QUANTCONNECT DOCUMENTATION

Learn to use QuantConnect and Explore Features

WRITING ALGORITHMS

Learn tools you need to build algorithmic trading strategies

Build on a mature, flexible, feature-complete API managing billions of dollars capital, and used by 5,000+ investors every month.

Table of Content

- 1 Key Concepts
- 1.1 Getting Started
- 1.2 Algorithm Engine
- 1.3 Multi-Asset Modeling
- 1.4 Time Modeling
- 1.4.1 Periods
- 1.4.2 Timeslices
- 1.4.3 Time Zones
- 1.5 Security Identifiers
- 1.6 Event Handlers
- 1.7 Python and LEAN
- 1.8 Duck Typing
- 1.9 Research Guide
- 1.10 Behavioral Finance
- 1.11 Globals and Statics
- 1.12 Libraries
- 1.13 Debugging Tools
- 1.14 Glossary
- 2 Initialization
- 3 Securities
- 3.1 Key Concepts
- 3.2 Requesting Data
- 3.3 Handling Data
- 3.4 Market Hours
- 3.5 Filtering Data
- 3.6 Asset Classes
- 3.6.1 US Equity
- 3.6.1.1 Requesting Data
- 3.6.1.2 Handling Data
- 3.6.1.3 Corporate Actions
- 3.6.1.4 Corporate Fundamentals
- 3.6.1.5 Shorting
- 3.6.1.6 Data Preparation
- 3.6.1.7 Market Hours
- 3.6.2 Equity Options
- 3.6.2.1 Requesting Data
- 3.6.2.1.1 Universes
- 3.6.2.1.2 Individual Contracts
- 3.6.2.2 Handling Data
- 3.6.2.3 Greeks and Implied Volatility
- 3.6.2.3.1 Key Concepts
- 3.6.2.3.2 Indicators
- 3.6.2.4 Market Hours
- 3.6.3 Crypto
- 3.6.3.1 Requesting Data
- 3.6.3.2 Handling Data
- 3.6.3.3 Holdings
- 3.6.3.4 Market Hours
- 3.6.4 Crypto Futures
- 3.6.4.1 Requesting Data
- 3.6.4.2 Handling Data

- 3.6.4.3 Market Hours
- 3.6.5 Forex
- 3.6.5.1 Requesting Data
- 3.6.5.2 Handling Data
- 3.6.5.3 Market Hours
- 3.6.6 Futures
- 3.6.6.1 Requesting Data
- 3.6.6.1.1 Universes
- 3.6.6.1.2 Individual Contracts
- 3.6.6.2 Handling Data
- 3.6.6.3 Market Hours
- 3.6.7 Future Options
- 3.6.7.1 Requesting Data
- 3.6.7.1.1 Universes
- 3.6.7.1.2 Individual Contracts
- 3.6.7.2 Handling Data
- 3.6.7.3 Greeks and Implied Volatility
- 3.6.7.3.1 Key Concepts
- 3.6.7.3.2 Indicators
- 3.6.7.4 Market Hours
- 3.6.8 Index
- 3.6.8.1 Requesting Data
- 3.6.8.2 Handling Data
- 3.6.8.3 Market Hours
- 3.6.9 Index Options
- 3.6.9.1 Requesting Data
- 3.6.9.1.1 Universes
- 3.6.9.1.2 Individual Contracts
- 3.6.9.2 Handling Data
- 3.6.9.3 Greeks and Implied Volatility
- 3.6.9.3.1 Key Concepts
- 3.6.9.3.2 Indicators
- 3.6.9.4 Market Hours
- 3.6.10 CFD
- 3.6.10.1 Requesting Data
- 3.6.10.2 Handling Data
- 3.6.10.3 Market Hours
- 3.6.11 India Equity
- 3.6.11.1 Market Hours
- 4 Portfolio
- 4.1 Key Concepts
- 4.2 Holdings
- 4.3 Cashbook
- 5 Universes
- 5.1 Key Concepts
- 5.2 Settings
- 5.3 Equity
- 5.3.1 Liquidity Universes
- 5.3.2 Fundamental Universes
- 5.3.3 ETF Constituents Universes
- 5.3.4 Chained Universes
- 5.3.5 Legacy Fundamental Universes

- 5.3.6 Alternative Data Universes
- 5.4 Equity Options
- 5.5 Equity CFD
- 5.6 Futures
- 5.7 Future Options
- 5.8 Index Options
- 5.9 Crypto
- 5.10 Custom Universes
- 5.11 Dataless Scheduled Universes
- 6 Datasets
- 6.1 Overview
- 6.2 QuantConnect
- 6.2.1 Binance Crypto Future Margin Rate Data
- 6.2.2 Bybit Crypto Future Margin Rate Data
- 6.2.3 CFD Data
- 6.2.4 Cash Indices
- 6.2.5 FOREX Data
- 6.2.6 Fear and Greed
- 6.2.7 International Future Universe
- 6.2.8 US ETF Constituents
- 6.2.9 US Equities Short Availability
- 6.2.10 US Equity Coarse Universe
- 6.2.11 US Equity Option Universe
- 6.2.12 US Equity Security Master
- 6.2.13 US Future Option Universe
- 6.2.14 US Future Universe
- 6.2.15 US Futures Security Master
- 6.2.16 US Index Option Universe
- 6.3 AlgoSeek
- 6.3.1 US Equities
- 6.3.2 US Equity Options
- 6.3.3 US Future Options
- 6.3.4 US Futures
- 6.3.5 US Index Options
- 6.4 Morningstar
- 6.4.1 US Fundamental Data
- 6.5 TickData
- 6.5.1 International Futures
- 6.6 CoinAPI
- 6.6.1 Binance Crypto Future Price Data
- 6.6.2 Binance Crypto Price Data
- 6.6.3 Binance US Crypto Price Data
- 6.6.4 Bitfinex Crypto Price Data
- 6.6.5 Bybit Crypto Future Price Data
- 6.6.6 Bybit Crypto Price Data
- 6.6.7 Coinbase Crypto Price Data
- 6.6.8 Kraken Crypto Price Data
- 6.7 Benzinga
- 6.7.1 Benzinga News Feed
- 6.8 Blockchain
- 6.8.1 Bitcoin Metadata
- 6.9 Brain

- 6.9.1 Brain Language Metrics on Company Filings
- 6.9.2 Brain ML Stock Ranking
- 6.9.3 Brain Sentiment Indicator
- 6.9.4 Brain Wikipedia Page Views
- 6.10 CoinGecko
- 6.10.1 Crypto Market Cap
- 6.11 EOD Historical Data
- 6.11.1 Economic Events
- 6.11.2 Macroeconomics Indicators
- 6.11.3 Upcoming Dividends
- 6.11.4 Upcoming Earnings
- 6.11.5 Upcoming IPOs
- 6.11.6 Upcoming Splits
- 6.12 Energy Information Administration
- 6.12.1 US Energy Information Administration (EIA)
- 6.13 ExtractAlpha
- 6.13.1 Cross Asset Model
- 6.13.2 Estimize
- 6.13.3 Tactical
- 6.13.4 True Beats
- 6.14 FRED
- 6.14.1 US Federal Reserve (FRED)
- 6.15 Federal Reserve Bank of St Louis
- 6.15.1 US Interest Rate
- 6.16 Kavout
- 6.16.1 Composite Factor Bundle
- 6.17 Nasdaq
- 6.17.1 Data Link
- 6.18 Quiver Quantitative
- 6.18.1 CNBC Trading
- 6.18.2 Corporate Lobbying
- 6.18.3 Insider Trading
- 6.18.4 US Congress Trading
- 6.18.5 US Government Contracts
- 6.18.6 WallStreetBets
- 6.19 RegAlytics
- 6.19.1 US Regulatory Alerts - Financial Sector
- 6.20 Securities and Exchange Commission
- 6.20.1 US SEC Filings
- 6.21 Smart Insider
- 6.21.1 Corporate Buybacks
- 6.22 Tiingo
- 6.22.1 Tiingo News Feed
- 6.23 Treasury Department
- 6.23.1 US Treasury Yield Curve
- 6.24 US Department of Agriculture
- 6.24.1 USDA Fruit And Vegetables
- 6.25 VIX Central
- 6.25.1 VIX Central Contango
- 7 Importing Data
- 7.1 Key Concepts
- 7.2 Streaming Data

- 7.2.1 Key Concepts
- 7.2.2 Custom Securities
- 7.2.2.1 Key Concepts
- 7.2.2.2 CSV Format Example
- 7.2.2.3 JSON Format Example
- 7.2.3 Custom Universes
- 7.2.3.1 Key Concepts
- 7.2.3.2 CSV Format Example
- 7.2.3.3 JSON Format Example
- 7.2.4 Precomputed ML Predictions
- 7.3 Bulk Downloads
- 8 Consolidating Data
- 8.1 Getting Started
- 8.2 Consolidator Types
- 8.2.1 Time Period Consolidators
- 8.2.2 Calendar Consolidators
- 8.2.3 Count Consolidators
- 8.2.4 Mixed-Mode Consolidators
- 8.2.5 Renko Consolidators
- 8.2.5.1 Renko Consolidators
- 8.2.5.2 Classic Renko Consolidators
- 8.2.5.3 Volume Renko Consolidators
- 8.2.6 Range Consolidators
- 8.2.6.1 Range Consolidators
- 8.2.6.2 Classic Range Consolidators
- 8.2.7 Combining Consolidators
- 8.3 Consolidator History
- 8.4 Updating Indicators
- 9 Historical Data
- 9.1 Getting Started
- 9.2 History Requests
- 9.3 History Responses
- 9.4 Asset Classes
- 9.4.1 US Equities
- 9.4.2 Equity Options
- 9.4.3 Crypto
- 9.4.4 Crypto Futures
- 9.4.5 Forex
- 9.4.6 Futures
- 9.4.7 Future Options
- 9.4.8 Index
- 9.4.9 Index Options
- 9.4.10 CFD
- 9.5 Alternative Data
- 9.6 Custom Data
- 9.7 Universe Data
- 9.8 Warm Up Periods
- 9.9 Rolling Window
- 9.10 Common Errors
- 9.11 Live Trading
- 10 Trading and Orders
- 10.1 Key Concepts

- 10.2 Order Management
- 10.2.1 Order Tickets
- 10.2.2 Transaction Manager
- 10.3 Order Types
- 10.3.1 Market Orders
- 10.3.2 Limit Orders
- 10.3.3 Limit if Touched Orders
- 10.3.4 Stop Market Orders
- 10.3.5 Stop Limit Orders
- 10.3.6 Trailing Stop Orders
- 10.3.7 Market On Open Orders
- 10.3.8 Market On Close Orders
- 10.3.9 Combo Market Orders
- 10.3.10 Combo Limit Orders
- 10.3.11 Combo Leg Limit Orders
- 10.3.12 Option Exercise Orders
- 10.3.13 Other Order Types
- 10.4 Pre-trade Risk Control
- 10.5 Position Sizing
- 10.6 Liquidating Positions
- 10.7 Crypto Trades
- 10.8 Option Strategies
- 10.8.1 Bear Call Spread
- 10.8.2 Bear Put Spread
- 10.8.3 Bull Call Spread
- 10.8.4 Bull Put Spread
- 10.8.5 Long Call Butterfly
- 10.8.6 Short Call Butterfly
- 10.8.7 Long Put Butterfly
- 10.8.8 Short Put Butterfly
- 10.8.9 Long Call Calendar Spread
- 10.8.10 Short Call Calendar Spread
- 10.8.11 Long Put Calendar Spread
- 10.8.12 Short Put Calendar Spread
- 10.8.13 Covered Call
- 10.8.14 Covered Put
- 10.8.15 Long Iron Butterfly
- 10.8.16 Short Iron Butterfly
- 10.8.17 Long Iron Condor
- 10.8.18 Short Iron Condor
- 10.8.19 Naked Call
- 10.8.20 Naked Put
- 10.8.21 Protective Call
- 10.8.22 Protective Put
- 10.8.23 Protective Collar
- 10.8.24 Long Straddle
- 10.8.25 Short Straddle
- 10.8.26 Long Strangle
- 10.8.27 Short Strangle
- 10.8.28 Conversion
- 10.8.29 Reverse Conversion
- 10.8.30 Long Box Spread

- 10.8.31 Short Box Spread
- 10.8.32 Long Jelly Roll
- 10.8.33 Short Jelly Roll
- 10.8.34 Bear Call Ladder
- 10.8.35 Bear Put Ladder
- 10.8.36 Bull Call Ladder
- 10.8.37 Bull Put Ladder
- 10.8.38 Long Call Backspread
- 10.8.39 Short Call Backspread
- 10.8.40 Long Put Backspread
- 10.8.41 Short Put Backspread
- 10.9 Order Properties
- 10.10 Order Events
- 10.11 Order Errors
- 10.12 Trade Statistics
- 10.13 Trading Calendar
- 10.14 Financial Advisors
- 11 Reality Modeling
- 11.1 Key Concepts
- 11.2 Trade Fills
- 11.2.1 Key Concepts
- 11.2.2 Supported Models
- 11.2.2.1 Equity Model
- 11.2.2.2 Future Model
- 11.2.2.3 Future Option Model
- 11.2.2.4 Immediate Model
- 11.2.2.5 Latest Price Model
- 11.3 Slippage
- 11.3.1 Key Concepts
- 11.3.2 Supported Models
- 11.4 Transaction Fees
- 11.4.1 Key Concepts
- 11.4.2 Supported Models
- 11.5 Brokerages
- 11.5.1 Key Concepts
- 11.5.2 Supported Models
- 11.5.2.1 QuantConnect Paper Trading
- 11.5.2.2 Interactive Brokers
- 11.5.2.3 TradeStation
- 11.5.2.4 Tastytrade
- 11.5.2.5 Alpaca
- 11.5.2.6 Charles Schwab
- 11.5.2.7 Binance
- 11.5.2.8 Bybit
- 11.5.2.9 Tradier
- 11.5.2.10 Kraken
- 11.5.2.11 Coinbase
- 11.5.2.12 Bitfinex
- 11.5.2.13 dYdX
- 11.5.2.14 Terminal Link
- 11.5.2.15 SSC Eze
- 11.5.2.16 Trading Technologies

- 11.5.2.17 Wolverine
- 11.5.2.18 Oanda
- 11.6 Brokerage Message Handler
- 11.7 Buying Power
- 11.8 Settlement
- 11.8.1 Key Concepts
- 11.8.2 Supported Models
- 11.9 Options Models
- 11.9.1 Pricing
- 11.9.2 Volatility
- 11.9.2.1 Key Concepts
- 11.9.2.2 Supported Models
- 11.9.3 Exercise
- 11.9.4 Assignment
- 11.10 Risk Free Interest Rate
- 11.10.1 Key Concepts
- 11.10.2 Supported Models
- 11.11 Dividend Yield
- 11.11.1 Key Concepts
- 11.11.2 Supported Models
- 11.12 Margin Interest Rate
- 11.12.1 Key Concepts
- 11.12.2 Supported Models
- 11.13 Margin Calls
- 11.14 Short Availability
- 11.14.1 Key Concepts
- 11.14.2 Supported Providers
- 12 Scheduled Events
- 13 Indicators
- 13.1 Supported Indicators
- 13.2 Key Concepts
- 13.3 Manual Indicators
- 13.4 Automatic Indicators
- 13.5 Plotting Indicators
- 13.6 Combining Indicators
- 13.7 Custom Indicators
- 13.8 Indicator Universes
- 13.9 Rolling Window
- 14 Object Store
- 15 Optimization
- 15.1 Parameters
- 15.2 Walk Forward Optimization
- 16 Machine Learning
- 16.1 Key Concepts
- 16.2 Training Models
- 16.3 Popular Libraries
- 16.3.1 Aesera
- 16.3.2 Copula
- 16.3.3 GPlearn
- 16.3.4 Hmmlearn
- 16.3.5 Keras
- 16.3.6 PyTorch

- 16.3.7 Scikit-Learn
- 16.3.8 Tensorflow
- 16.3.9 Tslearn
- 16.3.10 XGBoost
- 16.4 Hugging Face
- 16.4.1 Key Concepts
- 16.4.2 Popular Models
- 16.4.2.1 Chronos-T5
- 16.4.2.2 DistilBERT
- 16.4.2.3 FinBERT
- 16.4.2.4 Sentiment Analysis
- 16.4.2.5 Fill-Mask
- 16.4.2.6 Text Generation
- 16.4.2.7 Chronos-Bolt
- 17 Algorithm Framework
- 17.1 Overview
- 17.2 Universe Selection
- 17.2.1 Key Concepts
- 17.2.2 Universe Settings
- 17.2.3 Manual Universes
- 17.2.4 Fundamental Universes
- 17.2.5 ETF Constituents Universes
- 17.2.6 Scheduled Universes
- 17.2.7 Futures Universes
- 17.2.8 Options Universes
- 17.2.9 Legacy Fundamental Universes
- 17.3 Alpha
- 17.3.1 Key Concepts
- 17.3.2 Supported Models
- 17.4 Portfolio Construction
- 17.4.1 Key Concepts
- 17.4.2 Supported Models
- 17.4.3 Supported Optimizers
- 17.5 Risk Management
- 17.5.1 Key Concepts
- 17.5.2 Supported Models
- 17.6 Execution
- 17.6.1 Key Concepts
- 17.6.2 Supported Models
- 17.7 Hybrid Algorithms
- 17.8 Insight Manager
- 18 Charting
- 19 Logging
- 20 Statistics
- 20.1 Algorithm Statistics
- 20.2 Runtime Statistics
- 20.3 Trade Statistics
- 21 Live Trading
- 21.1 Key Concepts
- 21.2 Brokerages
- 21.3 Data Providers
- 21.4 Trading and Orders

- 21.4.1 Financial Advisors
- 21.5 Reconciliation
- 21.6 Notifications
- 21.7 Commands
- 21.8 Arbitrage
- 21.9 Charting and Logging
- 21.10 Signal Exports
- 21.10.1 Key Concepts
- 21.10.2 Collective2
- 21.10.3 Numerai
- 21.10.4 vBase
- 22 Strategy Library
- 23 API Reference
- 24 Migrations
- 24.1 Zipline
- 24.1.1 Initialization
- 24.1.2 Using Data
- 24.1.3 Ordering
- 24.1.4 Logging and Plotting
- 24.1.5 Quick Reference

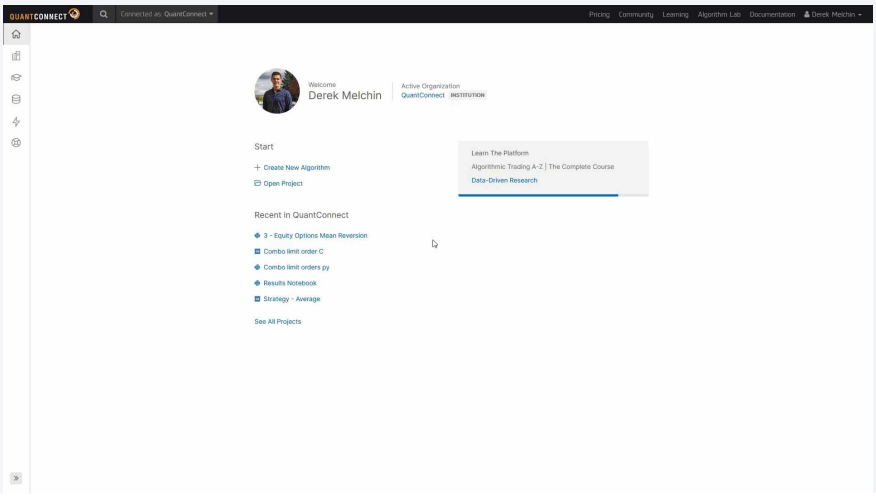
Key Concepts

Key Concepts

Getting Started

GET STARTED WITH WRITING ALGORITHMS

Guide through creating a project, running your first backtest, and live algo trading.



Follow these steps to create, backtest, and paper trade a new algorithm:

1. Create a new project.

The process to creating a new project depends on if you use the Cloud Platform , Local Platform , or CLI .

2. Backtest the default algorithm.

The process to backtest the algorithm depends on if you use the Cloud Platform , Local Platform , or CLI .

3. Deploy the algorithm to live paper trading.

The process to deploy the algorithm live depends on if you use the Cloud Platform , Local Platform , or CLI .

Key Concepts

Algorithm Engine

Introduction

LEAN Engine is an open-source algorithmic trading engine built for easy strategy research, backtesting, and live trading. We integrate with common data providers and brokerages, so you can quickly deploy algorithmic trading strategies. The core of the LEAN Engine is written in C#, but it operates seamlessly on Linux, Mac and Windows operating systems. To use it, you can write algorithms in Python 3.11 or C#. QuantConnect maintains the LEAN project and uses it to drive the web-based algorithmic trading platform on the website.

Since LEAN is open-source, you aren't locked-in to the QuantConnect platform. You can run LEAN with your own infrastructure, data, and brokerage connections. If you use QuantConnect, our team of engineers manage the infrastructure, you can run the latest version of LEAN with all of its brokerage connections, and you can utilize the datasets in the Dataset Market .

Your Algorithm and LEAN

To create a trading algorithm with LEAN, define a subclass of the `QCAlgorithm` class. LEAN loads your algorithm into the engine during the building and compilation process. LEAN runs an algorithm manager that synchronizes the data your algorithm requests, injects the data into your algorithm so you can place trades, processes your orders, and then updates your algorithm state.

The LEAN engine manages your portfolio and data feeds, so you focus on your algorithm strategy and execution. We automatically provide basic portfolio management and reality modeling underneath the hood. The `QCAlgorithm` class provides some key helper properties for you to use, including the Security Manager, Portfolio Manager, Transactions Manager, Notification Manager, and Scheduling Manager. The class also has hundreds of helper methods to make the API easy to use.

The Securities property is a dictionary of `Security` objects. Each asset (Equity, Forex pair, etc) in your algorithm has a `Security` object. All the models for a security live on these objects. For example, `self.Securities["IBM"].FeeModel` and `self.Securities["IBM"].Price` return the fee model and price of IBM, respectively.

The Portfolio is a dictionary of `SecurityHolding` objects. These classes track the profits, losses, fees, and quantity of individual portfolio holdings. For example, `self.portfolio["IBM"].last_trade_profit` returns the profit of your last IBM trade.

Other helpers like Transactions , Schedule , Notify , and Universe have their own helper methods.

```py
class QCAlgorithm:
    securities   # Array of Security objects.
    portfolio    # Array of SecurityHolding objects
    transactions # Transactions helper
    schedule     # Scheduling helper
    notify       # Email, SMS helper
    universe     # Universe helper

    # Set up Requested Data, Cash, Time Period.
    def initialize(self) -> None:

    # Other Event Handlers
    def on_data(self, slice: Slice) -> None:
    def on_end_of_day(self, symbol: Symbol) -> None:
    def on_end_of_algorithm(self) -> None:

    # Indicator Helpers
    def sma(self, symbol: Symbol, period: int) -> SimpleMovingAverage:
```

Only define one algorithm class per project. To trade multiple strategies in a single algorithm, implement an Algorithm Framework design with multiple Alpha models . To be notified when you can define more than one algorithm class per project, subscribe to GitHub Issue #6968 .

Threads in LEAN

LEAN is multi-threaded and attempts to consume as much CPU as possible to perform given work as quickly as possible. The analysis engine loads data parallel and synchronizes it to generate the requested security information.

The client algorithm is plugged into LEAN, and has its events triggered synchronously in backtesting. The primary bottle neck to LEAN execution is

executing client code.

In live trading, most events are synchronous as in backtesting, however order events are triggered immediately from the brokerage thread (asynchronously) to ensure the lowest latency. For example; a strategy using hourly data, with a limit order could fill between data bars. This fill event is triggered when it occurs in live trading.

We recommend using thread safe collections when possible, or locks around collections to ensure they are thread-safe.

Batch vs Stream Analysis

Backtesting platforms come in two general varieties, batch processing and event streaming. Batch processing backtesting is much simpler. It loads all data into an array and passes it to your algorithm for analysis. Because your algorithm has access to future data points, it is easy to introduce look-ahead bias . Most home-grown analysis tools are batch systems.

QuantConnect/LEAN is a streaming analysis system. In live trading, your algorithm receives data points one after another over time. QuantConnect models this in backtesting, streaming data to your algorithm in fast-forward mode. Because of this, you can't access price data beyond the Time Frontier . Although streaming analysis is slightly trickier to understand, it allows your algorithm to seamlessly work in backtests and live trading with no code changes.

Event Flow

When you deploy an algorithm, LEAN first calls the initialize method. In live mode, the engine loads your holdings and open orders from your brokerage account to add data subscriptions and populate the `securities` , `portfolio` , and `transactions` objects. LEAN receives the data from the subscriptions, synchronizes the data to create a timeslice , and then performs the following steps. Your algorithm can spend up to 10 minutes on each timeslice unless you call the train method.

1. If it's a backtest, check if there are Scheduled Events in the past that didn't fire because there was no data between the previous slice and the current slice. LEAN automatically creates a Scheduled Event to call the `on_end_of_day` method at the end of each day.

   In live mode, Scheduled Events occur in a separate thread from the algorithm manager, so they run at the correct time.

2. Update the algorithm time.
3. Update the `current_slice` .
4. Pass the `SymbolChangedEvents` to the `on_symbol_changed_events` method.
5. Cancel all open orders for securities that changed their ticker .
6. Add a `Security` object to the `securities` collection for each new security in the universe.
7. Update the `Security` objects with the latest data.
8. Update the `Cash` objects in the CashBook with the latest data.
9. Process fill models for non-market orders.
10. Submit market on open orders to liquidate Equity Option contracts if the underlying Equity has a split warning .
11. Process margin calls .
12. If it's time to settle unsettled cash , perform settlement.
13. Call the on_securities_changed method with the latest security changes.
14. Apply dividends to the portfolio.
15. For securities that have a split warning, update their portfolio holdings and adjust their open orders to account for the split.
16. Update consolidators with the latest data.
17. Pass the `Dividends` to the `on_dividends` method.
18. Pass the `Splits` to the `on_splits` method.
19. Pass the `Delistings` to the `on_delistings` method.
20. Pass the `Slice` to the `on_data` method.
21. Perform universe selection .
22. Pass the `Slice` to the `update` method of each Alpha model .
23. Pass the `Insight` objects from the Alpha model to the Portfolio Construction model .
24. Pass the `PortfolioTarget` objects from the Portfolio Construction model to each Risk Management model .
25. Pass the risk-adjusted `PortfolioTarget` objects from the Risk Management models to the Execution model .

When your algorithm stops executing, LEAN calls the `on_end_of_algorithm` method .

Python Support

The LEAN engine is written in C#, but you can create algorithms in C# or Python. If you program in Python, LEAN uses Python.Net to bridge between the C# engine and your algorithm. It can be slow to move from Python to C#. If you access C# objects in your Python algorithm, it's fastest to only access them once and save a reference if you need to access them again.

```py
# Store the security holding in a variable for better readability and performance.
security_holding = self.portfolio[self._symbol]
avg_price = security_holding.average_price
quantity = security_holding.quantity

# Avoid indexing the portfolio object multiple times for each attribute.
avg_price = self.portfolio[self._symbol].average_price
quantity = self.portfolio[self._symbol].quantity
```

Key Concepts

Multi-Asset Modeling

Introduction

We designed LEAN as a multi-asset platform with out-of-the-box support for multiple securities, so you can model complex portfolios like hedge funds.

Asset Portfolio

The portfolio manages the individual securities it contains. It tracks the cost of holding each security. It aggregates the performance of the individual securities in the portfolio to produce statistics like net profit and drawdown . The portfolio also holds information about each currency in its cashbook.

Cashbooks

We designed LEAN to be a multi-currency platform. LEAN can trade Forex, Cryptocurrencies, and other assets that are quoted in other currencies. A benefit of supporting multiple currencies is that as we add new asset classes from new countries, LEAN is already prepared to transact in those assets by using their quote currency. For instance, we added the India Equity market, which quotes assets in the INR currency.

The portfolio manages your currencies in its cashbook, which models the cash as a ledger of transactions. When you buy assets, LEAN uses the currencies in your cashbook to purchase the asset and pay the transaction fees. For more information about the cashbook, see Cashbook .

Buying Power

We model the margin requirements of each asset and reflect that in the buying power available to the algorithm. We source Futures margins from CME SPAN margins. Equity margin is 2x for standard margin accounts and 4x intraday for Pattern Day Trading accounts. For more information about buying power modeling, see Buying Power .

Examples

The following examples demonstrate some common practices for multi-asset modeling.

Example 1: BTC Spot-Crypto Future Arbitration

This algorithm demonstrates an arbitration between Spot and Crypto Future BTCUSDT using a BTC cash account. If one's price is above 0.5% of another's, we sell the relatively overpriced one and buy the counter side. To ensure the cash position is sufficient to open the buy position, we order the buy side after the sell side is filled and the cash is replenished.

```py
class MultiAssetModelingAlgorithm(QCAlgorithm):

    _threshold = 0.005

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the last price to set the initial price of the BTCUSDT holdings.
        self.settings.seed_initial_prices = True

        # Simulate a cash Bybit account.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.CASH)
        self.set_account_currency("USDT", 0)
        spot_balance = 2  # BTC
        self.set_cash('BTC', spot_balance)

        # Request BTCUSD spot and future data to trade their price discrepancies.
        self._spot = self.add_crypto("BTCUSDT", market=Market.BYBIT)
        self._future = self.add_crypto_future("BTCUSDT", market=Market.BYBIT)

        # Simulate the portfolio is holding BTC cash initially via BTCUSDT position.
        # Note that the performance will also be affected by the BTC performance in the default account currency.
        self._spot.holdings.set_holdings(average_price=self._spot.price, quantity=spot_balance)
        self._trade_quantity = spot_balance/2

    def on_data(self, slice: Slice) -> None:
        # Wait for the Slice to contain QuoteBar objects for both assets.
        if not(self._spot.symbol in slice.quote_bars and self._future.symbol in slice.quote_bars):
            return
        # If the spot price is higher than the future price more than the threshold,
        # Do arbitration by selling the spot BTC and buying the future.
        if self._spot.price >= self._future.price * (1 + self._threshold) and not self._future.holdings.is_long:
            self.sell(self._spot.symbol, self._trade_quantity)
        # If the future price is higher than the spot price more than the threshold,
        # Do arbitration by buying the spot BTC and selling the future.
        if self._future.price >= self._spot.price * (1 + self._threshold) and self._future.holdings.is_long:
            self.sell(self._future.symbol, self._trade_quantity)

    def on_order_event(self, order_event: OrderEvent) -> None:
        # Order the buy-side of the arb only when the BTC is sold and USDT is obtained.
        if not(order_event.quantity < 0 and order_event.status == OrderStatus.FILLED):
            return
        security = self._future if order_event.symbol == self._spot.symbol else self._spot
        # Calculate the initial margin needed, we need to sell the same amount of BTC to obtain sufficient USDT for trade.
        margin = security.buying_power_model.get_initial_margin_requirement(
            InitialMarginParameters(security, self._trade_quantity)
        ).value
        # Check if USDT cash is sufficient to open the position.
        if self.portfolio.cash_book["USDT"].amount >= margin:
            self.buy(security.symbol, self._trade_quantity)
```

Example 2: BTC Spot-Future Arbitration

This algorithm implements a similar logic to the above example of an arbitration algorithm between spot BTCUSD and Future BTC using a cash account. If one's price is above 0.5% of another's, we sell the relatively overpriced one and buy the counter side. Note that we need to calculate the future value of the Future fairly compared to the spot BTC. Also, we need to rollover any Futures contracts.

PY

```
class MultiAssetModelingAlgorithm(QCAlgorithm):

    _threshold = 0.005

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set cash to sell Futures.
        self.set_cash(1_000_000)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add BTCUSD spot and Future data to trade their price
        # discrepancies.
        self._spot = self.add_crypto("BTCUSD", market=Market.COINBASE)
        self._future = self.add_future(
            Futures.Currencies.BTC,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST
        )
        self._future.set_filter(0, 62)

    def on_data(self, slice: Slice) -> None:
        # Get the current bar for both markets.
        spot = slice.bars.get(self._spot.symbol)
        future = slice.quote_bars.get(self._future.mapped)
        if not (spot and future):
            return
        # Use forward price fairly to compare to the spot BTC price.
        discount_factor = (
            self.risk_free_interest_rate_model.get_interest_rate(slice.time)
            * (self._future.mapped.id.date - slice.time).total_seconds()
            / 60 / 60 / 24 / 365
        )
        btc_future_fv = future.close * np.exp(discount_factor)
        # If the spot price is above the Future price by more than the
        # threshold, sell the spot BTC and buy the Future to arbitrage.
        if (not self._spot.holdings.is_short and
            spot.close >= btc_future_fv * (1 + self._threshold)):
            self.sell(
                self._spot.symbol,
                self._future.symbol_properties.contract_multiplier
            )
            self.buy(self._future.mapped, 1)
        # If the Future price is above the spot price by more than the
        # threshold, sell the Future BTC and buy the spot BTC to arbitrage.
        elif (not self._spot.holdings.is_long and
            btc_future_fv >= spot.close * (1 + self._threshold)):
            self.sell(self._future.mapped, 1)
            self.buy(
                self._spot.symbol,
                self._future.symbol_properties.contract_multiplier
            )
        # When prices converge, exit the arbitrage trade.
        elif ((self._spot.holdings.is_long and btc_future_fv <= spot.close) or
            (self._spot.holdings.is_short and spot.close <= btc_future_fv)):
            self.liquidate()

    def on_symbol_changed_events(
            self, symbol_changed_events: SymbolChangedEvents) -> None:
        # Get Symbol Change Event of the Continuous Future (change
        # in mapped contract) to roll over.
        changed_event = symbol_changed_events.get(self._future.symbol)
        if not changed_event:
            return
        old_symbol = changed_event.old_symbol
        # Add the new contract.
        new_symbol = self.add_future_contract(changed_event.new_symbol).symbol
        tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
        quantity = self.portfolio[old_symbol].quantity
        # Rolling over: liquidate the position of the old mapped
        # contract and switch to the new mapped contract.
        self.liquidate(old_symbol, tag=tag)
        if quantity:
            self.market_order(new_symbol, quantity, tag=tag)
```

Key Concepts

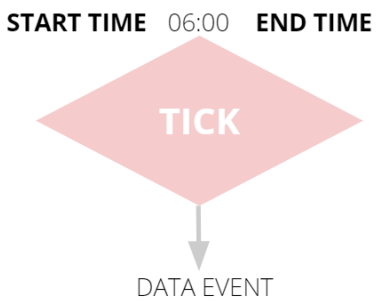
Time Modeling

Time Modeling

Periods

Introduction

Data comes in two different "shapes" according to the time period it covers: point values or period values. In QuantConnect, ticks are point values, and bars are a period values. These data formats have different properties, which control when LEAN emits them into your algorithm.



Start and End Time

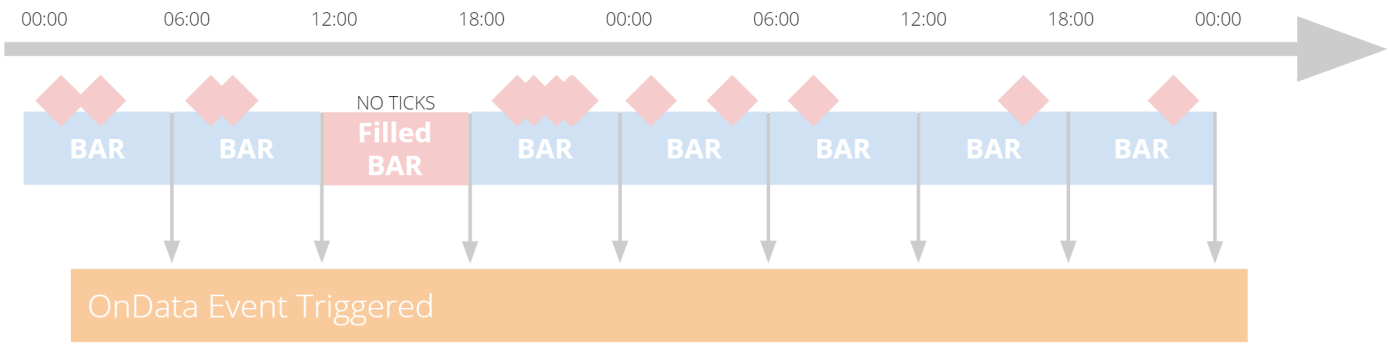
Bars have a start and end time that represent the time period of which the bar aggregates data. LEAN passes the bar to your algorithm at the end time so that you don't receive the bar before it was actually available. Free online data providers commonly timestamp their bars to the start time of the bar and include the bar close price, making your research prone to look ahead bias.

Carefully consider the end-time of data to avoid one-off errors when you import the data into QC or compare indicator values across different platforms. Generally, bar timestamps can be represented as follows: `bar.end_time = bar.time + bar.period`.

Period Values

Bars aggregate data across a period of time into a single object. We make this easy for you by pre-aggregating billions of raw trade-ticks into `TradeBar` objects and quote-ticks into `QuoteBar` objects.

We don't know the close of an intraday bar until the start of the next bar, which can sometimes be confusing. For daily bars, the bar will include all the ticks from market open to market close, but LEAN will emit it to your algorithm at market close. As a result, if you analyze the Friday data and then place an order, LEAN sends the order to your brokerage on Friday after the market close.



When there are no ticks during a period, LEAN emits the previous bar. This is the default behavior for bar data and it's referred to as "filling the data forward". You can enable and disable this setting when you create the security subscription .

We provide bar data in second, minute, hour, and daily bar formats. To create other periods of bars, see Consolidating Data .

Daily Periods

LEAN emits daily bars at market close or midnight, depending on your `daily_precise_end_time` setting . To be notified when a security has finished trading for the day, add an `on_end_of_day` method to your algorithm. LEAN calls this method for each security every day.

```
# Perform end-of-day logic for a given symbol.
def on_end_of_day(self, symbol: Symbol) -> None:
    pass
```

Point Values

Point values have no period because they occur at a singular point in time. Examples of point data includes ticks, open interest, and news releases. Tick data represents a single trade or quote in the market. It's a discrete event with no period, so the `time` and `end_time` properties are the same. LEAN emits ticks as soon as they arrive and doesn't fill them forward.

Examples

The following examples demonstrate some common practices for period time modeling.

Example 1: Tick Signals & Daily Bars

This example shows using a tick/pointwise signal from the Smart Insider Intention dataset while trading Starbuck's stock with daily bars. To liquidate the position after two trading days, you can count the number of `TradeBar` objects you get in the `on_data` method.

PY

```
class PeriodTimeModelingAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add an Equity asset. We will enter with a market on open order
        # and hold for 2 full days, so daily resolution is sufficient.
        self._equity = self.add_equity("SBUX", Resolution.DAILY)
        # Add insider trade intention as the sentiment trading signal.
        self._smart_insider_intention = self.add_data(
            SmartInsiderIntention, self._equity.symbol
        )
        # Add a member to track the number of days until the position
        # should be liquidated.
        self._hold_days = 0

    def on_data(self, slice: Slice):
        # Buy Starbuck's stock whenever we receive a buyback intention
        # or transaction notification due to volatility.
        if self._smart_insider_intention.symbol in slice:
            self.set_holdings(self._equity.symbol, 1)
            # Hold the position for 2 days to fully digest the insider
            # intention sentiment.
            self._hold_days = 2
        # Count down the holding day per each daily trade bar received.
        if self._equity.holdings.invested and self._equity.symbol in slice.bars:
            self._hold_days -= 1
            # Liquidate the position if 2 days were fully counted.
            if self._hold_days <= 0:
                self.liquidate(self._equity.symbol)
```

Time Modeling

Timeslices

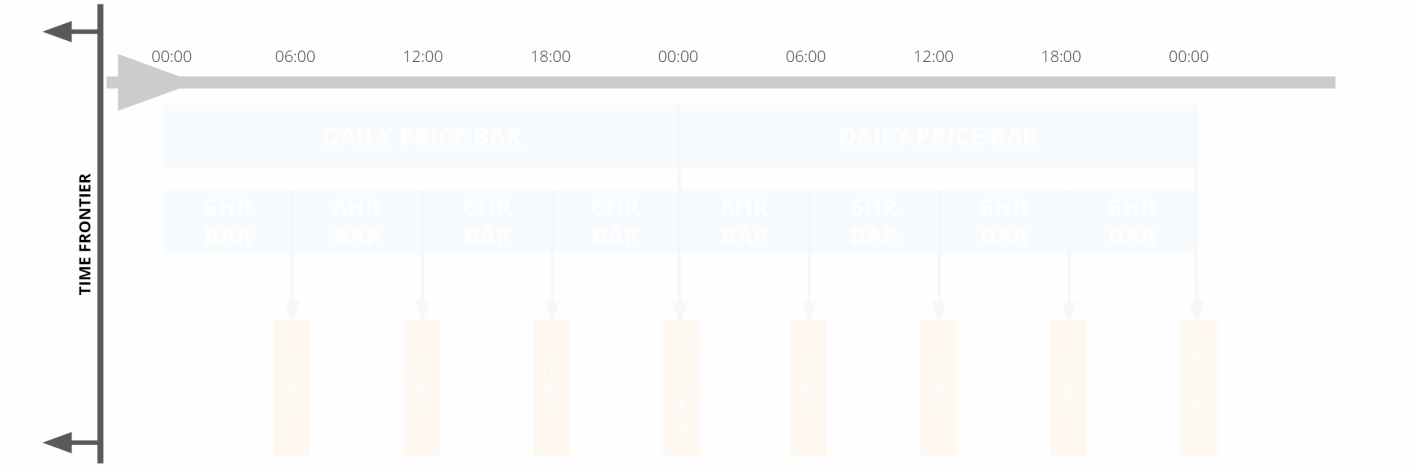
Introduction

The core technology behind QuantConnect algorithmic trading is an event-based, streaming analysis system called LEAN . LEAN attempts to model the stream of time as accurately as possible, presenting data ("events") to your algorithms in the order it arrives, as you would experience in reality.

All QuantConnect algorithms have this time-stream baked in as the primary event handler, `on_data` . The `Slice` object this method receives represents all of the data at a moment of time, a time-slice . No matter what data you request, you receive it in the order created according to simulated algorithm time. By only letting your algorithm see the present and past moments, we can prevent the most common quantitative-analysis error, look-ahead bias .

Time Frontier

If you request data for multiple securities and multiple resolutions, it can create a situation where one of your data subscriptions is ready to emit, but another subscription with a longer period is still be constructing its bar. Furthermore, if you request multiple datasets that have different time zones, your algorithm will receive the daily bars of each dataset at market close for the respective asset. To coordinate the data in these situations, we use the `end_time` of each data point to signal when LEAN should transmit it to your algorithm.



Once your algorithm reaches the `end_time` of a data point, LEAN sends the data to your `on_data` method. For intraday bars, this is the beginning of the next period. For daily bars, it's market close or midnight, depending on your `daily_precise_end_time` setting . To avoid look-ahead bias, you can only access data from before this Time Frontier. The `time` property of your algorithm is always equal to the Time Frontier.

Properties

`Slice` objects have the following properties:

Get Time Slices

To get the current `Slice` object, define an `on_data` method or use the `current_slice` property of your algorithm. The `Slice` contains all the data for a given moment in time. The and properties are `Symbol` /string indexed dictionaries. The property is a list of ticks for that moment of time, indexed by the `Symbol` . To check which data formats are available for each asset class, see the Data Formats page in the Asset Classes chapter.

The `Slice` object gives you the following ways to access your data:

- Indexing the `Slice` , which returns a dynamic object of your type.

```
# Get the current dynamic object for the specified symbol by indexing the Slice object.
def on_data(self, slice: Slice) -> None:
    data = slice[self._symbol]
```

With minute and second resolution data, the dynamic type is `TradeBar` for Equities and `QuoteBar` for other asset classes.

- Indexing the static properties, which returns the type you specify.

```
# Get the current trade and quote bars for a specific symbol from the data slice.
def on_data(self, slice: Slice) -> None:
    trade_bar = slice.bars[self._symbol]
    quote_bar = slice.quote_bars[self._symbol]
```

Check if the `Slice` contains the data you're looking for before you index it. If there is little trading, or you are in the same time loop as when you added the security, it may not have any data. Even if you enabled fill-forward for a security subscription, you should check if the data exists in the dictionary before you try to index it. To check if the `Slice` contains for a security, call the `contains_key` method. Note: if the `Slice` object doesn't contain any market data but it contains auxiliary data, the `slice.contains_key(symbol)` method can return true while `slice[symbol]` returns `None` .

```
# Check if the slice contains data for the symbol and retrieve it if it's available.
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self._symbol) and slice[self._symbol]:
        data = slice[self._symbol]
```

Examples

The following examples demonstrate some common practices for timeslices time modeling.

Example 1: 6-Hour Consolidated Bars

```
class TimeslicesTimeModelingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request AAPL data to trade it. We need a resolution denser than 6-hour for the consolidator.
        self.aapl = self.add_equity("AAPL", Resolution.Hour).symbol

        # Create a 6-hour consolidator for smoothing the noise.
        self.consolidator = TradeBarConsolidator(timedelta(hours=6))
        # Subscribe the consolidator to update with the security's data automatically.
        self.subscription_manager.add_consolidator(self.aapl, self.consolidator)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.aapl)
        if bar and self.consolidator.working_bar is not None:
            # Trade on a rising trend, suggested by the current close is above the past hour open
            # and the past hour open is above the 6-hour bar open.
            if self.consolidator.working_bar.open < bar.open < bar.close:
                self.set_holdings(self.aapl, 1)
            # Trade on a down trend, suggested by the current close is below the past hour open
            # and the past hour open is below the 6-hour bar open.
            elif self.consolidator.working_bar.open > bar.open > bar.close:
                self.set_holdings(self.aapl, -1)
            # Otherwise, do not hold a position if there is no deterministic trend.
            else:
                self.liquidate()
```

Time Modeling

Time Zones

Introduction

LEAN is an international trading engine, so it can operate across time zones. The exchange, your algorithm, and the data in your algorithm can all have different time zones.

Exchange Time Zone

Exchanges are located in different areas, so they can have different time zones. LEAN tracks the time zone of each exchange so that you receive the correct data at the right point in time in your algorithm. If you have multiple exchanges in your algorithm from different time zones, LEAN synchronizes them to your algorithm time. Furthermore, if you set up time period consolidators , LEAN consolidates the data based on the data time zone.

To get the time zone of the exchange that a `Security` trades on, use its `exchange.hours.time_zone` property.

```
# Get the time zone of the exchange for a specific asset.
time_zone = self.securities[self._symbol].exchange.hours.time_zone
```

Algorithm Time Zone

LEAN supports international trading across multiple time zones and markets, so it needs a reference time zone for the algorithm to set the `time` . The default time zone is Eastern Time (ET), which is UTC-4 in summer and UTC-5 in winter. To set a different time zone, call the `set_time_zone` method. This method accepts either a string following the IANA Time Zone database convention or a NodaTime .DateTimeZone object. If you pass a string, the method converts it to a `NodaTime.DateTimeZone` object. The `TimeZones` class provides the following helper attributes to create `NodaTime.DateTimeZone` objects:

```
self.set_time_zone("Europe/London")
self.set_time_zone(TimeZones.CHICAGO)
```

To get the time zone of your algorithm, use the `time_zone` property.

```
# The algorithm timezone property can assist with multi-asset trading.
time_zone = self.time_zone
```

To get the algorithm time in Coordinated Universal Time (UTC), use the `utc_time` property.

```
# Access the current UTC time to coordinate multi-timezone events or improve logging.
utc_time = self.utc_time
```

The `time` and `utc_time` objects have no time zone. LEAN maintains their state to be consistent.

Data Time Zone

Datasets can have different time zones because the time zone is up to the data provider, where they are located, and where they collect the data. LEAN converts their time zones to their exchange time zone and synchronizes them to your algorithm time.

To get the time zone of a dataset, view the listing page in the Dataset Market or call the `GetDataTimeZone` method.

```
# Get the time zone of an asset.
time_zone = self.market_hours_database.get_data_time_zone(Market.USA, self._symbol, SecurityType.EQUITY)
```

Examples

The following examples demonstrate some common practices for time zone modeling.

Example 1: Futures Time Zone

The following algorithm changes its time zone to be the same as the EMini SP500 Futures exchange, i.e., Chicago time zone. It is comparable in validating data across different sources and has a more convenient time notification without further conversion.

```py
class TimeZoneModelingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request ES future data for trading.
        self._future = self.add_future("ES", Resolution.Minute,
            extended_market_hours=True,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            contract_depth_offset=0)

        # Set the time zone to be the same as the Future (Chicago).
        # It will make the time in the backtest more comparable and easily analyzed with any other data sources.
        # Performance of the backtest will only be altered by this change if the trading logic depends on the Time property.
        self.set_time_zone(self._future.exchange.time_zone)

        self.ticket = None

    def on_data(self, slice: Slice) -> None:
        if not self.portfolio.invested and not self.ticket:
            contract = self.securities[self._future.mapped]
            # Set a limit price that repsects the brokerage's precision requirements.
            limit_price = self._adjust_limit_price(contract, contract.price * 0.995)
            self.ticket = self.limit_order(contract.symbol, 1, limit_price)

        # Cancel unfilled orders if they are not filled within 2 minutes to avoid stale fills.
        # Note that the Time property in the order ticket is in UTC timezone.
        if self.ticket.status != OrderStatus.Filled and self.ticket.time + timedelta(minutes=2) < self.utc_time:
            # You can change to a different time zone and compare the log differences.
            self.ticket.cancel(f"{self.time} :: utc: {self.utc_time} :: Order canceled due to placed over 2 minutes")

            self.ticket = None

    def _adjust_limit_price(self, security, price, round_up=True):
        pip = security.price_variation_model.get_minimum_price_variation(
            GetMinimumPriceVariationParameters(security, price)
        )
        return (int(price / pip) + int(round_up)) * pip
```

Key Concepts

Security Identifiers

Introduction

We implemented `Symbol` objects in the LEAN open-source project as a way to identify or "finger-print" tradable assets so that no further database look-up is required. All QuantConnect and LEAN Algorithm API methods use `Symbol` objects to identify assets.

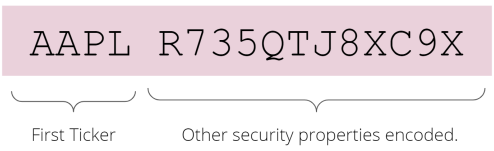
Encoding Symbols

`SecurityIdentifier` objects have several public properties to uniquely identify each asset. The following table describes these properties:

Property	Data Type	Description
`market`	`str`	The market in which the asset trades
`security_type`	`SecurityType`	The asset class
`option_style`	`OptionStyle`	American or European Option style
`option_right`	`OptionRight`	Call or put Option type
`strike_price`	`float`	The Option contract strike price
`date`	`datetime`	Earliest listing date for Equities; expiry for Futures and Options; December 30, 1899 for continuous Futures contracts and Indices.
`has_underlying`	`bool`	A flag to represent if its a derivative asset with another underlying asset

We encode the preceding properties to create `Symbol` objects. We do our best to hide the details of this process from your algorithm, but you'll occasionally see it come through as an encoded hash like `AAPL R735QTJ8XC9X`. The first half of the encoded string represents the first ticker AAPL was listed under. The other letters at the end of the string represent the serialized form of the preceding `SecurityIdentifier` properties. You may also see an encoded has like `AAPL XL7X5I241SO6|AAPL R735QTJ8XC9X` for a derivative contract. In this case, the part before the `|` is contract hash and the part after the `|` is the underlying hash. This serialization approach lets LEAN assign a short, unique string to octillions of different security objects.

Apple Inc. (same name since listed).



Google Class-C Shares (renamed to GOOG).

LEAN assumes the ticker you pass to the add_equity method is the current ticker of the Equity asset. To access this ticker, use the `value` property of the `Symbol` object.

```
# Add equity for "GOOG" and print symbol id value, symbol value and encoded symbol id.
self._symbol = self.add_equity("GOOG").symbol
self.debug(self._symbol.id.value) # Prints "GOOCV"
self.debug(self._symbol.value)    # Prints "GOOG"
self.debug(str(self._symbol.id))  # Prints "GOOCV VP83T1ZUHROL"
```

If you create the security subscription with a universe selection function, the `value` property of the `Symbol` object is the point-in-time ticker.

Decoding Symbols

When a `SecurityIdentifier` is serialized to a string, it looks something like `SPY R735QTJ8XC9X`. This two-part string is a base64 encoded set of data. Encoding all of the properties into a short format allows dense communication without requiring a third-party list or look-up. Most of the time, you will not need to work with these encoded strings. However, to deserialize the string into a `Symbol` object, call the `symbol` method.

```
# Extract the market, security type, and value from the symbol with ID "GOOCV VP83T1ZUHROL".
symbol = self.symbol("GOOCV VP83T1ZUHROL")
symbol.id.market      # => "USA"
symbol.security_type  # => SecurityType.EQUITY
symbol.value          # => "GOOCV"
```

Tickers

Tickers are a string shortcode representation for an asset. Some examples of popular tickers include "AAPL" for Apple Corporation and "IBM" for International Business Machines Corporation. These tickers often change when the company rebrands or they undergo a merger or reverse merger.

The ticker of an asset is not the same as the `Symbol`. `Symbol` objects are permanent and track the underlying entity. When a company rebrands or changes its name, the `Symbol` object remains constant, giving algorithms a way to reliably track assets over time.

Tickers are also often reused by different brokerages. For example Coinbase, a leading US Crypto Brokerage lists the "BTCUSD" ticker for trading. Its competitor, Bitfinex, also lists "BTCUSD". You can trade both tickers with LEAN. `Symbol` objects allow LEAN to identify which market you reference in your algorithms.

To create a `Symbol` object for a point-in-time ticker, call the `generate_equity` method to create the security identifier and then call the `Symbol` constructor. For example, Heliogen, Inc. changed their ticker from ATHN to HLGN on December 31, 2021. To convert the ATHN ticker to the Equity `Symbol`, type:

```
# Generate an Equity security identifier for a ticker and then create a symbol with it.
ticker = "ATHN"
security_id = SecurityIdentifier.generate_equity(ticker, Market.USA, mapping_resolve_date=datetime(2021, 12, 1))
symbol = Symbol(security_id, ticker)
```

In the preceding code snippet, the `mapping_resolve_date` must be a date when the point-in-time ticker was trading.

Symbol Cache

To make using the API easier, we have built the Symbol Cache technology, which accepts strings and tries to guess which `Symbol` object you might mean. With the Symbol Cache, many methods can accept a string such as "IBM" instead of a complete `Symbol` object. We highly recommend you don't rely on this technology and instead save explicit references to your securities when you need them.

```
# Example 1: Relying On Symbol Cache:
self.add_equity("IBM")          # Add by IBM string ticker, save reference to Symbol Cache.
self.market_order("IBM", 100)   # Determine refering to IBM Equity from Symbol Cache.
self.history("AAPL", 14)        # Makes a guess referring to AAPL Equity.

# Example 2: Correctly Using Symbols:
self.ibm = self.add_equity("IBM").symbol   # Add IBM Equity string ticker, save Symbol.
self.market_order(self.ibm, 100)           # Use IBM Symbol in future API calls.

self.aapl = Symbol.create("AAPL", SecurityType.EQUITY, Market.USA)
self.history(self.aapl, 14)
```

Markets

The `market` property distinguishes between tickers that have the same string value but represent different underlying assets. A prime example of this is the various market makers who have different prices for EURUSD. We store this data separately and as they have different fill prices, we treat the execution venues as different markets.

The following table shows the available markets for each security type:

Security Type	Market(s)
`SecurityType.EQUITY`	`Market.USA`
`SecurityType.OPTION`	`Market.USA`

`SecurityType.CRYPTO`	`Market.BINANCE`
	`Market.BINANCE_US`
	`Market.BITFINEX`
	`Market.BYBIT`
	`Market.COINBASE`
	`Market.KRAKEN`

`SecurityType.CRYPTO_FUTURE`	`Market.BINANCE`
	`Market.BYBIT`
	`Market.DYDX`

`SecurityType.FOREX`	`Market.OANDA`

`SecurityType.FUTURE`	`Market.CBOT`
	`Market.CFE`
	`Market.CME`
	`Market.COMEX`
	`Market.EUREX`
	`Market.HKFE`
	`Market.ICE`
	`Market.INDIA`
	`Market.NYMEX`
	`Market.NYSELIFFE`
	`Market.SGX`

`SecurityType.FUTURE_OPTION`	`Market.CBOT`
	`Market.COMEX`
	`Market.NYSELIFFE`

`SecurityType.INDEX`	`Market.EUREX`
	`Market.HKFE`
	`Market.OSE`
	`Market.USA`

`SecurityType.INDEX_OPTION`	`Market.USA`
`SecurityType.CFD`	`Market.INTERACTIVE_BROKERS` `Market.OANDA`

Industry Standard Identifiers

You can convert industry-standard security identifiers like CUSIP, FIGI, ISIN, and SEDOL to `Symbol` objects and you can convert `Symbol` objects to industry-standard security identifiers.

CUSIP

To convert a Committee on Uniform Securities Identification Procedures (CUSIP) number to a `Symbol` or a `Symbol` to a CUSIP number, call the `cusip` method.

```py
# Get the CUSIP for the symbol and vice versa.
symbol = self.cusip("03783310") # AAPL Symbol
cusip = self.cusip(symbol) # AAPL CUSIP (03783310)
```

FIGI

To convert a Financial Instrument Global Identifier (FIGI) to a `Symbol` or a `Symbol` to a FIGI, call the `composite_figi` method.

```
# Get the FIGI for the symbol and vice versa.
symbol = self.composite_figi("BBG000B9XRY4") # AAPL Symbol
figi = self.composite_figi(symbol) # AAPL FIGI (BBG000B9XRY4)
```

ISIN

To convert an International Securities Identification Number (ISIN) to a `Symbol` or a `Symbol` to an ISIN, call the `isin` method.

```
# Get the ISIN for the symbol and vice versa.
symbol = self.isin("US0378331005") # AAPL Symbol
isin = self.isin(symbol) # AAPL ISIN (US0378331005)
```

SEDOL

To convert a Stock Exchange Daily Official List (SEDOL) number to a `Symbol` or a `Symbol` to a SEDOL number, call the `sedol` method.

```
# Get the SEDOL for the symbol and vice versa
symbol = self.sedol("2046251") # AAPL Symbol
sedol = self.sedol(symbol) # AAPL SEDOL (2046251)
```

Examples

The following examples demonstrate some common practices for using the security identifier.

Example 1: Future Contract Expiry

The following example demonstrates the use of the security identifier of a Future contract to access its expiry date for contract selection and schedule a singular non-repeating event to rollover using `future.symbol.id.date` .

```
class SecurityIdentifierExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price data in order to allow the mapped contract to fill despite sparse data.
        self.settings.seed_initial_prices = True
        self._last_scheduled_event = None

    def on_warmup_finished(self) -> None:
        # Since we do not subscribe to any data source, we can use the on_warmup_finished event handler instead of OnData for the initial
position.
        es = Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)
        self.open_future_position(es, self.time + timedelta(30), 1)

    def open_future_position(self, symbol: Symbol, min_expiry: DateTime, quantity: float) -> None:
        # Select the next mapped future contract with the nearest expiry,
        # which should be further than the current contract.
        selected = selected = [c.symbol for c in self.futures_chain(symbol) if c.expiry > min_expiry][0]
        # Subscribe to its data to trade it.
        self._future = self.add_future_contract(selected, extended_market_hours=True)

        # Buy back the corresponding number of contracts as the previous mapped contract.
        self.market_order(self._future.symbol, quantity)

        # Schedule the next rollover to be 1 week before the expiry to avoid excessive volatility.
        # It will only be triggered once 7 days before the expiry and discarded afterward.
        if self._last_scheduled_event:
            self.schedule.remove(self._last_scheduled_event)
        self.schedule.on(
            self.date_rules.on([self._future.symbol.id.date - timedelta(7)]),
            self.time_rules.at(9, 30),
            self.roll_over
        )

    def roll_over(self) -> None:
        # Save the current number of contracts to roll over to the next mapped contract.
        quantity = self._future.holdings.quantity
        # Liquidate the to-be-expired contract to roll over the position.
        self.liquidate(self._future.symbol)

        self.open_future_position(self._future.symbol.canonical, self._future.symbol.id.date, quantity)
```

Key Concepts

Event Handlers

Introduction

Event handlers are called during an algorithm execution to pass information to your strategy. Most of the events are not needed for simple strategies, but can be helpful for debugging issues in more complex algorithms.

Data Events

```py
# Get data for a specific symbol to analyze its price movements or trading signals.
def on_data(self, slice: Slice) -> None:
    my_data = slice.get(self._symbol)

# Get split information to adjust holdings and calculations for the symbol.
def on_splits(self, splits: Splits) -> None:
    split = splits.get(self._symbol)

# Capture dividend events to account for income or adjust the strategy based on the dividend payout.
def on_dividends(self, dividends: Dividends) -> None:
    dividend = dividends.get(self._symbol)

# Track events when a security changes its ticker, allowing the algorithm to adapt to these changes.
def on_symbol_changed_events(self, symbol_changed_events: SymbolChangedEvents) -> None:
    symbol_changed_event = symbol_changed_events.get(self._symbol)

# Handle delisting events to manage the position or adjust the strategy when the symbol is delisted.
def on_delistings(self, delistings: Delistings) -> None:
    delisting = delistings.get(self._symbol)
```

The `on_data` method is the primary event handler for receiving financial data events to your algorithm. It is triggered sequentially at the point in time the data is available; in backtesting and live. For daily data, this means the event is processed at market close. In this method, all data for a given moment of time is grouped in a single event, including custom data types. This data is passed with the Slice object.

When fill-forward is enabled for your asset, the OnData event handler will be called regularly even if there was no new data. This is the default behavior.

In backtesting, if your algorithm takes a long time to process a slice, the following slice objects queue up and the next event triggers when your algorithm finishes processing the current slice. In live trading, if your algorithm takes longer to process the current slice than the time period that the slice spans, the next event triggers when your algorithm finishes processing the current slice, but the slice of the following event is the most recent slice. For example, say your algorithm consumes second resolution Crypto data but it takes your algorithm 3.5 seconds to process each slice. In backtesting, you'll get every slice. In live trading, if you deploy at 12:00:00 AM Coordinated Universal Time (UTC), you'll get the first slice at 12:00:01 AM UTC (spanning 12:00:00 AM UTC to 12:00:01 AM UTC) and you'll get the second slice at 12:00:04.5 AM UTC (roughly spanning 12:00:03 AM UTC to 12:00:04 AM UTC).

The `on_splits` , `on_dividends` , `on_symbol_changed_events` , and `on_delistings` event handlers provide data for their respective types in an isolated way. However, all of the data for these corporate actions is also available in the `Slice` in `on_data` .

To perform intensive computation before the market opens, use a Scheduled Event or the train method.

For more information on the `Slice` object and OnData event, see Handling Data .

Securities Changed Event

```
# Log the universe changes.
def on_securities_changed(self, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        self.debug(f"{self.time}: Added {security.symbol}")

    for security in changes.removed_securities:
        self.debug(f"{self.time}: Removed {security.symbol}")

        if security.invested:
            self.liquidate(security.symbol, "Removed from Universe")
```

The `on_securities_changed` event notifies the algorithm when assets are added or removed from the universe. This can be due to changes in the Universe constituents , delisting , contract expiration, or an explicit call to the remove_security method.

The event is triggered immediately when the asset is removed from the universe; however the data feed for the asset may remain active if the algorithm has open orders.

For more information, see how to use Security Changed Events .

Order Events

```
# Track filled orders and get the details of each fill.
def on_order_event(self, order_event: OrderEvent) -> None:
    order = self.transactions.get_order_by_id(order_event.order_id)
    if order_event.status == OrderStatus.FILLED:
        self.debug(f"{self.time}: {order.type}: {order_event}")

# Track Option assignment events to handle assigned Equity holdings.
def on_assignment_order_event(self, assignment_event: OrderEvent) -> None:
    self.log(str(assignment_event))
```

The `on_order_event` method notifies the algorithm of new orders, and changes in the order status such as fill events and cancelations. For options assignment events there is a dedicated event handler `on_assignment_order_event` .

In backtesting order events are triggered synchronously after the main data events. In live trading, order events are asynchronously as they occur. To avoid infinite loops, we recommend not to place orders in the OnOrderEvent.

For more information, see how to use Order Events .

Brokerage Events

```
# Track brokerage messages.
def on_brokerage_message(self, message: BrokerageMessageEvent) -> None:
    if message.type == BrokerageMessageType.RECONNECT:
        self.log(f"{self.time}: {message.type}: Message: {message.message}")

# Track brokerage disconnections and respond to connectivity interruptions.
def on_brokerage_disconnect(self) -> None:
    self.log(f"Brokerage disconnection detected at {self.time}")

 # Track brokerage reconnections to resume trading operations.
def on_brokerage_reconnect(self) -> None:
    self.log(f"Brokerage reconnected at {self.time}")
```

The `on_brokerage_disconnect` and `on_brokerage_reconnect` event handlers pass information to the algorithm about the brokerage connection status. This can be helpful for considering when to place a trade when a brokerage API is unreliable or under maintenance. The `on_brokerage_message` event handler provides information from the brokerages, including the disconnect and reconnect messages. Message content varies with each brokerage.

Brokerage events are triggered asynchronously in live trading, and are not created in backtesting.

Margin Call Events

```
# Track when remaining margin is low.
def on_margin_call_warning(self) -> None:
    self.log("Margin call warning, 5% margin remaining")

# Review and adjust liquidation orders in response to a margin call.
def on_margin_call(self, requests): -> List[SubmitOrderRequest]:
    # Modify order requests to choose what to liquidate.
    return requests
```

The `on_margin_call_warning` and `on_margin_call` event handlers provide information and control over how to reduce algorithm leverage when performance is poor.

The `on_margin_call_warning` method is called when there is less than 5% margin remaining to give you an opportunity to reduce leverage before an official margin call is performed by the brokerage. The `on_margin_call` event handler is passed a list of `SubmitOrderRequest` objects which will be executed on exiting the method.

Margin events are called before the data events are processed.

The following demonstration processes suggested orders and modifies the qualities to liquidate more than necessary and prevent a second margin call. For more information, see how to handle Margin Calls .

Warmup Finished Event

```
# This event handler runs when the algorithm is done warming up.
def on_warmup_finished(self) -> None:
    pass # Done warming up.
```

The `on_warmup_finished` event handler runs once initialization and warm up is complete.

Command Events

```
# An event handler for receiving external commands.
def on_command(self, data):
    self.log(f'Got command at {self.time} with data: {data}')
    return True
```

The `on_command` event handler runs when you send external commands to your algorithm.

End Of Algorithm Events

```
# The final step after algorithm execution.
def on_end_of_algorithm(self) -> None:
    self.debug("Algorithm done")
```

The `on_end_of_algorithm` event handler is when the algorithm has finished executing as its final step. This event handler is helpful for performing final analysis, or saving algorithm state.

Event Flow

When you deploy an algorithm, LEAN first calls the initialize method. In live mode, the engine loads your holdings and open orders from your brokerage account to add data subscriptions and populate the `securities` , `portfolio` , and `transactions` objects. LEAN receives the data from the subscriptions, synchronizes the data to create a timeslice , and then performs the following steps. Your algorithm can spend up to 10 minutes on each timeslice unless you call the train method.

1. If it's a backtest, check if there are Scheduled Events in the past that didn't fire because there was no data between the previous slice and the current slice. LEAN automatically creates a Scheduled Event to call the `on_end_of_day` method at the end of each day.

   In live mode, Scheduled Events occur in a separate thread from the algorithm manager, so they run at the correct time.

2. Update the algorithm time.
3. Update the `current_slice` .

4. Pass the `SymbolChangedEvents` to the `on_symbol_changed_events` method.
5. Cancel all open orders for securities that changed their ticker .
6. Add a `Security` object to the `securities` collection for each new security in the universe.
7. Update the `Security` objects with the latest data.
8. Update the `Cash` objects in the CashBook with the latest data.
9. Process fill models for non-market orders.
10. Submit market on open orders to liquidate Equity Option contracts if the underlying Equity has a split warning .
11. Process margin calls .
12. If it's time to settle unsettled cash , perform settlement.
13. Call the on_securities_changed method with the latest security changes.
14. Apply dividends to the portfolio.
15. For securities that have a split warning, update their portfolio holdings and adjust their open orders to account for the split.
16. Update consolidators with the latest data.
17. Pass the `Dividends` to the `on_dividends` method.
18. Pass the `Splits` to the `on_splits` method.
19. Pass the `Delistings` to the `on_delistings` method.
20. Pass the `Slice` to the `on_data` method.
21. Perform universe selection .
22. Pass the `Slice` to the `update` method of each Alpha model .
23. Pass the `Insight` objects from the Alpha model to the Portfolio Construction model .
24. Pass the `PortfolioTarget` objects from the Portfolio Construction model to each Risk Management model .
25. Pass the risk-adjusted `PortfolioTarget` objects from the Risk Management models to the Execution model .

When your algorithm stops executing, LEAN calls the `on_end_of_algorithm` method .

Key Concepts

Python and LEAN

Introduction

The LEAN open-source package powers the core functionality of QuantConnect. Strategies plugin to LEAN to request data, and process trades. LEAN is written in C# and accessed in python with a defined API. Most of these interactions are using native python types, with a few key exceptions. To comfortably use LEAN you should learn to recognize error messages when you hit a non-native API, and how to convert back to a native type.

Enumerable Types

When LEAN returns an enumerable, the object should be converted to a `list()` to be easily read by python classes. Popular API methods which return an enumerable are shown below:

Historical Data

QuantConnect also provides methods to request historical data that return a pandas dataframe, or an enumerable. If loading a large amount of data using an enumerable streams the results reducing RAM usage.

The enumerable types might look like the `MemoizingEnumerable` that is a raw C# type accessible in python.

```
qb = QuantBook()
start = datetime(2023, 2, 1)
end = datetime(2023, 2, 4)
ibm = qb.AddEquity('IBM', Resolution.Minute)
bars = qb.History[TradeBar](ibm.Symbol, start, end, Resolution.Minute)
bars
```

✓ 0.3s Python

```
<QuantConnect.Util.MemoizingEnumerable[TradeBar] object at 0x7fd5313209c0>
```

You can directly enumerate this object with standard python `for` loops, or to convert it to a native type, wrap the object in the `list()` :

```
# Loop over enumerable
import numpy as np
close = []
for bar in bars:
    close.append( bar.Close )
print( np.mean(close) )

# Manipulate as a list
barList =  list(bars)
len(barList)
```

✓ 0.8s Python

```
135.29654786324787

1170
```

Underlying Plugin Architecture

Your strategy code implements the QCAlgorithm class, which can communicate with the LEAN Engine. These method calls are done at the same speed as cython, directly invoking calls at the C layer of python. Your algorithm is imported as syntax checked, interpreted python.

Key Concepts

Duck Typing

Introduction

Duck typing is popular type system in Python. The language documentation defines duck typing as follows:

> A programming style which does not look at an object's type to determine if it has the right interface; instead, the method or attribute is simply called or used ("If it looks like a duck and quacks like a duck, it must be a duck.") By emphasizing interfaces rather than specific types, well-designed code improves its flexibility by allowing polymorphic substitution.
>
> Duck-typing avoids tests using `type()` or `isinstance()` . (Note, however, that duck-typing can be complemented with abstract base classes .) Instead, it typically employs `hasattr()` tests or EAFP programming.

You can use duck-typing to add attributes to all non-primitive types in Python

```py
# Duck-typing enables you to add attributes to non-primitive types in Python.
class Duck:
    pass

duck = Duck()
duck.quacks = "Quack!"
print(duck.quacks)
```

Custom Security Properties

You can add attributes to the `Security` object. For example, you can add an exponential moving average .

```py
# Add an EMA indicator to the SPY Equity object.
equity = self.add_equity("SPY")
equity.ema = self.ema(equity.symbol, 10, Resolution.DAILY)
```

This feature is helpful because you can get the `Security` object from the `securities` object.

```py
# Get the SPY Equity object from the securities object to get the EMA indicator.
ema = self.securities["SPY"].ema.current.value
```

Other Types

You can add properties to all types. However, these properties live in the Python space, so you can't access them without a reference. The following example demonstrates that you can add an exponential moving average to the `symbol` attribute of a `Security` object.

```py
# Add an EMA to the SPY security.
equity = self.add_equity("SPY")
self._symbol = equity.symbol
self._symbol.ema = self.ema(self._symbol, 10)
ema = self._symbol.ema.current.value

ema = self.securities["SPY"].symbol.ema.current.value
```

Examples

The following examples demonstrate some common practices for duck typing.

Example 1: Store EMA Indicator

The following example implements a trend-following strategy using the EMA indicator for the top 20 liquid equities. It uses duck typing to store the

EMA indicator in each Security object , so the indicator is discarded together with the `Security` when it is removed from the universe.

```py
class DuckTypingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Weekly renewal on the universe to allow time to capitalize on the trend.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Trade on the top 20 liquid equities since they have higher volume and hence price movements.
        self._universe = self.add_universe(self.universe.top(20))

    def on_data(self, slice: Slice) -> None:
        for kvp in self._universe.members:
            symbol, security = kvp.key, kvp.value

            # Trade on updated price data.
            bar = slice.bars.get(symbol)
            if bar:
                ema = security.ema.current.value
                # Switch to long if the current price is above EMA, suggesting an uptrend.
                if bar.close > ema and not self.portfolio[symbol].is_long:
                    self.set_holdings(symbol, 0.025)
                elif bar.close < ema and not self.portfolio[symbol].is_short:
                    self.set_holdings(symbol, -0.025)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for removed in changes.removed_securities:
            # Liquidate if it is not in the top 50 liquidity stocks anymore.
            self.liquidate(removed.symbol)
            # Release resources that update automatic indicators
            self.deregister_indicator(removed.ema)

        for added in changes.added_securities:
            # Use duck typing to create an EMA indicator for trend trading.
            # Discarded together when the security is removed from the universe, you won't need to handle it.
            added.ema = self.ema(added.symbol, 60, Resolution.DAILY)
            # Warm up the EMA indicator to ensure its readiness for immediate usage.
            self.warm_up_indicator(added.symbol, added.ema, Resolution.DAILY)
```

Key Concepts

Research Guide

Introduction

We aim to teach and inspire our community to create high-performing algorithmic trading strategies. We measure our success by the profits community members create through their live trading. As such, we try to build the best quantitative research techniques possible into the product to encourage a robust research process.

Hypothesis-Driven Research

We recommend you develop an algorithmic trading strategy based on a central hypothesis. You should develop an algorithm hypothesis at the start of your research and spend the remaining time exploring how to test your theory. If you find yourself deviating from your core theory or introducing code that isn't based around that hypothesis, you should stop and go back to thesis development.

Wang et al. (2014) illustrate the danger of creating your hypothesis based on test results. In their research, they examined the earnings yield factor in the technology sector over time. During 1998-1999, before the tech bubble burst, the factor was unprofitable. If you saw the results and then decided to bet against the factor during 2000-2002, you would have lost a lot of money because the factor performed extremely well during that time.

Hypothesis development is somewhat of an art and requires creativity and great observation skills. It is one of the most powerful skills a quant can learn. We recommend that an algorithm hypothesis follow the pattern of cause and effect. Your aim should be to express your strategy in the following sentence:

> *A change in {cause} leads to an {effect}.*

To search for inspiration, consider causes from your own experience, intuition, or the media. Generally, causes of financial market movements fall into the following categories:

- Human psychology
- Real-world events/fundamentals
- Invisible financial actions

Consider the following examples:

Cause	***leads to***	**Effect**
Share class stocks are the same company, so any price divergence is irrational...		A perfect pairs trade. Since they are the same company, the price will revert.
New stock addition to the S&P500 Index causes fund managers to buy up stock...		An increase in the price of the new asset in the universe from buying pressure.
Increase in sunshine-hours increases the production of oranges...		An increase in the supply of oranges, decreasing the price of Orange Juice Futures.
Allegations of fraud by the CEO causes investor faith in the stock to fall...		A collapse of stock prices for the company as people panic.
FDA approval of a new drug opens up new markets for the pharmaceutical company...		A jump in stock prices for the company.
Increasing federal interest rates restrict lending from banks, raising interest rates...		Restricted REIT leverage and lower REIT ETF returns.

There are millions of potential alpha strategies to explore, each of them a candidate for an algorithm. Once you have chosen a strategy, we recommend exploring it for no more than 8-32 hours, depending on your coding ability.

Data Mining Driven Research

An alternative view is to follow any statistical anomaly without explaining it. In this case, you can use statistical techniques to identify the

discontinuities and eliminate them when their edge is gone. Apparently, Renaissance Technologies has data mining models like this.

Overfitting

Overfitting occurs when you fine-tune the parameters of an algorithm to fit the detail and noise of backtesting data to the extent that it negatively impacts the performance of the algorithm on new data. The problem is that the parameters don't necessarily apply to new data and thus negatively impact the algorithm's ability to generalize and trade well in all market conditions. The following table shows ways that overfitting can manifest itself:

Data Practice	Description
Data Dredging	Performing many statistical tests on data and only paying attention to those that come back with significant results.
Hyper-Tuning Parameters	Manually changing algorithm parameters to produce better results without altering the test data.
Overfit Regression Models	Regression, machine learning, or other statistical models with too many variables will likely introduce overfitting to an algorithm.
Stale Testing Data	Not changing the backtesting data set when testing the algorithm. Any improvements might not be able to be generalized to different datasets.

An algorithm that is dynamic and generalizes to new data is more valuable to funds and individual investors. It is more likely to survive across different market conditions and apply to new asset classes and markets.

If you have a collection of factors, you can backtest over a period of time to find the best-performing factors for the time period. If you then narrow the collection of factors to just the best-performing ones and backtest over the same period, the backtest will show great results. However, if you take the same best-performing factors and backtest them on an out-of-sample dataset, the performance will almost always underperform the in-sample period. To avoid issues with overfitting, follow these guidelines:

- Use walk-forward optimization to train your models on historical data and test them on future data.
- Test your strategy with live paper trading.
- Test your model on different asset classes and markets.

Look-ahead Bias

Look-ahead bias occurs when you use information from the future to inform decisions in the present. An example of look-ahead bias is using financial statement data to make trading decisions at the end of the reporting period instead of when the financial statement data was released. Another example is using updated financial statement data before the updated figures were actually available. Wang et al. (2014) show that using the date of when the period ends instead of when the data is actually available can increase the performance of the earnings yield factor by 60%.

Another culprit of look-ahead bias is adjusted price data. Splits and reverse splits can improve liquidity or attract certain investors, causing the performance of the stock to be different than without the split or reverse split. Wang et al (2014) build a portfolio using the 25 lowest priced stocks in the S&P index based on adjusted and non-adjusted prices. The portfolio based on adjusted prices greatly outperformed the one with raw prices in terms of return and Sharpe ratio . In this case, if you were to analyze the low price factor with adjusted prices, it would lead you to believe the factor is very informative, but it would not perform well out-of-sample with raw data.

Look-ahead bias can also occur if you set the universe to assets that have performed well during the backtest period or initialize indicators with values that have performed well during the backtest period. To avoid issues with look-ahead bias, trade a dynamic universe of assets and use point-in-time data. If point-in-time data is not available for the dataset you use, apply a reporting lag. Since the backtesting environment is event-driven and you can't pass the time frontier, it naturally helps to reduce the risk of look-ahead bias.

Survivorship Bias

Survivorship bias occurs when you test a strategy using only the securities that have survived to the end of the testing period and omit securities that have been delisted. If you use the current constituents of an index and backtest over the past, you'll most likely outperform the index because many of the underperformers have dropped out of the index over time and outperformers have been added to the index. In this case, the current constituent universe would consist of only outperformers. This technique is a form of look-ahead bias because if you were trading the strategy in real-time throughout the backtest period, you would not know the current index constituents until today.

If you analyze a dataset that has survivorship bias, you can discover results that are opposite of the true results. For example, Wang et al. (2014) analyze the low volatility factor using two universes. The first universe was the current S&P 500 constituents and the second universe was the point-

in-time constituents. When they used the point-in-time constituents, the low volatility quintile outperformed the high volatility quintile. However, when they used the current constituents, the high volatility quintile significantly outperformed the low volatility quintile.

To avoid issues with survivorship bias, trade a dynamic universe of assets and use the datasets in the Dataset Market . We thoroughly vet the datasets we add to the market to ensure they're free of survivorship bias.

Outliers

Outliers in a dataset can have large impacts on how models train. In some cases, you may leave outliers in a dataset because they can contain useful information. In other cases, you may want to transform the data to handle the outlier data points. There are several common methods to handle outliers.

Winsorization Method

The winsorization method removes outliers at the x% of the extremes. For example, if you winsorize at 1%, it removes the 1% of data points with the lowest value and the 1% of data points with the highest value. This threshold percentage is subjective, so it can result in overfitting.

IQR Method

The interquartile range method removes data points that fall outside of the interval $[Q_1 - k(Q_3 - Q_1),\ Q_3 + k(Q_3 - Q_1)]$ for some constant $k \geq 0$. This method can exclude up to 25% of observations on each side of the extremes. The interquartile range method doesn't work if the data is skewed or non-normal. Therefore, the disadvantage to this method is you need to review the factor distribution properties. If you need to, you can normalize the data with z-scores.

$$Z = \frac{x - \mu}{\sigma}$$

Factor Ranking Method

The factor ranking method ranks the data values instead of taking their raw values. With this method, you don't need to remove any outlier data points. This method transforms the data into a uniform distribution. After converting the data to a uniform distribution, Wang et al. (2014) perform an inverse normal transformation to make the factor values normally distributed. Wang et al. found this ranking technique outperforms the z-score transformation, suggesting that the distance between factor scores doesn't add useful information.

Short Availability

To open a short position, you need to borrow shares from an investor or brokerage that owns them. Wang et al. (2014) show that some factors have a larger edge for long positions and some have a larger edge for short positions. Their research found that the factors that perform better on the short side usually target more securities that are on hard-to-borrow lists than factors that perform better on the long side. Their research also shows that a large part of the returns from these short-biased factors come from securities that are on hard-to-borrow lists. Therefore, if you don't account for short availability, your backtest results can be unrealistic and not reproducible in live trading.

To avoid issues with short availability, add a shortable provider to your algorithm. The shortable provider only lets you open a short trade if there are actually shares available to borrow. By default, LEAN doesn't simulate the cost of borrowing shorts in backtests, but you can enable them by setting a margin interest model.

Examples

The following examples demonstrate some common practices for applying research.

Example 1: Hypothesis-Driven Research

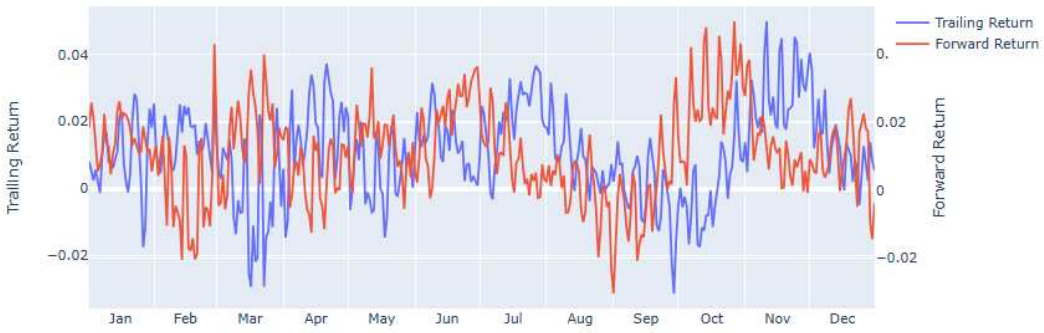
The following example hypothesized that 2 different classes of Google stocks are cointegrated with each other, and we can capitalize spread reversal from occasional, irrational price divergence. To prove it, we make use of the augmented Dickey-Fuller test to verify the cointegration relationship.

```py
from sklearn.linear_model import LinearRegression
from statsmodels.tsa.stattools import adfuller

class HypothesisDrivenResearchAlgorithm(QCAlgorithm):
    # The threshold that the spread/residual of the cointegrated series triggers a trade.
    threshold = 0
    # Store the coefficient and intercept of the cointegrated series for calculating the spread of a new data point.
    coefficients = [0, 0]
    # Store the price series of each symbol for cointegration calculation.
    windows = {}
```

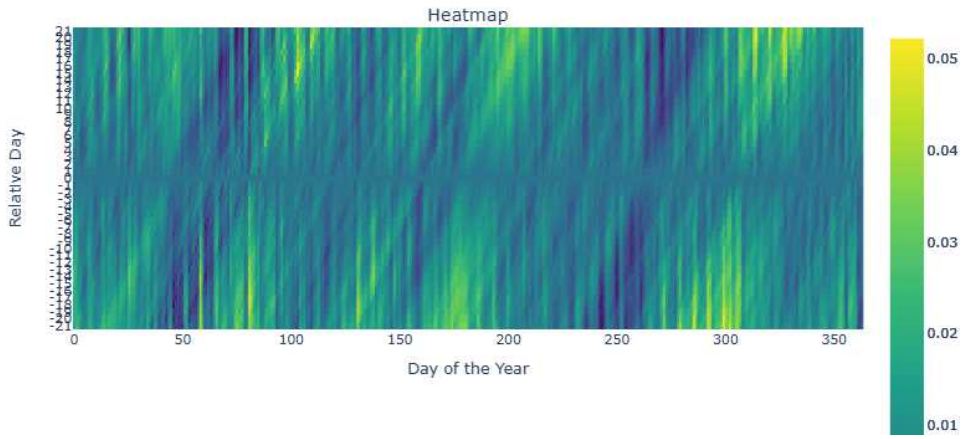
```
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Subscribe to 2 classes of Google stocks to trade their price divergence.
        self.goog1 = self.add_equity("GOOGL", Resolution.MINUTE).symbol        # Class A
        self.goog2 = self.add_equity("GOOG", Resolution.MINUTE).symbol         # Class C

        for symbol in [self.goog1, self.goog2]:
            self.windows[symbol] = RollingWindow(252)

            # Add a consolidator to aggregate a daily bar to update the window's daily price series.
            consolidator = TradeBarConsolidator(timedelta(1))
            consolidator.data_consolidated += lambda _, bar: self.windows[bar.symbol].add(bar.close)
            # Subscribe to the consolidator to update automatically.
            self.subscription_manager.add_consolidator(symbol, consolidator)

            # Warm up the rolling window's daily price series with historical data.
            history = self.history[TradeBar](symbol, 253, Resolution.DAILY)
            for bar in history:
                consolidator.update(bar)

        # Adjust the cointegration factor between the 2 classes' monthly price series.
        self.schedule.on(
            self.date_rules.month_start(),
            self.time_rules.at(0, 1),
            self.calculate_cointegration
        )

        self.calculate_cointegration()

    def on_data(self, slice: Slice) -> None:
        bar1 = slice.quote_bars.get(self.goog1)
        bar2 = slice.quote_bars.get(self.goog2)
        if bar1 and bar2:
            # Calculate the current cointegrated series spread.
            residual = self.coefficients[0] * bar2.close + self.coefficients[1] - bar1.close

            # If the residual is lower than the negative threshold, it means class A's price is much higher than it should be compared to
class C.
            # We sell class A and buy class C to bet on their price convergence.
            if residual < -self.threshold and not self.portfolio[self.goog1].is_short:
                self.set_holdings(self.goog1, -0.5)
                self.set_holdings(self.goog2, 0.5 * self.coefficients[0])
            # If the residual is higher than the threshold, it means class A price is much lower than what it should be compared to class
C.
            # We buy class A and sell class C to bet on their price convergence.
            elif residual > self.threshold and not self.portfolio[self.goog1].is_long:
                self.set_holdings(self.goog1, 0.5)
                self.set_holdings(self.goog2, -0.5 * self.coefficients[0])
            # Close positions of the price are converged.
            elif (self.portfolio[self.goog1].is_short and residual > 0) or (self.portfolio[self.goog1].is_long and residual < 0):
                self.liquidate()

    def calculate_cointegration(self) -> None:
        # Lag direction is unimportant; it is just a sign flip in the linear regression, so we don't need to flip the window order.
        y = np.array(list(self.windows[self.goog1])).reshape(-1, 1)
        x = np.array(list(self.windows[self.goog2])).reshape(-1, 1)

        # Perform Linear Regression on both price series to investigate their relationship.
        lr = LinearRegression().fit(x, y)
        slope = lr.coef_[0]
        intercept = lr.intercept_

        # Calculate the residuals series to check if it is stationary, meaning if the 2 price series move together.
        residuals = y - (intercept + slope * x)

        # Check if the residuals are stationary using the augmented Dickey-Fuller test.
        # Reject the null hypothesis of a unit root is present if test statistic <= -3.45 (approximate α=0.05 for n=250)
        # Which means no unit root for difference series and the residuals are stationary.
        adf_reject = adfuller(residuals)[0] <= -3.45
        if adf_reject:
            # If cointegrated, update the positional sizing ratio and the spread threshold of the trade trigger.
            self.coefficients = [slope, intercept]
            self.threshold = 2 * np.std(residuals)
        else:
            # If not cointegrated, liquidate and set the size to zeros for no positions.
            self.liquidate()
            self.coefficients = [0, 0]
            self.threshold = 100000000          # An arbitrarily large number that the class A price will never reach.
```

Example 2: Data-Driven Research

By exploring the return pattern of SPY in 2020, we use that information to invest in 2021. Since the macroeconomic environment is similar (low interest rate), we assume the market seasonality would follow through. We can observe from the figures below that an around 50-day cycle, like a sine function, exists, and the next cycle starts around Jan 2021. Hence, we switch long and short every 25 days in 2021.



Return Analysis



PY

```
class DataDrivenResearchAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Request SPY data to trade it.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol
        # Add a warm-up period so SPY has a price before the first trade.
        self.set_warm_up(timedelta(7))

    def on_warmup_finished(self) -> None:
        # According to the data, the first cycle is downgoing.
        self.set_holdings(self.spy, -1)
        # Schedule a switch 25 days later.
        self.last_scheduled_event = self.schedule.on(
            self.date_rules.On(self.time + timedelta(25)),
            self.time_rules.at(9, 30),
            self.switch
        )

    def switch(self) -> None:
        # Switch long/short after the cycle change.
        if self.portfolio[self.spy].is_long:
            self.set_holdings(self.spy, -1)
        else:
            self.set_holdings(self.spy, 1)

        # Schedule the next switch 25 days later.
        self.schedule.remove(self.last_scheduled_event)
        self.last_scheduled_event = self.schedule.on(
            self.date_rules.On(self.time + timedelta(25)),
            self.time_rules.at(9, 30),
            self.switch
        )
```

Key Concepts

Behavioral Finance

Introduction

Behavioral finance studies how cognitive biases and emotional responses cause investors to make irrational decisions. These systematic errors create persistent market inefficiencies that algorithmic trading strategies can exploit. Understanding the behavioral "why" behind a strategy helps you design better hypotheses, avoid overfitting to noise, and recognize when an edge might disappear.

Traditional finance assumes that market participants act rationally and that prices reflect all available information. In practice, investors consistently deviate from rationality in predictable ways. They follow the crowd, anchor to irrelevant reference points, overreact to recent news, and avoid realizing losses. These patterns repeat across markets and time periods because they stem from deep-seated cognitive tendencies.

Algorithmic trading is well suited to exploit behavioral biases because algorithms execute rules without emotion. However, the algo trader who designs the rules is still susceptible to the same biases. Knowing which biases drive your strategy and which biases threaten your decision-making process makes you a more effective quant.

This section covers four strategy families and the biases behind them:

- Momentum - Assets that have performed well recently tend to continue performing well. Exploits herd mentality, anchoring bias, and confirmation bias.
- Mean Reversion - Assets that deviate from historical averages tend to revert. Exploits overreaction, the disposition effect, and availability bias.
- Value Factor - Undervalued assets tend to appreciate as the market corrects its pessimism. Exploits loss aversion, recency bias, and status quo bias.
- Quality Factor - High-quality companies tend to outperform because investors chase exciting stories instead. Exploits narrative fallacy, overconfidence, and the endowment effect.

The section also includes a reference page covering the 12 most dangerous investment biases and example algorithms for each strategy type.

Momentum

Momentum strategies buy assets that have performed well recently and sell assets that have performed poorly. The core observation is that recent winners tend to keep winning and recent losers tend to keep losing over intermediate time horizons (3 to 12 months). This effect has been documented across equities, bonds, commodities, and currencies.

Behavioral biases that create momentum

Several cognitive biases cause momentum to persist in financial markets. **Herd mentality** amplifies trends as investors follow the crowd into rising assets. **Anchoring bias** causes slow adjustment to new information, creating gradual price drifts that momentum captures. **Confirmation bias** reinforces trends as investors seek information that supports their existing positions and dismiss contradictory signals.

Algorithmic approach

A typical cross-sectional momentum algorithm follows these steps:

- Define a universe of assets (for example, US equities with sufficient liquidity).
- Rank each asset by its trailing return over a lookback period (commonly 12 months with a 1-month skip to avoid short-term reversal).
- Go long the top decile of performers. Optionally, go short the bottom decile.
- Rebalance the portfolio monthly or at a fixed interval.

The 1-month skip at the end of the lookback period is important because very short-term returns (1 month or less) tend to reverse rather than continue. This short-term reversal is a separate effect driven by liquidity and microstructure, not the behavioral biases that drive intermediate-term momentum.

Biases that threaten momentum traders

While momentum strategies exploit behavioral biases in others, the algo trader designing and managing the strategy is susceptible to several biases:

- **Herd mentality** - Piling into popular momentum strategies when they are already crowded increases exposure to momentum crashes.
- **Anchoring** - Anchoring to a recent period of strong momentum returns can cause you to over-allocate to the strategy.
- **Overconfidence** - Holding momentum positions too long because you believe the trend will continue beyond what the evidence supports.

Mean Reversion

Mean reversion strategies bet that assets which have deviated significantly from their historical average will revert back. When a price moves far above or below its mean, the strategy takes a contrarian position and profits when the price returns to normal levels. Mean reversion operates on shorter time horizons than momentum, typically days to weeks.

Behavioral biases that create reversion opportunities

Several cognitive biases cause prices to overshoot and then revert. **Recency bias** causes investors to overweight recent events and extrapolate short-term trends, creating overreactions that correct over time. The **disposition effect** drives investors to sell winners too early and hold losers too long, creating predictable reversion patterns. **Availability bias** causes dramatic events to receive outsized attention, producing temporary mispricings that fade as investors process the actual impact.

Algorithmic approach

A typical mean reversion algorithm follows these steps:

- Select one or more assets and compute a measure of deviation from the mean. Common measures include z-scores, Bollinger Bands, and RSI.
- When the deviation exceeds a threshold (for example, price drops below the lower Bollinger Band), enter a contrarian position.
- Exit the position when the price reverts to the mean or crosses a take-profit level.
- Use stop-losses to protect against cases where the deviation reflects a genuine regime change rather than a temporary overreaction.

Pairs trading is a common mean reversion variant. You identify two correlated assets, monitor the spread between them, and trade the convergence when the spread widens beyond its historical norm.

Biases that threaten mean reversion traders

Mean reversion traders face their own behavioral pitfalls:

- **Recency bias** - Extrapolating a recent deviation as the "new normal" instead of recognizing it as a temporary overreaction. This can cause you to exit too early or avoid entering at all.
- **Overconfidence** - Believing a deviation will revert when it actually reflects a fundamental shift. Not every drop is a buying opportunity; sometimes the new price is the correct price.

Value Factor

Value strategies buy assets that appear undervalued relative to their fundamentals and sell assets that appear overvalued. The value premium is one of the most studied anomalies in finance: stocks with low price-to-earnings ratios, low price-to-book ratios, or high dividend yields have historically outperformed their expensive peers over long horizons.

Behavioral biases that create value opportunities

Several cognitive biases cause the market to misprice value stocks. **Loss aversion** causes investors to dump declining stocks to avoid further pain, pushing prices below intrinsic value. **Recency bias** leads investors to overweight recent poor performance and project it into the future, treating temporarily struggling companies as permanently impaired. **Status quo bias** makes investors reluctant to buy unfamiliar or out-of-favor stocks, leaving undervalued stocks cheap for longer.

Algorithmic approach

A typical value algorithm follows these steps:

- Define a universe of equities with sufficient liquidity and market capitalization.
- Screen for value using fundamental data: low P/E ratio, low price-to-book ratio, high dividend yield, or a composite of multiple value factors.
- Rank the universe by the value metric and select the most undervalued stocks.
- Construct an equal-weight or value-weight portfolio of the selected stocks.
- Rebalance quarterly to allow time for the value thesis to play out while keeping the portfolio current.

Value strategies require patience because undervalued stocks can remain cheap for extended periods. The quarterly rebalance frequency reflects this longer time horizon compared to momentum or mean reversion strategies.

Biases that threaten value traders

Value investors face specific behavioral challenges:

- **Loss aversion** - Value stocks often continue to decline after you buy them. The same bias that creates the opportunity also makes it difficult to hold positions through drawdowns.
- **Narrative fallacy** - Requiring a compelling "story" for why a cheap stock will recover. Sometimes the quantitative signal is sufficient, and waiting for a narrative can cause you to miss the opportunity.

Quality Factor

Quality factor strategies invest in companies with strong fundamentals: high profitability, stable earnings, low financial leverage, and consistent growth. These "boring but profitable" companies tend to outperform over time, despite receiving less attention than high-growth or turnaround stories.

Behavioral biases that create the quality premium

Several cognitive biases cause investors to undervalue quality stocks. The **narrative fallacy** draws investors toward stocks with exciting stories while steady, predictable businesses get ignored. **Overconfidence bias** leads investors to take concentrated bets on speculative stocks while overlooking quality companies with proven track records. The **endowment effect** makes investors reluctant to sell overvalued glamour stocks and rotate into higher-quality alternatives.

Algorithmic approach

A typical quality factor algorithm follows these steps:

- Define a universe of equities with sufficient liquidity and market capitalization.
- Screen for quality using fundamental metrics: high return on equity (ROE), low debt-to-equity ratio, stable or growing earnings, and strong gross margins.
- Rank the universe by a composite quality score that combines multiple metrics.
- Construct a portfolio of the highest-ranked quality stocks.
- Rebalance quarterly as new fundamental data becomes available.

Quality strategies pair well with value strategies because they address a common criticism of pure value investing: buying cheap stocks that are cheap for good reason (value traps). Adding a quality filter helps you avoid deteriorating businesses and focus on undervalued companies with strong fundamentals.

Biases that threaten quality traders

Quality factor investors face their own behavioral challenges:

- **Narrative fallacy** - Even algo traders can be tempted to override quality signals in favor of an exciting growth story. Trust the quantitative metrics over qualitative narratives.
- **Overconfidence** - Believing that a high-risk, high-reward bet will pay off better than a diversified portfolio of quality companies. The quality premium is modest but consistent, and chasing larger returns often destroys value.

Dangerous Investment Biases

The following 12 cognitive biases are among the most dangerous for investors. Each bias creates exploitable market inefficiencies, but each can also undermine your own trading decisions. For every bias, consider two questions: how can your algorithm exploit this bias in others, and how can you prevent this bias from affecting your algorithm design?

1. Anchoring bias

Anchoring is the tendency to over-rely on the first piece of information you encounter. Investors anchor to purchase prices, 52-week highs, analyst price targets, or round numbers. This causes them to adjust too slowly when new information changes an asset's fair value.

Susceptible strategies: Momentum (slow adjustment creates the trends that momentum exploits), Value Factor (anchoring to past prices instead of current intrinsic value).

Algorithmic antidote: Use relative metrics (percentile ranks, z-scores) instead of absolute price levels. Recompute signals from scratch at each rebalance rather than adjusting from previous values.

2. Availability bias

Availability bias is the tendency to overweight information that is easy to recall, usually because it is recent, dramatic, or emotionally charged. A vivid market crash receives more mental weight than a slow, steady recovery. This causes investors to overreact to dramatic events and create temporary mispricings.

Susceptible strategies: Mean Reversion (dramatic events cause temporary mispricings that revert as emotions fade).

Algorithmic antidote: Base decisions on systematic data analysis rather than salient events. Weight all data points equally unless you have a quantitative reason to do otherwise.

3. Confirmation bias

Confirmation bias is the tendency to seek information that supports your existing beliefs and ignore information that contradicts them. An investor who is bullish on a stock will focus on positive news and dismiss negative signals. This selective processing reinforces trends and delays corrections.

Susceptible strategies: Momentum (reinforces the belief that trends will continue), All types (ignoring disconfirming signals in backtests).

Algorithmic antidote: Validate strategies with out-of-sample data and stress tests. Actively look for conditions where your strategy fails. If you can't find scenarios where it loses money, your testing process likely has confirmation bias built into it.

4. Disposition effect

The disposition effect is the tendency to sell winning investments and hold losing ones. Investors want to realize gains (which feels good) and avoid realizing losses (which feels bad). This creates predictable patterns: winning stocks face selling pressure and losing stocks face holding pressure, both of which eventually correct.

Susceptible strategies: Mean Reversion (disposition-driven selling and holding creates reversion patterns), Momentum (undermines trend following by encouraging premature profit-taking).

Algorithmic antidote: Remove your entry price from the decision process entirely. Use trailing stops or signal-based exits rather than profit targets tied to your cost basis.

5. Endowment effect

The endowment effect is the tendency to value something more simply because you own it. Investors demand a higher price to sell a stock they own than they would pay to buy the same stock. This creates inertia in portfolios and causes investors to hold overvalued positions longer than they should.

Susceptible strategies: Quality Factor (overvaluing flashy holdings over fundamentally strong alternatives), Value Factor (refusing to sell a holding that has become overvalued).

Algorithmic antidote: Rebalance on a fixed schedule using objective ranking criteria. An algorithm that ranks the entire universe at each rebalance does not "remember" what it owns and is immune to the endowment effect.

6. Herd mentality

Herd mentality is the tendency to follow the crowd rather than think independently. When investors see others buying, they buy. When they see others selling, they sell. This collective behavior amplifies price movements beyond what fundamentals justify and creates the trends that momentum strategies exploit.

Susceptible strategies: Momentum (crowd behavior amplifies trends that the strategy rides).

Algorithmic antidote: Monitor crowding metrics and position concentration. Reduce exposure when too many market participants are on the same side of a trade. Momentum crashes often occur when a crowded trade reverses suddenly.

7. Loss aversion

Losses feel approximately twice as painful as equivalent gains feel pleasurable. This asymmetry causes investors to hold losing positions too long (hoping to break even) and sell winning positions too early (locking in gains). Loss aversion creates undervaluation in declining stocks and suppresses returns in rising stocks.

Susceptible strategies: Value Factor (difficulty holding underperformers through drawdowns), Momentum (cutting winners short instead of letting

them run).

Algorithmic antidote: Define exit rules based on quantitative signals, not profit or loss relative to your entry price. Your algorithm should not know or care about its purchase price when making sell decisions.

8. Narrative fallacy

The narrative fallacy is the tendency to create compelling stories to explain random events. Investors prefer stocks with an exciting narrative (a disruptive technology, a charismatic founder) over stocks with strong but boring fundamentals. This preference channels capital toward glamour stocks and away from quality and value stocks.

Susceptible strategies: Quality Factor (investors prefer exciting stories over fundamentals), Value Factor (requiring a "story" to justify buying an undervalued stock instead of trusting the data).

Algorithmic antidote: Let quantitative signals drive decisions. If your algorithm requires a human-readable narrative to justify every trade, you are introducing narrative bias into a systematic process.

9. Overconfidence bias

Overconfidence is the tendency to overestimate your knowledge, ability, or the precision of your predictions. Overconfident investors trade too frequently, take concentrated positions, and underestimate risk. In algo trading, overconfidence manifests as overfitting to historical data and deploying strategies with insufficient out-of-sample validation.

Susceptible strategies: All types (overtrading, over-concentration, and insufficient diversification).

Algorithmic antidote: Use conservative position sizing. Require a strategy to pass multiple independent tests before deploying capital. Track your prediction accuracy honestly and adjust your confidence accordingly.

10. Recency bias

Recency bias is the tendency to overweight recent events when making decisions. Investors extrapolate recent trends, assuming that what happened last quarter will continue next quarter. This causes overreaction to short-term news and creates mispricings that revert over time.

Susceptible strategies: Mean Reversion (overreaction creates the mean-reversion opportunities), Value Factor (recent poor performance causes investors to undervalue fundamentally sound companies).

Algorithmic antidote: Use longer lookback periods that incorporate multiple market regimes. Evaluate strategy performance across full market cycles, not just recent periods.

11. Status quo bias

Status quo bias is the preference for the current state of affairs. Changing your portfolio feels risky, even when the evidence supports a change. Investors stick with familiar holdings and avoid unfamiliar opportunities, which delays the correction of mispricings.

Susceptible strategies: Value Factor (reluctance to buy out-of-favor or unfamiliar stocks).

Algorithmic antidote: Automate rebalancing so that portfolio changes happen systematically. An algorithm that executes a defined rebalancing rule does not experience the discomfort of change.

12. Sunk cost fallacy

The sunk cost fallacy is the tendency to continue investing in something because of the resources already committed, even when continuing is irrational. Investors double down on losing positions because they have "already invested so much" rather than evaluating the position on its current merits.

Susceptible strategies: Value Factor (averaging down on a stock when the original value thesis is broken).

Algorithmic antidote: Evaluate every position at every rebalance as if you were building the portfolio from scratch. The question is not "should I hold this?" but "would I buy this today at this price?"

Examples

The following examples demonstrate algorithmic strategies that exploit behavioral biases. Each example targets a different strategy family and the biases that create its edge.

Example 1: Cross-Sectional Momentum Strategy

The following algorithm exploits herd mentality and anchoring bias by going long equities with the strongest trailing 12-month returns (skipping the most recent month to avoid short-term reversal). It rebalances monthly using a fundamental universe to identify the top momentum stocks.

```py
class MomentumBiasAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Use a universe of liquid US equities.
        self.universe_settings.resolution = Resolution.DAILY
        self.add_universe(self.fundamental_selection)

        # Rebalance on the first trading day of each month.
        self.schedule.on(
            self.date_rules.month_start(),
            self.time_rules.after_market_open("SPY", 30),
            self.rebalance
        )

        self.selected = []

    def fundamental_selection(self, fundamental: list[Fundamental]) -> list[Symbol]:
        # Filter for liquid equities with prices above $5.
        filtered = [x for x in fundamental if x.has_fundamental_data and x.price > 5 and x.dollar_volume > 10000000]
        sorted_by_volume = sorted(filtered, key=lambda x: x.dollar_volume, reverse=True)[:500]
        return [x.symbol for x in sorted_by_volume]

    def rebalance(self) -> None:
        # Calculate trailing 12-month return with a 1-month skip for each security.
        momentum = {}

        for symbol, security in self.active_securities.items():
            # Request 13 months of history: 12-month return skipping the most recent month.
            history = self.history[TradeBar](symbol, 280, Resolution.DAILY)
            history_list = list(history)
            if len(history_list) < 252:
                continue

            # Skip the most recent 21 trading days (~1 month) to avoid short-term reversal.
            end_price = history_list[-22].close
            start_price = history_list[0].close
            if start_price == 0:
                continue

            momentum[symbol] = float(end_price / start_price - 1)

        if not momentum:
            return

        # Select the top 10% of momentum stocks.
        top_count = max(1, len(momentum) // 10)
        sorted_momentum = sorted(momentum.items(), key=lambda x: x[1], reverse=True)[:top_count]
        self.selected = [x[0] for x in sorted_momentum]

        # Equal-weight the portfolio across the selected stocks.
        weight = 1.0 / len(self.selected)
        for symbol in self.selected:
            self.set_holdings(symbol, weight)

        # Liquidate positions no longer in the selection.
        for holding in self.portfolio.values():
            if holding.invested and holding.symbol not in self.selected:
                self.liquidate(holding.symbol)
```

Example 2: Bollinger Band Mean Reversion Strategy

The following algorithm exploits overreaction and the disposition effect. It uses Bollinger Bands on SPY to identify overbought and oversold conditions. When the price drops below the lower band, the algorithm enters a long position betting on reversion to the mean. It exits when the price returns to the middle band.

```
class MeanReversionBiasAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Subscribe to SPY daily data.
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol

        # Create a 20-day Bollinger Band with 2 standard deviations.
        self.bollinger = self.bb(self.spy, 20, 2, MovingAverageType.SIMPLE, Resolution.DAILY)

        # Warm up the indicator with historical data.
        self.set_warm_up(20, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or not self.bollinger.is_ready:
            return
        if self.spy not in slice.bars:
            return

        price = slice.bars[self.spy].close

        if not self.portfolio[self.spy].invested:
            # Enter long when price drops below the lower Bollinger Band (oversold / overreaction).
            if price < self.bollinger.lower_band.current.value:
                self.set_holdings(self.spy, 1, tag=f"Enter long at {price:.2f} (lower band:
{self.bollinger.lower_band.current.value:.2f})")
        elif self.portfolio[self.spy].is_long:
            # Exit when price reverts to the middle band (mean).
            if price >= self.bollinger.middle_band.current.value:
                self.liquidate(self.spy, tag=f"Exit at {price:.2f} (middle band: {self.bollinger.middle_band.current.value:.2f})")
```

Example 3: Fundamental Value Strategy

The following algorithm exploits loss aversion and recency bias by screening for undervalued equities using the price-to-earnings ratio. Investors dump stocks with recent poor performance, pushing prices below intrinsic value. The algorithm identifies these opportunities using fundamental data and rebalances quarterly.

```py
class ValueBiasAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Use a fundamental universe for value screening.
        self.universe_settings.resolution = Resolution.DAILY
        self.add_universe(self.fundamental_selection)

        self.selected = []
        self.last_rebalance_month = -1

    def fundamental_selection(self, fundamental: list[Fundamental]) -> list[Symbol]:
        # Only rebalance quarterly (January, April, July, October).
        if self.time.month not in [1, 4, 7, 10]:
            return Universe.UNCHANGED
        if self.time.month == self.last_rebalance_month:
            return Universe.UNCHANGED
        self.last_rebalance_month = self.time.month

        # Screen for value: liquid stocks with positive earnings and low P/E ratios.
        with_earnings = [x for x in fundamental
                         if x.has_fundamental_data and x.price > 5 and x.dollar_volume > 5000000
                         and 0 < x.valuation_ratios.pe_ratio < 50
                         and x.market_cap > 1e9]

        if not with_earnings:
            return []

        # Select the 20 stocks with the lowest P/E ratio (most undervalued).
        sorted_by_pe = sorted(with_earnings, key=lambda x: x.valuation_ratios.pe_ratio)[:20]
        self.selected = [x.symbol for x in sorted_by_pe]
        return self.selected

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate stocks removed from the universe.
        for security in changes.removed_securities:
            if security.invested:
                self.liquidate(security.symbol)

        # Equal-weight the portfolio across selected value stocks.
        if self.selected:
            weight = 1.0 / len(self.selected)
            for symbol in self.selected:
                self.set_holdings(symbol, weight)
```

Example 4: Quality Factor Strategy

The following algorithm exploits narrative fallacy and overconfidence by screening for high-quality companies that the market overlooks in favor of more exciting stories. It uses return on equity and debt-to-equity ratio as quality metrics, constructs an equal-weight portfolio, and rebalances quarterly.

PY

```
class QualityBiasAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Use a fundamental universe for quality screening.
        self.universe_settings.resolution = Resolution.DAILY
        self.add_universe(self.fundamental_selection)

        self.selected = []
        self.last_rebalance_month = -1

    def fundamental_selection(self, fundamental: list[Fundamental]) -> list[Symbol]:
        # Only rebalance quarterly (January, April, July, October).
        if self.time.month not in [1, 4, 7, 10]:
            return Universe.UNCHANGED
        if self.time.month == self.last_rebalance_month:
            return Universe.UNCHANGED
        self.last_rebalance_month = self.time.month

        # Screen for quality: liquid stocks with high ROE and low debt-to-equity.
        quality = [x for x in fundamental
                   if x.has_fundamental_data and x.price > 5 and x.dollar_volume > 5000000
                   and x.operation_ratios.roe.value > 0
                   and 0 <= x.operation_ratios.total_debt_equity_ratio.value < 1.5
                   and x.market_cap > 1e9]

        if not quality:
            return []

        # Rank by ROE descending (higher ROE = higher quality).
        sorted_by_roe = sorted(quality, key=lambda x: x.operation_ratios.roe.value, reverse=True)[:20]
        self.selected = [x.symbol for x in sorted_by_roe]
        return self.selected

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate stocks removed from the universe.
        for security in changes.removed_securities:
            if security.invested:
                self.liquidate(security.symbol)

        # Equal-weight the portfolio across selected quality stocks.
        if self.selected:
            weight = 1.0 / len(self.selected)
            for symbol in self.selected:
                self.set_holdings(symbol, weight)
```

Key Concepts

Globals and Statics

Introduction

It is often convenient to access the QCAlgorithm API in classes outside of the QCAlgorithm class. To achieve this, pass an instance reference or access a global static variable. Whenever possible, we recommend using an instance reference to keep your global namespace clean.

API Access by Algorithm Instance

You can access the QCAlgorithm API by passing the `self` object into a constructor of your target class. The class constructor receives it as a variable you can use to initialize your algorithm. The following algorithm demonstrates this process:

```py
class CustomPartialFillModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data for trading.
        equity = self.add_equity("SPY", Resolution.HOUR)
        self._spy = equity.symbol
        self._holdings = equity.holdings

        # Set the fill model with the algorithm instance.
        equity.set_fill_model(CustomPartialFillModel(self))

    def on_data(self, data: Slice) -> None:
        # To get all SPY open orders, use the transaction manager.
        open_orders = self.transactions.get_open_orders(self._spy)
        if len(open_orders) != 0: return

        # Hold SPY during 10th-20th in each month.
        if self.time.day > 10 and self._holdings.quantity <= 0:
            self.market_order(self._spy, 105, True)
        elif self.time.day > 20 and self._holdings.quantity >= 0:
            self.market_order(self._spy, -100, True)

class CustomPartialFillModel(FillModel):
    def __init__(self, algorithm: QCAlgorithm) -> None:
        self._algorithm = algorithm
        self._absolute_remaining_by_order_id = {}

    def market_fill(self, asset: Security, order: MarketOrder) -> OrderEvent:
        absolute_remaining = self._absolute_remaining_by_order_id.get(order.id, order.absolute_quantity)
        fill = super().market_fill(asset, order)

        # Allow at most 10 shares to be filled at once.
        fill.fill_quantity = np.sign(order.quantity) * 10

        # If the remaining shares are less than or equal to 10, the order will be fully filled.
        if (min(abs(fill.fill_quantity), absolute_remaining) == absolute_remaining):
            fill.fill_quantity = np.sign(order.quantity) * absolute_remaining
            fill.status = OrderStatus.FILLED
            self._absolute_remaining_by_order_id.pop(order.id, None)
        # Otherwise, the order is only partially filled out, and we need to update the remaining quantity.
        else:
            fill.status = OrderStatus.PARTIALLY_FILLED
            self._absolute_remaining_by_order_id[order.id] = absolute_remaining - abs(fill.fill_quantity)
            price = fill.fill_price
            # Use the instance of the algorithm to log the information
            self._algorithm.debug(f"{self._algorithm.time} - Partial Fill - Remaining {self._absolute_remaining_by_order_id[order.id]}
Price - {price}")

        return fill
```

API Access by Global Static Variable

Occasionally, passing the QCAlgorithm object to your class constructor is impossible, so you need to use global static variables to access the API for debugging and initialization purposes. The most common case is custom data implementations , where the LEAN Engine is creating the objects, and you're unable to specify a custom constructor.

To create a global static variable, assign the reference to the global variable in the algorithm's `initialize` method. This assignment sets the active instance of the variable to the global static. Then, in your custom class, you can access the QCAlgorithm API through the global. The following

algorithm demonstrates this process:

```py
class MyAlgorithmInstance(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Instantiate a custom type as a global variable in the QCAlgorithm instance.
        # Assign self to Static Cape Variable.
        Cape.algorithm = self
        # Request data for trading or signal.
        self._symbol = self.add_data(Cape, "CAPE")

    def on_data(self, data: Slice) -> None:
        cape = data.get(self._symbol)
        if not cape:
            return
        self.plot("CAPE", "Value", cape.value)

class Cape(PythonData):
    def get_source(self, config: SubscriptionDataConfig, date:datetime, isLiveMode: bool) -> SubscriptionDataSource:
        Cape.algorithm.debug("Test Static: GetSource")
        return SubscriptionDataSource("https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/CAPE.csv", SubscriptionTransportMedium.REMOTE_FILE)

    def reader(self, config: SubscriptionDataConfig, line: str, date:datetime, isLiveMode: bool) -> BaseData:
        index = Cape()
        index.symbol = config.symbol
        data = [x.strip() for x in line.split(',')]
        if len(data) < 12 or line.startswith('Date'):
            return index

        try:
            # Construct the custom data type with input data entry.
            index.value = float(data[12])
            index.time = datetime.strptime(data[0], "%Y.%m")
        except ValueError:
            Cape.algorithm.debug("Test Static 2: ValueError")
        return index
```

Key Concepts

Libraries

Introduction

Libraries (or packages) are third-party software that you can use in your projects. You can use many of the available open-source libraries to complement the classes and methods that you create. Libraries reduce your development time because it's faster to use a pre-built, open-source library than to write the functionality. Libraries can be used in backtesting, research, and live trading. The environments support various libraries for machine learning, plotting, and data processing. As members often request new libraries, we frequently add new libraries to the underlying docker image that runs the Lean engine.

This feature is primarily for Python algorithms as not all Python libraries are compatible with each other. We've bundled together different sets of libraries into distinct environments. To use the libraries of an environment, set the environment in your project and add the relevant `import` statement of a library at the top of your file.

Supported Libraries

The following libraries are supported in each environment :

Add New Libraries

To request a new library, contact us . We will add the library to the queue for review and deployment. Since the libraries run on our servers, we need to ensure they are secure and won't cause harm. The process of adding new libraries takes 2-4 weeks to complete. View the list of libraries currently under review on the Issues list of the Lean GitHub repository .

Project Libraries

Project libraries are QuantConnect projects you can merge into your project to avoid duplicating code files. If you have tools that you use across several projects, create a library.

The process to create a library depends on if you use the Cloud Platform , Local Platform , or CLI .

Key Concepts

Debugging Tools

Introduction

When your algorithm throws errors or doesn't operate how you expect, use debugging tools to diagnose and resolve the issue. The tools to debug LEAN algorithms include a built-in debugger, logs, charts, and the Object Store.

Debugger

The debugger is a built-in tool to help you debug coding errors while backtesting. The debugger enables you to slow down the code execution, step through the program line-by-line, and inspect the variables to understand the internal state of the program. The learn how to start and operate the backtest debugger in your development environment, see the Debugging page in the Cloud Platform , Local Platform , or CLI documentation.

Logs

Algorithms can record string messages ('log statements') to a file for analysis after a backtest is complete, or as a live algorithm is running. These records can assist in debugging logical flow errors in the project code or recording key decision moments. Backtests can process millions of data points. If you place a log statement in the wrong place, it can generate gigabytes of logs, which can be difficult to sift through. When you add to the log file, ensure they are readable.

```py
# Logging records messages to a file so you can debug errors and record key decisions.
self.log(f"Signal triggered. Placing limit order: {limit}")
```

Debug statements are the same as log statements, but `debug` method statements are orange in the Cloud Terminal to provide more emphasis. Debug statements are streamed to the Cloud Terminal while the backtest is running. This can be helpful for real-time updates or monitoring in live-trading.

```py
# Debugging streams messages to the Cloud Terminal for real-time analysis.
self.debug("Exposure to the {sector_name} sector is above 50%!")
```

For more information about log statements, see Logging .

Charts

Plotting values over time can help you understand its range, identify outliers in data, and see times when your algorithm generates unusual values. Before you create a plot, develop a theory about what you expect to see before you view it to gain a deeper understanding of your algorithm. For example, a plot of an asset's price standard deviations should be between -4 and +4, with any outliers happening during extreme market conditions like the COVID pandemic in March 2020. LEAN provides a powerful charting API. You can create time series plots with just a single line of code.

```py
# Create time series plots to understand the asset range and identify outliers.
self.plot("Deviations", "aapl", asset_deviation)
```

For more information about creating charts, see Charting .

Object Store

The Object Store is a key-value data store for low-latency information storage and retrieval. During a backtest, you can build large objects you'd like to analyze and write them for later analysis. This workflow can be helpful when the objects are large and plotting is impossible or when you want to perform analysis across many backtests.

```py
# The Object Store provides a key-value data store optimized for fast access and retrieval of information.
self.object_store.save("key", value)
```

For more information about the Object Store, see Object Store . For a specific example of saving indicator values during a backtest into the Object

Store and then plotting them in the Research Environment, see Example for Plotting .

Performance Chart

QuantConnect provides a series of metrics to help you analyze the computational performance of your algorithms. To enable the Performance Chart , set the `self.settings.performance_sample_period` attribute to the desired sampling period:

```
self.settings.performance_sample_period = timedelta(7)
```

Python Profilers from the Python Standard Library enable you to find the functions and methods in your algorithm that consume most of the runtime. To analyze your algorithm's efficiency with the Python Profilers, follow these steps:

1. Open the project in the Cloud Platform , Local Platform , or a local IDE if you use the CLI.
2. At the top of the main.py file, import the classes you need and enable the `Profile` .

   ```
   from cProfile import Profile
   from pstats import Stats
   from io import StringIO

   # Create the profile and start collecting profiling data.
   profile = Profile()
   profile.enable()
   ```

3. Define or extend the on_end_of_algorithm method to `disable` the `Profile` and save information on the most time-consuming functions to the Object Store .

   ```
   class FunctionTimeConsumptionAlgorithm(QCAlgorithm):
       def initialize(self):
           pass

       def on_end_of_algorithm(self):
           # Stop collecting profiling data
           profile.disable()
           stream = StringIO()
           # Save the top 20 time-consuming functions to a file in the Object Store.
           Stats(profile, stream=stream).sort_stats('cumulative').print_stats(20)
           # Save the profiling data to the Object Store using the algorithm ID.
           self.object_store.save(f"{self.algorithm_id}_profile", stream.getvalue())
   ```

4. Open the Research Environment and create a `QuantBook` .

   ```
   qb = QuantBook()
   ```

5. Get the backtest Id and read the profiling data from the Object Store.

   The process to get the backtest Id depends on if you use the Cloud Platform , Local Platform , or CLI .

   ```
   backtest_id = "8b16cec0c44f75188d82f9eadb310e17"
   profile_output = qb.object_store.read(f"{backtest_id}_profile")
   print(profile_output)
   ```

Memory Metrics

The static `OS` class in the `QuantConnect` namespace exposes memory and CPU metrics that you can read from your algorithm to monitor resource usage during backtests and live trading.

The following table lists the static members of the `OS` class that report memory and CPU usage:

Member	Data Type	Description
`APPLICATION_MEMORY_USED`	`int`	Private memory allocated to the current process, including managed and unmanaged memory, in megabytes.
`TOTAL_PHYSICAL_MEMORY_USED`	`int`	Managed-runtime RAM usage (sampled with `gc.get_total_memory`), in megabytes. LEAN's memory monitor samples this value.
`CPU_USAGE`	`float`	Total CPU usage as a percentage, sampled on a background thread.
`get_server_statistics()`	`Dictionary[str, str]`	Snapshot dictionary with `CPU Usage`, `Used RAM`, `Total RAM`, `Hostname`, and `LEAN Version` entries.

The following examples show how to read these metrics from an algorithm:

```py
# Log the current memory usage of the algorithm process.
self.log(f"Total physical memory used: {OS.TOTAL_PHYSICAL_MEMORY_USED} MB")
self.log(f"Application memory used: {OS.APPLICATION_MEMORY_USED} MB")

# Log the current CPU usage as a percentage.
self.log(f"CPU usage: {OS.CPU_USAGE}%")

# Log a full snapshot of the server statistics.
for kvp in OS.get_server_statistics():
    self.log(f"{kvp.key}: {kvp.value}")
```

LEAN monitors the process's RAM usage during backtests and live trading so it can stop the algorithm gracefully when it approaches the node's memory limit, instead of letting the runtime crash. To avoid killing an algorithm over a short-lived spike, LEAN does not act on the raw reading — it smooths the sampled value first and checks the smoothed value against the limit.

LEAN samples `OS.TOTAL_PHYSICAL_MEMORY_USED` once per minute and feeds each sample into an exponential moving average. The engine sets `emaPeriod = 60`, which gives you roughly a 1-hour rolling window.

```py
# Update the smoothed memory reading each minute.
sample = OS.TOTAL_PHYSICAL_MEMORY_USED
memory_used = int((ema_period - 1) / ema_period * memory_used + (1 / ema_period) * sample)
```

If the smoothed value exceeds the node's memory limit, LEAN terminates the algorithm with the runtime message `Execution Security Error: Memory Usage Maxed Out - {memoryCap}MB max, with last sample of {lastSample}MB.`, where `{memoryCap}` is the node's configured limit and `{lastSample}` is the most recent reading.

JIT Compilation

To increase the speed of some methods, you can use decorators like `@jit` or `@lru_cache`. The `@jit` decorator from Numba is for Just-In-Time (JIT) compilation. It compiles Python code to machine code at runtime to increase the speed of loops and mathematical operations. However, Numba can't compile all Python code. It performs best with NumPy arrays. If you add the `@jit` decorator to your methods, it can make the debugging process more challenging since the code is compiled to machine code. The following code snippet shows an example of using the `@jit` decorator:

```py
import numpy as np
from numba import jit
import time

# Without JIT
def slow_function(arr):
    result = 0
    for i in range(len(arr)):
        result += np.sin(arr[i]) * np.cos(arr[i])
    return result

# With JIT
@jit(nopython=True)
def fast_function(arr):
    result = 0
    for i in range(len(arr)):
        result += np.sin(arr[i]) * np.cos(arr[i])
    return result

# Example usage
arr = np.random.rand(1000000)

# Test without JIT.
start = time.time()
slow_function(arr)
print(f"Slow function took: {time.time() - start} seconds")

# Test with JIT.
start = time.time()
fast_function(arr)  # This will compile first, then run.
print(f"Fast function took: {time.time() - start} seconds")
```

Slow function took: 1.1557221412658691 seconds

Fast function took: 0.32973551750183105 seconds

The `@lru_cache` decorator from the `functools` module is for Least Recently Used (LRU) caching. It caches the result of a function based on its arguments, so that the function body doesn't have to repeatedly execute when you call the function multiple times with the same inputs. The `maxsize` argument of the decorator defines the number of results to cache. For example, `@lru_cache(maxsize=512)` means that it should cache a maximum of 512 results. If you add the `@lru_cache` decorator to your methods, make sure your function is completely dependent on the arguments (not any global state variables) or else the caching process may lead to incorrect results. The following code snippet shows an example of using the `@lru_cache` decorator:

```py
from functools import lru_cache
import time

# Without caching
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# With caching
@lru_cache(maxsize=512)
def fibonacci_cached(n):
    if n < 2:
        return n
    return fibonacci_cached(n-1) + fibonacci_cached(n-2)

n = 30

# Test without caching.
start = time.time()
for _ in range(n):
    fibonacci(n)
print(f"Fibonacci without cache took: {time.time() - start} seconds")

# Test with caching.
start = time.time()
for _ in range(n):
    fibonacci_cached(n)
print(f"Fibonacci with cache took: {time.time() - start} seconds")
```

Fibonacci without cache took: 4.387492418289185 seconds

Fibonacci with cache took: 9.322166442871094e-05 seconds

Key Concepts

Glossary

alpha

The excess return of an algorithm relative to its benchmark after adjusting for risk. Alpha measures the value added by the algorithm's strategy beyond what could be explained by exposure to market movements.

It is calculated as

$$\alpha = R_p - \left[R_f + \beta\left(R_b - R_f\right)\right]$$

where R_p is the portfolio return, R_f is the risk-free rate, β is the portfolio's beta, and R_b is the benchmark return. A positive alpha indicates the algorithm outperformed its risk-adjusted benchmark.

annual standard deviation

A statistical measure that describes the dispersion of annual returns relative to the mean annual return. It is the square root of the annual variance. Assuming independent and identically distributed daily returns, it is calculated as

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

where σ_{daily} is the standard deviation of daily returns and 252 is the typical number of trading days in a year.

annual variance

A statistical measure that describes the dispersion of annual returns relative to the mean annual return. Assuming independent and identically distributed daily returns, it is calculated as

$$\sigma^2_{\text{annual}} = \sigma^2_{\text{daily}} \times 252$$

where σ^2_{daily} is the variance of daily returns and 252 is the typical number of trading days in a year.

average loss

The average rate of return for unprofitable trades.

It is calculated as

$$\bar{L} = \frac{1}{n_L}\sum_{i=1}^{n_L} R_i$$

where n_L is the number of unprofitable trades and R_i is the return of each unprofitable trade.

average win

The average rate of return for profitable trades.

It is calculated as

$$\bar{W} = \frac{1}{n_W}\sum_{i=1}^{n_W} R_i$$

where n_W is the number of profitable trades and R_i is the return of each profitable trade.

beta

The scale and direction of an algorithm's returns relative to movements in the underlying benchmark.

It is calculated as

$$\beta = \frac{\text{Cov}(R_p, R_b)}{\text{Var}(R_b)}$$

where R_p is the portfolio return and R_b is the benchmark return. A beta of 1 indicates the algorithm moves in lockstep with the benchmark, a beta greater than 1 indicates amplified movements, and a negative beta indicates inverse movements.

capacity

The maximum amount of money an algorithm can trade before its performance degrades from market impact. Capacity is determined by the liquidity of the assets the algorithm trades and the size of positions it takes.

compounding annual return

The annual percentage return that would be required to grow a portfolio from its starting value to its ending value.

It is calculated as

$$\text{CAGR} = \left(\frac{e}{s}\right)^{\frac{1}{y}} - 1$$

where s is starting equity, e is ending equity, and y is the number of years in the backtest period.

day trade

Buying and selling the same asset within one trading day. In the US Equity market, the Financial Industry Regulatory Authority (FINRA) monitors day trading activity and enforces the pattern day trader rule.

drawdown

The largest peak to trough decline in an algorithm's equity curve.

It is calculated as

$$1 - \frac{v_{\text{min}}^{t \geq s}}{v_{\text{max}}^{s}}$$

where v_{max}^{s} is the maximum equity value up to time s and $v_{\text{min}}^{t \geq s}$ is the minimum equity value at time t where $t \geq s$.

equity

The total portfolio value if all of the holdings were sold at current market rates. Equity equals the sum of cash and the market value of all open positions.

expectancy

The expected return per trade.

It is calculated as

$$E = (W_r \times \bar{W}) + (L_r \times \bar{L})$$

where W_r is the win rate, $\bar{W}$ is the average win, L_r is the loss rate, and $\bar{L}$ is the average loss. A positive expectancy indicates the algorithm is expected to be profitable over time.

holdings

The absolute sum of the items in the portfolio. Holdings represent the total market value of all positions, regardless of whether they are long or short.

implied volatility

The volatility of an Option's underlying asset that, when input into an Options pricing model, returns a theoretical value equal to the current market price of the Option.

An initial estimate can be calculated as

$$\sigma \approx \frac{P}{S}\sqrt{\frac{2\pi}{T}}$$

where P is the Option price, S is the underlying asset price, and T is the time to expiration in years. For more information, see Brenner and Subrahmanyam (1988) .

information ratio

The amount of excess return relative to the benchmark per unit of tracking error.

It is calculated as

$$IR = \frac{R_p - R_b}{\sigma_{R_p - R_b}}$$

where R_p is the portfolio return, R_b is the benchmark return, and $\sigma_{R_p - R_b}$ is the tracking error (the standard deviation of the difference between the portfolio and benchmark returns). A higher information ratio indicates more consistent outperformance.

intrinsic value

(Call Option) The price of an asset minus the strike price if the price is above the strike price, otherwise zero.

It is calculated as

$IV_{\text{call}} = \max$

(Put Option) The strike price minus the price of an asset if the price is below the strike price, otherwise zero.

It is calculated as

```
IV_{\text{put}} = \max(K - S,\, 0)
```

where `S` is the current price of the underlying asset and `K` is the strike price of the Option contract.

look ahead bias

The practice of making decisions using information that would not be available until some time in the future. Look-ahead bias invalidates backtest results because the algorithm uses data it would not have access to in live trading.

loss rate

The proportion of trades that were not profitable.

It is calculated as

```
L_r = \frac{n_L}{N}
```

where `n_L` is the number of unprofitable trades and `N` is the total number of trades. The loss rate equals `1 - W_r`, where `W_r` is the win rate.

lowest capacity asset

The asset an algorithm traded that has the lowest capacity. This asset acts as the bottleneck that limits the total capital the algorithm can deploy without degrading performance.

net profit

(Percent) The rate of return across the entire trading period.

It is calculated as

```
\text{Net Profit (\%)} = \frac{e - s}{s} \times 100
```

(Dollar-value) The dollar-value return across the entire trading period.

It is calculated as

```
\text{Net Profit (\$)} = e - s
```

where `s` is starting equity and `e` is ending equity.

out of the money amount

(Call Option) The strike price minus the price of an asset if the price is below the strike price, otherwise zero.

It is calculated as

```
OTM_{\text{call}} = \max(K - S,\, 0)
```

(Put Option) The price of an asset minus the strike price if the price is above the strike price, otherwise zero.

It is calculated as

$$OTM_{\text{put}} = \max(S - K,\, 0)$$

where S is the current price of the underlying asset and K is the strike price of the Option contract.

pattern day trader

A trader who executes four or more day trades in the US Equity market within five business days. FINRA requires pattern day traders to maintain a minimum account equity of $25,000.

payoff

The profit or loss that an Option buyer or seller makes from a trade. Payoff is determined by the difference between the Option's intrinsic value at expiration and the premium paid or received.

Pearson correlation

A measure of the linear relationship between two variables, ranging from -1 to 1.

It is calculated as

$$\rho_{XY} = \frac{\text{Cov}(X,\, Y)}{\sigma_X \sigma_Y}$$

where $\text{Cov}(X, Y)$ is the covariance of the two variables and σ_X, σ_Y are their standard deviations. A value of 1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship.

Probabilistic Sharpe ratio

The probability that the estimated Sharpe ratio of an algorithm is greater than a benchmark.

It is calculated as

$$P\left(\hat{SR} > SR^{\ast}\right) = CDF\left(\frac{(\hat{SR} - SR^{\ast})\sqrt{n-1}}{\sqrt{1 - \hat{\gamma}_{3}\hat{SR} + \frac{\hat{\gamma}_{4}-1}{4}\hat{SR}^{2}}}\right)$$

where $SR^{\ast}$ is the Sharpe ratio of the benchmark, $\hat{SR}$ is the Sharpe ratio of the algorithm, n is the number of trading days, $\hat{\gamma}_{3}$ is the skewness of the algorithm's returns, $\hat{\gamma}_{4}$ is the kurtosis of the algorithm's returns, and CDF is the normal cumulative distribution function. For more information about the PSR, see Bailey and López de Prado (2012) .

profit loss ratio

The ratio of the average win rate to the average loss rate.

It is calculated as

$$PLR = \frac{\bar{W}}{|\bar{L}|}$$

where $\bar{W}$ is the average win and $\bar{L}$ is the average loss. A profit-loss ratio greater than 1 indicates that the average winning trade is larger than the average losing trade.

put call parity

A relationship between the prices of a European call Option, a European put Option, the underlying asset, and the risk-free rate.

It is defined as

$$C(t,K) - P(t,K) = e^{(q-r)t}(S_T - K)$$

where C is the call price, P is the put price, K is the strike price, S_T is the underlying asset price, r is the risk-free rate, q is the dividend yield, and t is the time to expiration.

return

The rate of return across the entire trading period.

It is calculated as

$$R = \frac{e - s}{s}$$

where s is starting equity and e is ending equity.

risk free interest rate

The interest rate an investor can expect to earn on an investment that carries zero risk, such as the interest paid on a 10-year highly rated government treasury note. The risk-free rate serves as a baseline for evaluating the performance of risky investments.

Sharpe ratio

A measure of the risk-adjusted return, developed by William Sharpe.

It is calculated as

$$SR = \frac{E[R_p - R_b]}{\sigma_p}$$

where R_p is the return of the portfolio, R_b is the return of the benchmark, and σ_p is the standard deviation of the portfolio's excess returns. By default, LEAN uses a 0% risk-free rate, so $R_b = 0$. For more information about the Sharpe ratio, see Sharpe (1994) .

Sortino ratio

A measure of the risk-adjusted return, developed by Frank Sortino. Unlike the Sharpe ratio, the Sortino ratio only penalizes returns falling below a user-specified target or required rate of return.

It is calculated as

$$S = \frac{R_p - R_f}{\sigma_d}$$

where R_p is the portfolio return, R_f is the risk-free rate, and σ_d is the downside deviation (the standard deviation of negative returns only). By using downside deviation, the Sortino ratio does not penalize upside volatility.

tail value at risk

The expected loss of a portfolio beyond the Value at Risk threshold, also known as Conditional Value at Risk (CVaR).

Assuming normally distributed returns, it is calculated as

$$\text{TVaR}_{\alpha} = \mu + \sigma \times \frac{\phi[\Phi^{-1}(\alpha)]}{1 - \alpha}$$

where μ is the mean return, σ is the standard deviation, α is the confidence level, ϕ is the standard normal density function, and Φ^{-1} is the inverse of the standard normal cumulative distribution function.

total fees

The total quantity of fees paid for all the transactions during the algorithm's trading period. Total fees include brokerage commissions, exchange fees, and other transaction costs.

total net profit

The rate of return across the entire trading period.

It is calculated as

$$\text{Total Net Profit} = \frac{e - s}{s}$$

where s is starting equity and e is ending equity.

total trades

The number of orders that were filled or partially filled during the algorithm's trading period.

tracking error

A measure of how closely a portfolio follows the index to which it is benchmarked.

It is calculated as

$$TE = \sigma(R_p - R_b)$$

where R_p is the portfolio return, R_b is the benchmark return, and σ denotes the standard deviation. A tracking error of 0 indicates a perfect match with the benchmark.

Treynor ratio

A measurement of the returns earned in excess of the risk-free rate per unit of benchmark risk, developed by Jack Treynor.

It is calculated as

$$T = \frac{R_p - R_f}{\beta}$$

where R_p is the portfolio return, R_f is the risk-free rate, and β is the portfolio's beta. Unlike the Sharpe ratio, which uses total risk (standard deviation), the Treynor ratio uses systematic risk (beta).

unrealized

The amount of profit a portfolio would capture if it liquidated all open positions and paid the fees for transacting and crossing the spread. Unrealized profit becomes realized profit when the positions are closed.

volume

The total value of assets traded for all of an algorithm's transactions during the trading period.

win rate

The proportion of trades that were profitable after transaction fees.

It is calculated as

$$W_r = \frac{n_W}{N}$$

where n_W is the number of profitable trades and N is the total number of trades.

z score

The number of standard deviations a data point is from the mean of a dataset.

It is calculated as

$$Z = \frac{x - \mu}{\sigma}$$

where x is the data point, μ is the mean, and σ is the standard deviation. A z-score of 0 indicates the data point equals the mean. Values beyond ± 3 are commonly considered outliers.

Initialization

Introduction

The `initialize` method is the entry point of your algorithm where you define a series of settings, including security subscriptions, starting cash balances, and warm-up periods. LEAN only calls the `initialize` method one time, at the start of your algorithm.

Set Dates

To set the date range of backtests, call the `set_start_date` and `set_end_date` methods. The dates you provide are based in the algorithm time zone. By default, the end date is yesterday, one millisecond before midnight. In live trading, LEAN ignores the start and end dates.

```
PY
# Set the start and end dates for the algorithm, defining the backtesting period.
self.set_start_date(2013, 1, 5)                 # Set the start date to January 5, 2013.
self.set_end_date(2015, 1, 5)                   # Set the end date to January 5, 2015.
self.set_end_date(datetime.now() - timedelta(7)) # Set the end date to last week.
```

Organization managers can enforce all backtests to end a certain number of months before the current date to reduce the chance of overfitting. For more information, see Out-of-Sample Period .

Set Account Currency

The algorithm equity curve, benchmark, and performance statistics are denominated in the account currency. To set the account currency and your starting cash, call the `set_account_currency` method. By default, the account currency is USD and your starting cash is $100,000. If you call the `set_account_currency` method, you must call it before you call the `set_cash` method or add data . If you call the `set_account_currency` method more than once, only the first call takes effect.

```
PY
# Set the account currency and your starting cash.
self.set_account_currency("BTC") # Set the account currency to Bitcoin and its quantity to 100,000 BTC.
self.set_account_currency("INR") # Set the account currency to Indian Rupees and its quantity to 100,000 INR.
self.set_account_currency("BTC", 10) # Set the account currency to Bitcoin and its quantity to 10 BTC.
```

Set Cash

To set your starting cash in backtests, call the `set_cash` method. By default, your starting cash is $100,000 USD. In live trading, LEAN ignores the `set_cash` method and uses the cash balances in your brokerage account instead.

```
PY
# Set the initial cash balance for the account for supported currencies.
self.set_cash(100000)       # Set the quantity of the account currency to 100,000.
self.set_cash("BTC", 10)    # Set the Bitcoin quantity to 10.
self.set_cash("EUR", 10000) # Set the EUR quantity to 10,000.
```

Set Brokerage and Cash Model

We model your algorithm with margin modeling by default, but you can select a cash account type. Cash accounts don't allow leveraged trading, whereas Margin accounts can support leverage on your account value. To set your brokerage and account type, call the `set_brokerage_model` method. For more information about each brokerage and the account types they support, see the brokerage integration documentation. For more information about the reality models that the brokerage models set, see Supported Models .

```
PY
# Set the brokerage model and account type to model fees, supported order types, and available margin.
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE, AccountType.MARGIN)
```

The `AccountType` enumeration has the following members:

Set Universe Settings

The universe settings of your algorithm configure some properties of the universe constituents, including derivatives such as futures and options.

The following table describes the properties of the `UniverseSettings` object:

Property: `asynchronous` Should the universe selection run asynchronously to boost speed? Data Type: `bool` | Default Value: `False`
Property: `extended_market_hours` Should assets also feed extended market hours? You only receive extended market hours data if you create the subscription with an intraday resolution. If you create the subscription with daily resolution, the daily bars only reflect the regular trading hours. Data Type: `bool` | Default Value: `False`
Property: `fill_forward` Should asset data fill forward? Data Type: `bool` | Default Value: `True`
Property: `minimum_time_in_universe` What's the minimum time assets should be in the universe? Data Type: `timedelta` | Default Value: `timedelta(1)`
Property: `resolution` What resolution should assets use? Data Type: `Resolution` | Default Value: `Resolution.MINUTE`
Method: `schedule.on` What days should universe selection run? If it's `None` , it's the data resolution. For example, US Equity fundamentals universes are daily at midnight. Argument Type: `IDateRule` | Default Value: `None`
Property: `contract_depth_offset` What offset from the current front month should be used for continuous Future contracts ? 0 uses the front month and 1 uses the back month contract. This setting is only available for Future assets. Data Type: `int` | Default Value: `0`
Property: `data_mapping_mode` How should continuous Future contracts be mapped? This setting is only available for Future assets. Data Type: `DataMappingMode` | Default Value: `DataMappingMode.OpenInterest`

Property: `data_normalization_mode` How should historical prices be adjusted? This setting is only available for Equity and Futures assets. Data Type: `DataNormalizationMode` | Default Value: `DataNormalizationMode.ADJUSTED`
Property: `leverage` What leverage should assets use in the universe? This setting is not available for derivative assets. Data Type: `float` | Default Value: `Security.NULL_LEVERAGE`

To set the `universe_settings` , update the preceding properties in the `initialize` method before you add the universe. These settings are globals, so they apply to all universes you create.

```py
# Request second resolution data. This will be slow!
self.universe_settings.resolution = Resolution.SECOND
self.add_universe(self.universe.top(50))
```

For more information about universe settings, see the related documentation for classic and framework algorithms.

Set Security Initializer

Instead of configuring global universe settings, you can individually configure the settings of each security in the universe with a security initializer. Security initializers let you apply any security-level reality model or special data requests on a per-security basis. To set the security initializer, in the `initialize` method, call the `set_security_initializer` method and then define the security initializer.

```py
# A custom security initializer can override default models such as
# setting new fee and fill models for the security.
self.set_security_initializer(self._custom_security_initializer)

def _custom_security_initializer(self, security: Security) -> None:
    security.set_fee_model(ConstantFeeModel(0, "USD"))
```

For simple requests, you can use the functional implementation of the security initializer. This style lets you configure the security object with one line of code.

```py
# Disable the trading fees for each security by using lambda function
# for the set_security_initializer argument.
self.set_security_initializer(lambda security: security.set_fee_model(ConstantFeeModel(0, "USD")))
```

If you call the `set_security_initializer` method, it overwrites the default security initializer. The default security initializer uses the security-level reality models of the brokerage model to set the following reality models of each security:

- Fill
- Slippage
- Fee
- Buying Power
- Settlement
- Short availability
- Margin Interest Rate

The default security initializer also sets the leverage of each security and intializes each security with a seeder function. To extend upon the default security initializer instead of overwriting it, call `add_security_initializer` before you create the subscriptions.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fee_model(ConstantFeeModel(0, "USD"))
```

Add Data

You can subscribe to asset, fundamental, alternative, and custom data. The Dataset Market provides 400TB of data that you can easily import into your algorithms.

Asset Data

To subscribe to asset data, call one of the asset subscription methods like `add_equity` or `add_forex` . Each asset class has its own method to create subscriptions. For more information about how to create subscriptions for each asset class, see Asset Classes .

```
self.add_equity("AAPL")  # Add Apple 1 minute bars (minute by default)
self.add_forex("EURUSD", Resolution.SECOND) # Add EURUSD 1 second bars
```

In live trading, you define the securities you want, but LEAN also gets the securities in your live portfolio and sets their resolution to the lowest resolution of the subscriptions you made. For example, if you create subscriptions in your algorithm for securities with Second, Minute, and Hour resolutions, the assets in your live portfolio are given a resolution of Second.

Alternative Data

To add alternative datasets to your algorithms, call the `add_data` method. For full examples, in the Datasets chapter, select a dataset and see the Requesting Data section.

Custom Data

To add custom data to your algorithms, call the `add_data` method. For more information about custom data, see Importing Data .

Limitations

There is no official limit to how much data you can add to your algorithms, but there are practical resource limitations. Each security subscription requires about 5MB of RAM, so larger machines let you run algorithms with bigger universes. For more information about our cloud nodes, see Resources .

Set Indicators and Consolidators

You can create and warm-up indicators in the `initialize` method.

```
self._symbol = self.add_equity("SPY").symbol
self._sma = self.sma(self._symbol, 20)
self.warm_up_indicator(self._symbol, self._sma)
```

Set Algorithm Settings

The following table describes the `AlgorithmSettings` properties:

Property: `automatic_indicator_warm_up` A flag that defines if automatic indicators are ready on creation. It does not account for built-in `RollingWindow` that stores their historical values , use the `warm_up_indicator` or the `indicator_history` method instead. Data Type: `bool` | Default Value: `False`

Property: `seed_initial_prices`

Determines whether to seed initial prices for all selected and manually added securities.

Data Type: `bool` | Default Value: `False`

Property: `daily_precise_end_time`

A flag that defines if daily bars should have an end_time that matches the market close time (`True`) or the following midnight (`False`).

Data Type: `bool` | Default Value: `True`

Property: `free_portfolio_value`

The buying power buffer value.

To successfully update the `free_portfolio_value` , you must update it after the `initialize` method.

Data Type: `float` | Default Value: `250`

Property: `free_portfolio_value_percentage`

The buying power buffer percentage value.

Data Type: `float` | Default Value: `0.0025`

Property: `ignore_unknown_asset_holdings`

Determines whether to terminate the algorithm when an asset holding is not supported by Lean or the brokerage.

Data Type: `bool` | Default Value: `True`

Property: `liquidate_enabled`

A flag to enable and disable the `liquidate` method.

Data Type: `bool` | Default Value: `True`

Property: `max_absolute_portfolio_target_percentage`

The absolute maximum valid total portfolio value target percentage.

Data Type: `float` | Default Value: `1000000000`

Property: `min_absolute_portfolio_target_percentage`

The absolute minimum valid total portfolio value target percentage.

Data Type: `float` | Default Value: `0.0000000001`

Property: `minimum_order_margin_portfolio_percentage` The minimum order margin portfolio percentage to ignore bad orders and orders with small sizes. Data Type: `float` | Default Value: `0.001`
Property: `performance_sample_period` Performance tracking sample period to use if any, useful to debug performance issues. See Performance Chart . Data Type: `timedelta` | Default Value: `timedelta(0)`
Property: `rebalance_portfolio_on_insight_changes` Rebalance the portfolio when you emit new insights or when insights expire. Data Type: `bool/NoneType` | Default Value: `True`
Property: `rebalance_portfolio_on_security_changes` Rebalance the portfolio when your universe changes. Data Type: `bool/NoneType` | Default Value: `True`
Property: `stale_price_time_span` The minimum time span elapsed to consider a market fill price as stale Data Type: `timedelta` | Default Value: `timedelta(hours=1)`
Property: `trading_days_per_year` Number of trading days per year for this algorithm's portfolio statistics . Data Type: `int` | Default Value: `252`
Property: `warm_up_resolution` The resolution to use during the warm-up period Data Type: `Resolution/NoneType` | Default Value: `None`

To change the `settings` , update some of the preceding properties.

```
# Set the algorithm behavior using the settings property on your algorithm.
self.settings.rebalance_portfolio_on_security_changes = False
self.settings.trading_days_per_year = 365
```

Set Benchmark

The benchmark performance is input to calculate several statistics on your algorithm, including alpha and beta . To set a benchmark for your algorithm, call the `set_benchmark` method. You can set the benchmark to a security, a constant value, or a value from a custom data source . If you don't set a brokerage model , the default benchmark is SPY. If you set a brokerage model, the model defines the default benchmark.

```
# Set the benchmark to IBM
self.set_benchmark("IBM")

# Set the benchmark to a constant value of 0
self.set_benchmark(lambda x: 0)

# Set the benchmark to a value from a custom data source
symbol = self.add_data(CustomData, "CustomData", Resolution.HOUR).symbol
self.set_benchmark(symbol)
```

If you pass a ticker to the `set_benchmark` method, LEAN checks if you have a subscription for it. If you have a subscription for it, LEAN uses the security subscription. If you don't have a subscription for it, LEAN creates a US Equity subscription with the ticker. Since the ticker you pass may not reference a US Equity, we recommend you subscribe to the benchmark security before you call the `set_benchmark` method.

Set Time Zone

LEAN supports international trading across multiple time zones and markets, so it needs a reference time zone for the algorithm to set the `time` . The default time zone is Eastern Time (ET), which is UTC-4 in summer and UTC-5 in winter. To set a different time zone, call the `set_time_zone` method. This method accepts either a string following the IANA Time Zone database convention or a NodaTime .DateTimeZone object. If you pass a string, the method converts it to a `NodaTime.DateTimeZone` object. The `TimeZones` class provides the following helper attributes to create `NodaTime.DateTimeZone` objects:

```
self.set_time_zone("Europe/London")
self.set_time_zone(TimeZones.CHICAGO)
```

The algorithm time zone may be different from the exchange time zone . If the time zones are different, it might appear like there is a lag between the algorithm time and the first bar of a history request, but this is just the difference in time zone. All the data is internally synchronized in Coordinated Universal Time (UTC) and arrives in the same Slice object. A slice is a sliver of time with all the data available for this moment.

To keep trades easy to compare between asset classes, we mark all orders in UTC time.

Set Warm Up Period

You may need some historical data at the start of your algorithm to prime technical indicators or populate historical data arrays. The warm-up period pumps data into your algorithm from before the start date. To set a warm-up period, call the `set_warm_up` method. The warm-up feature uses the subscriptions you add in `initialize` to gather the historical data that warms up the algorithm. If you don't create security subscriptions in the `initialize` method, the warm-up won't occur.

```
# Wind time back 7 days from the start date
self.set_warm_up(timedelta(7))

# Feed in 100 trading days worth of data before the start date
self.set_warm_up(100, Resolution.DAILY)

# If you don't provide a resolution argument, it uses the lowest resolution in your subscriptions
self.set_warm_up(100)
```

Set Name and Tags

You can categorize your backtest with a name and tags. To set the algorithm `name` , call the `set_name` method.

```
# Set the name of your backtest to help categorize your results.
self.set_name("Backtest Name")
```

The `set_name` method overwrites the names of backtests of an optimization , which contains the parameter values. To keep these values, create a name that includes them.

```
# Setting the backtest name to a mix of the parameters can help identify backtests in optimization results.
fast_period = self.get_parameter("fast_period", 100);
mid_period = self.get_parameter("mid_period", 200);
slow_period = self.get_parameter("slow_period", 300);
self.set_name(f"Backtest Name ({fast_period},{mid_period},{slow_period})");
```

To add tags to you algorithm, call the `add_tag` method.

```
# Add additional information to the algorithm with tags for better organization in the backtest results.
self.add_tag("Long Only")
self.add_tag("Momentum")
```

Your algorithm can add 20 tags. The name and each tag can have up to 200 characters or else they are truncated.

Post Initialization

After the `initialize` method, the `post_initialize` method performs post-initialization routines, so don't override it. To be notified when the algorithm is ready to begin trading, define an `on_warmup_finished` method. This method executes even if you don't set a warm-up period.

```
# Notify yourself when the algorithm is ready to begin trading.
def on_warmup_finished(self) -> None:
    self.log("Algorithm Ready")
```

Examples

The following examples demonstrate some common practices for initializing algorithms.

Example 1: Set the Data Seed

Some data feeds might not have data on its first fetch, making the algorithm prone to error. The following algorithm seeds the initial price of each asset to its last known price to avoid issues.

```
# Seed with the last known price of assets to prevent $0 security price errors.
def initialize(self):
    self.settings.seed_initial_prices = True
```

Example 2: Enable Extended Market Hours

Some traders use pre-market data for filtering assets. The following algorithm enables extended market hour data for assets in the universe.

```
# Add a universe of assets with extended market hours data.
def initialize(self):
    self.universe_settings.extended_market_hours = True
```

Example 3: Train Models When the Warm Up Finishes

The following algorithm uses the `on_warmup_finished` event handler to train a model with data from the warm-up period.

```
# Train machine learning models after the algorithm warm up is done, so your models
# are ready as soon as the algorithm starts trading.
def on_warmup_finished(self) -> None:
    self.train(self.my_training_method)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
BasicTemplateAlgorithm.py Python BasicTemplateCryptoAlgorithm.py Python BasicTemplateForexAlgorithm.py Python
BasicTemplateFuturesAlgorithm.py Python BasicTemplateOptionsAlgorithm.py Python

Securities

Securities

Key Concepts

Introduction

A security is an individual financial asset that you can trade on an exchange. LEAN models these unique assets with a `Security` object, which the `add_equity` , `add_crypto` , and similar methods return. The `securities` property of the `QCAlgorithm` class is a dictionary where the keys are `Symbol` objects and the values are `Security` objects. The `securities` dictionary contains all of the securities that have been in the algorithm during its lifetime.

```
PY
# Add the SPY.
self._symbol = self.add_equity("SPY").symbol
```

Price

To get the current price of a security, index the `securities` dictionary and then access the `price` member.

```
PY
# Get the price of the security.
price = self.securities[self._symbol].price
```

Holdings

The `Holdings` property of a `Security` object contains information about the investment state and history of the security.

```
PY
# Use the holdings object to get information on the asset position, such as the quantity and average price.
holdings = self.securities[self._symbol].holdings
quantity = holdings.quantity
```

Alternatively, you can get the current holdings of a security with the `portfolio` dictionary.

```
PY
# Get the quantity of a security you hold through the portfolio dictionary.
holdings = self.portfolio[self._symbol]
quantity = holdings.quantity
```

Reality Models

`Security` objects contain references to the security level reality models . To customize the behavior of each security, configure its reality models.

Tradable Status

The `is_tradable` property shows whether you can trade a security. The property value is true when the security is in the universe, even if the data starts at a later day. Indices, canonical Option securities, and continuous Futures contracts are not tradable. In live mode, custom data objects are also not tradable, even if the custom data represents a tradable asset.

```
PY
# Check if the security is tradable.
tradable = self.securities[self._symbol].is_tradable
```

Symbol Properties

The `symbol_properties` property of a `Security` contains properties for that specific security, such as the lot size and the minimum price variation.

```
# Round orders to a brokerage's lot size and minimum price variation using the symbol_properties.
symbol_properties = self.securities[self._symbol].symbol_properties
lot_size = symbol_properties.lot_size
minimum_price_variation = symbol_properties.minimum_price_variation
```

Market Hours

The `exchange` property of a `Security` contains information about the exchange that lists the security. For example, the `exchange.hours` property contains information on the trading days and hours .

```
# Using the exchange object, check if the exchange's regular trading hours are open at the given time.
exchange = self.securities[self._symbol].exchange
hours = exchange.hours
is_open_now = hours.is_open(self.time, extended_market_hours=False)
```

Custom Security Properties

You can add attributes to the `Security` object. For example, you can add an exponential moving average .

```
# Add an EMA indicator to the SPY Equity object.
equity = self.add_equity("SPY")
equity.ema = self.ema(equity.symbol, 10, Resolution.DAILY)
```

This feature is helpful because you can get the `Security` object from the `securities` object.

```
# Get the SPY Equity object from the securities object to get the EMA indicator.
ema = self.securities["SPY"].ema.current.value
```

Properties

The `Security` type has the following properties:

Examples

The following examples demonstrate some common practices for securities.

Example 1: Accessing Properties

The following example shows how to access various propeties of the `Security` object.

```
class SecuritiesUsageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        self.spy = self.add_equity("SPY")

    def on_data(self, slice: Slice) -> None:
        # OHLCV data.
        price = self.spy.close
        volume = self.spy.volume
        # Quote data.
        quote_size = self.spy.bid_size
        quote_price = self.spy.ask_price
        # Symbol Properties.
        lot_size = self.spy.symbol_properties.lot_size
        # Fundamentals cached data.
        pe_ratio = self.spy.fundamentals.valuation_ratios.pe_ratio
        # Properties, e.g. is tradeble, is market open, leverage, ...
        market_open = Extensions.is_market_open(self.spy, extended_market_hours=False)
        leverage = self.spy.leverage
        # Models, e.g. margin model, buying power model, ...
        margin_model = self.spy.margin_model
        # Portfolio Holdings reference.
        holding = self.spy.holdings
```

Securities

Requesting Data

Introduction

When you add a security to your algorithm, your algorithm creates a data subscription and receives data updates for that security or custom data source .

Add Assets

To create an Equity subscription, call the `add_equity` method and pass the `ticker` argument. The `ticker` argument represents the current ticker.

```py
# Add the SPY ETF.
equity = self.add_equity("SPY")
```

The `add_equity` method creates a subscription for a single Equity asset and adds it to your user-defined universe.

For more information about adding assets, see Asset Classes .

Add Universes

Universe selection is the process of selecting a basket of assets you may trade. When you add a universe to your algorithm, LEAN sends a large dataset into a filter function you define. LEAN automatically subscribes to these new assets and adds them to your algorithm.

```py
# Use the _fundamental_filter_function to add the 100 most liquid stocks to the universe.
class MyFundamentalUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._fundamental_filter_function)

    # Get the top 100 stocks by dollar volume.
    def _fundamental_filter_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)
        return [c.symbol for c in sorted_by_dollar_volume[:100]]
```

For more information about universes, see Universes .

Resolutions

Resolution is the duration of time that's used to sample a data source. Market data is sampled and saved in disk with five resolutions. The `Resolution` enumeration has the following members:

- `Resolution.TICK`
- `Resolution.SECOND`
- `Resolution.MINUTE`
- `Resolution.HOUR`
- `Resolution.DAILY`

For more information about the resolutions of each asset class, follow these steps:

1. Open the Asset Classes documentation.
2. Click an asset class.
3. Click Requesting Data .
4. Scroll down to the Resolutions section.

You can subscribe to daily resolution for an asset for updating indicator and subscribe to another asset for denser resolution for trading using the indicator.

```
def initialize(self):
    self.add_equity("VIX", Resolution.DAILY)
    self.add_equity("SPY", Resolution.MINUTE)
```

To get data with multiple or customizable resolutions for an asset, use consolidators .

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

For more information about how to enable and disable fill forward for each asset class, follow these steps:

1. Open the Asset Classes documentation.
2. Click an asset class.
3. Click Requesting Data .
4. Scroll down to the Fill Forward section.

Example: Disable Fill Forward Data

For some illiquid securities or alternative data, using forward filled data might not be an accurate estimation nor usable information. To avoid this, disable fill forward.

```
def initialize(self):
    self.add_data(BrainSentiment7Days, symbol, fill_forward=False)
```

Security Changed Events

When you add and remove assets, LEAN notifies your algorithm through the `on_securities_changed` event handler. The event handler receives a `SecurityChanges` object, which contains references to the added and removed securities. To access the added securities, iterate the `changes.added_securities` property. To get the removed securities, iterate the `changes.removed_securities` property.

```
# Use on_securities_changed to know when an asset is added or removed.
def on_securities_changed(self, changes: SecurityChanges) -> None:
    # Iterate through the added assets.
    for security in changes.added_securities:
        self.debug(f"{self.time}: Added {security.symbol}")

    # Iterate through the removed assets.
    for security in changes.removed_securities:
        self.debug(f"{self.time}: Removed {security.symbol}")

        if security.invested:
            self.liquidate(security.symbol, "Removed from Universe")
```

The preceding example liquidates securities that leave the universe. You can use this event to create indicators when a new security enters the universe.

Remove Subscriptions

To remove a security subscription, call the `remove_security` method.

```
# Remove the security subscription.
self.remove_security(self._symbol)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings.

Examples

The following examples demonstrate some common practices on the subject of requesting data.

Example 1: Add Linked Alternative Data for Universe Constituents

Many traders select universe constituents and then allocate positions with two independent sets of logic. Position sizing usually requires additional information, like alternative data factors. To manage alternative data subscriptions for all the universe constituents, subscribe and unsubscribe from the data in the `on_securities_changed` method. The following example demonstrates adds the KavoutCompositeFactorBundle when assets enter the universe.

```py
class RequestSecuritiesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe of the 25 most liquid US Equities.
        self.add_universe(self.universe.top(25))

    def on_data(self, slice: Slice) -> None:
        # Invest by the updated Kavout Factor data.
        factor_data = slice.get(KavoutCompositeFactorBundle)
        if factor_data:
            sorted_by_factor_score = sorted(
                factor_data.items(),
                key=lambda x: x[1].growth + x[1].value_factor + x[1].quality + x[1].momentum + x[1].low_volatility,
                reverse=True
            )
            sorted_by_factor_score = [x for x in sorted_by_factor_score if self.securities[x[0].underlying].price][:10]
            top_factor_targets = [PortfolioTarget(x[0].underlying, 0.1) for x in sorted_by_factor_score]
            self.set_holdings(top_factor_targets, True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Iterate securities that entered the universe.
        for security in changes.added_securities:
            # Subscribe to the factor data for the new security.
            # Save the alternative data symbol in the security object.
            security.alt_data_symbol = self.add_data(KavoutCompositeFactorBundle, security.symbol, Resolution.DAILY).symbol

        # Iterate securities that left the universe.
        for security in changes.removed_securities:
            # Unsubscribe from the altnerative data updates.
            self.remove_security(security.alt_data_symbol)
```

Example 2: Add Linked Custom Data

QuantConnect Cloud has a lot of data, but not everything. To import some external data into your algorithm, define a custom data source . The following algorithm loads Bitstamp's BTCUSD trading activity from a CSV file and plots it:

PY

```
# Define the algorithm.
class LinkedCustomDataExampleAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add Bitcoin and the custom dataset.
        btc = self.add_crypto('BTCUSD')
        btc.data_symbol = self.add_data(Bitstamp, btc.symbol.value, Resolution.DAILY).symbol
        # Plot the price from the custom dataset.
        self.plot_indicator('BTC Price', self.identity(btc.data_symbol))

# Define the custom data source.
class Bitstamp(PythonData):
    # Define the get_source method to load the data from its location.
    def get_source(self, config, date, is_live_mode):
        source = "https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-data/bitstampusd.csv"
        # REMOTE_FILE in the next line means the data file in online storage.
        return SubscriptionDataSource(source, SubscriptionTransportMedium.REMOTE_FILE)

    # Define the reader method to parse the CSV rows.
    def reader(self, config, line, date, is_live_mode):
        # Ignore empty rows.
        if not line.strip():
            return None

        # Instantiate a new data object.
        coin = Bitstamp()
        coin.symbol = config.symbol

        # Ignore non-numeric lines.
        if not line[0].isdigit():
            return None

        # Split the CSV row by its commas.
        data = line.split(',')

        # Ignore invalid data.
        coin.value = float(data[4])
        if coin.value == 0:
            return None

        # Parse the row.
        coin.time = datetime.strptime(data[0], "%Y-%m-%d")
        coin.end_time = coin.time + timedelta(1)
        coin["Open"] = float(data[1])
        coin["High"] = float(data[2])
        coin["Low"] = float(data[3])
        coin["Close"] = coin.value
        coin["VolumeBTC"] = float(data[5])
        coin["VolumeUSD"] = float(data[6])
        coin["WeightedPrice"] = float(data[7])
        return coin
```

Securities

Handling Data

Introduction

If you create a data subscription, your algorithm receives data updates for that security or custom data source .

Get Prices

LEAN uses data updates to set the current prices of a security and compute the contribution of securities you hold to the portfolio value. To get the current prices of a security, find the `Security` object in the `securities` dictionary and access the price attributes.

```
PY
# Use the securities object as a dictionary to get the current ask and bid price of the SPY.
security = self.securities["SPY"]
price = security.price
bid_price = security.bid_price
ask_price = security.ask_price
```

Data Events

The `Slice` that LEAN passes to the `on_data` method represents all of the data for your subscriptions at a single point in time. The `Slice` object contains data like `Tick` objects, `TradeBar` objects, `QuoteBar` objects, corporate actions , and chains for Option and Future contracts. You can use the data in the `Slice` to make trading decisions.

To access data in the `Slice` , index it with the security `Symbol` . If you use the security ticker instead of the `Symbol` , LEAN automatically finds the `Symbol` .

```
PY
# Use the OnData method to get the data for a symbol in the current time slice.
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self._symbol):
        my_data = slice[self._symbol]
```

The following table shows the `Slice` property to index based on the data format you want:

Data Format	Slice Property
`TradeBar`	`bars`
`QuoteBar`	`quote_bars`
`Tick`	`ticks`

```
PY
# Get the tradebar from the slice.
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self._symbol):
        bar = slice.bars[self._symbol]
```

If you just index the `Slice` instead of the preceding properties, it returns the correct object. For instance, if your data subscription provides `QuoteBar` objects and you index the `Slice` with the security `Symbol` , it returns the `QuoteBar` .

For more information about the `Slice` class, see Timeslices .

Cache

To get the most recent data for a security, call the `get_last_data` method on the `Security` object.

```
# Get the most recent data for a security.
data = self.securities["SPY"].get_last_data()
```

The `cache` property of a `Security` object provides additional price information since you can access `TradeBar` , `QuoteBar` , and `Tick` data.

```
# Get additional price information for the security.
cache = self.securities["SPY"].cache
trade_bar = cache.get_data[TradeBar]()
quote_bar = cache.get_data[QuoteBar]()
ticks = cache.get_all[Tick]()
```

Market Session

The `Session` property of a `Security` object contains the open, high, low, close, volume, and open interest of trading sessions. The `Session` object is a `RollingWindow` object with size of zero by default. To enable market session tracking, set the size of the `Session` object to 2, so the previous session data is at index 1.

```
session = self.securities[self._symbol].session
session.size = 2
today_open = session.open
today_high = session.high
today_low = session.low
today_cumulative_volume = session.volume
if session.is_ready:
    yesterday_close = session[1].close
```

Increase the size of the `Session` object to store more session data. You can use cached data in the `Session` property instead of requesting historical daily data .

```
equity = self.add_equity("SPY")
# Increase the session lookback to 252 from default (0)
equity.session.size = 252
# Get all closing prices in the session history
closes = [x.close for x in session][::-1]
```

For more information about rolling windows, see Rolling Window .

Examples

The following examples demonstrate some common practices for handling data.

Example 1: Get QuoteBar Data

The following algorithm updates an indicator with the mid-price of a quote bar to increase the accuracy of the indicator values for illiquid assets. Then, make use of the SMA indicator to trade the SMA cross strategy.

```py
class HandlingSecuritiesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        self._symbol = self.add_equity("SPY").symbol
        self._indicator = SimpleMovingAverage(250)

        self.set_warm_up(timedelta(350))

    def on_data(self, slice: Slice) -> None:
        # Check if the QuoteBars contain SPY quote data.
        if self._symbol not in slice.quote_bars:
            return
        quote_bar = slice.quote_bars[self._symbol]
        # Calculate the mid price by averaging the bid and ask price.
        mid_price = (quote_bar.bid.close + quote_bar.ask.close) * 0.5
        # Update the SMA indicator with the mid price.
        self._indicator.update(quote_bar.end_time, mid_price)

        # Wait for the algorithm and indicator to be warmed up.
        if self.is_warming_up or not self._indicator.is_ready:
            return
        # Trade SMA cross strategy.
        weight = -0.5 if self._indicator.current.value > mid_price else 0.5
        self.set_holdings(self._symbol, weight)
```

Example 2: Get Tick Data

The following algorithm calculates the bid and ask sizes of the latest SPY quote ticks and then trades at the top of each hour. During each rebalance, it buys SPY if bid size is greater than ask size since supply is greater than demand in this case. If bid size is smaller than ask size, it sells SPY since demand is greater than supply in this case.

```py
class BidAskSizeTickAlgorithm(QCAlgorithm):
    # Set up variables to save the bid and ask sizes.
    _bid_size = 0
    _ask_size = 0
    _hour = -1

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 10)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        self._symbol = self.add_equity('SPY', Resolution.TICK).symbol

    def on_data(self, slice: Slice) -> None:
        # Check if the Slice contain SPY tick data.
        ticks = slice.ticks.get(self._symbol)
        if ticks:
            # Iterate all the ticks to calculate the bid and ask sizes.
            for tick in ticks:
                # Make sure the tick data is a quote instead of a trade.
                if tick.tick_type == TickType.QUOTE:
                    # Update the bid and ask sizes.
                    self._bid_size += tick.bid_size
                    self._ask_size += tick.ask_size
        # Trade at the top of each hour.
        if slice.time.hour != self._hour:
            # Invest based on supply-demand relationship from all ticks.
            # If bid size is above ask size, supply is greater than
            # demand, which drives the price up. Otherwise, supply is
            # lower than demand, which drives the price down.
            self.set_holdings(
                self._symbol, 0.5 if self._bid_size > self._ask_size else -0.5
            )
            # Reset the bid and ask size variables.
            self._bid_size = 0
            self._ask_size = 0
            self._hour = slice.time.hour
```

Example 3: Get US Equity Fundamental Data

The following algorithm gets the latest fundamental data of all the US Equities in the algorithm. We invest in the stocks with the highest PE Ratio equally to ride on the popularity.

```
class HandlingSecuritiesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request data for a selected list of equities for trading.
        for ticker in ["AAPL", "MSFT", "TSLA", "GOOG", "AMZN"]:
            self.add_equity(ticker)

    def on_data(self, slice: Slice) -> None:
        pe_ratios = {}

        # Iterate active security objects.
        for kvp in self.active_securities:
            symbol = kvp.key
            security = kvp.value

            # Get the Fundamental cache.
            fundamental = security.fundamentals
            # Get the sector code.
            sector_code = fundamental.asset_classification.morningstar_sector_code

            pe_ratios[symbol] = fundamental.valuation_ratios.pe_ratio

        # Sort by PE ratio to get the most popular stocks.
        sorted_by_pe_ratio = sorted(pe_ratios.items(), key=lambda x: x[1])
        targets = [PortfolioTarget(x[0], 1/3) for x in sorted_by_pe_ratio[-3:]]
        targets.extend([PortfolioTarget(x[0], 0) for x in sorted_by_pe_ratio[:2]])
        # Invest equally in the highest PE Ratio stocks for evenly dissipate capital risk.
        self.set_holdings(targets)
```

Example 4: Get Option Greeks

The following algorithm gets SPY Option Greek values. Sell a straddle with the ATM options, selected by Option Delta (closest to 0.5).

```
class HandlingSecuritiesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add an SPY Option universe.
        option = self.add_option("SPY")
        # Set the Option universe filter.
        option.set_filter(lambda x: x.strikes(-5, 5).expiration(0, 60))
        # Save a reference of the canonical symbol.
        self._symbol = option.symbol

    def on_data(self, slice: Slice) -> None:
        # Try to get the Option contracts within the Option chain.
        chain = slice.option_chains.get(self._symbol)
        if not self.portfolio.invested and chain:
            for contract in chain:
                # Get the implied volatility and greeks of each Option contract.
                iv = contract.implied_volatility
                greeks = contract.greeks
                delta = greeks.delta
                gamma = greeks.gamma
                vega = greeks.vega
                theta = greeks.theta
                rho = greeks.rho

            # Invest in a straddle strategy to capitalize the volatility
            # Trade the ATM options with the closest expiry after 7 days.
            expiry = min(x.expiry for x in chain)
            selected = sorted([x for x in chain if x.expiry == expiry],
                key=lambda x: abs(x.greeks.delta - 0.5))[0]
            option_strategy = OptionStrategies.straddle(selected.symbol.canonical, selected.strike, selected.expiry)
            self.sell(option_strategy, 1)
```

Example 5: Get Asset Sentiment Values

The following example gets sentiment values from the Brain Sentiment Indicator dataset. It simply buys AAPL when the sentiment is positive and liquidates the position when the sentiment is negative.

```py
class HandlingSecuritiesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of AAPL with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add the 7-day sentiment data for AAPL.
        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self._dataset_7day_symbol = self.add_data(BrainSentimentIndicator7Day, self._symbol).symbol

    def on_data(self, slice: Slice) -> None:
        # Check if the current slice contains the 7-day sentiment data.
        if slice.contains_key(self._dataset_7day_symbol):
            data_point = slice[self._dataset_7day_symbol]
            # Log the sentiment value.
            self.log(f"{self._dataset_7day_symbol} sentiment at {slice.time}: {data_point.sentiment}")

            # Invest if the sentiment score is above 0, which indicates positive sentiment.
            if data_point.sentiment > 0:
                self.market_order(self._symbol, 1)
            # Liquidate otherwise.
            elif data_point.sentiment < 0 and self.portfolio[self._symbol].invested:
                self.liquidate(self._symbol)
```

Example 6: Market Session as History

Using `Session` instead of history requests or rolling window to cache the last 252 OHLCV bars, the following algorithm buys SPY accordingly when the market opens above the 252-day max of closing prices.

```py
class MarketSessionAsDailyHistory(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        equity = self.add_equity("SPY")

        # Increase the session lookback to 252 from default (0)
        # since our strategy break out the 252-day max of closing prices
        equity.session.size = 252
        # Warm up with daily data for speed
        self.set_warm_up(equity.session.size, Resolution.DAILY)

        self.schedule.on(self.date_rules.every_day("SPY"), self.time_rules.after_market_open("SPY", 1), self.trade_break_out)

    def trade_break_out(self):
        '''Break-out strategy. If the market opens above the 252-day max of closing prices,
        allocate all in SPY. Liquidate if it opens below the mean.
        '''
        session = self.securities["SPY"].session
        # The session is ready when the number of samples is greater than the size (252)
        if self.is_warming_up or not session.is_ready:
            return

        # Create a pandas Series of the closing prices of session
        # to use the max and mean methods
        closes = pd.Series([x.close for x in session][::-1])
        _open, _max, _mean = session.open, closes.max(), closes.mean()

        # Invest all in SPY if the open break out the 252-day max and liquidate below the mean
        if _open > _max:
            self.set_holdings("SPY", 1, tag=f'{_open=:.2f}, {_max=:.2f}')
        if _open < _mean:
            self.liquidate("SPY", tag=f'{_open=:.2f}, {_mean=:.2f}')
```

Securities

Market Hours

Introduction

The `exchange` property of a `Security` object contains information about the exchange that lists the security, including its time zone and the trading days and hours. To get the exchange market hours, use the `exchange.hours` property on a `Security` object.

```
# Get properties of the exchange that lists the security.
security_exchange_hours = self.securities["SPY"].exchange.hours
```

Market Status

To check if the exchange is open at a specific time, call the `is_open` method.

```
# Check if the exchange is currently open.
is_open_now = security_exchange_hours.is_open(self.time, extended_market_hours=False)

# Check if the exchange is open at any point in time over a specific interval.
is_open = security_exchange_hours.is_open(self.time, self.time + timedelta(days=1), extended_market_hours=False)
```

To check if the exchange is open on a specific date, call the `is_date_open` method.

```
# Check if the exchange is open at some time today.
is_open = security_exchange_hours.is_date_open(self.time, extended_market_hours=True)
```

To get the next market open time, call the `get_next_market_open` method.

```
# Get the next market open time.
market_open_time = security_exchange_hours.get_next_market_open(self.time, extended_market_hours=False)
```

To get the next market close time, call the `get_next_market_close` method.

```
# Get the next market close time.
market_close_time = security_exchange_hours.get_next_market_close(self.time, extended_market_hours=False)
```

To get the previous trading day, call the `get_previous_trading_day` method.

```
# Get the previous trading day.
day = security_exchange_hours.get_previous_trading_day(self.time)
```

To get the next trading day, call the `get_next_trading_day` method.

```
# Get the next trading day.
day = security_exchange_hours.get_next_trading_day(self.time)
```

Extended Market Hours

By default, your security subscriptions only cover regular trading hours. To subscribe to pre and post-market trading hours for a specific asset, enable the `extended_market_hours` argument when you create the security subscription.

```
self.add_equity("SPY", extended_market_hours=True)
```

You only receive extended market hours data if you create the subscription with minute, second, or tick resolution. If you create the subscription with daily or hourly resolution, the bars only reflect the regular trading hours.

Properties

The `SecurityExchange` object has the following attributes:

Examples

The follow examples demonstrate some common practices for trading with market hours data.

Example 1: Only Within 1 Hour Of Market Open And Close

Buying SPY shortly after market holidays or weekend breaks can be a profitable strategy. Historically, the market has tended to experience positive returns in the trading sessions immediately following extended market closures. This is often attributed to the release of pent-up demand and information during the downtime. By positioning into SPY at the open or within the first few trading hours after a holiday or weekend, investors may be able to capture this post-break bounce in the broader market.

```
class MarketHourExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.spy = self.add_equity("SPY", Resolution.HOUR)

    def on_data(self, slice: Slice) -> None:
        # Check if the previous day is holiday
        previous_day = self.time.replace(hour=0, minute=0) - timedelta(1)
        if previous_day in self.spy.exchange.hours.holidays:
            # Buy if it is a holiday a day before
            self.set_holdings(self.spy.symbol, 1)
        elif self.portfolio[self.spy.symbol].invested:
            # Liquidate otherwise if any open position
            self.liquidate(self.spy.symbol)
```

Example 2: Pre-market Short Limit Order

A pre-market short limit order is a type of order placed before the regular trading session begins. It allows a trader to take a short position at a price higher than the current market price, with the intention of selling the position shortly after the market opens and profiting from the expected price decline. This strategy is often used by traders who anticipate a decline in the asset's value and want to capitalize on it.

```
class MarketHourExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add SPY data with extended market hours.
        self._spy = self.add_equity("SPY", extended_market_hours=True)

    def on_data(self, slice: Slice) -> None:
        if self._spy.symbol not in slice.quote_bars:
            return
        # During pre-market hours, place an unmarketable limit order.
        if (not self._spy.invested and
            self._is_before_market_open_extended_hours(slice.time) and
            not list(self.transactions.get_open_order_tickets(self._spy.symbol))):
            self.limit_order(self._spy.symbol, -10, self._spy.bid_price * 1.005)
        # If we hold a position during regular trading hours, liquidate
        # it.
        elif (self._spy.invested and
            self._spy.exchange.hours.is_open(slice.time, False)):
            self.liquidate(self._spy.symbol)

    def _is_before_market_open_extended_hours(self, time: datetime) -> bool:
        return (
            # It's currently extended market hours.
            self._spy.exchange.hours.is_open(time, True) and
            # The next market open for regular trading hours is today.
            self._spy.exchange.hours.get_next_market_open(
                time, False
            ).day == time.day
        )
```

However, it is important to note that pre-market trading can be more volatile and riskier than regular trading, and the intended price may not be achieved on this strategy.

Example 3: Last Hour Trades

Establishing a scheduled trading event in the hour before market close can be an effective strategy. This allows the trader to analyze market conditions, identify potential trade opportunities, and prepare orders ahead of the close. By emitting their trading insights and planned actions to the market in this critical final hour, the trader can influence other market participants and potentially gain an edge.

```PY
class MarketHourExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.spy = self.add_equity("SPY")

        # Set up a portfolio construction model to handle insights for trading
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_DAY))

        # Set up a scheduled event on 1-hour before market close to open position
        self.schedule.on(
            self.date_rules.every_day(self.spy.symbol),
            self.time_rules.before_market_close(self.spy.symbol, 60),
            self.last_hour_trade
        )

    def last_hour_trade(self) -> None:
        # Get the time of next market close
        market_close_time = self.spy.exchange.hours.get_next_market_close(self.time, extended_market_hours=False)
        # Emit an insight that expires 1 minutye before the next market close
        self.emit_insights(Insight.price(self.spy.symbol, market_close_time - timedelta(minutes=1), InsightDirection.UP))
```

Securities

Filtering Data

Introduction

Unfiltered raw data can be faulty for a number of reasons, including invalid data entry. Moreover, high-frequency traders can deploy bait-and-switch strategies by submitting bait orders to deceive other market participants, making raw data noisy and untradeable. To avoid messing up with our trading logic and model training, you can filter out suspicious raw data with a data filter.

Set Models

To set a data filter for a security, call the `set_data_filter` property on the `Security` object.

```
PY
# Use the set_data_filter method to use the SecurityDataFilter on the SPY ETF data.
spy = self.add_equity("SPY")
spy.set_data_filter(SecurityDataFilter())
```

You can also set the data filter model in a security initializer . If your algorithm has a universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```
PY
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_data_filter(SecurityDataFilter())
```

Default Behavior

The following table shows the default data filter for each security type:

Security Type	Default Filter
`Equity`	EquityDataFilter
`Option`	OptionDataFilter
`Forex`	ForexDataFilter
`Index`	IndexDataFilter
`Cfd`	CfdDataFilter
Others	SecurityDataFilter

None of the preceding filters filter out any data.

Model Structure

Data filtering models must implement a `filter` method, which receives `Security` and `BaseData` objects and then returns a `boolean` object that represents if the data point should be filtered out.

```
PY
# Include or exclude a data point from the algorithm.
class MyDataFilter(SecurityDataFilter):
    def filter(self, vehicle: Security, data: BaseData) -> bool:
        return True
```

Examples

The following examples demonstrate some common practices for filtering data.

Example 1: Filter Out Outliers

When analyzing high-frequency price data, it's important to filter out potential outliers and anomalies that may skew the analysis. One effective method is to use simple moving average (SMA) and standard deviation indicators to identify ticks that significantly deviate from the short-term trend. By comparing each tick to the indicator values, you can flag any data points that fall outside a threshold (for example, three standard deviations). This filtration process removes suspicious or erroneous price information from entering your algorithm, ensuring a cleaner dataset for trading.

```
PY
class CustomDataFilterAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 3)
        equity = self.add_equity("AAPL", Resolution.TICK)
        # Create the indicators.
        equity.sma = self.sma(equity.symbol, 100)
        equity.std = IndicatorExtensions.of(StandardDeviation(100), equity.sma, True)
        # Set the data filter.
        equity.set_data_filter(CustomDataFilter())


class CustomDataFilter(SecurityDataFilter):

    def filter(self, vehicle: Security, data: BaseData) -> bool:
        # Wait until the indicators are ready.
        security = vehicle
        if not (security.sma.is_ready and security.std.is_ready):
            return True # Keep the data point.
        # Check if the current value is within 3 standard deviations of the mean.
        # Return True (keep) or False (discard).
        sma = security.sma.current.value
        std = security.std.current.value
        return sma - 3*std <= data.value <= sma + 3*std
```

Example 2: Filter Out Major Exchanges

When you trade illiquid financial instruments, it can be advantageous to focus on the BATS exchange since its quote data may not fully reflect the fair market value. Due to the lower trading volume and visibility of BATS, the quotes there may lag behind the true value of illiquid assets. A carefully designed algorithm can analyze the BATS feed to identify situations where the quotes appears to be undervalued compared to the asset's intrinsic worth. By executing trades to capture this disconnect, rather than arbitraging between exchanges, you may be able to profit from the market inefficiencies present in the less liquid instrument. The following example demonstrates how to only consume data from the BATS exchange:

```
PY
class BatsDataFilterAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 3)
        equity = self.add_equity("AAPL", Resolution.TICK)
        # Set the data filter.
        equity.set_data_filter(BatsDataFilter())


class BatsDataFilter(SecurityDataFilter):

    def filter(self, vehicle: Security, data: BaseData) -> bool:
        # Get the tick object.
        tick = Tick(data)
        # Return True (keep) or False (discard).
        return tick and tick.exchange == Exchange.BATS.name
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomSecurityDataFilterRegressionAlgorithm.py Python TickDataFilteringAlgorithm.py Python

Securities

Asset Classes

You can trade any of the following asset classes. Click one to learn more.

US Equity

Equity Options

Crypto

Crypto Futures

Forex

Futures

Future Options

Index

Index Options

CFD

India Equity

See Also

Universes
Importing Data

Asset Classes

US Equity

US Equities represent partial ownership in a US corporation.

Requesting Data

Handling Data

Corporate Actions

Corporate Fundamentals

Shorting

Data Preparation

Market Hours

See Also

BasicTemplateAlgorithm.py
BasicTemplateAlgorithm.cs

US Equity

Requesting Data

Introduction

Request US Equity data in your algorithm to receive a feed of asset prices in the `on_data` method.

Every stock traded in the US Securities Information Processors (SIP) CTA/UTP feed is available. Over-the-Counter (OTC) trades are not included. For more information about the specific dataset we use for backtests, see the US Equities dataset listing .

To trade US Equities live, you can use the QuantConnect data provider or one of the brokerage data providers .

Create Subscriptions

To create an Equity subscription, in the `initialize` method, call the `add_equity` method. The `add_equity` method returns an `Equity` object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
PY
self._symbol = self.add_equity("SPY").symbol
```

The `add_equity` method creates a subscription for a single Equity asset and adds it to your user-defined universe. To create a dynamic universe of Equities, add an Equity universe or an Equity Universe Selection model .

To view the supported assets in the US Equities dataset, see the Data Explorer . It doesn't include Over-the-Counter (OTC) stocks.

Resolutions

The following table shows the available resolutions and data formats for Equity subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`			✓	✓
`SECOND`	✓	✓		
`MINUTE`	✓	✓		
`HOUR`	✓			
`DAILY`	✓			

The default resolution for Equity subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_equity` method.

```
PY
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

To create custom resolution periods, see Consolidating Data .

Asset Primary Exchange

When a stock like Apple is listed, it's listed on Nasdaq. The open auction tick on Nasdaq is the price that's used as the official open of the day. NYSE, BATS, and other exchanges also have opening auctions, but the only official opening price for Apple is the opening auction on the exchange where it was listed.

Supported Markets

LEAN groups all of the US Equity exchanges under `Market.USA` . In live mode, the brokerage routes the orders to the exchange that provides the best price.

To set the market for a security, pass a `market` argument to the `add_equity` method.

```
self._symbol = self.add_equity("SPY", market=Market.USA).symbol
```

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self.add_equity("SPY", fill_forward=False)
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. In backtests, the default leverage for margin accounts is 2x leverage and leverage is not available for cash accounts. To change the amount of leverage you can use for a security, pass a `leverage` argument to the `add_equity` method.

```
self._symbol = self.add_equity("SPY", leverage=3).symbol
```

In live trading, the brokerage determines how much leverage you may use. For more information about the leverage they provide, see Brokerages .

Extended Market Hours

By default, your security subscriptions only cover regular trading hours. To subscribe to pre and post-market trading hours for a specific asset, enable the `extended_market_hours` argument when you create the security subscription.

```
self.add_equity("SPY", extended_market_hours=True)
```

You only receive extended market hours data if you create the subscription with minute, second, or tick resolution. If you create the subscription with daily or hourly resolution, the bars only reflect the regular trading hours.

To view the schedule of regular and extended market hours, see Market Hours .

Data Normalization

The data normalization mode defines how historical data is adjusted for corporate actions . The data normalization mode affects the data that LEAN passes to `on_data` and the data from history requests . By default, LEAN adjusts US Equity data for splits and dividends to produce a smooth price curve, but the following data normalization modes are available for Equity subscriptions:

If you use `ADJUSTED` , `SPLIT_ADJUSTED` , or `TOTAL_RETURN` , we use the entire split and dividend history to adjust historical prices. This process ensures you get the same adjusted prices, regardless of the backtest end date.

To set the data normalization mode for a security, pass a `data_normalization_mode` argument to the `add_equity` method.

```
self._symbol = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
```

The `SCALED_RAW` data normalization is only for history requests . When you use `SCALED_RAW` , we use the split and dividend history before the algorithm's current time to adjust historical prices. The `SCALED_RAW` data normalization model enables you to warm up indicators with adjusted data when you subscribe to `RAW` security data.

Properties

The `add_equity` method returns an `Equity` object, which have the following properties:

Examples

The following examples demonstrate some common practices for requesting US Equity data.

Example 1: Add Multiple Assets

The following algorithm adds multiple US Equities with extended market hours data. Extended market hours data enables you to monitor price movements during pre-market and after-hours trading sessions.

```py
class USEquityDemoAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Create a list of tickers you want to add.
        tickers = ["AAPL", "TSLA", "GOOG"]
        # Add extended market hours data for each US Equity.
        self._equities = [self.add_equity(ticker, extended_market_hours=True) for ticker in tickers]
```

Example 2: Import External Trade Signals

Say you source trading signals from outside your algorithm and organize them into a CSV file on Dropbox with the following structure:

```
#ticker,source1,source2,source3,source4,source5
AAPL,1.25,1.10,1.32,1.13,1.21
AMZN,1.15,1.05,1.07,1.11,1.03
GOOG,0.97,0.99,0.98,1.04,0.95
META,1.04,1.03,1.10,1.04,0.98
MSFT,0.92,0.96,0.98,1.01,0.98
```

The following algorithm uses a Scheduled Event to periodically pull the CSV file from Dropbox , parse its signals, and place trades. To determine the target weight for each asset, it takes the mean score from all the sources. If the mean score is greater than 1, it allocates 20% of the portfolio to the asset. Otherwise, it allocates -20% to the asset.

```py
from io import StringIO

class USEquityDemoAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        self.settings.seed_initial_prices = True

        # Add the US Equities.
        self.signal_by_ticker = {self.add_equity(ticker, Resolution.DAILY).symbol: 1
            for ticker in "AAPL,AMZN,GOOG,META,MSFT".split(',')}

        # Set up a Scheduled Event to download the data file and place orders.
        self.schedule.on(self.date_rules.month_start(), self.time_rules.midnight, self._download_data_and_place_orders)

    def _download_data_and_place_orders(self):
        # Download the file from DropBox.
        file_content = self.download("<DropBoxUrl>&dl=1")
        if file_content:
            # Convert the file content into trading signals.
            self.signal_by_ticker = pd.read_csv(StringIO(file_content), index_col=0).mean(axis=1)

        # Place orders according to the signals.
        targets = [
            PortfolioTarget(symbol, 0.2 if signal >= 1 else -0.2)
            for symbol, signal in self.signal_by_ticker.items()
        ]
        self.set_holdings(targets, True)
```

US Equity

Handling Data

Introduction

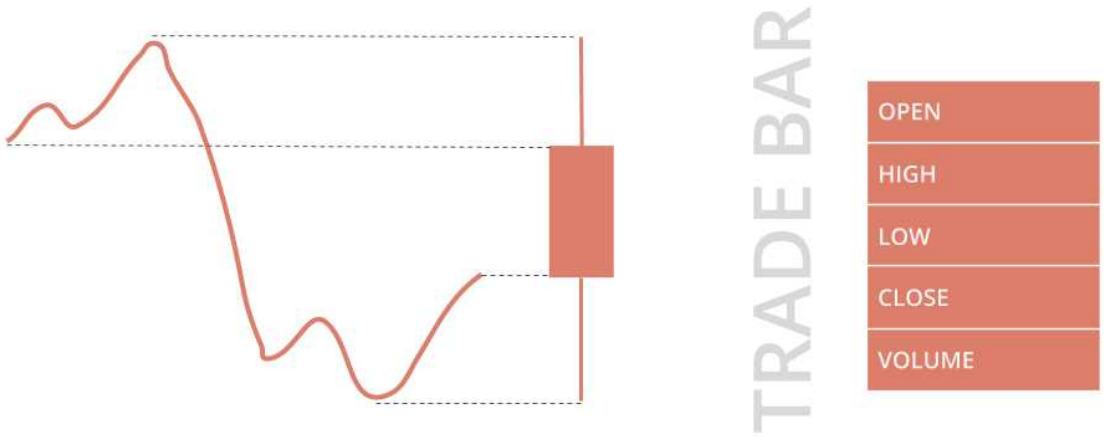
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the security ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for US Equity data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the security `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._symbol)   # None if not found
```

You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

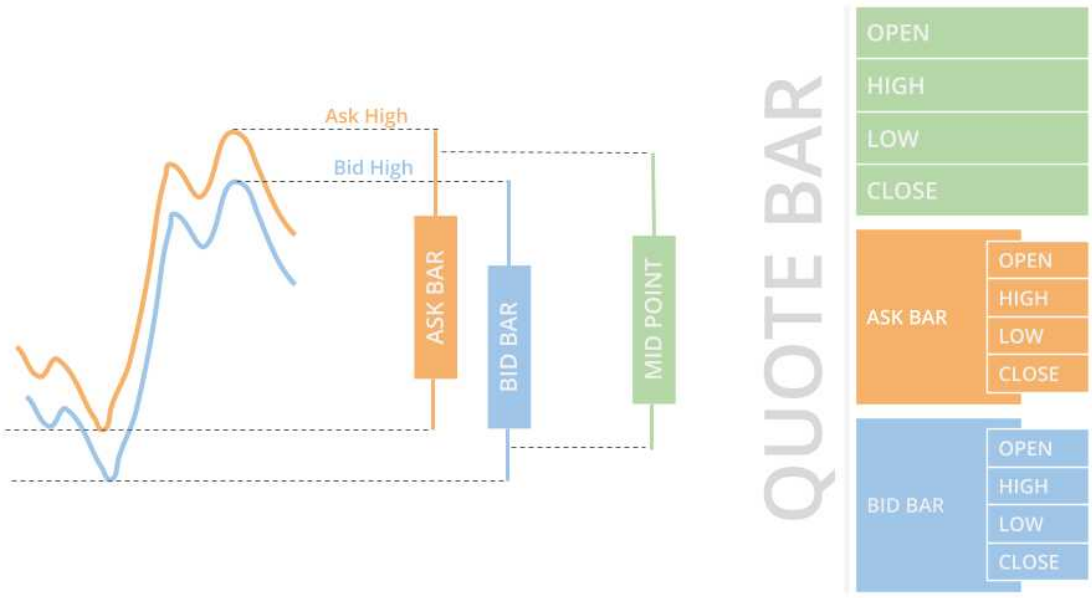
```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

We adjust the daily open and close price of bars to reflect the official auction prices .

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the security `symbol` . If the security doesn't actively get quotes or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._symbol)   # None if not found
```

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the security. A quote tick is an offer to buy or sell the security at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for

your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

If we detect a tick that may be suspicious , we set its `suspicious` flag to true.

Other Data Formats

For more information about data formats available for US Equities, see Corporate Actions .

Examples

The following examples demonstrate some common practices for handling US Equity data.

Example 1: Various Data Resolutions and Formats

The following algorithm handles three data resolutions (Daily, Minute, and Tick) and three data formats (TradeBar, QuoteBar, and Tick). The Equity subscriptions provide the ticks and daily TradeBar objects. To create the 5-minute QuoteBar objects, it consolidates the AAPL ticks.

The algorithm enters a long position in AAPL when the following conditions are met:

- The 16-day Fractal Adaptive Moving Average (FRAMA) of QQQ is increasing, which indicates the trend is up.
- The Relative Strength Index (RSI) of AAPL is above 70, which indicates AAPL has been rising from large buying pressure.

The algorithm enters a short position in AAPL when the following conditions are met:

- The 16-day FRAMA of QQQ is rising, which indicates the trend is down.
- The RSI of AAPL is above, which indicates AAPL has been declining from large selling pressure.

The algorithm liquidates the portfolio when the FRAMA changes direction.

PY

```
class USEquityExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 4)
        self.set_end_date(2024, 9, 10)
        # Add QQQ and AAPL data.
        self._qqq = self.add_equity("QQQ", Resolution.DAILY)
        self._aapl = self.add_equity("AAPL", Resolution.TICK)
        # Create the indicators.
        self._qqq.frama = self.frama(self._qqq.symbol, 16)
        self._aapl.rsi = RelativeStrengthIndex(10)
        # Create a 5-minute consolidator to aggregate AAPL tick data into quote bars.
        consolidator = TickQuoteBarConsolidator(timedelta(minutes=5))
        # Subscribe the consolidator for automatic updates.
        self.subscription_manager.add_consolidator(self._aapl.symbol, consolidator)
        # Attach an event handler handler to update the RSI indicator with the consolidated bars.
        consolidator.data_consolidated += lambda _, bar: self._aapl.rsi.update(bar.end_time, bar.close)
        # Warm up the indicators.
        self.warm_up_indicator(self._qqq.symbol, self._qqq.frama)
        for quote_bar in self.history[QuoteBar](self._aapl.symbol, 10, Resolution.MINUTE):
            self._aapl.rsi.update(quote_bar.end_time, quote_bar.close)

    def on_data(self, slice: Slice) -> None:
        # Ensure AAPL ticks are in the current slice.
        if self._aapl.symbol not in slice.ticks:
            return
        # Get the indicator values.
        frama = self._qqq.frama.current.value
        prev_frama = self._qqq.frama.previous.value
        rsi = self._aapl.rsi.current.value

        # Check if we have an AAPL position.
        if self._aapl.holdings.invested:
            # If there is divergence between our position and the Frama direction, exit the position.
            if (self._aapl.holdings.is_long and frama < prev_frama or
                self._aapl.holdings.is_short and frama > prev_frama):
                self.liquidate(self._aapl.symbol)
        # Look for a long entry.
        elif frama > prev_frama and rsi >= 70:
            self.set_holdings(self._aapl.symbol, 0.5)
        # Look for a short entry.
        elif frama < prev_frama and rsi <= 30:
            self.set_holdings(self._aapl.symbol, -0.5)
```

US Equity

Corporate Actions

Introduction

US Equity subscriptions provide notifications for splits, dividends, symbol changes, and delistings.

Splits

When a company does a stock split, the number of shares each shareholder owns increases and the price of each share decreases. When a company does a reverse stock split, the number of shares each shareholder owns decreases and the price of each share increases. A company may perform a stock split or a reverse stock split to adjust the price of their stock so that more investors trade it and the liquidity increases.

When a stock split or reverse stock split occurs for an Equity in your algorithm, LEAN sends a `Split` object to the `on_data` method. You receive `Split` objects when a split is in the near future and when it occurs. To know if the split occurs in the near future or now, check the `type` property.

If you backtest without the `RAW` data normalization mode , the splits are factored into the price and volume. If you backtest with the `RAW` data normalization mode or trade live, when a split occurs, LEAN automatically adjusts your positions based on the `split_factor` . If the post-split quantity isn't a valid lot size , LEAN credits the remaining value to your cashbook in your account currency. If you have indicators or consolidators in your algorithm, reset and warm-up them up with `SCALED_RAW` data when splits occur so that the data in your indicators and consolidators account for the price adjustments that the splits cause. For examples, see Reset Indicators or Reset Consolidators .

To get the `Split` objects, index the `splits` object with the security `symbol` . The `splits` object may not contain data for your `Symbol` . To avoid issues, check if the `splits` object contains data for your security before you index it with the security `Symbol` .

```
PY
def on_data(self, slice: Slice) -> None:
    split = slice.splits.get(self._symbol)
    if split:
        pass


def on_splits(self, splits: Splits) -> None:
    split = splits.get(self._symbol)
    if split:
        pass
```

You can also iterate through the `splits` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `Split` objects.

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, split in slice.splits.items():
        pass

def on_splits(self, splits: Splits) -> None:
    for symbol, split in splits.items():
        pass
```

To get historical splits, make a history request with the `Split` type.

```
PY
splits_df = self.history(Split, symbol, timedelta(4*365))
splits = self.history[Split](symbol, timedelta(4*365))
```

LEAN stores the data for stock splits in factor files. To view some example factor files, see the LEAN GitHub repository . In backtests, your algorithm receives `Split` objects at midnight. In live trading, your algorithm receives `Split` objects when the factor files are ready.

If you hold an Option contract for an underlying Equity when a split occurs, LEAN closes your Option contract position.

If a split event occurs before your order is filled, the unfilled portion of the order is adjusted automatically, where its quantity is multiplied by the split factor and the limit/stop/trigger price (if any) is divided by the split factor.

`Split` objects have the following properties:

Dividends

A dividend is a payment that a company gives to shareholders to distribute profits. When a dividend payment occurs for an Equity in your algorithm, LEAN sends a `Dividend` object to the `on_data` method.

If you backtest with the `ADJUSTED` or `TOTAL_RETURN` data normalization mode, the dividends are factored into the price. If you backtest with the other data normalization modes or trade live, when a dividend payment occurs, LEAN automatically adds the payment amount to your cashbook. If you have indicators or consolidators in your algorithm, reset and warm-up them up with `SCALED_RAW` data when dividend payments occur so that the data in your indicators and consolidators account for the price adjustments that the dividend causes. For examples, see Reset Indicators or Reset Consolidators .

To get the `Dividend` objects, index the `dividends` object with the security `symbol` . The `dividends` object may not contain data for your `symbol` . To avoid issues, check if the `dividends` object contains data for your security before you index it with the security `symbol` .

```
PY
def on_data(self, slice: Slice) -> None:
    dividend = slice.dividends.get(self._symbol)
    if dividend:
        pass

def on_dividends(self, dividends: Dividends) -> None:
    dividend = dividends.get(self._symbol)
    if dividend:
        pass
```

You can also iterate through the `dividends` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `Dividend` objects.

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, dividend in slice.dividends.items():
        pass

def on_dividends(self, dividends: Dividends) -> None:
    for symbol, dividend in dividends.items():
        pass
```

To get historical dividends, make a history request with the `Dividend` type.

```
PY
dividends_df = self.history(Dividend, symbol, timedelta(4*365))
dividends = self.history[Split](Dividend, timedelta(4*365))
```

For a full example, see the DividendAlgorithm in the LEAN GitHub repository.

`Dividend` objects have the following properties:

Symbol Changes

The benefit of the `Symbol` class is that it always maps to the same security, regardless of their trading ticker. When a company changes its trading ticker, LEAN sends a `SymbolChangedEvent` to the `on_data` method.

To get the `SymbolChangedEvent` objects, index the `symbol_changed_events` object with the security `symbol` . The `symbol_changed_events` object may not contain data for your `symbol` . To avoid issues, check if the `symbol_changed_events` object contains data for your security before you index it with the security `symbol` .

```
PY
def on_data(self, slice: Slice) -> None:
    symbol_changed_event = slice.symbol_changed_events.get(self._symbol)
    if symbol_changed_event:
        pass

def on_symbol_changed_events(self, symbol_changed_events: SymbolChangedEvents) -> None:
    symbol_changed_event = symbol_changed_events.get(self._symbol)
    if symbol_changed_event:
        pass
```

You can also iterate through the `symbol_changed_events` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `SymbolChangedEvent` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, symbol_changed_event in slice.symbol_changed_events.items():
        pass


def on_symbol_changed_events(self, symbol_changed_events: SymbolChangedEvents) -> None:
    for symbol, symbol_changed_event in symbol_changed_events.items():
        pass
```

If you have an open order for a security when they change their ticker, LEAN cancels your order. To keep your order, in the `on_order_event` method, get the quantity and `Symbol` of the cancelled order and submit a new order.

```
def on_order_event(self, order_event: OrderEvent) -> None:
    if order_event.status == OrderStatus.CANCELED:
        ticket = self.transactions.get_order_ticket(order_event.order_id)
        if "symbol changed event" in ticket.tag:
            self.transactions.add_order(ticket.submit_request)
```

LEAN stores the data for ticker changes in map files. To view some example map files, see the LEAN GitHub repository .

`SymbolChangedEvent` objects have the following properties:

Delistings

When a company is delisting from an exchange, LEAN sends a `Delisting` object to the `on_data` method. You receive `Delisting` objects when a delisting is in the near future and when it occurs. To know if the delisting occurs in the near future or now, check the `Type` property.

To get the `Delisting` objects, index the `delistings` object with the security `symbol` . The `delistings` objects may not contain data for your `symbol` . To avoid issues, check if the `delistings` object contains data for your security before you index it with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    delisting = slice.delistings.get(self._symbol)
    if delisting:
        pass

def on_delistings(self, delistings: Delistings) -> None:
    delisting = delistings.get(self._symbol)
    if delisting:
        pass
```

You can also iterate through the `delistings` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `Delisting` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, delisting in slice.Delistings.items():
        pass

def on_delistings(self, delistings: Delistings) -> None:
    for symbol, delisting in delistings.items():
        pass
```

The delist warning occurs on the final trading day of the stock to give you time to gracefully exit out of positions before LEAN automatically liquidates them.

```
if delisting.type == DelistingType.WARNING:
    # Liquidate with MarketOnOpenOrder on delisting warning
    quantity = self.portfolio[symbol].quantity
    if quantity != 0:
        self.market_on_open_order(symbol, -quantity)
```

The `on_securities_changed` event notifies the algorithm when delisted assets are removed from the universe.

To improve iteration speed, LEAN removes delisted securities from the `securities` primary collection. To access all the securities, iterate the `securities.total` property.

```py
# Access all the securities.
for security in self.securities.total:
    pass

# Exclude delisted securities.
for security in self.securities.values():
    pass
```

For a full example, see the DelistingEventsAlgorithm in the LEAN GitHub repository.

`Delisting` objects have the following properties:

Live Trading Considerations

In backtesting, corporate actions occurs at midnight Eastern Time (ET). In live trading, the live data for corporate actions arrives at 6/7 AM ET, so that's when they occur.

Examples

The following examples demonstrate some common practices for US Equity corporate actions.

Example 1: Resetting Indicators on Split Events

The following algorithm adds raw TSLA data and uses it to update an Exponential Moving Average (EMA) indicator. When a split occurs, it resets and warms up the indicator with scaled raw data to ensure the indicator doesn't reflect price discontinuities.

```py
class USEquityCorporateActionExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add raw US Equity data.
        self._equity = self.add_equity("TSLA", Resolution.DAILY, data_normalization_mode=DataNormalizationMode.RAW)
        # Create an EMA indicator.
        self._equity.ema = self.ema(self._equity.symbol, 50)

    def on_splits(self, splits: Splits) -> None:
        # Wait until the split has occurred.
        split = splits[self._equity.symbol]
        if split.type != SplitType.SPLIT_OCCURRED:
            return
        # Reset the indicator.
        self._equity.ema.reset()
        # Warm up the indicator with scaled raw data.
        history = self.history[TradeBar](
            self._equity.symbol, self._equity.ema.warm_up_period,
            data_normalization_mode=DataNormalizationMode.SCALED_RAW
        )
        for bar in history:
            self._equity.ema.update(bar.end_time, bar.close)
```

Example 2: Reinvesting Dividend Payments

The following algorithm adds raw SPY data. It then holds the SPY and re-invests the dividend payments into additional shares.

PY

```
class USEquityCorporateActionExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add raw Equity data.
        self._spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW)

    def on_data(self, slice: Slice) -> None:
        # Buy and hold.
        if not self._spy.holdings.invested:
            self.set_holdings(self._spy.symbol, 0.5)

    def on_dividends(self, dividends: Dividends) -> None:
        # Get the total dividend payment amount.
        total_dividend_amount = dividends[self._spy.symbol].distribution * self._spy.holdings.quantity
        # Calculate the number of shares you can buy with the dividend payment.
        quantity = np.floor(total_dividend_amount / self._spy.price)
        if quantity:
            # Place an order to re-invest the dividends.
            self.market_order(self._spy.symbol, quantity)
```

US Equity

Corporate Fundamentals

Introduction

Corporate fundamental data contains all the information on the underlying company of an Equity asset and the information in their financial statements. Since corporate data contains information not found in price and alternative data, adding corporate data to your trading strategies provides you with more information so you can make more informed trading decisions. Corporate fundamental data is available through the US Fundamental Data from Morningstar .

Direct Access

To get fundamental data for Equities in your algorithm, use the `fundamentals` property of the `Equity` objects. The fundamental data represent the company's fundamentals for the current algorithm time.

```
fundamentals = self.securities[self._symbol].fundamentals
```

To get fundamental data for Equities, regardless of whether or not you have subscribed to them in your algorithm, call the `fundamentals` method. If you pass one `Symbol` , the method returns a `Fundamental` object. If you pass a list of `Symbol` objects, the method returns a list of `Fundamental` objects. The fundamental data represents the corporate fundamentals for the current algorithm time.

```
# Single asset
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals(ibm)

# Multiple assets
nb = Symbol.create("NB", SecurityType.EQUITY, Market.USA)
fundamentals = self.fundamentals([ nb, ibm ])
```

Data Availability

Some assets don't have fundamentals (for example, ETFs) and the Morningstar dataset doesn't provide fundamentals for all US Equities. To check if fundamental data is available for an asset, use the `has_fundamental_data` property.

```
has_fundamental_data = self.securities[self._symbol].fundamentals.has_fundamental_data
```

Object References

If you save a reference to the `fundamentals` object or its properties, you can access the fundamental properties as they change over time.

```
self._fundamentals = self.securities[self._symbol].fundamentals
earning_ratios = self.fundamentals.earning_ratios
```

Universe Selection

You can access fundamental data in the selection function of your fundamental universe .

```
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._fundamental_function)

    def _fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        return [c.symbol for c in fundamental if c.has_fundamental_data]
```

Historical Data

To get historical fundamental data, call the `history` method. The return type depends on how you call the method.

```
PY
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)

# DataFrame of fundamental data for a given asset
df_history = self.history(Fundamental, ibm, timedelta(30), flatten=True)

# Fundamental objects
fundamental_history = self.history[Fundamental](ibm, timedelta(30))

# Fundamentals objects for all US Equities (including delisted companies)
fundamentals_history = self.history[Fundamentals](timedelta(30))

# DataFrame of fundamental data for universe constituents
df_history = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series of fundamental data for universe constituents
series_history = self.history(self._universe, 30, Resolution.DAILY)
for (universe_symbol, time), fundamental in series_history.items():
    highest_market_cap = sorted(fundamental, key=lambda x: x.market_cap)[-5:]
```

For more information about historical fundamental data, see Equity Fundamental Data .

Data Updates

The US Fundamental dataset only provides the originally-reported figures. If there was a mistake in reporting a figure, the data value isn't fixed later. In live trading, new fundamental data is available to your algorithms at approximately 6/7 AM Eastern Time (ET) each day. The majority of the corporate data update occurs once per month, but the financial ratios update daily. If there is no new data for a period of time, the previous data is filled forward.

Properties

The US Fundamentals dataset provides Fundamental objects. To filter Fundamental objects, you can use the `MorningstarSectorCode` , `MorningstarIndustryGroupCode` , and `MorningstarIndustryCode` enumeration values.

Fundamentals Attributes

Fundamentals objects have the following attributes:

MorningstarSectorCode Enumeration

Sectors are large super categories of data. To access the sector of an Equity, use the **morningstar_sector_code** property.

```
PY
filtered = [x for x in fundamental if x.asset_classification.morningstar_sector_code == MorningstarSectorCode.TECHNOLOGY]
```

The **MorningstarSectorCode** enumeration has the following members:

MorningstarIndustryGroupCode Enumeration

Industry groups are clusters of related industries which tie together. To access the industry group of an Equity, use the **morningstar_industry_group_code** property.

```
PY
filtered = [x for x in fundamental if x.asset_classification.morningstar_industry_group_code ==
MorningstarIndustryGroupCode.APPLICATION_SOFTWARE]
```

The **MorningstarIndustryGroupCode** enumeration has the following members:

MorningstarIndustryCode Enumeration

Industries are the finest level of classification available and are the individual industries according to the Morningstar classification system. To access the industry group of an Equity, use the **morningstar_industry_code** property:

```
filtered = [x for x in fundamental if x.asset_classification.morningstar_industry_code == MorningstarIndustryCode.SOFTWARE_APPLICATION]
```

The **MorningstarIndustryCode** enumeration has the following members:

Exchange Id Values

Exchange Id is mapped to represent the exchange that lists the Equity. To access the exchange Id of an Equity, use the **primary_exchange_id** property.

```
filtered = [x for x in fundamental if x.company_reference.primary_exchange_id == "NAS"]
```

The exchanges are represented by the following string values:

String Representation	Exchange
NYS	New York Stock Exchange (NYSE)
NAS	NASDAQ
ASE	American Stock Exchange (AMEX)
TSE	Tokyo Stock Exchange
AMS	Amsterdam Internet Exchange
SGO	Santiago Stock Exchange
XMAD	Madrid Stock Exchange
ASX	Australian Securities Exchange
BVMF	B3 (stock exchange)
LON	London Stock Exchange
TKS	Istanbul Stock Exchange Settlement and Custody Bank
SHG	Shanghai Exchange
LIM	Lima Stock Exchange
FRA	Frankfurt Stock Exchange
JSE	Johannesburg Stock Exchange
MIL	Milan Stock Exchange
TAE	Tel Aviv Stock Exchange
STO	Stockholm Stock Exchange
ETR	Deutsche Boerse Xetra Core
PAR	Paris Stock Exchange
BUE	Buenos Aires Stock Exchange
KRX	Korea Exchange
SWX	SIX Swiss Exchange

PINX	Pink Sheets (OTC)
CSE	Canadian Securities Exchange
PHS	Philippine Stock Exchange
MEX	Mexican Stock Exchange
TAI	Taiwan Stock Exchange
IDX	Indonesia Stock Exchange
OSL	Oslo Stock Exchange
BOG	Colombia Stock Exchange
NSE	National Stock Exchange of India
HEL	Nasdaq Helsinki
MISX	Moscow Exchange
HKG	Hong Kong Stock Exchange
IST	Istanbul Stock Exchange
BOM	Bombay Stock Exchange
TSX	Toronto Stock Exchange
BRU	Brussels Stock Exchange
BATS	BATS Global Markets
ARCX	NYSE Arca
GREY	Grey Market (OTC)
DUS	Dusseldorf Stock Exchange
BER	Berlin Stock Exchange
ROCO	Taipei Exchange
CNQ	Canadian Trading and Quotation System Inc.
BSP	Bangko Sentral ng Pilipinas
NEOE	NEO Exchange

Examples

The following examples demonstrate some common practices for US Equity corporate fundamentals.

Example 1: Capturing the Overreaction to Financial Reports

The following algorithm is based on the hypothesis that investors overreact to new financial report releases. It buys TSLA stock on every new 10Q or 8K filing and then liquidates the position after seven days.

```py
class USEquityCorporateFundamentalsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add Equity data.
        self._equity = self.add_equity("TSLA")
        # Add a member to track the last financial statement filing date.
        self._last_filing_date = None

    def on_data(self, slice: Slice) -> None:
        # Get the most recent filing date.
        last_filing_date = self._equity.fundamentals.financial_statements.file_date.value

        # When new financial statements are released, buy TSLA.
        if self._last_filing_date and last_filing_date != self._last_filing_date:
            self.set_holdings(self._equity.symbol, 1)
            self._exit_time = self.time + timedelta(7)

        # Liquidate the position after 1 week.
        if self._equity.holdings.invested and self.time >= self._exit_time:
            self.liquidate()

        # Update the last filing date to identify future changes.
        self._last_filing_date = last_filing_date
```

Example 2: Diversifying Across Sectors

The follow algorithm aims to reduce concentration risk in any one sector. It selects the 500 largest Equities at the start of each month and then forms a portfolio that gives equal weight to each sector and equal weight to the assets within each sector.

```py
class USEquityCorporateFundamentalsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10_000_000)
        # Define the universe settings.
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start('SPY'))
        # Add a universe of the 500 largest Equities.
        self._universe = self.add_universe(
            lambda fundamental: [
                f.symbol for f in sorted(fundamental, key=lambda f: f.market_cap)[-500:]
            ]
        )
        # Create a Scheduled Event to rebalance the portfolio each month.
        self.schedule.on(self.date_rules.month_start('SPY', 1), self.time_rules.midnight, self._rebalance)

    def _rebalance(self) -> None:
        # Wait until there are asset in the universe.
        if not self._universe.selected:
            return

        # Group the universe constituents by sector.
        symbols_by_sector = {}
        for symbol in self._universe.selected:
            equity = self.securities[symbol]
            if not equity.price:
                continue
            sector_code = equity.fundamentals.asset_classification.morningstar_sector_code
            if sector_code not in symbols_by_sector:
                symbols_by_sector[sector_code] = []
            symbols_by_sector[sector_code].append(symbol)

        # Create targets to invest equally in each sector and invest equally in the assets within each sector.
        targets = []
        sector_count = len(symbols_by_sector)
        for symbols in symbols_by_sector.values():
            weight = 1 / sector_count / len(symbols)
            targets.extend([PortfolioTarget(symbol, weight) for symbol in symbols])

        # Place the trades.
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

US Equity

Shorting

Introduction

In a regular long position, you buy shares and then sell them later to close the trade. In a short position, you borrow shares, sell them, and then buy the shares back later to close the trade. Short positions let you profit when a security's price decreases while you have the position open, but there are risks involved. For example, you can get a margin call and you may incur borrowing costs.

Holdings Accounting

If you have an open long position, your portfolio has a positive quantity of shares for that security. In contrast, if you have an open short position, your portfolio has a negative quantity of shares for that security.

Short Availability

To short sell, you need to borrow the shares from another investor or your brokerage. The short availability is the number of shares available for you to borrow. If you don't model the short availability in backtests, you could have a strategy that performs well in backtesting but doesn't trade in live mode because the shares aren't available to borrow. If you assume your short orders are successful in backtests, your backtesting results likely won't represent the true performance of actually trading the strategy during the backtest period. To get the short availability in your algorithms, use the US Equities Short Availability dataset . For more information about this topic, see Short Availability .

Borrowing Costs

In live trading, your brokerage charges an interest fee when you borrow shares to short a security. In backtesting, use the ShortMarginInterestRateModel .

To get valid borrowing rates, use the InteractiveBrokersShortableProvider .

```py
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # For US Equity, use the ShortMarginInterestRateModel with IB data
    security.set_shortable_provider(InteractiveBrokersShortableProvider())
    security.margin_interest_rate_model = ShortMarginInterestRateModel()
```

Examples

The following examples demonstrate some common practices for shorting US Equities:

Example 1: Shorting Penny Stock Top Gainers

The following algorithm adds a universe of penny stocks that have experienced the greatest upward momentum over the last two trading days. For each asset in the universe, it monitors an hourly Bollinger Bands indicator. When an asset breaks above the upper band, it shorts the asset. It exits each position when the asset drops 10% from the entry price or when the price reverts back below the middle Bollinger Band.

PY

```
class USEquityShortingExampleAlgorithm(QCAlgorithm):

    _roc_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Enable automatic indicator warm-up.
        self.settings.automatic_indicator_warm_up = True
        # Add a universe of low-cap penny stocks that have high upward momentum.
        self.add_universe(self._select_assets)

    def _select_assets(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select all the penny stocks that have a momentum score.
        penny_stocks = []
        for f in fundamentals:
            # Create a RateOfChange indicator for each asset.
            self._roc_by_symbol[f.symbol] = self._roc_by_symbol.get(f.symbol, RateOfChange(1))
            # Update the indicator an ensure it's ready.
            # Filter out assets above $5M market cap or $1 share price.
            if not self._roc_by_symbol[f.symbol].update(f.end_time, f.price) or f.market_cap > 5e6 or f.price > 1:
                continue
            penny_stocks.append(f.symbol)

        # Select the 20 penny stocks with the greatest momentum.
        return sorted(penny_stocks, key=lambda symbol: self._roc_by_symbol[symbol].current.value)[-20:]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Iterate through the assets that entered the universe.
        for security in changes.added_securities:
            # Set the shortable provider of the asset to ensure realistic short availability.
            security.set_shortable_provider(InteractiveBrokersShortableProvider())
            # Create a BollingerBand indicator for this asset.
            security.bbands = self.bb(security.symbol, 20, 2, resolution=Resolution.HOUR)
            # Attach an event handler to the indicator to track its updates.
            security.bbands.updated += self.on_bb_updated

        # Liquidate and remove the indicator updates for assets that leave the universe.
        for security in changes.removed_securities:
            self.deregister_indicator(security.bbands)
            self.liquidate(security.symbol)

    def on_bb_updated(self, bb: object, _: IndicatorDataPoint) -> None:
        # When the indicator updates, ensure it's ready.
        if not bb.is_ready:
            return
        security = self.securities[bb.current.symbol]
        # Short the asset if the price is above the upper band.
        if not security.holdings.invested and security.price > bb.upper_band.current.value:
            # Get the number of shares avaiable for shorting.
            shortable_quantity = security.shortable_provider.shortable_quantity(security.symbol, self.time)
            # Calculate the target quantity.
            quantity = max(self.calculate_order_quantity(security.symbol, -0.1), -shortable_quantity if shortable_quantity else 0)
            if quantity == 0: return
            # Short the asset.
            self.market_order(security.symbol, quantity)
            # Place a take-profit order 10% below the current price.
            self.limit_order(security.symbol, -quantity, 0.9 * security.bid_price)
        # If the limit order isn't hit before the price drops below the middle band,
        # exit the position and cancel the limit order.
        elif security.holdings.invested and security.price < bb.middle_band.current.value:
            self.liquidate(security.symbol)
```

US Equity

Data Preparation

Introduction

The US Equities dataset provides price data for backtests and live trading.

Sourcing

The QuantConnect data provider consolidates US Equity market data across all of the exchanges. Over-the-Counter (OTC) trades are excluded. The data is powered by the Securities Information Processor (SIP), so it has 100% market coverage. In contrast, free platforms that display data feeds like the Better Alternative Trading System (BATS) only have about 6-7% market coverage .

We provide live splits, dividends, and corporate actions for US companies. We deliver them to your algorithm before the trading day starts.

Bar Building

We aggregate ticks to build bars.

The bar-building process can exclude ticks. If a tick is excluded, its volume is aggregated in the bar but its price is not aggregated in the bar. Ticks are excluded if any of the following statements are true:

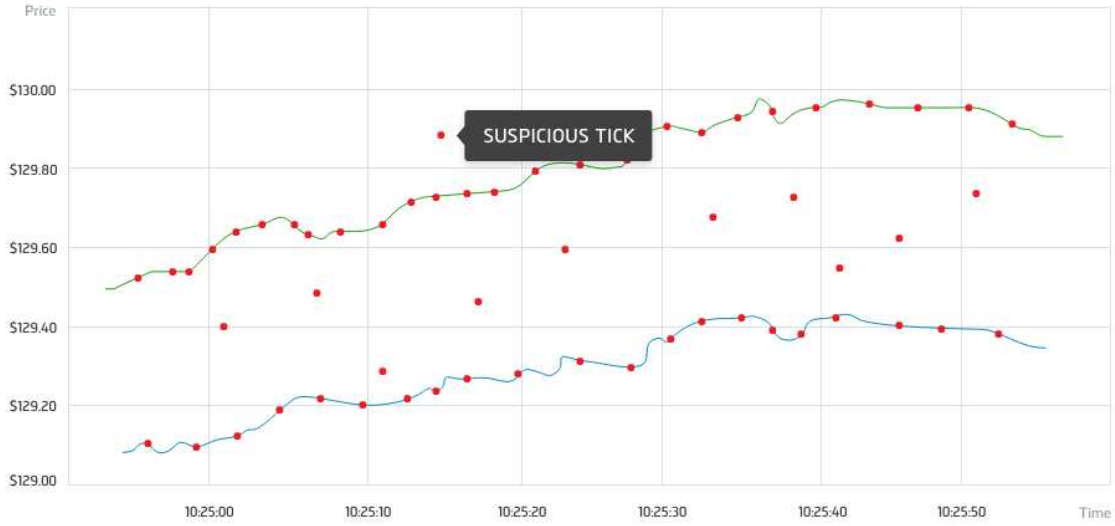
- The tick is suspicious.
- The tick is from the FINRA exchange and meets our price and volume thresholds.
- The trade has none of the following included `TradeConditionFlags` and at least one of the following excluded `TradeConditionFlags` :
- The quote has a size of less than 100 shares.
- The quote has none of the following included `QuoteConditionFlags` and at least one of the following excluded `QuoteConditionFlags` :
- The quote has one of the following `QuoteConditionFlags` :

In the preceding tables, Participant refers to the entities on page 19 of the Consolidated Tape System Multicast Output Binary Specification .

Suspicious Ticks

Tick price data is raw and unfiltered, so it can contain a lot of noise. If a tick is not tradable, we flag it as suspicious. This process makes the bars a more realistic representation of what you could execute in live trading. If you use tick data, avoid using suspicious ticks in your algorithms as informative data points. We recommend only using tick data if you understand the risks and are able to perform your own tick filtering. Ticks are flagged as suspicious in the following situations:

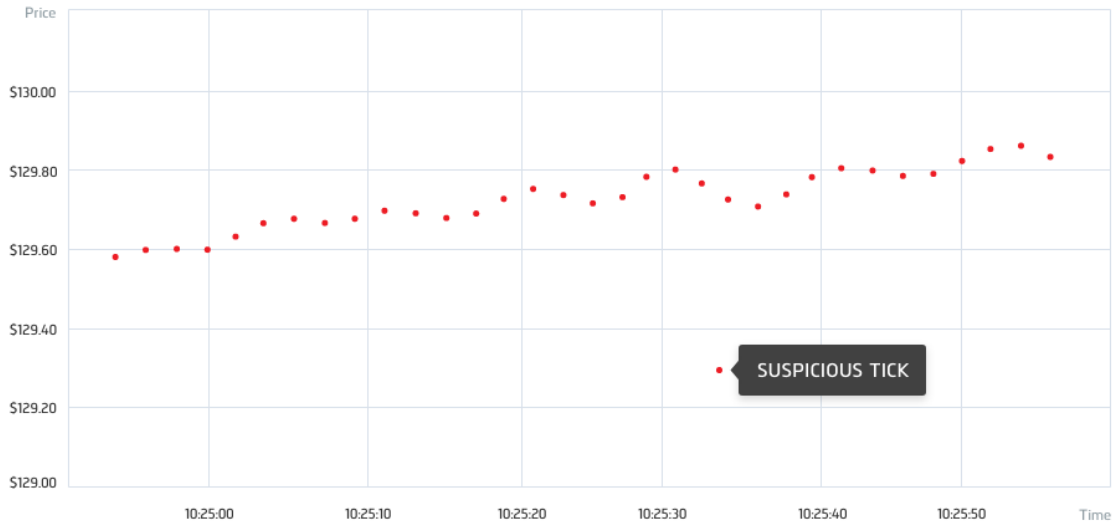
- The tick occurs below the best bid or above the best ask



This image shows a tick that occurred above the best ask price of a security. The green line represents the best ask of the security, the blue

line represents the best bid of the security, and the red dots represent trade ticks. The ticks between the best bid and ask occur from filling hidden orders. The tick that occurred above the best ask price is flagged as suspicious.

- The tick occurs far from the current market price



This image shows a tick that occurred far from the price of the security. The red dots represent trade ticks. The tick that occurred far from the market price is flagged as suspicious.

- The tick occurs on a dark pool
- The tick is rolled back
- The tick is reported late

Market Auction Prices

The opening and closing price of the day is set by very specific opening and closing auction ticks. When a stock like Apple is listed, it's listed on Nasdaq. The open auction tick on Nasdaq is the price that's used as the official open of the day. NYSE, BATS, and other exchanges also have opening auctions, but the only official opening price for Apple is the opening auction on the exchange where it was listed.

We set the opening and closing prices of the first and last bars of the day to the official auction prices. This process is used for second, minute, hour, and daily bars for the 9:30 AM and 4:00 PM Eastern Time (ET) prices. In contrast, other platforms might not be using the correct opening and closing prices.

The official auction prices are usually emitted 2-30 seconds after the market open and close. We do our best to use the official opening and closing prices in the bars we build, but the delay can be so large that there isn't enough time to update the opening and closing price of the bar before it's injected into your algorithms. For example, if you subscribe to second resolution data, we wait until the end of the second for the opening price but most second resolution data won't get the official opening price. If you subscribe to minute resolution data, we wait until the end of the minute for the opening auction price. Most of the time, you'll get the actual opening auction price with minute resolution data, but there are always exceptions. Nasdaq and NYSE can have delays in publishing the opening auction price, but we don't have control over those issues and we have to emit the data on time so that you get the bar you are expecting.

Live and Backtesting Differences

In live trading, bars are built using the exchange timestamps with microsecond accuracy. This microsecond-by-microsecond processing of the ticks can mean that the individual bars between live trading and backtesting can have slightly different ticks. As a result, it's possible for a tick to be counted in different bars between backtesting and live trading, which can lead to bars having slightly different open, high, low, close, and volume values.

Data Availability

In live trading, live data is available in real-time. In backtests, live data is available at the following pre-market trading session.

Examples

The following examples demonstrate some common practices related to our US Equity data preparation process.

Example 1: Trading at the Opening Auction

The following algorithm creates a Scheduled Event that places a Market On Open order , which fill at the official opening auction price.

```py
class USEquityDataPreparationExampleAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add Equity data.
        self._spy = self.add_equity("SPY", extended_market_hours=True).symbol
        # Create a Scheduled Event that rebalances with market on open orders.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.before_market_open(self._spy, 30),
            self._rebalance
        )

    def _rebalance(self):
        quantity = self.calculate_order_quantity(self._spy, 0.1)
        if quantity:
            self.market_on_open_order(self._spy, quantity)
```

Example 2: Demonstrate Using Tick Flags to Identify Opening Auction Tick

This example demonstrates using the tick flags to identify the official opening price on the security primary exchange. All exchanges have an opening auction to determine the market on open price, but only the primary listed exchange is used as the official open price for the asset. The algorithm subscribes to the raw tick data of QQQ ETF and logs the auction price tick data of the asset.

```py
class USEquityDataPreparationExampleAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 5)
        self.set_end_date(2024, 9, 5)
        # Subscribe to tick data of a security and find the exchange code.
        self._asset = self.add_equity(ticker = "QQQ", resolution = Resolution.TICK, extended_market_hours = True, data_normalization_mode
= DataNormalizationMode.RAW)
        # Define flags to help filter opening auctions in the OnData method.
        self.officialOpen = TradeConditionFlags.REGULAR | TradeConditionFlags.OFFICIAL_OPEN
        self.openingPrints = TradeConditionFlags.REGULAR | TradeConditionFlags.OPENING_PRINTS
        # Display exchange code and trade condition flags on the console.
        self.debug(f"Search Codes: Ex:{self._asset.primary_exchange.code} OO: {self.officialOpen} OP: {self.openingPrints}")

    def on_data(self, data):
        # Create a list of opening auction tick data.
        trades = [
            tick for tick in self._asset.cache.get_all[Tick]()
            if  tick.TickType == TickType.TRADE and
                tick.Price > 0 and
                tick.SaleCondition and
                (
                    tick.ParsedSaleCondition == self.officialOpen or
                    tick.ParsedSaleCondition == self.openingPrints
                )
        ]
        # Log the opening auction tick price
        for tick in trades:
            self.log(f"{self.Time},{tick.Price},{tick.Quantity},{tick.ParsedSaleCondition},{tick.ExchangeCode},{tick.Exchange}")
```

US Equity

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the US Equity market.

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

Equity Options

Equity Options are a financial derivative that gives the holder the right (but not the obligation) to buy or sell the underlying Equity, such as Apple, at the stated exercise price.

Requesting Data

Handling Data

Greeks and Implied Volatility

Market Hours

See Also

BasicTemplateOptionsAlgorithm.py
BasicTemplateOptionsAlgorithm.cs
Introduction to Options
Applied Options

Equity Options

Requesting Data

Request Equity Options data in your algorithm to receive a feed of contract prices in the `on_data` method. For more information about the specific dataset we use for backtests, see the US Equity Options dataset listing . To trade Equity Options live, you can use one of the brokerage data providers . We currently only support American-style Options for US Equity Options.

To request Equity Options data, call one of the following methods:

- `self.add_option` to add a basket of contracts for a single underlying Equity
- `self.add_option_contract` to add individual contracts for an underlying Equity

Request and Filter Contracts for Option Strategies

`self.add_option`

Request and Filter All Contracts

`self.add_option`

Select Individual Contracts

`self.add_option_contract`

See Also

Introduction to Options
Applied Options

Requesting Data

Universes

Introduction

The `add_option` method enables you to select a basket of Option contracts for an underlying Equity. To form your universe of contracts, you can filter them down by their strike price and expiry or you can choose to a subset of the contracts that form popular Option strategies . If you want to subscribe to individual contracts one-by-one instead of a set of contracts, see Individual Contracts .

Create Universes

To add a universe of Equity Option contracts, in the `initialize` method, call the `add_option` method. This method returns an `Option` object, which has a `set_filter` method you can call to filter the set of tradable contract down to just the contracts you want.

```
PY
class BasicOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.universe_settings.asynchronous = True
        option = self.add_option("SPY")
        option.set_filter(self._filter)
        self._symbol = option.symbol

    def _filter(self, universe):
        return universe.expiration(0, 7).delta(0.35, 0.75)

    def on_data(self, data):
        chain = data.option_chains.get(self._symbol)
        if chain:
            contract = sorted(chain, key=lambda x: (x.expiry, x.greeks.delta))[0]
```

For more information about the `add_option` method, see Create Universes .

Filter by Investment Strategy

Options trading strategies consist of simultaneously buying or selling one or more Option contracts of the same underlying Equity with different features.

```
PY
# Example 1: Select a Straddle
option.set_filter(lambda option_filter_universe: option_filter_universe.straddle(30))

# Example 2: Select the contracts (including weeklys) that expire in the next 30 days that form an Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 30).iron_condor(30, 5, 10))

# Example 3: Select the 0DTE contracts that form an Strangle
option.set_filter(lambda option_filter_universe: option_filter_universe.expiration(0, 0).strangle(30, 5, -10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contracts for Option strategies:

`naked_call(min_days_till_expiry: int, strike_from_atm: float)` Selects a call contract to form Naked Call , Covered Call , or Protective Call Option strategies.
`naked_put(min_days_till_expiry: int, strike_from_atm: float)` Selects a put contract to form Naked Put , Covered Put , or Protective Put Option strategies.
`call_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two call contracts to form Bull Call Spread or Bear Call Spread Option strategies.

`put_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)`

Selects two put contracts to form Bull Put Spread or Bear Put Spread Option strategies.

`call_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects two call contracts to form Long Call Calendar Spread or Short Call Calendar Spread Option strategies.

`put_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects two put contracts to form Long Put Calendar Spread or Short Put Calendar Spread Option strategies.

`strangle(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)`

Selects two contracts to form Long Strangle or Short Strangle Option strategies.

`straddle(min_days_till_expiry: int)`

Selects two contracts to form Long Straddle or Short Straddle Option strategies.

`protective_collar(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)`

Selects two contracts to form Protective Collar Option strategies.

`conversion(min_days_till_expiry: int, strike_from_atm: float)`

Selects two contracts to form Conversion or Reverse Conversion Option strategies.

`call_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects three contracts to form Long Call Butterfly or Short Call Butterfly Option strategies.

`put_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects three contracts to form Long Put Butterfly or Short Put Butterfly Option strategies.

`iron_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects four contracts to form Long Iron Butterfly or Short Iron Butterfly Option strategies.

`iron_condor(min_days_till_expiry: int, near_strike_spread: float, far_strike_spread: float)`

Selects four contracts to form Long Iron Condor or Short Iron Condor Option strategies.

`box_spread(min_days_till_expiry: int, strike_spread: float)`

Selects four contracts to form Box Spread or Short Box Spread Option strategies.

`jelly_roll(strike_from_atm: float, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects four contracts to form Jelly Roll or Short Jelly Roll Option strategies.

`call_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)`

Selects four contracts to form Bear Call Ladder or Bull Call Ladder Option strategies.

`put_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Put Ladder or Bull Put Ladder Option strategies.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

Filter by Implied Volatility and Greeks

Option price models compute the theoretical price of Option contracts, their implied volatility, and their Greek values. Theoretical prices can help you detect undervalued and overvalued contracts, implied volatility can provide you insight into the upcoming volatility of the underlying security, and Greek values can help you hedge your portfolio.

```py
# Example 1: Select the contracts with delta between 0.25 and 0.75
option.set_filter(lambda option_filter_universe: option_filter_universe.delta(0.25, 0.75))

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days with implied volatility below 20%
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 90))

# Example 3: Select the contracts (including weeklys) that expire in the next 30 days with implied volatility below 20% that forms an Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 30).iron_condor(30, 5, 10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for a range of implied volatility, Greek values, and open interest:

`iv(min: float, max: float)`
`implied_volatility(min: float, max: float)`
 	
Selects a contract with implied volatility within the range you set.

`d(min: float, max: float)`
`delta(min: float, max: float)`
 	
Selects a contract with delta within the range you set.

`g(min: float, max: float)`
`gamma(min: float, max: float)`
 	
Selects a contract with gamma within the range you set.

`r(min: float, max: float)`
`rho(min: float, max: float)`
 	
Selects a contract with rho within the range you set.

`v(min: float, max: float)`
`vega(min: float, max: float)`
 	
Selects a contract with vega within the range you set.

`t(min: float, max: float)`
`theta(min: float, max: float)`
 	
Selects a contract with theta within the range you set.

`oi(min: int, max: int)`
`open_interest(min: float, max: float)`
 	
Selects a contract with open interest within the range you set.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

To perform thorough filtering on the `OptionFilterUniverse` , define an isolated filter method.

```
# Select the put contracts with the lowest strike price.
option.set_filter(self._contract_selector)

def _contract_selector(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
    return option_filter_universe \
        .delta(0.5, 1.5) \
        .gamma(0.0001, 0.0006) \
        .vega(0.01, 1.5) \
        .theta(-2.0, -0.5) \
        .rho(0.5, 3.0) \
        .implied_volatility(1, 3) \
        .open_interest(100,500)
```

Filter by Other Contract Properties

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Option` object.

```
# Select contracts that have a strike price within 1 strike level above and below the underlying price
option.set_filter(min_strike=-1, max_strike=1)

# Select contracts that expire within 30 days
option.set_filter(min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select contracts that have a strike price within 1 strike level and expire within 30 days
option.set_filter(min_strike=-1, max_strike=1, min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only())
```

The following table describes the available filter techniques:

`set_filter(minStrike: int, maxStrike: int)` Selects the contracts that have a strike price within a minimum and maximum strike level relative to the underlying price. For example, say the underlying price is 302 and there are strikes at every 5. If you set `m_strike` to -1 and `max_strike` to 1, LEAN selects the contracts that have a strike of 300 or 305. This filter runs asynchronously by default.
`set_filter(minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(minStrike: int, maxStrike: int, minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire and have a strike within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[OptionFilterUniverse], OptionFilterUniverse])` Selects the contracts that a function selects.

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
# Example 1: Select the front month call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only().front_month())

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 90))
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Default Filter

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard and weeklys
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at the first bar of every day. When the filter selects a contract that isn't currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

Historical Data

To get historical chains for an Equity Option, see Historical Data .

Examples

Example 1: Selecting for 0DTE Contracts

0DTE Options are Option contracts that expire on the same day you trade them. To create an universe with 0DTE Options, call the `set_filter` method with expiration set. In order to trade with the contracts with most liquidity, limit the strike range from 10 strikes above and below the current price level.

```
class EquityOptionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_option("SPY")
        # Filter for 0-DTE contracts within +/-10 strike range.
        # They have the highest volatility but low premium, providing sufficient liquidity and higher profit margin.
        option.set_filter(lambda u: u.expiration(0, 0).strikes(-10, 10))
```

Example 2: Straddle

You can also create an universe that return the option contracts that form an option strategy . In this example, we demonstrate filtering for a long straddle strategy that expires 30 days later.

```
class EquityOptionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_option("SPY")
        self._symbol = option.symbol
        # Using straddle() method, it will only return the best-matched ATM call and put contracts expiring after 30 days.
        # It provides better accuracy in filtering, and subscribe only to the needed contracts to save computation resources.
        option.set_filter(lambda u: u.straddle(30))

    def on_data(self, slice: Slice) -> None:
        # Only wants the option chain of the selected symbol.
        chain = slice.option_chains.get(self._symbol)
        if not self.portfolio.invested and chain:
            # There should only be 1 expiry and 1 strike from the 2 contracts returned, getting from either contract is fine.
            expiry = list(chain)[0].expiry
            strike = list(chain)[0].strike

            # Forms a long straddle option strategy using abstraction method ensure accuracy.
            long_straddle = OptionStrategies.straddle(self._symbol, strike, expiry)
            self.buy(long_straddle, 1)
```

Example 3: Custom Selector

Using a custom selection function, you can filter the option universe to obtain the contracts that you need without subscribing the redundants. This can ensure the accuracy of the filter, as well as avoiding unnecessary subscriptions that drag the algorithm computational performance. In this example, we try to obtain a put contract that expires 30 days later that is 5 below the current underlying price and a call contract that expires 60 days later that is 10 above the current underlying price.

PY

```
class EquityOptionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price with last known price to ensure the underlying price data is available on initial option contract filtering.
        self.settings.seed_initial_prices = True
        self.underlying = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW)
        # Subscribe to underlying data for ATM calculation using the update underlying price.
        # Set data normalization mode to raw is required to ensure strike price and underlying price is comparable.
        option = self.add_option(self.underlying.symbol)
        option.set_filter(lambda u: u.contracts(self.selection))

    def selection(self, contracts: List[Symbol]) -> List[Symbol]:
        selected = []

        # Select the best-match put contracts that expires 30 days later that is $5 below the current underlying price.
        # We should first filter the hard requirement, then sort the reminders for faster computation.
        puts = [x for x in contracts
            if x.id.option_right == OptionRight.PUT and \
            x.id.date >= self.time + timedelta(30) and \
            x.id.strike_price <= self.underlying.price - 5
        ]
        if len(puts) > 0:
            expiry = min(x.id.date for x in puts)
            put = sorted([x for x in puts if x.id.date == expiry],
                key=lambda x: x.id.strike_price,
                reverse=True)[0]
            selected.append(put)

        # Select the best-match call contract that expires 60 days later that is $10 above the current underlying price.
        calls = [x for x in contracts
            if x.id.option_right == OptionRight.CALL and \
            x.id.date >= self.time + timedelta(60) and \
            x.id.strike_price >= self.underlying.price + 10
        ]
        if len(calls) > 0:
            expiry = min(x.id.date for x in calls)
            call = sorted([x for x in calls if x.id.date == expiry],
                key=lambda x: x.id.strike_price)[0]
            selected.append(call)

        return selected
```

Requesting Data

Individual Contracts

Introduction

The `add_option_contract` method enables you to add an individual Option contract to your algorithm. To check which contracts are currently available to add to your algorithm, use the `option_chain` method. If you want to subscribe to a set of contracts instead of individual contracts one-by-one, see Universes .

Create Subscriptions

Before you can subscribe to an Option contract, you must configure the underlying Equity and get the contract `Symbol` .

```
class BasicOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        self._contract_symbol = None

    def on_data(self, data):
        if self._contract_symbol:
            return
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        expiry = chain.expiry.min()
        self._contract_symbol = chain[
            (chain.expiry == expiry) &
            (chain.right == OptionRight.CALL) &
            (chain.delta < 0.7) &
            (chain.delta > 0.3)
        ].sort_values('openinterest').index[-1]
        self.add_option_contract(self._contract_symbol)
```

Configure the Underlying Equity

If you want to subscribe to the underlying Equity in the `initialize` method, set the Equity data normalization to `DataNormalizationMode.RAW` .

```
self._underlying = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
```

If your algorithm has a dynamic universe of Equities, before you add the Equity universe in the `initialize` method, set the universe data normalization mode to `DataNormalizationMode.RAW` .

```
self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
```

Get Contract Symbols

To subscribe to an Option contract, you need the contract `Symbol` . The preferred method to getting Option contract `Symbol` objects is to use the `option_chain` method. This method returns an `OptionChain` object, which represent an entire chain of Option contracts for a single underlying security. You can even format the chain data into a DataFrame where each row in the DataFrame represents a single contract. With the chain, sort and filter the data to find the specific contract(s) you want to trade.

```
# Get the contracts available to trade (in DataFrame format).
chain = self.option_chain(self._underlying, flatten=True).data_frame

# Select a contract.
expiry = chain.expiry.min()
self._contract_symbol = chain[
    # Select call contracts with the closest expiry.
    (chain.expiry == expiry) &
    (chain.right == OptionRight.CALL) &
    # Select contracts with a 0.3-0.7 delta.
    (chain.delta > 0.3) &
    (chain.delta < 0.7)
    # Select the contract with the largest open interest.
].sort_values('openinterest').index[-1]
```

Alternatively, you can create a Option contract `Symbol` object directly parsing its OSI ticker using the `SymbolRepresentation.parse_option_ticker_osi` method.

```
# Create the contract Symbol for the SPY Call 564 3 Sept 2024.
self._contract_symbol = SymbolRepresentation.parse_option_ticker_osi('SPY 240903C00564000')
```

Subscribe to Contracts

To create an Equity Option contract subscription, pass the contract `Symbol` to the `add_option_contract` method. Save a reference to the contract `symbol` so you can easily access the Option contract in the OptionChain that LEAN passes to the `on_data` method. This method returns an `Option` object. To override the default pricing model of the Option, set a pricing model .

```
option = self.add_option_contract(self._contract_symbol)
option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

The `add_option_contract` method creates a subscription for a single Option contract and adds it to your user-defined universe. To create a dynamic universe of Option contracts, add an Equity Options universe or an Options Universe Selection model .

Warm Up Contract Prices

If you subscribe to an Option contract with `add_option_contract` , you'll need to wait until the next `Slice` to receive data and trade the contract. To trade the contract in the same time step you subscribe to the contract, set the current price of the contract in a security initializer .

```
self.settings.seed_initial_prices = True
```

Supported Assets

To view the supported assets in the US Equities dataset, see the Data Explorer .

Resolutions

The following table shows the available resolutions and data formats for Equity Option contract subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`				
`SECOND`				
`MINUTE`	✓	✓		
`HOUR`	✓	✓		
`DAILY`	✓	✓		

The default resolution for Option contract subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the

`add_option_contract` method.

```
self.add_option_contract(self._contract_symbol, Resolution.MINUTE)
```

To create custom resolution periods, see Consolidating Data .

Supported Markets

LEAN groups all of the US Equity exchanges under `Market.USA` . You don't need to pass a `market` argument to the `add_option_contract` method because the contract `symbol` already contains the market.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self.add_option_contract(self._contract_symbol, fill_forward=False)
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. Options are already leveraged products, so you can't change their leverage.

Extended Market Hours

By default, your security subscriptions only cover regular trading hours. To subscribe to pre and post-market trading hours for a specific asset, enable the `extended_market_hours` argument when you create the security subscription.

```
self.add_option_contract(self._contract_symbol, extended_market_hours=True)
```

You only receive extended market hours data if you create the subscription with minute, second, or tick resolution. If you create the subscription with daily or hourly resolution, the bars only reflect the regular trading hours.

To view the schedule of regular and extended market hours, see Market Hours .

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . By default, LEAN doesn't adjust Equity Options data for splits and dividends of their underlying. If you change the data normalization mode, it won't change the outcome.

If you hold an Option contract when a corporate action occurs for the underlying Equity, LEAN automatically closes your position.

Remove Subscriptions

To remove a contract subscription that you created with `add_option_contract` , call the `remove_option_contract` method. This method is an alias for `remove_security` .

```
self.remove_option_contract(self._contract_symbol)
```

The `remove_option_contract` method cancels your open orders for the contract and liquidates your holdings.

Properties

The `add_option_contract` method returns an `Option` object, which have the following properties:

Helper Methods

The `Option` object provides methods you can use for basic calculations. These methods require the underlying price. To get the `Option` object and

the `Security` object for its underlying in any function, use the Option `symbol` to access the value in the `securities` object.

```
option = self.securities[self._contract_symbol]
underlying = self.securities[self._contract_symbol.underlying]
underlying_price = underlying.price
```

To get the Option payoff , call the `get_pay_off` method.

```
pay_off = option.get_pay_off(underlying_price)
```

To get the Option intrinsic value , call the `get_intrinsic_value` method.

```
intrinsic_value = option.get_intrinsic_value(underlying_price)
```

To get the Option out-of-the-money amount , call the `out_of_the_money_amount` method.

```
otm_amount = option.out_of_the_money_amount(underlying_price)
```

To check whether the Option can be automatic exercised, call the `is_auto_exercised` method.

```
is_auto_exercised = option.is_auto_exercised(underlying_price)
```

Exceptions and Edge Cases

The following sections explain exceptions and edge cases with subscribing to individual Option contracts.

Manually Creating Option Symbol Objects

To subscribe to an Option contract, you need the contract `Symbol` . You can get the contract `Symbol` from the `create_option` or `option_chain` methods. If you use the `create_option` method, you need to provide the details of an existing contract.

```
self._contract_symbol = Symbol.create_option(self._underlying, Market.USA,
    OptionStyle.AMERICAN, OptionRight.CALL, 365, datetime(2022, 6, 17))
```

Default Underlying Subscription Settings

If you subscribe to an Equity Option contract but don't have a subscription to the underlying Equity, LEAN automatically subscribes to the underlying Equity with the following settings:

Setting	Value
Fill forward	Same as the Option contract
Leverage	0
Extended Market Hours	Same as the Option contract
Data Normalization	`DataNormalizationMode.RAW`

In this case, you still need the Equity `Symbol` to subscribe to Equity Option contracts. If you don't have access to it, create it.

```
self._underlying = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
```

Overriding the Initial Implied Volatility Guess

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

Examples

The following examples demonstrate some common practices for requesting individual Equity Option contract data.

Example 1: Covered Call

A cover call consists of a short call and with a lot of the underlying equity. Although it capped the maximum profit if the underlying skyrocketted, it also provide extra credit received while speculating the underlying will rise.

```PY
class EquityOptionExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._chain = pd.DataFrame()
        # Seed the security price to ensure the retrieval of the ATM calls at the initial filtering.
        self.settings.seed_initial_prices = True
        # Set the data normalization mode as raw for option strike-price comparability.
        self.aapl = self.add_equity("AAPL", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Filter an updated option list at market open everyday by a scheduled event.
        self.schedule.on(
            self.date_rules.every_day(self.aapl),
            self.time_rules.at(9, 0),
            self.refresh_option_list
        )

    def refresh_option_list(self) -> None:
        # Get all tradable option contracts for AAPL at the current time for filtering.
        chain = self.option_chain(self.aapl, flatten=True).data_frame
        if chain.empty:
            return
        # Select the calls expires within 30 days and within $5 strike from ATM as leg of the covered call.
        # $5 buffer is given on selecting the ATM call due to price movement.
        expiry_threshold = self.time + timedelta(30)
        self._chain = chain[
            (chain.expiry < expiry_threshold) &
            (chain.right == OptionRight.CALL) &
            (abs(chain.strike - chain.underlyinglastprice) <= 5)
        ]
        for symbol in self._chain.index:
            self.add_option_contract(symbol)

    def on_data(self, slice: Slice) -> None:
        if not self.portfolio.invested and self.aapl in slice.bars and not self._chain.empty:
            # To form a covered call, get the contract closest to ATM and expiry.
            self._chain['abs_moneyness'] = abs(self._chain.strike - slice.bars[self.aapl].price)
            expiry = self._chain.expiry.min()
            contract = self._chain[self._chain.expiry == expiry].sort_values('abs_moneyness').index[0]

            # Covered call involves shorting 1 ATM call and ordering 1 lot of underlying.
            self.market_order(contract, -1)
            self.market_order(self.aapl, self.securities[contract].symbol_properties.contract_multiplier)

    def on_order_event(self, order_event: OrderEvent) -> None:
        # Close AAPL position if the short call is not exercised (OTM).
        # If it is exercised, the underlying will be used to settle the contract automatically.
        if order_event.ticket.order_type == OrderType.OPTION_EXERCISE \
        and not order_event.is_in_the_money:
            self.market_order(self.aapl, -self.securities[order_event.symbol].symbol_properties.contract_multiplier)
```

Example 2: 0-DTE Bull Put Spread

0DTE options often trades with high volume and volatility, providing arbitration opportunities and higher profit margin on spread type trading. In this example, we make use of 0-DTE SPY options to trade bull put spread option strategy.

```
class EquityOptionExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1);
        self.set_end_date(2024, 12, 27)
        self.legs = []
        # Seed the underlying security price to ensure accurate filtering for puts of $5 above/below current market price.
        self.settings.seed_initial_prices = True
        # Set the data normalization mode as raw for option strike-price comparability.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Filter an updated option list at market open everyday by a scheduled event.
        self.schedule.on(self.date_rules.every_day(self.spy), self.time_rules.before_market_open(self.spy, 10), self.refresh_option_list)

        # Use a scheduled event to close all positions before market close.
        self.schedule.on(self.date_rules.every_day(self.spy), self.time_rules.before_market_close(self.spy, 1), self.exit_position)

    def refresh_option_list(self) -> None:
        # Get all tradable option contracts for SPY at the current time for filtering.
        contracts = self.option_chain(self.spy)
        # Select the 0-DTE puts by setting expiry within 1 day.
        filtered_contracts = [x for x in contracts
            if x.expiry < self.time + timedelta(1) and x.right == OptionRight.PUT]
        # Ensure at least 2 contracts available to form a put spread.
        if len(filtered_contracts) < 2:
            self.legs = []
            return

        # To form a put spread, select and subscribe to put contracts $5 above and below from the current underlying price.
        underlying_price = self.securities[self.spy].price
        itm_put = sorted(filtered_contracts, key=lambda x: abs(x.strike - underlying_price - 5))[0]
        otm_put = sorted(filtered_contracts, key=lambda x: abs(x.strike - underlying_price + 5))[0]
        def get_leg(contract, quantity):
            self.add_option_contract(contract.symbol)
            return Leg.create(contract.symbol, quantity)
        self.legs = [get_leg(itm_put, -1), get_leg(otm_put, 1)]

    def on_data(self, slice: Slice) -> None:
        # To avoid over-trading, limit the position opening to before 3pm.
        # To ensure the put spread formed correctly, make sure at least 2 contracts selected.
        if not self.portfolio.invested and self.time.hour < 15 and len(self.legs) == 2:
            # A bull put spread involves buying a lower-strike put and selling a high-strike put
            [self.market_order(leg.symbol, leg.quantity) for leg in self.legs]

    def exit_position(self) -> None:
        # Exit all positions by removing the contract before market close to avoid option assignment and overnight risk.
        [self.remove_option_contract(leg.symbol, f'Exit position') for leg in self.legs]
        self.legs = []
```

Example 3: Select Option Contracts by Greeks

The following algorithm creates a weekly Scheduled Event that adds a universe of contracts with a Delta >= 0.99. This universe is useful for creating a hedge replicate portfolio for arbitrage.

```
class EquityOptionExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the underlying asset with raw data to enable Option strike-price comparability.
        self._underlying = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Add a Scheduled Event to select the Option contracts to trade each week.
        self.schedule.on(
            self.date_rules.week_start(self._underlying),
            self.time_rules.at(7, 30),
            self._add_contracts
        )

    def _add_contracts(self) -> None:
        self._universe = []
        # Get all the tradable Option contracts for SPY at the current time.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if not chain.empty:
            # Add the contracts that have the nearest expiry after 7 days and a delta >= 0.99.
            expiry_threshold = self.time + timedelta(7)
            expiry = chain[chain.expiry >= expiry_threshold].expiry.min()
            self._universe = [
                self.add_option_contract(symbol)
                for symbol in chain[(chain.expiry == expiry) & (chain.delta >= 0.99)].index
            ]
        # Plot the number of contracts in the universe.
        self.plot('Universe', 'Size', len(self._universe))
```

Example 4: Wheel Strategy

The Wheel strategy is a popular trading strategy for Options that enables traders to build a steady flow of income from Equity assets they want to hold for the long term.

```py
class EquityOptionExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 5)
        self.set_end_date(2024, 9, 5)
        self.set_cash(1000000000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Set the data normalization mode as raw for option strike-price comparability.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set OTM threshold for wheel strategy profit margin.
        self.otm_threshold = 0.05

    def on_data(self, slice: Slice) -> None:
        # To use the latest underlying price to filter the option contract, ensure the SPY in the bar data.
        # Open short put contract position only when the last wheel is completed.
        if not self.portfolio.invested and self.spy in slice.bars:
            # Initiate the wheel by shorting a least-OTM put contract that the strike is below the threshold.
            symbol = self.get_target_contract(OptionRight.PUT, slice.bars[self.spy].price * (1 - self.otm_threshold))
            self.set_holdings(symbol, -0.2)
        # Open short call contract position only when the put is assigned (portfolio with the underlying) to close the wheel and
underlying position by the call assignment.
        elif self.portfolio[self.spy].invested and self.spy in slice.bars:
            # Short the corresponding number of a least-OTM call contract that the strike is above the threshold.
            symbol = self.get_target_contract(OptionRight.CALL, slice.bars[self.spy].price * (1 + self.otm_threshold))
            self.market_order(symbol, self.portfolio[self.spy].quantity /
self.securities[self.spy].symbol_properties.contract_multipliers)

    def get_target_contract(self, right: OptionRight, target_price: float) -> Symbol:
        # Get all tradable option contracts for SPY at the current time for filtering.
        contract_symbols = self.option_chain(self.spy)
        if not contract_symbols:
            return
        # Filter for the least-OTM contract that is off by the threshold to form the wheel.
        # Expiry is set to be at least 30 days to earn the time decay, which is highest in the last month.
        expiry = min(x.id.date for x in contract_symbols)
        filtered = [x for x in contract_symbols \
            if x.id.date == expiry \
            and x.id.option_right == right \
            and (x.id.strike_price >= target_price if right == OptionRight.CALL else x.id.strike_price <= target_price)]
        sorted_by_strikes = sorted(filtered, key=lambda x: x.id.strike_price)
        selected = sorted_by_strikes[0] if right == OptionRight.CALL else sorted_by_strikes[-1]
        # Request the selected contract data for trading.
        return self.add_option_contract(selected).symbol
```

For more details, refer to the Wheel Strategy research post .

Example 5: Scan and Update Option Chain Every 5 Minutes

The following example shows how to update the Option chain every five minutes. The `OptionChainManager` class implements the selection logic and manages the contract subscriptions.

```py
class OptionChainFullExample(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(100000)

        self.universe_settings.asynchronous = True
        self.universe_settings.minimum_time_in_universe = timedelta(minutes=0)
        # Seed the security price to ensure the underlying price data is ready at the initial filtering
        self.settings.seed_initial_prices = True
        # Set the data normalization mode as raw for option strike-price comparability
        spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up a OptionChainManager to filter the contracts based on latest data by request
        self._chain_manager = {
            Symbol.create_canonical_option(spy): OptionChainManager(-10, 10, 0, 7)
        }
        # Daily population of the available contracts to ensure the contracts are tradable
        self._populate_option_chain()
        self.schedule.on(
            self.date_rules.every_day(spy), self.time_rules.after_market_open(spy, 1), self._populate_option_chain
        )
        # Filter for contract in every 5 minutes interval through scheduled event
        self.schedule.on(self.date_rules.every_day(spy), self.time_rules.every(timedelta(minutes=5)), self._filter)

    def _populate_option_chain(self):
        # The contract list is updated daily, so we can get it and apply
        # the expiration filter as soon as the market opens.
        for symbol, manager in self._chain_manager.items():
            manager.set_chain(self.option_chain(symbol), self.time)
        self._filter()
```

```
        self._filter()

    def _filter(self):
        for symbol, manager in self._chain_manager.items():
            manager.select(self, symbol)

    def on_data(self, slice: Slice) -> None:
        # Iterate the saved symbol and chain manager to obtain only the contract wanted
        for symbol, _ in self._chain_manager.items():
            chain = slice.option_chains.get(symbol)
            if not chain:
                continue
            expiry = min([x.expiry for x in chain])
            contracts = [
                x for x in chain
                if x.expiry == expiry and x.right == OptionRight.CALL and self.securities[x.symbol].is_tradable
            ]
            if not contracts:
                continue
            atm_call = sorted(contracts, key=lambda x: abs(chain.underlying.price-x.strike))[0]

            if not self.portfolio[atm_call.symbol].invested:
                self.market_order(atm_call.symbol, 1)


class OptionChainManager:
    _chain = []
    _symbols = set([])

    def __init__(self, min_strike, max_strike, min_expiry, max_expiry):
        self._min_strike = min_strike
        self._max_strike = max_strike
        self._min_expiry = min_expiry
        self._max_expiry = max_expiry

    def set_chain(self, chain: OptionChain, time: datetime) -> None:
        # Expiry criteria will not affect intra-day universe filtering, so it is done in a daily basis in higher level
        self._chain = [x for x in chain if self._min_expiry <= (x.expiry - time).days <= self._max_expiry]

    def select(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        if not self._chain:
            return
        if symbol.is_canonical():
            symbol = symbol.underlying

        # Select by strike range from ATM
        strikes = sorted(set(x.strike for x in self._chain))
        spot = algorithm.securities[symbol].price
        atm = sorted(strikes, key=lambda x: abs(spot-x))[0]
        index = strikes.index(atm)
        min_strike = strikes[max(0, index + self._min_strike)]
        max_strike = strikes[min(len(strikes) - 1, index + self._max_strike)]
        symbols = set(x.symbol for x in self._chain if min_strike <= x.strike <= max_strike)

        # Also remove data subscription on the contracts being filtered out to release computation resources
        for symbol in self._symbols - symbols:
            if algorithm.remove_option_contract(symbol):
                self._symbols.remove(symbol)
        # Request data subscription on the newly selected contracts for trading
        for symbol in symbols - self._symbols:
            self._symbols.add(symbol)
            algorithm.add_option_contract(symbol)
```

Equity Options

Handling Data

Introduction

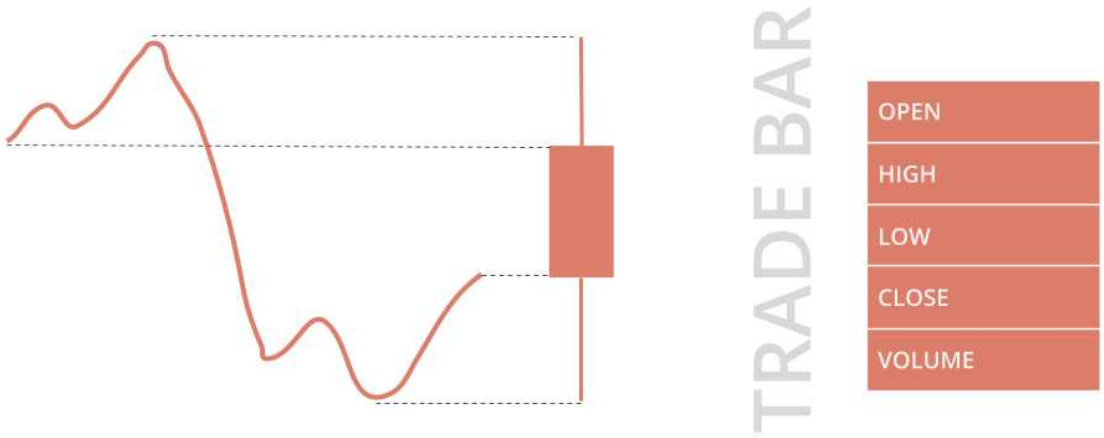
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the contract ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Equity Options data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively trade or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._contract_symbol)   # None if not found
```

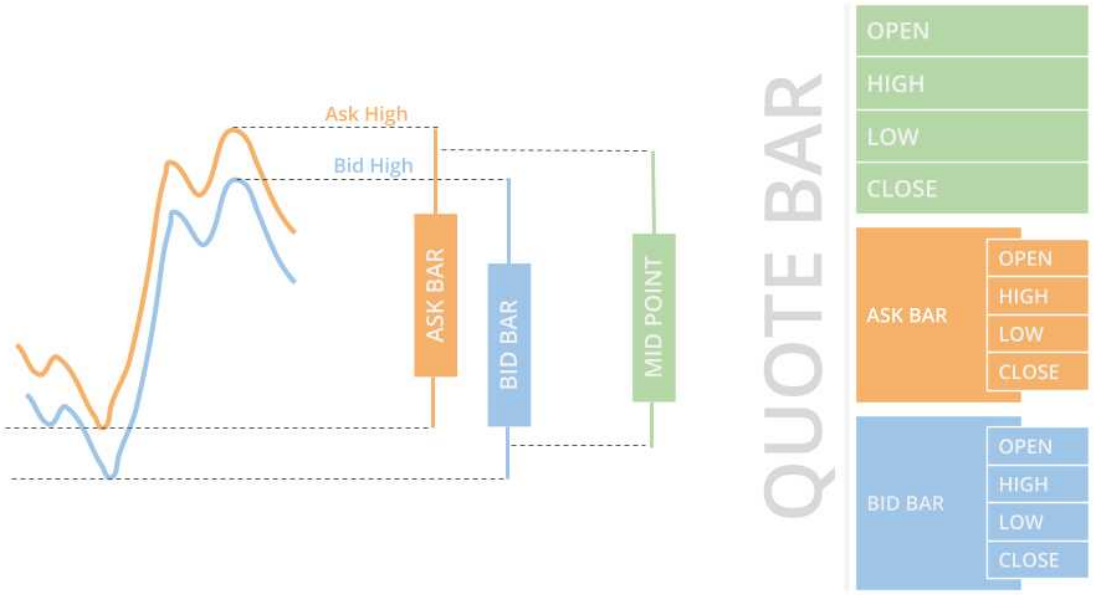
You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively get quotes or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._contract_symbol)   # None if not found
```

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , index the `option_chains` property of the `Slice` with the canonical `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol (None if no OptionChain return)
    chain = slice.option_chains.get(self._contract_symbol.Canonical)
    if chain:
        # Get all contracts if the OptionChain contains any member
        contracts = chain.contracts
```

You can also loop through the `option_chains` property to get each `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-OptionChain key-value pairs
    for canonical_symbol, chain in slice.option_chains.items():
        contracts = chain.contracts
```

`OptionChain` objects have the following properties:

Option Contracts

`OptionContract` objects represent the data of a single Option contract in the market. To get the Option contracts in the `Slice` , use the `contracts` property of the `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            price = contract.price
```

Greeks and Implied Volatility

To get the Greeks and implied volatility of an Option contract, use the `greeks` and `implied_volatility` members.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get greeks data of the selected contract
            delta = contract.greeks.delta
            iv = contract.implied_volatility
```

LEAN only calculates Greeks and implied volatility when you request them because they are expensive operations. If you invoke the `greeks` property, the Greeks aren't calculated. However, if you invoke the `greeks.delta` , LEAN calculates the delta. To avoid unecessary computation in your algorithm, only request the Greeks and implied volatility when you need them. For more information about the Greeks and implied volatility, see Options Pricing .

Open Interest

Open interest is the number of outstanding contracts that haven't been settled. It provides a measure of investor interest and the market liquidity, so it's a popular metric to use for contract selection. Open interest is calculated once per day. To get the latest open interest value, use the `open_interest` property of the `Option` or `option_contract` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the option_chains using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get the open interest of the selected contracts
            open_interest = contract.open_interest
```

Properties

`OptionContract` objects have the following properties:

Examples

Example 1: Get Mid Price For Individual Contracts

This example shows how to handle `QuoteBar` data for shortlisted Equity Option contracts to calculate mid price using bid close and ask close data,

while filter individual option contracts and request data using `self.option_chain` method for the contracts that expires within the current week. Using mid price, we can examine the market fair value of the Option and compare with model theoretical price.

```
class EquityOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)

        self.contracts = []

        # Seed the price with last known price to ensure the underlying price data is available on initial option contract filtering.
        self.settings.seed_initial_prices = True
        # Subscribe to underlying data for ATM calculation using the update underlying price.
        # Set data normalization mode to raw is required to ensure strike price and underlying price is comparable.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Update the tradable contracts daily before market open since the option contract list provider populate them daily.
        self.schedule.on(
            self.date_rules.every_day(self.spy),
            self.time_rules.at(9, 0),
            self.update_contracts
        )

    def update_contracts(self) -> None:
        # Get all contracts that expiring this week to trade with, subscribe to data for trading need.
        contracts = self.option_chain(self.spy)
        self.contracts = [self.add_option_contract(x).symbol for x in contracts
            if x.id.date < Expiry.end_of_week(self.time)]

    def on_data(self, slice: Slice) -> None:
        # Only focus on filtered list of option contracts to trade.
        for contract in self.contracts:
            # Mid price can only be calculated when quote bar data is available.
            quote = slice.quote_bars.get(contract)
            if quote and quote.bid is not None and quote.ask is not None:
                # Mid price = average of bid close price and ask close price.
                mid_price = (quote.bid.close + quote.ask.close) * 0.5
```

Example 2: Get Mid Price For Universe

This example shows how to handle `QuoteBar` data for shortlisted Equity Option contracts to calculate mid price using bid close and ask close data, while request data through universe selection function using `set_filter` method for the contracts that expires within the current week. Using mid price, we can examine the market fair value of the Option and compare with model theoretical price.

```
class EquityOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)

        # Seed the price with last known price to ensure the underlying price data is available on initial option contract filtering.
        self.settings.seed_initial_prices = True
        # Subscribe to underlying data for ATM calculation using the update underlying price.
        # Set data normalization mode to raw is required to ensure strike price and underlying price is comparable.
        spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Subscribe to SPY option data.
        option = self.add_option(spy)
        self._symbol = option.symbol
        # We wish to only trade the contracts expiring within the same week since they have the highest volume.
        option.set_filter(lambda u: u.contracts(lambda x: [s for s in x if s.id.date <= Expiry.end_of_week(self.time)]))

    def on_data(self, slice: Slice) -> None:
        # Only want to obtain the option chain of the selected symbol.
        chain = slice.option_chains.get(self._symbol)
        if not chain:
            return

        for contract in chain:
            # Mid price = average of bid close price and ask close price
            mid_price = (contract.bid_price + contract.ask_price) * 0.5
```

Example 3: Get Instant Delta

The option greeks change rapidly, so we need to obtain the instant greeks to accurately calculate the hedge size for arbitration or replication portfolio. You can call the `Greeks` property from the `OptionChain` data to access various greeks. In this example, we will demonstrate how to obtain the contract with delta closest to 0.4 among all call contracts expiring the same week.

```py
class EquityOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 5)
        self.set_end_date(2024, 9, 5)

        # Seed the price with last known price to ensure the underlying price data is available on initial option contract filtering.
        self.settings.seed_initial_prices = True
        # Subscribe to underlying data for ATM calculation using the update underlying price.
        # Set data normalization mode to raw is required to ensure strike price and underlying price is comparable.
        spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Subscribe to SPY option data.
        option = self.add_option(spy)
        self._symbol = option.symbol
        # We wish to only trade the contracts expiring within the same week since they have the highest volume.
        # 0.4 Delta will only appear in call contracts.
        option.set_filter(lambda u: u.delta(0.2, 0.6).contracts(lambda x: [s for s in x if s.id.date <= Expiry.end_of_week(self.time)]))

    def on_data(self, slice: Slice) -> None:
        # Only want to obtain the option chain of the selected symbol.
        chain = slice.option_chains.get(self._symbol)
        if not chain:
            return

        # Get the contract with Delta closest to 0.4 to trade.
        # The arbitary delta criterion might be set due to hedging need or risk adjustment.
        selected = sorted(chain, key=lambda x: abs(x.greeks.delta - 0.4))[0]
```

Example 4: Delta-Hedged Short Straddle

This example demonstrates a delta-hedged short straddle strategy that sells an at-the-money (ATM) straddle with 7-30 days to expiration and continuously neutralizes the portfolio delta using the underlying shares. The `StraddleSelector` helper class picks the farthest expiry within the DTE range and the strike closest to the spot price. On each slice , the combined Option delta is accumulated from the `OptionChain` and the net portfolio delta is computed by adding the underlying position. The `DeltaHedger` helper enforces a minimum share threshold and a rehedge band to avoid over-trading, and applies a time delay between successive hedges. A Scheduled Event closes the straddle on expiration day and assignment events are handled to keep the underlying position clean.

```py
class DeltaHedgedStraddleAlgo(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)
        self.settings.seed_initial_prices = True
        self._spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW)
        # Initialize the straddle selector with 7-30 day DTE range.
        self._straddle_selector = StraddleSelector(min_dte=7, max_dte=30)
        # Initialize the delta hedger with 30 share minimum, 20 share rehedge band, and 4 hour delay.
        self._hedger = DeltaHedger(30, 20, timedelta(hours=4))
        self._canonical_option = Symbol.create_canonical_option(self._spy)
        # Set the straddle position size to 50% of portfolio value.
        self._straddle_weight = 0.5
        self._short_straddle = None
        self._exit_day = None
        # Schedule the rebalance method to run 30 minutes after market open.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.after_market_open(self._spy, 30),
            self._rebalance
        )
        # Schedule the close expiring method to run 60 minutes before market close.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.before_market_close(self._spy, 60),
            self._close_expiring
        )
        self.set_warm_up(timedelta(45))

    def on_data(self, data):
        if self.is_warming_up or not data.bars:
            return
        chain = data.option_chains.get(self._canonical_option)
        if not chain or not self._short_straddle:
            return
        # Accumulate the delta from all option legs in the straddle.
        for leg in self._short_straddle.option_legs:
            contract = chain.contracts.get(leg.symbol)
            if contract:
                self._delta += contract.greeks.delta
        # Calculate the net portfolio delta including underlying and options.
        net_delta = self._hedger.compute_net_delta(self, self._short_straddle.option_legs, self._spy.holdings.quantity, self._delta)
        # Reset the delta accumulator for the next iteration.
        self._delta = 0
        # Exit if it's not time to hedge or net delta is zero.
```

```
        if not (self._hedger.should_hedge(self.time) and net_delta):
            return
        # Calculate the hedge quantity needed to neutralize delta.
        hedge_qty = self._hedger.get_quantity(net_delta)
        if hedge_qty:
            self.market_order(self._spy, hedge_qty, tag=f"Daily delta hedge ({net_delta:.2f} share delta)")
            # Record the hedge operation with timestamp and delta.
            self._hedger.record_hedge(self.time, net_delta)
            # Update the net delta after the hedge.
            net_delta += hedge_qty
        self.plot("Portfolio Delta", "Delta", net_delta)

    def _rebalance(self):
        if self.is_warming_up or self.portfolio.invested:
            return
        chain = self.option_chain(self._spy)
        if not chain:
            return
        # Use the straddle selector to find the best ATM straddle.
        result = self._straddle_selector.select(chain, self._spy.price, self.time.date())
        if not result:
            return
        self._delta, expiry, strike = result
        self._short_straddle = OptionStrategies.short_straddle(self._canonical_option, strike, expiry)
        # Calculate the number of contracts based on portfolio value and weight.
        qty = int(self.portfolio.total_portfolio_value * self._straddle_weight / (self._spy.price * 100.0))
        if qty:
            self.buy(self._short_straddle, qty, tag=f"Sell ATM straddle exp {expiry.date()}")
        self._hedger.reset()
        self._exit_day = expiry.date()

    def _close_expiring(self):
        if not self._short_straddle or not self._exit_day or self.time.date() < self._exit_day:
            return
        # Liquidate all positions before expiration.
        self.liquidate(tag="Expiry hedge liquidation")

    def on_order_event(self, order_event):
        # Handle option assignment events.
        if order_event.status == OrderStatus.FILLED and order_event.is_assignment:
            # Liquidate all option legs of the straddle.
            for leg in self._short_straddle.option_legs:
                self.liquidate(leg.symbol, tag="Assignment liquidation")
            # Determine the direction based on whether it's a call or put assignment.
            direction = 1 if self.securities[order_event.symbol].right == OptionRight.CALL else -1
            self.market_order(self._spy, -self._spy.holdings.quantity + order_event.quantity * direction * 100)
            self._short_straddle = None


class StraddleSelector:

    def __init__(self, min_dte, max_dte):
        # Store the minimum days to expiration for contract selection.
        self._min_dte = min_dte
        # Store the maximum days to expiration for contract selection.
        self._max_dte = max_dte

    def select(self, chain, spot_price, current_date):
        # Filter contracts to those within the specified DTE range.
        contracts = [
            contract for contract in chain
            if self._min_dte <= (contract.expiry.date() - current_date).days <= self._max_dte
        ]
        if not contracts:
            return
        # Select the farthest expiration date from the filtered contracts.
        expiry = max(contract.expiry for contract in contracts)
        # Filter to only contracts with the selected expiration.
        contracts = [contract for contract in contracts if contract.expiry == expiry]
        # Find the strike price closest to the current spot price.
        strike = min(contracts, key=lambda c: abs(c.strike - spot_price)).strike
        # Filter to only contracts with the selected strike.
        contracts = [contract for contract in contracts if contract.strike == strike]
        # Ensure we have both call and put for a straddle.
        if len(contracts) < 2:
            return
        # Return the combined delta, expiry, and strike for the straddle.
        return sum(contract.greeks.delta for contract in contracts), expiry, strike


class DeltaHedger:

    def __init__(self, min_shares, rehedge_band, hedge_delay):
        # Store the minimum number of shares required to execute a hedge.
        self._min_shares = min_shares
        # Store the rehedge band threshold to prevent excessive hedging.
        self._rehedge_band = rehedge_band
        # Store the time delay between hedging operations.
        self._hedge_delay = hedge_delay
        self.reset()

    def reset(self):
        self._next_hedge_time = datetime.min
        self._last_hedged_delta = None
```

```
    def should_hedge(self, current_time):
        # Check if enough time has passed since the last hedge.
        return current_time >= self._next_hedge_time

    def compute_net_delta(self, algorithm, legs, underlying_quantity, contract_delta):
        # Calculate the net portfolio delta by combining underlying and option positions.
        return underlying_quantity + sum(
            algorithm.securities[leg.symbol].holdings.quantity * contract_delta * 100
            for leg in legs
        )

    def get_quantity(self, net_delta):
        if (
            self._last_hedged_delta is not None
            and abs(net_delta - self._last_hedged_delta) < self._rehedge_band
            and abs(net_delta) < self._min_shares + self._rehedge_band
        ):
            return
        # Calculate the required hedge quantity to neutralize the net delta.
        hedge_qty = int(round(-net_delta))
        if abs(hedge_qty) < self._min_shares:
            return
        # Return the calculated hedge quantity.
        return hedge_qty

    def record_hedge(self, current_time, net_delta):
        # Record the delta value after this hedge operation.
        self._last_hedged_delta = net_delta
        # Set the next allowed hedge time based on the configured delay.
        self._next_hedge_time = current_time + self._hedge_delay
```

Equity Options

Greeks and Implied Volatility

The Greeks are measures that describe the sensitivity of an Option's price to various factors like underlying price changes (Delta), time decay (Theta), volatility (Vega), and interest rates (Rho), while Implied Volatility (IV) represents the market's expectation of the underlying asset's volatility over the life of the Option.

Key Concepts

Concepts on Greeks, IV, and IV modeling

Indicators

Implementation of IV and Greeks as LEAN indicators

See Also

Option Pricing
Risk Free Interest Rate
Dividend Yield

Greeks and Implied Volatility

Key Concepts

Introduction

LEAN provides multiple ways for you to get the Greeks and implied volatility (IV) of Option contracts. There is a daily, pre-calculated dataset based on the end of the previous trading day. There is also a stream of values that are calculated on-the-fly as your algorithm runs.

Greeks on Universes

The Greeks and IV values in the filter function are the daily, pre-calculated values based on the end of the previous trading day.

```py
def initialize(self):
    option = self.add_option("SPY")
    option.set_filter(lambda universe: universe.delta(0.3, 0.7).expiration(0,7))
```

To calculate the values, we use our implementation of the forward tree pricing model, which accounts for the interest rate and dividend payments.

You can't customize the Greeks and IV values that the filter function receives. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

Greek History

The Greeks and IV values that you get from a history request of the Option universe are the daily, pre-calculated values based on the end of the previous trading day.

```py
class OptionHistoryAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_option('SPY')
        # DataFrame example:
        history_df = self.history(option.symbol, 5, flatten=True)
        # OptionUniverse objects example:
        history = self.history[OptionUniverse](option.symbol, 5)
        for chain in history:
            end_time = chain.end_time
            filtered_contracts = [c for c in chain if c.greeks.delta > 0.3]
            for contract in filtered_contracts:
                symbol = contract.symbol
                expiry = contract.id.date
                strike = contract.id.strike_price
                price = contract.close
                iv = contract.implied_volatility
```

You can't customize the Greeks and IV values that you get from a history request. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

Greeks on Data Events

The Greeks and IV values that you get from the current Slice are from the price model calculations. These values aren't the same as the values you get in the universe filter function.

```
class OptionDataEventsAlgorithm(QCAlgorithm):

    def initialize(self):
        option = self.add_option("GOOG")
        option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()

    def on_data(self, data):
        for canonical_symbol, chain in data.option_chains.items():
            for contract in chain:
                iv = contract.implied_volatility
                delta = contract.greeks.delta
```

To view all the models LEAN supports, see Supported Models .

Greek Indicators

The Greeks and IV values that you get from the indicators are calculated in the same way as the values you get in the filter function, but they can reflect intraday prices instead of just the daily closing prices.

For Equity Options, the `indicator_history` uses the `SCALED_RAW` data normalization mode when requesting historical prices.

Examples

For detailed examples on using Option Greeks and Implied Volatility indicators in algorithm, you may refer to the Greeks Indicators page examples.

Greeks and Implied Volatility

Indicators

Introduction

Option indicators enable you to calculate daily and intraday values for the Greeks and implied volatility of Option contracts. This page explains the most popular Option indicators and how to use them with LEAN.

Parameters

The following table describes the arguments that the automatic Option indicators methods accept:

Argument	Data Type	Description	Default Value
`symbol`	`Symbol`	The contract to use when calculating the indicator values.	
`mirror_option`	`Symbol`	The mirror contract to use in parity type calculations.	`None`
`risk_free_rate`	`float`	The risk-free interest rate. If you don't provide a value, the default value is the US primary credit rate from the Interest Rate Provider Model .	`None`
`dividend_yield`	`float`	The dividend yield rate. If you don't provide a value, the default value comes from the Dividend Yield Provider Model .	`None`
`option_model`	`OptionPricingModelType`	The Option pricing model that's used to calculate the Greeks. If you don't provide a value, the default value is `OptionPricingModelType.BinomialCoxRossRubinstein` for Equity Options.	`None`
`resolution`	`Resolution`	The resolution of the indicator data. If you don't provide a value, the default value is the resolution of the subscription you have for the Option contract(s).	`None`

To perform implied volatility (IV) smoothing with a put-call pair, pass one of the contracts as the `symbol` argument and pass the other contract in the pair as the `mirror_option` argument. The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, call the `set_smoothing_function` method of the `ImpliedVolatility` class/property.

Several different Option pricing models are supported to calculate the IV and Greeks. The following table describes the `OptionPricingModelType` enumeration members:

Implied Volatility

Implied volatility, , is the market's expectation for the future volatility of an asset and is implied by the price of the assets's Options contracts. You can't observe it in the market but you can derive it from the price of an Option. For more information about implied volatility, see Implied Volatility .

Automatic Indicators

To create an automatic indicator for implied volatility, call the `QCAlgorithm.iv` method with the Option contract `symbol` object(s).

```
class AutomaticImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            contract1.iv = self.iv(call, put)
            contract2.iv = self.iv(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the ImpliedVolatility indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.iv
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `iv` method, see Parameters and Using IV Indicator .

Manual Indicators

To create a manual indicator for implied volatility, call the `ImpliedVolatility` constructor.

```
class ManualImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset
```

```
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.ImpliedVolatility = ImpliedVolatility(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].ImpliedVolatility
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the ImpliedVolatility indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `ImpliedVolatility` constructor, see Using IV Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the `iv` method or `ImpliedVolatility` constructor and then call the `set_smoothing_function` method of the resulting `ImpliedVolatility` object. The follow table describes the arguments of the custom function:

Argument	Data Type	Description
`iv`	`float`	The IV of the Option contract.
`mirror_iv`	`float`	The IV of the mirror Option contract.

The method must return a `float` as the smoothened IV.

```
def initialize(self):
    option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.PUT, 505, datetime(2014, 6, 27))
    self.add_option_contract(option)

    mirror_option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.CALL, 505, datetime(2014, 6, 27))
    self.add_option_contract(mirror_option)

    self._iv = self.iv(option, mirror_option)
    # Example: The average of the call-put pair.
    self._iv.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

PY

Delta

Delta, , is the rate of change of the Option price with respect to the price of the underlying asset. It measures the first-order sensitivity of the price to a movement in underlying price. For example, an Option delta of 0.4 means that if the underlying asset moves by 1%, then the value of the Option moves by 0.4 1% = 0.4%. For more information about delta, see Delta .

Automatic Indicators

To create an automatic indicator for delta, call the `QCAlgorithm.d` method with the Option contract `symbol` object(s).

```py
class AutomaticDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Delta indicators.
            contract1.d = self.d(call, put)
            contract2.d = self.d(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Delta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.d
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `d` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Delta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `d` method, see Using D Indicator .

Manual Indicators

To create a manual indicator for delta, call the `Delta` constructor.

```py
class ManualDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Delta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Delta = Delta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Delta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Delta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Delta` constructor, see Using D Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
delta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Gamma

Gamma, , is the rate of change of the portfolio's delta with respect to the underlying asset's price. It represents the second-order sensitivity of the Option to a movement in the underlying asset's price. For more information about Gamma, see Gamma .

Automatic Indicators

To create an automatic indicator for gamma, call the `QCAlgorithm.g` method with the Option contract `symbol` object(s).

```py
class AutomaticGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Gamma indicators.
            contract1.g = self.g(call, put)
            contract2.g = self.g(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Gamma indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.g
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `g` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Gamma. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `g` method, see Using G Indicator .

Manual Indicators

To create a manual indicator for gamma, call the `Gamma` constructor.

```py
class ManualGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Gamma indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Gamma = Gamma(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Gamma
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Gamma indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Gamma` constructor, see Using G Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
gamma_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Vega

Vega, , is the rate of change in the value of the Option with respect to the volatility of the underlying asset. For more information about vega, see Vega .

Automatic Indicators

To create an automatic indicator for vega, call the `QCAlgorithm.v` method with the Option contract `symbol` object(s).

```
class AutomaticVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Vega indicators.
            contract1.v = self.v(call, put)
            contract2.v = self.v(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Vega indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.v
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `v` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Vega. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `v` method, see Using V Indicator .

Manual Indicators

To create a manual indicator for vega, call the Vega constructor.

PY

```
class ManualVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Vega indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Vega = Vega(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Vega
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Vega indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Vega` constructor, see Using V Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
vega_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Theta

Theta, , is the rate of change of the value of the Option with respect to the passage of time. It is also known to as the time decay of an Option. For more information about theta, see Theta .

Automatic Indicators

To create an automatic indicator for theta, call the `QCAlgorithm.t` method with the Option contract `symbol` object(s).

```py
class AutomaticThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Theta indicators.
            contract1.t = self.t(call, put)
            contract2.t = self.t(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Theta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.t
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `t` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating theta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `t` method, see Using T Indicator .

Manual Indicators

To create a manual indicator for theta, call the `Theta` constructor.

```py
class ManualThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Theta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Theta = Theta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Theta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Theta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Theta` constructor, see Using T Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
theta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Rho

Rho, , is the rate of change of the value of a derivative with respect to the interest rate . It is usually small and not a big issue in practice unless the Option is deep in-the-money and has a long horizon. In this case, the interest rate matters because you need to discount a larger cash flow over a longer horizon. For more information about rho, see Rho .

Automatic Indicators

To create an automatic indicator for rho, call the `QCAlgorithm.r` method with the Option contract `symbol` object(s).

```py
class AutomaticRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Rho indicators.
            contract1.r = self.r(call, put)
            contract2.r = self.r(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Rho indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.r
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `r` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating rho If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `r` method, see Using R Indicator .

Manual Indicators

To create a manual indicator for rho, call the Rho constructor.

```py
class ManualRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_option_contract(call)
            contract2 = self.add_option_contract(put)

            # Create and save the automatic Rho indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Rho = Rho(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Rho
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Rho indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Rho` constructor, see Using R Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
rho_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Examples

Demonstration Algorithms
OptionIndicatorsRegressionAlgorithm.py Python OptionIndicatorsMirrorContractsRegressionAlgorithm.py Python

Equity Options

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the Equity Option market.

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

Crypto

A Cryptocurrency is a digital currency that's secured by cryptography, which makes it nearly impossible to counterfeit or double-spend.

Requesting Data

Handling Data

Holdings

Market Hours

See Also

BasicTemplateCryptoAlgorithm.py
BasicTemplateCryptoAlgorithm.cs

Crypto

Requesting Data

Introduction

Request Crypto data in your algorithm to receive a feed of asset prices in the `on_data` method. For more information about the specific datasets we use for backtests, see the CoinAPI datasets . To trade Crypto live, you can use the QuantConnect data provider .

Create Subscriptions

To create a Crypto subscription, in the `initialize` method, call the `add_crypto` method. The `add_crypto` method returns a `Crypto` object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_crypto("BTCUSD").symbol
```

The `add_crypto` method creates a subscription for a single Crypto asset and adds it to your user-defined universe. To create a dynamic universe of Crypto assets, add a Crypto universe .

To view the supported assets, see the CoinAPI datasets .

Fungible Assets

Fungible assets are assets that are interchangeable with each other. For example, every Bitcoin is effectively identical to every other Bitcoin. In contrast, non-fungible tokens (NFTs) are non-fungible because each one is unique. An example of an NFT is the Bored Ape Yacht Club collection. LEAN doesn't currently support trading NFTs.

Resolutions

The following table shows the available resolutions and data formats for Crypto subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`			✓	✓
`SECOND`	✓	✓		
`MINUTE`	✓	✓		
`HOUR`	✓	✓		
`DAILY`	✓	✓		

The default resolution for Crypto subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_crypto` method.

```
self._symbol = self.add_crypto("BTCUSD", Resolution.DAILY).symbol
```

To create custom resolution periods, see Consolidating Data .

Supported Markets

The following `Market` enumeration members are available for Crypto:

To set the market for a security, pass a `market` argument to the `add_crypto` method.

```
self._symbol = self.add_crypto("BTCUSD", market=Market.COINBASE).symbol
```

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self._symbol = self.add_crypto("BTCUSD", fill_forward=False).symbol
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. The amount of leverage available to you depends on the brokerage you use. To change the amount of leverage you can use for a security, pass a `leverage` argument to the `add_crypto` method.

```
self._symbol = self.add_crypto("BTCUSD", leverage=3).symbol
```

For more information about the leverage each brokerage provides, see Brokerages .

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Properties

The `add_crypto` method returns a `Crypto` object, which have the following properties:

Examples

The following examples demonstrate some common practices for requesting Crypto data.

Example 1: Initialize Binance Algorithms

The following algorithm requests data for the BTC/USDT and ETH/USDT Crypto pairs on Binance. The Binance brokerage model provides realistic fee and fill modeling for the backtest. Since the quote currency of both pairs is USDT, set the account currency to USDT instead of USD. Otherwise, some order will fail since the brokerage doesn't automatically convert USD to USDT for you.

```
class CryptoExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the brokerage and account type for accurate fee and margin behavior.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.CASH)

        # In some Crypto brokerages, USD is not a valid currency to trade.
        # Therefore, set the account currency to USDT and add the starting cash.
        self.set_account_currency("USDT", 100000)

        # Subscribe to the Crypto pairs.
        # You don't need to specify the market because the Binance brokerage model already does that.
        self.add_crypto("BTCUSDT", market=Market.BINANCE)
        self.add_crypto("ETHUSDT", market=Market.BINANCE)
```

Crypto

Handling Data

Introduction

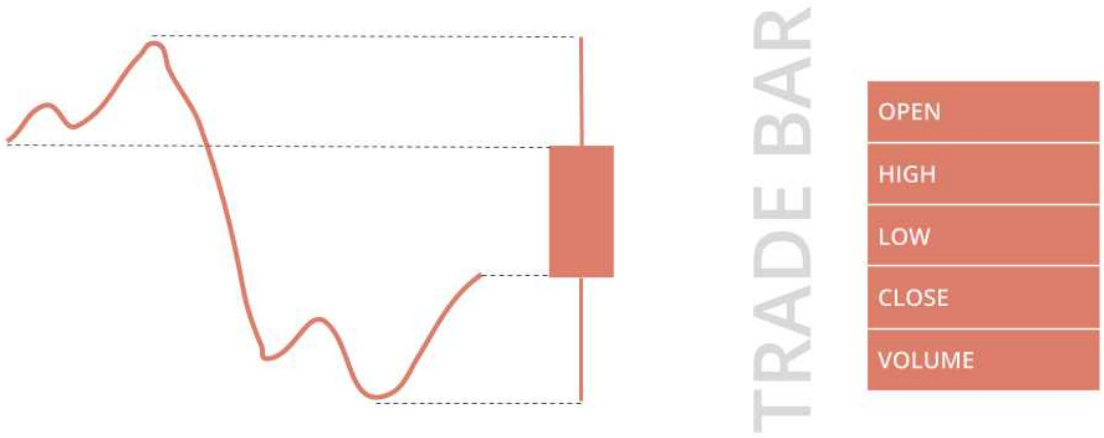
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the security ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Crypto data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the security `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._symbol)   # None if not found
```

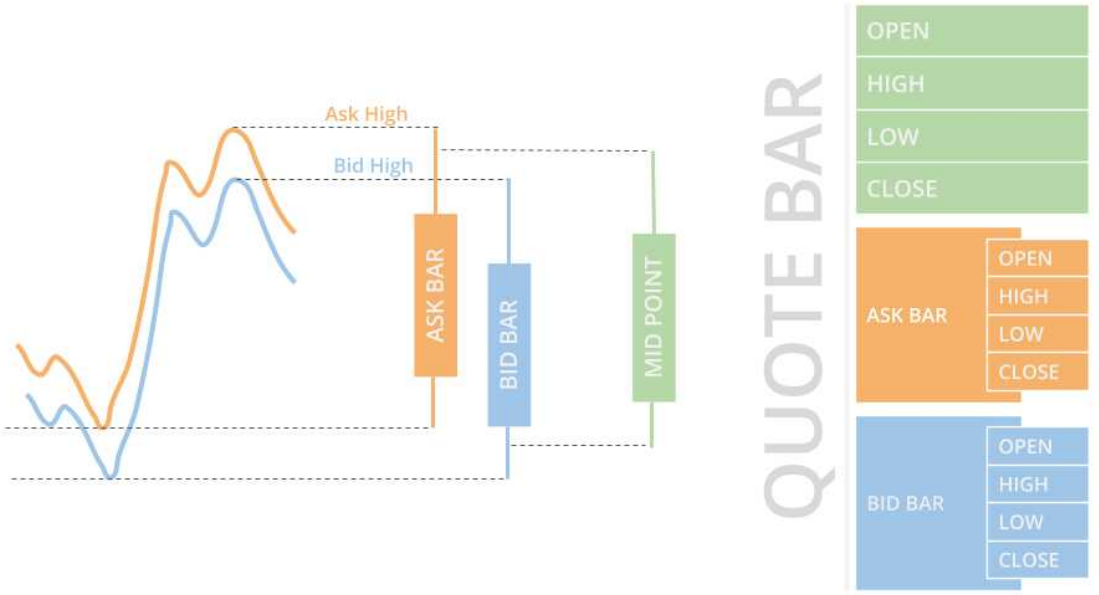
You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the security `symbol` . If the security doesn't actively get quotes or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._symbol)   # None if not found
```

PY

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

PY

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the security. A quote tick is an offer to buy or sell the security at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Crypto data.

Example 1: Dollar Cost Average BTC

Dollar cost averaging (DCA) is where you consistently invest a fixed dollar amount into an asset on a regular basis (for example, monthly), regardless of the asset's price. It can reduce the volatility in your PnL due to slowly increasing the position size over time. The following algorithm demonstrates a DCA investment into BTC. It buys $3,000 USD worth of BTC every midnight for 30 consecutive days.

```
class CryptoExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set a day count variable for counting the days of the DCA trade.
        self._day_count = 0
        # Set the brokerage and account type to match your brokerage environment for accurate fee and margin behavior.
        self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.CASH)
        # For daily DCA purchases, subscribe to daily asset data.
        self._symbol = self.add_crypto("BTCUSD", Resolution.DAILY, Market.BITFINEX).symbol

    def on_data(self, slice: Slice) -> None:
        # If you haven't invested for 30 consecutive days yet, continue buying.
        if self._symbol in slice.bars and self._day_count < 30:
            # Increase the day count.
            self._day_count += 1
            # Calculate the order size for $3,000 USD using the current price.
            quantity = 3_000 / slice.bars[self._symbol].close
            # Round down to observe the lot size.
            lot_size = self.securities[self._symbol].symbol_properties.lot_size
            quantity = int(quantity / lot_size) * lot_size
            if quantity:
                self.market_order(self._symbol, quantity)
```

Example 2: Inter-Market Spread

There is always a small discrepancy in the price of the same Crypto pair trading on different exchanges. Although you can't currently live trade on algorithm with multiple brokerages, you can study the cointegration pattern and implement 2 live nodes to capture the arbitrage opportunity. The following algorithm demonstrates how to obtain the spread between the BTCUSD pair on Kraken and on Coinbase:

PY

```
class CryptoExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Subscribe to BTC/USD on 2 different exchanges.
        self.kraken_btc = self.add_crypto("BTCUSD", market=Market.KRAKEN).symbol
        self.coinbase_btc = self.add_crypto("BTCUSD", market=Market.COINBASE).symbol

    def on_data(self, slice: Slice) -> None:
        # Only calculate the spread if the prices on both exchanges are in the current Slice.
        if self.kraken_btc in slice.bars and self.coinbase_btc in slice.bars:
            # Calculate the spread between the 2 exchanges, making sure the comparison is always in the same direction.
            spread = slice.bars[self.kraken_btc].close - slice.bars[self.coinbase_btc].close
```

Crypto

Holdings

Introduction

Crypto holdings require special consideration because not all brokerages track your positions in each currency pair.

Cash vs Margin Brokerage Accounts

Some of the Crypto brokerages integrated into QuantConnect support cash and margin accounts while some only support cash accounts. If you want to short Crypto assets or use leverage, you need a margin account. Follow these steps to view which account types each brokerage supports:

1. Open the brokerage guides .
2. Click a Crypto brokerage.
3. Scroll down to the Account Types section.

Virtual Pairs

All fiat and Crypto currencies are individual assets. When you buy a pair like BTCUSD, you trade USD for BTC. In this case, LEAN removes some USD from your portfolio cashbook and adds some BTC. The virtual pair BTCUSD represents your position in that trade, but the virtual pair doesn't actually exist. It simply represents an open trade. If you call the Liquidate method with the `symbol` of a virtual pair, the method only liquidates the quantity in your virtual pair. For more information about liquidating Crypto postitions, see Crypto Trades .

Access Cypto Holdings

The `cash_book` stores the quantity of your Crypto holdings. To access your Crypto holdings, index the `cash_book` with the currency ticker. The values of the `CashBook` dictionary are `Cash` objects, which have the following properties:

```
btc_quantity = self.portfolio.cash_book['BTC'].amount
btc_value = self.portfolio.cash_book['BTC'].value_in_account_currency
```

To access the virtual pair of your Crypto trades, index the `portfolio` object with the pair `symbol` .

```
security_holding  = self.portfolio[self._btc_usd_symbol]
```

Stateful Redeployments

When you make a trade inside of a running algorithm, LEAN tracks the virtual position state for you, but it won't survive between deployments. Some brokerages save your virtual pairs, so you can load them into your algorithm when you stop and redeploy it. Depending on the brokerage you select, you may be able to manually add virtual pairs when you deploy live algorithms with the LEAN CLI or cloud deployment wizard .

Examples

The following examples demonstrate some common practices for crypto holdings.

Example 1: ETH-BTC Proxy Trading

The following algorithm trades trend-following of the ETHBTC crypto pair using a 20-day EMA indicator. To reduce friction (e.g. slippage), we trade the more liquid and popular BTCUSDT and ETHUSDT pair instead. For example, if ETHBTC is above EMA (uptrend), we sell all BTCUSDT holdings and buy ETHUSDT with the same USDT value, going flat on BTC while holding ETH.

PY

```
class CryptoHoldingsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_account_currency("USDT", 1000000)


        # We would like to trade the EMA cross between 2 popular cryptos: BTC & ETH,
        # so we request ETHBTC data to find trading opportunities.
        self.ethbtc = self.add_crypto("ETHBTC", Resolution.DAILY, market=Market.COINBASE).symbol
        # Trade through BTCUSDT & ETHUSDT, though, since stable coin trades have lower friction costs and higher liquidity.
        btcusdt = self.add_crypto("BTCUSDT", Resolution.DAILY, market=Market.COINBASE)
        ethusdt = self.add_crypto("ETHUSDT", Resolution.DAILY, market=Market.COINBASE)
        self.btcusdt = btcusdt.symbol
        self.ethusdt = ethusdt.symbol

        # Simulate an account with various crypto cash through holdings.
        # Set the cash book directly with the crypto amounts
        btcusdt.holdings.set_holdings(btcusdt.price, 5)
        btcusdt.base_currency.add_amount(5)
        ethusdt.holdings.set_holdings(ethusdt.price, 22.5)
        ethusdt.base_currency.add_amount(22.5)

        # Add automatic updating of the EMA indicator for trend trade signal emission.
        self._ema = self.ema(self.ethbtc, 20, Resolution.DAILY)
        # Warm up the indicator for its readiness usage immediately.
        self.warm_up_indicator(self.ethbtc, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.ethbtc)
        btc = slice.bars.get(self.btcusdt)
        eth = slice.bars.get(self.ethusdt)

        if bar and self._ema.is_ready and btc and eth:
            ema = self._ema.current.value
            btc_holdings = self.portfolio.cash_book["BTC"].amount
            eth_holdings = self.portfolio.cash_book["ETH"].amount

            # ETHBTC's current price is higher than EMA, suggesting an uptrend.
            if bar.close > ema and not self.portfolio[self.btcusdt].is_short:
                # To follow the up trend of ETHBTC, sell ALL BTCUSDT and buy ETHUSDT with the same value.
                if btc_holdings > 0:
                    btc_value = btc_holdings * btc.close
                    # Sell all BTC
                    self.market_order(self.btcusdt, -btc_holdings)
                    # Buy ETH with the same USDT value
                    eth_size = btc_value / eth.close
                    self.market_order(self.ethusdt, eth_size)
            # ETHBTC's current price is below the EMA, suggesting a downtrend.
            elif bar.close < ema and not self.portfolio[self.btcusdt].is_long:
                # To follow the downtrend of ETHBTC, sell ALL ETHUSDT and buy BTCUSDT with the same value.
                if eth_holdings > 0:
                    eth_value = eth_holdings * eth.close
                    # Sell all ETH
                    self.market_order(self.ethusdt, -eth_holdings)
                    # Buy BTC with the same USDT value
                    btc_size = eth_value / btc.close
                    self.market_order(self.btcusdt, btc_size)
```

Crypto

Market Hours

Introduction

The Crypto markets are open 24/7. There are no pre-market hours, post-market hours, holidays, late opens, or early closes.

Asset Classes

Crypto Futures

Crypto Perpetual Futures are derivative financial contracts that obligate parties to buy or sell an asset at an unspecified future date.

Requesting Data

Handling Data

Market Hours

See Also

BasicTemplateCryptoFutureAlgorithm.py
BasicTemplateCryptoFutureAlgorithm.cs

Crypto Futures

Requesting Data

Introduction

Request Crypto Futures data in your algorithm to receive a feed of asset prices in the `on_data` method. For more information about the specific datasets we use for backtests, see the Binance Crypto Future Price Data dataset listing . To trade Crypto Futures live, you can use the QuantConnect data provider .

Create Subscriptions

To create a Crypto Futures subscription, in the `initialize` method, call the `add_crypto_future` method. The `add_crypto_future` method returns a `CryptoFuture` object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_crypto_future("BTCUSD").symbol
```
PY

The `add_crypto_future` method creates a subscription for a single Crypto Futures asset and adds it to your user-defined universe.

To view the supported assets in the Crypto Futures dataset, see Supported Assets .

Resolutions

The following table shows the available resolutions and data formats for Crypto Futures contract subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`			✓	✓
`SECOND`	✓	✓		
`MINUTE`	✓	✓		
`HOUR`	✓	✓		
`DAILY`	✓	✓		

The default resolution for Crypto Futures subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_crypto_future` method.

```
self._symbol = self.add_crypto_future("BTCUSD", Resolution.DAILY).symbol
```
PY

To create custom resolution periods, see Consolidating Data .

Supported Markets

The following `Market` enumeration members are available for Cryptofuture:

```
self._symbol = self.add_crypto_future("BTCUSD", market=Market.BINANCE).symbol
```
PY

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you

disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
PY
self._symbol = self.add_crypto_future("BTCUSD", fill_forward=False).symbol
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. The amount of leverage available to you depends on the brokerage you use. To change the amount of leverage you can use for a security, pass a `leverage` argument to the `add_crypto_future` method.

```
PY
self._symbol = self.add_crypto_future("BTCUSD", leverage=3).symbol
```

For more information about the leverage each brokerage provides, see Brokerages .

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Properties

The `add_crypto_future` method returns a `CryptoFuture` object, which have the following properties:

Examples

The following examples demonstrate some common practices for requesting Crypto Futures data.

Example 1: Respect Lot Sizes

Like Futures, Crypto Futures contracts have a fixed discrete lot size to trade instead of fractional like spot Crypto pairs. The following algorithm demonstrates how to place orders that respect the lot size to avoid rounding errors.

```
PY
class CryptoFutureAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set brokerage and account type to match your brokerage environment for accurate fee and margin behavior.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)
        # In the Binance brokerage, you can't trade with USD.
        # Set the account currency as USDT and add the starting cash.
        self.set_account_currency("USDT", 100_000)
        # Subscribe to the BTCUSDT perpetual Future contract.
        self._symbol = self.add_crypto_future("BTCUSDT").symbol

    def on_data(self, slice: Slice) -> None:
        # Only place orders when market is open since market on open orders aren't supported.
        if not self.portfolio.invested and self.is_market_open(self._symbol):
            # Get the lot size from the symbol properties. Placing an order that respects the lot size
            # ensures the order is valid and allows accurate profit and risk calculations for the whole portfolio.
            lot_size = self.securities[self._symbol].symbol_properties.lot_size
            # This example demonstrates an initial desired order size of 2.5 contracts, which will be rounded
            # to 2 contracts since the lot size is 1.
            quantity = 2.5 // lot_size * lot_size
            self.market_order(self._symbol, quantity)
```

Crypto Futures

Handling Data

Introduction

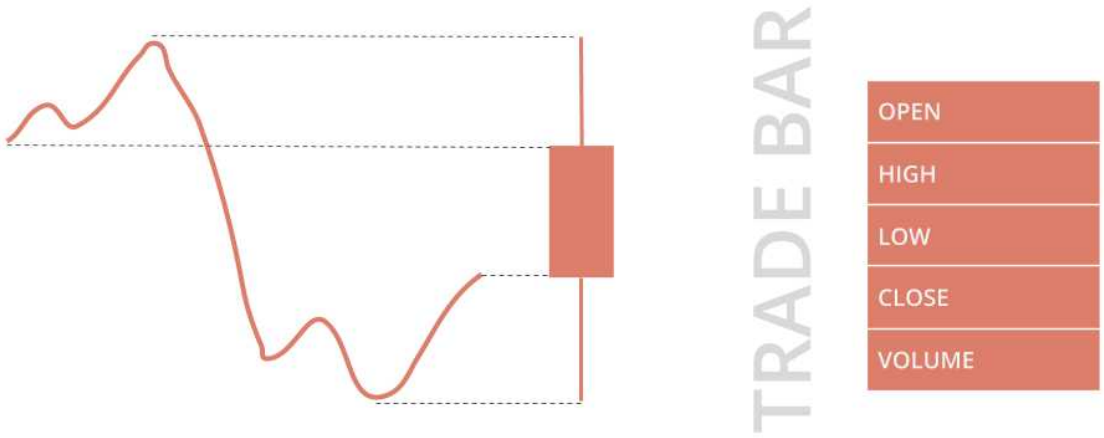
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the security ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Crypto Futures data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the security `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._symbol)   # None if not found
```

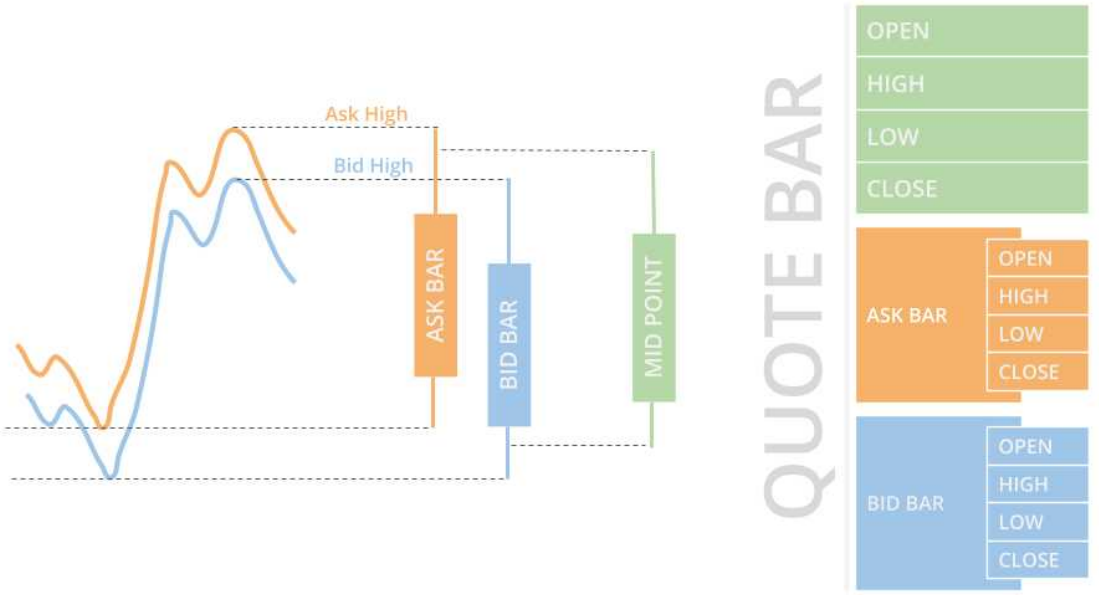
You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the security `symbol` . If the security doesn't actively get quotes or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._symbol)   # None if not found
```

PY

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

PY

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the security. A quote tick is an offer to buy or sell the security at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the security doesn't actively trade or you are in the same time step as when you added the security subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your security before you index the `Slice` with the security `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

Margin Interest Rates

`MarginInterestRate` objects contain the margin interest rate, which is a cost associated with trading on margin. To get the `MarginInterestRate` objects in the `Slice` , index the `margin_interest_rates` property of the `Slice` with the Crypto Future `Symbol` . The `margin_interest_rates` property of the `Slice` may not contain data for your `Symbol` . To avoid issues, check if the property contains data for your Crypto Future before you index it with the Crypto Future `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    margin_interest_rate = slice.margin_interest_rates.get(self._symbol)
    if margin_interest_rate:
        interest_rate = margin_interest_rate.interest_rate
```

You can also iterate through the `margin_interest_rates` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `margin_interest_rate` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, margin_interest_rate in slice.margin_interest_rates.items():
        interest_rate = margin_interest_rate.interest_rate
```

`MarginInterestRate` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Crypto Futures data.

Example 1: Adjust Holdings on Margin Payments

Perpetual futures have a mechanism called funding , where if you're holding a position at certain times (the funding timestamp), you might have to pay or receive funding based on the difference between the perpetual contract price and the spot price. This example demonstrates how to adjust your position in Crypto Future based on the funding rate. The funding is settled in the quote currency, which is USDT in this case. When you receive USDT, the algorithm increases its position size in BTCUSDT. When you pay USDT, the algorithm decreases its position size.

```py
class CryptoFutureAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set brokerage and account type to match your brokerage environment for accurate fee and margin behavior.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)
        # In the Binance brokerage, you can't trade with USD.
        # Set the account currency as USDT and add the starting cash.
        self.set_account_currency("USDT", 1000000)
        # Subscribe to the BTCUSDT perpetual Future contract.
        btcusdt = self.add_crypto_future("BTCUSDT")
        self._symbol = btcusdt.symbol
        # Save the lot size to avoid order errors.
        self._lot_size = btcusdt.symbol_properties.lot_size
        # Set the starting BTC balance to 10.
        btcusdt.base_currency.set_amount(10)
        # Create a member to track the current day.
        self._day = -1

    def on_data(self, slice: Slice) -> None:
        # Only place orders when the market is open since market on open orders aren't supported.
        if not self.portfolio.invested and self.is_market_open(self._symbol):
            # Open a long position in the perpetual Crypto Future.
            self.market_order(self._symbol, 10)
        # Only run the following logic once per day.
        if self._day == self.time.day:
            return
        # Get the current margin interest rate.
        interest_rate = slice.margin_interest_rates.get(self._symbol)
        if not interest_rate:
            return
        # Calculate the funding payment.
        holding = self.portfolio[self._symbol]
        position_value = holding.get_quantity_value(holding.quantity).amount
        interest_rate = slice.margin_interest_rates[self._symbol].interest_rate
        funding = interest_rate * position_value / holding.security.price
        # Increase/decrease the position size based on the funding payment.
        quantity = -funding // self._lot_size * self._lot_size
        if quantity:
            self.market_order(self._symbol, quantity)
            # Plot the portfolio state.
            self.plot("CashBook", "USDT", self.portfolio.cash_book['USDT'].amount)
            self.plot("CashBook", "BTC", self.portfolio.cash_book['BTC'].amount)
            self.plot("Quantity", "BTCUSDT", self.portfolio[self._symbol].quantity)
        self._day = self.time.day
```

Example 2: Future-Spot Arbitrage

Long-short arbitrage involves simultaneously trading BTCUSDT in the spot market and BTCUSDT Futures with the same size but in the opposite direction. The following algorithm monitors the spread between these markets. If the spot price exceeds the Future price by a threshold, it shorts the spot market and buys the Future contract (and vice vera for the other way around), assuming their discrepancies will be wiped out within a short period due to market efficiency. It closes the positions after the spread inverts.

PY

```
class CryptoFutureArbitrageAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set brokerage and account type to match your brokerage
        # environment for accurate fee and margin behavior.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)
        # In the Binance brokerage, you can't trade with USD.
        # Set the account currency to USDT and add the starting cash.
        self.set_account_currency("USDT", 50_000_000)
        # Add the BTCUSDT spot and Futures markets.
        self._spot = self.add_crypto(
            "BTCUSDT", fill_forward=False, market=Market.BINANCE
        )
        self._future = self.add_crypto_future(
            "BTCUSDT", fill_forward=False, market=Market.BINANCE
        )
        self._multiplier = self._future.symbol_properties.contract_multiplier
        self._threshold = 1.005  # 0.5%

    def on_data(self, slice: Slice) -> None:
        # Wait for there to be a TradeBar for each asset, so we know
        # the prices are up-to-date.
        for security in [self._spot, self._future]:
            if security.symbol not in slice.bars:
                return
        spot = self._spot.price
        future = self._future.price
        # Buy low sell high: If one's price is above another by the
        # set threshold, sell it and buy the other security. The
        # threshold ensures there is enough profit potential to overcome
        # the fees and slippage. Make sure to equalize the actual order
        # size with the contract multiplier.
        if not self._spot.invested:
            if spot >= future * self._threshold:
                self._enter(-1, 1)
            elif future >= spot * self._threshold:
                self._enter(1, -1)
            return
        # When the mispricing converges, close both positions to earn
        # the spread.
        if ((self._spot.holdings.is_short and spot < future) or
            (self._spot.holdings.is_long and spot > future)):
            # Liquidate the spot position.
            quantity = min(
                self._spot.holdings.quantity,
                self._spot.base_currency.amount
            )
            if quantity < 0:
                quantity /= (1 - 0.001)  # To cover the taker fees.
            lot_size = self._spot.symbol_properties.lot_size
            quantity = -round(quantity / lot_size) * lot_size
            self.market_order(self._spot.symbol, quantity, tag='Liquidate Spot')
            # Liquidate the Future position.
            self.liquidate(self._future.symbol, tag='Liquidate Future')

    def _enter(self, spot_sign, future_sign):
        quantity = self.calculate_order_quantity(self._future.symbol, 0.5)
        if not quantity:
            return
        # Enter the spot position.
        spot_quantity = spot_sign * quantity * self._multiplier
        self.market_order(self._spot.symbol, spot_quantity, tag='Spot')
        # Enter the Future position.
        self.market_order(self._future, future_sign * quantity, tag='Future')
```

Crypto Futures

Market Hours

Introduction

The Crypto Futures markets are open 24/7. There are no pre-market hours, post-market hours, holidays, late opens, or early closes.

Asset Classes

Forex

Forex markets are for trading foreign currencies.

Requesting Data

Handling Data

Market Hours

See Also

BasicTemplateForexAlgorithm.py
BasicTemplateForexAlgorithm.cs

Forex

Requesting Data

Introduction

Request Forex data in your algorithm to receive a feed of exchange rates in the `on_data` method. For more information about the specific dataset we use for backtests, see the FOREX dataset listing . To trade Forex live, you can use the QuantConnect data provider .

Create Subscriptions

To create a Forex pair subscription, in the `initialize` method, call the `add_forex` method. The `add_forex` method returns a `Forex` object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_forex("EURUSD").symbol
```

PY

To view the supported Forex pairs, see Supported Assets .

Resolutions

The following table shows the available resolutions and data formats for Forex subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`				✓
`SECOND`		✓		
`MINUTE`		✓		
`HOUR`		✓		
`DAILY`		✓		

The default resolution for Forex subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_forex` method.

```
self._symbol = self.add_forex("EURUSD", Resolution.DAILY).symbol
```

PY

To create custom resolution periods, see Consolidating Data .

Supported Markets

The only market available for Forex pairs is `Market.OANDA` , so you don't need to pass a `market` argument to the `add_forex` method.

```
self._symbol = self.add_forex("EURUSD", market=Market.OANDA).symbol
```

PY

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self._symbol = self.add_forex("EURUSD", fill_forward=False).symbol
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. The OANDA brokerage let's you use up to 50x leverage on margin accounts. To change the amount of leverage you can use for a Forex pair, pass a `leverage` argument to the `add_forex` method.

```
self._symbol = self.add_forex("EURUSD", leverage=35).symbol
```

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Properties

The `add_forex` method returns a `Forex` object, which have the following properties:

Examples

The following examples demonstrate some common practices for requesting Forex data.

Example 1: Add All Forex Pairs

The following algorithm adds all the Forex pairs that trade on the OANDA exchange . To get all of the pairs from the Symbol Properties Database , call the `symbol_properties_database.get_symbol_properties_list` method. Adding all of the pairs is particularly useful for spotting arbitrage opportunities among the pairs.

```
class ForexExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._forex_pairs = [
            # Add all of the pairs. You don't need to specify the market because the Symbol object already does.
            self.add_forex(x.key.symbol, Resolution.DAILY)
            # Get all of the OANDA Forex pairs from the Symbol Properties Database.
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.OANDA, SecurityType.FOREX)
        ]
```

Forex

Handling Data

Introduction

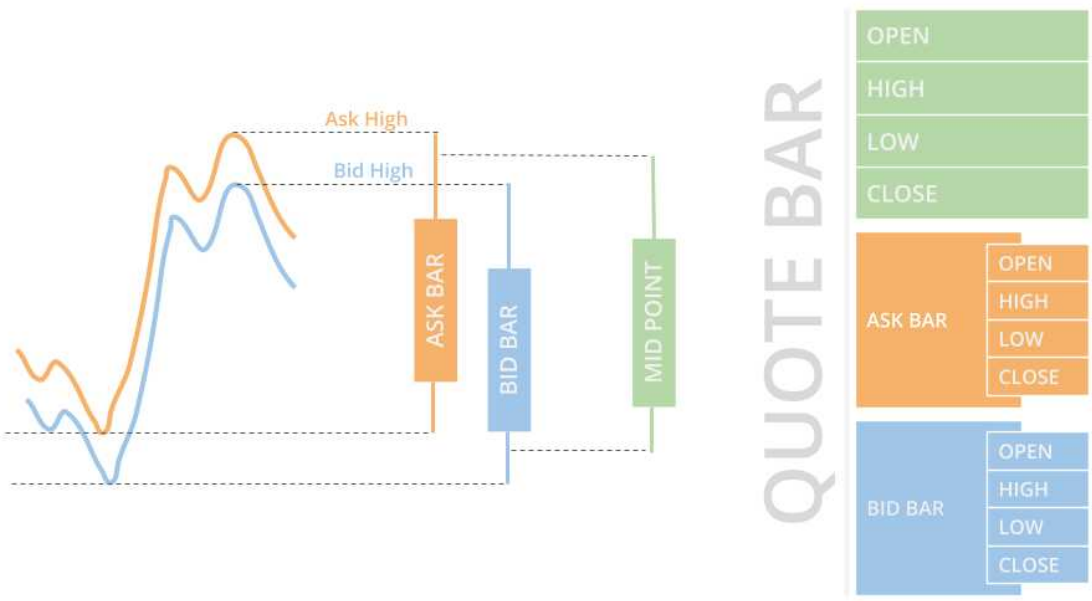
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `QuoteBars` `DataDictionary` is made up of `QuoteBar` objects. To access individual data points in the dictionary, you can index the dictionary with the Forex pair ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Forex data, see Resolutions .

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the Forex pair `symbol` . If the Forex pair doesn't actively get quotes or you are in the same time step as when you added the Forex pair subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your Forex pair before you index the `Slice` with the Forex pair `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._symbol)   # None if not found
```

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the Forex pair. A quote tick is an offer to buy or sell the Forex pair at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the Forex pair doesn't actively trade or you are in the same time step as when you added the Forex pair subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your Forex pair before you index the `Slice` with the Forex pair `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Forex data.

Example 1: Trade the USDJPY Spread

USDJPY is a Forex pair where the reserve banks of the base currency and quote currency act at different time, which results in a 12-hour period of seasonal peak volume. To reduce slippage costs, place orders when bid-ask spread is relatively low. The following algorithm demonstrates how to utilize the Minimum indicator to check for a narrow spread.

```py
class ForexExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the USDJPY trading pair.
        self._forex = self.add_forex("USDJPY")
        # Create a Minimum indicator to track the lowest bid-ask spread for the past 12 hours.
        self._forex.spread_low = Minimum(12*60)
        # Warm up the indicator so it's immediately ready to use.
        self.warm_up_indicator(self._forex.symbol, self._forex.spread_low)

    def on_data(self, slice: Slice) -> None:
        # Ensure we have quote data for USDJPY in the current slice.
        if self._forex.symbol not in slice.quote_bars:
            return
        quote_bar = slice.quote_bars[self._forex.symbol]
        # Bid-ask spread = Ask price - Bid price
        bid_ask_spread = quote_bar.ask.close = quote_bar.bid.close
        # Update the spread minimum indicator to calculate the lowest bid-ask spread over the last 12 hours.
        self._forex.spread_low.update(quote_bar.end_time, bid_ask_spread)

        # Trade if the current spread is the lowest bid-ask spread, since it is the most efficient,
        # liquid price with lowest slippage.
        if not self.portfolio.invested and bid_ask_spread == self._forex.spread_low.current.value:
            self.market_order(self._forex.symbol, 1000)

        # Plot the bid-ask spread to validate the minimum bid-ask spread calculation.
        self.plot("Bid-Ask Spread", "spread", bid_ask_spread)
```

Example 2: 5-Minute EURUSD Trend Reversal

The following algorithm applies a mean reveral strategy to EURUSD. To smooth out the noise of minute resolution data, it consolidates the data into 5-minute bars and then calculates their 200-bar Exponential Moving Average (EMA). After every time the indicator updates, the algorithm checks for a trading opportuniity. When the current EURUSD price is below the EMA, it takes a long position. Otherwise, it takes a short position.

```py
class ForexExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add EURUSD minute data.
        self._forex = self.add_forex("EURUSD")
        # Create an EMA indicator to estimate the trend of EURUSD.
        self._forex.ema = ExponentialMovingAverage(200)
        # Attach an event handler to the indicator to track its updates.
        self._forex.ema.updated += self.on_updated
        # Create a consolidator that produces 5-minute quote bars to reduce reduce noise.
        consolidator = QuoteBarConsolidator(timedelta(minutes=5))
        # Define the consolidator handler so it updates the EMA with the consoildated bars.
        consolidator.DataConsolidated += lambda _, bar: self._forex.ema.update(bar.end_time, bar.close)
        # Warm up the indicator with historical data.
        for bar in self.history[QuoteBar](self._forex.symbol, 1000):
            consolidator.update(bar)
        # Subscribe the consolidator for automatic updates.
        self.subscription_manager.add_consolidator(self._forex.symbol, consolidator)

    def on_updated(self, sender: object, point: IndicatorDataPoint) -> None:
        # Wait until the indicator is ready and the Forex pair has a price.
        if not self._forex.ema.is_ready or not self._forex.price:
            return
        # Rebalance the portfolio. If the current price is below the EMA, long. Otherwise, short.
        self.set_holdings(self._forex.symbol, 0.5 if self._forex.price <= point.value else -0.5)
```

Example 3: New York Time Alignment

The following algorithm creates daily Forex bars that close at 4:30 PM Eastern Time (ET). This technique is helpful when you want to align the daily bars with the close of the New York.

PY

```
class ForexExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add EURUSD minute data.
        self._forex = self.add_forex("EURUSD")
        # Create a daily bar consolidator with a custom period (4:30 PM-4:30 PM Eastern Time).
        consolidator = QuoteBarConsolidator(self._consolidation_period)
        # Attach a consolidation handler to plot the consolidated bars.
        consolidator.data_consolidated += lambda _, bar: self.plot("Consolidated Bar", "Close", bar.close)
        # Subscribe the consolidator for automatic updates.
        self.subscription_manager.add_consolidator(self._forex.symbol, consolidator)

    def _consolidation_period(self, dt: datetime) -> CalendarInfo:
        # Set the start time to be 4:30 PM of the previous day.
        start = dt.replace(hour=16, minute=30, second=0, microsecond=0)
        period = timedelta(1)
        if dt.hour < 17:
            start -= period
        # Set the period to 1 day to make daily bars.
        return CalendarInfo(start, period)
```

Forex

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the Forex market.

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

Futures

A Futures contract is an agreement to buy or sell a particular commodity, asset, or security at a predetermined price at a specified time in the future.

Requesting Data

Handling Data

Market Hours

See Also

BasicTemplateFuturesAlgorithm.py
BasicTemplateFuturesAlgorithm.cs

Futures

Requesting Data

Request Futures data in your algorithm to receive a feed of contract prices in the `on_data` method. For more information about the specific dataset we use for backtests, see the US Futures dataset listing . To trade Futures live, you can use the QuantConnect data provider or one of the brokerage data providers .

To request Futures data, call one of the following methods:

- `self.add_future` to add a basket of contracts for a single Future
- `self.add_future_contract` to add individual contracts for an underlying Future

Request Continuous Contracts

`self.add_future`

Request and Filter All Contracts

`self.add_future`

Select Individual Contracts

`self.add_future_contract`

Requesting Data

Universes

Introduction

The `add_future` method enables you to select a basket of Future contracts for an underlying Future. To form your universe of contracts, you can filter them down by their expiry. If you want to subscribe to individual contracts one-by-one instead of a set of contracts, see Individual Contracts .

Create Universes

To add a universe of Future contracts, in the `initialize` method, call the `add_future` method. This method returns an `Future` object, which has a `set_filter` method you can call to filter the set of tradable contract down to just the contracts you want.

```
PY
class BasicFutureAlgorithm(QCAlgorithm):

    def initialize(self):
        self.future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            Resolution.DAILY,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self.future.set_filter(0, 182)

    def on_data(self, data):
        for continuous_symbol, chain in data.future_chains.items():
            for symbol, contract in chain.contracts.items():
                expiry = contract.expiry
```

For more information about the `add_future` method, see Create Universes .

Continous Contracts

By default, LEAN only subscribes to the continuous Future contract. A continuous Future contract represents a series of separate contracts stitched together to form a continuous price. If you need a lot of historical data to warm up an indicator, apply the indicator to the continuous contract price series. The `Future` object has a `symbol` property and a `mapped` property. The price of the `symbol` property is the adjusted price of the continuous contract. The price of the `mapped` property is the raw price of the currently selected contract in the continuous contract series.

```
PY
# Get the adjusted price of the continuous contract.
adjusted_price = self.securities[self._future.symbol].price

# Get the raw price of the currently selected contract in the continuous contract series.
raw_price = self.securities[self._future.mapped].price
```

The continuous Futures contract isn't a tradable security . You must place orders for a specific Futures contract. To access the currently selected contract in the continuous contract series, use the `mapped` property of the `Future` object.

```
PY
# Place a market order for the currently selected contract in the continuous contract series.
self.market_order(self._future.mapped, 1)
```

For more information, see Continous Contracts .

Filter by Contract Properties

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Future` object. The following table describes the available filter techniques:

`set_filter(minExpiry: int, maxExpiry: int)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[FutureFilterUniverse], FutureFilterUniverse])` Selects the contracts that a function selects.

```
# Select the contracts that expire within 182 days.
self._future.set_filter(0, 182)

# Select the front month contract.
self._future.set_filter(lambda future_filter_universe: future_filter_universe.front_month())
```

The following table describes the filter methods of the `FutureFilterUniverse` class:

`standards_only()` Selects standard contracts
`front_month()` Selects the front month contract
`back_months()` Selects the non-front month contracts
`back_month()` Selects the back month contracts
`expiration(min_expiry: timedelta, max_expiry: timedelta)` Selects contracts that expire within a range of dates relative to the current day
`expiration(min_expiry_days: int, max_expiry_days: int)` Selects contracts that expire within a range of dates relative to the current day
`contracts(contracts: list[Symbol])` Selects a list of contracts
`contracts(contractSelector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects

The preceding methods return an `FutureFilterUniverse` , so you can chain the methods together.

```
# Select the front month standard contracts
self._future.set_filter(lambda future_filter_universe: future_filter_universe.standards_only().front_month())
```

You can also define an isolated filter method.

```
# In Initialize
self._future.set_filter(self._contract_selector)

def _contract_selector(self,
    future_filter_universe: Callable[[FutureFilterUniverse], FutureFilterUniverse]) -> FutureFilterUniverse:
    return future_filter_universe.standards_only().front_month()
```

Default Filter

By default, LEAN doesn't add any contracts to the FuturesChain it passes to the `on_data` method.

Historical Data

To get historical data for a Futures universe, see Historical Data .

Examples

The following examples demonstrate some common practices for requesting Futures universe data.

Example 1: Rollover

Future contracts expire monthly or quarterly in most cases. Hence, if we hold Future position in month or quarter end, we must consider rolling over to the mapped contract. The following algorithm shows how to buy and roll over to the next front month Future contract. We make use of the universe selection filter to select the front month contract and order the next mapped contract during the previous one expires.

```
class FutureExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price with last known price to ensure the contract price data is available for the limit order.
        self.settings.seed_initial_prices = True
        future = self.add_future(Futures.Indices.SP_500_E_MINI, extended_market_hours=True)
        self._future = future.symbol
        # We only want to hold position of the front month contract.
        future.set_filter(lambda u: u.front_month())

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate if expired (or not being the front month contract anymore) and exit universe.
        for removed in changes.removed_securities:
            self.liquidate(removed.symbol)

        for added in changes.added_securities:
            # Make sure the newly added contract is an actual mapped tradable contract.
            if not added.symbol.is_canonical():
                # Roll over by ordering the same quantity.
                # Use limit order since market on open order is not supported on Future and avoid extreme quote filling.
                self.limit_order(added.symbol, 1, added.price)
```

Example 2: Continuous Future Indicator

One of the major applications of Continuous Futures is to obtain a smooth price series to feed into indicators. This technique ensures the indicator gets the price data that is comparable to the current contract in the continuous contract series. This example demonstrates a Exponential Moving Average crossover strategy based on the continuous ES contract data.

```py
class FutureEMACrossAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Define a threshold for the EMA crossover.
        self._threshold = 0.01  # 1% above/below the EMA.
        # Use backward ratio normalization for the continuous contract
        # so the indicator receives a smooth price series.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
        )
        # Only add the front-month contract.
        self._future.set_filter(lambda u: u.front_month())
        # Create a 252-day EMA indicator as a trend estimator.
        self._future.ema = self.ema(self._future.symbol, 252, Resolution.DAILY)
        # Warm up the EMA indicator to so we can trade right away.
        self.warm_up_indicator(self._future.symbol, self._future.ema)

    def on_data(self, slice: Slice) -> None:
        # Ensure the TradeBar data is available for the Future before
        # placing trades.
        bar = slice.bars.get(self._future.symbol)
        if not bar:
            return
        # Buy the mapped contract if the trend is estimated to go up
        # (price above the EMA).
        if bar.close >= self._future.ema.current.value * (1+self._threshold):
            self.set_holdings(self._future.mapped, 0.1)
        # Short the mapped contract if the trend is estimated to go down
        # (price below the EMA).
        elif bar.close <= self._future.ema.current.value * (1-self._threshold):
            self.set_holdings(self._future.mapped, -0.1)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate the contract when it is no longer the front-month
        # contract (expired).
        for contract in changes.removed_securities:
            self.liquidate(contract.symbol)
```

Example 3: Contango

In Future trading, contango refers to the far-to-expiry Future contract price is higher than the spot price due to various reasons, such as storage fee and insurance of the commodities. The following example shows a contango trading by shorting the far contract that the price is above a threshold compared to the front month contract price and buying the front month contract to earn the premium in between.

```py
class FutureExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add Gold Futures.
        future = self.add_future(Futures.Metals.GOLD)
        self._future = future.symbol
        # Limit the expiration to within 6 months, as the longer the expiration, the higher the price uncertainty.
        future.set_filter(lambda u: u.expiration(0, 183))

    def on_data(self, slice: Slice) -> None:
        # Get Future chain only for the selected Future contract.
        chain = slice.future_chains.get(self._future)
        if not self.portfolio.invested and chain:
            # It takes 2 contracts with different expiries to form a horizontal spread arbitration to earn price difference in contango.
            if len(list(chain)) < 2:
                return
            sorted_by_expiry = sorted(chain, key=lambda x: x.expiry)
            far_contract = sorted_by_expiry[-1]
            near_contract = sorted_by_expiry[0]

            # Check if the far contract price is 1% higher than the near one.
            # If so, short the far contract and buy the near one to earn the horizontal spread premium.
            if far_contract.bid_price >= near_contract.ask_price * 1.01:
                self.market_order(far_contract.symbol, -1)
                self.market_order(near_contract.symbol, 1)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate if expired (or not being the front month contract anymore) and exit universe.
        for removed in changes.removed_securities:
            self.liquidate()
```

Requesting Data

Individual Contracts

Introduction

The `add_future_contract` method enables you to add an individual Future contract to your algorithm. To check which contracts are currently available to add to your algorithm, call the `futures_chain` method. If you want to subscribe to a set of contracts instead of individual contracts one-by-one, see Universes .

Create Subscriptions

Before you can subscribe to a Future contract, you must have the underlying Future `Symbol` to get the contract `Symbol` .

```
class BasicFutureAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._continuous = self.add_future(Futures.Indices.SP_500_E_MINI).symbol
        self._contract_symbol = None

    def on_data(self, data):
        if not self._contract_symbol:
            self._contract_symbol = list(self.futures_chain(self._continuous))[0].symbol
            self.add_future_contract(self._contract_symbol)
```

Get Contract Symbols

To subscribe to a Future contract, you need the contract `Symbol` . The preferred method to getting Future contract `Symbol` objects is to use the `futures_chain` method. This method returns a `FuturesChain` object, which represent an entire chain of Option contracts for a single underlying security. You can even format the chain data into a DataFrame where each row in the DataFrame represents a single contract. With the chain, sort and filter the data to find the specific contract(s) you want to trade.

```
# Select the first contract in the chain, which is the front-month contract.
self._contract_symbol = list(self.futures_chain(self._continuous))[0].symbol
```

Alternatively, you can create a Future contract `Symbol` object directly parsing its ticker using the `SymbolRepresentation.parse_future_symbol` method.

```
# Create the contract Symbol for the December 2025 E-mini S&P 500 Future.
self._contract_symbol = SymbolRepresentation.parse_future_symbol('ESZ25')
# Create the contract Symbol using the 1-digit year format.
self._contract_symbol = SymbolRepresentation.parse_future_symbol('ESZ5')
# To avoid ambiguity, specify the 4-digit year.
self._contract_symbol = SymbolRepresentation.parse_future_symbol('ESZ5', 2015)
```

Subscribe to Contracts

To create an Future contract subscription, pass the contract `Symbol` to the `add_future_contract` method. Save a reference to the contract `symbol` so you can easily access the Future contract in the FutureChain that LEAN passes to the `on_data` method. This method returns an `Future` object.

```
self.add_future_contract(self._contract_symbol)
```

The `add_future_contract` method creates a subscription for a single Future contract and adds it to your user-defined universe. To create a dynamic universe of Futures contracts, add an Future universe or an Future Universe Selection model .

Warm Up Contract Prices

If you subscribe to an Future contract with `add_future_contract` , you'll need to wait until the next `Slice` to receive data and trade the contract. To trade the contract in the same time step you subscribe to the contract, set the current price of the contract in a security initializer .

```
self.settings.seed_initial_prices = True
```

Supported Assets

To view the supported assets in the Futures dataset, see the Supported Assets .

Resolutions

The following table shows the available resolutions and data formats for Futures subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`			✓	✓
`SECOND`	✓	✓		
`MINUTE`	✓	✓		
`HOUR`	✓	✓		
`DAILY`	✓	✓		

The default resolution for Futures contract subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_future_contract` method.

```
self.add_future_contract(self._contract_symbol, Resolution.DAILY)
```

To create custom resolution periods, see Consolidating Data .

Supported Markets

The following `Market` enumeration members are available for Futures:

Historical data for backtesting is unavailable for `ICE` , `INDIA` , `SGX` and `NYSELIFFE` . In live trading, LEAN sources this data from your brokerage or a third-party data provider .

You don't need to pass a `market` argument to the `add_future_contract` method because the contract `Symbol` already contains the market.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self.add_future_contract(self._contract_symbol, fill_forward=False)
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. Futures are already leveraged products, so you can't change their leverage with the default margin model .

Extended Market Hours

By default, your security subscriptions only cover regular trading hours. To subscribe to pre and post-market trading hours for a specific asset, enable the `extended_market_hours` argument when you create the security subscription.

```
self.add_future_contract(self._contract_symbol, extended_market_hours=True)
```

You only receive extended market hours data if you create the subscription with minute, second, or tick resolution. If you create the subscription with daily or hourly resolution, the bars only reflect the regular trading hours.

To view the schedule of regular and extended market hours, see Market Hours .

In general, we model most Futures market hours with the following segments:

Market Segment	Time
Pre-market	00:00:00 to 09:30:00
Market	09:30:00 to 17:00:00
Post-market	18:00:00 to 00:00:00

We model it this way because some Futures, like VIX, have pre- and post-market hours, so we standardized it. With this segmentation, if you set a Scheduled Events for the market open, it's set for 9:30 AM instead of midnight.

Continuous Contracts

A continuous Futures contract represents a single contract that maps to other contracts over time as the rollover rules are met. For more information about continuous Futures contracts, see Continuous Contracts .

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request for Futures contracts. If you change the data normalization mode, it won't change the outcome.

The following data normalization modes are available for continuous Futures contracts :

Properties

The `add_future_contract` method returns a `Future` object, which have the following properties:

Examples

The following examples demonstrate common practices for requesting individual Future contract data.

Example 1: Add the Front-Month Contract

The following algorithm adds the front-month contract, with a two-day buffer, for the E-Mini S&P 500 Future and plots the daily closing price of the contract three days before month end. Then, trade an EMA cross strategy.

PY

```
class FutureExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request the future underlying data for ATM strike selection.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
        ).symbol

        # Create an EMA indicator for trade signal generation.
        self._ema = self.ema(self._future, 20, Resolution.DAILY)
        # Warm up the indicator for immediate readiness to trade with.
        self.warm_up_indicator(self._future, self._ema, Resolution.DAILY)

        # Set a scheduled event to select the option contract periodically.
        self.schedule.on(
            # 3 days offset to select in prior.
            self.date_rules.month_end(3),
            self.time_rules.at(0, 1),
            self.select_contract
        )
        self.select_contract()

    def select_contract(self) -> None:
        # Get all the contracts that are currently tradable.
        chain = self.futures_chain(self._future)
        # Select the front-month contract due to liquidity, with a 2-day buffer to avoid volatility.
        contracts = [c for c in chain if c.expiry >= self.time + timedelta(2)]
        if not contracts:
            return
        contract = contracts[0].symbol
        # Request the Futures contract for trading.
        self._contract_symbol = self.add_future_contract(contract).symbol

    def on_data(self, slice: Slice) -> None:
        # Trade based on updated data.
        bar = slice.bars.get(self._contract_symbol)
        if bar and self.is_market_open(self._contract_symbol):
            # Trade trend-following strategy with EMA cross.
            if bar.close > self._ema.current.value and not self.portfolio[self._future].is_long:
                self.set_holdings(self._contract_symbol, 0.5)
            elif bar.close < self._ema.current.value and not self.portfolio[self._future].is_short:
                self.set_holdings(self._contract_symbol, -0.5)
```

Futures

Handling Data

Introduction

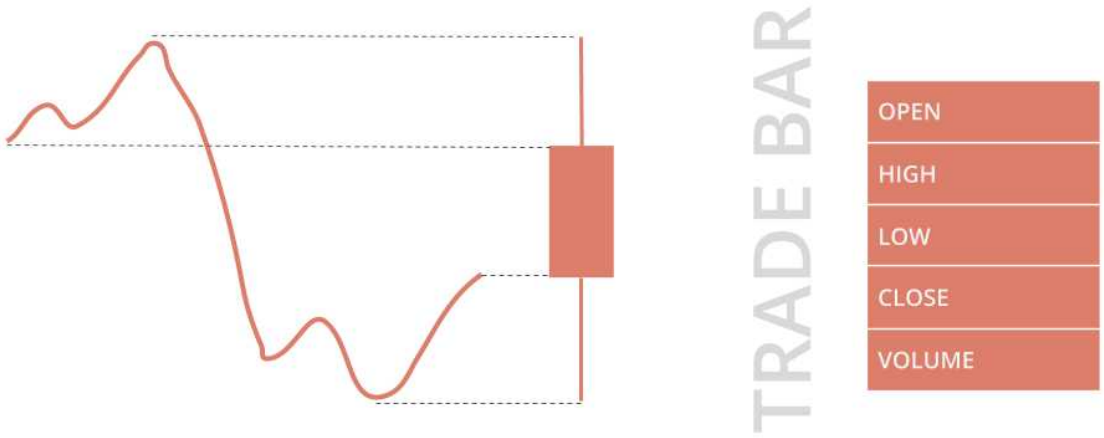
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the contract ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Futures data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively trade or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._contract_symbol)   # None if not found
```

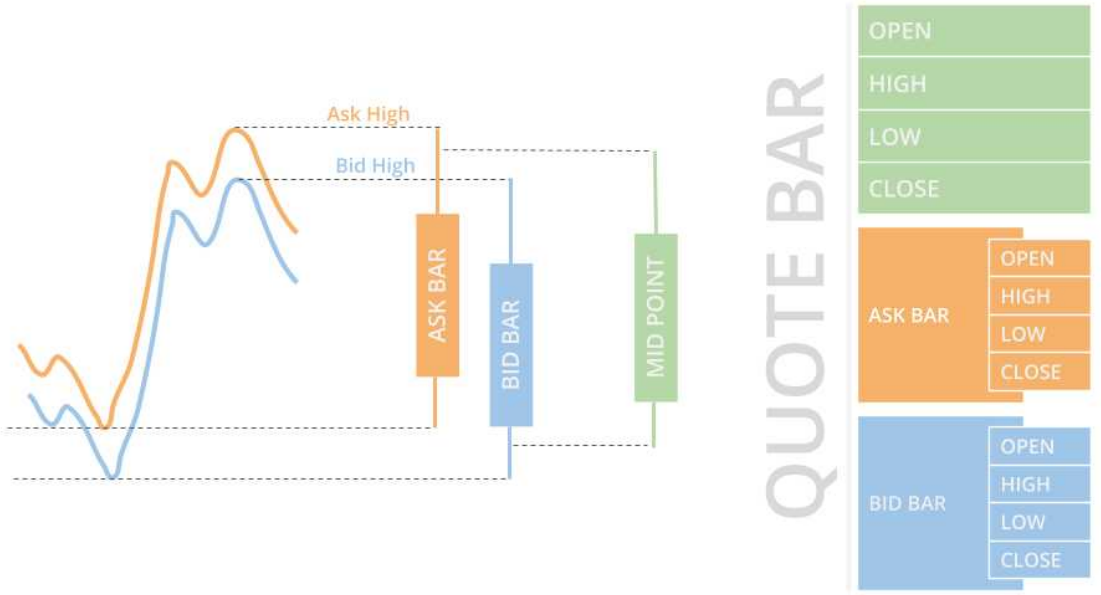
You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively get quotes or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._contract_symbol)   # None if not found
```

PY

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

PY

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the contract. A quote tick is an offer to buy or sell the contract at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the contract doesn't actively trade or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._contract_symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

Futures Chains

`FuturesChain` objects represent an entire chain of contracts for a single underlying Future. To get the `FuturesChain`, index the `futures_chains` property of the `Slice` with the continuous contract `Symbol`.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the FutureChain using the canonical symbol (None if no FutureChain return)
    chain = slice.futures_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get all contracts if the FutureChain contains any member
        contracts = chain.contracts
```

You can also loop through the `futures_chains` property to get each `FuturesChain`.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-FutureChain key-value pairs
    for continuous_contract_symbol, chain in slice.futures_chains.items():
        contracts = chain.contracts
```

`FuturesChain` objects have the following properties:

Futures Contracts

`FuturesContract` objects represent the data of a single Futures contract in the market. To get the Futures contracts in the `Slice`, use the `contracts` property of the `FuturesChain`.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the FutureChain using the canonical symbol
    chain = slice.future_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data (None if not contained)
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            price = contract.last_price
```

Open interest is the number of outstanding contracts that haven't been settled. It provides a measure of investor interest and the market liquidity, so it's a popular metric to use for contract selection. Open interest is calculated once per day. To get the latest open interest value, use the `open_interest` property of the `Future` or `future_contract`.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the futures_chains using the canonical symbol
    chain = slice.futures_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get the open interest of the selected contracts
            open_interest = contract.open_interest
```

`FuturesContract` objects have the following properties:

Symbol Changes

When the continuous contract rolls over, LEAN passes a `SymbolChangedEvent` to your `on_data` method, which contains the old contract `Symbol` and the new contract `Symbol` . To get the `SymbolChangedEvent` , use the `symbol_changed_events` property of the `Slice` . You can use the `SymbolChangedEvent` to roll over contracts.

```
def on_data(self, slice: Slice) -> None:
    for symbol, changed_event in  slice.symbol_changed_events.items():
        old_symbol = changed_event.old_symbol
        new_symbol = changed_event.new_symbol
        tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
        quantity = self.portfolio[old_symbol].quantity

        # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
        self.liquidate(symbol=old_symbol, tag=tag)
        if quantity: self.market_order(new_symbol, quantity, tag=tag)
        self.log(tag)
```

LEAN passes a `SymbolChangedEvent` to your `on_symbol_changed_events` method as well. LEAN only invokes this method when the continuous contract rolls over.

```
def on_symbol_changed_events(self, symbol_changed_events: SymbolChangedEvents) -> None:
    for symbol, changed_event in symbol_changed_events.items():
        old_symbol = changed_event.old_symbol
        new_symbol = changed_event.new_symbol
        tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
        quantity = self.portfolio[old_symbol].quantity

        # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
        self.liquidate(symbol=old_symbol, tag=tag)
        if quantity: self.market_order(new_symbol, quantity, tag=tag)
        self.log(tag)
```

In backtesting, the `SymbolChangedEvent` occurs at midnight Eastern Time (ET). In live trading, the live data for continuous contract mapping arrives at 6/7 AM ET, so that's when it occurs.

`SymbolChangedEvent` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Futures data.

Example 1: Rollovers

Spot Future is referred to as the continuous Future contract, which is usually mapped by the front month contract or the contract with the most open interest. When a contract expires or is very close to expiring, traders usually rollover from the current contract to the next contract to avoid price settlement and remain invested. The following algorithm demonstrates rolling over with limit orders .

```py
class FutureExampleAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the E-mini Futures and set the continuous contract mapping criteria for the rollovers.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

    def on_data(self, data):
        # Place the initial order so you can start rolling over contracts later.
        if not self.portfolio.invested and not self.transactions.get_open_orders():
            # Buy the contract that's currently selected in the continous contract series.
            self.market_order(self._future.mapped, 1)

    # Track rollover events.
    def on_symbol_changed_events(self, symbol_changed_events):
        for symbol, changed_event in symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = changed_event.new_symbol
            # The quantity to roll over should be consistent.
            quantity = self.portfolio[old_symbol].quantity
            # Rolling over: Liquidate the old mapped contract and switch to the newly mapped contract.
            tag = f"Rollover: {old_symbol} -> {new_symbol}"
            self.liquidate(old_symbol, tag=tag)
            if quantity:
                # Place a limit order to avoid extreme quote filling.
                new_contract = self.securities[new_symbol]
                self.limit_order(
                    new_symbol, quantity,
                    # To avoid warnings, round the target limit price to a price that respects
                    # the minimum price variation for the Future.
                    self._get_limit_price(new_contract, new_contract.price),
                    tag=tag
                )

    def _get_limit_price(self, security, target_limit_price, round_up=True):
        parameters = GetMinimumPriceVariationParameters(security, target_limit_price)
        pip = security.price_variation_model.get_minimum_price_variation(parameters)
        return (int(target_limit_price / pip) + int(round_up)) * pip
```

Example 2: Micro Gold vs Gold

The following example demonstrates how to use the `SymbolPropertiesDatabase` to obtain the contract multipliers of both Micro Gold and Gold Future contracts. The algorithm uses it to invest $500k of nominal value into each contract.

PY

```
class FutureNominalBuysAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)
        # Set the continuous contract mapping criteria for both Futures
        # since we want to trade the most liquid contracts.
        self._futures = []
        for ticker in [Futures.Metals.GOLD, Futures.Metals.MICRO_GOLD]:
            future = self.add_future(
                ticker,
                extended_market_hours=True,
                data_mapping_mode=DataMappingMode.OPEN_INTEREST,
                data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
                contract_depth_offset=0
            )
            future.set_filter(0, 180)
            self._futures.append(future)

    def on_data(self, slice: Slice) -> None:
        # Wait until we aren't invested and we have bars for all Futures.
        if (not self.portfolio.invested and
            all(future.symbol in slice.bars for future in self._futures)):
            for future in self._futures:
                # Calculate the order size for $500k. Get the quotient
                # after dividing by the contract multiplier since the
                # order size must be whole number.
                quantity = (
                    500_000
                    / future.price
                    // future.symbol_properties.contract_multiplier
                )
                if not quantity:
                    continue
                self.market_order(future.mapped, quantity)
```

Futures

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the CME Future market.

Search by symbol (e.g. ES, NQ

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

Future Options

Future Option contracts give the buyer a window of opportunity to buy or sell the underlying Future contract at a specific price.

Requesting Data

Handling Data

Greeks and Implied Volatility

Market Hours

See Also

BasicTemplateFutureOptionAlgorithm.py
BasicTemplateFutureOptionAlgorithm.cs

Future Options

Requesting Data

Request Future Options data in your algorithm to receive a feed of contract prices in the `on_data` method. For more information about the specific dataset we use for backtests, see the US Future Options dataset listing . To trade Future Options live, you can use one of the brokerage data providers . We currently only support European-style Options for US Future Options.

To request Future Options data, call one of the following methods:

- `self.add_future_option` to add a basket of contracts for a single underlying Future
- `self.add_future_option_contract` to add individual contracts for an underlying Future

Request and Filter Contracts for Option Strategies

`self.add_future_option`

Request and Filter All Contracts

`self.add_future_option`

Select Individual Contracts

`self.add_future_option_contract`

See Also

Introduction to Options
Applied Options

Requesting Data

Universes

Introduction

The `add_future_option` method enables you to select a basket of Option contracts for an underlying Future. To form your universe of contracts, you can filter them down by their strike price and expiry or you can choose a subset of the contracts that form popular Option strategies . If you want to subscribe to individual contracts one-by-one instead of a set of contracts, see Individual Contracts .

Create Universes

To add a universe of Future Option contracts, in the `initialize` method, call the `add_future_option` method. Pass the `option_filter` argument to filter the set of tradable contract down to just the contracts you want.

```
PY
class BasicFutureOptionAlgorithm(QCAlgorithm):
    def initialize(self):
        self._future = self.add_future(Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        self._future.set_filter(0, 182)

        self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-1, 1))

    def on_data(self, data):
        # Iterate all Option chains to find the desired future and future options contracts
        for canonical_symbol, chain in data.option_chains.items():
            future_contract = canonical_symbol.underlying
            for symbol, contract in chain.contracts.items():
                strike = contract.strike

        # Get the Option chain for the mapped contract
        mapped_chain = data.option_chains.get(Symbol.create_canonical_option(self._future.mapped))
        if mapped_chain:
            for symbol, contract in mapped_chain.contracts.items():
                strike = contract.strike
```

For more information about the `add_future_option` method, see Create Universes .

Filter by Investment Strategy

Options trading strategies consist of simultaneously buying or selling one or more Option contracts of the same underlying Future with different features. The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for Option strategies.

The following table describes the filter methods of the `OptionFilterUniverse` class that select contracts for Option strategies:

`naked_call(min_days_till_expiry: int, strike_from_atm: float)` Selects a call contract to form Naked Call , Covered Call , or Protective Call Option strategies.
`naked_put(min_days_till_expiry: int, strike_from_atm: float)` Selects a put contract to form Naked Put , Covered Put , or Protective Put Option strategies.
`call_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two call contracts to form Bull Call Spread or Bear Call Spread Option strategies.
`put_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two put contracts to form Bull Put Spread or Bear Put Spread Option strategies.

`call_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects two call contracts to form Long Call Calendar Spread or Short Call Calendar Spread Option strategies.

`put_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects two put contracts to form Long Put Calendar Spread or Short Put Calendar Spread Option strategies.

`strangle(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)`

Selects two contracts to form Long Strangle or Short Strangle Option strategies.

`straddle(min_days_till_expiry: int)`

Selects two contracts to form Long Straddle or Short Straddle Option strategies.

`protective_collar(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)`

Selects two contracts to form Protective Collar Option strategies.

`conversion(min_days_till_expiry: int, strike_from_atm: float)`

Selects two contracts to form Conversion or Reverse Conversion Option strategies.

`call_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects three contracts to form Long Call Butterfly or Short Call Butterfly Option strategies.

`put_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects three contracts to form Long Put Butterfly or Short Put Butterfly Option strategies.

`iron_butterfly(min_days_till_expiry: int, strike_spread: float)`

Selects four contracts to form Long Iron Butterfly or Short Iron Butterfly Option strategies.

`iron_condor(min_days_till_expiry: int, near_strike_spread: float, far_strike_spread: float)`

Selects four contracts to form Long Iron Condor or Short Iron Condor Option strategies.

`box_spread(min_days_till_expiry: int, strike_spread: float)`

Selects four contracts to form Box Spread or Short Box Spread Option strategies.

`jelly_roll(strike_from_atm: float, min_near_days_till_expiry: int, min_far_days_till_expiry: int)`

Selects four contracts to form Jelly Roll or Short Jelly Roll Option strategies.

`call_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)`

Selects four contracts to form Bear Call Ladder or Bull Call Ladder Option strategies.

`put_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)`

Selects four contracts to form Bear Put Ladder or Bull Put Ladder Option strategies.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
PY

# Example 1: Select a Straddle
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.straddle(30, 5, 10))

# Example 2: Select the contracts (including weeklys) that expire in the next 30 days that form an Iron Condor
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0,
30).iron_condor(30, 5, 10))

# Example 3: Select the front month contracts that form an Strangle
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.front_month().strangle(30, 5, -10))
```

Filter by Other Contract Properties

To set a contract filter, in the `initialize` method, call the `add_future_option` method and pass the `option_filter` argument.

```
PY

# Select contracts that have a strike price within 1 strike level above and below the underlying price
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-1,1))

# Select contracts that expire within 30 days
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.expiration(0,30))

# Select contracts that have a strike price within 1 strike level and expire within 30 days
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-1,1).expiration(0,30))

# Select call contracts
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.calls_only())
```

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts. This filter doesn't currently work. To add this functionality to our roadmap , react to this GitHub Issue with a thumbs up emoji.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

PY

```
# Example 1: Select the front month call contracts
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.calls_only().front_month())

# Example 2: Select the contracts that expire in the next 90 days
self.add_future_option(self._future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 90))
```

To perform thorough filtering on the `OptionFilterUniverse` , define an isolated filter method.

```py
class BasicFutureOptionAlgorithm(QCAlgorithm):
    def initialize(self):
        self.add_future_option(self._future.symbol, self._selector)

    def _selector(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        symbols = list(option_filter_universe.puts_only())
        strike = min([symbol.id.strike_price for symbol in symbols])
        symbols = [symbol for symbol in symbols if symbol.id.strike_price == strike]
        return option_filter_universe.contracts(symbols)
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Default Filter

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard and weeklys
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at the first bar of every day. When the filter selects a contract that isn't currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

Historical Data

To get historical data for a Future Options universe, see Historical Data .

Examples

The following examples demonstrate some common practices for requesting Future Option data through universe filtering.

Example 1: Call Spread

The following example demonstrates how to implement a bull call spread option strategy using universe filtering. We have to first subscribe to the underlying Future (with filtering), then call `self.add_future_option` with the underlying `Symbol` and the filtering function. We use `call_spread` method in the `OptionFilterUniverse` object to return only the 2 best match call contracts that forms a bull call spread.

```py
class FutureOptionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Filter the underlying continuous Futures to narrow the FOP spectrum.
        self.underlying = self.add_future(Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        self.underlying.set_filter(0, 182)
        # Use CallSpread filter to obtain the 2 best-matched contracts that forms a call spread.
        # It simplifies from further filtering and reduce computation on redundant subscription.
        self.add_future_option(self.underlying.symbol, lambda u: u.call_spread(5, 5, -5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return
        # Create canonical symbol for the mapped future contract, since we need that to access the option chain.
        symbol = Symbol.create_canonical_option(self.underlying.mapped)

        # Get option chain data for the mapped future only.
        # It requires 2 contracts with different strikes to form a call spread, so we make sure at least 2 contracts are present.
        chain = slice.option_chains.get(symbol)
        if not chain or len(list(chain)) < 2:
            return

        # Separate the contracts by strike, as we need to access their strike.
        expiry = min([x.expiry for x in chain])
        sorted_by_strike = sorted([x.strike for x in chain])
        itm_strike = sorted_by_strike[0]
        otm_strike = sorted_by_strike[-1]

        # Use abstraction method to order a bull call spread to avoid manual error.
        option_strategy = OptionStrategies.bull_call_spread(symbol, itm_strike, otm_strike, expiry)
        self.buy(option_strategy, 1)
```

Note that since both the underlying Future and the Future Option are expiring on the same day and are cash-settling in most cases, Lean can exercise the Future Option into account cash automatically at expiry and we do not need to handle the option exercise/assignment event.

Requesting Data

Individual Contracts

Introduction

The `add_future_option_contract` method enables you to add an individual Option contract to your algorithm. To check which contracts are currently available to add to your algorithm, use the `option_chain` method. If you want to subscribe to a set of contracts instead of individual contracts one-by-one, see Universes .

Create Subscriptions

Before you can subscribe to an Option contract, you must configure the underlying Future and get the contract `Symbol` .

```
class BasicFutureOptionAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        self._contract_symbol = None

    def on_data(self, data):
        if not self._contract_symbol:
            # Method 1: Add the desired option chain for the mapped contract
            self._contract_symbol = self._select_option_contract(self._future.mapped)
            self.add_future_option_contract(self._contract_symbol)

            # Method 2: Get all future contracts from the Future chain provider
            future_contract_symbol = list(self.futures_chain(self._future.symbol))[0].symbol
            symbol = self._select_option_contract(future_contract_symbol)
            self.add_future_option_contract(symbol)

            # Method 3: Iterate all Future chains to find the desired future options contracts
            for symbol, chain in data.future_chains.items():
                expiry = min([contract.expiry for contract in chain])
                future_contract = next(contract for contract in chain if contract.expiry == expiry)
                symbol = self._select_option_contract(future_contract.symbol)
                self.add_future_option_contract(symbol)

    def _select_option_contract(self, future_contract_symbol):
        chain = self.option_chain(future_contract_symbol, flatten=True).data_frame
        expiry = chain.expiry.min()
        return chain[
            (chain.expiry == expiry) & (chain.right == OptionRight.CALL)
        ].sort_values('strike').index[-1]
```

Configure the Underlying Futures Contracts

In most cases, you should subscribe to the underlying Futures contract before you subscribe to a Futures Option contract.

```
self._future = self.add_future(
    Futures.Indices.SP_500_E_MINI,
    extended_market_hours=True,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
    contract_depth_offset=0
)
```

To get a `Symbol` of a specific Futures contract, use the `mapped` property of the `Future` object or the `symbol` property of the `FutureContract` objects in the `Slice.future_chains` collection.

Get Contract Symbols

To subscribe to a Future Option contract, you need the contract `Symbol` . The preferred method to getting Option contract `Symbol` objects is to use

the option_chain method. This method returns an OptionChain object, which represent an entire chain of Option contracts for a single underlying security. You can even format the chain data into a DataFrame where each row in the DataFrame represents a single contract. With the chain, sort and filter the data to find the specific contract(s) you want to trade.

```
PY
# Get the contracts available to trade (in DataFrame format).
chain = self.option_chain(future_contract_symbol, flatten=True).data_frame

# Select a contract.
expiry = chain.expiry.min()
self._contract_symbol = chain[
    # Select call contracts with the closest expiry.
    (chain.expiry == expiry) & (chain.right == OptionRight.CALL)
    # Select the contract with the lowest strike price.
].sort_values('strike').index[0]
```

Alternatively, you can create a Future Option contract Symbol object directly parsing its OSI ticker using the SymbolRepresentation.parse_option_ticker_osi method.

```
PY
# Create the contract Symbol for the Sept 2024 E-mini S&P 500 Call 7900 20 Sept 2024.
self._contract_symbol = SymbolRepresentation.parse_option_ticker_osi('ESU24 240920C07900000', SecurityType.FUTURE_OPTION,
OptionStyle.AMERICAN, Market.CME)
# Create the contract Symbol using the 1-digit year format.
self._contract_symbol = SymbolRepresentation.parse_option_ticker_osi('ESU4 240920C07900000', SecurityType.FUTURE_OPTION,
OptionStyle.AMERICAN, Market.CME)
```

Subscribe to Contracts

To create a Future Option contract subscription, pass the contract Symbol to the add_future_option_contract method. Save a reference to the contract symbol so you can easily access the Option contract in the OptionChain that LEAN passes to the on_data method. This method returns an Option object. To override the default pricing model of the Option, set a pricing model .

```
PY
option = self.add_future_option_contract(self._contract_symbol)
option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

Warm Up Contract Prices

If you subscribe to a Future Option contract with add_future_option_contract , you'll need to wait until the next Slice to receive data and trade the contract. To trade the contract in the same time step you subscribe to the contract, set the current price of the contract in a security initializer .

```
PY
self.settings.seed_initial_prices = True
```

Supported Assets

To view the supported assets in the US Future Options dataset, see the Supported Assets .

Resolutions

The following table shows the available resolutions and data formats for Future Option contract subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
TICK				
SECOND				
MINUTE	✓	✓		
HOUR	✓	✓		
DAILY	✓	✓		

There is only one resolution option, so you don't need to pass a `resolution` argument to the `add_future_option_contract` method.

```
self.add_future_option_contract(self._option_contract_symbol, Resolution.MINUTE)
```

To create custom resolution periods, see Consolidating Data .

Supported Markets

The following `Market` enumeration members are available for Future Options:

You don't need to pass a `Market` argument to the `add_future_option_contract` method because the contract `Symbol` already contains the market.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self.add_future_option_contract(self._option_contract_symbol, fill_forward=False)
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. Future Options are already leveraged products, so you can't change their leverage.

Extended Market Hours

By default, your security subscriptions only cover regular trading hours. To subscribe to pre and post-market trading hours for a specific asset, enable the `extended_market_hours` argument when you create the security subscription.

```
self.add_future_option_contract(self._option_contract_symbol, extended_market_hours=True)
```

You only receive extended market hours data if you create the subscription with minute, second, or tick resolution. If you create the subscription with daily or hourly resolution, the bars only reflect the regular trading hours.

To view the schedule of regular and extended market hours, see Market Hours .

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Remove Subscriptions

To remove a contract subscription that you created with `add_future_option_contract` , call the `remove_option_contract` method. This method is an alias for `remove_security` .

```
self.remove_option_contract(self._option_contract_symbol)
```

The `remove_option_contract` method cancels your open orders for the contract and liquidates your holdings.

Properties

The `add_future_option_contract` method returns an `Option` object, which have the following properties:

Helper Methods

The `Option` object provides methods you can use for basic calculations. These methods require the underlying price. To get the `Option` object and the `Security` object for its underlying in any function, use the Option `symbol` to access the value in the `securities` object.

```py
option = self.securities[self._option_contract_symbol]
underlying = self.securities[self._option_contract_symbol.underlying]
underlying_price = underlying.price
```

To get the Option payoff , call the `get_pay_off` method.

```py
pay_off = option.get_pay_off(underlying_price)
```

To get the Option intrinsic value , call the `get_intrinsic_value` method.

```py
intrinsic_value = option.get_intrinsic_value(underlying_price)
```

To get the Option out-of-the-money amount , call the `out_of_the_money_amount` method.

```py
otm_amount = option.out_of_the_money_amount(underlying_price)
```

To check whether the Option can be automatic exercised, call the `is_auto_exercised` method.

```py
is_auto_exercised = option.is_auto_exercised(underlying_price)
```

Future Options

Handling Data

Introduction

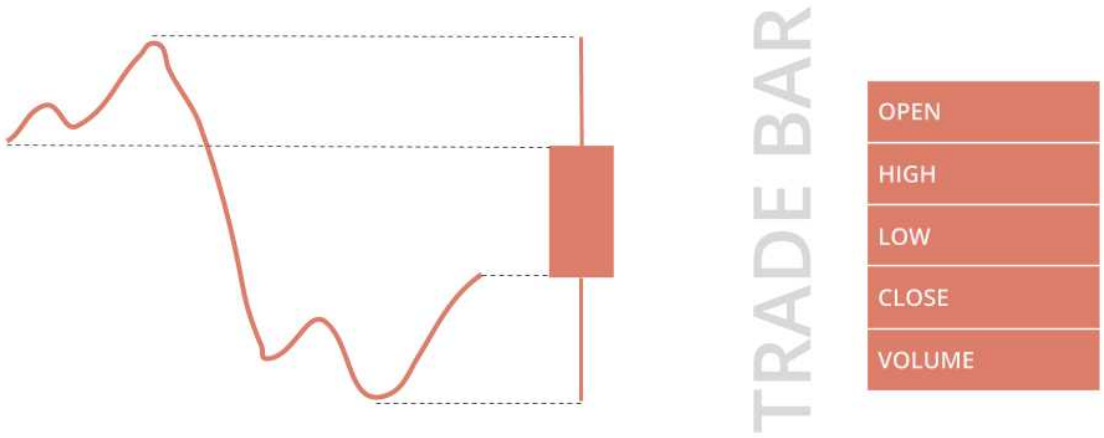
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the Futures Option contract ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Future Options data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the Option contract `symbol` . If the Option contract doesn't actively trade or you are in the same time step as when you added the Option contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your Option contract before you index the `Slice` with the Option contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._option_contract_symbol)   # None if not found
```

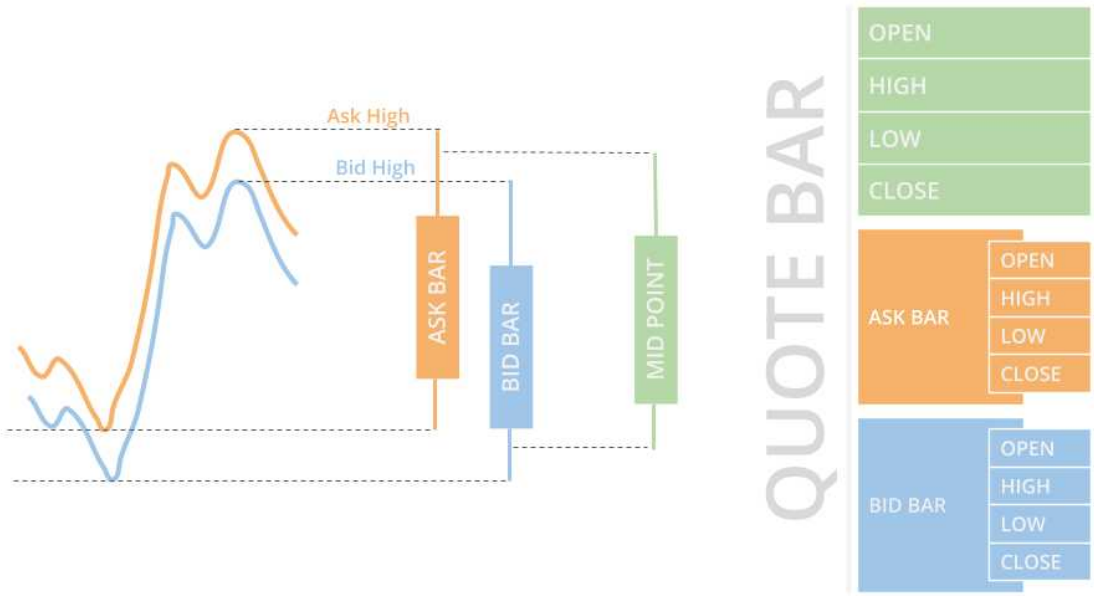
You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the Option contract `symbol` . If the Option contract doesn't actively get quotes or you are in the same time step as when you added the Option contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your Option contract before you index the `Slice` with the Option contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._option_contract_symbol)   # None if not found
```

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Futures Chains

`FuturesChain` objects represent an entire chain of contracts for a single underlying Future. To get the `FuturesChain` , index the `futures_chains` property of the `Slice` with the continuous contract `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the FutureChain using the canonical symbol (None if no FutureChain return)
    chain = slice.futures_chains.get(self._future_contract_symbol.canonical)
    if chain:
        # Get all contracts if the FutureChain contains any member
        contracts = chain.contracts
```

You can also loop through the `futures_chains` property to get each `FuturesChain` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-FutureChain key-value pairs
    for continuous_contract_symbol, chain in slice.futures_chains.items():
        contracts = chain.contracts
```

`FuturesChain` objects have the following properties:

Futures Contracts

`FuturesContract` objects represent the data of a single Futures contract in the market. To get the Futures contracts in the `Slice` , use the `contracts` property of the `FuturesChain` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the FutureChain using the canonical symbol
    chain = slice.future_chains.get(self._future_contract_symbol.canonical)
    if chain:
        # Get individual contract data (None if not contained)
        contract = chain.contracts.get(self._future_contract_symbol)
        if contract:
            price = contract.last_price
```

`FuturesContract` objects have the following properties:

Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , index the `option_chains` property of the `Slice` with the canonical `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol (None if no OptionChain return)
    chain = slice.option_chains.get(self._option_contract_symbol.Canonical)
    if chain:
        # Get all contracts if the OptionChain contains any member
        contracts = chain.contracts
```

You can also loop through the `option_chains` property to get each `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-OptionChain key-value pairs
    for canonical_fop_symbol, chain in slice.option_chains.items():
        contracts = chain.contracts
```

`OptionChain` objects have the following properties:

Option Contracts

`OptionContract` objects represent the data of a single Option contract in the market. To get the Option contracts in the `Slice` , use the `contracts` property of the `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._option_contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._option_contract_symbol)
        if contract:
            price = contract.price
```

You can also iterate through the `futures_chains` first.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-FuturesChain key-value pairs
    for continuous_future_symbol, futures_chain in slice.futures_chains.items():
        # Select a Future Contract and create its canonical FOP Symbol
        futures_contract = [contract for contract in futures_chain][0]
        canonical_fop_symbol = Symbol.create_canonical_option(futures_contract.symbol)
        # Try to get the OptionChain using the canonical FOP symbol
        option_chain = slice.option_chains.get(canonical_fop_symbol)
        if option_chain:
            # Get individual contract data
            option_contract = option_chain.contracts.get(self._option_contract_symbol)
            if option_contract:
                price = option_contract.price
```

Greeks and Implied Volatility

To get the Greeks and implied volatility of an Option contract, use the `greeks` and `implied_volatility` members.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._option_contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._option_contract_symbol)
        if contract:
            # Get greeks data of the selected contract
            delta = contract.greeks.delta
            iv = contract.implied_volatility
```

LEAN only calculates Greeks and implied volatility when you request them because they are expensive operations. If you invoke the `greeks` property, the Greeks aren't calculated. However, if you invoke the `greeks.delta` , LEAN calculates the delta. To avoid unecessary computation in your algorithm, only request the Greeks and implied volatility when you need them. For more information about the Greeks and implied volatility, see Options Pricing .

Open Interest

Open interest is the number of outstanding contracts that haven't been settled. It provides a measure of investor interest and the market liquidity, so it's a popular metric to use for contract selection. Open interest is calculated once per day. To get the latest open interest value, use the `open_interest` property of the `Option` or `option_contract` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the option_chains using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get the open interest of the selected contracts
            open_interest = contract.open_interest
```

Properties

`OptionContract` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Future Option data.

Example 1: Monthly Protective Put

The following algorithm shows how to perform monthly selection on individual ES Future Option contract to implement a protective put option strategy to hedge speculation on S&P500 Future. It is a useful tool to hedge the excessive risk on leverage using Futures to trade.

```
class FutureOptionExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000000)

        # Seed the security price to ensure the underlying price is available at the initial filtering.
        self.settings.seed_initial_prices = True
        # Set data normalization mode to raw for fair strike price comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        # Subscribe the underlying since the updated price is needed for filtering.
        self.underlying = self.add_future(Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.RAW,
            contract_depth_offset=0)
        # Filter the underlying continuous Futures to narrow the FOP spectrum.
        self.underlying.set_filter(0, 31)

        # Schedule a monthly event on selection of future-future option pair, since the portfolio rebalance on a monthly basis.
        self.schedule.on(
            self.date_rules.month_start(self.underlying.symbol),
            self.time_rules.after_market_open(self.underlying.symbol, 0),
            self.selection_and_rebalance
        )

    def selection_and_rebalance(self) -> None:
        # Get all available put FOP contract for the mapped underlying contract, since the trade liquidity and volatility is the highest.
        contract_symbols = self.option_chain(self.underlying.mapped)
        contract_symbols = [symbol for symbol in contract_symbols if symbol.id.option_right == OptionRight.PUT]
        # Select the ATM put expires the same date as the underlying. The max expiry of the FOP will expire the same time as the front
month future.
        expiry = max(symbol.id.date for symbol in contract_symbols)
        filtered_symbols = [symbol for symbol in contract_symbols if symbol.id.date == expiry]
        selected = sorted(filtered_symbols, key=lambda symbol: abs(symbol.id.strike_price -
self.securities[self.underlying.mapped].price))[0]
        # Request the FOP contract data for trading.
        contract = self.add_future_option_contract(selected)

        # A Protective Put consists of long a lot of the underlying, and long a put contract.
        self.market_order(self.underlying.mapped, contract.symbol_properties.contract_multiplier)
        self.market_order(contract.symbol, 1)
```

Example 2: Weekly Covered Call

The below example demonstrates a weekly-renewing covered call strategy to collect credit of selling the option. It filters the ATM call contract that expires within the current week at week start using `set_filter` filtering function.

```
class FutureOptionExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000000)
        # Set data normalization mode to raw for fair strike price comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW

        # Subscribe the underlying since the updated price is needed for filtering.
        self.underlying = self.add_future(Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.RAW,
            contract_depth_offset=0)
        # Filter the underlying continuous Futures to narrow the FOP spectrum.
        self.underlying.set_filter(0, 182)
        # Filter for the current-week-expiring calls to formulate a covered call that expires at the end of week.
        self.add_future_option(self.underlying.symbol, lambda u: u.calls_only().expiration(0, 5))

    def on_data(self, slice: Slice) -> None:
        # Create canonical symbol for the mapped future contract, since option chains are mapped by canonical symbol.
        symbol = Symbol.create_canonical_option(self.underlying.mapped)

        # Get option chain data for the mapped future, as both the underlying and FOP have the highest liquidity among all other
contracts.
        chain = slice.option_chains.get(symbol)
        if not self.portfolio.invested and chain:
            # Obtain the ATM call that expires at the end of week, such that both underlying and the FOP expires the same time.
            expiry = max(x.expiry for x in chain)
            atm_call = sorted([x for x in chain if x.expiry == expiry],
                key=lambda x: abs(x.strike - x.underlying_last_price))[0]

            # Use abstraction method to order a covered call to avoid manual error.
            option_strategy = OptionStrategies.covered_call(symbol, atm_call.strike,expiry)
            self.buy(option_strategy, 1)
```

Note that since both the underlying Future and the Future Option are expiring on the same day and are cash-settling in most cases, Lean can

exercise the Future Option into account cash automatically at expiry and we do not need to handle the option exercise/assignment event.

Future Options

Greeks and Implied Volatility

The Greeks are measures that describe the sensitivity of an Option's price to various factors like underlying price changes (Delta), time decay (Theta), volatility (Vega), and interest rates (Rho), while Implied Volatility (IV) represents the market's expectation of the underlying asset's volatility over the life of the Option.

Key Concepts

Concepts on Greeks, IV, and IV modeling

Indicators

Implementation of IV and Greeks as LEAN indicators

See Also

Option Pricing
Risk Free Interest Rate
Dividend Yield

Greeks and Implied Volatility

Key Concepts

Introduction

LEAN provides multiple ways for you to get the Greeks and implied volatility (IV) of Option contracts. There is a daily, pre-calculated dataset based on the end of the previous trading day. There is also a stream of values that are calculated on-the-fly as your algorithm runs.

Greeks on Universes

This feature is coming soon for Future Options.

Greek History

This feature is coming soon for Future Options.

Greeks on Data Events

The Greeks and IV values that you get from the current Slice are from the price model calculations. These values aren't the same as the values you get in the universe filter function.

```py
class BasicFutureOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        self.add_future_option(self._future.symbol, lambda universe: universe.strikes(-5, 5))

    def on_data(self, data):
        for chain in data.option_chains.values():
            for contract in chain:
                iv = contract.implied_volatility
                delta = contract.greeks.delta

    def _custom_security_initializer(self, security: Security) -> None:
        if security.type == SecurityType.FUTURE_OPTION:
            security.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

To view all the models LEAN supports, see Supported Models .

Greek Indicators

Option indicators provide daily and intraday values for the Greeks and implied volatility of Future Option contracts.

Examples

For detailed examples on using Option Greeks and Implied Volatility indicators in algorithm, you may refer to the Greeks Indicators page examples.

Greeks and Implied Volatility

Indicators

Introduction

Option indicators enable you to calculate daily and intraday values for the Greeks and implied volatility of Option contracts. This page explains the most popular Option indicators and how to use them with LEAN.

Parameters

The following table describes the arguments that the automatic Option indicators methods accept:

Argument	Data Type	Description	Default Value
`symbol`	`Symbol`	The contract to use when calculating the indicator values.	
`mirror_option`	`Symbol`	The mirror contract to use in parity type calculations.	`None`
`risk_free_rate`	`float`	The risk-free interest rate. If you don't provide a value, the default value is the US primary credit rate from the Interest Rate Provider Model .	`None`
`dividend_yield`	`float`	The dividend yield rate. If you don't provide a value, the default value comes from the Dividend Yield Provider Model .	`None`
`option_model`	`OptionPricingModelType`	The Option pricing model that's used to calculate the Greeks. If you don't provide a value, the default value is `OptionPricingModelType.BlackScholes` for European-style Options.	`None`
`resolution`	`Resolution`	The resolution of the indicator data. If you don't provide a value, the default value is the resolution of the subscription you have for the Option contract(s).	`None`

To perform implied volatility (IV) smoothing with a put-call pair, pass one of the contracts as the `symbol` argument and pass the other contract in the pair as the `mirror_option` argument. The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, call the `set_smoothing_function` method of the `ImpliedVolatility` class/property.

Several different Option pricing models are supported to calculate the IV and Greeks. The following table describes the `OptionPricingModelType` enumeration members:

Implied Volatility

Implied volatility, , is the market's expectation for the future volatility of an asset and is implied by the price of the assets's Options contracts. You can't observe it in the market but you can derive it from the price of an Option. For more information about implied volatility, see Implied Volatility .

Automatic Indicators

To create an automatic indicator for implied volatility, call the `QCAlgorithm.iv` method with the Option contract `symbol` object(s).

```py
class AutomaticImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            contract1.iv = self.iv(call, put)
            contract2.iv = self.iv(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the ImpliedVolatility indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.iv
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `iv` method, see Parameters and Using IV Indicator .

Manual Indicators

To create a manual indicator for implied volatility, call the `ImpliedVolatility` constructor.

```py
class ManualImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
```

```
    def Initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.ImpliedVolatility = ImpliedVolatility(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].ImpliedVolatility
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)
```

```
                # Get the current value of the ImpliedVolatility indicator.
                value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `ImpliedVolatility` constructor, see Using IV Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the `iv` method or `ImpliedVolatility` constructor and then call the `set_smoothing_function` method of the resulting `ImpliedVolatility` object. The follow table describes the arguments of the custom function:

Argument	Data Type	Description
`iv`	`float`	The IV of the Option contract.
`mirror_iv`	`float`	The IV of the mirror Option contract.

The method must return a `float` as the smoothened IV.

PY

```
def initialize(self):
    option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.PUT, 505, datetime(2014, 6, 27))
    self.add_option_contract(option)

    mirror_option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.CALL, 505, datetime(2014, 6, 27))
    self.add_option_contract(mirror_option)

    self._iv = self.iv(option, mirror_option)
    # Example: The average of the call-put pair.
    self._iv.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Delta

Delta, , is the rate of change of the Option price with respect to the price of the underlying asset. It measures the first-order sensitivity of the price to a movement in underlying price. For example, an Option delta of 0.4 means that if the underlying asset moves by 1%, then the value of the Option moves by 0.4 1% = 0.4%. For more information about delta, see Delta .

Automatic Indicators

To create an automatic indicator for delta, call the `QCAlgorithm.d` method with the Option contract `symbol` object(s).

```py
class AutomaticDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Delta indicators.
            contract1.d = self.d(call, put)
            contract2.d = self.d(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Delta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.d
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

The follow table describes the arguments that the `d` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Delta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `d` method, see Using D Indicator .

Manual Indicators

To create a manual indicator for delta, call the `Delta` constructor.

```py
class ManualDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Delta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Delta = Delta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)
```

```
    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Delta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Delta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `Delta` constructor, see Using D Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
delta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Gamma

Gamma, , is the rate of change of the portfolio's delta with respect to the underlying asset's price. It represents the second-order sensitivity of the Option to a movement in the underlying asset's price. For more information about Gamma, see Gamma .

Automatic Indicators

To create an automatic indicator for gamma, call the `QCAlgorithm.g` method with the Option contract `symbol` object(s).

```py
class AutomaticGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Gamma indicators.
            contract1.g = self.g(call, put)
            contract2.g = self.g(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Gamma indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.g
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

The follow table describes the arguments that the `g` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
iv_model	OptionPricingModelType	The Option pricing model to use to estimate the IV when calculating Gamma. If you don't provide a value, the default value is to match the option_model parameter.	None

For more information about the g method, see Using G Indicator .

Manual Indicators

To create a manual indicator for gamma, call the Gamma constructor.

```py
class ManualGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Gamma indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Gamma = Gamma(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)
```

```
    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Gamma
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Gamma indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `Gamma` constructor, see Using G Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
gamma_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Vega

Vega, , is the rate of change in the value of the Option with respect to the volatility of the underlying asset. For more information about vega, see Vega .

Automatic Indicators

To create an automatic indicator for vega, call the `QCAlgorithm.v` method with the Option contract `symbol` object(s).

PY

```
class AutomaticVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Vega indicators.
            contract1.v = self.v(call, put)
            contract2.v = self.v(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Vega indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.v
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

The follow table describes the arguments that the v method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
iv_model	OptionPricingModelType	The Option pricing model to use to estimate the IV when calculating Vega. If you don't provide a value, the default value is to match the option_model parameter.	None

For more information about the v method, see Using V Indicator .

Manual Indicators

To create a manual indicator for vega, call the Vega constructor.

```py
class ManualVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Vega indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Vega = Vega(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)
```

```
    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Vega
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Vega indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `Vega` constructor, see Using V Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
vega_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Theta

Theta, , is the rate of change of the value of the Option with respect to the passage of time. It is also known to as the time decay of an Option. For more information about theta, see Theta .

Automatic Indicators

To create an automatic indicator for theta, call the `QCAlgorithm.t` method with the Option contract `symbol` object(s).

```py
class AutomaticThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Theta indicators.
            contract1.t = self.t(call, put)
            contract2.t = self.t(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Theta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.t
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

The follow table describes the arguments that the `t` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating theta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `t` method, see Using T Indicator .

Manual Indicators

To create a manual indicator for theta, call the `Theta` constructor.

```py
class ManualThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Theta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Theta = Theta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)
```

```
    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Theta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Theta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `Theta` constructor, see Using T Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
# Example: Average IV of the call-put pair.
theta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Rho

Rho, , is the rate of change of the value of a derivative with respect to the interest rate . It is usually small and not a big issue in practice unless the Option is deep in-the-money and has a long horizon. In this case, the interest rate matters because you need to discount a larger cash flow over a longer horizon. For more information about rho, see Rho .

Automatic Indicators

To create an automatic indicator for rho, call the `QCAlgorithm.r` method with the Option contract `symbol` object(s).

```py
class AutomaticRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Rho indicators.
            contract1.r = self.r(call, put)
            contract2.r = self.r(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Rho indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.r
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

The follow table describes the arguments that the `r` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating rho If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `r` method, see Using R Indicator .

Manual Indicators

To create a manual indicator for rho, call the `Rho` constructor.

PY

```
class ManualRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        self._future.set_filter(0, 182)
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = ConstantDividendYieldModel(0)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._future.symbol),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._future.mapped is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._future.mapped, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_future_option_contract(call)
            contract2 = self.add_future_option_contract(put)

            # Create and save the automatic Rho indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Rho = Rho(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)
```

```
    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Rho
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                ).value
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Rho indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._future.mapped and self.portfolio[self._future.mapped].invested:
            self.liquidate(self._future.mapped)
```

For more information about the `Rho` constructor, see Using R Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
rho_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Examples

Demonstration Algorithms
OptionIndicatorsRegressionAlgorithm.py Python OptionIndicatorsMirrorContractsRegressionAlgorithm.py Python

Future Options

Market Hours

Introduction

The Future Option markets have the same hours as the Futures market. For more information, see the supported contracts and the market hours of Future markets .

Asset Classes

Index

There are cash Indices and volatility Indices. The former is a hypothetical portfolio that represents a segment of the financial market and the latter shows the expected level of price fluctuation in an Index Option.

Requesting Data

Handling Data

Market Hours

See Also

BasicTemplateIndexAlgorithm.py
BasicTemplateIndexAlgorithm.cs

Index

Requesting Data

Introduction

Request Index data in your algorithm to receive a feed of Index prices in the `on_data` method. For more information about the specific dataset we use for backtests, see the US Cash Indices dataset listing . To trade live with Index data, you can use one of the brokerage data providers .

Create Subscriptions

To create an Index subscription, in the `initialize` method, call the `add_index` method. The `add_index` method returns an `Index` object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_index("VIX").symbol
```

PY

To view the supported assets in the US Cash Indices dataset, see the Supported Indices .

Resolutions

The following table shows the available resolutions and data formats for Index subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
`TICK`				
`SECOND`				
`MINUTE`	✓			
`HOUR`	✓			
`DAILY`	✓			

The default resolution for Index subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_index` method.

```
self._symbol = self.add_index("VIX", Resolution.DAILY).symbol
```

PY

To create custom resolution periods, see Consolidating Data .

Supported Markets

The only market available for Indices is `Market.USA` , so you don't need to pass a `market` argument to the `add_index` method.

```
self._symbol = self.add_index("VIX", market=Market.USA).symbol
```

PY

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self._symbol = self.add_index("VIX", fill_forward=False).symbol
```

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Properties

The `add_index` method returns an `Index` object, which have the following properties:

Examples

The following examples demonstrate some common practices for requesting Index data.

Example 1: VIX EMA Cross

The following algorithm adds daily VIX Index data and uses the Exponential Moving Average (EMA) indicator to create trading signals. The algorithm uses a 30-day slow EMA and a 15-day fast EMA. It enters a long SPY position when the fast EMA crosses above the slow EMA and liquidates the position when the fast EMA crosses below the slow EMA.

```
class IndexDemoAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the daily VIX and SPY data.
        self._vix = self.add_index("VIX", Resolution.DAILY).symbol
        self._spy = self.add_equity("SPY", Resolution.DAILY).symbol
        # Create the fast and slow EMA indicators.
        self._ema_fast = self.ema(self._vix, 15)
        self._ema_slow = self.ema(self._vix, 30)
        # Define a small tolerance to reduce trading churn.
        self._tolerance = 0.00015

    def on_data(self, slice: Slice) -> None:
        # Check if the indicators are ready.
        if not self._ema_slow.is_ready:
            return
        # When the fast VIX EMA crosses above the slow EMA, buy SPY.
        if (not self.portfolio.invested and
            self._ema_fast.current.value > self._ema_slow.current.value * (1 + self._tolerance)):
            self.set_holdings(self._spy, 0.8)
        # When the fast VIX EMA crosses below the slow EMA, liquidate the position.
        elif (self.portfolio.invested and
            self._ema_fast.current.value < self._ema_slow.current.value * (1 - self._tolerance)):
            self.liquidate(self._spy)
```

Index

Handling Data

Introduction

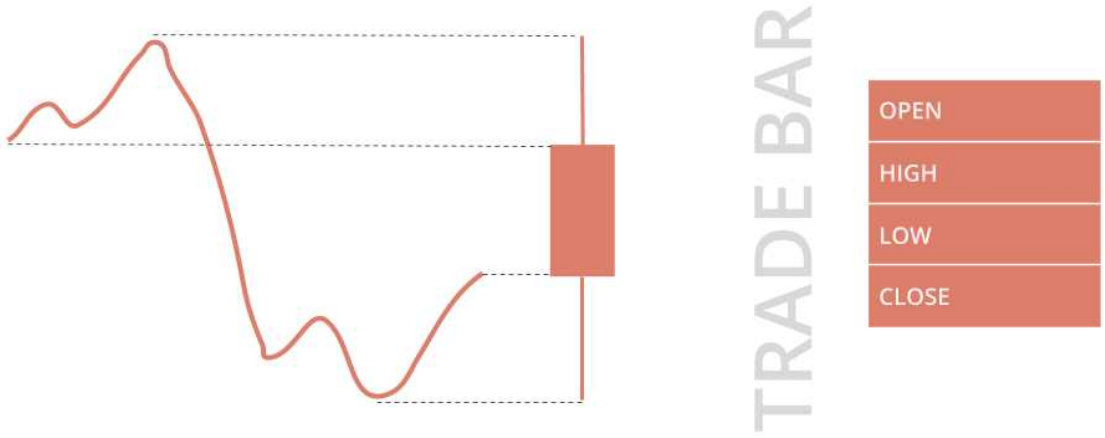
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `Ticks` `DataDictionary` is made up of `Tick` objects. To access individual data points in the dictionary, you can index the dictionary with the Index ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Index data, see Resolutions .

Bars

You can't trade Indices, but `TradeBar` objects are bars that represent the open, high, low, and close of an Index price over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the Index `Symbol` . The `Slice` may not contain data for your `Symbol` at every time step. To avoid issues, check if the `Slice` contains data for your Index before you index the `Slice` with the Index `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        trade_bar = slice.bars[self._symbol]
        value = trade_bar.value
```

`TradeBar` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Index data.

Example 1: Trading ES Futures Based on the SPX Index RSI

The following algorithm adds daily data for the SPX Index and uses the Relative Strength Index (RSI) indicator to generate trading signals for the E-Mini S&P 500 Future. When the RSI crosses below 30, it buys the front-month ES contract. When the RSI crosses above 70, it liquidates the portfolio.

PY

```
class IndexDemoAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add daily SPX Index data.
        self._spx = self.add_index("SPX", Resolution.DAILY)
        # Create an RSI indicator for the SPX index.
        self._spx.rsi = self.rsi(self._spx.symbol, 15)
        # Add the ES front-month Futures contract.
        self._es = self.add_future(Futures.Indices.SP_500_E_MINI, data_mapping_mode=DataMappingMode.OPEN_INTEREST)
        self._es.set_filter(lambda future_filter_universe: future_filter_universe.standards_only().front_month())

    def on_data(self, slice: Slice) -> None:
        # Check if the indicator is ready and we have the front-month contract data.
        if not self._spx.rsi.is_ready or self._es.mapped not in slice.bars:
            return

        # If the SPX RSI is <30, buy the ES.
        if not self.portfolio.invested and self._spx.rsi.current.value < 30:
            self.set_holdings(self._es.mapped, 0.5)

        # If the SPX RSI is >70, liquidate the portfolio.
        elif self.portfolio.invested and self._spx.rsi.current.value > 70:
            self.liquidate()
```

Index

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the US Indices market.

Search by symbol (e.g. SPX, V

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

Index Options

Index Options are a financial derivative that gives the holder the right (but not the obligation) to buy or sell the value of an underlying Index, such as the S&P 500 index, at the stated exercise price. No actual assets are bought or sold.

Requesting Data

Handling Data

Greeks and Implied Volatility

Market Hours

See Also

BasicTemplateIndexOptionsAlgorithm.py
BasicTemplateIndexOptionsAlgorithm.cs

Index Options

Requesting Data

Request Index Options data in your algorithm to receive a feed of contract prices in the `on_data` method. For more information about the specific dataset we use for backtests, see the US Index Options dataset listing . To trade Index Options live, you can use one of the brokerage data providers . Index Options are exclusively European-style Options.

To request Index Options data, call one of the following methods:

- `self.add_index_option` to add a basket of contracts for a single underlying Index
- `self.add_index_option_contract` to add individual contracts for an underlying Index

Request and Filter Contracts for Option Strategies

```
self.add_index_option
```

Request and Filter All Contracts

```
self.add_index_option
```

Select Individual Contracts

```
self.add_index_option_contract
```

See Also

Introduction to Options
Applied Options

Requesting Data

Universes

Introduction

The `add_index_option` method enables you to select a basket of Option contracts for an underlying Index. To form your universe of contracts, you can filter them down by their strike price and expiry or you can choose to a subset of the contracts that form popular Option strategies . If you want to subscribe to individual contracts one-by-one instead of a set of contracts, see Individual Contracts .

Create Universes

To add a universe of Index Option contracts, in the `initialize` method, call the `add_index_option` method. This method returns an `Option` object, which has a `set_filter` method you can call to filter the set of tradable contract down to just the contracts you want.

```
PY
class BasicIndexOptionAlgorithm(QCAlgorithm):
    def initialize(self):
        self.universe_settings.asynchronous = True
        option = self.add_index_option("SPX","SPXW")
        option.set_filter(self._filter)
        self._symbol = option.symbol

    def _filter(self, universe):
        return universe.expiration(0, 7).delta(0.35, 0.75)

    def on_data(self, data):
        chain = data.option_chains.get(self._symbol)
        if chain:
            contract = sorted(chain, key=lambda x: (x.expiry, x.greeks.delta))[0]
```

For more information about the `add_index_option` method, see Create Universes .

Filter by Investment Strategy

Options trading strategies consist of simultaneously buying or selling one or more Option contracts of the same underlying Index with different features. The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for Option strategies.

The following table describes the filter methods of the `OptionFilterUniverse` class that select contracts for Option strategies:

`naked_call(min_days_till_expiry: int, strike_from_atm: float)` Selects a call contract to form Naked Call , Covered Call , or Protective Call Option strategies.
`naked_put(min_days_till_expiry: int, strike_from_atm: float)` Selects a put contract to form Naked Put , Covered Put , or Protective Put Option strategies.
`call_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two call contracts to form Bull Call Spread or Bear Call Spread Option strategies.
`put_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two put contracts to form Bull Put Spread or Bear Put Spread Option strategies.
`call_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two call contracts to form Long Call Calendar Spread or Short Call Calendar Spread Option strategies.

`put_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two put contracts to form Long Put Calendar Spread or Short Put Calendar Spread Option strategies.
`strangle(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Long Strangle or Short Strangle Option strategies.
`straddle(min_days_till_expiry: int)` Selects two contracts to form Long Straddle or Short Straddle Option strategies.
`protective_collar(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Protective Collar Option strategies.
`conversion(min_days_till_expiry: int, strike_from_atm: float)` Selects two contracts to form Conversion or Reverse Conversion Option strategies.
`call_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Call Butterfly or Short Call Butterfly Option strategies.
`put_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Put Butterfly or Short Put Butterfly Option strategies.
`iron_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Long Iron Butterfly or Short Iron Butterfly Option strategies.
`iron_condor(min_days_till_expiry: int, near_strike_spread: float, far_strike_spread: float)` Selects four contracts to form Long Iron Condor or Short Iron Condor Option strategies.
`box_spread(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Box Spread or Short Box Spread Option strategies.
`jelly_roll(strike_from_atm: float, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects four contracts to form Jelly Roll or Short Jelly Roll Option strategies.
`call_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Call Ladder or Bull Call Ladder Option strategies.
`put_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Put Ladder or Bull Put Ladder Option strategies.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```py
# Example 1: Select a Straddle
option.set_filter(lambda option_filter_universe: option_filter_universe.straddle(30))

# Example 2: Select the contracts (including weeklys) that expire in the next 30 days that form an Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 30).iron_condor(30, 5, 10))

# Example 3: Select the 0DTE contracts that form an Strangle
option.set_filter(lambda option_filter_universe: option_filter_universe.expiration(0, 0).strangle(30, 5, -10))
```

Filter by Implied Volatility and Greeks

Option price models compute the theoretical price of Option contracts, their implied volatility, and their Greek values. Theoretical prices can help you detect undervalued and overvalued contracts, implied volatility can provide you insight into the upcoming volatility of the underlying security, and Greek values can help you hedge your portfolio. The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for a range of implied volatility, Greek values, and open interest.

The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for a range of implied volatility, Greek values, and open interest:

`iv(min: float, max: float)`
`implied_volatility(min: float, max: float)`
 	
Selects a contract with implied volatility within the range you set.

`d(min: float, max: float)`
`delta(min: float, max: float)`
 	
Selects a contract with delta within the range you set.

`g(min: float, max: float)`
`gamma(min: float, max: float)`
 	
Selects a contract with gamma within the range you set.

`r(min: float, max: float)`
`rho(min: float, max: float)`
 	
Selects a contract with rho within the range you set.

`v(min: float, max: float)`
`vega(min: float, max: float)`
 	
Selects a contract with vega within the range you set.

`t(min: float, max: float)`
`theta(min: float, max: float)`
 	
Selects a contract with theta within the range you set.

`oi(min: int, max: int)`
`open_interest(min: float, max: float)`
 	
Selects a contract with open interest within the range you set.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```py
# Example 1: Select the contracts with delta between 0.25 and 0.75
option.set_filter(lambda option_filter_universe: option_filter_universe.delta(0.25, 0.75))

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days with implied volatility below 20%
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 90))

# Example 3: Select the contracts (including weeklys) that expire in the next 30 days with implied volatility below 20% that forms an
Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 30).iron_condor(30, 5, 10))
```

Filter by Other Contract Properties

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Option` object.

```py
# Select contracts that have a strike price within 1 strike level above and below the underlying price
option.set_filter(min_strike=-1, max_strike=1)

# Select contracts that expire within 30 days
option.set_filter(min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select contracts that have a strike price within 1 strike level and expire within 30 days
option.set_filter(min_strike=-1, max_strike=1, min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only())
```

The following table describes the available filter techniques:

`set_filter(minStrike: int, maxStrike: int)` Selects the contracts that have a strike price within a minimum and maximum strike level relative to the underlying price. For example, say the underlying price is 302 and there are strikes at every 5. If you set `m_strike` to -1 and `max_strike` to 1, LEAN selects the contracts that have a strike of 300 or 305. This filter runs asynchronously by default.
`set_filter(minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(minStrike: int, maxStrike: int, minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire and have a strike within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[OptionFilterUniverse], OptionFilterUniverse])` Selects the contracts that a function selects.

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
# Example 1: Select the front month call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only().front_month())

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 90))
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Default Filter

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard and weeklys
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at the first bar of every day. When the filter selects a contract that isn't currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

Historical Data

To get historical chains for an Index Option, see Historical Data .

Examples

The following examples demonstrate some common practices for requesting Index Options universe data.

Example 1: Selecting for 0DTE Contracts

0DTE Options are option contracts that expire on the same day they are traded. To create a universe with 0DTE options, call the or `set_filter` method. In this example, we will select 0DTE contracts within 1 strike price level and a Delta between 0.25 and 0.75. Then, we will go long on the highest strike price call contract of the SPX index and hold until expiry.

```py
class BasicIndexOptionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        # Subscribe to the option chain.
        self._option = self.add_index_option("SPX", "SPXW")
        # Filter the option universe to only select 0DTE options.
        self._option.set_filter(lambda u: u.expiration(0, 0).strikes(-1, 1))
        # Filter the option universe by Delta. The last set_filter call prevails.
        # self._option.set_filter(lambda option_filter_universe: option_filter_universe.delta(0.25, 0.75))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return
        # Get the option chain data.
        chain = slice.option_chains.get(self._option.symbol)
        if not chain:
            return
        # Sorted the call Option contracts according to their strike prices.
        calls = sorted([contract for contract in chain if contract.right == OptionRight.CALL], key=lambda x: x.strike)
        if not calls:
            return
        # Buy 1 0DTE call option contract for the SPX index.
        self.Buy(calls[0].symbol, 1)
```

Requesting Data

Individual Contracts

Introduction

The `add_option_contract` method enables you to add an individual Option contract to your algorithm. To check which contracts are currently available to add to your algorithm, use the `option_chain` method. If you want to subscribe to a set of contracts instead of individual contracts one-by-one, see Universes .

Create Subscriptions

Before you can subscribe to an Index Option contract, you must configure the underlying Index and get the contract `symbol` .

```
class BasicIndexOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._underlying = self.add_index("SPX").symbol
        self._contract_symbol = None

    def on_data(self, data):
        if self._contract_symbol:
            return
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        expiry = chain.expiry.min()
        self._contract_symbol = chain[
            (chain.expiry == expiry) &
            (chain.right == OptionRight.CALL) &
            (chain.delta < 0.7) &
            (chain.delta > 0.3)
        ].sort_values('openinterest').index[-1]
        self.add_option_contract(self._contract_symbol)
```

PY

Configure the Underlying Index

In most cases, you should subscribe to the underlying Index before you subscribe to an Index Option contract.

```
self._underlying = self.add_index("SPX").symbol
```

PY

Get Contract Symbols

To subscribe to an Option contract, you need the contract `Symbol` . The preferred method to getting Option contract `Symbol` objects is to use the `option_chain` method. This method returns an `OptionChain` object, which represent an entire chain of Option contracts for a single underlying security. You can even format the chain data into a DataFrame where each row in the DataFrame represents a single contract. With the chain, sort and filter the data to find the specific contract(s) you want to trade.

```
# Get the contracts available to trade (in DataFrame format).
#   Option A: Standard contracts.
chain = self.option_chain(
    Symbol.create_canonical_option(self._underlying, Market.USA, "?SPX"), flatten=True
).data_frame

#  Option B: Weekly contracts.
#chain = self.option_chain(
#    Symbol.create_canonical_option(self._underlying, "SPXW", Market.USA, "?SPXW"), flatten=True
#).data_frame

# Select a contract.
expiry = chain.expiry.min()
self._contract_symbol = chain[
    # Select call contracts with the closest expiry.
    (chain.expiry == expiry) &
    (chain.right == OptionRight.CALL) &
    # Select contracts with a 0.3-0.7 delta.
    (chain.delta < 0.7) &
    (chain.delta > 0.3)
    # Select the contract with the largest open interest.
].sort_values('openinterest').index[-1]
```

Alternatively, you can create a Index Option contract `Symbol` object directly parsing its OSI ticker using the `SymbolRepresentation.parse_option_ticker_osi` method.

```
# Create the contract Symbol for the SPX Call 5650 3 Sept 2024.
self._contract_symbol = SymbolRepresentation.parse_option_ticker_osi('SPXW 240903C05650000', SecurityType.INDEX_OPTION,
OptionStyle.EUROPEAN, Market.USA)
```

Subscribe to Contracts

To create an Index Option contract subscription, pass the contract `Symbol` to the `add_index_option_contract` method. Save a reference to the contract `Symbol` so you can easily access the contract in the OptionChain that LEAN passes to the `on_data` method. To override the default pricing model of the Option, set a pricing model .

```
option = self.add_index_option_contract(self._contract_symbol)
option.PriceModel = OptionPriceModels.black_scholes()
```

The `add_index_option_contract` method creates a subscription for a single Index Option contract and adds it to your user-defined universe. To create a dynamic universe of Index Option contracts, add an Index Option universe .

Warm Up Contract Prices

If you subscribe to an Index Option contract with `add_index_option_contract` , you'll need to wait until the next `Slice` to receive data and trade the contract. To trade the contract in the same time step you subscribe to the contract, set the current price of the contract in a security initializer .

```
self.settings.seed_initial_prices = True
```

Supported Assets

To view the supported assets in the US Index Options dataset, see Supported Assets .

Resolutions

The following table shows the available resolutions and data formats for Index Option contract subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
TICK				
SECOND				
MINUTE	✓	✓		
HOUR	✓	✓		
DAILY	✓	✓		

The default resolution for Index Option subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_index_option_contract` method.

```
self.add_index_option_contract(self._contract_symbol, Resolution.HOUR)
```

To create custom resolution periods, see Consolidating Data .

Supported Markets

The only market available for Index Options is `Market.USA` . You don't need to pass a `market` argument to the `add_index_option_contract` method because the contract `Symbol` already contains the market.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self.add_index_option_contract(self._contract_symbol, fill_forward=False)
```

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. Options are already leveraged products, so you can't change their leverage.

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Remove Subscriptions

To remove a contract subscription that you created with `add_index_option_contract` , call the `remove_option_contract` method. This method is an alias for `remove_security` .

```
self.remove_option_contract(self._contract_symbol)
```

The `remove_option_contract` method cancels your open orders for the contract and liquidates your holdings.

Properties

The `add_index_option_contract` method returns an `Option` object, which have the following properties:

Helper Methods

The `Option` object provides methods you can use for basic calculations. These methods require the underlying price. To get the `Option` object and the `Security` object for its underlying in any function, use the Option `symbol` to access the value in the `securities` object.

```
option = self.securities[self._contract_symbol]
underlying = self.securities[self._contract_symbol.underlying]
underlying_price = underlying.price
```

To get the Option payoff , call the `get_pay_off` method.

```
pay_off = option.get_pay_off(underlying_price)
```

To get the Option intrinsic value , call the `get_intrinsic_value` method.

```
intrinsic_value = option.get_intrinsic_value(underlying_price)
```

To get the Option out-of-the-money amount , call the `out_of_the_money_amount` method.

```
otm_amount = option.out_of_the_money_amount(underlying_price)
```

To check whether the Option can be automatic exercised, call the `is_auto_exercised` method.

```
is_auto_exercised = option.is_auto_exercised(underlying_price)
```

Exceptions and Edge Cases

The following sections explain exceptions and edge cases with subscribing to individual Option contracts.

Manually Creating Option Symbol Objects

To subscribe to an Option contract, you need the contract `Symbol` . To get Index Option contract `Symbol` objects, call the `create_option` or `option_chain` methods. If you use the `create_option` method, you need to know the specific contract details.

```
# Standard contracts
self._contract_symbol = Symbol.create_option(self._underlying, Market.USA,
    OptionStyle.EUROPEAN, OptionRight.CALL, 3650, datetime(2022, 6, 17))

# Weekly contracts
self._weekly_contract_symbol = Symbol.create_option(self._underlying, "SPXW", Market.USA,
    OptionStyle.EUROPEAN, OptionRight.CALL, 3650, datetime(2022, 6, 17))
```

Default Underlying Subscription Settings

If you subscribe to an Index Option contract but don't have a subscription to the underlying Index, LEAN automatically subscribes to the underlying Index and sets its fill forward property to match that of the Index Option contract. In this case, you still need the Index `symbol` to subscribe to Index Option contracts. If you don't have access to it, create it.

```
self._underlying = Symbol.create("SPX", SecurityType.INDEX, Market.USA)
```

Overriding the Initial Implied Volatility Guess

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

Examples

The following examples demonstrate common practices for requesting individual index option contract data.

Example 1: 5-Minute Option Chain

The following example shows how to update the Option chain every five minutes. The custom `OptionChainManager` class implements the selection

logic and manages the contract subscriptions.

```py
class OptionChainFullExample(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 10, 31)
        self.set_cash(500000)
        self.universe_settings.asynchronous = True
        self.universe_settings.minimum_time_in_universe = timedelta(minutes=0)

        # Seed the last price to ensure filtering using the underlying price is available immediately.
        self.settings.seed_initial_prices = True

        # Request the index underlying data for filtering.
        spx = self.add_index("SPX").symbol
        # Create the 5-minute option chain manager on SPXW weekly index option.
        self._chain_manager = {
            Symbol.create_canonical_option(spx, "SPXW", Market.USA, "?SPXW"): OptionChainManager(-10, 10, 0, 1)
        }
        # Populate the updated option chain immediately to trade with.
        self._populate_option_chain()

        # Set schedule event to populate the option chain at market open, since the option contracts are updating daily.
        self.schedule.on(
            self.date_rules.every_day(spx), self.time_rules.after_market_open(spx, 1), self._populate_option_chain
        )
        # Set a scheduled event to filter the closed ATM calls every 5 minutes.
        self.schedule.on(self.date_rules.every_day(spx), self.time_rules.every(timedelta(minutes=5)), self._filter)

    def _populate_option_chain(self) -> None:
        # The contract list is updated daily, so we can get it and apply
        # the expiration filter as soon as the market opens.
        for symbol, manager in self._chain_manager.items():
            manager.set_chain(self.option_chain(symbol), self.time)
        self._filter()

    def _filter(self) -> None:
        for symbol, manager in self._chain_manager.items():
            manager.select(self, symbol)

    def on_data(self, slice: Slice) -> None:
        for symbol, _ in self._chain_manager.items():
            # Only trade on updated data.
            chain = slice.option_chains.get(symbol)
            if not chain:
                continue

            # Filter the closest ATM call contract and trade.
            expiry = min([x.expiry for x in chain])
            contracts = [
                x for x in chain
                if x.expiry == expiry and x.right == OptionRight.CALL and self.securities[x.symbol].is_tradable
            ]
            if not contracts:
                continue
            atm_call = sorted(contracts, key=lambda x: abs(chain.underlying.price-x.strike))[0]
            if not self.portfolio[atm_call.symbol].invested:
                self.market_order(atm_call.symbol, 1)


class OptionChainManager:
    _chain = []
    _symbols = set([])

    def __init__(self, min_strike: int, max_strike: int, min_expiry: int, max_expiry: int) -> None:
        self._min_strike = min_strike
        self._max_strike = max_strike
        self._min_expiry = min_expiry
        self._max_expiry = max_expiry

    def set_chain(self, chain:OptionChain, time: datetime) -> None:
        # Filter the expiry daily only since contract list is updated daily.
        self._chain = [x for x in chain if self._min_expiry <= (x.expiry - time).days <= self._max_expiry]

    def select(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        if not self._chain:
            return
        if symbol.is_canonical():
            symbol = symbol.underlying

        # Filter the contracts with strike range spread between the preset level.
        strikes = sorted(set(x.strike for x in self._chain))
        spot = algorithm.securities[symbol].price
        atm = sorted(strikes, key=lambda x: abs(spot-x))[0]
        index = strikes.index(atm)
        min_strike = strikes[max(0, index + self._min_strike)]
        max_strike = strikes[min(len(strikes) - 1, index + self._max_strike)]
        symbols = set(x.symbol for x in self._chain if min_strike <= x.strike <= max_strike)

        # Remove subscriptions if the contracts are not being ATM anymore.
        for symbol in self._symbols - symbols:
```

```
        for symbol in self._symbols - symbols:
            if algorithm.remove_option_contract(symbol):
                self._symbols.remove(symbol)
        # Request data of the newly identified ATM contracts.
        for symbol in symbols - self._symbols:
            self._symbols.add(symbol)
            algorithm.add_option_contract(symbol)
```

Index Options

Handling Data

Introduction

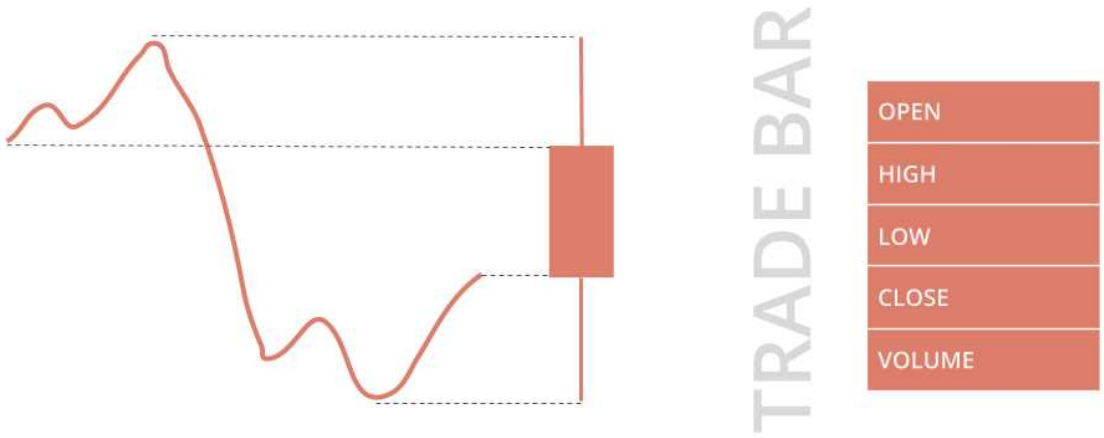
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `TradeBars` `DataDictionary` is made up of `TradeBar` objects. To access individual data points in the dictionary, you can index the dictionary with the contract ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for Index Options data, see Resolutions .

Trades

`TradeBar` objects are price bars that consolidate individual trades from the exchanges. They contain the open, high, low, close, and volume of trading activity over a period of time.



To get the `TradeBar` objects in the `Slice` , index the `Slice` or index the `bars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively trade or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped TradeBar of the symbol if any
    trade_bar = slice.bars.get(self._contract_symbol)   # None if not found
```

PY

You can also iterate through the `TradeBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `TradeBar` objects.

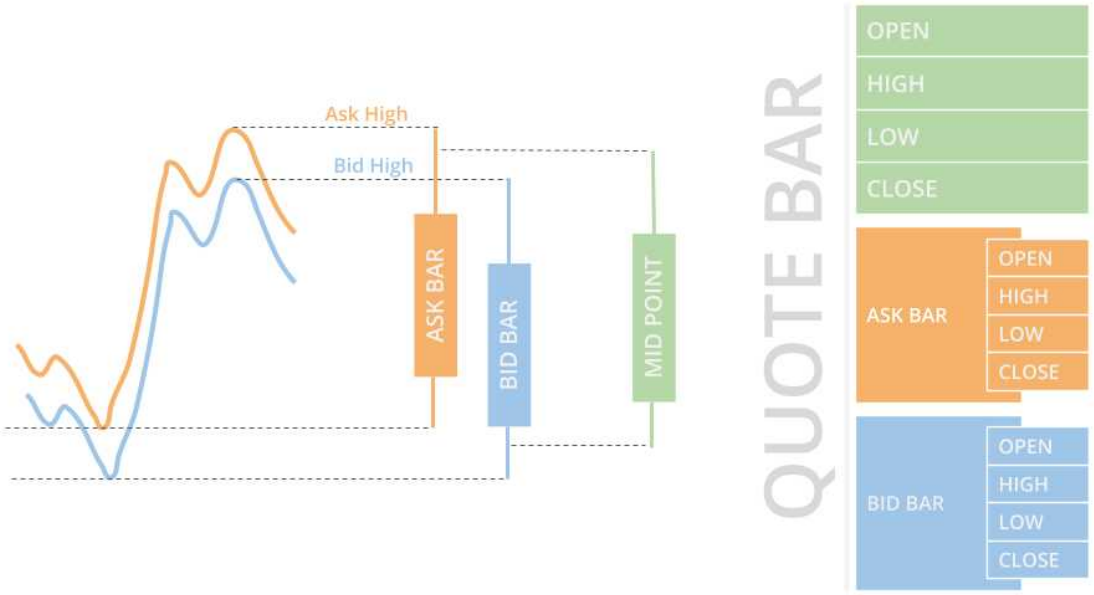
```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-TradeBar key-value pairs
    for symbol, trade_bar in slice.bars.items():
        close_price = trade_bar.close
```

PY

`TradeBar` objects have the following properties:

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the contract `symbol` . If the contract doesn't actively get quotes or you are in the same time step as when you added the contract subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your contract before you index the `Slice` with the contract `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._contract_symbol)   # None if not found
```

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , index the `option_chains` property of the `Slice` with the canonical `Symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol (None if no OptionChain return)
    chain = slice.option_chains.get(self._contract_symbol.Canonical)
    if chain:
        # Get all contracts if the OptionChain contains any member
        contracts = chain.contracts
```

You can also loop through the `option_chains` property to get each `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-OptionChain key-value pairs
    for canonical_symbol, chain in slice.option_chains.items():
        contracts = chain.contracts
```

`OptionChain` objects have the following properties:

Option Contracts

`OptionContract` objects represent the data of a single Option contract in the market. To get the Option contracts in the `Slice` , use the `contracts` property of the `OptionChain` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            price = contract.price
```

Greeks and Implied Volatility

To get the Greeks and implied volatility of an Option contract, use the `greeks` and `implied_volatility` members.

```
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get greeks data of the selected contract
            delta = contract.greeks.delta
            iv = contract.implied_volatility
```

LEAN only calculates Greeks and implied volatility when you request them because they are expensive operations. If you invoke the `greeks` property, the Greeks aren't calculated. However, if you invoke the `greeks.delta` , LEAN calculates the delta. To avoid unecessary computation in your algorithm, only request the Greeks and implied volatility when you need them. For more information about the Greeks and implied volatility, see Options Pricing .

Open Interest

Open interest is the number of outstanding contracts that haven't been settled. It provides a measure of investor interest and the market liquidity, so it's a popular metric to use for contract selection. Open interest is calculated once per day. To get the latest open interest value, use the `open_interest` property of the `Option` or `option_contract` .

```
def on_data(self, slice: Slice) -> None:
    # Try to get the option_chains using the canonical symbol
    chain = slice.option_chains.get(self._contract_symbol.canonical)
    if chain:
        # Get individual contract data
        contract = chain.contracts.get(self._contract_symbol)
        if contract:
            # Get the open interest of the selected contracts
            open_interest = contract.open_interest
```

Properties

`OptionContract` objects have the following properties:

Examples

The following examples demonstrate some common practices for handling Index Options.

Example 1: Supply-Demand Pressure Of Individual Contract

This example shows how to handle `QuoteBar` data for the SPXW Index Option contracts to trade based on the supply-demand pressure, while filter individual option contracts and request data using `self.option_chain` method for the contracts that expire that day.

```py
class IndexOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000000)

        self.contracts = []

        # Subscribe to underlying data for ATM calculation using the updated underlying price.
        # Set data normalization mode to raw is required to ensure the strike and underlying prices are comparable.
        self.spxw = self.add_index_option("SPX", "SPXW")
        self.spxw.set_filter(lambda x: x.expiration(0, 3).strikes(-5, 5))

        # Update the tradable contracts daily before the market opens since the option contract list provider populates them daily.
        self.schedule.on(
            self.date_rules.every_day(self.spxw.symbol),
            self.time_rules.at(9, 0),
            self.update_contracts
        )

    def update_contracts(self) -> None:
        # Get all contracts that expire today, and subscribe to data for trading needs.
        # We do not need to liquidate or handle settlement since all contracts are cash-settled after market close.
        # We only get the strike contracts within $10 of the underlying for higher liquidity and less slippage.
        contracts = self.option_chain(self.spxw.symbol)
        self.contracts = [x.symbol for x in contracts
            if x.expiry < self.time + timedelta(1) and\
            abs(x.underlying_last_price - x.strike) < 10]

    def on_data(self, slice: Slice) -> None:
        # Only focus on a filtered list of option contracts to trade and select the ATM call.
        calls = [x for x in self.contracts if x.id.option_right == OptionRight.CALL]
        if not calls:
            return
        contract = sorted(
            calls,
            key=lambda x: abs(self.securities[x.underlying].price - x.id.strike_price)
        )[0]

        # Trade based on the updated data.
        quote = slice.quote_bars.get(contract)
        if quote and quote.bid is not None and quote.ask is not None and not self.portfolio.invested:
            # if the last trading price is close to the ask price, the sell pressure is higher, so we short.
            diff = quote.ask.close - 2 * quote.close + quote.bid.close
            if diff < 0:
                self.set_holdings(contract, -0.05)
            else:
                self.set_holdings(contract, 0.05)
```

Example 2: Supply-Demand Pressure Of Universe

This example shows how to handle `QuoteBar` data for shortlisted Index Option contracts to calculate mid-price using bid close and ask close data, while request data through universe selection function using `set_filter` method for the contracts that expire within the current week. Using mid-price, we can examine the market fair value of the Option and compare it with the model theoretical price.

```py
class IndexOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 15)
        self.set_cash(100000000)

        # Subscribe to SPXW option data.
        option = self.add_index_option("SPX", "SPXW")
        self._symbol = option.symbol
        # We wish only to trade the contracts expiring within the same week since they have the highest volume.
        option.set_filter(lambda u: u.contracts(lambda x: [s for s in x if s.id.date <= Expiry.end_of_week(self.time)]))

    def on_data(self, slice: Slice) -> None:
        # Only want to obtain the option chain of the selected symbol.
        chain = slice.option_chains.get(self._symbol)
        if not chain:
            return

        # Select the ATM call to trade.
        contract = sorted(
            [x for x in chain if x.right == OptionRight.CALL],
            key=lambda x: abs(x.underlying_last_price - x.strike)
        )[0]

        # If the last trading price is close to the asking price, the selling pressure is higher, so we are short.
        diff = contract.ask_price - 2 * contract.last_price + contract.bid_price
        if diff < 0:
            self.set_holdings(contract, -0.05)
        else:
            self.set_holdings(contract, 0.05)
```

Example 3: Get Instant Delta

The option Greeks change rapidly, so we need to obtain the instant Greeks to calculate the hedge size for arbitration or replication portfolio accurately. You can call the `Greeks` property from the `OptionChain` data to access various Greeks. In this example, we will demonstrate how to obtain a contract with a Delta closest to 0.4 among all call contracts expiring the same week.

```py
class IndexOptionHandlingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Subscribe to SPXW option data.
        option = self.add_index_option("SPX", "SPXW")
        self._symbol = option.symbol
        # We wish only to trade the contracts expiring within the same week since they have the highest volume.
        # 0.4 Delta will only appear in call contracts.
        option.set_filter(lambda u: u.delta(0.2, 0.6).contracts(lambda x: [s for s in x if s.id.date <= Expiry.end_of_week(self.time)]))

    def on_data(self, slice: Slice) -> None:
        # Only want to obtain the option chain of the selected symbol.
        chain = slice.option_chains.get(self._symbol)
        if not chain or self.portfolio.invested:
            return

        # Get the contract with Delta closest to 0.4 to trade.
        # The arbitrary delta criterion might be set due to hedging need or risk adjustment.
        selected = sorted(chain, key=lambda x: abs(x.greeks.delta - 0.4))[0]

        # Hold a naked call on the selected contract to speculate an uprise of the SPX index with leverage.
        self.market_order(selected.symbol, 1)
```

Index Options

Greeks and Implied Volatility

The Greeks are measures that describe the sensitivity of an Option's price to various factors like underlying price changes (Delta), time decay (Theta), volatility (Vega), and interest rates (Rho), while Implied Volatility (IV) represents the market's expectation of the underlying asset's volatility over the life of the Option.

Key Concepts

Concepts on Greeks, IV, and IV modeling

Indicators

Implementation of IV and Greeks as LEAN indicators

See Also

Option Pricing
Risk Free Interest Rate
Dividend Yield

Greeks and Implied Volatility

Key Concepts

Introduction

LEAN provides multiple ways for you to get the Greeks and implied volatility (IV) of Option contracts. There is a daily, pre-calculated dataset based on the end of the previous trading day. There is also a stream of values that are calculated on-the-fly as your algorithm runs.

Greeks on Universes

The Greeks and IV values in the filter function are the daily, pre-calculated values based on the end of the previous trading day.

```py
def initialize(self):
    option = self.add_index_option("SPX")
    option.set_filter(lambda universe: universe.delta(0.3, 0.7).expiration(0,7))
```

To calculate the values, we use our implementation of the Black-Scholes pricing model, which accounts for the interest rate and dividend payments when applicable. For example, VIX doesn't have dividends. We use SPY for SPX and QQQ for NDX.

You can't customize the Greeks and IV values that the filter function receives. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

Greek History

The Greeks and IV values that you get from a history request of the Option universe are the daily, pre-calculated values based on the end of the previous trading day.

```py
class OptionHistoryAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_index_option('SPX')
        # DataFrame example:
        history_df = self.history(option.symbol, 5, flatten=True)
        # OptionUniverse objects example:
        history = self.history[OptionUniverse](option.symbol, 5)
        for chain in history:
            end_time = chain.end_time
            filtered_contracts = [c for c in chain if c.greeks.delta > 0.3]
            for contract in filtered_contracts:
                symbol = contract.symbol
                expiry = contract.id.date
                strike = contract.id.strike_price
                price = contract.close
                iv = contract.implied_volatility
```

You can't customize the Greeks and IV values that you get from a history request. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

Greeks on Data Events

The Greeks and IV values that you get from the current Slice are from the price model calculations. These values aren't the same as the values you get in the universe filter function.

```
class OptionDataEventsAlgorithm(QCAlgorithm):

    def initialize(self):
        option = self.add_index_option("VIX", "VIXW")
        option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()

    def on_data(self, data):
        for canonical_symbol, chain in data.option_chains.items():
            for contract in chain:
                iv = contract.implied_volatility
                delta = contract.greeks.delta
```

PY

To view all the models LEAN supports, see Supported Models .

Greek Indicators

The Greeks and IV values that you get from the indicators are calculated in the same way as the values you get in the filter function, but they can reflect intraday prices instead of just the daily closing prices.

Examples

For detailed examples on using Option Greeks and Implied Volatility indicators in algorithm, you may refer to the Greeks Indicators page examples.

Greeks and Implied Volatility

Indicators

Introduction

Option indicators enable you to calculate daily and intraday values for the Greeks and implied volatility of Option contracts. This page explains the most popular Option indicators and how to use them with LEAN.

Parameters

The following table describes the arguments that the automatic Option indicators methods accept:

Argument	Data Type	Description	Default Value
`symbol`	`Symbol`	The contract to use when calculating the indicator values.	
`mirror_option`	`Symbol`	The mirror contract to use in parity type calculations.	`None`
`risk_free_rate`	`float`	The risk-free interest rate. If you don't provide a value, the default value is the US primary credit rate from the Interest Rate Provider Model .	`None`
`dividend_yield`	`float`	The dividend yield rate. If you don't provide a value, the default value comes from the Dividend Yield Provider Model .	`None`
`option_model`	`OptionPricingModelType`	The Option pricing model that's used to calculate the Greeks. If you don't provide a value, the default value is `OptionPricingModelType.BlackScholes` for Index Options.	`None`
`resolution`	`Resolution`	The resolution of the indicator data. If you don't provide a value, the default value is the resolution of the subscription you have for the Option contract(s).	`None`

To perform implied volatility (IV) smoothing with a put-call pair, pass one of the contracts as the `symbol` argument and pass the other contract in the pair as the `mirror_option` argument. The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, call the `set_smoothing_function` method of the `ImpliedVolatility` class/property.

Several different Option pricing models are supported to calculate the IV and Greeks. The following table describes the `OptionPricingModelType` enumeration members:

Implied Volatility

Implied volatility, , is the market's expectation for the future volatility of an asset and is implied by the price of the assets's Options contracts. You can't observe it in the market but you can derive it from the price of an Option. For more information about implied volatility, see Implied Volatility .

Automatic Indicators

To create an automatic indicator for implied volatility, call the `QCAlgorithm.iv` method with the Option contract `symbol` object(s).

```py
class AutomaticImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            contract1.iv = self.iv(call, put)
            contract2.iv = self.iv(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the ImpliedVolatility indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.iv
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `iv` method, see Parameters and Using IV Indicator .

Manual Indicators

To create a manual indicator for implied volatility, call the `ImpliedVolatility` constructor.

```py
class ManualImpliedVolatilityIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
```

```
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic ImpliedVolatility indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.ImpliedVolatility = ImpliedVolatility(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].ImpliedVolatility
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the ImpliedVolatility indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `ImpliedVolatility` constructor, see Using IV Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the `iv` method or `ImpliedVolatility` constructor and then call the `set_smoothing_function` method of the resulting `ImpliedVolatility` object. The follow table describes the arguments of the custom function:

Argument	Data Type	Description
`iv`	`float`	The IV of the Option contract.
`mirror_iv`	`float`	The IV of the mirror Option contract.

The method must return a `float` as the smoothened IV.

```
def initialize(self):
    option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.PUT, 505, datetime(2014, 6, 27))
    self.add_option_contract(option)

    mirror_option = Symbol.create_option("AAPL", Market.USA, OptionStyle.AMERICAN, OptionRight.CALL, 505, datetime(2014, 6, 27))
    self.add_option_contract(mirror_option)

    self._iv = self.iv(option, mirror_option)
    # Example: The average of the call-put pair.
    self._iv.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

PY

Delta

Delta, , is the rate of change of the Option price with respect to the price of the underlying asset. It measures the first-order sensitivity of the price to a movement in underlying price. For example, an Option delta of 0.4 means that if the underlying asset moves by 1%, then the value of the Option moves by 0.4 1% = 0.4%. For more information about delta, see Delta .

Automatic Indicators

To create an automatic indicator for delta, call the `QCAlgorithm.d` method with the Option contract `symbol` object(s).

```py
class AutomaticDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Delta indicators.
            contract1.d = self.d(call, put)
            contract2.d = self.d(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Delta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.d
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `d` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Delta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `d` method, see Using D Indicator .

Manual Indicators

To create a manual indicator for delta, call the `Delta` constructor.

```py
class ManualDeltaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Delta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Delta = Delta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Delta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Delta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Delta` constructor, see Using D Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
delta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Gamma

Gamma, , is the rate of change of the portfolio's delta with respect to the underlying asset's price. It represents the second-order sensitivity of the Option to a movement in the underlying asset's price. For more information about Gamma, see Gamma .

Automatic Indicators

To create an automatic indicator for gamma, call the `QCAlgorithm.g` method with the Option contract `symbol` object(s).

```py
class AutomaticGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Gamma indicators.
            contract1.g = self.g(call, put)
            contract2.g = self.g(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Gamma indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.g
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `g` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Gamma. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `g` method, see Using G Indicator .

Manual Indicators

To create a manual indicator for gamma, call the `Gamma` constructor.

```py
class ManualGammaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Gamma indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Gamma = Gamma(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Gamma
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Gamma indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Gamma` constructor, see Using G Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
gamma_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Vega

Vega, , is the rate of change in the value of the Option with respect to the volatility of the underlying asset. For more information about vega, see Vega .

Automatic Indicators

To create an automatic indicator for vega, call the `QCAlgorithm.v` method with the Option contract `symbol` object(s).

```
class AutomaticVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Vega indicators.
            contract1.v = self.v(call, put)
            contract2.v = self.v(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Vega indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.v
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `v` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating Vega. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `v` method, see Using V Indicator .

Manual Indicators

To create a manual indicator for vega, call the Vega constructor.

```py
class ManualVegaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Vega indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Vega = Vega(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Vega
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Vega indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Vega` constructor, see Using V Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

PY

```
# Example: Average IV of the call-put pair.
vega_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Theta

Theta, , is the rate of change of the value of the Option with respect to the passage of time. It is also known to as the time decay of an Option. For more information about theta, see Theta .

Automatic Indicators

To create an automatic indicator for theta, call the `QCAlgorithm.t` method with the Option contract `symbol` object(s).

```py
class AutomaticThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Theta indicators.
            contract1.t = self.t(call, put)
            contract2.t = self.t(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Theta indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.t
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `t` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating theta. If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `t` method, see Using T Indicator .

Manual Indicators

To create a manual indicator for theta, call the `Theta` constructor.

```py
class ManualThetaIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Theta indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Theta = Theta(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Theta
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Theta indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Theta` constructor, see Using T Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
theta_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Rho

Rho, , is the rate of change of the value of a derivative with respect to the interest rate . It is usually small and not a big issue in practice unless the Option is deep in-the-money and has a long horizon. In this case, the interest rate matters because you need to discount a larger cash flow over a longer horizon. For more information about rho, see Rho .

Automatic Indicators

To create an automatic indicator for rho, call the `QCAlgorithm.r` method with the Option contract `symbol` object(s).

```py
class AutomaticRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol

        # Set up a Scheduled Event to select contracts and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )
        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Rho indicators.
            contract1.r = self.r(call, put)
            contract2.r = self.r(put, call)

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Get the Rho indicator of each contract.
        for canonical, chain in slice.option_chains.items():
            for symbol in chain:
                option = self.securities[symbol]
                indicator = option.r
        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

The follow table describes the arguments that the `r` method accepts in addition to the standard parameters :

Argument	Data Type	Description	Default Value
`iv_model`	`OptionPricingModelType`	The Option pricing model to use to estimate the IV when calculating rho If you don't provide a value, the default value is to match the `option_model` parameter.	`None`

For more information about the `r` method, see Using R Indicator .

Manual Indicators

To create a manual indicator for rho, call the Rho constructor.

```py
class ManualRhoIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(500_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Subscribe to the underlying asset.
        self._underlying = self.add_index('SPX').symbol
        # Set up the dividend yield provider for the underlying.
        self._dividend_yield_provider = DividendYieldProvider(self._underlying)
        # Define the Option pricing model.
        self._option_pricing_model = OptionPricingModelType.FORWARD_TREE

        # Set up a Scheduled Event to select contract and create the indicators every day before market open.
        self.schedule.on(
            self.date_rules.every_day(self._underlying),
            self.time_rules.at(9, 0),
            self._update_contracts_and_greeks
        )

        self._options = None

    def _update_contracts_and_greeks(self) -> None:
        if self._underlying is None:
            return
        # Get all the tradable Option contracts.
        chain = self.option_chain(self._underlying, flatten=True).data_frame
        if chain.empty:
            return

        # Filter the contracts down. For example, ATM contracts with atleast 1 month until expiry.
        chain = chain[chain.expiry > self.time + timedelta(30)]
        expiry = chain.expiry.min()
        chain = chain[chain.expiry == expiry]
        chain.loc[:, 'abs_strike_delta'] = abs(chain['strike'] - chain['underlyinglastprice'])
        abs_strike_delta = chain['abs_strike_delta'].min()
        chain = chain[chain['abs_strike_delta'] == abs_strike_delta]

        # Group the contracts into call/put pairs.
        contracts_pair_sizes = chain.groupby(['expiry', 'strike']).count()['right']
        paired_contracts = contracts_pair_sizes[contracts_pair_sizes == 2].index
        expiries = [x[0] for x in paired_contracts]
        strikes = [x[1] for x in paired_contracts]
        symbols = [
            idx[-1] for idx in chain[
                chain['expiry'].isin(expiries) & chain['strike'].isin(strikes)
            ].reset_index().groupby(['expiry', 'strike', 'right', 'symbol']).first().index
        ]
        pairs = [(symbols[i], symbols[i+1]) for i in range(0, len(symbols), 2)]

        for call, put in pairs:
            # Subscribe to both contracts.
            contract1 = self.add_index_option_contract(call)
            contract2 = self.add_index_option_contract(put)

            # Create and save the automatic Rho indicators.
            for contract_a, contract_b in [(contract1, contract2), (contract2, contract1)]:
                contract_a.Rho = Rho(
                        contract_a.symbol, self.risk_free_interest_rate_model,
                        self._dividend_yield_provider, contract_b.symbol, self._option_pricing_model
                    )

            self._options = (call, put)

    def on_data(self, slice: Slice) -> None:
        # Iterate through the indicators.
        for canonical, chain in slice.option_chains.items():
            for option in chain:
                underlying = option.symbol.underlying
                indicator = self.securities[option].Rho
                mirror = Symbol.create_option(
                    underlying.value, option.id.market, option.id.option_style,
                    OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                    option.id.strike_price, option.id.date
                )
                # Check if price data is available for both contracts and the underlying asset.
                q = slice.quote_bars
                b = slice.bars
                if option in q and mirror in q and underlying in b:
                    data_points = [
                        IndicatorDataPoint(option, q[option].end_time, q[option].close),
                        IndicatorDataPoint(mirror, q[mirror].end_time, q[mirror].close),
                        IndicatorDataPoint(
                            underlying, b[underlying].end_time, b[underlying].close
                        )
```

```
                    ]
                    for data_point in data_points:
                        indicator.update(data_point)

                    # Get the current value of the Rho indicator.
                    value = indicator.current.value

        # Sell straddle as an example to trade.
        if not self.portfolio.invested and self._options:
            self.sell(self._options[0], 1)
            self.sell(self._options[1], 1)
        # Liquidate any assigned positions.
        if not self._underlying and self.portfolio[self._underlying].invested:
            self.liquidate(self._underlying)
```

For more information about the `Rho` constructor, see Using R Indicator .

Volatility Smoothing

The default IV smoothing method uses the one contract in the pair that's at-the-money or out-of-money to calculate the IV. To change the smoothing function, pass a `mirror_option` argument to the indicator method or constructor and then call the `set_smoothing_function` method of the `implied_volatility` property of the indicator.

```
PY
# Example: Average IV of the call-put pair.
rho_indicator.implied_volatility.set_smoothing_function(lambda iv, mirror_iv: (iv + mirror_iv) * 0.5)
```

Examples

Demonstration Algorithms
OptionIndicatorsRegressionAlgorithm.py Python OptionIndicatorsMirrorContractsRegressionAlgorithm.py Python

Index Options

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the US Index Option market.

Search by symbol (e.g. SPX, V

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

CFD

A contract for difference (CFD) is a contract between a buyer and a seller that stipulates that the buyer must pay the seller the difference between the current value of an asset and its value at contract time.

Requesting Data

Handling Data

Market Hours

See Also

BasicTemplateCfdAlgorithm.py
BasicTemplateCfdAlgorithm.cs

CFD

Requesting Data

Introduction

Request Contract for Difference (CFD) data in your algorithm to receive a feed of contract prices in the `on_data` method. QuantConnect supports CFD trading with OANDA and Interactive Brokers .

OANDA Subscriptions

To create a CFD subscription, in the `initialize` method, call the `add_cfd` method. The `add_cfd` method returns a `Cfd` Security object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_cfd("XAUUSD").symbol
```

PY

To view the supported CFD contracts, see Supported Assets . For more information about the specific dataset we use for backtests, see the OANDA CFD dataset listing . To trade OANDA CFDs live, use our QuantConnect data provider .

Interactive Brokers Subscriptions

To create a CFD subscription for Interactive Brokers (IB) , in the `initialize` method, call the `add_cfd` method and set the `market` parameter to `Market.INTERACTIVE_BROKERS` . The `add_cfd` method returns a `Cfd` Security object, which contains a `symbol` property. Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
self._symbol = self.add_cfd("SPY", market=Market.INTERACTIVE_BROKERS).symbol
```

PY

Historical data for backtesting IB CFDs is unavailable.

In live trading, include the Interactive Brokers data provider when you deploy the algorithm to access IB CFD data. You can use this data provider for paper or real-money accounts. IB provides data for Indexes, Metals, Forex, and Global Stock CFDs. To view the Index and Forex CFD products that IB supports in live trading, see the Interactive Brokers Supported Contracts . To view the Equity CFD products, see the Products Search page on the IB website.

If you live in the EU and can't trade US ETFs due to regulation, you can trade their CFD equivalents in live mode.

```
security_type = SecurityType.CFD if self.live_mode else SecurityType.EQUITY
market=Market.INTERACTIVE_BROKERS if self.live_mode else Market.USA
self._symbol = Symbol.create("SPY", security_type, market)
self.add_security(self._symbol)
```

PY

Market hours for IB CFDs are not modeled. You cannot use them for scheduled events or to get market open/close times . To create a scheduled event, use a ticker of any US Equity that you haven't added as a IB CFD or create a US Equity `Symbol` .

```
def initialize(self):
    us_equity_symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
    # Call the _before_market_open method 10 minutes before open every day US Equity stocks are trading.
    self.schedule.on(
        self.date_rules.every_day(us_equity_symbol),
        self.time_rules.before_market_open(us_equity_symbol, 10),
        self._before_market_open
    )
```

PY

Resolutions

The following table shows the available resolutions and data formats for CFD subscriptions:

Resolution	TradeBar	QuoteBar	Trade Tick	Quote Tick
TICK				✓
SECOND		✓		
MINUTE		✓		
HOUR		✓		
DAILY		✓		

The default resolution for CFD subscriptions is `Resolution.MINUTE` . To change the resolution, pass a `resolution` argument to the `add_cfd` method.

```
self._symbol = self.add_cfd("XAUUSD", Resolution.DAILY).symbol
```

PY

To create custom resolution periods, see Consolidating Data .

Supported Markets

The following `Market` enumeration members are available for CFDs:

To set the market for a security, pass a `market` argument to the `add_cfd` method.

```
self._symbol = self.add_cfd("XAUUSD", market=Market.OANDA).symbol
```

PY

The brokerage models have a default market for each asset class. If you set a brokerage model, you may not need to specify the market to use. In backtesting, set the market for CFDs to OANDA.

Fill Forward

Fill forward means if there is no data point for the current slice , LEAN uses the previous data point. Fill forward is the default data setting. If you disable fill forward, you may get stale fills or you may see trade volume as zero.

To disable fill forward for a security, set the `fill_forward` argument to false when you create the security subscription.

```
self._symbol = self.add_cfd("XAUUSD", fill_forward=False).symbol
```

PY

Margin and Leverage

LEAN models buying power and margin calls to ensure your algorithm stays within the margin requirements. The specific margin available depends on your brokerage destination. To change the amount of leverage you can use for a CFD contract, pass a `leverage` argument to the `add_cfd` method.

```
self._symbol = self.add_cfd("XAUUSD", leverage=35).symbol
```

PY

Data Normalization

The data normalization mode doesn't affect the data that LEAN passes to `on_data` or the data from history request . If you change the data normalization mode, it won't change the outcome.

Interactive Brokers Stock CFDs apply corporate actions to the price of the asset, including paying dividends in cash to your account. This behavior is not modeled today and splits in live trading will result in price discontinuities. You should monitor the underlying Equity for events and use the underlying Equity for indicators.

Properties

The `add_cfd` method returns a `Cfd` object, which have the following properties:

Interactive Brokers Supported Contracts

The following table shows the CFD contracts that IB supports:

Contract Available (107)					
AUDCAD	AUDCHF	AUDCNH	AUDHKD	AUDJPY	AUDNZD
AUDSGD	AUDUSD	AUDZAR	CADCHF	CADCNH	CADHKD
CADJPY	CHFCNH	CHFCZK	CHFDKK	CHFHUF	CHFJPY
CHFNOK	CHFPLN	CHFSEK	CHFZAR	CNHHKD	CNHJPY
DKKJPY	DKKNOK	DKKSEK	EURAUD	EURCAD	EURCHF
EURCNH	EURCZK	EURDKK	EURGBP	EURHKD	EURHUF
EURILS	EURJPY	EURMXN	EURNOK	EURNZD	EURPLN
EURRUB	EURSEK	EURSGD	EURUSD	EURZAR	GBPAUD
GBPCAD	GBPCHF	GBPCNH	GBPCZK	GBPDKK	GBPHKD
GBPHUF	GBPJPY	GBPMXN	GBPNOK	GBPNZD	GBPPLN
GBPSEK	GBPSGD	GBPUSD	GBPZAR	HKDJPY	IBAU200
IBCH20	IBDE40	IBES35	IBEU50	IBFR40	IBGB100
IBHK50	IBJP225	IBNL25	IBUS30	IBUS500	IBUST100
MXNJPY	NOKJPY	NOKSEK	NZDCAD	NZDCHF	NZDJPY
NZDUSD	SEKJPY	SGDCNH	SGDJPY	USDCAD	USDCHF
USDCNH	USDCZK	USDDKK	USDHKD	USDHUF	USDILS
USDJPY	USDMXN	USDNOK	USDPLN	USDRUB	USDSEK
USDSGD	USDZAR	XAGUSD	XAUUSD	ZARJPY	

Examples

The following examples demonstrate some common practices for requesting CFD data.

Example 1: SMA Crossing

The following strategy purchases ETFs when their fast-moving average crosses over the slow-moving average. In live mode, the strategy purchases their CFD equivalents.

PY

```
class EtfCfdSwapMovingAverageCrossAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request the data to feed into indicators and trade.
        # In live mode, request CFD data for trading.
        # CFD is more useful in non-trading hours to reduce slippage when market prices jump at market opening.
        tickers = ["SPY", "BND", "GLD", "QQQ"]
        security_type = SecurityType.CFD if self.live_mode else SecurityType.EQUITY
        for ticker in tickers:
            # Create fast and slow-moving average indicators to trade SAM cross trend-following strategy.
            equity = self.add_equity(ticker, Resolution.DAILY)
            equity.sma_fast = self.sma(equity.symbol, 50)
            equity.sma_slow = self.sma(equity.symbol, 200)
            equity.target_vehicle = self.add_cfd(ticker, Resolution.DAILY, market=Market.INTERACTIVE_BROKERS).symbol if self.live_mode
else equity.symbol

        # Create an schduled event to plot the prices.
        # We need to create a symbol representing the US Equity market
        # because market hours for IB CFDs are not modeled
        market = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.schedule.on(self.date_rules.every_day(market), self.time_rules.midnight, self._plot_prices)

    def on_data(self, data: Slice) -> None:
        for security in self.securities.values():
            # Trade the SMA cross on the updated SMA indicator signals.
            # If fast MA is above slow MA, the price is in an uptrend, and we buy.
            if security.sma_fast > security.sma_slow:
                self.set_holdings(security.target_vehicle, 0.25)
            # Else, the price is in a downtrend, and we are short.
            else:
                self.set_holdings(security.target_vehicle, 0)

    def _plot_prices(self):
        for security in self.securities.values():
            self.plot(security.symbol.value, 'EOD', security.price)
```

CFD

Handling Data

Introduction

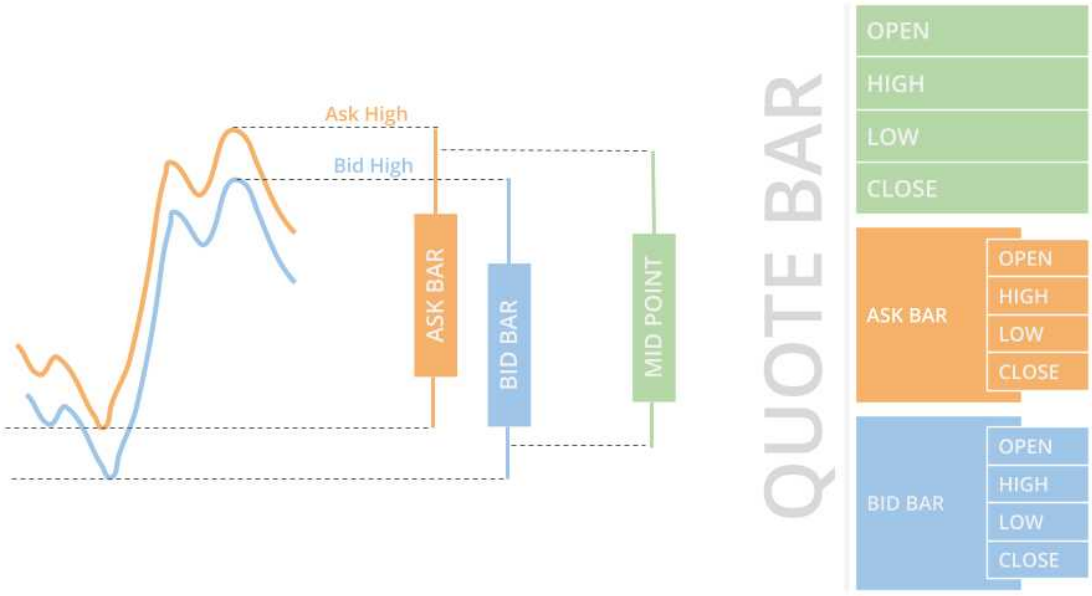
LEAN passes the data you request to the `on_data` method so you can make trading decisions. The `Slice` object that the `on_data` method receives groups all the data together at a single moment in time. To access the `Slice` outside of the `on_data` method, use the `current_slice` property of your algorithm.

All the data formats use `DataDictionary` objects to group data by `Symbol` and provide easy access to information. The plural of the type denotes the collection of objects. For instance, the `QuoteBars` `DataDictionary` is made up of `QuoteBar` objects. To access individual data points in the dictionary, you can index the dictionary with the CFD ticker or `symbol` , but we recommend you use the `symbol` .

To view the resolutions that are available for CFD data, see Resolutions .

Quotes

`QuoteBar` objects are bars that consolidate NBBO quotes from the exchanges. They contain the open, high, low, and close prices of the bid and ask. The `open` , `high` , `low` , and `close` properties of the `QuoteBar` object are the mean of the respective bid and ask prices. If the bid or ask portion of the `QuoteBar` has no data, the `open` , `high` , `low` , and `close` properties of the `QuoteBar` copy the values of either the `bid` or `ask` instead of taking their mean.



To get the `QuoteBar` objects in the `Slice` , index the `QuoteBars` property of the `Slice` with the CFD `symbol` . If the CFD doesn't actively get quotes or you are in the same time step as when you added the CFD subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your CFD before you index the `Slice` with the CFD `symbol` .

```
def on_data(self, slice: Slice) -> None:
    # Obtain the mapped QuoteBar of the symbol if any
    quote_bar = slice.quote_bars.get(self._symbol)   # None if not found
```

PY

You can also iterate through the `QuoteBars` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `QuoteBar` objects.

```
def on_data(self, slice: Slice) -> None:
    # Iterate all received Symbol-QuoteBar key-value pairs
    for symbol, quote_bar in slice.quote_bars.items():
        ask_price = quote_bar.ask.close
```

`QuoteBar` objects let LEAN incorporate spread costs into your simulated trade fills to make backtest results more realistic.

`QuoteBar` objects have the following properties:

Ticks

`Tick` objects represent a single trade or quote at a moment in time. A trade tick is a record of a transaction for the CFD. A quote tick is an offer to buy or sell the CFD at a specific price.

Trade ticks have a non-zero value for the `quantity` and `price` properties, but they have a zero value for the `bid_price` , `bid_size` , `ask_price` , and `ask_size` properties. Quote ticks have non-zero values for `bid_price` and `bid_size` properties or have non-zero values for `ask_price` and `ask_size` properties. To check if a tick is a trade or a quote, use the `ticktype` property.

In backtests, LEAN groups ticks into one millisecond buckets. In live trading, LEAN groups ticks into ~70-millisecond buckets. To get the `Tick` objects in the `Slice` , index the `Ticks` property of the `Slice` with a `symbol` . If the CFD doesn't actively trade or you are in the same time step as when you added the CFD subscription, the `Slice` may not contain data for your `symbol` . To avoid issues, check if the `Slice` contains data for your CFD before you index the `Slice` with the CFD `symbol` .

```
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks.get(self._symbol, [])   # Empty if not found
    for tick in ticks:
        price = tick.price
```

You can also iterate through the `Ticks` dictionary. The keys of the dictionary are the `Symbol` objects and the values are the `list[Tick]` objects.

```
def on_data(self, slice: Slice) -> None:
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            price = tick.price
```

Tick data is raw and unfiltered, so it can contain bad ticks that skew your trade results. For example, some ticks come from dark pools, which aren't tradable. We recommend you only use tick data if you understand the risks and are able to perform your own online tick filtering.

`Tick` objects have the following properties:

Examples

The following examples demonstrate common practices for handling CFD data.

Example 1: Buy And Hold

The following algorithm is a buy-and-hold strategy on the DE30EUR CFD contract. We only place orders when quote data is available to avoid stale fills. Also, we set the algorithm time zone to Berlin and the account currency to Euros for a more convenient comparison.

```py
class CfdExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the timezone as Berlin to conveniently compare the data.
        self.set_time_zone(TimeZones.BERLIN)
        # Set the account currency to EUR to trade the DAX CFD.
        self.set_account_currency("EUR", 10000)
        # Request the CFD data to trade.
        self._dax = self.add_cfd("DE30EUR").symbol

    def on_data(self, slice: Slice) -> None:
        # Trade based on updated data; CFDs only have quote data.
        bar = slice.quote_bars.get(self._dax)
        if bar:
            # Buy and hold DAX CFD.
            if not self.portfolio[self._dax].is_long:
                self.set_holdings(self._dax, 0.5)
```

Example 2: Global Indices

The following algorithm buys and holds various Index CFDs that trade in different market hours and currencies during their opening hours.

```py
class CfdExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Let's select CFD contracts that trade in different market
        # hours so the algorithm is always invested.
        for ticker in ['DE30EUR', 'SG30SGD', 'US30USD']:
            # Add the CFD.
            symbol = self.add_cfd(ticker).symbol
            # Set scheduled events to hold each CFD contract during
            # their regular trading hours.
            # Buy after market open.
            self.schedule.on(
                self.date_rules.every_day(symbol),
                self.time_rules.after_market_open(symbol, 1),
                lambda symbol=symbol: self.set_holdings(symbol, 0.3)
            )
            # Liquidate before market close.
            self.schedule.on(
                self.date_rules.every_day(symbol),
                self.time_rules.before_market_close(symbol, 1),
                lambda symbol=symbol: self.liquidate(symbol)
            )
```

CFD

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the CFD market.

Search by symbol (e.g. XAUUS

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Asset Classes

India Equity

India Equity

Market Hours

Introduction

This page shows the trading hours, holidays, and time zone of the India Equity market.

Pre-market Hours

Regular Trading Hours

Post-market Hours

Holidays

Early Closes

Late Opens

Time Zone

Portfolio

Portfolio

Key Concepts

Introduction

The `portfolio` object provides information about the investment state and history of all of the securities that have been in the algorithm during its lifetime.

The `portfolio` property of the `QCAlgorithm` class is a dictionary where the keys are `Symbol` objects and the values are `SecurityHolding` objects. The `SecurityHolding` object provides information about the investment state and history of a security.

Investment Status

To get the investment status of your strategy, access the `invested` property of a `portfolio` object.

PY

```
# Get the investment status and return True if the portfolio is invested.
invested = self.portfolio.invested
```

Holdings Status

You can identify a position as long- or short-biased based on the sign of the holding quantity. Long positions have a positive quantity and short positions have a negative quantity. The `SecurityHolding` object has the following properties to describe the holding status.

PY

```
# Create a SecurityHolding object.
holdings = self.portfolio[self._symbol]
# Check the holding quantity of the security.
quantity = holdings.quantity
# Check the investing status of the security.
invested = holdings.invested
# Check if the strategy is long or short the security.
is_long = holdings.is_long
is_short = holdings.is_short
```

The quantity can be lower than the minimum order size. If the quantity is less than the lot size of the asset, the `invested` property is `False` .

Buying Power

To get the maximum buying power in your account currency you can use for a given `Symbol` and order direction, call the `get_buying_power` method.

PY

```
# Check the total amount of buying power available. Margin available varies by order direction and current holdings.
available_buying_power = self.portfolio.get_buying_power(self._symbol, OrderDirection.BUY)
```

For more information about buying power, see Buying Power .

Cost Averaging Accounting

LEAN uses the cost averaging accounting method, which determines the cost of your holdings by taking a weighted average of all your purchase prices. For example, say you place the following buy orders:

1. Buy 10 ABC @ $10
2. Buy 5 ABC @ $11
3. Buy 20 ABC @ $14
4. Buy 3 ABC @ $9

In the preceding example, the average cost of your ABC position is (10*10 + 5*11 + 20*14 + 3*9) / (10 + 5 + 20 + 3) = 12.1579/share. In contrast, if you use the first-in, first-out (FIFO) accounting method, the cost of the first 10 shares is 10/share, not 12.1579/share.

To get the cost of your security holdings, use the `holdings_cost` property of the SecurityHolding object . If you fill buy and sell orders, the holdings cost is the product of the holding quantity and the average price. For example, the following table shows how the average price and holdings cost changes with each buy and sell order order in a long position:

Order Quantity	Fill Price ($)	Holding Quantity	Average Price ($)	Holdings Cost ($)
2	10	2	(2 * 10) / 2 = 10	2 * 10 = 20
-1	11	1	(1 * 10) / 1 = 10	1 * 10 = 10
1	12	2	(1 * 10 + 1 * 12) / 2 = 11	2 * 11 = 22
-2	13	0	0	0 * 0 = 0

The following table shows how the average price and holdings cost changes with each buy and sell order order in a short position:

Order Quantity	Fill Price ($)	Holding Quantity	Average Price ($)	Holdings Cost ($)
-2	10	-2	(-2 * 10) / -2 = 10	-2 * 10 = -20
1	11	-1	(-1 * 10) / -1 = 10	-1 * 10 = -10
-1	12	-2	(-1 * 10 + (-1) * 12) / -2 = 11	-2 * 11 = -22
2	13	0	0	0 * 0 = 0

Note that when you decrease the position size without closing the trade, the average price doesn't change because the denominator and the coefficients in the numerator of its formula are scaled by the quotient of the current holding quantity and the previous holding quantity. For instance, if the last row in the preceding table had an order quantity 1, the holding quantity would be -1 and the average price would be

$$\frac{(-1 * q) * 10 + (-1 * q) * 12}{-2 * q} =\frac{(-1 * \frac{-1}{-2}) * 10 + (-1 * \frac{-1}{-2}) * 12}{-2 * \frac{-1}{-2}} =\frac{\frac{-1}{2} * 10 + \frac{-1}{2} * 12}{-1 } =\frac{-5 - 6}{-1} = 11$$

Performance Statistics

The `portfolio` object has a collection of properties that provide the combined state of holdings and the cashbook .

```py
# Access key statistics of the portfolio using the properties of the portfolio object.
value = self.portfolio.total_portfolio_value
total_margin_used = self.portfolio.total_margin_used
margin_remaining = self.portfolio.margin_remaining
total_net_profit = self.portfolio.total_net_profit
total_unrealised_profit = self.portfolio.total_unrealised_profit
total_fees = self.portfolio.total_fees
cash = self.portfolio.cash
```

Properties

The `portfolio` object has the following properties:

Examples

The following examples demonstrate common practices for handling the portfolio.

Example 1: Martingale

The following algorithm trades a Martingale strategy on AAPL. The trend indicated by an EMA indicator follows the trend with a $1 per share stop-loss and a $2 take-profit level. Yet, if the last trade is lost, it will double the next trading size to recover the last loss.

PY

```
class PortfolioExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request AAPL's data to trade.
        self.aapl = self.add_equity("AAPL").symbol

        # Create an EMA indicator to generate trading data.
        self._ema = self.ema(self.aapl, 20, Resolution.DAILY)
        # Warm up the indicator to provide an immediate trade signal.
        self.warm_up_indicator(self.aapl, self._ema, Resolution.DAILY)

        self.next_quantity = 1

    def on_data(self, slice: Slice) -> None:
        # Ride on the AAPL trend indicated by the EMA indicator.
        if not self.portfolio.invested:
            if self.securities[self.aapl].price >= self._ema.current.value:
                self.market_order(self.aapl, self.next_quantity)
            else:
                self.market_order(self.aapl, -self.next_quantity)
        else:
            holding = self.portfolio[self.aapl]
            # Stop loss at $1 per share.
            if holding.unrealized_profit <= -1 * self.next_quantity:
                self.liquidate(self.aapl)
                # Double the next quantity to recover this loss.
                self.next_quantity *= 2
            # Take profit at $2 per share.
            elif holding.unrealized_profit >= 2:
                self.liquidate(self.aapl)
                self.next_quantity = 1
```

Portfolio

Holdings

Introduction

The `portfolio` is a dictionary where the key is a `Symbol` and the value is a `SecurityHolding` .

```
# Create shortcut helper to the SPY SecurityHolding object from the Portfolio collection.
security_holding = self.portfolio["SPY"]
```

Properties

`SecurityHolding` objects have the following properties:

```
#  Save shortcut to SecurityHolding object, and fetch quantity and invested status.
security_holding = self.portfolio["SPY"]
quantity = security_holding.quantity
invested = security_holding.invested
```

`FutureHolding` objects also have `settled_profit` and `unsettled_profit` properties.

Get Total Close Profit

To get the profit of a position holding if you closed the position, call the `total_close_profit` method. The value this method returns is denominated in your account currency and accounts for order fees.

```
profit = self.portfolio["SPY"].total_close_profit()
```

The `total_close_profit` method accepts the following optional arguments:

Argument	Data Type	Description	Default Value
`include_fees`	`bool`	Whether to reduce the profit based on the estimated fee from the fee model .	`True`
`exit_price`	`float/NoneType`	A hypothetical exit price to use for the profit calculation. If you don't provide a value, it uses the bid price for sell orders or the ask price for buy orders.	`None`
`entry_price`	`float/NoneType`	A hypothetical exit price to use for the profit calculation. If you don't provide a value, it uses the average price of the `SecurityHolding` .	`None`
`quantity`	`float/NoneType`	The quantity to liquidate. If you don't provide a value, it uses the quantity of the `SecurityHolding` .	`None`

LEAN uses this method to define the `unrealized_profit` property.

Get Quantity Value

To get the value of a security at any quantity, call the `get_quantity_value` method. The value this method returns is denominated in your account currency.

```py
# Get the quantity value at the current price
value_at_current_price = self.portfolio["SPY"].get_quantity_value(100)

# Get the quantity value at a specific price
value_at_specific_price = self.portfolio["SPY"].get_quantity_value(100, price=30)
```

Set Initial Holdings

When you deploy a backtest or a new paper trading algorithm, the default initial portfolio state contains $100,000 USD. However, in the initialize method, you can define the initial portfolio state to something else. To set the initial cash balances , call `set_cash` with the currency ticker and quantity. To set multiple currencies, call the method multiple times.

```py
self.set_cash(100000)       # Set the quantity of the account currency to 100,000
self.set_cash("BTC", 10)    # Set the Bitcoin quantity to 10
self.set_cash("EUR", 10000) # Set the EUR quantity to 10,000
```

To set the initial security holdings, subscribe to the security so that it enters your portfolio , then call the `SecurityHolding.set_holdings` method with the average purchase price and quantity.

```py
if not self.live_mode:
    self.portfolio["SPY"].set_holdings(average_price=500, quantity=100)
```

Examples

The following examples demonstrate common practices for setting the portfolio holdings.

Example 1: Initial Holdings

The following algorithm simulates an account with multiple initial positions and cash. Note that these initial holdings will affect the algorithm's performance.

```py
class PortfolioExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Simulate initial cash in multiple currencies.
        # Note that all currencies other than the account currency will affect the performance.
        self.set_account_currency("USD", 10000)
        self.set_cash("EUR", 1000)
        self.set_cash("JPY", 1000000)
        self.set_cash("HKD", 10000)

        # Add a security initializer that seeds the price of each asset with its
        # last known price. This ensures the holdings we add below will initialize
        # with a non-zero value.
        self.settings.seed_initial_prices = True

        # Request data for some securities in the account already.
        spy = self.add_equity("SPY").symbol
        aapl = self.add_equity("AAPL").symbol
        btcusd = self.add_crypto("BTCUSD").symbol
        xauusd = self.add_cfd("XAUUSD").symbol

        # Simulate that the account initially holds some assets.
        # Note that all initial holdings will affect the performance.
        self.portfolio[spy].set_holdings(average_price=420, quantity=102)
        self.portfolio[aapl].set_holdings(average_price=120, quantity=25)
        self.portfolio[btcusd].set_holdings(average_price=45000, quantity=0.125)
        self.portfolio[xauusd].set_holdings(average_price=1000, quantity=100)
```

Portfolio

Cashbook

Introduction

The `CashBook` is a dictionary where the keys are currency tickers and the values are `Cash` objects. The `Cash` objects track the amount of each currency in the portfolio. As you buy and sell securities, LEAN credits and debits the `Cash` objects to reflect your cash holdings.

Account Currency

The default account currency is USD, but you can change it. All of the properties of the Portfolio object that return a currency value denominate the currency value in your account currency. Depending on your account currency and security subscriptions, LEAN may add internal security subscriptions to calculate the `value_in_account_currency` . For example, if you only add BTCETH to your algorithm and set the account currency to USDT, LEAN adds BTCUSDT and ETHUSDT as internal feeds.

To get your account currency, use the `account_currency` property.

```
account_currency = self.account_currency
```

PY

To set your account currency, in the `initialize` method, call the `set_account_currency` method. You can only set the account currency once. LEAN ignores each additional call to `set_account_currency` .

```
self.set_account_currency("EUR")
```

PY

Settled vs Unsettled Cash

The `portfolio` has two independent cashbooks. The `cash_book` tracks settled cash and the `unsettled_cash_book` tracks unsettled cash, which you can't spend. If you trade with a margin account, trades settle immediately so LEAN credits and debits the `cash_book` at the time of the trade. In some cases, transactions can take time to settle. For example, Equity trades in a cash account settle in T+3. Therefore, if you sell shares of stock on Monday, the transaction settles on Thursday. In contrast, Option trades settle in T+1.

```
settled_cash_book = self.portfolio.cash_book
unsettled_cash_book = self.portfolio.unsettled_cash_book
```

PY

Track Cash Balances

To get the balance of a currency in the cash book, use the `amount` property.

```
# Return the cash book balance in US Dollar amount.
usd = self.portfolio.cash_book["USD"].amount
# Return the cash book balance in Bitcoin amount.
btc = self.portfolio.cash_book["BTC"].amount
```

PY

To get the value of a currency in the cash book, denominated in your account currency, use the `value_in_account_currency` property.

```
# Return the cash book value of Ethereum in account currency (US Dollar, Euro, etc.) amount.
eth_value = self.portfolio.cash_book["ETH"].value_in_account_currency
```

PY

Deposits and Withdrawals

In backtests, you can add and remove cash from the cash book. To add or remove cash, call the `add_amount` method.

```
# Adjust the backtesting cashbook balances to add 100 USD and subtract 1.5 BTC.
new_usd_balance = self.portfolio.cash_book["USD"].add_amount(100)
new_btc_balance = self.portfolio.cash_book["BTC"].add_amount(-1.5)
```

In live trading, add and withdraw cash through your brokerage account. If you adjust the cash balances in your algorithm with the `add_amount` method, the cash balance in your algorithm will be out of sync with the cash balance in your brokerage account.

Currency Symbols

A currency symbol is a graphic that represents the currency name. For example, $ is for dollars, € for euros, and ฿ for Bitcoin. To get the symbol of a currency in the cash book, use `currency_symbol` property.

```
usd_symbol = self.portfolio.cash_book["USD"].currency_symbol
```

Conversion Rates

To get the conversion rate for a currency in the cashbook to your account currency, use the `conversion_rate` property.

```
eur_conversion_rate = self.portfolio.cash_book["EUR"].conversion_rate
```

Examples

The following examples demonstrate common practices for controlling the portfolio cashbook.

Example 1: Dollar Cost Averaging

The following algorithm deposits USD 1000 on every month's start and buys SPY with the deposited cash. It results in a dollar cost averaging investment strategy on the overall market.

```
class PortfolioCashbookAlgorithm(QCAlgorithm):
    deposit = 1000

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000)

        # Request SPY data to trade the overall market representative.
        self.spy = self.add_equity("SPY").symbol

        # Deposit each month start and invest in SPY by dollar cost averaging.
        self.schedule.on(
            self.date_rules.month_start(self.spy),
            self.time_rules.after_market_open(self.spy, 0),
            self.deposit_and_rebalance
        )

        # To warm up the price data of SPY to calculate the quantity to be brought.
        self.set_warm_up(1)

    def deposit_and_rebalance(self) -> None:
        # Deposit the account currency's preset level ($1000) at the month's start.
        # Simulate the monthly salary deposit for dollar cost-averaging investment in the market.
        self.portfolio.cash_book[self.account_currency].add_amount(self.deposit)

        # Calculate the number of shares that can be invested using the deposited amount.
        quantity = self.deposit // self.securities[self.spy].price
        self.market_order(self.spy, quantity)
```

Example 2: Crypto Cashbook

This example trades an EMA cross strategy on the ETH-BTC crypto pair in a cash account. To obtain the maximum tradable order size, we need to use the cash book and check the amount of BTC in long ETHBTC trades and in short ETHBTC trades to avoid insufficient initial margin.

PY

```
class PortfolioCashbookAlgorithm(QCAlgorithm):

    buffer = 0.01

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the account currency as USDT and set the starting cash in a cash account.
        self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)
        self.set_account_currency("USDT", 100)
        self.set_cash("BTC", 100)
        self.set_cash("ETH", 2000)

        # We would like to trade the EMA cross between 2 popular cryptos BTC & ETH,
        # so we are requesting ETHBTC data to find trading opportunities.
        self.ethbtc = self.add_crypto("ETHBTC", Resolution.MINUTE, Market.COINBASE).symbol

        # Add automatic-updating EMA indicator for trend trade signal emission.
        self._ema = self.ema(self.ethbtc, 50, Resolution.DAILY)
        # Warm up the indicator for its readiness usage immediately.
        self.warm_up_indicator(self.ethbtc, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.ethbtc)
        if not (bar and self._ema.is_ready):
            return
        ema = self._ema.current.value
        # ETHBTC's current price is higher than EMA, suggesting an uptrend.
        if bar.close > ema and not self.portfolio[self.ethbtc].is_long:
            # Convert to the corresponding quantity of the quote currency.
            quantity = self.portfolio.cash_book["BTC"].amount * (1 - self.buffer) / bar.close
        # ETHBTC's current price is below the EMA, suggesting a downtrend.
        elif bar.close < ema and not self.portfolio[self.ethbtc].is_short:
            quantity = -self.portfolio.cash_book["ETH"].amount
        else:
            return
        # Round down to observe the lot size.
        lot_size = self.securities[self.ethbtc].symbol_properties.lot_size
        quantity = int(quantity / lot_size) * lot_size
        if quantity:
            self.market_order(self.ethbtc, quantity)
```

Universes

Universes

Key Concepts

Introduction

Universe selection is the process of selecting a basket of assets you may trade. Dynamic universe selection increase diversification and decrease selection bias in your algorithm.

How Universe Selection Works

When you add a universe to your algorithm, LEAN sends a large dataset into a filter function you define. Your filter function needs to return a list of `Symbol` objects. LEAN automatically subscribes to these new symbols and adds them to your algorithm. Your algorithm can do almost anything inside your filter functions, but the goal should be to narrow down the set of securities to the securities that are most applicable for your algorithm.

Selection Functions

The following example filter function selects the 100 most liquid US Equities:

```py
# Use a fundamental filter function to select the 100 most liquid Equities.
class MyFundamentalUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._fundamental_filter_function)

    def _fundamental_filter_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)
        return [c.symbol for c in sorted_by_dollar_volume[:100]]
```

To learn how to define filter functions for other asset classes, custom data universes , or alternative data universes , see the following pages in this Universes chapter.

Security Changed Events

When your universe adds and removes assets, LEAN notifies your algorithm through the `on_securities_changed` event handler. The event handler receives a `SecurityChanges` object, which contains references to the added and removed securities. To access the added securities, check the `changes.added_securities` method property. To access the removed securities, check the `changes.removed_securities` method property.

```py
# Use the on_securities_changed method to know when an asset is added or removed.
def on_securities_changed(self, changes: SecurityChanges) -> None:
    # Iterate through the added securities.
    for security in changes.added_securities:
        self.debug(f"{self.time}: Added {security.symbol}")

    # Iterate through the removed securities.
    for security in changes.removed_securities:
        self.debug(f"{self.time}: Removed {security.symbol}")

        if security.invested:
            self.liquidate(security.symbol, tag="Removed from Universe")
```

The preceding example liquidates securities that leave the universe. In this case, LEAN creates a market on open order and frees up buying power when the market opens.

Custom Security Properties

If you need to save data or create objects for each security in the universe, add custom members to the respective `Security` objects . This technique is useful if you want to track stop loss levels or add indicators for each asset in the universe.

```py
# Add custom properties to the security by duck-typing.
def on_securities_changed(self, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        # Create an SMA indicator with 10 periods for the asset.
        # Use duck typing to store it on the security object.
        security.indicator = self.sma(security.Symbol, 10)

        # Warm up the indicator with historical data.
        self.warm_up_indicator(security.symbol, security.indicator)

    # Remove the automatic indicator updates when the security leaves the universe.
    for security in changes.removed_securities:
        self.deregister_indicator(security.indicator)
```

Select Current Constituents

If you don't want to make any changes to the current universe, return `Universe.UNCHANGED` from your filter functions.

```py
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._my_fundamental_filter_function)

    # Use Universe.UNCHANGED to leave the universe unchanged.
    def _my_fundamental_filter_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        return Universe.UNCHANGED
```

Universe Manager

The universe manager tracks all the universes in your algorithm. If you add multiple universe, you can access the constituents of each individual universe. To access the constituents of a universe in a multi-universe algorithm, save references to each universe when you add them.

```py
# In the initialize method, add a new universe.
self.universe_settings.asynchronous = True
self._universe = self.add_universe(self._my_fundamental_filter_function)

# Use the universe_manager to get information about the assets in each universe.
universe_members = self.universe_manager[self._universe.configuration.symbol].members
for kvp in universe_members:
    symbol = kvp.key # The symbol of the asset.
    security = kvp.value # The security in the universe.
```

When you remove an asset from a universe, LEAN usually removes the security from the `Members` collection and removes the security subscription. However, it won't remove the security in any of the following situations:

- You own the security.
- You have an open order for the security.
- The security wasn't in the universe long enough to meet the `minimum_time_in_universe` setting.

To get only the assets that are currently in the universe, see Selected Securities .

Active Securities

The `QCAlgorithm.active_securities` is a collection of `Universe.members` of all universes. The `active_securities` property of the algorithm class contains all of the assets currently in your algorithm. It is a dictionary where the key is a `Symbol` and the value is a `Security` . When you remove an asset from a universe, LEAN usually removes the security from the `active_securities` collection and removes the security subscription. However, it won't remove the security in any of the following situations:

- You own the security.
- You have an open order for the security.
- The security wasn't in the universe long enough to meet the `minimum_time_in_universe` setting.

When LEAN removes the security, the `Security` object remains in the `securities` collection for record-keeping purposes, like tracking fees and trading volume.

To improve iteration speed, LEAN removes delisted securities from the `securities` primary collection. To access all the securities, iterate the `securities.total` property.

```
# Access all the securities.
for security in self.securities.total:
    pass

# Exclude delisted securities.
for security in self.securities.values():
    pass
```

To get only the assets that are currently in the universe, see Selected Securities .

Selected Securities

The `selected` property of your `Universe` contains references to all the assets that are currently in the universe. The `Universe.selected` property differs from the `Universe.members` property because the `Universe.members` property can contain more assets than `Universe.selected` when they have assets with open positions, open order orders, or assets that haven't met the minimum time requirement. To access the `Universe` object, save a reference to the result of the `add_universe` method. The following algorithm demonstrates how to use the `Universe.selected` property to create simple rebalancing strategies:

```
# Rebalance after market open to hold the 5 most liquid Equities.
class SimpleRebalancingAlgorithm(QCAlgorithm):

    def initialize(self):
        symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        # Schedule the universe selection to run at the start of each week.
        date_rule = self.date_rules.week_start(symbol)
        self.universe_settings.schedule.on(date_rule)
        # Add a universe of the top 5 most liquid Equities.
        universe = self.add_universe(self.universe.top(5))
        # 30 minutes after market open, rebalance the portfolio.
        self.schedule.on(
            date_rule,
            self.time_rules.after_market_open(symbol, 30),
            lambda: self.set_holdings(
                [PortfolioTarget(symbol, 1/len(universe.selected)) for symbol in universe.selected],
                liquidate_existing_holdings=True
            )
        )
```

Derivative Universes

In a regular universe, you select a basket of assets from the entire universe of securities. In a derivative universe, you select a basket of contracts for an underlying asset. The following derivative universes are available:

- Equity Options
- Futures
- Future Options
- Index Options

Historical Asset Prices

LEAN supports over 100 indicators you can use to generate your trading signals. However, in some cases, it's more efficient to maintain a matrix of historical prices for all the assets in your universe instead of using indicators. For example, say you want to calculate the correlation coefficients between all n asset in your universe using a 1-year lookback period. If you were to use the Correlation indicator, when a new asset enters the universe, you would usually need to make a 1-year history request for n assets to warm-up the n new correlation indicators. These large and frequent history requests can make your algorithm very slow.

The following algorithm demonstrates an alternative, more efficient approach. It defines a DataFrame in the initialize method . When a new asset enters the universe, it adds a column of historical prices for that asset to the DataFrame. Then, each day, it adds a single row of daily prices to the DataFrame, which covers all the assets in the universe. This approach reduces the size and frequency of history requests you need to make to get the data.

```py
class MaintainHistoricalDailyUniversePriceDataAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)

        # Add a universe of daily data.
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(
            lambda fundamentals: [f.symbol for f in sorted(fundamentals, key=lambda f: f.market_cap)[-10:]]
        )

        # Create a DataFrame to store the historical data.
        self._all_history = pd.DataFrame()
        # Define the lookback period.
        self._lookback = 252  # Trading days.

        # Create a Scheduled Event to record new daily prices and
        # rebelance the portfolio.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.at(0, 1), # One minute after `on_securities_changed` runs (in backtests)
            self._rebalance
        )

    def on_securities_changed(self, changes):
        # Remove the historical prices of assets that leave the universe.
        for security in changes.removed_securities:
            if security.symbol in self._all_history.columns:
                self._all_history.drop(security.symbol, axis=1, inplace=True)

        # Warm-up the historical data of assets that enter the universe.
        history = self.history(
            [security.symbol for security in changes.added_securities], self._lookback+1, Resolution.DAILY
        )
        if not history.empty:
            # If you trade at market open, it might make more sense to generate signals
            # based on daily opening prices. Drop the last row in `history` because
            # `_rebalance` will add it.
            self._all_history = self._all_history.join(history.open.unstack(0).iloc[:-1], how='outer')

    def _rebalance(self):
        # Add yesterday's open price to the DataFrame of historical prices.
        self._all_history = pd.concat([
            self._all_history,
            self.history(list(self._universe.selected), 1, Resolution.DAILY).open.unstack(0)
        ])
        # Trim the history to the lookback window size.
        self._all_history = self._all_history.iloc[-self._lookback:]

        # To avoid trading errors, select the assets that have a price.
        history = self._all_history[[symbol for symbol in self._all_history.columns if self.securities[symbol].price]]

        # Calculate asset signals for this rebalance.
        # For example, set the signal to give greater weight to uncorrelated assets.
        signal_by_symbol = 1/history.dropna(axis=1).corr().abs().sum()
        signal_by_symbol /= signal_by_symbol.sum()

        # Rebalance the portfolio based on the signals.
        self.set_holdings([PortfolioTarget(symbol, signal) for symbol, signal in signal_by_symbol.items()],
liquidate_existing_holdings=True)
```

Examples

You may refer to the following table for examples of various asset classes:

Asset Class	Universe Type	Example
Equities	Liquidity Universes	Example
Equities	Fundamental Universes	Example
Equities	ETF Constituents Universes	Example
Equities	Chained Universes	Example
Equities	Alternative Data Universes	Example
Equity Options	Equity Option Universes	Example
CFDs	Equity CFD Universes	Example
Futures	Future Universes	Example
Future Options	Future Option Universes	Example
Index Options	Index Option Universes	Example
Cryptos	Crypto Universes	Example
Any	Custom Universes	Example
Any	Dataless Scheduled Universes	Example

Universes

Settings

Introduction

Universe settings and security initializers enable you to configure some properties of the securities in a universe.

Resolution

The `resolution` setting defines the time period of the asset data. The `Resolution` enumeration has the following members:

To view which resolutions are available for the asset class of your universe, follow these steps:

1. Open the Asset Classes documentation page.
2. Click an asset class.
3. Click Requesting Data .
4. On the Requesting Data page, in the table of contents, click Resolutions .

The default value is `Resolution.MINUTE` . To change the resolution, in the Initialize method, pass a `resolution` argument to the universe creation method.

```
PY
# Add daily Equity Option data for the SPY.
self.add_option("SPY", resolution=Resolution.DAILY)
```

Leverage

The `leverage` setting is a `float` that defines the maximum amount of leverage you can use for a single asset in a non-derivative universe. The default value is `Security.NULL_LEVERAGE` (0). To change the leverage, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
PY
# Assign 2x leverage for all securities in the universe.
self.universe_settings.leverage = 2.0
# Add a universe of the 50 most liquid US Equities.
self.add_universe(self.universe.top(50))
```

Fill Forward

The `fill_forward` setting is a `bool` that defines whether or not too fill forward data. The default value is `True` . To disable fill forward in non-derivative universes, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
PY
# Disable fill-forward data before creating the universe (it's enabled by default).
self.universe_settings.fill_forward = False
# Add the 50 Equities to the universe that have the most dollar trading volume.
self.add_universe(self.universe.top(50))
```

To disable fill forward in derivative universes, pass a `fill_forward` argument to the universe creation method.

```
PY
# Disable fill-forward in the VIX Index Option universe.
self.add_index_option("VIX", fill_forward=False)
```

Extended Market Hours

The `extended_market_hours` setting is a `bool` that defines the trading schedule. If it's `True` , your algorithm receives price data for all trading hours. If it's `False` , your algorithm receives price data only for regular trading hours. The default value is `False` .

You only receive extended market hours data if you create the subscription with an intraday resolution. If you create the subscription with daily

resolution, the daily bars only reflect the regular trading hours.

To view the trading schedule of an asset, open Asset Classes , click an asset class, then click Market Hours .

To enable extended market hours in non-derivative universes, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
# Enable pre- and post-market hours data for assets in the universe.
self.universe_settings.extended_market_hours = True
# Add the 50 Equities to the universe that have the most dollar trading volume.
self.add_universe(self.universe.top(50))
```

To enable extended market hours in derivative universes, pass an `extended_market_hours` argument to the universe creation method.

```
# Request pre- and post-market hours data for a Futures universe.
self.add_future(Futures.Currencies.BTC, extended_market_hours=True)
```

Minimum Time in Universe

The `minimum_time_in_universe` setting is a `timedelta` object that defines the minimum amount of time an asset must be in the universe before the universe can remove it. The default value is `timedelta(1)` . To change the minimum time, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
# Keep each security in the universe for a minimum of 7 days.
self.universe_settings.minimum_time_in_universe = timedelta(7)
# Add the top 50 most liquid Equities to the universe.
self.add_universe(self.universe.top(50))
```

Data Normalization Mode

The `data_normalization_mode` setting is an enumeration that defines how historical data is adjusted. This setting is only applicable for US Equities and Futures.

US Equities

In the case of US Equities, the data normalization mode affects how historical data is adjusted for corporate actions . To view all the available options, see Data Normalization . The default value is `DataNormalizationMode.ADJUSTED` . To change the data normalization mode, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
# Pipe raw price data (unadjusted for splits and dividends) into the algorithm.
# Equity Options require raw data and it improves the accuracy of modeling historical periods.
self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
# Add a universe of the 50 most liquid US Equities.
self.add_universe(self.universe.top(50))
```

Futures

In the case of Futures, the data normalization mode affects how historical data of two contracts is stitched together to form the continuous contract . To view all the available options, see Data Normalization . The default value is `DataNormalizationMode.ADJUSTED` . To change the data normalization mode, in the Initialize method, pass a `data_normalization_mode` argument to the `add_future` method.

```
# Set a specific data normalization mode for the continuous contract
# with the data_normalization_mode parameter of the add_future method.
self.add_future(Futures.Currencies.BTC, data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO)
```

Contract Depth Offset

The `contract_depth_offset` setting is an `int` that defines which contract to use for the continuous Futures contract. 0 is the front month contract, 1 is the following back month contract, and 3 is the second back month contract. The default value is 0. To change the contract depth offset, in the

Initialize method, pass a `contract_depth_offset` argument to the `add_future` method.

```
# To adjust the contract depth offset to the second back month contract,
# pass the value 3 to the contract_depth_offset argument in the add_future method.
self.add_future(Futures.Currencies.BTC, contract_depth_offset=3)
```

Asynchronous Selection

The `asynchronous` setting is a `bool` that defines whether or not LEAN can run universe selection asynchronously, utilizing concurrent execution to increase the speed of your algorithm. The default value for this setting is `False` . If you enable this setting, abide by the following rules:

- Don't make any history requests in your filter function . History requests can only provide data up to the algorithm time , but if your filter function runs asynchronously, the algorithm time may not be what you expect.
- Don't use the portfolio, security, or orders state. The `portfolio` , `securities` , and `transactions` objects are also functions of the algorithm time.
- If your filter function updates a class variable, don't update the class variable anywhere else in your algorithm.

To enable asynchronous universe selection, in the Initialize method, adjust the algorithm's `universe_settings` before you add the universe.

```
# To speed up algorithm performance, enable asynchronous universe selection.
// Only use this when the code is self-contained and stateless.
self.universe_settings.asynchronous = True
self.add_universe(self.universe.top(50))
```

Schedule

The `schedule` setting defines the selection schedule of the universe. Most universes run on a daily schedule. To change the selection schedule, call the `universe_settings.schedule.on` method with an `IDateRule` object before you add the universe.

```
# Run universe selection on a monthly basis.
self.universe_settings.schedule.on(self.date_rules.month_start())
self.add_universe(self.universe.top(50))
```

The following table describes the supported `DateRules` :

Member	Description
`self.date_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `DateRules` object used in all methods in this table. The default time zone is the algorithm time zone .
`self.date_rules.on(year: int, month: int, day: int)`	Trigger an event on a specific date.
`self.date_rules.on(dates: list[datetime])`	Trigger an event on specific dates.
`self.date_rules.every_day()`	Trigger an event all seven days of the week.
`self.date_rules.every_day(symbol: Symbol, extended_market_hours: bool = False)`	Trigger an event every day a specific symbol is trading.
`self.date_rules.every(days: list[DayOfWeek])`	Trigger an event on specific days throughout the week. To view the `DayOfWeek` enum members, see DayOfWeek Enum in the .NET documentation.
`self.date_rules.month_start(days_offset: int = 0)`	Trigger an event on the first day of each month plus an offset.
`self.date_rules.month_start(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the first tradable date of each month for a specific symbol plus an offset.
`self.date_rules.month_end(days_offset: int = 0)`	Trigger an event on the last day of each month minus an offset.
`self.date_rules.month_end(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the last tradable date of each month for a specific symbol minus an offset.
`self.date_rules.week_start(days_offset: int = 0)`	Trigger an event on the first day of each week plus an offset.
`self.date_rules.week_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each week for a specific symbol plus an offset.
`self.date_rules.week_end(days_offset: int = 0)`	Trigger an event on the last day of each week minus an offset.
`self.date_rules.week_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each week for a specific symbol minus an offset.
`self.date_rules.year_start(days_offset: int = 0)`	Trigger an event on the first day of each year plus an offset.
`self.date_rules.year_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each year for a specific symbol plus an offset.
`self.date_rules.year_end(days_offset: int = 0)`	Trigger an event on the last day of each year minus an offset.
`self.date_rules.year_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each year for a specific symbol minus an offset.
`self.date_rules.today`	Trigger an event once today.
`self.date_rules.tomorrow`	Trigger an event once tomorrow.

To define custom date rules, create a `FuncDateRule` object. The `FuncDateRule` constructor expects a `name` argument of type `str` and a `get_dates_function` argument of type `Callable[[datetime, datetime], list[datetime]]` . The `get_dates_function` function receives the start and end dates of the algorithm and returns a list of dates for the date rule. In live trading, the end date is 12/31/2025. The following example demonstrates how to define a date rule that represents the 10th day of each month:

```py
# Create a date rule that specifies the 10th day of each month.
date_rule = FuncDateRule(
    name="10th_day_of_the_month",
    get_dates_function=lambda start, end: [
        datetime(year, month, 10)
        for year in range(start.year, end.year) for month in range(1,12)
    ]
)
```

Some date rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.date_rules.every_day('ES')` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

In live trading, scheduled universes run at roughly 8 AM Eastern Time to ensure there is enough time for the data to process.

Configure Universe Securities

Instead of configuring global universe settings, you can individually configure the settings of each security in the universe with a security initializer. Security initializers let you apply any security-level reality model or special data requests on a per-security basis. To set the security initializer, in the `initialize` method, call the `set_security_initializer` method and then define the security initializer.

```
# A custom security initializer can override default models such as
# setting new fee and fill models for the security.
self.set_security_initializer(self._custom_security_initializer)

def _custom_security_initializer(self, security: Security) -> None:
    security.set_fee_model(ConstantFeeModel(0, "USD"))
```

For simple requests, you can use the functional implementation of the security initializer. This style lets you configure the security object with one line of code.

```
# Disable the trading fees for each security by using lambda function
# for the set_security_initializer argument.
self.set_security_initializer(lambda security: security.set_fee_model(ConstantFeeModel(0, "USD")))
```

If you call the `set_security_initializer` method, it overwrites the default security initializer. The default security initializer uses the security-level reality models of the brokerage model to set the following reality models of each security:

- Fill
- Slippage
- Fee
- Buying Power
- Settlement
- Short availability
- Margin Interest Rate

The default security initializer also sets the leverage of each security and intializes each security with a seeder function. To extend upon the default security initializer instead of overwriting it, call `add_security_initializer` before you create the subscriptions.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fee_model(ConstantFeeModel(0, "USD"))
```

Examples

The following examples demonstrate some common practices for universe settings.

Example 1: Weekly-Updating Liquid Universe

The following algorithm demonstrates daily EMA cross, trading on the most liquid stocks. The universe is set to be updated weekly. Various universe settings have been set to simulate the brokerage environment best and for trading needs.

PY

```
class UniverseSettingsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # To avoid over-updating the universe, which causes insufficient time to earn the trend, update the universe weekly.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        self.universe_settings.minimum_time_in_universe = timedelta(7)
        # Rebalancing frequency is daily, so we only need to subscribe to daily resolution.
        self.universe_settings.resolution = Resolution.DAILY
        # To best simulate IB's margin requirement, set the leverage to 50%.
        self.universe_settings.leverage = 2
        # Since we trade by market order, we cannot use extended market hours data to fill.
        self.universe_settings.extended_market_hours = False
        # We want to trade the EMA with raw price but not altered by splits.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.SPLIT_ADJUSTED

                # Only trade on the top 10 most traded stocks since they have the most popularity to drive trends.
        self.add_universe(self.universe.top(10))

    def on_data(self, slice: Slice) -> None:
        for symbol, bar in slice.bars.items():
            # Trade the trend by EMA crossing.
            ema = self.securities[symbol].ema.current.value
            if bar.close > ema:
                self.set_holdings(symbol, 0.05)
            else:
                self.set_holdings(symbol, -0.05)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for removed in changes.removed_securities:
            # Liquidate the ones leaving the universe since we will not trade their EMA anymore.
            self.liquidate(removed.symbol)
            # Deregister the EMA indicator to free resources
            self.deregister_indicator(removed.ema)

        for added in changes.added_securities:
            # Create EMA indicator for trend trading signals.
            added.ema = self.ema(added.symbol, 50, Resolution.DAILY)
            # Warm up the EMA indicator to ensure its readiness for immediate use.
            self.warm_up_indicator(added.symbol, added.ema, Resolution.DAILY)
```

Universes

Equity

You can create any of the following dynamic universes of equity. Click one to learn more.

Liquidity Universes

Fundamental Universes

ETF Constituents Universes

Chained Universes

Legacy Fundamental Universes

Alternative Data Universes

See Also

US Equity

Equity

Liquidity Universes

Introduction

A liquidity universe lets you select a set of stocks with the greatest dollar volume in the market.

Create Universes

To add a dollar volume universe, call the `universe.dollar_volume.top` helper method and pass the result to the `add_universe` method.

```py
# Add the 50 stocks with the highest dollar volume.
self.universe_settings.asynchronous = True
self.add_universe(self.universe.top(50))
```

Selection Frequency

Equity universes run on a daily basis by default. To adjust the selection schedule, see Schedule .

Examples

The following examples demonstrate some common liquidity universes for US Equities.

Example 1: Minute-Resolution Universe

The following algorithm asynchronously selects the 10 most liquid US Equities as the universe. It uses minute resolution data to provide the most realistic results.

```py
class MinuteLiquidUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Use asynchronous universe selection to speed up your algorithm.
        self.universe_settings.asynchronous = True
        # Use minute resolution data so orders fill at realistic prices.
        self.universe_settings.resolution = Resolution.MINUTE
        # Add a universe of the 10 most liquid US Equities.
        self.add_universe(self.universe.top(10))
```

Example 2: Trade Liquid Assets off Bollinger Bands

The following algorithm asynchronously selects the 500 most liquid US Equities as the universe. Each day, it forms a portfolio of the assets in the universe that are below their lower Bollinger Band .

```py
class AsynchronousLiquidUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.automatic_indicator_warm_up = True
        # When a new asset enters the universe, seed its current price so you can trade it right away.
        self.settings.seed_initial_prices = True
        # Use asynchronous universe selection to speed up the algorithm.
        self.universe_settings.asynchronous = True
        # Use daily data. It doesn't affect accuracy since the trades fill at the daily market open.
        self.universe_settings.resolution = Resolution.DAILY
        # Add a universe of the 500 most liquid Equities.
        self._universe = self.add_universe(self.universe.top(500))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Create Bollinger Band indicators for the universe constituents.
        for security in changes.added_securities:
            security.bb = self.bb(security.symbol, 60, 2)
        # Deregister the indicators when assets leave the universe.
        for security in changes.removed_securities:
            self.deregister_indicator(security.bb)

    def on_data(self, slice: Slice) -> None:
        # Ensure there are TradeBar objects in the current slice.
        if not slice.bars:
            return
        # Select the Equities that are below their lower Bollinger Band.
        securities = [self.securities[symbol] for symbol in self._universe.selected]
        selected = [
            security for security in securities
            if security.bb.is_ready and security.price and security.price < security.bb.lower_band.current.value
        ]
        # Form an equal-weighted portfolio with the selected assets.
        weight = 1 / len(selected)
        self.set_holdings([PortfolioTarget(security.symbol, weight) for security in selected], liquidate_existing_holdings=True)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
UniverseSelectionDefinitionsAlgorithm.py Python

Equity

Fundamental Universes

Introduction

A fundamental universe lets you select stocks based on corporate fundamental data. This data is powered by Morningstar® and includes approximately 8,100 tickers (including delisted companies and excluding ETFs, ADRs, and OTC) with 900 properties each.

Create Universes

To add a fundamental universe, in the `initialize` method, pass a filter function to the `add_universe` method. The filter function receives a list of `Fundamental` objects and must return a list of `Symbol` objects. The `Symbol` objects you return from the function are the constituents of the fundamental universe and LEAN automatically creates subscriptions for them. Don't call `add_equity` in the filter function.

```
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        # Add a fundamental universe with a custom filter function.
        self._universe = self.add_universe(self._fundamental_function)

    def _fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select US Equities that have fundamental data.
        return [c.symbol for c in fundamental if c.has_fundamental_data]
```

`Fundamental` objects have the following attributes:

Example

The simplest example of accessing the fundamental object would be harnessing the iconic PE ratio for a stock. This is a ratio of the price it commands to the earnings of a stock. The lower the PE ratio for a stock, the more affordable it appears.

```
# In the initialize method:
self.universe_settings.asynchronous = True
# Add a fundamental universe with a custom filter function.
self._universe = self.add_universe(self._fundamental_selection_function)

def _fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
    # Select assets that have a price > $10, fundamental data, and a price_to_earnings ratio.
    filtered = [f for f in fundamental if f.price > 10 and f.has_fundamental_data and not np.isnan(f.valuation_ratios.pe_ratio)]
    # Select the 100 most liquid assets.
    sorted_by_dollar_volume = sorted(filtered, key=lambda f: f.dollar_volume, reverse=True)[:100]
    # Select the 10 assets with the lowest P/E ratios.
    sorted_by_pe_ratio = sorted(sorted_by_dollar_volume, key=lambda f: f.valuation_ratios.pe_ratio)[:10]
    # Return the selected securities.
    return [f.symbol for f in sorted_by_pe_ratio]
```

Asset Categories

In addition to valuation ratios, the US Fundamental Data from Morningstar has many other data point attributes, including over 200 different categorization fields for each US stock. Morningstar groups these fields into sectors, industry groups, and industries.

Sectors are large super categories of data. To get the sector of a stock, use the `morningstar_sector_code` property.

```
# Select the US Equities in the technology sector.
tech = [x for x in fundamental if x.asset_classification.morningstar_sector_code == MorningstarSectorCode.TECHNOLOGY]
```

Industry groups are clusters of related industries that tie together. To get the industry group of a stock, use the `morningstar_industry_group_code` property.

```
# Select the US Equities in the agricluture industry group.
ag = [x for x in fundamental if x.asset_classification.morningstar_industry_group_code == MorningstarIndustryGroupCode.AGRICULTURE]
```

Industries are the finest level of classification available. They are the individual industries according to the Morningstar classification system. To get the industry of a stock, use the `morningstar_industry_code` property.

```
# Select the US Equities in the coal industry.
coal = [x for x in fundamental if x.asset_classification.morningstar_industry_code == MorningstarIndustryCode.COAL]
```

Practical Limitations

Fundamental universes allow you to select an unlimited universe of assets to analyze. Each asset in the universe consumes approximately 5MB of RAM, so you may quickly run out of memory if your universe filter selects many assets. If you backtest your algorithms in the Algorithm Lab, familiarize yourself with the RAM capacity of your backtesting and live trading nodes . To keep your algorithm fast and efficient, only subscribe to the assets you need.

Data Availability

`Fundamental` objects can have NaN values for some of their properties. Before you sort the `Fundamental` objects by one of the properties, filter out the objects that have a NaN value for the property.

```
def _fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
    # Select objects that have a value for the fundamental property.
    filtered = [f for f in fundamental if f.has_fundamental_data and not np.isnan(f.valuation_ratios.pe_ratio)]
    # Sort the objects by the fundamental property.
    sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
    return [f.symbol for f in sortedByPeRatio[:10] ]
```

Direct Access

To get fundamental data for Equities in your algorithm, use the `fundamentals` property of the `Equity` objects. The fundamental data represent the company's fundamentals for the current algorithm time.

```
fundamentals = self.securities[self._symbol].fundamentals
```

To get fundamental data for Equities, regardless of whether or not you have subscribed to them in your algorithm, call the `fundamentals` method. If you pass one `Symbol` , the method returns a `Fundamental` object. If you pass a list of `Symbol` objects, the method returns a list of `Fundamental` objects. The fundamental data represents the corporate fundamentals for the current algorithm time.

```
# Single asset
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals(ibm)

# Multiple assets
nb = Symbol.create("NB", SecurityType.EQUITY, Market.USA)
fundamentals = self.fundamentals([ nb, ibm ])
```

Data Availability

Some assets don't have fundamentals (for example, ETFs) and the Morningstar dataset doesn't provide fundamentals for all US Equities. To check if fundamental data is available for an asset, use the `has_fundamental_data` property.

```
has_fundamental_data = self.securities[self._symbol].fundamentals.has_fundamental_data
```

Object References

If you save a reference to the `fundamentals` object or its properties, you can access the fundamental properties as they change over time.

```
self._fundamentals = self.securities[self._symbol].fundamentals
earning_ratios = self.fundamentals.earning_ratios
```

Historical Data

To get historical fundamental data, call the `history` method. The return type depends on how you call the method.

```
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)

# DataFrame of fundamental data for a given asset
df_history = self.history(Fundamental, ibm, timedelta(30), flatten=True)

# Fundamental objects
fundamental_history = self.history[Fundamental](ibm, timedelta(30))

# Fundamentals objects for all US Equities (including delisted companies)
fundamentals_history = self.history[Fundamentals](timedelta(30))

# DataFrame of fundamental data for universe constituents
df_history = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series of fundamental data for universe constituents
series_history = self.history(self._universe, 30, Resolution.DAILY)
for (universe_symbol, time), fundamental in series_history.items():
    highest_market_cap = sorted(fundamental, key=lambda x: x.market_cap)[-5:]
```

For more information about historical fundamental data, see Equity Fundamental Data .

Selection Frequency

Equity universes run on a daily basis by default. To adjust the selection schedule, see Schedule .

Live Trading Considerations

The live data for fundamental universe selection arrives at 6/7 AM Eastern Time (ET), so fundamental universe selection runs for live algorithms between 7 and 8 AM ET. This timing allows you to place trades before the market opens. Don't schedule anything for midnight because the universe selection data isn't ready yet.

Examples

The following examples demonstrate some common practices for fundamental universes.

Example 1: 500 Stocks >10/Share and >10M in Daily Trading Volume

The following algorithm selects the 500 most liquid US Equities above 10/share and 10M in daily volume.

```
class LiquidNonPennyStocksUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Configure the universe to update at the start of each month. Most of the top 500 doesn't change very
        # frequently.
        self.universe_settings.schedule.on(self.date_rules.month_start())
        # Add a universe with custom selection rules for filtering.
        self.add_universe(
            lambda fundamental: [
                x.symbol for x in sorted(
                    [f for f in fundamental if f.price > 10 and f.dollar_volume > 10_000_000],
                    key=lambda f: f.dollar_volume
                )[-500:]
            ]
        )
```

Example 2: 10 Stocks Above Their 200-Day EMA With >$1B of Daily Trading Volume

Another common request is to filter the universe by a technical indicator, such as only picking stocks above their 200-day Exponential Moving Average (EMA). The `Fundamental` object has daily price and volume information, so you can do any price-related analysis. The following algorithm defines a separate class to contain the indicator of each asset.

```py
class UpTrendLiquidUniverseAlgorithm(QCAlgorithm):

    _selection_data_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        # Add the custom universe.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self._universe = self.add_universe(self._select_assets)
        # Add a warm-up period to warm up the indicators.
        self.set_warm_up(timedelta(300))

    def _select_assets(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Update the indicator of all stocks in the universe dataset and
        # get the subset of stocks that have their indicator ready.
        ready_stocks = [
            f for f in fundamentals
            if self._selection_data_by_symbol.setdefault(f.symbol, SelectionData(self, f, 200)).update(f)
        ]
        # As assests leave the Fundamental dataset, delete their SelectionData object.
        for symbol in self._selection_data_by_symbol.keys() - {f.symbol for f in fundamentals}:
            del self._selection_data_by_symbol[symbol]
        # During warm-up, keep the universe empty.
        if self.is_warming_up:
            return []
        # Select the Equities that are above their EMA and have a daily volume of $1B.
        # These assets are in an uptrend and are very liquid.
        selected = [self._selection_data_by_symbol[f.symbol] for f in ready_stocks]
        selected = [x for x in selected if x.is_above_ema and x.volume > 1_000_000_000]
        # Select the 10 most liquid Equities to avoid extra slippage.
        return [x.symbol for x in sorted(selected, key=lambda x: x.volume)[-10:]]


# Create a separate class to contain the EMA information of each asset.
class SelectionData:

    def __init__(self, algorithm, f, period):
        self._algorithm = algorithm
        self._price_scale_factor = f.price_scale_factor
        # Create an EMA indicator for trend estimation and filtering.
        self.symbol = f.symbol
        self._ema = ExponentialMovingAverage(period)
        self.is_above_ema = False
        self.volume = 0

    # Update your variables and indicators with the latest data.
    def update(self, f):
        # If there hasn't been a split or dividend since the last trading
        # day, just update the indicator like normal.
        if f.price_scale_factor == self._price_scale_factor:
            return self._update(f.end_time, f.volume, f.price)
        # Otherwise, reset the indicator and warm it up with the new
        # adjusted history.
        self._price_scale_factor = f.price_scale_factor
        self._ema.reset()
        history = self._algorithm.history[TradeBar](
            self.symbol,
            self._ema.warm_up_period,
            Resolution.DAILY,
            data_normalization_mode=DataNormalizationMode.SCALED_RAW
        )
        for bar in history:
            self._update(bar.end_time, bar.volume, bar.close)
        return self._ema.is_ready

    def _update(self, end_time, volume, price):
        self.volume = volume * price
        if self._ema.update(end_time, price):
            self.is_above_ema = price > self._ema.current.value
        return self._ema.is_ready
```

In this example, the `SelectionData` class group variables for the universe selection and updates the indicator of each asset. We highly recommend you follow this pattern to keep your algorithm tidy and bug free. The following snippet shows an example implementation of the `SelectionData` class, but you can make this whatever you need to store your custom universe filters. Note that the preceding `SelectionData` class uses a manual EMA indicator instead of the automatic version . For more information about universes that select assets based on indicators, see Indicator Universes . You need to use a `SelectionData` class instead of assigning the EMA to the `Fundamental` object because you can't create custom attributes on `Fundamental` objects like you can with `Security` objects.

Example 3: 10 Stocks Furthest Above their 10-day SMA of Volume

The process to get the 10-day Simple Moving Average (SMA) stock volume is the same process as in Example 2. First, define a `SelectionData` class

that performs the averaging. This class tracks the ratio of today's volume relative to historical volumes. You can use this ratio to select assets that are above their 10-day SMA and sort the results by the Equities that have had the biggest jump since yesterday.

```py
class HighRelativeVolumeUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        # Add the custom universe.
        self._selection_data_by_symbol = {}
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self._universe = self.add_universe(self._select_assets)
        # Add a warm-up period to warm up the indicators.
        self.set_warm_up(timedelta(30))

    def _select_assets(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Update the indicator of all stocks in the universe dataset and
        # get the subset of stocks that have their indicator ready.
        ready_stocks = [
            f for f in fundamentals
            if self._selection_data_by_symbol.setdefault(f.symbol, SelectionData(self, f, 10)).update(f)
        ]
        # As assests leave the Fundamental dataset, delete their SelectionData object.
        for symbol in self._selection_data_by_symbol.keys() - {f.symbol for f in fundamentals}:
            del self._selection_data_by_symbol[symbol]
        # During warm-up, keep the universe empty.
        if self.is_warming_up:
            return []
        # Select the Equities with higher trading volume than their SMA, indicating higher capital flow.
        selected = [self._selection_data_by_symbol[f.symbol] for f in ready_stocks]
        selected = [sd for sd in selected if sd.volume_ratio > 1]
        # Select the 10 Equities with the highest relative volume, since they have the highest capactity
        # for scalp-trading or intra-day movement.
        return [x.symbol for x in sorted(selected, key=lambda x: x.volume_ratio)[-10:]]


# Create a separate class to contain the SMA information of each asset.
class SelectionData:

    def __init__(self, algorithm, f, period):
        self._algorithm = algorithm
        self._price_scale_factor = f.price_scale_factor
        # Create an SMA of volume to track the popularity of the stock.
        self.symbol = f.symbol
        self._sma = SimpleMovingAverage(period)

    # Update your variables and indicators with the latest data.
    def update(self, f):
        # If there hasn't been a split or dividend since the last trading
        # day, just update the indicator like normal.
        if f.price_scale_factor == self._price_scale_factor:
            return self._update(f.end_time, f.volume)
        # Otherwise, reset the indicator and warm it up with the new
        # adjusted history.
        self._price_scale_factor = f.price_scale_factor
        self._sma.reset()
        history = self._algorithm.history[TradeBar](
            self.symbol,
            self._sma.warm_up_period,
            Resolution.DAILY,
            data_normalization_mode=DataNormalizationMode.SCALED_RAW
        )
        for bar in history:
            self._update(bar.end_time, bar.volume)
        return self._sma.is_ready

    def _update(self, end_time, volume):
        # Update the SMA with today's data and calculate the relative volume position for filtering.
        if self._sma.update(end_time, volume):
            self.volume_ratio = volume / self._sma.current.value if self._sma.current.value != 0 else -1
        return self._sma.is_ready
```

Example 4: 10 "Fastest Moving" Stocks With a 50-Day EMA > 200 Day EMA

You can construct complex universe filters with the `SelectionData` helper class pattern. To view a full example of this algorithm, see the EmaCrossUniverseSelectionAlgorithm in the LEAN GitHub repository or take the related Boot Camp lesson .

Example 5: Piotroski F-Score

To view this example, see the Piotroski F-Score Investing Research post.

Example 6: Stocks Far Above Their SMA

The following example shows how to select a universe of US Equities based on their price and SMA indicator. The `SelectionData` class keeps track of the SMA indicator for each stock in the universe dataset. When a split or dividend occurs for a stock, the data in its indicator becomes invalid because it doesn't account for the price adjustments that the split or dividend causes. The `SelectionData` class resets and warms up the indicator with the `SCALED_RAW` data normalization mode , which gives you accurate indicator values to use in your universe selection after each corporate action.

PY

```
class EquityIndicatorUniverseSelectionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        # Add a universe of US Equities based on an indicator.
        self._selection_data_by_symbol = {}
        self._universe = self.add_universe(self._select_assets)
        # Add a warm-up period to warm up the indicators in the universe selection.
        self.set_warm_up(timedelta(60))

    def _select_assets(self, fundamentals):
        # Update the indicator of all stocks in the universe dataset and
        # get the subset of stocks that have their indicator ready.
        ready_stocks = [
            f for f in fundamentals
            if self._selection_data_by_symbol.setdefault(f.symbol, SelectionData(self, f)).update(f)
        ]
        # As assests leave the Fundamental dataset, delete their SelectionData object.
        for symbol in self._selection_data_by_symbol.keys() - {f.symbol for f in fundamentals}:
            del self._selection_data_by_symbol[symbol]
        # During warm-up, keep the universe empty.
        if self.is_warming_up:
            return []
        # Select a subset of the stocks based on the indicator.
        # Example: 10 stocks furthest above their SMA.
        factor_by_symbol = {
            f.symbol: f.price / self._selection_data_by_symbol[f.symbol].indicator.current.value
            for f in ready_stocks
        }
        return sorted(
            {k: v for k, v in factor_by_symbol.items() if v > 0},
            key=lambda symbol: factor_by_symbol[symbol]
        )[-100:]


class SelectionData:

    def __init__(self, algorithm, f):
        self._algorithm = algorithm
        self._price_scale_factor = f.price_scale_factor
        self.indicator = SimpleMovingAverage(21)

    def update(self, f):
        # If there hasn't been a split or dividend since the last trading
        # day, just update the indicator like normal.
        if f.price_scale_factor == self._price_scale_factor:
            return self.indicator.update(f.end_time, f.price)
        # Otherwise, reset the indicator and warm it up with the new
        # adjusted history.
        self._price_scale_factor = f.price_scale_factor
        self.indicator.reset()
        history = self._algorithm.history[TradeBar](
            f.symbol,
            self.indicator.warm_up_period,
            Resolution.DAILY,
            data_normalization_mode=DataNormalizationMode.SCALED_RAW
        )
        for bar in history:
            self.indicator.update(bar)
        return self.indicator.is_ready
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
FundamentalUniverseSelectionAlgorithm.py Python

Equity

ETF Constituents Universes

Introduction

An ETF constituents universe lets you select a universe of securities in an exchange traded fund. The US ETF Constituents dataset includes 2,650 US ETFs you can use to create your universe.

Create Universes

To add an ETF Constituents universe, call the `universe.etf` method.

```
class ETFConstituentsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self.universe.etf("SPY"))
```

The following table describes the `etf` method arguments:

Argument: `etf_ticker` The ETF ticker. To view the supported ETFs in the US ETF Constituents dataset, see Supported ETFs . Data Type: `str` | Default Value: `None`
Argument: `universe_settings` The universe settings . If you don't provide an argument, it uses the algorithm `universe_settings` . Data Type: `UniverseSettings` | Default Value: `None`
Argument: `universe_filter_func` A function to select some of the ETF constituents for the universe. If you don't provide an argument, it selects all of the constituents. Data Type: `Callable[[List[ETFConstituentUniverse]], List[Symbol]]` | Default Value: `None`

To select a subset of the ETF constituents, provide a `universe_filter_func` argument. The filter function receives `ETFConstituentUniverse` objects, which represent one of the ETF constituents. `ETFConstituentUniverse` objects have the following attributes:

```
class ETFConstituentsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self._universe = self.universe.etf("SPY", self.universe_settings, self._etf_constituents_filter)
        self.add_universe(self._universe)

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Get the 10 securities with the largest weight in the index
        selected = sorted(
            constituents, key=lambda c: c.weight or 0.0, reverse=True
        )[:10]
        return [c.symbol for c in selected]
```

Historical Data

To get historical ETF constituents data, call the `history` method with the `Universe` object and the lookback period.

```py
# DataFrame object:
df_history = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series object:
series_history = self.history(self._universe, 30, Resolution.DAILY)
for (universe_symbol, time), constituents in history.items():
    for constituent in constituents:
        self.log(f'{constituent.symbol} weight at {constituent.end_time}: {constituent.weight}')
```

For more information about ETF Constituents data, see US ETF Constituents .

Selection Frequency

Equity universes run on a daily basis by default. To adjust the selection schedule, see Schedule .

Examples

The following examples demonstrate some common practices for ETF constituent universes.

Example 1: Short the Smallest ETF Constituents

A subset of the SPY constituents outperform the SPY while many of the constituents underperform the overall index. In an attempt to exclude the ETF constituents that underperform the index, the following algorithm buys the SPY and shorts the 100 assets in the index with the smallest weight in the ETF.

```py
class ETFConstituentsUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10_000_000)
        self.settings.minimum_order_margin_portfolio_percentage = 0
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # To avoid over-trading and high transaction costs, refilter and rebalance weekly.
        self.universe_settings.schedule.on(self.date_rules.week_end())
        # Add the SPY ETF.
        self._spy = self.add_equity("SPY").symbol
        # Add a universe of the SPY constituents.
        self._universe = self.add_universe(self.universe.etf(self._spy, universe_filter_func=self._select_assets))
        # Create a Scheduled Event to rebalance the portfolio.
        self.schedule.on(self.date_rules.week_start(), self.time_rules.at(9, 0), self._rebalance)

    def _select_assets(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Cache the constituent weights in a dictionary for filtering and position sizing.
        self._etf_weight_by_symbol = {c.symbol: c.weight for c in constituents if c.weight}
        # Select the 100 consituents with the smallest weight in the ETF.
        # They should have negative excess return.
        return [symbol for symbol, _ in sorted(self._etf_weight_by_symbol.items(), key=lambda x: x[1])[:100]]

    def _rebalance(self) -> None:
        # Get the ETF weight of all the assets currently in the universe.
        weight_by_symbol = {
            symbol: self._etf_weight_by_symbol[symbol] for symbol in self._universe.selected
            # To avoid trading errors, skip assets that have no price yet.
            if self.securities[symbol].price
        }
        # Buy the SPY to eliminate systematic risk.
        targets = [PortfolioTarget(self._spy, 1-sum(weight_by_symbol.values()))]
        # Sell the 100 ETF constituents with the lowest weight due to expected negative excess return.
        targets.extend(
            [PortfolioTarget(symbol, -weight) for symbol, weight in weight_by_symbol.items()]
        )
        # Place orders to rebalance the portfolio.
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Example 2: SPY Constituents in an Uptrend

The following example chains an ETF constituents universe and a fundamental universe . It first selects all the constituents of the SPY ETF and then filters them down in the fundamental universe filter to select the assets that are trading above their average price over the last 200 days. The output of the fundamental universe selection method is the output of the chained universe. The `Fundamental` objects that the fundamental universe filter function recieves contains the prices of the ETF constituents. By chaining the ETF constituents universe into the fundamental universe, you can update the indicators with the price instead of making a history request .

```py
class ChainedUniverseAlgorithm(QCAlgorithm):

    _selection_data_by_symbol = {}

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset that enters the universe with its last known price so you can trade it
        # on the same morning it enters the universe without getting warnings.
        self.settings.seed_initial_prices = True
        # Add a chained universe that selects the SPY constituents trading above their 200-day SMA.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self._universe = self.add_universe(self.universe.etf(spy), self._fundamental_selection)
        # Trade daily at market open since the trading signal is generated on a daily resolution.
        self.schedule.on(self.date_rules.every_day(spy), self.time_rules.after_market_open(spy, 1), self._rebalance)

    def _fundamental_selection(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Create/Update an SMA indicator for each asset that enters the ETF.
        universe_symbols = []
        for f in fundamental:
            universe_symbols.append(f.symbol)
            if f.symbol not in self._selection_data_by_symbol:
                self._selection_data_by_symbol[f.symbol] = SelectionData(self, f.symbol)
            self._selection_data_by_symbol[f.symbol].update(self.time, f.adjusted_price)

        # Remove indicators for assets that are no longer in the ETF to release the computational resources.
        symbols_to_remove = [s for s in self._selection_data_by_symbol.keys() if s not in universe_symbols]
        for symbol in symbols_to_remove:
            self._selection_data_by_symbol.pop(symbol)

        # Select the Equities trading above their SMA as we estimate them to be in uptrend and still rising.
        selected = [
            symbol for symbol, selection_data in self._selection_data_by_symbol.items() if selection_data.is_above_sma
        ]

        # Plot the results.
        self.plot("Universe", "Possible", len(list(fundamental)))
        self.plot("Universe", "Selected", len(selected))

        return selected

    def _rebalance(self) -> None:
        # Form an equal-weighted portfolio with the Equities that are above their SMA.
        symbols = [symbol for symbol in self._universe.selected if self.securities[symbol].price]
        if not symbols:
            return
        weight = 1 / len(symbols)
        self.set_holdings([PortfolioTarget(symbol, weight) for symbol in symbols], liquidate_existing_holdings=True)


# Define a separate class to contain the SMA indicator.
class SelectionData(object):

    def __init__(self, algorithm, symbol, period=200):
        #  Create the SMA indicator for trend detection and filtering.
        self._sma = SimpleMovingAverage(period)
        # Warm up the SMA so you can immediately use it.
        algorithm.warm_up_indicator(symbol, self._sma, Resolution.DAILY)

    def update(self, time, value):
        self.is_above_sma = self._sma.update(time, value) and value > self._sma.current.value
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
ETFConstituentUniverseRSIAlphaModelAlgorithm.py Python UniverseOnlyRegressionAlgorithm.py Python

Equity

Chained Universes

Introduction

You can combine ("chain") universes together to fetch fundamental and alternative data on a specific subset of assets. Universes filter input data and return `Symbol` objects. The only requirement is that `Symbol` objects the filter returns are a subset of the input data. The source of the `Symbol` objects is unrestricted, so you can feed the output of one universe into another.

Filter Pattern

Universes filter a large set of Symbol objects by a coarse filter to quickly reduce the data processing requirement. This is often a first step before applying a second filter or requesting alternative data. For example, a strategy might only be interested in easily tradable liquid assets so quickly eliminates all stocks with less than $1M USD / day in trading volume.

The order of your filters can improve the speed of your research. By applying filters that narrow the universe the most, or are the lightest weight first, you can significantly reduce the amount of data your algorithm processes. Unless necessary, you can also not return any selections from earlier filters to further improve research speed, keeping only the universe data for later filters.

Universe Data Weights

To speed up your algorithm, request the lightest weight data first before chaining heavier filters or adding alternative data. The following table shows the size each dataset:

Name	**Data Size / Weight**
US Equities (Fundamental - Dollar Volume only)	Light (100 MB)
US Equities (Fundamental)	Heavy (up to 20 GB)
US Equity Options	Huge (200 TB)
US Index Options	Medium (500 GB)
US Futures	Medium (500 GB)
US Futures Options	Medium (500 GB)
Crypto	Light (1 GB)
Alternative / General	Light (100 MB - 2 GB)
Alternative / Tiingo News	Medium (200 GB)

Chain Fundamental and Alternative Data

The following example chains a fundamental universe and a QuiverCNBCsUniverse alternative universe . It first selects the 100 most liquid US Equities and then filters them down to those mentioned by CNBC commentator/trader Jim Cramer. The output of the alternative universe selection method is the output of the chained universe.

```py
from AlgorithmImports import *

class ChainedUniverseAlgorithm(QCAlgorithm):

    _fundamental = []

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_universe(self._fundamental_filter_function)
        self.add_universe(QuiverCNBCsUniverse, self._mad_money_selection)

    def _fundamental_filter_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Filter the top 100 liquid equities of the last trading day, and save the symbols for the next filtering.
        sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)
        self._fundamental = [c.symbol for c in sorted_by_dollar_volume[:100]]
        return Universe.UNCHANGED

    def _mad_money_selection(self, alt_coarse: List[QuiverCNBCsUniverse]) -> List[Symbol]:
        # Filter the equities being commented on by CNBC analyst Cramer, then select the ones that intersect with the fundamental
universe.
        madmoney = [d.symbol for d in alt_coarse if 'Cramer' in d.traders]
        return list(set(self._fundamental) & set(madmoney))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Request CNBC data for the selected stocks.
        for added in changes.added_securities:
            self.add_data(QuiverCNBCs, added.symbol)

    def on_data(self, data: Slice) -> None:
        # Prices in the slice from the universe selection
        # Alternative data in a slice from OnSecuritiesChanged Addition
        # for ticker,bar in data.bars.items():
        #     pass
        for dataset_symbol, data_points in data.get(QuiverCNBCs).items():
            for data_point in data_points:
                self.debug(f"{dataset_symbol} traders at {data.time}: {data_point.traders}")
```

Chain Fundamental and US Equity Options

The following example chains a fundamental universe and an Equity Options universe . It first selects 10 stocks with the lowest PE ratio and then selects their front-month call Option contracts. The output of both universes is the output of the chained universe.

```py
from AlgorithmImports import *

class ChainedUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Need to set data normalization mode to raw for options to compare the strike price fairly.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        # Filter for equities with fundamental data first.
        universe = self.add_universe(self._fundamental_function)
        # Based on the filtered equities, request an option universe with them as underlying.
        self.add_universe_options(universe, self._option_filter_function)

    def _fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Filter for equities with the lowest PE Ratio first.
        filtered = [f for f in fundamental if not np.isnan(f.valuation_ratios.pe_ratio)]
        sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
        return [f.symbol for f in sorted_by_pe_ratio[:10]]

    def _option_filter_function(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Select the front month ATM calls.
        return option_filter_universe.strikes(0, 2).front_month().calls_only()

    def on_data(self, data: Slice) -> None:
        # Iterate through each Option chain.
        for canonical_symbol, chain in data.option_chains.items():
            equity_symbol = canonical_symbol.underlying
            for contract in chain:
                contract_symbol = contract.symbol
                price = contract.last_price
```

Chain ETF and Fundamental

The following example chains a fundamental universe and an ETF constituents universe . It first selects all the constituents of the QQQ ETF and then filters them down to select the 10 assets with the lowest PE ratio. The output of the fundamental universe selection method is the output of the

chained universe.

```
from AlgorithmImports import *


class ChainedUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Select QQQ constituents first, then by fundamental data.
        self._universe = self.add_universe(
            self.universe.etf("QQQ", Market.USA, self.universe_settings, self._etf_constituents_filter),
            self._fundamental_selection
        )

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Select all QQQ constituents.
        return [c.symbol for c in constituents]

    def _fundamental_selection(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Filter for the lowest PE Ratio stocks among QQQ constituents for undervalued stocks.
        filtered = [f for f in fundamental if not np.isnan(f.valuation_ratios.pe_ratio)]
        sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
        return [f.symbol for f in sorted_by_pe_ratio[:10]]

    def on_data(self, data):
        for symbol in self._universe.selected:
            pe_ratio = self.securities[symbol].fundamentals.valuation_ratios.pe_ratio
```

Chain ETF and Alternative Data

The following example chains an ETF universe and a QuiverCNBCsUniverse alternative universe . It first selects all constituents of SPY and then filters them down to those mentioned by CNBC commentator/trader Jim Cramer. The output of the alternative universe selection method is the output of the chained universe.

```
from AlgorithmImports import *

class ChainedUniverseAlgorithm(QCAlgorithm):

    _etf = []

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Save all SPY constituents for the next filtering.
        self.add_universe(self.universe.etf("SPY", Market.USA, self.universe_settings, self._etf_constituents_filter))
        # Next filtering based on CNBC data.
        self.add_universe(QuiverCNBCsUniverse, self._mad_money_selection)

    def _etf_constituents_filter(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Save all SPY constituents for the next filtering.
        self._etf = [c.symbol for c in constituents]
        return Universe.UNCHANGED

    def _mad_money_selection(self, alt_coarse: List[QuiverCNBCsUniverse]) -> List[Symbol]:
        # Filter the equities being commented on by CNBC analyst Cramer, then select the ones in SPY constituents.
        madmoney = [d.symbol for d in alt_coarse if 'Cramer' in d.traders]
        return list(set(self._etf) & set(madmoney))

    def on_securities_changed(self, changes):
        # Request CNBC data for the selected stocks.
        for added in changes.added_securities:
            self.add_data(QuiverCNBCs, added.symbol)

    def on_data(self, data):
        # Prices in the slice from the universe selection
        # Alternative data in a slice from OnSecuritiesChanged Addition
        # for ticker,bar in data.bars.items():
        #     pass
        for dataset_symbol, data_points in data.get(QuiverCNBCs).items():
            for data_point in data_points:
                self.debug(f"{dataset_symbol} traders at {data.time}: {data_point.traders}")
```

Chain ETF and US Equity Options

The following example chains an ETF constituents universe and an Equity Options universe . It first selects the 10 largest-weighted constituents of QQQ and then selects their front-month call Option contracts. The output of both universes is the output of the chained universe.

PY

```
from AlgorithmImports import *


class ChainedUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Set the data normalization mode to raw for fair strike price comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW

        # First-pass filtering from QQQ constituents.
        etf_universe = self.universe.etf("QQQ", Market.USA, self.universe_settings, self._etf_constituents_filter)
        self.add_universe(etf_universe)
        # Filter for equity options from the selected QQQ constituents next.
        self.add_universe_options(etf_universe, self._option_filter_function)

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Select the top 10 weighted QQQ constituents for most liquid options.
        sorted_by_weight = sorted(constituents, key=lambda x: x.weight, reverse=True)
        return [c.symbol for c in sorted_by_weight[:10]]

    def _option_filter_function(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Select the front month ATM standard calls.
        return option_filter_universe.strikes(0, 2).front_month().calls_only()

    def on_data(self, data: Slice) -> None:
        # Iterate through each Option chain.
        for canonical_symbol, chain in data.option_chains.items():
            equity_symbol = canonical_symbol.underlying
            for contract in chain:
                contract_symbol = contract.symbol
                price = contract.last_price
```

Equity

Legacy Fundamental Universes

Introduction

Warning: This API for Universe Selection was deprecated on November 2023. Please refer to the new Fundamental Universe API .

There are several ways to create an Equities universe. You can select a universe based on `CoarseFundamental` data or the constituents of an ETF, and then you can further filter your universe down with corporate fundamentals. The following sections explain each of these techniques in detail.

Coarse Universe Selection

A coarse universe enables you pick a set of stocks based on their trading volume, price, or whether they have fundamental data. To add a coarse universe, in the `initialize` method, pass a filter function to the `add_universe` method. The coarse filter function receives a list of `CoarseFundamental` objects and must return a list of `Symbol` objects. The `Symbol` objects you return from the function are the constituents of the universe and LEAN automatically creates subscriptions for them. Don't call `add_equity` in the filter function.

```py
class MyCoarseUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._coarse_filter_function)

    def _coarse_filter_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        sorted_by_dollar_volume = sorted(coarse, key=lambda x: x.dollar_volume, reverse=True)
        return [c.symbol for c in sorted_by_dollar_volume[:100]]
```

`CoarseFundamental` objects have the following attributes:

The total number of stocks in the US Equity Security Master dataset is 30,000 but your coarse filter function won't receive all of these at one time because the US Equity Security Master dataset is free of survivorship bias and some of the securities have delisted over time. The number of securities that are passed into your coarse filter function depends on the date of your algorithm. Currently, there are about 10,000 securities that LEAN passes into your coarse filter function.

Fundamentals Selection

A fundamental universe lets you select stocks based on corporate fundamental data. This data is powered by Morningstar® and includes approximately 8,100 tickers with 900 properties each. This universe doesn't include ETFs, ADRs, or OTC equities. Due to the sheer volume of information, fundamental selection is performed on the output of another universe filter. Think of this process as a 2-stage filter. An initial filter function selects a set of stocks and then a fine fundamental filter function selects a subset of those stocks.

QuantConnect Coarse and Fine Universe Selection

To add a fundamental universe, in the `initialize` method, pass two filter functions to the `add_universe` method. The first filter function can be a coarse universe filter , dollar volume filter , or an ETF constituents filter . The second filter function receives a list of `FineFundamental` objects and must return a list of `Symbol` objects. The list of `FineFundamental` objects contains a subset of the `Symbol` objects that the first filter function returned. The `Symbol` objects you return from the second function are the constituents of the fundamental universe and LEAN automatically creates

subscriptions for them. Don't call `add_equity` in the filter function.

> **Tip:** Only 8,100 assets have fundamental data. If your first filter function receives `CoarseFundamental` data, you should only select assets that have a true value for their `has_fundamental_data` property.

```
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self.add_universe(self._coarse_filter_function, self._fine_fundamental_function)

    def _coarse_filter_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        # In addition to further coarse universe selection, ensure the security has fundamental data
        return [c.symbol for c in coarse if c.has_fundamental_data]

    def _fine_fundamental_function(self, fine: List[FineFundamental]) -> List[Symbol]:
        # Return a list of Symbols
        return []
```

`FineFundamental` objects have the following attributes:

Example

The simplest example of accessing the fundamental object would be harnessing the iconic PE ratio for a stock. This is a ratio of the price it commands to the earnings of a stock. The lower the PE ratio for a stock, the more affordable it appears.

```
# In Initialize:
self.universe_settings.asynchronous = True
self.add_universe(self._coarse_selection_function, self._fine_selection_function)

def _coarse_selection_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
    sorted_by_dollar_volume = sorted(coarse, key=lambda x: x.dollar_volume, reverse=True)
    filtered = [x.symbol for x in sorted_by_dollar_volume if x.has_fundamental_data]
    return filtered[:50]

def _fine_selection_function(self, fine: List[FineFundamental]) -> List[Symbol]:
    sorted_by_pe_ratio = sorted(fine, key=lambda x: x.valuation_ratios.pe_ratio, reverse=False)
    return [x.symbol for x in sorted_by_pe_ratio[:10]]
```

Asset Categories

In addition to valuation ratios, the US Fundamental Data from Morningstar has many other data point attributes, including over 200 different categorization fields for each US stock. Morningstar groups these fields into sectors, industry groups, and industries.

Sectors are large super categories of data. To get the sector of a stock, use the `MorningstarSectorCode` property.

```
tech = [x for x in fine if x.asset_classification.morningstar_sector_code == MorningstarSectorCode.TECHNOLOGY]
```

Industry groups are clusters of related industries that tie together. To get the industry group of a stock, use the `MorningstarIndustryGroupCode` property.

```
ag = [x for x in fine if x.asset_classification.morningstar_industry_group_code == MorningstarIndustryGroupCode.AGRICULTURE]
```

Industries are the finest level of classification available. They are the individual industries according to the Morningstar classification system. To get the industry of a stock, use the `MorningstarIndustryCode` .

```
coal = [x for x in fine if x.asset_classification.morningstar_industry_code == MorningstarSectorCode.COAL]
```

Practical Limitations

Like coarse universes, fine universes allow you to select an unlimited universe of assets to analyze. Each asset in the universe consumes approximately 5MB of RAM, so you may quickly run out of memory if your universe filter selects many assets. If you backtest your algorithms in the

Algorithm Lab, familiarize yourself with the RAM capacity of your backtesting and live trading nodes . To keep your algorithm fast and efficient, only subscribe to the assets you need.

Live Trading Considerations

The live data for fundamental universe selection arrives at 6/7 AM Eastern Time (ET), so fundamental universe selection runs for live algorithms between 7 and 8 AM ET. This timing allows you to place trades before the market opens. Don't schedule anything for midnight because the universe selection data isn't ready yet.

Examples

The legacy fundamental universe is deprecated. For examples of the new fundamental universe API, see Examples .

Equity

Alternative Data Universes

Introduction

An alternative data universe lets you select a basket of Equities based on an alternative dataset that's linked to them. If you use an alternative data universe, you limit your universe to only the securities in the dataset, which avoids unnecessary subscriptions.

Supported Datasets

The following alternative datasets support universe selection:

- Brain Language Metrics on Company Filings
- Brain ML Stock Ranking
- Brain Sentiment Indicator
- Crypto Market Cap
- Upcoming Dividends
- Upcoming Earnings
- Upcoming IPOs
- Upcoming Splits
- CNBC Trading
- Corporate Lobbying
- Insider Trading
- US Congress Trading
- US Government Contracts
- WallStreetBets
- Corporate Buybacks

Examples

The following examples demonstrate some common alternative data universes.

Example 1: Brain Sentiment Universe

The following algorithm uses the Brain Sentiment Indicator dataset to create a universe of US Equities that have some article mentions and the most positive sentiment. It then forms a sentiment-weighted portfolio at the start of each month.

```py
class BrainSentimentUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add an Equity universe that updates at the start of each month.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self.add_universe(BrainSentimentIndicatorUniverse, self._select_assets)
        # Create a Scheduled Event to rebalance the portfolio at the start of each month.
        self.schedule.on(self.date_rules.month_start(spy), self.time_rules.at(9, 45), self._rebalance)

    def _select_assets(self, alt_data: List[BrainSentimentIndicatorUniverse]) -> List[Symbol]:
        # Select the assets with recent mentions and a sentiment score.
        alt_data = [x for x in alt_data if x.total_article_mentions_30_days and x.sentiment_30_days]
        # Select the 10 assets with the greatest sentiment score.
        selected = sorted(alt_data, key=lambda x: x.sentiment_30_days)[-10:]
        # Save the sentiment scores for rebalancing later.
        self._sentiment_by_symbol = {x.symbol: x.sentiment_30_days for x in selected}
        # Return the Symbol objects of the selected assets.
        return list(self._sentiment_by_symbol.keys())

    def _rebalance(self) -> None:
        # To avoid trading errors, filter out the assets that don't have a price yet.
        self._sentiment_by_symbol = {
            symbol: sentiment for symbol, sentiment in self._sentiment_by_symbol.items()
            if self.securities[symbol].price
        }
        # Form a sentiment-weighted portfolio.
        sentiment_sum = sum(list(self._sentiment_by_symbol.values()))
        self.set_holdings(
            [PortfolioTarget(symbol, sentiment/sentiment_sum) for symbol, sentiment in self._sentiment_by_symbol.items()],
            liquidate_existing_holdings=True
        )
```

Example 2: Insiders Trading Universe

Insiders have more information to evaluate the overall prospect of the company, so following their trades can be useful. The following algorithm uses the Insider Trading to create a universe of US Equities that insiders have recently purchased. It then invests equally into the companies with positive insider trades, which may provide extra confidence in their expected return, and hold for 3 months.

```py
class InsiderTradingUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Keep each security in the universe for a minimum of 30 days to
        # digest the insiders purchase sentiment.
        self.universe_settings.minimum_time_in_universe = timedelta(30)
        # Add the QuiverInsiderTrading dataset for insider trade
        # detection and filtering.
        self._universe = self.add_universe(
            QuiverInsiderTradingUniverse, self._select_assets
        )
        # Add a Scheduled Event to rebalance the portfolio each day.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def _select_assets(
            self, alt_data: List[QuiverInsiderTradingUniverse]) -> List[Symbol]:
        # Calculate the net dollar volume of insider trades for each
        # asset.
        dollar_volume_by_symbol = {}
        for data in alt_data:
            symbol = data.symbol
            if not data.price_per_share:
                continue
            if symbol not in dollar_volume_by_symbol:
                dollar_volume_by_symbol[symbol] = 0
            dollar_volume_by_symbol[symbol] += data.shares * data.price_per_share
        # Select the 10 assets with the largest dollar volume of insider
        # trades.
        return [
            symbol for symbol, _ in sorted(
                dollar_volume_by_symbol.items(), key=lambda kvp: kvp[1]
            )[-10:]
        ]

    def _rebalance(self):
        # Equally invest in insider buying companies to evenly
        # dissipate the capital risk.
        symbols = [s for s in self._universe.selected if self.securities[s].price]
        self.set_holdings(
            [PortfolioTarget(symbol, 1/len(symbols)) for symbol in symbols],
            True
        )
```

Example 3: Share Buyback Universe

The following algorithm uses the Corporate Buybacks dataset to create a universe of US Equities that have announced an upcoming share buyback program:

PY

```
class SmartInsiderIntentionUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe that selects assets at the start of each week.
        # The universe should select Equities that have announced a
        # material buyback plan, since they have confidence in their
        # future prospect and the reduction in supply can drive their
        # price up.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        self.add_universe(SmartInsiderIntentionUniverse, self._select_assets)
        # Add a Scheduled Event to rebalance the portfolio at the start
        # of each week. The weekly schedule allows time to capitalize
        # the market swings from the positive sentiment.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.week_start(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def _select_assets(
            self,
            alt_coarse: List[SmartInsiderIntentionUniverse]) -> List[Symbol]:
        self._buyback_size = {}
        selected = []
        for d in alt_coarse:
            # Select assets that have a material buyback size to have
            # strong confidence.
            if d.amount_value and d.amount_value > 10_000_000:
                # Record the buyback size in the dictionary.
                self._buyback_size[d.symbol] = d.amount_value
                selected.append(d.symbol)
        return selected

    def _rebalance(self) -> None:
        # Ignore illiquid assets that have no price yet.
        self._buyback_size = {
            symbol: amount
            for symbol, amount in self._buyback_size.items()
            if self.securities[symbol].price
        }
        # Get the sum of all buyback size so we can scale the portfolio
        # weights based on the buyback size.
        buyback_sum = sum(self._buyback_size.values())
        # Place the trades to rebalance the portfolio.
        self.set_holdings(
            [
                PortfolioTarget(symbol, size / buyback_sum)
                for symbol, size in self._buyback_size.items()
            ]
        )

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate assets when they leave the universe.
        for removed in changes.removed_securities:
            self.liquidate(removed.symbol)
            self._buyback_size.pop(removed.symbol, None)
```

Universes

Equity Options

Introduction

An Equity Options universe lets you select a basket of contracts for a single Option. LEAN models Option subscriptions as a universe of Option contracts.

Create Universes

To add a universe of Equity Option contracts, in the `initialize` method, call the `add_option` method. This method returns an `Option` object, which contains the canonical `symbol`. You can't trade with the canonical Option `symbol`, but save a reference to it so you can easily access the Option contracts in the OptionChain that LEAN passes to the `on_data` method.

```
class BasicOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.universe_settings.asynchronous = True
        option = self.add_option("SPY")
        option.set_filter(self._filter)
        self._symbol = option.symbol

    def _filter(self, universe):
        return universe.expiration(0, 7).delta(0.35, 0.75)

    def on_data(self, data):
        chain = data.option_chains.get(self._symbol)
        if chain:
            contract = sorted(chain, key=lambda x: (x.expiry, x.greeks.delta))[0]
```

PY

The following table describes the `add_option` method arguments:

Argument	Data Type	Description
`ticker`	`str`	The underlying Equity ticker. To view the supported underlying Equity tickers, see Supported Assets .
`resolution`	`Resolution`	The resolution of the market data. To view the supported resolutions, see Resolutions . The Equity resolution must be less than or equal to the Equity Option resolution. For example, if you set the Equity resolution to minute, then you must set the Equity Option resolution to minute, hour, or daily.
`market`	`str`	The underlying Equity market.
`fill_forward`	`bool`	If true, the current slice contains the last available data even if there is no data at the current time.
`leverage`	`float`	The leverage for this Equity.
`extended_market_hours`	`bool`	A flag that signals if LEAN should send data during pre- and post-market trading hours.

If you don't set any of these parameters, LEAN uses the `self.universe_settings` values. See Set Universe Settings for more information.

If you add an Equity Option universe but don't have a subscription to the underlying Equity, LEAN automatically subscribes to the underlying Equity with the following settings:

Setting	Value
Fill forward	Same as the Option universe
Leverage	0
Extended Market Hours	Same as the Option universe
Data Normalization	`DataNormalizationMode.RAW`

If you already have a subscription to the underlying Equity but it's not `RAW` data normalization, LEAN automatically changes it to `RAW` .

Filter by Investment Strategy

Options trading strategies consist of simultaneously buying or selling one or more Option contracts of the same underlying Equity, but with different features.

```py
# Example 1: Select a Straddle
option.set_filter(lambda option_filter_universe: option_filter_universe.straddle(30))

# Example 2: Select the contracts (including weeklys) that expire in the next 30 days that form an Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 30).iron_condor(30, 5, 10))

# Example 3: Select the 0DTE contracts that form an Strangle
option.set_filter(lambda option_filter_universe: option_filter_universe.expiration(0, 0).strangle(30, 5, -10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contracts for Option strategies:

`naked_call(min_days_till_expiry: int, strike_from_atm: float)` Selects a call contract to form Naked Call , Covered Call , or Protective Call Option strategies.
`naked_put(min_days_till_expiry: int, strike_from_atm: float)` Selects a put contract to form Naked Put , Covered Put , or Protective Put Option strategies.
`call_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two call contracts to form Bull Call Spread or Bear Call Spread Option strategies.
`put_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two put contracts to form Bull Put Spread or Bear Put Spread Option strategies.
`call_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two call contracts to form Long Call Calendar Spread or Short Call Calendar Spread Option strategies.
`put_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two put contracts to form Long Put Calendar Spread or Short Put Calendar Spread Option strategies.
`strangle(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Long Strangle or Short Strangle Option strategies.
`straddle(min_days_till_expiry: int)` Selects two contracts to form Long Straddle or Short Straddle Option strategies.

`protective_collar(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Protective Collar Option strategies.
`conversion(min_days_till_expiry: int, strike_from_atm: float)` Selects two contracts to form Conversion or Reverse Conversion Option strategies.
`call_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Call Butterfly or Short Call Butterfly Option strategies.
`put_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Put Butterfly or Short Put Butterfly Option strategies.
`iron_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Long Iron Butterfly or Short Iron Butterfly Option strategies.
`iron_condor(min_days_till_expiry: int, near_strike_spread: float, far_strike_spread: float)` Selects four contracts to form Long Iron Condor or Short Iron Condor Option strategies.
`box_spread(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Box Spread or Short Box Spread Option strategies.
`jelly_roll(strike_from_atm: float, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects four contracts to form Jelly Roll or Short Jelly Roll Option strategies.
`call_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Call Ladder or Bull Call Ladder Option strategies.
`put_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Put Ladder or Bull Put Ladder Option strategies.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

Filter by Implied Volatility and Greeks

Option price models compute the theoretical price of Option contracts, their implied volatility, and their Greek values. Theoretical prices can help you detect undervalued and overvalued contracts, implied volatility can provide you insight into the upcoming volatility of the underlying security, and Greek values can help you hedge your portfolio.

```py
# Example 1: Select the contracts with delta between 0.25 and 0.75
option.set_filter(lambda option_filter_universe: option_filter_universe.delta(0.25, 0.75))

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days with implied volatility below 20%
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 90))

# Example 3: Select the contracts (including weeklys) that expire in the next 30 days with implied volatility below 20% that forms an
Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 30).iron_condor(30, 5, 10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for a range of implied volatility, Greek values, and open interest:

`iv(min: float, max: float)`
`implied_volatility(min: float, max: float)`
 	
Selects a contract with implied volatility within the range you set.

`d(min: float, max: float)`
`delta(min: float, max: float)`
 	
Selects a contract with delta within the range you set.

`g(min: float, max: float)`
`gamma(min: float, max: float)`
 	
Selects a contract with gamma within the range you set.

`r(min: float, max: float)`
`rho(min: float, max: float)`
 	
Selects a contract with rho within the range you set.

`v(min: float, max: float)`
`vega(min: float, max: float)`
 	
Selects a contract with vega within the range you set.

`t(min: float, max: float)`
`theta(min: float, max: float)`
 	
Selects a contract with theta within the range you set.

`oi(min: int, max: int)`
`open_interest(min: float, max: float)`
 	
Selects a contract with open interest within the range you set.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

To perform thorough filtering on the `OptionFilterUniverse` , define an isolated filter method.

```
# Select the put contracts with the lowest strike price.
option.set_filter(self._contract_selector)

def _contract_selector(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
    return option_filter_universe \
        .delta(0.5, 1.5) \
        .gamma(0.0001, 0.0006) \
        .vega(0.01, 1.5) \
        .theta(-2.0, -0.5) \
        .rho(0.5, 3.0) \
        .implied_volatility(1, 3) \
        .open_interest(100,500)
```

Filter by Other Contract Properties

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Option` object.

```py
# Select contracts that have a strike price within 1 strike level above and below the underlying price
option.set_filter(min_strike=-1, max_strike=1)

# Select contracts that expire within 30 days
option.set_filter(min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select contracts that have a strike price within 1 strike level and expire within 30 days
option.set_filter(min_strike=-1, max_strike=1, min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only())
```

The following table describes the available filter techniques:

`set_filter(minStrike: int, maxStrike: int)` Selects the contracts that have a strike price within a minimum and maximum strike level relative to the underlying price. For example, say the underlying price is 302 and there are strikes at every 5. If you set `m_strike` to -1 and `max_strike` to 1, LEAN selects the contracts that have a strike of 300 or 305. This filter runs asynchronously by default.
`set_filter(minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(minStrike: int, maxStrike: int, minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire and have a strike within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[OptionFilterUniverse], OptionFilterUniverse])` Selects the contracts that a function selects.

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
# Example 1: Select the front month call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only().front_month())

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 90))
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Default Filter

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard and weeklys
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at the first bar of every day. When the filter selects a contract that isn't currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

Navigate Intraday Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , index the `option_chains` property of the `Slice` with the canonical `symbol` . After you get the `OptionChain` , you can sort and filter the Option contracts in the chain.

```
PY
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._symbol)
    if chain:
        # Example: Find 5 put contracts that are closest to at-the-money (ATM) and have the farthest expiration
        contracts = [x for x in chain if x.right == OptionRight.PUT]
        contracts = sorted(sorted(contracts, \
            key = lambda x: abs(chain.underlying.price - x.strike)), \
            key = lambda x: x.expiry, reverse=True)[:5]

        # Select the contract with the delta closest to -0.5
        contract = sorted(contracts, key=lambda x: abs(-0.5 - x.greeks.delta))[0]
```

You can also loop through the `option_chains` property to get each `OptionChain` .

```
PY
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-OptionChain key-value pairs
    for canonical_symbol, chain in slice.option_chains.items():
        contracts = chain.contracts
```

`OptionChain` objects have the following properties:

Navigate Daily Option Chains

To get the daily, pre-calculated Greeks and implied volaility of all the currently tradable contracts, call the `option_chain` method. This method returns a `DataHistory[OptionUniverse]` object, which you can format into a DataFrame or iterate through. Each row in the DataFrame and each `OptionUniverse` object represents a single contract. The data this method returns contains information on all the currently tradable contracts, not just the contracts that pass your filter.

```
PY
# In the initialize method, create a Scheduled Event to get the Option
# chain and rebalance the portfolio.
self.schedule.on(
    self.date_rules.week_start(self._symbol),
    self.time_rules.after_market_open(self._symbol, 1),
    self._rebalance
  )

# Define the method.
def _rebalance(self):
    daily_option_chain = self.option_chain(self._symbol, flatten=True)

    # Get the DataFrame format.
    df = daily_option_chain.data_frame
    delta_by_symbol = df.delta

    # Get the OptionUniverse objects format.
    for option_universe in daily_option_chain:
        close = option_universe.close
        oi = option_universe.open_interest
        delta = option_universe.greeks.delta
```

The `option_chain` method was previously known as the `option_chain_provider.get_option_contract_list` method.

`OptionUniverse` objects have the following properties:

Greeks and Implied Volatility

There are several ways to get the implied volatility and Greeks of contracts.

The Greeks and IV values in the filter function are the daily, pre-calculated values based on the end of the previous trading day.

```
def initialize(self):
    option = self.add_option("SPY")
    option.set_filter(lambda universe: universe.delta(0.3, 0.7).expiration(0,7))
```

To calculate the values, we use our implementation of the forward tree pricing model, which accounts for the interest rate and dividend payments.

You can't customize the Greeks and IV values that the filter function receives. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

To override the default pricing model of the Option, set a pricing model . If you set a price model, it sets the price model of the individual contracts in the universe.

```
// Set price_model field to use the Binomial Cox-Ross-Rubinstein model to price the Options.
option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

Historical Data

To get historical chains for an Equity Option, call the `history[OptionUniverse]` method with the canonical Option `Symbol` . This method returns the entire Option chain for each trading day, not the subset of contracts that pass your universe filter.

```
class OptionHistoryAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_option('SPY')
        # DataFrame example:
        history_df = self.history(option.symbol, 5, flatten=True)
        # OptionUniverse objects example:
        history = self.history[OptionUniverse](option.symbol, 5)
        for chain in history:
            end_time = chain.end_time
            filtered_contracts = [c for c in chain if c.greeks.delta > 0.3]
            for contract in filtered_contracts:
                symbol = contract.symbol
                expiry = contract.id.date
                strike = contract.id.strike_price
                price = contract.close
                iv = contract.implied_volatility
```

The Greeks and IV values that you get from a history request of the Option universe are the daily, pre-calculated values based on the end of the previous trading day. To get the intraday values or to customize the Greeks and IV calculations, create some Option indicators .

The method represents each contract with an `OptionUniverse` object, which have the following properties:

Selection Frequency

By default, Equity Option universes run at the first time step of each day to select their contracts.

Examples

The following examples demonstrate some common Equity Option universes.

Example 1: 0DTE Contracts

0DTE Options are Option contracts that expire on the same day you trade them. The following algorithm selects 0DTE Option contracts for the SPY that fall within 3 strikes of the underlying price.

```py
class ZeroDTEOptionUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_option('SPY')
        option.set_filter(lambda u: u.expiration(0, 0).strikes(-3, 3))
```

For trading full examples of Equity Option Universe, please refer to the Securities > Equity Options > Requesting Data > Universes page.

Universes

Equity CFD

Introduction

An Equity CFD universe lets you select a set of Equity CFD assets. To select the underlying Equities, you can use a fundamental universe to select assets based on corporate fundamental data, an ETF constituents universe to select the CFD versions of stocks in an exchange-traded fund, or an alternative data universe to select assets based on alternative data. This feature is powerful for European clients seeking to trade US ETF products.

Only the Interactive Brokers CFD integration supports trading Stock-CFD products. QuantConnect doesn't have historical data for Interactive Brokers CFD products; however, you can use the `live_mode` flag to swap to the CFD equivalents for live trading. In live trading, you must include Interactive Brokers as a data provider to trade Equity-CFD products.

Create Universes Using Fundamentals

To add a fundamental universe, in the `initialize` method, pass a filter function to the `add_universe` method. The filter function receives a list of `Fundamental` objects and must return a list of `Symbol` objects. The `Symbol` objects you return from the function are the constituents of the fundamental universe and LEAN automatically creates subscriptions for them. In live mode, call the `Symbol.create` method to swap for a CFD version of the same `Symbol` . Don't call `add_cfd` in the filter function.

```
PY
# Create a universe for CFD securities based on fundamental data.
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        # Pass the filter function as an argument to the AddUniverse method.
        self._universe = self.add_universe(self._fundamental_function)

    def _fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        symbols = [c.symbol for c in fundamental if c.has_fundamental_data]
        if self.live_mode:
            return [Symbol.create(x.value, SecurityType.CFD, Market.INTERACTIVE_BROKERS) for x in symbols]
        return symbols
```

Example

The simplest example of accessing the fundamental object would be harnessing the iconic PE ratio for a stock. This is a ratio of the price it commands to the earnings of a stock. The lower the PE ratio for a stock, the more affordable it appears.

```
PY
# Select the top 50 by dollar volume and then select the top 10 by their PE ratio.
self.universe_settings.asynchronous = True
self._universe = self.add_universe(self._fundamental_selection_function)

def _fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
    filtered = [f for f in fundamental if f.price > 10 and f.has_fundamental_data and not np.isnan(f.valuation_ratios.pe_ratio)]
    sorted_by_dollar_volume = sorted(filtered, key=lambda f: f.dollar_volume, reverse=True)[:100]
    sorted_by_pe_ratio = sorted(sorted_by_dollar_volume, key=lambda f: f.valuation_ratios.pe_ratio, reverse=False)[:10]
    symbols = [f.symbol for f in sorted_by_pe_ratio]
    if self.live_mode:
        return [Symbol.create(x.value, SecurityType.CFD, Market.INTERACTIVE_BROKERS) for x in symbols]
    return symbols
```

Practical Limitations

Fundamental universes allow you to select an unlimited universe of assets to analyze. Each asset in the universe consumes approximately 5MB of RAM, so you may quickly run out of memory if your universe filter selects many assets. If you backtest your algorithms in the Algorithm Lab, familiarize yourself with the RAM capacity of your backtesting and live trading nodes . To keep your algorithm fast and efficient, only subscribe to the assets you need.

Create Universes using ETF Constituents

To add an ETF Constituents universe, in the `initialize` method, call the `universe.etf` method with a filter function, and pass it to the `add_universe` method. The filter function receives a list of `ETFConstituentUniverse` objects and must return a list of `Symbol` objects. The `Symbol` objects you return

from the function are the constituents of the fundamental universe and LEAN automatically creates subscriptions for them. In live mode, call the `Symbol.create` method to swap for a CFD version of the same `Symbol` . Don't call `add_cfd` in the filter function.

```
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self._universe = self.universe.etf("SPY", self.universe_settings, self._etf_constituents_filter)
        self.add_universe(self._universe)

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Get the 10 securities with the largest weight in the index
        selected = sorted(constituents, key=lambda c: c.weight or 0.0, reverse=True)[:10]
        symbols = [c.symbol for c in selected]
        if self.live_mode:
            return [Symbol.create(x.value, SecurityType.CFD, Market.INTERACTIVE_BROKERS) for x in symbols]
        return symbols
```

PY

Selection Frequency

Equity universes run on a daily basis by default. To adjust the selection schedule, see Schedule .

Live Trading Considerations

The live data for fundamental universe selection arrives at 6/7 AM Eastern Time (ET), so fundamental universe selection runs for live algorithms between 7 and 8 AM ET. This timing allows you to place trades before the market opens. Don't schedule anything for midnight because the universe selection data isn't ready yet.

Examples

The following examples demonstrate common practices for implementing Equity CFD's universe.

Example 1: Short Bottom-Weighted SPY Constituents

A subset of the SPY constituents outperforms the SPY, while many underperform the overall index. To exclude the ETF constituents that underperform the index, the following algorithm buys the SPY CFD and shorts the CFD of the 100 constituents in the index with the smallest weight in the ETF.

PY

```
class EquityCfdUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10_000_000)
        self.settings.minimum_order_margin_portfolio_percentage = 0
        # Request SPY to trade. In live mode, request the IB CFD
        self._spy = self.add_cfd("SPY", market= Market.InteractiveBrokers).symbol if self.live_mode else self.add_equity("SPY").symbol
        # The SPY ETF provides market hours for scheduled events.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        # Refilter and rebalance weekly to avoid over-trading and high transaction costs.
        self.universe_settings.schedule.on(self.date_rules.week_end(spy))
        # Add a universe to select the CFD contracts of the SPY constituents.
        self._universe = self.add_universe(self.universe.etf(spy, universe_filter_func=self._select_assets))

        # Create a scheduled event to rebalance the portfolio.
        self.schedule.on(
            self.date_rules.week_start(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def _select_assets(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Cache the constituent weights in a dictionary for filtering and position sizing.
        self._etf_weight_by_symbol = {c.symbol: c.weight for c in constituents if c.weight}
        # Select the CFD contracts of the bottom-weighted 100 constituents.
        # They should have negative excess returns.
        def get_symbol(symbol):
            if self.live_mode:
                return Symbol.create(symbol.value, SecurityType.CFD, Market.INTERACTIVE_BROKERS)
            return symbol
        return [
            get_symbol(symbol) for symbol, _ in sorted(
                self._etf_weight_by_symbol.items(),
                key=lambda x: x[1]
            )[:100]
        ]

    def _rebalance(self) -> None:
        # Get the ETF weight of all the assets currently in the universe.
        weight_by_symbol = { symbol: self._etf_weight_by_symbol[symbol] for symbol in self._universe.selected }
        # Sell the CFD of the 100 ETF constituents with the lowest weight due to expected negative excess return.
        targets = [PortfolioTarget(symbol, -weight) for symbol, weight in weight_by_symbol.items()]
        # Buy the SPY CFD to eliminate systematic risk.
        targets.append(PortfolioTarget(self._spy, 1 - sum(weight_by_symbol.values())))
        # Place orders to rebalance the portfolio.
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Universes

Futures

Introduction

A Futures universe lets you select a basket of contracts for a single Future. LEAN models Future subscriptions as a universe of Future contracts. A Future universe is similar to an Option universe , except Future contracts don't have a strike price, so the universe filter primarily focuses on the contract expiration date.

Create Universes

To add a universe of Future contracts, in the `initialize` method, call the `add_future` method. This method returns an `Future` object, which contains the continuous contract `symbol` . The continuous contract `symbol` is the key to access the contracts in the FutureChain that LEAN passes to the `on_data` method. The continuous contract `mapped` property references the current contract in the continuous contract series. When you create the Future subscription, save a reference to the `Future` object so you can use it later in your algorithm.

```
class BasicFutureAlgorithm(QCAlgorithm):
    def initialize(self):
        self.universe_settings.asynchronous = True
        self._future = self.add_future(Futures.Currencies.BTC,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.LAST_TRADING_DAY,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        self._future.set_filter(0,62)

    def on_data(self, data):
        if self.portfolio.invested:
            return
        continuous_trade_bar = data.bars.get(self._future.symbol)
        mapped_trade_bar = data.bars.get(self._future.mapped)
        self.market_order(self._future.mapped, 1)

    # Track events when security changes its ticker allowing algorithm to adapt to these changes.
    def on_symbol_changed_events(self, symbol_changed_events):
        for symbol, changed_event in  symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = changed_event.new_symbol
            quantity = self.portfolio[old_symbol].quantity

            # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
            tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
            self.liquidate(old_symbol, tag=tag)
            if quantity: self.market_order(new_symbol, quantity, tag=tag)
```

The following table describes the `add_future` method arguments:

Argument: `ticker` The Future ticker. To view the supported assets in the US Futures dataset, see Supported Assets . Data Type: `str`
Argument: `resolution` The resolution of the market data. To view the supported resolutions, see Resolutions . If you don't provide a value, it uses `Resolution.MINUTE` by default. Data Type: `Resolution/NoneType`

Argument: `market` The Futures market. To view the supported markets in the US Futures dataset, see Supported Markets . If you don't provide a value, it uses the default Future market of your brokerage model . Data Type: `str`
Argument: `fill_forward` If true, the current slice contains the last available data even if there is no data at the current time. Data Type: `bool`
Argument: `leverage` The leverage for this Future. Data Type: `float`
Argument: `extended_market_hours` If true, use data from the pre and post market sessions Data Type: `bool`
Argument: `data_mapping_mode` The contract mapping mode to use for the continuous future contract Data Type: `DataMappingMode/NoneType`
Argument: `data_normalization_mode` The price scaling mode to use for the continuous future contract Data Type: `DataNormalizationMode/NoneType`
Argument: `contract_depth_offset` The continuous future contract desired offset from the current front month. For example, 0 is the front month, 1 is the back month contract. Data Type: `int`

If you don't set any of these parameters, LEAN uses the `self.universe_settings` values. See Set Universe Settings for more information.

Continous Contracts

By default, LEAN only subscribes to the continuous Future contract. A continuous Future contract represents a series of separate contracts stitched together to form a continuous price. If you need a lot of historical data to warm up an indicator, apply the indicator to the continuous contract price series. The `Future` object has a `symbol` property and a `mapped` property. The price of the `symbol` property is the adjusted price of the continuous contract. The price of the `mapped` property is the raw price of the currently selected contract in the continuous contract series.

```
# Get the adjusted price of the continuous contract.
adjusted_price = self.securities[self._future.symbol].price

# Get the raw price of the currently selected contract in the continuous contract series.
raw_price = self.securities[self._future.mapped].price
```

The continuous Futures contract isn't a tradable security . You must place orders for a specific Futures contract. To access the currently selected contract in the continuous contract series, use the `mapped` property of the `Future` object.

```
# Place a market order for the currently selected contract in the continuous contract series.
self.market_order(self._future.mapped, 1)
```

To configure how LEAN identifies the current Future contract in the continuous series and how it forms the adjusted price between each contract, provide `data_mapping_mode` , `data_normalization_mode` , and `contract_depth_offset` arguments to the `add_future` method. The `Future` object that the `add_future` method returns contains a `mapped` property that references the current contract in the continuous contract series. As the contracts roll over, the `mapped` property references the next contract in the series and you receive a `SymbolChangedEvent` object in the `on_data` method. The `SymbolChangedEvent` references the old contract `Symbol` and the new contract `Symbol` . You can use `SymbolChangedEvents` to roll over contracts.

```
class BasicFutureAlgorithm(QCAlgorithm):
    # Track when the continuous contract switches from one contract to the next.
    def on_symbol_changed_events(self, symbol_changed_events):
        for symbol, changed_event in  symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = changed_event.new_symbol
            quantity = self.portfolio[old_symbol].quantity

            # Rolling over: To liquidate the old mapped contract and switch to the new mapped contract.
            tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
            self.liquidate(old_symbol, tag=tag)
            if quantity: self.market_order(new_symbol, quantity, tag=tag)
```

In backtesting, the `SymbolChangedEvent` occurs at midnight Eastern Time (ET). In live trading, the live data for continuous contract mapping arrives at 6/7 AM ET, so that's when it occurs.

Data Normalization Modes

The `data_normalization_mode` argument defines how the price series of two contracts are stitched together when the contract rollovers occur. The following `DataNormalizatoinMode` enumeration members are available for continuous contracts:

We use the entire Futures history to adjust historical prices. This process ensures you get the same adjusted prices, regardless of the backtest end date.

Data Mapping Modes

The `data_mapping_mode` argument defines when contract rollovers occur. The `DataMappingMode` enumeration has the following members:

`Futures.Indices.VIX` (VX) doesn't support continous contract rolling with `DataMappingMode.OPEN_INTEREST` and `DataMappingMode.OPEN_INTEREST_ANNUAL` .

Contract Depth Offsets

The `contract_depth_offset` argument defines which contract to use. 0 is the front month contract, 1 is the following back month contract, and 2 is the second back month contract.

Filter Contracts

By default, LEAN doesn't add any contracts to the FuturesChain it passes to the `on_data` method.

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Future` object. The following table describes the available filter techniques:

`set_filter(minExpiry: int, maxExpiry: int)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[FutureFilterUniverse], FutureFilterUniverse])` Selects the contracts that a function selects.

```py
# Select the contracts that expire within 182 days.
self._future.set_filter(0, 182)

# Select the front month contract.
self._future.set_filter(lambda future_filter_universe: future_filter_universe.front_month())
```

The following table describes the filter methods of the `FutureFilterUniverse` class:

`standards_only()` Selects standard contracts
`front_month()` Selects the front month contract
`back_months()` Selects the non-front month contracts
`back_month()` Selects the back month contracts
`expiration(min_expiry: timedelta, max_expiry: timedelta)` Selects contracts that expire within a range of dates relative to the current day
`expiration(min_expiry_days: int, max_expiry_days: int)` Selects contracts that expire within a range of dates relative to the current day
`contracts(contracts: list[Symbol])` Selects a list of contracts
`contracts(contractSelector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects

The preceding methods return an `FutureFilterUniverse` , so you can chain the methods together.

```py
# Select the front month standard contracts
self._future.set_filter(lambda future_filter_universe: future_filter_universe.standards_only().front_month())
```

You can also define an isolated filter method.

```py
# In Initialize
self._future.set_filter(self._contract_selector)

def _contract_selector(self,
    future_filter_universe: Callable[[FutureFilterUniverse], FutureFilterUniverse]) -> FutureFilterUniverse:
    return future_filter_universe.standards_only().front_month()
```

Navigate Futures Chains

`FuturesChain` objects represent an entire chain of contracts for a single underlying Future. They have the following properties:

To get the `FuturesChain` , index the `futures_chains` property of the `Slice` with the continuous contract `Symbol` .

```py
def on_data(self, slice: Slice) -> None:
    chain = slice.futures_chains.get(self._symbol)
    if chain:
        # Example: Select the contract with the greatest open interest
        contract = sorted(chain, key=lambda contract: contract.open_interest, reverse=True)[0]
```

You can also loop through the `futures_chains` property to get each `FuturesChain` .

```py
def on_data(self, slice: Slice) -> None:
    for continuous_contract_symbol, chain in slice.futures_chains.items():
        pass
```

Historical Data

To get historical Future universe data, call the `history` method. This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. The return type depends on how you call the method.

```py
class FutureHistoryAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        # DataFrame example:
        history_df = self.history(FutureUniverse, future.symbol, 5, flatten=True)
        # Series example:
        history_series = self.history(FutureUniverse, future.symbol, 5, flatten=False)
        # FutureUniverse objects example:
        history = self.history[FutureUniverse](future.symbol, 5)
        for universe in history:
            end_time = universe.end_time
            for contract in universe:
                symbol = contract.symbol
                expiry = contract.id.date
                price = contract.close
                oi = contract.open_interest
```

The method represents each contract with a `FutureUniverse` object, which have the following properties:

Selection Frequency

By default, Futures universes run at the first time step of each day to select their contracts.

Examples

The following examples demonstrate some common Futures universes.

Example 1: Contracts Expiring On Quarter End

To reduce the need for rolling over, you can filter Future contracts that expire only on quarter ends to save transaction costs. The following example trades a simple EMA cross strategy on SP500 EMini Future contracts that expire on quarter ends.

```py
class QuarterlyFuturesAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)

        # Request ES Future data for trading.
        self.future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
        )
        # We only filter the ones expiring on quarter ends to reduce the transaction cost needed for rolling over contracts.
        self.future.set_filter(lambda future_filter_universe: future_filter_universe.expiration_cycle([3, 6, 9, 12]))

        # Create an EMA indicator to track trends for trade signal generation.
        self._ema = self.ema(self.future.symbol, 20, Resolution.DAILY)
        # Warm up the indicator for immediate readiness.
        self.warm_up_indicator(self.future.symbol, self._ema, Resolution.DAILY)

        # A variable to control the algorithm trading in daily frequency only, since the indicator signal is by daily.
        self._day = -1

    def on_data(self, slice: Slice) -> None:
        # Get the chain to trade from the filtered contracts.
        chain = slice.future_chains.get(self.future.symbol)
        if self._day != slice.time.day and chain:
            # Select the contract with the nearest expiry.
            contract = min(chain, key=lambda x: x.expiry)

            # Trade an EMA cross strategy to follow the trend.
            quantity = self.portfolio[self.future.symbol].quantity
            if contract.last_price > self._ema.current.value and quantity <= 0:
                self.market_order(contract.symbol, 1-quantity)
            elif contract.last_price < self._ema.current.value and quantity >= 0:
                self.market_order(contract.symbol, -1-quantity)

            self._day = slice.time.day
```

Example 2: Contracts Expiring Within One Week

The trading volume of Futures reaches its maximum during the last week of the trading period. The following algorithm selects contracts that expire within a week:

PY

```
class WeeklyFuturesAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add ES Futures for trading. The continuous contract data is
        # only used for indicator values.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST
        )
        # Select the contract expiring within a week since they have
        # the highest liquidity.
        self._future.set_filter(lambda universe: universe.expiration(0, 7))
        # Create a Bollinger Band indicator to identify extreme prices
        # for trade signal generation.
        self._bb = self.bb(self._future.symbol, 60, 2)
        # Warm up the indicator for so we can trade right away.
        self.warm_up_indicator(self._future.symbol, self._bb)

    def on_data(self, slice: Slice) -> None:
        # Get the current Futures chain, if there is one.
        chain = slice.future_chains.get(self._future.symbol)
        if not chain:
            return
        # Select the contract with the nearest expiry.
        contract = min(chain, key=lambda x: x.expiry)
        # Trade a mean-reversal strategy based on Bollinger Band signal.
        holding = self.portfolio[contract.symbol]
        price = contract.last_price
        # Case 1: Short
        if not holding.is_short and price > self._bb.upper_band.current.value:
            target_quantity = -1
        # Case 2: Long
        elif not holding.is_long and price < self._bb.lower_band.current.value:
            target_quantity = 1
        # Case 3: Exit
        elif ((holding.is_short and price < self._bb.middle_band.current.value) or
            (holding.is_long and price > self._bb.middle_band.current.value)):
            target_quantity = 0
        # Case 4: Hold
        else:
            return
        self.market_order(contract.symbol, target_quantity - holding.quantity)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
EmaCrossFuturesFrontMonthAlgorithm.py Python

Universes

Future Options

Introduction

A Future Option universe lets you select a basket of Option contracts on the contracts in a Futures universe .

Create Universes

To add a universe of Future Option contracts, in the `initialize` method, define a Future universe and then pass the canonical `symbol` to the `add_future_option` method.

```
self.universe_settings.asynchronous = True
future = self.add_future(Futures.Metals.GOLD)
future.set_filter(0, 90)
self.add_future_option(future.symbol)
```

The following table describes the `add_future_option` method arguments:

Argument	Data Type	Description
`symbol`	`Symbol`	The continuous Future contract Symbol. To view the supported assets in the US Future Options dataset, see Supported Assets .
`option_filter`	`Callable[[OptionFilterUniverse], OptionFilterUniverse]`	A function that selects Future Option contracts

The subscription characteristics such as resolution, market, and fill forward behavior are inherited from the `self.universe_settings` values. See Set Universe Settings for more information.

To override the default pricing model of the Option, set a pricing model in a security initializer.

```
# In Initialize
seeder = SecuritySeeder.NULL
self.set_security_initializer(MySecurityInitializer(self.brokerage_model, seeder, self))

# Outside of the algorithm class
class MySecurityInitializer(BrokerageModelSecurityInitializer):

    def __init__(self, brokerage_model: IBrokerageModel, security_seeder: ISecuritySeeder, QCAlgorithm algorithm) -> None:
        super().__init__(brokerage_model, security_seeder)
        self._algorithm = algorithm;

    def initialize(self, security: Security) -> None:
        # First, call the superclass definition
        # This method sets the reality models of each security using the default reality models of the brokerage model
        super().initialize(security)

        # Next, set the price model
        if security.type == SecurityType.FUTURE_OPTION: # Option type
            security.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

Filter Contracts

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard type (weeklies and non-standard contracts are not available)
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at every time step in your algorithm. When the filter selects a contract that isn't

currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

To set a contract filter, in the `initialize` method, pass a filter function to the `add_future_option` method. The following table describes the available filter techniques:

```
self.add_future_option(future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-1, 1))
```

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts. This filter doesn't currently work. To add this functionality to our roadmap , react to this GitHub Issue with a thumbs up emoji.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
self.add_future_option(future.symbol, lambda option_filter_universe: option_filter_universe.strikes(-1, 1).calls_only())
```

To perform thorough filtering on the `OptionFilterUniverse` , define an isolated filter method.

```
# In Initialize
self.add_future_option(future.Symbol, self._contract_selector)

def _contract_selector(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
    symbols = option_filter_universe.PutsOnly()
    strike = min([symbol.id.strike_price for symbol in symbols])
    symbols = [symbol for symbol in symbols if symbol.id.strike_price == strike]
    return option_filter_universe.contracts(symbols)
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Navigate Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , loop through the `option_chains` property. After you get the `OptionChain` , you can sort and filter the Option contracts in the chain.

```
def on_data(self, slice: Slice) -> None:
    for _, option_chain in slice.option_chains.items():
        # Example: Find 5 put contracts that are closest to at-the-money (ATM) and have the farthest expiration
        contracts = [x for x in option_chain if x.right == OptionRight.PUT]
        contracts = sorted(sorted(contracts, \
            key=lambda x: abs(option_chain.underlying.price - x.strike)), \
            key=lambda x: x.expiry, reverse=True)[:5]

        # Select the contract with the delta closest to -0.5
        contract = sorted(contracts, key=lambda x: abs(-0.5 - x.greeks.delta))[0]
```

You can also iterate through the `futures_chains` first.

```
def on_data(self, slice: Slice) -> None:
    for continuous_future_symbol, futures_chain in slice.futures_chains.items():
        # Select a Future Contract and create its canonical FOP Symbol
        futures_contract = [contract for contract in futures_chain][0]
        canonical_fop_symbol = Symbol.create_canonical_option(futures_contract.symbol)
        fop_chain = slice.option_chains.get(canonical_fop_symbol)
        if fop_chain:
            for contract in fop_chain:
                pass
```

`OptionChain` objects have the following properties:

Historical Data

To get historical Future Option universe data, call the `history` method with the canoncial Option `Symbol` . This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. The return type depends on how you call the method.

```py
class FutureOptionsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Select a Futures contract.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        futures_contract = list(self.futures_chain(future.symbol))[0]
        # Create the canoncial Option Symbol for this Futures contract.
        canonical = Symbol.create_canonical_option(futures_contract.symbol)
        # Get the historical Option universe data for this Futures contract.
        # DataFrame example:
        history_df = self.history(OptionUniverse, canonical, 3, flatten=True)
        # Series example:
        history_series = self.history(OptionUniverse, canonical, 3, flatten=False)
        # OptionUniverse example:
        history = self.history[OptionUniverse](canonical, 3)
        for universe in history:
            end_time = universe.end_time
            for contract in universe:
                symbol = contract.symbol
                expiry = contract.id.date
                price = contract.close
```

The method represents each contract with a `OptionUniverse` object, which have the following properties:

Selection Frequency

By default, Future Option universes run at the first time step of each day to select their contracts.

Examples

The following examples demonstrate some common Future Option universes.

Example 1: 0DTE Contracts

0DTE Options are Option contracts that expire on the same day you trade them. The following algorithm selects 0DTE Future Option contracts for the E-mini S&P 500 that fall within 3 strikes of the underlying price.

```py
class ZeroDTEFutureOptionUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(0, 90)
        self.add_future_option(
            future.symbol,
            lambda u: u.expiration(0, 0).strikes(-3, 3)
        )
```

For full trading examples of Future Options, see Examples .

Universes

Index Options

Introduction

An Index Option universe lets you select a basket of Option contracts on an Index.

Create Universes

To add a universe of Index Option contracts, in the `initialize` method, call the `add_index_option` method. This method returns an `Option` object, which contains the canonical `symbol`. You can't trade with the canonical Option `symbol`, but save a reference to it so you can easily access the Option contracts in the OptionChain that LEAN passes to the `on_data` method. The method to create the universe depends on if the Index Options you want require a target ticker.

Create Non-Standard Universes

Non-standard Option contracts are popular among Option traders. To create a universe of non-standard Index Options like weekly VIX contracts, pass the index Symbol and target Option ticker to the `add_index_option` method.

```
class WeeklyIndexOptionAlgorithm(QCAlgorithm):

    def initialize(self):
        underlying = self.add_index("VIX").symbol
        option = self.add_index_option(underlying, "VIXW")
        option.set_filter(lambda universe: universe.expiration(0, 7).delta(0.35, 0.75))
        self._symbol = option.symbol

    def on_data(self, data):
        chain = data.option_chains.get(self._symbol)
        if chain:
            contract = sorted(chain, key=lambda x: (x.expiry, x.greeks.delta))[0]
```

The following table describes the `add_index_option` method arguments for non-standard universes:

Argument	Data Type	Description
`underlying`	`Symbol`	The underlying Index `Symbol`. To view the supported indices, see Supported Assets.
`target_option`	`str`	The target Option ticker. To view the supported target Options, see Supported Assets.
`resolution`	`Resolution`	The resolution of the market data. To view the supported resolutions, see Resolutions. The Index resolution must be less than or equal to the Index Option resolution. For example, if you set the Index resolution to minute, then you must set the Index Option resolution to minute, hour, or daily.
`market`	`str`	The Index Option market.
`fill_forward`	`bool`	If true, the current slice contains the last available data even if there is no data at the current time.

If you don't set any of these parameters, LEAN uses the `self.universe_settings` values. See Set Universe Settings for more information.

If you add an Option universe for an underlying Index that you don't have a subscription for, LEAN automatically subscribes to the underlying Index.

Create Standard Universes

To create a universe of Index Options based on an index like VIX, SPX, or NDX, pass the index ticker to the `add_index_option` method.

```
option = self.add_index_option("SPX")
option.set_filter(lambda universe: universe.expiration(0, 60).delta(0.35, 0.75))
self._symbol = option.symbol
```

The following table describes the `add_index_option` method arguments for standard universes:

Argument	Data Type	Description
`ticker`	`str`	The underlying Index ticker. To view the supported indices, see Supported Assets .
`resolution`	`Resolution`	The resolution of the market data. To view the supported resolutions, see Resolutions .
`market`	`str`	The Index Option market.
`fill_forward`	`bool`	If true, the current slice contains the last available data even if there is no data at the current time.

If you don't set any of these parameters, LEAN uses the `self.universe_settings` values. See Set Universe Settings for more information.

If you add an Option universe for an underlying Index that you don't have a subscription for, LEAN automatically subscribes to the underlying Index and sets its fill forward property to match that of the Index Option universe.

Filter by Investment Strategy

Options trading strategies consist of simultaneously buying or selling one or more Option contracts on the same underlying Index, but with different features.

```
# Example 1: Select a Straddle
option.set_filter(lambda option_filter_universe: option_filter_universe.straddle(30))

# Example 2: Select the contracts (including weeklys) that expire in the next 30 days that form an Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 30).iron_condor(30, 5, 10))

# Example 3: Select the 0DTE contracts that form an Strangle
option.set_filter(lambda option_filter_universe: option_filter_universe.expiration(0, 0).strangle(30, 5, -10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contracts for Option strategies:

`naked_call(min_days_till_expiry: int, strike_from_atm: float)` Selects a call contract to form Naked Call , Covered Call , or Protective Call Option strategies.
`naked_put(min_days_till_expiry: int, strike_from_atm: float)` Selects a put contract to form Naked Put , Covered Put , or Protective Put Option strategies.
`call_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two call contracts to form Bull Call Spread or Bear Call Spread Option strategies.
`put_spread(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two put contracts to form Bull Put Spread or Bear Put Spread Option strategies.
`call_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two call contracts to form Long Call Calendar Spread or Short Call Calendar Spread Option strategies.

`put_calendar_spread(strike_from_atm: int, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects two put contracts to form Long Put Calendar Spread or Short Put Calendar Spread Option strategies.
`strangle(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Long Strangle or Short Strangle Option strategies.
`straddle(min_days_till_expiry: int)` Selects two contracts to form Long Straddle or Short Straddle Option strategies.
`protective_collar(min_days_till_expiry: int, higher_strike_from_atm: float, lower_strike_from_atm: float)` Selects two contracts to form Protective Collar Option strategies.
`conversion(min_days_till_expiry: int, strike_from_atm: float)` Selects two contracts to form Conversion or Reverse Conversion Option strategies.
`call_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Call Butterfly or Short Call Butterfly Option strategies.
`put_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects three contracts to form Long Put Butterfly or Short Put Butterfly Option strategies.
`iron_butterfly(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Long Iron Butterfly or Short Iron Butterfly Option strategies.
`iron_condor(min_days_till_expiry: int, near_strike_spread: float, far_strike_spread: float)` Selects four contracts to form Long Iron Condor or Short Iron Condor Option strategies.
`box_spread(min_days_till_expiry: int, strike_spread: float)` Selects four contracts to form Box Spread or Short Box Spread Option strategies.
`jelly_roll(strike_from_atm: float, min_near_days_till_expiry: int, min_far_days_till_expiry: int)` Selects four contracts to form Jelly Roll or Short Jelly Roll Option strategies.
`call_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Call Ladder or Bull Call Ladder Option strategies.
`put_ladder(min_days_till_expiry: int, higher_strike_from_atm: float, middle_strike_from_atm: float, lower_strike_from_atm: float)` Selects four contracts to form Bear Put Ladder or Bull Put Ladder Option strategies.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

Filter by Implied Volatility and Greeks

Option price models compute the theoretical price of Option contracts, their implied volatility, and their Greek values. Theoretical prices can help you

detect undervalued and overvalued contracts, implied volatility can provide you insight into the upcoming volatility of the underlying security, and Greek values can help you hedge your portfolio.

```py
# Example 1: Select the contracts with delta between 0.25 and 0.75
option.set_filter(lambda option_filter_universe: option_filter_universe.delta(0.25, 0.75))

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days with implied volatility below 20%
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 90))

# Example 3: Select the contracts (including weeklys) that expire in the next 30 days with implied volatility below 20% that forms an
Iron Condor
option.set_filter(lambda option_filter_universe: option_filter_universe.iv(0, 20).expiration(0, 30).iron_condor(30, 5, 10))
```

The following table describes the filter methods of the `OptionFilterUniverse` class that select contract for a range of implied volatility, Greek values, and open interest:

`iv(min: float, max: float)`
`implied_volatility(min: float, max: float)`
 	
Selects a contract with implied volatility within the range you set.

`d(min: float, max: float)`
`delta(min: float, max: float)`
 	
Selects a contract with delta within the range you set.

`g(min: float, max: float)`
`gamma(min: float, max: float)`
 	
Selects a contract with gamma within the range you set.

`r(min: float, max: float)`
`rho(min: float, max: float)`
 	
Selects a contract with rho within the range you set.

`v(min: float, max: float)`
`vega(min: float, max: float)`
 	
Selects a contract with vega within the range you set.

`t(min: float, max: float)`
`theta(min: float, max: float)`
 	
Selects a contract with theta within the range you set.

`oi(min: int, max: int)`
`open_interest(min: float, max: float)`
 	
Selects a contract with open interest within the range you set.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

To perform thorough filtering on the `OptionFilterUniverse` , define an isolated filter method.

```py
# Select the put contracts with the lowest strike price.
option.set_filter(self._contract_selector)

def _contract_selector(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
    return option_filter_universe \
        .delta(0.5, 1.5) \
        .gamma(0.0001, 0.0006) \
        .vega(0.01, 1.5) \
        .theta(-2.0, -0.5) \
        .rho(0.5, 3.0) \
        .implied_volatility(1, 3) \
        .open_interest(100,500)
```

Filter by Other Contract Properties

To set a contract filter, in the `initialize` method, call the `set_filter` method of the `Option` object.

```py
# Select contracts that have a strike price within 1 strike level above and below the underlying price
option.set_filter(min_strike=-1, max_strike=1)

# Select contracts that expire within 30 days
option.set_filter(min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select contracts that have a strike price within 1 strike level and expire within 30 days
option.set_filter(min_strike=-1, max_strike=1, min_expiry=timedelta(days=0), maxExpiry=timedelta(days=30))

# Select call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only())
```

The following table describes the available filter techniques:

`set_filter(minStrike: int, maxStrike: int)` Selects the contracts that have a strike price within a minimum and maximum strike level relative to the underlying price. For example, say the underlying price is 302 and there are strikes at every 5. If you set `m_strike` to -1 and `max_strike` to 1, LEAN selects the contracts that have a strike of 300 or 305. This filter runs asynchronously by default.
`set_filter(minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire within the range you set. This filter runs asynchronously by default.
`set_filter(minStrike: int, maxStrike: int, minExpiry: timedelta, maxExpiry: timedelta)` Selects the contracts that expire and have a strike within the range you set. This filter runs asynchronously by default.
`set_filter(universeFunc: Callable[[OptionFilterUniverse], OptionFilterUniverse])` Selects the contracts that a function selects.

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

```
# Example 1: Select the front month call contracts
option.set_filter(lambda option_filter_universe: option_filter_universe.calls_only().front_month())

# Example 2: Select the contracts (including weeklys) that expire in the next 90 days
option.set_filter(lambda option_filter_universe: option_filter_universe.strikes(-20, 20).expiration(0, 90))
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

Default Filter

By default, LEAN subscribes to the Option contracts that have the following characteristics:

- Standard and weeklys
- Within 1 strike price of the underlying asset price
- Expire within 35 days

To adjust the universe of contracts, set a filter. The filter usually runs at the first bar of every day. When the filter selects a contract that isn't currently in your universe, LEAN adds the new contract data to the next `Slice` that it passes to the `on_data` method.

Navigate Intraday Option Chains

`OptionChain` objects represent an entire chain of Option contracts for a single underlying security.

To get the `OptionChain` , index the `option_chains` property of the `Slice` with the canonical `symbol` . After you get the `OptionChain` , you can sort and filter the Option contracts in the chain.

```
PY
def on_data(self, slice: Slice) -> None:
    # Try to get the OptionChain using the canonical symbol
    chain = slice.option_chains.get(self._symbol)
    if chain:
        # Example: Find 5 put contracts that are closest to at-the-money (ATM) and have the farthest expiration
        contracts = [x for x in chain if x.right == OptionRight.PUT]
        contracts = sorted(sorted(contracts, \
            key = lambda x: abs(chain.underlying.price - x.strike)), \
            key = lambda x: x.expiry, reverse=True)[:5]

        # Select the contract with the delta closest to -0.5
        contract = sorted(contracts, key=lambda x: abs(-0.5 - x.greeks.delta))[0]
```

You can also loop through the `option_chains` property to get each `OptionChain` .

```
PY
def on_data(self, slice: Slice) -> None:
    # Iterate all received Canonical Symbol-OptionChain key-value pairs
    for canonical_symbol, chain in slice.option_chains.items():
        contracts = chain.contracts
```

`OptionChain` objects have the following properties:

Navigate Daily Option Chains

To get the daily, pre-calculated Greeks and implied volaility of all the currently tradable contracts, call the `option_chain` method. This method returns a `DataHistory[OptionUniverse]` object, which you can format into a DataFrame or iterate through. Each row in the DataFrame and each `OptionUniverse` object represents a single contract. The data this method returns contains information on all the currently tradable contracts, not just the contracts that pass your filter.

```
PY
# In the initialize method, create a Scheduled Event to get the Option
# chain and rebalance the portfolio.
self.schedule.on(
    self.date_rules.week_start(self._symbol),
    self.time_rules.after_market_open(self._symbol, 1),
    self._rebalance
  )

# Define the method.
def _rebalance(self):
    daily_option_chain = self.option_chain(self._symbol, flatten=True)

    # Get the DataFrame format.
    df = daily_option_chain.data_frame
    delta_by_symbol = df.delta

    # Get the OptionUniverse objects format.
    for option_universe in daily_option_chain:
        close = option_universe.close
        oi = option_universe.open_interest
        delta = option_universe.greeks.delta
```

The `option_chain` method was previously known as the `option_chain_provider.get_option_contract_list` method.

`OptionUniverse` objects have the following properties:

Greeks and Implied Volatility

There are several ways to get the implied volatility and Greeks of contracts.

The Greeks and IV values in the filter function are the daily, pre-calculated values based on the end of the previous trading day.

```
PY
def initialize(self):
    option = self.add_index_option("SPX")
    option.set_filter(lambda universe: universe.delta(0.3, 0.7).expiration(0,7))
```

To calculate the values, we use our implementation of the Black-Scholes pricing model, which accounts for the interest rate and dividend payments when applicable. For example, VIX doesn't have dividends. We use SPY for SPX and QQQ for NDX.

You can't customize the Greeks and IV values that the filter function receives. However, you can create indicators to customize how the Greeks and IV are calculated for the contracts already in your universe.

To override the default pricing model of the Option, set a pricing model . If you set a price model, it sets the price model of the individual contracts in the universe.

```
PY
// Set price_model field to use the Binomial Cox-Ross-Rubinstein model to price the Options.
option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

Historical Data

To get historical chains for an Index Option, call the `history[OptionUniverse]` method with the canonical Option `Symbol` . This method returns the entire Option chain for each trading day, not the subset of contracts that pass your universe filter.

```
PY
class OptionHistoryAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        option = self.add_index_option('SPX');
        # DataFrame example:
        history_df = self.history(option.symbol, 5, flatten=True)
        # OptionUniverse objects example:
        history = self.history[OptionUniverse](option.symbol, 5)
        for chain in history:
            end_time = chain.end_time
            filtered_contracts = [c for c in chain if c.greeks.delta > 0.3]
            for contract in filtered_contracts:
                symbol = contract.symbol
                expiry = contract.id.date
                strike = contract.id.strike_price
                price = contract.close
                iv = contract.implied_volatility
```

The Greeks and IV values that you get from a history request of the Option universe are the daily, pre-calculated values based on the end of the previous trading day. To get the intraday values or to customize the Greeks and IV calculations, create some Option indicators .

The method represents each contract with an `OptionUniverse` object, which have the following properties:

Selection Frequency

By default, Index Option universes run at the first time step of each day to select their contracts.

Examples

The following examples demonstrate some common Index Option universes.

Example 1: 0DTE Contracts

0DTE Options are Option contracts that expire on the same day you trade them. The following algorithm selects 0DTE Option contracts for the SPX Index that fall within 3 strikes of the underlying price.

```py
class ZeroDTEIndexOptionUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        index = self.add_index('SPX')
        index_option = self.add_index_option(index.symbol, 'SPXW')
        index_option.set_filter(lambda u: u.expiration(0, 0).strikes(-3, 3))
```

For trading full examples of Index Option Universe, please refer to the Securities > Index Options > Requesting Data > Universes page.

Universes

Crypto

Introduction

A Crypto universe lets you select a basket of Cryptocurrencies based on `CryptoUniverse` data.

Crypto Universes

To add a universe of Cryptocurrencies, in the `initialize` method, pass a `CryptoUniverse` to the `add_universe` method.

```py
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.universe_settings.resolution = Resolution.DAILY
    self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)

    # Add crypto universe selection
    self._universe = self.add_universe(CryptoUniverse.coinbase(lambda universe_day: [c.symbol for c in universe_day]))
```

The following table shows the helper methods to create Crypto universes for the supported exchanges:

Brokerage Name	Helper Method	Example
`BrokerageName.BINANCE`	`CryptoUniverse.binance`	Example
`BrokerageName.BINANCE_US`	`CryptoUniverse.binance_us`	Example
`BrokerageName.BITFINEX`	`CryptoUniverse.bitfinex`	Example
`BrokerageName.BYBIT`	`CryptoUniverse.bybit`	Example
`BrokerageName.COINBASE`	`CryptoUniverse.coinbase`	Example
`BrokerageName.KRAKEN`	`CryptoUniverse.kraken`	Example

The following table describes the `CryptoUniverse` method arguments:

Argument	Data Type	Description	Default Value
`selector`	`Callable[[List[CryptoUniverse]], List[Symbol]]`	A function to select some of the Cryptocurrencies for the universe.	
`universe_settings`	`UniverseSettings`	The universe settings .	`None`

The filter function receives `CryptoUniverse` objects, which represent one of the Cryptocurrencies in the market. The `Symbol` objects that the filter function returns represent the universe constituents. `CryptoUniverse` objects have the following attributes:

To perform thorough filtering on the `CryptoUniverse` objects, define an isolated filter method.

```py
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.universe_settings.resolution = Resolution.DAILY
    self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)

    # Add crypto universe selection
    self._universe = self.add_universe(CryptoUniverse.coinbase(self._universe_filter))


def _universe_filter(self, universe_day):
    # Define the universe selection function
    return [cf.symbol for cf in universe_day if cf.volume >= 100 and cf.volume_in_usd > 10000]
```

Historical Data

To get historical Cryptocurrency universe data, call the `history` method with the `Universe` object and the lookback period. The return type is a multi-index `pandas.Series` or `pandas.DataFrame` containing `CryptoUniverse` data.

```
PY
# Get 30 days of history for the Crypto universe.
# DataFrame object:
df_history = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series object:
series_history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

For more information about Cryptocurrency data, see Crypto .

Alternative Data Universes

An alternative data universe lets you select a basket of Cryptocurrencies based on an alternative dataset that's linked to them. If you use an alternative data universe, you limit your universe to only the securities in the dataset, which avoids unnecessary subscriptions. Currently, only the Crypto Market Cap alternative dataset supports universe selection for Crypto.

Selection Frequency

Crypto universes run on a daily basis by default. To adjust the selection schedule, see Schedule .

Live Trading Considerations

In live mode, the pipeline has a 16-hour delay. Your algorithm receives the `CryptoUniverse` objects at around 16:00-16:30 Coordinated Universal Time (UTC) each day, depending on the data processing task.

Examples

The following examples demonstrate some common Crypto universes.

Example 1: Highly Liquid Crypto Universe

To fairly compare the liquidity between Crypto pairs, use the `volume_in_usd` property of the `CryptoUniverse` objects during universe selection. The following algorithm selects the 10 Crypto pairs with the greatest USD volume on Bitfinex:

```
PY
class HighlyLiquidCryptoUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.add_universe(CryptoUniverse.bitfinex(self._select_assets))

    def _select_assets(self, data):
        selected = sorted(
            [x for x in data if x.volume_in_usd],
            key=lambda x: x.volume_in_usd
        )[-10:]
        return [x.symbol for x in selected]
```

Example 2: Highly Volatile Crypto Universe

The following algorithm creates a universe of volatile Crypto pairs on Binance by selecting the pairs with the largest trading range in the previous day:

```
class HighlyVolatileCryptoUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.add_universe(CryptoUniverse.binance(self._select_assets))

    def _select_assets(self, data):
        selected = sorted(data, key=lambda x: (x.high - x.low) / x.low)[-10:]
        return [x.symbol for x in selected]
```

Example 3: Large Cap Crypto Universe

The following algorithm uses the CoinGecko Crypto Market Cap dataset to create a universe of the largest Crypto pairs on Coinbase. To avoid trading errors, it only selects pairs that have a quote currency that matches your account currency . At noon each day, it rebalances the portfolio to give equal weight to each coin in the universe.

```
class LargeCapCryptoUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the account currency. USD is already the default value. Change it here if you want.
        self.set_account_currency("USD")
        # Get the pairs that our brokerage supports and have a quote currency that
        # matches your account currency. We need this list in the universe selection function.
        self._market = Market.COINBASE
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(self._market)
            if x.value.quote_currency == self.account_currency
        ]
        # Add a universe of Cryptocurrencies.
        self._universe = self.add_universe(CoinGeckoUniverse, 'CoinGeckoUniverse', Resolution.HOUR, self._select_assets)
        # Add a Sheduled Event to rebalance the portfolio.
        self.schedule.on(self.date_rules.every_day(), self.time_rules.at(12, 0), self._rebalance)

    def _select_assets(self, data: List[CoinGecko]) -> List[Symbol]:
        # Select the coins that our brokerage supports and have a quote currency that matches
        # our account currency.
        tradable_coins = [d for d in data if d.coin + self.account_currency in self._market_pairs]
        # Select the largest coins and create their Symbol objects.
        return [
            c.create_symbol(self._market, self.account_currency)
            for c in sorted(tradable_coins, key=lambda x: x.market_cap)[-10:]
        ]

    def _rebalance(self):
        if not self._universe.selected:
            return
        symbols = [symbol for symbol in self._universe.selected if self.securities[symbol].price]
        # Liquidate coins that are no longer in the universe.
        targets = [PortfolioTarget(symbol, 0) for symbol, holding in self.portfolio.items() if holding.invested and symbol not in
symbols]
        # Form an equal weighted portfolio of the coins in the universe.
        targets += [PortfolioTarget(symbol, 0.5/len(symbols)) for symbol in symbols]
        # Place orders to rebalance the portfolio.
        self.set_holdings(targets)
```

Example 4: Indicator Universe

The following example shows how to select a universe of Crypto pairs based on indicators.

PY

```
class CryptoIndicatorUniverseSelectionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Crypto pairs based on indicators.
        self._selection_data_by_symbol = {}
        self._universe = self.add_universe(CryptoUniverse.coinbase(self._select_assets))
        # Add a warm-up period to warm up the indicators in the universe selection.
        self.set_warm_up(timedelta(60))

    def _select_assets(self, data):
        # Update the indicator of all Crypto pairs in the universe dataset and
        # get the subset of them that have their indicator ready.
        ready_pairs = [
            c for c in data
            if self._selection_data_by_symbol.setdefault(c.symbol, SelectionData()).update(c)
        ]
        # As Crypto pairs leave the universe dataset, delete their SelectionData objects.
        for symbol in self._selection_data_by_symbol.keys() - {c.symbol for c in data}:
            del self._selection_data_by_symbol[symbol]
        # During warm-up, keep the universe empty.
        if self.is_warming_up:
            return []
        # Select a subset of the Crypto pairs based on their indicators.
        # Example: Pairs trading above their bollinger band with high volume.
        filtered = []
        for c in ready_pairs:
            selection_data = self._selection_data_by_symbol[c.symbol]
            if c.close > selection_data.bb.upper_band.current.value:
                filtered.append(c)
        filtered = sorted(filtered, key=lambda c: self._selection_data_by_symbol[c.symbol].mean_daily_usd_volume)[-10:]
        return [c.symbol for c in filtered]


class SelectionData:

    def __init__(self):
        self.mean_daily_usd_volume = SimpleMovingAverage(30)
        self.bb = BollingerBands(30, 2)

    def update(self, c):
        # Update all the indicators and return true if they are all ready.
        return (
            self.mean_daily_usd_volume.update(c.end_time, c.volume_in_usd) &
            self.bb.update(TradeBar(c.end_time, c.symbol, c.open, c.high, c.low, c.close, c.volume_in_usd))
        )
```

Universes

Custom Universes

Introduction

A custom universe lets you select a basket of assets from a custom dataset.

Data Sources

You can gather your custom data from any of the following sources:

The data source should serve data in chronological order and each data point should have a unique timestamp. Each request has a 1 second overhead, so bundle samples together for fast execution.

Define Custom Universe Types

Custom universes should extend the `PythonData` class. Extensions of the `PythonData` class must implement a `get_source` and `reader` method.

The `get_source` method in your custom data class instructs LEAN where to find the data. This method must return a `SubscriptionDataSource` object, which contains the data location and format (`SubscriptionTransportMedium`). You can even change source locations for backtesting and live modes. We support many different data sources.

The `reader` method of your custom data class takes one line of data from the source location and parses it into one of your custom objects. You can add as many properties to your custom data objects as you need, but must set `symbol` and `end_time` properties. When there is no useable data in a line, the method should return `None` . LEAN repeatedly calls the `reader` method until the date/time advances or it reaches the end of the file.

```py
# Example custom universe data; it is virtually identical to other custom data types.
class MyCustomUniverseDataClass(PythonData):

    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live_mode: bool) -> SubscriptionDataSource:
        return SubscriptionDataSource(@"your-remote-universe-data", SubscriptionTransportMedium.REMOTE_FILE)

    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live_mode: bool) -> BaseData:
        items = line.split(",")

        # Generate required data, then return an instance of your class.
        data = MyCustomUniverseDataClass()
        data.end_time = datetime.strptime(items[0], "%Y-%m-%d")
        # define Time as exactly 1 day earlier Time
        data.time = data.end_time - timedelta(1)
        data.symbol = Symbol.create(items[1], SecurityType.CRYPTO, Market.BITFINEX)
        data["CustomAttribute1"] = int(items[2])
        data["CustomAttribute2"] = float(items[3])
        return data
```

Your `reader` method should return objects in chronological order. If an object has a timestamp that is the same or earlier than the timestamp of the previous object, LEAN ignores it.

If you need to create multiple objects in your `reader` method from a single `line` , follow these steps:

1. In the `get_source` method, pass `FileFormat.UNFOLDING_COLLECTION` as the third argument to the `SubscriptionDataSource` constructor.
2. In the `reader` method, order the objects by their timestamp and then return a `BaseDataCollection(end_time, config.symbol, objects)` where `objects` is a list of your custom data objects.

```py
class MyCustomUniverseDataClass(PythonData):

    def get_source(self, config, date, isLive):
        return SubscriptionDataSource("your-data-source-url", SubscriptionTransportMedium.REMOTE_FILE, FileFormat.UNFOLDING_COLLECTION)

    def reader(self, config, line, date, isLive):
        json_response = json.loads(line)

        end_time = datetime.strptime(json_response[-1]["date"], '%Y-%m-%d') + timedelta(1)

        data = list()

        for json_datum in json_response:
            datum = MyCustomUniverseDataClass()
            datum.symbol = Symbol.create(json_datum["Ticker"], SecurityType.EQUITY, Market.USA)
            datum.time = datetime.strptime(json_datum["date"], '%Y-%m-%d')
            datum.end_time = datum.time + timedelta(1)
            datum['CustomAttribute1'] = int(json_datum['Attr1'])
            datum.value = float(json_datum['Attr1'])
            data.append(datum)

        return BaseDataCollection(end_time, config.symbol, data)
```

Initialize Custom Universes

To add a custom universe to your algorithm, in the `initialize` method, pass your universe type and a selector function to the `add_universe` method. The selector function receives a list of your custom objects and must return a list of `Symbol` objects. In the selector function definition, you can use any of the properties of your custom data type. The `Symbol` objects that you return from the selector function set the constituents of the universe.

```py
# In Initialize
self._universe = self.add_universe(MyCustomUniverseDataClass, "myCustomUniverse", Resolution.DAILY, self.selector_function)

# Define the selector function
def selector_function(self, data: list[MyCustomUniverseDataClass]) -> list[Symbol]:
    sorted_data = sorted([ x for x in data if x["CustomAttribute1"] > 0 ],
                         key=lambda x: x["CustomAttribute2"],
                         reverse=True)
    return [x.symbol for x in sorted_data[:5]]
```

Historical Data

To get historical custom universe data, call the `history` method with the `Universe` object and the lookback period. The return type is a `pandas.DataFrame` where the columns contain the custom type attributes.

```py
# DataFrame where teh columns are the universe attributes:
history_df = self.history(self._universe, 30, flatten=True)

# Series where the values are lists of universe objects:
history_series = self.history(self._universe, 30)
for time, data in history_series.iterrows():
    for single_stock_data in data:
        self.log(f"{single_stock_data.symbol} CustomAttribute1 at {single_stock_data.end_time}: {single_stock_data['CustomAttribute1']}")
```

Selection Frequency

Custom universes run on a schedule based on the `end_time` of your custom data objects. To adjust the selection schedule, see Schedule .

Examples

The following examples demonstrate some common practices for Custom Universe.

Example 1: Sourcing from the Object Store

This project demonstrates how to read custom data from the Object Store , and then use it to define a universe and place trades. The following research environment file generates the demo universe data, which contains a daily set of assets and their respective signals:

```
# Set a random seed to ensure reproducibility.
import random
np.random.seed(0)
# Select the asset weights for each trading day.
indices = [[x] * 3 for x in pd.bdate_range('2015-01-01', '2024-12-31')]
weights = list(np.random.dirichlet((10, 5, 3), size=(len(indices),)).flatten())
# Select the universe for each trading day.
equities = []
for i in range(len(indices)):
    random.seed(i)
    equities.extend(list(random.sample(["SPY", "TLT", "GLD", "USO", "IWM"], 3)))
# Organize the data into a DataFrame.
df = pd.DataFrame({"Date": [x for y in indices for x in y], "Symbol": equities, "Weight": weights})
# Save the DataFrame as a CSV in the Object Store.
df.to_csv(QuantBook().object_store.get_file_path("portfolio-targets.csv"), index=False)
```

The following algorithm file reads the preceding CSV file from the Object Store and uses its contents to form the daily universe and place trades:

```
class CustomUniverseExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add a universe that reads from the Object Store.
        self._universe = self.add_universe(
            CustomUniverseData, "CustomUniverse", Resolution.DAILY, self._selector_function
        )
        # Add a Scheduled Event to rebalance the portfolio.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            lambda: self.set_holdings(
                [PortfolioTarget(symbol, self._weight_by_symbol[symbol]) for symbol in self._universe.selected],
                liquidate_existing_holdings=True
            )
        )

    def _selector_function(self, alt_coarse: List[PythonData]) -> List[Symbol]:
        # Select the symbols that have a significant weight in the custom universe data to avoid
        # small-size trades that erode returns. Save the weight to use during the rebalance.
        self._weight_by_symbol = {d.symbol: d.weight for d in alt_coarse if d["weight"] > 0.05}
        return list(self._weight_by_symbol.keys())


class CustomUniverseData(PythonData):

    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live_mode: bool) -> SubscriptionDataSource:
        # Define the location and format of the data file.
        return SubscriptionDataSource(
            "portfolio-targets.csv", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.CSV
        )

    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live_mode: bool) -> BaseData:
        # Skip the header row.
        if not line[0].isnumeric():
            return None
        # Split the line by each comma.
        items = line.split(",")
        # Parse the data from the CSV file.
        data = CustomUniverseData()
        data.end_time = datetime.strptime(items[0], "%Y-%m-%d")
        data.time = data.end_time - timedelta(1)
        data.symbol = Symbol.create(items[1], SecurityType.EQUITY, Market.USA)
        data["weight"] = float(items[2])
        return data
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
DropboxUniverseSelectionAlgorithm.py Python DropboxBaseDataUniverseSelectionAlgorithm.py Python

Universes

Dataless Scheduled Universes

Introduction

A dataless scheduled universe let's you select a set of assets on a specific schedule. You can control which days the other types of universe run by adjusting the Schedule universe setting . However, the `schedule` universe setting doesn't accept a `TimeRule` argument, so you can't control the time of the day they run. In contrast, a dataless scheduled universe accepts a `TimeRule` argument, but its selection function only receives the algorithm time.

Create Universes

To add a dataless scheduled universe, in the `initialize` method, call the `add_universe` method with a `ScheduledUniverse` object.

```
PY
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.asynchronous = True
        self._universe = self.add_universe(
            ScheduledUniverse(
                self.date_rules.month_start(),
                self.time_rules.at(8, 0),
                self._select_symbols
            )
        )

    def _select_symbols(self, dt):
        return [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`date_rule`	`IDateRule`	Date rule that defines what days the universe selection function runs	
`time_rule`	`ITimeRule`	Time rule that defines what times on each day selected by date rule the universe selection function runs	
`Callable[[datetime], List[Symbol]]`	`selector`	Selector function that accepts the current date time and returns the universe's `Symbol` objects	
`settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

Date Rules

The following table describes the supported `DateRules` :

Member	Description
`self.date_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `DateRules` object used in all methods in this table. The default time zone is the algorithm time zone .
`self.date_rules.on(year: int, month: int, day: int)`	Trigger an event on a specific date.
`self.date_rules.on(dates: list[datetime])`	Trigger an event on specific dates.
`self.date_rules.every_day()`	Trigger an event all seven days of the week.
`self.date_rules.every_day(symbol: Symbol, extended_market_hours: bool = False)`	Trigger an event every day a specific symbol is trading.
`self.date_rules.every(days: list[DayOfWeek])`	Trigger an event on specific days throughout the week. To view the `DayOfWeek` enum members, see DayOfWeek Enum in the .NET documentation.
`self.date_rules.month_start(days_offset: int = 0)`	Trigger an event on the first day of each month plus an offset.
`self.date_rules.month_start(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the first tradable date of each month for a specific symbol plus an offset.
`self.date_rules.month_end(days_offset: int = 0)`	Trigger an event on the last day of each month minus an offset.
`self.date_rules.month_end(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the last tradable date of each month for a specific symbol minus an offset.
`self.date_rules.week_start(days_offset: int = 0)`	Trigger an event on the first day of each week plus an offset.
`self.date_rules.week_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each week for a specific symbol plus an offset.
`self.date_rules.week_end(days_offset: int = 0)`	Trigger an event on the last day of each week minus an offset.
`self.date_rules.week_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each week for a specific symbol minus an offset.
`self.date_rules.year_start(days_offset: int = 0)`	Trigger an event on the first day of each year plus an offset.
`self.date_rules.year_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each year for a specific symbol plus an offset.
`self.date_rules.year_end(days_offset: int = 0)`	Trigger an event on the last day of each year minus an offset.
`self.date_rules.year_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each year for a specific symbol minus an offset.
`self.date_rules.today`	Trigger an event once today.
`self.date_rules.tomorrow`	Trigger an event once tomorrow.

To define custom date rules, create a `FuncDateRule` object. The `FuncDateRule` constructor expects a `name` argument of type `str` and a `get_dates_function` argument of type `Callable[[datetime, datetime], list[datetime]]` . The `get_dates_function` function receives the start and end dates of the algorithm and returns a list of dates for the date rule. In live trading, the end date is 12/31/2025. The following example demonstrates how to define a date rule that represents the 10th day of each month:

```py
# Create a date rule that specifies the 10th day of each month.
date_rule = FuncDateRule(
    name="10th_day_of_the_month",
    get_dates_function=lambda start, end: [
        datetime(year, month, 10)
        for year in range(start.year, end.year) for month in range(1,12)
    ]
)
```

Some date rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.date_rules.every_day('ES')` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

Time Rules

The following table describes the supported `TimeRules` :

Member	Description
`self.time_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `TimeRules` object used in all methods in this table, except when a different time zone is given. The default time zone is the algorithm time zone .
`self.time_rules.before_market_open(symbol: Symbol, minutes_before_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_open(symbol: Symbol, minutes_after_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.before_market_close(symbol: Symbol, minutes_before_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_close(symbol: Symbol, minutes_after_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.every(interval: timedelta)`	Trigger an event every period interval starting at midnight. This time rule triggers every period, regardless of whether or not the market is open.
`self.time_rules.now`	Trigger an event at the current time of day.
`self.time_rules.midnight`	Trigger an event at midnight.
`self.time_rules.noon`	Trigger an event at noon.
`self.time_rules.at(hour: int, minute: int, second: int = 0)`	Trigger an event at a specific time of day (e.g. 13:10).
`self.time_rules.at(hour: int, minute: int, second: int, time_zone: DateTimeZone)`	Trigger an event at a specific time of day in the given time zone (e.g. 13:10 UTC).

To define custom time rules, create a `FuncTimeRule` object. The `FuncTimeRule` constructor expects a `name` argument of type `str` and a `create_utc_event_times_function` argument of type `Callable[[list[datetime]], list[datetime]]` . The function receives the list of dates from the date rule and then returns a list of `datetime` that define the time rule.

```
time_rule = FuncTimeRule(
    name="CustomTimeRule",
    create_utc_event_times_function=lambda dates: [d + timedelta(hours=10) for d in dates]
)
```

Some time rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.time_rules.before_market_open('ES', 15)` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

Selection Frequency

Dataless scheduled universes run at whatever selection schedule you define by the `date_rule` and `time_rule` arguments.

Examples

The following examples demonstrate some common practices for dataless scheduled universes.

Example 1: Download External Universe Files

The following algorithm downloads a CSV file from Dropbox. The first column in the file contains the universe date. The second column in the file contains a list of Equity tickers that represents the universe for the day. The dataless scheduled universe parses the file contents each trading day and adds the universe constituents.

```py
from io import StringIO


class DatalessScheduledUniverseExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Download the initial universe file.
        self._download_universe_file()
        # Add the custom universe.
        self.universe_settings.extended_market_hours = True
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        date_rule = self.date_rules.every_day(spy)
        self._universe = self.add_universe(
            ScheduledUniverse(date_rule, self.time_rules.at(8, 0), self._select_assets)
        )
        # Schedule rebalances for market open.
        self.schedule.on(date_rule, self.time_rules.after_market_open(spy, 0), self._rebalance)

    def _download_universe_file(self):
        # Download the universe CSV file. Dropbox links require the "dl=1" URL parameter.
        file = self.download(
            "https://www.dropbox.com/scl/fi/fbrxitk4ec3w91nse1raa/df.csv?rlkey=7r042rukzkthp7y1srloyhkov&st=5r4sdfwd&dl=1"
        )
        # Convert the CSV file data into a dictionary where the key is the date and
        # the value is a comma-separated string of stock tickers.
        df = pd.read_csv(StringIO(file), index_col=0).iloc[:, 0]
        df.index = pd.to_datetime(df.index).date
        self._tickers_by_date = df.to_dict()

    def _select_assets(self, dt: datetime) -> List[Symbol]:
        # When live trading, re-download the CSV file each day to get the new rows.
        if self.live_mode:
            self._download_universe_file()
        # Get the current day's data from the CSV file.
        data = self._tickers_by_date.get(dt.date(), '')
        # If there isn't an entry for the current date, return an empty universe.
        if not data:
            return []
        # Convert the stock tickers in the CSV file to Symbol objects.
        return [Symbol.create(x, SecurityType.EQUITY, Market.USA) for x in data.split(",")]

    def _rebalance(self) -> None:
        # Form an equal-weighted portfolio of all the universe constituents.
        symbols = list(self._universe.selected)
        weight = 1 / len(symbols)
        self.set_holdings([PortfolioTarget(symbol, weight) for symbol in symbols], liquidate_existing_holdings=True)
```

Example 2: Quarter End Selection

The following algorithm selects SPY on the last month of each quarter. For the remaining months, it selects no assets.

```
class DatalessScheduledUniverseDemoAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.add_universe(
            ScheduledUniverse(
                self.date_rules.month_start(),
                self.time_rules.at(8, 0),
                self._select_assets
            )
        )

    def _select_assets(self, dt: datetime) -> List[Symbol]:
        if dt.month % 3 == 0:
            return [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
        return []
```

Example 3: Third Week VIX

Standard Options expire at end of the third week of each month. The following algorithm selects VIX-related products on the third week to trade the foreseeable increase in volatility.

```
class DatalessScheduledUniverseDemoAlgorithm(QCAlgorithm):

    def initialize(self):
        self._month = -1
        self._week = -1
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.add_universe(
            ScheduledUniverse(
                self.date_rules.week_start(),
                self.time_rules.at(8, 0),
                self._select_assets
            )
        )

    def _select_assets(self, dt: datetime) -> List[Symbol]:
        if dt.month == self._month:
            self._week += 1
            if self._week == 3:
                return [Symbol.create("VXZ", SecurityType.EQUITY, Market.USA)]
            return []

        self._month = dt.month
        self._week = 0
        return []
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomDataUniverseScheduledRegressionAlgorithm.py Python

Datasets

Datasets

Overview

Introduction

It's common to use price and fundamental data points to inform trading decisions. Alternative datasets are datasets that aren't classified as market or fundamental data. Alternative data research began gaining popularity in 2007 after the financial crisis and its usage has continued to grow over the years. As usage has grown, so has the number of unique alternative datasets that are available for investors. The benefit of alternative datasets is that it's easier to find alpha in these datasets because they receive less attention from investment researchers and these datasets contain information not found in market and fundamental datasets. Most traders research with market and fundamental datasets, so it's difficult to find alpha in them.

Methodology

The journey of raw data from its source to its integration with a trading platform can be complex. Alternative data is generally created through individuals, business processes, and sensors. Individuals create content on social media platforms, write reviews on e-commerce sites, and visit corporate Wikipedia pages. Businesses process transactions, file SEC reports, and issue company announcements. Sensors collect location data, gather satellite images, and monitor customer behavior.

Data providers can capture these forms of data for traders to inform trading decisions in ways that supplement traditional market and fundamental datasets. With over a million terabytes of data on the internet, there is virtually a limitless amount of information that can be informative for investors when analyzing a security's performance in the marketplace.

Examples

A common example of alternative data in a trading system is using sentiment analysis on news sources to predict the future price movements of Equities. A second example is using announcements regarding corporate share buyback programs to trading Equities. A third example is using global supply and demand information for US Crude Products to inform Futures trades. To view all of the datasets we have available, see the left navigation menu.

Challenges

A challenge of working with datasets is that it can be difficult to find a reliable, high-quality, and inexpensive data source to use in live algorithms. If a data source has survivorship bias, insufficient history, or hasn't been cleaned and processed properly, the data can cause poor trading performance. We thoroughly vet the datasets we add to the Dataset Market to ensure they are free of survivorship bias and have a reliable live feed. All of the available datasets have already been cleaned and processed, so you can just focus on the strategy research and development.

Future Outlook

The Dataset Market is still in the early phases of development. Over time, we will add more markets and alternative datasets. Alternative datasets are an ever-expanding data source, so it's expected to continue to grow into the future. In today's digital world, more data is created each year than in the previous year. To take advantage of the trend, we have implemented a new onboarding process to integrate many new alternative data sources into the QuantConnect platform. With the Dataset Market, you can import a new alternative dataset into your trading algorithm with just a single line of code. The Dataset Market gives everyone the ability to access high-quality alternative datasets that were previously only accessible to major hedge funds.

Live Trading Considerations

We receive most daily alternative data at 7 AM Eastern Time (ET) and start processing it. In live trading, your algorithm receives this data within 30 minutes of 7 AM ET. In backtesting, this data has a timestamp of midnight, so your algorithm receives it about seven hours earlier than it does in live mode.

In backtests, your algorithm receives data at perfect timing. If you request minute resolution data, your algorithm receives the bars at the top of each minute. In live trading, bars have a slight delay, so you may receive them milliseconds after the top of each minute.

Note that there will be delay for data available to be backtested. Live feed data is available for backtesting after 24-48 hours normally. Consider using Paper live trading if close monitoring is needed.

Datasets

QuantConnect

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Binance Crypto Future Margin Rate Data

Bybit Crypto Future Margin Rate Data

CFD Data

Cash Indices

FOREX Data

Fear and Greed

International Future Universe

US ETF Constituents

US Equities Short Availability

US Equity Coarse Universe

US Equity Option Universe

US Equity Security Master

US Future Option Universe

US Future Universe

US Futures Security Master

US Index Option Universe

QuantConnect

Binance Crypto Future Margin Rate Data

Introduction

The Binance Crypto Future Margin Rate Data by QuantConnect is for crypto-currency futures margin interest data points. The data covers 718 Cryptocurrency pairs, starts in August 2020, and is delivered on a daily update frequency. This dataset is created by downloading data using Binance API.

This dataset is an important companion to the Binance Crypto Future Price Data dataset because it contains information on margin interest data to model margin costs.

For more information about the Binance Crypto Future Margin Rate Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the Binance Crypto Future Margin Rate dataset:

```
def initialize(self) -> None:
    self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
    #self.set_brokerage_model(BrokerageName.BINANCE_COIN_FUTURES, AccountType.MARGIN)

    self.crypto_future_symbol = self.add_crypto_future("BTCBUSD", Resolution.MINUTE).symbol
```

The Binance Crypto Future Margin Rate data is added to your algorithm along with the market data when you add a crypto future subscription.

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	August 2020
Asset Coverage	718 Crypto Futures Pairs
Data Density	Regular
Resolution	Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Binance Crypto Future Margin Rate data to your algorithm, call the `add_crypto_future` method. Save a reference to the Crypto Future `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Binance Stable Coin for USD
        self.set_account_currency("BUSD")
        self.set_cash(100000)

        self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
        self.set_brokerage_model(BrokerageName.BINANCE_COIN_FUTURES, AccountType.MARGIN)

        crypto_future = self.add_crypto_future("BTCBUSD", Resolution.MINUTE)
        # perpetual futures does not have a filter function
        self.btcbusd = crypto_future.symbol
```

For more information about creating Crypto Future subscriptions, see Requesting Data .

Accessing Data

To get the current Binance Crypto Margin Rate data, index the `margin_interest_rates` property of the current `Slice` with the Crypto Future `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```py
def on_data(self, slice: Slice) -> None:
    if self.btcbusd in slice.margin_interest_rates:
        interest_rate = slice.margin_interest_rates[self.btcbusd].interest_rate
        self.log(f"{self.btcbusd} close at {slice.time}: {interest_rate}")
```

You can also iterate through all of the data objects in the current `Slice` .

```py
def on_data(self, slice: Slice) -> None:
    for symbol, margin_interest_rate in slice.margin_interest_rates.items():
        interest_rate = margin_interest_rate.interest_rate
        self.log(f"{symbol} close at {slice.time}: {interest_rate}")
```

For more information about accessing Crypto Future data, see Handling Data .

Historical Data

To get historical Binance Crypto Future Margin Rate data, call the `history` method with `MarginInterestRate` data type and the Crypto Future `Symbol` . If there is no data in the period you request, the history result is empty.

```py
# DataFrame
history_df = self.history(MarginInterestRate, self.btcbusd, 100, Resolution.DAILY)

# MarginInterestRate objects
history = self.history[MarginInterestRate](self.btcbusd, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcbusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Crypto Future pairs:

Pairs Available (718)					
0GUSDT	1000000BOBUSDT	1000000MOGUSDT	1000BONKUSDC	1000BONKUSDT	1000CATUSDT
1000CHEEMSUSDT	1000FLOKIUSDT	1000LUNCBUSD	1000LUNCUSDT	1000PEPEUSDC	1000PEPEUSDT
1000RATSUSDT	1000SATSUSDT	1000SHIBBUSD	1000SHIBUSDC	1000SHIBUSDT	1000WHYUSDT
1000XECUSDT	1000XUSDT	1INCHUSDT	1MBABYDOGEUSDT	2ZUSDT	4USDT
A2ZUSDT	AAVEUSD	AAVEUSDC	AAVEUSDT	ACEUSDT	ACHUSDT
ACTUSDT	ACXUSDT	ADABUSD	ADAUSD	ADAUSDC	ADAUSDT
AERGOUSDT	AEROUSDT	AEVOUSDT	AGIXBUSD	AGIXUSDT	AGLDUSDT
AGTUSDT	AI16ZUSDT	AIAUSDT	AINUSDT	AIOTUSDT	AIOUSDT
AIUSDT	AIXBTUSDT	AKEUSDT	AKTUSDT	ALCHUSDT	ALGOUSD
ALGOUSDT	ALICEUSDT	ALLUSDT	ALPACAUSDT	ALPHAUSDT	ALPINEUSDT
ALTUSDT	AMBBUSD	AMBUSDT	ANCBUSD	ANIMEUSDT	ANKRUSDT
ANTUSDT	APEBUSD	APEUSD	APEUSDT	API3USDT	APTBUSD
APTUSD	APTUSDT	ARBUSDC	ARBUSDT	ARCUSDT	ARIAUSDT
ARKMUSDT	ARKUSDT	ARPAUSDT	ARUSDT	ASRUSDT	ASTERUSDT
ASTRUSDT	ATAUSDT	ATHUSDT	ATOMUSD	ATOMUSDT	AUCTIONBUSD
AUCTIONUSDT	AUDIOUSDT	AUSDT	AVAAIUSDT	AVAUSDT	AVAXBUSD
AVAXUSD	AVAXUSDC	AVAXUSDT	AVNTUSDT	AWEUSDT	AXLUSDT
AXSUSD	AXSUSDT	B2USDT	B3USDT	BABYUSDT	BADGERUSDT
BAKEUSDT	BALUSDT	BANANAS31USDT	BANANAUSDT	BANDUSDT	BANKUSDT
BANUSDT	BARDUSDT	BASUSDT	BATUSDT	BBUSDT	BCHUSD
BCHUSDC	BCHUSDT	BDXNUSDT	BEAMXUSDT	BELUSDT	BERAUSDT
BICOUSDT	BIDUSDT	BIGTIMEUSDT	BIOUSDC	BIOUSDT	BLESSUSDT
BLUAIUSDT	BLUEBIRDUSDT	BLURUSDT	BLZUSDT	BMTUSDT	BNBBUSD
BNBUSD	BNBUSDC	BNBUSDT	BNTUSDT	BNXUSDT	BOMEUSDC
BOMEUSDT	BONDUSDT	BRETTUSDT	BROCCOLI714USDT	BROCCOLIF3BUSDT	BRUSDT
BSVUSDT	BSWUSDT	BTCBUSD	BTCDOMUSDT	BTCSTUSDT	BTCUSD
BTCUSDC	BTCUSDT	BTRUSDT	BTSUSDT	BULLAUSDT	BUSDT
C98USDT	CAKEUSDT	CARVUSDT	CATIUSDT	CELOUSDT	CELRUSDT
CETUSUSDT	CFXUSDT	CGPTUSDT	CHESSUSDT	CHILLGUYUSDT	CHRUSDT
CHZUSD	CHZUSDT	CKBUSDT	CLOUSDT	COAIUSDT	COCOSUSDT
COMBOUSDT	COMPUSDT	COOKIEUSDT	COSUSDT	COTIUSDT	COWUSDT

CROSSUSDT	CRVUSDC	CRVUSDT	CTKUSDT	CTSIUSDT	CUDISUSDT
CUSDT	CVCUSDT	CVXBUSD	CVXUSDT	CYBERUSDT	DAMUSDT
DARUSDT	DASHUSDT	DEEPUSDT	DEFIUSDT	DEGENUSDT	DEGOUSDT
DENTUSDT	DEXEUSDT	DFUSDT	DGBUSDT	DIAUSDT	DMCUSDT
DODOBUSD	DODOXUSDT	DOGEBUSD	DOGEUSD	DOGEUSDC	DOGEUSDT
DOGSUSD	DOGSUSDT	DOLOUSDT	DOODUSDT	DOTBUSD	DOTUSD
DOTUSDT	DRIFTUSDT	DUSDT	DUSKUSDT	DYDXUSDT	DYMUSDT
EDENUSDT	EDUUSDT	EGLDUSD	EGLDUSDT	EIGENUSDT	ENAUSDC
ENAUSDT	ENJUSDT	ENSOUSDT	ENSUSD	ENSUSDT	EOSUSD
EOSUSDT	EPICUSDT	EPTUSDT	ERAUSDT	ESPORTSUSDT	ETCBUSD
ETCUSD	ETCUSDT	ETHBTC	ETHBUSD	ETHFIUSDC	ETHFIUSDT
ETHUSD	ETHUSDC	ETHUSDT	ETHWUSDT	EULUSDT	EVAAUSDT
FARTCOINUSDT	FETUSDT	FFUSDT	FHEUSDT	FIDAUSDT	FILBUSD
FILUSD	FILUSDC	FILUSDT	FIOUSDT	FISUSDT	FLMUSDT
FLOCKUSDT	FLOWUSDT	FLUIDUSDT	FLUXUSDT	FOOTBALLUSDT	FORMUSDT
FORTHUSDT	FRONTUSDT	FTMBUSD	FTMUSD	FTMUSDT	FTTBUSD
FTTUSDT	FUNUSDT	FUSDT	FXSUSDT	GALABUSD	GALAUSD
GALAUSDT	GALBUSD	GALUSDT	GASUSDT	GHSTUSDT	GIGGLEUSDT
GLMRUSDT	GLMUSDT	GMTBUSD	GMTUSD	GMTUSDT	GMXUSDT
GOATUSDT	GPSUSDT	GRASSUSDT	GRIFFAINUSDT	GRTUSDT	GTCUSDT
GUNUSDT	GUSDT	HAEDALUSDT	HANAUSDT	HBARUSDC	HBARUSDT
HEIUSDT	HEMIUSDT	HFTUSDT	HIFIUSDT	HIGHUSDT	HIPPOUSDT
HIVEUSDT	HMSTRUSDT	HNTUSDT	HOLOUSDT	HOMEUSDT	HOOKUSDT
HOTUSDT	HUMAUSDT	HUSDT	HYPERUSDT	HYPEUSDT	ICNTUSDT
ICPBUSD	ICPUSDT	ICXUSD	ICXUSDT	IDEXUSDT	IDOLUSDT
IDUSDT	ILVUSDT	IMXUSDT	INITUSDT	INJUSDT	INUSDT
IOSTUSDT	IOTAUSDT	IOTXUSDT	IOUSDT	IPUSDC	IPUSDT
JASMYUSDT	JELLYJELLYUSDT	JOEUSDT	JSTUSDT	JTOUSDT	JUPUSDT
KAIAUSDT	KAITOUSDC	KAITOUSDT	KASUSDT	KAVAUSDT	KDAUSDT
KERNELUSDT	KEYUSDT	KGENUSDT	KLAYUSDT	KMNOUSDT	KNCUSD
KNCUSDT	KOMAUSDT	KSMUSDT	LABUSDT	LAUSDT	LAYERUSDT
LDOBUSD	LDOUSDT	LEVERBUSD	LEVERUSDT	LIGHTUSDT	LINAUSDT
LINEAUSDT	LINKBUSD	LINKUSD	LINKUSDC	LINKUSDT	LISTAUSDT

LITUSDT	LOKAUSDT	LOOMUSDT	LPTUSDT	LQTYUSDT	LRCUSDT
LSKUSDT	LTCBUSD	LTCUSD	LTCUSDC	LTCUSDT	LUMIAUSDT
LUNA2BUSD	LUNA2USDT	LYNUSDT	MAGICUSDT	MANAUSD	MANAUSDT
MANTAUSDT	MASKUSDT	MATICBUSD	MATICUSD	MATICUSDC	MATICUSDT
MAVIAUSDT	MAVUSDT	MBLUSDT	MBOXUSDT	MDTUSDT	MELANIAUSDT
MEMEFIUSDT	MEMEUSDT	MERLUSDT	METISUSDT	METUSDT	MEUSDT
MEWUSDT	MILKUSDT	MINAUSDT	MIRAUSDT	MITOUSDT	MKRUSDT
MLNUSDT	MOCAUSDT	MONUSDT	MOODENGUSDT	MORPHOUSDT	MOVEUSDT
MOVRUSDT	MTLUSDT	MUBARAKUSDT	MUSDT	MYROUSDT	MYXUSDT
NAORISUSDT	NEARBUSD	NEARUSD	NEARUSDC	NEARUSDT	NEIROETHUSDT
NEIROUSDT	NEOUSDC	NEOUSDT	NEWTUSDT	NFPUSDT	NILUSDT
NKNUSDT	NMRUSDT	NOMUSDT	NOTUSDT	NTRNUSDT	NULSUSDT
NXPCUSDT	OBOLUSDT	OCEANUSDT	OGNUSDT	OGUSDT	OLUSDT
OMGUSDT	OMNIUSDT	OMUSDT	ONDOUSDT	ONEUSDT	ONGUSDT
ONTUSDT	OPENUSDT	OPUSD	OPUSDT	ORBSUSDT	ORCAUSDT
ORDERUSDT	ORDIUSDC	ORDIUSDT	OXTUSDT	PARTIUSDT	PAXGUSDT
PENDLEUSDT	PENGUUSDC	PENGUUSDT	PEOPLEUSDT	PERPUSDT	PHAUSDT
PHBBUSD	PHBUSDT	PIPPINUSDT	PIXELUSDT	PLAYUSDT	PLUMEUSDT
PNUTUSDC	PNUTUSDT	POLUSDT	POLYXUSDT	PONKEUSDT	POPCATUSDT
PORT3USDT	PORTALUSDT	POWRUSDT	PROMPTUSDT	PROMUSDT	PROVEUSDT
PTBUSDT	PUFFERUSDT	PUMPBTCUSDT	PUMPUSDT	PUNDIXUSDT	PYTHUSDT
QNTUSDT	QTUMUSDT	QUICKUSDT	QUSDT	RADUSDT	RAREUSDT
RAYSOLUSDT	RAYUSDT	RDNTUSDT	RECALLUSDT	REDUSDT	REEFUSDT
REIUSDT	RENDERUSDT	RENUSDT	RESOLVUSDT	REZUSDT	RIFUSDT
RIVERUSDT	RLCUSDT	RNDRUSDT	RONINUSDT	ROSEUSD	ROSEUSDT
RPLUSDT	RSRUSDT	RUNEUSD	RUNEUSDT	RVNUSDT	RVVUSDT
SAFEUSDT	SAGAUSDT	SAHARAUSDT	SANDBUSD	SANDUSD	SANDUSDT
SANTOSUSDT	SAPIENUSDT	SCRTUSDT	SCRUSDT	SCUSDT	SEIUSDT
SFPUSDT	SHELLUSDT	SIGNUSDT	SIRENUSDT	SKATEUSDT	SKLUSDT
SKYAIUSDT	SKYUSDT	SLERFUSDT	SLPUSDT	SNTUSDT	SNXUSDT
SOLBUSD	SOLUSD	SOLUSDC	SOLUSDT	SOLVUSDT	SOMIUSDT
SONICUSDT	SOONUSDT	SOPHUSDT	SPELLUSDT	SPKUSDT	SPXUSDT
SQDUSDT	SRMUSDT	SSVUSDT	STBLUSDT	STEEMUSDT	STGUSDT

STMXUSDT	STORJUSDT	STOUSDT	STPTUSDT	STRAXUSDT	STRKUSDT
STXUSDT	SUIUSD	SUIUSDC	SUIUSDT	SUNUSDT	SUPERUSDT
SUSDT	SUSHIUSDT	SWARMSUSDT	SWELLUSDT	SXPUSDT	SXTUSDT
SYNUSDT	SYRUPUSDT	SYSUSDT	TACUSDT	TAGUSDT	TAIKOUSDT
TAKEUSDT	TANSSIUSDT	TAOUSDT	TAUSDT	THETAUSD	THETAUSDT
THEUSDT	TIAUSDC	TIAUSDT	TLMBUSD	TLMUSDT	TNSRUSDT
TOKENUSDT	TOMOUSDT	TONUSDT	TOSHIUSDT	TOWNSUSDT	TRADOORUSDT
TRBUSDT	TREEUSDT	TROYUSDT	TRUMPUSDC	TRUMPUSDT	TRUTHUSDT
TRUUSDT	TRXBUSD	TRXUSD	TRXUSDT	TSTUSDT	TURBOUSDT
TUSDT	TUTUSDT	TWTUSDT	UBUSDT	UMAUSDT	UNFIUSDT
UNIBUSD	UNIUSD	UNIUSDC	UNIUSDT	USDCUSDT	USELESSUSDT
USTCUSDT	USUALUSDT	UXLINKUSDT	VANAUSDT	VANRYUSDT	VELODROMEUSDT
VELVETUSDT	VETUSD	VETUSDT	VFYUSDT	VICUSDT	VIDTUSDT
VINEUSDT	VIRTUALUSDT	VOXELUSDT	VTHOUSDT	VVVUSDT	WALUSDT
WAVESBUSD	WAVESUSDT	WAXPUSDT	WCTUSDT	WIFUSD	WIFUSDC
WIFUSDT	WLDUSD	WLDUSDC	WLDUSDT	WLFIUSDC	WLFIUSDT
WOOUSDT	WUSDT	XAIUSDT	XANUSDT	XCNUSDT	XEMUSDT
XLMUSD	XLMUSDT	XMRUSD	XMRUSDT	XNYUSDT	XPINUSDT
XPLUSDT	XRPBUSD	XRPUSD	XRPUSDC	XRPUSDT	XTZUSD
XTZUSDT	XVGUSDT	XVSUSDT	YALAUSDT	YBUSDT	YFIUSDT
YGGUSDT	ZBTUSDT	ZECUSDT	ZENUSDT	ZEREBROUSDT	ZETAUSDT
ZILUSD	ZILUSDT	ZKCUSDT	ZKJUSDT	ZKUSDT	ZORAUSDT
ZRCUSDT	ZROUSDT	ZRXUSDT	[illegible]USDT		

Example Applications

The Binance Crypto Future Margin Rate dataset enables correct margin cost so you can accurately design strategies for Cryptocurrencies with term structure. Examples include the following strategies:

- Horizontal/Diagonal arbitrage with the underlying cryptocurrencies
- Trade Contango/Backwardation predictions
- Hedge for illiquid cryptocurrencies

Classic Algorithm Example

The following example algorithm holds a long position in the BTCUSDT perpetual Future contract on the Binance exchange. When the margin interest rate is relatively low, it flips to a short position.

```py
from AlgorithmImports import *


class BinanceCryptoFutureDataAlgorithm(QCAlgorithm):

    # Add an indicator to track the low of trailing margin interest rates.
    _low_interest_rate = Minimum(10)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to USDT, since USD cannot be used to trade.
        self.set_cash("USDT", 100_000)
        # Binance Futures Exchange accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
        # Requesting data, we only trade on BTCUSDT Future in Binance Future exchange
        # Perpetual futures does not have a filter function.
        self._future = self.add_crypto_future("BTCUSDT", Resolution.DAILY)
        # Historical data
        history = self.history[MarginInterestRate](self._future.symbol, 10, Resolution.DAILY)
        for data_point in history:
            self._low_interest_rate.update(data_point.end_time, data_point.interest_rate)

    def on_data(self, slice: Slice) -> None:
        # Note that you may want to access the margin interest of the crypto future to calculate if it would impact a trade's PnL
        # Or you can calculate the trade size on keeping the quote currency constant
        if self._future.symbol not in slice.margin_interest_rates:
            return
        interest_rate = slice.margin_interest_rates[self._future.symbol].interest_rate
        self.plot('Margin Interest Rate', str(self._future.symbol), interest_rate)
        # Short when margin interest rates are relatively low. Otherwise, long.
        if interest_rate <= self._low_interest_rate.current.value:
            if not self._future.holdings.is_short:
                self.set_holdings(self._future.symbol, -0.5)
        elif not self._future.holdings.is_long:
            self.set_holdings(self._future.symbol, 1)
        # Update the indicator.
        self._low_interest_rate.update(self.time, interest_rate)
```

Framework Algorithm Example

The following example algorithm holds a long position in the BTCUSDT perpetual Future contract on the Binance exchange. When the margin interest rate is relatively low, it flips to a short position.

```py
from AlgorithmImports import *


class BinanceCryptoFutureDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.free_portfolio_value_percentage = 0.1
        # Set Account Currency to USDT, since USD cannot be used to trade.
        self.set_cash("USDT", 100000)
        # Binance Futures Exchange accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.leverage = 2
        # We only trade on BTCUSDT Future in Binance Future exchange
        symbols = Symbol.create("BTCUSDT", SecurityType.CRYPTO_FUTURE, Market.BINANCE)
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Custom alpha model to emit insights based on the Crypto Future price data
        self.add_alpha(CryptoFutureAlphaModel())
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class CryptoFutureAlphaModel(AlphaModel):

    _min_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []
        for symbol, low_interest_rate in self._min_by_symbol.items():
            # Note that you may want to access the margin interest of the crypto future to calculate if it would impact a trade's PnL
            # Or you can calculate the trade size on keeping the quote currency constant
            if symbol not in slice.margin_interest_rates:
                continue
            interest_rate = slice.margin_interest_rates[symbol].interest_rate
            algorithm.plot('Margin Interest Rate', str(symbol), interest_rate)
            # Short when margin interest rates are relatively low. Otherwise, long.
            if interest_rate <= low_interest_rate.current.value:
                direction = InsightDirection.DOWN
            else:
                direction = InsightDirection.UP
            insights.append(Insight.price(symbol, timedelta(1), direction))
            # Update the indicator.
            low_interest_rate.update(slice.time, interest_rate)
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            # Add an indicator to track the low of trailing margin interest rates.
            self._min_by_symbol[symbol] = Minimum(10)
            # Historical data
            history = algorithm.history[MarginInterestRate](symbol, 10, Resolution.DAILY)
            for data_point in history:
                self._min_by_symbol[symbol].update(data_point.end_time, data_point.interest_rate)
        for security in changes.removed_securities:
            self._min_by_symbol.pop(security.symbol, None)
```

Data Point Attributes

The Binance Crypto Future Margin Rate dataset provides `MarginInterestRates` objects with the following attributes:

QuantConnect

Bybit Crypto Future Margin Rate Data

Introduction

The Bybit Crypto Future Margin Rate Data by QuantConnect is for Cryptocurrency Futures margin interest data points. The data covers 631 Cryptocurrency pairs, starts in August 2020, and is delivered on a daily update frequency. This dataset is created by downloading data using Bybit API.

This dataset is an important companion to the Bybit Crypto Future Price Data dataset because it contains information on margin interest data to model margin costs.

For more information about the Bybit Crypto Future Margin Rate Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the Bybit Crypto Future Margin Rate dataset:

```
def Initialize(self) -> None:
    self.SetBrokerageModel(BrokerageName.Bybit, AccountType.Margin)

    self.crypto_future_symbol = self.AddCryptoFuture("BTCBUSD", Resolution.Minute).Symbol
```

The Bybit Crypto Future Margin Rate data is added to your algorithm along with the market data when you add a crypto future subscription.

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	August 2020
Asset Coverage	631 Crypto Futures Pairs
Data Density	Regular
Resolution	Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Bybit Crypto Future Margin Rate data to your algorithm, call the `add_crypto_future` method. Save a reference to the Crypto Future `Symbol` so you can access the data later in your algorithm.

```
from AlgorithmImports import *
from QuantConnect.DataSource import *

class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Tether
        self.set_account_currency("USDT", 100000)

        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        crypto_future = self.add_crypto_future("BTCUSDT", Resolution.MINUTE)
        # perpetual futures does not have a filter function
        self.btcusdt = crypto_future.symbol
```

For more information about creating Crypto Future subscriptions, see Requesting Data .

Accessing Data

To get the current Bybit Crypto Margin Rate data, index the `margin_interest_rates` property of the current `Slice` with the Crypto Future `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
        if self.btcusdt in slice.margin_interest_rates:
            interest_rate = slice.margin_interest_rates[self.btcusdt].interest_rate
            self.log(f"{self.btcusdt} close at {slice.time}: {interest_rate}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    for symbol, margin_interest_rate in slice.margin_interest_rates.items():
        interest_rate = margin_interest_rate.interest_rate
        self.log(f"{symbol} close at {slice.time}: {interest_rate}")
```

For more information about accessing Crypto Future data, see Handling Data .

Historical Data

To get historical Bybit Crypto Future Margin Rate data, call the `history` method with `MarginInterestRate` data type and the Crypto Future `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(MarginInterestRate, self.btcusdt, 100, Resolution.DAILY)

# MarginInterestRate objects
history = self.history[MarginInterestRate](self.btcusdt, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```
self.remove_security(self.btcusdt)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Crypto Future pairs:

Pairs Available (631)					
1CATUSDT	1INCHUSDT	A8USDT	AAVEUSD	AAVEUSDT	ACEUSDT
ACHUSDT	ACTUSDT	ACXUSDT	ADAUSD	ADAUSDT	AERGOUSDT
AEROUSDT	AEVOUSDT	AGIUSDT	AGIXUSDT	AGLDUSDT	AI16ZUSDT
AIDOGEUSDT	AIOZUSDT	AIUSDT	AIXBTUSDT	AKROUSDT	AKTUSDT
ALCHUSDT	ALEOUSDT	ALGOUSDT	ALICEUSDT	ALPACAUSDT	ALPHAUSDT
ALTUSDT	ALUUSDT	AMBUSDT	ANIMEUSDT	ANKRUSDT	ANTUSDT
APEUSDT	APEXUSDT	API3USDT	APTUSD	APTUSDT	APUUSDT
ARBUSDT	ARCUSDT	ARKMUSDT	ARKUSDT	ARPAUSDT	ARUSDT
ASTRUSDT	ATAUSDT	ATHUSDT	ATOMUSDT	AUCTIONUSDT	AUDIOUSDT
AVAAIUSDT	AVAILUSDT	AVAUSDT	AVAXUSD	AVAXUSDT	AVLUSDT
AXLUSDT	AXSUSDT	B3USDT	BABYDOGEUSDT	BABYUSDT	BADGERUSDT
BAKEUSDT	BALUSDT	BANANAS31USDT	BANANAUSDT	BANDUSDT	BANKUSDT
BANUSDT	BATUSDT	BBUSDT	BCHUSD	BCHUSDT	BEAMUSDT
BEERUSDT	BELUSDT	BENDOGUSDT	BERAUSDT	BICOUSDT	BIGTIMEUSDT
BILLYUSDT	BIOUSDT	BLASTUSDT	BLUEUSDT	BLURUSDT	BLZUSDT
BMTUSDT	BNBUSDT	BNTUSDT	BNXUSDT	BOBAUSDT	BOMEUSDT
BONDUSDT	BONKUSDT	BRETTUSDT	BROCCOLIUSDT	BRUSDT	BSVUSDT
BSWUSDT	BTCUSD	BTCUSDT	BTTUSDT	BUSDUSDT	BUZZUSDT
C98USDT	CAKEUSDT	CARVUSDT	CATIUSDT	CATSUSDT	CATUSDT
CEEKUSDT	CELOUSDT	CELRUSDT	CETUSUSDT	CFXUSDT	CGPTUSDT
CHEEMSUSDT	CHESSUSDT	CHILLGUYUSDT	CHRUSDT	CHZUSDT	CKBUSDT
CLANKERUSDT	CLOUDUSDT	COMBOUSDT	COMPUSDT	COOKIEUSDT	COOKUSDT
COQUSDT	COREUSDT	COSUSDT	COTIUSDT	COVALUSDT	COWUSDT
CPOOLUSDT	CROUSDT	CRVUSDT	CTCUSDT	CTKUSDT	CTSIUSDT
CVCUSDT	CVXUSDT	CYBERUSDT	DAOUSDT	DARKUSDT	DARUSDT
DASHUSDT	DATAUSDT	DBRUSDT	DEEPUSDT	DEGENUSDT	DENTUSDT
DEXEUSDT	DGBUSDT	DODOUSDT	DOGEUSD	DOGEUSDT	DOGSUSDT
DOGUSDT	DOODUSDT	DOP1USDT	DOTUSD	DOTUSDT	DRIFTUSDT
DUCKUSDT	DUSKUSDT	DYDXUSDT	DYMUSDT	EDUUSDT	EGLDUSDT
EIGENUSDT	ELONUSDT	ELXUSDT	ENAUSDT	ENJUSDT	ENSUSDT
EOSUSD	EOSUSDT	EPICUSDT	EPTUSDT	ETCUSD	ETCUSDT
ETHBTCUSDT	ETHFIUSDT	ETHUSD	ETHUSDT	ETHWUSDT	FARTCOINUSDT

FBUSDT	FDUSDUSDT	FETUSDT	FHEUSDT	FIDAUSDT	FILUSD
FILUSDT	FIOUSDT	FIREUSDT	FITFIUSDT	FLMUSDT	FLOCKUSDT
FLOKIUSDT	FLOWUSDT	FLRUSDT	FLUXUSDT	FORMUSDT	FORTHUSDT
FOXYUSDT	FRONTUSDT	FTMUSDT	FTNUSDT	FUELUSDT	FUNUSDT
FUSDT	FWOGUSDT	FXSUSDT	GALAUSDT	GALUSDT	GASUSDT
GEMSUSDT	GFTUSDT	GIGAUSDT	GLMRUSDT	GLMUSDT	GMEUSDT
GMTUSDT	GMXUSDT	GNOUSDT	GOATUSDT	GODSUSDT	GOMININGUSDT
GORKUSDT	GPSUSDT	GPTUSDT	GRASSUSDT	GRIFFAINUSDT	GRTUSDT
GTCUSDT	GUNUSDT	GUSDT	HAEDALUSDT	HBARUSDT	HEIUSDT
HFTUSDT	HIFIUSDT	HIGHUSDT	HIPPOUSDT	HIVEUSDT	HMSTRUSDT
HNTUSDT	HOOKUSDT	HOTUSDT	HPOS10IUSDT	HYPERUSDT	HYPEUSDT
ICPUSDT	ICXUSDT	IDEXUSDT	IDUSDT	ILVUSDT	IMXUSDT
INITUSDT	INJUSDT	IOSTUSDT	IOTAUSDT	IOTXUSDT	IOUSDT
IPUSDT	IQ50USDT	JASMYUSDT	JELLYJELLYUSDT	JOEUSDT	JSTUSDT
JTOUSDT	JUPUSDT	JUSDT	KAIAUSDT	KAITOUSDT	KASUSDT
KAVAUSDT	KDAUSDT	KERNELUSDT	KEYUSDT	KLAYUSDT	KMNOUSDT
KNCUSDT	KOMAUSDT	KSMUSDT	L3USDT	LADYSUSDT	LAIUSDT
LDOUSDT	LEVERUSDT	LINAUSDT	LINKUSD	LINKUSDT	LISTAUSDT
LITUSDT	LOOKSUSDT	LOOMUSDT	LPTUSDT	LQTYUSDT	LRCUSDT
LSKUSDT	LTCUSD	LTCUSDT	LTOUSDT	LUCEUSDT	LUMIAUSDT
LUNA2USDT	LUNCUSDT	MAGICUSDT	MAJORUSDT	MANAUSD	MANAUSDT
MANEKIUSDT	MANTAUSDT	MAPOUSDT	MASAUSDT	MASKUSDT	MATICUSDT
MAVIAUSDT	MAVUSDT	MAXUSDT	MBLUSDT	MBOXUSDT	MCUSDT
MDTUSDT	MELANIAUSDT	MEMEFIUSDT	MEMEUSDT	MERLUSDT	METISUSDT
MEUSDT	MEWUSDT	MICHIUSDT	MILKUSDT	MINAUSDT	MKRUSDT
MLNUSDT	MNTUSD	MNTUSDT	MOBILEUSDT	MOCAUSDT	MOGUSDT
MONUSDT	MOODENGUSDT	MORPHOUSDT	MOTHER1USDT	MOTHER2USDT	MOTHERUSDT
MOVEUSDT	MOVRUSDT	MTLUSDT	MUBARAKUSDT	MULTIUSDT	MUMUUSDT
MVLUSDT	MYRIAUSDT	MYROUSDT	NCUSDT	NEARUSD	NEARUSDT
NEIROCTOUSDT	NEIROETHUSDT	NEOUSDT	NFPUSDT	NFTUSDT	NILUSDT
NKNUSDT	NMRUSDT	NOTUSDT	NSUSDT	NTRNUSDT	NULSUSDT
NYANUSDT	OBOLUSDT	OBTUSDT	OCEANUSDT	OGNUSDT	OGUSDT
OKBUSDT	OLUSDT	OMGUSDT	OMNIUSDT	OMUSDT	ONDOUSDT

ONEUSDT	ONGUSDT	ONTUSDT	OPUSD	OPUSDT	ORBSUSDT
ORCAUSDT	ORDERUSDT	ORDIUSDT	ORNUSDT	OSMOUSDT	OXTUSDT
PARTIUSDT	PAXGUSDT	PEAQUSDT	PEIPEIUSDT	PENDLEUSDT	PENGUSDT
PENGUUSDT	PEOPLEUSDT	PEPEUSDT	PERPUSDT	PHAUSDT	PHBUSDT
PIPPINUSDT	PIRATEUSDT	PIXELUSDT	PIXFIUSDT	PLUMEUSDT	PNUTUSDT
POLUSDT	POLYXUSDT	PONKEUSDT	POPCATUSDT	PORTALUSDT	POWRUSDT
PRCLUSDT	PRE05EOSUSDT	PRE08EOSUSDT	PRIMEUSDT	PROMPTUSDT	PROMUSDT
PROSUSDT	PUFFERUSDT	PUMPUSDT	PUNDIXUSDT	PUNDUUSDT	PYRUSDT
PYTHUSDT	QIUSDT	QNTUSDT	QTUMUSDT	QUBICUSDT	QUICKUSDT
RADUSDT	RAREUSDT	RATSUSDT	RAYDIUMUSDT	RAYUSDT	RDNTUSDT
REDUSDT	REEFUSDT	RENDERUSDT	RENUSDT	REQUSDT	REXUSDT
REZUSDT	RFCUSDT	RIFSOLUSDT	RIFUSDT	RLCUSDT	RNDRUSDT
ROAMUSDT	RONINUSDT	RONUSDT	ROSEUSDT	RPLUSDT	RSRUSDT
RSS3USDT	RUNEUSDT	RVNUSDT	SAFEUSDT	SAGAUSDT	SANDUSDT
SATSUSDT	SCAUSDT	SCRTUSDT	SCRUSDT	SCUSDT	SDUSDT
SEIUSDT	SENDUSDT	SERAPHUSDT	SFPUSDT	SHELLUSDT	SHIB1000USDT
SIGNUSDT	SILLYUSDT	SIRENUSDT	SKLUSDT	SLERFUSDT	SLFUSDT
SLPUSDT	SNTUSDT	SNXUSDT	SOLAYERUSDT	SOLOUSDT	SOLUSD
SOLUSDT	SOLVUSDT	SONICUSDT	SPECUSDT	SPELLUSDT	SPXUSDT
SSVUSDT	STARLUSDT	STEEMUSDT	STGUSDT	STMXUSDT	STORJUSDT
STOUSDT	STPTUSDT	STRAXUSDT	STRKUSDT	STXUSDT	SUIUSD
SUIUSDT	SUNDOGUSDT	SUNUSDT	SUPERUSDT	SUSDT	SUSHIUSDT
SWARMSUSDT	SWEATUSDT	SWELLUSDT	SXPUSDT	SXTUSDT	SYNUSDT
SYSUSDT	TAIKOUSDT	TAIUSDT	TAOUSDT	TEST01SWELLUSDT	THETAUSDT
THEUSDT	TIAUSDT	TLMUSDT	TNSRUSDT	TOKENUSDT	TOMIUSDT
TOMOUSDT	TONUSD	TONUSDT	TOSHIUSDT	TRBUSDT	TROYUSDT
TRUMPUSDT	TRUUSDT	TRXUSDT	TSTBSCUSDT	TURBOUSDT	TUSDT
TUTUSDT	TWTUSDT	UMAUSDT	UNFIUSDT	UNIUSD	UNIUSDT
UROUSDT	USDCUSDT	USDEUSDT	USTCUSDT	USUALUSDT	UXLINKUSDT
VANAUSDT	VANRYUSDT	VELODROMEUSDT	VELOUSDT	VETUSDT	VGXUSDT
VICUSDT	VIDTUSDT	VINEUSDT	VINUUSDT	VIRTUALUSDT	VOXELUSDT
VRAUSDT	VRUSDT	VTHOUSDT	VVVUSDT	WALUSDT	WAVESUSDT

WAXPUSDT	WCTUSDT	WENUSDT	WHYUSDT	WIFUSD	WIFUSDT
WLDUSDT	WOOUSDT	WSMUSDT	WUSDT	XAIUSDT	XAUTUSDT
XCHUSDT	XCNUSDT	XDCUSDT	XECUSDT	XEMUSDT	XIONUSDT
XLMUSD	XLMUSDT	XMRUSDT	XNOUSDT	XRDUSDT	XRPUSD
XRPUSDT	XTERUSDT	XTZUSDT	XUSDT	XVGUSDT	XVSUSDT
YFIIUSDT	YFIUSDT	YGGUSDT	ZBCNUSDT	ZCXUSDT	ZECUSDT
ZENTUSDT	ZENUSDT	ZEREBROUSDT	ZETAUSDT	ZEUSUSDT	ZILUSDT
ZKFUSDT	ZKJUSDT	ZKUSDT	ZORAUSDT	ZRCUSDT	ZROUSDT
ZRXUSDT					

Example Applications

The Binance Crypto Future Margin Rate dataset enables correct margin cost so you can accurately design strategies for Cryptocurrencies with term structure. Examples include the following strategies:

- Horizontal/Diagonal arbitrage with the underlying Cryptocurrencies
- Trade Contango/Backwardation predictions
- Hedge for illiquid Cryptocurrencies

Classic Algorithm Example

The following example algorithm holds a long position in the BTCUSDT perpetual Future contract on the Bybit exchange. When the margin interest rate is relatively low, it flips to a short position.

```py
from AlgorithmImports import *


class BybitCryptoFutureDataAlgorithm(QCAlgorithm):

    # Add an indicator to track the low of trailing margin interest rates.
    _low_interest_rate = Minimum(10)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to USDT, since USD cannot be used to trade.
        self.set_cash("USDT", 100_000)
        # Bybit  accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Requesting data, we only trade on BTCUSDT Future in Bybit  exchange
        # Perpetual futures does not have a filter function.
        self._future = self.add_crypto_future("BTCUSDT", Resolution.DAILY)
        # Historical data
        history = self.history[MarginInterestRate](self._future.symbol, 10, Resolution.DAILY)
        for data_point in history:
            self._low_interest_rate.update(data_point.end_time, data_point.interest_rate)

    def on_data(self, slice: Slice) -> None:
        # Note that you may want to access the margin interest of the crypto future to calculate if it would impact a trade's PnL
        # Or you can calculate the trade size on keeping the quote currency constant
        if self._future.symbol not in slice.margin_interest_rates:
            return
        interest_rate = slice.margin_interest_rates[self._future.symbol].interest_rate
        self.plot('Margin Interest Rate', str(self._future.symbol), interest_rate)
        # Short when margin interest rates are relatively low. Otherwise, long.
        if interest_rate <= self._low_interest_rate.current.value:
            if not self._future.holdings.is_short:
                self.set_holdings(self._future.symbol, -0.5)
        elif not self._future.holdings.is_long:
            self.set_holdings(self._future.symbol, 1)
        # Update the indicator.
        self._low_interest_rate.update(self.time, interest_rate)
```

Framework Algorithm Example

The following example algorithm holds a long position in the BTCUSDT perpetual Future contract on the Bybit exchange. When the margin interest rate is relatively low, it flips to a short position.

```py
from AlgorithmImports import *


class BybitCryptoFutureDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.free_portfolio_value_percentage = 0.1
        # Set Account Currency to USDT, since USD cannot be used to trade.
        self.set_cash("USDT", 100000)
        # Bybit accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.leverage = 2
        # We only trade on BTCUSDT Future in Bybit exchange
        symbols = Symbol.create("BTCUSDT", SecurityType.CRYPTO_FUTURE, Market.BYBIT)
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Custom alpha model to emit insights based on the Crypto Future price data
        self.add_alpha(CryptoFutureAlphaModel())
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class CryptoFutureAlphaModel(AlphaModel):

    _min_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []
        for symbol, low_interest_rate in self._min_by_symbol.items():
            # Note that you may want to access the margin interest of the crypto future to calculate if it would impact a trade's PnL
            # Or you can calculate the trade size on keeping the quote currency constant
            if symbol not in slice.margin_interest_rates:
                continue
            interest_rate = slice.margin_interest_rates[symbol].interest_rate
            algorithm.plot('Margin Interest Rate', str(symbol), interest_rate)
            # Short when margin interest rates are relatively low. Otherwise, long.
            if interest_rate <= low_interest_rate.current.value:
                direction = InsightDirection.DOWN
            else:
                direction = InsightDirection.UP
            insights.append(Insight.price(symbol, timedelta(1), direction))
            # Update the indicator.
            low_interest_rate.update(slice.time, interest_rate)
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            # Add an indicator to track the low of trailing margin interest rates.
            self._min_by_symbol[symbol] = Minimum(10)
            # Historical data
            history = algorithm.history[MarginInterestRate](symbol, 10, Resolution.DAILY)
            for data_point in history:
                self._min_by_symbol[symbol].update(data_point.end_time, data_point.interest_rate)
        for security in changes.removed_securities:
            self._min_by_symbol.pop(security.symbol, None)
```

Data Point Attributes

The Binance Crypto Future Margin Rate dataset provides `MarginInterestRate` objects, which have the following attributes:

QuantConnect

CFD Data

Introduction

The CFD Data by QuantConnect serves 51 contracts for differences (CFD). The data starts as early as May 2002 and is delivered on any frequency from tick to daily. This dataset is created by QuantConnect processing raw tick data from OANDA.

CFD data does not include ask and bid sizes.

For more information about the CFD Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the CFD dataset:

```py
self.xauusd = self.add_cfd("XAUUSD", Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	Mixed, earliest starts May 2002
Asset Coverage	51 Contracts
Data Density	Dense
Resolution	Tick, Second, Minute, Hour, & Daily
Timezone	Mixed, in which the contract is listed*
Market Hours	Always Open , except from Friday 5 PM EST to Sunday 5 PM EST. Index CFDs depends on the underlying market hour*

* E.g.: DE30EUR tracks DAX30 Index, which is listed in Europe/Berlin timezone.

Requesting Data

To add CFD data to your algorithm, call the `add_cfd` method. Save a reference to the CFD `Symbol` so you can access the data later in your algorithm.

```py
class CfdAlgorithm (QCAlgorithm):
    def initialize(self) -> None:
        self.set_account_currency('EUR');

        self.set_start_date(2019, 2, 20)
        self.set_end_date(2019, 2, 21)
        self.set_cash('EUR', 100000)

        self.de30eur = self.add_cfd('DE30EUR').symbol

        self.set_benchmark(self.de30eur)
```

For more information about creating CFD subscriptions, see Requesting Data .

Accessing Data

To get the current CFD data, index the `quote_bars` , or `ticks` properties of the current `Slice` with the CFD `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.de30eur in slice.quote_bars:
        quote_bar = slice.quote_bars[self.de30eur]
        self.log(f"{self.de30eur} bid at {slice.time}: {quote_bar.bid.close}")

    if self.de30eur in slice.ticks:
        ticks = slice.ticks[self.de30eur]
        for tick in ticks:
            self.log(f"{self.de30eur} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing CFD data, see Handling Data .

Historical Data

To get historical CFD data, call the `history` method with the CFD `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.de30eur, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.de30eur, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.de30eur, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a CFD subscription, call the `remove_security` method.

```
PY
self.remove_security(self.de30eur)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings.

Supported Assets

The following table shows the available contracts:

Contract Available (51)					
AU200AUD	BCOUSD	CH20CHF	CORNUSD	DE10YBEUR	DE30EUR
EU50EUR	FR40EUR	HK33HKD	JP225USD	NAS100USD	NATGASUSD
NL25EUR	SG30SGD	SOYBNUSD	SPX500USD	SUGARUSD	UK100GBP
UK10YBGBP	US2000USD	US30USD	USB02YUSD	USB05YUSD	USB10YUSD
USB30YUSD	WHEATUSD	WTICOUSD	XAGAUD	XAGCAD	XAGCHF
XAGEUR	XAGGBP	XAGHKD	XAGJPY	XAGNZD	XAGSGD
XAGUSD	XAUAUD	XAUCAD	XAUCHF	XAUEUR	XAUGBP
XAUHKD	XAUJPY	XAUNZD	XAUSGD	XAUUSD	XAUXAG
XCUUSD	XPDUSD	XPTUSD			

Example Applications

The CFD price data enables you to trade CFD assets in your algorithm. Examples include the following strategies:

- Exploring the daily worldwide news cycles with CFDs that track international indices.
- Trading price movements of commodities with no delivery of physical goods. For example, pairs trading between gold and silver, corn and wheat, brent and crude oil, etc.

Classic Algorithm Example

The following example algorithm implements a pairs trading strategy using Gold and Silver CFDs, XAUUSD and XAGUSD, respectively. When the spread is higher than one standard deviation above its mean, the algorithm buys the spread (buy XAUUSD and sell XAGUSD). When the spread is lower than one standard deviation below its mean, it sells the spread (buy XAGUSD and sell XAUUSD).

```py
from AlgorithmImports import *


class SMAPairsTrading(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request gold and sliver spot CFDs for trading their spread difference, assuming their spread series is cointegrated
        self.add_cfd('XAUUSD', Resolution.HOUR)
        self.add_cfd('XAGUSD', Resolution.HOUR)

        # Use 500-step mean and SD indicator on determine the spread relative difference for trading signal generation
        self.pair = [ ]
        self.spread_mean = SimpleMovingAverage(500)
        self.spread_std = StandardDeviation(500)

    def on_data(self, slice: Slice) -> None:
        # Update the indicator with updated spread difference, such that the an updated cointegration threshold is calculated for trade
inception
        spread = self.pair[1].price - self.pair[0].price
        self.spread_mean.update(self.time, spread)
        self.spread_std.update(self.time, spread)

        spread_mean = self.spread_mean.current.value
        upperthreshold = spread_mean  + self.spread_std.current.value
        lowerthreshold = spread_mean  - self.spread_std.current.value

        # If the spread is higher than upper threshold, bet their spread series will revert to mean
        if spread > upperthreshold:
            self.set_holdings(self.pair[0].symbol, 1)
            self.set_holdings(self.pair[1].symbol, -1)
        elif spread < lowerthreshold:
            self.set_holdings(self.pair[0].symbol, -1)
            self.set_holdings(self.pair[1].symbol, 1)
        # Close positions if mean reverted
        elif (self.portfolio[self.pair[0].symbol].quantity > 0 and spread < spread_mean)\
        or (self.portfolio[self.pair[0].symbol].quantity < 0 and spread > spread_mean):
            self.liquidate()

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self.pair = [x for x in changes.added_securities]

        #1. Call for 500 bars of history data for each symbol in the pair and save to the variable history
        history = self.history([x.symbol for x in self.pair], 500)
        #2. Unstack the Pandas data frame to reduce it to the history close price
        history = history.close.unstack(level=0)
        #3. Iterate through the history tuple and update the mean and standard deviation with historical data
        for tuple in history.itertuples():
            self.spread_mean.update(tuple[0], tuple[2]-tuple[1])
            self.spread_std.update(tuple[0], tuple[2]-tuple[1])
```

Framework Algorithm Example

The following example algorithm implements a pairs trading strategy using Gold and Silver CFDs, XAUUSD and XAGUSD, respectively. When the spread is higher than one standard deviation above its mean, the algorithm buys the spread (buy XAUUSD and sell XAGUSD). When the spread is lower than one standard deviation below its mean, it sells the spread (buy XAGUSD and sell XAUUSD).

```py
from AlgorithmImports import *


class GoldSilverPairsTradingAlgorithm (QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.HOUR
        # Custom universe contains only gold and sliver spot CFDs for trading their spread difference, assuming their spread series is
cointegrated
        self.set_universe_selection(ManualUniverseSelectionModel
        (
            [ Symbol.create(x, SecurityType.CFD, Market.OANDA) for x in ["XAUUSD", "XAGUSD"] ]
        ))
        # Custom alpha model to emit trade insights based on the gold-sliver price spread
        self.add_alpha(PairsTradingAlphaModel())
        # Equal weighting trades assuming the spread is cointegrated by 1:1 ratio
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

class PairsTradingAlphaModel(AlphaModel):

    def __init__(self) -> None:
        # Use 500-step mean and SD indicator on determine the spread relative difference for trading signal generation
        self.pair = [ ]
        self.spread_mean = SimpleMovingAverage(500)
        self.spread_std = StandardDeviation(500)
        # Assume efficient mean reversal happens within 2 hours
        self.period = timedelta(hours=2)

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Update the indicator with updated spread difference, such that the an updated cointegration threshold is calculated for trade
inception
        spread = self.pair[1].price - self.pair[0].price
        self.spread_mean.update(algorithm.time, spread)
        self.spread_std.update(algorithm.time, spread)

        upperthreshold = self.spread_mean.current.value + self.spread_std.current.value
        lowerthreshold = self.spread_mean.current.value - self.spread_std.current.value

        # If the spread is higher than upper threshold, bet their spread series will revert to mean
        if spread > upperthreshold:
            return Insight.group(
                [
                    Insight.price(self.pair[0].symbol, self.period, InsightDirection.UP),
                    Insight.price(self.pair[1].symbol, self.period, InsightDirection.DOWN)
                ])
        elif spread < lowerthreshold:
            return Insight.group(
                [
                    Insight.price(self.pair[0].symbol, self.period, InsightDirection.DOWN),
                    Insight.price(self.pair[1].symbol, self.period, InsightDirection.UP)
                ])

        return []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        self.pair = [x for x in changes.added_securities]

        #1. Call for 500 bars of history data for each symbol in the pair and save to the variable history
        history = algorithm.history([x.symbol for x in self.pair], 500)
        #2. Unstack the Pandas data frame to reduce it to the history close price
        history = history.close.unstack(level=0)
        #3. Iterate through the history tuple and update the mean and standard deviation with historical data
        for tuple in history.itertuples():
            self.spread_mean.update(tuple[0], tuple[2]-tuple[1])
            self.spread_std.update(tuple[0], tuple[2]-tuple[1])
```

Data Point Attributes

The CFD dataset provides `QuoteBar` and `Tick` objects.

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

QuantConnect

Cash Indices

Introduction

The Cash Indices dataset by QuantConnect covers 125 US Indices and 3 International indices. The data starts on various dates from January 1998 and is delivered on any frequency from minute to daily.

For more information about the Cash Indices dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Live Data

The Cash Indices dataset does not provide a live data feed. To receive Cash Indices' data in your live algorithm, you must add a brokerage or a third-party data provider (see Cloud Platform > Datasets).

Getting Started

The following snippet demonstrates how to request data from the Cash Indices dataset:

```
self.vix = self.add_index("VIX", Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Coverage	125 US Indices and 3 International indices (HSI, SX5E and N225)
Data Density	Dense
Resolution	Minute, Hour, & Daily
Timezone	New York for US Indices, refer to `Supported Assets` below for International indices
Market Hours	Regular Only

Requesting Data

To add Cash Indices data to your algorithm, call the `add_index` method. Save a reference to the Index `Symbol` so you can access the data later in your algorithm.

```
class CashIndexAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        self.vix = self.add_index("VIX").symbol
```

For more information about creating Index subscriptions, see Requesting Data .

Accessing Data

To get the current Cash Indices data, index the `bars` property of the current `Slice` with the Index `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.vix in slice.bars:
        trade_bar = slice.bars[self.vix]
        self.log(f"{self.vix} close at {slice.time}: {trade_bar.close}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")
```

For more information about accessing Index data, see Handling Data .

Historical Data

To get historical Cash Indices data, call the `history` method with the Index `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.vix, 100, Resolution.DAILY)

# TradeBar objects
history_bars = self.history[TradeBar](self.vix, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove an Index subscription, call the `remove_security` method.

```
PY
self.remove_security(self.vix)
```

Supported Indices

US Cash Indices

The following table shows the US Cash indices with Index Option support:

Ticker	Index	Expiry	Start Date
VIX	S&P500	30 Days	Jul 2003
SPX	S&P500	-	Jan 1998
NDX	NASDAQ-100	-	Jan 1998
RUT	Russell 2000	-	Jan 2008

VIX - CBOE Volatility Index

The Cboe Volatility Index (VIX) is a real-time index that represents the market's expectations for the relative strength of near-term price changes of the S&P 500 Index (SPX). Because it's derived from the prices of SPX index Options with near-term expiration dates, it generates a 30-day forward projection of volatility. Volatility, or how fast prices change, is often seen as a way to gauge market sentiment, and in particular, the degree of fear

among market participants.

SPX - S&P 500 Index

The S&P 500 Index, or the Standard & Poor's 500 Index, is a market-capitalization-weighted index of the 500 largest publicly-traded companies in the U.S. It is not an exact list of the top 500 U.S. companies by market capitalization because there are other criteria included in the index. The index is widely regarded as the best gauge of large-cap U.S. Equities.

NDX - Nasdaq 100 Index

The Nasdaq-100 Index is a modified market-capitalization-weighted index composed of securities issued by 100 of the largest non-financial companies listed on the Nasdaq Stock Market (Nasdaq). The index includes companies from various industries except for the financial industry, like commercial and investment banks. These non-financial sectors include retail, biotechnology, industrial, technology, health care, and others.

RUT - Russell 2000 Index

The Russell 2000 Index is a small-cap U.S. stock market index that makes up the smallest 2,000 stocks in the Russell Index. It was started by the Frank Russell Company in 1984. The index is maintained by FTSE Russell, a subsidiary of the London Stock Exchange Group (LSEG).

All US Cash Indices

The following table shows all US Cash indices:

Indices Available (125). Note: QuantConnect is working on its modeling: market hours and symbol properties.					
AEX	ASX	BKX	BRR	BRTI	BXD
BXM	BXN	BXR	CEX	CLL	COMP
COR1M	COR1Y	COR30D	COR3M	COR6M	COR9M
CXU	DAX	DJCIAGC	DJCICC	DJCIEN	DJCIGC
DJCIGR	DJCIIK	DJCIKC	DJCISB	DJCISI	DJI
DJINET	DJR	DJTTR	DJX	DRG	DUX
DVS	DWCF	DWCPF	DWLG	DWLV	DXL
DXY	EVZ	FVX	GVZ	HGX	LOVOL
MID	MIDG	MIDV	MRUT	NDX	NQX
NYA	NYFANG	NYXBT	OEX	OSX	OVX
PUT	REIT	RUA	RUI	RUT	RVX
RXM	SET	SGX	SKEW	SMILE	SMLG
SP500LVOL	SP600	SPEN	SPGSCI	SPMPG	SPMPV
SPRI	SPRO	SPSIBI	SPSPG	SPSPV	SPSV
SPTMI	SPX	SPXPG	SPXPV	SVX	TNX
TYX	UKX	UTIL	UTY	VIF	VIN
VIX	VIX1D	VIX1Y	VIX3M	VIX6M	VIX9D
VOLI	VPD	VPN	VVIX	VWA	VWB
VXD	VXN	VXO	VXSLV	VXTH	VXTLT
W5000	WPUT	XAU	XAX	XDA	XDB
XEO	XMI	XNDX	XSP	XSR	

International Cash Indices

The following table shows the International Cash indices:

Ticker	Index	Start Date	Time Zone	Currency
HSI	Hang Seng Index	Dec 2006	Asia/Hong Kong	HKD
SX5E	EUREX EU STOXX	Jul 2003	Europe/Berlin	EUR
N225	Nikkei 225	Jul 2003	Asia/Tokyo	JPY

Example Applications

The Cash Indices enable you to incorporate popular indices into your trading algorithms. Examples include the following use cases:

- Exploring the difference between the Index and the ETF that tracks it
- Using these indices as the underlying asset fo Index Options strategies
- Understanding the stock market's level of expected forward-looking volatility, also known as the "fear index". When the VIX starts moving higher, it tells you that traders are getting nervous. When the VIX starts moving lower, it tells you that traders are gaining confidence.

Classic Algorithm Example

The following example algorithm tracks the 80-day EMA and 200-day EMA of SPX. When the short EMA crosses above the long EMA, the algorithm buys SPY. Otherwise, it holds cash.

```py
from AlgorithmImports import *


class IndexDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request SPY data as a trading vehicle for SPX
        self.spy = self.add_equity("SPY").symbol

        # Request SPX data for trade signal generation
        spx = self.add_index("SPX").symbol

        # Create short and long-term EMA indicators for trend estimation to trade
        self.ema_fast = self.EMA(spx, 80, Resolution.DAILY)
        self.ema_slow = self.EMA(spx, 200, Resolution.DAILY)
        self.set_warm_up(200, Resolution.DAILY)

        # Historical data
        history = self.history(spx, 60, Resolution.DAILY)
        self.debug(f'We got {len(history.index)} items from our history request')

    def on_data(self, slice: Slice) -> None:
        # Trade signals required indicators to be ready
        if self.is_warming_up or not self.ema_slow.is_ready:
            return

        # If short-term EMA is above long-term, it indicates an up trend, so we buy SPY
        if not self.portfolio.invested and self.ema_fast > self.ema_slow:
            self.set_holdings(self.spy, 1)
        # If it is the reverse, it indicates a downtrend, and we liquidate any position
        elif self.ema_fast < self.ema_slow:
            self.liquidate()
```

Framework Algorithm Example

The following example algorithm tracks the 80-day EMA and 200-day EMA of SPX. When the short EMA crosses above the long EMA, the algorithm buys SPY. Otherwise, it holds cash.

```
PY
```

```
from AlgorithmImports import *


class IndexDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Universe contains only SPY as a trading vehicle for SPX
        self.set_universe_selection(ManualUniverseSelectionModel
        (
            Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        ))
        # Custom alpha model that emits insights based on SPX index data
        self.set_alpha(SpxEmaCrossAlphaModel(self))
        # Equally investing can dissipate non-systematic risky event's capital concentration risk evenly
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_MONTH))


class SpxEmaCrossAlphaModel(AlphaModel):

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Request SPX data for trade signal generation
        spx = algorithm.add_index("SPX").symbol

        # Create short and long-term EMA indicators for trend estimation to trade
        self.ema_fast = algorithm.EMA(spx, 80, Resolution.DAILY)
        self.ema_slow = algorithm.EMA(spx, 200, Resolution.DAILY)
        algorithm.set_warm_up(200, Resolution.DAILY)

        # Historical data
        history = algorithm.history(spx, 60, Resolution.DAILY)
        algorithm.debug(f'We got {len(history.index)} items from our history request')

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Trade signals required indicators to be ready
        if algorithm.is_warming_up or not self.ema_slow.is_ready:
            return []

        # If short-term EMA is above long-term, it indicates an up trend, so we buy SPY
        if not algorithm.portfolio.invested and self.ema_fast > self.ema_slow:
            return [Insight.price(kvp.key, Expiry.END_OF_MONTH, InsightDirection.UP)
                for kvp in algorithm.active_securities if kvp.value.is_tradable]
        # If it is the reverse, it indicates downtrend, and we liquidate any position
        elif self.ema_fast < self.ema_slow:
            return [Insight.price(kvp.key, Expiry.END_OF_MONTH, InsightDirection.FLAT)
                for kvp in algorithm.active_securities if kvp.value.is_tradable]

        return []
```

Data Point Attributes

The Cash Indices dataset provides `TradeBar` object.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuantConnect

FOREX Data

Introduction

The FOREX Data by QuantConnect serves 71 foreign exchange (FOREX) pairs, starts on various dates from January 2007, and is delivered on any frequency from tick to daily. This dataset is created by QuantConnect processing raw tick data from OANDA.

FOREX data does not include ask and bid sizes.

For more information about the FOREX Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the FOREX dataset:

```
self.eurusd = self.add_forex("EURUSD", Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2007
Asset Coverage	71 Currency pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hour, & Daily
Timezone	UTC
Market Hours	Always Open , except from Friday 5 PM EST to Sunday 5 PM EST

Requesting Data

To add FOREX data to your algorithm, call the `add_forex` method. Save a reference to the Forex `Symbol` so you can access the data later in your algorithm.

```
class ForexAlgorithm (QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 2, 20)
        self.set_end_date(2019, 2, 21)
        self.set_cash(100000)

        self.eurusd = self.add_forex('EURUSD', Resolution.MINUTE).symbol

        self.set_benchmark(self.eurusd)
```

For more information about creating Forex subscriptions, see Requesting Data .

Accessing Data

To get the current Forex data, index the `quote_bars` , or `ticks` properties of the current `Slice` with the Forex `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if self.eurusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.eurusd]
        self.log(f"{self.eurusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.eurusd in slice.ticks:
        ticks = slice.ticks[self.eurusd]
        for tick in ticks:
            self.log(f"{self.eurusd} price at {slice.time}: {tick.price}")
```

PY

You can also iterate through all of the data objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

PY

For more information about accessing Forex data, see Handling Data .

Historical Data

To get historical Forex data, call the `history` method with the Forex `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.eurusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.eurusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.eurusd, timedelta(seconds=10), Resolution.TICK)
```

PY

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a Forex pair subscription, call the `remove_security` method.

```
self.remove_security(self.eurusd)
```

PY

The `remove_security` method cancels your open orders for the security and liquidates your holdings.

Supported Assets

The following table shows the available Forex pairs:

Pairs Available (71)					
AUDCAD	AUDCHF	AUDHKD	AUDJPY	AUDNZD	AUDSGD
AUDUSD	CADCHF	CADHKD	CADJPY	CADSGD	CHFHKD
CHFJPY	CHFZAR	EURAUD	EURCAD	EURCHF	EURCZK
EURDKK	EURGBP	EURHKD	EURHUF	EURJPY	EURNOK
EURNZD	EURPLN	EURSEK	EURSGD	EURTRY	EURUSD
EURZAR	GBPAUD	GBPCAD	GBPCHF	GBPHKD	GBPJPY
GBPNZD	GBPPLN	GBPSGD	GBPUSD	GBPZAR	HKDJPY
NZDCAD	NZDCHF	NZDHKD	NZDJPY	NZDSGD	NZDUSD
SGDCHF	SGDHKD	SGDJPY	TRYJPY	USDCAD	USDCHF
USDCNH	USDCZK	USDDKK	USDHKD	USDHUF	USDINR
USDJPY	USDMXN	USDNOK	USDPLN	USDSAR	USDSEK
USDSGD	USDTHB	USDTRY	USDZAR	ZARJPY	

Example Applications

The FOREX price data enables you to trade currency pairs in the global market. Examples include the following strategies:

- Exploring the impact that daily worldwide news cycles have on international currencies
- Carry Trade: borrow from a lower interest currency pair to fund the purchase of a currency pair with a higher interest rate

Classic Algorithm Example

The following example algorithm implements a FOREX carry trade. It buys the FOREX pair of the country with the lowest interest rate and sells the FOREX pair of the country with the highest interest rate.

```py
from AlgorithmImports import *


class ForexCarryTradeAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(25000)

        # We will use hard-coded interest rates for each base currency
        self.rates = {
            "USDAUD": 1.5,    # Australia
            "USDCAD": 0.5,    # Canada
            "USDCNY": 4.35,   # China
            "USDEUR": 0.0,    # Euro Area
            "USDINR": 6.5,    # India
            "USDJPY": -0.1,   # Japan
            "USDMXN": 4.25,   # Mexico
            "USDTRY": 7.5,    # Turkey
            "USDZAR": 7.0     # South Africa
        }

        # Subscribe to forex data for trading
        for ticker in self.rates:
            self.add_forex(ticker, Resolution.DAILY)

        # Use a month counter as variable to control rebalancing
        self.month = -1

    def on_data(self, slice: Slice) -> None:
        # Monthly rebalance checker
        if self.month == self.time.month:
            return
        self.month = self.time.month

        # Long the pair with highest interest rate and sell the pair with the lowest to earn the max monetary inflation difference
between the two
        sorted_rates = sorted(self.rates.items(), key = lambda x: x[1])
        self.set_holdings(sorted_rates[0][0], -0.5)
        self.set_holdings(sorted_rates[-1][0], 0.5)
```

Framework Algorithm Example

The following example algorithm implements a FOREX carry trade. It buys the FOREX pair of the country with the lowest interest rate and sells the FOREX pair of the country with the highest interest rate.

```py
from AlgorithmImports import *


class ForexCarryTradeAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(25000)

        def to_symbol(ticker: str) -> Symbol:
            return Symbol.create(ticker, SecurityType.FOREX, Market.OANDA)

        # We will use hard-coded interest rates for each base currency
        rates = {
            to_symbol("USDAUD"): 1.5,    # Australia
            to_symbol("USDCAD"): 0.5,    # Canada
            to_symbol("USDCNY"): 4.35,   # China
            to_symbol("USDEUR"): 0.0,    # Euro Area
            to_symbol("USDINR"): 6.5,    # India
            to_symbol("USDJPY"): -0.1,   # Japan
            to_symbol("USDMXN"): 4.25,   # Mexico
            to_symbol("USDTRY"): 7.5,    # Turkey
            to_symbol("USDZAR"): 7.0     # South Africa
        }

        self.set_universe_selection(ManualUniverseSelectionModel(list(rates.keys())))
        # A custom alpha model to emit insight according to interest rate
        self.set_alpha(IntestRatesAlphaModel(rates))
        # Equal size to capitalize the monetary value of quote currency only based on interest rate difference
        # For dollar-neutral to save margin cost
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_MONTH))

class IntestRatesAlphaModel(AlphaModel):
    def __init__(self, rates: float) -> None:
        self.rates = rates
        # Variable to control the rebalancing time
        self.month = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Monthly rebalance
        if self.month == algorithm.time.month:
            return []
        self.month = algorithm.time.month

        # # Long the pair with highest interest rate and sell the pair with the lowest to earn the max monetary inflation difference
between the two
        sorted_rates = sorted(self.rates.items(), key = lambda x: x[1])
        return Insight.group(
            Insight.price(sorted_rates[0][0], Expiry.END_OF_MONTH, InsightDirection.UP),
            Insight.price(sorted_rates[-1][0], Expiry.END_OF_MONTH, InsightDirection.DOWN)
        )
```

Data Point Attributes

The FOREX dataset provides `QuoteBar` and `Tick` objects.

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

QuantConnect

Fear and Greed

Introduction

The Fear and Greed dataset by QuantConnect provides data points to quantify the degree of fear and greed in the US Equity market. The data starts in 2014 and is delivered on a daily frequency, delayed by two days. This dataset is created by aggregating seven indicators, including market volatility, stock market breadth, and put-call ratios.

For more information about the Fear and Greed dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the Fear and Greed dataset:

```
PY
self._dataset_symbol = self.add_data(FearGreedIndex, 'FG').symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	July 2014
Data Density	Dense
Resolution	Daily
Timezone	New York

The latest daily data points are about two days since the VIX and SPX prices are inputs into the index and their latest daily prices are also two days old. New data is available at 9 AM EST.

Requesting Data

To add Fear and Greed data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class FearAndGreedDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dataset_symbol = self.add_data(FearGreedIndex, "FG").symbol
```

Accessing Data

To get the current Fear and Greed data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self._dataset_symbol):
        data_point = slice[self._dataset_symbol]
        self.log(f"{self._dataset_symbol} value at {slice.time}: {data_point.value}")
```

Historical Data

To get historical Fear and Greed data, call the `history` method with the dataset `Symbol`. If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self._dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history = self.history[FearGreedIndex](self._dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests.

Remove Subscription

To remove a subscription to the Fear and Greed dataset, call the `remove_security` method.

```
self.remove_security(self._dataset_symbol)
```

Example Applications

The Fear and Greed dataset quantifies the degree of fear and greed in the US Equity market. Example use cases include the following strategies:

- Accumulate long-term positions when the market is fearful (the index is low).
- Short risky assets when the market is greedy.
- Trade butterfly Option strategies based on the market volatility indicator.

Classic Algorithm Example

The following example algorithm buys SPY when the Fear and Greed index is less than 25 (extreme fear) and sells when it's above 75 (extreme greed).

```
from AlgorithmImports import *


class FearAndGreedExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Add the market index to trade.
        self._equity = self.add_equity('SPY', Resolution.DAILY)
        # Add the Fear and Greed dataset.
        self._dataset_symbol = self.add_data(FearGreedIndex, 'FG').symbol

    def on_data(self, data: Slice) -> None:
        # Wait until the Fear and Greed dataset has new data.
        if self._dataset_symbol not in data:
            return
        value = data[self._dataset_symbol].value
        # Buy when the market is fearful.
        if not self._equity.invested and value < 25:
            self.set_holdings(self._equity.symbol, 1)
        # Sell when the market is greedy.
        elif self._equity.invested and value > 75:
            self.liquidate()
```

Framework Example

The following example algorithm buys SPY when the Fear and Greed index is less than 25 (extreme fear) and sells when it's above 75 (extreme greed).

```py
from AlgorithmImports import *


class FearAndGreedExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Trade SPY on a daily basis.
        self.universe_settings.resolution = Resolution.DAILY
        symbols = [Symbol.create('SPY', SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Add a custom Alpha model that emits insights based on the Fear and Greed dataset.
        self.add_alpha(FearAndGreedAlphaModel(self))
        # Set the equal weighted PCM so the algorithm places trades.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


# Define the custom alpha model.
class FearAndGreedAlphaModel(AlphaModel):

    def __init__(self, algorithm):
        # Create a list to track the current universe.
        self._securities = []
        # Add the Fear and Greed dataset.
        self._dataset_symbol = algorithm.add_data(FearGreedIndex, 'FG').symbol

    def update(self, algorithm: QCAlgorithm, data: Slice) -> List[Insight]:
        # Wait until the Fear and Greed dataset has new data.
        if self._dataset_symbol not in data:
            return []
        # Wait until the Fear and Greed index is extreme.
        value = data[self._dataset_symbol].value
        if 25 <= value <= 75:
            return []
        insights = []
        for security in self._securities:
            # Buy when the market is fearful.
            if not security.invested and value < 25:
                direction = InsightDirection.UP
            # Sell when the market is greedy.
            elif security.invested and value > 75:
                direction = InsightDirection.FLAT
            else:
                continue
            insights.append(Insight.price(security.symbol, timedelta(2*365), direction))
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # When assets enter the universe, save them.
        for security in changes.added_securities:
            self._securities.append(security)
        # When assets leave the universe, remove them.
        for security in changes.removed_securities:
            if security in self._securities:
                self._securities.remove(security)
```

Data Point Attributes

The Fear and Greed dataset provides `FearGreedIndex` objects, which have the following attributes:

QuantConnect

International Future Universe

Introduction

The International Future Universe dataset by QuantConnect lists the available International Future contracts, their daily trading volume, and Open Interest. The data covers 4 contracts (DAX, FESX, HSI, and NKD), starts in July 1998, and is delivered on daily frequency. This dataset is created by monitoring the trading activity on the EUREX, HKFE, and CME.

This dataset depends on the US Futures Security Master dataset because the US Futures Security Master dataset contains information on symbol changes of the contracts.

This dataset `does not` contain market data. For market data, see International Futures by TickData and US Futures by AlgoSeek for NKD.

For more information about the International Future Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The International Futures Universe dataset provides data for contract filtering/selection:

```py
hsi = self.add_future(Futures.Indices.HANG_SENG, Resolution.MINUTE)          # "HSI"
hsi.set_filter(0, 90)
dax = self.add_future(Futures.Indices.DAX, Resolution.MINUTE)                # "DAX"
dax.set_filter(0, 90)
fesx = self.add_future(Futures.Indices.EURO_STOXX_50, Resolution.MINUTE)     # "FESX"
fesx.set_filter(0, 90)
nkd = self.add_future(Futures.Indices.NIKKEI_225_DOLLAR, Resolution.MINUTE)  # "NKD"
nkd.set_filter(0, 90)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	July 1998 (for details, see `Supported Assets`)
Coverage	4 Contracts
Data Density	Dense
Resolution	Daily
Timezone	Various (for details, see `Supported Assets`)
Market Hours	Regular and Extended

Requesting Data

To add International Future Universe data to your algorithm, call the `add_future` method. Save a reference to the Future object so you can access the data later in your algorithm. To define which contracts should be in your universe, specify the filter when requesting the Future data.

The `add_future` method provides a daily stream of Future chain data. To get the most recent daily chain, call the `futures_chain` method with the underlying Future Symbol. The `futures_chain` method returns data on all the tradable contracts, not just the contracts that pass your universe filter.

```py
class InternationalFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2013, 12, 20)
        self.set_end_date(2014, 2, 20)
        self.set_cash(1000000)
        self.universe_settings.asynchronous = True
        self._future = self.add_future(Futures.Indices.HANG_SENG)
        # Set our contract filter for this Future chain.
        self._future.set_filter(lambda universe: universe.standards_only().front_month())
        # Get the entire Futures chain for the current day.
        chain = self.futures_chain(self._future.symbol, flatten=True).data_frame
```

For more information about creating Future Universes, see Futures .

Accessing Data

For information about accessing International Future Universe data, see Futures .

Historical Data

You can get historical International Future Universe data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical International Future Universe data in an algorithm, call the `history` method with continuous Future `Symbol` and a lookback period. This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. If there is no data for the period you requested, the history result is empty.

```py
# Add the Future and save a reference to it.
future = self.add_future(Futures.Indices.HANG_SENG)

# DataFrame example where the columns are the FutureUniverse attributes:
history_df = self.history(FutureUniverse, future.symbol, 5, flatten=True)

# Series example where the values are lists of FutureUniverse objects:
history_series = self.history(FutureUniverse, future.symbol, 5, flatten=False)

# FutureUniverse objects example:
history = self.history[FutureUniverse](future.symbol, 5)
```

For more information about historical International Future Universe data in algorithms, see Contracts .

Historical Data In Research

To get historical International Future Universe data in the Research Environment, call the `history` method with continuous Future `Symbol` and a time period. This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. If there is no data for the period you requested, the history result is empty.

```py
# Add the Future and save a reference to it.
future = qb.add_future(Futures.Indices.HANG_SENG)

# DataFrame example where the columns are the FutureUniverse attributes:
history_df = qb.history(FutureUniverse, future.symbol, datetime(2025, 1, 1), datetime(2025, 4, 1), flatten=True)

# Series example where the values are lists of FutureUniverse objects:
history_series = qb.history(FutureUniverse, future.symbol, 5, flatten=False)

# FutureUniverse objects example:
history = qb.history[FutureUniverse](future.symbol, datetime(2025, 1, 1), datetime(2025, 4, 1))
```

For more information about historical International Future Universe data in the Research Environment, see Daily Prices History .

Supported Assets

The following table shows the available Futures:

Ticker	Future	Start Date	Time Zone	Currency
HSI	Hang Seng Index Futures	Jan 2010	Asia/Hong Kong	HKD
DAX	German Index Futures	Jan 2010	Europe/Berlin	EUR
FESX	EURO STOXX 50 Index Futures	Jul 1998	Europe/Berlin	EUR
NKD	Nikkei 225 Index Futures	Jan 2007	America/Chicago	USD

Example Applications

The International Futures Universe dataset enables you to design Futures strategies accurately. Examples include the following strategies:

- Buying the Futures contract with the most open interest to reduce slippage and market impact
- Trade speculation on an International Index
- Trading bull calendar spreads to reduce volatility and margin requirements

Classic Algorithm Example

The following example algorithm uses the ZigZag indicator to determine the trend of Hang Seng Index. It then trades the Index with HSI Futures.

```py
from AlgorithmImports import *


class InternationalFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the time zone to HKT to make it more comparable with the exchange.
        self.set_time_zone(TimeZones.HONG_KONG)
        # Set the account currency as HKD to trade HSI Futures.
        self.set_account_currency("HKD", 1000000)
        # Seed the last price of the contracts for filling.
        self.settings.seed_initial_prices = True

        # Request HSI Futures to trade.
        # Note that we will trade the contract with the highest open interest for liquidity.
        self.hsi_future = self.add_future(
            Futures.Indices.HANG_SENG,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.LAST_TRADING_DAY,
            contract_depth_offset=0
        )
        # Adds contracts that expiry within 90 days. We will trade the farthest contract
        self.hsi_future.set_filter(0,90)
        # Request the corresponding underlying Index for feeding indicator for trade signal generation.
        hsi_index = self.add_index("HSI").symbol

        # Create a ZigZag indicator to trade Hang Seng Index price pivot points.
        self._zz = self.zz(hsi_index, 0.15, 5, Resolution.DAILY)
        # Warm up indicator for immediate readiness to trade.
        self.warm_up_indicator(hsi_index, self._zz, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        # Only place trade if the Future contracts is in market opening hours to avoid stale fills.
        if self.is_market_open(self.hsi_future.symbol) and self._zz.is_ready:
            pivot = self._zz.pivot_type
            # If the last pivot point is a low point, the current trend is increasing after this low point.
            if pivot == PivotPointType.LOW:
                contracts = sorted([x.symbol for x in slice.future_chains.get(self.hsi_future.symbol)],
                    key=lambda x: x.id.date)
                self.set_holdings(contracts[-1], 0.2)
            # If the last pivot point is a high point, the current trend is decreasing after this high point.
            if pivot == PivotPointType.HIGH:
                contracts = sorted([x.symbol for x in slice.future_chains.get(self.hsi_future.symbol)],
                    key=lambda x: x.id.date)
                self.set_holdings(contracts[-1], -0.2)
```

Data Point Attributes

The International Future Universe dataset provides `FutureFilterUniverse` and `FutureUniverse` objects.

FutureFilterUniverse Attributes

`FutureFilterUniverse` objects have the following attributes:

FutureUniverse Attributes

`FutureUniverse` objects have the following attributes:

QuantConnect

US ETF Constituents

Introduction

The US ETF Constituents dataset by QuantConnect tracks the constituents and weighting of US Equities in 2,650 ETF listings. The data starts in June 2009 and is delivered on a daily basis (monthly basis before January 2015). This dataset is created by tracking the host ETF websites and can be delayed by up to 1 week.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US ETF Constituents dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 160,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US ETF Constituents dataset:

```
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    # Use the following method for a Classic Algorithm
    self._universe = self.add_universe(self.universe.etf("SPY", Market.USA, self.universe_settings, self.etf_constituents_filter))

    symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
    # Use the following method for a Framework Algorithm
    self.add_universe_selection(ETFConstituentsUniverseSelectionModel(symbol, self.universe_settings, self.etf_constituents_filter))

    def etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Add all Symbols of the ETFConstituentUniverse
       return [x.symbol for x in constituents]
```

PY

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	June 2009
Asset Coverage	2,650 US ETF Listings
Data Density	Dense
Resolution	Daily (Monthly before Jan 2015)
Timezone	New York

Requesting Data

To add US ETF Constituents data to your algorithm, call the `add_universe` and `universe.etf` methods. To select which constituents occupy the universe, provide the ETF `Symbol` and a selection function.

```
class ETFConstituentUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2018, 1, 1)
        self.set_end_date(2020, 8, 25)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        self._universe = self.add_universe(self.universe.etf("SPY", self.universe_settings, self.etf_constituents_filter))
```

For more information about universe settings, see Settings .

Accessing Data

To access the US ETF Constituent data, use the `ETFConstituentUniverse` objects in your selection function. The data is available in daily resolution. The `Symbol` objects you return from your selection function defines the universe constituents.

```
def etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
    for c in constituents:
        self.debug(f'{c.end_time} :: {c.last_update} :: {c.weight} :: {c.shares_held} :: {c.market_value}')
    return [x.symbol for x in constituents]
```

Historical Data

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the ETFConstituentUniverse attributes:
df_history = self.history(self.universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of ETFConstituentUniverse objects:
series_history = self.history(self.universe, 30, Resolution.DAILY)
for (universe_symbol, time), constituents in series_history.items():
    for constituent in constituents:
        self.log(f'{constituent.symbol} weight at {constituent.end_time}: {constituent.weight}')
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the ETFConstituentUniverse attributes:
df_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of ETFConstituentUniverse objects:
series_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), constituents in series_history.items():
    for constituent in constituents:
       print(f"{constituent.symbol} weight at {constituent.end_time}: {constituent.weight}")
```

You can call the `history` method in Research.

Supported ETFs

The following table shows the available ETFs:

ETF Tickers					
AADR	AAIT	AAWW	AAXJ	ABCS	ABEQ
ACCU	ACES	ACIM	ACIO	ACSG	ACSI

ACT	ACTV	ACWF	ACWI	ACWV	ACWX
ADFI	ADIV	ADME	ADRA	ADRD	ADRE
ADRU	AEMB	AESR	AFK	AFLG	AFMC
AFSM	AFTY	AGEM	AGG	AGGE	AGGH
AGGP	AGIH	AGII	AGLS	AGNG	AGOV
AGQ	AGRG	AGRH	AGT	AGZ	AHYB
AIA	AIEQ	AIIQ	AIQ	AIRR	ALFA
ALFI	ALTL	ALTS	ALTY	AMCA	AMER
AMID	AMLP	AMLX	AMOM	AMPS	AMZA
ANEW	ANGL	AOA	AOK	AOM	AOR
AOTG	APRZ	AQWA	ARCM	ARGT	ARKG
ARKK	ARKQ	ARKW	ARKX	ARVR	ASEA
ASET	ASHR	ASHX	AUGZ	AUSE	AUSF
AVDE	AVDV	AVEM	AVES	AVGE	AVIE
AVIV	AVLV	AVRE	AVSC	AVSD	AVSE
AVSU	AVUS	AVUV	AWAY	AXDI	AXEN
AXFN	AXHE	AXID	AXIT	AXJL	AXJS
AXJV	AXMT	AXSL	AXTE	AXUT	BAB
BABS	BATT	BBAG	BBAX	BBC	BBCA
BBCB	BBEU	BBH	BBHY	BBIN	BBLU
BBMC	BBP	BBRC	BBRE	BBSA	BBSC
BBUS	BCHP	BCNA	BDAT	BDH	BDRY
BECO	BEDZ	BEMO	BERN	BETZ	BFIT
BFOR	BFTR	BHH	BIB	BIBL	BICK
BIDS	BIGD	BIK	BIKR	BITQ	BITS
BIV	BIZD	BJK	BKCH	BKCI	BKEM
BKES	BKF	BKGI	BKIE	BKIS	BKLC
BKMC	BKSE	BKUS	BLCN	BLDG	BLES
BLHY	BLKC	BLLD	BLOK	BLV	BMED
BND	BNDC	BNDW	BNDX	BNE	BNGE
BOB	BOSS	BOTZ	BOUT	BPAY	BRF
BRGL	BRNY	BRZU	BSCB	BSCC	BSCD

BSCE	BSCF	BSCG	BSCH	BSCI	BSCJ
BSCK	BSEA	BSJC	BSJD	BSJE	BSJF
BSJG	BSJH	BSJI	BSJJ	BSV	BTAH
BTAL	BTEC	BTEK	BTHM	BUFD	BUFF
BUFG	BUFQ	BUFR	BUFT	BUG	BUL
BUYZ	BUZ	BUZZ	BWX	BWZ	BYLD
BYOB	BYTE	CACG	CALF	CARZ	CATH
CBOE	CBON	CCON	CCOR	CCXE	CDC
CDEI	CDL	CDX	CEFA	CEFS	CEMB
CEY	CEZ	CFA	CFCV	CFGE	CFO
CGDV	CGGO	CGGR	CGUS	CGW	CGXU
CHAU	CHB	CHEP	CHGX	CHIC	CHIH
CHII	CHIK	CHIL	CHIQ	CHIR	CHIS
CHIX	CHLD	CHNA	CIBR	CID	CIL
CIRC	CIU	CIZ	CLDL	CLIX	CLNR
CLOU	CLRG	CLSA	CLSC	CLSM	CLY
CLYH	CMBS	CMDT	CMF	CN	CNBS
CNCR	CNDA	CNDF	CNHX	CNRG	CNSF
CNTR	CNXT	COLX	COMB	COMG	COMT
CONG	COPX	COWZ	CPI	CQQQ	CRAK
CRBA	CRBI	CRBN	CRBQ	CRED	CRO
CROP	CRPT	CRUZ	CSA	CSB	CSD
CSF	CSJ	CSM	CSML	CTEC	CTEX
CTRU	CU	CUMB	CURE	CUT	CVIE
CVLC	CVMC	CVSE	CVY	CWEB	CWI
CWS	CXSE	CYA	CZA	DALI	DALT
DAPP	DAT	DAX	DBA	DBAP	DBAU
DBAW	DBB	DBBR	DBC	DBE	DBEF
DBEM	DBES	DBEU	DBEZ	DBGR	DBIF
DBIT	DBIZ	DBJP	DBKO	DBLV	DBMX
DBN	DBO	DBRE	DBSE	DBSP	DBU
DBUK	DDBI	DDEZ	DDIV	DDLS	DDM

DDWM	DEB	DECZ	DEEF	DEEP	DEF
DEFA	DEFN	DEHP	DEM	DEMG	DENT
DES	DESC	DEUS	DEW	DEWJ	DEZU
DFAC	DFAE	DFAI	DFAR	DFAS	DFAT
DFAU	DFAX	DFE	DFEM	DFEN	DFEV
DFGR	DFHY	DFIC	DFIS	DFIV	DFJ
DFLV	DFND	DFNL	DFNV	DFSE	DFSI
DFSU	DFSV	DFUS	DFUV	DGRE	DGRO
DGRS	DGRW	DGS	DGT	DHDG	DHS
DIA	DIEM	DIG	DIHP	DIM	DINT
DISV	DIV	DIVA	DIVB	DIVD	DIVI
DIVO	DIVS	DIVZ	DJD	DJIA	DKA
DLN	DLS	DMAT	DMDV	DMRE	DMRI
DMRL	DMRM	DMRS	DMXF	DND	DNH
DNL	DOG	DOL	DON	DOO	DOZR
DPK	DPST	DQML	DRIV	DRLL	DRN
DRSK	DRW	DSC	DSCF	DSG	DSI
DSPC	DSTL	DSTX	DSUM	DSV	DTD
DTEC	DTH	DTN	DTOX	DTRE	DUDE
DUHP	DURA	DUSA	DUSL	DVAL	DVEM
DVLU	DVOL	DVP	DVY	DVYA	DVYE
DWAQ	DWAS	DWAT	DWAW	DWCR	DWEQ
DWFI	DWIN	DWLD	DWLV	DWM	DWMC
DWPP	DWSH	DWTR	DWUS	DWX	DXD
DXGE	DXJ	DXJC	DXJF	DXJH	DXJR
DXJS	DXJT	DXKW	DXPS	DXUS	DYHG
DYLD	DYLS	DYNF	DZK	EAFD	EAOA
EAOK	EAOM	EAOR	EASG	EASI	EATZ
EBIZ	EBLU	EBND	ECH	ECLN	ECNS
ECON	ECOW	ECOZ	EDBI	EDC	EDEN
EDIV	EDOC	EDOG	EDOM	EDOW	EDUT
EEB	EEG	EEHB	EELV	EEM	EEMA
EEMD	EEME	EEML	EEMO	EEMS	EEMV

EEMX	EEN	EEO	EES	EEZ	EFA
EFAD	EFAV	EFAX	EFG	EFIV	EFN
EFNL	EFRA	EFV	EGIS	EGPT	EGRW
EGUS	EIDO	EINC	EIPX	EIRL	EIS
EKAR	EKG	EKH	ELG	ELR	ELV
EMAG	EMB	EMBB	EMBH	EMBU	EMCA
EMCD	EMCG	EMCH	EMCR	EMDD	EMDG
EMDI	EMDV	EMEM	EMER	EMEY	EMFM
EMFN	EMFQ	EMFT	EMG	EMGC	EMGD
EMGF	EMHD	EMHY	EMIF	EMLC	EMLP
EMM	EMMT	EMPW	EMQQ	EMRE	EMSG
EMSO	EMT	EMTL	EMV	EMXC	EMXF
EMZA	ENFR	ENGN	ENOR	ENRG	ENTR
ENY	ENZL	EOPS	EPHE	EPI	EPOL
EPP	EPRE	EPRF	EPRO	EPS	EPU
EQAL	EQIN	EQL	EQLT	EQRR	EQUL
EQWL	EQWM	EQWS	ERET	ERGF	ERM
ERSX	ERUS	ERX	ESG	ESGA	ESGD
ESGE	ESGF	ESGG	ESGL	ESGN	ESGS
ESGU	ESGV	ESGW	ESGY	ESIX	ESML
ESMV	ESNG	ESPO	ESR	ETHO	ETPA
EUDG	EUDV	EUFL	EUFN	EUMF	EUMV
EURL	EURZ	EUSA	EUSC	EUXL	EVAL
EVEN	EVX	EWA	EWAC	EWC	EWCO
EWD	EWEB	EWEF	EWEM	EWG	EWGS
EWH	EWHS	EWI	EWJ	EWK	EWL
EWM	EWMC	EWMD	EWN	EWO	EWP
EWQ	EWRE	EWRI	EWRM	EWRS	EWS
EWSC	EWSM	EWSS	EWT	EWU	EWUS
EWW	EWX	EWY	EWZ	EWZS	EXB
EXI	EXT	EYLD	EZA	EZJ	EZM
EZU	EZY	FAA	FAB	FAD	FAIL
FAN	FAS	FATT	FAUS	FBCG	FBCV

FBM	FBND	FBT	FBZ	FCA	FCAN
FCD	FCEF	FCG	FCHI	FCL	FCLD
FCOM	FCOR	FCPI	FCQ	FCTR	FCV
FCVT	FDD	FDEM	FDEV	FDG	FDHT
FDIG	FDIS	FDIV	FDL	FDLO	FDM
FDMO	FDN	FDNI	FDRR	FDRV	FDT
FDTS	FDV	FDVV	FDWM	FEBZ	FEDM
FEDX	FEEM	FEFN	FEG	FEHY	FEIG
FEM	FEMB	FEMS	FENY	FEP	FEUS
FEUZ	FEVR	FEX	FFHG	FFL	FFR
FFSG	FFTG	FFTI	FFTY	FGD	FGEM
FGM	FGRO	FHC	FHK	FHLC	FIDI
FIDU	FIL	FILL	FINU	FINX	FIO
FISR	FITE	FIVA	FIVG	FIW	FJP
FKL	FKO	FKU	FLAG	FLAU	FLAX
FLBR	FLCA	FLCH	FLCO	FLEE	FLEH
FLFR	FLG	FLGB	FLGR	FLHK	FLIN
FLIO	FLIY	FLJH	FLJP	FLKR	FLLA
FLLV	FLM	FLMB	FLMI	FLMX	FLN
FLOT	FLQD	FLQE	FLQG	FLQH	FLQL
FLQM	FLQS	FLRG	FLRT	FLRU	FLSA
FLSP	FLSW	FLTB	FLTR	FLTW	FLV
FLYT	FLZA	FM	FMAG	FMAT	FMB
FMET	FMF	FMIL	FMK	FMM	FMU
FMV	FNCF	FNCL	FNDA	FNDB	FNDC
FNDE	FNDF	FNDX	FNG	FNGG	FNI
FNIO	FNK	FNTC	FNX	FNY	FOC
FOMO	FONE	FOS	FOVL	FPA	FPE
FPFD	FPRO	FPX	FPXE	FPXI	FQAL
FRAK	FRDM	FREL	FRI	FRL	FRN
FRNW	FSBD	FSMD	FSMO	FSST	FSTA
FSYD	FSZ	FTA	FTAG	FTC	FTCS

FTEC	FTGC	FTGS	FTHI	FTLB	FTLS
FTQ	FTQI	FTRI	FTSL	FTSM	FTVA
FTW	FTXD	FTXG	FTXH	FTXL	FTXN
FTXO	FTXR	FTY	FUI	FUTY	FV
FVAL	FVC	FVD	FVI	FVL	FWDB
FWDD	FWDI	FXD	FXEU	FXG	FXH
FXI	FXL	FXN	FXO	FXR	FXU
FXZ	FYC	FYLG	FYT	FYX	FZB
GAA	GAF	GAL	GAMR	GARD	GASL
GASX	GBDV	GBF	GBGR	GBIL	GBLD
GBLO	GBUY	GCC	GCOR	GCOW	GDAT
GDIV	GDJJ	GDMA	GDNA	GDOC	GDVD
GDX	GDXJ	GDXX	GEM	GEMD	GENY
GERJ	GERM	GEUR	GEX	GFGF	GFIN
GFOF	GGEM	GHII	GHS	GHYB	GHYG
GIGB	GIGE	GII	GINN	GIVE	GIY
GK	GLCN	GLDE	GLIF	GLIN	GLOF
GLOV	GLRY	GMAN	GMET	GMF	GMFL
GMFS	GML	GMM	GMOM	GMTB	GNAT
GNMA	GNOM	GNR	GNRX	GOAT	GOAU
GOEX	GOP	GOVT	GPAL	GQRE	GREI
GREK	GRES	GRI	GRID	GRMY	GRNB
GRNR	GRPC	GRV	GSAX	GSD	GSEE
GSEU	GSEW	GSFP	GSG	GSGO	GSID
GSIE	GSIG	GSJY	GSLC	GSMA	GSPY
GSRA	GSSC	GSST	GSUS	GSY	GTAA
GTEK	GTIP	GTO	GULF	GUNR	GUR
GURU	GUSA	GUSH	GVAL	GVI	GVIP
GVT	GWL	GWX	GXC	GXF	GXG
GXTG	GYEN	GYLD	HACK	HACV	HACW
HAHA	HAIL	HAO	HAP	HAPI	HAPY
HART	HAUD	HAUZ	HAWX	HBTA	HBU
HCOM	HCRF	HDAW	HDEE	HDEF	HDEZ

HDG	HDGE	HDGI	HDIV	HDLS	HDMV
HDRO	HDUS	HDV	HDWM	HDWX	HECO
HEDJ	HEEM	HEET	HEFA	HEFV	HEGE
HEGJ	HELX	HEMV	HEQT	HERD	HERO
HEUS	HEUV	HEWC	HEWG	HEWI	HEWJ
HEWL	HEWP	HEWU	HEWW	HEWY	HEZU
HFEZ	HFGO	HFND	HFXE	HFXI	HFXJ
HGEM	HGEU	HGI	HGSD	HHH	HIBL
HIDE	HILO	HIPR	HIPS	HISF	HJEN
HJPX	HKK	HLAL	HLGE	HMTM	HNDL
HOLD	HOM	HOMZ	HOTL	HSCZ	HSMV
HSPX	HTEC	HTUS	HUSE	HUSV	HVAL
HVOL	HVPW	HYD	HYDD	HYDR	HYEM
HYG	HYGH	HYGI	HYGV	HYHG	HYIH
HYLD	HYLG	HYLS	HYLV	HYXU	IAH
IAI	IAK	IAT	IAU	IBB	IBBJ
IBBQ	IBCB	IBCC	IBCD	IBCE	IBD
IBDA	IBDB	IBDC	IBDD	IBDF	IBDH
IBDJ	IBDK	IBDL	IBDM	IBDN	IBDO
IBDP	IBDQ	IBDT	IBHA	IBHB	IBHC
IBHD	IBHE	IBLC	IBLN	IBMD	IBME
IBMF	IBMG	IBMH	IBMI	IBND	IBRN
IBUY	ICAN	ICAP	ICF	ICLN	ICOL
ICOW	ICSH	ICVT	IDAT	IDEV	IDHB
IDHD	IDHQ	IDLB	IDLV	IDMO	IDNA
IDOG	IDRV	IDU	IDV	IDVO	IDX
IECS	IEDI	IEF	IEFA	IEFN	IEHS
IEI	IEIH	IEIL	IEIS	IELG	IEME
IEMG	IEO	IESM	IETC	IEUR	IEUS
IEV	IEZ	IFAS	IFEU	IFGL	IFLY
IFNA	IFRA	IFSM	IFV	IGBH	IGE
IGEM	IGF	IGHG	IGIB	IGM	IGN

IGOV	IGRO	IGSB	IGU	IGV	IGW
IHAK	IHDG	IHE	IHF	IHI	IHY
IIH	IJH	IJJ	IJK	IJNK	IJR
IJS	IJT	ILDR	ILF	ILTB	IMFL
IMOM	IMTM	INCO	INDA	INDF	INDL
INDS	INDY	INFL	INFR	INKM	INNO
INTF	INXX	IOIL	IOO	IPAC	IPAY
IPD	IPF	IPFF	IPK	IPKW	IPN
IPO	IPOS	IPS	IPU	IPW	IQDE
IQDF	IQDG	IQDY	IQIN	IQLT	IQM
IQSI	IQSU	IRBO	IRO	IRV	IRY
ISCF	ISDS	ISDX	ISEM	ISHG	ISHP
ISI	ISMD	ISRA	IST	ISTB	ISVL
ISZE	ITA	ITAN	ITB	ITEQ	ITF
ITIP	ITM	ITOT	IUS	IUSB	IUSG
IUSS	IUSV	IVAL	IVDG	IVE	IVEG
IVES	IVLC	IVLU	IVOG	IVOL	IVOO
IVOV	IVRA	IVSG	IVV	IVW	IWB
IWC	IWD	IWF	IWFH	IWIN	IWL
IWM	IWN	IWO	IWP	IWR	IWS
IWTR	IWV	IWW	IWX	IWY	IWZ
IXC	IXG	IXJ	IXN	IXP	IXUS
IYC	IYE	IYF	IYG	IYH	IYJ
IYK	IYLD	IYM	IYR	IYT	IYW
IYY	IYZ	IZRL	JANZ	JAVA	JBBB
JCO	JCPB	JCPI	JCTR	JDG	JDIV
JEMA	JEPI	JETS	JGLD	JGRO	JHCB
JHCS	JHDV	JHEM	JHID	JHMA	JHMC
JHMD	JHME	JHMF	JHMH	JHMI	JHML
JHMM	JHMS	JHMT	JHMU	JHPI	JHSC
JIDA	JIG	JIRE	JKD	JKE	JKF
JKG	JKH	JKI	JKJ	JKK	JKL
JMBS	JMEE	JMIN	JMOM	JMST	JMUB

JNUG	JOET	JPED	JPEM	JPEU	JPGB
JPGE	JPHF	JPHY	JPIE	JPIN	JPMB
JPME	JPMV	JPNL	JPRE	JPSE	JPST
JPUS	JQUA	JRE	JRNY	JSCP	JSMD
JSML	JUSA	JUST	JVAL	JXI	JZRO
KALL	KARS	KBA	KBE	KBUY	KBWB
KBWC	KBWD	KBWI	KBWP	KBWR	KBWX
KBWY	KCE	KCNY	KDFI	KDIV	KEJI
KEMP	KEMQ	KEMX	KESG	KFVG	KFYP
KGHG	KGRN	KGRO	KIE	KLCD	KLD
KLDW	KLEM	KLIP	KLNE	KME	KMED
KMLM	KNG	KNGS	KNOW	KOCG	KOIN
KOKU	KOL	KOMP	KORP	KORU	KRBN
KRE	KRMA	KROO	KROP	KRU	KSCD
KURE	KVLE	KWEB	KWT	KXI	LABU
LALT	LATM	LBJ	LBTA	LCTD	LCTU
LDEM	LDRI	LDRS	LDSF	LEAD	LEGR
LEMB	LEND	LFEQ	LGEM	LGLV	LGOV
LIT	LIV	LKOR	LLDM	LLEM	LLSC
LLSP	LMBS	LNGR	LOPX	LOUP	LOWC
LQD	LQDH	LRGE	LRGF	LRNZ	LSAF
LSAT	LSLT	LTL	LUXE	LVHB	LVHD
LVHE	LVHI	LVIN	LVL	LVOL	LVUS
LWPE	LYFE	MAAX	MAGA	MAKX	MAMB
MARB	MARZ	MATF	MATH	MATL	MAUI
MBB	MBCC	MBOX	MBSD	MCEF	MCHI
MCRO	MCSE	MDCP	MDD	MDEV	MDIV
MDLL	MDY	MDYG	MDYV	MEAR	MEME
MES	META	METV	MEXX	MFDX	MFEM
MFMS	MFUL	MFUS	MGC	MGK	MGV
MID	MIDE	MIDF	MIDU	MIDZ	MILN
MINC	MISL	MJ	MJUS	MKH	MLN
MLPA	MLPX	MMIN	MMIT	MMLG	MMSC

MMTM	MNA	MNM	MOAT	MOHR	MOM
MONY	MOO	MOOD	MOON	MORT	MOTE
MOTG	MOTI	MOTO	MPRO	MRAD	MRGR
MSGR	MSOS	MTK	MTUM	MUB	MULT
MUSI	MVIN	MVP	MVPS	MVV	MXDU
MXI	MZG	MZN	MZO	NACP	NAIL
NANR	NASH	NASI	NDIV	NDJI	NDVG
NEAR	NEED	NERD	NETL	NFLT	NFO
NFRA	NIFE	NIWM	NLR	NOBL	NOMO
NORW	NSCS	NSPI	NSPY	NTKI	NTSE
NTSI	NUAG	NUBD	NUCL	NUDM	NUDV
NUEM	NUGO	NUGT	NUHY	NULC	NULG
NULV	NUMG	NUMV	NURE	NUSA	NUSC
NUSI	NVQ	NWLG	NXTE	NXTG	NY
NYC	NYCC	NYF	NZAC	OASI	OBOR
OCEN	OCIO	OEF	OEUR	OEW	OGEM
OGIG	OIH	OLD	OMFL	OMFS	OMOM
OND	ONEF	ONEK	ONEO	ONEQ	ONEV
ONEY	ONG	ONLN	ONOF	ONTL	OOTO
OPD	OQAL	ORG	OSCV	OSIZ	OTP
OTR	OUSA	OUSM	OVB	OVF	OVL
OVLC	OVLH	OVLU	OVM	OVOL	OVS
OVT	OWNS	OYLD	PABU	PACA	PAF
PAGG	PALC	PAMC	PAO	PAVE	PAWZ
PBD	PBDM	PBE	PBEE	PBJ	PBP
PBS	PBSM	PBTQ	PBUS	PBW	PCA
PCEF	PDBC	PDEV	PDN	PDP	PEGA
PEJ	PEK	PEX	PEXL	PEY	PEZ
PFA	PFF	PFFA	PFFD	PFFR	PFI
PFIG	PFIX	PFM	PFUT	PFXF	PGAL
PGF	PGHY	PGJ	PGRO	PGX	PHB
PHDG	PHO	PHYL	PIC	PICB	PICK
PID	PIE	PILL	PIN	PINK	PIO

PIQ	PIV	PIZ	PJB	PJF	PJG
PJM	PJP	PKB	PKN	PKOL	PKW
PLAT	PLCY	PLDR	PLND	PLRG	PLTL
PLTM	PLW	PMA	PMNA	PMOM	PMPT
PMR	PNQI	PNXQ	POTX	PPA	PPH
PPI	PPLC	PPSC	PQBW	PQDI	PQIN
PQLC	PQSC	PQSG	PQSV	PQY	PQZ
PRAY	PREF	PRF	PRFZ	PRME	PRN
PRNT	PSAU	PSC	PSCC	PSCD	PSCE
PSCF	PSCH	PSCI	PSCM	PSCT	PSCU
PSDN	PSET	PSFF	PSI	PSIL	PSJ
PSK	PSL	PSMB	PSMC	PSMG	PSMM
PSP	PSQ	PSR	PSTL	PSY	PTE
PTEU	PTF	PTH	PTIN	PTJ	PTLC
PTMC	PTNQ	PTO	PTRP	PUI	PUTW
PUW	PVAL	PVI	PWB	PWC	PWJ
PWND	PWO	PWP	PWT	PWV	PWY
PWZ	PXE	PXF	PXH	PXI	PXJ
PXLC	PXLG	PXLV	PXMC	PXMG	PXMV
PXN	PXQ	PXR	PXSC	PXSG	PXSV
PXUS	PY	PYH	PYZ	PZA	PZD
PZI	PZJ	PZT	QABA	QAI	QARP
QAT	QCAN	QCLN	QCLR	QDEF	QDEM
QDF	QDIV	QDPL	QDXU	QDYN	QED
QEFA	QEH	QEM	QEMM	QGBR	QGEM
QGRO	QGTA	QID	QINC	QINT	QLC
QLD	QLS	QLT	QLTA	QLTB	QLTC
QLV	QLVD	QLVE	QMJ	QMN	QMOM
QPT	QPX	QQC	QQD	QQEW	QQJG
QQMG	QQQ	QQQA	QQQE	QQQJ	QQQM
QQQN	QQQQ	QQXT	QRFT	QRMI	QSY
QTEC	QTR	QTUM	QUAL	QUS	QVAL

QVM	QVML	QVMM	QVMS	QWLD	QXGG
QXMI	QXRR	QXTR	QXUS	QYLD	QYLG
RAAX	RAFE	RALS	RAVI	RAYS	RBIN
RBL	RBLD	RBUS	RCD	RDIV	RDMX
RDOG	RDVI	RDVY	RECS	REDV	REEM
REET	REFA	REGL	REIT	REM	REMG
REMX	RESE	RESI	RESP	RETL	REVS
REZ	RFAP	RFDA	RFDI	RFEM	RFEU
RFF	RFFC	RFG	RFUN	RFV	RGI
RGLB	RGRO	RHS	RIDV	RIET	RIGS
RIGZ	RING	RISN	RKH	RLY	RNDM
RNDV	RNEM	RNLC	RNMC	RNRG	RNSC
ROAM	ROB	ROBO	ROBT	ROCI	RODE
RODM	ROGS	ROI	ROKT	ROM	ROMO
ROOF	RORE	RORO	ROSC	ROUS	RPG
RPQ	RPV	RPX	RRF	RRGR	RSCO
RSP	RSPE	RSPY	RSU	RSUN	RSX
RSXJ	RTH	RTL	RTM	RTR	RTYD
RULE	RUSL	RUSS	RVRS	RWCD	RWDC
RWDE	RWED	RWG	RWGV	RWIU	RWJ
RWK	RWL	RWLS	RWM	RWO	RWR
RWSL	RWUI	RWV	RWVG	RWW	RWX
RXI	RXL	RYE	RYF	RYH	RYJ
RYLD	RYLG	RYT	RYU	RZG	RZV
SAA	SAEF	SAGG	SATO	SAVN	SBEU
SBIO	SBUS	SCAP	SCHA	SCHB	SCHC
SCHD	SCHE	SCHF	SCHG	SCHH	SCHI
SCHJ	SCHK	SCHM	SCHO	SCHP	SCHQ
SCHR	SCHV	SCHX	SCHY	SCHZ	SCIF
SCIN	SCJ	SCLP	SCOG	SCTR	SCZ
SDAG	SDCI	SDEM	SDG	SDGA	SDIV
SDOG	SDOW	SDS	SDSI	SDVY	SDY

SEA	SECT	SEMI	SENT	SEPZ	SFY
SFYF	SFYX	SGDJ	SGDM	SGGG	SGQI
SH	SHBT	SHE	SHLD	SHMO	SHNY
SHOC	SHV	SHVY	SHY	SHYD	SHYG
SIL	SILJ	SIMS	SIPE	SIXA	SIXH
SIXL	SIXS	SIZ	SIZE	SKOR	SKYY
SLBT	SLDR	SLIM	SLOW	SLQD	SLT
SLVP	SLVY	SLX	SLY	SLYG	SLYV
SMB	SMCP	SMD	SMDV	SMDY	SMH
SMIG	SMIN	SMLE	SMLF	SMLL	SMLV
SMMD	SMMV	SMOG	SNPE	SNSR	SNUG
SOCL	SOLR	SOVB	SOXL	SOXQ	SOXX
SPAK	SPBC	SPCX	SPD	SPDV	SPDW
SPEM	SPEU	SPGM	SPGP	SPHB	SPHD
SPHQ	SPLG	SPLV	SPMD	SPMO	SPMV
SPQQ	SPRE	SPSB	SPSM	SPTM	SPUC
SPUN	SPUS	SPUU	SPVM	SPVU	SPXE
SPXH	SPXL	SPXN	SPXS	SPXT	SPXU
SPXV	SPY	SPYB	SPYC	SPYD	SPYG
SPYV	SPYX	SQEW	SQLV	SQQQ	SRET
SRVR	SSAM	SSFI	SSLY	SSO	SSPX
SSPY	SSUS	SSXU	STH	STIP	STLC
STLG	STLV	STMB	STOT	STRV	STSB
STXD	STXG	STXK	STXV	STXX	SUB
SUBZ	SULR	SUNY	SUPL	SURE	SUSA
SUSC	SUSL	SVAL	SVOL	SWAN	SWAR
SWH	SWIN	SXQG	SXUS	SYE	SYG
SYLD	SYTL	SYUS	SYV	SZNE	TAIL
TAN	TAO	TAWK	TAXF	TAXR	TBF
TBLU	TBT	TCHF	TCHI	TCHP	TCTL
TDD	TDH	TDIV	TDN	TDSA	TDSB
TDSC	TDSD	TDSE	TDTF	TDTT	TDV
TDVG	TDX	TECB	TECL	TEMP	TENG

TEQI	TERM	TETF	TFIV	TFLO	TFLT
TGEM	TGN	TGR	TGRW	THCX	THD
THHY	THNQ	THRK	TILT	TINT	TINY
TIP	TIPX	TLDH	TLEH	TLH	TLT
TLTD	TLTE	TMDV	TMF	TMFC	TMFE
TMFG	TMFM	TMFS	TMFX	TMW	TNA
TOK	TOKE	TOLZ	TONS	TOTL	TPAY
TPHD	TPHE	TPIF	TPLC	TPLE	TPOR
TPSC	TPYP	TQQQ	TRFM	TRND	TRPL
TRSK	TRTY	TSPA	TTAC	TTAI	TTFS
TTH	TUR	TUSA	TUTI	TUTT	TWEB
TWM	TWOK	TWON	TYD	TYLG	TYNE
TYNS	TZD	TZE	TZG	TZI	TZL
TZO	TZV	TZW	TZY	UAE	UAV
UBD	UBIO	UBOT	UBT	UCC	UCOM
UCYB	UDBI	UDIV	UDOW	UEVM	UGE
UGEM	UIVM	UJB	UK	UKF	UKK
UKW	ULQ	ULST	ULTR	ULVM	UMDD
UMI	UOP	UPRO	UPW	UPWD	URA
URE	URTH	URTY	USAI	USD	USEQ
USHG	USIG	USLB	USMC	USMF	USMV
USPX	USRT	USSD	USSG	UST	USVM
USWD	USXF	UTES	UTH	UTLF	UTLT
UTRN	UTSL	UUP	UVDV	UVG	UVT
UVU	UWC	UWM	UXI	UYG	UYM
VALQ	VALX	VAMO	VAW	VB	VBK
VBR	VCAR	VCIT	VCLN	VCLO	VCLT
VCR	VCSH	VDC	VDE	VEA	VEGA
VEGI	VERS	VESH	VETS	VEU	VFH
VFIN	VFLQ	VFMF	VFMO	VFMV	VFQY
VFVA	VGEM	VGFO	VGIT	VGK	VGSH
VGT	VHT	VICE	VIG	VIGI	VIOG
VIOO	VIOV	VIRS	VIS	VIXH	VLLV

VLML	VLSM	VLU	VLUE	VMOT	VNAM
VNLA	VNM	VNQ	VNQI	VO	VOE
VONE	VONG	VONV	VOO	VOOG	VOOV
VOT	VOX	VPC	VPL	VPN	VPOP
VPU	VR	VRAI	VRP	VSDA	VSGX
VSL	VSMV	VSPY	VSS	VT	VTC
VTHR	VTI	VTRN	VTV	VTWG	VTWO
VTWV	VUG	VV	VWID	VWO	VXF
VXUS	VYM	VYMI	WANT	WBAL	WBIA
WBIB	WBIC	WBID	WBIE	WBIF	WBIG
WBIH	WBII	WBIL	WBIN	WBIR	WBIT
WBIY	WCAT	WCBR	WCHN	WCLD	WDIV
WDNA	WDRW	WDTI	WEAR	WEBL	WEXP
WFH	WFVK	WGRO	WINN	WIP	WIZ
WKLY	WLDR	WLTH	WMCR	WMH	WNDY
WOMN	WOOD	WPS	WREI	WSKY	WTAI
WTRE	WTRX	WTV	WUGI	WUSA	WWJD
WWOW	WXSP	XAR	XB	XBB	XBI
XBUY	XCCC	XCEM	XCLR	XCOM	XDAT
XDNA	XES	XFIV	XGC	XHB	XHE
XHLF	XHMO	XHS	XHYC	XHYD	XHYE
XHYF	XHYH	XHYI	XHYT	XITK	XJH
XJR	XKFS	XKII	XKST	XLB	XLBS
XLBT	XLC	XLE	XLES	XLF	XLFS
XLG	XLI	XLIS	XLK	XLKS	XLP
XLPS	XLRE	XLSR	XLU	XLUS	XLV
XLVO	XLVS	XLY	XLYS	XME	XMHQ
XMLV	XMMO	XMPT	XMVM	XMX	XNTK
XONE	XOP	XOUT	XPH	XPND	XRLV
XRMI	XRO	XRT	XSD	XSHD	XSHQ
XSLV	XSMO	XSOE	XSVM	XSVN	XSW
XT	XTEN	XTH	XTL	XTN	XTR
XTRE	XTWO	XTWY	XUSA	XVOL	XVV

XWEB	XYLD	XYLG	YAO	YDIV	YESR
YINN	YLCO	YLD	YLDE	YMLI	YMLP
YOLO	YPRO	YPS	YUMY	YYY	ZCAN
ZDEU	ZGBR	ZGEN	ZHOK	ZIG	ZLRG
ZMLP	ZSML				

Example Applications

The ETF Constituents dataset provides an excellent source of tradable universes for strategies without selection bias. When you use an ETF universe, the original ETF can serve as an excellent benchmark for your strategy performance. Other use cases include the following:

- Creating an index-tracking algorithm for customized passive portfolio management
- Performing statistical arbitrage with the base ETF

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 10 largest US Equities in the SPY ETF. Each day, the algorithm forms a dollar-neutral and market-neutral portfolio by buying the 10 ETF constituents and shorting the SPY ETF.

```py
from AlgorithmImports import *


class ETFConstituentUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Add the SPY to trade.
        self._spy = self.add_equity("SPY").symbol
        # Add an ETF constituents universe that selects the large caps
        # in SPY. Save the universe object so you can get history
        # for the universe.
        universe = self.add_universe(
            self.universe.etf(
                self._spy,
                universe_filter_func=self._select_assets
            )
        )
        # Get historical universe data.
        history = self.history(universe, 30, Resolution.DAILY, flatten=True)
        # Show an example of wrangling the historical data.
        etf_weights = history.weight.unstack(1)
        # Create a dictionary to store the ETF weights each day.
        self._weight_by_symbol = {}
        # Add a Scheduled Event to rebalance the portfolio each day.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.after_market_open(self._spy, 1),
            self._rebalance
        )

    def _select_assets(
            self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Select the 10 largest stocks in the SPY.
        selected = sorted(
            [c for c in constituents if c.weight],
            key=lambda c: c.weight
        )[-10:]
        # Save the weights for position sizing.
        self._weight_by_symbol = {c.symbol: c.weight for c in selected}
        # Return the selected assets.
        return list(self._weight_by_symbol.keys())

    def _rebalance(self) -> None:
        # Create a long-short portfolio to earn the excess return of the
        # top 10 weighted stocks from SPY.
        spy_weight = sum(self._weight_by_symbol.values())
        targets = [PortfolioTarget(self._spy, -0.5)]
        for symbol, weight in self._weight_by_symbol.items():
            targets.append(PortfolioTarget(symbol, 0.5*weight/spy_weight))
        # Liquidate the stocks that aren't in top 10.
        self.set_holdings(targets, True)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 10 largest US Equities in the SPY ETF. Each day, the algorithm forms a dollar-neutral and market-neutral portfolio by buying the 10 ETF constituents and shorting the SPY ETF.

```py
class ETFConstituentUniverseFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = False
        self.universe_settings.resolution = Resolution.MINUTE
        self.weight_by_symbol = {}

        # Add universe selection on SPY's constituents to select only from large cap stocks
        # Save the universe to access its members for historical data call
        spy = self.add_equity("SPY").symbol
        self.add_universe_selection(ETFConstituentsUniverseSelectionModel(spy, self.universe_settings, self.etf_constituents_filter))

        # Add alpha model that set normalized weight as investment insight
        self.add_alpha(ETFConstituentsAlphaModel(self, spy))

        # Set up portfolio construction model that invest by the insight weights
        pcm = InsightWeightingPortfolioConstructionModel()
        # Avoid excessive rebalance on insight changes
        pcm.rebalance_on_insight_changes = False
        self.set_portfolio_construction(pcm)

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # The top 10 weighted securities are considered better active selections
        # Save the weights for position sizing
        selected = sorted([c for c in constituents if c.weight],
            key=lambda c: c.weight, reverse=True)[:10]
        self.weight_by_symbol = {c.symbol: c.weight for c in selected}

        return list(self.weight_by_symbol.keys())

class ETFConstituentsAlphaModel(AlphaModel):

    def __init__(self, algorithm: QCAlgorithm, etf: Symbol) -> None:
        self.algorithm = algorithm
        self.etf = etf
        self.day = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Rebalance daily since selection is on daily basis
        if self.day == algorithm.time.day:
            return []

        self.day = algorithm.time.day

        insights = []

        # Create a long-short portfolio to earn excess return of the top 10 weighted stocks from SPY
        etf_weight = sum(self.algorithm.weight_by_symbol.values())
        if etf_weight> 0:
            # Invest half the portfolio by normalized weights of the top 10 constituents
            for symbol, weight in self.algorithm.weight_by_symbol.items():
                if algorithm.securities.contains_key(symbol):
                    insights.append(Insight.price(symbol, Expiry.END_OF_DAY, InsightDirection.UP, weight=0.5*weight/etf_weight))

            # Short the other half with SPY, looking to profit from the active selection
            insights.append(Insight.price(self.etf, Expiry.END_OF_DAY, InsightDirection.DOWN, weight=0.5))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # Liquidate the ones not in top 10 weights
        symbols = [x.symbol for x in changes.removed_securities if x.invested]
        algorithm.liquidate(symbols, tag='Removed From Universe')
```

Research Example

The following example lists ETF constituents with the greatest weight in the SPY:

```py
qb = QuantBook()

# Add the ETF
qb.spy = qb.add_equity("SPY").symbol

# Add ETF Universe Selection
def etf_constituents_filter(constituents):
    # Take the top 10 weighted constituents
    selected = sorted([c for c in constituents if c.weight],
        key=lambda c: c.weight, reverse=True)[:10]
    return [c.symbol for c in selected]

universe = qb.add_universe(qb.universe.etf(qb.spy, qb.universe_settings, etf_constituents_filter))

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), constituents in universe_history.items():
    for constituent in constituents:
        print(f"{constituent.symbol} weight at {constituent.end_time}: {constituent.weight}")
```

Data Point Attributes

The ETF Constituents dataset provides `ETFConstituentUniverse` objects, which have the following attributes:

QuantConnect

US Equities Short Availability

Introduction

The US Equity Short Availability dataset provides the available shares for open short positions and their borrowing cost in the US Equity market. The data covers 10,500 US Equities, starts in January 2018, and is delivered on a daily frequency. This dataset is created using information from the exchanges.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US Equities Short Availability dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippets demonstrate how to request data from the US Equities Short Availability dataset.

Interactive Brokers Data

```
security.set_shortable_provider(InteractiveBrokersShortableProvider())
```

Axos Clearing Data

```
security.set_shortable_provider(LocalDiskShortableProvider("axos"))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2018
Asset Coverage	10,500 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

To add US Equities Short Availability data to your algorithm, set the shortable provider of each US Equity in your algorithm.

```
class ShortAvailabilityDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        security = self.add_equity("AAPL")
        # Set shortable provider as IB
        security.set_shortable_provider(InteractiveBrokersShortableProvider())
        self._symbol = security.symbol
```

Accessing Data

To check how many shares are available for a security to short, call the `shortable_quantity` method of the `shortable_provider`

```
shortable_provider = self.securities[self._symbol].shortable_provider
# Get the shortable quantity of the selected symbol at the selected time
shortable_quantity = shortable_provider.shortable_quantity(self._symbol, self.time)

# Check if there are a certain quantity of shares available
quantity = 100;
is_shortable_quantity = self.shortable(self._symbol, quantity)
```

To check borrowing cost, call the `fee_rate` or `rebate_rate` method of the `shortable_provider`

```
fee_rate = shortable_provider.fee_rate(self._symbol, self.time);
rebate_rate = shortable_provider.rebate_rate(self._symbol, self.time);
```

To get valid borrowing rates, use the InteractiveBrokersShortableProvider .

Example Applications

The US Equities Short Availability dataset enables you to accurately design strategies harnessing information about short availability. Examples include the following use cases:

- Avoiding short orders that the brokerage will reject
- Selecting securities based on how many shares are available to short

Classic Algorithm Example

The following example algorithm shorts GameStop every day there are shares available to short. If the algorithm receives a margin call, it liquidates the position and start again on the next day.

```py
from AlgorithmImports import *

class ShortAvailabilityDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000)
        # Seed the security price as the last known price, such that the price data is immediately available at initial rebalance
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)

        self.equity = self.add_equity("GME")

        # Set up daily rebalance scheduled event, since shortable quantity is updated daily
        self.schedule.on(
            self.date_rules.every_day(self.equity.symbol),
            self.time_rules.after_market_open(self.equity.symbol, 10),
            self.rebalance)

    def rebalance(self) -> None:
        symbol = self.equity.symbol

        # You can obtain the shortable quantity to decide the submission of a short order
        shortable_quantity = self.equity.shortable_provider.shortable_quantity(symbol, self.time)
        if not shortable_quantity:
            shortable_quantity = 0
        # Fee and rebate rate is also available, such that you can calculate the expected return and decide if the margin is worthwhile
        self.plot('Total Shortable Quantity', symbol, shortable_quantity)
        self.plot('Borrowing Cost', "Fee Rate", self.equity.shortable_provider.fee_rate(symbol, self.time))
        self.plot('Borrowing Cost', "Rebate Rate", self.equity.shortable_provider.rebate_rate(symbol, self.time))

        # Test whether we can short the desired quantity
        quantity = self.calculate_order_quantity(symbol, -1)
        if quantity and self.shortable(symbol, quantity):
            self.market_order(symbol, quantity)

    def on_margin_call_warning(self) -> None:
        self.liquidate()

    def _custom_security_initializer(self, security: Security) -> None:
        # Set the shortable provider as your broker for accurate short reality modeling
        security.set_shortable_provider(InteractiveBrokersShortableProvider())
```

Framework Algorithm Example

The following example algorithm shorts GameStop every day there are shares available to short. If the algorithm receives a margin call, it liquidates the position and start again on the next day.

```py
from AlgorithmImports import *


class ShortAvailabilityDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:

        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000)
        # Seed the security price as the last known price, such that the price data is immediately available at initial rebalance
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)

        self.set_universe_selection(ManualUniverseSelectionModel(
            [Symbol.create("GME", SecurityType.EQUITY, Market.USA)]))

        # Emit down-direction insights to short all securities in the universe
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.DOWN, timedelta(1)))
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ShortableExecutionModel())

    # On Margin Call, emit flat-direction insights to liquidate the positions
    def on_margin_call_warning(self) -> None:
        self.emit_insights([Insight.price(kvp.Key, timedelta(1), InsightDirection.FLAT)
            for kvp in self.securities if kvp.Value.invested])

    def _custom_security_initializer(self, security: Security) -> None:
        # Set the shortable provider as IB
        security.set_shortable_provider(InteractiveBrokersShortableProvider())

class ShortableExecutionModel(ExecutionModel):
    def __init__(self) -> None:
        self.targets_collection = PortfolioTargetCollection()

    def execute(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> None:
        '''Immediately submits orders for the specified portfolio targets.
        Args:
            algorithm: The algorithm instance
            targets: The portfolio targets to be ordered'''

        # for performance we check count value, OrderByMarginImpact and ClearFulfilled are expensive to call
        self.targets_collection.add_range(targets)
        if self.targets_collection.count > 0:
            for target in self.targets_collection.order_by_margin_impact(algorithm):
                # calculate remaining quantity to be ordered
                quantity = OrderSizing.get_unordered_quantity(algorithm, target)
                # If the quantity is negative, ensure that the security is shortable
                if quantity > 0 or algorithm.shortable(target.symbol, quantity):
                    algorithm.market_order(target.symbol, quantity)

            self.targets_collection.clear_fulfilled(algorithm)
```

Data Point Attributes

The US Equities Short Availability data is a `Symbol` /decimal pair for `ShortQuantity` , `FeeRate` , and `RebateRate` .

QuantConnect

US Equity Coarse Universe

Introduction

The US Equity Coarse Universe dataset by QuantConnect is a daily universe of all trading stocks in the US for a given day with the end of day price and volume. The data covers 30,000 US Equities in total, with approximately 8,000 Equities per day. The data starts in January 1998 and is delivered each trading day. This dataset is created by taking the closing auction price tick from the daily L1 trade and quote exchange dumps.

This dataset depends on the US Equities by AlgoSeek dataset because universe selection adds and removes market data subscriptions and on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

QuantConnect/LEAN combines the data of this dataset with MorningStar Fundamental data in runtime. You can use this dataset in local development without the Morningstar counterpart.

For more information about the US Equity Coarse Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US Equity Coarse Universe dataset:

- Direct Access using the Security object

```
equity = self.add_equity("IBM")
ibm_fundamental = equity.fundamentals
```

- Direct Access using the Symbol object

```
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals
```

- Universe Selection

```
def initialize(self) -> None:
    # Universe selection
    self._universe = self.add_universe(self.fundamental_filter_function)

def fundamental_filter_function(self, fundamental: List[Fundamental]):
    # Sort all equities with price above $10 by dollar volume
    sorted_by_dollar_volume = sorted([f for f in fundamental if f.price > 10],
                                     key=lambda f: f.dollar_volume, reverse=True)
    # Take the top 10
    return [f.symbol for f in sorted_by_dollar_volume[:10]]
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Asset Coverage	30,000 US Equities
Data Density	Dense
Resolution	Daily
Timezone	New York

This dataset doesn't include Over-the-Counter (OTC) stocks.

Requesting Data

You don't need any special code to request US Coarse Fundamental Data. You can access the current and historical fundamental data for any of the US Equities that this dataset includes.

```
PY
class CoarseFundamentalDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 7, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True

        # Option 1: Subscribe to individual US Equity assets
        self.add_equity("IBM")

        # Option 2A: Create a fundamental universe (classic version)
        self._universe = self.add_universe(self.fundamental_function)

        # Option 2B: Create a fundamental universe (framework version)
        self.add_universe_selection(FundamentalUniverseSelectionModel(self.fundamental_function))
```

For more information about universe settings, see Settings .

Accessing Data

If you add a fundamental universe to your algorithm, you can access fundamental data in the universe selection function.

```
PY
def fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
    sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)[:3]
    for cf in sorted_by_dollar_volume:
        self.debug(f"{cf.end_time} :: {cf.symbol} : {cf.adjusted_price} :: {cf.dollar_volume}")

    return [ x.symbol for x in sorted_by_dollar_volume]
```

To get fundamental data for Equities in your algorithm, use the `fundamentals` property of the `Equity` objects. The fundamental data represent the corporate fundamentals for the current algorithm time.

```
PY
fundamentals = self.securities[symbol].fundamentals
```

To get fundamental data for Equities, regardless of whether or not you have subscribed to them in your algorithm, call the `fundamentals` method. If you pass one `Symbol` , the method returns a `Fundamental` object. If you pass a list of `Symbol` objects, the method returns a list of `Fundamental` objects.

```
PY
# Single asset
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals(ibm)

# Multiple assets
nb = Symbol.create("NB", SecurityType.EQUITY, Market.USA)
fundamentals = self.fundamentals([ nb, ibm ])
```

Historical Data

You can get historical fundamental data in an algorithm and in the Research Environment.

Historical Data in Algorithms

To get historical fundamental data in an algorithm, call the `history` method with `Fundamental` type and the Equity `Symbol` . If there is no data in the period you request, the history result is empty.

```py
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)

# DataFrame of fundamental data for a given asset
df_history = self.history(Fundamental, ibm, timedelta(30), flatten=True)

# Fundamental objects
fundamental_history = self.history[Fundamental](ibm, timedelta(30))

# Fundamentals objects for all US Equities (including delisted companies)
fundamentals_history = self.history[Fundamentals](timedelta(30))

# DataFrame of fundamental data for universe constituents
df_history = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series of fundamental data for universe constituents
series_history = self.history(self._universe, 30, Resolution.DAILY)
for (universe_symbol, time), fundamental in series_history.items():
    most_liquid = sorted(fundamental, key=lambda x: x.dollar_volume)[-5:]
```

Historical Universe Data in Research

To get historical universe data in the Research Environment, call the `universe_history` method with the `Universe` object and the lookback period. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame of fundamental data for universe constituents
df_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series of fundamental data for universe constituents
series_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), fundamentals in series_history.items():
    for fundamental in fundamentals:
        print(f"{fundamental.symbol} dollar volume at {fundamental.end_time}: {fundamental.dollar_volume}")
```

For more information about historical US Equity fundamental data, see Equity Fundamental Data .

Example Applications

The US Equity Coarse Universe dataset enables you to accurately design a universe of US Equities. Examples include the following strategies:

- Selecting securities with the largest dollar volume
- Selecting securities within a specific price range
- Selecting securities that have fundamental data available (together with MorningStar Fundamental data in `Fundamental` objects)

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the three most liquid US Equities. Each time the universe changes, the algorithm forms an equal-weighted portfolio of the three companies.

```py
from AlgorithmImports import *


class USEquityCoarseUniverseConstituentsDataAlgorithm(QCAlgorithm):

    _number_of_symbols = 3
    _changes = None

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Set Asynchronous to True to improve speed performance.
        self.universe_settings.asynchronous = True
        # Add a universe of US Equities.
        universe = self.add_universe(self._select_assets)
        # Get historical data for the universe.
        history = self.history(universe, 30, Resolution.DAILY, flatten=True)

    def _select_assets(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select the top traded stocks, since they are the most popular
        # with high capital flow.
        return [
            x.symbol for x in sorted(
                fundamental, key=lambda x: x.dollar_volume
            )[-self._number_of_symbols:]
        ]

    def on_data(self, slice: Slice) -> None:
        # If we have no changes, do nothing.
        if not slice.bars or self._changes is None:
            return
        # Liquidate removed securities, since they are not the most
        # popular stocks.
        for security in self._changes.removed_securities:
            if security.invested:
                self.liquidate(security.symbol)
        # We want 1/N allocation for each security in our universe to
        # evenly dissipate risk.
        for security in self._changes.added_securities:
            self.set_holdings(security.symbol, 1 / self._number_of_symbols)
        self._changes = None

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._changes = changes
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the three most liquid US Equities and forms an equal-weighted portfolio with them.

```py
from AlgorithmImports import *


class USEquityCoarseUniverseConstituentsDataAlgorithm(QCAlgorithm):

    _number_of_symbols = 3

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Set Asynchronous to True to improve speed performance
        self.universe_settings.asynchronous = True

        # Requesting data
        self.set_universe_selection(
            FundamentalUniverseSelectionModel(self.fundamental_selection_function))

        # we want 1/N allocation in each security in our universe to evenly dissipate risk
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select the top traded stocks, since they are the most popular with high capital flow
        sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)
        return [ x.symbol for x in sorted_by_dollar_volume[:self._number_of_symbols] ]
```

Research Example

The following example lists the three most liquid US Equities:

```py
qb = QuantBook()

# Add Fundamental Universe Selection
def fundamental_selection_function(fundamentals):
    selected = sorted(fundamentals, key=lambda x: x.dollar_volume, reverse=True)[:10]
    return [x.symbol for x in selected]

universe = qb.add_universe(fundamental_selection_function)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), fundamentals in universe_history.items():
    for fundamental in fundamentals:
        print(f"{fundamental.symbol} dollar volume at {fundamental.end_time}: {fundamental.dollar_volume}")
```

Data Point Attributes

The US Equity Coarse Universe dataset provides its data within `Fundamental` objects, which have the following . The attributes from `market_cap` to `asset_classification` listed in the following widget are NaN until this dataset is combined with the MorningStar Fundamental dataset.

QuantConnect

US Equity Option Universe

Introduction

The US Equity Option Universe dataset by QuantConnect lists the available US Equity Options contracts and the current Implied Volatility and Greeks. The data covers 4,000 Symbols, starts in January 2012, and is delivered on a daily update frequency. To create this dataset, we use our implementation of the forward tree pricing model, which accounts for the interest rate, dividend payments, and daily closing prices. The values in this dataset are the same values you can get from daily indicators with mirror Options.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes of the underlying security.

This dataset `does not` contain market data. For market data, see US Equity Options by AlgoSeek .

For more information about the US Equity Option Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US Equity Options Universe dataset:

```
option = self.add_option("GOOG")
self.option_symbol = option.symbol
option.set_filter(lambda universe: universe.delta(0.4, 0.6))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012
Asset Coverage	4,000 Symbols
Data Density	Dense
Resolution	Daily
Timezone	New York

Requesting Data

To add US Equity Options Universe data to your algorithm, call the `add_option` method. Save a reference to the Equity Option `Symbol` so you can access the data later in your algorithm. To define which contracts should be in your universe, call the `set_filter` method of the Option object.

The `add_option` method provides a daily stream of Option chain data. To get the most recent daily chain, call the `option_chain` method with the underlying Equity Symbol. The `option_chain` method returns data on all the tradable contracts, not just the contracts that pass your universe filter.

```
class USEquityOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        option = self.add_option("GOOG")
        self.option_symbol = option.symbol
        # Set our strike/expiry filter for this option chain
        option.set_filter(self._option_filter)
        # Get the entire Option chain for the current day.
        chain = self.option_chain(option.symbol.underlying, flatten=True).data_frame

    def _option_filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Contracts can be filtered by greeks, implied volatility, open interest:
        return universe \
            .delta(0.5, 1.5) \
            .gamma(0.0001, 0.0006) \
            .vega(0.01, 1.5) \
            .theta(-2.0, -0.5) \
            .rho(0.5, 3.0) \
            .implied_volatility(1, 3) \
            .open_interest(100,500)
```

The Equity resolution must be less than or equal to the Equity Option resolution. For example, if you set the Equity resolution to minute, then you must set the Equity Option resolution to minute, hour, or daily.

For more information about creating US Equity Option Universes, see Equity Options .

Accessing Data

For information about accessing US Equity Options Universe data, see Equity Options .

Historical Data

You can get historical US Equity Options Universe data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Equity Options Universe data in an algorithm, call the `history` method with the canonical Equity Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter. If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.option_symbol, timedelta(3), flatten=True)

# OptionUniverse objects
history = self.history[OptionUniverse](self.option_symbol, timedelta(3))
```

For more information about historical Equity Options Universe data in algorithms, see Historical Data .

Historical Data In Research

To get historical US Equity Options Universe data in the Research Environment, call the `history` method with the canonical Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter.

```
qb = QuantBook()
option = qb.add_option("GOOG")
history = qb.history(option.symbol, datetime(2020, 6, 1), datetime(2020, 6, 5), flatten=True)
```

For more information about historical Equity Options Universe data in the Research Environment, see Universes .

Supported Assets

To view the supported assets in the US Equity Options Universe dataset, see the Data Explorer .

Example Applications

The US Equity Options Universe dataset enables you to accurately design Option strategies. Examples include the following strategies:

- Buying put Options to hedge against downward price movement in positive Equity positions
- Exploiting arbitrage opportunities that arise when the price of Option contracts deviate from their theoretical value

Classic Algorithm Example

The following example algorithm subscribes to Google put options that fall within delta range between -1 and -0.95, open interest range between 10 and 1000, and expire within seven days. Within this Option chain, the algorithm holds the put Option contract that has the minimum delta (closest to -1) during market hour to hedge the underlying intra-day movement completely. It avoid volatility from sentiment and only earns from inter-day movement from longer-term factors. When the contract expires, the algorithm rolls over to the next contract that meets this criteria.

```py
from AlgorithmImports import *

class USEquityOptionsUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)

        # Asynchronous can use computational resources efficiently
        self.universe_settings.asynchronous = True
        # Subscribe to the underlying for the underlying position
        # Set the data normalization mode to raw for strike price comparability
        self.equity = self.add_equity("GOOG", data_normalization_mode=DataNormalizationMode.RAW).symbol
        # Requesting option data and filter for the hedge candidates
        option = self.add_option(self.equity)
        option.set_filter(self.option_filter)
        self.option_symbol = option.symbol

        # Set scheduled event to buy a hedge option contract at market open to eliminate the intra-day movement
        self.schedule.on(
            self.date_rules.every_day(self.equity),
            self.time_rules.after_market_open(self.equity, 1),
            self.buy_hedge_contract
        )

        # Set a scheduled event to sell the hedge contract before market close, since we want to earn from inter-day movement
        # Leave 2 minutes contingency to fill
        self.schedule.on(
            self.date_rules.every_day(self.equity),
            self.time_rules.before_market_close(self.equity, 2),
            self.sell_hedge_contract
        )

        self.hedge = None

    def option_filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Select the contracts with delta very close to -1 and high open interest
        # This can effectively hedge most of the price change of the underlying and ensure the liquidity
        # Make sure the contract is expiring close for its tradbility
        return universe.puts_only().expiration(2, 7).delta(-1, -0.95).open_interest(10, 1000)

    def buy_hedge_contract(self) -> None:
        chain = self.current_slice.option_chains.get(self.option_symbol)
        if chain:
            # Order the underlying if not hold, the order size should match the option contract
            # Order only if option chain data ready for hedging
            if not self.portfolio[self.equity].invested:
                self.market_order(self.equity, self.securities[self.option_symbol].symbol_properties.contract_multiplier)

            # Get the contract with delta closest to -1 (lowest possible delta)
            contract = sorted(chain, key=lambda x: x.greeks.delta)[0]
            self.hedge = contract.symbol
            # Buy 1 deep ITM put with delta close to -1 to eliminate the intraday movement
            self.market_order(self.hedge, 1)

    def sell_hedge_contract(self) -> None:
        # Check if any hedge contract position, if so, liquidate before market close to expose to underlying overnight movement
        if self.hedge:
            self.liquidate(self.hedge)
            self.hedge = None
```

Framework Algorithm Example

The following example algorithm demonstrating a Gamma Scalping strategy through framework algorithm. It filters NVDA options with expiration between 30 to 90 days and open interest between 50 and 1000, due to liquidity concern and lower Gamma fluctuation. Assuming a significant upward trend is expecting, it orders a long straddle strategy from the strike price with Delta-neutral and the highest Gamma to earn the highest profit from upward underlying movement.

PY

```
from AlgorithmImports import *

class USEquityOptionsUniverseFrameworkAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Asynchronous can use computational resources efficiently
        self.universe_settings.asynchronous = True

        # Universe selection that select based on liquidity of the option contracts (by open interest)
        self.add_universe_selection(EquityOptionsUniverseSelectionModel())
        # Custom alpha model that use Delta and Gamma to signal insights
        self.add_alpha(OptionDeltaGammaAlphaModel())
        # To maintain long-short size equal, use a PCM that order a single contract
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class EquityOptionsUniverseSelectionModel(OptionUniverseSelectionModel):
    def __init__(self) -> None:
        # Daily update with the select_option_chain_symbols function
        super().__init__(timedelta(1), self.select_option_chain_symbols)

    def select_option_chain_symbols(self, dt: datetime) -> List[Symbol]:
        # Select only NVDA options as our focus, which is a highly traded volatile equity with great upward momentum
        return [Symbol.create("NVDA", SecurityType.OPTION, Market.USA)]

    def filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # To ensure the liquidity and tradability, make sure the option is not 0-DTE and have a fair open interest
        # A longer TTM will have lower Gamma thus more stable in the local delta-neutral position
        return universe.expiration(30, 90).open_interest(50, 1000)

class OptionDeltaGammaAlphaModel(AlphaModel):
    def __init__(self) -> None:
        # A day count variable to control the alpha model only trade once a day
        self._day = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        if self._day != slice.time.day:
            # We open position for each option underlying as separate bet
            for _, chain in slice.option_chains.items():
                # For theta-neutral, select the same expiry for both call and put
                expiry = min(x.expiry for x in chain)
                contracts = [x for x in chain if x.expiry == expiry]

                # Calculate delta and gamma per strike price group for later filtering, ensure both call and put available for option
strategy
                delta_gamma_symbols = []
                strikes = set(x.strike for x in contracts if len([y for y in contracts if y.strike == x.strike]) == 2)
                for strike in strikes:
                    # Get both call and put for their aggregated delta and gamma
                    call = next(filter(lambda x: x.right == OptionRight.CALL and x.strike == strike, contracts))
                    put = next(filter(lambda x: x.right == OptionRight.PUT and x.strike == strike, contracts))
                    delta_gamma_symbols.append((call.greeks.delta + put.greeks.delta, call.greeks.gamma + put.greeks.gamma, call.symbol,
put.symbol))

                if len(delta_gamma_symbols) == 0:
                    continue

                # We want a delta-neutral position, so it is likely to be ATM (like a long straddle)
                # Less than 2d.p. difference is non-significant, which we can risk for better reward
                # Assuming the market direction is up in most scenario, we try to get the strike providing max overall Gamma
                # Make sure the aggregated gamma to be positive to bet on large uptrend
                # So it will earn more when the price really go up higher and higher, but locally immune for small noise
                filtered = [item for item in delta_gamma_symbols if round(item[0], 2) <= 0.01 and item[1] > 0]
                if len(filtered) == 0:
                    continue
                selected = sorted(filtered, key=lambda x: x[1], reverse=True)[0]

                # Provide trade signal and roll the day count to let the position stay for the whole day
                selected_call = selected[2]
                selected_put = selected[3]
                insights.extend([
                    Insight.price(selected_call, Expiry.END_OF_DAY, InsightDirection.UP),
                    Insight.price(selected_put, Expiry.END_OF_DAY, InsightDirection.UP)
                ])
                self._day = slice.time.day

        return insights

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Use a whole number target to order the exact number of share for size-matching
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Equity Options Universe dataset provides `OptionFilterUniverse` , `OptionUniverse` , and `OptionChain` objects.

OptionFilterUniverse Attributes

`OptionFilterUniverse` objects have the following attributes:

OptionUniverse Attributes

`OptionUniverse` objects have the following attributes:

OptionChain Attributes

`OptionChain` objects have the following attributes:

QuantConnect

US Equity Security Master

Introduction

The US Equity Security Master dataset by QuantConnect tracks US Equity corporate actions, including splits, dividends, delistings, mergers, and ticker changes through history. The data covers approximately 27,500 US Equities, starts in January 1998, and is delivered on a daily update frequency. You can easily download and install the dataset with the LEAN CLI so it's ready to use by LEAN. LEAN automatically handles all corporate actions and passes them into your algorithm as events .

This is `NOT` the underlying Equity data (US Equities dataset), which you need to purchase separately with a license from AlgoSeek. This security master dataset is required to purchase the US Equities or US Equities Options datasets.

For more information about the US Equity Security Master dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Data Summary

Data is delivered as a daily updated zip archive of map and factor files. The data is designed to be used in the LEAN Engine and cannot be consumed another way. The following table shows the dataset properties:

Property	Value
Start Date	January 1998
Data Points	Splits, Dividends, Mergers, IPO, & Delistings
Asset Coverage	27,500 US Equities
Resolution	Daily
Timezone	New York

This is `not` the underlying Equity data (US Equities dataset), which you need to purchase separately with a license from AlgoSeek. This security master dataset is required to purchase the US Equities or US Equities Options datasets.

Getting Started

You don't need any special code to utilize the US Equity Security Master. It automatically loads when you request US Equities data .

Accessing Splits

To get the current split data, index the `splits` property of the current `Slice` with the Equity `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```py
def on_data(self, slice: Slice) -> None:
    # Check if any splits for the symbol
    if slice.splits.contains_key(self._symbol):
        # If so, get the mapped split object
        split = slice.splits[self._symbol]
        split_type = {0: "Warning", 1: "SplitOccurred"}.get(split.type)
        self.log(f"Split: {split.symbol}\t{split.split_factor}\t{split.reference_price}\t{split_type}")
```

For more information about accessing splits, see Splits .

Accessing Dividends

To get the current dividend data, index the `dividends` property of the current `Slice` with the Equity `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    # Check if any dividend for the symbol
    if slice.dividends.contains_key(self._symbol):
        # If so, get the mapped dividend object
        dividend = slice.dividends[self._symbol]
        self.log(f'Dividend: {dividend.symbol}\t{dividend.distribution}\t{dividend.reference_price}')
```

For more information about accessing dividends, see Dividends .

Accessing Delistings

To get the current Delistings data, index the `delistings` property of the current `Slice` with the Equity `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    # Check if any delisting for the symbol
    if slice.delistings.contains_key(self._symbol):
        # If so, get the mapped delisting object
        delisting = slice.delistings[self._symbol]
        delisting_type = {0: "Warning", 1: "Delisted"}.get(delisting.type)
        self.log(f'Delistings: {delisting_type}')
```

For more information about accessing delistings, see Delistings .

Accessing Symbol Change Events

To get the current Symbol change events, index the `symbol_changed_events` property of the current `Slice` with the Equity `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    # Check if any symbol change event for the symbol
    if slice.symbol_changed_events.contains_key(self._symbol):
        # If so, get the mapped SymbolChangeEvent object
        symbol_changed_event = slice.symbol_changed_events[self._symbol]
        self.log(f"Symbol changed: {symbol_changed_event.old_symbol} -> {symbol_changed_event.new_symbol}")
```

For more information about accessing Symbol change events, see Symbol Changes .

Historical Data

To get historical US Equity Security Master data, call the `history` method with the data type and the Equity `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# Splits
split_history_df = self.history(Split, self._symbol, timedelta(5*365))
split_history = self.history[Split](self._symbol, timedelta(5*365))

# Dividends
dividend_history_df = self.history(Dividend, self._symbol, timedelta(5*365))
dividend_history = self.history[Dividend](self._symbol, timedelta(5*365))

# Symbol Changes
symbol_change_history_df = self.history(SymbolChangedEvent, self._symbol, timedelta(5*365))
symbol_change_history = self.history[SymbolChangedEvent](self._symbol, timedelta(5*365))

# Delistings
delisting_history_df = self.history(Delisting, self._symbol, timedelta(5*365))
delisting_history = self.history[Delisting](self._symbol, timedelta(5*365))
```

For more information about historical data, see History Requests .

Live Trading Considerations

In backtesting, corporate actions occurs at midnight (ET). In live trading, the live data for corporate actions arrives at 6/7 AM ET, so that's when they occur.

Supported Assets

To view the supported assets in the US Equity Security Master dataset, see the Data Explorer . This dataset doesn't include Over-the-Counter (OTC) stocks.

Example Applications

The US Security Master enables you to accurately design strategies harnessing any core corporate actions. Examples include the following strategies:

- Post-dividend announcement trading strategies.
- Trading on new Equities by monitoring for IPOs.
- Harnessing split announcements for reverse-split announcement momentum.

Classic Algorithm Example

The following example algorithm logs the `Split` , `Dividend` , `Delisting` , and `SymbolChangedEvent` objects of Apple:

```py
from AlgorithmImports import *

class USEquitySecurityMasterAlgorithm (QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        self.equity = self.add_equity("AAPL", Resolution.DAILY).symbol

    def on_data(self, slice: Slice) -> None:
        # Accessing Data - Splits
        split = slice.splits.get(self.equity)
        if split:
            self.debug(f"{self.time} >> SPLIT >> {split.symbol} - {split.split_factor} - {self.portfolio.cash} -
{self.portfolio[self.equity].price}")

        # Accessing Data - Dividends
        dividend = slice.dividends.get(self.equity)
        if dividend:
            self.debug(f"{self.time} >> DIVIDEND >> {dividend.symbol} - {dividend.distribution} - {self.portfolio.cash} -
{self.portfolio[self.equity].price}")

        # Accessing Data - Delisting
        delisting = slice.delistings.get(self.equity)
        if delisting:
            delistingType = {0: "Warning", 1: "Delisted"}.get(delisting.type)
            self.debug(f"{self.time} >> DELISTING >> {delisting.symbol} - {delistingType}")

        # Accessing Data - Symbol Changed Event
        symbolChangedEvent = slice.symbol_changed_events.get(self.equity)
        if symbolChangedEvent:
            self.debug(f"{self.time} >> SYMBOL CHANGED >> {symbolChangedEvent.old_symbol} -> {symbolChangedEvent.new_symbol}")
```

Framework Algorithm Example

The following algorithm demonstrates the payments for cash dividends in backtesting. When the data normalization mode is `Raw` , your portfolio receives cash dividends.

```py
from AlgorithmImports import *

class PaymentAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # this will use the Tradier Brokerage open order split behavior
        # forward split will modify open order to maintain order value
        # reverse split open orders will be canceled
        self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW

        # MSFT: Splits and Dividends
        # GOOG: Symbol Changed Event
        # AAA.1: Delisting

        self.set_universe_selection(ManualUniverseSelectionModel(
            Symbol.create("MSFT", SecurityType.EQUITY, Market.USA)))

        self.set_alpha(PaymentAlphaModel())
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(BracketExecutionModel())

class PaymentAlphaModel(AlphaModel):

    symbol = Symbol.EMPTY

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Accessing Data - Splits
        split = slice.splits.get(self.symbol)
        if split:
            algorithm.debug(f"{algorithm.time} >> SPLIT >> {split.symbol} - {split.split_factor} - {algorithm.portfolio.cash} -
{algorithm.portfolio[self.symbol].price}")

        # Accessing Data - Dividends
        dividend = slice.dividends.get(self.symbol)
        if dividend:
            algorithm.debug(f"{algorithm.time} >> DIVIDEND >> {dividend.symbol} - {dividend.distribution} - {algorithm.portfolio.cash} -
{algorithm.portfolio[self.symbol].price}")

        # Accessing Data - Delistings
        delisting = slice.delistings.get(self.symbol)
        if delisting:
            delistingType = {0: "Warning", 1: "Delisted"}.get(delisting.type)
            algorithm.debug(f"{algorithm.time} >> DELISTING >> {delisting.symbol} - {delistingType}")

        # Accessing Data - Symbol Changed Events
        symbolChangedEvent = slice.symbol_changed_events.get(self.symbol)
        if symbolChangedEvent:
            algorithm.debug(f"{algorithm.time} >> SYMBOL CHANGED >> {symbolChangedEvent.old_symbol} -> {symbolChangedEvent.new_symbol}")

        bar = slice.bars.get(self.symbol)
        return [Insight.price(self.symbol, timedelta(1), InsightDirection.UP)] if bar else []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        self.symbol = list(changes.added_securities)[0].symbol

class BracketExecutionModel(ExecutionModel):

    def __init__(self) -> None:
        '''Initializes a new instance of the ImmediateExecutionModel class'''
        self.targets_collection = PortfolioTargetCollection()

    def execute(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> None:

        # for performance we check count value, OrderByMarginImpact and ClearFulfilled are expensive to call
        self.targets_collection.add_range(targets)
        if self.targets_collection.count > 0:
            for target in self.targets_collection.order_by_margin_impact(algorithm):
                # calculate remaining quantity to be ordered
                quantity = OrderSizing.get_unordered_quantity(algorithm, target)
                if quantity != 0 and algorithm.transactions.orders_count == 0:
                    bar = algorithm.securities[target.symbol].get_last_data()
                    algorithm.market_order(target.symbol, quantity)
                    # place some orders that won't fill, when the split comes in they'll get modified to reflect the split
                    algorithm.stop_market_order(target.symbol, -quantity, bar.low/2)
                    algorithm.limit_order(target.symbol, -quantity, bar.high*2)

            self.targets_collection.clear_fulfilled(algorithm)
```

Data Point Attributes

The US Equity Security Master dataset provides `Split`, `Dividend`, `Delisting`, and `SymbolChangedEvent` objects.

Split Attributes

When a split or merger occurs, we pass the previous `Symbol` data into your algorithm. `Split` objects have the following attributes:

Dividend Attributes

Dividend events are triggered on the payment date. `Dividend` objects have the following attributes:

Delisting Attributes

When a security is delisted, we notify your algorithm. `Delisting` objects have the following attributes:

SymbolChangedEvent Attributes

When a security changes their ticker, we notify your algorithm. `SymbolChangedEvent` objects have the following attributes:

QuantConnect

US Future Option Universe

Introduction

The US Future Option Universe dataset by QuantConnect lists the available US Future Options contracts and the current open interest. The data covers 15 Monthly Future contracts, starts in January 2012, and is delivered on a daily update frequency. This dataset is created by monitoring the trading activity on the CME, CBOT, NYMEX, and COMEX markets.

The US Future Option Universe dataset depends on the US Future Universe dataset because the US Future Universe dataset contains the universe of underlying Futures contracts. This dataset also depends on the US Futures Security Master dataset because the US Futures Security Master dataset contains information on symbol changes of the contracts.

This dataset `does not` contain market data. For market data, see US Future Options by AlgoSeek .

For more information about the US Future Option Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US Future Options Universe dataset:

```
future = self.add_future(Futures.Metals.GOLD, Resolution.MINUTE)
future.set_filter(0, 90)
self.add_future_option(future.symbol, lambda universe: universe.strikes(-1, 1))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012
Asset Coverage	15 Monthly Future Contracts. Standard expires only. No weeklies or 0DTE contracts.
Data Density	Dense
Resolution	Daily
Timezone	New York
Market Hours	Regular and Extended

Requesting Data

To add US Future Options Universe data to your algorithm, call the `add_future_option` method. To define which contracts should be in your universe, specify the filter when requesting the Future Option data.

The `add_future_option` method provides a daily stream of Option chain data. To get the most recent daily chain, call the `option_chain` method with the underlying Future Symbol. The `option_chain` method returns data on all the tradable contracts, not just the contracts that pass your universe filter.

```
class USFutureOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True

        self.future = self.add_future(Futures.Metals.GOLD, Resolution.MINUTE)
        self.future.set_filter(0, 90)
        # Set our strike/expiry filter for this option chain
        self.add_future_option(self.future.symbol, self._option_filter)

    def on_data(self, slice: Slice) -> None:
        # Get the entire Option chain for the current day.
        symbol = Symbol.create_canonical_option(self.future.mapped)
        chain = self.option_chain(symbol, flatten=True).data_frame

    def _option_filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Contracts can be filtered by strike, and expiration
        return universe.strikes(-1, 1)
```

The Future resolution must be less than or equal to the Future Option resolution. For example, if you set the Future resolution to minute, then you must set the Future Option resolution to minute, hour, or daily.

For more information about creating US Future Option universes, see Future Options .

Accessing Data

For information about accessing US Future Options Universe data, see Future Options .

Historical Data

You can get historical US Future Options Universe data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Future Options Universe data in an algorithm, call the `history` method with the canonical mapped Future Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter. If there is no data in the period you request, the history result is empty.

```
future_option_symbol = Symbol.create_canonical_option(self.future.mapped)
# DataFrame
history_df = self.history(future_option_symbol, timedelta(10), flatten=True)
# OptionUniverse objects
history = self.history[OptionUniverse](future_option_symbol, timedelta(10))
```

For more information about historical US Future Options Universe data in algorithms, see Historical Data .

Historical Data In Research

To get historical US Future Options Universe data in the Research Environment, call the `history` method with the canonical Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter.

```
qb = QuantBook()
future = qb.add_future(Futures.Metals.GOLD, Resolution.MINUTE)
future.set_filter(0, 90)
symbol = Symbol.create_canonical_option(future.mapped)
history = qb.history(symbol, datetime(2020, 6, 1), datetime(2020, 6, 5), flatten=True)
```

For more information about historical Future Options Universe data in the Research Environment, see Universes .

Supported Assets

The following list shows the available (15) Futures Options:

- `Futures.Energy.CRUDE_OIL_WTI` : Crude Oil WTI Futures (NYMEX: LO | Underlying: CL)
- `Futures.Energy.GASOLINE` : Gasoline RBOB Futures (NYMEX: OB | Underlying: RB)
- `Futures.Energy.HEATING_OIL` : Heating Oil Futures (NYMEX: OH | Underlying: HO)

- `Futures.Energy.NATURAL_GAS` : Natural Gas Futures (NYMEX: ON | Underlying: NG)
- `Futures.Financials.Y_10_TREASURY_NOTE` : 10Y U.S. Treasury Note Futures (CBOT: OZN | Underlying: ZN)
- `Futures.Financials.Y_2_TREASURY_NOTE` : 2Y U.S. Treasury Note Futures (CBOT: OZT | Underlying: ZT)
- `Futures.Financials.Y_30_TREASURY_BOND` : 30Y U.S. Treasury Bond Futures (CBOT: OZB | Underlying: ZB)
- `Futures.Grains.CORN` : Corn Futures (CBOT: OZC | Underlying: ZC)
- `Futures.Grains.SOYBEANS` : Soybeans Futures (CBOT: OZS | Underlying: ZS)
- `Futures.Grains.WHEAT` : Default wheat contract is SRWWheat (CBOT: OZW | Underlying: ZW)
- `Futures.Indices.NASDAQ_100_E_MINI` : E-mini NASDAQ 100 Futures (CME: NQ)
- `Futures.Indices.SP_500_E_MINI` : E-mini S&P 500 Futures (CME: ES)
- `Futures.Metals.COPPER` : Copper Futures (COMEX: HXE | Underlying: HG)
- `Futures.Metals.GOLD` : Gold Futures (COMEX: OG | Underlying: GC)
- `Futures.Metals.SILVER` : Silver Futures (COMEX: SO | Underlying: SI)

Example Applications

The US Future Options dataset enables you to accurately design Future Option strategies. Examples include the following strategies:

- Selling out of the money Future Option contracts to collect the premium that the Option buyer pays
- Buying put Options to hedge against downward price movement in Future contracts you bought
- Exploiting arbitrage opportunities that arise when the price of Option contracts deviate from their theoretical value

Classic Algorithm Example

The following example demonstrates a bull call spread Option strategy using universe filtering.

```
PY
from AlgorithmImports import *


class FutureOptionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Filter the underlying continuous Futures to narrow the FOP spectrum.
        self.underlying = self.add_future(Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        self.underlying.set_filter(0, 182)
        # Use CallSpread filter to obtain the 2 best-matched contracts that forms a call spread.
        # It simplifies from further filtering and reduce computation on redundant subscription.
        self.add_future_option(self.underlying.symbol, lambda u: u.call_spread(5, 5, -5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return
        # Create canonical symbol for the mapped future contract, since we need that to access the option chain.
        symbol = Symbol.create_canonical_option(self.underlying.mapped)

        # Get option chain data for the mapped future only.
        # It requires 2 contracts with different strikes to form a call spread, so we make sure at least 2 contracts are present.
        chain = slice.option_chains.get(symbol)
        if not chain or len(list(chain)) < 2:
            return

        # Separate the contracts by strike, as we need to access their strike.
        expiry = min([x.expiry for x in chain])
        sorted_by_strike = sorted([x.strike for x in chain])
        itm_strike = sorted_by_strike[0]
        otm_strike = sorted_by_strike[-1]

        # Use abstraction method to order a bull call spread to avoid manual error.
        option_strategy = OptionStrategies.bull_call_spread(symbol, itm_strike, otm_strike, expiry)
        self.buy(option_strategy, 1)
```

Data Point Attributes

The US Future Options Universe dataset provides `OptionFilterUniverse` and `OptionUniverse` objects.

OptionFilterUniverse Attributes

`OptionFilterUniverse` objects have the following attributes:

OptionUniverse Attributes

`OptionUniverse` objects have the following attributes:

QuantConnect

US Future Universe

Introduction

The US Future Universe dataset by QuantConnect lists the available US Future contracts, their daily trading volume, and Open Interest. The data covers the 157 most liquid contracts, starts in May 2009, and is delivered on daily frequency. This dataset is created by monitoring the trading activity on the CFE, CBOT, CME, COMEX, NYMEX, and ICE *.

This dataset depends on the US Futures Security Master dataset because the US Futures Security Master dataset contains information on symbol changes of the contracts.

This dataset `does not` contain market data. For market data, see US Futures by AlgoSeek .

For more information about the US Future Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US Future Universe dataset:

```py
future = self.add_future(Futures.Metals.GOLD)
future.set_filter(0, 90)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	May 2009
Asset Coverage	157 Futures
Data Density	Dense
Resolution	Daily
Timezone	• Chicago (CFE, CME or CBOT) • New York (ICE, NYMEX or COMEX)
Market Hours	Regular and Extended

Requesting Data

To add US Future Universe data to your algorithm, call the `add_future` method. Save a reference to the Future object so you can access the data later in your algorithm. To define which contracts should be in your universe, specify the filter when requesting the Future data.

The `add_future` method provides a daily stream of Future chain data. To get the most recent daily chain, call the `futures_chain` method with the underlying Future Symbol. The `futures_chain` method returns data on all the tradable contracts, not just the contracts that pass your universe filter.

```
class USFutureDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        self._future = self.add_future(Futures.Metals.GOLD)
        # Set our contract filter for this Future chain.
        self._future.set_filter(lambda universe: universe.standards_only().front_month())
        # Get the entire Futures chain for the current day.
        chain = self.futures_chain(self._future.symbol, flatten=True).data_frame
```

For more information about creating US Future universes, see Futures .

Accessing Data

For information about accessing US Future Universe data, see Futures .

Historical Data

You can get historical US Future Universe data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Future Universe data in an algorithm, call the `history` method with continuous Future `Symbol` and a lookback period. This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. If there is no data for the period you requested, the history result is empty.

```
# Add the Future and save a reference to it.
future = self.add_future(Futures.Metals.GOLD)

# DataFrame example where the columns are the FutureUniverse attributes:
history_df = self.history(FutureUniverse, future.symbol, 5, flatten=True)

# Series example where the values are lists of FutureUniverse objects:
history_series = self.history(FutureUniverse, future.symbol, 5, flatten=False)

# FutureUniverse objects example:
history = self.history[FutureUniverse](future.symbol, 5)
```

For more information about historical US Future Universe data in algorithms, see Contracts .

Historical Data In Research

To get historical US Future Universe data in the Research Environment, call the `history` method with continuous Future `Symbol` and a time period. This method returns the all the available contracts for each trading day, not the subset of contracts that pass your universe filter. If there is no data for the period you requested, the history result is empty.

```
# Add the Future and save a reference to it.
future = qb.add_future(Futures.Metals.GOLD)

# DataFrame example where the columns are the FutureUniverse attributes:
history_df = qb.history(FutureUniverse, future.symbol, datetime(2025, 1, 1), datetime(2025, 4, 1), flatten=True)

# Series example where the values are lists of FutureUniverse objects:
history_series = qb.history(FutureUniverse, future.symbol, 5, flatten=False)

# FutureUniverse objects example:
history = qb.history[FutureUniverse](future.symbol, datetime(2025, 1, 1), datetime(2025, 4, 1))
```

For more information about historical US Future Universe data in the Research Environment, see Daily Prices History .

Supported Assets

The following list shows the available (157) Futures:

- `Futures.Currencies.AUD` : Australian Dollar Futures (CME: 6A)
- `Futures.Currencies.AUDCAD` : Australian Dollar/Canadian Dollar Futures (CME: ACD)

- `Futures.Currencies.AUDJPY` : Australian Dollar/Japanese Yen Futures (CME: AJY)
- `Futures.Currencies.AUDNZD` : Australian Dollar/New Zealand Dollar Futures (CME: ANE)
- `Futures.Currencies.BRL` : Brazillian Real Futures (CME: 6L)
- `Futures.Currencies.BTC` : Bitcoin Futures (CME: BTC)
- `Futures.Currencies.BTIC_MICRO_BTC` : BTIC on Micro Bitcoin Futures (CME: MIB)
- `Futures.Currencies.BTIC_MICRO_ETHER` : BTIC on Micro Ether Futures (CME: MRB)
- `Futures.Currencies.CAD` : Canadian Dollar Futures (CME: 6C)
- `Futures.Currencies.CADJPY` : Canadian Dollar/Japanese Yen Futures (CME: CJY)
- `Futures.Currencies.CHF` : Swiss Franc Futures (CME: 6S)
- `Futures.Currencies.ETH` : Ether Futures (CME: ETH)
- `Futures.Currencies.EUR` : Euro FX Futures (CME: 6E)
- `Futures.Currencies.EURAUD` : Euro/Australian Dollar Futures (CME: EAD)
- `Futures.Currencies.EURCAD` : Euro/Canadian Dollar Futures (CME: ECD)
- `Futures.Currencies.EURO_FX_EMINI` : E-mini Euro FX Futures (CME: E7)
- `Futures.Currencies.EURSEK` : Euro/Swedish Krona Futures (CME: ESK)
- `Futures.Currencies.GBP` : British Pound Futures (CME: 6B)
- `Futures.Currencies.JAPANESE_YEN_EMINI` : E-mini Japanese Yen Futures (CME: J7)
- `Futures.Currencies.JPY` : Japanese Yen Futures (CME: 6J)
- `Futures.Currencies.MICRO_AUD` : Micro AUD/USD Futures (CME: M6A)
- `Futures.Currencies.MICRO_BTC` : Micro Bitcoin Futures (CME: MBT)
- `Futures.Currencies.MICRO_CAD` : Micro USD/CAD Futures (CME: M6C)
- `Futures.Currencies.MICRO_CADUSD` : Micro CAD/USD Futures (CME: MCD)
- `Futures.Currencies.MICRO_CHF` : Micro CHF/USD Futures (CME: MSF)
- `Futures.Currencies.MICRO_ETHER` : Micro Ether Futures (CME: MET)
- `Futures.Currencies.MICRO_EUR` : Micro EUR/USD Futures (CME: M6E)
- `Futures.Currencies.MICRO_GBP` : Micro GBP/USD Futures (CME: M6B)
- `Futures.Currencies.MICRO_INRUSD` : Micro INR/USD Futures (CME: MIR)
- `Futures.Currencies.MICRO_JPY` : Micro JPY/USD Futures (CME: MJY)
- `Futures.Currencies.MICRO_USDCHF` : Micro USD/CHF Futures (CME: M6S)
- `Futures.Currencies.MICRO_USDCNH` : Micro USD/CNH Futures (CME: MNH)
- `Futures.Currencies.MICRO_USDJPY` : Micro USD/JPY Futures (CME: M6J)
- `Futures.Currencies.MXN` : Mexican Peso Futures (CME: 6M)
- `Futures.Currencies.NZD` : New Zealand Dollar Futures (CME: 6N)
- `Futures.Currencies.RUB` : Russian Ruble Futures (CME: 6R)
- `Futures.Currencies.STANDARD_SIZE_USD_OFFSHORE_RMBCNH` : Standard-Size USD/Offshore RMB (CNH) Futures (CME: CNH)
- `Futures.Currencies.ZAR` : South African Rand Futures (CME: 6Z)
- `Futures.Energy.ARGUS_LL_SVS_WTI_ARGUS_TRADE_MONTH` : Argus LLS vs. WTI (Argus) Trade Month Futures (NYMEX: AE5)
- `Futures.Energy.ARGUS_PROPANE_FAR_EAST_INDEX` : Argus Propane Far East Index Futures (NYMEX: A7E)
- `Futures.Energy.ARGUS_PROPANE_SAUDI_ARAMCO` : Argus Propane (Saudi Aramco) Futures (NYMEX: A9N)
- `Futures.Energy.BRENT_CRUDE_OIL_VS_DUBAI_CRUDE_OIL_PLATTS` : Brent Crude Oil vs. Dubai Crude Oil (Platts) Futures (NYMEX: ADB)
- `Futures.Energy.BRENT_LAST_DAY_FINANCIAL` : Brent Last Day Financial Futures (NYMEX: BZ)
- `Futures.Energy.CHICAGO_ETHANOL_PLATTS` : Chicago Ethaanol (Platts) Futures (NYMEX: CU)
- `Futures.Energy.CONWAY_PROPANE_OPIS` : Conway Propane (OPIS) Futures (NYMEX: A8K)
- `Futures.Energy.CRUDE_OIL_WTI` : Crude Oil WTI Futures (NYMEX: CL)
- `Futures.Energy.DUBAI_CRUDE_OIL_PLATTS_FINANCIAL` : Dubai Crude Oil (Platts) Financial Futures (NYMEX: DCB)
- `Futures.Energy.EAST_WEST_GASOLINE_SPREAD_PLATTS_ARGUS` : East-West Gasoline Spread (Platts-Argus) Futures (NYMEX: EWG)
- `Futures.Energy.EAST_WEST_NAPHTHA_JAPAN_C_FVS_CARGOES_CIFNWE_SPREAD_PLATTS` : East-West Naphtha: Japan C&F vs. Cargoes CIF NWE Spread (Platts) Futures (NYMEX: EWN)
- `Futures.Energy.ETHANOL` : Ethanol Futures (CBOT: EH)
- `Futures.Energy.ETHANOL_T_2_FOB_RDAM_INCLUDING_DUTY_PLATTS` : Ethanol T2 FOB Rdam Including Duty (Platts) Futures (NYMEX: AZ1)
- `Futures.Energy.EUROPEAN_NAPHTHA_PLATTS_CRACK_SPREAD` : European Naphtha (Platts) Crack Spread Futures (NYMEX: EN)
- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS` : European Propane CIF ARA (Argus) Futures (NYMEX: APS)

- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS_VS_NAPHTHA_CARGOES_CIFNWE_PLATTS` : European Propane CIF ARA (Argus) vs. Naphtha Cargoes CIF NWE (Platts) Futures (NYMEX: EPN)
- `Futures.Energy.FREIGHT_ROUTE_TC_14_BALTIC` : Freight Route TC14 (Baltic) Futures (NYMEX: FRC)
- `Futures.Energy.GASOLINE` : Gasoline RBOB Futures (NYMEX: RB)
- `Futures.Energy.GASOLINE_EUROBOB_OXY_NWE_BARGES_ARGUS` : Gasoline Euro-bob Oxy NWE Barges (Argus) Futures (NYMEX: B7H)
- `Futures.Energy.GROUP_THREE_SUBOCTANE_GASOLINE_PLATTS_VS_RBOB` : Group Three Sub-octane Gasoliine (Platts) vs. RBOB Futures (NYMEX: AA8)
- `Futures.Energy.GROUP_THREE_ULSD_PLATTS_VS_NY_HARBOR_ULSD` : Group Three ULSD (Platts) vs. NY Harbor ULSD Futures (NYMEX: AA6)
- `Futures.Energy.GULF_COAST_CBOB_GASOLINE_A_2_PLATTS_VS_RBOB_GASOLINE` : Gulf Coast CBOB Gasoline A2 (Platts) vs. RBOB Gasoline Futures (NYMEX: CRB)
- `Futures.Energy.GULF_COAST_HSFO_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Gulf Coast HSFO (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: GCU)
- `Futures.Energy.HEATING_OIL` : Heating Oil Futures (NYMEX: HO)
- `Futures.Energy.LOS_ANGELES_CARBOB_GASOLINE_OPI_SVS_RBOB_GASOLINE` : Los Angeles CARBOB Gasoline (OPIS) vs. RBOB Gasoline Futures (NYMEX: AJL)
- `Futures.Energy.LOS_ANGELES_CARB_DIESEL_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles CARB Diesel (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AKL)
- `Futures.Energy.LOS_ANGELES_JET_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles Jet (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AJS)
- `Futures.Energy.MARS_ARGUS_VS_WTI_FINANCIAL` : Mars (Argus) vs. WTI Financial Futures (NYMEX: AYX)
- `Futures.Energy.MARS_ARGUS_VS_WTI_TRADE_MONTH` : Mars (Argus) vs. WTI Trade Month Futures (NYMEX: AYV)
- `Futures.Energy.MICRO_CRUDE_OIL_WTI` : Micro WTI Crude Oil Futures (NYMEX: MCL)
- `Futures.Energy.MICRO_EUROPEAN_FOB_RDAM_MARINE_FUEL_ZERO_POINT_FIVE_PERCENT_BARGES_PLATTS` : Micro European FOB Rdam Marine Fuel 0.5% Barges (Platts) Futures (NYMEX: R5O)
- `Futures.Energy.MICRO_EUROPEAN_THREE_POINT_FIVE_PERCENT_OIL_BARGES_FOB_RDAM_PLATTS` : Micro European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: MEF)
- `Futures.Energy.MICRO_GASOIL_ZERO_POINT_ONE_PERCENT_BARGES_FOBARA_PLATTS` : Micro Gasoil 0.1% Barges FOB ARA (Platts) Futures (NYMEX: M1B)
- `Futures.Energy.MICRO_SINGAPORE_FOB_MARINE_FUEL_ZERO_POINT_FIVE_PERCET_PLATTS` : Micro Singapore FOB Marine Fuel 0.5% (Platts) Futures (NYMEX: S5O)
- `Futures.Energy.MICRO_SINGAPORE_FUEL_OIL_380_CST_PLATTS` : Micro Singapore Fuel Oil 380CST (Platts) Futures (NYMEX: MAF)
- `Futures.Energy.MINI_EUROPEAN_THREE_POINT_PERCENT_FIVE_FUEL_OIL_BARGES_PLATTS` : Mini European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: A0D)
- `Futures.Energy.MINI_SINGAPORE_FUEL_OIL_180_CST_PLATTS` : Mini Singapore Fuel Oil 180 cst (Platts) Futures (NYMEX: A0F)
- `Futures.Energy.MONT_BELVIEU_ETHANE_OPIS` : Mont Belvieu Ethane (OPIS) Futures (NYMEX: AC0)
- `Futures.Energy.MONT_BELVIEU_LDH_PROPANE_OPIS` : Mont Belvieu LDH Propane (OPIS) Futures (NYMEX: B0)
- `Futures.Energy.MONT_BELVIEU_NATURAL_GASOLINE_OPIS` : Mont Belvieu Natural Gasoline (OPIS) Futures (NYMEX: A7Q)
- `Futures.Energy.MONT_BELVIEU_NORMAL_BUTANE_OPIS` : Mont Belvieu Normal Butane (OPIS) Futures (NYMEX: AD0)
- `Futures.Energy.NATURAL_GAS` : Natural Gas Futures (NYMEX: NG)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_LAST_DAY_FINANCIAL` : Natural Gas (Henry Hub) Last-day Financial Futures (NYMEX: HH)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_PENULTIMATE_FINANCIAL` : Natural Gas (Henry Hub) Penultimate Financial Futures (NYMEX: HP)
- `Futures.Energy.ONE_PERCENT_FUEL_OIL_CARGOES_FOBNWE_PLATTS_VS_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : 1% Fuel Oil Cargoes FOB NWE (Platts) vs. 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: FSS)
- `Futures.Energy.PREMIUM_UNLEADED_GASOLINE_10_PPM_FOBMED_PLATTS` : Premium Unleaded Gasoline 10 ppm FOB MED (Platts) Futures (NYMEX: A3G)
- `Futures.Energy.PROPANE_NON_LDH_MONT_BELVIEU_OPIS` : Propane Non-LDH Mont Belvieu (OPIS) Futures (NYMEX: A1R)
- `Futures.Energy.RBOB_GASOLINE_CRACK_SPREAD` : RBOB Gasoline Crack Spread Futures (NYMEX: ARE)
- `Futures.Energy.RBOB_GASOLINE_VS_EUROBOB_OXY_NWE_BARGES_ARGUS_THREE_HUNDRED_FIFTY_THOUSAND_GALLONS` : RBOB Gasoline vs. Euro-bob Oxy NWE Barges (Argus) (350,000 gallons) Futures (NYMEX: EXR)
- `Futures.Energy.SINGAPORE_FUEL_OIL_380_CST_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Singapore Fuel Oil 380 cst (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: EVC)
- `Futures.Energy.SINGAPORE_GASOIL_PLATTS_VS_LOW_SULPHUR_GASOIL_FUTURES` : Singapore Gasoil (Platts) vs. Low Sulphur Gasoil Futures (NYMEX: AGA)

- `Futures.Energy.SINGAPORE_MOGAS_92_UNLEADED_PLATTS_BRENT_CRACK_SPREAD` : Singapore Mogas 92 Unleaded (Platts) Brent Crack Spread Futures (NYMEX: D1N)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD` : 3.5% Fuel Oil Barges FOB Rdam (Platts) Crack Spread Futures (NYMEX: FO)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD_1000_MT` : 3.5% Fuel Oil Barges FOB Rdam (Platts) Crack Spread (1000mt) Futures (NYMEX: BOO)
- `Futures.Energy.WTI_BRENT_FINANCIAL` : WTI-Brent Financial Futures (NYMEX: BK)
- `Futures.Energy.WTI_FINANCIAL` : WTI Financial Futures (NYMEX: CSX)
- `Futures.Energy.WTI_HOUSTON_ARGUS_VS_WTI_TRADE_MONTH` : WTI Houston (Argus) vs. WTI Trade Month Futures (NYMEX: HTT)
- `Futures.Energy.WTI_HOUSTON_CRUDE_OIL` : WTI Houston Crude Oil Futures (NYMEX: HCL)
- `Futures.Financials.EURO_DOLLAR` : EuroDollar Futures (CME: GE)
- `Futures.Financials.FIVE_YEAR_USDMAC_SWAP` : 5-Year USD MAC Swap Futures (CBOT: F1U)
- `Futures.Financials.MICRO_Y_10_TREASURY_NOTE` : Micro 10-Year Yield Futures (CBOT: 10Y)
- `Futures.Financials.MICRO_Y_2_TREASURY_BOND` : Micro 2-Year Yield Futures (CBOT: 2YY)
- `Futures.Financials.MICRO_Y_30_TREASURY_BOND` : Micro 30-Year Yield Futures (CBOT: 30Y)
- `Futures.Financials.MICRO_Y_5_TREASURY_BOND` : Micro 5-Year Yield Futures (CBOT: 5YY)
- `Futures.Financials.ULTRA_TEN_YEAR_US_TREASURY_NOTE` : Ultra 10-Year U.S. Treasury Note Futures (CBOT: TN)
- `Futures.Financials.ULTRA_US_TREASURY_BOND` : Ultra U.S. Treasury Bond Futures (CBOT: UB)
- `Futures.Financials.Y_10_TREASURY_NOTE` : 10Y U.S. Treasury Note Futures (CBOT: ZN)
- `Futures.Financials.Y_2_TREASURY_NOTE` : 2Y U.S. Treasury Note Futures (CBOT: ZT)
- `Futures.Financials.Y_30_TREASURY_BOND` : 30Y U.S. Treasury Bond Futures (CBOT: ZB)
- `Futures.Financials.Y_5_TREASURY_NOTE` : 5Y U.S. Treasury Note Futures (CBOT: ZF)
- `Futures.Forestry.LUMBER` : Lumber Futures (CME: LBR)
- `Futures.Forestry.RANDOM_LENGTH_LUMBER` : Random Length Lumber Futures (CME: LBS)
- `Futures.Grains.BLACK_SEA_CORN_FINANCIALLY_SETTLED_PLATTS` : Black Sea Corn Financially Settled (Platts) Futures (CBOT: BCF)
- `Futures.Grains.BLACK_SEA_WHEAT_FINANCIALLY_SETTLED_PLATTS` : Black Sea Wheat Financially Settled (Platts) Futures (CBOT: BWF)
- `Futures.Grains.CORN` : Corn Futures (CBOT: ZC)
- `Futures.Grains.HRW_WHEAT` : KC HRW Wheat Futures (CBOT: KE)
- `Futures.Grains.OATS` : Oats Futures (CBOT: ZO)
- `Futures.Grains.SOYBEANS` : Soybeans Futures (CBOT: ZS)
- `Futures.Grains.SOYBEAN_MEAL` : Soybean Meal Futures (CBOT: ZM)
- `Futures.Grains.SOYBEAN_OIL` : Soybean Oil Futures (CBOT: ZL)
- `Futures.Grains.WHEAT` : Default wheat contract is SRWWheat (CBOT: ZW)
- `Futures.Indices.BLOOMBERG_COMMODITY_INDEX` : Bloomberg Commodity Index Futures (CBOT: AW)
- `Futures.Indices.DOW_30_E_MINI` : E-mini Dow Indu 30 Futures (CBOT: YM)
- `Futures.Indices.DOW_JONES_REAL_ESTATE` : Dow Jones Real Estate futures on CME (CME: RX)
- `Futures.Indices.FTSE_EMERGING_EMINI` : E-mini FTSE Emerging Index Futures (CME: EI)
- `Futures.Indices.MICRO_DOW_30_E_MINI` : Micro E-mini Dow Jones Industrial Average Index Futures (CBOT: MYM)
- `Futures.Indices.MICRO_NASDAQ_100_E_MINI` : Micro E-mini Nasdaq-100 Index Futures (CME: MNQ)
- `Futures.Indices.MICRO_RUSSELL_2000_E_MINI` : Micro E-mini Russell 2000 Index Futures (CME: M2K)
- `Futures.Indices.MICRO_SP_500_E_MINI` : Micro E-mini S&P 500 Index Futures (CME: MES)
- `Futures.Indices.NASDAQ_100_BIOTECHNOLOGY_E_MINI` : E-mini Nasdaq-100 Biotechnology Index Futures (CME: BIO)
- `Futures.Indices.NASDAQ_100_E_MINI` : E-mini NASDAQ 100 Futures (CME: NQ)
- `Futures.Indices.NIKKEI_225_DOLLAR` : Nikkei-225 Dollar Futures (CME: NKD)
- `Futures.Indices.NIKKEI_225_YEN_CME` : Nikkei-225 Yen denominated Futures on CME (CME: NIY)
- `Futures.Indices.RUSSELL_1000_E_MINI` : E-mini Russell 1000 futures on CME (CME: RS1)
- `Futures.Indices.RUSSELL_2000_E_MINI` : E-mini Russell 2000 Futures (CME: RTY)
- `Futures.Indices.SPGSCI_COMMODITY` : S&P-GSCI Commodity Index Futures (CME: GD)
- `Futures.Indices.SP_400_MID_CAP_EMINI` : E-mini S&P MidCap 400 Futures (CME: EMD)
- `Futures.Indices.SP_500_ANNUAL_DIVIDEND_INDEX` : (CME: SDA)
- `Futures.Indices.SP_500_E_MINI` : E-mini S&P 500 Futures (CME: ES)
- `Futures.Indices.TOPIXYEN` : YEN Denominated Topix Index Futures on CME (CME: TPY)

- `Futures.Indices.USD_DENOMINATED_IBOVESPA` : USD-Denominated Ibovespa Index Futures (CME: IBV)
- `Futures.Indices.VIX` : CBOE Volatility Index Futures (CFE: VX)
- `Futures.Meats.FEEDER_CATTLE` : Feeder Cattle Futures (CME: GF)
- `Futures.Meats.LEAN_HOGS` : Lean Hogs Futures (CME: HE)
- `Futures.Meats.LIVE_CATTLE` : Live Cattle Futures (CME: LE)
- `Futures.Metals.ALUMINIUM_EUROPEAN_PREMIUM_DUTY_PAID_METAL_BULLETIN` : Aluminium European Premium Duty-Paid (Metal Bulletin) Futures (COMEX: EDP)
- `Futures.Metals.ALUMINUM_MWUS_TRANSACTION_PREMIUM_PLATTS_25_MT` : Aluminum MW U.S. Transaction Premium Platts (25MT) Futures (COMEX: AUP)
- `Futures.Metals.COPPER` : Copper Futures (COMEX: HG)
- `Futures.Metals.GOLD` : Gold Futures (COMEX: GC)
- `Futures.Metals.MICRO_GOLD` : Micro Gold Futures (COMEX: MGC)
- `Futures.Metals.MICRO_GOLD_TAS` : Micro Gold TAS Futures (COMEX: MGT)
- `Futures.Metals.MICRO_PALLADIUM` : Micro Palladium Futures (NYMEX: PAM)
- `Futures.Metals.MICRO_SILVER` : Micro Silver Futures (COMEX: SIL)
- `Futures.Metals.PALLADIUM` : Palladium Futures (NYMEX: PA)
- `Futures.Metals.PLATINUM` : Platinum Futures (NYMEX: PL)
- `Futures.Metals.SILVER` : Silver Futures (COMEX: SI)
- `Futures.Metals.US_MIDWEST_DOMESTIC_HOT_ROLLED_COIL_STEEL_CRU_INDEX` : U.S. Midwest Domestic Hot-Rolled Coil Steel (CRU) Index Futures (NYMEX: HRC)
- `Futures.Softs.SUGAR_11` : Sugar #11 Futures ICE (ICE: SB)
- `Futures.Softs.SUGAR_11_CME` : Sugar #11 Futures CME (NYMEX: YO)

Example Applications

The US Future Universe dataset enables you to accurately design Futures strategies. Examples include the following strategies:

- Buying the Futures contract with the most open interest to reduce slippage and market impact
- Trading bull calendar spreads to reduce volatility and margin requirements
- Trading bull calendar spreads to speculate contango

Classic Algorithm Example

The following example algorithm selects and buys the front-month Mini Gold Futures contract and sells the front-month Micro Gold Futures contract. When the front-month contract changes, the algorithm rebalances the portfolio.

```py
from AlgorithmImports import *

class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)
        self.universe_settings.asynchronous = True
        # Requesting data on gold and micro-gold contracts.
        # Filter the universe to trade only the contracts expiring within 90 days to ensure liquidity.
        self.mini_gold = self.add_future(Futures.Metals.GOLD)
        self.mini_gold.set_filter(0, 90)

        self.micro_gold = self.add_future(Futures.Metals.MICRO_GOLD)
        self.micro_gold.set_filter(0, 90)

        # Save a cache of the mapped symbol to trade.
        self.contract = {self.mini_gold.symbol: None, self.micro_gold.symbol: None}

    def on_data(self, slice: Slice) -> None:
        for symbol, chain in slice.future_chains.items():
            if symbol in self.contract:
                # Select the contract with the greatest open interest to trade with the most efficiency.
                most_liquid_contract = sorted(chain, key=lambda contract: contract.open_interest, reverse=True)[0]

                if self.contract[symbol] is None or most_liquid_contract.symbol != self.contract[symbol].symbol:
                    # Liquidate any unmapped contracts.
                    if self.contract[symbol] is not None:
                        self.liquidate(self.contract[symbol].symbol)
                    self.contract[symbol] = most_liquid_contract

                    # Buy mini-gold and short micro-gold contracts as planned.
                    if symbol == self.mini_gold.symbol:
                        self.market_order(self.contract[symbol].symbol, 1)
                    elif symbol == self.micro_gold.symbol:
                        self.market_order(self.contract[symbol].symbol, -1)
```

Framework Algorithm Example

The following example algorithm selects and buys the front-month Mini Gold Futures contract and sells the front-month Micro Gold Futures contract. When the front-month contract changes, the algorithm rebalances the portfolio.

```py
from AlgorithmImports import *

class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Set up an universe selection model that selects the front month contract
        self.set_universe_selection(FrontMonthFutureUniverseSelectionModel())
        self.add_alpha(ConstantFuturesAlphaModel())
        # A portfolio construction model that only order a single share per insight signal
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class FrontMonthFutureUniverseSelectionModel(FutureUniverseSelectionModel):
    def __init__(self,) -> None:
        # Daily updating with select_future_chain_symbols function
        super().__init__(timedelta(1), self.select_future_chain_symbols)

    def select_future_chain_symbols(self, utcTime: datetime) -> List[Symbol]:
        # Select gold and micro gold contracts for the strategy need
        future_pairs = [
            (Futures.Metals.GOLD, Market.COMEX),
            (Futures.Metals.MICRO_GOLD, Market.COMEX)
        ]
        return [Symbol.create(pair[0], SecurityType.FUTURE, pair[1]) for pair in future_pairs]

    def filter(self, filter: FutureFilterUniverse) -> FutureFilterUniverse:
        # Filter only front month contract for liquidity and most informed information
        return filter.front_month().only_apply_filter_at_market_open()

class ConstantFuturesAlphaModel(AlphaModel):
    # Long gold and short micro gold in this strategy
    long_symbol = Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
    short_symbol = Symbol.create(Futures.Metals.MICRO_GOLD, SecurityType.FUTURE, Market.COMEX)

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        if algorithm.portfolio.invested:
            return []

        insights = []
        # For both gold and micro gold, select the front month contract (only contract) in the chain
        for kvp in slice.future_chains:
            chain = [contract for contract in kvp.Value]
            contract = chain[0]

            # Long gold and short micro gold as planned
            if kvp.Key == self.long_symbol:
                insights.append(Insight.price(contract.symbol, contract.expiry + timedelta(days=1), InsightDirection.UP))
            elif kvp.Key == self.short_symbol:
                insights.append(Insight.price(contract.symbol, contract.expiry + timedelta(days=1), InsightDirection.DOWN))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = algorithm.history(security.symbol, 10, Resolution.MINUTE)
            algorithm.debug(f"We got {len(history)} from our history request for {security.symbol}")

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Single share only using integer portfolio target
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Future Universe dataset provides `FutureFilterUniverse` and `FutureUniverse` objects.

FutureFilterUniverse Attributes

`FutureFilterUniverse` objects have the following attributes:

FutureUniverse Attributes

`FutureUniverse` objects have the following attributes:

QuantConnect

US Futures Security Master

Introduction

The US Futures Security Master dataset by QuantConnect provides mapping reference data for the most liquid contracts of the CME Group exchanges, calculated with popular rolling techniques. The data covers 157 root Future contracts, starts in May 2009, and is delivered on a daily frequency with a zip file with all the contract mappings. This dataset is created by daily processing of the US historical Future chains.

This dataset, paired with the US Futures dataset, supports the following rolling techniques: forward panama canal, backwards panama canal, and backwards ratio. You can set the specific rolling date to occur on the last trading day, first day month, or the day when the contract with the greatest open interest changes.

VIX Futures don't support continous contract rolling with open interest.

This is `NOT` the underlying Futures data (US Futures dataset), which you need to purchase separately with a license from AlgoSeek. This security master dataset is required to purchase the US Futures or US Future Options datasets.

For more information about the US Futures Security Master dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

You don't need any special code to utilize the US Futures Security Master. It automatically loads when you request US Futures data .

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	May 2009
Asset Coverage	157 Liquid Futures
Data Density	Regular
Resolution	Daily

VIX Futures don't support continous contract rolling with open interest.

Key Concept

The US Futures Security Master lets you to construct continuous Futures, allowing the access of normalized historical data of the underlying assets, as well as trading the “lead” Future contracts for those assets.

Continuous Futures refer to sets of rolling lead Future contracts during their actively trading periods. Since Future contracts expire at their maturities, to analyze the historical price of a Future and to apply technical indicators, you need the continuous Future price series.

To access the continuous Future, use the Future's `symbol` property.

```
# Set the data normalization mode and data mapping mode to select the contract as the continuous contract with price adjustment for
smooth price series
future = self.add_future(Futures.Energy.CRUDE_OIL_WTI,
    data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    contract_depth_offset=0)
self.continuous_contract_symbol = future.symbol
```

The `dataNormalizationMode` and `dataMappingMode` arguments makes the transition of the underlying contracts seemless. However, the Future `Symbol` doesn't map to an underlying Future contract. It works fine to trade within backtests, but could be subjected to friction costs during live trading since the order price could be a normalized price. For more information about this topic, see the `Live Trading Considerations` section.

Data Normalization Modes

The data normalization mode defines how the price series of two contracts are stitched together when the contract rollovers occur. The `DataNormalizatoinMode` enumeration has the following members available for continuous contracts:

If you use a data normalization mode that's not in the preceding list, LEAN automatically converts it to `DataNormalizationMode.BackwardsRatio` .

Data Mapping Modes

The data mapping mode defines when contract rollovers occur. The `DataMappingMode` enumeration has the following members:

VIX Futures don't support continous contract rolling with `DataMappingMode.OPEN_INTEREST` and `DataMappingMode.OPEN_INTEREST_ANNUAL` .

Tracking Contract Changes

As the contracts roll over, the `mapped` property of the `Future` object references the next contract in the series and you receive a `SymbolChangedEvent` . To get the current `Symbol` change events, index the `symbol_changed_events` property of the current `Slice` with the continuous Futures `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your Future at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    # Get Symbol Change Event of the Continuous Future (change in mapped contract)
    changed_event = slice.symbol_changed_events.get(self.continuous_future_symbol)
    if changed_event:
        old_symbol = changed_event.old_symbol
        new_symbol = self.add_future_contract(changed_event.new_symbol).symbol
        tag = f"Rollover - Symbol changed at {self.time}: {old_symbol} -> {new_symbol}"
        quantity = self.portfolio[old_symbol].quantity

        # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
        self.liquidate(old_symbol, tag = tag)
        if quantity != 0: self.market_order(new_symbol, quantity, tag = tag)
        self.log(tag)
```

`SymbolChangedEvent` objects have the following attributes:

Live Trading Considerations

You can trade continuous Futures, but the continuous Future `Symbol` doesn't map to a single underlying Future contract. Instead, it represents a set of rolling contracts. Thus, the prices could be frictional during a contract rollover, which could be catastrophic in live trading! For live trading, you should place your orders directly on the underlying contracts. To get the current underlying contract in the continuous Future series, use the `mapped` property.

```
# Get the mapped symbol
current_contract = self.continuous_contract.mapped
self.buy(current_contract, 1)
```

Data Format

If you download the files in the US Futures Security Master, you get a factor file and a map file for each of the exchanges with supported continuous Futures. To view the files, see the */data/future/<exchange_name>* directory under your LEAN CLI base directory.

For the factor file, it is a `.zip` collection of REST API styled `.csv` files for each Future `Symbol` , including the date, scaling factors for each type of data

normalization and the data mapping mode that indicates the Symbol changing event is on that day for that mapping mode. The following line is an example line in the `.csv` file:

```
{"Date":"2009-10-31T00:00:00","BackwardsRatioScale":[0.99141630901287553648068669 53,1.0,1.0],"BackwardsPanamaCanalScale":[-2.0,0.0,0.0],"ForwardPanamaCanalScale":[0.0,0.0,0.0],"DataMappingMode":1}
```

For the map file, it is a `.zip` collection of `.csv` files for each Future `Symbol` , including the date, new underlying contract Symbol, the exchange code, and the data mapping mode that indicates the Symbol changing event is on that day for that mapping mode. The following line is an example line in the `.csv` file:

```
20091130,aw uii3j0m6zbj9,CBOT,1
```

Historical Data

You can get historical US Future Security Master data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Future Security Master data in an algorithm, call the `history` method with continuous Future `Symbol` and a lookback period. This method returns the old and new contract of each rollover. If there is no data for the period you requested, the history result is empty.

```py
# Add the Future and define its rollover settings.
future = self.add_future(
    Futures.Indices.SP_500_E_MINI,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
    contract_depth_offset=0
)

# DataFrame example where the columns are the SymbolChangedEvent attributes:
history_df = self.history(SymbolChangedEvent, future.symbol, timedelta(365))

# SymbolChangedEvent objects example:
history = self.history[SymbolChangedEvent](future.symbol, timedelta(365))
```

For more information about historical US Future Security Master data in algorithms, see Symbol Changes .

Historical Data In Research

To get historical US Future Security Master data in the Research Environment, call the `history` method with continuous Future `Symbol` and a time period. This method returns the old and new contract of each rollover. If there is no data for the period you requested, the history result is empty.

```py
# Add the Future and define its rollover settings.
future = qb.add_future(
    Futures.Indices.SP_500_E_MINI,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
    contract_depth_offset=0
)

# DataFrame example where the columns are the SymbolChangedEvent attributes:
history_df = qb.history(SymbolChangedEvent, future.symbol, datetime(2023, 1, 1), datetime(2025, 1, 1))

# SymbolChangedEvent objects example:
history = qb.history[SymbolChangedEvent](future.symbol, datetime(2023, 1, 1), datetime(2025, 1, 1))
```

For more information about historical US Future Security Master data in the Research Environment, see Symbol Changes History .

Supported Assets

The following list shows the available (157) Futures:

- `Futures.Currencies.AUD` : Australian Dollar Futures (CME: 6A)
- `Futures.Currencies.AUDCAD` : Australian Dollar/Canadian Dollar Futures (CME: ACD)
- `Futures.Currencies.AUDJPY` : Australian Dollar/Japanese Yen Futures (CME: AJY)
- `Futures.Currencies.AUDNZD` : Australian Dollar/New Zealand Dollar Futures (CME: ANE)
- `Futures.Currencies.BRL` : Brazillian Real Futures (CME: 6L)
- `Futures.Currencies.BTC` : Bitcoin Futures (CME: BTC)
- `Futures.Currencies.BTIC_MICRO_BTC` : BTIC on Micro Bitcoin Futures (CME: MIB)

- `Futures.Currencies.BTIC_MICRO_ETHER` : BTIC on Micro Ether Futures (CME: MRB)
- `Futures.Currencies.CAD` : Canadian Dollar Futures (CME: 6C)
- `Futures.Currencies.CADJPY` : Canadian Dollar/Japanese Yen Futures (CME: CJY)
- `Futures.Currencies.CHF` : Swiss Franc Futures (CME: 6S)
- `Futures.Currencies.ETH` : Ether Futures (CME: ETH)
- `Futures.Currencies.EUR` : Euro FX Futures (CME: 6E)
- `Futures.Currencies.EURAUD` : Euro/Australian Dollar Futures (CME: EAD)
- `Futures.Currencies.EURCAD` : Euro/Canadian Dollar Futures (CME: ECD)
- `Futures.Currencies.EURO_FX_EMINI` : E-mini Euro FX Futures (CME: E7)
- `Futures.Currencies.EURSEK` : Euro/Swedish Krona Futures (CME: ESK)
- `Futures.Currencies.GBP` : British Pound Futures (CME: 6B)
- `Futures.Currencies.JAPANESE_YEN_EMINI` : E-mini Japanese Yen Futures (CME: J7)
- `Futures.Currencies.JPY` : Japanese Yen Futures (CME: 6J)
- `Futures.Currencies.MICRO_AUD` : Micro AUD/USD Futures (CME: M6A)
- `Futures.Currencies.MICRO_BTC` : Micro Bitcoin Futures (CME: MBT)
- `Futures.Currencies.MICRO_CAD` : Micro USD/CAD Futures (CME: M6C)
- `Futures.Currencies.MICRO_CADUSD` : Micro CAD/USD Futures (CME: MCD)
- `Futures.Currencies.MICRO_CHF` : Micro CHF/USD Futures (CME: MSF)
- `Futures.Currencies.MICRO_ETHER` : Micro Ether Futures (CME: MET)
- `Futures.Currencies.MICRO_EUR` : Micro EUR/USD Futures (CME: M6E)
- `Futures.Currencies.MICRO_GBP` : Micro GBP/USD Futures (CME: M6B)
- `Futures.Currencies.MICRO_INRUSD` : Micro INR/USD Futures (CME: MIR)
- `Futures.Currencies.MICRO_JPY` : Micro JPY/USD Futures (CME: MJY)
- `Futures.Currencies.MICRO_USDCHF` : Micro USD/CHF Futures (CME: M6S)
- `Futures.Currencies.MICRO_USDCNH` : Micro USD/CNH Futures (CME: MNH)
- `Futures.Currencies.MICRO_USDJPY` : Micro USD/JPY Futures (CME: M6J)
- `Futures.Currencies.MXN` : Mexican Peso Futures (CME: 6M)
- `Futures.Currencies.NZD` : New Zealand Dollar Futures (CME: 6N)
- `Futures.Currencies.RUB` : Russian Ruble Futures (CME: 6R)
- `Futures.Currencies.STANDARD_SIZE_USD_OFFSHORE_RMBCNH` : Standard-Size USD/Offshore RMB (CNH) Futures (CME: CNH)
- `Futures.Currencies.ZAR` : South African Rand Futures (CME: 6Z)
- `Futures.Energy.ARGUS_LL_SVS_WTI_ARGUS_TRADE_MONTH` : Argus LLS vs. WTI (Argus) Trade Month Futures (NYMEX: AE5)
- `Futures.Energy.ARGUS_PROPANE_FAR_EAST_INDEX` : Argus Propane Far East Index Futures (NYMEX: A7E)
- `Futures.Energy.ARGUS_PROPANE_SAUDI_ARAMCO` : Argus Propane (Saudi Aramco) Futures (NYMEX: A9N)
- `Futures.Energy.BRENT_CRUDE_OIL_VS_DUBAI_CRUDE_OIL_PLATTS` : Brent Crude Oil vs. Dubai Crude Oil (Platts) Futures (NYMEX: ADB)
- `Futures.Energy.BRENT_LAST_DAY_FINANCIAL` : Brent Last Day Financial Futures (NYMEX: BZ)
- `Futures.Energy.CHICAGO_ETHANOL_PLATTS` : Chicago Ethaanol (Platts) Futures (NYMEX: CU)
- `Futures.Energy.CONWAY_PROPANE_OPIS` : Conway Propane (OPIS) Futures (NYMEX: A8K)
- `Futures.Energy.CRUDE_OIL_WTI` : Crude Oil WTI Futures (NYMEX: CL)
- `Futures.Energy.DUBAI_CRUDE_OIL_PLATTS_FINANCIAL` : Dubai Crude Oil (Platts) Financial Futures (NYMEX: DCB)
- `Futures.Energy.EAST_WEST_GASOLINE_SPREAD_PLATTS_ARGUS` : East-West Gasoline Spread (Platts-Argus) Futures (NYMEX: EWG)
- `Futures.Energy.EAST_WEST_NAPHTHA_JAPAN_C_FVS_CARGOES_CIFNWE_SPREAD_PLATTS` : East-West Naphtha: Japan C&F vs. Cargoes CIF NWE Spread (Platts) Futures (NYMEX: EWN)
- `Futures.Energy.ETHANOL` : Ethanol Futures (CBOT: EH)
- `Futures.Energy.ETHANOL_T_2_FOB_RDAM_INCLUDING_DUTY_PLATTS` : Ethanol T2 FOB Rdam Including Duty (Platts) Futures (NYMEX: AZ1)
- `Futures.Energy.EUROPEAN_NAPHTHA_PLATTS_CRACK_SPREAD` : European Naphtha (Platts) Crack Spread Futures (NYMEX: EN)
- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS` : European Propane CIF ARA (Argus) Futures (NYMEX: APS)
- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS_VS_NAPHTHA_CARGOES_CIFNWE_PLATTS` : European Propane CIF ARA (Argus) vs. Naphtha Cargoes CIF NWE (Platts) Futures (NYMEX: EPN)
- `Futures.Energy.FREIGHT_ROUTE_TC_14_BALTIC` : Freight Route TC14 (Baltic) Futures (NYMEX: FRC)
- `Futures.Energy.GASOLINE` : Gasoline RBOB Futures (NYMEX: RB)
- `Futures.Energy.GASOLINE_EUROBOB_OXY_NWE_BARGES_ARGUS` : Gasoline Euro-bob Oxy NWE Barges (Argus) Futures (NYMEX: B7H)

- `Futures.Energy.GROUP_THREE_SUBOCTANE_GASOLINE_PLATTS_VS_RBOB` : Group Three Sub-octane Gasoliine (Platts) vs. RBOB Futures (NYMEX: AA8)
- `Futures.Energy.GROUP_THREE_ULSD_PLATTS_VS_NY_HARBOR_ULSD` : Group Three ULSD (Platts) vs. NY Harbor ULSD Futures (NYMEX: AA6)
- `Futures.Energy.GULF_COAST_CBOB_GASOLINE_A_2_PLATTS_VS_RBOB_GASOLINE` : Gulf Coast CBOB Gasoline A2 (Platts) vs. RBOB Gasoline Futures (NYMEX: CRB)
- `Futures.Energy.GULF_COAST_HSFO_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Gulf Coast HSFO (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: GCU)
- `Futures.Energy.HEATING_OIL` : Heating Oil Futures (NYMEX: HO)
- `Futures.Energy.LOS_ANGELES_CARBOB_GASOLINE_OPI_SVS_RBOB_GASOLINE` : Los Angeles CARBOB Gasoline (OPIS) vs. RBOB Gasoline Futures (NYMEX: AJL)
- `Futures.Energy.LOS_ANGELES_CARB_DIESEL_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles CARB Diesel (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AKL)
- `Futures.Energy.LOS_ANGELES_JET_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles Jet (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AJS)
- `Futures.Energy.MARS_ARGUS_VS_WTI_FINANCIAL` : Mars (Argus) vs. WTI Financial Futures (NYMEX: AYX)
- `Futures.Energy.MARS_ARGUS_VS_WTI_TRADE_MONTH` : Mars (Argus) vs. WTI Trade Month Futures (NYMEX: AYV)
- `Futures.Energy.MICRO_CRUDE_OIL_WTI` : Micro WTI Crude Oil Futures (NYMEX: MCL)
- `Futures.Energy.MICRO_EUROPEAN_FOB_RDAM_MARINE_FUEL_ZERO_POINT_FIVE_PERCENT_BARGES_PLATTS` : Micro European FOB Rdam Marine Fuel 0.5% Barges (Platts) Futures (NYMEX: R5O)
- `Futures.Energy.MICRO_EUROPEAN_THREE_POINT_FIVE_PERCENT_OIL_BARGES_FOB_RDAM_PLATTS` : Micro European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: MEF)
- `Futures.Energy.MICRO_GASOIL_ZERO_POINT_ONE_PERCENT_BARGES_FOBARA_PLATTS` : Micro Gasoil 0.1% Barges FOB ARA (Platts) Futures (NYMEX: M1B)
- `Futures.Energy.MICRO_SINGAPORE_FOB_MARINE_FUEL_ZERO_POINT_FIVE_PERCET_PLATTS` : Micro Singapore FOB Marine Fuel 0.5% (Platts) Futures (NYMEX: S5O)
- `Futures.Energy.MICRO_SINGAPORE_FUEL_OIL_380_CST_PLATTS` : Micro Singapore Fuel Oil 380CST (Platts) Futures (NYMEX: MAF)
- `Futures.Energy.MINI_EUROPEAN_THREE_POINT_PERCENT_FIVE_FUEL_OIL_BARGES_PLATTS` : Mini European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: A0D)
- `Futures.Energy.MINI_SINGAPORE_FUEL_OIL_180_CST_PLATTS` : Mini Singapore Fuel Oil 180 cst (Platts) Futures (NYMEX: A0F)
- `Futures.Energy.MONT_BELVIEU_ETHANE_OPIS` : Mont Belvieu Ethane (OPIS) Futures (NYMEX: AC0)
- `Futures.Energy.MONT_BELVIEU_LDH_PROPANE_OPIS` : Mont Belvieu LDH Propane (OPIS) Futures (NYMEX: B0)
- `Futures.Energy.MONT_BELVIEU_NATURAL_GASOLINE_OPIS` : Mont Belvieu Natural Gasoline (OPIS) Futures (NYMEX: A7Q)
- `Futures.Energy.MONT_BELVIEU_NORMAL_BUTANE_OPIS` : Mont Belvieu Normal Butane (OPIS) Futures (NYMEX: AD0)
- `Futures.Energy.NATURAL_GAS` : Natural Gas Futures (NYMEX: NG)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_LAST_DAY_FINANCIAL` : Natural Gas (Henry Hub) Last-day Financial Futures (NYMEX: HH)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_PENULTIMATE_FINANCIAL` : Natural Gas (Henry Hub) Penultimate Financial Futures (NYMEX: HP)
- `Futures.Energy.ONE_PERCENT_FUEL_OIL_CARGOES_FOBNWE_PLATTS_VS_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : 1% Fuel Oil Cargoes FOB NWE (Platts) vs. 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: FSS)
- `Futures.Energy.PREMIUM_UNLEADED_GASOLINE_10_PPM_FOBMED_PLATTS` : Premium Unleaded Gasoline 10 ppm FOB MED (Platts) Futures (NYMEX: A3G)
- `Futures.Energy.PROPANE_NON_LDH_MONT_BELVIEU_OPIS` : Propane Non-LDH Mont Belvieu (OPIS) Futures (NYMEX: A1R)
- `Futures.Energy.RBOB_GASOLINE_CRACK_SPREAD` : RBOB Gasoline Crack Spread Futures (NYMEX: ARE)
- `Futures.Energy.RBOB_GASOLINE_VS_EUROBOB_OXY_NWE_BARGES_ARGUS_THREE_HUNDRED_FIFTY_THOUSAND_GALLONS` : RBOB Gasoline vs. Eurobob Oxy NWE Barges (Argus) (350,000 gallons) Futures (NYMEX: EXR)
- `Futures.Energy.SINGAPORE_FUEL_OIL_380_CST_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Singapore Fuel Oil 380 cst (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: EVC)
- `Futures.Energy.SINGAPORE_GASOIL_PLATTS_VS_LOW_SULPHUR_GASOIL_FUTURES` : Singapore Gasoil (Platts) vs. Low Sulphur Gasoil Futures (NYMEX: AGA)
- `Futures.Energy.SINGAPORE_MOGAS_92_UNLEADED_PLATTS_BRENT_CRACK_SPREAD` : Singapore Mogas 92 Unleaded (Platts) Brent Crack Spread Futures (NYMEX: D1N)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD` : 3.5% Fuel Oil Barges FOB Rdam (Platts) Crack Spread Futures (NYMEX: FO)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD_1000_MT` : 3.5% Fuel Oil Barges FOB Rdam

(Platts) Crack Spread (1000mt) Futures (NYMEX: BOO)

- `Futures.Energy.WTI_BRENT_FINANCIAL` : WTI-Brent Financial Futures (NYMEX: BK)
- `Futures.Energy.WTI_FINANCIAL` : WTI Financial Futures (NYMEX: CSX)
- `Futures.Energy.WTI_HOUSTON_ARGUS_VS_WTI_TRADE_MONTH` : WTI Houston (Argus) vs. WTI Trade Month Futures (NYMEX: HTT)
- `Futures.Energy.WTI_HOUSTON_CRUDE_OIL` : WTI Houston Crude Oil Futures (NYMEX: HCL)
- `Futures.Financials.EURO_DOLLAR` : EuroDollar Futures (CME: GE)
- `Futures.Financials.FIVE_YEAR_USDMAC_SWAP` : 5-Year USD MAC Swap Futures (CBOT: F1U)
- `Futures.Financials.MICRO_Y_10_TREASURY_NOTE` : Micro 10-Year Yield Futures (CBOT: 10Y)
- `Futures.Financials.MICRO_Y_2_TREASURY_BOND` : Micro 2-Year Yield Futures (CBOT: 2YY)
- `Futures.Financials.MICRO_Y_30_TREASURY_BOND` : Micro 30-Year Yield Futures (CBOT: 30Y)
- `Futures.Financials.MICRO_Y_5_TREASURY_BOND` : Micro 5-Year Yield Futures (CBOT: 5YY)
- `Futures.Financials.ULTRA_TEN_YEAR_US_TREASURY_NOTE` : Ultra 10-Year U.S. Treasury Note Futures (CBOT: TN)
- `Futures.Financials.ULTRA_US_TREASURY_BOND` : Ultra U.S. Treasury Bond Futures (CBOT: UB)
- `Futures.Financials.Y_10_TREASURY_NOTE` : 10Y U.S. Treasury Note Futures (CBOT: ZN)
- `Futures.Financials.Y_2_TREASURY_NOTE` : 2Y U.S. Treasury Note Futures (CBOT: ZT)
- `Futures.Financials.Y_30_TREASURY_BOND` : 30Y U.S. Treasury Bond Futures (CBOT: ZB)
- `Futures.Financials.Y_5_TREASURY_NOTE` : 5Y U.S. Treasury Note Futures (CBOT: ZF)
- `Futures.Forestry.LUMBER` : Lumber Futures (CME: LBR)
- `Futures.Forestry.RANDOM_LENGTH_LUMBER` : Random Length Lumber Futures (CME: LBS)
- `Futures.Grains.BLACK_SEA_CORN_FINANCIALLY_SETTLED_PLATTS` : Black Sea Corn Financially Settled (Platts) Futures (CBOT: BCF)
- `Futures.Grains.BLACK_SEA_WHEAT_FINANCIALLY_SETTLED_PLATTS` : Black Sea Wheat Financially Settled (Platts) Futures (CBOT: BWF)
- `Futures.Grains.CORN` : Corn Futures (CBOT: ZC)
- `Futures.Grains.HRW_WHEAT` : KC HRW Wheat Futures (CBOT: KE)
- `Futures.Grains.OATS` : Oats Futures (CBOT: ZO)
- `Futures.Grains.SOYBEANS` : Soybeans Futures (CBOT: ZS)
- `Futures.Grains.SOYBEAN_MEAL` : Soybean Meal Futures (CBOT: ZM)
- `Futures.Grains.SOYBEAN_OIL` : Soybean Oil Futures (CBOT: ZL)
- `Futures.Grains.WHEAT` : Default wheat contract is SRWWheat (CBOT: ZW)
- `Futures.Indices.BLOOMBERG_COMMODITY_INDEX` : Bloomberg Commodity Index Futures (CBOT: AW)
- `Futures.Indices.DOW_30_E_MINI` : E-mini Dow Indu 30 Futures (CBOT: YM)
- `Futures.Indices.DOW_JONES_REAL_ESTATE` : Dow Jones Real Estate futures on CME (CME: RX)
- `Futures.Indices.FTSE_EMERGING_EMINI` : E-mini FTSE Emerging Index Futures (CME: EI)
- `Futures.Indices.MICRO_DOW_30_E_MINI` : Micro E-mini Dow Jones Industrial Average Index Futures (CBOT: MYM)
- `Futures.Indices.MICRO_NASDAQ_100_E_MINI` : Micro E-mini Nasdaq-100 Index Futures (CME: MNQ)
- `Futures.Indices.MICRO_RUSSELL_2000_E_MINI` : Micro E-mini Russell 2000 Index Futures (CME: M2K)
- `Futures.Indices.MICRO_SP_500_E_MINI` : Micro E-mini S&P 500 Index Futures (CME: MES)
- `Futures.Indices.NASDAQ_100_BIOTECHNOLOGY_E_MINI` : E-mini Nasdaq-100 Biotechnology Index Futures (CME: BIO)
- `Futures.Indices.NASDAQ_100_E_MINI` : E-mini NASDAQ 100 Futures (CME: NQ)
- `Futures.Indices.NIKKEI_225_DOLLAR` : Nikkei-225 Dollar Futures (CME: NKD)
- `Futures.Indices.NIKKEI_225_YEN_CME` : Nikkei-225 Yen denominated Futures on CME (CME: NIY)
- `Futures.Indices.RUSSELL_1000_E_MINI` : E-mini Russell 1000 futures on CME (CME: RS1)
- `Futures.Indices.RUSSELL_2000_E_MINI` : E-mini Russell 2000 Futures (CME: RTY)
- `Futures.Indices.SPGSCI_COMMODITY` : S&P-GSCI Commodity Index Futures (CME: GD)
- `Futures.Indices.SP_400_MID_CAP_EMINI` : E-mini S&P MidCap 400 Futures (CME: EMD)
- `Futures.Indices.SP_500_ANNUAL_DIVIDEND_INDEX` : (CME: SDA)
- `Futures.Indices.SP_500_E_MINI` : E-mini S&P 500 Futures (CME: ES)
- `Futures.Indices.TOPIXYEN` : YEN Denominated Topix Index Futures on CME (CME: TPY)
- `Futures.Indices.USD_DENOMINATED_IBOVESPA` : USD-Denominated Ibovespa Index Futures (CME: IBV)
- `Futures.Indices.VIX` : CBOE Volatility Index Futures (CFE: VX)
- `Futures.Meats.FEEDER_CATTLE` : Feeder Cattle Futures (CME: GF)
- `Futures.Meats.LEAN_HOGS` : Lean Hogs Futures (CME: HE)
- `Futures.Meats.LIVE_CATTLE` : Live Cattle Futures (CME: LE)

- `Futures.Metals.ALUMINIUM_EUROPEAN_PREMIUM_DUTY_PAID_METAL_BULLETIN` : Aluminium European Premium Duty-Paid (Metal Bulletin) Futures (COMEX: EDP)
- `Futures.Metals.ALUMINUM_MWUS_TRANSACTION_PREMIUM_PLATTS_25_MT` : Aluminum MW U.S. Transaction Premium Platts (25MT) Futures (COMEX: AUP)
- `Futures.Metals.COPPER` : Copper Futures (COMEX: HG)
- `Futures.Metals.GOLD` : Gold Futures (COMEX: GC)
- `Futures.Metals.MICRO_GOLD` : Micro Gold Futures (COMEX: MGC)
- `Futures.Metals.MICRO_GOLD_TAS` : Micro Gold TAS Futures (COMEX: MGT)
- `Futures.Metals.MICRO_PALLADIUM` : Micro Palladium Futures (NYMEX: PAM)
- `Futures.Metals.MICRO_SILVER` : Micro Silver Futures (COMEX: SIL)
- `Futures.Metals.PALLADIUM` : Palladium Futures (NYMEX: PA)
- `Futures.Metals.PLATINUM` : Platinum Futures (NYMEX: PL)
- `Futures.Metals.SILVER` : Silver Futures (COMEX: SI)
- `Futures.Metals.US_MIDWEST_DOMESTIC_HOT_ROLLED_COIL_STEEL_CRU_INDEX` : U.S. Midwest Domestic Hot-Rolled Coil Steel (CRU) Index Futures (NYMEX: HRC)
- `Futures.Softs.SUGAR_11` : Sugar #11 Futures ICE (ICE: SB)
- `Futures.Softs.SUGAR_11_CME` : Sugar #11 Futures CME (NYMEX: YO)

Example Applications

The US Futures Security Master enables you to design strategies harnessing continuous Futures contracts. Examples include the following strategies:

- Trading cyclical patterns in commodity Futures.
- Buying gold Futures as an inflation hedge with automatic contract roll-overs.
- Detecting arbitrage opportunities between index Futures and Equities.

Classic Algorithm Example

The following example algorithm buys the continuous Future contract for Crude Oil when its price rises above its simple moving average. When its price drops below its simple moving average, the algorithm sells the continuous contract. Each time the continuous Future contract rolls over, the algorithm logs the event.

```py
from AlgorithmImports import *


class USFuturesSecurityMasterDataClassicAlgorithm(QCAlgorithm):
    # Define a threshold for the SMA cross.
    _threshold = 0.01

    def initialize(self) -> None:
        self.set_cash(1000000)
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to
        # avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a Future. Set the continuous contract mapping criteria to
        # the contract with highest open interest to provide the best
        # price information for trend estimation.
        self._future = self.add_future(
            Futures.Energy.CRUDE_OIL_WTI,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            contract_depth_offset=0
        )
        # Historical data: Get the contract rollovers for the trailing
        # year.
        history = self.history(
            SymbolChangedEvent, self._future.symbol, timedelta(365)
        )
        # Set up an SMA indicator for to estimate the trend direction.
        self._sma = self.sma(self._future.symbol, 10, Resolution.DAILY)
        # Warm up the SMA indicator so we can trade right away.
        self.warm_up_indicator(self._future.symbol, self._sma)

    def on_data(self, slice: Slice) -> None:
        # Wait until we have the latest TradeBar and the indicator is
        # ready.
        if not (self._future.symbol in slice.bars and self._sma.is_ready):
            return
        contract = self.portfolio[self._future.mapped]
        # Long during an uptrend.
        if (not contract.is_long and
            self._future.price > self._sma.current.value * (1+self._threshold)):
            quantity = 1
        # Short during an downtrend.
        elif (not contract.is_short and
            self._future.price < self._sma.current.value * (1-self._threshold)):
            quantity = -1
        else:
            return
        self.market_order(self._future.mapped, quantity - contract.quantity)

    def on_symbol_changed_events(self, symbol_changed_events):
        for symbol, changed_event in symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = changed_event.new_symbol
            # The quantity to roll over should be consistent.
            quantity = self.portfolio[old_symbol].quantity
            if not quantity:
                continue
            # Rolling over: Liquidate the old mapped contract and switch
            # to the new mapped contract.
            tag = f"Rollover: {old_symbol} -> {new_symbol}"
            self.liquidate(old_symbol, tag=tag)
            self.market_order(new_symbol, quantity, tag=tag)
```

Framework Algorithm Example

The following example algorithm buys the continuous Future contract for Crude Oil when its price rises above its simple moving average. When its price drops below its simple moving average, the algorithm sells the continuous contract. Each time the contract rolls over, the algorithm logs the event.

PY

```
from AlgorithmImports import *


class USFuturesSecurityMasterDataFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_cash(1000000)
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Setting the continuous contract mapping criteria, the contract with highest open interest provide the best price information
for trend estimation
        self.add_future(Futures.Energies.CRUDE_OIL_WTI,
                        data_normalization_mode = DataNormalizationMode.BACKWARDS_RATIO,
                        data_mapping_mode = DataMappingMode.OPEN_INTEREST,
                        contract_depth_offset = 0)
        self.add_alpha(ContinuousFuturesAlphaModel())
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class ContinuousFuturesAlphaModel(AlphaModel):
    # 1% margin to reassure trend direction
    threshold = 0.01
    symbol = None
    continuous_contract = None

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []
        # Check if contract data available, since all trades are based on that only
        if self.continuous_contract_symbol is None or self.continuous_contract is None:
            return insights

        # Up-to-date handling of switching the mapped contract for trade liquidity
        for symbol, changed_event in slice.symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            if algorithm.insights.has_active_insights(old_symbol, algorithm.utc_time):
                new_symbol = changed_event.new_symbol
                tag = f"Rollover - Symbol changed at {algorithm.time}: {old_symbol} -> {new_symbol}"
                last_insight = sorted(algorithm.insights[old_symbol], key=lambda x: x.close_time_utc)[-1]
                insights.append(Insight.price(new_symbol, last_insight.close_time_utc, last_insight.direction, tag= tag))
                algorithm.insights.clear([old_symbol])
                algorithm.log(tag)

        mapped_symbol = self.continuous_contract.mapped
        # Make sure trade decisions are based on newly received data
        if not slice.bars.contains_key(self.continuous_contract_symbol) or not self.sma.is_ready or not mapped_symbol:
            return insights

        direction = None
        # Long if trend up by threshold to follow the trend
        if slice.bars[self.continuous_contract_symbol].price > self.sma.current.value * (1+self.threshold) and not
algorithm.portfolio[mapped_symbol].is_long:
            direction = InsightDirection.UP
        # Short if trend down by threshold to follow the trend
        elif slice.bars[self.continuous_contract_symbol].price < self.sma.current.value * (1-self.threshold) and not
algorithm.portfolio[mapped_symbol].is_short:
            direction = InsightDirection.DOWN

        if direction:
            insights.append(Insight.price(mapped_symbol, timedelta(days=14), direction))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            # Set up SMA indicator for trend direction estimator, only for the canonical symbol
            if symbol.is_canonical():
                self.continuous_contract = security
                self.continuous_contract_symbol = symbol
                self.sma = algorithm.SMA(self.continuous_contract_symbol, 10, Resolution.DAILY)

                # Historical data
                history = algorithm.history(symbol, 60*24*10, Resolution.MINUTE)
                algorithm.debug(f"We got {len(history)} from our history request for {symbol}")

                if history.empty:
                    continue
                # Warm up the SMA indicator for its readiness for immediate use
                for time, row in history.droplevel(0).loc[self.continuous_contract_symbol].iterrows():
                    self.sma.update(IndicatorDataPoint(time, row.close))

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Futures Security Master dataset provides `SymbolChangedEvent` objects, which have the following attributes:

QuantConnect

US Index Option Universe

Introduction

The US Index Option Universe dataset by QuantConnect lists the available US Index Options contracts and the current Implied Volatility and Greeks. The data covers European Option contracts for 3 US Indices: SPX, VIX, and NDX. It starts in January 2012 and is delivered on a daily update frequency. To create this dataset, we use our implementation of the forward tree pricing model, which accounts for the interest rate, dividend payments, and daily closing prices. The values in this dataset are the same values you can get from daily indicators with mirror Options.

This dataset `does not` contain market data. For market data, see US Index Options by AlgoSeek .

For more information about the US Index Option Universe dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

QuantConnect was founded in 2012 to serve quants everywhere with the best possible algorithmic trading technology. Seeking to disrupt a notoriously closed-source industry, QuantConnect takes a radically open-source approach to algorithmic trading. Through the QuantConnect web platform, more than 50,000 quants are served every month.

Getting Started

The following snippet demonstrates how to request data from the US Index Options Universe dataset:

```
option = self.add_index_option('VIX')
option.set_filter(lambda universe: universe.delta(0.4, 0.6))
self.option_symbol = option.symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012
Asset Coverage	7 Index Options
Data Density	Dense
Resolution	Daily
Timezone	New York

Specification Over Time

According to the SPX Options contract specification , some SPX contracts expire every month and SPXW contracts expires every day. Before 2021, you could only trade SPX contracts with the following expiration dates:

- Expires within the next 4 months
- Expires in September within the next 14 months
- Expires in January, March, or June within the next 2 years
- Expires in December within the next 3 years

During this time, SPXW didn't have 0DTE every day.

`Sources` :

- Cboe Options Exchange to List Three Long-Dated SPX Options Expirations, Beginning November 1, 2021

- S&P 500 Weekly Options Now Expire Five Days a Week

Requesting Data

To add US Index Options Universe data to your algorithm, call the `add_index_option` method. Save a reference to the Index Option `Symbol` so you can access the data later in your algorithm. To define which contracts should be in your universe, call the `set_filter` method of the `IndexOption` object.

The `add_index_option` method provides a daily stream of Option chain data. To get the most recent daily chain, call the `option_chain` method with the canonical Index Option Symbol. The `option_chain` method returns data on all the tradable contracts, not just the contracts that pass your universe filter.

```
PY
class IndexOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(1_000_000)
        self.universe_settings.asynchronous = True
        self.index_symbol = self.add_index('VIX').symbol

        standard_option = self.add_index_option(self.index_symbol)
        standard_option.set_filter(self._option_filter)
        self.standard_option_symbol = standard_option.symbol
        standard_chain = self.option_chain(standard_option.symbol, flatten=True).data_frame

        weekly_option = self.add_index_option(self.index_symbol, "VIXW")
        weekly_option.set_filter(self._option_filter)
        self.weekly_option_symbol = weekly_option.symbol
        weekly_chain = self.option_chain(weekly_option.symbol, flatten=True).data_frame
        weekly_chain = weekly_chain[
            weekly_chain.index.map(lambda symbol: not IndexOptionSymbol.is_standard(symbol))
        ]


    def _option_filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Contracts can be filtered by greeks, implied volatility, open interest:
        return universe \
            .delta(0.5, 1.5) \
            .gamma(0.0001, 0.0006) \
            .vega(0.01, 1.5) \
            .theta(-2.0, -0.5) \
            .rho(0.5, 3.0) \
            .implied_volatility(1, 3) \
            .open_interest(100,500)
```

The Index resolution must be less than or equal to the Index Option resolution. For example, if you set the Index resolution to minute, then you must set the Index Option resolution to minute, hour, or daily.

For more information about creating US Index Option subscriptions, see Index Options .

Accessing Data

For information about accessing US Equity Options data, see Handling Data .

Historical Data

You can get historical US Index Options data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Index Options Universe data in an algorithm, call the `history` method with the canonical Index Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter. If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.standard_option_symbol, timedelta(3))

# OptionUniverse objects
history = self.history[OptionUniverse](self.standard_option_symbol, timedelta(3))
```

For more information about Index Options Universe data in algorithms, see Historical Data .

Historical Data In Research

To get historical US Index Options Universe data in the Research Environment, call the `history` method with the canonical Option `Symbol` . This method returns data on all of the tradable contracts, not just the contracts that pass your universe filter.

```py
qb = QuantBook()
index_symbol = qb.add_index('VIX').symbol
option = qb.add_index_option(index_symbol) # or qb.add_index_option(index_symbol, "VIXW")
history = qb.history(option.symbol, datetime(2020, 6, 1), datetime(2020, 6, 5))
```

For more information about historical Index Options Universe data in the Research Environment, see Universes .

Supported Assets

The following table shows the available Index Options:

Underlying Index	Underlying Ticker	Target Ticker	Standard Contracts	Weekly Contracts	Tradable on Expiry Day
NASDAQ-100	NDX		✓		
NASDAQ-100	NDX	NDXP	✓		✓
NASDAQ-100	NDX	NQX	✓	✓	✓
Russell 2000	RUT		✓		
Russell 2000	RUT	RUTW		✓	✓
S&P500	SPX		✓		
S&P500	SPX	SPXW		✓	✓
S&P500	VIX		✓		
S&P500	VIX	VIXW		✓	

For more information about each underlying Index, see Supported Indices .

Example Applications

The US Index Options Universe dataset enables you to accurately design strategies for Index Options. Examples include the following strategies:

- Buying VIX call Options to hedge against upcoming volatility
- Buying VIX put Options to capture the natural downward price movement in the VIX index
- Buying SPX put Options to protect against downward price movement in the S&P 500

Classic Algorithm Example

The following example algorithm subscribes to SPX put options that fall within delta range between -1 and -0.95, open interest range between 10 and 1000, and expire within seven days. Within this Option chain, the algorithm holds the put Option contract that has the minimum delta (closest to -1) during market hour to hedge the underlying intra-day movement completely. It avoid volatility from sentiment and only earns from inter-day movement from longer-term factors. When the contract expires, the algorithm rolls over to the next contract that meets this criteria.

```
from AlgorithmImports import * class IndexOptionsUniverseAlgorithm(QCAlgorithm): def initialize(self) -> None: self.set
_start_date(2024, 9, 1) self.set_end_date(2024, 12, 31) self.set_cash(10000000) # Asynchronous can use computational re
sources efficiently self.universe_settings.asynchronous = True # Subscribe to the underlying for the underlying positio
n # Set the data normalization mode to raw for strike price comparability self.index = self.add_index("SPX").symbol # R
equesting option data and filter for the hedge candidates option = self.add_index_option(self.index) option.set_filter(
self.option_filter) self.option_symbol = option.symbol # Set scheduled event to buy a hedge option contract at market o
pen to eliminate the intra-day movement self.schedule.on( self.date_rules.every_day(self.index), self.time_rules.after_
market_open(self.index, 1), self.buy_hedge_contract ) # Set a scheduled event to sell the hedge contract before market
close, since we want to earn from inter-day movement # Leave 2 minutes contingency to fill self.schedule.on( self.date_
rules.every_day(self.index), self.time_rules.before_market_close(self.index, 2), self.sell_hedge_contract ) self.hedge
= None def option_filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse: # Select the contracts with del
ta very close to -1 and high open interest # This can effectively hedge most of the price change of the underlying and
ensure the liquidity # Make sure the contract is expiring close for its tradbility return universe.puts_only().expirati
on(2, 7).delta(-1, -0.95).open_interest(10, 1000) def buy_hedge_contract(self) -> None: chain = self.current_slice.opti
on_chains.get(self.option_symbol) if chain: # Order the underlying if not hold, the order size should match the option
contract # Order only if option chain data ready for hedging if not self.portfolio[self.index].invested: self.market_or
der(self.index, self.securities[self.option_symbol].symbol_properties.contract_multiplier) # Get the contract with delt
a closest to -1 (lowest possible delta) contract = sorted(chain, key=lambda x: x.greeks.delta)[0] self.hedge = contract
.symbol # Buy 1 deep ITM put with delta close to -1 to eliminate the intraday movement self.market_order(self.hedge, 1)
def sell_hedge_contract(self) -> None: # Check if any hedge contract position, if so, liquidate before market close to
expose to underlying overnight movement if self.hedge: self.liquidate(self.hedge) self.hedge = None
```

Framework Algorithm Example

The following example algorithm demonstrating a Gamma Scalping strategy through framework algorithm. It filters SPX options with expiration between 30 to 90 days and open interest between 50 and 1000, due to liquidity concern and lower Gamma fluctuation. Assuming a significant upward trend is expecting, it orders a long straddle strategy from the strike price with Delta-neutral and the highest Gamma to earn the highest profit from upward underlying movement.

PY

```
from AlgorithmImports import *

class IndexOptionsUniverseFrameworkAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)
        # Asynchronous can use computational resources efficiently
        self.universe_settings.asynchronous = True

        # Universe selection that select based on liquidity of the option contracts (by open interest)
        self.add_universe_selection(IndexOptionsUniverseSelectionModel())
        # Custom alpha model that use Delta and Gamma to signal insights
        self.add_alpha(OptionDeltaGammaAlphaModel())
        # To maintain long-short size equal, use a PCM that order a single contract
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class IndexOptionsUniverseSelectionModel(OptionUniverseSelectionModel):
    def __init__(self) -> None:
        # Daily update with the select_option_chain_symbols function
        super().__init__(timedelta(1), self.select_option_chain_symbols)

    def select_option_chain_symbols(self, dt: datetime) -> List[Symbol]:
        # Select only SPX options as our focus, which is a highly traded volatile index with great upward momentum
        return [Symbol.create("SPX", SecurityType.INDEX_OPTION, Market.USA)]

    def filter(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # To ensure the liquidity and tradability, make sure the option is not 0-DTE and have a fair open interest
        # A longer TTM will have lower Gamma thus more stable in the local delta-neutral position
        return universe.expiration(30, 90).open_interest(50, 1000)

class OptionDeltaGammaAlphaModel(AlphaModel):
    def __init__(self) -> None:
        # A day count variable to control the alpha model only trade once a day
        self._day = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        if self._day != slice.time.day:
            # We open position for each option underlying as separate bet
            for _, chain in slice.option_chains.items():
                # For theta-neutral, select the same expiry for both call and put
                expiry = min(x.expiry for x in chain)
                contracts = [x for x in chain if x.expiry == expiry]

                # Calculate delta and gamma per strike price group for later filtering, ensure both call and put available for option
strategy
                delta_gamma_symbols = []
                strikes = set(x.strike for x in contracts if len([y for y in contracts if y.strike == x.strike]) == 2)
                for strike in strikes:
                    # Get both call and put for their aggregated delta and gamma
                    call = next(filter(lambda x: x.right == OptionRight.CALL and x.strike == strike, contracts))
                    put = next(filter(lambda x: x.right == OptionRight.PUT and x.strike == strike, contracts))
                    delta_gamma_symbols.append((call.greeks.delta + put.greeks.delta, call.greeks.gamma + put.greeks.gamma, call.symbol,
put.symbol))

                if len(delta_gamma_symbols) == 0:
                    continue

                # We want a delta-neutral position, so it is likely to be ATM (like a long straddle)
                # Less than 2d.p. difference is non-significant, which we can risk for better reward
                # Assuming the market direction is up in most scenario, we try to get the strike providing max overall Gamma
                # Make sure the aggregated gamma to be positive to bet on large uptrend
                # So it will earn more when the price really go up higher and higher, but locally immune for small noise
                filtered = [item for item in delta_gamma_symbols if round(item[0], 2) <= 0.01 and item[1] > 0]
                if len(filtered) == 0:
                    continue
                selected = sorted(filtered, key=lambda x: x[1])[0]

                # Provide trade signal and roll the day count to let the position stay for the whole day
                selected_call = selected[2]
                selected_put = selected[3]
                insights.extend([
                    Insight.price(selected_call, Expiry.END_OF_DAY, InsightDirection.UP),
                    Insight.price(selected_put, Expiry.END_OF_DAY, InsightDirection.UP)
                ])
                self._day = slice.time.day

        return insights

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Use a whole number target to order the exact number of share for size-matching
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Index Options Universe dataset provides `OptionFilterUniverse` , `OptionUniverse` , and `OptionChain` objects.

OptionFilterUniverse Attributes

`OptionFilterUniverse` objects have the following attributes:

OptionUniverse Attributes

`OptionUniverse` objects have the following attributes:

OptionChain Attributes

`OptionChain` objects have the following attributes:

Datasets

AlgoSeek

AlgoSeek is a leading historical intraday US market data provider offering the most comprehensive and detailed market data and analytics products in the financial industry covering Equities, Futures, Options, cash FOREX, and Cryptocurrencies. AlgoSeek data is built for quantitative trading and machine learning. For more information about AlgoSeek, visit algoseek.com .

US Equities

US Equity Options

US Future Options

US Futures

US Index Options

AlgoSeek

US Equities

Introduction

The US Equities dataset by AlgoSeek is survivorship bias-free daily coverage of every stock traded in the US Securities Information Processors (SIP) CTA/UTP feed since 1998. The dataset covers approximately 27,500 securities, starts in January 1998, and is delivered in any resolution from tick to daily. The Data is collected from the full SIP feed via our Equinix co-located servers, including all trades and quotes published to every exchange as well as FINRA. Over-the-Counter (OTC) trades are not included.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US Equities dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

AlgoSeek is a leading historical intraday US market data provider offering the most comprehensive and detailed market data and analytics products in the financial industry covering Equities, Futures, Options, cash FOREX, and Cryptocurrencies. AlgoSeek data is built for quantitative trading and machine learning. For more information about AlgoSeek, visit algoseek.com .

Getting Started

AlgoSeek is the default US Equities dataset on QuantConnect. The following snippet demonstrates how to request data from the US Equities dataset:

```py
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Asset Coverage	27,500 US Equities
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	New York
Market Hours	Regular and Extended

Requesting Data

To add US Equities data to your algorithm, call the `add_equity` method. Save a reference to the Equity `Symbol` so you can access the data later in your algorithm.

```py
class USEquityDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2018, 1, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)
        # Subscribe to AAPL data
        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
```

For more information about creating US Equity subscriptions, see Requesting Data or US Equity Universes .

Accessing Data

To get the current US Equities data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Equity `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    # Access data: TradeBar data
    if self.aapl in slice.bars:
        trade_bar = slice.bars[self.aapl]
        self.log(f"{self.aapl} close at {slice.time}: {trade_bar.close}")

    # Access data: QuoteBar data
    if self.aapl in slice.quote_bars:
        quote_bar = slice.quote_bars[self.aapl]
        self.log(f"{self.aapl} bid at {slice.time}: {quote_bar.bid.close}")

    # Access data: Ticks data
    if self.aapl in slice.ticks:
        ticks = slice.ticks[self.aapl]
        for tick in ticks:
            self.log(f"{self.aapl} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    # Iterate all TradeBar received
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    # Iterate all QuoteBar received
    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    # Iterate all Ticks received
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing US Equities data, see Handling Data .

Historical Data

To get historical US Equity data, call the `history` method with the Equity `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.aapl, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.aapl, 100, Resolution.DAILY)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.aapl, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.aapl, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a universe of US Equities, see Equity Universes .

Remove Subscriptions

To unsubscribe from a US Equity that you added with the `add_equity` method, call the `remove_security` method.

```
self.remove_security(self.aapl)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings.

Supported Assets

To view the supported assets in the US Equities dataset, see the Data Explorer . This dataset doesn't include Over-the-Counter (OTC) stocks.

Example Applications

The US Equities dataset enables you to accurately design Equity trading strategies. Examples include the following strategies:

- Momentum strategies using historical returns on the premise that the momentum will continue
- Value strategies using fundamental factors on the premise that the price of undervalued securities will rise
- Factor investing with periodic rebalancing

Classic Algorithm Example

The following example algorithm buys and holds Apple stock:

```
from AlgorithmImports import *


class USEquityDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting single equity data, since we only trade AAPL
        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol

        # Historical data
        history = self.history(self.aapl, 60, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def on_data(self, slice: Slice) -> None:
        # Check if the current slice containing AAPL and if we hold any position
        # As we make use of the most updated price data to decide the order size
        if slice.contains_key(self.aapl) and slice[self.aapl] is not None and not self.portfolio.invested:
            self.set_holdings(self.aapl, 1)
```

Framework Algorithm Example

The following example algorithm buys and holds Apple stock:

```
from AlgorithmImports import *


class USEquityDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY
        # To select only AAPL, use a manual selection universe
        symbols = [Symbol.create("AAPL", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))

        # Constant investment signal
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(days=7), 0.025, None))

        # Invest in all members equally
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

Data Point Attributes

The US Equities dataset provides `TradeBar` , `QuoteBar` , and `Tick` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

AlgoSeek

US Equity Options

Introduction

The US Equity Options data by AlgoSeek provides Option data, including prices, strikes, expires, and open interest. The data covers 4,000 Symbols, starts in January 2012, and is delivered on a minute frequency. This dataset is created by monitoring Options Price Reporting Authority (OPRA) data feed, which consolidates last sale and quotation information originating from the national securities exchanges that have been approved by the Securities and Exchange Commission.

This dataset depends on the following datasets:

- US Equity Security Master - data on splits, dividends, and symbol changes for the underlying Equities.
- US Equity Option Universe - data on the available contracts and their daily Greeks and implied volatility values.
- US Equities - trade and quote data on the underlying Equities.

For more information about the US Equity Options dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

AlgoSeek was in 2014 with the goal of providing the highest quality, most accurate, ready-to-use data in the financial data industry. AlgoSeek provides access to Equities, ETFs, ETNs, Equity Indices, Equity Options, Futures, and Future Options for quantitative firms and traders.

Getting Started

The following snippet demonstrates how to request data from the US Equity Options dataset:

```py
option = self.add_option("GOOG")
self.option_symbol = option.symbol
option.set_filter(-2, +2, 0, 180)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012*
Asset Coverage	4,000 Symbols
Data Density	Dense
Resolution	Minute, Hourly, & Daily
Timezone	New York
Market Hours	Regular Only

* Some data is available before this date. In 2012, AlgoSeek started to fetch data from 48 OPRA channels instead of 24, increasing the quality of the data.

Requesting Data

To add US Equity Options data to your algorithm, call the `add_option` method. Save a reference to the Equity Option `Symbol` so you can access the data later in your algorithm.

```
class USEquityOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Request GOOG option data
        option = self.add_option("GOOG")
        self.option_symbol = option.symbol
        # Set our strike/expiry filter for this option chain
        option.set_filter(-2, +2, 0, 180)
```

The Equity resolution must be less than or equal to the Equity Option resolution. For example, if you set the Equity resolution to minute, then you must set the Equity Option resolution to minute, hour, or daily.

For more information about creating US Equity Option subscriptions, see Requesting Data or Equity Options Universes .

Accessing Data

To get the current US Equity Options data, index the `option_chains` property of the current `Slice` with the canonical Equity Option `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your Index Option at every time step. To avoid issues, call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    # Get the wanted option chain with the canonical symbol
    chain = slice.option_chains.get(self.option_symbol)
    if chain:
        # Iterate the option contracts in chain
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

You can also iterate through all of the `OptionChain` objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all option chains of all symbols
    for canonical_symbol, chain in slice.option_chains.items():
        # Iterate the option contracts in chain
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

For more information about accessing US Equity Options data, see Handling Data .

Historical Data

You can get historical US Equity Options data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Equity Options data in an algorithm, call the `history` method with the Equity Option contract `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame of trade and quote data
history_df = self.history(contract.symbol, 100, Resolution.MINUTE)

# DataFrame of open interest data
history_oi_df = self.history(OpenInterest, contract.symbol, 100, Resolution.MINUTE)

# TradeBar objects
history_trade_bars = self.history[TradeBar](contract.symbol, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](contract.symbol, 100, Resolution.MINUTE)

# OpenInterest objects
history_oi = self.history[OpenInterest](contract.symbol, 100, Resolution.MINUTE)
```

For more information about historical data in algorithms, see History Requests .

Historical Data In Research

To get historical US Equity Options data in the Research Environment, call the `history` or `option_history` method. The `history` method returns the price, volume, and open interest history for some given Option contract(s). The `option_history` method returns the price and volume history for the contracts that pass your daily universe filter.

```py
qb = QuantBook()
option = qb.add_option("GOOG")
option.set_filter(-2, 2, 0, 90)
history = qb.option_history(option.symbol.underlying, datetime(2020, 6, 1), datetime(2020, 6, 5))
history_df = history.data_frame
expiries = history.get_expiry_dates()
strikes = history.get_strikes()
```

To get historical data for arbitrary US Equity Option contracts instead of just the that pass your universe filter, call the `history` method like you would in an algorithm, but on the `QuantBook` object. For more information about historical data in the Research Environment, see Key Concepts .

Historical Greeks and IV Data

To get historical data for the Greeks and implied volatility of Equity Options, see the US Equity Option Universe dataset.

Supported Assets

To view the supported assets in the US Equity Options dataset, see the Data Explorer .

Example Applications

The US Equity Options dataset enables you to accurately design Option strategies. Examples include the following strategies:

- Buying put Options to hedge against downward price movement in positive Equity positions
- Exploiting arbitrage opportunities that arise when the price of Option contracts deviates from their theoretical value

Classic Algorithm Example

The following example algorithm subscribes to Google Options that fall within two strikes of the underlying stock price and expire within seven days. Within this Option chain, the algorithm buys the call Option contract that has the furthest expiry and has its strike price closest to the underlying stock price. When the contract expires, the algorithm rolls over to the next contract that meets this criteria.

```py
from AlgorithmImports import *

class USEquityOptionsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Requesting data
        self.underlying = self.add_equity("GOOG").symbol
        option = self.add_option("GOOG")
        self.option_symbol = option.symbol
        # To speculate trade the underlying with a low cost, filter for the ATM calls that expiring in the current week
        # -2/+2 strike buffer is given for small price change
        option.set_filter(lambda u: u.calls_only().strikes(-2, +2).expiration(0, 6))

        self.contract = None

    def on_data(self, slice: Slice) -> None:
        # If the Option contract is exercised, close the underlying position.
        if self.portfolio[self.underlying].invested:
            self.liquidate(self.underlying)
        # If the Option contract expires, rollover to the next contract.
        if self.contract and not self.portfolio[self.contract.symbol].invested:
            self.contract = None
        # Select with the lastest option chain data only
        chain = slice.option_chains.get(self.option_symbol)
        if not self.contract and chain:
            # Select the call contracts with the furthest expiration (week end)
            furthest_expiry = max([c.expiry for c in chain])
            furthest_expiry_calls = [contract for contract in chain if contract.expiry == furthest_expiry]
            # Get the ATM call for speculate trade with low cost and limited loss
            self.contract = sorted(furthest_expiry_calls, key = lambda x: abs(chain.underlying.price - x.strike))[0]
            self.market_order(self.contract.symbol, 1)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 10, Resolution.MINUTE)
            self.debug(f"We got {len(history)} from our history request for {security.symbol}")
```

Framework Algorithm Example

The following example algorithm buys a call Option contract for Google that falls within one strike of the underlying stock price and expires within seven days. When the contract expires, the algorithm rolls over to the next contract that meets this criteria.

```py
from AlgorithmImports import *

class USEquityOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Requesting data
        self.set_universe_selection(EarliestExpiringWeeklyAtTheMoneyCallOptionUniverseSelectionModel())

        self.set_alpha(ConstantOptionsAlphaModel())

        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())


class EarliestExpiringWeeklyAtTheMoneyCallOptionUniverseSelectionModel(OptionUniverseSelectionModel):

    def __init__(self) -> None:
        # Daily update with the select_option_chain_symbols function
        super().__init__(timedelta(1), self.select_option_chain_symbols)

    def select_option_chain_symbols(self, utcTime: datetime) -> List[Symbol]:
        # Always select only GOOG options as our focus
        return [ Symbol.create("GOOG", SecurityType.OPTION, Market.USA) ]

    def filter(self, filter: OptionFilterUniverse) -> OptionFilterUniverse:
        # To speculate trade the underlying with a low cost, filter for the ATM calls that expiring in the current week
        # -1/+1 strike buffer is given for small price change
        return (filter.calls_only()
                      .strikes(-1, -1)
                      .expiration(0, 7))


class ConstantOptionsAlphaModel(AlphaModel):

    underlying = None
    contract = None

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:

        insights = []

        # Liquidate the underlying if the option is being exercised/assigned
        if algorithm.portfolio[self.underlying].invested:
            insights.append(Insight.price(self.underlying, timedelta(days=7), InsightDirection.FLAT))

        if self.contract is not None and algorithm.portfolio[self.contract.symbol].invested:
            return insights

        # Get the ATM call for speculate trade with low cost and limited loss that expires at week end
        for kvp in slice.option_chains:
            chain = kvp.Value
            expiry = max(x.expiry for x in chain)
            self.contract = sorted([x for x in chain if x.expiry == expiry],
                                           key=lambda x: abs(x.strike - x.underlying_last_price))[0]
            insights.append(Insight.price(self.contract.symbol, self.contract.expiry + timedelta(days=1), InsightDirection.UP))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            if security.type == SecurityType.EQUITY:
                self.underlying = security.symbol
            else:
                # Historical data
                history = algorithm.history(security.symbol, 10, Resolution.MINUTE)
                algorithm.debug(f"We got {len(history)} from our history request for {security.symbol}")


class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):

    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Equity Options dataset provides `TradeBar` , `QuoteBar` , and `OpenInterest` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

OpenInterest Attributes

`OpenInterest` objects have the following attributes:

AlgoSeek

US Future Options

Introduction

The US Future Options dataset by AlgoSeek provides Option data on US Future contracts, including prices, strikes, and expires. The data covers 15 Monthly Future contracts, starts in January 2012, and is delivered on a minute frequency. This dataset is created by monitoring the trading activity on the CME, CBOT, NYMEX, and COMEX markets.

This dataset depends on the following datasets:

- US Future Option Universe - data on the available US Future Options contracts and their current Open Interest.
- US Futures Security Master - data to construct continuous Futures.
- US Future Universe - data on the available US Future contracts, their daily trading volume, and Open Interest.
- US Futures - data on the underlying Futures contracts, including price, volume, Open Interest, and expiry.

For more information about the US Future Options dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

AlgoSeek is a leading historical intraday US market data provider offering the most comprehensive and detailed market data and analytics products in the financial industry covering equities, futures, options, cash forex, and cryptocurrencies. AlgoSeek data is built for quantitative trading and machine learning. For more information about AlgoSeek, visit algoseek.com .

Getting Started

The following snippet demonstrates how to request data from the US Future Options dataset:

```
PY
future = self.add_future(Futures.Metals.GOLD, Resolution.MINUTE)
future.set_filter(0, 90)
self.add_future_option(future.symbol, lambda universe: universe.strikes(-5, +5))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012
Asset Coverage	15 Monthly Future Contracts. Standard expires only*.
Data Density	Dense
Resolution	Minute, Hourly, & Daily
Timezone	New York
Market Hours	Regular and Extended

* No weeklies or 0DTE contracts.

Requesting Data

To add US Future Options data to your algorithm, call the `add_future_option` method.

```
class FutureOptionDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 1, 28)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        future = self.add_future(Futures.Metals.GOLD, Resolution.MINUTE)
        future.set_filter(0, 90)
        self.add_future_option(future.symbol, lambda universe: universe.strikes(-5, +5))
```

The Future resolution must be less than or equal to the Future Option resolution. For example, if you set the Future resolution to minute, then the Future Option resolution must be minute, hour, or daily.

For more information about creating Future Options subscriptions, see Requesting Data or Future Options Universes .

Accessing Data

To get the current Future Options data, iterate through the `option_chains` property of the current `Slice` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your Future Options at every time step.

```
def on_data(self, slice: Slice) -> None:
    for canonical_fop_symbol, chain in slice.option_chains.items():
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

You can also iterate through the `futures_chains` in the current `Slice` first.

```
def on_data(self, slice: Slice) -> None:
    for continuous_future_symbol, futures_chain in slice.futures_chains.items():
        # Select a Future Contract and create its canonical FOP Symbol
        futures_contract = [contract for contract in futures_chain][0]
        canonical_fop_symbol = Symbol.create_canonical_option(futures_contract.symbol)
        option_chain = slice.option_chains.get(canonical_fop_symbol)
        if option_chain:
            for fop_contract in option_chain:
                 self.log(f"{fop_contract.symbol} price at {slice.time}: {fop_contract.last_price}")
```

For more information about accessing Future Options data, see Handling Data .

Historical Data

You can get historical US Future Options data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Future Options data in an algorithm, call the `history` method with the Future Option contract `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame of trade and quote data
history_df = self.history(contract.symbol, 100, Resolution.MINUTE)

# DataFrame of open interest data
history_oi_df = self.history(OpenInterest, contract.symbol, 100, Resolution.MINUTE)

# TradeBar objects
history_trade_bars = self.history[TradeBar](contract.symbol, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](contract.symbol, 100, Resolution.MINUTE)

# OpenInterest objects
history_oi = self.history[OpenInterest](contract.symbol, 100, Resolution.MINUTE)
```

For more information about historical data in algorithms, see History Requests .

Historical Data In Research

To get historical US Future Options data in the Research Environment, call the `history` or `option_history` method. The `history` method returns the

price, volume, and open interest history for some given Future Option contract(s). The `option_history` method returns the price and volume history for the contracts that pass your daily universe filter.

```py
qb = QuantBook()
future = qb.add_future(Futures.Indices.SP_500_E_MINI)
start_date = datetime(2024, 1, 2)
future_contract_symbol = sorted(
    qb.future_chain_provider.get_future_contract_list(future.symbol, start_date),
    key=lambda s: s.id.date
)[0]
history = qb.option_history(
    future_contract_symbol, start_date, future_contract_symbol.id.date, Resolution.HOUR
)
history_df = history.data_frame
expiries = history.get_expiry_dates()
strikes = history.get_strikes()
```

To get historical data for arbitrary US Equity Option contracts instead of just the that pass your universe filter, call the `history` method like you would in an algorithm, but on the `QuantBook` object. For more information about historical data in the Research Environment, see Key Concepts .

Supported Assets

The following list shows the available (15) Futures Options:

- `Futures.Energy.CRUDE_OIL_WTI` : Crude Oil WTI Futures (NYMEX: LO | Underlying: CL)
- `Futures.Energy.GASOLINE` : Gasoline RBOB Futures (NYMEX: OB | Underlying: RB)
- `Futures.Energy.HEATING_OIL` : Heating Oil Futures (NYMEX: OH | Underlying: HO)
- `Futures.Energy.NATURAL_GAS` : Natural Gas Futures (NYMEX: ON | Underlying: NG)
- `Futures.Financials.Y_10_TREASURY_NOTE` : 10Y U.S. Treasury Note Futures (CBOT: OZN | Underlying: ZN)
- `Futures.Financials.Y_2_TREASURY_NOTE` : 2Y U.S. Treasury Note Futures (CBOT: OZT | Underlying: ZT)
- `Futures.Financials.Y_30_TREASURY_BOND` : 30Y U.S. Treasury Bond Futures (CBOT: OZB | Underlying: ZB)
- `Futures.Grains.CORN` : Corn Futures (CBOT: OZC | Underlying: ZC)
- `Futures.Grains.SOYBEANS` : Soybeans Futures (CBOT: OZS | Underlying: ZS)
- `Futures.Grains.WHEAT` : Default wheat contract is SRWWheat (CBOT: OZW | Underlying: ZW)
- `Futures.Indices.NASDAQ_100_E_MINI` : E-mini NASDAQ 100 Futures (CME: NQ)
- `Futures.Indices.SP_500_E_MINI` : E-mini S&P 500 Futures (CME: ES)
- `Futures.Metals.COPPER` : Copper Futures (COMEX: HXE | Underlying: HG)
- `Futures.Metals.GOLD` : Gold Futures (COMEX: OG | Underlying: GC)
- `Futures.Metals.SILVER` : Silver Futures (COMEX: SO | Underlying: SI)

Example Applications

The US Future Options dataset enables you to accurately design Future Option strategies. Examples include the following strategies:

- Selling out of the money Future Option contracts to collect the premium that the Option buyer pays
- Buying put Options to hedge against downward price movement in Future contracts you bought
- Exploiting arbitrage opportunities that arise when the price of Option contracts deviates from their theoretical value

Classic Algorithm Example

The following example demonstrates a weekly-renewing covered call strategy to collect credit of selling the option. It filters the ATM call contract that expires within the current week at week start using `SetFilter` filtering function.

```py
from AlgorithmImports import *

class FutureOptionExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(5_000_000)
        # Subscribe the underlying since the updated price is needed for filtering
        self.underlying = self.add_future(Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        # Filter the underlying continuous Futures to narrow the FOP spectrum
        self.underlying.set_filter(0, 182)
        # Filter for the current-week-expiring calls to formulate a covered call that expires at the end of week
        self.add_future_option(self.underlying.symbol, lambda u: u.calls_only().expiration(0, 5))

    def on_data(self, slice: Slice) -> None:
        # Create canonical symbol for the mapped future contract, since option chains are mapped by canonical symbol
        symbol = Symbol.create_canonical_option(self.underlying.mapped)

        # Get option chain data for the mapped future, as both the underlying and FOP have the highest liquidity among all other
contracts
        chain = slice.option_chains.get(symbol)
        if not self.portfolio.invested and chain:
            # Obtain the ATM call that expires at the end of week, such that both underlying and the FOP expires the same time
            expiry = max(x.expiry for x in chain)
            atm_call = sorted([x for x in chain if x.expiry == expiry],
                key=lambda x: abs(x.strike - x.underlying_last_price))[0]

            # Use abstraction method to order a covered call to avoid manual error
            option_strategy = OptionStrategies.covered_call(symbol, atm_call.strike,expiry)
            self.buy(option_strategy, 1)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            if security.type == SecurityType.FUTURE_OPTION:
                # Historical data
                history = self.history(security.symbol, 10, Resolution.MINUTE)
```

Framework Algorithm Example

The following example demonstrates a daily renewing speculation trade on upward movement of Gold with FOP using algorithm framework. Using FOP, traders can achieve lower cost to get similar absolute profit, given the FOP is selected to be deep ITM with high Delta value.

```py
from AlgorithmImports import *

class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Override an option universe selection model to select the FOPs
        self.set_universe_selection(GoldFOPUniverseSelectionModel(self))
        # Handle the FOP trading logic in an alpha model
        self.add_alpha(ConstantFutureOptionsAlphaModel())
        # To order a single contract per insight, use a custom portfolio construction model
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())


class GoldFOPUniverseSelectionModel(OptionUniverseSelectionModel):

    # Daily renewed universe since option contract list is updated on a daily basis
    def __init__(self, algorithm) -> None:
        self._algorithm = algorithm
        super().__init__(timedelta(1), self.select_fop_symbols)

    def select_fop_symbols(self, utc_time: datetime) -> List[Symbol]:
        #  Create the underlying symbol to get the FOP contracts later
        future_symbol = Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
        # Get all gold FOP contracts of the front month gold contract expiring within 3 months, since their liquidity is the highest
        future_contract_list = self._algorithm.future_chain_provider.get_future_contract_list(future_symbol, self._algorithm.time)
        return [Symbol.create_canonical_option(x) for x in future_contract_list
                    if x.id.date <= self._algorithm.time + timedelta(90)]

    def filter(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Filter for the ATM calls that expires latest (same expiry as the underlying)
        # +/-5 strike range buffer for price movement
        return option_filter_universe.back_month().strikes(-5, +5).calls_only()


class ConstantFutureOptionsAlphaModel(AlphaModel):

    # A dictionary to cache the mapped FOP of the underlying Future for filtering wanted option chain data and check if invested
    option_contract_by_future_underlying_contract = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        for kvp in slice.option_chains:
            # Liquidate underlying Future contract after Option assignment
            underlying_future_contract = kvp.key.underlying
            if algorithm.portfolio[underlying_future_contract].invested:
                algorithm.insights.cancel([underlying_future_contract])
                del self.option_contract_by_future_underlying_contract[underlying_future_contract]

            # Do not repeatly invest in the same underlying's FOP
            chain = [contract for contract in kvp.value if algorithm.securities[contract.symbol].is_tradable]
            if not chain or underlying_future_contract in self.option_contract_by_future_underlying_contract:
                continue

            # Select the Option contract with the lowest strike price to speculate trade the underlying with lowest cost and highest
delta
            contract = sorted(chain, key=lambda x: x.strike)[0]
            insights.append(Insight.price(contract.symbol, contract.expiry + timedelta(1), InsightDirection.UP))
            self.option_contract_by_future_underlying_contract[underlying_future_contract] = contract

        return insights


class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):

    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Use integer target to create a portfolio target to trade a single contract
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Future Options dataset provides `TradeBar` , `QuoteBar` , and `OpenInterest` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

OpenInterest Attributes

`OpenInterest` objects have the following attributes:

AlgoSeek

US Futures

Introduction

The US Futures dataset by AlgoSeek provides Futures data, including price, volume, open interest, and expiry. The data covers the 157 most liquid contracts, starts in May 2009, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on the CFE *, CBOT, CME, COMEX, NYMEX, and ICE *.

This dataset also depends on the US Futures Security Master because the US Futures Security Master dataset contains information to construct continuous Futures.

For more information about the US Futures dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

AlgoSeek is a leading historical intraday US market data provider offering the most comprehensive and detailed market data and analytics products in the financial industry covering equities, futures, options, cash forex, and cryptocurrencies. AlgoSeek data is built for quantitative trading and machine learning. For more information about AlgoSeek, visit algoseek.com .

Getting Started

The following snippet demonstrates how to request data from the US Futures dataset:

```
future = self.add_future(Futures.Metals.GOLD)
future.set_filter(0, 90)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	May 2009
Asset Coverage	157 Futures
Data Density	Dense
Resolution	Tick, Second, Minute, Hour, & Daily
Timezone	• Chicago (CFE, CME or CBOT) • New York (ICE, NYMEX or COMEX)
Market Hours	Regular and Extended

This dataset only includes Sugar (SB) from the ICE exchange.

This data provider doesn't include a live data feed for the CFE market (VIX).

LBS (Random Length Lumber) was deprecated for LBR (Lumber).

Volume Discrepancies

The volume of the daily trade bars from this dataset is different from the daily volume that CME and other platforms like Yahoo Finance report because the CME includes pit trades in their daily volume calculation. In contrast, the volume of the daily trade bars in this dataset (from AlgoSeek) doesn't include pit trades because pit trades are over-the-counter, which algorithms can't trade.

Another discrepancy occurs from the start and end times of the daily bars. Daily bars from the CME and Yahoo Finance span from 5 PM Central Time (CT) to 4 PM CT on the following day. In contrast, QuantConnect consolidates daily bars from from 12 AM Eastern Time (ET) to 12 AM ET the following day. Therefore, to calculate the daily volume in the same way that CME does (excluding the pit trades), sum the volume of intraday bars that span 5 PM CT to 4 PM CT on the following day. For an example implementation, see Examples .

Requesting Data

To add US Futures data to your algorithm, call the `add_future` method. Save a reference to the Future so you can access the data later in your algorithm.

```
class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2013, 12, 20)
        self.set_end_date(2014, 2, 20)
        self.set_cash(1000000)
        self.universe_settings.asynchronous = True
        # Request Gold Future data
        future = self.add_future(Futures.Metals.GOLD)
        # Set filter to obtain the contracts expire within 90 days
        future.set_filter(0, 90)
        self.future_symbol = future.symbol
```

PY

For more information about creating Future subscriptions, see Requesting Data or Futures Universes .

Accessing Data

To get the current US Futures data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Future `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    # Access data: TradeBar data
    if self.future_symbol in slice.bars:
        trade_bar = slice.bars[self.future_symbol]
        self.log(f"{self.future_symbol} close at {slice.time}: {trade_bar.close}")

    # Access data: QuoteBar data
    if self.future_symbol in slice.quote_bars:
        quote_bar = slice.quote_bars[self.future_symbol]
        self.log(f"{self.future_symbol} bid at {slice.time}: {quote_bar.bid.close}")

    # Access data: Ticks data
    if self.future_symbol in slice.ticks:
        ticks = slice.ticks[self.future_symbol]
        for tick in ticks:
            self.log(f"{self.future_symbol} price at {slice.time}: {tick.price}")
```

PY

You can also iterate through all of the data objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all TradeBar received
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    # Iterate all QuoteBar received
    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    # Iterate all Ticks received
    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

PY

For more information about accessing US Futures data, see Handling Data .

Historical Data

You can get historical US Futures data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Futures data in an algorithm, call the `history` method with the canonical Futures `Symbol` or a Futures contract `Symbol` . If there is no data in the period you request, the history result is empty.

```py
# DataFrame objects
contract_history_df = self.history(contract.symbol, 100, Resolution.MINUTE)
continuous_history_df = self.history(self.future_symbol,
    start=self.time - timedelta(days=15),
    end=self.time,
    resolution=Resolution.MINUTE,
    fill_forward=False,
    extended_market_hours=False,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    data_normalization_mode=DataNormalizationMode.RAW,
    contract_depth_offset=0)

# TradeBar objects
contract_history_trade_bars = self.history[TradeBar](contract.symbol, 100, Resolution.MINUTE)
continous_history_trade_bars = self.history[TradeBar](self.future_symbol, 100, Resolution.MINUTE)

# QuoteBar objects
contract_history_quote_bars = self.history[QuoteBar](contract.symbol, 100, Resolution.MINUTE)
continous_history_quote_bars = self.history[QuoteBar](self.future_symbol, 100, Resolution.MINUTE)

# Tick objects
contract_history_ticks = self.history[Tick](contract.symbol, timedelta(seconds=10), Resolution.TICK)
continous_history_ticks = self.history[Tick](self.future_symbol, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data in algorithms, see History Requests . For more information about the price adjustments for continuous contracts, see Continous Contracts .

Historical Data In Research

To get historical US Futures data in the Research Environment for an entire Futures chain, call the `future_history` method with the canonical Future `Symbol` .

```py
qb = QuantBook()
future = qb.add_future(Futures.Metals.GOLD)
future.set_filter(0, 90)
history = qb.future_history(future.symbol, datetime(2020, 6, 1), datetime(2020, 6, 5))
history_df = history.data_frame
all_history = history.get_all_data()
expiries = history.get_expiry_dates()
```

To get historical data for a single US Futures contract or the continuous Futures contract, call the `history` method like you would in an algorithm but on the `QuantBook` object. For more information about historical data in the Research Environment, see Futures .

Supported Assets

The following list shows the available (157) Futures:

- `Futures.Currencies.AUD` : Australian Dollar Futures (CME: 6A)
- `Futures.Currencies.AUDCAD` : Australian Dollar/Canadian Dollar Futures (CME: ACD)
- `Futures.Currencies.AUDJPY` : Australian Dollar/Japanese Yen Futures (CME: AJY)
- `Futures.Currencies.AUDNZD` : Australian Dollar/New Zealand Dollar Futures (CME: ANE)
- `Futures.Currencies.BRL` : Brazillian Real Futures (CME: 6L)
- `Futures.Currencies.BTC` : Bitcoin Futures (CME: BTC)
- `Futures.Currencies.BTIC_MICRO_BTC` : BTIC on Micro Bitcoin Futures (CME: MIB)
- `Futures.Currencies.BTIC_MICRO_ETHER` : BTIC on Micro Ether Futures (CME: MRB)
- `Futures.Currencies.CAD` : Canadian Dollar Futures (CME: 6C)
- `Futures.Currencies.CADJPY` : Canadian Dollar/Japanese Yen Futures (CME: CJY)
- `Futures.Currencies.CHF` : Swiss Franc Futures (CME: 6S)
- `Futures.Currencies.ETH` : Ether Futures (CME: ETH)
- `Futures.Currencies.EUR` : Euro FX Futures (CME: 6E)
- `Futures.Currencies.EURAUD` : Euro/Australian Dollar Futures (CME: EAD)
- `Futures.Currencies.EURCAD` : Euro/Canadian Dollar Futures (CME: ECD)
- `Futures.Currencies.EURO_FX_EMINI` : E-mini Euro FX Futures (CME: E7)

- `Futures.Currencies.EURSEK` : Euro/Swedish Krona Futures (CME: ESK)
- `Futures.Currencies.GBP` : British Pound Futures (CME: 6B)
- `Futures.Currencies.JAPANESE_YEN_EMINI` : E-mini Japanese Yen Futures (CME: J7)
- `Futures.Currencies.JPY` : Japanese Yen Futures (CME: 6J)
- `Futures.Currencies.MICRO_AUD` : Micro AUD/USD Futures (CME: M6A)
- `Futures.Currencies.MICRO_BTC` : Micro Bitcoin Futures (CME: MBT)
- `Futures.Currencies.MICRO_CAD` : Micro USD/CAD Futures (CME: M6C)
- `Futures.Currencies.MICRO_CADUSD` : Micro CAD/USD Futures (CME: MCD)
- `Futures.Currencies.MICRO_CHF` : Micro CHF/USD Futures (CME: MSF)
- `Futures.Currencies.MICRO_ETHER` : Micro Ether Futures (CME: MET)
- `Futures.Currencies.MICRO_EUR` : Micro EUR/USD Futures (CME: M6E)
- `Futures.Currencies.MICRO_GBP` : Micro GBP/USD Futures (CME: M6B)
- `Futures.Currencies.MICRO_INRUSD` : Micro INR/USD Futures (CME: MIR)
- `Futures.Currencies.MICRO_JPY` : Micro JPY/USD Futures (CME: MJY)
- `Futures.Currencies.MICRO_USDCHF` : Micro USD/CHF Futures (CME: M6S)
- `Futures.Currencies.MICRO_USDCNH` : Micro USD/CNH Futures (CME: MNH)
- `Futures.Currencies.MICRO_USDJPY` : Micro USD/JPY Futures (CME: M6J)
- `Futures.Currencies.MXN` : Mexican Peso Futures (CME: 6M)
- `Futures.Currencies.NZD` : New Zealand Dollar Futures (CME: 6N)
- `Futures.Currencies.RUB` : Russian Ruble Futures (CME: 6R)
- `Futures.Currencies.STANDARD_SIZE_USD_OFFSHORE_RMBCNH` : Standard-Size USD/Offshore RMB (CNH) Futures (CME: CNH)
- `Futures.Currencies.ZAR` : South African Rand Futures (CME: 6Z)
- `Futures.Energy.ARGUS_LL_SVS_WTI_ARGUS_TRADE_MONTH` : Argus LLS vs. WTI (Argus) Trade Month Futures (NYMEX: AE5)
- `Futures.Energy.ARGUS_PROPANE_FAR_EAST_INDEX` : Argus Propane Far East Index Futures (NYMEX: A7E)
- `Futures.Energy.ARGUS_PROPANE_SAUDI_ARAMCO` : Argus Propane (Saudi Aramco) Futures (NYMEX: A9N)
- `Futures.Energy.BRENT_CRUDE_OIL_VS_DUBAI_CRUDE_OIL_PLATTS` : Brent Crude Oil vs. Dubai Crude Oil (Platts) Futures (NYMEX: ADB)
- `Futures.Energy.BRENT_LAST_DAY_FINANCIAL` : Brent Last Day Financial Futures (NYMEX: BZ)
- `Futures.Energy.CHICAGO_ETHANOL_PLATTS` : Chicago Ethaanol (Platts) Futures (NYMEX: CU)
- `Futures.Energy.CONWAY_PROPANE_OPIS` : Conway Propane (OPIS) Futures (NYMEX: A8K)
- `Futures.Energy.CRUDE_OIL_WTI` : Crude Oil WTI Futures (NYMEX: CL)
- `Futures.Energy.DUBAI_CRUDE_OIL_PLATTS_FINANCIAL` : Dubai Crude Oil (Platts) Financial Futures (NYMEX: DCB)
- `Futures.Energy.EAST_WEST_GASOLINE_SPREAD_PLATTS_ARGUS` : East-West Gasoline Spread (Platts-Argus) Futures (NYMEX: EWG)
- `Futures.Energy.EAST_WEST_NAPHTHA_JAPAN_C_FVS_CARGOES_CIFNWE_SPREAD_PLATTS` : East-West Naphtha: Japan C&F vs. Cargoes CIF NWE Spread (Platts) Futures (NYMEX: EWN)
- `Futures.Energy.ETHANOL` : Ethanol Futures (CBOT: EH)
- `Futures.Energy.ETHANOL_T_2_FOB_RDAM_INCLUDING_DUTY_PLATTS` : Ethanol T2 FOB Rdam Including Duty (Platts) Futures (NYMEX: AZ1)
- `Futures.Energy.EUROPEAN_NAPHTHA_PLATTS_CRACK_SPREAD` : European Naphtha (Platts) Crack Spread Futures (NYMEX: EN)
- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS` : European Propane CIF ARA (Argus) Futures (NYMEX: APS)
- `Futures.Energy.EUROPEAN_PROPANE_CIFARA_ARGUS_VS_NAPHTHA_CARGOES_CIFNWE_PLATTS` : European Propane CIF ARA (Argus) vs. Naphtha Cargoes CIF NWE (Platts) Futures (NYMEX: EPN)
- `Futures.Energy.FREIGHT_ROUTE_TC_14_BALTIC` : Freight Route TC14 (Baltic) Futures (NYMEX: FRC)
- `Futures.Energy.GASOLINE` : Gasoline RBOB Futures (NYMEX: RB)
- `Futures.Energy.GASOLINE_EUROBOB_OXY_NWE_BARGES_ARGUS` : Gasoline Euro-bob Oxy NWE Barges (Argus) Futures (NYMEX: B7H)
- `Futures.Energy.GROUP_THREE_SUBOCTANE_GASOLINE_PLATTS_VS_RBOB` : Group Three Sub-octane Gasoliine (Platts) vs. RBOB Futures (NYMEX: AA8)
- `Futures.Energy.GROUP_THREE_ULSD_PLATTS_VS_NY_HARBOR_ULSD` : Group Three ULSD (Platts) vs. NY Harbor ULSD Futures (NYMEX: AA6)
- `Futures.Energy.GULF_COAST_CBOB_GASOLINE_A_2_PLATTS_VS_RBOB_GASOLINE` : Gulf Coast CBOB Gasoline A2 (Platts) vs. RBOB Gasoline Futures (NYMEX: CRB)
- `Futures.Energy.GULF_COAST_HSFO_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Gulf Coast HSFO (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: GCU)
- `Futures.Energy.HEATING_OIL` : Heating Oil Futures (NYMEX: HO)
- `Futures.Energy.LOS_ANGELES_CARBOB_GASOLINE_OPI_SVS_RBOB_GASOLINE` : Los Angeles CARBOB Gasoline (OPIS) vs. RBOB Gasoline Futures

(NYMEX: AJL)

- `Futures.Energy.LOS_ANGELES_CARB_DIESEL_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles CARB Diesel (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AKL)
- `Futures.Energy.LOS_ANGELES_JET_OPI_SVS_NY_HARBOR_ULSD` : Los Angeles Jet (OPIS) vs. NY Harbor ULSD Futures (NYMEX: AJS)
- `Futures.Energy.MARS_ARGUS_VS_WTI_FINANCIAL` : Mars (Argus) vs. WTI Financial Futures (NYMEX: AYX)
- `Futures.Energy.MARS_ARGUS_VS_WTI_TRADE_MONTH` : Mars (Argus) vs. WTI Trade Month Futures (NYMEX: AYV)
- `Futures.Energy.MICRO_CRUDE_OIL_WTI` : Micro WTI Crude Oil Futures (NYMEX: MCL)
- `Futures.Energy.MICRO_EUROPEAN_FOB_RDAM_MARINE_FUEL_ZERO_POINT_FIVE_PERCENT_BARGES_PLATTS` : Micro European FOB Rdam Marine Fuel 0.5% Barges (Platts) Futures (NYMEX: R5O)
- `Futures.Energy.MICRO_EUROPEAN_THREE_POINT_FIVE_PERCENT_OIL_BARGES_FOB_RDAM_PLATTS` : Micro European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: MEF)
- `Futures.Energy.MICRO_GASOIL_ZERO_POINT_ONE_PERCENT_BARGES_FOBARA_PLATTS` : Micro Gasoil 0.1% Barges FOB ARA (Platts) Futures (NYMEX: M1B)
- `Futures.Energy.MICRO_SINGAPORE_FOB_MARINE_FUEL_ZERO_POINT_FIVE_PERCET_PLATTS` : Micro Singapore FOB Marine Fuel 0.5% (Platts) Futures (NYMEX: S5O)
- `Futures.Energy.MICRO_SINGAPORE_FUEL_OIL_380_CST_PLATTS` : Micro Singapore Fuel Oil 380CST (Platts) Futures (NYMEX: MAF)
- `Futures.Energy.MINI_EUROPEAN_THREE_POINT_PERCENT_FIVE_FUEL_OIL_BARGES_PLATTS` : Mini European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: A0D)
- `Futures.Energy.MINI_SINGAPORE_FUEL_OIL_180_CST_PLATTS` : Mini Singapore Fuel Oil 180 cst (Platts) Futures (NYMEX: A0F)
- `Futures.Energy.MONT_BELVIEU_ETHANE_OPIS` : Mont Belvieu Ethane (OPIS) Futures (NYMEX: AC0)
- `Futures.Energy.MONT_BELVIEU_LDH_PROPANE_OPIS` : Mont Belvieu LDH Propane (OPIS) Futures (NYMEX: B0)
- `Futures.Energy.MONT_BELVIEU_NATURAL_GASOLINE_OPIS` : Mont Belvieu Natural Gasoline (OPIS) Futures (NYMEX: A7Q)
- `Futures.Energy.MONT_BELVIEU_NORMAL_BUTANE_OPIS` : Mont Belvieu Normal Butane (OPIS) Futures (NYMEX: AD0)
- `Futures.Energy.NATURAL_GAS` : Natural Gas Futures (NYMEX: NG)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_LAST_DAY_FINANCIAL` : Natural Gas (Henry Hub) Last-day Financial Futures (NYMEX: HH)
- `Futures.Energy.NATURAL_GAS_HENRY_HUB_PENULTIMATE_FINANCIAL` : Natural Gas (Henry Hub) Penultimate Financial Futures (NYMEX: HP)
- `Futures.Energy.ONE_PERCENT_FUEL_OIL_CARGOES_FOBNWE_PLATTS_VS_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : 1% Fuel Oil Cargoes FOB NWE (Platts) vs. 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: FSS)
- `Futures.Energy.PREMIUM_UNLEADED_GASOLINE_10_PPM_FOBMED_PLATTS` : Premium Unleaded Gasoline 10 ppm FOB MED (Platts) Futures (NYMEX: A3G)
- `Futures.Energy.PROPANE_NON_LDH_MONT_BELVIEU_OPIS` : Propane Non-LDH Mont Belvieu (OPIS) Futures (NYMEX: A1R)
- `Futures.Energy.RBOB_GASOLINE_CRACK_SPREAD` : RBOB Gasoline Crack Spread Futures (NYMEX: ARE)
- `Futures.Energy.RBOB_GASOLINE_VS_EUROBOB_OXY_NWE_BARGES_ARGUS_THREE_HUNDRED_FIFTY_THOUSAND_GALLONS` : RBOB Gasoline vs. Eurobob Oxy NWE Barges (Argus) (350,000 gallons) Futures (NYMEX: EXR)
- `Futures.Energy.SINGAPORE_FUEL_OIL_380_CST_PLATTS_VS_EUROPEAN_THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS` : Singapore Fuel Oil 380 cst (Platts) vs. European 3.5% Fuel Oil Barges FOB Rdam (Platts) Futures (NYMEX: EVC)
- `Futures.Energy.SINGAPORE_GASOIL_PLATTS_VS_LOW_SULPHUR_GASOIL_FUTURES` : Singapore Gasoil (Platts) vs. Low Sulphur Gasoil Futures (NYMEX: AGA)
- `Futures.Energy.SINGAPORE_MOGAS_92_UNLEADED_PLATTS_BRENT_CRACK_SPREAD` : Singapore Mogas 92 Unleaded (Platts) Brent Crack Spread Futures (NYMEX: D1N)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD` : 3.5% Fuel Oil Barges FOB Rdam (Platts) Crack Spread Futures (NYMEX: FO)
- `Futures.Energy.THREE_POINT_FIVE_PERCENT_FUEL_OIL_BARGES_FOB_RDAM_PLATTS_CRACK_SPREAD_1000_MT` : 3.5% Fuel Oil Barges FOB Rdam (Platts) Crack Spread (1000mt) Futures (NYMEX: BOO)
- `Futures.Energy.WTI_BRENT_FINANCIAL` : WTI-Brent Financial Futures (NYMEX: BK)
- `Futures.Energy.WTI_FINANCIAL` : WTI Financial Futures (NYMEX: CSX)
- `Futures.Energy.WTI_HOUSTON_ARGUS_VS_WTI_TRADE_MONTH` : WTI Houston (Argus) vs. WTI Trade Month Futures (NYMEX: HTT)
- `Futures.Energy.WTI_HOUSTON_CRUDE_OIL` : WTI Houston Crude Oil Futures (NYMEX: HCL)
- `Futures.Financials.EURO_DOLLAR` : EuroDollar Futures (CME: GE)
- `Futures.Financials.FIVE_YEAR_USDMAC_SWAP` : 5-Year USD MAC Swap Futures (CBOT: F1U)
- `Futures.Financials.MICRO_Y_10_TREASURY_NOTE` : Micro 10-Year Yield Futures (CBOT: 10Y)
- `Futures.Financials.MICRO_Y_2_TREASURY_BOND` : Micro 2-Year Yield Futures (CBOT: 2YY)

- `Futures.Financials.MICRO_Y_30_TREASURY_BOND` : Micro 30-Year Yield Futures (CBOT: 30Y)
- `Futures.Financials.MICRO_Y_5_TREASURY_BOND` : Micro 5-Year Yield Futures (CBOT: 5YY)
- `Futures.Financials.ULTRA_TEN_YEAR_US_TREASURY_NOTE` : Ultra 10-Year U.S. Treasury Note Futures (CBOT: TN)
- `Futures.Financials.ULTRA_US_TREASURY_BOND` : Ultra U.S. Treasury Bond Futures (CBOT: UB)
- `Futures.Financials.Y_10_TREASURY_NOTE` : 10Y U.S. Treasury Note Futures (CBOT: ZN)
- `Futures.Financials.Y_2_TREASURY_NOTE` : 2Y U.S. Treasury Note Futures (CBOT: ZT)
- `Futures.Financials.Y_30_TREASURY_BOND` : 30Y U.S. Treasury Bond Futures (CBOT: ZB)
- `Futures.Financials.Y_5_TREASURY_NOTE` : 5Y U.S. Treasury Note Futures (CBOT: ZF)
- `Futures.Forestry.LUMBER` : Lumber Futures (CME: LBR)
- `Futures.Forestry.RANDOM_LENGTH_LUMBER` : Random Length Lumber Futures (CME: LBS)
- `Futures.Grains.BLACK_SEA_CORN_FINANCIALLY_SETTLED_PLATTS` : Black Sea Corn Financially Settled (Platts) Futures (CBOT: BCF)
- `Futures.Grains.BLACK_SEA_WHEAT_FINANCIALLY_SETTLED_PLATTS` : Black Sea Wheat Financially Settled (Platts) Futures (CBOT: BWF)
- `Futures.Grains.CORN` : Corn Futures (CBOT: ZC)
- `Futures.Grains.HRW_WHEAT` : KC HRW Wheat Futures (CBOT: KE)
- `Futures.Grains.OATS` : Oats Futures (CBOT: ZO)
- `Futures.Grains.SOYBEANS` : Soybeans Futures (CBOT: ZS)
- `Futures.Grains.SOYBEAN_MEAL` : Soybean Meal Futures (CBOT: ZM)
- `Futures.Grains.SOYBEAN_OIL` : Soybean Oil Futures (CBOT: ZL)
- `Futures.Grains.WHEAT` : Default wheat contract is SRWWheat (CBOT: ZW)
- `Futures.Indices.BLOOMBERG_COMMODITY_INDEX` : Bloomberg Commodity Index Futures (CBOT: AW)
- `Futures.Indices.DOW_30_E_MINI` : E-mini Dow Indu 30 Futures (CBOT: YM)
- `Futures.Indices.DOW_JONES_REAL_ESTATE` : Dow Jones Real Estate futures on CME (CME: RX)
- `Futures.Indices.FTSE_EMERGING_EMINI` : E-mini FTSE Emerging Index Futures (CME: EI)
- `Futures.Indices.MICRO_DOW_30_E_MINI` : Micro E-mini Dow Jones Industrial Average Index Futures (CBOT: MYM)
- `Futures.Indices.MICRO_NASDAQ_100_E_MINI` : Micro E-mini Nasdaq-100 Index Futures (CME: MNQ)
- `Futures.Indices.MICRO_RUSSELL_2000_E_MINI` : Micro E-mini Russell 2000 Index Futures (CME: M2K)
- `Futures.Indices.MICRO_SP_500_E_MINI` : Micro E-mini S&P 500 Index Futures (CME: MES)
- `Futures.Indices.NASDAQ_100_BIOTECHNOLOGY_E_MINI` : E-mini Nasdaq-100 Biotechnology Index Futures (CME: BIO)
- `Futures.Indices.NASDAQ_100_E_MINI` : E-mini NASDAQ 100 Futures (CME: NQ)
- `Futures.Indices.NIKKEI_225_DOLLAR` : Nikkei-225 Dollar Futures (CME: NKD)
- `Futures.Indices.NIKKEI_225_YEN_CME` : Nikkei-225 Yen denominated Futures on CME (CME: NIY)
- `Futures.Indices.RUSSELL_1000_E_MINI` : E-mini Russell 1000 futures on CME (CME: RS1)
- `Futures.Indices.RUSSELL_2000_E_MINI` : E-mini Russell 2000 Futures (CME: RTY)
- `Futures.Indices.SPGSCI_COMMODITY` : S&P-GSCI Commodity Index Futures (CME: GD)
- `Futures.Indices.SP_400_MID_CAP_EMINI` : E-mini S&P MidCap 400 Futures (CME: EMD)
- `Futures.Indices.SP_500_ANNUAL_DIVIDEND_INDEX` : (CME: SDA)
- `Futures.Indices.SP_500_E_MINI` : E-mini S&P 500 Futures (CME: ES)
- `Futures.Indices.TOPIXYEN` : YEN Denominated Topix Index Futures on CME (CME: TPY)
- `Futures.Indices.USD_DENOMINATED_IBOVESPA` : USD-Denominated Ibovespa Index Futures (CME: IBV)
- `Futures.Indices.VIX` : CBOE Volatility Index Futures (CFE: VX)
- `Futures.Meats.FEEDER_CATTLE` : Feeder Cattle Futures (CME: GF)
- `Futures.Meats.LEAN_HOGS` : Lean Hogs Futures (CME: HE)
- `Futures.Meats.LIVE_CATTLE` : Live Cattle Futures (CME: LE)
- `Futures.Metals.ALUMINIUM_EUROPEAN_PREMIUM_DUTY_PAID_METAL_BULLETIN` : Aluminium European Premium Duty-Paid (Metal Bulletin) Futures (COMEX: EDP)
- `Futures.Metals.ALUMINUM_MWUS_TRANSACTION_PREMIUM_PLATTS_25_MT` : Aluminum MW U.S. Transaction Premium Platts (25MT) Futures (COMEX: AUP)
- `Futures.Metals.COPPER` : Copper Futures (COMEX: HG)
- `Futures.Metals.GOLD` : Gold Futures (COMEX: GC)
- `Futures.Metals.MICRO_GOLD` : Micro Gold Futures (COMEX: MGC)
- `Futures.Metals.MICRO_GOLD_TAS` : Micro Gold TAS Futures (COMEX: MGT)
- `Futures.Metals.MICRO_PALLADIUM` : Micro Palladium Futures (NYMEX: PAM)

- `Futures.Metals.MICRO_SILVER` : Micro Silver Futures (COMEX: SIL)
- `Futures.Metals.PALLADIUM` : Palladium Futures (NYMEX: PA)
- `Futures.Metals.PLATINUM` : Platinum Futures (NYMEX: PL)
- `Futures.Metals.SILVER` : Silver Futures (COMEX: SI)
- `Futures.Metals.US_MIDWEST_DOMESTIC_HOT_ROLLED_COIL_STEEL_CRU_INDEX` : U.S. Midwest Domestic Hot-Rolled Coil Steel (CRU) Index Futures (NYMEX: HRC)
- `Futures.Softs.SUGAR_11` : Sugar #11 Futures ICE (ICE: SB)
- `Futures.Softs.SUGAR_11_CME` : Sugar #11 Futures CME (NYMEX: YO)

Example Applications

The US Futures dataset enables you to accurately design Futures strategies. Examples include the following strategies:

- Buying the Futures contract with the most open interest to reduce slippage and market impact
- Trading bull calendar spreads to reduce volatility and margin requirements

Classic Algorithm Example

The following example algorithm buys the Mini Gold Futures contract with the greatest open interest and sells the Micro Gold Futures contract with the greatest open interest. When the open interest of a different contract exceeds the open interest of a contract in the portfolio, the algorithm rebalances the portfolio.

```py
from AlgorithmImports import *

class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)
        # Setting the continuous contract mapping criteria for both Gold and Micro Gold contracts, since we want to order the highest
liquidity contracts
        self.settings.seed_initial_prices = True
        self.gold = self.add_future(Futures.Metals.GOLD,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        self.micro_gold = self.add_future(Futures.Metals.MICRO_GOLD,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0)
        # The contract multiplier is cached in the Security symbol_properties property from the symbol properties database
        self.gold_multiplier = self.gold.symbol_properties.contract_multiplier
        self.micro_gold_multiplier = self.micro_gold.symbol_properties.contract_multiplier

    def on_data(self, slice: Slice) -> None:
        # Make sure to calculate the order size by the most updated price data of both contracts
        if not self.portfolio.invested and self.gold.symbol in slice.bars and self.micro_gold.symbol in slice.bars:
            # Calculate the order size for $500k
            # Get the quotient after dividing the contract multiplier since the order size must be whole number
            gold_quantity = 500000 / slice.bars[self.gold.symbol].close // self.gold_multiplier
            micro_gold_quantity = 500000 / slice.bars[self.micro_gold.symbol].close // self.micro_gold_multiplier

            self.market_order(self.gold.mapped, gold_quantity)
            self.market_order(self.micro_gold.mapped, micro_gold_quantity)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 10, Resolution.MINUTE)
            self.debug(f"We got {len(history)} from our history request for {security.symbol}")
```

Framework Algorithm Example

The following example algorithm buys the front-month Mini Gold Futures contract and sells the front-month Micro Gold Futures contract. When the front-month contract changes, the algorithm rebalances the portfolio.

```py
from AlgorithmImports import *

class USFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        # Set up an universe selection model that selects the front month contract
        self.set_universe_selection(FrontMonthFutureUniverseSelectionModel())
        self.add_alpha(ConstantFuturesAlphaModel())
        # A portfolio construction model that only order a single share per insight signal
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class FrontMonthFutureUniverseSelectionModel(FutureUniverseSelectionModel):
    def __init__(self,) -> None:
        # Daily updating with select_future_chain_symbols function
        super().__init__(timedelta(1), self.select_future_chain_symbols)

    def select_future_chain_symbols(self, utcTime: datetime) -> List[Symbol]:
        # Select gold and micro gold contracts for the strategy need
        future_pairs = [
            (Futures.Metals.GOLD, Market.COMEX),
            (Futures.Metals.MICRO_GOLD, Market.COMEX)
        ]
        return [Symbol.create(pair[0], SecurityType.FUTURE, pair[1]) for pair in future_pairs]

    def filter(self, filter: FutureFilterUniverse) -> FutureFilterUniverse:
        # Filter only front month contract for liquidity and most informed information
        return filter.front_month().only_apply_filter_at_market_open()

class ConstantFuturesAlphaModel(AlphaModel):
    # Long gold and short micro gold in this strategy
    long_symbol = Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
    short_symbol = Symbol.create(Futures.Metals.MICRO_GOLD, SecurityType.FUTURE, Market.COMEX)

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        if algorithm.portfolio.invested:
            return []

        insights = []
        # For both gold and micro gold, select the front month contract (only contract) in the chain
        for symbol, contracts in slice.future_chains.items():
            chain = [contract for contract in contracts]
            contract = chain[0]

            # Long gold and short micro gold as planned
            if symbol == self.long_symbol:
                insights.append(Insight.price(contract.symbol, contract.expiry + timedelta(days=1), InsightDirection.UP))
            elif symbol == self.short_symbol:
                insights.append(Insight.price(contract.symbol, contract.expiry + timedelta(days=1), InsightDirection.DOWN))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = algorithm.history(security.symbol, 10, Resolution.MINUTE)
            algorithm.debug(f"We got {len(history)} from our history request for {security.symbol}")

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Single share only using integer portfolio target
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Data Point Attributes

The US Futures dataset provides `FuturesChain` , `Future` , and `OpenInterest` objects. To configure the continuous Future settings, use the `DataNormalizationMode` and `DataMappingMode` enumerations.

DataNormalizationMode Values

The `DataNormalizationMode` enumeration has the following values:

DataMappingMode Values

The `DataMappingMode` enumeration has the following values:

Future Attributes

`Future` objects have the following attributes:

FuturesChain Attributes

`FuturesChain` objects have the following attributes:

OpenInterest Attributes

`OpenInterest` objects have the following attributes:

AlgoSeek

US Index Options

Introduction

The US Index Options dataset by AlgoSeek provides Option data, including prices, strikes, expires, and open interest. The data covers European Option contracts for 3 US Indices: SPX, VIX, and NDX. It starts from January 2012 and is delivered on minute resolution. This dataset is created by monitoring the Options Price Reporting Authority (OPRA) data feed, which consolidates last sale and quotation information originating from the national securities exchanges that have been approved by the Securities and Exchange Commission.

The US Index Options dataset depends on the US Index Option Universe dataset because the US Index Options Universe dataset contains information on the available contracts, including their daily Greeks and implied volatility values.

For more information about the US Index Options dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

AlgoSeek is a leading historical intraday US market data provider offering the most comprehensive and detailed market data and analytics products in the financial industry covering Equities, Futures, Options, cash FOREX, and Cryptocurrencies. AlgoSeek data is built for quantitative trading and machine learning. For more information about AlgoSeek, visit algoseek.com .

Getting Started

The following snippet demonstrates how to request data from the US Index Options dataset:

```py
self.index_symbol = self.add_index('VIX').symbol
option = self.add_index_option(self.index_symbol)
option.set_filter(-2, 2, 0, 90)
self.option_symbol = option.symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2012
Asset Coverage	7 Index Options
Data Density	Regular
Resolution	Minute, Hourly, & Daily
Timezone	New York
Market Hours	Regular Only

Specification Over Time

According to the SPX Options contract specification , some SPX contracts expire every month and SPXW contracts expires every day. Before 2021, you could only trade SPX contracts with the following expiration dates:

- Expires within the next 4 months
- Expires in September within the next 14 months
- Expires in January, March, or June within the next 2 years
- Expires in December within the next 3 years

During this time, SPXW didn't have 0DTE every day.

Sources :

- Cboe Options Exchange to List Three Long-Dated SPX Options Expirations, Beginning November 1, 2021
- S&P 500 Weekly Options Now Expire Five Days a Week

Requesting Data

To add US Index Options data to your algorithm, call the `add_index_option` method. Save a reference to the Index Option `Symbol` so you can access the data later in your algorithm.

```
PY
class IndexOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1);
        self.set_end_date(2021, 6, 1);
        self.set_cash(1000000);
        self.universe_settings.asynchronous = True
        self.index_symbol = self.add_index('VIX').symbol

        standard_option = self.add_index_option(self.index_symbol)
        standard_option.set_filter(-2, 2, 0, 90)
        self.standard_option_symbol = standard_option.symbol

        weekly_option = self.add_index_option(self.index_symbol, "VIXW")
        weekly_option.set_filter(-2, 2, 0, 90)
        self.weekly_option_symbol = weekly_option.symbol
```

The Index resolution must be less than or equal to the Index Option resolution. For example, if you set the Index resolution to minute, then you must set the Index Option resolution to minute, hour, or daily.

For more information about creating US Index Option subscriptions, see Requesting Data or Index Options Universes .

Accessing Data

To get the current US Index Options data, index the `option_chains` property of the current `Slice` with the canonical Index Option `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your Index Option at every time step.

```
PY
def on_data(self, slice: Slice) -> None:
    standard_chain = slice.option_chains.get(self.standard_option_symbol)
    if standard_chain:
        for contract in standard_chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")

    weekly_chain = slice.option_chains.get(self.weekly_option_symbol)
    if weekly_chain:
        for contract in weekly_chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

You can also iterate through all of the `OptionChain` objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for canonical_symbol, chain in slice.option_chains.items():
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

For more information about accessing US Index Options data, see Handling Data .

Historical Data

You can get historical US Index Options data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical US Index Options data in an algorithm, call the `history` method with the Index Option contract `Symbol` . If there is no data in the period you request, the history result is empty.

```py
# DataFrame of trade and quote data
history_df = self.history(contract.symbol, 100, Resolution.MINUTE)

# DataFrame of open interest data
history_oi_df = self.history(OpenInterest, contract.symbol, 100, Resolution.MINUTE)

# TradeBar objects
history_trade_bars = self.history[TradeBar](contract.symbol, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](contract.symbol, 100, Resolution.MINUTE)

# OpenInterest objects
history_oi = self.history[OpenInterest](contract.symbol, 100, Resolution.MINUTE)
```

For more information about historical data in algorithms, see History Requests .

Historical Data In Research

To get historical US Index Options data in the Research Environment, call the `history` or `option_history` method. The `history` method returns the price, volume, and open interest history for some given Option contract(s). The `option_history` method returns the price and volume history for the contracts that pass your daily universe filter.

```py
qb = QuantBook()
index_symbol = qb.add_index('VIX').symbol
option = qb.add_index_option(index_symbol) # or qb.add_index_option(index_symbol, "VIXW")
option.set_filter(-2, 2, 0, 90)
history = qb.option_history(option.symbol, datetime(2020, 6, 1), datetime(2020, 6, 5))
history_df = history.data_frame
expiries = history.get_expiry_dates()
strikes = history.get_strikes()
```

To get historical data for arbitrary US Index Option contracts instead of just the that pass your universe filter, call the `history` method like you would in an algorithm, but on the `QuantBook` object. For more information about historical data in the Research Environment, see Key Concepts .

Historical Greeks and IV Data

To get historical data for the Greeks and implied volatility of Index Options, see the US Index Option Universe dataset.

Supported Assets

The following table shows the available Index Options:

Underlying Index	Underlying Ticker	Target Ticker	Standard Contracts	Weekly Contracts	Tradable on Expiry Day
NASDAQ-100	NDX		✓		
NASDAQ-100	NDX	NDXP	✓		✓
NASDAQ-100	NDX	NQX	✓	✓	✓
Russell 2000	RUT		✓		
Russell 2000	RUT	RUTW		✓	✓
S&P500	SPX		✓		
S&P500	SPX	SPXW		✓	✓
S&P500	VIX		✓		
S&P500	VIX	VIXW		✓	

For more information about each underlying Index, see Supported Indices .

Example Applications

The US Index Options dataset enables you to accurately design strategies for Index Options. Examples include the following strategies:

- Buying VIX call Options to hedge against upcoming volatility
- Buying VIX put Options to capture the natural downward price movement in the VIX index
- Buying SPX put Options to protect against downward price movement in the S&P 500

Classic Algorithm Example

The following example algorithm buys the VIX Index Option Bull Call Spread with the furthest expiration:

```py
from AlgorithmImports import *

class IndexOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(200000)
        # Asynchronous can use computational resources efficiently
        self.universe_settings.asynchronous = True
        # Filter to get ATM calls expiring in 180 days to form the Bull Call Spread
        option = self.add_index_option("SPX")
        option.set_filter(lambda u: u.calls_only().strikes(-2, +2).expiration(0, 180))
        self.option_symbol = option.symbol

    def on_data(self, slice: Slice) -> None:
        if not self.portfolio.invested and self.is_market_open(self.option_symbol):
            # Make sure getting the updated VIX option chain
            chain = slice.option_chains.get(self.option_symbol)
            if chain:
                expiry = max([c.expiry for c in chain])
                call_contracts = sorted([c for c in chain if c.expiry == expiry],
                    key=lambda c: c.strike)

                # Need 2 contracts to form a call spread
                if len(call_contracts) < 2:
                    return

                # Obtain 2 call contracts with different strike price to form the call spread
                longCall, shortCall = call_contracts[0:2]

                # Use all the buying power, but need to ensure the order size of the long and short call are the same
                quantity = min([
                    abs(self.calculate_order_quantity(shortCall.symbol, -1)),
                    abs(self.calculate_order_quantity(longCall.symbol, 1))])
                if not quantity:
                    return

                # Bull call spread consists of long a lower-strike call and short a higher-strike call
                self.market_order(shortCall.symbol, -quantity)
                self.market_order(longCall.symbol, quantity)

                expected_margin_usage = max((longCall.strike - shortCall.strike) *
self.securities[longCall.symbol].symbol_properties.contract_multiplier * quantity, 0)
                if expected_margin_usage != self.portfolio.total_margin_used:
                    raise Exception("Unexpect margin used!")


    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            if security.type == SecurityType.INDEX_OPTION:
                # Historical data
                history = self.history(security.symbol, 10, Resolution.MINUTE)
```

Framework Algorithm Example

The following example algorithm buys the VIX Index Option Bull Call Spread with the furthest expiration:

```py
from AlgorithmImports import *

class IndexOptionsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(200000)
        self.universe_settings.asynchronous = True
        self.set_universe_selection(VIXIndexOptionUniverseSelectionModel())

        self.set_alpha(BullCallSpreadAlphaModel())

        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())


class VIXIndexOptionUniverseSelectionModel(OptionUniverseSelectionModel):

    def __init__(self):
        super().__init__(timedelta(1), self.option_chain_symbol_selector)

    def option_chain_symbol_selector(self, utc_time: datetime) -> List[Symbol]:
        # Only interest in VIX options
        return [Symbol.create("VIX", SecurityType.INDEX_OPTION, Market.USA)]

    def filter(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        # Filter to get ATM calls expiring in 180 days to form the Bull Call Spread
        return option_filter_universe.calls_only().strikes(-2, +2).expiration(0, 180)


class BullCallSpreadAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        if algorithm.portfolio.invested:
            return []

        for canonical_symbol, chain in slice.option_chains.items():
            if not chain:
                return []
            # Bull call spread legs are having the same expiry
            expiry = max([c.expiry for c in chain])
            call_contracts = sorted([c for c in chain if c.expiry == expiry],
                key=lambda c: c.strike)

            # Need 2 contracts to form a call spread
            if len(call_contracts) < 2:
                return []

            # Obtain 2 call contracts with different strike price to form the call spread
            long_call, short_call = call_contracts[0:2]

            # Bull call spread consists of long a lower-strike call and short a higher-strike call
            return Insight.group(
                [
                    Insight.price(long_call.symbol, expiry + timedelta(1), InsightDirection.UP),
                    Insight.price(short_call.symbol, expiry + timedelta(1), InsightDirection.DOWN)
                ])
        return []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            if security.type == SecurityType.INDEX_OPTION:
                # Historical data
                history = algorithm.history(security.symbol, 10, Resolution.MINUTE)


class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):

    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        if not insights:
            return []
        # Only a single contract toi ensure both legs size is equal
        quantity = min([abs(algorithm.calculate_order_quantity(i.symbol, int(i.direction)))
            for i in insights])

        return [PortfolioTarget(i.symbol, quantity * int(i.direction))
            for i in insights]
```

Data Point Attributes

The US Index Options dataset provides `TradeBar`, `QuoteBar`, and `OpenInterest` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

OpenInterest Attributes

`OpenInterest` objects have the following attributes:

Datasets

Morningstar

Morningstar was founded by Joe Mansueto in 1984 with the goal of empowering investors by connecting people to the investing information and tools they need. Morningstar provides access extensive line of products and services for individual investors, financial advisors, asset managers, and retirement plan providers. Morningstar provides data on approximately 525,000 investment offerings including stocks, mutual funds, and similar vehicles, along with real-time global market data on nearly 18 million equities, indexes, futures, options, commodities, and precious metals, in addition to foreign exchange and Treasury markets. Morningstar also offers investment management services through its investment advisory subsidiaries, with $244 billion in assets under advisement or management as of 2021.

US Fundamental Data

Morningstar

US Fundamental Data

Introduction

The US Fundamental Data by Morningstar tracks US Equity fundamentals. The data covers 8,000 US Equities, starts in January 1998, and is delivered on a daily frequency. The data does not include ETFs, ADRs, or OTC equities. This dataset is created by using a combination of string matching, Regular Expressions, and Machine Learning to gather the fundamental data published by companies.

For older symbols, the file date is approximated 45 days after the as of date. When a filing date is present on the Morningstar data, it is used. As we are a quant platform, all the data is loaded using "As Original Reported" figures. If there was a mistake reporting the figure, this will not be fixed later. The market typically responds quickly to these initially reported figures. Data is available for multiple periods depending on the property. Periods available include: 1 mo, 2 mo, 3 mo, 6 mo, 9 mo, 12 mo, 1 Yr, 2 Yr, 3 Yr, and 5 Yr. Morningstar symbols cover the NYSE, NASDAQ, AMEX, and BATS exchanges.

In live trading, Morningstar data is delivered to your algorithm at approximately 6 am each day. The majority of the fundamental data update occurs once per month. This includes updates to all of the key information for each security Morningstar supports. On top of this monthly update, there are daily updates of the financial ratios.

As Morningstar data arrives, it updates the master copy and is passed into your algorithm, similar to how TradeBars are fill-forwarded in your data feed. If there have been no updates for a day, you'll receive the same fundamental data.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

QuantConnect/LEAN combines the data of this dataset with US Coarse Universe data in runtime.

For more information about the US Fundamental Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Morningstar was founded by Joe Mansueto in 1984 with the goal of empowering investors by connecting people to the investing information and tools they need. Morningstar provides access extensive line of products and services for individual investors, financial advisors, asset managers, and retirement plan providers. Morningstar provides data on approximately 525,000 investment offerings including stocks, mutual funds, and similar vehicles, along with real-time global market data on nearly 18 million equities, indexes, futures, options, commodities, and precious metals, in addition to foreign exchange and Treasury markets. Morningstar also offers investment management services through its investment advisory subsidiaries, with $244 billion in assets under advisement or management as of 2021.

Getting Started

The following snippets demonstrate how to request data from the US Fundamental dataset:

- Direct Access using the Security object

```
equity = self.add_equity("IBM")
ibm_fundamental = equity.fundamentals
```

- Direct Access using the Symbol object

```
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals(ibm)
```

- Universe Selection

```
def initialize(self) -> None:
    self._universe = self.add_universe(self.fundamental_filter_function)

def fundamental_filter_function(self, fundamental: List[Fundamental]):
    filtered = [f for f in fundamental if f.has_fundamental_data and f.price > 10 and not np.isnan(f.valuation_ratios.pe_ratio)]
    sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
    return [f.symbol for f in sorted_by_pe_ratio[:10]]
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Asset Coverage	8,000 US Equities *
Corporate Indicators / Tracked Fields	900 Fields
Data Density	Sparse
Resolution	Daily
Timezone	New York

* The data does not include ETFs, ADRs, or OTC equities.

Requesting Data

You don't need any special code to request US Fundamental Data. You can access the current and historical fundamental data for any of the US Equities that this dataset includes.

```
class MorningStarDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 7, 1)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True

        # Option 1: Subscribe to individual US Equity assets
        self.add_equity("IBM")

        # Option 2A: Create a fundamental universe (classic version)
        self._universe = self.add_universe(self.fundamental_function)

        # Option 2B: Create a fundamental universe (framework version)
        self.add_universe_selection(FundamentalUniverseSelectionModel(self.fundamental_function))
```

For more information about universe settings, see Settings .

Accessing Data

If you add a fundamental universe to your algorithm, you can access fundamental data in the universe selection function.

```
def fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
    return [f.symbol for f in fundamental if f.price > 10 and f.valuation_ratios.pe_ratio > 30]
```

To get fundamental data for Equities in your algorithm, use the `fundamentals` property of the `Equity` objects. The fundamental data represent the corporate fundamentals for the current algorithm time.

```
fundamentals = self.securities[symbol].fundamentals
```

To get fundamental data for Equities, regardless of whether or not you have subscribed to them in your algorithm, call the `fundamentals` method. If

you pass one `Symbol`, the method returns a `Fundamental` object. If you pass a list of `Symbol` objects, the method returns a list of `Fundamental` objects.

```
# Single asset
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)
ibm_fundamental = self.fundamentals(ibm)

# Multiple assets
nb = Symbol.create("NB", SecurityType.EQUITY, Market.USA)
fundamentals = self.fundamentals([ nb, ibm ])
```

Historical Data

You can get historical fundamental data in an algorithm and in the Research Environment.

Historical Data in Algorithms

To get historical fundamental data in an algorithm, call the `history` method with `Fundamental` type and the Equity `Symbol` or the `Universe` object. If there is no data in the period you request, the history result is empty.

```
ibm = Symbol.create("IBM", SecurityType.EQUITY, Market.USA)

# DataFrame object where the columns are the Fundamental attributes
asset_df_history = self.history(Fundamental, ibm, timedelta(30), flatten=True)

# Fundamental objects
fundamental_history= self.history[Fundamental](ibm, timedelta(30))

# Fundamentals objects for all US Equities (including delisted companies)
fundamentals_history = self.history[Fundamentals](timedelta(30))

# Multi-index Series objects where the values are lists of Fundamental objects
series_history = self.history(self.universe, 30, Resolution.DAILY)
for (universe_symbol, time), fundamentals in series_history.items():
    highest_market_cap = sorted(fundamentals, key=lambda x: x.market_cap)[-5:]
```

Historical Data in the Research Environment

To get historical data in the Research Environment, call any of the preceding methods or call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. Alternatively, call the `get_fundamental` method with the Equity `Symbol`, a `Fundamental` property, a start date, and an end date. If there is no data in the period you request, the history result is empty.

```
# DataFrame object where the columns are the Fundamental attributes
universe_history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Multi-index Series objects where the values are lists of Fundamental objects
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), fundamentals in universe_history.items():
    for fundamental in fundamentals:
        print(f"{fundamental.symbol} market cap at {fundamental.end_time}: {fundamental.market_cap}")

# DataFrame of a single Fundamental attribute
history = qb.get_fundamental(symbol, "ValuationRatios.pe_ratios", datetime(2021, 1, 1), datetime(2021, 7, 1))
```

For more information about historical US Equity fundamental data, see Equity Fundamental Data .

Example Applications

The US Fundamentals dataset enables you to design strategies harnessing fundamental data points. Examples include the following strategies:

- Ranking a universe of securities by a value factor like the Piotroski F-score and buying a subset of the universe with the best factor ranking
- Using the Morningstar asset classification to target specific industries or to ensure your strategy is diversified across several sectors
- Trading securities that recently performed an IPO

Classic Algorithm Example

The following example algorithm creates a dynamic universe of US Equities. The first stage filter selects the 100 most liquid US Equities and the second stage filter selects the 10 assets with the greatest PE ratio. The algorithm then forms an equal-weighted portfolio with the 10 assets in the universe.

```py
from AlgorithmImports import *

class MorningStarDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_size = 10
        # Add universe selection to filter with fundamental data
        self.add_universe(self.fundamental_selection_function)

    def fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Make sure fundamental data and PE ratio available
        # To avoid penny stocks with high volatility, set price above $1
        selected = [f for f in fundamental if f.has_fundamental_data and f.price > 1 and not np.isnan(f.valuation_ratios.pe_ratio)]
        # Filter the top 100 high dollar volume equities to ensure liquidity
        sorted_by_dollar_volume = sorted(selected, key=lambda f: f.dollar_volume, reverse=True)[:100]
        # Get the top 10 PE Ratio stocks to follow speculations of a large number of capital
        sorted_by_pe_ratio = sorted(sorted_by_dollar_volume, key=lambda f: f.valuation_ratios.pe_ratio, reverse=True)
[:self.universe_size]
        return [ f.symbol for f in sorted_by_pe_ratio ]


    def on_data(self, slice: Slice) -> None:
        # if we have no changes, do nothing
        if self._changes is None or not slice.bars: return

        # liquidate removed securities, as not being the most popular stocks anymore
        for security in self._changes.removed_securities:
            if security.invested:
                self.liquidate(security.symbol)

        # we want 1/N allocation in each security in our universe to evenly dissipate risk
        for security in self._changes.added_securities:
            self.set_holdings(security.symbol, 1 / self.universe_size)

        self._changes = None

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._changes = changes
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities. The first stage filter selects the 100 most liquid US Equities and the second stage filter selects the 10 assets with the greatest PE ratio. The algorithm then forms an equal-weighted portfolio with the 10 assets in the universe.

```py
from AlgorithmImports import *

class MorningStarDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Add universe selection to filter with fundamental data
        self.universe_settings.resolution = Resolution.DAILY
        self.set_universe_selection(CustomFundamentalUniverseSelectionModel())

        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))

        # we want 1/N allocation in each security in our universe to evenly dissipate risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())


class CustomFundamentalUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamental: List[Fundamental]) -> List[Symbol]:
        # Make sure fundamental data and PE ratio available
        # To avoid penny stocks with high volatility, set price above $1
        # Filter the top 100 high dollar volume equities to ensure liquidity
        sorted_by_dollar_volume = sorted([x for x in fundamental if x.price > 1 and not np.isnan(x.valuation_ratios.pe_ratio)],
            key=lambda x: x.dollar_volume, reverse=True)[:100]
        # Get the top 10 PE Ratio stocks to follow speculations of a large number of capital
        sorted_by_pe_ratio = sorted(sorted_by_dollar_volume, key=lambda x: x.valuation_ratios.pe_ratio, reverse=True)[:10]

        # Take the top entries from our sorted collection
        return [ x.symbol for x in sorted_by_pe_ratio ]
```

Research Example

The following example selects the 100 most liquid US Equities and lists the 10 assets with the greatest PE ratio.:

```py
qb = QuantBook()

# Add Fundamental Universe Selection
def fundamental_selection_function(fundamentals):
    selected = [f for f in fundamentals if not np.isnan(f.valuation_ratios.pe_ratio)]
    sorted_by_dollar_volume = sorted(selected, key=lambda f: f.dollar_volume, reverse=True)[:100]

    sorted_by_pe_ratio = sorted(sorted_by_dollar_volume,
        key=lambda f: f.valuation_ratios.pe_ratio, reverse=True)[:10]
    return [ f.symbol for f in sorted_by_pe_ratio ]

universe = qb.add_universe(fundamental_selection_function)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (universe_symbol, time), fundamentals in universe_history.items():
    for fundamental in fundamentals:
        print(f"{fundamental.symbol} PE ratio volume at {fundamental.end_time}: {fundamental.valuation_ratios.pe_ratio}")
```

Data Point Attributes

The US Fundamentals dataset provides `Fundamental` objects. To filter `Fundamental` objects, you can use one of the enumeration helpers.

Fundamental Attributes

`Fundamental` objects have the following attributes:

MorningstarSectorCode Enumeration

Sectors are large super categories of data. To access the sector of an Equity, use the `MorningstarSectorCode` property.

```py
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.morningstar_sector_code == MorningstarSectorCode.TECHNOLOGY]
```

The `MorningstarSectorCode` enumeration has the following members:

MorningstarIndustryGroupCode Enumeration

Industry groups are clusters of related industries that tie together. To access the industry group of an Equity, use the `MorningstarIndustryGroupCode` property.

```
PY
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.morningstar_industry_group_code == MorningstarIndustryGroupCode.APPLICATION_SOFTWARE]
```

The `MorningstarIndustryGroupCode` enumeration has the following members:

MorningstarIndustryCode Enumeration

Industries are the finest classification level available and are the individual industries according to the Morningstar classification system. To access the industry group of an Equity, use the `MorningstarIndustryCode` property.

```
PY
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.morningstar_industry_code == MorningstarIndustryCode.SOFTWARE_APPLICATION]
```

The `MorningstarIndustryCode` enumeration has the following members:

StockType Enumeration

The Stock Type classification groups companies according to the underlying fundamentals of their business. It answers the question: "If I buy this stock, what kind of company am I buying?" The emphasis is on income statement, balance sheet, and cash-flow data—not price data or valuation multiples. To access the stock type of an Equity, use the `StockType` property.

```
PY
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.stock_type == StockType.AGGRESSIVE_GROWTH]
```

The `StockType` enumeration has the following members:

StyleBox Enumeration

The Morningstar Equity Style Box is a grid that provides a graphical representation of the investment style of stocks and portfolios. It classifies securities according to market capitalization (vertical axis) and value-growth scores (horizontal axis). For stocks, the central column represents the core style. To access the style box of an Equity, use the `StyleBox` property.

```
PY
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.style_box == StyleBox.LARGE_VALUE]
```

The `StyleBox` enumeration has the following members:

MorningstarEconomySphereCode Enumeration

Sectors are consolidated into three major economic spheres or Super Sectors: Cyclical, Defensive, and Sensitive. To access the economy sphere (Super Sector) of an Equity, use the `MorningstarEconomySphereCode` property.

```
PY
filtered_fundamentals = [x for x in fundamental
    if x.asset_classification.morningstar_economy_sphere_code == MorningstarEconomySphereCode.CYCLICAL]
```

The `MorningstarEconomySphereCode` enumeration has the following members:

Exchange ID Values

The exchange ID represents the exchange that lists the Equity. To access the exchange ID of an Equity, use the `PrimaryExchangeID` property.

```py
filtered_fundamentals = [x for x in fundamental
    if x.company_reference.primary_exchange_id == "NAS"]
```

The exchanges are represented by the following string values:

String Representation	Exchange
NYS	New York Stock Exchange (NYSE)
NAS	NASDAQ
ASE	American Stock Exchange (AMEX)
TSE	Tokyo Stock Exchange
AMS	Amsterdam Internet Exchange
SGO	Santiago Stock Exchange
XMAD	Madrid Stock Exchange
ASX	Australian Securities Exchange
BVMF	B3 (stock exchange)
LON	London Stock Exchange
TKS	Istanbul Stock Exchange Settlement and Custody Bank
SHG	Shanghai Exchange
LIM	Lima Stock Exchange
FRA	Frankfurt Stock Exchange
JSE	Johannesburg Stock Exchange
MIL	Milan Stock Exchange
TAE	Tel Aviv Stock Exchange
STO	Stockholm Stock Exchange
ETR	Deutsche Boerse Xetra Core
PAR	Paris Stock Exchange
BUE	Buenos Aires Stock Exchange
KRX	Korea Exchange
SWX	SIX Swiss Exchange
PINX	Pink Sheets (OTC)
CSE	Canadian Securities Exchange
PHS	Philippine Stock Exchange
MEX	Mexican Stock Exchange
TAI	Taiwan Stock Exchange

IDX	Indonesia Stock Exchange
OSL	Oslo Stock Exchange
BOG	Colombia Stock Exchange
NSE	National Stock Exchange of India
HEL	Nasdaq Helsinki
MISX	Moscow Exchange
HKG	Hong Kong Stock Exchange
IST	Istanbul Stock Exchange
BOM	Bombay Stock Exchange
TSX	Toronto Stock Exchange
BRU	Brussels Stock Exchange
BATS	BATS Global Markets
ARCX	NYSE Arca
GREY	Grey Market (OTC)
DUS	Dusseldorf Stock Exchange
BER	Berlin Stock Exchange
ROCO	Taipei Exchange
CNQ	Canadian Trading and Quotation System Inc.
BSP	Bangko Sentral ng Pilipinas
NEOE	NEO Exchange

Datasets

TickData

TickData was founded by a futures broker and a programmer in 1984 as the first company in the world to offer historical tick-by-tick prices on the futures and index markets. TickData provides access to comprehensive and detailed market data and analytics products in the financial industry covering Equities, Futures, Options, cash FOREX, and Cash Indices.

International Futures

TickData

International Futures

Introduction

The International Futures dataset by TickData provides Futures data, including price, volume, open interest, and expiry. The data covers the 4 contracts, DAX, FESX, HSI, and NKD, starting in July 1998 and is delivered on any frequency from tick to daily. This dataset is created by TickData, which negotiates directly with exchanges for their official archive and partners with real-time data providers with direct exchange connections and multiple redundant ticker plants.

This dataset also depends on the US Futures Security Master because the US Futures Security Master dataset contains information to construct continuous Futures.

For more information about the International Futures dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

TickData was founded by a futures broker and a programmer in 1984 as the first company in the world to offer historical tick-by-tick prices on the futures and index markets. TickData provides access to comprehensive and detailed market data and analytics products in the financial industry covering Equities, Futures, Options, cash FOREX, and Cash Indices.

Getting Started

The following snippet demonstrates how to request data from the International Futures dataset:

```py
hsi = self.add_future(Futures.Indices.HANG_SENG, Resolution.MINUTE).symbol          # "HSI"
dax = self.add_future(Futures.Indices.DAX, Resolution.MINUTE).symbol                # "DAX"
fesx = self.add_future(Futures.Indices.EURO_STOXX_50, Resolution.MINUTE).symbol     # "FESX"
nkd = self.add_future(Futures.Indices.NIKKEI_225_DOLLAR, Resolution.MINUTE).symbol  # "NKD"
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	July 1998, see `Supported Assets` below for details
Coverage	4 Contracts
Data Density	Dense
Resolution	Minute, Hour, & Daily
Timezone	Various, refer to `Supported Assets` below
Market Hours	Regular and Extended

Volume Discrepancies

For Futures contracts trading in the US, for example, NKD, see US Futures .

Requesting Data

To add International Futures data to your algorithm, call the `add_future` method. Save a reference to the Future so you can access the data later in your algorithm.

```
class InternationalFuturesDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2013, 12, 20)
        self.set_end_date(2014, 2, 20)
        self.set_cash(1000000)
        self.universe_settings.asynchronous = True
        # Request Hang Seng Index Futures data
        future = self.add_future(Futures.Indices.HANG_SENG)
        # Set filter to obtain the contracts expire within 90 days
        future.set_filter(0, 90)
        self.future_symbol = future.symbol
```

For more information about creating Future subscriptions, see Requesting Data or Futures Universes .

Accessing Data

To get the current International Futures data, index the `futures_chains` property of the current `Slice` with the canonical Futures `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your Future at every time step.

```
def on_data(self, slice: Slice) -> None:
    # Get future chain of the canonical symbol
    chain = slice.futures_chains.get(self.future_symbol)
    if chain:
        # Iterate each contract in the future chain
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

You can also iterate through all of the `FuturesChain` objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    # Iterate all future chains
    for canonical_symbol, chain in slice.futures_chains.items():
        # Iterate each contract in the future chain
        for contract in chain:
            self.log(f"{contract.symbol} price at {slice.time}: {contract.last_price}")
```

For more information about accessing Futures data, see Handling Data .

Historical Data

You can get historical International Futures data in an algorithm and the Research Environment.

Historical Data In Algorithms

To get historical International Futures data in an algorithm, call the `history` method with the canonical Futures `Symbol` or a Futures contract `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame objects
contract_history_df = self.history(contract.symbol, 100, Resolution.MINUTE)
continuous_history_df = self.history(self.future_symbol,
    start=self.time - timedelta(days=15),
    end=self.time,
    resolution=Resolution.MINUTE,
    fill_forward=False,
    extended_market_hours=False,
    data_mapping_mode=DataMappingMode.OPEN_INTEREST,
    data_normalization_mode=DataNormalizationMode.RAW,
    contract_depth_offset=0)

# TradeBar objects
contract_history_trade_bars = self.history[TradeBar](contract.symbol, 100, Resolution.MINUTE)
continous_history_trade_bars = self.history[TradeBar](self.future_symbol, 100, Resolution.MINUTE)

# QuoteBar objects
contract_history_quote_bars = self.history[QuoteBar](contract.symbol, 100, Resolution.MINUTE)
continous_history_quote_bars = self.history[QuoteBar](self.future_symbol, 100, Resolution.MINUTE)

# Tick objects
contract_history_ticks = self.history[Tick](contract.symbol, timedelta(seconds=10), Resolution.TICK)
continous_history_ticks = self.history[Tick](self.future_symbol, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data in algorithms, see History Requests . For more information about the price adjustments for continuous contracts, see Continous Contracts .

Historical Data In Research

To get historical International Futures data in the Research Environment for an entire Futures chain, call the `future_history` method with the canonical Future `Symbol` .

```py
qb = QuantBook()
future = qb.add_future(Futures.Indices.HANG_SENG)
future.set_filter(0, 90)
history = qb.future_history(future.symbol, datetime(2020, 6, 1), datetime(2020, 6, 5))
history_df = history.data_frame
all_history = history.get_all_data()
expiries = history.get_expiry_dates()
```

To get historical data for a single International Futures contract or the continuous Futures contract, call the `history` method like you would in an algorithm but on the `QuantBook` object. For more information about historical data in the Research Environment, see Futures .

Supported Assets

The following table shows the available Futures:

Ticker	Future	Start Date	Time Zone	Currency
HSI	Hang Seng Index Futures	Jan 2010	Asia/Hong Kong	HKD
DAX	German Index Futures	Jan 2010	Europe/Berlin	EUR
FESX	EURO STOXX 50 Index Futures	Jul 1998	Europe/Berlin	EUR
NKD	Nikkei 225 Index Futures	Jan 2007	America/Chicago	USD

Example Applications

The International Futures dataset enables you to design Futures strategies accurately. Examples include the following strategies:

- Buying the Futures contract with the most open interest to reduce slippage and market impact
- Trade speculation on an International Index
- Trading bull calendar spreads to reduce volatility and margin requirements

Classic Algorithm Example -- HSI Futures

The following example algorithm uses ZigZag to determine the trend of Hang Seng Index. To trade the Indices, we make use of the HSI Futures.

```py
from AlgorithmImports import *


class InternationalFuturesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the time zone to HKT to make it more comparable with the exchange.
        self.set_time_zone(TimeZones.HONG_KONG)
        # Set the account currency as HKD to trade HSI Futures.
        self.set_account_currency("HKD", 1000000)
        # Seed the last price of the contracts for filling.
        self.settings.seed_initial_prices = True

        # Request HSI Futures to trade.
        # Note that we will trade the contract with the highest open interest for liquidity.
        self.hsi_future = self.add_future(
            Futures.Indices.HANG_SENG,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.LAST_TRADING_DAY,
            contract_depth_offset=0
        )
        # Request the corresponding underlying Index for feeding indicator for trade signal generation.
        hsi_index = self.add_index("HSI").symbol

        # Create a ZigZag indicator to trade Hang Seng Index price pivot points.
        self._zz = self.zz(hsi_index, 0.15, 5, Resolution.DAILY)
        # Warm up indicator for immediate readiness to trade.
        self.warm_up_indicator(hsi_index, self._zz, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        # Only place trade if the Future contracts is in market opening hours to avoid stale fills.
        if self.is_market_open(self.hsi_future.symbol) and self._zz.is_ready:
            pivot = self._zz.pivot_type
            # If the last pivot point is a low point, the current trend is increasing after this low point.
            if pivot == PivotPointType.LOW and not self.portfolio[self.hsi_future.symbol].is_long:
                self.set_holdings(self.hsi_future.mapped, 0.2)
            # If the last pivot point is a high point, the current trend is decreasing after this high point.
            elif pivot == PivotPointType.HIGH and not self.portfolio[self.hsi_future.symbol].is_short:
                self.set_holdings(self.hsi_future.mapped, -0.2)

        # Handle rollover in case the current mapped contract changes.
        for _, changed_event in slice.symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = self.add_future_contract(changed_event.new_symbol).symbol
            quantity = self.portfolio[old_symbol].quantity
            # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
            self.liquidate(old_symbol)
            if quantity != 0:
                self.market_order(new_symbol, quantity)
```

Classic Algorithm Example -- FESX Futures

The following example algorithm uses the MACD indicator to determine the trend of the EuroStoxx50 Index. To trade the Index, we make use of the FESX Futures.

```py
from AlgorithmImports import *


class InternationalFuturesDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the time zone to Berlin to make it more comparable with the exchange.
        self.set_time_zone(TimeZones.BERLIN)
        # Set the account currency as EUR to trade FSX Futures.
        self.set_account_currency("EUR", 1000000)
        # Seed the last price of the contracts for filling.
        self.settings.seed_initial_prices = True

        # Request FESX Futures to trade.
        # Note that we will trade the contract with the highest open interest for liquidity.
        self.fesx_future = self.add_future(
            Futures.Indices.EURO_STOXX_50,
            extended_market_hours=True,
            data_mapping_mode=DataMappingMode.LAST_TRADING_DAY,
            contract_depth_offset=0
        )
        # Request the corresponding underlying Index for feeding indicator for trade signal generation.
        fesx_index = self.add_index("SX5E").symbol

        # Create a MACD indicator to trade EuroStoxx trend changes on the fast-slow term trend convergence.
        self._macd = self.macd(fesx_index, 12, 26, 9, resolution=Resolution.DAILY)
        # Warm up indicator for immediate readiness to trade.
        self.warm_up_indicator(fesx_index, self._macd, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        # Only place trade if the Future contracts is in market opening hours to avoid stale fills.
        if self.is_market_open(self.fesx_future.symbol) and self._macd.is_ready:
            # To identify a sensitive change in trend, use the change in the histogram value of the MACD.
            histogram = self._macd.histogram
            histogram_chg = histogram.current.value - histogram.previous.value
            # Positive histogram value increasing means the fast trend is rising faster than the slow trend, indicating a strong uptrend.
            if histogram_chg > 0 and histogram.current.value > 0 and not self.portfolio[self.fesx_future.symbol].is_long:
                self.set_holdings(self.fesx_future.mapped, 0.2)
            # Negative histogram value decreasing means the slow trend is dropping faster than the fast trend, indicating a strong
downtrend.
            elif histogram_chg < 0 and histogram.current.value < 0 and not self.portfolio[self.fesx_future.symbol].is_short:
                self.set_holdings(self.fesx_future.mapped, -0.2)

        # Handle rollover in case the current mapped contract changes.
        for _, changed_event in slice.symbol_changed_events.items():
            old_symbol = changed_event.old_symbol
            new_symbol = self.add_future_contract(changed_event.new_symbol).symbol
            quantity = self.portfolio[old_symbol].quantity
            # Rolling over: to liquidate any position of the old mapped contract and switch to the newly mapped contract
            self.liquidate(old_symbol)
            if quantity != 0:
                self.market_order(new_symbol, quantity)
```

Data Point Attributes

The International Futures dataset provides `Future` , `FuturesChain` , and `OpenInterest` objects. To configure the continuous Future settings, use the `DataNormalizationMode` and `DataMappingMode` enumerations.

Future Attributes

`Future` objects have the following attributes:

FuturesChain Attributes

`FuturesChain` objects have the following attributes:

OpenInterest Attributes

`OpenInterest` objects have the following attributes:

DataNormalizationMode Values

The `DataNormalizationMode` enumeration has the following values:

DataMappingMode Values

The `DataMappingMode` enumeration has the following values:

Datasets

CoinAPI

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Binance Crypto Future Price Data

Binance Crypto Price Data

Binance US Crypto Price Data

Bitfinex Crypto Price Data

Bybit Crypto Future Price Data

Bybit Crypto Price Data

Coinbase Crypto Price Data

Kraken Crypto Price Data

CoinAPI

Binance Crypto Future Price Data

Introduction

The Binance Crypto Future Price Data by CoinAPI is for Crypto-currency futures price and volume data points. The data covers 718 Cryptocurrency pairs, starts in August 2020, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Binance.

The Binance Crypto Future Margin Rate Data dataset provides margin interest data to model margin costs.

For more information about the Binance Crypto Future Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Binance Crypto Future Price dataset:

```
def initialize(self) -> None:
    self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
    #self.set_brokerage_model(BrokerageName.BINANCE_COIN_FUTURES, AccountType.MARGIN)

    self.crypto_future_symbol = self.add_crypto_future("BTCBUSD", Resolution.MINUTE).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	August 2020
Asset Coverage	718 Crypto Futures Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Binance Crypto Future Price data to your algorithm, call the `add_crypto_future` method. Save a reference to the Crypto Future `Symbol` so you can access the data later in your algorithm.

```
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Binance Stable Coin for USD
        self.set_account_currency("BUSD")
        self.set_cash(100000)

        self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
        self.set_brokerage_model(BrokerageName.BINANCE_COIN_FUTURES, AccountType.MARGIN)

        crypto_future = self.add_crypto_future("BTCBUSD", Resolution.MINUTE)
        # perpetual futures does not have a filter function
        self.btcbusd = crypto_future.symbol
```

For more information about creating Crypto Future subscriptions, see Requesting Data .

Accessing Data

To get the current Binance Crypto Future Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto Future `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if self.btcbusd in slice.bars:
        trade_bar = slice.bars[self.btcbusd]
        self.log(f"{self.btcbusd} close at {slice.time}: {trade_bar.close}")

    if self.btcbusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcbusd]
        self.log(f"{self.btcbusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcbusd in slice.ticks:
        ticks = slice.ticks[self.btcbusd]
        for tick in ticks:
            self.log(f"{self.btcbusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto Future data, see Handling Data .

Historical Data

To get historical Binance Crypto Future Price data, call the `history` method with the Crypto Future `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.btcbusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcbusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcbusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcbusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To unsubscribe from a Crypto Future contract that you added with the `add_crypto_future` method, call the `remove_security` method.

```
self.remove_security(self.btcbusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your Crypto Future holdings.

Supported Assets

The following table shows the available Crypto Future pairs:

Pairs Available (718)					
0GUSDT	1000000BOBUSDT	1000000MOGUSDT	1000BONKUSDC	1000BONKUSDT	1000CATUSDT
1000CHEEMSUSDT	1000FLOKIUSDT	1000LUNCBUSD	1000LUNCUSDT	1000PEPEUSDC	1000PEPEUSDT
1000RATSUSDT	1000SATSUSDT	1000SHIBBUSD	1000SHIBUSDC	1000SHIBUSDT	1000WHYUSDT
1000XECUSDT	1000XUSDT	1INCHUSDT	1MBABYDOGEUSDT	2ZUSDT	4USDT
A2ZUSDT	AAVEUSD	AAVEUSDC	AAVEUSDT	ACEUSDT	ACHUSDT
ACTUSDT	ACXUSDT	ADABUSD	ADAUSD	ADAUSDC	ADAUSDT
AERGOUSDT	AEROUSDT	AEVOUSDT	AGIXBUSD	AGIXUSDT	AGLDUSDT
AGTUSDT	AI16ZUSDT	AIAUSDT	AINUSDT	AIOTUSDT	AIOUSDT
AIUSDT	AIXBTUSDT	AKEUSDT	AKTUSDT	ALCHUSDT	ALGOUSD
ALGOUSDT	ALICEUSDT	ALLUSDT	ALPACAUSDT	ALPHAUSDT	ALPINEUSDT
ALTUSDT	AMBBUSD	AMBUSDT	ANCBUSD	ANIMEUSDT	ANKRUSDT
ANTUSDT	APEBUSD	APEUSD	APEUSDT	API3USDT	APTBUSD
APTUSD	APTUSDT	ARBUSDC	ARBUSDT	ARCUSDT	ARIAUSDT
ARKMUSDT	ARKUSDT	ARPAUSDT	ARUSDT	ASRUSDT	ASTERUSDT
ASTRUSDT	ATAUSDT	ATHUSDT	ATOMUSD	ATOMUSDT	AUCTIONBUSD
AUCTIONUSDT	AUDIOUSDT	AUSDT	AVAAIUSDT	AVAUSDT	AVAXBUSD
AVAXUSD	AVAXUSDC	AVAXUSDT	AVNTUSDT	AWEUSDT	AXLUSDT
AXSUSD	AXSUSDT	B2USDT	B3USDT	BABYUSDT	BADGERUSDT
BAKEUSDT	BALUSDT	BANANAS31USDT	BANANAUSDT	BANDUSDT	BANKUSDT
BANUSDT	BARDUSDT	BASUSDT	BATUSDT	BBUSDT	BCHUSD
BCHUSDC	BCHUSDT	BDXNUSDT	BEAMXUSDT	BELUSDT	BERAUSDT
BICOUSDT	BIDUSDT	BIGTIMEUSDT	BIOUSDC	BIOUSDT	BLESSUSDT
BLUAIUSDT	BLUEBIRDUSDT	BLURUSDT	BLZUSDT	BMTUSDT	BNBBUSD
BNBUSD	BNBUSDC	BNBUSDT	BNTUSDT	BNXUSDT	BOMEUSDC
BOMEUSDT	BONDUSDT	BRETTUSDT	BROCCOLI714USDT	BROCCOLIF3BUSDT	BRUSDT

BSVUSDT	BSWUSDT	BTCBUSD	BTCDOMUSDT	BTCSTUSDT	BTCUSD
BTCUSDC	BTCUSDT	BTRUSDT	BTSUSDT	BULLAUSDT	BUSDT
C98USDT	CAKEUSDT	CARVUSDT	CATIUSDT	CELOUSDT	CELRUSDT
CETUSUSDT	CFXUSDT	CGPTUSDT	CHESSUSDT	CHILLGUYUSDT	CHRUSDT
CHZUSD	CHZUSDT	CKBUSDT	CLOUSDT	COAIUSDT	COCOSUSDT
COMBOUSDT	COMPUSDT	COOKIEUSDT	COSUSDT	COTIUSDT	COWUSDT
CROSSUSDT	CRVUSDC	CRVUSDT	CTKUSDT	CTSIUSDT	CUDISUSDT
CUSDT	CVCUSDT	CVXBUSD	CVXUSDT	CYBERUSDT	DAMUSDT
DARUSDT	DASHUSDT	DEEPUSDT	DEFIUSDT	DEGENUSDT	DEGOUSDT
DENTUSDT	DEXEUSDT	DFUSDT	DGBUSDT	DIAUSDT	DMCUSDT
DODOBUSD	DODOXUSDT	DOGEBUSD	DOGEUSD	DOGEUSDC	DOGEUSDT
DOGSUSD	DOGSUSDT	DOLOUSDT	DOODUSDT	DOTBUSD	DOTUSD
DOTUSDT	DRIFTUSDT	DUSDT	DUSKUSDT	DYDXUSDT	DYMUSDT
EDENUSDT	EDUUSDT	EGLDUSD	EGLDUSDT	EIGENUSDT	ENAUSDC
ENAUSDT	ENJUSDT	ENSOUSDT	ENSUSD	ENSUSDT	EOSUSD
EOSUSDT	EPICUSDT	EPTUSDT	ERAUSDT	ESPORTSUSDT	ETCBUSD
ETCUSD	ETCUSDT	ETHBTC	ETHBUSD	ETHFIUSDC	ETHFIUSDT
ETHUSD	ETHUSDC	ETHUSDT	ETHWUSDT	EULUSDT	EVAAUSDT
FARTCOINUSDT	FETUSDT	FFUSDT	FHEUSDT	FIDAUSDT	FILBUSD
FILUSD	FILUSDC	FILUSDT	FIOUSDT	FISUSDT	FLMUSDT
FLOCKUSDT	FLOWUSDT	FLUIDUSDT	FLUXUSDT	FOOTBALLUSDT	FORMUSDT
FORTHUSDT	FRONTUSDT	FTMBUSD	FTMUSD	FTMUSDT	FTTBUSD
FTTUSDT	FUNUSDT	FUSDT	FXSUSDT	GALABUSD	GALAUSD
GALAUSDT	GALBUSD	GALUSDT	GASUSDT	GHSTUSDT	GIGGLEUSDT
GLMRUSDT	GLMUSDT	GMTBUSD	GMTUSD	GMTUSDT	GMXUSDT
GOATUSDT	GPSUSDT	GRASSUSDT	GRIFFAINUSDT	GRTUSDT	GTCUSDT
GUNUSDT	GUSDT	HAEDALUSDT	HANAUSDT	HBARUSDC	HBARUSDT
HEIUSDT	HEMIUSDT	HFTUSDT	HIFIUSDT	HIGHUSDT	HIPPOUSDT
HIVEUSDT	HMSTRUSDT	HNTUSDT	HOLOUSDT	HOMEUSDT	HOOKUSDT
HOTUSDT	HUMAUSDT	HUSDT	HYPERUSDT	HYPEUSDT	ICNTUSDT
ICPBUSD	ICPUSDT	ICXUSD	ICXUSDT	IDEXUSDT	IDOLUSDT
IDUSDT	ILVUSDT	IMXUSDT	INITUSDT	INJUSDT	INUSDT
IOSTUSDT	IOTAUSDT	IOTXUSDT	IOUSDT	IPUSDC	IPUSDT

JASMYUSDT	JELLYJELLYUSDT	JOEUSDT	JSTUSDT	JTOUSDT	JUPUSDT
KAIAUSDT	KAITOUSDC	KAITOUSDT	KASUSDT	KAVAUSDT	KDAUSDT
KERNELUSDT	KEYUSDT	KGENUSDT	KLAYUSDT	KMNOUSDT	KNCUSD
KNCUSDT	KOMAUSDT	KSMUSDT	LABUSDT	LAUSDT	LAYERUSDT
LDOBUSD	LDOUSDT	LEVERBUSD	LEVERUSDT	LIGHTUSDT	LINAUSDT
LINEAUSDT	LINKBUSD	LINKUSD	LINKUSDC	LINKUSDT	LISTAUSDT
LITUSDT	LOKAUSDT	LOOMUSDT	LPTUSDT	LQTYUSDT	LRCUSDT
LSKUSDT	LTCBUSD	LTCUSD	LTCUSDC	LTCUSDT	LUMIAUSDT
LUNA2BUSD	LUNA2USDT	LYNUSDT	MAGICUSDT	MANAUSD	MANAUSDT
MANTAUSDT	MASKUSDT	MATICBUSD	MATICUSD	MATICUSDC	MATICUSDT
MAVIAUSDT	MAVUSDT	MBLUSDT	MBOXUSDT	MDTUSDT	MELANIAUSDT
MEMEFIUSDT	MEMEUSDT	MERLUSDT	METISUSDT	METUSDT	MEUSDT
MEWUSDT	MILKUSDT	MINAUSDT	MIRAUSDT	MITOUSDT	MKRUSDT
MLNUSDT	MOCAUSDT	MONUSDT	MOODENGUSDT	MORPHOUSDT	MOVEUSDT
MOVRUSDT	MTLUSDT	MUBARAKUSDT	MUSDT	MYROUSDT	MYXUSDT
NAORISUSDT	NEARBUSD	NEARUSD	NEARUSDC	NEARUSDT	NEIROETHUSDT
NEIROUSDT	NEOUSDC	NEOUSDT	NEWTUSDT	NFPUSDT	NILUSDT
NKNUSDT	NMRUSDT	NOMUSDT	NOTUSDT	NTRNUSDT	NULSUSDT
NXPCUSDT	OBOLUSDT	OCEANUSDT	OGNUSDT	OGUSDT	OLUSDT
OMGUSDT	OMNIUSDT	OMUSDT	ONDOUSDT	ONEUSDT	ONGUSDT
ONTUSDT	OPENUSDT	OPUSD	OPUSDT	ORBSUSDT	ORCAUSDT
ORDERUSDT	ORDIUSDC	ORDIUSDT	OXTUSDT	PARTIUSDT	PAXGUSDT
PENDLEUSDT	PENGUUSDC	PENGUUSDT	PEOPLEUSDT	PERPUSDT	PHAUSDT
PHBBUSD	PHBUSDT	PIPPINUSDT	PIXELUSDT	PLAYUSDT	PLUMEUSDT
PNUTUSDC	PNUTUSDT	POLUSDT	POLYXUSDT	PONKEUSDT	POPCATUSDT
PORT3USDT	PORTALUSDT	POWRUSDT	PROMPTUSDT	PROMUSDT	PROVEUSDT
PTBUSDT	PUFFERUSDT	PUMPBTCUSDT	PUMPUSDT	PUNDIXUSDT	PYTHUSDT
QNTUSDT	QTUMUSDT	QUICKUSDT	QUSDT	RADUSDT	RAREUSDT
RAYSOLUSDT	RAYUSDT	RDNTUSDT	RECALLUSDT	REDUSDT	REEFUSDT
REIUSDT	RENDERUSDT	RENUSDT	RESOLVUSDT	REZUSDT	RIFUSDT
RIVERUSDT	RLCUSDT	RNDRUSDT	RONINUSDT	ROSEUSD	ROSEUSDT
RPLUSDT	RSRUSDT	RUNEUSD	RUNEUSDT	RVNUSDT	RVVUSDT
SAFEUSDT	SAGAUSDT	SAHARAUSDT	SANDBUSD	SANDUSD	SANDUSDT

SANTOSUSDT	SAPIENUSDT	SCRTUSDT	SCRUSDT	SCUSDT	SEIUSDT
SFPUSDT	SHELLUSDT	SIGNUSDT	SIRENUSDT	SKATEUSDT	SKLUSDT
SKYAIUSDT	SKYUSDT	SLERFUSDT	SLPUSDT	SNTUSDT	SNXUSDT
SOLBUSD	SOLUSD	SOLUSDC	SOLUSDT	SOLVUSDT	SOMIUSDT
SONICUSDT	SOONUSDT	SOPHUSDT	SPELLUSDT	SPKUSDT	SPXUSDT
SQDUSDT	SRMUSDT	SSVUSDT	STBLUSDT	STEEMUSDT	STGUSDT
STMXUSDT	STORJUSDT	STOUSDT	STPTUSDT	STRAXUSDT	STRKUSDT
STXUSDT	SUIUSD	SUIUSDC	SUIUSDT	SUNUSDT	SUPERUSDT
SUSDT	SUSHIUSDT	SWARMSUSDT	SWELLUSDT	SXPUSDT	SXTUSDT
SYNUSDT	SYRUPUSDT	SYSUSDT	TACUSDT	TAGUSDT	TAIKOUSDT
TAKEUSDT	TANSSIUSDT	TAOUSDT	TAUSDT	THETAUSD	THETAUSDT
THEUSDT	TIAUSDC	TIAUSDT	TLMBUSD	TLMUSDT	TNSRUSDT
TOKENUSDT	TOMOUSDT	TONUSDT	TOSHIUSDT	TOWNSUSDT	TRADOORUSDT
TRBUSDT	TREEUSDT	TROYUSDT	TRUMPUSDC	TRUMPUSDT	TRUTHUSDT
TRUUSDT	TRXBUSD	TRXUSD	TRXUSDT	TSTUSDT	TURBOUSDT
TUSDT	TUTUSDT	TWTUSDT	UBUSDT	UMAUSDT	UNFIUSDT
UNIBUSD	UNIUSD	UNIUSDC	UNIUSDT	USDCUSDT	USELESSUSDT
USTCUSDT	USUALUSDT	UXLINKUSDT	VANAUSDT	VANRYUSDT	VELODROMEUSDT
VELVETUSDT	VETUSD	VETUSDT	VFYUSDT	VICUSDT	VIDTUSDT
VINEUSDT	VIRTUALUSDT	VOXELUSDT	VTHOUSDT	VVVUSDT	WALUSDT
WAVESBUSD	WAVESUSDT	WAXPUSDT	WCTUSDT	WIFUSD	WIFUSDC
WIFUSDT	WLDUSD	WLDUSDC	WLDUSDT	WLFIUSDC	WLFIUSDT
WOOUSDT	WUSDT	XAIUSDT	XANUSDT	XCNUSDT	XEMUSDT
XLMUSD	XLMUSDT	XMRUSD	XMRUSDT	XNYUSDT	XPINUSDT
XPLUSDT	XRPBUSD	XRPUSD	XRPUSDC	XRPUSDT	XTZUSD
XTZUSDT	XVGUSDT	XVSUSDT	YALAUSDT	YBUSDT	YFIUSDT
YGGUSDT	ZBTUSDT	ZECUSDT	ZENUSDT	ZEREBROUSDT	ZETAUSDT
ZILUSD	ZILUSDT	ZKCUSDT	ZKJUSDT	ZKUSDT	ZORAUSDT
ZRCUSDT	ZROUSDT	ZRXUSDT	[illegible]USDT		

Example Applications

The Binance Crypto Future Price dataset enables you to accurately design strategies for Cryptocurrencies with term structure. Examples include the following strategies:

- Horizontal/Diagonal arbitrage with the underlying cryptocurrencies
- Trade Contango/Backwardation predictions

- Hedge for illiquid cryptocurrencies

Classic Algorithm Example

The following example algorithm buys BTCUSDT perpetual future contract if the last day's close price was close to ask close price than bid close price, sells short of that in opposite, through the Binance exchange:

```
from AlgorithmImports import * class BinanceCryptoFutureDataAlgorithm(QCAlgorithm): def initialize(self) -> None: self.
set_start_date(2024, 9, 1) self.set_end_date(2024, 12, 31) # Set the account currency to USDT, since you can't trade wi
th # USD and the algorithm doesn't automatically convert USD & USDT # holdings. self.set_account_currency("USDT", 10000
0) # Binance Futures Exchange accepts both Cash and Margin account # types. Select the one you need for the best realit
y modeling. self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN) # Add BTCUSDT data from the Bin
ance Future exchange. # Perpetual futures does not have a filter function. self._btc = self.add_crypto_future("BTCUSDT"
, Resolution.DAILY) # Historical data history = self.history(self._btc, 10, Resolution.DAILY) def on_data(self, slice:
Slice) -> None: # Trade only based on updated price data. if self._btc.symbol not in slice.bars or self._btc.symbol not
in slice.quote_bars: return # Scalp-trade the bid-ask spread based on the supply-demand strength if self._btc.price - s
elf._btc.bid_price > self._btc.ask_price - self._btc.price: self.set_holdings(self._btc, -0.5) else: self.set_holdings(
self._btc, 1)
```

Framework Algorithm Example

The following example algorithm hold a long BTCUSDT Future portfolio if the last day's close price was close to ask close price than bid close price, while hold short of that in opposite, through the Binance exchange using the algorithm framework implementation:

```
from AlgorithmImports import * class BinanceCryptoFutureDataAlgorithm(QCAlgorithm): def initialize(self) -> None: self.
set_start_date(2024, 9, 1) self.set_end_date(2024, 12, 31) # Set the account currency to USDT, since you can't trade wi
th # USD and the algorithm doesn't automatically convert USD & USDT # holdings. self.set_account_currency("USDT", 10000
0) # Binance Futures Exchange accepts both Cash and Margin account types, select the one you need for the best reality
modeling. self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN) self.universe_settings.resolution
= Resolution.DAILY self.universe_settings.leverage = 2 # We only trade on BTCUSDT Future in Binance Future exchange sym
bols = Symbol.create("BTCUSDT", SecurityType.CRYPTO_FUTURE, Market.BINANCE) self.add_universe_selection(ManualUniverseS
electionModel(symbols)) # Custom alpha model to emit insights based on the Crypto Future price data self.add_alpha(Cryp
toFutureAlphaModel()) # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair se
lf.set_portfolio_construction(EqualWeightingPortfolioConstructionModel()) self.set_execution(ImmediateExecutionModel())
class CryptoFutureAlphaModel(AlphaModel): def __init__(self) -> None: self.symbols = [] def update(self, algorithm: QCA
lgorithm, slice: Slice) -> List[Insight]: insights = [] for symbol in self.symbols: # Trade only based on updated price
data if not slice.bars.contains_key(symbol) or not slice.quote_bars.contains_key(symbol): continue quote = slice.quote_
bars[symbol] price = slice.bars[symbol].price # Scalp-trade the bid-ask spread based on the supply-demand strength if p
rice - quote.bid.close > quote.ask.close - price: direction = InsightDirection.DOWN else: direction = InsightDirection.
UP insights.append(Insight.price(symbol, timedelta(1), direction)) return insights def on_securities_changed(self, algo
rithm: QCAlgorithm, changes: SecurityChanges) -> None: for security in changes.added_securities: symbol = security.symb
ol self.symbols.append(symbol) # Historical data history = algorithm.history(symbol, 10, Resolution.DAILY) for security
in changes.removed_securities: symbol = security.symbol if symbol in self.symbols: self.symbols.remove(symbol)
```

Data Point Attributes

The Binance Crypto Future Price dataset provides `TradeBar` , `QuoteBar` , and `Tick` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CoinAPI

Binance Crypto Price Data

Introduction

The Binance Crypto Price Data by CoinAPI is for Cryptocurrency price and volume data points. The data covers 3,320 Cryptocurrency pairs, starts in July 2017, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Binance.

For more information about the Binance Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Binance Crypto Price dataset:

```py
# Binance accepts both Cash and Margin account types only.
self.set_brokerage_model(BrokerageName.BINANCE, AccountType.CASH)
#self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)

self.btcbusd = self.add_crypto("BTCBUSD", Resolution.MINUTE, Market.BINANCE).symbol

self._universe = self.add_universe(CryptoUniverse.binance(self.universe_selection_filter))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	July 2017
Asset Coverage	3,320 Currency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Binance Crypto Price data to your algorithm, call the `add_crypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Binance Stable Coin for USD
        self.set_account_currency("BUSD")
        self.set_cash(100000)

        # Binance accepts both Cash and Margin account types.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)

        self.btcbusd = self.add_crypto("BTCBUSD", Resolution.MINUTE, Market.BINANCE).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Binance Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```py
def on_data(self, slice: Slice) -> None:
    if self.btcbusd in slice.bars:
        trade_bar = slice.bars[self.btcbusd]
        self.log(f"{self.btcbusd} close at {slice.time}: {trade_bar.close}")

    if self.btcbusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcbusd]
        self.log(f"{self.btcbusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcbusd in slice.ticks:
        ticks = slice.ticks[self.btcbusd]
        for tick in ticks:
            self.log(f"{self.btcbusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```py
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Binance Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```py
# DataFrame
history_df = self.history(self.btcbusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcbusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcbusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcbusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Binance Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```py
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)
    self._universe = self.add_universe(CryptoUniverse.binance(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subcriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcbusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (3320)					
0GBNB	0GFDUSD	0GTRY	0GUSDC	0GUSDT	1000CATBNB

1000CATFDUSD	1000CATTRY	1000CATUSDC	1000CATUSDT	1000CHEEMSUSDC	1000CHEEMSUSDT
1000SATSFDUSD	1000SATSTRY	1000SATSUSDC	1000SATSUSDT	1INCHBTC	1INCHBUSD
1INCHUSDT	1MBABYDOGEFDUSD	1MBABYDOGETRY	1MBABYDOGEUSDC	1MBABYDOGEUSDT	2ZBNB
2ZFDUSD	2ZTRY	2ZUSDC	2ZUSDT	A2ZUSDC	A2ZUSDT
AAVEBKRW	AAVEBNB	AAVEBRL	AAVEBTC	AAVEBUSD	AAVEETH
AAVEFDUSD	AAVETRY	AAVEUSDC	AAVEUSDT	ABTC	ACABTC
ACABUSD	ACATRY	ACAUSDT	ACEBNB	ACEBTC	ACEFDUSD
ACETRY	ACEUSDT	ACHBTC	ACHBUSD	ACHTRY	ACHUSDC
ACHUSDT	ACMBTC	ACMBUSD	ACMTRY	ACMUSDT	ACTBRL
ACTEUR	ACTFDUSD	ACTTRY	ACTUSDC	ACTUSDT	ACXFDUSD
ACXTRY	ACXUSDC	ACXUSDT	ADAAUD	ADABIDR	ADABKRW
ADABNB	ADABRL	ADABTC	ADABUSD	ADAETH	ADAEUR
ADAFDUSD	ADAGBP	ADAJPY	ADAPAX	ADARUB	ADATRY
ADATUSD	ADAUSDC	ADAUSDT	ADXBNB	ADXBTC	ADXBUSD
ADXETH	ADXUSDT	AEBNB	AEBTC	AEETH	AERGOBTC
AERGOBUSD	AERGOUSDT	AETH	AEURUSDT	AEVOBNB	AEVOBTC
AEVOFDUSD	AEVOTRY	AEVOUSDC	AEVOUSDT	AFDUSD	AGIBNB
AGIBTC	AGIETH	AGIXBTC	AGIXBUSD	AGIXFDUSD	AGIXTRY
AGIXUSDT	AGLDBNB	AGLDBTC	AGLDBUSD	AGLDUSDT	AIBNB
AIBTC	AIFDUSD	AIONBNB	AIONBTC	AIONBUSD	AIONETH
AIONUSDT	AITRY	AITUSD	AIUSDT	AIXBTTRY	AIXBTUSDC
AIXBTUSDT	AKROBTC	AKROBUSD	AKROUSDT	ALCXBTC	ALCXBUSD
ALCXUSDT	ALGOBIDR	ALGOBNB	ALGOBTC	ALGOBUSD	ALGOETH
ALGOFDUSD	ALGOPAX	ALGORUB	ALGOTRY	ALGOTUSD	ALGOUSDC
ALGOUSDT	ALICEBIDR	ALICEBNB	ALICEBTC	ALICEBUSD	ALICETRY
ALICEUSDT	ALPACABNB	ALPACABTC	ALPACABUSD	ALPACAUSDT	ALPHABNB
ALPHABTC	ALPHABUSD	ALPHAUSDT	ALPINEBTC	ALPINEBUSD	ALPINEEUR
ALPINETRY	ALPINEUSDT	ALTBNB	ALTBTC	ALTFDUSD	ALTTRY
ALTUSDC	ALTUSDT	AMBBNB	AMBBTC	AMBBUSD	AMBETH
AMBUSDT	AMPBNB	AMPBTC	AMPBUSD	AMPTRY	AMPUSDT
ANCBNB	ANCBTC	ANCBUSD	ANCUSDT	ANIMEBNB	ANIMEFDUSD
ANIMETRY	ANIMEUSDC	ANIMEUSDT	ANKRBNB	ANKRBTC	ANKRBUSD

ANKRPAX	ANKRTRY	ANKRTUSD	ANKRUSDC	ANKRUSDT	ANTBNB
ANTBTC	ANTBUSD	ANTUSDT	ANYBTC	ANYBUSD	ANYUSDT
APEAUD	APEBNB	APEBRL	APEBTC	APEBUSD	APEETH
APEEUR	APEFDUSD	APEGBP	APETRY	APEUSDC	APEUSDT
API3BNB	API3BTC	API3BUSD	API3TRY	API3USDC	API3USDT
APPCBNB	APPCBTC	APPCETH	APTBRL	APTBTC	APTBUSD
APTETH	APTEUR	APTFDUSD	APTJPY	APTTRY	APTUSDC
APTUSDT	ARBBTC	ARBETH	ARBEUR	ARBFDUSD	ARBNB
ARBRUB	ARBTC	ARBTRY	ARBTUSD	ARBUSD	ARBUSDC
ARBUSDT	ARDRBNB	ARDRBTC	ARDRETH	ARDRUSDT	ARFDUSD
ARKBTC	ARKBUSD	ARKETH	ARKMBNB	ARKMBTC	ARKMFDUSD
ARKMRUB	ARKMTRY	ARKMTUSD	ARKMUSDC	ARKMUSDT	ARKTRY
ARKUSDT	ARNBTC	ARNETH	ARPABNB	ARPABTC	ARPABUSD
ARPAETH	ARPARUB	ARPATRY	ARPAUSDT	ARTRY	ARUSDC
ARUSDT	ASRBTC	ASRBUSD	ASRTRY	ASRUSDT	ASTBTC
ASTERTRY	ASTERUSDC	ASTERUSDT	ASTETH	ASTRBTC	ASTRBUSD
ASTRETH	ASTRUSDT	ASTUSDT	ATABNB	ATABTC	ATABUSD
ATAUSDT	ATMBTC	ATMBUSD	ATMTRY	ATMUSDT	ATOMBIDR
ATOMBNB	ATOMBRL	ATOMBTC	ATOMBUSD	ATOMETH	ATOMEUR
ATOMFDUSD	ATOMPAX	ATOMTRY	ATOMTUSD	ATOMUSDC	ATOMUSDT
ATRY	AUCTIONBTC	AUCTIONBUSD	AUCTIONFDUSD	AUCTIONTRY	AUCTIONUSDC
AUCTIONUSDT	AUDBUSD	AUDIOBTC	AUDIOBUSD	AUDIOTRY	AUDIOUSDT
AUDUSDC	AUDUSDT	AUSDC	AUSDT	AUTOBTC	AUTOBUSD
AUTOUSDT	AVABNB	AVABTC	AVABUSD	AVAUSDT	AVAXAUD
AVAXBIDR	AVAXBNB	AVAXBRL	AVAXBTC	AVAXBUSD	AVAXETH
AVAXEUR	AVAXFDUSD	AVAXGBP	AVAXTRY	AVAXTUSD	AVAXUSDC
AVAXUSDT	AVNTTRY	AVNTUSDC	AVNTUSDT	AWEBTC	AWEUSDT
AXLBTC	AXLFDUSD	AXLTRY	AXLUSDT	AXSAUD	AXSBIDR
AXSBNB	AXSBRL	AXSBTC	AXSBUSD	AXSETH	AXSTRY
AXSUSDC	AXSUSDT	BABYBNB	BABYEUR	BABYFDUSD	BABYTRY
BABYUSDC	BABYUSDT	BADGERBTC	BADGERBUSD	BADGERUSDT	BAKEBNB
BAKEBTC	BAKEBUSD	BAKETRY	BAKEUSDT	BALBNB	BALBTC
BALBUSD	BALUSDT	BANANABNB	BANANABTC	BANANAFDUSD	BANANAS31USDC

BANANAS31USDT	BANANATRY	BANANAUSDC	BANANAUSDT	BANDBNB	BANDBTC
BANDBUSD	BANDTRY	BANDUSDT	BARBTC	BARBUSD	BARDBNB
BARDFDUSD	BARDTRY	BARDUSDC	BARDUSDT	BARTRY	BARUSDT
BATBNB	BATBTC	BATBUSD	BATETH	BATPAX	BATTUSD
BATUSDC	BATUSDT	BBBNB	BBBTC	BBFDUSD	BBTRY
BBUSDC	BBUSDT	BCCBNB	BCCBTC	BCCETH	BCCUSDT
BCDBTC	BCDETH	BCHABCBTC	BCHABCBUSD	BCHABCPAX	BCHABCTUSD
BCHABCUSDC	BCHABCUSDT	BCHABUSD	BCHBNB	BCHBTC	BCHBUSD
BCHEUR	BCHFDUSD	BCHJPY	BCHPAX	BCHSVBTC	BCHSVPAX
BCHSVTUSD	BCHSVUSDC	BCHSVUSDT	BCHTRY	BCHTUSD	BCHUSDC
BCHUSDT	BCNBNB	BCNBTC	BCNETH	BCPTBNB	BCPTBTC
BCPTETH	BCPTPAX	BCPTTUSD	BCPTUSDC	BDOTDOT	BEAMBNB
BEAMBTC	BEAMUSDT	BEAMXTRY	BEAMXUSDC	BEAMXUSDT	BEARBUSD
BEARUSDT	BELBNB	BELBTC	BELBUSD	BELETH	BELTRY
BELUSDT	BERABNB	BERABTC	BERAFDUSD	BERATRY	BERAUSDC
BERAUSDT	BETABNB	BETABTC	BETABUSD	BETAETH	BETAUSDT
BETHBUSD	BETHETH	BETHUSDT	BFUSDUSDC	BFUSDUSDT	BGBPUSDC
BICOBTC	BICOBUSD	BICOUSDT	BIFIBNB	BIFIBUSD	BIFIUSDT
BIGTIMEUSDC	BIGTIMEUSDT	BIOBNB	BIOFDUSD	BIOTRY	BIOUSDC
BIOUSDT	BKRWBUSD	BKRWUSDT	BLURBTC	BLURFDUSD	BLURTRY
BLURUSDC	BLURUSDT	BLZBNB	BLZBTC	BLZBUSD	BLZETH
BLZFDUSD	BLZUSDT	BMTBNB	BMTFDUSD	BMTTRY	BMTUSDC
BMTUSDT	BNBARS	BNBAUD	BNBBEARBUSD	BNBBEARUSDT	BNBBIDR
BNBBKRW	BNBBRL	BNBBTC	BNBBULLBUSD	BNBBULLUSDT	BNBBUSD
BNBDAI	BNBETH	BNBEUR	BNBFDUSD	BNBGBP	BNBIDRT
BNBJPY	BNBNGN	BNBPAX	BNBRUB	BNBTRY	BNBTUSD
BNBUAH	BNBUSDC	BNBUSDP	BNBUSDS	BNBUSDT	BNBUST
BNBZAR	BNSOLSOL	BNSOLUSDT	BNTBTC	BNTBUSD	BNTETH
BNTUSDT	BNXBNB	BNXBTC	BNXBUSD	BNXTRY	BNXUSDC
BNXUSDT	BOMEBTC	BOMEEUR	BOMEFDUSD	BOMETRY	BOMEUSDC
BOMEUSDT	BONDBNB	BONDBTC	BONDBUSD	BONDETH	BONDUSDT
BONKBRL	BONKFDUSD	BONKTRY	BONKUSDC	BONKUSDT	BOTBTC
BOTBUSD	BQXBTC	BQXETH	BRDBNB	BRDBTC	BRDETH

BROCCOLI714USDC	BROCCOLI714USDT	BSWBNB	BSWBUSD	BSWETH	BSWTRY
BSWUSDT	BTCAEUR	BTCARS	BTCAUD	BTCBBTC	BTCBIDR
BTCBKRW	BTCBRL	BTCBUSD	BTCDAI	BTCEUR	BTCEURI
BTCFDUSD	BTCGBP	BTCGYEN	BTCIDRT	BTCJPY	BTCMXN
BTCNGN	BTCPAX	BTCPLN	BTCRON	BTCRUB	BTCSTBTC
BTCSTBUSD	BTCSTUSDT	BTCTRY	BTCTUSD	BTCUAH	BTCUSDC
BTCUSDP	BTCUSDS	BTCUSDT	BTCUST	BTCVAI	BTCZAR
BTGBTC	BTGBUSD	BTGETH	BTGUSDT	BTSBNB	BTSBTC
BTSBUSD	BTSETH	BTSUSDT	BTTBNB	BTTBRL	BTTBTC
BTTBUSD	BTTCBUSD	BTTCTRY	BTTCUSDC	BTTCUSDT	BTTEUR
BTTPAX	BTTTRX	BTTTRY	BTTTUSD	BTTUSDC	BTTUSDT
BULLBUSD	BULLUSDT	BURGERBNB	BURGERBUSD	BURGERETH	BURGERUSDT
BUSDBIDR	BUSDBKRW	BUSDBRL	BUSDBVND	BUSDDAI	BUSDIDRT
BUSDNGN	BUSDPLN	BUSDRON	BUSDRUB	BUSDTRY	BUSDUAH
BUSDUSDT	BUSDVAI	BUSDZAR	BZRXBNB	BZRXBTC	BZRXBUSD
BZRXUSDT	C98BNB	C98BRL	C98BTC	C98BUSD	C98USDT
CAKEAUD	CAKEBNB	CAKEBRL	CAKEBTC	CAKEBUSD	CAKEGBP
CAKETRY	CAKETUSD	CAKEUSDC	CAKEUSDT	CATIBNB	CATIBRL
CATIFDUSD	CATITRY	CATIUSDC	CATIUSDT	CBNB	CDTBTC
CDTETH	CELOBTC	CELOBUSD	CELOTRY	CELOUSDT	CELRBNB
CELRBTC	CELRBUSD	CELRETH	CELRUSDT	CETUSTRY	CETUSUSDC
CETUSUSDT	CFDUSD	CFXBTC	CFXBUSD	CFXTRY	CFXTUSD
CFXUSDC	CFXUSDT	CGPTUSDC	CGPTUSDT	CHATBTC	CHATETH
CHESSBNB	CHESSBTC	CHESSBUSD	CHESSUSDC	CHESSUSDT	CHRBNB
CHRBTC	CHRBUSD	CHRETH	CHRUSDT	CHZBNB	CHZBRL
CHZBTC	CHZBUSD	CHZEUR	CHZFDUSD	CHZGBP	CHZTRY
CHZUSDC	CHZUSDT	CITYBNB	CITYBTC	CITYBUSD	CITYTRY
CITYUSDT	CKBBTC	CKBBUSD	CKBTRY	CKBUSDC	CKBUSDT
CLOAKBTC	CLOAKETH	CLVBNB	CLVBTC	CLVBUSD	CLVUSDT
CMTBNB	CMTBTC	CMTETH	CNDBNB	CNDBTC	CNDETH
COCOSBNB	COCOSBTC	COCOSBUSD	COCOSTRY	COCOSUSDT	COMBOBNB
COMBOTRY	COMBOUSDT	COMPBNB	COMPBTC	COMPBUSD	COMPTRY
COMPTUSD	COMPUSDC	COMPUSDT	COOKIEUSDC	COOKIEUSDT	COSBNB

COSBTC	COSBUSD	COSTRY	COSUSDT	COTIBNB	COTIBTC
COTIBUSD	COTITRY	COTIUSDT	COVERBUSD	COVERETH	COWTRY
COWUSDC	COWUSDT	CREAMBNB	CREAMBUSD	CREAMUSDT	CRVBNB
CRVBTC	CRVBUSD	CRVETH	CRVTRY	CRVUSDC	CRVUSDT
CTKBNB	CTKBTC	CTKBUSD	CTKUSDT	CTRY	CTSIBNB
CTSIBTC	CTSIBUSD	CTSIUSDT	CTXCBNB	CTXCBTC	CTXCBUSD
CTXCUSDT	CUSDC	CUSDT	CVCBNB	CVCBTC	CVCBUSD
CVCETH	CVCUSDC	CVCUSDT	CVPBUSD	CVPETH	CVPUSDT
CVXBTC	CVXBUSD	CVXUSDC	CVXUSDT	CYBERBNB	CYBERBTC
CYBERETH	CYBERFDUSD	CYBERTRY	CYBERTUSD	CYBERUSDC	CYBERUSDT
DAIBNB	DAIBTC	DAIBUSD	DAIJPY	DAIUSDT	DARBNB
DARBTC	DARBUSD	DARETH	DAREUR	DARTRY	DARUSDT
DASHBNB	DASHBTC	DASHBUSD	DASHETH	DASHUSDT	DATABTC
DATABUSD	DATAETH	DATAUSDT	DCRBNB	DCRBTC	DCRBUSD
DCRUSDT	DEGOBTC	DEGOBUSD	DEGOUSDT	DENTBTC	DENTBUSD
DENTETH	DENTTRY	DENTUSDT	DEXEBUSD	DEXEETH	DEXEUSDT
DFBUSD	DFETH	DFUSDC	DFUSDT	DGBBTC	DGBBUSD
DGBUSDT	DGDBTC	DGDETH	DIABNB	DIABTC	DIABUSD
DIAUSDT	DLTBNB	DLTBTC	DLTETH	DNTBTC	DNTBUSD
DNTETH	DNTUSDT	DOCKBTC	DOCKBUSD	DOCKETH	DOCKUSDT
DODOBTC	DODOBUSD	DODOUSDT	DOGEAUD	DOGEBIDR	DOGEBNB
DOGEBRL	DOGEBTC	DOGEBUSD	DOGEEUR	DOGEFDUSD	DOGEGBP
DOGEJPY	DOGEPAX	DOGERUB	DOGETRY	DOGETUSD	DOGEUSDC
DOGEUSDT	DOGSBNB	DOGSBRL	DOGSFDUSD	DOGSTRY	DOGSUSDC
DOGSUSDT	DOLOBNB	DOLOFDUSD	DOLOTRY	DOLOUSDC	DOLOUSDT
DOTAUD	DOTBIDR	DOTBKRW	DOTBNB	DOTBRL	DOTBTC
DOTBUSD	DOTETH	DOTEUR	DOTFDUSD	DOTGBP	DOTNGN
DOTRUB	DOTTRY	DOTTUSD	DOTUSDC	DOTUSDT	DREPBNB
DREPBTC	DREPBUSD	DREPUSDT	DTRY	DUSDT	DUSKBNB
DUSKBTC	DUSKBUSD	DUSKPAX	DUSKUSDC	DUSKUSDT	DYDXBNB
DYDXBTC	DYDXBUSD	DYDXETH	DYDXFDUSD	DYDXTRY	DYDXUSDC
DYDXUSDT	DYMBTC	DYMFDUSD	DYMTRY	DYMUSDT	EASYBTC
EASYETH	EDENBNB	EDENFDUSD	EDENTRY	EDENUSDC	EDENUSDT

EDOBTC	EDOETH	EDUBNB	EDUBTC	EDUEUR	EDUTRY
EDUTUSD	EDUUSDT	EGLDBNB	EGLDBTC	EGLDBUSD	EGLDETH
EGLDEUR	EGLDFDUSD	EGLDRON	EGLDTRY	EGLDUSDC	EGLDUSDT
EIGENBTC	EIGENFDUSD	EIGENTRY	EIGENUSDC	EIGENUSDT	ELFBTC
ELFBUSD	ELFETH	ELFUSDT	ENABNB	ENABRL	ENABTC
ENAEUR	ENAFDUSD	ENATRY	ENAUSDC	ENAUSDT	ENGBTC
ENGETH	ENJBNB	ENJBRL	ENJBTC	ENJBUSD	ENJETH
ENJEUR	ENJGBP	ENJTRY	ENJUSDC	ENJUSDT	ENSBNB
ENSBTC	ENSBUSD	ENSFDUSD	ENSOBNB	ENSOFDUSD	ENSOTRY
ENSOUSDC	ENSOUSDT	ENSTRY	ENSUSDC	ENSUSDT	EOSAUD
EOSBEARBUSD	EOSBEARUSDT	EOSBNB	EOSBTC	EOSBULLBUSD	EOSBULLUSDT
EOSBUSD	EOSETH	EOSEUR	EOSFDUSD	EOSPAX	EOSTRY
EOSTUSD	EOSUSDC	EOSUSDT	EPICUSDC	EPICUSDT	EPSBTC
EPSBUSD	EPSUSDT	EPXBUSD	EPXUSDT	ERABNB	ERAFDUSD
ERATRY	ERAUSDC	ERAUSDT	ERDBNB	ERDBTC	ERDBUSD
ERDPAX	ERDUSDC	ERDUSDT	ERNBNB	ERNBUSD	ERNUSDT
ETCBNB	ETCBRL	ETCBTC	ETCBUSD	ETCETH	ETCEUR
ETCFDUSD	ETCGBP	ETCPAX	ETCTRY	ETCTUSD	ETCUSDC
ETCUSDT	ETHAEUR	ETHARS	ETHAUD	ETHBEARBUSD	ETHBEARUSDT
ETHBIDR	ETHBKRW	ETHBRL	ETHBTC	ETHBULLBUSD	ETHBULLUSDT
ETHBUSD	ETHDAI	ETHEUR	ETHEURI	ETHFDUSD	ETHFIBNB
ETHFIBTC	ETHFIFDUSD	ETHFITRY	ETHFIUSDC	ETHFIUSDT	ETHGBP
ETHJPY	ETHMXN	ETHNGN	ETHPAX	ETHPLN	ETHRUB
ETHTRY	ETHTUSD	ETHUAH	ETHUSDC	ETHUSDP	ETHUSDT
ETHUST	ETHZAR	EULBNB	EULFDUSD	EULTRY	EULUSDC
EULUSDT	EURAEUR	EURBUSD	EUREURI	EURIUSDC	EURIUSDT
EURUSDC	EURUSDT	EVXBTC	EVXETH	EZBTC	EZETH
FARMBNB	FARMBTC	FARMBUSD	FARMETH	FARMUSDT	FDUSDBUSD
FDUSDTRY	FDUSDUSDC	FDUSDUSDT	FETBNB	FETBRL	FETBTC
FETBUSD	FETFDUSD	FETTRY	FETUSDC	FETUSDT	FFBNB
FFFDUSD	FFTRY	FFUSDC	FFUSDT	FIDABNB	FIDABTC
FIDABUSD	FIDATRY	FIDAUSDT	FILBNB	FILBTC	FILBUSD

FILETH	FILFDUSD	FILTRY	FILUSDC	FILUSDT	FIOBNB
FIOBTC	FIOBUSD	FIOUSDT	FIROBTC	FIROBUSD	FIROETH
FIROUSDT	FISBIDR	FISBRL	FISBTC	FISBUSD	FISTRY
FISUSDT	FLMBNB	FLMBTC	FLMBUSD	FLMUSDT	FLOKIFDUSD
FLOKITRY	FLOKITUSD	FLOKIUSDC	FLOKIUSDT	FLOWBNB	FLOWBTC
FLOWBUSD	FLOWUSDT	FLUXBTC	FLUXBUSD	FLUXUSDC	FLUXUSDT
FORBNB	FORBTC	FORBUSD	FORMTRY	FORMUSDC	FORMUSDT
FORTHBTC	FORTHBUSD	FORTHUSDT	FORUSDT	FRONTBTC	FRONTBUSD
FRONTETH	FRONTTRY	FRONTTUSD	FRONTUSDC	FRONTUSDT	FTMAUD
FTMBIDR	FTMBNB	FTMBRL	FTMBTC	FTMBUSD	FTMETH
FTMEUR	FTMFDUSD	FTMPAX	FTMRUB	FTMTRY	FTMTUSD
FTMUSDC	FTMUSDT	FTTBNB	FTTBTC	FTTBUSD	FTTETH
FTTUSDT	FUELBTC	FUELETH	FUNBNB	FUNBTC	FUNETH
FUNUSDC	FUNUSDT	FXSBTC	FXSBUSD	FXSUSDT	GALAAUD
GALABNB	GALABRL	GALABTC	GALABUSD	GALAETH	GALAEUR
GALAFDUSD	GALATRY	GALAUSDC	GALAUSDT	GALBNB	GALBRL
GALBTC	GALBUSD	GALETH	GALEUR	GALTRY	GALUSDT
GASBTC	GASBUSD	GASFDUSD	GASTRY	GASUSDT	GBPBUSD
GBPUSDT	GFTBUSD	GFTUSDT	GHSTBUSD	GHSTETH	GHSTUSDT
GLMBTC	GLMBUSD	GLMETH	GLMRBNB	GLMRBTC	GLMRBUSD
GLMRUSDT	GLMUSDT	GMTAUD	GMTBNB	GMTBRL	GMTBTC
GMTBUSD	GMTETH	GMTEUR	GMTGBP	GMTTRY	GMTUSDC
GMTUSDT	GMXBTC	GMXBUSD	GMXUSDC	GMXUSDT	GNOBNB
GNOBTC	GNOBUSD	GNOUSDT	GNSBTC	GNSUSDT	GNTBNB
GNTBTC	GNTETH	GOBNB	GOBTC	GPSBNB	GPSFDUSD
GPSTRY	GPSUSDC	GPSUSDT	GRSBTC	GRSETH	GRTBTC
GRTBUSD	GRTETH	GRTEUR	GRTFDUSD	GRTTRY	GRTUSDC
GRTUSDT	GTCBNB	GTCBTC	GTCBUSD	GTCUSDT	GTOBNB
GTOBTC	GTOBUSD	GTOETH	GTOPAX	GTOTUSD	GTOUSDC
GTOUSDT	GTRY	GUNBNB	GUNFDUSD	GUNTRY	GUNUSDC
GUNUSDT	GUSDT	GVTBTC	GVTETH	GXSBNB	GXSBTC
GXSETH	GXSUSDT	HAEDALBNB	HAEDALFDUSD	HAEDALTRY	HAEDALUSDC
HAEDALUSDT	HARDBNB	HARDBTC	HARDBUSD	HARDUSDT	HBARBNB

HBARBTC	HBARBUSD	HBARFDUSD	HBARTRY	HBARUSDC	HBARUSDT
HCBTC	HCETH	HCUSDT	HEGICBUSD	HEGICETH	HEIBTC
HEIUSDC	HEIUSDT	HEMIBNB	HEMIFDUSD	HEMITRY	HEMIUSDC
HEMIUSDT	HFTBTC	HFTBUSD	HFTUSDT	HIFIETH	HIFIUSDT
HIGHBNB	HIGHBTC	HIGHBUSD	HIGHTRY	HIGHUSDT	HIVEBNB
HIVEBTC	HIVEBUSD	HIVEFDUSD	HIVETRY	HIVEUSDC	HIVEUSDT
HMSTRBNB	HMSTRFDUSD	HMSTRTRY	HMSTRUSDC	HMSTRUSDT	HNTBTC
HNTBUSD	HNTUSDT	HOLOBNB	HOLOFDUSD	HOLOTRY	HOLOUSDC
HOLOUSDT	HOMEBNB	HOMEFDUSD	HOMETRY	HOMEUSDC	HOMEUSDT
HOOKBNB	HOOKBTC	HOOKBUSD	HOOKUSDT	HOTBNB	HOTBRL
HOTBTC	HOTBUSD	HOTETH	HOTEUR	HOTTRY	HOTUSDT
HSRBTC	HSRETH	HUMABNB	HUMAFDUSD	HUMATRY	HUMAUSDC
HUMAUSDT	HYPERBNB	HYPERFDUSD	HYPERTRY	HYPERUSDC	HYPERUSDT
ICNBTC	ICNETH	ICPBNB	ICPBTC	ICPBUSD	ICPETH
ICPEUR	ICPFDUSD	ICPRUB	ICPTRY	ICPUSDC	ICPUSDT
ICXBNB	ICXBTC	ICXBUSD	ICXETH	ICXUSDT	IDBNB
IDBTC	IDEUR	IDEXBNB	IDEXBTC	IDEXBUSD	IDEXFDUSD
IDEXUSDC	IDEXUSDT	IDTRY	IDTUSD	IDUSDT	ILVBNB
ILVBTC	ILVBUSD	ILVUSDC	ILVUSDT	IMXBNB	IMXBTC
IMXBUSD	IMXTRY	IMXUSDC	IMXUSDT	INITBNB	INITFDUSD
INITTRY	INITUSDC	INITUSDT	INJBNB	INJBTC	INJBUSD
INJETH	INJFDUSD	INJTRY	INJTUSD	INJUSDC	INJUSDT
INSBTC	INSETH	IOBNB	IOBTC	IOFDUSD	IOSTBTC
IOSTBUSD	IOSTETH	IOSTUSDT	IOTABNB	IOTABTC	IOTABUSD
IOTAETH	IOTAFDUSD	IOTATRY	IOTAUSDC	IOTAUSDT	IOTRY
IOTXBTC	IOTXBUSD	IOTXETH	IOTXJPY	IOTXUSDT	IOUSDC
IOUSDT	IQBNB	IQBUSD	IQUSDT	IRISBNB	IRISBTC
IRISBUSD	IRISUSDT	JASMYBNB	JASMYBTC	JASMYBUSD	JASMYETH
JASMYEUR	JASMYTRY	JASMYUSDT	JOEBTC	JOEBUSD	JOETRY
JOEUSDT	JSTBNB	JSTBTC	JSTBUSD	JSTUSDT	JTOFDUSD
JTOTRY	JTOUSDC	JTOUSDT	JUPFDUSD	JUPTRY	JUPUSDC
JUPUSDT	JUVBTC	JUVBUSD	JUVTRY	JUVUSDC	JUVUSDT

KAIAUSDC	KAIAUSDT	KAITOBNB	KAITOBRL	KAITOBTC	KAITOFDUSD
KAITOTRY	KAITOUSDC	KAITOUSDT	KAVABNB	KAVABTC	KAVABUSD
KAVAETH	KAVAUSDT	KDABTC	KDABUSD	KDAUSDT	KEEPBNB
KEEPBTC	KEEPBUSD	KEEPUSDT	KERNELBNB	KERNELFDUSD	KERNELTRY
KERNELUSDC	KERNELUSDT	KEYBTC	KEYBUSD	KEYETH	KEYUSDT
KLAYBNB	KLAYBTC	KLAYBUSD	KLAYUSDT	KMDBTC	KMDBUSD
KMDETH	KMDUSDT	KMNOUSDC	KMNOUSDT	KNCBNB	KNCBTC
KNCBUSD	KNCETH	KNCUSDT	KP3RBNB	KP3RBUSD	KP3RUSDT
KSMAUD	KSMBNB	KSMBTC	KSMBUSD	KSMETH	KSMTRY
KSMUSDT	LABNB	LAFDUSD	LATRY	LAUSDC	LAUSDT
LAYERBNB	LAYERBTC	LAYERFDUSD	LAYERTRY	LAYERUSDC	LAYERUSDT
LAZIOBTC	LAZIOBUSD	LAZIOEUR	LAZIOTRY	LAZIOUSDT	LDOBTC
LDOBUSD	LDOFDUSD	LDOTRY	LDOTUSD	LDOUSDC	LDOUSDT
LENDBKRW	LENDBTC	LENDBUSD	LENDETH	LENDUSDT	LEVERBUSD
LEVERTRY	LEVERUSDT	LINABNB	LINABTC	LINABUSD	LINAUSDT
LINEABNB	LINEAFDUSD	LINEATRY	LINEAUSDC	LINEAUSDT	LINKAUD
LINKBKRW	LINKBNB	LINKBRL	LINKBTC	LINKBUSD	LINKETH
LINKEUR	LINKFDUSD	LINKGBP	LINKJPY	LINKNGN	LINKPAX
LINKTRY	LINKTUSD	LINKUSDC	LINKUSDT	LISTABNB	LISTABRL
LISTAFDUSD	LISTATRY	LISTAUSDC	LISTAUSDT	LITBTC	LITBUSD
LITETH	LITUSDT	LOKABNB	LOKABTC	LOKABUSD	LOKAUSDT
LOOMBNB	LOOMBTC	LOOMBUSD	LOOMETH	LOOMTRY	LOOMUSDT
LPTBNB	LPTBTC	LPTBUSD	LPTJPY	LPTTRY	LPTUSDC
LPTUSDT	LQTYBTC	LQTYFDUSD	LQTYUSDT	LRCBNB	LRCBTC
LRCBUSD	LRCETH	LRCTRY	LRCUSDT	LSKBNB	LSKBTC
LSKBUSD	LSKETH	LSKUSDT	LTCBNB	LTCBRL	LTCBTC
LTCBUSD	LTCETH	LTCEUR	LTCFDUSD	LTCGBP	LTCJPY
LTCNGN	LTCPAX	LTCRUB	LTCTRY	LTCTUSD	LTCUAH
LTCUSDC	LTCUSDT	LTOBNB	LTOBTC	LTOBUSD	LTOUSDT
LUMIATRY	LUMIAUSDT	LUNAAUD	LUNABIDR	LUNABNB	LUNABRL
LUNABTC	LUNABUSD	LUNAETH	LUNAEUR	LUNAGBP	LUNATRY
LUNAUSDT	LUNAUST	LUNBTC	LUNCBUSD	LUNCTRY	LUNCUSDT
LUNETH	MAGICBTC	MAGICBUSD	MAGICFDUSD	MAGICTRY	MAGICUSDC

MAGICUSDT	MANABIDR	MANABNB	MANABRL	MANABTC	MANABUSD
MANAETH	MANATRY	MANAUSDT	MANTABNB	MANTABTC	MANTAFDUSD
MANTATRY	MANTAUSDC	MANTAUSDT	MASKBNB	MASKBUSD	MASKTRY
MASKUSDC	MASKUSDT	MATICAUD	MATICBIDR	MATICBNB	MATICBRL
MATICBTC	MATICBUSD	MATICETH	MATICEUR	MATICFDUSD	MATICGBP
MATICJPY	MATICRUB	MATICTRY	MATICTUSD	MATICUSDC	MATICUSDT
MAVBTC	MAVTRY	MAVTUSD	MAVUSDC	MAVUSDT	MBLBNB
MBLBTC	MBLBUSD	MBLUSDT	MBOXBNB	MBOXBTC	MBOXBUSD
MBOXTRY	MBOXUSDT	MCBNB	MCBTC	MCBUSD	MCOBNB
MCOBTC	MCOETH	MCOUSDT	MCUSDT	MDABTC	MDAETH
MDTBNB	MDTBTC	MDTBUSD	MDTUSDT	MDXBNB	MDXBTC
MDXBUSD	MDXUSDT	MEBTC	MEFDUSD	MEMEBNB	MEMEETH
MEMEFDUSD	MEMETRY	MEMETUSD	MEMEUSDC	MEMEUSDT	METISBTC
METISFDUSD	METISTRY	METISUSDT	METRY	MEUSDC	MEUSDT
MFTBNB	MFTBTC	MFTETH	MFTUSDT	MINABNB	MINABTC
MINABUSD	MINATRY	MINAUSDT	MIRABNB	MIRAFDUSD	MIRATRY
MIRAUSDC	MIRAUSDT	MIRBTC	MIRBUSD	MIRUSDT	MITHBNB
MITHBTC	MITHUSDT	MITOBNB	MITOFDUSD	MITOTRY	MITOUSDC
MITOUSDT	MKRBNB	MKRBTC	MKRBUSD	MKRTRY	MKRUSDC
MKRUSDT	MLNBNB	MLNBTC	MLNBUSD	MLNUSDT	MOBBTC
MOBBUSD	MOBUSDT	MODBTC	MODETH	MORPHOBNB	MORPHOFDUSD
MORPHOTRY	MORPHOUSDC	MORPHOUSDT	MOVEBNB	MOVEBTC	MOVEFDUSD
MOVETRY	MOVEUSDC	MOVEUSDT	MOVRBNB	MOVRBTC	MOVRBUSD
MOVRTRY	MOVRUSDT	MTHBTC	MTHETH	MTLBTC	MTLBUSD
MTLETH	MTLTRY	MTLUSDT	MUBARAKTRY	MUBARAKUSDC	MUBARAKUSDT
MULTIBTC	MULTIBUSD	MULTIUSDT	NANOBNB	NANOBTC	NANOBUSD
NANOETH	NANOUSDT	NASBNB	NASBTC	NASETH	NAVBNB
NAVBTC	NAVETH	NBSBTC	NBSUSDT	NCASHBNB	NCASHBTC
NCASHETH	NEARBNB	NEARBRL	NEARBTC	NEARBUSD	NEARETH
NEAREUR	NEARFDUSD	NEARJPY	NEARRUB	NEARTRY	NEARUSDC
NEARUSDT	NEBLBNB	NEBLBTC	NEBLBUSD	NEBLUSDT	NEIROBRL
NEIROEUR	NEIROFDUSD	NEIROJPY	NEIROTRY	NEIROUSDC	NEIROUSDT
NEOBNB	NEOBTC	NEOBUSD	NEOETH	NEOPAX	NEORUB

NEOTRY	NEOTUSD	NEOUSDC	NEOUSDT	NEWTBNB	NEWTFDUSD
NEWTTRY	NEWTUSDC	NEWTUSDT	NEXOBTC	NEXOBUSD	NEXOUSDT
NFPBNB	NFPBTC	NFPFDUSD	NFPTRY	NFPTUSD	NFPUSDT
NILBNB	NILFDUSD	NILTRY	NILUSDC	NILUSDT	NKNBNB
NKNBTC	NKNBUSD	NKNUSDT	NMRBTC	NMRBUSD	NMRTRY
NMRUSDC	NMRUSDT	NOMFDUSD	NOMTRY	NOMUSDC	NOMUSDT
NOTBNB	NOTBRL	NOTEUR	NOTFDUSD	NOTTRY	NOTUSDC
NOTUSDT	NPXSBTC	NPXSETH	NPXSUSDC	NPXSUSDT	NTRNBNB
NTRNBTC	NTRNTRY	NTRNUSDT	NUAUD	NUBNB	NUBTC
NUBUSD	NULSBNB	NULSBTC	NULSBUSD	NULSETH	NULSUSDT
NURUB	NUUSDT	NXPCBNB	NXPCFDUSD	NXPCTRY	NXPCUSDC
NXPCUSDT	NXSBNB	NXSBTC	NXSETH	OAXBTC	OAXETH
OAXUSDT	OCEANBNB	OCEANBTC	OCEANBUSD	OCEANTRY	OCEANUSDT
OGBTC	OGBUSD	OGNBNB	OGNBTC	OGNBUSD	OGNTRY
OGNUSDT	OGTRY	OGUSDT	OMBTC	OMBUSD	OMGBNB
OMGBTC	OMGBUSD	OMGETH	OMGUSDT	OMNIBNB	OMNIBRL
OMNIBTC	OMNIFDUSD	OMNITRY	OMNIUSDC	OMNIUSDT	OMTRY
OMUSDC	OMUSDT	ONDOTRY	ONDOUSDC	ONDOUSDT	ONEBIDR
ONEBNB	ONEBTC	ONEBUSD	ONEETH	ONEPAX	ONETRY
ONETUSD	ONEUSDC	ONEUSDT	ONGBNB	ONGBTC	ONGUSDT
ONTBNB	ONTBTC	ONTBUSD	ONTETH	ONTPAX	ONTTRY
ONTUSDC	ONTUSDT	OOKIBNB	OOKIBUSD	OOKIETH	OOKIUSDT
OPBNB	OPBTC	OPBUSD	OPENBNB	OPENFDUSD	OPENTRY
OPENUSDC	OPENUSDT	OPETH	OPEUR	OPFDUSD	OPTRY
OPTUSD	OPUSDC	OPUSDT	ORCAFDUSD	ORCATRY	ORCAUSDC
ORCAUSDT	ORDIBTC	ORDIFDUSD	ORDITRY	ORDITUSD	ORDIUSDC
ORDIUSDT	ORNBTC	ORNBUSD	ORNUSDT	OSMOBTC	OSMOBUSD
OSMOUSDC	OSMOUSDT	OSTBNB	OSTBTC	OSTETH	OXTBTC
OXTBUSD	OXTUSDT	PARTIBNB	PARTIFDUSD	PARTITRY	PARTIUSDC
PARTIUSDT	PAXBNB	PAXBTC	PAXBUSD	PAXETH	PAXGBNB
PAXGBTC	PAXGBUSD	PAXGTRY	PAXGUSDC	PAXGUSDT	PAXTUSD
PAXUSDT	PDABTC	PDAUSDT	PENDLEBTC	PENDLEFDUSD	PENDLETRY
PENDLETUSD	PENDLEUSDC	PENDLEUSDT	PENGUBNB	PENGUFDUSD	PENGUTRY

PENGUUSDC	PENGUUSDT	PEOPLEBNB	PEOPLEBTC	PEOPLEBUSD	PEOPLEETH
PEOPLEFDUSD	PEOPLETRY	PEOPLEUSDC	PEOPLEUSDT	PEPEBRL	PEPEEUR
PEPEFDUSD	PEPEJPY	PEPETRY	PEPETUSD	PEPEUSDC	PEPEUSDT
PERLBNB	PERLBTC	PERLUSDC	PERLUSDT	PERPBTC	PERPBUSD
PERPUSDT	PHABTC	PHABUSD	PHATRY	PHAUSDC	PHAUSDT
PHBBNB	PHBBTC	PHBBUSD	PHBPAX	PHBTRY	PHBTUSD
PHBUSDC	PHBUSDT	PHXBNB	PHXBTC	PHXETH	PIVXBNB
PIVXBTC	PIVXUSDT	PIXELBNB	PIXELBTC	PIXELFDUSD	PIXELTRY
PIXELUSDC	PIXELUSDT	PLABNB	PLABTC	PLABUSD	PLAUSDT
PLUMEBNB	PLUMEFDUSD	PLUMETRY	PLUMEUSDC	PLUMEUSDT	PNTBTC
PNTUSDT	PNUTBRL	PNUTBTC	PNUTEUR	PNUTFDUSD	PNUTTRY
PNUTUSDC	PNUTUSDT	POABNB	POABTC	POAETH	POEBTC
POEETH	POLBNB	POLBRL	POLBTC	POLETH	POLEUR
POLFDUSD	POLJPY	POLSBNB	POLSBTC	POLSBUSD	POLSUSDT
POLTRY	POLUSDC	POLUSDT	POLYBNB	POLYBTC	POLYBUSD
POLYUSDT	POLYXBTC	POLYXBUSD	POLYXTRY	POLYXUSDT	PONDBTC
PONDBUSD	PONDUSDT	PORTALBNB	PORTALBTC	PORTALFDUSD	PORTALTRY
PORTALUSDT	PORTOBTC	PORTOBUSD	PORTOEUR	PORTOTRY	PORTOUSDT
POWRBNB	POWRBTC	POWRBUSD	POWRETH	POWRUSDT	PPTBTC
PPTETH	PROMBNB	PROMBTC	PROMBUSD	PROMUSDT	PROSBUSD
PROSETH	PROSUSDT	PROVEBNB	PROVEFDUSD	PROVETRY	PROVEUSDC
PROVEUSDT	PSGBTC	PSGBUSD	PSGTRY	PSGUSDT	PUMPTRY
PUMPUSDC	PUMPUSDT	PUNDIXBUSD	PUNDIXETH	PUNDIXUSDC	PUNDIXUSDT
PYRBTC	PYRBUSD	PYRUSDT	PYTHBTC	PYTHFDUSD	PYTHTRY
PYTHUSDC	PYTHUSDT	QIBNB	QIBTC	QIBUSD	QIUSDT
QKCBTC	QKCBUSD	QKCETH	QKCUSDT	QLCBNB	QLCBTC
QLCETH	QNTBNB	QNTBTC	QNTBUSD	QNTUSDC	QNTUSDT
QSPBNB	QSPBTC	QSPETH	QTUMBNB	QTUMBTC	QTUMBUSD
QTUMETH	QTUMTRY	QTUMUSDC	QTUMUSDT	QUICKBNB	QUICKBTC
QUICKBUSD	QUICKTUSD	QUICKUSDT	RADBNB	RADBTC	RADBUSD
RADTRY	RADUSDT	RAMPBTC	RAMPBUSD	RAMPUSDT	RAREBNB
RAREBRL	RAREBTC	RAREBUSD	RARETRY	RAREUSDC	RAREUSDT
RAYBNB	RAYBUSD	RAYFDUSD	RAYTRY	RAYUSDC	RAYUSDT

RCNBNB	RCNBTC	RCNETH	RDNBNB	RDNBTC	RDNETH
RDNTBTC	RDNTTRY	RDNTTUSD	RDNTUSDT	REDBTC	REDFDUSD
REDTRY	REDUSDC	REDUSDT	REEFBIDR	REEFBTC	REEFBUSD
REEFTRY	REEFUSDT	REIBNB	REIBUSD	REIETH	REIUSDT
RENBNB	RENBTC	RENBTCBTC	RENBTCETH	RENBUSD	RENDERBRL
RENDERBTC	RENDEREUR	RENDERFDUSD	RENDERTRY	RENDERUSDC	RENDERUSDT
RENUSDT	REPBNB	REPBTC	REPBUSD	REPUSDT	REQBTC
REQBUSD	REQETH	REQUSDT	RESOLVBNB	RESOLVFDUSD	RESOLVTRY
RESOLVUSDC	RESOLVUSDT	REZBNB	REZBTC	REZFDUSD	REZTRY
REZUSDC	REZUSDT	RGTBNB	RGTBTC	RGTBUSD	RGTUSDT
RIFBTC	RIFUSDT	RLCBNB	RLCBTC	RLCBUSD	RLCETH
RLCUSDT	RNDRBRL	RNDRBTC	RNDRBUSD	RNDREUR	RNDRFDUSD
RNDRTRY	RNDRUSDC	RNDRUSDT	RONINBTC	RONINFDUSD	RONINTRY
RONINUSDT	ROSEBNB	ROSEBTC	ROSEBUSD	ROSEETH	ROSETRY
ROSEUSDC	ROSEUSDT	RPLBTC	RPLBUSD	RPLUSDC	RPLUSDT
RPXBNB	RPXBTC	RPXETH	RSRBNB	RSRBTC	RSRBUSD
RSRFDUSD	RSRTRY	RSRUSDC	RSRUSDT	RUNEAUD	RUNEBNB
RUNEBTC	RUNEBUSD	RUNEETH	RUNEEUR	RUNEFDUSD	RUNEGBP
RUNETRY	RUNETUSD	RUNEUSDC	RUNEUSDT	RVNBTC	RVNBUSD
RVNTRY	RVNUSDC	RVNUSDT	SAGABNB	SAGABTC	SAGAFDUSD
SAGATRY	SAGAUSDC	SAGAUSDT	SAHARABNB	SAHARAFDUSD	SAHARATRY
SAHARAUSDC	SAHARAUSDT	SALTBTC	SALTETH	SANDAUD	SANDBIDR
SANDBNB	SANDBRL	SANDBTC	SANDBUSD	SANDETH	SANDFDUSD
SANDTRY	SANDUSDC	SANDUSDT	SANTOSBRL	SANTOSBTC	SANTOSBUSD
SANTOSTRY	SANTOSUSDT	SBNB	SBTC	SCBTC	SCBUSD
SCETH	SCRBTC	SCRFDUSD	SCRTBTC	SCRTBUSD	SCRTETH
SCRTRY	SCRTUSDT	SCRUSDT	SCUSDT	SEIBNB	SEIBTC
SEIFDUSD	SEIJPY	SEITRY	SEITUSD	SEIUSDC	SEIUSDT
SETH	SEUR	SFDUSD	SFPBTC	SFPBUSD	SFPUSDT
SHELLBNB	SHELLBTC	SHELLFDUSD	SHELLTRY	SHELLUSDC	SHELLUSDT
SHIBAUD	SHIBBRL	SHIBBUSD	SHIBDOGE	SHIBEUR	SHIBFDUSD
SHIBGBP	SHIBJPY	SHIBRUB	SHIBTRY	SHIBTUSD	SHIBUAH
SHIBUSDC	SHIBUSDT	SIGNBNB	SIGNFDUSD	SIGNTRY	SIGNUSDC

SIGNUSDT	SKLBTC	SKLBUSD	SKLTRY	SKLUSDC	SKLUSDT
SKYBNB	SKYBTC	SKYCOINBNB	SKYCOINBTC	SKYCOINETH	SKYETH
SKYTRY	SKYUSDC	SKYUSDT	SLFBTC	SLFTRY	SLFUSDC
SLFUSDT	SLPBIDR	SLPBNB	SLPBUSD	SLPETH	SLPTRY
SLPUSDT	SNGLSBTC	SNGLSETH	SNMBTC	SNMBUSD	SNMETH
SNTBTC	SNTBUSD	SNTETH	SNTUSDT	SNXBNB	SNXBTC
SNXBUSD	SNXETH	SNXTRY	SNXUSDC	SNXUSDT	SOLARS
SOLAUD	SOLBIDR	SOLBNB	SOLBRL	SOLBTC	SOLBUSD
SOLETH	SOLEUR	SOLFDUSD	SOLGBP	SOLJPY	SOLMXN
SOLRUB	SOLTRY	SOLTUSD	SOLUSDC	SOLUSDT	SOLVBNB
SOLVFDUSD	SOLVTRY	SOLVUSDT	SOMIBNB	SOMIFDUSD	SOMITRY
SOMIUSDC	SOMIUSDT	SOPHBNB	SOPHFDUSD	SOPHTRY	SOPHUSDC
SOPHUSDT	SPARTABNB	SPELLBNB	SPELLBTC	SPELLBUSD	SPELLTRY
SPELLUSDT	SPKBNB	SPKFDUSD	SPKTRY	SPKUSDC	SPKUSDT
SRMBIDR	SRMBNB	SRMBTC	SRMBUSD	SRMUSDT	SSVBTC
SSVBUSD	SSVETH	SSVTUSD	SSVUSDC	SSVUSDT	STEEMBNB
STEEMBTC	STEEMBUSD	STEEMETH	STEEMUSDC	STEEMUSDT	STGBTC
STGBUSD	STGUSDT	STMXBTC	STMXBUSD	STMXETH	STMXTRY
STMXUSDT	STOBNB	STOFDUSD	STORJBTC	STORJBUSD	STORJETH
STORJTRY	STORJUSDT	STORMBNB	STORMBTC	STORMETH	STORMUSDT
STOTRY	STOUSDC	STOUSDT	STPTBNB	STPTBTC	STPTBUSD
STPTUSDT	STRATBNB	STRATBTC	STRATBUSD	STRATETH	STRATUSDT
STRAXBTC	STRAXBUSD	STRAXETH	STRAXTRY	STRAXUSDT	STRKBTC
STRKFDUSD	STRKTRY	STRKUSDC	STRKUSDT	STRY	STXBNB
STXBTC	STXBUSD	STXFDUSD	STXTRY	STXUSDC	STXUSDT
SUBBTC	SUBETH	SUIBNB	SUIBRL	SUIBTC	SUIEUR
SUIFDUSD	SUIJPY	SUITRY	SUITUSD	SUIUSDC	SUIUSDT
SUNBTC	SUNBUSD	SUNTRY	SUNUSDT	SUPERBTC	SUPERBUSD
SUPERFDUSD	SUPERTRY	SUPERUSDT	SUSDBTC	SUSDC	SUSDETH
SUSDT	SUSDUSDT	SUSHIBNB	SUSHIBTC	SUSHIBUSD	SUSHITRY
SUSHIUSDC	SUSHIUSDT	SWRVBNB	SWRVBUSD	SXPAUD	SXPBIDR
SXPBNB	SXPBTC	SXPBUSD	SXPEUR	SXPGBP	SXPTRY
SXPUSDT	SXTBNB	SXTFDUSD	SXTTRY	SXTUSDC	SXTUSDT

SYNBTC	SYNUSDC	SYNUSDT	SYRUPTRY	SYRUPUSDC	SYRUPUSDT
SYSBNB	SYSBTC	SYSBUSD	SYSETH	SYSUSDT	TAOBTC
TAOFDUSD	TAOTRY	TAOUSDC	TAOUSDT	TBUSD	TCTBNB
TCTBTC	TCTUSDT	TFUELBNB	TFUELBTC	TFUELBUSD	TFUELPAX
TFUELTUSD	TFUELUSDC	TFUELUSDT	THEBNB	THEBTC	THEFDUSD
THETABNB	THETABTC	THETABUSD	THETAETH	THETAEUR	THETAFDUSD
THETATRY	THETAUSDC	THETAUSDT	THETRY	THEUSDC	THEUSDT
TIABTC	TIAFDUSD	TIATRY	TIATUSD	TIAUSDC	TIAUSDT
TKOBIDR	TKOBTC	TKOBUSD	TKOUSDT	TLMBNB	TLMBTC
TLMBUSD	TLMFDUSD	TLMTRY	TLMUSDC	TLMUSDT	TNBBTC
TNBETH	TNSRBTC	TNSRFDUSD	TNSRTRY	TNSRUSDC	TNSRUSDT
TNTBTC	TNTETH	TOMOBNB	TOMOBTC	TOMOBUSD	TOMOTRY
TOMOUSDC	TOMOUSDT	TONBTC	TONFDUSD	TONTRY	TONUSDC
TONUSDT	TORNBNB	TORNBTC	TORNBUSD	TORNUSDT	TOWNSBNB
TOWNSFDUSD	TOWNSTRY	TOWNSUSDC	TOWNSUSDT	TRBBNB	TRBBTC
TRBBUSD	TRBFDUSD	TRBTRY	TRBUSDC	TRBUSDT	TREEBNB
TREEFDUSD	TREETRY	TREEUSDC	TREEUSDT	TRIBEBNB	TRIBEBTC
TRIBEBUSD	TRIBEUSDT	TRIGBNB	TRIGBTC	TRIGETH	TROYBNB
TROYBTC	TROYBUSD	TROYTRY	TROYUSDC	TROYUSDT	TRUBTC
TRUBUSD	TRUMPBRL	TRUMPEUR	TRUMPFDUSD	TRUMPJPY	TRUMPTRY
TRUMPUSDC	TRUMPUSDT	TRURUB	TRUTRY	TRUUSDT	TRXAUD
TRXBNB	TRXBTC	TRXBUSD	TRXETH	TRXEUR	TRXFDUSD
TRXJPY	TRXNGN	TRXPAX	TRXTRY	TRXTUSD	TRXUSDC
TRXUSDT	TRXXRP	TSTFDUSD	TSTTRY	TSTUSDC	TSTUSDT
TURBOFDUSD	TURBOTRY	TURBOUSDC	TURBOUSDT	TUSDBNB	TUSDBTC
TUSDBTUSD	TUSDBUSD	TUSDC	TUSDETH	TUSDT	TUSDTRY
TUSDUSDT	TUTUSDC	TUTUSDT	TVKBTC	TVKBUSD	TVKUSDT
TWTBTC	TWTBUSD	TWTTRY	TWTUSDC	TWTUSDT	UFTBUSD
UFTETH	UFTUSDT	UMABTC	UMABUSD	UMATRY	UMAUSDC
UMAUSDT	UNFIBNB	UNFIBTC	UNFIBUSD	UNFIETH	UNFITRY
UNFIUSDT	UNIAUD	UNIBNB	UNIBTC	UNIBUSD	UNIETH
UNIEUR	UNIFDUSD	UNITRY	UNIUSDC	UNIUSDT	USD1USDT
USDCBNB	USDCBRL	USDCBUSD	USDCCZK	USDCPAX	USDCPLN

USDCRON	USDCTRY	USDCTUSD	USDCUSDT	USDEUSDC	USDEUSDT
USDPBUSD	USDPUSDT	USDSBUSDS	USDSBUSDT	USDSPAX	USDSTUSD
USDSUSDC	USDSUSDT	USDTARS	USDTBIDR	USDTBKRW	USDTBRL
USDTBVND	USDTCOP	USDTCZK	USDTDAI	USDTGYEN	USDTIDRT
USDTMXN	USDTNGN	USDTPLN	USDTRON	USDTRUB	USDTTRY
USDTUAH	USDTVAI	USDTZAR	USTBTC	USTBUSD	USTCBUSD
USTCFDUSD	USTCTRY	USTCUSDT	USTUSDT	USUALBTC	USUALFDUSD
USUALTRY	USUALUSDC	USUALUSDT	UTKBTC	UTKBUSD	UTKUSDC
UTKUSDT	VANABNB	VANAFDUSD	VANATRY	VANAUSDC	VANAUSDT
VANRYBTC	VANRYTRY	VANRYUSDC	VANRYUSDT	VELODROMEUSDC	VELODROMEUSDT
VENBNB	VENBTC	VENETH	VENUSDT	VETBNB	VETBTC
VETBUSD	VETETH	VETEUR	VETGBP	VETTRY	VETUSDC
VETUSDT	VGXBTC	VGXETH	VGXUSDT	VIABNB	VIABTC
VIAETH	VIBBTC	VIBBUSD	VIBEBTC	VIBEETH	VIBETH
VIBUSDT	VICBTC	VICTRY	VICUSDT	VIDTBTC	VIDTBUSD
VIDTUSDT	VIRTUALTRY	VIRTUALUSDC	VIRTUALUSDT	VITEBNB	VITEBTC
VITEBUSD	VITEUSDT	VOXELBNB	VOXELBTC	VOXELBUSD	VOXELETH
VOXELUSDT	VTHOBNB	VTHOBUSD	VTHOTRY	VTHOUSDT	WABIBNB
WABIBTC	WABIETH	WALBNB	WALFDUSD	WALTRY	WALUSDC
WALUSDT	WANBNB	WANBTC	WANETH	WANUSDT	WAVESBNB
WAVESBTC	WAVESBUSD	WAVESETH	WAVESEUR	WAVESPAX	WAVESRUB
WAVESTRY	WAVESTUSD	WAVESUSDC	WAVESUSDT	WAXPBNB	WAXPBTC
WAXPBUSD	WAXPUSDT	WBETHETH	WBETHUSDT	WBTC	WBTCBTC
WBTCBUSD	WBTCETH	WBTCUSDT	WCTBNB	WCTFDUSD	WCTTRY
WCTUSDC	WCTUSDT	WFDUSD	WIFBRL	WIFBTC	WIFEUR
WIFFDUSD	WIFTRY	WIFUSDC	WIFUSDT	WINBNB	WINBRL
WINBTC	WINBUSD	WINEUR	WINGBNB	WINGBTC	WINGBUSD
WINGETH	WINGSBTC	WINGSETH	WINGUSDT	WINTRX	WINUSDC
WINUSDT	WLDBTC	WLDEUR	WLDFDUSD	WLDRUB	WLDTRY
WLDUSDC	WLDUSDT	WLFIBRL	WLFIEUR	WLFITRY	WLFIUSDC
WLFIUSDT	WNXMBNB	WNXMBTC	WNXMBUSD	WNXMUSDT	WOOBNB
WOOBTC	WOOBUSD	WOOUSDT	WPRBTC	WPRETH	WRXBNB
WRXBTC	WRXBUSD	WRXEUR	WRXUSDT	WTCBNB	WTCBTC

WTCETH	WTCUSDT	WTRY	WUSDC	WUSDT	XAIBNB
XAIBTC	XAIFDUSD	XAITRY	XAITUSD	XAIUSDC	XAIUSDT
XECBUSD	XECTRY	XECUSDT	XEMBNB	XEMBTC	XEMBUSD
XEMETH	XEMUSDT	XLMBNB	XLMBTC	XLMBUSD	XLMETH
XLMEUR	XLMFDUSD	XLMJPY	XLMPAX	XLMTRY	XLMTUSD
XLMUSDC	XLMUSDT	XMRBNB	XMRBTC	XMRBUSD	XMRETH
XMRUSDT	XNOBTC	XNOBUSD	XNOETH	XNOUSDT	XPLBNB
XPLFDUSD	XPLTRY	XPLUSDC	XPLUSDT	XRPAUD	XRPBEARBUSD
XRPBEARUSDT	XRPBIDR	XRPBKRW	XRPBNB	XRPBRL	XRPBTC
XRPBULLBUSD	XRPBULLUSDT	XRPBUSD	XRPETH	XRPEUR	XRPFDUSD
XRPGBP	XRPJPY	XRPMXN	XRPNGN	XRPPAX	XRPRUB
XRPTRY	XRPTUSD	XRPUSDC	XRPUSDT	XTZBNB	XTZBTC
XTZBUSD	XTZETH	XTZTRY	XTZUSDC	XTZUSDT	XUSDUSDT
XVGBTC	XVGBUSD	XVGETH	XVGTRY	XVGTUSD	XVGUSDT
XVSBNB	XVSBTC	XVSBUSD	XVSTRY	XVSUSDT	XZCBNB
XZCBTC	XZCETH	XZCUSDT	XZCXRP	YBBNB	YBFDUSD
YBTRY	YBUSDC	YBUSDT	YFIBNB	YFIBTC	YFIBUSD
YFIEUR	YFIIBNB	YFIIBTC	YFIIBUSD	YFIIUSDT	YFIUSDT
YGGBNB	YGGBTC	YGGBUSD	YGGTRY	YGGUSDC	YGGUSDT
YOYOBNB	YOYOBTC	YOYOETH	ZBTBNB	ZBTFDUSD	ZBTTRY
ZBTUSDC	ZBTUSDT	ZECBNB	ZECBTC	ZECBUSD	ZECETH
ZECPAX	ZECTUSD	ZECUSDC	ZECUSDT	ZENBNB	ZENBTC
ZENBUSD	ZENETH	ZENUSDC	ZENUSDT	ZILBIDR	ZILBNB
ZILBTC	ZILBUSD	ZILETH	ZILEUR	ZILTRY	ZILUSDT
ZKBTC	ZKCBNB	ZKCFDUSD	ZKCTRY	ZKCUSDC	ZKCUSDT
ZKFDUSD	ZKTRY	ZKUSDC	ZKUSDT	ZROBTC	ZROFDUSD
ZROTRY	ZROUSDC	ZROUSDT	ZRXBNB	ZRXBTC	ZRXBUSD
ZRXETH	ZRXUSDT				

Example Applications

The Binance Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Binance exchange:

```
from AlgorithmImports import * class CoinAPIDataAlgorithm(QCAlgorithm): def initialize(self) -> None: self.set_start_da
te(2024, 9, 1) self.set_end_date(2024, 12, 31) # Set the account currency to USDT, since you can't trade with # USD and
the algorithm doesn't automatically convert USD & USDT # holdings. self.set_account_currency("USDT", 100000) self.unive
rse_settings.asynchronous = True # Binance accepts both Cash and Margin account types. Select the # one you need for th
e best reality modeling. self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN) # Add BTCUSDT data. self._
btc = self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BINANCE) # Historical data history = self.history(self._btc.
symbol, 30, Resolution.DAILY) # Add a Crypto universe that selects trading pairs on Binance. universe = self.add_univer
se(CryptoUniverse.binance(self._select_assets)) # Historical Universe data universe_history = self.history(universe, 30
, Resolution.DAILY) for (universe_symbol, time), universe_day in universe_history.items(): for universe_item in univers
e_day: symbol = universe_item.symbol price = universe_item.close def _select_assets(self, universe_day): # Filter for m
aterially traded Crypto pairs with significant # size and dollar volume, assuming higher capital flow in for # higher r
eturn. return [universe_item.symbol for universe_item in universe_day if universe_item.volume >= 100 and universe_item.
volume_in_usd > 10_000_000] def on_data(self, slice: Slice) -> None: # Speculate-invest all available free cash on BTCU
SDT, repecting # the order quantity restrictions to avoid invalid orders. if self.portfolio.cash_book['BTC'].amount !=
0: return # Define a buffer so that we can scale down the order size, # avoiding trading errors. buffer = max( BinanceF
eeModel.TAKER_TIER_1_FEE, self.settings.free_portfolio_value_percentage ) quantity = ( self.portfolio.cash_book['USDT']
.amount / self._btc.ask_price * (1-buffer) ) # Check if the quantity exceeds the minimum order size. On # Binance, the
minimum order size is measured in the quote # currency (USDT in this case). minimum_order_size = ( self._btc.symbol_pro
perties.minimum_order_size / self._btc.ask_price ) if abs(quantity) < minimum_order_size: return self.market_order(self
._btc, quantity)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Binance exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

PY

```
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to Tether, since USD and USDT will not auto-convert and USD cannot be used to trade
        self.set_account_currency("USDT", 100000)
        self.settings.free_portfolio_value_percentage = 0.05

        # Binance accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)

        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        self.universe_settings.resolution = Resolution.MINUTE
        # Add Crypto Universe Selection that select crypto pairs in Binance exchange
        self.add_universe(CryptoUniverse.binance(self.universe_selection_filter))
        # Get the pairs on Binance that have USDT as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.BINANCE)
            if x.value.quote_currency == self.account_currency
        ]

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(minutes = 20), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection_filter(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USDT.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the Binance exchange:

```py
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.BINANCE).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.binance(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Binance Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

CoinAPI

Binance US Crypto Price Data

Introduction

The Binance US Crypto Price Data by CoinAPI is for Cryptocurrency price and volume data points. The data covers 604 Cryptocurrency pairs, starts in October 2019, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Binance US.

For more information about the Binance US Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippets demonstrates how to set the brokerage model, request data, and perform universe selection with the Binance US dataset:

```
# Binance US only accepts cash accounts
self.set_brokerage_model(BrokerageName.BINANCE_US, AccountType.CASH)

self.btcbusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BINANCE_US).symbol

self._universe = AddUniverse(CryptoUniverse.binance_us(self.universe_selection_filter));
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	October 2019
Asset Coverage	604 Cryptocurrency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Binance US Crypto Price data to your algorithm, call the `add_crypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        # BinanceUS accepts Cash account type only, AccountType.MARGIN will result in an exception.
        self.set_brokerage_model(BrokerageName.BINANCE_US, AccountType.CASH)

        self.btcbusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BINANCE_US).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Binance US Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcbusd in slice.bars:
        trade_bar = slice.bars[self.btcbusd]
        self.log(f"{self.btcbusd} close at {slice.time}: {trade_bar.close}")

    if self.btcbusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcbusd]
        self.log(f"{self.btcbusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcbusd in slice.ticks:
        ticks = slice.ticks[self.btcbusd]
        for tick in ticks:
            self.log(f"{self.btcbusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Binance US Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcbusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcbusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcbusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcbusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Binance US Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```
PY
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.set_brokerage_model(BrokerageName.BINANCE_US, AccountType.CASH)
    self._universe = self.add_universe(CryptoUniverse.binance_us(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcbusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (604)					
1000MOGUSDT	1000REKTUSDT	1INCHUSD	1INCHUSD4	1INCHUSDT	A2ZUSDT
AAVEUSD	AAVEUSD4	AAVEUSDT	ACHUSD	ACHUSD4	ACHUSDT
ADABTC	ADABUSD	ADAETH	ADAUSD	ADAUSD4	ADAUSDC
ADAUSDT	ADXUSDT	AIXBTUSDT	ALGOBUSD	ALGOUSD	ALGOUSD4
ALGOUSDT	ALICEUSD	ALICEUSD4	ALICEUSDT	ALPINEUSD	ALPINEUSD4
ALPINEUSDT	AMPUSD	ANIMEUSDT	ANKRUSD	ANKRUSD4	ANKRUSDT
ANTUSD	ANTUSD4	ANTUSDT	APEUSD	APEUSD4	APEUSDT
API3USD	API3USD4	API3USDT	APTUSD	APTUSD4	APTUSDT
ARBUSD	ARBUSDT	ASTERUSDT	ASTRUSD	ASTRUSD4	ASTRUSDT

ATOMBTC	ATOMUSD	ATOMUSD4	ATOMUSDT	AUDIOUSD	AUDIOUSD4
AUDIOUSDT	AUSDT	AVAXBTC	AVAXUSD	AVAXUSD4	AVAXUSDT
AXLUSD	AXLUSD4	AXLUSDT	AXSUSD	AXSUSD4	AXSUSDT
BALUSD	BALUSD4	BALUSDT	BANDUSD	BANDUSD4	BANDUSDT
BATUSD	BATUSD4	BATUSDT	BCHBTC	BCHUSD	BCHUSD4
BCHUSDT	BICOUSD	BICOUSD4	BICOUSDT	BLURUSD	BLURUSDT
BNBBTC	BNBBUSD	BNBUSD	BNBUSD4	BNBUSDT	BNTUSD
BNTUSD4	BNTUSDT	BONDUSD	BONDUSD4	BONDUSDT	BONKUSD
BONKUSDT	BOSONUSD	BOSONUSD4	BOSONUSDT	BRETTUSDT	BTCBUSD
BTCDAI	BTCUSD	BTCUSD4	BTCUSDC	BTCUSDT	BTCUST
BTRSTUSD	BTRSTUSD4	BTRSTUSDT	BUSDUSD	BUSDUSD4	BUSDUSDT
CELOUSD	CELOUSD4	CELOUSDT	CELRUSD	CELRUSD4	CELRUSDT
CHZUSD	CHZUSD4	CHZUSDT	CLVUSD	CLVUSD4	CLVUSDT
COMPUSD	COMPUSD4	COMPUSDT	COTIUSD	COTIUSD4	COTIUSDT
CRVUSD	CRVUSD4	CRVUSDT	CTSIUSD	CTSIUSD4	CTSIUSDT
CUDOSUSDT	DAIUSD	DAIUSD4	DAIUSDT	DARUSD	DARUSD4
DARUSDT	DASHUSD	DASHUSD4	DASHUSDT	DATAUSDT	DGBUSD
DGBUSD4	DGBUSDT	DIAUSD	DIAUSD4	DIAUSDT	DOGEBTC
DOGEUSD	DOGEUSD4	DOGEUSDT	DOODUSDT	DOTBTC	DOTUSD
DOTUSD4	DOTUSDT	DUSDT	EGLDUSD	EGLDUSD4	EGLDUSDT
EIGENUSD	EIGENUSDT	ENAUSDT	ENJUSD	ENJUSD4	ENJUSDT
ENSUSD	ENSUSD4	ENSUSDT	EOSBUSD	EOSUSD	EOSUSD4
EOSUSDT	ETCUSD	ETCUSD4	ETCUSDT	ETHBTC	ETHBUSD
ETHDAI	ETHUSD	ETHUSD4	ETHUSDC	ETHUSDT	FARTCOINUSDT
FETUSD	FETUSD4	FETUSDT	FILUSD	FILUSD4	FILUSDT
FLOKI8USD	FLOKI8USDT	FLOKIUSD	FLOKIUSDT	FLOWUSD	FLOWUSD4
FLOWUSDT	FLUXUSD	FLUXUSD4	FLUXUSDT	FORTHUSD	FORTHUSD4
FORTHUSDT	FORTUSDT	FTMUSD	FTMUSD4	FTMUSDT	GALAUSD
GALAUSD4	GALAUSDT	GALUSD	GALUSD4	GALUSDT	GIGAUSDT
GLMUSD	GLMUSD4	GLMUSDT	GRTUSD	GRTUSD4	GRTUSDT
GTCUSD	GTCUSD4	GTCUSDT	GUSDT	HBARBUSD	HBARUSD
HBARUSD4	HBARUSDT	HNTUSD	HNTUSD4	HNTUSDT	HYPEUSD
HYPEUSDT	ICPUSD	ICPUSD4	ICPUSDT	ICXUSD	ICXUSD4

ICXUSDT	ILVUSD	ILVUSD4	ILVUSDT	IMXUSD	IMXUSD4
IMXUSDT	IOSTUSD	IOSTUSDT	IOTAUSD	IOTAUSD4	IOTAUSDT
IOTXUSDT	JAMUSD	JAMUSD4	JAMUSDT	JASMYUSD	JASMYUSD4
JASMYUSDT	JTOUSDT	JUPUSD	JUPUSDT	KAITOUSDT	KAVAUSD
KAVAUSD4	KAVAUSDT	KDAUSD	KDAUSD4	KDAUSDT	KNCUSD
KNCUSD4	KNCUSDT	KSHIBUSD	KSHIBUSD4	KSMUSD	KSMUSD4
KSMUSDT	LAYERUSDT	LAZIOUSD	LAZIOUSD4	LAZIOUSDT	LDOUSD
LDOUSD4	LDOUSDT	LINKBTC	LINKUSD	LINKUSD4	LINKUSDT
LOKAUSD	LOKAUSD4	LOKAUSDT	LOOMUSD	LOOMUSD4	LOOMUSDT
LPTBUSD	LPTUSD	LPTUSD4	LPTUSDT	LRCBTC	LRCUSD
LRCUSD4	LRCUSDT	LSKUSD	LSKUSD4	LSKUSDT	LTCBTC
LTCUSD	LTCUSD4	LTCUSDT	LTOUSD	LTOUSD4	LTOUSDT
MAGICUSDT	MANABTC	MANABUSD	MANAUSD	MANAUSD4	MANAUSDT
MASKUSD	MASKUSD4	MASKUSDT	MATICBTC	MATICBUSD	MATICETH
MATICUSD	MATICUSD4	MATICUSDT	METISUSD	METISUSDT	MEUSD
MEUSDT	MEWUSDT	MKRUSD	MKRUSD4	MKRUSDT	MOODENGUSDT
MXCUSD	MXCUSD4	MXCUSDT	NANOUSD	NEARBUSD	NEARUSD
NEARUSD4	NEARUSDT	NEIROUSDT	NEOUSD	NEOUSD4	NEOUSDT
NMRUSD	NMRUSD4	NMRUSDT	NOBODYUSDT	OCEANUSD	OCEANUSD4
OCEANUSDT	OGNUSD	OGNUSD4	OGNUSDT	OMGBUSD	OMGUSD
OMGUSD4	OMGUSDT	ONDOUSDT	ONEBUSD	ONEUSD	ONEUSD4
ONEUSDT	ONGUSDT	ONTUSD	ONTUSD4	ONTUSDT	OPUSD
OPUSD4	OPUSDT	ORBSUSDT	ORCAUSDT	OXTUSD	OXTUSD4
OXTUSDT	PAXGUSD	PAXGUSD4	PAXGUSDT	PENGUUSDT	PEPEUSD
PEPEUSDT	PNUTUSDT	POLBTC	POLETH	POLUSD	POLUSDT
POLYBTC	POLYBUSD	POLYUSD	POLYUSDT	POLYXUSD	POLYXUSD4
POLYXUSDT	PONDUSD	PONDUSD4	PONDUSDT	POPCATUSDT	PORTOUSD
PORTOUSD4	PORTOUSDT	PROMUSD	PROMUSD4	PROMUSDT	PUMPUSDT
QNTUSD	QNTUSD4	QNTUSDT	QTUMUSD	QTUMUSD4	QTUMUSDT
RADUSD	RADUSD4	RADUSDT	RAREUSD	RAREUSD4	RAREUSDT
REEFUSD	REEFUSD4	REEFUSDT	RENDERUSD	RENDERUSDT	RENUSD
RENUSD4	RENUSDT	REPBUSD	REPUSD	REQUSD	REQUSD4
REQUSDT	RLCUSD	RLCUSD4	RLCUSDT	RNDRUSD	RNDRUSD4

RNDRUSDT	ROSEUSD	ROSEUSD4	ROSEUSDT	RVNUSD	RVNUSD4
RVNUSDT	SANDUSD	SANDUSD4	SANDUSDT	SANTOSUSD	SANTOSUSD4
SANTOSUSDT	SHIBBUSD	SHIBUSD	SHIBUSDT	SKLUSD	SKLUSD4
SKLUSDT	SKYUSDT	SLPUSD	SLPUSD4	SLPUSDT	SNXUSD
SNXUSD4	SNXUSDT	SOLBTC	SOLBUSD	SOLETH	SOLUSD
SOLUSD4	SOLUSDC	SOLUSDT	SPELLUSD	SPELLUSD4	SPELLUSDT
SPXUSDT	SRMUSD	SRMUSDT	STGUSD	STGUSD4	STGUSDT
STMXUSD	STMXUSDT	STORJUSD	STORJUSD4	STORJUSDT	SUIUSD
SUIUSDT	SUSD	SUSDT	SUSHIUSD	SUSHIUSD4	SUSHIUSDT
SYSUSD	SYSUSD4	SYSUSDT	TFUELUSD	TFUELUSD4	TFUELUSDT
THETAUSD	THETAUSD4	THETAUSDT	TLMUSD	TLMUSD4	TLMUSDT
TOSHIUSDT	TRACUSD	TRACUSDT	TRUMPUSD	TRUMPUSDT	TRXBTC
TRXBUSD	TRXUSD	TRXUSD4	TRXUSDT	TURBOUSDT	TUSD
TUSD4	TUSDT	TUSDUSD	TUSDUSD4	TUSDUSDT	UNIBTC
UNIUSD	UNIUSD4	UNIUSDT	USDCBUSD	USDCUSD	USDCUSD4
USDCUSDT	USDTUSD	USDTUSD4	USELESSUSDT	USTUSD	USTUSDT
VETBTC	VETUSD	VETUSD4	VETUSDT	VIRTUALUSDT	VITEUSD
VITEUSD4	VITEUSDT	VOXELUSD	VOXELUSD4	VOXELUSDT	VTHOUSD
VTHOUSD4	VTHOUSDT	WAVESUSD	WAVESUSD4	WAVESUSDT	WAXPUSD
WAXPUSD4	WAXPUSDT	WBTCBTC	WIFUSDT	WLDUSDT	XDCUSDT
XECUSD	XECUSDT	XLMUSD	XLMUSD4	XLMUSDT	XNOUSD
XNOUSD4	XNOUSDT	XRPBTC	XRPBUSD	XRPUSD	XRPUSD4
XRPUSDT	XTZBTC	XTZBUSD	XTZUSD	XTZUSD4	XTZUSDT
YFIUSD	YFIUSD4	YFIUSDT	ZECUSD	ZECUSD4	ZECUSDT
ZENUSD	ZENUSD4	ZENUSDT	ZILBUSD	ZILUSD	ZILUSD4
ZILUSDT	ZRXUSD	ZRXUSD4	ZRXUSDT		

Example Applications

The Binance US Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Binance US exchange:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.universe_settings.asynchronous = True
        # Set USDT as the account currency so we can trade BTCUSDT.
        self.set_account_currency('USDT', 100_000)
        # Binance US accepts Cash account type only. AccountType.MARGIN
        # results in an exception.
        self.set_brokerage_model(BrokerageName.BINANCE_US, AccountType.CASH)
        # Add BTCUSDT data.
        self._btc = self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BINANCE_US)
        # Historical data
        history = self.history(self._btc.symbol, 30, Resolution.DAILY)
        # Add a Crypto universe that selects Crypto pairs on Binance US.
        universe = self.add_universe(CryptoUniverse.binance_us(self._select_assets))
        # Historical Universe data
        universe_history = self.history(universe, 30, Resolution.DAILY)
        for (universe_symbol, time), universe_day in universe_history.items():
            for universe_item in universe_day:
                symbol = universe_item.symbol
                price = universe_item.close

    def _select_assets(self, universe_day):
        # Filter for materially traded Crypto pairs with significant
        # size and dollar volume, assuming higher capital flow in for
        # higher return.
        return [universe_item.symbol for universe_item in universe_day
                if universe_item.volume >= 100
                and universe_item.volume_in_usd > 10_000_000]

    def on_data(self, slice: Slice) -> None:
        # Speculate-invest all available free cash on BTCUSDT, repecting
        # the order quantity restrictions to avoid invalid orders.
        if self.portfolio.cash_book['BTC'].amount != 0:
            return
        # Define a buffer so that we can scale down the order size,
        # avoiding trading errors.
        buffer = max(
            BinanceFeeModel.TAKER_TIER_1_FEE,
            self.settings.free_portfolio_value_percentage
        )
        quantity = (
            self.portfolio.cash_book['USDT'].amount
            / self._btc.ask_price
            * (1-buffer)
        )
        # Check if the quantity exceeds the minimum order size. On
        # Binance US, the minimum order size is measured in the quote
        # currency (USDT in this case).
        minimum_order_size = (
            self._btc.symbol_properties.minimum_order_size
            / self._btc.ask_price
        )
        if abs(quantity) < minimum_order_size:
            return
        self.market_order(self._btc, quantity)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Binance US exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)
        self.settings.free_portfolio_value_percentage = 0.05
        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        # Add Crypto Universe Selection that select crypto pairs in BinanceUS exchange
        self.add_universe(CryptoUniverse.binance_us(self._select_assets))
        # Get the pairs on Binance US that have USD as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.BINANCE_US)
            if x.value.quote_currency == self.account_currency
        ]
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(minutes=20), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def _select_assets(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USD.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request")
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the BinanceUS exchange:

```py
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.BINANCE_US).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.binance_us(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Binance US Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

CoinAPI

Bitfinex Crypto Price Data

Introduction

The Bitfinex Crypto Price Data by CoinAPI is for Cryptocurrency price and volume data points. The data covers 570 Cryptocurrency pairs, starts in January 2013, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Bitfinex.

For more information about the Bitfinex Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Bitfinex Crypto Price dataset:

```py
# Bitfinex accepts both Cash and Margin type account.
self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.CASH)
#self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)

self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BITFINEX).symbol

self._universe = self.add_universe(CryptoUniverse.bitfinex(self.universe_selection_filter))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2013
Asset Coverage	570 Currency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Bitfinex Crypto Price data to your algorithm, call the `add_crypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        # Bitfinex accepts both Cash and Margin type account.
        self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)

        self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BITFINEX).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Bitfinex Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcusd in slice.bars:
        trade_bar = slice.bars[self.btcusd]
        self.log(f"{self.btcusd} close at {slice.time}: {trade_bar.close}")

    if self.btcusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcusd]
        self.log(f"{self.btcusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcusd in slice.ticks:
        ticks = slice.ticks[self.btcusd]
        for tick in ticks:
            self.log(f"{self.btcusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Bitfinex Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Bitfinex Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```
PY
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)
    self._universe = self.add_universe(CryptoUniverse.bitfinex(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (570)					
1INCHUSD	1INCHUSDT	AAABBB	AAVEUSD	AAVEUSDT	ADABTC
ADAUSD	ADAUSDT	AIOZUSD	AIXUSD	AIXUSDT	ALBTUSD
ALBTUSDT	ALGOBTC	ALGOUSD	ALGOUSDT	ALT11M2507USD	ALT11M2507USDT
ALT11M250830USD	ALT11M250830USDT	ALT11M251029USD	ALT11M251029USDT	ALT2612USD	ALT2612USDT
AMPLBTC	AMPLUSD	AMPLUSDT	ANCUSD	ANCUSDT	ANTBTC
ANTETH	ANTUSD	APENFTUSD	APEUSD	APEUSDT	APTUSD
APTUSDT	ARBUSD	ARBUSDT	ATHUSD	ATHUSDT	ATOMBTC
ATOMETH	ATOMUSD	ATOMUSDT	AUSDTUSD	AUSDTUSDT	AVAXBTC

AVAXUSD	AVAXUSDT	AXSUSD	AXSUSDT	AZEROUSD	AZEROUSDT
B21USD	B21USDT	B2MUSD	B2MUSDT	BALUSD	BALUSDT
BANDUSD	BANDUSDT	BATUSD	BATUSDT	BCHABCUSD	BCHNUSD
BESTUSD	BFTUSD	BGBUSD	BGBUSDT	BLASTUSD	BLASTUSDT
BMIUSD	BMIUSDT	BNTUSD	BOBAUSD	BOBAUSDT	BONKUSD
BONKUSDT	BOSONUSD	BOSONUSDT	BSVBTC	BSVUSD	BTCCNHT
BTCEUR	BTCEURQ	BTCEURR	BTCEURT	BTCGBP	BTCJPY
BTCMIM	BTCTRY	BTCUSD	BTCUSDQ	BTCUSDR	BTCUSDT
BTCXAUT	BTCXCHF	BTGBTC	BTGUSD	BTSEUSD	BTTUSD
BXNUSD	BXNUSDT	CCDBTC	CCDUSD	CCDUSDT	CELOUSD
CELOUSDT	CELUSD	CELUSDT	CHEXUSD	CHSBBTC	CHSBUSD
CHSBUSDT	CHZUSD	CHZUSDT	CLOUSD	CNHCNHT	CNHTUSD
COMPUSD	COMPUSDT	CRVUSD	CRVUSDT	CTKUSD	CTKUSDT
DAIBTC	DAIETH	DAIUSD	DASHBTC	DASHUSD	DATAUSD
DCRUSD	DGBUSD	DOGEBTC	DOGEUSD	DOGEUSDT	DOPUSD
DOPUSDT	DORAUSD	DORAUSDT	DOTBTC	DOTUSD	DOTUSDT
DUSKBTC	DUSKUSD	DVFUSD	DYMUSD	DYMUSDT	EDOUSD
EGLDUSD	EGLDUSDT	EIGENUSD	EIGENUSDT	ENAUSD	ENAUSDT
ENJUSD	EOSBTC	EOSETH	EOSEUR	EOSGBP	EOSJPY
EOSUSD	EOSUSDT	ESSUSD	ETCBTC	ETCUSD	ETCUSDT
ETH2ETH	ETH2USD	ETH2USDT	ETHBTC	ETHEUR	ETHEURT
ETHGBP	ETHJPY	ETHUSD	ETHUSDT	ETHWUSD	ETHWUSDT
ETHXAUT	ETPBTC	ETPUSD	EURQEUR	EURQUSD	EURQUSDT
EURREUR	EURRUSD	EURRUSDT	EURSUSD	EURTEUR	EURTUSD
EURTUSDT	EURUSDT	EXOUSD	FCLUSD	FCLUSDT	FETUSD
FETUSDT	FILUSD	FILUSDT	FLOKIUSD	FLOKIUSDT	FORTHUSD
FORTHUSDT	FTMUSD	FTMUSDT	FTTUSD	FTTUSDT	FUNUSD
GALAUSD	GALAUSDT	GBPEURT	GBPUSDT	GLMUSD	GMMTUSD
GMMTUSDT	GNOUSD	GOMININGUSD	GOMININGUSDT	GOTEUR	GOTUSD
GRTUSD	GRTUSDT	GTXUSD	GTXUSDT	GUNUSD	GUNUSDT
HILSVUSD	HIXUSD	HIXUSDT	HMSTRUSD	HMSTRUSDT	HMTUSD
HMTUSDT	HTXDAOUSD	HTXDAOUSDT	HTXUSD	HTXUSDT	ICEUSD
ICPBTC	ICPUSD	ICPUSDT	IDUSD	IDUSDT	IOTABTC

IOTAETH	IOTAEUR	IOTAGBP	IOTAJPY	IOTAUSD	IQXUSD
IQXUSDT	JASMYUSD	JASMYUSDT	JPYUSDT	JSTBTC	JSTUSD
JSTUSDT	JUPUSD	JUPUSDT	JUSTICEUSD	JUSTICEUSDT	JXXUSD
JXXUSDT	KAIAUSD	KAIAUSDT	KAIUSD	KAIUSDT	KANUSD
KANUSDT	KARATEUSD	KARATEUSDT	KAVAUSD	KAVAUSDT	KMNOUSD
KMNOUSDT	KNCBTC	KNCUSD	KSMUSD	KSMUSDT	LDOUSD
LDOUSDT	LEOBTC	LEOEOS	LEOETH	LEOUSD	LEOUSDT
LIFIIIUSD	LIFIIIUSDT	LINKUSD	LINKUSDT	LRCUSD	LTCBTC
LTCUSD	LTCUSDT	LUNABTC	LUNAETH	LUNAUSD	LUNAUSDT
LYMUSD	MANABTC	MANAUSD	MANEKIUSD	MANEKIUSDT	MATICBTC
MATICUSD	MATICUSDT	MDOGBTC	MDOGUSD	MDOGUSDT	MEWUSD
MEWUSDT	MIMUSD	MIMUSDT	MIRUSD	MIRUSDT	MKRUSD
MKRUSDT	MLNUSD	MOBUSD	MOBUSDT	MPCUSD	MPCUSDT
MXNTUSD	NEARUSD	NEARUSDT	NEOBTC	NEOETH	NEOEUR
NEOGBP	NEOJPY	NEOUSD	NEOUSDT	NEXOBTC	NEXOUSD
NEXOUSDT	NOTUSD	NOTUSDT	NYMUSD	NYMUSDT	OCEANUSD
OCEANUSDT	ODEUSD	OMGBTC	OMGETH	OMGUSD	OMNIUSD
OPXUSD	OPXUSDT	ORSUSD	OXYUSD	OXYUSDT	PASSUSD
PAXUSD	PAXUSDT	PEPEUSD	PEPEUSDT	PLANETSUSD	PLANETSUSDT
PLUUSD	PNGUSD	PNGUSDT	PNKETH	PNKUSD	POLUSD
POLUSDT	QRDOUSD	QRDOUSDT	QSHUSD	QTFBTC	QTFUSD
QTUMBTC	QTUMUSD	RBTCBTC	RBTCUSD	REEFUSD	REEFUSDT
REP2BTC	REP2USD	REQUSD	ROSEUSD	ROSEUSDT	RRTUSD
SANBTC	SANDUSD	SANDUSDT	SANETH	SANUSD	SCROLLUSD
SCROLLUSDT	SEIUSD	SEIUSDT	SGBUSD	SGBUSDT	SHIBUSD
SHIBUSDT	SHMUSD	SHMUSDT	SNTUSD	SNXUSD	SNXUSDT
SOLBTC	SOLUSD	SOLUSDT	SONICUSD	SONICUSDT	SPECUSD
SPECUSDT	SPELLMIM	SPELLUSD	SPELLUSDT	SRMUSD	SRMUSDT
STGUSD	STGUSDT	STORJUSD	STRKUSD	STRKUSDT	STXUSD
STXUSDT	SUIUSD	SUIUSDT	SUKUUSD	SUKUUSDT	SUNUSD
SUNUSDT	SUSHIUSD	SUSHIUSDT	SWEATUSD	SWEATUSDT	SXXUSD
SXXUSDT	TERRAUSTUSD	TERRAUSTUSDT	TESTADATESTUSD	TESTALGOTESTUSD	TESTAPTTESTUSD
TESTAVAXTESTUSD	TESTBTCTESTUSD	TESTBTCTESTUSDT	TESTDOGETESTUSD	TESTDOTTESTUSD	TESTEOSTESTUSD

TESTETHTESTUSD	TESTFILTESTUSD	TESTLTCTESTUSD	TESTMATICTESTUSD	TESTMATICTESTUSDT	TESTNEARTESTUSD
TESTSOLTESTUSD	TESTXAUTTESTUSD	TESTXTZTESTUSD	THETAUSD	THETAUSDT	TIAUSD
TIAUSDT	TITAN1GBP	TITAN1USD	TITAN2GBP	TITAN2USD	TLOSUSD
TLOSUSDT	TOKENUSD	TOKENUSDT	TOMIUSD	TOMIUSDT	TONUSD
TONUSDT	TRADEUSD	TRADEUSDT	TRXBTC	TRXETH	TRXEUR
TRXUSD	TRXUSDT	TRYUSDT	TURBOUSD	TURBOUSDT	TUSDUSD
TUSDUSDT	UNIUSD	UNIUSDT	UOPUSD	UOPUSDT	UOSBTC
UOSUSD	USDCUSD	USDCUSDT	USDFUSD	USDFUSDT	USDQUSD
USDQUSDT	USDRUSD	USDRUSDT	USDTCNHT	USDTMXNT	USDTUSD
USTBLUSD	USTBLUSDT	UTKUSD	UXLINKUSD	UXLINKUSDT	VEEUSD
VELARUSD	VELARUSDT	VELOUSD	VELOUSDT	VETBTC	VETUSD
VETUSDT	VSYBTC	VSYUSD	WAVESUSD	WAVESUSDT	WAXUSD
WBTCBTC	WBTCUSD	WHBTUSD	WHBTUSDT	WIFUSD	WIFUSDT
WILDUSD	WILDUSDT	WNCGUSD	WNCGUSDT	WOOUSD	WOOUSDT
XAUTBTC	XAUTUSD	XAUTUSDT	XCHFUSD	XDCUSD	XDCUSDT
XLMBTC	XLMETH	XLMUSD	XLMUSDT	XMRBTC	XMRUSD
XMRUSDT	XRAUSD	XRDBTC	XRDUSD	XRPBTC	XRPUSD
XRPUSDT	XSNUSD	XTPUSD	XTPUSDT	XTZBTC	XTZUSD
XTZUSDT	XVGUSD	YFIUSD	YFIUSDT	ZCNUSD	ZECBTC
ZECUSD	ZILBTC	ZILUSD	ZKXUSD	ZKXUSDT	ZMTUSD
ZMTUSDT	ZROUSD	ZROUSDT	ZRXBTC	ZRXETH	ZRXUSD

Example Applications

The Bitfinex Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Bitfinex exchange:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.universe_settings.asynchronous = True
        self.set_cash(100_000)
        # Bitfinex accepts both Cash and Margin account types. Select the
        # one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)
        # Add BTCUSD data.
        self._btc = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BITFINEX)
        # Historical data
        history = self.history(self._btc.symbol, 30, Resolution.DAILY)
        # Add a Crypto universe that selects Crypto pairs on Bitfinex.
        universe = self.add_universe(CryptoUniverse.bitfinex(self._select_assets))
        # Historical Universe data
        universe_history = self.history(universe, 30, Resolution.DAILY)
        for (universe_symbol, time), universe_day in universe_history.items():
            for universe_item in universe_day:
                symbol = universe_item.symbol
                price = universe_item.close

    def _select_assets(self, universe_day):
        # Filter for materially traded Crypto pairs with significant
        # size and dollar volume, assuming higher capital flow in for
        # higher return.
        return [universe_item.symbol for universe_item in universe_day
                if universe_item.volume >= 100
                and universe_item.volume_in_usd > 10_000_000]

    def on_data(self, slice: Slice) -> None:
        # Speculate-invest all available free cash on BTCUSD, repecting
        # the order quantity restrictions to avoid invalid orders.
        if self.portfolio.cash_book['BTC'].amount != 0:
            return
        # Define a buffer so that we can scale down the order size,
        # avoiding trading errors.
        buffer = max(
            BitfinexFeeModel.TAKER_FEE,
            self.settings.free_portfolio_value_percentage
        )
        quantity = (
            self.portfolio.cash_book['USD'].amount
            / self._btc.ask_price
            * (1-buffer)
        )
        # Check if the quantity exceeds the minimum order size. On
        # Bitfinex, the minimum order size is measured in the base
        # currency (BTC in this case).
        if abs(quantity) < self._btc.symbol_properties.minimum_order_size:
            return
        self.market_order(self._btc, quantity)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Bitfinex exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

```
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.free_portfolio_value_percentage = 0.05
        # Bitfinex accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)

        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        self.universe_settings.resolution = Resolution.MINUTE
        # Add Crypto Universe Selection that select crypto pairs in Bitfinex exchange
        self.add_universe(CryptoUniverse.bitfinex(self._select_assets))
        # Get the pairs on Bitfinex that have USD as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.BITFINEX)
            if x.value.quote_currency == self.account_currency
        ]
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(hours=4), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def _select_assets(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USD.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request")
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the Bitfinex exchange:

```
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.BITFINEX).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.bitfinex(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Bitfinex Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

CoinAPI

Bybit Crypto Future Price Data

Introduction

The Bybit Crypto Future Price Data by CoinAPI is for Cryptocurrency Futures price and volume data points. The data covers 631 Cryptocurrency pairs, starts in August 2020, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Bybit.

The Bybit Crypto Future Margin Rate Data dataset provides margin interest rate data to model margin costs.

For more information about the Bybit Crypto Future Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Bybit Crypto Future Price dataset:

```
PY
def initialize(self) -> None:
    self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

    self.crypto_future_symbol = self.add_crypto_future("BTCUSDT", Resolution.MINUTE).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	October 2019
Asset Coverage	631 Crypto Futures Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Bybit Crypto Future Price data to your algorithm, call the `add_crypto_future` method. Save a reference to the Crypto Future `Symbol` so you can access the data later in your algorithm.

```
PY
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Tether
        self.set_account_currency("USDT", 100000)

        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        crypto_future = self.add_crypto_future("BTCUSDT", Resolution.MINUTE)
        self.btcusdt = crypto_future.symbol
```

For more information about creating Crypto Future subscriptions, see Requesting Data .

Accessing Data

To get the current Bybit Crypto Future Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto Future `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcusdt in slice.bars:
        trade_bar = slice.bars[self.btcusdt]
        self.log(f"{self.btcusdt} close at {slice.time}: {trade_bar.close}")

    if self.btcusdt in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcusdt]
        self.log(f"{self.btcusdt} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcusdt in slice.ticks:
        ticks = slice.ticks[self.btcusdt]
        for tick in ticks:
            self.log(f"{self.btcusdt} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto Future data, see Handling Data .

Historical Data

To get historical Bybit Crypto Future Price data, call the `history` method with the Crypto Future `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcusdt, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcusdt, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcusdt, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcusdt, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To unsubscribe from a Crypto Future contract that you added with the `add_crypto_future` method, call the `remove_security` method.

```
PY
self.remove_security(self.btcusdt)
```

The `remove_security` method cancels your open orders for the security and liquidates your Crypto Future holdings.

Supported Assets

The following table shows the available Crypto Future pairs:

Pairs Available (631)					
1CATUSDT	1INCHUSDT	A8USDT	AAVEUSD	AAVEUSDT	ACEUSDT
ACHUSDT	ACTUSDT	ACXUSDT	ADAUSD	ADAUSDT	AERGOUSDT
AEROUSDT	AEVOUSDT	AGIUSDT	AGIXUSDT	AGLDUSDT	AI16ZUSDT
AIDOGEUSDT	AIOZUSDT	AIUSDT	AIXBTUSDT	AKROUSDT	AKTUSDT
ALCHUSDT	ALEOUSDT	ALGOUSDT	ALICEUSDT	ALPACAUSDT	ALPHAUSDT
ALTUSDT	ALUUSDT	AMBUSDT	ANIMEUSDT	ANKRUSDT	ANTUSDT
APEUSDT	APEXUSDT	API3USDT	APTUSD	APTUSDT	APUUSDT
ARBUSDT	ARCUSDT	ARKMUSDT	ARKUSDT	ARPAUSDT	ARUSDT
ASTRUSDT	ATAUSDT	ATHUSDT	ATOMUSDT	AUCTIONUSDT	AUDIOUSDT
AVAAIUSDT	AVAILUSDT	AVAUSDT	AVAXUSD	AVAXUSDT	AVLUSDT
AXLUSDT	AXSUSDT	B3USDT	BABYDOGEUSDT	BABYUSDT	BADGERUSDT
BAKEUSDT	BALUSDT	BANANAS31USDT	BANANAUSDT	BANDUSDT	BANKUSDT
BANUSDT	BATUSDT	BBUSDT	BCHUSD	BCHUSDT	BEAMUSDT
BEERUSDT	BELUSDT	BENDOGUSDT	BERAUSDT	BICOUSDT	BIGTIMEUSDT
BILLYUSDT	BIOUSDT	BLASTUSDT	BLUEUSDT	BLURUSDT	BLZUSDT
BMTUSDT	BNBUSDT	BNTUSDT	BNXUSDT	BOBAUSDT	BOMEUSDT
BONDUSDT	BONKUSDT	BRETTUSDT	BROCCOLIUSDT	BRUSDT	BSVUSDT
BSWUSDT	BTCUSD	BTCUSDT	BTTUSDT	BUSDUSDT	BUZZUSDT
C98USDT	CAKEUSDT	CARVUSDT	CATIUSDT	CATSUSDT	CATUSDT
CEEKUSDT	CELOUSDT	CELRUSDT	CETUSUSDT	CFXUSDT	CGPTUSDT
CHEEMSUSDT	CHESSUSDT	CHILLGUYUSDT	CHRUSDT	CHZUSDT	CKBUSDT
CLANKERUSDT	CLOUDUSDT	COMBOUSDT	COMPUSDT	COOKIEUSDT	COOKUSDT
COQUSDT	COREUSDT	COSUSDT	COTIUSDT	COVALUSDT	COWUSDT
CPOOLUSDT	CROUSDT	CRVUSDT	CTCUSDT	CTKUSDT	CTSIUSDT
CVCUSDT	CVXUSDT	CYBERUSDT	DAOUSDT	DARKUSDT	DARUSDT
DASHUSDT	DATAUSDT	DBRUSDT	DEEPUSDT	DEGENUSDT	DENTUSDT
DEXEUSDT	DGBUSDT	DODOUSDT	DOGEUSD	DOGEUSDT	DOGSUSDT
DOGUSDT	DOODUSDT	DOP1USDT	DOTUSD	DOTUSDT	DRIFTUSDT
DUCKUSDT	DUSKUSDT	DYDXUSDT	DYMUSDT	EDUUSDT	EGLDUSDT
EIGENUSDT	ELONUSDT	ELXUSDT	ENAUSDT	ENJUSDT	ENSUSDT
EOSUSD	EOSUSDT	EPICUSDT	EPTUSDT	ETCUSD	ETCUSDT

ETHBTCUSDT	ETHFIUSDT	ETHUSD	ETHUSDT	ETHWUSDT	FARTCOINUSDT
FBUSDT	FDUSDUSDT	FETUSDT	FHEUSDT	FIDAUSDT	FILUSD
FILUSDT	FIOUSDT	FIREUSDT	FITFIUSDT	FLMUSDT	FLOCKUSDT
FLOKIUSDT	FLOWUSDT	FLRUSDT	FLUXUSDT	FORMUSDT	FORTHUSDT
FOXYUSDT	FRONTUSDT	FTMUSDT	FTNUSDT	FUELUSDT	FUNUSDT
FUSDT	FWOGUSDT	FXSUSDT	GALAUSDT	GALUSDT	GASUSDT
GEMSUSDT	GFTUSDT	GIGAUSDT	GLMRUSDT	GLMUSDT	GMEUSDT
GMTUSDT	GMXUSDT	GNOUSDT	GOATUSDT	GODSUSDT	GOMININGUSDT
GORKUSDT	GPSUSDT	GPTUSDT	GRASSUSDT	GRIFFAINUSDT	GRTUSDT
GTCUSDT	GUNUSDT	GUSDT	HAEDALUSDT	HBARUSDT	HEIUSDT
HFTUSDT	HIFIUSDT	HIGHUSDT	HIPPOUSDT	HIVEUSDT	HMSTRUSDT
HNTUSDT	HOOKUSDT	HOTUSDT	HPOS10IUSDT	HYPERUSDT	HYPEUSDT
ICPUSDT	ICXUSDT	IDEXUSDT	IDUSDT	ILVUSDT	IMXUSDT
INITUSDT	INJUSDT	IOSTUSDT	IOTAUSDT	IOTXUSDT	IOUSDT
IPUSDT	IQ50USDT	JASMYUSDT	JELLYJELLYUSDT	JOEUSDT	JSTUSDT
JTOUSDT	JUPUSDT	JUSDT	KAIAUSDT	KAITOUSDT	KASUSDT
KAVAUSDT	KDAUSDT	KERNELUSDT	KEYUSDT	KLAYUSDT	KMNOUSDT
KNCUSDT	KOMAUSDT	KSMUSDT	L3USDT	LADYSUSDT	LAIUSDT
LDOUSDT	LEVERUSDT	LINAUSDT	LINKUSD	LINKUSDT	LISTAUSDT
LITUSDT	LOOKSUSDT	LOOMUSDT	LPTUSDT	LQTYUSDT	LRCUSDT
LSKUSDT	LTCUSD	LTCUSDT	LTOUSDT	LUCEUSDT	LUMIAUSDT
LUNA2USDT	LUNCUSDT	MAGICUSDT	MAJORUSDT	MANAUSD	MANAUSDT
MANEKIUSDT	MANTAUSDT	MAPOUSDT	MASAUSDT	MASKUSDT	MATICUSDT
MAVIAUSDT	MAVUSDT	MAXUSDT	MBLUSDT	MBOXUSDT	MCUSDT
MDTUSDT	MELANIAUSDT	MEMEFIUSDT	MEMEUSDT	MERLUSDT	METISUSDT
MEUSDT	MEWUSDT	MICHIUSDT	MILKUSDT	MINAUSDT	MKRUSDT
MLNUSDT	MNTUSD	MNTUSDT	MOBILEUSDT	MOCAUSDT	MOGUSDT
MONUSDT	MOODENGUSDT	MORPHOUSDT	MOTHER1USDT	MOTHER2USDT	MOTHERUSDT
MOVEUSDT	MOVRUSDT	MTLUSDT	MUBARAKUSDT	MULTIUSDT	MUMUUSDT
MVLUSDT	MYRIAUSDT	MYROUSDT	NCUSDT	NEARUSD	NEARUSDT
NEIROCTOUSDT	NEIROETHUSDT	NEOUSDT	NFPUSDT	NFTUSDT	NILUSDT
NKNUSDT	NMRUSDT	NOTUSDT	NSUSDT	NTRNUSDT	NULSUSDT
NYANUSDT	OBOLUSDT	OBTUSDT	OCEANUSDT	OGNUSDT	OGUSDT

OKBUSDT	OLUSDT	OMGUSDT	OMNIUSDT	OMUSDT	ONDOUSDT
ONEUSDT	ONGUSDT	ONTUSDT	OPUSD	OPUSDT	ORBSUSDT
ORCAUSDT	ORDERUSDT	ORDIUSDT	ORNUSDT	OSMOUSDT	OXTUSDT
PARTIUSDT	PAXGUSDT	PEAQUSDT	PEIPEIUSDT	PENDLEUSDT	PENGUSDT
PENGUUSDT	PEOPLEUSDT	PEPEUSDT	PERPUSDT	PHAUSDT	PHBUSDT
PIPPINUSDT	PIRATEUSDT	PIXELUSDT	PIXFIUSDT	PLUMEUSDT	PNUTUSDT
POLUSDT	POLYXUSDT	PONKEUSDT	POPCATUSDT	PORTALUSDT	POWRUSDT
PRCLUSDT	PRE05EOSUSDT	PRE08EOSUSDT	PRIMEUSDT	PROMPTUSDT	PROMUSDT
PROSUSDT	PUFFERUSDT	PUMPUSDT	PUNDIXUSDT	PUNDUUSDT	PYRUSDT
PYTHUSDT	QIUSDT	QNTUSDT	QTUMUSDT	QUBICUSDT	QUICKUSDT
RADUSDT	RAREUSDT	RATSUSDT	RAYDIUMUSDT	RAYUSDT	RDNTUSDT
REDUSDT	REEFUSDT	RENDERUSDT	RENUSDT	REQUSDT	REXUSDT
REZUSDT	RFCUSDT	RIFSOLUSDT	RIFUSDT	RLCUSDT	RNDRUSDT
ROAMUSDT	RONINUSDT	RONUSDT	ROSEUSDT	RPLUSDT	RSRUSDT
RSS3USDT	RUNEUSDT	RVNUSDT	SAFEUSDT	SAGAUSDT	SANDUSDT
SATSUSDT	SCAUSDT	SCRTUSDT	SCRUSDT	SCUSDT	SDUSDT
SEIUSDT	SENDUSDT	SERAPHUSDT	SFPUSDT	SHELLUSDT	SHIB1000USDT
SIGNUSDT	SILLYUSDT	SIRENUSDT	SKLUSDT	SLERFUSDT	SLFUSDT
SLPUSDT	SNTUSDT	SNXUSDT	SOLAYERUSDT	SOLOUSDT	SOLUSD
SOLUSDT	SOLVUSDT	SONICUSDT	SPECUSDT	SPELLUSDT	SPXUSDT
SSVUSDT	STARLUSDT	STEEMUSDT	STGUSDT	STMXUSDT	STORJUSDT
STOUSDT	STPTUSDT	STRAXUSDT	STRKUSDT	STXUSDT	SUIUSD
SUIUSDT	SUNDOGUSDT	SUNUSDT	SUPERUSDT	SUSDT	SUSHIUSDT
SWARMSUSDT	SWEATUSDT	SWELLUSDT	SXPUSDT	SXTUSDT	SYNUSDT
SYSUSDT	TAIKOUSDT	TAIUSDT	TAOUSDT	TEST01SWELLUSDT	THETAUSDT
THEUSDT	TIAUSDT	TLMUSDT	TNSRUSDT	TOKENUSDT	TOMIUSDT
TOMOUSDT	TONUSD	TONUSDT	TOSHIUSDT	TRBUSDT	TROYUSDT
TRUMPUSDT	TRUUSDT	TRXUSDT	TSTBSCUSDT	TURBOUSDT	TUSDT
TUTUSDT	TWTUSDT	UMAUSDT	UNFIUSDT	UNIUSD	UNIUSDT
UROUSDT	USDCUSDT	USDEUSDT	USTCUSDT	USUALUSDT	UXLINKUSDT
VANAUSDT	VANRYUSDT	VELODROMEUSDT	VELOUSDT	VETUSDT	VGXUSDT
VICUSDT	VIDTUSDT	VINEUSDT	VINUUSDT	VIRTUALUSDT	VOXELUSDT
VRAUSDT	VRUSDT	VTHOUSDT	VVVUSDT	WALUSDT	WAVESUSDT

WAXPUSDT	WCTUSDT	WENUSDT	WHYUSDT	WIFUSD	WIFUSDT
WLDUSDT	WOOUSDT	WSMUSDT	WUSDT	XAIUSDT	XAUTUSDT
XCHUSDT	XCNUSDT	XDCUSDT	XECUSDT	XEMUSDT	XIONUSDT
XLMUSD	XLMUSDT	XMRUSDT	XNOUSDT	XRDUSDT	XRPUSD
XRPUSDT	XTERUSDT	XTZUSDT	XUSDT	XVGUSDT	XVSUSDT
YFIIUSDT	YFIUSDT	YGGUSDT	ZBCNUSDT	ZCXUSDT	ZECUSDT
ZENTUSDT	ZENUSDT	ZEREBROUSDT	ZETAUSDT	ZEUSUSDT	ZILUSDT
ZKFUSDT	ZKJUSDT	ZKUSDT	ZORAUSDT	ZRCUSDT	ZROUSDT
ZRXUSDT					

Example Applications

The Bybit Crypto Future Price dataset enables you to accurately design strategies for Crypto Futures with term structure. Examples include the following strategies:

- Horizontal/Diagonal arbitrage with the underlying Cryptocurrencies
- Trade Contango/Backwardation predictions
- Hedge for illiquid Cryptocurrencies

Classic Algorithm Example

The following example algorithm buys BTCUSDT perpetual future contract if the last day's close price was close to ask close price than bid close price, sells short of that in opposite, through the Bybit exchange:

```py
from AlgorithmImports import *


class BybitCryptoFutureDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to Tether, since USD and USDT will not auto-convert and USD cannot be used to trade
        self.set_account_currency("USDT", 100000)
        # Bybit accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        # Requesting data, we only trade on BTCUSDT Future in Bybit exchange
        crypto_future = self.add_crypto_future("BTCUSDT", Resolution.DAILY)
        # perpetual futures does not have a filter function
        self.btcusdt = crypto_future.symbol

        # Historical data
        history = self.history(self.btcusdt, 10, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        # Trade only based on updated price data
        if not slice.bars.contains_key(self.btcusdt) or not slice.quote_bars.contains_key(self.btcusdt):
            return

        quote = slice.quote_bars[self.btcusdt]
        price = slice.bars[self.btcusdt].price

        # Scalp-trade the bid-ask spread based on the supply-demand strength
        if price - quote.bid.close > quote.ask.close - price:
            self.set_holdings(self.btcusdt, -1)
        else:
            self.set_holdings(self.btcusdt, 1)
```

Framework Algorithm Example

The following example algorithm hold a 100% long BTCUST future portfolio if the last day's close price was close to ask close price than bid close price, while hold short of that in opposite, through the Bybit exchange using the algorithm framework implementation:

```py
from AlgorithmImports import *


class BybitCryptoFutureDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to Tether, since USD and USDT will not auto-convert and USD cannot be used to trade
        self.set_account_currency("USDT", 100000)
        # Bybit accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.leverage = 2
        # We only trade on BTCUSDT Future in Bybit exchange
        symbols = [Symbol.create("BTCUSDT", SecurityType.CRYPTO_FUTURE, Market.BYBIT)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Custom alpha model to emit insights based on the Crypto Future price data
        self.add_alpha(CryptoFutureAlphaModel())
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())

class CryptoFutureAlphaModel(AlphaModel):

    def __init__(self) -> None:
        self.symbols = []

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        for symbol in self.symbols:
            # Trade only based on updated price data
            if not slice.bars.contains_key(symbol) or not slice.quote_bars.contains_key(symbol):
                continue
            quote = slice.quote_bars[symbol]
            price = slice.bars[symbol].price

            # Scalp-trade the bid-ask spread based on the supply-demand strength
            if price - quote.bid.close > quote.ask.close - price:
                insights.append(Insight.price(symbol, timedelta(1), InsightDirection.DOWN))
            else:
                insights.append(Insight.price(symbol, timedelta(1), InsightDirection.UP))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            self.symbols.append(symbol)

            # Historical data
            history = algorithm.history(symbol, 10, Resolution.DAILY)

        for security in changes.removed_securities:
            symbol = security.symbol
            if symbol in self.symbols:
                self.symbols.remove(symbol)
```

Data Point Attributes

The Bybit Crypto Future Price dataset provides `TradeBar` , `QuoteBar` , and `Tick` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CoinAPI

Bybit Crypto Price Data

Introduction

The Bybit Crypto Price Data by CoinAPI is for Cryptocurrency price and volume data points. The data covers 956 Cryptocurrency pairs, starts in July 2017, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Bybit.

For more information about the Bybit Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Bybit Crypto Price dataset:

```py
# Bybit accepts both Cash and Margin account types only.
self.set_brokerage_model(BrokerageName.BYBIT, AccountType.CASH)
#self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

self.btcusdt = self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BYBIT).symbol

self._universe = self.add_universe(CryptoUniverse.bybit(self.universe_selection_filter))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	April 2022
Asset Coverage	956 Currency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Bybit Crypto Price data to your algorithm, call the `add_crypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)

        # Set Account Currency to Tether
        self.set_account_currency("USDT", 100000)

        # Bybit accepts both Cash and Margin account types.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        self.btcusdt = self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BYBIT).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Bybit Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcusdt in slice.bars:
        trade_bar = slice.bars[self.btcusdt]
        self.log(f"{self.btcusdt} close at {slice.time}: {trade_bar.close}")

    if self.btcusdt in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcusdt]
        self.log(f"{self.btcusdt} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcusdt in slice.ticks:
        ticks = slice.ticks[self.btcusdt]
        for tick in ticks:
            self.log(f"{self.btcusdt} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Bybit Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcusdt, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcusdt, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcusdt, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcusdt, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Bybit Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```
PY
def initialize(self) -> None:
    self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)
    self.universe_settings.asynchronous = True
    self._universe = self.add_universe(CryptoUniverse.bybit(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcusdt)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (956)					
1INCHADA	1INCHETH	1INCHUSDE	1INCHUSDT	1SOLUSDT	3PUSDT
5IREUSDC	5IREUSDT	A8USDT	AARKUSDT	AAVEACH	AAVEUSDC
AAVEUSDT	ACAUSDC	ACAUSDT	ACHUSDC	ACHUSDT	ACMUSDT
ACSACH	ACSMNT	ACSUSDC	ACSUSDT	ADA2LUSDT	ADA2SUSDT
ADABTC	ADAETH	ADAEUR	ADAMNT	ADAUSDC	ADAUSDT
AEGUSDT	AEROUSDT	AEVOUSDT	AFCUSDT	AFGUSDT	AGIUSDT
AGIXUSDT	AGLAUSDT	AGLDUSDT	AI16ZUSDT	AIOZUSDT	AIXBTUSDT
AKIUSDT	ALCHUSDT	ALGOBTC	ALGOUSDT	ALTUSDT	AMIUSDT

ANIMEUSDT	ANKRUSDT	APE2LUSDT	APE2SUSDT	APEUSDC	APEUSDT
APEXUSDC	APEXUSDT	APPUSDT	APRSUSDT	APTRUSDT	APTUSDC
APTUSDT	ARBUSDC	ARBUSDT	ARKMUSDT	ARTYUSDT	ARUSDT
ATHUSDT	ATOM2LUSDT	ATOM2SUSDT	ATOMUSDC	ATOMUSDT	AUDETH
AUDUSDT	AURORAUSDT	AURYUSDT	AVAILUSDT	AVAUSDT	AVAX2LUSDT
AVAX2SUSDT	AVAXEUR	AVAXUSDC	AVAXUSDT	AVLUSDT	AXLUSDT
AXSUSDT	AZYUSDT	B3TRUSDC	B3TRUSDT	B3USDT	BABY1USDT
BABYDOGEBTC	BABYDOGEETH	BABYDOGEUSDT	BANUSDT	BARUSDT	BATBTC
BATETH	BATUSDT	BBDETH	BBDUSDC	BBDUSDT	BBLUSDT
BBQUSDT	BBSOLSOL	BBSOLUSDC	BBSOLUSDT	BBUSDT	BCHUSDC
BCHUSDT	BCUTUSDT	BEAMUSDT	BELUSDT	BENUSDT	BERAUSDT
BICOUSDT	BLASTUSDT	BLOCKUSDT	BLURUSDT	BMTUSDT	BNBETH
BNBUSDC	BNBUSDT	BNTUSDT	BOBAUSDT	BOBUSDC	BOBUSDT
BOMEUSDC	BOMEUSDT	BONKUSDC	BONKUSDT	BONUSETH	BONUSEUR
BONUSUSDC	BONUSUSDT	BRAWLUSDT	BRETTUSDC	BRETTUSDT	BRLETH
BRUSDT	BRZUSDT	BTC2SUSDT	BTC3LUSDC	BTC3LUSDT	BTC3SUSDT
BTCBETH	BTCBRL	BTCBRZ	BTCCUSDT	BTCDAI	BTCETH
BTCEUR	BTCPLN	BTCTRY	BTCUSDC	BTCUSDE	BTCUSDQ
BTCUSDR	BTCUSDT	BTGUSDT	BTTBIT	BTTUSDT	BUBBLEUSDT
BUSDUSDT	C98USDT	CAKEUSDT	CANDYUSDT	CAPOUSDT	CAPSUSDT
CARVUSDT	CATBNBUSDT	CATIEUR	CATIUSDC	CATIUSDT	CATSUSDT
CATUSDT	CBKUSDT	CBXUSDT	CELOUSDT	CELUSDT	CGPTUSDT
CHILLGUYUSDT	CHRPUSDT	CHZBIT	CHZEUR	CHZUSDC	CHZUSDT
CITYUSDT	CLOUDUSDT	CMETHUSDT	CMPUSDT	COMPBIT	COMPUSDT
COMUSDT	COOKIEUSDT	COOKUSDT	COPBIT	COPETH	COPUSDT
COQUSDT	COREUSDC	COREUSDT	CORNUSDT	COTUSDT	COUSDT
CPOOLUSDT	CRDSUSDT	CRVBTC	CRVETH	CRVUSDT	CSPRUSDT
CTAUSDT	CTCETH	CTCUSDT	CTTUSDT	CULTUSDT	CUSDUSDT
CWARUSDT	CYBERUSDT	DAIADA	DAIBIT	DAIETH	DAIUSDC
DAIUSDT	DAPPUSDT	DBRUSDT	DCRBIT	DCRUSDT	DECHATUSDT
DEEPUSDT	DEFIUSDT	DEFYUSDT	DEGENUSDT	DEVTUSDT	DFIUSDT
DGBUSDT	DIAMUSDT	DICEUSDT	DLCUSDT	DMAILUSDT	DOGE2LUSDT

DOGE2SUSDT	DOGEETH	DOGEEUR	DOGEUSDC	DOGEUSDT	DOGSEUR
DOGSUSDC	DOGSUSDT	DOLOUSDT	DOMEUSDT	DOODUSDT	DOP1USDT
DOT3LUSDT	DOT3SUSDT	DOTBTC	DOTETH	DOTUSDC	DOTUSDT
DPXUSDT	DRIFTUSDT	DSRUNUSDT	DUELUSDT	DYDXUSDC	DYDXUSDT
DYMUSDT	DZDUSDT	DZOOUSDT	ECOXUSDT	EGLDUSDC	EGLDUSDT
EGOUSDT	EGP1USDT	EIGENUSDT	ELDAUSDT	ELIXUSDT	ELTUSDT
ELXUSDT	ENAEUR	ENAUSDT	ENJUSDT	ENSUSDT	EOS2LUSDT
EOS2SUSDT	EOS3LUSDT	EOS3SUSDT	EOSBTC	EOSETH	EOSMNT
EOSTUSD	EOSUSDC	EOSUSDT	EPTUSDT	ERTHAUSDT	ESEUSDT
ETC2LUSDT	ETC2SUSDT	ETCETH	ETCUSDT	ETH3LUSDT	ETH3SUSDT
ETHBRL	ETHBTC	ETHDAI	ETHEUR	ETHFIUSDT	ETHPLN
ETHTRY	ETHUSDC	ETHUSDE	ETHUSDT	ETHWUSDT	EUROCUSDT
EVERUSDT	EVERYUSDT	EXVGUSDT	FAMEUSDT	FARUSDT	FBUSDT
FCDUSDT	FETUSDC	FETUSDT	FHEUSDT	FIDAUSDT	FILUSDC
FILUSDT	FIREUSDT	FITFIUSDT	FLIPUSDT	FLOCKUSDT	FLOKIUSDC
FLOKIUSDT	FLOWBIT	FLOWUSDT	FLRUSDT	FLTUSDT	FLUIDUSDT
FMBUSDT	FMCUSDT	FONUSDT	FORTUSDT	FOXYUSDT	FTM2LUSDT
FTM2SUSDT	FTMEUR	FTMUSDC	FTMUSDT	FTTUSDC	FTTUSDT
FUELUSDT	FUSDT	FXSBIT	FXSUSDT	G3USDT	G7USDT
GALAUSDT	GALAXISUSDT	GALFTUSDT	GALUSDT	GAMEUSDT	GCAKEUSDT
GENEUSDT	GGMUSDT	GGUSDT	GLMRUSDT	GMBIT	GMRXUSDT
GMT2LUSDT	GMT2SUSDT	GMTUSDC	GMTUSDT	GMUSDC	GMUSDT
GMXBIT	GMXETH	GMXUSDT	GNSUSDT	GOATUSDT	GODSUSDT
GPSUSDT	GPTUSDT	GRAPEUSDT	GRASSUSDT	GRTBIT	GRTUSDT
GSTSUSDT	GSTUSDT	GSWIFTUSDT	GTAIUSDT	GUMMYUSDT	GUSDT
HAEDALUSDT	HATUSDT	HBARUSDT	HEROUSDT	HFTUSDC	HFTUSDT
HLGUSDT	HMSTRUSDC	HMSTRUSDT	HNTBIT	HNTUSDT	HONUSDT
HOOKUSDT	HOTUSDT	HPOS10IUSDT	HTETH	HTXUSDT	HVHUSDT
HYPERUSDT	ICPUSDC	ICPUSDT	ICXUSDT	IDUSDT	IMXUSDT
INITUSDT	INJUSDC	INJUSDT	INSPUSDT	INTERUSDT	INTXUSDT
IOUSDT	IPUSDT	IRLUSDT	IZIUSDT	JASMYUSDC	JASMYUSDT
JEFFUSDT	JSTUSDT	JTOUSDT	JUPUSDT	JUSDT	JUVUSDT

KAIAUSDT	KARATEUSDT	KASTAUSDT	KASUSDC	KASUSDT	KAVAUSDT
KCALUSDT	KCSUSDT	KDAUSDT	KILOUSDT	KLAYUSDT	KMABIT
KMAUSDT	KMNOUSDT	KMONUSDT	KOKUSDT	KONUSDT	KRLUSDT
KROUSDT	KSMUSDT	KUBUSDT	KUNCIUSDT	L3USDT	LADYSUSDT
LAIUSDT	LAVAUSDT	LAYERUSDT	LDOUSDC	LDOUSDT	LENDSUSDT
LEVERUSDT	LFTUSDT	LFWUSDT	LGXUSDT	LINGUSDT	LINK2LUSDT
LINK2SUSDT	LINKEUR	LINKUSDC	LINKUSDT	LISUSDT	LLUSDT
LMWRUSDT	LNK2SUSDT	LOOKSUSDT	LRCUSDT	LTC2LUSDT	LTC2SUSDT
LTCBTC	LTCEUR	LTCUSDC	LTCUSDT	LUCEUSDT	LUNAIUSDT
LUNAUSDT	LUNCUSDC	LUNCUSDT	MAGICUSDT	MAJORUSDT	MAKUSDT
MANABTC	MANAUSDC	MANAUSDT	MANTAUSDT	MASAUSDT	MASKUSDT
MATIC2LUSDT	MATIC2SUSDT	MATICBTC	MATICUSDC	MATICUSDT	MAVIAUSDT
MBOXUSDT	MBSUSDT	MBXUSDT	MCGUSDT	MCRTUSDT	MCTUSDT
MDAOUSDT	MEEETH	MEEUSDC	MEEUSDT	MELOSUSDT	MEMEFIUSDT
MEMEUSDT	MERLUSDT	METHETH	METHUSDT	MEUSDT	MEWUSDC
MEWUSDT	MFERUSDT	MIBRUSDT	MILKUSDT	MINABTC	MINAETH
MINAMNT	MINAUSDC	MINAUSDT	MINUUSDT	MIXUSDT	MKRUSDT
MLKUSDT	MMCUSDT	MNRYUSDT	MNTBTC	MNTETH	MNTUSDC
MNTUSDT	MNZUSDT	MOCAUSDT	MODEUSDT	MOGUSDT	MOJOUSDT
MONUSDT	MORPHOUSDT	MOVEUSDT	MOVEZUSDT	MOVRUSDT	MOZUSDT
MPLXUSDT	MSTARUSDT	MTCUSDT	MTKUSDT	MUSDUSDT	MVLUSDT
MVUSDT	MXMUSDT	MXUSDT	MYRIAUSDT	MYROUSDT	N3USDT
NAKAUSDT	NAVXUSDT	NEAREUR	NEARUSDC	NEARUSDT	NEIROCTOUSDT
NEIROUSDT	NEONUSDT	NESSUSDT	NETHETH	NETHUSDT	NEXOUSDT
NEXTUSDT	NFTUSDT	NGLUSDT	NIBIUSDT	NLKUSDT	NOTUSDC
NOTUSDT	NRNUSDT	NSUSDT	NUTSUSDT	NUUSDT	NYANUSDT
NYMUSDT	OASUSDT	OBOLUSDT	OBTUSDT	OBXUSDT	ODOSUSDT
OIKUSDT	OKGUSDT	OLUSDT	OMGUSDT	OMNICATUSDC	OMNICATUSDT
OMNIUSDC	OMNIUSDT	OMNUSDT	OMUSDT	ONDOEUR	ONDOUSDC
ONDOUSDT	ONEUSDT	OPUSDC	OPUSDT	ORDERUSDT	ORDIUSDT
ORTUSDT	PAALUSDT	PARAMUSDT	PARTIUSDT	PAWSUSDT	PAXGUSDT
PBUXUSDT	PELLUSDT	PENDLEUSDT	PENGUUSDT	PENUSDT	PEOPLEUSDT
PEPE2USDC	PEPE2USDT	PEPEEUR	PEPEUSDC	PEPEUSDT	PERPUSDT

PHPUSDT	PINEYEUSDT	PIPUSDT	PIRATEUSDT	PIXFIUSDT	PLANETUSDT
PLAYUSDT	PLTUSDT	PLUMEUSDT	PLYUSDT	PNUTUSDT	POKTUSDT
POLUSDT	PONKEUSDT	POPCATUSDT	PORT3USDT	PORTALUSDT	POSIUSDC
POSIUSDT	PPTUSDT	PRCLUSDT	PRIMALUSDT	PRIMEUSDT	PSGUSDT
PSPUSDT	PSTAKEUSDT	PTCUSDT	PTUUSDT	PUFFERUSDT	PUFFUSDT
PUMLXUSDT	PUMPUSDT	PURSEUSDT	PYTHUSDT	PYUSDUSDT	QMALLUSDT
QNTUSDT	QORPOUSDT	QTUMUSDT	RACAUSDT	RAINUSDT	RATSUSDT
RDNTUSDT	REALUSDT	REDUSDT	RENDERUSDT	RENUSDT	RLTMUSDT
RNDRUSDT	ROAMUSDT	RONDUSDT	ROOTUSDT	ROSEUSDT	ROUTEUSDT
RPKUSDT	RPLUSDT	RSS3USDT	RUBUSDT	RUBYUSDT	RUNBIT
RUNBTC	RUNEUSDT	RUNUSDT	RVNUSDT	SAFEUSDT	SAILUSDT
SAITAMAUSDT	SALDUSDT	SAND2LUSDT	SAND2SUSDT	SANDBTC	SANDUSDC
SANDUSDT	SAROSUSDT	SATSUSDT	SCAUSDT	SCRTUSDT	SCRUSDT
SCUSDT	SDAGLD	SDBIT	SDUSDT	SEIEUR	SEILORUSDT
SEIUSDC	SEIUSDT	SENDUSDT	SEORUSDT	SERAPHUSDT	SFUNDUSDT
SHARKUSDT	SHIBEUR	SHIBUSDC	SHIBUSDT	SHILLUSDT	SHRAPUSDT
SIDUSUSDT	SIGNUSDT	SISUSDT	SLGUSDC	SLGUSDT	SLPUSDT
SMILEUSDT	SNXUSDT	SOCIALUSDT	SOLBRL	SOLBTC	SOLETH
SOLEUR	SOLOUSDT	SOLUSDC	SOLUSDE	SOLUSDT	SOLVUSDT
SOLXRP	SONBIT	SONICUSDT	SONUSDT	SOSOUSDT	SOSUSDT
SPARTAUSDT	SPECUSDT	SPELLUSDT	SPXUSDT	SQDUSDT	SQRUSDT
SQTUSDT	SRMUSDT	SSVUSDT	STARUSDT	STATUSDT	STETHEUR
STETHUSDT	STGUSDT	STOPUSDT	STREAMUSDT	STRKUSDC	STRKUSDT
STRMUSDT	STXUSDT	SUIAUSDT	SUIUSDC	SUIUSDT	SUNDOGUSDT
SUNUSDT	SUPRAUSDT	SUSDT	SUSHIUSDT	SVLUSDT	SWEATUSDT
SWELLUSDC	SWELLUSDT	SXTUSDT	SYNRUSDT	TAIKOUSDT	TAIUSDT
TAMAUSDT	TAPUSDT	TAVAUSDT	TELUSDT	TENETUSDT	TFUELUSDT
THBEUR	THBUSDC	THBUSDT	THETAUSDT	THNUSDT	THRUSTUSDT
TIAUSDC	TIAUSDT	TIMEUSDT	TNSRUSDT	TOKENUSDT	TOMIUSDT
TOMSUSDT	TONEUR	TONUSDC	TONUSDT	TOSHIUSDT	TRCUSDT
TRIBEUSDT	TRUMPUSDC	TRUMPUSDT	TRVLUSDT	TRXUSDC	TRXUSDT
TSTUSDT	TURBOSUSDT	TUSDT	TUSDUSDT	TVKUSDT	TWTUSDT

ULTIUSDT	UMAUSDT	UNIUSDC	UNIUSDT	USDCBRL	USDCEUR
USDCUSDT	USDCZEC	USDDUSDT	USDEEUR	USDEUSDC	USDEUSDT
USDQUSDT	USDRUSDT	USDTAUD	USDTBRL	USDTBRZ	USDTBUSDT
USDTETH	USDTEUR	USDTPLN	USDTTRY	USDTUSDE	USDTZEC
USDUSDT	USDYUSDT	USTCUSDT	UXLINKUSDT	UYUUSDT	VANAUSDT
VANRYUSDT	VEGAUSDT	VELARUSDT	VELAUSDT	VELOUSDT	VENOMUSDT
VETUSDT	VEXTUSDT	VICUSDT	VINUUSDT	VIRTUALUSDT	VPADUSDT
VPRUSDT	VRAUSDT	VRTXUSDT	VTHOUSDT	VVUSDT	VVVUSDT
WALUSDT	WAVESUSDT	WAXPUSDT	WBTCBTC	WBTCUSDT	WCTUSDT
WEETHETH	WELLUSDT	WEMIXUSDT	WENUSDT	WIFEUR	WIFUSDC
WIFUSDT	WLDEUR	WLDUSDC	WLDUSDT	WLKNUSDT	WOOUSDT
WUSDT	WWYUSDT	XAIUSDT	XARUSDT	XAUTUSDT	XAVAUSDT
XCADUSDT	XDCUSDT	XECETH	XECUSDC	XECUSDT	XEMUSDT
XETAUSDT	XIONUSDT	XLMBTC	XLMUSDC	XLMUSDT	XRP3LETH
XRP3LUSDT	XRP3SUSDT	XRPBTC	XRPETH	XRPEUR	XRPUSDC
XRPUSDT	XTERUSDT	XTZUSDT	XUSDT	XWGUSDT	XYMUSDT
XZKUSDT	YFIBIT	YFIUSDT	ZAMUSDT	ZAPUSDT	ZARUSDT
ZBCBIT	ZBCUSDT	ZECBTC	ZECETH	ZECTFUEL	ZECUSDT
ZENBTC	ZENDUSDT	ZENETH	ZENTUSDT	ZENUSDC	ZENUSDT
ZEREBROUSDT	ZEROUSDT	ZETAUSDT	ZEXUSDT	ZIGUSDT	ZILUSDT
ZKFUSDT	ZKJETH	ZKJUSDT	ZKLUSDT	ZKUSDC	ZKUSDT
ZORAUSDT	ZRCUSDT	ZROUSDC	ZROUSDT	ZRXACS	ZRXETH
ZRXUSDT	ZTXUSDT				

Example Applications

The Bybit Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Bybit exchange:

```
from AlgorithmImports import * class CoinAPIDataAlgorithm(QCAlgorithm): def initialize(self) -> None: self.set_start_da
te(2024, 9, 1) self.set_end_date(2024, 12, 31) # Set the account currency to USDT, since you can't trade with # USD and
the algorithm doesn't automatically convert USD & USDT # holdings. self.set_account_currency("USDT", 100_000) self.univ
erse_settings.asynchronous = True # Bybit accepts both Cash and Margin account types. Select the # one you need for the
best reality modeling. self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN) # Add BTCUSDT data. self._btc
= self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BYBIT) # Historical data history = self.history(self._btc.symbol
, 30, Resolution.DAILY) # Add a Crypto universe that selects trading pairs on Bybit. universe = self.add_universe(Crypt
oUniverse.bybit(self._select_assets)) # Historical Universe data universe_history = self.history(universe, 30, Resoluti
on.DAILY) for (universe_symbol, time), universe_day in universe_history.items(): for universe_item in universe_day: sym
bol = universe_item.symbol price = universe_item.close def _select_assets(self, universe_day): # Filter for materially
traded Crypto pairs with significant # size and dollar volume, assuming higher capital flow in for # higher return. ret
urn [universe_item.symbol for universe_item in universe_day if universe_item.volume >= 100 and universe_item.volume_in_
usd > 10_000_000] def on_data(self, slice: Slice) -> None: # Speculate-invest all available free cash on BTCUSDT, repec
ting # the order quantity restrictions to avoid invalid orders. if self.portfolio.cash_book['BTC'].amount != 0: return
# Define a buffer so that we can scale down the order size, # avoiding trading errors. buffer = max( BybitFeeModel.TAKE
R_NON_VIP_FEE, self.settings.free_portfolio_value_percentage ) quantity = ( self.portfolio.cash_book['USDT'].amount / s
elf._btc.ask_price * (1-buffer) ) # Check if the quantity exceeds the minimum order size. On # Bybit, the minimum order
size is measured in the base # currency (BTC in this case). if abs(quantity) < self._btc.symbol_properties.minimum_orde
r_size: return self.market_order(self._btc, quantity)
```

```
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Bybit exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

PY

```
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set Account Currency to Tether, since USD and USDT will not auto-convert and USD cannot be used to trade
        self.set_account_currency("USDT", 100000)
        self.settings.free_portfolio_value_percentage = 0.05

        # Bybit accepts both Cash and Margin account types, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)

        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        self.universe_settings.resolution = Resolution.MINUTE
        # Add Crypto Universe Selection that select crypto pairs in Bybit exchange
        self.add_universe(CryptoUniverse.bybit(self.universe_selection_filter))
        # Get the pairs on Bybit that have USDT as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.BYBIT)
            if x.value.quote_currency == self.account_currency
        ]

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(minutes = 20), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection_filter(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USDT.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request")
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the Bybit exchange:

```py
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.BYBIT).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.bybit(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Bybit Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

CoinAPI

Coinbase Crypto Price Data

Introduction

The Coinbase Crypto Price Data by CoinAPI provides Cryptocurrency price and volume data points. The data covers 970 Cryptocurrency pairs, starts in January 2015, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on Coinbase.

For more information about the Coinbase Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Coinbase Crypto Price dataset:

```py
# Coinbase only accepts Cash account type
self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)

self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.COINBASE).symbol

self._universe = self.add_universe(CryptoUniverse.coinbase(self.universe_selection_filter))
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2015
Asset Coverage	970 Currency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Coinbase Crypto Price data to your algorithm, call the `add_crypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        # Coinbase accepts Cash account type only, AccountType.MARGIN will result in an exception.
        self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)

        self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.COINBASE).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Coinbase Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcusd in slice.bars:
        trade_bar = slice.bars[self.btcusd]
        self.log(f"{self.btcusd} close at {slice.time}: {trade_bar.close}")

    if self.btcusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcusd]
        self.log(f"{self.btcusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcusd in slice.ticks:
        ticks = slice.ticks[self.btcusd]
        for tick in ticks:
            self.log(f"{self.btcusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Coinbase Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Coinbase Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```
PY
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)
    self._universe = self.add_universe(CryptoUniverse.coinbase(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (970)					
00USD	00USDC	1INCHBTC	1INCHEUR	1INCHGBP	1INCHUSD
1INCHUSDC	A8USD	A8USDC	AAVEBTC	AAVEEUR	AAVEGBP
AAVEUSD	AAVEUSDC	ABTUSD	ABTUSDC	ACHUSD	ACHUSDC
ACSUSD	ACSUSDC	ACXUSD	ACXUSDC	ADABTC	ADAETH
ADAEUR	ADAGBP	ADAUSD	ADAUSDC	ADAUSDT	AERGOUSD
AERGOUSDC	AEROUSD	AEROUSDC	AGLDUSD	AGLDUSDC	AGLDUSDT
AIOZUSD	AIOZUSDC	AKTUSD	AKTUSDC	ALCXEUR	ALCXUSD
ALCXUSDC	ALCXUSDT	ALEOUSD	ALEOUSDC	ALEPHUSD	ALEPHUSDC
ALGOBTC	ALGOEUR	ALGOGBP	ALGOUSD	ALGOUSDC	ALICEUSD

ALICEUSDC	ALTUSD	ALTUSDC	AMPUSD	AMPUSDC	ANKRBTC
ANKREUR	ANKRGBP	ANKRUSD	ANKRUSDC	ANTUSD	ANTUSDC
APEEUR	APEUSD	APEUSDC	APEUSDT	API3USD	API3USDC
API3USDT	APTUSD	APTUSDC	APTUSDT	ARBUSD	ARBUSDC
ARKMUSD	ARKMUSDC	ARPAEUR	ARPAUSD	ARPAUSDC	ARPAUSDT
ASMUSD	ASMUSDC	ASMUSDT	ASTUSD	ASTUSDC	ATAUSD
ATAUSDC	ATHUSD	ATHUSDC	ATOMBTC	ATOMEUR	ATOMGBP
ATOMUSD	ATOMUSDC	ATOMUSDT	AUCTIONEUR	AUCTIONUSD	AUCTIONUSDC
AUCTIONUSDT	AUDIOUSD	AUDIOUSDC	AURORAUSD	AURORAUSDC	AVAXBTC
AVAXEUR	AVAXUSD	AVAXUSDC	AVAXUSDT	AVTUSD	AVTUSDC
AXLUSD	AXLUSDC	AXSBTC	AXSEUR	AXSUSD	AXSUSDC
AXSUSDT	B3USD	B3USDC	BADGEREUR	BADGERUSD	BADGERUSDC
BADGERUSDT	BALBTC	BALUSD	BALUSDC	BANDBTC	BANDEUR
BANDGBP	BANDUSD	BANDUSDC	BATBTC	BATETH	BATEUR
BATUSD	BATUSDC	BCHBTC	BCHEUR	BCHGBP	BCHUSD
BCHUSDC	BERAUSD	BERAUSDC	BICOEUR	BICOUSD	BICOUSDC
BICOUSDT	BIGTIMEUSD	BIGTIMEUSDC	BITUSD	BITUSDC	BLASTUSD
BLASTUSDC	BLURUSD	BLURUSDC	BLZUSD	BLZUSDC	BNTBTC
BNTEUR	BNTGBP	BNTUSD	BNTUSDC	BOBAUSD	BOBAUSDC
BONDUSD	BONKUSD	BONKUSDC	BTCEUR	BTCGBP	BTCUSD
BTCUSDC	BTCUSDT	BTCUST	BTRSTBTC	BTRSTEUR	BTRSTGBP
BTRSTUSD	BTRSTUSDC	BTRSTUSDT	C98USD	C98USDC	CBETHETH
CBETHUSD	CBETHUSDC	CELRUSD	CELRUSDC	CGLDBTC	CGLDEUR
CGLDGBP	CGLDUSD	CGLDUSDC	CHZEUR	CHZGBP	CHZUSD
CHZUSDC	CHZUSDT	CLANKERUSD	CLANKERUSDC	CLVEUR	CLVGBP
CLVUSD	CLVUSDC	CLVUSDT	COMPBTC	COMPUSD	COMPUSDC
COOKIEUSD	COOKIEUSDC	CORECHAINUSD	CORECHAINUSDC	COTIUSD	COTIUSDC
COVALUSD	COVALUSDC	COVALUSDT	COWUSD	COWUSDC	CROEUR
CROUSD	CROUSDC	CROUSDT	CRVBTC	CRVEUR	CRVGBP
CRVUSD	CRVUSDC	CTSIBTC	CTSIUSD	CTSIUSDC	CTXEUR
CTXUSD	CTXUSDC	CTXUSDT	CVCUSD	CVCUSDC	CVXUSD
CVXUSDC	DAIUSD	DAIUSDC	DARUSD	DARUSDC	DASHBTC
DASHUSD	DASHUSDC	DDXEUR	DDXUSD	DDXUSDT	DEGENUSD

DEGENUSDC	DESOEUR	DESOUSD	DESOUSDC	DESOUSDT	DEXTUSD
DEXTUSDC	DIAEUR	DIAUSD	DIAUSDC	DIAUSDT	DIMOUSD
DIMOUSDC	DNTUSD	DNTUSDC	DOGEBTC	DOGEEUR	DOGEGBP
DOGEUSD	DOGEUSDC	DOGEUSDT	DOGINMEUSD	DOGINMEUSDC	DOTBTC
DOTEUR	DOTGBP	DOTUSD	DOTUSDC	DOTUSDT	DREPUSD
DREPUSDC	DRIFTUSD	DRIFTUSDC	DYPUSD	DYPUSDC	EDGEUSD
EDGEUSDC	EGLDUSD	EGLDUSDC	EIGENUSD	EIGENUSDC	ELAUSD
ELAUSDC	ENJBTC	ENJUSD	ENJUSDC	ENJUSDT	ENSEUR
ENSUSD	ENSUSDC	ENSUSDT	EOSBTC	EOSEUR	EOSUSD
EOSUSDC	ERNUSD	ERNUSDC	ETCBTC	ETCEUR	ETCGBP
ETCUSD	ETCUSDC	ETHBTC	ETHDAI	ETHEUR	ETHFIUSD
ETHFIUSDC	ETHGBP	ETHUSD	ETHUSDC	ETHUSDT	EURCEUR
EURCUSD	EURCUSDC	EUROCEUR	EUROCUSD	EUROCUSDC	FAIUSD
FAIUSDC	FARMUSD	FARMUSDC	FARMUSDT	FETUSD	FETUSDC
FETUSDT	FIDAEUR	FIDAUSD	FIDAUSDC	FIDAUSDT	FILBTC
FILEUR	FILGBP	FILUSD	FILUSDC	FISUSD	FISUSDC
FLOKIUSD	FLOKIUSDC	FLOWUSD	FLOWUSDC	FLOWUSDT	FLRUSD
FLRUSDC	FORTHBTC	FORTHEUR	FORTHGBP	FORTHUSD	FORTHUSDC
FORTUSD	FORTUSDC	FOXUSD	FOXUSDC	FOXUSDT	FXUSD
FXUSDC	GALAEUR	GALAUSD	GALAUSDC	GALAUSDT	GALUSD
GALUSDC	GFIUSD	GFIUSDC	GHSTUSD	GHSTUSDC	GIGAUSD
GIGAUSDC	GLMUSD	GLMUSDC	GMTUSD	GMTUSDC	GMTUSDT
GNOUSD	GNOUSDC	GNTUSDC	GODSUSD	GODSUSDC	GRTBTC
GRTEUR	GRTGBP	GRTUSD	GRTUSDC	GSTUSD	GSTUSDC
GTCUSD	GTCUSDC	GUSD	GUSDC	GUSDUSD	GUSDUSDC
GYENUSD	GYENUSDC	HBARUSD	HBARUSDC	HBARUSDT	HFTUSD
HFTUSDC	HIGHUSD	HIGHUSDC	HNTUSD	HNTUSDC	HONEYUSD
HONEYUSDC	HOPRUSD	HOPRUSDC	ICPBTC	ICPEUR	ICPGBP
ICPUSD	ICPUSDC	ICPUSDT	IDEXUSD	IDEXUSDC	IDEXUSDT
ILVUSD	ILVUSDC	IMXUSD	IMXUSDC	IMXUSDT	INDEXUSD
INDEXUSDC	INJUSD	INJUSDC	INVUSD	INVUSDC	IOTXEUR
IOTXUSD	IOTXUSDC	IOUSD	IOUSDC	IPUSD	IPUSDC
JASMYUSD	JASMYUSDC	JASMYUSDT	JTOUSD	JTOUSDC	KAITOUSD

KAITOUSDC	KARRATUSD	KARRATUSDC	KAVAUSD	KAVAUSDC	KEEPUSD
KERNELUSD	KERNELUSDC	KEYCATUSD	KEYCATUSDC	KNCBTC	KNCUSD
KNCUSDC	KRLEUR	KRLUSD	KRLUSDC	KRLUSDT	KSMUSD
KSMUSDC	L3USD	L3USDC	LCXEUR	LCXUSD	LCXUSDC
LCXUSDT	LDOUSD	LDOUSDC	LINKBTC	LINKETH	LINKEUR
LINKGBP	LINKUSD	LINKUSDC	LINKUSDT	LITUSD	LITUSDC
LOKAUSD	LOKAUSDC	LOOMUSD	LOOMUSDC	LPTUSD	LPTUSDC
LQTYEUR	LQTYUSD	LQTYUSDC	LQTYUSDT	LRCBTC	LRCUSD
LRCUSDC	LRCUSDT	LRDSUSD	LRDSUSDC	LSETHETH	LSETHUSD
LSETHUSDC	LTCBTC	LTCEUR	LTCGBP	LTCUSD	LTCUSDC
MAGICUSD	MAGICUSDC	MANABTC	MANAETH	MANAEUR	MANAUSD
MANAUSDC	MANTLEUSD	MANTLEUSDC	MASKEUR	MASKGBP	MASKUSD
MASKUSDC	MASKUSDT	MATHUSD	MATHUSDC	MATICBTC	MATICEUR
MATICGBP	MATICUSD	MATICUSDC	MATICUSDT	MCO2USD	MCO2USDT
MDTUSD	MDTUSDC	MDTUSDT	MEDIAUSD	MEDIAUSDC	METISUSD
METISUSDC	MEUSD	MEUSDC	MINAEUR	MINAUSD	MINAUSDC
MINAUSDT	MIRBTC	MIREUR	MIRGBP	MIRUSD	MKRBTC
MKRUSD	MKRUSDC	MLNUSD	MLNUSDC	MNDEUSD	MNDEUSDC
MOBILEUSD	MOBILEUSDC	MOGUSD	MOGUSDC	MONAUSD	MONAUSDC
MOODENGUSD	MOODENGUSDC	MORPHOUSD	MORPHOUSDC	MOVEUSD	MOVEUSDC
MPLUSD	MPLUSDC	MSOLUSD	MSOLUSDC	MTLUSD	MTLUSDC
MUSDUSD	MUSEUSD	MUSEUSDC	NCTEUR	NCTUSD	NCTUSDC
NCTUSDT	NEARUSD	NEARUSDC	NEARUSDT	NEONUSD	NEONUSDC
NKNBTC	NKNEUR	NKNGBP	NKNUSD	NKNUSDC	NMRBTC
NMREUR	NMRGBP	NMRUSD	NMRUSDC	NUBTC	NUEUR
NUGBP	NUUSD	OCEANUSD	OCEANUSDC	OGNBTC	OGNUSD
OGNUSDC	OMGBTC	OMGEUR	OMGGBP	OMGUSD	OMNIUSD
OMNIUSDC	ONDOUSD	ONDOUSDC	OPUSD	OPUSDC	OPUSDT
ORCAUSD	ORCAUSDC	ORNBTC	ORNUSD	ORNUSDC	ORNUSDT
OSMOUSD	OSMOUSDC	OXTUSD	OXTUSDC	PAXGUSD	PAXGUSDC
PAXUSD	PAXUSDC	PAXUSDT	PENDLEUSD	PENDLEUSDC	PENGUUSD
PENGUUSDC	PEPEUSD	PEPEUSDC	PERPEUR	PERPUSD	PERPUSDC
PERPUSDT	PIRATEUSD	PIRATEUSDC	PLAUSD	PLAUSDC	PLUUSD

PLUUSDC	PNGUSD	PNGUSDC	PNUTUSD	PNUTUSDC	POLSUSD
POLSUSDC	POLSUSDT	POLUSD	POLUSDC	POLYUSD	POLYUSDT
PONDUSD	PONDUSDC	POPCATUSD	POPCATUSDC	POWREUR	POWRUSD
POWRUSDC	POWRUSDT	PRCLUSD	PRCLUSDC	PRIMEUSD	PRIMEUSDC
PROMPTUSD	PROMPTUSDC	PROUSD	PROUSDC	PRQUSD	PRQUSDC
PUNDIXUSD	PUNDIXUSDC	PYRUSD	PYRUSDC	PYTHUSD	PYTHUSDC
PYUSDUSD	PYUSDUSDC	QIUSD	QIUSDC	QNTUSD	QNTUSDC
QNTUSDT	QSPUSD	QSPUSDT	QUICKUSD	RADBTC	RADEUR
RADGBP	RADUSD	RADUSDC	RADUSDT	RAIUSD	RAIUSDC
RAREUSD	RAREUSDC	RARIUSD	RARIUSDC	RBNUSD	RBNUSDC
REDUSD	REDUSDC	RENBTC	RENDERUSD	RENDERUSDC	RENUSD
RENUSDC	REPBTC	REPUSD	REQBTC	REQEUR	REQGBP
REQUSD	REQUSDC	REQUSDT	REZUSD	REZUSDC	RGTUSD
RLCBTC	RLCUSD	RLCUSDC	RLYEUR	RLYGBP	RLYUSD
RLYUSDT	RNDREUR	RNDRUSD	RNDRUSDC	RNDRUSDT	RONINUSD
RONINUSDC	ROSEUSD	ROSEUSDC	ROSEUSDT	RPLUSD	RPLUSDC
RSRUSD	RSRUSDC	SAFEUSD	SAFEUSDC	SANDUSD	SANDUSDC
SANDUSDT	SDUSD	SDUSDC	SEAMUSD	SEAMUSDC	SEIUSD
SEIUSDC	SHDWUSD	SHDWUSDC	SHIBEUR	SHIBGBP	SHIBUSD
SHIBUSDC	SHIBUSDT	SHPINGEUR	SHPINGUSD	SHPINGUSDC	SHPINGUSDT
SKLBTC	SKLEUR	SKLGBP	SKLUSD	SKLUSDC	SNTUSD
SNTUSDC	SNXBTC	SNXEUR	SNXGBP	SNXUSD	SNXUSDC
SOLBTC	SOLETH	SOLEUR	SOLGBP	SOLUSD	SOLUSDC
SOLUSDT	SPAUSD	SPAUSDC	SPELLUSD	SPELLUSDC	SPELLUSDT
STGUSD	STGUSDC	STORJBTC	STORJUSD	STORJUSDC	STRKUSD
STRKUSDC	STXUSD	STXUSDC	STXUSDT	SUIUSD	SUIUSDC
SUKUEUR	SUKUUSD	SUKUUSDC	SUKUUSDT	SUPERUSD	SUPERUSDC
SUPERUSDT	SUSHIBTC	SUSHIETH	SUSHIEUR	SUSHIGBP	SUSHIUSD
SUSHIUSDC	SWELLUSD	SWELLUSDC	SWFTCUSD	SWFTCUSDC	SXTUSD
SXTUSDC	SYLOUSD	SYLOUSDC	SYNUSD	SYNUSDC	SYRUPUSD
SYRUPUSDC	TAOUSD	TAOUSDC	TIAUSD	TIAUSDC	TIMEUSD
TIMEUSDC	TNSRUSD	TNSRUSDC	TOSHIUSD	TOSHIUSDC	TRACEUR

TRACUSD	TRACUSDC	TRACUSDT	TRBBTC	TRBUSD	TRBUSDC
TRIBEUSD	TRUBTC	TRUEUR	TRUMPUSD	TRUMPUSDC	TRUUSD
TRUUSDC	TRUUSDT	TURBOUSD	TURBOUSDC	TUSD	TUSDC
TVKUSD	TVKUSDC	UMABTC	UMAEUR	UMAGBP	UMAUSD
UMAUSDC	UNFIUSD	UNFIUSDC	UNIBTC	UNIEUR	UNIGBP
UNIUSD	UNIUSDC	UPIUSD	UPIUSDT	USDCEUR	USDCGBP
USDTEUR	USDTGBP	USDTUSD	USDTUSDC	USTEUR	USTUSD
USTUSDT	VARAUSD	VARAUSDC	VELOUSD	VELOUSDC	VETUSD
VETUSDC	VGXEUR	VGXUSD	VGXUSDT	VOXELUSD	VOXELUSDC
VTHOUSD	VTHOUSDC	VVVUSD	VVVUSDC	WAMPLUSD	WAMPLUSDC
WAXLUSD	WAXLUSDC	WBTCBTC	WBTCUSD	WBTCUSDC	WCFGBTC
WCFGEUR	WCFGGBP	WCFGUSD	WCFGUSDC	WCFGUSDT	WELLUSD
WELLUSDC	WIFUSD	WIFUSDC	WLDUSD	WLDUSDC	WLUNABTC
WLUNAEUR	WLUNAGBP	WLUNAUSD	WLUNAUSDT	XCNUSD	XCNUSDC
XLMBTC	XLMEUR	XLMUSD	XLMUSDC	XLMUSDT	XRPBTC
XRPEUR	XRPGBP	XRPUSD	XRPUSDC	XRPUSDT	XTZBTC
XTZEUR	XTZGBP	XTZUSD	XTZUSDC	XYOBTC	XYOEUR
XYOUSD	XYOUSDC	XYOUSDT	YFIBTC	YFIIUSD	YFIUSD
YFIUSDC	ZECBTC	ZECUSD	ZECUSDC	ZENBTC	ZENUSD
ZENUSDC	ZENUSDT	ZETACHAINUSD	ZETACHAINUSDC	ZETAUSD	ZETAUSDC
ZKUSD	ZKUSDC	ZORAUSD	ZORAUSDC	ZROUSD	ZROUSDC
ZRXBTC	ZRXEUR	ZRXUSD	ZRXUSDC		

Example Applications

The Coinbase Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Coinbase exchange:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.universe_settings.asynchronous = True
        self.set_cash(100_000)
        # Coinbase accepts Cash account type only. AccountType.MARGIN
        # results in an exception.
        self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)
        # Add BTCUSD data.
        self._btc = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.COINBASE)
        # Historical data
        history = self.history(self._btc.symbol, 30, Resolution.DAILY)
        # Add a Crypto universe that selects Crypto pairs on Coinbase.
        universe = self.add_universe(CryptoUniverse.coinbase(self._select_assets))
        # Historical Universe data
        universe_history = self.history(universe, 30, Resolution.DAILY)
        for (universe_symbol, time), universe_day in universe_history.items():
            for universe_item in universe_day:
                symbol = universe_item.symbol
                price = universe_item.close

    def _select_assets(self, universe_day):
        # Filter for materially traded Crypto pairs with significant
        # size and dollar volume, assuming higher capital flow in for
        # higher return.
        return [universe_item.symbol for universe_item in universe_day
                if universe_item.volume >= 100
                and universe_item.volume_in_usd > 10_000_000]

    def on_data(self, slice: Slice) -> None:
        # Speculate-invest all available free cash on BTCUSD, repecting
        # the order quantity restrictions to avoid invalid orders.
        if self.portfolio.cash_book['BTC'].amount != 0:
            return
        # Define a buffer so that we can scale down the order size,
        # avoiding trading errors.
        buffer = max(
            CoinbaseFeeModel.TAKER_ADVANCED_1,
            self.settings.free_portfolio_value_percentage
        )
        quantity = (
            self.portfolio.cash_book['USD'].amount
            / self._btc.ask_price
            * (1-buffer)
        )
        # Check if the quantity exceeds the minimum order size. On
        # Coinbase, the minimum order size is measured in the base
        # currency (BTC in this case).
        if abs(quantity) < self._btc.symbol_properties.minimum_order_size:
            return
        self.market_order(self._btc, quantity)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Coinbase exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

```
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 2, 1)
        self.set_cash(100000)
        self.settings.free_portfolio_value_percentage = 0.05
        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        self.universe_settings.resolution = Resolution.MINUTE
        # Add Crypto Universe Selection that select crypto pairs in Coinbase exchange
        self.add_universe(CryptoUniverse.coinbase(self._select_assets))
        # Get the pairs on Coinbase that have USD as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.COINBASE)
            if x.value.quote_currency == self.account_currency
        ]
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(hours=4), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def _select_assets(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USD.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request")
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the Coinbase exchange:

```
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.COINBASE).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.coinbase(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Coinbase Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

CoinAPI

Kraken Crypto Price Data

Introduction

The Kraken Crypto Price Data by CoinAPI provides Cryptocurrency price and volume data points. The data covers 1,262 Cryptocurrency pairs, starts in October 2013, and is delivered on any frequency from tick to daily. This dataset is created by monitoring the trading activity on the Cryptocurrency markets/exchanges supported by QuantConnect.

For more information about the Kraken Crypto Price Data dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinAPI was founded by Artur Pietrzyk in 2016 with the goal of providing real-time and historical cryptocurrency market data, collected from hundreds of exchanges. CoinAPI provides access to Cryptocurrencies for traders, market makers, and developers building third-party applications.

Getting Started

The following snippet demonstrates how to request data from the Kraken Crypto Price dataset:

```py
# Kraken accepts both Cash and Margin type account.
self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.CASH)
#self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)

self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.KRAKEN).symbol

self._universe = self.add_universe(CryptoUniverse.kraken(self.universe_selection_filter));
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	October 2013
Asset Coverage	1,262 Currency Pairs
Data Density	Dense
Resolution	Tick, Second, Minute, Hourly, & Daily
Timezone	UTC
Market Hours	Always Open

Requesting Data

To add Kraken Crypto Price data to your algorithm, call the `AddCrypto` method. Save a reference to the Crypto `Symbol` so you can access the data later in your algorithm.

```py
class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 6, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        # Kraken accepts both Cash and Margin type account.
        self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)

        self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.KRAKEN).symbol
```

For more information about creating Crypto subscriptions, see Requesting Data .

Accessing Data

To get the current Kraken Crypto Price data, index the `bars` , `quote_bars` , or `ticks` properties of the current `Slice` with the Crypto `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.btcusd in slice.bars:
        trade_bar = slice.bars[self.btcusd]
        self.log(f"{self.btcusd} close at {slice.time}: {trade_bar.close}")

    if self.btcusd in slice.quote_bars:
        quote_bar = slice.quote_bars[self.btcusd]
        self.log(f"{self.btcusd} bid at {slice.time}: {quote_bar.bid.close}")

    if self.btcusd in slice.ticks:
        ticks = slice.ticks[self.btcusd]
        for tick in ticks:
            self.log(f"{self.btcusd} price at {slice.time}: {tick.price}")
```

You can also iterate through all of the data objects in the current `Slice` .

```
PY
def on_data(self, slice: Slice) -> None:
    for symbol, trade_bar in slice.bars.items():
        self.log(f"{symbol} close at {slice.time}: {trade_bar.close}")

    for symbol, quote_bar in slice.quote_bars.items():
        self.log(f"{symbol} bid at {slice.time}: {quote_bar.bid.close}")

    for symbol, ticks in slice.ticks.items():
        for tick in ticks:
            self.log(f"{symbol} price at {slice.time}: {tick.price}")
```

For more information about accessing Crypto data, see Handling Data .

Historical Data

To get historical Kraken Crypto Price data, call the `history` method with the Crypto `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.btcusd, 100, Resolution.DAILY)

# TradeBar objects
history_trade_bars = self.history[TradeBar](self.btcusd, 100, Resolution.MINUTE)

# QuoteBar objects
history_quote_bars = self.history[QuoteBar](self.btcusd, 100, Resolution.MINUTE)

# Tick objects
history_ticks = self.history[Tick](self.btcusd, timedelta(seconds=10), Resolution.TICK)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Kraken Crypto pairs, call the `add_universe` method with a `CryptoUniverse` object. A Crypto universe uses a selection function to select Crypto pairs based on their OHLCV and dollar volume of the previous day as of midnight Coordinated Universal Time (UTC).

```
PY
def initialize(self) -> None:
    self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)
    self.universe_settings.asynchronous = True
    self._universe = self.add_universe(CryptoUniverse.kraken(self.universe_selection_filter))

def universe_selection_filter(self, universe_day):
    return [c.symbol for c in universe_day if c.volume >= 100 and c.volume_in_usd > 10000]
```

For more information about universe settings, see Settings .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object, and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), universe_day in history.items():
    for universe_item in universe_day:
        self.log(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, and the lookback period. The `universe_history` returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CryptoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CryptoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in universe_history.items():
    for universe_item in universe_day:
        print(f"{universe_item.symbol} price at {universe_item.end_time}: {universe_item.close}")
```

You can call the `history` method in Research.

Remove Subscriptions

To unsubscribe from a Crypto pair that you added with the `add_crypto` method, call the `remove_security` method.

```py
self.remove_security(self.btcusd)
```

The `remove_security` method cancels your open orders for the security and liquidates your holdings in the virtual pair .

Supported Assets

The following table shows the available Cryptocurrency pairs:

Pairs Available (1262)					
1INCHEUR	1INCHUSD	AAVEAUD	AAVEBTC	AAVEETH	AAVEEUR
AAVEGBP	AAVEUSD	ACAEUR	ACAUSD	ACHEUR	ACHUSD
ACTEUR	ACTUSD	ACXEUR	ACXUSD	ADAAUD	ADABTC
ADAETH	ADAEUR	ADAGBP	ADAUSD	ADAUSDC	ADAUSDT
ADXEUR	ADXUSD	AEROEUR	AEROUSD	AEUR	AEVOEUR
AEVOUSD	AGLDEUR	AGLDUSD	AI16ZEUR	AI16ZUSD	AI16ZUSDC
AI16ZUSDT	AIOZEUR	AIOZUSD	AIREUR	AIRUSD	AIXBTEUR
AIXBTUSD	AKTEUR	AKTUSD	ALCHEUR	ALCHUSD	ALCXEUR

ALCXUSD	ALGOBTC	ALGOETH	ALGOEUR	ALGOGBP	ALGOUSD
ALGOUSDC	ALGOUSDT	ALICEEUR	ALICEUSD	ALPHAEUR	ALPHAUSD
ALTEUR	ALTUSD	ANKRBTC	ANKREUR	ANKRGBP	ANKRUSD
ANLOGEUR	ANLOGUSD	ANONEUR	ANONUSD	ANTBTC	ANTETH
ANTEUR	ANTUSD	APEEUR	APENFTEUR	APENFTUSD	APEUSD
APEUSDC	APEUSDT	API3EUR	API3USD	APTEUR	APTUSD
APUEUR	APUUSD	ARBEUR	ARBUSD	ARCEUR	ARCUSD
AREUR	ARKMEUR	ARKMUSD	ARPAEUR	ARPAUSD	ARUSD
ASRREUR	ASRRUSD	ASTREUR	ASTRUSD	ATHEUR	ATHUSD
ATLASEUR	ATLASUSD	ATOMAUD	ATOMBTC	ATOMETH	ATOMEUR
ATOMGBP	ATOMUSD	ATOMUSDC	ATOMUSDT	AUCTIONEUR	AUCTIONUSD
AUDIOEUR	AUDIOUSD	AUDJPY	AUDUSD	AURAEUR	AURAUSD
AUSD	AVAAIEUR	AVAAIUSD	AVAXEUR	AVAXUSD	AVAXUSDC
AVAXUSDT	AXSEUR	AXSUSD	B3EUR	B3USD	BABYEUR
BABYUSD	BADGEREUR	BADGERUSD	BALBTC	BALETH	BALEUR
BALUSD	BANANAS31EUR	BANANAS31USD	BANDEUR	BANDUSD	BATBTC
BATETH	BATEUR	BATJPY	BATUSD	BCHAUD	BCHBTC
BCHETH	BCHEUR	BCHGBP	BCHJPY	BCHUSD	BCHUSDC
BCHUSDT	BDXNEUR	BDXNUSD	BEAMEUR	BEAMUSD	BERAEUR
BERAUSD	BERAUSDC	BERAUSDT	BICOEUR	BICOUSD	BIGTIMEEUR
BIGTIMEUSD	BIOEUR	BIOUSD	BITEUR	BITUSD	BLUREUR
BLURUSD	BLZEUR	BLZUSD	BMTEUR	BMTUSD	BNBEUR
BNBUSD	BNBUSDC	BNBUSDT	BNCEUR	BNCUSD	BNTBTC
BNTEUR	BNTGBP	BNTUSD	BOBAEUR	BOBAUSD	BODENEUR
BODENUSD	BONDEUR	BONDUSD	BONKEUR	BONKUSD	BRICKEUR
BRICKUSD	BSXEUR	BSXUSD	BTCAED	BTCAUD	BTCCAD
BTCCHF	BTCDAI	BTCEUR	BTCGBP	BTCJPY	BTCPYUSD
BTCUSD	BTCUSDC	BTCUSDQ	BTCUSDR	BTCUSDT	BTTEUR
BTTUSD	C98EUR	C98USD	CAKEEUR	CAKEUSD	CATEUR
CATUSD	CCDEUR	CCDUSD	CELOEUR	CELOUSD	CELREUR
CELRUSD	CFGEUR	CFGUSD	CHEEMSEUR	CHEEMSUSD	CHEXEUR
CHEXUSD	CHILLHOUSEEUR	CHILLHOUSEUSD	CHREUR	CHRUSD	CHZEUR
CHZUSD	CLANKEREUR	CLANKERUSD	CLOUDEUR	CLOUDUSD	CLVEUR

CLVUSD	CMETHETH	CMETHUSD	COMPBTC	COMPETH	COMPEUR
COMPUSD	COOKIEEUR	COOKIEUSD	COQEUR	COQUSD	CORNEUR
CORNUSD	COTIEUR	COTIUSD	COWEUR	COWUSD	CPOOLEUR
CPOOLUSD	CQTEUR	CQTUSD	CROEUR	CROUSD	CROUSDC
CROUSDT	CRVBTC	CRVETH	CRVEUR	CRVUSD	CSMEUR
CSMUSD	CTSIEUR	CTSIUSD	CVCEUR	CVCUSD	CVXEUR
CVXUSD	CXTEUR	CXTUSD	CYBEREUR	CYBERUSD	DAIEUR
DAIUSD	DAIUSDT	DASHBTC	DASHEUR	DASHUSD	DBREUR
DBRUSD	DEEPEUR	DEEPUSD	DEGENEUR	DEGENUSD	DENTEUR
DENTUSD	DMCEUR	DMCUSD	DOGEUR	DOGSEUR	DOGSUSD
DOGUSD	DOLOEUR	DOLOUSD	DOTAUD	DOTBTC	DOTETH
DOTEUR	DOTGBP	DOTJPY	DOTUSD	DOTUSDC	DOTUSDT
DRIFTEUR	DRIFTUSD	DRVEUR	DRVUSD	DUCKEUR	DUCKUSD
DYDXEUR	DYDXUSD	DYMEUR	DYMUSD	EDGEEUR	EDGEUSD
EGLDEUR	EGLDUSD	EIGENEUR	EIGENUSD	ELXEUR	ELXUSD
ENAEUR	ENAUSD	ENJBTC	ENJEUR	ENJGBP	ENJJPY
ENJUSD	ENSEUR	ENSUSD	EOSBTC	EOSETH	EOSEUR
EOSUSD	EOSUSDC	EOSUSDT	EPTEUR	EPTUSD	ESEUR
ESUSD	ESXEUR	ESXUSD	ETCBTC	ETCETH	ETCEUR
ETCUSD	ETH2.SETH	ETHAED	ETHAUD	ETHBTC	ETHCAD
ETHCHF	ETHDAI	ETHEUR	ETHFIEUR	ETHFIUSD	ETHGBP
ETHJPY	ETHPYUSD	ETHUSD	ETHUSDC	ETHUSDT	ETHWETH
ETHWEUR	ETHWUSD	EULEUR	EULUSD	EURAUD	EURCAD
EURCHF	EURGBP	EURJPY	EUROPEUR	EUROPUSD	EUROPUSDC
EURQEUR	EURQUSD	EURREUR	EURRUSD	EURRUSDC	EURRUSDT
EURTEUR	EURTUSD	EURTUSDT	EURUSD	EWTBTC	EWTEUR
EWTGBP	EWTUSD	FARMEUR	FARMUSD	FARTCOINEUR	FARTCOINUSD
FARTCOINUSDC	FARTCOINUSDT	FETEUR	FETUSD	FHEEUR	FHEUSD
FIDAEUR	FIDAUSD	FILAUD	FILBTC	FILETH	FILEUR
FILGBP	FILUSD	FISEUR	FISUSD	FLOKIEUR	FLOKIUSD
FLOWBTC	FLOWETH	FLOWEUR	FLOWGBP	FLOWUSD	FLREUR
FLRUSD	FLUXEUR	FLUXUSD	FLYEUR	FLYUSD	FORTHEUR
FORTHUSD	FTMEUR	FTMUSD	FWOGEUR	FWOGUSD	FXSEUR

FXSUSD	GAIAEUR	GAIAUSD	GALAEUR	GALAUSD	GALEUR
GALUSD	GARIEUR	GARIUSD	GBPUSD	GEUR	GFIEUR
GFIUSD	GHIBLIEUR	GHIBLIUSD	GHSTBTC	GHSTEUR	GHSTGBP
GHSTUSD	GIGAEUR	GIGAUSD	GLMREUR	GLMRUSD	GMTEUR
GMTUSD	GMXEUR	GMXUSD	GNOBTC	GNOETH	GNOEUR
GNOUSD	GOATEUR	GOATUSD	GOMININGEUR	GOMININGUSD	GRASSEUR
GRASSUSD	GRIFFAINEUR	GRIFFAINUSD	GRTAUD	GRTBTC	GRTETH
GRTEUR	GRTGBP	GRTUSD	GSTEUR	GSTUSD	GTCEUR
GTCUSD	GUNEUR	GUNUSD	GUSD	HBAREUR	HBARUSD
HDXEUR	HDXUSD	HFTEUR	HFTUSD	HIPPOEUR	HIPPOUSD
HMSTREUR	HMSTRUSD	HNTEUR	HNTUSD	HONEYEUR	HONEYUSD
HOUSEEUR	HOUSEUSD	HPOS10IEUR	HPOS10IUSD	ICNTEUR	ICNTUSD
ICPEUR	ICPUSD	ICXBTC	ICXETH	ICXEUR	ICXUSD
IDEXEUR	IDEXUSD	IMXEUR	IMXUSD	INITEUR	INITUSD
INJEUR	INJUSD	INTREUR	INTRUSD	IPEUR	IPUSD
JAILSTOOLEUR	JAILSTOOLUSD	JASMYEUR	JASMYUSD	JITOSOLEUR	JITOSOLSOL
JITOSOLUSD	JOEEUR	JOEUSD	JSTEUR	JSTUSD	JTOEUR
JTOUSD	JUNOEUR	JUNOUSD	JUPEUR	JUPUSD	KAITOEUR
KAITOUSD	KAREUR	KARUSD	KASEUR	KASUSD	KAVABTC
KAVAETH	KAVAEUR	KAVAUSD	KEEPBTC	KEEPETH	KEEPEUR
KEEPUSD	KERNELEUR	KERNELUSD	KETEUR	KETUSD	KEUR
KEYEUR	KEYUSD	KILTEUR	KILTUSD	KINEUR	KINTEUR
KINTUSD	KINUSD	KMNOEUR	KMNOUSD	KNCBTC	KNCETH
KNCEUR	KNCUSD	KOBANEUR	KOBANUSD	KP3REUR	KP3RUSD
KSMAUD	KSMBTC	KSMDOT	KSMETH	KSMEUR	KSMGBP
KSMUSD	KTAEUR	KTAUSD	KUJIEUR	KUJIUSD	KUSD
L3EUR	L3USD	LAYEREUR	LAYERUSD	LCXEUR	LCXUSD
LDOEUR	LDOUSD	LINKAUD	LINKBTC	LINKETH	LINKEUR
LINKGBP	LINKJPY	LINKUSD	LINKUSDC	LINKUSDT	LITEUR
LITUSD	LMWREUR	LMWRUSD	LOCKINEUR	LOCKINUSD	LOFIEUR
LOFIUSD	LPTBTC	LPTEUR	LPTGBP	LPTUSD	LQTYEUR
LQTYUSD	LRCEUR	LRCUSD	LSETHETH	LSETHEUR	LSETHUSD
LSKBTC	LSKETH	LSKEUR	LSKUSD	LSSOLEUR	LSSOLSOL

LSSOLUSD	LTCAUD	LTCBTC	LTCETH	LTCEUR	LTCGBP
LTCJPY	LTCUSD	LTCUSDC	LTCUSDT	LUNA2EUR	LUNA2USD
LUNAEUR	LUNAUSD	MANABTC	MANAETH	MANAEUR	MANAUSD
MANAUSDC	MANAUSDT	MASKEUR	MASKUSD	MATEUR	MATICBTC
MATICEUR	MATICGBP	MATICPOL	MATICUSD	MATICUSDC	MATICUSDT
MATUSD	MCEUR	MCUSD	MEEUR	MELANIAEUR	MELANIAUSD
MELANIAUSDC	MELANIAUSDT	MEMEEUR	MEMEUSD	MERLEUR	MERLUSD
METHETH	METHUSD	METISEUR	METISUSD	MEUR	MEUSD
MEWEUR	MEWUSD	MICHIEUR	MICHIUSD	MINABTC	MINAEUR
MINAGBP	MINAUSD	MIRAEUR	MIRAUSD	MIREUR	MIRUSD
MKRBTC	MKREUR	MKRGBP	MKRUSD	MLNBTC	MLNETH
MLNEUR	MLNUSD	MNGOEUR	MNGOUSD	MNTEUR	MNTUSD
MOGEUR	MOGUSD	MOODENGEUR	MOODENGUSD	MOONEUR	MOONUSD
MORPHOEUR	MORPHOUSD	MOVEEUR	MOVEUSD	MOVREUR	MOVRUSD
MSOLEUR	MSOLUSD	MUBARAKEUR	MUBARAKUSD	MULTIEUR	MULTIUSD
MUSD	MVEUR	MVUSD	MXCEUR	MXCUSD	NANOBTC
NANOETH	NANOEUR	NANOUSD	NEAREUR	NEARUSD	NEIROEUR
NEIROUSD	NILEUR	NILUSD	NMREUR	NMRUSD	NODLEUR
NODLUSD	NOSEUR	NOSUSD	NOTEUR	NOTUSD	NPCEUR
NPCUSD	NTRNEUR	NTRNUSD	NYMBTC	NYMEUR	NYMUSD
OCEANBTC	OCEANEUR	OCEANGBP	OCEANUSD	ODOSEUR	ODOSUSD
OGNEUR	OGNUSD	OMEUR	OMGBTC	OMGETH	OMGEUR
OMGJPY	OMGUSD	OMNIEUR	OMNIUSD	OMUSD	ONDOEUR
ONDOUSD	OPEUR	OPUSD	ORCAEUR	ORCAUSD	ORDEREUR
ORDERUSD	OSMOEUR	OSMOUSD	OXTBTC	OXTETH	OXTEUR
OXTUSD	OXYEUR	OXYUSD	PARTIEUR	PARTIUSD	PAXGBTC
PAXGETH	PAXGEUR	PAXGUSD	PDAEUR	PDAUSD	PEAQEUR
PEAQUSD	PENDLEEUR	PENDLEUSD	PENGUEUR	PENGUUSD	PENGUUSDC
PENGUUSDT	PEPEAUD	PEPECAD	PEPEEUR	PEPEGBP	PEPEUSD
PERPEUR	PERPUSD	PHAEUR	PHAUSD	PLUMEEUR	PLUMEUSD
PNUTEUR	PNUTUSD	POLEUR	POLISEUR	POLISUSD	POLSEUR
POLSUSD	POLUSD	PONDEUR	PONDUSD	PONKEEUR	PONKEUSD

POPCATEUR	POPCATGBP	POPCATUSD	PORTALEUR	PORTALUSD	POWREUR
POWRUSD	PRCLEUR	PRCLUSD	PRIMEEUR	PRIMEUSD	PROEUR
PROMPTEUR	PROMPTUSD	PROUSD	PSTAKEEUR	PSTAKEUSD	PUFFEREUR
PUFFERUSD	PUMPEUR	PUMPSOL	PUMPUSD	PYTHEUR	PYTHUSD
PYUSDEUR	PYUSDUSD	QIEUR	QIUSD	QNTEUR	QNTUSD
QTUMBTC	QTUMETH	QTUMEUR	QTUMUSD	RADEUR	RADUSD
RAIINEUR	RAIINUSD	RAREEUR	RAREUSD	RARIBTC	RARIEUR
RARIGBP	RARIUSD	RAYEUR	RAYUSD	RBCEUR	RBCUSD
REDEUR	REDUSD	RENBTC	RENDEREUR	RENDERUSD	RENEUR
RENGBP	RENUSD	REPBTC	REPETH	REPEUR	REPUSD
REPV2BTC	REPV2ETH	REPV2EUR	REPV2USD	REQEUR	REQUSD
RETARDIOEUR	RETARDIOUSD	REZEUR	REZUSD	RIZEEUR	RIZEUSD
RLCEUR	RLCUSD	RLUSDEUR	RLUSDUSD	RLUSDUSDC	RLUSDUSDT
RNDREUR	RNDRUSD	ROOKEUR	ROOKUSD	RPLEUR	RPLUSD
RSREUR	RSRUSD	RUJIEUR	RUJIUSD	RUNEEUR	RUNEUSD
SAFEEUR	SAFEUSD	SAGAEUR	SAGAUSD	SAHARAEUR	SAHARAUSD
SAMOEUR	SAMOUSD	SANDBTC	SANDEUR	SANDGBP	SANDUSD
SBREUR	SBRUSD	SCBTC	SCETH	SCEUR	SCRTEUR
SCRTUSD	SCUSD	SDNEUR	SDNUSD	SEIEUR	SEIUSD
SEUR	SGBEUR	SGBUSD	SHIBEUR	SHIBUSD	SHIBUSDC
SHIBUSDT	SIGMAEUR	SIGMAUSD	SKYEUR	SKYUSD	SNEKEUR
SNEKUSD	SNXAUD	SNXBTC	SNXETH	SNXEUR	SNXGBP
SNXUSD	SOGNIEUR	SOGNIUSD	SOLAUD	SOLBTC	SOLCAD
SOLETH	SOLEUR	SOLGBP	SOLUSD	SOLUSDC	SOLUSDT
SONICEUR	SONICUSD	SOSOEUR	SOSOUSD	SPELLEUR	SPELLUSD
SPICEEUR	SPICEUSD	SPKEUR	SPKUSD	SPXEUR	SPXUSD
SRMBTC	SRMEUR	SRMGBP	SRMUSD	SSVEUR	SSVUSD
STEPEUR	STEPUSD	STGEUR	STGUSD	STORJBTC	STORJETH
STORJEUR	STORJUSD	STRDEUR	STRDUSD	STRKEUR	STRKUSD
STXEUR	STXUSD	SUIEUR	SUIGBP	SUIUSD	SUNDOGEUR
SUNDOGUSD	SUNEUR	SUNUSD	SUPEREUR	SUPERUSD	SUSD
SUSDC	SUSDT	SUSHIBTC	SUSHIEUR	SUSHIGBP	SUSHIUSD
SWARMSEUR	SWARMSUSD	SWEATEUR	SWEATUSD	SWELLEUR	SWELLUSD

SXTEUR	SXTUSD	SYNEUR	SYNUSD	SYRUPEUR	SYRUPUSD
TACEUR	TACUSD	TANSSIEUR	TANSSIUSD	TAOEUR	TAOUSD
TBTCBTC	TBTCETH	TBTCEUR	TBTCUSD	TEEREUR	TEERUSD
TERMEUR	TERMUSD	TEUR	TIAEUR	TIAUSD	TITCOINEUR
TITCOINUSD	TLMEUR	TLMUSD	TNSREUR	TNSRUSD	TOKEEUR
TOKENEUR	TOKENUSD	TOKEUSD	TONEUR	TONUSD	TONUSDC
TONUSDT	TOSHIEUR	TOSHIUSD	TRACEUR	TRACUSD	TREEEUR
TREEUSD	TREMPEUR	TREMPUSD	TRUEUR	TRUMPEUR	TRUMPUSD
TRUMPUSDC	TRUMPUSDT	TRUUSD	TRXBTC	TRXETH	TRXEUR
TRXUSD	TRXUSDD	TURBOEUR	TURBOUSD	TUSD	TUSDEUR
TUSDUSD	TVKEUR	TVKUSD	UFDEUR	UFDUSD	UMAEUR
UMAUSD	UNFIEUR	UNFIUSD	UNIBTC	UNIETH	UNIEUR
UNITEEUR	UNITEUSD	UNIUSD	USDAED	USDCAD	USDCAUD
USDCCAD	USDCCHF	USDCEUR	USDCGBP	USDCHF	USDCUSD
USDCUSDT	USDDEUR	USDDUSD	USDGEUR	USDGUSD	USDGUSDC
USDJPY	USDQEUR	USDQUSD	USDQUSDC	USDQUSDT	USDREUR
USDRUSD	USDRUSDC	USDRUSDT	USDSEUR	USDSUSD	USDTAUD
USDTCAD	USDTCHF	USDTEUR	USDTGBP	USDTJPY	USDTUSD
USDUCEUR	USDUCUSD	USTEUR	USTUSD	USTUSDC	USTUSDT
USUALEUR	USUALUSD	VANRYEUR	VANRYUSD	VELODROMEEUR	VELODROMEUSD
VINEEUR	VINEUSD	VIRTUALEUR	VIRTUALUSD	VIRTUALUSDC	VIRTUALUSDT
VSNEUR	VSNUSD	VVVEUR	VVVUSD	WALEUR	WALUSD
WAVESBTC	WAVESETH	WAVESEUR	WAVESUSD	WAXLEUR	WAXLUSD
WBTCBTC	WBTCEUR	WBTCUSD	WCTEUR	WCTUSD	WELLEUR
WELLUSD	WENEUR	WENUSD	WEUR	WIFEUR	WIFGBP
WIFUSD	WINEUR	WINUSD	WLDEUR	WLDUSD	WOOEUR
WOOUSD	WUSD	XCNEUR	XCNUSD	XDGAUD	XDGBTC
XDGCAD	XDGEUR	XDGGBP	XDGUSD	XDGUSDC	XDGUSDT
XLMAUD	XLMBTC	XLMEUR	XLMGBP	XLMUSD	XMRBTC
XMREUR	XMRUSD	XMRUSDC	XMRUSDT	XNYEUR	XNYUSD
XRPAUD	XRPBTC	XRPCAD	XRPETH	XRPEUR	XRPGBP
XRPJPY	XRPRLUSD	XRPUSD	XRPUSDC	XRPUSDT	XRTEUR
XRTUSD	XTZAUD	XTZBTC	XTZETH	XTZEUR	XTZGBP

XTZUSD	XTZUSDC	XTZUSDT	YFIAUD	YFIBTC	YFIETH
YFIEUR	YFIGBP	YFIUSD	YGGEUR	YGGUSD	ZECBTC
ZECEUR	ZECUSD	ZEREBROEUR	ZEREBROUSD	ZETAEUR	ZETAUSD
ZEUSEUR	ZEUSUSD	ZEXEUR	ZEXUSD	ZKEUR	ZKUSD
ZORAEUR	ZORAUSD	ZROEUR	ZROUSD	ZRXBTC	ZRXEUR
ZRXGBP	ZRXUSD				

Example Applications

The Kraken Crypto Price dataset enables you to accurately design strategies for Cryptocurrencies. Examples include the following strategies:

- Buy and hold
- Trading Cryptocurrency volatility and price action
- Allocating a small portion of your portfolio to Cryptocurrencies to hedge against inflation

Classic Algorithm Example

The following example algorithm buys and holds Bitcoin through the Kraken exchange:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.universe_settings.asynchronous = True
        self.set_cash(100_000)
        # Kraken accepts both Cash and Margin account types. Select the
        # one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)
        # Add BTCUSD data.
        self._btc = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.KRAKEN)
        # Historical data
        history = self.history(self._btc.symbol, 30, Resolution.DAILY)
        # Add a Crypto universe that selects Crypto pairs on Kraken.
        universe = self.add_universe(CryptoUniverse.kraken(self._select_assets))
        # Historical Universe data
        universe_history = self.history(universe, 30, Resolution.DAILY)
        for (universe_symbol, time), universe_day in universe_history.items():
            for universe_item in universe_day:
                symbol = universe_item.symbol
                price = universe_item.close

    def _select_assets(self, universe_day):
        # Filter for materially traded Crypto pairs with significant
        # size and dollar volume, assuming higher capital flow in for
        # higher return.
        return [universe_item.symbol for universe_item in universe_day
                if universe_item.volume >= 100
                and universe_item.volume_in_usd > 10_000_000]

    def on_data(self, slice: Slice) -> None:
        # Speculate-invest all available free cash on BTCUSD, repecting
        # the order quantity restrictions to avoid invalid orders.
        if self.portfolio.cash_book['BTC'].amount != 0:
            return
        # Define a buffer so that we can scale down the order size,
        # avoiding trading errors.
        buffer = max(
            KrakenFeeModel.TAKER_TIER_1_CRYPTO_FEE,
            self.settings.free_portfolio_value_percentage
        )
        quantity = (
            self.portfolio.cash_book['USD'].amount
            / self._btc.ask_price
            * (1-buffer)
        )
        # Check if the quantity exceeds the minimum order size. On
        # Kraken, the minimum order size is measured in the base
        # currency (BTC in this case).
        if abs(quantity) < self._btc.symbol_properties.minimum_order_size:
            return
        self.market_order(self._btc, quantity)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of Crypto pairs on the Kraken exchange and then forms a equal-weighted portfolio of all the pairs in the universe:

```py
from AlgorithmImports import *


class CoinAPIDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.free_portfolio_value_percentage = 0.05

        # Kraken accepts both Cash and Margin type account, select the one you need for the best reality modeling.
        self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)

        # Warm up the security with the last known price to avoid conversion error
        self.settings.seed_initial_prices = True

        # Get the pairs on Kraken that have USD as the quote currency.
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(Market.KRAKEN)
            if x.value.quote_currency == self.account_currency
        ]
        # Add Crypto Universe Selection that select crypto pairs in Kraken exchange
        self.add_universe(CryptoUniverse.kraken(self.universe_selection_filter))

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(hours=4), 0.025, None))
        # Equally invest to evenly dissipate the capital concentration risk of inidividual crypto pair
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection_filter(self, universe_day: List[CryptoUniverse]) -> List[Symbol]:
        # Select pairs where the quote currency is USD.
        pairs = [c for c in universe_day if c.symbol.value in self._market_pairs]
        # Filter for materially traded crypto pairs with significant dollar volume, assuming higher capital flow in for higher return.
        pairs = sorted(pairs, key=lambda c: c.volume_in_usd)[-3:]
        return [c.symbol for c in pairs]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Historical data
            history = self.history(security.symbol, 30, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request")
```

Research Example

The following example lists crypto-currency pairs with the greatest dollar volume in the Kraken exchange:

```py
qb = QuantBook()

# Add Cryptocurrency pair
symbol = qb.add_crypto("BTCUSDT", market=Market.KRAKEN).symbol

# Historical data
history = qb.history(symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f'{time} {symbol} {row.close}')

# Add Crypto Universe Selection
def universe_selection_filter(universe_day):
    selected = sorted([x for x in universe_day
        if x.volume_in_usd and x.volume_in_usd > 10000],
        key=lambda x: x.volume_in_usd, reverse=True)[:5]
    return [x.symbol for x in selected]

universe = qb.add_universe(CryptoUniverse.kraken(universe_selection_filter))

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), universe_day in history.items():
    print(f'=== {time} ===')
    for universe_item in sorted(universe_day, key=lambda x: x.volume_in_usd, reverse=True):
        print(f"{universe_item.symbol}: {universe_item.volume_in_usd}")
```

Data Point Attributes

The Kraken Crypto Price dataset provides `TradeBar` , `QuoteBar` , `Tick` , and `CryptoUniverse` objects.

TradeBar Attributes

`TradeBar` objects have the following attributes:

QuoteBar Attributes

`QuoteBar` objects have the following attributes:

Tick Attributes

`Tick` objects have the following attributes:

CryptoUniverse Attributes

`CryptoUniverse` objects have the following attributes:

Datasets

Benzinga

Benzinga was founded by Jason Raznick in 2010 with goal of connecting the world with news, data, and education that makes the path to financial prosperity easier for everyone, everyday. Benzinga provides access to real-time news for individual investors.

Benzinga News Feed

Benzinga

Benzinga News Feed

Introduction

The Benzinga News Feed dataset by Benzinga tracks US Equity news releases. The data covers about 1,250 articles per day across 8,000 Equities, starts in January 2016, and is delivered on a second frequency. This dataset is created by structuring the content produced by Benzinga's editorial team.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Benzinga News Feed dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Benzinga was founded by Jason Raznick in 2010 with goal of connecting the world with news, data, and education that makes the path to financial prosperity easier for everyone, everyday. Benzinga provides access to real-time news for individual investors.

Getting Started

The following snippet demonstrates how to request data from the Benzinga News Feed dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(BenzingaNews, self.symbol).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	September 2017
Asset Coverage	8,000 Equities
Data Density	Sparse
Resolution	Second (1,250 Articles/Day)
Timezone	New York

Requesting Data

To add Benzinga News Feed data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class BenzingaNewsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.dataset_symbol = self.add_data(BenzingaNews, self.aapl).symbol
```

Accessing Data

To get the current Benzinga News Feed data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm

as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY

def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        article = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} title at {slice.time}: {article.title}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY

def on_data(self, slice: Slice) -> None:
    for dataset_symbol, article in slice.get(BenzingaNews).items():
        self.log(f"{dataset_symbol} title at {slice.time}: {article.title}")
```

Historical Data

To get historical Benzinga News Feed data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY

# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[BenzingaNews](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY

self.remove_security(self.dataset_symbol)
```

If you subscribe to Benzinga News Feed data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Benzinga News Feed enables you to accurately design strategies harnessing real-time news releases. Examples include the following strategies:

- Creating a dictionary of sentiment scores for various words and assigning a sentiment score to the content of each news release
- Calculating the sentiment of news releases with Natural Language Processing (NLP)
- Trading securities when their news releases that Benzinga tags with current buzzwords

Classic Algorithm Example

The following example algorithm parses the Benzinga news articles related to Apple. If the sentiment is positive, the algorithm buys Apple. Otherwise, it holds cash.

```py
from AlgorithmImports import *

class BenzingaNewsDataAlgorithm(QCAlgorithm):

    current_holdings = 0
    target_holdings = 0
    # A custom word-score map for calculating the total sentiment score
    word_scores = {
        'good': 1, 'great': 1, 'best': 1, 'growth': 1,
        'bad': -1, 'terrible': -1, 'worst': -1, 'loss': -1}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting data to obtain the updated news for sentiment score calculation
        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.benzinga_symbol = self.add_data(BenzingaNews, self.aapl).symbol

        # Historical data
        history = self.history(self.benzinga_symbol, 14, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def on_data(self, slice: Slice) -> None:
        if slice.contains_key(self.benzinga_symbol):
            # Assign a sentiment score to the news article by specific word appearance scoring
            content_words = slice[self.benzinga_symbol].contents.lower()
            score = 0
            for word, word_score in self.word_scores.items():
                score += (content_words.count(word) * word_score)
            self.target_holdings = int(score > 0)

        # Ensure we have AAPL data in the current Slice to avoid stale filling
        if not (slice.contains_key(self.aapl) and slice[self.aapl] is not None and not slice[self.aapl].is_fill_forward):
            return

        # Buy or sell if the sentiment has changed from our current holdings
        if self.current_holdings != self.target_holdings:
            self.set_holdings(self.aapl, self.target_holdings)
            self.current_holdings = self.target_holdings
```

Framework Algorithm Example

The following example algorithm parses the Benzinga news articles related to Apple. If the sentiment is positive, the algorithm buys Apple. Otherwise, it holds cash.

```py
from AlgorithmImports import *


class BenzingaNewsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        symbols = [ Symbol.create("AAPL", SecurityType.EQUITY, Market.USA) ]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))

        self.add_alpha(BenzingaNewsAlphaModel())

        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())


class BenzingaNewsAlphaModel(AlphaModel):

    symbol_data_by_symbol = {}
    # A custom word-score map for calculating the total sentiment score
    word_scores = {'good': 1, 'great': 1, 'best': 1, 'growth': 1,
                   'bad': -1, 'terrible': -1, 'worst': -1, 'loss': -1}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        for symbol, symbol_data in self.symbol_data_by_symbol.items():
            if slice.contains_key(symbol_data.benzinga_symbol):
                # Assign a sentiment score to the news article by specific word appearance scoring
                content_words = slice[symbol_data.benzinga_symbol].contents.lower()
                score = 0
                for word, word_score in self.word_scores.items():
                    score += (content_words.count(word) * word_score)
                symbol_data.target_direction = InsightDirection.UP if score > 0 else InsightDirection.FLAT

            # Ensure we have security data in the current Slice to avoid stale filling
            if not (slice.contains_key(symbol) and slice[symbol] is not None and not slice[symbol].is_fill_forward):
                continue

            # Buy or sell if the sentiment has changed from our current holdings
            if symbol_data.current_direction != symbol_data.target_direction:
                symbol_data.current_direction = symbol_data.target_direction
                insights.append(Insight.price(symbol, timedelta(days=14), symbol_data.target_direction))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            self.symbol_data_by_symbol[symbol] = SymbolData(algorithm, symbol)

        for security in changes.removed_securities:
            symbol_data = self.symbol_data_by_symbol.pop(security.symbol, None)
            if symbol_data:
                symbol_data.dispose()

class SymbolData:

    current_direction = InsightDirection.FLAT
    target_direction = InsightDirection.FLAT

    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        self.algorithm = algorithm

        # Requesting data to obtain the updated news for sentiment score calculation
        self.benzinga_symbol = algorithm.add_data(BenzingaNews, symbol).symbol

        # Historical data
        history = algorithm.history(self.benzinga_symbol, 14, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request")

    def dispose(self) -> None:
        # Unsubscribe from Benzinga news feed for this security to release computational resources
        self.algorithm.remove_security(self.benzinga_symbol)
```

Data Point Attributes

The Benzinga News Feed dataset provides `BenzingaNews` objects, which have the following attributes:

Datasets

Blockchain

Blockchain is a website that publishes data related to Bitcoin. It has been online since 2011 and publishes the Bitcoin Metadata history back to 2009.

Bitcoin Metadata

Blockchain

Bitcoin Metadata

Introduction

The Bitcoin Metadata dataset by Blockchain provides 23 fundamental metadata of Bitcoin directly fetched from the Bitcoin blockchain. The data starts in January 2009 and delivered on a daily frequency. This dataset contains mining statistics like hash rate and miner revenue; transaction metadata like transaction per block, transaction fee, and number of addresses; and blockchain metadata like blockchain size and block size.

For more information about the Bitcoin Metadata dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Blockchain is a website that publishes data related to Bitcoin. It has been online since 2011 and publishes the Bitcoin Metadata history back to 2009.

Getting Started

The following snippet demonstrates how to request data from the Bitcoin Metadata dataset:

```
self.btcusd = self.add_crypto("BTCUSD", Resolution.DAILY, Market.BITFINEX).symbol
self.dataset_symbol = self.add_data(BitcoinMetadata, self.btcusd).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2009
Coverage	Bitcoin blockchain
Data Density	Regular
Resolution	Daily
Timezone	UTC

Requesting Data

To add Bitcoin Metadata data to your algorithm, call the `add_data` method with the BTCUSD `Symbol` . Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class BlockchainBitcoinMetadataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.btcusd = self.add_crypto("BTCUSD", Resolution.DAILY, Market.BITFINEX).symbol
        self.dataset_symbol = self.add_data(BitcoinMetadata, self.btcusd).symbol
```

Accessing Data

To get the current Bitcoin Metadata data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} miner revenue at {slice.time}: {data_point.miners_revenue}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(BlockchainBitcoinData).items():
        self.log(f"{dataset_symbol} miner revenue at {slice.time}: {data_point.miners_revenue}")
```

Historical Data

To get historical Bitcoin Metadata data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[BlockchainBitcoinData](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Bitcoin Metadata dataset enables you to incorporate metadata from the Bitcoin blockchain into your strategies. Examples include the following strategies:

- Comparing mining and transaction statistics to provide insight on the supply-demand relationship of the Bitcoin blockchain service.
- Measuring the activity and popularity of the Bitcoin blockchain to predict the price movements of the Cryptocurrency.

Classic Algorithm Example

The following example algorithm tracks the transaction-to-hash-rate ratio of the Bitcoin network. The algorithm holds Bitcoin when the ratio increases. Otherwise, it holds dollars.

```py
from AlgorithmImports import *


class BlockchainBitcoinMetadataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request BTCUSD as the trading vehicle on Bitcoin Metadata
        self.btcusd = self.add_crypto("BTCUSD", Resolution.MINUTE, Market.BITFINEX).symbol
        # Request Bitcoin Metadata for trade signal generation
        self.bitcoin_metadata_symbol = self.add_data(BitcoinMetadata, self.btcusd).symbol

        # Historical data
        history = self.history(BitcoinMetadata, self.bitcoin_metadata_symbol, 60, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for {self.btcusd} Blockchain Bitcoin Metadata")

        # Cache the last supply-demand ratio for comparison
        self.last_demand_supply = None

    def on_data(self, slice: Slice) -> None:
        # Trade only based on updated Bitcoin Metadata
        data = slice.get(BitcoinMetadata)

        if self.bitcoin_metadata_symbol in data and data[self.bitcoin_metadata_symbol] != None and
data[self.bitcoin_metadata_symbol].hash_rate:
            # Calculate the supply-demand ratio to estimate the microeconomy structure of Bitcoin for scalp-trading
            # Transaction number as demand, hash production rate as supply
            current_demand_supply = data[self.bitcoin_metadata_symbol].numberof_transactions /
data[self.bitcoin_metadata_symbol].hash_rate

            # Comparing the average transaction-to-hash-rate ratio changes, buy Bitcoin if demand is higher than supply, sell vice versa
            if self.last_demand_supply != None and current_demand_supply > self.last_demand_supply:
                self.set_holdings(self.btcusd, 1)
            else:
                self.set_holdings(self.btcusd, 0)

            self.last_demand_supply = current_demand_supply
```

Framework Algorithm Example

The following example algorithm tracks the transaction-to-hash-rate ratio of the Bitcoin network. The algorithm holds Bitcoin when the ratio increases. Otherwise, it holds dollars.

```py
from AlgorithmImports import *


class BlockchainBitcoinMetadataFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Universe contains only BTCUSD as the trading vehicle on Bitcoin Metadata
        self.add_universe_selection(
            ManualUniverseSelectionModel(
            Symbol.create("BTCUSD", SecurityType.CRYPTO, Market.BITFINEX)
        ))
        # Custom alpha model that emit insights based on Bitcoin Metadata
        self.add_alpha(BlockchainBitcoinMetadataAlphaModel())
        # Equally invest to evenly dissipate the capital concentration risk from non-sysmtematic risky events
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class BlockchainBitcoinMetadataAlphaModel(AlphaModel):

    def __init__(self) -> None:
        self.bitcoin_metadata_symbol_by_symbol = {}
        # Cache the last supply-demand ratio for comparison
        self.last_demand_supply = {}

    def update(self, algorithm:QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade only based on updated Bitcoin Metadata
        data = slice.Get(BitcoinMetadata)

        for symbol, bitcoin_metadata_symbol in self.bitcoin_metadata_symbol_by_symbol.items():
            if data.contains_key(bitcoin_metadata_symbol) and data[bitcoin_metadata_symbol] != None and
data[bitcoin_metadata_symbol].hash_rate:
                # Calculate the supply-demand ratio to estimate the microeconomy structure of the crypto pair for scalp-trading
                # Transaction number as demand, hash production rate as supply
                current_demand_supply = data[bitcoin_metadata_symbol].numberof_transactions / data[bitcoin_metadata_symbol].hash_rate

                # Comparing the average transaction-to-hash-rate ratio changes, buy coin if demand is higher than supply
                if symbol in self.last_demand_supply and current_demand_supply > self.last_demand_supply[symbol]:
                    insights.append(Insight.price(symbol, timedelta(1), InsightDirection.UP))

                self.last_demand_supply[symbol] = current_demand_supply

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol

            # Request Bitcoin Metadata for trade signal generation
            bitcoin_metadata_symbol = algorithm.add_data(BitcoinMetadata, symbol).symbol

            self.bitcoin_metadata_symbol_by_symbol[symbol] = bitcoin_metadata_symbol

            # Historical data
            history = algorithm.history(BitcoinMetadata, bitcoin_metadata_symbol, 60, Resolution.DAILY)
```

Data Point Attributes

The Bitcoin Metadata dataset provides `BitcoinMetadata` objects, which have the following attributes:

Datasets

Brain

Brain is a Research Company that creates proprietary datasets and algorithms for investment strategies, combining experience in financial markets with strong competencies in Statistics, Machine Learning, and Natural Language Processing. The founders share a common academic background of research in Physics as well as extensive experience in Financial markets.

Brain Language Metrics on Company Filings

Brain ML Stock Ranking

Brain Sentiment Indicator

Brain Wikipedia Page Views

Brain

Brain Language Metrics on Company Filings

Introduction

The Brain Language Metrics on Company Filings dataset provides the results of an NLP system that monitors several language metrics on 10-K and 10-Q company reports for US Equities. The data covers 5,000 US Equities, starts in January 2010, and is delivered on a daily frequency. The dataset is made of two parts; the first one includes the language metrics of the most recent 10-K or 10-Q report for each firm, namely:

1. Financial sentiment
2. Percentage of words belonging to financial domain classified by language types (e.g. “litigious” or “constraining” language)
3. Readability score
4. Lexical metrics such as lexical density and richness
5. Text statistics such as the report length and the average sentence length

The second part includes the differences between the two most recent 10-Ks or 10-Qs reports of the same period for each company, namely:

1. Difference of the various language metrics (e.g. delta sentiment, delta readability score, delta percentage of a specific language type etc.)
2. Similarity metrics between documents, also with respect to a specific language type (for example similarity with respect to “litigious” language or “uncertainty” language)

The analysis is available for the whole report and for specific sections of the report (e.g. Risk Factors and MD&A).

For more information, refer to Brain's summary paper .

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Brain Language Metrics on Company Filings dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Brain is a Research Company that creates proprietary datasets and algorithms for investment strategies, combining experience in financial markets with strong competencies in Statistics, Machine Learning, and Natural Language Processing. The founders share a common academic background of research in Physics as well as extensive experience in Financial markets.

Getting Started

The following snippet demonstrates how to request data from the Brain Language Metrics on Company Filings dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_10k_symbol = self.add_data(BrainCompanyFilingLanguageMetrics10K , self.aapl).symbol
self.dataset_all_symbol = self.add_data(BrainCompanyFilingLanguageMetricsAll, self.aapl).symbol

self.universe_10k = self.add_universe(BrainCompanyFilingLanguageMetricsUniverse10K, self.universe_selection)
self.universe_all = self.add_universe(BrainCompanyFilingLanguageMetricsUniverseAll, self.universe_selection)
```

PY

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2010
Asset Coverage*	5,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

The coverage includes all assets since the start date. It increases over time.

Requesting Data

To add Brain Language Metrics on Company Filings data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class BrainCompanyFilingNLPDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2010, 1, 1)
        self.set_end_date(2021, 7, 8)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_10k_symbol = self.add_data(BrainCompanyFilingLanguageMetrics10K, self.aapl).symbol
        self.dataset_all_symbol = self.add_data(BrainCompanyFilingLanguageMetricsAll, self.aapl).symbol
```

Accessing Data

To get the current Brain Language Metrics on Company Filings data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_10k_symbol):
        data_point = slice[self.dataset_10k_symbol]
        self.log(f"{self.dataset_10k_symbol} report sentiment at {slice.time}: {data_point.report_sentiment.sentiment}")

    if slice.contains_key(self.dataset_all_symbol):
        data_point = slice[self.dataset_all_symbol]
        self.log(f"{self.dataset_all_symbol} report sentiment at {slice.time}: {data_point.report_sentiment.sentiment}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(BrainCompanyFilingLanguageMetrics10K).items():
        self.log(f"{dataset_symbol} report sentiment at {slice.time}: {data_point.report_sentiment.sentiment}")

    for dataset_symbol, data_point in slice.get(BrainCompanyFilingLanguageMetricsAll).items():
        self.log(f"{dataset_symbol} report sentiment at {slice.time}: {data_point.report_sentiment.sentiment}")
```

Historical Data

To get historical Brain Language Metrics on Company Filings data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrames
ten_k_history_df = self.history(self.dataset_10k_symbol, 100, Resolution.DAILY)
all_history_df = self.history(self.dataset_all_symbol, 100, Resolution.DAILY)
history_df = self.history([self.dataset_10k_symbol, self.dataset_all_symbol], 100, Resolution.DAILY)

# Dataset objects
ten_k_history_bars = self.history[BrainCompanyFilingLanguageMetrics10K](self.dataset_10k_symbol, 100, Resolution.DAILY)
all_history_bars = self.history[BrainCompanyFilingLanguageMetricsAll](self.dataset_all_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Brain Language Metrics on Company Filings data, call the `add_universe` method with the `BrainCompanyFilingLanguageMetricsUniverseAll` class or the `BrainCompanyFilingLanguageMetricsUniverse10K` class and a selection function.

```py
def initialize(self) -> None:
    self._universe = self.add_universe(BrainCompanyFilingLanguageMetricsUniverseAll, self.universe_selection)

def universe_selection(self, alt_coarse: List[BrainCompanyFilingLanguageMetricsUniverseAll]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
                if d.report_sentiment.sentiment > 0 \
                and d.management_discussion_analysis_of_financial_condition_and_results_of_operations.sentiment > 0]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the universe object attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of the universe objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), universeDay in universe_history.items():
    for language_metrics in universeDay:
        self.log(f"{language_metrics.symbol} sentiment at {language_metrics.end_time}: {language_metrics.report_sentiment.sentiment}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the universe object attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of the universe objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), universeDay in universe_history.items():
    for language_metrics in universeDay:
        print(f"{language_metrics.symbol} sentiment at {language_metrics.end_time}: {language_metrics.report_sentiment.sentiment}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```py
self.remove_security(self.dataset_10k_symbol)
self.remove_security(self.dataset_all_symbol)
```

If you subscribe to Brain Language Metrics on Company Filings data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Brain Language Metrics on Company Filings dataset enables you to test strategies using language metrics and their differences gathered from 10K and 10Q reports. Examples include the following strategies:

- Using the similarity among reports to determine position sizing of securities. Some examples are discussed in Lazy Prices, Cohen et al. 2018 and The Positive Similarity of Company Filings and the Cross-section of Stock Returns, M. Padyšák 2020 .
- Using the sentiment of the latest report to determine the portfolio allocation to give to each security in the universe.
- Using levels of uncertainty, readability, or litigious language in the report to determine position sizing of securities.

Classic Algorithm Example

The following example algorithm buys Apple when the sentiment of their 10K report is positive. Otherwise, it holds cash.

```py
from AlgorithmImports import *


class BrainCompanyFilingNLPDataAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Requesting data -- we aim to obtain a sentiment score from the company filings
        # Combining both fundamental and sentiment factor, as well as past performance and future provision
        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(BrainCompanyFilingLanguageMetrics10K , self._symbol).symbol

        # Historical data
        history = self.history(self.dataset_symbol, 365, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for {self.dataset_symbol}")

    def on_data(self, data):
        # Trade base on the updated report sentiment
        if data.contains_key(self.dataset_symbol):
            sentiment = data[self.dataset_symbol].report_sentiment.sentiment
            # Buy for a positive sentiment score for the positive return projection
            self.set_holdings(self._symbol, int(sentiment > 0))
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities that have positive sentiment in their 10K report and then forms an equal-weighted portfolio:

```
PY
from AlgorithmImports import *


class BrainCompanyFilingNLPDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.free_portfolio_value_percentage = 0.1
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        # Filter based on company filing data
        self.add_universe(BrainCompanyFilingLanguageMetricsUniverse10K, self.universe_selection)

        self.add_alpha(BrainCompanyFilingNLPAlphaModel())

        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, alt_coarse: List[BrainCompanyFilingLanguageMetricsUniverse10K]) -> List[Symbol]:
        # Filter for the stocks with positive sentiment score for the positive return projection
        # Combining both fundamental and sentiment factor, as well as past performance and future provision
        return [d.symbol for d in alt_coarse \
                if d.report_sentiment.sentiment is not None and d.report_sentiment.sentiment > 0]

class BrainCompanyFilingNLPAlphaModel(AlphaModel):

    _securities = []

    def update(self, algorithm: QCAlgorithm, slice: Slice):
        # Signal to invest in the selected universe, which are expected to have a positive return due to positive fundamentals and future
prospect
        return [
            Insight.price(security.symbol, timedelta(days=1), InsightDirection.UP)
            for security in self._securities
            if security.price and security.is_tradable
        ]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self._securities.append(security)
        for security in changes.removed_securities:
            if security in self._securities:
                self._securities.remove(security)
```

Research Example

The following example lists US Equities having the highest 2-day rank.

```
PY
qb = QuantBook()

# Requesting Data
aapl = qb.AddEquity("AAPL", Resolution.Daily).Symbol
symbol = qb.AddData(BrainCompanyFilingLanguageMetrics10K, aapl).Symbol

# Historical data
history = qb.History(BrainCompanyFilingLanguageMetrics10K, symbol, 180, Resolution.Daily)
for (symbol, time), row in history.iterrows():
    print(f"{symbol} sentiment at {time}: {row['reportsentiment'].Sentiment}")

# Add Universe Selection
def UniverseSelection(alt_coarse: List[BrainCompanyFilingLanguageMetricsUniverse10K]) -> List[Symbol]:
    return [d.Symbol for d in sorted([x for x in alt_coarse if x.ReportSentiment.Sentiment],
        key=lambda x: x.ReportSentiment.Sentiment, reverse=True)[:10]]

universe = qb.AddUniverse(BrainCompanyFilingLanguageMetricsUniverse10K, UniverseSelection)

# Historical Universe data
universe_history = qb.UniverseHistory(universe, qb.Time-timedelta(10), qb.Time)
for (_, time), universeDay in universe_history.items():
    for language_metrics in universeDay:
        print(f"{language_metrics.Symbol} sentiment at {language_metrics.EndTime}: {language_metrics.ReportSentiment.Sentiment}")
```

Data Point Attributes

The Brain Language Metrics on Company Filings dataset provides `BrainCompanyFilingLanguageMetrics` and `BrainCompanyFilingLanguageMetricsUniverse` objects.

BrainCompanyFilingLanguageMetrics Attributes

`BrainCompanyFilingLanguageMetrics` objects have the following attributes:

BrainCompanyFilingLanguageMetricsUniverse Attributes

`BrainCompanyFilingLanguageMetricsUniverse` objects have the following attributes:

Brain

Brain ML Stock Ranking

Introduction

The Brain ML Stock Ranking dataset by Brain generates a daily ranking for US Equities based on their predicted ranking of future returns relative to the universe median across four-time horizons: next 2, 3, 5, 10, and 21 days (one trading month). The data covers 1,000 US Equities (universe updated yearly by including the largest 1,000 US companies of the previous year), starts in January 2010, and is delivered on a daily frequency. This dataset is created by a voting scheme of machine learning classifiers that non-linearly combine a variety of features with a series of techniques aimed at mitigating the well-known overfitting problem for financial data with a low signal-to-noise ratio. Examples of features are time-varying stock-specific features like price and volume-related metrics or fundamentals; time-fixed stock-specific features like the sector and other database information; market regime features such as volatility and other financial stress indicators; calendar features representing possible anomalies, for example, the month of the year.

More precisely the ML Stock Ranking score is related to the confidence of a Machine Learning classifier in predicting top or bottom quintile returns for the next N trading days (e.g. next 21 trading days) for a stock with the respect to the median of the universe and ranges from -1 to +1.

A negative score means that the system is more confident that the stock belongs to the lower returns quintile, a positive score means that the system is more confident that the stock belongs to the higher returns quintile. It is important to note that the score has a meaning only if used to compare different stocks to perform a ranking.

Typical use is to download the score for a large stock universe for a given day, e.g. 500 stocks or the full universe of 1000 stocks, order the stocks by mlAlpha score and go long the top K stocks, or build a long-short strategy going long the top K and short the bottom K stocks.

For more information, refer to Brain's summary paper .

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Brain ML Stock Ranking dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Brain is a Research Company that creates proprietary datasets and algorithms for investment strategies, combining experience in financial markets with strong competencies in Statistics, Machine Learning, and Natural Language Processing. The founders share a common academic background of research in Physics as well as extensive experience in Financial markets.

Getting Started

The following snippet demonstrates how to request data from the Brain ML Stock Ranking dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(BrainStockRanking2Day, self.aapl).symbol

self._universe = self.add_universe(BrainStockRankingUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2010
Asset Coverage*	1,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

The coverage includes all assets since the start date. It increases over time.

Requesting Data

To add Brain ML Stock Ranking data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class BrainMLRankingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 7, 8)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.two_day_symbol = self.add_data(BrainStockRanking2Day, self.aapl).symbol
        self.three_day_symbol = self.add_data(BrainStockRanking3Day, self.aapl).symbol
        self.five_day_symbol = self.add_data(BrainStockRanking5Day, self.aapl).symbol
        self.ten_day_symbol = self.add_data(BrainStockRanking10Day, self.aapl).symbol
        self.month_symbol = self.add_data(BrainStockRanking21Day, self.aapl).symbol
```

Accessing Data

To get the current Brain ML Stock Ranking data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.two_day_symbol):
        data_point = slice[self.two_day_symbol]
        self.log(f"{self.two_day_symbol} rank at {slice.time}: {data_point.rank}")

    if slice.contains_key(self.three_day_symbol):
        data_point = slice[self.three_day_symbol]
        self.log(f"{self.three_day_symbol} rank at {slice.time}: {data_point.rank}")

    if slice.contains_key(self.five_day_symbol):
        data_point = slice[self.five_day_symbol]
        self.log(f"{self.five_day_symbol} rank at {slice.time}: {data_point.rank}")

    if slice.contains_key(self.ten_day_symbol):
        data_point = slice[self.ten_day_symbol]
        self.log(f"{self.ten_day_symbol} rank at {slice.time}: {data_point.rank}")

    if slice.contains_key(self.month_symbol):
        data_point = slice[self.month_symbol]
        self.log(f"{self.month_symbol} rank at {slice.time}: {data_point.rank}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(BrainStockRanking2Day).items():
        self.log(f"{dataset_symbol} rank at {slice.time}: {data_point.rank}")

    for dataset_symbol, data_point in slice.get(BrainStockRanking3Day).items():
        self.log(f"{dataset_symbol} rank at {slice.time}: {data_point.rank}")

    for dataset_symbol, data_point in slice.get(BrainStockRanking5Day).items():
        self.log(f"{dataset_symbol} rank at {slice.time}: {data_point.rank}")

    for dataset_symbol, data_point in slice.get(BrainStockRanking10Day).items():
        self.log(f"{dataset_symbol} rank at {slice.time}: {data_point.rank}")

    for dataset_symbol, data_point in slice.get(BrainStockRanking21Day).items():
        self.log(f"{dataset_symbol} rank at {slice.time}: {data_point.rank}")
```

Historical Data

To get historical Brain ML Stock Ranking data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrames
two_day_history_df = self.history(self.two_day_symbol, 100, Resolution.DAILY)
three_day_history_df = self.history(self.three_day_symbol, 100, Resolution.DAILY)
five_day_history_df = self.history(self.five_day_symbol, 100, Resolution.DAILY)
ten_day_history_df = self.history(self.ten_day_symbol, 100, Resolution.DAILY)
thirty_day_history_df = self.history(self.month_symbol, 100, Resolution.DAILY)
history_df = self.history([self.two_day_symbol,
                           self.three_day_symbol,
                           self.five_day_symbol,
                           self.ten_day_symbol,
                           self.month_symbol], 100, Resolution.DAILY)

# Dataset objects
two_day_history_bars = self.history[BrainStockRanking2Day](self.two_day_symbol, 100, Resolution.DAILY)
three_day_history_bars = self.history[BrainStockRanking3Day](self.three_day_symbol, 100, Resolution.DAILY)
five_day_history_bars = self.history[BrainStockRanking5Day](self.five_day_symbol, 100, Resolution.DAILY)
ten_day_history_bars = self.history[BrainStockRanking10Day](self.ten_day_symbol, 100, Resolution.DAILY)
month_history_bars = self.history[BrainStockRanking21Day](self.month_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Brain ML Stock Ranking data, call the `add_universe` method with the `BrainStockRankingUniverse` class and a selection function.

```
def initialize(self) -> None:
    self._universe = self.add_universe(BrainStockRankingUniverse, self.universe_selection)

def universe_selection(self, alt_coarse: List[BrainStockRankingUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
                if d.rank2_days > 0 \
                and d.rank3_days > 0 \
                and d.rank5_days > 0]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
PY

# DataFrame example where the columns are the BrainStockRankingUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of BrainStockRankingUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), ranks in universe_history.items():
    for rank in ranks:
        self.log(f"{rank.symbol} 2-day rank at {rank.end_time}: {rank.rank2_days}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
PY

# DataFrame example where the columns are the BrainStockRankingUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of BrainStockRankingUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), ranks in universe_history.items():
    for rank in ranks:
        print(f"{rank.symbol} 2-day rank at {rank.end_time}: {rank.rank2_days}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY

self.remove_security(self.two_day_symbol)
self.remove_security(self.three_day_symbol)
self.remove_security(self.five_day_symbol)
self.remove_security(self.ten_day_symbol)
self.remove_security(self.month_symbol)
```

If you subscribe to Brain ML Stock Ranking data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Brain ML Stock Ranking dataset enables you to test strategies using the machine learning ranking provided by Brain. Examples include the following strategies:

- Constructing a portfolio of securities with each security's weight in the portfolio reflecting its Brain ML Stock Ranking
- Buying stocks with the largest Brain ML Stock Ranking
- Building a market-neutral strategy based on the top N and bottom N stocks in the Brain ML Stock Ranking

Classic Algorithm Example

The following example algorithm constructs a portfolio where the weight of each security in the portfolio is scaled based on its Brain ML Ranking. It gives a larger allocation to securities that have a higher Brain ML Ranking.

```py
from AlgorithmImports import *


class BrainMLRankingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # We cherry picked 5 largest stocks, high trading volume provides better information and credibility for ML ranking
        tickers = ["AAPL", "TSLA", "MSFT", "F", "KO"]
        self.symbol_by_dataset_symbol = {}
        for ticker in tickers:
            # Requesting data to get 2 days estimated relative ranking
            symbol = self.add_equity(ticker, Resolution.DAILY).symbol
            dataset_symbol = self.add_data(BrainStockRanking2Day, symbol).symbol
            self.symbol_by_dataset_symbol[dataset_symbol] = symbol

            # Historical data
            history = self.history(dataset_symbol, 365, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request for {symbol}")

    def on_data(self, slice: Slice) -> None:
        # Collect rankings for all selected symbols for ranking them
        points = slice.Get(BrainStockRanking2Day)
        if points is None:
            return
        symbols = []
        ranks = []
        for point in points.Values:
            symbols.append(self.symbol_by_dataset_symbol[point.symbol])
            ranks.append(point.rank)

        # Rank each symbol's Brain ML ranking relative to each other for positional sizing
        if len(ranks) == 0:
            return
        ranks = [sorted(ranks).index(rank) + 1 for rank in ranks]

        # Place orders according to the ML ranking, the better the rank, the higher the estimated return and hence weight
        for i, rank in enumerate(ranks):
            weight = rank / sum(ranks)
            self.set_holdings(symbols[i], weight)
```

Framework Algorithm Example

The following example algorithm constructs a portfolio where the weight of each security in the portfolio is scaled based on its Brain ML Ranking. It gives a larger allocation to securities that have a higher Brain ML Ranking.

PY

```
from AlgorithmImports import *


class BrainMLRankingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY
        # Select based on ML ranking data
        self.add_universe(BrainStockRankingUniverse, self.universe_selection)

        self.add_alpha(BrainMLRankingAlphaModel())

        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, alt_coarse: List[BrainStockRankingUniverse]) -> List[Symbol]:
        # Select the ones that expected to out-perform the median of the whole market in 2-3 days
        return [d.symbol for d in alt_coarse \
                if d.Rank2Days > 0.2  \
                and d.Rank3Days > 0.2]

class BrainMLRankingAlphaModel(AlphaModel):

    symbol_data_by_symbol = {}
    symbol_by_dataset_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Collect rankings for all selected symbols for ranking them
        points = slice.Get(BrainStockRanking2Day)
        if points is None:
            return []
        symbols = []
        ranks = []
        for point in points.Values:
            symbols.append(self.symbol_by_dataset_symbol[point.symbol])
            ranks.append(point.rank)

        # Rank each symbol's Brain ML ranking relative to each other for positional sizing
        if len(ranks) == 0:
            return []
        ranks = [sorted(ranks).index(rank) + 1 for rank in ranks]

        # Place orders according to the ML ranking, the better the rank, the higher the estimated return and hence weight
        for i, rank in enumerate(ranks):
            weight = rank / sum(ranks)
            insights.append(Insight.price(symbols[i], timedelta(days=7), InsightDirection.UP, None, None, None, weight))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            symbol_data = SymbolData(algorithm, symbol)
            self.symbol_data_by_symbol[symbol] = symbol_data
            self.symbol_by_dataset_symbol[symbol_data.dataset_symbol] = symbol

        for security in changes.removed_securities:
            symbol_data = self.symbol_data_by_symbol.pop(security.symbol, None)
            if symbol_data:
                symbol_data.dispose()

            for dataset_symbol, symbol in self.symbol_by_dataset_symbol.items():
                if symbol == security.symbol:
                    self.symbol_by_dataset_symbol.pop(dataset_symbol)
                    break


class SymbolData:
    def __init__(self, algorithm, symbol):
        self.algorithm = algorithm

        # Requesting data to get 2 days estimated relative ranking
        self.dataset_symbol = algorithm.add_data(BrainStockRanking2Day, symbol).symbol

        # Historical data
        history = algorithm.history(self.dataset_symbol, 365, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request for {symbol}")

    def dispose(self):
        # Unsubscribe from the Brain ML Ranking feed for this security to release computation resources
        self.algorithm.remove_security(self.dataset_symbol)
```

Research Example

The following example lists US Equities having the highest 2-day rank.

```py
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(BrainStockRanking2Day, aapl).symbol

# Historical data
history = qb.history(BrainStockRanking2Day, symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f"{symbol} rank at {time}: {row['rank']}")

# Add Universe Selection
def universe_selection(alt_coarse: List[BrainStockRankingUniverse]) -> List[Symbol]:
    return [d.symbol for d in sorted(alt_coarse, key=lambda x: x.Rank2Days, reverse=True)[:10]]

universe = qb.add_universe(BrainStockRankingUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(5), qb.time)
for (_, time), ranks in universe_history.items():
    for rank in ranks:
        print(f"{rank.symbol} 2-day rank at {rank.end_time}: {rank.Rank2Days}")
```

Data Point Attributes

The Brain ML Stock Ranking dataset provides `BrainStockRankingBase` and `BrainStockRankingUniverse` objects.

BrainStockRankingBase Attributes

`BrainStockRankingBase` objects have the following attributes:

BrainStockRankingUniverse Attributes

`BrainStockRankingUniverse` objects have the following attributes:

Brain

Brain Sentiment Indicator

Introduction

The Brain Sentiment Indicator dataset by Brain tracks the public sentiment around US Equities. The data covers 4,500 US Equities, starts in August 2016, and is delivered on a daily frequency. This dataset is created by analyzing financial news using Natural Language Processing techniques while taking into account the similarity and repetition of news on the same topic. The sentiment score assigned to each stock ranges from -1 (most negative) to +1 (most positive). The sentiment score corresponds to the average sentiment for each piece of news. The score is updated daily and is available on two time scales: 7 days and 30 days. For more information, see Brain's summary paper .

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Brain Sentiment Indicator dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Brain is a Research Company that creates proprietary datasets and algorithms for investment strategies, combining experience in financial markets with strong competencies in Statistics, Machine Learning, and Natural Language Processing. The founders share a common academic background of research in Physics as well as extensive experience in Financial markets.

Getting Started

The following snippet demonstrates how to request data from the Brain Sentiment Indicator dataset:

```
PY
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_7day_symbol = self.add_data(BrainSentimentIndicator7Day, self.aapl).symbol
self.dataset_30day_symbol = self.add_data(BrainSentimentIndicator30Day, self.aapl).symbol

self._universe = self.add_universe(BrainSentimentIndicatorUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	August 2016
Asset Coverage*	4,500 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

The coverage includes all assets since the start date. It increases over time.

Requesting Data

To add Brain Sentiment Indicator data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class BrainSentimentDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2021, 7, 8)
        self.set_cash(100000)

        symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_7day_symbol = self.add_data(BrainSentimentIndicator7Day, symbol).symbol
        self.dataset_30day_symbol = self.add_data(BrainSentimentIndicator30Day, symbol).symbol
```

Accessing Data

To get the current Brain Sentiment Indicator data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_7day_symbol):
        data_point = slice[self.dataset_7day_symbol]
        self.log(f"{self.dataset_7day_symbol} sentiment at {slice.time}: {data_point.sentiment}")

    if slice.contains_key(self.dataset_30day_symbol):
        data_point = slice[self.dataset_30day_symbol]
        self.log(f"{self.dataset_30day_symbol} sentiment at {slice.time}: {data_point.sentiment}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(BrainSentimentIndicator7Day).items():
        self.log(f"{dataset_symbol} sentiment at {slice.time}: {data_point.sentiment}")

    for dataset_symbol, data_point in slice.get(BrainSentimentIndicator30Day).items():
        self.log(f"{dataset_symbol} sentiment at {slice.time}: {data_point.sentiment}")
```

Historical Data

To get historical Brain Sentiment Indicator data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrames
week_history_df = self.history(self.dataset_7day_symbol, 100, Resolution.DAILY)
month_history_df = self.history(self.dataset_30day_symbol, 100, Resolution.DAILY)
history_df = self.history([self.dataset_7day_symbol, self.dataset_30day_symbol], 100, Resolution.DAILY)

# Dataset objects
week_history_bars = self.history[BrainSentimentIndicator7Day](self.dataset_7day_symbol, 100, Resolution.DAILY)
month_history_bars = self.history[BrainSentimentIndicator30Day](self.dataset_30day_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Brain Sentiment Indicator data, call the `add_universe` method with the `BrainSentimentIndicatorUniverse` class and a selection function.

```
def initialize(self) -> None:
    self._universe = self.add_universe(BrainSentimentIndicatorUniverse, self.universe_selection)

def universe_selection(self, alt_coarse: List[BrainSentimentIndicatorUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
                if d.total_article_mentions7_days > 0 \
                and d.sentiment7_days]
```

The Brain Sentiment Indicator universe runs at 7 AM Eastern Time (ET) in live trading. For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame example where the columns are the BrainSentimentIndicatorUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of BrainSentimentIndicatorUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), sentiments in universe_history.items():
    for sentiment in sentiments:
        self.log(f"{sentiment.symbol} 7-day sentiment at {sentiment.end_time}: {sentiment.sentiment7_days}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame example where the columns are the BrainSentimentIndicatorUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of BrainSentimentIndicatorUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), sentiments in universe_history.items():
    for sentiment in sentiments:
        print(f"{sentiment.symbol} 7-day sentiment at {sentiment.end_time}: {sentiment.sentiment7_days}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.dataset_7day_symbol)
self.remove_security(self.dataset_30day_symbol)
```

If you subscribe to Brain Sentiment Indicator data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Brain Sentiment Indicator dataset enables you to incorporate sentiment from financial news sources into your strategies. Examples include the following strategies:

- Buying when the public sentiment for a security is increasing
- Short selling when the public sentiment for a security is decreasing
- Scaling the position sizing of securities based on how many times they are mentioned in financial news articles
- Sector rotation based on news sentiment

Classic Algorithm Example

The following example algorithm buys Apple when the 30-day Brain Sentiment indicator increases. Otherwise, it remains in cash.

```
from AlgorithmImports import *


class BrainSentimentDataAlgorithm(QCAlgorithm):

    latest_sentiment_value = None
    target_holdings = 0

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting the processed longer term (30-day) sentiment score data for sentiment trading
        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(BrainSentimentIndicator30Day, self.aapl).symbol

        # Historical data
        history = self.history(self.dataset_symbol, 100, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for {self.dataset_symbol}")
        if history.empty:
            return

        # Warm up historical sentiment values, cache for comparing last sentiment score to trade, making it immediately tradable signal
        previous_sentiment_values = history.loc[self.dataset_symbol].sentiment.values
        for sentiment in previous_sentiment_values:
            self.update(sentiment)

    def update(self, sentiment: float) -> None:
        # Comparing the last sentiment score and decide to buy if the sentiment increases to ride the popularity
        if self.latest_sentiment_value is not None:
            self.target_holdings = int(sentiment > self.latest_sentiment_value)
        self.latest_sentiment_value = sentiment

    def on_data(self, slice: Slice) -> None:
        # Update trade direction based on updated data
        if slice.contains_key(self.dataset_symbol):
            sentiment = slice[self.dataset_symbol].sentiment
            self.update(sentiment)

        # Ensure we have security data in the current slice to avoid stale fill
        if not (slice.contains_key(self.aapl) and slice[self.aapl] is not None):
            return

        # Buy if sentiment increase, liquidate otherwise to ride on the popularity of the equity
        if self.target_holdings != self.portfolio.invested:
            self.set_holdings(self.aapl, self.target_holdings)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities that have been mentioned in an article over the last seven days. It then buys the subset of Equities that have increasing sentiment and forms an equal-weighted portfolio.

```
from AlgorithmImports import *


class BrainSentimentDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 10)
        self.set_cash(100000)

        self.settings.minimum_order_margin_portfolio_percentage = 0
        self.universe_settings.resolution = Resolution.DAILY
        # Filter base on sentiment data
        self.add_universe(BrainSentimentIndicatorUniverse, self.universe_selection)

        self.add_alpha(BrainSentimentAlphaModel())

        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, alt_coarse: List[BrainSentimentIndicatorUniverse]) -> List[Symbol]:
        # Filter for any sentiment on last 7 days to trade on sentiment news
        return [d.symbol for d in alt_coarse \
        if d.SentimentalArticleMentions7Days is not None and d.SentimentalArticleMentions7Days > 0]


class BrainSentimentAlphaModel(AlphaModel):

    symbol_data_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []
```

```
        insights = []

        for symbol, symbol_data in self.symbol_data_by_symbol.items():
            # Update trade direction based on updated data
            if slice.contains_key(symbol_data.dataset_symbol) and slice[symbol_data.dataset_symbol] is not None:
                sentiment = slice[symbol_data.dataset_symbol].sentiment
                symbol_data.update(sentiment)

            # Ensure we have security data in the current slice to avoid stale fill
            if not (slice.contains_key(symbol) and slice[symbol] is not None):
                continue

            # Buy if sentiment increase, liquidate otherwise to ride on the popularity of the equity
            if symbol_data.target_direction == InsightDirection.UP != algorithm.portfolio[symbol].invested:
                insights.append(Insight.price(symbol, timedelta(days=100), symbol_data.target_direction))
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            self.symbol_data_by_symbol[symbol] = SymbolData(algorithm, symbol)

        for security in changes.removed_securities:
            symbol_data = self.symbol_data_by_symbol.pop(security.symbol, None)
            if symbol_data:
                symbol_data.dispose()


class SymbolData:

    target_direction = InsightDirection.FLAT
    _latest_sentiment_value = None

    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        self.algorithm = algorithm

        # Requesting the processed longer term (30-day) sentiment score data for sentiment trading
        self.dataset_symbol = algorithm.add_data(BrainSentimentIndicator30Day, symbol).symbol

        # Historical data
        history = algorithm.history(self.dataset_symbol, 100, Resolution.DAILY)
        if history.empty or 'sentiment' not in history.columns:
            return

        # Warm up historical sentiment values, cache for comparing last sentiment score to trade, making it immediately tradable signal
        previous_sentiment_values = history.loc[self.dataset_symbol].sentiment.values
        for sentiment in previous_sentiment_values:
            self.update(sentiment)

    def dispose(self) -> None:
        # Unsubscribe from the Brain Sentiment feed for this security to release computational resources
        self.algorithm.remove_security(self.dataset_symbol)

    def update(self, sentiment: float) -> None:
        # Comparing the last sentiment score and decide to buy if the sentiment increases to ride the popularity
        if self._latest_sentiment_value is not None:
            if sentiment > self._latest_sentiment_value:
                self.target_direction = InsightDirection.UP
            else:
                self.target_direction = InsightDirection.FLAT
        self._latest_sentiment_value = sentiment
```

Research Example

The following example lists US Equities having the highest 7-day sentiment.

```
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(BrainSentimentIndicator30Day, aapl).symbol

# Historical data
history = qb.history(BrainSentimentIndicator30Day, symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f"{symbol} sentiment at {time}: {row['sentiment']}")

# Add Universe Selection
def universe_selection(alt_coarse: List[BrainSentimentIndicatorUniverse]) -> List[Symbol]:
    return [d.symbol for d in sorted([x for x in alt_coarse if x.SentimentalArticleMentions7Days],
        key=lambda x: x.SentimentalArticleMentions7Days, reverse=True)[:10]]

universe = qb.add_universe(BrainSentimentIndicatorUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(5), qb.time)
for (_, time), sentiments in universe_history.items():
    for sentiment in sentiments:
        print(f"{sentiment.symbol} 7-day sentiment at {sentiment.end_time}: {sentiment.SentimentalArticleMentions7Days}")
```

Data Point Attributes

The Brain Sentiment Indicator dataset provides `BrainSentimentIndicatorBase` and `BrainSentimentIndicatorUniverse` objects.

BrainSentimentIndicatorBase Attributes

`BrainSentimentIndicatorBase` objects have the following attributes:

BrainSentimentIndicatorUniverse Attributes

`BrainSentimentIndicatorUniverse` objects have the following attributes:

Brain

Brain Wikipedia Page Views

Introduction

The Brain Wikipedia Page Views dataset by Brain tracks investor attention toward large U.S. equities by monitoring traffic on company Wikipedia pages. The dataset covers approximately 1,000 of the largest U.S. companies, broadly corresponding to the Russell 1000, and is available starting from February 2023. Data is delivered at a daily frequency.

The dataset is constructed by analyzing the number of views on each company's Wikipedia page, including both the main page and any secondary pages that redirect to it. Only English-language Wikipedia pages are considered. The data is published daily, typically by 12:00 PM UTC, and reflects attention metrics calculated relative to the reporting date.

In addition to raw page-view counts, the dataset includes buzz metrics, which normalize current attention levels relative to historical behavior. Buzz scores are computed as the distance between current page views and the company's own historical average over the previous six months, expressed in units of standard deviations. As a result, positive buzz values indicate unusually high attention, while negative values indicate lower-than-normal attention.

Metrics are provided across multiple time horizons, including 1-day, 7-day, and 30-day rolling windows, enabling users to study short-term spikes in attention as well as more persistent trends. These features make the dataset well-suited for research into investor attention, behavioral finance signals, event-driven strategies, and alpha models that incorporate information demand.

This dataset depends on the US Equity Security Master dataset, which provides essential information such as symbol mappings, corporate actions, and identifier consistency required for accurate historical analysis and backtesting.

For more information about the Brain Wikipedia Page Views dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Brain is a Research Company that creates proprietary datasets and algorithms for investment strategies, combining experience in financial markets with strong competencies in Statistics, Machine Learning, and Natural Language Processing. The founders share a common academic background of research in Physics as well as extensive experience in Financial markets.

Getting Started

The following snippet demonstrates how to request data from the Brain Wikipedia Page Views dataset:

```PY
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(BrainWikipediaPageViews, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	February 2023
Asset Coverage*	~1,000 US Equities (approximately Russell 1000)
Data Density	Sparse
Resolution	Daily
Timezone	UTC

* The coverage includes the largest U.S. equities since the start date and may evolve over time.

Requesting Data

To add Brain Wikipedia Page Views Indicator data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class BrainWikipediaPageViews(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 7, 8)
        self.set_cash(100000)
        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(BrainWikipediaPageViews, self.aapl).symbol
```

Accessing Data

To get the current Brain Wikipedia Page Views Indicator data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} sentiments at {slice.time}: {data_point.NumberViews1}, {data_point.Buzz1}"
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(BrainWikipediaPageViews).items():
         self.log(f"{dataset_symbol} metric at {slice.time}: {data_point.NumberViews1} , {data_point.Buzz1}")
```

Historical Data

To get historical Brain Wikipedia Page Views data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)
```

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.dataset_symbol)
```

Example Applications

The Brain Wikipedia Page Views dataset enables you to incorporate investor attention signals derived from Wikipedia traffic into your strategies. Examples include the following strategies:

- Buying securities that experience unusually high spikes in Wikipedia page views, indicating elevated public interest.
- Avoiding or reducing exposure to securities with declining or abnormally low attention relative to their historical baseline.
- Scaling position sizes based on short-term or long-term attention intensity using page view and buzz metrics.
- Building cross-sectional ranking strategies that favor stocks with sustained increases in public visibility.

Classic Algorithm Example

The following example algorithm ranks large-cap U.S. stocks daily using Brain Wikipedia Page Views attention and buzz metrics, then allocates portfolio weights proportionally to each stock's relative attention score.

```py
from AlgorithmImports import *

class BrainWikipediaPageViewsRankingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2025, 1, 1)
        self.set_end_date(2025, 11, 23)
        self.set_cash(100000)

        # Seed prices so we don't get "no price" trading errors
        self.settings.seed_initial_prices = True

        # We cherry-picked 5 largest stocks, high trading volume provides better information and credibility for ML ranking
        tickers = ["AAPL", "TSLA", "MSFT", "F", "KO"]

        for ticker in tickers:
            # Requesting data to get 2 days estimated relative ranking
            equity = self.add_equity(ticker, Resolution.DAILY)
            equity.dataset_symbol = self.add_data(BrainWikipediaPageViews, equity.symbol).symbol

            # Historical data
            history = self.history(equity.dataset_symbol, 365, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request for {equity.symbol}")

    def on_data(self, slice: Slice) -> None:
        points = slice.get(BrainWikipediaPageViews)

        if not points:
            return

        scores = {}

        for point in points.Values:
            # Example composite signal:
            number_of_views = point.NumberViews1 if point.NumberViews1 else 0
            buzz1day = point.Buzz1 if point.Buzz1 else 0

            symbol = point.symbol.underlying
            scores[symbol] = buzz1day * math.sqrt(number_of_views)

        if len(scores) == 0:
            return

        # Rank by score (higher score = better)
        ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)

        # Convert ranks to weights (same style as your example)
        ranks = list(range(len(ranked), 0, -1))  # best gets highest rank number
        denom = sum(ranks)
        top = ", ".join([f"{sym.value}:{sc:.4f}" for sym, sc in ranked[:3]])
        self.debug(f"{self.time.date()} top: {top}")

        # Place orders according to the ML ranking, the better the rank, the higher the estimated return and hence weight
        for i, (sym, sc) in enumerate(ranked):
            w = ranks[i] / denom
            self.set_holdings(sym, w)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of liquid U.S. equities, ranks them daily using Brain Wikipedia Page Views attention and buzz metrics, and emits weighted bullish insights for the most attention-driven stocks using the Alpha Framework.

```py
from AlgorithmImports import *
import math

class BrainWikipediaPageViewsFrameworkRankingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2025, 1, 1)
        self.set_end_date(2025, 11, 23)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY

        # Universe: pick liquid names
        self.add_universe(self._select)
        alpha = BrainWikipediaPageViewsRankingAlpha()
        self.add_alpha(alpha)

        self.schedule.on(
            self.date_rules.every_day(),
            self.time_rules.at(9, 0),
            alpha.emit_insights
        )

        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())
        self.add_risk_management(NullRiskManagementModel())
        self.set_execution(ImmediateExecutionModel())
```

```
        self.set_warmup(30, Resolution.DAILY)

    def _select(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Liquid universe
        filtered = [c for c in fundamental if c.market_cap and c.price > 10]
        filtered.sort(key=lambda c: c.dollar_volume, reverse=True)

        # Keep it manageable
        return [c.symbol for c in filtered[:50]]


class BrainWikipediaPageViewsRankingAlpha(AlphaModel):
    def __init__(self):
        self.symbol_data_by_symbol = {}
        self.symbol_by_dataset_symbol = {}

        # Cache latest score per equity symbol
        self.latest_scores = {}     # Symbol -> float
        self.last_score_date = None # date of the latest cache refresh (algorithm.time.date())

        # Store algorithm reference for scheduled emission
        self.algorithm = None

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        self.algorithm = algorithm

        points = slice.get(BrainWikipediaPageViews)
        if points is None:
            return []

        for point in points.values():
            # Map dataset symbol -> equity symbol
            if point.symbol not in self.symbol_by_dataset_symbol:
                continue

            sym = self.symbol_by_dataset_symbol[point.symbol]

            number_of_views = point.NumberViews1 if point.NumberViews1 else 0
            buzz1day = point.Buzz1 if point.Buzz1 else 0

            score = buzz1day + math.sqrt(number_of_views)
            self.latest_scores[sym] = score

        self.last_score_date = algorithm.time.date()

        return []

    def emit_insights(self) -> None:
        algorithm = self.algorithm
        if not algorithm:
            return

        if self.last_score_date != algorithm.time.date():
            return

        paired = []
        for sym, sc in self.latest_scores.items():
            sec = algorithm.securities.get(sym)
            if sec is None or not sec.has_data or sec.price is None or sec.price <= 0:
                continue
            if sc == 0:
                continue
            paired.append((sym, sc))

        if not paired:
            return

        # Rank (higher is better)
        paired.sort(key=lambda x: x[1], reverse=True)

        n = len(paired)
        ranks = list(range(n, 0, -1))  # best gets largest rank
        denom = sum(ranks)

        insights = []
        for i, (sym, sc) in enumerate(paired):
            weight = ranks[i] / denom
            insights.append(
                Insight.price(sym, timedelta(days=7), InsightDirection.UP, None, None, None, weight)
            )

        # Emit insights into the framework
        algorithm.emit_insights(insights)

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            if symbol in self.symbol_data_by_symbol:
                continue

            sd = SymbolData(algorithm, symbol)
            self.symbol_data_by_symbol[symbol] = sd
            self.symbol_by_dataset_symbol[sd.dataset_symbol] = symbol
```

```
        for security in changes.removed_securities:
            symbol = security.symbol

            sd = self.symbol_data_by_symbol.pop(symbol, None)
            if sd is not None:
                sd.dispose()

            to_remove = None
            for ds_sym, eq_sym in self.symbol_by_dataset_symbol.items():
                if eq_sym == symbol:
                    to_remove = ds_sym
                    break
            if to_remove is not None:
                self.symbol_by_dataset_symbol.pop(to_remove, None)

            self.latest_scores.pop(symbol, None)


class SymbolData:
    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol):
        self.algorithm = algorithm

        # Subscribe to Brain Wikipedia Page Views metrics for this equity symbol
        self.dataset_symbol = algorithm.add_data(BrainWikipediaPageViews, symbol, Resolution.DAILY).symbol

        history = algorithm.history(self.dataset_symbol, 365, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request for {symbol}")

    def dispose(self):
        # Unsubscribe custom data subscription
        self.algorithm.remove_security(self.dataset_symbol)
```

Research Example

The following example retrieves historical Brain Wikipedia Page Views data for Apple and prints the daily number of Wikipedia page views over the past six months.

```
PY
qb = QuantBook()
# Requesting Data
aapl = qb.AddEquity("AAPL", Resolution.Daily).Symbol
symbol = qb.AddData(BrainWikipediaPageViews, aapl).Symbol

# Historical data
history = qb.History(BrainWikipediaPageViews, symbol, 180, Resolution.Daily)
for (symbol, time), row in history.iterrows():
    print(f"{symbol} views at {time}: {row['numberviews1']}")
```

Data Point Attributes

The Brain Wikipedia Page Views dataset provides `BrainWikipediaPageViews` objects.

`BrainWikipediaPageViews` objects have the following attributes:

Datasets

CoinGecko

CoinGecko was founded in 2014 by TM Lee (CEO) and Bobby Ong (COO) with the mission to democratize the access of crypto data and empower users with actionable insights. We also deep dive into the crypto space to deliver valuable insights to our users through our cryptocurrency reports, as well as our publications, newsletter, and more.

Crypto Market Cap

CoinGecko

Crypto Market Cap

Introduction

The Crypto Market Cap dataset by CoinGecko tracks the market cap of cryptocurrencies. The data covers 620 cryptocurrencies that are supported by QuantConnect, starts in 28 April 2013, and is delivered on a daily frequency. This dataset is created by scraping CoinGecko's Market Chart.

For more information about the Crypto Market Cap dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

CoinGecko was founded in 2014 by TM Lee (CEO) and Bobby Ong (COO) with the mission to democratize the access of crypto data and empower users with actionable insights. We also deep dive into the crypto space to deliver valuable insights to our users through our cryptocurrency reports, as well as our publications, newsletter, and more.

Getting Started

The following snippet demonstrates how to request data from the CoinGecko Crypto Market Cap dataset:

```
self.btc = self.add_data(CoinGecko, "BTC").symbol
self._universe = self.add_universe(CoinGeckoUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	28 April 2013
Asset Coverage	620 cryptocurrencies
Resolution	Daily
Timezone	UTC

Requesting Data

To add CoinGecko Crypto Market Cap data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
from AlgorithmImports import *

class CoinGeckoMarketCapDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.btcusd = self.add_crypto("BTCUSD", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(CoinGecko, "BTC").symbol
```

Accessing Data

To get the current Crypto Market Cap data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} market cap::volume at {slice.time}: {data_point.market_cap}::{data_point.volume}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(CoinGecko).items():
        self.log(f"{dataset_symbol} market cap::volume at {slice.time}: {data_point.market_cap}::{data_point.volume}")
```

Historical Data

To get historical CoinGecko Crypto Market Cap data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[CoinGecko](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of Cryptos based on CoinGecko Crypto Market Cap data, call the `add_universe` method with the `CoinGeckoUniverse` class and a selection function. Note that the filtered output is a list of names of the coins. If corresponding tradable crypto pairs are preferred, call `create_symbol(market, quoteCurrency)` method for each output item.

```
class CryptoMarketCapUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_account_currency("USD")
        self._market = Market.COINBASE
        self._market_pairs = [
            x.key.symbol
            for x in self.symbol_properties_database.get_symbol_properties_list(self._market)
            if x.value.quote_currency == self.account_currency
        ]
        self._universe = self.add_universe(CoinGeckoUniverse, 'CoinGeckoUniverse', Resolution.DAILY, self._select_assets)

    def _select_assets(self, data: List[CoinGecko]) -> List[Symbol]:
        for datum in data:
            self.debug(f'{datum.coin},{datum.market_cap},{datum.price}')

        # Select the coins that our brokerage supports and have a quote currency that matches
        # our account currency.
        tradable_coins = [d for d in data if d.coin + self.account_currency in self._market_pairs]
        # Select the largest coins and create their Symbol objects.
        return [
            c.create_symbol(self._market, self.account_currency)
            for c in sorted(tradable_coins, key=lambda x: x.market_cap)[-10:]
        ]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CoinGeckoUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of CoinGeckoUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), coins in universe_history.items():
    for coin in coins:
        self.log(f"{coin.symbol.value} market cap at {coin.end_time}: {coin.market_cap}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the CoinGeckoUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of CoinGeckoUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), coins in universe_history.items():
    for coin in coins:
        print(f"{coin.symbol.value} market cap at {coin.end_time}: {coin.market_cap}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove your subscription to CoinGecko Crypto Market Cap data, call the `remove_security` method.

```py
self.remove_security(self.dataset_symbol)
```

Supported Assets

The following table shows the available Cryptocurrencies:

Supported Cryptocurrency					
1INCH	AAA	AAPL	AAVE	ACA	ACH
ACM	ADA	ADX	AE	AERGO	AGIX
AGLD	AION	AKRO	ALBT	ALCX	ALEPH
ALGO	ALICE	ALPACA	ALPHA	ALPINE	AMB
AMC	AMP	AMPL	ANC	ANKR	ANT
ANY	APE	API3	APPC	APT	AR
ARDR	ARK	ARPA	ASD	ASM	ASR
AST	ASTR	ATA	ATLAS	ATM	ATOM
AUCTION	AUDIO	AURY	AUTO	AVA	AVAX
AVT	AXS	B21	BADGER	BAKE	BAL
BAND	BAO	BAR	BAT	BB	BCC
BCD	BCH	BCN	BCPT	BDOT	BEAM
BEL	BEST	BETA	BETH	BFT	BICO
BIDR	BIFI	BIT	BITO	BLT	BLZ

BMI	BNB	BNT	BNTX	BNX	BOBA
BOND	BOSON	BOT	BRD	BRL	BRZ
BSV	BSW	BTC	BTCB	BTCST	BTG
BTRST	BTS	BTSE	BTT	BULL	BURGER
BUSD	BVOL	BYND	BZRX	C98	CAKE
CCD	CDT	CEL	CELO	CELR	CFX
CGC	CHAT	CHESS	CHEX	CHR	CHSB
CHZ	CITY	CKB	CLO	CLOAK	CLV
CMT	CND	CNHT	COCOS	COIN	COMP
CONV	COPE	COS	COTI	COVAL	COVER
CQT	CREAM	CRO	CRON	CRV	CTK
CTSI	CTX	CTXC	CUSDT	CVC	CVP
CVX	DAI	DAR	DASH	DATA	DAWN
DCR	DDX	DEGO	DENT	DESO	DEXE
DF	DFL	DGB	DGD	DIA	DLT
DMG	DNT	DOCK	DODO	DOGE	DORA
DOT	DREP	DUSK	DVF	DYDX	EDEN
EGLD	ELF	EMB	ENG	ENJ	ENS
EOS	EPS	EPX	ERN	ESS	ETC
ETH	ETH2	ETHE	ETP	EURS	EURT
EVX	EWT	EXO	EZ	FARM	FB
FCL	FET	FIDA	FIL	FIO	FIRO
FIS	FLM	FLOW	FLUX	FOR	FORTH
FOX	FRONT	FTM	FTT	FUEL	FUN
FX	FXS	GAL	GALA	GAS	GENE
GFI	GHST	GLD	GLM	GLMR	GLXY
GME	GMT	GMX	GNO	GNT	GO
GODS	GOG	GOT	GRS	GRT	GT
GTC	GTO	GTX	GVT	GYEN	HARD
HBAR	HC	HEGIC	HFT	HGET	HIGH
HIVE	HMT	HNT	HOLY	HOOK	HOT
HT	HUM	HXRO	ICE	ICN	ICP
ICX	ID	IDEX	IDRT	ILV	IMX

INJ	INS	INTER	INV	IOST	IOTX
IQ	IRIS	JASMY	JET	JOE	JST
JUV	KAI	KAN	KAR	KAVA	KDA
KEEP	KEY	KIN	KLAY	KMD	KNC
KP3R	KRL	KSHIB	KSM	LAZIO	LCX
LDO	LEND	LEO	LEVER	LINA	LINK
LIT	LOKA	LOOKS	LOOM	LPT	LQTY
LRC	LSK	LTC	LTO	LUA	LUN
LUNA	LUNC	LYM	MAGIC	MANA	MAPS
MASK	MATH	MATIC	MBL	MBOX	MBS
MC	MCB	MCO	MCO2	MDA	MDT
MDX	MEDIA	MER	MFT	MIM	MINA
MIR	MITH	MKR	MLN	MNGO	MOB
MOD	MOVR	MPL	MSOL	MSTR	MTA
MTH	MTL	MULTI	MUSD	NAS	NAV
NBS	NCASH	NCT	NEAR	NEBL	NEO
NEXO	NKN	NMR	NPXS	NU	NULS
NXS	OAX	OCEAN	ODE	OG	OGN
OKB	OM	OMG	OMNI	ONE	ONG
ONT	OOKI	OP	ORBS	ORCA	ORN
ORS	OSMO	OST	OXT	OXY	PASS
PAXG	PEOPLE	PERL	PERP	PHA	PHB
PHX	PIVX	PLA	PLANETS	PLU	PNG
PNK	PNT	POA	POE	POLIS	POLS
POLY	POLYX	POND	PORT	PORTO	POWR
PPT	PRISM	PRO	PROM	PROS	PSG
PTU	PUNDIX	PYR	QI	QKC	QLC
QNT	QRDO	QSP	QTF	QTUM	QUICK
RAD	RAI	RAMP	RARE	RARI	RAY
RBN	RBTC	RCN	RDN	REAL	REEF
REI	REN	RENBTC	REP	REQ	RGT
RIF	RLC	RLY	RNDR	RON	ROOK
ROSE	RRT	RSR	RUNE	RVN	SALT

SAN	SAND	SANTOS	SC	SCRT	SFP
SGB	SHIB	SHPING	SKL	SKY	SLND
SLP	SLRS	SNGLS	SNM	SNT	SNX
SNY	SOL	SOS	SPARTA	SPELL	SPY
SRM	SSV	STARS	STEEM	STEP	STETH
STG	STMX	STORJ	STORM	STPT	STRAX
STSOL	STX	SUB	SUKU	SUN	SUPER
SUSD	SUSHI	SWRV	SXP	SYS	T
TBTC	TCT	TFUEL	THETA	TKO	TLM
TLOS	TNB	TNT	TOMO	TORN	TRAC
TRADE	TRB	TRIBE	TROY	TRU	TRX
TRYB	TSLA	TULIP	TUSD	TVK	TWT
UFT	UMA	UNFI	UNI	UOS	UPI
USDC	USDP	USDS	USDT	USTC	UTK
VAI	VEE	VELO	VEN	VET	VGX
VIA	VIB	VIBE	VIDT	VITE	VOXEL
VTHO	WABI	WAN	WAVES	WAXP	WBTC
WCFG	WILD	WIN	WING	WINGS	WNCG
WNDR	WNXM	WOO	WPR	WRX	WTC
XAUT	XCHF	XDC	XDG	XEC	XEM
XLM	XMR	XNO	XRA	XRD	XRP
XTZ	XVG	XVS	XYO	YFI	YFII
YGG	ZCN	ZEC	ZEN	ZIL	ZM
ZMT	ZRX				

Example Applications

The CoinGecko Crypto Market Cap dataset provides information on the size of the crypto coin and can be used to compare the size of one coin to another. Examples include the following strategies:

- Construct a major crypto index fund.
- Invest in the cryptos with the fastest growth in market size.
- Red flag stop when there might be a crypto bank run.

Classic Algorithm Example

The following example algorithm buy BTCUSD when the market cap of BTC is rising, while sell it when the market cap of BTC is dropping.

```py
from AlgorithmImports import *


class CoinGeckoAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request BTCUSD crypto data for trading
        self.crypto_symbol = self.add_crypto("BTCUSD").symbol
        # Request CoinGecko Market Cap data of BTC for trade signal generation
        self.custom_data_symbol = self.add_data(CoinGecko, "BTC").symbol
        # Use RollingWindow to save the last 2 market cap data for capital flow analysis
        self.window = RollingWindow(2)

    def on_data(self, slice: Slice) -> None:
        # Trade based on updated market cap data
        data = slice.get(CoinGecko)
        if data and self.custom_data_symbol in data:
            # Update RollingWindow for updated comparison
            self.window.add(data[self.custom_data_symbol])
            if not self.window.is_ready:
                return

            # Buy BTCUSD if the market cap of BTC is increasing, which suggests the capital flow towards BTC market and drive up the
demand
            if self.window[0].market_cap > self.window[1].market_cap:
                self.set_holdings(self.crypto_symbol, 1)
            # Sell otherwise, since the capital is flowing out and the demand of BTC lowered
            else:
                self.set_holdings(self.crypto_symbol, -0.5)

    def on_order_event(self, orderEvent: OrderEvent) -> None:
        if orderEvent.status == OrderStatus.FILLED:
            self.debug(f'Purchased Crypto: {orderEvent.symbol}')
```

Framework Algorithm Example

The following example algorithm buy BTCUSD when the market cap of BTC is rising, while sell it when the market cap of BTC is dropping with framework implementation.

```py
from AlgorithmImports import *


class CoinGeckoAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request CoinGecko Market Cap data of BTC for trade signal generation
        symbol = self.add_data(CoinGecko, "BTC").symbol
        # Request BTCUSD crypto data for trading
        crypto = self.add_crypto("BTCUSD", market=Market.COINBASE).symbol
        symbol_dict = {symbol: crypto}
        # Use RollingWindow to save the last 2 market cap data for capital flow analysis
        window = {symbol: RollingWindow[CoinGecko](2)}

        # Custom alpha model that emit insights based on updated market cap data
        self.add_alpha(CoinGeckoAlphaModel(symbol_dict, window))
        # Equal invest to evenly dissipate capital concentration risk from non-systematic individual risky event
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())


class CoinGeckoAlphaModel(AlphaModel):

    def __init__(self, symbol_dict: Dict[Symbol, Symbol], window: Dict[Symbol, RollingWindow[CoinGecko]]) -> None:
        self.symbol_dict = symbol_dict
        self.window = window

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade based on updated market cap data
        data = slice.Get(CoinGecko)
        for dataset_symbol, crypto_symbol in self.symbol_dict.items():
            if not data.contains_key(dataset_symbol):
                continue

            # Update RollingWindow for updated comparison
            self.window[dataset_symbol].add(data[dataset_symbol])

            # Can only compare market cap if the RollingWindow is ready
            window = self.window[dataset_symbol]
            if not window.is_ready:
                continue

            # Buy BTCUSD if the market cap of BTC is increasing, which suggests the capital flow towards BTC market and drive up the
demand
            if window[0].market_cap > window[1].market_cap:
                insight = Insight.price(crypto_symbol, timedelta(1), InsightDirection.UP, weight=1)
                insights.append(insight)
            # Sell otherwise, since the capital is flowing out and the demand of BTC lowered
            else:
                insight = Insight.price(crypto_symbol, timedelta(1), InsightDirection.DOWN, weight=0.25)
                insights.append(insight)

        return insights
```

Research Example

The following example lists US Equities having the highest 7-day sentiment.

```py
qb = QuantBook()

# Requesting Data
symbol = qb.add_data(CoinGecko, "BTC").symbol

# Historical data
history = qb.history(CoinGecko, symbol, 30, Resolution.DAILY)
for (symbol, time), row in history.iterrows():
    print(f"{symbol} sentiment at {time}: {row['marketcap']}")

# Add Universe Selection
def universe_selection(alt_coarse: List[CoinGeckoUniverse]) -> List[Symbol]:
    return [d.symbol for d in sorted([x for x in alt_coarse if x.market_cap],
        key=lambda x: x.market_cap, reverse=True)[:10]]

universe = qb.add_universe(CoinGeckoUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(10), qb.time)
for (_, time), coins in universe_history.items():
    for coin in coins:
        print(f"{coin.symbol.value} market cap at {coin.end_time}: {coin.market_cap}")
```

Data Point Attributes

The Crypto Market Cap dataset provides `CoinGecko` and `CoinGeckoUniverse` objects.

CoinGecko Attributes

`CoinGecko` objects have the following attributes:

CoinGeckoUniverse Attributes

`CoinGeckoUniverse` objects have the following attributes:

Datasets

EOD Historical Data

EODHD was a France financial data provider founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomy indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly.

Economic Events

Macroeconomics Indicators

Upcoming Dividends

Upcoming Earnings

Upcoming IPOs

Upcoming Splits

EOD Historical Data

Economic Events

Introduction

The Economic Events dataset, provided by EODHD, offers daily alerts for major economic events or announcements of global markets within the upcoming 7 days, with estimation and previous record if available. The data starts in January 2019, and is delivered on a daily frequency.

For more information about the Economic Events dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EODHD was a France financial data provider founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomy indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly.

Getting Started

The following snippet demonstrates how to request data from the Economic Events dataset:

```py
self.add_data(EODHDEconomicEvents, Country.UNITED_STATES)
self.add_data(EODHDEconomicEvents, EODHD.Events.UnitedStates.CPI)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2019
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Economic Events data to your algorithm, call the `add_data` method.

```py
class EconomicEventsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        ticker = EODHD.Events.UnitedStates.CPI
        # ticker = Country.UNITED_STATES
        self.dataset_symbol = self.add_data(EODHDEconomicEvents, ticker).symbol
```

Accessing Data

To get the current Economic Events data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    events = slice.get(EODHDEconomicEvents).get(self.dataset_symbol)
    for event in events:
        self.log(f"{event.event_type} of {event.country} over {event.event_period} with be held at {event.event_time} with estimated
value {event.estimate}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for symbol, events in slice.get(EODHDEconomicEvents).items():
        for event in events:
            self.log(f"{event.event_type} of {event.country} over {event.event_period} with be held at {event.event_time} with estimated
value {event.estimate}")
```

Historical Data

To get historical Economic Events data, call the `history` method with the type `EODHDEconomicEvents` cast and the underlying `Symbol` . If there is no data in the period you request, the history result is empty.

```
history_bars = self.history[EODHDEconomicEvents](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Economic Events dataset provides timely notifications about upcoming economic events, allowing traders to trade on that volatility. Examples include the following strategies:

- Long straddle to trade the volatility of the economic events.
- Trade market representative ETFs based on estimated figures for the upcoming events.
- Statistical arbitrage on two or more correlated global markets based on economic events of different locations.

Classic Algorithm Example

The following example algorithm trade Industrial sector ETF based on US PMI estimated change direction. If the estimated PMI of the upcoming announcement is above the previous PMI, buy the Industrial ETF, and sell it otherwise.

```py
from AlgorithmImports import *


class EODHDEconomicEventsAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Use industrial sector ETF as a vehicle to trade.
        self.equity_symbol = self.add_equity("XLI").symbol
        # Request US PMI economic event data to generate trade signals.
        ticker = EODHD.Events.UnitedStates.MARKIT_MANUFACTURING_PURCHASING_MANAGERS_INDEX
        self.dataset_symbol = self.add_data(EODHDEconomicEvents, ticker).symbol

    def on_data(self, slice):
        # Trade based on the updated economic events.
        if self.dataset_symbol in slice.get(EODHDEconomicEvents):
            # Use the Manufacturing Index to generate trade signals on manufacturing industry vehicles.
            # Make sure previous and estimate are available to estimate the direction of the industry.
            event = slice[self.dataset_symbol].data[0]
            if event.previous and event.estimate:
                # If the estimated PMI is higher than the previous PMI, the manufacturing ETF price is expected to rise.
                if event.previous > event.estimate:
                    self.set_holdings(self.equity_symbol, 1)
                # Otherwise, it is expected manufacturing ETF prices will drop.
                else:
                    self.set_holdings(self.equity_symbol, -1)
```

Framework Algorithm Example

The following example algorithm trade Industrial sector ETF based on US PMI estimated change direction using algorithm framework. If the estimated PMI of the upcoming announcement is above the previous PMI, buy the Industrial ETF, sell it otherwise.

```py
from AlgorithmImports import *


class EODHDEconomicEventsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Use industrial sector ETF as a vehicle to trade.
        symbol = Symbol.create("XLI", SecurityType.EQUITY, Market.USA)
        self.add_universe_selection(ManualUniverseSelectionModel(symbol))
        # Custom alpha model to emit insights according to economic events.
        self.add_alpha(EconomicEventAlphaModel(self))
        # Equal weighting position sizing to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_MONTH))

class EconomicEventAlphaModel(AlphaModel):
    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Request US PMI economic event data to generate trade signals.
        ticker = EODHD.Events.UnitedStates.MARKIT_MANUFACTURING_PURCHASING_MANAGERS_INDEX
        self.dataset_symbol = algorithm.add_data(EODHDEconomicEvents, ticker).symbol
        self._symbol = None

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Trade based on the updated economic events.
        if self.dataset_symbol not in slice.get(EODHDEconomicEvents):
            return []
        # Use the US Manufacturing Index to generate trade signals on manufacturing industry vehicles.
        # Make sure previous and estimate are available to estimate the direction of the industry.
        event = slice[self.dataset_symbol].data[0]
        # If the estimated PMI is higher than the previous PMI, the manufacturing ETF price is expected to rise.
        # Otherwise, it is expected manufacturing ETF prices will drop.
        direction = InsightDirection.UP if event.previous > event.estimate else InsightDirection.DOWN
        return [Insight.price(self._symbol, timedelta(5), direction)]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            self._symbol = added.symbol
```

Data Point Attributes

The EODHD Economic Events dataset provides `EODHDEconomicEvent` objects, which have the following attributes:

The `EODHDEconomicEvents` objects represent a collection of `EODHDEconomicEvent` object

Country Enumeration

To filter the countries of the economic events, you can make use of the `Country` enumeration. The `Country` enumeration has the following members:

Events Enumeration

To filter the event types of the economic events, you can make use of the `Events` enumeration. The `Events` enumeration has the following members:

EOD Historical Data

Macroeconomics Indicators

Introduction

The Macroeconomic Indicators dataset, provided by EODHD, provides data on 39 major macroeconomic indicators of 249 countries and regions. The data starts in January 1998, and is delivered on indicators update.

For more information about the Macroeconomics Indicators dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EODHD was a France financial data provider founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomy indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly.

Getting Started

The following snippet demonstrates how to request data from the Macroeconomic Indicators dataset:

```
PY
self.add_data(EODHDMacroIndicators, Country.UNITED_STATES)
self.add_data(EODHDMacroIndicators, EODHD.MacroIndicators.UnitedStates.GDP_CURRENT_USD)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Macroeconomic Indicators data to your algorithm, call the `add_data` method.

```
PY
class EODHDMacroIndicatorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(
            EODHDMacroIndicators,
            EODHD.MacroIndicators.UnitedStates.GDP_CURRENT_USD    # Country.UNITED_STATES
        ).symbol
```

Accessing Data

To get the current Macroeconomic Indicators data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{data_point.frequency} {data_point.indicator} of {data_point.country} at {data_point.time} with value
{data_point.value}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for _, data_point in slice.get(EODHDMacroIndicators).items():
        self.log(f"{data_point.frequency} {data_point.indicator} of {data_point.country} at {data_point.time} with value
{data_point.value}")
```

Historical Data

To get historical Macroeconomic Indicators data, call the `history` method with the type `EODHDMacroIndicators` cast and the underlying `Symbol` . If there is no data in the period you request, the history result is empty.

```
history_bars = self.history[EODHDMacroIndicators](self.dataset_symbol, 100)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Macroeconomic Indicators dataset provides data on global macro indicators, allowing traders to react to market sentiment. Examples include the following strategies:

- Long straddle to trade the market shock brought by the macro indicators.
- Hold long or short position of the market representative ETF based on the expected market direction.
- Statistical arbitrage on 2 or more correlated global markets based on market strength indicated by macro indicators.

Classic Algorithm Example

The following example algorithm holds long position of SPY if the US GDP is in positive growth, while holds short position if it is the opposite case.

```
from AlgorithmImports import *


class EODHDMacroIndicatorsAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.equity_symbol = self.add_equity("SPY", Resolution.Daily).symbol

        ticker = EODHD.MacroIndicators.UnitedStates.GDP_GROWTH_ANNUAL
        self.dataset_symbol = self.add_data(EODHDMacroIndicators, ticker).Symbol

    def on_data(self, slice):
        indicators = slice.get(EODHDMacroIndicators).get(self.dataset_symbol)
        if indicators:
            gdp = indicators.data[0].value
            self.SetHoldings(self.equity_symbol, 1 if gdp > 0 else -1)
```

Framework Algorithm Example

The following example algorithm holds the long position of SPY if the US GDP is in positive growth while holding a short position if it is the opposite case. It is implemented in the framework algorithm.

```py
from AlgorithmImports import *


class EODHDMacroIndicatorsAlgorithm(QCAlgorithm):

   def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Use market ETF as a vehicle to trade.
        symbol = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.add_universe_selection(ManualUniverseSelectionModel(symbol))
        # Custom alpha model to emit insights according to macroeconomic indicators.
        self.add_alpha(MacroIndicatorAlphaModel(self))
        # Equal weighting position sizing to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_MONTH))


class MacroIndicatorAlphaModel(AlphaModel):

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Request US GDP growth indicator data to generate trade signal.
        ticker = EODHD.MacroIndicators.UnitedStates.GDP_GROWTH_ANNUAL
        self.dataset_symbol = algorithm.add_data(EODHDMacroIndicators, ticker).symbol
        self._symbol = None

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Trade based on the updated macroeconomic indicator.
        indicators = slice.get(EODHDMacroIndicators).get(self.dataset_symbol)
        if not indicators:
            return []
        gdp = indicators.data[0].value
        # If the GDP gap is positive, the market ETF is expected to rise.
        # Otherwise, it is expected the market ETF will drop.
        direction = InsightDirection.UP if gdp > 0 else InsightDirection.DOWN
        return [Insight.price(self._symbol, timedelta(5), direction)]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            self._symbol = added.symbol
```

Data Point Attributes

The EODHD Macro Indicators dataset provides `EODHDMacroIndicators` objects, which have the following attributes:

Country Enumeration

To filter the country of the macro indicators, you can make use of the `Country` enumeration. The `Country` enumeration has the following members:

Frequency Enumeration

To filter the indicator frequency of the macro indicators, you can make use of the `Frequency` enumeration. The `Frequency` enumeration has the following members:

EOD Historical Data

Upcoming Dividends

Introduction

The Upcoming Dividends dataset, provided by EODHD, offers daily alerts for US Equities that will have a dividend event within the upcoming 7 days. The data starts in January 2015 and is delivered on a daily frequency.

Compared to US Equity Security Master as a benchmark, the Upcoming Dividends dataset has a 98.56% coverage of all dividend events, while having a 99.71% precision on the exact dividend dates of the covered ones and a 99.90% precision within +/- 3 days.

For more information about the Upcoming Dividends dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EOD Historical Data (EODHD) is a financial data provider based in France, and founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomic indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly. For more information about EODHD, visit https://eodhd.com/ .

Getting Started

The following snippet demonstrates how to request data from the Upcoming Dividends dataset:

```
self.add_data(EODHDUpcomingDividends, "dividends")
self.add_universe(EODHDUpcomingDividends, self.selection_function)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2015
Data Density	Sparse
Resolution	Daily
Timezone	New York

Universe Selection

To select a dynamic universe of US Equities based on the Upcoming Dividends dataset, call the `add_universe` method with a `EODHDUpcomingDividends` cast.

```
def initialize(self) -> None:
    self._universe = self.add_universe(EODHDUpcomingDividends, self.universe_selection_filter)

def universe_selection_filter(self, dividends: List[EODHDUpcomingDividends]) -> List[Symbol]:
    return [d.symbol for d in dividends if d.dividend_date <= self.time + timedelta(1) and d.dividend > 0.05]
```

For more information about universe settings, see Settings .

Requesting Data

To add Upcoming Dividends data to your algorithm, call the `add_data` method.

```
class UpcomingDividendsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(EODHDUpcomingDividends, "dividends").symbol
```

Accessing Data

To get the current Upcoming Dividends data, call the `get(EODHDUpcomingDividends)` method from the current `Slice` and index the result with the security `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    upcomings_dividends = slice.get(EODHDUpcomingDividends)
    if upcomings_dividends and self._symbol in upcomings_dividends:
        upcomings_dividends_data_point = upcomings_dividends[self._symbol]
        self.log(f"{self._symbol} will pay dividend at {upcomings_dividends_data_point.dividend_date} with dividend per share of
${upcomings_dividends_data_point.dividend}")
```

You can also iterate through all of the dataset objects in the current `Slice`

```
def on_data(self, slice: Slice) -> None:
    for equity_symbol, upcomings_dividends_data_point in slice.get(EODHDUpcomingDividends).items():
        self.log(f"{equity_symbol} will pay dividend at {upcomings_dividends_data_point.dividend_date} with dividend per share of
${upcomings_dividends_data_point.dividend}")
```

Historical Data

To get historical Upcoming Dividends data, call the `history` method with the type `EODHDUpcomingDividends` cast and the period of request. If there is no data in the period you request, the history result is empty.

```
history = self.history[EODHDUpcomingDividends](timedelta(100), Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Upcoming Dividends dataset allows traders to trade the price change due to dividends. Examples include the following strategies:

- Short the stocks on dividend report day to earn the dividend discount on pricing.
- Avoid volatility on securities with upcoming dividends

Classic Algorithm Example

The following example algorithm shorts each equity in equal size with an upcoming dividend by the next day. It selects stocks with dividend recording over $0.5 per share to capitalize on the price shock momentum due to dividend payment.

```py
from AlgorithmImports import *


class UpcomingDividendsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Universe consists of equities with upcoming dividend events.
        self._universe = self.add_universe(EODHDUpcomingDividends, self.selection)
        # Add a Scheduled Event to rebalance the portfolio every morning
        # based on upcoming dividend signals.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def selection(self, dividends: List[EODHDUpcomingDividends]) -> List[Symbol]:
        # Select the stocks with upcoming dividend record date, with a sufficient dividend size.
        return [x.symbol for x in dividends if x.dividend_date < self.time + timedelta(1) and x.dividend > 0.5]

    def _rebalance(self) -> None:
        # Equally invest in each member of the universe to evenly dissipate the capital risk.
        symbols = [s for s in self._universe.selected if self.securities[s].price]
        total_count = len(symbols)
        targets = [PortfolioTarget(symbol, -1. / total_count) for symbol in symbols]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Framework Algorithm Example

The following example implements a strategy of shorting each equity in equal size with an upcoming dividend by the next day using the framework. It selects stocks with dividend recording over $0.5 per share to capitalize on the price shock momentum due to dividend payment.

```py
from AlgorithmImports import *


class UpcomingDividendsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Trade on a daily basis based on upcoming dividend signals daily.
        self.universe_settings.resolution = Resolution.DAILY
        # Universe consists of equities with upcoming dividend events.
        self._universe = self.add_universe(EODHDUpcomingDividends, self.selection)

        # A constant alpha model will emit insights for the stocks with dividend events in the upcoming day.
        # It is expecting a price shock due to pricing after the dividend, which might affect some momentum traders.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.DOWN, timedelta(1)))

        # Equal weighting for each signal to dissipate the capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_DAY))

    def selection(self, dividends: List[EODHDUpcomingDividends]) -> List[Symbol]:
        # Select the stocks with upcoming dividend record dates with a sufficient dividend size.
        return [x.symbol for x in dividends if x.dividend_date < self.time + timedelta(1) and x.dividend > 0.5]
```

Data Point Attributes

The EODHD Upcoming Dividends dataset provides `EODHDUpcomingDividends` objects, which have the following attributes:

EOD Historical Data

Upcoming Earnings

Introduction

The Upcoming Earnings dataset, provided by EOD Historical Data (EODHD), is a daily universe of US Equities with an earnings report publication in the upcoming 7 days. The data starts in January 1998 and is delivered on a daily frequency.

Compared to Nasdaq's Earning Reports as a benchmark, the Upcoming Earnings dataset has captured 96.79% of the scheduled earnings report, with a 97.25% exact-date precision of the captured events, while having a 99.28% precision within +/- 3 days. Note that the Upcoming Earnings dataset also contains unscheduled special earning reports.

For more information about the Upcoming Earnings dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EOD Historical Data (EODHD) is a financial data provider based in France, and founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomic indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly. For more information about EODHD, visit https://eodhd.com/ .

Getting Started

The following snippet demonstrates how to request data from the Upcoming Earnings dataset:

```
self.add_universe(EODHDUpcomingEarnings, self.selection_function)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Data Density	Sparse
Resolution	Daily
Timezone	New York

Universe Selection

To select a dynamic universe of US Equities based on the Upcoming Earnings dataset, call the `add_universe` method with a `EODHDUpcomingEarnings` cast.

```
def initialize(self) -> None:
    self._universe = self.add_universe(EODHDUpcomingEarnings, self.universe_selection_filter)

def universe_selection_filter(self, earnings: List[EODHDUpcomingEarnings]) -> List[Symbol]:
    return [d.symbol for d in earnings if d.report_date <= self.time + timedelta(3) and d.estimate > 0]
```

For more information about universe settings, see Settings .

Requesting Data

This dataset is designed for universe selection. However, you can add Upcoming Earnings data to your algorithm using the `add_data` method.

```
class UpcomingEarningsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self._dataset_symbol = self.add_data(EODHDUpcomingEarnings, "earnings").symbol
```

Accessing Data

To get the current Upcoming Earnings data, call the `get(EODHDUpcomingEarnings)` method from the current `Slice` and index the result with the security `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    upcomings_earnings_for_symbol = slice.get(EODHDUpcomingEarnings).get(self._symbol)
        if upcomings_earnings_for_symbol:
            self.log(f"{self._symbol} will report earnings at {upcomings_earnings_for_symbol.report_date}
{upcomings_earnings_for_symbol.report_time} with estimated EPS {upcomings_earnings_for_symbol.estimate}")
```

You can also iterate through all of the dataset objects in the current `Slice`

```
def on_data(self, slice: Slice) -> None:
    for equity_symbol, upcomings_earnings_data_point in slice.get(EODHDUpcomingEarnings).items():
        self.log(f"{equity_symbol} will report earnings at {upcomings_earnings_data_point.report_date}
{upcomings_earnings_data_point.report_time} with estimated EPS {upcomings_earnings_data_point.estimate}")
```

Historical Data

To get historical Upcoming Earnings for the universe, call the `history` method with the type `EODHDUpcomingEarning` .

```
history = self.history(EODHDUpcomingEarnings, timedelta(100), Resolution.DAILY)
```

To get historical Upcoming Earnings data for a known security, call the `history` method with the type `EODHDUpcomingEarning` cast and the security `Symbol` .

```
history = self.history[EODHDUpcomingEarnings](timedelta(100), Resolution.DAILY).loc[self._symbol]
```

If there is no data in the period you request, the history result is empty. For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Upcoming Earnings dataset provides timely notifications about earnings announcements, allowing traders to make capitalize on potential price movements and manage risks effectively. Examples include the following strategies:

- Short Straddle to trade on heightened volatility during earnings report.
- Filter universe for the stocks with or without upcoming earnings report to trade or avoid volatility.
- Hold the stocks with upcoming earnings estimated to be positive.

Classic Algorithm Example

The following example algorithm long a straddle on the stocks with upcoming earnings report within 5 days to trade the volatility. It allows 5 days to

build up the volatility and capitalize the positions 1 day after the earnings announcement.

```
class UpcomingEarningsExampleAlgorithm(QCAlgorithm):
    options_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Seed the last price as price since we need to use the underlying price for option contract filtering when it join the universe.
        self.settings.seed_initial_prices = True

        # Trade on daily basis based on daily upcoming earnings signals.
        self.universe_settings.resolution = Resolution.DAILY
        # Option trading requires raw price for strike price comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        # Universe consists of equities with upcoming earnings events.
        self.add_universe(EODHDUpcomingEarnings, self.selection)

    def selection(self, earnings: List[EODHDUpcomingEarnings]) -> List[Symbol]:
        # We do not want to lock our fund too early, so filter for stocks is in lower volatility but will go up.
        # Assuming 5 days before the earnings report publish is less volatile.
        # We do not want depository due to their low liquidity.
        for i in range(5, 0, -1):
            target_date = self.time + timedelta(i)
            symbols = set(d.symbol for d in earnings if d.report_date == target_date)
            if symbols:
                self.plot('Universe', 'Count', len(symbols))
                return list(symbols)
        return []

    def on_securities_changed(self, changes: SecurityChanges) -> None:

        # Actions only based on the equity universe changes.
        for added in [security for security in changes.added_securities if security.type == SecurityType.EQUITY]:
            # Select the option contracts to construct a straddle to trade the volatility.
            call, put = self.select_option_contracts(added.symbol)
            if not call or not put:
                continue
            self.options_by_symbol[added.symbol] = (call, put)
            # Request the option contract data for trading.
            call = self.add_option_contract(call).symbol
            put = self.add_option_contract(put).symbol
            # Long a straddle by shorting the selected ATM call and put.
            self.combo_market_order([
                    Leg.create(call, 1),
                    Leg.create(put, 1)
                ],
                1
            )

        # Actions only based on the equity universe changes.
        for removed in [security for security in changes.removed_securities if security.type == SecurityType.EQUITY]:
            # Liquidate any assigned position.
            self.liquidate(removed.symbol)
            # Liquidate the option positions and capitalize the volatility 1-day after the earnings announcement.
            contracts = self.options_by_symbol.pop(removed.symbol, None)
            if contracts:
                for contract in contracts:
                    self.remove_option_contract(contract)

    def select_option_contracts(self, underlying: Symbol) -> Tuple[Symbol, Symbol]:
        # Get all tradable option contracts for filtering.
        option_contract_list = self.option_chain(underlying)

        # Expiry at least 30 days later to have a smaller theta to reduce time decay loss.
        # Yet also be ensure liquidity over the volatility fluctuation hence we take the closet expiry after that.
        long_expiries = [x for x in option_contract_list if x.id.date >= self.time + timedelta(30)]
        if len(long_expiries) < 2:
            return None, None
        expiry = min(x.id.date for x in long_expiries)
        filtered_contracts = [x for x in option_contract_list if x.id.date == expiry]

        # Select ATM call and put to form a straddle for trading the volatility.
        strike = sorted(filtered_contracts,
            key=lambda x: abs(x.id.strike_price - self.securities[underlying].price))[0].id.strike_price
        atm_contracts = [x for x in filtered_contracts if x.id.strike_price == strike]
        if len(atm_contracts) < 2:
            return None, None

        atm_call = next(filter(lambda x: x.id.option_right == OptionRight.CALL, atm_contracts))
        atm_put = next(filter(lambda x: x.id.option_right == OptionRight.PUT, atm_contracts))
        return atm_call, atm_put
```

Framework Algorithm Example

The following example algorithm long a straddle on the stocks with upcoming earnings report within 5 days to trade the volatility implemented in

framework algorithm. It allows 5 days to build up the volatility and capitalize the positions 1 day after the earnings announcement.

PY

```
class UpcomingEarningsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Seed the last price as price since we need to use the underlying price for option contract filtering when it join the universe.
        self.settings.seed_initial_prices = True

        # Trade on daily basis based on daily upcoming earnings signals.
        self.universe_settings.resolution = Resolution.DAILY
        # Option trading requires raw price for strike price comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        # Universe consists of equities with upcoming earnings events.
        self.add_universe(EODHDUpcomingEarnings, self.selection)

        # A custom alpha model that emit signal based on upcoming earnings events.
        self.add_alpha(UpcomingEarningsAlphaModel())

        # To trade 1 contract per signal to ensure equal size per leg of the short straddle.
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

    def selection(self, earnings: List[EODHDUpcomingEarnings]) -> List[Symbol]:
        # We do not want to lock our fund too early, so filter for stocks is in lower volatility but will go up.
        # Assuming 5 days before the earnings report publish is less volatile.
        # We do not want depository due to their low liquidity.
        for i in range(5, 0, -1):
            target_date = self.time + timedelta(i)
            symbols = set(d.symbol for d in earnings if d.report_date == target_date)
            if symbols:
                self.plot('Universe', 'Count', len(symbols))
                return list(symbols)
        return []

class UpcomingEarningsAlphaModel(AlphaModel):
    options_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Iterate the newly added equities to trade their volatility.
        while len(self.options_by_symbol) > 0:
            key = list(self.options_by_symbol.keys())[0]
            # Obtain the selected option contracts to trade and remove the key-value pair to avoid repeat ordering.
            contracts = self.options_by_symbol.pop(key)
            # Long straddle (long the ATM call and put contracts) for 7 days to trade the volatility.
            # Assuming the volatility will go up due to earning announcement and reach highest volatility at 2-3 days after the report.
            insights.extend([
                Insight.price(contracts[0], timedelta(7), InsightDirection.UP),
                Insight.price(contracts[1], timedelta(7), InsightDirection.UP)
            ])

        # Remove any assigned positions of the underlying.
        for symbol, holding in algorithm.portfolio.items():
            if holding.type == SecurityType.EQUITY and holding.invested:
                insights.append(Insight.price(symbol, timedelta(7), InsightDirection.FLAT))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # Actions only based on the equity universe changes.
        for added in [x for x in changes.added_securities if x.type == SecurityType.EQUITY]:
            # Select the option contracts to construct a straddle to trade the volatility.
            call, put = self.select_option_contracts(algorithm, added.symbol, algorithm.time)
            if not call or not put:
                continue
            # Request the option contract data for trading.
            call = algorithm.add_option_contract(call).symbol
            put = algorithm.add_option_contract(put).symbol
            self.options_by_symbol[added.symbol] = (call, put)

    def select_option_contracts(self, algorithm: QCAlgorithm, underlying: Symbol, time: datetime) -> Tuple[Symbol, Symbol]:
        # Get all tradable option contracts for filtering.
        option_contract_list = algorithm.option_chain(underlying)

        # Expiry at least 30 days later to have a smaller theta to reduce time decay loss.
        # Yet also be ensure liquidity over the volatility fluctuation hence we take the closet expiry after that.
        long_expiries = [x for x in option_contract_list if x.id.date >= time + timedelta(30)]
        if len(long_expiries) < 2:
            return None, None
        expiry = min(x.id.date for x in long_expiries)
        filtered_contracts = [x for x in option_contract_list if x.id.date == expiry]

        # Select ATM call and put to form a straddle for trading the volatility.
        strike = sorted(filtered_contracts,
            key=lambda x: abs(x.id.strike_price - algorithm.securities[underlying].price))[0].id.strike_price
        atm_contracts = [x for x in filtered_contracts if x.id.strike_price == strike]
        if len(atm_contracts) < 2:
            return None, None
```

```
            return None, None

        atm_call = next(filter(lambda x: x.id.option_right == OptionRight.CALL, atm_contracts))
        atm_put = next(filter(lambda x: x.id.option_right == OptionRight.PUT, atm_contracts))
        return atm_call, atm_put

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        if not insights:
            return []
        # Only a single contract to ensure both legs size is equal
        quantity = min([abs(algorithm.calculate_order_quantity(i.symbol, int(i.direction)))
            for i in insights]) / 100
        return [PortfolioTarget(i.symbol, int(i.direction)) for i in insights]
```

Data Point Attributes

The EODHD Upcoming Earnings dataset provides `EODHDUpcomingEarnings` objects, which have the following attributes:

To filter the EODHD Upcoming Earnings data with their reporting timing, use the `EODHD.ReportTime` enum, which have the following attributes:

EOD Historical Data

Upcoming IPOs

Introduction

The Upcoming IPOs dataset, provided by EODHD, offers daily alerts for US Equities that will start their IPOs or have any updates on their IPO registrations within the upcoming 7 days. The data started in February 2013 and is delivered on a daily basis.

`Notice that this dataset might have a +/-2 days accuracy due to the data provider.`

For more information about the Upcoming IPOs dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EODHD was a France financial data provider founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomy indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly.

Getting Started

The following snippet demonstrates how to request data from the Upcoming IPOs dataset:

```
self.add_data(EODHDUpcomingIPOs, "ipos")
self.add_universe(EODHDUpcomingIPOs, self.selection_function)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	February 2013
Data Density	Sparse
Resolution	Daily
Timezone	New York

Universe Selection

To select a dynamic universe of US Equities based on the Upcoming IPOs dataset, call the `add_universe` method with a `EODHDUpcomingIPOs` cast.

```
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self._universe = self.add_universe(EODHDUpcomingIPOs, self.universe_selection_filter)

def universe_selection_filter(self, ipos: List[EODHDUpcomingIPOs]) -> List[Symbol]:
    # confirmed non-penny stock IPO that launches within 7 days.
    return [d.symbol for d in ipos if d.deal_type in [DealType.EXPECTED, DealType.PRICED] and d.ipo_date and min(x for x in
[d.lowest_price, d.highest_price, d.offer_price] if x) > 1]
```

For more information about universe settings, see Settings .

Requesting Data

To add Upcoming IPOs data to your algorithm, call the `add_data` method.

```
class UpcomingIPOsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(EODHDUpcomingIPOs, "ipos").symbol
```

Accessing Data

To get the current Upcoming IPOs data, call the `get(EODHDUpcomingIPOs)` method from the current `Slice` . Then, iterate through all of the dataset objects in the current `Slice`

```
def on_data(self, slice: Slice) -> None:
    for equity_symbol, upcomings_ipos_data_point in slice.get(EODHDUpcomingIPOs).items():
        self.log(f"{equity_symbol} will start IPO at {upcomings_ipos_data_point.ipo_date} with price
{upcomings_ipos_data_point.offer_price} and {upcomings_ipos_data_point.shares} shares")
```

Historical Data

To get historical Upcoming IPOs data, call the `history` method with the type `EODHDUpcomingIPOs` cast and the period of request. If there is no data in the period you request, the history result is empty.

```
history_bars = self.history[EODHDUpcomingIPOs](timedelta(100), Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Upcoming IPOs dataset provides timely notifications about upcoming IPOs start, allowing traders to capitalize on the high volatility of new stocks. Examples include the following strategies:

- Long straddle to trade the volatility of the new IPO stock.
- Arbitration on fair price versus IPO price.
- Use SMA of IPO number to estimate the IPO trend and market popularity.

Classic Algorithm Example

Example 1: New Stock

The following example algorithm holds new stocks for its IPO day to capitalize its hype.

```py
from AlgorithmImports import *


class UpcomingIPOsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Filter for new stocks to trade their hype using EODHDUpcomingIPOs.
        self._universe = self.add_universe(EODHDUpcomingIPOs, self.selection)

    def selection(self, ipos: List[EODHDUpcomingIPOs]) -> List[Symbol]:
        # Select the stocks that IPO starts today and traded in Nasdaq
        return [
            x.symbol for x in ipos
            if (x.ipo_date and
                x.ipo_date <= self.time + timedelta(1) and
                x.exchange == Exchange.NASDAQ)
        ]

    def on_data(self, slice: Slice) -> None:
        # Invest in new stocks and trade on their first day for their hype.
        # Equally invest in each new stocks to evenly dissipate the capital risk.
        for symbol in self._universe.selected:
            security = self.securities[symbol]
            if not security.holdings.invested and security.price:
                self.set_holdings(symbol, 0.1/len(self._universe.selected))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate the new stocks traded today/yesterday to capitalize the first-day hype.
        for removed in changes.removed_securities:
            self.liquidate(removed.symbol)
```

Example 2: Market Popularity

The following example algorithm buy QQQ if the number of IPO in Nasdaq is in increasing trend, sell otherwise.

```py
from AlgorithmImports import *


class UpcomingIPOsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Create a EMA indicator to estimate the trend of IPO number to reflect the market popularity.
        self._ema = ExponentialMovingAverage(50)

        self.universe_settings.resolution = Resolution.DAILY
        # Filter for new stocks to trade their hype using EODHDUpcomingIPOs.
        self.add_universe(EODHDUpcomingIPOs, self.selection)
        # Request QQQ data to trade.
        self.qqq = self.add_equity("QQQ", Resolution.DAILY).symbol

        self.set_warm_up(50, Resolution.DAILY)

    def selection(self, ipos: List[EODHDUpcomingIPOs]) -> List[Symbol]:
        # Filter for the stocks that IPO starts today and traded in Nasdaq.
        universe = [x.symbol for x in ipos if x.ipo_date and x.ipo_date <= self.time + timedelta(1) and x.exchange == Exchange.NASDAQ]
        # Feed to the EMA indicator.
        self._ema.update(self.time, len(universe))
        return Universe.UNCHANGED

    def on_data(self, slice: Slice) -> None:
        if not self.is_warming_up and self._ema.is_ready:
            # If the EMA is decreasing, we estimate the market popularity is decreasing, so we sell QQQ.
            if self._ema.previous.value > self._ema.current.value:
                self.set_holdings(self.qqq, -1)
            # Otherwise, we estimate the market popularity is increasing, so we buy QQQ.
            else:
                self.set_holdings(self.qqq, 1)
```

Framework Algorithm Example

The following example algorithm holds new stocks for its IPO day to capitalize its hype using framework algorithm.

```py
from AlgorithmImports import *


class UpcomingIPOsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Filter for new stocks to trade their hype using EODHDUpcomingIPOs.
        self.add_universe(EODHDUpcomingIPOs, self.selection)

        # Invest in new stocks and trade on their first day for their hype.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1), None, None, 0.01))

        # Equally invest in each new stocks to evenly dissipate the capital risk.
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel(Expiry.END_OF_DAY))

    def selection(self, ipos: List[EODHDUpcomingIPOs]) -> List[Symbol]:
        # Select the stocks that IPO starts today and traded in Nasdaq
        return [
            x.symbol for x in ipos
            if (x.ipo_date and
                x.ipo_date <= self.time + timedelta(1) and
                x.exchange == Exchange.NASDAQ)
        ]
```

Data Point Attributes

The EODHD Upcoming IPOs dataset provides `EODHDUpcomingIPOs` objects, which have the following attributes:

EOD Historical Data

Upcoming Splits

Introduction

The Upcoming Splits dataset, provided by EODHD, offers daily alerts for US Equities that will have a split event within the upcoming 7 days. The data starts in January 2010, and is delivered on a daily frequency.

Compared to US Equity Security Master as a benchmark, the Upcoming Splits dataset has an 85.18% coverage of all splits since 2015, while having a 92.74% precision on the exact split dates of the covered ones and a 97.18% precision within +/- 3 days.

For more information about the Upcoming Splits dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

EOD Historical Data (EODHD) is a financial data provider based in France, and founded in April 2015. They focus on providing clean financial data, including stock prices, splits, dividends, fundamentals, macroeconomic indicators, technical indicators, and alternative data sources, through 24/7 API seamlessly. For more information about EODHD, visit https://eodhd.com/ .

Getting Started

The following snippet demonstrates how to request data from the Upcoming Splits dataset:

```
self.add_universe(EODHDUpcomingSplits, self.selection_function)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2010
Data Density	Sparse
Resolution	Daily
Timezone	New York

Universe Selection

To select a dynamic universe of US Equities based on the Upcoming Splits dataset, call the `add_universe` method with a `EODHDUpcomingSplits` cast.

```
def initialize(self) -> None:
    self._universe = self.add_universe(EODHDUpcomingSplits, self.universe_selection_filter)

def universe_selection_filter(self, splits: List[EODHDUpcomingSplits]) -> List[Symbol]:
    return [d.symbol for d in splits if d.split_date <= self.time + timedelta(3) and d.split_factor > 1]
```

For more information about universe settings, see Settings .

Requesting Data

To add Upcoming Splits data to your algorithm, call the `add_data` method.

```
class UpcomingSplitsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(EODHDUpcomingSplits, "splits").symbol
```

Accessing Data

To get the current Upcoming Splits data, call the `get(EODHDUpcomingSplits)` method from the current `Slice` and index the result with the security `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your security at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    upcomings_splits = slice.get(EODHDUpcomingSplits)
    if upcomings_splits and self._symbol in upcomings_splits:
        upcomings_splits_data_point = upcomings_splits[self._symbol]
        self.log(f"{self._symbol} will split at {upcomings_splits_data_point.split_date} with split factor
{upcomings_splits_data_point.split_factor}")
```

You can also iterate through all of the dataset objects in the current `Slice`

```
def on_data(self, slice: Slice) -> None:
    for equity_symbol, upcomings_splits_data_point in slice.get(EODHDUpcomingSplits).items():
        self.log(f"{equity_symbol} will split at {upcomings_splits_data_point.split_date} with split factor
{upcomings_splits_data_point.split_factor}")
```

Historical Data

To get historical Upcoming Splits data, call the `history` method with the type `EODHDUpcomingSplits` cast and the period of request. If there is no data in the period you request, the history result is empty.

```
history = self.history[EODHDUpcomingSplits](timedelta(100), Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The Upcoming Splits dataset provides timely notifications about upcoming share split or reverse split events, allowing traders to capitalize on potential price movements and manage risks effectively. Examples include the following strategies:

- Splits into shares with lower price will provide higher liquidity for market making algorithms.
- Select a universe of stocks with upcoming splits event and trade their volatility using options.
- Buy stocks with split factor higher than 1, sell vice versa for self-defeating ones.

Classic Algorithm Example

The following example algorithm holds each equity in equal size with upcoming splits within 7 days. It selects stocks that split up to more shares with lower price and tries to capitalize the higher demand of the more afforable stocks.

```
from AlgorithmImports import *


class UpcomingSplitsExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Universe consists of equities with upcoming splits events.
        self._universe = self.add_universe(EODHDUpcomingSplits, self.selection)
        # Add a Scheduled Event to rebalance the portfolio every morning
        # based on upcoming splits signals.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def selection(self, splits: List[EODHDUpcomingSplits]) -> List[Symbol]:
        # Split (more shares with lower price) will make the stock more affordable and drive up the demand.
        # Hence, include all stocks that will have a split within the next 7 days.
        # Note that spliting up the stock means the split factor > 1.
        return [x.symbol for x in splits if x.split_factor > 1]

    def _rebalance(self) -> None:
        # Equally invest in each member of the universe to evenly dissipate the capital risk.
        symbols = [s for s in self._universe.selected if self.securities[s].price]
        total_count = len(symbols)
        targets = [PortfolioTarget(symbol, 1. / total_count) for symbol in symbols]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Framework Algorithm Example

The following example algorithm holds each equity in equal size with upcoming splits within 7 days using algorithm framework. It selects stocks that split up to more shares with lower price and tries to capitalize the higher demand of the more afforable stocks.

```
class UpcomingSplitsExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.free_portfolio_value_percentage = 0.1

        # Trade on daily basis based on daily upcoming splits signals.
        self.universe_settings.resolution = Resolution.DAILY
        # Universe consists of equities with upcoming splits events.
        self.add_universe(EODHDUpcomingSplits, self.selection)

        # Constant alpha model to emit insights for the stocks with split up event within a week, estimating their demand and price will
go up.
        # Signal stays for 2 days to fully digest the driven up demands.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(2)))

        # Equal weighting for each signal to evenly dissipate the capital risk.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Expiry.END_OF_DAY))

    def selection(self, splits: List[EODHDUpcomingSplits]) -> List[Symbol]:
        # Split (more shares with lower price) will make the stock more affordable and drive up the demand.
        # Hence, include the stocks with the lowest split factor that will have a split within the next 7 days.
        # Note that spliting up the stock means the split factor > 1.
        count = 5
        splits = [x for x in splits if x.split_factor > 1]
        if len(splits) > count:
            splits = sorted([x for x in splits if x.split_factor > 1], key=lambda x: x.split_factor)[:count]
        return [x.symbol for x in splits]
```

Data Point Attributes

The EODHD Upcoming Splits dataset provides `EODHDUpcomingSplits` objects, which have the following attributes:

Datasets

Energy Information Administration

The Treasury Department is the executive agency responsible for promoting economic prosperity and ensuring the financial security of the United States. The Department is responsible for a wide range of activities such as advising the President on economic and financial issues, encouraging sustainable economic growth, and fostering improved governance in financial institutions. The Department of the Treasury operates and maintains systems that are critical to the nation's financial infrastructure, such as the production of coin and currency, the disbursement of payments to the American public, revenue collection, and the borrowing of funds necessary to run the federal government.

US Energy Information Administration (EIA)

Energy Information Administration

US Energy Information Administration (EIA)

Introduction

The US Energy Information Administration (EIA) datasets by the Department of the Treasury tracks national and international oil production and consumption. The data covers 190 datasets, starts in January 1991, and is delivered on a daily frequency. This dataset is created by QuantConnect processing and caching the EIA archives.

For more information about the US Energy Information Administration (EIA) dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The Treasury Department is the executive agency responsible for promoting economic prosperity and ensuring the financial security of the United States. The Department is responsible for a wide range of activities such as advising the President on economic and financial issues, encouraging sustainable economic growth, and fostering improved governance in financial institutions. The Department of the Treasury operates and maintains systems that are critical to the nation's financial infrastructure, such as the production of coin and currency, the disbursement of payments to the American public, revenue collection, and the borrowing of funds necessary to run the federal government.

Getting Started

The following snippet demonstrates how to request data from the EIA dataset:

```
self.dataset_symbol = self.add_data(USEnergy, USEnergy.Petroleum.UnitedStates.WEEKLY_NET_IMPORTS_OF_TOTAL_PETROLEUM_PRODUCTS).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1991
Asset Coverage	190 Datasets
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

To add EIA data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class USEnergyDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(USEnergy,
USEnergy.Petroleum.UnitedStates.WEEKLY_NET_IMPORTS_OF_TOTAL_PETROLEUM_PRODUCTS).symbol
```

Accessing Data

To get the current EIA data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you

index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} value at {slice.time}: {data_point.value}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(USEnergy).items():
        self.log(f"{dataset_symbol} value at {slice.time}: {data_point.value}")
```

Historical Data

To get historical EIA data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[USEnergy](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove your subscription to EIA data, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Supported Datasets

The following table shows the accessor code you need to add each EIA dataset to your algorithm:

Symbol	Accessor Code	Description
`UnitedStates`		
PET.WGFRPUS2.W	`USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderAdjustedNetProductionOfFinishedMotorGasoline`	U.S. Refiner and Blender Adjusted Net Production of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WGFSTUS1.W	`USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfFinishedMotorGasoline`	U.S. Ending Stocks of Finished Motor Gasoline in Thousand Barrels (Mbbl)
PET.WGFUPUS2.W	`USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfFinishedMotorGasoline`	U.S. Product Supplied of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WCSSTUS1.W	`USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfCrudeOilInSpr`	U.S. Ending Stocks of Crude Oil in SPR in Thousand Barrels (Mbbl)
PET.WDGRPUS2.W	`USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfDistillateFuelOilGreaterThan500PpmSulfur`	U.S. Refiner and Blender Net Production of Distillate Fuel Oil Greater than 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WDGSTUS1.W	`USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfDistillateFuelOilGreaterThan500PpmSulfur`	U.S. Ending Stocks of Distillate Fuel Oil, Greater Than 500 ppm Sulfur in Thousand Barrels (Mbbl)

PET.WDIEXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfTotalDistillate	U.S. Exports of Total Distillate in Thousand Barrels per Day (Mbbl/d)
PET.WDIIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfDistillateFuelOil	U.S. Imports of Distillate Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WDIRPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfDistillateFuelOil	U.S. Refiner and Blender Net Production of Distillate Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WKJSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfKeroseneTypeJetFuel	U.S. Ending Stocks of Kerosene-Type Jet Fuel in Thousand Barrels (Mbbl)
PET.WKJUPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfKeroseneTypeJetFuel	U.S. Product Supplied of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WGTIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfTotalGasoline	U.S. Imports of Total Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WGTSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfTotalGasoline	U.S. Ending Stocks of Total Gasoline in Thousand Barrels (Mbbl)
PET.WGIRIUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyGrossInputsIntoRefineries	U.S. Gross Inputs into Refineries in Thousand Barrels per Day (Mbbl/d)
PET.WGRIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfReformulatedMotorGasoline	U.S. Imports of Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WGRRPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfReformulatedMotorGasoline	U.S. Refiner and Blender Net Production of Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WGRSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedMotorGasoline	U.S. Ending Stocks of Reformulated Motor Gasoline in Thousand Barrels (Mbbl)
PET.WDISTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfDistillateFuelOil	U.S. Ending Stocks of Distillate Fuel Oil in Thousand Barrels (Mbbl)
PET.WDIUPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfDistillateFuelOil	U.S. Product Supplied of Distillate Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WKMRPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfMilitaryKeroseneTypeJetFuel	U.S. Refiner and Blender Net Production of Military Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WOCLEUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyOperableCrudeOilDistillationCapacity	U. S. Operable Crude Oil Distillation Capacity in Thousand Barrels per Calendar Day (Mbbl/d)
PET.WPLSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyPropyleneNonfuelUseStocksAtBulkTerminals	U.S. Propylene Nonfuel Use Stocks at Bulk Terminals in Thousand Barrels (Mbbl)
PET.WPRSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfPropaneAndPropylene	U.S. Ending Stocks of Propane and Propylene in Thousand Barrels (Mbbl)
PET.WPULEUS3.W	USEnergy.Petroleum.UnitedStates.WeeklyPercentUtilizationOfRefineryOperableCapacity	U.S. Percent Utilization of Refinery Operable Capacity in Percent (%)
PET.WREEXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfResidualFuelOil	U.S. Exports of Residual Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WREIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfResidualFuelOil	U.S. Imports of Residual Fuel Oil in Thousand Barrels per Day (Mbbl/d)

PET.WKCRPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfCommercialKeroseneTypeJetFuel	U.S. Refiner and Blender Net Production of Commercial Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WKJEXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfKeroseneTypeJetFuel	U.S. Exports of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WKJIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfKeroseneTypeJetFuel	U.S. Imports of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WKJRPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfKeroseneTypeJetFuel	U.S. Refiner and Blender Net Production of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.WCESTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksExcludingSprOfCrudeOil	U.S. Ending Stocks excluding SPR of Crude Oil in Thousand Barrels (Mbbl)
PET.WCREXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfCrudeOil	U.S. Exports of Crude Oil in Thousand Barrels per Day (Mbbl/d)
PET.WCRFPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyFieldProductionOfCrudeOil	U.S. Field Production of Crude Oil in Thousand Barrels per Day (Mbbl/d)
PET.WCRIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfCrudeOil	U.S. Imports of Crude Oil in Thousand Barrels per Day (Mbbl/d)
PET.WCRNTUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyNetImportsOfCrudeOil	U.S. Net Imports of Crude Oil in Thousand Barrels per Day (Mbbl/d)
PET.WCRRIUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetInputOfCrudeOil	U.S. Refiner Net Input of Crude Oil in Thousand Barrels per Day (Mbbl/d)
PET.WRERPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfResidualFuelOil	U.S. Refiner and Blender Net Production of Residual Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WRESTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfResidualFuelOil	U.S. Ending Stocks of Residual Fuel Oil in Thousand Barrels (Mbbl)
PET.WREUPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfResidualFuelOil	U.S. Product Supplied of Residual Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.WRPEXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfTotalPetroleumProducts	U.S. Exports of Total Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WRPIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfTotalPetroleumProducts	U.S. Imports of Total Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WRPNTUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyNetImportsOfTotalPetroleumProducts	U.S. Net Imports of Total Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WRPUPUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfPetroleumProducts	U.S. Product Supplied of Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WTESTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksExcludingSprOfCrudeOilAndPetroleumProducts	U.S. Ending Stocks excluding SPR of Crude Oil and Petroleum Products in Thousand Barrels (Mbbl)
PET.WTTEXUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfCrudeOilAndPetroleumProducts	U.S. Exports of Crude Oil and Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WTTIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfCrudeOilAndPetroleumProducts	U.S. Imports of Crude Oil and Petroleum Products in Thousand Barrels per Day (Mbbl/d)
PET.WTTNTUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyNetImportsOfCrudeOilAndPetroleumProducts	U.S. Net Imports of Crude Oil and Petroleum Products in Thousand Barrels per Day (Mbbl/d)

PET.WTTSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfCrudeOilAndPetroleumProducts	U.S. Ending Stocks of Crude Oil and Petroleum Products in Thousand Barrels (Mbbl)
PET.WUOSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfUnfinishedOils	U.S. Ending Stocks of Unfinished Oils in Thousand Barrels (Mbbl)
PET.WG6TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfOtherFinishedConventionalMotorGasoline	U.S. Refiner and Blender Net Production of Other Finished Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WD0TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfDistillateFuelOil0To15PpmSulfur	U.S. Refiner and Blender Net Production of Distillate Fuel Oil, 0 to 15 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WD1ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Ending Stocks of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels (Mbbl)
PET.WD1TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductionOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Production of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WG1ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedMotorGasolineWithFuelAlcohol	U.S. Ending Stocks of Reformulated Motor Gasoline with Fuel ALcohol in Thousand Barrels (Mbbl)
PET.WCRSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfCrudeOil	U.S. Ending Stocks of Crude Oil in Thousand Barrels (Mbbl)
PET.WCSIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyCrudeOilImportsBySpr	U.S. Crude Oil Imports by SPR in Thousand Barrels per Day (Mbbl/d)
PET.WBCIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfGasolineBlendingComponents	U.S. Imports of Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WBCSTUS1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfGasolineBlendingComponents	U.S. Ending Stocks of Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.WCEIMUS2.W	USEnergy.Petroleum.UnitedStates.WeeklyCommercialCrudeOilImportsExcludingSpr	U.S. Commercial Crude Oil Imports Excluding SPR in Thousand Barrels per Day (Mbbl/d)
PET.WPRTP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerBlenderAndGasPlantNetProductionOfPropaneAndPropylene	U.S. Refiner, Blender, and Gas Plant Net Production of Propane and Propylene in Thousand Barrels per Day (Mbbl/d)
PET.WG1TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfFinishedReformulatedMotorGasolineWithEthanol	U.S. Refiner and Blender Net Production of Finished Reformulated Motor Gasoline with Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.WG3ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedMotorGasolineNonOxygentated	U.S. Ending Stocks of Reformulated Motor Gasoline, Non-Oxygentated in Thousand Barrels (Mbbl)
PET.WG4ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalMotorGasoline	U.S. Ending Stocks of Conventional Motor Gasoline in Thousand Barrels (Mbbl)
PET.WG4TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfConventionalMotorGasoline	U.S. Refiner and Blender Net Production of Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WG5ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalMotorGasolineWithFuelEthanol	U.S. Ending Stocks of Conventional Motor Gasoline with Fuel Ethanol in Thousand Barrels (Mbbl)

PET.WG5TP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfFinishedConventionalMotorGasolineWithEthanol	U.S. Refiner and Blender Net Production of Finished Conventional Motor Gasoline with Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.WG6ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfOtherConventionalMotorGasoline	U.S. Ending Stocks of Other Conventional Motor Gasoline in Thousand Barrels (Mbbl)
PET.WO6RI_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfConventionalCbobGasolineBlendingComponents	U.S. Refiner and Blender Net Input of Conventional CBOB Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO6ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalCbobGasolineBlendingComponents	U.S. Ending Stocks of Conventional CBOB Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.WO7RI_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfConventionalGtabGasolineBlendingComponents	U.S. Refiner and Blender Net Input of Conventional GTAB Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO7ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalGtabGasolineBlendingComponents	U.S. Ending Stocks of Conventional GTAB Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.WO9RI_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfConventionalOtherGasolineBlendingComponents	U.S. Refiner and Blender Net Input of Conventional Other Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO9ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalOtherGasolineBlendingComponents	U.S. Ending Stocks of Conventional Other Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.W_EPD2F_PWR_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyNo2HeatingOilWholesaleResalePrice	U.S. No. 2 Heating Oil Wholesale/Resale Price in Dollars per Gallon ($/gal)
PET.W_EPC0_SKA_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyCrudeOilStocksInTransitOnShipsFromAlaska	U.S. Crude Oil Stocks in Transit (on Ships) from Alaska in Thousand Barrels (Mbbl)
PET.W_EPC0_VSD_NUS_DAYS.W	USEnergy.Petroleum.UnitedStates.WeeklyDaysOfSupplyOfCrudeOilExcludingSpr	U.S. Days of Supply of Crude Oil excluding SPR in Number of Days (Days)
PET.W_EPD0_VSD_NUS_DAYS.W	USEnergy.Petroleum.UnitedStates.WeeklyDaysOfSupplyOfTotalDistillate	U.S. Days of Supply of Total Distillate in Number of Days (Days)
PET.W_EPD2F_PRS_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyWeeklyNo2HeatingOilResidentialPrice	U.S. Weekly No. 2 Heating Oil Residential Price in Dollars per Gallon ($/gal)
PET.WPRUP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfPropaneAndPropylene	U.S. Product Supplied of Propane and Propylene in Thousand Barrels per Day (Mbbl/d)
PET.WWOUP_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyProductSuppliedOfOtherOils	U.S. Product Supplied of Other Oils in Thousand Barrels per Day (Mbbl/d)
PET.WBCRI_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfGasolineBlendingComponents	U.S. Refiner and Blender Net Input of Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WD0ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfDistillateFuelOil0To15PpmSulfur	U.S. Ending Stocks of Distillate Fuel Oil, 0 to 15 ppm Sulfur in Thousand Barrels (Mbbl)
PET.W_EPJK_VSD_NUS_DAYS.W	USEnergy.Petroleum.UnitedStates.WeeklyDaysOfSupplyOfKeroseneTypeJetFuel	U.S. Days of Supply of Kerosene-Type Jet Fuel in Number of Days (Days)

PET.W_EPM0_VSD_NUS_DAYS.W	USEnergy.Petroleum.UnitedStates.WeeklyDaysOfSupplyOfTotalGasoline	U.S. Days of Supply of Total Gasoline in Number of Days (Days)
PET.W_EPPA_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfAsphaltAndRoadOil	U.S. Ending Stocks of Asphalt and Road Oil in Thousand Barrels (Mbbl)
PET.W_EPPK_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfKerosene	U.S. Ending Stocks of Kerosene in Thousand Barrels (Mbbl)
PET.W_EPDM10_VUA_NUS_2.W	USEnergy.Petroleum.UnitedStates.WeeklySupplyAdjustmentOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Supply Adjustment of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WG5IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfConventionalMotorGasolineWithFuelEthanol	U.S. Imports of Conventional Motor Gasoline with Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.WG6IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfOtherConventionalMotorGasoline	U.S. Imports of Other Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WD0IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfDistillateFuelOil0To15PpmSulfur	U.S. Imports of Distillate Fuel Oil, 0 to 15 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WD1IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Imports of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WD2IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfDistillateFuelOilGreaterThan500To2000PpmSulfur	U.S. Imports of Distillate Fuel Oil, Greater than 500 to 2000 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WPRIM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfPropaneAndPropylene	U.S. Imports of Propane and Propylene in Thousand Barrels per Day (Mbbl/d)
PET.WO7IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfConventionalGtabGasolineBlendingComponents	U.S. Imports of Conventional GTAB Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WD3IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfDistillateFuelOilGreaterThan2000PpmSulfur	U.S. Imports of Distillate Fuel Oil, Greater than 2000 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.WG1IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfReformulatedMotorGasolineWithFuelAlcohol	U.S. Imports of Reformulated Motor Gasoline with Fuel ALcohol in Thousand Barrels per Day (Mbbl/d)
PET.WG4IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfConventionalMotorGasoline	U.S. Imports of Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.WO9IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfConventionalOtherGasolineBlendingComponents	U.S. Imports of Conventional Other Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO6IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfConventionalCbobGasolineBlendingComponents	U.S. Imports of Conventional CBOB Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.W_EPPK_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfKerosene	U.S. Blender Net Production of Kerosene in Thousand Barrels per Day (Mbbl/d)
PET.W_EPPK_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfKerosene	U.S. Refiner Net Production of Kerosene in Thousand Barrels per Day (Mbbl/d)
PET.W_EPPO6_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfOtherOilsExcludingFuelEthanol	U.S. Ending Stocks of Other Oils (Excluding Fuel Ethanol) in Thousand Barrels (Mbbl)

PET.W_EPPR_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfResidualFuelOil	U.S. Refiner Net Production of Residual Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0R_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfReformulatedMotorGasoline	U.S. Blender Net Production of Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0R_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfReformulatedMotorGasoline	U.S. Refiner Net Production of Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPOOXE_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfFuelEthanol	U.S. Ending Stocks of Fuel Ethanol in Thousand Barrels (Mbbl)
PET.W_EPD0_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfDistillateFuelOil	U.S. Blender Net Production of Distillate Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.W_EPD0_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfDistillateFuelOil	U.S. Refiner Net Production of Distillate Fuel Oil in Thousand Barrels per Day (Mbbl/d)
PET.W_EPJK_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfKeroseneTypeJetFuel	U.S. Blender Net Production of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.W_EPJK_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfKeroseneTypeJetFuel	U.S. Refiner Net Production of Kerosene-Type Jet Fuel in Thousand Barrels per Day (Mbbl/d)
PET.W_EPLLPA_PRS_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyPropaneResidentialPrice	U.S. Propane Residential Price in Dollars per Gallon ($/gal)
PET.W_EPLLPA_PWR_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyPropaneWholesaleResalePrice	U.S. Propane Wholesale/Resale Price in Dollars per Gallon ($/gal)
PET.W_EPOBGRR_YIR_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfMotorGasolineBlendingComponentsRbob	U.S. Refiner and Blender Net Input of Motor Gasoline Blending Components, RBOB in Thousand Barrels per Day (Mbbl/d)
PET.W_EPL0XP_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfNgplsLrgsExcludingPropanePropylene	U.S. Ending Stocks of NGPLs/LRGs (Excluding Propane/Propylene) in Thousand Barrels (Mbbl)
PET.W_EPLLPZ_VSD_NUS_DAYS.W	USEnergy.Petroleum.UnitedStates.WeeklyDaysOfSupplyOfPropanePropylene	U.S. Days of Supply of Propane/Propylene in Number of Days (Days)
PET.W_EPM0C_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfConventionalMotorGasoline	U.S. Blender Net Production of Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0C_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfConventionalMotorGasoline	U.S. Refiner Net Production of Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_VUA_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklySupplyAdjustmentOfFinishedMotorGasoline	U.S. Supply Adjustment of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfFinishedMotorGasoline	U.S. Blender Net Production of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_YPR_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfFinishedMotorGasoline	U.S. Refiner and Blender Net Production of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfFinishedMotorGasoline	U.S. Refiner Net Production of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)

PET.W_EPD00H_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfDistillateFuelOilGreaterThan500PpmSulfur	U.S. Blender Net Production of Distillate Fuel Oil, Greater Than 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPD00H_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfDistillateFuelOilGreaterThan500PpmSulfur	U.S. Refiner Net Production of Distillate Fuel Oil, Greater Than 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPDM10_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Blender Net Production of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPDM10_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfDistillateFuelOilGreaterThan15To500PpmSulfur	U.S. Refiner Net Production of Distillate Fuel Oil, Greater than 15 to 500 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPDXL0_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfDistillateFuelOil0To15PpmSulfur	U.S. Blender Net Production of Distillate Fuel Oil, 0 to 15 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPDXL0_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfDistillateFuelOil0To15PpmSulfur	U.S. Refiner Net Production of Distillate Fuel Oil, 0 to 15 ppm Sulfur in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CA_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfConventionalMotorGasolineWithFuelEthanol	U.S. Blender Net Production of Conventional Motor Gasoline with Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CA_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfConventionalMotorGasolineWithFuelEthanol	U.S. Refiner Net Production of Conventional Motor Gasoline with Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CO_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfOtherConventionalMotorGasoline	U.S. Blender Net Production of Other Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CO_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfOtherConventionalMotorGasoline	U.S. Refiner Net Production of Other Conventional Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0RA_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfReformulatedMotorGasolineWithFuelAlcohol	U.S. Blender Net Production of Reformulated Motor Gasoline with Fuel ALcohol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0RA_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfReformulatedMotorGasolineWithFuelAlcohol	U.S. Refiner Net Production of Reformulated Motor Gasoline with Fuel ALcohol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPOOXE_YOP_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyOxygenatePlantProductionOfFuelEthanol	U.S. Oxygenate Plant Production of Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAL55_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfMotorGasolineFinishedConventionalEd55AndLower	U.S. Blender Net Production of Motor Gasoline, Finished, Conventional, Ed55 and Lower in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAL55_YPT_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfFinishedConventionalMotorGasolineEd55AndLower	U.S. Refiner and Blender Net Production of Finished Conventional Motor Gasoline, Ed 55 and Lower in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAL55_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfMotorGasolineFinishedConventionalEd55AndLower	U.S. Refiner Net Production of Motor Gasoline, Finished, Conventional, Ed55 and Lower in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_EEX_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfFinishedMotorGasoline	U.S. Exports of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0F_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfFinishedMotorGasoline	U.S. Imports of Finished Motor Gasoline in Thousand Barrels per Day (Mbbl/d)

PET.W_EPM0RO_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfOtherReformulatedMotorGasoline	U.S. Blender Net Production of Other Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0RO_YPT_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfOtherFinishedReformulatedMotorGasoline	U.S. Refiner and Blender Net Production of Other Finished Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0RO_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfOtherReformulatedMotorGasoline	U.S. Refiner Net Production of Other Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPOBGRR_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfMotorGasolineBlendingComponentsRbob	U.S. Ending Stocks of Motor Gasoline Blending Components, RBOB in Thousand Barrels (Mbbl)
PET.W_EPOOXE_YIR_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetInputOfFuelEthanol	U.S. Refiner and Blender Net Input of Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAG55_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfMotorGasolineFinishedConventionalGreaterThanEd55	U.S. Imports of Motor Gasoline, Finished, Conventional, Greater than Ed55 in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAL55_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfMotorGasolineFinishedConventionalEd55AndLower	U.S. Imports of Motor Gasoline, Finished, Conventional, Ed55 and Lower in Thousand Barrels per Day (Mbbl/d)
PET.W_EPC0_IMU_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyCrudeOilImportsForSprByOthers	U.S. Crude Oil Imports for SPR by Others in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAG55_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalMotorGasolineGreaterThanEd55	U.S. Ending Stocks of Conventional Motor Gasoline, Greater than Ed55 in Thousand Barrels (Mbbl)
PET.W_EPOOXE_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfFuelEthanol	U.S. Imports of Fuel Ethanol in Thousand Barrels per Day (Mbbl/d)
PET.W_EPL0XP_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfLiquefiedPetroleumGassesLessPropanePropylene	U.S. Imports of Liquefied Petroleum Gasses Less Propane/Propylene in Thousand Barrels per Day (Mbbl/d)
PET.W_EPLLPZ_EEX_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfPropaneAndPropylene	U.S. Exports of Propane and Propylene in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0RO_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfOtherReformulatedMotorGasoline	U.S. Imports of Other Reformulated Motor Gasoline in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAG55_YPB_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyBlenderNetProductionOfMotorGasolineFinishedConventionalGreaterThanEd55	U.S. Blender Net Production of Motor Gasoline, Finished, Conventional, Greater Than Ed55 in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAG55_YPT_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerAndBlenderNetProductionOfFinishedConventionalMotorGasolineGreaterThanEd55	U.S. Refiner and Blender Net Production of Finished Conventional Motor Gasoline, Greater than Ed 55 in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAG55_YPY_NUS_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyRefinerNetProductionOfFinishedConventionalMotorGasolineGreaterThanEd55	U.S. Refiner Net Production of Finished Conventional Motor Gasoline, Greater than Ed 55 in Thousand Barrels per Day (Mbbl/d)
PET.W_EPM0CAL55_SAE_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfConventionalMotorGasolineEd55AndLower	U.S. Ending Stocks of Conventional Motor Gasoline, Ed55 and Lower in Thousand Barrels (Mbbl)
PET.W_EPPK_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfKerosene	U.S. Imports of Kerosene in Thousand Barrels per Day (Mbbl/d)

PET.W_EPPO4_EEX_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyExportsOfOtherOils	U.S. Exports of Other Oils in Thousand Barrels per Day (Mbbl/d)
PET.W_EPPO6_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfOtherOilsExcludingFuelEthanol	U.S. Imports of Other Oils (Excluding Fuel Ethanol) in Thousand Barrels per Day (Mbbl/d)
PET.W_EPOBGRR_IM0_NUS-Z00_MBBLD.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsFromAllCountriesOfMotorGasolineBlendingComponentsRbob	U.S. Imports from All Countries of Motor Gasoline Blending Components, RBOB in Thousand Barrels per Day (Mbbl/d)
PET.EMM_EPMR_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyRegularAllFormulationsRetailGasolinePrices	U.S. Regular All Formulations Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMM_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyMidgradeAllFormulationsRetailGasolinePrices	U.S. Midgrade All Formulations Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMP_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyPremiumAllFormulationsRetailGasolinePrices	U.S. Premium All Formulations Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPM0_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyAllGradesAllFormulationsRetailGasolinePrices	U.S. All Grades All Formulations Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPM0R_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyAllGradesReformulatedRetailGasolinePrices	U.S. All Grades Reformulated Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMMR_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyMidgradeReformulatedRetailGasolinePrices	U.S. Midgrade Reformulated Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMPR_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyPremiumReformulatedRetailGasolinePrices	U.S. Premium Reformulated Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMRU_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyRegularConventionalRetailGasolinePrices	U.S. Regular Conventional Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMRR_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyRegularReformulatedRetailGasolinePrices	U.S. Regular Reformulated Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMD_EPD2D_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyNo2DieselRetailPrices	U.S. No 2 Diesel Retail Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMPU_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyPremiumConventionalRetailGasolinePrices	U.S. Premium Conventional Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPMMU_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyMidgradeConventionalRetailGasolinePrices	U.S. Midgrade Conventional Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMM_EPM0U_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyAllGradesConventionalRetailGasolinePrices	U.S. All Grades Conventional Retail Gasoline Prices in Dollars per Gallon ($/gal)
PET.EMD_EPD2DXL0_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyNo2DieselUltraLowSulfur015PpmRetailPrices	U.S. No 2 Diesel Ultra Low Sulfur (0-15 ppm) Retail Prices in Dollars per Gallon ($/gal)

PET.W_EPC0_SAX_NUS_MBBL.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksExcludingSprAndIncludingLeaseStockOfCrudeOil	U.S. Ending Stocks excluding SPR and including Lease Stock of Crude Oil in Thousand Barrels (Mbbl)
PET.EMD_EPD2DM10_PTE_NUS_DPG.W	USEnergy.Petroleum.UnitedStates.WeeklyNo2DieselLowSulfur15500PpmRetailPrices	U.S. No 2 Diesel Low Sulfur (15-500 ppm) Retail Prices in Dollars per Gallon ($/gal)
PET.WO3IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfReformulatedRbobWithAlcoholGasolineBlendingComponents	U.S. Imports of Reformulated RBOB with Alcohol Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO4IM_NUS-Z00_2.W	USEnergy.Petroleum.UnitedStates.WeeklyImportsOfReformulatedRbobWithEtherGasolineBlendingComponents	U.S. Imports of Reformulated RBOB with Ether Gasoline Blending Components in Thousand Barrels per Day (Mbbl/d)
PET.WO2ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedGtabGasolineBlendingComponents	U.S. Ending Stocks of Reformulated GTAB Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.WO3ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedRbobWithAlcoholGasolineBlendingComponents	U.S. Ending Stocks of Reformulated RBOB with Alcohol Gasoline Blending Components in Thousand Barrels (Mbbl)
PET.WO4ST_NUS_1.W	USEnergy.Petroleum.UnitedStates.WeeklyEndingStocksOfReformulatedRbobWithEtherGasolineBlendingComponents	U.S. Ending Stocks of Reformulated RBOB with Ether Gasoline Blending Components in Thousand Barrels (Mbbl)
EquatorialGuinea		
PET.W_EPC0_IM0_NUS-NEK_MBBLD.W	USEnergy.Petroleum.EquatorialGuinea.WeeklyImportsFromEquatorialGuineaOfCrudeOil	U.S. Imports from Equatorial Guinea of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Iraq		
PET.W_EPC0_IM0_NUS-NIZ_MBBLD.W	USEnergy.Petroleum.Iraq.WeeklyImportsFromIraqOfCrudeOil	U.S. Imports from Iraq of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Kuwait		
PET.W_EPC0_IM0_NUS-NKU_MBBLD.W	USEnergy.Petroleum.Kuwait.WeeklyImportsFromKuwaitOfCrudeOil	U.S. Imports from Kuwait of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Mexico		
PET.W_EPC0_IM0_NUS-NMX_MBBLD.W	USEnergy.Petroleum.Mexico.WeeklyImportsFromMexicoOfCrudeOil	U.S. Imports from Mexico of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Nigeria		
PET.W_EPC0_IM0_NUS-NNI_MBBLD.W	USEnergy.Petroleum.Nigeria.WeeklyImportsFromNigeriaOfCrudeOil	U.S. Imports from Nigeria of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Norway		
PET.W_EPC0_IM0_NUS-NNO_MBBLD.W	USEnergy.Petroleum.Norway.WeeklyImportsFromNorwayOfCrudeOil	U.S. Imports from Norway of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Russia		
PET.W_EPC0_IM0_NUS-NRS_MBBLD.W	USEnergy.Petroleum.Russia.WeeklyImportsFromRussiaOfCrudeOil	U.S. Imports from Russia of Crude Oil in Thousand Barrels per Day (Mbbl/d)
SaudiArabia		
PET.W_EPC0_IM0_NUS-NSA_MBBLD.W	USEnergy.Petroleum.SaudiArabia.WeeklyImportsFromSaudiArabiaOfCrudeOil	U.S. Imports from Saudi Arabia of Crude Oil in Thousand Barrels per Day (Mbbl/d)

UnitedKingdom		
PET.W_EPC0_IM0_NUS-NUK_MBBLD.W	USEnergy.Petroleum.UnitedKingdom.WeeklyImportsFromUnitedKingdomOfCrudeOil	U.S. Imports from United Kingdom of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Venezuela		
PET.W_EPC0_IM0_NUS-NVE_MBBLD.W	USEnergy.Petroleum.Venezuela.WeeklyImportsFromVenezuelaOfCrudeOil	U.S. Imports from Venezuela of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Algeria		
PET.W_EPC0_IM0_NUS-NAG_MBBLD.W	USEnergy.Petroleum.Algeria.WeeklyImportsFromAlgeriaOfCrudeOil	U.S. Imports from Algeria of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Angola		
PET.W_EPC0_IM0_NUS-NAO_MBBLD.W	USEnergy.Petroleum.Angola.WeeklyImportsFromAngolaOfCrudeOil	U.S. Imports from Angola of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Brazil		
PET.W_EPC0_IM0_NUS-NBR_MBBLD.W	USEnergy.Petroleum.Brazil.WeeklyImportsFromBrazilOfCrudeOil	U.S. Imports from Brazil of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Canada		
PET.W_EPC0_IM0_NUS-NCA_MBBLD.W	USEnergy.Petroleum.Canada.WeeklyImportsFromCanadaOfCrudeOil	U.S. Imports from Canada of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Congo		
PET.W_EPC0_IM0_NUS-NCF_MBBLD.W	USEnergy.Petroleum.Congo.WeeklyImportsFromCongoBrazzavilleOfCrudeOil	U.S. Imports from Congo (Brazzaville) of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Colombia		
PET.W_EPC0_IM0_NUS-NCO_MBBLD.W	USEnergy.Petroleum.Colombia.WeeklyImportsFromColombiaOfCrudeOil	U.S. Imports from Colombia of Crude Oil in Thousand Barrels per Day (Mbbl/d)
Ecuador		
PET.W_EPC0_IM0_NUS-NEC_MBBLD.W	USEnergy.Petroleum.Ecuador.WeeklyImportsFromEcuadorOfCrudeOil	U.S. Imports from Ecuador of Crude Oil in Thousand Barrels per Day (Mbbl/d)

Example Applications

The EIA dataset enables you to monitor national and international oil production and consumption in you trading strategies. Examples include the following strategies:

- Trading petroleum companies when there is a change in net imports of petroleum products
- Trading country ETFs when there is a change in the country's net import of resources
- Adjusting exposure to vehicle manufacturer stocks when the supply of gasoline is higher/lower than historical levels

Classic Algorithm Example

The following example algorithm buy a petroleum company when net imports of petroleum products are increasing. When the net imports are decreasing, it short sells the company.

```py
from AlgorithmImports import *


class USEnergyDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting data, we trade Exxon Mobil Corp (XOM) as the proxy of petroleum investment
        self._xom = self.add_equity("XOM", Resolution.DAILY).symbol
        us_energy_symbol = self.add_data(USEnergy, USEnergy.Petroleum.UnitedStates.WEEKLY_NET_IMPORTS_OF_TOTAL_PETROLEUM_PRODUCTS).symbol

        # Historical data
        history = self.history(USEnergy, us_energy_symbol, 60, Resolution.DAILY)
        self.log(f"We got {len(history)} items from our history request")

        # Get latest value for net imports of petroleum products
        self.previous_value = history.loc[us_energy_symbol].values[-1, -1]

    def on_data(self, slice: Slice) -> None:
        # Gather the current net imports of petroleum products
        points = slice.get(USEnergy).values()
        if not points:
            return
        current_value = points[-1].value
        self.plot('Net Imports', 'Petroleum', current_value)

        # Buy when net imports of petroleum products are increasing
        if current_value > self.previous_value:
            self.set_holdings(self._xom, 1)

        # Short sell when net imports of petroleum products are decreasing
        elif current_value < self.previous_value:
            self.set_holdings(self._xom, -1)

        self.previous_value = current_value
```

Framework Algorithm Example

The following example algorithm buy a petroleum company when net imports of petroleum products are increasing. When the net imports are decreasing, it short sells the company.

```
PY
```

```
from AlgorithmImports import *


class USEnergyDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY
        # We only trade Exxon Mobil Corp (XOM) as the proxy of petroleum investment
        symbols = [Symbol.create("XOM", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # A custom alpha model that emit trade insights based on US Energy data
        self.add_alpha(USEnergyAlphaModel(self))
        # Equally invest to dissipate non-systematic capital concentration risk on individual stock
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class USEnergyAlphaModel(AlphaModel):

    tradable_symbols = []

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Request US weekly net import petroleum products data for trade signal generation
        self.us_energy_symbol = algorithm.add_data(USEnergy,
USEnergy.Petroleum.UnitedStates.WEEKLY_NET_IMPORTS_OF_TOTAL_PETROLEUM_PRODUCTS).symbol

        # Historical data
        history = algorithm.history(USEnergy, self.us_energy_symbol, 60, Resolution.DAILY)
        algorithm.log(f"We got {len(history)} items from our history request")
        # Get latest value for net imports of petroleum products for trade system readiness
        self.previous_value = history.loc[self.us_energy_symbol].values[-1, -1]

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Trade based on the current net imports of petroleum products
        points = slice.get(USEnergy).values()
        if not points:
            return []
        current_value = points[-1].value

        insights = []

        for symbol in self.tradable_symbols:
            # Buy when net imports of petroleum products are increasing, assuming the demand increases that bring up the petroleum price
            # Hence the inventory and revenue of the proxy increase, supporting the stock price goes up
            if current_value > self.previous_value:
                insights += [Insight.price(symbol, timedelta(days=7), InsightDirection.UP)]

            # Short sell when net imports of petroleum products are decreasing, decreasing demand lowers the petroleum price and the
proxy stock price
            elif current_value < self.previous_value:
                insights += [Insight.price(symbol, timedelta(days=7), InsightDirection.DOWN)]

        self.previous_value = current_value
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.tradable_symbols.append(security.symbol)
```

Data Point Attributes

The EIA dataset provides `USEnergy` objects, which have the following attributes:

Datasets

ExtractAlpha

ExtractAlpha was founded by Vinesh Jha in 2013 with the goal of providing alternative data for investors. ExtractAlpha's rigorously researched data sets and quantitative stock selection models leverage unique sources and analytical techniques, allowing users to gain an investment edge.

Cross Asset Model

Estimize

Tactical

True Beats

ExtractAlpha

Cross Asset Model

Introduction

The Cross Asset Model by ExtractAlpha provides stock scoring values based on the trading activity in the Options market. Since the Options market has a higher proportion of institutional traders than the Equities market, the Options market is composed of investors who are more informed and information-driven on average. The data covers a dynamic universe of over 3,000 US Equities, starts in July 2005, and is delivered on a daily frequency. This dataset is created by feature engineering on the Options market put-call spread, volatility skewness, and volume.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Cross Asset Model dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

ExtractAlpha was founded by Vinesh Jha in 2013 with the goal of providing alternative data for investors. ExtractAlpha's rigorously researched data sets and quantitative stock selection models leverage unique sources and analytical techniques, allowing users to gain an investment edge.

Getting Started

The following snippet demonstrates how to request data from the Cross Asset Model dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(ExtractAlphaCrossAssetModel, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	July 2005
Asset Coverage	Over 3,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Cross Asset Model data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class ExtractAlphaCrossAssetModelDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(ExtractAlphaCrossAssetModel, self.aapl).symbol
```

Accessing Data

To get the current Cross Asset Model data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} score at {slice.time}: {data_point.score}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(ExtractAlphaCrossAssetModel).items():
        self.log(f"{dataset_symbol} score at {slice.time}: {data_point.score}")
```

Historical Data

To get historical Cross Asset Model data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[ExtractAlphaCrossAssetModel](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

If you subscribe to Cross Asset Model data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Cross Asset Model dataset by ExtractAlpha enables you to utilize Options market information to extract alpha. Examples include the following strategies:

- Predicting price and volatility changes in Equities.
- Signaling arbitrage opportunities between Options and underlying assets.
- Using it as a stock selection indicator.

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, the algorithm forms an equal-weighted dollar-neutral portfolio of the 10 companies most likely to outperform and the 10 companies most likely to underperform.

```py
from AlgorithmImports import *

class ExtractAlphaCrossAssetModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self._points = {}
        self.add_universe(self._select_assets)
        # Add a Scheduled Event to rebalance the portfolio each day.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 30),
            self._rebalance
        )

    def _select_assets(self, coarse: List[Fundamental]) -> List[Symbol]:
        # Select non-penny stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by cross asset model data
        sorted_by_dollar_volume = sorted(
            [x for x in coarse if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]

    def on_data(self, slice: Slice) -> None:
        # Get the current data from the cross asset model.
        points = slice.get(ExtractAlphaCrossAssetModel)
        if points:
            self._points = points
            ## Demonstrate how to iterate through the data and access its members:
            #for dataset_symbol, model in points.items():
            #    self.quit(
            #        f"{self.time} -- "
            #        f"Asset Symbol: {dataset_symbol.underlying}; "
            #        f"Score: {model.score} "
            #    )

    def _rebalance(self):
        # Long the ones with the highest return estimates based on option trade data
        # Short the lowest return ones
        sorted_by_score = sorted(
            [
                x for x in self._points.items()
                # Remove assets that have no price or score.
                if self.securities[x[0].underlying].price and x[1].score
            ],
            key=lambda x: x[1].score
        )
        long_symbols = [x[0].underlying for x in sorted_by_score[-10:]]
        short_symbols = [x[0].underlying for x in sorted_by_score[:10]]

        # Liquidate the ones without a strong trading signal
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        long_targets = [PortfolioTarget(symbol, 0.05) for symbol in long_symbols]
        short_targets = [PortfolioTarget(symbol, -0.05) for symbol in short_symbols]
        self.set_holdings(long_targets + short_targets, True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting cross asset model data for trading signal generation
            security.cross_asset_model = self.add_data(
                ExtractAlphaCrossAssetModel, security.symbol
            ).symbol
            # Historical Data
            history = self.history(security.cross_asset_model, 2, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the cross asset model data for this asset when it leaves the universe.
            self.remove_security(security.cross_asset_model)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, the algorithm forms an equal-weighted dollar-neutral portfolio of the 10 companies most likely to outperform and the 10 companies most likely to underperform.

```py
from AlgorithmImports import *

class ExtractAlphaCrossAssetModelFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_universe_selection(LiquidEquitiesUniverseSelectionModel())
        # Custom alpha model emits insights based on cross asset model data
        self.add_alpha(ExtractAlphaCrossAssetModelAlphaModel())
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class LiquidEquitiesUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select non-penny stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by cross asset model data
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]


class ExtractAlphaCrossAssetModelAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Get the current data from the cross asset model.
        points = slice.get(ExtractAlphaCrossAssetModel)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, model in points.items():
        #    algorithm.quit(
        #        f"{algorithm.time} -- "
        #        f"Asset Symbol: {dataset_symbol.underlying}; "
        #        f"Score: {model.score} "
        #    )

        # Drop factors for assets that have no price or score.
        points = [
            x for x in points.items()
            if algorithm.securities[x[0].underlying].price and x[1].score
        ]
        # Only rebalance when there are new data points from the cross asset models.
        if not points:
            return []

        # Long the ones with the highest return estimates based on option trade data
        # Short the lowest return ones
        sorted_by_score = sorted(points, key=lambda x: x[1].score)
        insight_directions = (
            [(x, InsightDirection.DOWN) for x in sorted_by_score[:10]] +
            [(x, InsightDirection.UP) for x in sorted_by_score[-10:]]
        )
        return [
            Insight.price(x[0].underlying, Expiry.END_OF_DAY, direction)
            for x, direction in insight_directions
        ]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting cross asset model data for trading signal generation
            security.cross_asset_model = algorithm.add_data(
                ExtractAlphaCrossAssetModel, security.symbol
            ).symbol
            # Historical Data
            history = algorithm.history(security.cross_asset_model, 2, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the cross asset model data for this asset when it leaves the universe.
            algorithm.remove_security(security.cross_asset_model)
```

Data Point Attributes

The Cross Asset Model dataset provides `ExtractAlphaCrossAssetModel` objects, which have the following attributes:

ExtractAlpha

Estimize

Introduction

The Estimize dataset by ExtractAlpha estimates the financials of companies, including EPS, and revenues. The data covers over 2,800 US-listed Equities' EPS/Revenue. The data starts in January 2011 and is updated on a daily frequency. The data is sparse, and it doesn't have new updates every day. This dataset is crowdsourced from a community of 100,000+ contributors via the data provider's web platform.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Estimize dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

ExtractAlpha was founded by Vinesh Jha in 2013 with the goal of providing alternative data for investors. ExtractAlpha's rigorously researched data sets and quantitative stock selection models leverage unique sources and analytical techniques, allowing users to gain an investment edge.

Getting Started

The following snippet demonstrates how to request data from the Estimize dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.estimize_consensus_symbol = self.add_data(EstimizeConsensus, self.symbol).symbol
self.estimize_estimate_symbol = self.add_data(EstimizeEstimate, self.symbol).symbol
self.estimize_release_symbol = self.add_data(EstimizeRelease, self.symbol).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2011
Asset Coverage	2,800 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Estimize data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class ExtractAlphaEstimizeDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.estimize_consensus_symbol = self.add_data(EstimizeConsensus, self.aapl).symbol
        self.estimize_estimate_symbol = self.add_data(EstimizeEstimate, self.aapl).symbol
        self.estimize_release_symbol = self.add_data(EstimizeRelease, self.aapl).symbol
```

Accessing Data

To get the current Estimize data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.estimize_consensus_symbol):
        data_point = slice[self.estimize_consensus_symbol]
        self.log(f"{self.estimize_consensus_symbol} mean at {slice.time}: {data_point.mean}")

    if slice.contains_key(self.estimize_estimate_symbol):
        data_point = slice[self.estimize_estimate_symbol]
        self.log(f"{self.estimize_estimate_symbol} EPS at {slice.time}: {data_point.eps}")

    if slice.contains_key(self.estimize_release_symbol):
        data_point = slice[self.estimize_release_symbol]
        self.log(f"{self.estimize_release_symbol} EPS at {slice.time}: {data_point.eps}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(EstimizeConsensus).items():
        self.log(f"{dataset_symbol} mean at {slice.time}: {data_point.mentions}")

    for dataset_symbol, data_point in slice.get(EstimizeEstimate).items():
        self.log(f"{dataset_symbol} EPS at {slice.time}: {data_point.eps}")

    for dataset_symbol, data_point in slice.get(EstimizeRelease).items():
        self.log(f"{dataset_symbol} EPS at {slice.time}: {data_point.eps}")
```

Historical Data

To get historical Estimize data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrames
consensus_history_df = self.history(self.estimize_consensus_symbol, 100, Resolution.DAILY)
estimate_history_df = self.history(self.estimize_estimate_symbol, 100, Resolution.DAILY)
release_history_df = self.history(self.estimize_release_symbol, 100, Resolution.DAILY)
history_df = self.history([
    self.estimize_consensus_symbol,
    self.estimize_estimate_symbol,
    self.estimize_release_symbol], 100, Resolution.DAILY)

# Dataset objects
consensus_history_bars = self.history[EstimizeConsensus](self.estimize_consensus_symbol, 100, Resolution.DAILY)
estimate_history_bars = self.history[EstimizeEstimate](self.estimize_estimate_symbol, 100, Resolution.DAILY)
release_history_bars = self.history[EstimizeRelease](self.estimize_release_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.estimize_consensus_symbol)
self.remove_security(self.estimize_estimate_symbol)
self.remove_security(self.estimize_release_symbol)
```

If you subscribe to Estimize data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Estimize dataset enables you to estimate the financial data of a company more accurately for alpha. Examples include the following use cases:

- Fundamental estimates for ML regression/classification models

- Arbitrage/Sentiment trading on market “surprise” from ordinary expectations based on the better expectation by the dataset
- Using industry-specific KPIs to predict the returns of individual sectors

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 250 most liquid US Equities. Each morning, the algorithm forms an equal-weighted with the companies that have a new EPS estimate.

```py
from AlgorithmImports import *

class ExtractAlphaEstimizeAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a dictionary to hold the latest Consensus data since this dataset is sparse.
        self._consensus_by_symbol = {}
        # Add a universe of liquid US Equities.
        self.add_universe(self._select_assets)
        # Add a Scheduled Event to rebalance the portfolio each day.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            self._rebalance
        )

    def _select_assets(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select the non-penny stocks with the highest dollar volume,
        # since they have more stable price (lower risk) and more
        # informed insights from high market activities
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-250:]]

    def on_data(self, slice: Slice) -> None:
        # Record the latest Consensus data.
        consensus = slice.get(EstimizeConsensus)
        if not consensus:
            return
        for dataset_symbol, value in consensus.items():
            self._consensus_by_symbol[dataset_symbol.underlying] = value

    def _rebalance(self):
        if not self._consensus_by_symbol:
            return
        # If there has been new Consensus data released since the previous
        # market open, form an equal-weighted portfolio of all the assets
        # that have new Consensus data.
        symbols = [
            symbol for symbol in self._consensus_by_symbol.keys()
            if self.securities[symbol].price
        ]
        targets = [PortfolioTarget(symbol, 1/len(symbols)) for symbol in symbols]
        self.set_holdings(targets, True)
        self._consensus_by_symbol.clear()

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting data for trading signal generation
            security.consensus = self.add_data(EstimizeConsensus, security.symbol).symbol
            # Historical data
            history = self.history(security.consensus, 100, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the Consensus data for this asset when it leaves the universe.
            self.remove_security(security.consensus)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 250 most liquid US Equities. Each morning, the algorithm emits insights for the companies that have a new EPS estimate.

```py
from AlgorithmImports import *

class ExtractAlphaEstimizeFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_universe_selection(LiquidEquitiesUniverseSelectionModel())
        # Custom alpha model that emit signal according to Estimize data
        self.add_alpha(ExtractAlphaEstimizeAlphaModel())
        # Invest equally to evenly dissipate capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class LiquidEquitiesUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select the non-penny stocks with the highest dollar volume,
        # since they have more stable price (lower risk) and more
        # informed insights from high market activities
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-250:]]


class ExtractAlphaEstimizeAlphaModel(AlphaModel):

    # Add a dictionary to hold the latest Consensus data since this dataset is sparse.
    _consensus_by_symbol = {}
    _day = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Record the latest Consensus data.
        consensus = slice.get(EstimizeConsensus)
        if consensus:
            for dataset_symbol, value in consensus.items():
                self._consensus_by_symbol[dataset_symbol.underlying] = value
        # Wait for the market open to rebalance
        if self._day == algorithm.time.day or not slice.bars:
            return []
        self._day = algorithm.time.day
        # If there has been new Consensus data released since the previous
        # market open, emit insights for all the assets that have new Consensus data.
        if not self._consensus_by_symbol:
            return []
        symbols = [
            symbol for symbol in self._consensus_by_symbol.keys()
            if algorithm.securities[symbol].price
        ]
        self._consensus_by_symbol.clear()
        return [Insight.price(symbol, timedelta(30), InsightDirection.UP) for symbol in symbols]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting data for trading signal generation
            security.consensus = algorithm.add_data(EstimizeConsensus, security.symbol).symbol
            # Historical data
            history = algorithm.history(security.consensus, 100, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the Consensus data for this asset when it leaves the universe.
            algorithm.remove_security(security.consensus)
```

Data Point Attributes

The Estimize dataset provides `EstimizeConsensus` , `EstimizeEstimate` , and `EstimizeRelease` objects.

EstimizeConsensus Attributes

`EstimizeConsensus` objects have the following attributes:

EstimizeEstimate Attributes

`EstimizeEstimate` objects have the following attributes:

EstimizeRelease Attributes

`EstimizeRelease` objects have the following attributes:

ExtractAlpha

Tactical

Introduction

The Tactical dataset by ExtractAlpha is a stock scoring algorithm that captures the technical dynamics of individual US Equities over one to ten trading day horizons. It can assist a longer-horizon investor in timing their entry or exit points or be used in combination with existing systematic or qualitative strategies with similar holding periods.

The data covers a dynamic universe of around 4,700 US Equities per day on average, starts in January 2000, and is delivered on a daily frequency. The Tactical dataset expands upon simple reversal, liquidity, and seasonality factors to identify stocks that are likely to trend or reverse.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Tactical dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

ExtractAlpha was founded by Vinesh Jha in 2013 with the goal of providing alternative data for investors. ExtractAlpha's rigorously researched data sets and quantitative stock selection models leverage unique sources and analytical techniques, allowing users to gain an investment edge.

Getting Started

The following snippet demonstrates how to request data from the Tactical dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(ExtractAlphaTacticalModel, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2000
Asset Coverage	5,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Tactical data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class ExtractAlphaTacticalModelDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(ExtractAlphaTacticalModel, self.aapl).symbol
```

Accessing Data

To get the current Tactical data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} score at {slice.time}: {data_point.score}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(ExtractAlphaTacticalModel).items():
        self.log(f"{dataset_symbol} score at {slice.time}: {data_point.score}")
```

Historical Data

To get historical Tactical data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_df = self.history[ExtractAlphaTacticalModel](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.dataset_symbol)
```

If you subscribe to Tactical data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Tactical dataset enables you to gain insight into short-term stock dynamics for trading. Examples include the following strategies:

- Optimizing entry and exit times in a portfolio construction model.
- Using the raw factor values as technical indicators.
- Inputting the data into machine learning classifier models as trend/reversal labels.

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms an equal-weighted dollar-neutral portfolio of the 10 companies that are most likely to outperform and the 10 that are most likely to underperform.

```py
from AlgorithmImports import *

class ExtractAlphaTacticalModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self.add_universe(self._select_assets)

    def _select_assets(self, coarse: List[Fundamental]) -> List[Symbol]:
        # Select non-penny stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by tactical data
        sorted_by_dollar_volume = sorted(
            [x for x in coarse if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]

    def on_data(self, slice: Slice) -> None:
        # Get the current data from the tactical dataset.
        points = slice.get(ExtractAlphaTacticalModel)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, model in points.items():
        #    self.quit(
        #        f"{self.time} -- "
        #        f"Asset Symbol: {dataset_symbol.underlying}; "
        #        f"Score: {model.score} "
        #    )

        # Remove assets that have no price.
        points = [x for x in points.items() if self.securities[x[0].underlying].price]
        # Only rebalance when there are new tactical factors.
        if not points:
            return

        # Long the ones with the highest return estimates riding from tactical strategies
        # Short the lowest that predicted stock price goes down
        sorted_by_score = sorted(points, key=lambda x: x[1].score)
        long_symbols = [x[0].underlying for x in sorted_by_score[-10:]]
        short_symbols = [x[0].underlying for x in sorted_by_score[:10]]
        long_targets = [PortfolioTarget(symbol, 0.05) for symbol in long_symbols]
        short_targets = [PortfolioTarget(symbol, -0.05) for symbol in short_symbols]
        self.set_holdings(long_targets + short_targets, True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting tactical data for trading signal generation
            security.tactical_model = self.add_data(
                ExtractAlphaTacticalModel, security.symbol
            ).symbol
            # Historical Data
            history = self.history(security.tactical_model, 2, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the tactical model data for this asset when it leaves the universe.
            self.remove_security(security.tactical_model)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms an equal-weighted dollar-neutral portfolio of the 10 companies that are most likely to outperform and the 10 that are most likely to underperform.

```py
from AlgorithmImports import *

class ExtractAlphaTacticalModelFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self.add_universe_selection(LiquidEquitiesUniverseSelectionModel())
        # Custom alpha model to generate trading signal based on tactical data
        self.add_alpha(ExtractAlphaTacticalModelAlphaModel())
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class LiquidEquitiesUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select non-penny stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by tactical data
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data and x.price > 4],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]


class ExtractAlphaTacticalModelAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Get the current data from the tactical dataset.
        points = slice.get(ExtractAlphaTacticalModel)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, model in points.items():
        #    algorithm.quit(
        #        f"{algorithm.time} -- "
        #        f"Asset Symbol: {dataset_symbol.underlying}; "
        #        f"Score: {model.score} "
        #    )

        # Remove assets that have no price or score.
        points = [x for x in points.items() if algorithm.securities[x[0].underlying].price]

        # Long the ones with the highest return estimates riding from tactical strategies
        # Short the lowest that predicted stock price goes down
        sorted_by_score = sorted(points, key=lambda x: x[1].score)
        insight_directions = (
            [(x, InsightDirection.DOWN) for x in sorted_by_score[:10]] +
            [(x, InsightDirection.UP) for x in sorted_by_score[-10:]]
        )
        return [
            Insight.price(x[0].underlying, Expiry.END_OF_DAY, direction)
            for x, direction in insight_directions
        ]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting tactical data for trading signal generation
            security.tactical_model = algorithm.add_data(
                ExtractAlphaTacticalModel, security.symbol
            ).symbol
            # Historical Data
            history = algorithm.history(security.tactical_model, 2, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the tactical model data for this asset when it leaves the universe.
            algorithm.remove_security(security.tactical_model)
```

Data Point Attributes

The Tactical dataset provides `ExtractAlphaTacticalModel` objects, which have the following attributes:

ExtractAlpha

True Beats

Introduction

The True Beats dataset by ExtractAlpha provides quantitative predictions of EPS and Revenues for US Equities. The data covers a dynamic universe of around 4,000-5,000 US-listed Equities on a daily average. The data starts in January 2000 and is delivered on a daily frequency. This dataset is created by incorporating the opinions of expert analysts, historical earnings, revenue trends for the company and its peers, and metrics on company earnings management.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the True Beats dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

ExtractAlpha was founded by Vinesh Jha in 2013 with the goal of providing alternative data for investors. ExtractAlpha's rigorously researched data sets and quantitative stock selection models leverage unique sources and analytical techniques, allowing users to gain an investment edge.

Getting Started

The following snippet demonstrates how to request data from the True Beats dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(ExtractAlphaTrueBeats, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2000
Asset Coverage	Over 5,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add True Beats data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class ExtractAlphaTrueBeatsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(ExtractAlphaTrueBeats, self.aapl).symbol
```

Accessing Data

To get the current True Beats data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} True beat at {slice.time}: {data_point.true_beat}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(ExtractAlphaTrueBeats).items():
        self.log(f"{dataset_symbol} True beat at {slice.time}: {data_point.true_beat}")
```

Historical Data

To get historical True Beats data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY, flatten=True)

# Series
history_series = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[ExtractAlphaTrueBeats](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

If you subscribe to True Beats data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The True Beats dataset enables you to predict EPS and revenue of US-listed Equities for trading. Examples include the following strategies:

- Finding surprise in EPS or revenue for sentiment/arbitrage trading
- Stock or sector selections based on EPS or revenue predictions
- Calculate expected return by valuation models based on EPS or revenue predictions (e.g. Black-Litterman)

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms an equal-weighted dollar-neutral portfolio of the 10 best and 10 worst companies based on their predicted earnings per share.

```
from AlgorithmImports import *

class ExtractAlphaTrueBeatsDataAlgorithm(QCAlgorithm):

    _true_beats = None

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(100_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe that runs at the start of each month.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        date_rule = self.date_rules.month_start(spy)
        self.universe_settings.schedule.on(date_rule)
        self.add_universe(self._select_assets)
        # Add a Scheduled Event to rebalance the portfolio at the start of each month.
        self.schedule.on(date_rule, self.time_rules.before_market_close(spy, 30), self._rebalance)

    def _select_assets(self, coarse: List[Fundamental]) -> List[Symbol]:
        # Select the stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by true beats data
        sorted_by_dollar_volume = sorted(
            [x for x in coarse if x.has_fundamental_data],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]

    def on_data(self, slice: Slice) -> None:
        # Get the current data from the true beats dataset.
        points = slice.get(ExtractAlphaTrueBeats)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, true_beats in points.items():
        #    for true_beat in true_beats:
        #        self.quit(
        #            f"{self.time} -- "
        #            f"Asset Symbol: {dataset_symbol.underlying}; "
        #            f"Fiscal Year: {true_beat.fiscal_period.fiscal_year} "
        #            f"Fiscal Quarter: {true_beat.fiscal_period.fiscal_quarter} "
        #            f"True Beat: {true_beat.true_beat} "
        #        )
        if points:
            self._true_beats = points

    def _rebalance(self):
        eps_prediction_by_symbol = {}
        for dataset_symbol, true_beats in self._true_beats.items():
            # Select the subset of TrueBeat objects that have the EPS earnings metric,
            # a value for the fiscal year, and a value for the fiscal quarter.
            true_beats = [
                tb for tb in true_beats
                if (tb.earnings_metric == ExtractAlphaTrueBeatEarningsMetric.EPS and
                    tb.fiscal_period.fiscal_quarter)
            ]
            if not true_beats:
                continue
            # Select the TrueBeat object of the closest fiscal quarter.
            eps_prediction_by_symbol[dataset_symbol.underlying] = sorted(
                true_beats,
                key=lambda tb: (tb.fiscal_period.fiscal_year, tb.fiscal_period.fiscal_quarter)
            )[0].true_beat

        # Long the ones with the highest earning and revenue estimates due to fundamental factor may bring stock price up
        # Short the lowest that predicted to bring stock price down
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        sorted_by_eps_prediction = sorted(eps_prediction_by_symbol.items(), key=lambda kvp: kvp[1])
        long_targets = [PortfolioTarget(symbol, 0.05) for symbol, _ in sorted_by_eps_prediction[-10:]]
        short_targets = [PortfolioTarget(symbol, -0.05) for symbol, _ in sorted_by_eps_prediction[:10]]
        self.set_holdings(long_targets + short_targets, True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting true beats data for trading signal generation
            security.true_beats = self.add_data(ExtractAlphaTrueBeats, security.symbol).symbol
            # Historical Data
            history = self.history(security.true_beats, 10, Resolution.DAILY, flatten=True)
        for security in changes.removed_securities:
            # Remove the true beats data for this asset when it leaves the universe.
            self.remove_security(security.true_beats)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms an equal-weighted dollar-neutral portfolio of the 10 best and 10 worst companies based on their predicted earnings per share.

```
from AlgorithmImports import *
```

```
from AlgorithmImports import *

class ExtractAlphaTrueBeatsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 30)
        self.set_cash(100_000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe that runs at the start of each month.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        date_rule = self.date_rules.month_start(spy)
        self.universe_settings.schedule.on(date_rule)
        self.add_universe_selection(LiquidEquitiesUniverseSelectionModel())
        # Custom alpha model to trade based on true beats data signals
        self.add_alpha(ExtractAlphaTrueBeatsAlphaModel())
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class LiquidEquitiesUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select non-penny stocks with highest dollar volume due to better informed information from more market activities
        # Only the ones with fundamental data are supported by true beats data
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]


class ExtractAlphaTrueBeatsAlphaModel(AlphaModel):

    _month = -1
    _insights = []

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Rebalance at the start of each month.
        if self._month == algorithm.time.month:
            return []
        # Trade only based on the updated true beats data
        points = slice.get(ExtractAlphaTrueBeats)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, true_beats in points.items():
        #    for true_beat in true_beats:
        #        algorithm.quit(
        #            f"{algorithm.time} -- "
        #            f"Asset Symbol: {dataset_symbol.underlying}; "
        #            f"Fiscal Year: {true_beat.fiscal_period.fiscal_year} "
        #            f"Fiscal Quarter: {true_beat.fiscal_period.fiscal_quarter} "
        #            f"True Beat: {true_beat.true_beat} "
        #        )
        if not points:
            return []
        self._month = algorithm.time.month

        # Get the EPS prediction of each asset.
        eps_prediction_by_symbol = {}
        for dataset_symbol, true_beats in points.items():
            # Select the subset of TrueBeat objects that have the EPS earnings metric,
            # a value for the fiscal year, and a value for the fiscal quarter.
            true_beats = [
                tb for tb in true_beats
                if (tb.earnings_metric == ExtractAlphaTrueBeatEarningsMetric.EPS and
                    tb.fiscal_period.fiscal_quarter)
            ]
            if not true_beats:
                continue
            # Select the TrueBeat object of the closest fiscal quarter.
            eps_prediction_by_symbol[dataset_symbol.underlying] = sorted(
                true_beats,
                key=lambda tb: (tb.fiscal_period.fiscal_year, tb.fiscal_period.fiscal_quarter)
            )[0].true_beat

        # Cancel the previous insights.
        algorithm.insights.cancel(self._insights)
        # Emit the new insights.
        # Long the ones with the highest earning and revenue estimates due to fundamental factor may bring stock price up
        # Short the lowest that predicted to bring stock price down
        # Invest equally and dollar-neutral to evenly dissipate capital risk and hedge systematic risk
        self._insights = []
        sorted_by_eps_prediction = sorted(eps_prediction_by_symbol.items(), key=lambda kvp: kvp[1])
        for symbol, _ in sorted_by_eps_prediction[-10:]:
            self._insights.append(Insight.price(symbol, timedelta(35), InsightDirection.UP))
        for symbol, _ in sorted_by_eps_prediction[:10]:
            self._insights.append(Insight.price(symbol, timedelta(35), InsightDirection.DOWN))
        return self._insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting true beats data for trading signal generation
```

```
            security.true_beats = algorithm.add_data(ExtractAlphaTrueBeats, security.symbol).symbol
            # Historical Data
            history = algorithm.history(security.true_beats, 10, Resolution.DAILY, flatten=True)
        for security in changes.removed_securities:
            # Remove the true beats data for this asset when it leaves the universe.
            algorithm.remove_security(security.true_beats)
```

Data Point Attributes

The True Beats dataset provides `ExtractAlphaTrueBeats` and `ExtractAlphaTrueBeat` objects.

ExtractAlphaTrueBeats

`ExtractAlphaTrueBeats` objects have the following attributes:

ExtractAlphaTrueBeat

`ExtractAlphaTrueBeat` objects have the following attributes:

Datasets

FRED

The Research Division of the Federal Reserve bank of St. Louis, MO expands the frontier of economic knowledge by producing high-quality original research in the areas of macroeconomics, money and banking, and applied microeconomics. They contribute to monetary policy discussions by advising the Bank president on a range of topics, especially in preparation for Federal Open Market Committee (FOMC) meetings. The Research Division is in the top 1% of all economics research departments worldwide.

US Federal Reserve (FRED)

FRED

US Federal Reserve (FRED)

Introduction

The Federal Reserve Economic Data (FRED) by the Research Division of the Federal Reserve bank of St. Louis, MO provides various time series relating to macro-economic data. The data covers 560 datasets, starts in January 1999, and is delivered on a daily frequency. The data is created by aggregating daily updates from more than 85 public and proprietary sources.

For more information about the US Federal Reserve (FRED) dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The Research Division of the Federal Reserve bank of St. Louis, MO expands the frontier of economic knowledge by producing high-quality original research in the areas of macroeconomics, money and banking, and applied microeconomics. They contribute to monetary policy discussions by advising the Bank president on a range of topics, especially in preparation for Federal Open Market Committee (FOMC) meetings. The Research Division is in the top 1% of all economics research departments worldwide.

Getting Started

The following snippet demonstrates how to request data from the FRED dataset:

```
PY
self.dataset_symbol = self.add_data(Fred, "USARECDM", Resolution.DAILY).symbol
```

QuantConnect provides helpers to find some of the available datasets from FRED:

```
PY
self.dataset_symbol = self.add_data(Fred, Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_TROUGH,
Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1990
Asset Coverage	560 Datasets
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

To add FRED data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class FredAlternativeDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2003, 1, 1)
        self.set_end_date(2019, 10, 11)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(Fred, Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_TROUGH,
Resolution.DAILY).symbol
```

Accessing Data

To get the current FRED data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} value at {slice.time}: {data_point.value}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(Fred).items():
        self.log(f"{dataset_symbol} value at {slice.time}: {data_point.value}")
```

Historical Data

To get historical FRED data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[Fred](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove your subscription to FRED data, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Supported Datasets

The following lists show the accessor code you need to add each FRED dataset to your algorithm. This list does `not` include all fields from the FRED database. Please refer to FRED website for additional fields.

CBOE

- `Fred.CBOE.VIX_ON_GOOGLE` : CBOE Equity VIX on Google (in Index) (`VXGOGCLS`)
- `Fred.CBOE.VXD` : CBOE DJIA Volatility Index (in Index) (`VXDCLS`)
- `Fred.CBOE.VIX_ON_GOLDMAN_SACHS` : CBOE Equity VIX on Goldman Sachs (in Index) (`VXGSCLS`)
- `Fred.CBOE.VIX_ON_IBM` : CBOE Equity VIX on IBM (in Index) (`VXIBMCLS`)
- `Fred.CBOE.VIX_ON_AMAZON` : CBOE Equity VIX on Amazon (in Index) (`VXAZNCLS`)
- `Fred.CBOE.VXO` : CBOE S&P 100 Volatility Index: VXO (in Index) (`VXOCLS`)
- `Fred.CBOE.VXN` : CBOE NASDAQ 100 Volatility Index (in Index) (`VXNCLS`)

- Fred.CBOE.TEN_YEAR_TREASURY_NOTE_VOLATILITY_FUTURES : CBOE 10-Year Treasury Note Volatility Futures (in Index) (VXTYN)
- Fred.CBOE.RVX : CBOE Russell 2000 Volatility Index (in Index) (RVXCLS)
- Fred.CBOE.SP_500_THREE_MONTH_VOLATILITY_INDEX : CBOE S&P 500 3-Month Volatility Index (in Index) (VXVCLS)
- Fred.CBOE.VIX_ON_APPLE : CBOE Equity VIX on Apple (in Index) (VXAPLCLS)
- Fred.CBOE.GOLD_MINERS_ETF_VOLATILITY_INDEX : CBOE Gold Miners ETF Volatility Index (in Index) (VXGDXCLS)
- Fred.CBOE.CHINA_ETF_VOLATILITY_INDEX : CBOE China ETF Volatility Index (in Index) (VXFXICLS)
- Fred.CBOE.BRAZIL_ETF_VOLATILITY_INDEX : CBOE Brazil ETF Volatility Index (in Index) (VXEWZCLS)
- Fred.CBOE.EMERGING_MARKETS_ETF_VOLATILITY_INDEX : CBOE Emerging Markets ETF Volatility Index (in Index) (VXEEMCLS)
- Fred.CBOE.EURO_CURRENCY_ETF_VOLATILITY_INDEX : CBOE EuroCurrency ETF Volatility Index (in Index) (EVZCLS)
- Fred.CBOE.GOLD_ETF_VOLATILITY_INDEX : CBOE Gold ETF Volatility Index (in Index) (GVZCLS)
- Fred.CBOE.CRUDE_OIL_ETF_VOLATILITY_INDEX : CBOE Crude Oil ETF Volatility Index (in Index) (OVXCLS)
- Fred.CBOE.SILVER_ETF_VOLATILITY_INDEX : CBOE Silver ETF Volatility Index (in Index) (VXSLVCLS)
- Fred.CBOE.ENERGY_SECTOR_ETF_VOLATILITY_INDEX : CBOE Energy Sector ETF Volatility Index (in Index) (VXXLECLS)
- Fred.CBOE.VIX : CBOE Volatility Index: VIX (in Index) (VIXCLS)

Central Bank Interventions

- Fred.CentralBankInterventions.JAPANESE_BANK_PURCHASES_OF_DM_EURO_AGAINST_JPY : Japan Intervention: Japanese Bank purchases of DM/Euro against JPY (in 100 Million Yen) (JPINTDDMEJPY)
- Fred.CentralBankInterventions.JAPANESE_BANK_PURCHASES_OF_USD_AGAINST_DM : Japan Intervention: Japanese Bank purchases of USD against DM (in 100 Million Yen) (JPINTDEXR)
- Fred.CentralBankInterventions.JAPANESE_BANK_PURCHASES_OF_USD_AGAINST_RUPIAH : Japan Intervention: Japanese Bank purchases of USD against Rupiah (in 100 Million Yen) (JPINTDUSDRP)
- Fred.CentralBankInterventions.US_INTERVENTION_IN_MARKET_TRANSACTIONS_IN_THE_JPY_USD : U.S. Intervention: in Market Transactions in the JPY/USD (Millions of USD) (in Millions of USD) (USINTDMRKTJPY)
- Fred.CentralBankInterventions.US_INTERVENTION_WITH_CUSTOMER_TRANSACTIONS_IN_OTHER_CURRENCIES : U.S. Intervention: With-Customer Transactions in Other Currencies (Millions of USD) (in Millions of USD) (USINTDCSOTH)
- Fred.CentralBankInterventions.US_INTERVENTION_WITH_CUSTOMER_TRANSACTIONS_IN_THE_JPY_USD : U.S. Intervention: With-Customer Transactions in the JPY/USD (Millions of USD) (in Millions of USD) (USINTDCSJPY)
- Fred.CentralBankInterventions.US_INTERVENTION_WITH_CUSTOMER_TRANSACTIONS_IN_THE_DEM_USD_EURO : U.S. Intervention: With-Customer Transactions in the DEM/USD (Euro since 1999) (Millions of USD) (in Millions of USD) (USINTDCSDM)
- Fred.CentralBankInterventions.US_INTERVENTION_IN_MARKET_TRANSACTIONS_IN_OTHER_CURRENCIES : U.S. Intervention: in Market Transactions in Other Currencies (Millions of USD) (in Millions of USD) (USINTDMRKTOTH)
- Fred.CentralBankInterventions.CENTRAL_BANK_OF_TURKEY_PURCHASES_OF_USD : Turkish Intervention: Central Bank of Turkey Purchases of USD (Millions of USD) (in Millions of USD) (TRINTDEXR)
- Fred.CentralBankInterventions.JAPANESE_BANK_PURCHASES_OF_USD_AGAINST_JPY : Japan Intervention: Japanese Bank purchases of USD against JPY (in 100 Million Yen) (JPINTDUSDJPY)
- Fred.CentralBankInterventions.US_INTERVENTION_IN_MARKET_TRANSACTIONS_IN_THE_DEM_USD_EURO : U.S. Intervention: in Market Transactions in the DEM/USD (Euro since 1999) (Millions of USD) (in Millions of USD) (USINTDMRKTDM)
- Fred.CentralBankInterventions.SWISS_NATIONAL_BANK_PURCHASES_OF_DEM_AGAINST_CHF_MILLIONS_OF_DEM : Swiss Intervention: Swiss National Bank Purchases of DEM against CHF (Millions of DEM) (in Millions of DEM) (CHINTDCHFDM)
- Fred.CentralBankInterventions.SWISS_NATIONAL_BANK_PURCHASES_OF_USD_AGAINST_DEM : Swiss Intervention: Swiss National Bank Purchases of USD against DEM (Millions of USD) (in Millions of USD) (CHINTDUSDDM)
- Fred.CentralBankInterventions.SWISS_NATIONAL_BANK_PURCHASES_OF_USD_AGAINST_JPY : Swiss Intervention: Swiss National Bank Purchases of USD against JPY (Millions of USD) (in Millions of USD) (CHINTDUSDJPY)
- Fred.CentralBankInterventions.SWISS_NATIONAL_BANK_PURCHASES_OF_USD_AGAINST_CHF : Swiss Intervention: Swiss National Bank Purchases of USD against CHF (Millions of USD) (in Millions of USD) (CHINTDCHFUSD)
- Fred.CentralBankInterventions.BANCO_DE_MEXICO_PURCHASE_ON_THE_USD : Mexican Intervention: Banco de Mexico Purchase on the USD (in Millions of USD) (MEXINTDUSD)

Commercial Paper

- Fred.CommercialPaper.THREE_MONTH_AA_NONFINANCIAL_COMMERCIAL_PAPER_RATE : 3-Month AA Nonfinancial Commercial Paper Rate (in Percent) (DCPN3M)

- `Fred.CommercialPaper.ONE_MONTH_AA_NONFINANCIAL_COMMERCIAL_PAPER_RATE` : 1-Month AA Nonfinancial Commercial Paper Rate (in Percent) (`DCPN30`)
- `Fred.CommercialPaper.TWO_MONTH_AA_NONFINANCIAL_COMMERCIAL_PAPER_RATE` : 2-Month AA Nonfinancial Commercial Paper Rate (in Percent) (`DCPN2M`)
- `Fred.CommercialPaper.THREE_MONTH_AA_FINANCIAL_COMMERCIAL_PAPER_RATE` : 3-Month AA Financial Commercial Paper Rate (in Percent) (`DCPF3M`)
- `Fred.CommercialPaper.TWO_MONTH_AA_FINANCIAL_COMMERCIAL_PAPER_RATE` : 2-Month AA Financial Commercial Paper Rate (in Percent) (`DCPF2M`)
- `Fred.CommercialPaper.ONE_MONTH_AA_FINANCIAL_COMMERCIAL_PAPER_RATE` : 1-Month AA Financial Commercial Paper Rate (in Percent) (`DCPF1M`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Number of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (`NONFIN14A2P2VOL`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Number of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (`NONFIN59A2P2VOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN59A2P2AMT`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_NONFINANCIAL` : Number of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (`NONFIN4180AAVOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_ASSET_BACKED` : Total Value of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (`ABGT80AAAMT`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN4180AAAMT`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Number of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (`NONFIN4180A2P2VOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN4180A2P2AMT`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_NONFINANCIAL` : Number of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (`NONFIN2140AAVOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN2140AAAMT`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Number of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (`NONFIN2140A2P2VOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN2140A2P2AMT`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_NONFINANCIAL` : Number of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (`NONFIN14AAVOL`)
- `Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL` : Number of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (`NONFIN1020A2P2VOL`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_NONFINANCIAL` : Total Value of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (`NONFIN1020AAAMT`)
- `Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_ASSET_BACKED` : Total Value of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (`AB2140AAAMT`)

- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_NONFINANCIAL : Number of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (NONFIN1020AAVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL : Total Value of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFIN14A2P2AMT)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_NONFINANCIAL : Total Value of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFIN14AAAMT)
- Fred.CommercialPaper.TOTAL_VALUEOF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_1_AND_4_DAYS : Total Value of Commercial Paper Issues with a Maturity Between 1 and 4 Days (in Millions of Dollars) (MKT14MKTAMT)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL : Total Value of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFIN1020A2P2AMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FINGT80AAVOL)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FIN1020AAVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FIN14AAAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FIN14AAVOL)
- Fred.CommercialPaper.TOTAL_VALUEOF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_10_AND_20_DAYS : Total Value of Commercial Paper Issues with a Maturity Between 10 and 20 Days (in Millions of Dollars) (MKT1020MKTAMT)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_10_AND_20_DAYS : Number of Commercial Paper Issues with a Maturity Between 10 and 20 Days (in Number) (MKT1020MKTVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FIN2140AAAMT)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_1_AND_4_DAYS : Number of Commercial Paper Issues with a Maturity Between 1 and 4 Days (in Number) (MKT14MKTVOL)
- Fred.CommercialPaper.TOTAL_VALUEOF_ISSUERSOF_COMMERCIAL_PAPERWITHA_MATURITY_BETWEEN_21_AND_40_DAYS : Total Value of Issuers of Commercial Paper with a Maturity Between 21 and 40 Days (in Millions of Dollars) (MKT2140MKTAMT)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_21_AND_40_DAYS : Number of Commercial Paper Issues with a Maturity Between 21 and 40 Days (in Number) (MKT2140MKTVOL)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FIN2140AAVOL)
- Fred.CommercialPaper.TOTAL_VALUEOF_ISSUERSOF_COMMERCIAL_PAPERWITHA_MATURITY_BETWEEN_41_AND_80_DAYS : Total Value of Issuers of Commercial Paper with a Maturity Between 41 and 80 Days (in Millions of Dollars) (MKT4180MKTAMT)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_NONFINANCIAL : Total Value of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFIN59AAAMT)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_41_AND_80_DAYS : Number of Commercial Paper Issues with a Maturity Between 41 and 80 Days (in Number) (MKT4180MKTVOL)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_BETWEEN_5_AND_9_DAYS : Number of Commercial Paper Issues with a Maturity Between 5 and 9 Days (in Number) (MKT59MKTVOL)
- Fred.CommercialPaper.TOTAL_VALUEOF_ISSUERSOF_COMMERCIAL_PAPERWITHA_MATURITY_GREATER_THAN_80_DAYS : Total Value of Issuers of Commercial Paper with a Maturity Greater Than 80 Days (in Millions of Dollars) (MKTGT80MKTAMT)
- Fred.CommercialPaper.NUMBEROF_COMMERCIAL_PAPER_ISSUESWITHA_MATURITY_GREATER_THAN_80_DAYS : Number of Commercial Paper Issues with a Maturity Greater Than 80 Days (in Number) (MKTGT80MKTVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FIN4180AAAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FIN4180AAVOL)

- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_ASSET_BACKED : Total Value of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (AB4180AAAMT)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FIN59AAAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_FINANCIAL : Number of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Number) (FIN59AAVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FINGT80AAAMT)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_FINANCIAL : Total Value of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Financial Commercial Paper Rates (in Millions of Dollars) (FIN1020AAAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_21_AND_40_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Between 21 and 40 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (AB2140AAVOL)
- Fred.CommercialPaper.TOTAL_VALUEOF_ISSUERSOF_COMMERCIAL_PAPERWITHA_MATURITY_BETWEEN_5_AND_9_DAYS : Total Value of Issuers of Commercial Paper with a Maturity Between 5 and 9 Days (in Millions of Dollars) (MKT59MKTAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (ABGT80AAVOL)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_NONFINANCIAL : Number of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (NONFIN59AAVOL)
- Fred.CommercialPaper.FIFTEEN_DAY_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : 15-Day AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD15NB)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_ASSET_BACKED : Total Value of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (AB59AAAMT)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_41_AND_80_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Between 41 and 80 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (AB4180AAVOL)
- Fred.CommercialPaper.FIFTEEN_DAY_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 15-Day A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D15NB)
- Fred.CommercialPaper.SEVEN_DAY_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 7-Day A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D07NB)
- Fred.CommercialPaper.OVERNIGHT_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : Overnight A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D01NB)
- Fred.CommercialPaper.NINETY_DAY_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 90-Day AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD90NB)
- Fred.CommercialPaper.OVERNIGHT_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : Overnight AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD01NB)
- Fred.CommercialPaper.THREE_0_DAY_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 30-Day A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D30NB)
- Fred.CommercialPaper.SIXTY_DAY_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 60-Day AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD60NB)
- Fred.CommercialPaper.THREE_0_DAY_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 30-Day AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD30NB)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL : Total Value of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFINGT80A2P2AMT)
- Fred.CommercialPaper.THREE_0_DAY_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : 30-Day AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD30NB)
- Fred.CommercialPaper.SIXTY_DAY_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : 60-Day AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD60NB)
- Fred.CommercialPaper.NINETY_DAY_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : 90-Day AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD90NB)
- Fred.CommercialPaper.FIFTEEN_DAY_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 15-Day AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD15NB)

- Fred.CommercialPaper.SEVEN_DAY_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 7-Day AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD07NB)
- Fred.CommercialPaper.SEVEN_DAY_AA_ASSETBACKED_COMMERCIAL_PAPER_INTEREST_RATE : 7-Day AA Asset-backed Commercial Paper Interest Rate (in Percent) (RIFSPPAAAD07NB)
- Fred.CommercialPaper.OVERNIGHT_AA_FINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : Overnight AA Financial Commercial Paper Interest Rate (in Percent) (RIFSPPFAAD01NB)
- Fred.CommercialPaper.SIXTY_DAY_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 60-Day A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D60NB)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_5_AND_9_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Between 5 and 9 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (AB59AAVOL)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (AB14AAVOL)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_A_2_P_2_NONFINANCIAL : Number of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the A2/P2 Nonfinancial Commercial Paper Rates (in Number) (NONFINGT80A2P2VOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_1_AND_4_DAYS_USED_FOR_AA_ASSET_BACKED : Total Value of Issues, with a Maturity Between 1 and 4 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (AB14AAAMT)
- Fred.CommercialPaper.NINETY_DAY_A_2_P_2_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 90-Day A2/P2 Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNA2P2D90NB)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_ASSET_BACKED : Number of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Number) (AB1020AAVOL)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_NONFINANCIAL : Total Value of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Millions of Dollars) (NONFINGT80AAAMT)
- Fred.CommercialPaper.OVERNIGHT_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : Overnight AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD01NB)
- Fred.CommercialPaper.TOTAL_VALUE_OF_ISSUES_WITH_MATURITY_BETWEEN_10_AND_20_DAYS_USED_FOR_AA_ASSET_BACKED : Total Value of Issues, with a Maturity Between 10 and 20 Days, Used in Calculating the AA Asset-Backed Commercial Paper Rates (in Millions of Dollars) (AB1020AAAMT)
- Fred.CommercialPaper.SEVEN_DAY_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 7-Day AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD07NB)
- Fred.CommercialPaper.NINETY_DAY_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 90-Day AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD90NB)
- Fred.CommercialPaper.FIFTEEN_DAY_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 15-Day AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD15NB)
- Fred.CommercialPaper.THREE_0_DAY_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 30-Day AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD30NB)
- Fred.CommercialPaper.SIXTY_DAY_AA_NONFINANCIAL_COMMERCIAL_PAPER_INTEREST_RATE : 60-Day AA Nonfinancial Commercial Paper Interest Rate (in Percent) (RIFSPPNAAD60NB)
- Fred.CommercialPaper.NUMBER_OF_ISSUES_WITH_MATURITY_GREATER_THAN_80_DAYS_USED_FOR_AA_NONFINANCIAL : Number of Issues, with a Maturity Greater Than 80 Days, Used in Calculating the AA Nonfinancial Commercial Paper Rates (in Number) (NONFINGT80AAVOL)
- Fred.CommercialPaper.THREE_MONTH_COMMERCIAL_PAPER_MINUS_FEDERAL_FUNDS_RATE : 3-Month Commercial Paper Minus Federal Funds Rate (in Percent) (CPFF)

ICEB of AML

- Fred.ICEBofAML.AAAA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML AAA-A Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM1BRRAAA2ACRPITRIV)
- Fred.ICEBofAML.AAAAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML AAA-A US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM1RAAA2ALCRPIUSTRIV)
- Fred.ICEBofAML.ASIA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Asia Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMRACRPIASIATRIV)
- Fred.ICEBofAML.ASIA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Asia US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMALLCRPIASIAUSTRIV)

- Fred.ICEBofAML.BAND_LOWER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML B and Lower Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM4BRRBLCRPITRIV)
- Fred.ICEBofAML.BAND_LOWER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML B and Lower US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM4RBLLCRPIUSTRIV)
- Fred.ICEBofAML.BB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML BB Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM3BRRBBCRPITRIV)
- Fred.ICEBofAML.BBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML BB US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM3RBBLCRPIUSTRIV)
- Fred.ICEBofAML.BBB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML BBB Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM2BRRBBBCRPITRIV)
- Fred.ICEBofAML.BBBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML BBB US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM2RBBBLCRPIUSTRIV)
- Fred.ICEBofAML.CROSSOVER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Crossover Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEM5BCOCRPITRIV)
- Fred.ICEBofAML.CROSSOVER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Crossover US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMXOCOLCRPIUSTRIV)
- Fred.ICEBofAML.EMERGING_MARKETS_CORPORATE_PLUS_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Emerging Markets Corporate Plus Index Total Return Index Value (in Index) (BAMLEMCBPITRIV)
- Fred.ICEBofAML.EURO_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Euro Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMEBCRPIETRIV)
- Fred.ICEBofAML.EMEA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Europe, the Middle East, and Africa (EMEA) Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMRECRPIEMEATRIV)
- Fred.ICEBofAML.EMEAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Europe, the Middle East, and Africa (EMEA) US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMELLCRPIEMEAUSTRIV)
- Fred.ICEBofAML.FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Financial Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMFSFCRPITRIV)
- Fred.ICEBofAML.FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Financial US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMFLFLCRPIUSTRIV)
- Fred.ICEBofAML.HIGH_GRADE_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML High Grade Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMIBHGCRPITRIV)
- Fred.ICEBofAML.HIGH_GRADE_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML High Grade US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMHGHGLCRPIUSTRIV)
- Fred.ICEBofAML.HIGH_YIELD_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML High Yield Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMHBHYCRPITRIV)
- Fred.ICEBofAML.HIGH_YIELD_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML High Yield US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMHYHYLCRPIUSTRIV)
- Fred.ICEBofAML.LATIN_AMERICA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Latin America Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMRLCRPILATRIV)
- Fred.ICEBofAML.LATIN_AMERICA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Latin America US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMLLLCRPILAUSTRIV)
- Fred.ICEBofAML.NON_FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Non-Financial Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMNSNFCRPITRIV)
- Fred.ICEBofAML.NON_FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Non-Financial US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMNFNFLCRPIUSTRIV)
- Fred.ICEBofAML.US_CORPORATE_MASTER_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate Master Option-Adjusted Spread (in Percent) (BAMLC0A0CM)
- Fred.ICEBofAML.US_HIGH_YIELD_MASTER_II_OPTION_ADJUSTED_SPREAD : ICE BofAML US High Yield Master II Option-Adjusted Spread (in Percent) (BAMLH0A0HYM2)
- Fred.ICEBofAML.US_CORPORATE_1_TO_3_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 1-3 Year Option-Adjusted Spread (in Percent) (BAMLC1A0C13Y)
- Fred.ICEBofAML.US_CORPORATE_10_TO_15_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 10-15 Year Option-Adjusted Spread (in

Percent) (BAMLC7A0C1015Y)

- Fred.ICEBofAML.US_CORPORATE_MORE_THAN_15_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 15+ Year Option-Adjusted Spread (in Percent) (BAMLC8A0C15PY)
- Fred.ICEBofAML.US_CORPORATE_3_TO_5_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 3-5 Year Option-Adjusted Spread (in Percent) (BAMLC2A0C35Y)
- Fred.ICEBofAML.US_CORPORATE_5_TO_7_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 5-7 Year Option-Adjusted Spread (in Percent) (BAMLC3A0C57Y)
- Fred.ICEBofAML.US_CORPORATE_7_TO_10_YEAR_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate 7-10 Year Option-Adjusted Spread (in Percent) (BAMLC4A0C710Y)
- Fred.ICEBofAML.PUBLIC_SECTOR_ISSUERS_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Public Sector Issuers US Emerging Markets Liquid Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMPUPUBSLCRPIUSTRIV)
- Fred.ICEBofAML.US_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Emerging Markets Corporate Plus Sub-Index Total Return Index Value (in Index) (BAMLEMUBCRPIUSTRIV)
- Fred.ICEBofAML.US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Emerging Markets Liquid Corporate Plus Index Total Return Index Value (in Index) (BAMLEMCLLCRPIUSTRIV)
- Fred.ICEBofAML.EURO_HIGH_YIELD_INDEX_TOTAL_RETURN_INDEX_VALUE : ICE BofAML Euro High Yield Index Total Return Index Value (in Index) (BAMLHE00EHYITRIV)
- Fred.ICEBofAML.US_CORP_1_TO_3_YEARS_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp 1-3yr Total Return Index Value (in Index) (BAMLCC1A013YTRIV)
- Fred.ICEBofAML.US_CORP_10_TO_15_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp 10-15yr Total Return Index Value (in Index) (BAMLCC7A01015YTRIV)
- Fred.ICEBofAML.US_CORP_MORE_THAN_15_YEARS_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp 15+yr Total Return Index Value (in Index) (BAMLCC8A015PYTRIV)
- Fred.ICEBofAML.US_CORPE_TO_5_YEARS_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp 3-5yr Total Return Index Value (in Index) (BAMLCC2A035YTRIV)
- Fred.ICEBofAML.US_CORP_5_TO_7_YEARS_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp 5-7yr Total Return Index Value (in Index) (BAMLCC3A057YTRIV)
- Fred.ICEBofAML.US_CORPORATE_7_TO_10_YEARS_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corporate 7-10yr Total Return Index Value (in Index) (BAMLCC4A0710YTRIV)
- Fred.ICEBofAML.US_CORP_A_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp A Total Return Index Value (in Index) (BAMLCC0A3ATRIV)
- Fred.ICEBofAML.US_CORP_AA_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp AA Total Return Index Value (in Index) (BAMLCC0A2AATRIV)
- Fred.ICEBofAML.US_CORP_AAA_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp AAA Total Return Index Value (in Index) (BAMLCC0A1AAATRIV)
- Fred.ICEBofAML.US_HIGH_YIELD_B_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US High Yield B Total Return Index Value (in Index) (BAMLHYH0A2BTRIV)
- Fred.ICEBofAML.US_HIGH_YIELD_BB_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US High Yield BB Total Return Index Value (in Index) (BAMLHYH0A1BBTRIV)
- Fred.ICEBofAML.US_CORP_BBB_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp BBB Total Return Index Value (in Index) (BAMLCC0A4BBBTRIV)
- Fred.ICEBofAML.US_HIGH_YIELD_CC_COR_BELOW_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US High Yield CCC or Below Total Return Index Value (in Index) (BAMLHYH0A3CMTRIV)
- Fred.ICEBofAML.US_CORP_MASTER_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US Corp Master Total Return Index Value (in Index) (BAMLCC0A0CMTRIV)
- Fred.ICEBofAML.US_HIGH_YIELD_MASTER_II_TOTAL_RETURN_INDEX_VALUE : ICE BofAML US High Yield Master II Total Return Index Value (in Index) (BAMLHYH0A0HYM2TRIV)
- Fred.ICEBofAML.AAAA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML AAA-A Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM1BRRAAA2ACRPIOAS)
- Fred.ICEBofAML.AAAAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML AAA-A US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM1RAAA2ALCRPIUSOAS)
- Fred.ICEBofAML.ASIA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Asia Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMRACRPIASIAOAS)

- Fred.ICEBofAML.ASIA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Asia US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMALLCRPIASIAUSOAS)
- Fred.ICEBofAML.BAND_LOWER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML B and Lower Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM4BRRBLCRPIOAS)
- Fred.ICEBofAML.BAND_LOWER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML B and Lower US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM4RBLLCRPIUSOAS)
- Fred.ICEBofAML.BB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML BB Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM3BRRBBCRPIOAS)
- Fred.ICEBofAML.BBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML BB US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM3RBBLCRPIUSOAS)
- Fred.ICEBofAML.BBB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML BBB Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM2BRRBBBCRPIOAS)
- Fred.ICEBofAML.BBBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML BBB US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM2RBBBLCRPIUSOAS)
- Fred.ICEBofAML.CROSSOVER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Crossover Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEM5BCOCRPIOAS)
- Fred.ICEBofAML.CROSSOVER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Crossover US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMXOCOLCRPIUSOAS)
- Fred.ICEBofAML.EMERGING_MARKETS_CORPORATE_PLUS_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Emerging Markets Corporate Plus Index Option-Adjusted Spread (in Percent) (BAMLEMCBPIOAS)
- Fred.ICEBofAML.EURO_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Euro Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMEBCRPIEOAS)
- Fred.ICEBofAML.EMEA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Europe, the Middle East, and Africa (EMEA) Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMRECRPIEMEAOAS)
- Fred.ICEBofAML.EMEAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Europe, the Middle East, and Africa (EMEA) US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMELLCRPIEMEAUSOAS)
- Fred.ICEBofAML.FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Financial Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMFSFCRPIOAS)
- Fred.ICEBofAML.FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Financial US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMFLFLCRPIUSOAS)
- Fred.ICEBofAML.HIGH_GRADE_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML High Grade Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMIBHGCRPIOAS)
- Fred.ICEBofAML.HIGH_GRADE_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML High Grade US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMHGHGLCRPIUSOAS)
- Fred.ICEBofAML.HIGH_YIELD_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML High Yield Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMHBHYCRPIOAS)
- Fred.ICEBofAML.HIGH_YIELD_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML High Yield US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMHYHYLCRPIUSOAS)
- Fred.ICEBofAML.LATIN_AMERICA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Latin America Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMRLCRPILAOAS)
- Fred.ICEBofAML.LATIN_AMERICA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Latin America US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMLLLCRPILAUSOAS)
- Fred.ICEBofAML.NON_FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Non-Financial Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMNSNFCRPIOAS)
- Fred.ICEBofAML.NON_FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Non-Financial US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMNFNFLCRPIUSOAS)
- Fred.ICEBofAML.PUBLIC_SECTOR_ISSUERS_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Public Sector Issuers US Emerging Markets Liquid Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMPUPUBSLCRPIUSOAS)
- Fred.ICEBofAML.US_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML US Emerging Markets Corporate Plus Sub-Index Option-Adjusted Spread (in Percent) (BAMLEMUBCRPIUSOAS)

- Fred.ICEBofAML.US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML US Emerging Markets Liquid Corporate Plus Index Option-Adjusted Spread (in Percent) (BAMLEMCLLCRPIUSOAS)
- Fred.ICEBofAML.EURO_HIGH_YIELD_INDEX_OPTION_ADJUSTED_SPREAD : ICE BofAML Euro High Yield Index Option-Adjusted Spread (in Percent) (BAMLHE00EHYIOAS)
- Fred.ICEBofAML.US_CORPORATE_A_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate A Option-Adjusted Spread (in Percent) (BAMLC0A3CA)
- Fred.ICEBofAML.US_CORPORATE_AA_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate AA Option-Adjusted Spread (in Percent) (BAMLC0A2CAA)
- Fred.ICEBofAML.US_CORPORATE_AAA_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate AAA Option-Adjusted Spread (in Percent) (BAMLC0A1CAAA)
- Fred.ICEBofAML.US_HIGH_YIELD_B_OPTION_ADJUSTED_SPREAD : ICE BofAML US High Yield B Option-Adjusted Spread (in Percent) (BAMLH0A2HYB)
- Fred.ICEBofAML.US_HIGH_YIELD_BB_OPTION_ADJUSTED_SPREAD : ICE BofAML US High Yield BB Option-Adjusted Spread (in Percent) (BAMLH0A1HYBB)
- Fred.ICEBofAML.US_CORPORATE_BBB_OPTION_ADJUSTED_SPREAD : ICE BofAML US Corporate BBB Option-Adjusted Spread (in Percent) (BAMLC0A4CBBB)
- Fred.ICEBofAML.US_HIGH_YIELD_CC_COR_BELOW_OPTION_ADJUSTED_SPREAD : ICE BofAML US High Yield CCC or Below Option-Adjusted Spread (in Percent) (BAMLH0A3HYC)
- Fred.ICEBofAML.AAAA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML AAA-A Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM1BRRAAA2ACRPIEY)
- Fred.ICEBofAML.AAAAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML AAA-A US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM1RAAA2ALCRPIUSEY)
- Fred.ICEBofAML.ASIA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Asia Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMRACRPIASIAEY)
- Fred.ICEBofAML.ASIA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Asia US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMALLCRPIASIAUSEY)
- Fred.ICEBofAML.BAND_LOWER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML B and Lower Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM4BRRBLCRPIEY)
- Fred.ICEBofAML.BAND_LOWER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML B and Lower US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM4RBLLCRPIUSEY)
- Fred.ICEBofAML.BB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML BB Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM3BRRBBCRPIEY)
- Fred.ICEBofAML.BBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML BB US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM3RBBLCRPIUSEY)
- Fred.ICEBofAML.BBB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML BBB Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM2BRRBBBCRPIEY)
- Fred.ICEBofAML.BBBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML BBB US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM2RBBBLCRPIUSEY)
- Fred.ICEBofAML.CROSSOVER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Crossover Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEM5BCOCRPIEY)
- Fred.ICEBofAML.CROSSOVER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Crossover US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMXOCOLCRPIUSEY)
- Fred.ICEBofAML.EMERGING_MARKETS_CORPORATE_PLUS_INDEX_EFFECTIVE_YIELD : ICE BofAML Emerging Markets Corporate Plus Index Effective Yield (in Percent) (BAMLEMCBPIEY)
- Fred.ICEBofAML.EURO_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Euro Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMEBCRPIEEY)
- Fred.ICEBofAML.EURO_HIGH_YIELD_INDEX_EFFECTIVE_YIELD : ICE BofAML Euro High Yield Index Effective Yield (in Percent) (BAMLHE00EHYIEY)
- Fred.ICEBofAML.EMEA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Europe, the Middle East, and Africa (EMEA) Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMRECRPIEMEAEY)
- Fred.ICEBofAML.EMEAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Europe, the Middle East, and Africa (EMEA) US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMELLCRPIEMEAUSEY)
- Fred.ICEBofAML.FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Financial Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMFSFCRPIEY)

- Fred.ICEBofAML.FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Financial US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMFLFLCRPIUSEY)
- Fred.ICEBofAML.HIGH_GRADE_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML High Grade Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMIBHGCRPIEY)
- Fred.ICEBofAML.HIGH_GRADE_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML High Grade US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMHGHGLCRPIUSEY)
- Fred.ICEBofAML.HIGH_YIELD_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML High Yield Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMHBHYCRPIEY)
- Fred.ICEBofAML.HIGH_YIELD_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML High Yield US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMHYHYLCRPIUSEY)
- Fred.ICEBofAML.LATIN_AMERICA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Latin America Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMRLCRPILAEY)
- Fred.ICEBofAML.LATIN_AMERICA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Latin America US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMLLLCRPILAUSEY)
- Fred.ICEBofAML.NON_FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Non-Financial Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMNSNFCRPIEY)
- Fred.ICEBofAML.NON_FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Non-Financial US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMNFNFLCRPIUSEY)
- Fred.ICEBofAML.PUBLIC_SECTOR_ISSUERS_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML Public Sector Issuers US Emerging Markets Liquid Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMPUPUBSLCRPIUSEY)
- Fred.ICEBofAML.US_CORPORATE_1_THREE_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 1-3 Year Effective Yield (in Percent) (BAMLC1A0C13YEY)
- Fred.ICEBofAML.US_CORPORATE_10_TO_15_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 10-15 Year Effective Yield (in Percent) (BAMLC7A0C1015YEY)
- Fred.ICEBofAML.US_CORPORATE_MORE_THAN_15_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 15+ Year Effective Yield (in Percent) (BAMLC8A0C15PYEY)
- Fred.ICEBofAML.US_CORPORATE_3_TO_5_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 3-5 Year Effective Yield (in Percent) (BAMLC2A0C35YEY)
- Fred.ICEBofAML.US_CORPORATE_5_TO_7_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 5-7 Year Effective Yield (in Percent) (BAMLC3A0C57YEY)
- Fred.ICEBofAML.US_CORPORATE_7_TO_10_YEAR_EFFECTIVE_YIELD : ICE BofAML US Corporate 7-10 Year Effective Yield (in Percent) (BAMLC4A0C710YEY)
- Fred.ICEBofAML.US_CORPORATE_A_EFFECTIVE_YIELD : ICE BofAML US Corporate A Effective Yield (in Percent) (BAMLC0A3CAEY)
- Fred.ICEBofAML.US_CORPORATE_AA_EFFECTIVE_YIELD : ICE BofAML US Corporate AA Effective Yield (in Percent) (BAMLC0A2CAAEY)
- Fred.ICEBofAML.US_CORPORATE_AAA_EFFECTIVE_YIELD : ICE BofAML US Corporate AAA Effective Yield (in Percent) (BAMLC0A1CAAAEY)
- Fred.ICEBofAML.US_HIGH_YIELD_B_EFFECTIVE_YIELD : ICE BofAML US High Yield B Effective Yield (in Percent) (BAMLH0A2HYBEY)
- Fred.ICEBofAML.US_HIGH_YIELD_BB_EFFECTIVE_YIELD : ICE BofAML US High Yield BB Effective Yield (in Percent) (BAMLH0A1HYBBEY)
- Fred.ICEBofAML.US_CORPORATE_BBB_EFFECTIVE_YIELD : ICE BofAML US Corporate BBB Effective Yield (in Percent) (BAMLC0A4CBBBEY)
- Fred.ICEBofAML.US_HIGH_YIELD_CC_COR_BELOW_EFFECTIVE_YIELD : ICE BofAML US High Yield CCC or Below Effective Yield (in Percent) (BAMLH0A3HYCEY)
- Fred.ICEBofAML.US_CORPORATE_MASTER_EFFECTIVE_YIELD : ICE BofAML US Corporate Master Effective Yield (in Percent) (BAMLC0A0CMEY)
- Fred.ICEBofAML.US_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_EFFECTIVE_YIELD : ICE BofAML US Emerging Markets Corporate Plus Sub-Index Effective Yield (in Percent) (BAMLEMUBCRPIUSEY)
- Fred.ICEBofAML.US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_INDEX_EFFECTIVE_YIELD : ICE BofAML US Emerging Markets Liquid Corporate Plus Index Effective Yield (in Percent) (BAMLEMCLLCRPIUSEY)
- Fred.ICEBofAML.US_HIGH_YIELD_MASTER_II_EFFECTIVE_YIELD : ICE BofAML US High Yield Master II Effective Yield (in Percent) (BAMLH0A0HYM2EY)
- Fred.ICEBofAML.AAAA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML AAA-A Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM1BRRAAA2ACRPISYTW)
- Fred.ICEBofAML.AAAAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML AAA-A US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM1RAAA2ALCRPIUSSYTW)
- Fred.ICEBofAML.ASIA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Asia Emerging Markets

Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMRACRPIASIASYTW)

- Fred.ICEBofAML.ASIA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Asia US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMALLCRPIASIAUSSYTW)
- Fred.ICEBofAML.BAND_LOWER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML B and Lower Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM4BRRBLCRPISYTW)
- Fred.ICEBofAML.BAND_LOWER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML B and Lower US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM4RBLLCRPIUSSYTW)
- Fred.ICEBofAML.BB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML BB Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM3BRRBBCRPISYTW)
- Fred.ICEBofAML.BBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML BB US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM3RBBLCRPIUSSYTW)
- Fred.ICEBofAML.BBB_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML BBB Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM2BRRBBBCRPISYTW)
- Fred.ICEBofAML.BBBUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML BBB US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM2RBBBLCRPIUSSYTW)
- Fred.ICEBofAML.CROSSOVER_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Crossover Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEM5BCOCRPISYTW)
- Fred.ICEBofAML.CROSSOVER_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Crossover US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMXOCOLCRPIUSSYTW)
- Fred.ICEBofAML.EMERGING_MARKETS_CORPORATE_PLUS_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Emerging Markets Corporate Plus Index Semi-Annual Yield to Worst (in Percent) (BAMLEMCBPISYTW)
- Fred.ICEBofAML.EURO_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Euro Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMEBCRPIESYTW)
- Fred.ICEBofAML.EURO_HIGH_YIELD_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Euro High Yield Index Semi-Annual Yield to Worst (in Percent) (BAMLHE00EHYISYTW)
- Fred.ICEBofAML.EMEA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Europe, the Middle East, and Africa (EMEA) Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMRECRPIEMEASYTW)
- Fred.ICEBofAML.EMEAUS_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Europe, the Middle East, and Africa (EMEA) US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMELLCRPIEMEAUSSYTW)
- Fred.ICEBofAML.FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Financial Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMFSFCRPISYTW)
- Fred.ICEBofAML.FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Financial US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMFLFLCRPIUSSYTW)
- Fred.ICEBofAML.HIGH_GRADE_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML High Grade Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMIBHGCRPISYTW)
- Fred.ICEBofAML.HIGH_GRADE_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML High Grade US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMHGHGLCRPIUSSYTW)
- Fred.ICEBofAML.HIGH_YIELD_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML High Yield Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMHBHYCRPISYTW)
- Fred.ICEBofAML.HIGH_YIELD_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML High Yield US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMHYHYLCRPIUSSYTW)
- Fred.ICEBofAML.LATIN_AMERICA_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Latin America Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMRLCRPILASYTW)
- Fred.ICEBofAML.LATIN_AMERICA_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Latin America US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMLLLCRPILAUSSYTW)
- Fred.ICEBofAML.NON_FINANCIAL_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Non-Financial Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMNSNFCRPISYTW)
- Fred.ICEBofAML.NON_FINANCIAL_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Non-Financial US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMNFNFLCRPIUSSYTW)
- Fred.ICEBofAML.PRIVATE_SECTOR_ISSUERS_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Private Sector Issuers Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMPTPRVICRPISYTW)

- Fred.ICEBofAML.PRIVATE_SECTOR_ISSUERS_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Private Sector Issuers US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMPVPRIVSLCRPIUSSYTW)
- Fred.ICEBofAML.PUBLIC_SECTOR_ISSUERS_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Public Sector Issuers Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMPBPUBSICRPISYTW)
- Fred.ICEBofAML.PUBLIC_SECTOR_ISSUERS_US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML Public Sector Issuers US Emerging Markets Liquid Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMPUPUBSLCRPIUSSYTW)
- Fred.ICEBofAML.US_CORPORATE_1_TO_3_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 1-3 Year Semi-Annual Yield to Worst (in Percent) (BAMLC1A0C13YSYTW)
- Fred.ICEBofAML.US_CORPORATE_10_TO_15_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 10-15 Year Semi-Annual Yield to Worst (in Percent) (BAMLC7A0C1015YSYTW)
- Fred.ICEBofAML.US_CORPORATE_MORE_THAN_15_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 15+ Year Semi-Annual Yield to Worst (in Percent) (BAMLC8A0C15PYSYTW)
- Fred.ICEBofAML.US_CORPORATE_3_TO_5_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 3-5 Year Semi-Annual Yield to Worst (in Percent) (BAMLC2A0C35YSYTW)
- Fred.ICEBofAML.US_CORPORATE_5_TO_7_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 5-7 Year Semi-Annual Yield to Worst (in Percent) (BAMLC3A0C57YSYTW)
- Fred.ICEBofAML.US_CORPORATE_7_TO_10_YEAR_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate 7-10 Year Semi-Annual Yield to Worst (in Percent) (BAMLC4A0C710YSYTW)
- Fred.ICEBofAML.US_CORPORATE_A_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate A Semi-Annual Yield to Worst (in Percent) (BAMLC0A3CASYTW)
- Fred.ICEBofAML.US_CORPORATE_AA_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate AA Semi-Annual Yield to Worst (in Percent) (BAMLC0A2CAASYTW)
- Fred.ICEBofAML.US_CORPORATE_AAA_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate AAA Semi-Annual Yield to Worst (in Percent) (BAMLC0A1CAAASYTW)
- Fred.ICEBofAML.US_HIGH_YIELD_B_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US High Yield B Semi-Annual Yield to Worst (in Percent) (BAMLH0A2HYBSYTW)
- Fred.ICEBofAML.US_HIGH_YIELD_BB_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US High Yield BB Semi-Annual Yield to Worst (in Percent) (BAMLH0A1HYBBSYTW)
- Fred.ICEBofAML.US_CORPORATE_BBB_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate BBB Semi-Annual Yield to Worst (in Percent) (BAMLC0A4CBBBSYTW)
- Fred.ICEBofAML.US_HIGH_YIELD_CC_COR_BELOW_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US High Yield CCC or Below Semi-Annual Yield to Worst (in Percent) (BAMLH0A3HYCSYTW)
- Fred.ICEBofAML.US_CORPORATE_MASTER_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Corporate Master Semi-Annual Yield to Worst (in Percent) (BAMLC0A0CMSYTW)
- Fred.ICEBofAML.US_EMERGING_MARKETS_CORPORATE_PLUS_SUB_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Emerging Markets Corporate Plus Sub-Index Semi-Annual Yield to Worst (in Percent) (BAMLEMUBCRPIUSSYTW)
- Fred.ICEBofAML.US_EMERGING_MARKETS_LIQUID_CORPORATE_PLUS_INDEX_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US Emerging Markets Liquid Corporate Plus Index Semi-Annual Yield to Worst (in Percent) (BAMLEMCLLCRPIUSSYTW)
- Fred.ICEBofAML.US_HIGH_YIELD_MASTER_II_SEMI_ANNUAL_YIELDTO_WORST : ICE BofAML US High Yield Master II Semi-Annual Yield to Worst (in Percent) (BAMLH0A0HYM2SYTW)

LIBOR

- Fred.LIBOR.SPOT_NEXT_BASED_ON_SWISS_FRANC : Spot Next London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (CHFONTD156N)
- Fred.LIBOR.SPOT_NEXT_BASED_ON_JAPANESE_YEN : Spot Next London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (JPYONTD156N)
- Fred.LIBOR.SIX_MONTH_BASED_ON_JAPANESE_YEN : 6-Month London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (JPY6MTD156N)
- Fred.LIBOR.THREE_MONTH_BASED_ON_JAPANESE_YEN : 3-Month London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (JPY3MTD156N)

- `Fred.LIBOR.SIX_MONTH_BASED_ON_USD` : 6-Month London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD6MTD156N`)
- `Fred.LIBOR.ONE_MONTH_BASED_ON_JAPANESE_YEN` : 1-Month London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (`JPY1MTD156N`)
- `Fred.LIBOR.TWELVE_MONTH_BASED_ON_JAPANESE_YEN` : 12-Month London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (`JPY12MD156N`)
- `Fred.LIBOR.TWELVE_MONTH_BASED_ON_BRITISH_POUND` : 12-Month London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP12MD156N`)
- `Fred.LIBOR.ONE_MONTH_BASED_ON_BRITISH_POUND` : 1-Month London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP1MTD156N`)
- `Fred.LIBOR.ONE_WEEK_BASED_ON_BRITISH_POUND` : 1-Week London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP1WKD156N`)
- `Fred.LIBOR.TWO_MONTH_BASED_ON_BRITISH_POUND` : 2-Month London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP2MTD156N`)
- `Fred.LIBOR.THREE_MONTH_BASED_ON_BRITISH_POUND` : 3-Month London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP3MTD156N`)
- `Fred.LIBOR.ONE_WEEK_BASED_ON_JAPANESE_YEN` : 1-Week London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (`JPY1WKD156N`)
- `Fred.LIBOR.TWO_MONTH_BASED_ON_JAPANESE_YEN` : 2-Month London Interbank Offered Rate (LIBOR), based on Japanese Yen (in Percent) (`JPY2MTD156N`)
- `Fred.LIBOR.SIX_MONTH_BASED_ON_SWISS_FRANC` : 6-Month London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF6MTD156N`)
- `Fred.LIBOR.THREE_MONTH_BASED_ON_SWISS_FRANC` : 3-Month London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF3MTD156N`)
- `Fred.LIBOR.ONE_MONTH_BASED_ON_USD` : 1-Month London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD1MTD156N`)
- `Fred.LIBOR.TWELVE_MONTH_BASED_ON_SWISS_FRANC` : 12-Month London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF12MD156N`)
- `Fred.LIBOR.TWELVE_MONTH_BASED_ON_USD` : 12-Month London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD12MD156N`)
- `Fred.LIBOR.ONE_MONTH_BASED_ON_SWISS_FRANC` : 1-Month London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF1MTD156N`)
- `Fred.LIBOR.ONE_WEEK_BASED_ON_SWISS_FRANC` : 1-Week London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF1WKD156N`)
- `Fred.LIBOR.TWO_MONTH_BASED_ON_SWISS_FRANC` : 2-Month London Interbank Offered Rate (LIBOR), based on Swiss Franc (in Percent) (`CHF2MTD156N`)
- `Fred.LIBOR.TWELVE_MONTH_BASED_ON_EURO` : 12-Month London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR12MD156N`)
- `Fred.LIBOR.SIX_MONTH_BASED_ON_BRITISH_POUND` : 6-Month London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBP6MTD156N`)
- `Fred.LIBOR.ONE_MONTH_BASED_ON_EURO` : 1-Month London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR1MTD156N`)
- `Fred.LIBOR.TWO_MONTH_BASED_ON_EURO` : 2-Month London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR2MTD156N`)
- `Fred.LIBOR.THREE_MONTH_BASED_ON_EURO` : 3-Month London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR3MTD156N`)
- `Fred.LIBOR.SIX_MONTH_BASED_ON_EURO` : 6-Month London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR6MTD156N`)
- `Fred.LIBOR.OVERNIGHT_BASED_ON_EURO` : Overnight London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EURONTD156N`)
- `Fred.LIBOR.ONE_WEEK_BASED_ON_USD` : 1-Week London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD1WKD156N`)
- `Fred.LIBOR.TWO_MONTH_BASED_ON_USD` : 2-Month London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD2MTD156N`)
- `Fred.LIBOR.THREE_MONTH_BASED_ON_USD` : 3-Month London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USD3MTD156N`)
- `Fred.LIBOR.OVERNIGHT_BASED_ON_USD` : Overnight London Interbank Offered Rate (LIBOR), based on U.S. Dollar (in Percent) (`USDONTD156N`)
- `Fred.LIBOR.ONE_WEEK_BASED_ON_EURO` : 1-Week London Interbank Offered Rate (LIBOR), based on Euro (in Percent) (`EUR1WKD156N`)
- `Fred.LIBOR.OVERNIGHT_BASED_ON_BRITISH_POUND` : Overnight London Interbank Offered Rate (LIBOR), based on British Pound (in Percent) (`GBPONTD156N`)

OECD Recession Indicators

- `Fred.OECDRecessionIndicators.FOUR_BIG_EUROPEAN_COUNTRIES_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Four Big European Countries from the Peak through the Trough (in +1 or 0) (`4BIGEURORECDM`)

- `Fred.OECDRecessionIndicators.AUSTRALIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Australia from the Peak through the Trough (in +1 or 0) (`AUSRECDM`)
- `Fred.OECDRecessionIndicators.AUSTRIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Austria from the Peak through the Trough (in +1 or 0) (`AUTRECDM`)
- `Fred.OECDRecessionIndicators.BELGIUM_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Belgium from the Peak through the Trough (in +1 or 0) (`BELRECDM`)
- `Fred.OECDRecessionIndicators.BRAZIL_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Brazil from the Peak through the Trough (in +1 or 0) (`BRARECDM`)
- `Fred.OECDRecessionIndicators.CANADA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Canada from the Peak through the Trough (in +1 or 0) (`CANRECDM`)
- `Fred.OECDRecessionIndicators.SWITZERLAND_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Switzerland from the Peak through the Trough (in +1 or 0) (`CHERECDM`)
- `Fred.OECDRecessionIndicators.CHILE_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Chile from the Peak through the Trough (in +1 or 0) (`CHLRECDM`)
- `Fred.OECDRecessionIndicators.CHINA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for China from the Peak through the Trough (in +1 or 0) (`CHNRECDM`)
- `Fred.OECDRecessionIndicators.CZECH_REPUBLIC_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the Czech Republic from the Peak through the Trough (in +1 or 0) (`CZERECDM`)
- `Fred.OECDRecessionIndicators.GERMANY_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Germany from the Peak through the Trough (in +1 or 0) (`DEURECDM`)
- `Fred.OECDRecessionIndicators.DENMARK_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Denmark from the Peak through the Trough (in +1 or 0) (`DNKRECDM`)
- `Fred.OECDRecessionIndicators.SPAIN_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Spain from the Peak through the Trough (in +1 or 0) (`ESPRECDM`)
- `Fred.OECDRecessionIndicators.ESTONIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Estonia from the Peak through the Trough (in +1 or 0) (`ESTRECDM`)
- `Fred.OECDRecessionIndicators.EURO_AREA_FROM_PEAK_THROUGH_THE_TROUGH` : Retrieved from FRED, Federal Reserve Bank of St. Louis; https://fred.stlouisfed.org/series/ESTRECDM (`EURORECDM`)
- `Fred.OECDRecessionIndicators.FINLAND_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Finland from the Peak through the Trough (in +1 or 0) (`FINRECDM`)
- `Fred.OECDRecessionIndicators.FRANCE_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for France from the Peak through the Trough (in +1 or 0) (`FRARECDM`)
- `Fred.OECDRecessionIndicators.UNITED_KINGDOM_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the United Kingdom from the Peak through the Trough (in +1 or 0) (`GBRRECDM`)
- `Fred.OECDRecessionIndicators.GREECE_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Greece from the Peak through the Trough (in +1 or 0) (`GRCRECDM`)
- `Fred.OECDRecessionIndicators.HUNGARY_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Hungary from the Peak through the Trough (in +1 or 0) (`HUNRECDM`)
- `Fred.OECDRecessionIndicators.INDONESIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Indonesia from the Peak through the Trough (in +1 or 0) (`IDNRECDM`)
- `Fred.OECDRecessionIndicators.INDIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for India from the Peak through the Trough (in +1 or 0) (`INDRECDM`)
- `Fred.OECDRecessionIndicators.IRELAND_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Ireland from the Peak through the Trough (in +1 or 0) (`IRLRECDM`)
- `Fred.OECDRecessionIndicators.ISRAEL_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Israel from the Peak through the Trough (in +1 or 0) (`ISRRECDM`)
- `Fred.OECDRecessionIndicators.ITALY_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Italy from the Peak through the Trough (in +1 or 0) (`ITARECDM`)
- `Fred.OECDRecessionIndicators.JAPAN_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Japan from the Peak through the Trough (in +1 or 0) (`JPNRECDM`)
- `Fred.OECDRecessionIndicators.KOREA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Korea from the Peak through the Trough (in +1 or 0) (`KORRECDM`)

- `Fred.OECDRecessionIndicators.LUXEMBOURG_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Luxembourg from the Peak through the Trough (in +1 or 0) (`LUXRECDM`)
- `Fred.OECDRecessionIndicators.MAJOR_FIVE_ASIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Major 5 Asia from the Peak through the Trough (in +1 or 0) (`MAJOR5ASIARECDM`)
- `Fred.OECDRecessionIndicators.MEXICO_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Mexico from the Peak through the Trough (in +1 or 0) (`MEXRECDM`)
- `Fred.OECDRecessionIndicators.MAJOR_SEVEN_COUNTRIES_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Major Seven Countries from the Peak through the Trough (in +1 or 0) (`MSCRECDM`)
- `Fred.OECDRecessionIndicators.NAFTA_AREA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for NAFTA Area from the Peak through the Trough (in +1 or 0) (`NAFTARECDM`)
- `Fred.OECDRecessionIndicators.NETHERLANDS_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Netherlands from the Peak through the Trough (in +1 or 0) (`NDLRECDM`)
- `Fred.OECDRecessionIndicators.NORWAY_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Norway from the Peak through the Trough (in +1 or 0) (`NORRECDM`)
- `Fred.OECDRecessionIndicators.NEW_ZEALAND_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for New Zealand from the Peak through the Trough (in +1 or 0) (`NZLRECDM`)
- `Fred.OECDRecessionIndicators.OECD_EUROPE_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for OECD Europe from the Peak through the Trough (in +1 or 0) (`OECDEUROPERECDM`)
- `Fred.OECDRecessionIndicators.OECD_AND_NONMEMBER_ECONOMIES_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for OECD and Non-member Economies from the Peak through the Trough (in +1 or 0) (`OECDNMERECDM`)
- `Fred.OECDRecessionIndicators.OECD_TOTAL_AREA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the OECD Total Area from the Peak through the Trough (in +1 or 0) (`OECDRECDM`)
- `Fred.OECDRecessionIndicators.POLAND_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Poland from the Peak through the Trough (in +1 or 0) (`POLRECDM`)
- `Fred.OECDRecessionIndicators.PORTUGAL_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Portugal from the Peak through the Trough (in +1 or 0) (`PRTRECDM`)
- `Fred.OECDRecessionIndicators.RUSSIAN_FEDERATION_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Russian Federation from the Peak through the Trough (in +1 or 0) (`RUSRECDM`)
- `Fred.OECDRecessionIndicators.SLOVAK_REPUBLIC_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the Slovak Republic from the Peak through the Trough (in +1 or 0) (`SVKRECDM`)
- `Fred.OECDRecessionIndicators.SLOVENIA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Slovenia from the Peak through the Trough (in +1 or 0) (`SVNRECDM`)
- `Fred.OECDRecessionIndicators.SWEDEN_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Sweden from the Peak through the Trough (in +1 or 0) (`SWERECDM`)
- `Fred.OECDRecessionIndicators.TURKEY_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Turkey from the Peak through the Trough (in +1 or 0) (`TURRECDM`)
- `Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the United States from the Peak through the Trough (in +1 or 0) (`USARECDM`)
- `Fred.OECDRecessionIndicators.SOUTH_AFRICA_FROM_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for South Africa from the Peak through the Trough (in +1 or 0) (`ZAFRECDM`)
- `Fred.OECDRecessionIndicators.FOUR_BIG_EUROPEAN_COUNTRIES_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Four Big European Countries from the Period following the Peak through the Trough (in +1 or 0) (`4BIGEURORECD`)
- `Fred.OECDRecessionIndicators.AUSTRALIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Australia from the Period following the Peak through the Trough (in +1 or 0) (`AUSRECD`)
- `Fred.OECDRecessionIndicators.AUSTRIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Austria from the Period following the Peak through the Trough (in +1 or 0) (`AUTRECD`)
- `Fred.OECDRecessionIndicators.BELGIUM_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Belgium from the Period following the Peak through the Trough (in +1 or 0) (`BELRECD`)
- `Fred.OECDRecessionIndicators.BRAZIL_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Brazil from the Period following the Peak through the Trough (in +1 or 0) (`BRARECD`)
- `Fred.OECDRecessionIndicators.CANADA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Canada from the Period following the Peak through the Trough (in +1 or 0) (`CANRECD`)

- `Fred.OECDRecessionIndicators.SWITZERLAND_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Switzerland from the Period following the Peak through the Trough (in +1 or 0) (`CHERECD`)
- `Fred.OECDRecessionIndicators.CHILE_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Chile from the Period following the Peak through the Trough (in +1 or 0) (`CHLRECD`)
- `Fred.OECDRecessionIndicators.CHINA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for China from the Period following the Peak through the Trough (in +1 or 0) (`CHNRECD`)
- `Fred.OECDRecessionIndicators.CZECH_REPUBLIC_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the Czech Republic from the Period following the Peak through the Trough (in +1 or 0) (`CZERECD`)
- `Fred.OECDRecessionIndicators.GERMANY_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Germany from the Period following the Peak through the Trough (in +1 or 0) (`DEURECD`)
- `Fred.OECDRecessionIndicators.DENMARK_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Denmark from the Period following the Peak through the Trough (in +1 or 0) (`DNKRECD`)
- `Fred.OECDRecessionIndicators.SPAIN_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Spain from the Period following the Peak through the Trough (in +1 or 0) (`ESPRECD`)
- `Fred.OECDRecessionIndicators.ESTONIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Estonia from the Period following the Peak through the Trough (in +1 or 0) (`ESTRECD`)
- `Fred.OECDRecessionIndicators.EURO_AREA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Euro Area from the Period following the Peak through the Trough (in +1 or 0) (`EURORECD`)
- `Fred.OECDRecessionIndicators.FINLAND_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Finland from the Period following the Peak through the Trough (in +1 or 0) (`FINRECD`)
- `Fred.OECDRecessionIndicators.FRANCE_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for France from the Period following the Peak through the Trough (in +1 or 0) (`FRARECD`)
- `Fred.OECDRecessionIndicators.UNITED_KINGDOM_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for the United Kingdom from the Period following the Peak through the Trough (in +1 or 0) (`GBRRECD`)
- `Fred.OECDRecessionIndicators.GREECE_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Greece from the Period following the Peak through the Trough (in +1 or 0) (`GRCRECD`)
- `Fred.OECDRecessionIndicators.HUNGARY_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Hungary from the Period following the Peak through the Trough (in +1 or 0) (`HUNRECD`)
- `Fred.OECDRecessionIndicators.INDONESIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Indonesia from the Period following the Peak through the Trough (in +1 or 0) (`IDNRECD`)
- `Fred.OECDRecessionIndicators.INDIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for India from the Period following the Peak through the Trough (in +1 or 0) (`INDRECD`)
- `Fred.OECDRecessionIndicators.IRELAND_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Ireland from the Period following the Peak through the Trough (in +1 or 0) (`IRLRECD`)
- `Fred.OECDRecessionIndicators.ISRAEL_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Israel from the Period following the Peak through the Trough (in +1 or 0) (`ISRRECD`)
- `Fred.OECDRecessionIndicators.ITALY_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Italy from the Period following the Peak through the Trough (in +1 or 0) (`ITARECD`)
- `Fred.OECDRecessionIndicators.JAPAN_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Japan from the Period following the Peak through the Trough (in +1 or 0) (`JPNRECD`)
- `Fred.OECDRecessionIndicators.KOREA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Korea from the Period following the Peak through the Trough (in +1 or 0) (`KORRECD`)
- `Fred.OECDRecessionIndicators.LUXEMBOURG_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Luxembourg from the Period following the Peak through the Trough (in +1 or 0) (`LUXRECD`)
- `Fred.OECDRecessionIndicators.MAJOR_FIVE_ASIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Major 5 Asia from the Period following the Peak through the Trough (in +1 or 0) (`MAJOR5ASIARECD`)
- `Fred.OECDRecessionIndicators.MEXICO_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Mexico from the Period following the Peak through the Trough (in +1 or 0) (`MEXRECD`)
- `Fred.OECDRecessionIndicators.MAJOR_SEVEN_COUNTRIES_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for Major Seven Countries from the Period following the Peak through the Trough (in +1 or 0) (`MSCRECD`)
- `Fred.OECDRecessionIndicators.NAFTA_AREA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH` : OECD based Recession Indicators for NAFTA Area from the Period following the Peak through the Trough (in +1 or 0) (`NAFTARECD`)

- Fred.OECDRecessionIndicators.NETHERLANDS_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Netherlands from the Period following the Peak through the Trough (in +1 or 0) (NDLRECD)
- Fred.OECDRecessionIndicators.NORWAY_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Norway from the Period following the Peak through the Trough (in +1 or 0) (NORRECD)
- Fred.OECDRecessionIndicators.NEW_ZEALAND_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for New Zealand from the Period following the Peak through the Trough (in +1 or 0) (NZLRECD)
- Fred.OECDRecessionIndicators.OECD_EUROPE_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for OECD Europe from the Period following the Peak through the Trough (in +1 or 0) (OECDEUROPERECD)
- Fred.OECDRecessionIndicators.OEC_DAND_NONMEMBER_ECONOMIES_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for OECD and Non-member Economies from the Period following the Peak through the Trough (in +1 or 0) (OECDNMERECD)
- Fred.OECDRecessionIndicators.OECD_TOTAL_AREA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for the OECD Total Area from the Period following the Peak through the Trough (in +1 or 0) (OECDRECD)
- Fred.OECDRecessionIndicators.POLAND_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Poland from the Period following the Peak through the Trough (in +1 or 0) (POLRECD)
- Fred.OECDRecessionIndicators.PORTUGAL_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : Retrieved from FRED, Federal Reserve Bank of St. Louis; https://fred.stlouisfed.org/series/POLRECD (PRTRECD)
- Fred.OECDRecessionIndicators.RUSSIAN_FEDERATION_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Russian Federation from the Period following the Peak through the Trough (in +1 or 0) (RUSRECD)
- Fred.OECDRecessionIndicators.SLOVAK_REPUBLIC_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for the Slovak Republic from the Period following the Peak through the Trough (in +1 or 0) (SVKRECD)
- Fred.OECDRecessionIndicators.SLOVENIA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Slovenia from the Period following the Peak through the Trough (in +1 or 0) (SVNRECD)
- Fred.OECDRecessionIndicators.SWEDEN_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Sweden from the Period following the Peak through the Trough (in +1 or 0) (SWERECD)
- Fred.OECDRecessionIndicators.TURKEY_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for Turkey from the Period following the Peak through the Trough (in +1 or 0) (TURRECD)
- Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for the United States from the Period following the Peak through the Trough (in +1 or 0) (USARECD)
- Fred.OECDRecessionIndicators.SOUTH_AFRICA_FROM_PERIOD_FOLLOWING_PEAK_THROUGH_THE_TROUGH : OECD based Recession Indicators for South Africa from the Period following the Peak through the Trough (in +1 or 0) (ZAFRECD)
- Fred.OECDRecessionIndicators.FOUR_BIG_EUROPEAN_COUNTRIES_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Four Big European Countries from the Peak through the Period preceding the Trough (in +1 or 0) (4BIGEURORECDP)
- Fred.OECDRecessionIndicators.AUSTRALIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Australia from the Peak through the Period preceding the Trough (in +1 or 0) (AUSRECDP)
- Fred.OECDRecessionIndicators.AUSTRIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Austria from the Peak through the Period preceding the Trough (in +1 or 0) (AUTRECDP)
- Fred.OECDRecessionIndicators.BELGIUM_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Belgium from the Peak through the Period preceding the Trough (in +1 or 0) (BELRECDP)
- Fred.OECDRecessionIndicators.BRAZIL_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Brazil from the Peak through the Period preceding the Trough (in +1 or 0) (BRARECDP)
- Fred.OECDRecessionIndicators.CANADA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Canada from the Peak through the Period preceding the Trough (in +1 or 0) (CANRECDP)
- Fred.OECDRecessionIndicators.SWITZERLAND_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Switzerland from the Peak through the Period preceding the Trough (in +1 or 0) (CHERECDP)
- Fred.OECDRecessionIndicators.CHILE_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Chile from the Peak through the Period preceding the Trough (in +1 or 0) (CHLRECDP)
- Fred.OECDRecessionIndicators.CHINA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for China from the Peak through the Period preceding the Trough (in +1 or 0) (CHNRECDP)
- Fred.OECDRecessionIndicators.CZECH_REPUBLIC_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for the Czech Republic from the Peak through the Period preceding the Trough (in +1 or 0) (CZERECDP)
- Fred.OECDRecessionIndicators.GERMANY_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH : OECD based Recession Indicators for Germany from the Peak through the Period preceding the Trough (in +1 or 0) (DEURECDP)

- `Fred.OECDRecessionIndicators.DENMARK_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Denmark from the Peak through the Period preceding the Trough (in +1 or 0) (`DNKRECDP`)
- `Fred.OECDRecessionIndicators.SPAIN_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Spain from the Peak through the Period preceding the Trough (in +1 or 0) (`ESPRECDP`)
- `Fred.OECDRecessionIndicators.ESTONIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Estonia from the Peak through the Period preceding the Trough (in +1 or 0) (`ESTRECDP`)
- `Fred.OECDRecessionIndicators.EURO_AREA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Euro Area from the Peak through the Period preceding the Trough (in +1 or 0) (`EURORECDP`)
- `Fred.OECDRecessionIndicators.FINLAND_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Finland from the Peak through the Period preceding the Trough (in +1 or 0) (`FINRECDP`)
- `Fred.OECDRecessionIndicators.FRANCE_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for France from the Peak through the Period preceding the Trough (in +1 or 0) (`FRARECDP`)
- `Fred.OECDRecessionIndicators.UNITED_KINGDOM_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for the United Kingdom from the Peak through the Period preceding the Trough (in +1 or 0) (`GBRRECDP`)
- `Fred.OECDRecessionIndicators.GREECE_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Greece from the Peak through the Period preceding the Trough (in +1 or 0) (`GRCRECDP`)
- `Fred.OECDRecessionIndicators.HUNGARY_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Hungary from the Peak through the Period preceding the Trough (in +1 or 0) (`HUNRECDP`)
- `Fred.OECDRecessionIndicators.INDONESIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Indonesia from the Peak through the Period preceding the Trough (in +1 or 0) (`IDNRECDP`)
- `Fred.OECDRecessionIndicators.INDIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for India from the Peak through the Period preceding the Trough (in +1 or 0) (`INDRECDP`)
- `Fred.OECDRecessionIndicators.IRELAND_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Ireland from the Peak through the Period preceding the Trough (in +1 or 0) (`IRLRECDP`)
- `Fred.OECDRecessionIndicators.ISRAEL_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Israel from the Peak through the Period preceding the Trough (in +1 or 0) (`ISRRECDP`)
- `Fred.OECDRecessionIndicators.ITALY_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Italy from the Peak through the Period preceding the Trough (in +1 or 0) (`ITARECDP`)
- `Fred.OECDRecessionIndicators.JAPAN_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Japan from the Peak through the Period preceding the Trough (in +1 or 0) (`JPNRECDP`)
- `Fred.OECDRecessionIndicators.KOREA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Korea from the Peak through the Period preceding the Trough (in +1 or 0) (`KORRECDP`)
- `Fred.OECDRecessionIndicators.LUXEMBOURG_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Luxembourg from the Peak through the Period preceding the Trough (in +1 or 0) (`LUXRECDP`)
- `Fred.OECDRecessionIndicators.MAJOR_FIVE_ASIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Major 5 Asia from the Peak through the Period preceding the Trough (in +1 or 0) (`MAJOR5ASIARECDP`)
- `Fred.OECDRecessionIndicators.MEXICO_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Mexico from the Peak through the Period preceding the Trough (in +1 or 0) (`MEXRECDP`)
- `Fred.OECDRecessionIndicators.MAJOR_SEVEN_COUNTRIES_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Major Seven Countries from the Peak through the Period preceding the Trough (in +1 or 0) (`MSCRECDP`)
- `Fred.OECDRecessionIndicators.NAFTA_AREA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for NAFTA Area from the Peak through the Period preceding the Trough (in +1 or 0) (`NAFTARECDP`)
- `Fred.OECDRecessionIndicators.NETHERLANDS_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Netherlands from the Peak through the Period preceding the Trough (in +1 or 0) (`NDLRECDP`)
- `Fred.OECDRecessionIndicators.NORWAY_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Norway from the Peak through the Period preceding the Trough (in +1 or 0) (`NORRECDP`)
- `Fred.OECDRecessionIndicators.NEW_ZEALAND_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : Retrieved from FRED, Federal Reserve Bank of St. Louis; https://fred.stlouisfed.org/series/NORRECDP (`NZLRECDP`)
- `Fred.OECDRecessionIndicators.OECD_EUROPE_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for OECD Europe from the Peak through the Period preceding the Trough (in +1 or 0) (`OECDEUROPERECDP`)
- `Fred.OECDRecessionIndicators.OEC_DAND_NONMEMBER_ECONOMIES_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for OECD and Non-member Economies from the Peak through the Period preceding the Trough (in +1 or 0) (`OECDNMERECDP`

)

- `Fred.OECDRecessionIndicators.OECD_TOTAL_AREA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for the OECD Total Area from the Peak through the Period preceding the Trough (in +1 or 0) (`OECDRECDP`)
- `Fred.OECDRecessionIndicators.POLAND_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Poland from the Peak through the Period preceding the Trough (in +1 or 0) (`POLRECDP`)
- `Fred.OECDRecessionIndicators.PORTUGAL_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Portugal from the Peak through the Period preceding the Trough (in +1 or 0) (`PRTRECDP`)
- `Fred.OECDRecessionIndicators.RUSSIAN_FEDERATION_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Russian Federation from the Peak through the Period preceding the Trough (in +1 or 0) (`RUSRECDP`)
- `Fred.OECDRecessionIndicators.SLOVAK_REPUBLIC_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for the Slovak Republic from the Peak through the Period preceding the Trough (in +1 or 0) (`SVKRECDP`)
- `Fred.OECDRecessionIndicators.SLOVENIA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Slovenia from the Peak through the Period preceding the Trough (in +1 or 0) (`SVNRECDP`)
- `Fred.OECDRecessionIndicators.SWEDEN_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Sweden from the Peak through the Period preceding the Trough (in +1 or 0) (`SWERECDP`)
- `Fred.OECDRecessionIndicators.TURKEY_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for Turkey from the Peak through the Period preceding the Trough (in +1 or 0) (`TURRECDP`)
- `Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for the United States from the Peak through the Period preceding the Trough (in +1 or 0) (`USARECDP`)
- `Fred.OECDRecessionIndicators.SOUTH_AFRICA_FROM_PEAK_THROUGH_THE_PERIOD_PRECEDINGTHE_TROUGH` : OECD based Recession Indicators for South Africa from the Peak through the Period preceding the Trough (in +1 or 0) (`ZAFRECDP`)

Trade Weighted Indexes

- `Fred.TradeWeightedIndexes.MAJOR_CURRENCIES_GOODS` : Trade Weighted U.S. Dollar Index: Major Currencies, Goods (in Index Mar 1973=100) (`DTWEXM`)
- `Fred.TradeWeightedIndexes.OTHER_IMPORTANT_TRADING_PARTNERS_GOODS` : Trade Weighted U.S. Dollar Index: Other Important Trading Partners, Goods (in Index Jan 1997=100) (`DTWEXO`)
- `Fred.TradeWeightedIndexes.BROAD_GOODS` : Trade Weighted U.S. Dollar Index: Broad, Goods (in Index Jan 1997=100) (`DTWEXB`)
- `Fred.TradeWeightedIndexes.ADVANCED_FOREIGN_ECONOMIES_GOODS_AND_SERVICES` : Trade Weighted U.S. Dollar Index: Advanced Foreign Economies, Goods and Services (in Index Jan 2006=100) (`DTWEXAFEGS`)
- `Fred.TradeWeightedIndexes.BROAD_GOODS_AND_SERVICES` : Trade Weighted U.S. Dollar Index: Broad, Goods and Services (in Index Jan 2006=100) (`DTWEXBGS`)
- `Fred.TradeWeightedIndexes.EMERGING_MARKETS_ECONOMIES_GOODS_AND_SERVICES` : Trade Weighted U.S. Dollar Index: Emerging Markets Economies, Goods and Services (in Index Jan 2006=100) (`DTWEXEMEGS`)

Example Applications

The FRED dataset enables you to accurately design strategies utilizing macroeconomic indicators. Examples include the following strategies:

- Trading on macroeconomic factors
- Macroeconomic risk modeling

Classic Algorithm Example

The following example algorithm buys SPY when the FRED Peak to Trough value is 1. Otherwise, it holds cash.

```py
from AlgorithmImports import *


class FredAlternativeDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol

        # Requesting FED US peak-to-trough OECD recession indicators for trade signal generation
        self.fred_peak_to_trough = self.add_data(Fred, Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_TROUGH,
Resolution.DAILY).symbol

        # Historical data
        history = self.history(self.fred_peak_to_trough, 60, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def on_data(self, slice: Slice) -> None:
        # Trade with updated FED peak-to-trough indicator
        if slice.contains_key(self.fred_peak_to_trough) and slice.contains_key(self.spy):
            peak_to_trough = slice.Get(Fred, self.fred_peak_to_trough).value

            # Buy SPY if peak to trough value is 0, which is the expansionary period
            if peak_to_trough == 0 and not self.portfolio.invested:
                self.set_holdings(self.spy, 1)

            # Liquidate holdings if peak to trough value is 1, which is recessionary period
            elif peak_to_trough == 1 and self.portfolio.invested:
                self.liquidate(self.spy)
```

Framework Algorithm Example

The following example algorithm buys SPY when the FRED Peak to Trough value is 1. Otherwise, it holds cash.

```
from AlgorithmImports import *


class FredAlternativeDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Universe contains only SPY as the market representative
        symbols = [ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Invest equally to evenly dissipate the capital concentration risk
        self.add_alpha(FredAlphaModel(self))
        # Custom alpha model that use FED data as trade signals
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class FredAlphaModel(AlphaModel):

    symbols = []
    peak_to_trough_value = None

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Requesting FED US peak-to-trough OECD recession indicators for trade signal generation
        self.fred_peak_to_trough = algorithm.add_data(Fred, Fred.OECDRecessionIndicators.UNITED_STATES_FROM_PEAK_THROUGH_THE_TROUGH,
Resolution.DAILY).symbol

        # Historical data
        history = algorithm.history(self.fred_peak_to_trough, 60, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request")
        if history.empty:
            return
        self.peak_to_trough_value = history.loc[self.fred_peak_to_trough].value.iloc[-1]


    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade with updated FED peak-to-trough indicator
        if slice.contains_key(self.fred_peak_to_trough):
            self.peak_to_trough_value = slice.Get(Fred, self.fred_peak_to_trough).value

        # Ensure we have a FRED peak to trough value
        if self.peak_to_trough_value is None:
            return []

        for symbol in self.symbols:
            # Ensure we have data for the security in the current Slice to avoid stale filling
            if not (slice.contains_key(symbol) and slice[symbol] is not None):
                continue

            # Buy SPY if peak to trough value is 0, which is the expansionary period
            if self.peak_to_trough_value == 0 and not algorithm.portfolio[symbol].invested:
                insights += [Insight.price(symbol, timedelta(days=365*2), InsightDirection.UP)]

            # Liquidate holdings if peak to trough value is 1, which is recessionary period
            elif self.peak_to_trough_value == 1 and algorithm.portfolio.invested:
                insights += [Insight.price(symbol, timedelta(days=365*2), InsightDirection.FLAT)]

        return insights


    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.symbols.append(security.symbol)

        for security in changes.removed_securities:
            if symbol in self.symbols:
                self.symbols.remove(security.symbol)
```

Data Point Attributes

The FRED dataset provides `Fred` objects, which have the following attributes:

Datasets

Federal Reserve Bank of St Louis

The Federal Reserve Bank of St. Louis, often referred to as the St. Louis Fed, is one of the 12 regional banks that make up the United States Federal Reserve System. It is responsible for the Eighth Federal Reserve District, which includes the states of Arkansas, Illinois, Indiana, Kentucky, Mississippi, Missouri, and Tennessee, as well as portions of eastern Kansas and southern Illinois.

The St. Louis Fed, like other regional banks, participates in the formulation and implementation of monetary policy in the United States. It contributes to the Federal Open Market Committee (FOMC) meetings, where key decisions regarding interest rates and other monetary policy tools are made.

US Interest Rate

Federal Reserve Bank of St Louis

US Interest Rate

Introduction

The US Interest Rate dataset provides the primary credit rate from the Federal Open Market Committee (FOMC). The data starts in January 2003 and is updated on a daily frequency. This dataset is created using information from the FOMC meetings.

For more information about the US Interest Rate dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The Federal Reserve Bank of St. Louis, often referred to as the St. Louis Fed, is one of the 12 regional banks that make up the United States Federal Reserve System. It is responsible for the Eighth Federal Reserve District, which includes the states of Arkansas, Illinois, Indiana, Kentucky, Mississippi, Missouri, and Tennessee, as well as portions of eastern Kansas and southern Illinois.

The St. Louis Fed, like other regional banks, participates in the formulation and implementation of monetary policy in the United States. It contributes to the Federal Open Market Committee (FOMC) meetings, where key decisions regarding interest rates and other monetary policy tools are made.

Getting Started

The following snippet demonstrates how to access data from the US Interest Rate dataset:

```
interest_rate = self.risk_free_interest_rate_model.get_interest_rate(self.time)
```

To find the average interest rate between two dates, call the `GetRiskFreeRate` method.

```
avg_risk_free_rate = RiskFreeInterestRateModelExtensions.get_risk_free_rate(
    self.risk_free_interest_rate_model,
    self.time-timedelta(365), self.time
)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2003
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

You don't need any special code to request US Interest Rate data. `QCAlgorithm` automatically subscribes to the data by setting its default risk free interest rate model .

Accessing Data

To get the current US Interest Rate data, call the `get_interest_rate` method of the `RiskFreeInterestRateModel` object with the current time.

```
interest_rate = self.risk_free_interest_rate_model.get_interest_rate(self.time)
```

Historical Data

To get the average risk free interest rate for a window of time, call the `get_risk_free_rate` method with the start date and end date.

```
PY
risk_free_rate = RiskFreeInterestRateModelExtensions.get_risk_free_rate(
    self.risk_free_interest_rate_model,
    self.time-timedelta(365), self.time
)
```

To get the average risk free interest rate for a set of dates, call the `get_average_risk_free_rate` method with the list of dates.

```
PY
risk_free_rate = RiskFreeInterestRateModelExtensions.get_average_risk_free_rate(
    self.risk_free_interest_rate_model,
    [self.time, self.time-timedelta(180), self.time-timedelta(365)]
)
```

Example Applications

The US Interest Rate dataset provides an important economic indicator. Examples include the following applications:

- Accurately calculating indicators that are a function of the risk-free rate, like Sharpe ratios.
- Forming a portfolio of assets that have a history of outperforming when the rate is increasing/decreasing.
- Canceling orders in a Risk Management model when the expected return of the asset is less than the risk-free rate.

Classic Algorithm Example

The following example algorithm plots the current interest rate and the last year's average.

```
PY
from AlgorithmImports import *


class RiskFreeInterestRateModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_equity("SPY")

    def on_end_of_day(self, symbol):
        self.set_holdings(symbol, 1)
        # Get the average risk free rate of the last year at the current time
        risk_free_rate = RiskFreeInterestRateModelExtensions.get_risk_free_rate(self.risk_free_interest_rate_model, self.time -
timedelta(365), self.time)
        # Plot the current interest rate and the 1-year average rate for comparison
        self.plot('Interest', 'EOD', self.risk_free_interest_rate_model.get_interest_rate(self.time))
        self.plot('Interest', '1Y-RW', risk_free_rate)
```

Datasets

Kavout

Kavout was created by ex-Googlers and the founding team used to work at Google, Microsoft, Baidu, and financial firms with a proven track record of building many mission-critical machine learning systems where billions of data points were processed in real-time to predict the best outcome for core search ranking, ads monetization, recommendations, and trading platforms.

Their mission is to build machine investing solutions to find alpha with adaptive learning algorithms and to create an edge by assimilating vast quantities of complex data through the latest AI and Machine Learning methods to generate signals to uncover hidden, dynamic, and nonlinear patterns in the financial markets.

Composite Factor Bundle

Kavout

Composite Factor Bundle

Introduction

The Composite Factor Bundle dataset by Kavout provides ensemble scores for popular market factors. Kavout signals are machine-learning enhanced scores that capture the returns of systematic factors such as quality, value, momentum, growth, and low volatility. There are many different anomalies discovered by researchers and practitioners across these factor categories and there is no good common definition of each style across the literature. Kavout creates an ensemble score for each style that gauges the different factors considered in the literature and industry practice.

In this data set, you will find Kavout's proprietary signals for quality, value, momentum, growth, and low volatility, which have been adopted by some of the multi-billion dollar quant funds in New York and London. Each signal is generated by an ensemble model consisting of inputs from hundreds of anomalies. The data is generated on a daily basis and covers all the stocks traded in US major markets such as NYSE and Nasdaq since 2003. You could leverage this abundant set of signals to construct and backtest your strategies.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Composite Factor Bundle dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Kavout was created by ex-Googlers and the founding team used to work at Google, Microsoft, Baidu, and financial firms with a proven track record of building many mission-critical machine learning systems where billions of data points were processed in real-time to predict the best outcome for core search ranking, ads monetization, recommendations, and trading platforms.

Their mission is to build machine investing solutions to find alpha with adaptive learning algorithms and to create an edge by assimilating vast quantities of complex data through the latest AI and Machine Learning methods to generate signals to uncover hidden, dynamic, and nonlinear patterns in the financial markets.

Getting Started

The following snippet demonstrates how to request data from the Composite Factor Bundle dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(KavoutCompositeFactorBundle, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2003
Asset Coverage	8,000 US Equities
Data Density	Regular
Resolution	Daily
Timezone	UTC

Requesting Data

To add Composite Factor Bundle data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class KavoutCompositeFactorBundleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(KavoutCompositeFactorBundle, self.aapl).symbol
```

```

```

Accessing Data

To get the current Composite Factor Bundle data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} momentum at {slice.time}: {data_point.momentum}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(KavoutCompositeFactorBundle).items():
        self.log(f"{dataset_symbol} momentum at {slice.time}: {data_point.momentum}")
```

Historical Data

To get historical Composite Factor Bundle data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[KavoutCompositeFactorBundle](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

If you subscribe to Composite Factor Bundle data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Composite Factor Bundle dataset enables you to access the performance of 5 different factors in order to engineer strategies. Examples include the following strategies:

- Performing return-risk optimization based on performance and volatility scoring.
- Weighing stocks based on regression analysis in factor-vector space.

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms a equal-weighted dollar-neutral portfolio of the 10 companies most likely to outperform and the 10 companies most likely to underperform.

PY

```
from AlgorithmImports import *

class KavoutCompositeFactorBundleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_universe(self._select_assets)

    def _select_assets(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Filter for the highly traded stocks for more informed data from frequent market activities, which may translate to more
accurate prediction
        # Factors scores are only available for the ones with fundamentals
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]

    def on_data(self, slice: Slice) -> None:
        # Get the current data from the Kavout dataset.
        points = slice.get(KavoutCompositeFactorBundle)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, factors in points.items():
        #    self.quit(
        #        f"{self.time} -- "
        #        f"Asset Symbol: {dataset_symbol.underlying}; "
        #        f"Growth factor: {factors.growth} "
        #    )

        # Drop factors for assets that have no price.
        points = [x for x in points.items() if self.securities[x[0].underlying].price]
        # Only rebalance when there are new Kavout factors.
        if not points:
            return

        # Long the stocks with highest factor scores, which indicate higher return from various factors
        # Short the ones with lowest factor scores for lower return estimates
        sorted_by_score = sorted(points, key=self._total_score)
        long_symbols = [x[0].underlying for x in sorted_by_score[-10:]]
        short_symbols = [x[0].underlying for x in sorted_by_score[:10]]

        # Invest in equal-size and dollar-neutral to evenly dissipate individual capital risk, avoid non-systematic risk, and better
margin
        long_targets = [PortfolioTarget(symbol, 0.05) for symbol in long_symbols]
        short_targets = [PortfolioTarget(symbol, -0.05) for symbol in short_symbols]
        self.set_holdings(long_targets + short_targets, True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting factor bundle data for trade signal generation
            security.kavout_symbol = self.add_data(KavoutCompositeFactorBundle, security.symbol).symbol
            # Historical Data
            history = self.history(security.kavout_symbol, 2, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the factor bundle data for this asset when it leaves the universe.
            self.remove_security(security.kavout_symbol)

    def _total_score(self, value) -> float:
        # Return the total score to integrate overall likelihood to outcompete, take equal weighting for each factor
        value = value[1]
        return value.growth + value.low_volatility + value.momentum + value.quality + value.value_factor
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 100 most liquid US Equities. Each day, it then forms a equal-weighted dollar-neutral portfolio of the 10 companies most likely to outperform and the 10 companies most likely to underperform.

```py
from AlgorithmImports import *

class KavoutCompositeFactorBundleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_universe_selection(LiquidEquitiesUniverseSelectionModel())
        # Custom alpha model to emit insights based on factor bundle data
        self.add_alpha(KavoutCompositeFactorBundleAlphaModel())
        # Invest in equal-size and dollar-neutral to evenly dissipate individual capital risk, avoid non-systematic risk, and better
margin
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())


class LiquidEquitiesUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Filter for the highly traded stocks for more informed data from frequent market activities, which may translate to more
accurate prediction
        # Factors scores are only available for the ones with fundamentals
        sorted_by_dollar_volume = sorted(
            [x for x in fundamentals if x.has_fundamental_data],
            key=lambda x: x.dollar_volume
        )
        return [x.symbol for x in sorted_by_dollar_volume[-100:]]


class KavoutCompositeFactorBundleAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Get the current data from the Kavout dataset.
        points = slice.get(KavoutCompositeFactorBundle)
        ## Demonstrate how to iterate through the data and access its members:
        #for dataset_symbol, factors in points.items():
        #    algorithm.quit(
        #        f"{algorithm.time} -- "
        #        f"Asset Symbol: {dataset_symbol.underlying}; "
        #        f"Growth factor: {factors.growth} "
        #    )

        # Drop factors for assets that have no price.
        points = [x for x in points.items() if algorithm.securities[x[0].underlying].price]
        # Only rebalance when there are new Kavout factors.
        if not points:
            return []

        # Long the stocks with highest factor scores, which indicate higher return from various factors
        # Short the ones with lowest factor scores for lower return estimates
        sorted_by_score = sorted(points, key=self._total_score)
        long_symbols = [x[0].underlying for x in sorted_by_score[-10:]]
        short_symbols = [x[0].underlying for x in sorted_by_score[:10]]

        insights = []
        for symbol in long_symbols:
            insights.append(Insight.price(symbol, Expiry.END_OF_DAY, InsightDirection.UP))
        for symbol in short_symbols:
            insights.append(Insight.price(symbol, Expiry.END_OF_DAY, InsightDirection.DOWN))
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting factor bundle data for trade signal generation
            security.kavout_symbol = algorithm.add_data(KavoutCompositeFactorBundle, security.symbol).symbol
            # Historical Data
            history = algorithm.history(security.kavout_symbol, 10, Resolution.DAILY)
        for security in changes.removed_securities:
            # Remove the factor bundle data for this asset when it leaves the universe.
            algorithm.remove_security(security.kavout_symbol)

    def _total_score(self, value) -> float:
        # Return the total score to integrate overall likelihood to outcompete, take equal weighting for each factor
        value = value[1]
        return value.growth + value.low_volatility + value.momentum + value.quality + value.value_factor
```

Data Point Attributes

The Composite Factor Bundle dataset provides `KavoutCompositeFactorBundle` objects, which have the following attributes:

Datasets

Nasdaq

" Nasdaq " was initially an acronym for the National Association of Securities Dealers Automated Quotations. It was founded in 1971 by the National Association of Securities Dealers (NASD), now known as the Financial Industry Regulatory Authority (FINRA), with the goal to provide financial services and operate stock exchanges.

On Dec. 4th, 2018, Nasdaq announced it had acquired Quandl, Inc. , a leading provider of alternative and core financial data. Quandl was founded by Tammer Kamel in 2012, with goal of making it easy for anyone to find and use high-quality data effectively in their professional decision-making. In 2021, Quandl was replaced by Nasdaq Data Link .

Data Link

Nasdaq

Data Link

Introduction

The Data Link dataset by Nasdaq, previously known as Quandl, is a collection of alternative data sets. It has indexed millions of time-series datasets from over 400 sources, which start in different years. This dataset is delivered on several frequencies, but the free data sets have often a daily frequency.

For more information about the Data Link dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

" Nasdaq " was initially an acronym for the National Association of Securities Dealers Automated Quotations. It was founded in 1971 by the National Association of Securities Dealers (NASD), now known as the Financial Industry Regulatory Authority (FINRA), with the goal to provide financial services and operate stock exchanges.

On Dec. 4th, 2018, Nasdaq announced it had acquired Quandl, Inc. , a leading provider of alternative and core financial data. Quandl was founded by Tammer Kamel in 2012, with goal of making it easy for anyone to find and use high-quality data effectively in their professional decision-making. In 2021, Quandl was replaced by Nasdaq Data Link .

Getting Started

The following snippet demonstrates how to request data from the Data Link dataset:

```py
# For premium datasets, provide your API Key
# NasdaqDataLink.set_auth_code(self.get_parameter("nasdaq-data-link-api-key"))

self.bitcoin_chain_symbol = self.add_data(NasdaqDataLink, "QDL/BCHAIN", Resolution.DAILY).symbol
# This dataset has one data column ("Value")

self.bitfinex_exchange_rate_symbol = self.add_data(NasdaqCustomColumns, "QDL/BITFINEX", Resolution.DAILY).symbol
# This dataset has multiple data columns. Create a subclass to set the default value column.

class NasdaqCustomColumns(NasdaqDataLink):
    def __init__(self) -> None:
        # Select the column "mid".
        self.value_column_name = "mid"
```

Our Nasdaq Data Link integration supports any dataset from Nasdaq that has a URL starting with `data.nasdaq.com/data/` . To import a dataset from Data Link, you need to get the dataset code. Follow these steps to get the dataset code:

1. Open the Data Link catalog .
2. *(Optional)* Use the filters on the left side bar to narrow down the catalog results.
3. Click one of the data products.
4. On the data product page, click one of the table names.
5. In the top-right corner of the dataset page, copy the Nasdaq Data Link Code.

Data Summary

The data summary depends on the specific Data Link dataset you use. Follow these steps to view the data summary of a dataset:

1. Open the Data Link catalog .
2. Click one of the data products.
3. On the data product page, click one of the table names.
4. View the data frequency and description in the left sidebar.
5. Above the chart, click `Table` to view the start date, end date, and data point attributes.

Backward Compatibility

QuantConnect/LEAN has supported Quandl since 2015. On January, 12, 2022, we moved the implementation from the LEAN GitHub repository to the

Lean.DataSource.NasdaqDataLink GitHub repository . Through this transition, we maintained backward compatibility. All of the preceding code snippets work if you replace `NasdaqDataLink` with `Quandl` .

Requesting Data

To add Data Link data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm. If there is more than one value column in the Data Link dataset, to set the `Value` property of the data objects, create a sublcass of the `NasdaqDataLink` class and set its `value_column_name` property to the column name.

```
PY
class NasdaqDataLinkDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2022, 1, 1)
        self.set_end_date(2022, 7, 1)
        self.set_cash(100000)

        # For premium datasets, provide your API Key
        # NasdaqDataLink.set_auth_code(self.get_parameter("nasdaq-data-link-api-key"))

        self.bitcoin_chain_symbol = self.add_data(NasdaqDataLink, "QDL/BCHAIN", Resolution.DAILY).symbol
        # This dataset has one data column ("Value")
        # Source : https://data.nasdaq.com/databases/BCHAIN

        self.bitfinex_exchange_rate_symbol = self.add_data(NasdaqCustomColumns, "QDL/BITFINEX", Resolution.DAILY).symbol
        # This dataset has multiple data columns
        # Source: https://data.nasdaq.com/databases/BITFINEX

class NasdaqCustomColumns(NasdaqDataLink):
    def __init__(self) -> None:
        # Select the column "mid".
        self.value_column_name = "mid"
```

Accessing Data

To get the current Data Link data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

To get the default value of the dataset, use the `value` property. To get the value of a specific column in the dataset, call the `get_storage_dictionary` method and index the result with the column name.

```
PY
def on_data(self, slice: Slice) -> None:
    point = slice.get(self.bitcoin_chain_symbol)
    if point:
        value = point.value
        self.log(f"{self.bitcoin_chain_symbol} at {slice.time}: {value}")

    point = slice.get(self.bitfinex_exchange_rate_symbol)
    if point:
        value = point.value
        storage_dictionary = point.get_storage_dictionary()
        volume = storage_dictionary["volume"]
        self.log(f"{self.bitfinex_exchange_rate_symbol} Bitfinex exchange rate at {slice.time}: {value}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(NasdaqDataLink).items():
        self.log(f"{dataset_symbol} value at {slice.time}: {data_point.value}")

    for dataset_symbol, data_point in slice.get(NasdaqCustomColumns).items():
        self.log(f"{dataset_symbol} value at {slice.time}: {data_point.value}")
```

Historical Data

The process to get historical Data Link data depends on your environment.

In Algorithms

To get historical Data Link data in an algorithm, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrames
bitcoin_chain_df = self.history(self.bitcoin_chain_symbol, 100, Resolution.DAILY)
bitfinex_exchange_rate_df = self.history(self.bitfinex_exchange_rate_symbol, 100, Resolution.DAILY)

# Dataset objects
bitcoin_chain_history = self.history[NasdaqDataLink](self.bitcoin_chain_symbol, 100, Resolution.DAILY)
bitfinex_exchange_rate_history = self.history[NasdaqCustomColumns](self.bitfinex_exchange_rate_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

In Research Notebooks

To get historical Data Link data in the Research Environment, call the `download` method with the data URL.

```
from io import StringIO
data = qb.download("https://data.nasdaq.com/api/v3/datatables/QDL/BCHAIN.csv?")
df = pd.read_csv(StringIO(data), index_col=0)
```

To receive a notification when you can use the `history` method in the Research Environment to get data from Data Link, subscribe to Lean.DataSource.NasdaqDataLink GitHub Issue #10 .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.bitcoin_chain_symbol)
self.remove_security(self.bitfinex_exchange_rate_symbol)
```

Example Applications

The Nasdaq Data Link sources allow you to explore different kinds of data in their database to develop trading strategies. Examples include the following strategies:

- Using alternative data to regress market regime/asset price.
- Backtesting exotic derivatives or private Equity investments.

Classic Algorithm Example

The following example algorithm subscribes to the Bitfinex Crypto Exchange Rate datasets. The algorithm takes a long position in Bitfinex BTCUSD.

```
from AlgorithmImports import *


class NasdaqImporterAlgorithm(QCAlgorithm):

    def initialize(self):
        self.nasdaq_code = "QDL/BITFINEX"
        ## Optional argument - personal token necessary for restricted dataset
        # NasdaqDataLink.set_auth_code(self.get_parameter("nasdaq-data-link-api-key"))
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(25000)
        # Request nasdaq data link data to trade
        self.add_data(NasdaqCustomColumns, self.nasdaq_code, Resolution.DAILY, TimeZones.NEW_YORK)
        # Create SMA indicator that feed with nasdaq data link data, which can be used to determine trend for trend following trade
        self.sma = self.SMA(self.nasdaq_code, 14)

    def on_data(self, data):
        if not self.portfolio.hold_stock:
            self.set_holdings(self.nasdaq_code, 1)
            self.debug("Purchased {0} >> {1}".format(self.nasdaq_code, self.time))

        self.plot(self.nasdaq_code, "PriceSMA", self.sma.current.value)

# NasdaqDataLink often doesn't use close columns so need to tell LEAN which is the "value" column.
class NasdaqCustomColumns(NasdaqDataLink):
    def __init__(self):
        # Define ValueColumnName: cannot be None, Empty or non-existant column name
        super().__init__("mid")
        self.value_column_name = "mid"
```

Framework Algorithm Example

The following example algorithm subscribes to the Bitcoin Chain dataset and Bitfinex Crypto Exchange Rate dataset. The algorithm takes a long position in Bitfinex BTCUSD.

PY

```
from AlgorithmImports import *


class NasdaqDataLinkFrameworkAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        ### Requesting Data
        # For premium datasets, provide your API Key
        # NasdaqDataLink.set_auth_code(self.get_parameter("nasdaq-data-link-api-key"))

        # Add nasdaq data link data as trading vehicle
        bitfinex_exchange_rate_symbol = self.add_data(NasdaqDataLink, 'QDL/BCHAIN', Resolution.DAILY).symbol

        ### Historical Data
        history = self.history(bitfinex_exchange_rate_symbol, 1000, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for {bitfinex_exchange_rate_symbol} Nasdaq Data Link")

        # Custom alpha model that emit insights based on updated nasdaq data link data
        self.add_alpha(NasdaqDataLinkAlphaModel(self))

        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(lambda time: None))


class NasdaqDataLinkAlphaModel(AlphaModel):
    def __init__(self, algorithm: QCAlgorithm) -> None:
        ### Requesting NASDAQ data link custom data
        self.bitfinex_exchange_rate_symbol = algorithm.add_data(NasdaqCustomColumns, "QDL/BITFINEX", Resolution.DAILY).symbol
        # This dataset has multiple data columns, so create a custom class to set the value column name

        ### Historical Data
        history = algorithm.history(self.bitfinex_exchange_rate_symbol, 100, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request for {self.bitfinex_exchange_rate_symbol} Nasdaq Data
Link")

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade only based on updated NASDAQ data link data
        for dataset_symbol, data_point in slice.Get(NasdaqCustomColumns).items():
            if dataset_symbol != self.bitfinex_exchange_rate_symbol or algorithm.portfolio[dataset_symbol].invested:
                continue
            insights.append(Insight.price(dataset_symbol, timedelta(days=6*22), InsightDirection.UP))

        return insights

# NasdaqDataLink often doesn't use close columns so need to tell LEAN which is the "value" column.
class NasdaqCustomColumns(NasdaqDataLink):
    def __init__(self) -> None:
        # Select the column "mid".
        self.value_column_name = "mid"
```

Datasets

Quiver Quantitative

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

CNBC Trading

Corporate Lobbying

Insider Trading

US Congress Trading

US Government Contracts

WallStreetBets

Quiver Quantitative

CNBC Trading

Introduction

The CNBC Trading dataset by Quiver Quantitative tracks the recommendations made by media personalities on CNBC and their historical performance. The data covers over 1,500 US Equities, starts in December 2020, and is delivered on a daily frequency. This dataset covers recommendations made on Mad Money, Halftime Report, and Fast Money.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the CNBC Trading dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the CNBC Trading dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverCNBCs, self.aapl).symbol

self._universe = self.add_universe(QuiverCNBCsUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	December 25, 2020
Asset Coverage	1,515 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add CNBC Trading data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class QuiverCNBCDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverCNBCs, self.aapl).symbol
```

Accessing Data

To get the current CNBC Trading data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY

def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} direction at {slice.time}: {data_point.direction}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY

def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_points in slice.get(QuiverCNBCs).items():
        for data_point in data_points:
            self.log(f"{dataset_symbol} direction at {slice.time}: {data_point.direction}")
```

Historical Data

To get historical CNBC Trading data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY

# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[QuiverCNBCs](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on CNBC Trading data, call the `add_universe` method with the `QuiverCNBCsUniverse` class and a selection function.

```
PY

def initialize(self):
    self._uinverse = self.add_universe(QuiverCNBCsUniverse, self.universe_selection)

def universe_selection(self, alt_coarse: List[QuiverCNBCsUniverse]) -> List[Symbol]:
    cnbc_data_by_symbol = {}

    for datum in alt_coarse:
        symbol = datum.symbol

        if symbol not in cnbc_data_by_symbol:
            cnbc_data_by_symbol[symbol] = []
        cnbc_data_by_symbol[symbol].append(datum)

    # define our selection criteria
    return [symbol for symbol, d in cnbc_data_by_symbol.items()
            if len([x for x in d if x.direction == OrderDirection.BUY]) >= 3]
```

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverCNBCsUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverCNBCsUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), cbncs in universe_history.items():
    for cbnc in cbncs:
        self.log(f"{cbnc.symbol} traders at {cbnc.end_time}: {cbnc.traders}")
```

PY

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverCNBCsUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverCNBCsUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), cbncs in universe_history.items():
    for cbnc in cbncs:
        print(f"{cbnc.symbol} traders at {cbnc.end_time}: {cbnc.traders}")
```

PY

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

PY

If you subscribe to CNBC Trading data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Quiver Quantitative CNBC Trading dataset enables you to create strategies using the latest recommendations made by media personalities on CNBC. Examples include the following strategies:

- Taking short positions in securities that were mentioned by Jim Cramer (CNBC commentator) in the last week
- Trading securities that were most/least discussed across CNBC programs over the last year

Classic Algorithm Example

The following example algorithm buys Apple stock if the net recommendation of media personalities on CNBC for Apple is positive. Otherwise, it holds cash.

```py
from AlgorithmImports import *


class QuiverCNBCsAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        # Subscribe to CNBC data for AAPL to generate trade signal
        self.dataset_symbol = self.add_data(QuiverCNBCs, self.aapl).symbol
        # Define a mapping from OrderDirection to int.
        self._int_by_direction = {
            OrderDirection.BUY: 1,
            OrderDirection.HOLD: 0,
            OrderDirection.SELL: -1
        }

        # history request
        history = self.history(self.dataset_symbol, 10, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self.dataset_symbol}.")

    def on_data(self, slice: Slice) -> None:
        for cnbcs in slice.Get(QuiverCNBCs).values():
            # Using mean prediction from CNBC analysts to be the trade signal
            # If the average CNBC insight is upward movement, invest AAPL
            if np.mean([self._int_by_direction[cnbc.direction] for cnbc in cnbcs]) > 0:
                self.set_holdings(self.aapl, 1)
            else:
                self.set_holdings(self.aapl, 0)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities that have at least 3 positive opinions from CNBC sources. Each day, it then forms a equal-weighted portfolio with all the securities in the universe.

```py
from AlgorithmImports import *


class QuiverCNBCsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.dataset_symbol_by_symbol = {}
        # Filter universe based on CNBC data
        self.add_universe(QuiverCNBCsUniverse, self.universe_selection)

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))

        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection(self, data: List[QuiverCNBCsUniverse]) -> List[Symbol]:
        cnbc_data_by_symbol = {}

        for datum in data:
            symbol = datum.symbol

            if symbol not in cnbc_data_by_symbol:
                cnbc_data_by_symbol[symbol] = []
            cnbc_data_by_symbol[symbol].append(datum)

        # Select the stocks with at least 3 CNBC analysts to suggest buy, reassuring the signal
        return [symbol for symbol, d in cnbc_data_by_symbol.items()
                if len([x for x in d if x.direction == OrderDirection.BUY]) >= 3]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting CNBC Data
            symbol = security.symbol
            dataset_symbol = self.add_data(QuiverCNBCs, symbol).symbol
            self.dataset_symbol_by_symbol[symbol] = dataset_symbol

            # Historical Data
            history = self.history(dataset_symbol, 10, Resolution.DAILY)
            self.debug(f"We got {len(history)} items from our history request on {dataset_symbol}.")

        for security in changes.removed_securities:
            dataset_symbol = self.dataset_symbol_by_symbol.pop(security.symbol, None)
            if dataset_symbol:
                # Remove subscription of CNBC data to release computation resources
                self.remove_security(dataset_symbol)
```

Research Example

The following example lists US Equities mentioned by Jim Cramer.

```py
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(QuiverCNBCs, aapl).symbol

# Historical data
history = qb.history(QuiverCNBCs, symbol, 60, Resolution.DAILY)
for (symbol, time), cbncs in history.items():
    for cbnc in cbncs:
        print(f"{cbnc.symbol} traders at {cbnc.end_time}: {cbnc.traders}")

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverCNBCsUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if 'Cramer' in d.traders]

universe = qb.add_universe(QuiverCNBCsUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(60), qb.time)
for (_, time), cbncs in universe_history.items():
    for cbnc in cbncs:
        print(f"{cbnc.symbol} traders at {cbnc.end_time}: {cbnc.traders}")
```

Data Point Attributes

The Quiver Quantitative CNBC Trading dataset provides `QuiverCNBCs` , `QuiverCNBC` , and `QuiverCNBCsUniverse` objects.

QuiverCNBCs

`QuiverCNBCs` objects have the following attributes:

QuiverCNBC

`QuiverCNBC` objects have the following attributes:

QuiverCNBCsUniverse

`QuiverCNBCsUniverse` objects have the following attributes:

Quiver Quantitative

Corporate Lobbying

Introduction

The Corporate Lobbying dataset by Quiver Quantitative tracks the lobbying activity of US Equities. The Lobbying Disclosure Act of 1995 requires lobbyists in the United States to disclose information about their activities, such as their clients, which issues they are lobbying on, and how much they are being paid. Quiver Quantiative scrapes this data and maps it to stock tickers to track which companies are spending money for legislative influence.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Corporate Lobbying dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the Corporate Lobbying dataset:

```
PY
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverLobbyings, self.symbol).symbol

self._universe = self.add_universe(QuiverLobbyingUniverse, self.universe_selection_filter)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 4, 1999
Asset Coverage	1,418 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Corporate Lobbying data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class QuiverLobbyingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverLobbyings, symbol).symbol
```

Accessing Data

To get the current Corporate Lobbying data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} amount at {slice.time}: {data_point.amount}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_points in slice.get(QuiverLobbyings).items():
        for data_point in data_points:
            self.log(f"{dataset_symbol} amount at {slice.time}: {data_point.amount}")
```

Historical Data

To get historical Corporate Lobbying data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[QuiverLobbyings](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Corporate Lobbying data, call the `add_universe` method with the `QuiverLobbyingUniverse` class and a selection function.

```
PY
def initialize(self):
    self._universe = self.add_universe(QuiverLobbyingUniverse, "QuiverLobbyingUniverse", Resolution.DAILY, self.universe_selection)

def universe_selection(self, alt_coarse: List[QuiverLobbyingUniverse]) -> List[Symbol]:
    lobby_data_by_symbol = {}

    for datum in alt_coarse:
        symbol = datum.symbol

        if symbol not in lobby_data_by_symbol:
            lobby_data_by_symbol[symbol] = []
        lobby_data_by_symbol[symbol].append(datum)

    return [symbol for symbol, d in lobby_data_by_symbol.items()
            if sum([x.amount for x in d]) >= 100000]
```

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverLobbyingUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverLobbyingUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (symbol, time), lobbyings in universe_history.items():
    for lobbying in lobbyings:
        print(f"{lobbying.symbol} issue at {lobbying.end_time}: {lobbying.issue}")
```

PY

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverLobbyingUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverLobbyingUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (symbol, time), lobbyings in universe_history.items():
    for lobbying in lobbyings:
        print(f"{lobbying.symbol} issue at {lobbying.end_time}: {lobbying.issue}")
```

PY

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

PY

If you subscribe to Corporate Lobbying data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Corporate Lobbying dataset enables you to create strategies using the latest information on lobbying activity. Examples include the following strategies:

- Trading securities that have spent the most on lobbying over the last quarter
- Trading securities that have had the biggest change in lobbying spend for privacy legislation over the last year

Classic Algorithm Example

The following example algorithm tracks the lobbying activity of Apple. When the company initiates a lobbying activity worth more than 50K, the algorithm buys Apple stock. When the company initiates a lobbying activity worth less than 10K, the algorithm short sells Apple stock.

```py
from AlgorithmImports import *


class QuiverLobbyingDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        # Subscribe to lobbying data for AAPL to generate trade signal
        self._dataset_symbol = self.add_data(QuiverLobbyings, self._symbol).symbol

        # history request
        history = self.history(self._dataset_symbol, 10, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self._dataset_symbol}.")

    def on_data(self, slice: Slice) -> None:
        # Trade only base on lobbying data
        for lobbyings in slice.get(QuiverLobbyings).values():
            # Buy if over 50000 lobbying amount, suggesting a favored political prospect and sentiment
            if any([lobbying.amount > 50000 for lobbying in lobbyings]):
                self.set_holdings(self._symbol, 1)
            # Sell if below 10000 lobbying amount, suggesting a less favorable political prospect and sentiment
            elif any([lobbying.amount < 10000 for lobbying in lobbyings]):
                self.set_holdings(self._symbol, -1)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities that have at least $100K worth of new lobbying activity. Each day, it then forms an equal-weighted portfolio with all of the securities in the universe.

```py
from AlgorithmImports import *


class QuiverLobbyingDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # To hold the lobbying dataset symbol for managing subscription
        self.dataset_symbol_by_symbol = {}

        # Filter universe based on the lobbying data
        self.add_universe(QuiverLobbyingUniverse, self.universe_selection)

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))

        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection(self, data: List[QuiverLobbyingUniverse]) -> List[Symbol]:
        lobby_data_by_symbol = {}

        for datum in data:
            symbol = datum.symbol

            if symbol not in lobby_data_by_symbol:
                lobby_data_by_symbol[symbol] = []
            lobby_data_by_symbol[symbol].append(datum)

        # Select and invest all stocks with lobbying amount above 100000, suggesting a more favorable political prospect and sentiment
        return [symbol for symbol, d in lobby_data_by_symbol.items()
                if sum([x.amount for x in d if x.amount]) >= 100000]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting lobbying data for trading
            symbol = security.symbol
            dataset_symbol = self.add_data(QuiverLobbyings, symbol).symbol
            self.dataset_symbol_by_symbol[symbol] = dataset_symbol
            # Historical Data
            history = self.history(dataset_symbol, 10, Resolution.DAILY)

        for security in changes.removed_securities:
            dataset_symbol = self.dataset_symbol_by_symbol.pop(security.symbol, None)
            if dataset_symbol:
                # Remove lobbying data subscription to release computation resources
                self.remove_security(dataset_symbol)
```

Research Example

The following example lists US Equities that have brought health related issues.

```py
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(QuiverLobbyings, aapl).symbol

# Historical data
history = qb.history(QuiverLobbyings, symbol, 60, Resolution.DAILY)
for (symbol, time), lobbyings in history.items():
    for lobbying in lobbyings:
        print(f"{lobbying.symbol} issue at {lobbying.end_time}: {lobbying.issue}")

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverLobbyingUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if 'health' in d.issue.lower()]

universe = qb.add_universe(QuiverLobbyingUniverse, universe_selection)

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(60), qb.time)
for (symbol, time), lobbyings in history.items():
    for lobbying in lobbyings:
        print(f"{lobbying.symbol} issue at {lobbying.end_time}: {lobbying.issue}")
```

Data Point Attributes

The Quiver Quantitative Corporate Lobbying dataset provides `QuiverLobbyings` , `QuiverLobbying` , and `QuiverLobbyingUniverse` objects.

QuiverLobbyings

`QuiverLobbyings` objects have the following attributes:

QuiverLobbying

`QuiverLobbying` objects have the following attributes:

QuiverLobbyingUniverse

`QuiverLobbyingUniverse` objects have the following attributes:

Quiver Quantitative

Insider Trading

Introduction

Corporate insiders are required to disclose purchases or sales of their own stock within two business days of when they occur. Using these disclosures, we collect data on insider trading activity, which can give hints on whether executives are bullish or bearish on their own companies. Here is a blog that we did on this dataset: https://www.quiverquant.com/blog/081121

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Insider Trading dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the Insider Trading dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverInsiderTrading, aapl).symbol
self._universe = self.add_universe(QuiverInsiderTradingUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	25 April 2014
Asset Coverage	4994 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add Insider Trading data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class QuiverInsiderTradingDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverInsiderTrading, symbol).symbol
```

Accessing Data

To get the current Insider Trading data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} shares at {slice.time}: {data_point.shares}")
```

Note that `data_points` is a list of `QuiverInsiderTrading` object since we can have several trades per symbol in a day. You need to iterate `data_points` to access the data.

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_points in slice.get(QuiverInsiderTrading).items():
        for data_point in data_points:
            self.log(f"{dataset_symbol} shares at {slice.time}: {data_point.shares}")
```

Historical Data

To get historical Insider Trading data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
self.history[QuiverInsiderTrading](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Insider Trading data, call the `add_universe` method with the `QuiverInsiderTradingUniverse` class and a selection function.

```
PY
class InsiderTradingUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2023, 1, 1)
        self._universe = self.add_universe(QuiverInsiderTradingUniverse, self._select_assets)

    def _select_assets(self, alt_coarse: List[QuiverInsiderTradingUniverse]) -> List[Symbol]:
        dollar_volume_by_symbol = {}
        for data in alt_coarse:
            symbol = data.symbol
            if not data.price_per_share:
                continue
            if symbol not in dollar_volume_by_symbol:
                dollar_volume_by_symbol[symbol] = 0
            dollar_volume_by_symbol[symbol] += (data.shares or 0) * data.price_per_share
        return [
            symbol for symbol, _ in sorted(dollar_volume_by_symbol.items(), key=lambda kvp: kvp[1])[-10:]
        ]
```

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverInsiderTradingUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverInsiderTradingUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), insiders in universe_history.items():
    for insider in insiders:
        if insider.price_per_share:
            self.log(f"{insider.symbol} volume at {insider.end_time}: {insider.shares * insider.price_per_share}")
```

PY

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverInsiderTradingUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverInsiderTradingUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), insiders in universe_history.items():
    for insider in insiders:
        if insider.price_per_share:
            print(f"{insider.symbol} volume at {insider.end_time}: {insider.shares * insider.price_per_share}")
```

PY

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

PY

If you subscribe to Insider Trading data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Quiver Quantitative Insider Trading dataset enables researchers to create strategies using the latest information on insider trading activity. Examples include:

- Taking a short position in securities that have had the most insider selling over the last 5 days
- Buying any security that has had over $100,000 worth of shares purchased by insiders in the last month

Classic Algorithm Example

In this example, we would 100% long AAPL if there was any insider buying event. Otherwise, we liquidate it.

```py
from AlgorithmImports import *


class QuiverInsiderTradingAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        # Subscribe to insider trade data for AAPL to generate trade signal
        self.dataset_symbol = self.add_data(QuiverInsiderTrading, self._symbol).symbol

        # history request
        history = self.history(self.dataset_symbol, 10, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self.dataset_symbol}.")

    def on_data(self, slice: Slice) -> None:
        # Trade only base on insider trade data
        for insider_trades in slice.Get(QuiverInsiderTrading).values():
            for insider_trade in insider_trades:
                # Any buy insider trade will result in buying, assuming insider have confidence in stock price with more informed
information and projection
                if insider_trade.shares > 0:
                    self.set_holdings(self._symbol, 1)
                # Any sell insider trade will result in liquidation, assuming insiders believe the stock price has reached maximum or
poor future confidence
                else:
                    self.liquidate(self._symbol)
```

Framework Algorithm Example

In this example, we long a equal-sized portfolio with equities that having insider trading for over $100000 USD in the previous day.

```py
from AlgorithmImports import *


class QuiverInsiderTradingDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # To hold the insider trade dataset symbol for managing subscription
        self.data_symbols = {}

        # Filter universe using insider trade data
        self.add_universe(QuiverInsiderTradingUniverse, self.universe_selection)

        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))

        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def universe_selection(self, data: List[QuiverInsiderTradingUniverse]) -> List[Symbol]:
        symbol_data = {}

        for datum in data:
            symbol = datum.symbol

            if symbol not in symbol_data:
                symbol_data[symbol] = []
            symbol_data[symbol].append(datum)

        # Select the ones with insider trading dollar amount above 100k, assuming insider have confidence in stock price with more
informed information and projection
        return [symbol for symbol, d in symbol_data.items()
                if sum([x.shares * x.price_per_share for x in d if x.shares and x.price_per_share]) >= 100000]

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting insider trade data for trading
            symbol = security.symbol
            dataset_symbol = self.add_data(QuiverInsiderTrading, symbol).symbol
            self.data_symbols[symbol] = dataset_symbol
            # Historical Data
            history = self.history(dataset_symbol, 10, Resolution.DAILY)

        for security in changes.removed_securities:
            dataset_symbol = self.data_symbols.pop(security.symbol, None)
            if dataset_symbol:
                # Remove insider trade data subscription to release computation resources
                self.remove_security(dataset_symbol)
```

Research Example

The following example lists US Equities having insider trading for over $10,000,000.

```py
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(QuiverInsiderTrading, aapl).symbol

# Historical data
history = qb.history(QuiverInsiderTrading, symbol, 60, Resolution.DAILY)
for (symbol, time), insiders in history.items():
    for insider in insiders:
        print(f"{insider.symbol} shares at {insider.end_time}: {insider.shares}")

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverInsiderTradingUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if d.price_per_share and d.shares * d.price_per_share >= 10000000]

universe = qb.add_universe(QuiverInsiderTradingUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(5), qb.time)
for (_, time), insiders in universe_history.items():
    for insider in insiders:
        if insider.price_per_share:
            print(f"{insider.symbol} volume at {insider.end_time}: {insider.shares * insider.price_per_share}")
```

Data Point Attributes

The Insider Trading dataset provides `QuiverInsiderTrading` and `QuiverInsiderTradingUniverse` object.

QuiverInsiderTrading

`QuiverInsiderTrading` objects have the following attributes:

QuiverInsiderTradingUniverse

`QuiverInsiderTradingUniverse` objects have the following attributes:

Quiver Quantitative

US Congress Trading

Introduction

The US Congress Trading dataset by Quiver Quantitative tracks US Equity trades made by members of Congress in the Senate and the House of Representatives. The data covers 1,800 US Equities, starts in January 2016, and is delivered on a daily frequency. This dataset is created by scraping SEC reports.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US Congress Trading dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the US Congress Trading dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverCongress, self.aapl).symbol

self._universe = self.add_universe(QuiverQuantCongressUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2016
Asset Coverage	1,800 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add US Congress Trading data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class QuiverCongressDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverCongress, symbol).symbol
```

Accessing Data

To get the current US Congress Trading data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} transaction amount at {slice.time}: {data_point.amount}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(QuiverCongress).items():
        self.log(f"{dataset_symbol} transaction amount at {slice.time}: {data_point.amount}")
```

Historical Data

To get historical US Congress Trading data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[QuiverCongress](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on US Congress Trading data, call the `add_universe` method with the `QuiverQuantCongressUniverse` class and a selection function.

```
def initialize(self) -> None:
    self._universe = self.add_universe(QuiverQuantCongressUniverse, self.universe_selection)

def universe_selection(self, alt_coarse: List[QuiverQuantCongresssUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
        if d.amount > 200000 and d.transaction == OrderDirection.BUY]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverQuantCongressUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverQuantCongressUniverse objects:
history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), trades in universe_history.items():
    for trade in trades:
        self.log(f"{trade.symbol} amount at {trade.end_time}: {trade.amount} {trade.representative}") {trade.Representative}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```py
# DataFrame example where the columns are the QuiverQuantCongressUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverQuantCongressUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), trades in universe_history.items():
    for trade in trades:
        print(f"{trade.symbol} amount at {trade.end_time}: {trade.amount} {trade.representative}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```py
self.remove_security(self.dataset_symbol)
```

If you subscribe to US Congress Trading data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The US Congress Trading dataset enables you to take immediate action on trades made by informed Members of Congress. Examples include the following strategies:

- Following the trades of specific representatives on the premise that the representatives are more informed
- Assigning a long/short-bias to securities on a daily frequency based on how Members of Congress are trading them

Classic Algorithm Example

The following example algorithm follows the net bias of the trades made by Members of Congress each day. When Members of Congress are net buyers, the algorithm buys. When they are net sellers, the algorithm short sells.

```py
from AlgorithmImports import *


class QuiverCongressDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting data per underlying data to subscribe to updated congress members' trade information
        aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        quiver_congress_symbol = self.add_data(QuiverCongress, aapl).symbol

        # Historical data
        history = self.history(QuiverCongress, quiver_congress_symbol, 60, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request");

    def on_data(self, slice: Slice) -> None:
        congress_by_symbol = slice.Get(QuiverCongress)

        # Determine net direction of Congress trades for each security to estimate the sentiment direction due to political factor
        net_quantity_by_symbol = {}
        for symbol, points in congress_by_symbol.items():
            symbol = symbol.underlying
            if symbol not in net_quantity_by_symbol:
                net_quantity_by_symbol[symbol] = 0
            # Voting weight is by order size
            for point in points:
                net_quantity_by_symbol[symbol] += (1 if point.transaction == OrderDirection.BUY else -1) * point.amount

        for symbol, net_quantity in net_quantity_by_symbol.items():
            if net_quantity == 0:
                self.liquidate(symbol)
                continue
            # Buy when Congress members have bought, short otherwise
            self.set_holdings(symbol, 1 if net_quantity > 0 else -1)
```

Framework Algorithm Example

The following example algorithm follows the net bias of the trades made by Members of Congress each day. When Members of Congress are net buyers, the algorithm buys. When they are net sellers, the algorithm short sells.

```py
from AlgorithmImports import *


class QuiverCongressDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Filter according to QuiverCongress data
        self.add_universe(QuiverQuantCongressUniverse, "QuiverQuantCongresssUniverse", Resolution.DAILY, self.universe_selection)
        self.add_alpha(CongressAlphaModel())
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel(lambda time: None))
        self.add_risk_management(NullRiskManagementModel())
        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, alt_coarse: List[QuiverQuantCongressUniverse]) -> List[Symbol]:
        # Only include the ones with large size buy, since they are estimated to be materially confident to go up
        return [d.symbol for d in alt_coarse if d.amount > 10_000 and d.transaction == OrderDirection.BUY]


class CongressAlphaModel(AlphaModel):

    symbol_data_by_symbol = {}
    insight_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        congress_by_symbol = slice.get(QuiverCongress)
        if congress_by_symbol:
            self.insight_by_symbol.clear()
            # Determine net direction of Congress trades for each security to estimate the sentiment direction due to political factor
            net_quantity_by_symbol = {}
            for symbol, points in congress_by_symbol.items():
                symbol = symbol.underlying
                if symbol not in net_quantity_by_symbol:
                    net_quantity_by_symbol[symbol] = 0
                # Voting weight is by order size
                for point in points:
                    net_quantity_by_symbol[symbol] += (1 if point.transaction == OrderDirection.BUY else -1) * point.amount

            for symbol, net_quantity in net_quantity_by_symbol.items():
                # Buy when Congress members have bought, as they may have advance information to be confident to the return direction
                if net_quantity > 0 and not algorithm.portfolio[symbol].is_long:
                    direction = InsightDirection.UP
                # Short sell when Congress members have sold
                elif net_quantity < 0 and not algorithm.portfolio[symbol].is_short:
                    direction = InsightDirection.DOWN
                else:
                    continue
                self.insight_by_symbol[symbol] = Insight.price(symbol, timedelta(7), direction, weight=0.5)
        # To avoid error messages, emit the insights when the algorithm has a price for the asset.
        return [
            self.insight_by_symbol.pop(symbol)
            for symbol, i in list(self.insight_by_symbol.items())
            if algorithm.securities[symbol].price
        ]

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            self.symbol_data_by_symbol[symbol] = SymbolData(algorithm, symbol)

        for security in changes.removed_securities:
            symbol_data = self.symbol_data_by_symbol.pop(security.symbol, None)
            if symbol_data:
                symbol_data.dispose()


class SymbolData:

    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol):
        self.algorithm = algorithm

        # Requesting data per underlying data to subscribe to updated congress members' trade information
        self.quiver_congress_symbol = algorithm.add_data(QuiverCongress, symbol).symbol

        # Historical data
        history = algorithm.history(self.quiver_congress_symbol, 14, Resolution.DAILY)

    def dispose(self) -> None:
        # Unsubscribe from Quiver Congress feed for this security to release computational resources
        self.algorithm.remove_security(self.quiver_congress_symbol)
```

Research Example

The following example lists trades made by Members of Congress daily as buyers.

```py
qb = QuantBook()

# Add QuiverCongress
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
quiver_congress_symbol = qb.add_data(QuiverCongress, aapl).symbol

# Historical data
history = qb.history(QuiverCongress, quiver_congress_symbol, 60, Resolution.DAILY)
for (symbol, time), trades in history.items():
    for trade in trades:
        print(f'{time} {trade.symbol.value} {trade.representative} {trade.amount}')

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverQuantCongressUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if d.transaction == OrderDirection.BUY]

universe = qb.add_universe(QuiverQuantCongressUniverse, universe_selection)

# Historical Universe data
history = qb.universe_history(universe, qb.time-timedelta(60), qb.time)
for (univere_symbol, time), trades in history.items():
    for trade in trades:
        print(f'{time} {trade.symbol.value} {trade.representative} {trade.amount}')
```

Data Point Attributes

The US Congress Trading dataset provides `QuiverCongressDataPoint` , `QuiverCongress` , and `QuiverQuantCongressUniverse` objects.

QuiverCongressDataPoint Attributes

`QuiverCongressDataPoint` object has the following attributes:

QuiverCongress Attributes

`QuiverCongress` object has the following attributes:

QuiverQuantCongressUniverse Attributes

`QuiverQuantCongressUniverse` object has the following attributes:

Quiver Quantitative

US Government Contracts

Introduction

The US Government Contracts dataset by Quiver Quantitative tracks the transactions of government contracts with publicly traded companies. The data covers over 700 US Equities, starts in October 2019, and is delivered on a daily frequency. Quiver Quantitative creates this dataset by using the API for USASpending.gov , which has the official open data source of federal spending information. The rows in this dataset only show new contracts, not payments or modifications to existing contracts. The dollar amounts are based on the total dollars obligated from each contract.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US Government Contracts dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the US Government Contracts dataset:

```
PY
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverGovernmentContract, self.aapl).symbol

self._universe = self.add_universe(QuiverGovernmentContractUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	October 15, 2019
Asset Coverage	748 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add US Government Contracts data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
PY
class QuiverGovernmentContractDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        symbol = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverGovernmentContract, symbol).symbol
```

Accessing Data

To get the current US Government Contract data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} amount at {slice.time}: {data_point.amount}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_points in slice.get(QuiverGovernmentContract).items():
        for data_point in data_points:
            self.log(f"{dataset_symbol} amount at {slice.time}: {data_point.amount}")
```

Historical Data

To get historical US Government Contracts data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[QuiverGovernmentContract](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on US Government Contract data, call the `add_universe` method with the `QuiverGovernmentContractUniverse` class and a selection function.

```
PY
def initialize(self):
    self._universe = self.add_universe(QuiverGovernmentContractUniverse, self.universe_selection)

def universe_selection(self, alt_coase: List[QuiverGovernmentContractUniverse]) -> List[Symbol]:
    gov_contract_data_by_symbol = {}

    for datum in alt_coarse:
        symbol = datum.symbol

        if symbol not in gov_contract_data_by_symbol:
            gov_contract_data_by_symbol[symbol] = []
        gov_contract_data_by_symbol[symbol].append(datum)

    return [symbol for symbol, d in gov_contract_data_by_symbol.items()
            if len(d) >= 3 and sum([x.amount for x in d]) > 50000]
```

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverGovernmentContractUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverGovernmentContractUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (_, time), contracts in universe_history.items():
    for contract in contracts:
        self.log(f"{contract.symbol} amount at {contract.end_time}: {contract.amount}")
```

PY

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverGovernmentContractUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverGovernmentContractUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (_, time), contracts in universe_history.items():
    for contract in contracts:
        print(f"{contract.symbol} amount at {contract.end_time}: {contract.amount}")
```

PY

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

PY

If you subscribe to US Government Contracts data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Quiver Quantitative US Government Contracts dataset enables you to create strategies using the latest information on government contracts activity. Examples include the following strategies:

- Buying securities that have received the most new government contract awards over the last month
- Trading securities that have had the biggest change in government contracts awards over the last year

Classic Algorithm Example

The following example algorithm buys Apple stock when they receive a new government contract worth over 50K. If they receive a new contract worth under 10K, the algorithm short sells Apple.

```py
from AlgorithmImports import *


class QuiverGovernmentContractAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        # Subscribe to government contract data for AAPL to generate trade signal
        self.dataset_symbol = self.add_data(QuiverGovernmentContract, self.aapl).symbol

        # history request
        history = self.history(self.dataset_symbol, 10, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self.dataset_symbol}.")

    def on_data(self, slice: Slice) -> None:
        # Trade only base on government contract data
        for gov_contracts in slice.Get(QuiverGovernmentContract).values():
            # Buy if over 50000 government contract amount, suggesting a large income
            if any([gov_contract.amount > 50000 for gov_contract in gov_contracts]):
                self.set_holdings(self.aapl, 1)
            # Sell if below 10000 government contract amount, suggesting a smaller than usual income
            elif any([gov_contract.amount < 10000 for gov_contract in gov_contracts]):
                self.set_holdings(self.aapl, -1)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities that have just received a government contract worth at least $5K. Each day, it then forms an equal-weighted dollar-neutral portfolio with the 10 companies that received the largest contracts and the 10 companies that received the smallest contracts.

```py
from AlgorithmImports import *


class QuiverGovernmentContractDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Filter universe using government contract data
        self.add_universe(QuiverGovernmentContractUniverse, self.universe_selection)

        # Custom alpha model that emit insights based on updated government contract data
        self.add_alpha(QuiverGovernmentContractAlphaModel())

        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, data: List[QuiverGovernmentContractUniverse]) -> List[Symbol]:
        gov_contract_data_by_symbol = {}

        for datum in data:
            symbol = datum.symbol

            if symbol not in gov_contract_data_by_symbol:
                gov_contract_data_by_symbol[symbol] = []
            gov_contract_data_by_symbol[symbol].append(datum)

        # Only select the stocks with over 5000 government contracts, which is considered material information
        return [symbol for symbol, d in gov_contract_data_by_symbol.items()
                if sum([x.amount for x in d]) > 5000]

class QuiverGovernmentContractAlphaModel(AlphaModel):

    def __init__(self) -> None:
        # A variable to control the rebalancing time
        self.last_time = datetime.min
        # To hold the government contract dataset symbol for managing subscription
        self.dataset_symbol_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        if self.last_time > algorithm.time: return []

        # Trade signal only based on government contract data
        data_points = slice.Get(QuiverGovernmentContract)

        if not data_points: return []

        gov_contracts = {}
        # To aggregate all data per symbol for analysis
```

```
        # To aggregate all data per symbol for analysis
        for data_point in data_points:
            if not algorithm.securities[data_point.key.underlying].price:
                continue
            if data_point.Key not in gov_contracts:
                gov_contracts[data_point.Key] = 0

            for gov_contract in data_point.Value:
                gov_contracts[data_point.Key] += gov_contract.amount

        # Long the top 10 highest government contract amount, predicting a higher expected income and return
        # Short the lowest 10 government contract amount, predicting a lower expected income and return
        sorted_by_gov_contracts = sorted(gov_contracts.items(), key=lambda x: x[1])
        long_symbols = [x[0].underlying for x in sorted_by_gov_contracts[-10:]]
        short_symbols = [x[0].underlying for x in sorted_by_gov_contracts[:10]]

        insights = []
        for symbol in long_symbols:
            insights.append(Insight.price(symbol, Expiry.END_OF_DAY, InsightDirection.UP))
        for symbol in short_symbols:
            insights.append(Insight.price(symbol, Expiry.END_OF_DAY, InsightDirection.DOWN))

        self.last_time = Expiry.END_OF_DAY(algorithm.Time)

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Requesting government contract data for trade signal generation
            symbol = security.symbol
            dataset_symbol = algorithm.add_data(QuiverGovernmentContract, symbol).symbol
            self.dataset_symbol_by_symbol[symbol] = dataset_symbol
            # Historical Data
            history = algorithm.history(dataset_symbol, 10, Resolution.DAILY)

        for security in changes.removed_securities:
            dataset_symbol = self.dataset_symbol_by_symbol.pop(security.symbol, None)
            if dataset_symbol:
                # Remove government contract data subscription to release computation resources
                algorithm.remove_security(dataset_symbol)
```

Research Example

The following example lists all US Equities with Government contracts in the past year.

```
PY
qb = QuantBook()

# Requesting Data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(QuiverGovernmentContract, aapl).symbol

# Historical data
history = qb.history(QuiverGovernmentContract, symbol, 360, Resolution.DAILY)
for (symbol, time), contracts in history.items():
    for contract in contracts:
        print(f"{contract.symbol} amount at {contract.end_time}: {contract.amount}")

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverGovernmentContractUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse]

universe = qb.add_universe(QuiverGovernmentContractUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(360), qb.time)
for (_, time), contracts in universe_history.items():
    for contract in contracts:
        print(f"{contract.symbol} amount at {contract.end_time}: {contract.amount}")
```

Data Point Attributes

The US Government Contracts dataset provides `QuiverGovernmentContract` and `QuiverGovernmentContractUniverse` objects.

QuiverGovernmentContract

`QuiverGovernmentContract` objects have the following attributes:

QuiverGovernmentContractUniverse

`QuiverGovernmentContractUniverse` objects have the following attributes:

Quiver Quantitative

WallStreetBets

Introduction

The WallStreetBets dataset by Quiver Quantitative tracks daily mentions of different equities on Reddit's popular WallStreetBets forum. The data covers 6,000 Equities, starts in August 2018, and is delivered on a daily frequency. The dataset is created by scraping the daily discussion threads on r/WallStreetBets and parsing the comments for ticker mentions.

`This dataset ends in February 2025. You are free to use it on QuantConnect Cloud for research purposes.`

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the WallStreetBets dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Quiver Quantitative was founded by two college students in February 2020 with the goal of bridging the information gap between Wall Street and non-professional investors. Quiver allows retail investors to tap into the power of big data and have access to actionable, easy to interpret data that hasn't already been dissected by Wall Street.

Getting Started

The following snippet demonstrates how to request data from the WallStreetBets dataset:

```
PY
self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
self.dataset_symbol = self.add_data(QuiverWallStreetBets, self.aapl).symbol

self._universe = self.add_universe(QuiverWallStreetBetsUniverse, self.universe_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	August 2018
End Date	February 2025
Asset Coverage	6,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add WallStreetBets data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class QuiverWallStreetBetsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.DAILY).symbol
        self.dataset_symbol = self.add_data(QuiverWallStreetBets, self.aapl).symbol
```

Accessing Data

To get the current WallStreetBets data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} mentions at {slice.time}: {data_point.mentions}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_points in slice.get(QuiverWallStreetBets).items():
        for data_point in data_points:
            self.log(f"{dataset_symbol} mentions at {slice.time}: {data_point.mentions}")
```

Historical Data

To get historical WallStreetBets data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[QuiverWallStreetBets](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on WallStreetBets data, call the `add_universe` method with the `QuiverWallStreetBetsUniverse` class and a selection function.

```
def initialize(self) -> None:
    self.universe = self.add_universe(QuiverWallStreetBetsUniverse, self.universe_selection)

def universe_selection(self, alt_coarse: List[QuiverWallStreetBetsUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if d.mentions > 100  and d.rank < 100]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverWallStreetBetsUniverse attributes:
history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of QuiverWallStreetBetsUniverse objects:
universe_history = self.history(self._universe, 30, Resolution.DAILY)
for (univere_symbol, time), bets in universe_history.items():
    for bet in bets:
        self.log(f"{bet.symbol} mentions at {bet.end_time}: {bet.mentions}")
```

PY

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the QuiverWallStreetBetsUniverse attributes:
history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of QuiverWallStreetBetsUniverse objects:
universe_history = qb.universe_history(universe, qb.time-timedelta(30), qb.time)
for (univere_symbol, time), bets in universe_history.items():
    for bet in bets:
        print(f"{bet.symbol} rank at {bet.end_time}: {bet.rank}")
```

PY

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

PY

If you subscribe to WallStreetBets data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The WallStreetBets dataset enables you to create strategies using the latest activity on the WallStreetBets daily discussion thread. Examples include the following strategies:

- Trading any security that is being mentioned
- Trading securities that are receiving more/less mentions than they were previously
- Trading the security that is being mentioned the most/least for the day

Classic Algorithm Example

The following example algorithm creates a dynamic universe of US Equities based on daily WallStreetBets data. When a security is mentioned on r/WallStreetBets more than five times in a day, the algorithm buys the security. When a security is mentioned five time in a day or less, the algorithm short sells the security.

```py
from AlgorithmImports import *


class QuiverWallStreetBetsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        # Filter using wall street bet insights
        self._universe = self.add_universe(QuiverWallStreetBetsUniverse, self.universe_selection)

    def on_data(self, slice: Slice) -> None:
        points = slice.Get(QuiverWallStreetBets)
        for point in points.Values:
            symbol = point.symbol.underlying
            security = self.securities[symbol]
            if not security.is_tradable:
                continue

            # Buy if the stock was mentioned more than 5 times in the WallStreetBets daily discussion, which translate into high
popularity of rise
            if point.mentions > 5 and not security.holdings.is_long:
                self.market_order(symbol, 1)

            # Otherwise, short sell
            elif point.mentions <= 5 and not security.holdings.is_short:
                self.market_order(symbol, -1)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            # Requesting wall street bet data to obtain the trader's insights
            quiver_wsb_symbol = self.add_data(QuiverWallStreetBets, added.symbol).symbol

            # Historical data
            history = self.history(QuiverWallStreetBets, quiver_wsb_symbol, 60, Resolution.DAILY)

    def universe_selection(self, alt_coarse: List[QuiverWallStreetBetsUniverse]) -> List[Symbol]:
        # Select the ones with popularity (mentions) of better-than-others performance (rank)
        return [d.symbol for d in alt_coarse \
                    if d.mentions > 10 \
                    and d.rank < 100]
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of US Equities based on daily WallStreetBets data. When a security is mentioned on r/WallStreetBets more than five times in a day, the algorithm buys the security. When a security is mentioned five time in a day or less, the algorithm short sells the security.

```py
from AlgorithmImports import *


class QuiverWallStreetBetsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY
        # Filter using wall street bet insights
        self._universe = self.add_universe(QuiverWallStreetBetsUniverse, self.universe_selection)

        self.add_alpha(WallStreamBetsAlphaModel())

        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())

    def universe_selection(self, alt_coarse: List[QuiverWallStreetBetsUniverse]) -> List[Symbol]:
        # Select the ones with popularity (mentions) of better-than-others performance (rank)
        return [d.symbol for d in alt_coarse
                    if d.mentions > 10 and d.rank < 100]

class WallStreamBetsAlphaModel(AlphaModel):

    symbol_data_by_symbol = {}

    def __init__(self, mentions_threshold: int = 5) -> None:
        self.mentions_threshold = mentions_threshold

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        points = slice.Get(QuiverWallStreetBets)
        for point in points.Values:
            # Buy if the stock was mentioned more than 5 times in the WallStreetBets daily discussion, which translate into high
popularity of rise
            # Otherwise short sell
            target_direction = InsightDirection.UP if point.mentions > self.mentions_threshold else InsightDirection.DOWN
            self.symbol_data_by_symbol[point.symbol.underlying].target_direction = target_direction

        for symbol, symbol_data in self.symbol_data_by_symbol.items():
            # Ensure we have security data for the current Slice to avoid stale fill
            if not (slice.contains_key(symbol) and slice[symbol] is not None):
                continue

            if symbol_data.target_direction is not None:
                insights += [Insight.price(symbol, timedelta(1), symbol_data.target_direction)]
                symbol_data.target_direction = None

        return insights


    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            self.symbol_data_by_symbol[symbol] = SymbolData(algorithm, symbol)

        for security in changes.removed_securities:
            symbol_data = self.symbol_data_by_symbol.pop(security.symbol, None)
            if symbol_data:
                symbol_data.dispose()


class SymbolData:
    target_direction = None

    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        self.algorithm = algorithm

        # Requesting wall street bet data to obtain the trader's insights
        self.quiver_wsb_symbol = algorithm.add_data(QuiverWallStreetBets, symbol).symbol

        # Historical data
        history = algorithm.history(self.quiver_wsb_symbol, 60, Resolution.DAILY)

    def dispose(self) -> None:
        # Unsubscribe from the Quiver WallStreetBets feed for this security to release computationa resources
        self.algorithm.remove_security(self.quiver_wsb_symbol)
```

Research Example

The following example lists low-ranking US Equities that are mentioned more than ten times on r/WallStreetBets.

```py
qb = QuantBook()

# Requesting data
aapl = qb.add_equity("AAPL", Resolution.DAILY).symbol
symbol = qb.add_data(QuiverWallStreetBets, aapl).symbol

# Historical data
history = qb.history(QuiverWallStreetBets, symbol, 60, Resolution.DAILY)
for (symbol, time), bet in history.iterrows():
    print(f"{symbol} rank at {time}: {bet['rank']}")

# Add Universe Selection
def universe_selection(alt_coarse: List[QuiverWallStreetBetsUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse if d.mentions > 10 and d.rank < 100]

universe = qb.add_universe(QuiverWallStreetBetsUniverse, universe_selection)

# Historical Universe data
universe_history = qb.universe_history(universe, qb.time-timedelta(60), qb.time)
for (univere_symbol, time), bets in universe_history.items():
    for bet in bets:
        print(f"{bet.symbol} rank at {bet.end_time}: {bet.rank}")
```

Data Point Attributes

The WallStreetBets dataset provides `QuiverWallStreetBets` and `QuiverWallStreetBetsUniverse` objects.

QuiverWallStreetBets Attributes

`QuiverWallStreetBets` objects have the following attributes:

QuiverWallStreetBetsUniverse Attributes

`QuiverWallStreetBetsUniverse` objects have the following attributes:

Datasets

RegAlytics

RegAlytics was founded by Mary Kopczynski, Aaron Heisler, Alexander Appugliese, and Werner Pauliks in 2019 with the goal of significantly reducing the time and cost required to mitigate regulatory risk. RegAlytics provides access to accurate and clean regulatory data from all global regulators in all sectors that is enriched by regulatory experts for risk and compliance teams everywhere. Please come to RegAlytics directly if you would like data on other sectors or countries!

US Regulatory Alerts - Financial Sector

RegAlytics

US Regulatory Alerts - Financial Sector

Introduction

The US Regulatory Alerts dataset by RegAlytics tracks changes from over 8,000 globally governing bodies. The data covers over 2.5 million alerts, starts from January 2020, and is delivered on a daily basis. This dataset is created by sourcing information from over 8,000 regulators and using proprietary technology to gather and structure the regulatory data. Once prepared, the data is thoroughly reviewed by RegAlytics' team of regulatory experts and delivered each morning by 8AM for industry use.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the US Regulatory Alerts - Financial Sector dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

RegAlytics was founded by Mary Kopczynski, Aaron Heisler, Alexander Appugliese, and Werner Pauliks in 2019 with the goal of significantly reducing the time and cost required to mitigate regulatory risk. RegAlytics provides access to accurate and clean regulatory data from all global regulators in all sectors that is enriched by regulatory experts for risk and compliance teams everywhere. Please come to RegAlytics directly if you would like data on other sectors or countries!

Getting Started

The following snippet demonstrates how to request data from the US Regulatory Alerts - Financial Sector dataset:

```
self.dataset_symbol = self.add_data(RegalyticsRegulatoryArticles, "REG").symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2020
Coverage	2,500,000 Alerts
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

To add US Regulatory Alerts data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class RegalyticsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(RegalyticsRegulatoryArticles, "REG").symbol
```

Accessing Data

To get the current US Regulatory Alerts data, index the current `Slice` with the dataset `Symbol` . `Slice` objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_points = slice[self.dataset_symbol]
        for data_point in data_points:
            self.log(f"{self.dataset_symbol} title at {slice.time}: {data_point.title}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    data = slice.get(RegalyticsRegulatoryArticles)
    if data:
        for articles in data.values():
            self.log(f"{self.time} {articles.to_string()}")

            for article in articles:
                self.log(f"{self.dataset_symbol} article title at {slice.time}: {article.title}")
```

Historical Data

To get historical US Regulatory Alerts data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame where the columns are the RegalyticsRegulatoryArticle attributes:
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY, flatten=True)

# Series where the values are lists of RegalyticsRegulatoryArticle objects:
history_series = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects:
history_bars =  self.history[RegalyticsRegulatoryArticles](self.dataset_symbol, 100, Resolution.DAILY)
```

Remove Subscriptions

To remove your subscription to US Regulatory Alerts data, call the `remove_security` method.

```
PY
self.remove_security(self.dataset_symbol)
```

Example Applications

This regulatory dataset enables you to accurately design strategies while mitigating regulatory risk. Examples include the following strategies:

- Temporarily increasing/decreasing exposure to securities when new regulations are announced
- Parsing the content of regulatory announcements to determine market or sector impact
- Country rotation based on regulatory alert sentiment

Classic Algorithm Example

The following example algorithm sells short 100% SPY if any predetermined negative sentimental phrase was present in any of the previous day's regulatory articles. Otherwise, it buys and hold 100% SPY.

```py
from AlgorithmImports import *


class RegalyticsDataAlgorithm(QCAlgorithm):
    # Pre-defined list of negative sentiment phrases as indicator for short selling, since these words will decrease confident in equity performance
    _negative_sentiment_phrases = [
        "emergency rule", "proposed rule change", "development of rulemaking"
    ]

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.spy = self.add_equity("SPY", extended_market_hours=True).symbol
        # Requesting data to receive updated regulatory news for timely short selling
        self.regalytics_symbol = self.add_data(RegalyticsRegulatoryArticles, "REG").symbol

        # Historical data for past articles
        history = self.history(self.regalytics_symbol, 7, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def on_data(self, slice: Slice) -> None:
        # Only trade on regulatory news data
        data = slice.get(RegalyticsRegulatoryArticles)
        if not data:
            return
        # If any of the negative phrases appeared in regulatory news, we expect a market drop for the day
        detected_negativity = False
        for articles in data.values():
            if detected_negativity:
                break
            detected_negativity = any(
                [p in article.title.lower() for p in self._negative_sentiment_phrases for article in articles]
            )
        self.transactions.cancel_open_orders()
        quantity = self.calculate_order_quantity(self.spy, -1 if detected_negativity else 1)
        if quantity:
            self.market_order(self.spy, quantity)
```

Framework Algorithm Example

The following example algorithm buys and holds the S&P 500 ETF, SPY. To reduce regulatory risk, it liquidates your holdings for 2 days following regulatory announcements regarding new rules.

```py
from AlgorithmImports import *


class RegalyticsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.DAILY
        # Trade SPY as the proxy of the whole equity market movement on the regulatory news
        symbols = [ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))

        self.add_alpha(RegAnalyticsAlphaModel(self))

        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(lambda time: None))

        self.add_risk_management(NullRiskManagementModel())

        self.set_execution(ImmediateExecutionModel())


class RegAnalyticsAlphaModel(AlphaModel):

    symbols = []
    last_news_date = datetime.min
    target_direction = InsightDirection.UP
    # Pre-defined list of negative sentiment phrases as indicator for short selling, since these words will decrease confident in equity
performance
    negative_sentiment_phrases = ["emergency rule", "proposed rule change", "development of rulemaking"]
    # Assuming the negative impact of the regulatory news will last for 2 days
    news_affect_duration = timedelta(days = 2)

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Requesting data to receive updated regulatory news for timely short selling
        self.regalytics_symbol = algorithm.add_data(RegalyticsRegulatoryArticles, "REG").symbol

        # Historical data for past articles
        history = algorithm.history(self.regalytics_symbol, 14, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} from our history request")

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Only trade on regulatory news data
        if slice.contains_key(self.regalytics_symbol) and slice[self.regalytics_symbol] is not None:
            for article in slice[self.regalytics_symbol]:
                title = article.title.lower()
                # If any of the negative phrases appeared in regulatory news, we expect a market drop for the day
                # Signal an exit from the market when regulatory articles with negative sentiment are released
                if any(phrase in title for phrase in self.negative_sentiment_phrases):
                    self.target_direction = InsightDirection.FLAT
                    self.last_news_date = slice.time

        # Signal an entry if we've waited long enough for the market to digest the negative news
        if self.last_news_date + self.news_affect_duration < slice.time:
            self.target_direction = InsightDirection.UP

        for symbol in self.symbols:
            # Ensure we have security data in the current Slice to avoid stale fill
            if not (slice.contains_key(symbol) and slice[symbol] is not None):
                continue

            if self.target_direction == InsightDirection.UP and not algorithm.portfolio[symbol].invested:
                insights += [Insight.price(symbol, timedelta(days=90), InsightDirection.UP)]
            elif self.target_direction == InsightDirection.FLAT and algorithm.portfolio[symbol].invested:
                insights += [Insight.price(symbol, self.news_affect_duration, InsightDirection.FLAT)]

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.symbols.append(security.symbol)

        for security in changes.removed_securities:
            symbol = security.symbol
            if symbol in self.symbols:
                self.symbols.remove(symbol)
```

Data Point Attributes

The US Regulatory Alerts dataset provides `RegalyticsRegulatoryArticle` and `RegalyticsRegulatoryArticles` objects.

RegalyticsRegulatoryArticle

`RegalyticsRegulatoryArticle` objects have the following attributes:

RegalyticsRegulatoryArticles

`RegalyticsRegulatoryArticles` objects have the following attributes:

Datasets

Securities and Exchange Commission

The mission of the U.S. Securities and Exchange Commission is to protect investors, maintain fair, orderly, and efficient markets, and facilitate capital formation. The SEC oversees the key participants in the securities world, including securities exchanges, securities brokers and dealers, investment advisors, and mutual funds. The SEC is concerned primarily with promoting the disclosure of important market-related information, maintaining fair dealing, and protecting against fraud.

US SEC Filings

Securities and Exchange Commission

US SEC Filings

Introduction

The US SEC Filings dataset provides the quarterly financial earning reports that the United States Securities and Exchange Commission (SEC) requires from publicly traded companies in the US. The data covers 15,000 US Equities, starts in January 1998, and is delivered on a daily frequency. The data is sourced from the SEC's Electronic Data Gathering, Analysis, and Retrieval (EDGAR) system. QuantConnect downloads and formats the Quarterly Financial Reports (10-Q) and Annual Financial Report (8-K) filings of companies into a format for easy consumption by LEAN.

For more information about the US SEC Filings dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The mission of the U.S. Securities and Exchange Commission is to protect investors, maintain fair, orderly, and efficient markets, and facilitate capital formation. The SEC oversees the key participants in the securities world, including securities exchanges, securities brokers and dealers, investment advisors, and mutual funds. The SEC is concerned primarily with promoting the disclosure of important market-related information, maintaining fair dealing, and protecting against fraud.

Getting Started

The following snippet demonstrates how to request data from the US SEC Filings dataset:

```py
self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
self.report_8k_symbol = self.add_data(SECReport8K, self.aapl).symbol
self.report_10k_symbol = self.add_data(SECReport10K, self.aapl).symbol
self.report_10q_symbol = self.add_data(SECReport10Q, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1998
Asset Coverage	15,000 US Equities
Data Density	Sparse
Resolution	Daily
Timezone	UTC

Requesting Data

To add US SEC Filings data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```py
class SECReportAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2019, 8, 21)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.report_8k_symbol = self.add_data(SECReport8K, self.aapl).symbol
        self.report_10k_symbol = self.add_data(SECReport10K, self.aapl).symbol
        self.report_10q_symbol = self.add_data(SECReport10Q, self.aapl).symbol
```

Accessing Data

To get the current US SEC Filings data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.report_8k_symbol):
        data_point = slice[self.report_8k_symbol]
        self.log(f"{self.report_8k_symbol} report count at {slice.time}: {len(data_point.report.documents)}")

    if slice.contains_key(self.report_10k_symbol):
        data_point = slice[self.report_10k_symbol]
        self.log(f"{self.report_10k_symbol} report count at {slice.time}: {len(data_point.report.documents)}")

    if slice.ContainsKey(self.report_10q_symbol):
        data_point = slice[self.report_10q_symbol]
        self.log(f"{self.report_10q_symbol} report count at {slice.Time}: {len(data_point.report.documents)}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(SECReport8K).items():
        self.log(f"{dataset_symbol} report count at {slice.time}: {len(data_point.report.documents)}")

    for dataset_symbol, data_point in slice.get(SECReport10K).items():
        self.log(f"{dataset_symbol} report count at {slice.time}: {len(data_point.report.documents)}")

    for dataset_symbol, data_point in slice.get(SECReport10Q).items():
        self.log(f"{dataset_symbol} report count at {slice.time}: {len(data_point.report.documents)}")
```

Historical Data

To get historical US SEC Filings data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrames
report_8k_history_df = self.history(self.report_8k_symbol, 100, Resolution.DAILY)
report_10k_history_df = self.history(self.report_10k_symbol, 100, Resolution.DAILY)
report_10q_history_df = self.history(self.report_10q_symbol, 100, Resolution.DAILY)
history_df = self.history([self.report_8k_symbol,
                           self.report_10k_symbol,
                           self.report_10q_symbol], 100, Resolution.DAILY)

# Dataset objects
report_8k_history_bars = self.history[SECReport8K](self.report_8k_symbol, 100, Resolution.DAILY)
report_10k_history_bars = self.history[SECReport10K](self.report_10k_symbol, 100, Resolution.DAILY)
report_10q_history_bars = self.history[SECReport10Q](self.report_10q_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.report_8k_symbol)
self.remove_security(self.report_10k_symbol)
self.remove_security(self.report_10q_symbol)
```

If you subscribe to US SEC Filings data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The US SEC Filings dataset enables you to create strategies using information from SEC reports. Examples include the following strategies:

- Extracting information about corporate earnings from the documents for further analysis
- Applying sentiment analysis to the text content of the documents (for example, keyword scoring and ranking)

Classic Algorithm Example

The following example algorithm creates a dynamic universe of the 10 most liquid US Equities. It then buys the securities that have an 8K report with more than 20,000 characters and forms an equal-weighted portfolio. Instead of trading based on how long the report is, you could use sentiment analysis by scoring keywords.

PY

```
from AlgorithmImports import *


class SECReport8KAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        self.add_universe(self.coarse_selector)
        self.dataset_symbol_by_symbol = {}
        self.long_symbols = []
        self.rebalance = False

        # Request underlying equity data.
        ibm = self.add_equity("IBM", Resolution.DAILY).symbol
        # Add news data for the underlying IBM asset for trade signal generation
        earnings_filing = self.add_data(SECReport10Q, ibm, Resolution.DAILY).symbol
        # Request 120 days of history with the SECReport10Q IBM custom data Symbol
        history = self.history(SECReport10Q, earnings_filing, 120, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def coarse_selector(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        # Select the top 10 traded companies with fundamental data, since the market activities can better reflect the fundamental factor
efficiently
        coarse = sorted([cf for cf in coarse if cf.has_fundamental_data],
            key=lambda cf: cf.dollar_volume, reverse=True)[:10]
        return [cf.symbol for cf in coarse]

    def on_data(self, slice: Slice) -> None:
        # Trade from SEC data
        for report in slice.Get(SECReport8K).Values:
            underlying_symbol = report.symbol.underlying
            # Skip the Symbol if it's no longer in the universe (insuffucient popularity to reach market efficiency of fundamental
factor)
            if underlying_symbol not in self.dataset_symbol_by_symbol:
                if underlying_symbol in self.long_symbols:
                    self.rebalance = True
                    self.long_symbols.remove(underlying_symbol)
                continue

            # Long the stocks with SEC report over 20000 text length, suggesting excessive insights or future projections are highlighted
            report_text_length = sum([len(i.text) for i in report.report.documents])
            if report_text_length > 20000:
                if underlying_symbol not in self.long_symbols:
                    self.rebalance = True
                    self.long_symbols.append(underlying_symbol)
            elif underlying_symbol in self.long_symbols:
                self.rebalance = True
                self.long_symbols.remove(underlying_symbol)

        if not self.rebalance:
            return
        self.rebalance = False

        # Invest equally to evenly dissipate the capital concentration risk
        portfolio_targets = []
        equal_weighting = 1 / len(self.long_symbols) if len(self.long_symbols) > 0 else 0
        for symbol, security_holding in self.portfolio.items():
            weight = 0
            if symbol in self.long_symbols:
                weight = equal_weighting
            elif not security_holding.invested:
                continue
            portfolio_targets.append(PortfolioTarget(symbol, weight))
        self.set_holdings(portfolio_targets)


    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # If added to universe, adds SECReport8K for trade signal generation
            self.dataset_symbol_by_symbol[security.symbol] = self.add_data(SECReport8K, security.symbol).symbol

        for security in changes.removed_securities:
            # Remove SEC data subscription to release computation resources
            dataset_symbol = self.dataset_symbol_by_symbol.pop(security.symbol, None)
            if dataset_symbol:
                self.remove_security(dataset_symbol)
```

Framework Algorithm Example

The following example algorithm creates a dynamic universe of the 10 most liquid US Equities. It then buys the securities that have an 8K report with more than 20,000 characters and forms an equal-weighted portfolio. Instead of trading based on how long the report is, you could use sentiment analysis by scoring keywords.

```py
from AlgorithmImports import *


class SECReport8KAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        # Filter for the most liquid stocks for trading due to high market activities for efficient pricing from fundamental factor
        self.add_universe_selection(MostLiquidFundamentalUniverseSelectionModel())
        # Custom alpha model that generates insights from SEC data
        self.add_alpha(SECReport8KAlphaModel())
        # Invest equally to evenly dissipate the capital concentration risk
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(lambda time: None))


class MostLiquidFundamentalUniverseSelectionModel(FundamentalUniverseSelectionModel):

    def select(self, algorithm: QCAlgorithm, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select the top 10 traded companies with fundamental data, since the market activities can better reflect the fundamental factor
efficiently
        sorted_by_dollar_volume = sorted(fundamental, key=lambda f: f.dollar_volume, reverse=True)
        return [f.symbol for f in sorted_by_dollar_volume[:10]]


class SECReport8KAlphaModel(AlphaModel):
    dataset_symbol_by_symbol = {}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade only based on SEC data
        for report in slice.get(SECReport8K).values():
            underlying_symbol = report.symbol.underlying
            if underlying_symbol not in self.dataset_symbol_by_symbol:
                continue

            # Long the stocks with SEC report over 20000 text length, suggesting excessive insights or future projections are highlighted
            report_text_length = sum([len(i.text) for i in report.report.documents])

            if report_text_length > 20000:
                direction = InsightDirection.UP
            elif algorithm.portfolio[underlying_symbol].invested:
                direction = InsightDirection.FLAT
            else:
                continue
            insights.append(Insight.price(underlying_symbol, timedelta(days=30), direction))

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # If added to universe, adds SECReport8K for trade signal generation
            self.dataset_symbol_by_symbol[security.symbol] = algorithm.add_data(SECReport8K, security.symbol).symbol

        for security in changes.removed_securities:
            # Remove SEC data subscription to release computation resources
            symbol_data = self.dataset_symbol_by_symbol.pop(security.symbol, None)
            if symbol_data:
                algorithm.remove_security(symbol_data)
```

Data Point Attributes

The US SEC Filings dataset provides `SECReport8K` , `SECReport10K` , and `SECReport10Q` objects.

Report 8K Attributes

`SECReport8K` objects have the following attributes:

Report 10K Attributes

`SECReport10K` objects have the following attributes:

Report 10Q Attributes

`SECReport10Q` objects have the following attributes:

Datasets

Smart Insider

Smart Insider was founded by Michael Tindale in 2016 with the goal of forming the most progressive insider data vendor in the field. Smart Insider provides access to buyback intention and transactions for quantitative researchers. In addition to their Corporate Buybacks dataset, Smart Insider provides data on stock trades made by US politicians and thousands of high net worth individuals around the globe.

Corporate Buybacks

Smart Insider

Corporate Buybacks

Introduction

The Corporate Buybacks dataset by Smart Insider tracks US Equities share buyback programs. The data covers 3,000 US Equities, starts in May 2015, and is delivered on a second frequency. This dataset is created by analyzing daily buyback announcements and by using secondary data sources to ensure records are accurate and complete.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Corporate Buybacks dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Smart Insider was founded by Michael Tindale in 2016 with the goal of forming the most progressive insider data vendor in the field. Smart Insider provides access to buyback intention and transactions for quantitative researchers. In addition to their Corporate Buybacks dataset, Smart Insider provides data on stock trades made by US politicians and thousands of high net worth individuals around the globe.

Getting Started

The following snippet demonstrates how to request data from the Corporate Buybacks dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
self.intention_symbol = self.add_data(SmartInsiderIntention, self.aapl).symbol
self.transaction_symbol = self.add_data(SmartInsiderTransaction, self.aapl).symbol

self.intention_universe = self.add_universe(SmartInsiderIntentionUniverse, self.intention_selection)
self.transaction_universe = self.add_universe(SmartInsiderTransactionUniverse, self.transaction_selection)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	May 2015
Asset Coverage	3,000 US Equities
Data Density	Sparse
Resolution	Second
Timezone	New York

Requesting Data

To add Corporate Buybacks data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class CorporateBuybacksDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2016, 1, 1)
        self.set_end_date(2021, 1, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.intention_symbol = self.add_data(SmartInsiderIntention, self.aapl).symbol
        self.transaction_symbol = self.add_data(SmartInsiderTransaction, self.aapl).symbol
```

Accessing Data

To get the current Corporate Buybacks data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.intention_symbol):
        data_point = slice[self.intention_symbol]
        self.log(f"{self.intention_symbol} intention amount at {slice.time}: {data_point.amount}")

    if slice.contains_key(self.transaction_symbol):
        data_point = slice[self.transaction_symbol]
        self.log(f"{self.transaction_symbol} transaction amount at {slice.time}: {data_point.amount}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, data_point in slice.get(SmartInsiderIntention).items():
        self.log(f"{dataset_symbol} intention amount at {slice.time}: {data_point.amount}")

    for dataset_symbol, data_point in slice.get(SmartInsiderTransaction).items():
        self.log(f"{dataset_symbol} transaction amount at {slice.time}: {data_point.amount}")
```

Historical Data

To get historical Corporate Buybacks data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrames
intention_history_df = self.history(self.intention_symbol, 100, Resolution.DAILY)
transaction_history_df = self.history(self.transaction_symbol, 100, Resolution.DAILY)
history_df = self.history([self.intention_symbol, self.transaction_symbol], 100, Resolution.DAILY)

# Dataset objects
intention_history_bars = self.history[SmartInsiderIntention](self.intention_symbol, 100, Resolution.DAILY)
transaction_history_bars = self.history[SmartInsiderTransaction](self.transaction_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Universe Selection

To select a dynamic universe of US Equities based on Corporate Buybacks data, call the `add_universe` method with the `SmartInsiderIntentionUniverse` class or the `SmartInsiderTransactionUniverse` and a selection function.

```
def initialize(self) -> None:
    self.intention_universe = self.add_universe(SmartInsiderIntentionUniverse, self.intention_selection)
    self.transaction_universe = self.add_universe(SmartInsiderTransactionUniverse, self.transaction_selection)

def intention_selection(self, alt_coarse: List[SmartInsiderIntentionUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
                if d.percentage > 0.005 \
                and d.usd_market_cap > 100000000]

def transaction_selection(self, alt_coarse: List[SmartInsiderTransactionUniverse]) -> List[Symbol]:
    return [d.symbol for d in alt_coarse \
                if d.buyback_percentage > 0.005 \
                and d.usd_market_cap > 100000000]
```

For more information about dynamic universes, see Universes .

Universe History

You can get historical universe data in an algorithm and in the Research Environment.

Historical Universe Data in Algorithms

To get historical universe data in an algorithm, call the `history` method with the `Universe` object and the lookback period. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the SmartInsiderIntentionUniverse attributes:
intention_history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of SmartInsiderIntentionUniverse objects:
intention_universe_history = self.history(self.intention_universe, 30, Resolution.DAILY)
for (_, time), intentions in intention_universe_history.items():
    for intention in intentions:
        self.log(f"{intention.symbol.value} amount value at {intention.end_time}: {intention.amount_value}")

# DataFrame example where the columns are the SmartInsiderTransactionUniverse attributes:
transaction_history_df = self.history(self._universe, 30, Resolution.DAILY, flatten=True)

# Series example where the values are lists of SmartInsiderTransactionUniverse objects:
transaction_universe_history = self.history(self.transaction_universe, 30, Resolution.DAILY)
for (_, time), transactions in transaction_universe_history.items():
    for transaction in transactions:
        self.log(f"{transaction.symbol.value} amount at {intention.end_time}: {intention.amount}")
```

Historical Universe Data in Research

To get historical universe data in research, call the `universe_history` method with the `Universe` object, a start date, and an end date. This method returns the filtered universe. If there is no data in the period you request, the history result is empty.

```
# DataFrame example where the columns are the SmartInsiderIntentionUniverse attributes:
intention_history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of SmartInsiderIntentionUniverse objects:
intention_universe_history = qb.universe_history(intention_universe, qb.time-timedelta(30), qb.time)
for (_, time), intentions in intention_universe_history.items():
    for intention in intentions:
        print(f"{intention.symbol.value} amount value at {intention.end_time}: {intention.amount_value}")

# DataFrame example where the columns are the SmartInsiderTransactionUniverse attributes:
transaction_history_df = qb.universe_history(universe, qb.time-timedelta(30), qb.time, flatten=True)

# Series example where the values are lists of SmartInsiderTransactionUniverse objects:
transaction_universe_history = qb.universe_history(transaction_universe, qb.time-timedelta(30), qb.time)
for (_, time), transactions in transaction_universe_history.items():
    for transaction in transactions:
        print(f"{transaction.symbol.value} amount at {transaction.end_time}: {transaction.amount}")
```

You can call the `history` method in Research.

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.intention_symbol)
self.remove_security(self.transaction_symbol)
```

If you subscribe to Corporate Buybacks data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Corporate Buybacks dataset enables you to design strategies using information on company buyback programs. Examples include the following strategies:

- Buying securities when the company announces an upcoming share buyback on the premise that the reduction in supply (shares outstanding) will drive up the remaining shares' price
- Buying securities when the company executes an upcoming share buyback on the premise that the reduction in supply (shares outstanding) will drive up the remaining shares' price

Classic Algorithm Example

The following example algorithm buys Apple stock when there is a buyback intention or transaction and then it sells the stock 3 days later:

```
from AlgorithmImports import *


class CorporateBuybacksDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol

        # Requesting insider trade intention news and actual trades to estimate the return, since insiders may have better information of
the future confidence
        self.smart_insider_intention = self.add_data(SmartInsiderIntention, self.aapl).symbol
        self.smart_insider_transaction = self.add_data(SmartInsiderTransaction, self.aapl).symbol

        # Historical data
        history = self.history(self.smart_insider_intention, 365, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for intentions")

        history = self.history(self.smart_insider_transaction, 365, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request for transactions")

    def on_data(self, slice: Slice) -> None:
        # Buy Apple whenever we receive a buyback intention or transaction notification, given the insiders may have confidence in the
future to buy more
        # This news may stimulate market popularity
        if slice.contains_key(self.smart_insider_intention) or slice.contains_key(self.smart_insider_transaction):
            self.set_holdings(self.aapl, 1)
            self.entry_time = self.time

        # Liquidate holdings 3 days after the latest entry
        # The market popularity and possible overbrought is cooled
        if self.portfolio.invested and self.time >= self.entry_time + timedelta(days=3):
            self.liquidate()
```

Framework Algorithm Example

The following example algorithm buys Apple stock when there is a buyback intention or transaction and then it sells the stock 3 days later:

```
from AlgorithmImports import *


class CorporateBuybacksDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Only interest in AAPL's insider trades information
        symbols = [Symbol.create("AAPL", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        self.add_alpha(CorporateBuybacksAlphaModel())
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class CorporateBuybacksAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Buy Apple whenever we receive a buyback intention or transaction notification, given the insiders may have confidence in the
future to buy more
        # This news may stimulate market popularity, so we hold for 3 days
        if slice.contains_key(self.smart_insider_intention) or slice.contains_key(self.smart_insider_transaction):
            return [Insight.price(self.aapl, timedelta(days=3), InsightDirection.UP)]
        return []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.aapl = security.symbol

            # Requesting insider trade intention news and actual trades to estimate the return, since insiders may have better
information of the future confidence
            self.smart_insider_intention = algorithm.add_data(SmartInsiderIntention, self.aapl).symbol
            self.smart_insider_transaction = algorithm.add_data(SmartInsiderTransaction, self.aapl).symbol

            # Historical data
            history = algorithm.history(self.smart_insider_intention, 365, Resolution.DAILY)
            algorithm.debug(f"We got {len(history)} items from our history request for intentions")

            history = algorithm.history(self.smart_insider_transaction, 365, Resolution.DAILY)
            algorithm.debug(f"We got {len(history)} items from our history request for transactions")
```

Research Example

The following example lists US Equities with the highest buyback amount.

```
PY
```

```python
from math import isnan
qb = QuantBook()

# Requesting Data
symbol = qb.AddEquity("AAPL").Symbol
intention_symbol = qb.AddData(SmartInsiderIntention, symbol).Symbol
transaction_symbol = qb.AddData(SmartInsiderTransaction, symbol).Symbol

# Historical data
intention_history = qb.History(SmartInsiderIntention, intention_symbol, 300, Resolution.Daily)
for (symbol, time), row in intention_history.iterrows():
    if isnan(row['amountvalue']): continue
    print(f"{symbol} amount value at {time}: {row['amountvalue']}")

transaction_history = qb.History(SmartInsiderTransaction, transaction_symbol, 300, Resolution.Daily)
for (symbol, time), row in transaction_history.iterrows():
    if isnan(row['amount']): continue
    print(f"{symbol} amount at {time}: {row['amount']}")

# Add Universe Selection for SmartInsiderIntention
def IntentionSelection(alt_coarse: List[SmartInsiderIntentionUniverse]) -> List[Symbol]:
    return [d.Symbol for d in sorted([x for x in alt_coarse if x.AmountValue],
        key=lambda x: x.AmountValue, reverse=True)[:10]]

intention_universe  = qb.AddUniverse(SmartInsiderIntentionUniverse, IntentionSelection)

# Historical Universe data
intention_universe_history = qb.UniverseHistory(intention_universe, qb.Time-timedelta(10), qb.Time)
for (_, time), intentions in intention_universe_history.items():
    for intention in intentions:
        print(f"{intention.Symbol.Value} amount value at {intention.EndTime}: {intention.AmountValue}")

# Add Universe Selection for SmartInsiderTransaction
def IntentionSelection(alt_coarse: List[SmartInsiderTransactionUniverse]) -> List[Symbol]:
    return [d.Symbol for d in sorted([x for x in alt_coarse if x.Amount],
        key=lambda x: x.Amount, reverse=True)[:10]]

transaction_universe  = qb.AddUniverse(SmartInsiderTransactionUniverse, IntentionSelection)

# Historical Universe data
transaction_universe_history = qb.UniverseHistory(transaction_universe, qb.Time-timedelta(10), qb.Time)
for (_, time), transactions in transaction_universe_history.items():
    for transaction in transactions:
        print(f"{transaction.Symbol.Value} amount at {transaction.EndTime}: {transaction.Amount}")
```

Data Point Attributes

The Corporate Buybacks dataset provides `SmartInsiderIntention`, `SmartInsiderIntentionUniverse`, `SmartInsiderTransaction`, and `SmartInsiderTransactionUniverse` objects.

SmartInsiderIntention Attributes

`SmartInsiderIntention` objects have the following attributes:

SmartInsiderIntentionUniverse Attributes

`SmartInsiderIntentionUniverse` objects have the following attributes:

SmartInsiderTransaction Attributes

`SmartInsiderTransaction` objects have the following attributes:

SmartInsiderTransactionUniverse Attributes

`SmartInsiderTransactionUniverse` objects have the following attributes:

Datasets

Tiingo

Tiingo was founded by Rishi Singh in 2014. Tiingo goes beyond traditional news sources and focuses on finding rich, quality content written by knowledgeable writers. Their proprietary algorithms scan unstructured, non-traditional news and other information sources while tagging companies, topics, and assets. This refined system is backed by over ten years of research and development, and is written by former institutional quant traders. Because of this dedicated approach, Tiingo's News API is a trusted tool used by quant funds, hedge funds, pension funds, social media companies, and tech companies around the world.

Tiingo News Feed

Tiingo

Tiingo News Feed

Introduction

The Tiingo News Feed dataset by Tiingo tracks US Equity news releases. The data covers 10,000 US Equities, starts in January 2014, and is delivered on a second frequency. This dataset is creating by Tiingo integrating over 120 different news providers into their platform.

This dataset depends on the US Equity Security Master dataset because the US Equity Security Master dataset contains information on splits, dividends, and symbol changes.

For more information about the Tiingo News Feed dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

Tiingo was founded by Rishi Singh in 2014. Tiingo goes beyond traditional news sources and focuses on finding rich, quality content written by knowledgeable writers. Their proprietary algorithms scan unstructured, non-traditional news and other information sources while tagging companies, topics, and assets. This refined system is backed by over ten years of research and development, and is written by former institutional quant traders. Because of this dedicated approach, Tiingo's News API is a trusted tool used by quant funds, hedge funds, pension funds, social media companies, and tech companies around the world.

Getting Started

The following snippet demonstrates how to request data from the Tiingo News Feed dataset:

```
self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
self.dataset_symbol = self.add_data(TiingoNews, self.aapl).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2014
Asset Coverage	10,000 US Equities
Data Density	Sparse
Resolution	Second
Timezone	UTC

Requesting Data

To add Tiingo News Feed data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class TiingoNewsDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2021, 1, 1)
        self.set_end_date(2021, 6, 1)
        self.set_cash(100000)

        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.dataset_symbol = self.add_data(TiingoNews, self.aapl).symbol
```

Accessing Data

To get the current Tiingo News Feed data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
PY
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        article = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} article description at {slice.time}: {article.description}")
```

To iterate through all of the articles in the current `Slice` , call the `get` method.

```
PY
def on_data(self, slice: Slice) -> None:
    for dataset_symbol, article in slice.get(TiingoNews).items():
        self.log(f"{dataset_symbol} article description at {slice.time}: {article.description}")
```

Historical Data

To get historical Tiingo News Feed data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
PY
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
self.history[TiingoNews](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
PY
self.remove_security(self.dataset_symbol)
```

If you subscribe to Tiingo News Feed data for assets in a dynamic universe, remove the dataset subscription when the asset leaves your universe. To view a common design pattern, see Track Security Changes .

Example Applications

The Tiingo News Feed enables you to accurately design strategies harnessing news articles on the companies you're trading. Examples include the following strategies:

- Creating a dictionary of sentiment scores for various words and assigning a sentiment score to the content of each news release
- Calculating the sentiment of news releases with Natural Language Processing (NLP)
- Trading securities when their news releases are tagged by Tiingo with current buzzwords
- Detecting impactful news in ETF constituents

Classic Algorithm Example

The following example algorithm assigns a sentiment score to each news article that's released for Apple. When the sentiment score is positive, the algorithm buys Apple stock. When the sentiment score is negative, it short sells Apple stock.

```py
from AlgorithmImports import *


class TiingoNewsDataAlgorithm(QCAlgorithm):

    current_holdings = 0
    target_holdings = 0
    # Custom word-score map to assign score for each word in article
    word_scores = {'good': 1, 'great': 1, 'best': 1, 'growth': 1,
                   'bad': -1, 'terrible': -1, 'worst': -1, 'loss': -1}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Requesting Tiingo news data to obtain the updated news articles to calculate the sentiment score
        self.aapl = self.add_equity("AAPL", Resolution.MINUTE).symbol
        self.tiingo_symbol = self.add_data(TiingoNews, self.aapl).symbol

        # Historical data
        history = self.history(self.tiingo_symbol, 14, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

    def on_data(self, slice: Slice) -> None:
        if slice.contains_key(self.tiingo_symbol):
            # Assign a sentiment score to the news article by the word-score map
            title_words = slice[self.tiingo_symbol].description.lower()
            score = 0
            for word, word_score in self.word_scores.items():
                if word in title_words:
                    score += word_score

            # Buy if aggregated sentiment score shows positive sentiment, sell vice versa
            if score > 0:
                self.target_holdings = 1
            elif score < 0:
                self.target_holdings = -1

        # Buy or short sell if the sentiment has changed from our current holdings
        if slice.contains_key(self.aapl) and self.current_holdings != self.target_holdings:
            self.set_holdings(self.aapl, self.target_holdings)
            self.current_holdings = self.target_holdings
```

Framework Algorithm Example

The following example algorithm assigns a sentiment score to each news article that's released for Apple. When the sentiment score is positive, the algorithm buys Apple stock. When the sentiment score is negative, it short sells Apple stock. The algorithm holds positions for 14 days.

```py
from AlgorithmImports import *


class TiingoNewsDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        symbols = [Symbol.create("AAPL", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        self.add_alpha(TiingoNewsAlphaModel())
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class TiingoNewsAlphaModel(AlphaModel):

    current_holdings = 0
    target_holdings = 0
    # Custom word-score map to assign score for each word in article
    word_scores = {'good': 1, 'great': 1, 'best': 1, 'growth': 1,
                   'bad': -1, 'terrible': -1, 'worst': -1, 'loss': -1}

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        if slice.contains_key(self.tiingo_symbol):
            # Assign a sentiment score to the news article by the word-score map
            title_words = slice[self.tiingo_symbol].description.lower()
            score = 0
            for word, word_score in self.word_scores.items():
                if word in title_words:
                    score += word_score

            # Buy if aggregated sentiment score shows positive sentiment, sell vice versa
            if score > 0:
                self.target_holdings = 1
            elif score < 0:
                self.target_holdings = -1

        # Buy or short sell if the sentiment has changed from our current holdings
        if slice.contains_key(self.aapl) and self.current_holdings != self.target_holdings:
            self.current_holdings = self.target_holdings
            direction = InsightDirection.UP if self.target_holdings == 1 else InsightDirection.DOWN
            return [Insight.price(self.aapl, timedelta(days=14), direction)]

        return []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.aapl = security.symbol

            # Requesting Tiingo news data to obtain the updated news articles to calculate the sentiment score
            self.tiingo_symbol = algorithm.add_data(TiingoNews, self.aapl).symbol

            # Historical data
            history = algorithm.history(self.tiingo_symbol, 14, Resolution.DAILY)
            algorithm.debug(f"We got {len(history)} items from our history request")
```

Data Point Attributes

The Tiingo News Feed dataset provides `TiingoNews` objects, which have the following attributes:

Datasets

Treasury Department

The Treasury Department is the executive agency responsible for promoting economic prosperity and ensuring the financial security of the United States. The Department is responsible for a wide range of activities such as advising the President on economic and financial issues, encouraging sustainable economic growth, and fostering improved governance in financial institutions. The Department of the Treasury operates and maintains systems that are critical to the nation's financial infrastructure, such as the production of coin and currency, the disbursement of payments to the American public, revenue collection, and the borrowing of funds necessary to run the federal government.

US Treasury Yield Curve

Treasury Department

US Treasury Yield Curve

Introduction

The US Treasury Yield Curve datasets tracks the yield curve rate from the US Department of the Treasury. The data starts in January 1990 and is delivered on a daily frequency. This dataset is calculated from composites of indicative, bid-side market quotations (not actual transactions) obtained by the Federal Reserve Bank of New York at or near 3:30 PM Eastern Time (ET) each trading day.

For more information about the US Treasury Yield Curve dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The Treasury Department is the executive agency responsible for promoting economic prosperity and ensuring the financial security of the United States. The Department is responsible for a wide range of activities such as advising the President on economic and financial issues, encouraging sustainable economic growth, and fostering improved governance in financial institutions. The Department of the Treasury operates and maintains systems that are critical to the nation's financial infrastructure, such as the production of coin and currency, the disbursement of payments to the American public, revenue collection, and the borrowing of funds necessary to run the federal government.

Getting Started

The following snippet demonstrates how to request data from the US Treasury Yield Curve dataset:

```py
self.dataset_symbol = self.add_data(USTreasuryYieldCurveRate, "USTYCR").symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 1990
Coverage	1 Dataset
Data Density	Sparse
Resolution	Daily
Timezone	New York

Requesting Data

To add US Treasury Yield Curve data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```py
class QuiverCongressDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2019, 1, 1)
        self.set_end_date(2020, 6, 1)
        self.set_cash(100000)

        self.dataset_symbol = self.add_data(USTreasuryYieldCurveRate, "USTYCR").symbol
```

Accessing Data

To get the current US Treasury Yield Curve data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the

data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} one month value at {slice.time}: {data_point.one_month}")
```

Historical Data

To get historical US Treasury Yield Curve data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[USTreasuryYieldCurveRate](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove your subscription to US Treasury Yield Curve data, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The US Treasury Yield Curve dataset enables you to monitor the yields of bonds with numerous maturities in your strategies. Examples include the following strategies:

- Short selling SPY when the yield curve inverts
- Buying short-term Treasuries and short selling long-term Treasuries when the yield curve becomes steeper (aka curve steepener trade)

Classic Algorithm Example

The following example algorithm short sells SPY for two years when the yield curve inverts:

```py
from AlgorithmImports import *


class USTreasuryDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:

        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request SPY as the market representative for trading
        self.spy_symbol = self.add_equity("SPY", Resolution.HOUR).symbol

        # Requesting yield curve data for trade signal generation (inversion)
        self.yield_curve_symbol = self.add_data(USTreasuryYieldCurveRate, "USTYCR").symbol

        # Historical data
        history = self.history(USTreasuryYieldCurveRate, self.yield_curve_symbol, 60, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from our history request")

        self.last_inversion = datetime.min

    def on_data(self, slice: Slice) -> None:
        # Trade only based on updated yield curve data
        if not slice.contains_key(self.yield_curve_symbol):
            return
        rates = slice[self.yield_curve_symbol]

        # Only advance if a year has gone by, since the inversion signal indicates longer term market regime that will not revert in a
short period
        if (self.time - self.last_inversion < timedelta(days=365)):
            return

        # Normally, 10y yield should be greater than 2y yield due to default risk accumulation
        # But if an inversion occurs, it means the market expects a recession in short term such that the near-expiry bond is more likely
to default
        # if there is a yield curve inversion after not having one for a year, short sell SPY for two years for the expected down market
        if (not self.portfolio.invested and rates.two_year > rates.ten_year):
            self.debug(f"{self.time} - Yield curve inversion! Shorting the market for two years")
            self.set_holdings(self.spy_symbol, -0.5)
            self.last_inversion = self.time
            return

        # If two years have passed, liquidate our position in SPY assuming the market starts resilience
        if (self.time - self.last_inversion >= timedelta(days=365 * 2)):
            self.liquidate(self.spy_symbol)
```

Framework Algorithm Example

The following example algorithm short sells SPY for two years when the yield curve inverts:

```py
from AlgorithmImports import *


class USTreasuryDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.resolution = Resolution.HOUR
        # Universe only have SPY as the market representative for trading
        symbols = [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Custom alpha model that emit insight according to US Treasury data
        self.add_alpha(USTreasuryAlphaModel(self))
        # Use insight weighting PCM to limit the size of investment, reduce the fluctuation of the portfolio value
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel(lambda time: None))


class USTreasuryAlphaModel(AlphaModel):

    spy_symbol = None
    last_inversion = datetime.min

    def __init__(self, algorithm: QCAlgorithm) -> None:
        # Requesting yield curve data for trade signal generation (inversion)
        self.yield_curve_symbol = algorithm.add_data(USTreasuryYieldCurveRate, "USTYCR").symbol

        # Historical data
        history = algorithm.history(self.yield_curve_symbol, 60, Resolution.DAILY)
        algorithm.debug(f"We got {len(history)} items from our history request")

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        # Trade only based on updated yield curve data
        if not (slice.contains_key(self.yield_curve_symbol) and self.spy_symbol is not None):
            return []
        rates = slice[self.yield_curve_symbol]

        # Only advance if a year has gone by, since the inversion signal indicates longer term market regime that will not revert in a
short period
        if (slice.time - self.last_inversion < timedelta(days=365)):
            return []

        # Normally, 10y yield should be greater than 2y yield due to default risk accumulation
        # But if an inversion occurs, it means the market expects a recession in short term such that the near-expiry bond is more likely
to default
        # if there is a yield curve inversion after not having one for a year, short sell SPY for two years for the expected down market
        if (not algorithm.portfolio.invested and rates.two_year > rates.ten_year):
            algorithm.debug(f"{slice.time} - Yield curve inversion! Shorting the market for two years")
            self.last_inversion = slice.time
            return [Insight.price(self.spy_symbol, slice.time + timedelta(days=2*365), InsightDirection.DOWN, None, None, None, 0.5)]
        return []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            self.spy_symbol = security.symbol
```

Data Point Attributes

The US Treasury Yield Curve dataset provides `USTreasuryYieldCurveRate` objects, which have the following attributes:

Datasets

US Department of Agriculture

The Economic Research Service (ERS) is a federal statistical agency of the U.S. Department of Agriculture (USDA) dedicated to conducting objective economic research that informs public and private decision-making across agriculture, food, the environment, and rural America. ERS serves policymakers at all levels of government, Congress, and industry groups by providing high-quality analysis on topics ranging from farm sector performance and food security to global trade and rural economic development. The USDA Economic Research Service (ERS) publishes retail price data for fruits and vegetables across multiple forms: Fresh, Canned, Frozen, Dried, and Juice.

USDA Fruit And Vegetables

US Department of Agriculture

USDA Fruit And Vegetables

Introduction

The USDA Fruit And Vegetables dataset by the U.S. Department of Agriculture (USDA) tracks retail price estimates for 75+ commonly consumed fresh and processed fruits and vegetables. Includes price per pound/pint and normalized price per edible cup equivalent.

For more information about the USDA Fruit And Vegetables dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

The Economic Research Service (ERS) is a federal statistical agency of the U.S. Department of Agriculture (USDA) dedicated to conducting objective economic research that informs public and private decision-making across agriculture, food, the environment, and rural America. ERS serves policymakers at all levels of government, Congress, and industry groups by providing high-quality analysis on topics ranging from farm sector performance and food security to global trade and rural economic development. The USDA Economic Research Service (ERS) publishes retail price data for fruits and vegetables across multiple forms: Fresh, Canned, Frozen, Dried, and Juice.

Getting Started

The following snippet demonstrates how to request data from the USDA Fruit and Vegetables dataset:

```
self.dataset_symbol = self.add_data(USDAFruitAndVegetables,
    USDAFruitAndVegetable.Symbols.Apples, Resolution.DAILY)
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	January 2013
Asset Coverage	75 Fruits and Vegetables
Resolution	Annual
Data Density	Sparse
Timezone	New York

Requesting Data

To add USDA Fruit And Vegetables data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class USDAFruitAndVegetablesAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2013, 1, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.dataset_symbol = self.add_data(USDAFruitAndVegetables, USDAFruitAndVegetable.Symbols.Apples, Resolution.DAILY)
```

Accessing Data

To get the current USDA Fruit And Vegetables data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        collection = slice[self.dataset_symbol]
        for data in collection.data:
            self.log(f"{symbol} [{data.form}]: ${data.price_per_cup_equivalent}")
```

To iterate through all of the dataset objects in the current `Slice` , call the `get` method.

```
def on_data(self, slice: Slice) -> None:
    for symbol, collection in slice.get(USDAFruitAndVegetables).items():
        # Iterate over all forms in the collection
        for data in collection.data:
            self.log(f"{symbol} [{data.form}]: ${data.price_per_cup_equivalent}")
```

Historical Data

To get historical USDA Fruit And Vegetables data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
history_df = self.history(self.dataset_symbol, datetime(2013, 1, 1), self.time, Resolution.DAILY)
```

Remove Subscriptions

To remove a subscription, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Supported Assets

The USDA Fruit And Vegetable dataset lists data for the following agricultural products:

Example Applications

The USDA Fruit and Vegetable dataset provides insights into the agricultural sector. Examples include the following strategies:

- Compare the prices across sub-products (fresh vs frozen)
- Compare the price trends across products (strawberry vs orange)
- Understand how the agricultural market correlates with the stock market

Classic Algorithm Example

The following example algorithm compares the price difference between fresh and frozen strawberries:

```py
from AlgorithmImports import *

class FreshVsFrozenAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2018, 1, 1)
        self.set_end_date(2023, 12, 31)

        # Single subscription gets both Fresh and Frozen forms
        self.strawberries = self.add_data(USDAFruitAndVegetables, USDAFruitAndVegetable.Symbols.Strawberries).symbol

        history = self.history(USDAFruitAndVegetables, self.strawberries, datetime(2013,1,1), self.time, flatten=True)

    def on_data(self, slice):
        if slice.contains_key(self.strawberries):
            collection = slice[self.strawberries]

            # Find Fresh and Frozen forms in the collection
            fresh_price = None
            frozen_price = None

            for data in collection.data:
                if data.form == "Fresh":
                    fresh_price = data.value
                elif data.form == "Frozen":
                    frozen_price = data.value

            if fresh_price and frozen_price:
                diff_pct = ((fresh_price / frozen_price) - 1) * 100
                self.log(f"{self.time.year}: Fresh ${fresh_price:.2f} vs Frozen ${frozen_price:.2f} ({diff_pct:+.1f}%)")
```

Data Point Attributes

The USDA Fruit And Vegetable dataset provides `USDAFruitAndVegetables` and `USDAFruitAndVegetable` objects, which have the following attributes:

USDAFruitAndVegetables Attributes

USDAFruitAndVegetable Attributes

Datasets

VIX Central

VIX Central was founded by Eli Mintz in 2012 with goal of displaying historical VIX term structures in a simple and intuitive interface. VIX Central provides access to real-time and historical VIX data for individual investors.

VIX Central Contango

VIX Central

VIX Central Contango

Introduction

The VIX Central Contango dataset by VIX Central tracks VIX Futures (VX) contango data. The data covers 12 Futures contracts closest to expiry/maturity, starts in June 2010, and is delivered on a daily frequency. The dataset is created by QuantConnect downloading data from VIX Central website, which collects and analyses VIX and VX (VIX Futures) data.

Contango and Backwardation are terms used to describe if participants in the Futures market are overpaying or underpaying relative to the "spot" price of the underlying commodity when trading a Futures contract ("spot" price is the price of the actual commodity/asset at a given moment in time). Contango and backwardation can be used to determine forward-looking expectations of the commodity's spot price by the time the Future has expired/matured and is set to be delivered by participants of the Futures market. As Futures near their expiration/maturity date, contango and backwardation curves tend to converge on the spot price of the commodity at the time of expiration.

For more information about the VIX Central Contango dataset, including CLI commands and pricing, see the dataset listing .

About the Provider

VIX Central was founded by Eli Mintz in 2012 with goal of displaying historical VIX term structures in a simple and intuitive interface. VIX Central provides access to real-time and historical VIX data for individual investors.

Getting Started

The following snippet demonstrates how to request data from the VIX Central Contango dataset:

```
self.dataset_symbol = self.add_data(VIXCentralContango, "VIX", Resolution.DAILY).symbol
```

Data Summary

The following table describes the dataset properties:

Property	Value
Start Date	June 2010
Asset Coverage	1 Futures Chain with 12 contracts
Data Density	Regular
Resolution	Daily
Timezone	New York

Requesting Data

To add VIX Central Contango data to your algorithm, call the `add_data` method. Save a reference to the dataset `Symbol` so you can access the data later in your algorithm.

```
class VixCentralContangoAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2014, 1, 1)
        self.set_end_date(2018, 1, 1)
        self.set_cash(25000)

        self.dataset_symbol = self.add_data(VIXCentralContango, "VX", Resolution.DAILY).symbol
```

Accessing Data

To get the current VIX Central Contango data, index the current `Slice` with the dataset `Symbol` . Slice objects deliver unique events to your algorithm as they happen, but the `Slice` may not contain data for your dataset at every time step. To avoid issues, check if the `Slice` contains the data you want before you index it.

```
def on_data(self, slice: Slice) -> None:
    if slice.contains_key(self.dataset_symbol):
        data_point = slice[self.dataset_symbol]
        self.log(f"{self.dataset_symbol} front month at {slice.time}: {data_point.front_month}")
```

Historical Data

To get historical VIX Central Contango data, call the `history` method with the dataset `Symbol` . If there is no data in the period you request, the history result is empty.

```
# DataFrame
history_df = self.history(self.dataset_symbol, 100, Resolution.DAILY)

# Dataset objects
history_bars = self.history[VIXCentralContango](self.dataset_symbol, 100, Resolution.DAILY)
```

For more information about historical data, see History Requests .

Remove Subscriptions

To remove your subscription to VIX Central Contango data, call the `remove_security` method.

```
self.remove_security(self.dataset_symbol)
```

Example Applications

The VIX Central Contango dataset enables you to explore VIX Future contracts pricing data. Examples include the following strategies:

- Determining forward-looking expectations by Futures market participants of the underlying commodity's spot price by the time of expiration/maturity
- Creating cash-and-carry arbitrage strategies by trading the spread/convergence of the contango/backwardation curves as a Future nears expiration/maturity

Classic Algorithm Design

The following example algorithm buys SPY when the percentage change between contract F2 and F1 (`Contango_F2_Minus_F1`) is positive. Otherwise, it remains in cash.

```py
from AlgorithmImports import *


class VixCentralContangoAlgorithm (QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(25000)
        # SPY as market representative for macroeconomy indicator trading
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol
        # Request VIX Contango data for trade signal generation
        self.contango = self.add_data(VIXCentralContango, "VX", Resolution.DAILY).symbol

    def on_data(self, slice: Slice) -> None:
        # Trade base on the updated VIX contango data
        if self.contango not in slice:
            return
        contango_data = slice.get(VIXCentralContango, self.contango)
        ratio = contango_data.contango_f_2_minus_f_1 if contango_data else 0

        # Bet the market will go up if F2 - F1 price percentage change is positive, meaning the near-term market volatility is expected
to be low
        if not self.portfolio.invested and ratio > 0:
            self.set_holdings(self.spy, 1)
        # Liqudiate otherwise to avoid excessive market volatility
        elif ratio < 0:
            self.liquidate()
```

Framework Algorithm Design

The following example algorithm buys SPY when the percentage change between contract F2 and F1 (`Contango_F2_Minus_F1`) is positive. Otherwise, it remains in cash.

```py
from AlgorithmImports import *


class VixCentralContangoAlgorithm (QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(25000)

        self.universe_settings.resolution = Resolution.DAILY
        # Include SPY as market representative for macroeconomy indicator trading
        symbols = [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Custom alpha model that emit insights base on VIX contango data
        self.add_alpha(ContangoAlphaModel(self))
        # Equal weighting investment to evenly dissipate the capital concentration risk on non-systematic risky event
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class ContangoAlphaModel(AlphaModel):

    def __init__(self, algorithm: QCAlgorithm) -> None:
        self.symbols = []
        # Request VIX Contango data for trade signal generation
        self.contango = algorithm.add_data(VIXCentralContango, "VX", Resolution.DAILY).symbol

        # Historical data
        history = algorithm.history(VIXCentralContango, self.contango, 60, Resolution.DAILY)
        algorithm.debug(f"We got {len(history.index)} items from our history request")

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Trade base on the updated VIX contango data
        if self.contango not in slice:
            return []
        contango_data = slice.get(VIXCentralContango, self.contango)
        ratio = contango_data.contango_f_2_minus_f_1 if contango_data else 0

        # Bet the market will go up if F2 - F1 price percentage change is positive, meaning the near-term market volatility is expected
to be low
        if not algorithm.portfolio.invested and ratio > 0:
            for symbol in self.symbols:
                insights += [Insight.price(symbol, Expiry.ONE_MONTH, InsightDirection.UP)]
        # Liqudiate otherwise to avoid excessive market volatility
        elif ratio < 0:
            for symbol in self.symbols:
                insights += [Insight.price(symbol, Expiry.ONE_MONTH, InsightDirection.FLAT)]

        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for symbol in [x.symbol for x in changes.removed_securities]:
            if symbol in self.symbols:
                self.symbols.pop(symbol)

        self.symbols += [x.symbol for x in changes.added_securities]
```

Data Point Attributes

The VIX Central Contango dataset provides `VIXCentralContango` objects, which have the following attributes:

Importing Data

Importing Data

Key Concepts

Introduction

Custom data is your own external data that's not from the Dataset Market. You can use custom data to inform trading decisions and to simulate trades on unsupported securities. To get custom data into your algorithms, you download the entire file at once or read it line-by-line with a custom data reader. If you use a custom data reader, LEAN sends the data to the `on_data` method in your algorithm.

File Providers

The most common file providers to use are the Object Store , Dropbox, GitHub, and Google Sheets.

Object Store

The Object Store is the fastest file provider. If you import files from remote providers, you will be restricted by their rate limits and your download speed.

Dropbox

If you store your custom data in Dropbox, you need to create a link to the file and add `?dl=1` to the end of the file URL. To create file links, see How to share files or folders in the Dropbox documentation. The `?dl=1` parameter lets you download the direct file link, not the HTML page of the file.

GitHub

If you store your custom data in GitHub, you can use public or private repositories. When you download the data, use the raw file (for example, https://raw.githubusercontent.com/<organization>/<repo>/<path>). For instructions on accessing the raw file, see Viewing or copying the raw file content in the GitHub documentation.

Google Sheets

If you store your custom data in Google Sheets, you need to create a link to the file. To create file links, see Make Google Docs, Sheets, Slides & Forms public in the Google Docs documentation. Choose the CSV format to find `/pub?gid={SHEET_ID}&single=true&output=csv` parameter on the end of the link. Google Sheets don't support exporting data in JSON format.

Stream Custom Data

To receive your custom data in the `on_data` method, create a custom type and then create a data subscription. The custom data type tells LEAN where to get your data and how to read it.

All custom data types must extend the `PythonData` class and override the `get_source` and `reader` methods

```py
class MyCustomDataType(PythonData):
    def get_source(self,
         config: SubscriptionDataConfig,
         date: datetime,
         is_live: bool) -> SubscriptionDataSource:
        return SubscriptionDataSource("<sourceURL>", SubscriptionTransportMedium.REMOTE_FILE)

    def reader(self,
         config: SubscriptionDataConfig,
         line: str,
         date: datetime,
         is_live: bool) -> BaseData:

         if not (line.strip() and line[0].isdigit()):
            return None

         data = line.split(',')

        custom = MyCustomDataType()
        custom.time = datetime.strptime(data[0], '%Y%m%d')
        custom.end_time = custom.time + timedelta(1)
        custom.value = float(data[1])
        custom["Property1"] = float(data[2])
        return custom
```

For more information about custom data types, see Streaming Data .

Download Bulk Data

The `download` method downloads the content served from a local file or URL and then returns it as a string.

```py
content = self.download("<filePathOrURL>")
```

For more information about bulk downloads, see Bulk Downloads .

File Quotas

There are no limits to the number of files you can load from the Object Store during a single backtest or Research Environment session in QuantConnect Cloud.

The following table shows the number of remote files you can download during a single backtest or Research Environment session in QuantConnect Cloud:

Organization Tier	File Quota
Free	25
Quant Researcher	100
Team	250
Trading Firm	1,000
Institutional	Unlimited

Remote files can be up to 200 MB in size and can have names up to 200 characters long.

Rate Limits

We do not impose a rate limit on file downloads but often external providers do. Dropbox caps download speeds to 10 kb/s after 3-4 download requests. To ensure your algorithms run fast, only use a small number of small custom data files or use the Object Store.

Timeouts

In cloud algorithms, the download methods have a 10-second timeout period. If the methods don't download the data within 10 seconds, LEAN throws an error.

Importing Data

Streaming Data

Streaming Data

Key Concepts

Introduction

There are two techniques to import data into your algorithm. You can either manually import the entire file or stream the file line-by-line into your algorithm's `on_data` event. This page explores streaming a file's contents into your algorithm line-by-line. The data you import can be from a remote server or the Object Store .

Data Formats

Common data formats are CSV , JSON , XML , and ZIP but you can use any file type that can be read over the internet. For Excel files, double check the raw data format for parsing in the data reader, since data will be formatted for convenient visualization in Excel application view. To avoid confusion of data format, save the spreadsheet as a CSV file and open it in a text editor to confirm the raw data format.

If you import from a remote file provider , each request has a one-second overhead, so package your custom data to minimize requests. Bundle dates together where possible to speed up execution. The Object Store file provider gives you the fastest execution because you don't need to download the files on every execution.

Point-In-Time Tickers

Tickers are a string shortcode representation for an asset. Some examples of popular tickers include "AAPL" for Apple Corporation and "IBM" for International Business Machines Corporation. These tickers often change when the company rebrands or they undergo a merger or reverse merger.

The ticker of an asset is not the same as the `Symbol` . `Symbol` objects are permanent and track the underlying entity. When a company rebrands or changes its name, the `Symbol` object remains constant, giving algorithms a way to reliably track assets over time.

Tickers are also often reused by different brokerages. For example Coinbase, a leading US Crypto Brokerage lists the "BTCUSD" ticker for trading. Its competitor, Bitfinex, also lists "BTCUSD". You can trade both tickers with LEAN. `Symbol` objects allow LEAN to identify which market you reference in your algorithms.

To create a `Symbol` object for a point-in-time ticker, call the `generate_equity` method to create the security identifier and then call the `Symbol` constructor. For example, Heliogen, Inc. changed their ticker from ATHN to HLGN on December 31, 2021. To convert the ATHN ticker to the Equity `Symbol` , type:

```
# Generate an Equity security identifier for a ticker and then create a symbol with it.
ticker = "ATHN"
security_id = SecurityIdentifier.generate_equity(ticker, Market.USA, mapping_resolve_date=datetime(2021, 12, 1))
symbol = Symbol(security_id, ticker)
```

In the preceding code snippet, the `mapping_resolve_date` must be a date when the point-in-time ticker was trading.

For examples of point-in-time tickers in custom data types, refer to the CSV Format Example and JSON Format Example .

Set Data Sources

The `get_source` method in your custom data class instructs LEAN where to find the data.

```
class MyCustomDataType(PythonData):
    def get_source(self,
         config: SubscriptionDataConfig,
         date: datetime,
         is_live_mode: bool) -> SubscriptionDataSource:

         if is_live_mode:
            return SubscriptionDataSource("https://www.bitstamp.net/api/ticker/", SubscriptionTransportMedium.REST)

        source = f"http://my-ftp-server.com/{config.symbol.value}/{date:%Y%M%d}.csv"
        return SubscriptionDataSource(source, SubscriptionTransportMedium.REMOTE_FILE)

        # Example of loading from the Object Store:
        # return SubscriptionDataSource(Bitstamp.KEY, SubscriptionTransportMedium.OBJECT_STORE)

        # Example of loading a remote zip file:
        # return SubscriptionDataSource(
        #     "https://cdn.quantconnect.com/uploads/multi_csv_zipped_file.zip",
        #     SubscriptionTransportMedium.REMOTE_FILE,
        #     FileFormat.ZIP_ENTRY_NAME
        # )

        # Example of loading a remote zip file and accessing a CSV file inside it:
        # return SubscriptionDataSource(
        #     "https://cdn.quantconnect.com/uploads/multi_csv_zipped_file.zip#csv_file_10.csv",
        #     SubscriptionTransportMedium.REMOTE_FILE,
        #     FileFormat.ZIP_ENTRY_NAME
        # )

        # Example of loading a remote file with Basic authentication headers:
        # import base64
        # user = "username"
        # password = "12345"
        # byte_key = base64.b64encode(f"{user}:{password}".encode("ascii"))
        # authorization_headers = {
        #     "Authorization": f"Basic {byte_key.decode('ascii')}"
        # }
        # return SubscriptionDataSource(source, SubscriptionTransportMedium.REMOTE_FILE, FileFormat.CSV, authorization_headers)
```

The following table describes the arguments the `get_source` method accepts:

Argument	Data Type	Description
`config`	`SubscriptionDataConfig`	The subscription configuration
`date`	`datetime`	Date of this source file
`is_live_mode`	`bool`	`True` if algorithm is running in live mode

You can use these arguments to create `SubscriptionDataSource` objects representing different locations and formats. The following table describes the arguments the `SubscriptionDataSource` accepts:

Argument	Data Type	Description	Default Value
`source`	`str`	Data source location	
`transport_medium`	`SubscriptionTransportMedium`	The transport medium to be used to retrieve data from the source	
`format`	`FileFormat`	The format of the data within the source	`FileFormat.Csv`
`headers`	`dict[str, str]`	The headers to be used for this source. In cloud algorithms, each of the key-value pairs can consist of up to 1,000 characters.	`None`

The `FileFormat` enumeration has the following members:

The `SubscriptionTransportMedium` enumeration has the following members:

Member	Description	Example
`LOCAL_FILE`	The data comes from disk	Lean.DataSource.CBOE
`REMOTE_FILE`	The data is downloaded from a remote source	`Custom Securities Examples`
`REST`	The data comes from a rest call that is polled and returns a single line/data point of information	`live_mode` case of Demonstration Algorithm
`OBJECT_STORE`	The data comes from the object store	Example of Custom Data

Parse Custom Data

The `reader` method of your custom data class takes one line of data from the source location and parses it into one of your custom objects. You can add as many properties to your custom data objects as you need, but the following table describes the properties you must set. When there is no useable data in a line, the method should return `None` . LEAN repeatedly calls the `reader` method until the date/time advances or it reaches the end of the file.

Property	Description
`symbol`	You can set this property to `config.symbol` .
`time`	The time when the data sample starts.
`end_time`	The time when the data sample ends and when LEAN should add the sample to a Slice .
`value`	The default data point value (`float`).

The following table describes the arguments the `reader` method accepts:

Argument	Data Type	Description
`config`	`SubscriptionDataConfig`	The subscription configuration
`line`	`str`	Content from the requested data source
`date`	`datetime`	Date of this source file
`is_live_mode`	`bool`	`True` if algorithm is running in live mode

You can use these arguments to create `BaseData` objects from different sources.

```py
class MyCustomDataType(PythonData):
    def reader(self,
         config: SubscriptionDataConfig,
         line: str,
         date: datetime,
         is_live_mode: bool) -> BaseData:

        if not line.strip():
            return None

        custom = MyCustomDataType()
        custom.symbol = config.symbol

        if is_live_mode:
            data = json.loads(line)
            custom.end_time =  Extensions.convert_from_utc(datetime.utcnow(), config.exchange_time_zone)
            custom.value = data["value"]
            return custom

        if not line[0].isdigit():
            return None

        data = line.split(',')
        custom.end_time = datetime.strptime(data[0], '%Y%m%d') + timedelta(1)
        custom.value = float(data[1])
        return custom
```

Unsorted Data

By default, LEAN expects the data in chronological order. If the data is unsorted, set the `sort` property of the `SubscriptionDataSource` object to `True`.

```py
class MyCustomDataType(PythonData):
    def get_source(self,
            config: SubscriptionDataConfig,
            date: datetime,
            is_live_mode: bool) -> SubscriptionDataSource:

        source = f"http://my-ftp-server.com/{config.symbol.value}/{date:%Y%M%d}.csv"
        subscription = SubscriptionDataSource(source, SubscriptionTransportMedium.REMOTE_FILE)
        subscription.sort = True
        return subscription
```

LEAN uses `end_time` property of your custom data to sort it.

Object Store

The Object Store file provider gives you the fastest execution because you don't need to download the files on every execution. To upload the data files into the Object Store, use the Algorithm Lab , CLI , or Research Environment . To then pull the data from the Object Store into an algorithm, set the custom data source to read from the file in Object Store and parse the data in the `reader` method.

```py
class MyCustomDataType(PythonData):
    def get_source(self,
            config: SubscriptionDataConfig,
            date: datetime,
            is_live_mode: bool) -> SubscriptionDataSource:

        return SubscriptionDataSource("<your_csv_key>", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.CSV)
        # return new SubscriptionDataSource("<your_json_key>", SubscriptionTransportMedium.ObjectStore, FileFormat.UNFOLDING_COLLECTION);
```

Set Properties

To set the Symbol Properties of the custom data, provide a `SymbolProperties` object when you subscribe to the dataset. The ticker you pass to the `SymbolProperties` constructor and the `add_data` method must be the same.

To set the Exchange Hours of the custom data, provide a `SecurityExchangeHours` object when you subscribe to the dataset. The default hours are for the market to be open 24/7.

```
ticker = "ABC"
properties = SymbolProperties("description", "USD", 1, 0.01, 0.01, ticker)
hours = MarketHoursDatabase.get_entry(Market.USA, "SPY", SecurityType.EQUITY).exchange_hours
self.add_data(CustomData, ticker, properties, hours)
```

PY

Live Trading Considerations

In live trading, we pass custom data to your algorithm as soon as it arrives. The time it arrives may not align with the time of other slices . Design your algorithm to handle unsychronized data so that you don't run into issues.

Demonstration Algorithm

The following example algorithm implements a custom data source for the Bitstamp API.

PY

```
class CustomDataBitstampAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 9, 1)
        self.set_end_date(2020, 12, 31)
        self.set_cash(100000)
        # Define the symbol and "type" of our generic data:
        self._custom_data_symbol = self.add_data(Bitstamp, "BTC").symbol
        # Get some historical data.
        history = self.history(
            Bitstamp, self._custom_data_symbol, 200, Resolution.DAILY
        )

    def on_data(self, slice):
        # Get the data of the current day.
        data = slice.get(self._custom_data_symbol)
        if not data:
            return
        self.plot(self._custom_data_symbol, 'Price', data.close)


class Bitstamp(PythonData):

    def get_source(self, config, date, is_live_mode):
        if is_live_mode:
            return SubscriptionDataSource(
                'https://www.bitstamp.net/api/ticker/',
                SubscriptionTransportMedium.REST
            )
        return SubscriptionDataSource(
            "https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-data/bitstampusd.csv",
            SubscriptionTransportMedium.REMOTE_FILE
        )

    def reader(self, config, line, date, is_live_mode):
        if not line.strip():
            return None
        coin = Bitstamp()
        coin.symbol = config.symbol
        # In live trading, parse the JSON file.
        if is_live_mode:
            # Example Line Format:
            # {"high": "441.00", "last": "421.86", "timestamp": "1411606877", "bid": "421.96", "vwap": "428.58", "volume":
"14120.40683975", "low": "418.83", "ask": "421.99"}
            live_btc = json.loads(line)
            # If value is zero, return None
            coin.value = float(live_btc["last"])
            if coin.value == 0:
                return None
            coin.end_time =  Extensions.convert_from_utc(
                datetime.utcnow(), config.exchange_time_zone
            )
            coin.time = coin.end_time - timedelta(1)
            coin["Open"] = float(live_btc["open"])
            coin["High"] = float(live_btc["high"])
            coin["Low"] = float(live_btc["low"])
            coin["Close"] = coin.value
            coin["Ask"] = float(live_btc["ask"])
            coin["Bid"] = float(live_btc["bid"])
            coin["VolumeBTC"] = float(live_btc["volume"])
            coin["WeightedPrice"] = float(live_btc["vwap"])
            return coin

        # In backtests, parse the CSV file.
        # Example Line Format:
        # Date      Open   High    Low     Close   Volume (BTC)    Volume (Currency)   Weighted Price
        # 2011-09-13 5.8    6.0     5.65    5.97    58.37138238,    346.0973893944      5.929230648356
        if not line[0].isdigit():
            return None
        data = line.split(',')
        # If value is zero, return None
        coin.value = float(data[4])
        if coin.value == 0:
            return None
        coin.time = datetime.strptime(data[0], "%Y-%m-%d")
        coin.end_time = coin.time + timedelta(1)
        coin["Open"] = float(data[1])
        coin["High"] = float(data[2])
        coin["Low"] = float(data[3])
        coin["Close"] = coin.value
        coin["VolumeBTC"] = float(data[5])
        coin["VolumeUSD"] = float(data[6])
        coin["WeightedPrice"] = float(data[7])
        return coin
```

Streaming Data

Custom Securities

Custom Securities

Key Concepts

Introduction

To receive your custom data in the `on_data` method instead of in a bulk download, create a custom type and then create a data subscription. The custom data type tells LEAN where to get your data and how to read it. All custom data types must extend from `PythonData` and override the `reader` and `get_source` methods.

Unlike custom data, native QC data gets the full benefit of security modeling like splits and dividends. Custom data is ignorant of everything about the actual market. Things like market hours, delistings, and mergers don't work with custom data.

Create Subscriptions

After you define the custom data class, in the `initialize` method of your algorithm, call the `add_data` method. This method gives LEAN the type factory to create the objects, the name of the data, and the resolution at which to poll the data source for updates.

```py
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_data(MyCustomDataType, "<name>", Resolution.DAILY).symbol
```

The `resolution` argument should match the resolution of your custom dataset. The lowest reasonable resolution is every minute. Anything more frequent than every minute is very slow to execute. The frequency that LEAN checks the data source depends on the `resolution` argument. The following table shows the polling frequency of each resolution:

Resolution	Update Frequency
Daily	Every 30 minutes
Hour	Every 30 minutes
Minute	Every minute
Second	Every second
Tick	Constantly checks for new data

There are several other signatures for the `add_data` method.

```
self.add_data(type, ticker, resolution)
self.add_data(type, ticker, resolution, time_zone, fill_forward, leverage)
self.add_data(type, ticker, properties, exchange_hours, resolution, fill_forward, leverage)
```

Receive Custom Data

As your data reader reads your custom data file, LEAN adds the data points in the `Slice` it passes to your algorithm's `on_data` method. To collect the custom data, use the `symbol` or name of your custom data subscription. You can access the `value` and custom properties of your custom data class from the `Slice` . To access the custom properties, pass the property name to the `get_property` method .

```py
class MyAlgorithm(QCAlgorithm):
    def on_data(self, slice: Slice) -> None:
        if slice.contains_key(self._symbol):
            custom_data = slice[self._symbol]
            value = custom_data.value
            property1 = custom_data.property1
```

Debugging

In backtests, you can use the IDE debugging capabilities to inspect the variables to understand the internal state of the custom data implementation. For more information about backtest debugging, see the Debugging documentation for Cloud Platform , Local Platform , or the CLI .

In live trading, consider adding logging statements to help with debugging since the debugger isn't available. To log information from inside your custom data class, set a class member that references the algorithm.

```py
class MyCustomDataTypeAlgorithm(QCAlgorithm):
    def initialize(self):
        MyCustomDataType.ALGORITHM = self
        self._symbol = self.add_data(MyCustomDataType, "<name>", Resolution.DAILY).Symbol

class MyCustomDataType(PythonData):
    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live_mode: bool) -> BaseData:
        data = line.split(',')
        custom = MyCustomDataType()
        custom.symbol = config.symbol
        if not line[0].isdigit():
            # Display the line with the header
            MyCustomDataType.ALGORITHM.Debug(f"HEADER: {line}")
            return custom
        custom.end_time = datetime.strptime(data[0], '%Y%m%d') + timedelta(1)
        custom.value = float(data[1])
        return custom
```

Set the Benchmark

To set your custom data source as the benchmark, in the `initialize` method, call the `set_benchmark` method with the `Symbol` of your custom data subscription.

```py
self._symbol = self.add_data(MyCustomDataType, "<name>", Resolution.DAILY).symbol
self.set_benchmark(self._symbol)
```

Avoid Look-Ahead Bias

Look-ahead bias occurs when you make decisions with information that wouldn't be available until some time in the future. In backtesting, look-ahead bias occurs when you receive data earlier than it would actually be available in reality. If look-ahead bias seeps into your algorithm, it will perform differently between live trading and backtesting.

To avoid look-ahead bias, set the timestamp of data points to the time when the data would actually be available. A lot of external sources apply timestamps to data differently than we do. For instance, on some platforms, daily bars are displayed with the date that they opened. We display daily bars with the date they closed. If you set the `end_time` to the start of the bar, you'll receive the bar earlier in backtests than you would in live trading.

Time Modeling

Data types classes in LEAN inherit from the `BaseData` class that defines the `time` and `end_time` properties. These properties represent the time period of which the data was built. If the data type occurs in a singular point in time, they have no period, so `time` and `end_time` are the same. Regardless of the period, LEAN uses the time when the data sample ends, `end_time` , to add the sample to a `Slice` .

Historical Data

To get historical data points for a custom dataset, call the `history` method with the dataset `Symbol` . For more information about historical data, see Data Points .

Market Hours

The default market hours for custom securities is to be always open. To set the market hours, pass an `exchange_hours` argument to the `add_data` method.

```
symbol = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
symbol_properties = self.symbol_properties_database.get_symbol_properties(symbol.id.market, symbol, SecurityType.EQUITY, 'USD')
security_exchange_hours = self.market_hours_database.get_exchange_hours(symbol.id.market, symbol, SecurityType.EQUITY)
security = self.add_data(CustomType, contract, symbol_properties, security_exchange_hours, Resolution.HOUR)
```

Custom Securities

CSV Format Example

Introduction

This page explains how to import custom data of a single security sourced in CSV format.

Data Format

The following snippet shows an example CSV file:

```
Date,Open,High,Low,Close,SharesTraded,Tur11er(Rs.Cr)
1997-01-01,905.2,941.4,905.2,939.55,38948210,978.21
1997-01-02,941.95,944,925.05,927.05,49118380,1150.42
1997-01-03,924.3,932.6,919.55,931.65,35263845,866.74
...
2014-07-24,7796.25,7835.65,7771.65,7830.6,117608370,6271.45
2014-07-25,7828.2,7840.95,7748.6,7790.45,153936037,7827.61
2014-07-28,7792.9,7799.9,7722.65,7748.7,116534670,6107.78
```

The data in the file must be in chronological order.

Define Custom Types

To define a custom data type, inherit the `PythonData` class and override the GetSource and Reader methods.

PY

```
class MyCustomDataType(PythonData):

    def get_source(self,
         config: SubscriptionDataConfig,
         date: datetime,
         is_live: bool) -> SubscriptionDataSource:

        if not is_live:
            return SubscriptionDataSource("<custom_data_key>", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.CSV)
        return SubscriptionDataSource("https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/csv-data-example.csv", SubscriptionTransportMedium.REMOTE_FILE)

    def reader(self,
         config: SubscriptionDataConfig,
         line: str,
         date: datetime,
         is_live: bool) -> BaseData:

        if not (line.strip()):
            return None

        index = MyCustomDataType()
        index.symbol = config.symbol

        try:
            data = line.split(',')
            index.time = datetime.strptime(data[0], "%Y-%m-%d")
            index.end_time = index.time + timedelta(days=1)
            index.value = data[4]
            index["open"] = float(data[1])
            index["high"] = float(data[2])
            index["low"] = float(data[3])
            index["close"] = float(data[4])

        except ValueError:
            # Do nothing
            return None

        return index
```

Create Subscriptions

To create a subscription for the custom type, in the `initialize` method, call the `add_data` method. The `add_data` method returns a `Security` object, which contains a `Symbol` . Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_data(MyCustomDataType, "MyCustomDataType", Resolution.DAILY).symbol
```

The `add_data` method creates a subscription for a single custom data asset and adds it to your user-defined universe.

Receive Custom Data

As your data reader reads your custom data file, LEAN adds the data points in the `Slice` it passes to your algorithm's `on_data` method. To collect the custom data, use the `symbol` or name of your custom data subscription. You can access the `value` and custom properties of your custom data class from the `Slice` . To access the custom properties, pass the property name to the `get_property` method .

```
class MyAlgorithm(QCAlgorithm):
    def on_data(self, slice: Slice) -> None:
        if slice.contains_key(self._symbol):
            custom_data = slice[self._symbol]
            close = custom_data.close
```

If you add custom properties to your data object in the `reader` method, LEAN adds them as members to the data object in the `Slice` . To ensure the property names you add in the `reader` method follow the convention of member names, LEAN applies the following changes to the property names you provide in the `reader` method:

1. `-` and `.` characters are replaced with whitespace.
2. The first letter is capitalized.
3. Whitespace characters are removed.

For example, if you set a property name in the `reader` method to `['some-property.name']` , you can access it in the `on_data` method through the `Somepropertyname` member of your data object.

Demonstration Algorithms

BubbleAlgorithm.py Python CustomDataNIFTYAlgorithm.py Python DescendingCustomDataObjectStoreRegressionAlgorithm.py Python

Custom Securities

JSON Format Example

Introduction

This page explains how to import custom data of a single security sourced in JSON format.

Data Format

The following code snippet shows an example JSON file

```
[
    {
      "Date": "1997-01-01",
      "Open": 905.2,
      "High": 941.4,
      "Low": 905.2,
      "Close": 939.55,
      "SharesTraded": 38948210,
      "Turller(Rs.Cr)": 978.21
    },
    {
      "Date": "1997-01-02",
      "Open": 941.95,
      "High": 944,
      "Low": 925.05,
      "Close": 927.05,
      "SharesTraded": 49118380,
      "Turller(Rs.Cr)": 1150.42
    },
    ...
    {
      "Date": "2014-07-25",
      "Open": 7828.2,
      "High": 7840.95,
      "Low": 7748.6,
      "Close": 7790.45,
      "SharesTraded": 153936037,
      "Turller(Rs.Cr)": 7827.61
    },
    {
      "Date": "2014-07-28",
      "Open": 7792.9,
      "High": 7799.9,
      "Low": 7722.65,
      "Close": 7748.7,
      "SharesTraded": 116534670,
      "Turller(Rs.Cr)": 6107.78
    }
]
```

The data in the file must be in chronological order.

Define Custom Types

To define a custom data type, inherit the `PythonData` class and override the GetSource and Reader methods.

If you need to create multiple objects in your `reader` method from a single `line` , follow these steps:

1. In the `get_source` method, pass `FileFormat.UNFOLDING_COLLECTION` as the third argument to the `SubscriptionDataSource` constructor.
2. In the `reader` method, order the objects by their timestamp and then return a `BaseDataCollection(end_time, config.symbol, objects)` where `objects` is a list of your custom data objects.

```
class MyCustomDataType(PythonData):
    def get_source(self,
         config: SubscriptionDataConfig,
         date: datetime,
         is_live: bool) -> SubscriptionDataSource:
        if not is_live:
            return SubscriptionDataSource("<custom_data_key>", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.UNFOLDING_COLLECTION)
        return SubscriptionDataSource("https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/unfolding-collection-example.json", SubscriptionTransportMedium.REMOTE_FILE, FileFormat.UNFOLDING_COLLECTION)

    def reader(self,
         config: SubscriptionDataConfig,
         line: str,
         date: datetime,
         is_live: bool) -> BaseData:

        if not (line.strip()):
            return None

        objects = []
        data = json.loads(line)

        for datum in data:
            index = MyCustomDataType()
            index.symbol = config.symbol
            index.time = datetime.strptime(datum["Date"], "%Y-%m-%d")
            index.end_time = index.time + timedelta(1)
            index["Open"] = float(datum["Open"])
            index["High"] = float(datum["High"])
            index["Low"] = float(datum["Low"])
            index["Close"] = float(datum["Close"])
            index.value = index["Close"]
            objects.append(index)

        return BaseDataCollection(objects[-1].end_time, config.symbol, objects)
```

Create Subscriptions

To create a subscription for the custom type, in the `initialize` method, call the `add_data` method. The `add_data` method returns a `Security` object, which contains a `Symbol` . Save a reference to the `symbol` so you can use it in `on_data` to access the security data in the `Slice` .

```
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_data(MyCustomDataType, "MyCustomDataType", Resolution.DAILY).symbol
```

The `add_data` method creates a subscription for a single custom data asset and adds it to your user-defined universe.

Receive Custom Data

As your data reader reads your custom data file, LEAN adds the data points in the `Slice` it passes to your algorithm's `on_data` method. To collect the custom data, use the `symbol` or name of your custom data subscription. You can access the `value` and custom properties of your custom data class from the `Slice` . To access the custom properties, pass the property name to the `get_property` method .

```
class MyAlgorithm(QCAlgorithm):
    def on_data(self, slice: Slice) -> None:
        if slice.contains_key(self._symbol):
            custom_data = slice[self._symbol]
            close = custom_data.close
```

If you add custom properties to your data object in the `reader` method, LEAN adds them as members to the data object in the `Slice` . To ensure the property names you add in the `reader` method follow the convention of member names, LEAN applies the following changes to the property names you provide in the `reader` method:

1. `-` and `.` characters are replaced with whitespace.
2. The first letter is capitalized.
3. Whitespace characters are removed.

For example, if you set a property name in the `reader` method to `['some-property.name']` , you can access it in the `on_data` method through the `Somepropertyname` member of your data object.

Demonstration Algorithms

CustomDataBitcoinAlgorithm.py Python

Streaming Data

Custom Universes

Custom Universes

Key Concepts

Introduction

A custom universe lets you select a basket of assets from a custom dataset.

Initialize Universes

To add a custom universe to your algorithm, in the `initialize` method, pass your universe type and a selector function to the `add_universe` method.

```
self.add_universe(MyCustomUniverseDataClass, self._selector_function)
```

Receive Custom Data

The universe selector function receives a list of your custom objects and must return a list of `Symbol` objects. In the selector function definition, you can use any of the properties of your custom data type. The `Symbol` objects that you return from the selector function set the constituents of the universe.

```
class MyCustomUniverseAlgorithm(QCAlgorithm):
 def _selector_function(self, data: List[MyCustomUniverseDataClass]) -> List[Symbol]:
     sorted_data = sorted([ x for x in data if x["CustomAttribute1"] > 0 ],
                          key=lambda x: x["CustomAttribute2"],
                          reverse=True)
     return [x.symbol for x in sorted_data[:5]]
```

Historical Data

To get historical data for a custom data universe , call the `history` method with the `Universe` object. For more information about historical data, see Universes .

Custom Universes

CSV Format Example

Introduction

This page explains how to import custom data for universe selection sourced in CSV format.

Data Format

You must create a file with data in CSV format. Ensure the data in the file is in chronological order.

```
20170704,SPY,QQQ,FB,AAPL,IWM
20170706,QQQ,AAPL,IWM,FB,GOOGL
20170707,IWM,AAPL,FB,BAC,GOOGL
...
20170729,SPY,QQQ,FB,AAPL,IWM
20170801,QQQ,FB,AAPL,IWM,GOOGL
20170802,QQQ,IWM,FB,BAC,GOOGL
```

Define Custom Types

To define a custom data type, inherit the `PythonData` class and override the GetSource and Reader methods.

PY

```
class StockDataSource(PythonData):
    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live: bool) -> SubscriptionDataSource:
        if not is_live:
            return SubscriptionDataSource("csv-universe-example.csv", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.CSV)
        return SubscriptionDataSource("https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/csv-universe-example.csv", SubscriptionTransportMedium.REMOTE_FILE, FileFormat.CSV)

    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live: bool) -> BaseData:
        csv = line.split(',')
        stocks = StockDataSource()
        stocks.symbol = config.symbol
        stocks.line = line
        try:
            stocks.time = Extensions.convert_from_utc(datetime.utcnow(), config.data_time_zone) \
                if is_live else datetime.strptime(csv[0], "%Y%m%d")
            def point_in_time(ticker):
                # The tickers are point-in-time. We generate its security identifier for a given date
                # Then we create a Symbol where the value is the ticker
                sid = SecurityIdentifier.generate_equity(ticker, Market.USA, mapping_resolve_date=stocks.time)
                return Symbol(sid, ticker)
            stocks.symbols = [point_in_time (ticker) for ticker in csv[1:]]
        except:
            pass
        return stocks
```

Initialize Universe

To perform a universe selection with custom data, in the `initialize` method, call the `add_universe` method.

PY

```
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.add_universe(StockDataSource, self._filter_function)
```

Receive Custom Data

As your data reader reads your custom data file, LEAN adds the data points into a `List[StockDataSource])` object it passes to your algorithm's filter function. Your filter function needs to return a list of `Symbol` or `str` object. LEAN automatically subscribes to these new assets and adds them to your algorithm.

```
class MyAlgorithm(QCAlgorithm):
    def _filter_function(self, data: list[BaseData]) -> list[str]:
        symbols = []
        for item in data:
            symbols.extend(item.symbols)
        return symbols

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self.debug(str(changes))
```

If you add custom properties to your data object in the `reader` method, LEAN adds them as members to the data object in your filter method. To ensure the property names you add in the `reader` method follow the convention of member names, LEAN applies the following changes to the property names you provide in the `reader` method:

1. `-` and `.` characters are replaced with whitespace.
2. The first letter is capitalized.
3. Whitespace characters are removed.

For example, if you set a property name in the `reader` method to `['some-property.name']`, you can access it in your filter method through the `Somepropertyname` member of your data object.

Demonstration Algorithms

DropboxBaseDataUniverseSelectionAlgorithm.py Python

Custom Universes

JSON Format Example

Introduction

This page explains how to import custom data for universe selection sourced in JSON format.

Data Format

You must create a file with data in JSON format. Ensure the data in the file is in chronological order.

```
[
    {
      "Date": "20170704",
      "Symbols": ["SPY", "QQQ", "FB", "AAPL", "IWM"]
    },
    {
      "Date": "20170706",
      "Symbols": ["QQQ", "AAPL", "IWM", "FB", "GOOGL"]
    },
    ...
    {
      "Date": "20170801",
      "Symbols": ["QQQ", "FB", "AAPL", "IWM", "GOOGL"]
    },
    {
      "Date": "20170802",
      "Symbols": ["QQQ", "IWM", "FB", "BAC", "GOOGL"]
    }
]
```

Define Custom Types

To define a custom data type, inherit the `PythonData` class and override the GetSource and Reader methods.

If you need to create multiple objects in your `reader` method from a single `line` , follow these steps:

1. In the `get_source` method, pass `FileFormat.UNFOLDING_COLLECTION` as the third argument to the `SubscriptionDataSource` constructor.
2. In the `reader` method, order the objects by their timestamp and then return a `BaseDataCollection(end_time, config.symbol, objects)` where `objects` is a list of your custom data objects.

PY

```
class StockDataSource(PythonData):
    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live: bool) -> SubscriptionDataSource:
        if not is_live:
            return SubscriptionDataSource("universe-example.json", SubscriptionTransportMedium.OBJECT_STORE,
FileFormat.UNFOLDING_COLLECTION)
        return SubscriptionDataSource("https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/universe-example.json", SubscriptionTransportMedium.REMOTE_FILE, FileFormat.UNFOLDING_COLLECTION)

    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live: bool) -> BaseData:
        def convert(obj):
            stocks = StockDataSource()
            stocks.time = datetime.strptime(obj["Date"], "%Y%m%d")
            def point_in_time(ticker):
                # The tickers are point-in-time. We generate its security identifier for a given date
                # Then we create a Symbol where the value is the ticker
                sid = SecurityIdentifier.generate_equity(ticker, Market.USA, mapping_resolve_date=stocks.time)
                return Symbol(sid, ticker)
            stocks.symbols = [point_in_time (ticker) for ticker in obj["Symbols"]]
            return stocks

        objects = [convert(obj) for obj in json.loads(line)]

        return BaseDataCollection(objects[-1].end_time, config.symbol, objects)
```

Initialize Universe

To perform a universe selection with custom data, in the `initialize` method, call the `add_universe` method.

```py
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.add_universe(StockDataSource, self._filter_function)
```

Receive Custom Data

As your data reader reads your custom data file, LEAN adds the data points into a `List[StockDataSource])` object it passes to your algorithm's filter function. Your filter function needs to return a list of `Symbol` or `str` object. LEAN automatically subscribes to these new assets and adds them to your algorithm.

```py
class MyAlgorithm(QCAlgorithm):
    def _filter_function(self, data: list[BaseData]) -> list[str]:
        symbols = []
        for item in data:
            symbols.extend(item.symbols)
        return symbols

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self.debug(str(changes))
```

If you add custom properties to your data object in the `reader` method, LEAN adds them as members to the data object in your filter method. To ensure the property names you add in the `reader` method follow the convention of member names, LEAN applies the following changes to the property names you provide in the `reader` method:

1. `-` and `.` characters are replaced with whitespace.
2. The first letter is capitalized.
3. Whitespace characters are removed.

For example, if you set a property name in the `reader` method to `['some-property.name']`, you can access it in your filter method through the `Somepropertyname` member of your data object.

Demonstration Algorithms

DropboxBaseDataUniverseSelectionAlgorithm.py Python

Streaming Data

Precomputed ML Predictions

Introduction

QuantConnect's point-in-time backtesting engine is one of its greatest strengths: in machine-learning strategies , it forces every feature and prediction to be calculated using only the data available on that exact historical day. This delivers unmatched realism and eliminates look-ahead bias. At the same time, there are clear trade-offs. The same rigorous point-in-time discipline means that every small change to portfolio rules normally triggers a full rerun of the feature-engineering and model-inference pipeline inside the backtest engine.

To avoid re-running the entire feature-engineering and model-inference pipeline for every small change, you can precompute machine learning predictions in the Research Environment and then stream them into an algorithm as a custom universe dataset . This two-file pattern separates expensive feature engineering and model training from the algorithm, which becomes a pure portfolio engine. It provides the following benefits:

- **Speed** - Backtests run much faster because they skip feature calculation and model inference.
- **Iteration speed** - Test many thresholds or risk overlays without rerunning the ML pipeline.
- **Reproducibility** - Predictions are frozen in the Object Store, so there is no drift between backtest runs.

This page explains how to implement a project for precomputing machine learning predictions in the Research Environment and streaming them into a backtest as a custom universe dataset.

Create ML Predictions

In a Research Environment, gather universe and price history , calculate factors and labels, train a model, and export daily predictions as JSON to the Object Store . When you define factors in the notebook, strictly use trailing data as inputs. If you use future data, you may introduce look-ahead bias into the backtest.

PY

```
from sklearn.ensemble import GradientBoostingRegressor

# Define the training and validation periods.
validation_end = datetime(2026, 3, 12)  # or datetime.now()
validation_start = validation_end - timedelta(30)
train_end = validation_start
train_start = train_end - timedelta(90)

# Get the SP500 constituents history.
qb = QuantBook()
spy = qb.add_equity('SPY')
universe = qb.add_universe(qb.universe.etf(spy))
universe_history = qb.universe_history(universe, train_start, validation_end)
symbols_by_date = {
    time: [constituent.symbol for constituent in constituents]  # Timestamps are midnight Tuesday-Saturday
    for (_, time), constituents in universe_history.items()
}

# Get price history for all the assets in the universe.
symbols = list(set().union(*symbols_by_date.values()))
start_buffer = timedelta(40) # Add buffers to warm-up factors and labels
end_buffer = timedelta(5)
history = qb.history(symbols, train_start-start_buffer, validation_end+end_buffer, Resolution.DAILY)

# Calculate the feature and labels.
def add_features_and_label(asset_history):
    open_ = asset_history["open"]
    close = asset_history["close"]
    volume = asset_history["volume"]
    daily_return = close.pct_change()
    dataset = pd.DataFrame()
    # Feature 1: 10-day momentum
    dataset["momentum_10d"] = close / close.shift(10) - 1
    # Feature 2: 20-day volatility
    dataset["volatility_20d"] = daily_return.rolling(window=20).std()
    # Feature 3: Relative volume
    dataset["relative_volume"] = volume / volume.rolling(window=20).mean()
    # Label: return from next open to following open
    # The rebalancing logic in the algorithm uses market-on-open orders.
    # The opening auction is the earliest possible time to trade after
    # the factors update at market close.
    dataset["label"] = open_.shift(-2) / open_.shift(-1) - 1
    return dataset

dataset = history.groupby(level='symbol').apply(add_features_and_label).droplevel(0)
```

```
# Remove rows from the dataset for when the asset wouldn't have been in the universe.
indices = []
rows = []
for universe_dt, symbols in symbols_by_date.items():
    previous_close_dt = spy.exchange.hours.get_next_market_close(universe_dt - timedelta(1), False)
    for symbol in symbols:
        indices.append((universe_dt, symbol))
        rows.append(dataset.loc[(symbol, previous_close_dt)])
cleaned_dataset = pd.DataFrame(rows, index=pd.MultiIndex.from_tuples(indices, names=['time', 'symbol']))

# Split the dataset into train and validation subsets (and drop NaN rows).
t = cleaned_dataset.index.get_level_values("time")
train_data = cleaned_dataset[(train_start <= t) & (t < validation_start)].dropna()
validation_data = cleaned_dataset[(validation_start <= t) & (t <= validation_end)].dropna()
X_train = train_data.drop('label', axis=1)
y_train = train_data['label']
X_val = validation_data.drop('label', axis=1)
y_val = validation_data['label']

# Train the model.
model = GradientBoostingRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
model.fit(X_train, y_train)

# Calculate predictions on the validation set.
predictions = pd.Series(model.predict(X_val), index=X_val.index)

# Convert the predictions Series into a JSON object.
df = predictions.rename("prediction").reset_index()
df['symbol'] = df['symbol'].apply(str)
df['time'] = df['time'].dt.strftime('%Y-%m-%d')
predictions_json = [
    {
        'date': date,
        'prediction_by_symbol': group.set_index('symbol')['prediction'].to_dict()
    }
    for date, group in df.groupby('time')
]
# Save the predictions JSON to the Object Store.
key = 'research-to-backtest-factors.json'
qb.object_store.save(key, json.dumps(predictions_json))
```

Define A Custom Universe

In the main.py file of your project, define a custom data type that reads the JSON predictions from the Object Store.

```
PY
class PredictionUniverse(PythonData):

    def get_source(self, config, date, is_live_mode):
        # Read from the JSON file we saved in the Object Store.
        return SubscriptionDataSource(
            'research-to-backtest-factors.json',
            SubscriptionTransportMedium.OBJECT_STORE,
            FileFormat.UNFOLDING_COLLECTION
        )

    def reader(self, config, line, date, is_live):
        objects = []
        # Iterate through each day.
        for obj in json.loads(line):
            end_time = datetime.strptime(obj["date"], "%Y-%m-%d")
            # Iterate through each stock on this day.
            for symbol, prediction in obj['prediction_by_symbol'].items():
                stock = PredictionUniverse()
                stock.symbol = Symbol(SecurityIdentifier.parse(symbol), "")
                stock.end_time = end_time
                stock.value = prediction
                objects.append(stock)
        # The BaseDataCollection contains multiple PredictionUniverse objects
        # for each day. Set its `time` to match the `end_time` of one of the objects.
        # We pack the collection and timestamp it with the last timestamp (-1) to signal
        # that the collection has data prior to (or including) that timestamp.
        return BaseDataCollection(objects[-1].end_time, config.symbol, objects)
```

Stream Predictions in Backtests

In the main.py file of your project, subscribe to the custom universe type you defined in the preceding step. To rebalance the portfolio on each new set of daily predictions, add a Scheduled Event .

PY

```
class PredictionUniverseAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2026, 2, 10)
        self.set_cash(100_000)
        self.settings.seed_initial_prices = True
        self._return_prediction_threshold = 0
        # Add the universe of custom data.
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(PredictionUniverse, self._select_assets)
        # Add a Scheduled Event to rebalance the portfolio every day.
        self.schedule.on(self.date_rules.every_day('SPY'), self.time_rules.at(8, 0), self._rebalance)

    def _select_assets(self, data):
        # Select the assets with a return prediction above some threshold.
        return [stock.symbol for stock in data if stock.value > self._return_prediction_threshold]

    def _rebalance(self):
        # Form an equal-weighted portfolio with the assets in the universe.
        symbols = self._universe.selected
        if not symbols:
            return
        targets = [PortfolioTarget(symbol, 1/len(symbols)) for symbol in symbols]
        self.set_holdings(targets, True)
```

Importing Data

Bulk Downloads

Introduction

There are two techniques to import data into your algorithm. You can either manually import the entire file or stream the file line-by-line into your algorithm's `on_data` event. This page explores importing an entire file for manual use.

Instead of downloading the file from a remote file provider, you can upload the file to the Object Store (with the Algorithm Lab or with the CLI) for faster execution.

Recommended Use Cases

The batch import technique is outside of the LEAN's awareness or control, so it can't enforce good practices. However, the batch import technique is good for the loading the following datasets:

- Loading data into the Object Store
- Trained AI Models
- Well-defined historical price datasets
- Parameters and setting imports such as `Symbol` lists

Download Files

The `download` method downloads the content served from a local file or URL and then returns it as a string.

Basic Usage

```py
file = self.download("<filePathOrURL>")

# If your file is in CSV format, convert it to a DataFrame with the `read_csv` method.
from io import StringIO
import pandas as pd
df = pd.read_csv(StringIO(file))

# If your file is in JSON format, parse it with the `loads` method.
import json
data = json.loads(file)

# If your file is in XML format, parse it with the `fromstring` method.
import xml.etree.ElementTree as ET
root = ET.fromstring(file)
```

Download Method Arguments

The `download` method can accept header settings, a username, and a password for authentication.

Argument	Data Type	Description	Default Value
`address`	`str`	A string containing the URI to download	
`headers`	`Dict[str,str]`	Defines header values to add to the request	`dict()`
`user_name`	`str`	The user name associated with the credentials	`None`
`password`	`str`	The password for the user name associated with the credentials	`None`

Download Request Headers

```
header = { "1": "1" }
self.download(address, headers)
self.download(address, headers, user_name, password)
```

Save Files

When you download a remote file, save it into the Object Store so that you don't have to download the file again. If you need to import the file multiple times, it's faster to import it from the Object Store rather than repeatedly downloading the file from the remote file provider.

Transport Binary Data

Follow these steps to transport binary files with `joblib` :

1. Add the following imports to your local program:

```
import joblib
from io import BytesIO
from base64 import b64encode, b64decode
```

2. Serialize your object and save it to a file. This Base64 serialization is necessary because the default `joblib.dump` serialization produces a binary file that the `download` method can't read without data corruption.

```
buffer = BytesIO()
joblib.dump(my_object, buffer)
base64_str = b64encode(buffer.getvalue()).decode('ascii')
with open("my_model.b64", "w") as f:
    f.write(base64_str)
```

3. Save the file to one of the supported sources . For example, save the file to Dropbox.
4. Download the remote file into your project.

```
base64_str = self.download("<fileURL>")
```

5. Restore the object.

```
model_bytes = b64decode(base64_str.encode('ascii'))
restored_model = joblib.load(BytesIO(model_bytes))
```

Follow these steps to transport binary files with `pickle` :

1. Add the following imports to your local program:

```
import pickle
from base64 import b64encode, b64decode
```

2. Serialize your object and save it to a file.

```
pickle_bytes = pickle.dumps(my_object)
base64_str = b64encode(pickle_bytes).decode('ascii')
with open("my_model.b64", "w") as f:
    f.write(base64_str)
```

3. Save the file to one of the supported sources . For example, save the file to Dropbox.
4. Download the remote file into your project.

```
base64_str = self.download("<fileURL>")
```

5. Restore the object.

```
PY
model_bytes = b64decode(base64_str.encode('ascii'))
restored_model = pickle.loads(model_bytes)
```

Examples

The following examples demonstrate common practices for bulk downloading data.

Example 1: Download Machine Learning Model

The following algorithm makes use of a Scikit-Learn machine learning model to predict SPY price changes and place orders according to the prediction. To obtain the model, we either retrieve it from the Object Store if there exists any or download it from a Dropbox link using the `download` method.

```
PY
import joblib
from io import BytesIO
from base64 import b64decode, b64encode
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV

class BulkDownloadExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction, and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # 2-year data to train the model.
        training_length = 252*2
        self.training_data = RollingWindow(training_length)
        # Warm up the training dataset to train the model immediately.
        history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
        for trade_bar in history:
            self.training_data.add(trade_bar)

        # Retrieve the already trained model from the object store for immediate use.
        if self.object_store.contains_key("sklearn_model"):
            self.model = joblib.load(self.object_store.get_file_path("sklearn_model"))
        # Otherwise, bulk-download the base64-encoded model from an external source (Dropbox in this example).
        else:
            url = "https://www.dropbox.com/scl/fi/tj7wmpv1u2ysvejb0skpm/sklearn_model_b64?rlkey=pvcnuvqrp78oz1t73rytgnvdx&dl=1"
            base64_str = self.download(url)
            model_bytes = b64decode(base64_str.encode('ascii'))
            self.model = joblib.load(BytesIO(model_bytes))

        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.training_data)[::-1])
        daily_pct_change = training_df.pct_change().dropna()

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(daily_pct_change)-n_steps):
            features.append(daily_pct_change.iloc[i:i+n_steps].values.flatten())
            labels.append(daily_pct_change['close'].iloc[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()
        # Recalibrate the model based on updated data.
        if isinstance(self.model, GridSearchCV):
            self.model = self.model.fit(features, labels).best_estimator_
        else:
            self.model = self.model.fit(features, labels)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self.training_data.add(bar)

        # Get predictions by the updated features.
        features, _ = self.get_features_and_labels()
```

```
            prediction = self.model.predict(features[-1].reshape(1, -1))
            prediction = float(prediction)

            # If the predicted direction is going upward, buy SPY.
            if prediction > 0:
                self.set_holdings(self._symbol, 1)
            # If the predicted direction is going downward, sell SPY.
            elif prediction < 0:
                self.set_holdings(self._symbol, -1)

        def on_end_of_algorithm(self) -> None:
            # Store the model in the object store to retrieve it in other instances if the algorithm stops.
            model_key = "sklearn_model"
            joblib.dump(self.model, self.object_store.get_file_path(model_key))
            self._save_base64_encoded_version(model_key)

        def _save_base64_encoded_version(self, model_key):
            # Save a base64-encoded version of the model to upload to an external source (e.g., Dropbox).
            buffer = BytesIO()
            joblib.dump(self.model, buffer)
            content = b64encode(buffer.getvalue()).decode('ascii')
            path = self.object_store.get_file_path(f"{model_key}_b64")
            self.object_store.save(path, content)

            # Locally, you may want to use the open method
            #with open(path, "w") as f:
            #    f.write(content)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
DropboxCoarseFineAlgorithm.py Python NLTKSentimentTradingAlgorithm.py Python

Consolidating Data

Consolidating Data

Getting Started

Introduction

Consolidating data allows you to create bars of any length from smaller bars. Consolidation is commonly used to combine one-minute price bars into longer bars such as 10-20 minute bars. Consolidated bars are helpful because price movement over a longer period can sometimes contain less noise and bars of exotic length are less researched than standard bars, so they may present more opportunities to capture alpha.

To consolidate data, create a `Consolidator` object and register it for data. The built-in consolidators make it easy to create consolidated bars without introducing bugs. In the following sections, we will introduce the different types of consolidators and show you how to shape data into any form.

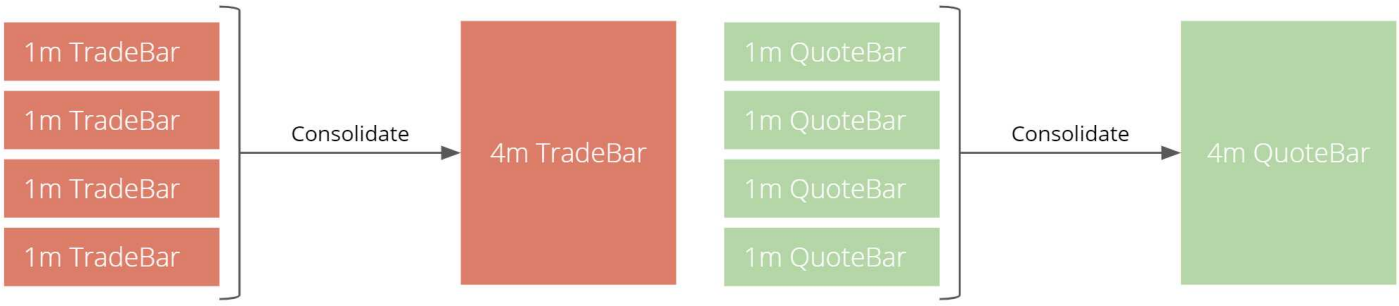
Basic Usage

The following code snippet demonstrates a simple consolidator that aggregates 1-minute bars into 10-minutes bars:

```
# In Initialize, subscribe to a security, create the consolidator, and register it for automatic updates
self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
self._consolidator = self.consolidate(self._symbol, timedelta(minutes=10), self._consolidation_handler)

# Define the consolidation handler
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    pass
```

Data Shapes and Sizes

Consolidators usually produce output data that is the same format as the input data. They aggregate small `TradeBar` objects into a large `TradeBar` , small `QuoteBar` objects into a large `QuoteBar` , and `Tick` objects into either a `TradeBar` or `QuoteBar` .



Many asset classes in QuantConnect have data for both trades and quotes. By default, consolidators aggregate data into `TradeBar` objects. Forex data is the only exception, which consolidators aggregate into `QuoteBar` objects since Forex data doesn't have `TradeBar` objects.

Consolidator Types

There are many different types of consolidators you can create. The process to create and update the consolidator depends on the input data format, output data format, and the consolidation technique.

Time Period Consolidators

Time period consolidators aggregate data based on a period of time like a number of seconds, minutes, or days. If a time period consolidator aggregates data over multiple days, the multi-day aggregation cycle starts and repeats based on the time stamp of the first data point you use to update the consolidator.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	TradeBarConsolidator
`QuoteBar`	`QuoteBar`	QuoteBarConsolidator
`Tick`	`TradeBar`	TickConsolidator
`Tick`	`QuoteBar`	TickQuoteBarConsolidator

Calendar Consolidators

Calendar consolidators aggregate data based on specific periods of time like a weekly basis, quarterly basis, or any schedule with specific start and end times. In contrast to time period consolidators, the bars that calendar consolidators produce always end on the same schedule, regardless of the first data point the consolidator receives.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	TradeBarConsolidator
`QuoteBar`	`QuoteBar`	QuoteBarConsolidator
`Tick`	`TradeBar`	TickConsolidator
`Tick`	`QuoteBar`	TickQuoteBarConsolidator

Count Consolidators

Count consolidators aggregate data based on a number of bars or ticks. This type of consolidator aggregates n samples together, regardless of the time period the samples cover. Managing count consolidators is similiar to managing time period consolidators , except you create count consolidators with an integer argument instead of a time-based argument.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	TradeBarConsolidator
`QuoteBar`	`QuoteBar`	QuoteBarConsolidator
`Tick`	`TradeBar`	TickConsolidator
`Tick`	`QuoteBar`	TickQuoteBarConsolidator

Mixed-Mode Consolidators

Mixed-mode consolidators are a combination of count consolidators and time period consolidators. This type of consolidator aggregates n samples together or aggregates samples over a specific time period, whichever happens first.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
TradeBar	TradeBar	TradeBarConsolidator
QuoteBar	QuoteBar	QuoteBarConsolidator
Tick	TradeBar	TickConsolidator
Tick	QuoteBar	TickQuoteBarConsolidator

Renko Consolidators

Most Renko consolidators aggregate bars based on a fixed price movement. The RenkoConsolidator produces Renko bars by their traditional definition. In the case of a $1 bar size, the RenkoConsolidator produces bars that have a body spanning 1. The opening price of the first bar is set to the closest 1 multiple of the first trade. When the price moves by at least 1, the first bar closes. If a bar is a rising bar, the following bar closes when the price moves 1 above the closing price of the previous bar or 1 below the opening price of the previous bar. If a bar is a falling bar, the following bar closes when the price moves 1 below the closing price of the previous bar or $1 above the opening price of the previous bar. If the price jumps multiple dollars in a single tick, the RenkoConsolidator produces multiple $1 bars in a single time step.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
TradeBar	TradeBar	RenkoConsolidator
QuoteBar	QuoteBar	RenkoConsolidator
Tick	TradeBar	RenkoConsolidator
Tick	QuoteBar	RenkoConsolidator

Classic Renko Consolidators

Most Renko consolidators aggregate bars based on a fixed price movement. The ClassicRenkoConsolidator produces a different type of Renko bars than the RenkoConsolidator . A ClassicRenkoConsolidator with a bar size of 1 produces a new bar that spans 1 every time an asset closes $1 away from the close of the previous bar. If the price jumps multiple dollars in a single tick, the ClassicRenkoConsolidator only produces one bar per time step where the open of each bar matches the close of the previous bar.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
TradeBar	TradeBar	ClassicRenkoConsolidator
QuoteBar	QuoteBar	ClassicRenkoConsolidator
Tick	TradeBar	ClassicRenkoConsolidator
Tick	QuoteBar	ClassicRenkoConsolidator

Volume Renko Consolidators

Volume Renko consolidators aggregate bars based on a fixed trading volume. These types of bars are commonly known as volume bars. A VolumeRenkoConsolidator with a bar size of 10,000 produces a new bar every time 10,000 units of the security trades in the market. If the trading volume of a single time step exceeds two step sizes (i.e. >20,000), the VolumeRenkoConsolidator produces multiple bars in a single time step.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	VolumeRenkoConsolidator
`Tick`	`TradeBar`	VolumeRenkoConsolidator

Range Consolidators

Most Range consolidators aggregate bars based on a fixed price movement. It is very much like a Renko Bar, but instead of consolidating a fix movement from the opening price, a Range Bar would fix the range of the whole bar, i.e. High - Low. A new bar is formed when the range of the whole bar reached a preset value, thus it completely neglects the time effect and focuses on the variance/information carried by the price movement.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	RangeConsolidator
`QuoteBar`	`QuoteBar`	RangeConsolidator
`Tick`	`TradeBar`	RangeConsolidator
`Tick`	`QuoteBar`	RangeConsolidator

Sequential Consolidators

Sequential consolidators wire two internal consolidators together such that the output of the first consolidator is the input to the second consolidator and the output of the second consolidator is the output of the sequential consolidator.

For more information about sequential consolidators, see Combining Consolidators .

Identity Consolidators

Identity consolidators are data pass through. This type of consolidator does not aggregate samples but instead passes each data point through as-is. You can use this consolidator to register indicators that require every data point without any aggregation or sequential consolidators.

Filtered Identity consolidators are a particular type that allows filtering the data. For example, you can filter out unwanted data points before they reach your indicators.

The following table shows which consolidator type to use based on the data format of the input and output:

Input	Output	Class Type
`TradeBar`	`TradeBar`	IdentityDataConsolidator[TradeBar]
`QuoteBar`	`QuoteBar`	IdentityDataConsolidator[QuoteBar]
`Tick`	`TradeBar`	FilteredIdentityDataConsolidator.for_tick_type(Ti
`Tick`	`QuoteBar`	FilteredIdentityDataConsolidator.for_tick_type(Ti

Execution Sequence

The algorithm manager calls events in the following order:

1. Scheduled Events
2. Consolidation event handlers
3. `on_data` event handler

This event flow is important to note. For instance, if your consolidation handlers or `on_data` event handler appends data to a `RollingWindow` and you use that `RollingWindow` in your Scheduled Event, when the Scheduled Event executes, the `RollingWindow` won't contain the most recent data.

The consolidators are called in the order that they are updated. If you register them for automatic updates, they are updated in the order that you register them.

Plot Consolidated Data

To plot consolidated bars, call the `plot` method with a `TradeBar` argument in the consolidation handler.

```
# In Initialize
equity = self.add_equity("SPY")
self.consolidate(equity.symbol, timedelta(minutes=10), self._consolidation_handler)

# Define the consolidation handler
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    self.plot("<chartName>", "<seriesName>", consolidated_bar)
```

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

Demonstration Algorithms
BasicTemplateFuturesConsolidationAlgorithm.py Python BasicTemplateOptionsConsolidationAlgorithm.py Python DataConsolidationAlgorithm.py Python MultipleSymbolConsolidationAlgorithm.py Python
Video Tutorials
Opening Range Breakout - Time Period Consolidation Python

Consolidating Data

Consolidator Types

Consolidator Types

Time Period Consolidators

Introduction

Time period consolidators aggregate data based on a period of time like a number of seconds, minutes, or days. If a time period consolidator aggregates data over multiple days, the multi-day aggregation cycle starts and repeats based on the time stamp of the first data point you use to update the consolidator.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `TradeBar` objects of the same size or larger. Follow these steps to create and manage a `TradeBar` consolidator based on a period of time:

1. Create the consolidator.

   To set the time period for the consolidator, you can use either a `timedelta` or `Resolution` object. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   - `timedelta` Periods

     ```
     self._consolidator = TradeBarConsolidator(timedelta(days=1))
     # Aliases:
     # self._consolidator = self.create_consolidator(timedelta(days=1), TradeBar)
     # self._consolidator = self.resolve_consolidator(self._symbol, timedelta(days=1))
     ```

     The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

     Use the `start_time` parameter of the TradeBarConsolidator type constructor to define a custom start time.

     ```
     self._consolidator = TradeBarConsolidator(timedelta(days=1),
         start_time=timedelta(hours=9, minutes=30))
     ```

   - `Resolution` Periods

     The `Resolution` enumeration has the following members:

     ```
     self._consolidator = TradeBarConsolidator.from_resolution(Resolution.DAILY)
     # Alias:
     # self._consolidator = self.resolve_consolidator(self._symbol, Resolution.DAILY)
     ```

   If the security subscription in your algorithm provides `TradeBar` and `QuoteBar` data, `resolve_consolidator` returns a `TradeBarConsolidator` .

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the TradeBarConsolidator type constructor to `timedelta(hours=9, minutes=30)`

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone . If you subscribe to minute resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to minute resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      trade_bar = slice.bars[self._symbol]
      self._consolidator.update(trade_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
  for trade_bar in history:
      self._consolidator.update(trade_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `Consolidate` helper method to create period consolidators and register them for automatic updates. With just one line of code,

you can create data in any time period based on a `timedelta` or `Resolution` object:

- `timedelta` Periods

```
self._consolidator = self.consolidate(self._symbol, timedelta(days=1), self._consolidation_handler)
```

- `Resolution` Periods

```
self._consolidator = self.consolidate(self._symbol, Resolution.DAILY, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    pass
```

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `QuoteBar` objects of the same size or larger. Follow these steps to create and manage a `QuoteBar` consolidator based on a period of time:

1. Create the consolidator.

   To set the time period for the consolidator, you can use either a `timedelta` or `Resolution` object. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   - `timedelta` Periods

     ```
     self._consolidator = QuoteBarConsolidator(timedelta(days=1))
     # Aliases:
     # self._consolidator = self.create_consolidator(timedelta(days=1), QuoteBar)
     # self._consolidator = self.resolve_consolidator(self._symbol, timedelta(days=1))
     ```

     The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

     Use the `start_time` parameter of the QuoteBarConsolidator type constructor to define a custom start time.

     ```
     self._consolidator = QuoteBarConsolidator(timedelta(days=1),
         start_time=timedelta(hours=9, minutes=30))
     ```

   - `Resolution` Periods

     The `Resolution` enumeration has the following members:

     ```
     self._consolidator = self.resolve_consolidator(self._symbol, Resolution.DAILY)
     ```

   If the security subscription in your algorithm provides `TradeBar` and `QuoteBar` data, `resolve_consolidator` returns a `TradeBarConsolidator` .

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the QuoteBarConsolidator type constructor to `timedelta(hours=9, minutes=30)`

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone . If you subscribe to minute resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to minute resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         quote_bar = slice.quote_bars[self._symbol]
         self._consolidator.update(quote_bar)

     # Example 2: Update the consolidator with data from a history request
     history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
     for quote_bar in history:
         self._consolidator.update(quote_bar)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `Consolidate` helper method to create period consolidators and register them for automatic updates. With just one line of code, you can create data in any time period based on a `timedelta` or `Resolution` object:

- `timedelta` Periods

```
self._consolidator = self.consolidate(self._symbol, timedelta(days=1), TickType.Quote, self._consolidation_handler)
```

- `Resolution` Periods

```
self._consolidator = self.consolidate(self._symbol, Resolution.DAILY, TickType.Quote, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: QuoteBar) -> None:
    pass
```

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `TradeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on a period of time:

1. Create the consolidator.

   To set the time period for the consolidator, you can use either a `timedelta` or `Resolution` object. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   - `timedelta` Periods

     ```
     self._consolidator = TickConsolidator(timedelta(milliseconds=100))
     # Aliases:
     # self._consolidator = self.create_consolidator(timedelta(milliseconds=100), Tick)
     # self._consolidator = self.resolve_consolidator(self._symbol, timedelta(milliseconds=100))
     ```

     The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

     Use the `start_time` parameter of the TickConsolidator type constructor to define a custom start time.

     ```
     self._consolidator = TickConsolidator(timedelta(milliseconds=100),
         start_time=timedelta(hours=9, minutes=30))
     ```

   - `Resolution` Periods

     The `Resolution` enumeration has the following members:

     ```
     self._consolidator = self.resolve_consolidator(self._symbol, Resolution.SECOND)
     ```

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the TickConsolidator type constructor to `timedelta(hours=9, minutes=30)`

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone . If you subscribe to tick resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to tick resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks[self._symbol]
    for tick in ticks:
        self._consolidator.update(tick)

# Example 2: Update the consolidator with data from a history request
ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
for tick in ticks:
    self._consolidator.update(tick)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `Consolidate` helper method to create period consolidators and register them for automatic updates. With just one line of code, you can create data in any time period based on a `timedelta` or `Resolution` object:

- `timedelta` Periods

```
self._consolidator = self.consolidate(self._symbol, timedelta(milliseconds=100), self._consolidation_handler)
```

- `Resolution` Periods

```
self._consolidator = self.consolidate(self._symbol, Resolution.SECOND, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    pass
```

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `QuoteBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on a period of time:

1. Create the consolidator.

   To set the time period for the consolidator, you can use either a `timedelta` or `Resolution` object. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   - `timedelta` Periods

     ```
     self._consolidator = TickQuoteBarConsolidator(timedelta(milliseconds=100))
     # Aliases:
     # self._consolidator = self.create_consolidator(timedelta(milliseconds=100), Tick, TickType.QUOTE)
     # self._consolidator = self.resolve_consolidator(self._symbol, timedelta(milliseconds=100))
     ```

     The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

     Use the `start_time` parameter of the TickQuoteBarConsolidator type constructor to define a custom start time.

     ```
     self._consolidator = TickQuoteBarConsolidator(timedelta(milliseconds=100),
         start_time=timedelta(hours=9, minutes=30))
     ```

   - `Resolution` Periods

     The `Resolution` enumeration has the following members:

     ```
     self._consolidator = self.resolve_consolidator(self._symbol, Resolution.SECOND)
     ```

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the TickQuoteBarConsolidator type constructor to `timedelta(hours=9, minutes=30)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method

executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone . If you subscribe to tick resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to tick resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks[self._symbol]
    for tick in ticks:
        self._consolidator.update(tick)

# Example 2: Update the consolidator with data from a history request
ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
for tick in ticks:
    self._consolidator.update(tick)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `Consolidate` helper method to create period consolidators and register them for automatic updates. With just one line of code, you can create data in any time period based on a `timedelta` or `Resolution` object:

- `timedelta` Periods

```
self._consolidator = self.consolidate(self._symbol, timedelta(milliseconds=100), TickType.Quote, self._consolidation_handler)
```

- `Resolution` Periods

```
self._consolidator = self.consolidate(self._symbol, Resolution.SECOND, TickType.Quote, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: QuoteBar) -> None:
    pass
```

Consolidate Other Data

`<dataType>` consolidators aggregate various types of data objects into `<dataType>` objects of the same size or larger. If you consolidate custom data or alternative datasets, check the definition of the data class to see its data type. Follow these steps to create and manage a `<dataType>` consolidator based on a period of time:

1. Create the consolidator.

   To set the time period for the consolidator, you can use either a `timedelta` or `Resolution` object. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   - `timedelta` Periods

     ```
     self._consolidator = DynamicDataConsolidator(timedelta(days=1))
     # Aliases:
     # self._consolidator = self.create_consolidator(timedelta(days=1), <dataType>)
     # self._consolidator = self.resolve_consolidator(self._symbol, timedelta(days=1))
     ```

     The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

     Use the `start_time` parameter of the DynamicDataConsolidator type constructor to define a custom start time.

     ```
     self._consolidator = DynamicDataConsolidator(timedelta(days=1),
         start_time=timedelta(hours=9, minutes=30))
     ```

   - `Resolution` Periods

     The `Resolution` enumeration has the following members:

     ```
     self._consolidator = self.resolve_consolidator(self._symbol, Resolution.DAILY)
     ```

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the DynamicDataConsolidator type constructor to `timedelta(hours=9, minutes=30)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: <dataType>) -> None:
    pass
```

   The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone .

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the data subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `<dataType>` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         if self._symbol in slice:
             self._consolidator.update(slice[self._symbol])

     # Example 2: Update the consolidator with data from a history request
     history = self.history[<dataType>](self._symbol, 30, Resolution.DAILY)
     for data in history:
         self._consolidator.update(data)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `Consolidate` helper method to create period consolidators and register them for automatic updates. With just one line of code, you can create data in any time period based on a `timedelta` or `Resolution` object:

- `timedelta` Periods

  ```
  self._consolidator = self.consolidate(self._symbol, timedelta(days=1), self._consolidation_handler)
  ```

- `Resolution` Periods

  ```
  self._consolidator = self.consolidate(self._symbol, Resolution.DAILY, self._consolidation_handler)
  ```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: <dataType>) -> None:
    pass
```

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate common practices for implementing time-period consolidators.

Example 1: RSI-MACD Trading

The following algorithm utilizes a 5-minute trade bar consolidator to feed the smoothed trade bar to an RSI indicator and a 15-minute quote bar consolidator to feed the smoothed quote bar to a MACD indicator, generating trade entry signals during an uptrend and oversold conditions while exiting when the trend turns down.

```py
class UniverseSettingsAlgorithm(QCAlgorithm):
    # Create RSI and MACD indicators for trade signal generation.
    _rsi = RelativeStrengthIndex(14)
    _macd = MovingAverageConvergenceDivergence(12, 26, 9)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data to feed to indicators and trade.
        self.spy = self.add_equity("SPY").symbol

        # Create a 5-minute trade bar consolidator to feed the smoothed bar to the RSI indicator for the price action entry signal.
        rsi_consolidator = TradeBarConsolidator(timedelta(minutes=5))
        # Create a 15-minute quote bar consolidator to feed the smoothed bar to the MACD consolidator for trend entry/exit signal.
        macd_consolidator = QuoteBarConsolidator(timedelta(minutes=15))

        # Subscribe the consolidators to SPY data to automatically update the indicators.
        self.register_indicator(self.spy, self._rsi, rsi_consolidator)
        self.register_indicator(self.spy, self._macd, macd_consolidator)

        # Warm up the indicators by updating the consolidators with historical data.
        trade_bars = self.history[TradeBar](self.spy, 5 * self._rsi.warm_up_period, Resolution.MINUTE)
        for bar in trade_bars:
            rsi_consolidator.update(bar)
        quote_bars = self.history[QuoteBar](self.spy, 15 * self._macd.warm_up_period, Resolution.MINUTE)
        for bar in quote_bars:
            macd_consolidator.update(bar)

    def on_data(self, slice: Slice) -> None:
        # Entry rule: MACD histogram is positive (positive trend) and RSI is below 30 (oversold) to maximize return.
        if self._rsi.current.value < 30 and self._macd.histogram.current.value > 0 and not self.portfolio[self.spy].is_long:
            self.set_holdings(self.spy, 0.5)
        # Exit rule: MACD histogram is below 0 (negative trend)
        elif self._macd.histogram.current.value < 0 and self.portfolio[self.spy].is_long:
            self.liquidate(self.spy)
```

Example 2: Create Daily `QuoteBar` Objects That Start at 5 PM

The following algorithm consolidates minute resolution Forex data into daily `QuoteBar` objects that open and close at 5 PM Eastern Time.

```py
class CustomStartTimeConsolidatorExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the USDJPY Forex pair.
        self._pair = self.add_forex("USDJPY")
        # Create an EMA indicator for the pair.
        self._pair.ema = ExponentialMovingAverage(10)
        # Create a QuoteBar consolidator with a custom start time.
        consolidator = QuoteBarConsolidator(timedelta(1), timedelta(hours=17))
        # Attach a consolidation handler that will receive the consolidated bars.
        consolidator.data_consolidated += self._consolidation_handler
        # Subscribe the consolidator for automatic updates with the prices of the pair.
        self.subscription_manager.add_consolidator(self._pair.symbol, consolidator)
        # Register the indicator for automatic updates with the consolidated bars.
        self.register_indicator(self._pair.symbol, self._pair.ema, consolidator)
        # Warm up the consolidator and indicator.
        history = self.history[QuoteBar](self._pair.symbol, 29000, Resolution.MINUTE)
        for bar in history:
            consolidator.update(bar)

    def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
        # Wait until the indicator is ready and the algorithm is running.
        if not self._pair.ema.is_ready or self.is_warming_up:
            return
        # Plot the closing price and the EMA.
        self.plot(consolidated_bar.symbol.value, 'Close', consolidated_bar.close)
        self.plot(consolidated_bar.symbol.value, 'EMA', self._pair.ema.current.value)

        if not self._pair.holdings.is_long and consolidated_bar.close > self._pair.ema.current.value:
            self.set_holdings(self._pair.symbol, 1)
        if consolidated_bar.close < self._pair.ema.current.value:
            self.liquidate()
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
BasicTemplateOptionsConsolidationAlgorithm.py Python DataConsolidationAlgorithm.py Python MultipleSymbolConsolidationAlgorithm.py Python

Consolidator Types

Calendar Consolidators

Introduction

Calendar consolidators aggregate data based on specific periods of time like a weekly basis, quarterly basis, or any schedule with specific start and end times. In contrast to time period consolidators, the bars that calendar consolidators produce always end on the same schedule, regardless of the first data point the consolidator receives.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `TradeBar` objects of the same size or larger. Follow these steps to create and manage a `TradeBar` consolidator based on custom start and end periods:

1. Create the consolidator.

   To set the time period for the consolidator, you can use the built-in `CalendarInfo` objects or create your own. The following list describes each technique:

   - Standard Periods

     The following table describes the helper methods that the `Calendar` class provides to create the built-in `CalendarInfo` objects:

Method	Description
`Calendar.WEEKLY`	Computes the start of week (previous Monday) of the given date/time
`Calendar.MONTHLY`	Computes the start of month (1st of the current month) of the given date/time
`Calendar.QUARTERLY`	Computes the start of quarter (1st of the starting month of the current quarter) of the given date/time
`Calendar.YEARLY`	Computes the start of year (1st of the current year) of the given date/time

     ```
     self._consolidator = TradeBarConsolidator(Calendar.WEEKLY)
     # Alias:
     # self._consolidator = self.create_consolidator(Calendar.WEEKLY, TradeBar)
     ```

     PY

   - Custom Periods

     If you need something more specific than the preceding time periods, define a method to set the start time and period of the consolidated bars. The method should receive a `datetime` object that's based in the exchange time zone and should return a `CalendarInfo` object, which contains the start time of the bar in the exchange time zone and the duration of the consolidation period. The following example demonstrates how to create a custom consolidator for weekly bars:

```
# Define a consolidation period method
def _consolidation_period(self, dt: datetime) -> CalendarInfo:
    period = timedelta(7)

    dt = dt.replace(hour=17, minute=0, second=0, microsecond=0)
    delta = 1+dt.weekday()
    if delta > 6:
        delta = 0
    start = dt-timedelta(delta)

    return CalendarInfo(start, period)

# Create the consolidator with the consolidation period method
self._consolidator = TradeBarConsolidator(self._consolidation_period)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    pass
```

If you use a custom consolidation period method, LEAN passes the consolidated bar to the consolidation handler when the consolidation period ends. The `time` and `end_time` properties of the consolidated bar reflect the exchange time zone, but the `time` property of the algorithm still reflects the algorithm time zone .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      trade_bar = slice.bars[self._symbol]
      self._consolidator.update(trade_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
  for trade_bar in history:
      self._consolidator.update(trade_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the

security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `consolidate` helper method to create calendar consolidators and register them for automatic updates.

```
self._consolidator = self.consolidate(self._symbol, Calendar.WEEKLY, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    pass
```

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `QuoteBar` objects of the same size or larger. Follow these steps to create and manage a `QuoteBar` consolidator based on custom start and end periods:

1. Create the consolidator.

   To set the time period for the consolidator, you can use the built-in `CalendarInfo` objects or create your own. The following list describes each technique:

   - Standard Periods

     The following table describes the helper methods that the `Calendar` class provides to create the built-in `CalendarInfo` objects:

Method	Description
`Calendar.WEEKLY`	Computes the start of week (previous Monday) of the given date/time
`Calendar.MONTHLY`	Computes the start of month (1st of the current month) of the given date/time
`Calendar.QUARTERLY`	Computes the start of quarter (1st of the starting month of the current quarter) of the given date/time
`Calendar.YEARLY`	Computes the start of year (1st of the current year) of the given date/time

     ```
     self._consolidator = QuoteBarConsolidator(Calendar.WEEKLY)
     # Alias:
     # self._consolidator = self.create_consolidator(Calendar.WEEKLY, QuoteBar)
     ```

   - Custom Periods

     If you need something more specific than the preceding time periods, define a method to set the start time and period of the consolidated bars. The method should receive a `datetime` object that's based in the exchange time zone and should return a `CalendarInfo` object, which contains the start time of the bar in the exchange time zone and the duration of the consolidation period. The following example demonstrates how to create a custom consolidator for weekly bars:

```
# Define a consolidation period method
def _consolidation_period(self, dt: datetime) -> CalendarInfo:
    period = timedelta(7)

    dt = dt.replace(hour=17, minute=0, second=0, microsecond=0)
    delta = 1+dt.weekday()
    if delta > 6:
        delta = 0
    start = dt-timedelta(delta)

    return CalendarInfo(start, period)

# Create the consolidator with the consolidation period method
self._consolidator = QuoteBarConsolidator(self._consolidation_period)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

If you use a custom consolidation period method, LEAN passes the consolidated bar to the consolidation handler when the consolidation period ends. The `time` and `end_time` properties of the consolidated bar reflect the exchange time zone, but the `time` property of the algorithm still reflects the algorithm time zone .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      quote_bar = slice.quote_bars[self._symbol]
      self._consolidator.update(quote_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
  for quote_bar in history:
      self._consolidator.update(quote_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the

security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `consolidate` helper method to create calendar consolidators and register them for automatic updates.

```
self._consolidator = self.consolidate(self._symbol, Calendar.WEEKLY, TickType.QUOTE, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: QuoteBar) -> None:
    pass
```

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `TradeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on custom start and end periods:

1. Create the consolidator.

   To set the time period for the consolidator, you can use the built-in `CalendarInfo` objects or create your own. The following list describes each technique:

   - Standard Periods

     The following table describes the helper methods that the `Calendar` class provides to create the built-in `CalendarInfo` objects:

Method	Description
`Calendar.WEEKLY`	Computes the start of week (previous Monday) of the given date/time
`Calendar.MONTHLY`	Computes the start of month (1st of the current month) of the given date/time
`Calendar.QUARTERLY`	Computes the start of quarter (1st of the starting month of the current quarter) of the given date/time
`Calendar.YEARLY`	Computes the start of year (1st of the current year) of the given date/time

     ```
     self._consolidator = TickConsolidator(Calendar.WEEKLY)
     # Alias:
     # self._consolidator = self.create_consolidator(Calendar.WEEKLY, Tick)
     ```

   - Custom Periods

     If you need something more specific than the preceding time periods, define a method to set the start time and period of the consolidated bars. The method should receive a `datetime` object that's based in the exchange time zone and should return a `CalendarInfo` object, which contains the start time of the bar in the exchange time zone and the duration of the consolidation period. The following example demonstrates how to create a custom consolidator for weekly bars:

```
# Define a consolidation period method
def _consolidation_period(self, dt: datetime) -> CalendarInfo:
    period = timedelta(7)

    dt = dt.replace(hour=17, minute=0, second=0, microsecond=0)
    delta = 1+dt.weekday()
    if delta > 6:
        delta = 0
    start = dt-timedelta(delta)

    return CalendarInfo(start, period)

# Create the consolidator with the consolidation period method
self._consolidator = TickConsolidator(self._consolidation_period)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    pass
```

If you use a custom consolidation period method, LEAN passes the consolidated bar to the consolidation handler when the consolidation period ends. The `time` and `end_time` properties of the consolidated bar reflect the exchange time zone, but the `time` property of the algorithm still reflects the algorithm time zone .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `consolidate` helper method to create calendar consolidators and register them for automatic updates.

```
self._consolidator = self.consolidate(self._symbol, Calendar.WEEKLY, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    pass
```

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `QuoteBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on custom start and end periods:

1. Create the consolidator.

   To set the time period for the consolidator, you can use the built-in `CalendarInfo` objects or create your own. The following list describes each technique:

   - Standard Periods

     The following table describes the helper methods that the `Calendar` class provides to create the built-in `CalendarInfo` objects:

Method	Description
`Calendar.WEEKLY`	Computes the start of week (previous Monday) of the given date/time
`Calendar.MONTHLY`	Computes the start of month (1st of the current month) of the given date/time
`Calendar.QUARTERLY`	Computes the start of quarter (1st of the starting month of the current quarter) of the given date/time
`Calendar.YEARLY`	Computes the start of year (1st of the current year) of the given date/time

     ```
     self._consolidator = TickQuoteBarConsolidator(Calendar.WEEKLY)
     # Alias:
     # self._consolidator = self.create_consolidator(Calendar.WEEKLY, Tick, TickType.QUOTE)
     ```

   - Custom Periods

     If you need something more specific than the preceding time periods, define a method to set the start time and period of the consolidated bars. The method should receive a `datetime` object that's based in the exchange time zone and should return a `CalendarInfo` object, which contains the start time of the bar in the exchange time zone and the duration of the consolidation period. The following example demonstrates how to create a custom consolidator for weekly bars:

```
# Define a consolidation period method
def _consolidation_period(self, dt: datetime) -> CalendarInfo:
    period = timedelta(7)

    dt = dt.replace(hour=17, minute=0, second=0, microsecond=0)
    delta = 1+dt.weekday()
    if delta > 6:
        delta = 0
    start = dt-timedelta(delta)

    return CalendarInfo(start, period)

# Create the consolidator with the consolidation period method
self._consolidator = TickQuoteBarConsolidator(self._consolidation_period)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

If you use a custom consolidation period method, LEAN passes the consolidated bar to the consolidation handler when the consolidation period ends. The `time` and `end_time` properties of the consolidated bar reflect the exchange time zone, but the `time` property of the algorithm still reflects the algorithm time zone .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `consolidate` helper method to create calendar consolidators and register them for automatic updates.

```
self._consolidator = self.consolidate(self._symbol, Calendar.WEEKLY, TickType.QUOTE, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: QuoteBar) -> None:
    pass
```

Consolidate Other Data

`<dataType>` consolidators aggregate various types of data objects into `<dataType>` objects of the same size or larger. If you consolidate custom data or alternative datasets, check the definition of the data class to see its data type. Follow these steps to create and manage a `<dataType>` consolidator based on custom start and end periods:

1. Create the consolidator.

   To set the time period for the consolidator, you can use the built-in `CalendarInfo` objects or create your own. The following list describes each technique:

   - Standard Periods

     The following table describes the helper methods that the `Calendar` class provides to create the built-in `CalendarInfo` objects:

Method	Description
`Calendar.WEEKLY`	Computes the start of week (previous Monday) of the given date/time
`Calendar.MONTHLY`	Computes the start of month (1st of the current month) of the given date/time
`Calendar.QUARTERLY`	Computes the start of quarter (1st of the starting month of the current quarter) of the given date/time
`Calendar.YEARLY`	Computes the start of year (1st of the current year) of the given date/time

     ```
     self._consolidator = DynamicDataConsolidator(Calendar.WEEKLY)
     # Alias:
     # self._consolidator = self.create_consolidator(Calendar.WEEKLY, <dataType>)
     ```

   - Custom Periods

     If you need something more specific than the preceding time periods, define a method to set the start time and period of the consolidated bars. The method should receive a `datetime` object that's based in the exchange time zone and should return a `CalendarInfo` object, which contains the start time of the bar in the exchange time zone and the duration of the consolidation period. The following example demonstrates how to create a custom consolidator for weekly bars:

```
# Define a consolidation period method
def _consolidation_period(self, dt: datetime) -> CalendarInfo:
    period = timedelta(7)

    dt = dt.replace(hour=17, minute=0, second=0, microsecond=0)
    delta = 1+dt.weekday()
    if delta > 6:
        delta = 0
    start = dt-timedelta(delta)

    return CalendarInfo(start, period)

# Create the consolidator with the consolidation period method
self._consolidator = DynamicDataConsolidator(self._consolidation_period)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: <dataType>) -> None:
    pass
```

If you use a custom consolidation period method, LEAN passes the consolidated bar to the consolidation handler when the consolidation period ends. The `time` and `end_time` properties of the consolidated bar reflect the exchange time zone, but the `time` property of the algorithm still reflects the algorithm time zone .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the data subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `<dataType>` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      if self._symbol in slice:
          self._consolidator.update(slice[self._symbol])

  # Example 2: Update the consolidator with data from a history request
  history = self.history[<dataType>](self._symbol, 30, Resolution.DAILY)
  for data in history:
      self._consolidator.update(data)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the

security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

You can also use the `consolidate` helper method to create calendar consolidators and register them for automatic updates.

```
self._consolidator = self.consolidate(self._symbol, Calendar.WEEKLY, self._consolidation_handler)
```

If you use the `consolidate` helper method, the consolidation handler doesn't receive an `object` argument.

```
def _consolidation_handler(self, consolidated_bar: <dataType>) -> None:
    pass
```

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate some common practices for calendar consolidators.

Example 1: Daily Futures Bars

Daily bars from the CME and Yahoo Finance represent the price action from 5 PM Central Time (CT) to 4 PM CT on the following day. In contrast, the regular daily resolution bars on QuantConnect represent the price action from 12 AM Eastern Time (ET) to 12 AM ET the following day. To create daily bars that represent the same period of time as the daily bars from the CME, the following algorithm consolidates minute bars (during regular and extended market hours) into daily bars that span from 5 PM CT to 4 PM CT on the following day. The algorithm then uses these consolidated daily bars to place take profit and stop loss orders as a function of the daily Average True Range .

```py
class CalendarConsolidatorExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Create an ATR indicator to set take profit and stop loss levels.
        self._atr = AverageTrueRange(10)
        # Create a day tracker to avoid overtrading.
        self._day = -1
        # Add ES Futures data and configure the continuous contract settings
        # so you can update the ATR with a smooth feed of prices.
        self._future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            extended_market_hours=True,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            contract_depth_offset=0
        )
        # Create a calendar consolidator to generate daily ES Futures bars
        # since market starts at 6pm EST to 5pm EST next day.
        consolidator = TradeBarConsolidator(self._future_trading_hour)
        consolidator.data_consolidated += self._on_consolidated
        # Warm up the consolidator and ATR indicator.
        history = self.history[TradeBar](self._future.symbol, 14500, Resolution.MINUTE)
        for bar in history:
            consolidator.update(bar)
        # Subscribe the consolidator for automatic updates with ES data.
        self.subscription_manager.add_consolidator(self._future.symbol, consolidator)

    def _future_trading_hour(self, dt: datetime) -> CalendarInfo:
        # Set the open time of the bar to be 6 PM EST.
        start = dt.replace(hour=0, minute=0, second=0, microsecond=0)
        if dt.hour < 18:
            start -= timedelta(hours=6)
        else:
            start += timedelta(hours=18)
        # Set the close time of the bar to be 5 PM EST the next day, which is 23 hours later.
        return CalendarInfo(start, timedelta(hours=23))

    def _on_consolidated(self, sender: object, bar: TradeBar) -> None:
        # Update the ATR indicator since we want the daily ATR to set the stop price.
        self._atr.update(bar)
        # Set the daily take profit and stop price levels as a function of the previous
        # close price and the ATR.
        self._take_profit_price = bar.close + self._atr.current.value * 2
        self._stop_loss_price = bar.close - self._atr.current.value * 2

    def on_data(self, slice: Slice) -> None:
        # Liquidate the current contract in the continuous contract series
        # when it's about to expire.
        changed_event = slice.symbol_changed_events.get(self._future.symbol)
        if changed_event:
            self.liquidate(changed_event.old_symbol)

        # If we're not invested, buy the current contract in the continuous contract series.
        if (not self.portfolio.invested and
            self.securities[self._future.mapped].price and
            self._day != slice.time.day):
            self.market_order(self._future.mapped, 1)
        # If we're already invested, check if we've hit the TP/SL level.
        elif self.portfolio.invested and self._future.symbol in slice.bars:
            current_price = slice.bars[self._future.symbol].close
            # Check if the current price is above profit taker or below stop loss.
            if current_price > self._take_profit_price or current_price < self._stop_loss_price:
                # Exit the position.
                self.liquidate()

        # Update the daily tracker so we only update our position once per day.
        self._day = slice.time.day
```

Example 2: Create Daily `QuoteBar` Objects That Start at 5 PM

The following algorithm consolidates minute resolution Forex data into daily `QuoteBar` objects that open and close at 5 PM Eastern Time.

```py
class CalendarConsolidatorExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add the USDJPY Forex pair.
        self._pair = self.add_forex("USDJPY")
        # Create an EMA indicator for the pair.
        self._pair.ema = ExponentialMovingAverage(10)
        # Create a QuoteBar consolidator with a custom consolidation period.
        consolidator = QuoteBarConsolidator(self._daily_forex_consolidation_period)
        # Attach a consolidation handler that will receive the consolidated bars.
        consolidator.data_consolidated += self._consolidation_handler
        # Subscribe the consolidator for automatic updates with the prices of the pair.
        self.subscription_manager.add_consolidator(self._pair.symbol, consolidator)
        # Register the indicator for automatic updates with the consolidated bars.
        self.register_indicator(self._pair.symbol, self._pair.ema, consolidator)
        # Warm up the consolidator and indicator.
        history = self.history[QuoteBar](self._pair.symbol, 29000, Resolution.MINUTE)
        for bar in history:
            consolidator.update(bar)

    # Define the consolidation period.
    def _daily_forex_consolidation_period(self, dt: datetime) -> CalendarInfo:
        # Set the start of the bar to be 5 PM ET.
        start = dt.replace(hour=0, minute=0, second=0, microsecond=0)
        if dt.hour < 17:
            start -= timedelta(hours=7)
        else:
            start += timedelta(hours=17)
        # Set the end of the bar to be 5 PM ET the next day.
        return CalendarInfo(start, timedelta(1))

    def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
        # Wait until the indicator is ready and the algorithm is running.
        if not self._pair.ema.is_ready or self.is_warming_up:
            return
        # Plot the closing price and the EMA.
        self.plot(str(consolidated_bar.symbol), 'Close', consolidated_bar.close)
        self.plot(str(consolidated_bar.symbol), 'EMA', self._pair.ema.current.value)
```

For a version of this algorithm with Time Period Consolidators, see Time Period Consolidators > Examples .

Example 3: Monthly Candlesticks

The following algorithm consolidates minute-resolution data into monthly `TradeBar` objects and then plots them as candlesticks .

```py
from dateutil.relativedelta import relativedelta


class CalendarConsolidatorExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add an asset.
        symbol = self.add_equity("SPY").symbol
        # Create a consolidator that produces monthly TradeBar objects.
        # This is the stardard way:
        #consolidator = TradeBarConsolidator(Calendar.MONTHLY)
        # However, you can acomplish the same thing with a custom period:
        consolidator = TradeBarConsolidator(self._custom_monthly)
        # Add a consolidation handler that plots the monthly bars as candlesticks.
        consolidator.data_consolidated += lambda _, bar: self.plot(
            "Monthly Candlestick", symbol.value, bar.open, bar.high, bar.low, bar.close
        )
        # Register the consolidator for automatic updates.
        self.subscription_manager.add_consolidator(symbol, consolidator)

    def _custom_monthly(self, dt: datetime) -> CalendarInfo:
        # Set the start time to the beginning of the month.
        start = dt.replace(day=1).date()
        # Set the end time to the beginning of next month.
        end = start + relativedelta(months=1)
        return CalendarInfo(start, end - start)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
DataConsolidationAlgorithm.py Python

Consolidator Types

Count Consolidators

Introduction

Count consolidators aggregate data based on a number of bars or ticks. This type of consolidator aggregates n samples together, regardless of the time period the samples cover. Managing count consolidators is similiar to managing time period consolidators , except you create count consolidators with an integer argument instead of a time-based argument.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `TradeBar` objects of the same size or larger. Follow these steps to create and manage a `TradeBar` consolidator based on a number of samples:

1. Create the consolidator.

   To create a count consolidator, pass the number of samples to the consolidator constructor.

   ```
   self._consolidator = TradeBarConsolidator(10)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
       pass
   ```

   When the consolidator receives the n -th data point, it passes the consolidated bar to the event handler.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    trade_bar = slice.bars[self._symbol]
    self._consolidator.update(trade_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
for trade_bar in history:
    self._consolidator.update(trade_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `QuoteBar` objects of the same size or larger. Follow these steps to create and manage a `QuoteBar` consolidator based on a number of samples:

1. Create the consolidator.

   To create a count consolidator, pass the number of samples to the consolidator constructor.

```
self._consolidator = QuoteBarConsolidator(10)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

   When the consolidator receives the n -th data point, it passes the consolidated bar to the event handler.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      quote_bar = slice.quote_bars[self._symbol]
      self._consolidator.update(quote_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
  for quote_bar in history:
      self._consolidator.update(quote_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `TradeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on a number of samples:

1. Create the consolidator.

   To create a count consolidator, pass the number of samples to the consolidator constructor.

   ```
   self._consolidator = TickConsolidator(10)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
       pass
   ```

   When the consolidator receives the n -th data point, it passes the consolidated bar to the event handler.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks[self._symbol]
    for tick in ticks:
        self._consolidator.update(tick)

# Example 2: Update the consolidator with data from a history request
ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
for tick in ticks:
    self._consolidator.update(tick)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `QuoteBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on a number of samples:

1. Create the consolidator.

To create a count consolidator, pass the number of samples to the consolidator constructor.

```
self._consolidator = TickQuoteBarConsolidator(10)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

When the consolidator receives the n -th data point, it passes the consolidated bar to the event handler.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         ticks = slice.ticks[self._symbol]
         for tick in ticks:
             self._consolidator.update(tick)

     # Example 2: Update the consolidator with data from a history request
     ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
     for tick in ticks:
         self._consolidator.update(tick)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Other Data

`<dataType>` consolidators aggregate various types of data objects into `<dataType>` objects of the same size or larger. If you consolidate custom data or alternative datasets, check the definition of the data class to see its data type. Follow these steps to create and manage a `<dataType>` consolidator based on a number of samples:

1. Create the consolidator.

   To create a count consolidator, pass the number of samples to the consolidator constructor.

   ```
   self._consolidator = DynamicDataConsolidator(10)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method

executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: <dataType>) -> None:
    pass
```

When the consolidator receives the n -th data point, it passes the consolidated bar to the event handler.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the data subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `<dataType>` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice:
        self._consolidator.update(slice[self._symbol])

# Example 2: Update the consolidator with data from a history request
history = self.history[<dataType>](self._symbol, 30, Resolution.DAILY)
for data in history:
    self._consolidator.update(data)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate common practices for implementing count consolidator.

Example 1: Tick Count Consolidator

```
class CountConsolidatorAlgorithm(QCAlgorithm):
    # Create EMA indicators for trade signal generation.
    _ema = ExponentialMovingAverage(14)
    # The _day variable controls trade daily when open.
    _day = -1

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 4)
        self.set_end_date(2024, 9, 5)

        # Request SPY data to feed to indicators and trade.
        self.spy = self.add_equity("SPY", Resolution.TICK).symbol

        # Create a 10000 trade tick consolidator to feed the "volume" bar to the EMA indicator for trend entry/exit signal.
        tick_consolidator = TickConsolidator(10000)

        # Subscribe the consolidators to SPY data to automatically update the indicators.
        self.register_indicator(self.spy, self._ema, tick_consolidator)

        self.set_warm_up(120)

    def on_data(self, slice: Slice) -> None:
        ticks = slice.ticks.get(self.spy)
        if self._ema.is_ready and ticks and self._day != slice.time.day:
            # Obtain the latest quote price from the latest tick.
            last_tick = sorted(ticks, key=lambda x: x.end_time, reverse=True)[0]

            ema = self._ema.current.value

            # Trade the trend by EMA crosses.
            if last_tick.price > ema and not self.portfolio[self.spy].is_long:
                self.set_holdings(self.spy, 0.5)
            elif last_tick.price < ema and not self.portfolio[self.spy].is_short:
                self.set_holdings(self.spy, -0.5)

            self._day = slice.time.day
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
DataConsolidationAlgorithm.py Python

Consolidator Types

Mixed-Mode Consolidators

Introduction

Mixed-mode consolidators are a combination of count consolidators and time period consolidators. This type of consolidator aggregates n samples together or aggregates samples over a specific time period, whichever happens first.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `TradeBar` objects of the same size or larger. Follow these steps to create and manage a `TradeBar` consolidator based on a period of time or a number of samples, whichever occurs first:

1. Create the consolidator.

   To create a mixed-mode consolidator, pass a `timedelta` object and an integer to the consolidator constructor. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   ```
   self._consolidator = TradeBarConsolidator(10, timedelta(days=1))
   ```
   PY

   The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the type constructor to `timedelta( timeDeltaStartTime?>)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```
   PY

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
       pass
   ```
   PY

   The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone or the number of samples, whichever occurs first. If you subscribe to minute resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to minute resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription

Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      trade_bar = slice.bars[self._symbol]
      self._consolidator.update(trade_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
  for trade_bar in history:
      self._consolidator.update(trade_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `QuoteBar` objects of the same size or larger. Follow these steps to create and manage a `QuoteBar` consolidator based on a period of time or a number of samples, whichever occurs first:

1. Create the consolidator.

   To create a mixed-mode consolidator, pass a `timedelta` object and an integer to the consolidator constructor. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   ```
   self._consolidator = QuoteBarConsolidator(10, timedelta(days=1))
   ```

   The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the type constructor to `timedelta( timeDeltaStartTime?>)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event

system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone or the number of samples, whichever occurs first. If you subscribe to minute resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to minute resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    quote_bar = slice.quote_bars[self._symbol]
    self._consolidator.update(quote_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
for quote_bar in history:
    self._consolidator.update(quote_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `TradeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on a period of time or a number of samples, whichever occurs first:

1. Create the consolidator.

To create a mixed-mode consolidator, pass a `timedelta` object and an integer to the consolidator constructor. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

```
self._consolidator = TickConsolidator(10, timedelta(milliseconds=100))
```

The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the type constructor to `timedelta( timeDeltaStartTime?>)`

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone or the number of samples, whichever occurs first. If you subscribe to tick resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to tick resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the

security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `QuoteBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on a period of time or a number of samples, whichever occurs first:

1. Create the consolidator.

   To create a mixed-mode consolidator, pass a `timedelta` object and an integer to the consolidator constructor. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   ```
   self._consolidator = TickQuoteBarConsolidator(10, timedelta(milliseconds=100))
   ```

   The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the type constructor to `timedelta( timeDeltaStartTime?>)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: QuoteBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone or the number of samples, whichever occurs first. If you subscribe to tick resolution data for Bitcoin and create an hourly consolidator, you receive consolidated bars at the top of each hour. However, if you subscribe to tick resolution data for the regular trading hours of US Equities and create a daily consolidator, you receive consolidated bars at market close or midnight, depending on your `daily_precise_end_time` setting .

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Other Data

`<dataType>` consolidators aggregate various types of data objects into `<dataType>` objects of the same size or larger. If you consolidate custom data or alternative datasets, check the definition of the data class to see its data type. Follow these steps to create and manage a `<dataType>` consolidator based on a period of time or a number of samples, whichever occurs first:

1. Create the consolidator.

   To create a mixed-mode consolidator, pass a `timedelta` object and an integer to the consolidator constructor. The consolidator time period must be greater than or equal to the resolution of the security subscription. For instance, you can aggregate minute bars into 10-minute bars, but you can't aggregate hour bars into 10-minute bars.

   ```
   self._consolidator = DynamicDataConsolidator(10, timedelta(days=1))
   ```

   The time period is relative to the exchange time zone , not the algorithm time zone . If you consolidate Crypto data into daily bars, the event handler receives the consolidated bars at midnight 12:00 AM Coordinated Universal Time (UTC), regardless of the algorithm time zone.

   If you consolidate on an hourly basis, the consolidator ends at the top of the hour, not every hour after the market open. For US Equities, that's 10 AM Eastern Time (ET), not 10:30 AM ET. To change this behavior, set the `start_time` parameter of the type constructor to `timedelta( timeDeltaStartTime?>)`

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event

system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: <dataType>) -> None:
    pass
```

The consolidation event handler receives bars when the consolidated bar closes based on the exchange time zone or the number of samples, whichever occurs first.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the data subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `<dataType>` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice:
        self._consolidator.update(slice[self._symbol])

# Example 2: Update the consolidator with data from a history request
history = self.history[<dataType>](self._symbol, 30, Resolution.DAILY)
for data in history:
    self._consolidator.update(data)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate common practices for implementing mixed-mode consolidators.

Example 1: Opening Range Breakout

The following algorithm demonstrates an opening range breakout strategy for BJK (a Gaming ETF). We create a 15-count/30-minute trade bar as the opening range for signal generation. If the close price at 10 am is higher than the high price of the aggregated opening bar, we long BJK and exit by a 2% trailing stop loss.

```
class MixedModeConsolidatorAlgorithm(QCAlgorithm):
    _high_price = 0

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request CARZ data to feed to the consolidators and trade.
        self._equity = self.add_equity("CARZ", Resolution.MINUTE).symbol

        # Create a 10-count-or-20-minute trade bar consolidator to create the opening range for trade signal generation.
        # CARZ is a less liquid ETF, we use a mixed-mode consolidator.
        self._consolidator = TradeBarConsolidator(10, timedelta(minutes=20))

        # Subscribe the consolidators to CARZ data for automatic updating.
        self.subscription_manager.add_consolidator(self._equity, self._consolidator)

        # We check if the entry condition is attained 30 minutes after the market opens.
        self.schedule.on(
            self.date_rules.every_day(self._equity),
            self.time_rules.after_market_open(self._equity, 30),
            self.entry
        )

        self.set_warm_up(1)

    def entry(self) -> None:
        # Check if the entry condition is met: the current price is higher than the high price of the open range.
        current = self.securities[self._equity].close
        if current > self._consolidator.consolidated.high:
            self.market_order(self._equity, 20)
            # Cache the current price for trailing stop loss need.
            self._high_price = current

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._equity)
        if bar and self.portfolio.invested:
            # Update the high price if the current high is higher to update the trailing stop price.
            if bar.high > self._high_price:
                self._high_price = bar.high

            # Exit position if the trailing stop loss is hit.
            if bar.close < self._high_price * 0.98:
                self.liquidate()
                self._high_price = 0
```

Consolidator Types

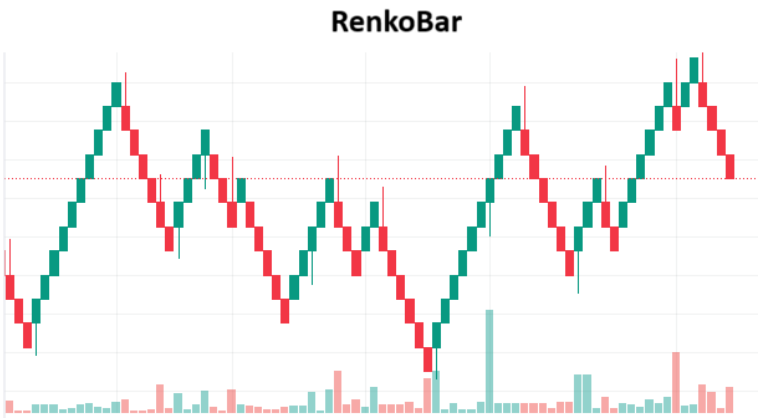
Renko Consolidators

Renko Consolidators

Renko Consolidators

Introduction

Most Renko consolidators aggregate bars based on a fixed price movement. The `RenkoConsolidator` produces Renko bars by their traditional definition. In the case of a $1 bar size, the `RenkoConsolidator` produces bars that have a body spanning 1. The opening price of the first bar is set to the closest 1 multiple of the first trade. When the price moves by at least 1, the first bar closes. If a bar is a rising bar, the following bar closes when the price moves 1 above the closing price of the previous bar or 1 below the opening price of the previous bar. If a bar is a falling bar, the following bar closes when the price moves 1 below the closing price of the previous bar or $1 above the opening price of the previous bar. If the price jumps multiple dollars in a single tick, the `RenkoConsolidator` produces multiple $1 bars in a single time step.



Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `RenkoBar` objects. Follow these steps to create and manage a `TradeBar` consolidator based on the traditional Renko bar rules:

1. Create the consolidator.

   To create a Renko consolidator, pass the bar size to the `RenkoConsolidator` constructor.

   ```
   # Create a Renko consolidator that emits a bar when the price moves $1
   self._consolidator = RenkoConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Renko bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         trade_bar = slice.bars[self._symbol]
         self._consolidator.update(trade_bar)

     # Example 2: Update the consolidator with data from a history request
     history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
     for trade_bar in history:
         self._consolidator.update(trade_bar)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `RenkoBar` objects. Follow these steps to create and manage a `QuoteBar` consolidator based on the traditional Renko bar rules:

1. Create the consolidator.

   To create a Renko consolidator, pass the bar size to the `RenkoConsolidator` constructor.

   ```
   # Create a Renko consolidator that emits a bar when the price moves $1
   self._consolidator = RenkoConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
PY
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Renko bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
PY
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
PY
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    quote_bar = slice.quote_bars[self._symbol]
    self._consolidator.update(quote_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
for quote_bar in history:
    self._consolidator.update(quote_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
PY
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `RenkoBar` objects. Follow these steps to create and manage a `Tick` consolidator based on the traditional Renko bar rules:

1. Create the consolidator.

To create a Renko consolidator, pass the bar size to the `RenkoConsolidator` constructor.

```
# Create a Renko consolidator that emits a bar when the price moves $1
self._consolidator = RenkoConsolidator(1)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Renko bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `RenkoBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on the traditional Renko bar rules:

1. Create the consolidator.

   To create a Renko consolidator, pass the bar size to the `RenkoConsolidator` constructor.

   ```
   # Create a Renko consolidator that emits a bar when the price moves $1
   self._consolidator = RenkoConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the price movement forms a new Renko bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         ticks = slice.ticks[self._symbol]
         for tick in ticks:
             self._consolidator.update(tick)

     # Example 2: Update the consolidator with data from a history request
     ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
     for tick in ticks:
         self._consolidator.update(tick)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Renko Consolidators

Classic Renko Consolidators

Introduction

Most Renko consolidators aggregate bars based on a fixed price movement. The `ClassicRenkoConsolidator` produces a different type of Renko bars than the `RenkoConsolidator` . A `ClassicRenkoConsolidator` with a bar size of `1` produces a new bar that spans `1` every time an asset closes $1 away from the close of the previous bar. If the price jumps multiple dollars in a single tick, the `ClassicRenkoConsolidator` only produces one bar per time step where the open of each bar matches the close of the previous bar.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `RenkoBar` objects. Follow these steps to create and manage a `TradeBar` consolidator based on the preceding Renko bar rules:

1. Create the consolidator.

   To create a classic Renko consolidator, pass the bar size to the `ClassicRenkoConsolidator` constructor.

   ```
   PY
   # Create a Classic Renko consolidator that emits a bar when the price moves $1
   self._consolidator = ClassicRenkoConsolidator(1)
   ```

   The `ClassicRenkoConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Renko bars
   - It ignores the volume of the input data
   - It enforces the open and close of each bar to be a multiple of the bar size

   To build the Renko bars with a different property than the `value` of the `IBaseData` object, provide a `selector` argument. The `selector` should be a function that receives the `IBaseData` object and returns a decimal value.

   ```
   PY
   self._consolidator = ClassicRenkoConsolidator(1, selector = lambda data: data.high)
   ```

   To add a non-zero `volume` property to the Renko bars, provide a `volume_selector` argument. The `volume_selector` should be a function that receives the `IBaseData` object and returns a decimal value.

   ```
   PY
   self._consolidator = ClassicRenkoConsolidator(1, volume_selector = lambda data: data.volume)
   ```

   To relax the requirement that the open and close of the Renko bars must be a multiple of bar size, disable the `even_bars` argument. If you disable `even_bars` , the open value of the first Renko bar is set to the first value from the `selector` . The following opening and closing Renko bar values are all multiples of the first value from the `selector`

   ```
   PY
   self._consolidator = ClassicRenkoConsolidator(1, even_bars=False)
   ```

2. Add an event handler to the consolidator.

   ```
   PY
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method

executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
PY
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new classic Renko bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
PY
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
PY
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    trade_bar = slice.bars[self._symbol]
    self._consolidator.update(trade_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
for trade_bar in history:
    self._consolidator.update(trade_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
PY
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `RenkoBar` objects. Follow these steps to create and manage a `QuoteBar` consolidator based on the preceding Renko bar rules:

1. Create the consolidator.

To create a classic Renko consolidator, pass the bar size to the `ClassicRenkoConsolidator` constructor.

```
PY
# Create a Classic Renko consolidator that emits a bar when the price moves $1
self._consolidator = ClassicRenkoConsolidator(1)
```

The `ClassicRenkoConsolidator` has the following default behavior:

- It uses the `value` property of the `IBaseData` object it receives to build the Renko bars
- It ignores the volume of the input data
- It enforces the open and close of each bar to be a multiple of the bar size

The following arguments enable you to create Renko bars that aggregate the excess liquidity on the bid.

```
self._consolidator = ClassicRenkoConsolidator(10, lambda data: data.value, lambda data: data.last_bid_size - data.last_ask_size)
```

To relax the requirement that the open and close of the Renko bars must be a multiple of bar size, disable the `even_bars` argument. If you disable `even_bars` , the open value of the first Renko bar is set to the first value from the `selector` . The following opening and closing Renko bar values are all multiples of the first value from the `selector`

```
self._consolidator = ClassicRenkoConsolidator(1, even_bars=False)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new classic Renko bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    quote_bar = slice.quote_bars[self._symbol]
    self._consolidator.update(quote_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
for quote_bar in history:
    self._consolidator.update(quote_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `RenkoBar` objects. Follow these steps to create and manage a `Tick` consolidator based on the preceding Renko bar rules:

1. Create the consolidator.

   To create a classic Renko consolidator, pass the bar size to the `ClassicRenkoConsolidator` constructor.

   ```
   # Create a Classic Renko consolidator that emits a bar when the price moves $1
   self._consolidator = ClassicRenkoConsolidator(1)
   ```

   The `ClassicRenkoConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Renko bars
   - It ignores the volume of the input data
   - It enforces the open and close of each bar to be a multiple of the bar size

   To relax the requirement that the open and close of the Renko bars must be a multiple of bar size, disable the `even_bars` argument. If you disable `even_bars` , the open value of the first Renko bar is set to the first value from the `selector` . The following opening and closing Renko bar values are all multiples of the first value from the `selector`

   ```
   self._consolidator = ClassicRenkoConsolidator(1, even_bars=False)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
       pass
   ```

The consolidation event handler receives bars when the price movement forms a new classic Renko bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         ticks = slice.ticks[self._symbol]
         for tick in ticks:
             self._consolidator.update(tick)

     # Example 2: Update the consolidator with data from a history request
     ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
     for tick in ticks:
         self._consolidator.update(tick)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `RenkoBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on the preceding Renko bar rules:

1. Create the consolidator.

   To create a classic Renko consolidator, pass the bar size to the `ClassicRenkoConsolidator` constructor.

   ```
   # Create a Classic Renko consolidator that emits a bar when the price moves $1
   self._consolidator = ClassicRenkoConsolidator(1)
   ```

   The `ClassicRenkoConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Renko bars
   - It ignores the volume of the input data
   - It enforces the open and close of each bar to be a multiple of the bar size

   To relax the requirement that the open and close of the Renko bars must be a multiple of bar size, disable the `even_bars` argument. If you disable `even_bars` , the open value of the first Renko bar is set to the first value from the `selector` . The following opening and closing Renko

bar values are all multiples of the first value from the `selector`

```
self._consolidator = ClassicRenkoConsolidator(1, even_bars=False)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RenkoBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new classic Renko bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate some common practices for implementing classic renko consolidator.

Example 1: 10 Dollar Bar Range SMA Cross

The following algorithm creates a dollar bar for SPY. Then, for each 10-dollar bar, we find their range (high - low) and feed to an SMA indicator, such that we obtain a trend indicator of the range of SPY. We trade VXX as the proxy of the SPY volatility based on the trend of the range SMA.

```py
class ClassicRenkoBarConsolidatorAlgorithm(QCAlgorithm):
    _high = 0
    _low = 10000
    _count = 0
    _sma = SimpleMovingAverage(10)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data to feed the consolidators and indicators for trade signal generation.
        self._spy = self.add_equity("SPY", Resolution.MINUTE).symbol
        # Request VXX data for trading.
        self._vxx = self.add_equity("VXX", Resolution.MINUTE).symbol

        # Create a dollar bar of SPY to track the volatility.
        self._consolidator = ClassicRenkoConsolidator(1)
        self._consolidator.data_consolidated += self.on_consolidated

        # Subscribe the consolidators to SPY data for automatic updating.
        self.subscription_manager.add_consolidator(self._spy, self._consolidator)

        self._sma.window.size = 2;
        self.set_warm_up(100)

    def on_consolidated(self, sender: object, bar: TradeBar) -> None:
        # Update the SPY range of the current batch.
        self._high = max(self._high, bar.high)
        self._low = min(self._low, bar.low)
        self._count += 1

        # Update SMA of SPY range of the last 10 Renko bars.
        if self._count >= 10:
            self._sma.update(bar.end_time, self._high - self._low)

            # Reset high and low.
            self._high = 0
            self._low = 10000
            self._count = 0

    def on_data(self, slice: Slice) -> None:
        if self._sma.is_ready:
            current = self._sma.current.value
            previous = self._sma.previous.value

            # If the range trend increases, buy VXX as a proxy for the increasing volatility.
            if current > previous and not self.portfolio[self._vxx].is_long:
                self.set_holdings(self._vxx, 0.5)
            # If the range trend decreases, sell VXX as a proxy for decreasing volatility.
            elif current < previous and not self.portfolio[self._vxx].is_short:
                self.set_holdings(self._vxx, -0.5)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
ClassicRenkoConsolidatorAlgorithm.py Python

Renko Consolidators

Volume Renko Consolidators

Introduction

Volume Renko consolidators aggregate bars based on a fixed trading volume. These types of bars are commonly known as volume bars. A `VolumeRenkoConsolidator` with a bar size of 10,000 produces a new bar every time 10,000 units of the security trades in the market. If the trading volume of a single time step exceeds two step sizes (i.e. >20,000), the `VolumeRenkoConsolidator` produces multiple bars in a single time step.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `VolumeRenkoBar` objects. Follow these steps to create and manage a `TradeBar` consolidator based on custom set volume traded:

1. Create the consolidator.

   To create a Volume Renko consolidator, pass the bar size to the `VolumeRenkoConsolidator` constructor.

   ```
   # Create a Volume Renko consolidator that emits a bar every time 10,000 units trade
   self._consolidator = VolumeRenkoConsolidator(10000)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: VolumeRenkoBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the trading volume forms a new Volume Renko bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    trade_bar = slice.bars[self._symbol]
    self._consolidator.update(trade_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
for trade_bar in history:
    self._consolidator.update(trade_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `VolumeRenkoBar` objects. Follow these steps to create and manage a `Tick` consolidator based on custom set volume traded:

1. Create the consolidator.

   To create a Volume Renko consolidator, pass the bar size to the `VolumeRenkoConsolidator` constructor.

   ```
   # Create a Volume Renko consolidator that emits a bar every time 10,000 units trade
   self._consolidator = VolumeRenkoConsolidator(10000)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: VolumeRenkoBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the trading volume forms a new Volume Renko bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  PY
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   PY
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
PY
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
PY
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

The following examples demonstrate common practices for implementing volume renko consolidator.

Example 1: Bollinger Bands From Volume Renko Bar

The following algorithm uses 100,000-share volume Renko bars to spot mean reversal opportunities on SPY with Bollinger Bands. Equal volume bars could provide more equal information in each bar, so the standard deviation and trade opportunities are more credible.

PY

```
class VolumeRenkoBarConsolidatorAlgorithm(QCAlgorithm):
    # We use a Bollinger band to signal mean reversal opportunities.
    _bbands = BollingerBands(20, 2, MovingAverageType.EXPONENTIAL)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data to feed for the consolidators, indicators, and trade signal generation.
        self._spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Create a 100,000-share bar of SPY to obtain a more equally-informed bar.
        self._consolidator = VolumeRenkoConsolidator(100000)

        # Subscribe the consolidators to SPY data to automatically update the indicator.
        self.register_indicator(self._spy, self._bbands, self._consolidator)

        # Warm up the indicator for immediate use.
        self.set_warm_up(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._spy) if not self.is_warming_up else None
        if self._bbands.is_ready and bar:
            upper_band = self._bbands.upper_band.current.value
            middle_band = self._bbands.middle_band.current.value
            lower_band = self._bbands.lower_band.current.value

            # Trade the mean reversal by short-selling SPY if the current price exceeds the upper threshold.
            if bar.close > upper_band and not self.portfolio[self._spy].is_long:
                self.set_holdings(self._spy, -0.5)
            # Trade the mean reversal by buying SPY if the price is below the lower threshold.
            elif bar.close < lower_band and not self.portfolio[self._spy].is_short:
                self.set_holdings(self._spy, 0.5)
            # Exit position if the price is mean-reverted that crosses the EMA reference.
            elif (self.portfolio[self._spy].quantity > 0 and bar.close > middle_band)\
            or (self.portfolio[self._spy].quantity < 0 and bar.close < middle_band):
                self.liquidate(self._spy)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
VolumeRenkoConsolidatorAlgorithm.py Python

Consolidator Types

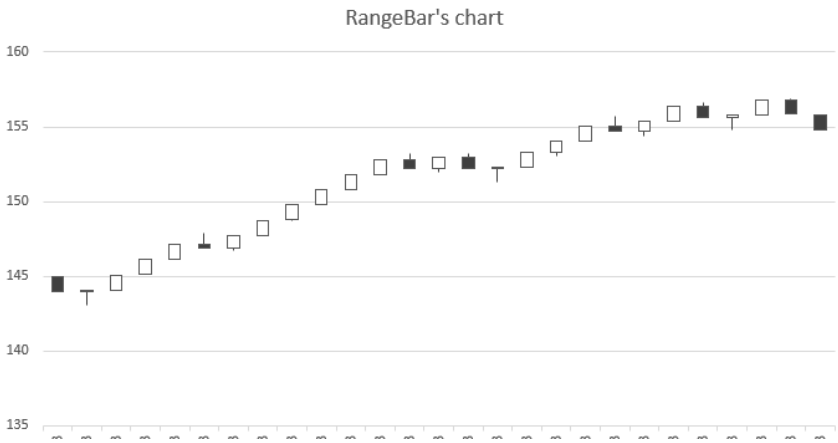
Range Consolidators

Range Consolidators

Range Consolidators

Introduction

Most Range consolidators aggregate bars based on a fixed price movement. It is very much like a Renko Bar, but instead of consolidating a fix movement from the opening price, a Range Bar would fix the range of the whole bar, i.e. High - Low. A new bar is formed when the range of the whole bar reached a preset value, thus it completely neglects the time effect and focuses on the variance/information carried by the price movement.



Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `RangeBar` objects. Follow these steps to create and manage a `TradeBar` consolidator based on the preset range of price movement:

1. Create the consolidator.

   To create a Range consolidator, pass the bar size to the `RangeConsolidator` constructor.

   ```
   # Create a Range consolidator that emits a bar when the high and low differs $1
   self._consolidator = RangeConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Range bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         trade_bar = slice.bars[self._symbol]
         self._consolidator.update(trade_bar)

     # Example 2: Update the consolidator with data from a history request
     history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
     for trade_bar in history:
         self._consolidator.update(trade_bar)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `RangeBar` objects. Follow these steps to create and manage a `QuoteBar` consolidator based on the preset range of price movement:

1. Create the consolidator.

   To create a Range consolidator, pass the bar size to the `RangeConsolidator` constructor.

   ```
   # Create a Range consolidator that emits a bar when the high and low differs $1
   self._consolidator = RangeConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Range bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    quote_bar = slice.quote_bars[self._symbol]
    self._consolidator.update(quote_bar)

# Example 2: Update the consolidator with data from a history request
history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
for quote_bar in history:
    self._consolidator.update(quote_bar)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `RangeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on the preset range of price movement:

1. Create the consolidator.

To create a Range consolidator, pass the bar size to the `RangeConsolidator` constructor.

```
# Create a Range consolidator that emits a bar when the high and low differs $1
self._consolidator = RangeConsolidator(1)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new Range bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `RangeBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on the preset range of price movement:

1. Create the consolidator.

   To create a Range consolidator, pass the bar size to the `RangeConsolidator` constructor.

   ```
   # Create a Range consolidator that emits a bar when the high and low differs $1
   self._consolidator = RangeConsolidator(1)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the price movement forms a new Range bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         ticks = slice.ticks[self._symbol]
         for tick in ticks:
             self._consolidator.update(tick)

     # Example 2: Update the consolidator with data from a history request
     ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
     for tick in ticks:
         self._consolidator.update(tick)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Range Consolidators

Classic Range Consolidators

Introduction

Most Range consolidators aggregate bars based on a fixed price range movement. The `ClassicRangeConsolidator` produces a different type of Range bars than the `RangeConsolidator` . A `ClassicRangeConsolidator` with a bar size of `1` produces a new bar that spans `1` every time an asset's price range moves $1. If the price jumps multiple dollars in a single tick, the `ClassicRangeConsolidator` only produces one bar per time step where the open of each bar matches the close of the previous bar.

Consolidate Trade Bars

`TradeBar` consolidators aggregate `TradeBar` objects into `RangeBar` objects. Follow these steps to create and manage a `TradeBar` consolidator based on the preceding Range bar rules:

1. Create the consolidator.

   To create a classic Range consolidator, pass the bar size to the `ClassicRangeConsolidator` constructor.

   ```
   # Create a Classic Range consolidator that emits a bar when the price range moves $1
   self._consolidator = ClassicRangeConsolidator(1)
   ```

   The `ClassicRangeConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Range bars
   - It ignores the volume of the input data
   - It enforces the high and low of each bar to be a multiple of the bar size

   To build the Range bars with a different property than the `value` of the `IBaseData` object, provide a `selector` argument. The `selector` should be a function that receives the `IBaseData` object and returns a decimal value.

   ```
   self._consolidator = ClassicRangeConsolidator(1, selector=lambda data: data.high)
   ```

   To add a non-zero `volume` property to the Range bars, provide a `volume_selector` argument. The `volume_selector` should be a function that receives the `IBaseData` object and returns a decimal value.

   ```
   self._consolidator = ClassicRangeConsolidator(1, volume_selector=lambda data: data.volume)
   ```

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
       pass
   ```

The consolidation event handler receives bars when the price movement forms a new classic Range bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `TradeBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

     ```
     # Example 1: Update the consolidator with data from the Slice object
     def on_data(self, slice: Slice) -> None:
         trade_bar = slice.bars[self._symbol]
         self._consolidator.update(trade_bar)

     # Example 2: Update the consolidator with data from a history request
     history = self.history[TradeBar](self._symbol, 30, Resolution.MINUTE)
     for trade_bar in history:
         self._consolidator.update(trade_bar)
     ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Bars

`QuoteBar` consolidators aggregate `QuoteBar` objects into `RangeBar` objects. Follow these steps to create and manage a `QuoteBar` consolidator based on the preceding Range bar rules:

1. Create the consolidator.

   To create a classic Range consolidator, pass the bar size to the `ClassicRangeConsolidator` constructor.

   ```
   # Create a Classic Range consolidator that emits a bar when the price range moves $1
   self._consolidator = ClassicRangeConsolidator(1)
   ```

   The `ClassicRangeConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Range bars
   - It ignores the volume of the input data
   - It enforces the high and low of each bar to be a multiple of the bar size

   The following arguments enable you to create Range bars that aggregate the excessive bid-ask spread on the asset. It can help market-making algorithms to track wider spread opportunities.

```
self._consolidator = ClassicRangeConsolidator(10, lambda data: data.value, lambda data: data.last_bid_size - data.last_ask_size)
```

2. Add an event handler to the consolidator.

```
self._consolidator.data_consolidated += self._consolidation_handler
```

LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

```
def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
    pass
```

The consolidation event handler receives bars when the price movement forms a new classic Range bar.

4. Update the consolidator.

You can automatically or manually update the consolidator.

- Automatic Updates

  To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

  ```
  self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
  ```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `QuoteBar` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      quote_bar = slice.quote_bars[self._symbol]
      self._consolidator.update(quote_bar)

  # Example 2: Update the consolidator with data from a history request
  history = self.history[QuoteBar](self._symbol, 30, Resolution.MINUTE)
  for quote_bar in history:
      self._consolidator.update(quote_bar)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

```
self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
```

If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Trade Ticks

`Tick` consolidators aggregate `Tick` objects into `RangeBar` objects. Follow these steps to create and manage a `Tick` consolidator based on the preceding Range bar rules:

1. Create the consolidator.

   To create a classic Range consolidator, pass the bar size to the `ClassicRangeConsolidator` constructor.

   ```
   # Create a Classic Range consolidator that emits a bar when the price range moves $1
   self._consolidator = ClassicRangeConsolidator(1)
   ```

   The `ClassicRangeConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Range bars
   - It ignores the volume of the input data
   - It enforces the high and low of each bar to be a multiple of the bar size

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the price movement forms a new classic Range bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

     To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

     ```
     self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
     ```

   - Manual Updates

     Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

```
# Example 1: Update the consolidator with data from the Slice object
def on_data(self, slice: Slice) -> None:
    ticks = slice.ticks[self._symbol]
    for tick in ticks:
        self._consolidator.update(tick)

# Example 2: Update the consolidator with data from a history request
ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
for tick in ticks:
    self._consolidator.update(tick)
```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Consolidate Quote Ticks

`Tick` quote bar consolidators aggregate `Tick` objects that represent quotes into `RangeBar` objects. Follow these steps to create and manage a `Tick` quote bar consolidator based on the preceding Range bar rules:

1. Create the consolidator.

   To create a classic Range consolidator, pass the bar size to the `ClassicRangeConsolidator` constructor.

   ```
   # Create a Classic Range consolidator that emits a bar when the price range moves $1
   self._consolidator = ClassicRangeConsolidator(1)
   ```

   The `ClassicRangeConsolidator` has the following default behavior:

   - It uses the `value` property of the `IBaseData` object it receives to build the Range bars
   - It ignores the volume of the input data
   - It enforces the high and low of each bar to be a multiple of the bar size

2. Add an event handler to the consolidator.

   ```
   self._consolidator.data_consolidated += self._consolidation_handler
   ```

   LEAN passes consolidated bars to the consolidator event handler in your algorithm. The most common error when creating consolidators is to put parenthesis `()` at the end of your method name when setting the event handler of the consolidator. If you use parenthesis, the method executes and the result is passed as the event handler instead of the method itself. Remember to pass the name of your method to the event system. Specifically, it should be `self._consolidation_handler` , not `self._consolidation_handler()` .

3. Define the consolidation handler.

   ```
   def _consolidation_handler(self, sender: object, consolidated_bar: RangeBar) -> None:
       pass
   ```

   The consolidation event handler receives bars when the price movement forms a new classic Range bar.

4. Update the consolidator.

   You can automatically or manually update the consolidator.

   - Automatic Updates

To automatically update a consolidator with data from the security subscription, call the `add_consolidator` method of the Subscription Manager.

```
self.subscription_manager.add_consolidator(self._symbol, self._consolidator)
```

- Manual Updates

  Manual updates let you control when the consolidator updates and what data you use to update it. If you need to warm up a consolidator with data outside of the warm-up period , you can manually update the consolidator. To manually update a consolidator, call its `update` method with a `Tick` object. You can update the consolidator with data from the Slice object in the `on_data` method or with data from a history request .

  ```
  # Example 1: Update the consolidator with data from the Slice object
  def on_data(self, slice: Slice) -> None:
      ticks = slice.ticks[self._symbol]
      for tick in ticks:
          self._consolidator.update(tick)

  # Example 2: Update the consolidator with data from a history request
  ticks = self.history[Tick](self._symbol, timedelta(minutes=3), Resolution.TICK)
  for tick in ticks:
      self._consolidator.update(tick)
  ```

5. If you create consolidators for securities in a dynamic universe and register them for automatic updates, remove the consolidator when the security leaves the universe.

   ```
   self.subscription_manager.remove_consolidator(self._symbol, self._consolidator)
   ```

   If you have a dynamic universe and don't remove consolidators, they compound internally, causing your algorithm to slow down and eventually die once it runs out of RAM. For an example of removing consolidators from universe subscriptions, see the GasAndCrudeOilEnergyCorrelationAlpha in the LEAN GitHub repository.

Reset Consolidators

To reset a consolidator, call its `reset` method.

```
self._consolidator.reset()
```

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your consolidators when splits and dividends occur. When a split or dividend occurs while the consolidator is in the process of building a bar, the open, high, and low may reflect prices from before the split or dividend. To avoid issues, call the consolidator's `reset` method and then warm it up with `SCALED_RAW` data from a history request .

```
def on_data(self, data: Slice):
    # When a split or dividend occurs...
    if (data.splits.contains_key(self._symbol) and data.splits[self._symbol].type == SplitType.SPLIT_OCCURRED or
        data.dividends.contains_key(self._symbol)):
        # If the consolidator is working on a bar...
        if self._consolidator.working_data:
            # Get adjusted prices for the time period of the working bar.
            history = self.history[TradeBar](self._symbol, self._consolidator.working_data.time, self.time,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
            # Reset the consolidator.
            self._consolidator.reset()
            # Warm-up the consolidator with the adjusted price data.
            for bar in history:
                self._consolidator.update(bar)
```

Examples

Demonstration Algorithms
ClassicRangeConsolidatorAlgorithm.py Python

Consolidator Types

Combining Consolidators

Introduction

Sequential consolidators wire two internal consolidators together such that the output of the first consolidator is the input to the second consolidator and the output of the second consolidator is the output of the sequential consolidator.

Create Consolidators

To create a sequential consolidator, create two consolidators and then pass them to the `SequentialConsolidator` constructor.

```py
# This first consolidator produces a consolidated bar after a day passes
one_day_consolidator = TradeBarConsolidator(timedelta(days=1))

# This second consolidators produces a consolidated bar after it sees 3 samples
three_count_consolidator = TradeBarConsolidator(3)

# This sequential consolidator aggregates three 1-day bars together
self.consolidator = SequentialConsolidator(one_day_consolidator, three_count_consolidator)
```

For more information about each type of consolidator, see Consolidator Types .

Examples

Demonstration Algorithms
DataConsolidationAlgorithm.py Python

Consolidating Data

Consolidator History

Introduction

This page explains how to save and access historical consolidated bars.

Save Consolidated Bars

To access historical bars that were passed to your consolidation handler, save the bars as you receive them. You can use a RollingWindow to save the consolidated bars and easily access them later on in your algorithm.

```
# Create a class member to store the RollingWindow
self._window = RollingWindow(2)

# In the consolidation handler, add consolidated bars to the RollingWindow
def _consolidation_handler(self, sender: object, consolidated_bar: TradeBar) -> None:
    self._window.add(consolidated_bar)
```

Get Historical Bars

If you save consolidated bars in a `RollingWindow`, you can access them by indexing the `RollingWindow`. `RollingWindow` objects operate on a first-in, first-out process to allow for reverse list access semantics with positive and negative indexing. Index 0 refers to the most recent item in the window and the largest index refers to the last item in the window. Index -1 refers to the latest item as well.

```
most_recent_bar = self._window[0]
previous_bar = self._window[1]
oldest_bar = self._window[-1]
```

To get the consolidated bar that was most recently removed from the `RollingWindow`, use the `most_recently_removed` property.

```
removed_bar = self._window.most_recently_removed
```

Examples

The following examples demonstrate some common practices for consolidator history.

Example 1: Price Action

The following algorithm trades breakout price action on the SPY five-minute trade bar. To do so, we must create a five-minute trade bar consolidator and a rolling window to hold 3 trade bars to check if the trade conditions are fulfilled.

PY

```
class ConsolidatorHistoryAlgorithm(QCAlgorithm):
    # To hold 3 consolidated trade bars to identify a breakout pattern.
    windows = RollingWindow(3)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # The breakout is based on a 5-minute consolidated trade bar.
        consolidator = TradeBarConsolidator(timedelta(minutes=5))
        # Subscribe for automatically updating the consolidator with SPY data.
        self.subscription_manager.add_consolidator(self.spy, consolidator)
        # Add a consolidator handler to check that the breakout condition is fulfilled and traded.
        consolidator.data_consolidated += self.on_consolidated

        self.set_warm_up(timedelta(15))

    def on_consolidated(self, sender: object, bar: TradeBar) -> None:
        self.windows.add(bar)

        if not self.is_warming_up and self.windows.is_ready:
            # Buy if the breakout price action is fulfilled.
            # 1. Increasing price trend.
            # 2. The last 3 bars are green.
            # 3. The 3rd and 2nd last bars range is decreasing.
            # 4. The last bar exceeds the 2nd last bar by double the 2nd last bar's range.
            second_last_range = self.windows[1].close - self.windows[1].open
            if bar.close > self.windows[1].close and self.windows[1].close > self.windows[2].close and\
            self.windows[2].close > self.windows[2].open and self.windows[1].close > self.windows[1].open and\
            self.windows[0].close > self.windows[0].open and self.windows[2].close - self.windows[2].open > second_last_range and\
            bar.close > self.windows[1].close + 2 * second_last_range:
                self.set_holdings(self.spy, 0.5)

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * 0.99 if order_event.fill_quantity > 0 else order_event.fill_price * 1.01
                self.stop_market_order(self.spy, -self.portfolio[self.spy].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * 1.02 if order_event.fill_quantity > 0 else order_event.fill_price * 0.98
                self.limit_order(self.spy, -self.portfolio[self.spy].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel any open order if stop loss or take profit order filled.
                self.transactions.cancel_open_orders()
```

Consolidating Data

Updating Indicators

Introduction

You can use consolidators to automatically update indicators in your algorithms. The consolidators can update your indicators at each time step or with aggregated bars. By default, LEAN updates data point indicators with the close price of the consolidated bars, but you can change it to a custom data field.

Standard Indicator Periods

If your algorithm has a static universe, you can create automatic indicators in just one line of code. When you create an automatic indicator, LEAN creates a consolidator, hooks it up for automatic updates, and then updates the indicator with the consolidated bars.

```
PY
 # Consolidate minute SPY data into 14-bar daily indicators
ema = self.ema("SPY", 14, Resolution.DAILY)
sma = self.sma("SPY", 14, Resolution.DAILY)
```

If your algorithm has a dynamic universe, create a manual indicator and then create a time period consolidator that updates the indicator at each time step. Keep a reference to the consolidator so you can remove it when your algorithm removes the security from the universe.

```
PY
def initialize(self):
    self._ema = ExponentialMovingAverage(14)
    self._consolidator = TradeBarConsolidator(1)
    self._consolidator.data_consolidated += self._on_data_consolidated
    self.subscription_manager.add_consolidator(self._symbol, self._consolidator)

# Define consolidator handler function as a class method
def _on_data_consolidated(self, sender, bar):
    self._ema.update(bar.end_time, bar.close)
```

Custom Indicator Periods

It's common to update indicators with price data that spans a normal time period like one minute or one day. It's less common to update an indicator with exotic time periods (for example, a 7-minute consolidated bar), so these types of indicators may provide more opportunities for alpha. To update indicators with exotic data, create a manual indicator and then call the `register_indicator` method. The `register_indicator` method wires up the indicator for automatic updates at the time interval you provide.

```
PY
# Calculate the SMA with 10 7-minute bars
self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
self._indicator = SimpleMovingAverage(10)
self.register_indicator(self._symbol, self._indicator, timedelta(minutes=7))
```

The `register_indicator` method can accept a `timedelta` , `Resolution` , or an unregistered consolidator. If you apply the indicator to a security in a dynamic universe, provide a consolidator so that you can remove it when your algorithm removes the security from the universe.

```
PY
# timedelta
self.register_indicator(self._symbol, self._indicator, timedelta(minutes=7))

# Resolution
self.register_indicator(self._symbol, self._indicator, Resolution.HOUR)

# Consolidator
self._consolidator = TradeBarConsolidator(35)
self.register_indicator(self._symbol, self._indicator, self._consolidator)
```

Custom Indicator Values

Data point indicators use only a single price data in their calculations. By default, those indicators use the closing price. For assets with `TradeBar`

data, that price is the `TradeBar` close price. For assets with `QuoteBar` data, that price is the mid-price of the bid closing price and the ask closing price. To create an indicator with the other fields like the `OPEN`, `HIGH`, `LOW`, or `CLOSE`, provide a `selector` argument to the `register_indicator` method.

```
# Define a 10-period RSI with indicator constructor
self._rsi = RelativeStrengthIndex(10, MovingAverageType.SIMPLE)

# Register the daily High price data to automatically update the indicator
self.register_indicator(self._symbol, self._rsi, Resolution.DAILY, Field.HIGH)
```

The `register_indicator` method can accept a `timedelta`, `Resolution`, or an unregistered consolidator. If you apply the indicator to a security in a dynamic universe, provide a consolidator so that you can remove it when your algorithm removes the security from the universe.

The `Field` class has the following `selector` properties:

Examples

The following examples demonstrate common practices for updating indicators using consolidators.

Example 1: Hammer Pattern With 5-Minute Consolidator

The following algorithm trades Hammer and Inverted Hammer candlestick patterns. The candles are constructed with five-minute SPY trade bars consolidated by a five-minute `TradeBarConsolidator`, while automatically updating with the `register_indicator` method. Both patterns are bullish, so we buy SPY if the pattern is signaled.

```
from QuantConnect.Indicators.CandlestickPatterns import *
class ConsolidatorUpdatingIndicatorsAlgorithm(QCAlgorithm):
    hammer = Hammer()
    inverted_hammer = InvertedHammer()

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # The candlestick patterns are based on a 5-minute consolidated trade bar.
        consolidator = TradeBarConsolidator(timedelta(minutes=5))
        # Subscribe for automatically updating the indicators with the 5-minute consolidator.
        self.register_indicator(self.spy, self.hammer, consolidator)
        self.register_indicator(self.spy, self.inverted_hammer, consolidator)
        # Add an event handler on candlestick indicators that are updated to trade the pattern.
        self.hammer.updated += self.on_updated
        self.inverted_hammer.updated += self.on_updated

        self.set_warm_up(1)

    def on_updated(self, sender: object, point: IndicatorDataPoint) -> None:
        # Both the hammer and inverted hammer patterns indicate bullish movement, so we buy SPY.
        if point.value == 1 and not self.portfolio[self.spy].is_long:
            self.set_holdings(self.spy, 0.5)

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * (0.99 if order_event.fill_quantity > 0 else 1.01)
                self.stop_market_order(self.spy, -self.portfolio[self.spy].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * (1.02 if order_event.fill_quantity > 0 else 0.98)
                self.limit_order(self.spy, -self.portfolio[self.spy].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel any open order if stop loss or take profit order filled.
                self.transactions.cancel_open_orders()
```

Historical Data

Historical Data

Getting Started

Introduction

There are two ways to request historical data in your algorithms: direct historical data requests and indirect algorithm warm up .

History Requests

History requests return all of the data you request as a single object. Follow these steps to get some historical data:

1. Create a new project.

   Projects contain files to run backtests, launch research notebooks, perform parameter optimizations, and deploy live trading strategies. You need to create projects in order to create strategies and share your work with other members.

   The process to create a new project depends on if you use the Cloud Platform , Local Platform , or CLI .

2. In the `initialize` method, add an asset .

   The `initialize` method is the entry point of your algorithm where you define a series of settings, including security subscriptions, starting cash balances, and warm-up periods. LEAN only calls the `initialize` method one time, at the start of your algorithm.

   ```
   spy = self.add_equity('SPY')
   ```
   PY

3. Call the `history` method with the asset's Symbol and a lookback period.

   `Symbol` objects are a way to identify or "finger-print" tradable assets so that no further database look-up is required. All QuantConnect and LEAN Algorithm API methods use `Symbol` objects to identify assets.

   ```
   history = self.history(spy.symbol, 5, Resolution.DAILY)
   ```
   PY

		close	high	low	open	volume
symbol	**time**					
SPY	**2025-02-12 16:00:00**	603.36	604.54	598.520	599.26	38247065.0
	2025-02-13 16:00:00	609.73	609.94	603.215	604.43	36445673.0
	2025-02-14 16:00:00	609.70	610.99	609.080	609.98	23498431.0
	2025-02-18 16:00:00	611.49	611.49	608.390	610.86	24030531.0
	2025-02-19 16:00:00	612.93	613.23	609.560	610.04	26847903.0

Warm Up Periods

Warm-up simulates winding back the clock from the time you deploy the algorithm. In a backest, this is the start date of your algorithm. In live trading, it's the current date. Follow these steps to add a warm-up period to the start of your algorithm:

1. Create a new project.

   The process to create a new project depends on if you use the Cloud Platform , Local Platform , or CLI .

2. In the `initialize` method, set the backtest dates and add an asset .

```
self.set_start_date(2024, 12, 1)
self.set_end_date(2024, 12, 2)
self.add_equity("SPY", Resolution.DAILY)
```

3. In the `initialize` method, call the `set_warm_up` method with the warm-up duration.

```
self.set_warm_up(10, Resolution.DAILY)
```

4. In the `on_data` method, log the time and warm-up state .

```
self.log(f"self.is_warming_up at {self.time}: {self.is_warming_up}")
```

```
Algorithm starting warm up...
self.is_warming_up at 2024-11-22 16:00:00: True
self.is_warming_up at 2024-11-25 16:00:00: True
self.is_warming_up at 2024-11-26 16:00:00: True
self.is_warming_up at 2024-11-27 16:00:00: True
self.is_warming_up at 2024-11-29 13:00:00: True
Algorithm finished warming up.
self.is_warming_up at 2024-12-02 16:00:00: False
```

For more information about warm-up, see Warm Up Periods .

Examples

The following examples demonstrate some common practices for historical data.

Example 1: Simple Moving Average

The following example demonstrates weekly-resampled historical data to calculate the 50-week Simple Moving Average to implement an SMA-cross strategy.

```
class HistoricalDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Request SPY data for historical data call and trading.
        self.spy = self.add_equity("SPY").symbol
        # Set a scheduled event to trade SPY with a weekly SMA signal.
        self.schedule.on(self.date_rules.week_start(self.spy), self.time_rules.after_market_open(self.spy, 1), self.trade)

    def trade(self) -> None:
        # Historical data call to calculate signal.
        history = self.history(self.spy, 252, Resolution.DAILY)
        # Obtain the weekly close price to calculate the weekly SMA.
        weekly_close = history.unstack(0).close.resample("W").last()
        # Calculate the current 50-week SMA as a trading signal.
        sma50 = weekly_close.rolling(50).mean().iloc[-1, 0]

        # Trade the SMA cross strategy.
        price = self.securities[self.spy].price
        if price > sma50 and not self.portfolio[self.spy].is_long:
            self.set_holdings(self.spy, 1)
        elif price < sma50 and not self.portfolio[self.spy].is_short:
            self.set_holdings(self.spy, -1)
```

Historical Data

History Requests

Introduction

There are two ways to request historical data in your algorithms: direct historical data requests and indirect algorithm warm up . You can use a direct historical data request at any time throughout your algorithm. It returns all of the data you request as a single object.

Trailing Data Samples

To get historical data for a trailing time period, call `history` method with an an integer and a `Resolution` . For example, if you pass an asset `Symbol` , 5, and `Resolution.MINUTE` as the arguments, it returns the data of the asset during the most recent 5 minutes in the asset's market hours . These trailing minute bars can cross multiple trading days.

PY

```
class TrailingDataSamplesHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('SPY').symbol
        # Get the minute-resolution TradeBar data of the asset over the trailing 5 minutes.
        history = self.history(TradeBar, symbol, 5, Resolution.MINUTE)
```

		close	high	low	open	volume
symbol	**time**					
SPY	**2024-12-18 15:56:00**	588.63	590.390	588.57	590.39	1053414.0
	2024-12-18 15:57:00	588.34	588.910	588.24	588.61	930643.0
	2024-12-18 15:58:00	588.11	588.460	588.07	588.34	1138812.0
	2024-12-18 15:59:00	587.92	588.325	587.70	588.10	1576391.0
	2024-12-18 16:00:00	586.28	587.940	585.89	587.92	5865463.0

If you don't pass a `Resolution` , it defaults to the resolution of the security subscription. The following example returns 3 days of data for QQQ and 3 minutes of data for SPY:

PY

```
class TrailingDataSamplesForMulitpleAssetsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 20)
        self.set_end_date(2024, 12, 31)
        # Add two assets with different resolutions.
        spy = self.add_equity('SPY', Resolution.MINUTE).symbol
        qqq = self.add_equity('QQQ', Resolution.DAILY).symbol
        # Get the trailing 3 bars of data for each asset.
        history = self.history(TradeBar, [spy, qqq], 3)
```

		close	high	low	open	volume
symbol	**time**					
QQQ	**2024-12-17 16:00:00**	535.80	537.49	534.130	536.410	25048673.0
	2024-12-18 16:00:00	516.47	536.87	515.010	535.110	47016560.0
	2024-12-19 16:00:00	514.17	521.75	513.830	521.110	42192908.0
SPY	**2024-12-19 15:58:00**	586.40	586.77	586.365	586.730	872817.0
	2024-12-19 15:59:00	586.50	586.83	586.380	586.420	1524079.0
	2024-12-19 16:00:00	586.10	586.53	585.850	586.495	4342748.0

If there is no data for the time period you request, the result has fewer samples. For instance, say an illiquid asset has no trading activity during the last 15 minutes of the trading day and you request the 10 most recent minute bars at market close, 4 PM Eastern Standard Time (ET). In this case, you won't get any data because LEAN will try to fetch data from 3:50 PM ET to 4 PM ET since the market was open during that time , but there were no trades for the asset. For more information about missing data points, see Missing Data Points .

Trailing Time Periods

To get historical data for a trailing time period, pass a `timedelta` to the `history` method. The days of the `timedelta` represent calendar days.

```
class TrailingTimePeriodHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('SPY').symbol
        # Get the minute-resolution TradeBar data of the asset over the trailing 3 days.
        history = self.history(TradeBar, symbol, timedelta(3), Resolution.MINUTE)
```

		close	high	low	open	volume
symbol	**time**					
SPY	**2024-12-16 09:31:00**	606.02	606.400	605.89	606.00	612650.0
	2024-12-16 09:32:00	606.01	606.305	605.94	605.99	132785.0
	...	...	...	...	...	...
	2024-12-18 15:59:00	587.92	588.325	587.70	588.10	1576391.0
	2024-12-18 16:00:00	586.28	587.940	585.89	587.92	5865463.0

If there is no data in the time period, the result is empty. For more information about missing data points, see Missing Data Points .

Date Ranges

To get historical data for a specific date range, call `history` method with start and end `datetime` objects. The `datetime` objects you provide are based in the algorithm time zone .

```
class DateRangeHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('SPY').symbol
        # Get the daily-resolution TradeBar data of the asset during 2020.
        history = self.history(TradeBar, symbol, datetime(2020, 1, 1), datetime(2021, 1, 1), Resolution.DAILY)
```

		close	high	low	open	volume
symbol	**time**					
SPY	**2020-01-02 16:00:00**	301.194352	301.194352	299.029520	299.998363	52757344.0
	2020-01-03 16:00:00	298.913630	300.053991	297.708370	297.782540	65558742.0
	2020-01-06 16:00:00	300.053991	300.137432	297.013028	297.087198	47199709.0
	2020-01-07 16:00:00	299.210309	299.956643	298.756019	299.479175	37979987.0
	2020-01-08 16:00:00	300.804960	302.038033	299.163953	299.451361	62265971.0
	...	...	...	...	...	...
	2020-12-24 13:00:00	348.496921	348.496921	347.033045	347.609151	22897100.0
	2020-12-28 16:00:00	351.490783	351.887446	350.451904	351.150786	34765681.0
	2020-12-29 16:00:00	350.820234	353.219101	350.225239	353.077436	48584327.0
	2020-12-30 16:00:00	351.320785	352.350220	350.933566	351.689115	46030043.0
	2020-12-31 16:00:00	353.105769	353.825429	350.612458	351.188564	58258603.0

If there is no data for the date range you request, the result is empty. For more information about missing data points, see Missing Data Points .

Flat Universe DataFrames

Most history requests return a flat DataFrame by default, where there is one column for each data point attribute. Universe history requests return a Series, where the values in the Series are lists of the universe data objects. For example, the following code snippet returns a Series where each value is a `List[ETFConstituentUniverse]` :

```
class SeriesUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add an ETF constituents universe for SPY so you can get its historical data.
        universe = self.add_universe(self.universe.etf('SPY'))
        # Get the trailing 5 days of universe data.
        history = self.history(universe, 5, Resolution.DAILY)
```

```
symbol                                                                  time
qc-universe-etf-constituents-a8cf32b9-ea94-488b-966f-aa2ebea9e40c 2T    2024-12-13    [A RPTMYV3VC57P: ¤0.00, AAPL: ¤0.00, ABBV VCY0...
                                                                        2024-12-14    [A RPTMYV3VC57P: ¤0.00, AAPL: ¤0.00, ABBV VCY0...
                                                                        2024-12-17    [A RPTMYV3VC57P: ¤0.00, AAPL: ¤0.00, ABBV VCY0...
                                                                        2024-12-18    [A RPTMYV3VC57P: ¤0.00, AAPL: ¤0.00, ABBV VCY0...
                                                                        2024-12-19    [A RPTMYV3VC57P: ¤0.00, AAPL: ¤0.00, ABBV VCY0...
Name: data, dtype: object
```

To get the data into a DataFrame instead, set the `flatten` argument to `True` . In this case, the DataFrame has one column for each data point attribute.

PY

```
# Get the trailing 5 days of universe data in DataFrame format
# so you can perform DataFrame wrangling operations.
history = self.history(universe, 5, Resolution.DAILY, flatten=True)
```

		lastupdate	period	sharesheld	weight
time	**symbol**				
2024-12-13	**A RPTMYV3VC57P**	2024-12-11	1 days	3667879.0	0.000799
	AAPL R735QTJ8XC9X	2024-12-11	1 days	190881735.0	0.072612
	ABBV VCY032R250MD	2024-12-11	1 days	22169383.0	0.005966
	ABNB XK8H247DY6W5	2024-12-11	1 days	5515085.0	0.001182
	ABT R735QTJ8XC9X	2024-12-11	1 days	21833687.0	0.003846
...	**...**	...	...	...	...
2024-12-19	**XYL V18KR26TE3XH**	2024-12-17	1 days	3065334.0	0.000572
	YUM R735QTJ8XC9X	2024-12-17	1 days	3548979.0	0.000739
	ZMH S6ZZPKTVDY05	2024-12-17	1 days	2570208.0	0.000429
	ZBRA R735QTJ8XC9X	2024-12-17	1 days	650402.0	0.000397
	ZTS VDRJHVQ4FNFP	2024-12-17	1 days	5703558.0	0.001505

Default Values

The following table describes the assumptions of the History API:

Argument	Assumption
`resolution`	LEAN guesses the resolution you request by looking at the securities you already have in your algorithm. If you have a security subscription in your algorithm with a matching `Symbol` , the history request uses the same resolution as the subscription. If you don't have a security subscription in your algorithm with a matching `Symbol` , `Resolution.MINUTE` is the default.

Additional Options

The `history` method accepts the following additional arguments:

Argument	Data Type	Description
fill_forward	bool/NoneType	True to fill forward missing data. Otherwise, false.
extended_market_hours	bool/NoneType	True to include extended market hours data. Otherwise, false.
data_mapping_mode	DataMappingMode/NoneType	The contract mapping mode to use for the security history request.
data_normalization_mode	DataNormalizationMode/NoneType	The price scaling mode to use for US Equities or continuous Futures contracts .
contract_depth_offset	int/NoneType	The desired offset from the current front month for continuous Futures contracts .

If you don't provide a value for some of the preceding options, it uses the option value from the security subscription.

```
self.future = self.add_future(Futures.Indices.SP_500_E_MINI)
history = self.history(
    symbols=[self.future.symbol],
    start=self.time - timedelta(days=15),
    end=self.time,
    resolution=Resolution.MINUTE,
    fill_forward=False,
    extended_market_hours=False,
    data_mapping_mode=DataMappingMode.LAST_TRADING_DAY,
    data_normalization_mode=DataNormalizationMode.RAW,
    contract_depth_offset=0
)
```

Examples

The following examples demonstrate some common practices for trading using historical requests.

Example 1: Mean-Variance Portfolio

The following algorithm constructs a monthly rebalance mean-variance portfolio using the top 20 liquid equities. The position sizing can be optimized by 1-year historical daily return of the universe members.

```
from Portfolio.MaximumSharpeRatioPortfolioOptimizer import MaximumSharpeRatioPortfolioOptimizer

class HistoricalRequestAlgorithm(QCAlgorithm):
    # Instantiate the optimizer to perform mean-variance optimization.
    # Mean-variance optimization will not consider a risk-free rate, so we use 0.
    _optimizer = MaximumSharpeRatioPortfolioOptimizer(0.0, 1.0, 0.0)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Seed the last price to set the initial price of the BTCUSDT holdings.
        self.settings.seed_initial_prices = True

        # Monthly renewal of the top 20 liquid universe to trade popular stocks.
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.top(20))

        # Set a scheduled event to rebalance the portfolio at the start of every month.
        self.schedule.on(self.date_rules.month_start(), self.time_rules.at(9, 31), self.rebalance)

    def rebalance(self) -> None:
        # Historical data request to get 1-year data for optimization.
        symbols = self._universe.selected
        history = self.history(symbols, 253, Resolution.DAILY).close.unstack(0).dropna()
        # Daily return on the universe members to calculate the optimized weights.
        returns = history.pct_change().dropna()

        # Calculate the optimized weights.
        weights = self._optimizer.optimize(returns)

        # Rebalance the portfolio according to the optimized weights.
        targets = [PortfolioTarget(symbol, size if abs(size) > 0.01 else 0) for symbol, size in zip(symbols, weights)]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
HistoryAlgorithm.py Python

Historical Data

History Responses

Introduction

When you make a history request, consider what data returns. Depending on how you request the data, history requests return a specific data type. The data that returns is in ascending order from oldest to newest. This order is necessary to use the data to warm up indicators.

DataFrames

The most popular return type is a DataFrame. To request a DataFrame of historical data, call the `history` method with at least one `Symbol` object.

```
class DataFrameHistoryResponseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2014, 12, 20)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol objects of some assets.
        symbols = [self.add_equity(ticker).symbol for ticker in ['SPY', 'QQQ']]
        # Get 3 days of daily data for the assets.
        history = self.history(symbols, 3, Resolution.DAILY)
```

		close	high	low	open	volume
symbol	**time**					
SPY	**2024-12-17 16:00:00**	604.29	605.16	602.89	604.22	38527534.0
	2024-12-18 16:00:00	586.28	606.40	585.89	604.00	80642184.0
	2024-12-19 16:00:00	586.10	593.00	585.85	591.43	70752398.0
QQQ	**2024-12-17 16:00:00**	535.80	537.49	534.13	536.41	25048673.0
	2024-12-18 16:00:00	516.47	536.87	515.01	535.11	47016560.0
	2024-12-19 16:00:00	514.17	521.75	513.83	521.11	42192908.0

The structure of the DataFrame depends on the dataset . In most cases, there is a mulit-index that contains the `Symbol` and a timestamp. The timestamps in the DataFrame are based on the exchange time zone . Each row of the DataFrame represents the prices at a point in time. Each column of the DataFrame is a property of that data (for example, open, high, low, and close (OHLC)).

If you intend to use the data in the DataFrame to create `TradeBar` or `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame.

Slices

To request `Slice` objects of historical data, call the `history` method without providing any `Symbol` objects. It returns data for all the data subscriptions in your notebook.

```
# Get the latest 3 data points of all the securities/datasets in the notebook, packaged into Slice objects.
history = self.history(3)
```

When your history request returns `Slice` objects, the `time` properties of these objects are based on the algorithm time zone, but the `end_time`

properties of the individual data objects (for example, `TradeBar` , `QuoteBar` , and `Tick` objects) are based on the exchange time zone . The `end_time` is the end of the sampling period and when the data is actually available. For more information about time periods, see Periods .

Lists

If you intend to use the data in the DataFrame to create objects like a `TradeBar` or `QuoteBar` , request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To request a list of objects containing the historical data, call one of the `history[ Type ]` methods.

```py
class ListHistoryResponseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('SPY', Resolution.DAILY).symbol
        # Create an indicator that is a function of historical bars.
        indicator = AverageTrueRange(21)
        # Get the latest TradeBar objects of the asset.
        history = self.history[TradeBar](symbol, indicator.warm_up_period)
        # Update the indicator with the historical TradeBar objects.
        for bar in history:
            indicator.update(bar)
```

The methods return different data types, depending on how you call it. The following table shows the return type for each input:

Input Generic `[ Type ]`	**Input** `Symbol` **Type**	**Output Data Type**
`TradeBar`	`Symbol`	`List[TradeBar]`
`TradeBar`	`List[Symbol]`	`List[TradeBars]`
`QuoteBar`	`Symbol`	`List[QuoteBar]`
`QuoteBar`	`List[Symbol]`	`List[QuoteBars]`
`Tick`	`Symbol`	`List[Tick]`
`Tick`	`List[Symbol]`	`List[Ticks]`
`alternativeDataClass` (ex: `CBOE`)	`Symbol`	`List[ alternativeDataClass ]` (ex: `List[CBOE]`)
`alternativeDataClass` (ex: `CBOE`)	`List[Symbol]`	`List[Dict[Symbol, alternativeDataClass ]]` (ex: `List[Dict[Symbol, CBOE]]`)

```py
class ListHistoryResponseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of some assets.
        spy = self.add_equity('SPY').symbol
        aapl = self.add_equity('AAPL').symbol

        # Example: List[QuoteBars]
        for quote_bars in self.history[QuoteBar]([spy, aapl], 3):
            t = quote_bars.time
            for symbol, quote_bar in quote_bars.items():
                bid = quote_bar.bid

        # Example: List[Ticks]
        for ticks in self.history[Tick]([spy, aapl], timedelta(3), Resolution.TICK):
            for symbol, tick in ticks.items():
                t = tick.time
                price = tick.price

        # Example: List[Dict[Symbol, QuiverInsiderTrading]]
        spy_insider = self.add_data(QuiverInsiderTrading, spy).symbol
        aapl_insider = self.add_data(QuiverInsiderTrading, aapl).symbol
        for data_by_symbol in self.history[QuiverInsiderTrading]([spy_insider, aapl_insider], 365, Resolution.DAILY):
            for dataset_symbol, insider_trading in data_by_symbol.items():
                asset_symbol = dataset_symbol.underlying
                t = insider_trading.end_time
                shares_traded = insider_trading.shares
```

When your history request returns a list of objects, the `time` properties of these objects are based on the time zone, but the `end_time` properties of the individual objects in the list are based on the exchange time zone . The `end_time` is the end of the sampling period and when the data is actually available. For more information about time periods, see Periods .

Missing Data Points

There are several cases where you may get fewer data points from a history request than you expect.

Illiquid Securities

The default behavior for history requests is to fill-forward the data. However, if an asset had no trading activity during the time period you request historical data for, you won't get any data for that asset.

Requesting Data Before an Asset's IPO

All assets have a day when they first started trading. If your history request includes time before an asset's initial public offering (IPO) date, you won't get any data before the IPO date because the data doesn't exist.

Requesting Data Outside Market Hours

History requests for a trailing number of data samples return data based on the market hours of assets, so they can wrap across multiple trading days to gather the amount of data you request. In contrast, history requests for trailing time periods return data based on a rolling calendar time and history requests for specific date ranges return data for whatever range you request. In these two latter types of history requests, if the period of time is outside of the market hours of an asset, you won't get any data.

Data Issues

Data issues are generally a result of human error or from mistakes in the data collection process. Any QuantConnect member can report a data issue , which our Data Team works to quickly resolve. However, while the data is still missing, your history requests won't be able to return the data. To view the list of current data issues, log in to the Algorithm Lab and then, in the left navigation bar, click Datasets > Data Issues .

Historical Data

Asset Classes

Asset Classes

US Equities

Introduction

This page explains how to get historical data for US Equities. Some of the data you can get include prices, fundamentals, corporate actions, and universe selection data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```
PY
class USEquityTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_equity('SPY').symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

		close	high	low	open	volume
symbol	**time**					
SPY	**2024-12-12 16:00:00**	604.33	607.160	604.33	606.64	24962360.0
	2024-12-13 16:00:00	604.21	607.100	602.82	606.38	29250856.0
	2024-12-16 16:00:00	606.79	607.775	605.22	606.00	33686372.0
	2024-12-17 16:00:00	604.29	605.160	602.89	604.22	38527534.0
	2024-12-18 16:00:00	586.28	606.400	585.89	604.00	80642184.0

```
PY
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
symbol  time
SPY     2024-12-13 16:00:00   -0.000199
        2024-12-16 16:00:00    0.004270
        2024-12-17 16:00:00   -0.004120
        2024-12-18 16:00:00   -0.029804
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
class USEquityQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_equity('SPY').symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

		askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
symbol	**time**														
SPY	**2024-12-18 15:56:00**	588.65	590.44	588.60	590.42	10700.0	588.61	590.40	588.56	590.38	700.0	588.630	590.420	588.580	590.400
	2024-12-18 15:57:00	588.37	588.92	588.25	588.65	100.0	588.34	588.89	588.24	588.61	100.0	588.355	588.905	588.245	588.630
	2024-12-18 15:58:00	588.13	588.50	588.08	588.37	100.0	588.11	588.47	588.06	588.34	500.0	588.120	588.485	588.070	588.355
	2024-12-18 15:59:00	587.93	588.34	587.70	588.13	8000.0	587.92	588.31	587.68	588.11	100.0	587.925	588.325	587.690	588.120
	2024-12-18 16:00:00	586.33	587.95	585.89	587.93	200.0	586.30	587.93	585.88	587.92	74700.0	586.315	587.940	585.885	587.925

```
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
symbol  time
SPY     2024-12-18 15:56:00    0.04
        2024-12-18 15:57:00    0.03
        2024-12-18 15:58:00    0.02
        2024-12-18 15:59:00    0.01
        2024-12-18 16:00:00    0.03
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
PY
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Request second or minute resolution data. Otherwise, the history request won't return any data.

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
PY
class USEquityTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_equity('SPY').symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

		askprice	asksize	bidprice	bidsize	exchange	lastprice	quantity
symbol	**time**							
SPY	**2024-12-17 09:30:00.000214**	0.00	0.0	604.17	1000.0	NASDAQ	604.17	0.0
	2024-12-17 09:30:00.000214	604.19	1900.0	0.00	0.0	ARCA	604.19	0.0
	2024-12-17 09:30:00.001110	NaN	NaN	NaN	NaN	NYSE	604.19	100.0
	2024-12-17 09:30:00.001480	NaN	NaN	NaN	NaN	ARCA	604.19	69.0
	2024-12-17 09:30:00.001482	0.00	0.0	604.17	1000.0	NASDAQ	604.17	0.0

```
PY
# Select the rows in the DataFrame that represent trades. Drop the bid/ask columns since they are NaN.
trade_ticks = history[history.quantity > 0].dropna(axis=1)
```

		exchange	lastprice	quantity
symbol	**time**			
SPY	**2024-12-17 09:30:00.001110**	NYSE	604.19	100.0
	2024-12-17 09:30:00.001480	ARCA	604.19	69.0
	2024-12-17 09:30:00.001483	ARCA	604.19	100.0
	2024-12-17 09:30:00.001483	ARCA	604.19	231.0
	2024-12-17 09:30:00.001564	NASDAQ	604.19	100.0

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
PY
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the quote size.
for tick in history:
    if tick.tick_type == TickType.Quote:
        t = tick.end_time
        size = max(tick.bid_size, tick.ask_size)
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
PY
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_equity('SPY')
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Fundamentals

To get historical fundamental data , call the `history` method with the `Fundamental` type and an asset's `Symbol` . This method returns a DataFrame with columns for the data point attributes .

```
PY
class USEquityFundamentalHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 27)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('AAPL').symbol
        # Get the 3 trailing daily Fundamental objects of the asset in DataFrame format.
        history = self.history(Fundamental, symbol, 3, Resolution.DAILY)
```

		adjusted	ab	ea	as	ay	pr	fil	fo	ga	gr	fin	pr	fis	fu	de	ad	op	pr	ra	sc	se	ty	ti	ev	va	vo
symbol	**time**																										
AAPL	**2024-12-17**	250.76	Q	A	C	C	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
	2024-12-18	253.20	Q	A	C	C	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
	2024-12-19	247.77	Q	A	C	C	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

PY

```
# Get the Current Ratio of each row.
current_ratios = history.apply(lambda row: row.operationratios.current_ratio.value, axis=1)
```

```
symbol  time
AAPL    2024-12-24    0.867313
        2024-12-25    0.867313
        2024-12-27    0.867313
dtype: float64
```

To avoid consuming unnecessary computational resources populating the DataFrame, you can instead request `Fundamental` objects. To get a list of `Fundamental` objects, call the `history[Fundamental]` method.

PY

```
# Get the 3 trailing daily Fundamental objects of an asset in Fundamental format.
history = self.history[Fundamental](symbol, 3, Resolution.DAILY)
# Iterate through each Fundamental object and access its properites.
for fundamental in history:
    symbol = fundamental.symbol
    pe_ratio = fundamental.valuation_ratios.pe_ratio
```

Splits

To get historical split data , call the `history` method with the `Split` type and an asset's `Symbol` . This method returns a DataFrame with columns for the reference price, split factor, and split type. Splits are a sparse dataset, so use a time period history request since most days have no data.

PY

```
class USEquitySplitHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('AAPL').symbol
        # Get the splits that occured for the stock over the last 5 years in DataFrame format.
        history = self.history(Split, symbol, timedelta(5*365))
```

		referenceprice	splitfactor	type	value
symbol	**time**				
AAPL	**2020-08-28**	0.00	0.25	0	0.00
	2020-08-31	499.23	0.25	1	499.23

In the preceding DataFrame, the `type` column represents the `SplitType` enumeration, where 0= `SplitType.WARNING` and 1= `SplitType.SPLIT_OCCURRED` .

```
# Select the prices where splits occurred.
split_prices = history[history.type == SplitType.SPLIT_OCCURRED].value
```

```
symbol  time
AAPL    2020-08-31    499.23
Name: value, dtype: float64
```

If you intend to use the data in the DataFrame to create `Split` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Split` objects instead of a DataFrame, call the `history[Split]` method.

```
# Get the splits that occured for a stock over the last 5 years in Split format.
history = self.history[Split](symbol, timedelta(5*365))
# Iterate through each Split object.
for split in history:
    # Select the time when each split occurred.
    if split.type == SplitType.SPLIT_OCCURRED:
        t = split.end_time
```

Dividends

To get historical dividend data , call the `history` method with the `Dividend` type and an asset's `Symbol` . This method returns a DataFrame with columns for the dividend payment and reference price. Dividends are a sparse dataset, so use a time period history request since most days have no data.

```
class USEquityDividendHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('AAPL').symbol
        # Get the dividends that the stock paid over the last 2 years in DataFrame format.
        history = self.history(Dividend, symbol, timedelta(2*365))
```

		distribution	referenceprice	value
symbol	**time**			
AAPL	**2022-12-23**	0.17	132.23	0.17
	2023-02-10	0.23	150.87	0.23
	2023-05-12	0.24	173.75	0.24
	2023-08-11	0.24	177.97	0.24
	2023-11-10	0.24	182.41	0.24

```
# Calculate the mean dividend payment.
mean_dividend = history.distribution.mean()
```

If you intend to use the data in the DataFrame to create `Dividend` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Dividend` objects instead of a DataFrame, call the `history[Dividend]` method.

```
# Get the dividends that a stock paid over the last 2 years in Dividend format.
history = self.history[Dividend](symbol, timedelta(2*365))
# Iterate through each Dividend object.
for dividend in history:
    distribution = dividend.distribution
```

Symbol Changes

To get historical symbol change data , call the `history` method with the `SymbolChangedEvent` type and an asset's `Symbol` . This method returns a DataFrame with columns for the old symbol and new symbol during each change.

```
class USEquitySymbolChangedEventHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('META').symbol
        # Get the symbol changes of the stock over the last 10 years in DataFrame format.
        history = self.history(SymbolChangedEvent, symbol, timedelta(10*365))
```

		newsymbol	oldsymbol
symbol	**time**		
META	**2022-06-09**	META	FB

```
# Select the dates of each ticker change.
dates = list(history.index.levels[1])
```

If you intend to use the data in the DataFrame to create `SymbolChangedEvent` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `SymbolChangedEvent` objects instead of a DataFrame, call the `history[SymbolChangedEvent]` method.

```
# Get the symbol changes of a stock over the last 10 years in SymbolChangedEvent format.
history = self.history[SymbolChangedEvent](symbol, timedelta(10*365))
# Iterate through each SymbolChangedEvent object.
for symbol_changed_event in history:
    t = symbol_changed_event.end_time
    old_symbol = symbol_changed_event.old_symbol
    new_symbol = symbol_changed_event.new_symbol
```

Delistings

To get historical delisting data , call the `history` method with the `Delisting` type and an asset's `Symbol` . This method returns a DataFrame with columns for the old symbol and new symbol during each change. Delistings are a sparse dataset, so use a time period history request since most days have no data.

```
class USEquityDelistingHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('BBBY').symbol
        # Get the deslistings of the asset over the last 10 years in DataFrame format.
        history = self.history(Delisting, symbol, timedelta(10*365))
```

		type
symbol	**time**	
BBBY	**2023-05-02**	0
	2023-05-03	1

In the preceding DataFrame, the `type` column represents the `DelistingType` enumeration, where 0= `DelistingType.WARNING` and 1= `DelistingType.DELISTED`.

```
PY
# Select the rows in the DataFrame that represent deslist warnings.
warnings = history[history.type == DelistingType.WARNING].type
```

```
symbol  time
BBBY    2023-05-02    0
Name: type, dtype: int64
```

If you intend to use the data in the DataFrame to create `Delisting` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Delisting` objects instead of a DataFrame, call the `history[Delisting]` method.

```
PY
# Get the deslistings of an asset over the last 10 years in Delisting format.
history = self.history[Delisting](symbol, timedelta(10*365))
# Iterate through each Deslisting object and access the warning dates.
for deslisting in history:
    if deslisting.type == DelistingType.WARNING:
        warning_date = deslisting.end_time
```

Universes

To get historical universe data , call the `history` method with the `Universe` object. This method returns a DataFrame with columns for the data point attributes.

```
PY
class USEquityUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of US Equities based on the constituents of an ETF.
        universe = self.add_universe(self.universe.etf('SPY'))
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		lastupdate	period	sharesheld	weight
time	**symbol**				
2024-12-19	**A RPTMYV3VC57P**	2024-12-17	1 days	3682327.0	0.000777
	AAPL R735QTJ8XC9X	2024-12-17	1 days	191633535.0	0.075065
	ABBV VCY032R250MD	2024-12-17	1 days	22256743.0	0.006032
	ABNB XK8H247DY6W5	2024-12-17	1 days	5536841.0	0.001131
	ABT R735QTJ8XC9X	2024-12-17	1 days	21919703.0	0.003837
...	**...**	...	...	...	...
2024-12-21	**XYL V18KR26TE3XH**	2024-12-19	1 days	2947931.0	0.000569
	YUM R735QTJ8XC9X	2024-12-19	1 days	3412693.0	0.000737
	ZMH S6ZZPKTVDY05	2024-12-19	1 days	2471688.0	0.000433
	ZBRA R735QTJ8XC9X	2024-12-19	1 days	625772.0	0.000400
	ZTS VDRJHVQ4FNFP	2024-12-19	1 days	5484351.0	0.001486

```py
# Select the 2 assets with the smallest weights in the ETF each day.
daily_smallest = history.groupby('time').apply(lambda x: x.nsmallest(2, 'weight')).reset_index(level=1, drop=True).weight
```

```
time        symbol
2024-12-19  AMTMW YM37RIGZUD0L    0.000053
            NWSVV VHJF6S7EZRL1    0.000068
2024-12-20  AMTMW YM37RIGZUD0L    0.000051
            NWSVV VHJF6S7EZRL1    0.000069
2024-12-21  AMTMW YM37RIGZUD0L    0.000048
            NWSVV VHJF6S7EZRL1    0.000069
Name: weight, dtype: float64
```

To get the data in the format of the objects that you receive in your universe filter function instead of a DataFrame, use `flatten=False`. This call returns a Series where the values are lists of the universe data objects.

```py
# Get the historical universe data over the last 30 days in a Series where
# the values in the series are lists of the universe selection objects.
history = self.history(universe, timedelta(30), flatten=False)
# Iterate through each day of universe selection.
for (universe_symbol, end_time), constituents in history.items():
    # Select the 10 largest assets in the ETF on this day.
    largest = sorted(constituents, key=lambda c: c.weight)[-10:]
```

Alternative Data

To get historical alternative data, call the `history` method with the dataset `Symbol`. This method returns a DataFrame that contains the data point attributes.

```
class USEquityAlternativeDataHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_equity('GME').symbol
        # Add the alternative dataset and save a reference to its Symbol.
        dataset_symbol = self.add_data(QuiverWallStreetBets, symbol).symbol
        # Get the trailing 5 days of QuiverWallStreetBets data for the asset in DataFrame format.
        history = self.history(dataset_symbol, 5, Resolution.DAILY)
```

		date	mentions	rank	sentiment
symbol	**time**				
GME.QuiverWallStreetBets	**2024-12-15**	2024-12-14	2	48	0.74245
	2024-12-16	2024-12-15	1	136	-0.51600
	2024-12-17	2024-12-16	8	53	0.13987
	2024-12-18	2024-12-17	17	32	0.08564
	2024-12-19	2024-12-18	4	117	0.61682

```
# Calculate the changes in sentiment.
sentiment_diff = history.sentiment.diff().iloc[1:]
```

```
symbol                    time
GME.QuiverWallStreetBets  2024-12-16   -1.25845
                          2024-12-17    0.65587
                          2024-12-18   -0.05423
                          2024-12-19    0.53118
Name: sentiment, dtype: float64
```

If you intend to use the data in the DataFrame to create `alternativeDataClass` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of dataset objects instead of a DataFrame, call the `history[ alternativeDataClass ]` method.

```
# Get the trailing 5 days of QuiverWallStreetBets data for an asset in QuiverWallStreetBets format.
history = self.history[QuiverWallStreetBets](dataset_symbol, 5, Resolution.DAILY)
# Iterate through each QuiverWallStreetBets object.
for data_point in history:
    t = data_point.end_time
    sentiment = data_point.sentiment
```

Some alternative datasets provide multiple entries per asset per time step . For example, the True Beats dataset provides EPS and revenue predictions for several upcoming quarters per asset per day. In this case, to organize the data into a DataFrame, set the `flatten` argument to `True` .

```
class USEquityTrueBeatsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Get the ExtractAlphaTrueBeats data for AAPL on 01/02/2024 organized in a flat DataFrame.
        aapl = self.add_equity("AAPL", Resolution.DAILY)
        aapl.true_beats = self.add_data(ExtractAlphaTrueBeats, aapl.symbol).symbol
        history = self.history(
            aapl.true_beats, datetime(2024, 1, 2), datetime(2024, 1, 3), Resolution.DAILY, flatten=True
        )
```

		analyst	estimates	count	earnings	metric	expected	reported	date	fiscal	quarter	fiscal	year	manage	ment	quoted	ly	time	trend	beat	true	beat
time	symbol																					
2024-01-02 12:30:00	AAPL.Extract	28	Alpha	True	Beats	0	2023-12-31	2024-01-25	-0.001241	4.0	2023	0.013694	True	2024-01-02 12:30:00	0.016899	0.029352						
	AAPL.Extract	28	Alpha	True	Beats	0	2024-09-30	2024-10-24	NaN	NaN	2024	NaN	False	2024-01-02 12:30:00	NaN	-0.020608						
	AAPL.Extract	24	Alpha	True	Beats	0	2024-03-31	2024-04-25	NaN	1.0	2024	NaN	True	2024-01-02 12:30:00	NaN	-0.016578						
	AAPL.Extract	24	Alpha	True	Beats	0	NaT	NaT	NaN	2.0	2024	NaN	True	2024-01-02 12:30:00	NaN	0.011859						
	AAPL.Extract	28	Alpha	True	Beats	0	NaT	NaT	NaN	3.0	2024	NaN	True	2024-01-02 12:30:00	NaN	0.017517						
	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...						
	AAPL.Extract	11	Alpha	True	Beats	0	NaT	NaT	NaN	1.0	2025	NaN	True	2024-01-02 12:30:00	NaN	-0.009165						
	AAPL.Extract	11	Alpha	True	Beats	0	NaT	NaT	NaN	2.0	2025	NaN	True	2024-01-02 12:30:00	NaN	-0.020219						
	AAPL.Extract	11	Alpha	True	Beats	0	NaT	NaT	NaN	3.0	2025	NaN	True	2024-01-02 12:30:00	NaN	-0.042210						
	AAPL.Extract	11	Alpha	True	Beats	0	NaT	NaT	NaN	4.0	2025	NaN	True	2024-01-02 12:30:00	NaN	0.000000						
	AAPL.Extract	10	Alpha	True	Beats	0	NaT	NaT	NaN	NaN	2026	NaN	False	2024-01-02 12:30:00	NaN	-0.001965						

PY

```
# Calculate the mean TrueBeat estimate for each day.
mean_true_beats_by_day = history.drop('time', axis=1).groupby('time').apply(lambda g: g.truebeat.mean())
```

```
time
2024-01-02 12:30:00   -0.007034
dtype: float64
```

For information on historical data for other alternative datasets, see the documentation in the Dataset Market .

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class USEquityIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_equity('SPY').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 16:00:00	599.186667	12582.92
2024-12-13 16:00:00	599.520952	12589.94
2024-12-16 16:00:00	600.160952	12603.38
2024-12-17 16:00:00	601.043810	12621.92
2024-12-18 16:00:00	600.954762	12620.05

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class USEquityMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_equity('AAPL').symbol
        reference_symbol = self.add_equity('SPY').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Short Availability

To get historical values for the borrow fee rate, borrow rebate rate, or shortable quantity of a US Equity, call the `fee_rate` , `rebate_rate` , or `shortable_quantity` methods of its shortable provider .

```
class USEquityShortAvailabilityHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add an asset and save a reference to its Equity object.
        security = self.add_equity('SPY')
        # Overwrite the default shortable provider to one that provides data.
        security.set_shortable_provider(InteractiveBrokersShortableProvider())
        # Select a time in the past.
        t = self.time - timedelta(30)
        # Pass the time argument to shortable provider to get historical values.
        fee_rate = security.shortable_provider.fee_rate(security.symbol, t)
        rebate_rate = security.shortable_provider.rebate_rate(security.symbol, t)
        shortable_quantity = security.shortable_provider.shortable_quantity(security.symbol, t)
```

For more information about security initializers, see Set Security Initializer .

Examples

The following examples demonstrate some common practices for trading using historical requests.

Example 1: Mean-Variance Portfolio

The following algorithm constructs a monthly rebalance mean-variance portfolio using the top 20 liquid equities. The position sizing can be optimized by 1-year historical daily return of the universe members.

```
from Portfolio.MaximumSharpeRatioPortfolioOptimizer import MaximumSharpeRatioPortfolioOptimizer

class HistoricalRequestAlgorithm(QCAlgorithm):
    # Instantiate the optimizer to perform mean-variance optimization.
    # Mean-variance optimization will not consider a risk-free rate, so we use 0.
    _optimizer = MaximumSharpeRatioPortfolioOptimizer(0.0, 1.0, 0.0)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Seed the last price to set the initial price of the BTCUSDT holdings.
        self.settings.seed_initial_prices = True

        # Monthly renewal of the top 20 liquid universe to trade popular stocks.
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.top(20))

        # Set a scheduled event to rebalance the portfolio at the start of every month.
        self.schedule.on(self.date_rules.month_start(), self.time_rules.at(9, 31), self.rebalance)

    def rebalance(self) -> None:
        # Historical data request to get 1-year data for optimization.
        symbols = self._universe.selected
        history = self.history(symbols, 253, Resolution.DAILY).close.unstack(0).dropna()
        # Daily return on the universe members to calculate the optimized weights.
        returns = history.pct_change().dropna()

        # Calculate the optimized weights.
        weights = self._optimizer.optimize(returns)

        # Rebalance the portfolio according to the optimized weights.
        targets = [PortfolioTarget(symbol, size if abs(size) > 0.01 else 0) for symbol, size in zip(symbols, weights)]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
HistoryAlgorithm.py Python

Asset Classes

Equity Options

Introduction

This page explains how to get historical data for Equity Options. Some of the data you can get include prices, open interest, Greeks, and universe selection data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```
class EquityOptionTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        equity = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW)
        symbol = sorted(self.option_chain(equity.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

PY

					close	high	low	open	volume
expiry	**strike**	**type**	**symbol**	**time**					
2025-01-17	**440.0**	**1**	**SPY 32OCGBPPW6DTY|SPY R735QTJ8XC9X**	**2024-12-13 16:00:00**	0.20	0.23	0.20	0.21	99.0
				2024-12-13 16:00:00	0.20	0.21	0.17	0.18	543.0
				2024-12-16 16:00:00	0.18	0.19	0.17	0.17	58.0
				2024-12-17 16:00:00	0.24	0.24	0.19	0.19	101.0
				2024-12-18 16:00:00	0.85	0.85	0.21	0.21	160.0

```
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

PY

```
expiry      strike  type  symbol                              time
2025-01-17  440.0   1     SPY 32OCGBPPW6DTY|SPY R735QTJ8XC9X  2024-12-13 16:00:00    0.000000
                                                              2024-12-16 16:00:00   -0.100000
                                                              2024-12-17 16:00:00    0.333333
                                                              2024-12-18 16:00:00    2.541667

Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
class EquityOptionsQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        equity = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW)
        symbol = sorted(self.option_chain(equity.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

					askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
expiry	strike	type	symbol	time														
2025-01-17	440.0	1	SPY 32OCGBPPW6DTY|SPY R735QTJ8XC9X	2024-12-18 15:56:00	0.76	0.85	0.71	0.75	72.0	0.74	0.74	0.17	0.60	72.0	0.75	0.795	0.440	0.675
				2024-12-18 15:57:00	0.79	0.80	0.76	0.76	143.0	0.77	0.78	0.73	0.74	60.0	0.78	0.790	0.745	0.750
				2024-12-18 15:58:00	0.80	0.80	0.78	0.79	169.0	0.78	0.78	0.75	0.77	60.0	0.79	0.790	0.765	0.780
				2024-12-18 15:59:00	0.82	0.83	0.79	0.80	72.0	0.80	0.80	0.76	0.78	72.0	0.81	0.815	0.775	0.790
				2024-12-18 16:00:00	0.91	0.91	0.82	0.82	346.0	0.87	0.89	0.80	0.80	72.0	0.89	0.900	0.810	0.810

```
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
expiry      strike  type  symbol                              time
2025-01-17  440.0   1     SPY 32OCGBPPW6DTY|SPY R735QTJ8XC9X  2024-12-18 15:56:00    0.02
                                                              2024-12-18 15:57:00    0.02
                                                              2024-12-18 15:58:00    0.02
                                                              2024-12-18 15:59:00    0.02
                                                              2024-12-18 16:00:00    0.04
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise,

LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```py
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```py
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        equity = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW)
        contract = sorted(self.option_chain(equity.symbol), key=lambda c: c.open_interest)[-1]
        self.add_option_contract(contract.symbol)
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Open Interest

To get historical open interest data , call the `history` method with the `OpenInterest` type and a security's `Symbol` . This method returns a DataFrame with a single column.

```py
class EquityOptionOpenInterestHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        equity = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW)
        symbol = sorted(self.option_chain(equity.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing daily OpenInterest objects of the security in DataFrame format.
        history = self.history(OpenInterest, symbol, 5, Resolution.DAILY)
```

					openinterest
expiry	**strike**	**type**	**symbol**	**time**	
2025-01-17	**440.0**	**1**	**SPY 250117P00440000**	**2024-12-13**	171751.0
				2024-12-16	172190.0
				2024-12-17	172157.0
				2024-12-18	172147.0
				2024-12-19	172099.0

```
# Calculate the daily change in open interest.
oi_delta = history.openinterest.diff().iloc[1:]
```

```
expiry      strike  type  symbol                 time
2025-01-17  440.0   1     SPY   250117P00440000  2024-12-16    439.0
                                                 2024-12-17    -33.0
                                                 2024-12-18    -10.0
                                                 2024-12-19    -48.0
Name: openinterest, dtype: float64
```

If you intend to use the data in the DataFrame to create `OpenInterest` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `OpenInterest` objects instead of a DataFrame, call the `history[OpenInterest]` method.

```
# Get the 5 trailing daily OpenInterest objects of the security in OpenInterest format.
history = self.history[OpenInterest](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for oi in history:
    t = oi.end_time
    open_interest = oi.value
```

Daily Option Chains

To get historical daily Option chain data , call the `history` method with the Option `Symbol` object. The data this method returns contains information on all the currently tradable contracts, not just the contracts that pass your filter. If you pass `flatten=True` , this method returns a DataFrame with columns for the data point attributes.

```
class EquityOptionDailyOptionChainHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add an Equity Option universe.
        option = self.add_option('SPY')
        # Get the trailing 5 daily Option chains in DataFrame format.
        history = self.history(option.symbol, 5, flatten=True)
```

		close	delta	gamma	high	implied_volatility	low	open	open_interest	rho	theta	underlying_value	vega	volume
time	symbol													
2024-12-13	SPY YOGVNNC[illegible]|SPY R735QTJ8XC9X	[illegible]	0.000000	0.000000	487.945	3.212331	484.225	486.74	1539.0	0.000000	0.000000	SPY: 604.33	484.815	0.000000	0.0
	SPY YOGVNNC[illegible]|SPY R735QTJ8XC9X	[illegible]	0.000000	0.000000	477.970	3.055466	474.210	476.91	26.0	0.000000	0.000000	SPY: 604.33	474.210	0.000000	0.0
	SPY YOGVNNC[illegible]|SPY R735QTJ8XC9X	[illegible]	0.000000	0.000000	467.965	2.910873	464.230	466.91	4.0	0.000000	0.000000	SPY: 604.33	464.835	0.000000	0.0
	SPY YOGVNNC[illegible]|SPY R735QTJ8XC9X	[illegible]	0.000000	0.000000	458.000	2.775398	454.245	456.92	18.0	0.000000	0.000000	SPY: 604.33	454.845	0.000000	0.0
	SPY YTG30NX[illegible]|SPY R735QTJ8XC9X	[illegible]	0.000000	0.000000	459.235	0.687831	456.270	458.77	33.0	0.000000	0.000000	SPY: 604.33	456.345	0.000000	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-12-19	SPY 33899RR[illegible]|SPY R735QTJ8XC9X	[illegible]	0.865013	0.000496	317.045	0.145628	293.900	297.42	1.0	-5.866155	0.038392	SPY: 586.28	314.995	0.573632	0.0
	SPY 337HP99[illegible]|SPY R735QTJ8XC9X	[illegible]	0.874953	0.000412	321.765	0.142599	299.020	302.42	0.0	-5.754009	0.039155	SPY: 586.28	319.995	0.452253	0.0
	SPY 33899RV[illegible]|SPY R735QTJ8XC9X	[illegible]	0.868866	0.000447	321.805	0.143224	298.995	302.47	0.0	-5.954058	0.039088	SPY: 586.28	319.990	0.511173	0.0
	SPY 337HP95[illegible]|SPY R735QTJ8XC9X	[illegible]	0.876073	0.000389	326.715	0.142820	304.030	307.43	0.0	-5.842472	0.040096	SPY: 586.28	324.995	0.428058	0.0
	SPY 33899RS[illegible]|SPY R735QTJ8XC9X	[illegible]	0.868426	0.000446	326.745	0.145502	303.975	307.46	1.0	-6.048355	0.039695	SPY: 586.28	324.995	0.517696	0.0

PY

```
# Select the 2 contracts with the greatest volume each day.
most_traded = history.groupby('time').apply(lambda x: x.nlargest(2, 'volume')).reset_index(level=1, drop=True).volume
```

```
time        symbol
2024-12-13  SPY 32NDZOIHRHLRA|SPY R735QTJ8XC9X    166689.0
            SPY 32NDZOIHXFXT2|SPY R735QTJ8XC9X    160265.0
2024-12-14  SPY YOCXVCLWHPT2|SPY R735QTJ8XC9X     137597.0
            SPY YOCXVCM2G1UU|SPY R735QTJ8XC9X     100401.0
2024-12-17  SPY 32NHXGVYLQYG6|SPY R735QTJ8XC9X    104700.0
            SPY YODXBFZKFH46|SPY R735QTJ8XC9X     103635.0
2024-12-18  SPY 32NIWWZBFX1IE|SPY R735QTJ8XC9X    106804.0
            SPY YOEWRJCELK6E|SPY R735QTJ8XC9X      75769.0
2024-12-19  SPY 32NKVT60AHJDY|SPY R735QTJ8XC9X    109298.0
            SPY YOFW7MPKOBC6|SPY R735QTJ8XC9X      94823.0
Name: volume, dtype: float64
```

To get the data in the format of `OptionUniverse` objects instead of a DataFrame, call the `history[OptionUniverse]` method.

```
# Get the historical OptionUniverse data over the last 30 days.
history = self.history[OptionUniverse](option.symbol, timedelta(30))
# Iterate through each daily Option chain.
for option_universe in history:
    t = option_universe.end_time
    # Select the contract with the most volume.
    most_traded = sorted(option_universe, key=lambda contract: contract.volume)[-1]
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class EquityOptionIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of the Option contract.
        underlying = self.add_equity('SPY').symbol
        symbol = sorted(self.option_chain(underlying), key=lambda c: c.open_interest)[-1].symbol
        # Get the 21-day SMA values of the contract for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 16:00:00	0.377619	7.930
2024-12-13 16:00:00	0.366190	7.690
2024-12-16 16:00:00	0.353810	7.430
2024-12-17 16:00:00	0.336667	7.070
2024-12-18 16:00:00	0.356429	7.485

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of multiple securities to compute their value (for example, the indicators for the Greeks and implied volatility). In this case, pass a list of the `Symbol` objects to the method.

```
class EquityOptionMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of the underlying asset.
        underlying = self.add_equity('SPY').symbol
        # Get the Option contract Symbol.
        option = sorted(self.option_chain(underlying), key=lambda c: c.open_interest)[-1].symbol
        # Get the Symbol of the mirror contract.
        mirror = Symbol.create_option(
            underlying, option.id.market, option.id.option_style,
            OptionRight.CALL if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
            option.id.strike_price, option.id.date
        )
        # Create the indicator.
        indicator = ImpliedVolatility(
            option, self.risk_free_interest_rate_model, DividendYieldProvider(underlying),
            mirror, OptionPricingModelType.FORWARD_TREE
        )
        # Get the historical values of the indicator over the last 60 trading minutes.
        history = self.indicator_history(indicator, [underlying, option, mirror], 60, Resolution.MINUTE)
        # Get the average IV value.
        iv_avg = history.data_frame.current.mean()
```

Examples

The following examples demonstrate some common practices for trading using historical requests.

Example 1: Trend Following on 0DTE Option Contract

This algorithm strategically trades 0DTE SPY options by analyzing bid and ask volumes shortly after the market opens. Using scheduled events, it effectively executes trades based on historical quote data, optimizing decision-making. The algorithm aims for timely entries and exits, ensuring efficient capital management and quick adaptability in volatile markets.

```
class ZeroDTEOptionsTradingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 10)  # Limit to a single day for 0DTE
        self.set_cash(100000)  # Starting cash

        # Request SPY option data for trading and signal generation.
        option = self.add_option("SPY")
        # We are interested in ATM 0DTE options since they are the most popular.
        option.set_filter(lambda u: u.expiration(0, 0).strikes(-3, 3))
        self._option = option.symbol

        # Schedule event to enter and exit option contract position.
        self.schedule.on(self.date_rules.every_day(self._option), self.time_rules.after_market_open(self._option, 16), self.trade_option)
        self.schedule.on(self.date_rules.every_day(self._option), self.time_rules.before_market_close(self._option, 15), self.liquidate)

    def trade_option(self) -> None:
        # Get the option chain for SPY to trade.
        option_chain = self.current_slice.option_chains.get(self._option, None)
        if option_chain:
            for option in option_chain:
                # Request historical quote data for signal generation.
                history = self.history(QuoteBar, option.symbol, 15, Resolution.Minute)
                if not history.empty:
                    # Calculate total bid and ask dollar volume to determine the capital directional force.
                    total_bid_volume = (history['bidclose'] * history['bidsize']).sum()
                    total_ask_volume = (history['askclose'] * history['asksize']).sum()

                    # Follow the capital flow to trade.
                    if total_bid_volume > total_ask_volume:
                        self.market_order(option.symbol, 1)
                    else:
                        self.market_order(option.symbol, -1)
```

Asset Classes

Crypto

Introduction

This page explains how to get historical data for Crypto. Some of the data you can get include prices, indicators, and universe selection data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```
PY
class CryptoTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto('BTCUSD').symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

		close	high	low	open	volume
symbol	**time**					
BTCUSD	**2024-12-15**	101399.99	102650.00	100600.00	101423.26	4054.541500
	2024-12-16	104439.88	105100.00	101221.34	101400.00	7216.743790
	2024-12-17	106099.81	107857.79	103289.21	104445.15	22263.157625
	2024-12-18	106150.00	108388.88	105337.97	106099.98	11729.293641
	2024-12-19	100150.73	106528.13	99939.82	106147.77	21659.470502

```
PY
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
symbol  time
BTCUSD  2024-12-16    0.029979
        2024-12-17    0.015894
        2024-12-18    0.000473
        2024-12-19   -0.056517
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```
PY
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
PY
class CryptoQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto('BTCUSD', market=Market.BITFINEX).symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

		askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
symbol	**time**														
BTCUSD	**2024-12-19 04:56:00**	100830.0	100830.0	100830.0	100830.0	0.496728	100820.0	100820.0	100820.0	100820.0	0.050586	100825.0	100825.0	100825.0	100825.0
	2024-12-19 04:57:00	100810.0	100830.0	100810.0	100830.0	1.200384	100800.0	100820.0	100800.0	100820.0	0.130504	100805.0	100825.0	100805.0	100825.0
	2024-12-19 04:58:00	100760.0	100810.0	100760.0	100810.0	0.520363	100750.0	100800.0	100750.0	100800.0	0.275894	100755.0	100805.0	100755.0	100805.0
	2024-12-19 04:59:00	100710.0	100760.0	100710.0	100760.0	1.716247	100700.0	100750.0	100700.0	100750.0	0.270080	100705.0	100755.0	100705.0	100755.0
	2024-12-19 05:00:00	100710.0	100710.0	100710.0	100710.0	0.712784	100700.0	100700.0	100700.0	100700.0	0.389266	100705.0	100705.0	100705.0	100705.0

```
PY
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
symbol  time
BTCUSD  2024-12-19 04:56:00    10.0
        2024-12-19 04:57:00    10.0
        2024-12-19 04:58:00    10.0
        2024-12-19 04:59:00    10.0
        2024-12-19 05:00:00    10.0
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
PY
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
class CryptoTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto('BTCUSD', market=Market.BITFINEX).symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

		askprice	asksize	bidprice	bidsize	lastprice	quantity
symbol	**time**						
BTCUSD	**2024-12-17 05:00:00.915569**	106390.0	1.072804	106380.0	16.521214	106385.0	0.0
	2024-12-17 05:00:01.119192	106390.0	1.302212	106380.0	16.521214	106385.0	0.0
	2024-12-17 05:00:05.981284	106390.0	1.131937	106380.0	16.521214	106385.0	0.0
	2024-12-17 05:00:07.302974	106390.0	1.498471	106380.0	16.521214	106385.0	0.0
	2024-12-17 05:00:07.514624	106390.0	1.631937	106380.0	16.521214	106385.0	0.0

```
# Select the rows in the DataFrame that represent trades. Drop the bid/ask columns since they are NaN.
trade_ticks = history[history.quantity > 0].dropna(axis=1)
```

		lastprice	quantity
symbol	**time**		
BTCUSD	**2024-12-17 05:00:13.647**	106380.0	0.201870
	2024-12-17 05:01:13.130	106390.0	0.000747
	2024-12-17 05:01:23.446	106380.0	0.197494
	2024-12-17 05:01:31.421	106390.0	0.031909
	2024-12-17 05:01:31.460	106390.0	0.000086

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the quote size.
for tick in history:
    if tick.tick_type == TickType.Quote:
        t = tick.end_time
        size = max(tick.bid_size, tick.ask_size)
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```py
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_crypto('BTCUSD')
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Universes

To get historical universe data , call the `history` method with the `Universe` object. This method doesn't apply your selection function . It returns all of the universe data in a DataFrame with columns for the data point attributes.

```py
class CryptoUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Cryptocurrencies on Coinbase.
        universe = self.add_universe(
            CryptoUniverse.coinbase(
                lambda constitutents: [c.symbol for c in constitutents]
            )
        )
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		close	high	low	open	price	volume	volumeinusd
time	symbol							
2024-12-19	00USD 2XR	0.050300	0.051500	0.049200	0.051500	0.050300	3.173495e+06	1.596268e+05
	00USDC 2XR	0.050300	0.051500	0.049200	0.051500	0.050300	3.173495e+06	1.596268e+05
	1INCHBTC 2XR	0.000004	0.000005	0.000004	0.000005	0.000004	9.815380e+03	4.584371e+03
	1INCHEUR 2XR	0.446000	0.467000	0.442000	0.465000	0.446000	9.007325e+04	4.215390e+04
	1INCHGBP 2XR	0.369000	0.385000	0.366000	0.381000	0.369000	3.932220e+04	1.845807e+04
...	...	...	...	...	...	...	...	...
2024-12-23	ZETAUSDC 2XR	0.574500	0.638600	0.562600	0.598400	0.574500	1.522481e+06	8.746654e+05
	ZROUSD 2XR	5.487000	6.363000	5.393000	5.635000	5.487000	1.312069e+05	7.199321e+05
	ZROUSDC 2XR	5.487000	6.363000	5.393000	5.635000	5.487000	1.312069e+05	7.199321e+05
	ZRXUSD 2XR	0.449944	0.509425	0.441590	0.476001	0.449944	3.371118e+06	1.516814e+06
	ZRXUSDC 2XR	0.449944	0.509425	0.441590	0.476001	0.449944	3.371118e+06	1.516814e+06

PY

```
# Select the 2 assets with the largest dollar volume each day.
most_traded = history.groupby('time').apply(lambda x: x.nlargest(2, 'volumeinusd')).reset_index(level=1, drop=True).volumeinusd
```

```
time        symbol
2024-12-19  BTCUSD         1.245065e+09
            BTCUSDC 2XR    1.245065e+09
2024-12-20  BTCUSD         2.169212e+09
            BTCUSDC 2XR    2.169212e+09
2024-12-21  BTCUSD         2.231721e+09
            BTCUSDC 2XR    2.231721e+09
2024-12-22  BTCUSD         2.077999e+09
            BTCUSDC 2XR    2.077999e+09
2024-12-23  BTCUSD         6.169765e+08
            BTCUSDC 2XR    6.169765e+08
Name: volumeinusd, dtype: float64
```

To get the data in the format of the objects that you receive in your universe filter function instead of a DataFrame, use `flatten=False`. This call returns a Series where the values are lists of the universe data objects.

PY

```
# Get the historical universe data over the last 30 days in a Series where
# the values in the series are lists of the universe selection objects.
history = self.history(universe, timedelta(30), flatten=False)
# Iterate through each day of universe selection.
for (universe_symbol, end_time), constituents in history.items():
    # Select the 10 assets with the largest dollar volume this day.
    most_liquid = sorted(constituents, key=lambda c: c.volume_in_usd)[-10:]
```

Alternative Data

To get historical alternative data, call the `history` method with the dataset `Symbol`. This method returns a DataFrame that contains the data point attributes.

```
class CryptoAlternativeDataHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an asset.
        symbol = self.add_crypto('BTCUSD').symbol
        # Add the alternative dataset and save a reference to its Symbol.
        dataset_symbol = self.add_data(BitcoinMetadata, symbol).symbol
        # Get the trailing 5 days of alternative data for the asset in DataFrame format.
        history = self.history(dataset_symbol, 5, Resolution.DAILY)
```

		average																				
symbol	time																					
BTCUSD.BitcoinMetadata	2024-12-15	1.555	462	252	0.96	29	99.784	5	0391	9	2	5	74	2	3	5	8	2	6	7	8	9	9	8	4	6	7	2..		897	8	0	4	3	7	4	0	6	1	3	8	2	1	0	7	2	6	9	5	0	7	3	7	0	1	9	7	9	6	1	2	2	1	4	2	6	3	0	6	5	1	4	9	3	3	.2167
	2024-12-16	1.501	462	276	0.9	5	6	9	18.200	1	0	3	9	1	9	6	6	4	9	6	2	8	2	6	1	2	3	8	7	4	2	6	6	3	8	8	2	6	7	2..		897	8	4	2	7	8	2	0	2	9	7	0	8	5	2	3	7	0	3	3	2	5	4	0	4	5	9	1	9	7	9	6	5	8	1	9	0	4	2	5	6	9	9	1	5	6	6	0	9	.890
	2024-12-17	1.599	362	297	0.2	8	6	6	20.902	5	8	2	4	2	5	6	5	5	4	9	6	3	7	2	9	6	3	5	2	4	8	6	6	9	8	6	7	2..		897	8	8	3	2	8	8	4	8	1	7	4	7	4	0	2	5	0	6	6	2	8	2	5	7	1	9	7	9	7	1	0	6	3	2	9	4	4	4	9	2	5	9	3	7	6	2	9	5	:
	2024-12-18	1.613	562	321	2.2	4	8	3	11.210	1	4	0	8	5	2	1	5	6	9	4	6	8	8	6	2	3	8	6	7	9	9	0	8	2	3	8	6	7	2..		897	9	2	1	5	0	4	8	6	0	4	9	9	0	7	4	0	9	4	8	4	5	8	7	9	5	5	1	9	7	9	7	5	1	0	3	2	7	0	8	3	9	9	0	3	2	4	7	7	8	6	.359	:
	2024-12-19	1.591	562	343	0.5	9	7	1	16.340	1	2	8	5	2	1	5	6	9	4	7	1	6	0	2	6	7	3	7	6	8	3	5	0	4	8	3	8	3	3	2		897	9	7	4	2	8	7	0	5	4	1	7	3	6	4	0	4	1	2	8	5	0	3	4	9	5	1	9	7	9	7	9	5	3	1	0	3	8	2	9	3	1	2	4	9	3	1	3	8	1	.368	:

```
# Calculate the growth in market cap.
growth = history.marketcapitalization.pct_change().iloc[1:]
```

```
symbol                  time
BTCUSD.BitcoinMetadata  2024-12-16    0.019420
                        2024-12-17    0.028577
                        2024-12-18    0.001342
                        2024-12-19   -0.030904
Name: marketcapitalization, dtype: float64
```

For information on historical data for other alternative datasets, see the documentation in the Dataset Market .

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class CryptoIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto('BTCUSD').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-15	97734.062857	2052415.32
2024-12-16	98039.381905	2058827.02
2024-12-17	98663.181905	2071926.82
2024-12-18	99340.640952	2086153.46
2024-12-19	99540.619048	2090353.00

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class CryptoMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_crypto('DOGEUSD').symbol
        reference_symbol = self.add_crypto('BTCUSD').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Examples

The following examples demonstrate some common practices for trading Cryptos using historical requests.

Example 1: Risk-Parity Crypto Pairs Portfolio

The following algorithm constructs a monthly rebalance risk-parity portfolio using the top 10 liquid Crypto pairs in the BitFinex market. A risk Parity portfolio can dissipate the risk of each crypto pair to reduce overall drawdown. The position sizing can be optimized by 1-year historical daily return of the universe members.

PY

```
from Portfolio.RiskParityPortfolioOptimizer import RiskParityPortfolioOptimizer

class CryptoHistoricalDataAlgorithm(QCAlgorithm):

    # Instantiate the optimizer to perform risk-parity optimization.
    # Risk Parity portfolio can dissipate the risk of each crypto pair to reduce overall drawdown.
    _optimizer = RiskParityPortfolioOptimizer(0.0, 1.0)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Seed the last price to set the initial price of the BTCUSDT holdings.
        self.settings.seed_initial_prices = True

        # Monthly renewal of the top 10 liquid universe to trade popular crypto pairs.
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(CryptoCoarseFundamentalUniverse(Market.BITFINEX, self.universe_settings, self.selection))

        # Set a scheduled event to rebalance the portfolio at the start of every month.
        self.schedule.on(
            self.date_rules.month_start(),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def selection(self, data: List[CryptoUniverse]) -> List[Symbol]:
        # Filter out the crypto pair with a price below $10 to avoid excessive volatility.
        filtered = [datum for datum in data if datum.volume_in_usd]
        # Select the top 10 dollar volume crypto pairs.
        sorted_by_volume_in_usd = sorted(filtered, key=lambda datum: datum.volume_in_usd, reverse=True)[:10]
        return [datum.symbol for datum in sorted_by_volume_in_usd]

    def rebalance(self) -> None:
        # Historical data request to get 1-year data for optimization.
        symbols = self._universe.selected
        history = self.history(symbols, 253, Resolution.DAILY).close.unstack(0).dropna()
        # Daily return on the universe members to calculate the optimized weights.
        returns = history.pct_change().dropna()

        # Calculate the optimized weights.
        weights = self._optimizer.optimize(returns)

        # Rebalance the portfolio according to the optimized weights.
        targets = [PortfolioTarget(symbol, size if abs(size) > 0.01 else 0) for symbol, size in zip(symbols, weights)]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Asset Classes

Crypto Futures

Introduction

This page explains how to get historical data for Crypto Futures. Some of the data you can get include prices, margin interest rates, and indicator data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```py
class CryptoFutureTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto_future('BTCUSD').symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

		close	high	low	open	volume
symbol	**time**					
BTCUSD	**2024-12-15**	101383.8	102683.9	100563.7	101403.8	16666498.0
	2024-12-16	104500.0	105323.5	101209.8	101383.9	26232316.0
	2024-12-17	106120.0	107872.1	103320.0	104500.0	44897036.0
	2024-12-18	106171.2	108496.9	105369.3	106120.0	41849071.0
	2024-12-19	100163.2	106550.2	99911.0	106171.2	65325574.0

```py
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
symbol  time
BTCUSD  2024-12-16    0.030737
        2024-12-17    0.015502
        2024-12-18    0.000482
        2024-12-19   -0.056588
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```py
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
class CryptoFutureQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto_future('BTCUSD').symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

		askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
symbol	**time**														
BTCUSD	**2024-12-19 04:56:00**	100744.2	100753.9	100727.5	100753.9	6420.0	100744.1	100753.8	100727.1	100753.8	4237.0	100744.15	100753.85	100727.3	100753.8
	2024-12-19 04:57:00	100727.3	100744.2	100727.3	100744.2	5674.0	100727.2	100744.1	100727.2	100744.1	742.0	100727.25	100744.15	100727.25	100744.1
	2024-12-19 04:58:00	100698.5	100727.3	100653.0	100727.3	6707.0	100698.4	100727.2	100652.8	100727.2	3719.0	100698.45	100727.25	100652.9	100727.2
	2024-12-19 04:59:00	100606.1	100698.5	100606.1	100698.5	1.0	100606.0	100698.4	100606.0	100698.4	5076.0	100606.05	100698.45	100606.05	100698.4
	2024-12-19 05:00:00	100644.1	100655.4	100606.1	100606.1	6005.0	100644.0	100655.3	100606.0	100606.0	611.0	100644.05	100655.35	100606.05	100606.0

```
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
symbol  time
BTCUSD  2024-12-19 04:56:00    0.1
        2024-12-19 04:57:00    0.1
        2024-12-19 04:58:00    0.1
        2024-12-19 04:59:00    0.1
        2024-12-19 05:00:00    0.1
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
PY
class CryptoFutureTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto_future('BTCUSD').symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

		askprice	asksize	bidprice	bidsize	lastprice	quantity
symbol	**time**						
BTCUSD	**2024-12-17 05:00:00.132868**	106537.1	12342.0	106537.0	35.0	106537.05	0.0
	2024-12-17 05:00:00.145098	106537.1	12353.0	106537.0	35.0	106537.05	0.0
	2024-12-17 05:00:00.424485	106537.1	12353.0	106537.0	355.0	106537.05	0.0
	2024-12-17 05:00:00.427716	106537.1	12353.0	106537.0	354.0	106537.05	0.0
	2024-12-17 05:00:00.431177	106537.1	12353.0	106537.0	355.0	106537.05	0.0

```
PY
# Select the rows in the DataFrame that represent trades. Drop the bid/ask columns since they are NaN.
trade_ticks = history[history.quantity > 0].dropna(axis=1)
```

		lastprice	quantity
symbol	**time**		
BTCUSD	**2024-12-17 05:00:01.008**	106537.1	2.0
	2024-12-17 05:00:02.729	106537.0	2.0
	2024-12-17 05:00:02.971	106537.0	5.0
	2024-12-17 05:00:05.977	106537.0	4.0
	2024-12-17 05:00:14.072	106537.1	56.0

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
PY
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the quote size.
for tick in history:
    if tick.tick_type == TickType.Quote:
        t = tick.end_time
        size = max(tick.bid_size, tick.ask_size)
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
PY
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_crypto_future('BTCUSD')
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Margin Interest Rates

To get historical margin interest rate data , call the `history` method with the `MarginInterestRate` type and a security's `Symbol` . This method returns a DataFrame with a single column for the interest rate.

```
PY
class CryptoFutureMarginInterestRateHistoryAlgorithmHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto_future('BTCUSD').symbol
        # Get the MarginInterestRate data of the security over the last 2 trading days in DataFrame format.
        history = self.history(MarginInterestRate, symbol, 2, Resolution.DAILY)
```

		interestrate
symbol	**time**	
BTCUSD	**2024-12-17 08:00:00**	0.0001
	2024-12-17 16:00:00	0.0001
	2024-12-18 00:00:00	0.0001
	2024-12-18 08:00:00	0.0001
	2024-12-18 16:00:00	0.0001
	2024-12-19 00:00:00	0.0001

```
PY
# Calculate the change in interest rates.
delta = history.interestrate.diff()[1:]
```

```
symbol  time
BTCUSD  2024-12-17 16:00:00    0.0
        2024-12-18 00:00:00    0.0
        2024-12-18 08:00:00    0.0
        2024-12-18 16:00:00    0.0
        2024-12-19 00:00:00    0.0
Name: interestrate, dtype: float64
```

If you intend to use the data in the DataFrame to create `MarginInterestRate` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `MarginInterestRate` objects

instead of a DataFrame, call the `history[MarginInterestRate]` method.

```
# Get the MarginInterestRate data of the security over the last 2 trading days in MarginInterestRate format.
history = self.history[MarginInterestRate](symbol, 2, Resolution.DAILY)
# Iterate through the MarginInterestRate objects and access their values
for margin_interest_rate in history:
    t = margin_interest_rate.end_time
    interest_rate = margin_interest_rate.interest_rate
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class CryptoFutureIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_crypto_future('BTCUSD').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-15	97759.090476	2052940.9
2024-12-16	98065.342857	2059372.2
2024-12-17	98689.038095	2072469.8
2024-12-18	99363.738095	2086638.5
2024-12-19	99563.576190	2090835.1

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class CryptoFutureMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_crypto_future('DOGEUSD').symbol
        reference_symbol = self.add_crypto_future('BTCUSD').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

PY

Examples

The following examples demonstrate common practices for trading cryptocurrency futures with historical data.

Example 1: Spot-Future Arbitrage

The example below uses linear regression with historical Spot and Future BTCUSDT data to obtain the cointegrating vector between them. We measure the spread between the two securities and trade mean reversal upon their spread divergence.

PY

```
from sklearn.linear_model import LinearRegression

class CointegrationVectorRegressionAlgorithm(QCAlgorithm):
    # Threshold of spread divergence to enter position.
    entry_threshold = 2

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_account_currency("USDT", 100000)
        self.set_cash("BTC", 2)

        # Request BTCUSDT Crypto and Crypto Future data to form the cointegrating vectors and trade.
        self._future = self.add_crypto_future("BTCUSDT", market=Market.BYBIT).symbol
        self._spot = self.add_crypto("BTCUSDT", market=Market.BYBIT).symbol

        # Initialize cointegration vector.
        self.cointegration_vector = np.array([0.0, 0.0])
        self.residual_mean = 0
        self.residual_std = 1

        # Daily recalibration on the cointegrating vectors.
        self.schedule.on(
            self.date_rules.every_day(self._future),
            self.time_rules.at(0, 1),
            self.recalibrate
        )

    def on_data(self, slice: Slice) -> None:
        y = slice.bars.get(self._future)
        x1 = slice.bars.get(self._spot)
        if y and x1:
            # Calculate the spread based on the cointegration vector.
            spread = self.calculate_spread(y.close, x1.close, self.cointegration_vector)
            normalized_spread = (spread - self.residual_mean) / self.residual_std

            # If the spread is too high, short the dependent and long the independent variables based on the weightings of the
cointegrating vectors.
            # Else, vice versa.
            total_weights = abs(self.cointegration_vector[1]) + 1
            if normalized_spread > self.entry_threshold and not self.portfolio[self._future].is_short:
                self.set_holdings(self._future, -0.5 / total_weights)
                self.set_holdings(self._spot, 0.5 * self.cointegration_vector[1] / total_weights)
            elif normalized_spread < -self.entry_threshold and not self.portfolio[self._future].is_long:
                self.set_holdings(self._future, 0.5 / total_weights)
                self.set_holdings(self._spot, -0.5 * self.cointegration_vector[1] / total_weights)

            # Exit if spread is converged.
            if (self.portfolio[self._future].is_short and normalized_spread < 0) or (self.portfolio[self._future].is_long and
normalized_spread > 0):
                self.liquidate()

    def recalibrate(self) -> None:
        # Calculate the cointegration vector from historical data.
        log_price = np.log(self.history([self._future, self._spot], 252, Resolution.DAILY).unstack(0).close.dropna())
        self.cointegration_vector = self.calculate_cointegrating_vector(log_price)
        residual = log_price.apply(lambda x: self.calculate_spread(x[self._future], x[self._spot], self.cointegration_vector), axis=1)
        self.residual_mean, self.residual_std = np.mean(residual), np.std(residual, ddof=1)

    def calculate_cointegrating_vector(self, log_price: pd.DataFrame) -> np.array:
        try:
            # Perform regression.
            lr = LinearRegression().fit(log_price[[self._spot]], log_price[self._future])

            # The coefficients array will contain the coefficients for each independent variable.
            return np.array([lr.intercept_, lr.coef_[0]])
        except:
            return self.cointegration_vector

    def calculate_spread(self, y: float, x1: float, cointegration_vector: np.array) -> float:
        # Using the cointegration vector to calculate the spread.
        return np.log(y) - (cointegration_vector[0] + np.log(x1) * cointegration_vector[1])
```

Asset Classes

Forex

Introduction

This page explains how to get historical data for Forex. Some of the data you can get include quotes, ticks, and indicator data.

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```py
class ForexQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_forex('EURUSD').symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

		askclose	askhigh	asklow	askopen	bidclose	bidhigh	bidlow	bidopen	close	high	low	open
symbol	**time**												
EURUSD	**2024-12-18 23:56:00**	1.03782	1.03809	1.03779	1.03809	1.03766	1.03794	1.03764	1.03794	1.037740	1.038015	1.037715	1.038015
	2024-12-18 23:57:00	1.03772	1.03789	1.03769	1.03782	1.03757	1.03773	1.03755	1.03766	1.037645	1.037810	1.037620	1.037740
	2024-12-18 23:58:00	1.03784	1.03786	1.03769	1.03772	1.03768	1.03771	1.03754	1.03757	1.037760	1.037785	1.037615	1.037645
	2024-12-18 23:59:00	1.03801	1.03807	1.03784	1.03784	1.03784	1.03790	1.03768	1.03768	1.037925	1.037985	1.037760	1.037760
	2024-12-19 00:00:00	1.03798	1.03803	1.03795	1.03801	1.03782	1.03788	1.03779	1.03784	1.037900	1.037955	1.037870	1.037925

```py
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
symbol  time
EURUSD  2024-12-18 23:56:00    0.00016
        2024-12-18 23:57:00    0.00015
        2024-12-18 23:58:00    0.00016
        2024-12-18 23:59:00    0.00017
        2024-12-19 00:00:00    0.00016
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the spread.
for quote_bar in history:
    t = quote_bar.end_time
    spread = quote_bar.ask.close - quote_bar.bid.close
```

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
class ForexTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_forex('EURUSD').symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

		askprice	bidprice	lastprice
symbol	**time**			
EURUSD	**2024-12-17 00:00:01.113566**	1.0509	1.05074	1.050820
	2024-12-17 00:00:01.158183	1.0509	1.05076	1.050830
	2024-12-17 00:00:08.514324	1.0509	1.05074	1.050820
	2024-12-17 00:00:09.034650	1.0509	1.05075	1.050825
	2024-12-17 00:00:09.222588	1.0509	1.05074	1.050820

```
# Calculate the spread.
spread = history.askprice - history.bidprice
```

```
symbol  time
EURUSD  2024-12-17 00:00:01.113566    0.00016
        2024-12-17 00:00:01.158183    0.00014
        2024-12-17 00:00:08.514324    0.00016
        2024-12-17 00:00:09.034650    0.00015
        2024-12-17 00:00:09.222588    0.00016
dtype: float64
```

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the spread.
for tick in history:
    t = tick.end_time
    spread = tick.bid_price - tick.ask_price
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
PY
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_forex('EURUSD')
        # Get the historical Slice objects over the last 5 minutes for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.MINUTE)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each QuoteBar in this Slice.
            for symbol, quote_bar in slice_.bars.items():
                midprice = quote_bar.close
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
PY
class ForexIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_forex('EURUSD').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
PY
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-13 19:00:00	1.051928	22.090485
2024-12-15 19:00:00	1.051755	22.086865
2024-12-16 19:00:00	1.051983	22.091650
2024-12-17 19:00:00	1.052336	22.099055
2024-12-18 19:00:00	1.051723	22.086175

```
PY
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's ask price.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.ASK_CLOSE)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class ForexMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_forex('AUDUSD').symbol
        reference_symbol = self.add_forex('EURUSD').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Examples

The following examples demonstrate some common practices for trading Forex with historical data.

Example 1: Arbitration with Cointegration

The below example uses linear regression with historical data of 3 related Forex pairs to obtain the cointegrating vector between them. We measure the spread between the 3 Forex pairs and trade mean reversal upon their spread divergence.

PY

```
from sklearn.linear_model import LinearRegression

class CointegrationVectorRegressionAlgorithm(QCAlgorithm):
    # Threshold of spread divergence to enter position.
    entry_threshold = 0.0001

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Request Forex data to form the cointegrating vectors and trade.
        self.currency_pairs = [self.add_forex(pair).symbol for pair in ["USDJPY", "EURUSD", "EURJPY"]]

        # Initialize cointegration vector
        self.cointegration_vector = np.array([0.0, 0.0, 0.0])

        # Daily recalibration on the cointegrating vectors.
        self.schedule.on(
            self.date_rules.every_day(self.currency_pairs[0]),
            self.time_rules.at(0, 1),
            self.recalibrate
        )

    def on_data(self, slice: Slice) -> None:
        y = slice.quote_bars.get(self.currency_pairs[0])
        x1 = slice.quote_bars.get(self.currency_pairs[1])
        x2 = slice.quote_bars.get(self.currency_pairs[2])
        if y and x1 and x2:
            # Calculate the spread based on the cointegration vector.
            spread = self.calculate_spread(y.close, x1.close, x2.close, self.cointegration_vector)

            # Exit if spread is converged.
            if (self.portfolio[self.currency_pairs[0]].is_short and spread < 0) or (self.portfolio[self.currency_pairs[0]].is_long and
spread > 0):
                self.liquidate()
                return

            # If the spread is too high, short the dependent and long the independent variables based on the weightings of the
cointegrating vectors.
            # Else, vice versa.
            total_weights = 1 / (abs(self.cointegration_vector[1]) + abs(self.cointegration_vector[2]) + 1)
            if spread > self.entry_threshold:
                self.set_holdings(self.currency_pairs[0], -total_weights)
                self.set_holdings(self.currency_pairs[1], self.cointegration_vector[1] * total_weights)
                self.set_holdings(self.currency_pairs[2], self.cointegration_vector[2] * total_weights)
            elif spread < -self.entry_threshold:
                self.set_holdings(self.currency_pairs[0], total_weights)
                self.set_holdings(self.currency_pairs[1], -self.cointegration_vector[1] * total_weights)
                self.set_holdings(self.currency_pairs[2], -self.cointegration_vector[2] * total_weights)

    def recalibrate(self) -> None:
        # Calculate the cointegration vector from historical data.
        log_price = np.log(self.history(self.currency_pairs, 252, Resolution.DAILY).unstack(0).close.dropna())
        self.cointegration_vector = self.calculate_cointegrating_vector(log_price)

    def calculate_cointegrating_vector(self, history: pd.DataFrame) -> np.array:
        try:
            # Perform regression.
            lr = LinearRegression().fit(history[self.currency_pairs[1:]], history[self.currency_pairs[0]])

            # The coefficients array will contain the coefficients for each independent variable.
            return np.array([lr.intercept_, lr.coef_[0], lr.coef_[1]])
        except:
            return self.cointegration_vector

    def calculate_spread(self, y: float, x1: float, x2: float, cointegration_vector: np.array) -> float:
        # Using the cointegration vector to calculate the spread.
        return np.log(y) - (cointegration_vector[0] + np.log(x1) * cointegration_vector[1] + np.log(x2) * cointegration_vector[2])
```

Asset Classes

Futures

Introduction

This page explains how to get historical data for Futures. Some of the data you can get include prices, open interest, indicators, and contract chain data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```py
class FutureTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        symbol = list(self.futures_chain(future.symbol))[0].symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

			close	high	low	open	volume
expiry	**symbol**	**time**					
2024-12-20 13:30:00	**ES YOGVNNAOI1OH**	**2024-12-12 17:00:00**	6058.75	6087.75	6055.50	6082.25	1152801.0
		2024-12-13 17:00:00	6051.25	6084.75	6041.25	6077.25	1027682.0
		2024-12-16 17:00:00	6076.50	6090.50	6064.75	6072.50	378263.0
		2024-12-17 17:00:00	6052.00	6063.50	6040.75	6052.75	278531.0
		2024-12-18 17:00:00	5872.25	6074.50	5840.00	6050.25	335048.0

```py
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
expiry               symbol          time
2024-12-20 13:30:00  ES YOGVNNAOI1OH  2024-12-13 17:00:00   -0.001238
                                      2024-12-16 17:00:00    0.004173
                                      2024-12-17 17:00:00   -0.004032
                                      2024-12-18 17:00:00   -0.029701
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
class FutureQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        symbol = list(self.futures_chain(future.symbol))[0].symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

			askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
expiry	symbol	time														
2024-12-20 13:30:00	ES YOGVNNAOI1OH	2024-12-12 17:00:00	6058.75	6087.75	6055.75	6082.50	19.0	6058.50	6087.50	6055.50	6082.25	52.0	6058.625	6087.625	6055.625	6082.37
		2024-12-13 17:00:00	6051.50	6085.00	6041.50	6077.50	59.0	6051.25	6084.75	6041.25	6077.25	1.0	6051.375	6084.875	6041.375	6077.37
		2024-12-16 17:00:00	6076.75	6090.75	6064.75	6072.50	10.0	6076.50	6090.50	6064.50	6072.25	7.0	6076.625	6090.625	6064.625	6072.37
		2024-12-17 17:00:00	6052.25	6063.50	6041.00	6053.00	12.0	6052.00	6063.25	6040.50	6052.75	1.0	6052.125	6063.375	6040.750	6052.87
		2024-12-18 17:00:00	5872.75	6074.75	5840.25	6050.25	2.0	5872.25	6074.50	5840.00	6050.00	1.0	5872.500	6074.625	5840.125	6050.12

```
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
expiry               symbol          time
2024-12-20 13:30:00  ES YOGVNNAOI1OH  2024-12-12 17:00:00    0.25
                                      2024-12-13 17:00:00    0.25
                                      2024-12-16 17:00:00    0.25
                                      2024-12-17 17:00:00    0.25
                                      2024-12-18 17:00:00    0.50
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the

`history[QuoteBar]` method.

```
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
class FutureTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        symbol = list(self.futures_chain(future.symbol))[0].symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

			askprice	asksize	bidprice	bidsize	lastprice	openinterest	quantity
expiry	**symbol**	**time**							
2024-12-20 13:30:00	**ES YOGVNNAOIOF**	**2024-12-17 09:30:00.001**	NaN	NaN	NaN	NaN	6052.75	NaN	1.0
		2024-12-17 09:30:00.001	0.0	0.0	6052.75	33.0	6052.75	NaN	0.0
		2024-12-17 09:30:00.001	6053.0	11.0	0.00	0.0	6053.00	NaN	0.0
		2024-12-17 09:30:00.001	6053.0	10.0	0.00	0.0	6053.00	NaN	0.0
		2024-12-17 09:30:00.001	6053.0	9.0	0.00	0.0	6053.00	NaN	0.0

```
# Select the rows in the DataFrame that represent trades. Drop the bid/ask columns since they are NaN.
trade_ticks = history[history.quantity > 0].dropna(axis=1)
```

			lastprice	quantity
expiry	**symbol**	**time**		
2024-12-20 13:30:00	**ES YOGVNNAOI1OH**	**2024-12-17 09:30:00.001**	6052.75	1.0
		2024-12-17 09:30:00.002	6053.00	1.0
		2024-12-17 09:30:00.002	6053.00	2.0
		2024-12-17 09:30:00.003	6052.75	1.0
		2024-12-17 09:30:00.003	6052.75	3.0

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the quote size.
for tick in history:
    if tick.tick_type == TickType.Quote:
        t = tick.end_time
        size = max(tick.bid_size, tick.ask_size)
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_future(Futures.Indices.SP_500_E_MINI)
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Continuous Contract Prices

To get historical trade, quote, or tick data for the continuous contract , use the `symbol` property of the `Future` object when you make the history request.

```
class FutureContinuousContractHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the Future and specify the contract rollover settings.
        future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        # Get the 5 trailing daily TradeBar objects of the continuous contract.
        history = self.history(TradeBar, future.symbol, 5, Resolution.DAILY)
```

			close	high	low	open	volume
expiry	**symbol**	**time**					
1899-12-30	**/ES**	**2024-12-12 17:00:00**	6132.582256	6161.935651	6129.292651	6156.368628	1152801.0
		2024-12-13 17:00:00	6124.990860	6158.899093	6114.869000	6151.307698	1027682.0
		2024-12-16 17:00:00	6150.548558	6164.719163	6138.655372	6146.499814	378263.0
		2024-12-17 17:00:00	6125.750000	6137.390140	6114.362907	6126.509140	278531.0
		2024-12-18 17:00:00	5939.500000	6148.000000	5906.500000	6124.000000	2009987.0

```
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
expiry      symbol  time
1899-12-30  /ES     2024-12-13 17:00:00   -0.001238
                    2024-12-16 17:00:00    0.004173
                    2024-12-17 17:00:00   -0.004032
                    2024-12-18 17:00:00   -0.030404
Name: close, dtype: float64
```

Symbol Changes

To get historical symbol change data , call the `history` method with the `SymbolChangedEvent` type and the continuous contract Symbol. This method returns a DataFrame that contains the old and new Symbol for each rollover.

```
class FutureSymbolChangeEventsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the Future and specify the contract rollover settings.
        future = self.add_future(
            Futures.Indices.SP_500_E_MINI,
            data_mapping_mode=DataMappingMode.OPEN_INTEREST,
            data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
            contract_depth_offset=0
        )
        # Get the contract rollovers for the trailing year.
        history = self.history(SymbolChangedEvent, future.symbol, timedelta(365))
```

			newsymbol	oldsymbol
expiry	**symbol**	**time**		
1899-12-30	**/ES**	**2024-03-13**	ES YJHOAMPYKQGX	ES YGT6HGVF2SQP
		2024-06-19	ES YLZ9Z50BJE2P	ES YJHOAMPYKQGX
		2024-09-18	ES YOGVNNAOI1OH	ES YLZ9Z50BJE2P
		2024-12-18	ES YQYHC5L1GPA9	ES YOGVNNAOI1OH

```
# Select the dates of each rollover.
dates = list(history.index.levels[2])
```

If you intend to use the data in the DataFrame to create `SymbolChangedEvent` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `SymbolChangedEvent` objects instead of a DataFrame, call the `history[SymbolChangedEvent]` method.

```
# Get the rollovers of the continuous contract during the last 10 years in SymbolChangedEvent format.
history = self.history[SymbolChangedEvent](symbol, timedelta(10*365))
# Iterate through each SymbolChangedEvent object.
for symbol_changed_event in history:
    t = symbol_changed_event.end_time
    old_symbol = symbol_changed_event.old_symbol
    new_symbol = symbol_changed_event.new_symbol
```

Open Interest

To get historical open interest data , call the `history` method with the `OpenInterest` type and a security's `Symbol` . This method returns a DataFrame with a single column.

```
class FutureOpenInterestHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        symbol = list(self.futures_chain(future.symbol))[0].symbol
        # Get the 5 trailing daily OpenInterest objects of the security in DataFrame format.
        history = self.history(OpenInterest, symbol, 5, Resolution.DAILY)
```

			openinterest
expiry	**symbol**	**time**	
2024-12-20 13:30:00	**ES YOGVNNAOI1OH**	**2024-12-14 19:00:00**	1631020.0
		2024-12-15 19:00:00	1625312.0
		2024-12-16 19:00:00	965789.0
		2024-12-17 19:00:00	687472.0
		2024-12-18 19:00:00	556365.0

```
# Calculate the daily change in open interest.
oi_delta = history.openinterest.diff().iloc[1:]
```

```
expiry               symbol           time
2024-12-20 13:30:00  ES YOGVNNAOI1OH  2024-12-15 19:00:00     -5708.0
                                      2024-12-16 19:00:00   -659523.0
                                      2024-12-17 19:00:00   -278317.0
                                      2024-12-18 19:00:00   -131107.0
Name: openinterest, dtype: float64
```

If you intend to use the data in the DataFrame to create `OpenInterest` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `OpenInterest` objects instead of a DataFrame, call the `history[OpenInterest]` method.

PY

```
# Get the 5 trailing daily OpenInterest objects of the security in OpenInterest format.
history = self.history[OpenInterest](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for oi in history:
    t = oi.end_time
    open_interest = oi.value
```

Contracts

To get historical Futures contracts, call the `history` method with the `FutureUniverse` type and the continuous contract `Symbol` . The data this method returns contains information on all the currently tradable contracts, not just the contracts that pass your filter. If you pass `flatten=True` , this method returns a DataFrame with columns for the data point attributes.

PY

```
class FutureContractsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add a Future.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        # Get the daily contract data over the last two trading days.
        history = self.history(FutureUniverse, future.symbol, 2, flatten=True)
```

		close	high	low	open	openinterest	value	volume
time	symbol							
2024-12-18	ES YOGVNNAOI1OH	6052.000	6063.500	6040.750	6052.750	687472.0	6052.000	278531.0
	ES YQYHC5L1GPA9	6125.750	6138.250	6114.250	6127.500	1902280.0	6125.750	997849.0
	ES YTG30NVEFCWT	6185.250	6198.750	6173.500	6184.250	5044.0	6185.250	197.0
	ES YVXOP65RE0HT	6238.250	6258.000	6229.375	6238.250	20.0	6238.250	0.0
	ES YYFADOG4CO3L	6299.125	6313.000	6057.000	6071.500	1106.0	6299.125	0.0
2024-12-19	ES YOGVNNAOI1OH	5872.250	6074.500	5840.000	6050.250	556365.0	5872.250	335048.0
	ES YQYHC5L1GPA9	5939.500	6148.000	5906.500	6124.000	2064361.0	5939.500	2009987.0
	ES YTG30NVEFCWT	6001.250	6205.000	5966.000	6181.250	5096.0	6001.250	536.0
	ES YVXOP65RE0HT	6054.250	6450.750	5934.375	6243.375	20.0	6054.250	0.0
	ES YYFADOG4CO3L	6315.000	6315.000	6315.000	6315.000	1116.0	6315.000	10.0
	ES Z0WW26QHB8PD	6171.375	6379.625	6142.000	6368.375	0.0	6171.375	0.0

```py
# Select the contract with the largest open interest each day.
most_oi = history.groupby('time').apply(lambda x: x.nlargest(1, 'openinterest')).reset_index(level=1, drop=True).openinterest
```

```
time        symbol
2024-12-18  ES YQYHC5L1GPA9    1902280.0
2024-12-19  ES YQYHC5L1GPA9    2064361.0
Name: openinterest, dtype: float64
```

If you intend to use the data in the DataFrame to create `FutureUniverse` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `FutureUniverse` objects instead of a DataFrame, call the `history[FutureUniverse]` method.

```py
# Get the 5 trailing daily FutureUniverse objects in FutureUniverse format.
history = self.history[FutureUniverse](future.symbol, 5, Resolution.DAILY)
# Iterate through the FutureUniverse objects and access their volumes.
for future_universe in history:
    t = future_universe.end_time
    most_oi = sorted(future_universe, key=lambda contract: contract.open_interest)[-1]
```

Contango

To get historical contango data for VX Futures , call the `history` method with the dataset `Symbol` . This method returns a DataFrame that contains the data point attributes.

```
class VIXContangoHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the dataset and save a reference to its Symbol.
        symbol = self.add_data(VIXCentralContango, "VIX", Resolution.DAILY).symbol
        # Get the trailing 7 days of contango data in DataFrame format.
        history = self.history(symbol, 7, Resolution.DAILY)
```

		contango_f2_minus_f1	contango_f3_minus_f1_div_2	contango_f4_minus_f1_div_3	f1	f2	f3	f4	f5	f6	f7	f8	frontmonthperiod
symbol	**time**												
VIX.VIXCentralContango	**2024-12-13**	0.1314	0.0498	0.0166	14.4062	16.2991	17.1553	17.9005	18.2460	18.5583	18.7923	18.9500	12 1 days
	2024-12-14	0.1212	0.0497	0.0166	14.5190	16.2782	17.1250	17.9474	18.2993	18.5750	18.8390	19.0500	12 1 days
	2024-12-17	0.1299	0.0457	0.0152	14.6997	16.6091	17.4592	18.1963	18.4965	18.7411	19.0282	19.1884	12 1 days
	2024-12-18	0.0765	0.0389	0.0130	15.7483	16.9531	17.6666	18.2998	18.4996	18.7185	19.0108	19.1500	12 1 days

```
# Calculate the changes in contango.
contango_diff = history.f1.diff().iloc[1:]
```

```
symbol                  time
VIX.VIXCentralContango  2024-12-14    0.1128
                        2024-12-17    0.1807
                        2024-12-18    1.0486
Name: f1, dtype: float64
```

If you intend to use the data in the DataFrame to create `VIXCentralContango` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of dataset objects instead of a DataFrame, call the `history[VIXCentralContango]` method.

```
# Get the trailing 7 days of contango data in VIXCentralContango format.
history = self.history[VIXCentralContango](symbol, 7, Resolution.DAILY)
# Iterate through each VIXCentralContango object.
for data_point in history:
    t = data_point.end_time
    value = data_point.contango_f_2_minus_f_1
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class FuturesIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_future(Futures.Indices.SP_500_E_MINI).symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 17:00:00	6150.577121	129162.119546
2024-12-13 17:00:00	6154.322490	129240.772299
2024-12-16 17:00:00	6162.919815	129421.316116
2024-12-17 17:00:00	6169.413410	129557.681602
2024-12-18 17:00:00	6166.050570	129487.061970

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class FuturesMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_future(Futures.Metals.GOLD).symbol
        reference_symbol = self.add_future(Futures.Indices.SP_500_E_MINI).symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Examples

The following examples demonstrate some common practices for trading Futures with historical data.

Example 1: Stock-Gold All-Weather Portfolio

The following algorithm constructs a monthly-rebalanced, risk-parity portfolio using S&P 500 E-Mini and Micro Gold Futures contracts. A risk-parity approach balances the contribution to risk from each asset class, helping reduce overall portfolio drawdowns and improve diversification. The position sizing is determined using 1-year of historical daily returns for the selected Futures universe.

PY

```
from Portfolio.RiskParityPortfolioOptimizer import RiskParityPortfolioOptimizer

class FuturesHistoricalDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)

        # Select the future contracts representing various asset classes to diversify the portfolio.
        self._futures = [self.add_future(
                x,
                fill_forward=True,
                extended_market_hours=True,
                data_normalization_mode=DataNormalizationMode.BACKWARDS_RATIO,
                data_mapping_mode=DataMappingMode.OPEN_INTEREST,
                contract_depth_offset=0
            ) for x in [Futures.Indices.SP_500_E_MINI, Futures.Metals.MICRO_GOLD]]

        # Instantiate the optimizer to perform risk-parity optimization.
        # Risk Parity portfolio can dissipate the risk of each crypto pair to reduce overall drawdown.
        self._optimizer = RiskParityPortfolioOptimizer(0.0, 1.0)

        # Set a scheduled event to rebalance the portfolio at the start of every month.
        self.schedule.on(
            self.date_rules.month_start(),
            self.time_rules.after_market_open(self._futures[0].symbol, 1),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Historical data request to get 1-year data for optimization.
        history = self.history([x.symbol for x in self._futures], 253, Resolution.DAILY, extended_market_hours=True, fill_forward=True)\
            .droplevel([0]).unstack(0).close.dropna()
        # Daily return on the universe members to calculate the optimized weights.
        returns = history.pct_change().dropna()

        # Calculate the optimized weights.
        weights = self._optimizer.optimize(returns)

        # Rebalance the portfolio according to the optimized weights.
        targets = [PortfolioTarget(future.mapped, size/2.0) for future, size in zip(self._futures, weights)]
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Asset Classes

Future Options

Introduction

This page explains how to get historical data for Future Options. Some of the data you can get include prices and indicator data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```
class FutureOptionsTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add a FOP universe.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        self.add_future_option(future.symbol, lambda universe: universe.front_month().strikes(-1, 0).calls_only())

    # Get trailing data whenever a new FOP contract enters the universe.
    def on_securities_changed(self, changes):
        for security in changes.added_securities:
            if security.type == SecurityType.FUTURE_OPTION:
                # Get the 3 trailing daily TradeBar objects of the security in DataFrame format.
                history = self.history(TradeBar, security.symbol, 3, Resolution.DAILY)
```

					close	high	low	open	volume
expiry	**strike**	**type**	**symbol**	**time**					
2024-12-20	**5870.0**	**1**	**ES 32NKVT5YYX1US|ES YOGVNNAOI1OH**	**2024-12-16 17:00:00**	2.15	2.15	1.45	1.80	227.0
				2024-12-17 17:00:00	2.35	3.05	2.30	2.30	14.0
				2024-12-18 17:00:00	38.75	43.75	1.75	1.85	399.0

```
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
expiry      strike  type  symbol                           time
2024-12-20  5870.0  1     ES 32NKVT5YYX1US|ES YOGVNNAOI1OH  2024-12-17 17:00:00     0.093023
                                                           2024-12-18 17:00:00    15.489362
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```py
# Get the 3 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 3, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```py
class FutureOptionsQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add a FOP universe.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        self.add_future_option(future.symbol, lambda universe: universe.front_month().strikes(-1, 0).calls_only())

    # Get trailing data whenever a new FOP contract enters the universe.
    def on_securities_changed(self, changes):
        for security in changes.added_securities:
            if security.type == SecurityType.FUTURE_OPTION:
                # Get the 3 trailing daily QuoteBar objects of the security in DataFrame format.
                history = self.history(QuoteBar, security.symbol, 3, Resolution.DAILY)
```

					askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
expiry	strike	type	symbol	time														
2024-12-20	5870.0	1	ES 32NKVT5YYX1US|ES YOGVNNAOI1OH	2024-12-16 17:00:00	2.5	2.55	1.45	1.85	2.0	2.35	2.40	1.35	1.75	46.0	2.425	2.475	1.4	1.800
				2024-12-17 17:00:00	2.7	3.15	2.15	3.05	179.0	2.45	3.05	2.05	2.90	46.0	2.575	3.100	2.1	2.975
				2024-12-18 17:00:00	40.0	215.50	1.55	2.30	3.0	36.50	60.50	0.05	2.15	2.0	38.250	138.00	0.8	2.225

```py
# Calculate the spread.
spread = history.askclose - history.bidclose
```

```
expiry      strike  type  symbol                          time
2024-12-20  5870.0  1     ES 32NKVT5YYX1US|ES YOGVNNAOI1OH  2024-12-16 17:00:00    0.15
                                                           2024-12-17 17:00:00    0.25
                                                           2024-12-18 17:00:00    3.50
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```py
# Get the 3 trailing daily QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 3, Resolution.DAILY)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```py
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        self.add_future_option(future.symbol, lambda universe: universe.front_month().strikes(-1, 0).calls_only())
        # Add a Scheduled Event that runs at the start of each month.
        self.schedule.on(self.date_rules.month_start(future.symbol), self.time_rules.after_market_open(future.symbol, 60), self._trade)

    def _trade(self):
        # Get the historical Slice objects over the last 30 minutes for all the subcriptions in your algorithm.
        for slice_ in self.history(30, Resolution.MINUTE):
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```py
class FutureOptionIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add a FOP universe.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        self.add_future_option(future.symbol, lambda universe: universe.front_month().strikes(0, 0).calls_only())

    def on_securities_changed(self, changes):
        for security in changes.added_securities:
            if security.type != SecurityType.FUTURE_OPTION:
                continue
            # Get the 21-day SMA values of the contract for the last 5 trading days.
            history = self.indicator_history(SimpleMovingAverage(21), security.symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```py
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 17:00:00	131.404762	2759.500
2024-12-13 17:00:00	132.101190	2774.125
2024-12-16 17:00:00	135.779762	2851.375
2024-12-17 17:00:00	138.047619	2899.000
2024-12-18 17:00:00	134.095238	2816.000

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the contract's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of multiple securities to compute their value (for example, the indicators for the Greeks and implied volatility). In this case, pass a list of the `Symbol` objects to the method.

```
class FutureOptionMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add a FOP universe.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        self.add_future_option(future.symbol, lambda universe: universe.front_month().strikes(0, 0).calls_only())

    def on_securities_changed(self, changes):
        for security in changes.added_securities:
            if security.type != SecurityType.FUTURE_OPTION:
                continue
            option = security.symbol
            # Get the Symbol of the mirror contract.
            mirror = Symbol.create_option(
                option.underlying, option.id.market, option.id.option_style,
                OptionRight.Call if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
                option.id.strike_price, option.id.date
            )
            # Create the indicator.
            indicator = ImpliedVolatility(
                option, self.risk_free_interest_rate_model, ConstantDividendYieldModel(0),
                mirror, OptionPricingModelType.FORWARD_TREE
            )
            # Get the historical values of the indicator over the last 10 minutes.
            history = self.indicator_history(indicator, [option.underlying, option, mirror], 10, Resolution.MINUTE)
            # Get the average IV value.
            iv_avg = history.data_frame.current.mean()
```

Examples

The following examples demonstrate some common practices for trading Future Options with historical data.

Example 1: Trend Following on Front Month ATM Option Contract

This algorithm strategically trades the front month S&P 500 EMini Future Options by analyzing bid and ask volumes shortly after the market opens. Using scheduled events, it effectively executes trades based on historical quote data, optimizing decision-making. The algorithm aims for timely entries and exits, ensuring efficient capital management and quick adaptability in volatile markets.

PY

```
class FutureOptionsTradingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)  # Limit to a single day for 0DTE
        self.set_cash(200000)  # Starting cash

        # Request ES option data for trading and signal generation.
        future = self.add_future(Futures.Indices.SP_500_E_MINI)
        future.set_filter(lambda universe: universe.front_month())
        # We are interested in ATM front-month options since they are the most popular.
        self.add_future_option(future.symbol, lambda u: u.expiration(0, 90).strikes(-3, 3))

        # Schedule event to enter and exit option contract position.
        self.schedule.on(self.date_rules.every_day(future.symbol), self.time_rules.after_market_open(future.symbol, 16),
self.trade_option)
        self.schedule.on(self.date_rules.every_day(future.symbol), self.time_rules.before_market_close(future.symbol, 15),
self.liquidate)

    def trade_option(self) -> None:
        # Get the option chain to trade.
        for option_chain in self.current_slice.option_chains.values():
            for option in option_chain:
                # Request historical quote data for signal generation.
                history = self.history(QuoteBar, option.symbol, 15, Resolution.Minute)
                if not history.empty:
                    # Calculate the total bid and ask for dollar volume to determine the capital directional force.
                    total_bid_volume = 0
                    total_ask_volume = 0
                    if 'bidclose' in history:
                        total_bid_volume += (history['bidclose'] * history['bidsize']).sum()
                    if 'askclose' in history:
                        total_ask_volume += (history['askclose'] * history['asksize']).sum()

                    # Follow the capital flow to trade.
                    if total_bid_volume > total_ask_volume:
                        self.market_order(option.symbol, 1)
                    else:
                        self.market_order(option.symbol, -1)
```

Asset Classes

Index

Introduction

This page explains how to get historical price and indicator data for Indices.

Bars

To get historical price data , call the `history` method with an Index `Symbol` . This method returns a DataFrame with columns for the open, high, low, and close.

```py
class IndexPriceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of an Index.
        symbol = self.add_index('SPX').symbol
        # Get the 5 trailing daily bars of the Index in DataFrame format.
        history = self.history(symbol, 5, Resolution.DAILY)
```

		close	high	low	open
symbol	**time**				
SPX	**2024-12-12 15:15:00**	6051.70	6079.68	6051.70	6074.05
	2024-12-13 15:15:00	6050.83	6078.58	6035.77	6068.48
	2024-12-16 15:15:00	6074.48	6085.19	6059.14	6064.04
	2024-12-17 15:15:00	6050.31	6057.68	6035.19	6046.33
	2024-12-18 15:15:00	5869.22	6070.67	5867.79	6048.69

```py
# Calculate the daily growth.
daily_growth = history.close.pct_change().iloc[1:]
```

```
symbol  time
SPX     2024-12-13 15:15:00   -0.000144
        2024-12-16 15:15:00    0.003909
        2024-12-17 15:15:00   -0.003979
        2024-12-18 15:15:00   -0.029931
Name: close, dtype: float64
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```py
# Get the 5 trailing daily bars of the Index in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their prices.
for trade_bar in history:
    price = trade_bar.price
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_index('SPX')
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class IndexIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_index('SPX').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 15:15:00	6003.055714	126064.17
2024-12-13 15:15:00	6006.096667	126128.03
2024-12-16 15:15:00	6012.036190	126252.76
2024-12-17 15:15:00	6020.634286	126433.32
2024-12-18 15:15:00	6019.442381	126408.29

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's high price.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.HIGH)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class IndexMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_index('NDX').symbol
        reference_symbol = self.add_index('SPX').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Examples

The following examples demonstrate some common practices for trading with Index historical data.

Example 1: Replicating Portfolio

The example below constructed a replicating SGX portfolio using the top 100 weighted SPY constituents. It provides an alternative way for retail investors to invest in SGX without access to certain financial instruments and a hedging idea on SGX close hours.

PY

```
from sklearn.linear_model import LinearRegression

class SGXMatchingTradingAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.set_warm_up(timedelta(7))

        # Request SGX Index data for matching.
        self._sgx = self.add_index("SGX").symbol
        self._spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        # We will try to use SPY constituents to reproduce the SGX index.
        self._universe = self.add_universe(
            self.universe.etf(
                self._spy,
                universe_filter_func=lambda x: [
                    y.symbol for y in sorted(
                        [y for y in x if y.weight],
                        key=lambda x: x.weight, reverse=True
                    )[:100]
                ]
            )
        )

        # Schedule an event to rebalance the portfolio.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.after_market_open(self._spy, 1),
            self.rebalance
        )

    def rebalance(self) -> None:
        if self.is_warming_up or not self._universe.selected:
            return
        symbols = list(self._universe.selected)
        # Obtain the historical data.
        history = self.history([self._sgx] + symbols, 252, Resolution.DAILY).unstack(0).close
        history = history.groupby(history.index.date).last().dropna()
        # Obtain the weights of SPY constituents that best reproduce SGX.
        weights = self.calculate_weights(history[symbols], history[self._sgx])
        self.set_holdings(
            [PortfolioTarget(symbol, weight) for symbol, weight in zip(symbols, weights)],
            liquidate_existing_holdings=True
        )

    def calculate_weights(self, history: pd.DataFrame, benchmark: pd.Series) -> pd.Series:
        # Use log price to eliminate the compounding effect.
        # Perform regression to obtain the best-replicating portfolio of SGX.
        lr = LinearRegression(fit_intercept=False).fit(np.log(history), np.log(benchmark))
        sum_ = sum(abs(lr.coef_))
        return [c/sum_ for c in lr.coef_]
```

Asset Classes

Index Options

Introduction

This page explains how to get historical data for Index Options. Some of the data you can get include prices, open interest, Greeks, and universe selection data.

Trades

To get historical trade data , call the `history` method with the `TradeBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and volume.

```
class IndexOptionTradeBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        index = self.add_index('SPX')
        symbol = sorted(self.option_chain(index.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing daily TradeBar objects of the security in DataFrame format.
        history = self.history(TradeBar, symbol, 5, Resolution.DAILY)
```

					close	high	low	open	volume
expiry	**strike**	**type**	**symbol**	**time**					
2024-12-20	**4000.0**	**1**	**SPX 32NKZCP89T8EM|SPX 31**	**2024-12-13 15:15:00**	0.05	0.07	0.05	0.07	5773.0
				2024-12-13 15:15:00	0.05	0.10	0.05	0.05	3052.0
				2024-12-16 15:15:00	0.03	0.10	0.03	0.10	6423.0
				2024-12-17 15:15:00	0.03	0.05	0.03	0.05	6085.0
				2024-12-18 15:15:00	0.85	1.00	0.03	0.03	10551.0

```
# Calculate the daily returns.
daily_returns = history.close.pct_change().iloc[1:]
```

```
expiry      strike  type  symbol                   time
2024-12-20  4000.0  1     SPX 32NKZCP89T8EM|SPX 31  2024-12-13 15:15:00     0.000000
                                                    2024-12-16 15:15:00    -0.400000
                                                    2024-12-17 15:15:00     0.000000
                                                    2024-12-18 15:15:00    27.333333
```

If you intend to use the data in the DataFrame to create `TradeBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `TradeBar` objects instead of a DataFrame, call the `history[TradeBar]` method.

```
# Get the 5 trailing daily TradeBar objects of the security in TradeBar format.
history = self.history[TradeBar](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for trade_bar in history:
    t = trade_bar.end_time
    volume = trade_bar.volume
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```
class IndexOptionQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        index = self.add_index('SPX')
        symbol = sorted(self.option_chain(index.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

					askclose	askhigh	asklow	askopen	asksize	bidclose	bidhigh	bidlow	bidopen	bidsize	close	high	low	open
expiry	strike	type	symbol	time														
2024-12-20	4000.0	1	SPX 32NKZCP89T8EM|SPX 31	2024-12-18 15:11:00	1.45	9.8	1.15	1.15	11.0	0.90	0.90	0.3	0.90	87.0	1.175	5.350	0.725	1.025
				2024-12-18 15:12:00	1.40	9.6	1.40	1.45	10.0	0.85	0.95	0.7	0.90	63.0	1.125	5.275	1.050	1.175
				2024-12-18 15:13:00	1.40	4.9	1.40	1.40	122.0	0.80	0.90	0.3	0.85	104.0	1.100	2.900	0.850	1.125
				2024-12-18 15:14:00	1.25	4.9	1.25	1.40	59.0	0.60	0.80	0.3	0.80	95.0	0.925	2.850	0.775	1.100
				2024-12-18 15:15:00	1.05	4.9	1.00	1.25	48.0	0.60	0.85	0.3	0.60	10.0	0.825	2.875	0.650	0.925

```
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
expiry      strike  type  symbol                   time
2024-12-20  4000.0  1     SPX 32NKZCP89T8EM|SPX 31  2024-12-18 15:11:00    0.55
                                                    2024-12-18 15:12:00    0.55
                                                    2024-12-18 15:13:00    0.60
                                                    2024-12-18 15:14:00    0.65
                                                    2024-12-18 15:15:00    0.45
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise,

LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the dollar volume on the bid.
for quote_bar in history:
    t = quote_bar.end_time
    bid_dollar_volume = quote_bar.last_bid_size * quote_bar.bid.close
```

Request minute, hour, or daily resolution data. Otherwise, the history request won't return any data.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        index = self.add_index('SPX')
        contract = sorted(self.option_chain(index.symbol), key=lambda c: c.open_interest)[-1]
        self.add_option_contract(contract.symbol)
        # Get the historical Slice objects over the last 5 days for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.DAILY)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each TradeBar in this Slice.
            for symbol, trade_bar in slice_.bars.items():
                close = trade_bar.close
```

Open Interest

To get historical open interest data , call the `history` method with the `OpenInterest` type and a security's `Symbol` . This method returns a DataFrame with a single column.

```
class IndexOptionOpenInterestHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        index = self.add_index('SPX')
        symbol = sorted(self.option_chain(index.symbol), key=lambda c: c.open_interest)[-1].symbol
        # Get the 5 trailing daily OpenInterest objects of the security in DataFrame format.
        history = self.history(OpenInterest, symbol, 5, Resolution.DAILY)
```

					openinterest
expiry	**strike**	**type**	**symbol**	**time**	
2024-12-20	**4000.0**	**1**	**SPX 32NKZCP89T8EM|SPX 31**	**2024-12-12 23:00:00**	306249.0
				2024-12-15 23:00:00	305821.0
				2024-12-16 23:00:00	301048.0
				2024-12-17 23:00:00	299501.0
				2024-12-18 23:00:00	294504.0

```
# Calculate the daily change in open interest.
oi_delta = history.openinterest.diff().iloc[1:]
```

```
expiry      strike  type  symbol                    time
2024-12-20  4000.0  1     SPX 32NKZCP89T8EM|SPX 31  2024-12-15 23:00:00    -428.0
                                                    2024-12-16 23:00:00   -4773.0
                                                    2024-12-17 23:00:00   -1547.0
                                                    2024-12-18 23:00:00   -4997.0
Name: openinterest, dtype: float64
```

If you intend to use the data in the DataFrame to create `OpenInterest` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `OpenInterest` objects instead of a DataFrame, call the `history[OpenInterest]` method.

```
# Get the 5 trailing daily OpenInterest objects of the security in OpenInterest format.
history = self.history[OpenInterest](symbol, 5, Resolution.DAILY)
# Iterate through the TradeBar objects and access their volumes.
for oi in history:
    t = oi.end_time
    open_interest = oi.value
```

Daily Option Chains

To get historical daily Option chain data , call the `history` method with the Option `Symbol` object. The data this method returns contains information on all the currently tradable contracts, not just the contracts that pass your filter. If you pass `flatten=True` , this method returns a DataFrame with columns for the data point attributes.

```
class IndexOptionDailyOptionChainHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add an Index Option universe.
        option = self.add_index_option('SPX')
        # Get the trailing 5 daily Option chains in DataFrame format.
        history = self.history(option.symbol, 5, flatten=True)
```

		close	delta	gamma	high	implied_volatility	low	open	openinterest	rho	theta	underlying_value	vega	volume
time	symbol													
2024-12-12	SPX YOGZ79IHUCOE|SPX 31		0.000000	0.000000	5897.45	0.000000	5865.00	5866.45	5280.0	0.051183	-0.030094	SPX: □6,084.20	5879.60	0.000000	0.0
	SPX YP8JPVHGP7Q9A|SPX 31		0.000000	0.000000	5898.45	0.000000	5864.30	5866.55	68.0	0.203532	-0.029968	SPX: □6,084.20	5878.95	0.000000	0.0
	SPX YQ70D5AGE4VTI|SPX 31		0.000000	0.000000	5892.70	0.000000	5862.20	5863.70	42.0	0.392162	-0.029810	SPX: □6,084.20	5876.70	0.000000	0.0
	SPX YQYKVRJ3JJ9DA|SPX 31		0.000000	0.000000	5889.75	0.000000	5856.85	5860.50	35.0	0.541633	-0.029685	SPX: □6,084.20	5873.70	0.000000	0.0
	SPX YRP5YAEJNL7MA|SPX 31		0.999970	0.000006	5888.45	1.651861	5857.20	5859.35	1.0	0.680416	-0.032476	SPX: □6,084.20	5873.15	0.004516	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-12-18	SPX 32RLQ2G4BQHCE|SPX 31		0.999435	0.000002	4751.20	0.268745	4727.55	4738.45	0.0	-44.100768	1.613144	SPX: □6,050.31	4740.75	0.077120	0.0
	SPX 32XJE2QF4AL7S|SPX 31		-0.675418	0.000079	5466.35	0.753369	5424.20	5445.05	28.0	-101.357524	-0.721391	SPX: □6,050.31	5445.85	21.834520	0.0
	SPX 337HSSRKHSSN2|SPX 31		-0.997608	0.000006	5012.40	0.139114	4990.20	4999.30	3.0	-214.914871	1.611553	SPX: □6,050.31	5001.30	0.638336	0.0
	SPX 33HG7ISPWZS26|SPX 31		-0.992253	0.000017	4595.60	0.118943	4574.20	4582.00	10.0	-303.860365	1.514502	SPX: □6,050.31	4583.80	2.232716	0.0
	SPX 33REM8T4V9UEA|SPX 31		-0.975511	0.000043	4205.90	0.111797	4185.60	4191.90	8.0	-379.486892	1.404206	SPX: □6,050.31	4193.20	6.947694	0.0

PY

```
# Select the 2 contracts with the greatest volume each day.
most_traded = history.groupby('time').apply(lambda x: x.nlargest(2, 'volume')).reset_index(level=1, drop=True).volume
```

```
time        symbol
2024-12-12  SPX YOGZ79IHUCOE|SPX 31     20200.0
            SPX 32NKZCPHP4626|SPX 31    18417.0
2024-12-13  SPX YOGZ79IHUCOE|SPX 31      7109.0
            SPX 32OCJVBQ3CMGE|SPX 31     5930.0
2024-12-14  SPX YOGZ79IHUCOE|SPX 31     17443.0
            SPX 32NKZCRZNW3RI|SPX 31    16458.0
2024-12-17  SPX YOGZ7C245MVI|SPX 31     32796.0
            SPX 32NKZCS1BFG9A|SPX 31    30285.0
2024-12-18  SPX YOGZ79IHUCOE|SPX 31     14183.0
            SPX YOGZ7C245MVI|SPX 31     13335.0
Name: volume, dtype: float64
```

To get the data in the format of `OptionUniverse` objects instead of a DataFrame, call the `history[OptionUniverse]` method.

```
# Get the historical OptionUniverse data over the last 30 days.
history = self.history[OptionUniverse](option.symbol, timedelta(30))
# Iterate through each daily Option chain.
for option_universe in history:
    t = option_universe.end_time
    # Select the contract with the most volume.
    most_traded = sorted(option_universe, key=lambda contract: contract.volume)[-1]
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
class IndexOptionIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of the Option contract.
        underlying = self.add_index('SPX').symbol
        symbol = sorted(self.option_chain(underlying), key=lambda c: c.open_interest)[-1].symbol
        # Get the 21-day SMA values of the contract for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-12 15:15:00	0.617857	12.975
2024-12-13 15:15:00	0.580952	12.200
2024-12-16 15:15:00	0.535714	11.250
2024-12-17 15:15:00	0.477381	10.025
2024-12-18 15:15:00	0.472619	9.925

```
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's volume.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of multiple securities to compute their value (for example, the indicators for the Greeks and implied volatility). In this case, pass a list of the `Symbol` objects to the method.

```py
class IndexOptionMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of the underlying asset.
        underlying = self.add_index('SPX').symbol
        # Get the Option contract Symbol.
        option = sorted(self.option_chain(underlying), key=lambda c: c.open_interest)[-1].symbol
        # Get the Symbol of the mirror contract.
        mirror = Symbol.create_option(
            underlying, option.id.market, option.id.option_style,
            OptionRight.CALL if option.id.option_right == OptionRight.PUT else OptionRight.PUT,
            option.id.strike_price, option.id.date
        )
        # Create the indicator.
        indicator = ImpliedVolatility(
            option, self.risk_free_interest_rate_model, DividendYieldProvider(underlying),
            mirror, OptionPricingModelType.FORWARD_TREE
        )
        # Get the historical values of the indicator over the last 60 trading minutes.
        history = self.indicator_history(indicator, [underlying, option, mirror], 60, Resolution.MINUTE)
        # Get the average IV value.
        iv_avg = history.data_frame.current.mean()
```

Examples

The following examples demonstrate some common practices for trading Index Options with historical data.

Example 1: Standard-Weekly Contracts Cointegration

The following example analyzes the cointegration relationship between the front-month ATM SPX and SPXW calls. By measuring their spread divergence, we trade mean reversal on their spread convergence.

```py
from sklearn.linear_model import LinearRegression

class IndexOptionHistoricalDataAlgorithm(QCAlgorithm):
    _threshold = 2
    _cointegration_vector = [0, 0]
    _spread_mean = 0
    _spread_sd = 1
    _spx_contract = None
    _spxw_contract = None

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)
        self._index = Symbol.create("SPX", SecurityType.INDEX, Market.USA)
        self._spx = Symbol.create_canonical_option(self._index)
        self._spxw = Symbol.create_canonical_option(self._index, "SPXW", Market.USA, "?SPXW")

        # Select the Index Options to analyze and trade by week.
        self.schedule.on(self.date_rules.week_start(), self.time_rules.at(9, 15), self.select_contract)

    def on_data(self, slice: Slice) -> None:
        if not (self._spx_contract and self._spxw_contract):
            return
        spxw = slice.quote_bars.get(self._spxw_contract)
        spx = slice.quote_bars.get(self._spx_contract)
        if not (spxw and spx):
            return
        # Obtain the current spread to see if there is any price divergence to trade.
        spread = self.calculate_spread(spxw.close, spx.close, self._cointegration_vector)
        z = (spread - self._spread_mean) / self._spread_sd
        # If the spread diverges above or below the threshold, trade to bet on mean reversion.
        if z > self._threshold and not self.portfolio[self._spx_contract].is_long:
            spx_qty = int(10 * self._cointegration_vector[1])
            spxw_qty = -10
            self.market_order(self._spx_contract, spx_qty)
            self.market_order(self._spxw_contract, spxw_qty)

        elif z < -self._threshold and not self.portfolio[self._spxw_contract].is_long:
            spx_qty = int(-10 * self._cointegration_vector[1])
            spxw_qty = 10
            self.market_order(self._spx_contract, spx_qty)
            self.market_order(self._spxw_contract, spxw_qty)

        # If prices converge, exit positions.
        if ((self.portfolio[self._spx_contract].is_long and z < 0) or
            (self.portfolio[self._spxw_contract].is_long and z > 0)):
            self.liquidate()

    def select_contract(self) -> None:
        self._spx_contract = None
```

```
        self._spxw_contract = None

        # Obtain the SPX ATM call contract since it is the most liquid to trade with.
        spx_contracts = [x for x in self.option_chain(self._spx) if x.expiry <= self.time + timedelta(30)]
        if not spx_contracts:
            return

        # Select the furthest expiry within the next 30 days
        expiry = max(x.expiry for x in spx_contracts)
        self._spx_contract = sorted(
            [x for x in spx_contracts
             if x.expiry == expiry and x.right == OptionRight.CALL],
            key=lambda x: abs(x.strike - x.underlying_last_price)
        )[0].symbol

        # Obtain the SPXW contract with the same strike, right, and expiry.
        spxw_contract = next(
            (
                x for x in self.option_chain(self._spxw)
                if (x.expiry == expiry and
                    x.right == OptionRight.CALL and
                    x.strike == self._spx_contract.id.strike_price)
            ),
            None
        )
        if not spxw_contract:
            return
        self._spxw_contract = spxw_contract.symbol

        # Subscribe to the contracts we will trade
        self.add_index_option_contract(self._spx_contract)
        self.add_index_option_contract(self._spxw_contract)

        # Obtain the historical data and find their cointegration relationship.
        history = self.history(QuoteBar, [self._spx_contract, self._spxw_contract], 1000).close.droplevel([0, 1, 2]).unstack(0)

        # Run linear regression to determine cointegration relationship
        self._cointegration_vector = self.calculate_cointegration_vector(history)

        residual = self.calculate_spread(history[self._spxw_contract], history[self._spx_contract], self._cointegration_vector)

        self._spread_mean = np.mean(residual)
        self._spread_sd = np.std(residual)

    def calculate_cointegration_vector(self, history):
        # Prepare log price series
        y = np.log(history[self._spxw_contract])
        x1 = np.log(history[self._spx_contract])

        # Linear regression for cointegration
        lr = LinearRegression().fit(x1.to_frame(), y)

        return [lr.intercept_, lr.coef_[0]]

    def calculate_spread(self, y, x1, cointegration_vector):
        return np.log(y) - (cointegration_vector[0] + np.log(x1) * cointegration_vector[1])
```

Asset Classes

CFD

Introduction

This page explains how to get historical quotes and indicator data for CFDs.

Quotes

To get historical quote data , call the `history` method with the `QuoteBar` type and a security's `Symbol` . This method returns a DataFrame with columns for the open, high, low, close, and size of the bid and ask quotes. The columns that don't start with "bid" or "ask" are the mean of the quote prices on both sides of the market.

```py
class CFDQuoteBarHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_cfd('XAUUSD').symbol
        # Get the 5 trailing minute QuoteBar objects of the security in DataFrame format.
        history = self.history(QuoteBar, symbol, 5, Resolution.MINUTE)
```

		askclose	askhigh	asklow	askopen	bidclose	bidhigh	bidlow	bidopen	close	high	low	open
symbol	**time**												
XAUUSD	**2024-12-18 23:56:00**	2607.96	2608.53	2607.91	2608.53	2607.49	2607.99	2607.43	2607.99	2607.725	2608.26	2607.670	2608.260
	2024-12-18 23:57:00	2608.31	2608.36	2607.92	2607.96	2607.74	2607.80	2607.45	2607.49	2608.025	2608.08	2607.685	2607.725
	2024-12-18 23:58:00	2608.47	2608.55	2608.28	2608.31	2607.89	2608.01	2607.71	2607.74	2608.180	2608.28	2607.995	2608.025
	2024-12-18 23:59:00	2609.48	2609.70	2608.43	2608.47	2609.05	2609.12	2607.89	2607.89	2609.265	2609.41	2608.160	2608.180
	2024-12-19 00:00:00	2609.48	2610.17	2609.48	2609.48	2609.10	2609.65	2609.03	2609.05	2609.290	2609.91	2609.255	2609.265

```py
# Calculate the spread at each minute.
spread = history.askclose - history.bidclose
```

```
symbol  time
XAUUSD  2024-12-18 23:56:00    0.47
        2024-12-18 23:57:00    0.57
        2024-12-18 23:58:00    0.58
        2024-12-18 23:59:00    0.43
        2024-12-19 00:00:00    0.38
dtype: float64
```

If you intend to use the data in the DataFrame to create `QuoteBar` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `QuoteBar` objects instead of a DataFrame, call the `history[QuoteBar]` method.

```
# Get the 5 trailing minute QuoteBar objects of the security in QuoteBar format.
history = self.history[QuoteBar](symbol, 5, Resolution.MINUTE)
# Iterate through each QuoteBar and calculate the spread.
for quote_bar in history:
    t = quote_bar.end_time
    spread = quote_bar.ask.close - quote_bar.bid.close
```

Ticks

To get historical tick data , call the `history` method with a security's `Symbol` and `Resolution.TICK` . This method returns a DataFrame that contains data on bids, asks, and last trade prices.

```
class CFDTickHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_cfd('XAUUSD').symbol
        # Get the trailing 2 days of ticks for the security in DataFrame format.
        history = self.history(symbol, timedelta(2), Resolution.TICK)
```

		askprice	bidprice	lastprice
symbol	**time**			
XAUUSD	**2024-12-17 00:00:00.047969**	2652.07	2651.61	2651.840
	2024-12-17 00:00:00.716746	2652.03	2651.53	2651.780
	2024-12-17 00:00:00.742011	2652.02	2651.49	2651.755
	2024-12-17 00:00:00.770819	2652.02	2651.47	2651.745
	2024-12-17 00:00:00.781622	2652.02	2651.49	2651.755

```
# Calculate the spread.
spread = history.askprice - history.bidprice
```

```
symbol  time
XAUUSD  2024-12-17 00:00:00.047969    0.46
        2024-12-17 00:00:00.716746    0.50
        2024-12-17 00:00:00.742011    0.53
        2024-12-17 00:00:00.770819    0.55
        2024-12-17 00:00:00.781622    0.53
dtype: float64
```

If you intend to use the data in the DataFrame to create `Tick` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of `Tick` objects instead of a DataFrame, call the `history[Tick]` method.

```
# Get the trailing 2 days of ticks for the security in Tick format.
history = self.history[Tick](symbol, timedelta(2), Resolution.TICK)
# Iterate through each quote tick and calculate the spread.
for tick in history:
    t = tick.end_time
    spread = tick.bid_price - tick.ask_price
```

Ticks are a sparse dataset, so request ticks over a trailing period of time or between start and end times.

Slices

To get historical Slice data, call the `history` method without passing any `Symbol` objects. This method returns `Slice` objects, which contain data points from all the datasets in your algorithm. If you omit the `resolution` argument, it uses the resolution that you set for each security and dataset when you created the subscriptions.

```
PY
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 1)
        self.set_end_date(2024, 12, 31)
        # Add some securities and datasets.
        self.add_cfd('XAUUSD')
        # Get the historical Slice objects over the last 5 minutes for all the subcriptions in your algorithm.
        history = self.history(5, Resolution.MINUTE)
        # Iterate through each historical Slice.
        for slice_ in history:
            # Iterate through each QuoteBar in this Slice.
            for symbol, quote_bar in slice_.bars.items():
                midprice = quote_bar.close
```

Indicators

To get historical indicator values, call the `indicator_history` method with an indicator and the security's `Symbol` .

```
PY
class CFDIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a security.
        symbol = self.add_cfd('SPX500USD').symbol
        # Get the 21-day SMA values of the security for the last 5 trading days.
        history = self.indicator_history(SimpleMovingAverage(21), symbol, 5, Resolution.DAILY)
```

To organize the data into a DataFrame, use the `data_frame` property of the result.

```
PY
# Organize the historical indicator data into a DataFrame to enable pandas wrangling.
history_df = history.data_frame
```

	current	rollingsum
2024-12-13 18:00:00	6033.309524	126699.5
2024-12-15 18:00:00	6039.390476	126827.2
2024-12-16 18:00:00	6045.090476	126946.9
2024-12-17 18:00:00	6049.180952	127032.8
2024-12-18 18:00:00	6044.276190	126929.8

```
PY
# Get the maximum of the SMA values.
sma_max = history_df.current.max()
```

The `indicator_history` method resets your indicator, makes a history request, and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
# Get the historical values of an indicator over the last 30 days, applying the indicator to the security's ask price.
history = self.indicator_history(indicator, symbol, timedelta(30), selector=Field.ASK_CLOSE)
```

Some indicators require the prices of two securities to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
class CFDMultiAssetIndicatorHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 19)
        self.set_end_date(2024, 12, 31)
        # Add the target and reference securities.
        target_symbol = self.add_cfd('XAUUSD').symbol
        reference_symbol = self.add_cfd('SPX500USD').symbol
        # Create a 21-period Beta indicator.
        beta = Beta("", target_symbol, reference_symbol, 21)
        # Get the historical values of the indicator over the last 10 trading days.
        history = self.indicator_history(beta, [target_symbol, reference_symbol], 10, Resolution.DAILY)
        # Get the average Beta value.
        beta_avg = history.data_frame.mean()
```

Examples

The following examples demonstrate some common practices for trading CFD with historical data.

Example 1: Demand Force Following

The following algorithm measures the historical bid-ask spread of XAUUSD CFD contract. If the current spread is higher than the historical spread, meaning the demand force is lower than usual, we sell the contract; else, we buy. To realize the directional speculation, we apply a 2% take profit risk management model and a 1% trailing stop loss risk management model.

```
class CFDHistoricalDataAlgorithm(QCAlgorithm):
    _threshold = 2
    _spread_mean = 0
    _spread_sd = 1

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Request XAUUSD CFD contract to trade.
        self._xauusd = self.add_cfd("XAUUSD").symbol

        # Update the spread mean and SD to calculate the Z-score accurately daily.
        self.schedule.on(
            self.date_rules.every_day(self._xauusd),
            self.time_rules.at(0, 0),
            self.update_spread
        )

        # Take profit at 2%.
        self.add_risk_management(MaximumUnrealizedProfitPercentPerSecurity(0.02))
        # Trailing stop loss at 1%.
        self.add_risk_management(TrailingStopRiskManagementModel(0.01))

    def update_spread(self) -> None:
        # Obtain historical quote data to obtain the mean and SD of the bid-ask spread.
        history = self.history(QuoteBar, self._xauusd, 390, Resolution.MINUTE).unstack(0)        # minute bar of 1 day
        if "askclose" in history and "bidclose" in history:
            spread = history.askclose - history.bidclose
            self._spread_mean = np.mean(spread.values)
            self._spread_sd = np.std(spread.values, ddof=1)

    def on_data(self, slice: Slice) -> None:
        quote = slice.quote_bars.get(self._xauusd)
        # Obtain the current bid-ask spread as a trade signal.
        if quote and quote.ask and quote.bid:
            spread = quote.ask.close - quote.bid.close
            z = (spread - self._spread_mean) / self._spread_sd

            # If the spread is lower than expected, expect demand to be higher than usual, and buy XAUUSD.
            if z <= -self._threshold:
                self.set_holdings(self._xauusd, 1)
            # If the spread is higher than expected, expect demand to be lower than usual and sell XAUUSD.
            elif z >= self._threshold:
                self.set_holdings(self._xauusd, -1)
```

Historical Data

Alternative Data

Introduction

Alternative datasets provide signals to inform trading decisions. To view all the alternative datasets available on QuantConnect, see the Dataset Market . This page explains how to get historical data for alternative datasets.

Data Points

To get historical alternative data points, call the `history` method with the dataset `Symbol` . This method returns a DataFrame that contains the data point attributes.

```py
class AlternativeDataHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 20)
        self.set_end_date(2024, 12, 31)
        # Get the Symbol of a dataset.
        dataset_symbol = self.add_data(Fred, 'RVXCLS').symbol
        # Get the trailing 5 days of Fred data in DataFrame format.
        history = self.history(dataset_symbol, 5, Resolution.DAILY)
```

		value
symbol	**time**	
RVXCLS.Fred	**2024-12-17**	23.02
	2024-12-18	24.01
	2024-12-19	32.76
	2024-12-20	29.90

```py
# Calculate the dataset's rate of change.
roc = history.pct_change().iloc[1:]
```

		value
symbol	**time**	
RVXCLS.Fred	**2024-12-18**	0.043006
	2024-12-19	0.364431
	2024-12-20	-0.087302

If you intend to use the data in the DataFrame to create `alternativeDataClass` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of dataset objects instead of a DataFrame, call the `history[ alternativeDataClass ]` method.

```py
# Get the trailing 5 days of Fred data for an asset in Fred format.
history = self.history[Fred](dataset_symbol, 5, Resolution.DAILY)
# Iterate through the historical data points.
for data_point in history:
    t = data_point.end_time
    value = data_point.value
```

Some alternative datasets provide multiple entries per asset per time step . For example, the US Regulatory Alerts dataset can provide multiple alerts per day. In this case, to organize the data into a DataFrame, set the `flatten` argument to `True` .

```
class RegalyticsHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 20)
        self.set_end_date(2024, 12, 31)
        # Get the all the Regalytics articles that were published over the last day, organized in a DataFrame.
        dataset_symbol = self.add_data(RegalyticsRegulatoryArticles, "REG").symbol
        history = self.history(dataset_symbol, 1, Resolution.DAILY, flatten=True)
```

		agencies	alerttype	...	time	title
time	symbol					
2024-12-20	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Experian Information Solutions, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Transunion Intermediate Holdings, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Transunion Intermediate Holdings, Inc.
	...	...	...	...	...	...
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.
	REG.RegalyticsRegulatoryArticles	[Consumer Financial Protection Bureau]	Complaint	...	2024-12-21	Complaint Filed: Equifax, Inc.

```
# Get all the unique alert types from the Regalytics articles.
alert_types = history.alerttype.unique()
```

```
array(['Complaint', 'Press release', 'Event', 'Litigation Release',
       'Grant Information', 'Media Release', 'News', 'Announcement',
       'Transcript', 'Decree', 'Decision', 'Regulation',
       'Executive Order', 'Media Advisory', 'Disaster Press Release',
       'Notice', 'Procurement', 'Meeting', 'News release', 'Contract',
       'Publication', 'Blog', 'Tabled Document', 'Resolution', 'Bill',
       'Concurrent Resolution', 'Opinions and Adjudicatory Orders',
       'Proposed rule', 'Technical Notice', 'Sanction', 'Order',
       'Statement', 'Rule', 'enforcement action', 'Report',
       'Statement|Release',
       'AWCs (Letters of Acceptance, Waiver, and Consent)'], dtype=object)
```

Universes

To get historical data for an alternative data universe , call the `history` method with the `Universe` object. Set the `flatten` argument to `True` to get a DataFrame that has columns for the data point attributes.

PY

```
class AltDataUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of US Equities based on an alternative dataset.
        universe = self.add_universe(BrainStockRankingUniverse)
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		rank10days	rank21days	rank2days	rank3days	rank5days	value
time	symbol						
2024-12-18	A RPTMYV3VC57P	-0.001895	0.005938	-0.007858	-0.006320	0.001771	-0.007858
	AAL VM9RIYHM8ACL	-0.003977	0.006520	-0.005671	-0.003738	-0.004715	-0.005671
	AAPL R735QTJ8XC9X	0.027450	0.037339	0.006018	0.001489	0.010102	0.006018
	ABBV VCY032R250MD	-0.002814	0.012297	-0.001717	-0.000679	-0.007454	-0.001717
	ABNB XK8H247DY6W5	0.020533	0.046173	-0.002303	0.007350	0.011252	-0.002303
...	...	...	...	...	...	...	...
2024-12-23	ZI XF2DKG2HHLK5	-0.038118	-0.037527	-0.035904	-0.041195	-0.041388	-0.035904
	ZM X3RPXTZRW09X	0.006784	0.020690	-0.005674	-0.008556	-0.002120	-0.005674
	ZS WSVU0MELFQED	0.010422	0.019619	-0.003743	-0.002079	0.002778	-0.003743
	ZTO WF2L9EOCSQCL	-0.021455	-0.015939	-0.018053	-0.020263	-0.024979	-0.018053
	ZTS VDRJHVQ4FNFP	0.030589	0.041400	0.024744	0.019891	0.031048	0.024744

PY

```
# Select the asset with the greatest value each day.
daily_winner = history.groupby('time').apply(lambda x: x.nlargest(1, 'value')).reset_index(level=1, drop=True).value
```

```
time        symbol
2024-12-18  FIC R735QTJ8XC9X    0.054204
2024-12-19  FIC R735QTJ8XC9X    0.073250
2024-12-20  FIC R735QTJ8XC9X    0.065142
2024-12-21  FIC R735QTJ8XC9X    0.065142
2024-12-22  FIC R735QTJ8XC9X    0.065142
2024-12-23  FIC R735QTJ8XC9X    0.065142
Name: value, dtype: float64
```

To get the data in the format of the objects that you receive in your universe filter function instead of a DataFrame, use `flatten=False` .

PY

```
# Get the historical universe data over the last 5 days in a Series where
# the values in the series are lists of the universe selection objects.
history = self.history(universe, timedelta(5), flatten=False)
# Select the asset with the greatest value each day.
for (universe_symbol, time), data in history.items():
    leader = sorted(data, key=lambda x: x.value)[-1]
```

Slices

To request `Slice` objects of historical data, call the `history` method without providing any `Symbol` objects. It returns data for all the data subscriptions in your algorithm, so the result may include more than just alternative data.

PY

```
class SliceHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add an asset and an alternative dataset.
        symbol = self.add_crypto('BTCUSD', Resolution.DAILY, Market.BITFINEX).symbol
        dataset_symbol = self.add_data(BitcoinMetadata, symbol).symbol
        # Get the latest 3 data points of all the securities/datasets in the algorithm, packaged into Slice objects.
        history = self.history(3)
        # Iterate through each Slice and get the synchronized data points at each moment in time.
        for slice_ in history:
            t = slice_.time
            if symbol in slice_:
                price = slice_[symbol].price
            if dataset_symbol in slice_:
                hash_rate = slice_[dataset_symbol].hash_rate
```

When your history request returns `Slice` objects, the `time` properties of these objects are based on the algorithm time zone, but the `end_time` properties of the individual data objects are based on the exchange time zone . The `end_time` is the end of the sampling period and when the data is actually available.

Sparse Datasets

A sparse dataset is a dataset that doesn't have data for every time step of its resolution. For example, the US Energy Information Administration (EIA) datasets have a daily resolution but the data for the "U.S. Ending Stocks of Finished Motor Gasoline in Thousand Barrels (Mbbl)" series only updates once a week. So when you request the trailing 30 days of historical data for it, you only get a few data points.

PY

```
class SparseDatasetHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a sparse dataset. In this case, the default resolution is daily.
        symbol = self.add_data(USEnergy, 'PET.WGFSTUS1.W').symbol
        # Get 30 days of history for the dataset.
        history = self.history(symbol, 30)
```

		value
symbol	**time**	
PET.WGFSTUS1.W.USEnergy	**2024-11-29**	14445.0
	2024-12-06	16528.0
	2024-12-13	16452.0
	2024-12-20	16833.0

Most alternative datasets have only one resolution, which is usually daily. To check if a dataset is sparse and to view its resolution(s), see the documentation in the Dataset Market .

Examples

The following examples demonstrate some common practices for trade with historical Alternative Data.

Example 1: Inverted Yield Curve

The following algorithm trades the smoothened yield curve difference, calculated from the difference between 10-year yield and 2-year yield historical data and smoothened by the exponential moving average. If the yield curve is inverted, short SPY from this market downturn indicator.

```py
class AlternativeHistoricalDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Requesting yield curve data for trade signal generation (inversion)
        self._yield_curve_symbol = self.add_data(USTreasuryYieldCurveRate, "USTYCR").symbol
        # Request XAUUSD CFD contract to trade.
        self._spy = self.add_equity("SPY").symbol

        # Ignore orders with very small sizes.
        self.settings.minimum_order_margin_portfolio_percentage = 0.01

        # Trade every day after the market opens with an updated smoothened yield curve difference.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.after_market_open(self._spy, 1),
            self.trade
        )

    def trade(self) -> None:
        # Obtain historical yield curve data to calculate the smoothened yield curve difference.
        history = self.history(self._yield_curve_symbol, 250, Resolution.DAILY).unstack(0)
        if "tenyear" in history and "twoyear" in history:
            diffs = history.tenyear - history.twoyear
            smoothened_diff = diffs.ewm(250).mean().values[-1]
            # If the smoothened yield curve difference is inverted, short SPY with the downside signal.
            self.set_holdings(self._spy, -1 if smoothened_diff < 0 else 1)
```

Historical Data

Custom Data

Introduction

You can import external datasets into your algorithm to use alongside other datasets from the Dataset Market. This page explains how to get historical data for custom datasets. Before you can get historical data for the dataset, define the `get_source` and `reader` methods of the custom data class. For examples of custom dataset implementations, see Key Concepts .

Slices

To request `Slice` objects of historical data, call the `history` method without providing any `Symbol` objects. It returns data for all the data subscriptions in your notebook, so the result may include more than just custom data.

```
# Get the latest 3 data points of all the securities/datasets in the notebook, packaged into Slice objects.
history = self.history(3)
```

When your history request returns `Slice` objects, the `time` properties of these objects are based on the algorithm time zone, but the `end_time` properties of the individual data objects are based on the exchange time zone . The `end_time` is the end of the sampling period and when the data is actually available.

Data Points

To get historical data points for a custom dataset, call the `history` method with the dataset `Symbol` . This method returns a DataFrame that contains the data point attributes of the dataset class. For an example definition of a custom data class, see the CSV Format Example .

```
class CustomSecurityHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add a custom dataset and save a reference to it's Symbol.
        dataset_symbol = self.add_data(MyCustomDataType, "MyCustomDataType", Resolution.DAILY).symbol
        # Get the trailing 5 days of MyCustomDataType data in DataFrame format.
        history = self.history(dataset_symbol, 5, Resolution.DAILY)
```

		close	high	low	open	value
symbol	**time**					
MYCUSTOMDATATYPE1M07S8	**2014-07-08**	7787.15	7792.00	7755.10	7780.40	7787.15
	2014-07-09	7623.20	7808.85	7595.90	7804.05	7623.20
	2014-07-10	7585.00	7650.10	7551.65	7637.95	7585.00

If you intend to use the data in the DataFrame to create `customDatasetClass` objects, request that the history request returns the data type you need. Otherwise, LEAN consumes unnecessary computational resources populating the DataFrame. To get a list of dataset objects instead of a DataFrame, call the `history[ customDatasetClass ]` method.

```
# Get the trailing 5 days of MyCustomDataType data for an asset in MyCustomDataType format.
history = self.history[MyCustomDataType](dataset_symbol, 5, Resolution.DAILY)
```

If the dataset provides multiple entries per time step , in the `get_source` method of your custom data class, return a `SubscriptionDataSource` that uses `FileFormat.UNFOLDING_COLLECTION` . To get the historical data of this custom data type in a DataFrame, set the `flatten` argument to `True` .

```
history = self.history(dataset_symbol, 1, Resolution.DAILY, flatten=True)
```

Universes

To get historical data for a custom data universe , call the `history` method with the `Universe` object. For an example definition of a custom data universe class, see the CSV Format Example .

```
class CustomDataUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add a universe from a custom data source and save a reference to it.
        universe = self.add_universe(
            StockDataSource, "my-stock-data-source", Resolution.DAILY, lambda data: [x.symbol for x in data]
        )
        # Get the historical universe data over the last 5 days in DataFrame format.
        history = self.history(universe, timedelta(5))
```

	symbols
time	
2017-07-05	[SPY, QQQ, FB, AAPL, IWM]
2017-07-06	[SPY, QQQ, FB, AAPL, IWM]
2017-07-07	[QQQ, AAPL, IWM, FB, GOOGL]
2017-07-08	[IWM, AAPL, FB, BAC, GOOGL]
2017-07-09	[AAPL, FB, GOOGL, GOOG, BAC]

```
# Count the number of assets in the universe each day.
universe_size_by_day = history.apply(lambda row: len(row['symbols']), axis=1)
```

```
time
2017-07-05    5
2017-07-06    5
2017-07-07    5
2017-07-08    5
2017-07-09    5
Name: symbols, dtype: int64
```

Missing Data Points

History requests for a trailing number of data samples return data based on the market hours of assets. The default market hours for custom securities is to be always open. Therefore, history requests for a trailing number of data samples may return fewer samples than you expect. To set the market hours of the dataset, see Market Hours .

Examples

The following examples demonstrate some common practices for trading with historical custom data.

Example 1: Custom Universe Data

The following algorithm trades based on a custom universe dataset. It obtains historical data on the universe's weights and uses the exponential moving average as the investment weight.

```py
class CustomUniverseExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add a universe that reads from the Object Store.
        self._universe = self.add_universe(
            CustomUniverseData, "CustomUniverse", Resolution.DAILY, lambda x: [y.symbol for y in x]
        )
        # Add a Scheduled Event to rebalance the portfolio.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.schedule.on(
            self.date_rules.every_day(spy),
            self.time_rules.after_market_open(spy, 1),
            self.rebalance
        )

    def rebalance(self) -> None:
        # We invest by the EMA of the given weight.
        history = self.history(self._universe, 5, Resolution.DAILY)
        if history.empty:
            return
        history = history.set_index('symbol', append=True).unstack('symbol').fillna(0)
        weights = history.ewm(5).mean().iloc[-1].droplevel(0)
        # Normalized weights.
        weights = weights / weights.sum()
        self.set_holdings([PortfolioTarget(symbol, weight) for symbol, weight in weights.items()], liquidate_existing_holdings=True)


class CustomUniverseData(PythonData):
    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live_mode: bool) -> SubscriptionDataSource:
        # Define the location and format of the data file.
        return SubscriptionDataSource(
            "portfolio-targets.csv", SubscriptionTransportMedium.OBJECT_STORE, FileFormat.CSV
        )

    def reader(self, config: SubscriptionDataConfig, line: str, date: datetime, is_live_mode: bool) -> BaseData:
        # Skip the header row.
        if not line[0].isnumeric():
            return None
        # Split the line by each comma.
        items = line.split(",")
        # Parse the data from the CSV file.
        data = CustomUniverseData()
        data.end_time = datetime.strptime(items[0], "%Y-%m-%d")
        data.time = data.end_time - timedelta(1)
        data['symbol'] = Symbol.create(items[1], SecurityType.EQUITY, Market.USA)
        data["weight"] = float(items[2])
        return data
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
HistoryAlgorithm.py Python

Historical Data

Universe Data

Introduction

Universe selection is the process of selecting a basket of assets you may trade. Dynamic universe selection increase diversification and decrease selection bias in your algorithm. To get the history of a universe, call the `history` method with the universe object. Set the `flatten` argument to `True` to organize the data in a DataFrame for easy pandas wrangling. Note that the `history` method doesn't filter the universe with your filter function. It simply returns all of the assets and data points in the universe dataset.

US Equity Fundamentals

The following example gets the history of a US Equity fundamental universe :

```
class USEquityFundamentalUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of US Equities based on the Fundamentals dataset.
        universe = self.add_universe(lambda fundamentals: [f.symbol for f in fundamentals])
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		adjusted																		
time	symbol																			
2024-12-19	A RPTMYV3VC57P	132.764597																		
	AA R735QTJ8XC9X	107.606519																		
	AA WF7IHVI76I5H	37.310000																		
	AAA XHPVXIJUWS4L	24.822918																		
	AAAU WX1IHYU092ESL	25.635000																		
...	...	...																		
2024-12-21	ZUO WTMF35A53QD	9.910000																		
	ZVIA XQD1725736UD	3.180000																		
	ZVSA Y4A7QNC417J9	1.140000																		
	ZYME WK2S2IM8SSPX	14.120000																		
	ZYXI X1ZPVR7SWYUD	8.250000																		

```
# Get the PE Ratio of the first row in the DataFrame.
pe_ratio = history.iloc[0].valuationratios.pe_ratio
```

US Equity ETF Constituents

The following example gets the history of a US Equity ETF constituents universe :

```
class ETFConstituentUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of US Equities based on the constituents of an ETF.
        universe = self.add_universe(self.universe.etf('SPY'))
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		lastupdate	period	sharesheld	weight
time	**symbol**				
2024-12-19	**A RPTMYV3VC57P**	2024-12-17	1 days	3682327.0	0.000777
	AAPL R735QTJ8XC9X	2024-12-17	1 days	191633535.0	0.075065
	ABBV VCY032R250MD	2024-12-17	1 days	22256743.0	0.006032
	ABNB XK8H247DY6W5	2024-12-17	1 days	5536841.0	0.001131
	ABT R735QTJ8XC9X	2024-12-17	1 days	21919703.0	0.003837
...	**...**	...	...	...	...
2024-12-21	**XYL V18KR26TE3XH**	2024-12-19	1 days	2947931.0	0.000569
	YUM R735QTJ8XC9X	2024-12-19	1 days	3412693.0	0.000737
	ZMH S6ZZPKTVDY05	2024-12-19	1 days	2471688.0	0.000433
	ZBRA R735QTJ8XC9X	2024-12-19	1 days	625772.0	0.000400
	ZTS VDRJHVQ4FNFP	2024-12-19	1 days	5484351.0	0.001486

```
# Select the 2 assets with the smallest weights in the ETF each day.
daily_smallest = history.groupby('time').apply(lambda x: x.nsmallest(2, 'weight')).reset_index(level=1, drop=True).weight
```

```
time        symbol
2024-12-19  AMTMW YM37RIGZUD0L    0.000053
            NWSVV VHJF6S7EZRL1    0.000068
2024-12-20  AMTMW YM37RIGZUD0L    0.000051
            NWSVV VHJF6S7EZRL1    0.000069
2024-12-21  AMTMW YM37RIGZUD0L    0.000048
            NWSVV VHJF6S7EZRL1    0.000069
Name: weight, dtype: float64
```

US Equity Alternative Data

The following example gets the history of a US Equity alternative data universe :

```
class USEquityAltDataUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of US Equities based on an alternative dataset.
        universe = self.add_universe(BrainStockRankingUniverse)
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		rank10days	rank21days	rank2days	rank3days	rank5days	value
time	symbol						
2024-12-18	A RPTMYV3VC57P	-0.001895	0.005938	-0.007858	-0.006320	0.001771	-0.007858
	AAL VM9RIYHM8ACL	-0.003977	0.006520	-0.005671	-0.003738	-0.004715	-0.005671
	AAPL R735QTJ8XC9X	0.027450	0.037339	0.006018	0.001489	0.010102	0.006018
	ABBV VCY032R250MD	-0.002814	0.012297	-0.001717	-0.000679	-0.007454	-0.001717
	ABNB XK8H247DY6W5	0.020533	0.046173	-0.002303	0.007350	0.011252	-0.002303
...	...	...	...	...	...	...	...
2024-12-23	ZI XF2DKG2HHLK5	-0.038118	-0.037527	-0.035904	-0.041195	-0.041388	-0.035904
	ZM X3RPXTZRW09X	0.006784	0.020690	-0.005674	-0.008556	-0.002120	-0.005674
	ZS WSVU0MELFQED	0.010422	0.019619	-0.003743	-0.002079	0.002778	-0.003743
	ZTO WF2L9EOCSQCL	-0.021455	-0.015939	-0.018053	-0.020263	-0.024979	-0.018053
	ZTS VDRJHVQ4FNFP	0.030589	0.041400	0.024744	0.019891	0.031048	0.024744

```
# Select the asset with the greatest value each day.
daily_winner = history.groupby('time').apply(lambda x: x.nlargest(1, 'value')).reset_index(level=1, drop=True).value
```

```
time        symbol
2024-12-18  FIC R735QTJ8XC9X    0.054204
2024-12-19  FIC R735QTJ8XC9X    0.073250
2024-12-20  FIC R735QTJ8XC9X    0.065142
2024-12-21  FIC R735QTJ8XC9X    0.065142
2024-12-22  FIC R735QTJ8XC9X    0.065142
2024-12-23  FIC R735QTJ8XC9X    0.065142
Name: value, dtype: float64
```

Equity Options

The following example gets the history of an Equity Options universe :

```
class EquityOptionUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Equity Option contracts.
        option = self.add_option('SPY')
        # Get 5 days of history for the universe.
        history = self.history(option.symbol, timedelta(5), flatten=True)
```

		close	delta	gamma	high	impliedvolatility	low	open	openinterest	rho	theta	underlyingvalue	vega	volume
time	symbol													
2024-12-19	SPY YOGVNNGGQDD|SPY R735QTJ8XC9X	[illegible]	1.000000	0.000000	486.910	0.000000	465.655	484.000	1543.0	0.000000	0.000000	SPY: □586.28	466.350	0.000000	7.0
	SPY YOGVNNGGSZB|SPY R735QTJ8XC9X	[illegible]	1.000000	0.000000	476.910	0.000000	455.575	474.010	31.0	0.009492	-0.000385	SPY: □586.28	455.800	-0.001621	5.0
	SPY YOGVNNGJ05E|SPY R735QTJ8XC9X	[illegible]	1.000000	0.000000	466.910	0.000000	445.750	463.985	9.0	0.010222	-0.001891	SPY: □586.28	445.960	-0.005638	5.0
	SPY YOGVNNGJ4HS|SPY R735QTJ8XC9X	[illegible]	1.000000	0.000000	456.915	0.000000	435.545	454.005	23.0	0.010952	-0.003397	SPY: □586.28	436.445	-0.009656	5.0
	SPY YTG30NXW1Q8|SPY R735QTJ8XC9X	[illegible]	1.000000	0.000000	458.475	0.722009	437.950	456.650	33.0	0.000000	0.000000	SPY: □586.28	438.380	0.000000	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-12-21	SPY 33899RR9ZZ3Q|SPY R735QTJ8XC9X	[illegible]	-0.867002	0.000479	357.380	0.141164	272.505	318.220	1.0	-5.758672	0.037828	SPY: □591.15	309.500	0.549488	0.0
	SPY 337HP99QFU5A|SPY R735QTJ8XC9X	[illegible]	-0.877658	0.000388	362.425	0.136831	277.490	323.190	0.0	-5.659564	0.039793	SPY: □591.15	314.500	0.414755	0.0
	SPY 33899RV2ZZ12|SPY R735QTJ8XC9X	[illegible]	-0.870742	0.000427	362.370	0.138433	277.495	322.940	0.0	-5.845895	0.038531	SPY: □591.15	314.315	0.481876	0.0
	SPY 337HP952Z1K8|SPY R735QTJ8XC9X	[illegible]	-0.877873	0.000381	367.425	0.138304	282.490	328.190	0.0	-5.749542	0.040433	SPY: □591.15	319.500	0.411621	0.0
	SPY 33899RS8JP5Z|SPY R735QTJ8XC9X	[illegible]	-0.869815	0.000438	367.370	0.141717	282.495	328.200	1.0	-5.941354	0.039119	SPY: □591.15	319.500	0.503219	0.0

```
# Find the open interest value at the 99th percentile.
high_open_interest = history.openinterest.quantile(0.99)
```

Index Options

The following example gets the history of an Index Options universe :

```
class IndexOptionsUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Index Option contracts.
        option = self.add_index_option('VIX')
        # Get 5 days of history for the universe.
        history = self.history(option.symbol, timedelta(5), flatten=True)
```

		close	delta	gamma	high	impliedvolatility	low	open	openinterest	rho	theta	underlyingvalue	vega	volume
time	symbol													
2024-12-19	VIX YPDGYCC[illegible]|VIX 31	[illegible]	0.000000	0.000000	11.950	0.000000	6.425	6.575	3868.0	0.008740	-0.001349	VIX: 27.65	0.000000	3747.0
	VIX YQ51GYL[illegible]|VIX 31	[illegible]	0.000000	0.000000	11.825	0.000000	8.225	8.550	59.0	0.015541	-0.001343	VIX: 27.65	0.000000	9.0
	VIX YPDGYCB[illegible]|VIX 31	[illegible]	0.000000	0.000000	11.050	0.000000	4.255	6.805	905.0	0.009711	-0.001499	VIX: 27.65	0.000000	250.0
	VIX YQ51GYJ[illegible]|VIX 31	[illegible]	0.000000	0.000000	10.325	0.000000	7.225	7.550	225.0	0.017268	-0.001492	VIX: 27.65	0.000000	112.0
	VIX YQVMJH[illegible]|VIX 31	[illegible]	0.000000	0.000000	10.225	0.000000	5.430	5.580	192.0	0.024495	-0.001486	VIX: 27.65	0.000000	22.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2024-12-21	VIX 32P91ME[illegible]|VIX 31	[illegible]	0.987482	0.001990	182.125	2.167690	176.675	177.700	1.0	-0.330896	0.025512	VIX: 18.37	179.925	0.002433	0.0
	VIX 32PZMOX[illegible]|VIX 31	[illegible]	0.984647	0.002316	180.850	1.849262	176.050	177.350	84.0	-0.475329	0.026046	VIX: 18.37	179.150	0.003485	0.0
	VIX 32QS6NN[illegible]|VIX 31	[illegible]	0.981929	0.002610	180.100	1.636117	174.450	176.875	20.0	-0.629100	0.026343	VIX: 18.37	178.425	0.004620	0.0
	VIX 32RQNAW[illegible]|VIX 31	[illegible]	0.981896	0.002619	179.300	1.433080	174.200	176.225	3.0	-0.812986	0.026931	VIX: 18.37	177.500	0.005274	0.0
	VIX 32SI7TJ[illegible]|VIX 31	[illegible]	0.982793	0.002528	178.575	1.305996	173.400	175.600	4.0	-0.958744	0.027301	VIX: 18.37	176.725	0.005495	0.0

```
# Get the daily trading volume across all the contracts.
daily_volume = history.groupby('time').apply(lambda x: x.volume.sum())
```

```
time
2024-12-19    1285753.0
2024-12-20    1374216.0
2024-12-21     833384.0
dtype: float64
```

Crypto

The following example gets the history of a Crypto universe :

```py
class CryptoUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 23)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Cryptocurrencies.
        universe = self.add_universe(CryptoUniverse.coinbase(lambda data: [x.symbol for x in data]))
        # Get 5 days of history for the universe.
        history = self.history(universe, timedelta(5), flatten=True)
```

		close	high	low	open	price	volume	volumeinusd
time	**symbol**							
2024-12-19	**00USD 2XR**	0.050300	0.051500	0.049200	0.051500	0.050300	3.173495e+06	1.596268e+05
	00USDC 2XR	0.050300	0.051500	0.049200	0.051500	0.050300	3.173495e+06	1.596268e+05
	1INCHBTC 2XR	0.000004	0.000005	0.000004	0.000005	0.000004	9.815380e+03	4.584371e+03
	1INCHEUR 2XR	0.446000	0.467000	0.442000	0.465000	0.446000	9.007325e+04	4.215390e+04
	1INCHGBP 2XR	0.369000	0.385000	0.366000	0.381000	0.369000	3.932220e+04	1.845807e+04
...	**...**	...	...	...	...	...	...	...
2024-12-23	**ZETAUSDC 2XR**	0.574500	0.638600	0.562600	0.598400	0.574500	1.522481e+06	8.746654e+05
	ZROUSD 2XR	5.487000	6.363000	5.393000	5.635000	5.487000	1.312069e+05	7.199321e+05
	ZROUSDC 2XR	5.487000	6.363000	5.393000	5.635000	5.487000	1.312069e+05	7.199321e+05
	ZRXUSD 2XR	0.449944	0.509425	0.441590	0.476001	0.449944	3.371118e+06	1.516814e+06
	ZRXUSDC 2XR	0.449944	0.509425	0.441590	0.476001	0.449944	3.371118e+06	1.516814e+06

```py
# Get the Crypto with the largest daily return each day.
history['roc'] = (history['close'] - history['open']) / history['open']
top_gainers = history.loc[history.groupby('time')['roc'].idxmax()]
```

		close	high	low	open	price	volume	volumeinusd	roc
time	symbol								
2024-12-19	PRQUSD 2XR	0.19020	0.21000	0.14930	0.15360	0.19020	1.934896e+07	3.680171e+06	0.238281
2024-12-20	NCTUSD 2XR	0.03696	0.03844	0.02740	0.02956	0.03696	2.584655e+08	9.552883e+06	0.250338
2024-12-21	VELOUSD 2XR	0.22099	0.23863	0.19002	0.20075	0.22099	8.206130e+07	1.813473e+07	0.100822
2024-12-22	ZENUSD 2XR	26.10600	28.39000	15.70500	15.83300	26.10600	5.834163e+05	1.523067e+07	0.648835
2024-12-23	AGLDUSD 2XR	2.15440	2.17030	1.30540	1.31010	2.15440	5.144987e+06	1.108436e+07	0.644455

Crypto Market Cap

The following example gets the history of a Crypto market cap universe :

PY

```
class CryptoMarketCapUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 12, 24)
        self.set_end_date(2024, 12, 31)
        # Add a universe of Cryptocurrencies.
        universe = self.add_universe(CoinGeckoUniverse)
        # Get 355 days of history for the universe.
        history = self.history(universe, timedelta(365), flatten=True)
```

		coin	marketcap	value	volume
time	symbol				
2023-12-26	DAPP.CoinGecko 2S	DAPP	0.000000e+00	0.000111	1.203664e+03
	FLUID.CoinGecko 2S	FLUID	3.385875e+07	1.886653	8.124432e+04
	PAAL.CoinGecko 2S	PAAL	7.678778e+07	0.118019	3.264158e+06
	TOSHI.CoinGecko 2S	TOSHI	5.358157e+07	0.000127	9.605914e+05
2023-12-27	DAPP.CoinGecko 2S	DAPP	0.000000e+00	0.000109	1.189876e+03
...	...	...	...	...	...
2024-12-24	VIRTUAL.CoinGecko 2S	VIRTUAL	3.016890e+09	3.014802	4.121598e+08
	XION.CoinGecko 2S	XION	7.990578e+07	3.126583	1.454954e+07
	ZAP.CoinGecko 2S	ZAP	3.094296e+06	0.032612	1.781765e+05
	ZEREBRO.CoinGecko 2S	ZEREBRO	3.018897e+08	0.300990	7.376285e+07
	ZRC.CoinGecko 2S	ZRC	1.660151e+08	0.075640	1.373892e+08

PY

```
# Select the largest Crypto each day.
largest_coin = history.groupby('time').apply(
    lambda x: x.nlargest(1, 'marketcap')
).reset_index(level=1, drop=True).marketcap
```

```
time        symbol
2023-12-26  PAAL.CoinGecko 2S       7.678778e+07
2023-12-27  PAAL.CoinGecko 2S       6.908865e+07
2023-12-28  PAAL.CoinGecko 2S       1.019541e+08
2023-12-29  PAAL.CoinGecko 2S       1.007006e+08
2023-12-30  PAAL.CoinGecko 2S       8.907086e+07
                                        ...
2024-12-20  VIRTUAL.CoinGecko 2S    2.297529e+09
2024-12-21  VIRTUAL.CoinGecko 2S    2.811267e+09
2024-12-22  VIRTUAL.CoinGecko 2S    2.476485e+09
2024-12-23  VIRTUAL.CoinGecko 2S    2.328428e+09
2024-12-24  VIRTUAL.CoinGecko 2S    3.016890e+09
Name: marketcap, Length: 365, dtype: float64
```

Custom Data

The following example gets the history of a custom data universe :

PY

```
# Define the custom data class outside of the algorithm class.
class StockDataSource(PythonData):

    def get_source(self, config: SubscriptionDataConfig, date: datetime, is_live: bool) -> SubscriptionDataSource:
        return SubscriptionDataSource(
            "https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-data/csv-universe-
example.csv",
            SubscriptionTransportMedium.REMOTE_FILE,
            FileFormat.CSV
        )

    def reader(self,config: SubscriptionDataConfig, line: str, date: datetime, is_live: bool) -> BaseData:
        stocks = StockDataSource()
        stocks.symbol = config.symbol
        if not (line.strip() and line[0].isdigit()):
            return stocks
        csv = line.split(',')
        try:
            stocks.time = datetime.strptime(csv[0], "%Y%m%d")
            stocks.end_time = stocks.time + timedelta(days=1)
            stocks["Symbols"] = csv[1:]
        except ValueError:
            StockDataSource.algorithm.log(f'{date}: ValueError')
        return stocks


class CustomDataUniverseHistoryAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Attach the algorithm instance to the custom data class to enable logging.
        StockDataSource.algorithm = self
        # Add a custom data universe.
        universe = self.add_universe(StockDataSource)
        # Get the universe history.
        history = self.history(universe, datetime(2018, 1, 1), datetime(2018, 1, 10), flatten=True)
```

		symbols
symbol	**time**	
STOCKDATASOURCE-USA-AE903F8D-F507-4638-BBFC-EE65760179AE.StockDataSource 2S	**2018-01-04**	[SPY, QQQ, AAPL, IWM, FB]
	2018-01-05	[AAPL, QQQ, IWM, FB, GOOGL]
	2018-01-06	[QQQ, IWM, FB, BAC, GOOGL]
	2018-01-07	[IWM, FB, GOOGL, BAC, GOOG]
	2018-01-08	[IWM, FB, GOOGL, BAC, GOOG]
	2018-01-09	[IWM, FB, GOOGL, BAC, GOOG]
	2018-01-10	[IWM, BAC, GOOGL, GOOG, IBM]

```
# Count the number of assets in the universe each day.
universe_size = history.apply(lambda row: len(row.symbols), axis=1).reset_index(level=0, drop=True)
```

```
time
2018-01-04    5
2018-01-05    5
2018-01-06    5
2018-01-07    5
2018-01-08    5
2018-01-09    5
2018-01-10    5
dtype: int64
```

Examples

The following examples demonstrate some common practices for trading with historical universe data.

Example 1: Historical Ranking Prediction

The following example forms an equal-weighted portfolio at the start of each month with the 5 assets that have the greatest 21-day ranking in the Brain ML Stock Ranking dataset.

```
class UniverseHistoricalDataAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)
        self.settings.seed_initial_prices = True

        # Add a monthly update to the universe of BrainStockRanking data.
        self.universe_settings.schedule.on(self.date_rules.month_start("SPY"))
        self._universe = self.add_universe(BrainStockRankingUniverse)

        # Trade on every month's start, based on the predicted rank in the next month.
        self.schedule.on(
            self.date_rules.month_start("SPY"),
            self.time_rules.at(9, 31),
            self.trade
        )

        self.set_warm_up(1)

    def trade(self) -> None:
        # Get the last 5 bars of history for the universe.
        history = self.history(self._universe, 5, fill_forward=False, flatten=True).rank21days.unstack(0)
        # Obtain the 5 highest average rank stocks to invest in equally.
        top_ranked = history.mean(axis=1).nlargest(n=5)
        self.set_holdings([PortfolioTarget(symbol, 0.2) for symbol in top_ranked.index], liquidate_existing_holdings=True)
```

Historical Data

Warm Up Periods

Introduction

LEAN supports an automated fast-forward system called "Warm Up". It simulates winding back the date you deployed your algorithm by a specific time period. In backtests, it changes the start date of your algorithm to an earlier time. In live trading, it fetches data from before your deployment and fast forwards through your strategy up to the present day. Warm Up is a great way to prepare your algorithm and its indicators for trading. You can't place trades during the warm-up period because the data feed is replaying historical data for setting algorithm state.

Set Warm Up Periods

You can set a warm-up period based on a number of bars or a period of time.

Trailing Data Samples

To set a warm-up based on a trailing number of data samples, in your algorithm's `initialize` method, call the `set_warm_up` method with an integer argument.

```py
# Feed in 100 bars before start date
self.set_warm_up(100)
```

LEAN calculates the start of the warm-up period for each of your security subscriptions by using the number of bars you specify, the resolution of each security, and the trading calendar of each security. After it calculates the start time of the warm-up period for each security, it sets the earliest start time as the start of the algorithm warm-up period. For instance, in the following example, the warm-up period consists of 100 minute resolution bars for SPY and as many second resolution bars for BTCUSD that occur from the start of the SPY warm-up period to the algorithm `start_date` .

```py
class WarmUpWithTrailingDataSamplesAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.add_equity("SPY", Resolution.MINUTE, fill_forward=False)
        self.add_crypto("BTCUSD", Resolution.SECOND, fill_forward=False)
        self.set_warm_up(100)
```

To use a specific resolution of data for the warm-up period, pass a `resolution` argument to the `set_warm_up` method.

```py
# Feed in data for 100 trading days before the start date
self.set_warm_up(100, Resolution.DAILY)
```

If you pass an integer and a resolution, the warm-up period consists of the number of bars and resolution you set, regardless of the resolution of your data subscriptions. In this case, LEAN calculates the start of the warm-up period for each of your security subscriptions just like it does when you only pass an integer argument. After it calculates the start time of the warm-up period for each security, it sets the earliest start time as the start of the algorithm warm-up period.

If you pass a resolution argument and you registered indicators or consolidators for automatic updates, the warm-up period resolution must be less than or equal to the lowest resolution of your automatic indicators and consolidators. For instance, in the following example, the indicators use minute resolution data, so you can set the warm-up period to use tick, second, or minute resolution data.

```
class WarmUpWithTrailingDataSamplesAndResolutionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._spy = self.add_equity("SPY", Resolution.MINUTE, fill_forward=False).symbol
        self._btc = self.add_crypto("BTCUSD", Resolution.SECOND, fill_forward=False).symbol

        self._spy_sma = self.sma(self._spy, 10)
        self._btc_sma = self.sma(self._btc, 10, Resolution.MINUTE)

        self.set_warm_up(100, Resolution.MINUTE)
```

Trailing Time Periods

To set a warm-up based on a trailing time period, in your algorithm's `initialize` method, call the `set_warm_up` method with a `timedelta` . The warm-up period consists of that `timedelta` before the start date and pipes in data that matches the resolution of your data subscriptions.

```
# Wind time back 7 days from start
self.set_warm_up(timedelta(7))
```

To use a specific resolution of data for the warm-up period, pass a `resolution` argument to the `set_warm_up` method.

```
# Feed in 100 days of daily data before the start date
self.set_warm_up(timedelta(days=100), Resolution.DAILY)
```

If you pass a `timedelta` and a resolution, the warm-up period consists of the time period and resolution you set, regardless of the resolution of your data subscriptions.

Warm Up Status

You can't place trades during the warm-up period because the data feed is replaying historical data for setting algorithm state. To check if the algorithm is currently in a warm up period, use the `is_warming_up` property.

```
class WarmUpStatusAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        symbol = self.add_equity('IBM', Resolution.DAILY).symbol
        self._ema = self.ema(symbol, 100)
        self.set_warmup(100)

    def on_data(self, data) -> None:
        if self.is_warming_up:
            return
        self.plot('EMA', 'Is ready', int(self._ema.is_ready))
```

Event Handler

The `on_warmup_finished` event handler executes when the warm up period ends, even if you don't set a warm up period.

```
def on_warmup_finished(self) -> None:
    pass # Done warming up
```

Missing Data Points

There are several cases where you may get fewer data points from during warm-up than you expect.

Illiquid Securities

The default behavior for LEAN is to fill-forward the data. However, if an asset had no trading activity during the time period you request historical data for, you won't get any data for that asset.

Requesting Data Before an Asset's IPO

All assets have a day when they first started trading. If your warm-up period includes time before an asset's initial public offering (IPO) date, you won't get any data before the IPO date because the data doesn't exist.

Requesting Data Outside Market Hours

Warming up with a trailing number of data samples means the data is based on the market hours of assets, so they can wrap across multiple trading days to gather the amount of data you request. In contrast, warming up with a trailing time period provides means the data is based on a rolling calendar time. In this latter types of warm-up, if the period of time is outside of the market hours of an asset, you won't get any data.

Data Issues

Data issues are generally a result of human error or from mistakes in the data collection process. Any QuantConnect member can report a data issue , which our Data Team works to quickly resolve. However, while the data is still missing, your warm-up won't be able to provide the data. To view the list of current data issues, log in to the Algorithm Lab and then, in the left navigation bar, click Datasets > Data Issues .

Data Provider Limitations

When you run warm-up, consider the functionality of the data provider you use. For example, the QuantConnect data provider has quotas on the amount of data you can request and the Interactive Brokers data provider can be slow, so it can timeout if you run warm-up with too many assets. For more information about all the data providers, see their respective documentation in Cloud Platform , Local Platform , or the CLI .

Warm Up vs History Requests

Warm-up and history requests both provide historical data. The approach you take depends on your specific use case. A benefit to using warm-up is that it only takes one line of code to pump historical data into your algorithm, which processes universe selection and updates all the indicators. A consequence to using warm-up is that it may provide more historica data than you actually need since it provides data on all the subscriptions in your algorithm. If you have derivative assets in your algorithm, warm-up can be very slow to process.

Examples

The following examples demonstrate some common practices for warm up.

Example 1: Warm Up Indicator

The following algorithm trades mean-reversion according to signals from a Bollinger Bands indicator. It uses warm up so the indicator is ready by the start date of the algorithm.

```py
class WarmUpPeriodsAlgorithm(QCAlgorithm):
    bbands = BollingerBands(20, 2)

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # The candlestick patterns are based on a daily consolidated trade bar.
        consolidator = TradeBarConsolidator(timedelta(1))
        # Subscribe to update the indicators with the 1-day consolidator automatically.
        self.register_indicator(self.spy, self.bbands, consolidator)
        # Add an event handler on candlestick indicators that are updated to trade the indicator.
        self.bbands.updated += self.on_updated

        # Warm up the indicator and the SPY price data.
        self.set_warm_up(20, Resolution.DAILY)

    def on_updated(self, sender: object, point: IndicatorDataPoint) -> None:
        if not sender.is_ready:
            return

        # Trade according to the updated indicator values.
        upper_band = sender.upper_band.current.value
        lower_band = sender.lower_band.current.value
        holdings = self.portfolio[self.spy]

        # Trade mean-reversal of the Bollinger Band.
        if holdings.price > upper_band and not holdings.is_short:
            self.set_holdings(self.spy, -0.5)
        if holdings.price < lower_band and not holdings.is_long:
            self.set_holdings(self.spy, 0.5)
```

Example 2: Warm Up Universe Selection

The following algorithm forms an equal-weighted portfolio with the 100 most liquid US Equities that are above their 60-day Exponential Moving Average (EMA). It uses the warm-up feature to prime the indicators in advance so it can start trading by the start date of the algorithm.

PY

```
class WarmUpPeriodsAlgorithm(QCAlgorithm):
    ema_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True

        # The universe is filtered by only the ones with prices above EMA, suggesting an uptrend.
        self._universe = self.add_universe(self.selection_by_ema)

        # Rebalance weekly.
        self.schedule.on(
            self.date_rules.week_start(),
            self.time_rules.at(9, 31),
            self.rebalance
        )

        self.set_warm_up(60, Resolution.DAILY)

    def selection_by_ema(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        for f in fundamentals:
            # Create an EMA indicator for the symbol if it is not available.
            self.ema_by_symbol[f.symbol] = self.ema_by_symbol.get(f.symbol, ExponentialMovingAverage(60))
            # Update the indicator using the updated price data to select the updated data.
            self.ema_by_symbol[f.symbol].update(f.end_time, f.adjusted_price)

        # We don't add new securities during the warm-up process
        if self.is_warming_up:
            return Universe.UNCHANGED

        # Select the ones with the current price above EMA to filter for the uptrend stocks.
        def is_uptrend(f):
            ema = self.ema_by_symbol[f.symbol]
            return ema.is_ready and ema.current.value > f.adjusted_price

        # Only trades the top 100 liquid stocks since their trend capitalizes quicker.
        fundamentals = sorted([f for f in fundamentals if is_uptrend(f)],
            key=lambda x: x.dollar_volume, reverse=True)

        return [f.symbol for f in fundamentals[:100]]

    def rebalance(self) -> None:
        #  Equally invest in uptrend stocks to dissipate capital risk evenly.
        weight = 1 / len(self._universe.selected) if self._universe.selected else 0
        if weight == 0:
            return
        targets = [PortfolioTarget(symbol, weight) for symbol in self._universe.selected]
        self.set_holdings(targets)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Exit position if the equity is below EMA since it becomes a downgoing trend.
        [self.liquidate(r.symbol) for r in changes.removed_securities]
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
WarmupAlgorithm.py Python

Historical Data

Rolling Window

Introduction

A `RollingWindow` is an array of a fixed-size that holds trailing data. It's more efficient to use `RollingWindow` objects to hold periods of data than to make multiple historical data requests . With a `RollingWindow` , you just update the latest data point while a `history` call fetches all of the data over the period you request. `RollingWindow` objects operate on a first-in, first-out process to allow for reverse list access semantics with positive and negative indexing. Index 0 refers to the most recent item in the window and the largest index refers to the last item in the window. Index -1 refers to the latest item as well.

Create Rolling Windows

To create rolling windows, instantiate the `RollingWindow` object with a maximum size.

```
PY
self._close_price_window = RollingWindow(4)
self._trade_bar_window = RollingWindow(2)
self._quote_bar_window = RollingWindow(2)
self._tuple_window = RollingWindow(2)
```

You can define the rolling window type with `RollingWindow[T]` , where `T` is the native or C# type of elements in the window.

```
PY
self._close_price_window = RollingWindow[float](4)
self._trade_bar_window = RollingWindow[TradeBar](2)
self._quote_bar_window = RollingWindow[QuoteBar](2)
```

The typeless version, `RollingWindow(int)` , allows for Python types. For example, `tuple` .

Add Data

To add data to a `RollingWindow` , call the `add` method.

```
PY
self._close_price_window.add(data["SPY"].close)
self._trade_bar_window.add(data["SPY"])
self._quote_bar_window.add(data["EURUSD"])
self._tuple_window.add((1, 2))
```

To update the data at a specific index, set the value for that index. If the index doesn't currently exist, it increases the size and fills the empty indices with a default value (zero or `None`).

```
PY
self._close_price_window[0] = data["SPY"].close
self._trade_bar_window[0] = data["SPY"]
self._quote_bar_window[0] = data["EURUSD"]
self._tuple_window[0] = (1, 2)
```

Warm Up

To warm up a `RollingWindow` , make a history request and then iterate through the result to add the data to the `RollingWindow` .

```
class WarmUpRollingWindowAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        spy = self.add_equity("SPY", Resolution.DAILY).symbol
        history_trade_bar = self.history[TradeBar](spy, 10, Resolution.DAILY)
        history_quote_bar = self.history[QuoteBar](spy, 10, Resolution.MINUTE)

        # Warm up the close price and trade bar rolling windows with the previous 10-day trade bar data
        close_price_window = RollingWindow(10)
        trade_bar_window = RollingWindow(10)
        for trade_bar in history_trade_bar:
            close_price_window.add(trade_bar.close)
            trade_bar_window.add(trade_bar)

        # Warm up the quote bar rolling window with the previous 10-minute quote bar data
        quote_bar_window = RollingWindow(10)
        for quote_bar in history_quote_bar:
            quote_bar_window.add(quote_bar)
```

Check Readiness

To check if a `RollingWindow` is full, use its `is_ready` flag.

```
if not self._close_price_window.is_ready:
    return
```

Adjust Size

To adjust the `RollingWindow` size, set the `size` property.

```
self._close_price_window.size = 3
self._trade_bar_window.size = 3
self._quote_bar_window.size = 3
self._tuple_window.size = 3
```

When you decrease the size, it removes the oldest values that no longer fit in the `RollingWindow` . When you explicitly increase the `size` member, it doesn't automatically add any new elements to the `RollingWindow` . However, if you set the value of an index in the `RollingWindow` and the index doesn't currently exist, it fills the empty indices with a default value (zero or `None`). For example, the following code increases the `size` to 10, sets the 10th element to 3, and sets the 4th-9th elements to the default value:

```
self._close_price_window[9] = 3
```

Access Data

`RollingWindow` objects operate on a first-in, first-out process to allow for reverse list access semantics with positive and negative indexing. Index 0 refers to the most recent item in the window and the largest index refers to the last item in the window. Index -1 refers to the latest item as well.

```
current_close = self._close_price_window[0]
previous_close = self._close_price_window[1]
oldest_close = self._close_price_window[self._close_price_window.count-1]
oldest_close = self._close_price_window[-1]   # Negative indexing
```

To get the item that was most recently removed from the `RollingWindow` , use the `most_recently_removed` property.

```
removed_close = self._close_price_window.most_recently_removed
```

Combine with Indicators

The `window` property of the indicators is a built-in `RollingWindow` that stores historical values. It holds a collection of `IndicatorDataPoint` objects, enabling quick access to the most recent historical indicator values for analysis, calculations, or comparisons in trading and financial strategies. Its

default size 2, but you can adjust its size. To warm up the indicator and its `RollingWindow`, call the `indicator_history` method with the window size as an argument.

```
class IndicatorRollingWindowAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        spy = self.add_equity("SPY").symbol
        qqq = self.add_equity("QQQ").symbol

        # Manual SMA indicator example.
        self._sma = SimpleMovingAverage(20)
        # Automatic Beta indicator example.
        self._beta = self.b(spy, qqq, 20)

        # Adjust the window size to 5 to access the previous 5 indicator data points.
        self._sma.window.size = 5
        self._beta.window.size = 5

        # Warm up the indicator and window for immediate usage.
        self.indicator_history(self._sma, spy, self._sma.window.size, Resolution.DAILY)
        # For indicators using 2+ symbols, use an iterable symbol list.
        self.indicator_history(self._beta, [spy, qqq], self._beta.window.size, Resolution.DAILY)
```

Combine with Consolidators

To store a history of consolidated bars, in the consolidation handler, add the consolidated bar to the `RollingWindow`.

```
self._consolidator.data_consolidated += self._on_data_consolidated

# Define consolidator handler function as a class method
def _on_data_consolidated(self, sender, consolidated_bar):
    self._trade_bar_window.add(consolidated_bar)
```

Market Session

The `Session` property of a `Security` object contains the open, high, low, close, volume, and open interest of trading sessions. The `Session` object is a `RollingWindow` object with size of zero by default. To enable market session tracking, set the size of the `Session` object to 2, so the previous session data is at index 1.

```
session = self.securities[self._symbol].session
session.size = 2
today_open = session.open
today_high = session.high
today_low = session.low
today_cumulative_volume = session.volume
if session.is_ready:
    yesterday_close = session[1].close
```

Increase the size of the `Session` object to store more session data. You can use cached data in the `Session` property instead of requesting historical daily data.

```
equity = self.add_equity("SPY")
# Increase the session lookback to 252 from default (0)
equity.session.size = 252
# Get all closing prices in the session history
closes = [x.close for x in session][::-1]
```

Cast to Other Types

You can cast a `RollingWindow` to a list or a DataFrame. If you cast it to a list, reverse the list so the most recent element is at the last index of the list. This is the order the elements would be in if you added the elements to the list with the `Add` method. To cast a `RollingWindow` to a DataFrame, the `RollingWindow` must contain `Slice`, `Tick`, `QuoteBar`, or `TradeBar` objects. If the `RollingWindow` contains ticks, the ticks must have unique timestamps.

```
closes = list(self._close_price_window)[::-1]  # Make sure to reverse the list.

tick_df = self.pandas_converter.get_data_frame[Tick](list(self._tick_window)[::-1])
trade_bar_df = self.pandas_converter.get_data_frame[TradeBar](list(self._trade_bar_window)[::-1])
quote_bar_df = self.pandas_converter.get_data_frame[QuoteBar](list(self._quote_bar_window)[::-1])
```

PY

Delete Data

To remove all of the elements from a `RollingWindow` , call the `reset` method.

```
self._close_price_window.reset()
```

PY

Examples

The following examples demonstrate some common practices for rolling windows.

Example 1: Price Actions

The following algorithm saves the trailing 3 `TradeBar` objects into a `RollingWindow` . When it identifies a volume contraction breakout price action pattern on the SPY, it buys to ride on the capital inflow. To exit positions, it places a 2% take profit and 1% stop loss order in the `on_order_event` method.

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up a rolling window to hold the last 3 trade bars for price action detection as the trade signal.
        self.windows = RollingWindow(3)
        # Warm up the rolling window.
        history = self.history[TradeBar](self.spy, 3, Resolution.MINUTE)
        for bar in history:
            self.windows.add(bar)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Trade the price action if the previous bars fulfill a contraction breakout.
            if self.contraction_action and self.breakout(bar.close):
                self.set_holdings(self.spy, 0.5)

            # Add the current bar to the window.
            self.windows.add(bar)

    def contraction_action(self) -> None:
        # We trade contraction type price action, where the buying preesure is increasing.
        # 1. The last 3 bars are green.
        # 2. The price is increasing in trend.
        # 3. The trading volume is increasing as well.
        # 4. The range of the bars are decreasing.
        return (
            self.windows[2].close > self.windows[2].open and
            self.windows[1].close > self.windows[1].open and
            self.windows[0].close > self.windows[0].open and
            self.windows[0].close > self.windows[1].close > self.windows[2].close and
            self.windows[0].volume > self.windows[1].volume > self.windows[2].volume and
            self.windows[2].close - self.windows[2].open > self.windows[1].close - self.windows[1].open > self.windows[0].close -
self.windows[0].open
        )

    def breakout(self, current_close: float) -> None:
        # Trade breakout from contraction: the breakout should be much greater than the contracted range of the last bar.
        return current_close - self.windows[0].close > (self.windows[0].close - self.windows[0].open) * 2

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * (0.99 if order_event.fill_quantity > 0 else 1.01)
                self.stop_market_order(self.spy, -self.portfolio[self.spy].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * (1.02 if order_event.fill_quantity > 0 else 0.98)
                self.limit_order(self.spy, -self.portfolio[self.spy].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel open orders if stop loss or take profit order fills.
                self.transactions.cancel_open_orders()
```

Example 2: Bid-Ask Spread

The following algorithm trades the microeconomy of SPY's supply-demand relationship. It buys when the current bid-ask spread is less than average spread over the last 20 QuoteBar objects, indicating demand is approaching supply. When the spread is greater than the average, it shorts SPY. To save the last spread values, it uses a `RollingWindow` .

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1);
        self.set_end_date(2024, 12, 31)

        # Add SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up a rolling window to hold the last 20 bar's bid-ask spread for trade signal generation.
        self.windows = RollingWindow(20)
        # Warm up for the rolling window with quote data.
        history = self.history[QuoteBar](self.spy, 20, Resolution.MINUTE)
        for bar in history:
            self.windows.add(bar.ask.close - bar.bid.close)

    def on_data(self, slice: Slice) -> None:
        bar = slice.quote_bars.get(self.spy)
        if bar:
            # Update the window with the current bid-ask spread.
            spread = bar.ask.close - bar.bid.close
            self.windows.add(spread)

            # Buy if the current spread is smaller than average, indicating demand is approaching supply. Buy force will drive up price.
            if spread < np.mean(list(self.windows)):
                self.set_holdings(self.spy, -0.5)
            # Short if the current spread is larger than average, indicating supply is gradually overwhelming demand.
            elif spread > np.mean(list(self.windows)):
                self.set_holdings(self.spy, 0.5)
```

Example 3: EMA Acceleration

The following algorithm trades SPY based on the direction and acceleration of its exponential moving average (EMA). It buys when the EMA is increasing and accelerating upward, indicating a strong uptrend. It short sells when the EMA is decreasing and accelerating downward, indicating a strong downtrend. To compare the current EMA value to its previous values, the algorithm utilizes the EMA's internal `RollingWindow` .

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data for trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up an automatic EMA indicator for trade signal generation.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Extend the size of the EMA's internal rolling window so we can access its historical values.
        self._ema.window.size = 3
        # Warm-up the indicator and its internal rolling window
        self.indicator_history(self._ema, self.spy, self._ema.window.size)

        # Schedule an event to rebalance SPY position at daily market open.
        self.schedule.on(self.date_rules.every_day(self.spy), self.time_rules.after_market_open(self.spy, 0), self.rebalance)

    def rebalance(self) -> None:
        # Buy if the current EMA is increasing with acceleration, indicating strong uptrend.
        if (self._ema.window[1].value < self._ema.current.value and
            self._ema.window[0].value - self._ema.window[1].value > self._ema.window[1].value - self._ema.window[2].value):
            self.set_holdings(self.spy, 0.5)
        # Short if the current EMA is decreasing with acceleration, indicating strong downtrend.
        elif (self._ema.window[1].value > self._ema.current.value and
            self._ema.window[0].value - self._ema.window[1].value < self._ema.window[1].value - self._ema.window[2].value):
            self.set_holdings(self.spy, -0.5)
        # Liquidate if no strong trend indicated.
        elif self.portfolio.invested:
            self.liquidate(self.spy)
```

Example 4: Market Session as History

Using `Session` instead of history requests or rolling window to cache the last 252 OHLCV bars, the following algorithm buys SPY accordingly when the market opens above the 252-day max of closing prices.

PY

```
class MarketSessionAsDailyHistory(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        equity = self.add_equity("SPY")

        # Increase the session lookback to 252 from default (0)
        # since our strategy break out the 252-day max of closing prices
        equity.session.size = 252
        # Warm up with daily data for speed
        self.set_warm_up(equity.session.size, Resolution.DAILY)

        self.schedule.on(self.date_rules.every_day("SPY"), self.time_rules.after_market_open("SPY", 1), self.trade_break_out)

    def trade_break_out(self):
        '''Break-out strategy. If the market opens above the 252-day max of closing prices,
        allocate all in SPY. Liquidate if it opens below the mean.
        '''
        session = self.securities["SPY"].session
        # The session is ready when the number of samples is greater than the size (252)
        if self.is_warming_up or not session.is_ready:
            return

        # Create a pandas Series of the closing prices of session
        # to use the max and mean methods
        closes = pd.Series([x.close for x in session][::-1])
        _open, _max, _mean = session.open, closes.max(), closes.mean()

        # Invest all in SPY if the open break out the 252-day max and liquidate below the mean
        if _open > _max:
            self.set_holdings("SPY", 1, tag=f'{_open=:.2f}, {_max=:.2f}')
        if _open < _mean:
            self.liquidate("SPY", tag=f'{_open=:.2f}, {_mean=:.2f}')
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
RollingWindowAlgorithm.py Python CustomVolatilityModelAlgorithm.py Python MultipleSymbolConsolidationAlgorithm.py Python

Historical Data

Common Errors

Introduction

Errors can occur when you request historical data. This page explains some of the most common ones you may encounter.

Indicator Readiness

Indicators can't produce a valid value until they have enough historical data to be ready . If your indicators require more data than what is currently available (for example, data before an asset's IPO), the `indicator_history` method will return fewer data points than you request and the warm-up techniques for manual and automatic indicators won't be able to make your indicators ready.

Trading During Warm Up

You can't place trades during the warm-up period because the data feed is replaying historical data for setting algorithm state. If you try to place a trade during warm-up, LEAN throws the following error:

Backtest Handled Error: This operation is not allowed in or during warm up: OrderRequest.Submit. Please move this code to the method.

Numerical Precision

Some factor files have INF split values, which indicate that the stock has so many splits that prices can't be calculated with correct numerical precision. To allow history requests with these symbols, we need to move the starting date forward when reading the data or use raw data normalization . If there are numerical precision errors in the factor files for a security in your history request, LEAN throws the following error:

"Warning: when performing history requests, the start date will be adjusted if there are numerical precision errors in the factor files."

Live Intraday Data Source

In live trading, if you make a history request for minute data at noon and the history period covers the start of the previous day to the present moment, the data from the previous day will be backtest data. The data of the current day will be live data that we collected throughout the morning. If you make this history request in a backtest, you might get slightly different data for the current day because of post-processing from the data vendor.

In cloud deployments, QuantConnect is the default data provider. If you remove it and use a different data provider, you won't get any price data from QuantConnect. However, we provide the following datasets:

- US Equity Security Master
- US Futures Security Master
- US Equity Option Universe
- US Index Option Universe
- Universe selection datasets
- Non-streaming alternative datasets

QC Data Provider Limitations

When you request historical data or run warm-up , the amount of historical data you can access depends on the resolution of your data subscriptions. The following table shows the amount of trailing historical data you can access for each data resolution:

Resolution	Available History
Daily	All historical data
Hour	All historical data
Minute	1 year
Second	2 months
Tick	1 month

In live trading, history requests for Index or Options with the QuantConnect data provider only returns data up to the end of the previous day. For example, if you make a history request for these asset classes at 1 PM Eastern Time (ET), you won't get any data for the current day.

For more information about the QuantConnect data provider, see QuantConnect .

Other Data Provider Limitations

LEAN supports other data providers, including IQFeed and Polygon. For more information about the limitations of these other data providers, see their respective documentation in Cloud Platform , Local Platform , or the CLI .

Brokerage Limitations

LEAN supports several brokerages that can provide historical data. However, some of them have limitations on historical data. For example, Interactive Brokers can be slow, so it can timeout if you request historical data for too many assets at once. For more information about these brokerages and the data they provide, see Datasets .

Historical Data

Live Trading

Introduction

When you make history requests in live trading, be aware of the associated data delays and the data provider's functionality.

Data Delays

Backtests have access to data up to t-1 to t-4 days, where t is the current day. The availability depends on the data type.

In live trading, history requests take 5-10 seconds to return the data. While LEAN processes the request, the algorithm doesn't receive any new data points.

Multiple Data Providers

When you deploy a live algorithm, you can add multiple data provider. If you use multiple data providers, the order you select them in defines their order of precedence in Lean. For example, if you set QuantConnect as the first provider and Interactive Brokers (IB) as the second provider, Lean only uses the IB data provider for securities that aren't available from the QuantConnect data provider. This configuration makes it possible to use the QuantConnect data provider for Equity universe selection and then place Options trades on the securities in the universe.

Data Provider Limitations

History requests from third-party data providers are much slower than from the QuantConnect data provider. If you need data from a third-party data provider, request small amounts or your algorithm may take longer than 10 minutes to process and cause a timeout. For more information about the data providers, see their respective documentation in Cloud Platform , Local Platform , or the CLI .

Trading and Orders

Trading and Orders

Key Concepts

Introduction

LEAN has dozens of methods to create, update, and cancel orders. You can place orders automatically with helper methods or manually through methods on the algorithm API. You can fetch, update, and cancel manual orders with order tickets . As the state of your orders change, LEAN creates events that notify your algorithm.

In backtesting, LEAN simulates order fills with historical data, but you can create your own fill, fee, slippage, and margin models via plugin points. You control how optimistic or pessimistic order fills are with transaction model classes. For more information about these types of models, see Reality Modeling .

Orders and Tickets

When you call one of the order methods to place an order, LEAN performs pre-order checks to ensure that the order meets some requirements and uses the last known price to check you have enough capital . To see the requirements of each order, see the Requirements section of the documentation for the order types and the Orders section of your brokerage model . If you can place the order, LEAN creates `Order` and OrderTicket objects. The order ticket is sent to your brokerage. As the brokerage processes your order, it returns another order ticket and it's compared against the order to see if the order is satisfied. Orders fill asynchronously in live trading, so if you want to change an order, you must request it with the order ticket. Order changes are not guaranteed since your order may fill by the time brokerage receives the request.

Order Types

LEAN supports the following order types:

- Market
- Limit
- Limit if touched
- Stop market
- Stop limit
- Trailing stop
- Market on open
- Market on close
- Combo market
- Combo limit
- Combo leg limit
- Option exercise

Symbol Properties

The `symbol_properties` are a property of the `Security` object. LEAN uses some of the `symbol_properties` to prevent invalid orders, and to calculate order quantities for a given target.

`SymbolProperties` objects have the following properties:

To get the `symbol_properties` , use the property on the `Security` object.

```py
symbol_properties = self.securities["BTCUSD"].symbol_properties
lot_size = symbol_properties.lot_size
minimum_order_size = symbol_properties.minimum_order_size
minimum_price_variation = symbol_properties.minimum_price_variation
```

LEAN uses the `minimum_price_variation` to round the `limit_price` , `stop_price` , and the `trigger_price` .

Quote Currency

The quote currency is the currency you must give the seller to buy an asset. For currency trades, the quote currency is the currency ticker on the right side of the currency pair. For other types of assets, the quote currency is usually USD, but the quote currency for India Equities is INR. To get the quote currency of a `Security` , use the `quote_currency` property.

```py
aapl_quote_currency = self.securities["AAPL"].quote_currency # USD
btcusdt_quote_currency = self.securities["BTCUSDT"].quote_currency # USDT
```

The `quote_currency` is a `Cash` object, which have the following properties:

You can use the `conversion_rate` property to calculate the value of the minimum price movement in the account currency

```py
cfd = self.securities["SG30SGD"]
quote_currency = cfd.quote_currency # SGD
contract_mutiplier = cfd.symbol_properties.contract_multiplier
minimum_price_variation = cfd.symbol_properties.minimum_price_variation

# Value of a pip in account currency
pip = minimum_price_variation * contract_mutiplier * quote_currency.conversion_rate
```

Trade Models

Reality models make backtests as realistic as possible to how the strategy would perform in live trading.

Split Adjustment

If a split event occurs before your order is filled, the unfilled portion of the order is adjusted automatically, where its quantity is multiplied by the split factor and the limit/stop/trigger price (if any) is divided by the split factor.

Open Order Margin Requirements

When an order fills, LEAN uses the portfolio information to check if you have enough capital to maintain the remaining open orders. If you don't, they are canceled.

The following table shows the order events of three orders of SPY 240109C00470000:

Time	Id	Type	Price	Quantity	Status
2024-01-08 09:31	1	Buy Market	0.91	20	`FILLED`
	2	Sell Stop Market	0.81	-20	`SUBMITTED`
2024-01-08 10:31	3	Sell Market	1.50	-10	`FILLED`
	2	Sell Stop Market	0.81	-20	`INVALID`

When the sell market order fills at 10:31, LEAN checks the sell stop order and sets its status to `OrderStatus.INVALID` due to insufficient buying power to complete it.

Asynchronous Execution

In live trading, we send your orders to the API of your brokerage and wait for their response to update the status of the order ticket from `NEW` to `SUBMITTED` , `CANCELED` , or `INVALID` . When you place multiple orders, waiting for responses from the brokerage delays the last orders of the batch. To avoid this delay, you can place orders asynchronously. When you place an order asynchronously, LEAN sends the order to your brokerage and

immediately continues executing your algorithm without waiting for a response. When the response from the brokerage arrives, LEAN updates the status of the order ticket accordingly, and your algorithm receives an order event .

Live Trading Considerations

In live trading, orders fill asynchronously. We send your order to the API of your brokerage and wait for their response to update the state of your algorithm and portfolio. The timing of live order fills doesn't depend on the resolution of your security subscriptions. When your order fills, the fill price and fee is set by your brokerage. You can add event handlers to your algorithm to monitor the brokerage connection and brokerage messages .

In backtesting, the trade fill timing depends on the resolution of your security subscription. For example, if you subscribe to a security with minute resolution data, the algorithm only receives data in one-minute time slices . As a result, the fill model can only evaluate if the order should fill on a minute-by-minute frequency.

Trading and Orders

Order Management

Order Management

Order Tickets

Introduction

When you create an order, you get an `OrderTicket` object to manage your order.

Properties

`OrderTicket` objects have the following attributes:

Track Orders

As the state of your order updates over time, your order ticket automatically updates. To track an order, you can check any of the preceding order ticket properties.

To get an order field, call the `get` method with an `OrderField` .

```py
def initialize(self) -> None:
    self._symbol = self.add_equity("SPY").symbol
    self._ticket = None

def on_data(self, slice: Slice) -> None:
    # Place order if not invested and save the order ticket for later retrival.
    if not self.portfolio.invested and self._symbol in slice.bars:
        self._ticket = self.limit_order(self._symbol, 10, slice.bars[self._symbol].close)
    # Get the limit price if the order is filled.
    elif self.portfolio.invested:
        limit_price = self._ticket.get(OrderField.LIMIT_PRICE)
```

The `OrderField` enumeration has the following members:

In addition to using order tickets to track orders, you can receive order events through the `on_order_event` event handler.

Update Orders

To update an order, use its `OrderTicket` . You can update other orders until they are filled or the brokerage prevents modifications. You just can't update orders during warm up and initialization .

Updatable Properties

The specific properties you can update depends on the order type. The following table shows the properties you can update for each order type.

Order Type	Updatable Properties				
	Tag	Quantity	LimitPrice	TriggerPrice	StopPrice
Market Order					
Limit Order	✓	✓	✓		
Limit If Touched Order	✓	✓	✓	✓	
Stop Market Order	✓	✓			✓
Stop Limit Order	✓	✓	✓		✓
Market On Open Order	✓	✓			
Market On Close Order	✓	✓			

Update Methods

To update an order, pass an `UpdateOrderFields` object to the `update` method. The method returns an `OrderResponse` to signal the success or failure of the update request.

```py
def initialize(self) -> None:
    self._symbol = self.add_equity("SPY").symbol
    self._ticket = None

def on_data(self, slice: Slice) -> None:
    # Place order if not invested and save the order ticket for later retrival.
    if not self._ticket and self._symbol in slice.bars:
        self._ticket = self.limit_order("SPY", 100, slice.bars[self._symbol].close * 0.98)
    # If order is placed, update the limit price to be 90% of the orginal.
    elif self._ticket != None and self._ticket.status == OrderStatus.SUBMITTED:
        # Update the order tag and limit price
        update_settings = UpdateOrderFields()
        update_settings.limit_price = self._ticket.get(OrderField.LIMIT_PRICE) * 0.9
        update_settings.tag = "Limit Price Updated for SPY Trade"
        response = self._ticket.update(update_settings)

        # Check if the update request is successfully submitted to the broker.
        # Note that it may not represent the order is updated successfully: during the order updating process, it may be filled or
canceled.
        if response.is_success:
            self.debug("Order update request is submitted successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_stop_price`
- `update_tag`

```py
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_stop_price(stop_price, tag)

response = ticket.update_tag(tag)
```

Update Order Requests

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Workaround for Brokerages That Don't Support Updates

Not all brokerages fully support order updates. To check what functionality your brokerage supports for order updates, see the Orders section of the documentation for your brokerage model . If your brokerage doesn't support order updates and you want to update an order, cancel the order . When you get an order event that confirms the order is no longer active, place a new order.

```
def initialize(self) -> None:
    self._symbol = self.add_equity("SPY").symbol
    self._ticket = None

def on_data(self, slice: Slice) -> None:
    # Place order if not invested and save the order ticket for later retrival.
    if not self._ticket and self._symbol in slice.bars:
        self._ticket = self.limit_order("SPY", 100, slice.bars[self._symbol].close)
    # If order is placed, cancel the order and place a new one as substituent.
    elif self._ticket != None and self._ticket.status == OrderStatus.SUBMITTED:
        self._ticket.cancel()

def on_order_event(self, order_event: OrderEvent) -> None:
    if (self._ticket and order_event.order_id == self._ticket.order_id and
        order_event.status == OrderStatus.CANCELED):
        # Place a new order
        quantity = self._ticket.quantity - self._ticket.quantity_filled
        limit_price = self.securities[self._ticket.symbol].price + 1
        self._ticket = self.limit_order(self._ticket.symbol, quantity, limit_price)
```

Cancel Orders

To cancel an order, call the `cancel` method on the `OrderTicket` . The method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
def initialize(self) -> None:
    self._symbol = self.add_equity("SPY").symbol
    self._ticket = None

def on_data(self, slice: Slice) -> None:
    # Place order if not invested and save the order ticket for later retrival.
    if not self._ticket and self._symbol in slice.bars:
        self._ticket = self.limit_order("SPY", 100, slice.bars[self._symbol].close)
    # If order is placed, cancel the order if it is not filled within 2 minutes.
    elif self._ticket != None and self._ticket.time < slice.time - timedelta(minutes=2) and self._ticket.status == OrderStatus.SUBMITTED:
        response = self._ticket.cancel("Canceled SPY trade")

        # Check if the cancel request is successfully submitted to the broker.
        # Note that it may not represent the order is canceled successfully: during the order updating process, it may be filled.
        if response.is_success:
            self.debug("Order cancel request successfully submitted")
```

You can't cancel market orders because they are immediately transmitted to the brokerage. You also can't cancel any orders during warm up and initialization .

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Order Response

When you update or cancel an order, LEAN returns an `OrderReponse` object, which have the following attributes:

If your order changes fail, check the `ErrorCode` or `ErrorMessage` . For more information about specific order errors, see the Order Response Error Reference .

To get most recent order response, call the `get_most_recent_order_response` method.

```
response = ticket.get_most_recent_order_response()
```

Examples

The following examples demonstrate some common practices for order ticket management.

Example 1: Timed Exit

The following algorithm is an EMA-crossover strategy. Using the filled time of the order ticket, we can control to exit the position after 45 minutes.

```
class OrderTicketExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._symbol = self.add_equity("SPY").symbol
        self._ticket = None
        # Set up EMA indicator for trade signal generation.
        self._ema = self.ema("SPY", 20, Resolution.DAILY)
        # Warm up the indicator for its readiness to use immediately.
        self.warm_up_indicator(self._symbol, self._ema)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        # Place order if not order yet and save the order ticket for later retrival.
        if not self._ticket and bar:
            # EMA crossover: buy if price is above EMA, which indicate uptrend.
            if self._ema.current.value < bar.close:
                self._ticket = self.limit_order("SPY", 100, slice.bars[self._symbol].close * 0.995)
            # EMA crossover: sell if price is below EMA, which indicate down trend.
            else:
                self._ticket = self.limit_order("SPY", -100, slice.bars[self._symbol].close * 1.005)
        # Exit position if it is in the portfolio for more than 45 minutes.
        elif self._ticket and self._ticket.time + timedelta(minutes=45) < self.utc_time:
            self.liquidate()
            self._ticket = None
```

Example 2: Intraday Stop Loss

The following algorithm demonstrates an intraday long position with a 10% stop loss order. Before the market closes, cancel the stop loss order if it is not filled and exit the position at market close with a market on close order.

```
class OrderTicketExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self._symbol = self.add_equity("QQQ").symbol
        self._ticket = None
        self._day = -1

        # Schedule an event to handle exiting position before market close if stop loss is not hit.
        # Market on close order requires ordering 15 minutes before market close.
        self.schedule.on(
            self.date_rules.every_day(self._symbol),
            self.time_rules.before_market_close(self._symbol, 16),
            self.exit_position
        )

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        # Place order if not invested. Make sure only trade once a day to avoid chasing the loss.
        if not self.portfolio.Invested and self._day != slice.time.day and bar:
            self.market_order(self._symbol, 10)
            # Update the day variable to avoid repeat ordering.
            self._day = slice.time.day

    def on_order_event(self, order_event: OrderEvent) -> None:
        # Make sure to order the stop loss order on the filled market order only.
        if order_event.status == OrderStatus.FILLED and order_event.fill_quantity > 0:
            # Place a 10% stop loss order to avoid large loss. Save the order ticket for tracking its status later.
            self._ticket = self.stop_market_order(order_event.symbol, -order_event.fill_quantity, order_event.fill_price*0.9)

    def exit_position(self) -> None:
        # Only need to cancel the order and handle exit position if the stop loss order is not fully filled.
        if self._ticket and self._ticket.status != OrderStatus.FILLED:
            self._ticket.cancel()
            # Market on close order to exit position at market close with the remaining quantity.
            self.market_on_close_order(self._symbol, -10 - self._ticket.quantity_filled)
```

Example 3: Crypto Trailing Stop Order

This example trades BTCUSD with a similar logic of Example 2, but apply a trailing stop loss of 10% instead. Crypto exchanges does not support trailing stop loss order. Yet, we can cancel and reorder a stop loss order with the updated stop loss price. If the market price of the crypto pair is above the previous level, update the stop loss order by the above.

```py
class OrderTicketExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Set the market argument to trade in specific exchange.
        self._symbol = self.add_crypto("BTCUSD", market=Market.COINBASE).symbol
        # A variable to hold the stop loss order to track if it should be updated.
        self._ticket = None
        # A day variable to avoid over-ordering (trade once per day)
        self._day = -1
        # A variable to hold the price of the high price in the lifetime of the open position for updating the stop loss price.
        self.high_price = 0
        # Update the stop loss when new high is 1% higher
        self.update_step = 1.01

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if not bar: return
        # Place order if not invested. Make sure only trade once a day to avoid chasing the loss.
        if not self.portfolio.invested and self._day != self.time.day:
            self.market_order(self._symbol, 1)
            # Place a 10% stop loss order to avoid large loss. Save the order ticket for tracking its status later.
            self._ticket = self.stop_market_order(self._symbol, -1, bar.close*0.9)
            self.high_price = bar.high * self.update_step
            # Update the day variable to avoid repeat ordering.
            self._day = self.time.day

        # Update the trailing stop order. Since order updating is not supported in crypto exchanges, we cancel and reorder.
        # Trailing stop price will only need to be updated when the market price is higher than the current price.
        if self._ticket and self._ticket.status in [OrderStatus.SUBMITTED, OrderStatus.PARTIALLY_FILLED] and bar.high > self.high_price:
            self._ticket.cancel()
            self._ticket = self.stop_market_order(self._symbol, -self.portfolio[self._symbol].quantity, bar.high*0.9)
            self.high_price = bar.high * self.update_step
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
OrderTicketDemoAlgorithm.py Python

Order Management

Transaction Manager

Introduction

The algorithm transactions manager (`SecurityTransactionManager`) contains a collection of helper methods to quickly access all your orders. To access the transaction manager, use the `transactions` property of your algorithm. If you save a reference to your order tickets , you shouldn't need to use the transaction manager. LEAN updates the order tickets as the brokerage processes your orders.

Get a Single Order Ticket

If you didn't save a reference to the order ticket when you created an order, you can call the `get_order_ticket` method to get it. You need to pass the order ID to the method. If you don't have the order ID, you can use the `last_order_id` property to get the order ID of the most recent order.

```
PY
order_id = self.transactions.last_order_id
ticket = self.transactions.get_order_ticket(order_id)
```

Get Order Tickets

To get order tickets, call the `get_order_tickets` or `get_open_order_tickets` method. You can pass in a filter function to filter all of the order tickets or pass a `Symbol` to get the order tickets for a specific asset.

```
PY
# Get all order tickets
order_tickets = self.transactions.get_order_tickets()

# Get order tickets that pass a filter
filtered_order_tickets = self.transactions.get_order_tickets(lambda order_ticket: order_ticket.symbol == symbol)

# Get all open order tickets
open_order_tickets = self.transactions.get_open_order_tickets()

# Get all open order tickets for a symbol
symbol_open_order_tickets = self.transactions.get_open_order_tickets(symbol)

# Get open order tickets that pass a filter
filtered_open_order_tickets = self.transactions.get_open_order_tickets(lambda order_ticket: order_ticket.quantity > 10)
```

Get a Single Order

To get a clone of a specific order, call the `get_order_by_id` method with the order Id. To get the order Id, use the `order_id` property of the order ticket or use the `LastOrderID` property if you want the most recent order.

```
PY
order_id = self.transactions.last_order_id
order = self.transactions.get_order_by_id(order_id)
```

Order objects are immutable and changes to the order object will not impact the trade. To make an update to an order you must use Order Tickets . `Order` objects have the following attributes:

Get Orders

To get a list of orders, call the `get_orders` , `get_open_orders` , or `get_orders_by_brokerage_id` method. These method returns a list of `Order` objects.

```py
# Get all completed orders
completed_Orders = self.transactions.get_orders()

# Get all completed orders that pass a filter
filtered_completed_orders = self.transactions.get_orders(lambda x: x.quantity > 10)

# Retrieve a list of all completed orders for a symbol
symbol_completed_orders = self.transactions.get_orders(lambda x: x.symbol == symbol)

# Get all open orders
open_orders = self.transactions.get_open_orders()

# Get all open orders that pass a filter
filtered_open_orders = self.transactions.get_open_orders(lambda x: x.quantity > 10)

# Retrieve a list of all open orders for a symbol
symbol_open_orders = self.transactions.get_open_orders(symbol)

# Get all open and completed orders that correspond to an Id that the brokerage assigned in live trading
orders_by_brokerage_id = self.transactions.get_orders_by_brokerage_id(brokerageId)
```

`Order` objects have the following attributes:

The `orders_count` property gets the current number of orders that have been processed.

Get Remaining Order Quantity

To get the unfilled quantity of open orders, call the `get_open_orders_remaining_quantity` method.

```py
# Get the quantity of all open orders
all_open_quantity = self.transactions.get_open_orders_remaining_quantity()

# Get the quantity of open orders that pass a filter
filtered_open_quantity = self.transactions.get_open_orders_remaining_quantity(
    lambda order_ticket: order_ticket.quantity > 10
)

# Get the quantity of open orders for a symbol
symbol_open_quantity = self.transactions.get_open_orders_remaining_quantity(symbol)
```

Cancel Orders

To cancel open orders, call the `cancel_open_orders` method. This method returns a list of `OrderTicket` objects that correspond to the canceled orders.

```py
 # Cancel all open orders
all_cancelled_orders = self.transactions.cancel_open_orders()

# Cancel orders related to IBM and apply a tag
ibm_cancelled_orders = self.transactions.cancel_open_orders("IBM", "Hit stop price")
```

Live Trading Considerations

LEAN doesn't save orders and order tickets across live deployments.

When you deploy your algorithm, LEAN fetches the open orders from your brokerage and populate the algorithm transactions manager (`SecurityTransactionManager`).

Examples

The following examples demonstrate some common practices for using the transaction manager.

Example 1: 2-Level Take Profit And Stop Loss

The following algorithm trades volatility using the CBOE VIX dataset . It makes use of the transaction manager to handle a 2-layer take profit and stop loss risk management.

PY

```
class TransactionManagerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request SPY data for trading and trade on daily signals so we use daily resolution.
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol
        #Subscribe to VIX data to generate trade signals.
        self.vix = self.add_data(CBOE, "VIX", Resolution.DAILY).symbol

        # Use the EMA indicator to generate trade signals from VIX.
        self._ema = self.ema(self.vix, 20, Resolution.DAILY)
        # Warm up the EMA indicator to enable immediate use.
        self.warm_up_indicator(self.vix, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        spy_bar = slice.bars.get(self.spy)
        if spy_bar:
            vix = self.securities[self.vix].price
            ema = self._ema.current.value

            # Trade based on volatility. Sell SPY if VIX is in an uptrend, indicating low market volatility.
            # Buy SPY if VIX is in a downtrend, indicating a volatile market.
            if vix > ema and not self.portfolio[self.spy].is_long:
                # Cancel all open orders to remove redundant take profit and stop loss orders.
                self.transactions.cancel_open_orders()

                quantity = 10 - self.portfolio[self.spy].quantity
                self.market_order(self.spy, quantity)
                # Stop Loss at 2%.
                self.stop_market_order(self.spy, -10, spy_bar.close * 0.98, tag="stop loss")
                # Take profit at 4%.
                self.limit_order(self.spy, -5, spy_bar.close * 1.04, tag="take profit")
            elif vix < ema and not self.portfolio[self.spy].is_short:
                # Cancel all open orders to remove redundant take profit and stop loss orders.
                self.transactions.cancel_open_orders()

                quantity = -10 - self.portfolio[self.spy].quantity
                self.market_order(self.spy, quantity)
                # Stop Loss at 2%.
                self.stop_market_order(self.spy, 10, spy_bar.close * 1.02, tag="stop loss")
                # Take profit at 4%.
                self.limit_order(self.spy, 5, spy_bar.close * 0.96, tag="take profit")

    def on_order_event(self, order_event: OrderEvent) -> None:
        # If the take profit order was filled out, we place the 2nd layer of take profit.
        take_profit_orders = list(self.transactions.get_order_tickets(lambda x: x.order_type == OrderType.LIMIT))
        if len(take_profit_orders) == 0:
            return
        last_take_profit_order_id = sorted(take_profit_orders, key=lambda x: x.time)[-1].order_id

        if order_event.status == OrderStatus.FILLED and order_event.id == last_take_profit_order_id:
            # Cancel all open orders to remove redundant stop loss orders.
            self.transactions.cancel_open_orders()
            # 2nd layer Take profit at extra 4%.
            take_profit_price = order_event.fill_price * (0.96 if order_event.fill_quantity > 0 else 1.04)
            self.limit_order(self.spy, 5, take_profit_price, tag="second-layer take profit")
        elif order_event.status == OrderStatus.FILLED and order_event.ticket.order_type == OrderType.STOP_MARKET:
            # Cancel all open orders to remove redundant take profit orders if stop loss.
            self.transactions.cancel_open_orders()
```

Trading and Orders

Order Types

You can place any of the following order types that your brokerage supports. Click one to learn more.

Market Orders

Limit Orders

Limit if Touched Orders

Stop Market Orders

Stop Limit Orders

Trailing Stop Orders

Market On Open Orders

Market On Close Orders

Combo Market Orders

Combo Limit Orders

Combo Leg Limit Orders

Option Exercise Orders

Other Order Types

See Also

Brokerages

Order Types

Market Orders

Introduction

Market orders are instructions to trade a specific number of units, regardless of the fill price. If your highest priority is to fill an order as fast as possible, use market orders. If the order book of the security you're trading has sufficient liquidity when the brokerage receives your order, it immediately fills. However, if there is not enough liquidity by the brokerage, you may need to wait a few minutes to fill the order, you may get partial fills, and you may incur additional costs from market impact.

Place Orders

To send a market order, call the `market_order` , `buy` , `sell` , or `order` method and provide a `Symbol` and quantity. If you don't have sufficient capital for the order, it's rejected. By default, market orders are synchronous and fill immediately.

```py
# Buy orders
self.market_order("IBM", 100)
self.buy("AAPL", 10)
self.order("SPY", 20)

# Sell orders
self.market_order("AMZN", -15)
self.sell("TSLA", 25)
self.order("SPY", -20)
```

You can also provide a tag and order properties to the `order` and `market_order` methods.

```py
self.market_order(symbol, quantity, tag=tag, order_properties=order_properties)
```

If you place a market order during pre-market or post-market hours, LEAN converts your order to market on open order. To override this behavior, create a custom fill model . To be notified when the default fill model allows you to submit market orders outside of regular trading hours, subscribe to GitHub Issue #3947 .

Monitor Order Fills

If the brokerage has sufficient liquidity in their order book, market orders fill immediately. Otherwise, you get partial fills. To monitor the fills of your order, save a reference to the order ticket .

```py
ticket = self.market_order("XLK", 10)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Cancel Orders

To cancel a market order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```py
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.market_order("IBM", 100, asynchronous=True);
```

Synchronous Timeouts

Market orders are synchronous by default, so your algorithm waits for the order to fill before moving to the next line of code. If your order takes longer than five seconds to fill, your algorithm continues executing even if the trade isn't filled. To adjust the timeout period, set the `transactions.market_order_fill_timeout` property.

```
 # Adjust the market fill-timeout to 30 seconds.
self.transactions.market_order_fill_timeout = timedelta(seconds=30)
```

Market orders may take a few minutes to fill for illiquid assets such as out-of-the-money Options or penny stocks.

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with market orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for market orders, see the Orders section of the brokerage model documentation .

Requirements

You can submit market orders during regular trading hours for all asset classes. For Futures and Future Options, you can place market orders during regular and extended market hours. For all other asset classes, if you place market orders at/after the last minute of regular market hours, LEAN converts the market orders into market-on-open orders as long as the asset class supports them . To view the trading hours of each asset class, follow these steps:

1. Open the Asset Classes documentation.
2. Click an asset class.
3. Click Market Hours .

Examples

The following backtest verifies the `market_order` behavior. The algorithm buys SPY on the first day and liquidates the position on the second day. The following table shows the first two trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T09:31:00Z	SPY	429.11	10	Market	Filled	4291.10	
2021-07-02T09:31:00Z	SPY	431.54	-10	Market	Filled	-4315.40	

On July 1, 2021, the algorithm buys SPY at $429.11. The fill model fills this order at the ask close price.

On July 2, 2021, the algorithm sells the SPY holdings at $431.54. The fill model fills this order at the bid close price.

To reproduce these results, backtest the following algorithm:

PY

```
class MarketOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 4)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if not self.portfolio.invested and self.time.day == 1:
            self.market_order("SPY", 10)

        elif self.portfolio.invested and self.time.day != 1:
            self.market_order("SPY", -10)
```

Order Types

Limit Orders

Introduction

Limit orders are instructions to trade a quantity of an asset at a specific price or a better price. A marketable limit order has a limit price that crosses the spread. An unmarketable limit order has a limit price that doesn't cross the spread. For example, if an asset has an ask price of 100 and you place a limit order to buy at 100 or higher, that's a marketable limit order that should immediately fill. If you place a limit order to buy at $99.99 or lower, that's an unmarketable limit order that should go on the order book. Unmarketable limit orders save you from paying spread costs. Some brokerages even charge you a lower transaction fee if you use unmarketable limit orders rather than marketable limit orders.

Limit orders are helpful in illiquid markets. You can use them to get a good entry price or to set a take-profit level on an existing holding. However, if the market trades away from your limit price, you may have to adjust your limit price or wait for the market to trade back to your limit price to fill the order.

Place Orders

To send a limit order, call the `limit_order` method with a `Symbol` , quantity, and limit price. You can also provide a tag and order properties to the `limit_order` method. If you do not have sufficient capital for the order, it's rejected.

```py
self.limit_order(symbol, quantity, limit_price, tag, order_properties)
```

To buy an asset with a marketable limit order, set the limit price to the current ask price or higher.

To sell an asset with a marketable limit order, set the limit price to the current bid price or lower.

To buy an asset with an unmarketable limit order, set the limit price below the current ask price.

```py
# Buy 1 Bitcoin when the price drops to $34,000
self.limit_order("BTCUSD", 1, 34000)
```

To sell an asset with an unmarketable limit order, set the limit price above the current bid price.

```
# Sell 1 Bitcoin when the price moves up to $60,000
self.limit_order("BTCUSD", -1, 60000)
```

Monitor Order Fills

Limit orders fill when the security price passes the limit price. To monitor the fills of your order, save a reference to the order ticket .

```
# Buy 10 shares of XLK at $140
ticket = self.limit_order("XLK", 10, 140)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, limit price, and tag of limit orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order ticket
ticket = self.limit_order("SPY", 100, 221.05, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = 80
update_settings.limit_price = 222.00
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_tag`

```
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a limit order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.limit_order("SPY", 10, 429.0, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with limit orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for limit orders, see the Orders section of the brokerage model documentation .

Requirements

Limit orders can be submitted at any time for all security types.

If your algorithm subscribes to extended market hours, they can be filled outside regular trading hours.

Examples

The following backtest verifies the `limit_order` method behavior. On the first day, the algorithm buys SPY with an unmarketable limit order. On the second day, the algorithm sells SPY with an unmarketable limit order and places another unmarketable limit order to buy SPY, which doesn't fill. The following table shows the first three trades in the backtest:

Submitted Time	Filled Time	Symbol	Limit Price	Filled Price	Quantity	Type	Status	Value	Tag
2021-07-01T09:31:00Z	2021-07-01T09:32:00Z	SPY	429.00	429.00	10	Limit	Filled	4290.00	Limit Price: ¤429.00
2021-07-02T09:31:00Z	2021-07-02T09:35:00Z	SPY	431.70	431.70	-10	Limit	Filled	-4317.00	Limit Price: ¤431.70
2021-07-02T09:31:00Z	/	SPY	400.00	/	10	Limit	Submitted	/	Limit Price: ¤400.00

On July 1, 2021 at 9:31 AM Eastern Time (ET), the algorithm places a buy limit order with a limit price of 429 when the ask low price is 428.81 and the ask high price is 429.15. The order fills at 9:32 AM ET at a price of 429. The fill model fills buy limit orders when the ask low price is less than the limit price. It sets the fill price of the order to the minimum of ask high price and the limit price.

On July 2, 2021 at 9:31 AM ET, the algorithm places a sell limit order at 431.70 and a buy limit order at 400. At the time of the order, the bid high price is 431.65, the bid low price is 431.49, and the ask low price is 431.50. The sell limit order fills at 9:35 AM ET at a price of 431.70 and the buy limit order doesn't fill. The fill model fills sell limit order when the bid high price is greater than the limit price. It sets the fill price of the order to the maximum of the bid low price and the limit price. The buy limit order doesn't fill because the ask low price is above the limit price for the remainder of the backtest period.

To reproduce these results, backtest the following algorithm:

PY

```
class LimitOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if self.time.day == 1 and self.time.hour == 9 and self.time.minute == 31:
            self.limit_order("SPY", 10, 429.0)

        elif self.time.day == 2 and self.time.hour == 9 and self.time.minute == 31:
            self.limit_order("SPY", -10, 431.7)
            self.limit_order("SPY", 10, 400)
```

Order Types

Limit if Touched Orders

Introduction

Limit if touched (LIT) orders are instructions to place a limit order once an asset touches a specific price level. A buy LIT order has a trigger price below the current market price and a sell LIT order has a trigger price above the current market price. In effect, LIT orders are the opposite of stop limit orders .

Place Orders

To send a LIT order, call the `limit_if_touched_order` method and provide a `Symbol` , quantity, trigger price, and limit price. You can also provide a tag and order properties to the `limit_if_touched_order` method. If you do not have sufficient capital for the order, it's rejected.

```
self.limit_if_touched_order(symbol, quantity, trigger_price, limit_price, tag, order_properties)
```

To buy an asset with a LIT order that has a marketable limit price, set the trigger price below the current market price and set the limit price above the trigger price.

```
# Once Bitcoin trades down to $36,000, place a limit order to buy 1 Bitcoin at $38,000
self.limit_if_touched_order("BTCUSD", 1, 33000, 36000)
```

To sell an asset with a LIT order that has a marketable limit price, set the trigger price above the current market price and set the limit price below the trigger price.

```
# Once Bitcoin trades up to $62,000, place a limit order to sell 1 Bitcoin at $59,000
self.limit_if_touched_order("BTCUSD", -1, 62000, 59000)
```

To buy an asset with a LIT order that has an unmarketable limit price, set the trigger price below the current market price and set the limit price below the trigger price.

```py
# Once Bitcoin trades down to $46,000, place a limit order to buy 1 Bitcoin at $36,000
self.limit_if_touched_order("BTCUSD", 1, 46000, 36000)
```

To sell an asset with a LIT order that has an unmarketable limit price, set the trigger price above the current market price and set the limit price above the trigger price.

```py
# Once Bitcoin trades up to $54,000, place a limit order to sell 1 Bitcoin at $62,000
self.limit_if_touched_order("BTCUSD", -1, 54000, 62000)
```

Monitor Order Fills

LIT orders fill when the security price touches the trigger price and is at least as favorable as the limit price. To monitor the fills of your order, save a reference to the order ticket .

```
# Once SPY trades down to $425, place a limit order to buy 10 shares at $415
ticket = self.limit_if_touched_order("SPY", 10, 425, 415)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, trigger price, limit price, and tag of LIT orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order ticket
ticket = self.limit_if_touched_order("SPY", 100, 350, 340, tag="Original tag")

# Update the order
update_order_fields = UpdateOrderFields()
update_order_fields.quantity = 80
update_order_fields.trigger_price = 380
update_order_fields.limit_price = 370
update_order_fields.tag = "New tag"
response = ticket.update(update_settings)

# Check the OrderResponse
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_trigger_price`
- `update_tag`

```
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_trigger_price(trigger_price, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a LIT order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.limit_if_touched_order("SPY", 10, 429.0, 428.95, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with LIT orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for LIT orders, see the Orders section of the brokerage model documentation .

Requirements

LIT orders can be submitted at any time for all security types.

If your algorithm subscribes to extended market hours, they can be filled outside regular trading hours.

Examples

The following backtest verifies the `limit_if_touched_order` behavior. The following table shows the trades in the backtest:

Submitted Time	Filled Time	Symbol	Trigger Price	Limit Price	Filled Price	Quantity	Type	Status	Value	Tag
2021-07-01T09:31:00Z	2021-07-01T09:38:00Z	SPY	429.00	428.95	428.95	10	Limit if touched	Filled	4285.00	Trigger Price: ¤429.00 Limit Price: ¤428.95
2021-07-02T09:31:00Z	2021-07-02T09:49:00Z	SPY	431.70	431.75	431.84	-10	Limit if touched	Filled	-4318.40	Trigger Price: ¤431.70 Limit Price: ¤431.75
2021-07-02T09:31:00Z	/	SPY	431.70	400.00	/	10	Limit if touched	Submitted	/	Trigger Price: ¤431.70 Limit Price: ¤400.00
/	/	SPY	400.00	400.00	/	10	Limit if touched	Submitted	/	Trigger Price: ¤400.00 Limit Price: ¤400.00

On July 1, 2021 at 9:31 AM Eastern Time (ET), the algorithm places a LIT order to buy SPY with a trigger price of 429 and a limit price of 428.95. At the time of the order submission, the low price was 428.80 and the bid close price was 429.10. The order fills at 9:38 AM ET at a price of $428.95. The fill model sets the limit price for buy orders when the low price is less than or equal to the trigger price, fills the order when the bid close price is less than or equal to the limit price, and sets the fill price to the minimum of the ask price and the limit price.

On July 2, 2021 at 9:31 AM ET, the algorithm places a LIT order to sell SPY with a trigger price of 431.70 and a limit price of 431.75. At the time of the order submission, the high price was 431.78 and the ask close price was 431.55. The order fills at 9:49 AM ET at a price of $431.84. The fill model sets the limit price for sell orders when the high price is greater than or equal to the trigger price, fills the order when the ask close price is greater than or equal to the limit price, and sets the fill price to the maximum of the bid price and the limit price.

On July 2, 2021, the algorithm places a third LIT order to buy SPY with a trigger price of 431.70 and a limit price of 400. The fill model sets the fill price, but it doesn't fill the order because the limit price is too low. On the same day, the algorithm places a fourth LIT order to buy SPY with a trigger price of 400 and a limit price of 400. The fill model doesn't set the limit price for this order because SPY doesn't touch the trigger price before the backtest ends.

To reproduce these results, backtest the following algorithm:

```
class LimitIfTouchedOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 4)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if self.time.day == 1 and self.time.hour == 9 and self.time.minute == 31:
            self.limit_if_touched_order("SPY", 10, 429.0, 428.95)

        elif self.time.day == 2 and self.time.hour == 9 and self.time.minute == 31:
            self.limit_if_touched_order("SPY", -10, 431.7, 431.75)
            self.limit_if_touched_order("SPY", 10, 431.7, 400)
            self.limit_if_touched_order("SPY", 10, 400, 400)
```

Order Types

Stop Market Orders

Introduction

Stop market orders fill as a market order when the asset reaches a specific price. A buy stop market order triggers when the asset price is greater than or equal to the stop price. A sell stop market order triggers when the asset price is less than or equal to the stop price. You can use stop market orders to buy breakouts or to mitigate the risk of large losses. However, if the market gaps past your stop price, the order may fill at a worse price than the stop price you set. There is no guarantee that a stop market order fills at the stop price.

Place Orders

To send a stop market order, call the `stop_market_order` method and provide a `Symbol` , quantity, and stop price. You can also provide a tag and order properties to the `stop_market_order` method. If you do not have sufficient capital for the order, it is rejected.

```py
self.stop_market_order(symbol, quantity, stop_price, tag, order_properties)
```

To buy an asset with a stop market order, set the stop price above the current ask price.

```py
# When Bitcoin trades up to $60,000, buy 1 Bitcoin
self.stop_market_order("BTCUSD", 1, 60000)
```

To sell an asset with a stop market order, set the stop price below the current bid price.

```py
# When Bitcoin trades down to $43,000, sell 1 Bitcoin
self.stop_market_order("BTCUSD", -1, 43000)
```

Monitor Order Fills

Stop market orders fill as a market order when the security price passes the stop price. To monitor the fills of your order, save a reference to the order ticket .

```py
# When XLK trades down to $130, sell 10 shares
ticket = self.stop_market_order("XLK", -10, 130)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, stop price, and tag of stop market orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```py
# Create a new order and save the order ticket
ticket = self.stop_market_order("SPY", -100, 415, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = -80
update_settings.stop_price = 400
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_quantity`
- `update_stop_price`
- `update_tag`

```py
response = ticket.update_quantity(quantity, tag)

response = ticket.update_stop_price(stop_price, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a stop market order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.stop_market_order("SPY", 10, 429.0, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with stop market orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for stop market orders, see the Orders section of the brokerage model documentation .

Requirements

Stop market orders can be submitted at any time for all security types.

Examples

The following backtest verifies the `stop_market_order` behavior. On even days, the algorithm buys SPY at the current market price and sells when the price drops 1%. On odd days, the algorithm shorts SPY and buys when the price rises 1%. The following table shows the first four trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T13:31:00Z	SPY	429.10	-1	Market	Filled	-429.10	
2021-07-01T13:31:00Z	SPY	433.44	1	Stop Market	Filled	433.44	Stop Price: 433.43
2021-07-02T17:04:00Z	SPY	433.44	1	Market	Filled	433.44	
2021-07-02T17:04:00Z	SPY	429.00	-1	Stop Market	Filled	-429.00	Stop Price: 429.10

On July 1, 2021, the algorithm shorts SPY at 429.10 and then buys it back at 433.44 when the stop price is 433.43. The stop price is 1% above the market price. Note that 429.10 is not the market price when the algorithm places the first two orders.429.10 is the fill price at the bid, but it's not far from the market price because 429.1 * 1.01 = 433.39. The fill price of the stop market order is 433.44, which, as expected, is higher than $433.43. The fill model assumes the worst-case scenario between the market price and the stop price. In this case, the worst-case scenario is the maximum of the market price and stop price.

On July 2, 2021, the algorithm buys SPY at 433.44 and sells it at 429 when the stop price is 429.10. The stop price is 1% below the market price. Note that 433.44 is not the market price when the algorithm places the second two orders.433.44 is the fill price at the ask, but it's not far off from the market price because 433.44 * 0.99 = 429.11. The fill price of the stop market order is 429, which, as expected, is lower than $429.10. The fill model assumes the worst-case scenario between the market price and the stop price. In this case, the worst-case scenario is the minimum of the market price and stop price.

To reproduce these results, backtest the following algorithm:

PY

```
class StopMarketOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 7)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if not self.portfolio.invested:
            quantity = 1
            factor = 0.99
            if self.time.day % 2 != 0:
                quantity = -1
                factor = 1.01

            self.market_order("SPY", quantity)
            self.stop_market_order("SPY", -quantity, data["SPY"].close * factor)
```

Order Types

Stop Limit Orders

Introduction

Stop limit orders are instructions to place a limit order once an asset passes a specific price level. They are similar to stop market orders , but use limit orders instead of market orders to give you more control over the fill price. A buy stop limit order has a stop price above the current market price and a sell stop limit order has a stop price below the current market price. In effect, they are the opposite of limit if touched orders .

Place Orders

To send a stop limit order, call the `stop_limit_order` method and provide a `Symbol` , quantity, stop price, and limit price. You can also provide a tag and order properties to the `stop_limit_order` method. If you do not have sufficient capital for the order, it is rejected.

```py
self.stop_limit_order(symbol, quantity, stop_rice, limit_price, tag, order_properties)
```

To buy an asset with a stop limit order that has a marketable limit price, set the stop price above the current market price and set the limit price above the stop price.

```py
# When Bitcoin trades up to $59,000, buy 1 Bitcoin with a limit order at $62,000
self.stop_limit_order("BTCUSD", 1, 59000, 62000)
```

To buy an asset with a stop limit order that has an unmarketable limit price, set the stop price above the current market price and set the limit price below the stop price.

```py
# When Bitcoin trades up to $52,000, buy 1 Bitcoin with a limit order at $41,000
self.stop_limit_order("BTCUSD", 1, 52000, 41000)
```

To sell an asset with a stop limit order that has an unmarketable limit price, set the stop price below the current market price and set the limit price above the stop price.

```py
# When Bitcoin trades down to $49,000, sell 1 Bitcoin with a limit order at $58,000
self.stop_limit_order("BTCUSD", -1, 49000, 58000)
```

To sell an asset with a stop limit order that has a marketable limit price, set the stop price below the current market price and set the limit price below the stop price.

```py
# When Bitcoin trades down to $37,000, sell 1 Bitcoin with a limit order at $34,000
self.stop_limit_order("BTCUSD", -1, 37000, 34000)
```

Monitor Order Fills

Stop limit orders fill as limit orders when the security price passes the stop price. To monitor the fills of your order, save a reference to the order ticket .

```
# When GLD trades up to $200, buy 10 shares with a limit order at $205
ticket = self.stop_limit_order("GLD", 10, 200, 205)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, stop price, limit price, and tag of stop limit orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order ticket
ticket = self.stop_limit_order("SPY", -10, 400, 390, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = -15
update_settings.stop_price = 415
update_settings.limit_price = 395
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_stop_price`
- `update_tag`

```
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_stop_price(stop_price, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.stop_limit_order("SPY", -10, 428.95, 429.00, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with stop limit orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for stop limit orders, see the Orders section of the brokerage model documentation .

Requirements

Stop limit orders can be submitted at any time for all security types.

If your algorithm subscribes to extended market hours, they can be filled outside regular trading hours.

Examples

The following backtest verifies the `stop_limit_order` behavior. The following table shows the trades in the backtest:

Submitted Time	Filled Time	Symbol	Stop Price	Limit Price	Filled Price	Quantity	Type	Status	Value	Tag
2021-07-01T09:31:00Z	2021-07-01T09:37:00Z	SPY	428.95	429.00	429.00	-10	Stop Limit	Filled	-4290.00	Stop Price: ¤428.95 Limit Price: ¤429.00
2021-07-02T09:31:00Z	2021-07-02T10:12:00Z	SPY	431.80	431.75	431.75	10	Stop Limit	Filled	4317.50	Stop Price: ¤431.80 Limit Price: ¤431.75

On July 1, 2021 at 9:31 AM Eastern Time (ET), the algorithm places a stop limit order to sell SPY with a stop price of 428.95 and a limit price of 429. At the time of the order submission, the ask high price is 429.16 and the ask close price is 429.11. The order fills at 9:37 AM ET at a price of $429. The fill model triggers the stop for buy orders when the ask high price is greater than the stop price, fills the order when the ask close price is less than the limit price, and sets the fill price to the maximum of the ask high price and the limit price.

On July 2, 2021 at 9:31 AM ET, the algorithm places a stop limit order to buy SPY with a stop price of 431.80 and a limit price of 431.75. At the time of the order submissions, the bid low price is 431.49 and the bid close price is 431.54. The order fills at 10:12 AM ET at a price of $431.75. The fill model triggers the stop for sell orders when bid low price is less than the stop price, fills the order when the ask close price is less than the limit price, and sets the fill price to the minimum of the ask high price and the limit price.

PY

```
class StopLimitOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 4)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if self.time.day == 1 and self.time.hour == 9 and self.time.minute == 31:
            self.stop_limit_order("SPY", -10, 428.95, 429.00)

        elif self.time.day == 2 and self.time.hour == 9 and self.time.minute == 31:
            self.stop_limit_order("SPY", 10, 431.80, 431.75)
```

Order Types

Trailing Stop Orders

Introduction

Trailing stop orders are stop market orders with a dynamic stop price. A sell trailing stop starts with the stop price at a fixed amount below the market price. As the market price rises, the stop price rises by the trail amount, but if the stock price falls, the stop loss price doesn't change. A buy trailing stop order starts with the stop price at a fixed amount above the market price. As the market price falls, the stop price falls by the trail amount, but if the stock price rises, the stop loss price doesn't change.

Trailing stop orders fill as a market order when the asset reaches the stop price. You can use trailing stop orders to buy breakouts or to mitigate the risk of large losses while protecting your gains. However, if the market gaps past your stop price, the order may fill at a worse price than the stop price you set. There's no guarantee that a trailing stop order fills at the stop price.

Place Orders

To send a trailing stop order, call the `trailing_stop_order` method and provide a `Symbol` , quantity, trailing amount, and the trailing type. You can also provide a tag and order properties to the `trailing_stop_order` method. If you do not have sufficient capital for the order, it is rejected.

```
self.trailing_stop_order(symbol, quantity, trailing_amount, trailing_as_percentage, tag, order_properties)
```

If you set the `trailing_as_percentage` parameter to `False` , the `trailing_amount` is denominated in the asset's quote currency .

Buy With a Currency-Based Trailing Stop

To buy an asset with a currency-based trailing stop order, pass the trailing amount and disable the `trailing_as_percentage` parameter.

```
# Buy 1 share of SPY through a trailing stop order that starts $30 above the current price
self.trailing_stop_order("SPY", 1, 30, False)
```

Buy With a Percent-Based Trailing Stop

To buy an asset with a percent-based trailing stop order, pass the trailing percent and enable the `trailing_as_percentage` parameter.

```py
# Buy 1 share of SPY through a trailing stop order that starts 7% above the current price
self.trailing_stop_order("SPY", 1, 0.07, True)
```

Sell With a Currency-Based Trailing Stop

To sell an asset with a currency-based trailing stop order, pass the trailing amount and disable the `trailing_as_percentage` parameter.

```py
# Sell 1 share of SPY through a trailing stop order that starts $20 below the current price
self.trailing_stop_order("SPY", -1, 20, False)
```

Sell With a Percent-Based Trailing Stop

To sell an asset with a percent-based trailing stop order, pass the trailing percent and enable the `trailing_as_percentage` parameter.

```
# Sell 1 share of SPY through a trailing stop order that starts 5% above the current price
self.trailing_stop_order("SPY", -1, 0.05, True)
```

Custom Starting Prices

In the preceding examples, LEAN calculates the stop price based on the current security price. If you want the order to start with a stop price closer to the security price than the `trailing_amount` , pass a `stop_price` parameter to the `trailing_stop_order` method.

```
self.trailing_stop_order(symbol, quantity, stop_price, trailing_amount, trailing_as_percentage, tag, order_properties)
```

Monitor Order Fills

Trailing stop orders fill as a market order when the security price passes the stop price. To monitor the fills of your order, save a reference to the order ticket .

```
# When XLK trades down to less than 5% of the current price
ticket = self.trailing_stop_order("XLK", -10, 0.05, trailing_as_percentage=True)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

The security's fill model automatically updates the stop price of trailing stop orders as the security's price moves away from the stop price. You can update the quantity, stop price, and tag of trailing stop orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order ticket
ticket = self.trailing_stop_order("SPY", -100, 415, 10, trailing_as_percentage=False, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = -80
update_settings.stop_price = 400
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
     self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_quantity`
- `update_stop_price`
- `update_tag`

```
response = ticket.update_quantity(quantity, tag)

response = ticket.update_stop_price(stop_price, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a trailing stop order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.trailing_stop_order("SPY", -100, 0.01, trailing_as_percentage=True, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with trailing stop orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for trailing stop orders, see the Orders section of the brokerage model documentation .

If your brokerage doesn't support them, you can implement a workaround with stop market orders . For an example, see the Buy and Hold with a Trailing Stop Bootcamp lesson.

Requirements

Trailing stop orders can be submitted at any time for all security types.

Examples

The following backtest verifies the `trailing_stop_order` behavior. On even days, the algorithm buys SPY at the current market price and sells when the price drops 1%. On odd days, the algorithm shorts SPY and buys when the price rises 1%. The following table shows the first four trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T13:31:00Z	SPY	429.10	-1	Market	Filled	-429.10	
2021-07-01T13:31:00Z	SPY	433.16	1	Trailing Stop	Filled	433.16	Stop Price: 433.431400 Trailing Amount: 1.00%
2021-07-02T16:53:00Z	SPY	433.16	1	Market	Filled	433.16	
2021-07-02T16:53:00Z	SPY	429.00	-1	Trailing Stop	Filled	-429.00	Stop Price: 428.823450 Trailing Amount: 1.00%

On July 1, 2021, the algorithm shorts SPY at 429.10 and then buys it back at 433.16 when the stop price is 433.16. The stop price is 0.95% above the market price. Note that 429.10 is not the market price when the algorithm places the first two orders. The market price is429.14, and the initial stop price is 429.14 * 1.01 = 433.4314. The market price drops to 428.86 at 9:40, and the stop price is set to 428.86 * 1.01 = 433.1486. The fill price of the stop market order is 433.16, which, as expected, is higher than 433.15. The fill model assumes the worst-case scenario between the market price and the stop price. In this case, the worst-case scenario is the maximum of the market price and stop price.

On July 2, 2021, the algorithm buys SPY at 433.16 and then sells it at 429 when the stop price is 429.00. The stop price is 0.96% below the market price. Note that 433.16 is not the market price when the algorithm places the second two orders. The market price is433.155, and the initial stop price is 433.155 * 0.99 = 428.82345. The market prices increased to 434.76 on July 7 at 3:35 PM, and the stop price is set to 434.76 * 0.99 = 430.4124. The fill price of the stop market order is 429, which, as expected, is lower than 430.41. The fill model assumes the worst-case scenario between the market price and the stop price. In this case, the worst-case scenario is the minimum of the market price and stop price in a gap down opening.

To reproduce these results, backtest the following algorithm:

PY

```
class TrailingStopOrderAlgorithm(QCAlgorithm):
    trailing_amount = 0.01

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 9)
        self.set_cash(100000)

        self.symbol = self.add_equity("SPY", data_normalization_mode = DataNormalizationMode.RAW).symbol

    def on_data(self, data):
        if not self.portfolio.invested:
            quantity = 1 if self.time.day % 2 == 0 else -1

            self.market_order(self.symbol, quantity)
            self.trailing_stop_order(self.symbol, -quantity, self.trailing_amount, True)
```

Order Types

Market On Open Orders

Introduction

Market on open (MOO) orders fill at the official opening auction price for a security. Market that operate 24/7 don't support MOO orders.

Place Orders

To send a MOO order, call the `market_on_open_order` method and provide a `Symbol` and quantity. Submit the order at least two minutes before the market opens to be included in the opening auction. If you do not have sufficient capital for the order, it's rejected.

```py
# Buy 100 shares of IBM at the market open
self.market_on_open_order("IBM", 100)

# Sell 100 shares of IBM at the market open
self.market_on_open_order("IBM", -100)
```

You can provide a tag and order properties to the `market_on_open_order` method.

```py
self.market_on_open_order(symbol, quantity, tag=tag, order_properties=order_properties)
```

Monitor Order Fills

MOO orders fill at the opening auction. If the auction is heavily skewed to one side of the market, your MOO order may not fill. To monitor the fills of your order, save a reference to the order ticket .

```py
# Buy 10 shares of SPY at the market open
ticket = self.market_on_open_order("SPY", 10)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

You won't know the fill price of the order until after the market opens. If the asset price moves before the market open to where you can't afford the quantity of the order, the brokerage rejects your order. To increase the probability of a successful trade, leave a sufficient buying power buffer to handle daily price gaps.

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity and tag of MOO orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```py
# Create a new order and save the order ticket
ticket = self.market_on_open_order("AAPL", 100, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = 75
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_quantity`
- `update_tag`

```
response = ticket.update_quantity(quantity, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a MOO order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.market_on_open_order("SPY", 10, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with MOO orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for MOO orders, see the Orders section of the brokerage model documentation .

Requirements

You can submit MOO orders at any time for Equity, Equity Options, and Index Options.

Markets that operate 24/7 don't support MOO orders. The Forex market doesn't operate during the weekend. However, MOO orders are invalid if you place them after this market closes for the weekend.

MOO orders don't support the `GoodTilDate` time in force . If you submit a MOO order with the `GoodTilDate` time in force, LEAN automatically adjusts the time in force to be `GoodTilCanceled` .

Examples

The following backtest verifies the `market_on_open_order` behavior. The following table shows the first trade in the backtest:

Submitted Time	Filled Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T10:31:00Z	2021-07-02T09:31:00Z	SPY	431.67	10	Market On Open	Filled	4316.70	

On July 1, 2021 at 10:31 AM Eastern Time (ET), the algorithm places a market on open order to buy SPY. The fill model fills the order on July 2, 2021 at 9:31 AM at a price of $431.67, which is the official opening auction price for July 2, 2021.

To reproduce these results, backtest the following algorithm:

```py
class MarketOnOpenOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 4)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if self.time.day == 1 and self.time.hour == 10 and self.time.minute == 31:
            self.market_on_open_order("SPY", 10)
```

Order Types

Market On Close Orders

Introduction

Market on close (MOC) orders fill at the official closing auction price for a security. Markets that operate 24/7 don't support MOC orders.

Place Orders

To send a MOC order, call the `market_on_close_order` method with a `Symbol` and quantity. If you don't have sufficient capital for the order, it is rejected.

```
PY
# Buy 100 shares of AAPL at the market open
self.market_on_close_order("AAPL", 100)

# Sell 100 shares of AAPL at the market open
self.market_on_close_order("AAPL", -100)
```

You can provide a tag and order properties to the `market_on_close_order` method.

```
PY
self.market_on_close_order(symbol, quantity, tag=tag, order_properties=order_properties)
```

By default, you must place MOC orders at least 15.5 minutes before the close, but some exchanges let you submit them closer to the market closing time. To adjust the buffer period that's required, set the `MarketOnCloseOrder.submission_time_buffer` property.

```
PY
MarketOnCloseOrder.submission_time_buffer = timedelta(minutes=10)
```

You can also place MOC orders after the market close.

Monitor Order Fills

MOC orders fill at the closing auction. If the auction is heavily skewed to one side of the market, your MOC order may not fill. To monitor the fills of your order, save a reference to the order ticket .

```
PY
# Buy 10 shares of SPY at the market close
ticket = self.market_on_close_order("SPY", 10)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

You won't know the fill price of the order until after the market closes. If the asset price moves before the market close to where you can't afford the quantity of the order, the brokerage rejects your order. To increase the probability of a successful trade, leave a sufficient buying power buffer to handle the price movements before the close.

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity and tag of MOC orders until the order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order ticket
ticket = self.market_on_open_order("SLV", 25, tag="original tag")

# Update the order
update_settings = UpdateOrderFields()
update_settings.quantity = 50
update_settings.tag = "new tag"
response = ticket.update(update_settings)

# Check if the update was successful
if response.is_success:
    self.debug("Order updated successfully")
```

To update individual fields of an order, call any of the following methods:

- `update_quantity`
- `update_tag`

```
response = ticket.update_quantity(quantity, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a MOC order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.market_on_close_order("SPY", 10, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with MOC orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for MOC orders, see the Orders section of the brokerage model documentation .

Requirements

By default, you must place MOC orders at least 15.5 minutes before the close, but some exchanges let you submit them closer to the market closing time. To adjust the buffer period that's required, set the `MarketOnCloseOrder.submission_time_buffer` property.

```
MarketOnCloseOrder.submission_time_buffer = timedelta(minutes=10)
```

Markets that operate 24/7 don't support MOC orders. The Forex market doesn't operate during the weekend.

MOC orders don't support the `GoodTilDate` time in force . If you submit a MOC order with the `GoodTilDate` time in force, LEAN automatically adjusts the time in force to be `GoodTilCanceled` .

Examples

The following backtest verifies the `market_on_close_order` behavior. The following table shows the first trade in the backtest:

Submitted Time	Filled Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T10:31:00Z	2021-07-01T16:00:00Z	SPY	430.43	10	Market On Close	Filled	4304.30	

On July 1, 2021 at 10:31 AM Eastern Time (ET), the algorithm places a market on open order to buy SPY. The fill model fills the order on July 1, 2021 at 4:00 PM ET at a price of $430.43, which is the official closing auction price for July 1, 2021.

To reproduce these results, backtest the following algorithm:

```
class MarketOnCloseOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 4)
        self.set_cash(100000)
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_equity("SPY")

    def on_data(self, data):
        if self.time.day == 1 and self.time.hour == 10 and self.time.minute == 31:
            self.market_on_close_order("SPY", 10)
```

Order Types

Combo Market Orders

Introduction

Combo market orders are individual orders that contain market orders for multiple securities. Combo market orders currently only work for trading Option contracts.

Place Orders

To send a combo market order, create multiple `Leg` objects to represent the legs of the combo order, then call the `combo_market_order` method. At least one leg must have a positive quantity and at least one leg must have a negative quantity. The legs must each target a unique contract. If you don't have sufficient capital for the order, it's rejected. By default, combo market orders are synchronous and fill immediately.

```
PY
for canonical_symbol, chain in slice.option_chains.items():
    # Select contracts
    contracts = [c for c in chain][:2]
    if len(contracts) < 2:
        return

    # Create order legs
    legs = []
    quantities = [1, -1]
    for i, contract in enumerate(contracts):
        legs.append(Leg.create(contract.symbol, quantities[i]))

    # Place order
    self.combo_market_order(legs, 1)
```

The quantity of the legs sets the ratio of the leg orders while the quantity argument of the `combo_market_order` method sets the combo order size and acts as a global multiplier. In the preceding example, if we set the global multiplier to two, then the algorithm buys two units of the first contract and sells two units of the second contract. The quantity argument of the `combo_market_order` method also sets the order direction of the combo order, which affects how the fill model fills the order.

You can also provide a tag and order properties to the `combo_market_order` method.

```
PY
self.combo_market_order(legs, quantity, tag=tag, order_properties=order_properties)
```

Monitor Order Fills

If the brokerage has sufficient liquidity in their order book, combo market orders fill immediately. Otherwise, you get partial fills. To monitor the fills of your order, save a reference to the order tickets .

```
PY
ticket = self.combo_market_order(legs, 1)
for ticket in tickets:
    self.debug(f"Symbol: {ticket.symbol}; Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Cancel Orders

To cancel a combo market order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
PY
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

PY

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.combo_market_order(legs, quantity, asynchronous=True);
```

PY

Synchronous Timeouts

Combo market orders are synchronous by default, so your algorithm waits for the order to fill before moving to the next line of code. If your order takes longer than five seconds to fill, your algorithm continues executing even if the trade isn't filled. To adjust the timeout period, set the `transactions.market_order_fill_timeout` property.

```
 # Adjust the market fill-timeout to 30 seconds.
self.transactions.market_order_fill_timeout = timedelta(seconds=30)
```

PY

Combo market orders may take a few minutes to fill for illiquid assets such as out-of-the-money Options.

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with combo market orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

PY

To check if your brokerage has any special requirements for combo market orders, see the Orders section of the brokerage model documentation .

Requirements

You can submit combo market orders during regular trading hours for all asset classes. For Futures and Future Options, you can place combo market orders during regular and extended market hours. For all other asset classes, if you place combo market orders at/after the last minute of regular market hours, LEAN converts the combo market orders into market-on-open orders as long as the asset class supports them . To view the trading hours of each asset class, follow these steps:

1. Open the Asset Classes documentation.
2. Click an asset class.
3. Click Market Hours .

Examples

The following backtest verifies the `combo_market_order` behavior. The algorithm buys one contract and sells one contract at the same time. The following table shows the two trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2015-12-24T09:31:00Z	GOOG 160115C00745000	16.90	1	Buy	Filled	16.90	
2015-12-24T09:31:00Z	GOOG 160115C00747500	14.20	-1	Sell	Filled	-14.20	

On December 24, 2015, the algorithm buys GOOG 16011SC00745000 at 16.90 and sells GOOG 160115C00747500 at 14.20. The fill model fills the buy order at the ask close price and fills the sell order at the bid close price.

To reproduce these results, backtest the following algorithm:

```py
class ComboMarketOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 12, 24)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        option = self.add_option("GOOG")
        option.set_filter(min_strike=-2, max_strike=2, min_expiry=timedelta(days=0), max_expiry=timedelta(days=180))

    def on_data(self, data: Slice):
        if self.portfolio.invested:
            return

        for canonical_symbol, chain in data.option_chains.items():
            contracts = [c for c in chain][:2]
            if len(contracts) < 2:
                return

            quantities = [1, -1]
            legs = []
            for i, contract in enumerate(contracts):
                legs.append(Leg.create(contract.symbol, quantities[i]))

            tickets = self.combo_market_order(legs, 1)
```

Order Types

Combo Limit Orders

Introduction

Combo limit orders are individual orders that contain limit orders for muliple securities. Combo limit orders are different from combo leg limit orders because you must set the limit price of all the leg orders to be the same with combo limit orders. With combo leg limit orders, you can create the order without forcing each leg to have the same limit price. Combo limit orders currently only work for trading Option contracts and their underlying Equities.

Place Orders

To send a combo limit order, create multiple `Leg` objects to represent the legs of the combo order, then call the `combo_limit_order` method. At least one leg must have a positive quantity and a tleast one leg must have a negative quantity. The legs must each target a unique contract, but don't set the `OrderPrice` property for any of the legs. If you don't have sufficient capital for the order, it's rejected.

```
PY
for canonical_symbol, chain in slice.option_chains.items():
    # Select contracts
    contracts = [c for c in chain][:2]
    if len(contracts) < 2:
        return

    # Create order legs
    legs = []
    quantities = [1, -1]
    for i, contract in enumerate(contracts):
        legs.append(Leg.create(contract.symbol, quantities[i]))

    # Calculate limit price
    limit_price = round(sum([self.securities[leg.symbol].close for leg in legs]) * 0.95, 2)

    # Place order
    self.combo_limit_order(legs, 1, limit_price)
```

The quantity of the legs sets the ratio of the leg orders while the quantity argument of the `combo_limit_order` method sets the combo order size and acts as a global multiplier. In the preceding example, if we set the global multiplier to two, then the algorithm buys two units of the first contract and sells two units of the second contract. The quantity also sets the order direction of the combo limit order, which affects how the fill model fills the order.

You can also provide a tag and order properties to the `combo_limit_order` method.

```
PY
self.combo_limit_order(legs, quantity, limit_price, tag=tag, order_properties=order_properties)
```

Monitor Order Fills

Combo limit orders fill all the legs at the same time. Each leg can fill when the sum of the security prices of the legs pass the limit price of the combo order. To monitor the fills of your order, save a reference to the order tickets .

```
PY
tickets = self.combo_limit_order(legs, 1, limit_price)
for ticket in tickets:
    self.debug(f"Symbol: {ticket.symbol}; Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, limit price, and tag of the limit orders in each leg until the combo order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . To update the limit price of the combo order, you only need to update the limit price of one of the leg orders. The `update`

method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order tickets
tickets = self.combo_limit_order(legs, 1, limit_price)

# Update the leg orders
for ticket in tickets:
    update_settings = UpdateOrderFields()
    update_settings.quantity = 2 * np.sign(ticket.quantity)
    update_settings.limit_price = ticket.get(OrderField.LIMIT_PRICE) + 0.01
    update_settings.tag = f"Update #{len(ticket.update_requests) + 1}"
    response = ticket.update(update_settings)

    # Check if the update was successful
    if response.is_success:
        self.debug(f"Order updated successfully for {ticket.symbol}")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_tag`

```
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a combo limit order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.combo_limit_order(legs, 1, limit_price, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with combo limit orders, set the brokerage model to a brokerage that supports them.

```py
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for combo limit orders, see the Orders section of the brokerage model documentation .

Requirements

Combo limit orders can be submitted at any time for all security types.

If your algorithm subscribes to extended market hours, they can be filled outside regular trading hours.

Examples

The following backtest verifies the `combo_limit_order` behavior. The algorithm buys one contract and sells one contract at the same time. The following table shows the two trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2015-12-24T09:31:00Z	GOOG 16011SC00745000	16.50	1	Buy	Filled	16.50	
2015-12-24T09:31:00Z	GOOG 160115C00747500	14.60	-1	Sell	Filled	-14.60	

On December 24, 2015 at 9:31 AM Eastern Time (ET), the algorithm places a combo limit order to buy one GOOG 16011SC00745000 contract and sell one GOOG 160115C00747500 contracts. The limit price is 75% of the contract spread, which equals 2.02. The combo order doesn't fill immediately because the contract spread 2.40 > 2.02. By 9:36 AM, the spread drops to1.90, which is below the limit price, so the fill model fills the combo limit order.

To reproduce these results, backtest the following algorithm:

PY

```
import itertools

class ComboMarketOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 12, 24)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        option = self.add_option("GOOG")
        option.set_filter(min_strike=-2, max_strike=2, min_expiry=timedelta(days=0), max_expiry=timedelta(days=180))

        self.tickets = []

    def on_data(self, slice: Slice):
        if len(self.tickets) == 0:
            for canonical_symbol, chain in slice.option_chains.items():
                # Select contracts
                contracts = [contract for contract in chain if contract.right == OptionRight.CALL]
                contracts = [(key, list(group)) for key, group in itertools.groupby(contracts, key=lambda x: x.expiry)]
                contracts.sort(key=lambda x: x[0])
                contracts = contracts[0][1]
                contracts.sort(key=lambda x: x.strike)

                if len(contracts) < 2:
                    return

                # Create order legs
                quantities = [1, -1]
                legs = []
                for i, contract in enumerate(contracts[:2]):
                    legs.append(Leg.create(contract.symbol, quantities[i]))

                # Calculate limit price
                self.limit_price = round((slice.quote_bars[contracts[0].symbol].ask.low - slice.quote_bars[contracts[1].symbol].bid.low)
* 0.75, 2)

                # Place order
                self.tickets = self.combo_limit_order(legs, 1, self.limit_price)
        else:
            if self.time.hour == 9 and self.time.minute == 37:
                self.quit()
                return

            price = 0
            for ticket in self.tickets:
                quote_bar = slice.quote_bars[ticket.symbol]
                price += ((quote_bar.ask if ticket.quantity > 0 else quote_bar.bid).low * np.sign(ticket.quantity))
            self.log(f"{self.time}. Limit price: {self.limit_price}; Aggregate price: {round(price, 2)}; Ready to fill: {price <
self.limit_price}")

    def on_order_event(self, orderEvent: OrderEvent) -> None:
        if orderEvent.status == OrderStatus.FILLED:
            self.log(f"{self.time} -- Order {orderEvent.order_id} filled at {orderEvent.fill_price}")
```

Order Types

Combo Leg Limit Orders

Introduction

Combo leg limit orders are individual orders that contain limit orders for muliple securities. Combo leg limit orders are different from combo limit orders because you can create combo leg limit orders without forcing each leg to have the same limit price. Combo leg limit orders currently only work for trading Option contracts.

Place Orders

To send a combo leg limit order, create multiple `Leg` objects to represent the legs of the combo order, then call the `combo_leg_limit_order` method. The legs must each target a unique contract. At least one leg must have a positive quantity and at least one leg must have a negative quantity. If you don't have sufficient capital for the order, it's rejected.

```py
for canonical_symbol, chain in slice.option_chains.items():
    # Select contracts
    contracts = [c for c in chain][:2]
    if len(contracts) < 2:
        return

    # Create order legs
    legs = []
    quantities = [1, -1]
    factors = [0.98, 1.02]
    for i, contract in enumerate(contracts):
        legs.append(Leg.create(contract.symbol, quantities[i], contract.last_price * factors[i]))

    # Place order
    self.combo_leg_limit_order(legs, 1)
```

The quantity of the legs sets the ratio of the leg orders while the quantity argument of the `combo_leg_limit_order` method sets the combo order size and acts as a global multiplier. In the preceding example, if we set the global multiplier to two, then the algorithm buys two units of the first contract and sells two units of the second contract. The quantity also sets the order direction of the combo limit order, which affects how the fill model fills the order.

You can also provide a tag and order properties to the `combo_leg_limit_order` method.

```py
self.combo_leg_limit_order(legs, quantity, tag=tag, order_properties=order_properties)
```

Monitor Order Fills

Combo leg limit orders fill all the legs at the same time. Each leg can fill when the security price passes the limit price of the leg. To monitor the fills of your order, save a reference to the order tickets .

```py
tickets = self.combo_leg_limit_order(legs, 1)
for ticket in tickets:
    self.debug(f"Symbol: {ticket.symbol}; Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models order fills in backtests, see Trade Fills .

Update Orders

You can update the quantity, limit price, and tag of the leg limit orders until the combo order fills or the brokerage prevents modifications. To update an order, pass an `UpdateOrderFields` object to the `update` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `update` method returns an OrderResponse to signal the success or failure of the update request.

```
# Create a new order and save the order tickets
tickets = self.combo_leg_limit_order(legs, 1)

# Update the leg orders
for ticket in tickets:
    direction = np.sign(ticket.quantity)
    update_settings = UpdateOrderFields()
    update_settings.quantity = 2 * direction
    update_settings.limit_price = ticket.get(OrderField.LIMIT_PRICE) + 0.01 * direction
    update_settings.tag = f"Update #{len(ticket.update_requests) + 1}"
    response = ticket.update(update_settings)

    # Check if the update was successful
    if response.is_success:
        self.debug(f"Order updated successfully for {ticket.symbol}")
```

To update individual fields of an order, call any of the following methods:

- `update_limit_price`
- `update_quantity`
- `update_tag`

```
response = ticket.update_limit_price(limit_price, tag)

response = ticket.update_quantity(quantity, tag)

response = ticket.update_tag(tag)
```

When you update an order, LEAN creates an `UpdateOrderRequest` object, which have the following attributes:

To get a list of `UpdateOrderRequest` objects for an order, call the `update_requests` method.

```
update_requests = ticket.update_requests()
```

Cancel Orders

To cancel a combo leg limit order, call the `cancel` method on the `OrderTicket` . If you don't have the order ticket, get it from the transaction manager . The `cancel` method returns an `OrderResponse` object to signal the success or failure of the cancel request.

```
response = ticket.cancel("Cancelled Trade")
if response.is_success:
    self.debug("Order successfully cancelled")
```

When you cancel an order, LEAN creates a `CancelOrderRequest` , which have the following attributes:

To get the `CancelOrderRequest` for an order, call the `cancel_order_request` method on the order ticket. The method returns `None` if the order hasn't been cancelled.

```
request = ticket.cancel_order_request()
```

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.combo_leg_limit_order(legs, 1, asynchronous=True);
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with combo leg limit orders, set the brokerage model to a

brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for combo leg limit orders, see the Orders section of the brokerage model documentation .

Requirements

Combo leg limit orders can be submitted at any time for all security types.

If your algorithm subscribes to extended market hours, they can be filled outside regular trading hours.

Examples

The following backtest verifies the `combo_leg_limit_order` behavior. The algorithm buys one contract and sells one contract at the same time. The following table shows the two trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2015-12-24T09:31:00Z	GOOG 16011SC00745000	16.10	2	Buy	Filled	32.20	Update #72
2015-12-24T09:31:00Z	GOOG 160115C00747500	14.11515	-2	Sell	Filled	-28.2303	Update #72

On December 24, 2015 at 9:31 AM Eastern Time (ET), the algorithm places a combo leg limit order to buy one GOOG 16011SC00745000 contract and sell two GOOG 160115C00747500 contracts. The limit price of both orders is 99.9% of the respective contract price, which is 16.2837 for GOOG 16011SC00745000 and 14.83515 for GOOG 160115C00747500. The combo order doesn't fill immediately, so the algorithm updates the leg orders at each time step. During the first update, the algorithm sets the quantity of the GOOG 160115C00747500 leg to -2. During each update, the limit price moves 0.01 closer to the market. That is, the limit price of GOOG 16011SC00745000 increases by0.01 and the limit price of GOOG 160115C00747500 decreases by $0.01. After the 72nd update, the ask low price is below the limit price of the leg to buy GOOG 16011SC00745000 and the bid high price is above the limit price of the leg to sell GOOG 160115C00747500, so the fill model fills the combo leg limit order at 10:44 AM ET.

To reproduce these results, backtest the following algorithm:

PY

```
import itertools

class ComboLegLimitOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 12, 24)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        option = self.add_option("GOOG")
        option.set_filter(min_strike=-2, max_strike=2, min_expiry=timedelta(days=0), max_expiry=timedelta(days=180))

        self.tickets = []

    def on_data(self, slice: Slice):
        if len(self.tickets) == 0:
            for canonical_symbol, chain in slice.option_chains.items():
                # Select contracts
                contracts = [contract for contract in chain if contract.right == OptionRight.CALL]
                contracts = [(key, list(group)) for key, group in itertools.groupby(contracts, key=lambda x: x.expiry)]
                contracts.sort(key=lambda x: x[0])
                contracts = contracts[0][1]
                contracts.sort(key=lambda x: x.strike)

                if len(contracts) < 2:
                    return

                # Create order legs
                quantities = [1, -2]
                legs = []
                for i, contract in enumerate(contracts[:2]):
                    legs.append(Leg.create(contract.symbol, quantities[i], slice[contract.symbol].close * .999))

                # Place order
                self.tickets = self.combo_leg_limit_order(legs, 1)
        else:
            if self.time.hour == 10 and self.time.minute == 45:
                self.quit()
                return

            for ticket in self.tickets:
                order_direction = np.sign(ticket.quantity)
                limit_price = ticket.get(OrderField.LIMIT_PRICE)

                # Log the limit prices and contract prices
                quote_bar = slice.quote_bars[ticket.symbol]
                current_price = quote_bar.ask.low if order_direction == 1 else quote_bar.bid.high
                ready_to_fill = current_price < limit_price if order_direction == 1 else current_price > limit_price
                self.log(f"{self.time} - {ticket.symbol}. Current price: {current_price}; Limit price: {round(limit_price, 2)};
Direction: {order_direction}; Ready to fill: {ready_to_fill}")

                # Update the leg orders
                update_settings = UpdateOrderFields()
                update_settings.quantity = 2 * order_direction
                update_settings.limit_price = limit_price + 0.01 * order_direction
                update_settings.tag = f"Update #{len(ticket.update_requests) + 1}"
                response = ticket.update(update_settings)

    def on_order_event(self, orderEvent: OrderEvent) -> None:
        if orderEvent.status == OrderStatus.FILLED:
            self.log(f"{self.time} -- Order {orderEvent.order_id} filled at {orderEvent.fill_price}")
```

Order Types

Option Exercise Orders

Introduction

If you buy an Option contract, you can exercise your right to buy or sell the underlying asset at the strike price. However, you don't need to exercise it. You can sell the Option or let it expire worthless if it's out of the money. If you hold a long position in an Option that expires in the money, LEAN automatically exercises it at the expiration date. If you sell an Option contract and the buyer exercises their right to buy or sell the underlying asset, you are assigned the Option and must trade with the buyer at the strike price.

Place Orders

You can exercise American-style Option contracts anytime before they expire. Depending on your brokerage and the delivery method, you may be able to exercise European-style Option contracts on their expiration date. To exercise an Option, call the `exercise_option` method with the Option contract `Symbol` and a quantity. If you do not have sufficient capital for the order, it's rejected. By default, Option exercise orders are synchronous and fill immediately.

```
PY
ticket = self.exercise_option(contract_symbol, quantity)
```

You can provide a tag and order properties to the `exercise_option` method.

```
PY
self.exercise_option(symbol, quantity, tag=tag, order_properties=order_properties)
```

Monitor Order Fills

To monitor the fills of your order, save a reference to the order ticket .

```
PY
ticket = self.exercise_option(contract_symbol, quantity)
self.debug(f"Quantity filled: {ticket.quantity_filled}; Fill price: {ticket.average_fill_price}")
```

For more information about how LEAN models Option exercise orders in backtests, see the Exercise Option model .

Option Assignments

If you sell an Option in a backtest, LEAN can simulate an Option exercise order on behalf of the buyer. By default, LEAN scans your portfolio every hour. It considers exercising American-style Options if they are within 4 days of their expiration and it considers exercising European-style Options on their day of expiration. If you have sold an Option that's 5% in-the-money and the Option exercise order is profitable after the cost of fees, LEAN exercises the Option. For more information about how we simulate Option assignments, see the Assignment reality model.

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
PY
ticket = self.exercise_option(contract_symbol, quantity, asynchronous=True);
```

Synchronous Timeouts

Option exercise orders are synchronous by default, so your algorithm waits for the order to fill before moving to the next line of code. If your order takes longer than five seconds to fill, your algorithm continues executing even if the trade isn't filled. To adjust the timeout period, set the `transactions.market_order_fill_timeout` property.

```
 # Adjust the market fill-timeout to 30 seconds.
self.transactions.market_order_fill_timeout = timedelta(seconds=30)
```

Brokerage Support

Each brokerage has a set of assets and order types they support. To avoid issues with Option exercise orders, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.QUANTCONNECT_BROKERAGE)
```

To check if your brokerage has any special requirements for Option exercise orders, see the Orders section of the brokerage model documentation .

Requirements

Option exercise orders can only be submitted for option contracts with a long position. European-style options cannot be exercised before their expiration date.

Examples

The following backtest verifies the `exercise_option` behavior. The following table shows the first three trades in the backtest:

Time	Symbol	Price	Quantity	Type	Status	Value	Tag
2021-07-01T09:31:00Z	SPY 221216C00085000	345.30	1	Buy Market	Filled	34530.00	
2021-07-01T09:31:00Z	SPY 221216C00085000	0.00	-1	Sell Option Exercise	Filled	0.00	Automatic Exercise
2021-07-01T09:31:00Z	SPY	85.00	100	Sell Option Exercise	Filled	8500.00	Option Exercise

The algorithm first brought a deep ITM option based on the set option selection conditions by 345.30, then exercise it actively, so 100 shares of SPY (in which 1 contract represents) was brought at its strike price at 85.00, with the option discarded from the portfolio.

To reproduce these results, backtest the following algorithm:

PY

```
class OptionExerciseOrderAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.settings.seed_initial_prices = True
        self.add_security_initializer(self.custom_security_initializer)

        spy = self.add_equity("SPY", Resolution.MINUTE)
        self.spy = spy.symbol

    def custom_security_initializer(self, security):
        security.set_data_normalization_mode(DataNormalizationMode.RAW)

    def on_data(self, data):
        if not self.portfolio.invested:
            contracts = self.option_chain_provider.get_option_contract_list(self.spy, self.time)
            contracts = sorted([x for x in contracts if x.id.option_right == OptionRight.CALL], key=lambda x: x.id.strike_price)
            if not contracts: return

            self.strike = contracts[0].id.strike_price

            contracts = sorted([x for x in contracts if x.id.strike_price == self.strike], key=lambda x: x.id.date)

            self.contract = contracts[0]
            self.add_option_contract(self.contract, Resolution.MINUTE)

            self.buy(self.contract, 1)

        if self.portfolio[self.contract].invested and self.strike < self.securities[self.spy].price:
            self.exercise_option(self.contract, 1)
            self.quit()
```

Order Types

Other Order Types

Introduction

We are often asked to support other order types like one cancels the other, trailing stop, and multi-leg orders. Currently, LEAN doesn't support these order types, but we will add them over time. Part of the difficulty of implementing them is the incomplete brokerage support.

One Cancels the Other Orders

One cancels the other (OCO) orders are a set of orders that when one fills, it cancels the rest of the orders in the set. An example is to set a take-profit and a stop-loss order right after you enter a position. In this example, when either the take-profit or stop-loss order fills, you cancel the other order. OCO orders usually create an upper and lower bound on the exit price of a trade.

When you place OCO orders, their price levels are usually relative to the fill price of an entry trade. If your entry trade is a synchronous market order, you can immediately get the fill price from the order ticket. If your entry trade doesn't execute immediately, you can get the fill price in the on_order_events event handler. Once you have the entry fill price, you can calculate the price levels for the OCO orders.

```py
# Get the fill price from the order ticket of a sync market order
self._market = self.market_order("SPY", 1)
fill_price = self._market.average_fill_price

# Get the fill price from the OnOrderEvent event handler
def on_order_event(self, order_event: OrderEvent) -> None:
    if order_event.status == OrderStatus.FILLED and order_event.ticket.order_type == OrderType.MARKET:
        fill_price = order_event.fill_price
```

After you have the target price levels, to implement the OCO orders, you can place active orders or track the security price to simulate the orders.

Place Active Orders

To place active orders for the OCO orders, use a combination of limit orders and stop limit orders . Place these orders so that their price levels that are far enough apart from each other. If their price levels are too close, several of the orders can fill in a single time step. When one of the orders fills, in the `on_order_event` event handler, cancel the other orders in the OCO order set.

```py
_stop_loss = None
_take_profit = None

def on_order_event(self, order_event: OrderEvent) -> None:
    if order_event.status != OrderStatus.FILLED:
        return

    match order_event.ticket.order_type:
        case OrderType.MARKET:
            self._stop_loss = self.stop_market_order(order_event.symbol, -order_event.fill_quantity, order_event.fill_price*0.95)
            self._take_profit = self.limit_order(order_event.symbol, -order_event.fill_quantity, order_event.fill_price*1.10)
        case OrderType.STOP_MARKET:
            self._take_profit.cancel()
        case OrderType.LIMIT:
            self._stop_loss.cancel()
```

Simulate Orders

To simulate OCO orders, track the asset price in the `on_data` method and place market or limit orders when asset price reaches the take-profit or stop-loss level. The benefit of manually simulating the OCO orders is that both of the orders can't fill in the same time step.

```
def on_data(self, slice: Slice) -> None:
    if not self.portfolio.invested:
        ticket = self.market_order("SPY", 1)
        self.entry_price = ticket.average_fill_price

    bar = slice.get("SPY")
    if bar:
        if bar.price >= self.entry_price * 1.10:
            self.liquidate(symbol="SPY", -1, tag="take profit")

        elif bar.price <= self.entry_price * 0.95:
            self.liquidate(symbol="SPY", -1, tag="stop loss")
```

Multi-Leg Orders

Multi-leg orders are orders that contain multiple sub-orders. Examples of multi-leg orders include Option strategies like spreads , straddles , and strangles . You can manually implement other types of multi-leg orders with the built-in order types.

Examples

The following example demonstrates a helper class for OCO orders. This is not an officially supported OCO order type as it's possible for both the take profit and stop loss orders to fill in the same bar. Before you use this approach, verify that your brokerage model supports stop market orders and limit orders.

```
class OCOTrade:

    def __init__(self, algorithm, symbol, quantity, entry_stop_price, take_profit_price, stop_loss_price):
        self._algorithm = algorithm
        self._symbol = symbol
        self._take_profit_price = take_profit_price
        self._stop_loss_price = stop_loss_price
        self._entry = algorithm.stop_market_order(symbol, quantity, entry_stop_price)
        self._take_profit = None
        self._stop_loss = None

    def on_order_event(self, order_event):
        if order_event.status != OrderStatus.FILLED:
            return
        _id = order_event.order_id
        if self._entry and self._entry.order_id == _id:
            quantity = order_event.fill_quantity
            self._take_profit = self._algorithm.limit_order(self._symbol, -quantity, self._take_profit_price)
            self._stop_loss = self._algorithm.stop_market_order(self._symbol, -quantity, self._stop_loss_price)
            self._entry = None
        elif self._take_profit and self._take_profit.order_id == _id:
            self._stop_loss.cancel()
            self._take_profit = self._stop_loss = None
        elif self._stop_loss and self._stop_loss.order_id == _id:
            self._take_profit.cancel()
            self._take_profit = self._stop_loss = None


class InsideDayBreakout(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000)
        self.set_time_zone(TimeZones.UTC)
        self.set_brokerage_model(BrokerageName.OANDA_BROKERAGE, AccountType.MARGIN)
        self._forex = self.add_forex("USDJPY")
        self._forex.session.size = 3
        for bar in self.history[QuoteBar](self._forex, 3):
            self._forex.session.update(bar)
        self._risk_pct = 7
        self._long_trade = None
        self._short_trade = None
        self.set_warm_up(timedelta(4))

    def on_warmup_finished(self):
        # Add a Scheduled event to rebalance the portfolio monthly.
        self.schedule.on(
            self.date_rules.every([DayOfWeek.MONDAY, DayOfWeek.TUESDAY, DayOfWeek.WEDNESDAY, DayOfWeek.THURSDAY, DayOfWeek.FRIDAY]),
            self.time_rules.after_market_open(self._forex, 1),
            self._rebalance
        )

    def _rebalance(self):
        if not self._forex.session.is_ready or self.portfolio.invested:
            return
        session = self._forex.session[1]
        high, low = session.high, session.low
        prev_high, prev_low = self._forex.session[2].high, self._forex.session[2].low
        if not (high < prev_high and low > prev_low):
```

```
            return
        # Replace any pending entry orders from a previous inside day.
        self.liquidate()
        inside_high = round(high, 3)
        inside_low = round(low, 3)
        day_range = inside_high - inside_low
        # Risk amount in USD.
        risk_usd = self.portfolio.cash * (self._risk_pct / 100)
        # Risk per unit in USD terms (day_range is in JPY, convert to USD via close price).
        risk_per_unit = day_range / session.close
        # OANDA max margin 50:1.
        risk_quantity = int(risk_usd / risk_per_unit)
        max_quantity = int((self.portfolio.cash * 48) / session.close)
        quantity = min(risk_quantity, max_quantity)
        if not quantity:
            return
        self._long_trade = OCOTrade(
            self, self._forex, quantity, inside_high,
            take_profit_price=round(inside_high + day_range, 3),
            stop_loss_price=inside_low
        )
        self._short_trade = OCOTrade(
            self, self._forex, -quantity, inside_low,
            take_profit_price=round(inside_low - day_range, 3),
            stop_loss_price=inside_high
        )

    def on_order_event(self, order_event):
        for trade in [self._long_trade, self._short_trade]:
            if trade is not None:
                trade.on_order_event(order_event)
```

Trading and Orders

Pre-trade Risk Control

Introduction

LEAN performs pre-order checks to ensure that the order meets some requirements to ensure that the order is valid and can be executed. These checks are include risk controls such as validation of order size and price limits, and whether the order can be executed by a given brokerage at a given time.

Basic Validation

Orders undergo basic validation checks to ensure they meet certain criteria before being submitted. The following list outlines some of the key checks performed:

1. **Tradable:** Ensures that the security is tradable or in the market.

   Your algorithm cannot place orders for indices , delisted securities, expired contracts, or any assets removed from your universe. For example, the option filter or universe selection removed a contract because it doesn't meet the criteria set by the filter.

2. **Market Hours:** Checks that the order can be placed at the current time.

   For example, market orders cannot be placed outside of regular trading hours. LEAN converts market orders to market-on-open orders if possible.

3. **Price:** Checks that the last known price is not zero.

   LEAN requires the last known price for buying power calculations .

4. **Order Size:** Ensures that the order quantity is not zero or less than the lot size.

LEAN ensures that your algorithm has a valid data subscription for the security and receives its market data before placing an order. For example, you can place a market order to trade SPY, LEAN calls the `self.add_equity("SPY")` for you and seeds initial prices.

Order Type

QuantConnect supports several order types . To see the requirements of each order, see the Requirements section of the documentation for the order types . LEAN doesn't allow placing orders that don't meet these requirements. For example, your algorithm cannot place a market-on-close order from cryto-currencies because they operate 24/7.

Buying Power

When you place an order, LEAN uses the buying power model to determine whether the order should be submitted so you avoid placing orders that would be rejected by the brokerage. Buying power calculations can be very complex and depend on many factors, including the brokerage or even the time of day. The main factors include your current portfolio holdings, available cash, and margin requirements.

The buying power calculations are restricted to portfolio your algorithm is connected to. QuantConnect doesn't support multi-account risk controls.

Brokerage

QuantConnect supports several brokerages . Each brokerage trades different asset classes and support different order types. LEAN doesn't allow placing orders that don't meet these requirements. For example, your algorithm cannot place a trailing stop order for most brokerages.

By default, LEAN uses the default brokerage model that suports all order types and all asset classes. To avoid issues with unsupported order types and asset classes when live trading, set the brokerage model to a brokerage that supports them.

```
self.set_brokerage_model(BrokerageName.TRADE_STATION)
```
PY

QuantConnect ensures your algorithm is always online, checking its status every few seconds. While a strategy is live it is also always connected to your brokerage, ensuring there is a route to quickly place trades when needed.

Trading and Orders

Position Sizing

Introduction

LEAN provides several methods to help you set specific portfolio weights for assets. These methods account for lot size and pre-calculated order fees.

Single Asset Targets

The `set_holdings` method calculates the number of asset units to purchase based on the portfolio weight you provide and then submits market orders . This method provides a quick way to set up a portfolio with a set of weights for assets. If you already have holdings, you may want to liquidate the existing holdings first to free up buying power.

```
 # Allocate 50% of buying power to IBM
self.set_holdings("IBM", 0.5)

# Allocate 50% of portfolio value to IBM, but liquidate other holdings first
self.set_holdings("IBM", 0.5, liquidate_existing_holdings=True)

# Provide a tag and order properties to the SetHoldings method
self.set_holdings(symbol, weight, liquidate_existing_holdings, tag, order_properties)
```

If the percentage you provide translates to an order quantity of 0 or to a small quantity that doesn't exceed the `minimum_order_margin_portfolio_percentage` algorithm setting , then the `set_holdings` method doesn't place an order and doesn't log anything.

Multiple Asset Targets

When you trade a weighted basket of assets, sometimes you must intelligently scale down existing positions before increasing allocations to other assets. If you call the `set_holdings` method with a list of `PortfolioTarget` objects, LEAN sorts the orders based on your position delta and then places the orders that reduce your position size in an asset before it places orders that increase your position size in an asset. When you call the `set_holdings` method with a list of `PortfolioTarget` objects, the decimal you pass to the `PortfolioTarget` constructor represents the portfolio weight. In this situation, don't use the `PortfolioTarget.percent` method.

```
# Purchase a portfolio of targets, processing orders intelligently.
# PortfolioTarget accepts a Symbol and decimal percent of buying power.
targets = [PortfolioTarget("SPY", 0.8), PortfolioTarget("IBM", 0.2)]
self.set_holdings(targets)
```

If you want the targets you pass to define the target portfolio state, enable the `liquidate_existing_holdings` argument.

```
self.set_holdings([PortfolioTarget("SPY", 0.8), PortfolioTarget("IBM", 0.2)], liquidate_existing_holdings=True)
```

The method also accepts tag and order properties arguments.

```
self.set_holdings(targets, liquidate_existing_holdings, tag, order_properties)
```

If the percentage you provide translates to an order quantity of 0 or to a small quantity that doesn't exceed the `minimum_order_margin_portfolio_percentage` algorithm setting , then the `set_holdings` method doesn't place an order and doesn't log anything.

Asynchronous Orders

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
self.set_holdings("IBM", 0.5, asynchronous=True);
```

When you set multiple asset targets, you likely want to send asynchronous orders.

```
targets = [PortfolioTarget("SPY", 0.8), PortfolioTarget("IBM", 0.2)]
self.set_holdings(targets, asynchronous=True)
```

Calculate Order Quantities

The `set_holdings` method uses market order to reach the target portfolio weight you provide. If you want to use a different order type, you need to specify the quantity to trade. To calculate the number of units to trade based on a portfolio weight, call the `calculate_order_quantity` method. The method calculates the quantity based on the current price of the asset and adjusts it for the fee model of the security. The target weight you provide is an unleveraged value. For instance, if you have 2x leverage and request a 100% weight, the method calculates the quantity that uses half of your available margin.

```
 # Calculate the fee-adjusted quantity of shares with given buying power
quantity = self.calculate_order_quantity("IBM", 0.4)
```

Buying Power Buffer

If you place a market on open order near the market close, the market can gap overnight to a worse open price. If the gap is large enough against your trade direction, you may not have sufficient buying power at the open to fill your trade. To ensure a high probability of order fills through market gaps and discontinuities, the `set_holdings` method assumes a 0.25% cash buffer. If LEAN rejects your orders due to buying power, widen the cash buffer through the algorithm `settings` property.

```
# Set the cash buffer to 5%
self.settings.free_portfolio_value_percentage = 0.05

# Set the cash buffer to $10,000
self.settings.free_portfolio_value = 10000
```

If you use `free_portfolio_value_percentage` , you must set it in the `initialize` or `post_initialize` event handler. If you use `free_portfolio_value` , you must set it after the `post_initialize` event handler.

Examples

The following examples demonstrate some common practices for position sizing.

Example 1: Covered Call

PY

```
class PositionSizingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10_000_000)

        # Seed the price to ensure securities have information for margin calculations.
        self.settings.seed_initial_prices = True

        # Request SPY data for trading. We use raw price data to compare the strike price fairly for ATM contract selection.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set a scheduled event to rebalance the covered call positions.
        self.schedule.on(
            self.date_rules.week_start(self.spy),
            self.time_rules.after_market_open(self.spy, 1),
            self.rebalance
        )

    def rebalance(self) -> None:
        option_chain = self.option_chain(self.spy)
        # Filter for the calls within 1-week expiry.
        expiry = max(x.expiry for x in option_chain if x.expiry <= self.time + timedelta(6))
        filtered = [x for x in option_chain if x.right == OptionRight.CALL and x.expiry == expiry]
        # Select the ATM call.
        atm_call = sorted(filtered, key=lambda x: abs(x.strike - self.securities[self.spy].price))[0]
        atm_call = self.add_option_contract(atm_call).symbol

        # Calculate the order size for the covered call.
        quantity = int(self.calculate_order_quantity(self.spy, 0.5) // 100)
        # Covered call involves 100 shares of SPY and -1 ATM call contract.
        if quantity > 0:
            self.combo_market_order(
                [Leg.create(self.spy, 100), Leg.create(atm_call, -1)],
                quantity,
                asynchronous=True
            )
```

Trading and Orders

Liquidating Positions

Introduction

The `liquidate` method lets you liquidate individual assets or your entire portfolio. The method creates market orders to close positions and returns the tickets of the liquidation orders. If you have pending open orders for the security when you call `liquidate` , LEAN tries to cancel them. The `liquidate` method works for all asset classes, except Crypto. To liquidate Crypto positions, see Crypto Trades .

Liquidate Individual Positions

To liquidate your holdings in an individual security, call the `liquidate` method and provide a ticker or `Symbol` .

```
# Liquidate all IBM in your portfolio
order_tickets = self.liquidate("IBM")
```

You can pass an order tag and properties to the `liquidate` method.

```
order_properties = OrderProperties()
order_properties.time_in_force = TimeInForce.DAY
order_tickets = self.liquidate("AAPL", tag="Liquidated", order_properties=order_properties)
```

Liquidate All Positions

To liquidate all of the positions in your portfolio, call the `liquidate` method without any ticker of `Symbol` arguments.

```
// Liquidate your entire portfolio
order_tickets = self.liquidate()
```

You can pass an order tag and properties to the `liquidate` method.

```
order_properties = OrderProperties()
order_properties.time_in_force = TimeInForce.DAY
order_tickets = self.liquidate(tag="Liquidated", order_properties=order_properties)
```

Asynchronous Liquidations

When you trade a large portfolio of assets, you may want to send orders in batches and not wait for the response of each one. To send asynchronous orders , set the `asynchronous` argument to `True` .

```
order_tickets = self.liquidate(asynchronous=True, order_properties=order_properties);
```

Enable and Disable Liquidations

By default, the `liquidate` method is functional. To enable and disable it, set the `liquidate_enabled` algorithm setting .

```
# Disable liquidations
self.settings.liquidate_enabled = False

# Enable liquidations
self.settings.liquidate_enabled = True
```

Market Closed Considerations

If you liquidate your positions when the market is closed, LEAN converts the orders into market on open orders. If your brokerage doesn't support market on open orders , the order is invalid.

Examples

The following examples demonstrate some common practices for liquidating positions.

Example 1: Liquidate Universe Removal

The following algorithm holds equal positions for the top 10 liquid universe to follow their hype while updating weekly. When a stock is removed from the universe, we liquidate the position in the `on_securities_changed` method.

```py
class LiquidatingPositionsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Update the universe weekly to allow time to capitalize on the hype.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Filter for the 10 most popular stocks to invest the hype.
        self._universe = self.add_universe(self.universe.top(10))

        # Set a schedule event to rebalance weekly.
        self.schedule.on(
            self.date_rules.week_start("SPY"),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Evenly invest in the universe members to dissipate the capital risk equally.
        count = len(self._universe.members)
        targets = [PortfolioTarget(x.key, 1 / count) for x in self._universe.members]
        self.set_holdings(targets)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        # Liquidate the ones leaving the universe since they are not the most popular.
        # Note that the liquidation will be market-on-open orders due to the time of securities removal.
        for removed in changes.removed_securities:
            self.liquidate(removed.symbol)
```

Example 2: Liquidate Existing Positions

The example below trades the same strategy as the previous one. Still, instead of liquidating the positions when the stocks leave the universe, we liquidate the position using the `liquidate_existing_holdings` argument in the `set_holdings` . Note that the time of the liquidation will be slightly delayed due to the time of the scheduled event.

```py
class LiquidatingPositionsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Update the universe weekly to allow time to capitalize on the hype.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Filter for the 10 most popular stocks to invest the hype.
        self._universe = self.add_universe(self.universe.top(10))

        # Set a scheduled event to rebalance weekly.
        self.schedule.on(
            self.date_rules.week_start("SPY"),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Evenly invest in the universe members to dissipate the capital risk equally.
        count = len(self._universe.members)
        targets = [PortfolioTarget(x.key, 1 / count) for x in self._universe.members]
        # We can use `liquidate_existing_holdings` arguments to liquidate the ones not in the universe anymore.
        self.set_holdings(targets, liquidate_existing_holdings=True)
```

Example 3: Liquidate Upon Assignment

The following algorithm sells SPY ATM puts weekly to earn the premium when SPY price is rising. In the case where the price of SPY drops and the put is being assigned, we liquidate the SPY position in the `on_assignment_order_event` .

PY

```
class LiquidatingPositionsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price with the last known price to ensure the underlying price data is available on initial option contract filtering.
        self.settings.seed_initial_prices = True

        # Request SPY data for option selection using its price.
        # Use raw data normalization mode since we need to compare the SPY price with the strike price.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set a scheduled event to rebalance weekly.
        self.schedule.on(
            self.date_rules.week_start(),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Select the 1-week ATM put SPY option to short.
        option_chain = self.option_chain(self.spy)
        expiry = max(x.expiry for x in option_chain if x.expiry < self.time + timedelta(6))
        atm_put = sorted([x for x in option_chain if x.expiry == expiry and x.right == OptionRight.PUT],
                         key=lambda x: abs(x.strike - self.securities[self.spy].price))[0]
        atm_put_symbol = self.add_option_contract(atm_put).symbol

        # Sell the ATM put option to earn the premium.
        self.sell(atm_put_symbol, 2)

    def on_assignment_order_event(self, assignment_event: OrderEvent) -> None:
        # Liquidate any assigned positions of SPY since we only earn the premium.
        self.liquidate(self.spy)
```

Trading and Orders

Crypto Trades

Introduction

All fiat and Crypto currencies are individual assets. When you buy a pair like BTCUSDT, you trade USDT for BTC. In this case, LEAN removes some USDT from your portfolio cash book and adds some BTC. The virtual pair BTCUSDT represents your position in the trade, but the virtual pair doesn't actually exist. It simply represents an open trade.

Place Trades

When you place Crypto trades, don't use the `calculate_order_quantity` or `set_holdings` methods. Instead, calculate the order quantity based on the currency amounts in your cash book and place manual orders.

The following code snippet demonstrates how to allocate 90% of your portfolio to BTC.

```
PY
def on_data(self, data: Slice):
    self.set_crypto_holdings(self._symbol, .9)

def set_crypto_holdings(self, symbol, percentage):
    crypto = self.securities[symbol]
    base_currency = crypto.base_currency

    # Calculate the target quantity in the base currency
    target_quantity = percentage * (self.portfolio.total_portfolio_value - self.settings.free_portfolio_value) /
base_currency.conversion_rate
    quantity = target_quantity - base_currency.amount

    # Round down to observe the lot size
    lot_size = crypto.symbol_properties.lot_size
    quantity = round(quantity / lot_size) * lot_size

    if self.is_valid_order_size(crypto, quantity):
        self.market_order(symbol, quantity)

# Brokerages have different order size rules
# Binance considers the minimum volume (price x quantity):
def is_valid_order_size(self, crypto, quantity):
    return abs(crypto.price * quantity) > crypto.symbol_properties.minimum_order_size
```

The preceding example doesn't take into account order fees. You can add a 0.1% buffer to accommodate it.

The following example demonstrates how to form an equal-weighted Crypto portfolio and stay within the cash buffer .

```
PY
def on_data(self, data: Slice):
    percentage = (1 - self.settings.free_portfolio_value_percentage) / len(self.symbols);
    for symbol in self.symbols:
        self.set_crypto_holdings(symbol, percentage)
```

You can replace the `settings.free_portfolio_value_percentage` for a class variable (e.g. `self.cash_buffer`).

When you place Crypto trades, ensure you have a sufficient balance of the base or quote currency before each trade. If you hold multiple assets and you want to put all of your capital into BTCUSDT, you need to first convert all your non-BTC assets into USDT and then purchase BTCUSDT.

By default, the account currency is USD and your starting cash is 100,000. Some Crypto exchanges don't support fiat currencies, which means you can't convert the 100,000 USD to the pair's quote currency . In this case, set your account currency to the quote currency with a positive quantity.

```
PY
self.set_account_currency("USDT", 1000)   # Set the account currency to Tether and its quantity to 1,000 USDT
```

For a full example of placing crypto trades, see the BasicTemplateCryptoAlgorithm .

Liquidate Positions

If you use the `liquidate` method to liquidate a Crypto position, it only liquidates the quantity of the virtual pair. Since the virtual pair BTCUSDT may not represent all of your BTC holdings, don't use the `liquidate` method to liquidate Crypto positions. Instead, calculate the order quantity based on the currency amounts in your cash book and place manual orders. The following code snippet demonstrates how to liquidate a BTCUSDT position.

```
PY
def on_data(self, data: Slice):
    self.liquidate_crypto(self._symbol)

def liquidate_crypto(self, symbol):
    crypto = self.securities[symbol]
    base_currency = crypto.base_currency

    # Avoid negative amount after liquidate
    quantity = min(crypto.holdings.quantity, base_currency.amount)

    # Round down to observe the lot size
    lot_size = crypto.symbol_properties.lot_size
    quantity = (round(quantity / lot_size) - 1) * lot_size

    if self.is_valid_order_size(crypto, quantity):
        self.market_order(symbol, -quantity)
```

The order fees don't respect the lot size. When you try to liquidate a position, the absolute value of the base currency quantity can be less than the lot size and greater than zero. In this case, your algorithm holds a position that you can't liquidate, yet `self.portfolio[symbol].invested` is `False` . For more information about `self.portfolio[symbol].invested` , see Holdings Status . The following code snippet demonstrates how to determine if you can liquidate a position:

```
PY
def on_data(self, data: Slice):
    crypto = self.securities[self._symbol]
    if abs(crypto.holdings.quantity) > crypto.symbol_properties.lot_size:
        self.liquidate_crypto(self._symbol)
```

Examples

The following examples demonstrate some common practices for crypto holdings.

Example 1: ETH-BTC Proxy Trading

The following algorithm trades trend-following of the ETHBTC crypto pair using a 20-day EMA indicator. To reduce friction (e.g. slippage), we trade the more liquid and popular BTCUSDT and ETHUSDT pair instead. For example, if ETHBTC is above EMA (uptrend), we sell all BTCUSDT holdings and buy ETHUSDT with the same USDT value, going flat on BTC while holding ETH.

PY

```
class CryptoHoldingsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_account_currency("USDT", 1000000)


        # We would like to trade the EMA cross between 2 popular cryptos: BTC & ETH,
        # so we request ETHBTC data to find trading opportunities.
        self.ethbtc = self.add_crypto("ETHBTC", Resolution.DAILY, market=Market.COINBASE).symbol
        # Trade through BTCUSDT & ETHUSDT, though, since stable coin trades have lower friction costs and higher liquidity.
        btcusdt = self.add_crypto("BTCUSDT", Resolution.DAILY, market=Market.COINBASE)
        ethusdt = self.add_crypto("ETHUSDT", Resolution.DAILY, market=Market.COINBASE)
        self.btcusdt = btcusdt.symbol
        self.ethusdt = ethusdt.symbol

        # Simulate an account with various crypto cash through holdings.
        # Set the cash book directly with the crypto amounts
        btcusdt.holdings.set_holdings(btcusdt.price, 5)
        btcusdt.base_currency.add_amount(5)
        ethusdt.holdings.set_holdings(ethusdt.price, 22.5)
        ethusdt.base_currency.add_amount(22.5)

        # Add automatic updating of the EMA indicator for trend trade signal emission.
        self._ema = self.ema(self.ethbtc, 20, Resolution.DAILY)
        # Warm up the indicator for its readiness usage immediately.
        self.warm_up_indicator(self.ethbtc, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.ethbtc)
        btc = slice.bars.get(self.btcusdt)
        eth = slice.bars.get(self.ethusdt)

        if bar and self._ema.is_ready and btc and eth:
            ema = self._ema.current.value
            btc_holdings = self.portfolio.cash_book["BTC"].amount
            eth_holdings = self.portfolio.cash_book["ETH"].amount

            # ETHBTC's current price is higher than EMA, suggesting an uptrend.
            if bar.close > ema and not self.portfolio[self.btcusdt].is_short:
                # To follow the up trend of ETHBTC, sell ALL BTCUSDT and buy ETHUSDT with the same value.
                if btc_holdings > 0:
                    btc_value = btc_holdings * btc.close
                    # Sell all BTC
                    self.market_order(self.btcusdt, -btc_holdings)
                    # Buy ETH with the same USDT value
                    eth_size = btc_value / eth.close
                    self.market_order(self.ethusdt, eth_size)
            # ETHBTC's current price is below the EMA, suggesting a downtrend.
            elif bar.close < ema and not self.portfolio[self.btcusdt].is_long:
                # To follow the downtrend of ETHBTC, sell ALL ETHUSDT and buy BTCUSDT with the same value.
                if eth_holdings > 0:
                    eth_value = eth_holdings * eth.close
                    # Sell all ETH
                    self.market_order(self.ethusdt, -eth_holdings)
                    # Buy BTC with the same USDT value
                    btc_size = eth_value / btc.close
                    self.market_order(self.btcusdt, btc_size)
```

Trading and Orders

Option Strategies

You can implement any of the following Option strategies in an algorithm. Click one to learn more.

Bear Call Spread

Bear Put Spread

Bull Call Spread

Bull Put Spread

Long Call Butterfly

Short Call Butterfly

Long Put Butterfly

Short Put Butterfly

Long Call Calendar Spread

Short Call Calendar Spread

Long Put Calendar Spread

Short Put Calendar Spread

Covered Call

Covered Put

Long Iron Butterfly

Short Iron Butterfly

Long Iron Condor

Short Iron Condor

Naked Call

Naked Put

Protective Call

Protective Put

Protective Collar

Long Straddle

Short Straddle

Long Strangle

Short Strangle

Conversion

Reverse Conversion

Long Box Spread

Short Box Spread

Long Jelly Roll

Short Jelly Roll

Bear Call Ladder

Bear Put Ladder

Bull Call Ladder

Bull Put Ladder

Long Call Backspread

Short Call Backspread

Long Put Backspread

Short Put Backspread

See Also

Equity Options
Future Options
Index Options

Option Strategies

Bear Call Spread

Introduction

Bear call spread , also known as short call spread , consists of selling an ITM call and buying an OTM call. Both calls have the same underlying Equity and the same expiration date. The ITM call serves as a hedge for the OTM call. The bear call spread profits from a drop in underlying asset price.

Implementation

Follow these steps to implement the bear call spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_spread(30, 5))
```

   The `call_spread` filter narrows the universe down to just the two contracts you need to form a bear call spread.

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls == 0:
        return

    # Select the ITM and OTM contract strike prices from the remaining contracts
    strikes = [x.strike for x in calls]
    otm_strike = max(strikes)
    itm_strike = min(strikes)
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bear_call_spread` method with the details of each leg and then pass the result to the `buy` method.

```
PY
option_strategy = OptionStrategies.bear_call_spread(self._symbol, itm_strike, otm_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
itm_call = [x for x in calls if x.strike == itm_strike][0]
otm_call = [x for x in calls if x.strike == otm_strike][0]

legs = [
    Leg.create(itm_call.symbol, -1),
    Leg.create(otm_call.symbol, 1)
]
self.combo_market_order(legs, 1)
```

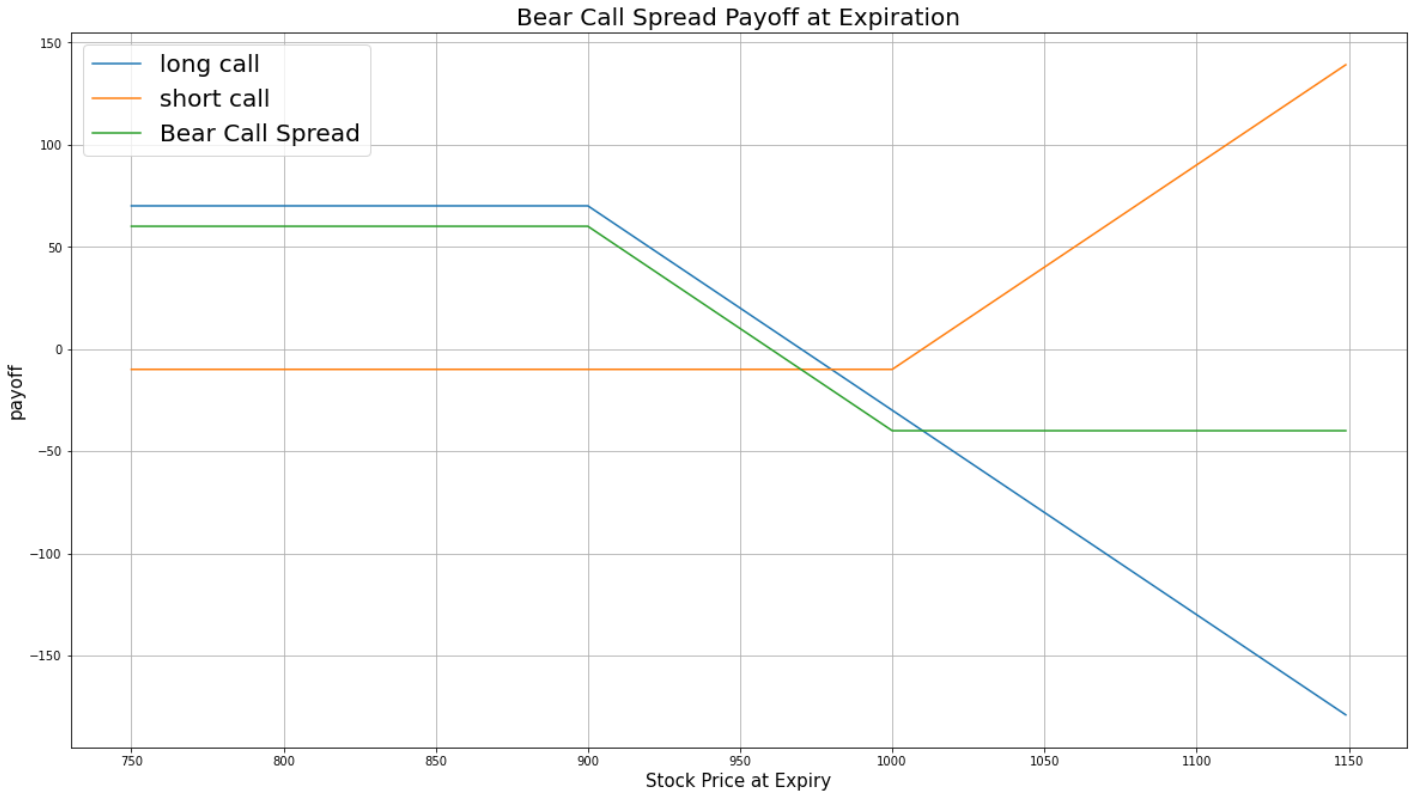
Strategy Payoff

The bear call spread is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ P_T & = & (C^{OTM}_T - C^{ITM}_T + C^{ITM}_0 - C^{OTM}_0)\times m - fee\\ \end{array} \begin{array}{rcll} \textrm{where} & C^{OTM}_T & = & \textrm{OTM call value at time T}\\ & C^{ITM}_T & = & \textrm{ITM call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM call strike price}\\ & K^{ITM} & = & \textrm{ITM call strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{ITM}_0 & = & \textrm{ITM call value at position opening (debit paid)}\\ & C^{OTM}_0 & = & \textrm{OTM call value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price is $1,078.92 at expiration. You sold an ITM call at a strike of $1,125.00 for $57.80 and bought an OTM call at a strike of $1,197.50 for $26.90 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit you receive from opening the trade, $C^{ITM}_0 - C^{OTM}_0$. If the price declines, both calls expire worthless.

The maximum loss is $K^{OTM} - K^{ITM} + C^{ITM}_0 - C^{OTM}_0$, which occurs when the underlying price is above the strike prices of both call Option contracts.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM call	26.90	1197.50
ITM call	57.80	1125.00
Underlying Equity at expiration	1078.92	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ & = & (1078.92-1197.50)^{+}\\ & = & 0\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ & = & (1078.92-1125.00)^{+}\\ & = & 0\\ P_T & = & (C^{OTM}_T - C^{ITM}_T + C^{ITM}_0 - C^{OTM}_0)\times m - fee\\ & = & (0-0+57.80-26.90)\times100-1.00\times2\\ & = & 3088\\ \end{array}$$

So, the strategy profits $3,088.

The following algorithm implements a bear call spread Option strategy:

```py
class BearCallSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol
        option.set_filter(
            lambda universe: universe.call_spread(30, 5)
        )

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Get the furthest expiry date of the contracts
        expiry = sorted(chain, key=lambda x: x.expiry, reverse=True)[0].expiry

        # Select the call Option contracts with the furthest expiry
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if len(calls) == 0:
            return

        # Select the ITM and OTM contracts
        ordered_calls = sorted(calls, key=lambda x: x.strike)
        itm_call = ordered_calls[0]
        otm_call = ordered_calls[-1]

        # Place the order
        option_strategy = OptionStrategies.bear_call_spread(
            self._symbol, itm_call.strike, otm_call.strike, expiry
        )
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionBearCallSpreadAlgorithm.py Python

Option Strategies

Bear Put Spread

Introduction

Bear put spread , also known as short put spread , consists of buying an ITM put and selling an OTM put. Both puts have the same underlying Equity and the same expiration date. The OTM put serves as a hedge for the ITM put. The bear put spread profits from a decline in underlying asset price.

Implementation

Follow these steps to implement the bear put spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_spread(30, 5))
```

   The `put_spread` filter narrows the universe down to just the two contracts you need to form a bear put spread.

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if not puts == 0:
        return

    # Select the ITM and OTM contract strike prices from the remaining contracts
    strikes = [x.strike for x in puts]
    otm_strike = min(strikes)
    itm_strike = max(strikes)
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bear_put_spread` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.bear_put_spread(self._symbol, itm_strike, otm_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
itm_put = [x for x in puts if x.strike == itm_strike][0]
otm_put = [x for x in puts if x.strike == otm_strike][0]

legs = [
    Leg.create(itm_put.symbol, 1),
    Leg.create(otm_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The bear put spread is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ P_T & = & (P^{ITM}_T - P^{OTM}_T + P^{OTM}_0 - P^{ITM}_0)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & P^{OTM}_T & = & \textrm{OTM put value at time T}\\ & P^{ITM}_T & = & \textrm{ITM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM put strike price}\\ & K^{ITM} & = & \textrm{ITM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & P^{ITM}_0 & = & \textrm{ITM put value at position opening (debit paid)}\\ & P^{OTM}_0 & = & \textrm{OTM put value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price is $829.08 at expiration. You sold an OTM put at a strike of $767.50 for $4.60 and bought an ITM put at a strike of $835.00 for $40.00 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{ITM} - K^{OTM} + P^{OTM}_0 - P^{ITM}_0$. If the underlying price is below than the strike prices of both put Option contracts, they are worth $(K - S_T)$ at expiration.

The maximum loss is the net debit you paid to open the position, $P^{OTM}_0 - P^{ITM}_0$.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM put	4.60	767.50
ITM put	40.00	835.00
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ & = & (767.50-829.08)^{+}\\ & = & 0\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ & = & (835.00-829.08)^{+}\\ & = & 5.92\\ P_T & = & (P^{ITM}_T - P^{OTM}_T + P^{OTM}_0 - P^{ITM}_0)\times m - fee\\ & = & (5.92-0+4.60-40.00)\times100-1.00\times2\\ & = & -2950\\ \end{array}$$

So, the strategy loses $2,950.

The following algorithm implements a bear put spread strategy:

```py
class BearPutSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_spread(30, 5)

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested: return

        # Get the OptionChain
        chain = slice.option_chains.get(self.symbol, None)
        if not chain: return

        # Get the furthest expiry date of the contracts
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # Select the put Option contracts with the furthest expiry
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # Select the ITM and OTM contract strike prices from the remaining contracts
        put_strikes = sorted([x.strike for x in puts])
        otm_strike = put_strikes[0]
        itm_strike = put_strikes[-1]

        option_strategy = OptionStrategies.bear_put_spread(self.symbol, itm_strike, otm_strike, expiry)
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionBearPutSpreadAlgorithm.py Python

Option Strategies

Bull Call Spread

Introduction

Bull call spread , also known as long call spread , consists of buying an ITM call and selling an OTM call. Both calls have the same underlying Equity and the same expiration date. The OTM call serves as a hedge for the ITM call. The bull call spread profits from a rise in underlying asset price.

Implementation

Follow these steps to implement the bull call spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_spread(30, 5))
```

   The `call_spread` filter narrows the universe down to just the two contracts you need to form a bull call spread.

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls == 0:
        return

    # Select the ITM and OTM contract strike prices from the remaining contracts
    strikes = [x.strike for x in calls]
    otm_strike = max(strikes)
    itm_strike = min(strikes)
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bull_call_spread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.bull_call_spread(self._symbol, itm_strike, otm_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
itm_call = [x for x in calls if x.strike == itm_strike][0]
otm_call = [x for x in calls if x.strike == otm_strike][0]

legs = [
    Leg.create(itm_call.symbol, 1),
    Leg.create(otm_call.symbol, -1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The bull call spread is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ P_T & = & (C^{ITM}_T - C^{OTM}_T + C^{OTM}_0 - C^{ITM}_0)\times m - fee\\ \end{array} \begin{array}{rcll} \textrm{where} & C^{OTM}_T & = & \textrm{OTM call value at time T}\\ & C^{ITM}_T & = & \textrm{ITM call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM call strike price}\\ & K^{ITM} & = & \textrm{ITM call strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{ITM}_0 & = & \textrm{ITM call value at position opening (debit paid)}\\ & C^{OTM}_0 & = & \textrm{OTM call value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price is $829.08 at expiration. You sold an OTM call at a strike of $835.00 for $3.00 and bought an ITM call at a strike of $767.50 for $41.00 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{OTM} - K^{ITM} + C^{OTM}_0 - C^{ITM}_0$. If the underlying price increases to exceed both strikes at expiration, both calls are worth $(S_T - K)$ at expiration.

The maximum loss is the net debit you paid to open the position, $C^{OTM}_0 - C^{ITM}_0$.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM call	3.00	835.00
ITM call	41.00	767.50
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ & = & (829.08-835.00)^{+}\\ & = & 0\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ & = & (829.08-767.50)^{+}\\ & = & 61.58\\ P_T & = & (C^{ITM}_T - C^{OTM}_T + C^{OTM}_0 - C^{ITM}_0)\times m - fee\\ & = & (61.58-0+3.00-41.00)\times100-1.00\times2\\ & = & 2356\\ \end{array}$$

So, the strategy profits $2,356.

The following algorithm implements a bull call spread strategy:

```py
class BullCallSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.call_spread(30, 5)

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested: return

        # Get the OptionChain
        chain = slice.option_chains.get(self.symbol, None)
        if not chain: return

        # Get the furthest expiration date of the contracts
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # Select the call Option contracts with the furthest expiry
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if len(calls) == 0: return

        # Select the ITM and OTM contract strike prices from the remaining contracts
        call_strikes = sorted([x.strike for x in calls])
        itm_strike = call_strikes[0]
        otm_strike = call_strikes[-1]

        option_strategy = OptionStrategies.bull_call_spread(self.symbol, itm_strike, otm_strike, expiry)
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionBullCallSpreadAlgorithm.py Python

Option Strategies

Bull Put Spread

Introduction

Bull put spread , also known as long put spread , consists of buying an OTM put and selling an ITM put. Both puts have the same underlying Equity and the same expiration date. The OTM put serves as a hedge for the ITM put. The bull put spread profits from a rise in underlying asset price.

Implementation

Follow these steps to implement the bull put spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_spread(30, 5))
```

   The `put_spread` filter narrows the universe down to just the two contracts you need to form a bull put spread.

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if not puts == 0:
        return

    # Select the ITM and OTM contract strike prices from the remaining contracts
    strikes = [x.strike for x in puts]
    otm_strike = min(strikes)
    itm_strike = max(strikes)
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bull_put_spread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.bull_put_spread(self._symbol, itm_strike, otm_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
itm_put = [x for x in puts if x.strike == itm_strike][0]
otm_put = [x for x in puts if x.strike == otm_strike][0]

legs = [
    Leg.create(itm_put.symbol, -1),
    Leg.create(otm_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

This is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ P_T & = & (P^{OTM}_T - P^{ITM}_T + P^{ITM}_0 - P^{OTM}_0)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & P^{OTM}_T & = & \textrm{OTM put value at time T}\\ & P^{ITM}_T & = & \textrm{ITM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM put strike price}\\ & K^{ITM} & = & \textrm{ITM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & P^{ITM}_0 & = & \textrm{ITM put value at position opening (credit received)}\\ & P^{OTM}_0 & = & \textrm{OTM put value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price is $829.08 at expiration. You sold an ITM put at a strike of $835.00 for $35.50 and bought an OTM put at a strike of $767.50 for $5.70 .

The following chart shows the payoff at expiration:

The maximum profit is the net credit you received when opening the position, $P^{ITM}_0 - P^{OTM}_0$. If the underlying price is higher than the strike prices of both put contracts at expiration, both puts expire worthless.

The maximum loss is $K^{OTM} - K^{ITM} + P^{ITM}_0 - P^{OTM}_0$.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM put	5.70	767.50
ITM put	35.50	835.00
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ & = & (767.50-829.08)^{+}\\ & = & 0\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ & = & (835.00-829.08)^{+}\\ & = & 5.92\\ P_T & = & (P^{OTM}_T - P^{ITM}_T + P^{ITM}_0 - P^{OTM}_0)\times m - fee\\ & = & (0-5.92+35.50-5.70)\times100-1.00\times2\\ & = & 2386\\ \end{array}$$

So, the strategy profits $2,386.

The following algorithm implements a bull put spread strategy:

```py
class BullPutSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_spread(30, 5)

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested: return

        # Get the OptionChain
        chain = slice.option_chains.get(self.symbol, None)
        if not chain: return

        # Get the furthest expiration date of the contracts
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # Select the put Option contracts with the furthest expiry
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # Select the ITM and OTM contract strikes from the remaining contracts
        put_strikes = sorted([x.strike for x in puts])
        otm_strike = put_strikes[0]
        itm_strike = put_strikes[-1]

        option_strategy = OptionStrategies.bull_put_spread(self.symbol, itm_strike, otm_strike, expiry)
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionBullPutSpreadAlgorithm.py Python

Option Strategies

Long Call Butterfly

Introduction

The long call butterfly strategy is the combination of a bull call spread and a bear call spread . In the call butterfly, all of the calls should have the same underlying Equity, the same expiration date, and the same strike price distance between the ITM-ATM and OTM-ATM call pairs. The long call butterfly consists of a long ITM call, a long OTM call, and 2 short ATM calls. This strategy profits from low volatility in the underlying Equity price.

Implementation

Follow these steps to implement the long call butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_butterfly(30, 5))
```

   The `call_spread` filter narrows the universe down to just the three contracts you need to form a long call butterfly.

2. In the `on_data` method, select the contracts of the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain.
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the furthest expiry date of the contracts.
    expiry = max([x.expiry for x in chain])

    # Select the call Option contracts with the furthest expiry.
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if len(calls) == 0:
        return

    # Select the target contracts.
    atm_call = sorted(calls, key=lambda x: abs(x.strike - chain.underlying.price))[0]
    itm_call = sorted(calls, key=lambda x: x.strike)[-2]
    otm_call = [x for x in calls if x.strike == atm_call.strike * 2 - itm_call.strike][0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.butterfly_call` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.butterfly_call(self._symbol, itm_call.strike, atm_call.strike, otm_call.strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(atm_call.symbol, -2),
    Leg.create(itm_call.symbol, 1),
    Leg.create(otm_call.symbol, 1)
]
self.combo_market_order(legs, 1)
```

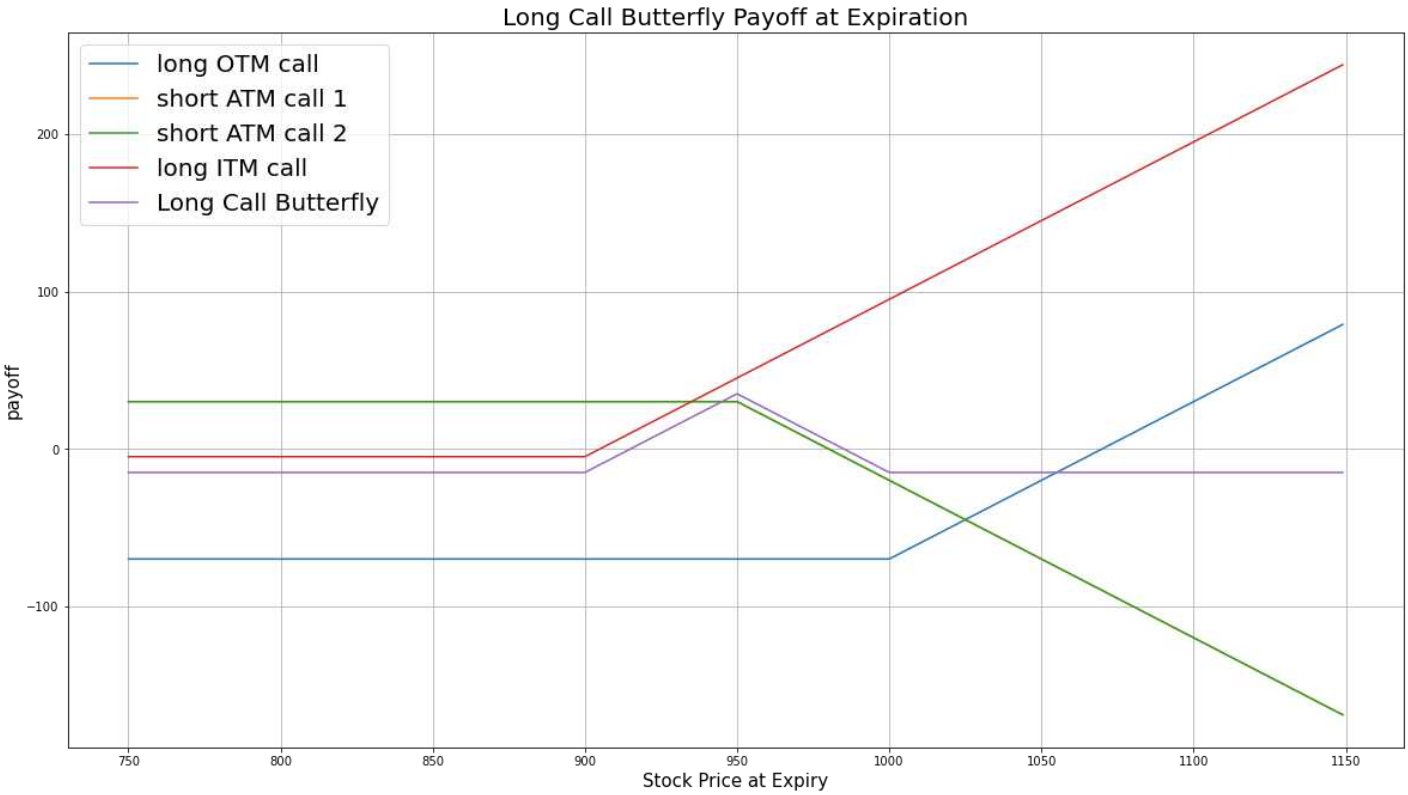
Strategy Payoff

The long call butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T &=& (S_T - K^{OTM})^{+}\\ C^{ITM}_T &=& (S_T - K^{ITM})^{+}\\ C^{ATM}_T &=& (S_T - K^{ATM})^{+}\\ P_T &=& (C^{OTM}_T + C^{ITM}_T - 2\times C^{ATM}_T + 2\times C^{ATM}_0 - C^{ITM}_0 - C^{OTM}_0)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & C^{OTM}_T &= \textrm{OTM call value at time T}\\ & C^{ITM}_T &= \textrm{ITM call value at time T}\\ & C^{ATM}_T &= \textrm{ATM call value at time T}\\ & S_T &= \textrm{Underlying asset price at time T}\\ & K^{OTM} &= \textrm{OTM call strike price}\\ & K^{ITM} &= \textrm{ITM call strike price}\\ & K^{ATM} &= \textrm{ATM call strike price}\\ & P_T &= \textrm{Payout total at time T}\\ & C^{ITM}_0 &= \textrm{ITM call value at position opening (debit paid)}\\ & C^{OTM}_0 &= \textrm{OTM call value at position opening (debit paid)}\\ & C^{ATM}_0 &= \textrm{OTM call value at position opening (credit received)}\\ & m &= \textrm{Contract multiplier}\\ & T &= \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $829.08 . You bought an OTM call at a strike of $767.50 for $4.90 , sold 2 ATM calls at a strike of $800.00 for $15.00 each, and bought an ITM call at a strike of $832.50 for $41.00 .

The following chart shows the payoff at expiration:



The maximum profit is $K^{ATM} - K^{ITM} + 2\times C^{ATM}_0 - C^{ITM}_0 - C^{OTM}_0$. It occurs when the underlying price is the same price at expiration as it was when opening the position and the payouts of the bull and bear call spreads are at their maximum.

The maximum loss is the net debit paid: $2\times C^{ATM}_0 - C^{ITM}_0 - C^{OTM}_0$. It occurs when the underlying price is less than ITM strike or greater than OTM strike at expiration.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the long call butterfly:

Asset	Price ($)	Strike ($)
OTM call	4.90	767.50
ATM call	15.00	800.00
ITM call	41.00	832.50
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ & = & (767.50-829.08)^{+}\\ & = & 0\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ & = & (832.50-829.08)^{+}\\ & = & 3.42\\ C^{ATM}_T & = & (S_T - K^{ATM})^{+}\\ & = & (800.00-829.08)^{+}\\ & = & 0\\ P_T & = & (C^{OTM}_T + C^{ITM}_T - 2\times C^{ATM}_T + 2\times C^{ATM}_0 - C^{ITM}_0 - C^{OTM}_0)\times m - fee\\ & = & (0+3.42-0\times2-4.90-41.00+15.00\times2)\times100-1.00\times4\\ & = & -1252 \end{array}

So, the strategy loses $1,252.

The following algorithm implements a long call butterfly Option strategy:

```py
class LongCallButterflyStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.strikes(-15, 15).expiration(timedelta(0), timedelta(31))

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # sorted the optionchain by expiration date and choose the furthest date
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # filter the call options from the contracts which expire on the furthest expiration date in the option chain.
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if len(calls) == 0: return

        # sort the call options with the same expiration date according to their strike price.
        call_strikes = sorted([x.strike for x in calls])

        # get at-the-money strike
        atm_strike = sorted(calls, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # Get the distance between lowest strike price and ATM strike, and highest strike price and ATM strike.
        # Get the lower value as the spread distance as equidistance is needed for both side.
        spread = min(abs(call_strikes[0] - atm_strike), abs(call_strikes[-1] - atm_strike))

        # select the strike prices for forming the option legs
        itm_strike = atm_strike - spread
        otm_strike = atm_strike + spread

        option_strategy = OptionStrategies.call_butterfly(self.symbol, otm_strike, atm_strike, itm_strike, expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
        # self.sell(option_strategy, 1) if short call butterfly
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionCallButterflyAlgorithm.py Python

Option Strategies

Short Call Butterfly

Introduction

The short call butterfly strategy is the combination of a bull call spread and a bear call spread . In the call butterfly, all of the calls should have the same underlying Equity, the same expiration date, and the same strike price distance between the ITM-ATM and OTM-ATM call pairs. The short call butterfly consists of a short ITM call, a short OTM call, and 2 long ATM calls. This strategy profits from high volatility in the underlying Equity price.

Implementation

Follow these steps to implement the short call butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_butterfly(30, 5))
```

   The `call_butterfly` filter narrows the universe down to just the three contracts you need to form a short call butterfly.

2. In the `on_data` method, select the contracts of the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the furthest expiry date of the contracts
    expiry = max([x.expiry for x in chain])

    # Select the call Option contracts with the furthest expiry
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if len(calls) == 0:
        return

    # Select the ATM, ITM and OTM contracts from the remaining contracts
    atm_call = sorted(calls, key=lambda x: abs(x.strike - chain.underlying.price))[0]
    itm_call = sorted(calls, key=lambda x: x.strike)[1]
    otm_call = [x for x in calls if x.strike == atm_call.strike * 2 - itm_call.strike][0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.short_butterfly_call` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.short_butterfly_call(self._symbol, otm_call.strike, atm_call.strike, itm_call.strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(atm_call.symbol, 2),
    Leg.create(itm_call.symbol, -1),
    Leg.create(otm_call.symbol, -1)
]
self.combo_market_order(legs, 1)
```

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

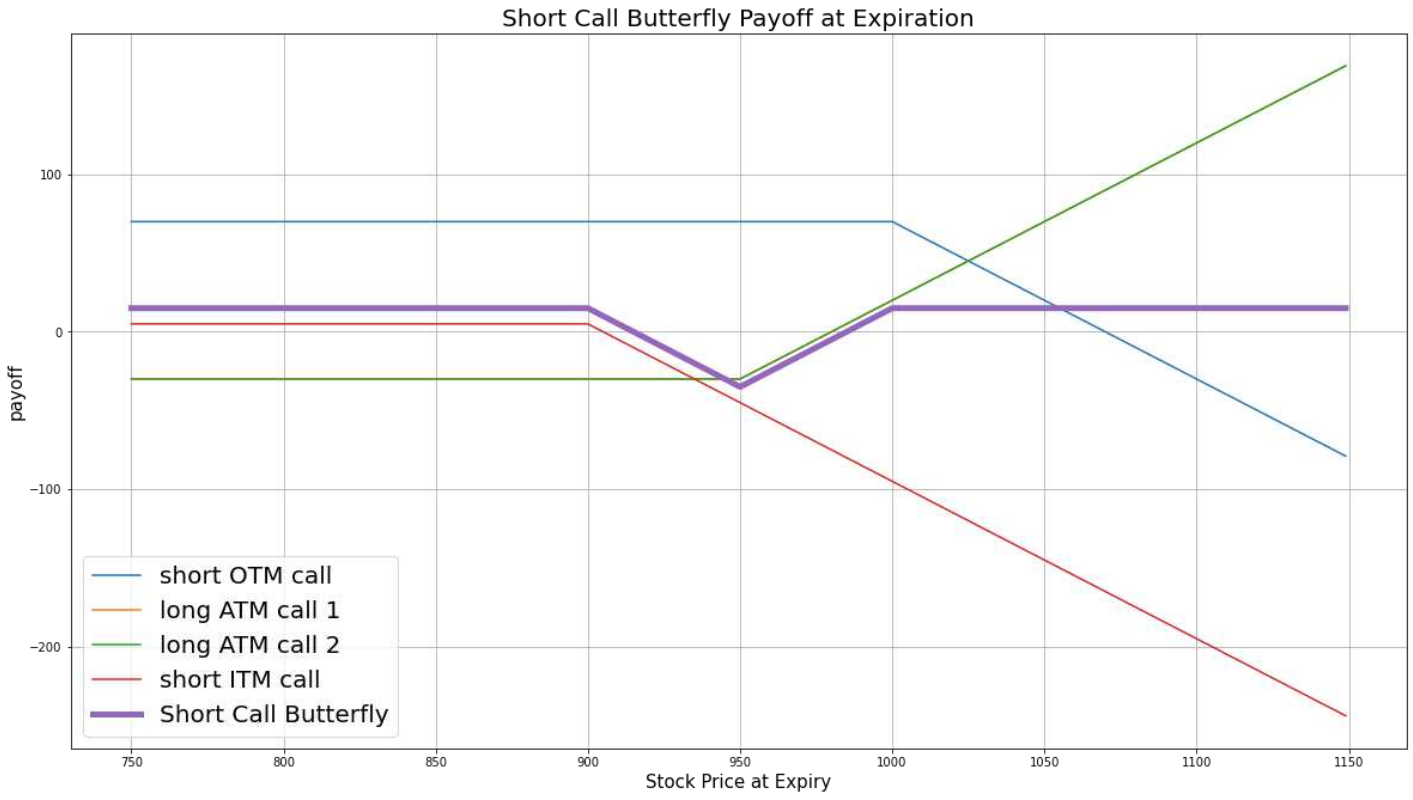
Strategy Payoff

The short call butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ C^{ATM}_T & = & (S_T - K^{ATM})^{+}\\ P_T & = & (2\times C^{ATM}_T - C^{OTM}_T - C^{ITM}_T - 2\times C^{ATM}_0 + C^{ITM}_0 + C^{OTM}_0)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & C^{OTM}_T & = & \textrm{OTM call value at time T}\\ & C^{ITM}_T & = & \textrm{ITM call value at time T}\\ & C^{ATM}_T & = & \textrm{ATM call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM call strike price}\\ & K^{ITM} & = & \textrm{ITM call strike price}\\ & K^{ATM} & = & \textrm{ATM call strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{ITM}_0 & = & \textrm{ITM call value at position opening (credit received)}\\ & C^{OTM}_0 & = & \textrm{OTM call value at position opening (credit received)}\\ & C^{ATM}_0 & = & \textrm{ATM call value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $829.08 . You bought an OTM call at a strike of $767.50 for $4.90 , sold 2 ATM calls at a strike of $800.00 for $15.00 each, and bought an ITM call at a strike of $832.50 for $41.00 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received: $C^{ITM}_0 + C^{OTM}_0 - 2\times C^{ATM}_0$. It occurs when the underlying price is less than ITM strike or greater than OTM strike at expiration.

The maximum loss is $K^{ATM} - K^{ITM} + C^{ITM}_0 + C^{OTM}_0 - 2\times C^{ATM}_0$. It occurs when the underlying price is at the same level as when you opened the trade.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the short call butterfly:

Asset	Price ($)	Strike ($)
OTM call	4.90	767.50
ATM call	15.00	800.00
ITM call	41.00	832.50
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{OTM})^{+}\\ & = & (767.50-829.08)^{+}\\ & = & 0\\ C^{ITM}_T & = & (S_T - K^{ITM})^{+}\\ & = & (832.50-829.08)^{+}\\ & = & 3.42\\ C^{ATM}_T & = & (S_T - K^{ATM})^{+}\\ & = & (800.00-829.08)^{+}\\ & = & 0\\ P_T & = & (-C^{OTM}_T - C^{ITM}_T + 2\times C^{ATM}_T - 2\times C^{ATM}_0 + C^{ITM}_0 + C^{OTM}_0)\times m - fee\\ & = & (-0-3.42+0\times2+4.90+41.00-15.00\times2)\times100-1.00\times4\\ & = & 1244 \end{array}$$

So, the strategy gains $1,244.

The following algorithm implements a short call butterfly Option strategy:

```py
class LongCallButterflyStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.call_butterfly(30, 5)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # sorted the optionchain by expiration date and choose the furthest date
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # filter the call options from the contracts which expire on the furthest expiration date in the option chain.
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if len(calls) == 0: return

        # sort the call options with the same expiration date according to their strike price.
        call_strikes = sorted([x.strike for x in calls])

        # get at-the-money strike
        atm_strike = sorted(calls, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # Get the distance between lowest strike price and ATM strike, and highest strike price and ATM strike.
        # Get the lower value as the spread distance as equidistance is needed for both side.
        spread = min(abs(call_strikes[0] - atm_strike), abs(call_strikes[-1] - atm_strike))

        # select the strike prices for forming the option legs
        itm_strike = atm_strike - spread
        otm_strike = atm_strike + spread

        option_strategy = OptionStrategies.short_butterfly_call(self.symbol, otm_strike, atm_strike, itm_strike, expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
LongAndShortButterflyCallStrategiesAlgorithm.py Python

Option Strategies

Long Put Butterfly

Introduction

Long Put butterfly is the combination of a bull put spread and a bear put spread . In this strategy, all the puts have the same underlying stock, the same expiration date, and the strike price distance of ITM-ATM and OTM-ATM put pairs are the same. The long put butterfly strategy consists of buying an ITM put, buying an OTM put, and selling 2 ATM puts. This strategy profits from low volatility.

Implementation

Follow these steps to implement the long put butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_butterfly(30, 5))
```

   The `put_butterfly` filter narrows the universe down to just the three contracts you need to form a long put butterfly.

2. In the `on_data` method, select the contracts of the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the furthest expiry date of the contracts
    expiry = max([x.expiry for x in chain])

    # Select the call Option contracts with the furthest expiry
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if len(puts) == 0:
        return

    # Select the ATM, ITM and OTM contracts from the remaining contracts
    atm_put = sorted(puts, key=lambda x: abs(x.strike - chain.underlying.price))[0]
    itm_put = sorted(puts, key=lambda x: x.strike)[-2]
    otm_put = [x for x in puts if x.strike == atm_put.strike * 2 - itm_put.strike][0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.butterfly_put` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.butterfly_put(self._symbol, itm_put.strike, atm_put.strike, otm_put.strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
legs = [
    Leg.create(atm_put.symbol, -2),
    Leg.create(itm_put.symbol, 1),
    Leg.create(otm_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

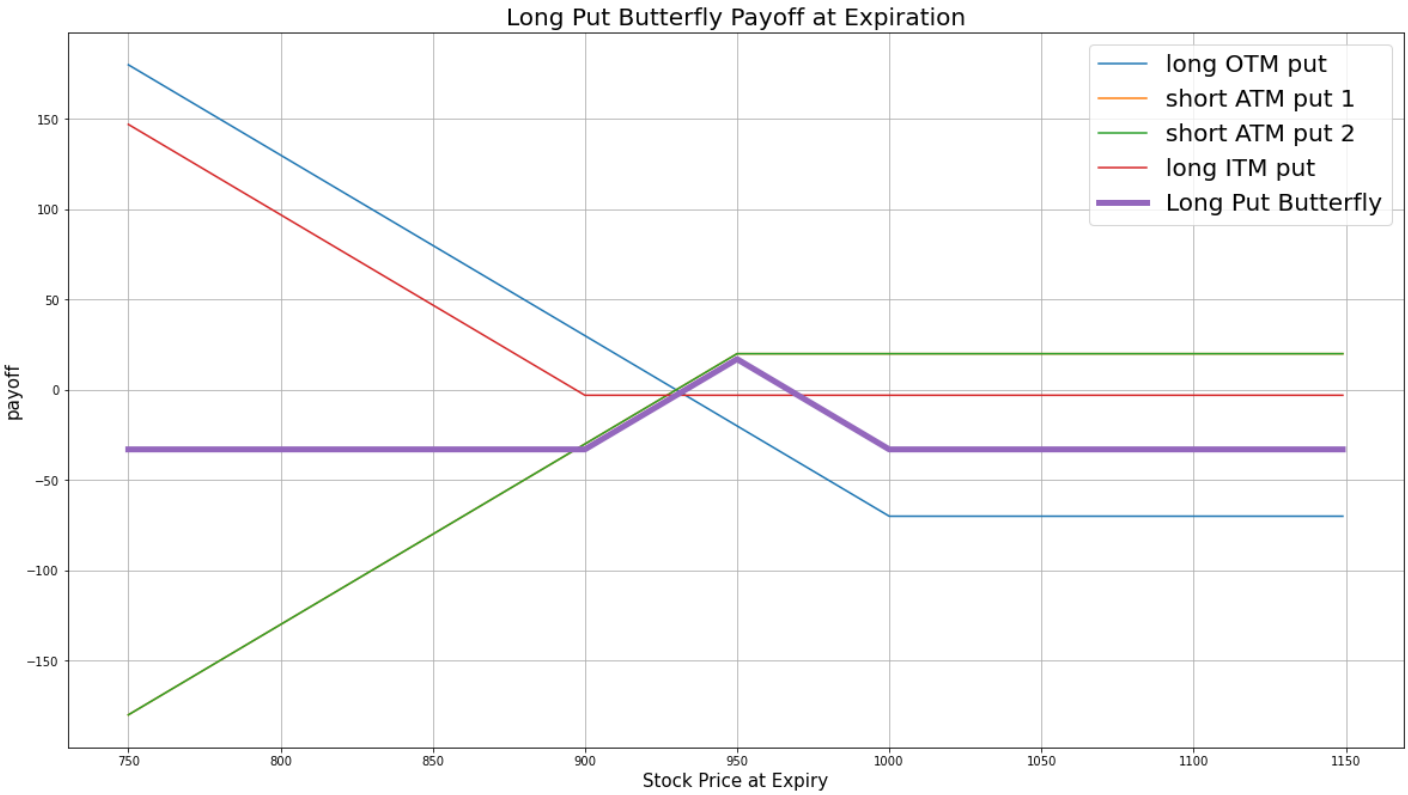
Strategy Payoff

The long put butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ P^{ATM}_T & = & (K^{ATM} - S_T)^{+}\\ P_T & = & (P^{OTM}_T + P^{ITM}_T - 2\times P^{ATM}_T + 2\times P^{ATM}_0 - P^{ITM}_0 - P^{OTM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & P^{OTM}_T & = & \textrm{OTM put value at time T}\\ & P^{ITM}_T & = & \textrm{ITM put value at time T}\\ & P^{ATM}_T & = & \textrm{ATM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{OTM} & = & \textrm{OTM put strike price}\\ & K^{ITM} & = & \textrm{ITM put strike price}\\ & K^{ATM} & = & \textrm{ATM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & P^{ITM}_0 & = & \textrm{ITM put value at position opening (debit paid)}\\ & P^{OTM}_0 & = & \textrm{OTM put value at position opening (debit paid)}\\ & P^{ATM}_0 & = & \textrm{ATM put value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $829.08 . You bought an ITM put at a strike of $832.50 for $37.80 , sold 2 ATM puts at a strike of $800.00 for $14.70 each, and bought an OTM put at a strike of $767.50 for $5.70 .

The following chart shows the payoff at expiration:



The maximum profit is $K^{ATM} - K^{OTM} + 2\times P^{ATM}_0 - P^{ITM}_0 - P^{OTM}_0$. It occurs when the underlying price is the same at expiration as it was when you open the trade. In this case, the payout of the combined bull put and bear put spreads are at their maximum.

The maximum loss is the net debit paid, $2\times P^{ATM}_0 - P^{ITM}_0 - P^{OTM}_0$. It occurs when the underlying price is below the ITM strike price or above the OTM strike price at expiration.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the long put butterfly algorithm:

Asset	Price ($)	Strike ($)
ITM put	37.80	832.50
ATM put	14.70	800.00
OTM put	5.70	767.50
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcl} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ & = & (829.08-832.50)^{+}\\ & = & 0\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ & = & (829.08-767.50)^{+}\\ & = & 61.58\\ P^{ATM}_T & = & (K^{ATM} - S_T)^{+}\\ & = & (829.08-800.00)^{+}\\ & = & 29.08\\ P_T & = & (P^{OTM}_T + P^{ITM}_T - 2\times P^{ATM}_T + 2\times P^{ATM}_0 - P^{ITM}_0 - P^{OTM}_0)\times m - fee\\ & = & (61.58+0-29.08\times2-5.70-37.80+14.70\times2)\times100-1.00\times4\\ & = & -1072 \end{array}$$

So, the strategy loses $1,072.

The following algorithm implements a long put butterfly Option strategy:

```py
class PutButterflyStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_butterfly(30, 5)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # sorted the optionchain by expiration date and choose the furthest date
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # filter the put options from the contracts which expire on the furthest expiration date in the option chain.
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # sort the put options with the same expiration date according to their strike price.
        put_strikes = sorted([x.strike for x in puts])

        # get at-the-money strike
        atm_strike = sorted(puts, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # Get the distance between lowest strike price and ATM strike, and highest strike price and ATM strike.
        # Get the lower value as the spread distance as equidistance is needed for both side.
        spread = min(abs(put_strikes[0] - atm_strike), abs(put_strikes[-1] - atm_strike))

        # select the strike prices for forming the option legs
        itm_strike = atm_strike + spread
        otm_strike = atm_strike - spread

        option_strategy = OptionStrategies.put_butterfly(self.symbol, itm_strike, atm_strike, otm_strike, expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
        # self.sell(option_strategy, 1) if short put butterfly
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionPutButterflyAlgorithm.py Python

Option Strategies

Short Put Butterfly

Introduction

Short Put butterfly is the combination of a bull put spread and a bear put spread . In this strategy, all the puts have the same underlying stock, the same expiration date, and the strike price distance of ITM-ATM and OTM-ATM put pairs are the same. The short put butterfly strategy consists of selling an ITM put, selling an OTM put, and buying 2 ATM puts. This strategy profits from a drastic change in underlying price.

Implementation

Follow these steps to implement the short put butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_butterfly(30, 5))
```

   The `put_butterfly` filter narrows the universe down to just the three contracts you need to form a short put butterfly.

2. In the `on_data` method, select the contracts of the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the furthest expiry date of the contracts
    expiry = max([x.expiry for x in chain])

    # Select the call Option contracts with the furthest expiry
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if len(puts) == 0:
        return

    # Select the ATM, ITM and OTM contracts from the remaining contracts
    atm_put = sorted(puts, key=lambda x: abs(x.strike - chain.underlying.price))[0]
    itm_put = sorted(puts, key=lambda x: x.strike)[-2]
    otm_put = [x for x in puts if x.strike == atm_put.strike * 2 - itm_put.strike][0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.short_butterfly_put` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.short_butterfly_put(self._symbol, itm_put.strike, atm_put.strike, otm_put.strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(atm_put.symbol, 2),
    Leg.create(itm_put.symbol, -1),
    Leg.create(otm_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

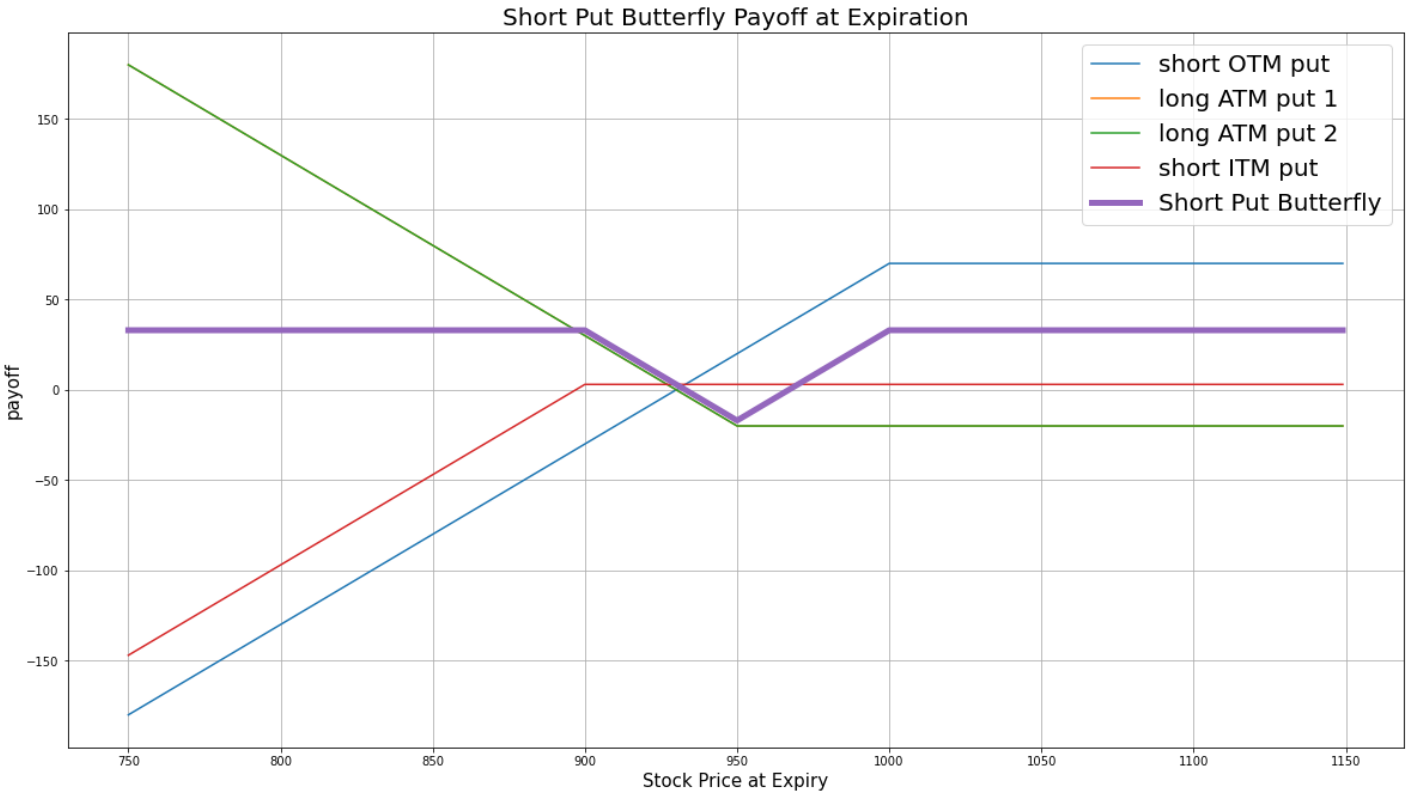
Strategy Payoff

The short put butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{OTM}_T &=& (K^{OTM} - S_T)^{+}\\ P^{ITM}_T &=& (K^{ITM} - S_T)^{+}\\ P^{ATM}_T &=& (K^{ATM} - S_T)^{+}\\ P_T &=& (2\times P^{ATM}_T - P^{OTM}_T - P^{ITM}_T - 2\times P^{ATM}_0 + P^{ITM}_0 + P^{OTM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & P^{OTM}_T &= \textrm{OTM put value at time T}\\ & P^{ITM}_T &= \textrm{ITM put value at time T}\\ & P^{ATM}_T &= \textrm{ATM put value at time T}\\ & S_T &= \textrm{Underlying asset price at time T}\\ & K^{OTM} &= \textrm{OTM put strike price}\\ & K^{ITM} &= \textrm{ITM put strike price}\\ & K^{ATM} &= \textrm{ATM put strike price}\\ & P_T &= \textrm{Payout total at time T}\\ & P^{ITM}_0 &= \textrm{ITM put value at position opening (credit received)}\\ & P^{OTM}_0 &= \textrm{OTM put value at position opening (credit received)}\\ & P^{ATM}_0 &= \textrm{ATM put value at position opening (debit paid)}\\ & m &= \textrm{Contract multiplier}\\ & T &= \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $829.08 . You bought an ITM put at a strike of $832.50 for $37.80 , sold 2 ATM puts at a strike of $800.00 for $14.70 each, and bought an OTM put at a strike of $767.50 for $5.70 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received, $P^{ITM}_0 + P^{OTM}_0 - 2\times P^{ATM}_0$. It occurs when the underlying price is below the ITM strike or above the OTM strike at expiration.

The maximum loss is $K^{ATM} - K^{OTM} - 2\times P^{ATM}_0 + P^{ITM}_0 + P^{OTM}_0$. It occurs when the underlying price at expiration is at the same price as when you opened the trade.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the short put butterfly algorithm:

Asset	Price ($)	Strike ($)
ITM put	37.80	832.50
ATM put	14.70	800.00
OTM put	5.70	767.50
Underlying Equity at expiration	829.08	-

Therefore, the payoff is

$$\begin{array}{rcl} P^{OTM}_T & = & (K^{OTM} - S_T)^{+}\\ & = & (829.08-832.50)^{+}\\ & = & 0\\ P^{ITM}_T & = & (K^{ITM} - S_T)^{+}\\ & = & (829.08-767.50)^{+}\\ & = & 61.58\\ P^{ATM}_T & = & (K^{ATM} - S_T)^{+}\\ & = & (829.08-800.00)^{+}\\ & = & 29.08\\ P_T & = & (-P^{OTM}_T - P^{ITM}_T + 2\times P^{ATM}_T - 2\times P^{ATM}_0 + P^{ITM}_0 + P^{OTM}_0)\times m - fee\\ & = & (-61.58-0+29.08\times2+5.70+37.80-14.70\times2)\times100-1.00\times4\\ & = & 1064 \end{array}$$

So, the strategy gains $1,064.

The following algorithm implements a short put butterfly Option strategy:

```py
class PutButterflyStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_butterfly(30, 5)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # sorted the optionchain by expiration date and choose the furthest date
        expiry = sorted(chain, key = lambda x: x.expiry, reverse=True)[0].expiry

        # filter the put options from the contracts which expire on the furthest expiration date in the option chain.
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # sort the put options with the same expiration date according to their strike price.
        put_strikes = sorted([x.strike for x in puts])

        # get at-the-money strike
        atm_strike = sorted(puts, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # Get the distance between lowest strike price and ATM strike, and highest strike price and ATM strike.
        # Get the lower value as the spread distance as equidistance is needed for both side.
        spread = min(abs(put_strikes[0] - atm_strike), abs(put_strikes[-1] - atm_strike))

        # select the strike prices for forming the option legs
        itm_strike = atm_strike + spread
        otm_strike = atm_strike - spread

        option_strategy = OptionStrategies.short_butterfly_put(self.symbol, itm_strike, atm_strike, otm_strike, expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
        # self.sell(option_strategy, 1) if short put butterfly
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
LongAndShortButterflyPutStrategiesAlgorithm.py Python

Option Strategies

Long Call Calendar Spread

Introduction

Long call calendar spread , also known as call horizontal spread , is a combination of a longer-term (far-leg/front-month) call and a shorter-term (near-leg/back-month) call, where all calls have the same underlying stock and the same strike price. The long call calendar spread consists of buying a longer-term call and selling a shorter-term call. This strategy profits from a decrease in the underlying price. It also profits from the time decay value because the theta \theta (the Option price decay by 1 day closer to maturity) of the shorter-term call is larger than longer-term call.

Implementation

Follow these steps to implement the long call calendar spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_calendar_spread(0, 30, 60))
```

   The `call_calendar_spread` filter narrows the universe down to just the two contracts you need to form a long call calendar spread.

2. In the `on_data` method, select the strike price and expiration dates of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the ATM strike
    atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

    # Select the ATM call Option contracts
    calls = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.CALL]
    if len(calls) == 0:
        return

    # Select the near and far expiry dates
    expiries = sorted([x.expiry for x in calls])
    near_expiry = expiries[0]
    far_expiry = expiries[-1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.call_calendar_spread` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.call_calendar_spread(self._symbol, atm_strike, near_expiry, far_expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
near_expiry_call = [x for x in calls if x.expiry == near_expiry][0]
far_expiry_call = [x for x in calls if x.expiry == far_expiry][0]

legs = [
    Leg.create(near_expiry_call.symbol, -1),
    Leg.create(far_expiry_call.symbol, 1)
]
self.combo_market_order(legs, 1)
```

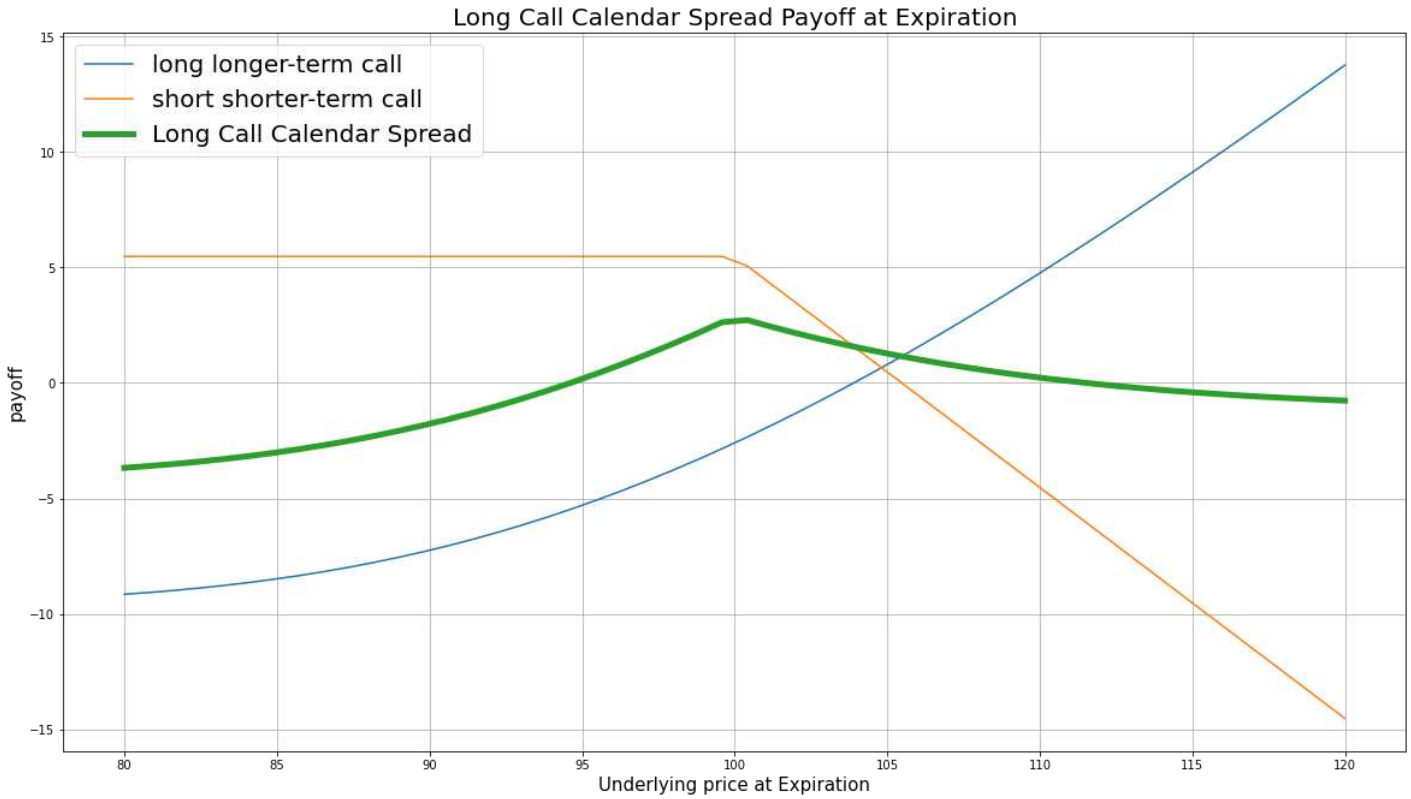
Strategy Payoff

The long call calendar spread is a limited-reward-limited-risk strategy. The payoff at the shorter-term expiration is

$$\begin{array}{rcll} C^{\textrm{short-term}}_T & = & (S_T - K)^{+}\\ P_T & = & (C^{\textrm{long-term}}_T - C^{\textrm{short-term}}_T + C^{\textrm{short-term}}_0 - C^{\textrm{long-term}}_0)\times m - fee\\ \end{array} \begin{array}{rcll} \textrm{where} & C^{\textrm{short-term}}_T & = & \textrm{Shorter term call value at time T}\\ & C^{\textrm{long-term}}_T & = & \textrm{Longer term call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{\textrm{short-term}}_0 & = & \textrm{Shorter term call value at position opening (credit received)}\\ & C^{\textrm{long-term}}_0 & = & \textrm{Longer term call value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of shorter term call expiration} \end{array}$$

In this example, you bought a longer-term call at a strike of $800.00 for $20.00 and sold a shorter-term call at the same strike for $11.30 .

The following chart shows the payoff at expiration:



The maximum profit is undetermined because it depends on the underlying volatility. It occurs when $S_T = S_0$ and the spread of the calls are at their maximum.

The maximum loss is the net debit paid, $C^{\textrm{short-term}}_0 - C^{\textrm{long-term}}_0$. It occurs when the underlying price moves very deep ITM or OTM so the values of both calls are close to zero.

If the Option is American Option, there is a risk of early assignment on the contract you sell. If the buyer exercises the call you sell, you could lose all the debit you received if you don't close the long call and the underlying price drops below the long call strike price.

Example

The following table shows the price details of the assets in the long call calendar spread:

Asset	Price ($)	Strike ($)
Longer-term call at the start of the trade	4.40	835.00
Shorter-term call at the start of the trade	36.80	767.50
Longer-term call at time T	31.35	835.00
Underlying Equity at time T	829.08	-

Therefore, the payoff at time T (the expiration of the short-term call) is

$$\begin{array}{rcll} C^{\textrm{short-term}}_T & = & (S_T - K)^{+}\\ & = & (828.07-800.00)^{+}\\ & = & 28.07\\ P_T & = & (C^{\textrm{long-term}}_T - C^{\textrm{short-term}}_T + C^{\textrm{short-term}}_0 - C^{\textrm{long-term}}_0)\times m - fee\\ & = & (31.35-28.07+11.30-20.00)\times100-1.00\times2\\ & = & -544 \end{array}$$

So, the strategy loses $544.

The following algorithm implements a long call calendar spread Option strategy:

```py
class LongCallCalendarSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.call_calendar_spread(0, 30, 60)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # get at-the-money strike
        atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # filter the call options from the contracts which is ATM in the option chain.
        calls = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.CALL]
        if len(calls) == 0: return

        # sorted the optionchain by expiration date
        expiries = sorted([x.expiry for x in calls], key = lambda x: x)

        # select the farest expiry as far-leg expiry, and the nearest expiry as near-leg expiry
        near_expiry = expiries[0]
        far_expiry = expiries[-1]

        option_strategy = OptionStrategies.call_calendar_spread(self.symbol, atm_strike, near_expiry, far_expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
        # self.sell(option_strategy, 1) if short call calendar spread
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionCallCalendarSpreadAlgorithm.py Python LongAndShortCallCalendarSpreadStrategiesAlgorithm.py Python

Option Strategies

Short Call Calendar Spread

Introduction

Call calendar spread , also known as call horizontal spread , is a combination of a longer-term (far-leg/front-month) call and a shorter-term (near-leg/back-month) call, where all calls have the same underlying stock and the same strike price. The short call calendar spread consists of selling a longer-term call and buying a shorter-term call. The strategy profits from from an increase in the underlying price.

Implementation

Follow these steps to implement the short call calendar spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_calendar_spread(0, 30, 60))
```

   The `call_calendar_spread` filter narrows the universe down to just the two contracts you need to form a short call calendar spread.

2. In the `on_data` method, select the strike price and expiration dates of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the ATM strike
    atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

    # Select the ATM call Option contracts
    calls = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.CALL]
    if len(calls) == 0:
        return

    # Select the near and far expiry dates
    expiries = sorted([x.expiry for x in calls])
    near_expiry = expiries[0]
    far_expiry = expiries[-1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_call_calendar_spread` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.short_call_calendar_spread(self._symbol, atm_strike, near_expiry, far_expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
near_expiry_call = [x for x in calls if x.expiry == near_expiry][0]
far_expiry_call = [x for x in calls if x.expiry == far_expiry][0]

legs = [
    Leg.create(near_expiry_call.symbol, 1),
    Leg.create(far_expiry_call.symbol, -1)
]
self.combo_market_order(legs, 1)
```

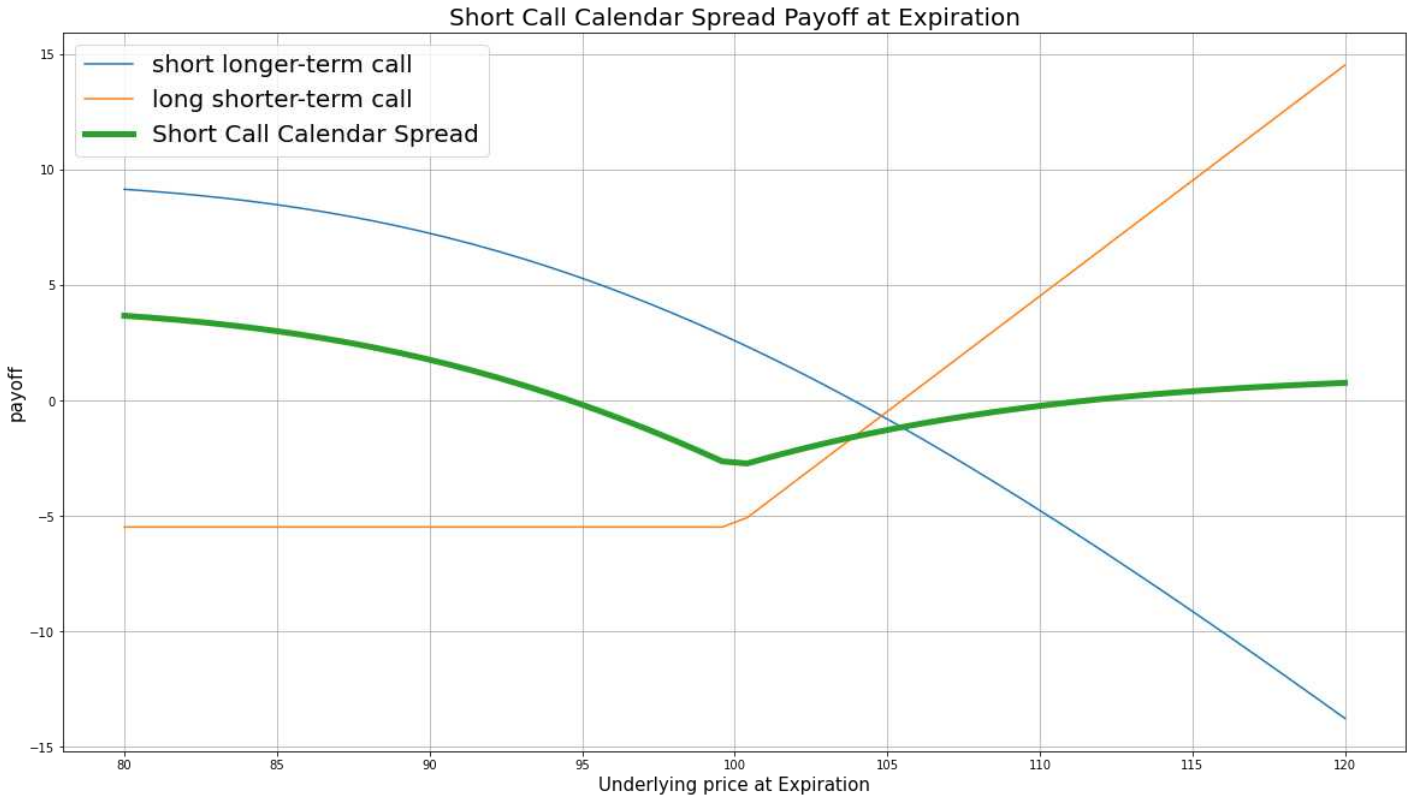
Strategy Payoff

The short call calendar spread is a limited-reward-limited-risk strategy. The payoff at the shorter-term expiration is

$$\begin{array}{rcll} C^{\textrm{short-term}}_T & = & (S_T - K)^{+}\\ P_T & = & (C^{\textrm{short-term}}_T - C^{\textrm{long-term}}_T + C^{\textrm{long-term}}_0 - C^{\textrm{short-term}}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{\textrm{short-term}}_T & = & \textrm{Shorter term call value at time T}\\ & C^{\textrm{long-term}}_T & = & \textrm{Longer term call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{\textrm{short-term}}_0 & = & \textrm{Shorter term call value at position opening (debit paid)}\\ & C^{\textrm{long-term}}_0 & = & \textrm{Longer term call value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of shorter term call expiration} \end{array}$$

In this example, you bought a longer-term call at a strike of $800.00 for $20.00 and sold a shorter-term call at the same strike for $11.30 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received, $C^{\textrm{long-term}}_0 - C^{\textrm{short-term}}_0$. It occurs when the underlying price moves very deep ITM or OTM so the values of both calls are close to zero.

The maximum loss is undetermined because it depends on the underlying volatility. It occurs when $S_T = S_0$ and the spread of the 2 calls are at their maximum.

If the Option is American Option, there is a risk of early assignment on the contract you sell. If you don't close the call positions together, the naked short call will have unlimited drawdown risk after the long call expires.

Example

The following table shows the price details of the assets in the short call calendar spread:

Asset	Price ($)	Strike ($)
Longer-term call at the start of the trade	4.40	835.00
Shorter-term call at the start of the trade	36.80	767.50
Longer-term call at time T	31.35	835.00
Underlying Equity at time T	829.08	-

Therefore, the payoff at time T (the expiration of the short-term call) is

$$\begin{array}{rcll} C^{\textrm{short-term}}_T & = & (S_T - K)^{+}\\ & = & (828.07-800.00)^{+}\\ & = & 28.07\\ P_T & = & (-C^{\textrm{long-term}}_T + C^{\textrm{short-term}}_T - C^{\textrm{short-term}}_0 + C^{\textrm{long-term}}_0)\times m - fee\\ & = & (-31.35+28.07-11.30+20.00)\times100-1.00\times2\\ & = & 540 \end{array}$$

So, the strategy gains $540.

The following algorithm implements a short call calendar spread Option strategy:

```
class LongCallCalendarSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.call_calendar_spread(0, 30, 60)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # get at-the-money strike
        atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # filter the call options from the contracts which is ATM in the option chain.
        calls = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.CALL]
        if len(calls) == 0: return

        # sorted the optionchain by expiration date
        expiries = sorted([x.expiry for x in calls], key = lambda x: x)

        # select the farest expiry as far-leg expiry, and the nearest expiry as near-leg expiry
        near_expiry = expiries[0]
        far_expiry = expiries[-1]

        option_strategy = OptionStrategies.short_call_calendar_spread(self.symbol, atm_strike, near_expiry, far_expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
LongAndShortCallCalendarSpreadStrategiesAlgorithm.py Python

Option Strategies

Long Put Calendar Spread

Introduction

Put calendar spread , also known as put horizontal spread , is a combination of a longer-term (far-leg/front-month) put and a shorter-term (near-leg/back-month) put, where both puts have the same underlying stock and the same strike price. The long put calendar spread consists of buying a longer-term put and selling a shorter-term put. This strategy profits from a decrease in price movement. The strategy also profits from the time decay value because the theta \theta (the Option price decay by 1 day closer to maturity) of the shorter-term put is larger than the longer-term put.

Implementation

Follow these steps to implement the long put calendar spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_calendar_spread(0, 30, 60))
```

   The `put_calendar_spread` filter narrows the universe down to just the two contracts you need to form a long put calendar spread.

2. In the `on_data` method, select the strike price and expiration dates of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the ATM strike
    atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

    # Select the ATM put Option contracts
    puts = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.PUT]
    if len(puts) == 0:
        return

    # Select the near and far expiry dates
    expiries = sorted([x.expiry for x in puts])
    near_expiry = expiries[0]
    far_expiry = expiries[-1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.put_calendar_spread` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.put_calendar_spread(self._symbol, atm_strike, near_expiry, far_expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
near_expiry_put = [x for x in puts if x.expiry == near_expiry][0]
far_expiry_put = [x for x in puts if x.expiry == far_expiry][0]

legs = [
    Leg.create(near_expiry_put.symbol, -1),
    Leg.create(far_expiry_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

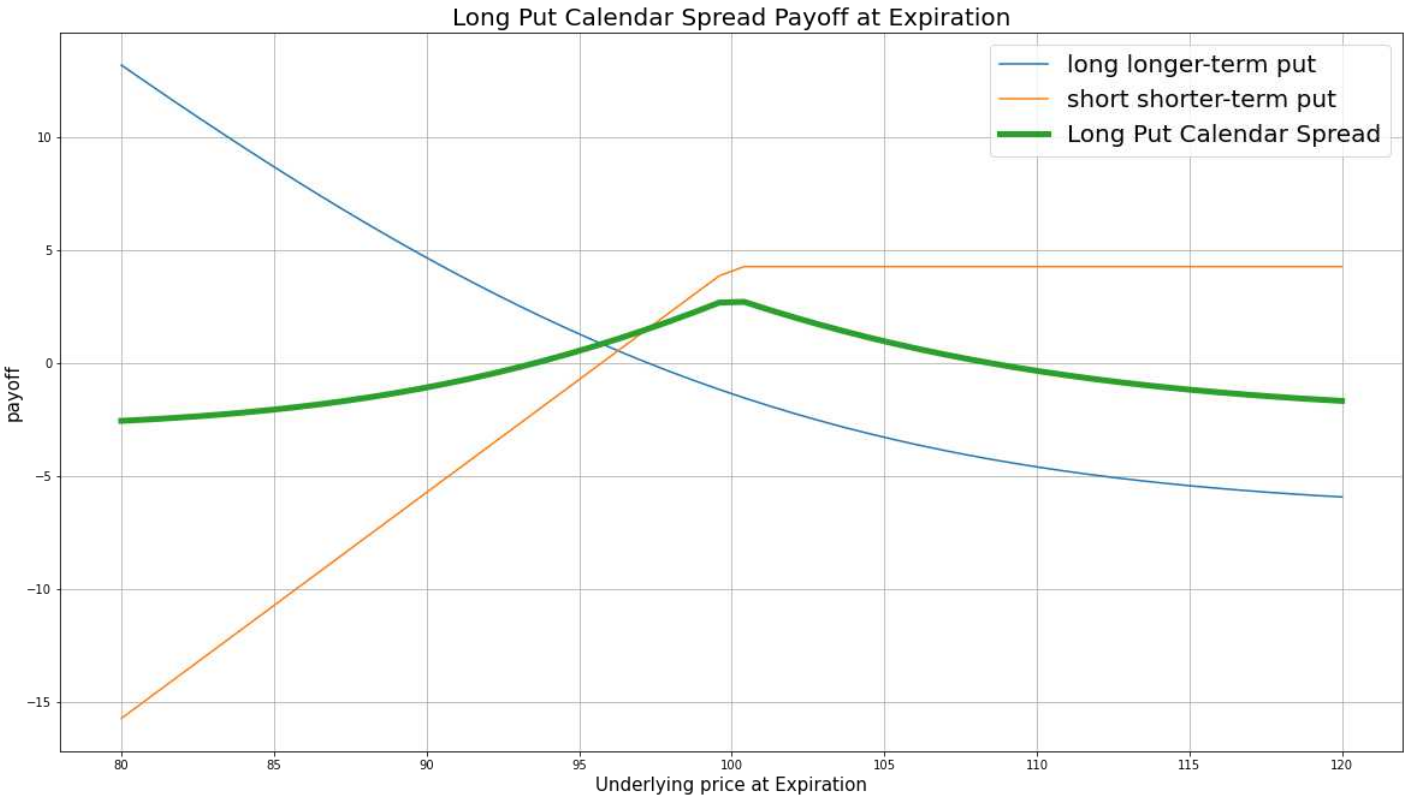
Strategy Payoff

The long put calendar spread is a limited-reward-limited-risk strategy. The payoff is taken at the shorter-term expiration. The payoff is

$$\begin{array}{rcll} P^{\textrm{short-term}}_T & = & (K - S_T)^{+}\\ P_T & = & (P^{\textrm{long-term}}_T - P^{\textrm{short-term}}_T + P^{\textrm{short-term}}_0 - P^{\textrm{long-term}}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & & \\ P^{\textrm{short-term}}_T & = & \textrm{Shorter term put value at time T}\\ P^{\textrm{long-term}}_T & = & \textrm{Longer term put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K & = & \textrm{Strike price}\\ P_T & = & \textrm{Payout total at time T}\\ P^{\textrm{short-term}}_0 & = & \textrm{Shorter term put value at position opening (credit received)}\\ P^{\textrm{long-term}}_0 & = & \textrm{Longer term put value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of shorter term put expiration} \end{array}$$

In this example, you bought a longer-term put at a strike of $800.00 for $19.30 and sold a shorter-term put at the same strike for $11.30 .

The following chart shows the payoff at expiration:



The maximum profit is undetermined because it depends on the underlying volatility. It occurs when $S_T = S_0$ and the spread of the puts are at their maximum.

The maximum loss is the net debit paid, $P^{\textrm{short-term}}_0 - P^{\textrm{long-term}}_0$. It occurs when the underlying price moves very deep ITM or OTM so the values of both puts are close to zero.

If the Option is American Option, there is a risk of early assignment on the contract you sell. Naked long puts pose risk of losing all the debit paid if you don't close the position with short put together and the price drops below its strike.

Example

The following table shows the price details of the assets in the long put calendar spread algorithm:

Asset	Price ($)	Strike ($)
Shorter-term put at position opening	11.30	800.00
Longer-term put at position opening	19.30	800.00
Longer-term put at shorter-term expiration	3.50	800.00
Underlying Equity at shorter-term expiration	828.07	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{\textrm{short-term}}_T & = & (K - S_T)^{+}\\ & = & (800.00-828.07)^{+}\\ & = & 0\\ P_T & = & (P^{\textrm{long-term}}_T - P^{\textrm{short-term}}_T + P^{\textrm{short-term}}_0 - P^{\textrm{long-term}}_0)\times m - fee\\ & = & (3.50-0+11.30-19.30)\times100-1.00\times2\\ & = & -452\\ \end{array}$$

So, the strategy loses $452.

The following algorithm implements a long put calendar spread Option strategy:

```py
class PutCalendarSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_calendar_spread(0, 30, 60)

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # get at-the-money strike
        atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # filter the put options from the contracts which is ATM in the option chain.
        puts = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # sorted the optionchain by expiration date
        expiries = sorted([x.expiry for x in puts], key = lambda x: x)

        # select the farest expiry as far-leg expiry, and the nearest expiry as near-leg expiry
        near_expiry = expiries[0]
        far_expiry = expiries[-1]

        option_strategy = OptionStrategies.put_calendar_spread(self.symbol, atm_strike, near_expiry, far_expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)
        # self.sell(option_strategy, 1) if short put calendar spread

    def on_end_of_algorithm(self):
        for symbol, sec in self.securities.items():
            self.log(f"{symbol} :: {sec.price}")
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionPutCalendarSpreadAlgorithm.py Python LongAndShortPutCalendarSpreadStrategiesAlgorithm.py Python

Option Strategies

Short Put Calendar Spread

Introduction

Put calendar spread , also known as put horizontal spread , is a combination of a longer-term (far-leg/front-month) put and a shorter-term (near-leg/back-month) put, where both puts have the same underlying stock and the same strike price. The short put calendar spread consists of selling a longer-term put and buying a shorter-term put. This strategy profits from an increase in price movement.

Implementation

Follow these steps to implement the short put calendar spread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_calendar_spread(0, 30, 60))
```

   The `put_calendar_spread` filter narrows the universe down to just the two contracts you need to form a short put calendar spread.

2. In the `on_data` method, select the strike price and expiration dates of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Get the ATM strike
    atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

    # Select the ATM put Option contracts
    puts = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.PUT]
    if len(puts) == 0:
        return

    # Select the near and far expiry dates
    expiries = sorted([x.expiry for x in puts])
    near_expiry = expiries[0]
    far_expiry = expiries[-1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_put_calendar_spread` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.short_put_calendar_spread(self._symbol, atm_strike, near_expiry, far_expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
near_expiry_put = [x for x in puts if x.expiry == near_expiry][0]
far_expiry_put = [x for x in puts if x.expiry == far_expiry][0]

legs = [
    Leg.create(near_expiry_put.symbol, 1),
    Leg.create(far_expiry_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

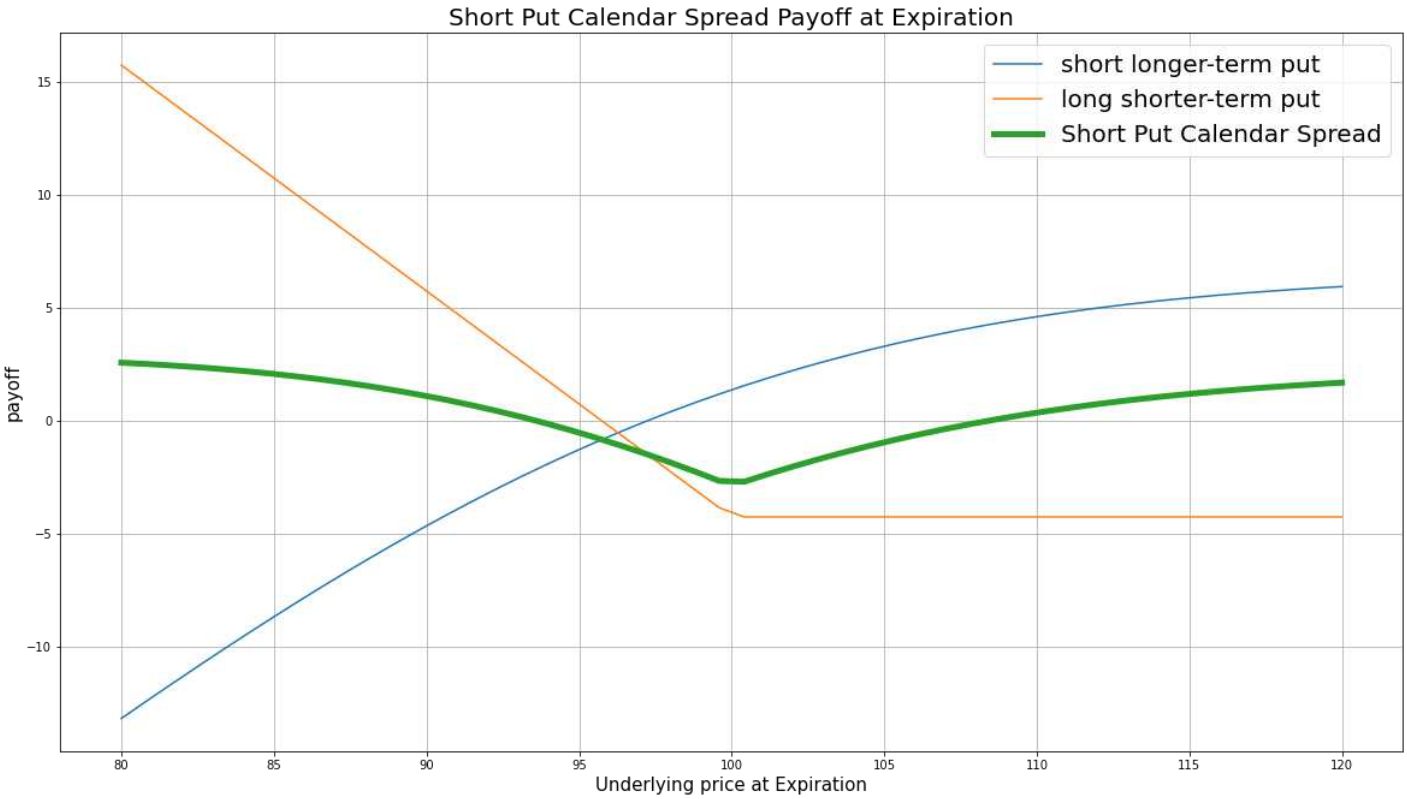
Strategy Payoff

The short put calendar spread is a limited-reward-limited-risk strategy. The payoff is taken at the shorter-term expiration. The payoff is

$$\begin{array}{rcll} P^{\textrm{short-term}}_T & = & (K - S_T)^{+}\\ P_T & = & (P^{\textrm{short-term}}_T - P^{\textrm{long-term}}_T + P^{\textrm{long-term}}_0 - P^{\textrm{short-term}}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & & \\ P^{\textrm{short-term}}_T & = & \textrm{Shorter term put value at time T}\\ P^{\textrm{long-term}}_T & = & \textrm{Longer term put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K & = & \textrm{Strike price}\\ P_T & = & \textrm{Payout total at time T}\\ P^{\textrm{short-term}}_0 & = & \textrm{Shorter term put value at position opening (debit paid)}\\ P^{\textrm{long-term}}_0 & = & \textrm{Longer term put value at position opening (credit received)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of shorter term put expiration} \end{array}$$

In this example, you bought a longer-term put at a strike of $800.00 for $19.30 and sold a shorter-term put at the same strike for $11.30 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received, $P^{\textrm{long-term}}_0 - P^{\textrm{short-term}}_0$. It occurs when the underlying price moves very deep ITM or OTM so the values of both puts are close to zero.

The maximum loss is undetermined because it depends on the underlying volatility. It occurs when $S_T = S_0$ and the spread of the 2 puts are at their maximum.

If the Option is American Option, there is a risk of early assignment on the contract you sell. Additionally, if you don't close the put positions together, the naked short put will have unlimited drawdown risk after the long put expires.

Example

The following table shows the price details of the assets in the short put calendar spread algorithm:

Asset	Price ($)	Strike ($)
Shorter-term put at position opening	11.30	800.00
Longer-term put at position opening	19.30	800.00
Longer-term put at shorter-term expiration	3.50	800.00
Underlying Equity at shorter-term expiration	828.07	-

Therefore, the payoff is

$$\begin{array}{rcl} P^{\textrm{short-term}}_T & = & (K - S_T)^{+}\\ & = & (800.00-828.07)^{+}\\ & = & 0\\ P_T & = & (-P^{\textrm{long-term}}_T + P^{\textrm{short-term}}_T - P^{\textrm{short-term}}_0 + P^{\textrm{long-term}}_0)\times m - fee\\ & = & (-3.50+0-11.30+19.30)\times100-1.00\times2\\ & = & 448\\ \end{array}$$

So, the strategy gains $448.

The following algorithm implements a short put calendar spread Option strategy:

```py
class PutCalendarSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(self.universe_func)

    def universe_func(self, universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return universe.put_calendar_spread(0, 30, 60)

    def on_data(self, data) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # get at-the-money strike
        atm_strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike

        # filter the put options from the contracts which is ATM in the option chain.
        puts = [i for i in chain if i.strike == atm_strike and i.right == OptionRight.PUT]
        if len(puts) == 0: return

        # sorted the optionchain by expiration date
        expiries = sorted([x.expiry for x in puts], key = lambda x: x)

        # select the farest expiry as far-leg expiry, and the nearest expiry as near-leg expiry
        near_expiry = expiries[0]
        far_expiry = expiries[-1]

        option_strategy = OptionStrategies.short_put_calendar_spread(self.symbol, atm_strike, near_expiry, far_expiry)
        # We open a position with 1 unit of the option strategy
        self.buy(option_strategy, 1)

    def on_end_of_algorithm(self) -> None:
        for symbol, sec in self.securities.items():
            self.log(f"{symbol} :: {sec.price}")
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
LongAndShortPutCalendarSpreadStrategiesAlgorithm.py Python

Option Strategies

Covered Call

Introduction

A Covered Call consists of a long position in a stock and a short position in call Options for the same amount of stock. Covered calls aim to profit from the Option premium by selling calls written on the stock you already own. At any time for American Options or at expiration for European Options, if the stock moves below the strike price, you keep the premium and still maintain the underlying Equity position. If the underlying price moves above the strike, the Option buyer can exercise the Options contract , which means you sell your stock at the strike price and keep the premium. Another risk of a covered call comes from the long stock position, which can drop in value.

Implementation

Follow these steps to implement the covered call strategy:

1. In the `initialize` method, set the start date, end date, starting cash, and Options universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_call(30, 0))
```

   The `naked_call` filter narrows the universe down to just the one contract you need to form a covered call.

2. In the `on_data` method, select the Option contract.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM call with the farthest expiry
    expiry = max([x.expiry for x in chain])
    call_contracts = sorted([x for x in chain
        if x.right == OptionRight.CALL and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not call_contracts:
        return

    atm_call = call_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.covered_call` method with the details of each leg and then pass the result to the `buy` method.

```
PY
covered_call = OptionStrategies.covered_call(self._symbol, atm_call.strike, expiry)
self.buy(covered_call, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(atm_call.symbol, -1),
    Leg.create(chain.underlying.symbol, chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcl} C^{K}_T & = & (S_T - K)^{+}\\ P_T & = & (S_T - S_0 + C^{K}_0 - C^{K}_T)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{K}_T & = \textrm{Call value at time T}\\ & S_T & = \textrm{Underlying asset price at time T}\\ & K & = \textrm{Call strike price}\\ & P_T & = \textrm{Payout total at time T}\\ & S_0 & = \textrm{Underlying asset price when the trade opened}\\ & C^{K}_0 & = \textrm{Call price when the trade opened (credit received)}\\ & m & = \textrm{Contract multiplier}\\ & T & = \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $187.07 at expiration. You held the underlying and sold a call at a strike of $185.00 for $3.35 .

The following chart shows the payoff at expiration:

The maximum profit is $K - S_T + C^{K}_0$, which occurs when the underlying price is at or above the strike price of the call at expiration.

The maximum loss is $S_0 - C^{K}_0$, which ocurrs when the underlying price drops.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	3.35	185.00
Underlying Equity at start of the trade	187.07	-
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcl} C^{K}_T & = & (S_T - K)^{+}\\ & = & (190.01 - 185)^{+}\\ & = & 5.01\\ P_T & = & (S_T - S_0 + C^{K}_0 - C^{K}_T)\times m - fee\\ & = & (190.01 - 187.07 + 3.35 - 5.01)\times m - fee\\ & = & 1.28 \times 100 - 2\\ & = & 126 \end{array}$$

So, the strategy gains $126.

The following algorithm implements a covered call strategy:

```py
class CoveredCallAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol

        option.set_filter(lambda universe: universe.naked_call(30, 0))

        self.call = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.call and self.portfolio[self.call].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM call with the farthest expiry
        expiry = max([x.expiry for x in chain])
        call_contracts = sorted([x for x in chain
            if x.right == OptionRight.CALL and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not call_contracts:
            return

        atm_call = call_contracts[0]

        covered_call = OptionStrategies.covered_call(self.symbol, atm_call.strike, expiry)
        self.buy(covered_call, 1)

        self.call = atm_call.symbol
```

Option Strategies

Covered Put

Introduction

A Covered Put consists of a short position in a stock and a short position in put Options for the same amount of stock. Covered puts aim to profit from the Option premium by selling puts written on the stock you already shorted. At any time for American Options or at expiration for European Options, if the stock moves below the strike price, you keep the premium and still maintain the underlying Equity position. If the underlying price moves above the strike, the Option buyer can exercise the Options contract , which mean you buy the stock at the strike price but you will still keep the premium. Another risk of a covered put comes from the short stock position, which can drop in value.

Implementation

Follow these steps to implement the covered put strategy:

1. In the `initialize` method, set the start date, end date, starting cash, and Options universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_put(30, 0))
```

   The `naked_put` filter narrows the universe down to just the one contract you need to form a covered put.

2. In the `on_data` method, select the Option contract.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM put with the farthest expiry
    expiry = max([x.expiry for x in chain])
    put_contracts = sorted([x for x in chain
        if x.right == OptionRight.PUT and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not put_contracts:
        return

    atm_put = put_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.covered_put` method with the details of each leg and then pass the result to the `buy` method.

```
PY
covered_put = OptionStrategies.covered_put(self._symbol, atm_put.strike, expiry)
self.buy(covered_put, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
legs = [
    Leg.create(atm_put.symbol, -1),
    Leg.create(chain.underlying.symbol, -chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

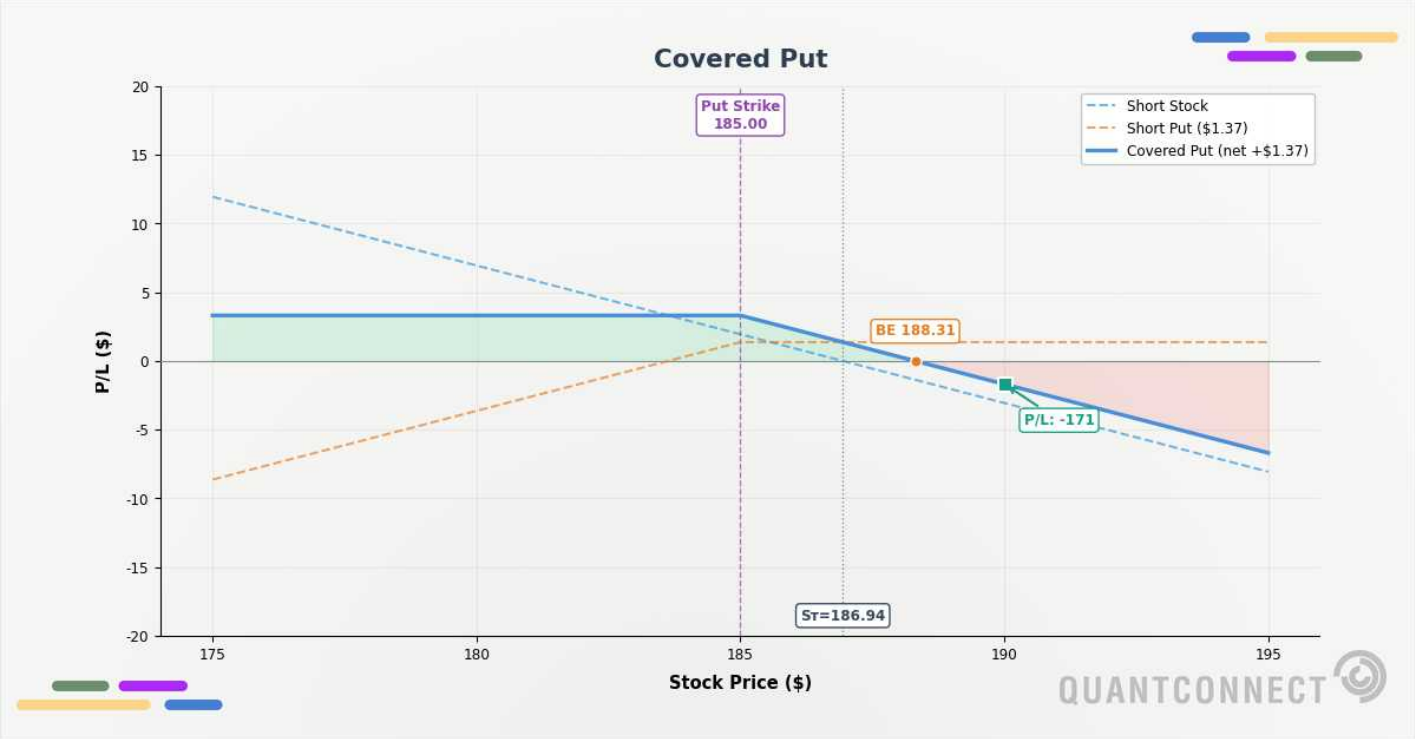
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcl} P^{K}_T & = & (K - S_T)^{+}\\ P_T & = & (S_0 - S_T + P^{K}_0 - P^{K}_T)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & P^{K}_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & P^{K}_0 & = & \textrm{Put price when the trade opened (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $186.94 at expiration. You held the underlying and sold a put at a strike of $185.00 for $1.37 .

The following chart shows the payoff at expiration:



The maximum profit is $S_T - K + P^{K}_0$. It occurs when the underlying price is at or below the strike price of the put at expiration.

If the underlying price increase, the maximum loss is unlimited.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Put	1.37	185.00
Underlying Equity at start of the trade	186.94	-
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcl} P^{K}_T & = & (K - S_T)^{+}\\ & = & (185 - 190.01)^{+}\\ & = & 0\\ P_T & = & (S_0 - S_T + P^{K}_0 - P^{K}_T)\times m - fee\\ & = & (186.94 - 190.01 + 1.37 - 0)\times m - fee\\ & = & -1.70 \times 100 - 2\\ & = & -172 \end{array}$$

So, the strategy loses $172.

The following algorithm implements a covered put strategy:

```py
class CoveredputAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.naked_put(30, 0))

        self.put = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.put and self.portfolio[self.put].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM put with the farthest expiry
        expiry = max([x.expiry for x in chain])
        put_contracts = sorted([x for x in chain
            if x.right == OptionRight.PUT and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not put_contracts:
            return

        atm_put = put_contracts[0]

        covered_put = OptionStrategies.covered_put(self.symbol, atm_put.strike, expiry)
        self.buy(covered_put, 1)

        self.put = atm_put.symbol
```

Option Strategies

Long Iron Butterfly

Introduction

The Long Iron Butterfly is an option strategy which involves four Option contracts. All the contracts have the same underlying stock and expiration, but the order of strike prices for the four contracts is A>B>C. The following table describes the strike price of each contract:

Position	Strike
1 OTM call	A
-1 ATM call	B
-1 ATM put	B
1 OTM put	C=B-(A-B)

The long iron butterfly consists of selling an OTM call, selling an OTM put, buying an ATM call, and buying an ATM put. This strategy profits from a decrease in price movement (implied volatility).

Implementation

Follow these steps to implement the long iron butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.iron_butterfly(30, 5));
```

   The `iron_butterfly` filter narrows the universe down to just the four contracts you need to form a long iron butterfly.

2. In the `on_data` method, select the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select expiry
    expiry = max([x.expiry for x in chain])

    # Separate the call and put contracts
    calls = [i for i in chain if i.right == OptionRight.CALL and i.expiry == expiry]
    puts = [i for i in chain if i.right == OptionRight.PUT and i.expiry == expiry]
    if not calls or not puts:
        return

    # Get the ATM and OTM strike prices
    atm_strike = sorted(calls, key = lambda x: abs(chain.underlying.price - x.strike))[0].strike
    otm_put_strike = min([x.strike for x in puts])
    otm_call_strike = 2 * atm_strike - otm_put_strike
```

3. In the `on_data` method, select the contracts and place the orders.

Approach A: Call the `OptionStrategies.iron_butterfly` method with the details of each leg and then pass the result to the `buy` method.

```
iron_butterfly = OptionStrategies.iron_butterfly(self._symbol, otm_put_strike, atm_strike, otm_call_strike, expiry)
self.buy(iron_butterfly, 2)
```

Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
# Select the contracts
atm_call = [x for x in calls if x.strike == atm_strike][0]
atm_put = [x for x in puts if x.strike == atm_strike][0]
otm_call = [x for x in calls if x.strike == otm_call_strike][0]
otm_put = [x for x in puts if x.strike == otm_put_strike][0]

legs = [
    Leg.create(atm_call.symbol, -1),
    Leg.create(atm_put.symbol, -1),
    Leg.create(otm_call.symbol, 1),
    Leg.create(otm_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

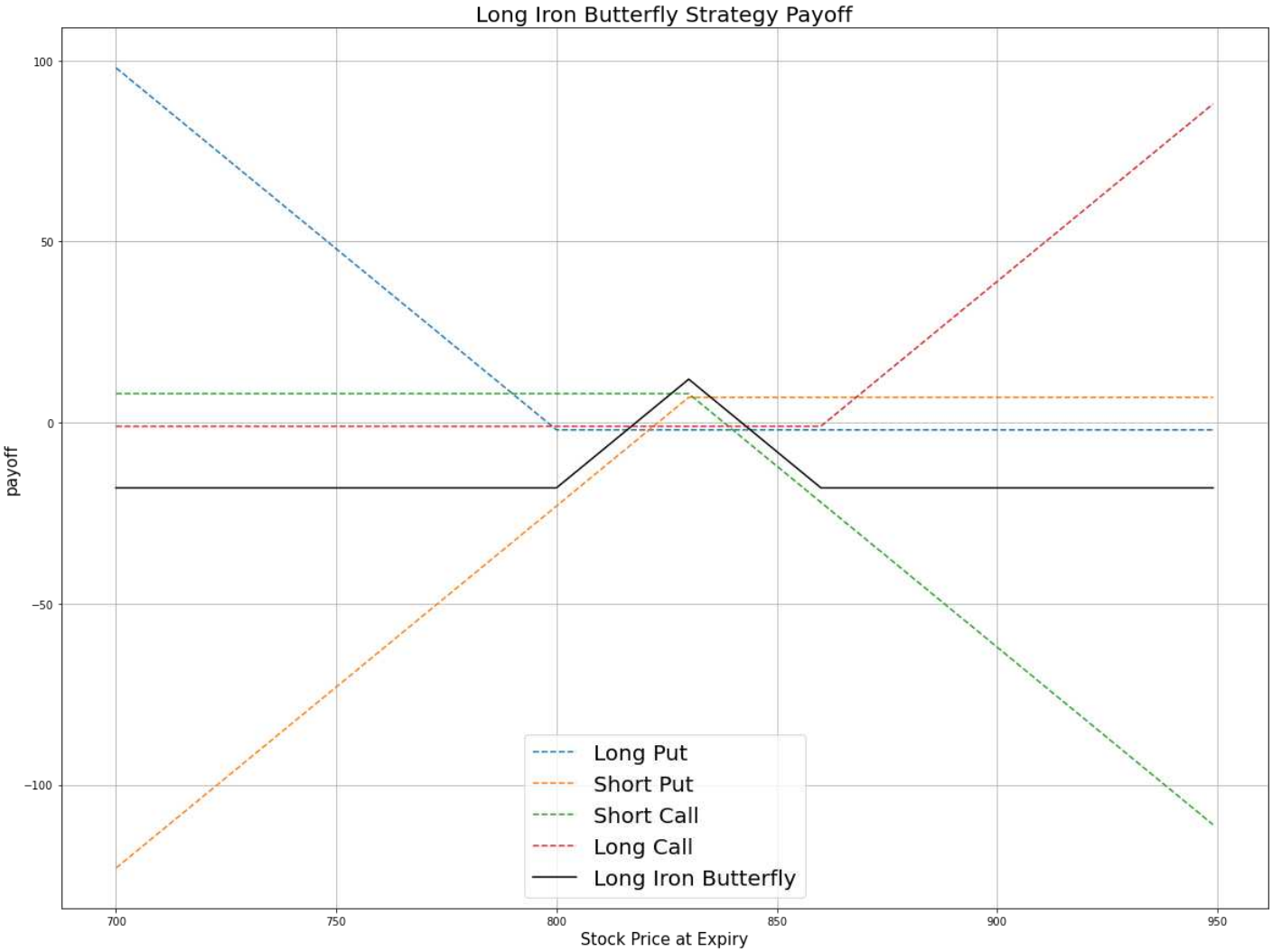
Strategy Payoff

The long call butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^C_{OTM})^{+}\\ C^{ATM}_T & = & (S_T - K^C_{ATM})^{+}\\ P^{OTM}_T & = & (K^P_{OTM} - S_T)^{+}\\ P^{ATM}_T & = & (K^P_{ATM} - S_T)^{+}\\ P_T & = & (C^{OTM}_T + P^{OTM}_T - C^{ATM}_T - P^{ATM}_T - C^{OTM}_0 - P^{OTM}_0 + C^{ATM}_0 + P^{ATM}_0)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & C^{OTM}_T & = & \textrm{OTM call value at time T}\\ & C^{ATM}_T & = & \textrm{ATM call value at time T}\\ & P^{OTM}_T & = & \textrm{OTM put value at time T}\\ & P^{ATM}_T & = & \textrm{ATM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^C_{OTM} & = & \textrm{OTM call strike price}\\ & K^C_{ATM} & = & \textrm{ATM call strike price}\\ & K^P_{OTM} & = & \textrm{OTM put strike price}\\ & K^P_{ATM} & = & \textrm{ATM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{OTM}_0 & = & \textrm{OTM call value at position opening (debit paid)}\\ & C^{ATM}_0 & = & \textrm{ATM call value at position opening (credit received)}\\ & P^{OTM}_0 & = & \textrm{OTM put value at position opening (debit paid)}\\ & P^{ATM}_0 & = & \textrm{ATM put value at position opening (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded an OTM call at $855.00 for $1.35 , an OTM put at $810.00 for $1.50 , an ATM call at $832.50 for $10.30 , and an ATM put at $832.50 for $9.50 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received, C^{ATM}_0 + P^{ATM}_0 - C^{OTM}_0 - P^{OTM}_0. It occurs when the underlying price stays the same as when you opened the trade.

The maximum loss is K^C_{OTM} - K^C_{ATM} - C^{ATM}_0 - P^{ATM}_0 + C^{OTM}_0 + P^{OTM}_0. It occurs when the underlying price is below the OTM put strike price or above the OTM call strike price at expiration.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM call	1.35	855.00
OTM put	1.50	810.00
ATM call	10.30	832.50
ATM put	9.50	832.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^C_{OTM})^{+}\\ & = & (843.25-855.00)^{+}\\ & = & 0\\ C^{ATM}_T & = & (S_T - K^C_{ATM})^{+}\\ & = & (843.25-832.50)^{+}\\ & = & 10.75\\ P^{OTM}_T & = & (K^P_{OTM} - S_T)^{+}\\ & = & (810.00-843.25)^{+}\\ & = & 0\\ P^{ATM}_T & = & (K^P_{ATM} - S_T)^{+}\\ & = & (832.50.00-843.25)^{+}\\ & = & 0\\ P_T & = & (C^{ATM}_T + P^{ATM}_T - C^{OTM}_T - P^{OTM}_T - C^{ATM}_0 - P^{ATM}_0 + C^{OTM}_0 + P^{OTM}_0)\times m - fee\\ & = & (10.75+0-0-0-10.30-9.50+1.35+1.50)\times100-1\times4\\ & = & -624 \end{array}

So, the strategy losses $624.

The following algorithm implements a long iron butterfly Option strategy:

```py
class LongButterflyOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.iron_butterfly(30, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select expiry
        expiry = max([x.expiry for x in chain])

        # Separate the call and put contracts
        calls = [i for i in chain if i.right == OptionRight.CALL and i.expiry == expiry]
        puts = [i for i in chain if i.right == OptionRight.PUT and i.expiry == expiry]
        if not calls or not puts:
            return

        # Get the ATM and OTM strike prices
        atm_strike = sorted(calls, key = lambda x: abs(x.strike - chain.underlying.price))[0].strike
        otm_put_strike = min([x.strike for x in puts])
        otm_call_strike = 2 * atm_strike - otm_put_strike

        iron_butterfly = OptionStrategies.iron_butterfly(self._symbol, otm_put_strike, atm_strike, otm_call_strike, expiry)
        self.buy(iron_butterfly, 1)
```

Option Strategies

Short Iron Butterfly

Introduction

The Short Iron Butterfly is an option strategy which involves four Option contracts. All the contracts have the same underlying stock and expiration, but the order of strike prices for the four contracts is A>B>C. The following table describes the strike price of each contract:

Position	Strike
1 OTM call	A
-1 ATM call	B
-1 ATM put	B
1 OTM put	C=B-(A-B)

The short call butterfly consists of buying an OTM call, buying an OTM put, selling an ATM call, and selling an ATM put. This strategy profits from an increase in price movement (implied volatility) and from time decay value since ATM options decay sharper.

Implementation

Follow these steps to implement the short iron butterfly strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.iron_butterfly(30, 5));
```

   The `iron_butterfly` filter narrows the universe down to just the four contracts you need to form a short iron butterly.

2. In the `on_data` method, select the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select expiry
    expiry = max([x.expiry for x in chain])

    # Separate the call and put contracts
    calls = [i for i in chain if i.right == OptionRight.CALL and i.expiry == expiry]
    puts = [i for i in chain if i.right == OptionRight.PUT and i.expiry == expiry]
    if not calls or not puts:
        return

    # Get the ATM and OTM strike prices
    atm_strike = sorted(calls, key = lambda x: abs(chain.underlying.price - x.strike))[0].strike
    otm_put_strike = min([x.strike for x in puts])
    otm_call_strike = 2 * atm_strike - otm_put_strike
```

3. In the `on_data` method, select the contracts and place the orders.

Approach A: Call the `OptionStrategies.short_iron_butterfly` method with the details of each leg and then pass the result to the `buy` method.

```
PY
short_iron_butterfly = OptionStrategies.short_iron_butterfly(self._symbol, otm_put_strike, atm_strike, otm_call_strike, expiry)
self.buy(short_iron_butterfly, 2)
```

Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
PY
# Select the contracts
atm_call = [x for x in calls if x.strike == atm_strike][0]
atm_put = [x for x in puts if x.strike == atm_strike][0]
otm_call = [x for x in calls if x.strike == otm_call_strike][0]
otm_put = [x for x in puts if x.strike == otm_put_strike][0]

legs = [
    Leg.create(atm_call.symbol, 1),
    Leg.create(atm_put.symbol, 1),
    Leg.create(otm_call.symbol, -1),
    Leg.create(otm_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

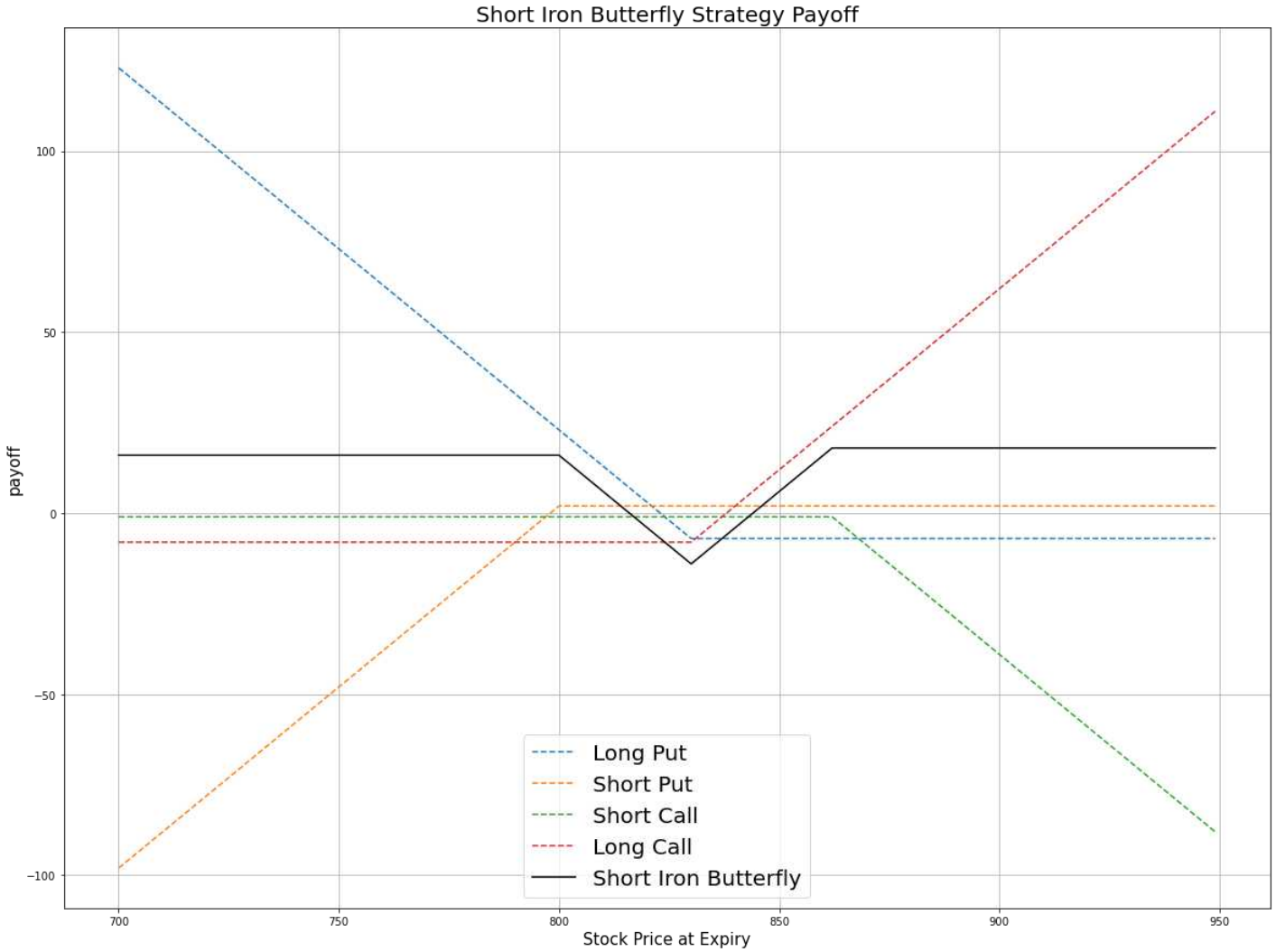
Strategy Payoff

The short iron butterfly is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{OTM}_T &=& (S_T - K^C_{OTM})^{+}\\ C^{ATM}_T &=& (S_T - K^C_{ATM})^{+}\\ P^{OTM}_T &=& (K^P_{OTM} - S_T)^{+}\\ P^{ATM}_T &=& (K^P_{ATM} - S_T)^{+}\\ P_T &=& (C^{ATM}_T + P^{ATM}_T - C^{OTM}_T - P^{OTM}_T - C^{ATM}_0 - P^{ATM}_0 + C^{OTM}_0 + P^{OTM}_0)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & C^{OTM}_T &=& \textrm{OTM call value at time T}\\ & C^{ATM}_T &=& \textrm{ATM call value at time T}\\ & P^{OTM}_T &=& \textrm{OTM put value at time T}\\ & P^{ATM}_T &=& \textrm{ATM put value at time T}\\ & S_T &=& \textrm{Underlying asset price at time T}\\ & K^C_{OTM} &=& \textrm{OTM call strike price}\\ & K^C_{ATM} &=& \textrm{ATM call strike price}\\ & K^P_{OTM} &=& \textrm{OTM put strike price}\\ & K^P_{ATM} &=& \textrm{ATM put strike price}\\ & P_T &=& \textrm{Payout total at time T}\\ & C^{OTM}_0 &=& \textrm{OTM call value at position opening (credit received)}\\ & C^{ATM}_0 &=& \textrm{ATM call value at position opening (debit paid)}\\ & P^{OTM}_0 &=& \textrm{OTM put value at position opening (credit received)}\\ & P^{ATM}_0 &=& \textrm{ATM put value at position opening (debit paid)}\\ & m &=& \textrm{Contract multiplier}\\ & T &=& \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded an OTM call at $855.00 for $2.35 , an OTM put at $810.00 for $2.75 , an ATM call at $832.50 for $8.10 , and an ATM put at $832.50 for $7.40 .

The following chart shows the payoff at expiration:



The maximum profit is K^C_{OTM} - K^C_{ATM} - C^{ATM}_0 - P^{ATM}_0 + C^{OTM}_0 + P^{OTM}_0. It occurs when the underlying price is below the OTM put strike price or above the OTM call strike price at expiration.

The maximum loss is the net debit paid, C^{OTM}_0 + P^{OTM}_0 - C^{ATM}_0 - P^{ATM}_0. It occurs when the underlying price stays the same as when you opened the trade.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
OTM call	2.35	855.00
OTM put	2.75	810.00
ATM call	8.10	832.50
ATM put	7.40	832.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^C_{OTM})^{+}\\ & = & (843.25-855.00)^{+}\\ & = & 0\\ C^{ATM}_T & = & (S_T - K^C_{ATM})^{+}\\ & = & (843.25-832.50)^{+}\\ & = & 10.75\\ P^{OTM}_T & = & (K^P_{OTM} - S_T)^{+}\\ & = & (810.00-843.25)^{+}\\ & = & 0\\ P^{ATM}_T & = & (K^P_{ATM} - S_T)^{+}\\ & = & (832.50.00-843.25)^{+}\\ & = & 0\\ P_T & = & (C^{OTM}_T + P^{OTM}_T - C^{ATM}_T - P^{ATM}_T - C^{OTM}_0 - P^{OTM}_0 + C^{ATM}_0 + P^{ATM}_0)\times m - fee\\ & = & (0+0-10.75-0-2.35-2.75+8.10+7.40)\times100-1\times4\\ & = & -39 \end{array}

So, the strategy losses $39.

The following algorithm implements a short iron butterfly Option strategy:

```py
class ShortButterflyOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.iron_butterfly(30, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select expiry
        expiry = max([x.expiry for x in chain])

        # Separate the call and put contracts
        calls = [i for i in chain if i.right == OptionRight.CALL and i.expiry == expiry]
        puts = [i for i in chain if i.right == OptionRight.PUT and i.expiry == expiry]
        if not calls or not puts:
            return

        # Get the ATM and OTM strike prices
        atm_strike = sorted(calls, key = lambda x: abs(x.strike - chain.underlying.price))[0].strike
        otm_put_strike = min([x.strike for x in puts])
        otm_call_strike = 2 * atm_strike - otm_put_strike

        short_iron_butterfly = OptionStrategies.short_iron_butterfly(self._symbol, otm_put_strike, atm_strike, otm_call_strike, expiry)
        self.buy(short_iron_butterfly, 1)
```

Option Strategies

Long Iron Condor

Introduction

The Long Iron Condor is an Option strategy that consists of four contracts. All the contracts have the same underlying Equity and expiration, but the order of strike prices is A>B>C>D. The following table describes the strike prices of each contract:

Position	Strike
1 far-OTM call	A
-1 near-OTM call	B, where B > underlying\ price
-1 near-OTM put	C, where C < underlying\ price
1 far-OTM put	D, where C-D = A-B

The long iron condor consists of buying a far OTM call, buying a far OTM put, selling a near ATM call, and selling a near ATM put. This strategy profits from an decrease in price movement (implied volatility) and time decay since ATM options decay sharper.

Implementation

Follow these steps to implement the long iron condor strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.iron_condor(30, 5, 10))
```

   The `iron_condor` filter narrows the universe down to just the four contracts you need to form a long iron condor.

2. In the `on_data` method, select the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio[self._symbol.underlying].invested:
        self.liquidate()

    if self.portfolio.invested or not self.is_market_open(self._symbol):
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find put and call contracts with the farthest expiry
    expiry = max([x.expiry for x in chain])
    chain = sorted([x for x in chain if x.expiry == expiry], key = lambda x: x.strike)

    put_contracts = [x for x in chain if x.right == OptionRight.PUT]
    call_contracts = [x for x in chain if x.right == OptionRight.CALL]

    if len(call_contracts) < 2 or len(put_contracts) < 2:
        return

    # Select the strategy legs
    near_call = call_contracts[0]
    far_call = call_contracts[1]
    near_put = put_contracts[1]
    far_put = [x for x in put_contracts if x.Strike == near_put.strike - far_call.strike + near_call.strike][0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.iron_condor` method with the details of each leg and then pass the result to the `buy` method.

```
PY
iron_condor = OptionStrategies.iron_condor(
    self._symbol,
    far_put.strike,
    near_put.strike,
    near_call.strike,
    far_call.strike,
    expiry)

self.buy(iron_condor, 2)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
PY
legs = [
    Leg.create(far_put.symbol, 1),
    Leg.create(near_put.symbol, -1),
    Leg.create(far_call.symbol, 1),
    Leg.create(near_call.symbol, -1)
]
self.combo_market_order(legs, 1)
```

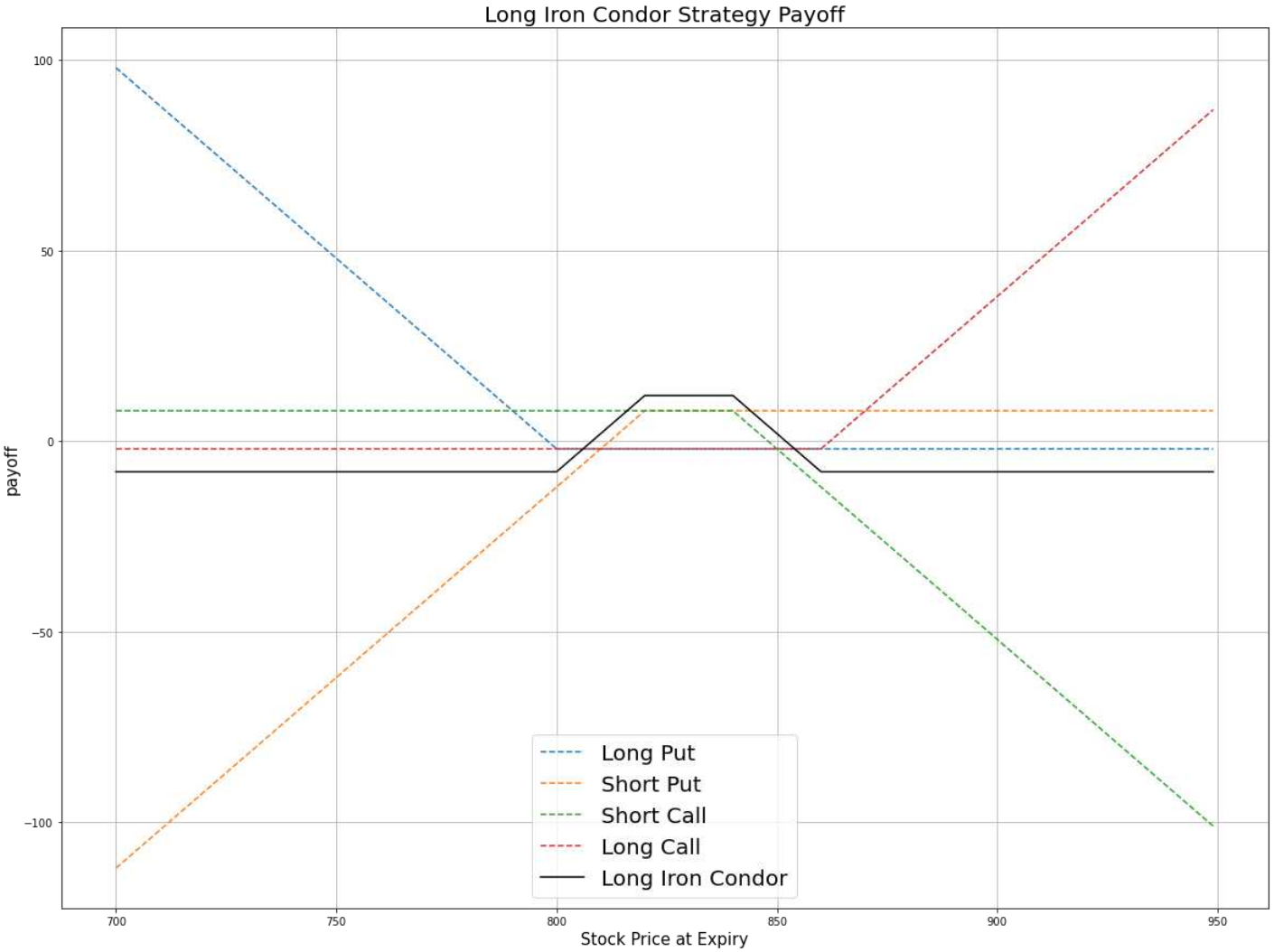
Strategy Payoff

This is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{far}_T & = & (S_T - K^C_{far})^{+}\\ C^{near}_T & = & (S_T - K^C_{near})^{+}\\ P^{far}_T & = & (K^P_{far} - S_T)^{+}\\ P^{near}_T & = & (K^P_{near} - S_T)^{+}\\ P_T & = & (C^{far}_T + P^{far}_T - C^{near}_T - P^{near}_T - C^{far}_0 - P^{far}_0 + C^{near}_0 + P^{near}_0)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & & & \\ C^{far}_T & = & \textrm{Far OTM call value at time T}\\ C^{near}_T & = & \textrm{Near OTM call value at time T}\\ P^{far}_T & = & \textrm{Far OTM put value at time T}\\ P^{near}_T & = & \textrm{Near ATM put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^C_{far} & = & \textrm{Far OTM call strike price}\\ K^C_{near} & = & \textrm{Near OTM call strike price}\\ K^P_{far} & = & \textrm{Far OTM put strike price}\\ K^P_{near} & = & \textrm{Near OTM put strike price}\\ P_T & = & \textrm{Payout total at time T}\\ C^{far}_0 & = & \textrm{Far OTM call value at position opening (credit received)}\\ C^{near}_0 & = & \textrm{Near OTM call value at position opening (debit paid)}\\ P^{far}_0 & = & \textrm{Far OTM put value at position opening (credit received)}\\ P^{near}_0 & = & \textrm{Near OTM put value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a far-OTM call at $857.50 for $1.85 , a near-OTM call at $852.50 for $1.65 , a far-OTM put at $815.00 for $3.80 , and a near-OTM put at $820.00 for $3.50 .

The following chart shows the payoff at expiration:



The maximum profit is the net credit received after commission when opening the trade, where K^P_{OTM} < S_T < K^C_{OTM}.

The maximum loss is K^C_{far} - K^C_{near} + C^{near}_0 + P^{near}_0 - C^{far}_0 - P^{far}_0, where K^P_{OTM} > S_T or S_T > K^C_{OTM}.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Far-OTM call	1.85	857.50
Far-OTM put	3.80	815.00
Near-OTM call	1.65	852.50
Near-OTM put	3.50	820.00
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} C^{far}_T & = & (S_T - K^C_{far})^{+}\\ & = & (843.25-857.50)^{+}\\ & = & 0\\ C^{near}_T & = & (S_T - K^C_{near})^{+}\\ & = & (843.25-852.50)^{+}\\ & = & 0\\ P^{far}_T & = & (K^P_{far} - S_T)^{+}\\ & = & (815.00-843.25)^{+}\\ & = & 0\\ P^{near}_T & = & (K^P_{near} - S_T)^{+}\\ & = & (820.00-843.25)^{+}\\ & = & 0\\ P_T & = & (C^{near}_T + P^{near}_T - C^{far}_T - P^{far}_T - C^{near}_0 - P^{near}_0 + C^{far}_0 + P^{far}_0)\times m - fee\\ & = & (0+0-0-0+1.65+3.50-1.85-3.80)\times100-1\times4\\ & = & -54 \end{array}

So, the strategy loses $54.

The following algorithm implements a long iron condor Option strategy:

```
class LongIronCondorOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.iron_condor(30, 5, 10))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested or not self.is_market_open(self._symbol):
            return

        chain = slice.option_chains.get(self._symbol)
        if not chain:
            return

        # Find put and call contracts with the farthest expiry
        expiry = max([x.expiry for x in chain])
        chain = sorted([x for x in chain if x.expiry == expiry], key = lambda x: x.strike)

        put_contracts = [x for x in chain if x.right == OptionRight.PUT]
        call_contracts = [x for x in chain if x.right == OptionRight.CALL]

        if len(call_contracts) < 2 or len(put_contracts) < 2:
            return

        # Select the strategy legs
        near_call = call_contracts[0]
        far_call = call_contracts[1]
        near_put = put_contracts[1]
        far_put = [x for x in put_contracts if x.Strike == near_put.strike - far_call.strike + near_call.strike][0]

        iron_condor = OptionStrategies.iron_condor(
            self._symbol,
            far_put.strike,
            near_put.strike,
            near_call.strike,
            far_call.strike,
            expiry)

        self.buy(iron_condor, 2)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
IndexOptionIronCondorAlgorithm.py Python

Option Strategies

Short Iron Condor

Introduction

The Short Iron Condor is an Option strategy that consists of four contracts. All the contracts have the same underlying Equity and expiration, but the order of strike prices is A>B>C>D. The following table describes the strike prices of each contract:

Position	Strike
-1 far-OTM call	A
1 near-OTM call	B, where B > underlying\ price
1 near-OTM put	C, where C < underlying\ price
-1 far-OTM put	D, where C-D = A-B

The short iron condor consists of selling a far OTM call, selling a far OTM put, buying a near OTM call, and buying a near OTM put. This strategy profits from a increase in price movement (implied volatility).

Implementation

Follow these steps to implement the short iron condor strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(500000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.iron_condor(30, 5, 10))
```

   The `iron_condor` filter narrows the universe down to just the four contracts you need to form a short iron condor.

2. In the `on_data` method, select the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio[self._symbol.underlying].invested:
        self.liquidate()

    if self.portfolio.invested or not self.is_market_open(self._symbol):
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find put and call contracts with the farthest expiry
    expiry = max([x.expiry for x in chain])
    chain = sorted([x for x in chain if x.expiry == expiry], key = lambda x: x.strike)

    put_contracts = [x for x in chain if x.right == OptionRight.PUT]
    call_contracts = [x for x in chain if x.right == OptionRight.CALL]

    if len(call_contracts) < 2 or len(put_contracts) < 2:
        return

    # Select the strategy legs
    far_put = put_contracts[0]
    near_put = put_contracts[1]
    near_call = call_contracts[0]
    far_call = call_contracts[1]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.short_iron_condor` method with the details of each leg and then pass the result to the `buy` method.

```
PY
short_iron_condor = OptionStrategies.short_iron_condor(
    self._symbol,
    far_put.strike,
    near_put.strike,
    near_call.strike,
    far_call.strike,
    expiry)

self.buy(short_iron_condor, 2)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
PY
legs = [
    Leg.create(far_put.symbol, -1),
    Leg.create(near_put.symbol, 1),
    Leg.create(far_call.symbol, -1),
    Leg.create(near_call.symbol, 1)
]
self.combo_market_order(legs, 1)
```

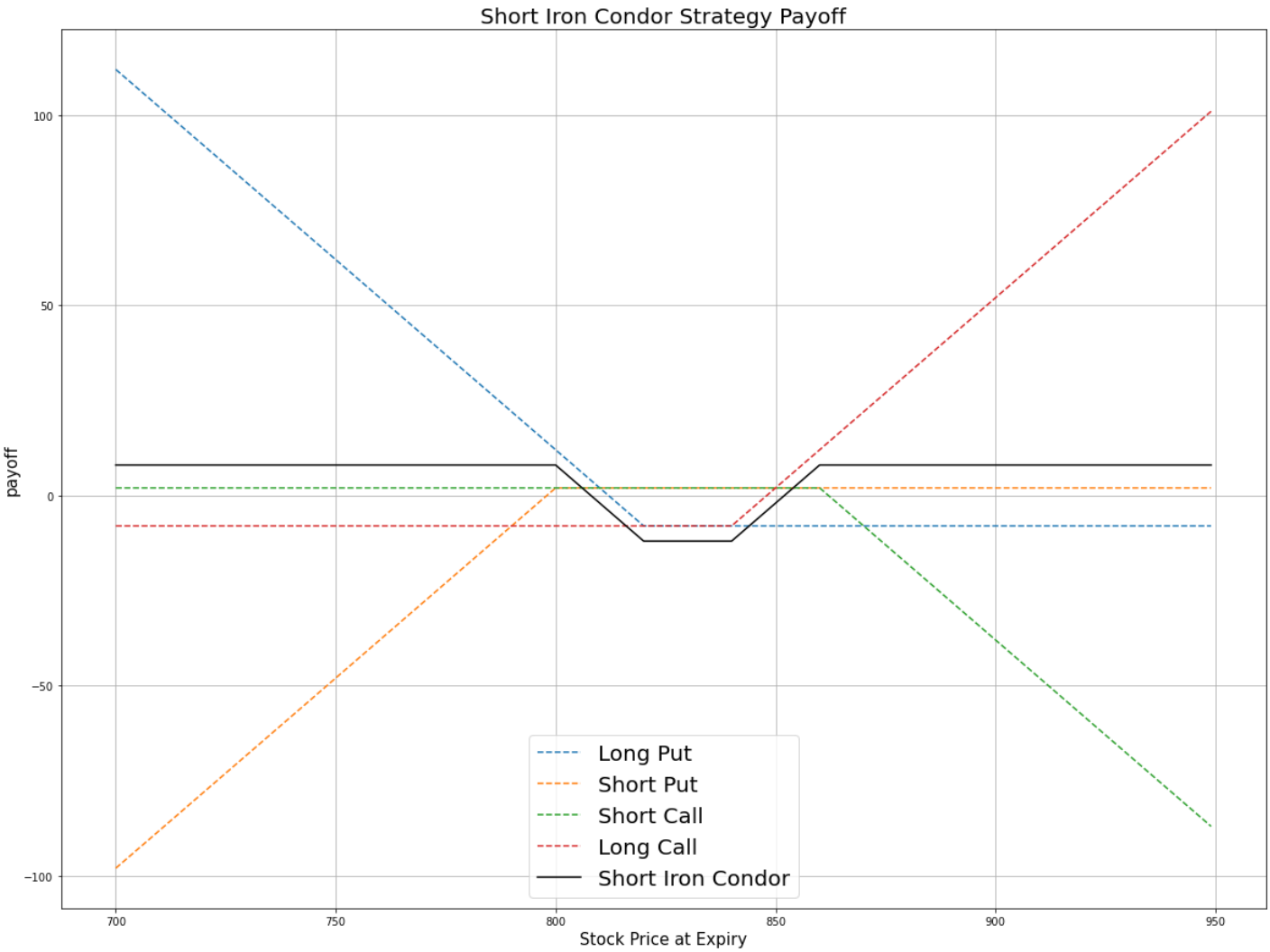
Strategy Payoff

This is a limited-reward-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{far}_T & = & (S_T - K^C_{far})^{+}\\ C^{near}_T & = & (S_T - K^C_{near})^{+}\\ P^{far}_T & = & (K^P_{far} - S_T)^{+}\\ P^{near}_T & = & (K^P_{near} - S_T)^{+}\\ P_T & = & (C^{near}_T + P^{near}_T - C^{far}_T - P^{far}_T - C^{near}_0 - P^{near}_0 + C^{far}_0 + P^{far}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{far}_T & = & \textrm{Far OTM call value at time T}\\ & C^{near}_T & = & \textrm{Near OTM call value at time T}\\ & P^{far}_T & = & \textrm{Far OTM put value at time T}\\ & P^{near}_T & = & \textrm{Near ATM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^C_{far} & = & \textrm{Far OTM call strike price}\\ & K^C_{near} & = & \textrm{Near OTM call strike price}\\ & K^P_{far} & = & \textrm{Far OTM put strike price}\\ & K^P_{near} & = & \textrm{Near OTM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{far}_0 & = & \textrm{Far OTM call value at position opening (credit received)}\\ & C^{near}_0 & = & \textrm{Near OTM call value at position opening (debit paid)}\\ & P^{far}_0 & = & \textrm{Far OTM put value at position opening (credit received)}\\ & P^{near}_0 & = & \textrm{Near OTM put value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a far-OTM call at $857.50 for $1.05 , a near-OTM call at $852.50 for $2.75 , a far-OTM put at $815.00 for $2.15 , and a near-OTM put at $820.00 for $4.80 .

The following chart shows the payoff at expiration:



The maximum profit is K^C_{far} - K^C_{near} - C^{near}_0 - P^{near}_0 + C^{far}_0 + P^{far}_0, where K^P_{OTM} > S_T or S_T > K^C_{OTM}.

The maximum loss is the net debit paid: C^{far}_0 + P^{far}_0 - C^{near}_0 - P^{near}_0, where K^P_{OTM} < S_T < K^C_{OTM}.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Far-OTM call	1.05	857.50
Far-OTM put	2.15	815.00
Near-OTM call	2.75	852.50
Near-OTM put	4.80	820.00
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} C^{far}_T & = & (S_T - K^C_{far})^{+}\\ & = & (843.25-857.50)^{+}\\ & = & 0\\ C^{near}_T & = & (S_T - K^C_{near})^{+}\\ & = & (843.25-852.50)^{+}\\ & = & 0\\ P^{far}_T & = & (K^P_{far} - S_T)^{+}\\ & = & (815.00-843.25)^{+}\\ & = & 0\\ P^{near}_T & = & (K^P_{near} - S_T)^{+}\\ & = & (820.00-843.25)^{+}\\ & = & 0\\ P_T & = & (C^{near}_T + P^{near}_T - C^{far}_T - P^{far}_T - C^{near}_0 - P^{near}_0 + C^{far}_0 + P^{far}_0)\times m - fee\\ & = & (0+0-0-0-2.75-4.80+1.05+2.15)\times100-1\times4\\ & = & -439 \end{array}

So, the strategy loses $439.

The following algorithm implements a short iron condor Option strategy:

PY

```
class ShortIronCondorOptionStrategy(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.iron_condor(30, 5, 10))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested or not self.is_market_open(self._symbol):
            return

        chain = slice.option_chains.get(self._symbol)
        if not chain:
            return

        # Find put and call contracts with the farthest expiry
        expiry = max([x.expiry for x in chain])
        chain = sorted([x for x in chain if x.expiry == expiry], key = lambda x: x.strike)

        put_contracts = [x for x in chain if x.right == OptionRight.PUT]
        call_contracts = [x for x in chain if x.right == OptionRight.CALL]

        if len(call_contracts) < 2 or len(put_contracts) < 2:
            return

        # Select the strategy legs
        near_call = call_contracts[0]
        far_call = call_contracts[1]
        near_put = put_contracts[1]
        far_put = [x for x in put_contracts if x.Strike == near_put.strike - far_call.strike + near_call.strike][0]

        short_iron_condor = OptionStrategies.short_iron_condor(
            self._symbol,
            far_put.strike,
            near_put.strike,
            near_call.strike,
            far_call.strike,
            expiry)

        self.buy(short_iron_condor, 2)
```

Option Strategies

Naked Call

Introduction

A Naked Call , also known as an uncovered call , consists of selling call Options without owning the underlying asset. It's called "naked" because you don't have any cover or protection in the form of owning the underlying asset, which exposes you to potentially unlimited risk. Naked calls aim to profit from the Option premium by selling calls. At any time for American Options or at expiration for European Options, if the stock moves below the strike price, you keep the premium. If the underlying price moves above the strike, the Option buyer can exercise the Options contract , which means you need buy the underlying at market price to fulfill your obligation to sell it at the strike price and keep the premium.

Implementation

Follow these steps to implement the naked call strategy:

1. In the `initialize` method, set the start date, end date, starting cash, and Options universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_call(30, 0))
```

   The `naked_call` filter narrows the universe down to just the one contract you need to form a naked call.

2. In the `on_data` method, select the Option contract.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM call with the farthest expiry
    expiry = max([x.expiry for x in chain])
    call_contracts = sorted([x for x in chain
        if x.right == OptionRight.CALL and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not call_contracts:
        return

    atm_call = call_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.naked_call` method with the details of each leg and then pass the result to the `buy` method.

```
PY
naked_call = OptionStrategies.naked_call(self._symbol, atm_call.strike, expiry)
self.buy(naked_call, 1)
```

   Approach B: Call the market_order or limit_order method.

```
PY
self.market_order(atm_call.symbol, -1)
```

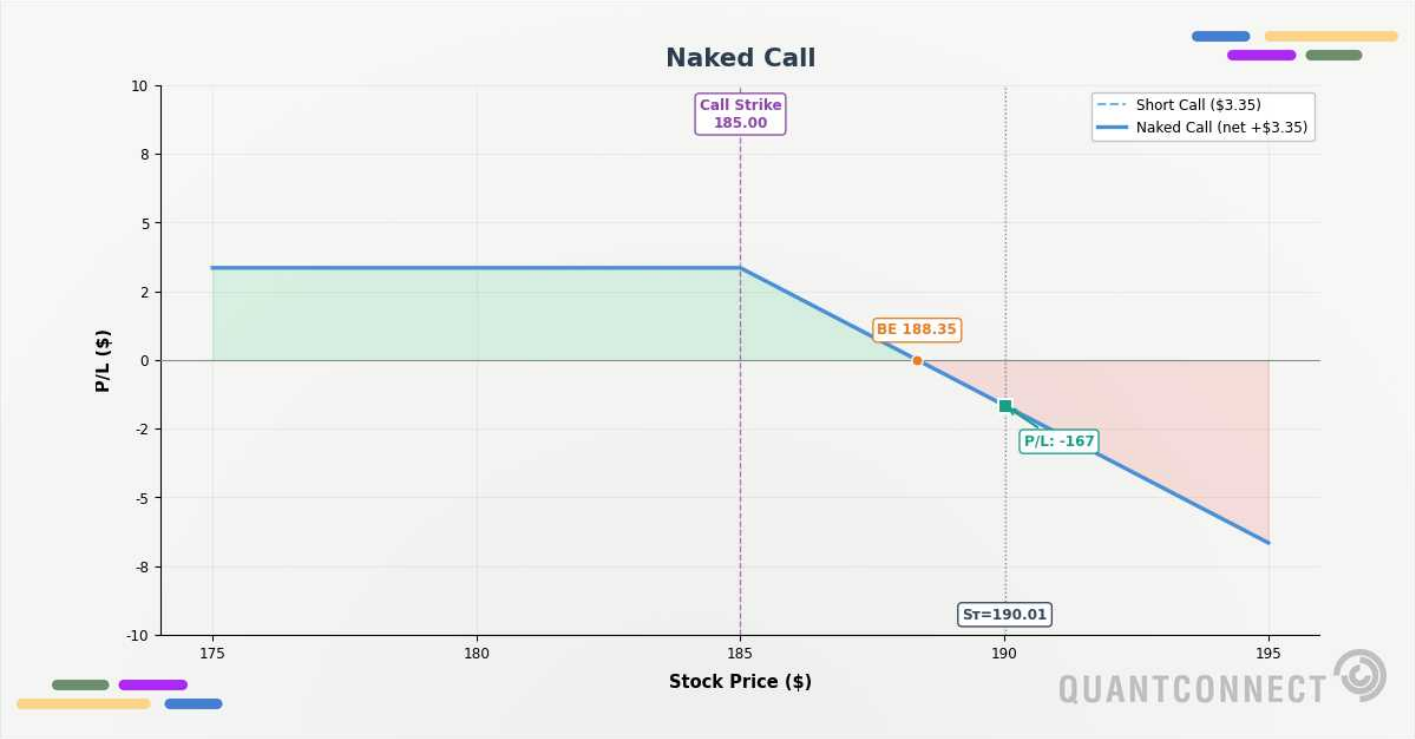
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{K}_T & = & (S_T - K)^{+}\\ P_T & = & (C^{K}_0 - C^{K}_T)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & & & \\ C^{K}_T & = & \textrm{Call value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K & = & \textrm{Call strike price}\\ P_T & = & \textrm{Payout total at time T}\\ C^{K}_0 & = & \textrm{Call price when the trade opened (credit received)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, you sold a call at a strike of $185.00 for $3.35 .

The following chart shows the payoff at expiration:



The maximum profit is C^{K}_0, which occurs when the underlying price is at or below the strike price of the call at expiration.

The maximum loss is unlimited because there is no limit to how high the underlying asset's price can rise.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	3.35	185.00
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{K}_T & = & (S_T - K)^{+}\\ & = & (190.01 - 185)^{+}\\ & = & 5.01\\ P_T & = & (C^{K}_0 - C^{K}_T)\times m - fee\\ & = & (3.35 - 5.01)\times m - fee\\ & = & -1.66 \times 100 - 1\\ & = & -167 \end{array}$$

So, the strategy loses $167.

The following algorithm implements a naked call Option strategy:

PY

```
class NakedCallAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.naked_call(30, 0))

        self.call = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.call and self.portfolio[self.call].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM call with the farthest expiry
        expiry = max([x.expiry for x in chain])
        call_contracts = sorted([x for x in chain
            if x.right == OptionRight.CALL and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not call_contracts:
            return

        atm_call = call_contracts[0]

        naked_call = OptionStrategies.naked_call(self.symbol, atm_call.strike, expiry)
        self.buy(naked_call, 1)

        self.call = atm_call.symbol
```

Option Strategies

Naked Put

Introduction

A Naked Put , also known as an uncovered put , consists of selling put Options without having a corresponding short position in the underlying security. It's called "naked" because you don't have any cover or protection in the form of a short position in the underlying asset, which exposes you to potentially unlimited risk. Naked puts aim to profit from the Option premium by selling puts. At any time for American Options or at expiration for European Options, if the stock moves above the strike price, you keep the premium. If the underlying price moves below the strike, the Option buyer can exercise the Options contract , which means you need sell the underlying at market price to fulfill your obligation to buy it at the strike price and keep the premium.

Implementation

Follow these steps to implement the naked put strategy:

1. In the `initialize` method, set the start date, end date, starting cash, and Options universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_put(30, 0))
```

   The `naked_put` filter narrows the universe down to just the one contract you need to form a naked put.

2. In the `on_data` method, select the Option contract.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM put with the farthest expiry
    expiry = max([x.expiry for x in chain])
    put_contracts = sorted([x for x in chain
        if x.right == OptionRight.PUT and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not put_contracts:
        return

    atm_put = put_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.naked_put` method with the details of each leg and then pass the result to the `buy` method.

```
naked_put = OptionStrategies.naked_put(self._symbol, atm_put.strike, expiry)
self.buy(naked_put, 1)
```

   Approach B: Call the market_order or limit_order method.

```
self.market_order(atm_put.symbol, -1)
```

Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} P^{K}_T & = & (K - S_T)^{+}\\ P_T & = & (P^{K}_0 - P^{K}_T)\times m - fee \end{array} \begin{array}{rcll} \textrm{where} & P^{K}_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & P^{K}_0 & = & \textrm{Put price when the trade opened (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, you sold a put at a strike of $185.00 for $1.37 .

The following chart shows the payoff at expiration:

The maximum profit is P^{K}_0, which occurs when the underlying price is at or above the strike price of the put at expiration.

The maximum loss is $K - P^{K}_0$, which ocurrs when the underlying price drops to zero.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Put	1.37	185.00
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{K}_T & = & (K - S_T)^{+}\\ & = & (185 - 190.01)^{+}\\ & = & 0.0\\ P_T & = & (P^{K}_0 - P^{K}_T)\times m - fee\\ & = & (1.37 - 0.0)\times m - fee\\ & = & 1.37 \times 100 - 1\\ & = & 136 \end{array}$$

So, the strategy gains $136.

The following algorithm implements a naked put Option strategy:

PY

```
class NakedPutAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.naked_put(30, 0))

        self.put = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.put and self.portfolio[self.put].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM put with the farthest expiry
        expiry = max([x.expiry for x in chain])
        put_contracts = sorted([x for x in chain
            if x.right == OptionRight.PUT and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not put_contracts:
            return

        atm_put = put_contracts[0]

        naked_put = OptionStrategies.naked_put(self.symbol, atm_put.strike, expiry)
        self.buy(naked_put, 1)

        self.put = atm_put.symbol
```

Option Strategies

Protective Call

Introduction

A Protective Call consists of a short position in a stock and a long position in a call Option for the same amount of stock. Protective calls aim to hedge the short position of a stock with a long ATM or slightly OTM call Option. At any time for American Options or at expiration for European Options, if the stock moves below the strike price, the Option contract becomes worthless but the short position acquires an unrealized gain. If the underlying price moves above the strike, you can exercise the Options contract and receive the underlying Equity, which closes your short position.

Implementation

Follow these steps to implement the protective call strategy:

1. In the `initialize` method, set the start date, end date, cash, and Options universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_call(30, 0))
```

   The `naked_call` filter narrows the universe down to just the one contract you need to form a protective call.

2. In the `on_data` method, select the Option contract.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM call with the farthest expiry
    expiry = max([x.expiry for x in chain])
    call_contracts = sorted([x for x in chain
        if x.right == OptionRight.CALL and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not call_contracts:
        return

    atm_call = call_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.protective_call` method with the details of each leg and then pass the result to the `buy` method.

```py
protective_call = OptionStrategies.protective_call(self._symbol, atm_call.strike, expiry)
self.buy(protective_call, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
legs = [
    Leg.create(atm_call.symbol, 1),
    Leg.create(chain.underlying.symbol, -chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{K}_T & = & (S_T - K)^{+}\\ P_T & = & (S_0 - S_T + C^{K}_T - C^{K}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{K}_T & = & \textrm{Call value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Call strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & C^{K}_0 & = & \textrm{Call price when the trade opened (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $186.94 at expiration. You held the underlying position and bought a call at a strike of $185.00 for $3.50 .

The following chart shows the payoff at expiration:

The maximum profit is $S_0 - C^{K}_0$, which occurs when the underlying price is 0.

The maximum loss is $S_0 - K - C^{K}_0$, which occurs when the underlying price is above the strike price.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	3.50	185.00
Underlying Equity at start of the trade	186.94	-
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{K}_T & = & (S_T - K)^{+}\\ & = & (190.01 - 185)^{+}\\ & = & 5.01\\ P_T & = & (S_0 - S_T + C^{K}_T - C^{K}_0)\times m - fee\\ & = & (186.94 - 190.01 + 5.01 - 3.50)\times m - fee\\ & = & -1.56 \times 100 - 2\\ & = & -158 \end{array}$$

So, the strategy loses $158.

The following algorithm implements a protective call Option strategy:

PY

```
class ProtectiveCallAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.naked_call(30, 0))

        self.call = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.call and self.portfolio[self.call].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM call with the farthest expiry
        expiry = max([x.expiry for x in chain])
        call_contracts = sorted([x for x in chain
            if x.right == OptionRight.CALL and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not call_contracts:
            return

        atm_call = call_contracts[0]

        protective_call = OptionStrategies.protective_call(self.symbol, atm_call.strike, expiry)
        self.buy(protective_call, 1)

        self.call = atm_call.symbol
```

Option Strategies

Protective Put

Introduction

A Protective Put consists of buying a long position in a stock and a long position in put Options for the same amount of stock. Protective puts aim to hedge the long position of a stock with a long ATM or slightly OTM put Option. At any time for American Options or at expiration for European Options, if the stock moves above the strike price, the Option contract becomes worthless but the long stock position acquires an unrealized gain. If the underlying price moves below the strike, you can exercise the Options contract , which sells your underlying position at the put Option strike price.

Implementation

Follow these steps to implement the protective put strategy:

1. In the `initialize` method, set the start date, end date, starting cash, and Options universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("IBM")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.naked_put(30, 0))
```

   The `naked_put` filter narrows the universe down to just the one contract you need to form a protective put.

2. In the `on_data` method, select the Option contract.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find ATM put with the farthest expiry
    expiry = max([x.expiry for x in chain])
    put_contracts = sorted([x for x in chain
        if x.right == OptionRight.PUT and x.expiry == expiry],
        key=lambda x: abs(chain.underlying.price - x.strike))

    if not put_contracts:
        return

    atm_put = put_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.protective_put` method with the details of each leg and then pass the result to the `buy` method.

```
PY
protective_put = OptionStrategies.protective_put(self._symbol, atm_put.strike, expiry)
self.buy(protective_put, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(atm_put.symbol, 1),
    Leg.create(chain.underlying.symbol, chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

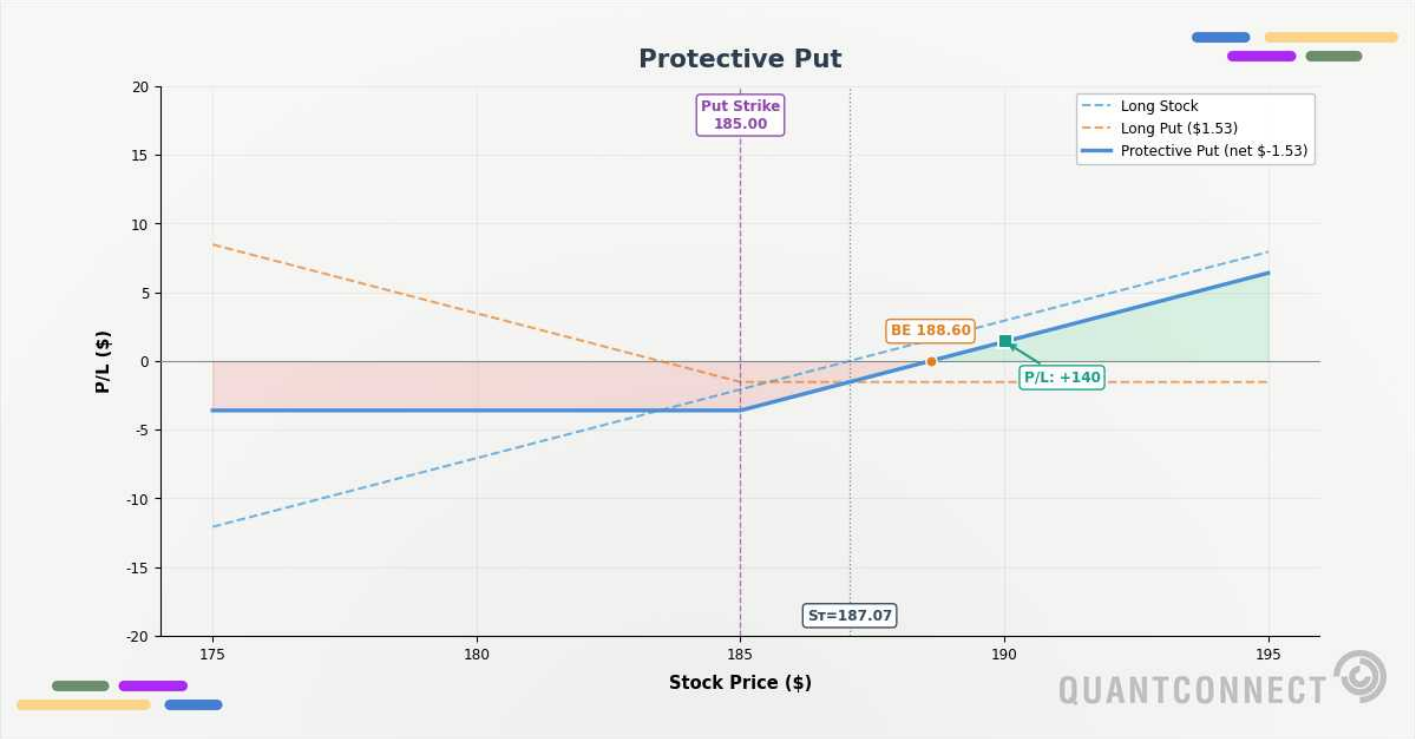
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcl} P^{K}_T & = & (K - S_T)^{+}\\ P_T & = & (S_T - S_0 + P^{K}_T - P^{K}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & P^{K}_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & P^{K}_0 & = & \textrm{Put price when the trade opened (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $187.07 at expiration. You held the underlying position and bought a put at a strike of $185.00 for $1.53 .

The following chart shows the payoff at expiration:



The maximum profit is $S_T - S_0 - P^{K}_0$, which occurs when the underlying price is above the $S_0 + P^{K}_0$.

The maximum loss is P^{K}_0, which occurs when the underlying price drops.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Put	1.53	185.00
Underlying Equity at start of the trade	187.07	-
Underlying Equity at expiration	190.01	-

Therefore, the payoff is

$$\begin{array}{rcl} P^{K}_T & = & (K - S_T)^{+}\\ & = & (185 - 190.1)^{+}\\ & = & 0\\ P_T & = & (S_T - S_0 + P^{K}_T - P^{K}_0)\times m - fee\\ & = & (190.01 - 187.07 + 0 - 1.53)\times m - fee\\ & = & 1.41 \times 100 - 2\\ & = & 139 \end{array}$$

So, the strategy gains $139.

The following algorithm implements a protective put Option strategy:

```py
class ProtectivePutAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("IBM")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.naked_put(30, 0))

        self.put = None

        # use the underlying equity as the benchmark
        self.set_benchmark(self.symbol.underlying)

    def on_data(self, slice: Slice) -> None:
        if self.put and self.portfolio[self.put].invested:
            return

        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find ATM put with the farthest expiry
        expiry = max([x.expiry for x in chain])
        put_contracts = sorted([x for x in chain
            if x.right == OptionRight.Put and x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if not put_contracts:
            return

        atm_put = put_contracts[0]

        protective_put = OptionStrategies.protective_put(self.symbol, atm_put.strike, expiry)
        self.buy(protective_put, 1)

        self.put = atm_put.symbol
```

Option Strategies

Protective Collar

Introduction

A Protective Collar is an Options strategy that consists of a covered call and a long put (protective put) with a lower strike price than the short call contract. In contrast to the covered call, the protective put component limits the drawdown of the strategy when the underlying price decreases too much.

Implementation

Follow these steps to implement the protective collar strategy:

1. In the `initialize` method, set the start date, set the end date, subscribe to the underlying Equity , and create an Option universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.protective_collar(30, -1, -10))
```

   The `protective_collar` filter narrows the universe down to just the two contracts you need to form a protective collar.

2. In the `on_data` method, select the Option contracts.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date
    expiry = max([x.expiry for x in chain])

    # Select the call and put contracts that expire on the selected date
    calls = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry]
    puts = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry]
    if not calls or not puts:
        return

    # Select the OTM contracts
    call = sorted(calls, key = lambda x: x.strike)[-1]
    put = sorted(puts, key = lambda x: x.strike)[0]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.protective_collar` method with the details of each leg and then pass the result to the `buy` method.

```
PY
protective_collar = OptionStrategies.protective_collar(self._symbol, call.strike, put.strike, expiry)
self.buy(protective_collar, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(call.symbol, -1),
    Leg.create(put.symbol, 1),
    Leg.create(chain.underlying.symbol, chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

This is a limited-profit-limited-loss strategy. The payoff is

$$\begin{array}{rcll} C_T & = & (S_T - K^{C})^{+}\\ P_T & = & (K^{P} - S_T)^{+}\\ Payoff_T & = & (S_T - S_0 - C_T + P_T + C_0 - P_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C_T & = & \textrm{Call value at time T}\\ & P_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{C} & = & \textrm{Call strike price}\\ & K^{P} & = & \textrm{Put strike price}\\ & Payoff_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & C_0 & = & \textrm{Call price when the trade opened (credit received)}\\ & P_0 & = & \textrm{Put price when the trade opened (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $833.17 at expiration. You held the underlying, bought a put at a strike of $822.50 for $6.00 , and sold a call at a strike of $845.00 for $2.85 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{C} - S_T + C_0 - P_0$. It occurs when the underlying price is at or above the strike price of the call at expiration.

The maximum profit is $S_T - K^{P} + C_0 - P_0$. It occurs when the underlying price is at or below the strike price of the put at expiration.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	2.85	845.00
Put	6.00	822.50
Underlying Equity at position opens	833.17	-
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

$$\begin{array}{rcll} C_T & = & (S_T - K^{C})^{+}\\ & = & (843.365 - 845.00)^{+}\\ & = & 0\\ P_T & = & (K^{P} - S_T)^{+}\\ & = & (822.50 - 843.365)^{+}\\ & = & 0\\ Payoff_T & = & (S_T - S_0 - C_T + P_T + C_0 - P_0)\times m - fee\\ & = & (843.25 - 833.17 - 0 + 0 + 2.85 - 6.00)\times100-1.00\times3\\ & = & 690\\ \end{array}$$

So, the strategy gains $690.

The following algorithm implements a protective collar Option strategy:

```py
class ProtectiveCollarOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        equity = self.add_equity("GOOG", Resolution.MINUTE)
        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.protective_collar(30, -1, -10))

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # choose the furthest expiration date within 30 days from now on
        expiry = sorted(chain, key = lambda x: x.expiry)[-1].expiry
        # filter the call options contracts
        call = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry]
        # filter the put options contracts
        put = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry]

        if not call or not put: return

        # select the strike prices of call and put contracts
        call_strike = sorted(call, key = lambda x: x.strike)[-1].strike
        put_strike = sorted(put, key = lambda x: x.strike)[0].strike

        protective_collar = OptionStrategies.protective_collar(self.symbol, call_strike, put_strike, expiry)
        self.buy(protective_collar, 1)

    def on_end_of_day(self, symbol):
        if symbol.value == "GOOG":
            self.log(f"{self.time}::{symbol}::{self.securities[symbol].price}")
```

Option Strategies

Long Straddle

Introduction

Long Straddle is an Options trading strategy that consists of buying an ATM call and an ATM put, where both contracts have the same underlying asset, strike price, and expiration date. This strategy aims to profit from volatile movements in the underlying stock, either positive or negative.

Implementation

Follow these steps to implement the long straddle strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.straddle(30))
```

   The `straddle` filter narrows the universe down to just the two contracts you need to form a long straddle.

2. In the `on_data` method, select the expiration date and strike price of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Find ATM options with the nearest expiry
    expiry = min([x.expiry for x in chain])
    strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.straddle` method with the details of each leg and then pass the result to the `buy` method.

```py
long_straddle = OptionStrategies.straddle(self._symbol, strike, expiry)
self.buy(long_straddle, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
contracts = [x for x in chain if x.expiry == expiry and x.strike == strike]
if len(contracts) < 2:
    return

atm_call = [x for x in contracts if x.right == OptionRight.CALL][0]
atm_put = [x for x in contracts if x.right == OptionRight.PUT][0]

legs = [
    Leg.create(atm_call.symbol, 1),
    Leg.create(atm_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

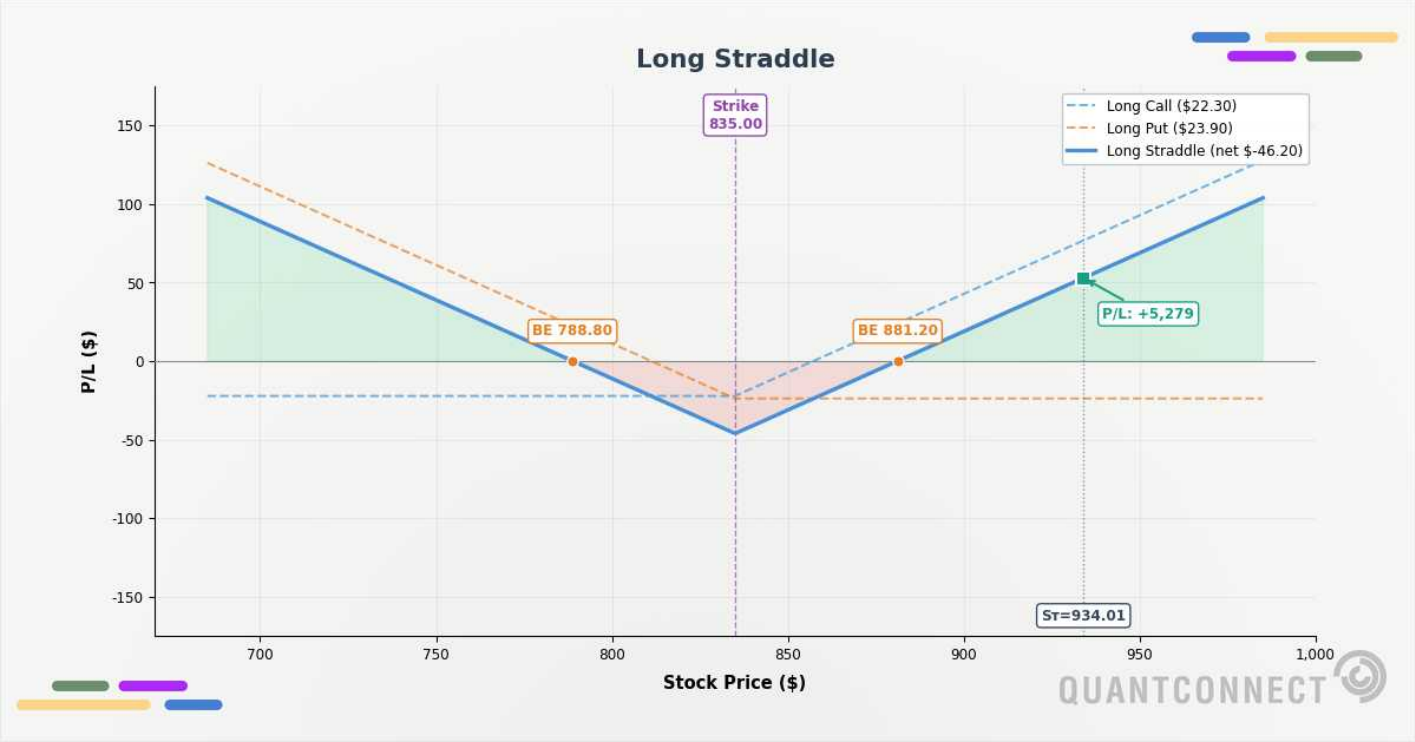
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{ATM}_T & = & (S_T - K^{C})^{+}\\ P^{ATM}_T & = & (K^{P} - S_T)^{+}\\ P_T & = & (C^{ATM}_T + P^{ATM}_T - C^{ATM}_0 - P^{ATM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{ATM}_T & = & \textrm{ATM call value at time T}\\ & P^{ATM}_T & = & \textrm{ATM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{C} & = & \textrm{ATM call strike price}\\ & K^{P} & = & \textrm{ATM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{ATM}_0 & = & \textrm{ATM call value at position opening (debit paid)}\\ & P^{ATM}_0 & = & \textrm{ATM put value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded a call and a put, both at a strike of $835.00 , for $22.30 and $23.90 respectively.

The following chart shows the payoff at expiration:



The maximum profit is unlimited if the underlying price rises to infinity or substantial, $K^{P} - C^{OTM}_0 - P^{OTM}_0$, if it drops to zero at expiration.

The maximum loss is the net debit paid, $C^{ATM}_0 + P^{ATM}_0$. It occurs when the underlying price is the same at expiration as it was when you opened the trade. In this case, both Options expire worthless.

Example

The following table shows the price details of the assets in the algorithm at Option expiration (2017-05-20):

Asset	Price ($)	Strike ($)
Call	22.30	835.00
Put	23.90	835.00
Underlying Equity at expiration	934.01	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{ATM}_T & = & (S_T - K^{C})^{+}\\ & = & (934.01-835.00)^{+}\\ & = & 99.01\\ P^{ATM}_T & = & (K^{P} - S_T)^{+}\\ & = & (835.00-934.01)^{+}\\ & = & 0\\ P_T & = & (C^{ATM}_T + P^{ATM}_T - C^{ATM}_0 - P^{ATM}_0)\times m - fee\\ & = & (99.01+0-22.3-23.9)\times100-1.00\times2\\ & = & 5279 \end{array}$$

So, the strategy gains $5,279.

The following algorithm implements a long straddle Option strategy:

PY

```
class LongStraddleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.straddle(30))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        chain = slice.option_chains.get(self.symbol, None)
        if not chain:
            return

        # Find ATM options with the nearest expiry
        expiry = min([x.expiry for x in chain])
        contracts = sorted([x for x in chain if x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if len(contracts) < 2:
            return

        # The first two contracts are the ATM Call and the ATM Put
        contracts = contracts[0:2]

        long_straddle = OptionStrategies.straddle(self.symbol, contracts[0].strike, expiry)
        self.buy(long_straddle, 1)
```

Option Strategies

Short Straddle

Introduction

A Short Straddle consists of selling a call and a put, where both contracts have the same underlying asset, strike price (normally at-the-money), and expiration date. If you enter a short straddle, you bet that the underlying asset will remain relatively stable and not experience significant price movements before the Option expires.

Implementation

Follow these steps to implement the short straddle strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.straddle(30))
```

   The `straddle` filter narrows the universe down to just the two contracts you need to form a short straddle.

2. In the `on_data` method, select the expiration date and strike price of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Find ATM options with the nearest expiry
    expiry = min([x.expiry for x in chain])
    strike = sorted(chain, key=lambda x: abs(x.strike - chain.underlying.price))[0].strike
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_straddle` method with the details of each leg and then pass the result to the `buy` method.

```py
short_straddle = OptionStrategies.short_straddle(self._symbol, contracts[0].strike, expiry)
self.buy(short_straddle, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
contracts = [x for x in chain if x.expiry == expiry and x.strike == strike]
if len(contracts) < 2:
    return

atm_call = [x for x in contracts if x.right == OptionRight.CALL][0]
atm_put = [x for x in contracts if x.right == OptionRight.PUT][0]

legs = [
    Leg.create(atm_call.symbol, -1),
    Leg.create(atm_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

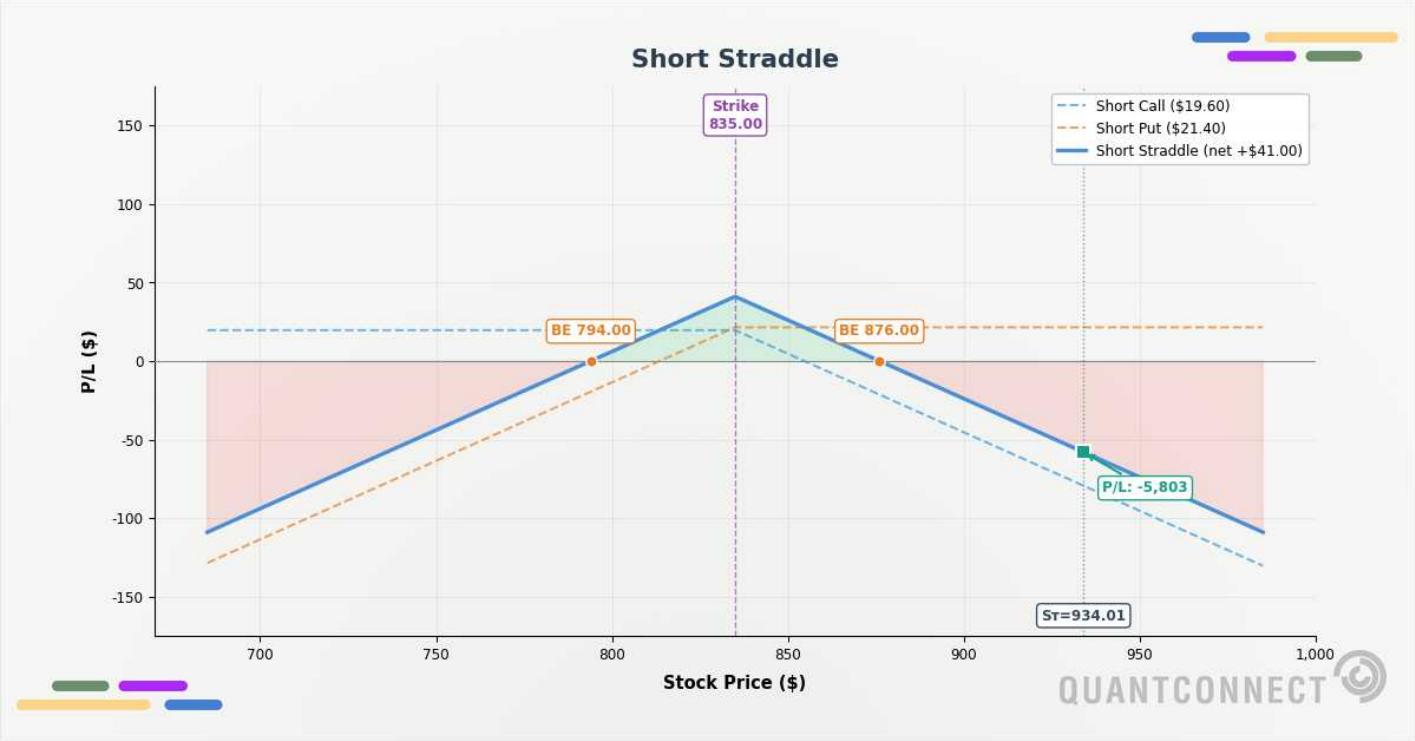
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{ATM}_T & = & (S_T - K^{C})^{+}\\ P^{ATM}_T & = & (K^{P} - S_T)^{+}\\ P_T & = & (-C^{ATM}_T - P^{ATM}_T + C^{ATM}_0 + P^{ATM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C^{ATM}_T & = & \textrm{ATM call value at time T}\\ & P^{ATM}_T & = & \textrm{ATM put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{C} & = & \textrm{ATM call strike price}\\ & K^{P} & = & \textrm{ATM put strike price}\\ & P_T & = & \textrm{Payout total at time T}\\ & C^{ATM}_0 & = & \textrm{ATM call value at position opening (debit paid)}\\ & P^{ATM}_0 & = & \textrm{ATM put value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded a call and a put, both at a strike of $835.00 , for $19.60 and $21.40 respectively.

The following chart shows the payoff at expiration:



The maximum profit is $C^{ATM}_0 + P^{ATM}_0$. It occurs when the price of the underlying asset remains exactly at the strike price upon expiration.

The maximum loss is unlimited if the underlying price rises to infinity or drops to zero at expiration.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm at Option expiration (2017-05-20):

Asset	Price ($)	Strike ($)
Call	19.6	835.00
Put	21.4	835.00
Underlying Equity at early (2017-05-15) call assignment	932.22	-
Underlying Equity at expiration	934.01	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{ATM}_T & = & (S_T - K^{C})^{+}\\ & = & (934.01-835.00)^{+}\\ & = & 99.01\\ P^{ATM}_T & = & (K^{P} - S_T)^{+}\\ & = & (835.00-934.01)^{+}\\ & = & 0\\ P_T & = & (-C^{ATM}_T - P^{ATM}_T + C^{ATM}_0 + P^{ATM}_0)\times m - fee\\ & = & (-99.01-0+19.6+21.4)\times 100 - 2.00\\ & = & -5803 \end{array}$$

So, the strategy loses $5,803. The early assigment doesn't influence the payoff.

The following algorithm implements a short straddle Option strategy:

PY

```
class LongStraddleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.straddle(30))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        chain = slice.option_chains.get(self.symbol, None)
        if not chain:
            return

        # Find ATM options with the nearest expiry
        expiry = min([x.expiry for x in chain])
        contracts = sorted([x for x in chain if x.expiry == expiry],
            key=lambda x: abs(chain.underlying.price - x.strike))

        if len(contracts) < 2:
            return

        # The first two contracts are the ATM Call and the ATM Put
        contracts = contracts[0:2]

        short_straddle = OptionStrategies.short_straddle(self.symbol, contracts[0].strike, expiry)
        self.buy(short_straddle, 1)
```

Option Strategies

Long Strangle

Introduction

Long Strangle is an Options trading strategy that consists of simultaneously buying an OTM put and an OTM call, where both contracts have the same underlying asset and expiration date. This strategy aims to profit from volatile movements in the underlying stock, either positive or negative.

Compared to a long straddle , the net debit of a long strangle is lower since OTM Options are cheaper. Additionally, the losing range of a long straddle is wider and the strike spread is wider.

Implementation

Follow these steps to implement the long strangle strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.strangle(30, 5, -10))
```

   The `strangle` filter narrows the universe down to just the two contracts you need to form a long strangle.

2. In the `on_data` method, select the contracts of the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chain.get(self._symbol)
    if not chain:
        return

    # Find options with the farthest expiry
    expiry = max([x.expiry for x in chain])
    contracts = [contract for contract in chain if contract.expiry == expiry]

    # Order the OTM calls by strike to find the nearest to ATM
    call_contracts = sorted([contract for contract in contracts
        if contract.right == OptionRight.CALL and
            contract.strike > chain.underlying.price],
        key=lambda x: x.Strike)
    if not call_contracts:
        return

    # Order the OTM puts by strike to find the nearest to ATM
    put_contracts = sorted([contract for contract in contracts
        if contract.right == OptionRight.PUT and
           contract.strike < chain.underlying.price],
        key=lambda x: x.Strike, reverse=True)
    if not put_contracts:
        return

    call = call_contracts[0]
    put = put_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.strangle` method with the details of each leg and then pass the result to the `buy` method.

```
long_strangle = OptionStrategies.strangle(self._symbol, call.strike, put.strike, expiry)
self.buy(long_strangle, 1)
```

Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(call.symbol, 1),
    Leg.create(put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

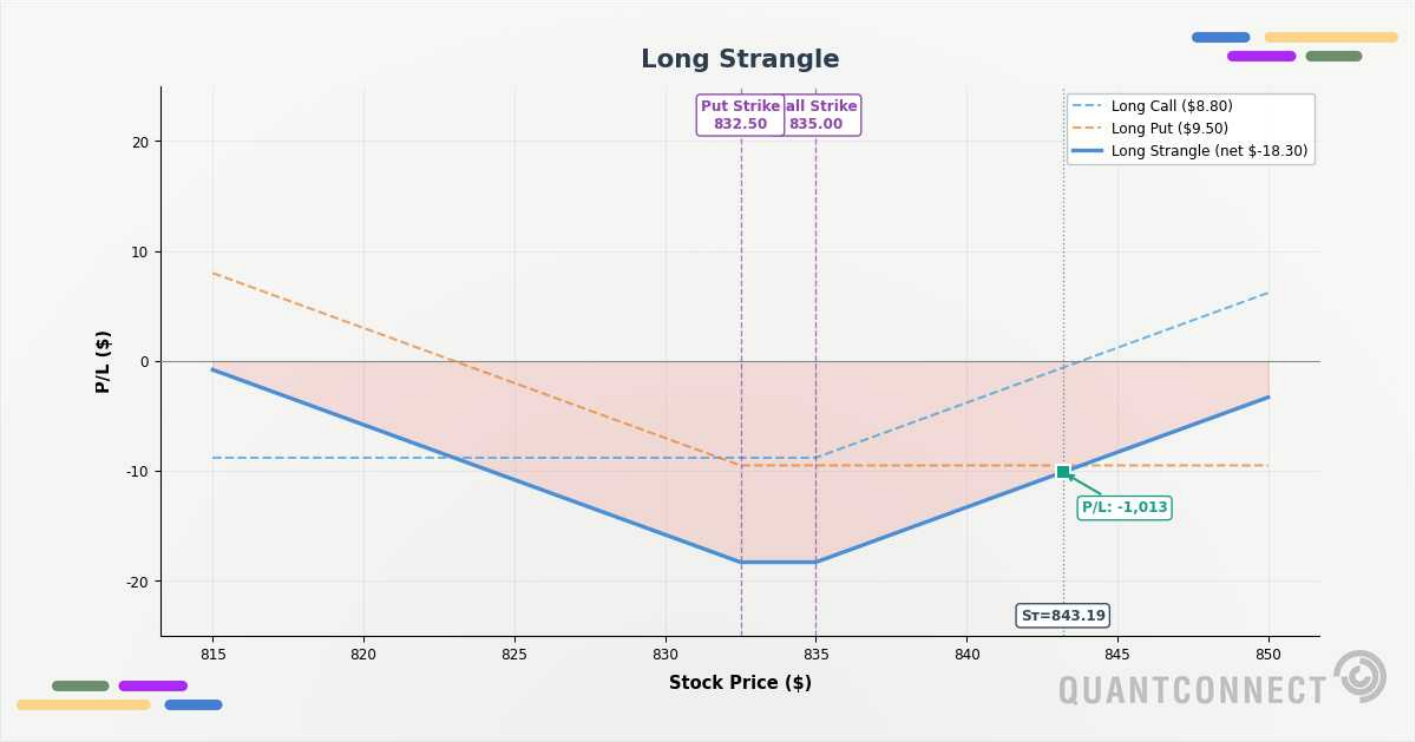
Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{C})^{+}\\ P^{OTM}_T & = & (K^{P} - S_T)^{+}\\ P_T & = & (C^{OTM}_T + P^{OTM}_T - C^{OTM}_0 - P^{OTM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{OTM}_T & = & \textrm{OTM call value at time T}\\ P^{OTM}_T & = & \textrm{OTM put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{C} & = & \textrm{OTM call strike price}\\ K^{P} & = & \textrm{OTM put strike price}\\ P_T & = & \textrm{Payout total at time T}\\ C^{OTM}_0 & = & \textrm{OTM call value at position opening (debit paid)}\\ P^{OTM}_0 & = & \textrm{OTM put value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded a call at a strike of $835.00 for $8.80 and a put at a strike of $832.50 for $9.50 .

The following chart shows the payoff at expiration:



The maximum profit is unlimited if the underlying price rises to infinity at expiration.

The maximum loss is the net debit paid, $C^{OTM}_0 + P^{OTM}_0$. It occurs when the underlying price at expiration is the same as when you opened the trade. In this case, both Options expire worthless.

Example

The following table shows the price details of the assets in the algorithm at Option expiration (04/22/2017):

Asset	Price ($)	Strike ($)
Call	8.80	835.00
Put	9.50	832.50
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{C})^{+}\\ & = & (843.19-835.00)^{+}\\ & = & 8.19\\ P^{OTM}_T & = & (K^{P} - S_T)^{+}\\ & = & (832.50-843.19)^{+}\\ & = & 0\\ P_T & = & (C^{OTM}_T + P^{OTM}_T - C^{OTM}_0 - P^{OTM}_0)\times m - fee\\ & = & (8.19+0-8.80-9.50)\times100-1.00\times2\\ & = & -1013 \end{array}$$

So, the strategy loses $1,013.

The following algorithm implements a long strangle Option strategy:

```py
class LongStrangleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.strangle(30, 5, -10))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find options with the nearest expiry
        expiry = max([x.expiry for x in chain])
        contracts = [contract for contract in chain if contract.expiry == expiry]

        # Order the OTM calls by strike to find the nearest to ATM
        call_contracts = sorted([contract for contract in contracts
            if contract.right == OptionRight.CALL and
               contract.strike > chain.underlying.price],
            key=lambda x: x.strike)
        if not call_contracts:
            return

        # Order the OTM puts by strike to find the nearest to ATM
        put_contracts = sorted([contract for contract in contracts
            if contract.right == OptionRight.PUT and
               contract.strike < chain.underlying.price],
            key=lambda x: x.strike, reverse=True)
        if not put_contracts:
            return

        call_strike = call_contracts[0].strike
        put_strike = put_contracts[0].strike

        long_strangle = OptionStrategies.strangle(self.symbol, call_strike, put_strike, expiry)
        self.buy(long_strangle, 1)
```

Option Strategies

Short Strangle

Introduction

Short Strangle is an Options trading strategy that consists of simultaneously selling an OTM put and an OTM call, where both contracts have the same underlying asset and expiration date. By doing so, the trader is essentially betting that the underlying asset will remain relatively stable and not experience significant price movements before the Options' expiration.

Compared to a short straddle , the net debit of a short strangle is lower since OTM Options are cheaper. Additionally, the winning range of a short straddle is wider and the strike spread is wider.

Implementation

Follow these steps to implement the short strangle strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG")
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.strangle(30, 5, -10))
```

   The `strangle` filter narrows the universe down to just the two contracts you need to form a short strangle.

2. In the `on_data` method, select the contracts of the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    chain = slice.option_chains.get(self._symbol)
    if not chain:
        return

    # Find options with the farthest expiry
    expiry = max([x.expiry for x in chain])
    contracts = [contract for contract in chain if contract.expiry == expiry]

    # Order the OTM calls by strike to find the nearest to ATM
    call_contracts = sorted([contract for contract in contracts
        if contract.right == OptionRight.CALL and
            contract.strike > chain.underlying.price],
        key=lambda x: x.strike)
    if not call_contracts:
        return

    # Order the OTM puts by strike to find the nearest to ATM
    put_contracts = sorted([contract for contract in contracts
        if contract.right == OptionRight.PUT and
           contract.strike < chain.underlying.price],
        key=lambda x: x.strike, reverse=True)
    if not put_contracts:
        return

    call = call_contracts[0]
    put = put_contracts[0]
```

3. In the `on_data` method, place the orders.

   Approach A: Call the `OptionStrategies.short_strangle` method with the details of each leg and then pass the result to the `buy` method.

```
short_strangle = OptionStrategies.short_strangle(self._symbol, call.strike, put.strike, expiry)
self.buy(short_strangle, 1)
```

Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
legs = [
    Leg.create(call.symbol, -1),
    Leg.create(put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The payoff of the strategy is

$$\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{C})^{+}\\ P^{OTM}_T & = & (K^{P} - S_T)^{+}\\ P_T & = & (-C^{OTM}_T - P^{OTM}_T + C^{OTM}_0 + P^{OTM}_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{OTM}_T & = & \textrm{OTM call value at time T}\\ P^{OTM}_T & = & \textrm{OTM put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{C} & = & \textrm{OTM call strike price}\\ K^{P} & = & \textrm{OTM put strike price}\\ P_T & = & \textrm{Payout total at time T}\\ C^{OTM}_0 & = & \textrm{OTM call value at position opening (debit paid)}\\ P^{OTM}_0 & = & \textrm{OTM put value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded a call at a strike of $835.00 for $8.00 and a put at a strike of $832.50 for $7.40 .

The following chart shows the payoff at expiration:

The maximum profit $C^{OTM}_0 + P^{OTM}_0$. It occurs when the underlying price at expiration remains within the range of the strike prices. In this case, both Options expire worthless.

The maximum loss is unlimited if the underlying price rises to infinity or substantial, $C^{OTM}_0 + P^{OTM}_0 - K^{P}$, if it drops to zero at expiration.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm at Option expiration (2017-04-22):

Asset	Price ($)	Strike ($)
Call	8.00	835.00
Put	7.40	832.50
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

\begin{array}{rcll} C^{OTM}_T & = & (S_T - K^{C})^{+}\\ & = & (843.19-835.00)^{+}\\ & = & 8.19\\ P^{OTM}_T & = & (K^{P} - S_T)^{+}\\ & = & (832.50-843.19)^{+}\\ & = & 0\\ P_T & = & (-C^{OTM}_T - P^{OTM}_T + C^{OTM}_0 + P^{OTM}_0)\times m - fee\\ & = & (-8.19-0+8.00+7.40)\times100-1.00\times2\\ & = & 719 \end{array}

So, the strategy gains $719.

The following algorithm implements a short strangle Option strategy:

```py
class ShortStrangleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG")
        self.symbol = option.symbol
        option.set_filter(lambda universe: universe.strangle(30, 5, -10))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self.symbol)
        if not chain:
            return

        # Find options with the nearest expiry
        expiry = max([x.expiry for x in chain])
        contracts = [contract for contract in chain if contract.expiry == expiry]

        # Order the OTM calls by strike to find the nearest to ATM
        call_contracts = sorted([contract for contract in contracts
            if contract.right == OptionRight.CALL and
               contract.strike > chain.underlying.price],
            key=lambda x: x.strike)
        if not call_contracts:
            return

        # Order the OTM puts by strike to find the nearest to ATM
        put_contracts = sorted([contract for contract in contracts
            if contract.right == OptionRight.PUT and
               contract.strike < chain.underlying.price],
            key=lambda x: x.strike, reverse=True)
        if not put_contracts:
            return

        call_strike = call_contracts[0].strike
        put_strike = put_contracts[0].strike

        short_strangle = OptionStrategies.short_strangle(self.symbol, call_strike, put_strike, expiry)
        self.buy(short_strangle, 1)
```

Option Strategies

Conversion

Introduction

A Conversion is a special case of a protective collar . It consist of holding one lot of the underlying security, shorting a call, and longing a put with lower strike price. However, the strategy now serves as an delta-neutral arbitration from Option mispricing instead of a hedge strategy. Note that it only attains a true profit when the risk-free return is greater than the risk-free interest rate .

Implementation

Follow these steps to implement the conversion strategy:

1. In the `initialize` method, set the start date, set the end date, subscribe to the underlying Equity , and create an Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.conversion(30, -5))
```

   The `conversion` filter narrows the universe down to just the two contracts you need to form a conversion.

2. In the `on_data` method, select the strike and expiry of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ATM strike price
    expiry = max([x.expiry for x in chain])
    strike = sorted(chain, key = lambda x: abs(x.strike - chain.underlying.price))[0].strike
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.conversion` method with the details of each leg and then pass the result to the `buy` method.

```
conversion = OptionStrategies.conversion(self._symbol, strike, expiry)
self.buy(conversion, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
# Select the call and put contracts
call = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry and x.strike == strike][0]
put = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry and x.strike == strike][0]

legs = [
    Leg.create(call.symbol, -1),
    Leg.create(put.symbol, 1),
    Leg.create(chain.underlying.symbol, chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

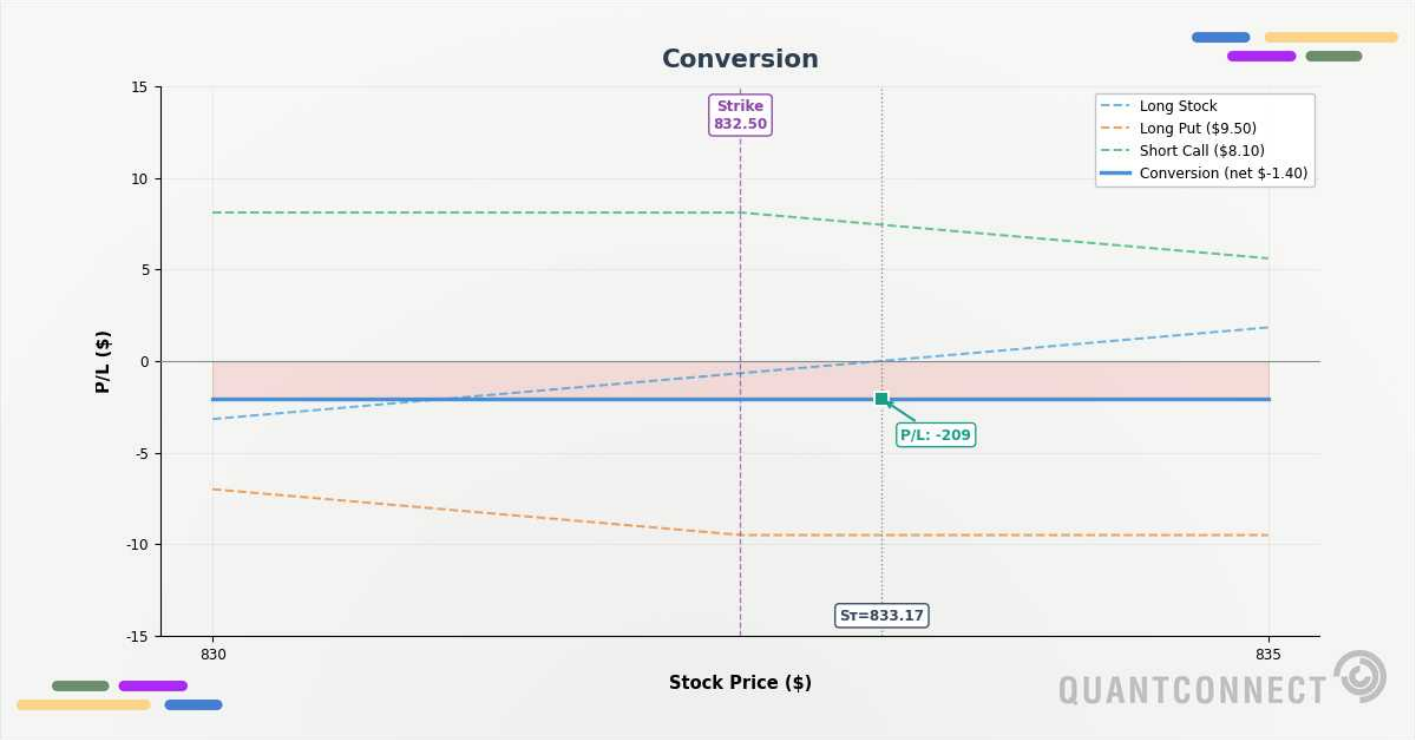
Strategy Payoff

This is a fixed payoff, delta-neutral strategy. The payoff is

$$\begin{array}{rcll} C_T & = & (S_T - K)^{+}\\ P_T & = & (K - S_T)^{+}\\ Payoff_T & = & (S_T - S_0 - C_T + P_T + C_0 - P_0)\times m - fee\\ & = & (K - S_0 + C_0 - P_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C_T & = & \textrm{Call value at time T}\\ & P_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Strike price}\\ & Payoff_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & C_0 & = & \textrm{Call price when the trade opened (credit received)}\\ & P_0 & = & \textrm{Put price when the trade opened (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $833.17 at expiration. You traded a call and a put at a strike of $832.50 for $8.10 and $9.50 respectively, and held the underlying.

The following chart shows the payoff at expiration:



The payoff is only dependent on the strike price and the initial asset prices.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	8.10	832.50
Put	9.50	832.50
Underlying Equity at position opens	833.17	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_T & = & (K - S_0 + C_0 - P_0)\times m - fee\\ & = & (832.50 - 833.17 + 8.10 - 9.50)\times100-1.00\times3\\ & = & -210.00\\ \end{array}$$

So, the strategy loses $210.

The following algorithm implements a conversion Option strategy:

PY

```
class ConversionOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        equity = self.add_equity("GOOG", Resolution.MINUTE)
        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda universe: universe.conversion(30, -5))

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # choose the furthest expiration date within 30 days from now on
        expiry = sorted(chain, key = lambda x: x.expiry)[-1].expiry

        # select ATM strike price
        strike = sorted(chain, key = lambda x: abs(x.Strike - chain.underlying.price))[0].strike

        # Order Strategy
        conversion = OptionStrategies.conversion(self.symbol, strike, expiry)
        self.buy(conversion, 1)

    def on_end_of_day(self, symbol: Symbol) -> None:
        if symbol.value == "GOOG":
            self.log(f"{self.time}::{symbol}::{self.securities[symbol].price}")
```

Option Strategies

Reverse Conversion

Introduction

A Reverse Conversion , or Reversal , is the inverse of a conversion and also a special case of a protective collar . It consist of shorting one lot of the underlying security, buying a call, and selling a put with lower strike price. However, the strategy now serves as an delta-neutral arbitration from Option mispricing instead of a hedge strategy. Note that it only attains a true profit when the risk-free return is greater than the risk-free interest rate .

Implementation

Follow these steps to implement the reverse conversion strategy:

1. In the `initialize` method, set the start date, set the end date, subscribe to the underlying Equity , and create an Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.conversion(30, -5))
```

   The `conversion` filter narrows the universe down to just the two contracts you need to form a reverse conversion.

2. In the `on_data` method, select the strike and expiry of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ATM strike price
    expiry = max([x.expiry for x in chain])
    strike = sorted(chain, key = lambda x: abs(x.strike - chain.underlying.price))[0].strike
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.reverse_conversion` method with the details of each leg and then pass the result to the `buy` method.

```
reverse_conversion = OptionStrategies.reverse_conversion(self._symbol, strike, expiry)
self.buy(reverse_conversion, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
# Select the call and put contracts
call = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry and x.strike == strike][0]
put = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry and x.strike == strike][0]

legs = [
    Leg.create(call.symbol, 1),
    Leg.create(put.symbol, -1),
    Leg.create(chain.underlying.symbol, -chain.underlying.symbol_properties.contract_multiplier)
]
self.combo_market_order(legs, 1)
```

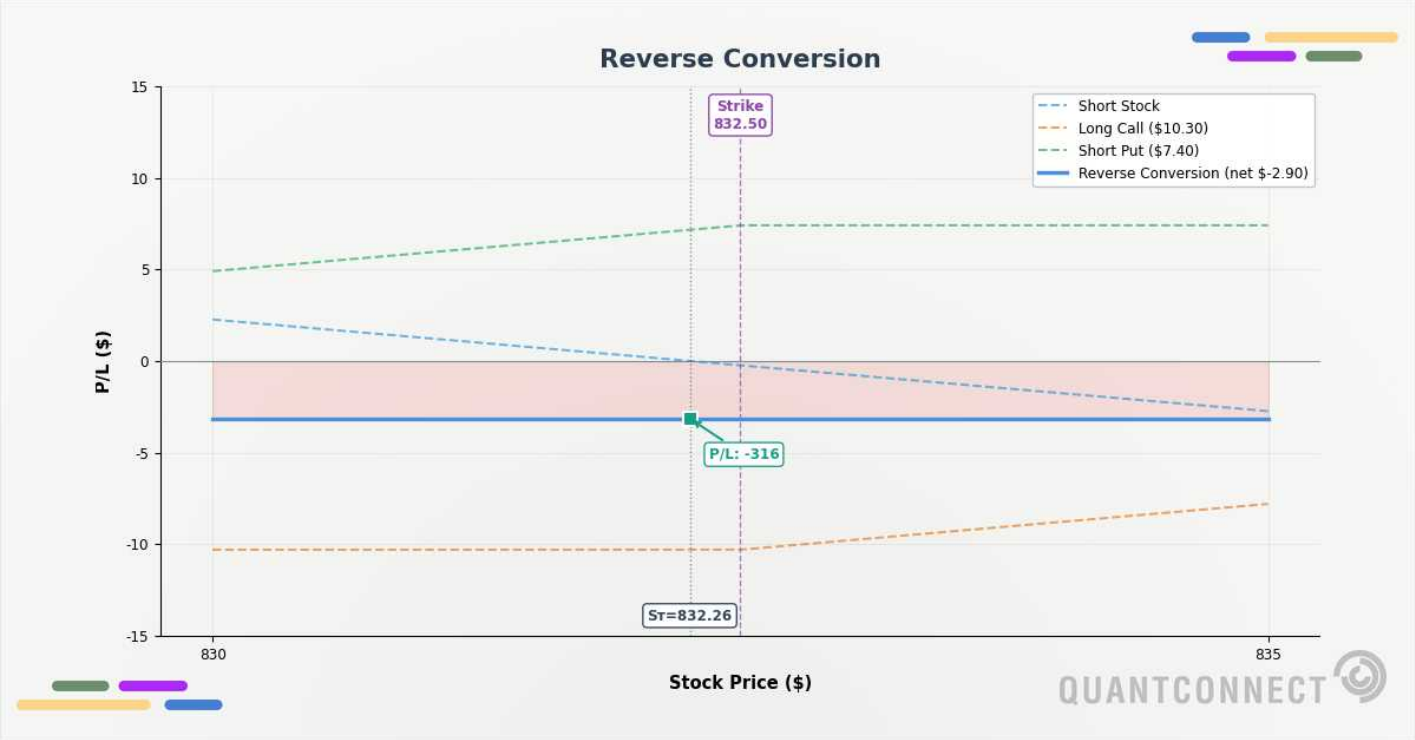
Strategy Payoff

This is a fixed payoff, delta-neutral strategy. The payoff is

$$\begin{array}{rcll} C_T & = & (S_T - K)^{+}\\ P_T & = & (K - S_T)^{+}\\ Payoff_T & = & (S_0 - S_T + C_T - P_T - C_0 + P_0)\times m - fee\\ & = & (S_0 - K - C_0 + P_0)\times m - fee \end{array} \begin{array}{rcl} \textrm{where} & C_T & = & \textrm{Call value at time T}\\ & P_T & = & \textrm{Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K & = & \textrm{Strike price}\\ & Payoff_T & = & \textrm{Payout total at time T}\\ & S_0 & = & \textrm{Underlying asset price when the trade opened}\\ & C_0 & = & \textrm{Call price when the trade opened (credit received)}\\ & P_0 & = & \textrm{Put price when the trade opened (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock traded at $832.26 at expiration. You traded a call and a put at a strike of $832.50 for $10.30 and $7.40 respectively, and held the underlying.

The following chart shows the payoff at expiration:



The payoff is only dependent on the strike price and the initial asset prices.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Call	10.30	832.50
Put	7.40	832.50
Underlying Equity at position opens	832.26	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_T & = & (S_0 - K - C_0 + P_0)\times m - fee\\ & = & (832.26 - 832.50 - 10.30 + 7.40)\times100-1.00\times3\\ & = & -317.00\\ \end{array}$$

So, the strategy loses $317.

The following algorithm implements a reverse conversion Option strategy:

PY

```
class ReverseConversionOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        equity = self.add_equity("GOOG", Resolution.MINUTE)
        option = self.add_option("GOOG", Resolution.MINUTE)
        self.symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda universe: universe.conversion(30, -5))

    def on_data(self, data: Slice) -> None:
        # avoid extra orders
        if self.portfolio.invested: return

        # Get the OptionChain of the self.symbol
        chain = data.option_chains.get(self.symbol, None)
        if not chain: return

        # choose the furthest expiration date within 30 days from now on
        expiry = sorted(chain, key = lambda x: x.expiry)[-1].expiry

        # select ATM strike price
        strike = sorted(chain, key = lambda x: abs(x.Strike - chain.underlying.price))[0].strike

        # Order Strategy
        reverse_conversion = OptionStrategies.reverse_conversion(self.symbol, strike, expiry)
        self.buy(reverse_conversion, 1)

    def on_end_of_day(self, symbol: Symbol) -> None:
        if symbol.value == "GOOG":
            self.log(f"{self.time}::{symbol}::{self.securities[symbol].price}")
```

Option Strategies

Long Box Spread

Introduction

A long Box Spread is the combination of a bull call spread and a bear put spread . It consist of buying an ITM call at strike A, selling an OTM put at strike A, buying an ITM put at strike B < A, and selling an OTM call at strike B, where all of the contracts have the same expiry date. This strategy serves as an delta-neutral arbitration from Option mispricing. Note that it only attains a true profit when the risk-free return is greater than the risk-free interest rate .

Implementation

Follow these steps to implement the long box spread strategy:

1. In the `initialize` method, set the start date, set the end date, subscribe to the underlying Equity , and create an Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.box_spread(30, 5))
```

   The `box_spread` filter narrows the universe down to just the four contracts you need to form a box spread.

2. In the `on_data` method, select the expiry and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ITM & OTM strike prices
    expiry = max([x.expiry for x in chain])
    contracts = [x for x in chain if x.expiry == expiry]
    lower_strike = min([x.strike for x in contracts])
    higher_strike = max([x.strike for x in contracts])
```

3. In the `on_data` method, select the contracts and place the order.

   Approach A: Call the `OptionStrategies.box_spread` method with the details of each leg and then pass the result to the `buy` method.

```py
box_spread = OptionStrategies.box_spread(self._symbol, higher_strike, lower_strike, expiry)
self.buy(box_spread, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
# Select the call and put contracts
itm_call = next(filter(lambda x: x.right == OptionRight.CALL and x.strike == lower_strike, contracts))
otm_call = next(filter(lambda x: x.right == OptionRight.CALL and x.strike == higher_strike, contracts))
itm_put = next(filter(lambda x: x.right == OptionRight.PUT and x.strike == higher_strike, contracts))
otm_put = next(filter(lambda x: x.right == OptionRight.PUT and x.strike == lower_strike, contracts))

legs = [
    Leg.create(itm_call.symbol, 1),
    Leg.create(itm_put.symbol, 1),
    Leg.create(otm_call.symbol, -1),
    Leg.create(otm_put.symbol, -1),
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

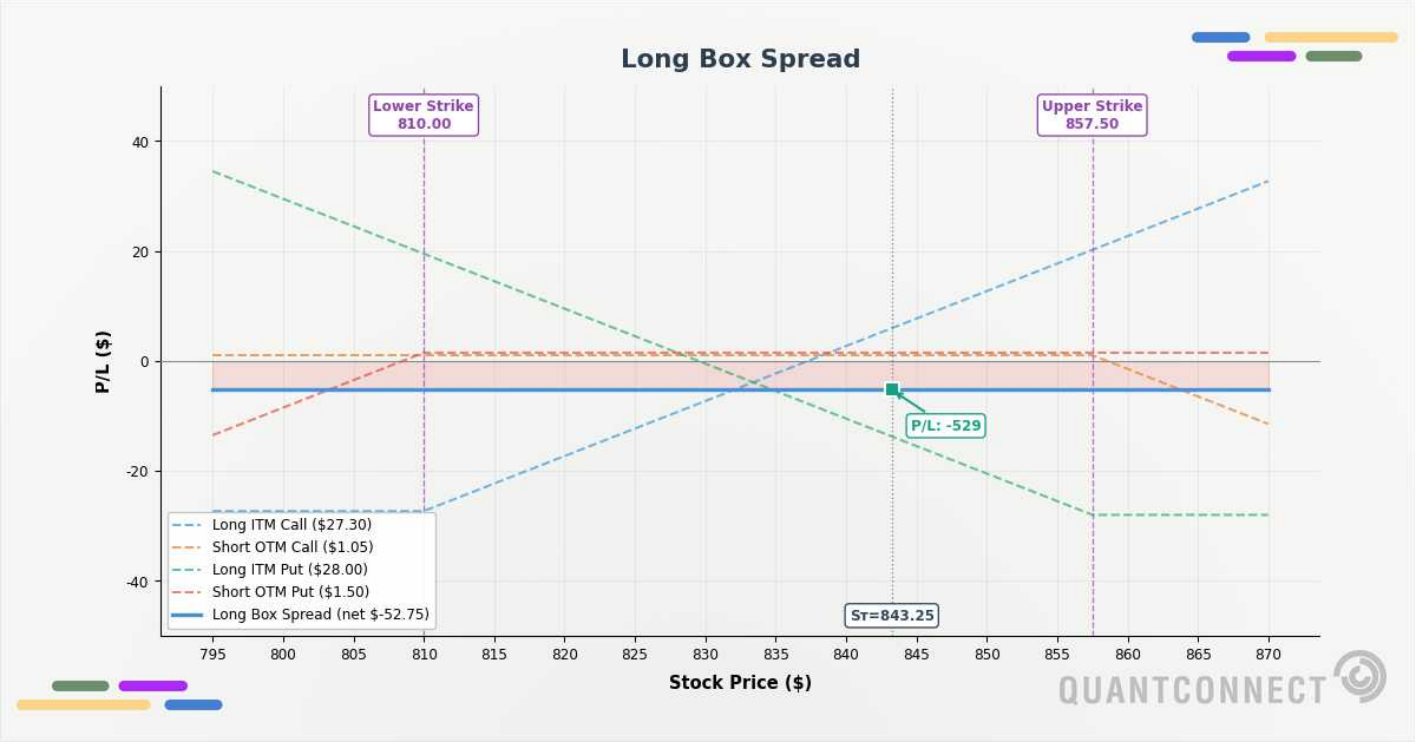
This is a fixed payoff, delta-neutral strategy. The payoff is

$$\begin{array}{rcll} C_T^{ITM} & = & (S_T - K_{-})^{+}\\ C_T^{OTM} & = & (S_T - K_{+})^{+}\\ P_T^{ITM} & = & (K_{+} - S_T)^{+}\\ P_T^{OTM} & = & (K_{-} - S_T)^{+}\\ Payoff_T & = & (C_T^{ITM} - C_{T_0}^{ITM} + P_T^{ITM} - P_{T_0}^{ITM} - C_T^{OTM} + C_{T_0}^{OTM} - P_T^{OTM} + P_{T_0}^{OTM})\times m - fee\\ & = & (K_{+} - K_{-} + C_{T_0}^{OTM} + P_{T_0}^{OTM} - C_{T_0}^{ITM} - P_{T_0}^{ITM})\times m - fee \end{array}$$

$$\begin{array}{rcll} \textrm{where} & & & \\ C_T^{ITM} & = & \textrm{ITM Call value at time T}\\ C_T^{OTM} & = & \textrm{OTM Call value at time T}\\ P_T^{ITM} & = & \textrm{ITM Put value at time T}\\ P_T^{OTM} & = & \textrm{OTM Put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K_{+} & = & \textrm{Higher strike price}\\ K_{-} & = & \textrm{Lower strike price}\\ Payoff_{T_1} & = & \textrm{Payout total at time T1}\\ C_{T_0}^{ITM} & = & \textrm{ITM Call price when the trade opened (debit paid)}\\ C_{T_0}^{OTM} & = & \textrm{OTM Call price when the trade opened (credit received)}\\ P_{T_0}^{ITM} & = & \textrm{ITM Put price when the trade opened (debit paid)}\\ P_{T_0}^{OTM} & = & \textrm{OTM Put price when the trade opened (credit received)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded an ITM call at $810.00 for $27.30 , an OTM call at $857.50 for $1.05 , an ITM put at $857.50 for $28.00 , and an OTM put at $810.00 for $1.50 .

The following chart shows the payoff at expiration:



The payoff is only dependent on the strike price and the initial asset prices.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
ITM Call	27.30	810.00
ITM Put	28.00	857.50
OTM Call	1.05	857.50
OTM Put	1.50	810.00
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_T & = & (K_{+} - K_{-} + C_0^{OTM} + P_0^{OTM} - C_0^{ITM} - P_0^{ITM})\times m - fee\\ & = & (857.50 - 810.00 + 1.05 + 1.50 - 27.30 - 28.00)\times100 - 1.00\times4\\ & = & -529.00\\ \end{array}$$

So, the strategy loses $529.

The following algorithm implements a long box spread Option strategy:

```py
class BoxSpreadStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.universe_settings.asynchronous = True

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.Symbol
        option.set_filter(lambda universe: universe.box_spread(30, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select an expiry date
        expiry = max([x.expiry for x in chain])

        # Select the strike prices of the contracts
        contracts = [x for x in chain if x.expiry == expiry]
        higher_strike = max([x.strike for x in contracts])
        lower_strike = min([x.strike for x in contracts])

        box_spread = OptionStrategies.box_spread(self._symbol, higher_strike, lower_strike, expiry)
        self.buy(box_spread, 1)

    def on_end_of_day(self, symbol: Symbol) -> None:
        if symbol == self._symbol.underlying:
            self.Log(f"{self.time}::{symbol}::{self.securities[symbol].price}")
```

Option Strategies

Short Box Spread

Introduction

A Short Box Spread is the inverse of a box spread , as well as the combination of a bear call spread and a bull put spread . It consists of buying an OTM call at strike A, selling an ITM put at strike A, buying an OTM put and strike B < A, and selling an ITM call at strike B, where all of the contracts have the same expiry date. This strategy serves as an delta-neutral arbitration from Option mispricing. Note that it only attains a true profit when the risk-free return is greater than the risk-free interest rate .

Implementation

Follow these steps to implement the short box spread strategy:

1. In the `initialize` method, set the start date, set the end date, subscribe to the underlying Equity , and create an Option universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.box_spread(30, 5))
```

   The `box_spread` filter narrows the universe down to just the four contracts you need to form a short box spread.

2. In the `on_data` method, select the strike and expiry of the contracts in the strategy legs.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ITM & OTM strike prices
    expiry = max([x.expiry for x in chain])
    contracts = [x for x in chain if x.expiry == expiry]
    lower_strike = min([x.strike for x in contracts])
    higher_strike = max([x.strike for x in contracts])
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_box_spread` method with the details of each leg and then pass the result to the `buy` method.

```
PY
short_box_spread = OptionStrategies.short_box_spread(self._symbol, higher_strike, lower_strike, expiry)
self.buy(short_box_spread, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
# Select the call and put contracts
itm_call = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry and x.strike == lower_strike][0]
otm_call = [x for x in chain if x.right == OptionRight.CALL and x.expiry == expiry and x.strike == higher_strike][0]
itm_put = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry and x.strike == higher_strike][0]
otm_put = [x for x in chain if x.right == OptionRight.PUT and x.expiry == expiry and x.strike == lower_strike][0]

legs = [
    Leg.create(itm_call.symbol, -1),
    Leg.create(itm_put.symbol, -1),
    Leg.create(otm_call.symbol, 1),
    Leg.create(otm_put.symbol, 1),
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

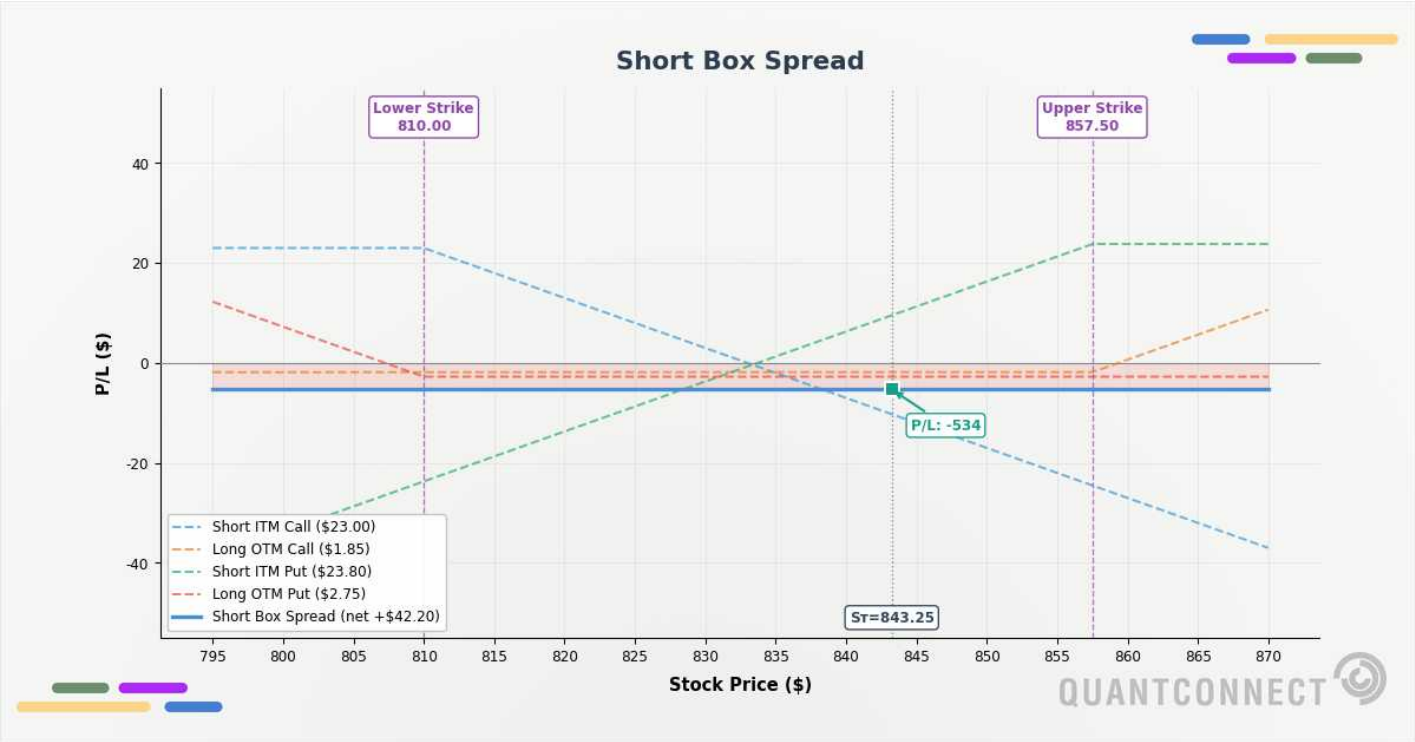
This is a fixed payoff, delta-neutral strategy. The payoff is

$$\begin{array}{rcl} C_T^{ITM} & = & (S_T - K_{-})^{+}\\ C_T^{OTM} & = & (S_T - K_{+})^{+}\\ P_T^{ITM} & = & (K_{+} - S_T)^{+}\\ P_T^{OTM} & = & (K_{-} - S_T)^{+}\\ Payoff_T & = & (C_{T_0}^{ITM} - C_T^{ITM} + P_{T_0}^{ITM} - P_T^{ITM} - C_{T_0}^{OTM} + C_T^{OTM} - P_{T_0}^{OTM} + P_T^{OTM})\times m - fee\\ & = & (K_{-} - K_{+} + C_{T_0}^{ITM} + P_{T_0}^{ITM} - C_{T_0}^{OTM} - P_{T_0}^{OTM})\times m - fee \end{array}$$

$$\begin{array}{rcl} \textrm{where} & C_T^{ITM} & = & \textrm{ITM Call value at time T}\\ & C_T^{OTM} & = & \textrm{OTM Call value at time T}\\ & P_T^{ITM} & = & \textrm{ITM Put value at time T}\\ & P_T^{OTM} & = & \textrm{OTM Put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K_{+} & = & \textrm{Higher strike price}\\ & K_{-} & = & \textrm{Lower strike price}\\ & Payoff_T & = & \textrm{Payout total at time T}\\ & C_{T_0}^{ITM} & = & \textrm{ITM Call price when the trade opened (debit paid)}\\ & C_{T_0}^{OTM} & = & \textrm{OTM Call price when the trade opened (credit received)}\\ & P_{T_0}^{ITM} & = & \textrm{ITM Put price when the trade opened (debit paid)}\\ & P_{T_0}^{OTM} & = & \textrm{OTM Put price when the trade opened (credit received)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, you traded an ITM call at $810.00 for $23.00 , an OTM call at $857.50 for $1.85 , an ITM put at $857.50 for $23.80 , and an OTM put at $810.00 for $2.75 .

The following chart shows the payoff at expiration:



The payoff is only dependent on the strike price and the initial asset prices.

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
ITM Call	23.00	810.00
ITM Put	23.80	857.50
OTM Call	1.85	857.50
OTM Put	2.75	810.00
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_T & = & (K_{-} - K_{+} + C_0^{ITM} + P_0^{ITM} - C_0^{OTM} - P_0^{OTM})\times m - fee\\ & = & (810.00 - 857.50 + 23.00 + 23.80 - 1.85 - 2.75)\times100 - 1.00\times4\\ & = & -534.00\\ \end{array}$$

So, the strategy loses $534.

The following algorithm implements a short box spread Option strategy:

```py
class ProtectiveCollarOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1);
        self.set_end_date(2024, 12, 31);
        self.set_cash(100000)

        self.universe_settings.asynchronous = True
        self.equity_symbol = self.add_equity("GOOG").symbol

        option = self.add_option("GOOG", Resolution.Minute)
        self.option_symbol = option.symbol
        option.set_filter(lambda universe: universe.box_spread(30, 5))

        self.invest_ever = False

    def on_data(self, slice: Slice) -> None:
        if self.invest_ever: return

        # Get the OptionChain
        chain = slice.option_chains.get(self.option_symbol, None)
        if not chain: return

        # Select an expiry date
        expiry = sorted(chain, key = lambda x: x.expiry)[-1].expiry

        # Select the strike prices of the contracts
        ordered_contracts = sorted(chain, key = lambda x: x.strike)
        higher_strike = ordered_contracts[-1].strike
        lower_strike = ordered_contracts[0].strike

        box_spread = OptionStrategies.short_box_spread(self.option_symbol, higher_strike, lower_strike, expiry)
        self.buy(box_spread, 1)

        self.invest_ever = True

    def on_end_of_day(self, symbol: Symbol) -> None:
        if symbol == self.equity_symbol:
            self.log(f"{self.time}::{symbol}::{self.securities[symbol].price}")
```

Option Strategies

Long Jelly Roll

Introduction

A long Jelly Roll , or simply Roll , is a combination of a long call calendar spread and a short put calendar spread . It consists of buying a put and selling a call of the same expiry, as well as buying a call and selling a put with a further expiry, where all of the contracts have the same strike price. This strategy serves as an arbitrage on Option mispricing due to the temporary disparity between the call spread and the put spread synthetic portfolios. It is a delta -, gamma -, vega -, and theta -neutral strategy, but sensitive to rho (interest rate) and phi (dividend yield).

Implementation

Follow these steps to implement the long jelly roll strategy:

1. In the `initialize` method, set the start date, set the end date, and create an Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda x: x.jelly_roll(5.0, 30, 60))
```

2. In the `on_data` method, select the expiry and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ITM & OTM strike prices
    strike = sorted([x.strike for x in chain], key=lambda x: abs(x - chain.underlying.price))[0]
    contracts = [x for x in chain if x.strike == strike]
    far_expiry = max([x.expiry for x in contracts])
    near_expiry = min([x.expiry for x in contracts])
```

3. In the `on_data` method, select the contracts and place the order.

   Approach A: Call the `OptionStrategies.jelly_roll` method with the details of each leg and then pass the result to the `buy` method.

```py
jelly_roll = OptionStrategies.jelly_roll(self._symbol, strike, near_expiry, far_expiry)
self.buy(jelly_roll, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
# Select the call and put contracts
near_call = next(filter(lambda x: x.right == OptionRight.CALL and x.expiry == near_expiry, contracts))
far_call = next(filter(lambda x: x.right == OptionRight.CALL and x.expiry == far_expiry, contracts))
near_put = next(filter(lambda x: x.right == OptionRight.PUT and x.expiry == near_expiry, contracts))
call_put = next(filter(lambda x: x.right == OptionRight.PUT and x.expiry == far_expiry, contracts))

legs = [
    Leg.create(near_call.symbol, -1),
    Leg.create(far_call.symbol, 1),
    Leg.create(near_put.symbol, 1),
    Leg.create(call_put.symbol, -1),
]
self.combo_market_order(legs, 1)
```

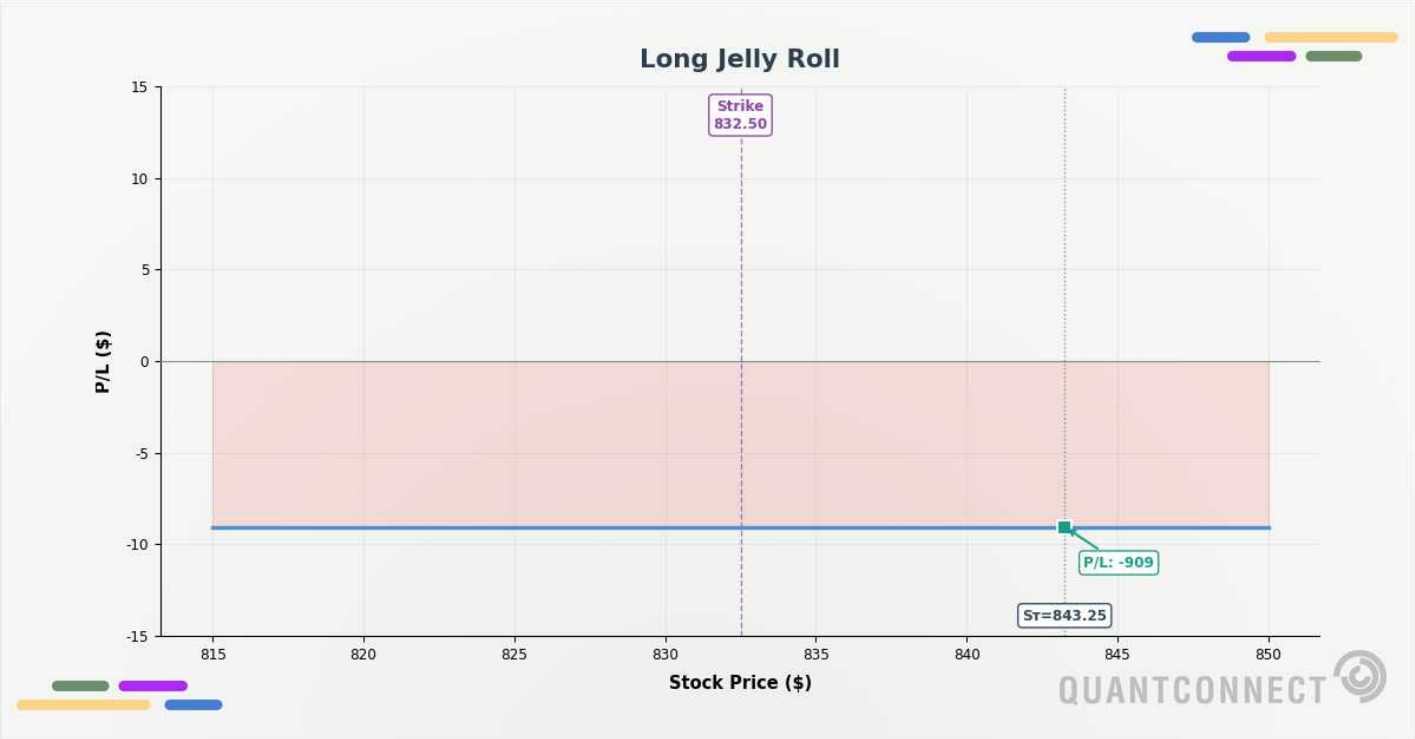
Strategy Payoff

This is a delta-, gamma-, vega-, and theta-neutral strategy. The payoff is

$$\begin{array}{rcll} C_{T_1}^{T_1} & = & (S_{T_1} - K)^{+}\\ P_{T_1}^{T_1} & = & (K - S_{T_1})^{+}\\ Payoff_{T_1} & = & (P_{T_1}^{T_1} - P_{T_0}^{T_1} - C_{T_1}^{T_1} + C_{T_0}^{T_1} + C_{T_1}^{T_2} - C_{T_0}^{T_2} - P_{T_1}^{T_2} + P_{T_0}^{T_2})\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & C_{T_1}^{T_1} & = & \textrm{Market value of Call with expiry at T1 at time T1}\\ C_{T_1}^{T_2} & = & \textrm{Market value of Call with expiry at T2 at time T1}\\ P_{T_1}^{T_1} & = & \textrm{Market value of Put with expiry at T1 at time T1}\\ P_{T_1}^{T_2} & = & \textrm{Market value of Put with expiry at T2 at time T1}\\ S_{T_1} & = & \textrm{Underlying asset price at time T1}\\ K & = & \textrm{Strike price}\\ Payoff_T & = & \textrm{Payout total at time T}\\ C_{T_0}^{T_1} & = & \textrm{Market value of Call with expiry at T1 when the trade opened}\\ C_{T_0}^{T_2} & = & \textrm{Market value of Call with expiry at T2 when the trade opened}\\ P_{T_0}^{T_1} & = & \textrm{Market value of Put with expiry at T2 when the trade opened}\\ P_{T_0}^{T_2} & = & \textrm{Market value of Put with expiry at T2 when the trade opened}\\ m & = & \textrm{Contract multiplier}\\ T_1 & = & \textrm{Time T1 as the near expiration}\\ T_2 & = & \textrm{Time T2 as the far expiration} \end{array}$$

In this example, you traded near-expiry and far-expiry calls and puts at a strike of $832.50 .

The following chart shows the payoff at expiration:



The payoff is dependent on the market prices of the Options, but in theory, if assuming call-put parity exists, the expected payoff would be

$$\begin{array}{rcll} Payoff_{T_1} & = & Payoff_{\textrm{call calendar spread}} - Payoff_{\textrm{put calendar spread}}\\ & = & K \times (T_2 - T_1) \times r - D \end{array} \begin{array}{rcl} \textrm{where} & r & = & \textrm{Continuous compounding interest rate}\\ D & = & \textrm{Dividend payment during the life of the option} \end{array}$$

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price at position open ($)	Price at the first expiry ($)	Strike ($)
Near-expiry Call	18.50	23.75	832.50
Near-expiry Put	22.50	12.85	832.50
Far-expiry Call	23.90	24.45	832.50
Far-expiry Put	19.00	13.70	832.50
Underlying Equity	-	843.2500	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_{T_1} & = & (P_{T_1}^{T_1} - P_0^{T_1} - C_{T_1}^{T_1} + C_0^{T_1} + C_{T_1}^{T_2} - C_0^{T_2} - P_{T_1}^{T_2} + P_0^{T_2})\times m - fee\\ & = & (12.85 - 22.50 - 23.75 + 18.50 + 24.45 - 23.90 - 13.70 + 19.00)\times100 - 1.00\times4\\ & = & -909.00\\ \end{array}$$

So, the strategy loses $909.

The following algorithm implements a long jelly roll Option strategy:

```py
class JellyRollOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.jelly_roll(5.0, 30, 60))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select an expiry date and ITM & OTM strike prices
        strike = sorted([x.strike for x in chain], key=lambda x: abs(x - chain.underlying.price))[0]
        contracts = [x for x in chain if x.strike == strike]
        far_expiry = max([x.expiry for x in contracts])
        near_expiry = min([x.expiry for x in contracts])

        jelly_roll = OptionStrategies.jelly_roll(self._symbol, strike, near_expiry, far_expiry)
        self.buy(jelly_roll, 1)
```

Option Strategies

Short Jelly Roll

Introduction

A Short Jelly Roll , or simply Short Roll , is a combination of a short call calendar spread and a long put calendar spread . It is the inverse of a jelly roll . It consist of selling a put and buying a call of the same expiry, as well as buying a put and selling a call with a further expiry, where all of the contracts have the same strike prices. This strategy serves as an arbitrage on Option mispricing due to the temporary disparity between the call spread and the put spread synthetic portfolios. It is a delta -, gamma -, vega -, and theta -neutral strategy, but sensitive to rho (interest rate) and phi (dividend yield).

Implementation

Follow these steps to implement the short jelly roll strategy:

1. In the `initialize` method, set the start date, set the end date, and create an Option universe .

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda x: x.jelly_roll(5.0, 30, 60))
```

2. In the `on_data` method, select the expiry and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select an expiry date and ITM & OTM strike prices
    strike = sorted([x.strike for x in chain], key=lambda x: abs(x - chain.underlying.price))[0]
    contracts = [x for x in chain if x.strike == strike]
    far_expiry = max([x.expiry for x in contracts])
    nearer_expiries = [x.expiry for x in contracts if x.expiry < far_expiry]
    if not nearer_expiries:
        return
    near_expiry = min(nearer_expiries)
```

3. In the `on_data` method, select the contracts and place the order.

   Approach A: Call the `OptionStrategies.jelly_roshort_jelly_rollll` method with the details of each leg and then pass the result to the `buy` method.

```py
short_jelly_roll = OptionStrategies.short_jelly_roll(self._symbol, strike, near_expiry, far_expiry)
self.buy(short_jelly_roll, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
# Select the call and put contracts
near_call = next(filter(lambda x: x.right == OptionRight.CALL and x.expiry == near_expiry, contracts))
far_call = next(filter(lambda x: x.right == OptionRight.CALL and x.expiry == far_expiry, contracts))
near_put = next(filter(lambda x: x.right == OptionRight.PUT and x.expiry == near_expiry, contracts))
call_put = next(filter(lambda x: x.right == OptionRight.PUT and x.expiry == far_expiry, contracts))

legs = [
    Leg.create(near_call.symbol, 1),
    Leg.create(far_call.symbol, -1),
    Leg.create(near_put.symbol, -1),
    Leg.create(call_put.symbol, 1),
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

This is a delta-, gamma-, vega-, and theta-neutral strategy. The payoff is

$$\begin{array}{rcll} C_{T_1}^{T_1} & = & (S_{T_1} - K)^{+}\\ P_{T_1}^{T_1} & = & (K - S_{T_1})^{+}\\ Payoff_{T_1} & = & (C_{T_1}^{T_1} - C_{T_0}^{T_1} - P_{T_1}^{T_1} + P_{T_0}^{T_1} + P_{T_1}^{T_2} - P_{T_0}^{T_2} - C_{T_1}^{T_2} + C_{T_0}^{T_2})\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & C_{T_1}^{T_1} & = & \textrm{Market value of Call with expiry at T1 at time T1}\\ C_{T_1}^{T_2} & = & \textrm{Market value of Call with expiry at T2 at time T1}\\ P_{T_1}^{T_1} & = & \textrm{Market value of Put with expiry at T1 at time T1}\\ P_{T_1}^{T_2} & = & \textrm{Market value of Put with expiry at T2 at time T1}\\ S_{T_1} & = & \textrm{Underlying asset price at time T1}\\ K & = & \textrm{Strike price}\\ Payoff_{T_1} & = & \textrm{Payout total at time T1}\\ C_{T_0}^{T_1} & = & \textrm{Market value of Call with expiry at T1 when the trade opened}\\ C_{T_0}^{T_2} & = & \textrm{Market value of Call with expiry at T2 when the trade opened}\\ P_{T_0}^{T_1} & = & \textrm{Market value of Put with expiry at T2 when the trade opened}\\ P_{T_0}^{T_2} & = & \textrm{Market value of Put with expiry at T2 when the trade opened}\\ m & = & \textrm{Contract multiplier}\\ T_1 & = & \textrm{Time T1 as the near expiration}\\ T_2 & = & \textrm{Time T2 as the far expiration} \end{array}$$

In this example, you traded near-expiry and far-expiry calls and puts at a strike of $832.50 .

The following chart shows the payoff at expiration:

The payoff is dependent on the market prices of the options, but in theory, if assuming call-put parity exists, the expected payoff would be

$$\begin{array}{rcll} Payoff_{T_1} & = & Payoff_{\textrm{put calendar spread}} - Payoff_{\textrm{call calendar spread}}\\ & = & D - K \times (T_2 - T_1) \times r \end{array} \begin{array}{rcl} \textrm{where} & r & = & \textrm{Continuous compounding interest rate}\\ D & = & \textrm{Dividend payment during the life of the option} \end{array}$$

If the Option is American Option, there is a risk of early assignment on the contracts you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price at position open ($)	Price at the first expiry ($)	Strike ($)
Near-expiry Call	22.80	23.75	832.50
Near-expiry Put	18.10	12.85	832.50
Far-expiry Call	19.50	24.45	832.50
Far-expiry Put	23.50	13.70	832.50
Underlying Equity	-	843.2500	-

Therefore, the payoff is

$$\begin{array}{rcll} Payoff_{T_1} & = & (C_{T_1}^{T_1} - C_0^{T_1} - P_{T_1}^{T_1} + P_0^{T_1} + P_{T_1}^{T_2} - P_0^{T_2} - C_{T_1}^{T_2} + C_0^{T_2})\times m - fee\\ & = & (23.75 - 22.80 - 12.85 + 18.10 + 13.70 - 23.50 - 24.45 + 19.50)\times100 - 1.00\times4\\ & = & -859.00\\ \end{array}$$

So, the strategy loses $859.

The following algorithm implements a short jelly roll Option strategy:

```py
class ShortJellyRollOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.jelly_roll(5.0, 30, 60))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select an expiry date and ITM & OTM strike prices
        strike = sorted([x.strike for x in chain], key=lambda x: abs(x - chain.underlying.price))[0]
        contracts = [x for x in chain if x.strike == strike]
        far_expiry = max([x.expiry for x in contracts])
        near_expiry = min([x.expiry for x in contracts])

        short_jelly_roll = OptionStrategies.short_jelly_roll(self._symbol, strike, near_expiry, far_expiry)
        self.buy(short_jelly_roll, 1)
```

Option Strategies

Bear Call Ladder

Introduction

Bear call ladder , also known as short call ladder , is a combination of a bear call spread and a long call with a higher strike price than the 2 legs of the call spread. All calls have the same underlying Equity and expiration date. This strategy profits from increasing volatility of the underlying asset. For instance, the underlying price moves away from its current price.

Implementation

Follow these steps to implement the bear call ladder strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_ladder(30, 5, 0, -5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in calls))
    if len(strikes) < 3:
        return

    low_strike = strikes[0]
    middle_strike = strikes[1]
    high_strike = strikes[2]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bear_call_ladder` method with the details of each leg and then pass the result to the `buy` method.

```
PY
option_strategy = OptionStrategies.bear_call_ladder(self._symbol, low_strike, middle_strike, high_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```py
low_strike_call = next(filter(lambda x: x.strike == low_strike, calls))
middle_strike_call = next(filter(lambda x: x.strike == middle_strike, calls))
high_strike_call = next(filter(lambda x: x.strike == high_strike, calls))

legs = [
    Leg.create(low_strike_call.symbol, -1),
    Leg.create(middle_strike_call.symbol, 1),
    Leg.create(high_strike_call.symbol, 1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The bear call ladding is an unlimited-profit-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ C^{mid}_T & = & (S_T - K^{mid})^{+}\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ Payoff_T & = & (C^{low}_0 - C^{low}_T + C^{mid}_T - C^{mid}_0 + C^{high}_T - C^{high}_0)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{low}_T & = & \textrm{Lower-strike call value at time T}\\ C^{mid}_T & = & \textrm{Middle-strike call value at time T}\\ C^{high}_T & = & \textrm{Higher-strike call value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{low} & = & \textrm{Lower-strike call strike price}\\ K^{mid} & = & \textrm{Middle-strike call strike price}\\ K^{high} & = & \textrm{Higher-strike call strike price}\\ C^{low}_0 & = & \textrm{Lower-strike call value at position opening (credit received)}\\ C^{mid}_0 & = & \textrm{Middle-strikeTM call value at position opening (debit paid)}\\ C^{high}_0 & = & \textrm{Higher-strike call value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a lower-strike call at $822.50 for $13.80 , a middle-strike call at $825.00 for $15.10 , and a higher-strike call at $827.50 for $13.10 .

The following chart shows the payoff at expiration:

The maximum profit is unlimited, which occurs when the underlying price increases indefinitely.

The maximum loss is $K^{low} - K^{mid} + C^{low}_0 - C^{mid}_0 - C^{high}_0$, which occurs when the underlying price is between the two higher strike prices.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike call	13.80	822.50
Middle-strike call	15.10	825.00
Higher-strike call	13.10	827.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ & = & (843.25-822.50)^{+}\\ & = & 20.75\\ C^{mid}_T & = & (S_T - K^{mid})^{+}\\ & = & (843.25-825.00)^{+}\\ & = & 18.25\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ & = & (843.25-827.50)^{+}\\ & = & 15.75\\ Payoff_T & = & (C^{low}_0 - C^{low}_T + C^{mid}_T - C^{mid}_0 + C^{high}_T - C^{high}_0)\times m - fee\\ & = & (13.80 - 20.75 + 18.25 - 15.10 + 15.75 - 13.10)\times100-1.00\times3\\ & = & -118\\ \end{array}

So, the strategy loses $118.

The following algorithm implements a bear call ladder Option strategy:

```py
class BearCallLadderOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.call_ladder(30, 5, 0, -5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if not calls:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in calls))
        if len(strikes) < 3:
            return

        low_strike = strikes[0]
        middle_strike = strikes[1]
        high_strike = strikes[2]

        option_strategy = OptionStrategies.bear_call_ladder(self._symbol, low_strike, middle_strike, high_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Bear Put Ladder

Introduction

Bear put ladder , also known as long put ladder , is a combination of a bear put spread and short put with a lower strike price than the 2 legs of the put spread. All puts have the same underlying Equity and expiration date. This strategy profits from low volatility of the underlying asset. For instance, the underlying price stays similar to its current price.

Implementation

Follow these steps to implement the bear put ladder strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_ladder(30, 5, 0, -5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if not puts:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in puts))
    if len(strikes) < 3:
        return

    low_strike = strikes[0]
    middle_strike = strikes[1]
    high_strike = strikes[2]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Put the `OptionStrategies.bear_put_ladder` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.bear_put_ladder(self._symbol, high_strike, middle_strike, low_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then put the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_put = next(filter(lambda x: x.strike == low_strike, puts))
middle_strike_put = next(filter(lambda x: x.strike == middle_strike, puts))
high_strike_put = next(filter(lambda x: x.strike == high_strike, puts))

legs = [
    Leg.create(low_strike_put.symbol, -1),
    Leg.create(middle_strike_put.symbol, -1),
    Leg.create(high_strike_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The bear put ladding is an limited-profit strategy. The payoff is

$$\begin{array}{rcll} P^{low}_T &=& (K^{low} - S_T)^{+}\\ P^{mid}_T &=& (K^{mid} - S_T)^{+}\\ P^{high}_T &=& (K^{high} - S_T)^{+}\\ Payoff_T &=& (P^{low}_0 - P^{low}_T + P^{mid}_0 - P^{mid}_T + P^{high}_T - P^{high}_0)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ P^{low}_T &=& \textrm{Lower-strike put value at time T}\\ P^{mid}_T &=& \textrm{Middle-strike put value at time T}\\ P^{high}_T &=& \textrm{Higher-strike put value at time T}\\ S_T &=& \textrm{Underlying asset price at time T}\\ K^{low} &=& \textrm{Lower-strike put strike price}\\ K^{mid} &=& \textrm{Middle-strike put strike price}\\ K^{high} &=& \textrm{Higher-strike put strike price}\\ P^{low}_0 &=& \textrm{Lower-strike put value at position opening (credit received)}\\ P^{mid}_0 &=& \textrm{Middle-strikeTM put value at position opening (debit paid)}\\ P^{high}_0 &=& \textrm{Higher-strike put value at position opening (debit paid)}\\ m &=& \textrm{Contract multiplier}\\ T &=& \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a lower-strike put at $822.50 for $3.80 , a middle-strike put at $825.00 for $4.70 , and a higher-strike put at $827.50 for $7.80 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{high} - K^{mid} + P^{low}_0 + P^{mid}_0 - P^{high}_0$, which occurs when the underlying price is between the two lower strike prices.

The maximum loss is $K^{high} - K^{mid} - K^{low} + P^{low}_0 + P^{mid}_0 - P^{high}_0$, which occurs when the underlying price decreases to $0 .

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike put	3.80	822.50
Middle-strike put	4.70	825.00
Higher-strike put	7.80	827.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ & = & (822.50-843.25)^{+}\\ & = & 0\\ P^{mid}_T & = & (K^{mid} - S_T)^{+}\\ & = & (825.00-843.25)^{+}\\ & = & 0\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ & = & (827.50-843.25)^{+}\\ & = & 0\\ Payoff_T & = & (P^{low}_0 - P^{low}_T + P^{mid}_0 - P^{mid}_T + P^{high}_T - P^{high}_0)\times m - fee\\ & = & (3.80 - 0 + 4.70 - 0 + 0 - 7.80)\times100-1.00\times3\\ & = & 67\\ \end{array}

So, the strategy gains $67.

The following algorithm implements a bear put ladder Option strategy:

```py
class BearPutLadderOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.put_ladder(30, 5, 0, -5))

    def on_data(self, slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if not puts:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in puts))
        if len(strikes) < 3:
            return

        low_strike = strikes[0]
        middle_strike = strikes[1]
        high_strike = strikes[2]

        option_strategy = OptionStrategies.bear_put_ladder(self._symbol, high_strike, middle_strike, low_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Bull Call Ladder

Introduction

Bull call ladder , also known as long call ladder , is a combination of a bull call spread and a short call with a higher strike price than the 2 legs of the call spread. All calls have the same underlying Equity and expiration date. This strategy profits from low volatility of the underlying asset. For instance, the underlying price stays similar to its current price.

Implementation

Follow these steps to implement the bull call ladder strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_ladder(30, 5, 0, -5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in calls))
    if len(strikes) < 3:
        return

    low_strike = strikes[0]
    middle_strike = strikes[1]
    high_strike = strikes[2]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.bull_call_ladder` method with the details of each leg and then pass the result to the `buy` method.

```
option_strategy = OptionStrategies.bull_call_ladder(self._symbol, low_strike, middle_strike, high_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_call = next(filter(lambda x: x.strike == low_strike, calls))
middle_strike_call = next(filter(lambda x: x.strike == middle_strike, calls))
high_strike_call = next(filter(lambda x: x.strike == high_strike, calls))

legs = [
    Leg.create(low_strike_call.symbol, 1),
    Leg.create(middle_strike_call.symbol, -1),
    Leg.create(high_strike_call.symbol, -1)
]
self.combo_market_order(legs, 1)
```

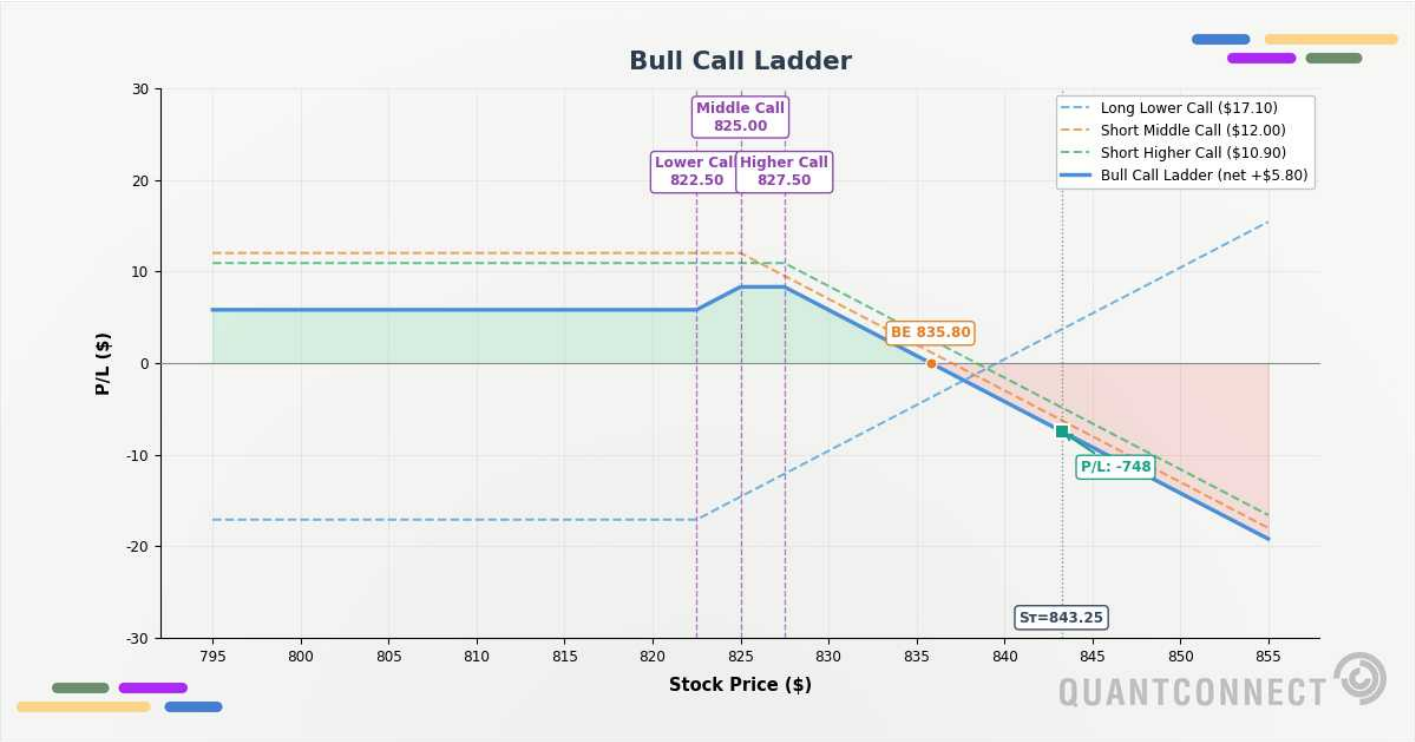
Strategy Payoff

The bull call spread is an limited-profit strategy. The payoff is

$$\begin{array}{rcll} C^{low}_T &=& (S_T - K^{low})^{+}\\ C^{mid}_T &=& (S_T - K^{mid})^{+}\\ C^{high}_T &=& (S_T - K^{high})^{+}\\ Payoff_T &=& (C^{low}_T - C^{low}_0 + C^{mid}_0 - C^{mid}_T + C^{high}_0 - C^{high}_T)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{low}_T &=& \textrm{Lower-strike call value at time T}\\ C^{mid}_T &=& \textrm{Middle-strike call value at time T}\\ C^{high}_T &=& \textrm{Higher-strike call value at time T}\\ S_T &=& \textrm{Underlying asset price at time T}\\ K^{low} &=& \textrm{Lower-strike call strike price}\\ K^{mid} &=& \textrm{Middle-strike call strike price}\\ K^{high} &=& \textrm{Higher-strike call strike price}\\ C^{low}_0 &=& \textrm{Lower-strike call value at position opening (credit received)}\\ C^{mid}_0 &=& \textrm{Middle-strikeTM call value at position opening (debit paid)}\\ C^{high}_0 &=& \textrm{Higher-strike call value at position opening (debit paid)}\\ m &=& \textrm{Contract multiplier}\\ T &=& \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a lower-strike call at $822.50 for $17.10 , a middle-strike call at $825.00 for $12.00 , and a higher-strike call at $827.50 for $10.90 .

The following chart shows the payoff at expiration:



The maximum profit is $K^{mid} - K^{low} - C^{low}_0 + C^{mid}_0 + C^{high}_0$, which occurs when the underlying price is between the two higher strike prices.

The maximum loss is unlimited, which occurs when the underlying price increases indefinitely.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike call	17.10	822.50
Middle-strike call	12.00	825.00
Higher-strike call	10.90	827.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ & = & (843.25-822.50)^{+}\\ & = & 20.75\\ C^{mid}_T & = & (S_T - K^{mid})^{+}\\ & = & (843.25-825.00)^{+}\\ & = & 18.25\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ & = & (843.25-827.50)^{+}\\ & = & 15.75\\ Payoff_T & = & (C^{low}_T - C^{low}_0 + C^{mid}_0 - C^{mid}_T + C^{high}_0 - C^{high}_T)\times m - fee\\ & = & (20.75 - 17.10 + 12.00 - 18.25 + 10.90 - 15.75)\times100-1.00\times3\\ & = & -748\\ \end{array}$$

So, the strategy loses $748.

The following algorithm implements a bull call ladder Option strategy:

```py
class BullCallLadderOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda x: x.call_ladder(30, 5, 0, -5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if not calls:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in calls))
        if len(strikes) < 3:
            return

        low_strike = strikes[0]
        middle_strike = strikes[1]
        high_strike = strikes[2]

        option_strategy = OptionStrategies.bull_call_ladder(self._symbol, low_strike, middle_strike, high_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Bull Put Ladder

Introduction

Bull put ladder , also known as short put ladder , is a combination of a bull put spread and short put with a lower strike price than the 2 legs of the put spread. All puts have the same underlying Equity and expiration date. This strategy profits from increasing volatility of the underlying asset. For instance, the underlying price moves away from its current price.

Implementation

Follow these steps to implement the bull put ladder strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe .

```
PY
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_ladder(30, 5, 0, -5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```
PY
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
    if not puts:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in puts))
    if len(strikes) < 3:
        return

    low_strike = strikes[0]
    middle_strike = strikes[1]
    high_strike = strikes[2]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Put the `OptionStrategies.bull_put_ladder` method with the details of each leg and then pass the result to the `buy` method.

```
PY
option_strategy = OptionStrategies.bull_put_ladder(self._symbol, high_strike, middle_strike, low_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then put the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_put = next(filter(lambda x: x.strike == low_strike, puts))
middle_strike_put = next(filter(lambda x: x.strike == middle_strike, puts))
high_strike_put = next(filter(lambda x: x.strike == high_strike, puts))

legs = [
    Leg.create(low_strike_put.symbol, 1),
    Leg.create(middle_strike_put.symbol, 1),
    Leg.create(high_strike_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The bull put spread is an limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{low}_T &=& (K^{low} - S_T)^{+}\\ P^{mid}_T &=& (K^{mid} - S_T)^{+}\\ P^{high}_T &=& (K^{high} - S_T)^{+}\\ Payoff_T &=& (P^{low}_T - P^{low}_0 + P^{mid}_T - P^{mid}_0 + P^{high}_0 - P^{high}_T)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ P^{low}_T &=& \textrm{Lower-strike put value at time T}\\ P^{mid}_T &=& \textrm{Middle-strike put value at time T}\\ P^{high}_T &=& \textrm{Higher-strike put value at time T}\\ S_T &=& \textrm{Underlying asset price at time T}\\ K^{low} &=& \textrm{Lower-strike put strike price}\\ K^{mid} &=& \textrm{Middle-strike put strike price}\\ K^{high} &=& \textrm{Higher-strike put strike price}\\ P^{low}_0 &=& \textrm{Lower-strike put value at position opening (credit received)}\\ P^{mid}_0 &=& \textrm{Middle-strikeTM put value at position opening (debit paid)}\\ P^{high}_0 &=& \textrm{Higher-strike put value at position opening (debit paid)}\\ m &=& \textrm{Contract multiplier}\\ T &=& \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.25 . You traded a lower-strike put at $822.50 for $6.00 , a middle-strike put at $825.00 for $4.70 , and a higher-strike put at $827.50 for $5.60 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{mid} + K^{low} - K^{high} - P^{low}_0 - P^{mid}_0 + P^{high}_0$, which occurs when the underlying price decreases to $0 .

The maximum loss is $K^{mid} - K^{high} - P^{low}_0 - P^{mid}_0 + P^{high}_0$, which occurs when the underlying price is between the two lower strike prices.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike put	6.00	822.50
Middle-strike put	4.70	825.00
Higher-strike put	5.60	827.50
Underlying Equity at expiration	843.25	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ & = & (822.50-843.25)^{+}\\ & = & 0\\ P^{mid}_T & = & (K^{mid} - S_T)^{+}\\ & = & (825.00-843.25)^{+}\\ & = & 0\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ & = & (827.50-843.25)^{+}\\ & = & 0\\ Payoff_T & = & (P^{low}_T - P^{low}_0 + P^{mid}_T - P^{mid}_0 + P^{high}_0 - P^{high}_T)\times m - fee\\ & = & (0 - 6.00 + 0 - 4.70 + 5.60 - 0)\times100-1.00\times3\\ & = & -513\\ \end{array}$$

So, the strategy loses $513.

The following algorithm implements a bull put ladder Option strategy:

```py
class BullPutLadderOptionStrategy(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol

        # set our strike/expiry filter for this option chain
        option.set_filter(lambda universe: universe.put_ladder(30, 5, 0, -5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        puts = [i for i in chain if i.expiry == expiry and i.right == OptionRight.PUT]
        if not puts:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in puts))
        if len(strikes) < 3:
            return

        low_strike = strikes[0]
        middle_strike = strikes[1]
        high_strike = strikes[2]

        option_strategy = OptionStrategies.bull_put_ladder(self._symbol, high_strike, middle_strike, low_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Long Call Backspread

Introduction

Long Call Backspread , consists of short 1 lower-strike call and long 2 higher strike calls. It is a combination of a bear call spread and a long call with the same strike price as the higher-strike leg the call spread. All calls have the same underlying Equity and expiration date. This strategy profits from increasing volatility of the underlying asset. For instance, the underlying price moves away from its current price.

Implementation

Follow these steps to implement the long call backspread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe . You can use the `call_spread` helper method in option universe filtering, since a call backspread consists of the same contracts as a call spread.

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_spread(20, 5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in calls))
    if len(strikes) < 2:
        return

    low_strike = strikes[0]
    high_strike = strikes[1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.call_backspread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.call_backspread(self._symbol, low_strike, high_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_call = next(filter(lambda x: x.strike == low_strike, calls))
high_strike_call = next(filter(lambda x: x.strike == high_strike, calls))

legs = [
    Leg.create(low_strike_call.symbol, -1),
    Leg.create(high_strike_call.symbol, 2)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The long call backspread is an unlimited-profit-limited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ Payoff_T & = & (C^{low}_0 - C^{low}_T + C^{high}_T \times 2 - C^{high}_0 \times 2)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{low}_T & = & \textrm{Lower-strike call value at time T}\\ C^{high}_T & = & \textrm{Higher-strike call value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{low} & = & \textrm{Lower-strike call strike price}\\ K^{high} & = & \textrm{Higher-strike call strike price}\\ C^{low}_0 & = & \textrm{Lower-strike call value at position opening (credit received)}\\ C^{high}_0 & = & \textrm{Higher-strike call value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.19 . You traded a lower-strike call at $825.00 for $12.00 and two higher-strike calls at $835.00 for $8.80 each.

The following chart shows the payoff at expiration:

The maximum profit is unlimited, which occurs when the underlying price increases indefinitely.

The maximum loss is $K^{low} - K^{high} + C^{low}_0 - C^{high}_0 \times 2$, which occurs when the underlying price is exactly at the higher strike at expiry.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike call	12.00	825.00
Higher-strike call	8.80	835.00
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ & = & (843.19-825.00)^{+}\\ & = & 18.19\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ & = & (843.19-835.00)^{+}\\ & = & 8.19\\ Payoff_T & = & (C^{low}_0 - C^{low}_T + C^{high}_T \times 2 - C^{high}_0 \times 2)\times m - fee\\ & = & (12.00 - 18.19 + 8.19\times2 - 8.80\times2)\times100-2.30\\ & = & -743.30\\ \end{array}$$

So, the strategy loses $743.30.

The following algorithm implements a long call backspread Option strategy:

```py
class BackspreadOptionStrategyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        self.universe_settings.asynchronous = True
        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol
        option.set_filter(lambda universe: universe.call_spread(20, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if not calls:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in calls))
        if len(strikes) < 2:
            return

        low_strike = strikes[0]
        high_strike = strikes[1]

        option_strategy = OptionStrategies.call_backspread(self._symbol, low_strike, high_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Short Call Backspread

Introduction

Short Call Backspread , consists of long 1 lower-strike call and short 2 higher strike calls. It is a combination of a bull call spread and a short call with the same strike price as the higher-strike leg the call spread. All calls have the same underlying Equity and expiration date. This strategy profits from stable, consistent price of the underlying asset. For instance, the underlying price stays at its current price.

Implementation

Follow these steps to implement the short call backspread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe . You can use the `call_spread` helper method in option universe filtering, since a call backspread consists of the same contracts as a call spread.

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.call_spread(20, 5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the call Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
    if not calls:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in calls))
    if len(strikes) < 2:
        return

    low_strike = strikes[0]
    high_strike = strikes[1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_call_backspread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.short_call_backspread(self._symbol, low_strike, high_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_call = next(filter(lambda x: x.strike == low_strike, calls))
high_strike_call = next(filter(lambda x: x.strike == high_strike, calls))

legs = [
    Leg.create(low_strike_call.symbol, 1),
    Leg.create(high_strike_call.symbol, -2)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The short call backspread is an limited-profit-unlimited-risk strategy. The payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ Payoff_T & = & (C^{low}_T - C^{low}_0 + C^{high}_0 \times 2 - C^{high}_T \times 2)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ C^{low}_T & = & \textrm{Lower-strike call value at time T}\\ C^{high}_T & = & \textrm{Higher-strike call value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{low} & = & \textrm{Lower-strike call strike price}\\ K^{high} & = & \textrm{Higher-strike call strike price}\\ C^{low}_0 & = & \textrm{Lower-strike call value at position opening (credit received)}\\ C^{high}_0 & = & \textrm{Higher-strike call value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.19 . You traded a lower-strike call at $825.00 for $15.10 and two higher-strike calls at $835.00 for $8.00 each.

The following chart shows the payoff at expiration:

The maximum profit is $K^{high} - K^{low} - C^{low}_0 + C^{high}_0 \times 2$, which occurs when the underlying price is exactly at the higher strike at expiry.

The maximum loss is unlimited, which occurs when the underlying price increases indefinitely.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike call	15.10	825.00
Higher-strike call	8.00	835.00
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

$$\begin{array}{rcll} C^{low}_T & = & (S_T - K^{low})^{+}\\ & = & (843.19-825.00)^{+}\\ & = & 18.19\\ C^{high}_T & = & (S_T - K^{high})^{+}\\ & = & (843.19-835.00)^{+}\\ & = & 8.19\\ Payoff_T & = & (C^{low}_T - C^{low}_0 - C^{high}_T \times 2 + C^{high}_0 \times 2)\times m - fee\\ & = & (18.19 - 15.10 - 8.19\times2 + 8.00\times2)\times100-2.30\\ & = & 268.70\\ \end{array}$$

So, the strategy gains $268.70.

The following algorithm implements a short call backspread Option strategy:

```py
class BackspreadOptionStrategyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        self.universe_settings.asynchronous = True
        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol
        option.set_filter(lambda universe: universe.call_spread(20, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        calls = [i for i in chain if i.expiry == expiry and i.right == OptionRight.CALL]
        if not calls:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in calls))
        if len(strikes) < 2:
            return

        low_strike = strikes[0]
        high_strike = strikes[1]

        option_strategy = OptionStrategies.short_call_backspread(self._symbol, low_strike, high_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Long Put Backspread

Introduction

Long Put Backspread , consists of short 1 higher-strike put and long 2 lower-strike puts. It is a combination of a bull put spread and a long put with the same strike price as the lower-strike leg the put spread. All puts have the same underlying Equity and expiration date. This strategy profits from increasing volatility of the underlying asset. For instance, the underlying price moves away from its current price.

Implementation

Follow these steps to implement the long put backspread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe . You can use the `put_spread` helper method in option universe filtering, since a put backspread consists of the same contracts as a put spread.

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_spread(20, 5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry]
    if not puts:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in puts))
    if len(strikes) < 2:
        return

    low_strike = strikes[0]
    high_strike = strikes[1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.put_backspread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.put_backspread(self._symbol, high_strike, low_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_put = next(filter(lambda x: x.strike == low_strike, puts))
high_strike_put = next(filter(lambda x: x.strike == high_strike, puts))

legs = [
    Leg.create(low_strike_put.symbol, 2),
    Leg.create(high_strike_put.symbol, -1)
]
self.combo_market_order(legs, 1)
```

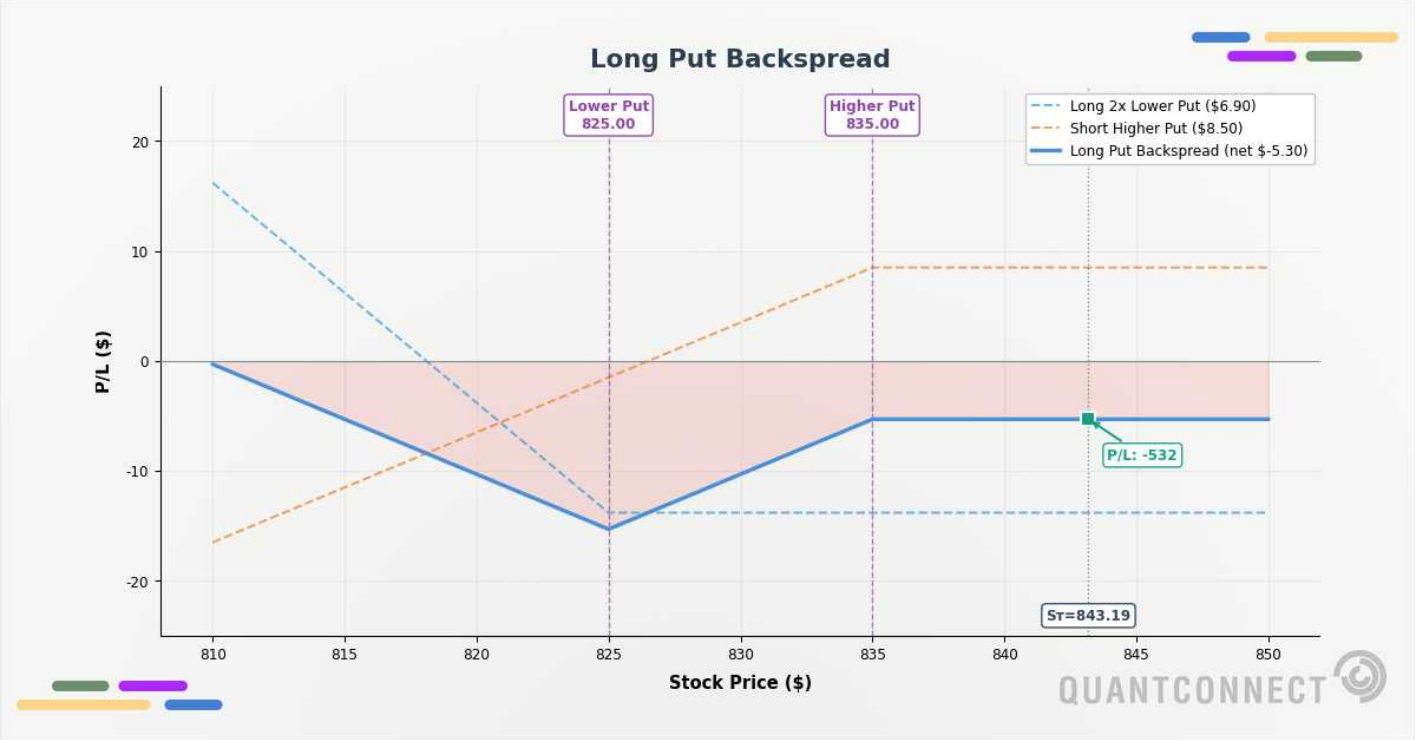
Strategy Payoff

The long put backspread is an unlimited-profit-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ Payoff_T & = & (P^{high}_0 - P^{high}_T + P^{low}_T \times 2 - P^{low}_0 \times 2)\times m - fee\\ \end{array} \begin{array}{rcl} \textrm{where} & & \\ P^{low}_T & = & \textrm{Lower-strike put value at time T}\\ P^{high}_T & = & \textrm{Higher-strike put value at time T}\\ S_T & = & \textrm{Underlying asset price at time T}\\ K^{low} & = & \textrm{Lower-strike put strike price}\\ K^{high} & = & \textrm{Higher-strike put strike price}\\ P^{low}_0 & = & \textrm{Lower-strike put value at position opening (credit received)}\\ P^{high}_0 & = & \textrm{Higher-strike put value at position opening (debit paid)}\\ m & = & \textrm{Contract multiplier}\\ T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.19 . You traded two lower-strike puts at $825.00 for $6.90 each and a higher-strike put at $835.00 for $8.50 .

The following chart shows the payoff at expiration:



The maximum profit is unlimited, which occurs when the underlying price increases indefinitely.

The maximum loss is $K^{low} - K^{high} + P^{high}_0 - P^{low}_0 \times 2$, which occurs when the underlying price is exactly at the lower strike at expiry.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike put	6.90	825.00
Higher-strike put	8.50	835.00
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ & = & (825.00-843.19)^{+}\\ & = & 0.00\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ & = & (835.00-843.19)^{+}\\ & = & 0.00\\ Payoff_T & = & (P^{high}_0 - P^{high}_T + P^{low}_T \times 2 - P^{low}_0 \times 2)\times m - fee\\ & = & (8.50 - 0.00 + 0.00\times2 - 6.90\times2)\times100-2.30\\ & = & -532.30\\ \end{array}$$

So, the strategy loses $532.30.

The following algorithm implements a long put backspread Option strategy:

```py
class BackspreadOptionStrategyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        self.universe_settings.asynchronous = True
        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol
        option.set_filter(lambda universe: universe.put_spread(20, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        puts = [i for i in chain if i.expiry == expiry]
        if not puts:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in puts))
        if len(strikes) < 2:
            return

        low_strike = strikes[0]
        high_strike = strikes[1]

        option_strategy = OptionStrategies.put_backspread(self._symbol, high_strike, low_strike, expiry)
        self.buy(option_strategy, 1)
```

Option Strategies

Short Put Backspread

Introduction

Short Put Backspread , consists of short 1 higher-strike put and short 2 lower-strike puts. It is a combination of a bear put spread and a short put with the same strike price as the lower-strike leg the put spread. All puts have the same underlying Equity and expiration date. This strategy profits from stable, consistent price of the underlying asset. For instance, the underlying price stays at its current price.

Implementation

Follow these steps to implement the short put backspread strategy:

1. In the `initialize` method, set the start date, end date, cash, and Option universe . You can use the `put_spread` helper method in option universe filtering, since a put backspread consists of the same contracts as a put spread.

```py
def initialize(self) -> None:
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(1000000)

    self.universe_settings.asynchronous = True
    option = self.add_option("GOOG", Resolution.MINUTE)
    self._symbol = option.symbol
    option.set_filter(lambda universe: universe.put_spread(20, 5))
```

2. In the `on_data` method, select the expiration and strikes of the contracts in the strategy legs.

```py
def on_data(self, slice: Slice) -> None:
    if self.portfolio.invested:
        return

    # Get the OptionChain
    chain = slice.option_chains.get(self._symbol, None)
    if not chain:
        return

    # Select the put Option contracts with the furthest expiry
    expiry = max([x.expiry for x in chain])
    puts = [i for i in chain if i.expiry == expiry]
    if not puts:
        return

    # Select the strike prices from the remaining contracts
    strikes = sorted(set(x.strike for x in puts))
    if len(strikes) < 2:
        return

    low_strike = strikes[0]
    high_strike = strikes[1]
```

3. In the `on_data` method, select the contracts and place the orders.

   Approach A: Call the `OptionStrategies.short_put_backspread` method with the details of each leg and then pass the result to the `buy` method.

```py
option_strategy = OptionStrategies.short_put_backspread(self._symbol, high_strike, low_strike, expiry)
self.buy(option_strategy, 1)
```

   Approach B: Create a list of `Leg` objects and then call the combo_market_order , combo_limit_order , or combo_leg_limit_order method.

```
low_strike_put = next(filter(lambda x: x.strike == low_strike, puts))
high_strike_put = next(filter(lambda x: x.strike == high_strike, puts))

legs = [
    Leg.create(low_strike_put.symbol, -2),
    Leg.create(high_strike_put.symbol, 1)
]
self.combo_market_order(legs, 1)
```

Strategy Payoff

The short put backspread is an unlimited-profit-limited-risk strategy. The payoff is

$$\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ Payoff_T & = & (P^{high}_T - P^{high}_0 - P^{low}_T \times 2 + P^{low}_0 \times 2)\times m - fee\\ \end{array} \begin{array}{rcll} \textrm{where} & P^{low}_T & = & \textrm{Lower-strike put value at time T}\\ & P^{high}_T & = & \textrm{Higher-strike put value at time T}\\ & S_T & = & \textrm{Underlying asset price at time T}\\ & K^{low} & = & \textrm{Lower-strike put strike price}\\ & K^{high} & = & \textrm{Higher-strike put strike price}\\ & P^{low}_0 & = & \textrm{Lower-strike put value at position opening (credit received)}\\ & P^{high}_0 & = & \textrm{Higher-strike put value at position opening (debit paid)}\\ & m & = & \textrm{Contract multiplier}\\ & T & = & \textrm{Time of expiration} \end{array}$$

In this example, the underlying stock price at expiration is $843.19 . You traded two lower-strike puts at $825.00 for $4.70 each and a higher-strike put at $835.00 for $10.90 .

The following chart shows the payoff at expiration:

The maximum profit is $K^{high} - K^{low} - P^{high}_0 + P^{low}_0 \times 2$, which occurs when the underlying price is exactly at the lower strike at expiry.

The maximum loss is $P^{low}_0 \times 2 - P^{high}_0 - S_T$, which occurs when the underlying price drops to zero.

If the Option is American Option, there is a risk of early assignment on the contract you sell.

Example

The following table shows the price details of the assets in the algorithm:

Asset	Price ($)	Strike ($)
Lower-Strike put	4.70	825.00
Higher-strike put	10.90	835.00
Underlying Equity at expiration	843.19	-

Therefore, the payoff is

$$\begin{array}{rcll} P^{low}_T & = & (K^{low} - S_T)^{+}\\ & = & (825.00-843.19)^{+}\\ & = & 0.00\\ P^{high}_T & = & (K^{high} - S_T)^{+}\\ & = & (835.00-843.19)^{+}\\ & = & 0.00\\ Payoff_T & = & (P^{high}_T - P^{high}_0 - P^{low}_T \times 2 + P^{low}_0 \times 2)\times m - fee\\ & = & (0 - 10.90 - 0.00\times2 + 4.70\times2)\times100-2.30\\ & = & -152.30\\ \end{array}$$

So, the strategy loses $152.30.

The following algorithm implements a short put backspread Option strategy:

```
class BackspreadOptionStrategyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        self.universe_settings.asynchronous = True
        option = self.add_option("GOOG", Resolution.MINUTE)
        self._symbol = option.symbol
        option.set_filter(lambda universe: universe.put_spread(20, 5))

    def on_data(self, slice: Slice) -> None:
        if self.portfolio.invested:
            return

        # Get the OptionChain
        chain = slice.option_chains.get(self._symbol, None)
        if not chain:
            return

        # Select the call Option contracts with the furthest expiry
        expiry = max([x.expiry for x in chain])
        puts = [i for i in chain if i.expiry == expiry]
        if not puts:
            return

        # Select the strike prices from the remaining contracts
        strikes = sorted(set(x.strike for x in puts))
        if len(strikes) < 2:
            return

        low_strike = strikes[0]
        high_strike = strikes[1]

        option_strategy = OptionStrategies.short_put_backspread(self._symbol, high_strike, low_strike, expiry)
        self.buy(option_strategy, 1)
```

Trading and Orders

Order Properties

Introduction

Order properties enable you to customize how the brokerage executes your orders. The `default_order_properties` of your algorithm sets the order properties for all of your orders. To adjust the order properties of an order, you can change the `default_order_properties` or pass an order properties object to the order method.

Time In Force

The `time_in_force` property determines how long an order should remain open if it doesn't fill. This property doesn't apply to market orders since they usually fill immediately. Time in force is useful to automatically cancel old trades. The following table describes the available `TimeInForce` options:

Member	Example	Description
`GOOD_TIL_CANCELED`	`TimeInForce.GOOD_TIL_CANCELED`	Order is valid until filled (default)
`DAY`	`TimeInForce.DAY`	Order is valid until filled or the market closes
`good_til_date(expiry: datetime)`	`time_in_force.good_til_date(datetime(2019, 6, 19, 12, 0, 0))`	Order is valid until filled or the specified expiration time

By default, orders remain open until they are canceled (`TimeInForce.GOOD_TIL_CANCELED`). To update the value, set the `default_order_properties.time_in_force` before you place an order or pass an `order_properties` argument to the order method.

PY

```
# Set a Limit Order to be good until market close
self.default_order_properties.time_in_force = TimeInForce.DAY
self.limit_order("IBM", 100, 120)

# Set a Limit Order to be good until noon
order_properties = OrderProperties()
order_properties.time_in_force = TimeInForce.good_til_date(datetime(2019, 6, 19, 12, 0, 0))
self.limit_order("IBM", 100, 120, order_properties=order_properties)
```

If you trade a market that's open 24 hours a day with daily data, `TimeInForce.DAY` won't work because the order cancels at the market close time but your algorithm receives daily data at midnight.

Market on open (MOO) and market on close (MOC) orders don't support the `good_til_date` time in force. If you submit a MOO or MOC order with the `good_til_date` time in force, LEAN automatically adjusts the time in force to be `GoodTilCanceled` .

The brokerage you use may not support all of the `TimeInForce` options. To see the options that your brokerage supports, see the Orders section of the brokerage model documentation .

Brokerage-Specific Properties

Some brokerages support additional order properties so you can customize how your orders execute. Some examples include the following order properties:

- Financial Advisor group orders
- An `outside_regular_trading_hours` property to let orders fill during pre-market and post-market trading hours
- A `post_only` property to force an order to only add liquidity to a market
- A `hidden` property to make an order not show on the order book
- A `reduce_only` property to signal the order must only decrease your position size
- `fee_in_base` and `fee_in_quote` properties to set which currency you pay fees in for a Crypto trade

To view the order properties your brokerage supports, see the Orders section of the brokerage model documentation .

Tags

You can tag orders to aid your strategy development. Tags can be any string of up to 100 characters. Tags aren't part of the `OrderProperties` object, but they are a property of the `Order` class you can set. To set an order tag, pass it as an argument when you create the order or use the order update methods. This is useful for debugging your algorithm's trading logic because you can include the indicator values or conditions that triggered the order.

```py
 # Tag an order on creation
ticket = self.limit_order("SPY", 100, 221.05, tag="Original tag")

# Update the tag with UpdateTag
ticket.update_tag("Updated tag")

# Update the tag with UpdateOrderFields
update_settings = UpdateOrderFields()
update_settings.tag = "Another tag"
ticket.update(update_settings)
```

Examples

The following examples demonstrate some common practices for utilizing order properties.

Example 1: Extended Market Hour Orders

The following algorithm trades buys or sells SPY during pre-market to capture momentum in the overnight gap. It exits the positions at 15 minutes after the market opens. To trade in the extended market hours without stale fills, it set the respective order properties.

```py
class OrderPropertiesAlgorithm(QCAlgorithm):
    _last_price = None

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add extended market hour SPY data for trading.
        self._spy = self.add_equity("SPY", extended_market_hours=True).symbol

        # Add a Scheduled Event to trade 2 hours enter a position during pre-market.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.before_market_open(self._spy, 120),
            self._open_position
        )
        # Add a Scheduled Event to exit the position 15 minutes after market open.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.after_market_open(self._spy, 15),
            self.liquidate
        )
        # Add a Scheduled Event to cache the daily close price so you can generate signals the next day.
        self.schedule.on(
            self.date_rules.every_day(self._spy),
            self.time_rules.before_market_close(self._spy),
            self._get_last_price
        )
        # Add a warm-up period so the algorithm trades on the first day in live mode.
        self.set_warm_up(timedelta(1))

    def _open_position(self) -> None:
        if self._last_price:
            current_price = self.securities[self._spy].price

            # Define the order properties to trade in pre-market. The order will only valid for 3 minutes to avoid
            # stale fills.
            order_properties = OrderProperties()
            order_properties.time_in_force = TimeInForce.good_til_date(self.time + timedelta(minutes=3))
            order_properties.outside_regular_trading_hours = True

            # Buy if the overnight direction is upward, indicating positive market sentiment.
            # Sell if the overnight direction is downward, indicating negative market sentiment.
            # Note that pre-market orders do not accept market order.
            quantity = 10 if current_price > self._last_price else -10
            self.limit_order(self._spy, quantity, current_price, order_properties=order_properties)

    def _get_last_price(self) -> None:
        self._last_price = self.securities[self._spy].price
```

Example 2: Financial Advisor Managing Client Funds

The following algorithm forms an equal-weight portfolio at the start of each week with the 10 most liquid US Equities. It demonstrates how order properties can give financial advisors easy control over client funds.

```py
class OrderPropertiesAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Define the order properties to trade with an FA group.
        self.default_order_properties = InteractiveBrokersOrderProperties()
        self.default_order_properties.fa_group = "TestGroupEQ"          # FA group
        self.default_order_properties.fa_method = "EqualQuantity"       # Allocation by net liquidation
        self.default_order_properties.account = "DU123456"              # FA account

        # Add a universe that selects the 10 most liquid US Equities at the start of each week.
        spy = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.week_start(spy))
        universe = self.add_universe(self.universe.top(10))

        # Add a Scheduled Event to rebalance at the start of each week.
        self.schedule.on(
            self.date_rules.week_start(spy),
            self.time_rules.at(10, 0),
            lambda: self.set_holdings(
                # Form an equal-weighted portfolio of the universe constituents.
                [PortfolioTarget(symbol, 0.1) for symbol in universe.selected],
                # Liquidate the assets that aren't in the universe anymore.
                liquidate_existing_holdings=True
            )
        )
```

Trading and Orders

Order Events

Introduction

An `OrderEvent` object represents an update to the state of an order. As the state of your orders change, we notify your algorithm with `OrderEvent` objects through the `on_order_event` and `on_assignment_order_event` event handlers.

Track Order Events

Each order generates events over its life as its status changes. Your algorithm receives these events through the `on_order_event` and `on_assignment_order_event` methods. The `on_order_event` event handler receives all order events. The `on_assignment_order_event` receives order events for Option assignments. The event handlers receive an `OrderEvent` object, which contains information about the order status.

```
PY
def on_order_event(self, order_event: OrderEvent) -> None:
    order = self.transactions.get_order_by_id(order_event.order_id)
    if order_event.status == OrderStatus.FILLED:
        self.debug(f"{self.time}: {order.type}: {order_event}")

def on_assignment_order_event(self, assignment_event: OrderEvent) -> None:
    self.log(str(assignment_event))
```

To get a list of all `OrderEvent` objects for an order, call the `order_events` method of the order ticket .

```
PY
order_events = order_ticket.order_events()
```

If you don't have the order ticket, get the order ticket from the TransactionManager .

Order States

Orders can have any of the following states:

Event Attributes

The `on_order_event` and `on_assignment_order_event` event handlers in your algorithm receive `OrderEvent` objects, which have the following attributes:

Examples

The following examples demonstrate some common practices for using order events.

Example 1: Illiquid Stock Partial Fill

The following algorithm trades EMA cross on CARZ, an illiquid ETF. To realistically simulate the fill behavior, we set a fill model to partially fill the orders with at most 50% of the previous bar's volume per fill. We cancel the remaining open order after the partial fill since we only trade on the updated information.

```py
class OrderEventsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request CARZ data to feed indicator and trade.
        equity = self.add_equity("CARZ")
        self.carz = equity.symbol
        # Set a custom partial fill model for the illiquid CARZ stock since it is more realistic.
        equity.set_fill_model(CustomPartialFillModel(self))

        # Create EMA indicator to generate trade signals.
        self._ema = self.ema(self.carz, 60, Resolution.DAILY)
        # Warm-up indicator for immediate readiness to use.
        self.warm_up_indicator(self.carz, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.carz)
        if bar and self._ema.is_ready:
            ema = self._ema.current.value
            # Trade EMA cross on CARZ for trend-following strategy.
            if bar.close > ema and not self.portfolio[self.carz].is_long:
                self.set_holdings(self.carz, 0.5)
            elif bar.close < ema and not self.portfolio[self.carz].is_short:
                self.set_holdings(self.carz, -0.5)

    def on_order_event(self, order_event: OrderEvent) -> None:
        # If an order is only partially filled, we cancel the rest to avoid trade on non-updated information.
        if order_event.status == OrderStatus.PARTIALLY_FILLED:
            self.transactions.cancel_open_orders()

# Implements a custom fill model that partially fills each order with a ratio of the previous trade bar.
class CustomPartialFillModel(FillModel):
    def __init__(self, algorithm: QCAlgorithm, ratio: float = 0.5) -> None:
        self.algorithm = algorithm
        self.absolute_remaining_by_order_id = {}
        # Save the ratio of the volume of the previous bar to fill the order.
        self.ratio = ratio

    def market_fill(self, asset: Security, order: MarketOrder) -> None:
        absolute_remaining = self.absolute_remaining_by_order_id.get(order.id, order. AbsoluteQuantity)

        fill = super().market_fill(asset, order)

        # Partially fill each order with at most 50% of the previous bar.
        fill.fill_quantity = np.sign(order.quantity) * asset.volume * self.ratio
        if (min(abs(fill.fill_quantity), absolute_remaining) == absolute_remaining):
            fill.fill_quantity = np.sign(order.quantity) * absolute_remaining
            fill.status = OrderStatus.FILLED
            self.absolute_remaining_by_order_id.pop(order.id, None)
        else:
            fill.status = OrderStatus.PARTIALLY_FILLED
            # Save the remaining quantity after it is partially filled.
            self.absolute_remaining_by_order_id[order.id] = absolute_remaining - abs(fill.fill_quantity)
            price = fill.fill_price

        return fill
```

Example 2: Price Actions

The following algorithm saves the trailing 3 `TradeBar` objects into a `RollingWindow` . When it identifies a volume contraction breakout price action pattern on the SPY, it buys to ride on the capital inflow. To exit positions, it places a 2% take profit and 1% stop loss order in the `on_order_event` method.

PY

```
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up a rolling window to hold the last 3 trade bars for price action detection as the trade signal.
        self.windows = RollingWindow(3)
        # Warm up the rolling window.
        history = self.history[TradeBar](self.spy, 3, Resolution.MINUTE)
        for bar in history:
            self.windows.add(bar)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Trade the price action if the previous bars fulfill a contraction breakout.
            if self.contraction_action and self.breakout(bar.close):
                self.set_holdings(self.spy, 0.5)

            # Add the current bar to the window.
            self.windows.add(bar)

    def contraction_action(self) -> None:
        # We trade contraction type price action, where the buying preesure is increasing.
        # 1. The last 3 bars are green.
        # 2. The price is increasing in trend.
        # 3. The trading volume is increasing as well.
        # 4. The range of the bars are decreasing.
        return (
            self.windows[2].close > self.windows[2].open and
            self.windows[1].close > self.windows[1].open and
            self.windows[0].close > self.windows[0].open and
            self.windows[0].close > self.windows[1].close > self.windows[2].close and
            self.windows[0].volume > self.windows[1].volume > self.windows[2].volume and
            self.windows[2].close - self.windows[2].open > self.windows[1].close - self.windows[1].open > self.windows[0].close -
self.windows[0].open
        )

    def breakout(self, current_close: float) -> None:
        # Trade breakout from contraction: the breakout should be much greater than the contracted range of the last bar.
        return current_close - self.windows[0].close > (self.windows[0].close - self.windows[0].open) * 2

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * (0.99 if order_event.fill_quantity > 0 else 1.01)
                self.stop_market_order(self.spy, -self.portfolio[self.spy].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * (1.02 if order_event.fill_quantity > 0 else 0.98)
                self.limit_order(self.spy, -self.portfolio[self.spy].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel open orders if stop loss or take profit order fills.
                self.transactions.cancel_open_orders()
```

Trading and Orders

Order Errors

Introduction

If an error occurs with one of your orders, LEAN logs the error message and stops executing.

Types of Order Errors

Errors can occur when you place orders or after you place them.

Errors at Order Submission

The following errors can occur when you submit an order:

- `ALGORITHM_WARMING_UP`
- `BROKERAGE_FAILED_TO_SUBMIT_ORDER`
- `BROKERAGE_MODEL_REFUSED_TO_UPDATE_ORDER`
- `CONVERSION_RATE_ZERO`
- `EUROPEAN_OPTION_NOT_EXPIRED_ON_EXERCISE`
- `EXCEEDED_MAXIMUM_ORDERS`
- `EXCEEDS_SHORTABLE_QUANTITY`
- `EXCHANGE_NOT_OPEN`
- `FOREX_BASE_AND_QUOTE_CURRENCIES_REQUIRED`
- `FOREX_CONVERSION_RATE_ZERO`
- `INSUFFICIENT_BUYING_POWER`
- `MARKET_ON_CLOSE_ORDER_TOO_LATE`
- `MISSING_SECURITY`
- `NON_EXERCISABLE_SECURITY`
- `NON_TRADABLE_SECURITY`
- `OPTION_ORDER_ON_STOCK_SPLIT`
- `ORDER_ALREADY_EXISTS`
- `ORDER_QUANTITY_LESS_THAN_LOT_SIZE`
- `ORDER_QUANTITY_ZERO`
- `PRE_ORDER_CHECKS_ERROR`
- `QUOTE_CURRENCY_REQUIRED`
- `SECURITY_HAS_NO_DATA`
- `UNSUPPORTED_REQUEST_TYPE`

Errors After Order Submission

The following errors can occur for active orders:

- `BROKERAGE_FAILED_TO_UPDATE_ORDER`
- `BROKERAGE_MODEL_REFUSED_TO_SUBMIT_ORDER`
- `INVALID_NEW_ORDER_STATUS`
- `INVALID_ORDER_STATUS`
- `INVALID_REQUEST`
- `REQUEST_CANCELED`
- `UNABLE_TO_FIND_ORDER`

Common Errors

There are a few common order errors you may experience.

Why is my order converted to a market on open order?

If you place a market order when the market is closed, LEAN automatically converts the order into market on open order. This most commonly happens when you use daily or hourly data, which your algorithm can receive when the market is closed. With the `daily_precise_end_time` setting enabled, your algorithm receives daily bars and the last hourly bar of each day at 4 PM Eastern Time (ET). To avoid LEAN from converting your market order to market on open orders, submit your market orders when the market is open. To check if the market is open, call the `is_market_open` method.

```py
# Check the market status before placing an order to avoid unintended market-on-open trades.
if self.is_market_open(self._symbol):
    self.market_order(self._symbol, quantity)
```

Why am I seeing the "stale price" warning?

Stale fills occur when you fill an order with price data that is timestamped an hour or more into the past. Stale fills usually only occur if you trade illiquid assets or if your algorithm uses daily data but you trade intraday with Scheduled Events. If your order is filled with stale data, the fill price may not be realistic. The pre-built fill models can only fill market orders with stale data. To adjust the length of time that needs to pass before an order is considered stale, set the `stale_price_time_span` setting.

```py
def initialize(self) -> None:
    # Adjust the stale price time span to be 10 minutes.
    self.settings.stale_price_time_span = timedelta(minutes=10)
```

Why do I get "Backtest Handled Error: The security with symbol '/ES' is marked as non-tradable"?

This error occurs when you place an order for a continuous Futures contract, which isn't a tradable security. To fix the issue, place the order for a specific Futures contract. To access the currently selected contract in the continuous contract series, use the `mapped` property of the `Future` object.

Why do I get "Backtest Handled Error: Order Error: ids: [1], Insufficient buying power to complete orders" from a Crypto order?

When you place Crypto trades, ensure you have a sufficient balance of the base or quote currency before each trade. If you hold multiple assets and you want to put all of your capital into BTCUSDT, you need to first convert all your non-BTC assets into USDT and then purchase BTCUSDT.

By default, the account currency is USD and your starting cash is 100,000. Some Crypto exchanges don't support fiat currencies, which means you can't convert the 100,000 USD to the pair's quote currency . In this case, set your account currency to the quote currency with a positive quantity.

```py
self.set_account_currency("USDT", 1000)   # Set the account currency to Tether and its quantity to 1,000 USDT
```

Error Code Reference

The `OrderError` enumeration has the following members:

Order Response Error Reference

The following sections explain why each `OrderResponseErrorCode` occurs and how to avoid it.

None

The `OrderResponseErrorCode.NONE` (0) error means there is no order response error.

Processing Error

The `OrderResponseErrorCode.PROCESSING_ERROR` (-1) error occurs in the following situations:

- When you submit a new order, but LEAN throws an exception while checking if you have sufficient buying power for the order.
- When you try to update or cancel an order, but LEAN throws an exception.

To investigate this order response error further, see the `HandleSubmitOrderRequest` , `UpdateOrder` , and `CancelOrder` methods of the BrokerageTransactionHandler in the LEAN GitHub repository.

Order Already Exists

The `OrderResponseErrorCode.ORDER_ALREADY_EXISTS` (-2) error occurs when you submit a new order but you already have an open order or a

completed order with the same order ID. This order response error usually comes from a concurrency issue.

To avoid this order response, don't place two asynchronous orders at the same time.

Insufficient Buying Power

The `OrderResponseErrorCode.INSUFFICIENT_BUYING_POWER` (-3) error occurs when you place an order but the buying power model determines you can't afford it.

To avoid this order response error, ensure you have enough margin remaining to cover the initial margin requirements of the order before placing it.

This error also commonly occurs when you place a market on open order with daily data. If you place the order with `set_holdings` or use `calculate_order_quantity` to determine the order quantity, LEAN calculates the order quantity based on the market close price. If the open price on the following day makes your order more expensive, then you may have insufficient buying power. To avoid the order response error in this case, either use intraday data and place trades when the market is open or adjust your buying power buffer .

```
self.settings.free_portfolio_value_percentage = 0.05
```

Brokerage Model Refused to Submit Order

The `OrderResponseErrorCode.BROKERAGE_MODEL_REFUSED_TO_SUBMIT_ORDER` (-4) error occurs when you place an order but the brokerage model determines it's invalid. The brokerage model usually checks your order meets the following requirements before sending it to the brokerage:

- Supported security types
- Supported order types and their respective requirements
- Supported time in force options
- The order size is larger than the minimum order size

Each brokerage model can have additional order requirements that the brokerage declares. To avoid this order response error, see the Orders section of the brokerage model documentation .

To investigate this order response error further, see the `CanSubmitOrder` method definition of your brokerage model . This order response error occurs when the `CanSubmitOrder` method returns `False` .

Brokerage Failed to Submit Order

The `OrderResponseErrorCode.BROKERAGE_FAILED_TO_SUBMIT_ORDER` (-5) error occurs when you place an order but the brokerage implementation fails to submit the order to your brokerage.

To investigate this order response error further, see the `PlaceOrder` method definition of your brokerage or the BacktestingBrokerage in the LEAN GitHub repository. This order response error occurs when the `PlaceOrder` method throws an error or returns `false` .

Brokerage Failed to Update Order

The `OrderResponseErrorCode.BROKERAGE_FAILED_TO_UPDATE_ORDER` (-6) error occurs when you try to update an order but the brokerage implementation fails to submit the order update request to your brokerage.

To avoid this order response error, see the Orders section of the brokerage model documentation .

To investigate this order response error further, see the `UpdateOrder` method definition of your brokerage or the BacktestingBrokerage in the LEAN GitHub repository. This order response error occurs when the `UpdateOrder` method throws an error or returns `false` .

Brokerage Failed to Cancel Order

The `OrderResponseErrorCode.BROKERAGE_FAILED_TO_CANCEL_ORDER` (-8) error occurs when you try to cancel an order but the brokerage implementation fails to submit the cancel request to your brokerage.

To investigate this order response error further, see the `CancelOrder` method definition of your brokerage or the BacktestingBrokerage in the LEAN GitHub repository. This order response error occurs when `CancelOrder` method throws an error or returns `false` .

Invalid Order Status

The `OrderResponseErrorCode.INVALID_ORDER_STATUS` (-9) error occurs when you try to update or cancel an order but the order is already complete.

An order is complete if it has `OrderStatus.FILLED` , `OrderStatus.CANCELED` , or `OrderStatus.INVALID` .

To avoid this order response error, check the `Status` of an order ticket or order event before you update or cancel the order.

```
if not OrderExtensions.is_closed(order_ticket.status):
    order_ticket.cancel()
```

Unable to Find Order

The `OrderResponseErrorCode.UNABLE_TO_FIND_ORDER` (-10) error occurs when you try to place, update, or cancel an order, but the `BrokerageTransactionHandler` doesn't have a record of the order ID.

To investigate this order response error further, see BrokerageTransactionHandler.cs in the LEAN GitHub repository. This order response error occurs when the `BrokerageTransactionHandler` can't find the order ID in it's `_completeOrders` or `_completeOrderTickets` dictionaries.

Order Quantity Zero

The `OrderResponseErrorCode.ORDER_QUANTITY_ZERO` (-11) error occurs when you place an order that has zero quantity or when you update an order to have a zero quantity. This error commonly occurs if you use the SetHoldings method but the portfolio weight you provide to the method is too small to translate into a non-zero order quantity.

To avoid this order response error, check if the quantity of the order is non-zero before you place the order. If you use the `set_holdings` method, replace it with a combination of the CalculateOrderQuantity and MarketOrder methods.

```
quantity = self.calculate_order_quantity(self._symbol, 0.05)
if quantity:
    self.market_order(self._symbol, quantity)
```

Unsupported Request Type

The `OrderResponseErrorCode.UNSUPPORTED_REQUEST_TYPE` (-12) error occurs in the following situations:

- When you try to exercise an Option contract for which you hold a short position
- When you try to exercise more Option contracts than you hold

To avoid this order response error, check the quantity of your holdings before you try to exercise an Option contract.

```
holding_quantity = self.portfolio[self._contract_symbol].quantity
if holding_quantity > 0:
    self.exercise_option(self._contract_symbol, max(holding_quantity, exercise_quantity))
```

Missing Security

The `OrderResponseErrorCode.MISSING_SECURITY` (-14) error occurs when you place an order for a security but you don't have a subscription for the security in your algorithm.

To avoid this order response error, create a subscription for each security you want to trade. To create subscriptions, see the Requesting Data page of the documentation for each asset class .

Exchange Not Open

The `OrderResponseErrorCode.EXCHANGE_NOT_OPEN` (-15) error occurs in the following situations:

- When you try to exercise an Option while the exchange is not open

  To avoid the order response error in this case, check if the exchange is open before you exercise an Option contract.

  ```
  if self.is_market_open(self.contract_symbol):
      self.exercise_option(self.contract_symbol, quantity)
  ```

- When you try to place a market on open order for a Futures contract or a Future Option contract

Security Price Zero

The `OrderResponseErrorCode.SECURITY_PRICE_ZERO` (-16) error occurs when you place an order or exercise an Option contract while the security price is 0. The security price can be 0 for the following reasons:

- The data is missing

  Investigate if it's a data issue. If it is a data issue, report it .

- The algorithm hasn't received data for the security yet

  If you subscribe to a security and place an order for the security in the same time step , you'll get this error. To avoid this order response error, enable the `seed_initial_prices` setting to seed new assets with their last known price.

  ```
  self.settings.seed_initial_prices = True
  ```
  PY

Forex Base and Quote Currencies Required

The `OrderResponseErrorCode.FOREX_BASE_AND_QUOTE_CURRENCIES_REQUIRED` (-17) error occurs when you place a trade for a Forex or Crypto pair but you don't have the base currency and quote currency in your cash book . This error should never occur. If it does, create a bug report.

Forex Conversion Rate Zero

The `OrderResponseErrorCode.FOREX_CONVERSION_RATE_ZERO` (-18) error occurs when you place a trade for a Forex or Crypto pair and LEAN can't convert the value of the base currency to your account currency. This error usually indicates a lack of data. Investigate the data and if there is some missing, report it .

Security Has No Data

The `OrderResponseErrorCode.SECURITY_HAS_NO_DATA` (-19) error occurs when you place an order for a security before your algorithm receives any data for it. If you subscribe to a security and place an order for the security in the same time step , you'll get this error. To avoid this order response error, enable the `seed_initial_prices` setting to seed new assets with their last known price.

```
self.settings.seed_initial_prices = True
```
PY

Exceeded Maximum Orders

The `OrderResponseErrorCode.EXCEEDED_MAXIMUM_ORDERS` (-20) error occurs when exceed your order quota in a backtest. The number of orders you can place in a single backtest depends on the tier of your organization . The following table shows the number of orders you can place on each tier:

Tier	**Orders Quota**
Free	10K
Quant Researcher	10M
Team	Unlimited
Trading Firm	Unlimited
Institution	Unlimited

To avoid this order response error, reduce the number of orders in your backtest or upgrade your organization .

Market on Close Order Too Late

The `OrderResponseErrorCode.MARKET_ON_CLOSE_ORDER_TOO_LATE` (-21) error occurs when you try to place a market on close (MOC) order too early in the trading day.

To avoid this order response error, place the MOC order closer to the market close or adjust the submission time buffer. By default, you must place

MOC orders at least 15.5 minutes before the close, but some exchanges let you submit them closer to the market closing time. To adjust the buffer period that's required, set the `MarketOnCloseOrder.submission_time_buffer` property.

```
MarketOnCloseOrder.submission_time_buffer = timedelta(minutes=10)
```

Invalid Request

The `OrderResponseErrorCode.INVALID_REQUEST` (-22) error occurs when you try to cancel an order multiple times.

To avoid this order response error, only try to cancel an order one time.

Request Canceled

The `OrderResponseErrorCode.REQUEST_CANCELED` (-23) error occurs when you try to cancel an order multiple times.

To avoid this order response error, only try to cancel an order one time.

Algorithm Warming Up

The `OrderResponseErrorCode.ALGORITHM_WARMING_UP` (-24) error occurs in the following situations:

- When you try to place, update, or cancel an order during the warm-up period
- When the Option assignment simulator assigns you to an Option during the warm-up period

To avoid this order response error, only manage orders after the warm-up period ends. To avoid trading during the warm-up period, add an `is_warming_up` guard to the top of the `on_data` method.

```
if self.is_warming_up: return
```

Brokerage Model Refused to Update Order

The `OrderResponseErrorCode.BROKERAGE_MODEL_REFUSED_TO_UPDATE_ORDER` (-25) error occurs in backtests when you try to update an order in a way that the brokerage model doesn't support.

To avoid this issue, see the Orders section of the brokerage model documentation to check its order requirements.

To investigate this order response error further, see the `CanUpdateOrder` method definition of your brokerage model .

Quote Currency Required

The `OrderResponseErrorCode.QUOTE_CURRENCY_REQUIRED` (-26) error occurs when you place an order for a Forex or Crypto pair and don't have the quote currency of the pair in your cash book . This error should never occur. If it does, create a bug report.

Conversion Rate Zero

The `OrderResponseErrorCode.CONVERSION_RATE_ZERO` (-27) error occurs when you place an order for a Forex or Crypto pair and LEAN can't convert the value of the quote currency in the pair to your account currency. This order response error usually indicates a lack of data. Investigate the data and if there is data missing, report it .

Non-Tradable Security

The `OrderResponseErrorCode.NON_TRADABLE_SECURITY` (-28) error occurs when you place an order for a security that's not tradable . To avoid this order response error, check if a security is tradable before you trade it.

```
if self.securities[self._symbol].is_tradable:
    self.market_order(self._symbol, quantity)
```

Non-Exercisable Security

The `OrderResponseErrorCode.NON_EXERCISABLE_SECURITY` (-29) error occurs when you call the exercise_option method with a `Symbol` that doesn't reference an Option contract.

Order Quantity Less Than Lot Size

The `OrderResponseErrorCode.ORDER_QUANTITY_LESS_THAN_LOT_SIZE` (-30) error occurs when you place an order with a quantity that's less than the lot size of the security.

To avoid this order response error, check if the order quantity is greater than or equal to the security lot size before you place an order.

```
lot_size = self.securities[self._symbol].symbol_properties.lot_size
if quantity >= lot_size:
    self.market_order(self._symbol, quantity)
```

Exceeds Shortable Quantity

The `OrderResponseErrorCode.EXCEEDS_SHORTABLE_QUANTITY` (-31) error occurs when you place an order to short a security but the shortable provider of the brokerage model states there isn't enough shares to borrow. The following algorithm demonstrates this error by using a custom shortable provider that limits shorts to 10 shares and then trying to short 12 shares.

```
# Demonstrate ExceedsShortableQuantity error with a custom shortable provider.
class ExceedsShortableQuantityAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2021, 7, 23)
        self.set_end_date(2021, 7, 30)
        self.set_cash(100000)
        self.set_brokerage_model(CustomShortBrokerageModel())
        self.add_equity("SPY", Resolution.MINUTE)

    def on_data(self, data):
        if not self.portfolio.invested:
            self.market_order("SPY", 1)
        else:
            # Try to short 12 shares, which exceeds the shortable quantity of 10.
            self.market_order("SPY", -12)

    def on_order_event(self, order_event):
        self.debug(f"{self.time} -- {order_event.message}")

# Custom shortable provider that limits shorts to 10 shares.
class CustomShortableProvider:
    def all_shortable_symbols(self, local_time):
        return None
    def shortable_quantity(self, symbol, local_time):
        return 10

# Custom brokerage model using the shortable provider.
class CustomShortBrokerageModel(DefaultBrokerageModel):
    def __init__(self):
        super().__init__(AccountType.MARGIN)
        self.shortable_provider = CustomShortableProvider()
```

To avoid this order response error, check if there are enough shares available before you place an order to short a security.

```
if quantity_to_borrow <= self.shortable_quantity(self._symbol):
    self.market_order(self._symbol, -quantity_to_borrow)
```

Invalid New Order Status

The `OrderResponseErrorCode.INVALID_NEW_ORDER_STATUS` (-32) error occurs in live trading when you try to update or cancel an order while it still has `OrderStatus.NEW` status.

To avoid this order response error, check the `status` property of the order ticket or order event before you update or cancel an order.

```
if self.order_ticket.status != OrderStatus.NEW:
    self.order_ticket.cancel()
```

European Option Not Expired on Exercise

The `OrderResponseErrorCode.EUROPEAN_OPTION_NOT_EXPIRED_ON_EXERCISE` (-33) error occurs when you try to exercise a European Option contract before its expiry date.

To avoid this order response error, check the type and expiry date of the contract before you exercise it.

```py
if self.contract_symbol.id.option_style == OptionStyle.EUROPEAN && self.contract_symbol.id.date == self.time.date:
    self.exercise_option(self.contract_symbol, quantity)
```

Option Order on Stock Split

The `OrderResponseErrorCode.OPTION_ORDER_ON_STOCK_SPLIT` (-34) error occurs when you try to submit an order for an Equity Option contract when the current time slice contains a split for the underlying Equity.

To avoid this order response error, check if the time slice has a split event for the underlying Equity of the contract before you place an order for the contract.

```py
if self.contract_symbol.underlying not in slice.splits:
    self.market_order(self.contract_symbol, quantity)
```

Market on Open Not Allowed During Regular Hours

The `OrderResponseErrorCode.MARKET_ON_OPEN_NOT_ALLOWED_DURING_REGULAR_HOURS` (-35) error occurs when you try to place a market on open order for an asset when it's during regular trading hours.

To avoid this order response error, place the order when the market is closed.

```py
if not self.is_market_open(self._symbol):
    self.market_on_open_order(self._symbol, quantity)
```

Warnings

The following sections explain why each some warnings occur and how to avoid them.

Limit Price Exceeds Precision Requirements

Limit prices must be a multiple of the security's `minimum_price_variation` property . Otherwise, LEAN rounds the limit price for you and throws the following warning message:

Backtest Handled Error: Warning: To meet brokerage precision requirements, order LimitPrice was rounded to 6136.00 from 6136.125

To avoid the warning, round the limit price to the nearest value that respects the `minimum_price_variation` before you place the order.

```py
def _adjust_limit_price(self, security, price, round_up=True):
    pip = security.price_variation_model.get_minimum_price_variation(
        GetMinimumPriceVariationParameters(security, price)
    )
    return (int(price / pip) + int(round_up)) * pip
```

Examples

The following examples demonstrate common practices for handling order errors.

Example 1: Pre-Open Order Error

The following algorithm simulate extended market hour trading on Tradier Brokerage . We place a market-on-open order at 7:30am and expect the order cannot submit to the broker since it is not supported, resulting into an error.

PY

```
class OrderErrorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Simulate Tradier brokerage, which does not support the market-on-open orders.
        self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE, AccountType.CASH)

        # Request extended market hour SPY data for trading.
        self.spy = self.add_equity("SPY", extended_market_hours=True).symbol

        # Set a scheduled event to trade 2 hours pre-open to place the market-on-open-order.
        self.schedule.on(
            self.date_rules.every_day(self.spy),
            self.time_rules.before_market_open(self.spy, 120),
            self.open_position
        )

    def open_position(self) -> None:
        # Buy on market open will result in an error since Tradier does not support it.
        self.market_on_open_order(self.spy, 10)

    # It will be triggered on live trading.
    def on_brokerage_message(self, message: BrokerageMessageEvent) -> None:
        if message.type == BrokerageMessageType.ERROR:
            self.log(f"{self.time}: {message.type}: Message: {message.message}")
```

Trading and Orders

Trade Statistics

Introduction

The `TradeBuilder` tracks the trades of your algorithm and calculates some statistics. Some of these statistics are a function of the risk free interest rate and the number of trading days per year . You can adjust how the `TradeBuilder` defines a trade and how it matches offsetting order fills.

Set Trade Builder

To set the `TradeBuilder` , in the `initialize` method, call the `set_trade_builder` method.

```
self.set_trade_builder(TradeBuilder(grouping_method, matching_method))
```

The following table describes the arguments the `TradeBuilder` constructor accepts:

Argument	Data Type	Description	Default Value
`grouping_method`	`FillGroupingMethod`	The method used to group order fills into trades	
`matching_method`	`FillMatchingMethod`	The method used to match offsetting order fills	

The `FillGroupingMethod` enumeration has the following members:

The `FillMatchingMethod` enumeration has the following members:

Default Behavior

The default `TradeBuilder` uses `FillGroupingMethod.FILL_TO_FILL` and `FillMatchingMethod.FIFO` .

Check Open Positions

To check if you have a position open for a security as defined by the `FillGroupingMethod` , call the `has_open_position` method.

```
# Check if the trade builder has recorded any active trades (unclosed positions)
# for a security.
has_open_position = self.trade_builder.has_open_position(self._symbol)
```

Get Closed Trades

To get your closed trades, use the `closed_trades` property.

```
trades = self.trade_builder.closed_trades
```

This property returns a list of `Trade` objects, which have the following attributes:

Examples

The following examples demonstrate some common practices for making use of trade statistics.

Example 1: Kelly Criterion Sizing Using Previous Trades

The following algorithm uses the previous trades to update the win probability and expected return of each trade of the EMA cross strategy of SPY, by the concept of Bayes Theorem. The updated figures are used for position sizing through Kelly Criterion.

PY

```
class TradeStatisticsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Warm-up indicator for immediate readiness.
        self.warm_up_indicator(self.spy, self._ema, Resolution.DAILY)

        # Set up a trade builder to track the trade statistics; we are interested in open-to-close round trade.
        self.set_trade_builder(TradeBuilder(FillGroupingMethod.FLAT_TO_FLAT, FillMatchingMethod.FIFO))

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if not bar:
            return
        # Trend-following strategy using price and EMA.
        # If the price is above EMA, SPY is in an uptrend, and we buy it.
        sign = 0
        if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
            sign = 1
        elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
            sign = -1
        else:
            return

        size = 1
        trades = self.trade_builder.closed_trades

        if len(trades) > 4:
            # Use the trade builder to obtain the win rate and % return of the last five trades to calculate position size.
            last_five_trades = sorted(trades, key=lambda x: x.exit_time)[-5:]
            prob_win = len([x for x in last_five_trades if x.is_win]) / 5
            win_size = np.mean([x.profit_loss / x.entry_price for x in last_five_trades])
            # Use the Kelly Criterion to calculate the order size.
            size = max(0, prob_win - (1 - prob_win) / win_size)

        self.set_holdings(self.spy, size * sign)
```

Trading and Orders

Trading Calendar

Introduction

The `trading_calendar` contains a variety of events relevant to the assets in your algorithm.

Trading Day Properties

`TradingDay` objects represent the trading events of a single day. They have the following properties:

Get a Single Trading Day

To get the trading events of the current day, call the `get_trading_day` method with no arguments.

```py
trading_day = self.trading_calendar.get_trading_day()
```

To get the trading events of a specific day, pass a `datetime` object to the `get_trading_day` method.

```py
trading_day = self.trading_calendar.get_trading_day(datetime(2022, 6, 1))
```

Get Trading Days

To get all the trading events across a range of dates, pass start and end `datetime` objects to the `get_trading_days` method.

```py
start_date = datetime(2022, 6, 1)
end_date = datetime(2022, 7, 1)
trading_days = self.trading_calendar.get_trading_days(start_date, end_date)
```

To get specific trading events across a range of dates, pass a `TradingDayType` enumeration member and the dates to the `get_days_by_type` method.

```py
trading_days = self.TradingCalendar.get_days_by_type(TradingDayType.OPTION_EXPIRATION, start_date, end_date)
```

The `TradingDayType` enumeration has the following members:

Trading and Orders

Financial Advisors

Introduction

Financial Advisor accounts enable certified professionals to use a single trading algorithm to manage several client accounts. Our Interactive Brokers integration enables you to place FA group orders if your IB account code starts with F, FA, or I.

Group Routing

To place trades using a subset of client accounts, create Account Groups in Trader Workstation and then define the `InteractiveBrokersOrderProperties` when you create orders.

```
self.default_order_properties = InteractiveBrokersOrderProperties()
self.default_order_properties.fa_group = "TestGroupEQ"
self.default_order_properties.fa_method = "Equal"
self.default_order_properties.account = "DU123456"
```

SecurityHolding objects aggregate your positions across all the account groups. If you have two groups where group A has 10 shares of SPY and group B has -10 shares of SPY, then `self.portfolio["SPY"].quantity` is zero.

Allocation Methods

LEAN supports several allocation methods for FA group orders. If you intend to use the same group allocation method for every order, set the `default_order_properties` of your algorithm, which sets the order properties for all of your orders.

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = InteractiveBrokersOrderProperties()
    self.default_order_properties.fa_group = "TestGroupEQ"
    self.default_order_properties.fa_method = "Equal"
    self.default_order_properties.account = "DU123456"

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    LimitOrder(_symbol, quantity, limitPrice);
```

To adjust the order properties of an order, change the `default_order_properties` or pass an order properties object to the order method . The following sections explain the FA group allocation methods.

Equal

This group allocation method distributes shares equally between all accounts in the group. When you use this method, you need to specify an order quantity.

For example, say your Account Group includes four accounts and you place an order to buy 400 shares of a stock. In this case, each account receives 100 shares. If your Account Group includes six accounts, each account receives 66 shares, and then 1 share is allocated to each account until all are distributed. After you submit the order, your algorithm receives order events to track the order progress. When all the shares are bought, the order status is `OrderStatus.FILLED` . If one of the accounts in the group can't afford 10 of the shares it needs to buy, 10 shares are cancelled and you'll only end up buying 390 shares in total.

```
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "Equal"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Net Liquidation Value

This group allocation method distributes shares based on the net liquidation value of each account. The system calculates ratios based on the net

liquidation value in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with Net Liquidation values of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```py
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "NetLiq"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Available Equity

This group allocation method distributes shares based on the amount of available equity in each account. The system calculates ratios based on the available equity in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with available equity of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```py
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "AvailableEquity"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Subscription Requirements

To use FA group orders through our Interactive Brokers integration, you need to connect as a member of a Trading Firm and Institution organization. If you aren't currently on either of these tiers, upgrade your organization .

Reality Modeling

Reality Modeling

Key Concepts

Introduction

Reality models make backtests as realistic as possible to how the strategy would perform in live trading. These reality models model the behavior of things like the portfolio, brokerage, fills, slippage, options, and strategy capacity . Some reality models are set on a security basis and some are set at the portfolio level. The default models assume you trade highly liquid assets. If you trade high volumes or on illiquid assets, you should create custom reality models to be more realistic.

Security Level Models

LEAN's philosophy is to make the models customizable per security as much as possible. The following models are security-level models:

- Fill
- Slippage
- Fee
- Brokerages
- Buying Power
- Settlement
- Short availability
- Option models
  - Pricing
  - Volatility
  - Exercise
  - Assignment
- Margin Interest Rate
- Dividend Yield

To set a security-level reality model, call the set reality model method on the `Security` object. To get the correct method, see the preceding documentation page for each type of model.

```py
# Set IBM to have a constant $1 transaction fee
self.securities["IBM"].set_fee_model(ConstantFeeModel(1))
```

You can also set the security-specific models inside a security initializer .

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fee_model(ConstantFeeModel(1))
```

Portfolio Level Models

A portfolio is a semi-closed system where the state updates over time as the value and quantity of assets in the portfolio fluctuate. It's a semi-closed system because the portfolio usually incurs transaction fees and you can add and deposit capital from the portfolio. LEAN models the portfolio

holdings to track and update the average price of each asset with every transaction.

You can set the following portfolio-level models:

- Margin call model

```
self.portfolio.set_margin_call_model(DefaultMarginCallModel(self.portfolio, self.default_order_properties))
```

- Risk free interest rate model

```
self.set_risk_free_interest_rate_model(InterestRateProvider())
```

Reality Modeling

Trade Fills

Trade Fills

Key Concepts

Introduction

A trade fill is the quantity and price at which your brokerage executes your order in the market. Fill models model how each type of order fills to accurately simulate the behavior of a real brokerage. Fill models determine the price and quantity of your fills, can incorporate spread costs, and work with the slippage model to add slippage into the fill price. If you trade US Equities, our built-in fill models can fill your orders at the official opening and closing auction prices .

Set Models

The brokerage model of your algorithm automatically sets the fill model for each security, but you can override it. To manually set the fill model of a security, call the `set_fill_model` method on the Security object.

PY

```
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # Set the fill model for the requested security to backtest with the most realistic scenario
    # ImmediateFillModel provide no delay from network issue or brokerage rules
    security.set_fill_model(ImmediateFillModel())
```

You can also set the fill model in a security initializer . If your algorithm has a dynamic universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

PY

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fill_model(ImmediateFillModel())
```

To view all the pre-built fill models, see Supported Models .

Default Behavior

The brokerage model of your algorithm automatically sets the fill model of each security. The default brokerage model is the `DefaultBrokerageModel` , which sets the EquityFillModel for Equities, the FutureFillModel for Futures, the FutureOptionFillModel for Future Options, and the ImmediateFillModel for all other asset classes.

Model Structure

Fill Models should extend the `FillModel` class. To implement your own fill model, override the methods in the `FillModel` class you wish to change. The class has a dedicated method for each order type. Most of the methods receive a `Security` and `Order` object and return an `OrderEvent` object that contains information about the order status, fill quantity, and errors.

```py
class CustomFillModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # In the Initialize method, set the custom fill model for an added security to use the custom model
        security = self.add_equity("SPY")
        security.set_fill_model(MyFillModel())

# Define the custom fill model outside of the algorithm
class MyFillModel(FillModel):

    def market_fill(self, asset: Security, order: MarketOrder) -> OrderEvent:
        return super().market_fill(asset, order)

    def limit_fill(self, asset: Security, order: LimitOrder) -> OrderEvent:
        return super().limit_fill(asset, order)

    def limit_if_touched_fill(self, asset: Security, order: LimitIfTouchedOrder) -> OrderEvent:
        return super().limit_if_touched_fill(asset, order)

    def stop_market_fill(self, asset: Security, order: StopMarketOrder) -> OrderEvent:
        return super().stop_market_fill(asset, order)

    def stop_limit_fill(self, asset: Security, order: StopLimitOrder) -> OrderEvent:
        return super().stop_limit_fill(asset, order)

    def trailing_stop_fill(self, asset: Security, order: TrailingStopOrder) -> OrderEvent:
        return super().trailing_stop_fill(asset, order)

    def market_on_open_fill(self, asset: Security, order: MarketOnOpenOrder) -> OrderEvent:
        return super().market_on_open_fill(asset, order)

    def market_on_close_fill(self, asset: Security, order: MarketOnCloseOrder) -> OrderEvent:
        return super().market_on_close_fill(asset, order)

    def combo_market_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_market_fill(order, parameters)

    def combo_limit_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_limit_fill(order, parameters)

    def combo_leg_limit_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_leg_limit_fill(order, parameters)
```

The `FillModelParameters` class has the following properties:

Partial Fills

In live trading, your orders can partially fill. For example, if you have a buy limit order at the bid price for 100 shares and someone sells 10 shares with a market order, your order is partially filled. In backtests, the pre-built fill models assume orders completely fill. To simulate partial fills in backtests, create a custom fill model.

Stale Fills

Stale fills occur when you fill an order with price data that is timestamped an hour or more into the past. Stale fills usually only occur if you trade illiquid assets or if your algorithm uses daily data but you trade intraday with Scheduled Events. If your order is filled with stale data, the fill price may not be realistic. The pre-built fill models can only fill market orders with stale data. To adjust the length of time that needs to pass before an order is considered stale, set the `stale_price_time_span` setting.

```py
def initialize(self) -> None:
    # Adjust the stale price time span to be 10 minutes.
    self.settings.stale_price_time_span = timedelta(minutes=10)
```

Examples

The following examples demonstrate common practices for implementing a custom fill model.

Example 1: Default Model Wrapper

The following algorithm defines a subclass of the default `FillModel` and overrides its built-in members so you can easily customize the parts you need.

```py
class CustomFillModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 10, 15)
        self.set_cash(100000)
        security = self.add_equity('SPY', Resolution.DAILY)
        security.set_fill_model(MyFillModel())

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyFillModel(FillModel):

    def market_fill(self, asset: Security, order: MarketOrder) -> OrderEvent:
        return super().market_fill(asset, order)

    def limit_fill(self, asset: Security, order: LimitOrder) -> OrderEvent:
        return super().limit_fill(asset, order)

    def limit_if_touched_fill(self, asset: Security, order: LimitIfTouchedOrder) -> OrderEvent:
        return super().limit_if_touched_fill(asset, order)

    def stop_market_fill(self, asset: Security, order: StopMarketOrder) -> OrderEvent:
        return super().stop_market_fill(asset, order)

    def stop_limit_fill(self, asset: Security, order: StopLimitOrder) -> OrderEvent:
        return super().stop_limit_fill(asset, order)

    def trailing_stop_fill(self, asset: Security, order: TrailingStopOrder) -> OrderEvent:
        return super().trailing_stop_fill(asset, order)

    def market_on_open_fill(self, asset: Security, order: MarketOnOpenOrder) -> OrderEvent:
        return super().market_on_open_fill(asset, order)

    def market_on_close_fill(self, asset: Security, order: MarketOnCloseOrder) -> OrderEvent:
        return super().market_on_close_fill(asset, order)

    def combo_market_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_market_fill(order, parameters)

    def combo_limit_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_limit_fill(order, parameters)

    def combo_leg_limit_fill(self, order: Order, parameters: FillModelParameters) -> List[OrderEvent]:
        return super().combo_leg_limit_fill(order, parameters)
```

Example 2: Volume Share Fill

The following algorithm longs the top 10 liquid constituents and short the bottom 10 liquid constituents of QQQ. To realistically fill the less liquid stocks, we implement a fill model that only fills with at most 30% of the previous second bar.

```py
class VolumeShareFillModelAlgorithm(QCAlgorithm):
    longs = []
    shorts = []

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # The security initializer applies the VolumeShareFillModel to all assets.
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._volume_share_fill_security_initializer)

        # Request extended market hour SPY data for trading.
        qqq = self.add_equity("QQQ").symbol

        # Weekly portfolio updating to allow time to capitalize on the popularity gap.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Set the resolution to second since the fill model is based on the second bar.
        self.universe_settings.resolution = Resolution.SECOND
        # Add universe to trade on the most and least liquid stocks among QQQ constituents.
        self.add_universe(
            self.universe.etf(qqq, Market.USA, self.universe_settings, lambda constituents: [c.symbol for c in constituents]),
            self.fundamental_selection
        )

        # Set a scheduled event to rebalance the portfolio at the start of every week.
        self.schedule.on(
            self.date_rules.week_start(qqq),
            self.time_rules.after_market_open(qqq),
            self.rebalance
        )

    def fundamental_selection(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        sorted_by_dollar_volume = sorted(fundamentals, key=lambda f: f.dollar_volume)
        # Add the 10 most liquid stocks to the universe to long later.
        self.longs = [f.symbol for f in sorted_by_dollar_volume[-10:]]
        # Add the 10 least liquid stocks to the universe to short later.
        self.shorts = [f.symbol for f in sorted_by_dollar_volume[:10]]

        return self.longs + self.shorts

    def rebalance(self) -> None:
        # Equally invest in the selected stocks to dissipate capital risk evenly.
        # Dollar neutral of long and short stocks to eliminate systematic risk, only capitalize the popularity gap.
        targets = [PortfolioTarget(symbol, 0.05) for symbol in self.longs]
        targets += [PortfolioTarget(symbol, -0.05) for symbol in self.shorts]

        # Liquidate the ones not being the most and least popular stocks to release funds for higher expected return trades.
        self.set_holdings(targets, liquidate_existing_holdings=True)

    def _volume_share_fill_security_initializer(self, security: Security) -> None:
        # Create a slippage model to fill only 30% of the volume of the previous second bar to fill illiquid stocks realistically.
        security.set_fill_model(VolumeShareFillModel(self, 0.3))

class VolumeShareFillModel(FillModel):
    def __init__(self, algorithm: QCAlgorithm, maximum_ratio: float = 0.3):
        self.algorithm = algorithm
        self.maximum_ratio = maximum_ratio
        self.absolute_remaining_by_order_id = {}

    def market_fill(self, asset, order):
        absolute_remaining = self.absolute_remaining_by_order_id.get(order.id, order. AbsoluteQuantity)

        fill = super().market_fill(asset, order)
        # Set the fill amount to 30% of the previous second trade bar.
        fill.fill_quantity = np.sign(order.quantity) * asset.volume * self.maximum_ratio

        if (min(abs(fill.fill_quantity), absolute_remaining) == absolute_remaining):
            fill.fill_quantity = np.sign(order.quantity) * absolute_remaining
            fill.status = OrderStatus.FILLED
            self.absolute_remaining_by_order_id.pop(order.id, None)
        else:
            fill.status = OrderStatus.PARTIALLY_FILLED
            self.absolute_remaining_by_order_id[order.id] = absolute_remaining - abs(fill.fill_quantity)
            price = fill.fill_price

        return fill
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomModelsAlgorithm.py Python CustomPartialFillModelAlgorithm.py Python ComboOrdersFillModelAlgorithm.py Python
ForwardDataOnlyFillModelAlgorithm.py Python

Trade Fills

Supported Models

Fill models determine the price and quantity of your fills. The following fill models are available:

Equity Model

Equity

Future Model

Futures

Future Option Model

Future Options

Immediate Model

All asset classes

Latest Price Model

All asset classes

See Also

Reality Modeling
Order Types

Supported Models

Equity Model

Introduction

The `EquityFillModel` is the default fill model if you trade Equity assets with the DefaultBrokerageModel . This fill model fills trades completely and immediately.

```
security.set_fill_model(EquityFillModel())
```

PY

The fill logic of each order depends on the order type, the data format of the security subscription, and the order direction. The following sections explain the fill logic of each order given these factors.

For more information about this model, see the class reference and implementation .

Market Orders

The model fills buy market orders at the best effort ask price plus slippage and fills sell market orders at the best effort bid price minus slippage.

To get the best effort bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the best effort ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the best effort bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

The model only fills market orders during regular trading hours.

Limit Orders

To fill limit orders, the model first gets the best effort `TradeBar` . To get the best effort `TradeBar` , the model checks the resolution of your security subscription. If your subscription provies `Tick` data, it gets the most recent batch of trade ticks and consolidates them to build a `TradeBar` . If your subscription provies `TradeBar` data, it gets the most recent `TradeBar` . If the `end_time` of the best effort `TradeBar` is less than or equal to the time you placed the order, the model waits until the next best effort `TradeBar` to fill the order.

Once the model has a valid best effort `TradeBar` , it can fill the order. The following table shows the fill condition and fill price of limit orders. The model only fills the order once the fill condition is met.

Order Direction	Fill Condition	Fill Price
Buy	low price < limit price	min(open price, limit price)
Sell	high price > limit price	max(open price, limit price)

The model only fills limit orders when the exchange is open.

The model won't fill limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Limit if Touched Orders

To fill limit if touched orders, the model first gets the best effort `TradeBar` . To get the best effort `TradeBar` , the model checks the resolution of your

security subscription. If your subscription provies `Tick` data, it gets the most recent batch of trade ticks and consolidates them to build a `TradeBar` . If your subscription provies `TradeBar` data, it gets the most recent `TradeBar` . If the `end_time` of the best effort `TradeBar` is less than or equal to the time you placed the order, the model waits until the next best effort `TradeBar` to fill the order.

After the model has a valid best effort `TradeBar` , it can check if the trigger price has been touched. The following table describes the trigger condition of limit if touched orders for each order direction:

Order Direction	Trigger Condition
Buy	low price <= trigger price
Sell	high price >= trigger price

Once the limit if touched order triggers, the model starts to check if it should fill the order. The following table shows the fill condition and fill price of limit if touched orders. The model only fills the order once the fill condition is met

Order Direction	Fill Condition	Fill Price
Buy	Best effort ask price <= limit price	min(best effort ask price, limit price)
Sell	Best effort bid price >= limit price	max(best effort bid price, limit price)

To get the best effort bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the best effort ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the best effort bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

The model only fills limit orders when the exchange is open.

The model won't trigger or fill limit if touched orders with stale data .

Stop Market Orders

To fill stop market orders, the model first gets the best effort `TradeBar` . To get the best effort `TradeBar` , the model checks the resolution of your security subscription. If your subscription provies `Tick` data, it gets the most recent batch of trade ticks and consolidates them to build a `TradeBar` . If your subscription provies `TradeBar` data, it gets the most recent `TradeBar` . If the `end_time` of the best effort `TradeBar` is less than or equal to the time you placed the order, the model waits until the next best effort `TradeBar` to fill the order.

Once the stop condition is met, the model fills the orders and sets the fill price.

Once the model has a valid best effort `TradeBar` , it can fill the order. The following table shows the stop condition and fill price of stop market orders. The model only fills the order once the stop condition is met.

Order Direction	Fill Condition	Fill Price
Buy	high price >= stop price	max(open price, stop price) + slippage
Sell	low price <= stop price	min(open price, stop price) - slippage

The model only fills stop market orders during regular trading hours.

The model won't fill stop market orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Limit Orders

The fill logic of stop limit orders depends on the data format of the security subscription and the order direction. The following table shows the fill price of stop limit orders given these factors. To determine the fill price of the order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

The following table describes how the fill model processes the order given the data format and order direction. Once the stop condition is met, the model starts to check the fill condition. Once the fill condition is met, the model fills the orders and sets the fill price.

Data Format	`TickType`	Order Direction	Stop Condition	Fill Condition	Fill Price
`Tick`	`Quote`	Buy	quote price > stop price	quote price < limit price	min(quote price, limit price)
`Tick`	`Quote`	Sell	quote price < stop price	quote price > limit price	max(quote price, limit price)
`Tick`	`Trade`	Buy	trade price > stop price	trade price < limit price	min(trade price, limit price)
`Tick`	`Trade`	Sell	trade price < stop price	trade price > limit price	max(trade price, limit price)
`QuoteBar`		Buy	ask high price > stop price	ask close price < limit price	min(ask high price, limit price)
`QuoteBar`		Sell	bid low price < stop price	bid close price > limit price	max(bid low price, limit price)
`TradeBar`		Buy	high price > stop price	close price < limit price	min(high price, limit price)
`TradeBar`		Sell	low price < stop price	close price > limit price	max(low price, limit price)

The model won't fill stop limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

The model only fills stop limit orders when the exchange is open.

Trailing Stop Orders

To fill trailing stop orders, the model first gets the best effort `TradeBar` . To get the best effort `TradeBar` , the model checks the resolution of your security subscription. If your subscription provies `Tick` data, it gets the most recent batch of trade ticks and consolidates them to build a `TradeBar` . If your subscription provies `TradeBar` data, it gets the most recent `TradeBar` . If the `end_time` of the best effort `TradeBar` is less than or equal to the time you placed the order, the model waits until the next best effort `TradeBar` to fill the order.

Once the stop condition is met, the model fills the orders and sets the fill price.

Once the model has a valid best effort `TradeBar` , it can fill the order. The following table shows the stop condition and fill price of trailing stop orders. The model only fills the order once the stop condition is met.

Order Direction	Fill Condition	Fill Price
Buy	high price >= stop price	max(open price, stop price) + slippage
Sell	low price <= stop price	min(open price, stop price) - slippage

While the stop condition is not met, the model updates the stop price under certain conditions. The following table shows the update condition and stop price value for currency-based trailing amounts:

Order Direction	Update Condition	Stop Price
Buy	stop price - low price <= trailing amount	low price + trailing amount
Sell	high price - stop price <= trailing amount	high price - trailing amount

The following table shows the update condition and stop price value for percentage-based trailing amounts:

Order Direction	Update Condition	Stop Price
Buy	stop price - low price <= low price * trailing amount	low price * (1 + trailing amount)
Sell	high price - stop price <= high price * trailing amount	high price * (1 - trailing amount)

The model only fills trailing stop orders during regular trading hours.

The model won't fill trailing stop orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Open Orders

The following table describes the fill price of market on open orders for each data format and order direction:

Data Format	Order Direction	Fill Price
`Tick`	Buy	If the model receives the official opening auction price within one minute, the order fills at official open price + slippage. After one minute, the order fills at the most recent trade price + slippage. If the security doesn't trade within the first two minutes, the order fills at the best effort ask price + slippage.
`Tick`	Sell	If the model receives the official opening auction price within one minute, the order fills at the official open price - slippage. After one minute, the order fills at the most recent trade price - slippage. If the security doesn't trade within the first two minutes, the order fills at the best effort bid price - slippage.
`TradeBar`	Buy	Open price + slippage
`TradeBar`	Sell	Open price - slippage
`QuoteBar`	Buy	Best effort ask price + slippage
`QuoteBar`	Sell	Best effort bid price - slippage

The model checks the data format in the following order:

1. `Tick`
2. `TradeBar`
3. `QuoteBar`

To get the best effort bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the best effort ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the best effort bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Market on Close Orders

The following table describes the fill price of market on close orders for each data format and order direction:

Data Format	Order Direction	Fill Price
`Tick`	Buy	If the model receives the official closing auction price within one minute after the close, the order fills at official close price + slippage. After one minute, the order fills at the most recent trade price + slippage. If the security doesn't trade within the first two minutes, the order fills at the best effort ask price + slippage.
`Tick`	Sell	If the model receives the official closing auction price within one minute after the close, the order fills at the official close price - slippage. After one minute, the order fills at the most recent trade price - slippage. If the security doesn't trade within the first two minutes after the close, the order fills at the best effort bid price - slippage.
`TradeBar`	Buy	Open price + slippage
`TradeBar`	Sell	Open price - slippage
`QuoteBar`	Buy	Best effort ask price + slippage
`QuoteBar`	Sell	Best effort bid price - slippage

The model checks the data format in the following order:

1. `Tick`
2. `TradeBar`
3. `QuoteBar`

To get the best effort bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the best effort ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the best effort bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Combo Market Orders

The fill logic of combo market orders depends on the data format of the security subscription and the order direction. The following table shows the fill price of combo market orders given these factors. To determine the fill price of the order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	Ask quote price + slippage
Tick	Quote	Sell	Bid quote price - slippage
Tick	Trade	Buy	Trade price + slippage
Tick	Trade	Sell	Trade price - slippage
QuoteBar		Buy	Ask close price + slippage
QuoteBar		Sell	Bid close price - slippage
TradeBar		Buy	Close price + slippage
TradeBar		Sell	Close price - slippage

The model only fills combo market orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity of each leg is the product of the leg order quantity and the combo market order quantity.

Combo Limit Orders

The fill logic of combo limit orders depends on the data format of the security subscription and the order direction. To determine the fill price of the order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

To fill combo limit orders, the fill model calculates the aggregate price of the combo order, which is the sum of prices for each security in the order legs. The price of each security is a function of the data format and order direction. Legs with a positive order quantity increase the aggregate price and legs with a negative quantity decrease the aggregate price. The following table shows how the fill model calculates the security prices.

Data Format	TickType	Combo Order Direction	Leg Order Direction	Price
Tick	Quote	Buy or sell	Buy	Ask price
Tick	Quote	Buy or sell	Sell	Bid price
Tick	Trade	Buy or sell	Buy or sell	Trade price
QuoteBar		Buy	Buy	Ask low price
QuoteBar		Buy	Sell	Bid low price
QuoteBar		Sell	Buy	Ask high price
QuoteBar		Sell	Sell	Bid high price
TradeBar		Buy	Buy or sell	Low price
TradeBar		Sell	Buy or sell	High price

After the fill model calculates the aggregate price of the combo order, it checks if it should fill the order. The following table describes the fill condition of the combo order and the fill price price of each leg:

Data Format	TickType	Combo Order Direction	Fill Condition	Leg Order Direction	Fill Price
Tick	Quote	Buy	Aggregate price < combo limit price	Buy or sell	Quote price
Tick	Quote	Sell	Aggregate price > combo limit price	Buy or sell	Quote price
Tick	Trade	Buy	Aggregate price < combo limit price	Buy or sell	Trade price
Tick	Trade	Sell	Aggregate price > combo limit price	Buy or sell	Trade price
QuoteBar		Buy	Aggregate price < combo limit price	Buy	Ask low price
QuoteBar		Buy	Aggregate price < combo limit price	Sell	Bid low price
QuoteBar		Sell	Aggregate price > combo limit price	Buy	Ask high price
QuoteBar		Sell	Aggregate price > combo limit price	Sell	Bid high price
TradeBar		Buy	Aggregate price < combo limit price	Buy or sell	Low price
TradeBar		Sell	Aggregate price > combo limit price	Buy or sell	High price

The model only fills combo limit orders if the data isn't stale and all the legs can fill in the same time step after the order time step. The fill quantity of each leg is the product of the leg order quantity and the combo order quantity.

Combo Leg Limit Orders

The fill logic of combo leg limit orders depends on the data format of the security subscription and the order direction. The following table shows the fill price of combo leg limit orders given these factors. To determine the fill price of the order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

The order direction in the table represents the order direction of the order leg, not the order direction of the combo order.

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	Ask price < limit price	min(ask price, limit price)
Tick	Quote	Sell	Bid price > limit price	max(bid price, limit price)
Tick	Trade	Buy	Trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	Trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	Ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	Bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	Low price < limit price	min(high price, limit price)
TradeBar		Sell	High price > limit price	max(low price, limit price)

The model only fills combo leg limit orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity is the product of the leg order quantity and the combo order quantity.

Supported Models

Future Model

Introduction

The `FutureFillModel` is the default fill model if you trade Futures contracts with the DefaultBrokerageModel . This fill model fills trades completely and immediately.

```
security.set_fill_model(FutureFillModel())
```

PY

The fill logic of each order depends on the order type, the data format of the security subscription, and the order direction. The following tables show the fill price of each order given these factors. To determine the fill price of an order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

For more information about this model, see the class reference and implementation .

Market Orders

The following table describes the fill price of market orders for each data format and order direction:

Data Format	`TickType`	Order Direction	Fill Price
`Tick`	`Quote`	Buy	quote price + slippage
`Tick`	`Quote`	Sell	quote price - slippage
`Tick`	`Trade`	Buy	trade price + slippage
`Tick`	`Trade`	Sell	trade price - slippage
`QuoteBar`		Buy	ask close price + slippage
`QuoteBar`		Sell	bid close price - slippage
`TradeBar`		Buy	close price + slippage
`TradeBar`		Sell	close price - slippage

The model only fills market orders if the exchange is open and it immediately fills them. If your algorithm places a market order at 10 AM, it fills at 10 AM.

Limit Orders

The following table describes the fill logic of limit orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	low price < limit price	min(high price, limit price)
TradeBar		Sell	high price > limit price	max(low price, limit price)

The model won't fill limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Limit if Touched Orders

The model converts a limit if touched order to a limit order when the trigger condition is met. The following table describes the trigger condition of limit if touched orders for each data format and order direction:

Data Format	TickType	Order Direction	Trigger Condition
Tick	Quote	Buy	quote price <= trigger price
Tick	Quote	Sell	quote price >= trigger price
Tick	Trade	Buy	trade price <= trigger price
Tick	Trade	Sell	trade price >= trigger price
TradeBar		Buy	low price <= trigger price
TradeBar		Sell	high price >= trigger price

Once the limit if touched order triggers, to fill the order, the model checks the bid price for buy orders and the ask price for sell orders.

To get the bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Buy orders fill when the bid price <= limit price and sell orders fill when the ask price >= limit price. The order fills at the limit price. The model won't trigger or fill limit if touched orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Market Orders

The following table describes the fill logic of stop market orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

The model only fills stop market orders when the exchange is open.

The model won't fill stop market orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Limit Orders

The following table describes the fill logic of stop limit orders for each data format and order direction. Once the stop condition is met, the model starts to check the fill condition. Once the fill condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price < stop price	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price > stop price	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price < stop price	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask high price > stop price	ask close price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid low price < stop price	bid close price > limit price	max(bid low price, limit price)
TradeBar		Buy	high price > stop price	close price < limit price	min(high price, limit price)
TradeBar		Sell	low price < stop price	close price > limit price	max(low price, limit price)

The model won't fill stop limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Trailing Stop Orders

The following table describes the fill logic of trailing stop orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

While the stop condition is not met, the model updates the stop price under certain conditions. The following table shows the update condition and stop price value for currency-based trailing amounts:

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= trailing amount	quote price + trailing amount
Tick	Quote	Sell	quote price - stop price <= trailing amount	quote price - trailing amount
Tick	Trade	Buy	stop price - trade price <= trailing amount	trade price + trailing amount
Tick	Trade	Sell	trade price - stop price <= trailing amount	trade price - trailing amount
QuoteBar		Buy	stop price - ask low price <= trailing amount	ask low price + trailing amount
QuoteBar		Sell	bid high price - stop price <= trailing amount	bid high price - trailing amount
TradeBar		Buy	stop price - low price <= trailing amount	low price + trailing amount
TradeBar		Sell	high price - stop price <= trailing amount	high price - trailing amount

The following table shows the update condition and stop price value for percentage-based trailing amounts.

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= quote price * trailing amount	quote price * (1 + trailing amount)
Tick	Quote	Sell	quote price - stop price <= quote price * trailing amount	quote price * (1 - trailing amount)
Tick	Trade	Buy	stop price - trade price <= trade price * trailing amount	trade price * (1 + trailing amount)
Tick	Trade	Sell	trade price - stop price <= trade price * trailing amount	trade price * (1 - trailing amount)
QuoteBar		Buy	stop price - ask low price <= ask low price * trailing amount	ask low price * (1 + trailing amount)
QuoteBar		Sell	bid high price - stop price <= bid high price * trailing amount	bid high price * (1 - trailing amount)
TradeBar		Buy	stop price - low price <= stop price * trailing amount	low price * (1 + trailing amount)
TradeBar		Sell	high price - stop price <= high price * trailing amount	high price * (1 - trailing amount)

The model only fills trailing stop orders when the exchange is open.

The model won't fill trailing stop orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Open Orders

The following table describes the fill price of market on open orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask open price + slippage
QuoteBar		Sell	bid open price - slippage
TradeBar		Buy	open price + slippage
TradeBar		Sell	open price - slippage

The model won't fill market on open orders during pre-market hours.

The model won't fill market on open orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Close Orders

The following table describes the fill price of market on close orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask close price + slippage
QuoteBar		Sell	bid close price - slippage
TradeBar		Buy	close price + slippage
TradeBar		Sell	close price - slippage

The model won't fill market on close orders with stale data or data with the order timestamp to avoid look-ahead bias.

Combo Market Orders

The following table describes the fill price of combo market orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	Ask quote price + slippage
Tick	Quote	Sell	Bid quote price - slippage
Tick	Trade	Buy	Trade price + slippage
Tick	Trade	Sell	Trade price - slippage
QuoteBar		Buy	Ask close price + slippage
QuoteBar		Sell	Bid close price - slippage
TradeBar		Buy	Close price + slippage
TradeBar		Sell	Close price - slippage

The model only fills combo market orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity of each leg is the product of the leg order quantity and the combo market order quantity.

Combo Limit Orders

To fill combo limit orders, the fill model calculates the aggregate price of the combo order, which is the sum of prices for each security in the order legs. The price of each security is a function of the data format and order direction. Legs with a positive order quantity increase the aggregate price and legs with a negative quantity decrease the aggregate price. The following table shows how the fill model calculates the security prices.

Data Format	TickType	Combo Order Direction	Leg Order Direction	Price
Tick	Quote	Buy or sell	Buy	Ask price
Tick	Quote	Buy or sell	Sell	Bid price
Tick	Trade	Buy or sell	Buy or sell	Trade price
QuoteBar		Buy	Buy	Ask low price
QuoteBar		Buy	Sell	Bid low price
QuoteBar		Sell	Buy	Ask high price
QuoteBar		Sell	Sell	Bid high price
TradeBar		Buy	Buy or sell	Low price
TradeBar		Sell	Buy or sell	High price

After the fill model calculates the aggregate price of the combo order, it checks if it should fill the order. The following table describes the fill condition of the combo order and the fill price price of each leg:

Data Format	TickType	Combo Order Direction	Fill Condition	Leg Order Direction	Fill Price
Tick	Quote	Buy	Aggregate price < combo limit price	Buy or sell	Quote price
Tick	Quote	Sell	Aggregate price > combo limit price	Buy or sell	Quote price
Tick	Trade	Buy	Aggregate price < combo limit price	Buy or sell	Trade price
Tick	Trade	Sell	Aggregate price > combo limit price	Buy or sell	Trade price
QuoteBar		Buy	Aggregate price < combo limit price	Buy	Ask low price
QuoteBar		Buy	Aggregate price < combo limit price	Sell	Bid low price
QuoteBar		Sell	Aggregate price > combo limit price	Buy	Ask high price
QuoteBar		Sell	Aggregate price > combo limit price	Sell	Bid high price
TradeBar		Buy	Aggregate price < combo limit price	Buy or sell	Low price
TradeBar		Sell	Aggregate price > combo limit price	Buy or sell	High price

The model only fills combo limit orders if the data isn't stale and all the legs can fill in the same time step after the order time step. The fill quantity of each leg is the product of the leg order quantity and the combo order quantity.

Combo Leg Limit Orders

The following table describes the fill logic of combo leg limit orders for each data format and order direction. The order direction in the table represents the order direction of the order leg, not the order direction of the combo order.

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	Ask price < limit price	min(ask price, limit price)
Tick	Quote	Sell	Bid price > limit price	max(bid price, limit price)
Tick	Trade	Buy	Trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	Trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	Ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	Bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	Low price < limit price	min(high price, limit price)
TradeBar		Sell	High price > limit price	max(low price, limit price)

The model only fills combo leg limit orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity is the product of the leg order quantity and the combo order quantity.

Supported Models

Future Option Model

Introduction

The `FutureOptionFillModel` is the default fill model if you trade Future Option contracts with the DefaultBrokerageModel . This fill model fills trades completely and immediately.

```
security.set_fill_model(FutureOptionFillModel())
```

PY

The fill logic of each order depends on the order type, the data format of the security subscription, and the order direction. The following tables show the fill price of each order given these factors. To determine the fill price of an order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

For more information about this model, see the class reference and implementation .

Market Orders

The following table describes the fill price of market orders for each data format and order direction:

Data Format	`TickType`	Order Direction	Fill Price
`Tick`	`Quote`	Buy	quote price + slippage
`Tick`	`Quote`	Sell	quote price - slippage
`Tick`	`Trade`	Buy	trade price + slippage
`Tick`	`Trade`	Sell	trade price - slippage
`QuoteBar`		Buy	ask close price + slippage
`QuoteBar`		Sell	bid close price - slippage
`TradeBar`		Buy	close price + slippage
`TradeBar`		Sell	close price - slippage

The model only fills market orders if the exchange is open and it immediately fills them. If your algorithm places a market order at 10 AM, it fills at 10 AM.

Limit Orders

The following table describes the fill logic of limit orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	low price < limit price	min(high price, limit price)
TradeBar		Sell	high price > limit price	max(low price, limit price)

The model won't fill limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Limit if Touched Orders

The model converts a limit if touched order to a limit order when the trigger condition is met. The following table describes the trigger condition of limit if touched orders for each data format and order direction:

Data Format	TickType	Order Direction	Trigger Condition
Tick	Quote	Buy	quote price <= trigger price
Tick	Quote	Sell	quote price >= trigger price
Tick	Trade	Buy	trade price <= trigger price
Tick	Trade	Sell	trade price >= trigger price
TradeBar		Buy	low price <= trigger price
TradeBar		Sell	high price >= trigger price

Once the limit if touched order triggers, to fill the order, the model checks the bid price for buy orders and the ask price for sell orders.

To get the bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Buy orders fill when the bid price <= limit price and sell orders fill when the ask price >= limit price. The order fills at the limit price. The model won't trigger or fill limit if touched orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Market Orders

The following table describes the fill logic of stop market orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

The model only fills stop market orders when the exchange is open.

The model won't fill stop market orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Limit Orders

The following table describes the fill logic of stop limit orders for each data format and order direction. Once the stop condition is met, the model starts to check the fill condition. Once the fill condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price < stop price	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price > stop price	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price < stop price	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask high price > stop price	ask close price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid low price < stop price	bid close price > limit price	max(bid low price, limit price)
TradeBar		Buy	high price > stop price	close price < limit price	min(high price, limit price)
TradeBar		Sell	low price < stop price	close price > limit price	max(low price, limit price)

The model won't fill stop limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Trailing Stop Orders

The following table describes the fill logic of trailing stop orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

While the stop condition is not met, the model updates the stop price under certain conditions. The following table shows the update condition and stop price value for currency-based trailing amounts:

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= trailing amount	quote price + trailing amount
Tick	Quote	Sell	quote price - stop price <= trailing amount	quote price - trailing amount
Tick	Trade	Buy	stop price - trade price <= trailing amount	trade price + trailing amount
Tick	Trade	Sell	trade price - stop price <= trailing amount	trade price - trailing amount
QuoteBar		Buy	stop price - ask low price <= trailing amount	ask low price + trailing amount
QuoteBar		Sell	bid high price - stop price <= trailing amount	bid high price - trailing amount
TradeBar		Buy	stop price - low price <= trailing amount	low price + trailing amount
TradeBar		Sell	high price - stop price <= trailing amount	high price - trailing amount

The following table shows the update condition and stop price value for percentage-based trailing amounts.

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= quote price * trailing amount	quote price * (1 + trailing amount)
Tick	Quote	Sell	quote price - stop price <= quote price * trailing amount	quote price * (1 - trailing amount)
Tick	Trade	Buy	stop price - trade price <= trade price * trailing amount	trade price * (1 + trailing amount)
Tick	Trade	Sell	trade price - stop price <= trade price * trailing amount	trade price * (1 - trailing amount)
QuoteBar		Buy	stop price - ask low price <= ask low price * trailing amount	ask low price * (1 + trailing amount)
QuoteBar		Sell	bid high price - stop price <= bid high price * trailing amount	bid high price * (1 - trailing amount)
TradeBar		Buy	stop price - low price <= stop price * trailing amount	low price * (1 + trailing amount)
TradeBar		Sell	high price - stop price <= high price * trailing amount	high price * (1 - trailing amount)

The model only fills trailing stop orders when the exchange is open.

The model won't fill trailing stop orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Open Orders

The following table describes the fill price of market on open orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask open price + slippage
QuoteBar		Sell	bid open price - slippage
TradeBar		Buy	open price + slippage
TradeBar		Sell	open price - slippage

The model won't fill market on open orders during pre-market hours.

The model won't fill market on open orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Close Orders

The following table describes the fill price of market on close orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask close price + slippage
QuoteBar		Sell	bid close price - slippage
TradeBar		Buy	close price + slippage
TradeBar		Sell	close price - slippage

The model won't fill market on close orders with stale data or data with the order timestamp to avoid look-ahead bias.

Combo Market Orders

The following table describes the fill price of combo market orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	Ask quote price + slippage
Tick	Quote	Sell	Bid quote price - slippage
Tick	Trade	Buy	Trade price + slippage
Tick	Trade	Sell	Trade price - slippage
QuoteBar		Buy	Ask close price + slippage
QuoteBar		Sell	Bid close price - slippage
TradeBar		Buy	Close price + slippage
TradeBar		Sell	Close price - slippage

The model only fills combo market orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity of each leg is the product of the leg order quantity and the combo market order quantity.

Combo Limit Orders

To fill combo limit orders, the fill model calculates the aggregate price of the combo order, which is the sum of prices for each security in the order legs. The price of each security is a function of the data format and order direction. Legs with a positive order quantity increase the aggregate price and legs with a negative quantity decrease the aggregate price. The following table shows how the fill model calculates the security prices.

Data Format	TickType	Combo Order Direction	Leg Order Direction	Price
Tick	Quote	Buy or sell	Buy	Ask price
Tick	Quote	Buy or sell	Sell	Bid price
Tick	Trade	Buy or sell	Buy or sell	Trade price
QuoteBar		Buy	Buy	Ask low price
QuoteBar		Buy	Sell	Bid low price
QuoteBar		Sell	Buy	Ask high price
QuoteBar		Sell	Sell	Bid high price
TradeBar		Buy	Buy or sell	Low price
TradeBar		Sell	Buy or sell	High price

After the fill model calculates the aggregate price of the combo order, it checks if it should fill the order. The following table describes the fill condition of the combo order and the fill price price of each leg:

Data Format	TickType	Combo Order Direction	Fill Condition	Leg Order Direction	Fill Price
Tick	Quote	Buy	Aggregate price < combo limit price	Buy or sell	Quote price
Tick	Quote	Sell	Aggregate price > combo limit price	Buy or sell	Quote price
Tick	Trade	Buy	Aggregate price < combo limit price	Buy or sell	Trade price
Tick	Trade	Sell	Aggregate price > combo limit price	Buy or sell	Trade price
QuoteBar		Buy	Aggregate price < combo limit price	Buy	Ask low price
QuoteBar		Buy	Aggregate price < combo limit price	Sell	Bid low price
QuoteBar		Sell	Aggregate price > combo limit price	Buy	Ask high price
QuoteBar		Sell	Aggregate price > combo limit price	Sell	Bid high price
TradeBar		Buy	Aggregate price < combo limit price	Buy or sell	Low price
TradeBar		Sell	Aggregate price > combo limit price	Buy or sell	High price

The model only fills combo limit orders if the data isn't stale and all the legs can fill in the same time step after the order time step. The fill quantity of each leg is the product of the leg order quantity and the combo order quantity.

Combo Leg Limit Orders

The following table describes the fill logic of combo leg limit orders for each data format and order direction. The order direction in the table represents the order direction of the order leg, not the order direction of the combo order.

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	Ask price < limit price	min(ask price, limit price)
Tick	Quote	Sell	Bid price > limit price	max(bid price, limit price)
Tick	Trade	Buy	Trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	Trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	Ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	Bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	Low price < limit price	min(high price, limit price)
TradeBar		Sell	High price > limit price	max(low price, limit price)

The model only fills combo leg limit orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity is the product of the leg order quantity and the combo order quantity.

Supported Models

Immediate Model

Introduction

The `ImmediateFillModel` is the default fill model if you trade non-Equity assets with the DefaultBrokerageModel . This fill model fills trades completely and immediately.

```
security.set_fill_model(ImmediateFillModel())
```

PY

The fill logic of each order depends on the order type, the data format of the security subscription, and the order direction. The following tables show the fill price of each order given these factors. To determine the fill price of an order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If your security subscription doesn't provide quote data, the fill model checks the most recent `TradeBar` .

For more information about this model, see the class reference and implementation .

Market Orders

The following table describes the fill price of market orders for each data format and order direction:

Data Format	`TickType`	Order Direction	Fill Price
`Tick`	`Quote`	Buy	quote price + slippage
`Tick`	`Quote`	Sell	quote price - slippage
`Tick`	`Trade`	Buy	trade price + slippage
`Tick`	`Trade`	Sell	trade price - slippage
`QuoteBar`		Buy	ask close price + slippage
`QuoteBar`		Sell	bid close price - slippage
`TradeBar`		Buy	close price + slippage
`TradeBar`		Sell	close price - slippage

The model only fills market orders if the exchange is open and it immediately fills them. If your algorithm places a market order at 10 AM, it fills at 10 AM.

Limit Orders

The following table describes the fill logic of limit orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	low price < limit price	min(high price, limit price)
TradeBar		Sell	high price > limit price	max(low price, limit price)

The model won't fill limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Limit if Touched Orders

The model converts a limit if touched order to a limit order when the trigger condition is met. The following table describes the trigger condition of limit if touched orders for each data format and order direction:

Data Format	TickType	Order Direction	Trigger Condition
Tick	Quote	Buy	quote price <= trigger price
Tick	Quote	Sell	quote price >= trigger price
Tick	Trade	Buy	trade price <= trigger price
Tick	Trade	Sell	trade price >= trigger price
TradeBar		Buy	low price <= trigger price
TradeBar		Sell	high price >= trigger price

Once the limit if touched order triggers, to fill the order, the model checks the bid price for buy orders and the ask price for sell orders.

To get the bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Buy orders fill when the bid price <= limit price and sell orders fill when the ask price >= limit price. The order fills at the limit price. The model won't trigger or fill limit if touched orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Market Orders

The following table describes the fill logic of stop market orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

The model only fills stop market orders when the exchange is open.

The model won't fill stop market orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Limit Orders

The following table describes the fill logic of stop limit orders for each data format and order direction. Once the stop condition is met, the model starts to check the fill condition. Once the fill condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price < stop price	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price > stop price	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price < stop price	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask high price > stop price	ask close price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid low price < stop price	bid close price > limit price	max(bid low price, limit price)
TradeBar		Buy	high price > stop price	close price < limit price	min(high price, limit price)
TradeBar		Sell	low price < stop price	close price > limit price	max(low price, limit price)

The model won't fill stop limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Trailing Stop Orders

The following table describes the fill logic of trailing stop orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

While the stop condition is not met, the model updates the stop price under certain conditions. The following table shows the update condition and stop price value for currency-based trailing amounts:

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= trailing amount	quote price + trailing amount
Tick	Quote	Sell	quote price - stop price <= trailing amount	quote price - trailing amount
Tick	Trade	Buy	stop price - trade price <= trailing amount	trade price + trailing amount
Tick	Trade	Sell	trade price - stop price <= trailing amount	trade price - trailing amount
QuoteBar		Buy	stop price - ask low price <= trailing amount	ask low price + trailing amount
QuoteBar		Sell	bid high price - stop price <= trailing amount	bid high price - trailing amount
TradeBar		Buy	stop price - low price <= trailing amount	low price + trailing amount
TradeBar		Sell	high price - stop price <= trailing amount	high price - trailing amount

The following table shows the update condition and stop price value for percentage-based trailing amounts.

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= quote price * trailing amount	quote price * (1 + trailing amount)
Tick	Quote	Sell	quote price - stop price <= quote price * trailing amount	quote price * (1 - trailing amount)
Tick	Trade	Buy	stop price - trade price <= trade price * trailing amount	trade price * (1 + trailing amount)
Tick	Trade	Sell	trade price - stop price <= trade price * trailing amount	trade price * (1 - trailing amount)
QuoteBar		Buy	stop price - ask low price <= ask low price * trailing amount	ask low price * (1 + trailing amount)
QuoteBar		Sell	bid high price - stop price <= bid high price * trailing amount	bid high price * (1 - trailing amount)
TradeBar		Buy	stop price - low price <= stop price * trailing amount	low price * (1 + trailing amount)
TradeBar		Sell	high price - stop price <= high price * trailing amount	high price * (1 - trailing amount)

The model only fills trailing stop orders when the exchange is open.

The model won't fill trailing stop orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Open Orders

The following table describes the fill price of market on open orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask open price + slippage
QuoteBar		Sell	bid open price - slippage
TradeBar		Buy	open price + slippage
TradeBar		Sell	open price - slippage

The model won't fill market on open orders during pre-market hours.

The model won't fill market on open orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Close Orders

The following table describes the fill price of market on close orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask close price + slippage
QuoteBar		Sell	bid close price - slippage
TradeBar		Buy	close price + slippage
TradeBar		Sell	close price - slippage

The model won't fill market on close orders with stale data or data with the order timestamp to avoid look-ahead bias.

Combo Market Orders

The following table describes the fill price of combo market orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	Ask quote price + slippage
Tick	Quote	Sell	Bid quote price - slippage
Tick	Trade	Buy	Trade price + slippage
Tick	Trade	Sell	Trade price - slippage
QuoteBar		Buy	Ask close price + slippage
QuoteBar		Sell	Bid close price - slippage
TradeBar		Buy	Close price + slippage
TradeBar		Sell	Close price - slippage

The model only fills combo market orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity of each leg is the product of the leg order quantity and the combo market order quantity.

Combo Limit Orders

To fill combo limit orders, the fill model calculates the aggregate price of the combo order, which is the sum of prices for each security in the order legs. The price of each security is a function of the data format and order direction. Legs with a positive order quantity increase the aggregate price and legs with a negative quantity decrease the aggregate price. The following table shows how the fill model calculates the security prices.

Data Format	TickType	Combo Order Direction	Leg Order Direction	Price
Tick	Quote	Buy or sell	Buy	Ask price
Tick	Quote	Buy or sell	Sell	Bid price
Tick	Trade	Buy or sell	Buy or sell	Trade price
QuoteBar		Buy	Buy	Ask low price
QuoteBar		Buy	Sell	Bid low price
QuoteBar		Sell	Buy	Ask high price
QuoteBar		Sell	Sell	Bid high price
TradeBar		Buy	Buy or sell	Low price
TradeBar		Sell	Buy or sell	High price

After the fill model calculates the aggregate price of the combo order, it checks if it should fill the order. The following table describes the fill condition of the combo order and the fill price price of each leg:

Data Format	TickType	Combo Order Direction	Fill Condition	Leg Order Direction	Fill Price
Tick	Quote	Buy	Aggregate price < combo limit price	Buy or sell	Quote price
Tick	Quote	Sell	Aggregate price > combo limit price	Buy or sell	Quote price
Tick	Trade	Buy	Aggregate price < combo limit price	Buy or sell	Trade price
Tick	Trade	Sell	Aggregate price > combo limit price	Buy or sell	Trade price
QuoteBar		Buy	Aggregate price < combo limit price	Buy	Ask low price
QuoteBar		Buy	Aggregate price < combo limit price	Sell	Bid low price
QuoteBar		Sell	Aggregate price > combo limit price	Buy	Ask high price
QuoteBar		Sell	Aggregate price > combo limit price	Sell	Bid high price
TradeBar		Buy	Aggregate price < combo limit price	Buy or sell	Low price
TradeBar		Sell	Aggregate price > combo limit price	Buy or sell	High price

The model only fills combo limit orders if the data isn't stale and all the legs can fill in the same time step after the order time step. The fill quantity of each leg is the product of the leg order quantity and the combo order quantity.

Combo Leg Limit Orders

The following table describes the fill logic of combo leg limit orders for each data format and order direction. The order direction in the table represents the order direction of the order leg, not the order direction of the combo order.

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	Ask price < limit price	min(ask price, limit price)
Tick	Quote	Sell	Bid price > limit price	max(bid price, limit price)
Tick	Trade	Buy	Trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	Trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	Ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	Bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	Low price < limit price	min(high price, limit price)
TradeBar		Sell	High price > limit price	max(low price, limit price)

The model only fills combo leg limit orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity is the product of the leg order quantity and the combo order quantity.

Supported Models

Latest Price Model

Introduction

The `LatestPriceFillModel` fills trades completely and immediately.

```
security.set_fill_model(LatestPriceFillModel())
```
PY

The fill logic of each order depends on the order type, the data format of the security subscription, and the order direction. The following tables show the fill price of each order given these factors. To determine the fill price of an order, the fill model first checks the most recent tick for the security. If your security subscription doesn't provide tick data, the fill model checks the most recent `QuoteBar` . If there is no `QuoteBar` available, it uses the OHLC prices from the `Security` object. If there is a `QuoteBar` available, the fill model gets the most recent `TradeBar` . If the `TradeBar` is more recent, it uses the `TradeBar` data. Otherwise, it uses the `QuoteBar` data.

For more information about this model, see the class reference and implementation .

Market Orders

The following table describes the fill price of market orders for each data format and order direction:

Data Format	`TickType`	Order Direction	Fill Price
`Tick`	`Quote`	Buy	quote price + slippage
`Tick`	`Quote`	Sell	quote price - slippage
`Tick`	`Trade`	Buy	trade price + slippage
`Tick`	`Trade`	Sell	trade price - slippage
`QuoteBar`		Buy	ask close price + slippage
`QuoteBar`		Sell	bid close price - slippage
`TradeBar`		Buy	close price + slippage
`TradeBar`		Sell	close price - slippage

The model only fills market orders if the exchange is open and it immediately fills them. If your algorithm places a market order at 10 AM, it fills at 10 AM.

Limit Orders

The following table describes the fill logic of limit orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	low price < limit price	min(high price, limit price)
TradeBar		Sell	high price > limit price	max(low price, limit price)

The model won't fill limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Limit if Touched Orders

The model converts a limit if touched order to a limit order when the trigger condition is met. The following table describes the trigger condition of limit if touched orders for each data format and order direction:

Data Format	TickType	Order Direction	Trigger Condition
Tick	Quote	Buy	quote price <= trigger price
Tick	Quote	Sell	quote price >= trigger price
Tick	Trade	Buy	trade price <= trigger price
Tick	Trade	Sell	trade price >= trigger price
TradeBar		Buy	low price <= trigger price
TradeBar		Sell	high price >= trigger price

Once the limit if touched order triggers, to fill the order, the model checks the bid price for buy orders and the ask price for sell orders.

To get the bid price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a buy quote, use the bid price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing bid price of the most recent `QuoteBar` .

To get the ask price, the model uses the following procedure:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a sell quote, use the ask price of the most recent quote tick.
2. If the subscription provides `QuoteBar` data, use the closing ask price of the most recent `QuoteBar` .

If neither of the preceding procedures yield a result, the model uses the following procedure to get the bid or ask price:

1. If the subscription provides `Tick` data and the most recent batch of ticks contains a tick of type `TickType.Trade` , use the last trade price.
2. If the subscription provides `TradeBar` data, use the closing bid price of the most recent `QuoteBar` .

Buy orders fill when the bid price <= limit price and sell orders fill when the ask price >= limit price. The order fills at the limit price. The model won't trigger or fill limit if touched orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Market Orders

The following table describes the fill logic of stop market orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

The model only fills stop market orders when the exchange is open.

The model won't fill stop market orders with stale data or data with the order timestamp to avoid look-ahead bias.

Stop Limit Orders

The following table describes the fill logic of stop limit orders for each data format and order direction. Once the stop condition is met, the model starts to check the fill condition. Once the fill condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	quote price < limit price	min(quote price, limit price)
Tick	Quote	Sell	quote price < stop price	quote price > limit price	max(quote price, limit price)
Tick	Trade	Buy	trade price > stop price	trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	trade price < stop price	trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	ask high price > stop price	ask close price < limit price	min(ask high price, limit price)
QuoteBar		Sell	bid low price < stop price	bid close price > limit price	max(bid low price, limit price)
TradeBar		Buy	high price > stop price	close price < limit price	min(high price, limit price)
TradeBar		Sell	low price < stop price	close price > limit price	max(low price, limit price)

The model won't fill stop limit orders with stale data or data with the order timestamp to avoid look-ahead bias.

Trailing Stop Orders

The following table describes the fill logic of trailing stop orders for each data format and order direction. Once the stop condition is met, the model fills the orders and sets the fill price.

Data Format	TickType	Order Direction	Stop Condition	Fill Price
Tick	Quote	Buy	quote price > stop price	max(stop price, quote price + slippage)
Tick	Quote	Sell	quote price < stop price	min(stop price, quote price - slippage)
Tick	Trade	Buy	trade price > stop price	max(stop price, last trade price + slippage)
Tick	Trade	Sell	trade price < stop price	min(stop price, last trade price - slippage)
QuoteBar		Buy	ask high price > stop price	max(stop price, ask close price + slippage)
QuoteBar		Sell	bid low price < stop price	min(stop price, bid close price - slippage)
TradeBar		Buy	high price > stop price	max(stop price, close price + slippage)
TradeBar		Sell	low price < stop price	min(stop price, close price - slippage)

While the stop condition is not met, the model updates the stop price under certain conditions. The following table shows the update condition and stop price value for currency-based trailing amounts:

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= trailing amount	quote price + trailing amount
Tick	Quote	Sell	quote price - stop price <= trailing amount	quote price - trailing amount
Tick	Trade	Buy	stop price - trade price <= trailing amount	trade price + trailing amount
Tick	Trade	Sell	trade price - stop price <= trailing amount	trade price - trailing amount
QuoteBar		Buy	stop price - ask low price <= trailing amount	ask low price + trailing amount
QuoteBar		Sell	bid high price - stop price <= trailing amount	bid high price - trailing amount
TradeBar		Buy	stop price - low price <= trailing amount	low price + trailing amount
TradeBar		Sell	high price - stop price <= trailing amount	high price - trailing amount

The following table shows the update condition and stop price value for percentage-based trailing amounts.

Data Format	TickType	Order Direction	Update Condition	Stop Price
Tick	Quote	Buy	stop price - quote price <= quote price * trailing amount	quote price * (1 + trailing amount)
Tick	Quote	Sell	quote price - stop price <= quote price * trailing amount	quote price * (1 - trailing amount)
Tick	Trade	Buy	stop price - trade price <= trade price * trailing amount	trade price * (1 + trailing amount)
Tick	Trade	Sell	trade price - stop price <= trade price * trailing amount	trade price * (1 - trailing amount)
QuoteBar		Buy	stop price - ask low price <= ask low price * trailing amount	ask low price * (1 + trailing amount)
QuoteBar		Sell	bid high price - stop price <= bid high price * trailing amount	bid high price * (1 - trailing amount)
TradeBar		Buy	stop price - low price <= stop price * trailing amount	low price * (1 + trailing amount)
TradeBar		Sell	high price - stop price <= high price * trailing amount	high price * (1 - trailing amount)

The model only fills trailing stop orders when the exchange is open.

The model won't fill trailing stop orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Open Orders

The following table describes the fill price of market on open orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask open price + slippage
QuoteBar		Sell	bid open price - slippage
TradeBar		Buy	open price + slippage
TradeBar		Sell	open price - slippage

The model won't fill market on open orders during pre-market hours.

The model won't fill market on open orders with stale data or data with the order timestamp to avoid look-ahead bias.

Market on Close Orders

The following table describes the fill price of market on close orders for each data format and order side:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	quote price + slippage
Tick	Quote	Sell	quote price - slippage
Tick	Trade	Buy	trade price + slippage
Tick	Trade	Sell	trade price - slippage
QuoteBar		Buy	ask close price + slippage
QuoteBar		Sell	bid close price - slippage
TradeBar		Buy	close price + slippage
TradeBar		Sell	close price - slippage

The model won't fill market on close orders with stale data or data with the order timestamp to avoid look-ahead bias.

Combo Market Orders

The following table describes the fill price of combo market orders for each data format and order direction:

Data Format	TickType	Order Direction	Fill Price
Tick	Quote	Buy	Ask quote price + slippage
Tick	Quote	Sell	Bid quote price - slippage
Tick	Trade	Buy	Trade price + slippage
Tick	Trade	Sell	Trade price - slippage
QuoteBar		Buy	Ask close price + slippage
QuoteBar		Sell	Bid close price - slippage
TradeBar		Buy	Close price + slippage
TradeBar		Sell	Close price - slippage

The model only fills combo market orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity of each leg is the product of the leg order quantity and the combo market order quantity.

Combo Limit Orders

To fill combo limit orders, the fill model calculates the aggregate price of the combo order, which is the sum of prices for each security in the order legs. The price of each security is a function of the data format and order direction. Legs with a positive order quantity increase the aggregate price and legs with a negative quantity decrease the aggregate price. The following table shows how the fill model calculates the security prices.

Data Format	TickType	Combo Order Direction	Leg Order Direction	Price
Tick	Quote	Buy or sell	Buy	Ask price
Tick	Quote	Buy or sell	Sell	Bid price
Tick	Trade	Buy or sell	Buy or sell	Trade price
QuoteBar		Buy	Buy	Ask low price
QuoteBar		Buy	Sell	Bid low price
QuoteBar		Sell	Buy	Ask high price
QuoteBar		Sell	Sell	Bid high price
TradeBar		Buy	Buy or sell	Low price
TradeBar		Sell	Buy or sell	High price

After the fill model calculates the aggregate price of the combo order, it checks if it should fill the order. The following table describes the fill condition of the combo order and the fill price price of each leg:

Data Format	TickType	Combo Order Direction	Fill Condition	Leg Order Direction	Fill Price
Tick	Quote	Buy	Aggregate price < combo limit price	Buy or sell	Quote price
Tick	Quote	Sell	Aggregate price > combo limit price	Buy or sell	Quote price
Tick	Trade	Buy	Aggregate price < combo limit price	Buy or sell	Trade price
Tick	Trade	Sell	Aggregate price > combo limit price	Buy or sell	Trade price
QuoteBar		Buy	Aggregate price < combo limit price	Buy	Ask low price
QuoteBar		Buy	Aggregate price < combo limit price	Sell	Bid low price
QuoteBar		Sell	Aggregate price > combo limit price	Buy	Ask high price
QuoteBar		Sell	Aggregate price > combo limit price	Sell	Bid high price
TradeBar		Buy	Aggregate price < combo limit price	Buy or sell	Low price
TradeBar		Sell	Aggregate price > combo limit price	Buy or sell	High price

The model only fills combo limit orders if the data isn't stale and all the legs can fill in the same time step after the order time step. The fill quantity of each leg is the product of the leg order quantity and the combo order quantity.

Combo Leg Limit Orders

The following table describes the fill logic of combo leg limit orders for each data format and order direction. The order direction in the table represents the order direction of the order leg, not the order direction of the combo order.

Data Format	TickType	Order Direction	Fill Condition	Fill Price
Tick	Quote	Buy	Ask price < limit price	min(ask price, limit price)
Tick	Quote	Sell	Bid price > limit price	max(bid price, limit price)
Tick	Trade	Buy	Trade price < limit price	min(trade price, limit price)
Tick	Trade	Sell	Trade price > limit price	max(trade price, limit price)
QuoteBar		Buy	Ask low price < limit price	min(ask high price, limit price)
QuoteBar		Sell	Bid high price > limit price	max(bid low price, limit price)
TradeBar		Buy	Low price < limit price	min(high price, limit price)
TradeBar		Sell	High price > limit price	max(low price, limit price)

The model only fills combo leg limit orders if all the following conditions are met:

- The exchange is open
- The data isn't stale
- All the legs can fill in the same time step after the order time step

The fill quantity is the product of the leg order quantity and the combo order quantity.

Reality Modeling

Slippage

Slippage

Key Concepts

Introduction

Slippage is the difference between the fill price you expect to get for an order and the actual fill price. Since the price can move in the direction of your trade or against the direction of your trade while you wait for the order to fill, slippage can be positive or negative. Slippage models model slippage to make backtest results more realistic.

Factors Impacting Slippage

There are many factors that can impact slippage, including the trading engine, brokerage connection, and market dynamics.

Trading Engine

How long does it take from when you place an order to when it's sent to the brokerage? Longer delays lead to more slippage.

Brokerage Connection

How long does it take for your brokerage to receive an order that you send? Slow internet connections, long travel distances, and poor infrastructure lead to more slippage

Market Dynamics

How volatile is the current market environment? More volatility leads to more slippage.

Does the market consist of sophisticated microsecond arbitrageurs? If your order creates an arbitrage opportunity, it can cause more slippage.

Set Models

The brokerage model of your algorithm automatically sets the slippage model for each security, but you can override it. To manually set the slippage model of a security, call the `set_slippage_model` method on the `Security` object.

```py
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # Set the slippage model for the requested security to backtest with the most realistic scenario
    # VolumeShareSlippageModel has slippage size affected by the volume of the order compared to the actual filled volume of the bar
    # It is only valid for the security with a volume property, while being more accurate with denser resolution
    security.set_slippage_model(VolumeShareSlippageModel())
```

You can also set the slippage model in a security initializer . If your algorithm has a dynamic universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_slippage_model(VolumeShareSlippageModel())
```

To view all the pre-built slippage models, see Supported Models .

Default Behavior

The brokerage model of your algorithm automatically sets the slippage model of each security. The default brokerage model is the `DefaultBrokerageModel`, which uses the NullSlippageModel to model zero slippage for all securities.

Model Structure

Slippage models should implement the `ISlippageModel` interface. Extensions of the `ISlippageModel` interface must implement the `get_slippage_approximation` method, which calculates the slippage quantity.

```
PY
class CustomSlippageModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the custom slippage model for an added security to use the custom model
        security = self.add_equity("SPY")
        security.set_slippage_model(MySlippageModel())

# Define the custom slippage model
class MySlippageModel:

    def get_slippage_approximation(self, asset: Security, order: Order) -> float:
        return asset.price * 0.0001 * np.log10(2*float(order.absolute_quantity))
```

Examples

The following examples demonstrate some common practices for implementing a custom slippage model.

Example 1: Logarithmic Scaling

The following algorithm defines a custom slippage model where the slippage increases logarithmically with order size, scaled by 0.01% of the asset price, modeling realistic market impact where larger orders have diminishing marginal price impact.

```
PY
class CustomSlippageModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        security = self.add_equity('SPY', Resolution.DAILY)
        security.set_slippage_model(MySlippageModel())

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MySlippageModel:

    def get_slippage_approximation(self, asset: Security, order: Order) -> float:
        return asset.price * 0.0001 * np.log10(2*float(order.absolute_quantity))
```

Example 2: Volume Ratio Capped Slippage Model

The following algorithm trades the microeconomic demand popularity gap between the top 10 and bottom 10 liquid constituents of QQQ. A custom slippage model simulates the slippage on less liquid stock fills. The slippage percent is set to be 2% multiplied by the ratio of the order quantity to the previous minute bar's volume, maximized at 2%.

```py
class RealityModelingSlippageAlgorithm(QCAlgorithm):
    longs = []
    shorts = []

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)

        # Request extended market hour QQQ data for trading.
        qqq = self.add_equity("QQQ").symbol

        # Weekly portfolio updating to allow time to capitalize on the popularity gap.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Add universe to trade on the most and least liquid stocks among QQQ constituents.
        self.add_universe(
            self.universe.etf(qqq, Market.USA, self.universe_settings, lambda constituents: [c.symbol for c in constituents]),
            self.fundamental_selection
        )

        # Set a scheduled event to rebalance the portfolio at the start of every week.
        self.schedule.on(
            self.date_rules.every_day(qqq),
            self.time_rules.after_market_open(qqq),
            self.rebalance
        )

    def fundamental_selection(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        sorted_by_dollar_volume = sorted(fundamentals, key=lambda f: f.dollar_volume)
        # Add the 10 most liquid stocks to the universe to long later.
        self.longs = [f.symbol for f in sorted_by_dollar_volume[-10:]]
        # Add the 10 least liquid stocks to the universe to short later.
        self.shorts = [f.symbol for f in sorted_by_dollar_volume[:10]]

        return self.longs + self.shorts

    def rebalance(self) -> None:
        # Equally invest in the selected stocks to dissipate capital risk evenly.
        # Dollar neutral of long and short stocks to eliminate systematic risk, only capitalize the popularity gap.
        targets = [PortfolioTarget(symbol, 0.05) for symbol in self.longs]
        targets += [PortfolioTarget(symbol, -0.05) for symbol in self.shorts]

        # Liquidate the stocks that are not the most and least popular to release funds for trades with higher expected returns.
        self.set_holdings(targets, liquidate_existing_holdings=True)

    def _custom_security_initializer(self, security):
        # To set the slippage model to the custom one for all stocks entering the universe.
        security.set_slippage_model(CustomSlippageModel())

class CustomSlippageModel:
    def get_slippage_approximation(self, asset: Security, order: Order) -> float:
        # We set a maximum 2% slippage that is linearly linked to the ratio of the order size versus the previous bar's volume.
        return asset.price * 0.02 * min(1, order.absolute_quantity / asset.volume)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomModelsAlgorithm.py Python

Slippage

Supported Models

Introduction

This page describes the pre-built slippage models in LEAN. If none of these models perform exactly how you want, create a custom slippage model .

Null Model

The `NullSlippageModel` sets the slippage of each order to zero. It's the default slippage model of the DefaultBrokerageModel .

```
security.set_slippage_model(NullSlippageModel.INSTANCE)
```

PY

For more information about this model, see the class reference and implementation .

Constant Model

The `ConstantSlippageModel` applies a constant percentage of slippage to each order.

```
security.set_slippage_model(ConstantSlippageModel(0.01))
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`slippage_percent`	`float`	The slippage percent for each order. The value must be in the interval (0, 1).	

For more information about this model, see the class reference and implementation .

Volume Share Model

The `VolumeShareSlippageModel` calculates the slippage of each order by multiplying the price impact constant by the square of the ratio of the order to the total volume. If the volume of the current bar is zero, the slippage percent is

$$volumeLimit^2 * priceImpact$$

where volumeLimit and priceImpact are custom input variables. If the volume of the current bar is positive, the slippage percent is

$$\min(\frac{|orderQuantity|}{barVolume}, volumeLimit)^2 * priceImpact$$

where orderQuantity is the quantity of the order and barVolume is the volume of the current bar. If the security subscription provides `TradeBar` data, the barVolume is the volume of the current bar. If the security subscription provides `QuoteBar` data, the barVolume is the bid or ask size of the current bar. CFD, Forex, and Crypto data often doesn't include volume. If there is no volume reported for these asset classes, the model returns zero slippage.

```
security.set_slippage_model(VolumeShareSlippageModel())
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`volume_limit`	`float`	Maximum percent of historical volume that can fill in each bar. 0.5 means 50% of historical volume. 1.0 means 100%.	0.025
`price_impact`	`float`	Scaling coefficient for price impact. Larger values will result in more simulated price impact. Smaller values will result in less simulated price impact.	0.1

For more information about this model, see the class reference and implementation .

Market Impact Model

The `MarketImpactSlippageModel` calculates the slippage with perspective to the market impact created by an order, which mimics the consumption of the order book and disruption of the supply-demand relationship at the microeconomic scale. The model is divided into distinct parts, the market impact estimation and the slippage that results from it.

Factors to Consider

The model includes factors that create market impact. Almgren, Thum, Hauptmann & Li (2005) suggested the following factors:

1. Execution Time

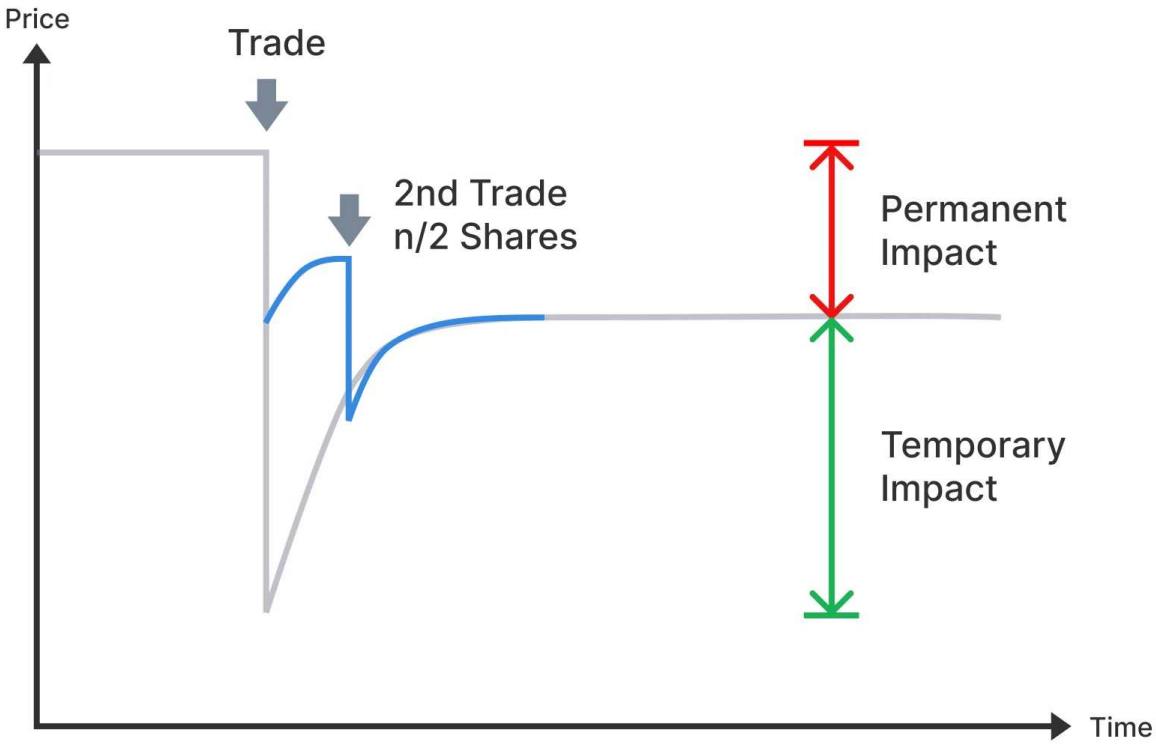
   Asset price time series is usually considered as Random walk (with/without drift) $S_t = S_{t-1} + f(S, t) + \epsilon$ or Brownian Motion $S_t = S_0 + exp((\mu-\frac{1}{2}\sigma^2)t + \sigma W_t)$ In either model, the variance of predictions increases over time ($Var(S)=\sigma\sqrt{t}$). In fact, this happens to most stock price models. Thus, time uncertainty should also apply to the concept of slippage. The longer it takes to go from order submission to execution, the further the fill price will be from the price at order submission.

2. Volatility of the Underlying Securities

   Like (1), the volatility also contributes to the uncertainty of the next price ($Var(S)=\sigma\sqrt{t}$). In most cases, volatility has a one-sided impact since the market's volatility is often introduced by overbuying or overselling from positive or negative sentiments.

3. Liquidity

   Liquidity plays a crucial role in the supply-demand resilience. When a stock is liquid, an order usually only takes a small fraction of the market depth. Hence, the gap between the bid and ask price can be refilled quickly due to high market participation and a small gap. However, when a stock is illiquid, a single order might already take the top few layers of the order book, creating a wide gap between the bid and ask price and volume, which makes a bigger shift in the equilibrium price. This will create an impact which is permanent in nature.

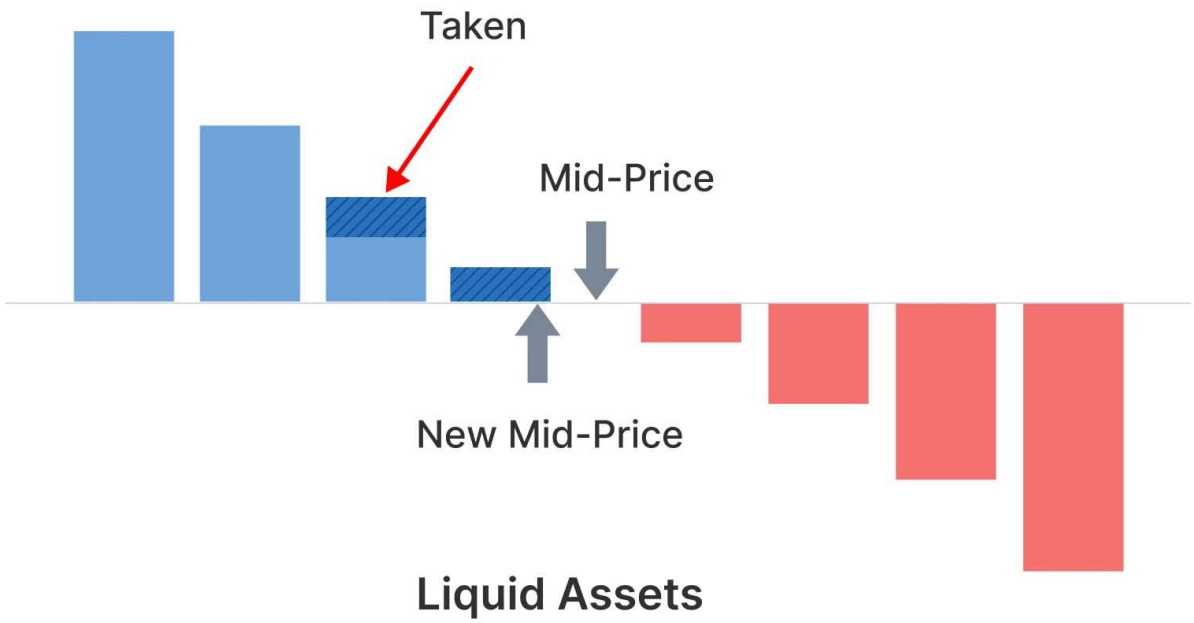


4. Order Size

   Last but not least, the order size directly affects the proportion of the order book/liquidity that the order consumes. Moreover, the larger the order is, the longer it takes to fill since the market might not have sufficient depth. This has interaction effects with the preceding factors.

Market Impact Decomposition

In the book "A Signal Processing Perspective on Financial Engineering", Feng & Palomer (2015) generalized scholars' opinions that market impact can be broken down into "permanent" and "temporary" components, with noise. The permanent impact of a trade refers to the price differences between equilibrium prices before and after the trade and the resilience of the order book. In contrast, temporary impact refers to the price difference between the instant price drop after a trade and the post-trade equilibrium.



Liquid Assets

Illiquid Assets

The model incorporates the preceding factors and estimates both the permanent and temporary impact components with two diffusion models as a function of $\nu=\frac{X}{VT}$, which is the normalized order size: $G\left(\nu\right)=\pm\gamma\left|\nu\right|^\alpha+\text{noise}$ $H\left(\nu\right)=\pm\eta\left|\nu\right|^\beta+\text{noise}$ where X is the order quantity, V is the average daily volume, and T is the time difference between order submission and filling (execution time). The other symbols are parameters that are calibrated from historical trade data. Together with the execution time, daily volatility σ, and liquidity factor δ, the complete models are as follows: $I=\sigma T \gamma\left|\frac{X}{VT}\right|^\alpha\left(\frac{\Theta}{V}\right)^\delta+\sigma\sqrt{T_{\text{post}}}\epsilon$ $J=\sigma T \eta\left|\frac{X}{VT}\right|^\beta+\sigma\sqrt{T_{\text{post}}}\epsilon$ where Θ is the share outstanding, T_{post} is the time difference between order

submission and the new equilibrium established, I is the permanent market impact, and J is the temporary market impact. The inverse of the turnover rate $\frac{\Theta}{V}$ contributes to the permanent impact. This relationship is reasonable since higher turnover means more market participants are in the market, so the order book depth is deeper.

Implementation

```
security.set_slippage_model(MarketImpactSlippageModel(self))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`algorithm`	`IAlgorithm`	The algorithm instance	
`non_negative`	`bool`	Indicator whether only non-negative slippage allowed.	`True`
`latency`	`float`	Seconds between order submission and order fill	0.075
`impact_time`	`float`	Seconds between order fill and the establishment of the new equilibrium	1800
`alpha`	`float`	Exponent of the permanent impact function	0.891
`beta`	`float`	Exponent of the temporary impact function	0.600
`gamma`	`float`	Coefficient of the permanent impact function	0.314
`eta`	`float`	Coefficient of the temporary impact function	0.142
`delta`	`float`	Liquidity scaling factor for permanent impact	0.267
`random_seed`	`int`	Random seed for generating gaussian noise	50

The default parameters of the `MarketImpactSlippageModel` are the parameters that the authors calibrated and shared. However, you may want to reference the original paper and recalibrate the parameters because of the following reasons:

- The default parameters are calibrated around 2 decades ago
- The trading time effect is not accounted (volume near market open/close is larger)
- The market environment does not have many market makers at that time

For more information about this model, see the class reference and implementation .

Reality Modeling

Transaction Fees

Transaction Fees

Key Concepts

Introduction

Your orders incur a transaction fee when a brokerage fills them in the market. LEAN uses transaction fee models in backtesting to model the live trading fees you would incur with the strategy. Transaction fee models make backtest results more realistic. To give your backtests the most accurate fees, LEAN contains transaction fee models that model the fee structure of many popular brokerages.

Set Models

The brokerage model of your algorithm automatically sets the fee model for each security, but you can override it. To manually set the fee model of a security, call the `set_fee_model` method on the `Security` object.

```
PY
def initialize(self):
    security = self.add_equity("SPY")
    # Set the fee model for the requested security to backtest with the most realistic scenario
    # Constant fee model of 0 dollar is accurate for the commission-free brokerage
    security.set_fee_model(ConstantFeeModel(0))
```

You can also set the fee model in a security initializer . If your algorithm has a dynamic universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```
PY
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fee_model(ConstantFeeModel(0))
```

In live trading, the `set_fee_model` method isn't ignored. If we use order helper methods like `set_holdings` , the fee model helps to calculate the order quantity. However, the algorithm doesn't update the cash book with the fee from the fee model. The algorithm uses the actual fee from the brokerage to update the cash book.

To view all the pre-built fee models, see Supported Models .

Default Behavior

The brokerage model of your algorithm automatically sets the fill model for each security. The default brokerage model is the `DefaultBrokerageModel` , which set the ConstantFeeModel with no fees for Forex, CFD, and Crypto assets and sets the InteractiveBrokersFeeModel for the remaining asset classes.

Model Structure

Fee models should extend the `FeeModel` class. Extensions of the `FeeModel` class must implement the `get_order_fee` method, which receives `OrderFeeParameters` and returns an `OrderFee` that represents a cash amount in a currency.

```py
class CustomFeeModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the custom fee model for an added security to use the custom model
        security = self.add_equity("SPY")
        security.set_fee_model(MyFeeModel())

# Define the custom fee model
class MyFeeModel(FeeModel):

    def get_order_fee(self, parameters: OrderFeeParameters) -> OrderFee:
        return OrderFee(CashAmount(0.5, 'USD'))
```

The `OrderFeeParameters` object has the following members:

Zero Transaction Fees

To model zero transaction fees, use the ConstantFeeModel with zero as `fee` and specify the `currency` .

```py
def initialize(self):
    security = self.add_equity("SPY")
    security.set_fee_model(ConstantFeeModel(fee=0, currency="USD"))
```

Negative Transaction Fees

If you short a security and receive interest payments, they are negative transaction fees.

Examples

The following examples demonstrate common practices for implementing a custom fee model.

Example 1: Fixed Fees

The following algorithm incurs a $0.50 fee for each order.

```py
class CustomFeeModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        security = self.add_equity('SPY', Resolution.DAILY)
        security.set_fee_model(MyFeeModel())

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyFeeModel(FeeModel):

    def get_order_fee(self, parameters: OrderFeeParameters) -> OrderFee:
        return OrderFee(CashAmount(0.5, 'USD'))
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomModelsAlgorithm.py Python RawPricesCoarseUniverseAlgorithm.py Python

Transaction Fees

Supported Models

Introduction

This page describes some of the pre-built fee models in LEAN. For more brokerage-specific fee models, see the brokerage model documentation . If none of these models perform exactly how you want, create a custom fee model .

Constant Model

The `ConstantFeeModel` applies the absolute value of a constant fee to each order. It's the default fee model of the DefaultBrokerageModel if you trade Forex, CFD, or Crypto assets. Use this model to set zero transaction fees .

```
security.set_fee_model(ConstantFeeModel(0.05))
```
PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`fee`	`float`	The order fee quantity	
`currency`	`str`	The order fee currency	"USD"

For more information about this model, see the class reference and implementation .

Alpaca Model

The `AlpacaFeeModel` models the fees of Alpaca .

```
security.set_fee_model(AlpacaFeeModel())
```
PY

The `AlpacaFeeModel` charges zero fees for Equity and Equity Options trading. For Crypto trades, it charges a 0.15% maker and 0.25% taker fee. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Alpaca adjusts your fees based on your 30-day trading volume, but we don't currently model these metrics to adjust fees.

The `AlpacaFeeModel` charges fees in the currency you receive from a trade. For example, if you buy ETHBTC, you pay fees in ETH. If you sell ETHBTC, you pay fees in BTC.

For more information about this model, see the class reference and implementation .

Binance Model

The `BinanceFeeModel` models the fees of spot trades on Binance and Binance US .

```
security.set_fee_model(BinanceFeeModel())
```
PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`m_fee`	`float`	Maker fee value	0.001
`t_fee`	`float`	Taker fee value	0.001

The `BinanceFeeModel` charges the order fees of Binance and Binance US at the VIP 0 level, which is a 0.1% maker and taker fee. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Binance adjusts your fees based on your 30-day trading volume and BNB balance, but we don't currently model these metrics to adjust fees.

The `BinanceFeeModel` charges fees in the currency you receive from a trade. For example, if you buy ETHBTC, you pay fees in ETH. If you sell ETHBTC, you pay fees in BTC.

For more information about this model, see the class reference and implementation .

Binance Futures Model

The `BinanceFuturesFeeModel` models the Binance Futures live trading fees .

```
security.set_fee_model(BinanceFuturesFeeModel())
```
PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`m_usdt_fee`	`float`	Maker fee value for USDT pair contracts	0.0002
`t_usdt_fee`	`float`	Taker fee value for USDT pair contracts	0.0004
`m_busd_fee`	`float`	Maker fee value for BUSD pair contracts	0.00012
`t_busd_fee`	`float`	Taker fee value for BUSD pair contracts	0.00036

By default, the `BinanceFuturesFeeModel` charges the order fees of Binance Futures at the Regular User level. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Binance adjusts your fees based on your 30-day trading volume and BNB balance, but we don't currently model these metrics to adjust fees.

The `BinanceFuturesFeeModel` charges fees in the currency you receive from a trade. For example, if you buy BTCUSD, you pay fees in BTC. If you sell BTCUSD, you pay fees in USD.

For more information about this model, see the class reference and implementation .

Binance Coin Futures Model

The `BinanceCoinFuturesFeeModel` models the Binance Coin Futures live trading fees .

```
security.set_fee_model(BinanceCoinFuturesFeeModel())
```
PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`m_fee`	`float`	Maker fee value	0.0001
`t_fee`	`float`	Taker fee value	0.0005

By default, the `BinanceCoinFuturesFeeModel` charges the order fees of Binance Coin Futures at the Regular User level. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Binance adjusts your fees based on your 30-day trading volume and BNB balance, but we don't currently model these metrics to adjust fees.

The `BinanceCoinFuturesFeeModel` charges fees in the currency you receive from a trade. For example, if you buy BTCUSDT, you pay fees in BTC. If you sell BTCUSDT, you pay fees in USDT.

For more information about this model, see the class reference and implementation .

Bitfinex Model

The `BitfinexFeeModel` models the Bitfinex live trading fees .

```
PY
security.set_fee_model(BitfinexFeeModel())
```

The `BitfinexFeeModel` charges a 0.1% maker fee and a 0.2% taker fee. If you place a limit order that hits a hidden order or you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you place a hidden order or you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Bitfinex adjusts your fees based on your 30-day trading volume and LEO balance, but we don't currently model these metrics to adjust fees.

The `BitfinexFeeModel` charges fees in the currency you receive from a trade. For example, if you buy ETHBTC, you pay fees in ETH. If you sell ETHBTC, you pay fees in BTC.

For more information about this model, see the class reference and implementation .

Bybit Model

The `BybitFeeModel` models the Bybit live trading fees .

```
PY
security.set_fee_model(BybitFeeModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`m_fee`	`float`	Maker fee value	0.001
`t_fee`	`float`	Taker fee value	0.001

The `BybitFeeModel` charges the order fees of Bybit at the VIP 0 level, which is a 0.1% maker and taker fee for spot trades. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Bybit adjusts your fees based on your 30-day trading volume, net borrowings, or asset balance, but we don't currently model these metrics to adjust fees. Bybit also charges different fees for fiat-Crypto pairs, (for example, USDT/EUR), but the `BybitFeeModel` charges the same fee rate for all pairs.

The `BybitFeeModel` charges fees in the currency you receive from a trade. For example, if you buy ETHBTC, you pay fees in ETH. If you sell ETHBTC, you pay fees in BTC.

For more information about this model, see the class reference and implementation .

Bybit Futures Model

The `BybitFuturesFeeModel` models the Bybit live trading fees .

```
PY
security.set_fee_model(BybitFuturesFeeModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maker_fee`	`float`	Maker fee value	0.0002
`taker_fee`	`float`	Taker fee value	0.00055

The `BybitFuturesFeeModel` charges the order fees of Bybit at the VIP 0 level, which is a 0.02% maker and a 0.055% taker fee for Crypto Futures trades. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Bybit adjusts your fees based on your 30-day trading volume, net borrowings, or asset balance, but we don't currently model these metrics to adjust fees.

The `BybitFuturesFeeModel` charges fees in the currency you receive from a trade. For example, if you buy BTCUSDT, you pay fees in BTC. If you sell BTCUSDT, you pay fees in USDT.

For more information about this model, see the class reference and implementation .

Charles Schwab Model

The `CharlesSchwabFeeModel` models the fees of Charles Schwab .

```
security.set_fee_model(CharlesSchwabFeeModel())
```
PY

The `CharlesSchwabFeeModel` charges $0 USD for Equity trades.

For more information about this model, see the class reference and implementation .

Coinbase Model

The `CoinbaseFeeModel` models the Coinbase live trading fees .

```
security.set_fee_model(CoinbaseFeeModel())
```
PY

The `CoinbaseFeeModel` models the order fees of Coinbase at the $50K-100K pricing tier for all Crypto pairs, so it charges a 0.6% maker and a 0.8% taker fee for most pairs. The following table shows the Coinbase Stable Pairs, which charge a 0% maker fee and a 0.001% taker fee:

Stable Pairs			
DAIUSDC	DAIUSD	GYENUSD	PAXUSD
PAXUSDT	MUSDUSD	USDCEUR	USDCGBP
USDTEUR	USDTGBP	USDTUSD	USDTUSDC
USTEUR	USTUSD	USTUSDT	WBTCBTC

If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. Coinbase adjusts your fees based on your 30-day trading volume, but we don't currently model trading volume to adjust fees.

The `CoinbaseFeeModel` models the adjustments Coinbase has made to their fees over time. The following table shows the fees for each time period:

Time Period (UTC)	Maker Fee (%)	Taker Fee (%)	Stable Coin
Time < 3/23/2019 1:30AM	0	0.3	
3/23/2019 1:30AM <= Time < 10/8/2019 12:30AM	0.15	0.25	
10/8/2019 12:30AM <= Time	0.5	0.5	
Time < 6/1/2022 3/23/2019 12:00AM	0	0.1%	✓

The `CoinbaseFeeModel` charges fees in the quote currency .

For more information about this model, see the class reference and implementation .

dYdX Model

The `dYdXFeeModel` models the dYdX live trading fees .

```
security.set_fee_model(dYdXFeeModel())
```

The `dYdXFeeModel` models the order fees of dYdX at the lowest pricing tier for all Cryptofuture pairs, so it charges a 0.01% maker and a 0.05% taker fee.

If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. dYdX adjusts your fees based on your 30-day trading volume, but we don't currently model trading volume to adjust fees.

The `dYdXFeeModel` charges fees in the quote currency .

For more information about this model, see the class reference and implementation .

Interactive Brokers Model

The `InteractiveBrokersFeeModel` models the fees of Interactive Brokers .

```
security.set_fee_model(InteractiveBrokersFeeModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`monthly_forex_trade_amount_in_us_dollars`	`float`	Monthly Forex dollar volume traded	0
`monthly_options_trade_amount_in_contracts`	`float`	Monthly Option contracts traded	0

The following table describes which currency the `InteractiveBrokersFeeModel` charges fees in for each asset class:

Asset Class	Fee Currency
US Equity	USD
India Equity	INR
Equity Options	USD
Forex	USD
US Futures	USD
Hong Kong Futures	The contract quote currency (CNH, HKD, or USD)
US Future Options	USD
Hong Kong Future Options	The contract quote currency (CNH, HKD, or USD)
Index Options	USD
CFD	The contract quote currency

The following sections describe the trading fees of each asset class.

For more information about this model, see the class reference and implementation .

US Equities

US Equity trades cost 0.005/share with a 1 minimum fee and a 0.5% maximum fee.

Equity Options and Index Options

Equity Options and Index Options fees are a function of your monthly volume and the premium of the contract you trade. The following table shows the fees for each volume tier:

Monthly Volume (Contacts)	Premium ($)	Fee per Contract ($)
<= 10,000	< 0.05	0.25
<= 10,000	0.05 <= premium < 0.10	0.50
<= 10,000	>= 0.10	0.70
10,000 < volume <= 50,000	< 0.05	0.25
10,000 < volume <= 50,000	>= 0.05	0.50
50,000 < volume <= 100,000	Any	0.25
> 100,000	Any	0.15

By default, LEAN models your fees at the tier with the lowest monthly volume. To adjust the fee tier, manually set the fee model and provide a `monthly_options_trade_amount_in_contracts` argument.

The following table shows how the model determines the premium for each order type and direction:

Order Type	Buy Order Premium	Sell Order Premium
Market	Bid price	Ask price
Limit	Limit price	Limit price
Limit if touched	Limit price	Limit price
Stop market	Stop price	Stop price
Stop limit	Limit price	Limit price
Trailing stop	Stop price	Stop price
Market on open	Bid price	Ask price
Market on close	Bid price	Ask price
Combo market	Bid price	Ask price
Combo limit	Limit price	Limit price
Combo leg limit	Limit price	Limit price

There is no fee to exercise Option contracts.

Forex

Forex fees are a function of your monthly Forex trading volume. The following table shows the fee tiers:

Monthly Volume (USD)	Commission Rate (%)	Minimum Fee ($)
<= 1B	0.002	2
1B < volume <= 2B	0.0015	1.5
2B < volume <= 5B	0.001	1.25
> 5B	0.0008	1

By default, LEAN models your fees at the tier with the lowest monthly volume. To adjust the fee tier, manually set the fee model and provide a `monthly_forex_trade_amount_in_us_dollars` argument.

US Futures

US Futures fees depend on the contracts you trade. The following table shows the base fee per contract for each Future:

Contract Symbol	Market	Name	Base Fee Per Contract ($)	Exchange Fee Per Contract ($)
E-mini Futures				
ES	CME	E-mini S&P 500 Futures	0.85	1.28
NQ	CME	E-mini Nasdaq-100 Futures	0.85	1.28
YM	CBOT	E-mini Dow ($5) Futures	0.85	1.28
RTY	CME	E-mini Russell 2000 Index Futures	0.85	1.28
EMD	CME	E-mini S&P MidCap 400 Futures	0.85	1.28

Micro E-mini Futures				
MYM	CBOT	Micro E-mini Dow Jones Industrial Average Index Futures	0.25	0.3
M2K	CME	Micro E-mini Russell 2000 Index Futures	0.25	0.3
MES	CME	Micro E-mini Standard and Poor's 500 Stock Price Index Futures	0.25	0.3
MNQ	CME	Micro E-mini Nasdaq-100 Index Futures	0.25	0.3
2YY	CBOT	Micro 2-Year Yield Futures	0.25	0.3
5YY	CBOT	Micro 5-Year Yield Futures	0.25	0.3
10Y	CBOT	Micro 10-Year Yield Futures	0.25	0.3
30Y	CBOT	Micro 30-Year Yield Futures	0.25	0.3
MCL	NYMEX	Micro WTI Crude Oil Futures	0.25	0.3
MGC	COMEX	Micro Gold Futures	0.25	0.3
SIL	COMEX	Micro Silver Futures	0.25	0.3
Cryptocurrency Futures				
BTC	CME	Bitcoin Futures	5	6
MIB	CME	BTIC on Micro Bitcoin Futures	2.25	2.5
MBT	CME	Micro Bitcoin Futures	2.25	2.5
MET	CME	Micro Ether Futures	0.2	0.2
MRB	CME	BTIC on Micro Ether Futures	0.2	0.2
E-mini FX Futures				
E7	CME	E-mini Euro FX Futures	0.5	0.85
J7	CME	E-mini Japanese Yen Futures	0.5	0.85
Micro E-mini FX Futures				
M6E	CME	Micro Euro/U.S. Dollar (EUR/USD) Futures	0.15	0.24
M6A	CME	Micro Australian Dollar/U.S. Dollar (AUD/USD) Futures	0.15	0.24
M6B	CME	Micro British Pound Sterling/U.S. Dollar (GBP/USD) Futures	0.15	0.24

MCD	CME	Micro Canadian Dollar/U.S.Dollar(CAD/USD) Futures	0.15	0.24
MJY	CME	Micro Japanese Yen/U.S. Dollar (JPY/USD) Futures	0.15	0.24
MSF	CME	Micro Swiss Franc/U.S. Dollar (CHF/USD) Futures	0.15	0.24
M6J	CME	Micro USD/JPY Futures	0.15	0.24
MIR	CME	Micro INR/USD Futures	0.15	0.24
M6C	CME	Micro USD/CAD Futures	0.15	0.24
M6S	CME	Micro USD/CHF Futures	0.15	0.24
MNH	CME	Micro USD/CNH Futures	0.15	0.24

If you trade a contract that's not in the preceding table, the base fee is 0.85/contract and the exchange fee is 1.60/contract.

In addition to the base fee and exchange fee, there is a $0.02/contract regulatory fee.

Futures Options

Futures Options fees depend on the contracts you trade. The following table shows the base fee per contract for each Future:

Contract Symbol	Market	Underlying Futures Name	Base Fee Per Contract ($)	Exchange Fee Per Contract ($)
E-mini Futures Options				
ES	CME	E-mini S&P 500 Futures	0.85	0.55
NQ	CME	E-mini Nasdaq-100 Futures	0.85	0.55
YM	CBOT	E-mini Dow ($5) Futures	0.85	0.55
RTY	CME	E-mini Russell 2000 Index Futures	0.85	0.55
EMD	CME	E-mini S&P MidCap 400 Futures	0.85	0.55
Micro E-mini Futures Options				
MYM	CBOT	Micro E-mini Dow Jones Industrial Average Index Futures	0.25	0.2
M2K	CME	Micro E-mini Russell 2000 Index Futures	0.25	0.2
MES	CME	Micro E-mini Standard and Poor's 500 Stock Price Index Futures	0.25	0.2
MNQ	CME	Micro E-mini Nasdaq-100 Index Futures	0.25	0.2
2YY	CBOT	Micro 2-Year Yield Futures	0.25	0.2
5YY	CBOT	Micro 5-Year Yield Futures	0.25	0.2
10Y	CBOT	Micro 10-Year Yield Futures	0.25	0.2
30Y	CBOT	Micro 30-Year Yield Futures	0.25	0.2
MCL	NYMEX	Micro WTI Crude Oil Futures	0.25	0.2
MGC	COMEX	Micro Gold Futures	0.25	0.2
SIL	COMEX	Micro Silver Futures	0.25	0.2
Cryptocurrency Futures Options				
BTC	CME	Bitcoin Futures	5	5
MIB	CME	BTIC on Micro Bitcoin Futures	1.25	2.5
MBT	CME	Micro Bitcoin Futures	1.25	2.5
MET	CME	Micro Ether Futures	0.1	0.2
MRB	CME	BTIC on Micro Ether Futures	0.1	0.2

If you trade a contract that's not in the preceding table, the base fee is 0.85/contract and the exchange fee is 1.60/contract.

In addition to the base fee and exchange fee, there is a $0.02/contract regulatory fee.

CFD

CFD trades cost 0.002% of the trade value. The following table shows the minimum fee for each quote currency:

Quote Currency	Fee
JPY	¥40
MKD	HK$10
Other	1

CFD trading fees are denominated in the quote currency of the CFD product.

Kraken Model

The `KrakenFeeModel` models the fees of Kraken .

```
security.set_fee_model(KrakenFeeModel())
```

The `KrakenFeeModel` model the order fees of Kraken. For trading pairs that contain only Crypto assets, this fee model models the lowest tier in Kraken's tiered fee structure, which is a 0.16% maker fee and a 0.26% taker fee. If you add liquidity to the order book by placing a limit order that doesn't cross the spread, you pay maker fees. If you remove liquidity from the order book by placing an order that crosses the spread, you pay taker fees. For trading pairs that have any of the following currencies as the base currency in the pair, the fee is 0.2%:

- CAD
- EUR
- GBP
- JPY
- USD
- USDT
- DAI
- USDC

The `KrakenFeeModel` charges fees in the quote currency for buy orders and in the base currency for sell orders. To override these settings, define the `KrakenOrderProperties` .

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = KrakenOrderProperties()
    self.default_order_properties.fee_in_base = True
    self.default_order_properties.fee_in_quote = False

def on_data(self, slice: Slice) -> None:
    # Override the default order properties
    order_properties = KrakenOrderProperties()
    order_properties.fee_in_quote = True
    order_properties.fee_in_base = False
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Kraken adjusts your fees based on your 30-day trading volume, but the `KrakenFeeModel` don't currently model trading volume to adjust fees.

For more information about this model, see the class reference and implementation .

Samco Model

The `SamcoFeeModel` models the fees of Samco .

```
security.set_fee_model(SamcoFeeModel())
```

The SamcoFeeModel models the order fees of Samco by its Equity Intraday fee structure. The following table shows the fees:

Charge Item	Fee
Brokerage Fee	₹20 per trade or 0.02% (whichever is lower)
Exchange Transaction Charge	0.00345%
Securities Transaction Tax	0.025%
Goods and Services Tax	18%
SEBI Charges	0.0001%
Stamp Duty	0.003%

The SamcoFeeModel charges fees in INR.

For more information about this model, see the class reference and implementation .

Tastytrade Model

The TastytradeFeeModel models the fees of Tastytrade .

```
security.set_fee_model(TastytradeFeeModel())
```
PY

The following sections describe the trading fees of each asset class.

For more information about this model, see the class reference and implementation .

US Equities

US Equity trades are free.

Equity Options

Equity Option trades cost $1 USD per contract to open a position, free to close.

Index Options

Index Option trades cost $1 USD per contract to open a position, free to close.

Futures

Futures trades cost $1.25 USD per contract.

Future Options

Future Option trades cost $2.5 USD per contract to open a position, free to close.

TradeStation Model

The TradeStationFeeModel models the fees of TradeStation .

```
security.set_fee_model(TradeStationFeeModel())
```
PY

The following sections describe the trading fees of each asset class.

For more information about this model, see the class reference and implementation .

US Equities

US Equity trades are free for US residents. For non-US residents, they cost $5 USD per order.

Equity Options

Equity Option trades cost 0.60 USD per contract. For non-US residents, they cost an extra 5 USD per order.

Index Options

Index Option trades cost 1.00 USD per contract. For non-US residents, they cost an extra 5 USD per order.

Futures

Futures trades cost $1.50 USD per contract.

Wolverine Model

The `WolverineFeeModel` models the fees of Wolverine Execution Services.

```
security.set_fee_model(WolverineFeeModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`fees_per_share`	`float` / `NoneType`	The fees per share to apply. If `None` , it uses 0.005.	`None`

The `WolverineFeeModel` charges fees in USD.

For more information about this model, see the class reference and implementation .

Zerodha Model

The `ZerodhaFeeModel` models the fees of Zerodha .

```
security.set_fee_model(ZerodhaFeeModel())
```

The `ZerodhaFeeModel` models the order fees of Zerodha with its Equity Intraday fee structure. The following table shows the fees:

Charge Item	Fee
Brokerage Fee	₹20 per trade or 0.03% (whichever is lower)
Exchange Transaction Charge	0.00345%
Securities Transaction Tax	0.025%
Goods and Services Tax	18%
SEBI Charges	0.0001%
Stamp Duty	0.003%

The `ZerodhaFeeModel` charges fees in INR.

For more information about this model, see the class reference and implementation .

Reality Modeling

Brokerages

Brokerages

Key Concepts

Introduction

Brokerages provide you with a connection to the market so you can fill trades. Brokerage models simulate the live behavior of a real brokerage. To avoid sending invalid orders for execution in live trading, the brokerage model validates your orders before LEAN sends them to the real brokerage. Brokerage models combine together all of the models relevant for a brokerage. If you set the appropriate brokerage model, the supported order types, default markets, and some of the security-level models are appropriately set in your algorithm.

Set Models

To set a brokerage model, in the `initialize` method, call the `set_brokerage_model` method with a `BrokerageName` and an `AccountType` . If you set a brokerage model, it overrides any security level models you manually set in your algorithm.

```
def initialize(self) -> None:
 # Set the brokerage model to backtest with the most realistic scenario to handle the order validity, margin rate and transaction fee
 self.set_brokerage_model(BrokerageName.OANDA_BROKERAGE) # Defaults to margin account
 self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN) # Overrides the default account type, which is AccountType.Cash
```

PY

In live trading, LEAN doesn't ignore your `set_brokerage_model` method calls. LEAN uses some of the brokerage model rules to catch invalid orders before they reach your real brokerage.

To view all the pre-built brokerage models, see Supported Models .

Default Behavior

The default brokerage model is the `DefaultBrokerageModel` , but LEAN has many other brokerage models you can use in your algorithms. For more information about the `DefaultBrokerageModel` , see QuantConnect Paper Trading .

Modeled Properties

Brokerage models model that following properties:

- Transaction fees
- Supported asset classes
- Account types
- Support for extended market hours
- Leverage for assets
- Trade settlement rules
- Order properties and updates

Model Structure

Brokerage models should extend the `DefaultBrokerageModel` class. Extensions of the `DefaultBrokerageModel` class should implement the following methods:

```py
class CustomBrokerageModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the custom brokerage model
        self.set_brokerage_model(MyBrokerageModel())

# Define the custom brokerage model outside of the algorithm
class MyBrokerageModel(DefaultBrokerageModel):
    default_markets = {SecurityType.EQUITY: Market.USA}
    required_free_buying_power_percent = 0

    def __init__(self, accountType: account_type = account_type.margin):
        self.account_type = accountType

    def can_submit_order(self, security: Security, order: Order,
         message: BrokerageMessageEvent) -> bool:
        return super().can_submit_order(security, order, message)

    def can_update_order(self, security: Security, order: Order,
         request: UpdateOrderRequest, message: BrokerageMessageEvent) -> bool:
        return super().can_update_order(security, order, request, message)

    def can_execute_order(self, security: Security, order: Order) -> bool:
        return super().can_execute_order(security, order)

    def apply_split(self, tickets: List[OrderTicket], split: Split) -> None:
        super().apply_split(tickets, split)

    def get_leverage(self, security: Security) -> float:
        return super().get_leverage(security)

    def get_benchmark(self, securities: SecurityManager) -> IBenchmark:
        return super().get_benchmark(securities)

    def get_fill_model(self, security: Security) -> IFillModel:
        return super().get_fill_model(security)

    def get_fee_model(self, security: Security) -> IFeeModel:
        return super().get_fee_model(security)

    def get_slippage_model(self, security: Security) -> ISlippageModel:
        return super().get_slippage_model(security)

    def get_settlement_model(self, security: Security) -> ISettlementModel:
        return super().get_settlement_model(security)

    def get_buying_power_model(self, security: Security) -> IBuyingPowerModel:
        return super().get_buying_power_model(security)

    def get_margin_interest_rate_model(self, security: Security) -> IMarginInterestRateModel:
        return super().get_margin_interest_rate_model(security)

    def get_shortable_provider(self, security: Security) -> IShortableProvider:
        return super().get_shortable_provider(security)
```

Custom brokerage models give you enormous control over your algorithm behavior and allow you to model virtually any brokerage in the world.

Live Trading Considerations

LEAN doesn't infer account type, leverage, margin requirements, or pattern day trading rules from your brokerage account. For example, if you have a cash account, and your algorithm doesn't set `AccountType.CASH`, LEAN will not automatically enforce cash account restrictions such as leverage of 1x or immediate settlement. You must manually configure your algorithm to adhere to these rules. If you set the leverage to 10x and you have a leverage of 2x, LEAN will place orders that can be rejected by your brokerage.

The default leverage of U.S. Equities is 2x. However, your brokerage can reduce the leverage of some securities based on market conditions. For example, leveraged ETFs such as TQQQ might have lower leverage during periods of high volatility. If your algorithm doesn't account for this, LEAN doesn't account for it and your orders can be rejected by your brokerage.

Examples

The following examples demonstrate common practices for implementing a custom brokerage model.

Example 1: Default Model Wrapper

The following algorithm defines a subclass of the `DefaultBrokerageModel` and overrides the built-in members so you can easily customize the parts you need.

PY

```
class CustomBrokerageModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.set_brokerage_model(MyBrokerageModel())
        self.add_equity('SPY', Resolution.DAILY)

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyBrokerageModel(DefaultBrokerageModel):
    default_markets = {SecurityType.EQUITY: Market.USA}
    required_free_buying_power_percent = 0

    def __init__(self, account_type: AccountType = AccountType.MARGIN):
        self.account_type = account_type

    def can_submit_order(self, security: Security, order: Order,
         message: BrokerageMessageEvent) -> bool:
        return super().can_submit_order(security, order, message)

    def can_update_order(self, security: Security, order: Order,
         request: UpdateOrderRequest, message: BrokerageMessageEvent) -> bool:
        return super().can_update_order(security, order, request, message)

    def can_execute_order(self, security: Security, order: Order) -> bool:
        return super().can_execute_order(security, order)

    def apply_split(self, tickets: List[OrderTicket], split: Split) -> None:
        super().apply_split(tickets, split)

    def get_leverage(self, security: Security) -> float:
        return super().get_leverage(security)

    def get_benchmark(self, securities: SecurityManager) -> IBenchmark:
        return super().get_benchmark(securities)

    def get_fill_model(self, security: Security) -> IFillModel:
        return super().get_fill_model(security)

    def get_fee_model(self, security: Security) -> IFeeModel:
        return super().get_fee_model(security)

    def get_slippage_model(self, security: Security) -> ISlippageModel:
        return super().get_slippage_model(security)

    def get_settlement_model(self, security: Security) -> ISettlementModel:
        return super().get_settlement_model(security)

    def get_buying_power_model(self, security: Security) -> IBuyingPowerModel:
        return super().get_buying_power_model(security)

    def get_margin_interest_rate_model(self, security: Security) -> IMarginInterestRateModel:
        return super().get_margin_interest_rate_model(security)

    def get_shortable_provider(self, security: Security) -> IShortableProvider:
        return super().get_shortable_provider(security)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
BrokerageModelAlgorithm.py Python

Brokerages

Supported Models

Brokerage models combine together all of the models relevant for a brokerage. The following brokerage models are available:

QuantConnect Paper Trading

Interactive Brokers

TradeStation

Tastytrade

Alpaca

Charles Schwab

Binance

Bybit

Tradier

Kraken

Coinbase

Bitfinex

dYdX

Terminal Link

SSC Eze

Trading Technologies

Wolverine

Oanda

See Also

Reality Modeling
Live Trading Brokerages

Supported Models

QuantConnect Paper Trading

Introduction

This page explains the `DefaultBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```
self.set_brokerage_model(BrokerageName.QUANT_CONNECT_BROKERAGE) # Defaults to margin account
self.set_brokerage_model(BrokerageName.QUANT_CONNECT_BROKERAGE, AccountType.MARGIN) # Overrides the default account type
```

PY

For more information about this model, see the class reference and implementation .

Asset Classes

The `DefaultBrokerageModel` supports trading for all asset classes .

Orders

The following sections describe how the `DefaultBrokerageModel` handles orders.

Order Types

The following table describes the available order types for each asset class that the `DefaultBrokerageModel` supports:

Order Type	US Equity	Equity Options	Crypto	Crypto Futures	Forex	CFD	Futures	Futures Options	Index Options
Market	✓	✓	✓	✓	✓	✓	✓	✓	✓
Limit	✓	✓	✓	✓	✓	✓	✓	✓	✓
Limit if touched	✓	✓	✓	✓	✓	✓	✓	✓	✓
Stop market	✓	✓	✓	✓	✓	✓	✓	✓	✓
Stop limit	✓	✓	✓	✓	✓	✓	✓	✓	✓
Market on open	✓	✓	✓		✓	✓			✓
Market on close	✓	✓	✓		✓	✓	✓	✓	✓
Combo market		✓						✓	✓
Combo limit		✓						✓	✓
Combo leg limit		✓						✓	✓
Exercise Option		✓ Not supported for cash-settled Options						✓	

In live trading, Option orders require a third-party data provider that supports Options. To view the data providers that support Options, see their respective documentation in Cloud Platform , Local Platform , or the CLI .

Time In Force

The `DefaultBrokerageModel` supports the following TimeInForce instructions:

- `DAY`
- `good_til_canceled`
- `good_til_date`

Updates

The `DefaultBrokerageModel` supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The following table shows the fill model that the `DefaultBrokerageModel` uses for each `SecurityType` :

`SecurityType`	**Fill Model**
`Equity`	EquityFillModel
`Future`	FutureFillModel
`FutureOption`	FutureOptionFillModel
Remaining `SecurityType` values	ImmediateFillModel

Slippage

The `DefaultBrokerageModel` uses the NullSlippageModel .

Fees

The `DefaultBrokerageModel` uses the ConstantFeeModel with no fees for Forex, CFD, Crypto, and Crypto Future assets and the InteractiveBrokersFeeModel for the remaining asset classes.

```
# For Forex, CFD, Crypto, and Crypto Future assets:
security.set_fee_model(ConstantFeeModel(0))

# For the remaining asset classes:
security.set_fee_model(InteractiveBrokersFeeModel())
```

Buying Power

The `DefaultBrokerageModel` sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equity Options	`OptionMarginModel`
Futures	`FutureMarginModel`
Future Options	`FuturesOptionsMarginModel`
Index Options	`OptionMarginModel`
Crypto	`CashBuyingPowerModel` for cash accounts or `BuyingPowerModel` for margin accounts
CryptoFuture	`CryptoFutureMarginModel`
Forex	`CashBuyingPowerModel` for cash accounts or `BuyingPowerModel` for margin accounts
Other	`BuyingPowerModel`

If you have a margin account, the `DefaultBrokerageModel` allows 2x leverage for Equities, 50x leverage for Forex and CFDs, and 1x leverage for the remaining asset classes.

Settlement

The following table shows which settlement model the `DefaultBrokerageModel` uses based on the security type and your account type:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the `DefaultBrokerageModel` uses the ImmediateSettlementModel .

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For Futures
security.set_settlement_model(FutureSettlementModel())

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `DefaultBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The following table describes the default markets of each `SecurityType` for the `DefaultBrokerageModel` :

SecurityType	Market
Equity	USA
Option	USA
Future	CME
FutureOption	CME
Index	USA
IndexOption	USA
Forex	Oanda
Cfd	Oanda
Crypto	Coinbase
CryptoFuture	Binance

Account Currency

The `DefaultBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Interactive Brokers

Introduction

This page explains the `InteractiveBrokersBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE, AccountType.CASH)
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `InteractiveBrokersBrokerageModel` supports the following asset classes:

- US Equities
- Equity Options
- Forex
- Futures
- Future Options
- Indices
- Index Options
- CFD

Orders

The `InteractiveBrokersBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the order types that the `InteractiveBrokersBrokerageModel` supports: supports. For specific details about each order type, refer to the IB documentation.

Order Type	IB Documentation Page
Market	Market Orders
Limit	Limit Orders
Limit if touched	Limit if Touched Orders
Stop market	Stop Orders
Stop limit	Stop-Limit Orders
Trailing stop	Trailing Stop Orders
Market on open	Market-on-Open (MOO) Orders
Market on close	Market-on-Close (MOC) Orders
Combo market	Spread Orders
Combo limit	Spread Orders
Combo leg limit	Spread Orders
Exercise Option	Options Exercise

The following table describes the available order types for each asset class that the `InteractiveBrokersBrokerageModel` supports:

Order Type	US Equity	Equity Options	Forex	Futures	Futures Options	Index Options	CFD
Market	✓	✓	✓	✓	✓	✓	✓
Limit	✓	✓	✓	✓	✓	✓	✓
Limit if touched	✓	✓	✓	✓	✓	✓	✓
Stop market	✓	✓	✓	✓	✓	✓	✓
Stopl imit	✓	✓	✓	✓	✓	✓	✓
Trailing stop	✓	✓	✓	✓	✓	✓	✓
Market on open	✓	✓			✓	✓	✓
Market on close	✓			✓			✓
Combo market		✓			✓	✓	
Combo limit		✓			✓	✓	
Combo leg limit		✓			✓	✓	
Exercise Option		✓ Not supported for cash-settled Options			✓		

Order Properties

The `InteractiveBrokersBrokerageModel` supports custom order properties. The following table describes the members of the

`InteractiveBrokersOrderProperties` object that you can set to customize order execution. The table does not include the methods for FA accounts .

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`outside_regular_trading_hours`	`bool`	A flag to signal that the order may be triggered and filled outside of regular trading hours.	`False`

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = InteractiveBrokersOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.outside_regular_trading_hours = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = InteractiveBrokersOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.outside_regular_trading_hours = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `InteractiveBrokersBrokerageModel` supports order updates .

Financial Advisor Group Orders

To place FA group orders, see Financial Advisors .

Fractional Trading

The `InteractiveBrokersBrokerageModel` doesn't support fractional trading .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Order Size Limits

The `InteractiveBrokersBrokerageModel` enforces the Spot Currency Minimum/Maximum Order Sizes from the IB website.

Fills

The following table shows the fill model that the `InteractiveBrokersBrokerageModel` uses for each `SecurityType` :

SecurityType	Fill Model
Equity	EquityFillModel
Future	FutureFillModel
FutureOption	FutureOptionFillModel
Remaining SecurityType values	ImmediateFillModel

Slippage

The InteractiveBrokersBrokerageModel uses the NullSlippageModel .

Fees

The InteractiveBrokersBrokerageModel uses the InteractiveBrokersFeeModel with the default argument values. We model current Interactive Brokers fees on all assets.

Buying Power

The InteractiveBrokersBrokerageModel sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equity Options	OptionMarginModel
Futures	FutureMarginModel
Future Options	FuturesOptionsMarginModel
Index Options	OptionMarginModel
Crypto	CashBuyingPowerModel for cash accounts or BuyingPowerModel for margin accounts
CryptoFuture	CryptoFutureMarginModel
Forex	CashBuyingPowerModel for cash accounts or BuyingPowerModel for margin accounts
Other	BuyingPowerModel

If you have a margin account, the InteractiveBrokersBrokerageModel allows 2x leverage for Equities, 50x leverage for Forex, 10x leverage for CFDs, and 1x leverage for the remaining asset classes.

Settlement

The InteractiveBrokersBrokerageModel uses the ImmediateSettlementModel in most cases. If you trade US Equities or Equity Options with a cash account, it uses the DelayedSettlementModel with the default settlement rules .

The following table shows which settlement model the InteractiveBrokersBrokerageModel uses based on the security type and your account type:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the InteractiveBrokersBrokerageModel uses the ImmediateSettlementModel .

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Interactive Brokers doesn't provide information on which assets aren't settled, so we assume each live trading session starts with its cash fully settled.

Margin Interest Rate

The `InteractiveBrokersBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The following table describes the default markets of each `SecurityType` for the `InteractiveBrokersBrokerageModel` :

SecurityType	Market
Equity	USA
Option	USA
Future	CME
FutureOption	CME
Index	USA
IndexOption	USA
Forex	Oanda
Cfd	InteractiveBrokers

Financial Advisors

IB supports FA accounts for Trading Firm and Institution organizations. FA accounts enable certified professionals to use a single trading algorithm to manage several client accounts.

To place trades using a subset of client accounts, create Account Groups in Trader Workstation and then define the `InteractiveBrokersOrderProperties` when you create orders.

```py
self.default_order_properties = InteractiveBrokersOrderProperties()
self.default_order_properties.fa_group = "TestGroupEQ"
self.default_order_properties.fa_method = "Equal"
self.default_order_properties.account = "DU123456"
```

SecurityHolding objects aggregate your positions across all the account groups. If you have two groups where group A has 10 shares of SPY and group B has -10 shares of SPY, then `self.portfolio["SPY"].quantity` is zero.

LEAN supports several allocation methods for FA group orders. If you intend to use the same group allocation method for every order, set the `default_order_properties` of your algorithm, which sets the order properties for all of your orders.

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = InteractiveBrokersOrderProperties()
    self.default_order_properties.fa_group = "TestGroupEQ"
    self.default_order_properties.fa_method = "Equal"
    self.default_order_properties.account = "DU123456"

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    LimitOrder(_symbol, quantity, limitPrice);
```

To adjust the order properties of an order, change the `default_order_properties` or pass an order properties object to the order method . The following sections explain the FA group allocation methods.

Equal

This group allocation method distributes shares equally between all accounts in the group. When you use this method, you need to specify an order quantity.

For example, say your Account Group includes four accounts and you place an order to buy 400 shares of a stock. In this case, each account receives 100 shares. If your Account Group includes six accounts, each account receives 66 shares, and then 1 share is allocated to each account until all are distributed. After you submit the order, your algorithm receives order events to track the order progress. When all the shares are bought, the order status is `OrderStatus.FILLED` . If one of the accounts in the group can't afford 10 of the shares it needs to buy, 10 shares are cancelled and you'll only end up buying 390 shares in total.

```
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "Equal"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Net Liquidation Value

This group allocation method distributes shares based on the net liquidation value of each account. The system calculates ratios based on the net liquidation value in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with Net Liquidation values of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "NetLiq"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Available Equity

This group allocation method distributes shares based on the amount of available equity in each account. The system calculates ratios based on the available equity in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with available equity of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "AvailableEquity"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Account Currency

The `InteractiveBrokersBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

TradeStation

Introduction

This page explains the `TradeStationBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.TRADE_STATION, AccountType.CASH)
self.set_brokerage_model(BrokerageName.TRADE_STATION, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `TradeStationBrokerageModel` supports trading the following asset classes:

- US Equities
- Equity Options
- Futures
- Index
- Index Options

Orders

The `TradeStationBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `TradeStationBrokerageModel` supports:

Order Type	Equity	Equity Options	Index Options	Futures
Market	✓	✓	✓	✓
Limit	✓	✓	✓	✓
Stop market	✓	✓	✓	✓
Stop limit	✓	✓	✓	✓
Trailing stop	✓	✓	✓	✓
Market on Open	✓			
Market on Close	✓			
Combo market	✓	✓	✓	✓
Combo limit	✓	✓	✓	✓

Order Properties

The `TradeStationBrokerageModel` supports custom order properties. The following table describes the members of the `TradeStationOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`outside_regular_trading_hours`	`bool`	A flag to signal that the order may be triggered and filled outside of regular trading hours.	`False`
`post_only`	`bool`	This flag will ensure the limit order executes only as a maker (no fee) order. If part of the order results in taking liquidity rather than providing, it will be rejected and no part of the order will execute. Equities only.	`False`

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = TradeStationOrderProperties()
    self.default_order_properties.outside_regular_trading_hours = False
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = TradeStationOrderProperties()
    order_properties.post_only = True
    order_properties.time_in_force = TimeInForce.DAY
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.outside_regular_trading_hours = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `TradeStationBrokerageModel` supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The following table shows the fill model that the `TradeStationBrokerageModel` uses for each `SecurityType` :

`SecurityType`	Fill Model
`Equity`	EquityFillModel
`Option`	ImmediateFillModel
`Future`	FutureFillModel

Slippage

The `TradeStationBrokerageModel` uses the NullSlippageModel .

Fees

The `TradeStationBrokersBrokerageModel` uses the TradeStationFeeModel with the default argument values. We model current TradeStation fees on all assets.

Buying Power

The `TradeStationBrokerageModel` sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equities	`SecurityMarginModel`
Equity Options	`OptionMarginModel`
Futures	`FutureMarginModel`

If you have a margin account, the `TradeStationBrokerageModel` allows 2x leverage for Equities and and 1x leverage for Equity Options and Futures.

Settlement

The following table shows which settlement model the `TradeStationBrokerageModel` uses based on the security type and your account type:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the `TradeStationBrokerageModel` uses the ImmediateSettlementModel .

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For Futures
security.set_settlement_model(FutureSettlementModel())

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `TradeStationBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The following table describes the default markets of each `SecurityType` for the `TradeStationBrokerageModel` :

`SecurityType`	`Market`
`Equity`	`USA`
`Option`	`USA`
`Future`	`CME`

Account Currency

The `TradeStationBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Tastytrade

Introduction

This page explains the `TastytradeBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.TASTYTRADE, AccountType.CASH)
self.set_brokerage_model(BrokerageName.TASTYTRADE, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `TastytradeBrokerageModel` supports the following asset classes:

- US Equities
- Equity Options
- Futures
- Future Options
- Indices
- Index Options

Orders

The `TastytradeBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `TastytradeBrokerageModel` supports:

Order Type	Equity	Equity Options	Futures	Future Options	Index Options
Market	✓	✓	✓	✓	✓
Limit	✓	✓	✓	✓	✓
Stop market	✓	✓	✓	✓	✓
Stop limit	✓	✓	✓	✓	✓
Combo limit	✓	✓	✓	✓	✓

Order Properties

The `TastytradeBrokerageModel` supports custom order properties. The following table describes the members of the `TastytradeOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = TastytradeOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = TastytradeOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `TastytradeBrokerageModel` supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The following table shows the fill model that the `TastytradeBrokerageModel` uses for each `SecurityType` :

`SecurityType`	Fill Model
`Equity`	EquityFillModel
`Future`	FutureFillModel
`FutureOption`	FutureOptionFillModel
Remaining `SecurityType` values	ImmediateFillModel

Slippage

The `TastytradeBrokerageModel` uses the NullSlippageModel .

Fees

The `TastytradeBrokersBrokerageModel` uses the TastytradeFeeModel with the default argument values. We model current Tastytrade fees on all assets.

Buying Power

The `TastytradeBrokerageModel` sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equities	`SecurityMarginModel`
Futures	`FutureMarginModel`
All Options Types	`OptionMarginModel`

If you have a margin account, the `TastytradeBrokerageModel` allows 2x leverage for Equities. The remaining asset classes are derivatives.

Settlement

The following table shows which settlement model the `TastytradeBrokerageModel` uses based on the security type and your account type:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the `TastytradeBrokerageModel` uses the ImmediateSettlementModel .

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For Futures
security.set_settlement_model(FutureSettlementModel())

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `TastytradeBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The following table describes the default markets of each `SecurityType` for the `TastytradeBrokerageModel` :

`SecurityType`	`Market`
`Equity`	`USA`
`Option`	`USA`
`Future`	`CME`
`FutureOption`	`CME`
`Index`	`USA`
`IndexOption`	`USA`

Supported Models

Alpaca

Introduction

This page explains the `AlpacaBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```
self.set_brokerage_model(BrokerageName.ALPACA, AccountType.CASH)
self.set_brokerage_model(BrokerageName.ALPACA, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation .

Asset Classes

The `AlpacaBrokerageModel` supports the following asset classes:

- US Equities
- Equity Options
- Crypto

Orders

The `AlpacaBrokerageModel` supports several order types, order properties, and most order updates.

Order Types

The following table describes the available order types for each asset class that the `AlpacaBrokerageModel` supports:

Order Type	Equity	Equity Options	Crypto
Market	✓	✓	✓
Limit	✓	✓	✓
Stop market	✓	✓	✓
Stop limit	✓	✓	✓
Market on Open	✓		
Market on Close	✓		

Order Properties

The `AlpacaBrokerageModel` supports custom order properties. The following table describes the members of the `AlpacaOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`outside_regular_trading_hours`	`bool`	A flag to signal that the order may be triggered and filled outside of regular trading hours.	`False`

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = AlpacaOrderProperties()
    self.default_order_properties.outside_regular_trading_hours = False
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = AlpacaOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.outside_regular_trading_hours = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `AlpacaBrokerageModel` supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The `AlpacaBrokerageModel` uses the EquityFillModel for Equity trades and the ImmediateFillModel for Option and Crypto trades.

Slippage

The `AlpacaBrokerageModel` uses the NullSlippageModel .

Fees

The `AlpacaBrokerageModel` uses the AlpacaFeeModel with the default argument values. We model current Alpaca fees on all assets.

Buying Power

The `AlpacaBrokerageModel` uses the `CashBuyingPowerModel` for cash accounts, the `SecurityMarginModel` for margin accounts, and the `OptionMarginModel` for Option trades.

If you have a margin account, the `AlpacaBrokerageModel` allows 2x leverage for Equities and 3x leverage for Crypto.

Settlement

The `AlpacaBrokerageModel` uses the ImmediateSettlementModel for margin accounts and the DelayedSettlementModel with the default settlement

rules for cash accounts with Equity and Equity Options.

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `AlpacaBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `AlpacaBrokerageModel` is `Market.USA` for Equity and Equity Options. It's `Market.COINBASE` for Crypto.

Account Currency

The `AlpacaBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Charles Schwab

Introduction

This page explains the `CharlesSchwabBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```py
self.set_brokerage_model(BrokerageName.CHARLES_SCHWAB, AccountType.CASH)
self.set_brokerage_model(BrokerageName.CHARLES_SCHWAB, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation .

Asset Classes

The `CharlesSchwabBrokerageModel` supports the following asset classes:

- US Equities
- Equity Options
- Index
- Index Options

Orders

The `CharlesSchwabBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `CharlesSchwabBrokerageModel` supports:

Order Type	Equity	Equity Options	Index Options
Market	✓	✓	✓
Limit	✓	✓	✓
Stop market	✓	✓	
Market on open	✓		
Market on close	✓		
Combo market	✓	✓	✓
Combo limit	✓	✓	✓

Order Properties

The `CharlesSchwabBrokerageModel` supports custom order properties. The following table describes the members of the `CharlesSchwabOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`extended_regular_trading_hours`	`bool`	If set to true, allows orders to also trigger or fill outside of regular trading hours.	`False`

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = CharlesSchwabOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.extended_regular_trading_hours = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `CharlesSchwabBrokerageModel` supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The `CharlesSchwabBrokerageModel` uses the EquityFillModel for Equity trades and the ImmediateFillModel for Equity and Index Options trades.

Slippage

The `CharlesSchwabBrokerageModel` uses the NullSlippageModel .

Fees

The `CharlesSchwabBrokerageModel` uses the CharlesSchwabFeeModel .

Buying Power

The `CharlesSchwabBrokerageModel` uses the `SecurityMarginModel` . If you have a margin account, the `CharlesSchwabBrokerageModel` allows up to 2x leverage.

Settlement

The `CharlesSchwabBrokerageModel` uses the ImmediateSettlementModel for margin accounts and the DelayedSettlementModel with the default settlement rules for cash accounts.

```
# For cash accounts:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For margin accounts:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `CharlesSchwabBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `CharlesSchwabBrokerageModel` is `Market.USA` .

Account Currency

The `CharlesSchwabBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Binance

Introduction

This page explains the Binance brokerage models, including the asset classes they supports, their default security-level models , and their default markets.

```py
# Binance Spot
self.set_brokerage_model(BrokerageName.BINANCE, AccountType.CASH)
self.set_brokerage_model(BrokerageName.BINANCE, AccountType.MARGIN)

# Binance Futures
self.set_brokerage_model(BrokerageName.BINANCE_FUTURES, AccountType.MARGIN)
self.set_brokerage_model(BrokerageName.BINANCE_COIN_FUTURES, AccountType.MARGIN)

# Binance US Spot
self.set_brokerage_model(BrokerageName.BINANCE_US, AccountType.CASH)
```

For more information about these models, see their respective class reference and implementation.

Model	Class Reference	Implementation
BinanceBrokerageModel		
BinanceFuturesBrokerageModel		
BinanceCoinFuturesBrokerageModel		
BinanceUSBrokerageModel		

Asset Classes

The Binance brokerage models support trading Crypto and Crypto Futures .

```py
self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BINANCE)
self.add_crypto_future("BTCUSD", Resolution.MINUTE, Market.BINANCE)
```

The Binance US brokerage model supports trading Crypto .

```py
self.add_crypto("BTCUSDT", Resolution.MINUTE, Market.BINANCE_US)
```

Orders

The Binance and Binance US brokerage models support several order types and order properties, but don't support order updates.

Order Types

The following table describes the available order types for each asset class that the Binance and Binance US brokerage models support:

Order Type	Crypto	Crypto Futures
Market	✓	✓
Limit	✓	✓
Stop Market		✓
Stop limit	✓	✓

Order Properties

The Binance and Binance US brokerage models supports custom order properties. The following table describes the members of the `BinanceOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`post_only`	`bool`	A flag to signal that the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled.	

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = BinanceOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.post_only = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = BinanceOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.post_only = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The Binance and Binance US brokerage models don't support order updates, but you can cancel an existing order and then create a new order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Fills

The Binance brokerage models use the ImmediateFillModel .

Slippage

The Binance brokerage models use the NullSlippageModel .

Fees

The following table shows the fee model that each Binance brokerage model uses:

Brokerage Model	Fee Model
`BinanceBrokerageModel`	BinanceFeeModel
`BinanceFuturesBrokerageModel`	BinanceFuturesFeeModel
`BinanceCoinFuturesBrokerageModel`	BinanceCoinFuturesFeeModel
`BinanceUSBrokerageModel`	BinanceFeeModel

The Binance brokerage models use the default argument values for each fee model so that it models the current Binance fee schedule.

Buying Power

The Binance brokerage models use the `CashBuyingPowerModel` for cash accounts and the `SecurityMarginModel` for margin accounts.

The following table shows the maximum leverage each brokerage model allows for margin accounts:

Brokerage Model	Maximum Leverage
`BinanceBrokerageModel`	3
`BinanceFuturesBrokerageModel`	25
`BinanceCoinFuturesBrokerageModel`	25

The `BinanceUSBrokerageModel` doesn't currently support margin trading.

Settlement

The Binance brokerage models use the ImmediateSettlementModel .

Margin Interest Rate

The following table shows the margin interest rate model that the Binance brokerages use:

Brokerage Model	Margin Interest Rate Model
`BinanceBrokerageModel`	NullMarginInterestRateModel
`BinanceFuturesBrokerageModel`	BinanceFutureMarginInterestRateModel for Crypto Perpetual Futures and NullMarginInterestRateModel for other assets
`BinanceCoinFuturesBrokerageModel`	BinanceFutureMarginInterestRateModel for Crypto Perpetual Futures and NullMarginInterestRateModel for other assets
`BinanceUSBrokerageModel`	NullMarginInterestRateModel

Default Markets

The following table shows the default market of the Binance brokerage models:

Brokerage Model	Market
`BinanceBrokerageModel`	`Market.BINANCE`
`BinanceFuturesBrokerageModel`	`Market.BINANCE`
`BinanceCoinFuturesBrokerageModel`	`Market.BINANCE`
`BinanceUSBrokerageModel`	`Market.BINANCE_US`

Account Currency

Most of the Binance brokerage models support fiat currencies like USD, but the `BinanceBrokerageModel` doesn't. When you use this brokerage

model, set the account currency to USDT or another Cryptocurrency that Binance supports .

Supported Models

Bybit

Introduction

This page explains the `BybitBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```
self.set_brokerage_model(BrokerageName.BYBIT, AccountType.CASH)
self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `BybitBrokerageModel` supports trading Crypto and Crypto Futures.

Orders

The `BybitBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `BybitBrokerageModel` supports:

Order Type	Crypto	Crypto Futures
Market	✓	✓
Limit	✓	✓
Stop market	✓	✓
Stop limit	✓	✓

Order Properties

The `BybitBrokerageModel` supports custom order properties. The following table describes the members of the `BybitBrokerageModel` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`post_only`	`bool`	A flag to signal that the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled. This order property is only available for limit orders.	
`reduce_only`	`bool/NoneType`	A flag to signal that the order must only reduce your current position size. For more information about this order property, see Reduce-Only Order on the Bybit website.	

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = BybitOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.post_only = False
    self.default_order_properties.reduce_only = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = BybitOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.post_only = True
    self.default_order_properties.reduce_only = False
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.post_only = False
    self.default_order_properties.reduce_only = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `BybitBrokerageModel` supports order updates for Crypto Future assets that have one of the following order states :

- `OrderStatus.NEW`
- `OrderStatus.PARTIALLY_FILLED`
- `OrderStatus.SUBMITTED`
- `OrderStatus.FILLED`

In cases where you can't update an order, you can cancel the existing order and then create a new order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Fills

The `BybitBrokerageModel` uses the ImmediateFillModel .

Slippage

The `BybitBrokerageModel` uses the NullSlippageModel .

Fees

The `BybitBrokerageModel` uses the BybitFeeModel for Crypto trades and the BybitFuturesFeeModel for Crypto Future trades. In both cases, the brokerage model uses the default argument values that models the Bybit fee schedule for VIP Level 0.

Buying Power

The `BybitBrokerageModel` sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Crypto	`CashBuyingPowerModel` for cash accounts or `SecurityMarginModel` for margin accounts
CryptoFuture	`CryptoFutureMarginModel`

If you have a margin account, the `BybitBrokerageModel` allows 10x leverage.

Settlement

The `BybitBrokerageModel` uses the ImmediateSettlementModel .

Margin Interest Rate

The `BybitBrokerageModel` uses the BybitFutureMarginInterestRateModel for perpetual Futures and the NullMarginInterestRateModel for every other asset.

Default Markets

The default market of the `BybitBrokerageModel` is `Market.BYBIT` .

Account Currency

The `BybitBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Tradier

Introduction

This page explains the `TradierBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE, AccountType.CASH)
self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `TradierBrokerageModel` supports trading US Equities and Equity Options.

Orders

The `TradierBrokerageModel` supports several order types, order properties, and most order updates.

Order Types

The following table describes the available order types for each asset class that the `TradierBrokerageModel` supports:

Order Type	Equity	Equity Options
Market	✓	✓
Limit	✓	✓
Stop market	✓	✓
Stop limit	✓	✓

Order Properties

The `TradierBrokerageModel` supports custom order properties. The following table describes the members of the `TradierOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`outside_regular_trading_hours`	`bool`	A flag to signal that the order may be triggered and filled outside of regular trading hours.	`False`

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = TradierOrderProperties()
    self.default_order_properties.outside_regular_trading_hours = False
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = TradierOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.outside_regular_trading_hours = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `TradierBrokerageModel` supports most order updates . To update the quantity of an order, cancel the order and then submit a new order with the desired quantity. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Extended Market Hours

The `TradierBrokerageModel` doesn't support extended market hours trading. If you place an order outside of regular trading hours, the order will be processed at market open.

Automatic Cancellations

If you have open orders for a security when it performs a reverse split, the `TradierBrokerageModel` automatically cancels your orders.

Errors

To view the order-related error codes from Tradier, see Error Responses in their documentation.

The `TradierBrokerageModel` validates your orders for the following errors before sending them to Tradier:

Error	Description
`ShortOrderIsGtc`	You can't short sell with the `GOOD_TIL_CANCELED` time in force
`SellShortOrderLastPriceBelow5`	You can't short sell stocks below $5
`IncorrectOrderQuantity`	The order quantity must be between 1 and 10,000,000 shares

Fills

The `TradierBrokerageModel` uses the EquityFillModel for Equity trades and the ImmediateFillModel for Option trades.

Slippage

The `TradierBrokerageModel` uses the NullSlippageModel .

Fees

The `TradierBrokerageModel` uses the ConstantFeeModel with zero fees.

```
security.set_fee_model(ConstantFeeModel(0))
```

Buying Power

The `TradierBrokerageModel` uses the `OptionMarginModel` for Option trades and the `SecurityMarginModel` for Equity trades.

If you have a margin account, the `TradierBrokerageModel` allows 2x leverage for Equities.

Settlement

The TradierBrokerageModel uses the ImmediateSettlementModel for margin accounts and the DelayedSettlementModel with the default settlement rules for cash accounts.

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The TradierBrokerageModel uses the NullMarginInterestRateModel .

Default Markets

The default market of the TradierBrokerageModel is Market.USA .

Account Currency

The TradierBrokerageModel doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Kraken

Introduction

This page explains the `KrakenBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```py
self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.CASH)
self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation .

Asset Classes

The `KrakenBrokerageModel` supports trading Crypto .

Orders

The `KrakenBrokerageModel` supports several order types and order properties, but it doesn't support order updates.

Order Types

The following table describes the available order types for each asset class that the `KrakenBrokerageModel` supports:

Order Type	Crypto
Market	✓
Limit	✓
Limit if touched	✓
Stop market	✓
Stop limit	✓

Order Properties

The `KrakenBrokerageModel` supports custom order properties. The following table describes the members of the `KrakenOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`post_only`	`bool`	A flag to signal that the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled.	
`fee_in_base`	`bool`	A flag to signal that the order fees should be paid in the base currency, which is the default behavior when selling. This flag must be the opposite of the `fee_in_quote` flag.	
`fee_in_quote`	`bool`	A flag to signal that the order fees should be paid in the quote currency, which is the default behavior when buying. This flag must be the opposite of the `fee_in_base` flag.	
`no_market_price_protection`	`bool`	A flag to signal that no Market Price Protection should be used.	
`conditional_order`	`Order`	An `Order` that's submitted when the primary order is executed. The `conditional_order` quantity must match the primary order quantity and the `conditional_order` direction must be the opposite of the primary order direction. This order property is only available for live algorithms.	

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = KrakenOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.post_only = False
    self.default_order_properties.fee_in_base = True
    self.default_order_properties.fee_in_quote = False
    self.default_order_properties.no_market_price_protection = True

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = KrakenOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.post_only = True
    order_properties.fee_in_base = False
    order_properties.fee_in_quote = True
    order_properties.no_market_price_protection = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.post_only = False
    order_properties.fee_in_base = True
    order_properties.fee_in_quote = False
    order_properties.no_market_price_protection = False
    order_properties.conditional_order = StopLimitOrder(self._symbol, -quantity, stop_limit_price, stop_price)
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `KrakenBrokerageModel` doesn't support order updates, but you can cancel an existing order and then create a new order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Fills

The `KrakenBrokerageModel` uses the ImmediateFillModel .

Slippage

The `KrakenBrokerageModel` uses the NullSlippageModel .

Fees

The `KrakenBrokerageModel` uses the KrakenFeeModel .

Buying Power

The `KrakenBrokerageModel` uses the `CashBuyingPowerModel` for cash accounts and the `SecurityMarginModel` for margin accounts.

If you have a margin account, the `KrakenBrokerageModel` allows 1x leverage for most Crypto pairs. The following table shows pairs that have additional leverage available:

Quote Currency	Base Currencies	Leverage
ADA	BTC, ETH, USD, EUR	3
BCH	BTC, USD, EUR	2
BTC	USD, EUR	5
DASH	BTC, USD, EUR	3
EOS	BTC, ETH, USD, EUR	3
ETH	BTC, USD, EUR	5
LINK	BTC, ETH, USD, EUR	3
LTC	BTC, USD, EUR	3
REP	BTC, ETH, USD, EUR	2
TRX	BTC, ETH, USD, EUR	3
USDC	USD, EUR	3
USDT	USD, EUR	2
XMR	BTC, USD, EUR	2
XRP	BTC, USD, EUR	3
XTZ	BTC, ETH, USD, EUR	2

Settlement

The `KrakenBrokerageModel` uses the ImmediateSettlementModel .

Margin Interest Rate

The `KrakenBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `KrakenBrokerageModel` is `Market.KRAKEN` .

Account Currency

The `KrakenBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Coinbase

Introduction

This page explains the `CoinbaseBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.COINBASE, AccountType.CASH)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `CoinbaseBrokerageModel` supports trading Crypto.

Orders

The `CoinbaseBrokerageModel` supports several order types and order properties, but it doesn't support order updates.

Order Types

The following table describes the available order types for each asset class that the `CoinbaseBrokerageModel` supports:

Order Type	Crypto
Market	✓
Limit	✓
Stop market	✓ Supported before 2019-03-23 in backtests. For reference, see the Coinbase Market Structure Update on the Coinbase website.
Stop limit	✓

Order Properties

The `CoinbaseBrokerageModel` supports custom order properties. The following table describes the members of the `CoinbaseOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The `TimeInForce` is supported.	`TimeInForce.GOOD_TIL_CANCELED`
`post_only`	`bool`	A flag that signals the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled.	
`self_trade_prevention_id`	`bool`	A flag that signals self-trade prevention is enabled for this order. Self-trade prevention helps prevent an order from crossing against the same user, reducing the risk of unintentional trades within the same account.	

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = CoinbaseOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.post_only = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = CoinbaseOrderProperties()
    order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    order_properties.post_only = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `CoinbaseBrokerageModel` doesn't support limit order updates, but you can cancel an existing limit order and then create a new limit order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Fills

The `CoinbaseBrokerageModel` uses the ImmediateFillModel .

Slippage

The `CoinbaseBrokerageModel` uses the NullSlippageModel .

Fees

The `CoinbaseBrokerageModel` uses the CoinbaseFeeModel .

Buying Power

The `CoinbaseBrokerageModel` uses the `CashBuyingPowerModel` . The brokerage model doesn't currently support margin trading.

Settlement

The `CoinbaseBrokerageModel` uses the ImmediateSettlementModel .

Margin Interest Rate

The `CoinbaseBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `CoinbaseBrokerageModel` is `Market.COINBASE` .

Account Currency

The `CoinbaseBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Bitfinex

Introduction

This page explains the `BitfinexBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.CASH)
self.set_brokerage_model(BrokerageName.BITFINEX, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `BitfinexBrokerageModel` supports trading Crypto.

Orders

The `BitfinexBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `BitfinexBrokerageModel` supports:

Order Type	Crypto
Market	✓
Limit	✓
Stop market	✓
Stop limit	✓

Order Properties

The `BitfinexBrokerageModel` supports custom order properties. The following table describes the members of the `BitfinexOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`hidden`	`bool`	A flag to signal that the order should be hidden. Hidden orders do not appear in the order book, so they do not influence other market participants. Hidden orders incur the taker fee.	
`post_only`	`bool`	A flag to signal that the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled.	

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = BitfinexOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.hidden = False
    self.default_order_properties.post_only = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = BitfinexOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.hidden = True
    order_properties.post_only = False
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.hidden = False
    order_properties.post_only = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `BitfinexBrokerageModel` supports order updates .

Fills

The `BitfinexBrokerageModel` uses the ImmediateFillModel .

Slippage

The `BitfinexBrokerageModel` uses the NullSlippageModel .

Fees

The `BitfinexBrokerageModel` uses the BitfinexFeeModel with the default argument values that models the current Bitfinex fee schedule.

Buying Power

The `BitfinexBrokerageModel` uses the `CashBuyingPowerModel` for cash accounts and the `SecurityMarginModel` for margin accounts.

If you have a margin account, the `BitfinexBrokerageModel` allows 3.3x leverage.

Settlement

The `BitfinexBrokerageModel` uses the ImmediateSettlementModel .

Margin Interest Rate

The `BitfinexBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `BitfinexBrokerageModel` is `Market.BITFINEX` .

Account Currency

The `BitfinexBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

dYdX

Introduction

This page explains the `dYdXBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```py
self.set_brokerage_model(BrokerageName.DYDX, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `dYdXBrokerageModel` supports trading Crypto futures.

Orders

The `dYdXBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `dYdXBrokerageModel` supports:

Order Type	Crypto Future
Market	✓
Limit	✓
Stop market	✓
Stop limit	✓

Order Properties

The `dYdXBrokerageModel` supports custom order properties. The following table describes the members of the `dYdXOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` [1] • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`gas_limit`	`int`	The maximum amount of gas to use for the order.	1_000_000
`good_til_block_offset`	`int`	The block height at which the order expires.	20
`IOC` [2]	`bool`	Enforces that an order only be placed on the book as a maker order. Note this means that validators will cancel any newly placed post only orders that would cross with other maker.	`False`
`post_only` [2]	`bool`	A flag to signal that the order must only add liquidity to the order book and not take liquidity from the order book. If part of the order results in taking liquidity rather than providing liquidity, the order is rejected without any part of it being filled. This order property is only available for limit orders.	`False`
`reduce_only`	`bool`	A flag to signal that the order must only reduce your current position size. For more information about this order property, see Reduce-Only Order on the dYdX website.	`False`

[1] `GOOD_TIL_CANCELED` is not fully supported. These orders expire in 90 days.

[2] You cannot set `post_only` when `IOC` is already set and vice-versa. Only one execution type can be active at a time.

PY

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = dYdXOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.reduce_only = False
    self.default_order_properties.post_only = False

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = dYdXOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.reduce_only = True
    order_properties.post_only = False
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.reduce_only = False
    order_properties.post_only = True
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `dYdXBrokerageModel` doesn't support order updates, but you can cancel an existing order and then create a new order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Fills

The `dYdXBrokerageModel` uses the ImmediateFillModel .

Slippage

The `dYdXBrokerageModel` uses the NullSlippageModel .

Fees

The `dYdXBrokerageModel` uses the dYdXFeeModel with the default argument values that models the current dYdX fee schedule.

Buying Power

The `dYdXBrokerageModel` uses the `SecurityMarginModel` and allows 3.3x leverage.

Settlement

The `dYdXBrokerageModel` uses the ImmediateSettlementModel .

Margin Interest Rate

The `dYdXBrokerageModel` uses the dYdXFutureMarginInterestRateModel .

Default Markets

The default market of the `dYdXBrokerageModel` is `Market.DYDX` .

Account Currency

The `dYdXBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Terminal Link

Introduction

This page explains the Terminal Link brokerage, including the asset classes it supports, its default security-level models , and its default markets.

Asset Classes

Terminal Link supports trading the following asset classes:

- Equities
- Equity Options
- Futures
- Index Options

Orders

Terminal Link enables you to create and manage Bloomberg™ orders.

Order Types

The following table describes the available order types for each asset class that Terminal Link supports:

Order Type	Equity	Equity Options	Futures	Index Options
Market	✓	✓	✓	✓
Limit	✓	✓	✓	✓
Stop market	✓	✓	✓	✓
Stop limit	✓	✓	✓	✓

Order Properties

We model custom order properties from the Bloomberg EMSX API. The following table describes the members of the `TerminalLinkOrderProperties` object that you can set to customize order execution:

Property	Data Type	Description	Default Value
`time_in_force`	`TimeInForce`	A TimeInForce instruction to apply to the order. The following instructions are supported: • `DAY` • `GOOD_TIL_CANCELED` • `good_til_date`	`TimeInForce.GOOD_TIL_CANCELED`
`notes`	`str`	The free form instructions that may be sent to the broker.	
`handling_instruction`	`str`	The instructions for handling the order or route. The values can be preconfigured or a value customized by the broker.	

`custom_notes_1`	`str`	Custom user order notes 1. For more information about custom order notes, see Custom Notes & Free Text Fields in the EMSX API documentation	
`custom_notes_2`	`str`	Custom user order notes 2.	
`custom_notes_3`	`str`	Custom user order notes 3.	
`custom_notes_4`	`str`	Custom user order notes 4.	
`custom_notes_5`	`str`	Custom user order notes 5.	
`account`	`str`	The EMSX account.	
`broker`	`str`	The EMSX broker code.	
`strategy`	`StrategyParameters`	An object that represents the EMSX order strategy details. You must append strategy parameters in the order that the EMSX API expects. The following strategy names are supported: "DMA", "DESK", "VWAP", "TWAP", "FLOAT", "HIDDEN", "VOLUMEINLINE", "CUSTOM", "TAP", "CUSTOM2", "WORKSTRIKE", "TAPNOW", "TIMED", "LIMITTICK", "STRIKE"	
`execution_instruction`	`str`	The execution instruction field.	
`automatic_position_sides`	`bool`	A flag that determines whether to automatically include the position side in the order direction (buy-to-open, sell-to-close, etc.) instead of the default (buy, sell).	
`position_side`	`OrderPosition/NoneType`	An `OrderPosition` object that specifies the position side in the order direction (buy-to-open, sell-to-close, etc.) instead of the default (buy, sell). This member has precedence over `automatic_position_sides` .	
`exchange`	`Exchange`	Defines the exchange name for sending the order to.	

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = TerminalLinkOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.strategy = TerminalLinkOrderProperties.StrategyParameters(
        "VWAP",
        [
            TerminalLinkOrderProperties.StrategyField("09:30:00"),
            TerminalLinkOrderProperties.StrategyField("10:30:00"),
            TerminalLinkOrderProperties.StrategyField(),
            TerminalLinkOrderProperties.StrategyField()
        ]
    )

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = TerminalLinkOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.account = "account1"
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)

    order_properties.time_in_force = TimeInForce.good_til_date(datetime(year, month, day))
    order_properties.account = "account2"
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

For more information about the format that the Bloomberg EMSX API expects, see Create Order and Route Extended Request in the EMSX API documentation and the createOrderAndRouteWithStrat documentation on the MathWorks website.

Get Open Orders

Terminal Link lets you access open orders .

Monitor Fills

Terminal Link allows you to monitor orders as they fill through order events .

Updates

Terminal Link doesn't support order updates , but you can cancel an existing order and then create a new order with the desired arguments. For more information about this workaround, see the Workaround for Brokerages That Don't Support Updates .

Cancellations

Terminal Link enables you to cancel open orders .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Brokerage-Side Orders

By default, your algorithm doesn't record orders that you submit to your account by third-parties instead of through LEAN. To accept these orders, create a custom brokerage message handler .

Fills

To view how we model fills in backtests, see Fills .

Slippage

To view how we model slippage in backtests, see Slippage .

Fees

To view how we model fees in backtests, see Fees .

Buying Power

To view how we model buying power in backtests, see Buying Power .

Settlement

To view how we model settlements in backtests, see Settlement .

Margin Interest Rate

To view how we model margin interest rates in backtests, see Margin Interest Rate .

Default Markets

To view the default markets of each asset class when you use Terminal Link, see Default Markets .

Account Currency

The Terminal Link brokerage doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

SSC Eze

Introduction

This page explains the `EzeBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```py
self.set_brokerage_model(BrokerageName.EZE, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation .

Asset Classes

The `EzeBrokerageModel` supports the following asset classes:

- US Equities
- Equity Options
- Futures
- Future Options
- Indices
- Index Options

Orders

The `EzeBrokerageModel` supports several order types, order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `EzeBrokerageModel` supports:

Order Type	Equity	Equity Options	Futures	Future Options	Index Options
Market	✓	✓	✓	✓	✓
Limit	✓	✓	✓	✓	✓
Stop market	✓	✓	✓	✓	✓
Stop limit	✓	✓	✓	✓	✓
Market on Open	✓				
Market on Close	✓				

Order Properties

The `EzeBrokerageModel` supports custom order properties. The following table describes the members of the `EzeOrderProperties` object that you can set to customize order execution.

Property	Data Type	Description	Default Value
time_in_force	TimeInForce	A TimeInForce instruction to apply to the order. The following instructions are supported: • DAY • GOOD_TIL_CANCELED • good_til_date	TimeInForce.GOOD_TIL_CANCELED
account	str	Sets a semi-colon separated list of trade or neutral accounts the user has permission for, e.g., "TAL;TEST;USER1;TRADE" or "TAL;TEST;USER2;NEUTRAL".	
account_type	str	Sets the account type for the order. E.g., **"119"** for margin orders in Eze EMS.	
notes	str	Sets the user message or notes.	
route	str	Sets the route name as shown in SS&C Eze EMS.	

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = EzeOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.notes = "Default order properties"

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = EzeOrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    order_properties.notes = "Default order properties"
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The EzeBrokerageModel supports order updates .

Handling Splits

If you're using raw data normalization and you have active orders with a limit, stop, or trigger price in the market for a US Equity when a stock split occurs, the following properties of your orders automatically adjust to reflect the stock split:

- Quantity
- Limit price
- Stop price
- Trigger price

Fills

The following table shows the fill model that the EzeBrokerageModel uses for each SecurityType :

SecurityType	Fill Model
Equity	EquityFillModel
Future	FutureFillModel
FutureOption	FutureOptionFillModel
Remaining SecurityType values	ImmediateFillModel

Slippage

The EzeBrokerageModel uses the NullSlippageModel .

Fees

The EzeBrokerageModel uses the EzeFeeModel which extends the ConstantFeeModel with with zero as fee for all assets.

Buying Power

The EzeBrokerageModel sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equities	SecurityMarginModel
Futures	FutureMarginModel
All Options Types	OptionMarginModel

If you have a margin account, the EzeBrokerageModel allows 2x leverage for Equities. The remaining asset classes are derivatives.

Settlement

The following table shows which settlement model the EzeBrokerageModel uses based on the security type and your account type:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the EzeBrokerageModel uses the ImmediateSettlementModel .

```py
# For US Equities with a cash account:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For Equity Options with a cash account:
security.set_settlement_model(DelayedSettlementModel(Option.DEFAULT_SETTLEMENT_DAYS, Option.DEFAULT_SETTLEMENT_TIME))

# For Futures
security.set_settlement_model(FutureSettlementModel())

# For remaining cases:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The EzeBrokerageModel uses the NullMarginInterestRateModel .

Default Markets

The following table describes the default markets of each `SecurityType` for the `EzeBrokerageModel` :

SecurityType	Market
Equity	USA
Option	USA
Future	CME
FutureOption	CME
Index	USA
IndexOption	USA

Supported Models

Trading Technologies

Introduction

This page explains the `TradingTechnologiesBrokerageModel`, including the asset classes it supports, its default security-level models, and its default markets.

```
self.set_brokerage_model(BrokerageName.TRADING_TECHNOLOGIES, AccountType.CASH)
self.set_brokerage_model(BrokerageName.TRADING_TECHNOLOGIES, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation.

Asset Classes

The `TradingTechnologiesBrokerageModel` supports trading Futures.

Orders

The `TradingTechnologiesBrokerageModel` supports several order types, some `TimeInForce` order properties, and order updates.

Order Types

The following table describes the available order types for each asset class that the `TradingTechnologiesBrokerageModel` supports:

Order Type	Futures
Market	✓
Limit	✓
Stop market	✓
Stop limit	✓

The `TradingTechnologiesBrokerageModel` enforces the following order rules:

- If you are buying (selling) with a `stop_market_order` or a `stop_limit_order`, the stop price of the order must be greater (less) than the current security price.
- If you are buying (selling) with a `stop_limit_order`, the limit price of the order must be greater (less) than the stop price.

Time In Force

The `TradingTechnologiesBrokerageModel` supports the `DAY` and `GOOD_TIL_CANCELED` TimeInForce order properties.

```
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    self.limit_order(self._symbol, quantity, limit_price)

    # Override the default order properties
    order_properties = OrderProperties()
    order_properties.time_in_force = TimeInForce.DAY
    self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Updates

The `TradingTechnologiesBrokerageModel` supports order updates.

Fills

The `TradingTechnologiesBrokerageModel` uses the FutureFillModel .

Slippage

The `TradingTechnologiesBrokerageModel` uses the NullSlippageModel .

Fees

The `TradingTechnologiesBrokerageModel` uses the ConstantFeeModel with zero fees.

```
security.set_fee_model(ConstantFeeModel(0))
```
PY

Buying Power

The `TradingTechnologiesBrokerageModel` uses the `FutureMarginModel` .

Settlement

The `TradingTechnologiesBrokerageModel` uses the FutureSettlementModel .

Margin Interest Rate

The `TradingTechnologiesBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `TradierBrokerageModel` is `Market.CME` .

Account Currency

The `TradingTechnologiesBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Wolverine

Introduction

This page explains the `WolverineBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```py
self.set_brokerage_model(BrokerageName.WOLVERINE, AccountType.CASH)
self.set_brokerage_model(BrokerageName.WOLVERINE, AccountType.MARGIN)
```

For more information about this model, see the class reference and implementation .

Asset Classes

The `WolverineBrokerageModel` supports trading US Equities .

Orders

The `WolverineBrokerageModel` supports one order type, but doesn't support order updates or extended market hours trading.

Order Types

The following table describes the available order types for each asset class that the `WolverineBrokerageModel` supports:

Order Type	Equity
Market	✓
Market on Close	✓
Limit	✓
Stop market	✓
Stop limit	✓

Updates

The `WolverineBrokerageModel` doesn't support order updates.

Extended Market Hours

The `WolverineBrokerageModel` doesn't support extended market hours trading. If you place an order outside of regular trading hours, the order is invalid.

Order Properties

The `WolverineBrokerageModel` supports custom order properties. The following table describes the members of the `WolverineOrderProperties` object that you can set to customize order execution:

Property	Description

`time_in_force`	`TimeInForce`
`exchange`	`Exchange`

exchange_post_fix	str
locate_broker	str
position_side	OrderPosition | NoneType

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = WolverineOrderProperties()
    self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
    self.default_order_properties.exchange = Exchange.SMART
    self.default_order_properties.exchange_post_fix = "-INCA-TX"
    self.default_order_properties.order_position = OrderPosition.BUY_TO_OPEN
    # Optionally, you can specify for equity shorts which broker will provide the stock to short
    # if not specified defaults to broker will provide
    self.default_order_properties.locate_broker = "abn"
```

If the `position_side` is not specified, the engine will determine it from the holdings and the order quantity. For example, if it's a sell short and will send `PositionSide SELL_SHORT` .

Fills

The `WolverineBrokerageModel` uses the EquityFillModel for Equity trades.

Slippage

The `WolverineBrokerageModel` uses the NullSlippageModel .

Fees

The `WolverineBrokerageModel` uses the WolverineFeeModel .

Buying Power

The `WolverineBrokerageModel` uses the `SecurityMarginModel` . If you have a margin account, the `WolverineBrokerageModel` allows up to 2x leverage.

Settlement

The `WolverineBrokerageModel` uses the ImmediateSettlementModel for margin accounts and the DelayedSettlementModel with the default settlement rules for cash accounts.

```py
# For cash accounts:
security.set_settlement_model(DelayedSettlementModel(Equity.DEFAULT_SETTLEMENT_DAYS, Equity.DEFAULT_SETTLEMENT_TIME))

# For margin accounts:
security.set_settlement_model(ImmediateSettlementModel())
```

Margin Interest Rate

The `WolverineBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `WolverineBrokerageModel` is `Market.USA` .

Account Currency

The `WolverineBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Supported Models

Oanda

Introduction

This page explains the `OandaBrokerageModel` , including the asset classes it supports, its default security-level models , and its default markets.

```
self.set_brokerage_model(BrokerageName.OANDA_BROKERAGE, AccountType.MARGIN)
```

PY

For more information about this model, see the class reference and implementation .

Asset Classes

The `OANDABrokerageModel` supports trading Forex and CFDs .

Orders

The `OandaBrokerageModel` supports several order types, the `TimeInForce` order property, and order updates.

Order Types

The following table describes the available order types for each asset class that the `OANDABrokerageModel` supports:

Order Type	Forex	CFD
Market	✓	✓
Limit	✓	✓
Stop market	✓	✓
Stop limit	✓	✓

Time In Force

The `OANDABrokerageModel` the `GOOD_TIL_CANCELED` TimeInForce .

```
self.default_order_properties.time_in_force = TimeInForce.GOOD_TIL_CANCELED
self.limit_order(self._symbol, quantity, limit_price)
```

PY

Updates

The `OANDABrokerageModel` supports order updates .

Fills

The `OandaBrokerageModel` uses the ImmediateFillModel .

Slippage

The `OandaBrokerageModel` uses the NullSlippageModel .

Fees

The `OandaBrokerageModel` uses the ConstantFeeModel with zero fees. OANDA offers "Spread" and "Core + Commission" pricing models. Our data is the spread dataset, so we model the spread pricing. To estimate OANDA Core+Commission data, use a custom fee model and a custom fill model .

```
security.set_fee_model(ConstantFeeModel(0))
```

PY

Buying Power

The `OandaBrokerageModel` uses the `SecurityMarginModel` and allows up to 50x leverage.

Settlement

The `OandaBrokerageModel` uses the ImmediateSettlementModel for Forex trades and the AccountCurrencyImmediateSettlementModel for CFD trades.

Margin Interest Rate

The `OandaBrokerageModel` uses the NullMarginInterestRateModel .

Default Markets

The default market of the `OandaBrokerageModel` is `Market.OANDA` .

Account Currency

The `OandaBrokerageModel` doesn't set a default currency. To change the algorithm's currency from USD to a different currency, see Set Account Currency .

Reality Modeling

Brokerage Message Handler

Introduction

When your brokerage sends a message, you receive this information in the algorithm and can take any actions. When you place an order through your brokerage platform instead of placing it through LEAN, you can decide whether the transaction handler should process the order. By default, the transaction handler only process orders that you place through LEAN. Custom brokerage message handlers enable you to instruct the transaction handler to process orders that you create directly through your brokerage's platform.

Set Models

To set a brokerage message handler, in the `initialize` method, call the `set_brokerage_message_handler` method.

```py
def initialize(self) -> None:
    # Set a custom brokerage message handler for the algorithm in Initialize method
    # It handles different type of returned message from the broker, helping you filter or abstract for the ones you care about
    self.set_brokerage_message_handler(MyBrokerageMessageHandler(self))
```

Default Behavior

The default brokerage message handler is the `DefaultBrokerageMessageHandler` . The following table describes how the `DefaultBrokerageMessageHandler` processes brokerage messages:

Brokerage Message Type	Action
`BrokerageMessageType. INFORMATION`	Sends a debug message for the algorithm.
`BrokerageMessageType. WARNING`	Sends an error message for the algorithm.
`BrokerageMessageType. ERROR`	Sets the algorithm runtime error and stops it.
`BrokerageMessageType. DISCONNECT`	Stops the algorithm after 15 minutes if the market is open. Otherwise, it stops five minutes after market open.
`BrokerageMessageType. RECONNECT`	Cancels the disconnection process.

The default brokerage message handler rejects orders you create through the brokerage. An example of this is when you place an order using the brokerage website or a third-party software.

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`algorithm`	`IAlgorithm`	The running algorithm.	
`initial_delay`	`timedelta/NoneType`	The amount of time LEAN will wait after a brokerage disconnection message for a reconnection message. If the reconnection message doesn't arrive before the time limit, LEAN stops running. If you don't provide a value, it uses `timedelta(minutes=15)` .	`None`
`open_threshold`	`timedelta/NoneType`	Defines how long before market open to re-check for brokerage reconnect message. If you don't provide a value, it uses `timedelta(minutes=5)` .	`None`

For more information about this model, see the class reference and implementation .

Model Structure

Brokerage message handlers extend the `DefaultBrokerageMessageHandler` class. Extensions of the `DefaultBrokerageMessageHandler` class should define the `handle_message` and `handle_order` methods.

The `handle_message` method processes the brokerage message event. It triggers any actions in the algorithm or notifications system required.

The `handle_order` method defines whether the transaction handler should process a new order you placed directly through the brokerage's website or third-party software.

```
PY
class CustomBrokerageMessageHandlerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the brokerage message handler
        self.set_brokerage_message_handler(MyBrokerageMessageHandler(self))

# Define the custom brokerage message handler
class MyBrokerageMessageHandler(DefaultBrokerageMessageHandler):

    def __init__(self, algorithm):
        self._algorithm = algorithm

    def handle_message(self, message: BrokerageMessageEvent) -> None:
        self._algorithm.debug(f"{self._algorithm.time} Event: {message.message}")

    def handle_order(self, event_args: NewBrokerageOrderNotificationEventArgs) -> bool:
        return False
```

The `BrokerageMessageEvent` class has the following properties:

The `NewBrokerageOrderNotificationEventArgs` class has the following properties:

Brokerage Support

Most QuantConnect brokerage integrations don't support brokerage-side orders. If you create an order directly through your brokerage instead of through LEAN, your algorithm won't process the order because the return value of the `handle_order` method is ignored.

Terminal Link is the exception that can support brokerage-side orders. The following algorithm demonstrates how to implement a custom brokerage handler that accepts orders with a target `TerminalLinkOrderProperties.custom_notes_1` value.

```
PY
class CustomBrokerageSideOrderHandlingAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2013, 10, 7)
        self.set_end_date(2013, 10, 11)
        self.set_brokerage_message_handler(CustomBrokerageMessageHandler(self))
        self.brokerage_side_order = None
        self.schedule.on(self.date_rules.every_day(), self.time_rules.every(timedelta(minutes=1)), self.check_brokerage_side_order)

    def check_brokerage_side_order(self):
        if not self.brokerage_side_order:
            return
        open_orders = self.transactions.get_open_orders()
        brokerage_order = next((x for x in open_orders if x.broker_id[0] == self.brokerage_side_order.broker_id[0]), None)
        if not brokerage_order:
            return
        if not self.securities.contains_key(brokerage_order.symbol):
            self.log(f"Security {brokerage_order.symbol} not found in algorithm's securities!")
        self.log(f"{self.time} :: Brokerage-side order found: {brokerage_order}")

class CustomBrokerageMessageHandler(DefaultBrokerageMessageHandler):
    def __init__(self, algorithm):
        super().__init__(algorithm)
        self._algorithm = algorithm

    def handle_order(self, event_args):
        order = event_args.order
        self._algorithm.brokerage_side_order = order

        # Only accept TerminalLink orders with a custom note "AcceptOrder".
        if not isinstance(order.properties, TerminalLinkOrderProperties):
            self._algorithm.log("handle_order(): order properties is not set to expected terminal link type, skipping")
            return False
        custom_notes1 = order.properties.custom_notes1
        result = custom_notes1 and "AcceptOrder" in custom_notes1
        self._algorithm.log(f"handle_order(): returning: {result}")
        return result
```

Examples

The following examples demonstrate common practices for implementing a custom brokerage message handler.

Example 1: Notification

The following algorithm buys and holds SPY. It implemented a custom brokerage message handler to send notifications to users through email and SMS in cases of disconnection, reconnections, and outside trades.

```py
class CustomBrokerageMessageHandlerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # To simulate the connection to IB brokerage.
        self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE, AccountType.MARGIN)
        # Set the brokerage message handler to our custom model.
        self.set_brokerage_message_handler(CustomBrokerageMessageHandler(self))

        # Request SPY data for trading.
        self.spy = self.add_equity("SPY").symbol

    def on_data(self, slice: Slice) -> None:
        # Buy and hold SPY as a strategy.
        if not self.portfolio.invested:
            self.set_holdings(self.spy, 1)

class CustomBrokerageMessageHandler(DefaultBrokerageMessageHandler):
    def __init__(self, algorithm: QCAlgorithm) -> None:
        self._algorithm = algorithm

    def handle_message(self, message: BrokerageMessageEvent) -> None:
        # Notify the user by SMS of the disconnection of the algorithm instance from the brokerage.
        if message.type == BrokerageMessageType.DISCONNECT:
            self._algorithm.notify.sms(
                phone_number="<YOUR_PHONE_NUMBER>",
                message="Your QC Algorithm instance 'CustomBrokerageMessageHandlerAlgorithm' was disconnected from the IB brokerage."
            )
        # Notify the user by SMS on reconnection of the algorithm instance to the brokerage after disconnection.
        elif message.type == BrokerageMessageType.RECONNECT:
            self._algorithm.notify.sms(
                phone_number="<YOUR_PHONE_NUMBER>",
                message="Your QC Algorithm instance 'CustomBrokerageMessageHandlerAlgorithm' was reconnected to the IB brokerage."
            )

    def handle_order(self, event_args: NewBrokerageOrderNotificationEventArgs) -> bool:
        # Notify the user by email on receiving an order from outside the algorithm.
        self._algorithm.notify.email(
            address="<YOUR_EMAIL_ADDRESS@example.com>",
            subject="Outside Order",
            message="Your QC Algorithm instance 'CustomBrokerageMessageHandlerAlgorithm' detected an outside order from the IB brokerage;
it has been processed to the algorithm."
        )
        # Process a new order placed through the brokerage's website or third-party software to handle it in the algorithm instance.
        return True
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomBrokerageSideOrderHandlingRegressionAlgorithm.py Python

Reality Modeling

Buying Power

Introduction

Buying power models (also known as margin models) control how much buying power or leverage your portfolio has to make trades. When you place an order, LEAN uses the buying power model to determine whether the order should be submitted so you avoid placing orders that would be rejected by the brokerage. Buying power calculations can be very complex and depend on many factors, including the brokerage or even the time of day. For example, the `PatternDayTradingMarginModel` lets you have 4x leverage during regular trading hours and 2x leverage during pre-market and post-market trading hours.

What is Buying Power?

Buying power is the total amount of money you can spend in your brokerage account. It depends on the security type and account type. On one hand, Option and Future contracts are leveraged securities with specific buying power rules. On the other hand, the buying power of cash accounts is just the cash in the account while the buying power of margin accounts is leveraged by the brokerage credit.

What is Margin?

Margin is a credit loan from your brokerage you receive after you deposit collateral into your margin account. You need margin to place short-biased trades and you can use margin to increase your buying power, but you incur interest fees. Margin is the dollar value of the loan that the brokerage gives you. A margin requirement of 25% means that to purchase 10,000 worth of securities, you need at least 2,500 worth of collateral in your brokerage account to open the trade and you can borrow the rest on margin. Maintenance margin is the minimum equity (equity = total portfolio value - borrowed funds) you must have in your brokerage account to stay in your positions. If the value of your portfolio falls below the maintenance margin, you receive a margin call . If you receive a margin call, you either need to add more capital to your brokerage account or the brokerage will liquidate some of your holdings to reduce your exposure and their risk.

Some securities have special margin rules. Derivatives are leveraged assets with a floating margin requirement. In particular, long Options have zero maintenance margin requirement and their initial margin requirement is only the premium that you pay upfront.

What is Leverage?

Leverage is using borrowed money to increase your buying power. Leverage has an inverse relationship with your margin requirement and maintenance margin. If you have a margin requirement of 50%, you can use up to 1 / 50% = 2 leverage. Trading with leverage can be risky. It can boost your returns on profitable trades but can make your losing trades more expensive. If you have 10,000 in your brokerage margin account and purchase 20,000 worth of securities, you are trading with 2x leverage. If the value of the securities in your portfolio drops by 50% when you have a 2x leverage position, you lose all of your equity.

What Are Position Groups?

A position group is the combination of holdings. It has lower margin requirement and maintenance margin than the sum of each position of the group. If you have a margin requirement of 29,150 to purchase an in-the-money call Option contract, and a margin requirement of 101,499 to sell an out-of-the money call Option contract, the total margin requirement is $130,649. However, these positions compose a bull call spread with a margin requirement of $0.

Set Models

The brokerage model of your algorithm automatically sets the buying power model for each security, but you can override it. To manually set the buying power model of a security, call the `set_buying_power_model` method on the `Security` object.

```
PY
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # Use 3x leverage on the selected security, which mimic the actual buying power of your broker
    security.set_buying_power_model(BuyingPowerModel(3))
```

You can also set the buying power model in a security initializer . If your algorithm has a universe, use the security initializer technique. In order to

initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_buying_power_model(BuyingPowerModel(3))
```

PY

You cannot change the position group buying power models.

Supported Models

The following buying power models are available:

- BuyingPowerModel
- CashBuyingPowerModel
- ConstantBuyingPowerModel
- CryptoFutureMarginModel
- NullBuyingPowerModel
- FutureMarginModel
- FuturesOptionsMarginModel
- OptionMarginModel
- PatternDayTradingMarginModel

The following position group buying power models are available:

- SecurityPositionGroupBuyingPowerModel
- OptionStrategyPositionGroupBuyingPowerModel

The SecurityMarginModel is an alias to the BuyingPowerModel .

Default Behavior

The brokerage model of your algorithm automatically sets the buying power model of each security. The default brokerage model is the `DefaultBrokerageModel` , which sets the buying power model based on the asset class of the security. The following table shows the default buying power model of each asset class:

Asset Class	Model
Equity Options	`OptionMarginModel`
Futures	`FutureMarginModel`
Future Options	`FuturesOptionsMarginModel`
Index Options	`OptionMarginModel`
Crypto	`CashBuyingPowerModel` for cash accounts or `BuyingPowerModel` for margin accounts
CryptoFuture	`CryptoFutureMarginModel`
Forex	`CashBuyingPowerModel` for cash accounts or `BuyingPowerModel` for margin accounts
Other	`BuyingPowerModel`

Model Structure

Buying power models should extend the `BuyingPowerModel` class. Extensions of the `BuyingPowerModel` class should implement the following methods:

```
class CustomBuyingPowerModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_equity("SPY")
        # Set the custom buying power model of the selected security to mimic its actual buying power in your used broker
        security.set_buying_power_model(MyBuyingPowerModel())

# Define the custom buying power model
class MyBuyingPowerModel(BuyingPowerModel):
    def __init__(self,
                 leverage: float = 2,
                 requiredFreeBuyingPowerPercent: float = 0):
        super().__init__(leverage, requiredFreeBuyingPowerPercent)

    def get_leverage(self, security: Security) -> float:
        return super().get_leverage(security)

    def set_leverage(self, security: Security, leverage: float) -> None:
        super().set_leverage(security, leverage)

    def get_initial_margin_required_for_order(self,
                 parameters: InitialMarginRequiredForOrderParameters) -> InitialMargin:
        return super().get_initial_margin_required_for_order(parameters)

    def get_maintenance_margin(self,
                 parameters: MaintenanceMarginParameters) -> MaintenanceMargin:
        return super().get_maintenance_margin(parameters)

    def get_margin_remaining(self,
                 portfolio: SecurityPortfolioManager,
                 security: Security,
                 direction: OrderDirection) -> float:
        return super().get_margin_remaining(portfolio, security, direction)

    def get_initial_margin_requirement(self,
                 parameters: InitialMarginParameters) -> InitialMargin:
        return super().get_initial_margin_requirement(parameters)

    def has_sufficient_buying_power_for_order(self,
                 parameters: HasSufficientBuyingPowerForOrderParameters
                ) -> HasSufficientBuyingPowerForOrderResult:
        return super().has_sufficient_buying_power_for_order(parameters)

    def get_maximum_order_quantity_for_delta_buying_power(self,
                 parameters: GetMaximumOrderQuantityForDeltaBuyingPowerParameters
                ) -> GetMaximumOrderQuantityResult:
        return super().get_maximum_order_quantity_for_delta_buying_power(parameters)

    def get_maximum_order_quantity_for_target_buying_power(self,
                 parameters: GetMaximumOrderQuantityForTargetBuyingPowerParameters
                ) -> GetMaximumOrderQuantityResult:
        return super().get_maximum_order_quantity_for_target_buying_power(parameters)

    def get_reserved_buying_power_for_position(self,
                 parameters: ReservedBuyingPowerForPositionParameters
                ) -> ReservedBuyingPowerForPosition:
        return super().get_reserved_buying_power_for_position(parameters)

    def get_buying_power(self,
                 parameters: BuyingPowerParameters) -> BuyingPower:
        return super().get_buying_power(parameters)
```

Disable Buying Power Models

You can disable order margin checks and opt to let your brokerage decide to accept or reject the trades. This is helpful in live trading if you have a more permissive brokerage margin allowance that what LEAN models. The default position group buying power models are helpful for Option trading strategies. However, it can be counterproductive if it's not a supported Option strategy . To disable the validations of the default position group buying power model, use the `NullSecurityPositionGroupModel` . To set the `NullSecurityPositionGroupModel` for the portfolio, during initialization , call the `set_positions` method with the `SecurityPositionGroupModel. NULL` argument.

```
self.portfolio.set_positions(SecurityPositionGroupModel.NULL)
```

To disable the validations of the default buying power model , use the `NullBuyingPowerModel` . To set the `NullBuyingPowerModel` for a security subscription, call the `set_buying_power_model` method with the `BuyingPowerModel. NULL` argument.

```
def initialize(self) -> None:
    equity = self.add_equity("SPY")
    # To disable any buying power of the selected security
    equity.set_buying_power_model(BuyingPowerModel.NULL)
    # Alias:
    # equity.set_margin_model(SecurityMarginModel.NULL)
```

You can also set the `NullBuyingPowerModel` in a security initializer . If your algorithm has a universe, use the security initializer technique. To set the buying power of securities in the security initializer, set the brokerage model , set the security initializer , and then create security subscriptions.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the security buying power
        security.set_buying_power_model(BuyingPowerModel.NULL)
```

Set Asset Leverage

The buying power model sets the leverage for each security in your algorithm, but you can override its leverage settings after the buying power model is set.

To set the leverage when you create a security subscription, pass in a `leverage` argument.

```
def initialize(self) -> None:
 # Set the leverage to 3x manually subjected to specific need
 self.add_equity("SPY", leverage=3)
```

You can also set the asset leverage in a security initializer. In order to set the leverage of securities in the security initializer, call `add_security_initializer` before you create security subscriptions and after you call `set_brokerage_model` . If you pass in a `leverage` argument when you create the security subscription, the `leverage` argument takes precedence over the `set_leverage` call in the security initializer.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the security leverage
        security.set_leverage(3)
```

To set the leverage for all securities in a universe, set the `universe_settings.leverage` property.

```
def initialize(self) ->None:
 # Set the leverage to 3x to all securities, it should only be used for a very narrow spectrum of security universe with the same leverage
 # E.g. US Equities universe composed of only primary stocks in NYSE exchange
 self.universe_settings.leverage = 3
```

In live trading, LEAN doesn't ignore the leverage you set. However, if you set a different leverage from what your brokerage provides, it creates a mismatch between the buying power in your algorithm and the buying power the brokerage gives you. In this case, orders can pass the validations in LEAN but your brokerage may reject them.

PDT Rule

If all of the following statements are true, you are classified as a pattern day trader:

- You reside in the United States.
- You trade in a margin account.
- You execute 4+ intraday US Equity trades within 5 business days.
- Your intraday US Equity trades represent more than 6% of your total trades.

Pattern day traders must maintain a minimum equity of $25,000 in their margin account to continue trading. For more information about pattern day trading, see Am I a Pattern Day Trader? on the FINRA website.

The `PatternDayTradingMarginModel` doesn't enforce minimum equity rules and doesn't limit your trades, but it adjusts your available leverage based on the market state. During regular market hours, you can use up to 4x leverage. During extended market hours, you can use up to 2x leverage. This model is not automatically enforced, see Live Trading Considerations . You must manually set it on your securities.

```
security.margin_model = PatternDayTradingMarginModel()
```

Get Initial Margin Requirements

The following sections explain how to check if you have enough margin remaining to cover the initial margin requirements of various order types.

Check Requirements of Regular Orders

Follow these steps to check if you have enough margin remaining to cover the initial margin requirements of an order:

1. Create an `InitialMarginParameters` object with the security and quantity you want to trade.

   ```
   def on_data(self, slice: Slice) -> None:
       security = self.securities["SPY"]
       quantity = 100
       parameter = InitialMarginParameters(security, quantity)
   ```

2. Call the `get_initial_margin_requirement` method of the security's buying power model with the `InitialMarginParameters` .

   ```
       initial_margin = security.buying_power_model.get_initial_margin_requirement(parameter)
   ```

   The `get_initial_margin_requirement` method returns an `InitialMargin` object, which have the following properties:

3. Compare the margin you have remaining against the initial margin requirement of the order.

   ```
       if self.portfolio.margin_remaining >= initial_margin.value:
           self.market_order(security.symbol, quantity)
       else:
           self.debug("You don't have sufficient margin for this order.")
   ```

Check Requirements of Option Strategy Orders

Follow these steps to check if you have enough margin remaining to cover the initial margin requirements of an Option strategy :

1. Create an `OptionStrategy` object with the strategy you want to trade and its buying power model.

   For example, create a Bull Put Spread strategy.

   ```
   def initialize(self) -> None:
       # Subscribe to option data and cache the canonical symbol to obtain the option data
       self._symbol = self.add_option("SPY").symbol

   def on_data(self, slice: Slice) > None:
       # Trade on updated option chain data
       chain = slice.option_chains.get(self._symbol)
       if not chain:
           return

       itm_strike = max(x.strike for x in chain)
       otm_strike = min(x.strike for x in chain)
       expiry = min(x.expiry for x in chain)

       option_strategy = OptionStrategies.bull_put_spread(self._symbol, itm_strike, otm_strike, expiry)
   ```

2. Create an `OptionStrategyPositionGroupBuyingPowerModel` object of the strategy.

```
buying_power_model = OptionStrategyPositionGroupBuyingPowerModel(option_strategy)
```

3. Create a list of `Position` objects from the strategy legs.

```
def get_symbol(leg):
    return Symbol.CreateOption(
        self._symbol.underlying, self._symbol.id.market, self._symbol.id.option_style,
        leg.right, leg.strike, leg.expiration
    )

positions = [Position(get_symbol(leg), leg.quantity, 1) for leg in option_strategy.option_legs]
```

4. Create a `PositionGroupInitialMarginParameters` objects with the portfolio and the position group.

```
position_group = PositionGroup(buying_power_model, 1, positions)
parameters = PositionGroupInitialMarginParameters(self.portfolio, position_group)
```

5. Call the `get_initial_margin_requirement` method of the strategy's buying power model with the `PositionGroupInitialMarginParameters` .

```
initial_margin = buying_power_model.get_initial_margin_requirement(parameters)
```

   The `get_initial_margin_requirement` method returns an `InitialMargin` object, which have the following properties:

6. Compare the margin you have remaining against the initial margin requirement of the order.

   In this case, the `initial_margin.value` is the initial margin requirement for a trade size of 1. For different trade sizes, multiply the `initial_margin.value` by the quantity.

```
strategy_quantity = 2
if self.portfolio.margin_remaining >= strategy_quantity * initial_margin.value:
    self.buy(option_strategy, strategy_quantity)
else:
    self.debug("You don't have sufficient margin for this order.")
```

Live Trading Considerations

LEAN doesn't infer account type, leverage, margin requirements, or pattern day trading rules from your brokerage account. For example, if you have a cash account, and your algorithm doesn't set `AccountType. CASH` , LEAN will not automatically enforce cash account restrictions such as leverage of 1x or immediate settlement. You must manually configure your algorithm to adhere to these rules. If you set the leverage to 10x and you have a leverage of 2x, LEAN will place orders that can be rejected by your brokerage.

The default leverage of U.S. Equities is 2x. However, your brokerage can reduce the leverage of some securities based on market conditions. For example, leveraged ETFs such as TQQQ might have lower leverage during periods of high volatility. If your algorithm doesn't account for this, LEAN doesn't account for it and your orders can be rejected by your brokerage.

Examples

The following examples demonstrate some common practices for implementing a custom buying power model.

Example 1: Always Sufficient Buying Power

The following algorithm trades protective call strategy. Sometimes, the brokerage's margin requirement differs from the algorithm's default. To avoid errors on order submission at the algorithm level, we can use the `SecurityPositionGroupModel.NULL` and `NullBuyingPowerModel` to disable buying power limitation so the order is sent directly to the broker.

```py
class BuyingPowerModelAlgorithm(QCAlgorithm):
    def Initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)

        # Disable the validations of the default position group buying power model
        # that evaluates the buying power of multi-leg strategies
        self.portfolio.set_positions(SecurityPositionGroupModel.NULL)

        # Request SPY data for trading. Set the data normalization mode to raw to allow for a fair comparison in strike price.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Set a scheduled event to rebalance the protective put strategy.
        self.schedule.on(
            self.date_rules.week_start(self.spy),
            self.time_rules.after_market_open(self.spy, 0),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Select the weekend expiring ATM put for the protective put strategy.
        chain = self.option_chain(self.spy)
        filtered = [x for x in chain if x.right == OptionRight.PUT and x.expiry <= self.time + timedelta(6)]
        if not filtered:
            return
        atm_put_contract = sorted(filtered,
            key=lambda x: (-abs(x.strike - x.underlying_last_price), x.expiry),
            reverse=True)[0]
        # Request the ATM put data for trading.
        atm_put = self.add_option_contract(atm_put_contract).symbol

        if self.portfolio[self.spy].invested:
            # If SPY is not assigned, we only need to buy the put.
            self.market_order(atm_put, 1, tag="Protective Put")
        else:
            # Order the protective put strategy by combo order.
            self.combo_market_order([
                    Leg.create(self.spy, 100),
                    Leg.create(atm_put, 1)
                ],
                1,
                tag="Protective Put"
            )

    def _custom_security_initializer(self, security: Security):
        # Do not allow buying power for options; only a hedging strategy with 0 margin is allowed.
        security.SetBuyingPowerModel(NullBuyingPowerModel())
```

Example 2: Default Model Wrapper

The following algorithm defines a subclass of the default `BuyingPowerModel` and overrides its built-in members so you can easily customize the parts you need.

PY

```
class CustomBuyingPowerModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        security = self.add_equity('SPY', Resolution.DAILY)
        security.set_buying_power_model(MyBuyingPowerModel())

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyBuyingPowerModel(BuyingPowerModel):

    def __init__(self,
         leverage: float = 2,
         required_free_buying_power_percent: float = 0):
        super().__init__(leverage, required_free_buying_power_percent)

    def get_leverage(self, security: Security) -> float:
        return super().get_leverage(security)

    def set_leverage(self, security: Security, leverage: float) -> None:
        super().set_leverage(security, leverage)

    def get_initial_margin_required_for_order(self,
         parameters: InitialMarginRequiredForOrderParameters) -> InitialMargin:
        return super().get_initial_margin_required_for_order(parameters)

    def get_maintenance_margin(self,
         parameters: MaintenanceMarginParameters) -> MaintenanceMargin:
        return super().get_maintenance_margin(parameters)

    def get_margin_remaining(self,
         portfolio: SecurityPortfolioManager,
         security: Security,
         direction: OrderDirection) -> float:
        return super().get_margin_remaining(portfolio, security, direction)

    def get_initial_margin_requirement(self,
         parameters: InitialMarginParameters) -> InitialMargin:
        return super().get_initial_margin_requirement(parameters)

    def has_sufficient_buying_power_for_order(self,
         parameters: HasSufficientBuyingPowerForOrderParameters
        ) -> HasSufficientBuyingPowerForOrderResult:
        return super().has_sufficient_buying_power_for_order(parameters)

    def get_maximum_order_quantity_for_delta_buying_power(self,
         parameters: GetMaximumOrderQuantityForDeltaBuyingPowerParameters
        ) -> GetMaximumOrderQuantityResult:
        return super().get_maximum_order_quantity_for_delta_buying_power(parameters)

    def get_maximum_order_quantity_for_target_buying_power(self,
         parameters: GetMaximumOrderQuantityForTargetBuyingPowerParameters
        ) -> GetMaximumOrderQuantityResult:
        return super().get_maximum_order_quantity_for_target_buying_power(parameters)

    def get_reserved_buying_power_for_position(self,
         parameters: ReservedBuyingPowerForPositionParameters
        ) -> ReservedBuyingPowerForPosition:
        return super().get_reserved_buying_power_for_position(parameters)

    def get_buying_power(self,
         parameters: BuyingPowerParameters) -> BuyingPower:
        return super().get_buying_power(parameters)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomBuyingPowerModelAlgorithm.py Python CustomModelsAlgorithm.py Python NullBuyingPowerOptionBullCallSpreadAlgorithm.py Python

Reality Modeling

Settlement

Settlement

Key Concepts

Introduction

After you trade an asset, the brokerage needs to settle the funds in your account. The most common type of settlement is immediate, where the funds are immediately available for trading after the transaction. In some cases, you may have delayed settlement, where you sell an asset and need to wait a few days to spend the cash you receive from the sale. A settlement model simulates these settlement rules.

Set Models

The brokerage model of your algorithm automatically sets the settlement model for each security, but you can override it. To manually set the settlement model of a security, call the `set_settlement_model` method on the `Security` object.

```py
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # Set a delayed settlement model that settles 7 days after the trade at 8 AM
    # This can better mimic actual settlement of some brokerage, providing more realistic fund and margin available
    security.set_settlement_model(DelayedSettlementModel(7, timedelta(hours=8)))
```

You can also set the settlement model in a security initializer . If your algorithm has a universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_settlement_model(DelayedSettlementModel(7, timedelta(hours=8)))
```

To view all the pre-built settlement models, see Supported Models .

Default Behavior

The brokerage model of your algorithm automatically sets the settlement model for each security. The default brokerage model is the `DefaultBrokerageModel` , which sets the settlement model based on the security type and your account type. The following table shows how it sets the settlement models:

Security Type	Account Type	Settlement Model
Equity	Cash	DelayedSettlementModel with the default settlement rules
Option	Cash	DelayedSettlementModel with the default settlement rules
Future	Any	FutureSettlementModel

For all other cases, the `DefaultBrokerageModel` uses the ImmediateSettlementModel .

The default delayed settlement rule for US Equity trades is T+2 at 8 AM Eastern Time (ET). For example, if you sell on Monday, the trade settles on Wednesday at 8 AM. The default delayed settlement rule for Future and Option contracts is T+1 at 8 AM.

Model Structure

Settlement models must extend the `ISettlementModel` interface. Extensions of the `ISettlementModel` interface must implement the `scan` and `apply_funds` methods. The `scan` method is automatically called at the top of each hour and it receives a `ScanSettlementModelParameters` object. The `apply_funds` method receives an `ApplyFundsSettlementModelParameters` object and applies the settlement rules. The `apply_funds` method is also automatically called, but its frequency depends on the security type. The `apply_funds` method is automatically called when you fill an order for the following security types:

- Equity
- Equity Options
- Crypto
- Forex
- Future Options
- Index Options

The `apply_funds` method is automatically called when you close a position for the following security types:

- Futures
- Crypto Futures
- CFD

```py
class CustomSettlementModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_equity("SPY")
        # Set the custom settlement model of the selected security to reflect actual scenario for more realistic margin available
        security.set_settlement_model(MySettlementModel())

# Define the custom settlement model outside of the algorithm
class MySettlementModel:
    def apply_funds(self, applyFundsParameters: ApplyFundsSettlementModelParameters) -> None:
        currency = applyFundsParameters.cash_amount.currency
        amount = applyFundsParameters.cash_amount.amount
        applyFundsParameters.portfolio.cash_book[currency].add_amount(amount)

    def scan(self, settlementParameters: ScanSettlementModelParameters) -> None:
        pass

    def get_unsettled_cash(self) -> CashAmount:
        return CashAmount(0, 'USD')
```

`ApplyFundsSettlementModelParameters` objects have the following properties:

`ScanSettlementModelParameters` objects have the following properties:

You likely don't need to create a custom settlement model because the supported models already implement immediate and delayed settlement rules.

Examples

The following examples demonstrate some common practices for implementing the settlement model.

Example 1: 2-day Settlement

The following algorithm simulates Interactive Brokers cash account. According to IB's website, cash positions will be settled in 48 hours. Thus, we can implement `DelayedSettlementModel` to mimic that. We hold the intra-day position of SPY to implement the settlement logic.

```py
# region imports
from AlgorithmImports import *
# endregion

class SettlementModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Simulate a cash account of IB.
        self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE, AccountType.CASH)
        # We need to set the settlement model in the security initializer.
        self.set_security_initializer(CustomSecurityInitializer(self.brokerage_model, SecuritySeeder.NULL))

        # Request SPY data for trading.
        self.spy = self.add_equity("SPY").symbol

    def on_data(self, slice: Slice) -> None:
        # Trade based on updated data.
        bar = slice.quote_bars.get(self.spy)
        if bar and not self.portfolio.invested:
            # Order based on the current available cash.
            quantity = (self.portfolio.cash - self.portfolio.unsettled_cash) //  bar.ask.close
            if quantity > 0:
                self.market_order(self.spy, quantity)

        # Exit position before market close; we only hold the intraday position.
        if slice.time.hour == 15 and slice.time.minute == 45:
            self.liquidate()

class CustomSecurityInitializer(BrokerageModelSecurityInitializer):
    def __init__(self, brokerage_model: IBrokerageModel, security_seeder: ISecuritySeeder):
        super().__init__(brokerage_model, security_seeder)
    def initialize(self, security: Security) -> None:
        super().initialize(security)
        # IB settles cash in 48 hours.
        security.set_settlement_model(DelayedSettlementModel(2, timedelta(hours=0)))
```

Example 2: No Unsettled Cash

The following algorithm implements a custom settlement model that states there is always $0 of unsettled cash.

```py

class CustomSettlementModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        security = self.add_equity('SPY')
        security.set_settlement_model(MySettlementModel())

        self.schedule.on(
            self.date_rules.every_day('SPY'),
            self.time_rules.every(timedelta(minutes=30)),
            self._plot_cash
        )

        self._traded = False

    def _plot_cash(self):
        self.Plot("Settled Cash", "USD", self.Portfolio.CashBook["USD"].Amount)
        self.Plot("Unsettled Cash", "USD", self.Portfolio.UnsettledCashBook["USD"].Amount)

    def on_data(self, data: Slice) -> None:
        if self._traded:
            return

        self.market_order('SPY', 1)
        self.market_order('SPY', -1)
        self._traded = True


class MySettlementModel:

    def apply_funds(self, apply_funds_parameters: ApplyFundsSettlementModelParameters) -> None:
        currency = apply_funds_parameters.cash_amount.currency
        amount = apply_funds_parameters.cash_amount.amount
        apply_funds_parameters.portfolio.cash_book[currency].add_amount(amount)

    def scan(self, settlement_parameters: ScanSettlementModelParameters) -> None:
        pass

    def get_unsettled_cash(self) -> CashAmount:
        return CashAmount(0, 'USD')
```

Settlement

Supported Models

Introduction

This page describes all of the pre-built settlement models in LEAN. For more brokerage-specific settlement models, see the brokerage model documentation . If none of these models perform exactly how you want, create a custom settlement model .

Immediate Model

The `ImmediateSettlementModel` immediately adds or removes the cash from your portfolio when your transactions fill.

```
security.set_settlement_model(ImmediateSettlementModel())
```

PY

For more information about this model, see the class reference and implementation .

Delayed Model

The `DelayedSettlementModel` immediately removes the cash from your portfolio when your buy orders fill. When your sell orders fill, it adds the cash to your unsettled cash book . When the settlement period ends, the unsettled cash is added to your portfolio.

```
security.set_settlement_model(DelayedSettlementModel(7, timedelta(hours=8)))
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`number_of_days`	`int`	The number of days required for settlement	
`time_of_day`	`timedelta`	The time of day used for settlement	

For more information about this model, see the class reference and implementation .

Account Currency Immediate Model

The `AccountCurrencyImmediateSettlementModel` applies cash settlement immediately and automatically converts the settlement cash into the account currency.

```
security.set_settlement_model(AccountCurrencyImmediateSettlementModel())
```

PY

For more information about this model, see the class reference and implementation .

Future Model

The `FutureSettlementModel` settles the daily profit and loss at the start of each day.

```
security.set_settlement_model(FutureSettlementModel())
```

PY

For more information about this model, see the class reference and implementation .

Reality Modeling

Options Models

Options Models

Pricing

Introduction

Option price models compute the theoretical price of Option contracts, their implied volatility (IV), and their Greek values. Theoretical prices can help you detect undervalued and overvalued contracts, IV can provide you insight into the upcoming volatility of the underlying security, and Greek values can help you hedge your portfolio.

Data Discrepancies

There are several ways for you to get the Greeks and IV values, but you may notice some discrepancies between them.

Greeks in Universe Selection

The Greeks and IV values in the universe filter function are the daily, pre-calculated values based on the end of the previous trading day. You can't customize the Greeks and IV values that the filter function receives.

Greeks in Chain Requests

The Greeks and IV values in the `OptionUniverse` objects that you get from calling the `option_chain` method are the daily, pre-calculated values based on the end of the previous trading day. You can't customize the Greeks and IV values that you get from this method.

Greeks in Data Events

The Greeks and IV values in the Slice that you get in the `on_data` method are the values from the Option price model. To customize the Greeks and IV values in the `Slice` , set a different price model.

Set Models

To set the price model of an Option, set its `price_model` property.

If you have access to the `Option` object when you subscribe to the Option universe or contract, you can set the price model immediately after you create the subscription.

```
PY
def initialize(self):
    self.universe_settings.asynchronous = True
    option = self.add_option("SPY")
    # SPY options are American Options, in which IV and greeks more accurately calculated using Binomial model
    option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

Otherwise, set the price model in a security initializer .

```
PY
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the price model
        if security.type == SecurityType.OPTION: # Option type
            security.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
```

Default Behavior

The default Option price model is the `OptionPriceModels.binomial_cox_ross_rubinstein` model for American Options or `OptionPriceModels.black_scholes` model for European Options.

Model Structure

Option price models should extend the `OptionPriceModel` class. Extensions of the `OptionPriceModel` class must implement the `evaluate` method, which receives `OptionPriceModelParameters` and returns an `OptionPriceModelResult` that contains the theoretical price, the implied volatility, and Greeks.

```
PY
class CustomOptionPriceModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the custom price model for an added option chain to use the custom model
        option = self.add_option("SPY")
        option.set_price_model(CustomOptionPriceModel())

# Define the custom price model
class CustomOptionPriceModel(CurrentPriceOptionPriceModel):
    def evaluate(self, parameters: OptionPriceModelParameters) -> OptionPriceModelResult:
        contract = parameters.contract
        underlying = contract.underlying_last_price
        strike = contract.strike
        greeks = Greeks(0.5, 0.2, 0.15, 0.05, 0.1, 2.0)

        if contract.right == OptionRight.CALL:
            intrinsic = max(0, underlying - strike)
        else:
            intrinsic = max(0, strike - underlying)
            # Delta and Rho are negative for a put
            greeks.delta *= -1
            greeks.rho *= -1

        theoretical_price = intrinsic + 1.0
        implied_volatility = 0.2

        return OptionPriceModelResult(theoretical_price, implied_volatility, greeks)
```

The `OptionPriceModelParameters` object has the following members:

Disable Pricing

To turn off the Option price calculations, use the `OptionPriceModels.null` model. This model sets the implied volatility and Greeks to 0, and sets the theoretical price to the current price.

```
PY
option.price_model = OptionPriceModels.null()
```

Legacy Models

On February, 19th, 2026, QuantConnect announced new price models for LEAN under the `OptionPriceModels` namespace. The legacy models are still available under the `OptionPriceModels.QuantLib` namespace. The default legacy Option price model was the `OptionPriceModels.QuantLib.binomial_cox_ross_rubinstein` model for American Options or `OptionPriceModels.QuantLib.black_scholes` model for European Options.

```
PY
# option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()   # New model
option.price_model = OptionPriceModels.QuantLib.binomial_cox_ross_rubinstein()   # Legacy model
```

Certain legacy models do not have a direct equivalent in the new price models. See Supported Models for available options.

```
PY
option.price_model = OptionPriceModels.QuantLib.crank_nicolson_fd()
```

Supported Models

LEAN supports the following Option price models.

Model `OptionPriceModels`	American Style	European Style
`binomial_cox_ross_rubinstein`	✓	✓
`black_scholes`		✓
`forward_tree`	✓	✓

These models implement the `IndicatorBasedOptionPriceModel` , which uses the implied volatility and Greeks indicators , with different methodologies.

```py
def initialize(self):
    spy_option = self.add_option("SPY")
    spy_option.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()
    spx_option = self.add_index_option("SPX")
    spx_option.price_model = OptionPriceModels.black_scholes()
```

LEAN supports the following legacy Option price models from the QuantLib library.

Model `OptionPriceModels.QuantLib`	American Style	European Style
`additive_equiprobabilities`	✓	✓
`barone_adesi_whaley`	✓	
`binomial_cox_ross_rubinstein`	✓	✓
`binomial_jarrow_rudd`	✓	✓
`binomial_joshi`	✓	✓
`binomial_leisen_reimer`	✓	✓
`binomial_tian`	✓	✓
`binomial_trigeorgis`	✓	✓
`bjerksund_stensland`	✓	
`black_scholes`		✓
`crank_nicolson_fd`	✓	✓
`integral`		✓

```py
def initialize(self):
    aapl_option = self.add_option("AAPL")
    aapl_option.price_model = OptionPriceModels.QuantLib.crank_nicolson_fd()
    spy_option = self.add_option("SPY")
    spy_option.price_model = OptionPriceModels.QuantLib.binomial_cox_ross_rubinstein()
    spx_option = self.add_index_option("SPX")
    spx_option.price_model = OptionPriceModels.QuantLib.black_scholes()
```

If you set the price model of an Option to a model with an incompatible style, LEAN throws an exception.

What are Greeks?

Option Greeks measure the exposure of Option price or Option delta to the movement of different factors such as the underlying price, time, and volatility. The Greeks are a function of implied volatility. For more information about them, see The Greek Letters .

LEAN provides indicator implementations of Option Greeks. It provides higher flexibility on Option price model selection, volatility modeling , and allows IV smoothing through call-put pair. The `IndicatorBasedOptionPriceModel` use these indicators to calculate the Greeks values. For details, see Option Indicators .

What Is Implied Volatility?

Implied volatility is the forecasted future volatility of a security.

LEAN provides an indicator implementation of implied volatility. It provides higher flexibility on Option price model selection, volatility modeling , and allows IV smoothing through call-put pair. The `IndicatorBasedOptionPriceModel` use these indicators to calculate the implied volatility values. For details, see Implied Volatility .

The legacy Option price models use the realized volatility to calculate the implied volatility. By default, they use the formula from Brenner and Subrahmanyam (1988) as the initial guess for implied volatility.

$$\frac{P}{S} \sqrt{\frac{2\pi}{T}}$$

where P is the Option contract price, S is the underlying price, and T is the time until Option expiration. If the volatility model of the underlying security is ready , the price model uses its value as the initial guess for implied volatility and as an input to calculate the theoretical contract prices.

What Is Volatility Smile?

The "volatility smile" is a term used in finance to describe a particular shape that can be observed when plotting IV against strike prices for options that share the same expiration date. Typically, when you plot IVs against strike prices, the curve tends to be concave, resembling a smile. This means that options with strike prices significantly above or below the current market price of the underlying asset tend to have higher IV than options with strike prices closer to the current market price.

Several theories attempt to explain the existence of the volatility smile. One explanation is that it arises from adjustments to the classic Black-Scholes-Merton (BSM) model, which assumes that asset prices follow a log-normal distribution. The volatility smile suggests that the market anticipates a higher probability of extreme price movements (fat tails) than the BSM model predicts. Another explanation is related to market dynamics and the demand for hedging. During periods of heightened market volatility, the volatility smile tends to become steeper, indicating a greater disparity in implied volatilities across different strike prices. This means that options, particularly those with out-of-the-money strike prices, become more expensive as investors seek protection against potential market downturns or increased price fluctuations. As a result, the volatility smile can serve as a valuable tool for investors and traders to assess market sentiment and gauge the level of risk in their portfolios.

According to some volatility models, such as the Heston model, the IV smile/surface should exhibit a smooth pattern. Any deviations from this smoothness, such as non-smooth spikes or irregularities, may signal potential arbitrage opportunities for traders.

Volatility Smirk

However, in some cases, rather than a traditional U-shaped curve, traders may observe what's known as a "smirk." This smirk occurs when the implied volatilities for higher-strike options are lower than those for ATM and lower-strike options. This deviation from the typical volatility smile pattern indicates a market expectation of a significant downward movement in the underlying asset's price. Traders interpret this as a signal that investors are allocating funds towards options with higher probabilities of such downward movements, resulting in increased demand for lower-strike options.

What Is Volatility Smoothing?

Volatility smoothing refers to the creation of a smooth volatility smile/surface. Derivative like options are favorable for maket-making and arbitration trading, due to the existence of fair price modeling. In the contemporary domain of high-frequency trading, the efficiency of calculating IV and Greeks assumes paramount significance. Thus, the quest for swift and refined IV surface computation becomes an inevitable quest for traders. Rather than relying solely on conventional methods involving slow-paced sampling and continuous parameter recalibration, the adoption of approximation techniques and stationary state analyses gains traction, especially in the context of IV estimation.

Accuracy Consideration

The Law of One Price stipulates the existence of a singular fair price for any given underlying asset. Consequently, according to the principles of put-call parity, each strike price for European options at expiration should correspond to a unique IV value. Within the realm of the Black-Scholes-Merton (BSM) framework, for instance, the concept of put-call parity yields insights into the equivalence of IVs for call and put options at identical strike prices and expirations. This equivalence serves as the foundation for formulating objective function: $C(t,K) - P(t,K)=e^{(q-r)t} (S_T - K) \\\Rightarrow e^{(q-r)t} [S_T (\Phi(d_{+}^{call})+\Phi(-d_{+}^{put})) - K (\Phi(d_{-}^{call})+\Phi(-d_{-}^{put}))] = e^{(q-r)t} (S_T - K) \\\Rightarrow \Phi(d_{+}^{call})+\Phi(-d_{+}^{put}) = 1\ \text{and}\ \Phi(d_{-}^{call})+\Phi(-d_{-}^{put}) = 1 \\\Rightarrow d_{\pm}^{call}=d_{\pm}^{put}$ Thus, we can formulate an objective function of $C_{market} + P_{market} = C_{BSM} + P_{BSM}$.

Alternatively, certain schools of thought contend that the bulk of historical volatility information is encapsulated within at-the-money (ATM) options,

with any residual volatility attributable to the extrinsic value of the option. Consequently, a strategy focused on out-of-the-money (OTM) options for IV calculations gains traction.

Speed Consideration

In the realm of high-frequency trading, where split-second decisions are the norm, the traditional process of sampling, evaluating, recalibrating, and deriving implied volatility expectations is simply unfeasible. In response, traders and analysts increasingly turn to polynomial regression techniques as a pragmatic solution for IV modeling.

For example, $IV_t = \beta_0 + \beta_1 M + \beta_2 M^2 + \beta_3 (T-t) + \beta_4 (T-t)^2 + \beta_5 M(T-t) \\\text{where} M = ln(S_t/K)$

Additionally, the utilization of advanced mathematical algorithms, such as the Fast Fourier Transform (FFT), enables the rapid computation of the entire Greek surface, enhancing speed and efficiency. While these approaches may entail minor trade-offs in terms of accuracy, the overarching objective remains the attainment of a seamlessly smooth implied volatility surface that fosters effective arbitrage strategies.

Examples

Demonstration Algorithms
BasicTemplateOptionsHistoryAlgorithm.py Python BasicTemplateOptionsPriceModel.py Python

Options Models

Volatility

Volatility

Key Concepts

Introduction

Volatility models measure the historical volatility of an asset. They are mostly used to calculate the volatility of the underlying security of an Option because the implied volatility of an Option contract needs an initial guess. The historical volatility doesn't need to be the standard deviation of the asset prices. The various volatility models in LEAN each have a unique methodology to calculate volatility.

LEAN also provides an indicator implementation of implied volatility. It provides higher flexibility on Option price model selection, volatility modeling , and allows IV smoothing through call-put pair. For details, see Implied Volatility .

Set Models

To set the volatility model of the underlying security of an Option, set the `VolatilityModel` property of the `Security` object. The volatility model can have a different resolution than the underlying asset subscription.

```py
def initialize(self) -> None:
    underlying_security = self.add_equity("SPY")
    # Usually, the historical and implied volatility is calculated using 30-day standard deviation of return
    underlying_security.volatility_model = StandardDeviationOfReturnsVolatilityModel(30)
```

You can also set the volatility model in a security initializer . If your algorithm has a universe of underlying assets, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the volatility model
        if security.Type == SecurityType.EQUITY:
            security.volatility_model = StandardDeviationOfReturnsVolatilityModel(30)
```

To view all the pre-built volatility models, see Supported Models .

Default Behavior

The default underlying volatility model for Equity Options and Index Options is the StandardDeviationOfReturnsVolatilityModel based on 30 days of daily resolution data. The default underlying volatility model for Future Options is the NullVolatilityModel .

Model Structure

Volatility models should extend the `BaseVolatilityModel` class. Extensions of the `BaseVolatilityModel` class must have `update` and `get_history_requirements` methods and a `volatility` property. The `update` method receives `Security` and `BaseData` objects and then updates the `volatility` . The `get_history_requirements` method receives `Security` and `datetime` objects and then returns a list of `HistoryRequest` objects that represent the history requests to warm up the model. Volatility models receive data at each time step in the algorithm to update their state.

```
class CustomVolatilityModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        underlying_security = self.add_index("VIX")
        # Apply custom volatility model for specific underlyings, often they are synthetic products or with different market calendar
        underlying_security.volatility_model = MyVolatilityModel()

# Define the custom volatility model outside of the algorithm
class MyVolatilityModel(BaseVolatilityModel):
    volatility: float = 0

    def set_subscription_data_config_provider(self,
        subscription_data_config_provider: ISubscriptionDataConfigProvider) -> None:
        super().set_subscription_data_config_provider(subscription_data_config_provider)

    def update(self, security: Security, data: BaseData) -> None:
        pass

    def get_history_requirements(
            self,
            security: Security,
            utc_time: datetime | date,
            resolution: Resolution | None = None,
            bar_count: int = 0) -> Iterable[HistoryRequest]:
        return super().get_history_requirements(security, utc_time, resolution, bar_count)
```

Warm Up Models

To use your volatility model as the inital guess for the implied volatility , warm up the volatility model of the underlying security. If you subscribe to all the Options in the `initialize` method, set a warm-up period to warm up their volatility models. The warm-up period should provide the volatility models with enough data to compute their values.

```
def initialize(self) -> None:
    # For default 30-day SD of return as volatility, you need 30+1 trading day to warm up
    self.set_warm_up(30, Resolution.DAILY)

def on_data(self, slice: Slice) -> None:
    # Only take the warmed-up volatility value into account for accuracy issue
    if self.is_warming_up:
        return
```

If you have a dynamic universe of underlying assets and add Option contracts to your algorithm with the `add_option_contract` , `add_index_option_contract` , or `add_future_option_contract` methods, warm up the volatility model when the underlying asset enters your universe. We recommend you do this inside a security initializer .

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite and warm up the volatility model
        if security.type == SecurityType.EQUITY:  # Underlying asset type
            security.volatility_model = StandardDeviationOfReturnsVolatilityModel(30)
            trade_bars = self.history[TradeBar](security.symbol, 30, Resolution.DAILY)
            for trade_bar in trade_bars:
                security.volatility_model.update(security, trade_bar)
```

Examples

The following examples demonstrate common practices for implementing a custom volatility model.

Example 1: Default Model Wrapper

The following algorithm defines a subclass of the `BaseVolatilityModel` and overrides its built-in members so you can easily customize the parts you need.

```py
class CustomVolatilityModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        security = self.add_equity('SPY', Resolution.DAILY)
        security.volatility_model = MyVolatilityModel()

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyVolatilityModel(BaseVolatilityModel):
    volatility: float = 0

    def set_subscription_data_config_provider(self,
        subscription_data_config_provider: ISubscriptionDataConfigProvider) -> None:
        super().set_subscription_data_config_provider(subscription_data_config_provider)

    def update(self, security: Security, data: BaseData) -> None:
        pass

    def get_history_requirements(
            self,
            security: Security,
            utc_time: datetime | date,
            resolution: Resolution | None = None,
            bar_count: int = 0) -> Iterable[HistoryRequest]:
        return super().get_history_requirements(security, utc_time, resolution, bar_count)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomVolatilityModelAlgorithm.py Python

Volatility

Supported Models

Introduction

This page describes all of the pre-built volatility models in LEAN. If none of these models perform exactly how you want, create a custom volatility model .

Null Model

The `NullVolatilityModel` sets the volatility of the security to zero. It's the default volatility model for the underlying asset of Future Options.

```
underlying_security.volatility_model = VolatilityModel.NULL
```

PY

For more information about this model, see the class reference and implementation .

Standard Deviation of Returns Model

The `StandardDeviationOfReturnsVolatilityModel` sets the volatility of the security to the annualized sample standard deviation of trailing returns. It's the default volatility model for the underlying asset of Equity Options and Index Options.

```
underlying_security.volatility_model = StandardDeviationOfReturnsVolatilityModel(30)
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`periods`	`int`	The max number of samples to use when calculating the standard deviation of returns. This value must be greater than two.	
`resolution`	`Resolution` / `NoneType`	The resolution of the price data used to calculate the standard deviation. This only has a material effect in live mode. For backtesting, this value does not cause any behavioral changes.	`None`
`update_frequency`	`timedelta` / `NoneType`	The frequency at which new values are inserted into the rolling window for the standard deviation calculation. If the value is `None` , it defaults to the `timedelta` representation of `resolution` . If the value and `resolution` are `None` , it defaults to a `timedelta` of one day.	`None`

For more information about this model, see the class reference and implementation .

Relative Standard Deviation Model

The `RelativeStandardDeviationVolatilityModel` sets the volatility of the security to the relative standard deviation of its price. In symbols, the value is

$$\frac{|\mu|}{\sigma}$$

where μ is the average of the samples and σ is the standard deviation of the samples.

```
underlying_security.volatility_model = RelativeStandardDeviationVolatilityModel(timedelta(days=1), 10)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`period_span`	`timedelta`	The period of time to wait between each sample of the security price	
`periods`	`int`	The number of samples to use to calculate the volatility value	

For more information about this model, see the class reference and implementation .

Options Models

Exercise

Introduction

If you exercise a long Option position or are assigned on your short Option position, LEAN processes an Option exercise order. The Option exercise model converts the Option exercise order into an OrderEvent .

Set Models

To set the exercise model of an Option, call the `set_option_exercise_model` method of the `Option` object inside a security initializer .

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the Option exercise model
        if security.Type == SecurityType.OPTION: # Option type
            security.set_option_exercise_model(DefaultExerciseModel())
```

Default Behavior

The default Option exercise model is the `DefaultExerciseModel` . The `DefaultExerciseModel` fills exercise orders to the full quantity with zero fees and applies an order tag to represent if the order is an exercise or assignment.

For more information about this model, see the class reference and implementation .

Model Structure

Option exercise models should extend the `DefaultExerciseModel` class. Extensions of the `DefaultExerciseModel` must implement the `option_exercise` method, which receives `Option` and `OptionExerciseOrder` objects and then returns a list of `OrderEvent` objects that contain the order fill information.

```py
class CustomOptionExerciseModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_option("SPY")
        # Set custom option exercise model for mimicking specific Brokerage most realistic actions
        security.set_option_exercise_model(MyOptionExerciseModel())

# Define the custom Option exercise model outside of the algorithm
class MyOptionExerciseModel(DefaultExerciseModel):
    def option_exercise(self, option: Option, order: OptionExerciseOrder) -> List[OrderEvent]:
        in_the_money = option.is_auto_exercised(option.underlying.close)
        is_assignment = in_the_money and option.holdings.is_short

        order_event = OrderEvent(
            order.id,
            option.symbol,
            Extensions.convert_to_utc(option.local_time, option.exchange.time_zone),
            OrderStatus.FILLED,
            Extensions.get_order_direction(order.quantity),
            0.0,
            order.quantity,
            OrderFee.ZERO,
            "Tag"
        )
        order_event.is_assignment = is_assignment
        return [ order_event ]
```

`OptionExerciseOrder` objects have the following properties:

The following table describes the arguments of the `OrderEvent` constructor:

Argument Details

Argument: `order_id`

Id of the parent order

Data Type: `int` | Default Value: `-`

Argument: `symbol`

Asset Symbol

Data Type: `Symbol` | Default Value: `-`

Argument: `utc_time`

Date/time of this event

Data Type: `datetime` | Default Value: `-`

Argument: `direction`

The direction of the order. The `OrderDirection` enumeration has the following members:

Data Type: `OrderDirection` | Default Value: `Hold`

Argument: `fill_price`

Fill price information if applicable

Data Type: `float` | Default Value: `0`

Argument: `fill_quantity`

Fill quantity

Data Type: `float` | Default Value: `0`

Argument: `order_fee`

The order fee. You can use `OrderFee.ZERO` or create an `OrderFee` object with a custom fee.

```
OrderFee(CashAmount(0.5, 'USD'))
```

Data Type: `OrderFee` | Default Value: `-`

Argument: `message`

Message from the exchange

Data Type: `str` | Default Value: `""`

`OrderEvent` objects have the following attributes:

Examples

The following examples demonstrate some common practices for implementing a custom Option exercise model.

Example 1: Cash Settlement

The following algorithm trades GOOG 30-day expiring straddle. Yet, instead of settling with the underlying stock, some brokerages will settle with cash for ITM options. To simulate this behavior, we can create a custom option exercise model.

```py
class OptionExerciseModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request GOOG option data for trading.
        security = self.add_option("GOOG")
        self.goog = security.symbol
        # Filter for the 2 ATM contracts expiring in 30 days to form a straddle strategy.
        security.set_filter(lambda universe: universe.straddle(30))
        # Set custom option exercise model for disabling exercise through security initializer.
        self.settings.seed_initial_prices = True

    def on_data(self, slice: Slice) -> None:
        # Open position on updated option chain data.
        chain = slice.option_chains.get(self.goog)
        if chain and not self.portfolio.invested:
            # Only one strike and expiry for the straddle universe.
            strike = min(x.strike for x in chain)
            expiry = min(x.expiry for x in chain)
            # Open the straddle position.
            option_straddle = OptionStrategies.straddle(self.goog, strike, expiry)
            self.buy(option_straddle, 5)

class MySecurityInitializer(BrokerageModelSecurityInitializer):
    def __init__(self, brokerageModel, securitySeeder):
        super().__init__(brokerageModel, securitySeeder)

    def initialize(self, security: Security) -> None:
        super().initialize(security)

        # Set the custom Option exercise model for Option securities
        if security.type == SecurityType.OPTION:
            security.set_option_exercise_model(MyOptionExerciseModel())

# Define the custom Option exercise model outside of the algorithm
class MyOptionExerciseModel(DefaultExerciseModel):
    def option_exercise(self, option: Option, order: OptionExerciseOrder) -> List[OrderEvent]:
        underlying = option.underlying
        utc_time = Extensions.convert_to_utc(option.local_time, option.exchange.time_zone)
        in_the_money = option.is_auto_exercised(underlying.close)
        # Cash settle: using payoff.
        payoff = option.get_intrinsic_value(underlying.close)

        # Only liquidate option positions, but do not add equity positions.
        order_event = OrderEvent(
            order.id,
            option.symbol,
            utc_time,
            OrderStatus.FILLED,
            Extensions.get_order_direction(order.quantity),
            payoff,
            order.quantity,
            OrderFee.ZERO,
            "Option Settlement"
        )
        order_event.is_in_the_money = in_the_money
        return [ order_event ]
```

Example 2: Default Model Implementation

The following algorithm implements the `DefaultExerciseModel` so you can easily customize the parts you need.

PY

```
class CustomExerciseModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self._custom_security_initializer)
        option = self.add_option("SPY")
        self.option_symbol = option.symbol

    def on_data(self, data: Slice):
        if self.portfolio.invested:
            return
        chain = data.option_chains.get(self.option_symbol)
        if chain:
            contract = sorted([x for x in chain if x.right == OptionRight.CALL],
                key=lambda x: x.strike)[0]
            self.market_order(contract.symbol, 1)
            self.exercise_option(contract.symbol, 1)
            self.quit()

    def _custom_security_initializer(self, security: Security) -> None:
        if security.type == SecurityType.OPTION:
            security.set_option_exercise_model(MyOptionExerciseModel(self))


class MyOptionExerciseModel(DefaultExerciseModel):

    def __init__(self, algorithm):
        self.algorithm = algorithm

    def option_exercise(self, option: Option, order: OptionExerciseOrder) -> List[OrderEvent]:
        order_events = []

        underlying = option.underlying
        utc_time = Extensions.convert_to_utc(option.local_time, option.exchange.time_zone)

        in_the_money = option.is_auto_exercised(underlying.close)
        is_assignment = in_the_money and option.holdings.is_short
        messages = Messages.DefaultExerciseModel

        order_event = OrderEvent(
            order.id,
            option.symbol,
            utc_time,
            OrderStatus.FILLED,
            Extensions.get_order_direction(order.quantity),
            0,
            order.quantity,
            OrderFee.ZERO,
            messages.contract_holdings_adjustment_fill_tag(in_the_money, is_assignment, option)
        )
        order_event.is_assignment = is_assignment
        order_event.is_in_the_money = in_the_money
        order_events.append(order_event)

        if in_the_money and option.exercise_settlement == SettlementType.PHYSICAL_DELIVERY:
            exercise_quantity = option.get_exercise_quantity(order.quantity);
            order_event =  OrderEvent(
                order.id,
                underlying.symbol,
                utc_time,
                OrderStatus.FILLED,
                Extensions.get_order_direction(exercise_quantity),
                option.strike_price,
                exercise_quantity,
                OrderFee.ZERO,
                messages.option_assignment if is_assignment else messages.option_exercise
            )
            order_event.is_in_the_money = True
            order_events.append(order_event)

        return order_events
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomOptionExerciseModelRegressionAlgorithm.py Python

Options Models

Assignment

Introduction

If you sell an Option in a backtest, an assignment model can simulate an Option exercise order on behalf of the buyer and assign you to complete the requirements of the contract.

Set Models

To set the assignment model of an Option, call the `set_option_assignment_model` method of the `Option` object inside a security initializer .

```
PY
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the assignment model
        if security.Type == SecurityType.OPTION: # Option type
            security.set_option_assignment_model(DefaultOptionAssignmentModel())
```

Default Behavior

The default Option assignment model is the `DefaultOptionAssignmentModel` . The `DefaultOptionAssignmentModel` scans your portfolio every hour. It considers exercising American-style Options if they are within 4 days of their expiration and it considers exercising European-style Options on their day of expiration. If you have sold an Option that's 5% in-the-money and the Option exercise order is profitable after the cost of fees, this model exercises the Option.

For more information about this model, see the class reference and implementation .

Model Structure

Option assignment models should extend the `NullOptionAssignmentModel` class. Extensions of the `NullOptionAssignmentModel` class must implement the `get_assignment` method, which automatically fires at the top of each hour and returns the Option assignments to generate.

```
PY
class CustomOptionAssignmentModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_option("SPY")
        # Set custom option assignment model for mimicking specific Brokerage most realistic actions
        security.set_option_assignment_model(MyOptionAssignmentModel())

# Define the custom Option assignment model outside of the algorithm
class MyOptionAssignmentModel(NullOptionAssignmentModel):

    def get_assignment(self, parameters: OptionAssignmentParameters) -> OptionAssignmentResult:
        option = parameters.option
        # Check if the contract is ITM
        if (option.right == OptionRight.CALL and option.underlying.price > option.strike_price or
            option.right == OptionRight.PUT and option.underlying.price < option.strike_price):
            return OptionAssignmentResult(option.holdings.absolute_quantity, "MyTag")
        return OptionAssignmentResult.NULL
```

The `OptionAssignmentParameters` object has the following members:

To exercise the Option, return an `OptionAssignmentResult` with a positive quantity. Otherwise, return `OptionAssignmentResult.Null` . The `OptionAssignmentResult` constructor accepts the following arguments:

Argument	Data Type	Description	Default Value
`quantity`	`float`	The quantity to assign	
`tag`	`str`	The order tag to use	

Disable Assignments

To disable early Option assignments, set the Option assignment model to the `NullOptionAssignmentModel` .

```
security.set_option_assignment_model(NullOptionAssignmentModel())
```

Examples

The following examples demonstrate some common practices for handling Option assignment.

Example 1: Liquidate After Assignment

The following algorithm trades a short straddle strategy on ATM SPY Options that expire within seven days. To capitalize the cash profit, it liquidates the underlying assigned Equity position in the `on_assignment_order_event` method.

```
class OptionAssignmentAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Add SPY Options data for trading.
        self._option = self.add_option("SPY")
        # Filter for the 2 ATM contracts that expire in 7 days to form a straddle.
        self._option.set_filter(lambda universe: universe.straddle(7))

    def on_data(self, slice: Slice) -> None:
        # Get the current Option chain.
        chain = slice.option_chains.get(self._option.symbol)
        # Open a new position if we're not already invested.
        if chain and not self.portfolio.invested:
            # Select the strike and expiry of the contracts.
            strike = list(chain)[0].strike
            expiry = list(chain)[0].expiry
            # Open the straddle position.
            option_strategy = OptionStrategies.short_straddle(self._option.symbol, strike, expiry)
            self.buy(option_strategy, 1)

    def on_assignment_order_event(self, assignment_event: OrderEvent) -> None:
        # Liquidate the assigned SPY position.
        self.market_order(
            assignment_event.symbol.underlying,
            -assignment_event.quantity * self._option.symbol_properties.contract_multiplier
        )
```

Example 2: Wheel Strategy

The following algorithm demonstrates a wheel Option strategy . The strategy is split into separate parts to generate consistent income: selling puts and selling covered call.

PY

```
class WheelStrategyAlgorithm(QCAlgorithm):
    # Define the OTM threshold. A greater value means lower risk of assignment but lower premiums to collect.
    _otm_threshold = 0.05

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Seed the initial price of the each new security so we can add and trade contracts in the same time step.
        self.settings.seed_initial_prices = True
        # Add SPY data for trading. Use raw data normalization mode to enable fair strike price comparison.
        self._equity = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW)

    def _get_target_contract(self, right: OptionRight, target_price: float) -> Symbol:
        # Get the Option chain.
        chain = self.option_chain(self._equity.symbol, flatten=True).data_frame
        # Select the contracts that are first to expire after 30 days since they have greater premiums and theta
        # to collect.
        expiry_threshold = self.time + timedelta(30)
        expiry = chain[chain.expiry > expiry_threshold].expiry.min()
        # Select the OTM Option contract to start the wheel or covered call.
        symbol = chain[
            (chain.expiry == expiry) &
            (chain.right == right) &
            (chain.strike <= target_price if right == OptionRight.PUT else chain.strike >= target_price)
        ].sort_values('strike', ascending=right == OptionRight.CALL).index[0]
        # Add the Option contract to the algorithm.
        return self.add_option_contract(symbol).symbol

    def on_data(self, slice: Slice) -> None:
        # Start the wheel by selling the OTM put.
        if not self.portfolio.invested and self.is_market_open(self._equity.symbol):
            symbol = self._get_target_contract(OptionRight.PUT, self._equity.price * (1-self._otm_threshold))
            self.set_holdings(symbol, -0.2)

    def on_assignment_order_event(self, assignment_event: OrderEvent) -> None:
        # After the put is assigned, look for an OTM call to form a covered call with the assigned underlying
        # position.
        symbol = self._get_target_contract(OptionRight.CALL, self._equity.price * (1+self._otm_threshold))
        self.market_order(symbol, -self._equity.holdings.quantity / 100)
```

Example 3: Assign ITM Contracts

The following algorithm assigns all in-the-money contracts at the top of each hour.

PY

```
class CustomOptionAssignmentAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.add_security_initializer(self._initialize_security)

        option = self.add_option('GOOG')
        option.set_filter(
            lambda option_filter_universe: option_filter_universe.calls_only().strikes(-65, 0).expiration(0, 30)
        )

    def _initialize_security(self, security: Security) -> None:
        # Overwrite the assignment model.
        if security.Type == SecurityType.OPTION: # Option type
            security.set_option_assignment_model(MyOptionAssignmentModel())

    def on_data(self, data) -> None:
        if self.portfolio.invested:
            return

        for canonical_symbol, chain in data.option_chains.items():
            min_expiry = min([contract.expiry for contract in chain])
            contracts = sorted(
                [contract for contract in chain if contract.expiry == min_expiry],
                key=lambda contract: contract.strike
            )
            if contracts:
                self.market_order(contracts[0].symbol, -1)


class MyOptionAssignmentModel(NullOptionAssignmentModel):

    def get_assignment(self, parameters: OptionAssignmentParameters) -> OptionAssignmentResult:
        option = parameters.option
        # Check if the contract is ITM.
        if (option.Right == OptionRight.CALL and option.Underlying.Price > option.StrikePrice or
            option.Right == OptionRight.PUT and option.Underlying.Price < option.StrikePrice):
            return OptionAssignmentResult(option.holdings.absolute_quantity, "MyTag")
        return OptionAssignmentResult.NULL
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomOptionAssignmentRegressionAlgorithm.py Python

Reality Modeling

Risk Free Interest Rate

Risk Free Interest Rate

Key Concepts

Introduction

The risk free interest rate is the hurdle rate for investments. If investors can get a rate of return with virtually no risk, an investment needs to offer the potential for a greater return to attract capital from rational investors. The risk free interest rate also impacts some performance statistics , like Sharpe and Sortino ratios. Risk free interest rate models provide the interest rate data to your algorithms to enable these calculations.

Set Models

To set the risk free interest rate model, in the Initialize method, call the `set_risk_free_interest_rate_model` method.

```py
def initialize(self) -> None:
    # Set the interest rate model for accurate option IV/greeks and result metrics like Sharpe Ratio
    # You should set it according to the main currency of your trading Securities
    self.set_risk_free_interest_rate_model(ConstantRiskFreeRateInterestRateModel(0.02))
```

To view all the pre-built risk free interest rate models, see Supported Models .

Default Behavior

The default risk free interest rate model is the InterestRateProvider , which provides the primary credit rate from the Federal Open Market Committee (FOMC).

Model Structure

Risk free interest rate models must extend the `IRiskFreeInterestRateModel` interface. Extensions of the `IRiskFreeInterestRateModel` interface must implement a `get_interest_rate` method. The `get_interest_rate` method returns the risk free interest rate for a given date.

```py
class CustomRiskFreeInterestRateModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # Set the interest rate model for accurate option IV/greeks and result metrics like Sharpe Ratio
        # You should set it according to the main currency of your trading Securities
        self.set_risk_free_interest_rate_model(MyRiskFreeInterestRateModel())

# Define the custom risk free interest rate model
class MyRiskFreeInterestRateModel(IRiskFreeInterestRateModel):
    def get_interest_rate(self, date: datetime | date) -> float:
        return 0.02
```

Disable Interest Rates

To disable the risk free interest rate, set the model to the `ConstantRiskFreeRateInterestRateModel` and set the risk free rate to zero.

```py
self.set_risk_free_interest_rate_model(ConstantRiskFreeRateInterestRateModel(0))
```

Get Interest Rate

To get the risk free interest rate for a specific time, call the `get_interest_rate` method with the time.

```
def on_data(self, slice: Slice) -> None:
 risk_free_interest_rate = self.risk_free_interest_rate_model.get_interest_rate(slice.time)
```

To get the average risk free interest rate for a window of time, call the `RiskFreeInterestRateModelExtensions.get_risk_free_rate` method with the model, the start date, and end date.

```
def on_data(self, slice: Slice) -> None:
 avg_risk_free_interest_rate = RiskFreeInterestRateModelExtensions.get_risk_free_rate(
  self.risk_free_interest_rate_model,
  self.time-timedelta(7), self.time
 )
```

To get the average risk free interest rate for a set of dates, call the `RiskFreeInterestRateModelExtensions.get_average_risk_free_rate` method with the model and list of dates.

```
def on_data(self, slice: Slice) -> None:
 avg_risk_free_interest_rate = RiskFreeInterestRateModelExtensions.get_average_risk_free_rate(
  self.risk_free_interest_rate_model,
  [self.time, self.time-timedelta(180), self.time-timedelta(365)]
 )
```

Examples

The following examples demonstrate common practices for implementing a custom risk free interest rate model.

Example 1: Fixed Rate

The following algorithm implements a custom risk free interest rate model with a fixed rate of 2%.

```
class CustomFillModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.set_risk_free_interest_rate_model(MyRiskFreeInterestRateModel())
        self.add_equity('SPY', Resolution.DAILY)

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyRiskFreeInterestRateModel:

    def get_interest_rate(self, date: datetime | date) -> float:
        return 0.02
```

Risk Free Interest Rate

Supported Models

Introduction

This page describes all of the pre-built risk free interest rate models in LEAN. If none of these models perform exactly how you want, create a custom risk free interest rate model .

Constant Model

The `ConstantRiskFreeRateInterestRateModel` returns a constant rate across time. It's the default risk free interest rate model.

```
self.set_risk_free_interest_rate_model(ConstantRiskFreeRateInterestRateModel(0.02))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`risk_free_rate`	`float`	The risk free interest rate	

For more information about this model, see the class reference and implementation .

Function Model

The `FuncRiskFreeRateInterestRateModel` calls a function you provide to get the risk free interest rate. The function you pass to the constructor effectively replaces the GetInterestRate method.

```
self.set_risk_free_interest_rate_model(FuncRiskFreeRateInterestRateModel(self.get_interest_rate_func))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`get_interest_rate_func`	`Callable[[datetime], float]`	A function that returns the risk free interest rate for a given date	

For more information about this model, see the class reference and implementation .

Interest Rate Provider Model

The `InterestRateProvider` returns the primary credit rate from the Federal Open Market Committee (FOMC).

```
self.set_risk_free_interest_rate_model(InterestRateProvider())
```

For more information about this model, see the class reference and implementation .

Reality Modeling

Dividend Yield

Dividend Yield

Key Concepts

Introduction

Dividends are periodic payments a company makes to its shareholders out of its earnings. The dividend yield refers to the rate of return an investor earns in the form of dividends from holding a stock.

In the context of Option pricing models , the dividend yield is a critical factor because it affects the value of the underlying stock. When calculating the theoretical price of an Option using these models, the dividend yield is factored into the equation.

Set Models

To set the dividend yield model for an Option indicator , pass a `DividendYieldProvider` as the `dividend_yield_model` parameter.

```py
def initialize(self) -> None:
    # Before creating the indicator, create the set dividend yield model
    # Using DividendYieldProvider will calculate the continuous dividend yield using all dividend payoffs in 1 year
    dividend_yield_model = DividendYieldProvider(symbol.underlying)
    self.iv = ImpliedVolatility(symbol, self.risk_free_interest_rate_model, dividend_yield_model, OptionPricingModelType.BLACK_SCHOLES)
```

To view all the pre-built dividend yield models, see Supported Models .

Default Behavior

For US Equity Options, the default dividend yield model is the DividendYieldProvider , which provides the continuous yield calculated from all dividend payoffs from the underlying Equity over the previous 350 days.

For other Option types, the default dividend yield model is the `DividendYieldProvider` , but it calculates the yield from the dividends of the SPY ETF.

Model Structure

Dividend yield models must extend the `IDividendYieldModel` interface. Extensions of the `IDividendYieldModel` interface must implement a `get_dividend_yield` method. The `get_dividend_yield` method returns the dividend yield for a given date.

```py
from datetime import date


class CustomDividendYieldModelExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Create the dividend yield model.
        dividend_yield_model = MyDividendYieldModel()
        # Add the Option contracts.
        underlying_symbol = self.add_equity('SPY', data_normalization_mode=DataNormalizationMode.RAW).symbol
        strike = 595
        expiry = datetime(2024, 12, 30)
        option = self._add_contract(underlying_symbol, OptionRight.PUT, strike, expiry)
        mirror = self._add_contract(underlying_symbol, OptionRight.CALL, strike, expiry)
        # Add the indicator.
        iv = ImpliedVolatility(option, self.risk_free_interest_rate_model, dividend_yield_model, mirror)

    def _add_contract(self, underlying_symbol, right, strike, expiry) -> Symbol:
        contract_symbol = Symbol.create_option(
            underlying_symbol, Market.USA, OptionStyle.AMERICAN, right, strike, expiry
        )
        self.add_option_contract(contract_symbol)
        return contract_symbol


# Define the custom dividend yield model
class MyDividendYieldModel(DividendYieldProvider):

    def get_dividend_yield(
            self,
            date_or_data: datetime | date | IBaseData,
            security_price: float | None = None) -> float:
        return 0.02
```

Examples

The following examples demonstrate common practices for implementing a custom dividend yield model.

Example 1: Fixed Yield

The following algorithm implements a custom dividend yield model with a fixed yield of 2%.

PY

```
from datetime import date


class CustomDividendYieldModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        option = self.add_option('MSFT')
        option.set_filter(0, 1, 0, 31)
        self._symbol = option.symbol
        self._deltas = {}
        self._dividend_yield_model = {self._symbol.underlying : MyDividendYieldModel()}

    def on_data(self, data: Slice) -> None:
        chain = data.option_chains.get(self._symbol)
        if not chain:
            return
        expiry = min([x.expiry for x in chain])
        symbols = [x.symbol for x in chain if x.expiry == expiry and x.right == OptionRight.CALL and x.symbol not in self._deltas]
        for symbol in symbols:
            self._deltas[symbol] = self._deltas.get(symbol, self._create_delta_indicator(symbol))

    def _create_delta_indicator(self, symbol: Symbol) -> Delta:
        dividend_yield_model = self._dividend_yield_model.get(symbol.underlying)
        delta = Delta(symbol.value, symbol, self.risk_free_interest_rate_model, dividend_yield_model)
        self.register_indicator(symbol, delta)
        self.register_indicator(symbol.underlying, delta)
        return delta

    def on_end_of_day(self, symbol) -> None:
        delta = self._deltas.get(symbol)
        if delta:
            self.plot('Delta', symbol.value, delta.current.value)


class MyDividendYieldModel(DividendYieldProvider):

    def get_dividend_yield(
            self,
            date_or_data: datetime | date | IBaseData,
            security_price: float | None = None) -> float:
        return 0.02
```

Dividend Yield

Supported Models

Introduction

This page describes all of the pre-built dividend yield models in LEAN. If none of these models perform exactly how you want, create a custom dividend yield model .

Constant Model

The `ConstantDividendYieldModel` returns a constant yield across time. It's the default dividend yield model for securities that don't pay dividends.

```
self.dividend_yield_model = ConstantDividendYieldModel(0.02)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`dividend_yield`	`float`	The dividend yield	

For more information about this model, see the class reference and implementation .

Dividend Yield Provider Model

The `DividendYieldProvider` uses historical dividend payments to estimate the annualized continuous dividend yield for a given date.

```
self.dividend_yield_model = DividendYieldProvider()
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`symbol`	`Symbol`	The symbol of a dividend-paying security. If you don't provide a value, it defaults to SPY.	

For more information about this model, see the class reference and implementation .

Reality Modeling

Margin Interest Rate

Margin Interest Rate

Key Concepts

Introduction

Margin interest is a cost associated with trading on margin. Margin interest rate models model margin interest cash flows by directly adding or removing cash from your portfolio.

Set Models

The brokerage model of your algorithm automatically sets the margin interest rate model for each security, but you can override it. To manually set the margin interest rate model of a security, assign a model to the `MarginInterestRateModel` property of the Security object.

```py
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # For US Equity, use the ShortMarginInterestRateModel with IB data
    security.set_shortable_provider(InteractiveBrokersShortableProvider())
    security.margin_interest_rate_model = ShortMarginInterestRateModel()
```

You can also set the margin interest rate model in a security initializer . If your algorithm has a dynamic universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_shortable_provider(InteractiveBrokersShortableProvider())
        security.margin_interest_rate_model = ShortMarginInterestRateModel()
```

To view all the pre-built margin interest rate models, see Supported Models .

Default Behavior

The brokerage model of your algorithm automatically sets the margin interest rate model of each security. The default brokerage model is the `DefaultBrokerageModel` , which sets the NullMarginInterestRateModel .

Model Structure

Margin interest rate models should implement the `IMarginInterestRateModel` interface. Extensions of the `IMarginInterestRateModel` interface must implement the `apply_margin_interest_rate` method, which applies margin interest payments to the portfolio.

```
class CustomMarginInterestRateModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_equity("SPY")
        # You should set it according to your broker information for the least actual-expected discrepancies
        security.set_margin_interest_rate_model(MyMarginInterestRateModel())

# Define the custom margin interest rate model
class MyMarginInterestRateModel:

    def apply_margin_interest_rate(self, margin_interest_rate_parameters: MarginInterestRateParameters) -> None:
        holdings = margin_interest_rate_parameters.security.holdings
        position_value = holdings.get_quantity_value(holdings.quantity)
        position_value.cash.add_amount(-1)
```

The `apply_margin_interest_rate` method is automatically called at the top of each hour.

Examples

The following examples demonstrate some common practices for implementing a custom margin interest rate model.

Example 1: Based On Interest Rate

The following algorithm buys and holds SPY, with 2x leverage and holds 1.5x of portfolio value. Hence, funds will be borrowed from the broker and are susceptible to margin interest. We implement a custom margin interest rate model to simulate the Interactive Brokers margin interest deduction. While following daily accrual and monthly accounting of the margin interest, we simplify the rate from tiered to fixed at 1% above the current interest rate.

```
class MarginInterestRateModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Request SPY data for trading.
        equity = self.add_equity("SPY")
        self.spy = equity.symbol
        # Set a custom margin interest rate model for the SPY security to mimic IB margin interest deduction.
        equity.set_margin_interest_rate_model(MyMarginInterestRateModel(self, self.spy))

    def on_data(self, slice: Slice) -> None:
        if not self.portfolio.invested:
            # Place a position that exceeds available cash to buy with borrowed funds.
            self.set_holdings(self.spy, 1.5)

# Define the custom margin interest rate model
class MyMarginInterestRateModel:
    accrued_margin_interest = {}

    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        # Save the algorithm instance since we must access the at-the-time interest rate to calculate the margin rate.
        self.algorithm = algorithm
        # IBKR accrues interest daily and posts actual interest monthly on the third business day of the following month.
        algorithm.schedule.on(
            algorithm.date_rules.month_start(symbol, 3),
            algorithm.time_rules.at(0, 0),
            self.margin_deduction
        )

    def margin_deduction(self) -> None:
        for cash, accrued_interest in self.accrued_margin_interest.items():
            # Monthly deduction of cash position with accrued interests.
            cash.add_amount(accrued_interest)
            # Reset the accrued interest accumulator.
            self.accrued_margin_interest[cash] = 0

    def apply_margin_interest_rate(self, margin_interest_rate_parameters: MarginInterestRateParameters) -> None:
        # IBKR accrues interest daily for overnight positions only.
        if self.algorithm.time.hour == 16:
            holdings = margin_interest_rate_parameters.security.holdings
            # Calculate the borrowed capital. The margin interest will only applied to the borrowed capital.
            borrowed_value = holdings.unlevered_absolute_holdings_cost - holdings.absolute_holdings_cost
            # Margin interest rate is 1% above the current risk-free interest rate.
            margin_rate = self.algorithm.risk_free_interest_rate_model.get_interest_rate(self.algorithm.time) + 0.01
            # Deduct the margin interest on the account cash position by the margin rate, multiplying the borrowed sum.
            cash = holdings.get_quantity_value(holdings.quantity).cash
            if cash not in self.accrued_margin_interest:
                self.accrued_margin_interest[cash] = 0
            # Daily rate will be divided by 365.
            self.accrued_margin_interest[cash] += borrowed_value * margin_rate / 365
```

Example 2: Fixed Dollar Rate

The following algorithm defines custom margin interest rate model that reduces your cash by $1 at the top of each hour.

```py
class CustomFillModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        security = self.add_equity('SPY', Resolution.DAILY)
        security.set_margin_interest_rate_model(MyMarginInterestRateModel())

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', 1)


class MyMarginInterestRateModel:

    def apply_margin_interest_rate(self, margin_interest_rate_parameters: MarginInterestRateParameters) -> None:
        holdings = margin_interest_rate_parameters.security.holdings
        position_value = holdings.get_quantity_value(holdings.quantity)
        position_value.cash.add_amount(-1)
```

Margin Interest Rate

Supported Models

Introduction

This page describes the pre-built margin interest rate models in LEAN. If none of these models perform exactly how you want, create a custom margin interest rate model .

Null Model

The `NullMarginInterestRateModel` doesn't charge any margin interest. It's the default margin interest rate of the DefaultBrokerageModel .

```
PY
security.margin_interest_rate_model = MarginInterestRateModel.NULL
```

For more information about this model, see the class reference and implementation .

Short Equity Model

The `ShortMarginInterestRateModel` simulates the margin cost and payments of your Equity holdings. This model use the fee rate to compute the amount due to borrowing the asset to short, and the rebate rate to compute the amount due to lending the asset for shorting. For more information about these rates providers, see Short Availability .

```
PY
security.set_shortable_provider(InteractiveBrokersShortableProvider())
security.margin_interest_rate_model = ShortMarginInterestRateModel()
```

For more information about this model, see the class reference and implementation .

Binance Futures Model

The `BinanceFutureMarginInterestRateModel` simulates the margin cost and payments of your Crypto Future holdings. When the funding rate is positive, the price of the perpetual contract is higher than the mark price, so traders who are long pay for short positions. Conversely, a negative funding rate indicates that perpetual prices are below the mark price, so traders who are short pay for long positions. The interest amount is

Nominal \ Value \ of \ Positions * Funding \ Rate

The interest amount is periodically credited or debited from your currency holdings while you hold open positions. The interest amount is charged at 12 AM, 8 AM, 4 PM in your algorithm time zone .

```
PY
security.margin_interest_rate_model = BinanceFutureMarginInterestRateModel()
```

For more information about this model, see the class reference and implementation .

Bybit Futures Model

The `BybitFutureMarginInterestRateModel` has the same behavior as the BinanceFutureMarginInterestRateModel .

```
PY
security.margin_interest_rate_model = BybitFutureMarginInterestRateModel()
```

For more information about this model, see the class reference and implementation .

dYdX Futures Model

The `dYdXFutureMarginInterestRateModel` simulates the margin cost and payments of your Crypto Future holdings. When the funding rate is positive, the price of the perpetual contract is higher than the mark price, so traders who are long pay for short positions. Conversely, a negative funding rate indicates that perpetual prices are below the mark price, so traders who are short pay for long positions. The interest amount is

Nominal \ Value \ of \ Positions * Funding \ Rate

The interest amount is periodically credited or debited from your currency holdings while you hold open positions. The interest amount is charged at 12 AM, 8 AM, and 4 PM in your algorithm time zone .

```
security.margin_interest_rate_model = dYdXFutureMarginInterestRateModel()
```

PY

For more information about this model, see the class reference and implementation .

Reality Modeling

Margin Calls

Introduction

If the value of your portfolio falls below the maintenance margin, you receive a margin call. If you receive a margin call, you either need to add more capital to your brokerage account or the brokerage will liquidate some of your holdings to reduce your exposure and their risk. A margin call model monitors the margin levels of your portfolio, issues margin call warnings, and submits orders when margin calls occur.

Set Models

To set the margin call model, call the `set_margin_call_model` method of the `portfolio` object.

```py
def initialize(self) -> None:
 # Set margin call model for the whole portfolio, better with reference to your broker information
 self.portfolio.set_margin_call_model(DefaultMarginCallModel(self.portfolio, self.default_order_properties))
```

Default Behavior

The brokerage model of your algorithm automatically sets the margin call model for the portfolio. The default brokerage model is the `DefaultBrokerageModel` , which sets the `DefaultMarginCallModel` . The `DefaultMarginCallModel` issues margin call warnings when the margin remaining in your portfolio is less than or equal to 5% of the total portfolio value. When your total margin used exceeds 10% of the total portfolio value, this model creates margin call orders. It sorts the generated margin call orders in ascending order by their unrealized profit and then synchronously executes each order until your portfolio is within the margin requirements.

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`portfolio`	`SecurityPortfolioManager`	The portfolio object to receive margin calls.	
`default_order_properties`	`IOrderProperties`	The default order properties to be use in margin call orders.	
`margin_buffer`	`float`	The percent margin buffer to use when checking whether the total margin used is above the total portfolio value to generate margin call orders.	`0.10` (10%)

For more information about this model, see the class reference and implementation .

Model Structure

Margin call models must extend the `DefaultMarginCallModel` class. Extensions of the `DefaultMarginCallModel` class can override the `get_margin_call_orders` and `execute_margin_call` methods.

The `get_margin_call_orders` method scans the portfolio and the updated data for a potential margin call situation that may get the equity below zero. The method must return a tuple where the first element is a list of `SubmitOrderRequest` objects that represent the margin call orders and the second element is a boolean that represents whether or not to issue a margin call warning.

The `execute_margin_call` method receives the list of `SubmitOrderRequest` objects from the `get_margin_call_orders` method, executes some of them, and returns a list of `OrderTicket` objects.

```
class CustomMarginCallModelExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # Set margin call model for the whole portfolio, better with reference to your broker information
        self.portfolio.set_margin_call_model(MyMarginCallModel(self.portfolio, self.default_order_properties))

# Define the custom margin call model
class MyMarginCallModel(DefaultMarginCallModel):
    def __init__(self,
                 portfolio: SecurityPortfolioManager,
                 default_order_properties: IOrderProperties):
        super().__init__(portfolio, default_order_properties)

    def execute_margin_call(self,
                 generatedMarginCallOrders: List[SubmitOrderRequest]) -> List[OrderTicket]:
        return super().execute_margin_call(generatedMarginCallOrders)

    def get_margin_call_orders(self,
                 issue_margin_call_warning: bool | None) -> tuple[List[SubmitOrderRequest], bool]:
        (margin_call_orders, issue_margin_call_warning) = super().get_margin_call_orders(issue_margin_call_warning)
        return (margin_call_orders, issue_margin_call_warning)
```

Disable Margin Calls

To disable margin calls, set the margin call model to the `NullMarginCallModel` .

```
def initialize(self) -> None:
 self.portfolio.margin_call_model = MarginCallModel.NULL
```

Monitor Margin Call Events

When the margin call model of your portfolio issues a margin call warning, we notify your algorithm through the `on_margin_call_warning` event handler.

```
def on_margin_call_warning(self) -> None:
    self.debug(f"Warning: Close to margin call")
```

Before we send the orders that the margin call model produces, we notify your algorithm through the `on_margin_call` event handler. This notification gives your algorithm a chance to liquidate some positions or modify the margin call orders. To modify the orders, adjust the list of `SubmitOrderRequest` objects the event handler receives.

```
def on_margin_call(self, requests: List[SubmitOrderRequest]) -> List[SubmitOrderRequest]:
    for i, order in enumerate(requests):
        # liquidate an extra 10% each time you get a margin call to give yourself more padding
        new_quantity = int(order.quantity * 1.1)
        requests[i] = SubmitOrderRequest(order.order_type, order.security_type,
                                         order.symbol, new_quantity, order.stop_price,
                                         order.limit_price, 0, self.time, "OnMarginCall")
    return requests
```

Submit Order Request

If you receive a margin call or create a margin call model, you'll need to work with `SubmitOrderRequest` objects. These objects have the following properties:

The following table describes the arguments of the `SubmitOrderRequest` constructor:

Argument	Data Type	Description	Default Value
order_type	OrderType	The order type to be submitted	
security_type	SecurityType	The security type of the asset	
symbol	Symbol	The symbol to be traded	
quantity	float	The number of units to order	
stop_price	float	The stop price for stop orders	
limit_price	float	The limit price for limit orders	
trigger_price	float	The trigger price for limit if touched orders	
time	datetime	The time this request was created	
tag	str	A custom tag for this request	
properties	IOrderProperties	The order properties for this request	None

Examples

The following examples demonstrate common practices for implementing a custom margin call model.

Example 1: Default Model Wrapper

The following algorithm defines a subclass of the DefaultMarginCallModel and overrides its built-in members so you can easily customize the parts you need.

```py
class CustomFillModelAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.portfolio.set_margin_call_model(MyMarginCallModel(self.portfolio, self.default_order_properties))
        self.add_equity('SPY', Resolution.DAILY)

    def on_data(self, data: Slice) -> None:
        if not self.portfolio.invested:
            self.set_holdings('SPY', -2)


class MyMarginCallModel(DefaultMarginCallModel):
    def __init__(self,
         portfolio: SecurityPortfolioManager,
         default_order_properties: IOrderProperties):
        super().__init__(portfolio, default_order_properties)

    def execute_margin_call(self,
         generatedMarginCallOrders: List[SubmitOrderRequest]) -> List[OrderTicket]:
        return super().execute_margin_call(generatedMarginCallOrders)

    def get_margin_call_orders(self,
         issue_margin_call_warning: bool | None) -> tuple[List[SubmitOrderRequest], bool]:
        (margin_call_orders, issue_margin_call_warning) = super().get_margin_call_orders(issue_margin_call_warning)
        return (margin_call_orders, issue_margin_call_warning)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
MarginCallEventsAlgorithm.py Python

Reality Modeling

Short Availability

Short Availability

Key Concepts

Introduction

To short a security, you need to borrow shares from another investor or organization that owns the shares. A shortable provider is a model that tracks the number of shares that are available for you to borrow. A shortable provider can make your backtest results more realistic because it doesn't let you place a short trade if there are no shares available for you to borrow.

Set Providers

The brokerage model of your algorithm automatically sets the settlement model for each security, but you can override it. To manually set the shortable provider of a security, call the `set_shortable_provider` method on the `Security` object.

PY

```
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # Get shortable stocks and quantity from local disk
    security.set_shortable_provider(LocalDiskShortableProvider("axos"))
```

You can also set the shortable provider in a security initializer. If your algorithm has a universe, use the security initializer technique. In order to initialize single security subscriptions with the security initializer, call `add_security_initializer` before you create the subscriptions.

PY

```
def initialize(self) -> None:
    # Set the security initializer before requesting data to apply to all requested securities afterwards
    self.set_security_initializer(self.custom_security_initializer)
    self.add_equity("SPY")

def custom_security_initializer(self, security: Security) -> None:
    security.set_shortable_provider(LocalDiskShortableProvider("axos"))
```

If you call the `set_security_initializer` method, it overwrites the default security initializer. The default security initializer uses the security-level reality models of the brokerage model to set the following reality models of each security:

- Fill
- Slippage
- Fee
- Buying Power
- Settlement
- Short availability
- Margin Interest Rate

To extend upon the default security initializer instead of overwriting it, call `add_security_initializer` before you create the subscriptions before you create the subscriptions.

```
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_shortable_provider(LocalDiskShortableProvider("axos"))
```

To view all the pre-built shortable providers, see Supported Providers .

Default Behavior

The brokerage model of your algorithm automatically sets the shortable provider for each security. The default brokerage model is the `DefaultBrokerageModel` , which uses the NullShortableProvider .

Provider Structure

Shortable providers must implement the `IShortableProvider` interface. Shortable providers that implement the `IShortableProvider` interface must implement the `fee_rate` , `rebate_rate` , and `shortable_quantity` methods. These methods receives a Symbol and the local time of the algorithm. The `fee_rate` method returns the borrow fee rate. The `rebate_rate` method returns the borrow rebate rate. The `shortable_quantity` returns the shortable quantity.

```
class CustomShortableProviderExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        security = self.add_equity("SPY")
        # Provide the selected security with your broker's shortable information
        security.set_shortable_provider(MyShortableProvider())

# Define the custom shortable provider
class MyShortableProvider(NullShortableProvider):

    def fee_rate(self, symbol: Symbol, local_time: datetime) -> float:
        return 0.0025

    def rebate_rate(self, symbol: Symbol, local_time: datetime) -> float:
        return 0.0507

    def shortable_quantity(self, symbol: Symbol, local_time: datetime) -> int:
        return 10000
```

Order Failures

If you place an order to short more shares than are available to borrow, LEAN rejects the order and applies the following tag: "Order exceeds maximum shortable quantity for Symbol ticker (requested short: quantity)".

Borrowing Costs

In live trading, your brokerage charges an interest fee when you borrow shares to short a security. In backtesting, use the ShortMarginInterestRateModel .

To get valid borrowing rates, use the InteractiveBrokersShortableProvider .

```
def initialize(self) -> None:
    security = self.add_equity("SPY")
    # For US Equity, use the ShortMarginInterestRateModel with IB data
    security.set_shortable_provider(InteractiveBrokersShortableProvider())
    security.margin_interest_rate_model = ShortMarginInterestRateModel()
```

Examples

The following examples demonstrate common practices for using a shortable provider.

Example 1: Fixed Rates

The following algorithm defines a custom short availability model with fixed values for the fee rate, rebate rate, and shortable quantity.

PY

```
class CustomShortableProviderAlgorithm(QCAlgorithm):

    def Initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self._security = self.add_equity("QQQ", Resolution.DAILY)
        self._security.set_shortable_provider(MyShortableProvider())

    def on_data(self, data: Slice) -> None:
        if self.portfolio.invested:
            return
        shortable_quantity = self._security.shortable_provider.shortable_quantity(self._security.symbol, self.Time)
        if not shortable_quantity:
            return
        fee_rate = self._security.shortable_provider.fee_rate(self._security.symbol, self.time)
        rebate_rate = self._security.shortable_provider.rebate_rate(self._security.symbol, self.time)
        self.set_holdings(self._security.symbol, -0.05, tag=f'Borrow fee rate {fee_rate}. Borrow rebate rate {rebate_rate}')


class MyShortableProvider(NullShortableProvider):

    def fee_rate(self, symbol: Symbol, local_time: datetime) -> float:
        return 0.0025

    def rebate_rate(self, symbol: Symbol, local_time: datetime) -> float:
        return 0.0507

    def shortable_quantity(self, symbol: Symbol, local_time: datetime) -> int:
        return 10000
```

Example 2: Sizing By Shortable Quantity

The following algorithm trades arbitrage of class A and class C price divergence of Google stocks using cointegration analysis. However, the position sizing may be limited by the available quantity to be shorted by the brokerage, breaking the paired trade. Thus, we need to make use of the `shortable_quantity` method of the shortable provider to calculate the quantity.

PY

```
from sklearn.linear_model import LinearRegression
from statsmodels.tsa.stattools import adfuller

class ShortableProviderAlgorithm(QCAlgorithm):
    # The threshold that the spread/residual of the cointegrated series triggers a trade.
    threshold = 0
    # Store the coefficient and intercept of the cointegrated series for calculating the spread of a new data point.
    coefficients = [0, 0]

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Subscribe to 2 classes of Google stocks to trade their price divergence.
        goog1 = self.add_equity("GOOGL", Resolution.DAILY)        # Class A
        goog2 = self.add_equity("GOOG", Resolution.DAILY)         # Class C
        self.goog1 = goog1.symbol
        self.goog2 = goog2.symbol
        # Set the shortable providers to simulate the available shares to trade realistically.
        goog1.set_shortable_provider(InteractiveBrokersShortableProvider())
        goog2.set_shortable_provider(InteractiveBrokersShortableProvider())

        # Use rolling windows to save the price data for cointegration analysis.
        goog1.window = RollingWindow(252)
        goog2.window = RollingWindow(252)
        # Warm up the rolling windows.
        history = self.history[TradeBar]([self.goog1, self.goog2], 252, Resolution.DAILY)
        for bar in history:
            goog1.window.add(bar[self.goog1].close)
            goog2.window.add(bar[self.goog2].close)

        # Adjust the cointegration factor between the 2 classes' monthly price series.
        self.schedule.on(
            self.date_rules.month_start(),
            self.time_rules.at(0, 1),
            self.calculate_cointegration
        )
        self.calculate_cointegration()

    def on_data(self, slice: Slice) -> None:
        bar1 = slice.bars.get(self.goog1)
        bar2 = slice.bars.get(self.goog2)
        if bar1 and bar2:
            # Update the rolling windows for cointegration analysis.
            self.securities[self.goog1].window.add(bar1.close)
            self.securities[self.goog2].window.add(bar2.close)

            # Calculate the current cointegrated series spread
```

```
            # Calculate the current cointegrated series spread.
            residual = self.coefficients[0] * bar2.close + self.coefficients[1] - bar1.close

            quantity = self.calculate_order_quantity(self.goog1, 0.2)
            # If the residual is lower than the negative threshold, it means class A's price is much higher than it should be compared to
class C.
            # We sell class A and buy class C to bet on their price convergence.
            if residual < -self.threshold and not self.portfolio[self.goog1].is_short:
                # Since it will be limited by shortable quantity, we max{shortable, 20% of portfolio value}.
                shortable_quantity = self.securities[self.goog1].shortable_provider.shortable_quantity(self.goog1, bar1.end_time)
                quantity = min(quantity, shortable_quantity)
                self.market_order(self.goog1, -quantity)
                self.market_order(self.goog2, quantity * self.coefficients[0])
            # If the residual is higher than the threshold, it means class A price is much lower than what it should be compared to class
C.
            # We buy class A and sell class C to bet on their price convergence.
            elif residual > self.threshold and not self.portfolio[self.goog1].is_long:
                # Since it will be limited by shortable quantity, we max{shortable, 20% of portfolio value}.
                shortable_quantity = self.securities[self.goog2].shortable_provider.shortable_quantity(self.goog2, bar2.end_time)
                quantity = min(quantity, shortable_quantity)
                self.market_order(self.goog1, quantity)
                self.market_order(self.goog2, -quantity * self.coefficients[0])
            # Close positions of the price are converged.
            elif (self.portfolio[self.goog1].is_short and residual > 0) or (self.portfolio[self.goog1].is_long and residual < 0):
                self.liquidate()

    def calculate_cointegration(self) -> None:
        # Lag direction is unimportant; it is just a sign flip in the linear regression, so we don't need to flip the window order.
        y = np.array(list(self.securities[self.goog1].window)).reshape(-1, 1)
        x = np.array(list(self.securities[self.goog2].window)).reshape(-1, 1)

        # Perform Linear Regression on both price series to investigate their relationship.
        lr = LinearRegression().fit(x, y)
        slope = lr.coef_[0]
        intercept = lr.intercept_

        # Calculate the residuals series to check if it is stationary, meaning if the 2 price series move together.
        residuals = y - (intercept + slope * x)

        # Check if the residuals are stationary using the augmented Dickey-Fuller test.
        # Reject the null hypothesis of a unit root is present if test statistic <= -3.45 (approximate α=0.05 for n=250)
        # This means no unit root exists for the difference series, and the residuals are stationary.
        adf_reject = adfuller(residuals)[0] <= -3.45
        if adf_reject:
            # If cointegrated, update the positional sizing ratio and the spread threshold of the trade trigger.
            self.coefficients = [slope, intercept]
            self.threshold = 2 * np.std(residuals)
        else:
            # If not cointegrated, liquidate and set the size to zeros for no positions.
            self.liquidate()
            self.coefficients = [0, 0]
            self.threshold = 100000000          # An arbitrarily large number that the class A price will never reach.
```

Other Examples

Example 3: Interactive Brokers Shortable Provider

The following algorithm uses the `InteractiveBrokersShortableProvider` to check the shortable quantity and borrowing costs of GME before placing a short order.

```
class InteractiveBrokersShortableProviderAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self._security = self.add_equity("GME", Resolution.DAILY)
        self._security.set_shortable_provider(InteractiveBrokersShortableProvider())

    def on_data(self, data: Slice) -> None:
        if self.portfolio.invested:
            return
        shortable_quantity = self._security.shortable_provider.shortable_quantity(self._security.symbol, self.time)
        if not shortable_quantity:
            return
        fee_rate = self._security.shortable_provider.fee_rate(self._security.symbol, self.time)
        rebate_rate = self._security.shortable_provider.rebate_rate(self._security.symbol, self.time)
        # Only short if the borrow fee is reasonable.
        if fee_rate < 0.05:
            quantity = int(min(self.calculate_order_quantity(self._security.symbol, -0.5), shortable_quantity))
            if quantity != 0:
                self.market_order(self._security.symbol, quantity, tag=f'Fee rate: {fee_rate}, Rebate rate: {rebate_rate}, Shortable:
{shortable_quantity}')
```

For more examples, see the following algorithms:

Demonstration Algorithms
CustomShortableProviderRegressionAlgorithm.py Python AllShortableSymbolsCoarseSelectionRegressionAlgorithm.py Python

Short Availability

Supported Providers

Introduction

This page describes all of the pre-built shortable providers in LEAN. .

Null Provider

The `NullShortableProvider` doesn't provide shortable data, so it allows you to infinitely short assets. It's the shortable provider of the `DefaultBrokerageModel` .

```
security.set_shortable_provider(NullShortableProvider())
```

For more information about this model, see the class reference and implementation .

Local Disk Provider

The `LocalDiskShortableProvider` sources short availability data from the local disk of a specific brokerage.

```
security.set_shortable_provider(LocalDiskShortableProvider("axos"))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`brokerage`	`str`	Brokerage to read data	

For more information about this model, see the class reference and implementation .

Interactive Brokers Provider

The `InteractiveBrokersShortableProvider` sources short availability data from Interactive Brokers.

```
security.set_shortable_provider(InteractiveBrokersShortableProvider())
```

For more information about this model, see the class reference and implementation .

Scheduled Events

Introduction

Scheduled Events let you trigger code to run at specific times of day, regardless of your algorithm's data subscriptions. It's easier and more reliable to execute time-based events with Scheduled Events than checking the current algorithm time in the `on_data` event handler.

Create Scheduled Events

To create a Scheduled Event, call the `schedule.on` method. The method expects a `DateRules` object, a `TimeRules` object, and a function to execute. The following examples demonstrate some common Scheduled Events.

Schedule Events Before Market Open

You may want to train a model or fetch historical data before the market opens. The following example demonstrates how to set a Scheduled Event for 10 minutes before the market opens.

```py
def initialize(self):
    # Call the _before_market_open method 10 minutes before open every day SPY is trading.
    self.schedule.on(
        self.date_rules.every_day('SPY'),
        self.time_rules.before_market_open('SPY', 10),
        self._before_market_open
    )
```

Schedule Events on the Last Trading Day of the Week

You may want to rebalance your portfolio on the last trading day of each week and factor in market holidays. The following example demonstrates how to set a Scheduled Event for the last trading day of each week 30 minutes before the market closes.

```py
def initialize(self):
# Call the _rebalance method 30 minutes before the end of the SPY trading week.
    self.schedule.on(
        self.date_rules.week_end('SPY'),
        self.time_rules.before_market_close('SPY', 30),
        self._rebalance
    )
```

Schedule Events on Regular Intervals Throughout the Trading Day

You may want to perform some action on a regular interval through each trading day. The following example demonstrates how to set a Scheduled Event for every 30 minutes through the trading day for SPY.

```py
def initialize(self):
    # Every day SPY is trading, call the '_some_action' method every 30 minutes.
    self.schedule.on(
        self.date_rules.every_day('SPY'),
        self.time_rules.every(timedelta(minutes=30)),
        self._some_action
    )
```

Date Rules

The following table describes the supported `DateRules` :

Member	Description
`self.date_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `DateRules` object used in all methods in this table. The default time zone is the algorithm time zone .
`self.date_rules.on(year: int, month: int, day: int)`	Trigger an event on a specific date.
`self.date_rules.on(dates: list[datetime])`	Trigger an event on specific dates.
`self.date_rules.every_day()`	Trigger an event all seven days of the week.
`self.date_rules.every_day(symbol: Symbol, extended_market_hours: bool = False)`	Trigger an event every day a specific symbol is trading.
`self.date_rules.every(days: list[DayOfWeek])`	Trigger an event on specific days throughout the week. To view the `DayOfWeek` enum members, see DayOfWeek Enum in the .NET documentation.
`self.date_rules.month_start(days_offset: int = 0)`	Trigger an event on the first day of each month plus an offset.
`self.date_rules.month_start(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the first tradable date of each month for a specific symbol plus an offset.
`self.date_rules.month_end(days_offset: int = 0)`	Trigger an event on the last day of each month minus an offset.
`self.date_rules.month_end(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the last tradable date of each month for a specific symbol minus an offset.
`self.date_rules.week_start(days_offset: int = 0)`	Trigger an event on the first day of each week plus an offset.
`self.date_rules.week_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each week for a specific symbol plus an offset.
`self.date_rules.week_end(days_offset: int = 0)`	Trigger an event on the last day of each week minus an offset.
`self.date_rules.week_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each week for a specific symbol minus an offset.
`self.date_rules.year_start(days_offset: int = 0)`	Trigger an event on the first day of each year plus an offset.
`self.date_rules.year_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each year for a specific symbol plus an offset.
`self.date_rules.year_end(days_offset: int = 0)`	Trigger an event on the last day of each year minus an offset.
`self.date_rules.year_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each year for a specific symbol minus an offset.
`self.date_rules.today`	Trigger an event once today.
`self.date_rules.tomorrow`	Trigger an event once tomorrow.

To define custom date rules, create a `FuncDateRule` object. The `FuncDateRule` constructor expects a `name` argument of type `str` and a `get_dates_function` argument of type `Callable[[datetime, datetime], list[datetime]]` . The `get_dates_function` function receives the start and end dates of the algorithm and returns a list of dates for the date rule. In live trading, the end date is 12/31/2025. The following example demonstrates how to define a date rule that represents the 10th day of each month:

```py
# Create a date rule that specifies the 10th day of each month.
date_rule = FuncDateRule(
    name="10th_day_of_the_month",
    get_dates_function=lambda start, end: [
        datetime(year, month, 10)
        for year in range(start.year, end.year) for month in range(1,12)
    ]
)
```

Some date rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.date_rules.every_day('ES')` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

Time Rules

The following table describes the supported `TimeRules` :

Member	Description
`self.time_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `TimeRules` object used in all methods in this table, except when a different time zone is given. The default time zone is the algorithm time zone .
`self.time_rules.before_market_open(symbol: Symbol, minutes_before_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_open(symbol: Symbol, minutes_after_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.before_market_close(symbol: Symbol, minutes_before_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_close(symbol: Symbol, minutes_after_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.every(interval: timedelta)`	Trigger an event every period interval starting at midnight. This time rule triggers every period, regardless of whether or not the market is open.
`self.time_rules.now`	Trigger an event at the current time of day.
`self.time_rules.midnight`	Trigger an event at midnight.
`self.time_rules.noon`	Trigger an event at noon.
`self.time_rules.at(hour: int, minute: int, second: int = 0)`	Trigger an event at a specific time of day (e.g. 13:10).
`self.time_rules.at(hour: int, minute: int, second: int, time_zone: DateTimeZone)`	Trigger an event at a specific time of day in the given time zone (e.g. 13:10 UTC).

To define custom time rules, create a `FuncTimeRule` object. The `FuncTimeRule` constructor expects a `name` argument of type `str` and a `create_utc_event_times_function` argument of type `Callable[[list[datetime]], list[datetime]]` . The function receives the list of dates from the date rule and then returns a list of `datetime` that define the time rule.

```
time_rule = FuncTimeRule(
    name="CustomTimeRule",
    create_utc_event_times_function=lambda dates: [d + timedelta(hours=10) for d in dates]
)
```

Some time rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.time_rules.before_market_open('ES', 15)` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

Remove Scheduled Events

If you no longer need a Scheduled Event in your algorithm, remove it so your algorithm doesn't execute unnecessary functions. To remove a Scheduled Event, save a reference to the Scheduled Event when you create it and then call the `remove` method to remove it.

```
PY
def initialize(self):
    # Remove a scheduled event by keeping a copy of the object on creation.
    self.scheduled_event = self.schedule.on(
        self.date_rules.every_day('SPY'),
        self.time_rules.after_market_open('SPY', 10),
        self._ten_minutes_after_open)

# Remove the scheduled event using the object reference in any method
self.schedule.remove(self.scheduled_event)
```

Common Errors

Common errors with Scheduled Events include stale fills and execution timeouts.

Stale Fills

A common error is to subscribe to daily resolution data and set a Scheduled Event intraday to place trades. If you trade intraday with daily data, you get stale fills.

Another common error is to set a Scheduled Event to trade immediately after the market open on illiquid securities. Illiquid securities can have no trades for the first few minutes after the market open. If you trade during this time, you get stale fills.

Execution Timeouts

If your Scheduled Event takes longer than 10 minutes to execute, your algorithm will timeout. To increase the amount of time that your Scheduled Event can run, replace your Scheduled Event with a training session .

Invoking the Callback Method

Make sure to not accidentally invote the event handler method by adding trailing braces `()` . Use `self._event_handler_func` , not `self._event_handler_func()` .

Execution Sequence

The algorithm manager calls events in the following order:

1. Scheduled Events
2. Consolidation event handlers
3. `on_data` event handler

This event flow is important to note. For instance, if your consolidation handlers or `on_data` event handler appends data to a `RollingWindow` and you use that `RollingWindow` in your Scheduled Event, when the Scheduled Event executes, the `RollingWindow` won't contain the most recent data.

Live Trading Considerations

In live trading, Scheduled Events execute in a parallel thread based on a real-time clock. If you set a Scheduled Event to fire at 10:00 AM, it executes at exactly 10:00 AM. In backtesting, Scheduled Events are part of the main algorithm manager loop, so they may not execute exactly when you set them. For example, if your algorithm subscribes to minute resolution US Equity data with regular trading hours and you set a Scheduled Event to occur at 2:00 AM, your Scheduled Event will execute at 9:31 AM when the next bar is fed into your algorithm.

The difference between live trading and backtesting is important to note because it can affect your algorithm's behavior. There are two common scenarios to consider.

Execution Timing and Backtest Timeouts

Take the following scenario:

- You set Scheduled Events for 2:00 AM, 3:00 AM, and 4:00 AM each day.
- Each Scheduled Event takes eight minutes to execute.
- Your algorithm only subscribes to US Equity securities without extended market hours (9:30 AM - 4:00 PM).

In this scenario, the Scheduled Events each fire at the correct time and execute without error in live trading. In backtesting, all of the Scheduled Events execute at 9:31 AM when your algorithm receives the first bar of the trading day. Since all the Scheduled Events take eight minutes to execute, the algorithm tries to execute all the Scheduled Events but reaches the 10-minute timeout and the backtest stops execution.

Live Data Delays

In backtests, your algorithm receives data at perfect timing. If you request minute resolution data, your algorithm receives the bars at the top of each minute. In live trading, bars have a slight delay, so you may receive them milliseconds after the top of each minute. Take the following scenario:

- You subscribe to minute resolution data
- You set a Scheduled Event for 10:00 AM
- The Scheduled Event checks the current asset price

In live trading, the Scheduled Event executes at exactly 10:00 AM but your algorithm may receive the 9:59-10:00 AM bar at 10:00:00.01 AM. Therefore, when you check the price in the Scheduled Event, the price from the 9:58-9:59 AM bar is the latest price. In backtesting, the Scheduled Event gets the price from the 9:59-10:00 AM bar since your algorithm receives the bar at perfect timing.

Examples

PY

```
def initialize(self):
    # Schedule an event to fire at a specific date/time using inline callback function.
    self.schedule.on(self.date_rules.on(2013, 10, 7),
                     self.time_rules.at(13, 0),
                     lambda: self.log(f"SpecificTime: Fired at : {self.time}"))

    # Schedule an event to fire every trading day for a security at 13:00:00 UTC.
    self.schedule.on(self.date_rules.every_day("SPY"),
                     self.time_rules.at(13, 0, 0, TimeZones.UTC),
                     lambda: self.log(f"EveryDay.SPY SpecificTime: Fired at: {self.time}"))

    # Schedule an event to fire every trading day for a security - 10 minutes before SPY's market open.
    self.schedule.on(self.date_rules.every_day("SPY"),
                     self.time_rules.before_market_open("SPY", 10),
                     lambda: self.log(f"EveryDay.SPY 10 min before open: Fired at: {self.time}"))

    # Schedule an event to fire every trading day for a security - 10 minutes after SPY's market open.
    self.schedule.on(self.date_rules.every_day("SPY"),
                     self.time_rules.after_market_open("SPY", 10),
                     lambda: self.log(f"EveryDay.SPY 10 min after open: Fired at: {self.time}"))

    # Schedule an event to fire every trading day for a security - 10 minutes before SPY's market close.
    self.schedule.on(self.date_rules.every_day("SPY"),
                     self.time_rules.before_market_close("SPY", 10),
                     lambda: self.log(f"EveryDay.SPY 10 min before close: Fired at: {self.time}"))

    # Schedule an event to fire every trading day for a security - 10 minutes after SPY's market close.
    self.schedule.on(self.date_rules.every_day("SPY"),
                     self.time_rules.after_market_close("SPY", 10),
                     lambda: self.log(f"EveryDay.SPY 10 min after close: Fired at: {self.time}"))

    # Schedule an event to fire on certain days of the week
    self.schedule.on(self.date_rules.every(DayOfWeek.MONDAY, DayOfWeek.FRIDAY),
                     self.time_rules.at(12, 0),
                     lambda: self.log(f"Mon/Fri at 12pm: Fired at: {self.time}"))

    # Schedule an event to fire once today at when this method is called (now)
    self.schedule.on(self.date_rules.today,
                     self.time_rules.now,
                     lambda: self.log(f"Now: Fired at: {self.time}"))

    # Schedule an event to fire once tomorrow at midnight
    self.schedule.on(self.date_rules.tomorrow,
                     self.time_rules.midnight,
                     lambda: self.log(f"Tomorrow at midnight: Fired at: {self.time}"))

    # Schedule an event to fire once today at noon
    self.schedule.on(self.date_rules.today,
                     self.time_rules.noon,
                     lambda: self.log(f"Today at noon: Fired at: {self.time}"))

    # Scan a method every ten minutes, every day.
    self.schedule.on(self.date_rules.every_day(),
                     self.time_rules.every(timedelta(minutes=10)),
                     self._liquidate_unrealized_losses)

    # Schedule an event to fire at the beginning of the month.
    # The symbol is optional but if specified, it will fire the first trading day
    # for that symbol of the month. Otherwise, it will fire on the first day of the month.
    self.schedule.on(self.date_rules.month_start("SPY"),
                     self.time_rules.after_market_open("SPY"),
                     self._rebalancing_code)

    # Schedule an event to fire at the end of the month, the symbol is optional.
```

```
        # If specified, it will fire the last trading day for that symbol of the month.
        # Otherwise, it will fire on the last day of the month.
        self.schedule.on(self.date_rules.month_end("SPY"),
                         self.time_rules.before_market_close("SPY"),
                         self._rebalancing_code)

        # Schedule an event to fire at the beginning of the week, the symbol is optional.
        # If specified, it will fire the first trading day for that symbol of the week.
        # Otherwise, it will fire on the first day of the week.
        self.schedule.on(self.date_rules.week_start("SPY"),
                         self.time_rules.after_market_open("SPY", 5),
                         self._rebalancing_code)

        # Schedule an event to fire at the end of the week, the symbol is optional.
        # If specified, it will fire the last trading day for that symbol of the week.
        # Otherwise, it will fire on the last day of the week.
        self.schedule.on(self.date_rules.week_end("SPY"),
                         self.time_rules.before_market_close("SPY", 5),
                         self._rebalancing_code)

    # The following methods from examples above should be defined in the algorithm body.
    def _liquidate_unrealized_losses(self) -> None:
        ''' if we have over 1000 dollars in unrealized losses, liquidate'''
        if self.portfolio.total_unrealized_profit < -1000:
            self.log(f"Liquidated due to unrealized losses at: {self.time}")
            self.liquidate()

    def _rebalancing_code(self) -> None:
        ''' Good spot for rebalancing code?'''
        pass
```

Demonstration Algorithm
ScheduledEventsAlgorithm.py Python

Indicators

Indicators

Supported Indicators

Indicators translate a stream of data points into a numerical value you can use to detect trading opportunities. LEAN provides more than 100 pre-built technical indicators and candlestick patterns you can use in your algorithms. You can use any of the following indicators. Click one to learn more.

Candlestick Patterns

Absolute Price Oscillator

Acceleration Bands

Accumulation Distribution

Accumulation Distribution Oscillator

Advance Decline Difference

Advance Decline Ratio

Advance Decline Volume Ratio

Alpha

Arms Index

Arnaud Legoux Moving Average

Aroon Oscillator

Augen Price Spike

Auto Regressive Integrated Moving Average

Average Directional Index

Average Directional Movement Index Rating

Average Range

Average True Range

Awesome Oscillator

Balance Of Power

Beta

Bollinger Bands

Chaikin Money Flow

Chaikin Oscillator

Chande Kroll Stop

Chande Momentum Oscillator

Choppiness Index

Commodity Channel Index

Connors Relative Strength Index

Coppock Curve

Correlation

Covariance

DeMarker Indicator

Delta

Derivative Oscillator

Detrended Price Oscillator

Donchian Channel

Double Exponential Moving Average

Ease Of Movement Value

Exponential Moving Average

Filtered Identity

Fisher Transform

Force Index

Fractal Adaptive Moving Average

Gamma

Heikin Ashi

Hilbert Transform

Hull Moving Average

Hurst Exponent

Ichimoku Kinko Hyo

Identity

Implied Volatility

Internal Bar Strength

Intraday Vwap

Kaufman Adaptive Moving Average

Kaufman Efficiency Ratio

Keltner Channels

Klinger Volume Oscillator

Know Sure Thing

Least Squares Moving Average

Linear Weighted Moving Average

Log Return

Mass Index

Maximum

McClellan Oscillator

McClellan Summation Index

McGinley Dynamic

Mean Absolute Deviation

Mesa Adaptive Moving Average

Mid Point

Mid Price

Minimum

Momentum

Momentum Percent

Momersion

Money Flow Index

Moving Average Convergence Divergence

New Highs New Lows

New Highs New Lows Volume

Normalized Average True Range

On Balance Volume

Parabolic Stop And Reverse

Parabolic Stop And Reverse Extended

Percentage Price Oscillator

Pivot Points High Low

Premier Stochastic Oscillator

Rate Of Change

Rate Of Change Percent

Rate Of Change Ratio

Regression Channel

Relative Daily Volume

Relative Moving Average

Relative Strength Index

Relative Vigor Index

Rho

Rogers Satchell Volatility

Schaff Trend Cycle

Sharpe Ratio

Simple Moving Average

Smoothed On Balance Volume

Sortino Ratio

Squeeze Momentum

Standard Deviation

Stochastic

Stochastic Relative Strength Index

Sum

Super Trend

Swiss Army Knife

T3 Moving Average

Target Downside Deviation

Theta

Time Profile

Time Series Forecast

Tom Demark Sequential

Triangular Moving Average

Triple Exponential Moving Average

Trix

True Range

True Strength Index

Ultimate Oscillator

Value At Risk

Variable Index Dynamic Average

Variance

Vega

Volume Profile

Volume Weighted Average Price Indicator

Volume Weighted Moving Average

Vortex

Wilder Accumulative Swing Index

Wilder Moving Average

Wilder Swing Index

Williams Percent R

Zero Lag Exponential Moving Average

Zig Zag

Supported Indicators

Candlestick Patterns

You can use any of the following candlestick patterns. Click one to learn more.

Abandoned Baby

Advance Block

Belt Hold

Breakaway

Closing Marubozu

Concealed Baby Swallow

Counterattack

Dark Cloud Cover

Doji

Doji Star

Dragonfly Doji

Engulfing

Evening Doji Star

Evening Star

Gap Side By Side White

Gravestone Doji

Hammer

Hanging Man

Harami

Harami Cross

High Wave Candle

Hikkake

Hikkake Modified

Homing Pigeon

Identical Three Crows

In Neck

Inverted Hammer

Kicking

Kicking By Length

Ladder Bottom

Long Legged Doji

Long Line Candle

Marubozu

Mat Hold

Matching Low

Morning Doji Star

Morning Star

On Neck

Piercing

Rickshaw Man

Rise Fall Three Methods

Separating Lines

Shooting Star

Short Line Candle

Spinning Top

Stalled Pattern

Stick Sandwich

Takuri

Tasuki Gap

Three Black Crows

Three Inside

Three Line Strike

Three Outside

Three Stars In South

Three White Soldiers

Thrusting

Tristar

Two Crows

Unique Three River

Up Down Gap Three Methods

Upside Gap Two Crows

Supported Indicators

Abandoned Baby

Introduction

Create a new Abandoned Baby candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using AbandonedBaby Indicator

To create an automatic indicator for `AbandonedBaby` , call the `abandoned_baby` helper method from the `QCAlgorithm` class. The `abandoned_baby` method creates a `AbandonedBaby` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class AbandonedBabyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._abandoned_baby = self.candlestick_patterns.abandoned_baby(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._abandoned_baby.is_ready:
            # The current value of self._abandoned_baby is represented by self._abandoned_baby.current.value
            self.plot("AbandonedBaby", "abandoned_baby", self._abandoned_baby.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `AbandonedBaby` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AbandonedBabyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._abandoned_baby = AbandonedBaby()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._abandoned_baby.update(bar)

        if self._abandoned_baby.is_ready:
            # The current value of self._abandoned_baby is represented by self._abandoned_baby.current.value
            self.plot("AbandonedBaby", "abandoned_baby", self._abandoned_baby.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `AbandonedBaby` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AbandonedBabyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._abandoned_baby = self.candlestick_patterns.abandoned_baby(self._symbol)

        indicator_history = self.indicator_history(self._abandoned_baby, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._abandoned_baby, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._abandoned_baby, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Advance Block

Introduction

Create a new Advance Block candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using AdvanceBlock Indicator

To create an automatic indicator for `AdvanceBlock` , call the `advance_block` helper method from the `QCAlgorithm` class. The `advance_block` method creates a `AdvanceBlock` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class AdvanceBlockAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advance_block = self.candlestick_patterns.advance_block(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._advance_block.is_ready:
            # The current value of self._advance_block is represented by self._advance_block.current.value
            self.plot("AdvanceBlock", "advance_block", self._advance_block.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `AdvanceBlock` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AdvanceBlockAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advance_block = AdvanceBlock()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._advance_block.update(bar)

        if self._advance_block.is_ready:
            # The current value of self._advance_block is represented by self._advance_block.current.value
            self.plot("AdvanceBlock", "advance_block", self._advance_block.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `AdvanceBlock` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AdvanceBlockAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advance_block = self.candlestick_patterns.advance_block(self._symbol)

        indicator_history = self.indicator_history(self._advance_block, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._advance_block, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._advance_block, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Belt Hold

Introduction

Create a new Belt Hold candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using BeltHold Indicator

To create an automatic indicator for `BeltHold` , call the `belt_hold` helper method from the `QCAlgorithm` class. The `belt_hold` method creates a `BeltHold` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class BeltHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._belt_hold = self.candlestick_patterns.belt_hold(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._belt_hold.is_ready:
            # The current value of self._belt_hold is represented by self._belt_hold.current.value
            self.plot("BeltHold", "belt_hold", self._belt_hold.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `BeltHold` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class BeltHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._belt_hold = BeltHold()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._belt_hold.update(bar)

        if self._belt_hold.is_ready:
            # The current value of self._belt_hold is represented by self._belt_hold.current.value
            self.plot("BeltHold", "belt_hold", self._belt_hold.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `BeltHold` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class BeltHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._belt_hold = self.candlestick_patterns.belt_hold(self._symbol)

        indicator_history = self.indicator_history(self._belt_hold, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._belt_hold, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._belt_hold, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Breakaway

Introduction

Create a new Breakaway candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Breakaway Indicator

To create an automatic indicator for `Breakaway` , call the `breakaway` helper method from the `QCAlgorithm` class. The `breakaway` method creates a `Breakaway` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class BreakawayAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._breakaway = self.candlestick_patterns.breakaway(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._breakaway.is_ready:
            # The current value of self._breakaway is represented by self._breakaway.current.value
            self.plot("Breakaway", "breakaway", self._breakaway.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Breakaway` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class BreakawayAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._breakaway = Breakaway()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._breakaway.update(bar)

        if self._breakaway.is_ready:
            # The current value of self._breakaway is represented by self._breakaway.current.value
            self.plot("Breakaway", "breakaway", self._breakaway.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Breakaway` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class BreakawayAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._breakaway = self.candlestick_patterns.breakaway(self._symbol)

        indicator_history = self.indicator_history(self._breakaway, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._breakaway, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._breakaway, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Closing Marubozu

Introduction

Create a new Closing Marubozu candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ClosingMarubozu Indicator

To create an automatic indicator for `ClosingMarubozu` , call the `closing_marubozu` helper method from the `QCAlgorithm` class. The `closing_marubozu` method creates a `ClosingMarubozu` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ClosingMarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._closing_marubozu = self.candlestick_patterns.closing_marubozu(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._closing_marubozu.is_ready:
            # The current value of self._closing_marubozu is represented by self._closing_marubozu.current.value
            self.plot("ClosingMarubozu", "closing_marubozu", self._closing_marubozu.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ClosingMarubozu` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ClosingMarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._closing_marubozu = ClosingMarubozu()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._closing_marubozu.update(bar)

        if self._closing_marubozu.is_ready:
            # The current value of self._closing_marubozu is represented by self._closing_marubozu.current.value
            self.plot("ClosingMarubozu", "closing_marubozu", self._closing_marubozu.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ClosingMarubozu` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ClosingMarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._closing_marubozu = self.candlestick_patterns.closing_marubozu(self._symbol)

        indicator_history = self.indicator_history(self._closing_marubozu, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._closing_marubozu, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._closing_marubozu, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Concealed Baby Swallow

Introduction

Create a new Concealed Baby Swallow candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ConcealedBabySwallow Indicator

To create an automatic indicator for `ConcealedBabySwallow` , call the `concealed_baby_swallow` helper method from the `QCAlgorithm` class. The `concealed_baby_swallow` method creates a `ConcealedBabySwallow` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ConcealedBabySwallowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._concealed_baby_swallow = self.candlestick_patterns.concealed_baby_swallow(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._concealed_baby_swallow.is_ready:
            # The current value of self._concealed_baby_swallow is represented by self._concealed_baby_swallow.current.value
            self.plot("ConcealedBabySwallow", "concealed_baby_swallow", self._concealed_baby_swallow.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ConcealedBabySwallow` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ConcealedBabySwallowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._concealed_baby_swallow = ConcealedBabySwallow()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._concealed_baby_swallow.update(bar)

        if self._concealed_baby_swallow.is_ready:
            # The current value of self._concealed_baby_swallow is represented by self._concealed_baby_swallow.current.value
            self.plot("ConcealedBabySwallow", "concealed_baby_swallow", self._concealed_baby_swallow.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ConcealedBabySwallow` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ConcealedBabySwallowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._concealed_baby_swallow = self.candlestick_patterns.concealed_baby_swallow(self._symbol)

        indicator_history = self.indicator_history(self._concealed_baby_swallow, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._concealed_baby_swallow, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._concealed_baby_swallow, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Counterattack

Introduction

Create a new Counterattack candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Counterattack Indicator

To create an automatic indicator for `Counterattack` , call the `counterattack` helper method from the `QCAlgorithm` class. The `counterattack` method creates a `Counterattack` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class CounterattackAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._counterattack = self.candlestick_patterns.counterattack(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._counterattack.is_ready:
            # The current value of self._counterattack is represented by self._counterattack.current.value
            self.plot("Counterattack", "counterattack", self._counterattack.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Counterattack` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class CounterattackAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._counterattack = Counterattack()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._counterattack.update(bar)

        if self._counterattack.is_ready:
            # The current value of self._counterattack is represented by self._counterattack.current.value
            self.plot("Counterattack", "counterattack", self._counterattack.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Counterattack` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class CounterattackAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._counterattack = self.candlestick_patterns.counterattack(self._symbol)

        indicator_history = self.indicator_history(self._counterattack, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._counterattack, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._counterattack, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Dark Cloud Cover

Introduction

Create a new Dark Cloud Cover candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using DarkCloudCover Indicator

To create an automatic indicator for `DarkCloudCover` , call the `dark_cloud_cover` helper method from the `QCAlgorithm` class. The `dark_cloud_cover` method creates a `DarkCloudCover` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class DarkCloudCoverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dark_cloud_cover = self.candlestick_patterns.dark_cloud_cover(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._dark_cloud_cover.is_ready:
            # The current value of self._dark_cloud_cover is represented by self._dark_cloud_cover.current.value
            self.plot("DarkCloudCover", "dark_cloud_cover", self._dark_cloud_cover.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `DarkCloudCover` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class DarkCloudCoverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dark_cloud_cover = DarkCloudCover()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._dark_cloud_cover.update(bar)

        if self._dark_cloud_cover.is_ready:
            # The current value of self._dark_cloud_cover is represented by self._dark_cloud_cover.current.value
            self.plot("DarkCloudCover", "dark_cloud_cover", self._dark_cloud_cover.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `DarkCloudCover` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DarkCloudCoverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dark_cloud_cover = self.candlestick_patterns.dark_cloud_cover(self._symbol)

        indicator_history = self.indicator_history(self._dark_cloud_cover, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dark_cloud_cover, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dark_cloud_cover, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Doji

Introduction

Create a new Doji candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Doji Indicator

To create an automatic indicator for `Doji` , call the `doji` helper method from the `QCAlgorithm` class. The `doji` method creates a `Doji` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class DojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji = self.candlestick_patterns.doji(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._doji.is_ready:
            # The current value of self._doji is represented by self._doji.current.value
            self.plot("Doji", "doji", self._doji.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Doji` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class DojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji = Doji()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._doji.update(bar)

        if self._doji.is_ready:
            # The current value of self._doji is represented by self._doji.current.value
            self.plot("Doji", "doji", self._doji.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Doji` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji = self.candlestick_patterns.doji(self._symbol)

        indicator_history = self.indicator_history(self._doji, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._doji, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._doji, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Doji Star

Introduction

Create a new Doji Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using DojiStar Indicator

To create an automatic indicator for `DojiStar` , call the `doji_star` helper method from the `QCAlgorithm` class. The `doji_star` method creates a `DojiStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class DojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji_star = self.candlestick_patterns.doji_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._doji_star.is_ready:
            # The current value of self._doji_star is represented by self._doji_star.current.value
            self.plot("DojiStar", "doji_star", self._doji_star.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `DojiStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class DojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji_star = DojiStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._doji_star.update(bar)

        if self._doji_star.is_ready:
            # The current value of self._doji_star is represented by self._doji_star.current.value
            self.plot("DojiStar", "doji_star", self._doji_star.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `DojiStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doji_star = self.candlestick_patterns.doji_star(self._symbol)

        indicator_history = self.indicator_history(self._doji_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._doji_star, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._doji_star, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Dragonfly Doji

Introduction

Create a new Dragonfly Doji candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using DragonflyDoji Indicator

To create an automatic indicator for `DragonflyDoji` , call the `dragonfly_doji` helper method from the `QCAlgorithm` class. The `dragonfly_doji` method creates a `DragonflyDoji` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class DragonflyDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dragonfly_doji = self.candlestick_patterns.dragonfly_doji(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._dragonfly_doji.is_ready:
            # The current value of self._dragonfly_doji is represented by self._dragonfly_doji.current.value
            self.plot("DragonflyDoji", "dragonfly_doji", self._dragonfly_doji.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `DragonflyDoji` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class DragonflyDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dragonfly_doji = DragonflyDoji()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._dragonfly_doji.update(bar)

        if self._dragonfly_doji.is_ready:
            # The current value of self._dragonfly_doji is represented by self._dragonfly_doji.current.value
            self.plot("DragonflyDoji", "dragonfly_doji", self._dragonfly_doji.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `DragonflyDoji` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DragonflyDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dragonfly_doji = self.candlestick_patterns.dragonfly_doji(self._symbol)

        indicator_history = self.indicator_history(self._dragonfly_doji, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dragonfly_doji, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dragonfly_doji, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Engulfing

Introduction

Create a new Engulfing candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Engulfing Indicator

To create an automatic indicator for `Engulfing` , call the `engulfing` helper method from the `QCAlgorithm` class. The `engulfing` method creates a `Engulfing` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class EngulfingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._engulfing = self.candlestick_patterns.engulfing(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._engulfing.is_ready:
            # The current value of self._engulfing is represented by self._engulfing.current.value
            self.plot("Engulfing", "engulfing", self._engulfing.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Engulfing` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class EngulfingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._engulfing = Engulfing()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._engulfing.update(bar)

        if self._engulfing.is_ready:
            # The current value of self._engulfing is represented by self._engulfing.current.value
            self.plot("Engulfing", "engulfing", self._engulfing.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Engulfing` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class EngulfingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._engulfing = self.candlestick_patterns.engulfing(self._symbol)

        indicator_history = self.indicator_history(self._engulfing, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._engulfing, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._engulfing, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Evening Doji Star

Introduction

Create a new Evening Doji Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using EveningDojiStar Indicator

To create an automatic indicator for `EveningDojiStar` , call the `evening_doji_star` helper method from the `QCAlgorithm` class. The `evening_doji_star` method creates a `EveningDojiStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class EveningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_doji_star = self.candlestick_patterns.evening_doji_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._evening_doji_star.is_ready:
            # The current value of self._evening_doji_star is represented by self._evening_doji_star.current.value
            self.plot("EveningDojiStar", "evening_doji_star", self._evening_doji_star.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `EveningDojiStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class EveningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_doji_star = EveningDojiStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._evening_doji_star.update(bar)

        if self._evening_doji_star.is_ready:
            # The current value of self._evening_doji_star is represented by self._evening_doji_star.current.value
            self.plot("EveningDojiStar", "evening_doji_star", self._evening_doji_star.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `EveningDojiStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class EveningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_doji_star = self.candlestick_patterns.evening_doji_star(self._symbol)

        indicator_history = self.indicator_history(self._evening_doji_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._evening_doji_star, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._evening_doji_star, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Evening Star

Introduction

Create a new Evening Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using EveningStar Indicator

To create an automatic indicator for `EveningStar` , call the `evening_star` helper method from the `QCAlgorithm` class. The `evening_star` method creates a `EveningStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class EveningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_star = self.candlestick_patterns.evening_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._evening_star.is_ready:
            # The current value of self._evening_star is represented by self._evening_star.current.value
            self.plot("EveningStar", "evening_star", self._evening_star.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `EveningStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class EveningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_star = EveningStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._evening_star.update(bar)

        if self._evening_star.is_ready:
            # The current value of self._evening_star is represented by self._evening_star.current.value
            self.plot("EveningStar", "evening_star", self._evening_star.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `EveningStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class EveningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._evening_star = self.candlestick_patterns.evening_star(self._symbol)

        indicator_history = self.indicator_history(self._evening_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._evening_star, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._evening_star, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Gap Side By Side White

Introduction

Create a new Gap Side By Side White candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using GapSideBySideWhite Indicator

To create an automatic indicator for `GapSideBySideWhite` , call the `gap_side_by_side_white` helper method from the `QCAlgorithm` class. The `gap_side_by_side_white` method creates a `GapSideBySideWhite` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class GapSideBySideWhiteAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gap_side_by_side_white = self.candlestick_patterns.gap_side_by_side_white(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._gap_side_by_side_white.is_ready:
            # The current value of self._gap_side_by_side_white is represented by self._gap_side_by_side_white.current.value
            self.plot("GapSideBySideWhite", "gap_side_by_side_white", self._gap_side_by_side_white.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `GapSideBySideWhite` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class GapSideBySideWhiteAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gap_side_by_side_white = GapSideBySideWhite()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._gap_side_by_side_white.update(bar)

        if self._gap_side_by_side_white.is_ready:
            # The current value of self._gap_side_by_side_white is represented by self._gap_side_by_side_white.current.value
            self.plot("GapSideBySideWhite", "gap_side_by_side_white", self._gap_side_by_side_white.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `GapSideBySideWhite` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class GapSideBySideWhiteAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gap_side_by_side_white = self.candlestick_patterns.gap_side_by_side_white(self._symbol)

        indicator_history = self.indicator_history(self._gap_side_by_side_white, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._gap_side_by_side_white, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._gap_side_by_side_white, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Gravestone Doji

Introduction

Create a new Gravestone Doji candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using GravestoneDoji Indicator

To create an automatic indicator for `GravestoneDoji` , call the `gravestone_doji` helper method from the `QCAlgorithm` class. The `gravestone_doji` method creates a `GravestoneDoji` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class GravestoneDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gravestone_doji = self.candlestick_patterns.gravestone_doji(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._gravestone_doji.is_ready:
            # The current value of self._gravestone_doji is represented by self._gravestone_doji.current.value
            self.plot("GravestoneDoji", "gravestone_doji", self._gravestone_doji.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `GravestoneDoji` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class GravestoneDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gravestone_doji = GravestoneDoji()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._gravestone_doji.update(bar)

        if self._gravestone_doji.is_ready:
            # The current value of self._gravestone_doji is represented by self._gravestone_doji.current.value
            self.plot("GravestoneDoji", "gravestone_doji", self._gravestone_doji.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `GravestoneDoji` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class GravestoneDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._gravestone_doji = self.candlestick_patterns.gravestone_doji(self._symbol)

        indicator_history = self.indicator_history(self._gravestone_doji, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._gravestone_doji, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._gravestone_doji, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Hammer

Introduction

Create a new Hammer candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Hammer Indicator

To create an automatic indicator for `Hammer` , call the `hammer` helper method from the `QCAlgorithm` class. The `hammer` method creates a `Hammer` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hammer = self.candlestick_patterns.hammer(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._hammer.is_ready:
            # The current value of self._hammer is represented by self._hammer.current.value
            self.plot("Hammer", "hammer", self._hammer.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Hammer` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hammer = Hammer()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hammer.update(bar)

        if self._hammer.is_ready:
            # The current value of self._hammer is represented by self._hammer.current.value
            self.plot("Hammer", "hammer", self._hammer.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Hammer` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hammer = self.candlestick_patterns.hammer(self._symbol)

        indicator_history = self.indicator_history(self._hammer, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._hammer, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._hammer, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Hanging Man

Introduction

Create a new Hanging Man candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using HangingMan Indicator

To create an automatic indicator for `HangingMan` , call the `hanging_man` helper method from the `QCAlgorithm` class. The `hanging_man` method creates a `HangingMan` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HangingManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hanging_man = self.candlestick_patterns.hanging_man(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._hanging_man.is_ready:
            # The current value of self._hanging_man is represented by self._hanging_man.current.value
            self.plot("HangingMan", "hanging_man", self._hanging_man.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `HangingMan` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HangingManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hanging_man = HangingMan()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hanging_man.update(bar)

        if self._hanging_man.is_ready:
            # The current value of self._hanging_man is represented by self._hanging_man.current.value
            self.plot("HangingMan", "hanging_man", self._hanging_man.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `HangingMan` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HangingManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hanging_man = self.candlestick_patterns.hanging_man(self._symbol)

        indicator_history = self.indicator_history(self._hanging_man, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._hanging_man, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._hanging_man, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Harami

Introduction

Create a new Harami candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Harami Indicator

To create an automatic indicator for `Harami` , call the `harami` helper method from the `QCAlgorithm` class. The `harami` method creates a `Harami` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HaramiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami = self.candlestick_patterns.harami(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._harami.is_ready:
            # The current value of self._harami is represented by self._harami.current.value
            self.plot("Harami", "harami", self._harami.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Harami` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HaramiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami = Harami()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._harami.update(bar)

        if self._harami.is_ready:
            # The current value of self._harami is represented by self._harami.current.value
            self.plot("Harami", "harami", self._harami.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Harami` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HaramiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami = self.candlestick_patterns.harami(self._symbol)

        indicator_history = self.indicator_history(self._harami, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._harami, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._harami, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Harami Cross

Introduction

Create a new Harami Cross candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using HaramiCross Indicator

To create an automatic indicator for `HaramiCross` , call the `harami_cross` helper method from the `QCAlgorithm` class. The `harami_cross` method creates a `HaramiCross` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HaramiCrossAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami_cross = self.candlestick_patterns.harami_cross(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._harami_cross.is_ready:
            # The current value of self._harami_cross is represented by self._harami_cross.current.value
            self.plot("HaramiCross", "harami_cross", self._harami_cross.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `HaramiCross` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HaramiCrossAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami_cross = HaramiCross()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._harami_cross.update(bar)

        if self._harami_cross.is_ready:
            # The current value of self._harami_cross is represented by self._harami_cross.current.value
            self.plot("HaramiCross", "harami_cross", self._harami_cross.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `HaramiCross` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HaramiCrossAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._harami_cross = self.candlestick_patterns.harami_cross(self._symbol)

        indicator_history = self.indicator_history(self._harami_cross, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._harami_cross, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._harami_cross, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

High Wave Candle

Introduction

Create a new High Wave Candle candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using HighWaveCandle Indicator

To create an automatic indicator for `HighWaveCandle` , call the `high_wave_candle` helper method from the `QCAlgorithm` class. The `high_wave_candle` method creates a `HighWaveCandle` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HighWaveCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._high_wave_candle = self.candlestick_patterns.high_wave_candle(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._high_wave_candle.is_ready:
            # The current value of self._high_wave_candle is represented by self._high_wave_candle.current.value
            self.plot("HighWaveCandle", "high_wave_candle", self._high_wave_candle.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `HighWaveCandle` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HighWaveCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._high_wave_candle = HighWaveCandle()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._high_wave_candle.update(bar)

        if self._high_wave_candle.is_ready:
            # The current value of self._high_wave_candle is represented by self._high_wave_candle.current.value
            self.plot("HighWaveCandle", "high_wave_candle", self._high_wave_candle.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `HighWaveCandle` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HighWaveCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._high_wave_candle = self.candlestick_patterns.high_wave_candle(self._symbol)

        indicator_history = self.indicator_history(self._high_wave_candle, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._high_wave_candle, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._high_wave_candle, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Hikkake

Introduction

Create a new Hikkake candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Hikkake Indicator

To create an automatic indicator for `Hikkake` , call the `hikkake` helper method from the `QCAlgorithm` class. The `hikkake` method creates a `Hikkake` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HikkakeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake = self.candlestick_patterns.hikkake(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._hikkake.is_ready:
            # The current value of self._hikkake is represented by self._hikkake.current.value
            self.plot("Hikkake", "hikkake", self._hikkake.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Hikkake` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HikkakeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake = Hikkake()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hikkake.update(bar)

        if self._hikkake.is_ready:
            # The current value of self._hikkake is represented by self._hikkake.current.value
            self.plot("Hikkake", "hikkake", self._hikkake.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Hikkake` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HikkakeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake = self.candlestick_patterns.hikkake(self._symbol)

        indicator_history = self.indicator_history(self._hikkake, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._hikkake, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._hikkake, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Hikkake Modified

Introduction

Create a new Hikkake Modified candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using HikkakeModified Indicator

To create an automatic indicator for `HikkakeModified` , call the `hikkake_modified` helper method from the `QCAlgorithm` class. The `hikkake_modified` method creates a `HikkakeModified` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HikkakeModifiedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake_modified = self.candlestick_patterns.hikkake_modified(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._hikkake_modified.is_ready:
            # The current value of self._hikkake_modified is represented by self._hikkake_modified.current.value
            self.plot("HikkakeModified", "hikkake_modified", self._hikkake_modified.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `HikkakeModified` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HikkakeModifiedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake_modified = HikkakeModified()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hikkake_modified.update(bar)

        if self._hikkake_modified.is_ready:
            # The current value of self._hikkake_modified is represented by self._hikkake_modified.current.value
            self.plot("HikkakeModified", "hikkake_modified", self._hikkake_modified.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `HikkakeModified` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HikkakeModifiedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hikkake_modified = self.candlestick_patterns.hikkake_modified(self._symbol)

        indicator_history = self.indicator_history(self._hikkake_modified, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._hikkake_modified, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._hikkake_modified, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Homing Pigeon

Introduction

Create a new Homing Pigeon candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using HomingPigeon Indicator

To create an automatic indicator for `HomingPigeon` , call the `homing_pigeon` helper method from the `QCAlgorithm` class. The `homing_pigeon` method creates a `HomingPigeon` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HomingPigeonAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._homing_pigeon = self.candlestick_patterns.homing_pigeon(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._homing_pigeon.is_ready:
            # The current value of self._homing_pigeon is represented by self._homing_pigeon.current.value
            self.plot("HomingPigeon", "homing_pigeon", self._homing_pigeon.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `HomingPigeon` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HomingPigeonAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._homing_pigeon = HomingPigeon()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._homing_pigeon.update(bar)

        if self._homing_pigeon.is_ready:
            # The current value of self._homing_pigeon is represented by self._homing_pigeon.current.value
            self.plot("HomingPigeon", "homing_pigeon", self._homing_pigeon.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `HomingPigeon` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HomingPigeonAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._homing_pigeon = self.candlestick_patterns.homing_pigeon(self._symbol)

        indicator_history = self.indicator_history(self._homing_pigeon, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._homing_pigeon, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._homing_pigeon, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Identical Three Crows

Introduction

Create a new Identical Three Crows candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using IdenticalThreeCrows Indicator

To create an automatic indicator for `IdenticalThreeCrows` , call the `identical_three_crows` helper method from the `QCAlgorithm` class. The `identical_three_crows` method creates a `IdenticalThreeCrows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class IdenticalThreeCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identical_three_crows = self.candlestick_patterns.identical_three_crows(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._identical_three_crows.is_ready:
            # The current value of self._identical_three_crows is represented by self._identical_three_crows.current.value
            self.plot("IdenticalThreeCrows", "identical_three_crows", self._identical_three_crows.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `IdenticalThreeCrows` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class IdenticalThreeCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identical_three_crows = IdenticalThreeCrows()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._identical_three_crows.update(bar)

        if self._identical_three_crows.is_ready:
            # The current value of self._identical_three_crows is represented by self._identical_three_crows.current.value
            self.plot("IdenticalThreeCrows", "identical_three_crows", self._identical_three_crows.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `IdenticalThreeCrows` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class IdenticalThreeCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identical_three_crows = self.candlestick_patterns.identical_three_crows(self._symbol)

        indicator_history = self.indicator_history(self._identical_three_crows, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._identical_three_crows, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._identical_three_crows, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

In Neck

Introduction

Create a new In Neck candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using InNeck Indicator

To create an automatic indicator for `InNeck` , call the `in_neck` helper method from the `QCAlgorithm` class. The `in_neck` method creates a `InNeck` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class InNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._in_neck = self.candlestick_patterns.in_neck(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._in_neck.is_ready:
            # The current value of self._in_neck is represented by self._in_neck.current.value
            self.plot("InNeck", "in_neck", self._in_neck.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `InNeck` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class InNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._in_neck = InNeck()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._in_neck.update(bar)

        if self._in_neck.is_ready:
            # The current value of self._in_neck is represented by self._in_neck.current.value
            self.plot("InNeck", "in_neck", self._in_neck.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `InNeck` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class InNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._in_neck = self.candlestick_patterns.in_neck(self._symbol)

        indicator_history = self.indicator_history(self._in_neck, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._in_neck, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._in_neck, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Inverted Hammer

Introduction

Create a new Inverted Hammer candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using InvertedHammer Indicator

To create an automatic indicator for `InvertedHammer` , call the `inverted_hammer` helper method from the `QCAlgorithm` class. The `inverted_hammer` method creates a `InvertedHammer` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class InvertedHammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._inverted_hammer = self.candlestick_patterns.inverted_hammer(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._inverted_hammer.is_ready:
            # The current value of self._inverted_hammer is represented by self._inverted_hammer.current.value
            self.plot("InvertedHammer", "inverted_hammer", self._inverted_hammer.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `InvertedHammer` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class InvertedHammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._inverted_hammer = InvertedHammer()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._inverted_hammer.update(bar)

        if self._inverted_hammer.is_ready:
            # The current value of self._inverted_hammer is represented by self._inverted_hammer.current.value
            self.plot("InvertedHammer", "inverted_hammer", self._inverted_hammer.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `InvertedHammer` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class InvertedHammerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._inverted_hammer = self.candlestick_patterns.inverted_hammer(self._symbol)

        indicator_history = self.indicator_history(self._inverted_hammer, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._inverted_hammer, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._inverted_hammer, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Kicking

Introduction

Create a new Kicking candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Kicking Indicator

To create an automatic indicator for `Kicking` , call the `kicking` helper method from the `QCAlgorithm` class. The `kicking` method creates a `Kicking` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class KickingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking = self.candlestick_patterns.kicking(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._kicking.is_ready:
            # The current value of self._kicking is represented by self._kicking.current.value
            self.plot("Kicking", "kicking", self._kicking.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Kicking` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class KickingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking = Kicking()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._kicking.update(bar)

        if self._kicking.is_ready:
            # The current value of self._kicking is represented by self._kicking.current.value
            self.plot("Kicking", "kicking", self._kicking.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Kicking` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class KickingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking = self.candlestick_patterns.kicking(self._symbol)

        indicator_history = self.indicator_history(self._kicking, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kicking, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kicking, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Kicking By Length

Introduction

Create a new Kicking By Length candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using KickingByLength Indicator

To create an automatic indicator for `KickingByLength` , call the `kicking_by_length` helper method from the `QCAlgorithm` class. The `kicking_by_length` method creates a `KickingByLength` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KickingByLengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking_by_length = self.candlestick_patterns.kicking_by_length(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._kicking_by_length.is_ready:
            # The current value of self._kicking_by_length is represented by self._kicking_by_length.current.value
            self.plot("KickingByLength", "kicking_by_length", self._kicking_by_length.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `KickingByLength` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class KickingByLengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking_by_length = KickingByLength()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._kicking_by_length.update(bar)

        if self._kicking_by_length.is_ready:
            # The current value of self._kicking_by_length is represented by self._kicking_by_length.current.value
            self.plot("KickingByLength", "kicking_by_length", self._kicking_by_length.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `KickingByLength` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class KickingByLengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kicking_by_length = self.candlestick_patterns.kicking_by_length(self._symbol)

        indicator_history = self.indicator_history(self._kicking_by_length, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kicking_by_length, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kicking_by_length, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Ladder Bottom

Introduction

Create a new Ladder Bottom candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using LadderBottom Indicator

To create an automatic indicator for `LadderBottom` , call the `ladder_bottom` helper method from the `QCAlgorithm` class. The `ladder_bottom` method creates a `LadderBottom` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class LadderBottomAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ladder_bottom = self.candlestick_patterns.ladder_bottom(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._ladder_bottom.is_ready:
            # The current value of self._ladder_bottom is represented by self._ladder_bottom.current.value
            self.plot("LadderBottom", "ladder_bottom", self._ladder_bottom.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `LadderBottom` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class LadderBottomAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ladder_bottom = LadderBottom()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._ladder_bottom.update(bar)

        if self._ladder_bottom.is_ready:
            # The current value of self._ladder_bottom is represented by self._ladder_bottom.current.value
            self.plot("LadderBottom", "ladder_bottom", self._ladder_bottom.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `LadderBottom` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class LadderBottomAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ladder_bottom = self.candlestick_patterns.ladder_bottom(self._symbol)

        indicator_history = self.indicator_history(self._ladder_bottom, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ladder_bottom, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ladder_bottom, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Long Legged Doji

Introduction

Create a new Long Legged Doji candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using LongLeggedDoji Indicator

To create an automatic indicator for `LongLeggedDoji` , call the `long_legged_doji` helper method from the `QCAlgorithm` class. The `long_legged_doji` method creates a `LongLeggedDoji` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class LongLeggedDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_legged_doji = self.candlestick_patterns.long_legged_doji(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._long_legged_doji.is_ready:
            # The current value of self._long_legged_doji is represented by self._long_legged_doji.current.value
            self.plot("LongLeggedDoji", "long_legged_doji", self._long_legged_doji.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `LongLeggedDoji` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class LongLeggedDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_legged_doji = LongLeggedDoji()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._long_legged_doji.update(bar)

        if self._long_legged_doji.is_ready:
            # The current value of self._long_legged_doji is represented by self._long_legged_doji.current.value
            self.plot("LongLeggedDoji", "long_legged_doji", self._long_legged_doji.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `LongLeggedDoji` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class LongLeggedDojiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_legged_doji = self.candlestick_patterns.long_legged_doji(self._symbol)

        indicator_history = self.indicator_history(self._long_legged_doji, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._long_legged_doji, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._long_legged_doji, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Long Line Candle

Introduction

Create a new Long Line Candle candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using LongLineCandle Indicator

To create an automatic indicator for `LongLineCandle` , call the `long_line_candle` helper method from the `QCAlgorithm` class. The `long_line_candle` method creates a `LongLineCandle` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class LongLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_line_candle = self.candlestick_patterns.long_line_candle(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._long_line_candle.is_ready:
            # The current value of self._long_line_candle is represented by self._long_line_candle.current.value
            self.plot("LongLineCandle", "long_line_candle", self._long_line_candle.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `LongLineCandle` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class LongLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_line_candle = LongLineCandle()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._long_line_candle.update(bar)

        if self._long_line_candle.is_ready:
            # The current value of self._long_line_candle is represented by self._long_line_candle.current.value
            self.plot("LongLineCandle", "long_line_candle", self._long_line_candle.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `LongLineCandle` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class LongLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._long_line_candle = self.candlestick_patterns.long_line_candle(self._symbol)

        indicator_history = self.indicator_history(self._long_line_candle, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._long_line_candle, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._long_line_candle, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Marubozu

Introduction

Create a new Marubozu candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Marubozu Indicator

To create an automatic indicator for `Marubozu` , call the `marubozu` helper method from the `QCAlgorithm` class. The `marubozu` method creates a `Marubozu` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._marubozu = self.candlestick_patterns.marubozu(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._marubozu.is_ready:
            # The current value of self._marubozu is represented by self._marubozu.current.value
            self.plot("Marubozu", "marubozu", self._marubozu.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Marubozu` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._marubozu = Marubozu()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._marubozu.update(bar)

        if self._marubozu.is_ready:
            # The current value of self._marubozu is represented by self._marubozu.current.value
            self.plot("Marubozu", "marubozu", self._marubozu.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Marubozu` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MarubozuAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._marubozu = self.candlestick_patterns.marubozu(self._symbol)

        indicator_history = self.indicator_history(self._marubozu, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._marubozu, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._marubozu, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Mat Hold

Introduction

Create a new Mat Hold candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using MatHold Indicator

To create an automatic indicator for `MatHold` , call the `mat_hold` helper method from the `QCAlgorithm` class. The `mat_hold` method creates a `MatHold` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MatHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mat_hold = self.candlestick_patterns.mat_hold(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._mat_hold.is_ready:
            # The current value of self._mat_hold is represented by self._mat_hold.current.value
            self.plot("MatHold", "mat_hold", self._mat_hold.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `MatHold` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MatHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mat_hold = MatHold()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._mat_hold.update(bar)

        if self._mat_hold.is_ready:
            # The current value of self._mat_hold is represented by self._mat_hold.current.value
            self.plot("MatHold", "mat_hold", self._mat_hold.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `MatHold` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MatHoldAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mat_hold = self.candlestick_patterns.mat_hold(self._symbol)

        indicator_history = self.indicator_history(self._mat_hold, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mat_hold, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mat_hold, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Matching Low

Introduction

Create a new Matching Low candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using MatchingLow Indicator

To create an automatic indicator for `MatchingLow` , call the `matching_low` helper method from the `QCAlgorithm` class. The `matching_low` method creates a `MatchingLow` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MatchingLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._matching_low = self.candlestick_patterns.matching_low(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._matching_low.is_ready:
            # The current value of self._matching_low is represented by self._matching_low.current.value
            self.plot("MatchingLow", "matching_low", self._matching_low.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `MatchingLow` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MatchingLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._matching_low = MatchingLow()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._matching_low.update(bar)

        if self._matching_low.is_ready:
            # The current value of self._matching_low is represented by self._matching_low.current.value
            self.plot("MatchingLow", "matching_low", self._matching_low.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `MatchingLow` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MatchingLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._matching_low = self.candlestick_patterns.matching_low(self._symbol)

        indicator_history = self.indicator_history(self._matching_low, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._matching_low, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._matching_low, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Morning Doji Star

Introduction

Create a new Morning Doji Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using MorningDojiStar Indicator

To create an automatic indicator for `MorningDojiStar` , call the `morning_doji_star` helper method from the `QCAlgorithm` class. The `morning_doji_star` method creates a `MorningDojiStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MorningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_doji_star = self.candlestick_patterns.morning_doji_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._morning_doji_star.is_ready:
            # The current value of self._morning_doji_star is represented by self._morning_doji_star.current.value
            self.plot("MorningDojiStar", "morning_doji_star", self._morning_doji_star.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `MorningDojiStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MorningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_doji_star = MorningDojiStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._morning_doji_star.update(bar)

        if self._morning_doji_star.is_ready:
            # The current value of self._morning_doji_star is represented by self._morning_doji_star.current.value
            self.plot("MorningDojiStar", "morning_doji_star", self._morning_doji_star.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `MorningDojiStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MorningDojiStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_doji_star = self.candlestick_patterns.morning_doji_star(self._symbol)

        indicator_history = self.indicator_history(self._morning_doji_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._morning_doji_star, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._morning_doji_star, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Morning Star

Introduction

Create a new Morning Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using MorningStar Indicator

To create an automatic indicator for `MorningStar` , call the `morning_star` helper method from the `QCAlgorithm` class. The `morning_star` method creates a `MorningStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MorningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_star = self.candlestick_patterns.morning_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._morning_star.is_ready:
            # The current value of self._morning_star is represented by self._morning_star.current.value
            self.plot("MorningStar", "morning_star", self._morning_star.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `MorningStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MorningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_star = MorningStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._morning_star.update(bar)

        if self._morning_star.is_ready:
            # The current value of self._morning_star is represented by self._morning_star.current.value
            self.plot("MorningStar", "morning_star", self._morning_star.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `MorningStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MorningStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._morning_star = self.candlestick_patterns.morning_star(self._symbol)

        indicator_history = self.indicator_history(self._morning_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._morning_star, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._morning_star, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

On Neck

Introduction

Create a new On Neck candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using OnNeck Indicator

To create an automatic indicator for `OnNeck` , call the `on_neck` helper method from the `QCAlgorithm` class. The `on_neck` method creates a `OnNeck` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class OnNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._on_neck = self.candlestick_patterns.on_neck(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._on_neck.is_ready:
            # The current value of self._on_neck is represented by self._on_neck.current.value
            self.plot("OnNeck", "on_neck", self._on_neck.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `OnNeck` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class OnNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._on_neck = OnNeck()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._on_neck.update(bar)

        if self._on_neck.is_ready:
            # The current value of self._on_neck is represented by self._on_neck.current.value
            self.plot("OnNeck", "on_neck", self._on_neck.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `OnNeck` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class OnNeckAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._on_neck = self.candlestick_patterns.on_neck(self._symbol)

        indicator_history = self.indicator_history(self._on_neck, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._on_neck, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._on_neck, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Piercing

Introduction

Create a new Piercing candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Piercing Indicator

To create an automatic indicator for `Piercing` , call the `piercing` helper method from the `QCAlgorithm` class. The `piercing` method creates a `Piercing` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class PiercingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._piercing = self.candlestick_patterns.piercing(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._piercing.is_ready:
            # The current value of self._piercing is represented by self._piercing.current.value
            self.plot("Piercing", "piercing", self._piercing.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Piercing` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class PiercingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._piercing = Piercing()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._piercing.update(bar)

        if self._piercing.is_ready:
            # The current value of self._piercing is represented by self._piercing.current.value
            self.plot("Piercing", "piercing", self._piercing.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Piercing` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class PiercingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._piercing = self.candlestick_patterns.piercing(self._symbol)

        indicator_history = self.indicator_history(self._piercing, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._piercing, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._piercing, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Rickshaw Man

Introduction

Create a new Rickshaw Man candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using RickshawMan Indicator

To create an automatic indicator for `RickshawMan` , call the `rickshaw_man` helper method from the `QCAlgorithm` class. The `rickshaw_man` method creates a `RickshawMan` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RickshawManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rickshaw_man = self.candlestick_patterns.rickshaw_man(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._rickshaw_man.is_ready:
            # The current value of self._rickshaw_man is represented by self._rickshaw_man.current.value
            self.plot("RickshawMan", "rickshaw_man", self._rickshaw_man.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `RickshawMan` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RickshawManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rickshaw_man = RickshawMan()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rickshaw_man.update(bar)

        if self._rickshaw_man.is_ready:
            # The current value of self._rickshaw_man is represented by self._rickshaw_man.current.value
            self.plot("RickshawMan", "rickshaw_man", self._rickshaw_man.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `RickshawMan` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RickshawManAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rickshaw_man = self.candlestick_patterns.rickshaw_man(self._symbol)

        indicator_history = self.indicator_history(self._rickshaw_man, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rickshaw_man, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rickshaw_man, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Rise Fall Three Methods

Introduction

Create a new Rise Fall Three Methods candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using RiseFallThreeMethods Indicator

To create an automatic indicator for `RiseFallThreeMethods` , call the `rise_fall_three_methods` helper method from the `QCAlgorithm` class. The `rise_fall_three_methods` method creates a `RiseFallThreeMethods` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RiseFallThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rise_fall_three_methods = self.candlestick_patterns.rise_fall_three_methods(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._rise_fall_three_methods.is_ready:
            # The current value of self._rise_fall_three_methods is represented by self._rise_fall_three_methods.current.value
            self.plot("RiseFallThreeMethods", "rise_fall_three_methods", self._rise_fall_three_methods.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `RiseFallThreeMethods` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RiseFallThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rise_fall_three_methods = RiseFallThreeMethods()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rise_fall_three_methods.update(bar)

        if self._rise_fall_three_methods.is_ready:
            # The current value of self._rise_fall_three_methods is represented by self._rise_fall_three_methods.current.value
            self.plot("RiseFallThreeMethods", "rise_fall_three_methods", self._rise_fall_three_methods.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `RiseFallThreeMethods` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RiseFallThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rise_fall_three_methods = self.candlestick_patterns.rise_fall_three_methods(self._symbol)

        indicator_history = self.indicator_history(self._rise_fall_three_methods, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rise_fall_three_methods, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rise_fall_three_methods, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Separating Lines

Introduction

Create a new Separating Lines candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using SeparatingLines Indicator

To create an automatic indicator for `SeparatingLines` , call the `separating_lines` helper method from the `QCAlgorithm` class. The `separating_lines` method creates a `SeparatingLines` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SeparatingLinesAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._separating_lines = self.candlestick_patterns.separating_lines(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._separating_lines.is_ready:
            # The current value of self._separating_lines is represented by self._separating_lines.current.value
            self.plot("SeparatingLines", "separating_lines", self._separating_lines.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `SeparatingLines` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SeparatingLinesAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._separating_lines = SeparatingLines()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._separating_lines.update(bar)

        if self._separating_lines.is_ready:
            # The current value of self._separating_lines is represented by self._separating_lines.current.value
            self.plot("SeparatingLines", "separating_lines", self._separating_lines.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `SeparatingLines` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class SeparatingLinesAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._separating_lines = self.candlestick_patterns.separating_lines(self._symbol)

        indicator_history = self.indicator_history(self._separating_lines, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._separating_lines, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._separating_lines, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Shooting Star

Introduction

Create a new Shooting Star candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ShootingStar Indicator

To create an automatic indicator for `ShootingStar` , call the `shooting_star` helper method from the `QCAlgorithm` class. The `shooting_star` method creates a `ShootingStar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ShootingStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._shooting_star = self.candlestick_patterns.shooting_star(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._shooting_star.is_ready:
            # The current value of self._shooting_star is represented by self._shooting_star.current.value
            self.plot("ShootingStar", "shooting_star", self._shooting_star.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ShootingStar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ShootingStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._shooting_star = ShootingStar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._shooting_star.update(bar)

        if self._shooting_star.is_ready:
            # The current value of self._shooting_star is represented by self._shooting_star.current.value
            self.plot("ShootingStar", "shooting_star", self._shooting_star.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ShootingStar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ShootingStarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._shooting_star = self.candlestick_patterns.shooting_star(self._symbol)

        indicator_history = self.indicator_history(self._shooting_star, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._shooting_star, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._shooting_star, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Short Line Candle

Introduction

Create a new Short Line Candle candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ShortLineCandle Indicator

To create an automatic indicator for `ShortLineCandle` , call the `short_line_candle` helper method from the `QCAlgorithm` class. The `short_line_candle` method creates a `ShortLineCandle` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ShortLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._short_line_candle = self.candlestick_patterns.short_line_candle(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._short_line_candle.is_ready:
            # The current value of self._short_line_candle is represented by self._short_line_candle.current.value
            self.plot("ShortLineCandle", "short_line_candle", self._short_line_candle.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ShortLineCandle` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ShortLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._short_line_candle = ShortLineCandle()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._short_line_candle.update(bar)

        if self._short_line_candle.is_ready:
            # The current value of self._short_line_candle is represented by self._short_line_candle.current.value
            self.plot("ShortLineCandle", "short_line_candle", self._short_line_candle.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ShortLineCandle` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ShortLineCandleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._short_line_candle = self.candlestick_patterns.short_line_candle(self._symbol)

        indicator_history = self.indicator_history(self._short_line_candle, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._short_line_candle, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._short_line_candle, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Spinning Top

Introduction

Create a new Spinning Top candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using SpinningTop Indicator

To create an automatic indicator for `SpinningTop` , call the `spinning_top` helper method from the `QCAlgorithm` class. The `spinning_top` method creates a `SpinningTop` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```py
class SpinningTopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._spinning_top = self.candlestick_patterns.spinning_top(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._spinning_top.is_ready:
            # The current value of self._spinning_top is represented by self._spinning_top.current.value
            self.plot("SpinningTop", "spinning_top", self._spinning_top.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `SpinningTop` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```py
class SpinningTopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._spinning_top = SpinningTop()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._spinning_top.update(bar)

        if self._spinning_top.is_ready:
            # The current value of self._spinning_top is represented by self._spinning_top.current.value
            self.plot("SpinningTop", "spinning_top", self._spinning_top.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `SpinningTop` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class SpinningTopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._spinning_top = self.candlestick_patterns.spinning_top(self._symbol)

        indicator_history = self.indicator_history(self._spinning_top, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._spinning_top, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._spinning_top, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Stalled Pattern

Introduction

Create a new Stalled Pattern candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using StalledPattern Indicator

To create an automatic indicator for `StalledPattern` , call the `stalled_pattern` helper method from the `QCAlgorithm` class. The `stalled_pattern` method creates a `StalledPattern` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class StalledPatternAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stalled_pattern = self.candlestick_patterns.stalled_pattern(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._stalled_pattern.is_ready:
            # The current value of self._stalled_pattern is represented by self._stalled_pattern.current.value
            self.plot("StalledPattern", "stalled_pattern", self._stalled_pattern.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `StalledPattern` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class StalledPatternAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stalled_pattern = StalledPattern()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._stalled_pattern.update(bar)

        if self._stalled_pattern.is_ready:
            # The current value of self._stalled_pattern is represented by self._stalled_pattern.current.value
            self.plot("StalledPattern", "stalled_pattern", self._stalled_pattern.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `StalledPattern` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class StalledPatternAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stalled_pattern = self.candlestick_patterns.stalled_pattern(self._symbol)

        indicator_history = self.indicator_history(self._stalled_pattern, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._stalled_pattern, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._stalled_pattern, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Stick Sandwich

Introduction

Create a new Stick Sandwich candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using StickSandwich Indicator

To create an automatic indicator for `StickSandwich` , call the `stick_sandwich` helper method from the `QCAlgorithm` class. The `stick_sandwich` method creates a `StickSandwich` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class StickSandwichAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stick_sandwich = self.candlestick_patterns.stick_sandwich(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._stick_sandwich.is_ready:
            # The current value of self._stick_sandwich is represented by self._stick_sandwich.current.value
            self.plot("StickSandwich", "stick_sandwich", self._stick_sandwich.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `StickSandwich` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class StickSandwichAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stick_sandwich = StickSandwich()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._stick_sandwich.update(bar)

        if self._stick_sandwich.is_ready:
            # The current value of self._stick_sandwich is represented by self._stick_sandwich.current.value
            self.plot("StickSandwich", "stick_sandwich", self._stick_sandwich.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `StickSandwich` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class StickSandwichAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stick_sandwich = self.candlestick_patterns.stick_sandwich(self._symbol)

        indicator_history = self.indicator_history(self._stick_sandwich, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._stick_sandwich, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._stick_sandwich, self._symbol, datetime(2024, 7, 1), datetime(2024,
7, 5), Resolution.MINUTE)
```

Supported Indicators

Takuri

Introduction

Create a new Takuri candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Takuri Indicator

To create an automatic indicator for `Takuri` , call the `takuri` helper method from the `QCAlgorithm` class. The `takuri` method creates a `Takuri` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TakuriAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._takuri = self.candlestick_patterns.takuri(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._takuri.is_ready:
            # The current value of self._takuri is represented by self._takuri.current.value
            self.plot("Takuri", "takuri", self._takuri.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Takuri` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class TakuriAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._takuri = Takuri()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._takuri.update(bar)

        if self._takuri.is_ready:
            # The current value of self._takuri is represented by self._takuri.current.value
            self.plot("Takuri", "takuri", self._takuri.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Takuri` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TakuriAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._takuri = self.candlestick_patterns.takuri(self._symbol)

        indicator_history = self.indicator_history(self._takuri, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._takuri, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._takuri, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Tasuki Gap

Introduction

Create a new Tasuki Gap candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using TasukiGap Indicator

To create an automatic indicator for `TasukiGap` , call the `tasuki_gap` helper method from the `QCAlgorithm` class. The `tasuki_gap` method creates a `TasukiGap` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TasukiGapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tasuki_gap = self.candlestick_patterns.tasuki_gap(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._tasuki_gap.is_ready:
            # The current value of self._tasuki_gap is represented by self._tasuki_gap.current.value
            self.plot("TasukiGap", "tasuki_gap", self._tasuki_gap.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `TasukiGap` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class TasukiGapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tasuki_gap = TasukiGap()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._tasuki_gap.update(bar)

        if self._tasuki_gap.is_ready:
            # The current value of self._tasuki_gap is represented by self._tasuki_gap.current.value
            self.plot("TasukiGap", "tasuki_gap", self._tasuki_gap.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `TasukiGap` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TasukiGapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tasuki_gap = self.candlestick_patterns.tasuki_gap(self._symbol)

        indicator_history = self.indicator_history(self._tasuki_gap, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tasuki_gap, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tasuki_gap, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Three Black Crows

Introduction

Create a new Three Black Crows candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeBlackCrows Indicator

To create an automatic indicator for `ThreeBlackCrows` , call the `three_black_crows` helper method from the `QCAlgorithm` class. The `three_black_crows` method creates a `ThreeBlackCrows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ThreeBlackCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_black_crows = self.candlestick_patterns.three_black_crows(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_black_crows.is_ready:
            # The current value of self._three_black_crows is represented by self._three_black_crows.current.value
            self.plot("ThreeBlackCrows", "three_black_crows", self._three_black_crows.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeBlackCrows` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ThreeBlackCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_black_crows = ThreeBlackCrows()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_black_crows.update(bar)

        if self._three_black_crows.is_ready:
            # The current value of self._three_black_crows is represented by self._three_black_crows.current.value
            self.plot("ThreeBlackCrows", "three_black_crows", self._three_black_crows.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeBlackCrows` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeBlackCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_black_crows = self.candlestick_patterns.three_black_crows(self._symbol)

        indicator_history = self.indicator_history(self._three_black_crows, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_black_crows, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_black_crows, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Three Inside

Introduction

Create a new Three Inside candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeInside Indicator

To create an automatic indicator for `ThreeInside` , call the `three_inside` helper method from the `QCAlgorithm` class. The `three_inside` method creates a `ThreeInside` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ThreeInsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_inside = self.candlestick_patterns.three_inside(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_inside.is_ready:
            # The current value of self._three_inside is represented by self._three_inside.current.value
            self.plot("ThreeInside", "three_inside", self._three_inside.current.value)
```

PY

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeInside` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ThreeInsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_inside = ThreeInside()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_inside.update(bar)

        if self._three_inside.is_ready:
            # The current value of self._three_inside is represented by self._three_inside.current.value
            self.plot("ThreeInside", "three_inside", self._three_inside.current.value)
```

PY

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeInside` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeInsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_inside = self.candlestick_patterns.three_inside(self._symbol)

        indicator_history = self.indicator_history(self._three_inside, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_inside, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_inside, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Three Line Strike

Introduction

Create a new Three Line Strike candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeLineStrike Indicator

To create an automatic indicator for `ThreeLineStrike` , call the `three_line_strike` helper method from the `QCAlgorithm` class. The `three_line_strike` method creates a `ThreeLineStrike` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ThreeLineStrikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_line_strike = self.candlestick_patterns.three_line_strike(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_line_strike.is_ready:
            # The current value of self._three_line_strike is represented by self._three_line_strike.current.value
            self.plot("ThreeLineStrike", "three_line_strike", self._three_line_strike.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeLineStrike` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ThreeLineStrikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_line_strike = ThreeLineStrike()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_line_strike.update(bar)

        if self._three_line_strike.is_ready:
            # The current value of self._three_line_strike is represented by self._three_line_strike.current.value
            self.plot("ThreeLineStrike", "three_line_strike", self._three_line_strike.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeLineStrike` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeLineStrikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_line_strike = self.candlestick_patterns.three_line_strike(self._symbol)

        indicator_history = self.indicator_history(self._three_line_strike, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_line_strike, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_line_strike, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Three Outside

Introduction

Create a new Three Outside candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeOutside Indicator

To create an automatic indicator for `ThreeOutside` , call the `three_outside` helper method from the `QCAlgorithm` class. The `three_outside` method creates a `ThreeOutside` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ThreeOutsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_outside = self.candlestick_patterns.three_outside(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_outside.is_ready:
            # The current value of self._three_outside is represented by self._three_outside.current.value
            self.plot("ThreeOutside", "three_outside", self._three_outside.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeOutside` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ThreeOutsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_outside = ThreeOutside()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_outside.update(bar)

        if self._three_outside.is_ready:
            # The current value of self._three_outside is represented by self._three_outside.current.value
            self.plot("ThreeOutside", "three_outside", self._three_outside.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeOutside` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeOutsideAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_outside = self.candlestick_patterns.three_outside(self._symbol)

        indicator_history = self.indicator_history(self._three_outside, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_outside, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_outside, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)
```

Supported Indicators

Three Stars In South

Introduction

Create a new Three Stars In South candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeStarsInSouth Indicator

To create an automatic indicator for `ThreeStarsInSouth` , call the `three_stars_in_south` helper method from the `QCAlgorithm` class. The `three_stars_in_south` method creates a `ThreeStarsInSouth` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ThreeStarsInSouthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_stars_in_south = self.candlestick_patterns.three_stars_in_south(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_stars_in_south.is_ready:
            # The current value of self._three_stars_in_south is represented by self._three_stars_in_south.current.value
            self.plot("ThreeStarsInSouth", "three_stars_in_south", self._three_stars_in_south.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeStarsInSouth` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ThreeStarsInSouthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_stars_in_south = ThreeStarsInSouth()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_stars_in_south.update(bar)

        if self._three_stars_in_south.is_ready:
            # The current value of self._three_stars_in_south is represented by self._three_stars_in_south.current.value
            self.plot("ThreeStarsInSouth", "three_stars_in_south", self._three_stars_in_south.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeStarsInSouth` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeStarsInSouthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_stars_in_south = self.candlestick_patterns.three_stars_in_south(self._symbol)

        indicator_history = self.indicator_history(self._three_stars_in_south, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_stars_in_south, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_stars_in_south, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Three White Soldiers

Introduction

Create a new Three White Soldiers candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using ThreeWhiteSoldiers Indicator

To create an automatic indicator for `ThreeWhiteSoldiers` , call the `three_white_soldiers` helper method from the `QCAlgorithm` class. The `three_white_soldiers` method creates a `ThreeWhiteSoldiers` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ThreeWhiteSoldiersAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_white_soldiers = self.candlestick_patterns.three_white_soldiers(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._three_white_soldiers.is_ready:
            # The current value of self._three_white_soldiers is represented by self._three_white_soldiers.current.value
            self.plot("ThreeWhiteSoldiers", "three_white_soldiers", self._three_white_soldiers.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `ThreeWhiteSoldiers` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ThreeWhiteSoldiersAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_white_soldiers = ThreeWhiteSoldiers()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._three_white_soldiers.update(bar)

        if self._three_white_soldiers.is_ready:
            # The current value of self._three_white_soldiers is represented by self._three_white_soldiers.current.value
            self.plot("ThreeWhiteSoldiers", "three_white_soldiers", self._three_white_soldiers.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `ThreeWhiteSoldiers` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThreeWhiteSoldiersAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._three_white_soldiers = self.candlestick_patterns.three_white_soldiers(self._symbol)

        indicator_history = self.indicator_history(self._three_white_soldiers, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._three_white_soldiers, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._three_white_soldiers, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Thrusting

Introduction

Create a new Thrusting candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Thrusting Indicator

To create an automatic indicator for `Thrusting` , call the `thrusting` helper method from the `QCAlgorithm` class. The `thrusting` method creates a `Thrusting` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ThrustingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._thrusting = self.candlestick_patterns.thrusting(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._thrusting.is_ready:
            # The current value of self._thrusting is represented by self._thrusting.current.value
            self.plot("Thrusting", "thrusting", self._thrusting.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Thrusting` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ThrustingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._thrusting = Thrusting()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._thrusting.update(bar)

        if self._thrusting.is_ready:
            # The current value of self._thrusting is represented by self._thrusting.current.value
            self.plot("Thrusting", "thrusting", self._thrusting.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Thrusting` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThrustingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._thrusting = self.candlestick_patterns.thrusting(self._symbol)

        indicator_history = self.indicator_history(self._thrusting, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._thrusting, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._thrusting, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Tristar

Introduction

Create a new Tristar candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using Tristar Indicator

To create an automatic indicator for `Tristar` , call the `tristar` helper method from the `QCAlgorithm` class. The `tristar` method creates a `Tristar` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TristarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tristar = self.candlestick_patterns.tristar(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._tristar.is_ready:
            # The current value of self._tristar is represented by self._tristar.current.value
            self.plot("Tristar", "tristar", self._tristar.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `Tristar` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TristarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tristar = Tristar()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._tristar.update(bar)

        if self._tristar.is_ready:
            # The current value of self._tristar is represented by self._tristar.current.value
            self.plot("Tristar", "tristar", self._tristar.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `Tristar` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TristarAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tristar = self.candlestick_patterns.tristar(self._symbol)

        indicator_history = self.indicator_history(self._tristar, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tristar, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tristar, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Two Crows

Introduction

Create a new Two Crows candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using TwoCrows Indicator

To create an automatic indicator for `TwoCrows` , call the `two_crows` helper method from the `QCAlgorithm` class. The `two_crows` method creates a `TwoCrows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._two_crows = self.candlestick_patterns.two_crows(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._two_crows.is_ready:
            # The current value of self._two_crows is represented by self._two_crows.current.value
            self.plot("TwoCrows", "two_crows", self._two_crows.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `TwoCrows` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class TwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._two_crows = TwoCrows()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._two_crows.update(bar)

        if self._two_crows.is_ready:
            # The current value of self._two_crows is represented by self._two_crows.current.value
            self.plot("TwoCrows", "two_crows", self._two_crows.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `TwoCrows` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._two_crows = self.candlestick_patterns.two_crows(self._symbol)

        indicator_history = self.indicator_history(self._two_crows, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._two_crows, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._two_crows, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Unique Three River

Introduction

Create a new Unique Three River candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using UniqueThreeRiver Indicator

To create an automatic indicator for `UniqueThreeRiver` , call the `unique_three_river` helper method from the `QCAlgorithm` class. The `unique_three_river` method creates a `UniqueThreeRiver` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class UniqueThreeRiverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._unique_three_river = self.candlestick_patterns.unique_three_river(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._unique_three_river.is_ready:
            # The current value of self._unique_three_river is represented by self._unique_three_river.current.value
            self.plot("UniqueThreeRiver", "unique_three_river", self._unique_three_river.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `UniqueThreeRiver` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class UniqueThreeRiverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._unique_three_river = UniqueThreeRiver()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._unique_three_river.update(bar)

        if self._unique_three_river.is_ready:
            # The current value of self._unique_three_river is represented by self._unique_three_river.current.value
            self.plot("UniqueThreeRiver", "unique_three_river", self._unique_three_river.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `UniqueThreeRiver` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class UniqueThreeRiverAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._unique_three_river = self.candlestick_patterns.unique_three_river(self._symbol)

        indicator_history = self.indicator_history(self._unique_three_river, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._unique_three_river, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._unique_three_river, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Up Down Gap Three Methods

Introduction

Create a new Up Down Gap Three Methods candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using UpDownGapThreeMethods Indicator

To create an automatic indicator for `UpDownGapThreeMethods` , call the `up_down_gap_three_methods` helper method from the `QCAlgorithm` class. The `up_down_gap_three_methods` method creates a `UpDownGapThreeMethods` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class UpDownGapThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._up_down_gap_three_methods = self.candlestick_patterns.up_down_gap_three_methods(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._up_down_gap_three_methods.is_ready:
            # The current value of self._up_down_gap_three_methods is represented by self._up_down_gap_three_methods.current.value
            self.plot("UpDownGapThreeMethods", "up_down_gap_three_methods", self._up_down_gap_three_methods.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `UpDownGapThreeMethods` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class UpDownGapThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._up_down_gap_three_methods = UpDownGapThreeMethods()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._up_down_gap_three_methods.update(bar)

        if self._up_down_gap_three_methods.is_ready:
            # The current value of self._up_down_gap_three_methods is represented by self._up_down_gap_three_methods.current.value
            self.plot("UpDownGapThreeMethods", "up_down_gap_three_methods", self._up_down_gap_three_methods.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `UpDownGapThreeMethods` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class UpDownGapThreeMethodsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._up_down_gap_three_methods = self.candlestick_patterns.up_down_gap_three_methods(self._symbol)

        indicator_history = self.indicator_history(self._up_down_gap_three_methods, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._up_down_gap_three_methods, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._up_down_gap_three_methods, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Upside Gap Two Crows

Introduction

Create a new Upside Gap Two Crows candlestick pattern to indicate the pattern's presence.

To view the implementation of this candlestick pattern, see the LEAN GitHub repository .

Using UpsideGapTwoCrows Indicator

To create an automatic indicator for `UpsideGapTwoCrows` , call the `upside_gap_two_crows` helper method from the `QCAlgorithm` class. The `upside_gap_two_crows` method creates a `UpsideGapTwoCrows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class UpsideGapTwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._upside_gap_two_crows = self.candlestick_patterns.upside_gap_two_crows(self._symbol)

    def on_data(self, slice: Slice) -> None:
        if self._upside_gap_two_crows.is_ready:
            # The current value of self._upside_gap_two_crows is represented by self._upside_gap_two_crows.current.value
            self.plot("UpsideGapTwoCrows", "upside_gap_two_crows", self._upside_gap_two_crows.current.value)
```

For more information about this method, see the CandlestickPatterns class .

You can manually create a `UpsideGapTwoCrows` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class UpsideGapTwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._upside_gap_two_crows = UpsideGapTwoCrows()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._upside_gap_two_crows.update(bar)

        if self._upside_gap_two_crows.is_ready:
            # The current value of self._upside_gap_two_crows is represented by self._upside_gap_two_crows.current.value
            self.plot("UpsideGapTwoCrows", "upside_gap_two_crows", self._upside_gap_two_crows.current.value)
```

For more information about this indicator, see its reference .

Indicator History

To get the historical data of the `UpsideGapTwoCrows` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class UpsideGapTwoCrowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._upside_gap_two_crows = self.candlestick_patterns.upside_gap_two_crows(self._symbol)

        indicator_history = self.indicator_history(self._upside_gap_two_crows, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._upside_gap_two_crows, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._upside_gap_two_crows, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Absolute Price Oscillator

Introduction

This indicator computes the Absolute Price Oscillator (APO) The Absolute Price Oscillator is calculated using the following formula: APO[i] = FastMA[i] - SlowMA[i]

To view the implementation of this indicator, see the LEAN GitHub repository .

Using APO Indicator

To create an automatic indicator for `AbsolutePriceOscillator` , call the `apo` helper method from the `QCAlgorithm` class. The `apo` method creates a `AbsolutePriceOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AbsolutePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._apo = self.apo(self._symbol, 10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._apo.is_ready:
            # The current value of self._apo is represented by self._apo.current.value
            self.plot("AbsolutePriceOscillator", "apo", self._apo.current.value)
            # Plot all attributes of self._apo
            self.plot("AbsolutePriceOscillator", "fast", self._apo.fast.current.value)
            self.plot("AbsolutePriceOscillator", "slow", self._apo.slow.current.value)
            self.plot("AbsolutePriceOscillator", "signal", self._apo.signal.current.value)
            self.plot("AbsolutePriceOscillator", "histogram", self._apo.histogram.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AbsolutePriceOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AbsolutePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._absolutepriceoscillator = AbsolutePriceOscillator(10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._absolutepriceoscillator.update(bar.end_time, bar.close)

        if self._absolutepriceoscillator.is_ready:
            # The current value of self._absolutepriceoscillator is represented by self._absolutepriceoscillator.current.value
            self.plot("AbsolutePriceOscillator", "absolutepriceoscillator", self._absolutepriceoscillator.current.value)
            # Plot all attributes of self._absolutepriceoscillator
            self.plot("AbsolutePriceOscillator", "fast", self._absolutepriceoscillator.fast.current.value)
            self.plot("AbsolutePriceOscillator", "slow", self._absolutepriceoscillator.slow.current.value)
            self.plot("AbsolutePriceOscillator", "signal", self._absolutepriceoscillator.signal.current.value)
            self.plot("AbsolutePriceOscillator", "histogram", self._absolutepriceoscillator.histogram.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AbsolutePriceOscillator` indicator properties:

Indicator History

To get the historical data of the `AbsolutePriceOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AbsolutePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._apo = self.apo(self._symbol, 10, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._apo, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._apo, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._apo, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        fast = indicator_history_df["fast"]
        slow = indicator_history_df["slow"]
        signal = indicator_history_df["signal"]
        histogram = indicator_history_df["histogram"]
```

Supported Indicators

Acceleration Bands

Introduction

The Acceleration Bands created by Price Headley plots upper and lower envelope bands around a moving average.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ABANDS Indicator

To create an automatic indicator for `AccelerationBands` , call the `abands` helper method from the `QCAlgorithm` class. The `abands` method creates a `AccelerationBands` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class AccelerationBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._abands = self.abands(self._symbol, 10, 4, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._abands.is_ready:
            # The current value of self._abands is represented by self._abands.current.value
            self.plot("AccelerationBands", "abands", self._abands.current.value)
            # Plot all attributes of self._abands
            self.plot("AccelerationBands", "middle_band", self._abands.middle_band.current.value)
            self.plot("AccelerationBands", "upper_band", self._abands.upper_band.current.value)
            self.plot("AccelerationBands", "lower_band", self._abands.lower_band.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AccelerationBands` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

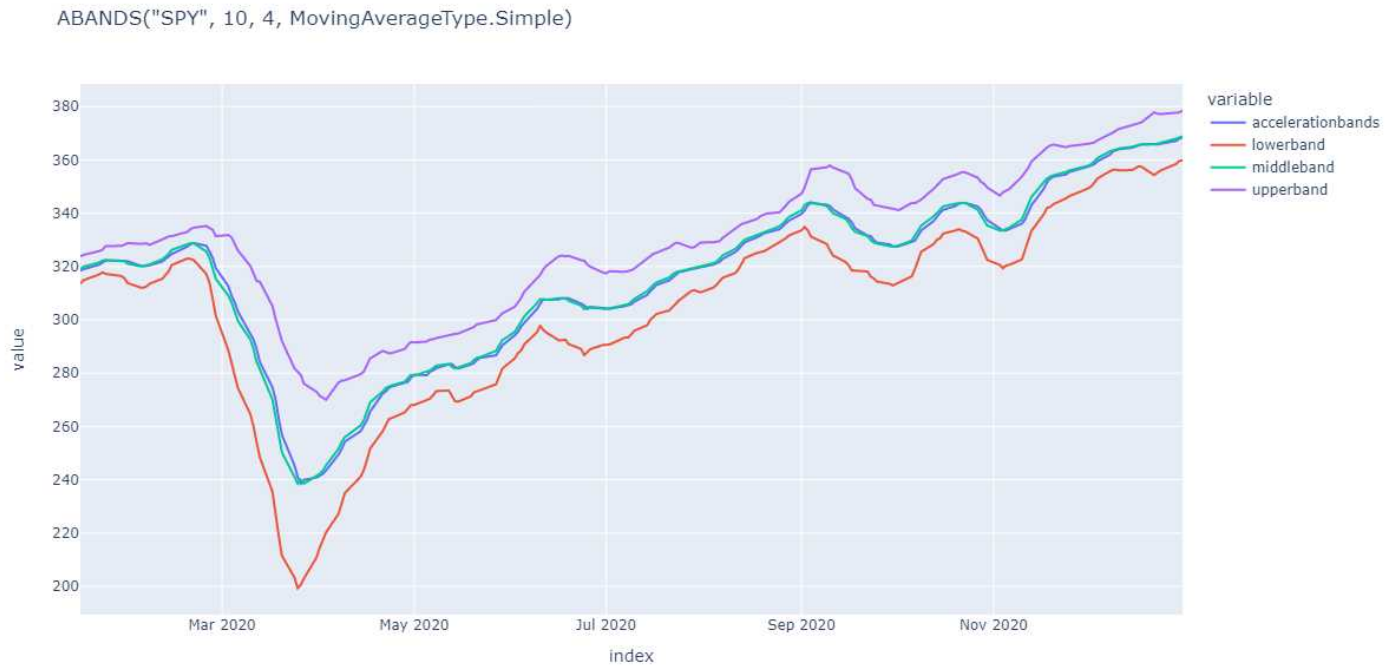
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AccelerationBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._accelerationbands = AccelerationBands(10, 4, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._accelerationbands.update(bar)

        if self._accelerationbands.is_ready:
            # The current value of self._accelerationbands is represented by self._accelerationbands.current.value
            self.plot("AccelerationBands", "accelerationbands", self._accelerationbands.current.value)
            # Plot all attributes of self._accelerationbands
            self.plot("AccelerationBands", "middle_band", self._accelerationbands.middle_band.current.value)
            self.plot("AccelerationBands", "upper_band", self._accelerationbands.upper_band.current.value)
            self.plot("AccelerationBands", "lower_band", self._accelerationbands.lower_band.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AccelerationBands` indicator properties:



Indicator History

To get the historical data of the `AccelerationBands` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```py
class AccelerationBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._abands = self.abands(self._symbol, 10, 4, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._abands, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._abands, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._abands, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        middle_band = indicator_history_df["middleband"]
        upper_band = indicator_history_df["upperband"]
        lower_band = indicator_history_df["lowerband"]
```

Supported Indicators

Accumulation Distribution

Introduction

This indicator computes the Accumulation/Distribution (AD) The Accumulation/Distribution is calculated using the following formula: AD = AD + ((Close - Low) - (High - Close)) / (High - Low) * Volume

To view the implementation of this indicator, see the LEAN GitHub repository .

Using AD Indicator

To create an automatic indicator for `AccumulationDistribution` , call the `ad` helper method from the `QCAlgorithm` class. The `ad` method creates a `AccumulationDistribution` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AccumulationDistributionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ad = self.ad(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._ad.is_ready:
            # The current value of self._ad is represented by self._ad.current.value
            self.plot("AccumulationDistribution", "ad", self._ad.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AccumulationDistribution` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AccumulationDistributionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._accumulationdistribution = AccumulationDistribution()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._accumulationdistribution.update(bar)

        if self._accumulationdistribution.is_ready:
            # The current value of self._accumulationdistribution is represented by self._accumulationdistribution.current.value
            self.plot("AccumulationDistribution", "accumulationdistribution", self._accumulationdistribution.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AccumulationDistribution` indicator properties:

Indicator History

To get the historical data of the `AccumulationDistribution` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AccumulationDistributionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ad = self.ad(self._symbol)

        indicator_history = self.indicator_history(self._ad, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ad, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ad, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Accumulation Distribution Oscillator

Introduction

This indicator computes the Accumulation/Distribution Oscillator (ADOSC) The Accumulation/Distribution Oscillator is calculated using the following formula: ADOSC = EMA(fast,AD) - EMA(slow,AD)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADOSC Indicator

To create an automatic indicator for `AccumulationDistributionOscillator` , call the `adosc` helper method from the `QCAlgorithm` class. The `adosc` method creates a `AccumulationDistributionOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AccumulationDistributionOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adosc = self.adosc(self._symbol, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._adosc.is_ready:
            # The current value of self._adosc is represented by self._adosc.current.value
            self.plot("AccumulationDistributionOscillator", "adosc", self._adosc.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AccumulationDistributionOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

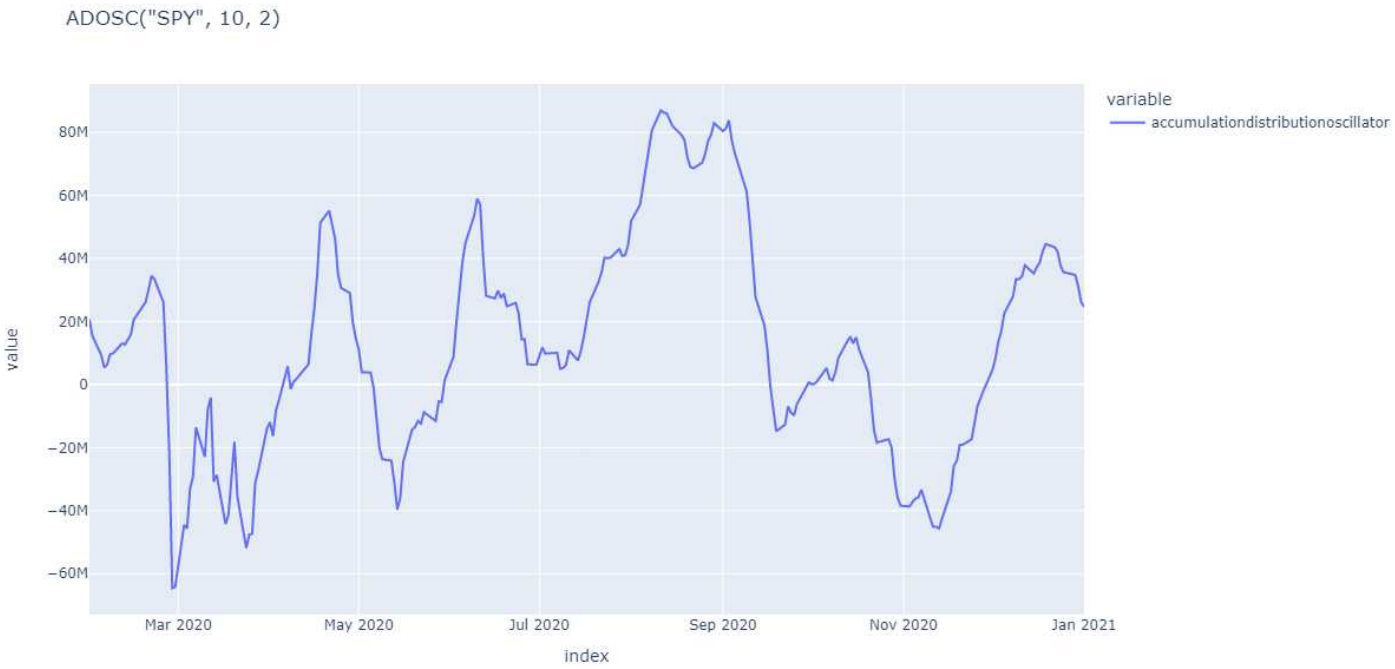
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AccumulationDistributionOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._accumulationdistributionoscillator = AccumulationDistributionOscillator(10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._accumulationdistributionoscillator.update(bar)

        if self._accumulationdistributionoscillator.is_ready:
            # The current value of self._accumulationdistributionoscillator is represented by
self._accumulationdistributionoscillator.current.value
            self.plot("AccumulationDistributionOscillator", "accumulationdistributionoscillator",
self._accumulationdistributionoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AccumulationDistributionOscillator` indicator properties:



Indicator History

To get the historical data of the `AccumulationDistributionOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AccumulationDistributionOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adosc = self.adosc(self._symbol, 10, 20)

        indicator_history = self.indicator_history(self._adosc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._adosc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._adosc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Advance Decline Difference

Introduction

The Advance Decline Difference compute the difference between the number of stocks that closed higher and the number of stocks that closed lower than their previous day's closing prices.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADDIFF Indicator

To create an automatic indicator for `AdvanceDeclineDifference` , call the `addiff` helper method from the `QCAlgorithm` class. The `addiff` method creates a `AdvanceDeclineDifference` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AdvanceDeclineDifferenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._addiff = self.addiff([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._addiff.is_ready:
            # The current value of self._addiff is represented by self._addiff.current.value
            self.plot("AdvanceDeclineDifference", "addiff", self._addiff.current.value)
```

To create an automatic indicator for `AdvanceDeclineDifference` using universe constituents, call the `addiff` helper method from the `QCAlgorithm` class. The `addiff` method creates a `AdvanceDeclineDifference` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class AdvanceDeclineDifferenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._addiff = self.addiff(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._addiff.is_ready:
            # The current value of self._addiff is represented by self._addiff.current.value
            self.plot("AdvanceDeclineDifference", "addiff", self._addiff.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AdvanceDeclineDifference` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

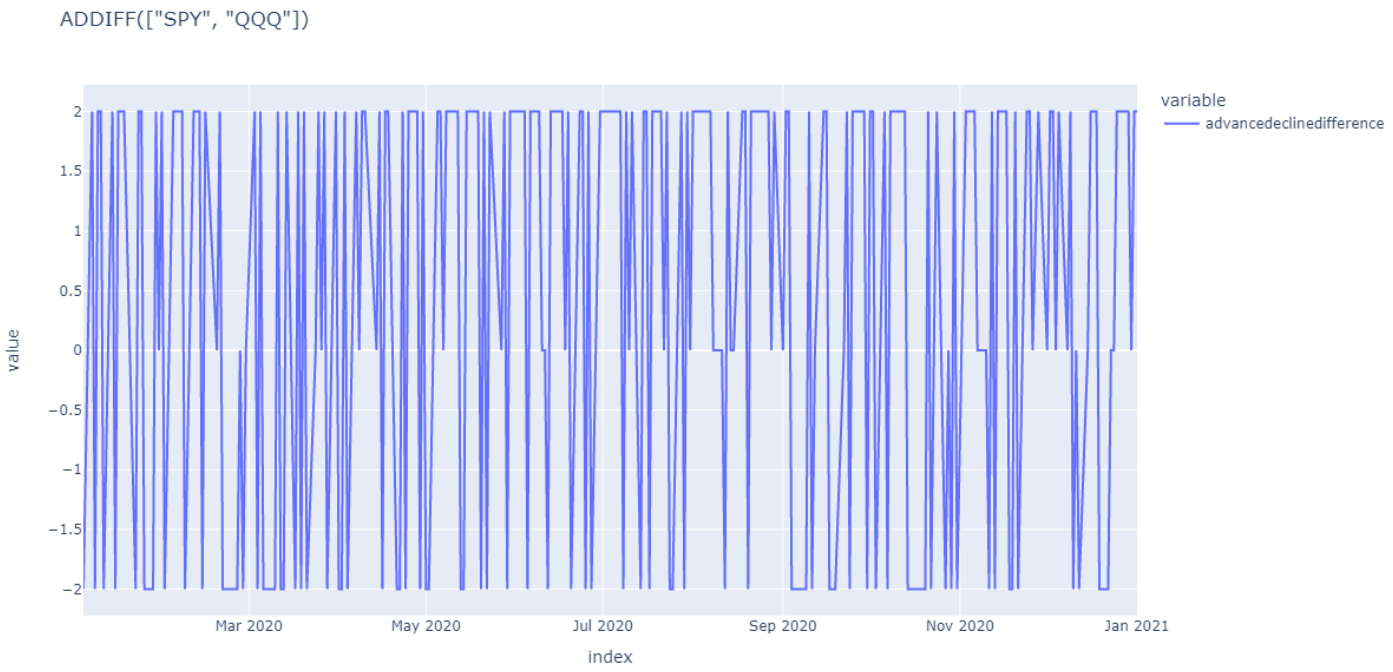
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AdvanceDeclineDifferenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advancedeclinedifference = AdvanceDeclineDifference()
        self._advancedeclinedifference.add(self._symbol);
        self._advancedeclinedifference.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._advancedeclinedifference.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._advancedeclinedifference.update(bar)

        if self._advancedeclinedifference.is_ready:
            # The current value of self._advancedeclinedifference is represented by self._advancedeclinedifference.current.value
            self.plot("AdvanceDeclineDifference", "advancedeclinedifference", self._advancedeclinedifference.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AdvanceDeclineDifference` indicator properties:



Indicator History

To get the historical data of the `AdvanceDeclineDifference` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AdvanceDeclineDifferenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._addiff = self.addiff([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._addiff, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._addiff, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._addiff, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Advance Decline Ratio

Introduction

The advance-decline ratio (ADR) compares the number of stocks that closed higher against the number of stocks that closed lower than their previous day's closing prices.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADR Indicator

To create an automatic indicator for `AdvanceDeclineRatio` , call the `adr` helper method from the `QCAlgorithm` class. The `adr` method creates a `AdvanceDeclineRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AdvanceDeclineRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adr = self.adr([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._adr.is_ready:
            # The current value of self._adr is represented by self._adr.current.value
            self.plot("AdvanceDeclineRatio", "adr", self._adr.current.value)
```

To create an automatic indicator for `AdvanceDeclineRatio` using universe constituents, call the `adr` helper method from the `QCAlgorithm` class. The `adr` method creates a `AdvanceDeclineRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class AdvanceDeclineRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._adr = self.adr(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._adr.is_ready:
            # The current value of self._adr is represented by self._adr.current.value
            self.plot("AdvanceDeclineRatio", "adr", self._adr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AdvanceDeclineRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

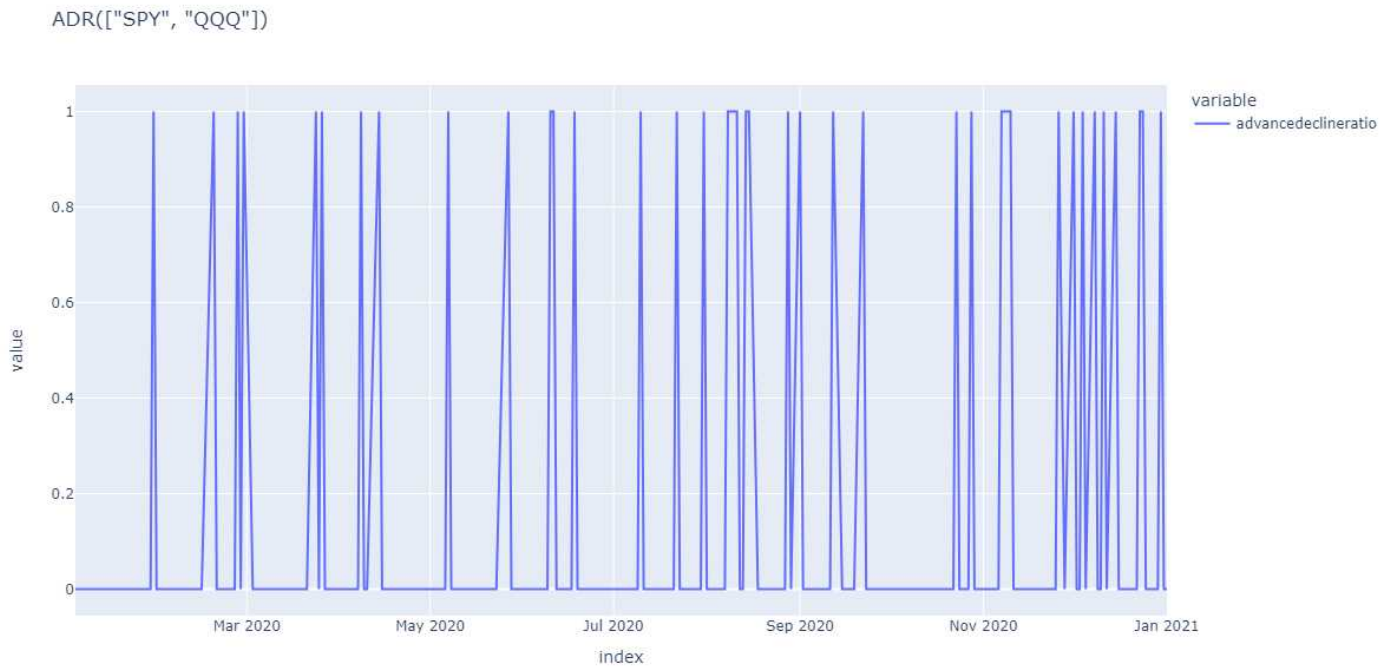
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AdvanceDeclineRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advancedeclineratio = AdvanceDeclineRatio()
        self._advancedeclineratio.add(self._symbol);
        self._advancedeclineratio.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._advancedeclineratio.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._advancedeclineratio.update(bar)

        if self._advancedeclineratio.is_ready:
            # The current value of self._advancedeclineratio is represented by self._advancedeclineratio.current.value
            self.plot("AdvanceDeclineRatio", "advancedeclineratio", self._advancedeclineratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AdvanceDeclineRatio` indicator properties:



Indicator History

To get the historical data of the `AdvanceDeclineRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AdvanceDeclineRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adr = self.adr([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._adr, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._adr, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._adr, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Advance Decline Volume Ratio

Introduction

The Advance Decline Volume Ratio is a Breadth indicator calculated as ratio of summary volume of advancing stocks to summary volume of declining stocks. AD Volume Ratio is used in technical analysis to see where the main trading activity is focused.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADVR Indicator

To create an automatic indicator for `AdvanceDeclineVolumeRatio` , call the `advr` helper method from the `QCAlgorithm` class. The `advr` method creates a `AdvanceDeclineVolumeRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AdvanceDeclineVolumeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advr = self.advr([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._advr.is_ready:
            # The current value of self._advr is represented by self._advr.current.value
            self.plot("AdvanceDeclineVolumeRatio", "advr", self._advr.current.value)
```

To create an automatic indicator for `AdvanceDeclineVolumeRatio` using universe constituents, call the `advr` helper method from the `QCAlgorithm` class. The `advr` method creates a `AdvanceDeclineVolumeRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class AdvanceDeclineVolumeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._advr = self.advr(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._advr.is_ready:
            # The current value of self._advr is represented by self._advr.current.value
            self.plot("AdvanceDeclineVolumeRatio", "advr", self._advr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AdvanceDeclineVolumeRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

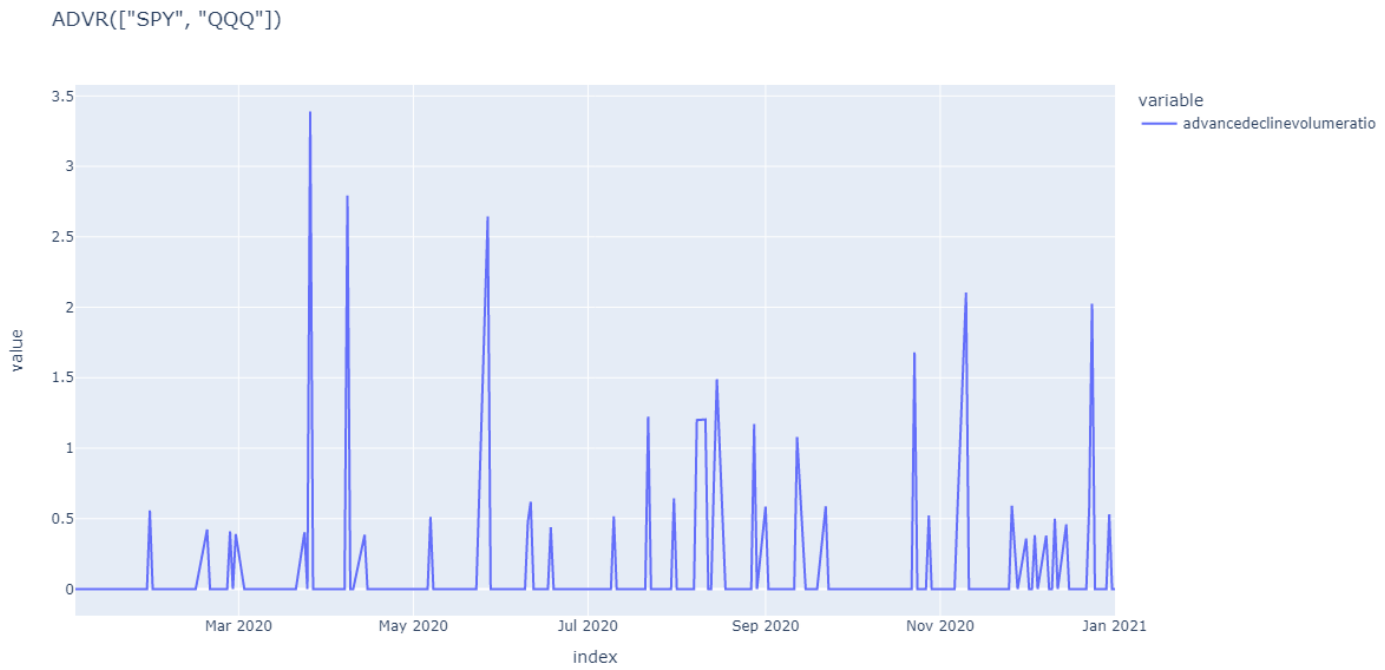
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AdvanceDeclineVolumeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advancedeclinevolumeratio = AdvanceDeclineVolumeRatio()
        self._advancedeclinevolumeratio.add(self._symbol);
        self._advancedeclinevolumeratio.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._advancedeclinevolumeratio.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._advancedeclinevolumeratio.update(bar)

        if self._advancedeclinevolumeratio.is_ready:
            # The current value of self._advancedeclinevolumeratio is represented by self._advancedeclinevolumeratio.current.value
            self.plot("AdvanceDeclineVolumeRatio", "advancedeclinevolumeratio", self._advancedeclinevolumeratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AdvanceDeclineVolumeRatio` indicator properties:



Indicator History

To get the historical data of the `AdvanceDeclineVolumeRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AdvanceDeclineVolumeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._advr = self.advr([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._advr, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._advr, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._advr, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Alpha

Introduction

In financial analysis, the Alpha indicator is used to measure the performance of an investment (such as a stock or ETF) relative to a benchmark index, often representing the broader market. Alpha indicates the excess return of the investment compared to the return of the benchmark index. The S P 500 index is frequently used as a benchmark in Alpha calculations to represent the overall market performance. Alpha is an essential tool for investors to understand the idiosyncratic returns of their investment that aren't caused by movement in the underlying benchmark.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using A Indicator

To create an automatic indicator for `Alpha` , call the `a` helper method from the `QCAlgorithm` class. The `a` method creates a `Alpha` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AlphaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._a = self.a(self._symbol, self._reference, 20)

    def on_data(self, slice: Slice) -> None:

        if self._a.is_ready:
            # The current value of self._a is represented by self._a.current.value
            self.plot("Alpha", "a", self._a.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Alpha` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

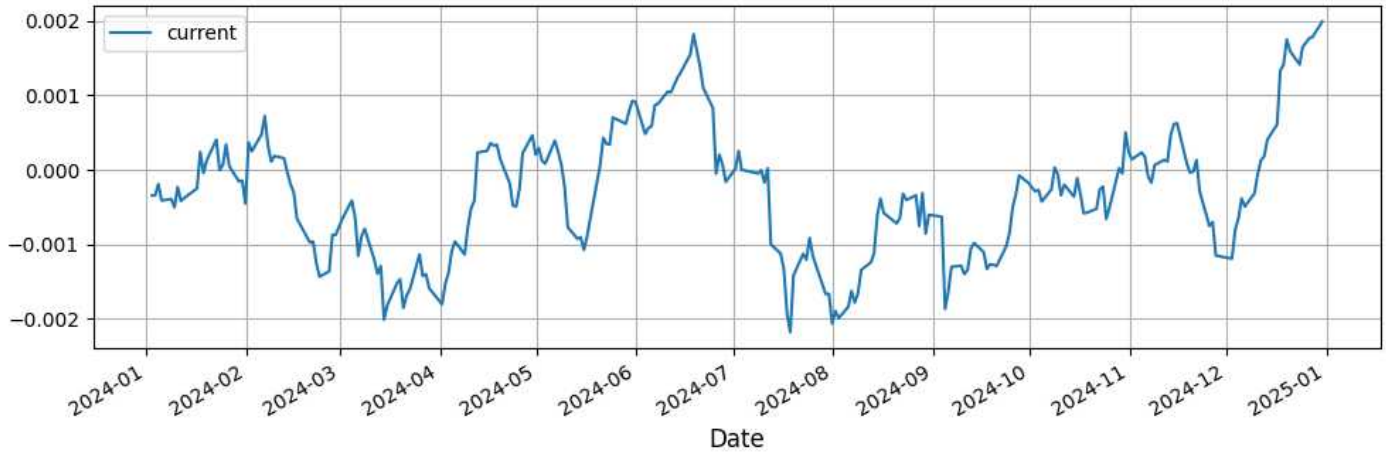
```
PY
class AlphaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._alpha = Alpha(self._symbol, self._reference, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._alpha.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._alpha.update(bar)

        if self._alpha.is_ready:
            # The current value of self._alpha is represented by self._alpha.current.value
            self.plot("Alpha", "alpha", self._alpha.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Alpha` indicator properties:

C#: A(_symbol, _reference, 20)
Py: self.a(self._symbol, self._reference, 20)



Indicator History

To get the historical data of the `Alpha` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AlphaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._a = self.a(self._symbol, self._reference, 20)

        indicator_history = self.indicator_history(self._a, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._a, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._a, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Arms Index

Introduction

The Arms Index, also called the Short-Term Trading Index (TRIN) is a technical analysis indicator that compares the number of advancing and declining stocks (AD Ratio) to advancing and declining volume (AD volume).

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TRIN Indicator

To create an automatic indicator for `ArmsIndex` , call the `trin` helper method from the `QCAlgorithm` class. The `trin` method creates a `ArmsIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ArmsIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trin = self.trin([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._trin.is_ready:
            # The current value of self._trin is represented by self._trin.current.value
            self.plot("ArmsIndex", "trin", self._trin.current.value)
            # Plot all attributes of self._trin
            self.plot("ArmsIndex", "ad_ratio", self._trin.ad_ratio.current.value)
            self.plot("ArmsIndex", "adv_ratio", self._trin.adv_ratio.current.value)
```

To create an automatic indicator for `ArmsIndex` using universe constituents, call the `trin` helper method from the `QCAlgorithm` class. The `trin` method creates a `ArmsIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class ArmsIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._trin = self.trin(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._trin.is_ready:
            # The current value of self._trin is represented by self._trin.current.value
            self.plot("ArmsIndex", "trin", self._trin.current.value)
            # Plot all attributes of self._trin
            self.plot("ArmsIndex", "ad_ratio", self._trin.ad_ratio.current.value)
            self.plot("ArmsIndex", "adv_ratio", self._trin.adv_ratio.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ArmsIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

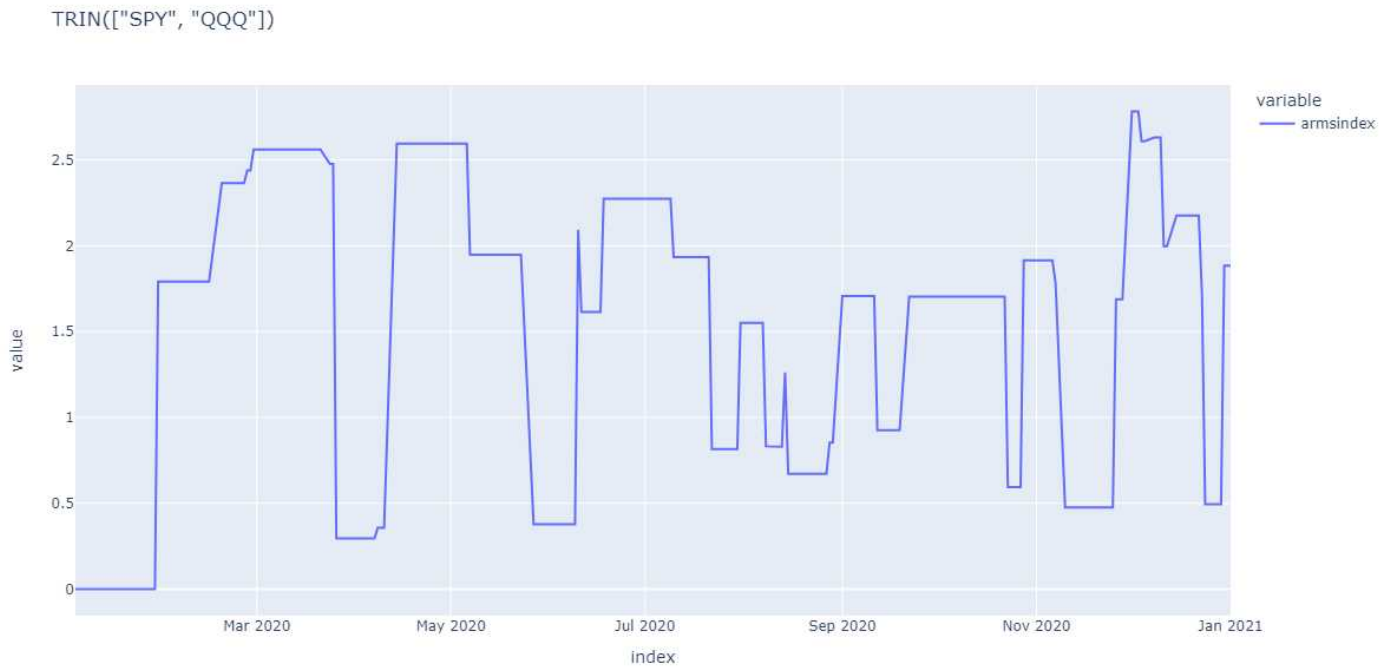
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ArmsIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._armsindex = ArmsIndex()
        self._armsindex.add(self._symbol);
        self._armsindex.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._armsindex.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._armsindex.update(bar)

        if self._armsindex.is_ready:
            # The current value of self._armsindex is represented by self._armsindex.current.value
            self.plot("ArmsIndex", "armsindex", self._armsindex.current.value)
            # Plot all attributes of self._armsindex
            self.plot("ArmsIndex", "ad_ratio", self._armsindex.ad_ratio.current.value)
            self.plot("ArmsIndex", "adv_ratio", self._armsindex.adv_ratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ArmsIndex` indicator properties:



Indicator History

To get the historical data of the `ArmsIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ArmsIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trin = self.trin([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._trin, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._trin, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._trin, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        ad_ratio = indicator_history_df["adratio"]
        adv_ratio = indicator_history_df["advratio"]
```

Supported Indicators

Arnaud Legoux Moving Average

Introduction

Smooth and high sensitive moving Average. This moving average reduce lag of the information but still being smooth to reduce noises. Is a weighted moving average, which weights have a Normal shape; the parameters Sigma and Offset affect the kurtosis and skewness of the weights respectively. source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ALMA Indicator

To create an automatic indicator for `ArnaudLegouxMovingAverage` , call the `alma` helper method from the `QCAlgorithm` class. The `alma` method creates a `ArnaudLegouxMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ArnaudLegouxMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._alma = self.alma(self._symbol, 10, 6, 0.85)

    def on_data(self, slice: Slice) -> None:

        if self._alma.is_ready:
            # The current value of self._alma is represented by self._alma.current.value
            self.plot("ArnaudLegouxMovingAverage", "alma", self._alma.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ArnaudLegouxMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

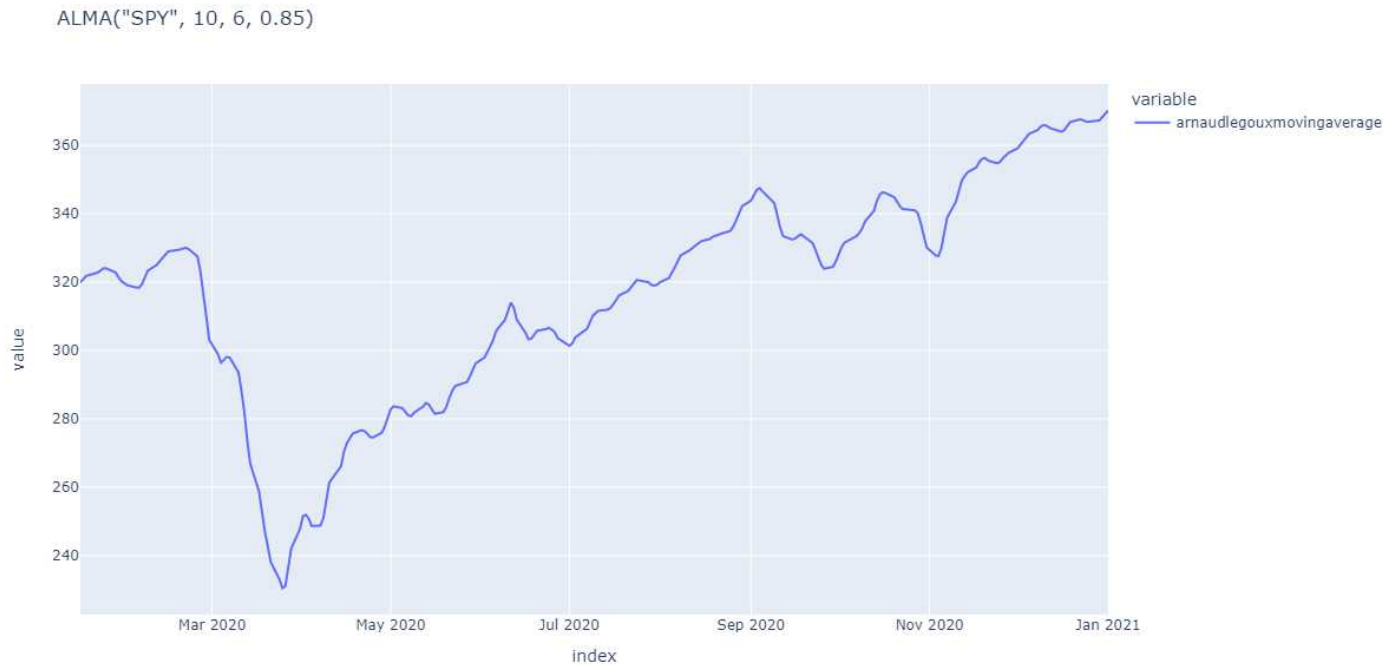
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ArnaudLegouxMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._arnaudlegouxmovingaverage = ArnaudLegouxMovingAverage(10, 6, 0.85)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._arnaudlegouxmovingaverage.update(bar.end_time, bar.close)

        if self._arnaudlegouxmovingaverage.is_ready:
            # The current value of self._arnaudlegouxmovingaverage is represented by self._arnaudlegouxmovingaverage.current.value
            self.plot("ArnaudLegouxMovingAverage", "arnaudlegouxmovingaverage", self._arnaudlegouxmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ArnaudLegouxMovingAverage` indicator properties:



Indicator History

To get the historical data of the `ArnaudLegouxMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ArnaudLegouxMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._alma = self.alma(self._symbol, 10, 6, 0.85)

        indicator_history = self.indicator_history(self._alma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._alma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._alma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Aroon Oscillator

Introduction

The Aroon Oscillator is the difference between AroonUp and AroonDown. The value of this indicator fluctuates between -100 and +100. An upward trend bias is present when the oscillator is positive, and a negative trend bias is present when the oscillator is negative. AroonUp/Down values over 75 identify strong trends in their respective direction.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using AROON Indicator

To create an automatic indicator for `AroonOscillator` , call the `aroon` helper method from the `QCAlgorithm` class. The `aroon` method creates a `AroonOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AroonOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._aroon = self.aroon(self._symbol, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._aroon.is_ready:
            # The current value of self._aroon is represented by self._aroon.current.value
            self.plot("AroonOscillator", "aroon", self._aroon.current.value)
            # Plot all attributes of self._aroon
            self.plot("AroonOscillator", "aroon_up", self._aroon.aroon_up.current.value)
            self.plot("AroonOscillator", "aroon_down", self._aroon.aroon_down.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AroonOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

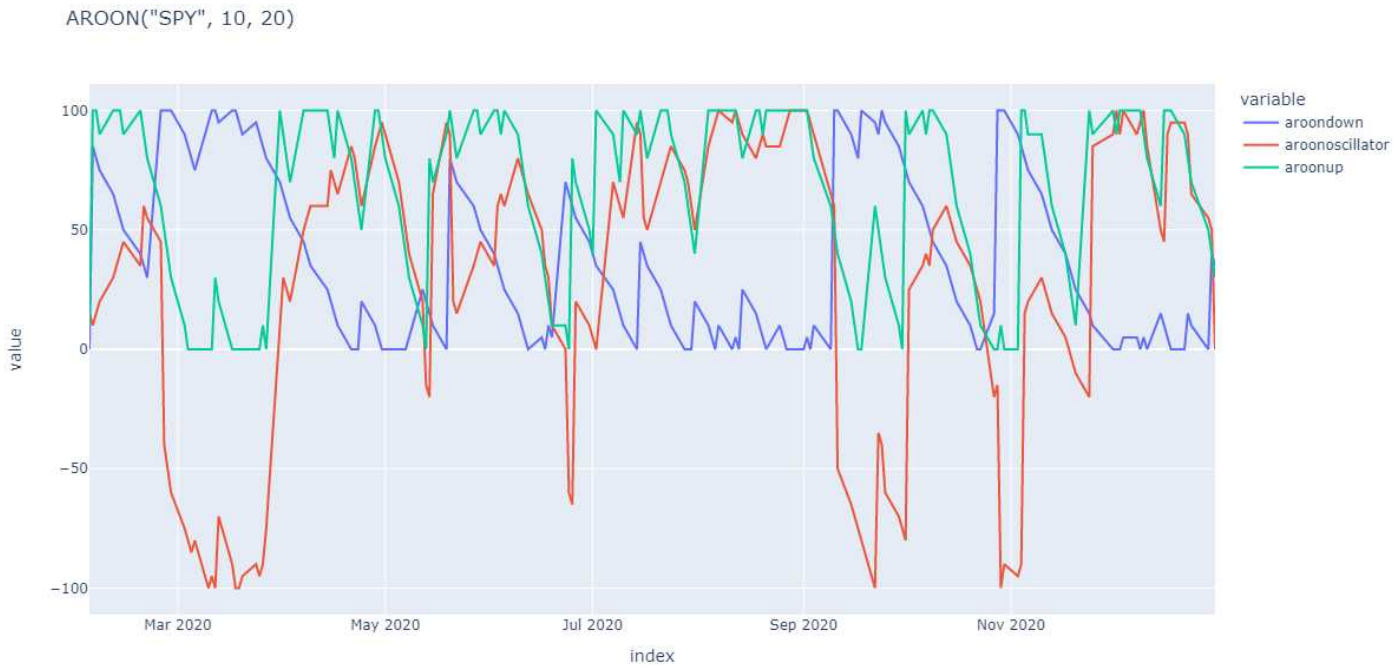
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AroonOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._aroonoscillator = AroonOscillator(10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._aroonoscillator.update(bar)

        if self._aroonoscillator.is_ready:
            # The current value of self._aroonoscillator is represented by self._aroonoscillator.current.value
            self.plot("AroonOscillator", "aroonoscillator", self._aroonoscillator.current.value)
            # Plot all attributes of self._aroonoscillator
            self.plot("AroonOscillator", "aroon_up", self._aroonoscillator.aroon_up.current.value)
            self.plot("AroonOscillator", "aroon_down", self._aroonoscillator.aroon_down.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AroonOscillator` indicator properties:



Indicator History

To get the historical data of the `AroonOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AroonOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._aroon = self.aroon(self._symbol, 10, 20)

        indicator_history = self.indicator_history(self._aroon, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._aroon, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._aroon, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        aroon_up = indicator_history_df["aroonup"]
        aroon_down = indicator_history_df["aroondown"]
```

Supported Indicators

Augen Price Spike

Introduction

The Augen Price Spike indicator is an indicator that measures price changes in terms of standard deviations. In the book, The Volatility Edge in Options Trading, Jeff Augen describes a method for tracking absolute price changes in terms of recent volatility, using the standard deviation. length = x closes = closeArray closes1 = closeArray shifted right by 1 closes2 = closeArray shifted right by 2 closeLog = np.log(np.divide(closes1, closes2)) SDev = np.std(closeLog) m = SDev * closes1[-1] spike = (closes[-1]-closes1[-1])/m return spike Augen Price Spike from TradingView https://www.tradingview.com/script/fC7Pn2X2-Price-Spike-Jeff-Augen/

To view the implementation of this indicator, see the LEAN GitHub repository .

Using APS Indicator

To create an automatic indicator for `AugenPriceSpike` , call the `aps` helper method from the `QCAlgorithm` class. The `aps` method creates a `AugenPriceSpike` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AugenPriceSpikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._aps = self.aps(self._symbol, 3)

    def on_data(self, slice: Slice) -> None:

        if self._aps.is_ready:
            # The current value of self._aps is represented by self._aps.current.value
            self.plot("AugenPriceSpike", "aps", self._aps.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AugenPriceSpike` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

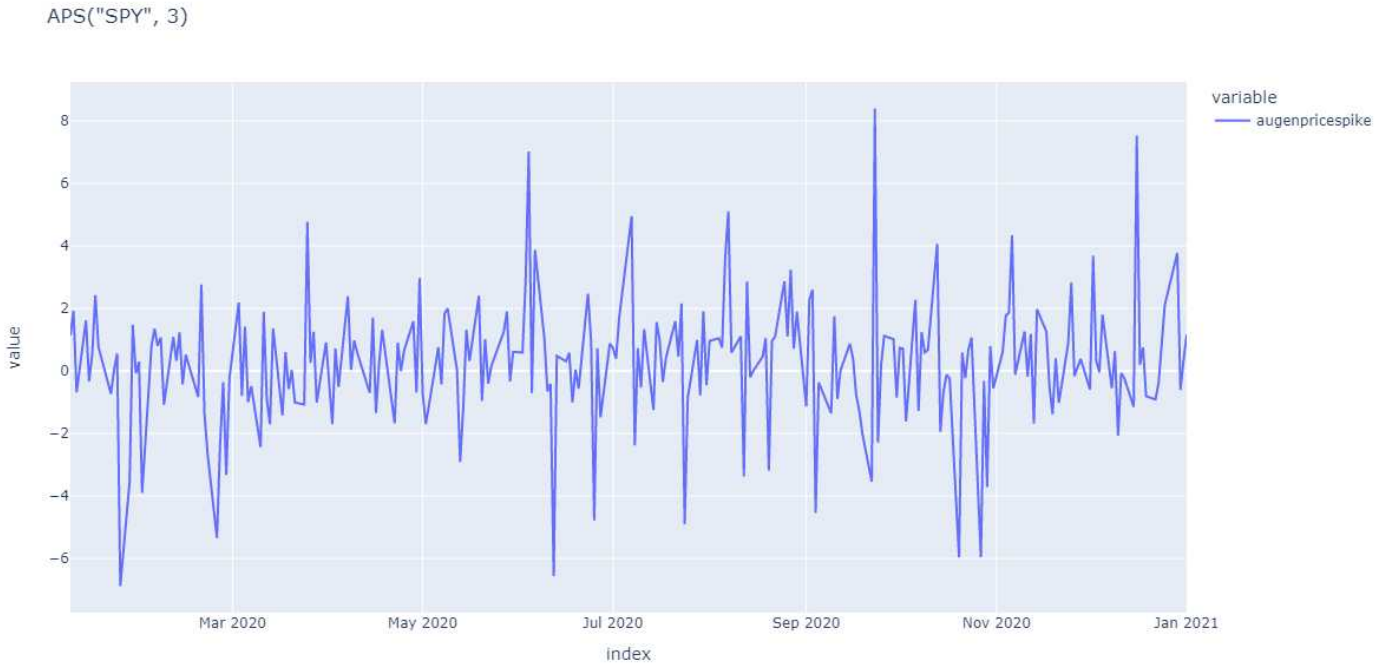
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AugenPriceSpikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._augenpricespike = AugenPriceSpike(3)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._augenpricespike.update(bar.end_time, bar.close)

        if self._augenpricespike.is_ready:
            # The current value of self._augenpricespike is represented by self._augenpricespike.current.value
            self.plot("AugenPriceSpike", "augenpricespike", self._augenpricespike.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AugenPriceSpike` indicator properties:



Indicator History

To get the historical data of the `AugenPriceSpike` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AugenPriceSpikeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._aps = self.aps(self._symbol, 3)

        indicator_history = self.indicator_history(self._aps, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._aps, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._aps, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Auto Regressive Integrated Moving Average

Introduction

An Autoregressive Intergrated Moving Average (ARIMA) is a time series model which can be used to describe a set of data. In particular,with X_t representing the series, the model assumes the data are of form (after differencing times): $X_t = c + \varepsilon_t + \Sigma_i \varphi_i X_{t-i} + \Sigma_i \theta_i \varepsilon_{t-i}$ where the first sum has an upper limit of and the second .

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ARIMA Indicator

To create an automatic indicator for `AutoRegressiveIntegratedMovingAverage` , call the `arima` helper method from the `QCAlgorithm` class. The `arima` method creates a `AutoRegressiveIntegratedMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class AutoRegressiveIntegratedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._arima = self.arima(self._symbol, 1, 1, 1, 20)

    def on_data(self, slice: Slice) -> None:

        if self._arima.is_ready:
            # The current value of self._arima is represented by self._arima.current.value
            self.plot("AutoRegressiveIntegratedMovingAverage", "arima", self._arima.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AutoRegressiveIntegratedMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

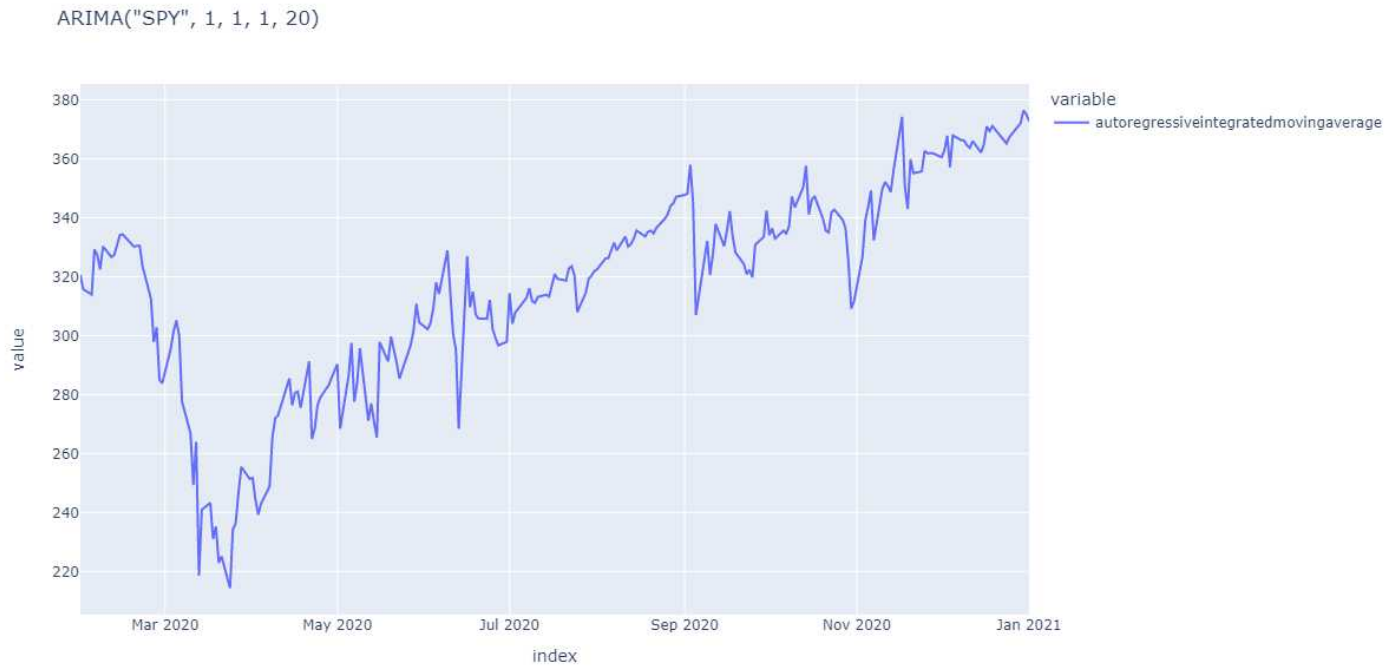
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AutoRegressiveIntegratedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._autoregressiveintegratedmovingaverage = AutoRegressiveIntegratedMovingAverage(1, 1, 1, 20, True)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._autoregressiveintegratedmovingaverage.update(bar.end_time, bar.close)

        if self._autoregressiveintegratedmovingaverage.is_ready:
            # The current value of self._autoregressiveintegratedmovingaverage is represented by
self._autoregressiveintegratedmovingaverage.current.value
            self.plot("AutoRegressiveIntegratedMovingAverage", "autoregressiveintegratedmovingaverage",
self._autoregressiveintegratedmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AutoRegressiveIntegratedMovingAverage` indicator properties:



Indicator History

To get the historical data of the `AutoRegressiveIntegratedMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AutoRegressiveIntegratedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._arima = self.arima(self._symbol, 1, 1, 1, 20)

        indicator_history = self.indicator_history(self._arima, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._arima, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._arima, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Average Directional Index

Introduction

This indicator computes Average Directional Index which measures trend strength without regard to trend direction. Firstly, it calculates the Directional Movement and the True Range value, and then the values are accumulated and smoothed using a custom smoothing method proposed by Wilder. For an n period smoothing, 1/n of each period's value is added to the total period. From these accumulated values we are therefore able to derived the 'Positive Directional Index' (+DI) and 'Negative Directional Index' (-DI) which is used to calculate the Average Directional Index. Computation source: https://stockcharts.com/school/doku.php?id=chart_school:technical_indicators:average_directional_index_adx

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADX Indicator

To create an automatic indicator for `AverageDirectionalIndex` , call the `adx` helper method from the `QCAlgorithm` class. The `adx` method creates a `AverageDirectionalIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AverageDirectionalIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adx = self.adx(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._adx.is_ready:
            # The current value of self._adx is represented by self._adx.current.value
            self.plot("AverageDirectionalIndex", "adx", self._adx.current.value)
            # Plot all attributes of self._adx
            self.plot("AverageDirectionalIndex", "positive_directional_index", self._adx.positive_directional_index.current.value)
            self.plot("AverageDirectionalIndex", "negative_directional_index", self._adx.negative_directional_index.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AverageDirectionalIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AverageDirectionalIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._averagedirectionalindex = AverageDirectionalIndex(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._averagedirectionalindex.update(bar)

        if self._averagedirectionalindex.is_ready:
            # The current value of self._averagedirectionalindex is represented by self._averagedirectionalindex.current.value
            self.plot("AverageDirectionalIndex", "averagedirectionalindex", self._averagedirectionalindex.current.value)
            # Plot all attributes of self._averagedirectionalindex
            self.plot("AverageDirectionalIndex", "positive_directional_index",
self._averagedirectionalindex.positive_directional_index.current.value)
            self.plot("AverageDirectionalIndex", "negative_directional_index",
self._averagedirectionalindex.negative_directional_index.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AverageDirectionalIndex` indicator properties:

Indicator History

To get the historical data of the `AverageDirectionalIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AverageDirectionalIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adx = self.adx(self._symbol, 20)

        indicator_history = self.indicator_history(self._adx, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._adx, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._adx, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        positive_directional_index = indicator_history_df["positivedirectionalindex"]
        negative_directional_index = indicator_history_df["negativedirectionalindex"]
```

Supported Indicators

Average Directional Movement Index Rating

Introduction

This indicator computes the Average Directional Movement Index Rating (ADXR). The Average Directional Movement Index Rating is calculated with the following formula: ADXR[i] = (ADX[i] + ADX[i - period + 1]) / 2

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ADXR Indicator

To create an automatic indicator for `AverageDirectionalMovementIndexRating` , call the `adxr` helper method from the `QCAlgorithm` class. The `adxr` method creates a `AverageDirectionalMovementIndexRating` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AverageDirectionalMovementIndexRatingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adxr = self.adxr(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._adxr.is_ready:
            # The current value of self._adxr is represented by self._adxr.current.value
            self.plot("AverageDirectionalMovementIndexRating", "adxr", self._adxr.current.value)
            # Plot all attributes of self._adxr
            self.plot("AverageDirectionalMovementIndexRating", "adx", self._adxr.adx.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AverageDirectionalMovementIndexRating` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

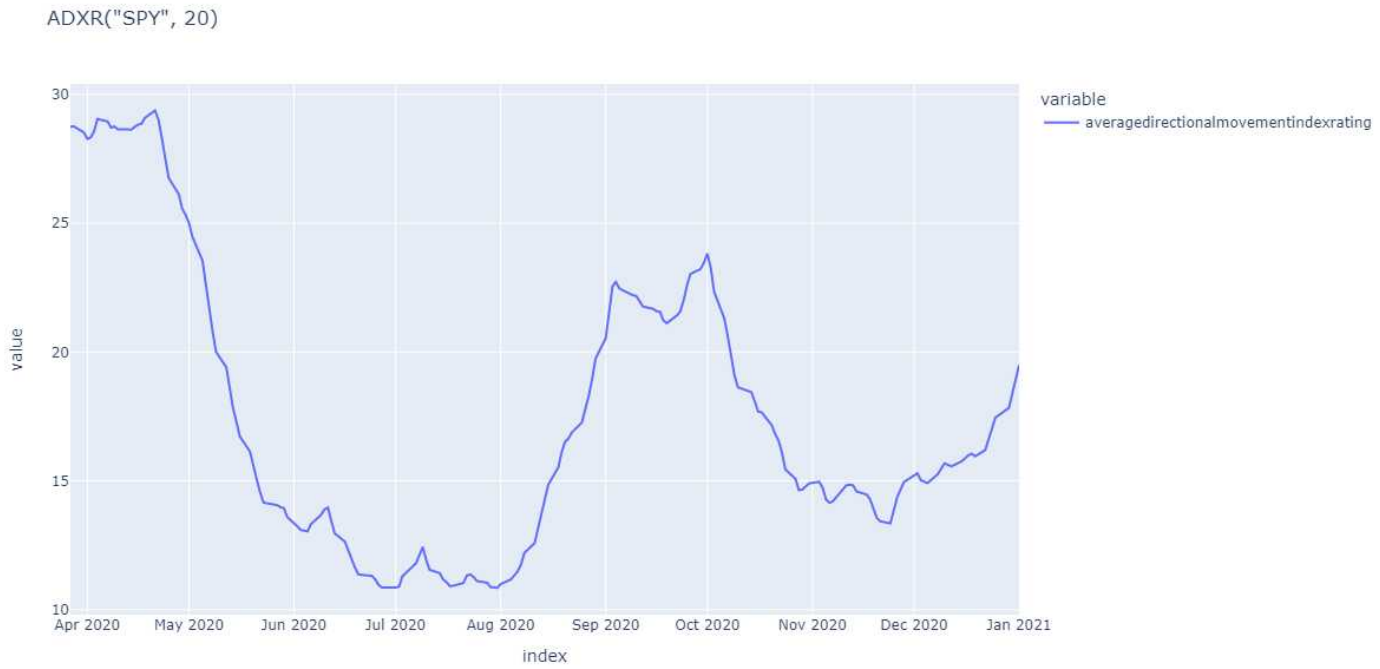
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AverageDirectionalMovementIndexRatingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._averagedirectionalmovementindexrating = AverageDirectionalMovementIndexRating(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._averagedirectionalmovementindexrating.update(bar)

        if self._averagedirectionalmovementindexrating.is_ready:
            # The current value of self._averagedirectionalmovementindexrating is represented by
self._averagedirectionalmovementindexrating.current.value
            self.plot("AverageDirectionalMovementIndexRating", "averagedirectionalmovementindexrating",
self._averagedirectionalmovementindexrating.current.value)
            # Plot all attributes of self._averagedirectionalmovementindexrating
            self.plot("AverageDirectionalMovementIndexRating", "adx", self._averagedirectionalmovementindexrating.adx.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AverageDirectionalMovementIndexRating` indicator properties:



Indicator History

To get the historical data of the `AverageDirectionalMovementIndexRating` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AverageDirectionalMovementIndexRatingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._adxr = self.adxr(self._symbol, 20)

        indicator_history = self.indicator_history(self._adxr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._adxr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._adxr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        adx = indicator_history_df["adx"]
```

Supported Indicators

Average Range

Introduction

This indicator represents the Average Range (AR) indicator, which calculates the average price range

To view the implementation of this indicator, see the LEAN GitHub repository .

Using AR Indicator

To create an automatic indicator for `AverageRange` , call the `ar` helper method from the `QCAlgorithm` class. The `ar` method creates a `AverageRange` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class AverageRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ar = self.ar(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._ar.is_ready:
            # The current value of self._ar is represented by self._ar.current.value
            self.plot("AverageRange", "ar", self._ar.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AverageRange` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

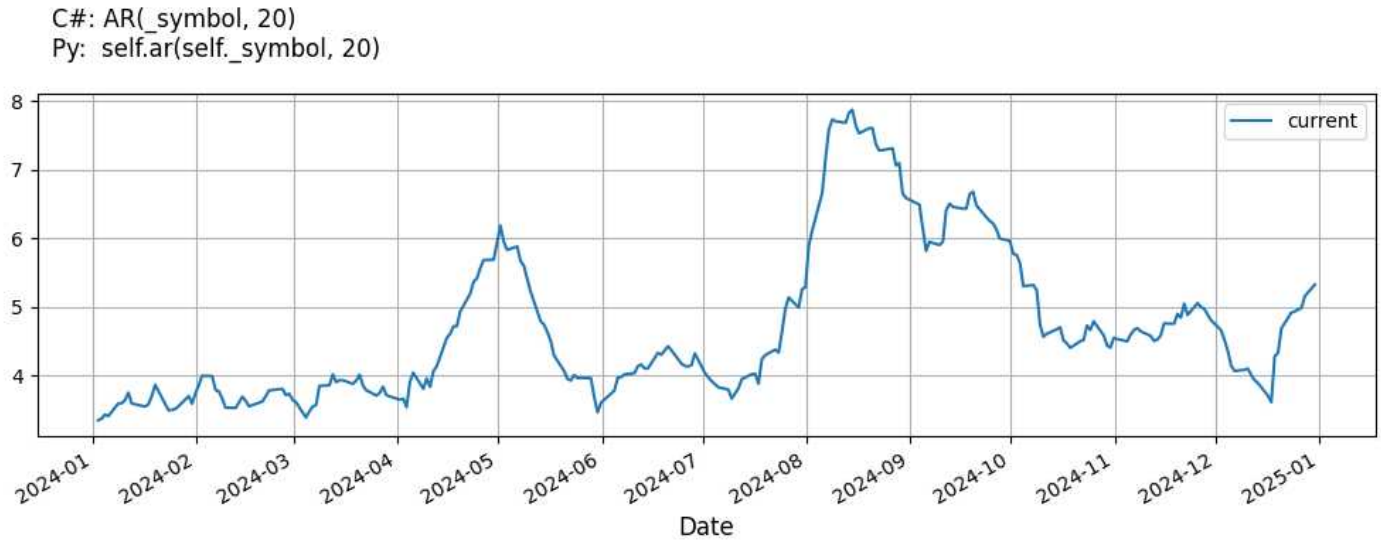
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class AverageRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._averagerange = AverageRange(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._averagerange.update(bar)

        if self._averagerange.is_ready:
            # The current value of self._averagerange is represented by self._averagerange.current.value
            self.plot("AverageRange", "averagerange", self._averagerange.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AverageRange` indicator properties:



Indicator History

To get the historical data of the `AverageRange` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class AverageRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ar = self.ar(self._symbol, 20)

        indicator_history = self.indicator_history(self._ar, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ar, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ar, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Average True Range

Introduction

The AverageTrueRange indicator is a measure of volatility introduced by Welles Wilder in his book: New Concepts in Technical Trading Systems. This indicator computes the TrueRange and then smoothes the TrueRange over a given period. TrueRange is defined as the maximum of the following: High - Low ABS(High - PreviousClose) ABS(Low - PreviousClose)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ATR Indicator

To create an automatic indicator for `AverageTrueRange` , call the `atr` helper method from the `QCAlgorithm` class. The `atr` method creates a `AverageTrueRange` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._atr = self.atr(self._symbol, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._atr.is_ready:
            # The current value of self._atr is represented by self._atr.current.value
            self.plot("AverageTrueRange", "atr", self._atr.current.value)
            # Plot all attributes of self._atr
            self.plot("AverageTrueRange", "true_range", self._atr.true_range.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AverageTrueRange` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

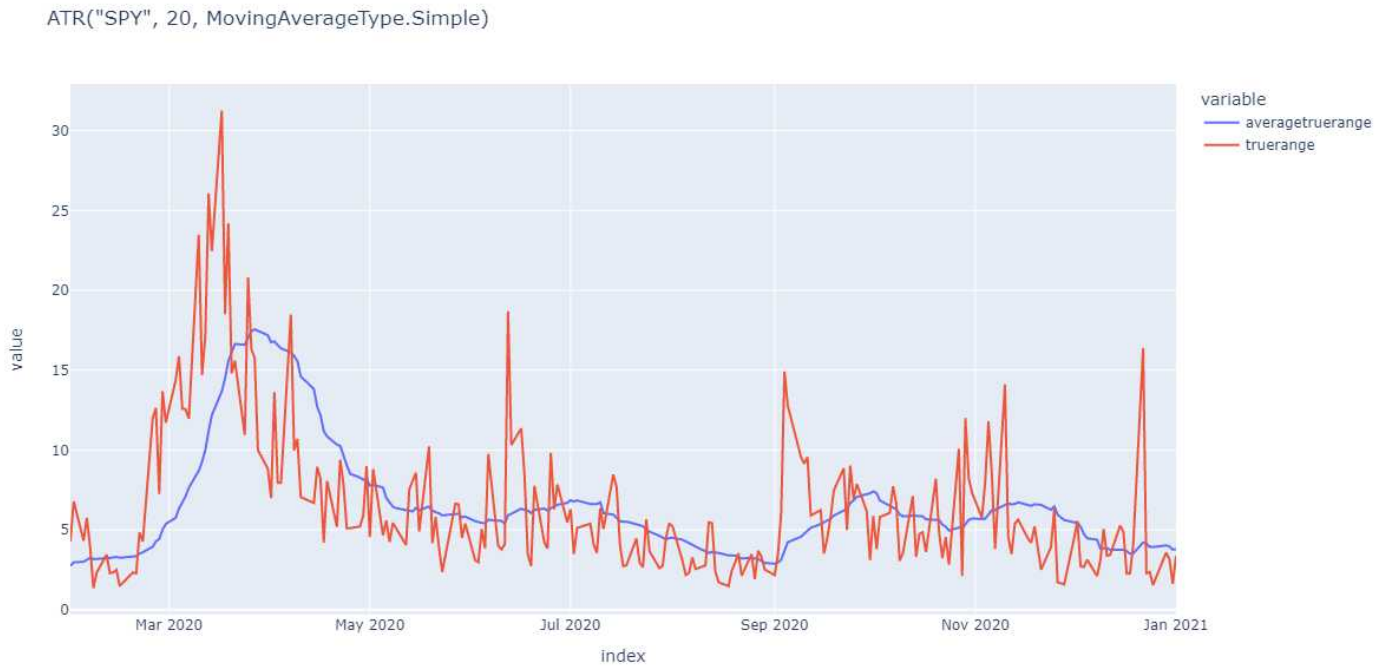
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._averagetruerange = AverageTrueRange(20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._averagetruerange.update(bar)

        if self._averagetruerange.is_ready:
            # The current value of self._averagetruerange is represented by self._averagetruerange.current.value
            self.plot("AverageTrueRange", "averagetruerange", self._averagetruerange.current.value)
            # Plot all attributes of self._averagetruerange
            self.plot("AverageTrueRange", "true_range", self._averagetruerange.true_range.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AverageTrueRange` indicator properties:



Indicator History

To get the historical data of the `AverageTrueRange` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class AverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._atr = self.atr(self._symbol, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._atr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._atr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._atr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        true_range = indicator_history_df["truerange"]
```

Supported Indicators

Awesome Oscillator

Introduction

The Awesome Oscillator Indicator tracks the price midpoint-movement of a security. Specifically, AO = MAfast[(H+L)/2] - MAslow[(H+L)/2] where MAfast and MAslow denote simple moving averages wherein fast has a shorter period. https://www.barchart.com/education/technical-indicators/awesome_oscillator

To view the implementation of this indicator, see the LEAN GitHub repository .

Using AO Indicator

To create an automatic indicator for `AwesomeOscillator` , call the `ao` helper method from the `QCAlgorithm` class. The `ao` method creates a `AwesomeOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class AwesomeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ao = self.ao(self._symbol, 10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._ao.is_ready:
            # The current value of self._ao is represented by self._ao.current.value
            self.plot("AwesomeOscillator", "ao", self._ao.current.value)
            # Plot all attributes of self._ao
            self.plot("AwesomeOscillator", "slow_ao", self._ao.slow_ao.current.value)
            self.plot("AwesomeOscillator", "fast_ao", self._ao.fast_ao.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `AwesomeOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

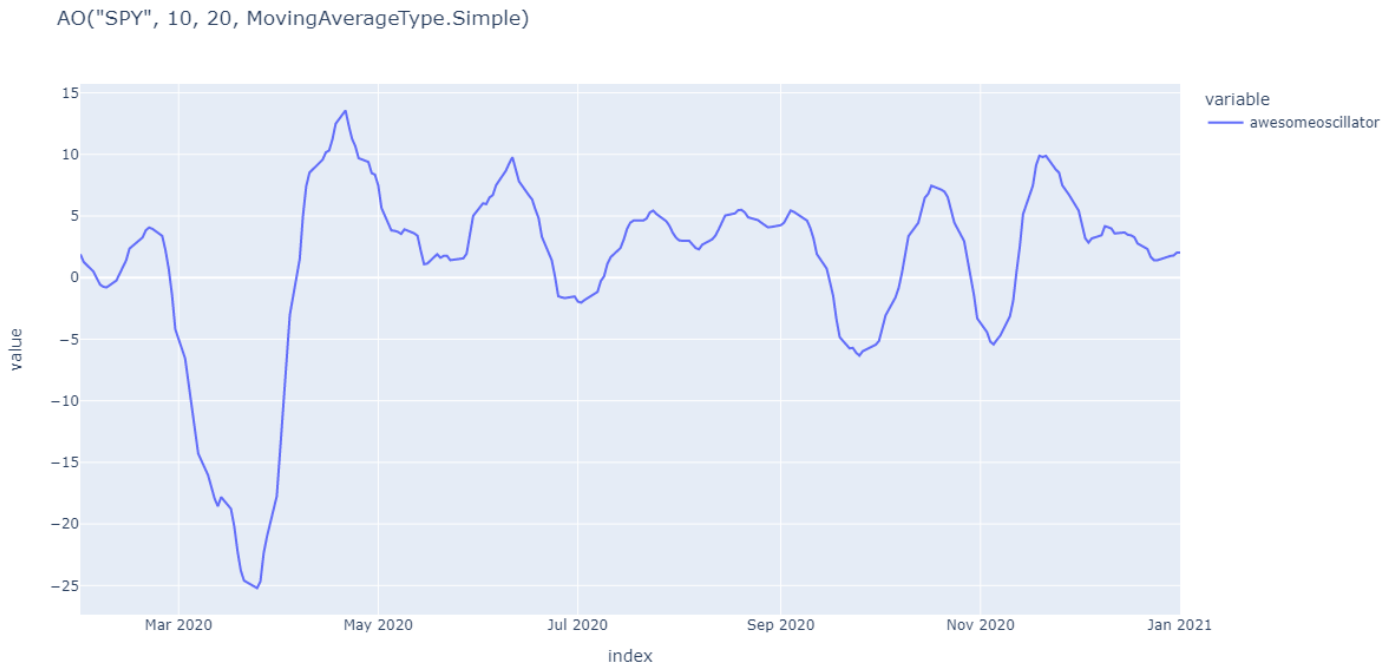
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class AwesomeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._awesomeoscillator = AwesomeOscillator(10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._awesomeoscillator.update(bar)

        if self._awesomeoscillator.is_ready:
            # The current value of self._awesomeoscillator is represented by self._awesomeoscillator.current.value
            self.plot("AwesomeOscillator", "awesomeoscillator", self._awesomeoscillator.current.value)
            # Plot all attributes of self._awesomeoscillator
            self.plot("AwesomeOscillator", "slow_ao", self._awesomeoscillator.slow_ao.current.value)
            self.plot("AwesomeOscillator", "fast_ao", self._awesomeoscillator.fast_ao.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `AwesomeOscillator` indicator properties:



Indicator History

To get the historical data of the `AwesomeOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class AwesomeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ao = self.ao(self._symbol, 10, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._ao, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ao, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ao, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        slow_ao = indicator_history_df["slowao"]
        fast_ao = indicator_history_df["fastao"]
```

Supported Indicators

Balance Of Power

Introduction

This indicator computes the Balance Of Power (BOP). The Balance Of Power is calculated with the following formula: BOP = (Close - Open) / (High - Low)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using BOP Indicator

To create an automatic indicator for `BalanceOfPower` , call the `bop` helper method from the `QCAlgorithm` class. The `bop` method creates a `BalanceOfPower` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class BalanceOfPowerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._bop = self.bop(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._bop.is_ready:
            # The current value of self._bop is represented by self._bop.current.value
            self.plot("BalanceOfPower", "bop", self._bop.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `BalanceOfPower` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

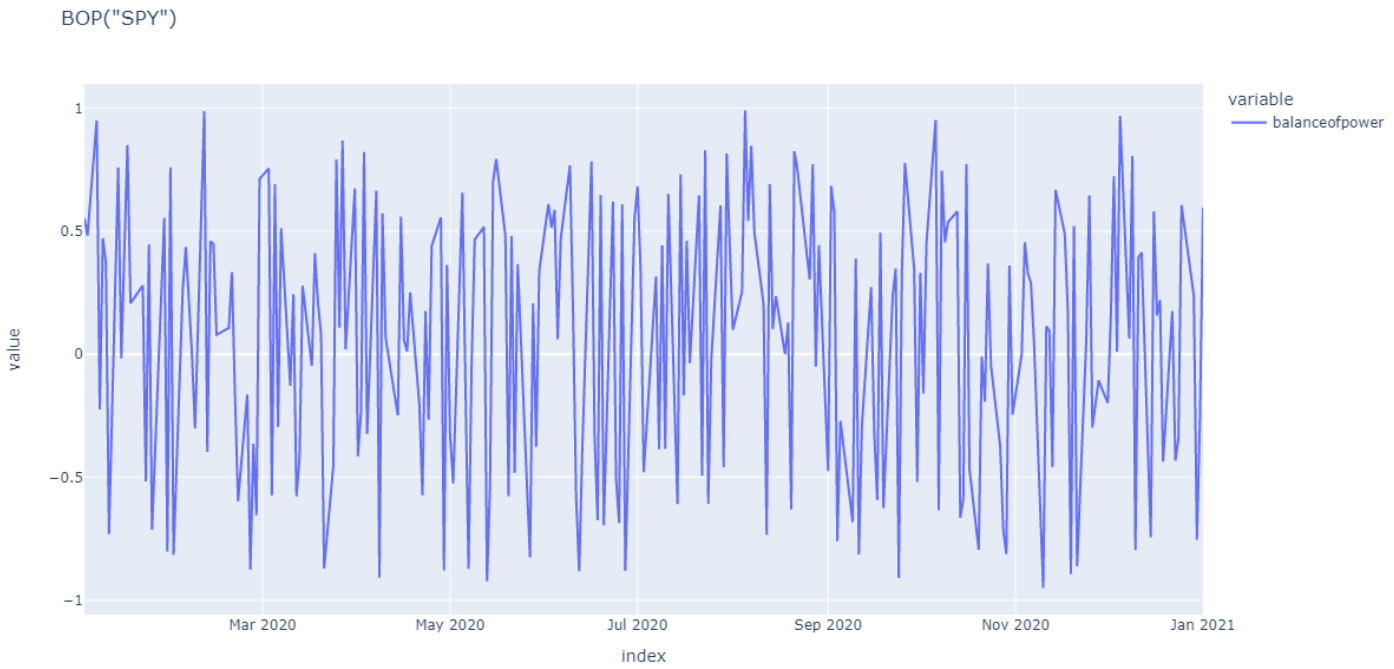
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class BalanceOfPowerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._balanceofpower = BalanceOfPower()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._balanceofpower.update(bar)

        if self._balanceofpower.is_ready:
            # The current value of self._balanceofpower is represented by self._balanceofpower.current.value
            self.plot("BalanceOfPower", "balanceofpower", self._balanceofpower.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `BalanceOfPower` indicator properties:



Indicator History

To get the historical data of the `BalanceOfPower` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class BalanceOfPowerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._bop = self.bop(self._symbol)

        indicator_history = self.indicator_history(self._bop, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._bop, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._bop, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Beta

Introduction

In technical analysis Beta indicator is used to measure volatility or risk of a target (ETF) relative to the overall risk (volatility) of the reference (market indexes). The Beta indicators compares target's price movement to the movements of the indexes over the same period of time. It is common practice to use the SPX index as a benchmark of the overall reference market when it comes to Beta calculations. The indicator only updates when both assets have a price for a time step. When a bar is missing for one of the assets, the indicator value fills forward to improve the accuracy of the indicator.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using B Indicator

To create an automatic indicator for `Beta` , call the `b` helper method from the `QCAlgorithm` class. The `b` method creates a `Beta` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class BetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._b = self.b(self._symbol, self._reference, 20)

    def on_data(self, slice: Slice) -> None:

        if self._b.is_ready:
            # The current value of self._b is represented by self._b.current.value
            self.plot("Beta", "b", self._b.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Beta` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class BetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._beta = Beta(self._symbol, self._reference, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._beta.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._beta.update(bar)

        if self._beta.is_ready:
            # The current value of self._beta is represented by self._beta.current.value
            self.plot("Beta", "beta", self._beta.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Beta` indicator properties:

Indicator History

To get the historical data of the `Beta` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class BetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._b = self.b(self._symbol, self._reference, 20)

        indicator_history = self.indicator_history(self._b, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._b, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._b, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Bollinger Bands

Introduction

This indicator creates a moving average (middle band) with an upper band and lower band fixed at k standard deviations above and below the moving average.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using BB Indicator

To create an automatic indicator for `BollingerBands` , call the `bb` helper method from the `QCAlgorithm` class. The `bb` method creates a `BollingerBands` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class BollingerBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._bb = self.bb(self._symbol, 30, 2)

    def on_data(self, slice: Slice) -> None:

        if self._bb.is_ready:
            # The current value of self._bb is represented by self._bb.current.value
            self.plot("BollingerBands", "bb", self._bb.current.value)
            # Plot all attributes of self._bb
            self.plot("BollingerBands", "standard_deviation", self._bb.standard_deviation.current.value)
            self.plot("BollingerBands", "middle_band", self._bb.middle_band.current.value)
            self.plot("BollingerBands", "upper_band", self._bb.upper_band.current.value)
            self.plot("BollingerBands", "lower_band", self._bb.lower_band.current.value)
            self.plot("BollingerBands", "band_width", self._bb.band_width.current.value)
            self.plot("BollingerBands", "percent_b", self._bb.percent_b.current.value)
            self.plot("BollingerBands", "price", self._bb.price.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `BollingerBands` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class BollingerBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._bollingerbands = BollingerBands(30, 2)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._bollingerbands.update(bar.end_time, bar.close)

        if self._bollingerbands.is_ready:
            # The current value of self._bollingerbands is represented by self._bollingerbands.current.value
            self.plot("BollingerBands", "bollingerbands", self._bollingerbands.current.value)
            # Plot all attributes of self._bollingerbands
            self.plot("BollingerBands", "standard_deviation", self._bollingerbands.standard_deviation.current.value)
            self.plot("BollingerBands", "middle_band", self._bollingerbands.middle_band.current.value)
            self.plot("BollingerBands", "upper_band", self._bollingerbands.upper_band.current.value)
            self.plot("BollingerBands", "lower_band", self._bollingerbands.lower_band.current.value)
            self.plot("BollingerBands", "band_width", self._bollingerbands.band_width.current.value)
            self.plot("BollingerBands", "percent_b", self._bollingerbands.percent_b.current.value)
            self.plot("BollingerBands", "price", self._bollingerbands.price.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `BollingerBands` indicator properties:

Indicator History

To get the historical data of the `BollingerBands` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class BollingerBandsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._bb = self.bb(self._symbol, 30, 2)

        indicator_history = self.indicator_history(self._bb, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._bb, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._bb, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        standard_deviation = indicator_history_df["standarddeviation"]
        middle_band = indicator_history_df["middleband"]
        upper_band = indicator_history_df["upperband"]
        lower_band = indicator_history_df["lowerband"]
        band_width = indicator_history_df["bandwidth"]
        percent_b = indicator_history_df["percentb"]
        price = indicator_history_df["price"]
```

Supported Indicators

Chaikin Money Flow

Introduction

The Chaikin Money Flow Index (CMF) is a volume-weighted average of accumulation and distribution over a specified period. CMF = n-day Sum of [((C - L) - (H - C)) / (H - L)) x Vol] / n-day Sum of Vol Where: n = number of periods, typically 21 H = high L = low C = close Vol = volume https://www.fidelity.com/learning-center/trading-investing/technical-analysis/technical-indicator-guide/cmf

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CMF Indicator

To create an automatic indicator for `ChaikinMoneyFlow` , call the `cmf` helper method from the `QCAlgorithm` class. The `cmf` method creates a `ChaikinMoneyFlow` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ChaikinMoneyFlowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cmf = self.cmf(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._cmf.is_ready:
            # The current value of self._cmf is represented by self._cmf.current.value
            self.plot("ChaikinMoneyFlow", "cmf", self._cmf.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ChaikinMoneyFlow` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

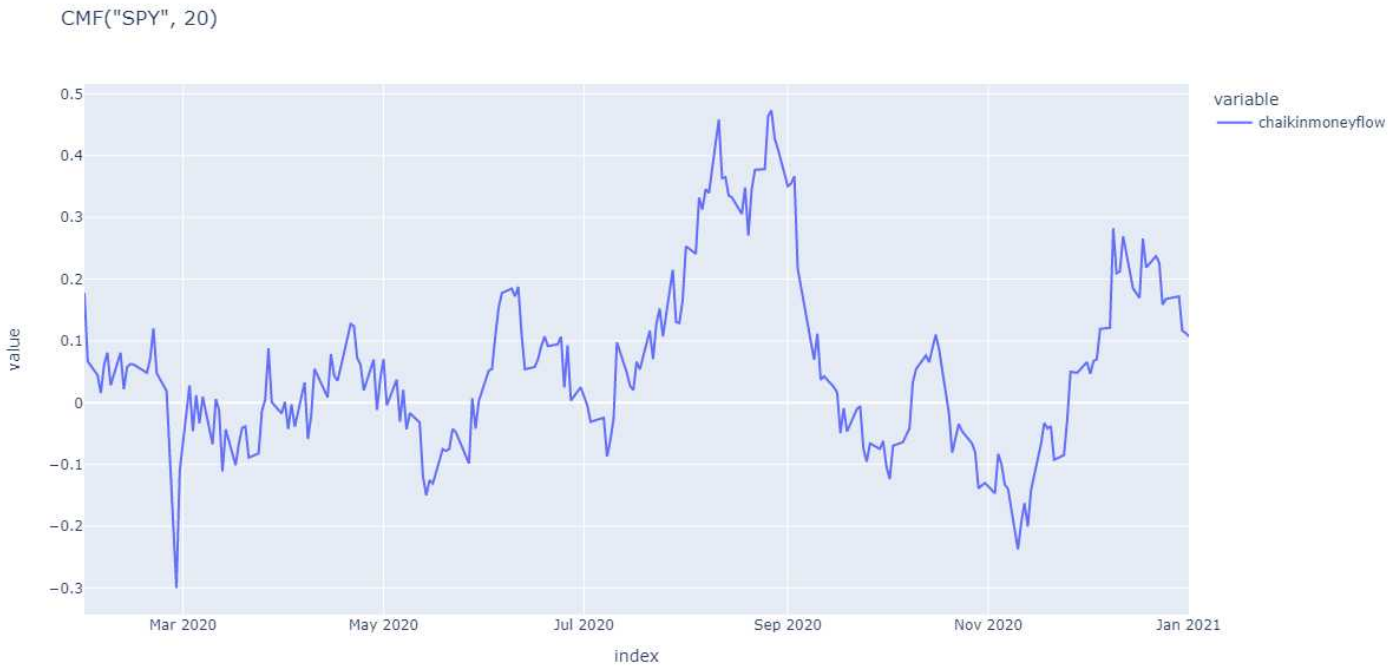
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ChaikinMoneyFlowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chaikinmoneyflow = ChaikinMoneyFlow(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._chaikinmoneyflow.update(bar)

        if self._chaikinmoneyflow.is_ready:
            # The current value of self._chaikinmoneyflow is represented by self._chaikinmoneyflow.current.value
            self.plot("ChaikinMoneyFlow", "chaikinmoneyflow", self._chaikinmoneyflow.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ChaikinMoneyFlow` indicator properties:



Indicator History

To get the historical data of the `ChaikinMoneyFlow` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class ChaikinMoneyFlowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cmf = self.cmf(self._symbol, 20)

        indicator_history = self.indicator_history(self._cmf, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cmf, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cmf, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Chaikin Oscillator

Introduction

This class is an alias for AccumulationDistributionOscillator (also known as Chaikin Oscillator).

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CO Indicator

To create an automatic indicator for `ChaikinOscillator` , call the `co` helper method from the `QCAlgorithm` class. The `co` method creates a `ChaikinOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ChaikinOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._co = self.co(self._symbol, 3, 10)

    def on_data(self, slice: Slice) -> None:

        if self._co.is_ready:
            # The current value of self._co is represented by self._co.current.value
            self.plot("ChaikinOscillator", "co", self._co.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ChaikinOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

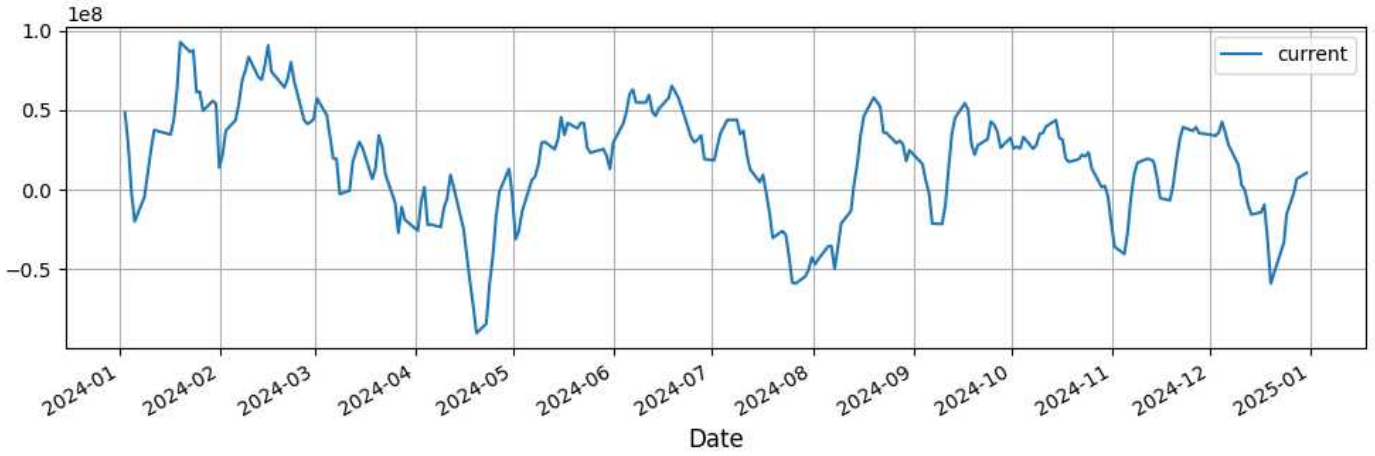
```
PY
class ChaikinOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chaikinoscillator = ChaikinOscillator(3,10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._chaikinoscillator.update(bar)

        if self._chaikinoscillator.is_ready:
            # The current value of self._chaikinoscillator is represented by self._chaikinoscillator.current.value
            self.plot("ChaikinOscillator", "chaikinoscillator", self._chaikinoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ChaikinOscillator` indicator properties:

C#: CO(_symbol, 3, 10)
Py: self.co(self._symbol, 3, 10)



Indicator History

To get the historical data of the `ChaikinOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ChaikinOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._co = self.co(self._symbol, 3, 10)

        indicator_history = self.indicator_history(self._co, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._co, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._co, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Chande Kroll Stop

Introduction

This indicator computes the short stop and lower stop values of the Chande Kroll Stop Indicator. It is used to determine the optimal placement of a stop-loss order.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CKS Indicator

To create an automatic indicator for `ChandeKrollStop` , call the `cks` helper method from the `QCAlgorithm` class. The `cks` method creates a `ChandeKrollStop` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ChandeKrollStopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cks = self.cks(self._symbol, 10, 1, 9)

    def on_data(self, slice: Slice) -> None:

        if self._cks.is_ready:
            # The current value of self._cks is represented by self._cks.current.value
            self.plot("ChandeKrollStop", "cks", self._cks.current.value)
            # Plot all attributes of self._cks
            self.plot("ChandeKrollStop", "short_stop", self._cks.short_stop.current.value)
            self.plot("ChandeKrollStop", "long_stop", self._cks.long_stop.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ChandeKrollStop` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

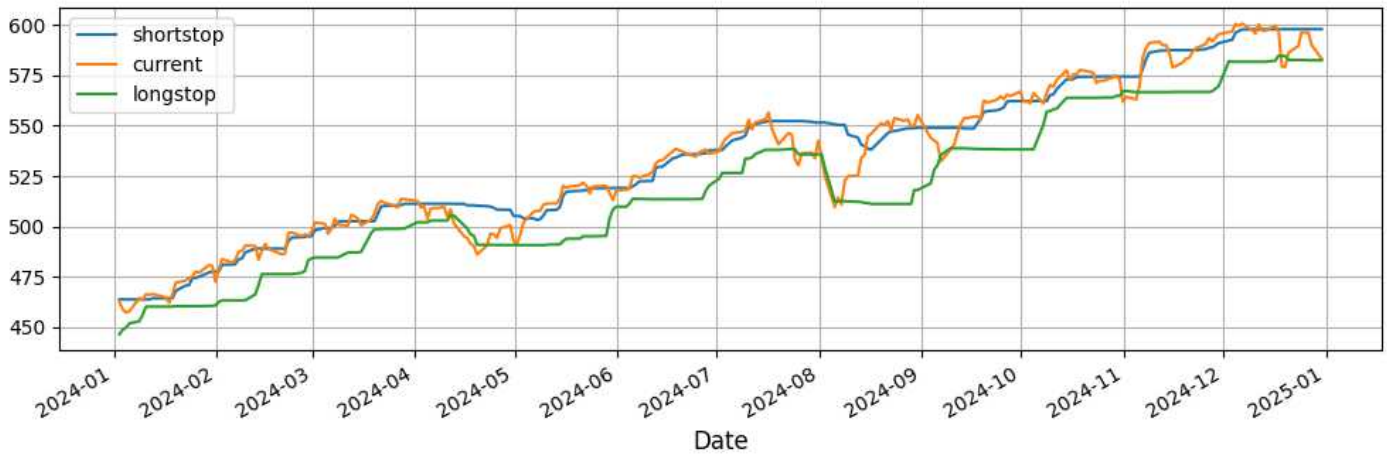
```
PY
class ChandeKrollStopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chandekrollstop = ChandeKrollStop(10, 1, 9)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._chandekrollstop.update(bar)

        if self._chandekrollstop.is_ready:
            # The current value of self._chandekrollstop is represented by self._chandekrollstop.current.value
            self.plot("ChandeKrollStop", "chandekrollstop", self._chandekrollstop.current.value)
            # Plot all attributes of self._chandekrollstop
            self.plot("ChandeKrollStop", "short_stop", self._chandekrollstop.short_stop.current.value)
            self.plot("ChandeKrollStop", "long_stop", self._chandekrollstop.long_stop.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ChandeKrollStop` indicator properties:

C#: CKS(_symbol, 10, 1, 9)
Py: self.cks(self._symbol, 10, 1, 9)



Indicator History

To get the historical data of the `ChandeKrollStop` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ChandeKrollStopAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cks = self.cks(self._symbol, 10, 1, 9)

        indicator_history = self.indicator_history(self._cks, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cks, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cks, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        short_stop = indicator_history_df["shortstop"]
        long_stop = indicator_history_df["longstop"]
```

Supported Indicators

Chande Momentum Oscillator

Introduction

This indicator computes the Chande Momentum Oscillator (CMO). CMO calculation is mostly identical to RSI. The only difference is in the last step of calculation: RSI = gain / (gain+loss) CMO = (gain-loss) / (gain+loss)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CMO Indicator

To create an automatic indicator for `ChandeMomentumOscillator` , call the `cmo` helper method from the `QCAlgorithm` class. The `cmo` method creates a `ChandeMomentumOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ChandeMomentumOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cmo = self.cmo(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._cmo.is_ready:
            # The current value of self._cmo is represented by self._cmo.current.value
            self.plot("ChandeMomentumOscillator", "cmo", self._cmo.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ChandeMomentumOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

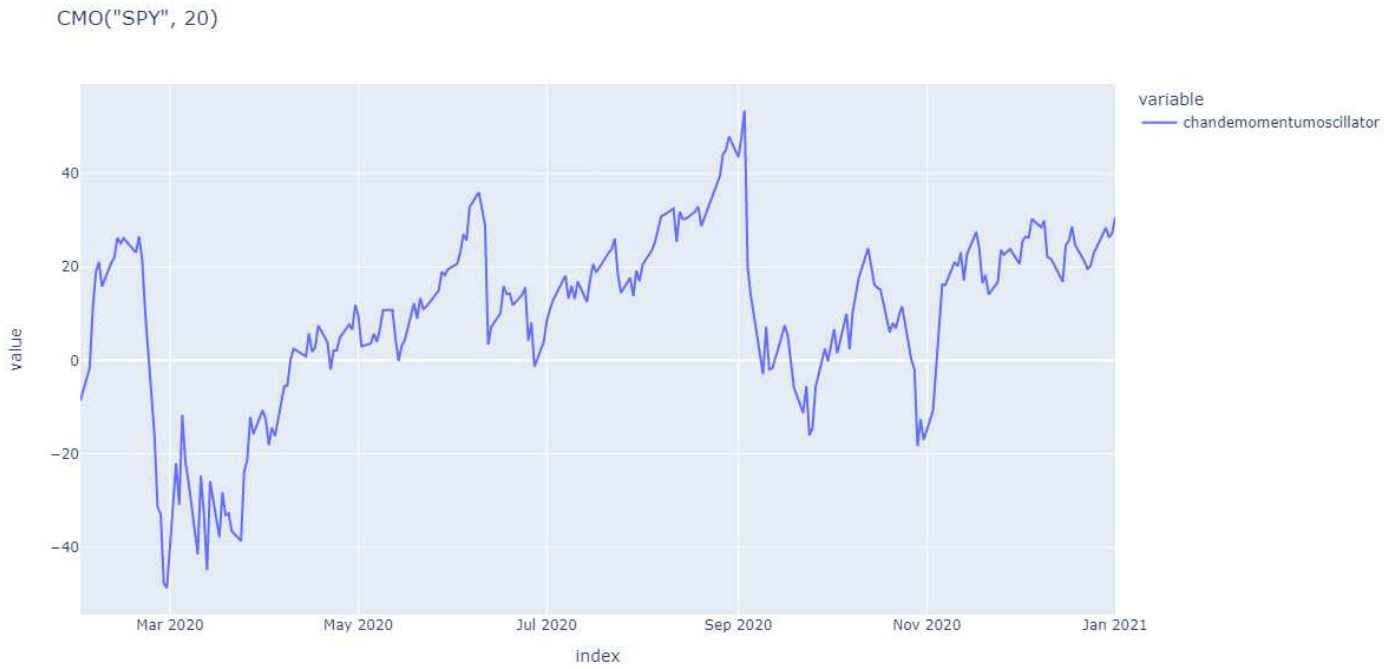
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ChandeMomentumOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chandemomentumoscillator = ChandeMomentumOscillator(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._chandemomentumoscillator.update(bar.end_time, bar.close)

        if self._chandemomentumoscillator.is_ready:
            # The current value of self._chandemomentumoscillator is represented by self._chandemomentumoscillator.current.value
            self.plot("ChandeMomentumOscillator", "chandemomentumoscillator", self._chandemomentumoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ChandeMomentumOscillator` indicator properties:



Indicator History

To get the historical data of the `ChandeMomentumOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ChandeMomentumOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cmo = self.cmo(self._symbol, 20)

        indicator_history = self.indicator_history(self._cmo, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cmo, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cmo, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Choppiness Index

Introduction

The ChoppinessIndex indicator is an indicator designed to determine if the market is choppy (trading sideways) or not choppy (trading within a trend in either direction)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CHOP Indicator

To create an automatic indicator for `ChoppinessIndex` , call the `chop` helper method from the `QCAlgorithm` class. The `chop` method creates a `ChoppinessIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ChoppinessIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chop = self.chop(self._symbol, 14)

    def on_data(self, slice: Slice) -> None:

        if self._chop.is_ready:
            # The current value of self._chop is represented by self._chop.current.value
            self.plot("ChoppinessIndex", "chop", self._chop.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ChoppinessIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

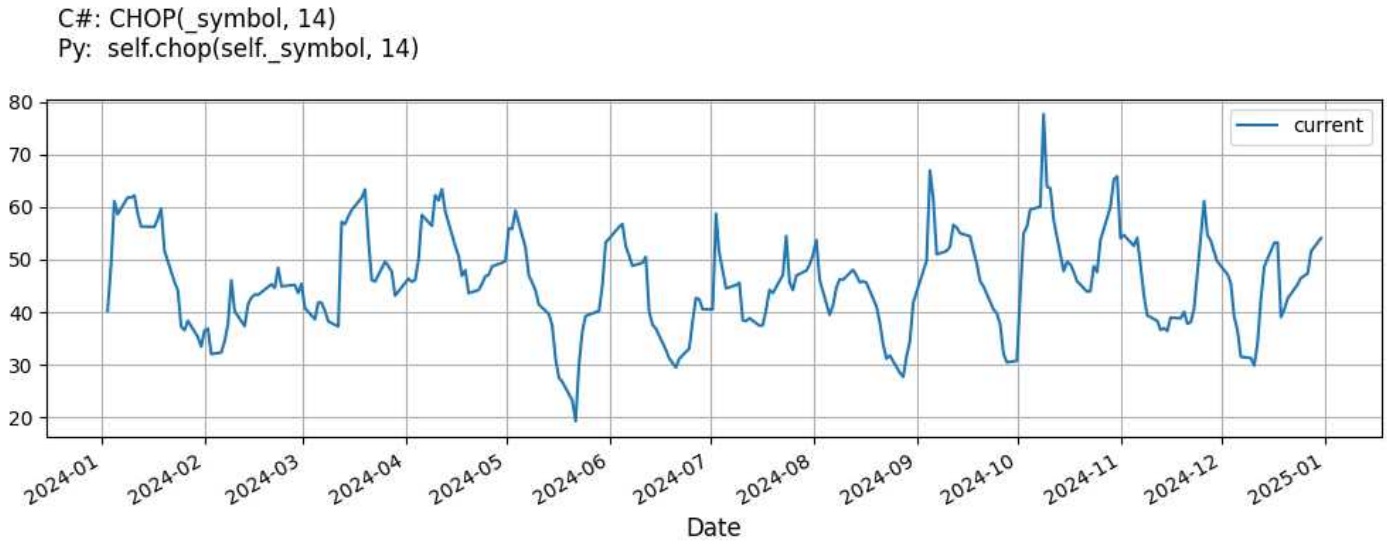
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ChoppinessIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._choppinessindex = ChoppinessIndex(14)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._choppinessindex.update(bar)

        if self._choppinessindex.is_ready:
            # The current value of self._choppinessindex is represented by self._choppinessindex.current.value
            self.plot("ChoppinessIndex", "choppinessindex", self._choppinessindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ChoppinessIndex` indicator properties:



Indicator History

To get the historical data of the `ChoppinessIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```py
class ChoppinessIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._chop = self.chop(self._symbol, 14)

        indicator_history = self.indicator_history(self._chop, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._chop, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._chop, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Commodity Channel Index

Introduction

This indicator represents the traditional commodity channel index (CCI) CCI = (Typical Price - 20-period SMA of TP) / (.015 * Mean Deviation) Typical Price (TP) = (High + Low + Close)/3 Constant = 0.015 There are four steps to calculating the Mean Deviation, first, subtract the most recent 20-period average of the typical price from each period's typical price. Second, take the absolute values of these numbers. Third, sum the absolute values. Fourth, divide by the total number of periods (20).

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CCI Indicator

To create an automatic indicator for `CommodityChannelIndex` , call the `cci` helper method from the `QCAlgorithm` class. The `cci` method creates a `CommodityChannelIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class CommodityChannelIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cci = self.cci(self._symbol, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._cci.is_ready:
            # The current value of self._cci is represented by self._cci.current.value
            self.plot("CommodityChannelIndex", "cci", self._cci.current.value)
            # Plot all attributes of self._cci
            self.plot("CommodityChannelIndex", "typical_price_average", self._cci.typical_price_average.current.value)
            self.plot("CommodityChannelIndex", "typical_price_mean_deviation", self._cci.typical_price_mean_deviation.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `CommodityChannelIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

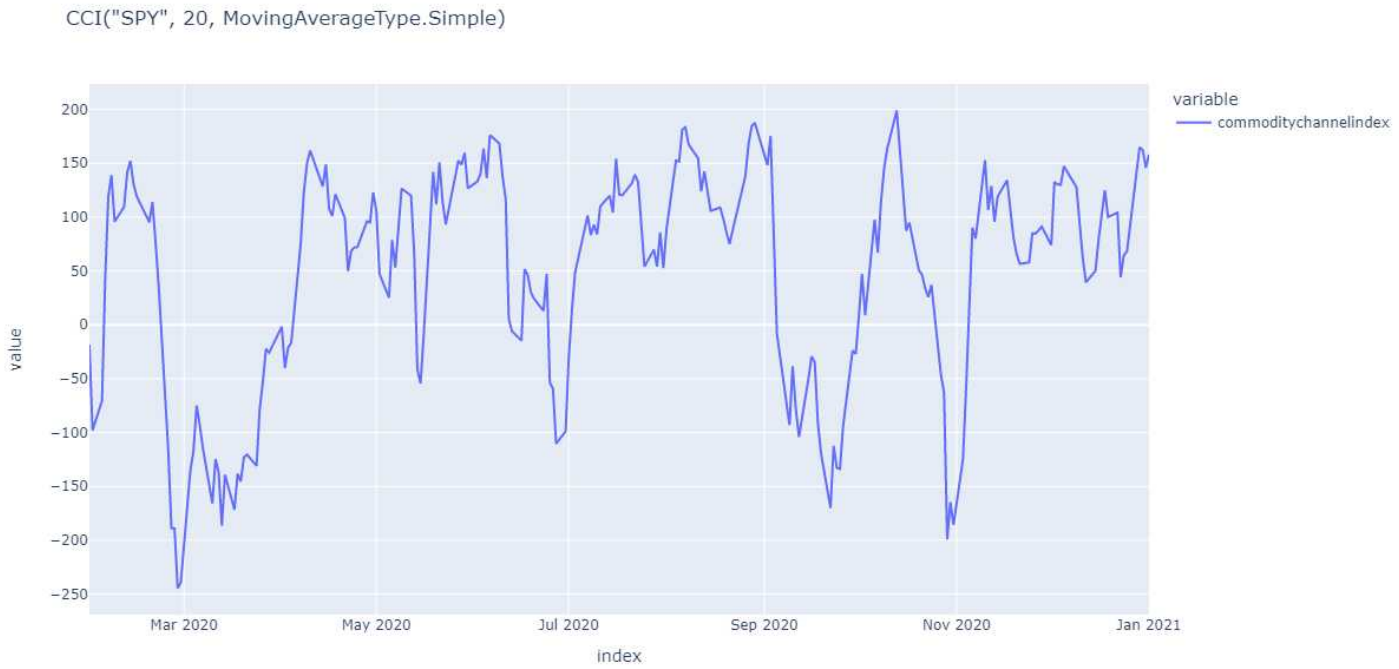
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class CommodityChannelIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._commoditychannelindex = CommodityChannelIndex(20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._commoditychannelindex.update(bar)

        if self._commoditychannelindex.is_ready:
            # The current value of self._commoditychannelindex is represented by self._commoditychannelindex.current.value
            self.plot("CommodityChannelIndex", "commoditychannelindex", self._commoditychannelindex.current.value)
            # Plot all attributes of self._commoditychannelindex
            self.plot("CommodityChannelIndex", "typical_price_average", self._commoditychannelindex.typical_price_average.current.value)
            self.plot("CommodityChannelIndex", "typical_price_mean_deviation",
self._commoditychannelindex.typical_price_mean_deviation.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `CommodityChannelIndex` indicator properties:



Indicator History

To get the historical data of the `CommodityChannelIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class CommodityChannelIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cci = self.cci(self._symbol, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._cci, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cci, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cci, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        typical_price_average = indicator_history_df["typicalpriceaverage"]
        typical_price_mean_deviation = indicator_history_df["typicalpricemeandeviation"]
```

Supported Indicators

Connors Relative Strength Index

Introduction

This indicator represents the Connors Relative Strength Index (CRSI), a combination of the traditional Relative Strength Index (RSI), a Streak RSI (SRSI), and Percent Rank. This index is designed to provide a more robust measure of market strength by combining momentum, streak behavior, and price change.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CRSI Indicator

To create an automatic indicator for `ConnorsRelativeStrengthIndex` , call the `crsi` helper method from the `QCAlgorithm` class. The `crsi` method creates a `ConnorsRelativeStrengthIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ConnorsRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._crsi = self.crsi(self._symbol, 3, 2, 100)

    def on_data(self, slice: Slice) -> None:

        if self._crsi.is_ready:
            # The current value of self._crsi is represented by self._crsi.current.value
            self.plot("ConnorsRelativeStrengthIndex", "crsi", self._crsi.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ConnorsRelativeStrengthIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

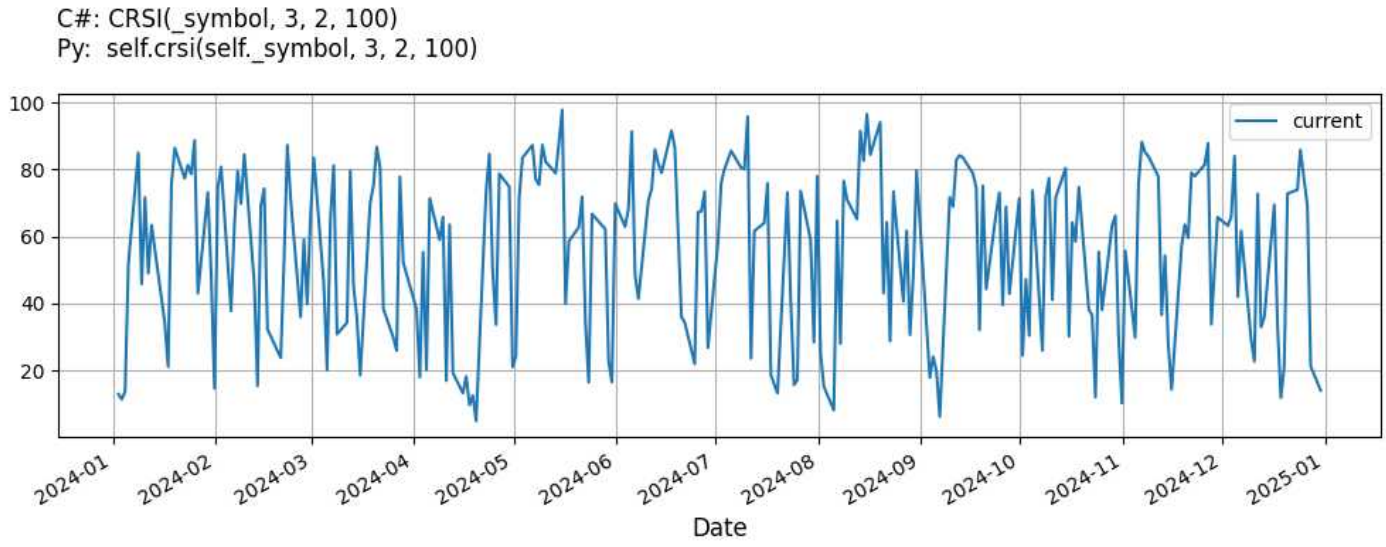
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ConnorsRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._connorsrelativestrengthindex = ConnorsRelativeStrengthIndex(3, 2, 100)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._connorsrelativestrengthindex.update(bar.end_time, bar.close)

        if self._connorsrelativestrengthindex.is_ready:
            # The current value of self._connorsrelativestrengthindex is represented by self._connorsrelativestrengthindex.current.value
            self.plot("ConnorsRelativeStrengthIndex", "connorsrelativestrengthindex", self._connorsrelativestrengthindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ConnorsRelativeStrengthIndex` indicator properties:



Indicator History

To get the historical data of the `ConnorsRelativeStrengthIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class ConnorsRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._crsi = self.crsi(self._symbol, 3, 2, 100)

        indicator_history = self.indicator_history(self._crsi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._crsi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._crsi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Coppock Curve

Introduction

A momentum indicator developed by Edwin “Sedge” Coppock in October 1965. The goal of this indicator is to identify long-term buying opportunities in the S&P500 and Dow Industrials. source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using CC Indicator

To create an automatic indicator for `CoppockCurve` , call the `cc` helper method from the `QCAlgorithm` class. The `cc` method creates a `CoppockCurve` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class CoppockCurveAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cc = self.cc(self._symbol, 11, 14, 10)

    def on_data(self, slice: Slice) -> None:

        if self._cc.is_ready:
            # The current value of self._cc is represented by self._cc.current.value
            self.plot("CoppockCurve", "cc", self._cc.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `CoppockCurve` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

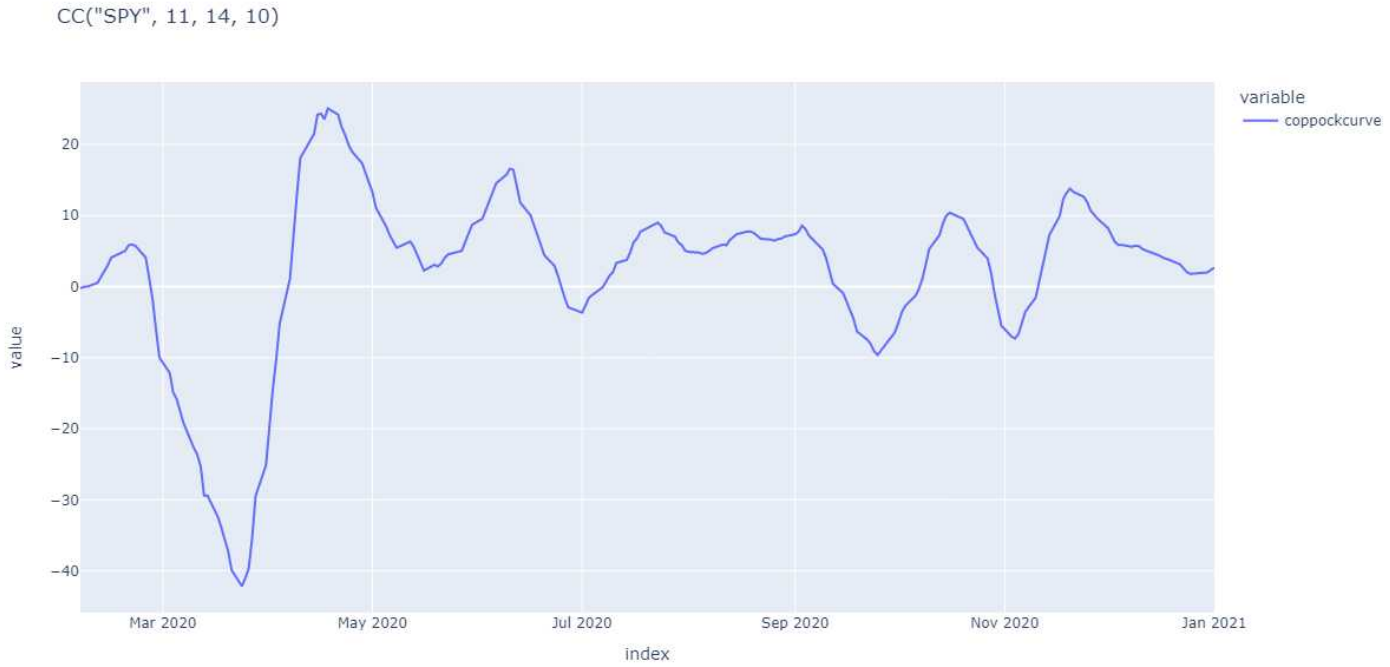
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class CoppockCurveAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._coppockcurve = CoppockCurve(11, 14, 10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._coppockcurve.update(bar.end_time, bar.close)

        if self._coppockcurve.is_ready:
            # The current value of self._coppockcurve is represented by self._coppockcurve.current.value
            self.plot("CoppockCurve", "coppockcurve", self._coppockcurve.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `CoppockCurve` indicator properties:



Indicator History

To get the historical data of the `CoppockCurve` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class CoppockCurveAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cc = self.cc(self._symbol, 11, 14, 10)

        indicator_history = self.indicator_history(self._cc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Correlation

Introduction

The Correlation Indicator is a valuable tool in technical analysis, designed to quantify the degree of relationship between the price movements of a target security (e.g., a stock or ETF) and a reference market index. It measures how closely the target's price changes are aligned with the fluctuations of the index over a specific period of time, providing insights into the target's susceptibility to market movements. A positive correlation indicates that the target tends to move in the same direction as the market index, while a negative correlation suggests an inverse relationship. A correlation close to 0 implies a weak or no linear relationship. Commonly, the SPX index is employed as the benchmark for the overall market when calculating correlation, ensuring a consistent and reliable reference point. This helps traders and investors make informed decisions regarding the risk and behavior of the target security in relation to market trends. The indicator only updates when both assets have a price for a time step. When a bar is missing for one of the assets, the indicator value fills forward to improve the accuracy of the indicator.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using C Indicator

To create an automatic indicator for `Correlation` , call the `c` helper method from the `QCAlgorithm` class. The `c` method creates a `Correlation` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class CorrelationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._c = self.c(self._symbol, self._reference, 20, CorrelationType.PEARSON)

    def on_data(self, slice: Slice) -> None:

        if self._c.is_ready:
            # The current value of self._c is represented by self._c.current.value
            self.plot("Correlation", "c", self._c.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Correlation` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

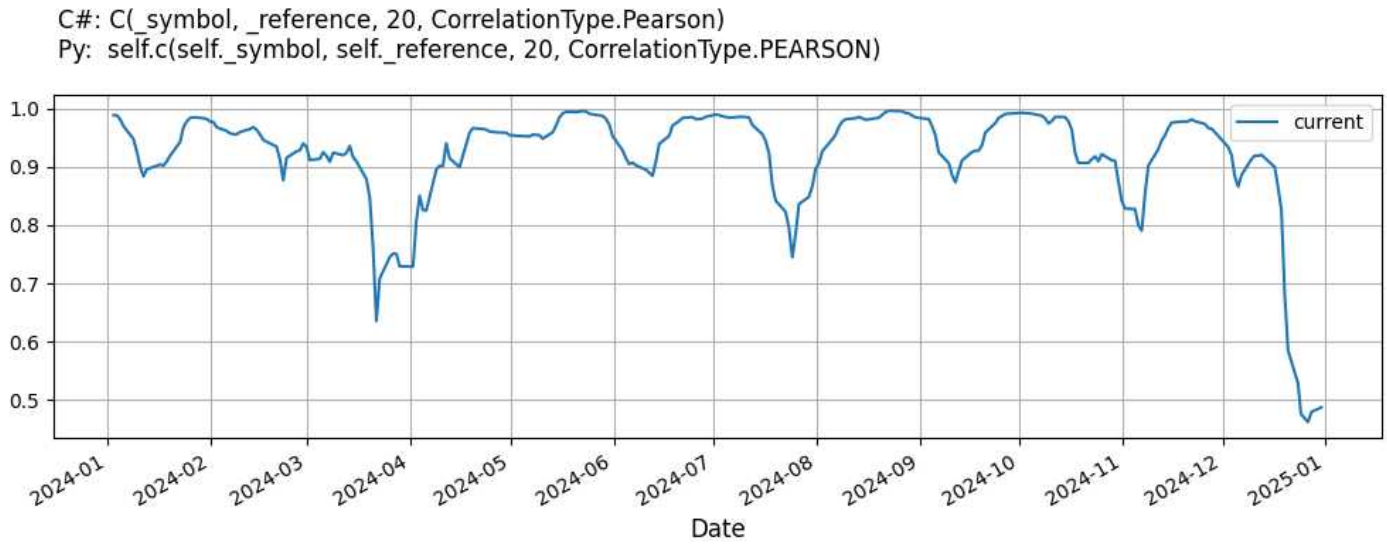
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class CorrelationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._correlation = Correlation(self._symbol, self._reference, 20, CorrelationType.PEARSON)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._correlation.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._correlation.update(bar)

        if self._correlation.is_ready:
            # The current value of self._correlation is represented by self._correlation.current.value
            self.plot("Correlation", "correlation", self._correlation.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Correlation` indicator properties:



Indicator History

To get the historical data of the `Correlation` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class CorrelationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._c = self.c(self._symbol, self._reference, 20, CorrelationType.PEARSON)

        indicator_history = self.indicator_history(self._c, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._c, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._c, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Covariance

Introduction

This indicator computes the Covariance of two assets using the given Look-Back period. The Covariance of two assets is a measure of their co-movement.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using COV Indicator

To create an automatic indicator for `Covariance` , call the `cov` helper method from the `QCAlgorithm` class. The `cov` method creates a `Covariance` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class CovarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cov = self.cov([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._cov.is_ready:
            # The current value of self._cov is represented by self._cov.current.value
            self.plot("Covariance", "cov", self._cov.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Covariance` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

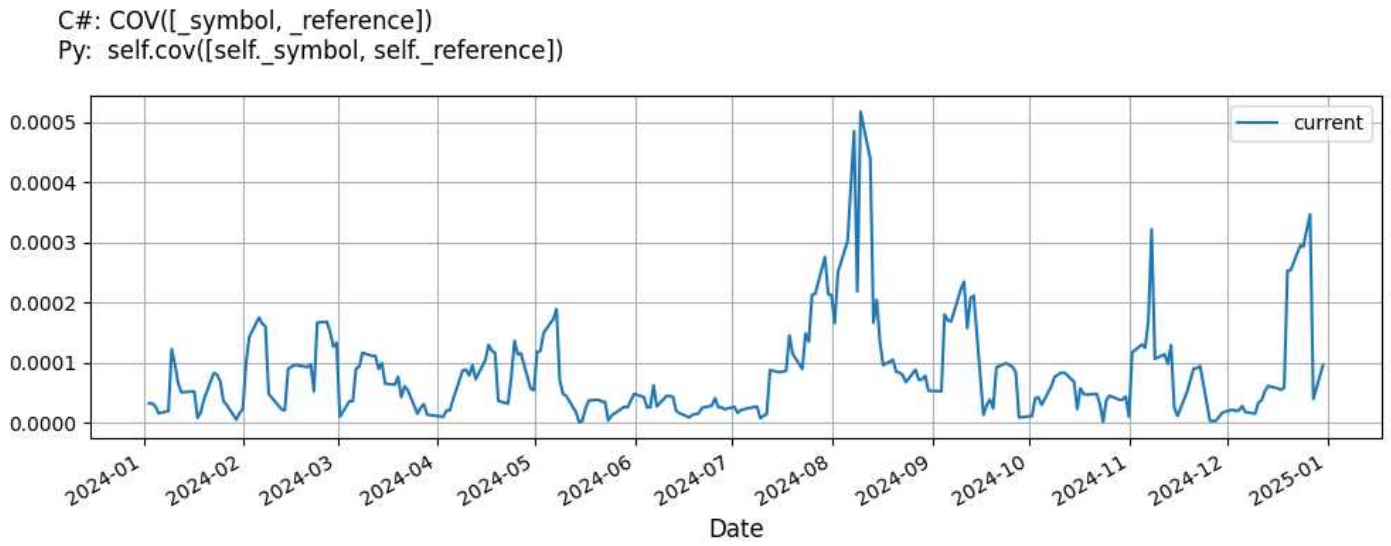
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class CovarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._covariance = Covariance(self._symbol, self._reference, 5)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._covariance.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._covariance.update(bar)

        if self._covariance.is_ready:
            # The current value of self._covariance is represented by self._covariance.current.value
            self.plot("Covariance", "covariance", self._covariance.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Covariance` indicator properties:



Indicator History

To get the historical data of the `Covariance` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class CovarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._cov = self.cov([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._cov, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._cov, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._cov, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

DeMarker Indicator

Introduction

In the DeMarker strategy, for some period of size N, set: DeMax = High - Previous High, and DeMin = Previous Low - Low where, in the prior, if either term is less than zero (DeMax or DeMin), set it to zero. We can now define the indicator itself, DEM, as: DEM = MA(DeMax)/(MA(DeMax)+MA(DeMin)) where MA denotes a Moving Average of period N.
https://www.investopedia.com/terms/d/demarkerindicator.asp

To view the implementation of this indicator, see the LEAN GitHub repository .

Using DEM Indicator

To create an automatic indicator for `DeMarkerIndicator` , call the `dem` helper method from the `QCAlgorithm` class. The `dem` method creates a `DeMarkerIndicator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class DeMarkerIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dem = self.dem(self._symbol, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._dem.is_ready:
            # The current value of self._dem is represented by self._dem.current.value
            self.plot("DeMarkerIndicator", "dem", self._dem.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `DeMarkerIndicator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

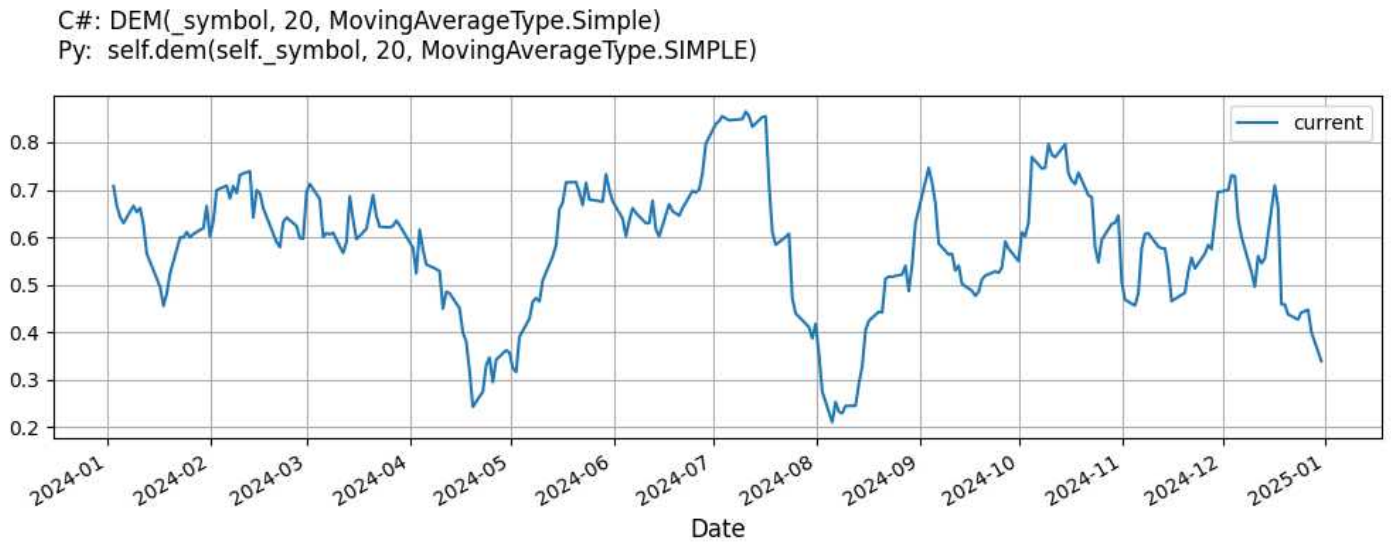
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class DeMarkerIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._demarkerindicator = DeMarkerIndicator(20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._demarkerindicator.update(bar)

        if self._demarkerindicator.is_ready:
            # The current value of self._demarkerindicator is represented by self._demarkerindicator.current.value
            self.plot("DeMarkerIndicator", "demarkerindicator", self._demarkerindicator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `DeMarkerIndicator` indicator properties:



Indicator History

To get the historical data of the `DeMarkerIndicator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class DeMarkerIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dem = self.dem(self._symbol, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._dem, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dem, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dem, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Delta

Introduction

Option Delta indicator that calculate the delta of an option

To view the implementation of this indicator, see the LEAN GitHub repository .

Using D Indicator

To create an automatic indicator for `Delta` , call the `d` helper method from the `QCAlgorithm` class. The `d` method creates a `Delta` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```py
class DeltaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._d = self.d(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._d.is_ready:
            # The current value of self._d is represented by self._d.current.value
            self.plot("Delta", "d", self._d.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Delta` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

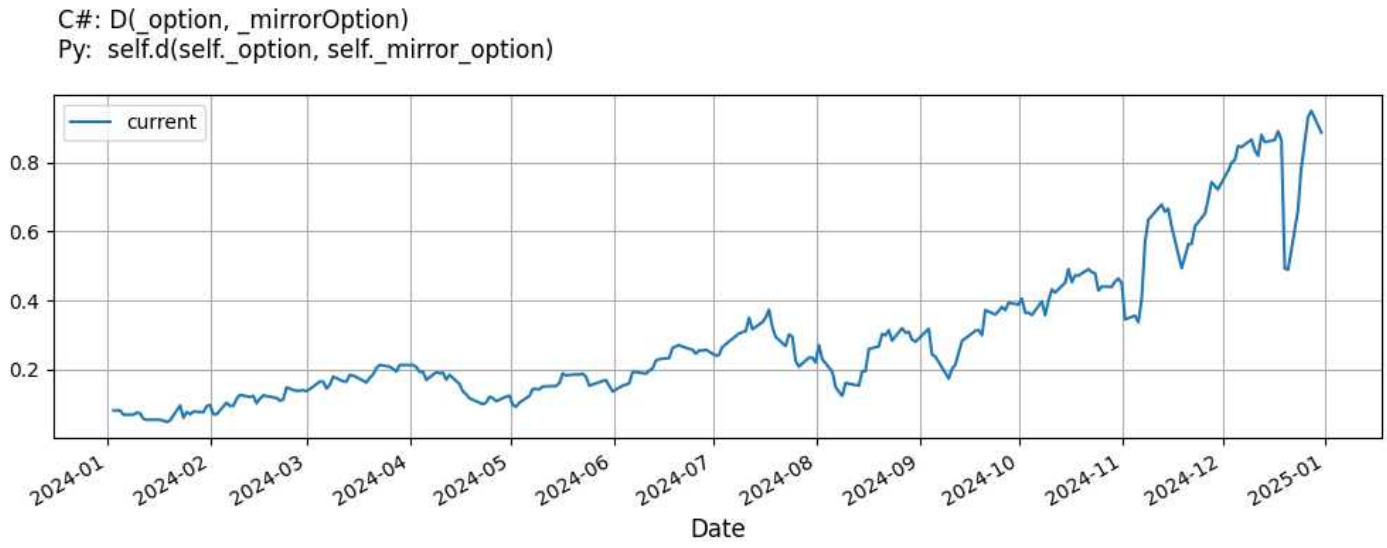
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```py
class DeltaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._delta = Delta(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._delta.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._delta.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._delta.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._delta.is_ready:
            # The current value of self._delta is represented by self._delta.current.value
            self.plot("Delta", "delta", self._delta.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Delta` indicator properties:



Indicator History

To get the historical data of the `Delta` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DeltaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._d = self.d(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._d, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._d, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._d, [self._symbol, self._option, self._mirror_option], datetime(2024,
7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Derivative Oscillator

Introduction

This indicator represents the Derivative Oscillator Indicator, utilizing a moving average convergence-divergence (MACD) histogram to a double-smoothed relative strength index (RSI).

To view the implementation of this indicator, see the LEAN GitHub repository .

Using DO Indicator

To create an automatic indicator for `DerivativeOscillator` , call the `do` helper method from the `QCAlgorithm` class. The `do` method creates a `DerivativeOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class DerivativeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._do = self.do(self._symbol, 14, 5, 3, 9)

    def on_data(self, slice: Slice) -> None:

        if self._do.is_ready:
            # The current value of self._do is represented by self._do.current.value
            self.plot("DerivativeOscillator", "do", self._do.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `DerivativeOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

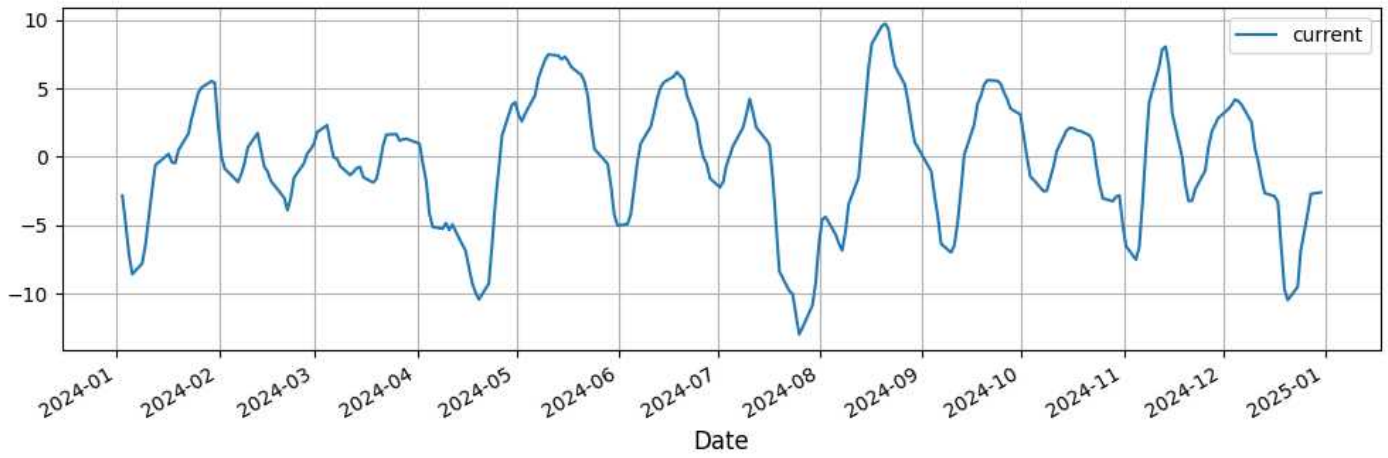
```
class DerivativeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._derivativeoscillator = DerivativeOscillator(14, 5, 3, 9)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._derivativeoscillator.update(bar.end_time, bar.close)

        if self._derivativeoscillator.is_ready:
            # The current value of self._derivativeoscillator is represented by self._derivativeoscillator.current.value
            self.plot("DerivativeOscillator", "derivativeoscillator", self._derivativeoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `DerivativeOscillator` indicator properties:

C#: DO(_symbol, 14, 5, 3, 9)
Py: self.do(self._symbol, 14, 5, 3, 9)



Indicator History

To get the historical data of the `DerivativeOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class DerivativeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._do = self.do(self._symbol, 14, 5, 3, 9)

        indicator_history = self.indicator_history(self._do, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._do, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._do, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Detrended Price Oscillator

Introduction

The Detrended Price Oscillator is an indicator designed to remove trend from price and make it easier to identify cycles. DPO does not extend to the last date because it is based on a displaced moving average. Is estimated as Price {X/2 + 1} periods ago less the X-period simple moving average. E.g.DPO(20) equals price 11 days ago less the 20-day SMA.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using DPO Indicator

To create an automatic indicator for `DetrendedPriceOscillator` , call the `dpo` helper method from the `QCAlgorithm` class. The `dpo` method creates a `DetrendedPriceOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class DetrendedPriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dpo = self.dpo(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._dpo.is_ready:
            # The current value of self._dpo is represented by self._dpo.current.value
            self.plot("DetrendedPriceOscillator", "dpo", self._dpo.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `DetrendedPriceOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

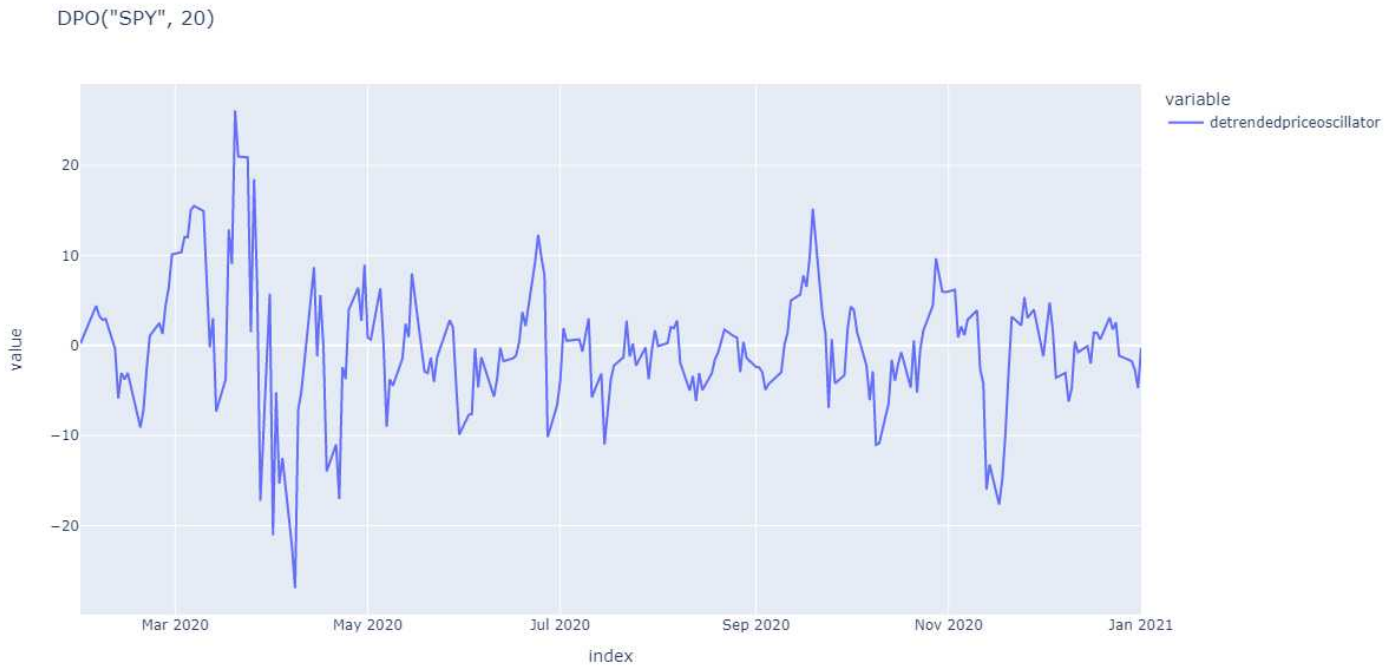
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class DetrendedPriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._detrendedpriceoscillator = DetrendedPriceOscillator(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._detrendedpriceoscillator.update(bar.end_time, bar.close)

        if self._detrendedpriceoscillator.is_ready:
            # The current value of self._detrendedpriceoscillator is represented by self._detrendedpriceoscillator.current.value
            self.plot("DetrendedPriceOscillator", "detrendedpriceoscillator", self._detrendedpriceoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `DetrendedPriceOscillator` indicator properties:



Indicator History

To get the historical data of the `DetrendedPriceOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class DetrendedPriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dpo = self.dpo(self._symbol, 20)

        indicator_history = self.indicator_history(self._dpo, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dpo, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dpo, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Donchian Channel

Introduction

This indicator computes the upper and lower band of the Donchian Channel. The upper band is computed by finding the highest high over the given period. The lower band is computed by finding the lowest low over the given period. The primary output value of the indicator is the mean of the upper and lower band for the given timeframe.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using DCH Indicator

To create an automatic indicator for `DonchianChannel` , call the `dch` helper method from the `QCAlgorithm` class. The `dch` method creates a `DonchianChannel` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class DonchianChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dch = self.dch(self._symbol, 20, 20)

    def on_data(self, slice: Slice) -> None:

        if self._dch.is_ready:
            # The current value of self._dch is represented by self._dch.current.value
            self.plot("DonchianChannel", "dch", self._dch.current.value)
            # Plot all attributes of self._dch
            self.plot("DonchianChannel", "upper_band", self._dch.upper_band.current.value)
            self.plot("DonchianChannel", "lower_band", self._dch.lower_band.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `DonchianChannel` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

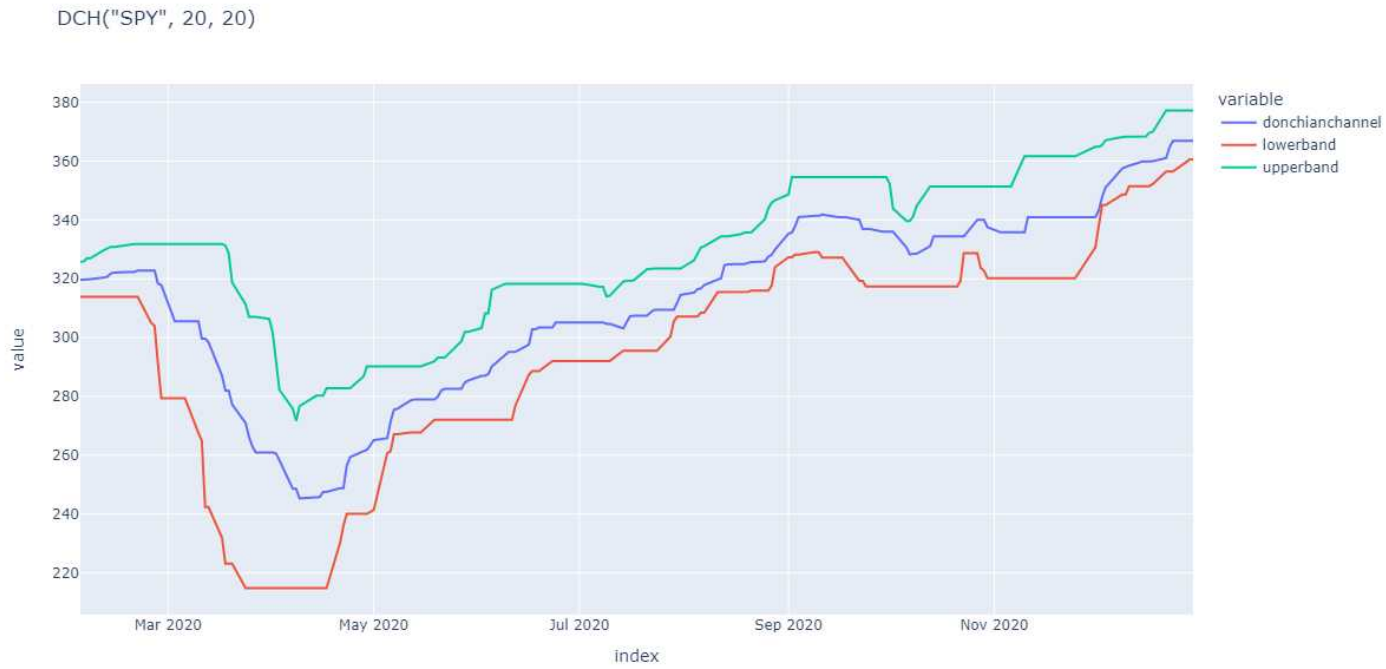
```
class DonchianChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._donchianchannel = DonchianChannel(20, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._donchianchannel.update(bar)

        if self._donchianchannel.is_ready:
            # The current value of self._donchianchannel is represented by self._donchianchannel.current.value
            self.plot("DonchianChannel", "donchianchannel", self._donchianchannel.current.value)
            # Plot all attributes of self._donchianchannel
            self.plot("DonchianChannel", "upper_band", self._donchianchannel.upper_band.current.value)
            self.plot("DonchianChannel", "lower_band", self._donchianchannel.lower_band.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `DonchianChannel` indicator properties:



Indicator History

To get the historical data of the `DonchianChannel` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class DonchianChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dch = self.dch(self._symbol, 20, 20)

        indicator_history = self.indicator_history(self._dch, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dch, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dch, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        upper_band = indicator_history_df["upperband"]
        lower_band = indicator_history_df["lowerband"]
```

Supported Indicators

Double Exponential Moving Average

Introduction

This indicator computes the Double Exponential Moving Average (DEMA). The Double Exponential Moving Average is calculated with the following formula: EMA2 = EMA(EMA(t,period),period) DEMA = 2 * EMA(t,period) - EMA2 The Generalized DEMA (GD) is calculated with the following formula: GD = (volumeFactor+1) * EMA(t,period) - volumeFactor * EMA2

To view the implementation of this indicator, see the LEAN GitHub repository .

Using DEMA Indicator

To create an automatic indicator for `DoubleExponentialMovingAverage` , call the `dema` helper method from the `QCAlgorithm` class. The `dema` method creates a `DoubleExponentialMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class DoubleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dema = self.dema(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._dema.is_ready:
            # The current value of self._dema is represented by self._dema.current.value
            self.plot("DoubleExponentialMovingAverage", "dema", self._dema.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `DoubleExponentialMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

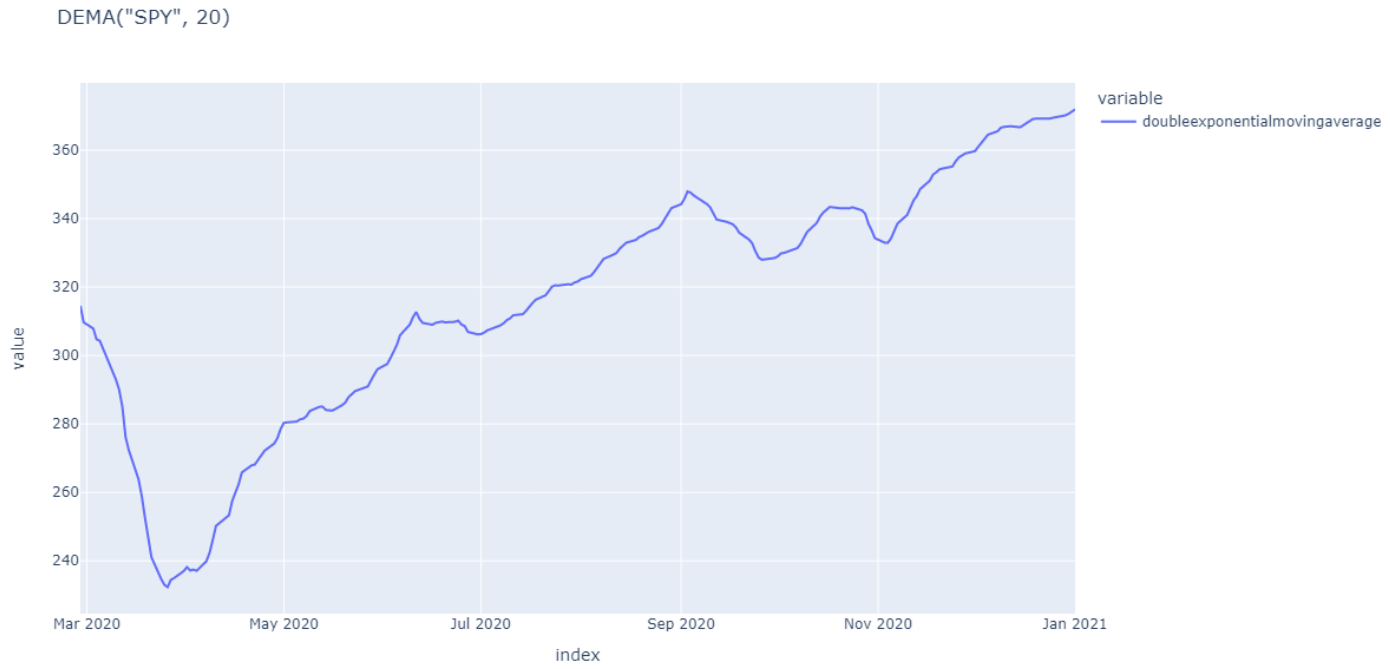
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class DoubleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._doubleexponentialmovingaverage = DoubleExponentialMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._doubleexponentialmovingaverage.update(bar.end_time, bar.close)

        if self._doubleexponentialmovingaverage.is_ready:
            # The current value of self._doubleexponentialmovingaverage is represented by
self._doubleexponentialmovingaverage.current.value
            self.plot("DoubleExponentialMovingAverage", "doubleexponentialmovingaverage",
self._doubleexponentialmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `DoubleExponentialMovingAverage` indicator properties:



Indicator History

To get the historical data of the `DoubleExponentialMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class DoubleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._dema = self.dema(self._symbol, 20)

        indicator_history = self.indicator_history(self._dema, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._dema, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._dema, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Ease Of Movement Value

Introduction

This indicator computes the n-period Ease of Movement Value using the following: MID = (high_1 + low_1)/2 - (high_0 + low_0)/2 RATIO = (currentVolume/10000) / (high_1 - low_1) EMV = MID/RATIO _SMA = n-period of EMV Returns _SMA source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using EMV Indicator

To create an automatic indicator for `EaseOfMovementValue` , call the `emv` helper method from the `QCAlgorithm` class. The `emv` method creates a `EaseOfMovementValue` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class EaseOfMovementValueAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._emv = self.emv(self._symbol, 1, 10000)

    def on_data(self, slice: Slice) -> None:

        if self._emv.is_ready:
            # The current value of self._emv is represented by self._emv.current.value
            self.plot("EaseOfMovementValue", "emv", self._emv.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `EaseOfMovementValue` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

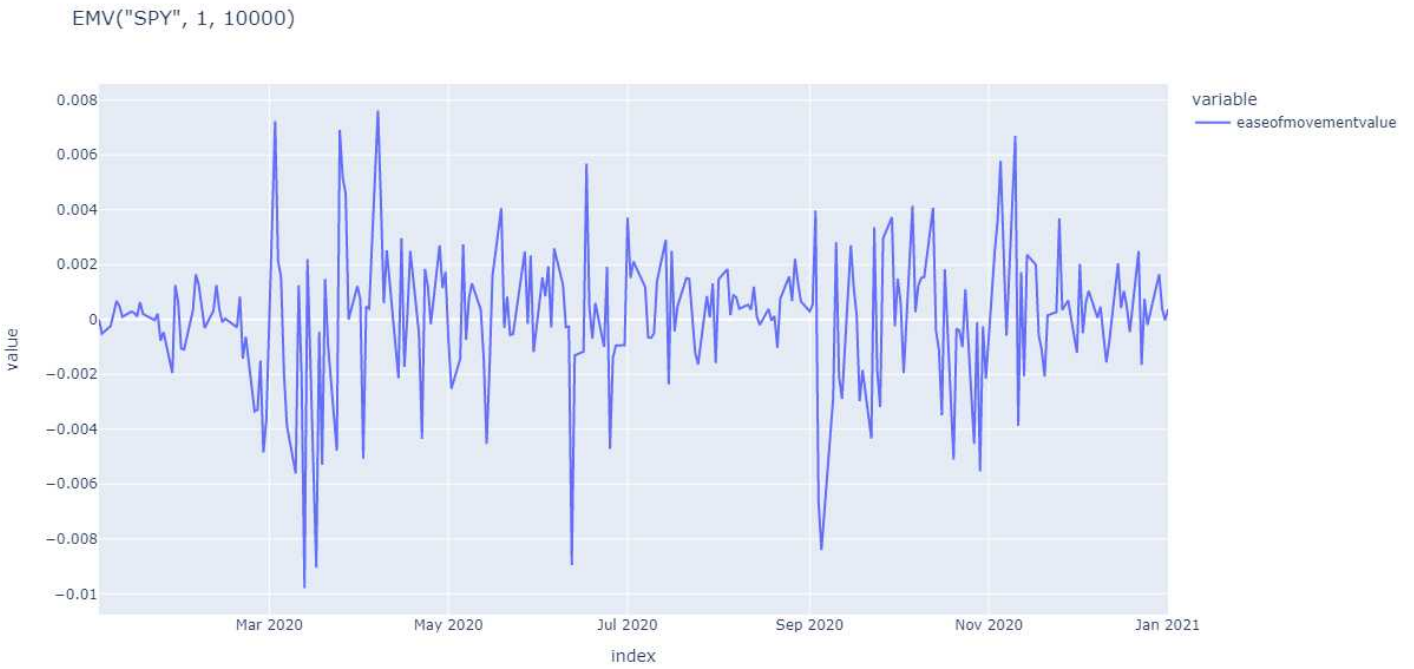
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class EaseOfMovementValueAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._easeofmovementvalue = EaseOfMovementValue(1, 10000)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._easeofmovementvalue.update(bar)

        if self._easeofmovementvalue.is_ready:
            # The current value of self._easeofmovementvalue is represented by self._easeofmovementvalue.current.value
            self.plot("EaseOfMovementValue", "easeofmovementvalue", self._easeofmovementvalue.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `EaseOfMovementValue` indicator properties:



Indicator History

To get the historical data of the `EaseOfMovementValue` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class EaseOfMovementValueAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._emv = self.emv(self._symbol, 1, 10000)

        indicator_history = self.indicator_history(self._emv, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._emv, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._emv, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Exponential Moving Average

Introduction

This indicator represents the traditional exponential moving average indicator (EMA). When the indicator is ready, the first value of the EMA is equivalent to the simple moving average. After the first EMA value, the EMA value is a function of the previous EMA value. Therefore, depending on the number of samples you feed into the indicator, it can provide different EMA values for a single security and lookback period. To make the indicator values consistent across time, warm up the indicator with all the trailing security price history.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using EMA Indicator

To create an automatic indicator for `ExponentialMovingAverage` , call the `ema` helper method from the `QCAlgorithm` class. The `ema` method creates a `ExponentialMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

PY

```
class ExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ema = self.ema(self._symbol, 20, 0.5)

    def on_data(self, slice: Slice) -> None:

        if self._ema.is_ready:
            # The current value of self._ema is represented by self._ema.current.value
            self.plot("ExponentialMovingAverage", "ema", self._ema.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ExponentialMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

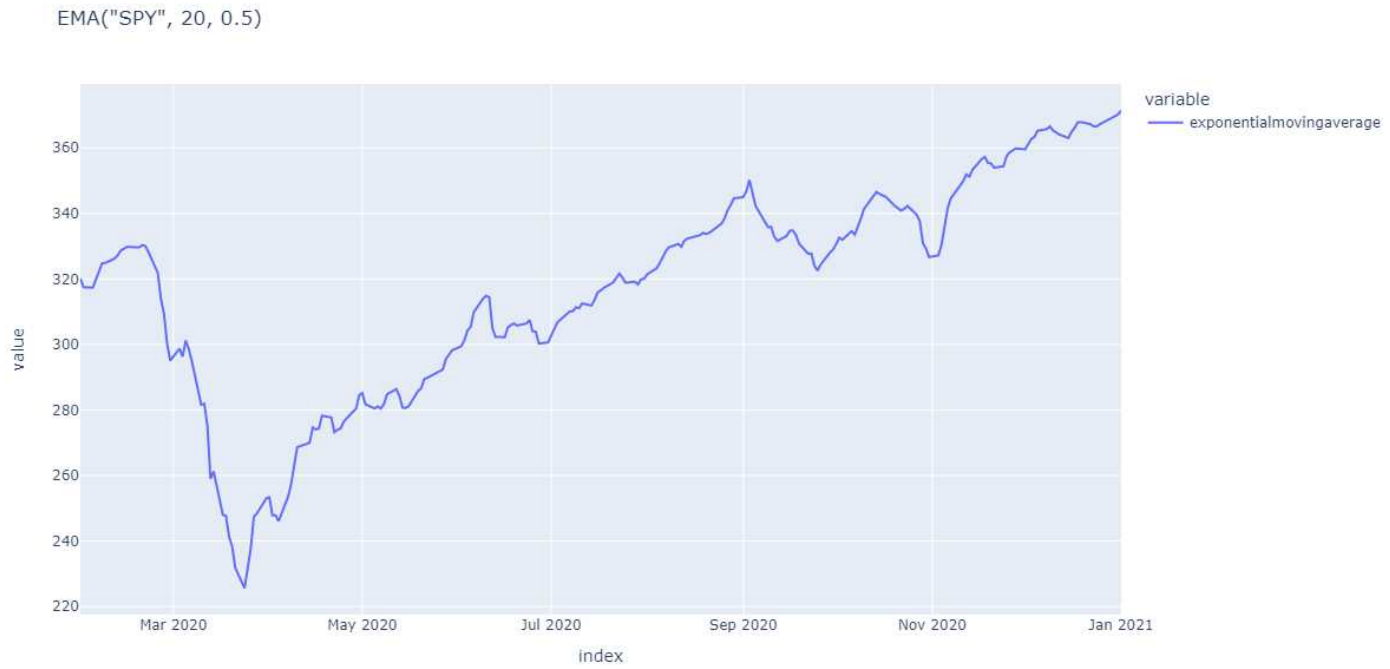
PY

```
class ExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._exponentialmovingaverage = ExponentialMovingAverage(20, 0.5)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._exponentialmovingaverage.update(bar.end_time, bar.close)

        if self._exponentialmovingaverage.is_ready:
            # The current value of self._exponentialmovingaverage is represented by self._exponentialmovingaverage.current.value
            self.plot("ExponentialMovingAverage", "exponentialmovingaverage", self._exponentialmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ExponentialMovingAverage` indicator properties:



Indicator History

To get the historical data of the `ExponentialMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class ExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ema = self.ema(self._symbol, 20, 0.5)

        indicator_history = self.indicator_history(self._ema, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ema, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ema, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Filtered Identity

Introduction

This indicator represents an indicator that is a ready after ingesting a single sample and always returns the same value as it is given if it passes a filter condition

To view the implementation of this indicator, see the LEAN GitHub repository .

Using FilteredIdentity Indicator

To create an automatic indicator for `FilteredIdentity` , call the `filtered_identity` helper method from the `QCAlgorithm` class. The `filtered_identity` method creates a `FilteredIdentity` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class FilteredIdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._filtered_identity = self.filtered_identity(self._symbol, filter=lambda x: x.close > x.open)

    def on_data(self, slice: Slice) -> None:

        if self._filtered_identity.is_ready:
            # The current value of self._filtered_identity is represented by self._filtered_identity.current.value
            self.plot("FilteredIdentity", "filtered_identity", self._filtered_identity.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `FilteredIdentity` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class FilteredIdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._filteredidentity = FilteredIdentity(filter=lambda x: x.close > x.open)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._filteredidentity.update(bar)

        if self._filteredidentity.is_ready:
            # The current value of self._filteredidentity is represented by self._filteredidentity.current.value
            self.plot("FilteredIdentity", "filteredidentity", self._filteredidentity.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `FilteredIdentity` indicator properties:

Indicator History

To get the historical data of the `FilteredIdentity` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class FilteredIdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._filtered_identity = self.filtered_identity(self._symbol, filter=lambda x: x.close > x.open)

        indicator_history = self.indicator_history(self._filtered_identity, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._filtered_identity, self._symbol, timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._filtered_identity, self._symbol, datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Fisher Transform

Introduction

The Fisher transform is a mathematical process which is used to convert any data set to a modified data set whose Probability Distribution Function is approximately Gaussian. Once the Fisher transform is computed, the transformed data can then be analyzed in terms of it's deviation from the mean. The equation is y = .5 * ln [1 + x / 1 - x] where x is the input y is the output ln is the natural logarithm The Fisher transform has much sharper turning points than other indicators such as MACD For more info, read chapter 1 of Cybernetic Analysis for Stocks and Futures by John F. Ehlers We are implementing the latest version of this indicator found at Fig. 4 of http://www.mesasoftware.com/papers/UsingTheFisherTransform.pdf

To view the implementation of this indicator, see the LEAN GitHub repository .

Using FISH Indicator

To create an automatic indicator for `FisherTransform` , call the `fish` helper method from the `QCAlgorithm` class. The `fish` method creates a `FisherTransform` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

PY

```
class FisherTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fish = self.fish(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._fish.is_ready:
            # The current value of self._fish is represented by self._fish.current.value
            self.plot("FisherTransform", "fish", self._fish.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `FisherTransform` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

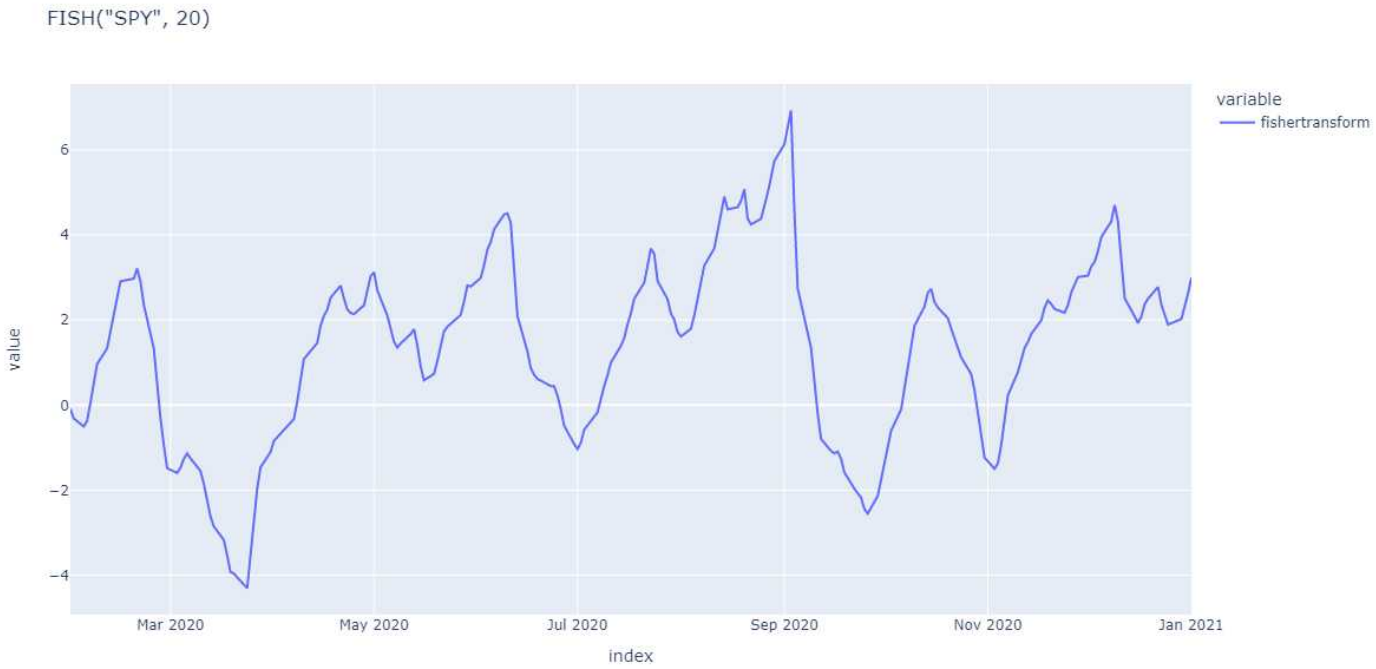
PY

```
class FisherTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fishertransform = FisherTransform(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._fishertransform.update(bar)

        if self._fishertransform.is_ready:
            # The current value of self._fishertransform is represented by self._fishertransform.current.value
            self.plot("FisherTransform", "fishertransform", self._fishertransform.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `FisherTransform` indicator properties:



Indicator History

To get the historical data of the `FisherTransform` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class FisherTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fish = self.fish(self._symbol, 20)

        indicator_history = self.indicator_history(self._fish, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._fish, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._fish, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Force Index

Introduction

The Force Index is calculated by comparing the current market price with the previous market price and multiplying its difference with the traded volume during a specific time period.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using FI Indicator

To create an automatic indicator for `ForceIndex` , call the `fi` helper method from the `QCAlgorithm` class. The `fi` method creates a `ForceIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ForceIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fi = self.fi(self._symbol, 13)

    def on_data(self, slice: Slice) -> None:

        if self._fi.is_ready:
            # The current value of self._fi is represented by self._fi.current.value
            self.plot("ForceIndex", "fi", self._fi.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ForceIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

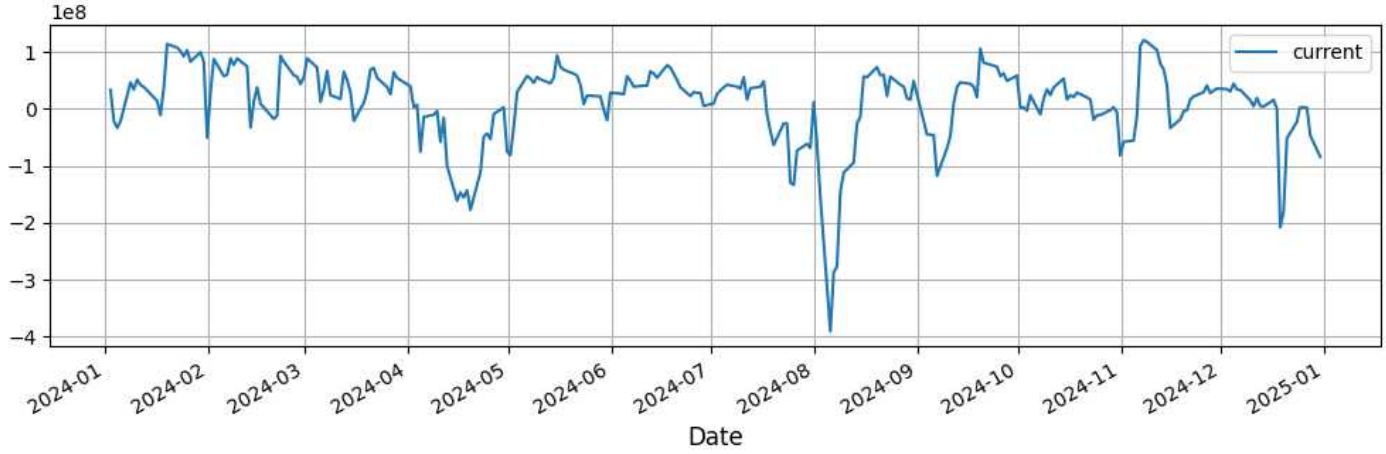
```
PY
class ForceIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._forceindex = ForceIndex(13)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._forceindex.update(bar)

        if self._forceindex.is_ready:
            # The current value of self._forceindex is represented by self._forceindex.current.value
            self.plot("ForceIndex", "forceindex", self._forceindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ForceIndex` indicator properties:

C#: FI(_symbol, 13)
Py: self.fi(self._symbol, 13)



Indicator History

To get the historical data of the `ForceIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class ForceIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fi = self.fi(self._symbol, 13)

        indicator_history = self.indicator_history(self._fi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._fi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._fi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Fractal Adaptive Moving Average

Introduction

The Fractal Adaptive Moving Average (FRAMA) by John Ehlers

To view the implementation of this indicator, see the LEAN GitHub repository .

Using FRAMA Indicator

To create an automatic indicator for `FractalAdaptiveMovingAverage` , call the `frama` helper method from the `QCAlgorithm` class. The `frama` method creates a `FractalAdaptiveMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class FractalAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._frama = self.frama(self._symbol, 20, 198)

    def on_data(self, slice: Slice) -> None:

        if self._frama.is_ready:
            # The current value of self._frama is represented by self._frama.current.value
            self.plot("FractalAdaptiveMovingAverage", "frama", self._frama.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `FractalAdaptiveMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

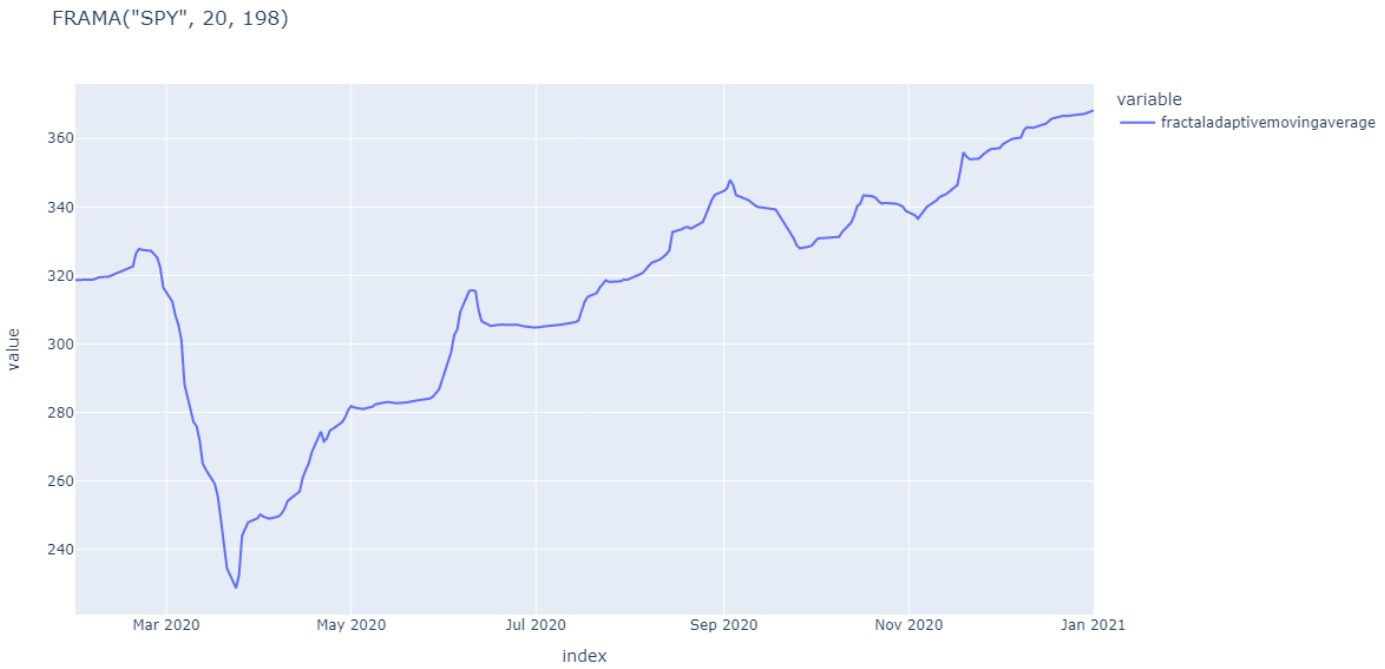
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class FractalAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fractaladaptivemovingaverage = FractalAdaptiveMovingAverage(20, 198)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._fractaladaptivemovingaverage.update(bar)

        if self._fractaladaptivemovingaverage.is_ready:
            # The current value of self._fractaladaptivemovingaverage is represented by self._fractaladaptivemovingaverage.current.value
            self.plot("FractalAdaptiveMovingAverage", "fractaladaptivemovingaverage", self._fractaladaptivemovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `FractalAdaptiveMovingAverage` indicator properties:



Indicator History

To get the historical data of the `FractalAdaptiveMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class FractalAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._frama = self.frama(self._symbol, 20, 198)

        indicator_history = self.indicator_history(self._frama, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._frama, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._frama, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Gamma

Introduction

Option Gamma indicator that calculate the gamma of an option

To view the implementation of this indicator, see the LEAN GitHub repository .

Using G Indicator

To create an automatic indicator for `Gamma` , call the `g` helper method from the `QCAlgorithm` class. The `g` method creates a `Gamma` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```py
class GammaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._g = self.g(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._g.is_ready:
            # The current value of self._g is represented by self._g.current.value
            self.plot("Gamma", "g", self._g.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Gamma` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

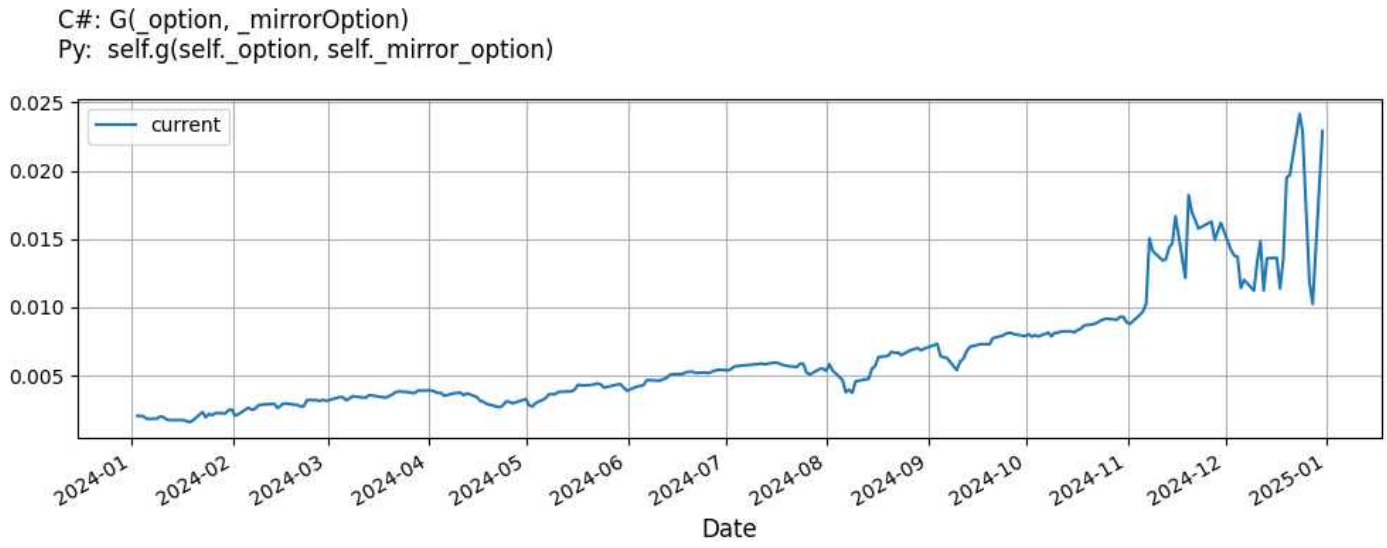
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class GammaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._gamma = Gamma(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._gamma.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._gamma.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._gamma.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._gamma.is_ready:
            # The current value of self._gamma is represented by self._gamma.current.value
            self.plot("Gamma", "gamma", self._gamma.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Gamma` indicator properties:



Indicator History

To get the historical data of the `Gamma` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class GammaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._g = self.g(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._g, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._g, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._g, [self._symbol, self._option, self._mirror_option], datetime(2024,
7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Heikin Ashi

Introduction

This indicator computes the Heikin-Ashi bar (HA) The Heikin-Ashi bar is calculated using the following formulas: HA_Close[0] = (Open[0] + High[0] + Low[0] + Close[0]) / 4 HA_Open[0] = (HA_Open[1] + HA_Close[1]) / 2 HA_High[0] = MAX(High[0], HA_Open[0], HA_Close[0]) HA_Low[0] = MIN(Low[0], HA_Open[0], HA_Close[0])

To view the implementation of this indicator, see the LEAN GitHub repository .

Using HeikinAshi Indicator

To create an automatic indicator for `HeikinAshi` , call the `heikin_ashi` helper method from the `QCAlgorithm` class. The `heikin_ashi` method creates a `HeikinAshi` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HeikinAshiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._heikin_ashi = self.heikin_ashi(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._heikin_ashi.is_ready:
            # The current value of self._heikin_ashi is represented by self._heikin_ashi.current.value
            self.plot("HeikinAshi", "heikin_ashi", self._heikin_ashi.current.value)
            # Plot all attributes of self._heikin_ashi
            self.plot("HeikinAshi", "open", self._heikin_ashi.open.current.value)
            self.plot("HeikinAshi", "high", self._heikin_ashi.high.current.value)
            self.plot("HeikinAshi", "low", self._heikin_ashi.low.current.value)
            self.plot("HeikinAshi", "close", self._heikin_ashi.close.current.value)
            self.plot("HeikinAshi", "volume", self._heikin_ashi.volume.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `HeikinAshi` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HeikinAshiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._heikinashi = HeikinAshi()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._heikinashi.update(bar)

        if self._heikinashi.is_ready:
            # The current value of self._heikinashi is represented by self._heikinashi.current.value
            self.plot("HeikinAshi", "heikinashi", self._heikinashi.current.value)
            # Plot all attributes of self._heikinashi
            self.plot("HeikinAshi", "open", self._heikinashi.open.current.value)
            self.plot("HeikinAshi", "high", self._heikinashi.high.current.value)
            self.plot("HeikinAshi", "low", self._heikinashi.low.current.value)
            self.plot("HeikinAshi", "close", self._heikinashi.close.current.value)
            self.plot("HeikinAshi", "volume", self._heikinashi.volume.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `HeikinAshi` indicator properties:

Indicator History

To get the historical data of the `HeikinAshi` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class HeikinAshiAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._heikin_ashi = self.heikin_ashi(self._symbol)

        indicator_history = self.indicator_history(self._heikin_ashi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._heikin_ashi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._heikin_ashi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7,
5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        open = indicator_history_df["open"]
        high = indicator_history_df["high"]
        low = indicator_history_df["low"]
        close = indicator_history_df["close"]
        volume = indicator_history_df["volume"]
```

Supported Indicators

Hilbert Transform

Introduction

This indicator computes the Hilbert Transform Indicator by John Ehlers. By using present and prior price differences, and some feedback, price values are split into their complex number components of real (inPhase) and imaginary (quadrature) parts. source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using HT Indicator

To create an automatic indicator for `HilbertTransform` , call the `ht` helper method from the `QCAlgorithm` class. The `ht` method creates a `HilbertTransform` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HilbertTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ht = self.ht(self._symbol, 7, 0.635, 0.338)

    def on_data(self, slice: Slice) -> None:

        if self._ht.is_ready:
            # The current value of self._ht is represented by self._ht.current.value
            self.plot("HilbertTransform", "ht", self._ht.current.value)
            # Plot all attributes of self._ht
            self.plot("HilbertTransform", "in_phase", self._ht.in_phase.current.value)
            self.plot("HilbertTransform", "quadrature", self._ht.quadrature.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `HilbertTransform` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HilbertTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hilberttransform = HilbertTransform(7, 0.635, 0.338)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hilberttransform.update(bar.end_time, bar.close)

        if self._hilberttransform.is_ready:
            # The current value of self._hilberttransform is represented by self._hilberttransform.current.value
            self.plot("HilbertTransform", "hilberttransform", self._hilberttransform.current.value)
            # Plot all attributes of self._hilberttransform
            self.plot("HilbertTransform", "in_phase", self._hilberttransform.in_phase.current.value)
            self.plot("HilbertTransform", "quadrature", self._hilberttransform.quadrature.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `HilbertTransform` indicator properties:

Indicator History

To get the historical data of the `HilbertTransform` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class HilbertTransformAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ht = self.ht(self._symbol, 7, 0.635, 0.338)

        indicator_history = self.indicator_history(self._ht, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ht, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ht, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        in_phase = indicator_history_df["inphase"]
        quadrature = indicator_history_df["quadrature"]
```

Supported Indicators

Hull Moving Average

Introduction

Produces a Hull Moving Average as explained at http://www.alanhull.com/hull-moving-average/ and derived from the instructions for the Excel VBA code at http://finance4traders.blogspot.com/2009/06/how-to-calculate-hull-moving-average.html

To view the implementation of this indicator, see the LEAN GitHub repository .

Using HMA Indicator

To create an automatic indicator for `HullMovingAverage` , call the `hma` helper method from the `QCAlgorithm` class. The `hma` method creates a `HullMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class HullMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hma = self.hma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._hma.is_ready:
            # The current value of self._hma is represented by self._hma.current.value
            self.plot("HullMovingAverage", "hma", self._hma.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `HullMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

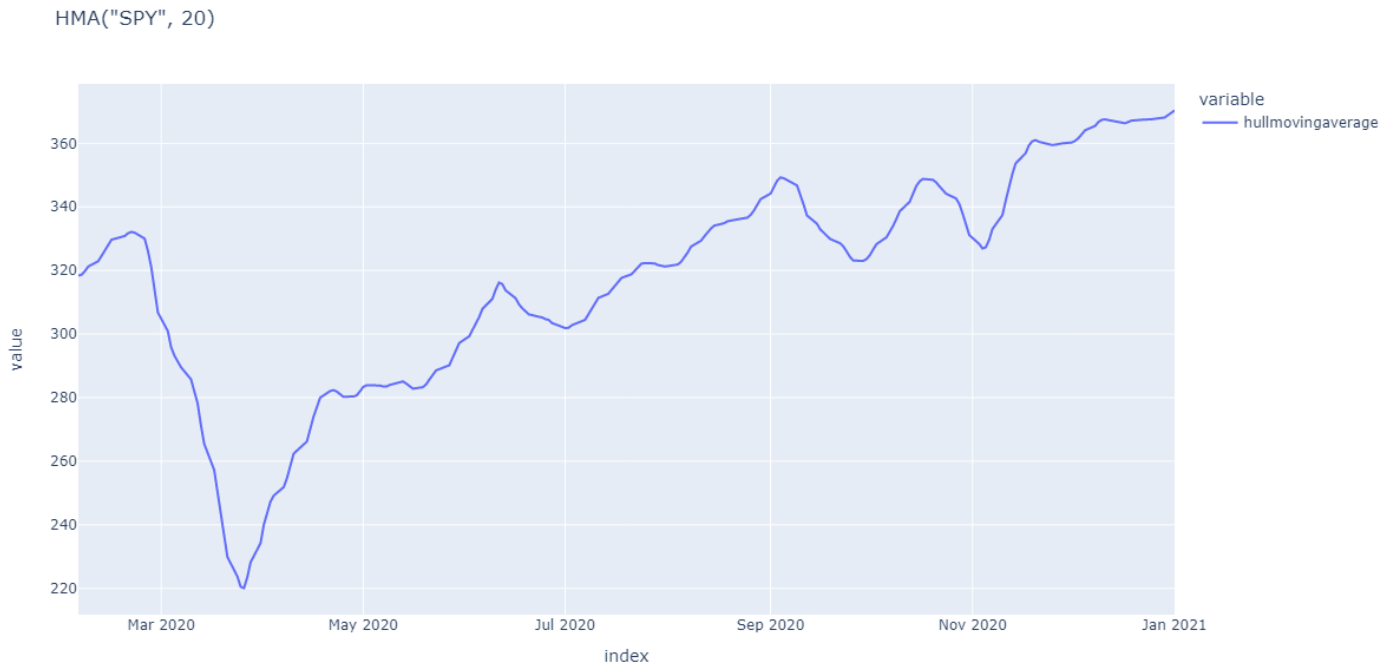
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class HullMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hullmovingaverage = HullMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hullmovingaverage.update(bar.end_time, bar.close)

        if self._hullmovingaverage.is_ready:
            # The current value of self._hullmovingaverage is represented by self._hullmovingaverage.current.value
            self.plot("HullMovingAverage", "hullmovingaverage", self._hullmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `HullMovingAverage` indicator properties:



Indicator History

To get the historical data of the `HullMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HullMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hma = self.hma(self._symbol, 20)

        indicator_history = self.indicator_history(self._hma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._hma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._hma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Hurst Exponent

Introduction

This indicator represents the Hurst Exponent indicator, which is used to measure the long-term memory of a time series. - H less than 0.5: Mean-reverting; high values followed by low ones, stronger as H approaches 0. - H equal to 0.5: Random walk (geometric). - H greater than 0.5: Trending; high values followed by higher ones, stronger as H approaches 1.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using HE Indicator

To create an automatic indicator for `HurstExponent` , call the `he` helper method from the `QCAlgorithm` class. The `he` method creates a `HurstExponent` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class HurstExponentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._he = self.he(self._symbol, 32)

    def on_data(self, slice: Slice) -> None:

        if self._he.is_ready:
            # The current value of self._he is represented by self._he.current.value
            self.plot("HurstExponent", "he", self._he.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `HurstExponent` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

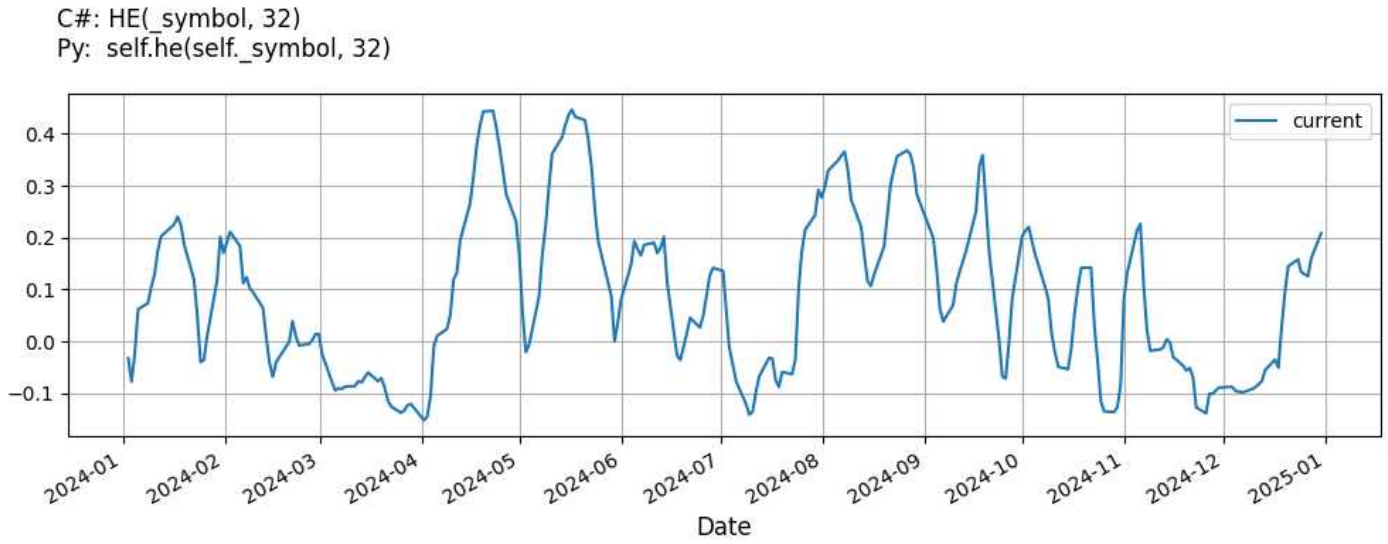
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class HurstExponentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._hurstexponent = HurstExponent(32)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._hurstexponent.update(bar.end_time, bar.close)

        if self._hurstexponent.is_ready:
            # The current value of self._hurstexponent is represented by self._hurstexponent.current.value
            self.plot("HurstExponent", "hurstexponent", self._hurstexponent.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `HurstExponent` indicator properties:



Indicator History

To get the historical data of the `HurstExponent` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class HurstExponentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._he = self.he(self._symbol, 32)

        indicator_history = self.indicator_history(self._he, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._he, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._he, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Ichimoku Kinko Hyo

Introduction

This indicator computes the Ichimoku Kinko Hyo indicator. It consists of the following main indicators: Tenkan-sen: (Highest High + Lowest Low) / 2 for the specific period (normally 9) Kijun-sen: (Highest High + Lowest Low) / 2 for the specific period (normally 26) Senkou A Span: (Tenkan-sen + Kijun-sen)/ 2 from a specific number of periods ago (normally 26) Senkou B Span: (Highest High + Lowest Low) / 2 for the specific period (normally 52), from a specific number of periods ago (normally 26)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ICHIMOKU Indicator

To create an automatic indicator for `IchimokuKinkoHyo` , call the `ichimoku` helper method from the `QCAlgorithm` class. The `ichimoku` method creates a `IchimokuKinkoHyo` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class IchimokuKinkoHyoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ichimoku = self.ichimoku(self._symbol, 9, 26, 17, 52, 26, 26)

    def on_data(self, slice: Slice) -> None:

        if self._ichimoku.is_ready:
            # The current value of self._ichimoku is represented by self._ichimoku.current.value
            self.plot("IchimokuKinkoHyo", "ichimoku", self._ichimoku.current.value)
            # Plot all attributes of self._ichimoku
            self.plot("IchimokuKinkoHyo", "tenkan", self._ichimoku.tenkan.current.value)
            self.plot("IchimokuKinkoHyo", "kijun", self._ichimoku.kijun.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_a", self._ichimoku.senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b", self._ichimoku.senkou_b.current.value)
            self.plot("IchimokuKinkoHyo", "chikou", self._ichimoku.chikou.current.value)
            self.plot("IchimokuKinkoHyo", "tenkan_maximum", self._ichimoku.tenkan_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "tenkan_minimum", self._ichimoku.tenkan_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "kijun_maximum", self._ichimoku.kijun_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "kijun_minimum", self._ichimoku.kijun_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b_maximum", self._ichimoku.senkou_b_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b_minimum", self._ichimoku.senkou_b_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_tenkan_senkou_a", self._ichimoku.delayed_tenkan_senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_kijun_senkou_a", self._ichimoku.delayed_kijun_senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_maximum_senkou_b", self._ichimoku.delayed_maximum_senkou_b.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_minimum_senkou_b", self._ichimoku.delayed_minimum_senkou_b.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `IchimokuKinkoHyo` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

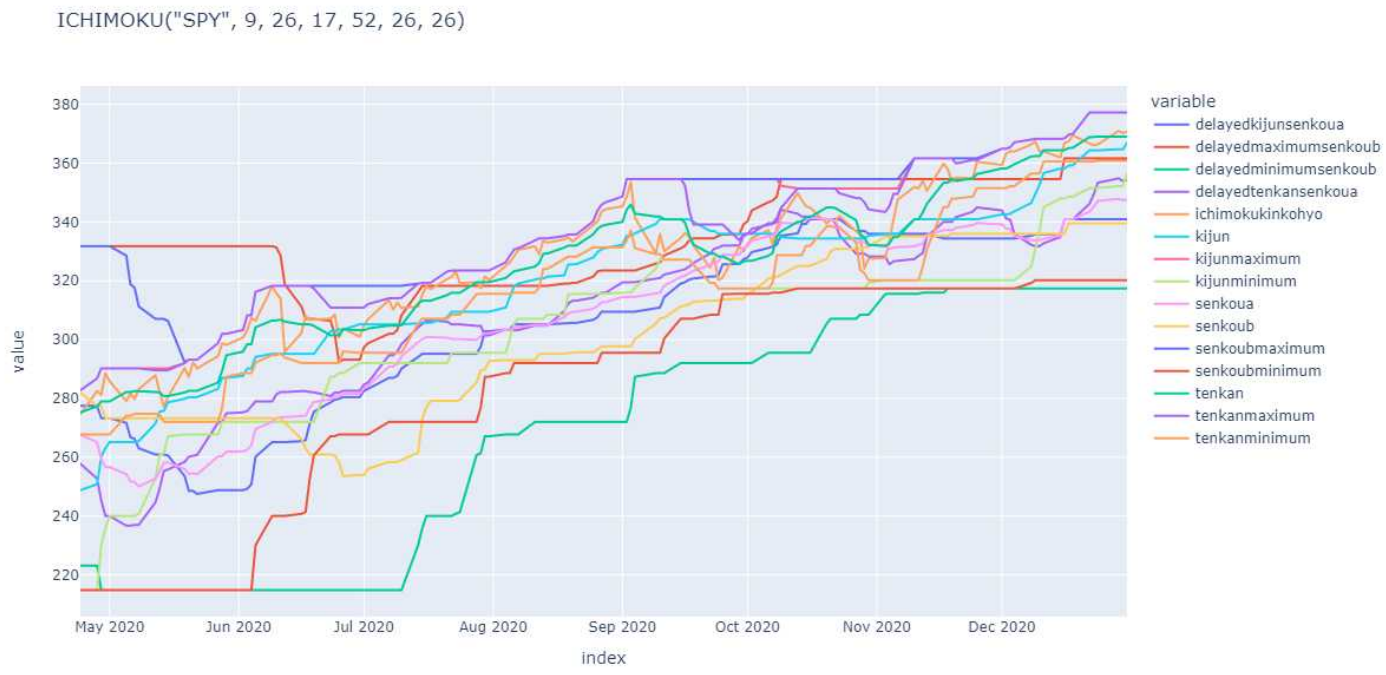
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class IchimokuKinkoHyoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ichimokukinkohyo = IchimokuKinkoHyo(9, 26, 17, 52, 26, 26)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._ichimokukinkohyo.update(bar)

        if self._ichimokukinkohyo.is_ready:
            # The current value of self._ichimokukinkohyo is represented by self._ichimokukinkohyo.current.value
            self.plot("IchimokuKinkoHyo", "ichimokukinkohyo", self._ichimokukinkohyo.current.value)
            # Plot all attributes of self._ichimokukinkohyo
            self.plot("IchimokuKinkoHyo", "tenkan", self._ichimokukinkohyo.tenkan.current.value)
            self.plot("IchimokuKinkoHyo", "kijun", self._ichimokukinkohyo.kijun.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_a", self._ichimokukinkohyo.senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b", self._ichimokukinkohyo.senkou_b.current.value)
            self.plot("IchimokuKinkoHyo", "chikou", self._ichimokukinkohyo.chikou.current.value)
            self.plot("IchimokuKinkoHyo", "tenkan_maximum", self._ichimokukinkohyo.tenkan_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "tenkan_minimum", self._ichimokukinkohyo.tenkan_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "kijun_maximum", self._ichimokukinkohyo.kijun_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "kijun_minimum", self._ichimokukinkohyo.kijun_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b_maximum", self._ichimokukinkohyo.senkou_b_maximum.current.value)
            self.plot("IchimokuKinkoHyo", "senkou_b_minimum", self._ichimokukinkohyo.senkou_b_minimum.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_tenkan_senkou_a", self._ichimokukinkohyo.delayed_tenkan_senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_kijun_senkou_a", self._ichimokukinkohyo.delayed_kijun_senkou_a.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_maximum_senkou_b", self._ichimokukinkohyo.delayed_maximum_senkou_b.current.value)
            self.plot("IchimokuKinkoHyo", "delayed_minimum_senkou_b", self._ichimokukinkohyo.delayed_minimum_senkou_b.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `IchimokuKinkoHyo` indicator properties:



Indicator History

To get the historical data of the `IchimokuKinkoHyo` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class IchimokuKinkoHyoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ichimoku = self.ichimoku(self._symbol, 9, 26, 17, 52, 26, 26)

        indicator_history = self.indicator_history(self._ichimoku, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ichimoku, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ichimoku, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        tenkan = indicator_history_df["tenkan"]
        kijun = indicator_history_df["kijun"]
        senkou_a = indicator_history_df["senkoua"]
        senkou_b = indicator_history_df["senkoub"]
        chikou = indicator_history_df["chikou"]
        tenkan_maximum = indicator_history_df["tenkanmaximum"]
        tenkan_minimum = indicator_history_df["tenkanminimum"]
        kijun_maximum = indicator_history_df["kijunmaximum"]
        kijun_minimum = indicator_history_df["kijunminimum"]
        senkou_b_maximum = indicator_history_df["senkoubmaximum"]
        senkou_b_minimum = indicator_history_df["senkoubminimum"]
        delayed_tenkan_senkou_a = indicator_history_df["delayedtenkansenkoua"]
        delayed_kijun_senkou_a = indicator_history_df["delayedkijunsenkoua"]
        delayed_maximum_senkou_b = indicator_history_df["delayedmaximumsenkoub"]
        delayed_minimum_senkou_b = indicator_history_df["delayedminimumsenkoub"]
```

Supported Indicators

Identity

Introduction

This indicator represents an indicator that is a ready after ingesting a single sample and always returns the same value as it is given.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using Identity Indicator

To create an automatic indicator for `Identity` , call the `identity` helper method from the `QCAlgorithm` class. The `identity` method creates a `Identity` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class IdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identity = self.identity(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._identity.is_ready:
            # The current value of self._identity is represented by self._identity.current.value
            self.plot("Identity", "identity", self._identity.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Identity` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

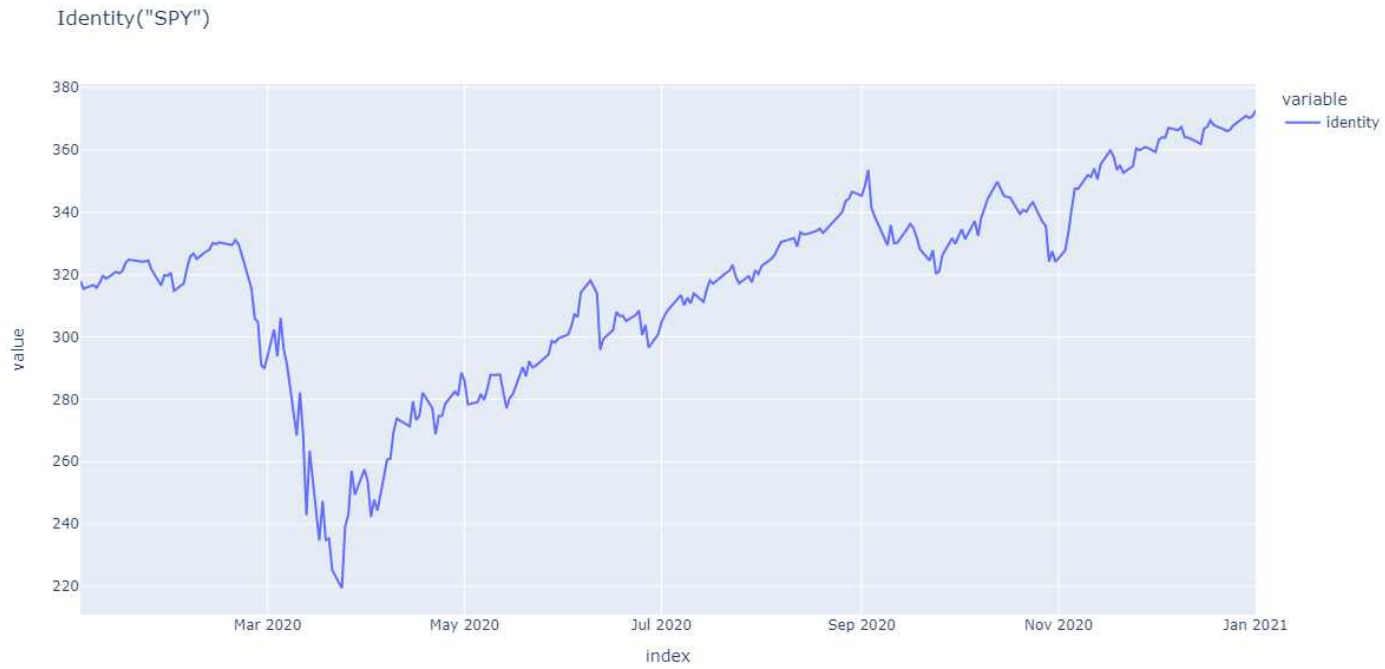
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class IdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identity = Identity()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._identity.update(bar.end_time, bar.close)

        if self._identity.is_ready:
            # The current value of self._identity is represented by self._identity.current.value
            self.plot("Identity", "identity", self._identity.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Identity` indicator properties:



Indicator History

To get the historical data of the `Identity` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class IdentityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._identity = self.identity(self._symbol)

        indicator_history = self.indicator_history(self._identity, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._identity, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._identity, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Implied Volatility

Introduction

Implied Volatility indicator that calculate the IV of an option using Black-Scholes Model

To view the implementation of this indicator, see the LEAN GitHub repository .

Using IV Indicator

To create an automatic indicator for `ImpliedVolatility` , call the `iv` helper method from the `QCAlgorithm` class. The `iv` method creates a `ImpliedVolatility` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ImpliedVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._iv = self.iv(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._iv.is_ready:
            # The current value of self._iv is represented by self._iv.current.value
            self.plot("ImpliedVolatility", "iv", self._iv.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ImpliedVolatility` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

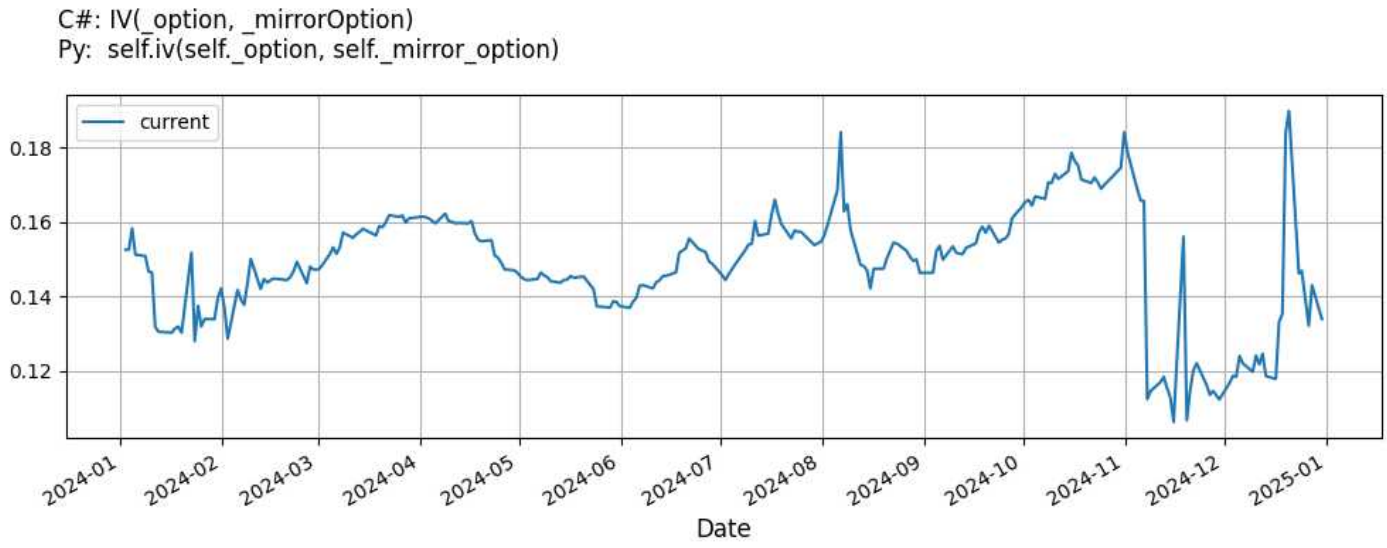
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ImpliedVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._impliedvolatility = ImpliedVolatility(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._impliedvolatility.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._impliedvolatility.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._impliedvolatility.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._impliedvolatility.is_ready:
            # The current value of self._impliedvolatility is represented by self._impliedvolatility.current.value
            self.plot("ImpliedVolatility", "impliedvolatility", self._impliedvolatility.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ImpliedVolatility` indicator properties:



Indicator History

To get the historical data of the `ImpliedVolatility` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ImpliedVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._iv = self.iv(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._iv, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._iv, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._iv, [self._symbol, self._option, self._mirror_option],
datetime(2024, 7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Internal Bar Strength

Introduction

The InternalBarStrenght indicator is a measure of the relative position of a period's closing price to the same period's high and low. The IBS can be interpreted to predict a bullish signal when displaying a low value and a bearish signal when presenting a high value.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using IBS Indicator

To create an automatic indicator for `InternalBarStrength` , call the `ibs` helper method from the `QCAlgorithm` class. The `ibs` method creates a `InternalBarStrength` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class InternalBarStrengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ibs = self.ibs(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._ibs.is_ready:
            # The current value of self._ibs is represented by self._ibs.current.value
            self.plot("InternalBarStrength", "ibs", self._ibs.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `InternalBarStrength` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

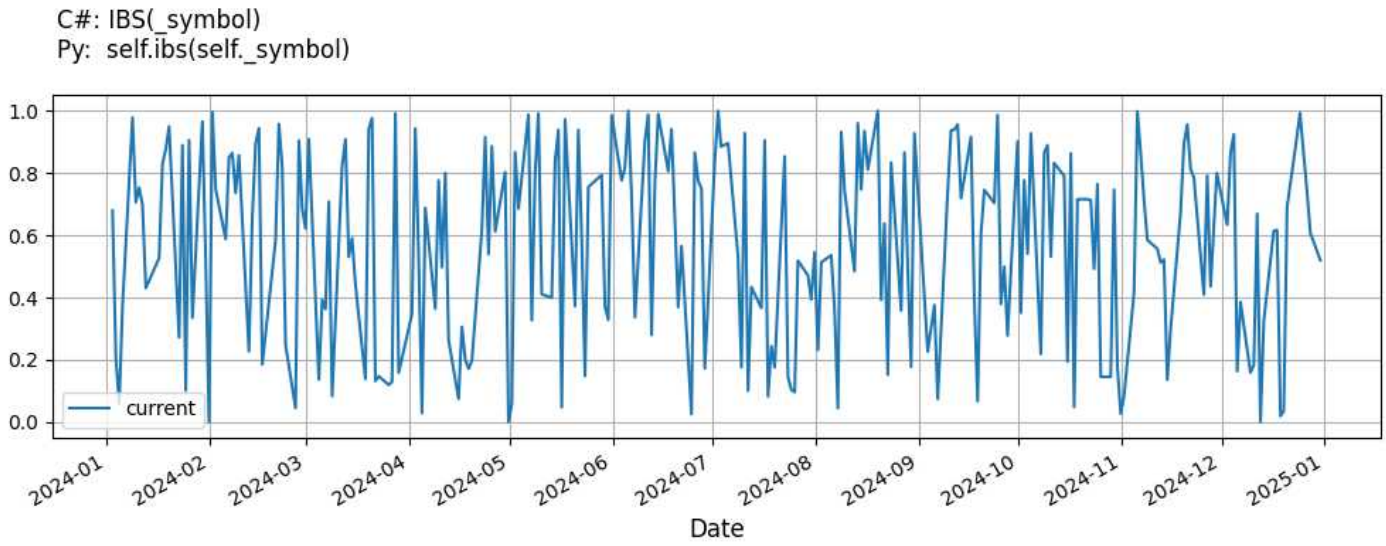
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class InternalBarStrengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._internalbarstrength = InternalBarStrength()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._internalbarstrength.update(bar)

        if self._internalbarstrength.is_ready:
            # The current value of self._internalbarstrength is represented by self._internalbarstrength.current.value
            self.plot("InternalBarStrength", "internalbarstrength", self._internalbarstrength.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `InternalBarStrength` indicator properties:



Indicator History

To get the historical data of the `InternalBarStrength` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```py
class InternalBarStrengthAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ibs = self.ibs(self._symbol)

        indicator_history = self.indicator_history(self._ibs, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ibs, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ibs, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Intraday Vwap

Introduction

Defines the canonical intraday VWAP indicator

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VWAP Indicator

To create an automatic indicator for `IntradayVwap` , call the `vwap` helper method from the `QCAlgorithm` class. The `vwap` method creates a `IntradayVwap` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class IntradayVwapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
        self._vwap = self.vwap(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._vwap.is_ready:
            # The current value of self._vwap is represented by self._vwap.current.value
            self.plot("IntradayVwap", "vwap", self._vwap.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `IntradayVwap` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

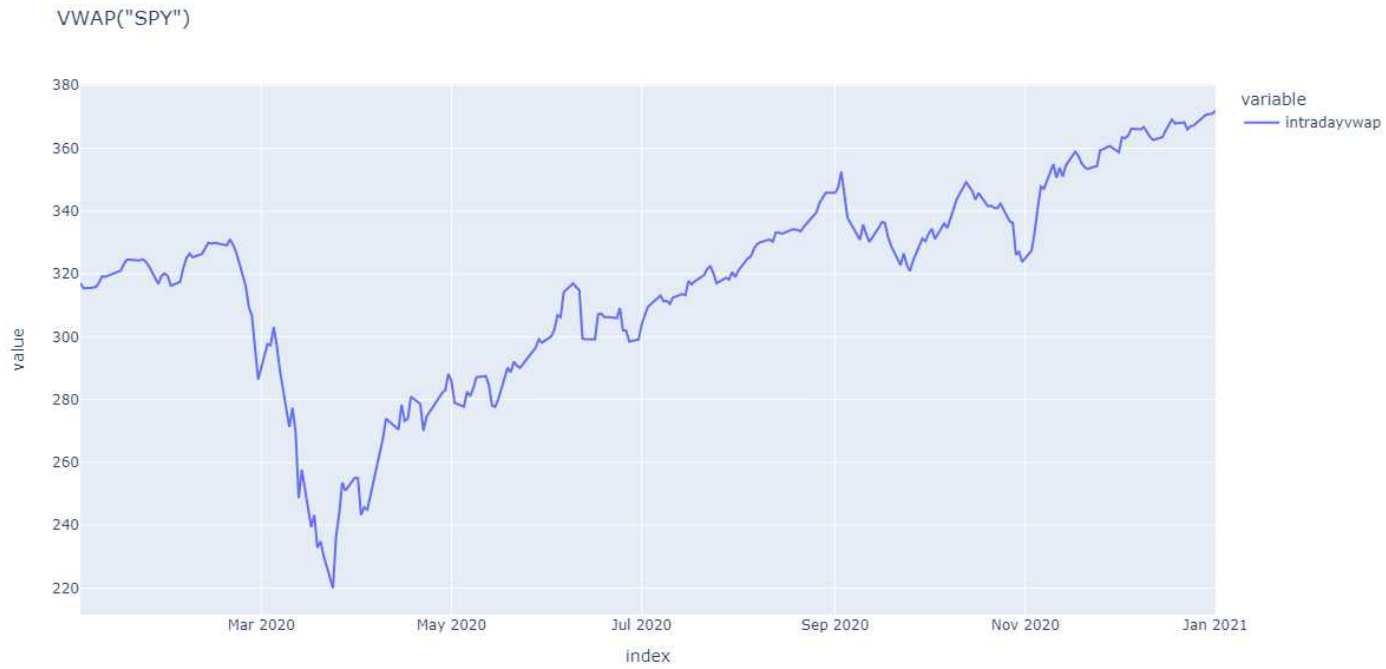
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class IntradayVwapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
        self._intradayvwap = IntradayVwap()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._intradayvwap.update(bar)

        if self._intradayvwap.is_ready:
            # The current value of self._intradayvwap is represented by self._intradayvwap.current.value
            self.plot("IntradayVwap", "intradayvwap", self._intradayvwap.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `IntradayVwap` indicator properties:



Indicator History

To get the historical data of the `IntradayVwap` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class IntradayVwapAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
        self._vwap = self.vwap(self._symbol)

        indicator_history = self.indicator_history(self._vwap, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vwap, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vwap, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Kaufman Adaptive Moving Average

Introduction

This indicator computes the Kaufman Adaptive Moving Average (KAMA). The Kaufman Adaptive Moving Average is calculated as explained here: http://stockcharts.com/school/doku.php?id=chart_school:technical_indicators:kaufman_s_adaptive_moving_average

To view the implementation of this indicator, see the LEAN GitHub repository .

Using KAMA Indicator

To create an automatic indicator for `KaufmanAdaptiveMovingAverage` , call the `kama` helper method from the `QCAlgorithm` class. The `kama` method creates a `KaufmanAdaptiveMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KaufmanAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kama = self.kama(self._symbol, 20, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._kama.is_ready:
            # The current value of self._kama is represented by self._kama.current.value
            self.plot("KaufmanAdaptiveMovingAverage", "kama", self._kama.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `KaufmanAdaptiveMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

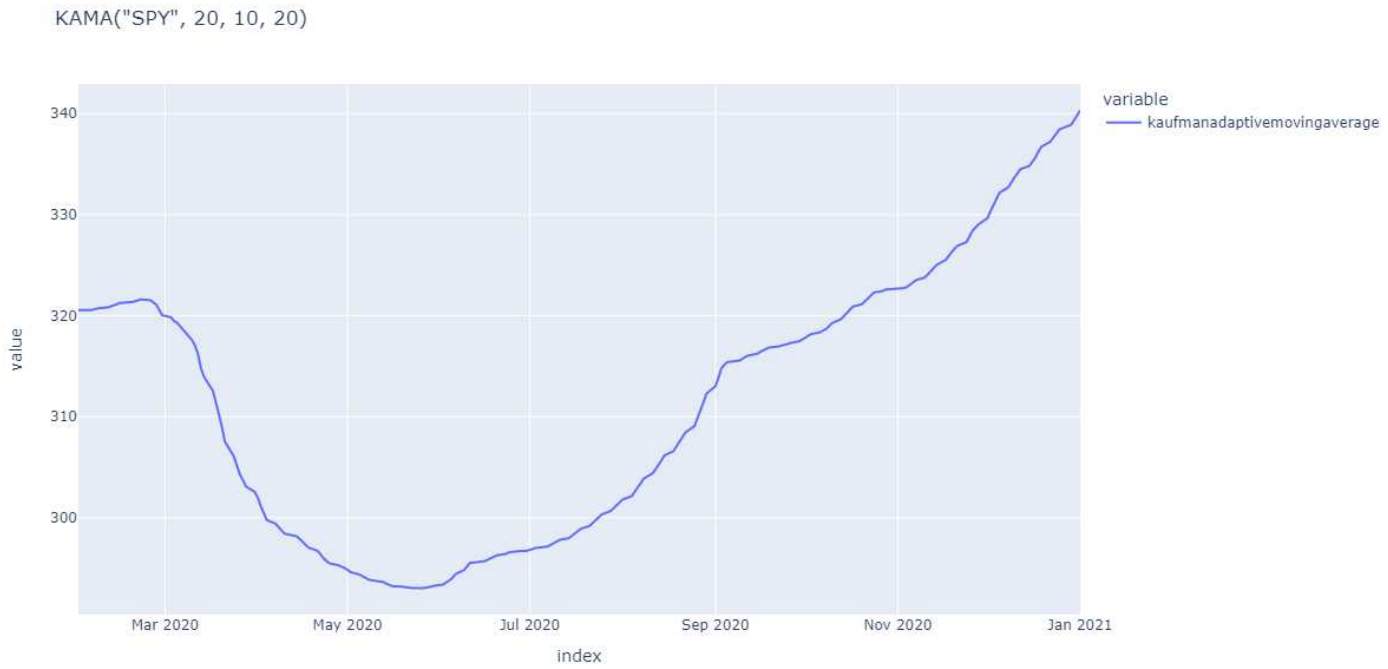
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class KaufmanAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kaufmanadaptivemovingaverage = KaufmanAdaptiveMovingAverage(20, 10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._kaufmanadaptivemovingaverage.update(bar.end_time, bar.close)

        if self._kaufmanadaptivemovingaverage.is_ready:
            # The current value of self._kaufmanadaptivemovingaverage is represented by self._kaufmanadaptivemovingaverage.current.value
            self.plot("KaufmanAdaptiveMovingAverage", "kaufmanadaptivemovingaverage", self._kaufmanadaptivemovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `KaufmanAdaptiveMovingAverage` indicator properties:



Indicator History

To get the historical data of the `KaufmanAdaptiveMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class KaufmanAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kama = self.kama(self._symbol, 20, 10, 20)

        indicator_history = self.indicator_history(self._kama, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kama, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kama, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Kaufman Efficiency Ratio

Introduction

This indicator computes the Kaufman Efficiency Ratio (KER). The Kaufman Efficiency Ratio is calculated as explained here: https://www.marketvolume.com/technicalanalysis/efficiencyratio.asp

To view the implementation of this indicator, see the LEAN GitHub repository .

Using KER Indicator

To create an automatic indicator for `KaufmanEfficiencyRatio` , call the `ker` helper method from the `QCAlgorithm` class. The `ker` method creates a `KaufmanEfficiencyRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KaufmanEfficiencyRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ker = self.ker(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._ker.is_ready:
            # The current value of self._ker is represented by self._ker.current.value
            self.plot("KaufmanEfficiencyRatio", "ker", self._ker.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `KaufmanEfficiencyRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

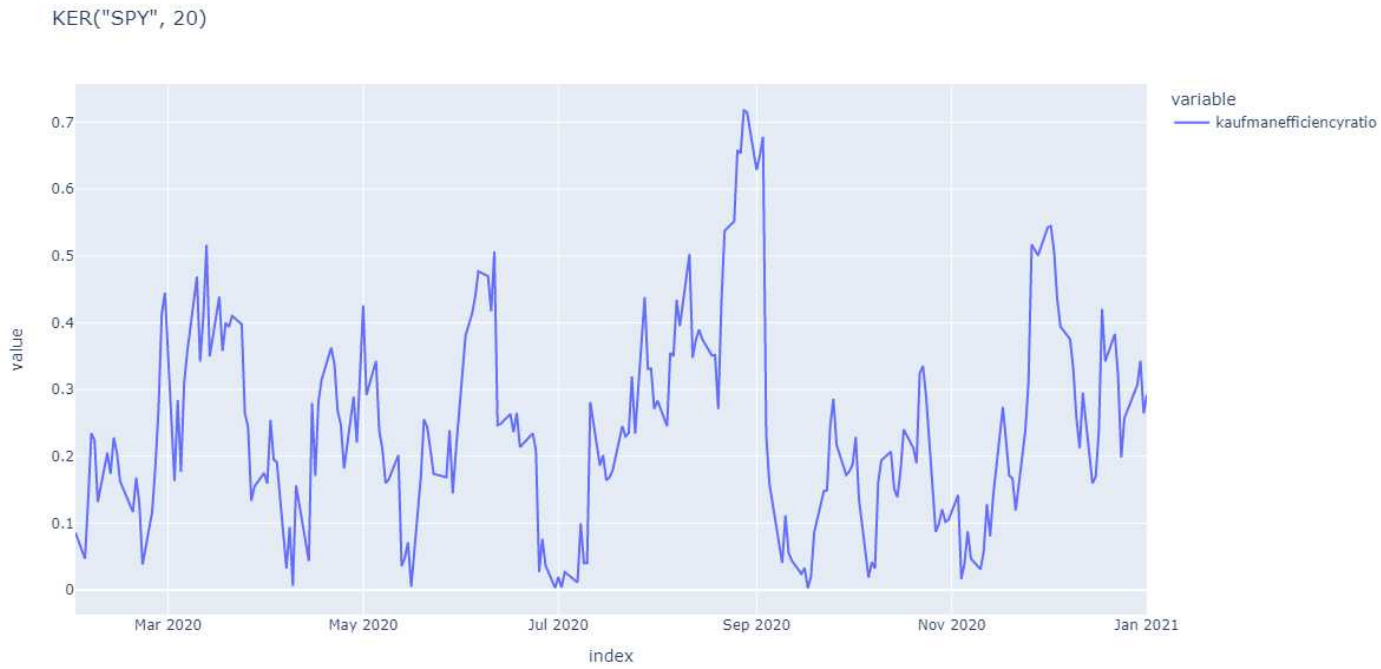
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class KaufmanEfficiencyRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kaufmanefficiencyratio = KaufmanEfficiencyRatio(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._kaufmanefficiencyratio.update(bar.end_time, bar.close)

        if self._kaufmanefficiencyratio.is_ready:
            # The current value of self._kaufmanefficiencyratio is represented by self._kaufmanefficiencyratio.current.value
            self.plot("KaufmanEfficiencyRatio", "kaufmanefficiencyratio", self._kaufmanefficiencyratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `KaufmanEfficiencyRatio` indicator properties:



Indicator History

To get the historical data of the `KaufmanEfficiencyRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class KaufmanEfficiencyRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ker = self.ker(self._symbol, 20)

        indicator_history = self.indicator_history(self._ker, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ker, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ker, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Keltner Channels

Introduction

This indicator creates a moving average (middle band) with an upper band and lower band fixed at k average True range multiples away from the middle band.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using KCH Indicator

To create an automatic indicator for `KeltnerChannels` , call the `kch` helper method from the `QCAlgorithm` class. The `kch` method creates a `KeltnerChannels` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KeltnerChannelsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kch = self.kch(self._symbol, 20, 2, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._kch.is_ready:
            # The current value of self._kch is represented by self._kch.current.value
            self.plot("KeltnerChannels", "kch", self._kch.current.value)
            # Plot all attributes of self._kch
            self.plot("KeltnerChannels", "middle_band", self._kch.middle_band.current.value)
            self.plot("KeltnerChannels", "upper_band", self._kch.upper_band.current.value)
            self.plot("KeltnerChannels", "lower_band", self._kch.lower_band.current.value)
            self.plot("KeltnerChannels", "average_true_range", self._kch.average_true_range.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `KeltnerChannels` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

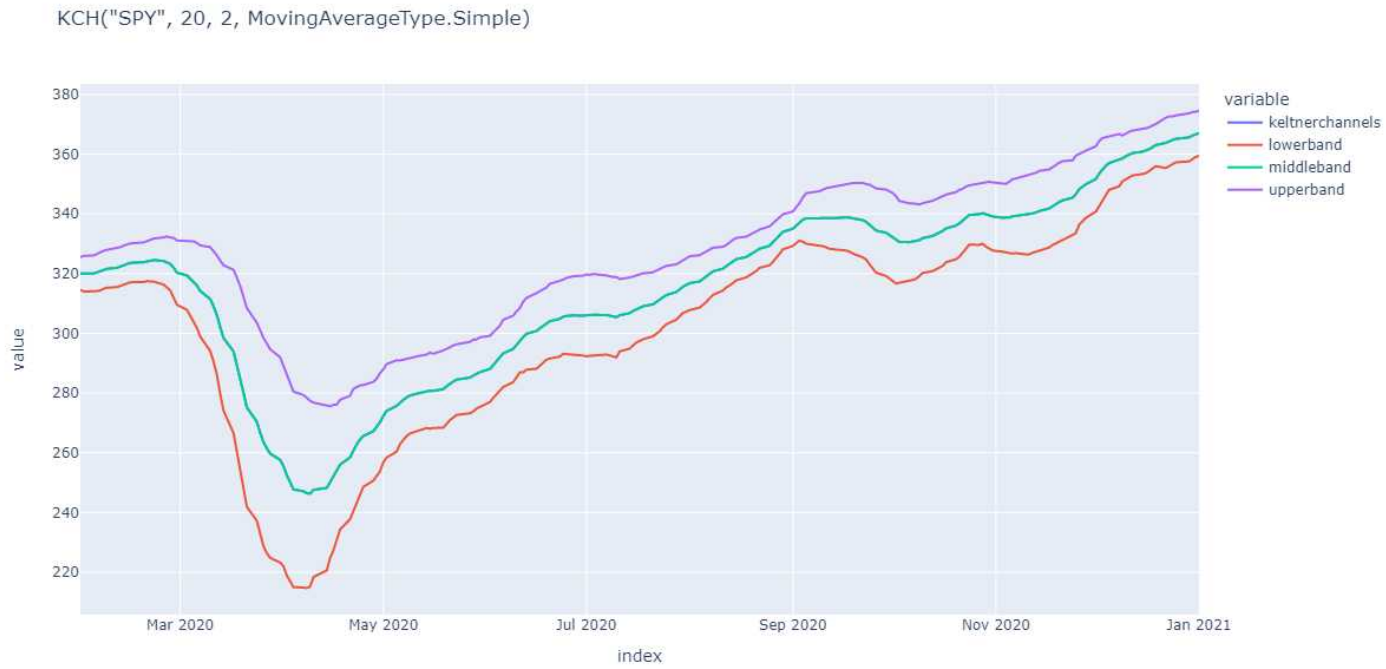
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class KeltnerChannelsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._keltnerchannels = KeltnerChannels(20, 2, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._keltnerchannels.update(bar)

        if self._keltnerchannels.is_ready:
            # The current value of self._keltnerchannels is represented by self._keltnerchannels.current.value
            self.plot("KeltnerChannels", "keltnerchannels", self._keltnerchannels.current.value)
            # Plot all attributes of self._keltnerchannels
            self.plot("KeltnerChannels", "middle_band", self._keltnerchannels.middle_band.current.value)
            self.plot("KeltnerChannels", "upper_band", self._keltnerchannels.upper_band.current.value)
            self.plot("KeltnerChannels", "lower_band", self._keltnerchannels.lower_band.current.value)
            self.plot("KeltnerChannels", "average_true_range", self._keltnerchannels.average_true_range.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `KeltnerChannels` indicator properties:



Indicator History

To get the historical data of the `KeltnerChannels` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class KeltnerChannelsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kch = self.kch(self._symbol, 20, 2, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._kch, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kch, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kch, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        middle_band = indicator_history_df["middleband"]
        upper_band = indicator_history_df["upperband"]
        lower_band = indicator_history_df["lowerband"]
        average_true_range = indicator_history_df["averagetruerange"]
```

Supported Indicators

Klinger Volume Oscillator

Introduction

The Klinger Volume Oscillator (KVO) is a technical indicator that analyzes the relationship between price movement and trading volume to assess the strength of market trends and identify potential trend reversals. As a volume-based oscillator, it measures the force behind price movements by incorporating volume data adjusted for price trends and specific conditions. Traders use the KVO to analyze its behavior relative to price action, looking for patterns such as divergences or crossovers that can provide insights into market trends and potential turning points.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using KVO Indicator

To create an automatic indicator for `KlingerVolumeOscillator` , call the `kvo` helper method from the `QCAlgorithm` class. The `kvo` method creates a `KlingerVolumeOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KlingerVolumeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kvo = self.kvo(self._symbol, 5, 10, 13)

    def on_data(self, slice: Slice) -> None:

        if self._kvo.is_ready:
            # The current value of self._kvo is represented by self._kvo.current.value
            self.plot("KlingerVolumeOscillator", "kvo", self._kvo.current.value)
            # Plot all attributes of self._kvo
            self.plot("KlingerVolumeOscillator", "signal", self._kvo.signal.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `KlingerVolumeOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

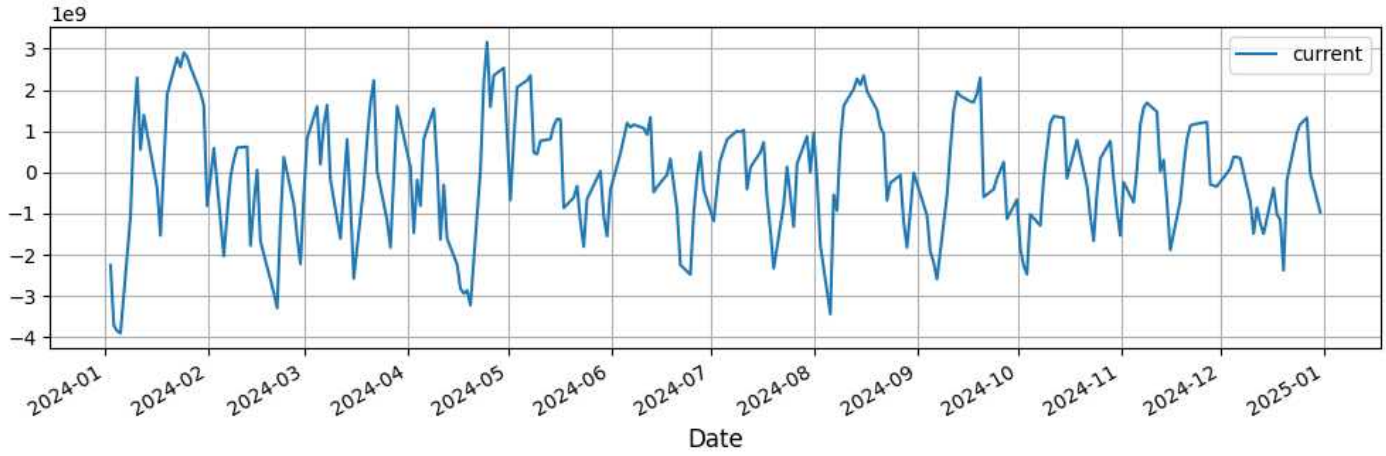
```
PY
class KlingerVolumeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._klingervolumeoscillator = KlingerVolumeOscillator(5, 10, 13)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._klingervolumeoscillator.update(bar)

        if self._klingervolumeoscillator.is_ready:
            # The current value of self._klingervolumeoscillator is represented by self._klingervolumeoscillator.current.value
            self.plot("KlingerVolumeOscillator", "klingervolumeoscillator", self._klingervolumeoscillator.current.value)
            # Plot all attributes of self._klingervolumeoscillator
            self.plot("KlingerVolumeOscillator", "signal", self._klingervolumeoscillator.signal.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `KlingerVolumeOscillator` indicator properties:

C#: KVO(_symbol, 5, 10, 13)
Py: self.kvo(self._symbol, 5, 10, 13)



Indicator History

To get the historical data of the `KlingerVolumeOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class KlingerVolumeOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kvo = self.kvo(self._symbol, 5, 10, 13)

        indicator_history = self.indicator_history(self._kvo, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kvo, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kvo, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        signal = indicator_history_df["signal"]
```

Supported Indicators

Know Sure Thing

Introduction

This indicator creates a moving average (middle band) with an upper band and lower band fixed at k standard deviations above and below the moving average.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using KST Indicator

To create an automatic indicator for `KnowSureThing` , call the `kst` helper method from the `QCAlgorithm` class. The `kst` method creates a `KnowSureThing` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class KnowSureThingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kst = self.kst(self._symbol, 5, 5, 10, 5, 15, 5, 25, 10, 9, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._kst.is_ready:
            # The current value of self._kst is represented by self._kst.current.value
            self.plot("KnowSureThing", "kst", self._kst.current.value)
            # Plot all attributes of self._kst
            self.plot("KnowSureThing", "roc_1", self._kst.roc_1.current.value)
            self.plot("KnowSureThing", "roc_2", self._kst.roc_2.current.value)
            self.plot("KnowSureThing", "roc_3", self._kst.roc_3.current.value)
            self.plot("KnowSureThing", "roc_4", self._kst.roc_4.current.value)
            self.plot("KnowSureThing", "roc_1_ma", self._kst.roc_1_ma.current.value)
            self.plot("KnowSureThing", "roc_2_ma", self._kst.roc_2_ma.current.value)
            self.plot("KnowSureThing", "roc_3_ma", self._kst.roc_3_ma.current.value)
            self.plot("KnowSureThing", "roc_4_ma", self._kst.roc_4_ma.current.value)
            self.plot("KnowSureThing", "signal_line", self._kst.signal_line.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `KnowSureThing` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

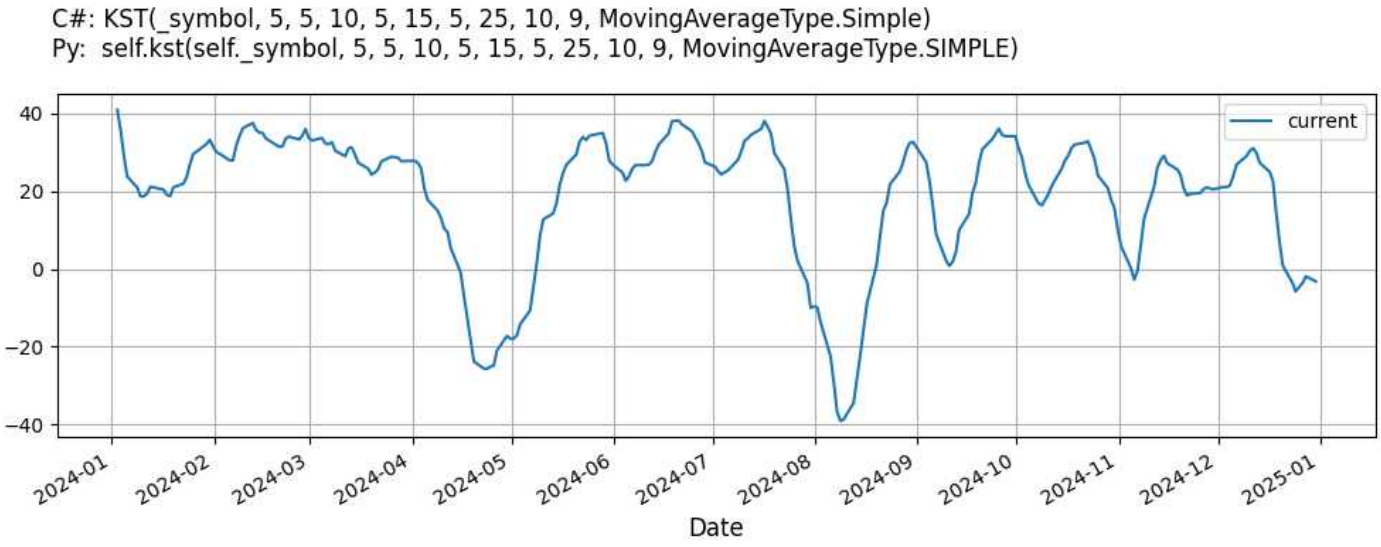
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class KnowSureThingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._knowsurething = KnowSureThing(5, 5, 10, 5, 15, 5, 25, 10, 9, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._knowsurething.update(bar.end_time, bar.close)

        if self._knowsurething.is_ready:
            # The current value of self._knowsurething is represented by self._knowsurething.current.value
            self.plot("KnowSureThing", "knowsurething", self._knowsurething.current.value)
            # Plot all attributes of self._knowsurething
            self.plot("KnowSureThing", "roc_1", self._knowsurething.roc_1.current.value)
            self.plot("KnowSureThing", "roc_2", self._knowsurething.roc_2.current.value)
            self.plot("KnowSureThing", "roc_3", self._knowsurething.roc_3.current.value)
            self.plot("KnowSureThing", "roc_4", self._knowsurething.roc_4.current.value)
            self.plot("KnowSureThing", "roc_1_ma", self._knowsurething.roc_1_ma.current.value)
            self.plot("KnowSureThing", "roc_2_ma", self._knowsurething.roc_2_ma.current.value)
            self.plot("KnowSureThing", "roc_3_ma", self._knowsurething.roc_3_ma.current.value)
            self.plot("KnowSureThing", "roc_4_ma", self._knowsurething.roc_4_ma.current.value)
            self.plot("KnowSureThing", "signal_line", self._knowsurething.signal_line.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the KnowSureThing indicator properties:



Indicator History

To get the historical data of the KnowSureThing indicator, call the self.indicator_history method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the indicator_history method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a resolution argument, it defaults to match the resolution of the security subscription.

PY

```
class KnowSureThingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._kst = self.kst(self._symbol, 5, 5, 10, 5, 15, 5, 25, 10, 9, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._kst, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._kst, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._kst, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        roc_1 = indicator_history_df["roc1"]
        roc_2 = indicator_history_df["roc2"]
        roc_3 = indicator_history_df["roc3"]
        roc_4 = indicator_history_df["roc4"]
        roc_1_ma = indicator_history_df["roc1ma"]
        roc_2_ma = indicator_history_df["roc2ma"]
        roc_3_ma = indicator_history_df["roc3ma"]
        roc_4_ma = indicator_history_df["roc4ma"]
        signal_line = indicator_history_df["signalline"]
```

Supported Indicators

Least Squares Moving Average

Introduction

The Least Squares Moving Average (LSMA) first calculates a least squares regression line over the preceding time periods, and then projects it forward to the current period. In essence, it calculates what the value would be if the regression line continued. source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using LSMA Indicator

To create an automatic indicator for `LeastSquaresMovingAverage` , call the `lsma` helper method from the `QCAlgorithm` class. The `lsma` method creates a `LeastSquaresMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

PY

```
class LeastSquaresMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._lsma = self.lsma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._lsma.is_ready:
            # The current value of self._lsma is represented by self._lsma.current.value
            self.plot("LeastSquaresMovingAverage", "lsma", self._lsma.current.value)
            # Plot all attributes of self._lsma
            self.plot("LeastSquaresMovingAverage", "intercept", self._lsma.intercept.current.value)
            self.plot("LeastSquaresMovingAverage", "slope", self._lsma.slope.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `LeastSquaresMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

PY

```
class LeastSquaresMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._leastsquaresmovingaverage = LeastSquaresMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._leastsquaresmovingaverage.update(bar.end_time, bar.close)

        if self._leastsquaresmovingaverage.is_ready:
            # The current value of self._leastsquaresmovingaverage is represented by self._leastsquaresmovingaverage.current.value
            self.plot("LeastSquaresMovingAverage", "leastsquaresmovingaverage", self._leastsquaresmovingaverage.current.value)
            # Plot all attributes of self._leastsquaresmovingaverage
            self.plot("LeastSquaresMovingAverage", "intercept", self._leastsquaresmovingaverage.intercept.current.value)
            self.plot("LeastSquaresMovingAverage", "slope", self._leastsquaresmovingaverage.slope.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `LeastSquaresMovingAverage` indicator properties:

Indicator History

To get the historical data of the `LeastSquaresMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class LeastSquaresMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._lsma = self.lsma(self._symbol, 20)

        indicator_history = self.indicator_history(self._lsma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._lsma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._lsma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        intercept = indicator_history_df["intercept"]
        slope = indicator_history_df["slope"]
```

Supported Indicators

Linear Weighted Moving Average

Introduction

This indicator represents the traditional Weighted Moving Average indicator. The weight are linearly distributed according to the number of periods in the indicator. For example, a 4 period indicator will have a numerator of (4 * window[0]) + (3 * window[1]) + (2 * window[2]) + window[3] and a denominator of 4 + 3 + 2 + 1 = 10 During the warm up period, IsReady will return False, but the LWMA will still be computed correctly because the denominator will be the minimum of Samples factorial or Size factorial and the computation iterates over that minimum value. The RollingWindow of inputs is created when the indicator is created. A RollingWindow of LWMAs is not saved. That is up to the caller.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using LWMA Indicator

To create an automatic indicator for `LinearWeightedMovingAverage` , call the `lwma` helper method from the `QCAlgorithm` class. The `lwma` method creates a `LinearWeightedMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class LinearWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._lwma = self.lwma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._lwma.is_ready:
            # The current value of self._lwma is represented by self._lwma.current.value
            self.plot("LinearWeightedMovingAverage", "lwma", self._lwma.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `LinearWeightedMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

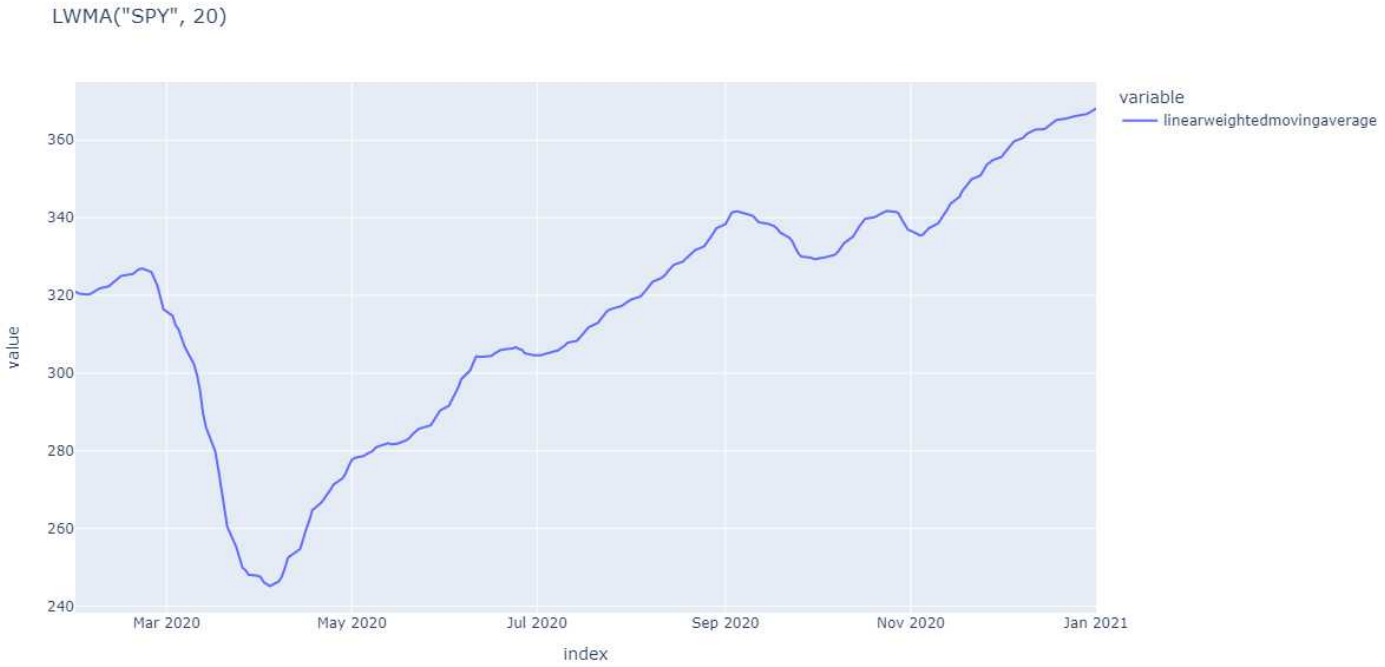
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class LinearWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._linearweightedmovingaverage = LinearWeightedMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._linearweightedmovingaverage.update(bar.end_time, bar.close)

        if self._linearweightedmovingaverage.is_ready:
            # The current value of self._linearweightedmovingaverage is represented by self._linearweightedmovingaverage.current.value
            self.plot("LinearWeightedMovingAverage", "linearweightedmovingaverage", self._linearweightedmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `LinearWeightedMovingAverage` indicator properties:



Indicator History

To get the historical data of the `LinearWeightedMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class LinearWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._lwma = self.lwma(self._symbol, 20)

        indicator_history = self.indicator_history(self._lwma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._lwma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._lwma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Log Return

Introduction

This indicator represents the LogReturn indicator (LOGR) - log returns are useful for identifying price convergence/divergence in a given period - logr = log (current price / last price in period)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using LOGR Indicator

To create an automatic indicator for `LogReturn` , call the `logr` helper method from the `QCAlgorithm` class. The `logr` method creates a `LogReturn` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class LogReturnAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._logr = self.logr(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._logr.is_ready:
            # The current value of self._logr is represented by self._logr.current.value
            self.plot("LogReturn", "logr", self._logr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `LogReturn` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

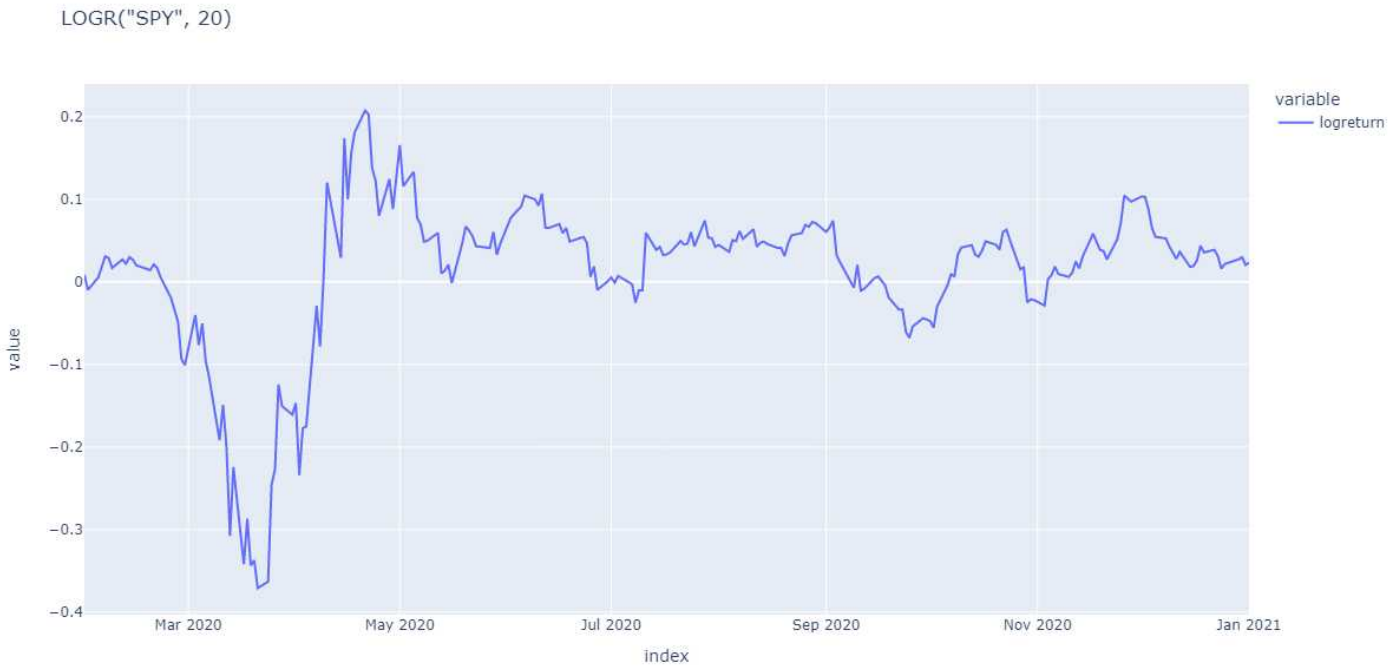
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class LogReturnAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._logreturn = LogReturn(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._logreturn.update(bar.end_time, bar.close)

        if self._logreturn.is_ready:
            # The current value of self._logreturn is represented by self._logreturn.current.value
            self.plot("LogReturn", "logreturn", self._logreturn.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `LogReturn` indicator properties:



Indicator History

To get the historical data of the `LogReturn` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class LogReturnAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._logr = self.logr(self._symbol, 20)

        indicator_history = self.indicator_history(self._logr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._logr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._logr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Mass Index

Introduction

The Mass Index uses the high-low range to identify trend reversals based on range expansions. In this sense, the Mass Index is a volatility indicator that does not have a directional bias. Instead, the Mass Index identifies range bulges that can foreshadow a reversal of the current trend. Developed by Donald Dorsey.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MASS Indicator

To create an automatic indicator for `MassIndex` , call the `mass` helper method from the `QCAlgorithm` class. The `mass` method creates a `MassIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MassIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mass = self.mass(self._symbol, 9, 25)

    def on_data(self, slice: Slice) -> None:

        if self._mass.is_ready:
            # The current value of self._mass is represented by self._mass.current.value
            self.plot("MassIndex", "mass", self._mass.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MassIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

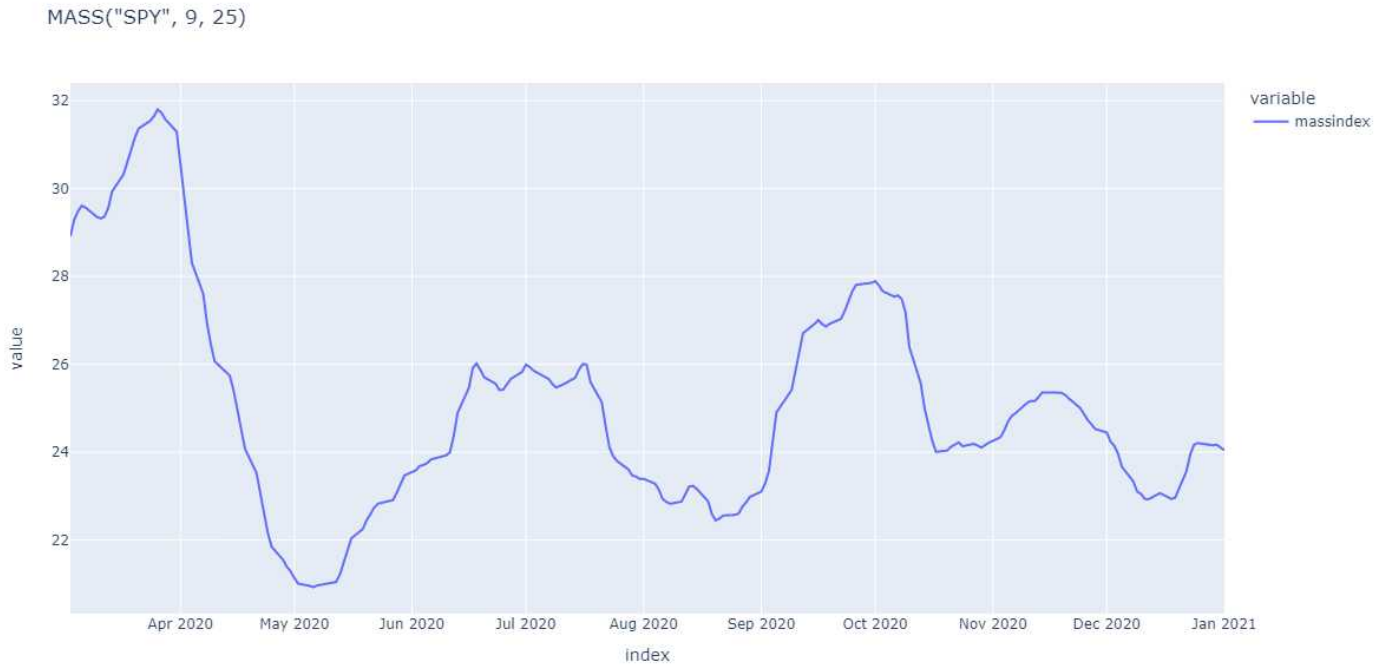
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MassIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._massindex = MassIndex(9, 25)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._massindex.update(bar)

        if self._massindex.is_ready:
            # The current value of self._massindex is represented by self._massindex.current.value
            self.plot("MassIndex", "massindex", self._massindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MassIndex` indicator properties:



Indicator History

To get the historical data of the `MassIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MassIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mass = self.mass(self._symbol, 9, 25)

        indicator_history = self.indicator_history(self._mass, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mass, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mass, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Maximum

Introduction

This indicator represents an indicator capable of tracking the maximum value and how many periods ago it occurred

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MAX Indicator

To create an automatic indicator for `Maximum` , call the `max` helper method from the `QCAlgorithm` class. The `max` method creates a `Maximum` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MaximumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._max = self.max(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._max.is_ready:
            # The current value of self._max is represented by self._max.current.value
            self.plot("Maximum", "max", self._max.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Maximum` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

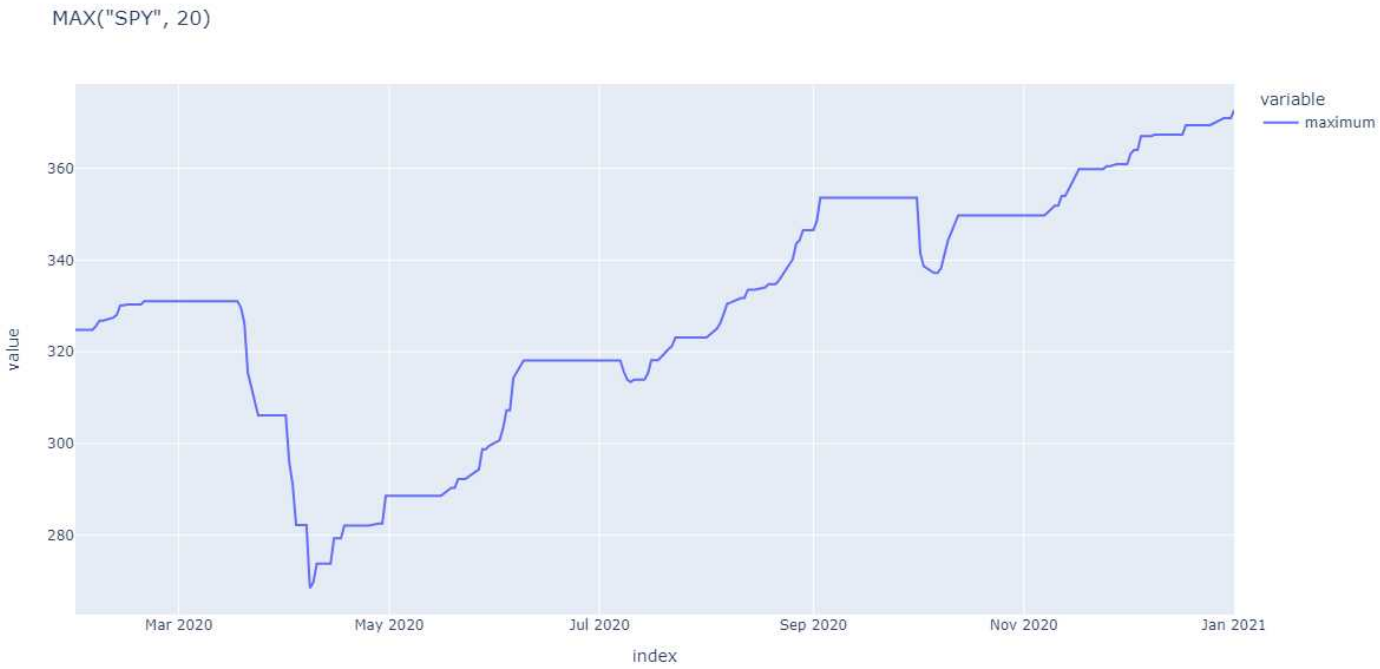
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MaximumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._maximum = Maximum(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._maximum.update(bar.end_time, bar.close)

        if self._maximum.is_ready:
            # The current value of self._maximum is represented by self._maximum.current.value
            self.plot("Maximum", "maximum", self._maximum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Maximum` indicator properties:



Indicator History

To get the historical data of the `Maximum` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MaximumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._max = self.max(self._symbol, 20)

        indicator_history = self.indicator_history(self._max, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._max, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._max, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

McClellan Oscillator

Introduction

The McClellan Oscillator is a market breadth indicator which was developed by Sherman and Marian McClellan. It is based on the difference between the number of advancing and declining periods.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MOSC Indicator

To create an automatic indicator for `McClellanOscillator` , call the `mosc` helper method from the `QCAlgorithm` class. The `mosc` method creates a `McClellanOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class McClellanOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mosc = self.mosc([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._mosc.is_ready:
            # The current value of self._mosc is represented by self._mosc.current.value
            self.plot("McClellanOscillator", "mosc", self._mosc.current.value)
            # Plot all attributes of self._mosc
            self.plot("McClellanOscillator", "ema_fast", self._mosc.ema_fast.current.value)
            self.plot("McClellanOscillator", "ema_slow", self._mosc.ema_slow.current.value)
            self.plot("McClellanOscillator", "ad_difference", self._mosc.ad_difference.current.value)
```

To create an automatic indicator for `McClellanOscillator` using universe constituents, call the `mosc` helper method from the `QCAlgorithm` class. The `mosc` method creates a `McClellanOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
class McClellanOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._mosc = self.mosc(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._mosc.is_ready:
            # The current value of self._mosc is represented by self._mosc.current.value
            self.plot("McClellanOscillator", "mosc", self._mosc.current.value)
            # Plot all attributes of self._mosc
            self.plot("McClellanOscillator", "ema_fast", self._mosc.ema_fast.current.value)
            self.plot("McClellanOscillator", "ema_slow", self._mosc.ema_slow.current.value)
            self.plot("McClellanOscillator", "ad_difference", self._mosc.ad_difference.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `McClellanOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

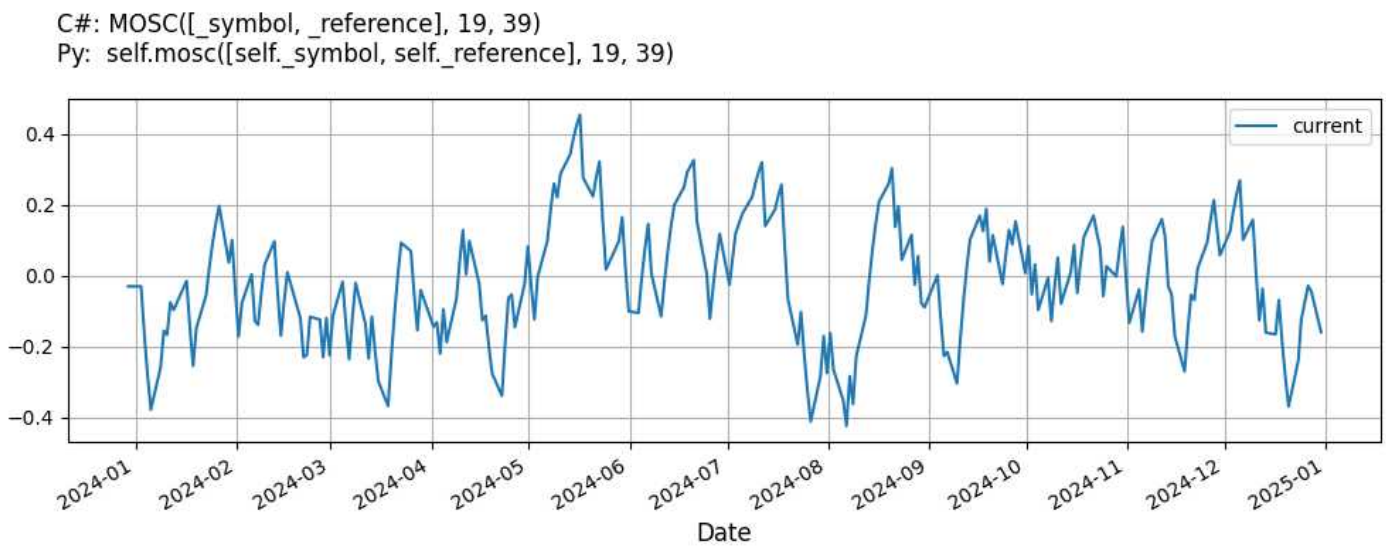
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class McClellanOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mcclellanoscillator = McClellanOscillator(19, 39)
        self._mcclellanoscillator.add(self._symbol);
        self._mcclellanoscillator.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._mcclellanoscillator.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._mcclellanoscillator.update(bar)

        if self._mcclellanoscillator.is_ready:
            # The current value of self._mcclellanoscillator is represented by self._mcclellanoscillator.current.value
            self.plot("McClellanOscillator", "mcclellanoscillator", self._mcclellanoscillator.current.value)
            # Plot all attributes of self._mcclellanoscillator
            self.plot("McClellanOscillator", "ema_fast", self._mcclellanoscillator.ema_fast.current.value)
            self.plot("McClellanOscillator", "ema_slow", self._mcclellanoscillator.ema_slow.current.value)
            self.plot("McClellanOscillator", "ad_difference", self._mcclellanoscillator.ad_difference.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `McClellanOscillator` indicator properties:



Indicator History

To get the historical data of the `McClellanOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class McClellanOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mosc = self.mosc([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._mosc, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mosc, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mosc, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        ema_fast = indicator_history_df["emafast"]
        ema_slow = indicator_history_df["emaslow"]
        ad_difference = indicator_history_df["addifference"]
```

Supported Indicators

McClellan Summation Index

Introduction

The McClellan Summation Index (MSI) is a market breadth indicator that is based on the rolling average of difference between the number of advancing and declining issues on a stock exchange. It is generally considered as is a long-term version of the

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MSI Indicator

To create an automatic indicator for `McClellanSummationIndex` , call the `msi` helper method from the `QCAlgorithm` class. The `msi` method creates a `McClellanSummationIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class McClellanSummationIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._msi = self.msi([self._symbol, self._reference])

    def on_data(self, slice: Slice) -> None:

        if self._msi.is_ready:
            # The current value of self._msi is represented by self._msi.current.value
            self.plot("McClellanSummationIndex", "msi", self._msi.current.value)
            # Plot all attributes of self._msi
            self.plot("McClellanSummationIndex", "mc_clellan_oscillator", self._msi.mc_clellan_oscillator.current.value)
```

PY

To create an automatic indicator for `McClellanSummationIndex` using universe constituents, call the `msi` helper method from the `QCAlgorithm` class. The `msi` method creates a `McClellanSummationIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
class McClellanSummationIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._msi = self.msi(list(self._universe.selected))

    def on_data(self, slice: Slice) -> None:

        if self._msi.is_ready:
            # The current value of self._msi is represented by self._msi.current.value
            self.plot("McClellanSummationIndex", "msi", self._msi.current.value)
            # Plot all attributes of self._msi
            self.plot("McClellanSummationIndex", "mc_clellan_oscillator", self._msi.mc_clellan_oscillator.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `McClellanSummationIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

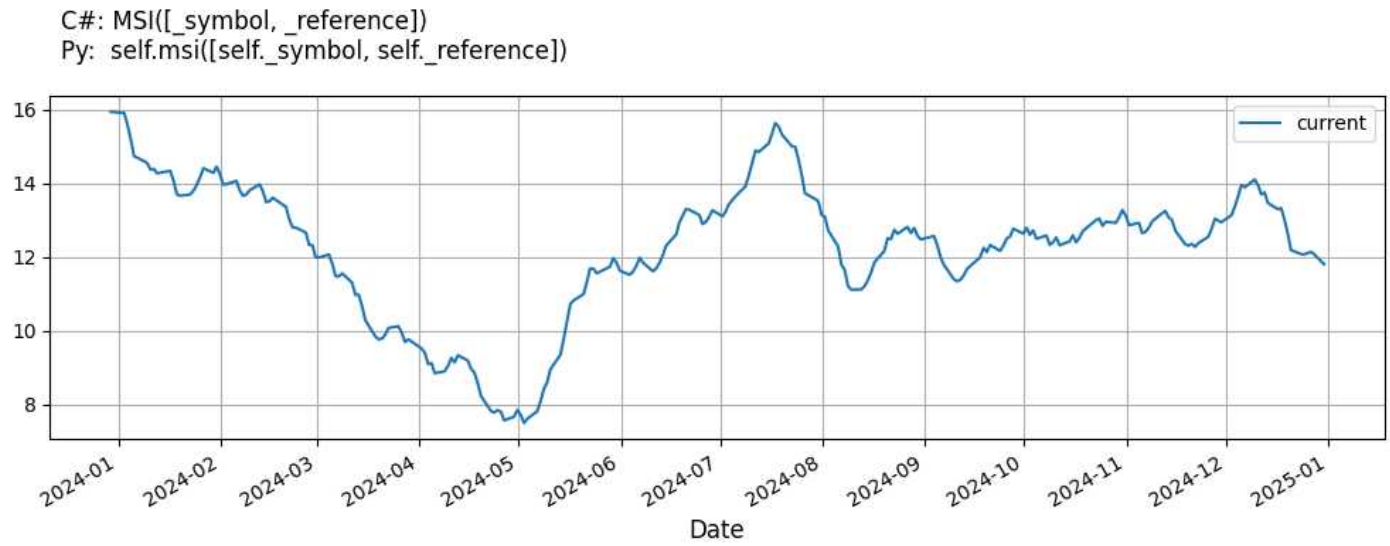
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class McClellanSummationIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mcclellansummationindex = McClellanSummationIndex(19, 39)
        self._mcclellansummationindex.add(self._symbol);
        self._mcclellansummationindex.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._mcclellansummationindex.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._mcclellansummationindex.update(bar)

        if self._mcclellansummationindex.is_ready:
            # The current value of self._mcclellansummationindex is represented by self._mcclellansummationindex.current.value
            self.plot("McClellanSummationIndex", "mcclellansummationindex", self._mcclellansummationindex.current.value)
            # Plot all attributes of self._mcclellansummationindex
            self.plot("McClellanSummationIndex", "mc_clellan_oscillator",
self._mcclellansummationindex.mc_clellan_oscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `McClellanSummationIndex` indicator properties:



Indicator History

To get the historical data of the `McClellanSummationIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class McClellanSummationIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._msi = self.msi([self._symbol, self._reference])

        indicator_history = self.indicator_history(self._msi, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._msi, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._msi, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        mc_clellan_oscillator = indicator_history_df["mcclellanoscillator"]
```

Supported Indicators

McGinley Dynamic

Introduction

This indicator represents the McGinley Dynamic (MGD) It is a type of moving average that was designed to track the market better than existing moving average indicators. It is a technical indicator that improves upon moving average lines by adjusting for shifts in market speed.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MGD Indicator

To create an automatic indicator for `McGinleyDynamic` , call the `mgd` helper method from the `QCAlgorithm` class. The `mgd` method creates a `McGinleyDynamic` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class McGinleyDynamicAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mgd = self.mgd(self._symbol, 10)

    def on_data(self, slice: Slice) -> None:

        if self._mgd.is_ready:
            # The current value of self._mgd is represented by self._mgd.current.value
            self.plot("McGinleyDynamic", "mgd", self._mgd.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `McGinleyDynamic` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

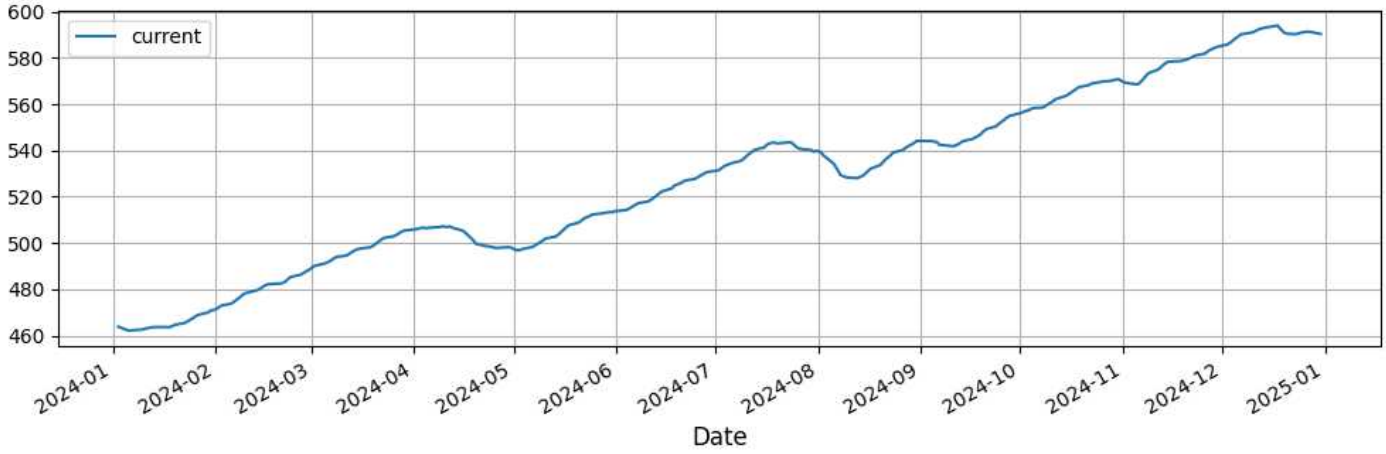
```
PY
class McGinleyDynamicAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mcginleydynamic = McGinleyDynamic(10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._mcginleydynamic.update(bar.end_time, bar.close)

        if self._mcginleydynamic.is_ready:
            # The current value of self._mcginleydynamic is represented by self._mcginleydynamic.current.value
            self.plot("McGinleyDynamic", "mcginleydynamic", self._mcginleydynamic.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `McGinleyDynamic` indicator properties:

C#: MGD(_symbol, 10)
Py: self.mgd(self._symbol, 10)



Indicator History

To get the historical data of the `McGinleyDynamic` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class McGinleyDynamicAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mgd = self.mgd(self._symbol, 10)

        indicator_history = self.indicator_history(self._mgd, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mgd, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mgd, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Mean Absolute Deviation

Introduction

This indicator computes the n-period mean absolute deviation.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MAD Indicator

To create an automatic indicator for `MeanAbsoluteDeviation` , call the `mad` helper method from the `QCAlgorithm` class. The `mad` method creates a `MeanAbsoluteDeviation` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MeanAbsoluteDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mad = self.mad(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._mad.is_ready:
            # The current value of self._mad is represented by self._mad.current.value
            self.plot("MeanAbsoluteDeviation", "mad", self._mad.current.value)
            # Plot all attributes of self._mad
            self.plot("MeanAbsoluteDeviation", "mean", self._mad.mean.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `MeanAbsoluteDeviation` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

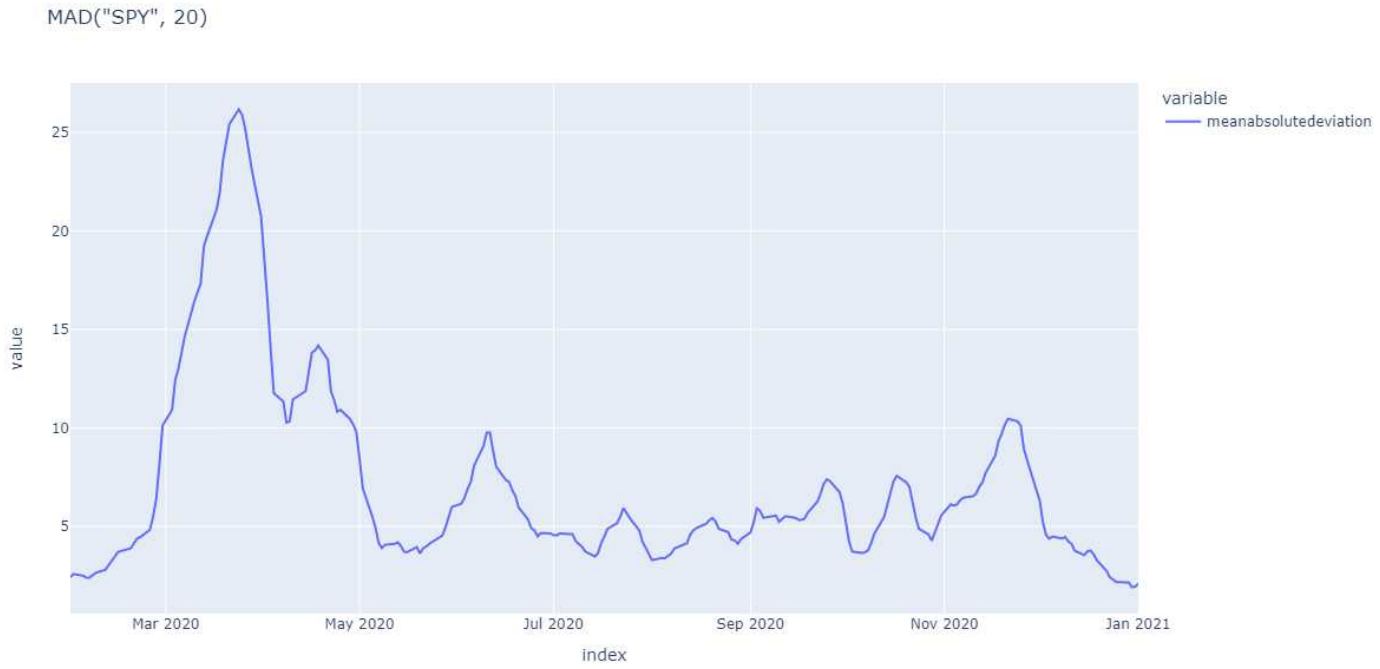
```
class MeanAbsoluteDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._meanabsolutedeviation = MeanAbsoluteDeviation(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._meanabsolutedeviation.update(bar.end_time, bar.close)

        if self._meanabsolutedeviation.is_ready:
            # The current value of self._meanabsolutedeviation is represented by self._meanabsolutedeviation.current.value
            self.plot("MeanAbsoluteDeviation", "meanabsolutedeviation", self._meanabsolutedeviation.current.value)
            # Plot all attributes of self._meanabsolutedeviation
            self.plot("MeanAbsoluteDeviation", "mean", self._meanabsolutedeviation.mean.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MeanAbsoluteDeviation` indicator properties:



Indicator History

To get the historical data of the `MeanAbsoluteDeviation` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MeanAbsoluteDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mad = self.mad(self._symbol, 20)

        indicator_history = self.indicator_history(self._mad, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mad, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mad, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        mean = indicator_history_df["mean"]
```

Supported Indicators

Mesa Adaptive Moving Average

Introduction

Implements the Mesa Adaptive Moving Average (MAMA) indicator along with the following FAMA (Following Adaptive Moving Average) as a secondary indicator. The MAMA adjusts its smoothing factor based on the market's volatility, making it more adaptive than a simple moving average.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MAMA Indicator

To create an automatic indicator for `MesaAdaptiveMovingAverage` , call the `mama` helper method from the `QCAlgorithm` class. The `mama` method creates a `MesaAdaptiveMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MesaAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mama = self.mama(self._symbol, 0.5, 0.05)

    def on_data(self, slice: Slice) -> None:

        if self._mama.is_ready:
            # The current value of self._mama is represented by self._mama.current.value
            self.plot("MesaAdaptiveMovingAverage", "mama", self._mama.current.value)
            # Plot all attributes of self._mama
            self.plot("MesaAdaptiveMovingAverage", "fama", self._mama.fama.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MesaAdaptiveMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

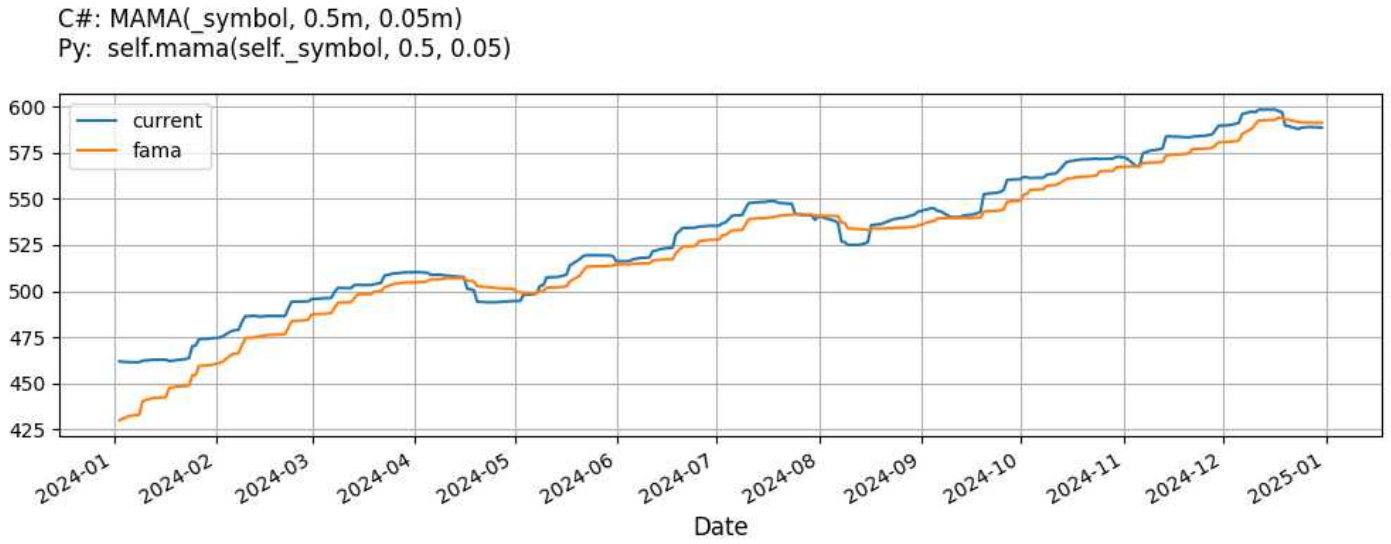
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MesaAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mesaadaptivemovingaverage = MesaAdaptiveMovingAverage(0.5, 0.05)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._mesaadaptivemovingaverage.update(bar)

        if self._mesaadaptivemovingaverage.is_ready:
            # The current value of self._mesaadaptivemovingaverage is represented by self._mesaadaptivemovingaverage.current.value
            self.plot("MesaAdaptiveMovingAverage", "mesaadaptivemovingaverage", self._mesaadaptivemovingaverage.current.value)
            # Plot all attributes of self._mesaadaptivemovingaverage
            self.plot("MesaAdaptiveMovingAverage", "fama", self._mesaadaptivemovingaverage.fama.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MesaAdaptiveMovingAverage` indicator properties:



Indicator History

To get the historical data of the `MesaAdaptiveMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MesaAdaptiveMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mama = self.mama(self._symbol, 0.5, 0.05)

        indicator_history = self.indicator_history(self._mama, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mama, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mama, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        fama = indicator_history_df["fama"]
```

Supported Indicators

Mid Point

Introduction

This indicator computes the MidPoint (MIDPOINT) The MidPoint is calculated using the following formula: MIDPOINT = (Highest Value + Lowest Value) / 2

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MIDPOINT Indicator

To create an automatic indicator for `MidPoint` , call the `midpoint` helper method from the `QCAlgorithm` class. The `midpoint` method creates a `MidPoint` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MidPointAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midpoint = self.midpoint(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._midpoint.is_ready:
            # The current value of self._midpoint is represented by self._midpoint.current.value
            self.plot("MidPoint", "midpoint", self._midpoint.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MidPoint` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

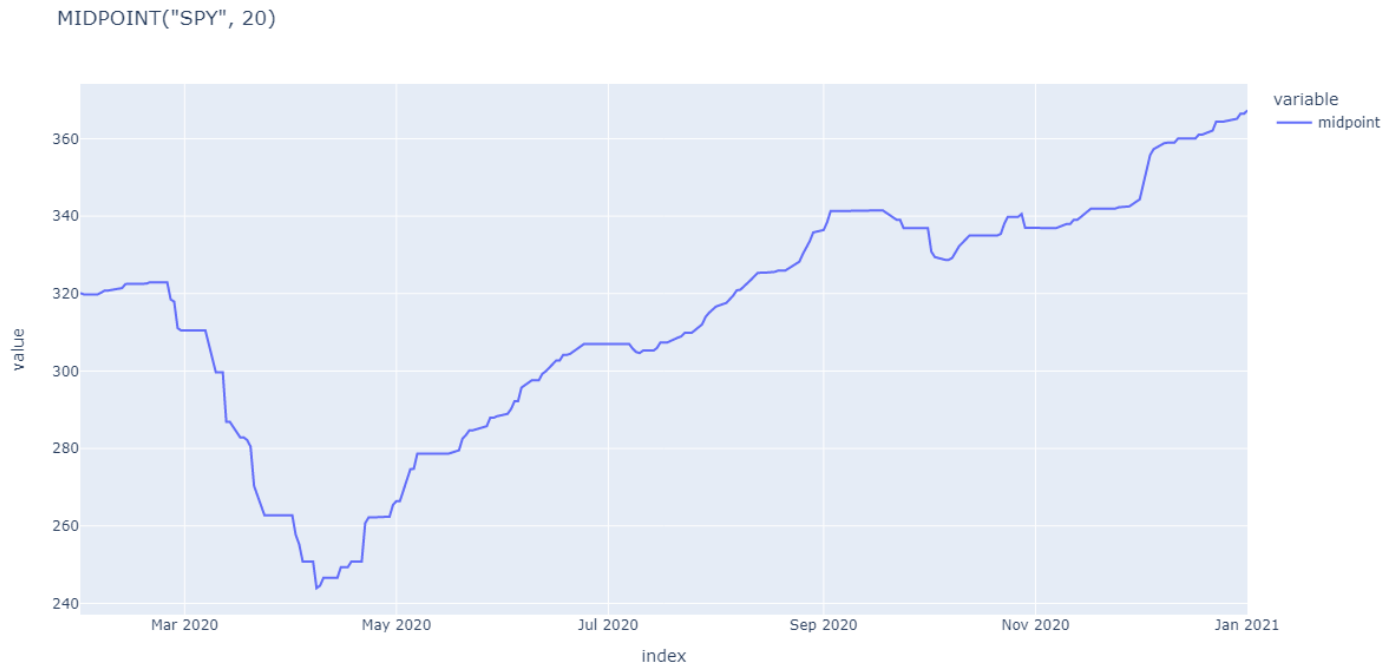
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MidPointAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midpoint = MidPoint(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._midpoint.update(bar.end_time, bar.close)

        if self._midpoint.is_ready:
            # The current value of self._midpoint is represented by self._midpoint.current.value
            self.plot("MidPoint", "midpoint", self._midpoint.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MidPoint` indicator properties:



Indicator History

To get the historical data of the `MidPoint` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MidPointAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midpoint = self.midpoint(self._symbol, 20)

        indicator_history = self.indicator_history(self._midpoint, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._midpoint, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._midpoint, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Mid Price

Introduction

This indicator computes the MidPrice (MIDPRICE). The MidPrice is calculated using the following formula: MIDPRICE = (Highest High + Lowest Low) / 2

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MIDPRICE Indicator

To create an automatic indicator for `MidPrice` , call the `midprice` helper method from the `QCAlgorithm` class. The `midprice` method creates a `MidPrice` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MidPriceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midprice = self.midprice(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._midprice.is_ready:
            # The current value of self._midprice is represented by self._midprice.current.value
            self.plot("MidPrice", "midprice", self._midprice.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MidPrice` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

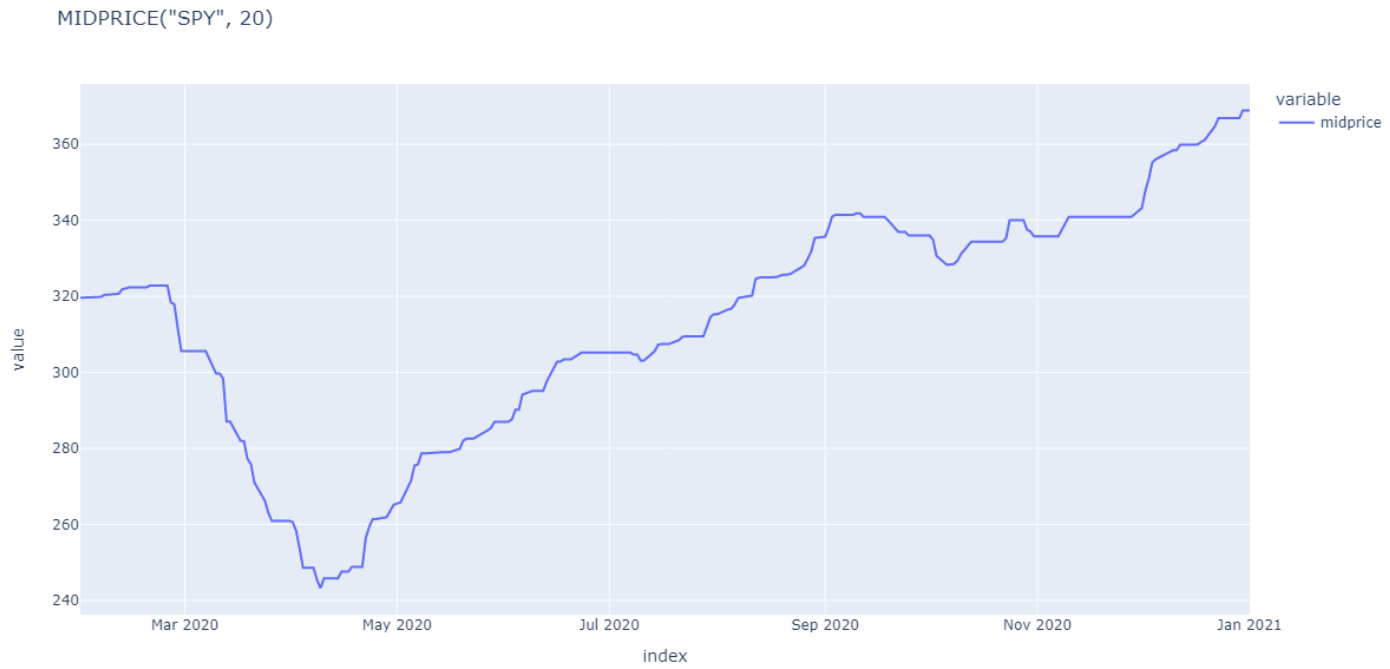
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MidPriceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midprice = MidPrice(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._midprice.update(bar)

        if self._midprice.is_ready:
            # The current value of self._midprice is represented by self._midprice.current.value
            self.plot("MidPrice", "midprice", self._midprice.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MidPrice` indicator properties:



Indicator History

To get the historical data of the `MidPrice` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MidPriceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._midprice = self.midprice(self._symbol, 20)

        indicator_history = self.indicator_history(self._midprice, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._midprice, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._midprice, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Minimum

Introduction

This indicator represents an indicator capable of tracking the minimum value and how many periods ago it occurred

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MIN Indicator

To create an automatic indicator for `Minimum` , call the `min` helper method from the `QCAlgorithm` class. The `min` method creates a `Minimum` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class MinimumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._min = self.min(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._min.is_ready:
            # The current value of self._min is represented by self._min.current.value
            self.plot("Minimum", "min", self._min.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Minimum` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

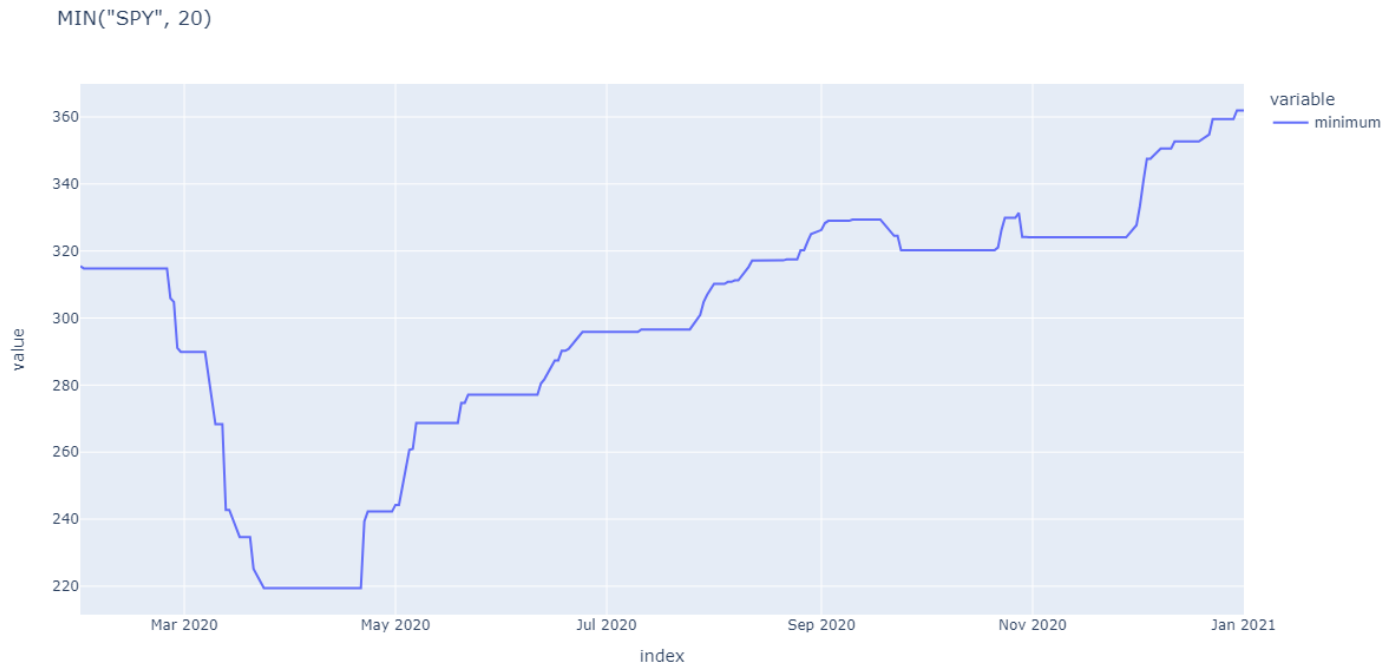
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class MinimumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._minimum = Minimum(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._minimum.update(bar.end_time, bar.close)

        if self._minimum.is_ready:
            # The current value of self._minimum is represented by self._minimum.current.value
            self.plot("Minimum", "minimum", self._minimum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Minimum` indicator properties:



Indicator History

To get the historical data of the `Minimum` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MinimumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._min = self.min(self._symbol, 20)

        indicator_history = self.indicator_history(self._min, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._min, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._min, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Momentum

Introduction

This indicator computes the n-period change in a value using the following: value_0 - value_n

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MOM Indicator

To create an automatic indicator for `Momentum` , call the `mom` helper method from the `QCAlgorithm` class. The `mom` method creates a `Momentum` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mom = self.mom(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._mom.is_ready:
            # The current value of self._mom is represented by self._mom.current.value
            self.plot("Momentum", "mom", self._mom.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Momentum` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momentum = Momentum(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._momentum.update(bar.end_time, bar.close)

        if self._momentum.is_ready:
            # The current value of self._momentum is represented by self._momentum.current.value
            self.plot("Momentum", "momentum", self._momentum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Momentum` indicator properties:

Indicator History

To get the historical data of the `Momentum` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mom = self.mom(self._symbol, 20)

        indicator_history = self.indicator_history(self._mom, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mom, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mom, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Momentum Percent

Introduction

This indicator computes the n-period percentage rate of change in a value using the following: 100 * (value_0 - value_n) / value_n This indicator yields the same results of RateOfChangePercent

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MOMP Indicator

To create an automatic indicator for `MomentumPercent` , call the `momp` helper method from the `QCAlgorithm` class. The `momp` method creates a `MomentumPercent` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MomentumPercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momp = self.momp(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._momp.is_ready:
            # The current value of self._momp is represented by self._momp.current.value
            self.plot("MomentumPercent", "momp", self._momp.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MomentumPercent` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

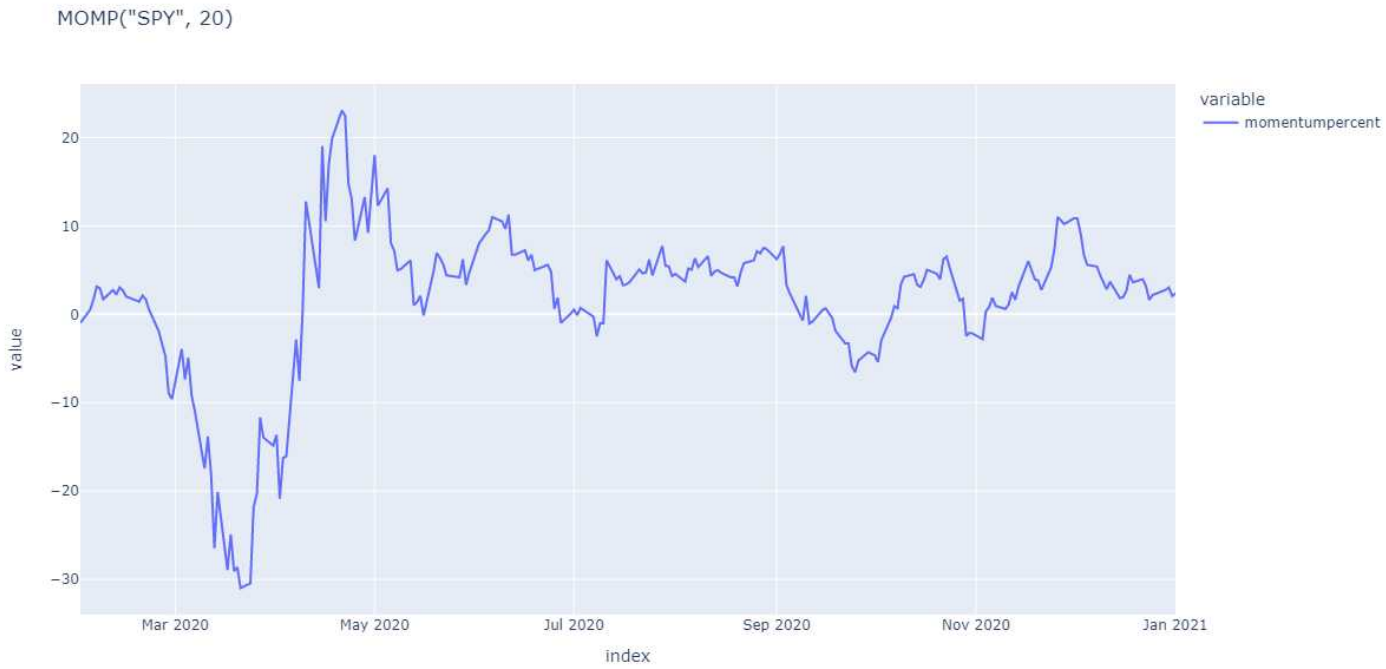
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MomentumPercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momentumpercent = MomentumPercent(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._momentumpercent.update(bar.end_time, bar.close)

        if self._momentumpercent.is_ready:
            # The current value of self._momentumpercent is represented by self._momentumpercent.current.value
            self.plot("MomentumPercent", "momentumpercent", self._momentumpercent.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MomentumPercent` indicator properties:



Indicator History

To get the historical data of the `MomentumPercent` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class MomentumPercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momp = self.momp(self._symbol, 20)

        indicator_history = self.indicator_history(self._momp, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._momp, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._momp, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Momersion

Introduction

Oscillator indicator that measures momentum and mean-reversion over a specified period n. source Harris, Michael. "Momersion Indicator." Price Action Lab., 13 Aug. 2015. Web.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MOMERSION Indicator

To create an automatic indicator for `Momersion` , call the `momersion` helper method from the `QCAlgorithm` class. The `momersion` method creates a `Momersion` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

PY

```
class MomersionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momersion = self.momersion(self._symbol, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._momersion.is_ready:
            # The current value of self._momersion is represented by self._momersion.current.value
            self.plot("Momersion", "momersion", self._momersion.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Momersion` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

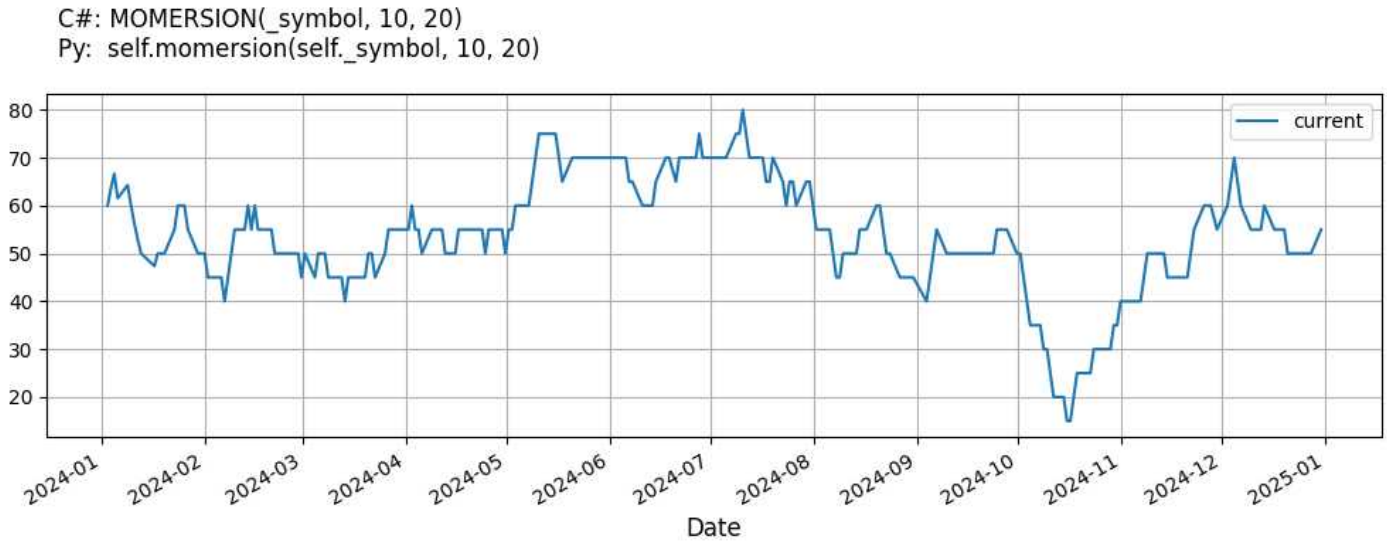
PY

```
class MomersionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momersion = Momersion(10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._momersion.update(bar.end_time, bar.close)

        if self._momersion.is_ready:
            # The current value of self._momersion is represented by self._momersion.current.value
            self.plot("Momersion", "momersion", self._momersion.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Momersion` indicator properties:



Indicator History

To get the historical data of the `Momersion` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class MomersionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._momersion = self.momersion(self._symbol, 10, 20)

        indicator_history = self.indicator_history(self._momersion, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._momersion, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._momersion, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Money Flow Index

Introduction

The Money Flow Index (MFI) is an oscillator that uses both price and volume to measure buying and selling pressure Typical Price = (High + Low + Close)/3 Money Flow = Typical Price x Volume Positive Money Flow = Sum of the money flows of all days where the typical price is greater than the previous day's typical price Negative Money Flow = Sum of the money flows of all days where the typical price is less than the previous day's typical price Money Flow Ratio = (14-period Positive Money Flow)/(14-period Negative Money Flow) Money Flow Index = 100 x Positive Money Flow / (Positive Money Flow + Negative Money Flow)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MFI Indicator

To create an automatic indicator for `MoneyFlowIndex` , call the `mfi` helper method from the `QCAlgorithm` class. The `mfi` method creates a `MoneyFlowIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MoneyFlowIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mfi = self.mfi(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._mfi.is_ready:
            # The current value of self._mfi is represented by self._mfi.current.value
            self.plot("MoneyFlowIndex", "mfi", self._mfi.current.value)
            # Plot all attributes of self._mfi
            self.plot("MoneyFlowIndex", "positive_money_flow", self._mfi.positive_money_flow.current.value)
            self.plot("MoneyFlowIndex", "negative_money_flow", self._mfi.negative_money_flow.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MoneyFlowIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

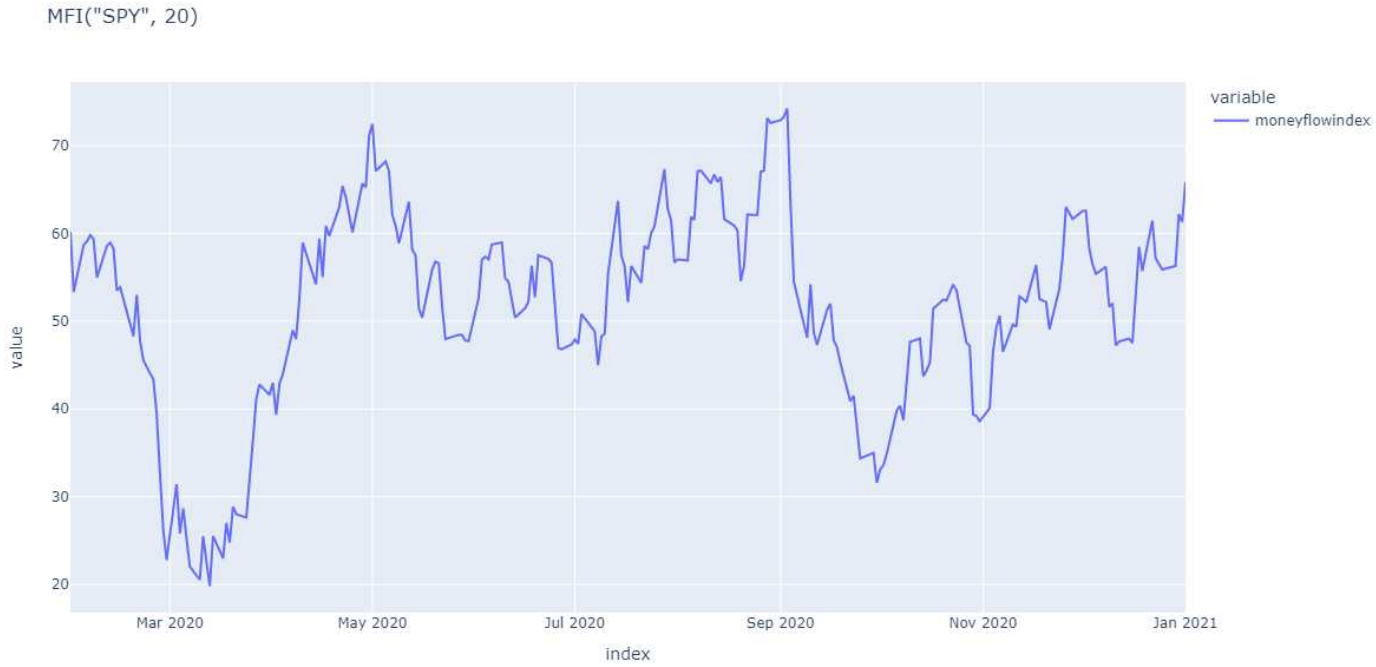
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MoneyFlowIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._moneyflowindex = MoneyFlowIndex(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._moneyflowindex.update(bar)

        if self._moneyflowindex.is_ready:
            # The current value of self._moneyflowindex is represented by self._moneyflowindex.current.value
            self.plot("MoneyFlowIndex", "moneyflowindex", self._moneyflowindex.current.value)
            # Plot all attributes of self._moneyflowindex
            self.plot("MoneyFlowIndex", "positive_money_flow", self._moneyflowindex.positive_money_flow.current.value)
            self.plot("MoneyFlowIndex", "negative_money_flow", self._moneyflowindex.negative_money_flow.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MoneyFlowIndex` indicator properties:



Indicator History

To get the historical data of the `MoneyFlowIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class MoneyFlowIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._mfi = self.mfi(self._symbol, 20)

        indicator_history = self.indicator_history(self._mfi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._mfi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._mfi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        positive_money_flow = indicator_history_df["positivemoneyflow"]
        negative_money_flow = indicator_history_df["negativemoneyflow"]
```

Supported Indicators

Moving Average Convergence Divergence

Introduction

This indicator creates two moving averages defined on a base indicator and produces the difference between the fast and slow averages.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using MACD Indicator

To create an automatic indicator for `MovingAverageConvergenceDivergence` , call the `macd` helper method from the `QCAlgorithm` class. The `macd` method creates a `MovingAverageConvergenceDivergence` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class MovingAverageConvergenceDivergenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._macd = self.macd(self._symbol, 12, 26, 9, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:

        if self._macd.is_ready:
            # The current value of self._macd is represented by self._macd.current.value
            self.plot("MovingAverageConvergenceDivergence", "macd", self._macd.current.value)
            # Plot all attributes of self._macd
            self.plot("MovingAverageConvergenceDivergence", "fast", self._macd.fast.current.value)
            self.plot("MovingAverageConvergenceDivergence", "slow", self._macd.slow.current.value)
            self.plot("MovingAverageConvergenceDivergence", "signal", self._macd.signal.current.value)
            self.plot("MovingAverageConvergenceDivergence", "histogram", self._macd.histogram.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `MovingAverageConvergenceDivergence` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

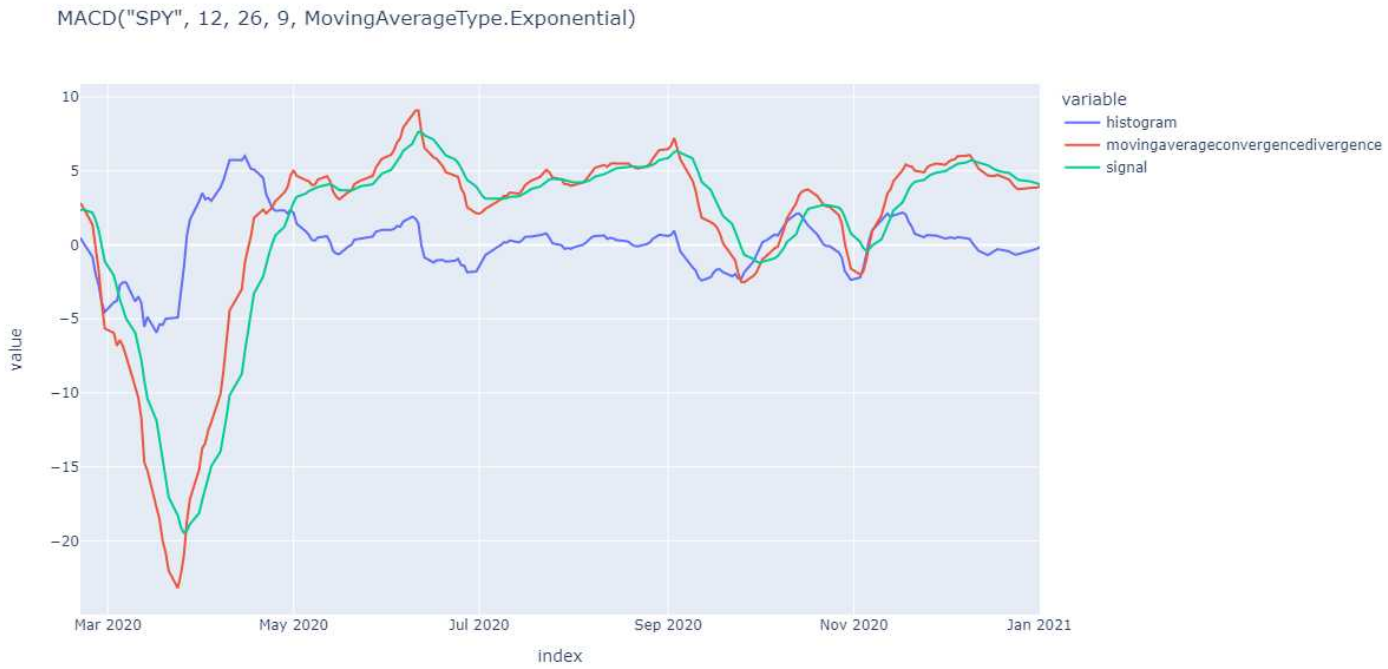
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class MovingAverageConvergenceDivergenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._movingaverageconvergencedivergence = MovingAverageConvergenceDivergence(12, 26, 9, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._movingaverageconvergencedivergence.update(bar.end_time, bar.close)

        if self._movingaverageconvergencedivergence.is_ready:
            # The current value of self._movingaverageconvergencedivergence is represented by
self._movingaverageconvergencedivergence.current.value
            self.plot("MovingAverageConvergenceDivergence", "movingaverageconvergencedivergence",
self._movingaverageconvergencedivergence.current.value)
            # Plot all attributes of self._movingaverageconvergencedivergence
            self.plot("MovingAverageConvergenceDivergence", "fast", self._movingaverageconvergencedivergence.fast.current.value)
            self.plot("MovingAverageConvergenceDivergence", "slow", self._movingaverageconvergencedivergence.slow.current.value)
            self.plot("MovingAverageConvergenceDivergence", "signal", self._movingaverageconvergencedivergence.signal.current.value)
            self.plot("MovingAverageConvergenceDivergence", "histogram",
self._movingaverageconvergencedivergence.histogram.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `MovingAverageConvergenceDivergence` indicator properties:



Indicator History

To get the historical data of the `MovingAverageConvergenceDivergence` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class MovingAverageConvergenceDivergenceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._macd = self.macd(self._symbol, 12, 26, 9, MovingAverageType.EXPONENTIAL)

        indicator_history = self.indicator_history(self._macd, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._macd, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._macd, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        fast = indicator_history_df["fast"]
        slow = indicator_history_df["slow"]
        signal = indicator_history_df["signal"]
        histogram = indicator_history_df["histogram"]
```

Supported Indicators

New Highs New Lows

Introduction

The NewHighsNewLows is a New Highs / New Lows indicator that accepts IBaseDataBar data as its input. This type is more of a shim/typedef to reduce the need to refer to things as NewHighsNewLows<IBaseDataBar>

To view the implementation of this indicator, see the LEAN GitHub repository .

Using NHNL Indicator

To create an automatic indicator for `NewHighsNewLows` , call the `nhnl` helper method from the `QCAlgorithm` class. The `nhnl` method creates a `NewHighsNewLows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class NewHighsNewLowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._nhnl = self.nhnl([self._symbol, self._reference], 2)

    def on_data(self, slice: Slice) -> None:

        if self._nhnl.is_ready:
            # The current value of self._nhnl is represented by self._nhnl.current.value
            self.plot("NewHighsNewLows", "nhnl", self._nhnl.current.value)
            # Plot all attributes of self._nhnl
            self.plot("NewHighsNewLows", "difference", self._nhnl.difference.current.value)
            self.plot("NewHighsNewLows", "ratio", self._nhnl.ratio.current.value)
```

To create an automatic indicator for `NewHighsNewLows` using universe constituents, call the `nhnl` helper method from the `QCAlgorithm` class. The `nhnl` method creates a `NewHighsNewLows` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class NewHighsNewLowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._nhnl = self.nhnl(list(self._universe.selected), 2)

    def on_data(self, slice: Slice) -> None:

        if self._nhnl.is_ready:
            # The current value of self._nhnl is represented by self._nhnl.current.value
            self.plot("NewHighsNewLows", "nhnl", self._nhnl.current.value)
            # Plot all attributes of self._nhnl
            self.plot("NewHighsNewLows", "difference", self._nhnl.difference.current.value)
            self.plot("NewHighsNewLows", "ratio", self._nhnl.ratio.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `NewHighsNewLows` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```py
class NewHighsNewLowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._newhighsnewlows = NewHighsNewLows("NHNL", 2)
        self._newhighsnewlows.add(self._symbol);
        self._newhighsnewlows.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._newhighsnewlows.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._newhighsnewlows.update(bar)

        if self._newhighsnewlows.is_ready:
            # The current value of self._newhighsnewlows is represented by self._newhighsnewlows.current.value
            self.plot("NewHighsNewLows", "newhighsnewlows", self._newhighsnewlows.current.value)
            # Plot all attributes of self._newhighsnewlows
            self.plot("NewHighsNewLows", "difference", self._newhighsnewlows.difference.current.value)
            self.plot("NewHighsNewLows", "ratio", self._newhighsnewlows.ratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `NewHighsNewLows` indicator properties:

Indicator History

To get the historical data of the `NewHighsNewLows` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class NewHighsNewLowsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._nhnl = self.nhnl([self._symbol, self._reference], 2)

        indicator_history = self.indicator_history(self._nhnl, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._nhnl, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._nhnl, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        difference = indicator_history_df["difference"]
        ratio = indicator_history_df["ratio"]
```

Supported Indicators

New Highs New Lows Volume

Introduction

The New Highs - New Lows Volume Ratio is a Breadth indicator calculated as ratio of summary volume of stocks reaching new high to summary volume of stocks reaching new low compared to high and low values in defined time period.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using NHNLV Indicator

To create an automatic indicator for `NewHighsNewLowsVolume` , call the `nhnlv` helper method from the `QCAlgorithm` class. The `nhnlv` method creates a `NewHighsNewLowsVolume` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class NewHighsNewLowsVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._nhnlv = self.nhnlv([self._symbol, self._reference], 2)

    def on_data(self, slice: Slice) -> None:

        if self._nhnlv.is_ready:
            # The current value of self._nhnlv is represented by self._nhnlv.current.value
            self.plot("NewHighsNewLowsVolume", "nhnlv", self._nhnlv.current.value)
            # Plot all attributes of self._nhnlv
            self.plot("NewHighsNewLowsVolume", "volume_ratio", self._nhnlv.volume_ratio.current.value)
            self.plot("NewHighsNewLowsVolume", "difference", self._nhnlv.difference.current.value)
            self.plot("NewHighsNewLowsVolume", "ratio", self._nhnlv.ratio.current.value)
```

To create an automatic indicator for `NewHighsNewLowsVolume` using universe constituents, call the `nhnlv` helper method from the `QCAlgorithm` class. The `nhnlv` method creates a `NewHighsNewLowsVolume` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In this case, you should call the helper method in the `on_securities_changed` method.

```
PY
class NewHighsNewLowsVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024,1,1)
        self.set_end_date(2024,12,31)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start())
        self._universe = self.add_universe(self.universe.etf('SPY'))

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        self._nhnlv = self.nhnlv(list(self._universe.selected), 2)

    def on_data(self, slice: Slice) -> None:

        if self._nhnlv.is_ready:
            # The current value of self._nhnlv is represented by self._nhnlv.current.value
            self.plot("NewHighsNewLowsVolume", "nhnlv", self._nhnlv.current.value)
            # Plot all attributes of self._nhnlv
            self.plot("NewHighsNewLowsVolume", "volume_ratio", self._nhnlv.volume_ratio.current.value)
            self.plot("NewHighsNewLowsVolume", "difference", self._nhnlv.difference.current.value)
            self.plot("NewHighsNewLowsVolume", "ratio", self._nhnlv.ratio.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `NewHighsNewLowsVolume` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```py
class NewHighsNewLowsVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._newhighsnewlowsvolume = NewHighsNewLowsVolume("NHNLV", 2)
        self._newhighsnewlowsvolume.add(self._symbol);
        self._newhighsnewlowsvolume.add(self._reference);

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._newhighsnewlowsvolume.update(bar)
        bar = slice.bars.get(self._reference)
        if bar:
            self._newhighsnewlowsvolume.update(bar)

        if self._newhighsnewlowsvolume.is_ready:
            # The current value of self._newhighsnewlowsvolume is represented by self._newhighsnewlowsvolume.current.value
            self.plot("NewHighsNewLowsVolume", "newhighsnewlowsvolume", self._newhighsnewlowsvolume.current.value)
            # Plot all attributes of self._newhighsnewlowsvolume
            self.plot("NewHighsNewLowsVolume", "volume_ratio", self._newhighsnewlowsvolume.volume_ratio.current.value)
            self.plot("NewHighsNewLowsVolume", "difference", self._newhighsnewlowsvolume.difference.current.value)
            self.plot("NewHighsNewLowsVolume", "ratio", self._newhighsnewlowsvolume.ratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `NewHighsNewLowsVolume` indicator properties:

Indicator History

To get the historical data of the `NewHighsNewLowsVolume` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class NewHighsNewLowsVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("QQQ", Resolution.DAILY).symbol
        self._reference = self.add_equity("SPY", Resolution.DAILY).symbol
        self._nhnlv = self.nhnlv([self._symbol, self._reference], 2)

        indicator_history = self.indicator_history(self._nhnlv, [ self._symbol, self._reference ], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._nhnlv, [ self._symbol, self._reference ], timedelta(days=10),
Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._nhnlv, [ self._symbol, self._reference ], datetime(2024, 7, 1),
datetime(2024, 7, 5), Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        volume_ratio = indicator_history_df["volumeratio"]
        difference = indicator_history_df["difference"]
        ratio = indicator_history_df["ratio"]
```

Supported Indicators

Normalized Average True Range

Introduction

This indicator computes the Normalized Average True Range (NATR). The Normalized Average True Range is calculated with the following formula: NATR = (ATR(period) / Close) * 100

To view the implementation of this indicator, see the LEAN GitHub repository .

Using NATR Indicator

To create an automatic indicator for `NormalizedAverageTrueRange` , call the `natr` helper method from the `QCAlgorithm` class. The `natr` method creates a `NormalizedAverageTrueRange` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class NormalizedAverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._natr = self.natr(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._natr.is_ready:
            # The current value of self._natr is represented by self._natr.current.value
            self.plot("NormalizedAverageTrueRange", "natr", self._natr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `NormalizedAverageTrueRange` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

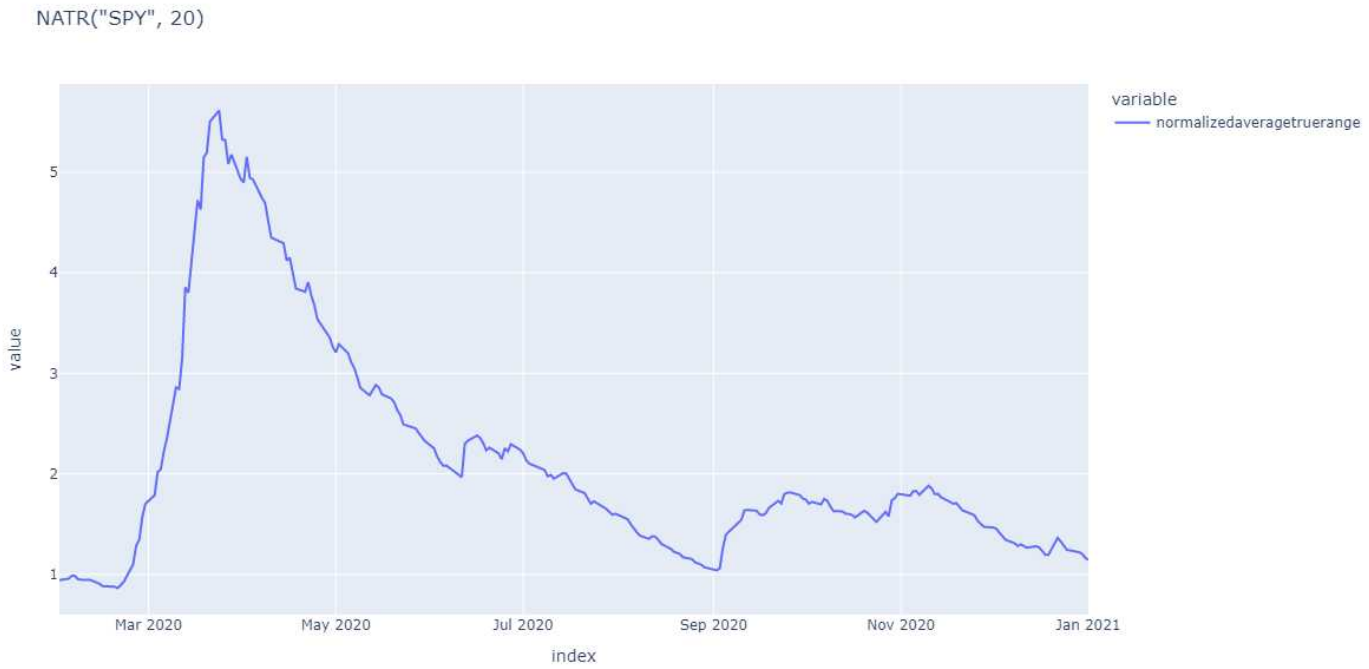
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class NormalizedAverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._normalizedaveragetruerange = NormalizedAverageTrueRange(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._normalizedaveragetruerange.update(bar)

        if self._normalizedaveragetruerange.is_ready:
            # The current value of self._normalizedaveragetruerange is represented by self._normalizedaveragetruerange.current.value
            self.plot("NormalizedAverageTrueRange", "normalizedaveragetruerange", self._normalizedaveragetruerange.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `NormalizedAverageTrueRange` indicator properties:



Indicator History

To get the historical data of the `NormalizedAverageTrueRange` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class NormalizedAverageTrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._natr = self.natr(self._symbol, 20)

        indicator_history = self.indicator_history(self._natr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._natr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._natr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

On Balance Volume

Introduction

This indicator computes the On Balance Volume (OBV). The On Balance Volume is calculated by determining the price of the current close price and previous close price. If the current close price is equivalent to the previous price the OBV remains the same, If the current close price is higher the volume of that day is added to the OBV, while a lower close price will result in negative value.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using OBV Indicator

To create an automatic indicator for `OnBalanceVolume` , call the `obv` helper method from the `QCAlgorithm` class. The `obv` method creates a `OnBalanceVolume` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class OnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._obv = self.obv(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._obv.is_ready:
            # The current value of self._obv is represented by self._obv.current.value
            self.plot("OnBalanceVolume", "obv", self._obv.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `OnBalanceVolume` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

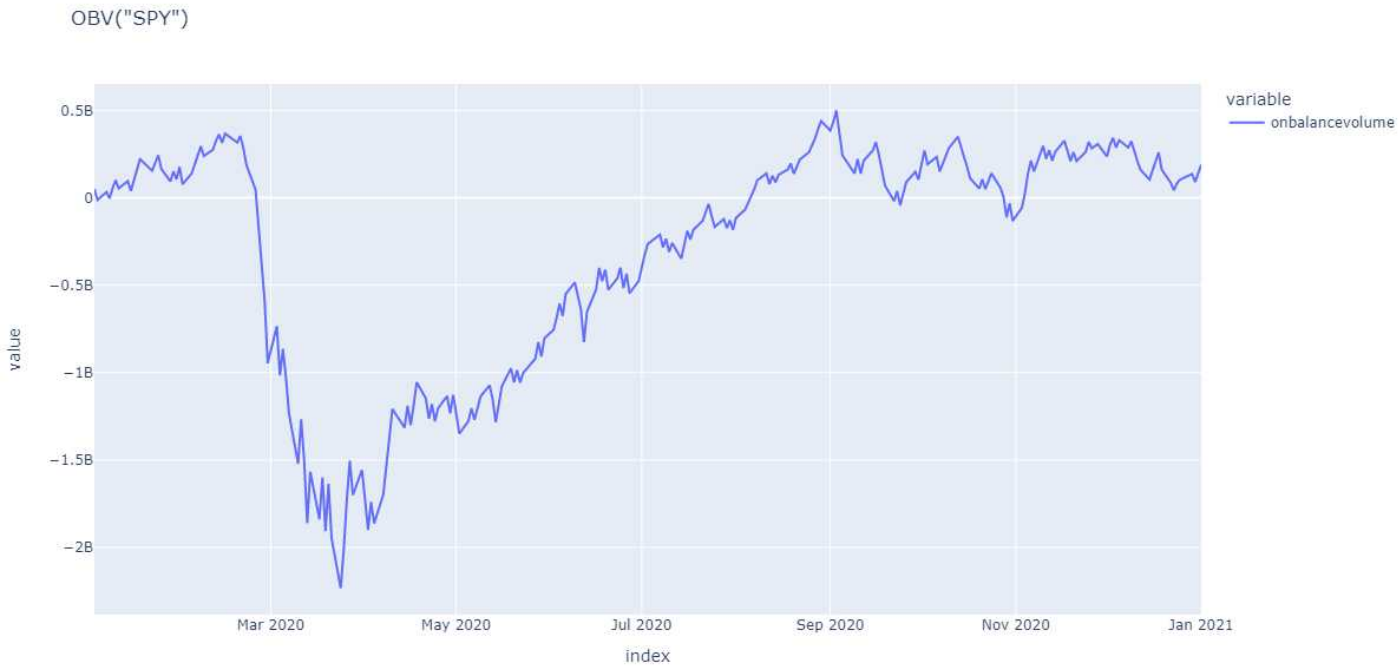
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class OnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._onbalancevolume = OnBalanceVolume()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._onbalancevolume.update(bar)

        if self._onbalancevolume.is_ready:
            # The current value of self._onbalancevolume is represented by self._onbalancevolume.current.value
            self.plot("OnBalanceVolume", "onbalancevolume", self._onbalancevolume.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `OnBalanceVolume` indicator properties:



Indicator History

To get the historical data of the `OnBalanceVolume` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class OnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._obv = self.obv(self._symbol)

        indicator_history = self.indicator_history(self._obv, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._obv, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._obv, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Parabolic Stop And Reverse

Introduction

Parabolic SAR Indicator Based on TA-Lib implementation

To view the implementation of this indicator, see the LEAN GitHub repository .

Using PSAR Indicator

To create an automatic indicator for `ParabolicStopAndReverse` , call the `psar` helper method from the `QCAlgorithm` class. The `psar` method creates a `ParabolicStopAndReverse` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ParabolicStopAndReverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._psar = self.psar(self._symbol, 0.02, 0.02, 0.2)

    def on_data(self, slice: Slice) -> None:

        if self._psar.is_ready:
            # The current value of self._psar is represented by self._psar.current.value
            self.plot("ParabolicStopAndReverse", "psar", self._psar.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ParabolicStopAndReverse` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

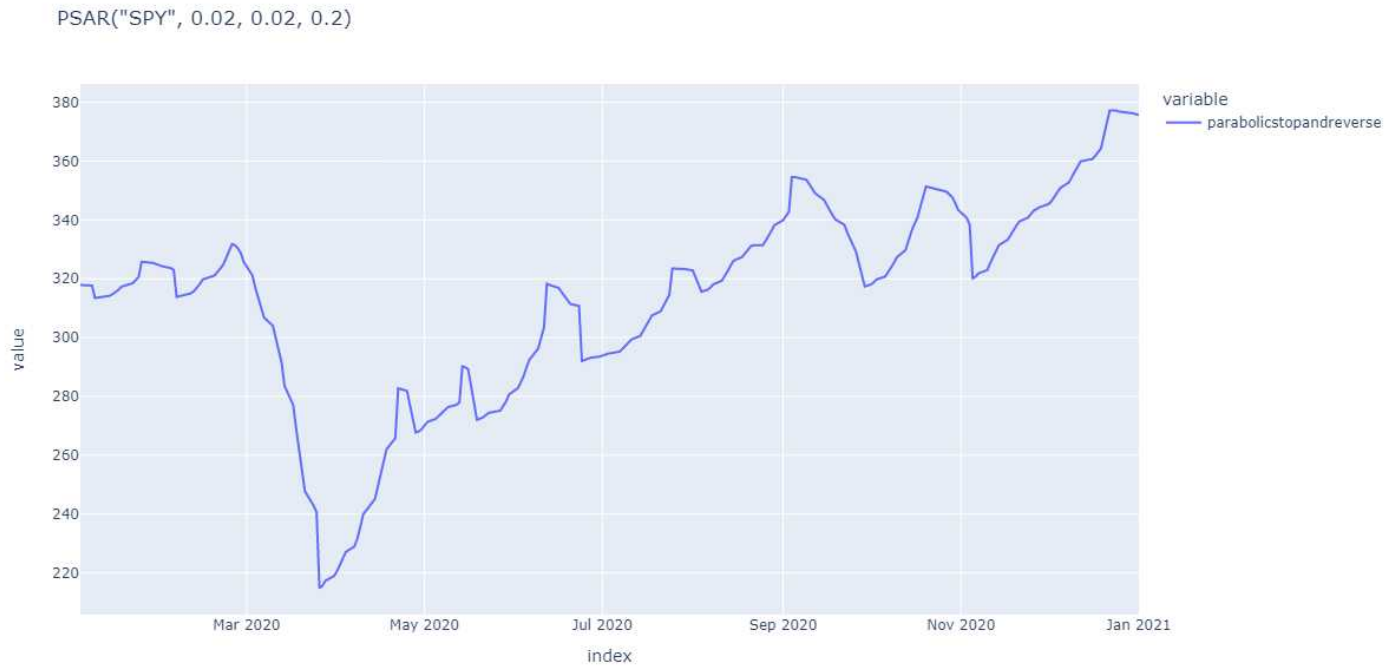
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ParabolicStopAndReverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._parabolicstopandreverse = ParabolicStopAndReverse(0.02, 0.02, 0.2)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._parabolicstopandreverse.update(bar)

        if self._parabolicstopandreverse.is_ready:
            # The current value of self._parabolicstopandreverse is represented by self._parabolicstopandreverse.current.value
            self.plot("ParabolicStopAndReverse", "parabolicstopandreverse", self._parabolicstopandreverse.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ParabolicStopAndReverse` indicator properties:



Indicator History

To get the historical data of the `ParabolicStopAndReverse` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class ParabolicStopAndReverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._psar = self.psar(self._symbol, 0.02, 0.02, 0.2)

        indicator_history = self.indicator_history(self._psar, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._psar, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._psar, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Parabolic Stop And Reverse Extended

Introduction

Parabolic SAR Extended Indicator Based on TA-Lib implementation

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SAREXT Indicator

To create an automatic indicator for `ParabolicStopAndReverseExtended` , call the `sarext` helper method from the `QCAlgorithm` class. The `sarext` method creates a `ParabolicStopAndReverseExtended` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ParabolicStopAndReverseExtendedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sarext = self.sarext(self._symbol, 0.0, 0.0, 0.02, 0.02, 0.2, 0.02, 0.02, 0.2)

    def on_data(self, slice: Slice) -> None:

        if self._sarext.is_ready:
            # The current value of self._sarext is represented by self._sarext.current.value
            self.plot("ParabolicStopAndReverseExtended", "sarext", self._sarext.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ParabolicStopAndReverseExtended` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

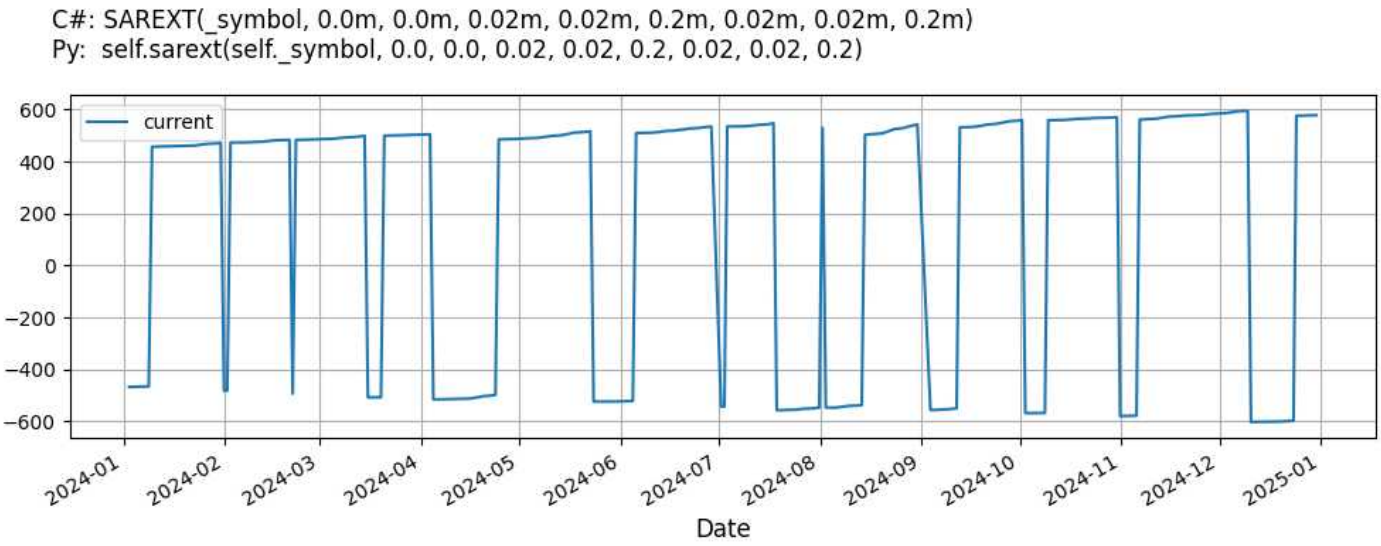
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ParabolicStopAndReverseExtendedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._parabolicstopandreverseextended = ParabolicStopAndReverseExtended(0.0, 0.0, 0.02, 0.02, 0.2, 0.02, 0.02, 0.2)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._parabolicstopandreverseextended.update(bar)

        if self._parabolicstopandreverseextended.is_ready:
            # The current value of self._parabolicstopandreverseextended is represented by
self._parabolicstopandreverseextended.current.value
            self.plot("ParabolicStopAndReverseExtended", "parabolicstopandreverseextended",
self._parabolicstopandreverseextended.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ParabolicStopAndReverseExtended` indicator properties:



Indicator History

To get the historical data of the `ParabolicStopAndReverseExtended` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```py
class ParabolicStopAndReverseExtendedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sarext = self.sarext(self._symbol, 0.0, 0.0, 0.02, 0.02, 0.2, 0.02, 0.02, 0.2)

        indicator_history = self.indicator_history(self._sarext, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sarext, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sarext, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Percentage Price Oscillator

Introduction

This indicator computes the Percentage Price Oscillator (PPO) The Percentage Price Oscillator is calculated using the following formula: PPO[i] = 100 * (FastMA[i] - SlowMA[i]) / SlowMA[i]

To view the implementation of this indicator, see the LEAN GitHub repository .

Using PPO Indicator

To create an automatic indicator for `PercentagePriceOscillator` , call the `ppo` helper method from the `QCAlgorithm` class. The `ppo` method creates a `PercentagePriceOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class PercentagePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ppo = self.ppo(self._symbol, 10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._ppo.is_ready:
            # The current value of self._ppo is represented by self._ppo.current.value
            self.plot("PercentagePriceOscillator", "ppo", self._ppo.current.value)
            # Plot all attributes of self._ppo
            self.plot("PercentagePriceOscillator", "fast", self._ppo.fast.current.value)
            self.plot("PercentagePriceOscillator", "slow", self._ppo.slow.current.value)
            self.plot("PercentagePriceOscillator", "signal", self._ppo.signal.current.value)
            self.plot("PercentagePriceOscillator", "histogram", self._ppo.histogram.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `PercentagePriceOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

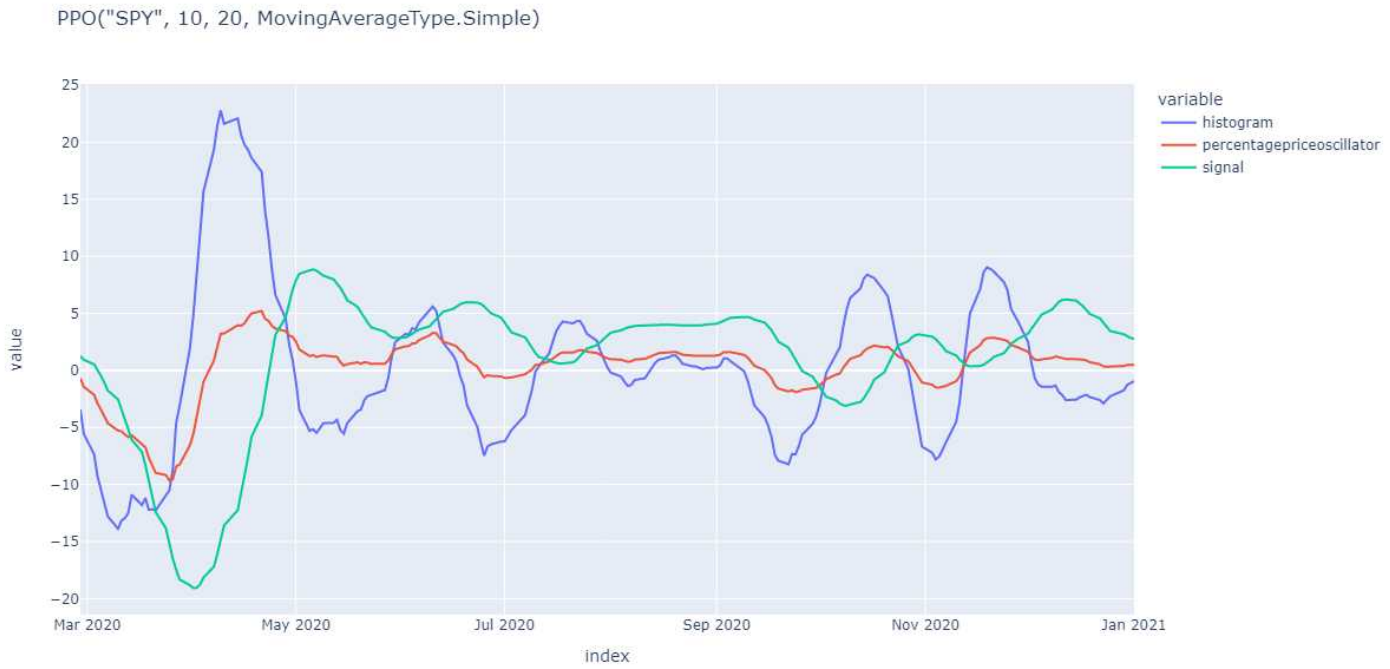
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class PercentagePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._percentagepriceoscillator = PercentagePriceOscillator(10, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._percentagepriceoscillator.update(bar.end_time, bar.close)

        if self._percentagepriceoscillator.is_ready:
            # The current value of self._percentagepriceoscillator is represented by self._percentagepriceoscillator.current.value
            self.plot("PercentagePriceOscillator", "percentagepriceoscillator", self._percentagepriceoscillator.current.value)
            # Plot all attributes of self._percentagepriceoscillator
            self.plot("PercentagePriceOscillator", "fast", self._percentagepriceoscillator.fast.current.value)
            self.plot("PercentagePriceOscillator", "slow", self._percentagepriceoscillator.slow.current.value)
            self.plot("PercentagePriceOscillator", "signal", self._percentagepriceoscillator.signal.current.value)
            self.plot("PercentagePriceOscillator", "histogram", self._percentagepriceoscillator.histogram.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `PercentagePriceOscillator` indicator properties:



Indicator History

To get the historical data of the `PercentagePriceOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class PercentagePriceOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ppo = self.ppo(self._symbol, 10, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._ppo, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ppo, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ppo, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        fast = indicator_history_df["fast"]
        slow = indicator_history_df["slow"]
        signal = indicator_history_df["signal"]
        histogram = indicator_history_df["histogram"]
```

Supported Indicators

Pivot Points High Low

Introduction

Pivot Points (High/Low), also known as Bar Count Reversals, indicator. https://www.fidelity.com/learning-center/trading-investing/technical-analysis/technical-indicator-guide/pivot-points-high-low

To view the implementation of this indicator, see the LEAN GitHub repository .

Using PPHL Indicator

To create an automatic indicator for `PivotPointsHighLow` , call the `pphl` helper method from the `QCAlgorithm` class. The `pphl` method creates a `PivotPointsHighLow` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class PivotPointsHighLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._pphl = self.pphl(self._symbol, 10, 10, 100)

    def on_data(self, slice: Slice) -> None:

        if self._pphl.is_ready:
            # The current value of self._pphl is represented by self._pphl.current.value
            self.plot("PivotPointsHighLow", "pphl", self._pphl.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `PivotPointsHighLow` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

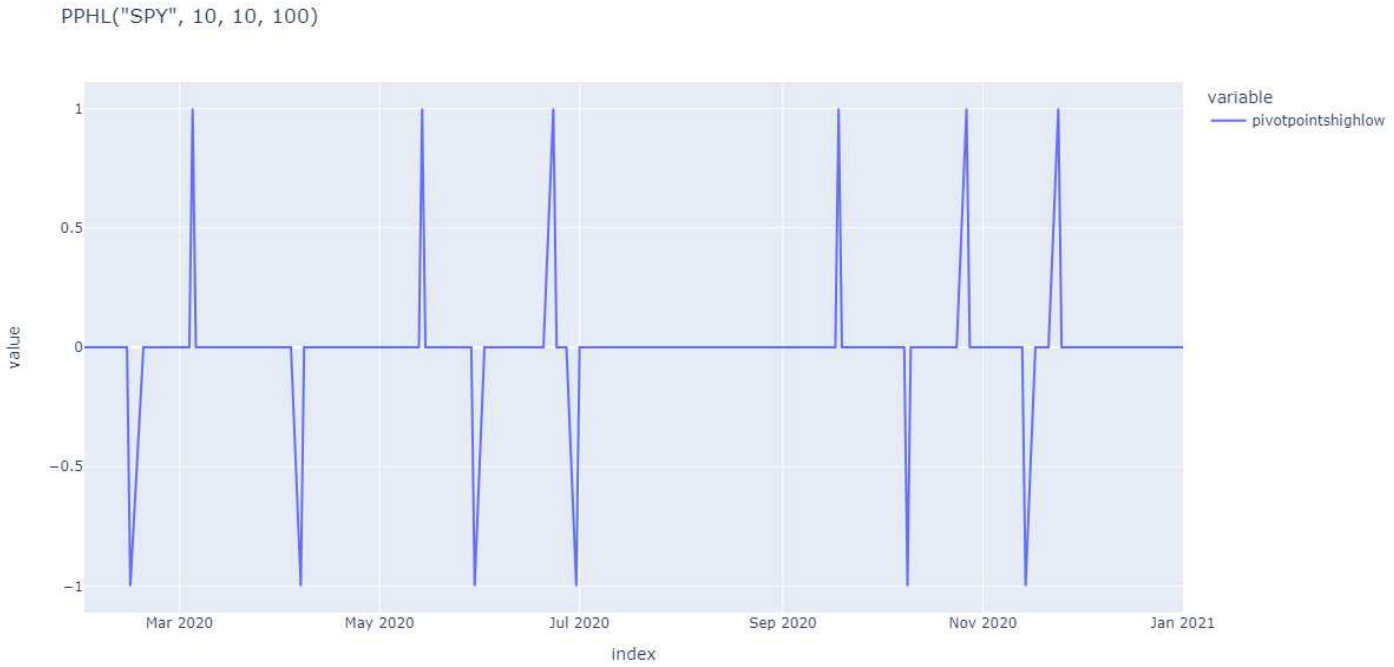
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class PivotPointsHighLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._pivotpointshighlow = PivotPointsHighLow(10, 10, 100)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._pivotpointshighlow.update(bar)

        if self._pivotpointshighlow.is_ready:
            # The current value of self._pivotpointshighlow is represented by self._pivotpointshighlow.current.value
            self.plot("PivotPointsHighLow", "pivotpointshighlow", self._pivotpointshighlow.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `PivotPointsHighLow` indicator properties:



Indicator History

To get the historical data of the `PivotPointsHighLow` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class PivotPointsHighLowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._pphl = self.pphl(self._symbol, 10, 10, 100)

        indicator_history = self.indicator_history(self._pphl, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._pphl, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._pphl, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Premier Stochastic Oscillator

Introduction

Premier Stochastic Oscillator (PSO) Indicator implementation. This indicator combines a stochastic oscillator with exponential moving averages to provide a normalized output between -1 and 1, which can be useful for identifying trends and potential reversal points in the market.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using PSO Indicator

To create an automatic indicator for `PremierStochasticOscillator` , call the `pso` helper method from the `QCAlgorithm` class. The `pso` method creates a `PremierStochasticOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class PremierStochasticOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._pso = self.pso(self._symbol, 14, 3)

    def on_data(self, slice: Slice) -> None:

        if self._pso.is_ready:
            # The current value of self._pso is represented by self._pso.current.value
            self.plot("PremierStochasticOscillator", "pso", self._pso.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `PremierStochasticOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

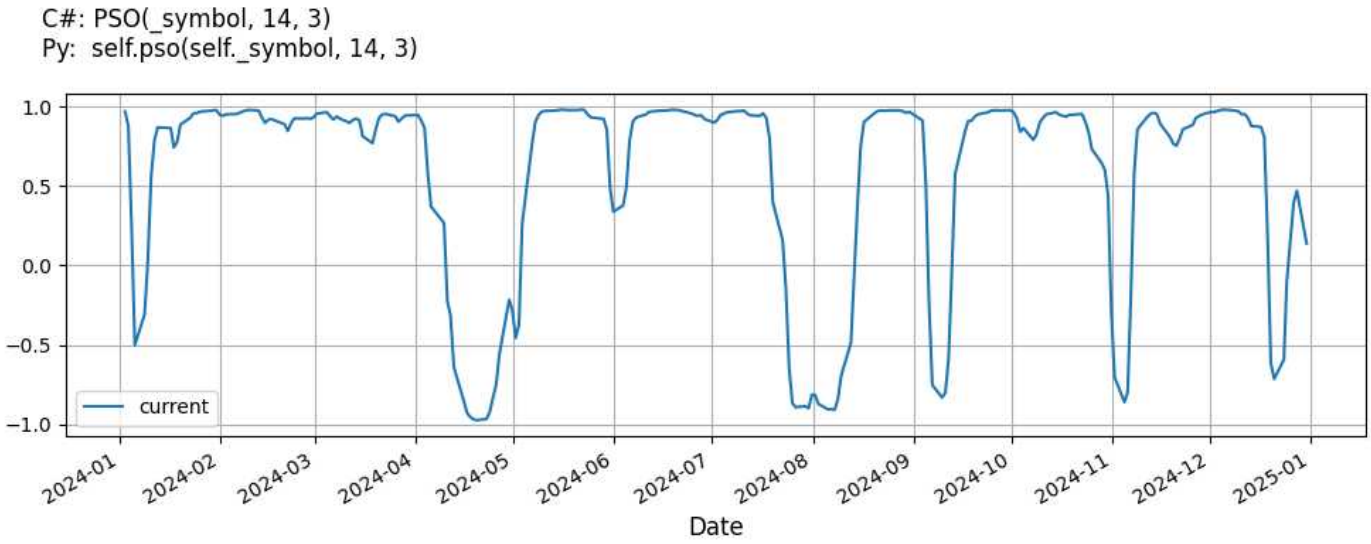
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class PremierStochasticOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._premierstochasticoscillator = PremierStochasticOscillator(14, 3)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._premierstochasticoscillator.update(bar)

        if self._premierstochasticoscillator.is_ready:
            # The current value of self._premierstochasticoscillator is represented by self._premierstochasticoscillator.current.value
            self.plot("PremierStochasticOscillator", "premierstochasticoscillator", self._premierstochasticoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `PremierStochasticOscillator` indicator properties:



Indicator History

To get the historical data of the `PremierStochasticOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class PremierStochasticOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._pso = self.pso(self._symbol, 14, 3)

        indicator_history = self.indicator_history(self._pso, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._pso, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._pso, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Rate Of Change

Introduction

This indicator computes the n-period rate of change in a value using the following: (value_0 - value_n) / value_n

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ROC Indicator

To create an automatic indicator for `RateOfChange` , call the `roc` helper method from the `QCAlgorithm` class. The `roc` method creates a `RateOfChange` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class RateOfChangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._roc = self.roc(self._symbol, 10)

    def on_data(self, slice: Slice) -> None:

        if self._roc.is_ready:
            # The current value of self._roc is represented by self._roc.current.value
            self.plot("RateOfChange", "roc", self._roc.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RateOfChange` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class RateOfChangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rateofchange = RateOfChange(10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rateofchange.update(bar.end_time, bar.close)

        if self._rateofchange.is_ready:
            # The current value of self._rateofchange is represented by self._rateofchange.current.value
            self.plot("RateOfChange", "rateofchange", self._rateofchange.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RateOfChange` indicator properties:

Indicator History

To get the historical data of the `RateOfChange` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RateOfChangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._roc = self.roc(self._symbol, 10)

        indicator_history = self.indicator_history(self._roc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._roc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._roc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Rate Of Change Percent

Introduction

This indicator computes the n-period percentage rate of change in a value using the following: 100 * (value_0 - value_n) / value_n

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ROCP Indicator

To create an automatic indicator for `RateOfChangePercent` , call the `rocp` helper method from the `QCAlgorithm` class. The `rocp` method creates a `RateOfChangePercent` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class RateOfChangePercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rocp = self.rocp(self._symbol, 10)

    def on_data(self, slice: Slice) -> None:

        if self._rocp.is_ready:
            # The current value of self._rocp is represented by self._rocp.current.value
            self.plot("RateOfChangePercent", "rocp", self._rocp.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RateOfChangePercent` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class RateOfChangePercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rateofchangepercent = RateOfChangePercent(10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rateofchangepercent.update(bar.end_time, bar.close)

        if self._rateofchangepercent.is_ready:
            # The current value of self._rateofchangepercent is represented by self._rateofchangepercent.current.value
            self.plot("RateOfChangePercent", "rateofchangepercent", self._rateofchangepercent.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RateOfChangePercent` indicator properties:

Indicator History

To get the historical data of the `RateOfChangePercent` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RateOfChangePercentAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rocp = self.rocp(self._symbol, 10)

        indicator_history = self.indicator_history(self._rocp, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rocp, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rocp, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Rate Of Change Ratio

Introduction

This indicator computes the Rate Of Change Ratio (ROCR). The Rate Of Change Ratio is calculated with the following formula: ROCR = price / prevPrice

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ROCR Indicator

To create an automatic indicator for `RateOfChangeRatio` , call the `rocr` helper method from the `QCAlgorithm` class. The `rocr` method creates a `RateOfChangeRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class RateOfChangeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rocr = self.rocr(self._symbol, 10)

    def on_data(self, slice: Slice) -> None:

        if self._rocr.is_ready:
            # The current value of self._rocr is represented by self._rocr.current.value
            self.plot("RateOfChangeRatio", "rocr", self._rocr.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `RateOfChangeRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class RateOfChangeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rateofchangeratio = RateOfChangeRatio(10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rateofchangeratio.update(bar.end_time, bar.close)

        if self._rateofchangeratio.is_ready:
            # The current value of self._rateofchangeratio is represented by self._rateofchangeratio.current.value
            self.plot("RateOfChangeRatio", "rateofchangeratio", self._rateofchangeratio.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RateOfChangeRatio` indicator properties:

Indicator History

To get the historical data of the `RateOfChangeRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RateOfChangeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rocr = self.rocr(self._symbol, 10)

        indicator_history = self.indicator_history(self._rocr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rocr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rocr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Regression Channel

Introduction

The Regression Channel indicator extends the with the inclusion of two (upper and lower) channel lines that are distanced from the linear regression line by a user defined number of standard deviations. Reference: http://www.onlinetradingconcepts.com/TechnicalAnalysis/LinRegChannel.html

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RC Indicator

To create an automatic indicator for `RegressionChannel` , call the `rc` helper method from the `QCAlgorithm` class. The `rc` method creates a `RegressionChannel` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class RegressionChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rc = self.rc(self._symbol, 20, 2)

    def on_data(self, slice: Slice) -> None:

        if self._rc.is_ready:
            # The current value of self._rc is represented by self._rc.current.value
            self.plot("RegressionChannel", "rc", self._rc.current.value)
            # Plot all attributes of self._rc
            self.plot("RegressionChannel", "linear_regression", self._rc.linear_regression.current.value)
            self.plot("RegressionChannel", "upper_channel", self._rc.upper_channel.current.value)
            self.plot("RegressionChannel", "lower_channel", self._rc.lower_channel.current.value)
            self.plot("RegressionChannel", "intercept", self._rc.intercept.current.value)
            self.plot("RegressionChannel", "slope", self._rc.slope.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RegressionChannel` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class RegressionChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._regressionchannel = RegressionChannel(20, 2)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._regressionchannel.update(bar.end_time, bar.close)

        if self._regressionchannel.is_ready:
            # The current value of self._regressionchannel is represented by self._regressionchannel.current.value
            self.plot("RegressionChannel", "regressionchannel", self._regressionchannel.current.value)
            # Plot all attributes of self._regressionchannel
            self.plot("RegressionChannel", "linear_regression", self._regressionchannel.linear_regression.current.value)
            self.plot("RegressionChannel", "upper_channel", self._regressionchannel.upper_channel.current.value)
            self.plot("RegressionChannel", "lower_channel", self._regressionchannel.lower_channel.current.value)
            self.plot("RegressionChannel", "intercept", self._regressionchannel.intercept.current.value)
            self.plot("RegressionChannel", "slope", self._regressionchannel.slope.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RegressionChannel` indicator properties:

Indicator History

To get the historical data of the `RegressionChannel` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RegressionChannelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rc = self.rc(self._symbol, 20, 2)

        indicator_history = self.indicator_history(self._rc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        linear_regression = indicator_history_df["linearregression"]
        upper_channel = indicator_history_df["upperchannel"]
        lower_channel = indicator_history_df["lowerchannel"]
        intercept = indicator_history_df["intercept"]
        slope = indicator_history_df["slope"]
```

Supported Indicators

Relative Daily Volume

Introduction

The Relative Daily Volume indicator is an indicator that compares current cumulative volume to the cumulative volume for a given time of day, measured as a ratio. Current volume from open to current time of day / Average over the past x days from open to current time of day

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RDV Indicator

To create an automatic indicator for `RelativeDailyVolume` , call the `rdv` helper method from the `QCAlgorithm` class. The `rdv` method creates a `RelativeDailyVolume` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RelativeDailyVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rdv = self.rdv(self._symbol, 2)

    def on_data(self, slice: Slice) -> None:

        if self._rdv.is_ready:
            # The current value of self._rdv is represented by self._rdv.current.value
            self.plot("RelativeDailyVolume", "rdv", self._rdv.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RelativeDailyVolume` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

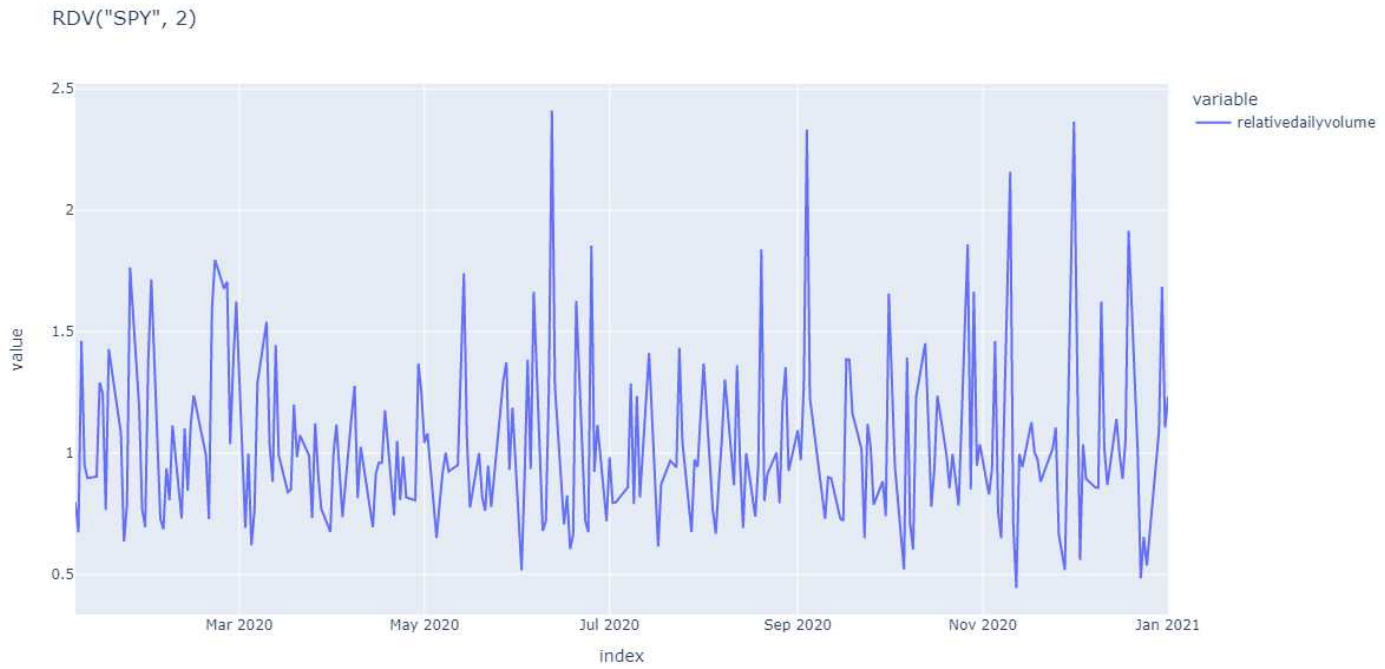
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RelativeDailyVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._relativedailyvolume = RelativeDailyVolume(2)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._relativedailyvolume.update(bar)

        if self._relativedailyvolume.is_ready:
            # The current value of self._relativedailyvolume is represented by self._relativedailyvolume.current.value
            self.plot("RelativeDailyVolume", "relativedailyvolume", self._relativedailyvolume.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RelativeDailyVolume` indicator properties:



Indicator History

To get the historical data of the `RelativeDailyVolume` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RelativeDailyVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rdv = self.rdv(self._symbol, 2)

        indicator_history = self.indicator_history(self._rdv, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rdv, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rdv, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Relative Moving Average

Introduction

This indicator represents the relative moving average indicator (RMA). RMA = SMA(3 x Period) - SMA(2 x Period) + SMA(1 x Period) per formula: https://www.hybrid-solutions.com/plugins/client-vtl-plugins/free/rma.html

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RMA Indicator

To create an automatic indicator for `RelativeMovingAverage` , call the `rma` helper method from the `QCAlgorithm` class. The `rma` method creates a `RelativeMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RelativeMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rma = self.rma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._rma.is_ready:
            # The current value of self._rma is represented by self._rma.current.value
            self.plot("RelativeMovingAverage", "rma", self._rma.current.value)
            # Plot all attributes of self._rma
            self.plot("RelativeMovingAverage", "short_average", self._rma.short_average.current.value)
            self.plot("RelativeMovingAverage", "medium_average", self._rma.medium_average.current.value)
            self.plot("RelativeMovingAverage", "long_average", self._rma.long_average.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RelativeMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RelativeMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._relativemovingaverage = RelativeMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._relativemovingaverage.update(bar.end_time, bar.close)

        if self._relativemovingaverage.is_ready:
            # The current value of self._relativemovingaverage is represented by self._relativemovingaverage.current.value
            self.plot("RelativeMovingAverage", "relativemovingaverage", self._relativemovingaverage.current.value)
            # Plot all attributes of self._relativemovingaverage
            self.plot("RelativeMovingAverage", "short_average", self._relativemovingaverage.short_average.current.value)
            self.plot("RelativeMovingAverage", "medium_average", self._relativemovingaverage.medium_average.current.value)
            self.plot("RelativeMovingAverage", "long_average", self._relativemovingaverage.long_average.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RelativeMovingAverage` indicator properties:

Indicator History

To get the historical data of the `RelativeMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RelativeMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rma = self.rma(self._symbol, 20)

        indicator_history = self.indicator_history(self._rma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        short_average = indicator_history_df["shortaverage"]
        medium_average = indicator_history_df["mediumaverage"]
        long_average = indicator_history_df["longaverage"]
```

Supported Indicators

Relative Strength Index

Introduction

This indicator represents the Relative Strength Index (RSI) developed by K. Welles Wilder. You can optionally specified a different moving average type to be used in the computation

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RSI Indicator

To create an automatic indicator for `RelativeStrengthIndex` , call the `rsi` helper method from the `QCAlgorithm` class. The `rsi` method creates a `RelativeStrengthIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rsi = self.rsi(self._symbol, 14)

    def on_data(self, slice: Slice) -> None:

        if self._rsi.is_ready:
            # The current value of self._rsi is represented by self._rsi.current.value
            self.plot("RelativeStrengthIndex", "rsi", self._rsi.current.value)
            # Plot all attributes of self._rsi
            self.plot("RelativeStrengthIndex", "average_loss", self._rsi.average_loss.current.value)
            self.plot("RelativeStrengthIndex", "average_gain", self._rsi.average_gain.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RelativeStrengthIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._relativestrengthindex = RelativeStrengthIndex(14)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._relativestrengthindex.update(bar.end_time, bar.close)

        if self._relativestrengthindex.is_ready:
            # The current value of self._relativestrengthindex is represented by self._relativestrengthindex.current.value
            self.plot("RelativeStrengthIndex", "relativestrengthindex", self._relativestrengthindex.current.value)
            # Plot all attributes of self._relativestrengthindex
            self.plot("RelativeStrengthIndex", "average_loss", self._relativestrengthindex.average_loss.current.value)
            self.plot("RelativeStrengthIndex", "average_gain", self._relativestrengthindex.average_gain.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RelativeStrengthIndex` indicator properties:

Indicator History

To get the historical data of the `RelativeStrengthIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rsi = self.rsi(self._symbol, 14)

        indicator_history = self.indicator_history(self._rsi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rsi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rsi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        average_loss = indicator_history_df["averageloss"]
        average_gain = indicator_history_df["averagegain"]
```

Supported Indicators

Relative Vigor Index

Introduction

The Relative Vigor Index (RVI) compares the ratio of the closing price of a security to its trading range. For illustration, let: a = Close−Open b = Close−Open of One Bar Prior to a c = Close−Open of One Bar Prior to b d = Close−Open of One Bar Prior to c e = High−Low of Bar a f = High−Low of Bar b g = High−Low of Bar c h = High−Low of Bar d Then let (a+2*(b+c)+d)/6 be NUM and (e+2*(f+g)+h)/6 be DENOM. RVI = SMA(NUM)/SMA(DENOM) for a specified period. https://www.investopedia.com/terms/r/relative_vigor_index.asp

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RVI Indicator

To create an automatic indicator for `RelativeVigorIndex` , call the `rvi` helper method from the `QCAlgorithm` class. The `rvi` method creates a `RelativeVigorIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class RelativeVigorIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rvi = self.rvi(self._symbol, 20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:

        if self._rvi.is_ready:
            # The current value of self._rvi is represented by self._rvi.current.value
            self.plot("RelativeVigorIndex", "rvi", self._rvi.current.value)
            # Plot all attributes of self._rvi
            self.plot("RelativeVigorIndex", "signal", self._rvi.signal.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `RelativeVigorIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class RelativeVigorIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._relativevigorindex = RelativeVigorIndex(20, MovingAverageType.SIMPLE)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._relativevigorindex.update(bar)

        if self._relativevigorindex.is_ready:
            # The current value of self._relativevigorindex is represented by self._relativevigorindex.current.value
            self.plot("RelativeVigorIndex", "relativevigorindex", self._relativevigorindex.current.value)
            # Plot all attributes of self._relativevigorindex
            self.plot("RelativeVigorIndex", "signal", self._relativevigorindex.signal.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RelativeVigorIndex` indicator properties:

Indicator History

To get the historical data of the `RelativeVigorIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RelativeVigorIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rvi = self.rvi(self._symbol, 20, MovingAverageType.SIMPLE)

        indicator_history = self.indicator_history(self._rvi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rvi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rvi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        signal = indicator_history_df["signal"]
```

Supported Indicators

Rho

Introduction

Option Rho indicator that calculate the rho of an option

To view the implementation of this indicator, see the LEAN GitHub repository .

Using R Indicator

To create an automatic indicator for `Rho` , call the `r` helper method from the `QCAlgorithm` class. The `r` method creates a `Rho` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RhoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._r = self.r(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._r.is_ready:
            # The current value of self._r is represented by self._r.current.value
            self.plot("Rho", "r", self._r.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Rho` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

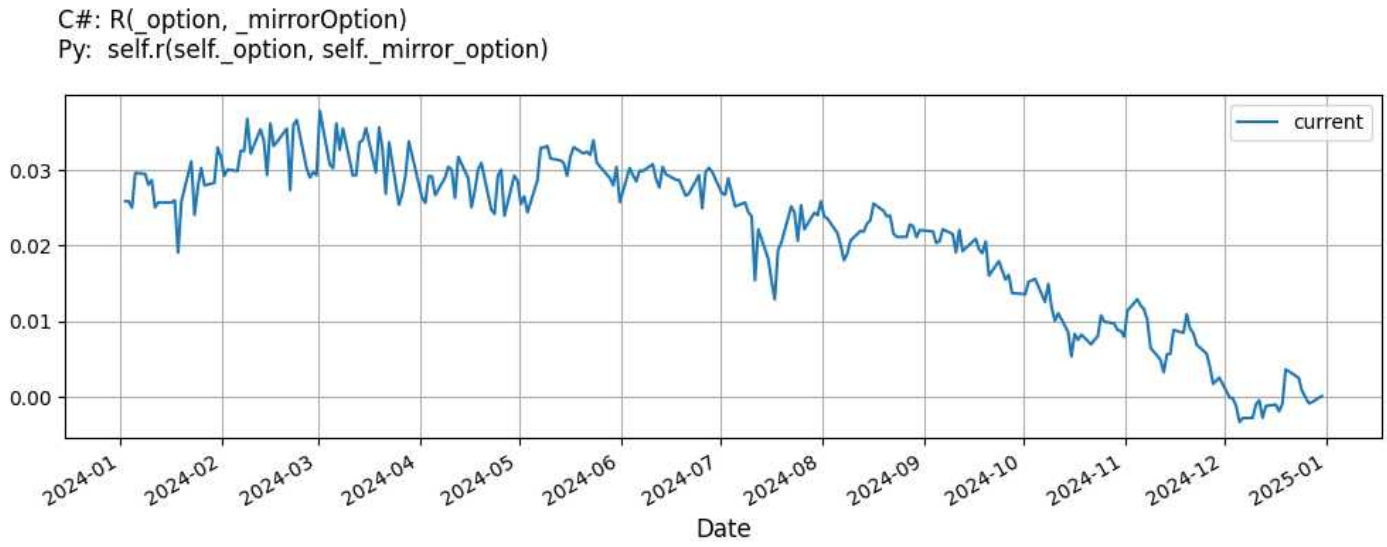
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RhoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._rho = Rho(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rho.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._rho.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._rho.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._rho.is_ready:
            # The current value of self._rho is represented by self._rho.current.value
            self.plot("Rho", "rho", self._rho.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Rho` indicator properties:



Indicator History

To get the historical data of the `Rho` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RhoAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._r = self.r(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._r, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._r, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._r, [self._symbol, self._option, self._mirror_option], datetime(2024,
7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Rogers Satchell Volatility

Introduction

This indicator computes the Rogers-Satchell Volatility It is an estimator for measuring the volatility of securities with an average return not equal to zero.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using RSV Indicator

To create an automatic indicator for `RogersSatchellVolatility` , call the `rsv` helper method from the `QCAlgorithm` class. The `rsv` method creates a `RogersSatchellVolatility` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class RogersSatchellVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rsv = self.rsv(self._symbol, 30)

    def on_data(self, slice: Slice) -> None:

        if self._rsv.is_ready:
            # The current value of self._rsv is represented by self._rsv.current.value
            self.plot("RogersSatchellVolatility", "rsv", self._rsv.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `RogersSatchellVolatility` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

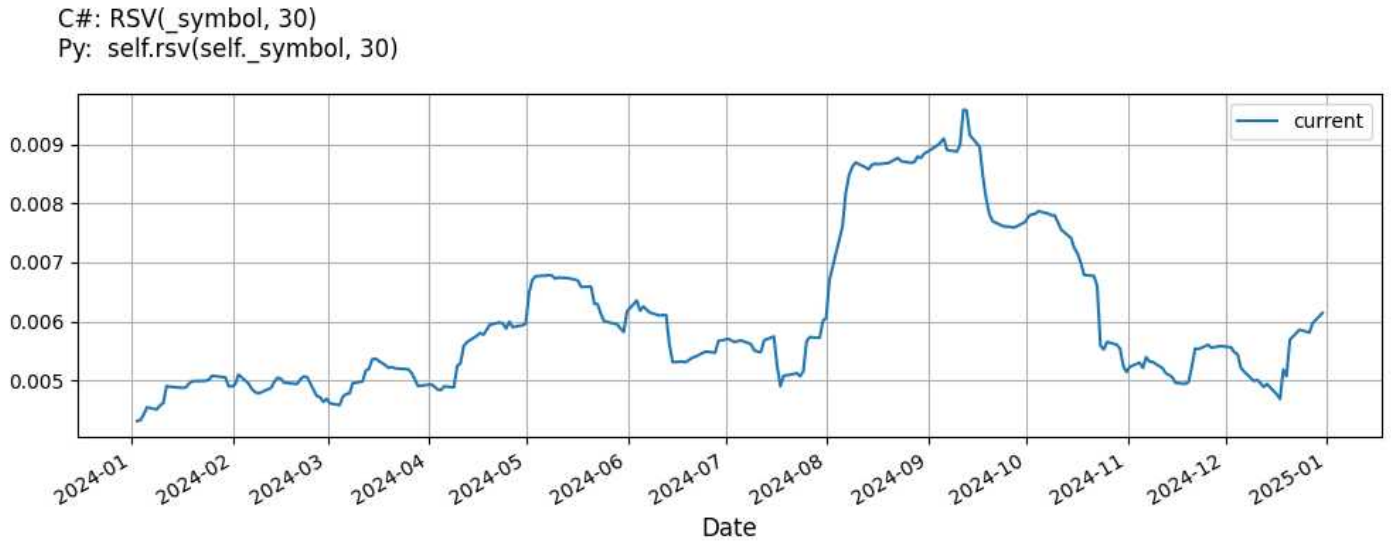
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class RogersSatchellVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rogerssatchellvolatility = RogersSatchellVolatility(30)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._rogerssatchellvolatility.update(bar)

        if self._rogerssatchellvolatility.is_ready:
            # The current value of self._rogerssatchellvolatility is represented by self._rogerssatchellvolatility.current.value
            self.plot("RogersSatchellVolatility", "rogerssatchellvolatility", self._rogerssatchellvolatility.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `RogersSatchellVolatility` indicator properties:



Indicator History

To get the historical data of the `RogersSatchellVolatility` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class RogersSatchellVolatilityAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._rsv = self.rsv(self._symbol, 30)

        indicator_history = self.indicator_history(self._rsv, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._rsv, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._rsv, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Schaff Trend Cycle

Introduction

This indicator creates the Schaff Trend Cycle

To view the implementation of this indicator, see the LEAN GitHub repository .

Using STC Indicator

To create an automatic indicator for `SchaffTrendCycle` , call the `stc` helper method from the `QCAlgorithm` class. The `stc` method creates a `SchaffTrendCycle` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SchaffTrendCycleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stc = self.stc(self._symbol, 5, 10, 20, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:

        if self._stc.is_ready:
            # The current value of self._stc is represented by self._stc.current.value
            self.plot("SchaffTrendCycle", "stc", self._stc.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SchaffTrendCycle` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

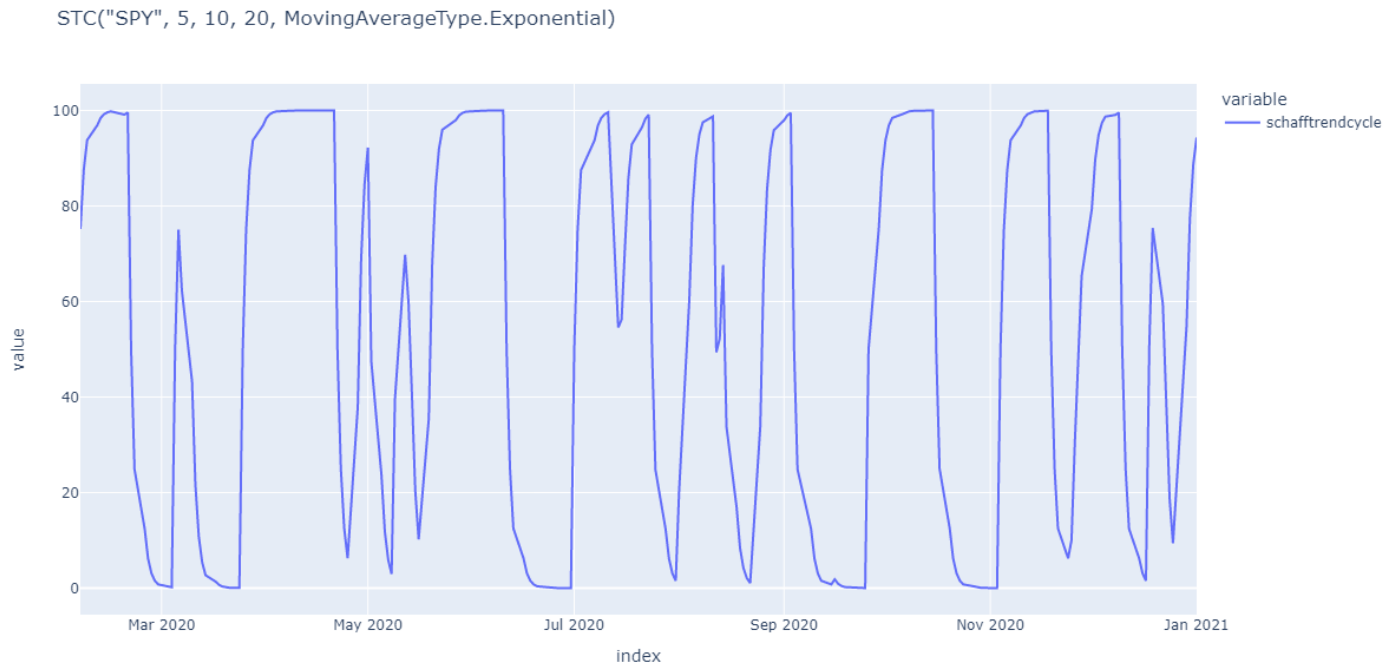
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SchaffTrendCycleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._schafftrendcycle = SchaffTrendCycle(5, 10, 20, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._schafftrendcycle.update(bar.end_time, bar.close)

        if self._schafftrendcycle.is_ready:
            # The current value of self._schafftrendcycle is represented by self._schafftrendcycle.current.value
            self.plot("SchaffTrendCycle", "schafftrendcycle", self._schafftrendcycle.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SchaffTrendCycle` indicator properties:



Indicator History

To get the historical data of the `SchaffTrendCycle` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class SchaffTrendCycleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stc = self.stc(self._symbol, 5, 10, 20, MovingAverageType.EXPONENTIAL)

        indicator_history = self.indicator_history(self._stc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._stc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._stc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Sharpe Ratio

Introduction

Calculation of the Sharpe Ratio (SR) developed by William F. Sharpe. Reference: https://www.investopedia.com/articles/07/sharpe_ratio.asp Formula: S(x) = (Rx - Rf) / stdDev(Rx) Where: S(x) - sharpe ratio of x Rx - average rate of return for x Rf - risk-free rate

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SR Indicator

To create an automatic indicator for `SharpeRatio` , call the `sr` helper method from the `QCAlgorithm` class. The `sr` method creates a `SharpeRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SharpeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sr = self.sr(self._symbol, 22, 0.03)

    def on_data(self, slice: Slice) -> None:

        if self._sr.is_ready:
            # The current value of self._sr is represented by self._sr.current.value
            self.plot("SharpeRatio", "sr", self._sr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SharpeRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

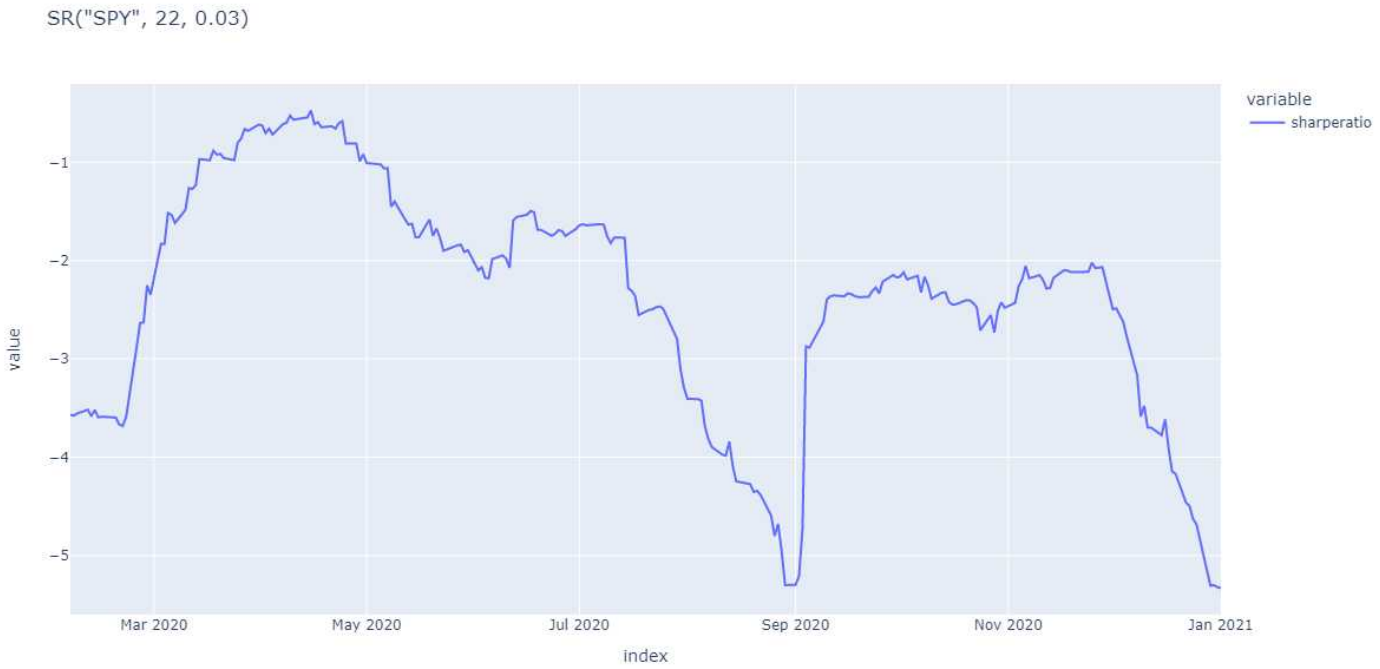
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SharpeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sharperatio = SharpeRatio(22, 0.03)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._sharperatio.update(bar.end_time, bar.close)

        if self._sharperatio.is_ready:
            # The current value of self._sharperatio is represented by self._sharperatio.current.value
            self.plot("SharpeRatio", "sharperatio", self._sharperatio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SharpeRatio` indicator properties:



Indicator History

To get the historical data of the `SharpeRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class SharpeRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sr = self.sr(self._symbol, 22, 0.03)

        indicator_history = self.indicator_history(self._sr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Simple Moving Average

Introduction

This indicator represents the traditional simple moving average indicator (SMA)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SMA Indicator

To create an automatic indicator for `SimpleMovingAverage` , call the `sma` helper method from the `QCAlgorithm` class. The `sma` method creates a `SimpleMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SimpleMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sma = self.sma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._sma.is_ready:
            # The current value of self._sma is represented by self._sma.current.value
            self.plot("SimpleMovingAverage", "sma", self._sma.current.value)
            # Plot all attributes of self._sma
            self.plot("SimpleMovingAverage", "rolling_sum", self._sma.rolling_sum.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SimpleMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

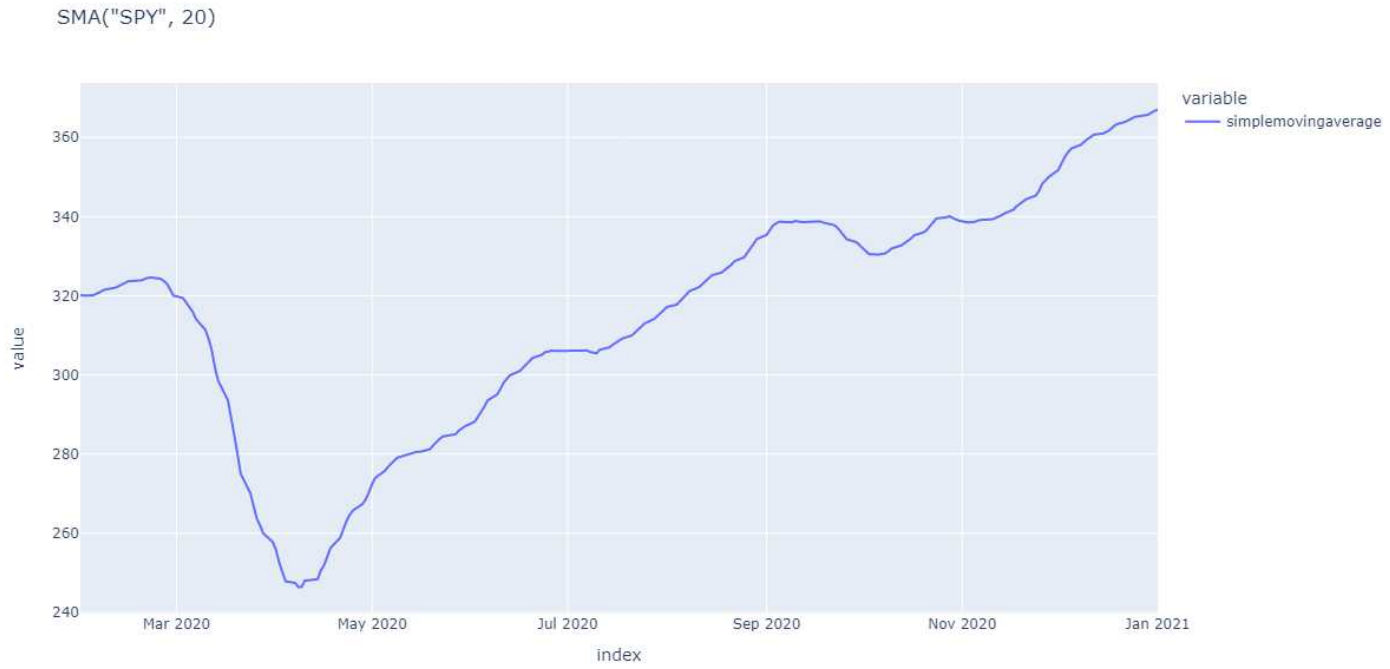
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SimpleMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._simplemovingaverage = SimpleMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._simplemovingaverage.update(bar.end_time, bar.close)

        if self._simplemovingaverage.is_ready:
            # The current value of self._simplemovingaverage is represented by self._simplemovingaverage.current.value
            self.plot("SimpleMovingAverage", "simplemovingaverage", self._simplemovingaverage.current.value)
            # Plot all attributes of self._simplemovingaverage
            self.plot("SimpleMovingAverage", "rolling_sum", self._simplemovingaverage.rolling_sum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SimpleMovingAverage` indicator properties:



Indicator History

To get the historical data of the `SimpleMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class SimpleMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sma = self.sma(self._symbol, 20)

        indicator_history = self.indicator_history(self._sma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        rolling_sum = indicator_history_df["rollingsum"]
```

Supported Indicators

Smoothed On Balance Volume

Introduction

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SOBV Indicator

To create an automatic indicator for `SmoothedOnBalanceVolume` , call the `sobv` helper method from the `QCAlgorithm` class. The `sobv` method creates a `SmoothedOnBalanceVolume` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SmoothedOnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sobv = self.sobv(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._sobv.is_ready:
            # The current value of self._sobv is represented by self._sobv.current.value
            self.plot("SmoothedOnBalanceVolume", "sobv", self._sobv.current.value)
            # Plot all attributes of self._sobv
            self.plot("SmoothedOnBalanceVolume", "on_balance_volume", self._sobv.on_balance_volume.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SmoothedOnBalanceVolume` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

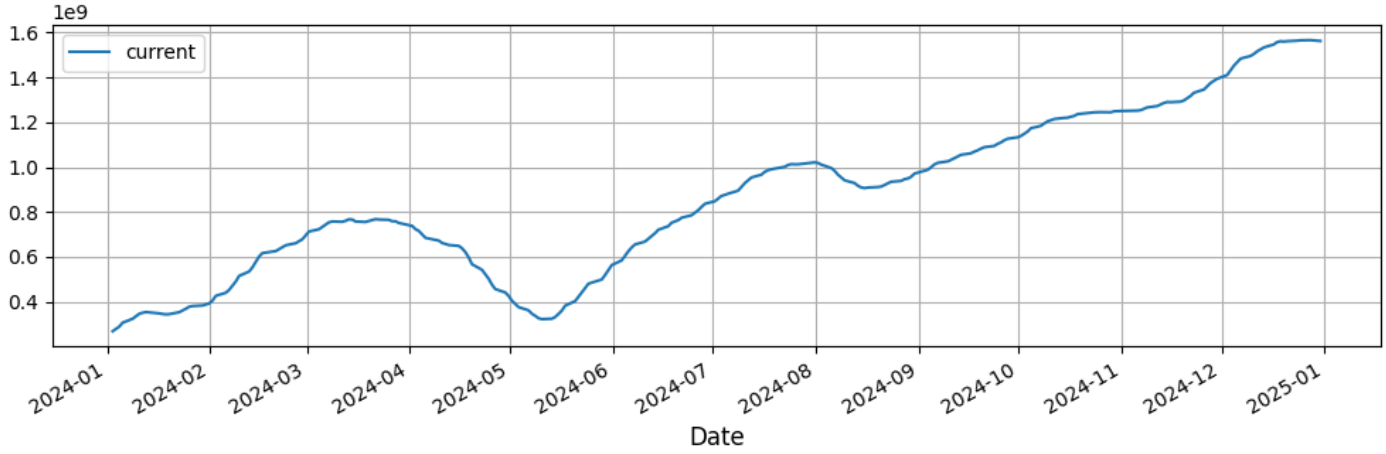
```
PY
class SmoothedOnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._smoothedonbalancevolume = SmoothedOnBalanceVolume(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._smoothedonbalancevolume.update(bar)

        if self._smoothedonbalancevolume.is_ready:
            # The current value of self._smoothedonbalancevolume is represented by self._smoothedonbalancevolume.current.value
            self.plot("SmoothedOnBalanceVolume", "smoothedonbalancevolume", self._smoothedonbalancevolume.current.value)
            # Plot all attributes of self._smoothedonbalancevolume
            self.plot("SmoothedOnBalanceVolume", "on_balance_volume", self._smoothedonbalancevolume.on_balance_volume.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SmoothedOnBalanceVolume` indicator properties:

C#: SOBV(_symbol, 20)
Py: self.sobv(self._symbol, 20)



Indicator History

To get the historical data of the `SmoothedOnBalanceVolume` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class SmoothedOnBalanceVolumeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sobv = self.sobv(self._symbol, 20)

        indicator_history = self.indicator_history(self._sobv, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sobv, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sobv, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        on_balance_volume = indicator_history_df["onbalancevolume"]
```

Supported Indicators

Sortino Ratio

Introduction

Calculation of the Sortino Ratio, a modification of the . Reference: https://www.cmegroup.com/education/files/rr-sortino-a-sharper-ratio.pdf Formula: S(x) = (R - T) / TDD Where: S(x) - Sortino ratio of x R - the average period return T - the target or required rate of return for the investment strategy under consideration. In Sortino's early work, T was originally known as the minimum acceptable return, or MAR. In his more recent work, MAR is now referred to as the Desired Target Return. TDD - the target downside deviation.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SORTINO Indicator

To create an automatic indicator for `SortinoRatio` , call the `sortino` helper method from the `QCAlgorithm` class. The `sortino` method creates a `SortinoRatio` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SortinoRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sortino = self.sortino(self._symbol, 22)

    def on_data(self, slice: Slice) -> None:

        if self._sortino.is_ready:
            # The current value of self._sortino is represented by self._sortino.current.value
            self.plot("SortinoRatio", "sortino", self._sortino.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SortinoRatio` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

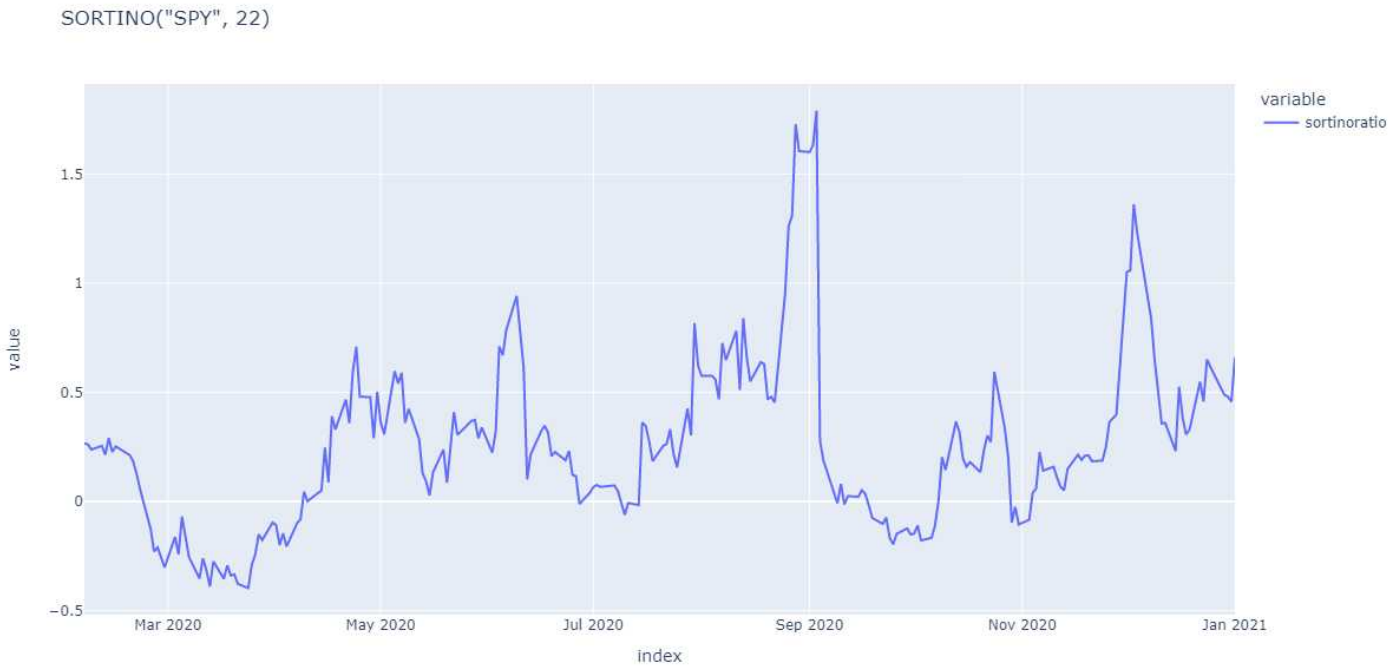
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SortinoRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sortinoratio = SortinoRatio(22)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._sortinoratio.update(bar.end_time, bar.close)

        if self._sortinoratio.is_ready:
            # The current value of self._sortinoratio is represented by self._sortinoratio.current.value
            self.plot("SortinoRatio", "sortinoratio", self._sortinoratio.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SortinoRatio` indicator properties:



Indicator History

To get the historical data of the `SortinoRatio` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class SortinoRatioAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sortino = self.sortino(self._symbol, 22)

        indicator_history = self.indicator_history(self._sortino, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sortino, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sortino, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Squeeze Momentum

Introduction

The SqueezeMomentum indicator calculates whether the market is in a "squeeze" condition, determined by comparing Bollinger Bands to Keltner Channels. When the Bollinger Bands are inside the Keltner Channels, the indicator returns 1 (squeeze on). Otherwise, it returns -1 (squeeze off).

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SM Indicator

To create an automatic indicator for `SqueezeMomentum` , call the `sm` helper method from the `QCAlgorithm` class. The `sm` method creates a `SqueezeMomentum` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SqueezeMomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sm = self.sm(self._symbol, 20, 2, 20, 1.5)

    def on_data(self, slice: Slice) -> None:

        if self._sm.is_ready:
            # The current value of self._sm is represented by self._sm.current.value
            self.plot("SqueezeMomentum", "sm", self._sm.current.value)
            # Plot all attributes of self._sm
            self.plot("SqueezeMomentum", "bollinger_bands", self._sm.bollinger_bands.current.value)
            self.plot("SqueezeMomentum", "keltner_channels", self._sm.keltner_channels.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SqueezeMomentum` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

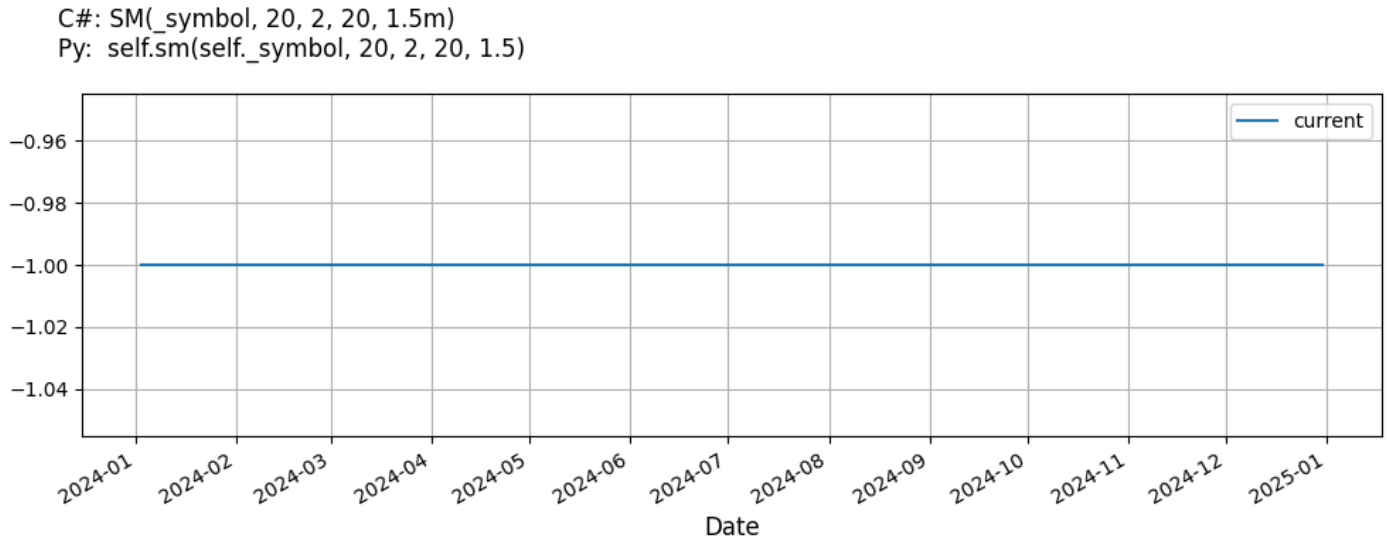
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SqueezeMomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._squeezemomentum = SqueezeMomentum(20, 2, 20, 1.5)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._squeezemomentum.update(bar)

        if self._squeezemomentum.is_ready:
            # The current value of self._squeezemomentum is represented by self._squeezemomentum.current.value
            self.plot("SqueezeMomentum", "squeezemomentum", self._squeezemomentum.current.value)
            # Plot all attributes of self._squeezemomentum
            self.plot("SqueezeMomentum", "bollinger_bands", self._squeezemomentum.bollinger_bands.current.value)
            self.plot("SqueezeMomentum", "keltner_channels", self._squeezemomentum.keltner_channels.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SqueezeMomentum` indicator properties:



Indicator History

To get the historical data of the `SqueezeMomentum` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class SqueezeMomentumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sm = self.sm(self._symbol, 20, 2, 20, 1.5)

        indicator_history = self.indicator_history(self._sm, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sm, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sm, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        bollinger_bands = indicator_history_df["bollingerbands"]
        keltner_channels = indicator_history_df["keltnerchannels"]
```

Supported Indicators

Standard Deviation

Introduction

This indicator computes the n-period population standard deviation.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using STD Indicator

To create an automatic indicator for `StandardDeviation` , call the `std` helper method from the `QCAlgorithm` class. The `std` method creates a `StandardDeviation` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class StandardDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._std = self.std(self._symbol, 22)

    def on_data(self, slice: Slice) -> None:

        if self._std.is_ready:
            # The current value of self._std is represented by self._std.current.value
            self.plot("StandardDeviation", "std", self._std.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `StandardDeviation` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

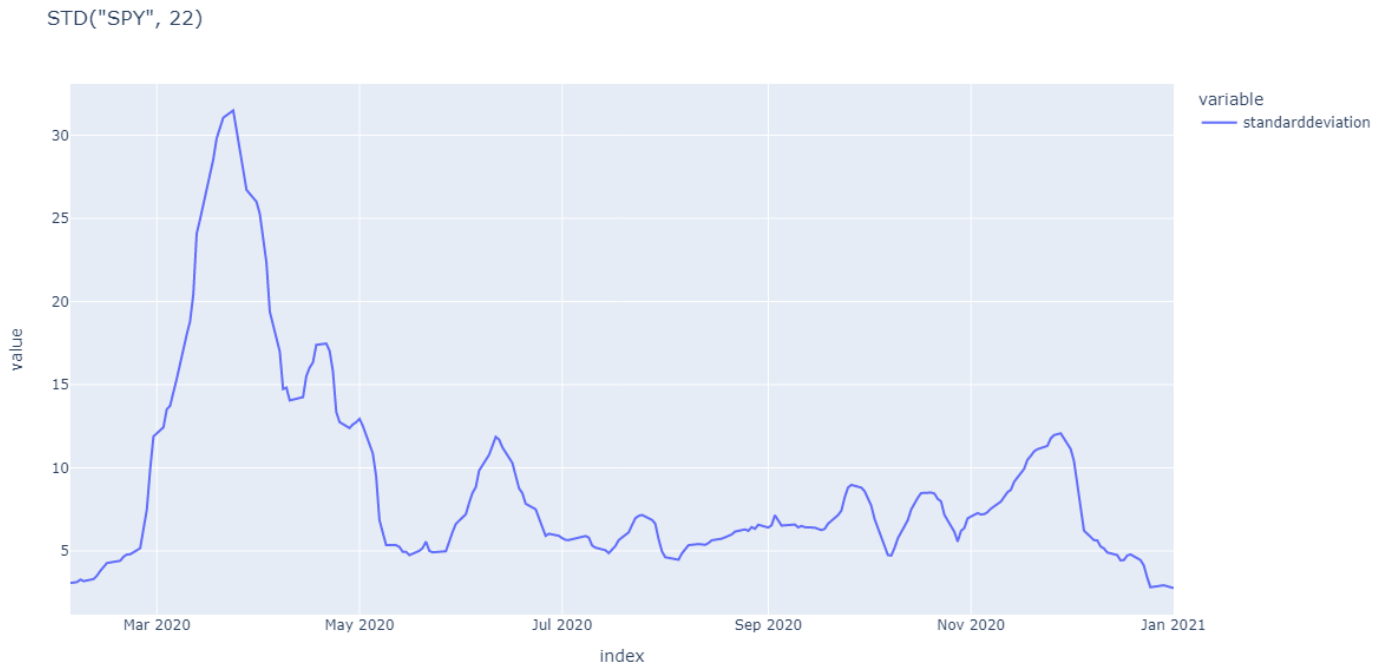
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class StandardDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._standarddeviation = StandardDeviation(22)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._standarddeviation.update(bar.end_time, bar.close)

        if self._standarddeviation.is_ready:
            # The current value of self._standarddeviation is represented by self._standarddeviation.current.value
            self.plot("StandardDeviation", "standarddeviation", self._standarddeviation.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `StandardDeviation` indicator properties:



Indicator History

To get the historical data of the `StandardDeviation` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class StandardDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._std = self.std(self._symbol, 22)

        indicator_history = self.indicator_history(self._std, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._std, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._std, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Stochastic

Introduction

This indicator computes the Slow Stochastics %K and %D. The Fast Stochastic %K is is computed by (Current Close Price - Lowest Price of given Period) / (Highest Price of given Period - Lowest Price of given Period) multiplied by 100. Once the Fast Stochastic %K is calculated the Slow Stochastic %K is calculated by the average/smoothed price of of the Fast %K with the given period. The Slow Stochastic %D is then derived from the Slow Stochastic %K with the given period.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using STO Indicator

To create an automatic indicator for `Stochastic` , call the `sto` helper method from the `QCAlgorithm` class. The `sto` method creates a `Stochastic` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class StochasticAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sto = self.sto(self._symbol, 20, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._sto.is_ready:
            # The current value of self._sto is represented by self._sto.current.value
            self.plot("Stochastic", "sto", self._sto.current.value)
            # Plot all attributes of self._sto
            self.plot("Stochastic", "fast_stoch", self._sto.fast_stoch.current.value)
            self.plot("Stochastic", "stoch_k", self._sto.stoch_k.current.value)
            self.plot("Stochastic", "stoch_d", self._sto.stoch_d.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Stochastic` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

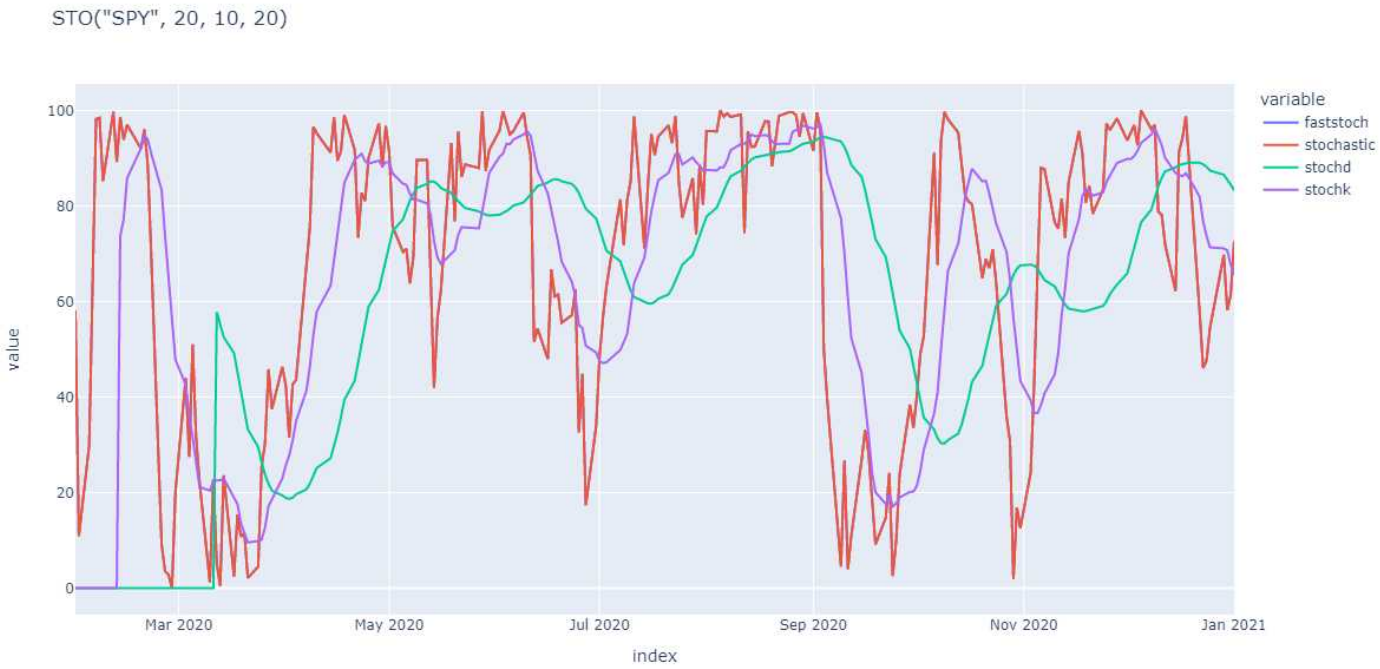
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class StochasticAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stochastic = Stochastic(20, 10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._stochastic.update(bar)

        if self._stochastic.is_ready:
            # The current value of self._stochastic is represented by self._stochastic.current.value
            self.plot("Stochastic", "stochastic", self._stochastic.current.value)
            # Plot all attributes of self._stochastic
            self.plot("Stochastic", "fast_stoch", self._stochastic.fast_stoch.current.value)
            self.plot("Stochastic", "stoch_k", self._stochastic.stoch_k.current.value)
            self.plot("Stochastic", "stoch_d", self._stochastic.stoch_d.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Stochastic` indicator properties:



Indicator History

To get the historical data of the `Stochastic` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class StochasticAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sto = self.sto(self._symbol, 20, 10, 20)

        indicator_history = self.indicator_history(self._sto, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sto, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sto, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        fast_stoch = indicator_history_df["faststoch"]
        stoch_k = indicator_history_df["stochk"]
        stoch_d = indicator_history_df["stochd"]
```

Supported Indicators

Stochastic Relative Strength Index

Introduction

Stochastic RSI, or simply StochRSI, is a technical analysis indicator used to determine whether an asset is overbought or oversold, as well as to identify current market trends. As the name suggests, the StochRSI is a derivative of the standard Relative Strength Index (RSI) and, as such, is considered an indicator of an indicator. It is a type of oscillator, meaning that it fluctuates above and below a center line.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SRSI Indicator

To create an automatic indicator for `StochasticRelativeStrengthIndex` , call the `srsi` helper method from the `QCAlgorithm` class. The `srsi` method creates a `StochasticRelativeStrengthIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class StochasticRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._srsi = self.srsi(self._symbol, 14, 14, 3, 3)

    def on_data(self, slice: Slice) -> None:

        if self._srsi.is_ready:
            # The current value of self._srsi is represented by self._srsi.current.value
            self.plot("StochasticRelativeStrengthIndex", "srsi", self._srsi.current.value)
            # Plot all attributes of self._srsi
            self.plot("StochasticRelativeStrengthIndex", "k", self._srsi.k.current.value)
            self.plot("StochasticRelativeStrengthIndex", "d", self._srsi.d.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `StochasticRelativeStrengthIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

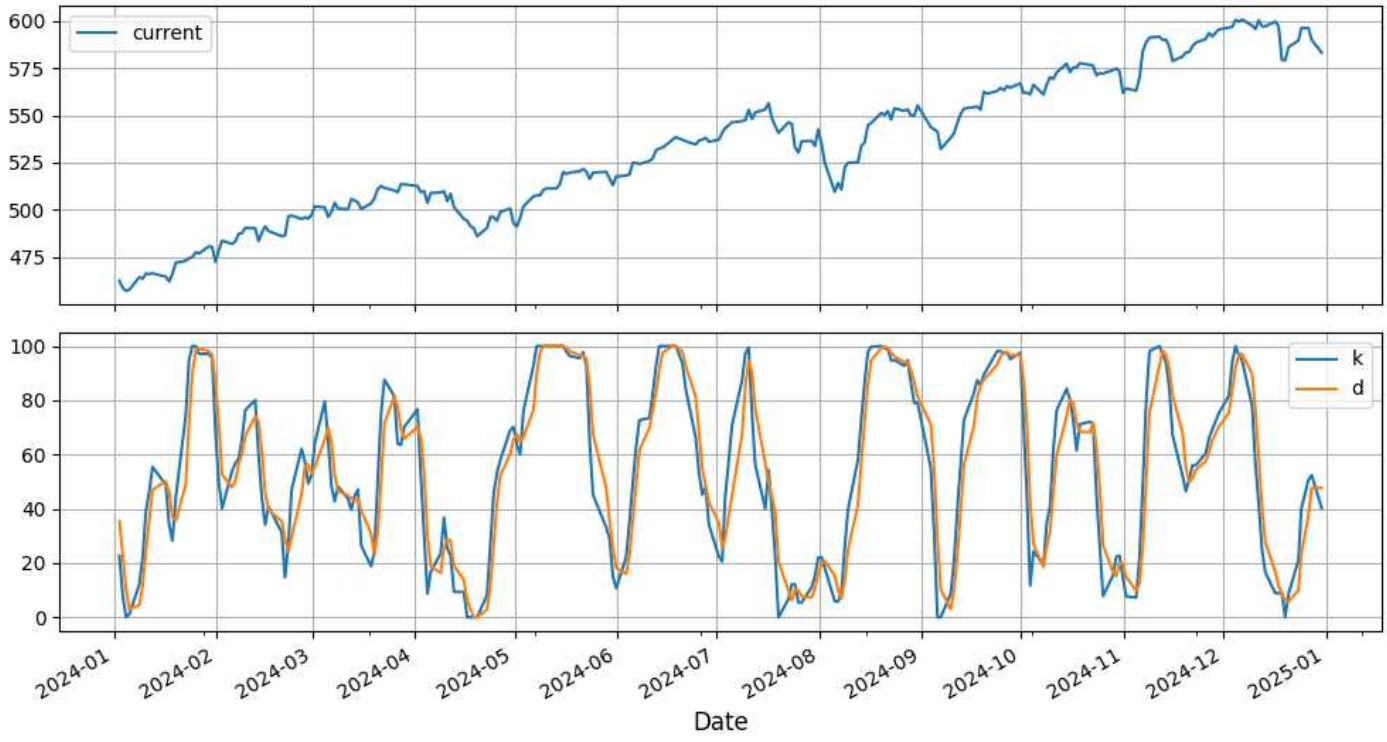
```
class StochasticRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._stochasticrelativestrengthindex = StochasticRelativeStrengthIndex(14, 14, 3, 3)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._stochasticrelativestrengthindex.update(bar.end_time, bar.close)

        if self._stochasticrelativestrengthindex.is_ready:
            # The current value of self._stochasticrelativestrengthindex is represented by
self._stochasticrelativestrengthindex.current.value
            self.plot("StochasticRelativeStrengthIndex", "stochasticrelativestrengthindex",
self._stochasticrelativestrengthindex.current.value)
            # Plot all attributes of self._stochasticrelativestrengthindex
            self.plot("StochasticRelativeStrengthIndex", "k", self._stochasticrelativestrengthindex.k.current.value)
            self.plot("StochasticRelativeStrengthIndex", "d", self._stochasticrelativestrengthindex.d.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `StochasticRelativeStrengthIndex` indicator properties:

C#: SRSI(_symbol, 14, 14, 3, 3)
Py: self.srsi(self._symbol, 14, 14, 3, 3)



Indicator History

To get the historical data of the `StochasticRelativeStrengthIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class StochasticRelativeStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._srsi = self.srsi(self._symbol, 14, 14, 3, 3)

        indicator_history = self.indicator_history(self._srsi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._srsi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._srsi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        k = indicator_history_df["k"]
        d = indicator_history_df["d"]
```

Supported Indicators

Sum

Introduction

This indicator represents an indicator capable of tracking the sum for the given period

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SUM Indicator

To create an automatic indicator for `Sum` , call the `sum` helper method from the `QCAlgorithm` class. The `sum` method creates a `Sum` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sum = self.sum(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._sum.is_ready:
            # The current value of self._sum is represented by self._sum.current.value
            self.plot("Sum", "sum", self._sum.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Sum` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

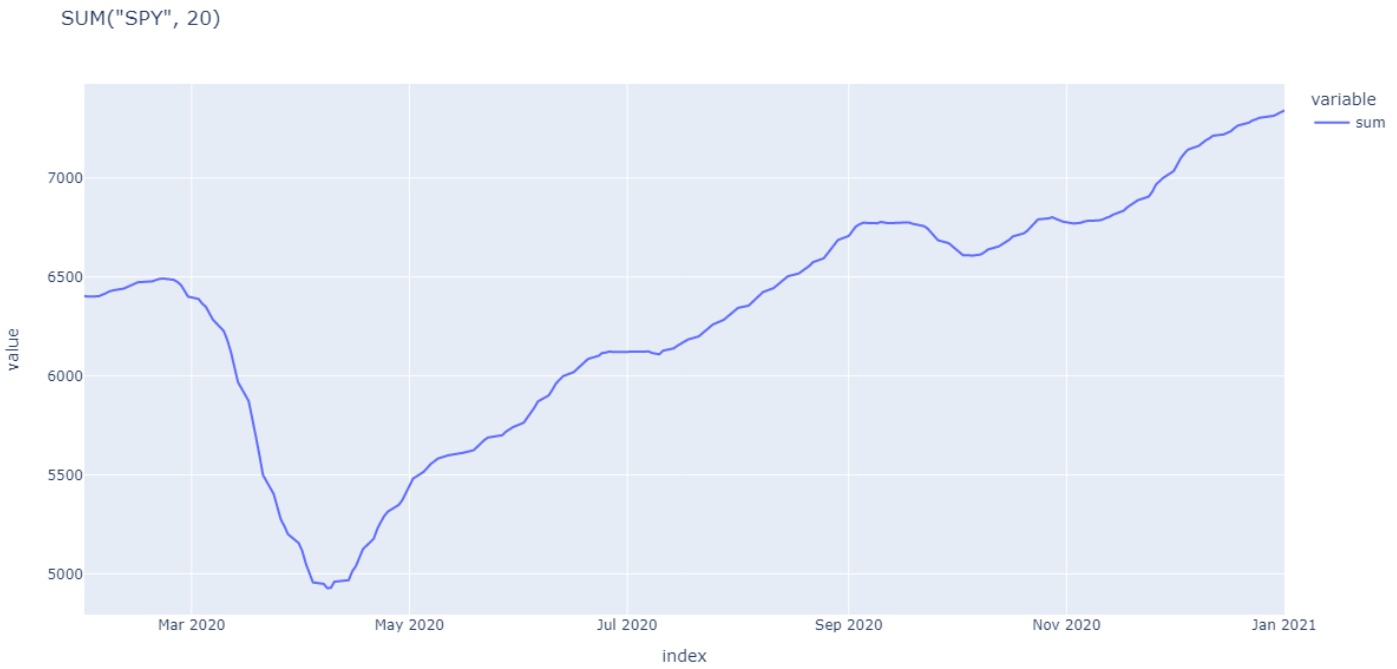
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sum = Sum(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._sum.update(bar.end_time, bar.close)

        if self._sum.is_ready:
            # The current value of self._sum is represented by self._sum.current.value
            self.plot("Sum", "sum", self._sum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Sum` indicator properties:



Indicator History

To get the historical data of the `Sum` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class SumAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._sum = self.sum(self._symbol, 20)

        indicator_history = self.indicator_history(self._sum, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._sum, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._sum, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Super Trend

Introduction

Super trend indicator. Formula can be found here via the excel file: https://tradingtuitions.com/supertrend-indicator-excel-sheet-with-realtime-buy-sell-signals/

To view the implementation of this indicator, see the LEAN GitHub repository .

Using STR Indicator

To create an automatic indicator for `SuperTrend` , call the `str` helper method from the `QCAlgorithm` class. The `str` method creates a `SuperTrend` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SuperTrendAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._str = self.str(self._symbol, 20, 2, MovingAverageType.WILDERS)

    def on_data(self, slice: Slice) -> None:

        if self._str.is_ready:
            # The current value of self._str is represented by self._str.current.value
            self.plot("SuperTrend", "str", self._str.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SuperTrend` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

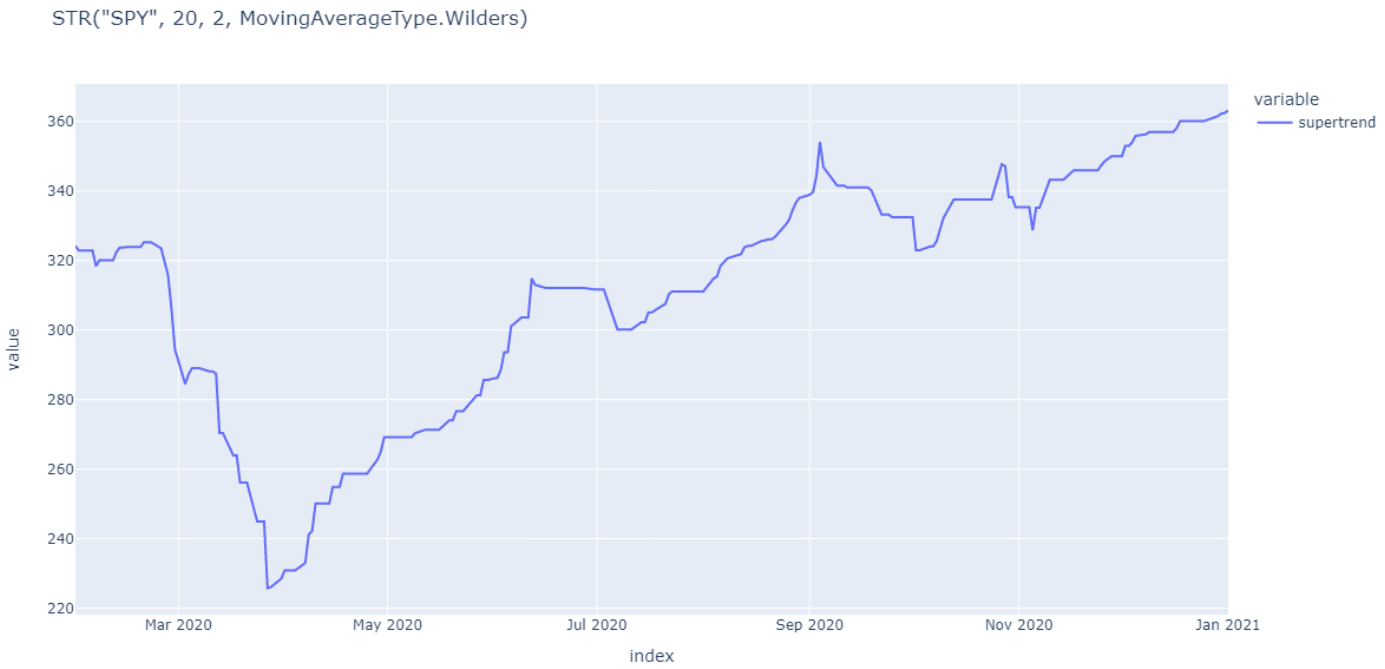
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SuperTrendAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._supertrend = SuperTrend(20, 2, MovingAverageType.WILDERS)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._supertrend.update(bar)

        if self._supertrend.is_ready:
            # The current value of self._supertrend is represented by self._supertrend.current.value
            self.plot("SuperTrend", "supertrend", self._supertrend.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SuperTrend` indicator properties:



Indicator History

To get the historical data of the `SuperTrend` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class SuperTrendAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._str = self.str(self._symbol, 20, 2, MovingAverageType.WILDERS)

        indicator_history = self.indicator_history(self._str, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._str, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._str, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Swiss Army Knife

Introduction

Swiss Army Knife indicator by John Ehlers

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SWISS Indicator

To create an automatic indicator for `SwissArmyKnife` , call the `swiss` helper method from the `QCAlgorithm` class. The `swiss` method creates a `SwissArmyKnife` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class SwissArmyKnifeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._swiss = self.swiss(self._symbol, 20, 0.2, SwissArmyKnifeTool.GAUSS)

    def on_data(self, slice: Slice) -> None:

        if self._swiss.is_ready:
            # The current value of self._swiss is represented by self._swiss.current.value
            self.plot("SwissArmyKnife", "swiss", self._swiss.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `SwissArmyKnife` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

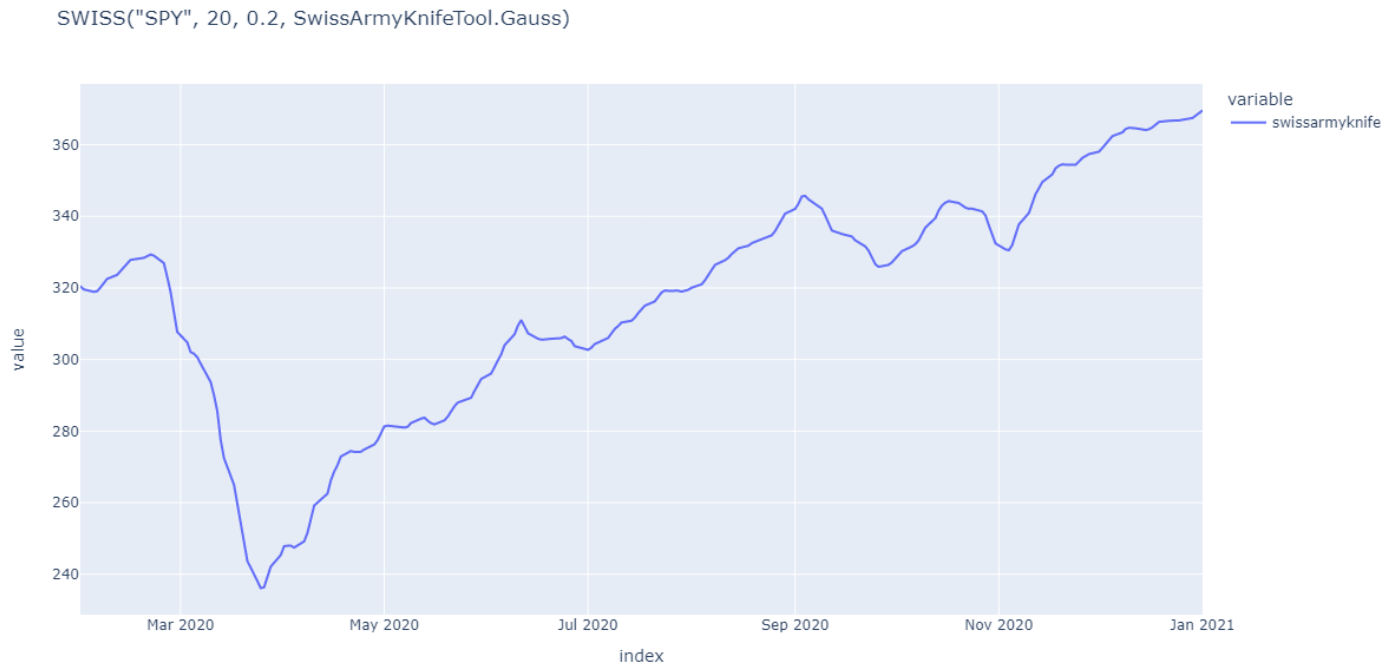
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class SwissArmyKnifeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._swissarmyknife = SwissArmyKnife(20, 0.2, SwissArmyKnifeTool.GAUSS)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._swissarmyknife.update(bar.end_time, bar.close)

        if self._swissarmyknife.is_ready:
            # The current value of self._swissarmyknife is represented by self._swissarmyknife.current.value
            self.plot("SwissArmyKnife", "swissarmyknife", self._swissarmyknife.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `SwissArmyKnife` indicator properties:



Indicator History

To get the historical data of the `SwissArmyKnife` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class SwissArmyKnifeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._swiss = self.swiss(self._symbol, 20, 0.2, SwissArmyKnifeTool.GAUSS)

        indicator_history = self.indicator_history(self._swiss, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._swiss, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._swiss, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

T3 Moving Average

Introduction

This indicator computes the T3 Moving Average (T3). The T3 Moving Average is calculated with the following formula: EMA1(x, Period) = EMA(x, Period) EMA2(x, Period) = EMA(EMA1(x, Period),Period) GD(x, Period, volumeFactor) = (EMA1(x, Period)*(1+volumeFactor)) - (EMA2(x, Period)* volumeFactor) T3 = GD(GD(GD(t, Period, volumeFactor), Period, volumeFactor), Period, volumeFactor);

To view the implementation of this indicator, see the LEAN GitHub repository .

Using T3 Indicator

To create an automatic indicator for `T3MovingAverage` , call the `t_3` helper method from the `QCAlgorithm` class. The `t_3` method creates a `T3MovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class T3MovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._t_3 = self.t_3(self._symbol, 30, 0.7)

    def on_data(self, slice: Slice) -> None:

        if self._t_3.is_ready:
            # The current value of self._t_3 is represented by self._t_3.current.value
            self.plot("T3MovingAverage", "t_3", self._t_3.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `T3MovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

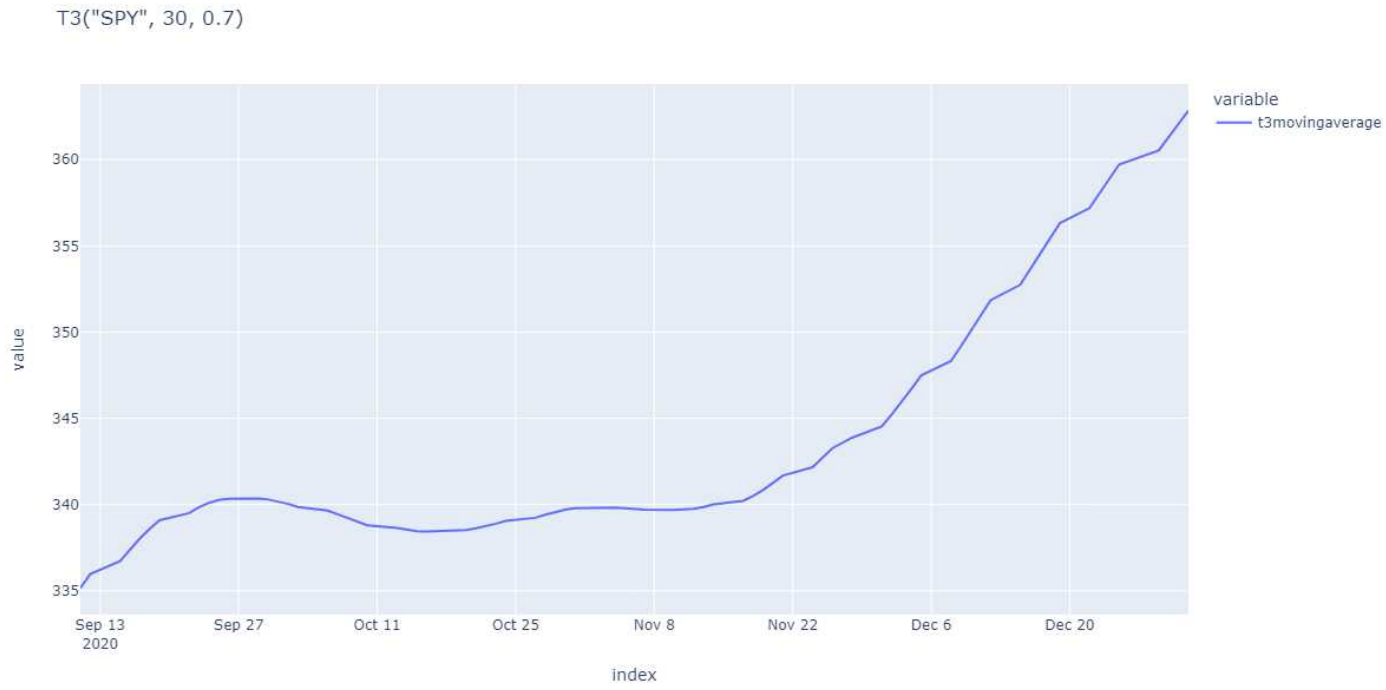
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class T3MovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._t3movingaverage = T3MovingAverage(30, 0.7)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._t3movingaverage.update(bar.end_time, bar.close)

        if self._t3movingaverage.is_ready:
            # The current value of self._t3movingaverage is represented by self._t3movingaverage.current.value
            self.plot("T3MovingAverage", "t3movingaverage", self._t3movingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `T3MovingAverage` indicator properties:



Indicator History

To get the historical data of the `T3MovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class T3MovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._t_3 = self.t_3(self._symbol, 30, 0.7)

        indicator_history = self.indicator_history(self._t_3, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._t_3, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._t_3, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Target Downside Deviation

Introduction

This indicator computes the n-period target downside deviation. The target downside deviation is defined as the root-mean-square, or RMS, of the deviations of the realized return's underperformance from the target return where all returns above the target return are treated as underperformance of 0. Reference: https://www.cmegroup.com/education/files/rr-sortino-a-sharper-ratio.pdf

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TDD Indicator

To create an automatic indicator for `TargetDownsideDeviation` , call the `tdd` helper method from the `QCAlgorithm` class. The `tdd` method creates a `TargetDownsideDeviation` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TargetDownsideDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tdd = self.tdd(self._symbol, 50)

    def on_data(self, slice: Slice) -> None:

        if self._tdd.is_ready:
            # The current value of self._tdd is represented by self._tdd.current.value
            self.plot("TargetDownsideDeviation", "tdd", self._tdd.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TargetDownsideDeviation` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

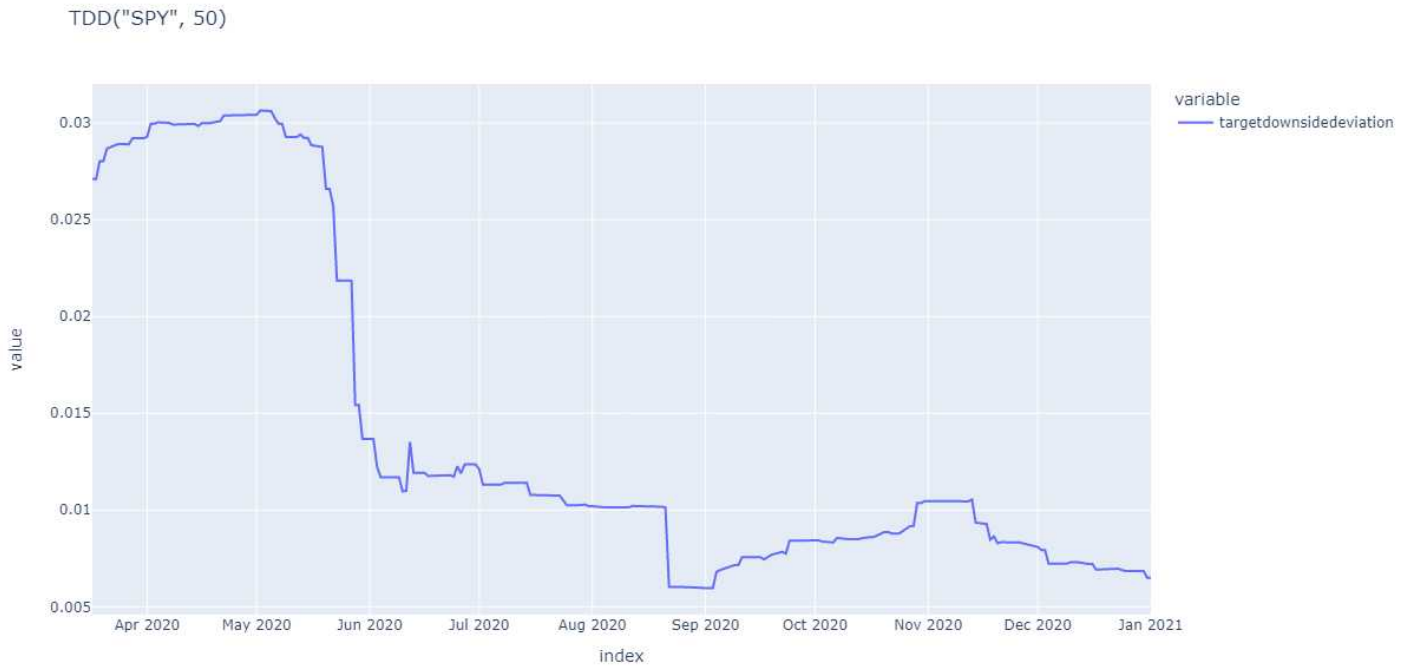
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TargetDownsideDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._targetdownsidedeviation = IndicatorExtensions.of(TargetDownsideDeviation(50), RateOfChange(1))

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._targetdownsidedeviation.update(bar.end_time, bar.close)

        if self._targetdownsidedeviation.is_ready:
            # The current value of self._targetdownsidedeviation is represented by self._targetdownsidedeviation.current.value
            self.plot("TargetDownsideDeviation", "targetdownsidedeviation", self._targetdownsidedeviation.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TargetDownsideDeviation` indicator properties:



Indicator History

To get the historical data of the `TargetDownsideDeviation` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TargetDownsideDeviationAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tdd = self.tdd(self._symbol, 50)

        indicator_history = self.indicator_history(self._tdd, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tdd, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tdd, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Theta

Introduction

Option Theta indicator that calculate the theta of an option

To view the implementation of this indicator, see the LEAN GitHub repository .

Using T Indicator

To create an automatic indicator for `Theta` , call the `t` helper method from the `QCAlgorithm` class. The `t` method creates a `Theta` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```py
class ThetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._t = self.t(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._t.is_ready:
            # The current value of self._t is represented by self._t.current.value
            self.plot("Theta", "t", self._t.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Theta` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

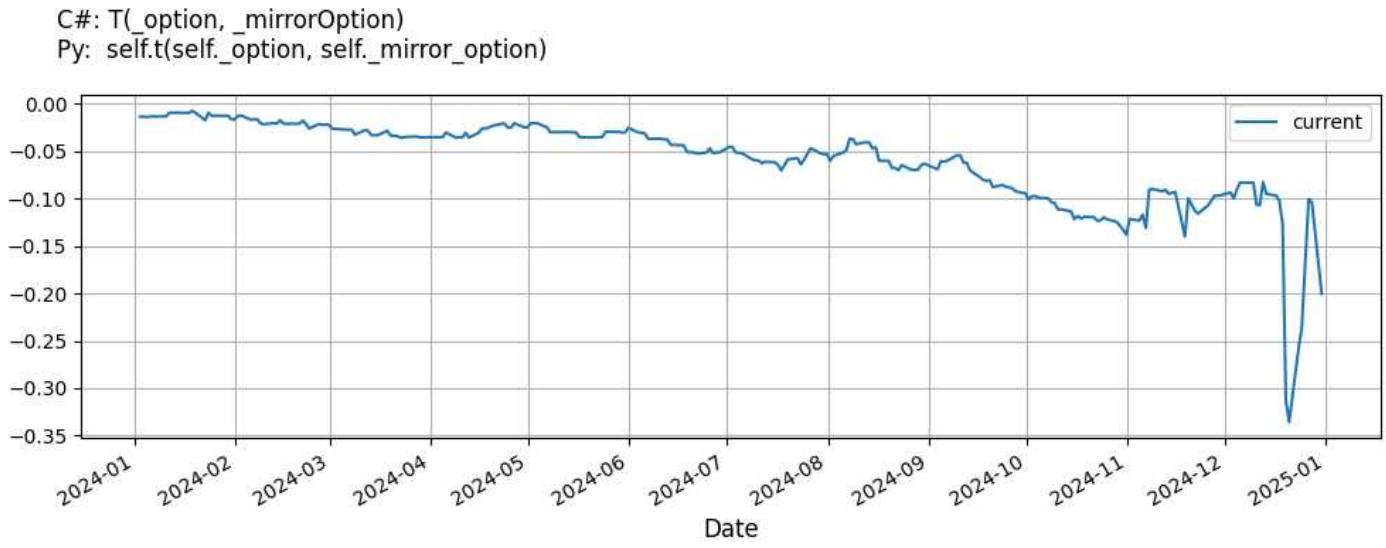
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ThetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._theta = Theta(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._theta.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._theta.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._theta.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._theta.is_ready:
            # The current value of self._theta is represented by self._theta.current.value
            self.plot("Theta", "theta", self._theta.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Theta` indicator properties:



Indicator History

To get the historical data of the `Theta` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ThetaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._t = self.t(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._t, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._t, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._t, [self._symbol, self._option, self._mirror_option], datetime(2024,
7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Time Profile

Introduction

This indicator represents an Indicator of the Market Profile with Time Price Opportunity (TPO) mode and its attributes

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TP Indicator

To create an automatic indicator for `TimeProfile` , call the `tp` helper method from the `QCAlgorithm` class. The `tp` method creates a `TimeProfile` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TimeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tp = self.tp(self._symbol, 3, 0.70, 0.05)

    def on_data(self, slice: Slice) -> None:

        if self._tp.is_ready:
            # The current value of self._tp is represented by self._tp.current.value
            self.plot("TimeProfile", "tp", self._tp.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `TimeProfile` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class TimeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._timeprofile = TimeProfile(3, 0.70, 0.05)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._timeprofile.update(bar)

        if self._timeprofile.is_ready:
            # The current value of self._timeprofile is represented by self._timeprofile.current.value
            self.plot("TimeProfile", "timeprofile", self._timeprofile.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TimeProfile` indicator properties:

Indicator History

To get the historical data of the `TimeProfile` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class TimeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tp = self.tp(self._symbol, 3, 0.70, 0.05)

        indicator_history = self.indicator_history(self._tp, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tp, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tp, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Time Series Forecast

Introduction

This indicator represents an indicator capable of predicting new values given previous data from a window. source

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TSF Indicator

To create an automatic indicator for `TimeSeriesForecast` , call the `tsf` helper method from the `QCAlgorithm` class. The `tsf` method creates a `TimeSeriesForecast` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TimeSeriesForecastAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tsf = self.tsf(self._symbol, 3)

    def on_data(self, slice: Slice) -> None:

        if self._tsf.is_ready:
            # The current value of self._tsf is represented by self._tsf.current.value
            self.plot("TimeSeriesForecast", "tsf", self._tsf.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TimeSeriesForecast` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

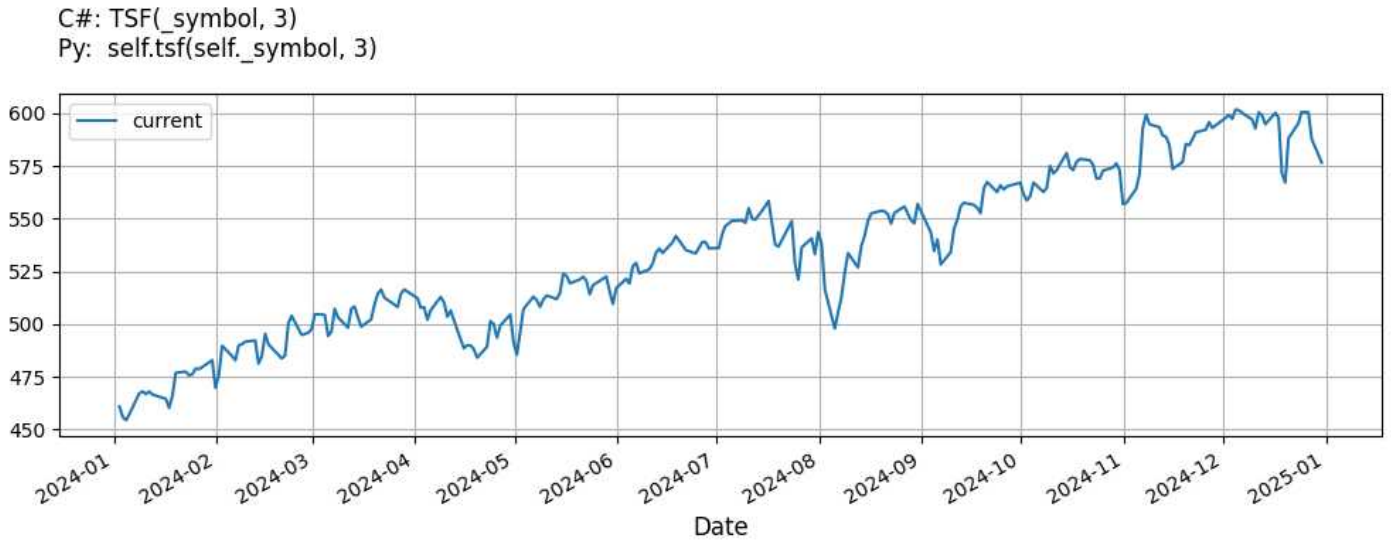
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TimeSeriesForecastAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._timeseriesforecast = TimeSeriesForecast(3)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._timeseriesforecast.update(bar.end_time, bar.close)

        if self._timeseriesforecast.is_ready:
            # The current value of self._timeseriesforecast is represented by self._timeseriesforecast.current.value
            self.plot("TimeSeriesForecast", "timeseriesforecast", self._timeseriesforecast.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TimeSeriesForecast` indicator properties:



Indicator History

To get the historical data of the `TimeSeriesForecast` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TimeSeriesForecastAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tsf = self.tsf(self._symbol, 3)

        indicator_history = self.indicator_history(self._tsf, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tsf, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tsf, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Tom Demark Sequential

Introduction

This indicator represents the Tom Demark Sequential indicator, which is used to identify potential trend exhaustion points. This implementation tracks both the setup and countdown phases, and can be extended to handle bullish and bearish setups, as well as qualifiers such as Perfect Setups and Countdown completions. - **Setup Phase**: Detects a trend by counting 9 consecutive bars where the close is less than (Buy Setup) or greater than (Sell Setup) the close 4 bars earlier. - **TDST Support/Resistance**: After a valid 9-bar setup completes, the indicator records the **lowest low** (for Sell Setup) or **highest high** (for Buy Setup) among the 9 bars. These are referred to as Support Price(for Buy Setup) and Resistance Price (for Sell Setup), and serve as critical thresholds that validate the continuation of the countdown phase. - **Countdown Phase**: Once a valid setup is completed, the indicator attempts to count 13 qualifying bars (not necessarily consecutive) where the close is less than (Buy Countdown) or greater than (Sell Countdown) the low/high 2 bars earlier. During this phase, the TDST Support/Resistance levels are checked — if the price breaks these levels, the countdown phase is reset, as the trend reversal condition is considered invalidated.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TDS Indicator

To create an automatic indicator for `TomDemarkSequential` , call the `tds` helper method from the `QCAlgorithm` class. The `tds` method creates a `TomDemarkSequential` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TomDemarkSequentialAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tds = self.tds(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._tds.is_ready:
            # The current value of self._tds is represented by self._tds.current.value
            self.plot("TomDemarkSequential", "tds", self._tds.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TomDemarkSequential` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

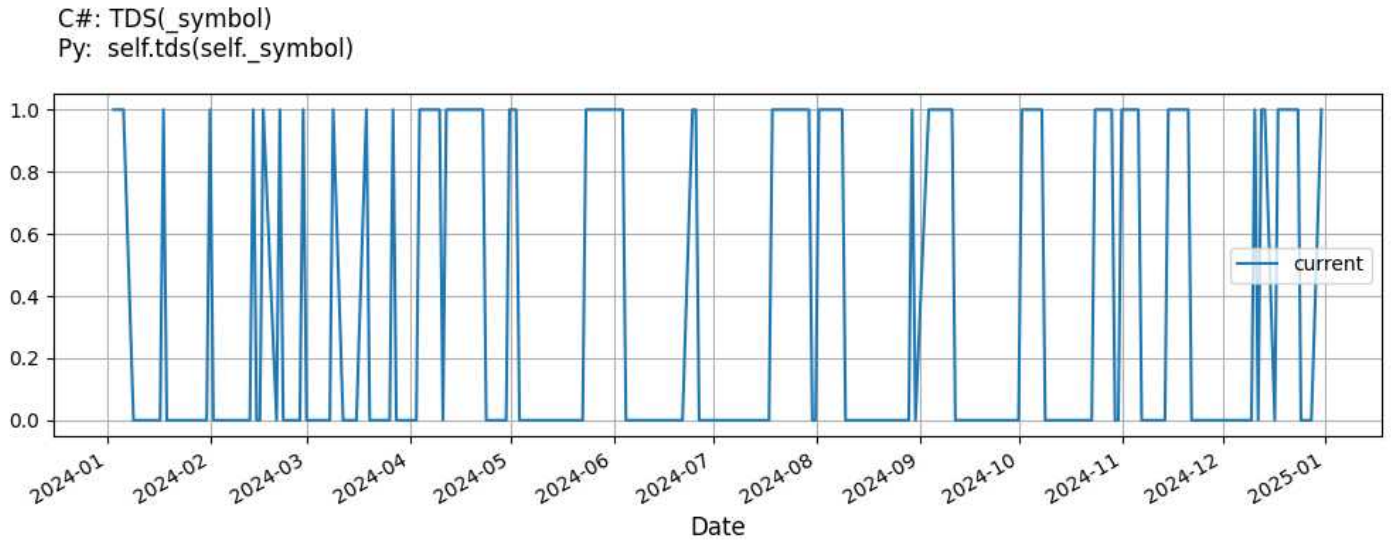
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TomDemarkSequentialAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tomdemarksequential = TomDemarkSequential()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._tomdemarksequential.update(bar)

        if self._tomdemarksequential.is_ready:
            # The current value of self._tomdemarksequential is represented by self._tomdemarksequential.current.value
            self.plot("TomDemarkSequential", "tomdemarksequential", self._tomdemarksequential.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TomDemarkSequential` indicator properties:



Indicator History

To get the historical data of the `TomDemarkSequential` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TomDemarkSequentialAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tds = self.tds(self._symbol)

        indicator_history = self.indicator_history(self._tds, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tds, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tds, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Triangular Moving Average

Introduction

This indicator computes the Triangular Moving Average (TRIMA). The Triangular Moving Average is calculated with the following formula: (1) When the period is even, TRIMA(x,period)=SMA(SMA(x,period/2),(period/2)+1) (2) When the period is odd, TRIMA(x,period)=SMA(SMA(x,(period+1)/2),(period+1)/2)

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TRIMA Indicator

To create an automatic indicator for `TriangularMovingAverage` , call the `trima` helper method from the `QCAlgorithm` class. The `trima` method creates a `TriangularMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TriangularMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trima = self.trima(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._trima.is_ready:
            # The current value of self._trima is represented by self._trima.current.value
            self.plot("TriangularMovingAverage", "trima", self._trima.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TriangularMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

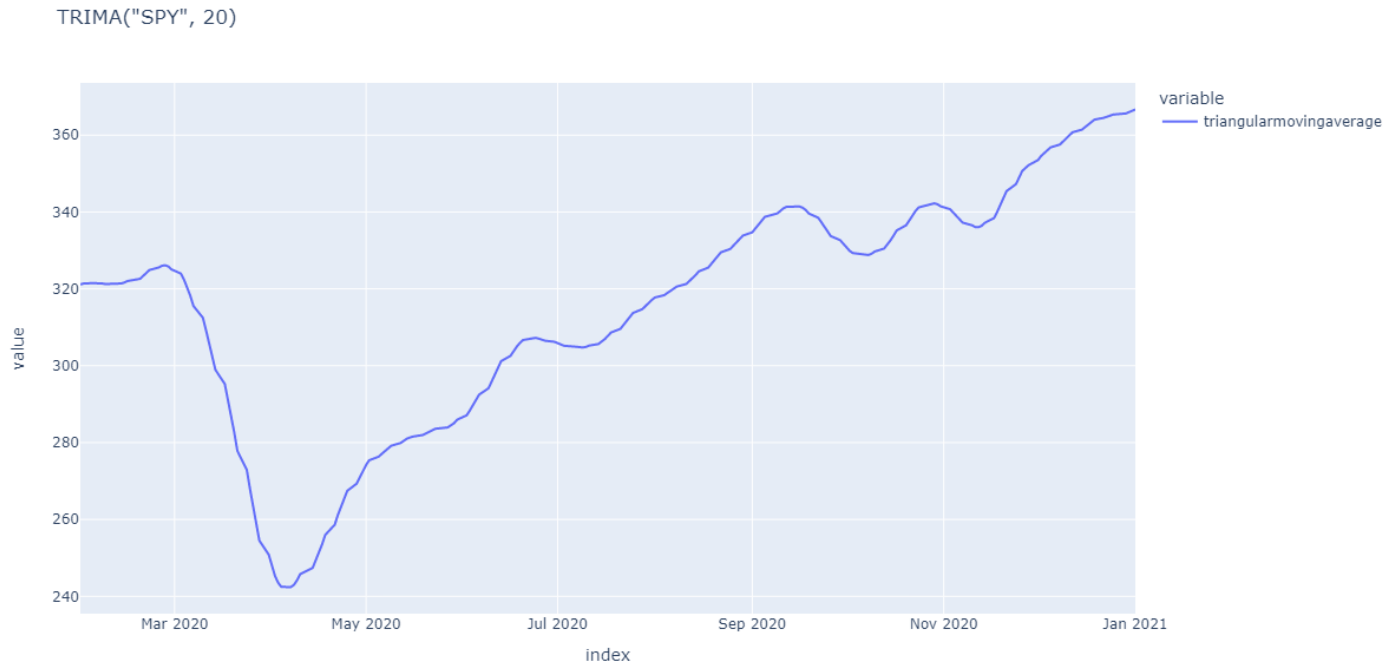
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TriangularMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._triangularmovingaverage = TriangularMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._triangularmovingaverage.update(bar.end_time, bar.close)

        if self._triangularmovingaverage.is_ready:
            # The current value of self._triangularmovingaverage is represented by self._triangularmovingaverage.current.value
            self.plot("TriangularMovingAverage", "triangularmovingaverage", self._triangularmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TriangularMovingAverage` indicator properties:



Indicator History

To get the historical data of the `TriangularMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class TriangularMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trima = self.trima(self._symbol, 20)

        indicator_history = self.indicator_history(self._trima, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._trima, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._trima, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Triple Exponential Moving Average

Introduction

This indicator computes the Triple Exponential Moving Average (TEMA). The Triple Exponential Moving Average is calculated with the following formula: EMA1 = EMA(t,period) EMA2 = EMA(EMA(t,period),period) EMA3 = EMA(EMA(EMA(t,period),period),period) TEMA = 3 * EMA1 - 3 * EMA2 + EMA3

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TEMA Indicator

To create an automatic indicator for `TripleExponentialMovingAverage` , call the `tema` helper method from the `QCAlgorithm` class. The `tema` method creates a `TripleExponentialMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TripleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tema = self.tema(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._tema.is_ready:
            # The current value of self._tema is represented by self._tema.current.value
            self.plot("TripleExponentialMovingAverage", "tema", self._tema.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TripleExponentialMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TripleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tripleexponentialmovingaverage = TripleExponentialMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._tripleexponentialmovingaverage.update(bar.end_time, bar.close)

        if self._tripleexponentialmovingaverage.is_ready:
            # The current value of self._tripleexponentialmovingaverage is represented by
self._tripleexponentialmovingaverage.current.value
            self.plot("TripleExponentialMovingAverage", "tripleexponentialmovingaverage",
self._tripleexponentialmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TripleExponentialMovingAverage` indicator properties:

Indicator History

To get the historical data of the `TripleExponentialMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TripleExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tema = self.tema(self._symbol, 20)

        indicator_history = self.indicator_history(self._tema, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tema, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tema, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Trix

Introduction

This indicator computes the TRIX (1-period ROC of a Triple EMA) The TRIX is calculated as explained here: http://stockcharts.com/school/doku.php?id=chart_school:technical_indicators:trix

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TRIX Indicator

To create an automatic indicator for `Trix` , call the `trix` helper method from the `QCAlgorithm` class. The `trix` method creates a `Trix` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TrixAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trix = self.trix(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._trix.is_ready:
            # The current value of self._trix is represented by self._trix.current.value
            self.plot("Trix", "trix", self._trix.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `Trix` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

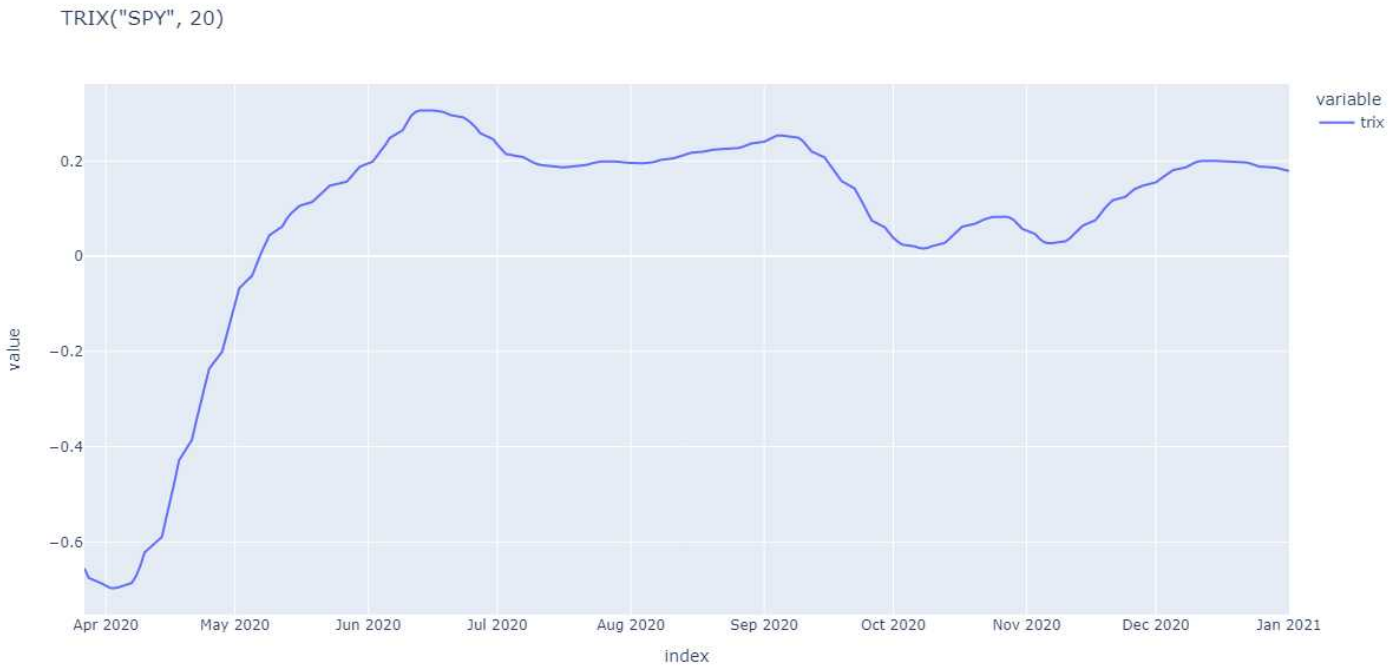
```
class TrixAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trix = Trix(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._trix.update(bar.end_time, bar.close)

        if self._trix.is_ready:
            # The current value of self._trix is represented by self._trix.current.value
            self.plot("Trix", "trix", self._trix.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Trix` indicator properties:



Indicator History

To get the historical data of the `Trix` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TrixAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._trix = self.trix(self._symbol, 20)

        indicator_history = self.indicator_history(self._trix, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._trix, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._trix, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

True Range

Introduction

This indicator computes the True Range (TR). The True Range is the greatest of the following values: value1 = distance from today's high to today's low. value2 = distance from yesterday's close to today's high. value3 = distance from yesterday's close to today's low.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TR Indicator

To create an automatic indicator for `TrueRange` , call the `tr` helper method from the `QCAlgorithm` class. The `tr` method creates a `TrueRange` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class TrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tr = self.tr(self._symbol)

    def on_data(self, slice: Slice) -> None:

        if self._tr.is_ready:
            # The current value of self._tr is represented by self._tr.current.value
            self.plot("TrueRange", "tr", self._tr.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TrueRange` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

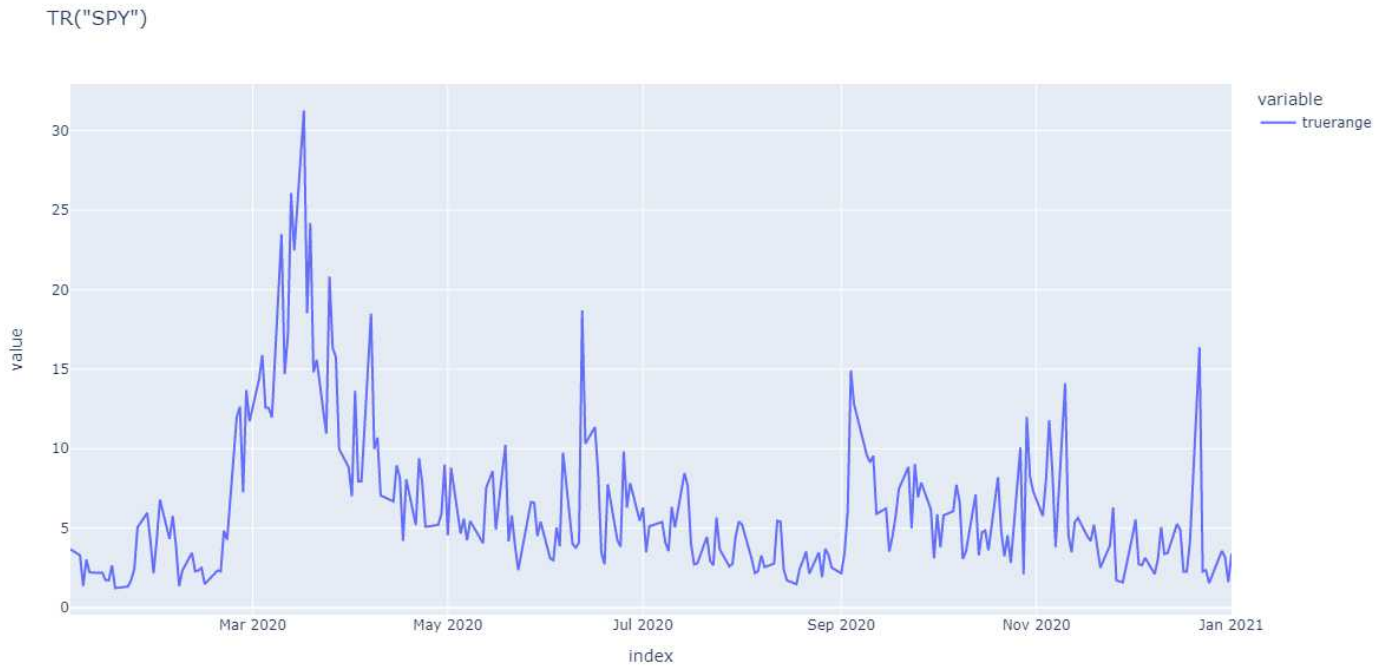
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class TrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._truerange = TrueRange()

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._truerange.update(bar)

        if self._truerange.is_ready:
            # The current value of self._truerange is represented by self._truerange.current.value
            self.plot("TrueRange", "truerange", self._truerange.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TrueRange` indicator properties:



Indicator History

To get the historical data of the `TrueRange` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TrueRangeAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tr = self.tr(self._symbol)

        indicator_history = self.indicator_history(self._tr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

True Strength Index

Introduction

This indicator computes the True Strength Index (TSI). The True Strength Index is calculated as explained here: https://school.stockcharts.com/doku.php?id=technical_indicators:True_strength_index Briefly, the calculation has three steps: 1. Smooth the momentum and the absolute momentum by getting an EMA of them (typically of period 25) 2. Double smooth the momentum and the absolute momentum by getting an EMA of their EMA (typically of period 13) 3. The TSI formula itself: divide the double-smoothed momentum over the double-smoothed absolute momentum and multiply by 100 The signal is typically a 7-to-12-EMA of the TSI.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using TSI Indicator

To create an automatic indicator for `TrueStrengthIndex` , call the `tsi` helper method from the `QCAlgorithm` class. The `tsi` method creates a `TrueStrengthIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class TrueStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tsi = self.tsi(self._symbol, 25, 13, 7, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:

        if self._tsi.is_ready:
            # The current value of self._tsi is represented by self._tsi.current.value
            self.plot("TrueStrengthIndex", "tsi", self._tsi.current.value)
            # Plot all attributes of self._tsi
            self.plot("TrueStrengthIndex", "signal", self._tsi.signal.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `TrueStrengthIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

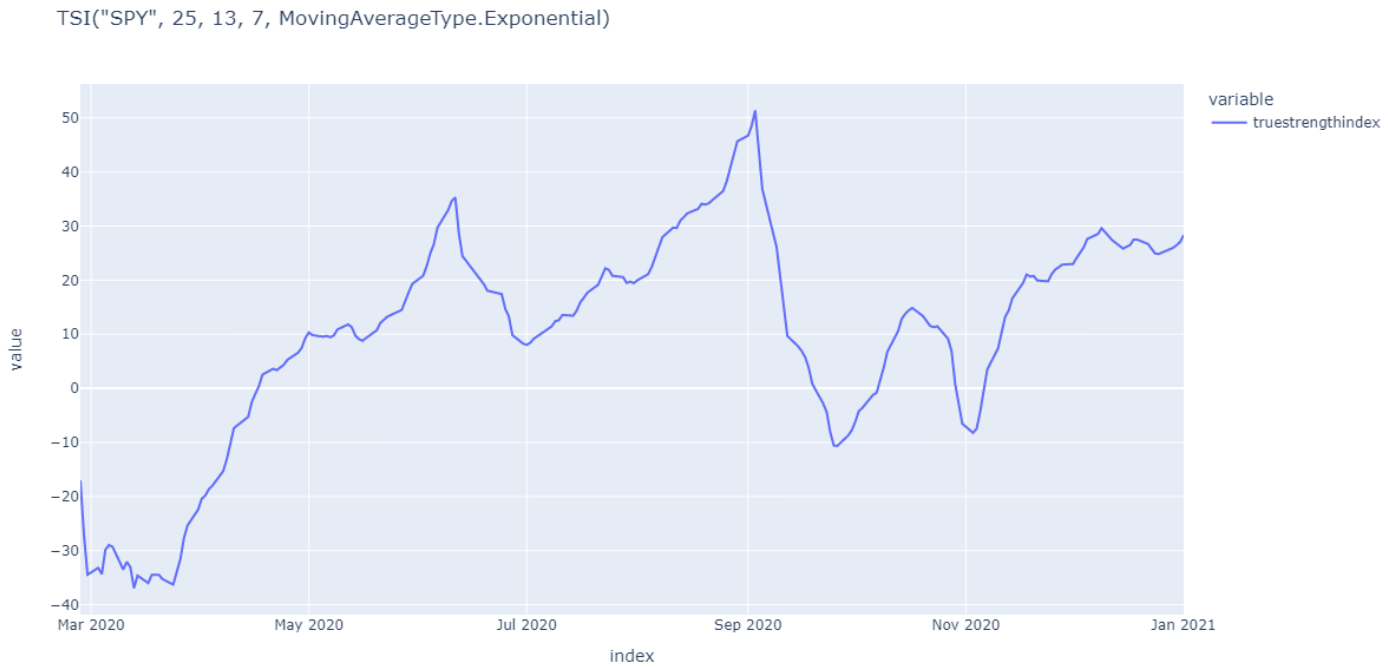
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class TrueStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._truestrengthindex = TrueStrengthIndex(25, 13, 7, MovingAverageType.EXPONENTIAL)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._truestrengthindex.update(bar.end_time, bar.close)

        if self._truestrengthindex.is_ready:
            # The current value of self._truestrengthindex is represented by self._truestrengthindex.current.value
            self.plot("TrueStrengthIndex", "truestrengthindex", self._truestrengthindex.current.value)
            # Plot all attributes of self._truestrengthindex
            self.plot("TrueStrengthIndex", "signal", self._truestrengthindex.signal.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `TrueStrengthIndex` indicator properties:



Indicator History

To get the historical data of the `TrueStrengthIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class TrueStrengthIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._tsi = self.tsi(self._symbol, 25, 13, 7, MovingAverageType.EXPONENTIAL)

        indicator_history = self.indicator_history(self._tsi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._tsi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._tsi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        signal = indicator_history_df["signal"]
```

Supported Indicators

Ultimate Oscillator

Introduction

This indicator computes the Ultimate Oscillator (ULTOSC) The Ultimate Oscillator is calculated as explained here: http://stockcharts.com/school/doku.php?id=chart_school:technical_indicators:ultimate_oscillator

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ULTOSC Indicator

To create an automatic indicator for `UltimateOscillator` , call the `ultosc` helper method from the `QCAlgorithm` class. The `ultosc` method creates a `UltimateOscillator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class UltimateOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ultosc = self.ultosc(self._symbol, 5, 10, 20)

    def on_data(self, slice: Slice) -> None:

        if self._ultosc.is_ready:
            # The current value of self._ultosc is represented by self._ultosc.current.value
            self.plot("UltimateOscillator", "ultosc", self._ultosc.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `UltimateOscillator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

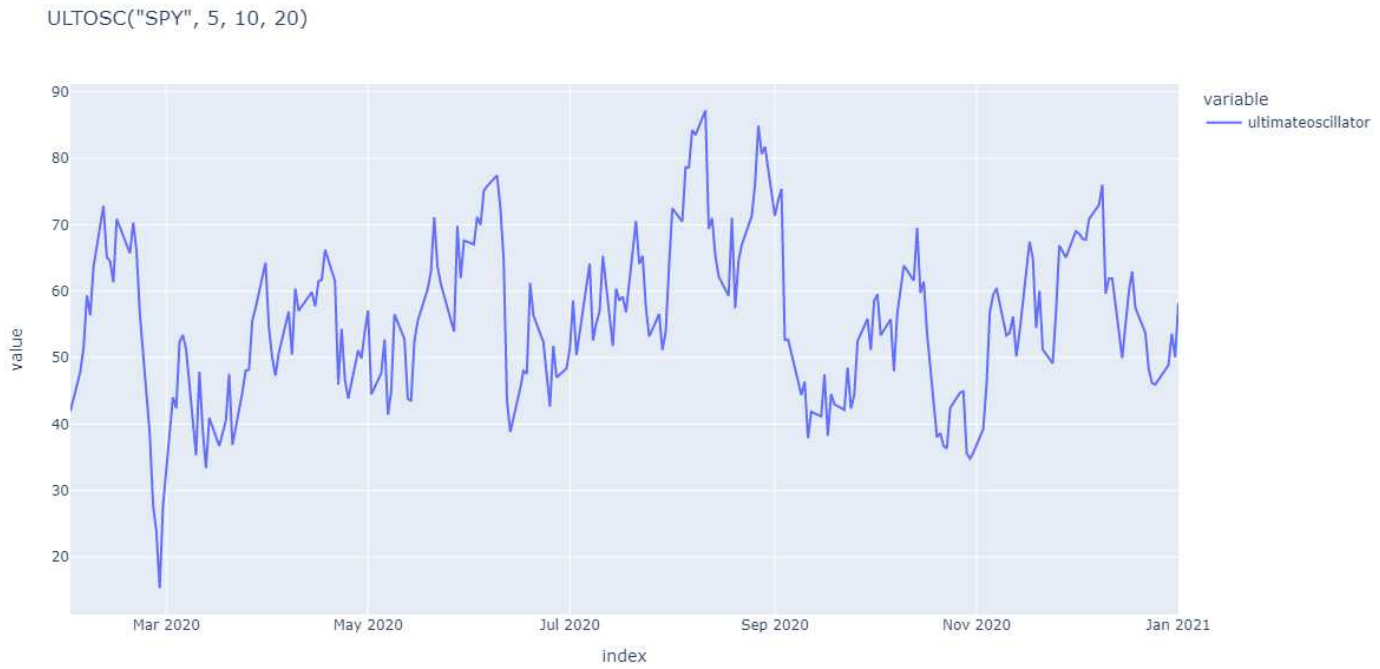
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class UltimateOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ultimateoscillator = UltimateOscillator(5, 10, 20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._ultimateoscillator.update(bar)

        if self._ultimateoscillator.is_ready:
            # The current value of self._ultimateoscillator is represented by self._ultimateoscillator.current.value
            self.plot("UltimateOscillator", "ultimateoscillator", self._ultimateoscillator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `UltimateOscillator` indicator properties:



Indicator History

To get the historical data of the `UltimateOscillator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class UltimateOscillatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ultosc = self.ultosc(self._symbol, 5, 10, 20)

        indicator_history = self.indicator_history(self._ultosc, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._ultosc, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._ultosc, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Value At Risk

Introduction

This indicator computes 1-day VaR for a specified confidence level and lookback period

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VAR Indicator

To create an automatic indicator for `ValueAtRisk` , call the `var` helper method from the `QCAlgorithm` class. The `var` method creates a `ValueAtRisk` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class ValueAtRiskAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._var = self.var(self._symbol, 252, 0.95)

    def on_data(self, slice: Slice) -> None:

        if self._var.is_ready:
            # The current value of self._var is represented by self._var.current.value
            self.plot("ValueAtRisk", "var", self._var.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ValueAtRisk` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

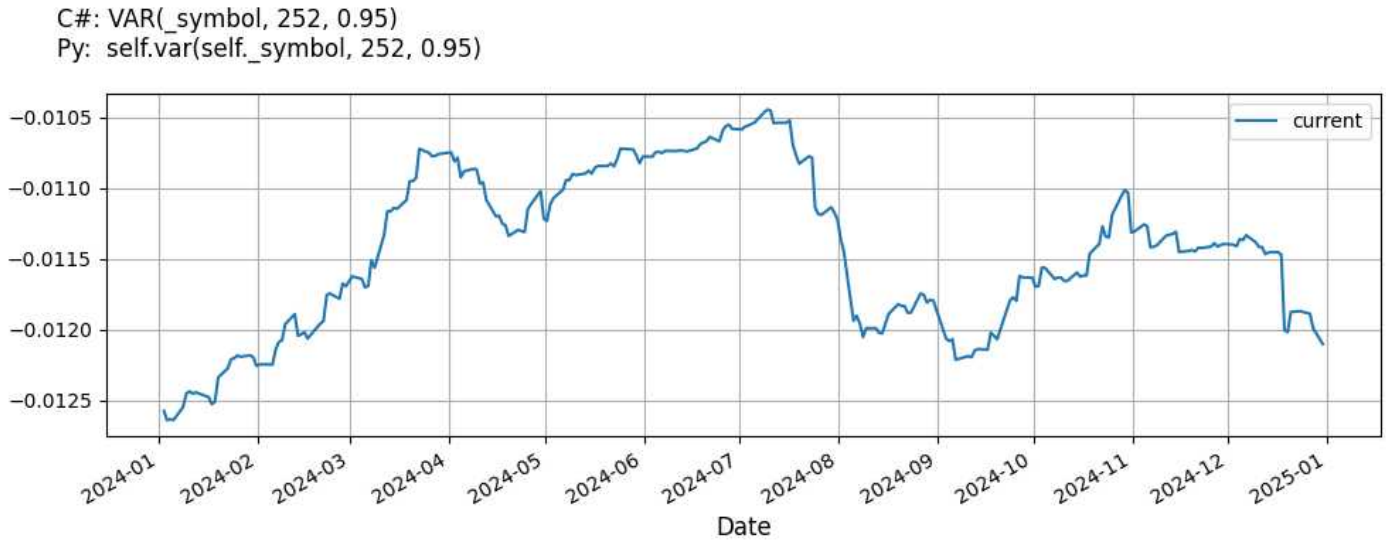
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class ValueAtRiskAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._valueatrisk = ValueAtRisk(252, 0.95)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._valueatrisk.update(bar.end_time, bar.close)

        if self._valueatrisk.is_ready:
            # The current value of self._valueatrisk is represented by self._valueatrisk.current.value
            self.plot("ValueAtRisk", "valueatrisk", self._valueatrisk.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ValueAtRisk` indicator properties:



Indicator History

To get the historical data of the `ValueAtRisk` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
PY
class ValueAtRiskAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._var = self.var(self._symbol, 252, 0.95)

        indicator_history = self.indicator_history(self._var, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._var, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._var, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Variable Index Dynamic Average

Introduction

This indicator computes the n-period adaptive weighted moving average indicator. VIDYAi = Pricei x F x ABS(CMOi) + VIDYAi-1 x (1 - F x ABS(CMOi)) where: VIDYAi - is the value of the current period. Pricei - is the source price of the period being calculated. F = 2/(Period_EMA+1) - is a smoothing factor. ABS(CMOi) - is the absolute current value of CMO. VIDYAi-1 - is the value of the period immediately preceding the period being calculated.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VIDYA Indicator

To create an automatic indicator for `VariableIndexDynamicAverage` , call the `vidya` helper method from the `QCAlgorithm` class. The `vidya` method creates a `VariableIndexDynamicAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class VariableIndexDynamicAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vidya = self.vidya(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._vidya.is_ready:
            # The current value of self._vidya is represented by self._vidya.current.value
            self.plot("VariableIndexDynamicAverage", "vidya", self._vidya.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `VariableIndexDynamicAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

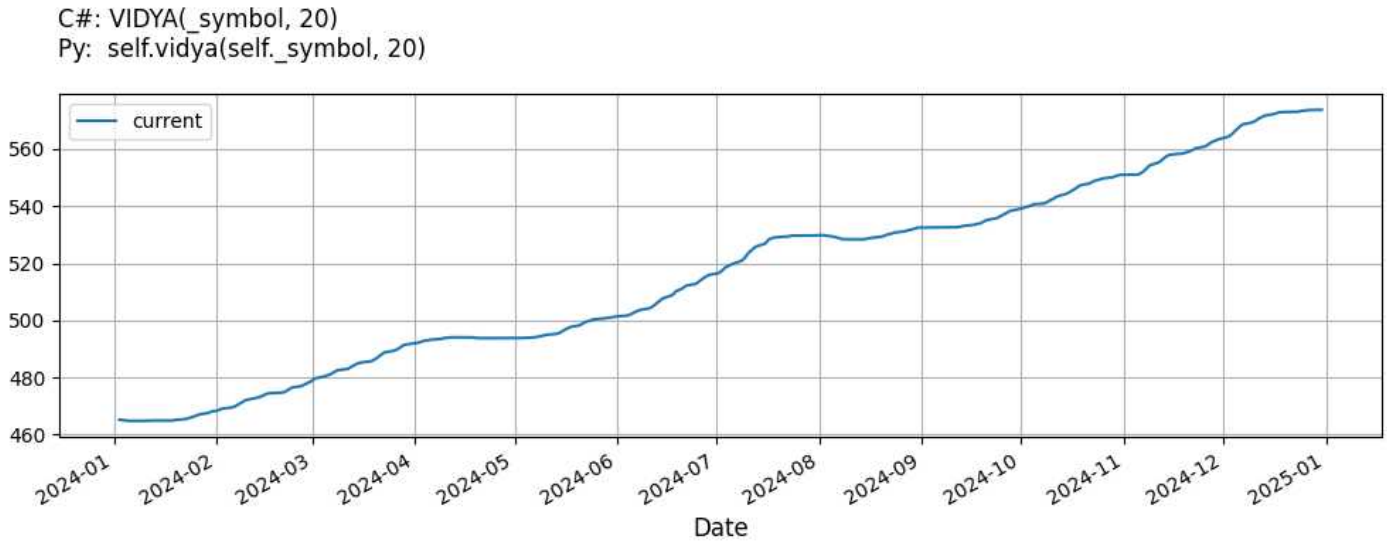
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class VariableIndexDynamicAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._variableindexdynamicaverage = VariableIndexDynamicAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._variableindexdynamicaverage.update(bar.end_time, bar.close)

        if self._variableindexdynamicaverage.is_ready:
            # The current value of self._variableindexdynamicaverage is represented by self._variableindexdynamicaverage.current.value
            self.plot("VariableIndexDynamicAverage", "variableindexdynamicaverage", self._variableindexdynamicaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `VariableIndexDynamicAverage` indicator properties:



Indicator History

To get the historical data of the `VariableIndexDynamicAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class VariableIndexDynamicAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vidya = self.vidya(self._symbol, 20)

        indicator_history = self.indicator_history(self._vidya, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vidya, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vidya, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Variance

Introduction

This indicator computes the n-period population variance.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using V Indicator

To create an automatic indicator for `Variance` , call the `v` helper method from the `QCAlgorithm` class. The `v` method creates a `Variance` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class VarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._v = self.v(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._v.is_ready:
            # The current value of self._v is represented by self._v.current.value
            self.plot("Variance", "v", self._v.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Variance` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

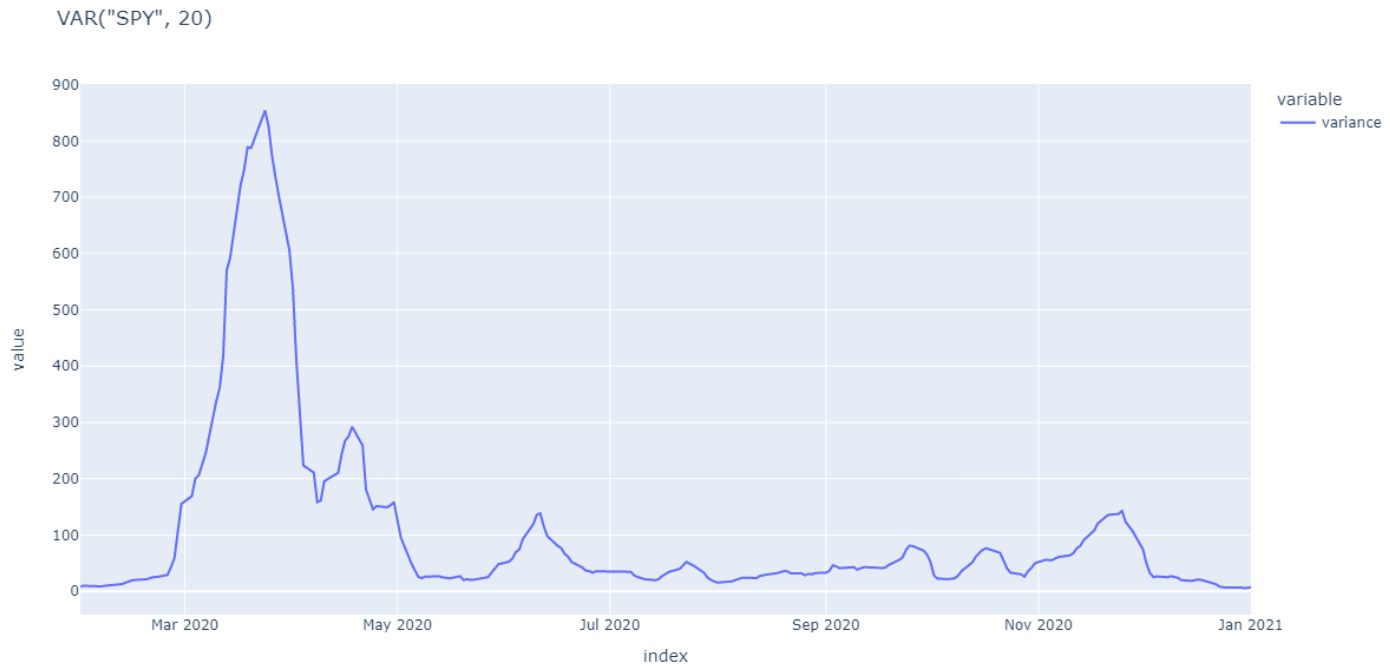
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class VarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._variance = Variance(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._variance.update(bar.end_time, bar.close)

        if self._variance.is_ready:
            # The current value of self._variance is represented by self._variance.current.value
            self.plot("Variance", "variance", self._variance.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Variance` indicator properties:



Indicator History

To get the historical data of the `Variance` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class VarianceAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._v = self.v(self._symbol, 20)

        indicator_history = self.indicator_history(self._v, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._v, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._v, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Vega

Introduction

Option Vega indicator that calculate the Vega of an option

To view the implementation of this indicator, see the LEAN GitHub repository .

Using V Indicator

To create an automatic indicator for `Vega` , call the `v` helper method from the `QCAlgorithm` class. The `v` method creates a `Vega` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class VegaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._v = self.v(self._option, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        if self._v.is_ready:
            # The current value of self._v is represented by self._v.current.value
            self.plot("Vega", "v", self._v.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Vega` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

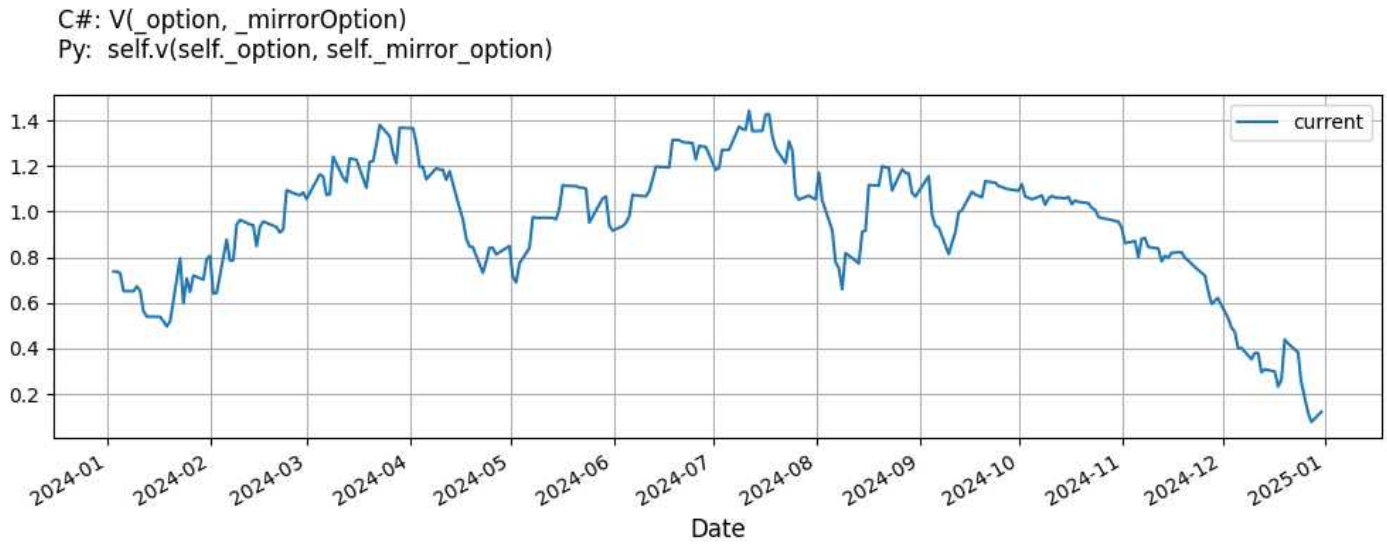
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class VegaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        interest_rate_model = InterestRateProvider()
        dividend_yield_model = DividendYieldProvider(self._symbol)

        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._vega = Vega(self._option, interest_rate_model, dividend_yield_model, self._mirror_option)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._vega.update(IndicatorDataPoint(self._symbol, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._option)
        if bar:
            self._vega.update(IndicatorDataPoint(self._option, bar.end_time, bar.close))
        bar = slice.quote_bars.get(self._mirror_option)
        if bar:
            self._vega.update(IndicatorDataPoint(self._mirror_option, bar.end_time, bar.close))

        if self._vega.is_ready:
            # The current value of self._vega is represented by self._vega.current.value
            self.plot("Vega", "vega", self._vega.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Vega` indicator properties:



Indicator History

To get the historical data of the `Vega` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```py
class VegaAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 20)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920C00564000");
        self._mirror_option = SymbolRepresentation.parse_option_ticker_osi("SPY 240920P00564000");
        self.add_option_contract(self._option, Resolution.DAILY);
        self.add_option_contract(self._mirror_option, Resolution.DAILY);

        self._v = self.v(self._option, self._mirror_option)

        indicator_history = self.indicator_history(self._v, [self._symbol, self._option, self._mirror_option], 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._v, [self._symbol, self._option, self._mirror_option],
timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._v, [self._symbol, self._option, self._mirror_option], datetime(2024,
7, 1), datetime(2024, 7, 5), Resolution.MINUTE)
```

Supported Indicators

Volume Profile

Introduction

This indicator represents an Indicator of the Market Profile with Volume Profile mode and its attributes

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VP Indicator

To create an automatic indicator for `VolumeProfile` , call the `vp` helper method from the `QCAlgorithm` class. The `vp` method creates a `VolumeProfile` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class VolumeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vp = self.vp(self._symbol, 3, 0.70, 0.05)

    def on_data(self, slice: Slice) -> None:

        if self._vp.is_ready:
            # The current value of self._vp is represented by self._vp.current.value
            self.plot("VolumeProfile", "vp", self._vp.current.value)
```

PY

For more information about this method, see the QCAlgorithm class .

You can manually create a `VolumeProfile` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class VolumeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._volumeprofile = VolumeProfile(3, 0.70, 0.05)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._volumeprofile.update(bar)

        if self._volumeprofile.is_ready:
            # The current value of self._volumeprofile is represented by self._volumeprofile.current.value
            self.plot("VolumeProfile", "volumeprofile", self._volumeprofile.current.value)
```

PY

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `VolumeProfile` indicator properties:

Indicator History

To get the historical data of the `VolumeProfile` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class VolumeProfileAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vp = self.vp(self._symbol, 3, 0.70, 0.05)

        indicator_history = self.indicator_history(self._vp, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vp, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vp, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Volume Weighted Average Price Indicator

Introduction

Volume Weighted Average Price (VWAP) Indicator: It is calculated by adding up the dollars traded for every transaction (price multiplied by number of shares traded) and then dividing by the total shares traded for the day.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VWAP Indicator

To create an automatic indicator for `VolumeWeightedAveragePriceIndicator` , call the `vwap` helper method from the `QCAlgorithm` class. The `vwap` method creates a `VolumeWeightedAveragePriceIndicator` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class VolumeWeightedAveragePriceIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vwap = self.vwap(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._vwap.is_ready:
            # The current value of self._vwap is represented by self._vwap.current.value
            self.plot("VolumeWeightedAveragePriceIndicator", "vwap", self._vwap.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `VolumeWeightedAveragePriceIndicator` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

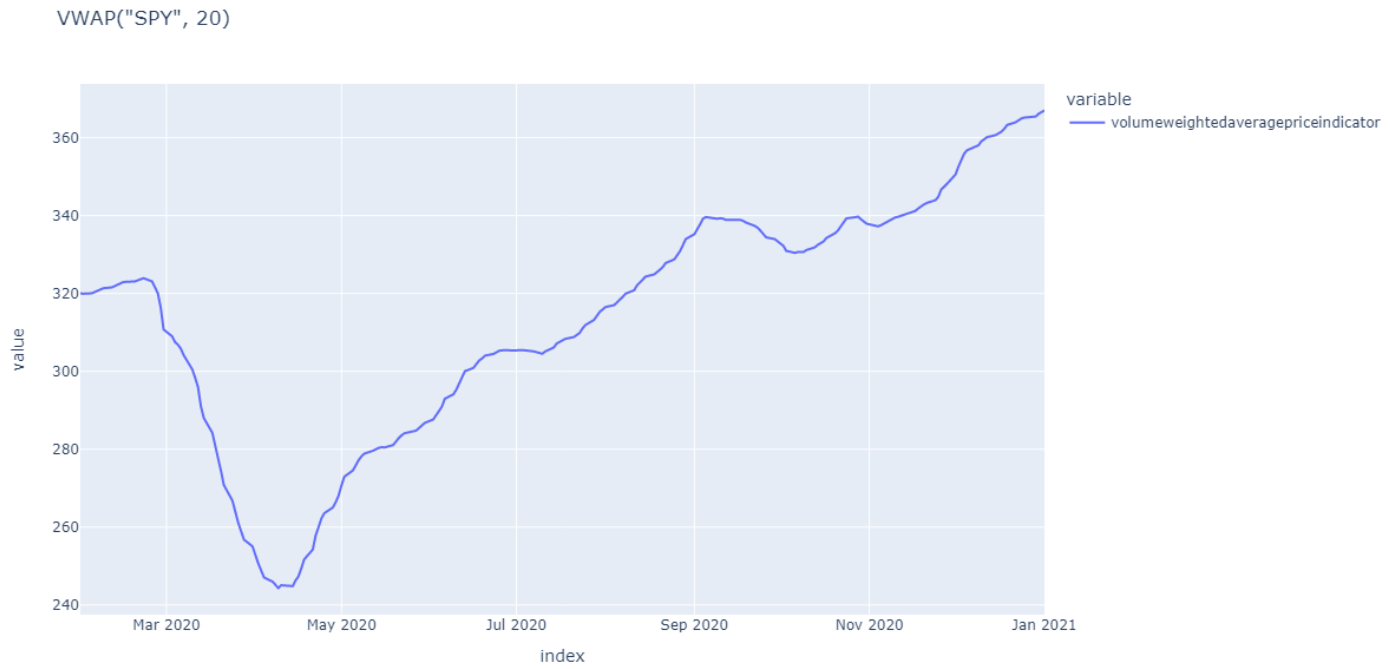
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class VolumeWeightedAveragePriceIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._volumeweightedaveragepriceindicator = VolumeWeightedAveragePriceIndicator(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._volumeweightedaveragepriceindicator.update(bar)

        if self._volumeweightedaveragepriceindicator.is_ready:
            # The current value of self._volumeweightedaveragepriceindicator is represented by
self._volumeweightedaveragepriceindicator.current.value
            self.plot("VolumeWeightedAveragePriceIndicator", "volumeweightedaveragepriceindicator",
self._volumeweightedaveragepriceindicator.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `VolumeWeightedAveragePriceIndicator` indicator properties:



Indicator History

To get the historical data of the `VolumeWeightedAveragePriceIndicator` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class VolumeWeightedAveragePriceIndicatorAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vwap = self.vwap(self._symbol, 20)

        indicator_history = self.indicator_history(self._vwap, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vwap, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vwap, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Volume Weighted Moving Average

Introduction

This indicator computes the Volume Weighted Moving Average (VWMA) It is a technical analysis indicator used by traders to determine the average price of an asset over a given period of time, taking into account both price and volume.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VWMA Indicator

To create an automatic indicator for `VolumeWeightedMovingAverage` , call the `vwma` helper method from the `QCAlgorithm` class. The `vwma` method creates a `VolumeWeightedMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class VolumeWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vwma = self.vwma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._vwma.is_ready:
            # The current value of self._vwma is represented by self._vwma.current.value
            self.plot("VolumeWeightedMovingAverage", "vwma", self._vwma.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `VolumeWeightedMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

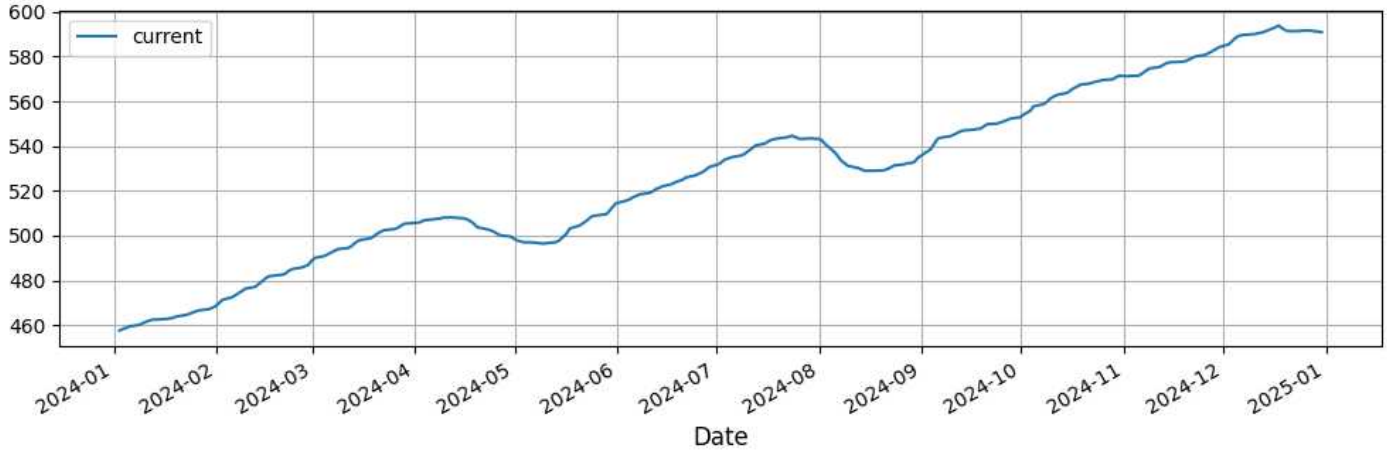
```
PY
class VolumeWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._volumeweightedmovingaverage = VolumeWeightedMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._volumeweightedmovingaverage.update(bar)

        if self._volumeweightedmovingaverage.is_ready:
            # The current value of self._volumeweightedmovingaverage is represented by self._volumeweightedmovingaverage.current.value
            self.plot("VolumeWeightedMovingAverage", "volumeweightedmovingaverage", self._volumeweightedmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `VolumeWeightedMovingAverage` indicator properties:

C#: VWMA(_symbol, 20)
Py: self.vwma(self._symbol, 20)



Indicator History

To get the historical data of the `VolumeWeightedMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class VolumeWeightedMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vwma = self.vwma(self._symbol, 20)

        indicator_history = self.indicator_history(self._vwma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vwma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vwma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Vortex

Introduction

This indicator represents the Vortex Indicator, which identifies the start and continuation of market trends. It includes components that capture positive (upward) and negative (downward) trend movements. This indicator compares the ranges within the current period to previous periods to calculate upward and downward movement trends.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using VTX Indicator

To create an automatic indicator for `Vortex` , call the `vtx` helper method from the `QCAlgorithm` class. The `vtx` method creates a `Vortex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class VortexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vtx = self.vtx(self._symbol, 14)

    def on_data(self, slice: Slice) -> None:

        if self._vtx.is_ready:
            # The current value of self._vtx is represented by self._vtx.current.value
            self.plot("Vortex", "vtx", self._vtx.current.value)
            # Plot all attributes of self._vtx
            self.plot("Vortex", "plus_vortex", self._vtx.plus_vortex.current.value)
            self.plot("Vortex", "minus_vortex", self._vtx.minus_vortex.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `Vortex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

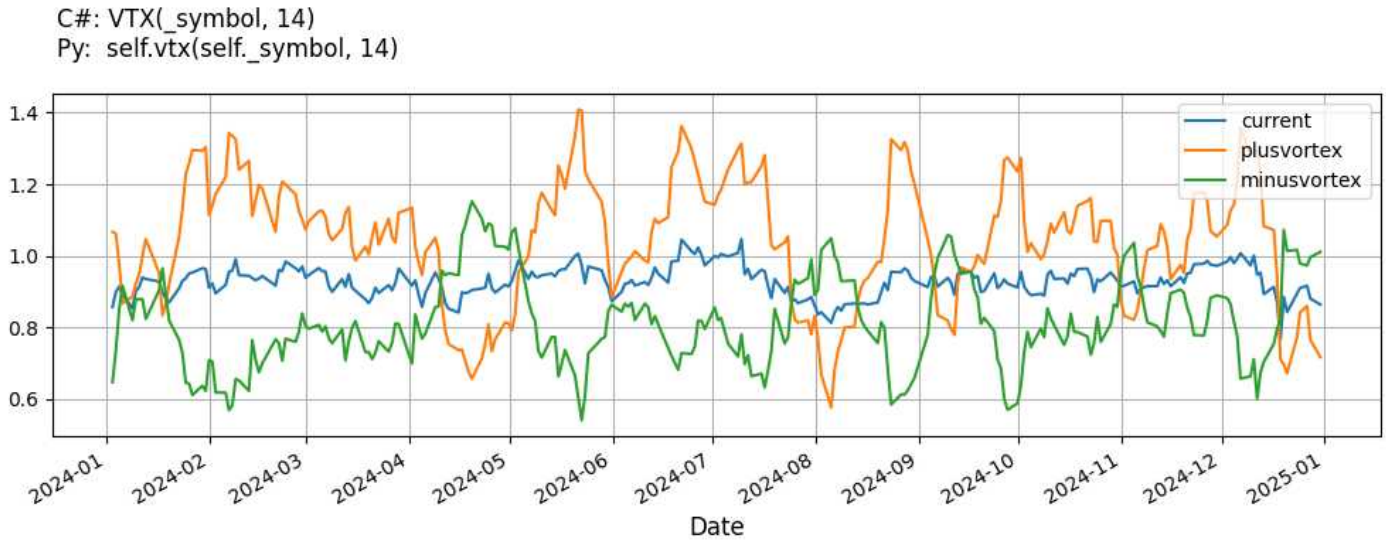
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class VortexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vortex = Vortex(14)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._vortex.update(bar)

        if self._vortex.is_ready:
            # The current value of self._vortex is represented by self._vortex.current.value
            self.plot("Vortex", "vortex", self._vortex.current.value)
            # Plot all attributes of self._vortex
            self.plot("Vortex", "plus_vortex", self._vortex.plus_vortex.current.value)
            self.plot("Vortex", "minus_vortex", self._vortex.minus_vortex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `Vortex` indicator properties:



Indicator History

To get the historical data of the `Vortex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class VortexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._vtx = self.vtx(self._symbol, 14)

        indicator_history = self.indicator_history(self._vtx, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._vtx, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._vtx, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        plus_vortex = indicator_history_df["plusvortex"]
        minus_vortex = indicator_history_df["minusvortex"]
```

Supported Indicators

Wilder Accumulative Swing Index

Introduction

This indicator calculates the Accumulative Swing Index (ASI) as defined by Welles Wilder in his book 'New Concepts in Technical Trading Systems'. $ASI_t = ASI_{t-1} + SI_t$ Where: ASI_{t-1} The for the previous period. SI_t The calculated for the current period.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ASI Indicator

To create an automatic indicator for `WilderAccumulativeSwingIndex` , call the `asi` helper method from the `QCAlgorithm` class. The `asi` method creates a `WilderAccumulativeSwingIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class WilderAccumulativeSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._asi = self.asi(self._symbol, 20, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:

        if self._asi.is_ready:
            # The current value of self._asi is represented by self._asi.current.value
            self.plot("WilderAccumulativeSwingIndex", "asi", self._asi.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `WilderAccumulativeSwingIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

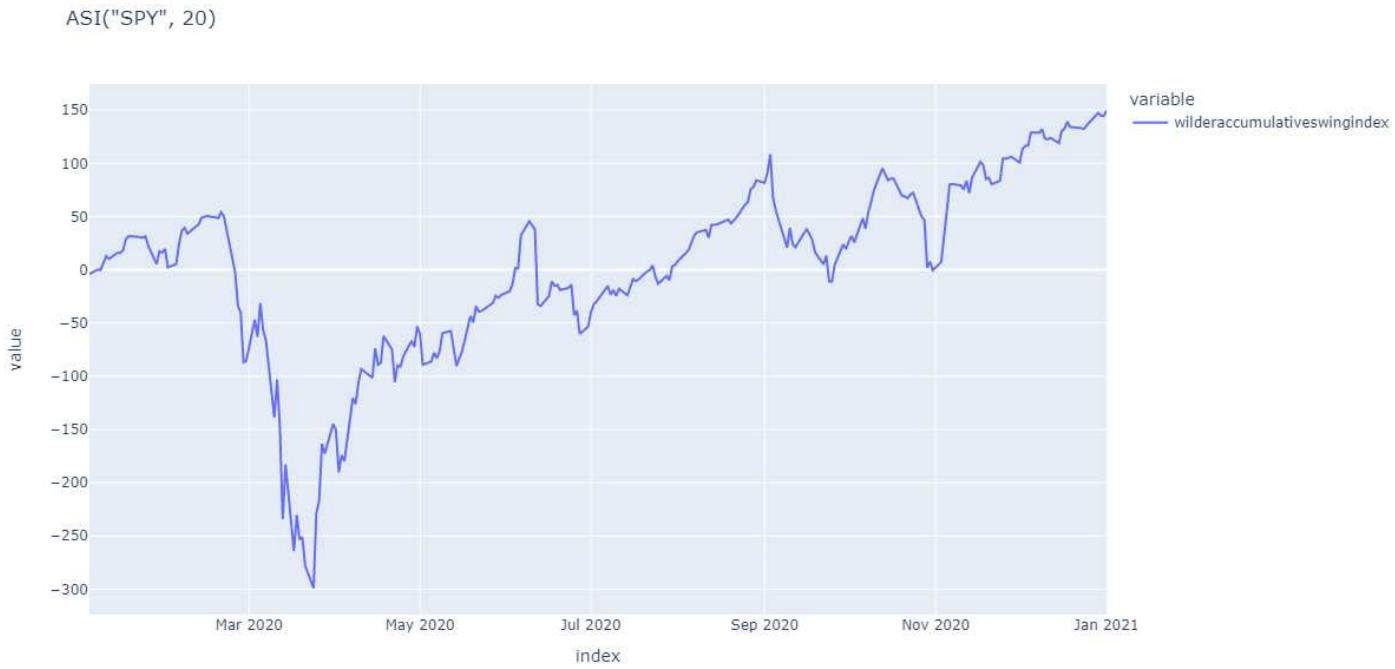
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class WilderAccumulativeSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wilderaccumulativeswingindex = WilderAccumulativeSwingIndex(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._wilderaccumulativeswingindex.update(bar)

        if self._wilderaccumulativeswingindex.is_ready:
            # The current value of self._wilderaccumulativeswingindex is represented by self._wilderaccumulativeswingindex.current.value
            self.plot("WilderAccumulativeSwingIndex", "wilderaccumulativeswingindex", self._wilderaccumulativeswingindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `WilderAccumulativeSwingIndex` indicator properties:



Indicator History

To get the historical data of the `WilderAccumulativeSwingIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class WilderAccumulativeSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._asi = self.asi(self._symbol, 20, Resolution.DAILY)

        indicator_history = self.indicator_history(self._asi, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._asi, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._asi, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Wilder Moving Average

Introduction

This indicator represents the moving average indicator defined by Welles Wilder in his book: New Concepts in Technical Trading Systems.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using WWMA Indicator

To create an automatic indicator for `WilderMovingAverage` , call the `wwma` helper method from the `QCAlgorithm` class. The `wwma` method creates a `WilderMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class WilderMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wwma = self.wwma(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._wwma.is_ready:
            # The current value of self._wwma is represented by self._wwma.current.value
            self.plot("WilderMovingAverage", "wwma", self._wwma.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `WilderMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

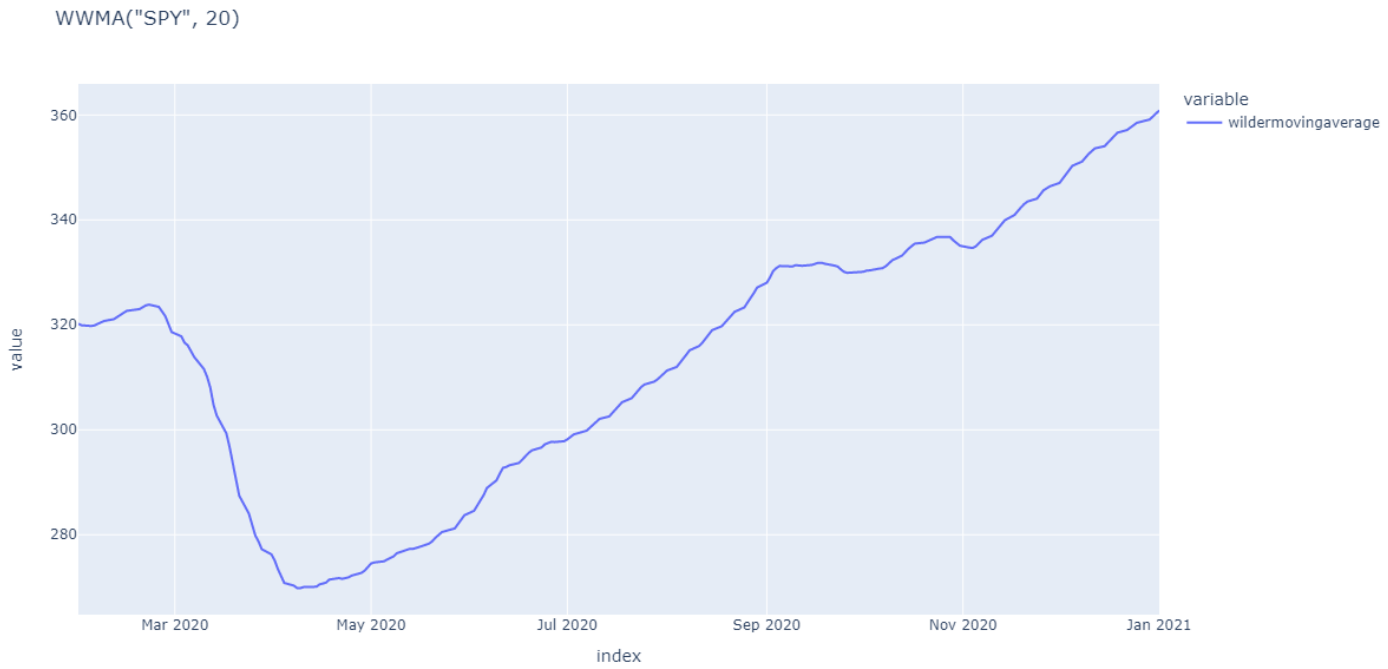
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class WilderMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wildermovingaverage = WilderMovingAverage(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._wildermovingaverage.update(bar.end_time, bar.close)

        if self._wildermovingaverage.is_ready:
            # The current value of self._wildermovingaverage is represented by self._wildermovingaverage.current.value
            self.plot("WilderMovingAverage", "wildermovingaverage", self._wildermovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `WilderMovingAverage` indicator properties:



Indicator History

To get the historical data of the `WilderMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class WilderMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wwma = self.wwma(self._symbol, 20)

        indicator_history = self.indicator_history(self._wwma, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._wwma, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._wwma, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Wilder Swing Index

Introduction

This indicator calculates the Swing Index (SI) as defined by Welles Wilder in his book 'New Concepts in Technical Trading Systems'. $SI_t = 50 * (N / R) * (K / T)$ Where: N Equals: $C_t - C_{t-1} + 0.5 * (C_t - O_t) + 0.25 * (C_{t-1} - O_{t-1})$ *See* R Found by selecting the expression with the largest value and then using the corresponding formula. Expression => Formula $|H_t - C_{t-1}|$ => $|H_t - C_t| - 0.5 * |L_t - C_{t-1}| + 0.25 * |C_{t-1} - O_{t-1}|$ $|L_t - C_{t-1}|$ => $|L_t - C_t| - 0.5 * |H_t - C_{t-1}| + 0.25 * |C_{t-1} - O_{t-1}|$ $|H_t - L_t|$ => $|H_t - L_{t-1}| + 0.25 * |C_{t-1} - O_{t-1}|$ *See* K Found by selecting the larger of the two expressions: $|H_t - C_{t-1}|$, $|L_t - C_{t-1}|$ *See* T The limit move, or the maximum change in price during the time period for the bar. Passed as limitMove via the constructor. *See*

To view the implementation of this indicator, see the LEAN GitHub repository .

Using SI Indicator

To create an automatic indicator for `WilderSwingIndex` , call the `si` helper method from the `QCAlgorithm` class. The `si` method creates a `WilderSwingIndex` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
class WilderSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._si = self.si(self._symbol, 20, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:

        if self._si.is_ready:
            # The current value of self._si is represented by self._si.current.value
            self.plot("WilderSwingIndex", "si", self._si.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `WilderSwingIndex` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

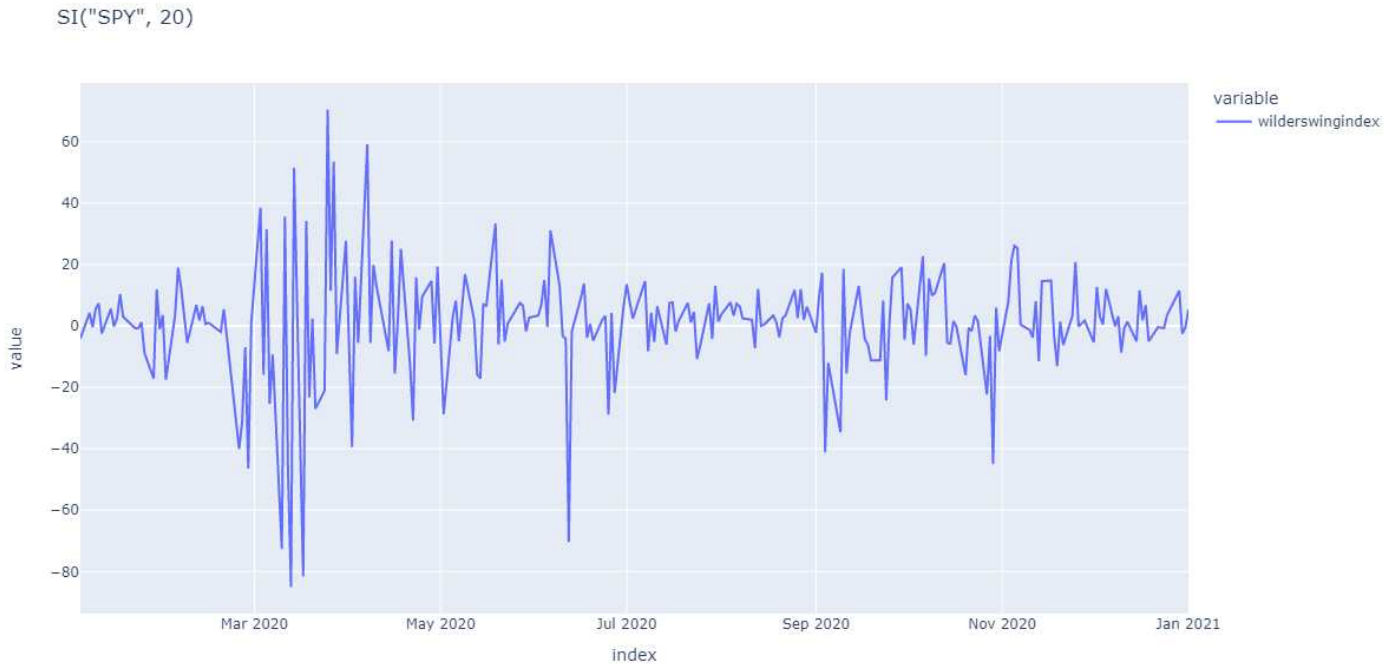
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
class WilderSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wilderswingindex = WilderSwingIndex(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._wilderswingindex.update(bar)

        if self._wilderswingindex.is_ready:
            # The current value of self._wilderswingindex is represented by self._wilderswingindex.current.value
            self.plot("WilderSwingIndex", "wilderswingindex", self._wilderswingindex.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `WilderSwingIndex` indicator properties:



Indicator History

To get the historical data of the `WilderSwingIndex` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class WilderSwingIndexAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._si = self.si(self._symbol, 20, Resolution.DAILY)

        indicator_history = self.indicator_history(self._si, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._si, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._si, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Williams Percent R

Introduction

Williams %R, or just %R, is the current closing price in relation to the high and low of the past N days (for a given N). The value of this indicator fluctuates between -100 and 0. The symbol is said to be oversold when the oscillator is below -80%, and overbought when the oscillator is above -20%.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using WILR Indicator

To create an automatic indicator for `WilliamsPercentR` , call the `wilr` helper method from the `QCAlgorithm` class. The `wilr` method creates a `WilliamsPercentR` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class WilliamsPercentRAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wilr = self.wilr(self._symbol, 20)

    def on_data(self, slice: Slice) -> None:

        if self._wilr.is_ready:
            # The current value of self._wilr is represented by self._wilr.current.value
            self.plot("WilliamsPercentR", "wilr", self._wilr.current.value)
            # Plot all attributes of self._wilr
            self.plot("WilliamsPercentR", "maximum", self._wilr.maximum.current.value)
            self.plot("WilliamsPercentR", "minimum", self._wilr.minimum.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `WilliamsPercentR` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

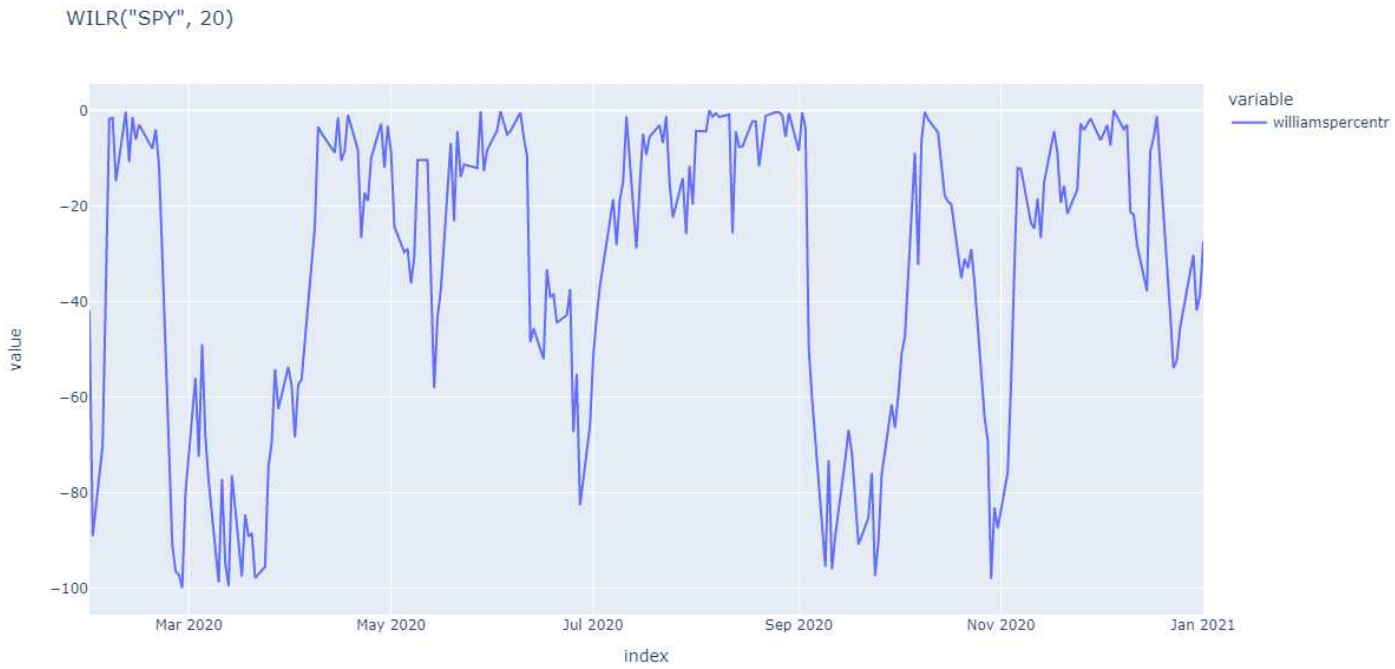
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class WilliamsPercentRAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._williamspercentr = WilliamsPercentR(20)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._williamspercentr.update(bar)

        if self._williamspercentr.is_ready:
            # The current value of self._williamspercentr is represented by self._williamspercentr.current.value
            self.plot("WilliamsPercentR", "williamspercentr", self._williamspercentr.current.value)
            # Plot all attributes of self._williamspercentr
            self.plot("WilliamsPercentR", "maximum", self._williamspercentr.maximum.current.value)
            self.plot("WilliamsPercentR", "minimum", self._williamspercentr.minimum.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `WilliamsPercentR` indicator properties:



Indicator History

To get the historical data of the `WilliamsPercentR` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
class WilliamsPercentRAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._wilr = self.wilr(self._symbol, 20)

        indicator_history = self.indicator_history(self._wilr, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._wilr, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._wilr, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)

        # Access all attributes of indicator_history
        indicator_history_df = indicator_history.data_frame
        maximum = indicator_history_df["maximum"]
        minimum = indicator_history_df["minimum"]
```

Supported Indicators

Zero Lag Exponential Moving Average

Introduction

This indicator represents the zero lag moving average indicator (ZLEMA) ie a technical indicator that aims is to eliminate the inherent lag associated to all trend following indicators which average a price over time.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ZLEMA Indicator

To create an automatic indicator for `ZeroLagExponentialMovingAverage` , call the `zlema` helper method from the `QCAlgorithm` class. The `zlema` method creates a `ZeroLagExponentialMovingAverage` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ZeroLagExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zlema = self.zlema(self._symbol, 10)

    def on_data(self, slice: Slice) -> None:

        if self._zlema.is_ready:
            # The current value of self._zlema is represented by self._zlema.current.value
            self.plot("ZeroLagExponentialMovingAverage", "zlema", self._zlema.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ZeroLagExponentialMovingAverage` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

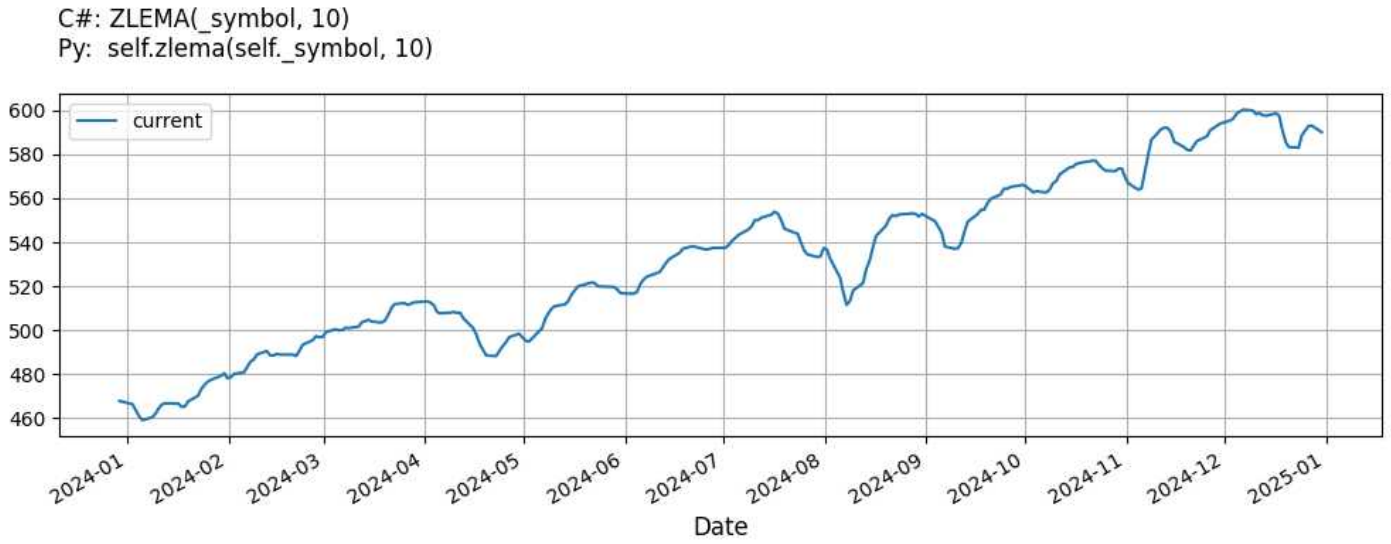
Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

```
PY
class ZeroLagExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zerolagexponentialmovingaverage = ZeroLagExponentialMovingAverage(10)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._zerolagexponentialmovingaverage.update(bar.end_time, bar.close)

        if self._zerolagexponentialmovingaverage.is_ready:
            # The current value of self._zerolagexponentialmovingaverage is represented by
self._zerolagexponentialmovingaverage.current.value
            self.plot("ZeroLagExponentialMovingAverage", "zerolagexponentialmovingaverage",
self._zerolagexponentialmovingaverage.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ZeroLagExponentialMovingAverage` indicator properties:



Indicator History

To get the historical data of the `ZeroLagExponentialMovingAverage` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ZeroLagExponentialMovingAverageAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zlema = self.zlema(self._symbol, 10)

        indicator_history = self.indicator_history(self._zlema, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._zlema, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._zlema, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Supported Indicators

Zig Zag

Introduction

The ZigZag indicator identifies significant turning points in price movements, filtering out noise using a sensitivity threshold and a minimum trend length. It alternates between high and low pivots to determine market trends.

To view the implementation of this indicator, see the LEAN GitHub repository .

Using ZZ Indicator

To create an automatic indicator for `ZigZag` , call the `zz` helper method from the `QCAlgorithm` class. The `zz` method creates a `ZigZag` object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. In most cases, you should call the helper method in the `initialize` method.

```
PY
class ZigZagAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zz = self.zz(self._symbol, 0.05, 1)

    def on_data(self, slice: Slice) -> None:

        if self._zz.is_ready:
            # The current value of self._zz is represented by self._zz.current.value
            self.plot("ZigZag", "zz", self._zz.current.value)
```

For more information about this method, see the QCAlgorithm class .

You can manually create a `ZigZag` indicator, so it doesn't automatically update. Manual indicators let you update their values with any data you choose.

Updating your indicator manually enables you to control when the indicator is updated and what data you use to update it. To manually update the indicator, call the `update` method. The indicator will only be ready after you prime it with enough data.

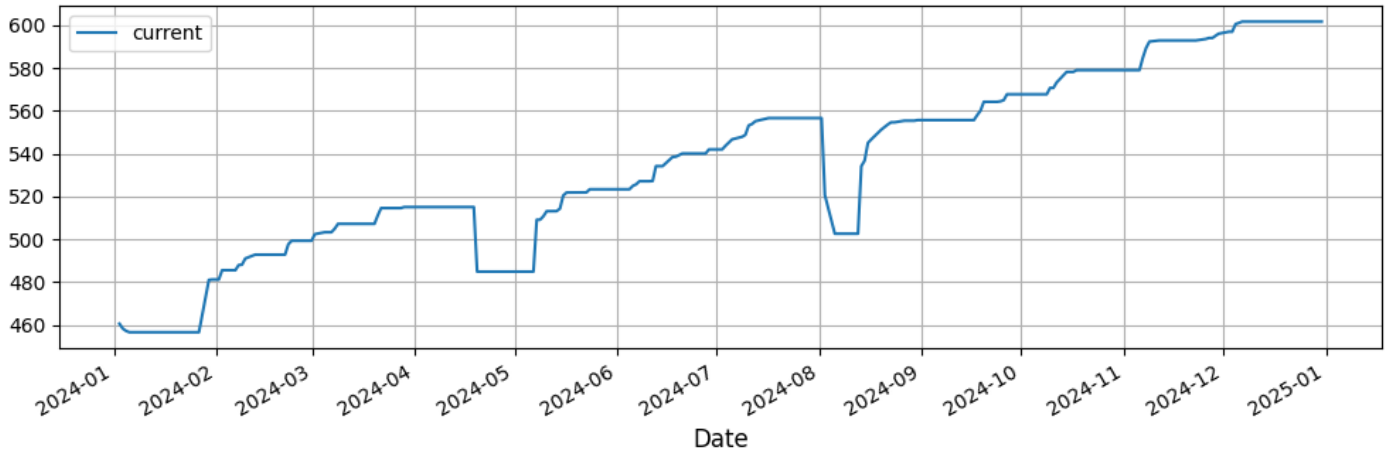
```
PY
class ZigZagAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zigzag = ZigZag(0.05, 1)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._symbol)
        if bar:
            self._zigzag.update(bar)

        if self._zigzag.is_ready:
            # The current value of self._zigzag is represented by self._zigzag.current.value
            self.plot("ZigZag", "zigzag", self._zigzag.current.value)
```

For more information about this indicator, see its reference .

Visualization

The following plot shows values for some of the `ZigZag` indicator properties:

C#: ZZ(_symbol, 0.05m, 1)
Py: self.zz(self._symbol, 0.05, 1)



Indicator History

To get the historical data of the `ZigZag` indicator, call the `self.indicator_history` method. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

PY

```
class ZigZagAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._zz = self.zz(self._symbol, 0.05, 1)

        indicator_history = self.indicator_history(self._zz, self._symbol, 100, Resolution.MINUTE)
        timedelta_indicator_history = self.indicator_history(self._zz, self._symbol, timedelta(days=10), Resolution.MINUTE)
        time_period_indicator_history = self.indicator_history(self._zz, self._symbol, datetime(2024, 7, 1), datetime(2024, 7, 5),
Resolution.MINUTE)
```

Indicators

Key Concepts

Introduction

Indicators translate a stream of data points into a numerical value you can use to detect trading opportunities. LEAN provides more than 100 pre-built technical indicators and candlestick patterns you can use in your algorithms. To view a list of all the indicators, see the Supported Indicators . One simple indicator to learn is the Identity indicator, which just returns the value of the asset.

```
pep = self.identity("PEP") # Pepsi ticker
```
PY

Design Philosophy

Technical indicators take in a stream of data points, `Bar` objects, or `TradeBar` objects and produce a continuous value. Candlestick patterns take in a stream of `Bar` or `TradeBar` objects and produce a value that's either 0, 1, or -1 to signify the presence of the pattern.

You can configure technical indicators and candlestick patterns to receive manual or automatic updates. With manual indicators, you update the indicator with whatever data you want and whenever you want. With automatic indicators, the algorithm automatically uses the security data to update the indicator value.

LEAN's indicators are implemented "on-line". This means as data pipes through, it updates the internal state of the indicator. Other platforms perform indicator math on an array of values in bulk which can be very inefficient when working on large volumes of data. You need to warm up your indicators and prime them with data to get them ready.

Indicator Types

You can classify an indicator as either a data point, bar, or TradeBar indicator. Their classification depends on the type of data they receive.

Data Point Indicators

Data point indicators use `IndicatorDataPoint` objects to compute their value. Some examples of data point indicators include the following:

- SimpleMovingAverage
- ExponentialMovingAverage
- Momentum

Bar Indicators

Bar indicators use `QuoteBar` or `TradeBar` objects to compute their value. Since Forex and CFD securities don't have `TradeBar` data, they use bar indicators. Candlestick patterns are examples of bar indicators. Some other examples of bar indicators include the following:

- WilliamsPercentR
- Stochastics
- AverageTrueRange

TradeBar Indicators

`TradeBar` indicators use `TradeBar` objects to compute their value. Some `TradeBar` indicators use the volume property of the `TradeBar` to compute their value. Some examples of `TradeBar` indicators include the following:

- Beta
- AdvanceDeclineVolumeRatio
- VolumeWeightedAveragePriceIndicator

Naming Convention

The class name to create a manual indicator is the indicator name spelled in pascal case. For example, to create a manual simple moving average indicator , use the `SimpleMovingAverage` class.

The method to create an automatic indicator is usually named after the acronym of the indicator name. For example, to create an automatic simple moving average indicator , use the `sma` method.

To view the class name and shortcut method of each indicator, see the Supported Indicators .

Create Indicators

There are two ways to create indicators. You can create indicators with the indicator constructor or use a helper method in the `QCAlgorithm` class. If you use the helper method, the indicator automatically updates as your algorithm receives new data.

```
PY
def initialize(self) -> None:
    # Create a manual indicator with the indicator constructor
    self.manual_bb = BollingerBands(20, 2)
    # Create an automatic indicator with the helper method
    self._symbol = self.add_crypto("BTCUSD").Symbol
    self.auto_bb = self.bb(self._symbol, 20, 2, Resolution.DAILY)
```

Set Historical Values Window Size

Indicators have a built-in `RollingWindow` that stores their historical values. The default window size is 2. To change the window size, set the `Window.Size` member.

```
PY
# Increase the Size to 10 for Bollinger Band
self.manual_bb.window.size = 10
self.auto_bb.window.size = 10

# Increase the Size to 20 for the middle band
self.manual_bb.middle_band.window.size = 20
self.auto_bb.middle_band.window.size = 20
```

When you decrease the size, it removes the oldest values that no longer fit in the `RollingWindow` . When you explicitly increase the `size` member, it doesn't automatically add any new elements to the `RollingWindow` . However, if you set the value of an index in the `RollingWindow` and the index doesn't currently exist, it fills the empty indices with `None` . For example, the following code increases the `size` to 10, sets the 10th element to the indicator current value, and sets the 6th-9th elements to `None` :

```
PY
self.manual_bb.window[9] = self.manual_bb.current
self.auto_bb.window[9] = self.auto_bb.current
```

Check Readiness

Indicators aren't always ready when you first create them. The length of time it takes to trust the indicator values depends on the indicator period. To know if an indicator is ready to use, use the `is_ready` property.

```
PY
if not self.auto_bb.is_ready:
    return
```

To get the number of samples the indicator has processed, use the `samples` property.

```
PY
samples = self.auto_bb.samples
```

The `RollingWindow` objects that store the historical values may not be full when the indicator is ready to use. To check if the `Window` is full, use its `is_ready` flag.

```
PY
if not self.auto_bb.window.is_ready:
    return
```

Track Update Events

When an indicator receives a new data point and updates its state, it triggers an update event. To be notified of update events, attach an event handler to the indicator object. The event handler receives 2 arguments: the indicator object and the latest `IndicatorDataPoint` it produced.

```py
# After you create the indicator, attach an event handler
self.auto_bb.updated += self.update_event_handler

# Define the event handler within the same class
def update_event_handler(self, indicator: object, indicator_data_point: IndicatorDataPoint) -> None:
    if indicator.is_ready:
        self.plot("Indicator", "Value", indicator_data_point.value)
```

Get Current Values

To access the indicator value, use the `current.value` property. Some indicators have one output and some indicators have multiple outputs. The `SimpleMovingAverage` indicator only has one output, the average price over the last n periods, so the `current.value` property returns this value. The `BollingerBand` indicator has multiple outputs because it has a simple moving average, an upper band, and a lower band. For indicators that have multiple outputs, refer to the Supported Indicators to see how to access the output values.

```py
sma = self.sma.current.value

current_price = self.auto_bb.current.value
bb_upper_band = self.auto_bb.upper_band.current.value
bb_lower_band = self.auto_bb.lower_band.current.value
```

You can implicitly cast indicators to the decimal version of their `current.value` property.

```py
if self.sma.current.value > self.auto_bb.upper_band.current.value:
    self.set_holdings(self._symbol, -0.1)
```

Get Historical Values

To access historical indicator values, you can use the indicator's built-in `RollingWindow` or you can call the `indicator_history` method to request values over a specific period of time.

Get Trailing Values

To access trailing indicator values, use reverse list access semantics with positive and negative indexing. The current (most recent) indicator value is at index 0, the previous value is at index 1 or the `previous` property, and so on until the length of the window. By default, the window length is 2. If you index the window with a number that's greater than the maximum index of the window, you get a `None` value and the window length grows to support the new index.

```py
current_bb = self.auto_bb.current # self.auto_bb[0]
previous_bb = self.auto_bb.previous # or self.auto_bb[1]
oldest_bb = self.auto_bb.window[-1]
none_value = self.auto_bb.window[100]

previous_upper_band = self.auto_bb.upper_band.previous
oldest_upper_band = self.auto_bb.upper_band.window[-1]
```

To access all of an indicator's historical values in the window, iterate through the indicator object.

```py
for value in self.auto_bb:
    self.log(f'{value}')
```

Request Historical Values

The `indicator_history` method also provides historical indicator values. This method resets your indicator, makes a history request , and updates the indicator with the historical data. Just like with regular history requests, the `indicator_history` method supports time periods based on a trailing number of bars, a trailing period of time, or a defined period of time. If you don't provide a `resolution` argument, it defaults to match the resolution of the security subscription.

```
# Get historical indicator values.
history = self.indicator_history(self.sma, self._symbol, timedelta(30))

# Organize the historical values into a DataFrame.
df = history.data_frame

# Iterate through the historical values.
for indicator_data_point in history:
    t = indicator_data_point.end_time
    value = indicator_data_point.value
```

To make the `indicator_history` method update the indicator with an alternative price field instead of the close (or mid-price) of each bar, pass a `selector` argument.

```
history = self.indicator_history(self.sma, self._symbol, timedelta(30), selector=Field.VOLUME)
```

Some indicators require the prices of two assets to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
self._spy = self.add_equity("SPY").symbol
self._aapl = self.add_equity('AAPL').symbol
self._beta = Beta("", self._aapl, self._spy, 21)
history = self.indicator_history(self._beta, [self._spy, self._aapl], 10, Resolution.DAILY)
```

If the indicator is an Option indicator with an underlying Equity, the `indicator_history` uses the `SCALED_RAW` data normalization mode when requesting historical prices.

Reset Indicators

To reset indicators, call the `reset` method.

```
self.auto_bb.reset()
```

The process to warm up your indicator depends on if it's a manual or automatic indicator .

If you are live trading Equities or backtesting Equities without the adjusted data normalization mode , reset your indicators when splits and dividends occur. If a split or dividend occurs, the data in your indicators becomes invalid because it doesn't account for the price adjustments that the split or dividend causes. To replace your indicator history with the newly-adjusted prices, call the `reset` method and then warm up the indicator with `SCALED_RAW` data from a history request .

```
# In OnData
if data.splits.contains_key(self._symbol) or data.dividends.contains_key(self._symbol):
    # Reset the indicator
    self._sma.reset()

    # Warm up the indicator
    trade_bars = self.history[TradeBar](self._symbol, self._sma.warm_up_period, Resolution.DAILY,
data_normalization_mode=DataNormalizationMode.SCALED_RAW)
    for trade_bar in trade_bars:
        self._sma.update(trade_bar.end_time, trade_bar.close)
```

Common Design Patterns

The location in your algorithm where you create and manage indicators depends on the type of universe you use.

One Asset In a Static Universe

If you only have one security in your algorithm, you can create indicators for it during initialization .

```
def initialize(self):
    self._symbol = self.add_equity("SPY").symbol
    self.settings.automatic_indicator_warm_up = True
    self._sma = self.sma(self._symbol, 20, Resolution.DAILY)
```

Multiple Assets or a Dynamic Universe

If your algorithm has multiple assets or a dynamic universe of assets, add custom members to the `Security` objects as you receive them in `on_securities_changed`.

```
def on_securities_changed(self, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        # Create an indicator
        security.indicator = self.sma(security.Symbol, 10)

        # Warm up the indicator
        self.warm_up_indicator(security.symbol, security.indicator)

    for security in changes.removed_securities:
        self.deregister_indicator(security.indicator)
```

Differences From Third-Party Indicators

The values of our indicators can sometimes produce different results than the indicators on other platforms. There can be several reasons for these discrepancies.

Price Differences

If you find differences in indicator values, compare the prices that are fed into the indicator on each platform. If the input data is slightly different, the indicator values will be different. To test if it's a difference in price data, feed in the data from the third-party platform into our indicators. We validate the indicators against third-party sources and when the values are the same, the indicator values are similar too.

Timestamp Differences

We timestamp our data to the time when the period ends. Many other platforms timestamp to the beginning of the candle.

Implementation Differences

Some indicators can have slightly different default arguments for their implementation. A common difference across platforms is to use a different `MovingAverageType` for the indicators. To view the default arguments for all our indicators, see Supported Indicators. To view the full implementation of our indicators, see the LEAN GitHub repository.

Warm Up Period Differences

Some platforms use all of the historical data to warm up their indicators while LEAN indicators have a fixed number of bars they need to warm up. As a result, indicators with long memory like the Exponential Moving Average can have slightly different values across platforms.

Risk Free Interest Rate Differences

Some indicators, like Sharpe ratios and Sortino ratios, are a function of the risk free interest rate. LEAN provides several different ways to define the interest rate, which can lead to different indicator values compared to other platforms.

Examples

Demonstration Algorithms
IndicatorSuiteAlgorithm.py Python EmaCrossUniverseSelectionAlgorithm.py Python MACDTrendAlgorithm.py Python
RegressionChannelAlgorithm.py Python TalibIndicatorsAlgorithm.py Python

Indicators

Manual Indicators

Introduction

Manual indicators are indicators that don't automatically update from the underlying security data. Manual updates let you customize how you update the indicator, such as using a custom field or using Renko Bars . It's not always necessary to create manual indicators though. Automatic indicators are easier to create and update.

Naming Convention

The class name to create a manual indicator is the indicator name spelled in pascal case. For example, to create a manual simple moving average indicator , use the `SimpleMovingAverage` class.

Create Manual Indicators

To create manual indicators, instantiate an indicator with its constructor. To view all of the available indicators and their constructors, see Supported Indicators .

```py
# Create an indicator
self.delay = Delay(20)

# Create a candlestick pattern
self.two_crows = TwoCrows()
```

You can track indicators by their name. To name a manual indicator, pass a string as the first argument to the constructor.

```py
# Name an indicator
self.delay = Delay("AAPL Past Price", 20)
self.log(self.delay.name)

# Name a candlestick pattern
self.two_crows = TwoCrows("Two Crows Pattern")
self.log(self.two_crows.name)
```

If you don't name an indicator, it's given a default name. For example, the default name for a `Delay` indicator is "Delay(period)".

Manual Updates

With manual updates, you control what data you use to update the indicator. For instance, you can use the 3:30 PM price in your daily moving average instead of the daily closing price or you can use the maximum temperature of the past 10 cloudy days.

To update an indicator, call the `update` method. The `update` method expects one of the following arguments:

- Time/decimal pair
- `IndicatorDataPoint`
- `QuoteBar`
- `TradeBar`
- Custom data bar

To view what data type you should use to update an indicator, see Supported Indicators .

You can update indicators at any point in your algorithm, but the most common places are during the `on_data` event handler or during a consolidation event handler.

```
def initialize(self) -> None:
    self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

    self._rsi = RelativeStrengthIndex(10, MovingAverageType.SIMPLE)
    self.ad = AccumulationDistribution()

    self.consolidator = TradeBarConsolidator(timedelta(3))   # 10 x 3 daily trade bars to warm up the RSI.
    self.consolidator = QuoteBarConsolidator(timedelta(3))   # 10 x 3 daily quote bars to warm up the RSI.
    self.consolidator = RenkoBarConsolidator(1)     # Renko consolidator that emits a bar when the price moves $1
    # Update the AD indicator with the consolidated bar
    self.consolidator.data_consolidated += self.on_data_consolidated
    self.subscription_manager.add_consolidator(self._symbol, self.consolidator)

def on_data_consolidated(self, sender, bar):
    self.ad.update(bar)

def on_data(self, slice: Slice) -> None:
    # Update the RSI indicator value with the new input close price every day
    if slice.bars.contains_key(self._symbol):
        bar = slice.bars[self._symbol]
        self.rsi.update(bar.end_time, bar.close)
```

Automatic Updates

With automatic updates, your indicators automatically update with the security data on a schedule you set. To configure automatic updates, create a consolidator and then call the `register_indicator` method. If you register an indicator for automatic updates, don't call the indicator's `update` method or else the indicator will receive double updates.

```
# Create a security subscription
self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol

# Create a manual indicator
self.indicator = RelativeStrengthIndex(10, MovingAverageType.SIMPLE)

# Create a consolidator
consolidator = TradeBarConsolidator(3)            # 10 x 3 min trade bars to warm up the RSI.
consolidator = QuoteBarConsolidator(4)            # 10 x 4 min quote bars to warm up the RSI
consolidator = RenkoConsolidator(1)               # Renko consolidator that emits a bar when the price moves $1
consolidator = IdentityDataConsolidator[TradeBar]()   # Identity consolidator that passes through each TradeBar data point

# Register the indicator to update with the consolidated data
self.register_indicator(self._symbol, self.indicator, consolidator)
```

Data point indicators use only a single price data in their calculations. By default, those indicators use the closing price. For assets with `TradeBar` data, that price is the `TradeBar` close price. For assets with `QuoteBar` data, that price is the mid-price of the bid closing price and the ask closing price. To create an indicator with the other fields like the `Open` , `High` , `Low` , or `Close` , provide a `selector` argument to the `register_indicator` method.

```
self.register_indicator(self._symbol, self.indicator, consolidator, Field.HIGH)
```

The `Field` class has the following `selector` properties:

To stop automatically updating an indicator, pass the indicator to the `deregister_indicator` property method.

```
self.deregister_indicator(self.indicator)
# Alias:
# self.unregister_indicator(self.indicator)
```

Warm Up Indicators

Indicators use historical data to compute their value. Before you start trading with an indicator, warm it up. There are several ways to warm-up manual indicators.

Manual Indicator Warm-up

If you have access to the `QCAlgorithm` object, you can manually warmup indicators with a history request .

```
# Create a manual indicator.
self._sma = SimpleMovingAverage(20)
# Get 20 bars of daily data.
history = algorithm.history[TradeBar](self._symbol, 20, Resolution.DAILY)
# Manually warm up the indicator with the historical close prices.
for bar in history:
    self._sma.update(bar.end_time, bar.close)
```

PY

Warm-up Helper Method

If an indicator inherits (implements) the `IIndicatorWarmUpPeriodProvider` interface, you can warm it up with the `warm_up_indicator` method.

```
# Warm up the indicator using the warm-up helper.
self._sma = SimpleMovingAverage(20)
algorithm.warm_up_indicator(self._symbol, self._sma)
```

PY

To warm up the indicator with a resolution that's different from the security resolution, pass a resolution or `timedelta` argument to the `warm_up_indicator` method. The resolution you provide should be greater than or equal to the security resolution. For example, if the security has minute resolution data, you should warm up the indicator with data that spans at least one minute.

```
# Warm-up with daily bars.
algorithm.warm_up_indicator(self._symbol, self._sma, Resolution.DAILY)

# Warm-up with 3-day bars.
algorithm.warm_up_indicator(self._symbol, self._sma, timedelta(days=3))
```

PY

The `warm_up_indicator` method uses the default `value` of the historical data to warm up the indicator. In most cases, this is the closing price. To warm up the indicator with an alternative price field, pass a `Field` argument to the method.

```
# Warm up the indicator using daily high prices instead of close prices.
algorithm.warm_up_indicator(self._symbol, self._sma, Resolution.DAILY, Field.HIGH)
```

PY

Some indicators require the prices of two assets to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
# Use the warm-up helper for an indicator that needs the data of 2 assets.
self._spy = self.add_equity("SPY").symbol
self._aapl = self.add_equity('AAPL').symbol
self._beta = Beta("", self._aapl, self._spy, 21)
self.warm_up_indicator([self._aapl, self._spy], self._beta, Resolution.DAILY)
```

PY

Algorithm Warm-up

If you create indicators at the beginning of your algorithm, you can set an algorithm warm-up period to warm up the indicators. When you set an algorithm warm-up period, the engine pumps data in and automatically updates all the indicators from before the start date of the algorithm. To ensure that all the indicators are ready after the algorithm warm-up period, choose a lookback period that contains sufficient data.

```
# In the initialize() method, request data and construct the indicators you need.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
self._sma = SimpleMovingAverage(20)
self.set_warm_up(20)

# In the on_data event handler, update your indicators.
# Use the self.is_warming_up flag to skip some code execution.
self._sma.update(data[self._symbol]) # Delete this line if you registered the indicator for automatic updates.
if self.is_warming_up:
    return
```

PY

Timing Considerations

In some cases, you might create and warm up the indicator during its sampling period. For example, say the security resolution is minute, the

indicator resolution is daily, and you create and warm-up the indicator at noon without using the `set_warm_up` method. In this example, the history request that gathers data to warm up the indicator won't contain any data from the current day. Furthermore, if you set up a consolidator to update the indicator, the consolidator also won't aggregate any data from before noon. It doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Liquidity Considerations

When LEAN makes a history request to warm up an indicator, it may not return enough data to warm up the indicator. After warming up an indicator, check for readiness , reset it , and increase the lookback period for a new history request if necessary.

```py
# In Initialize
symbol = self.add_equity("SPY").symbol
self._sma = SimpleMovingAverage(20)
self.warm_up_indicator(symbol, self._sma, Resolution.DAILY)
if not self._sma.is_ready:
    # The history request didn't return enough data to warm up the indicator.
    # Reset it, and increase the lookback period and try again.
    self._sma.reset()
    self.indicator_history(self._sma, symbol, 40, Resolution.DAILY)
```

Delay Indicator

The `Delay` indicator delays its input by a specified number of bars. It is mainly used to combine indicators, enabling you to set a delay without using built-in rolling windows . For example, you can create a displaced moving average by applying a `Delay` to a `SimpleMovingAverage` .

```py
# Create a 20-bar Delay indicator
self._delay = Delay(20)

# Create a displaced SMA by combining Delay with SMA
sma = SimpleMovingAverage(10)
displaced_sma = IndicatorExtensions.of(Delay(5), sma)
```

For a full example of using the `Delay` indicator to create a displaced SMA ribbon strategy, see Combining Indicators Examples .

For more information about this indicator, see the class reference and implementation .

Examples

The following examples demonstrate some common practices for using manual indicators.

Example 1: Universe Selection

The following algorithm selects the stocks within 5% of the 1-year maximum price among the top 100 liquid stocks. To do so, we make use of Maximum indicator to do so. Then, we hold the stocks with price > EMA > SMA , which indicates an upward accelerating trend.

PY

```
class ManualIndicatorAlgorithm(QCAlgorithm):
    maximum_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Select a popularity-based universe with indicators in a Selection function.
        self._universe = self.add_universe(self.selection)

        # Set a scheduled event to rebalance daily on the daily indicator signals.
        self.schedule.on(
            self.date_rules.every_day(),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def selection(self, fundamentals: list[Fundamental]) -> list[Symbol]:
        selected = []

        # Initially filtered for the top 100 liquid stocks first.
        filtered = sorted(fundamentals, key=lambda f: f.dollar_volume, reverse=True)[:100]

        for f in filtered:
            if f.symbol not in self.maximum_by_symbol:
                self.maximum_by_symbol[f.symbol] = Maximum(252)
                # Warm up the Maximum indicator to its readiness to use immediately.
                history = self.history[TradeBar](f.symbol, 252, Resolution.DAILY)
                for bar in history:
                    self.maximum_by_symbol[f.symbol].update(bar.end_time, bar.close)
            else:
                # Update the indicator with the last known adjusted price daily.
                self.maximum_by_symbol[f.symbol].update(f.end_time, f.adjusted_price)

            # Select to trade if the current price is within 5% of the maximum price of the last year.
            # Close to the maximum price provides evidence of high popularity for the fund to support the trend.
            if f.adjusted_price >= self.maximum_by_symbol[f.symbol].current.value * 0.95:
                selected.append(f.symbol)

        return selected

    def rebalance(self) -> None:
        def to_buy(symbol):
            security = self.securities[symbol]
            # Buy the stocks whose prices are above the EMA and the EMA is above the SMA, meaning their trend is upward accelerating.
            return security.price > security.ema.current.value > security.sma.current.value

        symbols_to_buy = [symbol for symbol in self._universe.selected if to_buy(symbol)]

        # Equally invest in the selected stocks to dissipate the capital risk evenly.
        count = len(symbols_to_buy)
        if count > 0:
            targets = [PortfolioTarget(symbol, 1 / count) for symbol in symbols_to_buy]
            # Liquidate the positions that are not on an upward trend or are not popular anymore.
            self.set_holdings(targets, liquidate_existing_holdings=True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            symbol = added.symbol
            # Create an EMA and an SMA manual indicator for trade signal generation.
            added.ema = ExponentialMovingAverage(20)
            added.sma = SimpleMovingAverage(20)

            # Warm up the indicators to ensure their readiness to use them immediately.
            self.warm_up_indicator(symbol, added.ema, Resolution.DAILY)
            self.warm_up_indicator(symbol, added.sma, Resolution.DAILY)

            # Subscribe to the indicators to update daily price data for updated trade signal generation.
            self.register_indicator(symbol, added.ema, Resolution.DAILY)
            self.register_indicator(symbol, added.sma, Resolution.DAILY)

        for removed in changes.removed_securities:
            # Cancel data subscription to release computation resources when leaving the universe.
            self.deregister_indicator(removed.ema)
            self.deregister_indicator(removed.sma)
```

Indicators

Automatic Indicators

Introduction

Automatic indicators are indicators that automatically update from the underlying security data. Automatic indicators do everything that manual indicators do, but with automatic updates registered for you.

Naming Convention

The method to create an automatic indicator is usually named after the acronym of the indicator name. For example, to create an automatic simple moving average indicator , use the `sma` method.

Create Automatic Indicators

To create automatic indicators, call the indicator helper method from the `QCAlgorithm` class. The indicator helper methods create an indicator object, hooks it up for automatic updates, and returns it so you can used it in your algorithm. To view the helper method for each indicator, see Supported Indicators . In most cases, you should call the helper method in the `initialize` method.

The indicator resolution must be greater than or equal to the resolution of the security subscription. For instance, if your security subscription is for minute resolution data, the indicator resolution should be minute, hour, or daily resolution.

```
PY
# In Initialize()
symbol = self.add_equity("SPY").symbol

#  - Create an indicator
self._sma = self.sma(symbol, 20, Resolution.DAILY)

#  - Create a candlestick pattern
patterns = CandlestickPatterns(self)
self.two_crows = patterns.two_crows(self._symbol)
```

When you create an indicator with a helper method, the indicator is given a default name. You can track indicators by their name. The default name for the `SimpleMovingAverage` indicator is " sma (period , ticker _ resolution)". For example, in the preceding code snippet, the `SimpleMovingAverage` indicator is named " sma (20,SPY_day)". To get the name of an indicator, inspect its `name` property.

```
PY
self.log(self.sma.name)
self.log(self.two_crows.name)
```

Alternative Price Fields

Data point indicators use only a single price data in their calculations. By default, those indicators use the closing price. For assets with `TradeBar` data, that price is the `TradeBar` close price. For assets with `QuoteBar` data, that price is the mid-price of the bid closing price and the ask closing price. To create an indicator with the other fields like the `OPEN` , `HIGH` , `LOW` , or `CLOSE` , provide a `selector` argument to the indicator helper method.

```
PY
# Define a 10-period daily RSI indicator with shortcut helper method
# Select the Open price to update the indicator
self._rsi = self.rsi("SPY", 10,  MovingAverageType.SIMPLE, Resolution.DAILY, Field.OPEN)
```

The `Field` class has the following `selector` properties:

Warm Up Indicators

Indicators compute their value based on historical data. Before you start trading with an indicator, warm it up. There are several ways to warm-up automatic indicators.

Algorithm Warm-up

You can set an algorithm warm-up period to warm up the indicators. When you set an algorithm warm-up period, the engine pumps data in and automatically updates all the indicators from before the start date. To ensure that all the indicators are ready after the algorithm warm-up period, choose a look-back period that contains sufficient data.

```
# In Initialize
symbol = self.add_equity("SPY").symbol
self._sma = self.sma(symbol, 20, Resolution.DAILY)
self.set_warm_up(20, Resolution.DAILY)

# In OnData
if self.is_warming_up:
    return
```

PY

Manual Indicator Warm-up

You can manually warm up indicators with a history request .

```
# In Initialize
symbol = self.add_equity("SPY").symbol
self._sma = self.sma(symbol, 20, Resolution.DAILY)

history = self.history(symbol, 20, Resolution.DAILY)
if not history.empty:
    for time, row in history.loc[symbol].iterrows():
        self._sma.update(time, row.close)
```

PY

Warm-up Helper Method

If an indicator inherits the `IIndicatorWarmUpPeriodProvider` class, you can warm it up with the `warm_up_indicator` method.

```
self._sma = self.sma(self._symbol, 20)
self.warm_up_indicator(self._symbol, self._sma)
```

PY

To warm up the indicator with a resolution that's different from the security resolution, pass a resolution or `timedelta` argument to the `warm_up_indicator` method. The resolution you provide should be greater than or equal to the security resolution. For example, if the security has minute resolution data, you should warm up the indicator with data that spans at least one minute. If the indicator resolution is different from the security resolution, provide the indicator resolution as the argument to properly warm up the indicator.

```
# Warm up with daily bars
self.warm_up_indicator(self._symbol, self._sma, Resolution.DAILY)

# Warm up with 3-day bars
self.warm_up_indicator(self._symbol, self._sma, timedelta(days=3))
```

PY

The `warm_up_indicator` method uses the default `value` of the historical data to warm up the indicator. In most cases, this is the closing price. To warm up the indicator with a different data field, pass a `Field` argument to the method.

```
self.warm_up_indicator(self._symbol, self._sma, Resolution.DAILY, Field.HIGH)
```

PY

Some indicators require the prices of two assets to compute their value (for example, Beta). In this case, pass a list of the `Symbol` objects to the method.

```
self._spy = self.add_equity("SPY").symbol
self._aapl = self.add_equity('AAPL').symbol
self._beta = self.B(self._aapl, self._spy, 21, Resolution.DAILY)
self.warm_up_indicator([self._aapl, self._spy], self._beta, Resolution.DAILY)
```

PY

Automatic Indicator Warm-up

You can set the `self.settings.automatic_indicator_warm_up` property to true before you create most indicators to automatically warm them up.

This technique doesn't work for all indicators.

```py
# In Initialize
symbol = self.add_equity("SPY").symbol
self.settings.automatic_indicator_warm_up = True
self._sma = self.sma(symbol, 20, Resolution.DAILY)
```

The `automatic_indicator_warm_up` settings does not account for built-in `RollingWindow` that stores their historical values , use the `warm_up_indicator` or the `indicator_history` method instead.

Timing Considerations

In some cases, you might create and warm up the indicator during its sampling period. For example, say the security resolution is minute, the indicator resolution is daily, and you create and warm-up the indicator at noon without using the `set_warm_up` method. In this example, the history request that gathers data to warm up the indicator won't contain any data from the current day. Furthermore, if you set up a consolidator to update the indicator, the consolidator also won't aggregate any data from before noon. It doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Liquidity Considerations

When LEAN makes a history request to warm up an indicator, it may not return enough data to warm up the indicator. After warming up an indicator, check for readiness , reset it , and increase the lookback period for a new history request if necessary.

```py
# In Initialize
symbol = self.add_equity("SPY").symbol
self.settings.automatic_indicator_warm_up = True
self._sma = self.sma(symbol, 20, Resolution.DAILY)
if not self._sma.is_ready:
    # The history request didn't return enough data to warm up the indicator.
    # Reset it, and increase the lookback period and try again.
    self._sma.reset()
    self.indicator_history(self._sma, symbol, 40, Resolution.DAILY)
```

Deregister Indicators

To stop the automatic updates of an indicator, pass the indicator to the `deregister_indicator` property method.

```py
self.deregister_indicator(self._sma)
# Alias:
# self.unregister_indicator(self._sma)
```

Common Mistakes

Avoid the following common mistakes when you use automatic indicators.

Using Indicator Values Before the Indicator Is Warmed Up

Indicators can provide inaccurate output values before they are warmed up. To avoid issues, always check the `is_ready` flag before you use indicator output values.

```py
# is_ready signals whether the indicator values are accurate and ready to use.
if self.indicator.is_ready:
    value = self.indicator.current.value
```

Manually Registering an Automatic Indicator for Updates

If you create an automatic indicator and then register it for automatics updates or call the `update` method, the indicator receives multiple input values during each update cycle. To avoid issues, create automatic indicators but don't register them for automatic updates or call the `update` method.

```
# Automatic indicator short-cuts create the indicator and register it for data updates.
self.indicator = self.sma(self._symbol, 5)

# Don't register the automatic indicators for data. They automatically get data.
# self.register_indicator(self._symbol, self.indicator, Resolution.DAILY)
```

Examples

The following examples demonstrate some common practices for using automatic indicators.

Example 1: Trade With Automatic Indicators in a Dynamic Universe

The following algorithm selects a universe of the 20 most liquid US Equities each day. At the start of each trading session, it then forms an equal-weighted portfolio of the stocks that have price > EMA > SMA , which indicates an upward accelerating trend. As assets enter the universe, it creates automatic indicators for them. As assets leave the universe, it deregisters the indicators.

```
class AutomaticIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add a universe of the 20 most liquid stocks to trade trends more efficiently.
        self._universe = self.add_universe(self.universe.top(20))

        # Set a Scheduled Event to rebalance at the start of each day with the daily indicator signals.
        self.schedule.on(self.date_rules.every_day(), self.time_rules.at(9, 31), self._rebalance)

    def _rebalance(self) -> None:
        # Buy the stocks where price is above EMA and EMA is above SMA, meaning its trend is upward and accelerating.
        securities = [self.securities[symbol] for symbol in self._universe.selected]
        symbols_to_buy = [
            security.symbol for security in securities
            if security.price > security.ema.current.value > security.sma.current.value
        ]

        # Equally invest in the selected stocks to evenly distribute the capital risk.
        count = len(symbols_to_buy)
        if count:
            targets = [PortfolioTarget(symbol, 1 / count) for symbol in symbols_to_buy]
            # Liquidate the positions that are no longer popular or in an uptrend.
            self.set_holdings(targets, liquidate_existing_holdings=True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            symbol = security.symbol
            # Create EMA & SMA indicators to generate trade signals. Use duck typing to save the indicator instances.
            security.ema = self.ema(symbol, 20, Resolution.DAILY)
            security.sma = self.sma(symbol, 20, Resolution.DAILY)

            # Warm up the indicators so they are ready to use immediately.
            self.warm_up_indicator(symbol, security.ema, Resolution.DAILY)
            self.warm_up_indicator(symbol, security.sma, Resolution.DAILY)

        for security in changes.removed_securities:
            # Stop updating the indicators of assets that leave the universe to release computation resources.
            self.deregister_indicator(security.ema)
            self.deregister_indicator(security.sma)
```

Indicators

Plotting Indicators

Introduction

LEAN provides helper methods to make it simple to create indicator plots.

Plot Update Events

To plot all of the values of some indicators, in the `initialize` method, call the `plot_indicator` method. The method plots each indicator value as the indicator updates. The method accepts up to four indicators.

```
PY
symbol = self.add_equity("SPY")
sma_short = self.sma(symbol, 10)
sma_long = self.sma(symbol, 20)
self.plot_indicator("<chartName>", sma_short, sma_long)
```

Plot Current Values

To plot the current value of indicators, call the `plot` method. The method accepts up to four indicators.

```
PY
# In Initialize
symbol = self.add_equity("SPY")
sma_short = self.sma(symbol, 10)
sma_long = self.sma(symbol, 20)

# In OnData
self.plot("<chartName>", sma_short, sma_long)
```

View Charts

The following table describes where you can access your charts, depending on how to deploy your algorithms:

Location	Algorithm Lab Algorithms	CLI Cloud Algorithms	CLI Local Algorithms
Backtest results page	✓	✓	
Live results page	✓	✓	
/backtests/read endpoint	✓	✓	
/live/read endpoint	✓	✓	
ReadBacktest method	✓	✓	
ReadLiveAlgorithm method	✓	✓	
Local JSON file in your <projectName> / backtests / <timestamp> or <projectName> / live / <timestamp> directory		✓	✓

Chart Limits

Not all indicators share the same base type(T), so you may not be able to plot them together since some indicators require points while others require `TradeBars` .

```
# Plot indicators that extend the "Indicator" type
self.plot_indicators("All Indicator Values", sma, rsi)
self.plot("Current Indicator Values", sma, rsi);

# Plot indicators that extend the "TradeBarIndicator" type
self.plot_indicators("All Indicator Values", atr, aroon)
self.plot("Current Indicator Values", atr, aroon);
```

If your indicator plots are complex, call the `plot` method with one indicator and plot its `current.value` property. For more information about plotting, see Charting .

Examples

The following examples demonstrate some common practices for plotting indicators.

Example 1: SMA Ribbon

The following algorithm trades simple moving average (SMA) crossings, which involves five SMA indicators to form an SMA ribbon. To visualize the ribbon, it plots each indicator with a different color.

```
class PlottingIndicatorAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data to update indicators and trade.
        self._spy = self.add_equity("SPY", Resolution.DAILY).symbol

        # Create a chart to plot the ribbon. To better visualize the plot, use different colors per series.
        chart = Chart("Ribbon")
        chart.add_series(Series("Price", SeriesType.CANDLE, "$"))
        chart.add_series(Series("SMA10", SeriesType.LINE, "$", Color.VIOLET))
        chart.add_series(Series("SMA20", SeriesType.LINE, "$", Color.BLUE))
        chart.add_series(Series("SMA30", SeriesType.LINE, "$", Color.GREEN))
        chart.add_series(Series("SMA40", SeriesType.LINE, "$", Color.YELLOW))
        chart.add_series(Series("SMA50", SeriesType.LINE, "$", Color.RED))

        # Create 5 SMA indicators to create the SMA ribbon.
        self._smas = {n: self._create_sma(n) for n in [10, 20, 30, 40, 50]}

    def _create_sma(self, n: int) -> SimpleMovingAverage:
        sma = self.sma(self._spy, n, Resolution.DAILY)
        # Plot the indicator value to create the ribbon when the indicator updates.
        sma.updated += lambda sender, point: self.plot("Ribbon", f"SMA{n}", point.value)
        # Warm up the indicator so it's ready to use immediately.
        self.warm_up_indicator(self._spy, sma)
        return sma

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self._spy)
        if bar:
            # Plot the candlestick when you get the daily bar.
            self.plot("Ribbon", "Price", bar.open, bar.high, bar.low, bar.close)

            # Follow strong upward trend.
            if self._is_uptrend(bar.close, self._smas):
                self.set_holdings(self._spy, 1)
            # Follow strong downward trend.
            elif self._is_downtrend(bar.close, self._smas):
                self.set_holdings(self._spy, -1)
            # Liquidate positions if the trend is less deterministic.
            else:
                self.liquidate()

    def _is_uptrend(self, price: float, smas: List[SimpleMovingAverage]) -> bool:
        # Uptrend occurs when price is above all SMAs and shorter SMAs are above longer SMAs.
        smas_list = [x[1].current.value for x in sorted(smas.items(), key=lambda x: x[0])]
        return all([smas_list[i] < smas_list[i-1] for i in range(1, len(smas_list))]) and price > smas_list[0]

    def _is_downtrend(self, price: float, smas: List[SimpleMovingAverage]) -> bool:
        # Downtrend occurs when price is below all SMAs and shorter SMAs are below longer SMAs.
        smas_list = [x[1].current.value for x in sorted(smas.items(), key=lambda x: x[0])]
        return all([smas_list[i] > smas_list[i-1] for i in range(1, len(smas_list))]) and price < smas_list[0]
```

Indicators

Combining Indicators

Introduction

Indicator extensions let you chain indications together like Lego blocks to create unique combinations. When you chain indicators together, the `current.value` property output of one indicator is the input of the following indicator. To chain indicators together with values other than the `current.value` property, create a custom indicator .

Addition

The `plus` extension sums the `current.value` property of two indicators or sums the `current.value` property of an indicator and a fixed value.

```
PY
# Sum the output of two indicators
min_ = self.min("SPY", 21)
std = self.std("SPY", 21)
min_plus_std = IndicatorExtensions.plus(min_, std)

# Sum the output of an indicator and a fixed value
min_plus_value = IndicatorExtensions.plus(min_, 10)
```

If you pass an indicator to the `plus` extension, you can name the composite indicator.

```
PY
named_indicator = IndicatorExtensions.plus(min_, std, "Buy Zone")
```

Subtraction

The `minus` extension subtracts the `current.value` property of two indicators or subtracts a fixed value from the `current.value` property of an indicator.

```
PY
# Subtract the output of two indicators
sma_short = self.sma("SPY", 14)
sma_long = self.sma("SPY", 21)
sma_difference = IndicatorExtensions.minus(sma_short, sma_long)

# Subtract a fixed value from the output of an indicator
sma_minus_value = IndicatorExtensions.minus(sma_short, 10)
```

If you pass an indicator to the `minus` extension, you can name the composite indicator.

```
PY
named_indicator = IndicatorExtensions.minus(sma_short, sma_long, "SMA Difference")
```

Multiplication

The `times` extension multiplies the `current.value` property of two indicators or multiplies a fixed value and the `current.value` property of an indicator.

```
PY
# Multiply the output of two indicators
ema_short = self.ema("SPY", 14)
ema_long = self.ema("SPY", 21)
ema_product = IndicatorExtensions.times(ema_short, ema_long)

# Multiply the output of an indicator and a fixed value
ema_times_value = IndicatorExtensions.times(ema_short, 1.5)
```

If you pass an indicator to the `times property` extension, you can name the composite indicator.

```
named_indicator = IndicatorExtensions.times(ema_short, ema_long, "EMA Product")
```

Division

The `over` extension divides the `current.value` property of an indicator by the `current.value` property of another indicator or a fixed value.

```
# Divide the output of two indicators
rsi_short = self.rsi("SPY", 14)
rsi_long = self.rsi("SPY", 21)
rsi_division = rsi_short.over(rsi_long)

# Divide the output of an indicator by a fixed value
rsi_half = IndicatorExtensions.over(rsi_short, 2)
```

If you pass an indicator to the `over` extension, you can name the composite indicator.

```
named_indicator = IndicatorExtensions.over(rsi_short, rsi_long, "RSI Division")
```

Weighted Average

The `weighted_by` extension calculates the average `current.value` property of an indicator over a lookback period, weighted by another indicator over the same lookback period. The value of the calculation is

$$\frac{\textbf{x} \cdot \textbf{y}}{ \sum\limits_{i=1}^{n} y_{i} }$$

where $\textbf{x}$ is a vector that contains the historical values of the first indicator,$\textbf{y}$ is a vector that contains the historical values of the second indicator, and n is the lookback period.

```
sma_short = self.sma("SPY", 14)
sma_long = self.sma("SPY", 21)
weighted_sma = IndicatorExtensions.weighted_by(sma_short, sma_long, 3)
```

Custom Chains

The `of` extension feeds an indicator's `current.value` property into the input of another indicator. The first argument of the `IndicatorExtensions.Of` method must be a manual indicator with no automatic updates. If you pass an indicator that has automatic updates as the argument, that first indicator is updated twice. The first update is from the security data and the second update is from the `IndicatorExtensions` class.

```
rsi = self.rsi("SPY", 14)
rsi_sma = IndicatorExtensions.of(SimpleMovingAverage(10), rsi) # 10-period SMA of the 14-period RSI
```

If you pass a manual indicator as the second argument, to update the indicator chain, update the second indicator. If you call the `update` method of the entire indicator chain, it won't update the chain properly.

Simple Moving Average

The `sma` extension calculates the simple moving average of an indicator's `current.value` property.

```
rsi = self.rsi("SPY", 14) # Create a RSI indicator
rsi_sma = IndicatorExtensions.SMA(rsi, 3) # Create an indicator to calculate the 3-period SMA of the RSI indicator
```

Exponential Moving Average

The `ema` extension calculates the exponential moving average of an indicator's `current.value` property.

```
rsi = self.rsi("SPY", 14) # Create a RSI indicator
rsi_ema = IndicatorExtensions.ema(rsi, 3) # Create an indicator to calculate the 3-period EMA of the RSI indicator
```

The `ema` extension can also accept a smoothing parameter that sets the percentage of data from the previous value that's carried into the next value.

```
rsi_ema = IndicatorExtensions.ema(rsi, 3, 0.1) # 10% smoothing factor
```

Maximum

The `max` extension calculates an indicator's maximum `current.value` property over a lookback window.

```
ema = self.ema("SPY", 14) # Create an EMA indicator
ema_max = IndicatorExtensions.max(ema, 10) # Create an indicator to calculate the maximum EMA over the last 10 periods
```

Minimum

The `min` extension calculates an indicator's minimum `current.value` property over a lookback window.

```
ema = self.ema("SPY", 14) # Create an EMA indicator
ema_min = IndicatorExtensions.min(ema, 10) # Create an indicator to calculate the minimum EMA over the last 10 periods
```

Examples

The following examples demonstrate some common practices for combining indicators.

Example 1: Volatility

The following algorithm trades a volatility strategy. By comparing SMA and the current value of the standard deviation of the return, we can estimate the current volatility regime is above or below average to trade the price volatility through strangle.

```
class CombiningIndicatorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request daily SPY data to feed the indicators and generate trade signals.
        # Use Raw data normalization mode to compare the strike price fairly.
        self.spy = self.add_equity("SPY", data_normalization_mode=DataNormalizationMode.RAW).symbol

        # Request option data to trade.
        option = self.add_option(self.spy)
        self._option = option.symbol
        # Filter for 7-day expiring options with $5 apart from the current price to trade volatility using strangle.
        option.set_filter(lambda universe: universe.strangle(7, 5, -5))

        # Create a return indicator to get the daily return of SPY.
        ret = self.roc(self.spy, 1, Resolution.DAILY)
        # Create an SD indicator to measure the 252-day SD of return to measure SPY's volatility.
        self._sd = IndicatorExtensions.of(StandardDeviation(252), ret)
        # Create a 20-day SMA indicator of the SD indicator to compare the average volatility.
        self._sma = IndicatorExtensions.of(SimpleMovingAverage(20), self._sd)

        # Warm up for immediate usage of indicators.
        self.set_warm_up(400, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up:
            return
        chain = slice.option_chains.get(self._option)
        if not self.portfolio.invested and chain:
            # Create a strangle strategy to trade the volatility forecast.
            sorted_strike = sorted([x.strike for x in chain])
            otm_call_strike = sorted_strike[-1]
            otm_put_strike = sorted_strike[0]
            expiry = list(chain)[0].expiry
            strangle = OptionStrategies.strangle(self._option, otm_call_strike, otm_put_strike, expiry)

            # If the current STD is above its SMA, we estimate the volatility will remain high due to volatility clustering.
            # Thus, we long the strangle to earn from the price displacement from the current level.
            if self._sd.current.value > self._sma.current.value:
                self.buy(strangle, 2)
            # If the current STD is below its SMA, we estimate the volatility will remain lower due to volatility clustering.
            # Thus, we short the strangle to earn from the price staying at the current level.
            elif self._sd.current.value < self._sma.current.value:
                self.sell(strangle, 2)

        elif self.portfolio[self.spy].invested:
            # Liquidate any assigned underlying positions.
            self.liquidate(self.spy)
```

Example 2: Displaced SMA Ribbon

The following algorithm trades trends indicated by SMA crossings. We use the `IndicatorExtensions.of` method to create a Delay indicator on SMA indicator.

PY

```
class DisplacedMovingAverageRibbon(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request daily SPY data for feeding indicator and trading.
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol

        # Create 6 15-day SMA indicators, with a 5-day delay between each indicator.
        count = 6
        offset = 5
        period = 15
        self.ribbon = []
        # Define our sma as the base of the ribbon.
        self.sma = SimpleMovingAverage(period)

        for x in range(count):
            # Define our offset to the zero SMA. These various offsets will create our 'displaced' ribbon.
            delay = Delay(offset*(x+1))
            # Using Delay indicator to create displaced SMA indicators.
            delayed_sma = IndicatorExtensions.of(delay, self.sma)
            # Register our new 'delayed_sma' for automatic updates on a daily resolution.
            self.register_indicator(self.spy, delayed_sma, Resolution.DAILY)
            self.ribbon.append(delayed_sma)

        # Plot indicators each time they update using the PlotIndicator function.
        for i in self.ribbon:
            self.plot_indicator("Ribbon", i)

    def on_data(self, data: Slice) -> None:
        # Trade only on updated data with ready-to-use indicators.
        if data[self.spy] is None: return
        if not all(x.is_ready for x in self.ribbon): return
        self.plot("Ribbon", "Price", data[self.spy].price)

        values = [x.current.value for x in self.ribbon]
        holding = self.portfolio[self.spy]
        # Buy SPY if the trend is upward.
        if (holding.quantity <= 0 and self.is_ascending(values)):
            self.set_holdings(self.spy, 1.0)
        # Liquidate if the trend is downwards.
        elif (holding.quantity > 0 and self.is_descending(values)):
            self.liquidate(self.spy)

    # Returns true if the SMA values are in ascending order, indicating an upward trend
    def is_ascending(self, values: List[float]) -> None:
        last = None
        for val in values:
            if last is None:
                last = val
                continue
            if last < val:
                return False
            last = val
        return True

    # Returns true if the SMA values are in descending order, indicating a downward trend
    def is_descending(self, values: List[float]) -> None:
        last = None
        for val in values:
            if last is None:
                last = val
                continue
            if last > val:
                return False
            last = val
        return True
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
DisplacedMovingAverageRibbon.py Python

Indicators

Custom Indicators

Introduction

LEAN supports over 100 pre-built indicators, but a custom indicator is an indicator you define. It receives input, performs some calculation, and sets its output value. Custom indicators are helpful when you want to achieve any of the following results:

- Use an indicator that LEAN doesn't currently implement
- Combine built-in indicators beyond the logic of Indicator Extensions
- Create your own unique indicator for trading

Define Indicators

Custom indicators must implement the `PythonIndicator` class. The indicator must have an `update` method and `name` , `time` , and `value` attributes. The `update` method must accept an `IndicatorDataPoint` , `QuoteBar` , or `TradeBar` and return a boolean that represents if the indicator is ready. The `time` attribute represents the last time you updated the indicator and the `value` attribute represents the current indicator value. The following definition provides an example of a custom simple moving average indicator.

```py
class CustomSimpleMovingAverage(PythonIndicator):
    def __init__(self, name, period):
        self.name = name
        self.warm_up_period = period
        self.time = datetime.min
        self.value = 0
        self.queue = deque(maxlen=period)

    def update(self, input: BaseData) -> bool:
        self.queue.appendleft(input.value)
        count = len(self.queue)
        self.time = input.time
        self.value = sum(self.queue) / count
        # Set the current attribute if you will update the indicator manually
        self.current = IndicatorDataPoint(input.end_time, self.value)
        # Call on_update to trigger an event (cannot be used in automatic updates)
        self.on_updated(self.current)
        return count == self.queue.maxlen

    # Define is_ready property if you will update the indicator manually
    @property
    def is_ready(self) -> bool:
        return len(self.queue) == self.queue.maxlen
```

The following definition provides an example of a custom money flow index indicator.

```py
class CustomMoneyFlowIndex(PythonIndicator):
    def __init__(self, name, period):
        super().__init__()
        self.name = name
        self.value = 0
        self.previous_typical_price = 0
        self.negative_money_flow = deque(maxlen=period)
        self.positive_money_flow = deque(maxlen=period)

    def update(self, input):
        if not isinstance(input, TradeBar):
            raise TypeError('CustomMoneyFlowIndex.update: input must be a TradeBar')

        typical_price = (input.high + input.low + input.close) / 3
        money_flow = typical_price * input.volume

        # We need to avoid double rounding errors
        if abs(self.previous_typical_price / typical_price - 1) < 1e-10:
            self.previous_typical_price = typical_price

        self.negative_money_flow.appendleft(money_flow if typical_price < self.previous_typical_price else 0)
        self.positive_money_flow.appendleft(money_flow if typical_price > self.previous_typical_price else 0)
        self.previous_typical_price = typical_price

        positive_money_flow_sum = sum(self.positive_money_flow)
        total_money_flow = positive_money_flow_sum + sum(self.negative_money_flow)

        self.value = 100
        if total_money_flow != 0:
            self.value *= positive_money_flow_sum / total_money_flow

        # Set the current attribute if you will update the indicator manually
        self.current = IndicatorDataPoint(input.end_time, self.value)

        # Call on_update to trigger an event (cannot be used in automatic updates)
        self.on_updated(self.current)

        return len(self.positive_money_flow) == self.positive_money_flow.maxlen

    # Define is_ready property if you will update the indicator manually
    @property
    def is_ready(self) -> bool:
        len(self.positive_money_flow) == self.positive_money_flow.maxlen
```

Create Indicators

You must define a custom indicator before you can create an instance of it.

To create a custom indicator, call the indicator constructor.

```py
self.custom_sma = CustomSimpleMovingAverage("My SMA", 10)
```

Updates

The process to update custom indicators is the same process you use to update manual indicators. If you update your custom indicator manually , you must set the `current` attribute, define the `is_ready` property, and call the `on_updated` method in the `update` method to trigger update events . When you register your custom indicator for automatic updates , these attributes are set automatically.

Warm Up Indicators

The process to warm up custom indicators is the same process you use to warm up manual indicators .

Examples

The following examples demonstrate some common practices for implementing custom indicators.

Example 1: Multiple Symbol Custom Indicator

The following algorithm implements a custom Cointegration price divergence indicator involving 2 symbols: GOOGL & GOOG. It trades the arbitrage between 2 cointegrated stocks on their price convergence after deviating more than 2 standard deviations.

```py
from sklearn.linear_model import LinearRegression
from statsmodels.tsa.stattools import adfuller

class CustomIndicatorsAlgorithm(QCAlgorithm):
```

```
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request 2 classes of Google stock data to feed the indicators to generate trade signals and trade.
        goog1 = self.add_equity("GOOGL").symbol
        goog2 = self.add_equity("GOOG").symbol

        # Create a custom money flow index to generate a trade signal.
        self.cointegration = Cointegration(self, goog1, goog2, 252, self.date_rules.month_start())
        # Add a handler to trade on updates.
        self.cointegration.updated += self.on_updated
        # Register the indicator to update automatically with daily data.
        self.register_indicator(goog1, self.cointegration, Resolution.DAILY)
        self.register_indicator(goog2, self.cointegration, Resolution.DAILY)

        # Warm up for immediate usage of indicators.
        self.warm_up_indicator(goog1, self.cointegration, Resolution.DAILY)
        self.warm_up_indicator(goog2, self.cointegration, Resolution.DAILY)

    def on_updated(self, sender: object, point: IndicatorDataPoint) -> None:
        if sender.is_ready:
            holding = self.portfolio[self.cointegration.symbol1]

            # If the residual is lower than -2x SD, it means class A price is much higher than what it should be compared to class C.
            # We sell class A and buy class C to bet on their price convergence.
            if point.value < -2 and not holding.is_short:
                self.set_holdings(self.cointegration.symbol1, -0.5)
                self.set_holdings(self.cointegration.symbol2, 0.5 * self.cointegration.ratio)
            # If the residual is higher than the threshold, it means class A price is much lower than what it should be compared to class
C.
            # We buy class A and sell class C to bet on their price convergence.
            elif point.value > 2 and not holding.is_long:
                self.set_holdings(self.cointegration.symbol1, 0.5)
                self.set_holdings(self.cointegration.symbol2, -0.5 * self.cointegration.ratio)
            # Close positions of the price are converged.
            elif (holding.is_short and point.value > 0) or (holding.is_long and point.value < 0):
                self.liquidate()

class Cointegration(PythonIndicator):
    # The standard deviation of the residuals is such that the returned indicator value is relative to the SD.
    _residual_sd = 1e10
    # Store the coefficient and intercept of the cointegrated series for calculating the spread of a new data point.
    _coefficients = [0, 0]
    _windows = {}

    @property
    def ratio(self) -> float:
        return self._coefficients[0]

    @property
    def is_ready(self) -> bool:
        return all(window.is_ready for window in self._windows.values()) and all(x != 0 for x in self._coefficients)

    def __init__(self, algorithm: QCAlgorithm, symbol1: Symbol, symbol2: Symbol, period: int, recalibrating_date_rule: IDateRule) ->
None:
        super().__init__("Cointegration")
        self.symbol1 = symbol1
        self.symbol2 = symbol2
        self.value = 0

        # Use rolling windows to save the price data for cointegration analysis.
        self._windows[symbol1] = RollingWindow(period)
        self._windows[symbol2] = RollingWindow(period)

        # Adjust the cointegration factor between the 2 classes' price series.
        algorithm.schedule.on(
            recalibrating_date_rule,
            algorithm.time_rules.at(23, 59),
            self.calculate_cointegration
        )

        self.warm_up_period = period

    def update(self, input: BaseData) -> bool:
        # Update the rolling windows for cointegration analysis.
        window = self._windows.get(input.symbol)
        if not window:
            raise Exception(f"{input.Symbol} is not part of the Cointegration relation.")
        window.add(IndicatorDataPoint(input.symbol, input.end_time, input.value))

        if not all(x == 0 for x in self._coefficients):
            # Calculate the updated cointegrated series spread only if all symbol data points are updated.
            if all(window[0].end_time == input.end_time for window in self._windows.values()):
                self.value = (self._coefficients[0] * self._windows[self.symbol2][0].value + self._coefficients[1] -
self._windows[self.symbol1][0].value) / self._residual_sd

        return self.is_ready

    def calculate_cointegration(self) -> None:
        # Lag direction is unimportant; it is just a sign flip in the linear regression, so we don't need to flip the window order.
        y = np.array([x.value for x in self._windows[self.symbol1]]).reshape(-1, 1)
        x = np.array([x.value for x in self._windows[self.symbol2]]).reshape(-1, 1)
```

```
        # Perform Linear Regression on both price series to investigate their relationship.
        lr = LinearRegression().fit(x, y)
        slope = float(lr.coef_[0])
        intercept = float(lr.intercept_)

        # Calculate the residuals series to check if it is stationary, meaning if the 2 price series move together.
        residuals = y - (intercept + slope * x)

        # Check if the residuals are stationary using the augmented Dickey-Fuller test.
        # This means no unit root exists for the difference series, and the residuals are stationary.
        critical = -1.941 + -0.2686 / self.warm_up_period + -3.365 / self.warm_up_period**2 + 31.223 / self.warm_up_period**3
        adf_reject = adfuller(residuals)[0] <= -3.45
        if adf_reject:
            # If cointegrated, update the positional sizing ratio and the spread threshold of the trade trigger.
            self._coefficients = [slope, intercept]
            self._residual_sd = float(np.std(residuals))
        else:
            self._coefficients = [0, 0]
            self._residual_sd = 100000000          # An arbitrarily large number that the class A price will never reach.
```

Example 2: Custom Money Flow Index

The following algorithm implements a custom Money Flow Index indicator, and updates it manually. We estimate the supply-demand balance of SPY and trade using the average money flow direction.

```py
from collections import deque

class CustomIndicatorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request daily SPY data to feed the indicators to generate trade signals and trade.
        self.spy = self.add_equity("SPY").symbol

        # Create a custom money flow index to generate a trade signal.
        self.custom_mfi = CustomMoneyFlowIndex(20)

        # Warm up for immediate usage of indicators.
        self.set_warm_up(20, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if not bar:
            return
        # Update the custom MFI with the updated trade bar to obtain the updated trade signal.
        self.custom_mfi.update(bar)
        # Don't trade until warm-up is done.
        if self.is_warming_up:
            return
        # Buy if the positive money flow is above negative, indicating demand is greater than supply, driving up the price.
        if self.custom_mfi.current.value > 50:
            self.set_holdings(self.spy, 1)
        # Sell if the positive money flow is below negative, indicating demand is less than supply, driving down the price.
        else:
            self.set_holdings(self.spy, -1)

class CustomMoneyFlowIndex(PythonIndicator):
    def __init__(self, period: int) -> None:
        super().__init__()
        self.value = 0
        self.previous_typical_price = 0
        self.negative_money_flow = deque(maxlen=period)
        self.positive_money_flow = deque(maxlen=period)

    def update(self, input: BaseData) -> bool:
        if not isinstance(input, TradeBar):
            raise TypeError('CustomMoneyFlowIndex.update: input must be a TradeBar')

        # Estimate the money flow by averaging the price multiplied by volume.
        typical_price = (input.high + input.low + input.close) / 3
        money_flow = typical_price * input.volume

        # We need to avoid double-rounding errors.
        if abs(self.previous_typical_price / typical_price - 1) < 1e-10:
            self.previous_typical_price = typical_price

        # Add the period money flow to calculate the aggregated money flow.
        self.negative_money_flow.appendleft(money_flow if typical_price < self.previous_typical_price else 0)
        self.positive_money_flow.appendleft(money_flow if typical_price > self.previous_typical_price else 0)
        self.previous_typical_price = typical_price

        positive_money_flow_sum = sum(self.positive_money_flow)
        total_money_flow = positive_money_flow_sum + sum(self.negative_money_flow)

        # Set the value to be the positive money flow ratio.
        self.value = 100
        if total_money_flow != 0:
            self.value *= positive_money_flow_sum / total_money_flow

        # Set the current attribute if you will update the indicator manually
        self.current = IndicatorDataPoint(input.end_time, self.value)

        # Call on_update to trigger an event
        self.on_updated(self.current)

        # Set the is_ready property to receive the required bars to fill all windows.
        return self.is_ready

    @property
    def is_ready(self) -> bool:
        return len(self.positive_money_flow) == self.positive_money_flow.maxlen
```

This version of the algorithm uses update events to trade when the custom indicator is updated. Call the `on_updated` method in the `update` method to trigger event.

```
from collections import deque

class CustomIndicatorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Request daily SPY data to feed the indicators to generate trade signals and trade.
        self.spy = self.add_equity("SPY").symbol

        # Create a custom money flow index to generate a trade signal.
        self.custom_mfi = CustomMoneyFlowIndex(20)
        self.custom_mfi.updated += self._on_update

        # Warm up for immediate usage of indicators.
        self.set_warm_up(20, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if not bar:
            return
        # Update the custom MFI with the updated trade bar to obtain the updated trade signal.
        self.custom_mfi.update(bar)

    def _on_update(self, sender, current):
        # Don't trade until warm-up is done.
        if self.is_warming_up:
            return
        # Buy if the positive money flow is above negative, indicating demand is greater than supply, driving up the price.
        if current.value > 50:
            self.set_holdings(self.spy, 1)
        # Sell if the positive money flow is below negative, indicating demand is less than supply, driving down the price.
        else:
            self.set_holdings(self.spy, -1)

class CustomMoneyFlowIndex(PythonIndicator):
    def __init__(self, period: int) -> None:
        super().__init__()
        self.value = 0
        self.previous_typical_price = 0
        self.negative_money_flow = deque(maxlen=period)
        self.positive_money_flow = deque(maxlen=period)

    def update(self, input: BaseData) -> bool:
        if not isinstance(input, TradeBar):
            raise TypeError('CustomMoneyFlowIndex.update: input must be a TradeBar')

        # Estimate the money flow by averaging the price multiplied by volume.
        typical_price = (input.high + input.low + input.close) / 3
        money_flow = typical_price * input.volume

        # We need to avoid double-rounding errors.
        if abs(self.previous_typical_price / typical_price - 1) < 1e-10:
            self.previous_typical_price = typical_price

        # Add the period money flow to calculate the aggregated money flow.
        self.negative_money_flow.appendleft(money_flow if typical_price < self.previous_typical_price else 0)
        self.positive_money_flow.appendleft(money_flow if typical_price > self.previous_typical_price else 0)
        self.previous_typical_price = typical_price

        positive_money_flow_sum = sum(self.positive_money_flow)
        total_money_flow = positive_money_flow_sum + sum(self.negative_money_flow)

        # Set the value to be the positive money flow ratio.
        self.value = 100
        if total_money_flow != 0:
            self.value *= positive_money_flow_sum / total_money_flow

        # Set the current attribute if you will update the indicator manually
        self.current = IndicatorDataPoint(input.end_time, self.value)

        # Call on_update to trigger an event
        self.on_updated(self.current)

        # Set the is_ready property to receive the required bars to fill all windows.
        return self.is_ready

    @property
    def is_ready(self) -> bool:
        return len(self.positive_money_flow) == self.positive_money_flow.maxlen
```

Example 3: Custom Annualized Volatility Indicator

The following algorithm implements a custom annualized log-return volatility indicator using a rolling window of close prices. It screens a dynamic Equity universe for overbought stocks whose annualized volatility exceeds 100% and enters short positions via limit orders using an equal-weight allocation, exiting when the Connors Relative Strength Index reverts below the exit threshold.

PY

```
class ConnorsCrash(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self._rsi_period = 3
        self._streak_period = 2
        self._pct_rank_period = 100
        self._vola_period = 100
        self._crsi_entry = 90
        self._crsi_exit = 30
        self._max_shorts = 40
        self._stop_trading_days_before_end = 1
        self.settings.automatic_indicator_warm_up = True
        self.settings.seed_initial_prices = True
        self.universe_settings.resolution = Resolution.DAILY
        # Select liquid, tradeable-priced equities for the universe.
        self._universe = self.add_universe(
            lambda fundamentals: [f.symbol for f in fundamentals if f.price > 5 and f.dollar_volume > 1e6]
        )
        self.schedule.on(
            self.date_rules.every_day('SPY'),
            self.time_rules.at(8, 0),
            self._rebalance
        )

    def on_securities_changed(self, changes):
        for security in changes.added_securities:
            # Attach ConnorsRSI indicator to each security.
            security.connors = self.crsi(security, self._rsi_period, self._streak_period, self._pct_rank_period)
            # Initialize and warm up the custom volatility indicator.
            security.volatility = CustomVolatility(self._vola_period)
            for bar in self.history[TradeBar](security, self._vola_period + 1):
                security.volatility.update(bar)
            self.register_indicator(security, security.volatility)
        for security in changes.removed_securities:
            self.deregister_indicator(security.connors)
            self.deregister_indicator(security.volatility)
            self.liquidate(security)

    def _rebalance(self):
        if not self._universe.selected:
            return
        securities = [self.securities[symbol] for symbol in self._universe.selected]
        securities = [s for s in securities if s.connors.is_ready and s.volatility.is_ready]
        filter_vola = [s for s in securities if s.volatility.value > 100]
        # Find currently invested short positions.
        short_positions = [s for s in securities if s.holdings.is_short]
        # Liquidate short positions when ConnorsRSI falls below exit threshold.
        for security in short_positions:
            if security.connors.current.value < self._crsi_exit:
                self.liquidate(security)
        # Exclude symbols with pending open orders.
        pending_symbols = {t.symbol for t in self.transactions.get_open_order_tickets()}
        # Find short entry candidates that are not currently invested (high volatility and high CRSI).
        short_candidates = sorted([
            s for s in filter_vola
            if (s.connors.current.value > self._crsi_entry and
            not s.holdings.invested and
            s.symbol not in pending_symbols)
        ], key=lambda s: (s.connors.current.value, s.volatility.value))
        # Set union: liquidated positions are only counted once.
        short_position_symbols = {s.symbol for s in short_positions}
        occupied = short_position_symbols | pending_symbols
        available_slots = self._max_shorts - len(occupied)
        if available_slots <= 0 or not short_candidates:
            return
        n_orders = min(len(short_candidates), available_slots)
        target_weight = -1 / self._max_shorts
        for security in short_candidates[-n_orders:]:
            quantity = int(self.calculate_order_quantity(security, target_weight))
            if quantity:
                self.limit_order(security, quantity, round(1.03 * security.price, 2))


class CustomVolatility(PythonIndicator):

    def __init__(self, period):
        super().__init__()
        self.value = 0
        self._window = RollingWindow[float](period)

    def update(self, input_: BaseData):
        # Annualized log-return volatility.
        price = input_.value
        if price <= 0:
            return
        self._window.add(price)
        if self._window.is_ready:
            prices = np.array(list(self._window)[::-1])
            log_diffs = np.diff(np.log(prices))
            self.value = np.std(log_diffs) * math.sqrt(252) * 100.0
```

```
                self.value = np.std(log_diffs) * math.sqrt(252) * 100.0
            return self.is_ready

        @property
        def is_ready(self) -> bool:
            return self._window.is_ready
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
CustomIndicatorAlgorithm.py Python

Indicators

Indicator Universes

Introduction

An indicator universe uses technical indicators to determine the constituents of the universe. Imagine a universe that only contains assets above their 10-day simple moving average. You can incorporate indicators into any of the types of universes in the Universes chapter . To create an indicator universe, define a helper class that contains the indicators and then define a universe that updates the indicators and selects assets.

Define SymbolData Objects

To make it easy to create and update indicators for each security in the universe, move the indicator logic into a class. In the universe definition, you can create an instance of this class for each security in the universe. The indicators you create in this class should be manual indicators so you can ensure they only update during universe selection.

```py
class SymbolData(object):
    def __init__(self, symbol):
        self._symbol = symbol
        self.tolerance = 1.01
        self.fast = ExponentialMovingAverage(100)
        self.slow = ExponentialMovingAverage(300)
        self.is_uptrend = False
        self.scale = 0

    def update(self, time, value):
        if self.fast.update(time, value) and self.slow.update(time, value):
            fast = self.fast.current.value
            slow = self.slow.current.value
            self.is_uptrend = fast > slow * self.tolerance

        if self.is_uptrend:
            self.scale = (fast - slow) / ((fast + slow) / 2.0)
```

You need to use a `SymbolData` class instead of assigning the indicators to the `Fundamental` object because you can't create custom attributes on `Fundamental` objects like you can with `Security` objects.

Define the Universe

You need to define SymbolData objects before you define the universe that selects securities.

When your universe function receives an object that contains all the possible securities, create a `SymbolData` object for each new security and update the remaining `SymbolData` objects with their daily price or some other data point. For example, the following universe definition selects US Equities that have the greatest difference between two moving averages.

```
class EmaCrossUniverseSelectionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        '''Initialise the data and resolution required, as well as the cash and start-end dates for your algorithm. All algorithms must
initialized.'''

        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.universe_settings.asynchronous = True
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.leverage = 2

        self.count = 10
        self.averages = { }

        # this add universe method accepts two parameters:
        # - fundamental selection function: accepts an IEnumerable<Fundamental> and returns an IEnumerable<Symbol>
        self.add_universe(self.fundamental_selection_function)


    # sort the data by daily dollar volume and take the top 'NumberOfSymbols'
    def fundamental_selection_function(self, fundamental: List[Fundamental]) -> List[Symbol]:

        # We are going to use a dictionary to refer the object that will keep the moving averages
        for f in fundamental:
            if f.symbol not in self.averages:
                self.averages[f.symbol] = SymbolData(f.symbol)

            # Updates the SymbolData object with current EOD price
            avg = self.averages[f.symbol]
            avg.update(f.end_time, f.adjusted_price)

        # Filter the values of the dict: we only want up-trending securities
        values = list(filter(lambda x: x.is_uptrend, self.averages.values()))

        # Sorts the values of the dict: we want those with greater difference between the moving averages
        values.sort(key=lambda x: x.scale, reverse=True)

        for x in values[:self.count]:
            self.log('symbol: ' + str(x.symbol.value) + '  scale: ' + str(x.scale))

        # we need to return only the symbol objects
        return [ x.symbol for x in values[:self.count] ]
```

Examples

The following examples demonstrate some common practices for using indicator universes.

Example 1: Universe Selection

The following algorithm selects the stocks within 5% of the 1-year maximum price among the top 100 liquid stocks. To do so, we make use of Maximum indicator to do so. Then, we hold the stocks with price > EMA > SMA , which indicates an upward accelerating trend.

```py
class ManualIndicatorAlgorithm(QCAlgorithm):
    maximum_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Select a popularity-based universe with indicators in a Selection function.
        self._universe = self.add_universe(self.selection)

        # Set a scheduled event to rebalance daily on the daily indicator signals.
        self.schedule.on(
            self.date_rules.every_day(),
            self.time_rules.at(9, 31),
            self.rebalance
        )

    def selection(self, fundamentals: list[Fundamental]) -> list[Symbol]:
        selected = []

        # Initially filtered for the top 100 liquid stocks first.
        filtered = sorted(fundamentals, key=lambda f: f.dollar_volume, reverse=True)[:100]

        for f in filtered:
            if f.symbol not in self.maximum_by_symbol:
                self.maximum_by_symbol[f.symbol] = Maximum(252)
                # Warm up the Maximum indicator to its readiness to use immediately.
                history = self.history[TradeBar](f.symbol, 252, Resolution.DAILY)
                for bar in history:
                    self.maximum_by_symbol[f.symbol].update(bar.end_time, bar.close)
            else:
                # Update the indicator with the last known adjusted price daily.
                self.maximum_by_symbol[f.symbol].update(f.end_time, f.adjusted_price)

            # Select to trade if the current price is within 5% of the maximum price of the last year.
            # Close to the maximum price provides evidence of high popularity for the fund to support the trend.
            if f.adjusted_price >= self.maximum_by_symbol[f.symbol].current.value * 0.95:
                selected.append(f.symbol)

        return selected

    def rebalance(self) -> None:
        def to_buy(symbol):
            security = self.securities[symbol]
            # Buy the stocks whose prices are above the EMA and the EMA is above the SMA, meaning their trend is upward accelerating.
            return security.price > security.ema.current.value > security.sma.current.value

        symbols_to_buy = [symbol for symbol in self._universe.selected if to_buy(symbol)]

        # Equally invest in the selected stocks to dissipate the capital risk evenly.
        count = len(symbols_to_buy)
        if count > 0:
            targets = [PortfolioTarget(symbol, 1 / count) for symbol in symbols_to_buy]
            # Liquidate the positions that are not on an upward trend or are not popular anymore.
            self.set_holdings(targets, liquidate_existing_holdings=True)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            symbol = added.symbol
            # Create an EMA and an SMA manual indicator for trade signal generation.
            added.ema = ExponentialMovingAverage(20)
            added.sma = SimpleMovingAverage(20)

            # Warm up the indicators to ensure their readiness to use them immediately.
            self.warm_up_indicator(symbol, added.ema, Resolution.DAILY)
            self.warm_up_indicator(symbol, added.sma, Resolution.DAILY)

            # Subscribe to the indicators to update daily price data for updated trade signal generation.
            self.register_indicator(symbol, added.ema, Resolution.DAILY)
            self.register_indicator(symbol, added.sma, Resolution.DAILY)

        for removed in changes.removed_securities:
            # Cancel data subscription to release computation resources when leaving the universe.
            self.deregister_indicator(removed.ema)
            self.deregister_indicator(removed.sma)
```

Example 2: Reset Indicators on Corporate Actions

The following example shows how to select a universe of US Equities based on their price and SMA indicator. The `SelectionData` class keeps track of the SMA indicator for each stock in the universe dataset. When a split or dividend occurs for a stock, the data in its indicator becomes invalid because it doesn't account for the price adjustments that the split or dividend causes. The `SelectionData` class resets and warms up the indicator with the `SCALED_RAW` data normalization mode , which gives you accurate indicator values to use in your universe selection after each corporate action.

```py
class EquityIndicatorUniverseSelectionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.settings.seed_initial_prices = True
        # Add a universe of US Equities based on an indicator.
        self._selection_data_by_symbol = {}
        self._universe = self.add_universe(self._select_assets)
        # Add a warm-up period to warm up the indicators in the universe selection.
        self.set_warm_up(timedelta(60))

    def _select_assets(self, fundamentals):
        # Update the indicator of all stocks in the universe dataset and
        # get the subset of stocks that have their indicator ready.
        ready_stocks = [
            f for f in fundamentals
            if self._selection_data_by_symbol.setdefault(f.symbol, SelectionData(self, f)).update(f)
        ]
        # As assests leave the Fundamental dataset, delete their SelectionData object.
        for symbol in self._selection_data_by_symbol.keys() - {f.symbol for f in fundamentals}:
            del self._selection_data_by_symbol[symbol]
        # During warm-up, keep the universe empty.
        if self.is_warming_up:
            return []
        # Select a subset of the stocks based on the indicator.
        # Example: 10 stocks furthest above their SMA.
        factor_by_symbol = {
            f.symbol: f.price / self._selection_data_by_symbol[f.symbol].indicator.current.value
            for f in ready_stocks
        }
        return sorted(
            {k: v for k, v in factor_by_symbol.items() if v > 0},
            key=lambda symbol: factor_by_symbol[symbol]
        )[-100:]


class SelectionData:

    def __init__(self, algorithm, f):
        self._algorithm = algorithm
        self._price_scale_factor = f.price_scale_factor
        self.indicator = SimpleMovingAverage(21)

    def update(self, f):
        # If there hasn't been a split or dividend since the last trading
        # day, just update the indicator like normal.
        if f.price_scale_factor == self._price_scale_factor:
            return self.indicator.update(f.end_time, f.price)
        # Otherwise, reset the indicator and warm it up with the new
        # adjusted history.
        self._price_scale_factor = f.price_scale_factor
        self.indicator.reset()
        history = self._algorithm.history[TradeBar](
            f.symbol,
            self.indicator.warm_up_period,
            Resolution.DAILY,
            data_normalization_mode=DataNormalizationMode.SCALED_RAW
        )
        for bar in history:
            self.indicator.update(bar)
        return self.indicator.is_ready
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
EmaCrossUniverseSelectionAlgorithm.py Python

Indicators

Rolling Window

Introduction

A `RollingWindow` is an array of a fixed-size that holds trailing data. It's more efficient to use `RollingWindow` objects to hold periods of data than to make multiple historical data requests . With a `RollingWindow` , you just update the latest data point while a `history` call fetches all of the data over the period you request. `RollingWindow` objects operate on a first-in, first-out process to allow for reverse list access semantics with positive and negative indexing. Index 0 refers to the most recent item in the window and the largest index refers to the last item in the window. Index -1 refers to the latest item as well.

Create Rolling Windows

To create rolling windows, instantiate the `RollingWindow` object with a maximum size.

```
PY
self._close_price_window = RollingWindow(4)
self._trade_bar_window = RollingWindow(2)
self._quote_bar_window = RollingWindow(2)
self._tuple_window = RollingWindow(2)
```

You can define the rolling window type with `RollingWindow[T]` , where `T` is the native or C# type of elements in the window.

```
PY
self._close_price_window = RollingWindow[float](4)
self._trade_bar_window = RollingWindow[TradeBar](2)
self._quote_bar_window = RollingWindow[QuoteBar](2)
```

The typeless version, `RollingWindow(int)` , allows for Python types. For example, `tuple` .

Add Data

To add data to a `RollingWindow` , call the `add` method.

```
PY
self._close_price_window.add(data["SPY"].close)
self._trade_bar_window.add(data["SPY"])
self._quote_bar_window.add(data["EURUSD"])
self._tuple_window.add((1, 2))
```

To update the data at a specific index, set the value for that index. If the index doesn't currently exist, it increases the size and fills the empty indices with a default value (zero or `None`).

```
PY
self._close_price_window[0] = data["SPY"].close
self._trade_bar_window[0] = data["SPY"]
self._quote_bar_window[0] = data["EURUSD"]
self._tuple_window[0] = (1, 2)
```

Warm Up

To warm up a `RollingWindow` , make a history request and then iterate through the result to add the data to the `RollingWindow` .

```
class WarmUpRollingWindowAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        spy = self.add_equity("SPY", Resolution.DAILY).symbol
        history_trade_bar = self.history[TradeBar](spy, 10, Resolution.DAILY)
        history_quote_bar = self.history[QuoteBar](spy, 10, Resolution.MINUTE)

        # Warm up the close price and trade bar rolling windows with the previous 10-day trade bar data
        close_price_window = RollingWindow(10)
        trade_bar_window = RollingWindow(10)
        for trade_bar in history_trade_bar:
            close_price_window.add(trade_bar.close)
            trade_bar_window.add(trade_bar)

        # Warm up the quote bar rolling window with the previous 10-minute quote bar data
        quote_bar_window = RollingWindow(10)
        for quote_bar in history_quote_bar:
            quote_bar_window.add(quote_bar)
```

Check Readiness

To check if a `RollingWindow` is full, use its `is_ready` flag.

```
if not self._close_price_window.is_ready:
    return
```

Adjust Size

To adjust the `RollingWindow` size, set the `size` property.

```
self._close_price_window.size = 3
self._trade_bar_window.size = 3
self._quote_bar_window.size = 3
self._tuple_window.size = 3
```

When you decrease the size, it removes the oldest values that no longer fit in the `RollingWindow` . When you explicitly increase the `size` member, it doesn't automatically add any new elements to the `RollingWindow` . However, if you set the value of an index in the `RollingWindow` and the index doesn't currently exist, it fills the empty indices with a default value (zero or `None`). For example, the following code increases the `size` to 10, sets the 10th element to 3, and sets the 4th-9th elements to the default value:

```
self._close_price_window[9] = 3
```

Access Data

`RollingWindow` objects operate on a first-in, first-out process to allow for reverse list access semantics with positive and negative indexing. Index 0 refers to the most recent item in the window and the largest index refers to the last item in the window. Index -1 refers to the latest item as well.

```
current_close = self._close_price_window[0]
previous_close = self._close_price_window[1]
oldest_close = self._close_price_window[self._close_price_window.count-1]
oldest_close = self._close_price_window[-1]   # Negative indexing
```

To get the item that was most recently removed from the `RollingWindow` , use the `most_recently_removed` property.

```
removed_close = self._close_price_window.most_recently_removed
```

Combine with Indicators

The `window` property of the indicators is a built-in `RollingWindow` that stores historical values. It holds a collection of `IndicatorDataPoint` objects, enabling quick access to the most recent historical indicator values for analysis, calculations, or comparisons in trading and financial strategies. Its

default size 2, but you can adjust its size. To warm up the indicator and its `RollingWindow`, call the `indicator_history` method with the window size as an argument.

```
class IndicatorRollingWindowAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        spy = self.add_equity("SPY").symbol
        qqq = self.add_equity("QQQ").symbol

        # Manual SMA indicator example.
        self._sma = SimpleMovingAverage(20)
        # Automatic Beta indicator example.
        self._beta = self.b(spy, qqq, 20)

        # Adjust the window size to 5 to access the previous 5 indicator data points.
        self._sma.window.size = 5
        self._beta.window.size = 5

        # Warm up the indicator and window for immediate usage.
        self.indicator_history(self._sma, spy, self._sma.window.size, Resolution.DAILY)
        # For indicators using 2+ symbols, use an iterable symbol list.
        self.indicator_history(self._beta, [spy, qqq], self._beta.window.size, Resolution.DAILY)
```

Combine with Consolidators

To store a history of consolidated bars, in the consolidation handler, add the consolidated bar to the `RollingWindow`.

```
self._consolidator.data_consolidated += self._on_data_consolidated

# Define consolidator handler function as a class method
def _on_data_consolidated(self, sender, consolidated_bar):
    self._trade_bar_window.add(consolidated_bar)
```

Market Session

The `Session` property of a `Security` object contains the open, high, low, close, volume, and open interest of trading sessions. The `Session` object is a `RollingWindow` object with size of zero by default. To enable market session tracking, set the size of the `Session` object to 2, so the previous session data is at index 1.

```
session = self.securities[self._symbol].session
session.size = 2
today_open = session.open
today_high = session.high
today_low = session.low
today_cumulative_volume = session.volume
if session.is_ready:
    yesterday_close = session[1].close
```

Increase the size of the `Session` object to store more session data. You can use cached data in the `Session` property instead of requesting historical daily data.

```
equity = self.add_equity("SPY")
# Increase the session lookback to 252 from default (0)
equity.session.size = 252
# Get all closing prices in the session history
closes = [x.close for x in session][::-1]
```

Cast to Other Types

You can cast a `RollingWindow` to a list or a DataFrame. If you cast it to a list, reverse the list so the most recent element is at the last index of the list. This is the order the elements would be in if you added the elements to the list with the `Add` method. To cast a `RollingWindow` to a DataFrame, the `RollingWindow` must contain `Slice`, `Tick`, `QuoteBar`, or `TradeBar` objects. If the `RollingWindow` contains ticks, the ticks must have unique timestamps.

```
closes = list(self._close_price_window)[::-1]  # Make sure to reverse the list.

tick_df = self.pandas_converter.get_data_frame[Tick](list(self._tick_window)[::-1])
trade_bar_df = self.pandas_converter.get_data_frame[TradeBar](list(self._trade_bar_window)[::-1])
quote_bar_df = self.pandas_converter.get_data_frame[QuoteBar](list(self._quote_bar_window)[::-1])
```

Delete Data

To remove all of the elements from a `RollingWindow` , call the `reset` method.

```
self._close_price_window.reset()
```

Examples

The following examples demonstrate some common practices for rolling windows.

Example 1: Price Actions

The following algorithm saves the trailing 3 `TradeBar` objects into a `RollingWindow` . When it identifies a volume contraction breakout price action pattern on the SPY, it buys to ride on the capital inflow. To exit positions, it places a 2% take profit and 1% stop loss order in the `on_order_event` method.

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up a rolling window to hold the last 3 trade bars for price action detection as the trade signal.
        self.windows = RollingWindow(3)
        # Warm up the rolling window.
        history = self.history[TradeBar](self.spy, 3, Resolution.MINUTE)
        for bar in history:
            self.windows.add(bar)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Trade the price action if the previous bars fulfill a contraction breakout.
            if self.contraction_action and self.breakout(bar.close):
                self.set_holdings(self.spy, 0.5)

            # Add the current bar to the window.
            self.windows.add(bar)

    def contraction_action(self) -> None:
        # We trade contraction type price action, where the buying preesure is increasing.
        # 1. The last 3 bars are green.
        # 2. The price is increasing in trend.
        # 3. The trading volume is increasing as well.
        # 4. The range of the bars are decreasing.
        return (
            self.windows[2].close > self.windows[2].open and
            self.windows[1].close > self.windows[1].open and
            self.windows[0].close > self.windows[0].open and
            self.windows[0].close > self.windows[1].close > self.windows[2].close and
            self.windows[0].volume > self.windows[1].volume > self.windows[2].volume and
            self.windows[2].close - self.windows[2].open > self.windows[1].close - self.windows[1].open > self.windows[0].close -
self.windows[0].open
        )

    def breakout(self, current_close: float) -> None:
        # Trade breakout from contraction: the breakout should be much greater than the contracted range of the last bar.
        return current_close - self.windows[0].close > (self.windows[0].close - self.windows[0].open) * 2

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * (0.99 if order_event.fill_quantity > 0 else 1.01)
                self.stop_market_order(self.spy, -self.portfolio[self.spy].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * (1.02 if order_event.fill_quantity > 0 else 0.98)
                self.limit_order(self.spy, -self.portfolio[self.spy].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel open orders if stop loss or take profit order fills.
                self.transactions.cancel_open_orders()
```

Example 2: Bid-Ask Spread

The following algorithm trades the microeconomy of SPY's supply-demand relationship. It buys when the current bid-ask spread is less than average spread over the last 20 QuoteBar objects, indicating demand is approaching supply. When the spread is greater than the average, it shorts SPY. To save the last spread values, it uses a `RollingWindow` .

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1);
        self.set_end_date(2024, 12, 31)

        # Add SPY data for signal generation and trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up a rolling window to hold the last 20 bar's bid-ask spread for trade signal generation.
        self.windows = RollingWindow(20)
        # Warm up for the rolling window with quote data.
        history = self.history[QuoteBar](self.spy, 20, Resolution.MINUTE)
        for bar in history:
            self.windows.add(bar.ask.close - bar.bid.close)

    def on_data(self, slice: Slice) -> None:
        bar = slice.quote_bars.get(self.spy)
        if bar:
            # Update the window with the current bid-ask spread.
            spread = bar.ask.close - bar.bid.close
            self.windows.add(spread)

            # Buy if the current spread is smaller than average, indicating demand is approaching supply. Buy force will drive up price.
            if spread < np.mean(list(self.windows)):
                self.set_holdings(self.spy, -0.5)
            # Short if the current spread is larger than average, indicating supply is gradually overwhelming demand.
            elif spread > np.mean(list(self.windows)):
                self.set_holdings(self.spy, 0.5)
```

Example 3: EMA Acceleration

The following algorithm trades SPY based on the direction and acceleration of its exponential moving average (EMA). It buys when the EMA is increasing and accelerating upward, indicating a strong uptrend. It short sells when the EMA is decreasing and accelerating downward, indicating a strong downtrend. To compare the current EMA value to its previous values, the algorithm utilizes the EMA's internal `RollingWindow` .

```py
class RollingWindowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add SPY data for trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol

        # Set up an automatic EMA indicator for trade signal generation.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Extend the size of the EMA's internal rolling window so we can access its historical values.
        self._ema.window.size = 3
        # Warm-up the indicator and its internal rolling window
        self.indicator_history(self._ema, self.spy, self._ema.window.size)

        # Schedule an event to rebalance SPY position at daily market open.
        self.schedule.on(self.date_rules.every_day(self.spy), self.time_rules.after_market_open(self.spy, 0), self.rebalance)

    def rebalance(self) -> None:
        # Buy if the current EMA is increasing with acceleration, indicating strong uptrend.
        if (self._ema.window[1].value < self._ema.current.value and
            self._ema.window[0].value - self._ema.window[1].value > self._ema.window[1].value - self._ema.window[2].value):
            self.set_holdings(self.spy, 0.5)
        # Short if the current EMA is decreasing with acceleration, indicating strong downtrend.
        elif (self._ema.window[1].value > self._ema.current.value and
            self._ema.window[0].value - self._ema.window[1].value < self._ema.window[1].value - self._ema.window[2].value):
            self.set_holdings(self.spy, -0.5)
        # Liquidate if no strong trend indicated.
        elif self.portfolio.invested:
            self.liquidate(self.spy)
```

Example 4: Market Session as History

Using `Session` instead of history requests or rolling window to cache the last 252 OHLCV bars, the following algorithm buys SPY accordingly when the market opens above the 252-day max of closing prices.

```py
class MarketSessionAsDailyHistory(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        equity = self.add_equity("SPY")

        # Increase the session lookback to 252 from default (0)
        # since our strategy break out the 252-day max of closing prices
        equity.session.size = 252
        # Warm up with daily data for speed
        self.set_warm_up(equity.session.size, Resolution.DAILY)

        self.schedule.on(self.date_rules.every_day("SPY"), self.time_rules.after_market_open("SPY", 1), self.trade_break_out)

    def trade_break_out(self):
        '''Break-out strategy. If the market opens above the 252-day max of closing prices,
        allocate all in SPY. Liquidate if it opens below the mean.
        '''
        session = self.securities["SPY"].session
        # The session is ready when the number of samples is greater than the size (252)
        if self.is_warming_up or not session.is_ready:
            return

        # Create a pandas Series of the closing prices of session
        # to use the max and mean methods
        closes = pd.Series([x.close for x in session][::-1])
        _open, _max, _mean = session.open, closes.max(), closes.mean()

        # Invest all in SPY if the open break out the 252-day max and liquidate below the mean
        if _open > _max:
            self.set_holdings("SPY", 1, tag=f'{_open=:.2f}, {_max=:.2f}')
        if _open < _mean:
            self.liquidate("SPY", tag=f'{_open=:.2f}, {_mean=:.2f}')
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
RollingWindowAlgorithm.py Python CustomVolatilityModelAlgorithm.py Python MultipleSymbolConsolidationAlgorithm.py Python

Object Store

Introduction

The Object Store is a file system that you can use in your algorithms to save, read, and delete data. The Object Store is organization-specific, so you can save or read data from the same Object Store in all of your organization's projects. The Object Store works like a key-value storage system where you can store regular strings, JSON encoded strings, XML encoded strings, and bytes. You can access the data you store in the Object Store from backtests, the Research Environment, and live algorithms.

Get All Stored Data

To get all of the keys and values in the Object Store, iterate through the `object_store` property.

```
for kvp in self.object_store:
    key = kvp.key
    value = kvp.value
```

To iterate through just the keys in the Object Store, iterate through the `keys` property.

```
for key in self.object_store.keys:
    continue
```

Create Sample Data

You need some data to store data in the Object Store.

Follow these steps to create some sample data:

1. Create a `string` .

   ```
   string_sample = "My string"
   ```

2. Create a `Bytes` object.

   ```
   bytes_sample = str.encode("My String")
   ```

Save Data

The Object Store saves objects under a key-value system. If you save objects in backtests, you can access them from the Research Environment. To avoid slowing down your backtests, save data once in the `on_end_of_algorithm` event handler. In live trading, you can save data more frequently like at the end of a `train` method or after universe selection.

If you run algorithms in QuantConnect Cloud, you need storage create permissions to save data in the Object Store.

If you don't have data to store, create some sample data .

You can save `Bytes` and `string` objects in the Object Store. To store data, you need to provide a key. If you provide a key that is already in the Object Store, it will overwrite the data at that location. To avoid overwriting objects from other projects in your organization, prefix the key with your project ID. You can find the project ID in the URL of your browser when you open a project. For example, the ID of the project at quantconnect.com/project/12345 is 12345.

Strings

To save a `string` object, call the `save` or `save_string` method.

```
save_successful = self.object_store.save(f"{self.project_id}/string_key", string_sample)
```

Bytes

To save a `Bytes` object (for example, zipped data), call the `save_bytes` method.

```
save_successful = self.object_store.save_bytes(f"{self.project_id}/bytes_key", bytes_sample)

zipped_data_sample = Compression.zip_bytes(bytes(string_sample, "utf-8"), "data")
zip_save_successful = self.object_store.save_bytes(f"{self.project_id}/bytesKey.zip", zipped_data_sample)
```

Read Data

Read data from the Object Store to import algorithm variables between deployments, import data from the Research Environment, or load trained machine learning models. To read data from the Object Store, you need to provide the key you used to store the object.

You can load `Bytes` and `string` objects from the Object Store.

Before you read data from the Object Store, check if the key exists.

```
if self.object_store.contains_key(key):
    # Read data
```

Strings

To read a `string` object, call the `read` or `read_string` method.

```
string_data = self.object_store.read(f"{self.project_id}/string_key")
```

Bytes

To read a `Bytes` object, call the `read_bytes` method.

```
byte_data = self.object_store.read_bytes(f"{self.project_id}/bytes_key")
```

Delete Data

Delete objects in the Object Store to remove objects that you no longer need. If you run algorithms in QuantConnect Cloud, you need storage delete permissions to delete data from the Object Store.

To delete objects from the Object Store, call the `delete` method. Before you delete data, check if the key exists. If you try to delete an object with a key that doesn't exist in the Object Store, the method raises an exception.

```
if self.object_store.contains_key(key):
    self.object_store.delete(key)
```

To delete all of the content in the Object Store, iterate through all the stored data.

```
for kvp in self.object_store:
    self.object_store.delete(kvp.key)
```

Cache Data

When you write to or read from the Object Store, the algorithm caches the data. The cache speeds up the algorithm execution because if you try to read the Object Store data again with the same key, it returns the cached data instead of downloading the data again. The cache speeds up

execution, but it can cause problems if you are trying to share data between two nodes under the same Object Store key. For example, consider the following scenario:

1. You open project A and save data under the key `123` .
2. You open project B and save new data under the same key `123` .
3. In project A, you read the Object Store data under the key `123` , expecting the data from project B, but you get the original data you saved in step #1 instead.

   You get the data from step 1 instead of step 2 because the cache contains the data from step 1.

To clear the cache, call the `Clear` method.

```
self.object_store.clear()
```

Get File Path

To get the file path for a specific key in the Object Store, call the `get_file_path` method. If the key you pass to the method doesn't already exist in the Object Store, it's added to the Object Store.

```
file_path = self.object_store.get_file_path(key)
```

Storage Quotas

If you run algorithms locally, you can store as much data as your hardware will allow. If you run algorithms in QuantConnect Cloud, you must stay within your storage quota . To find your storage quota programmatically, use the `max_size` attribute for storage size and `max_files` attribute for the maximum number of files.

```
max_size = self.object_store.max_size
max_files = self.object_store.max_files
```

If you need more storage space, edit your storage plan .

Example for DataFrames

Follow these steps to create a DataFrame, save it into the Object Store, and load it from the Object Store:

1. Get some historical data.

   ```
   spy = self.add_equity("SPY").symbol
   df = self.history(self.securities.keys, 360, Resolution.DAILY)
   ```

2. Get the file path for a specific key in the Object Store.

   ```
   file_path = self.object_store.get_file_path("df_to_csv")
   ```

3. Call the to_csv method to save the DataFrame in the Object Store as a CSV file.

   ```
   df.to_csv(file_path)   # File size: 32721 bytes
   ```

4. Call the read_csv method to load the CSV file from the Object Store.

   ```
   reread = pd.read_csv(file_path)
   ```

pandas supports saving and loading DataFrame objects in the following additional formats:

- XML

```
file_path = self.object_store.get_file_path("df_to_xml")
df.to_xml(file_path)   # File size: 87816 bytes
reread = pd.read_xml(file_path)
```

- JSON

```
file_path = self.object_store.get_file_path("df_to_json")
df.to_json(file_path)   # File size: 125250 bytes
reread = pd.read_json(file_path)
```

- Parquet

```
file_path = self.object_store.get_file_path("df_to_parquet")
df.to_parquet(file_path)   # File size: 23996 bytes
reread = pd.read_parquet(file_path)
```

- Pickle

```
file_path = self.object_store.get_file_path("df_to_pickle")
df.to_pickle(file_path)   # File size: 19868 bytes
reread = pd.read_pickle(file_path)
```

Example for Plotting

You can use the Object Store to plot data from your backtests and live algorithm in the Research Environment. The following example demonstrates how to plot a Simple Moving Average indicator that's generated during a backtest.

1. Create a algorithm, add a data subscription, and add a simple moving average indicator.

```
class ObjectStoreChartingAlgorithm(QCAlgorithm):
    def initialize(self):
        self.add_equity("SPY")

        self._content = ''
        self._sma = self.sma("SPY", 22)
```

   The algorithm will save `self._content` to the Object Store.

2. Save the indicator data as `string` in `self._content` .

```
def on_data(self, data: Slice):
    self.plot('SMA', 'Value', self._sma.current.value)
    self._content += f'{self._sma.current.end_time},{self._sma.current.value}\n'
```

3. In the OnEndOfAlgorithm method, save the indicator data to the Object Store.

```
def on_end_of_algorithm(self):
    self.object_store.save('sma_values_python', self._content)
```

4. Open the Research Environment and create a `QuantBook` .

```
qb = QuantBook()
```

5. Read the indicator data from the Object Store.

```
content = qb.object_store.read("sma_values_python")
```

The key you provide must be the same key you used to save the object.

6. Convert the data to a pandas object and create a chart.

```
data = {}
for line in content.split('\n'):
    csv = line.split(',')
    if len(csv) > 1:
        data[csv[0]] = float(csv[1])

series = pd.Series(data, index=data.keys())
series.plot()
```

```
class ObjectStoreChartingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.add_equity("SPY", Resolution.MINUTE)

        self._content = ''
        # Create SMA indicator for referencing.
        self._sma = self.sma("SPY", 22)

    def on_data(self, data: Slice) -> None:
        # Cache the indicator data point to save it.
        self._content += f'{self._sma.current.end_time},{self._sma.current.value}\n'

    def on_end_of_algorithm(self) -> None:
        # Save the indicator values to object store for logging.
        self.object_store.save('sma_values_python', self._content)
```

Example for Logging

You can use the Object Store to log order data from your backtests and live algorithms, then analyze it in the Research Environment. The following example demonstrates how to log orders from an Exponential Moving Average (EMA) cross strategy to a text file in the Object Store.

1. Create an algorithm, add a daily equity subscription, and create two EMA indicators.

```
class ObjectStoreLoggingAlgorithm(QCAlgorithm):
    def initialize(self):
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._ema_short = self.ema(self._symbol, 10)
        self._ema_long = self.ema(self._symbol, 30)
        # Add a header row.
        self._content = 'Time,Symbol,Price,Quantity,Tag\n'
```

The algorithm saves `self._content` to the Object Store at the end of the backtest.

2. In the `OnData` method, place market orders when the EMAs cross.

```
def on_data(self, data: Slice):
    if not self._ema_short.is_ready or not self._ema_long.is_ready:
        return

    ema_short = self._ema_short.current.value
    ema_long = self._ema_long.current.value
    if ema_short > ema_long and not self.portfolio[self._symbol].is_long:
        self.market_order(self._symbol, 100, tag=f'BUY: ema-short: {ema_short:.4f} > ema-long: {ema_long:.4f}')
    elif ema_short < ema_long and not self.portfolio[self._symbol].is_short:
        self.market_order(self._symbol, -100, tag=f'SELL: ema-short: {ema_short:.4f} < ema-long: {ema_long:.4f}')
```

3. In the `OnOrderEvent` method, log each fill to the content string.

```
def on_order_event(self, order_event: OrderEvent):
    if order_event.status != OrderStatus.FILLED:
        return

    self._content += (
        f'{order_event.utc_time.strftime("%Y-%m-%d")},'
        f'{order_event.symbol},'
        f'{order_event.fill_price},'
        f'{order_event.fill_quantity},'
        f'{order_event.ticket.tag}\n'
    )
```

4. In the OnEndOfAlgorithm method, save the content to the Object Store.

```
def on_end_of_algorithm(self):
    key = f'{self.project_id}-{self.algorithm_id}.txt'
    self.object_store.save(key, self._content)
    self.log(f'Saved order log to Object Store: {key}')
```

5. Open the Research Environment and create a `QuantBook` .

```
qb = QuantBook()
```

6. Read the data from the Object Store.

```
# Replace the key with the one from your backtest log.
content = qb.object_store.read("<project-id>-<algorithm-id>.txt")
```

   The key you provide must be the same key you used to save the object. Check the backtest log for the exact key.

7. Split the content into lines.

```
lines = content.strip().split('\n')
# Display the header.
print(lines[0])
```

8. Display the first few rows (head) and last few rows (tail).

```
# Show the first 5 data rows.
for line in lines[1:6]:
    print(line)

# Show the last 5 data rows.
for line in lines[-5:]:
    print(line)
```

9. Search for all rows tagged as "BUY".

```
# Filter lines that contain "BUY".
buy_orders = [line for line in lines[1:] if 'BUY' in line]
for line in buy_orders:
    print(line)
```

```
class ObjectStoreLoggingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 1, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        # Create short and long EMA indicators.
        self._ema_short = self.ema(self._symbol, 10)
        self._ema_long = self.ema(self._symbol, 30)
        # Add a header row.
        self._content = 'Time,Symbol,Price,Quantity,Tag\n'

    def on_data(self, data: Slice) -> None:
        if not self._ema_short.is_ready or not self._ema_long.is_ready:
            return

        # Place a market order when the EMAs cross.
        ema_short = self._ema_short.current.value
        ema_long = self._ema_long.current.value
        if ema_short > ema_long and not self.portfolio[self._symbol].is_long:
            self.market_order(self._symbol, 100, tag=f'BUY: ema-short: {ema_short:.4f} > ema-long: {ema_long:.4f}')
        elif ema_short < ema_long and not self.portfolio[self._symbol].is_short:
            self.market_order(self._symbol, -100, tag=f'SELL: ema-short: {ema_short:.4f} < ema-long: {ema_long:.4f}')

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status != OrderStatus.FILLED:
            return

        # Log each fill as a row.
        self._content += (
            f'{order_event.utc_time.strftime("%Y-%m-%d")},'
            f'{order_event.symbol},'
            f'{order_event.fill_price},'
            f'{order_event.fill_quantity},'
            f'{order_event.ticket.tag}\n'
        )

    def on_end_of_algorithm(self) -> None:
        # Save the order log to the Object Store.
        key = f'{self.project_id}-{self.algorithm_id}.txt'
        self.object_store.save(key, self._content)
        self.log(f'Saved order log to Object Store: {key}')
```

Example of Custom Data

Follow these steps to use the Object Store as a data source for custom data :

1. Create a custom data class that defines a storage key and implements the `get_source` method.

```
class Bitstamp(PythonData):
    KEY = 'bitstampusd.csv'
    def get_source(self, config, date, isLiveMode):
        return SubscriptionDataSource(Bitstamp.KEY, SubscriptionTransportMedium.OBJECT_STORE)
```

2. Create an algorithm that downloads data from an external source and saves it to the Object Store .

```
class ObjectStoreCustomDataAlgorithm(QCAlgorithm):
    def initialize(self):
        if not self.object_store.contains_key(Bitstamp.KEY):
            url = "https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-
data/bitstampusd.csv"
            content = self.download(url)
            self.object_store.save(Bitstamp.KEY, content)
```

3. Call the `add_data` method to subscribe to the custom type .

```
class ObjectStoreCustomDataAlgorithm(QCAlgorithm):
    def initialize(self):
        self.custom_data_symbol = self.add_data(Bitstamp, "BTC").symbol
```

4. Implement the `Reader` method for the custom data class.

```
class Bitstamp(PythonData):
    def reader(self, config, line, date, isLiveMode):
        # Example Line Format:
        # Date      Open   High    Low     Close   Volume (BTC)    Volume (Currency)   Weighted Price
        # 2011-09-13 5.8    6.0     5.65    5.97    58.37138238,    346.0973893944      5.929230648356
        if not line[0].isdigit():
            return None

        coin = Bitstamp()
        coin.symbol = config.symbol
        data = line.split(',')

        # If value is zero, return None
        coin.value = float(data[4])
        if coin.value == 0:
            return None

        coin.time = datetime.strptime(data[0], "%Y-%m-%d")
        coin.end_time = coin.time + timedelta(1)
        coin["Open"] = float(data[1])
        coin["High"] = float(data[2])
        coin["Low"] = float(data[3])
        coin["Close"] = coin.value
        coin["VolumeBTC"] = float(data[5])
        coin["VolumeUSD"] = float(data[6])
        coin["WeightedPrice"] = float(data[7])
        return coin
```

5. To confirm your algorithm is receiving the custom data, request some historical data , then log and plot the data from the `Slice` object.

```
class ObjectStoreCustomDataAlgorithm(QCAlgorithm):
    def initialize(self):
        history = self.history(Bitstamp, self.custom_data_symbol, 200, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self.custom_data_symbol}.")
    def on_data(self, slice):
        data = slice.get(Bitstamp).get( self.custom_data_symbol)
        if not data:
            return

        self.log(f'{data.end_time}: Close: {data.close}')
        self.plot(self.custom_data_symbol, 'Price', data.close)
```

The following algorithm provides a full example of a sourcing custom data from the Object Store:

```
class ObjectStoreCustomDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2012, 9, 13)
        self.set_end_date(2021, 6, 20)
        self.set_cash(100000)

        # Load the custom data from ObjectStore.
        if not self.object_store.contains_key(Bitstamp.KEY):
            url = "https://raw.githubusercontent.com/QuantConnect/Documentation/master/Resources/datasets/custom-data/bitstampusd.csv"
            content = self.download(url)
            self.object_store.save(Bitstamp.KEY, content)

        # Request custom data for plotting.
        # Define the symbol and "type" of our generic data.
        self.custom_data_symbol = self.add_data(Bitstamp, "BTC").symbol

        # Example historical call.
        history = self.history(Bitstamp, self.custom_data_symbol, 200, Resolution.DAILY)
        self.debug(f"We got {len(history)} items from historical data request of {self.custom_data_symbol}.")

    def on_data(self, slice: Slice) -> None:
        # Plot on updated data.
        data = slice.get(Bitstamp).get(self.custom_data_symbol)
        if not data:
            return

        self.log(f'{data.end_time}: Close: {data.close}')
        self.plot(self.custom_data_symbol, 'Price', data.close)


class Bitstamp(PythonData):
    KEY = 'bitstampusd.csv'
    def get_source(self, config, date, isLiveMode):
        # Load data from object store.
        return SubscriptionDataSource(Bitstamp.KEY, SubscriptionTransportMedium.OBJECT_STORE)

    def reader(self, config, line, date, isLiveMode):
        # Example Line Format:
        # Date      Open   High    Low     Close   Volume (BTC)    Volume (Currency)   Weighted Price
        # 2011-09-13 5.8    6.0     5.65    5.97    58.37138238,    346.0973893944      5.929230648356
        if not line[0].isdigit():
            return None

        coin = Bitstamp()
        coin.symbol = config.symbol
        data = line.split(',')

        # If value is zero, return None
        coin.value = float(data[4])
        if coin.value == 0:
            return None

        coin.time = datetime.strptime(data[0], "%Y-%m-%d")
        coin.end_time = coin.time + timedelta(1)
        coin["Open"] = float(data[1])
        coin["High"] = float(data[2])
        coin["Low"] = float(data[3])
        coin["Close"] = coin.value
        coin["VolumeBTC"] = float(data[5])
        coin["VolumeUSD"] = float(data[6])
        coin["WeightedPrice"] = float(data[7])
        return coin
```

Preserve Insights Between Deployments

Follow these steps to use the Object Store to preserve the algorithm state across live deployments:

1. Create an algorithm that defines a storage key and adds insights to the Insight Manager .

```
class ObjectStoreChartingAlgorithm(QCAlgorithm):
    def initialize(self):
        self.insight_key = f"{self.project_id}/insights"
        self.set_universe_selection(ManualUniverseSelectionModel([ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]))
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(5), 0.025, None))
```

2. At the top of the algorithm file, add the following imports:

```
from Newtonsoft.Json import JsonConvert
from System.Collections.Generic import List
```

`Insight` objects are C# objects, so you need the preceding C# libraries to serialize and deserialize them.

3. In the on_end_of_algorithm event handler of the algorithm, get the Insight objects and save them in the Object Store as a JSON object.

```
def on_end_of_algorithm(self):
    insights = self.insights.get_insights(lambda x: x.is_active(self.utc_time))
    content = ','.join([JsonConvert.SerializeObject(x) for x in insights])
    self.object_store.save(self.insight_key, f'[{content}]')
```

4. At the bottom of the `initialize` method, read the Insight objects from the Object Store and add them to the Insight Manager .

```
if self.object_store.contains_key(self.insight_key):
    insights = JsonConvert.DeserializeObject[List[Insight]](
        self.object_store.read(self.insights_key)
    )
    self.insights.add_range(insights)
```

The following algorithm provides a full example of preserving the Insight state between deployments:

```
from Newtonsoft.Json import JsonConvert
from System.Collections.Generic import List


class ObjectStoreInsightsAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.universe_settings.resolution = Resolution.DAILY

        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.insights_key = f"{self.project_id}/insights"

        # Read the file with the insights
        if self.object_store.contains_key(self.insights_key):
            # Load cached JSON key value pair from object store, then add to algorithm insights.
            insights = JsonConvert.DeserializeObject[List[Insight]](
                self.object_store.read(self.insights_key)
            )
            self.log(f"Read {len(insights)} insight(s) from the Object Store")
            self.insights.add_range(insights)

            # Delete the key to reuse it
            self.object_store.delete(self.insights_key)

        self.set_universe_selection(ManualUniverseSelectionModel([ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]))
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(5), 0.025, None))
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Resolution.DAILY))

    def on_end_of_algorithm(self) -> None:
        # Get all active insights.
        insights = self.insights.get_insights(lambda x: x.is_active(self.utc_time))
        # If we want to save all insights (expired and active), we can use
        # insights = self.insights.get_insights(lambda x: True)
        self.log(f"Save {len(insights)} insight(s) to the Object Store.")
        content = ','.join([JsonConvert.SerializeObject(x) for x in insights])
        self.object_store.save(self.insights_key, f'[{content}]')
```

Live Trading Considerations

If you update the Object Store from outside of your live trading node and then try to read the new content from inside your live trading algorithm, you may not get the new content because the Object Store caches data to speed up execution. To ensure you get the latest value for a specific key in the Object Store, clear the cache and then read the data .

```
self.object_store.clear()
if self.object_store.contains_key(key):
    value = self.object_store.read(key)
```

Optimization

Optimization

Parameters

Introduction

Parameters are project variables that your algorithm uses to define the value of internal variables like indicator arguments or the length of lookback windows. Parameters are stored outside of your algorithm code, but we inject the values of the parameters into your algorithm when you run a backtest, deploy a live algorithm, or launch an optimization job. To use parameters, set some parameters in your project and then load them into your algorithm.

Set Parameters

The process to set parameter values depends on the environment you use to write algorithms. See the tutorial in one of the following environments:

- Cloud Platform
- Local Platform
- LEAN CLI

Get Parameters

To get a parameter value into your algorithm, call the `get_parameter` method of the algorithm class.

```py
# Get the parameter string-value with this matching key.
# Project parameters are useful for optimization or securing variables outside the source-code.
parameter_value = self.get_parameter("Parameter Name")
```

The `get_parameter` method returns a string by default. If you provide a default parameter value, the method returns the parameter value as the same data type as the default value. If there are no parameters in your project that match the name you pass to the method and you provide a default value to the method, it returns the default value. The following table describes the arguments the `get_parameter` method accepts:

Argument	Data Type	Description	Default Value
`name`	`str`	The name of the parameter to get	
`default_value`	`str/int/double`	The default value to return	`None`

The following example algorithm gets the values of parameters of each data type:

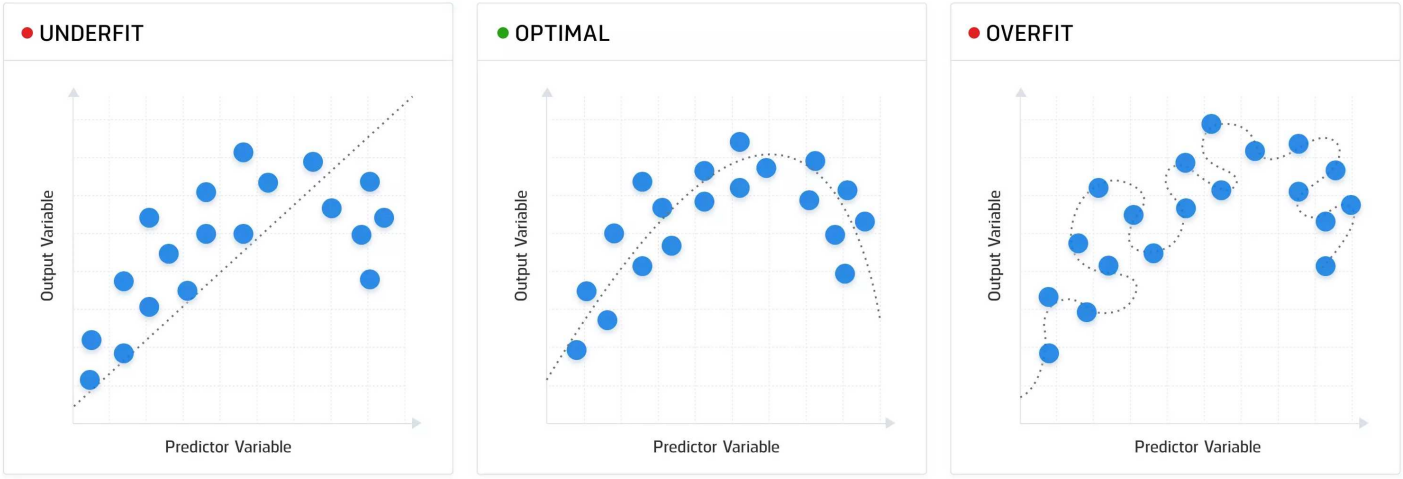
```py
# Example to get the values of parameters of each data type:
class ParameterizedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # Get the parameter value and return an integer.
        int_parameter_value = self.get_parameter("int_parameter_name", 100)

        # Get the parameter value and return a double.
        float_parameter_value = self.get_parameter("float_parameter_name", 0.95)

        # Get the parameter value as a string.
        string_parameter_value = self.get_parameter("parameter_name", "default_string_value")

        # Cast the string to an integer.
        parameter_value = int(string_parameter_value)
```

The parameter values are sent to your algorithm when you deploy the algorithm, so it's not possible to change the parameter values while the

algorithm runs.

Overfitting

Overfitting occurs when a function is fit too closely fit to a limited set of training data. Overfitting can occur in your trading algorithms if you have many parameters or select parameters values that worked very well in the past but are sensitive to small changes in their values. In these cases, your algorithm will likely be fine-tuned to fit the detail and noise of the historical data to the extent that it negatively impacts the live performance of your algorithm. The following image shows examples of underfit, optimally-fit, and overfit functions:



An algorithm that is dynamic and generalizes to new data is more likely to survive across different market conditions and apply to other markets.

Look-Ahead Bias

Look-ahead bias occurs when an algorithm makes decisions using data that would not have yet been available. For instance, in optimization jobs, you optimize a set of parameters over a historical backtesting period. After the optimizer finds the optimal parameter values, the backtest period becomes part of the in-sample data. If you run a backtest over the same period using the optimal parameters, look-ahead bias has seeped into your research. In reality, it would not be possible to know the optimal parameters during the testing period until after the testing period is over. To avoid issues with look-ahead bias, optimize on older historical data and test the optimal parameter values on recent historical data. Alternatively, apply walk forward optimization to optimize the parameters on smaller batches of history.

Live Trading Considerations

To update parameters in live mode, add a Schedule Event that downloads a remote file and uses its contents to update the parameter values.

```py
def initialize(self):
    self.parameters = { }
    if self.live_mode:
        def download_parameters():
            content = self.download(url_to_remote_file)
            # Convert content to self.parameters

        self.schedule.on(self.date_rules.every_day(), self.time_rules.every(timedelta(minutes=1)), download_parameters)
```

Examples

The following examples demonstrate common practices for using optimization parameters.

Example 1: Bond Candidate Selection

The following algorithm holds a 60/40 stock-bond portfolio rebalancing monthly. To select the best bond ETF to act as a proxy to trade bonds, we create a list of bond tickers and grid search the best using each to backtest. Using a `bond-id` parameter, which is an integer ranging from 0 to the list size - 1 with a step size of 1, we can perform optimization to test each of them.

```py
class OptimizationParameterAlgorithm(QCAlgorithm):
    # A list of bond ETFs as candidates to trade as bond proxies.
    _bond_list = [
        "TLT", "IEF", "SHY", "VCSH", "VCLT"
    ]

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Get the bond candidate to be tested.
        bond_id = self.get_parameter("bond-id", 0)

        # Request SPY and the selected bond data for trading.
        self.spy = self.add_equity("SPY", Resolution.MINUTE).symbol
        self.bond = self.add_equity(self._bond_list[bond_id], Resolution.MINUTE).symbol

        # Rebalance for a stock-bond portfolio monthly.
        self.schedule.on(
            self.date_rules.month_start(self.spy),
            self.time_rules.after_market_open(self.spy, 1),
            self.rebalance
        )

    def rebalance(self) -> None:
        # Hold an all-weather stock-bond 60/40 portfolio.
        self.set_holdings(self.spy, 0.6)
        self.set_holdings(self.bond, 0.4)
```

Example 2: Test Different Period

This algorithm holds SPY for the whole day following the direction from the previous day. To test which period is the most effective for this momentum-following strategy, we can set an `offset` parameter and adjust the backtest period accordingly.

```py
from dateutil.relativedelta import relativedelta
class OptimizationParameterAlgorithm(QCAlgorithm):
    last_price = 0

    def initialize(self) -> None:
        # Get the number of months to offset the start date.
        offset = self.get_parameter("offset", 0)
        # Create a variable start date with offset to test which periods the algorithm performs better.
        start_date = datetime(2024, 1, 1)
        self.set_start_date(start_date + relativedelta(months=offset))
        # One-month fixed period test per backtest.
        self.set_end_date(self.start_date + relativedelta(months=1))

        # Request daily SPY data for trading.
        self.spy = self.add_equity("SPY", Resolution.DAILY).symbol

        self.set_warm_up(1)

    def on_data(self, slice: Slice) -> None:
        # Trade based on updated data.
        bar = slice.bars.get(self.spy)
        if bar:
            # Trade with riding on the last day's momentum.
            if self.last_price != 0 and bar.close > self.last_price and not self.portfolio[self.spy].is_long:
                self.set_holdings(self.spy, 0.5)
            elif self.last_price != 0 and bar.close < self.last_price and not self.portfolio[self.spy].is_short:
                self.set_holdings(self.spy, -0.5)

            self.last_price = bar.close
```

Example 3: Hyperparameter Tuning

The following example algorithm demonstrates an EMA cross strategy. To optimize for the best setup to attain the greatest Sharpe Ratio, the algorithm grid searches the best hyperparameters (the EMA periods) that do so.

```py
class ParameterizedAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2024, 12, 31);
        self.set_cash(100000)
        # For warming up indicators.
        self.settings.automatic_indicator_warm_up = True

        # Request SPY data to feed the indicator and trade.
        self.spy = self.add_equity("SPY").symbol

        # Get and store the EMA settings to local parameters.
        fast_period = self.get_parameter("ema-fast", 50)
        slow_period = self.get_parameter("ema-slow", 200)

        # Set up the 2 EMA indicators for trade signal generation with the fast and slow periods from parameters.
        self._fast = self.ema("SPY", fast_period, Resolution.DAILY)
        self._slow = self.ema("SPY", slow_period, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Following the uptrend indicated by price up in nearer periods.
            if bar.close > self._fast.current.value > self._slow.current.value:
                self.set_holdings(self.spy, 0.5)
            # Following the downtrend indicated by price down in nearer periods.
            elif bar.close < self._fast.current.value < self._slow.current.value:
                self.set_holdings(self.spy, -0.5)
            # Liquidate if the trend is not strong.
            else:
                self.liquidate()
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
ParameterizedAlgorithm.py Python

Optimization

Walk Forward Optimization

Introduction

Walk forward optimization is the practice of periodically adjusting the logic or parameters of a strategy to optimize some objective function over a trailing window of time. For example, say your strategy is to hold a long position when an asset is trading above its simple moving average (SMA) and to hold a short position when it's trading below the SMA. If your objective is to maximize returns, how many trailing data points should you use to compute the SMA?

Benefits

The parameter values that maximize your algorithm's performance are typically a function of the start and end dates of your training dataset. Market conditions change over time and a strategy that works well during one period may not perform as well during another. By continually adjusting the strategy parameters based on recent data, walk-forward optimization tailors your strategy to the latest market trends and patterns.

Optimization Frequency

There is a tradeoff when it comes to how often you optimize your algorithm's parameters. On one hand, if you frequently optimize your parameters, your parameters will be fit to the most recent, most meaningful market data. On the other hand, if you wait a long time between optimization sessions, your algorithm will run faster and you reduce the chance of overfitting. To define your optimization schedule, pass DateRules and TimeRules arguments to the train method.

```
// To define the optimization schedule, pass self.date_rules and self.time_rules arguments to the train method.
symbol = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
self.train(
    self.date_rules.month_start(symbol),
    self.time_rules.midnight,
    lambda: self._do_wfo(self._optimization_func, max, objective)
)
```

EMA Crossover Example

Follow these steps to implement walk forward optimization for an exponential moving average (EMA) crossover strategy:

1. Define the backtest dates and starting cash.

```
# Set the start date, end date, and starting cash.
def initialize(self):
    self.set_start_date(2024, 9, 1)
    self.set_end_date(2024, 12, 31)
    self.set_cash(100_000)
```

2. Enable the automatic indicator warm up setting .

```
# Set the automatic_indicator_warm_up property to true to enable automatic algorithm warm-up.
def initialize(self):
    # . . .
    self.settings.automatic_indicator_warm_up = True
```

3. Subscribe to an asset .

```
# Add daily SPY Equity data.
def initialize(self):
    # . . .
    self._security = self.add_equity("SPY", Resolution.DAILY)
    self._symbol = self._security.symbol
```

4. Add some members for the EMA indicators.

```py
# Initialize members for short and long EMA indicators.
def initialize(self):
    # . . .
    self._short_ema = None
    self._long_ema = None
```

5. Define the trading rules.

```py
# Define the trading logic.
def on_data(self, data):
    if self.is_warming_up:
        return

    # Case 1: Short EMA is above long EMA.
    if (self._short_ema > self._long_ema and
        not self._security.holdings.is_long):
        self.set_holdings(self._symbol, 1)
    # Case 2: Short EMA is below long EMA.
    elif (self._short_ema < self._long_ema and
            not self._security.holdings.is_short):
        self.set_holdings(self._symbol, 0)
```

6. Define a method that will adjust the algorithm's behavior, given a set of optimal parameters.

```py
# Define a method to adjust the algorithm's behavior based on the new optimal parameters.
def _update_algorithm_logic(self, optimal_parameters):
    # Remove the old indicators.
    if self._short_ema:
        self.deregister_indicator(self._short_ema)
    if self._long_ema:
        self.deregister_indicator(self._long_ema)
    # Create the new indicators.
    self._short_ema = self.ema(
        self._symbol, optimal_parameters['short_ema'], Resolution.DAILY
    )
    self._long_ema = self.ema(
        self._symbol, optimal_parameters['long_ema'], Resolution.DAILY
    )
```

7. Back in `initialize` method, generate all the possible combinations of parameter values.

```py
# In the initialize method, generate all possible combinations of parameter values.
import itertools

def initialize(self):
    # . . .
    self._parameter_sets = self._generate_parameter_sets(
        {
            'short_ema': (10, 50, 10),  # min, max, step
            'long_ema': (60, 200, 10)
        }
    )
    # . . .

def _generate_parameter_sets(self, search_space):
    # Create ranges for each parameter.
    ranges = {
        parameter_name: np.arange(min_, max_ + step_size, step_size)
        for parameter_name, (min_, max_, step_size) in search_space.items()
    }

    # Calculate the cartesian product and create a list of dictionaries for
    # the parameter sets.
    return [
        dict(zip(ranges.keys(), combination))
        for combination in list(itertools.product(*ranges.values()))
    ]
```

8. Define the objective function.

```py
# Define an objective function to optimize the algorithm's overall performance.
def initialize(self):
    # . . .
    objective = self._cumulative_return
    # . . .

def _cumulative_return(self, daily_returns):
    return (daily_returns + 1).cumprod()[-1] - 1
```

9. Define the optimization function.

```py
# Evaluate parameter sets with the objective function.
def _optimization_func(self, data, parameter_set, objective):
    p1 = parameter_set['short_ema']
    p2 = parameter_set['long_ema']
    short_ema = data['close'].ewm(p1, min_periods=p1).mean()
    long_ema = data['close'].ewm(p2, min_periods=p2).mean()
    exposure = (short_ema - long_ema).dropna().apply(np.sign)\
        .replace(0, pd.NA).ffill().shift(1)
    # ^ shift(1) because we enter the position on the next day.
    asset_daily_returns = data['open'].pct_change().shift(-1)
    # ^ shift(-1) because we want each entry to be the return from
    # the current day to the next day.
    strategy_daily_returns = (exposure * asset_daily_returns).dropna()
    return objective(strategy_daily_returns)
```

10. Schedule the optimization sessions.

```py
# Schedule the WFO process to run at the start of each month at midnight.
def initialize(self):
    # . . .
    self.train(
        self.date_rules.month_start(self._symbol),
        self.time_rules.midnight,
        lambda: self._do_wfo(self._optimization_func, max, objective)
    )
    # . . .

def _do_wfo(self, optimization_func, min_max, objective):
    # Get the historical data you need to calculate the scores.
    prices = self.history(
        self._symbol, timedelta(365), Resolution.DAILY
    ).loc[self._symbol]

    # Calculate the score of each parameter set.
    scores = [
        optimization_func(prices, parameter_set, objective)
        for parameter_set in self._parameter_sets
    ]

    # Find the parameter set that minimizes/maximizes the objective function.
    optimal_parameters = self._parameter_sets[scores.index(min_max(scores))]

    # Adjust the algorithm's logic.
    self._update_algorithm_logic(optimal_parameters)
```

11. Add a warm-up period .

    The warm-up period ensures that the algorithm calls the `_do_wfo` method at least one time before the algorithm starts trading.

```py
# Set the warm-up period.
def initialize(self):
    # . . .
    self.set_warm_up(timedelta(45))
```

The following algorithm combines the code snippets above into a single algorithm:

```py
# region imports
from AlgorithmImports import *
import itertools
# endregion

# Walk forward optimization with an EMA crossover strategy on SPY.
class WalkForwardOptimizationEmaAlgorithm(QCAlgorithm):
    def initialize(self):
```

```
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self.settings.automatic_indicator_warm_up = True
        self._security = self.add_equity("SPY", Resolution.DAILY)
        self._symbol = self._security.symbol
        self._short_ema = None
        self._long_ema = None
        # Generate all possible combinations of parameter values.
        self._parameter_sets = self._generate_parameter_sets(
            {
                'short_ema': (10, 50, 10),  # min, max, step
                'long_ema': (60, 200, 10)
            }
        )
        # Schedule the WFO process to run at the start of each month.
        objective = self._cumulative_return
        self.train(
            self.date_rules.month_start(self._symbol),
            self.time_rules.midnight,
            lambda: self._do_wfo(self._optimization_func, max, objective)
        )
        self.set_warm_up(timedelta(45))

    def on_data(self, data):
        if self.is_warming_up:
            return
        # Case 1: Short EMA is above long EMA.
        if (self._short_ema > self._long_ema and
            not self._security.holdings.is_long):
            self.set_holdings(self._symbol, 1)
        # Case 2: Short EMA is below long EMA.
        elif (self._short_ema < self._long_ema and
                not self._security.holdings.is_short):
            self.set_holdings(self._symbol, 0)

    def _update_algorithm_logic(self, optimal_parameters):
        # Remove the old indicators.
        if self._short_ema:
            self.deregister_indicator(self._short_ema)
        if self._long_ema:
            self.deregister_indicator(self._long_ema)
        # Create the new indicators.
        self._short_ema = self.ema(
            self._symbol, int(optimal_parameters['short_ema']), Resolution.DAILY
        )
        self._long_ema = self.ema(
            self._symbol, int(optimal_parameters['long_ema']), Resolution.DAILY
        )

    def _generate_parameter_sets(self, search_space):
        # Create ranges for each parameter.
        ranges = {
            parameter_name: np.arange(min_, max_ + step_size, step_size)
            for parameter_name, (min_, max_, step_size) in search_space.items()
        }
        # Calculate the cartesian product.
        return [
            dict(zip(ranges.keys(), combination))
            for combination in list(itertools.product(*ranges.values()))
        ]

    def _cumulative_return(self, daily_returns):
        return (daily_returns + 1).cumprod()[-1] - 1

    def _optimization_func(self, data, parameter_set, objective):
        p1 = parameter_set['short_ema']
        p2 = parameter_set['long_ema']
        short_ema = data['close'].ewm(p1, min_periods=int(p1)).mean()
        long_ema = data['close'].ewm(p2, min_periods=int(p2)).mean()
        exposure = (short_ema - long_ema).dropna().apply(np.sign)\
            .replace(0, pd.NA).ffill().shift(1)
        asset_daily_returns = data['open'].pct_change().shift(-1)
        strategy_daily_returns = (exposure * asset_daily_returns).dropna()
        return objective(strategy_daily_returns)

    def _do_wfo(self, optimization_func, min_max, objective):
        # Get the historical data to calculate the scores.
        prices = self.history(
            self._symbol, timedelta(365), Resolution.DAILY
        ).loc[self._symbol]
        # Calculate the score of each parameter set.
        scores = [
            optimization_func(prices, parameter_set, objective)
            for parameter_set in self._parameter_sets
        ]
        # Find the parameter set that maximizes the objective function.
        optimal_parameters = self._parameter_sets[scores.index(min_max(scores))]
        # Adjust the algorithm's logic.
        self._update_algorithm_logic(optimal_parameters)
```

Ideas

The following list provides some ideas on how you could use walk forward optimization:

1. Find the range of PE ratios that mazimize your algorithm's performance, and then use that range in your fundamental universe filter.
2. Experiment with asset universe diversity and counts.
3. Take 10 fundamental factors (PE, market cap, etc...). For each optimization session, step through and picks one factor at a time, using it as a ranking factor for the universe. Find the factor that leads to the greatest return over the lookback window and use it as the sole factor for the following month.

Machine Learning

Machine Learning

Key Concepts

Introduction

Machine learning is a field of study that combines statistics and computer science to build intelligent systems that predict outcomes. You can use machine learning techniques in your trading strategies.

Supported Libraries

LEAN supports several machine learning libraries. You can import these packages and use them in your algorithms.

Name	Version	Language	Import Statement	Examp

Add New Libraries

To request a new library, contact us . We will add the library to the queue for review and deployment. Since the libraries run on our servers, we need to ensure they are secure and won't cause harm. The process of adding new libraries takes 2-4 weeks to complete. View the list of libraries currently under review on the Issues list of the Lean GitHub repository .

Save Models

After you train a model, you can save it into the Object Store . In QuantConnect Cloud, we back up your Object Store data on QuantConnect servers. In local algorithms, your local machine saves the Object Store data. If you save models in live algorithms, save them at the end of the training method so you can access the trained model again if your algorithm stops executing. If you save models in backtests, save them during the `on_end_of_algorithm` event handler so that saving multiple times doesn't slow down your backtest.

If you train models outside of QuantConnect (for example, on your local machine), you can upload the model files to the Object Store with the LEAN CLI or the Cloud API . For library-specific upload instructions, see the Upload Models section in each Popular Libraries tutorial.

To view examples of storing library-specific models, see Popular Libraries .

Load Models

You can load machine learning models from the Object Store or a custom data file like pickle. If you load models from the Object Store, before you load the model into your algorithm, in the `initialize` method, check if the Object Store already contains the model. To avoid look-ahead bias in backtests, don't train your model on the same data you use to test the model.

Increase Backtest Speed

If your backtests are slow because they recalculate features and retrain models on every run, you can precompute your ML predictions in the Research Environment and save them to the Object Store . Your algorithm then streams in the precomputed predictions as a custom universe dataset, skipping all feature engineering and model inference at runtime. This pattern makes backtests significantly faster and lets you iterate on portfolio logic without rerunning the ML pipeline. For a full walkthrough, see Precomputed ML Predictions .

Library Errors

LEAN gracefully handles runtime errors that your code raises. It generally informs the location and cause of the error before the algorithm is terminated. FATAL ERROR EXCEPTION refers to a critical failure from third-party libraries, such as machine learning models, that LEAN doesn't handle.

Most QuantConnect members experience the invalid memory access error (SIGBUS) or division by zero error (SIGFPE) marked as "7 Killed" and "8 Killed" in the error message, respectively. The following sections explain some common errors and what causes them.

Memory Issues

You may experience the following memory issues:

- Out of Memory (OOM) : The process requires more memory (RAM or GPU memory) than is available, leading to a crash.
- Memory Leaks : If the program fails to properly release memoryy, it can eventually exhaust available resources.
- Invalid Memory Access : Bugs in custom C/C++ extensions, kernels, or libraries (for example, CUDA kernels for GPU acceleration) can cause misaligned memory access or invalid memory operations.

Numerical Instability

You may experience the following numerical instability errors:

- Division by Zero : This can occur due to input data with zeroes or during calculations, especially in custom loss functions or optimization algorithms.
- NaN or Infinity Values : These can arise due to data with NaN values, improper initialization, exploding gradients, or unstable mathematical operations.

Software Bugs

You may experience the following software bugs:

- Code Errors : Bugs in the implementation of the model, data preprocessing, or training loop can cause crashes.
- Library Incompatibility : Using incompatible versions of libraries (for example, TensorFlow or PyTorch) or dependencies can lead to fatal errors.

Machine Learning

Training Models

Introduction

Algorithms usually must process each timeslice within 10 minutes, but the `train` method allows you to increase this time to train machine learning models. The length of time you can train depends on your training quotas.

Train Models

To train models immediately, call the `train` method and pass in the name of your training method.

```
// Run an ML model training session and extend method call timeouts.
self.train(self.my_method)
```

Immediate training is most useful for training your model when you first deploy your strategy to production or when the model's performance begins to degrade.

Schedule Training Sessions

You can schedule model training sessions in a similar way to a Scheduled Event. To schedule a training session do this, pass in a DateRules and TimeRules argument to the `train` method.

```
# Set TrainingMethod to be executed at 8:00 am every Sunday
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)
```

We recommend you schedule your training sessions for when the market is closed to get the best compute allocation. While the market is open, your CPU is occupied with processing incoming tick data and handling other LEAN events.

Training Quotas

Training resources are allocated with a leaky bucket algorithm where you can use a maximum of n-minutes in a single training session and the number of available minutes refills over time. This design gives you burst allocations when you need them and recharges the allowance to prepare for the next training.

Cloud Quotas

If you execute algorithms in QuantConnect Cloud, see Training Quotas for more information about the training quotas.

Local Quotas

If you execute algorithms locally, the following table shows the default settings for the leaky bucket algorithm:

Setting	Value
Capacity (minutes)	120
Time interval (minutes)	1440
Refill amount (minutes per time interval)	Capacity / 7

To allow virtually unlimited training for local algorithms, add the following key-value pairs to your Lean / Launcher / config.json file:

```
"scheduled-event-leaky-bucket-capacity" : 99999999,
"scheduled-event-leaky-bucket-time-interval-minutes" : 1,
"scheduled-event-leaky-bucket-refill-amount": 999999,
```

Check Model Readiness

In backtests, the `train` method is synchronous, so it blocks your algorithm execution while the model trains. In live trading, the `train` method is asynchronous, so ensure your model is trained before you continue the algorithm execution. Training occurs on a separate thread, so set a boolean flag to notify your algorithm of the model state. A semaphore is a thread-safe flag you can use to synchronize program operations across different threads.

```py
# Example of using a flag to signal the model is ready to use:
class SemaphoreTrainingAlgorithm(QCAlgorithm):

    model = None  # Model object
    model_is_training = False  # Model state flag

    def initialize(self) -> None:
        self.train(self.my_training_method)

    def my_training_method(self) -> None:
        self.model_is_training = True
        # Train the model here.
        self.model_is_training = False

    def on_data(self, slice: Slice) -> None:
        # Don't use the model while it is training.
        if self.model_is_training:
            return

        # Once training is complete, use the model.
        result = self.model.predict()
```

Examples

The following examples demonstrate some common practices for training the machine learning model.

Example 1: Scikit-Learn

The below algorithm makes use of `Scikit-Learn` library to predict the future price movement using the previous 5 OHLCV data. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

```
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
import joblib

class ScikitLearnExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self.symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # 2-year data to train the model.
        training_length = 252*2
        self.training_data = RollingWindow(training_length)
        # Warm up the training dataset to train the model immediately.
        history = self.history[TradeBar](self.symbol, training_length, Resolution.DAILY)
        for trade_bar in history:
            self.training_data.add(trade_bar)

        # Retrieve already trained model from object store to use immediately.
        if self.object_store.contains_key("sklearn_model"):
            file_name = self.object_store.get_file_path("sklearn_model")
            self.model = joblib.load(file_name)
        # Otherwise, grid serach with hyperparameter choices to create an optimal support vector regressor to predict price movement.
        else:
            param_grid = {'C': [.05, .1, .5, 1, 5, 10],
                          'epsilon': [0.001, 0.005, 0.01, 0.05, 0.1],
                          'gamma': ['auto', 'scale']}
            self.model = GridSearchCV(SVR(), param_grid, scoring='neg_mean_squared_error', cv=5)

        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.training_data)[::-1])
        daily_pct_change = training_df.pct_change().dropna()

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(daily_pct_change)-n_steps):
            features.append(daily_pct_change.iloc[i:i+n_steps].values.flatten())
            labels.append(daily_pct_change['close'].iloc[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()
        # Recalibrate the model based on updated data.
        if isinstance(self.model, GridSearchCV):
            self.model = self.model.fit(features, labels).best_estimator_
        else:
            self.model = self.model.fit(features, labels)

    def on_data(self, slice: Slice) -> None:
        if self.symbol in slice.bars:
            self.training_data.add(slice.bars[self.symbol])

        # Get prediction by the updated features.
        features, _ = self.get_features_and_labels()
        prediction = self.model.predict(features[-1].reshape(1, -1))
        prediction = float(prediction)

        # If the predicted direction is going upward, buy SPY.
        if prediction > 0:
            self.set_holdings(self.symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction < 0:
            self.set_holdings(self.symbol, -1)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        model_key = "sklearn_model"
        file_name = self.object_store.get_file_path(model_key)
        joblib.dump(self.model, file_name)
        self.object_store.save(model_key)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
TrainingExampleAlgorithm.py Python

Machine Learning

Popular Libraries

These are examples of using some of the most common machine learning libraries in an algorithm. Click one to learn more.

Aesera

Copula

GPlearn

Hmmlearn

Keras

PyTorch

Scikit-Learn

Tensorflow

Tslearn

XGBoost

Popular Libraries

Aesera

Introduction

This page explains how to build, train, deploy and store `Aesera` models.

To access Aesara, use the Legacy package environment. To set this environment, see the documentation in Cloud Platform or Local Platform .

Import Libraries

Import the `aesera` and `sklearn` libraries.

```
PY
from AlgorithmImports import *
import aesara
import aesara.tensor as at
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `sklearn` model and make predictions.

```
PY
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a logistic regression prediction model that uses the following features and labels:

Data Category	Description
Features	Normalized daily close price of the SPY over the last 5 days
Labels	Return direction of the SPY over the next day

The following image shows the time difference between the features and labels:

Follow the below steps to build the model:

1. Initialize variables.

```
# Declare Aesara's symbolic variables.
x = at.dmatrix("x")
y = at.dvector("y")

# Initialize a NumPy random number generator with a fixed seed for reproducibility.
rng = np.random.default_rng(100)

# Initialize the weight vector w randomly using shared so the model coefficients keep their values
# between training iterations (updates).
w = aesara.shared(rng.standard_normal(5), name="w")

# Initialize the bias term.
b = aesara.shared(0., name="b")
```

2. Construct the model graph.

```
# Construct the Aesara expression graph.
p_1 = 1 / (1 + at.exp(-at.dot(x, w) - b))           # Logistic transformation.
prediction = p_1 > 0.5                              # The prediction thresholded.
xent = y * at.log(p_1) - (1 - y) * at.log(1 - p_1)  # Cross-entropy log-loss function.
cost = xent.mean() + 0.01 * (w ** 2).sum()          # The cost to minimize (MSE).
gw, gb = at.grad(cost, [w, b])                      # Compute the gradient of the cost.
```

3. Compile the model.

```
# Define training and prediction functions with Aesara to optimize weights
# and biases during training and generate predictions.
self._train = aesara.function(
    inputs=[x, y],
    outputs=[prediction, xent],
    # Update weights and biases.
    updates=((w, w - 0.1 * gw), (b, b - 0.1 * gb)))
self.predict = aesara.function(inputs=[x], outputs=prediction)
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of training data.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training.
def get_features_and_labels(self, n_steps=5):
    training_df = self.PandasConverter.GetDataFrame[TradeBar](list(self.training_data)[::-1])['close']

    features = []
    for i in range(1, n_steps + 1):
        close = training_df.shift(i)[n_steps:-1]
        close.name = f"close-{i}"
        features.append(close)
    features = pd.concat(features, axis=1)
    # Normalize using the 5 day interval.
    features = MinMaxScaler().fit_transform(features.T).T[4:]

    Y = training_df.pct_change().shift(-1)[n_steps*2-1:-1].reset_index(drop=True)
    labels = np.array([1 if y > 0 else 0 for y in Y])   # Binary class.

    return features, labels

def my_training_method(self):
    features, labels = self.get_features_and_labels()
    D = (features, labels)
    self._train(D[0], D[1])
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model right now.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM.
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the current bar to the training data to ensure the model trains with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol])
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the feature set and make a prediction.
features, _ = self.get_features_and_labels()
prediction = self.predict(features[-1].reshape(1, -1))
prediction = float(prediction)
```

You can use the label prediction to place orders.

```
# Place orders based on the prediction.
if prediction == 1:
    self.set_holdings(self._symbol, 1)
elif prediction == 0:
    self.set_holdings(self._symbol, -1)
```

Save Models

Follow these steps to save `sklearn` models into the Object Store :

1. Set the key name you want to store the model under in the Object Store.

```
# Set the storage key to something representative.
model_key = f"{self.project_id}/model"
```

2. Call the `get_file_path` method with the key.

```
# Get the file path to save and access the model in the Object Store.
file_name = self.object_store.get_file_path(model_key)
```

   This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

```
# Serialize and save the model.
joblib.dump(self.predict, file_name)
```

   If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model <pathTo>/model
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `sklearn` models that you saved in the Object Store. To load a `sklearn` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the trained Aesera model from the Object Store.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `Aesera` library.

Example 1: Aesera

The below algorithm makes use of `Aesera` library to predict the future price movement using the previous 5 OHLCV data. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
import aesara
import aesara.tensor as at
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import joblib

class AeseraExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 252*2
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

        # Declare Aesara symbolic variables.
        x = at.dmatrix("x")
        y = at.dvector("y")

        # Initialize the weight vector w randomly using shared so the model coefficients keep
        # their values between training iterations (updates).
        rng = np.random.default_rng(100)
        w = aesara.shared(rng.standard_normal(5), name="w")
        # Initialize the bias term.
        b = aesara.shared(0., name="b")

        # Construct Aesara expression graph.
        p_1 = 1 / (1 + at.exp(-at.dot(x, w) - b))       # Probability that target = 1
        prediction = p_1 > 0.5                          # The prediction thresholded
        xent = y * at.log(p_1) - (1-y) * at.log(1-p_1)  # Cross-entropy log-loss function
        cost = xent.mean() + 0.01 * (w ** 2).sum()      # The cost to minimize
        gw, gb = at.grad(cost, [w, b])                  # Compute the gradient of the cost

        # Compile the model and train it later.
        self._train = aesara.function(
                inputs=[x, y],
                outputs=[prediction, xent],
                updates=((w, w - 0.1 * gw), (b, b - 0.1 * gb)))
        self.predict = aesara.function(inputs=[x], outputs=prediction)

    def on_warmup_finished(self) -> None:
        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.securities[self._symbol].session)[::-1])['close']

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        for i in range(1, n_steps + 1):
            close = training_df.shift(i)[n_steps:-1]
            close.name = f"close-{i}"
            features.append(close)
        features = pd.concat(features, axis=1)
        # Normalize using the 5 day interval.
        features = MinMaxScaler().fit_transform(features.T).T[4:]

        Y = training_df.pct_change().shift(-1)[n_steps*2-1:-1].reset_index(drop=True)
        # Train to predict the direction only.
        labels = np.array([1 if y > 0 else 0 for y in Y])   # binary class

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()
        D = (features, labels)
        # Recalibrate the model based on updated data.
        self._train(D[0], D[1])

        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        model_key = f"{self.project_id}/model_test_aesera"
        file_name = self.object_store.get_file_path(model_key)
        joblib.dump(self.predict, file_name)
        self.object_store.save(model_key)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or self._symbol not in slice.bars:
            return

        # Get prediction using the latest 5 OHLCV.
        features, _ = self.get_features_and_labels()
        prediction = self.predict(features[-1].reshape(1, -1))
        prediction = float(prediction)
```

```
        # If the predicted direction is going upward, buy SPY.
        if prediction == 1:
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction == 0:
            self.set_holdings(self._symbol, -1)
```

Popular Libraries

Copula

Introduction

This page explains how to build, train, deploy and store `Copula` models.

Import Libraries

Import the `copula` , `scipy` , and `sklearn` libraries.

```
PY
from AlgorithmImports import *
from copulas.bivariate import Bivariate
from scipy import stats
from sklearn.preprocessing import MinMaxScaler
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `copula` model and make predictions.

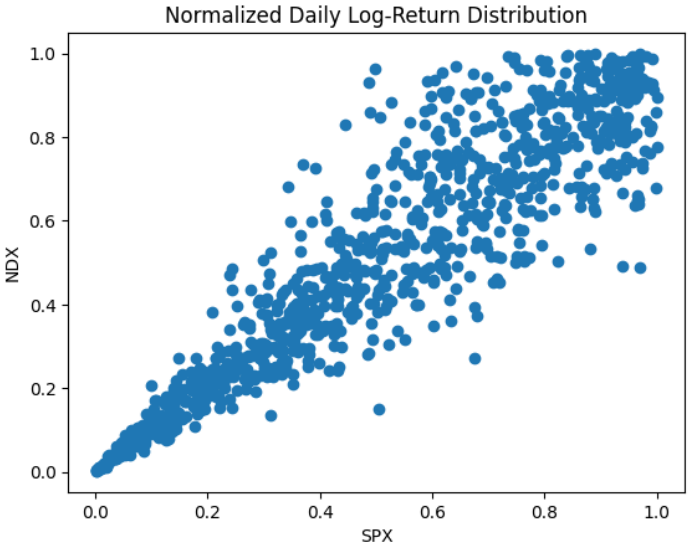
Since the algorithm is trading on options as speculation tools, to avoid insufficient buying power limited at the algorithm level, use `NullBuyingPowerModel` to bypass that.

```
PY
def initialize(self) -> None:
    # Request spx and ndx data for option pricing and selection.
    self.spx = self.add_index("SPX", Resolution.MINUTE).symbol
    self.ndx = self.add_index("NDX", Resolution.MINUTE).symbol

    self.set_security_initializer(self.custom_security_initializer)

def custom_security_initializer(self, security: Security) -> None:
    # Seed the last price for the initial filtering on option contracts using the price data.
    security.set_market_price(self.get_last_known_price(security))
    # Allow infinite margin to start the option positions.
    security.set_buying_power_model(NullBuyingPowerModel())
```

Build Models

In this example, build a copula model to estimate the log-return bivariate distribution of the paired securities, such that it can predict the correlated future price movement more accurately.

The following image shows the 1-year daily log-return bivariate distribution (normalized) of SPX and NDX indices as an example. As shown in the plot, although they exhibit a certain level of correlation, the tail is fat in both side, making normal distribution assumption unsuitable. In such case, we can make use of distributions from the extreme value theorem (Frank, Gumbel, or Clayton).



Follow the below steps to build the model:

1. Create a helper class to handle the copula trading.

```
class CopulaTrader:
    copula = None
    copula_name = None
    security1_scaler = MinMaxScaler()
    security2_scaler = MinMaxScaler()
    traded_pairs = []

    def __init__(self, algorithm: QCAlgorithm, security1: Symbol, security2: Symbol, alpha: float = 0.05, misprice_threshold: float
= 2.5) -> None:
        self._algorithm = algorithm
        # Interest rate and dividend yield providers to calculate fair option price later.
        self.interest_rate_provider = algorithm.risk_free_interest_rate_model
        self.security1_dividend_yield_provider = DividendYieldProvider(security1)
        self.security2_dividend_yield_provider = DividendYieldProvider(security2)
        # The significance level on KS test of the copula goodness-of-fit
        self.alpha = alpha
        # The threshold of the price divergence of a mispriced pair to have in order to start the positions.
        # Should at least over the transaction fee with some margin.
        self.misprice_threshold = misprice_threshold
```

2. Create a method to transform the raw data into daily log-return, such that the series is more stationary while capturing sufficient variance to accurately estimate the bivariate distribution.

```
class CopulaTrader:
    copula = None
    copula_name = None
    security1_scaler = MinMaxScaler()
    security2_scaler = MinMaxScaler()

    def _data_preparation(self, security1_prices: np.ndarray, security2_prices: np.ndarray) -> np.ndarray:
        # Calculate log price difference for fitting copulas, since the series has more stationary domain for prediction.
        security1_log_price = np.log(security1_prices)
        security2_log_price = np.log(security2_prices)
        security1_log_diff = np.diff(security1_log_price)
        security2_log_diff = np.diff(security2_log_price)
        # Scale to [0, 1] to fit the model.
        scaled_security1_data = self.security1_scaler.fit_transform(security1_log_diff.reshape(-1, 1))
        scaled_security2_data = self.security2_scaler.fit_transform(security2_log_diff.reshape(-1, 1))
        data = np.column_stack((scaled_security1_data, scaled_security2_data))
        return data
```

3. Create a method to fit the copula with the distribution yielding the best log-likelihood.

```
def _fit_bivariate_copula(self, data: np.ndarray) -> None:
    # Select the best-fitted bivariate copula by log-likelihood.
    self.copula = Bivariate.select_copula(data)
    self.copula_name = self.copula.copula_type
```

4. (Optional) Reconfirm it by a 2-sample KS test.

```py
def _fit_bivariate_copula(self, data: np.ndarray) -> None:
    # Continue from above

    # KS test to check if the actual data fits to the CDF of the copula good.
    samples = self.copula.sample(data.shape[0])
    p_value = self._ks2d2s(data[:, 0], samples[:, 0], data[:, 1], samples[:, 1])
    # Null hypothesis is the 2 datasets are having the same distribution, so if p < alpha, we reject the copula.
    if p_value < self.alpha:
        self.copula, self.copula_name = None, None
        self._algorithm.debug("No copula was fitted with statistically significant confidence.")
        return

    self._algorithm.debug(f"Best fitted copula distribution: {self.copula_name}, with tau: {self.copula.tau}")

def _ks2d2s(self, x1: np.ndarray, y1: np.ndarray, x2: np.ndarray, y2: np.ndarray) -> float:
    '''Two-dimensional Kolmogorov-Smirnov test on two samples.
    Adapted from: https://github.com/syrte/ndtest/blob/master/ndtest.py'''
    def quadct(x, y, xx, yy):
        n = len(xx)
        ix1, ix2 = xx <= x, yy <= y
        a = np.sum(ix1 & ix2) / n
        b = np.sum(ix1 & ~ix2) / n
        c = np.sum(~ix1 & ix2) / n
        d = 1 - a - b - c
        return a, b, c, d

    def maxdist(x1, y1, x2, y2):
        n1 = len(x1)
        D1 = np.empty((n1, 4))
        for i in range(n1):
            a1, b1, c1, d1 = quadct(x1[i], y1[i], x1, y1)
            a2, b2, c2, d2 = quadct(x1[i], y1[i], x2, y2)
            D1[i] = [a1 - a2, b1 - b2, c1 - c2, d1 - d2]

        D1[:, 0] -= 1 / n1

        dmin, dmax = -D1.min(), D1.max() + 1 / n1
        return max(dmin, dmax)

    def avgmaxdist(x1, y1, x2, y2):
        D1 = maxdist(x1, y1, x2, y2)
        D2 = maxdist(x2, y2, x1, y1)
        return (D1 + D2) / 2

    n1, n2 = len(x1), len(x2)
    D = avgmaxdist(x1, y1, x2, y2)

    sqen = np.sqrt(n1 * n2 / (n1 + n2))
    r1 = stats.pearsonr(x1, y1)[0]
    r2 = stats.pearsonr(x2, y2)[0]
    r = np.sqrt(1 - 0.5 * (r1**2 + r2**2))
    d = D * sqen / (1 + r * (0.25 - 0.75 / sqen))
    p = stats.kstwobign.sf(d)

    return p
```

5. Instantiate the helper class and fit the copula model.

```py
class CopulaTrader:
    def fit_bivariate_copula(self, security1_prices: np.ndarray, security2_prices: np.ndarray) -> None:
        training_data = self._data_preparation(security1_prices, security2_prices)
        self._fit_bivariate_copula(training_data)

class CopulaOptionTradingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # Set up the copula trading helper class.
        alpha = self.get_parameter("alpha", 0.05)
        misprice_threshold = self.get_parameter("misprice_threshold", 5)
        self.n_samples = self.get_parameter("n_samples", 1000)
        self.copula_trader = CopulaTrader(self, self.spx, self.ndx, alpha, misprice_threshold)

        # Fit the copula.
        # spx_prices & ndx_prices will be discussed later. They are 2 equal-size np array containing the raw price data of SPX and
NDX.
        self.copula_trader.fit_bivariate_copula(spx_prices, ndx_prices)
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a

history request .

```
def initialize(self) -> None:
    # Create a rolling window to save the spx and ndx price for fitting the copula on their relationship.
    # We use the recent 1 year to fit the model.
    self.windows = {self.spx: RollingWindow(252), self.ndx: RollingWindow(252)}
    # Warm up the rolling windows with historical data for its immediate use.
    for bar in self.history[TradeBar](self.spx, 252, Resolution.DAILY):
        self.windows[self.spx].add(bar.close)
    for bar in self.history[TradeBar](self.ndx, 252, Resolution.DAILY):
        self.windows[self.ndx].add(bar.close)
```

PY

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
def fit_copula(self) -> None:
    # Obtain the historical price series of the spx and ndx for fitting the copula.
    # Note that we need to invert the rolling window series.
    spx_prices = np.array(list(self.windows[self.spx]))[::-1]
    ndx_prices = np.array(list(self.windows[self.ndx]))[::-1]
    # Fit the copula.
    self.copula_trader.fit_bivariate_copula(spx_prices, ndx_prices)
```

PY

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Fit the copula model for initial usage.
self.train(self.fit_copula)
```

PY

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Recalibrate the copula on every month to keep its accuracy updated.
self.train(self.date_rules.month_start(), self.time_rules.at(8,0), self.fit_copula)
```

PY

Update Training Data

To update the training data as the algorithm executes, make use of a consolidator to automatically update the rolling windows when a daily bar is consolidated through a consolidator handler.

```
def initialize(self) -> None:
    # Set a daily consolidator to update the rolling windows automatically.
    self.consolidate(self.spx, Resolution.DAILY, lambda bar: self.windows[bar.symbol].add(bar.close))
    self.consolidate(self.ndx, Resolution.DAILY, lambda bar: self.windows[bar.symbol].add(bar.close))
```

PY

Trading

In this example, we trade option mispricing arbitration using the copula model. More precisely, we are using the copula model to generate forward price series samples to price the options and do arbitration between options of the 2 correlated underlying. Since the copula can provide a distribution more accurately, we have an edge on the option pricing information.

Follow the below steps to trade the above logic:

1. Create a method to generate the forward price series samples of the 2 underlyings.

```py
class CopulaTrader:
    def _get_simulated_paths(self, current_security1_price: float, current_security2_price: float, expiry: datetime, current:
datetime, n_samples: int) -> tuple[float, float]:
        # Get the year till expiry.
        dtm = (expiry - current).total_seconds() / 60 / 60 / 24

        future_log_prices = np.log(np.array([[[current_security1_price, current_security2_price]] * n_samples]))
        # Convert to estimated trading days left.
        for _ in range(int(dtm / 252 * 365)):
            # The future price is simulated from N stochastic simulations.
            future_log_diff = self.copula.sample(n_samples)
            # Convert the daily log-difference into future price.
            security1_log_diff = self.security1_scaler.inverse_transform(future_log_diff[:, 0].reshape(-1, 1))
            security2_log_diff = self.security2_scaler.inverse_transform(future_log_diff[:, 1].reshape(-1, 1))
            log_diff = np.column_stack((security1_log_diff, security2_log_diff))
            future_log_prices = np.concatenate([future_log_prices, np.array([future_log_prices[-1, :, :] + log_diff])], axis=0)

        self.paths = np.exp(future_log_prices)
        # Return the ultimate price at the last iteration, convert back to price from log-price.
        return self.paths[-1], dtm
```

2. Create a method to price the options through averaging the payoff over all simulated paths.

```py
class CopulaTrader:
    def _calculate_fair_option_price(self, future_prices: np.ndarray, ttm: float, strike: float, right: OptionRight, interest_rate:
float, dividend_yield: float, security2: bool = False) -> float:
        select_col = 1 if security2 else 0
        future_prices = future_prices[:, select_col]

        # Average of option payoffs among simulation paths. Make use of vectorize calculation here instead of util method.
        option_prices = np.maximum(future_prices - strike, 0) if right == OptionRight.CALL else np.maximum(strike - future_prices,
0)
        # Adjust the continuous dividend payoff yield and interest rate.
        return np.exp((dividend_yield - interest_rate) * ttm) * np.mean(option_prices)
```

3. Create a method to calculate the probability of winning (price converges to our model).

   We use the ratio of paths that the underlyings ever reach the strikes of the options, since ATM options are most efficiently priced and equal among all option pricing engine.

```py
class CopulaTrader:
    def _ratio_reached(self, data: np.ndarray, point: np.ndarray) -> float:
        # Count how many paths ever reached the point.
        count = 0
        for i in range(data.shape[1]):
            if point[0] <= max(data[:, i, 0]) and point[1] <= max(data[:, i, 1]) and point[0] >= min(data[:, i, 0]) and point[1] >=
min(data[:, i, 1]):
                count += 1
        # Calculate the ratio of paths that the point has ever been reached.
        return count / data.shape[1]
```

4. Create a method to calculate the aboves and output the mispriced option pairs.

```py
class CopulaTrader:
    def scan_option_mispricing_combinations(self, security1_options: List[OptionContract], security2_options: List[OptionContract],
current_security1_price: float, current_security2_price: float,
        current: datetime, n_samples: int = 1000) -> List[tuple[OptionContract, OptionContract, float, float]]:
        # Get the simluated price series till the expiry. Note that you should use single expiry for all contracts for easier
handling and stable theta.
        future_prices, dtm = self._get_simulated_paths(current_security1_price, current_security2_price,
security1_options[0].expiry, current, n_samples)

        mispriced_pairs = []
        interest_rate = self.interest_rate_provider.get_interest_rate(current)
        security1_dividend_yield = self.security1_dividend_yield_provider.get_dividend_yield(current)
        security2_dividend_yield = self.security2_dividend_yield_provider.get_dividend_yield(current)

        # Separate call and put options to avoid mixed right pairs, in order to keep theta close to 0.
        for right in [OptionRight.CALL, OptionRight.PUT]:
            sorted_security1_options = sorted([contract for contract in security1_options if contract.right == right], key=lambda x:
x.strike)
            sorted_security2_options = sorted([contract for contract in security2_options if contract.right == right], key=lambda x:
x.strike)
            ttm = dtm / 365

            # Calculate the security1-security2 fair prices and spot any significant mispricing.
            for i, security1_option in enumerate(sorted_security1_options):
                for j, security2_option in enumerate(sorted_security2_options):
                    # Skip the pairs already being traded.
                    if (security1_option.symbol, security2_option.symbol) in [(y.symbol for y in x[0]) for x in self.traded_pairs]:
                        continue

                    # Calculate the probability of price converges. Since the ATM options are the most efficiently priced, we use
that as a proxy for our win rate.
                    # The probability will be based on the whether the simulated paths have ever reached the point (so it was ever
ATM).
                    prob = self._ratio_reached(self.paths, [security1_option.strike, security2_option.strike])

                    security1_price = self._calculate_fair_option_price(future_prices, ttm, security1_option.strike, right,
interest_rate, security1_dividend_yield, False)
                    security2_price = self._calculate_fair_option_price(future_prices, ttm, security2_option.strike, right,
interest_rate, security2_dividend_yield, True)
                    # Add to trade stacks only if the mispricing is opposite in direction and above profit threshold.
                    security1_price_divergence = security1_price - security1_option.ask_price
                    security2_price_divergence = security2_price - security2_option.bid_price
                    price_divergence1 = security1_price_divergence - security2_price_divergence
                    price_divergence2 = security1_price - security1_option.bid_price - security2_price + security2_option.ask_price
                    price_divergence = price_divergence1 if abs(price_divergence1) > abs(price_divergence2) else price_divergence1
                    if abs(price_divergence) > self.misprice_threshold and np.sign(security1_price_divergence) !=
np.sign(security2_price_divergence):
                        mispriced_pairs.append((security1_option, security2_option, price_divergence, prob))

        self._algorithm.debug(f"Found {len(mispriced_pairs)} mispriced option contract pairs.")

        return mispriced_pairs
```

5. Create a method to get the optimal position sizings for the top-expected-return trading opportunities.

```py
class CopulaTrader:
    def trade_options(self, mispriced_pairs: List[tuple[OptionContract, OptionContract, float, float]], fund: float,
contract_multiplier: int = 100) -> dict[OptionContract, float]:
        orders = {}
        total_pct = 0

        # Only trade on the top 10 expected return pairs.
        top_expected_ret = sorted(mispriced_pairs, key=lambda x: abs(x[2]) * x[3], reverse=True)[:10]
        for security1_option, security2_option, price_divergence, prob in top_expected_ret:
            # Calculate the order size of the mispricing pairs.
            contract1_price = security1_option.ask_price if price_divergence > 0 else security1_option.bid_price
            contract2_price = security2_option.ask_price if price_divergence < 0 else security2_option.bid_price
            security1_option_order_size, security2_option_order_size, order_pct = self._calculate_order_size(price_divergence, fund,
prob, contract1_price, contract2_price, contract_multiplier)
            # Only order those with positive Kelly sizing and keep the exposure to cash only.
            # Keep 5% buffer to avoid order errors.
            if order_pct and total_pct + order_pct <= 0.95:
                if security1_option not in orders:
                    orders[security1_option] = 0
                if security2_option not in orders:
                    orders[security2_option] = 0
            # Aggregate the overall order sizes between all pairs.
                orders[security1_option] += security1_option_order_size
                orders[security2_option] += security2_option_order_size
                total_pct += order_pct
                self.traded_pairs.append(((security1_option, security2_option), (security1_option_order_size,
security2_option_order_size)))

        return orders

    def _calculate_order_size(self, price_divergence: float, fund: float, prob_win: float, security1_option_price: float,
security2_option_price: float, contract_multiplier: int) -> tuple[float, float, float]:
        # The profit percentage is based on the option prices.
        capital = security1_option_price + security2_option_price
        profit_pct = abs(price_divergence) / capital
        # Assume no recovery at expiry if the price does not converges, while no extra loss will be paid.
        # To improve, you may use a continuous distribution to model the aggregated profit.
        # Keep at most 5% per trade, since option has leverage effect.
        order_pct = min(prob_win - (1 - prob_win) / profit_pct, 0.05)
        # The order size should be the shared by both side of the legs.
        order_size = order_pct * fund / capital // contract_multiplier
        order_pct = order_size * capital / fund
        # If the optimal order size is negative, do not place order.
        if order_pct < 0:
            return 0, 0, 0
        # If price divergence above 0, meaning security1 option is underpriced and the security2 option is overpriced, so we buy the
security1 option and sell the security2 option.
        if price_divergence > 0:
            return order_size, -order_size, order_pct
        # Vice versa otherwise.
        return -order_size, order_size, order_pct
```

6. Create a method to handle the portfolio and exit the positions when price converges.

```py
class CopulaTrader:
    def exit_trades(self, current: datetime) -> dict[OptionContract, float]:
        # Get the parameters for fair price calculation.
        interest_rate = self.interest_rate_provider.get_interest_rate(current)
        security1_dividend_yield = self.security1_dividend_yield_provider.get_dividend_yield(current)
        security2_dividend_yield = self.security2_dividend_yield_provider.get_dividend_yield(current)

        exit_positions = {}
        for ((contract1, contract2), (size1, size2)) in self.traded_pairs.copy():
            # Get the simluated price series till the expiry.
            future_prices = self.paths[-1]
            ttm = (contract1.expiry - current).total_seconds() / 60 / 60 / 24 / 365
            # Check if the price discrepancy converges (relatively).
            contract1_price = self._calculate_fair_option_price(future_prices, ttm, contract1.strike, contract1.right,
interest_rate, security1_dividend_yield, False)
            contract2_price = self._calculate_fair_option_price(future_prices, ttm, contract2.strike, contract2.right,
interest_rate, security2_dividend_yield, True)
            contract1.mid_price = (contract1.bid_price + contract1.ask_price) * 0.5
            contract2.mid_price = (contract2.bid_price + contract2.ask_price) * 0.5
            contract1_divergence = contract1_price - contract1.mid_price
            contract2_divergence = contract2_price - contract2.mid_price
            price_divergence = contract1_divergence - contract2_divergence

            # If so, place the opposite size to exit the positions.
            if (size1 > 0 and price_divergence <= 0) or (size1 < 0 and price_divergence >= 0):
                if contract1 not in exit_positions:
                    exit_positions[contract1] = 0
                if contract2 not in exit_positions:
                    exit_positions[contract2] = 0
                exit_positions[contract1] -= size1
                exit_positions[contract2] -= size2
                # Remove the record and allow the pair to be traded again.
                self.traded_pairs.remove(((contract1, contract2), (size1, size2)))

        return exit_positions
```

7. Create a method in the base algorithm to open trades. Schedule its run.

   To simply implementation, this example will only open trades after the previous positions are all expired, but you can change it to scan by some time interval.

```py
class CopulaOptionTradingAlgorithm(QCAlgorithm):
    def open_trades(self) -> None:

        # Check if copula was fitted
        if self.copula_trader.copula is None:
            return

        # Obtain the option chain of the spx and the ndx for trading.
        spx_options = self.option_chain(self.spx)
        ndx_options = self.option_chain(self.ndx)

        # Only use European options since the computation of averaging option payoff to price is more efficient.
        # Only include the ones within 5% of the current price, since they are more liquid and less susceptible from slippage.
        spx_price = self.securities[self.spx].price
        ndx_price = self.securities[self.ndx].price

        european_spx_options = [contract for contract in spx_options if contract.style == OptionStyle.EUROPEAN and spx_price * 1.05
> contract.strike > spx_price * 0.95]
        european_ndx_options = [contract for contract in ndx_options if contract.style == OptionStyle.EUROPEAN and ndx_price * 1.05
> contract.strike > ndx_price * 0.95]

        if len(european_spx_options) == 0 or len(european_ndx_options) == 0:
            return

        # Get the nearest expiry after 7 days to trade them. We use a single expiry to trade with more stable theta.
        spx_expiries = set(contract.expiry for contract in european_spx_options if contract.expiry > self.time + timedelta(7))
        ndx_expiries = set(contract.expiry for contract in european_ndx_options if contract.expiry > self.time + timedelta(7))
        common_expiries = spx_expiries.intersection(ndx_expiries)
        if len(common_expiries) == 0:
            return
        expiry = min(common_expiries)
        filtered_spx_options = [contract for contract in european_spx_options if contract.expiry == expiry]
        filtered_ndx_options = [contract for contract in european_ndx_options if contract.expiry == expiry]
        # Use simulations to price options by the fitted copula distributions to spot mispricing option pairs.
        mispriced_option_pairs = self.copula_trader.scan_option_mispricing_combinations(filtered_spx_options, filtered_ndx_options,
spx_price, ndx_price, self.time, self.n_samples)
        # Obtain the trading plan from the mispriced options, calculated using Kelly Criterion.
        trade_plan = self.copula_trader.trade_options(mispriced_option_pairs, self.portfolio.cash, 100)
        # Order both plans. Request the option contract data to trade it.
        for contract, size in trade_plan.items():
            symbol = self.add_option_contract(contract).symbol
            self.market_order(symbol, size)

        # Schedule the next option trades to be after the current selected expiry to simplify path calculation.
        next_trading_day = self.securities[self.spx].exchange.hours.get_next_trading_day(expiry)
        self.schedule.on(
            self.date_rules.on(next_trading_day),
            self.time_rules.after_market_open(self.spx, 1),
            self.open_trades
        )
```

8. Scan every N minutes to exit the positions if the price discrepancy between a pair is converged.

```py
class CopulaOptionTradingAlgorithm(QCAlgorithm):
    def on_data(self, slice: Slice) -> None:
        # Scan every 5 minutes to check price convergence.
        if slice.time.minute % 5 == 0:
            exit_plan = self.copula_trader.exit_trades(self.time)
            # Exit the paired positions when the price converges.
            for contract, size in exit_plan.items():
                self.market_order(contract.symbol, size)
```

Save Models

Follow these steps to save `copula` models into the Object Store :

1. Set the key name you want to store the model under in the Object Store.

```py
# Set the storage key to something representative.
model_key = f"{self.project_id}/copula"
```

2. Call the `get_file_path` method with the key.

```
# Get the file path to save and access the model in the Object Store.
file_name = self.object_store.get_file_path(model_key)
```

This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

```
# Serialize and save the model.
joblib.dump(self.predict, self.copula_trader.copula)
```

If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/copula <pathTo>/copula
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `copula` models that you saved in the Object Store. To load a `copula` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the trained Aesera model from the Object Store.
def initialize(self) -> None:
    model_key = f"{self.project_id}/copula"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.copula_trader.copula = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The below demonstrates the full algorithm of the above example.

```
from AlgorithmImports import *
from copulas.bivariate import Bivariate
from scipy import stats
from sklearn.preprocessing import MinMaxScaler

class CopulaOptionTradingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        np.random.seed(42)
        self.set_start_date(2024, 12, 3)
        self.set_end_date(2024, 12, 4)
        self.set_cash(100000000)
        self.settings.seed_initial_prices = True
        self.add_security_initializer(self.custom_security_initializer)

        # Request spx and ndx data for option pricing and selection.
        self.spx = self.add_index("SPX", Resolution.MINUTE).symbol
```

```
        self.ndx = self.add_index("NDX", Resolution.MINUTE).symbol

        # Set up the copula trading helper class.
        alpha = self.get_parameter("alpha", 0.05)
        misprice_threshold = self.get_parameter("misprice_threshold", 5)
        self.n_samples = self.get_parameter("n_samples", 1000)
        self.copula_trader = CopulaTrader(self, self.spx, self.ndx, alpha, misprice_threshold)

        # Set warm-up period to have enough historical data for fitting the copula.
        lookback = 252
        self.securities[self.spx].session.size = lookback
        self.securities[self.ndx].session.size = lookback
        self.set_warm_up(lookback, Resolution.DAILY)

    def on_warmup_finished(self):
        # Fit the copula model for initial usage.
        self.train(self.fit_copula)
        # Recalibrate the copula on every month to keep its accuracy updated.
        self.train(self.date_rules.month_start(), self.time_rules.at(8,0), self.fit_copula)

        # Instantiate the next trade on the next market open.
        next_trading_day = self.securities[self.spx].exchange.hours.get_next_trading_day(self.time)
        self.schedule.on(
            self.date_rules.on(next_trading_day),
            self.time_rules.after_market_open(self.spx, 1),
            self.open_trades
        )

    def custom_security_initializer(self, security: Security) -> None:
        # Allow infinite margin to start the option positions.
        security.set_buying_power_model(NullBuyingPowerModel())

    def fit_copula(self) -> None:
        # Obtain the historical price series of the spx and ndx for fitting the copula.
        # Note that we need to invert the session series.
        spx_prices = np.array([x.close for x in self.securities[self.spx].session][::-1])
        ndx_prices = np.array([x.close for x in self.securities[self.ndx].session][::-1])
        # Fit the copula.
        self.copula_trader.fit_bivariate_copula(spx_prices, ndx_prices)

    def open_trades(self) -> None:

        # Check if copula was fitted
        if self.copula_trader.copula is None:
            return

        # Obtain the option chain of the spx and the ndx for trading.
        spx_options = self.option_chain(self.spx)
        ndx_options = self.option_chain(self.ndx)

        # Only use European options since the computation of averaging option payoff to price is more efficient.
        # Only include the ones within 5% of the current price, since they are more liquid and less susceptible from slippage.
        spx_price = self.securities[self.spx].price
        ndx_price = self.securities[self.ndx].price

        european_spx_options = [contract for contract in spx_options if contract.style == OptionStyle.EUROPEAN and spx_price * 1.05 >
contract.strike > spx_price * 0.95]
        european_ndx_options = [contract for contract in ndx_options if contract.style == OptionStyle.EUROPEAN and ndx_price * 1.05 >
contract.strike > ndx_price * 0.95]

        if len(european_spx_options) == 0 or len(european_ndx_options) == 0:
            return

        # Get the nearest expiry after 7 days to trade them. We use a single expiry to trade with more stable theta.
        spx_expiries = set(contract.expiry for contract in european_spx_options if contract.expiry > self.time + timedelta(7))
        ndx_expiries = set(contract.expiry for contract in european_ndx_options if contract.expiry > self.time + timedelta(7))
        common_expiries = spx_expiries.intersection(ndx_expiries)
        if len(common_expiries) == 0:
            return
        expiry = min(common_expiries)
        filtered_spx_options = [contract for contract in european_spx_options if contract.expiry == expiry]
        filtered_ndx_options = [contract for contract in european_ndx_options if contract.expiry == expiry]
        # Use simulations to price options by the fitted copula distributions to spot mispricing option pairs.
        mispriced_option_pairs = self.copula_trader.scan_option_mispricing_combinations(filtered_spx_options, filtered_ndx_options,
spx_price, ndx_price, self.time, self.n_samples)
        # Obtain the trading plan from the mispriced options, calculated using Kelly Criterion.
        trade_plan = self.copula_trader.trade_options(mispriced_option_pairs, self.portfolio.cash, 100)
        # Order both plans. Request the option contract data to trade it.
        for contract, size in trade_plan.items():
            symbol = self.add_option_contract(contract).symbol
            self.market_order(symbol, size)

        # Schedule the next option trades to be after the current selected expiry to simplify path calculation.
        next_trading_day = self.securities[self.spx].exchange.hours.get_next_trading_day(expiry)
        self.schedule.on(
            self.date_rules.on(next_trading_day),
            self.time_rules.after_market_open(self.spx, 1),
            self.open_trades
        )

    def on_data(self, slice: Slice) -> None:
        # Scan every 5 minutes to check price convergence.
        if slice.time.minute % 5 == 0:
            exit_plan = self.copula_trader.exit_trades(self.time)
```

```
            exit_plan = self.copula_trader.exit_trades(self.time)
            # Exit the paired positions when the price converges.
            for contract, size in exit_plan.items():
                self.market_order(contract.symbol, size)

class CopulaTrader:
    copula = None
    copula_name = None
    security1_scaler = MinMaxScaler()
    security2_scaler = MinMaxScaler()
    traded_pairs = []

    def __init__(self, algorithm: QCAlgorithm, security1: Symbol, security2: Symbol, alpha: float = 0.05, misprice_threshold: float =
2.5) -> None:
        self._algorithm = algorithm
        # Interest rate and dividend yield providers to calculate fair option price later.
        self.interest_rate_provider = algorithm.risk_free_interest_rate_model
        self.security1_dividend_yield_provider = DividendYieldProvider(security1)
        self.security2_dividend_yield_provider = DividendYieldProvider(security2)
        # The significance level on KS test of the copula goodness-of-fit
        self.alpha = alpha
        # The threshold of the price divergence of a mispriced pair to have in order to start the positions.
        # Should at least over the transaction fee with some margin.
        self.misprice_threshold = misprice_threshold

    def fit_bivariate_copula(self, security1_prices: np.ndarray, security2_prices: np.ndarray) -> None:
        training_data = self._data_preparation(security1_prices, security2_prices)
        self._fit_bivariate_copula(training_data)

    def _data_preparation(self, security1_prices: np.ndarray, security2_prices: np.ndarray) -> np.ndarray:
        # Calculate log price difference for fitting copulas, since the series has more stationary domain for prediction.
        security1_log_diff = np.diff(np.log(security1_prices))[:-1]
        security2_log_diff = np.diff(np.log(security2_prices))[:-1]
        # Scale to [0, 1] to fit the model.
        scaled_security1_data = self.security1_scaler.fit_transform(security1_log_diff.reshape(-1, 1))
        scaled_security2_data = self.security2_scaler.fit_transform(security2_log_diff.reshape(-1, 1))
        return np.column_stack((scaled_security1_data, scaled_security2_data))

    def _fit_bivariate_copula(self, data: np.ndarray) -> None:
        # Select the best-fitted bivariate copula by log-likelihood.
        self.copula = Bivariate.select_copula(data)
        self.copula_name = self.copula.copula_type

        # KS test to check if the actual data fits to the CDF of the copula good.
        samples = self.copula.sample(data.shape[0])
        p_value = self._ks2d2s(data[:, 0], samples[:, 0], data[:, 1], samples[:, 1])
        # Null hypothesis is the 2 datasets are having the same distribution, so if p < alpha, we reject the copula.
        if p_value < self.alpha:
            self.copula, self.copula_name = None, None
            self._algorithm.debug("No copula was fitted with statistically significant confidence.")
            return

        self._algorithm.debug(f"Best fitted copula distribution: {self.copula_name}, with tau: {self.copula.tau}")

    def _ks2d2s(self, x1: np.ndarray, y1: np.ndarray, x2: np.ndarray, y2: np.ndarray) -> float:
        '''Two-dimensional Kolmogorov-Smirnov test on two samples.
        Adapted from: https://github.com/syrte/ndtest/blob/master/ndtest.py'''
        def quadct(x, y, xx, yy):
            n = len(xx)
            ix1, ix2 = xx <= x, yy <= y
            a = np.sum(ix1 & ix2) / n
            b = np.sum(ix1 & ~ix2) / n
            c = np.sum(~ix1 & ix2) / n
            d = 1 - a - b - c
            return a, b, c, d

        def maxdist(x1, y1, x2, y2):
            n1 = len(x1)
            D1 = np.empty((n1, 4))
            for i in range(n1):
                a1, b1, c1, d1 = quadct(x1[i], y1[i], x1, y1)
                a2, b2, c2, d2 = quadct(x1[i], y1[i], x2, y2)
                D1[i] = [a1 - a2, b1 - b2, c1 - c2, d1 - d2]

            D1[:, 0] -= 1 / n1

            dmin, dmax = -D1.min(), D1.max() + 1 / n1
            return max(dmin, dmax)

        def avgmaxdist(x1, y1, x2, y2):
            D1 = maxdist(x1, y1, x2, y2)
            D2 = maxdist(x2, y2, x1, y1)
            return (D1 + D2) / 2

        n1, n2 = len(x1), len(x2)
        D = avgmaxdist(x1, y1, x2, y2)

        sqen = np.sqrt(n1 * n2 / (n1 + n2))
        r1 = stats.pearsonr(x1, y1)[0]
        r2 = stats.pearsonr(x2, y2)[0]
        r = np.sqrt(1 - 0.5 * (r1**2 + r2**2))
        d = D * sqen / (1 + r * (0.25 - 0.75 / sqen))
        p = stats.kstwobign.sf(d)
```

```
        return p

    def scan_option_mispricing_combinations(self, security1_options: List[OptionContract], security2_options: List[OptionContract],
current_security1_price: float, current_security2_price: float,
        current: datetime, n_samples: int = 1000) -> List[tuple[OptionContract, OptionContract, float, float]]:
        # Get the simluated price series till the expiry. Note that you should use single expiry for all contracts for easier handling
and stable theta.
        future_prices, dtm = self._get_simulated_paths(current_security1_price, current_security2_price, security1_options[0].expiry,
current, n_samples)

        mispriced_pairs = []
        interest_rate = self.interest_rate_provider.get_interest_rate(current)
        security1_dividend_yield = self.security1_dividend_yield_provider.get_dividend_yield(current)
        security2_dividend_yield = self.security2_dividend_yield_provider.get_dividend_yield(current)
        # Separate call and put options to avoid mixed right pairs, in order to keep theta close to 0.
        pairs_checked = 0
        for right in [OptionRight.CALL, OptionRight.PUT]:
            sorted_security1_options = sorted([contract for contract in security1_options if contract.right == right], key=lambda x:
x.strike)
            sorted_security2_options = sorted([contract for contract in security2_options if contract.right == right], key=lambda x:
x.strike)
            ttm = dtm / 365

            # Calculate the security1-security2 fair prices and spot any significant mispricing.
            for i, security1_option in enumerate(sorted_security1_options):
                for j, security2_option in enumerate(sorted_security2_options):
                    # Skip the pairs already being traded.
                    if (security1_option.symbol, security2_option.symbol) in [(y.symbol for y in x[0]) for x in self.traded_pairs]:
                        continue

                    # Calculate the probability of price converges. Since the ATM options are the most efficiently priced, we use that as
a proxy for our win rate.
                    # The probability will be based on the whether the simulated paths have ever reached the point (so it was ever ATM).
                    prob = self._ratio_reached(self.paths, [security1_option.strike, security2_option.strike])

                    security1_price = self._calculate_fair_option_price(future_prices, ttm, security1_option.strike, right,
interest_rate, security1_dividend_yield, False)
                    security2_price = self._calculate_fair_option_price(future_prices, ttm, security2_option.strike, right,
interest_rate, security2_dividend_yield, True)
                    # Add to trade stacks only if the mispricing is opposite in direction and above profit threshold.
                    security1_price_divergence = security1_price - security1_option.ask_price
                    security2_price_divergence = security2_price - security2_option.bid_price
                    price_divergence1 = security1_price_divergence - security2_price_divergence
                    price_divergence2 = security1_price - security1_option.bid_price - security2_price + security2_option.ask_price
                    price_divergence = price_divergence1 if abs(price_divergence1) > abs(price_divergence2) else price_divergence1
                    pairs_checked += 1
                    if abs(price_divergence) > self.misprice_threshold and np.sign(security1_price_divergence) !=
np.sign(security2_price_divergence):
                        mispriced_pairs.append((security1_option, security2_option, price_divergence, prob))

        self._algorithm.debug(f"Found {len(mispriced_pairs)} mispriced option contract pairs.")

        return mispriced_pairs

    def _get_simulated_paths(self, current_security1_price: float, current_security2_price: float, expiry: datetime, current: datetime,
n_samples: int) -> tuple[float, float]:
        # Get the year till expiry.
        dtm = (expiry - current).total_seconds() / 60 / 60 / 24

        future_log_prices = np.log(np.array([[[current_security1_price, current_security2_price]] * n_samples]))
        # Convert to estimated trading days left.
        for _ in range(int(dtm / 252 * 365)):
            # The future price is simulated from N stochastic simulations.
            future_log_diff = self.copula.sample(n_samples)
            # Convert the daily log-difference into future price.
            security1_log_diff = self.security1_scaler.inverse_transform(future_log_diff[:, 0].reshape(-1, 1))
            security2_log_diff = self.security2_scaler.inverse_transform(future_log_diff[:, 1].reshape(-1, 1))
            log_diff = np.column_stack((security1_log_diff, security2_log_diff))
            future_log_prices = np.concatenate([future_log_prices, np.array([future_log_prices[-1, :, :] + log_diff])], axis=0)

        self.paths = np.exp(future_log_prices)
        # Return the ultimate price at the last iteration, convert back to price from log-price.
        return self.paths[-1], dtm

    def _calculate_fair_option_price(self, future_prices: np.ndarray, ttm: float, strike: float, right: OptionRight, interest_rate:
float, dividend_yield: float, security2: bool = False) -> float:
        select_col = 1 if security2 else 0
        future_prices = future_prices[:, select_col]

        # Average of option payoffs among simulation paths. Make use of vectorize calculation here instead of util method.
        option_prices = np.maximum(future_prices - strike, 0) if right == OptionRight.CALL else np.maximum(strike - future_prices, 0)
        # Adjust the continuous dividend payoff yield and interest rate.
        return np.exp((dividend_yield - interest_rate) * ttm) * np.mean(option_prices)

    def _ratio_reached(self, data: np.ndarray, point: np.ndarray) -> float:
        # Count how many paths ever reached the point.
        count = 0
        for i in range(data.shape[1]):
            if point[0] <= max(data[:, i, 0]) and point[1] <= max(data[:, i, 1]) and point[0] >= min(data[:, i, 0]) and point[1] >=
min(data[:, i, 1]):
                count += 1
        # Calculate the ratio of paths that the point has ever been reached.
        return count / data.shape[1]
```

```
    def trade_options(self, mispriced_pairs: List[tuple[OptionContract, OptionContract, float, float]], fund: float, contract_multiplier:
int = 100) -> dict[OptionContract, float]:
        orders = {}
        total_pct = 0

        # Only trade on the top 10 expected return pairs.
        top_expected_ret = sorted(mispriced_pairs, key=lambda x: abs(x[2]) * x[3], reverse=True)[:10]
        for security1_option, security2_option, price_divergence, prob in top_expected_ret:
            # Calculate the order size of the mispricing pairs.
            contract1_price = security1_option.ask_price if price_divergence > 0 else security1_option.bid_price
            contract2_price = security2_option.ask_price if price_divergence < 0 else security2_option.bid_price
            security1_option_order_size, security2_option_order_size, order_pct = self._calculate_order_size(price_divergence, fund,
prob, contract1_price, contract2_price, contract_multiplier)
            # Only order those with positive Kelly sizing and keep the exposure to cash only.
            # Keep 5% buffer to avoid order errors.
            if order_pct and total_pct + order_pct <= 0.95:
                if security1_option not in orders:
                    orders[security1_option] = 0
                if security2_option not in orders:
                    orders[security2_option] = 0
            # Aggregate the overall order sizes between all pairs.
                orders[security1_option] += security1_option_order_size
                orders[security2_option] += security2_option_order_size
                total_pct += order_pct
                self.traded_pairs.append(((security1_option, security2_option), (security1_option_order_size,
security2_option_order_size)))

        return orders

    def _calculate_order_size(self, price_divergence: float, fund: float, prob_win: float, security1_option_price: float,
security2_option_price: float, contract_multiplier: int) -> tuple[float, float, float]:
        # The profit percentage is based on the option prices.
        capital = security1_option_price + security2_option_price
        profit_pct = abs(price_divergence) / capital
        # Assume no recovery at expiry if the price does not converges, while no extra loss will be paid.
        # To improve, you may use a continuous distribution to model the aggregated profit.
        # Keep at most 5% per trade, since option has leverage effect.
        order_pct = min(prob_win - (1 - prob_win) / profit_pct, 0.05)
        # The order size should be the shared by both side of the legs.
        order_size = order_pct * fund / capital // contract_multiplier
        order_pct = order_size * capital / fund
        # If the optimal order size is negative, do not place order.
        if order_pct < 0:
            return 0, 0, 0
        # If price divergence above 0, meaning security1 option is underpriced and the security2 option is overpriced, so we buy the
security1 option and sell the security2 option.
        if price_divergence > 0:
            return order_size, -order_size, order_pct
        # Vice versa otherwise.
        return -order_size, order_size, order_pct

    def exit_trades(self, current: datetime) -> dict[OptionContract, float]:
        # Get the parameters for fair price calculation.
        interest_rate = self.interest_rate_provider.get_interest_rate(current)
        security1_dividend_yield = self.security1_dividend_yield_provider.get_dividend_yield(current)
        security2_dividend_yield = self.security2_dividend_yield_provider.get_dividend_yield(current)

        exit_positions = {}
        for ((contract1, contract2), (size1, size2)) in self.traded_pairs.copy():
            # Get the simluated price series till the expiry.
            future_prices = self.paths[-1]
            ttm = (contract1.expiry - current).total_seconds() / 60 / 60 / 24 / 365
            # Check if the price discrepancy converges (relatively).
            contract1_price = self._calculate_fair_option_price(future_prices, ttm, contract1.strike, contract1.right, interest_rate,
security1_dividend_yield, False)
            contract2_price = self._calculate_fair_option_price(future_prices, ttm, contract2.strike, contract2.right, interest_rate,
security2_dividend_yield, True)
            contract1.mid_price = (contract1.bid_price + contract1.ask_price) * 0.5
            contract2.mid_price = (contract2.bid_price + contract2.ask_price) * 0.5
            contract1_divergence = contract1_price - contract1.mid_price
            contract2_divergence = contract2_price - contract2.mid_price
            price_divergence = contract1_divergence - contract2_divergence

            # If so, place the opposite size to exit the positions.
            if (size1 > 0 and price_divergence <= 0) or (size1 < 0 and price_divergence >= 0):
                if contract1 not in exit_positions:
                    exit_positions[contract1] = 0
                if contract2 not in exit_positions:
                    exit_positions[contract2] = 0
                exit_positions[contract1] -= size1
                exit_positions[contract2] -= size2
                # Remove the record and allow the pair to be traded again.
                self.traded_pairs.remove(((contract1, contract2), (size1, size2)))

        return exit_positions
```

Popular Libraries

GPlearn

Introduction

This page explains how to build, train, deploy and store `GPlearn` models.

To access GPlearn, use the Legacy package environment. To set this environment, see the documentation in Cloud Platform or Local Platform .

Import Libraries

Import the `gplearn` and `joblib` libraries.

```
PY
from AlgorithmImports import *
from gplearn.genetic import SymbolicRegressor, SymbolicTransformer
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `GPLearn` model and make predictions.

```
PY
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a genetic programming feature transformation model and a genetic programming regression prediction model using the following features and labels:

Data Category	Description
Features	Daily percent change of the close price of the SPY over the last 5 days
Labels	Daily percent return of the SPY over the next day

The following image shows the time difference between the features and labels:

Follow these steps to create a method to build the model:

1. Declare a set of functions to use for feature engineering.

```
# Pick some functions for feature engineering to explore transformations of the data.
function_set = ['add', 'sub', 'mul', 'div',
                'sqrt', 'log', 'abs', 'neg', 'inv',
                'max', 'min']
```

2. Call the `SymbolicTransformer` constructor with the preceding set of functions and then save it as a class variable.

```
# Initialize a SymbolicTransformer with a function set for feature engineering.
self.gp_transformer = SymbolicTransformer(function_set=function_set)
```

3. Call the `SymbolicRegressor` constructor to instantiate the regression model.

```
# Instantiate the regression model to perform symbolic regression, which involves
# mathematical expressions to fit the data and identify patterns.
self.model = SymbolicRegressor()
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of training data.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar.close)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare the feature and label data for training.
def get_features_and_labels(self, n_steps=5):
    training_df = list(self.training_data)[::-1]
    daily_pct_change = ((np.roll(training_df, -1) - training_df) / training_df)[:-1]

    features = []
    labels = []
    for i in range(len(daily_pct_change)-n_steps):
        features.append(daily_pct_change[i:i+n_steps])
        labels.append(daily_pct_change[i+n_steps])
    features = np.array(features)
    labels = np.array(labels)

    return features, labels

def my_training_method(self):
    features, labels = self.get_features_and_labels()

    # Perform feature engineering.
    self.gp_transformer.fit(features, labels)
    gp_features = self.gp_transformer.transform(features)
    new_features = np.hstack((features, gp_features))

    # Fit the regression model with transformed and raw features.
    self.model.fit(new_features, labels)
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model right now.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM.
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current close price to the `RollingWindow` that holds the training data.

```
# Add the current close to the training data to ensure the model trains with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol].close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the feature set.
features, _ = self.get_features_and_labels()

# Transform the features.
gp_features = self.gp_transformer.transform(features)
new_features = np.hstack((features, gp_features))

# Make a prediction.
prediction = self.model.predict(new_features)
prediction = float(prediction.flatten()[-1])
```

You can use the label prediction to place orders.

```
# Place orders based on the prediction.
if prediction > 0:
    self.set_holdings(self._symbol, 1)
elif prediction < 0:
    self.set_holdings(self._symbol, -1)
```

Save Models

Follow these steps to save `GPLearn` models into the Object Store :

1. Set the key names you want to store the models under in the Object Store.

   ```
   # Set the storage keys to something representative.
   transformer_model_key = f"{self.project_id}/transformer"
   regressor_model_key = f"{self.project_id}/regressor"
   ```

2. Call the `get_file_path` method with the keys.

   ```
   # Get the file paths to save and access the models in the Object Store.
   transformer_file_name = self.object_store.get_file_path(transformer_model_key)
   regressor_file_name = self.object_store.get_file_path(regressor_model_key)
   ```

   This method returns the file paths where the models will be stored.

3. Call the `dump` method the file paths.

```
# Serialize and save the models.
joblib.dump(self.gp_transformer, transformer_file_name)
joblib.dump(self.model, regressor_file_name)
```

If you dump the models using the `joblib` module before you save the models, you don't need to retrain the models.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/transformer <pathTo>/transformer
```

```
$ lean cloud object-store set <projectId>/regressor <pathTo>/regressor
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `GPLearn` models that you saved in the Object Store. To load a `GPLearn` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the GPLearn model from Object Store to use its saved state and update with new data if needed.
def initialize(self) -> None:
    transformer_model_key = f"{self.project_id}/transformer"
    regressor_model_key = f"{self.project_id}/regressor"
    if self.object_store.contains_key(transformer_model_key) and self.object_store.contains_key(regressor_model_key):
        transformer_file_name = self.object_store.get_file_path(transformer_model_key)
        regressor_file_name = self.object_store.get_file_path(regressor_model_key)
        self.gp_transformer = joblib.load(transformer_file_name)
        self.model = joblib.load(regressor_file_name)
```

The `contains_key` method returns a boolean that represents if `transformer_model_key` and `regressor_model_key` are in the Object Store. If the Object Store does not contain the keys, save the model using them before you proceed.

Examples

The following examples demonstrate some common practices for using `GPLearn` library.

Example 1: Genetic Learning

The below algorithm makes use of `GPLearn` library to predict the future price movement using the previous 5 OHLCV data. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

```
from gplearn.genetic import SymbolicRegressor, SymbolicTransformer
import joblib

class GPLearnExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

```
        # 2-year data to train the model.
        training_length = 252*2
        self.training_data = RollingWindow(training_length)
        # Warm up the training dataset to train the model immediately.
        history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
        for trade_bar in history:
            self.training_data.add(trade_bar.close)

        # Retrieve already trained model from object store to use immediately.
        transformer_model_key = f"{self.project_id}/Transformer"
        regressor_model_key = f"{self.project_id}/Regressor"
        if self.object_store.contains_key(transformer_model_key) and self.object_store.contains_key(regressor_model_key):
            transformer_file_name = self.object_store.get_file_path(transformer_model_key)
            regressor_file_name = self.object_store.get_file_path(regressor_model_key)

            self.gp_transformer = joblib.load(transformer_file_name)
            self.model = joblib.load(regressor_file_name)

        else:
            # Exhasting list of the non-linear transformation to apply on the variables.
            function_set = ['add', 'sub', 'mul', 'div',
                'sqrt', 'log', 'abs', 'neg', 'inv',
                'max', 'min']
            # Create and train the model to use the prediction right away.
            self.gp_transformer = SymbolicTransformer(function_set=function_set)
            self.model = SymbolicRegressor()
            self.train(self.my_training_method)

        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> tuple[list, list]:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = list(self.training_data)[::-1]
        daily_pct_change = ((np.roll(training_df, -1) - training_df) / training_df)[:-1]

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(daily_pct_change)-n_steps):
            features.append(daily_pct_change[i:i+n_steps])
            labels.append(daily_pct_change[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()

        # Feature engineering
        self.gp_transformer.fit(features, labels)
        gp_features = self.gp_transformer.transform(features)
        new_features = np.hstack((features, gp_features))

        # Fit the regression model with transformed and raw features.
        self.model.fit(new_features, labels)

    def on_data(self, slice: Slice) -> None:
        features, _ = self.get_features_and_labels()

        # Get transformed features
        gp_features = self.gp_transformer.transform(features)
        new_features = np.hstack((features, gp_features))

        # Get prediction using the latest 5 OHLCV.
        prediction = self.model.predict(new_features)
        prediction = float(prediction.flatten()[-1])

        # If the predicted direction is going upward, buy SPY.
        if prediction > 0:
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction < 0:
            self.set_holdings(self._symbol, -1)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        transformer_model_key = f"{self.project_id}/Transformer"
        regressor_model_key = f"{self.project_id}/Regressor"
        transformer_file_name = self.object_store.get_file_path(transformer_model_key)
        regressor_file_name = self.object_store.get_file_path(regressor_model_key)
        joblib.dump(self.gp_transformer, transformer_file_name)
        joblib.dump(self.model, regressor_file_name)
        self.object_store.save(transformer_model_key)
        self.object_store.save(regressor_model_key)
```

Popular Libraries

Hmmlearn

Introduction

This page explains how to build, train, deploy and store `Hmmlearn` models.

Import Libraries

Import the `hmmlearn` and `joblib` libraries.

```
PY
from AlgorithmImports import *
from hmmlearn import hmm
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `hmmlearn` model and make predictions.

```
PY
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, assume the market has only 2 regimes and the market returns follow a Gaussian distribution. Therefore, create a 2-component Hidden Markov Model with Gaussian emissions, which is equivalent to a Gaussian mixture model with 2 means.

To build the model, call the `GaussianHMM` constructor with the number of components, a covariance type, and the number of iterations:

```
PY
# Initialize GaussianHMM with 2 states for capturing patterns, full covariance to model complex relationships, and
# 100 iterations for sufficient training.
self.model = hmm.GaussianHMM(n_components=2, covariance_type="full", n_iter=100)
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
PY
# Fill a RollingWindow with 2 years of historical closing prices.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar.close)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features(self):
    training_df = np.array(list(self.training_data)[::-1])
    daily_pct_change = (np.roll(training_df, 1) - training_df) / training_df

    return daily_pct_change[1:].reshape(-1, 1)

def my_training_method(self):
    features = self.get_features()
    self.model.fit(features)
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM.
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current close price to the `RollingWindow` that holds the training data.

```
# Add the latest bar to the training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol].close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the current feature set and make a prediction.
new_feature = self.get_features()
prediction = self.model.predict(new_feature)
prediction = float(prediction[-1])
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if prediction == 1:
    self.set_holdings(self._symbol, 1)
else:
    self.liquidate(self._symbol)
```

Save Models

Follow these steps to save `hmmlearn` models into the Object Store :

1. Set the key name you want to store the model under in the Object Store.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model.hmm"
   ```

2. Call the `get_file_path` method with the key.

```
# Get the file path to correctly save and access the model in Object Store.
file_name = self.object_store.get_file_path(model_key)
```

This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

```
# Serialize the model and save it to the file.
joblib.dump(self.model, file_name)
```

If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model.hmm <pathTo>/model.hmm
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `hmmlearn` models that you saved in the Object Store. To load a `hmmlearn` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the hmmlearn model from Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model.hmm"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `Hmmlearn` library.

Example 1: Hidden Markov Model Regime Detection

The below algorithm makes use of `Hmmlearn` library to predict the future market regime through a hidden markov model using return data. If the regime is upward market, we hold SPY. Otherwise, we liquidate the portfolio. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
from hmmlearn import hmm
import joblib

class HmmlearnExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 252*2
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

        # Retrieve already trained model from object store to use immediately.
        self._model_key = f"{self.project_id}/model.hmm"
        if self.object_store.contains_key(self._model_key):
            file_name = self.object_store.get_file_path(self._model_key)
            self.model = joblib.load(file_name)
        # Create a 2-regime model otherwise to predict different variance regime markets.
        else:
            self.model = hmm.GaussianHMM(n_components=2, covariance_type="full", n_iter=100)

    def on_warmup_finished(self) -> None:
        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features(self) -> None:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = np.array([x.close for x in self.securities[self._symbol].session][::-1])
        daily_pct_change = np.diff(training_df) / training_df[:-1]

        return daily_pct_change.reshape(-1, 1)

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features = self.get_features()
        # Recalibrate the model based on updated data.
        self.model.fit(features)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or self._symbol not in slice.bars:
            return

        # Get prediction by the updated features.
        new_feature = self.get_features()
        prediction = self.model.predict(new_feature)
        prediction = float(prediction[-1])

        # If the predicted direction is going upward, buy SPY.
        if prediction == 1:
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, liquidate.
        else:
            self.liquidate(self._symbol)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        file_name = self.object_store.get_file_path(self._model_key)
        joblib.dump(self.model, file_name)
```

Popular Libraries

Keras

Introduction

This page explains how to build, train, deploy and store `Keras` models.

Import Libraries

Import the `keras` libraries.

```
PY
from AlgorithmImports import *
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
```

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `keras` model and make predictions.

```
PY
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a neural-network regression model that uses the following features and labels:

Data Category	Description
Features	Daily percent change of the open, high, low, close, and volume of the SPY over the last 5 days
Labels	Daily percent return of the SPY over the next day

The following image shows the time difference between the features and labels:

Follow the below steps to build the model:

1. In the `initialize` method, create a `Sequential` object with several layers.

```
# Build a neural network with Dense layers for feature extraction, Flatten for reshaping, and a final
# Dense layer for prediction.
self.model = Sequential([Dense(10, input_shape=(5,5), activation='relu'),
                         Dense(10, activation='relu'),
                         Flatten(),
                         Dense(1)])
```

Set the `input_shape` of the first layer to `(5, 5)` because each sample contains the percent change of 5 factors (percent change of the open, high, low, close, and volume) over the previous 5 days. Call the `Flatten` constructor because the input is 2-dimensional but the output is just a single value.

2. Call the `compile` method with a loss function, an optimizer, and a list of metrics to monitor.

```
# Configure the model to use mean squared error loss, Adam optimizer, and track mean absolute error
# and mean squared error for performance evaluation.
self.model.compile(loss='mse',
                   optimizer=Adam(),
                   metrics=['mae', 'mse'])
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of historical TradeBar objects.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features_and_labels(self, n_steps=5):
    training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.training_data)[::-1])
    daily_pct_change = training_df.pct_change().dropna()

    features = []
    labels = []
    for i in range(len(daily_pct_change)-n_steps):
        features.append(daily_pct_change.iloc[i:i+n_steps].values)
        labels.append(daily_pct_change['close'].iloc[i+n_steps])
    features = np.array(features)
    labels = np.array(labels)

    return features, labels

def my_training_method(self):
    features, labels = self.get_features_and_labels()
    self.model.fit(features, labels, epochs=5)
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the latest bar to training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol])
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the current feature set and make a prediction.
features, _ = self.get_features_and_labels()
features = features[-1].reshape(1, 5, 5)
prediction = float(self.model.predict(features)[-1])
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if prediction > 0:
    self.set_holdings(self._symbol, 1)
elif prediction < 0:
    self.set_holdings(self._symbol, -1)
```

Save Models

Follow these steps to save `keras` models into the Object Store :

1. Set the key name you want to store the model under in the Object Store.

   The key must end with a .keras extension for the native Keras format (recommended) or a .h5 extension.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model.keras"
   ```

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Call the `save` method with the file path.

   ```
   # Serialize the model and save it to the file.
   self.model.save(file_name)
   ```

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model.keras <pathTo>/model.keras
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `keras` models that you saved in the Object Store. To load a `keras` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load_model` method.

```
# Load the model from Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model.keras"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = load_model(file_name)
```

PY

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `Keras` library.

Example 1: Multi-layer Perceptron Model

The below algorithm makes use of `Keras` library to predict the future price movement using the previous 5 OHLCV data. The model is built with multi-layer perceptrons, ReLu activation function, and Adam optimization. It is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam

class KerasExampleAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)  # Set Strategy Cash
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Retrieve already trained model from object store to use immediately.
        self._model_key = f"{self.project_id}/model.keras"
        if self.object_store.contains_key(self._model_key):
            file_name = self.object_store.get_file_path(self._model_key)
            self.model = load_model(file_name)

        # Create a MLP model otherwise to predict price movement.
        else:
            self.model = Sequential([Dense(10, input_shape=(5,5), activation='relu'),
                            Dense(10, activation='relu'),
                            Flatten(),
                            Dense(1)])
            self.model.compile(loss='mse',
                    optimizer=Adam(),
                    metrics=['mae', 'mse'])

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 252*2
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

    def on_warmup_finished(self) -> None:
        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> tuple[float, float]:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.securities[self._symbol].session)[::-1])
        daily_pct_change = training_df.pct_change().dropna()

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(daily_pct_change)-n_steps):
            features.append(daily_pct_change.iloc[i:i+n_steps].values)
            labels.append(daily_pct_change['close'].iloc[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()
        # Recalibrate the model based on updated data.
        self.model.fit(features, labels, epochs=5)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or self._symbol not in slice.bars:
            return

        # Get prediction by the updated features.
        features, _ = self.get_features_and_labels()
        features = features[-1].reshape(1, 5, 5)
        prediction = float(self.model.predict(features)[-1])

        # If the predicted direction is going upward, buy SPY.
        if prediction > 0:
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction < 0:
            self.set_holdings(self._symbol, -1)

    def on_end_of_algorithm(self):
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        file_name = self.object_store.get_file_path(self._model_key)
        self.model.save(file_name)
```

Popular Libraries

PyTorch

Introduction

This page explains how to build, train, deploy and store `PyTorch` models.

Import Libraries

Import the `torch` and `joblib` libraries.

```
PY
from AlgorithmImports import *
import torch
from torch import nn
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `torch` model and make predictions.

```
PY
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a neural-network regression model that uses the following features and labels:

Data Category	Description
Features	The last 5 closing prices.
Labels	The following day's closing price

The following image shows the time difference between the features and labels:

Follow these steps to create a method to build the model:

1. Define a subclass of `nn.Module` to be the model.

   In this example, use the ReLU activation function for each layer.

```
# Define a feed-forward neural network with two hidden layers for learning complex features, ReLU
# activations to introduce non-linearity, and a single regression output for predicting continuous values.
class NeuralNetwork(nn.Module):
    # Define the model structure.
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(5, 5),   # input size, output size of the layer.
            nn.ReLU(),         # Relu non-linear transformation.
            nn.Linear(5, 5),
            nn.ReLU(),
            nn.Linear(5, 1),   # Output size = 1 for regression.
        )

    # Define the feed-forward prediction procedure.
    def forward(self, x):
        x = torch.from_numpy(x).float()
        result = self.linear_relu_stack(x)
        return result
```

2. Create an instance of the model and set its configuration to train on the GPU if it's available.

```
# Use GPU if it's available for faster computation. Otherwise, fallback to CPU.
device = 'cuda' if torch.cuda.is_available() else 'cpu'
# Initialize the model on the selected device.
self.model = NeuralNetwork().to(device)
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of historical closing prices.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar.close)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features_and_labels(self, n_steps=5):
    close_prices = list(self.training_data)[::-1]

    features = []
    labels = []
    for i in range(len(close_prices)-n_steps):
        features.append(close_prices[i:i+n_steps])
        labels.append(close_prices[i+n_steps])
    features = np.array(features)
    labels = np.array(labels)

    return features, labels

def my_training_method(self):
    features, labels = self.get_features_and_labels()

    # Set the loss and optimization functions.
    # In this example, use the mean squared error as the loss function and stochastic gradient descent as the optimizer.
    loss_fn = nn.MSELoss()
    learning_rate = 0.001
    optimizer = torch.optim.SGD(self.model.parameters(), lr=learning_rate)

    # Loop through each epoch and train the model.
    epochs = 5
    for t in range(epochs):
        # Loop through each batch to train the model.
        for batch, (feature, label) in enumerate(zip(features, labels)):
            # Calculate the prediction and loss.
            pred = self.model(feature)
            real = torch.from_numpy(np.array(label).flatten()).float()
            loss = loss_fn(pred, real)

            # Perform backpropagation.
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the latest bar to training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.Bars:
        self.training_data.add(slice.bars[self._symbol].close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and pass it to the model.

```
# Get the current feature set and make a prediction.
features, _ = self.get_features_and_labels()
prediction = self.model(features[-1].reshape(1, -1))
prediction = float(prediction.detach().numpy()[-1])
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if prediction > slice.bars[self._symbol].price:
    self.set_holdings(self._symbol, 1)
elif prediction < slice.bars[self._symbol].price:
    self.set_holdings(self._symbol, -1)
```

Save Models

Follow these steps to save `PyTorch` models into the Object Store :

1. Set the key name of the model to be stored in the Object Store.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model"
   ```

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

   ```
   # Serialize the model and save it to the file.
   joblib.dump(self.model, file_name)
   ```

   If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model <pathTo>/model
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `PyTorch` models that you saved in the Object Store. To load a `PyTorch` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the PyTorch model from Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `PyTorch` library.

Example 1: Multi-layer Perceptron Model

The below algorithm makes use of `PyTorch` library to predict the future price movement using the previous 5 OHLCV data. The model is built with multi-layer perceptrons, ReLu activation function, and stochastic gradient descent optimization. It is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
import torch
from torch import nn
import joblib

class PyTorchExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 252*2
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

        # Retrieve already trained model from object store to use immediately.
        self._model_key = f"{self.project_id}/pytorch-example-model"
        if self.live_mode and self.object_store.contains_key(self._model_key):
            file_name = self.object_store.get_file_path(self._model_key)
            self.model = joblib.load(file_name)
        # Create a MLP model otherwise to predict price movement.
        else:
            device = 'cuda' if torch.cuda.is_available() else 'cpu'
            self.model = NeuralNetwork().to(device)

    def on_warmup_finished(self) -> None:
        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        close_prices = [x.close for x in self.securities[self._symbol].session][::-1]

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(close_prices)-n_steps):
            features.append(close_prices[i:i+n_steps])
            labels.append(close_prices[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()

        # Set the loss and optimization functions.
        # In this example, use the mean squared error as the loss function and stochastic gradient descent as the optimizer.
        loss_fn = nn.MSELoss()
        learning_rate = 0.001
        optimizer = torch.optim.SGD(self.model.parameters(), lr=learning_rate)

        # Create a for-loop to train for preset number of epoch.
        epochs = 5
        for t in range(epochs):
            # Create a for-loop to fit the model per batch
            for batch, (feature, label) in enumerate(zip(features, labels)):
                # Compute prediction and loss.
                pred = self.model(feature)
                real = torch.from_numpy(np.array(label).flatten()).float()
                loss = loss_fn(pred, real)

                # Perform backpropagation.
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or self._symbol not in slice.bars:
```

```
            return

        # Get prediction by the updated features.
        features, __ = self.get_features_and_labels()
        prediction = self.model(features[-1].reshape(1, -1))
        if isinstance(prediction, np.ndarray):
            prediction = float(prediction[-1])  # No need for detach() on NumPy arrays
        elif isinstance(prediction, torch.Tensor):
            prediction = float(prediction.detach().numpy()[-1])

        # If the predicted direction is going upward, buy SPY.
        if prediction > slice.bars[self._symbol].price:
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction < slice.bars[self._symbol].price:
            self.set_holdings(self._symbol, -1)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        if not self.live_mode:
            return
        file_name = self.object_store.get_file_path(self._model_key)
        joblib.dump(self.model, file_name)
        self.object_store.save(self._model_key)

class NeuralNetwork(nn.Module):
    # Model Structure
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(5, 5),   # input size, output size of the layer
            nn.ReLU(),         # Relu non-linear transformation
            nn.Linear(5, 5),
            nn.ReLU(),
            nn.Linear(5, 1),   # Output size = 1 for regression
        )

    # Feed-forward training/prediction
    def forward(self, x):
        x = torch.from_numpy(x).float()   # Convert to tensor in type float
        result = self.linear_relu_stack(x)
        return result
```

Popular Libraries

Scikit-Learn

Introduction

This page explains how to build, train, deploy and store `Scikit-Learn` models.

Import Libraries

Import the `sklearn` and `joblib` libraries.

```py
from AlgorithmImports import *
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `sklearn` model and make predictions.

```py
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a support vector regression prediction model that uses the following features and labels:

Data Category	Description
Features	Daily percent change of the open, high, low, close, and volume of the SPY over the last 5 days
Labels	Daily percent return of the SPY over the next day

The following image shows the time difference between the features and labels:

To build the model, call the `GridSearchCV` constructor with the SVR model, the parameter grid, a scoring method, the number of cross-validation folds:

```
# Define a grid of hyperparameters to search over and perform cross-validation.
aram_grid = {'C': [.05, .1, .5, 1, 5, 10],
             'epsilon': [0.001, 0.005, 0.01, 0.05, 0.1],
             'gamma': ['auto', 'scale']}
self.model = GridSearchCV(SVR(), param_grid, scoring='neg_mean_squared_error', cv=5)
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of historical bars.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features_and_labels(self, n_steps=5):
    training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.training_data)[::-1])
    daily_pct_change = training_df.pct_change().dropna()

    features = []
    labels = []
    for i in range(len(daily_pct_change)-n_steps):
        features.append(daily_pct_change.iloc[i:i+n_steps].values.flatten())
        labels.append(daily_pct_change['close'].iloc[i+n_steps])
    features = np.array(features)
    labels = np.array(labels)

    return features, labels

def my_training_method(self):
    features, labels = self.get_features_and_labels()
    self.model = self.model.fit(features, labels).best_estimator_
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the latest bar to the training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol])
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the current feature set and make a prediction.
features, _ = self.get_features_and_labels()
prediction = self.model.predict(features[-1].reshape(1, -1))
prediction = float(prediction)
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if prediction > 0:
    self.SetHoldings(self._symbol, 1)
elif prediction < 0:
    self.SetHoldings(self._symbol, -1)
```

Save Models

Follow these steps to save `sklearn` models into the Object Store :

1. Set the key name you want to store the model under in the Object Store.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model"
   ```

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

   ```
   # Serialize the model and save it to the file.
   joblib.dump(self.model, file_name)
   ```

   If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model <pathTo>/model
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `sklearn` models that you saved in the Object Store. To load a `sklearn` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
# Load the sklearn model from the Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using Scikit-Learn library.

Example 1: Scikit-Learn

The below algorithm makes use of `Scikit-Learn` library to predict the future price movement using the previous 5 OHLCV data. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
import joblib

class ScikitLearnExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self.symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # 2-year data to train the model.
        training_length = 252*2
        self.training_data = RollingWindow(training_length)
        # Warm up the training dataset to train the model immediately.
        history = self.history[TradeBar](self.symbol, training_length, Resolution.DAILY)
        for trade_bar in history:
            self.training_data.add(trade_bar)

        # Retrieve already trained model from object store to use immediately.
        if self.object_store.contains_key("sklearn_model"):
            file_name = self.object_store.get_file_path("sklearn_model")
            self.model = joblib.load(file_name)
        # Otherwise, grid serach with hyperparameter choices to create an optimal support vector regressor to predict price movement.
        else:
            param_grid = {'C': [.05, .1, .5, 1, 5, 10],
                          'epsilon': [0.001, 0.005, 0.01, 0.05, 0.1],
                          'gamma': ['auto', 'scale']}
            self.model = GridSearchCV(SVR(), param_grid, scoring='neg_mean_squared_error', cv=5)

        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        # Train and predict the return data, which is more normalized and stationary.
        training_df = self.pandas_converter.get_data_frame[TradeBar](list(self.training_data)[::-1])
        daily_pct_change = training_df.pct_change().dropna()

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(daily_pct_change)-n_steps):
            features.append(daily_pct_change.iloc[i:i+n_steps].values.flatten())
            labels.append(daily_pct_change['close'].iloc[i+n_steps])
        features = np.array(features)
        labels = np.array(labels)

        return features, labels

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()
        # Recalibrate the model based on updated data.
        if isinstance(self.model, GridSearchCV):
            self.model = self.model.fit(features, labels).best_estimator_
        else:
            self.model = self.model.fit(features, labels)

    def on_data(self, slice: Slice) -> None:
        if self.symbol in slice.bars:
            self.training_data.add(slice.bars[self.symbol])

        # Get prediction by the updated features.
        features, _ = self.get_features_and_labels()
        prediction = self.model.predict(features[-1].reshape(1, -1))
        prediction = float(prediction)

        # If the predicted direction is going upward, buy SPY.
        if prediction > 0:
            self.set_holdings(self.symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        elif prediction < 0:
            self.set_holdings(self.symbol, -1)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        model_key = "sklearn_model"
        file_name = self.object_store.get_file_path(model_key)
        joblib.dump(self.model, file_name)
        self.object_store.save(model_key)
```

Popular Libraries

Tensorflow

Introduction

This page explains how to build, train, deploy and store `Tensorflow` models.

Import Libraries

Import the `tensorflow` library.

```py
from AlgorithmImports import *
import tensorflow as tf
```

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `tensorflow` model and make predictions.

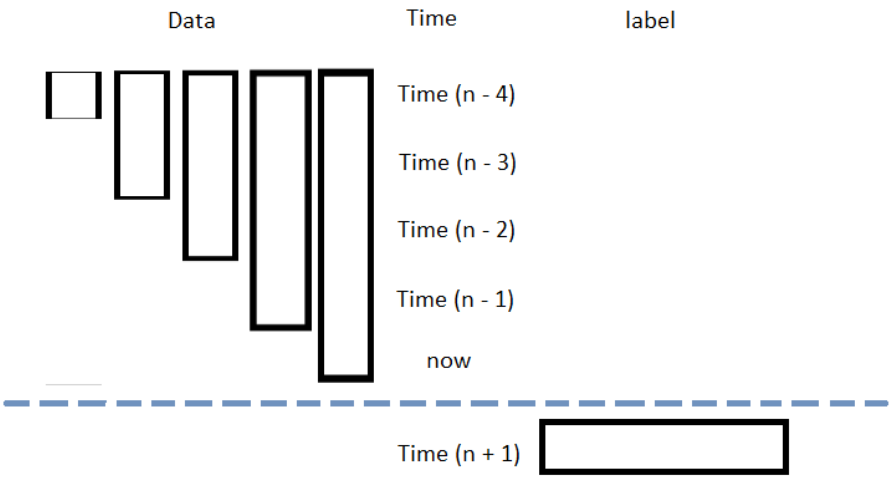
```py
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a neural-network regression prediction model that uses the following features and labels:

Data Category	Description
Features	The last 5 close price differencing compared to current price
Labels	The following day's price change

The following image shows the time difference between the features and labels:



Follow these steps to create a method to build the model:

1. Define some parameters, including the number of factors, neurons, and epochs.

```
# Set some hyperparameters:
num_factors = 5     # Set the number of input features.
num_neurons_1 = 10  # Set the neurons in each hidden layer to capture complexity.
num_neurons_2 = 10
num_neurons_3 = 5
self.epochs = 20    # Set the training epochs for sufficient learning.
self.learning_rate = 0.0001  # Set the learning rate for optimization control.
```

2. Create the model using built-in Keras API.

```
# Define the model with sequential layers: an input layer with ReLU activation to introduce non-linearity,
# two hidden layers for feature extraction, and an output layer for predictions.
self.model = tf.keras.Sequential([
    tf.keras.layers.Dense(num_neurons_1, activation=tf.nn.relu, input_shape=(num_factors,)),  # The input shape is required.
    tf.keras.layers.Dense(num_neurons_2, activation=tf.nn.relu),
    tf.keras.layers.Dense(num_neurons_3, activation=tf.nn.relu),
    tf.keras.layers.Dense(1)
])
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 300 days of historical closing prices.
training_length = 300
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar.close)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features_and_labels(self, lookback=5):
    lookback_series = []

    data = pd.Series(list(self.training_data)[::-1])
    for i in range(1, lookback + 1):
        df = data.diff(i)[lookback:-1]
        df.name = f"close-{i}"
        lookback_series.append(df)

    X = pd.concat(lookback_series, axis=1).reset_index(drop=True).dropna()
    Y = data.diff(-1)[lookback:-1].reset_index(drop=True)
    return X.values, Y.values

def my_training_method(self):
    features, labels = self.get_features_and_labels()

    # Define the loss function. We use MSE in this example.
    def loss_mse(target_y, predicted_y):
        return tf.reduce_mean(tf.square(target_y - predicted_y))

    # Train the model.
    optimizer = tf.keras.optimizers.Adam(learning_rate=self.learning_rate)
    for i in range(self.epochs):
        with tf.GradientTape() as t:
            loss = loss_mse(labels, self.model(features))

        jac = t.gradient(loss, self.model.trainable_weights)
        optimizer.apply_gradients(zip(jac, self.model.trainable_weights))
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, schedule some training sessions .

```
# Train the model every Sunday at 8:00 AM
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current close price to the `RollingWindow` that holds the training data.

```
# Add the latest closing price to the training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        self.training_data.add(slice.bars[self._symbol].close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `run` method with new features.

```
# Get the current feature set and make a prediction.
new_features, __ = self.get_features_and_labels()
prediction = self.model(new_features)
prediction = float(prediction.numpy()[-1])
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if prediction > 0:
    self.SetHoldings(self._symbol, 1)
elif prediction < 0:
    self.SetHoldings(self._symbol, -1)
```

Save Models

Follow these steps to save `Tensorflow` models into the Object Store :

1. Set the key name of the model to be stored in the Object Store.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model.keras"
   ```

   Note that the model has to have the suffix `.keras` .

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Call the `save` method with the model and file path.

```
# Serialize the model and save it to the file.
model.save(file_name)
```

4. Save the model to the file path.

```
# Save the model to object store to capture model state for reusability.
self.object_store.save(model_key)
```

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model.keras <pathTo>/model.keras
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `tensorflow` models that you saved in the Object Store. To load a `tensorflow` model from the Object Store, in the `initialize` method, get the file path to the saved model and then recall the graph and weights of the model.

```
# Load the tensorflow model from the Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model.keras"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = tf.keras.models.load_model(file_name)
```

The `contains_key` method returns a boolean that represents if the `model.keras` is in the Object Store. If the Object Store doesn't contain the keys, save the model using them before you proceed.

Examples

The following examples demonstrate some common practices for using `Keras` library.

Example 1: Multi-layer Perceptron Model

The below algorithm makes use of `Keras` library to predict the future price movement using the previous 5 OHLCV data. The model is built with multi-layer perceptrons, ReLu activation function, and Adam optimization. It is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
import tensorflow as tf

class TensorFlowAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Hyperparameter to create the MLP model.
        num_factors = 5
        num_neurons_1 = 10
        num_neurons_2 = 10
        num_neurons_3 = 5
        self.epochs = 100
        self.learning_rate = 0.0001

        # Create the MLP model with ReLu activiation.
        self.model = tf.keras.Sequential([
            tf.keras.layers.Dense(num_neurons_1, activation=tf.nn.relu, input_shape=(num_factors,)),  # input shape required
            tf.keras.layers.Dense(num_neurons_2, activation=tf.nn.relu),
            tf.keras.layers.Dense(num_neurons_3, activation=tf.nn.relu),
            tf.keras.layers.Dense(1)
        ])

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 500
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

    def on_warmup_finished(self) -> None:
        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.week_start(), self.time_rules.at(8, 0), self.my_training_method)

    def get_features_and_labels(self, lookback=5) -> None:
        lookback_series = []

        # Train and predict the N differencing data, which is more normalized and stationary.
        data = pd.Series([x.close for x in self.securities[self._symbol].session][::-1])
        for i in range(1, lookback + 1):
            df = data.diff(i)[lookback:-1]
            df.name = f"close-{i}"
            lookback_series.append(df)

        X = pd.concat(lookback_series, axis=1).reset_index(drop=True).dropna()
        Y = data.diff(-1)[lookback:-1].reset_index(drop=True)
        return X.values, Y.values

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features, labels = self.get_features_and_labels()

        # Define the loss function, we use MSE in this example
        def loss_mse(target_y, predicted_y):
            return tf.reduce_mean(tf.square(target_y - predicted_y))

        # Train the model with Adam optimization function.
        optimizer = tf.keras.optimizers.Adam(learning_rate=self.learning_rate)
        for i in range(self.epochs):
            with tf.GradientTape() as t:
                loss = loss_mse(labels, self.model(features))

            jac = t.gradient(loss, self.model.trainable_weights)
            optimizer.apply_gradients(zip(jac, self.model.trainable_weights))

    def on_data(self, data) -> None:
        if self.is_warming_up or not data.bars.contains_key(self._symbol):
            return

        # Get prediction by the updated features.
        new_features, __ = self.get_features_and_labels()
        prediction = float(self.model(new_features).numpy().flatten()[-1])

        # If the predicted direction is going upward, buy SPY, else sell.
        self.set_holdings(self._symbol, 1 if prediction > 0 else -1)
```

Popular Libraries

Tslearn

Introduction

This page explains how to build, train, deploy and store `Tslearn` models.

Import Libraries

Import the `tslearn` libraries.

```
PY
from AlgorithmImports import *
from tslearn.barycenters import softdtw_barycenter
from tslearn.clustering import TimeSeriesKMeans
```

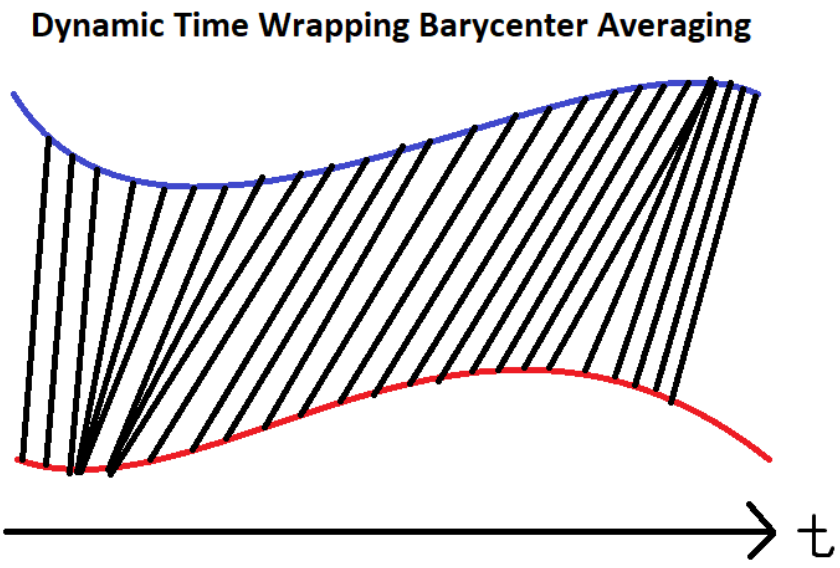
Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `tslearn` model.

```
PY
# Add securities and save a reference to their Symbol objects.
tickers = ["SPY", "QQQ", "DIA",
           "AAPL", "MSFT", "TSLA",
           "IEF", "TLT", "SHV", "SHY",
           "GLD", "IAU", "SLV",
           "USO", "XLE", "XOM"]
symbols = [self.add_equity(ticker, Resolution.DAILY).symbol for ticker in tickers]
```

Build Models

In this example, train a model that clusters the universe of Equities into distinct groups and then allocate an equal portion of the portfolio to each cluster. To cluster the securities, instead of using a real-time comparison, apply Dynamic Time Wrapping Barycenter Averaging (DBA) to their historical prices and then run a k-means clustering algorithm. DBA is a technique of averaging a few time-series into a single one without losing much of their information. Since not all time-series move efficiently like in ideal EMH assumption, this technique allows similarity analysis of different time-series with sticky lags. The following image shows a visualization of the process. For more information about the technical details, see Dynamic Time Warping in the tslearn documentation .



To perform DBA and then cluster the securities by k-means, create a TimeSeriesKMeans model:

```
# Create a model to cluster the time series into 6 groups using DTW for similarity measurement.
self.model = TimeSeriesKMeans(n_clusters=6, metric="dtw")
```

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```
# Fill a RollingWindow with 2 years of historical closing prices.
training_length = 252
self.training_data = {}
history = self.history(self.symbols, training_length, Resolution.DAILY).unstack(0).close
for symbol in self.symbols:
    self.training_data[symbol] = RollingWindow(training_length)
    for close_price in history[symbol]:
        self.training_data[symbol].add(close_price)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features(self):
    close_price = pd.DataFrame({symbol: list(data)[::-1] for symbol, data in self.training_data.items()})
    log_price = np.log(close_price)
    log_normal_price = (log_price - log_price.mean()) / log_price.std()

    return log_normal_price

def my_training_method(self):
    features = self.get_features()
    self.model.fit(features.T.values)
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```
# Train the model every Sunday at 8:00 AM
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the latest price to the training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    for kvp in slice.bars:
        self.training_data[kvp.key].add(kvp.value.close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the current feature set and make a prediction.
features = self.get_features()
self.labels = self.model.predict(features.T.values)
```

You can use the label prediction to place orders.

```
# Place orders based on the predicted clusters.
for i in set(self.labels):
    assets_in_cluster = features.columns[[n for n, k in enumerate(self.labels) if k == i]]
    size = 1/6/len(assets_in_cluster)
    self.set_holdings([PortfolioTarget(symbol, size) for symbol in assets_in_cluster])
```

Save Models

Follow these steps to save `tslearn` models into the Object Store :

1. Set the key name of the model to be stored in the Object Store.

   ```
   # Set the key to store the model in the Object Store for reuse across sessions.
   model_key = f"{self.project_id}/model.hdf5"
   ```

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Delete the current file to avoid a `FileExistsError` error when you save the model.

   ```
   import os
   os.remove(file_name)
   ```

4. Call the `to_hdf5` method with the file path.

   ```
   # Serialize the model and save it to the file.
   self.model.to_hdf5(file_name)
   ```

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model.hdf5 <pathTo>/model.hdf5
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `tslearn` models that saved in Object Store. To load a `tslearn` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `from_hdf5` method.

```
# Load the tslearn model from the Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model.hdf5"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = TimeSeriesKMeans.from_hdf5(file_name + ".hdf5")
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `Tslearn` library.

Example 1: Dynamic Time Wrapping (DTW) Clustering

The below algorithm makes use of `Tslearn` library to cluster stocks based on their recent year's price movement through DTW Barycenter Averaging. Then, we invest each cluster equally, while investing stocks within cluster equally as well. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

PY

```
from tslearn.barycenters import softdtw_barycenter
from tslearn.clustering import TimeSeriesKMeans
import joblib

class TslearnExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request stocks data for model training, clustering and trading.
        tickers = ["SPY", "QQQ", "DIA",
                "AAPL", "MSFT", "TSLA",
                "IEF", "TLT", "SHV", "SHY",
                "GLD", "IAU", "SLV",
                "USO", "XLE", "XOM"]
        self.symbols = [self.add_equity(ticker, Resolution.DAILY).symbol for ticker in tickers]

        # 1-year data to train the model.
        self._training_length = 252

        # Create the DTW model to cluster the stocks.
        self.model = TimeSeriesKMeans(n_clusters=6,   # We have 6 main groups
                                      metric="dtw")

        # Train the model to use the prediction right away.
        self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features(self) -> None:
        # Get historical close prices and cluster the log-return data, which is more normalized and stationary.
        close_price = self.history(self.symbols, self._training_length, Resolution.DAILY).unstack(0).close.dropna(axis=1)
        log_price = np.log(close_price)
        log_normal_price = (log_price - log_price.mean()) / log_price.std()

        return log_normal_price

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        features = self.get_features()
        # Recalibrate the model based on updated data.
        self.model.fit(features.T.values)

    def on_data(self, slice: Slice) -> None:
        if not hasattr(self.model, 'cluster_centers_'):
            return

        # Get prediction by the updated features.
        features = self.get_features()
        self.labels = self.model.predict(features.T.values)

        # Equally invest in each cluster, while investing each stocks within cluster equally to evenly dissipate capital risks at various
levels.
        for i in set(self.labels):
            assets_in_cluster = features.columns[[n for n, k in enumerate(self.labels) if k == i]]
            size = 1/6/len(assets_in_cluster)
            self.set_holdings([PortfolioTarget(symbol, size) for symbol in assets_in_cluster])

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        model_key = f"{self.project_id}/model_test.hdf5"
        # If the model already exists, delete it so the `to_hdf5` method doesn't raise an error.
        if self.object_store.contains_key(model_key):
            self.object_store.delete(model_key)
        file_name = self.object_store.get_file_path(model_key)
        self.model.to_hdf5(file_name)
```

Popular Libraries

XGBoost

Introduction

This page explains how to build, train, deploy and store `XGBoost` models.

Import Libraries

Import the `XGBoost` and `joblib` libraries.

```
from AlgorithmImports import *
import xgboost as xgb
import joblib
```

You need the `joblib` library to store models.

Create Subscriptions

In the `initialize` method, subscribe to some data so you can train the `xgboost` model and make predictions.

```
# Add a security and save a reference to its Symbol.
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
```

Build Models

In this example, build a gradient boost tree regression prediction model that uses the following features and labels:

Data Category	Description
Features	The last 5 closing prices
Labels	The following day's closing price

The following image shows the time difference between the features and labels:

Train Models

You can train the model at the beginning of your algorithm and you can periodically re-train it as the algorithm executes.

Warm Up Training Data

You need historical data to initially train the model at the start of your algorithm. To get the initial training data, in the `initialize` method, make a history request .

```py
# Fill a RollingWindow with 2 years of historical closing prices.
training_length = 252*2
self.training_data = RollingWindow(training_length)
history = self.history[TradeBar](self._symbol, training_length, Resolution.DAILY)
for trade_bar in history:
    self.training_data.add(trade_bar.close)
```

Define a Training Method

To train the model, define a method that fits the model with the training data.

```py
# Prepare feature and label data for training by processing the RollingWindow data into a time series.
def get_features_and_labels(self, n_steps=5):
    # Create a list of close prices from the rolling window.
    close_prices = np.array(list(self.training_data)[::-1])
    df = (np.roll(close_prices, -1) - close_prices) * 0.5 + close_prices * 0.5
    df = df[:-1]

    features = []
    labels = []
    for i in range(len(df)-n_steps):
        features.append(df[i:i+n_steps])
        labels.append(df[i+n_steps])

    features = np.array(features)
    labels = np.array(labels)
    # Standardize the features and labels arrays.
    features = (features - features.mean()) / features.std()
    labels = (labels - labels.mean()) / labels.std()

    # Load the NumPy array into a DMatrix object.
    d_matrix = xgb.DMatrix(features, label=labels)

    return d_matrix

def my_training_method(self):
    # Instantiate the DMatrix object with the features and labels array.
    d_matrix = self.get_features_and_labels()
    # Define the parameters for the XGBoost model.
    params = {
        'booster': 'gbtree',
        'colsample_bynode': 0.8,
        'learning_rate': 0.1,
        'lambda': 0.1,
        'max_depth': 5,
        'num_parallel_tree': 100,
        'objective': 'reg:squarederror',
        'subsample': 0.8,
      }
    # Create the model and fit it with the training data.
    self.model = xgb.train(params, d_matrix, num_boost_round=10)
```

Set Training Schedule

To train the model at the beginning of your algorithm, in the `initialize` method, call the `train` method.

```py
# Train the model initially to provide a baseline for prediction and decision-making.
self.train(self.my_training_method)
```

To periodically re-train the model as your algorithm executes, in the `initialize` method, call the `train` method as a Scheduled Event .

```py
# Train the model every Sunday at 8:00 AM.
self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8, 0), self.my_training_method)
```

Update Training Data

To update the training data as the algorithm executes, in the `on_data` method, add the current `TradeBar` to the `RollingWindow` that holds the training data.

```
# Add the latest price to the training data to ensure the model is trained with the most recent market data.
def on_data(self, slice: Slice) -> None:
    if self._symbol in slice.bars:
        # Update the training data with new most recent close price.
        self.training_data.add(slice.bars[self._symbol].close)
```

Predict Labels

To predict the labels of new data, in the `on_data` method, get the most recent set of features and then call the `predict` method.

```
# Get the current feature set and make a prediction.
new_d_matrix = self.get_features_and_labels(df)
prediction = self.model.predict(new_d_matrix)
prediction = prediction.flatten()
```

You can use the label prediction to place orders.

```
# Place orders based on the forecasted market direction.
if float(prediction[-1]) > float(prediction[-2]):
    self.set_holdings(self._symbol, 1)
else:
    self.set_holdings(self._symbol, -1)
```

Save Models

Follow these steps to save `xgboost` models into the Object Store :

1. Set the key name of the model to be stored in the Object Store.

   ```
   # Set the key to store the model in the Object Store so you can use it later.
   model_key = f"{self.project_id}/model"
   ```

2. Call the `get_file_path` method with the key.

   ```
   # Get the file path to correctly save and access the model in Object Store.
   file_name = self.object_store.get_file_path(model_key)
   ```

   This method returns the file path where the model will be stored.

3. Call the `dump` method the file path.

   ```
   # Serialize the model and save it to the file.
   joblib.dump(self.model, file_name)
   ```

   If you dump the model using the `joblib` module before you save the model, you don't need to retrain the model.

Upload Models

If you train models locally or in another environment, you can upload the model files to the Object Store so your algorithms can use them. Use the model key that matches the key your algorithm expects when it calls `object_store.get_file_path` .

Follow one of these approaches to upload your model files:

LEAN CLI

Run the `lean cloud object-store set` command to upload a local file to the Object Store.

```
$ lean cloud object-store set <projectId>/model <pathTo>/model
```

Replace `<projectId>` with your project Id and `<pathTo>` with the path to the local model file.

Cloud API

Use the Upload Object Store Files endpoint to upload a model file through the API.

Load Models

You can load and trade with pre-trained `xgboost` models that saved in Object Store. To load a `xgboost` model from the Object Store, in the `initialize` method, get the file path to the saved model and then call the `load` method.

```
PY
# Load the xgboost model from the Object Store to use its saved state and update it with new data if needed.
def initialize(self) -> None:
    model_key = f"{self.project_id}/model"
    if self.object_store.contains_key(model_key):
        file_name = self.object_store.get_file_path(model_key)
        self.model = joblib.load(file_name)
```

The `contains_key` method returns a boolean that represents if the `model_key` is in the Object Store. If the Object Store does not contain the `model_key` , save the model using the `model_key` before you proceed.

Examples

The following examples demonstrate some common practices for using `XGBoost` library.

Example 1: Gradient Boosting Model

The below algorithm makes use of `XGBoost` library to predict the future price movement using the previous 5 OHLCV data. The model is trained using rolling 2-year data. To ensure the model applicable to the current market environment, we recalibrate the model on every Sunday.

```
PY
import xgboost as xgb
import joblib

class XGBoostExampleAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        # Request SPY data for model training, prediction and trading.
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Use the session rolling window to cache 2-year daily data for training.
        training_length = 252*2
        self.securities[self._symbol].session.size = training_length
        self.set_warm_up(training_length, Resolution.DAILY)

        # Retrieve already trained model from object store to use immediately.
        self._model_key = f"{self.project_id}/xgboost-example-model"
        if self.live_mode and self.object_store.contains_key(self._model_key):
            file_name = self.object_store.get_file_path(self._model_key)
            self.model = joblib.load(file_name)

    def on_warmup_finished(self) -> None:
        if not hasattr(self, 'model'):
            # Train the model to use the prediction right away.
            self.train(self.my_training_method)
        # Recalibrate the model weekly to ensure its accuracy on the updated domain.
        self.train(self.date_rules.every(DayOfWeek.SUNDAY), self.time_rules.at(8,0), self.my_training_method)

    def get_features_and_labels(self, n_steps=5) -> None:
        # Train and predict the partial-differencing data, which is more stationary while more variance remaining.
        close_prices = np.array([x.close for x in self.securities[self._symbol].session][::-1])
        df = (np.roll(close_prices, -1) - close_prices) * 0.5 + close_prices * 0.5
        df = df[:-1]

        # Stack the data for 5-day OHLCV data per each sample to train with.
        features = []
        labels = []
        for i in range(len(df)-n_steps):
            features.append(df[i:i+n_steps])
            labels.append(df[i+n_steps])

        features = np.array(features)
        labels = np.array(labels)
        features = (features - features.mean()) / features.std()
        labels = (labels - labels.mean()) / labels.std()

        d_matrix = xgb.DMatrix(features, label=labels)
```

```
        return d_matrix

    def my_training_method(self) -> None:
        # Prepare the processed training data.
        d_matrix = self.get_features_and_labels()
        # Recalibrate the model based on updated data.
        params = {
            'booster': 'gbtree',
            'colsample_bynode': 0.8,
            'learning_rate': 0.1,
            'lambda': 0.1,
            'max_depth': 5,
            'num_parallel_tree': 100,
            'objective': 'reg:squarederror',
            'subsample': 0.8,
        }
        self.model = xgb.train(params, d_matrix, num_boost_round=2)

    def on_data(self, slice: Slice) -> None:
        if self.is_warming_up or self._symbol not in slice.bars:
            return

        # Get prediction by the updated features.
        new_d_matrix = self.get_features_and_labels()
        prediction = self.model.predict(new_d_matrix)
        prediction = prediction.flatten()

        # If the predicted direction is going upward, buy SPY.
        if float(prediction[-1]) > float(prediction[-2]):
            self.set_holdings(self._symbol, 1)
        # If the predicted direction is going downward, sell SPY.
        else:
            self.set_holdings(self._symbol, -1)

    def on_end_of_algorithm(self) -> None:
        # Store the model to object store to retrieve it in other instances in case the algorithm stops.
        if not self.live_mode:
            return
        file_name = self.object_store.get_file_path(self._model_key)
        joblib.dump(self.model, file_name)
        self.object_store.save(self._model_key)
```

Machine Learning

Hugging Face

Hugging Face

Key Concepts

Introduction

Hugging Face is a leading company in natural language processing (NLP), providing powerful tools and pre-trained models that can significantly enhance your trading algorithms on QuantConnect. By integrating Hugging Face's models, you can easily leverage advanced NLP capabilities for tasks such as sentiment analysis, news classification, and market forecasting without defining and training the model yourself. Hugging Face provides an easy way to leverage the research of community members to increase the sophistication of your trading algorithms.

Supported Models

Hundreds of thousands of Hugging Face models are publicly available and ready to use. To view all of them, see the Model Hub on the Hugging Face website. The Model card tab of each model repository explains an overview of how the model works, its requirements, and a quick start guide.

QuantConnect Cloud caches some of the most popular models to speed up your development workflow. The following table shows the cached models:

Name	Category	Example
ahmedrachid/FinancialBERT-Sentiment-Analysis	Text Classification	
amazon/chronos-t5-base	Time Series Forecasting	
amazon/chronos-t5-large	Time Series Forecasting	
amazon/chronos-t5-small	Time Series Forecasting	
amazon/chronos-t5-tiny	Time Series Forecasting	
autogluon/chronos-bolt-base	Time Series Forecasting	
autogluon/chronos-t5-base	Time Series Forecasting	
autogluon/chronos-t5-large	Time Series Forecasting	
autogluon/chronos-t5-tiny	Time Series Forecasting	
AutonLab/MOMENT-1-large	Time Series Forecasting	
AventIQ-AI/sentiment-analysis-for-stock-market-sentiment	Text Classification	
bardsai/finance-sentiment-fr-base	Text Classification	
cardiffnlp/twitter-roberta-base-sentiment-latest	Text Classification	
deepseek-ai/DeepSeek-R1-Distill-Llama-70B	Text Generation	

distilbert/distilbert-base-cased-distilled-squad	Question Answering	
distilbert/distilbert-base-uncased	Fill-Mask	
FacebookAI/roberta-base	Fill-Mask	
google-bert/bert-base-uncased	Fill-Mask	
google/gemma-7b	Text Generation	
ibm-granite/granite-timeseries-ttm-r1	Time Series Forecasting	
microsoft/deberta-base	Fill-Mask	
mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis	Text Classification	
nickmuchi/deberta-v3-base-finetuned-finance-text-classification	Text Classification	
nickmuchi/distilroberta-finetuned-financial-text-classification	Text Classification	
nickmuchi/sec-bert-finetuned-finance-classification	Text Classification	
openai-community/gpt2	Text Generation	
ProsusAI/finbert	Text Classification	
Salesforce/moirai-1.0-R-base	Time Series Forecasting	
Salesforce/moirai-1.0-R-large	Time Series Forecasting	
Salesforce/moirai-1.0-R-small	Time Series Forecasting	
StephanAkkerman/FinTwitBERT-sentiment	Text Classification	
yiyanghkust/finbert-tone	Text Classification	

To see the commit hash of the cached models, run the following algorithm in QC Cloud and then view the logs :

```py
# Scan and log all cached Hugging Face model revisions from the cache directory.
from huggingface_hub import scan_cache_dir

class HuggingFaceModelHashAlgorithm(QCAlgorithm):

    def initialize(self):
        cache_info = scan_cache_dir()
        cached_models_log = []
        for entry in cache_info.repos:
            revisions = [revision.commit_hash for revision in entry.revisions]
            cached_models_log.append(f'Repo: {entry.repo_id}. Revisions {str(revisions)}')
        self.quit('\n'.join(sorted(cached_models_log)))
```

Train Models

Hugging Face models are pre-trained, so you can quickly load them into your algorithms and use their output to inform your trading decisions. To customize the model for your specific use case, you can fine-tune the pre-trained model with your own training data.

Examples

You may refer to the following table for examples of popular Hugging Face models:

Model Name	Model Type	Example
Chronos-T5	Pretrained Time Series Forecasting Transformer Model	Example
FinBERT	Text Classification Transformer Model	Example

Hugging Face

Popular Models

These are examples of using some of the most common Hugging Face models in an algorithm. Click one to learn more.

Chronos-T5

Time series forecasting

DistilBERT

Question Answering

FinBERT

Sentiment analysis

Sentiment Analysis

Text classification

Fill-Mask

Feature extraction

Text Generation

Text generation

Chronos-Bolt

Time series forecasting

Popular Models

Chronos-T5

Introduction

This page explains how to use Chronos-T5 in LEAN trading algorithms. The model repository provides the following description:

> Chronos is a family of **pretrained time series forecasting models** based on language model architectures. A time series is transformed into a sequence of tokens via scaling and quantization, and a language model is trained on these tokens using the cross-entropy loss. Once trained, probabilistic forecasts are obtained by sampling multiple future trajectories given the historical context. Chronos models have been trained on a large corpus of publicly available time series data, as well as synthetic data generated using Gaussian processes.
>
> For details on Chronos models, training data and procedures, and experimental results, please refer to the paper Chronos: Learning the Language of Time Series .

Use Cases

The Chronos-T5 model is a time series forecasting model. The following use cases explain how you might utilize it in trading algorithms:

- Forecast the future equity curves for a set of assets, then pass them to an optimizer to determine the weights that maximize the future Sharpe ratio of the portfolio.
- Forecast the future volatility of an asset to manage risk and opimize portfolio allocations.
- Train the model to forecast the impact of specific events on an asset and then adjust your holdings in response to the event. For example, you could train the model to forecast the impact of news , corporate actions , or financial reports .

Load Pre-Trained Model

Follow these steps to load the pre-trained Chronos-T5 model:

1. Add the following imports to the top of your code file:

```
PY
from chronos import ChronosPipeline
import torch
```

2. Call the `ChronosPipeline.from_pretrained` method with the model path.

   In QuantConnect Cloud, the path to the tiny model is amazon / chronos-t5-tiny .

```
PY
# Load the ChronosPipeline model.
self._pipeline = ChronosPipeline.from_pretrained(
    "amazon/chronos-t5-tiny",
    device_map="cuda" if torch.cuda.is_available() else "cpu",
    torch_dtype=torch.bfloat16,
)
```

3. (Optional) Set the seed to enable reproducibility.

```
PY
# Set the seed.
from transformers import set_seed
set_seed(1, True)
```

Fine-Tune Model

The Chronos-T5 model is pre-trained, so you don't need to fine-tune it. Fine-tuning the model just tailors it to your specific use case. Follow these steps to fine-tune it:

1. Add the following imports to the top of your code file:

```py
import torch
from ast import literal_eval
from pathlib import Path
from functools import partial
from transformers import Trainer, TrainingArguments, set_seed
from gluonts.dataset.pandas import PandasDataset
from gluonts.itertools import Filter
from chronos import ChronosConfig, ChronosPipeline
from chronos.scripts.training.train import ChronosDataset, has_enough_observations, load_model
from chronos.scripts.training import train
from logging import getLogger, INFO
```

2. In the `initialize` method, define the model and some of its settings.

   In QuantConnect Cloud, the path to the tiny model is amazon/chronos-t5-tiny .

```py
def initialize(self):
    # Configure the model parameters to optimize performance.
    self._prediction_length = 3*21  # Three months of trading days
    self._device_map = "cuda" if torch.cuda.is_available() else "cpu"
    self._optimizer = 'adamw_torch_fused' if torch.cuda.is_available() else 'adamw_torch'
    self._model_name = "amazon/chronos-t5-tiny"
    self._model_path = self.object_store.get_file_path(
        f"{self.project_id}/llm/fine-tune/{self._model_name.replace('/', '-')}/"
    )
```

3. Define the fine-tuning method.

```py
# Configure the fine-tuning method and initiate the training.
def _train_chronos(
        self, training_data,
        probability: Optional[str] = None,
        context_length: int = 512,
        prediction_length: int = 64,
        min_past: int = 64,
        max_steps: int = 200_000,
        save_steps: int = 50_000,
        log_steps: int = 500,
        per_device_train_batch_size: int = 32,
        learning_rate: float = 1e-3,
        optim: str = "adamw_torch_fused",
        shuffle_buffer_length: int = 100,
        gradient_accumulation_steps: int = 2,
        model_id: str = "google/t5-efficient-tiny",
        model_type: str = "seq2seq",
        random_init: bool = False,
        tie_embeddings: bool = False,
        output_dir: str = "./output/",
        tf32: bool = True,
        torch_compile: bool = True,
        tokenizer_class: str = "MeanScaleUniformBins",
        tokenizer_kwargs: str = "{'low_limit': -15.0, 'high_limit': 15.0}",
        n_tokens: int = 4096,
        n_special_tokens: int = 2,
        pad_token_id: int = 0,
        eos_token_id: int = 1,
        use_eos_token: bool = True,
        lr_scheduler_type: str = "linear",
        warmup_ratio: float = 0.0,
        dataloader_num_workers: int = 1,
        max_missing_prop: float = 0.9,
        num_samples: int = 20,
        temperature: float = 1.0,
        top_k: int = 50,
        top_p: float = 1.0):

    # Set up logging for the train object.
    train.logger = getLogger()
    train.logger.setLevel(INFO)
    # Ensure the output_dir is a Path object.
    output_dir = Path(output_dir)
    # Convert probability from a string to a list, or set default if None.
    if isinstance(probability, str):
        probability = literal_eval(probability)
    elif probability is None:
        probability = [1.0 / len(training_data)] * len(training_data)
    # Convert the tokenizer_kwargs from a string to a dictionary.
    if isinstance(tokenizer_kwargs, str):
        tokenizer_kwargs = literal_eval(tokenizer_kwargs)
    # Enable reproducibility.
    set_seed(1, True)
    # Create datasets for training, filtered by criteria.
    train_datasets = [
```

```
            Filter(
                partial(
                    has_enough_observations,
                    min_length=min_past + prediction_length,
                    max_missing_prop=max_missing_prop,
                ),
                PandasDataset(data_frame, freq="D"),
            )
            for data_frame in training_data
        ]
        # Load the model with the specified configuration.
        model = load_model(
            model_id=model_id,
            model_type=model_type,
            vocab_size=n_tokens,
            random_init=random_init,
            tie_embeddings=tie_embeddings,
            pad_token_id=pad_token_id,
            eos_token_id=eos_token_id,
        )
        # Define the configuration for the Chronos tokenizer and other settings.
        chronos_config = ChronosConfig(
            tokenizer_class=tokenizer_class,
            tokenizer_kwargs=tokenizer_kwargs,
            n_tokens=n_tokens,
            n_special_tokens=n_special_tokens,
            pad_token_id=pad_token_id,
            eos_token_id=eos_token_id,
            use_eos_token=use_eos_token,
            model_type=model_type,
            context_length=context_length,
            prediction_length=prediction_length,
            num_samples=num_samples,
            temperature=temperature,
            top_k=top_k,
            top_p=top_p,
        )

        # Add extra items to model config so that it's saved in the ckpt.
        model.config.chronos_config = chronos_config.__dict__
        # Create a shuffled training dataset with the specified parameters.
        shuffled_train_dataset = ChronosDataset(
            datasets=train_datasets,
            probabilities=probability,
            tokenizer=chronos_config.create_tokenizer(),
            context_length=context_length,
            prediction_length=prediction_length,
            min_past=min_past,
            mode="training",
        ).shuffle(shuffle_buffer_length=shuffle_buffer_length)

        # Define the training arguments.
        training_args = TrainingArguments(
            output_dir=str(output_dir),
            per_device_train_batch_size=per_device_train_batch_size,
            learning_rate=learning_rate,
            lr_scheduler_type=lr_scheduler_type,
            warmup_ratio=warmup_ratio,
            optim=optim,
            logging_dir=str(output_dir / "train-logs"),
            logging_strategy="steps",
            logging_steps=log_steps,
            save_strategy="steps",
            save_steps=save_steps,
            report_to=["tensorboard"],
            max_steps=max_steps,
            gradient_accumulation_steps=gradient_accumulation_steps,
            dataloader_num_workers=dataloader_num_workers,
            tf32=tf32,  # remove this if not using Ampere GPUs (e.g., A100)
            torch_compile=torch_compile,
            ddp_find_unused_parameters=False,
            remove_unused_columns=False,
        )

        # Create a Trainer instance for training the model.
        trainer = Trainer(
            model=model,
            args=training_args,
            train_dataset=shuffled_train_dataset,
        )
        # Start the training process.
        trainer.train()
        # Save the trained model to the output directory.
        model.save_pretrained(output_dir)
        # Return the path to the output directory.
        return output_dir
```

4. Create DataFrame(s) that contain your training samples.

   Each DataFrame should have one column, named "target". The rows throughout the DataFrame should span a consistent time. If there is no

data for the time step (for example, the weekend price of an Equity asset), put NaN as the value for the time step. The following example demonstrates how to create DataFrames that contain one year of trailing prices for a set of Equities:

```
# Get historical equity curves.
history = self.history(symbols, timedelta(365), Resolution.DAILY)['close'].unstack(0)

# Gather the training data.
training_data_by_symbol = {}
for symbol in symbols:
    df = history[[symbol]].dropna()
    if df.shape[0] < 10: # Skip this asset if there is very little data.
        continue
    adjusted_df = df.reset_index()[['time', symbol]]
    adjusted_df = adjusted_df.rename(columns={str(symbol.id): 'target'})
    adjusted_df['time'] = pd.to_datetime(adjusted_df['time'])
    adjusted_df.set_index('time', inplace=True)
    adjusted_df = adjusted_df.resample('D').asfreq()
    training_data_by_symbol[symbol] = adjusted_df
```

For more information about history requests, see History Requests .

5. Call the fine-tuning method with the training data.

```
# Train the model with set parameters and save it to the output directory.
output_dir_path = self._train_chronos(
    list(training_data_by_symbol.values()),
    context_length=int(252/2), # 6 months
    prediction_length=self._prediction_length,
    optim=self._optimizer,
    model_id=self._model_name,
    output_dir=self._model_path,
    learning_rate=1e-5,
    # Requires Ampere GPUs (e.g., A100)
    tf32=False,
    max_steps=3
)
```

6. Load the fine-tuned model.

```
# Load the trained model from the output directory.
pipeline = ChronosPipeline.from_pretrained(
    output_dir_path,
    device_map=self._device_map,
    torch_dtype=torch.bfloat16,
)
```

Forecast Time Series

Follow these steps to forecast time series with Chronos-T5:

1. Get some historical data .

```
# Get historical closing prices.
history = self.history(symbols, timedelta(365), Resolution.DAILY)['close'].unstack(0)
```

2. Load the pre-trained or fine-tuned model.
3. Forecast the future time series.

```
# Generate forecasts for the assets using the trained pipeline.
all_forecasts = pipeline.predict(
    [
        torch.tensor(history[symbol].dropna())
        for symbol in symbols
    ],
    self._prediction_length
)
```

In this example, the model returns several future price paths for each asset.

4. Aggregate the future prices paths of each asset into a single price path for each asset.

   For example, take the median value of each time step in the future price paths.

```
# Create a DataFrame with median forecasts for each asset.
forecasts_df = pd.DataFrame(
    {
        symbol: np.quantile(
            all_forecasts[i].numpy(), 0.5, axis=0   # 0.5 = median
        )
        for i, symbol in enumerate(symbols)
    }
)
```

Examples

The following examples demonstrate usage of the Chronos-T5 model.

Example 1: Price Prediction

The following algorithm selects the most liquid assets at the beginning of each month. Once a quarter, it gets the trailing year of prices for all the assets in the universe and then forecasts the prices paths of all the assets over the upcoming quarter. It then uses the SciPy package to find the weights that maximize the future Sharpe ratio of the portfolio and rebalances the portfolio to those weights.

```
import torch
from chronos import ChronosPipeline
from scipy.optimize import minimize
from transformers import set_seed
# endregion

class HuggingFaceBaseModelDemo(QCAlgorithm):
    """
    This algorithm demonstrates how to use a pre-trained HuggingFace
    model. It uses the "amazon/chronos-t5-tiny" model to forecast the
    future equity curves of the 5 most liquid assets in the market,
    then it uses the SciPy package to find the portfolio weights
    that will maximize the future Sharpe ratio of the portfolio.
    The portfolio is rebalanced every 3 months.
    """

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.settings.min_absolute_portfolio_target_percentage = 0

        # Enable reproducibility.
        set_seed(1, True)

        # Load the pre-trained model.
        self._pipeline = ChronosPipeline.from_pretrained(
            "amazon/chronos-t5-tiny",
            device_map="cuda" if torch.cuda.is_available() else "cpu",
            torch_dtype=torch.bfloat16,
        )

        # Define the universe.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(
            self.universe.top(
                self.get_parameter('universe_size', 5)
            )
        )

        # Define some trading parameters.
        self._lookback_period = timedelta(
            365 * self.get_parameter('lookback_years', 1)
        )
        self._prediction_length = 3*21  # Three months of trading days

        # Schedule rebalances.
        self._last_rebalance = datetime.min
        self.schedule.on(
            self.date_rules.month_start(spy, 1),
            self.time_rules.midnight,
            self._trade
        )

        # Add warm up so the algorithm trades on deployment.
        self.set_warmup(timedelta(31))
```

```
    def _sharpe_ratio(
            self, weights, returns, risk_free_rate, trading_days_per_year=252):
        # Define how to calculate the Sharpe ratio so we can use
        # it to optimize the portfolio weights.

        # Calculate the annualized returns and covariance matrix.
        mean_returns = returns.mean() * trading_days_per_year
        cov_matrix = returns.cov() * trading_days_per_year

        # Calculate the Sharpe ratio.
        portfolio_return = np.sum(mean_returns * weights)
        portfolio_std = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
        sharpe_ratio = (portfolio_return - risk_free_rate) / portfolio_std

        # Return negative Sharpe ratio because we minimize this
        # function in optimization.
        return -sharpe_ratio

    def _optimize_portfolio(self, equity_curves):
        returns = equity_curves.pct_change().dropna()
        num_assets = returns.shape[1]
        initial_guess = num_assets * [1. / num_assets,]
        # Find portfolio weights that mazimize the forward Sharpe
        # ratio.
        result = minimize(
            self._sharpe_ratio,
            initial_guess,
            args=(
                returns,
                self.risk_free_interest_rate_model.get_interest_rate(self.time)
            ),
            method='SLSQP',
            bounds=tuple((0, 1) for _ in range(num_assets)),
            constraints=(
                {'type': 'eq', 'fun': lambda weights: np.sum(weights) - 1}
            )
        )
        return result.x

    def _trade(self):
        # Don't rebalance during warm-up.
        if self.is_warming_up:
            return
        # Only rebalance on a quarterly basis.
        if self.time - self._last_rebalance < timedelta(80):
            return
        self._last_rebalance = self.time

        symbols = list(self._universe.selected)

        # Get historical equity curves.
        history = self.history(symbols, self._lookback_period)['close'].unstack(0)

        # Forecast the future equity curves.
        all_forecasts = self._pipeline.predict(
            [
                torch.tensor(history[symbol].dropna())
                for symbol in symbols
            ],
            self._prediction_length
        )

        # Take the median forecast for each asset.
        forecasts_df = pd.DataFrame(
            {
                symbol: np.quantile(
                    all_forecasts[i].numpy(), 0.5, axis=0   # 0.5 = median
                )
                for i, symbol in enumerate(symbols)
            }
        )

        # Find the weights that maximize the forward Sharpe
        # ratio of the portfolio.
        optimal_weights = self._optimize_portfolio(forecasts_df)

        # Rebalance the portfolio.
        self.set_holdings(
            [
                PortfolioTarget(symbol, optimal_weights[i])
                for i, symbol in enumerate(symbols)
            ],
            True
        )
```

Example 2: Fine Tuning

The following algorithm expands the preceding algorithm by fine-tuning the model before each forecast:

PY

```
import torch
from scipy.optimize import minimize
from ast import literal_eval
from pathlib import Path
from functools import partial
from typing import List, Iterator, Optional, Dict
from torch.utils.data import IterableDataset, get_worker_info
from transformers import Trainer, TrainingArguments, set_seed
from gluonts.dataset.pandas import PandasDataset
from gluonts.itertools import Filter
from chronos import ChronosConfig, ChronosPipeline
from chronos.scripts.training.train import ChronosDataset, has_enough_observations, load_model
from chronos.scripts.training import train
from logging import getLogger, INFO
# endregion

class HuggingFaceFineTunedDemo(QCAlgorithm):
    """
    This algorithm demonstrates how to fine-tune a HuggingFace model.
    It uses the "amazon/chronos-t5-tiny" model to forecast the
    future equity curves of the 5 most liquid assets in the market,
    then it uses the SciPy package to find the portfolio weights
    that will maximize the future Sharpe ratio of the portfolio.
    The model is retrained and the portfolio is rebalanced every 3
    months.
    """

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.settings.daily_precise_end_time = False
        self.settings.min_absolute_portfolio_target_percentage = 0

        # Define the universe.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(
            self.universe.top(
                self.get_parameter('universe_size', 5)
            )
        )

        # Define some trading parameters.
        self._lookback_period = timedelta(
            365 * self.get_parameter('lookback_years', 1)
        )
        self._prediction_length = 3*21  # Three months of trading days

        # Schedule rebalances.
        self._last_rebalance = datetime.min
        self.schedule.on(
            self.date_rules.month_start(spy, 1),
            self.time_rules.midnight,
            self._trade
        )

        # Add warm up so the algorithm trades on deployment.
        self.set_warm_up(timedelta(31))

        # Define the model and some of its settings.
        self._device_map = "cuda" if torch.cuda.is_available() else "cpu"
        self._optimizer = 'adamw_torch_fused' if torch.cuda.is_available() else 'adamw_torch'
        self._model_name = "amazon/chronos-t5-tiny"
        self._model_path = self.object_store.get_file_path(
            f"{self.project_id}/llm/fine-tune/{self._model_name.replace('/', '-')}/"
        )

    def _sharpe_ratio(
            self, weights, returns, risk_free_rate, trading_days_per_year=252):
        # Define how to calculate the Sharpe ratio so we can use
        # it to optimize the portfolio weights.

        # Calculate the annualized returns and covariance matrix.
        mean_returns = returns.mean() * trading_days_per_year
        cov_matrix = returns.cov() * trading_days_per_year

        # Calculate the Sharpe ratio.
        portfolio_return = np.sum(mean_returns * weights)
        portfolio_std = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
        sharpe_ratio = (portfolio_return - risk_free_rate) / portfolio_std

        # Return negative Sharpe ratio because we minimize this
        # function in optimization.
        return -sharpe_ratio

    def _optimize_portfolio(self, equity_curves):
        returns = equity_curves.pct_change().dropna()
        num_assets = returns.shape[1]
        initial_guess = num_assets * [1. / num_assets,]
        # Find portfolio weights that mazimize the forward Sharpe
```

```
        # ratio.
        result = minimize(
            self._sharpe_ratio,
            initial_guess,
            args=(
                returns,
                self.risk_free_interest_rate_model.get_interest_rate(self.time)
            ),
            method='SLSQP',
            bounds=tuple((0, 1) for _ in range(num_assets)),
            constraints=(
                {'type': 'eq', 'fun': lambda weights: np.sum(weights) - 1}
            )
        )
        return result.x

    def _trade(self):
        # Don't rebalance during warm-up.
        if self.is_warming_up:
            return
        # Only rebalance on a quarterly basis.
        if self.time - self._last_rebalance < timedelta(80):
            return
        self._last_rebalance = self.time

        symbols = list(self._universe.selected)

        # Get historical equity curves.
        history = self.history(symbols, self._lookback_period)['close'].unstack(0)

        # Gather the training data.
        training_data_by_symbol = {}
        for symbol in symbols:
            df = history[[symbol]].dropna()
            if df.shape[0] < 10: # Skip this asset if there is very little data
                continue
            adjusted_df = df.reset_index()[['time', symbol]]
            adjusted_df = adjusted_df.rename(columns={symbol: 'target'})
            adjusted_df['time'] = pd.to_datetime(adjusted_df['time'])
            adjusted_df.set_index('time', inplace=True)
            adjusted_df = adjusted_df.resample('D').asfreq()
            training_data_by_symbol[symbol] = adjusted_df
        tradable_symbols = list(training_data_by_symbol.keys())

        # Fine-tune the model.
        output_dir_path = self._train_chronos(
            list(training_data_by_symbol.values()),
            context_length=int(252/2), # 6 months
            prediction_length=self._prediction_length,
            optim=self._optimizer,
            model_id=self._model_name,
            output_dir=self._model_path,
            learning_rate=1e-5,
            # Requires Ampere GPUs (e.g., A100)
            tf32=False,
            max_steps=3
        )

        # Load the fine-tuned model.
        pipeline = ChronosPipeline.from_pretrained(
            output_dir_path,
            device_map=self._device_map,
            torch_dtype=torch.bfloat16,
        )

        # Forecast the future equity curves.
        all_forecasts = pipeline.predict(
            [
                torch.tensor(history[symbol].dropna())
                for symbol in tradable_symbols
            ],
            self._prediction_length
        )

        # Take the median forecast for each asset.
        forecasts_df = pd.DataFrame(
            {
                symbol: np.quantile(
                    all_forecasts[i].numpy(), 0.5, axis=0   # 0.5 = median
                )
                for i, symbol in enumerate(tradable_symbols)
            }
        )

        # Find the weights that maximize the forward Sharpe
        # ratio of the portfolio.
        optimal_weights = self._optimize_portfolio(forecasts_df)

        # Rebalance the portfolio.
        self.set_holdings(
            [
                PortfolioTarget(symbol, optimal_weights[i])
                for i, symbol in enumerate(tradable_symbols)
            ],
```

```
            1,
            True
        )

    def _train_chronos(
            self, training_data,
            probability: Optional[str] = None,
            context_length: int = 512,
            prediction_length: int = 64,
            min_past: int = 64,
            max_steps: int = 200_000,
            save_steps: int = 50_000,
            log_steps: int = 500,
            per_device_train_batch_size: int = 32,
            learning_rate: float = 1e-3,
            optim: str = "adamw_torch_fused",
            shuffle_buffer_length: int = 100,
            gradient_accumulation_steps: int = 2,
            model_id: str = "google/t5-efficient-tiny",
            model_type: str = "seq2seq",
            random_init: bool = False,
            tie_embeddings: bool = False,
            output_dir: str = "./output/",
            tf32: bool = True,
            torch_compile: bool = True,
            tokenizer_class: str = "MeanScaleUniformBins",
            tokenizer_kwargs: str = "{'low_limit': -15.0, 'high_limit': 15.0}",
            n_tokens: int = 4096,
            n_special_tokens: int = 2,
            pad_token_id: int = 0,
            eos_token_id: int = 1,
            use_eos_token: bool = True,
            lr_scheduler_type: str = "linear",
            warmup_ratio: float = 0.0,
            dataloader_num_workers: int = 0,
            max_missing_prop: float = 0.9,
            num_samples: int = 20,
            temperature: float = 1.0,
            top_k: int = 50,
            top_p: float = 1.0):

        # Set up logging for the train object.
        train.logger = getLogger()
        train.logger.setLevel(INFO)
        # Ensure output_dir is a Path object.
        output_dir = Path(output_dir)
        # Convert probability from string to a list, or set default if
        # None.
        if isinstance(probability, str):
            probability = literal_eval(probability)
        elif probability is None:
            probability = [1.0 / len(training_data)] * len(training_data)
        # Convert tokenizer_kwargs from string to a dictionary.
        if isinstance(tokenizer_kwargs, str):
            tokenizer_kwargs = literal_eval(tokenizer_kwargs)
        # Enable reproducibility.
        set_seed(1, True)
        # Create datasets for training, filtered by criteria.
        train_datasets = [
            Filter(
                partial(
                    has_enough_observations,
                    min_length=min_past + prediction_length,
                    max_missing_prop=max_missing_prop,
                ),
                PandasDataset(data_frame, freq="D"),
            )
            for data_frame in training_data
        ]
        # Load the model with the specified configuration.
        model = load_model(
            model_id=model_id,
            model_type=model_type,
            vocab_size=n_tokens,
            random_init=random_init,
            tie_embeddings=tie_embeddings,
            pad_token_id=pad_token_id,
            eos_token_id=eos_token_id,
        )
        # Define the configuration for the Chronos
        # tokenizer and other settings.
        chronos_config = ChronosConfig(
            tokenizer_class=tokenizer_class,
            tokenizer_kwargs=tokenizer_kwargs,
            n_tokens=n_tokens,
            n_special_tokens=n_special_tokens,
            pad_token_id=pad_token_id,
            eos_token_id=eos_token_id,
            use_eos_token=use_eos_token,
            model_type=model_type,
            context_length=context_length,
            prediction_length=prediction_length,
            num_samples=num_samples,
            temperature=temperature,
            top_k=top_k,
```

```
        top_k=top_k,
        top_p=top_p,
    )

    # Add extra items to model config so that
    # it's saved in the ckpt.
    model.config.chronos_config = chronos_config.__dict__
    # Create a shuffled training dataset with the
    # specified parameters.
    shuffled_train_dataset = ChronosDataset(
        datasets=train_datasets,
        probabilities=probability,
        tokenizer=chronos_config.create_tokenizer(),
        context_length=context_length,
        prediction_length=prediction_length,
        min_past=min_past,
        mode="training",
    ).shuffle(shuffle_buffer_length=shuffle_buffer_length)

    # Define the training arguments.
    training_args = TrainingArguments(
        output_dir=str(output_dir),
        per_device_train_batch_size=per_device_train_batch_size,
        learning_rate=learning_rate,
        lr_scheduler_type=lr_scheduler_type,
        warmup_ratio=warmup_ratio,
        optim=optim,
        logging_dir=str(output_dir / "train-logs"),
        logging_strategy="steps",
        logging_steps=log_steps,
        save_strategy="steps",
        save_steps=save_steps,
        report_to=["tensorboard"],
        max_steps=max_steps,
        gradient_accumulation_steps=gradient_accumulation_steps,
        dataloader_num_workers=dataloader_num_workers,
        tf32=tf32,  # remove this if not using Ampere GPUs (e.g., A100)
        torch_compile=torch_compile,
        ddp_find_unused_parameters=False,
        remove_unused_columns=False,
    )

    # Create a Trainer instance for training the model.
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=shuffled_train_dataset,
    )
    # Start the training process.
    trainer.train()
    # Save the trained model to the output directory.
    model.save_pretrained(output_dir)
    # Return the path to the output directory.
    return output_dir
```

Popular Models

DistilBERT

Introduction

This page explains how to use DistilBERT in LEAN trading algorithms. The model repository provides the following description:

> The DistilBERT model was proposed in the blog post Smaller, faster, cheaper, lighter: Introducing DistilBERT, adistilled version of BERT , and the paper DistilBERT, adistilled version of BERT: smaller, faster, cheaper and lighter . DistilBERT is a small, fast, cheap and light Transformer model trained by distilling BERT base. It has 40% less parameters than bert-base-uncased, runs 60% faster while preserving over 95% of BERT's performances as measured on the GLUE language understanding benchmark. This model is a fine-tune checkpoint of DistilBERT-base-cased , fine-tuned using (a second step of) knowledge distillation on SQuAD v1.1 .

Use Cases

The DistilBERT model is a question answering model. The following use cases explain how you might utilize it in trading algorithms:

- Summarize the upcoming changes reported through regulatory articles in preparation of sentiment analysis.
- Extract relevant information from news feeds like Tiingo and Benzinga while reducing noise.
- Parse through US SEC Filings to find critical information about company performance, legal issues, or changes in management.

Load Pre-Trained Model

Follow these steps to load the pre-trained DistilBERT model:

1. Import the pipeline.

```
PY
from transformers import pipeline
```

2. Create a pipeline object with the DistilBERT model.

   In QuantConnect Cloud, the path is distilbert / distilbert-base-cased-distilled-squad .

```
PY
question_answerer = pipeline(
    "question-answering",
    model='distilbert/distilbert-base-cased-distilled-squad',
    local_files_only=True
)
```

Answer Questions

Follow these steps to answer questions with DistilBERT:

1. Load the model .
2. Define the context.

```
PY
context = """
Introduced on 2024-11-01.
The legislation introduces a flat corporate income tax rate, reducing the current graduated system
to 5.5% for taxable years starting from January 1, 2025, and further lowering it to 3.5% for years
beginning January 1, 2026. It establishes a bonus depreciation deduction for qualified property and
research expenditures, allowing immediate cost recovery. The bill also terminates various corporate
tax credits, exemptions, and deductions, including those related to motion picture production,
research and development, and historic structure rehabilitation, with most credits expiring after
June 30, 2025. Additionally, it repeals provisions for certain tax programs and credits, such as the
corporate tax apportionment program and various incentives for businesses. The changes aim to simplify
corporate taxation and reduce the number of available tax credits, with the new provisions effective
for income tax periods starting January 1, 2025, and franchise tax periods beginning January 1, 2026.
The legislation will take effect upon the governor's signature or after the designated period for
gubernatorial action.
"""
```

3. Create the question.

```py
question = "What do these changes accomplish?"
```

4. Pass the question and context to the pipeline.

```py
result = question_answerer(question=question, context=context)
```

The following dictionary is an example result:

```py
{
    'score': 0.19066356122493744,
    'start': 820,
    'end': 895,
    'answer': 'simplify corporate taxation and reduce the number of available tax credits'
}
```

Examples

Popular Models

FinBERT

Introduction

This page explains how to use FinBERT in LEAN trading algorithms. The model repository provides the following description:

> FinBERT is a pre-trained NLP model to analyze sentiment of financial text. It is built by further training the BERT language model in the finance domain, using a large financial corpus and thereby fine-tuning it for financial sentiment classification. Financial PhraseBank by Malo et al. (2014) is used for fine-tuning. For more details, please see the paper FinBERT: Financial Sentiment Analysis with Pre-trained Language Models and our related blog post on Medium.
>
> The model will give softmax outputs for three labels: positive, negative or neutral.

Use Cases

The FinBERT model is a sentiment analysis model. The following use cases explain how you might utilize it in trading algorithms:

- Analyze the sentiment of the latest news articles for specific companies , then form a long-short portfolio with assets that have the most postive and most negative news sentiment.
- Monitor the sentiment of regulatory alerts in a risk management model and liquidate holdings when sentiment is extremely negative.
- Generate sentences based on information from other datasets and then feed them into the model to determine the sentiment. For example, you could use the US Government Contracts dataset to create the string "The Department of State grants AAPL a contract for the purchase of mobile phones".

Load Pre-Trained Model

Follow these steps to load the pre-trained FinBERT model:

1. Import the model and tokenizer classes.

```
PY
from transformers import TFBertForSequenceClassification, BertTokenizer
```

2. Define the path were the model is stored.

   In QuantConnect Cloud, the path is ProsusAI / finbert .

```
PY
model_path = "ProsusAI/finbert"
```

3. Create a TFBertForSequenceClassification model.

```
PY
self._model = TFBertForSequenceClassification.from_pretrained(model_path, local_files_only=True)
```

4. Create a BertTokenizer object.

```
PY
self._tokenizer = BertTokenizer.from_pretrained(model_path, local_files_only=True)
```

5. (Optional) Set the seed to enable reproducibility.

```
PY
from transformers import set_seed
set_seed(1, True)
```

Fine-Tune Model

The FinBERT model is pre-trained, so you don't need to fine-tune it. Fine-tuning the model just tailors it to your specific use case. Follow these steps to fine-tune it:

1. Import the `Dataset` class.

```py
from datasets import Dataset
```

2. Load the pre-trained model .
3. Compile the model with an optimizer and loss function.

```py
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=3e-5),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
)
```

   You only need to compile the model once. TensorFlow offers many other optimizer and loss classes you can use.

4. Create a DataFrame that contains your training samples.

   The DataFrame should have two columns, named "text" and "label". The label must be an integer that represents the sentiment class. By default, the FinBERT model has three classes. Class zero represents negative sentiment, class one represents neutral sentiment, and class two represents positive sentiment.

```py
samples = pd.DataFrame(columns=['text', 'label'])
# Add rows to the DataFrame...
```

5. Convert the samples DataFrame to a `tf.data.Dataset` object.

```py
dataset = Dataset.from_pandas(samples)
```

6. Tokenize the text in each training sample.

```py
dataset = dataset.map(
    lambda sample: self._tokenizer(sample['text'], padding='max_length', truncation=True)
)
```

7. Call the model's prepare_tf_dataset method.

```py
dataset = model.prepare_tf_dataset(dataset, shuffle=True, tokenizer=self._tokenizer)
```

8. Call the model's fit method.

```py
model.fit(dataset, epochs=2)
```

Analyze Sentiment

Follow these steps to analyze the sentiment of some text with FinBERT:

1. Load the model .
2. (Optional) Fine-tune the model .
3. Get the text you want the model to analyze.

   The model can analyze a single sentence or a list of sentences.

```
content = "AAPL stock price spikes after record-breaking sales figures."
```

4. Tokenize the text(s).

```
inputs = self._tokenizer(content, padding=True, truncation=True, return_tensors='tf')
```

For more information about how to tokenize, see the PreTrainedTokenizer.__call__ reference on the Hugging Face website.

5. Perform dictionary unpacking on the preceding result and pass it to the model as input.

```
outputs = self._model(**inputs)
```

6. Apply softmax to the outputs to get probabilities of each sentiment class.

```
scores = tf.nn.softmax(outputs.logits, axis=-1).numpy()
```

The result of the preceding operation is a two dimensional numpy array. Each element in the two dimensional array is a list that contains the probability that the sentiment of the corresponding sentence is negative, neutral, or postiive, respectively. For example, you may get the following result if you use a single sentence as input. The result shows that the input is more positive than negative, but is likely neutral.

```
array([[0.21346861, 0.46771246, 0.318819]])
```

Examples

The following examples demonstrate usage of the FinBERT model.

Example 1: 10-day Sentiment Score

The following algorithm selects a volatile asset at the beginning of each month. It gets the Tiingo News articles that were released for the asset over the previous 10 days and then feeds them into the pre-trained FinBERT model. It then aggregates the sentiment scores of all the news releases. If the aggregated sentiment score is positive, it enters a long position for the month. If it's negative, it enters a short position for the month.

```
import tensorflow as tf
from transformers import TFBertForSequenceClassification, BertTokenizer, set_seed
from pathlib import Path

class FinbertBaseModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start("SPY"))
        self._universe = self.add_universe(
            lambda fundamental: [
                self.history(
                    [f.symbol for f in sorted(fundamental, key=lambda f: f.dollar_volume)[-10:]],
                    timedelta(365), Resolution.DAILY
                )['close'].unstack(0).pct_change().iloc[1:].std().idxmax()
            ]
        )

        set_seed(1, True)

        # Load the tokenizer and the model
        model_path = "ProsusAI/finbert"
        self._tokenizer = BertTokenizer.from_pretrained(model_path, local_files_only=True)
        self._model = TFBertForSequenceClassification.from_pretrained(model_path, local_files_only=True)

        self._last_rebalance_time = datetime.min
        self.set_warm_up(30, Resolution.DAILY)

    def on_warmup_finished(self):
        self._trade()
```

```
        self.schedule.on(self.date_rules.month_start("SPY", 1), self.time_rules.midnight, self._trade)

    def on_securities_changed(self, changes):
        for security in changes.removed_securities:
            self.remove_security(security.dataset_symbol)
        for security in changes.added_securities:
            security.dataset_symbol = self.add_data(TiingoNews, security.symbol).symbol

    def _trade(self):
        if self.is_warming_up or self.time - self._last_rebalance_time < timedelta(14):
            return

        # Get the target security.
        security = self.securities[list(self._universe.selected)[0]]

        # Get the latest news articles.
        articles = self.history[TiingoNews](security.dataset_symbol, 10, Resolution.DAILY)
        article_text = [article.description for article in articles]
        if not article_text:
            return

        # Prepare the input sentences
        inputs = self._tokenizer(article_text, padding=True, truncation=True, return_tensors='tf')

        # Get the model outputs
        outputs = self._model(**inputs)

        # Apply softmax to the outputs to get probabilities
        scores = tf.nn.softmax(outputs.logits, axis=-1).numpy()
        self.log(f"{str(scores)}")
        scores = self._aggregate_sentiment_scores(scores)

        self.plot("Sentiment Probability", "Negative", scores[0])
        self.plot("Sentiment Probability", "Neutral", scores[1])
        self.plot("Sentiment Probability", "Positive", scores[2])

        # Rebalance
        weight = 1 if scores[2] > scores[0] else -0.25
        self.set_holdings(security.symbol, weight, liquidate_existing_holdings=True)
        self._last_rebalance_time = self.time

    def _aggregate_sentiment_scores(self, sentiment_scores):
        n = sentiment_scores.shape[0]

        # Generate exponentially increasing weights
        weights = np.exp(np.linspace(0, 1, n))

        # Normalize weights to sum to 1
        weights /= weights.sum()

        # Apply weights to sentiment scores
        weighted_scores = sentiment_scores * weights[:, np.newaxis]

        # Aggregate weighted scores by summing them
        aggregated_scores = weighted_scores.sum(axis=0)

        return aggregated_scores
```

Example 2: 30-day Sentiment Score With Fine Tuning

The following algorithm selects a volatile asset at the beginning of each month. It gets the Tiingo News articles that were released for the asset over the previous 30 days to generate the training set. The label is the market return that occurs from the current news release to the next news release. The algorithm then fine-tunes the model, calculates the sentiment, and rebalances the portfolio.

```
PY
import tensorflow as tf
from transformers import TFBertForSequenceClassification, BertTokenizer, set_seed
from pathlib import Path
from datasets import Dataset
import pytz
import torch

class FinbertBaseModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self._universe = self.add_universe(
            lambda fundamental: [
                self.history(
                    [f.symbol for f in sorted(fundamental, key=lambda f: f.dollar_volume)[-10:]],
                    timedelta(365), Resolution.DAILY
                )['close'].unstack(0).pct_change().iloc[1:].std().idxmax()
            ]
```

```
        )

        set_seed(1, True)

        self._last_rebalance_time = datetime.min
        self.schedule.on(
            self.date_rules.month_start(spy, 1),
            self.time_rules.midnight,
            self._trade
        )

        self.set_warm_up(timedelta(30))

        self._model_name = "ProsusAI/finbert"
        self._tokenizer = BertTokenizer.from_pretrained(self._model_name)

    def on_warmup_finished(self):
        self._trade()

    def on_securities_changed(self, changes):
        for security in changes.removed_securities:
            self.remove_security(security.dataset_symbol)
        for security in changes.added_securities:
            security.dataset_symbol = self.add_data(
                TiingoNews, security.symbol
            ).symbol

    def _trade(self):
        if (self.is_warming_up or
            self.time - self._last_rebalance_time < timedelta(14)):
            return

        # Get the target security.
        security = self.securities[list(self._universe.selected)[0]]

        # Get samples to fine-tune the model
        samples = pd.DataFrame(columns=['text', 'label'])
        news_history = self.history(security.dataset_symbol, 30, Resolution.DAILY)
        if news_history.empty:
            return
        news_history = news_history.loc[security.dataset_symbol]['description']
        asset_history = self.history(
            security.symbol, timedelta(30), Resolution.SECOND
        ).loc[security.symbol]['close']
        for i in range(len(news_history.index)-1):
            # Get factor (article description).
            factor = news_history.iloc[i]
            if not factor:
                continue

            # Get the label (the market reaction to the news, for now).
            release_time = self._convert_to_eastern(news_history.index[i])
            next_release_time = self._convert_to_eastern(news_history.index[i+1])
            reaction_period = asset_history[
                (asset_history.index > release_time) &
                (asset_history.index < next_release_time + timedelta(seconds=1))
            ]
            if reaction_period.empty:
                continue
            label = (
                (reaction_period.iloc[-1] - reaction_period.iloc[0])
                / reaction_period.iloc[0]
            )

            # Save the training sample.
            samples.loc[len(samples), :] = [factor, label]

        samples = samples.iloc[-100:]

        if samples.shape[0] < 10:
            self.liquidate()
            return

        # Classify the market reaction into positive/negative/neutral.
        # 75% of the most negative labels => class 0 (negative)
        # 75% of the most postiive labels => class 2 (positive)
        # Remaining labels                => class 1 (netural)
        sorted_samples = samples.sort_values(by='label', ascending=False).reset_index(drop=True)
        percent_signed = 0.75
        positive_cutoff = (
            int(percent_signed
            * len(sorted_samples[sorted_samples.label > 0]))
        )
        negative_cutoff = (
            len(sorted_samples)
            - int(percent_signed * len(sorted_samples[sorted_samples.label < 0]))
        )
        sorted_samples.loc[list(range(negative_cutoff, len(sorted_samples))), 'label'] = 0
        sorted_samples.loc[list(range(positive_cutoff, negative_cutoff)), 'label'] = 1
        sorted_samples.loc[list(range(0, positive_cutoff)), 'label'] = 2

        # Load the pre-trained model.
        model = TFBertForSequenceClassification.from_pretrained(
            self._model_name, num_labels=3, from_pt=True
```

```
            self._model_name, num_labels=3, from_pt=True
        )
        # Compile the model.
        model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=3e-5),
            loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
        )
        # Create the training dataset.
        dataset = Dataset.from_pandas(sorted_samples)
        dataset = dataset.map(
            lambda sample: self._tokenizer(
                sample['text'], padding='max_length', truncation=True
            )
        )
        dataset = model.prepare_tf_dataset(
            dataset, shuffle=True, tokenizer=self._tokenizer
        )
        # Train the model.
        model.fit(dataset, epochs=2)
        # Prepare the input sentences.
        inputs = self._tokenizer(
            list(samples['text'].values), padding=True, truncation=True,
            return_tensors='tf'
        )

        # Get the model outputs.
        outputs = model(**inputs)

        # Apply softmax to the outputs to get probabilities.
        scores = tf.nn.softmax(outputs.logits, axis=-1).numpy()
        scores = self._aggregate_sentiment_scores(scores)

        self.plot("Sentiment Probability", "Negative", scores[0])
        self.plot("Sentiment Probability", "Neutral", scores[1])
        self.plot("Sentiment Probability", "Positive", scores[2])

        # Rebalance.
        weight = 1 if scores[2] > scores[0] else -0.25
        self.set_holdings(security.symbol, weight, liquidate_existing_holdings=True)
        self._last_rebalance_time = self.time

    def _convert_to_eastern(self, dt):
        return dt.astimezone(pytz.timezone('US/Eastern')).replace(tzinfo=None)

    def _aggregate_sentiment_scores(self, sentiment_scores):
        n = sentiment_scores.shape[0]

        # Generate exponentially increasing weights
        weights = np.exp(np.linspace(0, 1, n))

        # Normalize weights to sum to 1
        weights /= weights.sum()

        # Apply weights to sentiment scores
        weighted_scores = sentiment_scores * weights[:, np.newaxis]

        # Aggregate weighted scores by summing them
        aggregated_scores = weighted_scores.sum(axis=0)

        return aggregated_scores
```

These algorithms require a GPU node.

Popular Models

Sentiment Analysis

Introduction

This page explains how to use Hugging Face sentiment analysis models in LEAN trading algorithms. These models classify financial text into sentiment categories like positive, negative, and neutral. The following models are available:

- ahmedrachid/FinancialBERT-Sentiment-Analysis — A BERT model fine-tuned on financial data for sentiment analysis, classifying text as positive, negative, or neutral.
- bardsai/finance-sentiment-fr-base — A French-language financial sentiment classification model.
- cardiffnlp/twitter-roberta-base-sentiment-latest — A RoBERTa model fine-tuned on tweets for sentiment analysis.
- mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis — A DistilRoBERTa model fine-tuned on financial news for sentiment classification.
- nickmuchi/deberta-v3-base-finetuned-finance-text-classification — A DeBERTa model fine-tuned for financial text classification.
- nickmuchi/distilroberta-finetuned-financial-text-classification — A DistilRoBERTa model fine-tuned for financial text classification.
- nickmuchi/sec-bert-finetuned-finance-classification — A SEC-BERT model fine-tuned for financial document classification.
- StephanAkkerman/FinTwitBERT-sentiment — A BERT model fine-tuned on financial tweets for sentiment classification.

All of these models accept text input and return classification labels with confidence scores. You can use them with the Hugging Face `transformers` library to analyze the sentiment of financial news and social media posts, then use the results to inform trading decisions.

Examples

The following examples demonstrate usage of Hugging Face sentiment analysis models.

Example 1: News Sentiment Trading

The following algorithm selects the most volatile asset at the beginning of each month. It gets the Tiingo News articles that were released for the asset over the previous 10 days and then feeds them into a sentiment analysis model. It aggregates the sentiment scores of all the news releases. If the aggregated sentiment is positive, it enters a long position for the month. If it's negative, it enters a short position. You can replace the model name with any of the sentiment analysis models listed on the introduction page.

PY

```
from transformers import pipeline, set_seed

class SentimentAnalysisModelAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start("SPY"))
        self._universe = self.add_universe(
            lambda fundamental: [
                self.history(
                    [f.symbol for f in sorted(
                        fundamental, key=lambda f: f.dollar_volume
                    )[-10:]],
                    timedelta(365), Resolution.DAILY
                )['close'].unstack(0).pct_change().iloc[1:].std().idxmax()
            ]
        )

        set_seed(1, True)

        # Load the sentiment analysis pipeline.
        # Replace the model name with any supported sentiment model.
        self._sentiment_pipeline = pipeline(
            "text-classification",
            model="mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis"
        )

        self._last_rebalance_time = datetime.min
        self.set_warm_up(30, Resolution.DAILY)

    def on_warmup_finished(self):
```

```
        self._trade()
        self.schedule.on(
            self.date_rules.month_start("SPY", 1),
            self.time_rules.midnight,
            self._trade
        )

    def on_securities_changed(self, changes):
        for security in changes.removed_securities:
            self.remove_security(security.dataset_symbol)
        for security in changes.added_securities:
            security.dataset_symbol = self.add_data(
                TiingoNews, security.symbol
            ).symbol

    def _trade(self):
        if (self.is_warming_up or
            self.time - self._last_rebalance_time < timedelta(14)):
            return

        # Get the target security.
        security = self.securities[list(self._universe.selected)[0]]

        # Get the latest news articles.
        articles = self.history[TiingoNews](
            security.dataset_symbol, 10, Resolution.DAILY
        )
        article_text = [
            article.description for article in articles
            if article.description
        ]
        if not article_text:
            return

        # Run sentiment analysis on each article.
        # Truncate long articles to the model's max length.
        results = self._sentiment_pipeline(
            article_text, truncation=True, max_length=512
        )

        # Aggregate sentiment scores.
        positive_score = 0
        negative_score = 0
        for result in results:
            label = result['label'].lower()
            score = result['score']
            if 'pos' in label:
                positive_score += score
            elif 'neg' in label:
                negative_score += score

        self.plot("Sentiment", "Positive", positive_score)
        self.plot("Sentiment", "Negative", negative_score)

        # Rebalance based on sentiment.
        weight = 1 if positive_score > negative_score else -0.25
        self.set_holdings(
            security.symbol, weight,
            liquidate_existing_holdings=True
        )
        self._last_rebalance_time = self.time
```

Popular Models

Fill-Mask

Introduction

This page explains how to use Hugging Face fill-mask models in LEAN trading algorithms. Fill-mask models predict the most likely word to fill a masked position in a sentence. You can use them to extract text embeddings and build feature vectors from financial text. The following models are available:

- google-bert/bert-base-uncased — The original BERT base model (uncased), widely used for natural language understanding tasks.
- distilbert/distilbert-base-uncased — A distilled version of BERT that is 60% faster while retaining 97% of BERT's language understanding.
- FacebookAI/roberta-base — A robustly optimized BERT pretraining approach by Facebook AI.
- microsoft/deberta-base — A DeBERTa model by Microsoft that uses disentangled attention for improved language understanding.

These models are useful for extracting text embeddings from financial news. You can feed these embeddings into a downstream classifier or use cosine similarity to measure the semantic similarity between documents.

Examples

The following examples demonstrate usage of Hugging Face fill-mask models for feature extraction.

Example 1: Embedding-Based News Similarity

The following algorithm selects a volatile asset at the beginning of each month. It uses a fill-mask model to extract embeddings from Tiingo News articles. It then compares the average embedding of recent news to a reference "bullish" and "bearish" embedding. If the recent news is more similar to the bullish reference, it enters a long position. You can replace the model name with any of the fill-mask models listed on the introduction page.

```py
import torch
import numpy as np
from transformers import AutoTokenizer, AutoModel, set_seed

class FillMaskEmbeddingAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.universe_settings.resolution = Resolution.DAILY
        self.universe_settings.schedule.on(self.date_rules.month_start("SPY"))
        self._universe = self.add_universe(
            lambda fundamental: [
                self.history(
                    [f.symbol for f in sorted(
                        fundamental, key=lambda f: f.dollar_volume
                    )[-10:]],
                    timedelta(365), Resolution.DAILY
                )['close'].unstack(0).pct_change().iloc[1:].std().idxmax()
            ]
        )

        set_seed(1, True)

        # Load the model and tokenizer.
        # Replace with any fill-mask model (e.g., google-bert/bert-base-uncased).
        model_name = "distilbert/distilbert-base-uncased"
        self._tokenizer = AutoTokenizer.from_pretrained(model_name)
        self._model = AutoModel.from_pretrained(model_name)
        self._model.eval()

        # Create reference embeddings for bullish/bearish text.
        self._bullish_embedding = self._get_embedding(
            "Stock prices surged on strong earnings and revenue growth."
        )
        self._bearish_embedding = self._get_embedding(
            "Stock prices plunged on weak earnings and declining revenue."
        )

        self._last_rebalance_time = datetime.min
        self.set_warm_up(30, Resolution.DAILY)
```

```
    def on_warmup_finished(self):
        self._trade()
        self.schedule.on(
            self.date_rules.month_start("SPY", 1),
            self.time_rules.midnight,
            self._trade
        )

    def on_securities_changed(self, changes):
        for security in changes.removed_securities:
            self.remove_security(security.dataset_symbol)
        for security in changes.added_securities:
            security.dataset_symbol = self.add_data(
                TiingoNews, security.symbol
            ).symbol

    def _get_embedding(self, text):
        """Extract the [CLS] token embedding from the model."""
        inputs = self._tokenizer(
            text, return_tensors="pt", truncation=True, max_length=512
        )
        with torch.no_grad():
            outputs = self._model(**inputs)
        # Use the [CLS] token (first token) embedding.
        return outputs.last_hidden_state[:, 0, :].squeeze().numpy()

    def _cosine_similarity(self, a, b):
        return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

    def _trade(self):
        if (self.is_warming_up or
            self.time - self._last_rebalance_time < timedelta(14)):
            return

        # Get the target security.
        security = self.securities[list(self._universe.selected)[0]]

        # Get the latest news articles.
        articles = self.history[TiingoNews](
            security.dataset_symbol, 10, Resolution.DAILY
        )
        article_text = [
            article.description for article in articles
            if article.description
        ]
        if not article_text:
            return

        # Get embeddings for each article and average them.
        embeddings = [self._get_embedding(text) for text in article_text]
        avg_embedding = np.mean(embeddings, axis=0)

        # Compare to reference embeddings.
        bullish_sim = self._cosine_similarity(
            avg_embedding, self._bullish_embedding
        )
        bearish_sim = self._cosine_similarity(
            avg_embedding, self._bearish_embedding
        )

        self.plot("Similarity", "Bullish", bullish_sim)
        self.plot("Similarity", "Bearish", bearish_sim)

        # Rebalance based on similarity.
        weight = 1 if bullish_sim > bearish_sim else -0.25
        self.set_holdings(
            security.symbol, weight,
            liquidate_existing_holdings=True
        )
        self._last_rebalance_time = self.time
```

Popular Models

Text Generation

Introduction

This page explains how to use Hugging Face text generation models in LEAN trading algorithms. These models generate text given an input prompt, which you can use for tasks like summarizing financial data or generating structured analysis. The following models are available:

- openai-community/gpt2 — The GPT-2 language model by OpenAI, a lightweight model suitable for text generation tasks.
- google/gemma-7b — A 7B parameter model from Google's Gemma family, offering strong text generation capabilities. Requires a GPU node.
- deepseek-ai/DeepSeek-R1-Distill-Llama-70B — A 70B parameter reasoning model distilled from DeepSeek-R1 into a Llama architecture. Requires a GPU node with significant memory.

Text generation models can analyze market context and generate structured outputs. You can prompt them to classify market conditions or extract trading signals from financial text. Note that larger models like Gemma-7B and DeepSeek-70B require GPU nodes with sufficient memory.

Examples

The following examples demonstrate usage of Hugging Face text generation models.

Example 1: GPT-2 Market Condition Classifier

The following algorithm uses GPT-2 to classify market conditions based on recent price data. At the beginning of each month, it calculates trailing returns, volatility, and momentum for the universe of the 5 most liquid assets. It then prompts GPT-2 to complete a structured market analysis template and parses the generated text to determine position sizing.

```py
from transformers import pipeline, set_seed

class GPT2MarketAnalysisAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.settings.min_absolute_portfolio_target_percentage = 0

        set_seed(1, True)

        # Load the text generation pipeline with GPT-2.
        self._generator = pipeline(
            "text-generation",
            model="openai-community/gpt2"
        )

        # Define the universe.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(
            self.universe.top(
                self.get_parameter('universe_size', 5)
            )
        )

        self._last_rebalance = datetime.min
        self.schedule.on(
            self.date_rules.month_start(spy, 1),
            self.time_rules.midnight,
            self._trade
        )
        self.set_warm_up(timedelta(31))

    def _trade(self):
        if self.is_warming_up:
            return
        if self.time - self._last_rebalance < timedelta(25):
            return
        self._last_rebalance = self.time

        symbols = list(self._universe.selected)
        if not symbols:
            return
```

```
        # Get trailing 60-day price data.
        history = self.history(
            symbols, 60, Resolution.DAILY
        )['close'].unstack(0)

        scores = {}
        for symbol in symbols:
            prices = history[symbol].dropna()
            if len(prices) < 20:
                continue

            # Calculate features.
            returns_20d = (prices.iloc[-1] / prices.iloc[-20] - 1) * 100
            volatility = prices.pct_change().std() * np.sqrt(252) * 100

            # Create a structured prompt.
            prompt = (
                f"Stock analysis: 20-day return {returns_20d:.1f}%, "
                f"annualized volatility {volatility:.1f}%. "
                f"Market outlook:"
            )

            # Generate text.
            result = self._generator(
                prompt, max_new_tokens=30, num_return_sequences=1,
                do_sample=True, temperature=0.7
            )
            generated = result[0]['generated_text'].lower()

            # Parse sentiment from generated text.
            bullish_words = ['bullish', 'growth', 'strong', 'positive', 'upward', 'buy', 'rally']
            bearish_words = ['bearish', 'decline', 'weak', 'negative', 'downward', 'sell', 'crash']

            bull_count = sum(1 for w in bullish_words if w in generated)
            bear_count = sum(1 for w in bearish_words if w in generated)

            # Combine model signal with momentum.
            momentum_signal = 1 if returns_20d > 0 else -1
            model_signal = bull_count - bear_count
            scores[symbol] = momentum_signal + model_signal * 0.5

        if not scores:
            return

        # Normalize scores to portfolio weights.
        total = sum(abs(v) for v in scores.values())
        if total == 0:
            return
        weights = {s: v / total for s, v in scores.items()}

        # Rebalance.
        self.set_holdings(
            [
                PortfolioTarget(symbol, weight)
                for symbol, weight in weights.items()
            ],
            True
        )
```

Popular Models

Chronos-Bolt

Introduction

This page explains how to use Chronos-Bolt in LEAN trading algorithms. The model repository provides the following description:

> Chronos-Bolt models are a family of **lightweight, efficient time series forecasting models** . They are a follow-up to the original Chronos models, designed for faster inference and lower computational cost. Chronos-Bolt uses a T5-based encoder-decoder architecture where the encoder processes the historical context and the decoder directly generates quantile forecasts. Unlike the original Chronos models, Chronos-Bolt does not use tokenization, resulting in significantly faster inference. For details, refer to the paper Chronos: Learning the Language of Time Series .

Examples

The following examples demonstrate usage of the Chronos-Bolt model.

Example 1: Price Prediction

The following algorithm selects the most liquid assets at the beginning of each month. Once a quarter, it gets the trailing year of prices for all the assets in the universe and then forecasts the price paths over the upcoming quarter using Chronos-Bolt. It then uses the SciPy package to find the weights that maximize the future Sharpe ratio of the portfolio and rebalances the portfolio to those weights. Chronos-Bolt is a faster variant of Chronos-T5 that directly generates quantile forecasts.

```py
import torch
import numpy as np
from scipy.optimize import minimize
from chronos import ChronosBoltPipeline
from transformers import set_seed
# endregion

class ChronosBoltAlgorithm(QCAlgorithm):
    """
    This algorithm demonstrates how to use the Chronos-Bolt time
    series forecasting model. It forecasts the future equity curves
    of the 5 most liquid assets, then finds portfolio weights that
    maximize the future Sharpe ratio. The portfolio is rebalanced
    every 3 months.
    """

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        self.settings.min_absolute_portfolio_target_percentage = 0

        set_seed(1, True)

        # Load the pre-trained Chronos-Bolt model.
        self._pipeline = ChronosBoltPipeline.from_pretrained(
            "autogluon/chronos-bolt-base",
            device_map="cuda" if torch.cuda.is_available() else "cpu",
            torch_dtype=torch.bfloat16,
        )

        # Define the universe.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.universe_settings.schedule.on(self.date_rules.month_start(spy))
        self.universe_settings.resolution = Resolution.DAILY
        self._universe = self.add_universe(
            self.universe.top(
                self.get_parameter('universe_size', 5)
            )
        )

        self._lookback_period = timedelta(
            365 * self.get_parameter('lookback_years', 1)
        )
        self._prediction_length = 3 * 21  # Three months of trading days

        # Schedule rebalances.
        self._last_rebalance = datetime.min
        self.schedule.on(
```

```
            self.date_rules.month_start(spy, 1),
            self.time_rules.midnight,
            self._trade
        )
        self.set_warmup(timedelta(31))

    def _sharpe_ratio(
            self, weights, returns, risk_free_rate,
            trading_days_per_year=252):
        mean_returns = returns.mean() * trading_days_per_year
        cov_matrix = returns.cov() * trading_days_per_year
        portfolio_return = np.sum(mean_returns * weights)
        portfolio_std = np.sqrt(
            np.dot(weights.T, np.dot(cov_matrix, weights))
        )
        sharpe_ratio = (portfolio_return - risk_free_rate) / portfolio_std
        return -sharpe_ratio

    def _optimize_portfolio(self, equity_curves):
        returns = equity_curves.pct_change().dropna()
        num_assets = returns.shape[1]
        initial_guess = num_assets * [1. / num_assets]
        result = minimize(
            self._sharpe_ratio,
            initial_guess,
            args=(
                returns,
                self.risk_free_interest_rate_model.get_interest_rate(
                    self.time
                )
            ),
            method='SLSQP',
            bounds=tuple((0, 1) for _ in range(num_assets)),
            constraints=(
                {'type': 'eq', 'fun': lambda w: np.sum(w) - 1}
            )
        )
        return result.x

    def _trade(self):
        if self.is_warming_up:
            return
        if self.time - self._last_rebalance < timedelta(80):
            return
        self._last_rebalance = self.time

        symbols = list(self._universe.selected)

        # Get historical equity curves.
        history = self.history(
            symbols, self._lookback_period
        )['close'].unstack(0)

        # Forecast the future equity curves using Chronos-Bolt.
        # predict() returns (num_series, num_samples, prediction_length).
        all_forecasts = self._pipeline.predict(
            [
                torch.tensor(history[symbol].dropna())
                for symbol in symbols
            ],
            self._prediction_length
        )

        # Take the median forecast for each asset.
        forecasts_df = pd.DataFrame(
            {
                symbol: np.quantile(
                    all_forecasts[i].numpy(), 0.5, axis=0
                )
                for i, symbol in enumerate(symbols)
            }
        )

        # Find the weights that maximize the forward Sharpe ratio.
        optimal_weights = self._optimize_portfolio(forecasts_df)

        # Rebalance the portfolio.
        self.set_holdings(
            [
                PortfolioTarget(symbol, optimal_weights[i])
                for i, symbol in enumerate(symbols)
            ],
            True
        )
```

Algorithm Framework

Algorithm Framework

Overview

Introduction

Since QuantConnect was formed in 2013, the community has created more than 2 million algorithms. Although they all seek to achieve different results, they can all be abstracted into the same core features. The Algorithm Framework is our attempt to provide this well-defined structure to the community, encouraging good design and making your work more reusable. LEAN provides many pre-built Algorithm Framework models you can use in your algorithms.

Strategy Styles

The algorithm framework works best for portfolio rebalancing, ranking, and multi-factor style strategies. The default portfolio construction models are designed to weigh and invest in the assets as the signals values shift, aiming to be fully invested as much as possible.

Some strategies might find it difficult to fit this model. Technical entry signals are often tightly coupled with their exit criteria, they may not have defined hold periods, and they might follow the market to determine when to exit. This style of strategy can be addressed in a few ways:

1. Short Term Signals - Emitting insights for short periods until they are no longer valid.
2. Cancel Signals - Updating an insight's expiry time so that it's expired upon the exit signal.
3. Alpha Exit Signal - Treating exit as its own signal, a separate Alpha model could indicate exit.
4. Risk Model - Tracking exit using a risk model that attempts to lock in gains.

These techniques are still not ideal for many strategies. For example, with only a single signal, the built in `EqualWeightingPortfolioConstructionModel` will allocate 100% of buying power to a single asset, over concentrating risk. If a second signal comes a few days later, it would sell 50% of the portfolio to free cash and invest in the second asset. This can create high trading costs through portfolio churn.

With a custom portfolio construction model , signals can mean what you need for your specific strategy style. For example, you might create a " `MultiBetPortfolioConstructionModel` " that uses each insight to open a specific "bet" that is separately allocated capital and tracked.

Important Terminology

The following table defines important Algorithm Framework terminology:

Term	Description
Universe Selection model	A framework module that selects assets for your algorithm.
Alpha model	A framework module that generates trading signals on the assets in your universe.
Insight	An object that represents a trading signal. The Alpha model generates these objects for the Portfolio Construction model.
Portfolio Construction model	A framework module that determines position size targets based on the `Insight` objects it receives from the Alpha model.
PortfolioTarget	An object that represents the target position size for an asset in the universe. The Portfolio Construction model generates these objects for the Risk Management model.
Risk Management	A framework module that manages market risks and ensures the algorithm remains within target parameters. This model adjusts the `PortfolioTarget` objects it receives from the Portfolio Construction model before they reach the Execution model.
Execution	A framework module that places trades to reach the target risk-adjust portfolio based on the `PortfolioTarget` objects it receives from the Risk Management model.

Framework Advantages

We recommend you design strategies with the Algorithm Framework in most cases. The Algorithm Framework has many advantages over the classic algorithm design.

Pluggable Algorithm Modules

With the Algorithm Framework, your code can instantly utilize all modules built within the framework. The modules clip together in well-defined ways, which enable them to be shared and swapped interchangeably.

Focus On Your Strengths

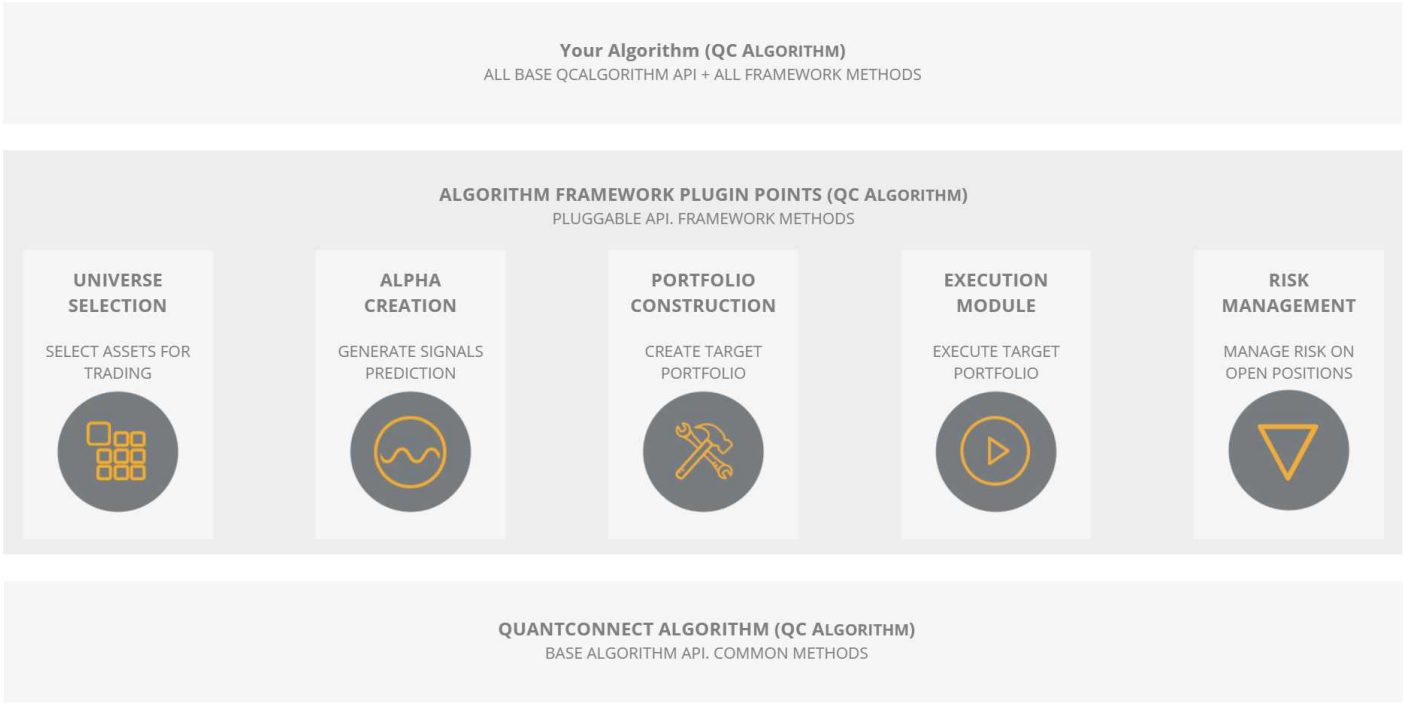
If you write code in modules, you can focus on your strengths. If you are great at building universes, you can build Universe Selection modules. If you have risk management experience, you could write reusable risk monitoring technology.

Reduce Development By Using Community Modules

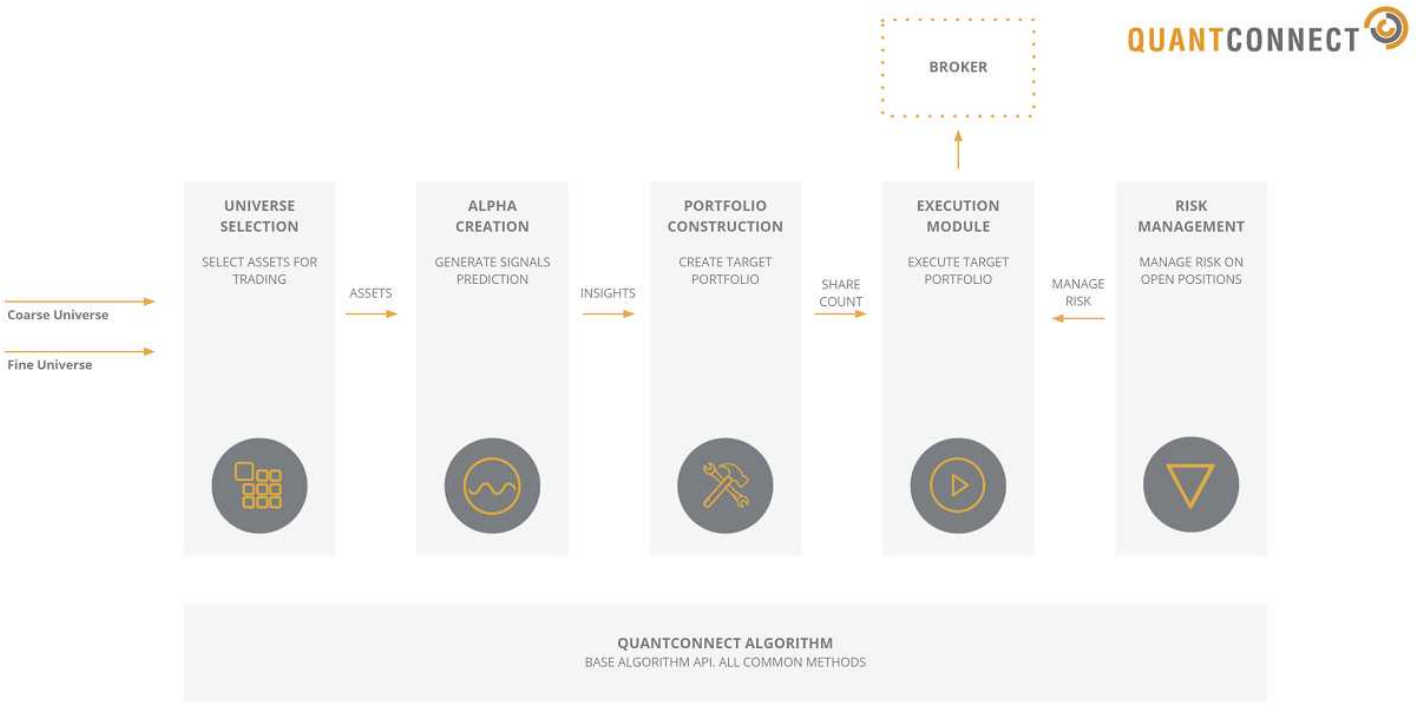
Easily share modules you've made between algorithms or pull in ones produced by the community. The strict separation of duties makes the Algorithm Framework perfect for reusing code between strategies.

System Architecture

The Algorithm Framework is built into the `QCAlgorithm` class, so your strategy can access all the normal methods you use for algorithm development. You should be able to copy and paste your algorithms across without any changes. You don't need a separate class.



The framework data output of each module flows into the following module. The assets that the Universe Selection model selects are fed into the Alpha model to generate trade signals (`Insight` objects). The `Insight` objects from the Alpha model are fed into the Portfolio Construction model to create `PortfolioTarget` objects, which contain the target number of units to hold for each asset. The `PortfolioTarget` objects from the Portfolio Construction model are fed into the Risk Management model to ensure the targets are within safe risk parameters and to adjust the `PortfolioTarget` objects if necessary. The `PortfolioTarget` objects from the Risk Management model are fed into the Execution model, which efficiently places trades to acquire the target portfolio.




```py
class MyFrameworkAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_universe_selection(EmaCrossUniverseSelectionModel())
        self.add_alpha(RsiAlphaModel())
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
        self.set_execution(ImmediateExecutionModel())
        self.add_risk_management(NullRiskManagementModel())
```

For simple strategies, it may seem like overkill to abstract out your algorithm concepts. However, even simple strategies can benefit from reusing the ecosystem of modules available in QuantConnect. Imagine pairing your EMA-cross Alpha model with a better execution system or plugging in an

open-source trailing stop Risk Management model .

Separation of Concerns

The separation of concerns principle is an algorithm design principle where individual components are isolated and their responsibilities don't overlap. The Algorithm Framework is designed to uphold this design principle. The individual framework components shouldn't rely on the state of the other framework components in order to operate.

In the Algorithm Framework, the models should not communicate. The universe constituents and the `Insight` objects you emit should be the same, regardless of what you select for the Portfolio Construction, Risk Management, and Execution models. If your algorithm logic naturally violates the separation of concerns design principle, it's more appropriate to use the classic or hybrid design .

Framework vs Classic Design

Select the classic or Algorithm Framework design based on the needs of your algorithm. If your algorithm uses special order types or passes information between the different algorithm components, use the classic design. If you can reuse existing modules and the Algorithm Framework modules can work in isolation, use the Algorithm Framework design.

Algorithm Framework

Universe Selection

Universe Selection

Key Concepts

Introduction

The Universe Selection model creates `Universe` objects, which select the assets for your algorithm. As the universe changes, we notify your algorithm through the `on_securities_changed` event handler. With this event handler, you can track the current universe constituents in other parts of your algorithm without breaking the separation of concerns design principle.

Types of Universe Selection

We have identified several types of universes that cover most people's requirements and built helper classes to make their implementation easier. The following table describes the types of pre-built Universe Selection models:

Universe Type	Description
Manual Universes	Universes that use a fixed, static set of assets
Fundamental Universes	Universes for US Equities that are based on coarse price or fundamental data
Scheduled Universes	Universes that trigger on regular, custom intervals
Futures Universes	Universes that subscribe to Future chains
Option Universes	Universes that subscribe to Option chains

Add Models

To add a Universe Selection model, in the `initialize` method, call the `AddUniverseSelection` method.

```
PY
# Adds a universe selection model to the algorithm, manually defining the set of securities to include in the algorithm.
symbols = [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
self.add_universe_selection(ManualUniverseSelectionModel(symbols))
```

To view all the pre-built Universe Selection models, see Types of Universe Selection .

Multi-Universe Algorithms

You can add multiple Universe Selection models to a single algorithm.

```
PY
# Configures asynchronous universe settings for faster updates and includes both manual and EMA cross universe selection models for
dynamic and static selection.
self.universe_settings.asynchronous = True
symbols = [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
self.add_universe_selection(ManualUniverseSelectionModel(symbols))

self.add_universe_selection(EmaCrossUniverseSelectionModel())
```

If you add multiple Universe Selection models, the algorithm subscribes to the constituents of all the models.

Model Structure

Universe Selection models should extend the `UniverseSelectionModel` class. Extensions of the `UniverseSelectionModel` class must implement the `create_universes` method, which receives an `algorithm` object and returns an array of `Universe` objects.

```py
# Extend UniverseSelectionModel to define a custom security universe tailored to the algorithm.
class MyUniverseSelectionModel(UniverseSelectionModel):

    # Creates the universes for this algorithm
    def create_universes(self, algorithm: QCAlgorithm) -> List[Universe]:
        universes = []
        return universes

    # Gets the next time the framework should invoke the `CreateUniverses` method to refresh the set of universes.
    def get_next_refresh_time_utc(self):
        return datetime.max
```

The `algorithm` argument that the methods receive is an instance of the base `QCAlgorithm` class, not your subclass of it.

Generally, you should be able to extend one of the pre-built Universe Selection types. If you need to do something that doesn't fit into the pre-built types, let us know and we'll create a new foundational type of Universe Selection model.

Track Security Changes

When the Universe Selection model adjusts the universe constituents, we notify your algorithm through the `on_securities_changed` event handler on your other framework components. The method receives `QCAlgorithm` and `SecurityChanges` objects. The `QCAlgorithm` object is an instance of the base QCAlgorithm class, not a reference to your algorithm object. To access the added securities, check the `changes.added_securities` method property. To access the removed securities, check the `changes.removed_securities` method property.

```py
# Log added and removed securities to update the universe and adapt the strategy.
def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        self.log(f"Added {security.symbol}")

    for security in changes.removed_securities:
        self.log(f"Removed {security.symbol}")
```

Live Trading Considerations

In live trading, the securities in your brokerage account are added to your user-defined universe. If your Universe Selection model doesn't select these securities, they are not removed from your user-defined universe. When you liquidate the positions, you can remove them from the user-defined universe. If you call the `remove_security` method, it automatically liquidates the position.

To see the securities in your user-defined universe that were loaded from your brokerage account, check your algorithm's active_securities in the on_warmup_finished event handler.

Examples

The following examples demonstrate some common practices for implementing the universe selection model.

Example 1: Liquid Universe Selection Model

The following algorithm selects a universe of the top 20 liquid equity through the framework universe selection model. Then, buying and holding these popular equities equally to bet on their high demand will drive up the price.

```py
class FrameworkUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Rebalance weekly to allow time to capitalize on the trend
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Since the selection is based on daily data, we only require daily data to trade daily.
        self.universe_settings.resolution = Resolution.DAILY
        # Add a universe selection model to select the most liquid stocks to trade with.
        self.add_universe_selection(FundamentalUniverseSelectionModel(self.selection))

        # Sent insights on buying and holding the most liquid stocks weekly.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Evenly dissipate the capital risk among selected stocks.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def selection(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select the top 20 liquid equity to trade the popularity, suggesting a higher demand to drive the price.
        sorted_by_dollar_volume = sorted(fundamental, key=lambda x: x.dollar_volume, reverse=True)
        return [x.symbol for x in sorted_by_dollar_volume[:20]]
```

Example 2: Multiple Universe Selection Models

The following algorithm implements two framework universe selection models to manually select the most liquid market ETFs and those with EMA crossing events to trade on trend changes. Then, buy and hold these equities equally to bet on the insights and dissipate the capital risk.

```py
class FrameworkUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Rebalance weekly to allow time to capitalize on the trend
        self.universe_settings.schedule.on(self.date_rules.week_start())
        # Since the selection is based on daily data, we only require daily data to trade daily.
        self.universe_settings.resolution = Resolution.DAILY
        # Add a universe selection model to select the most liquid ETFs manually.
        symbols = [
            Symbol.create("SPY", SecurityType.EQUITY, Market.USA),
            Symbol.create("QQQ", SecurityType.EQUITY, Market.USA)
        ]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))
        # Add another universe selection model to select the ones with EMA crosses.
        self.add_universe_selection(EmaCrossUniverseSelectionModel())

        # Sent insights on buying and holding the most liquid stocks weekly.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Evenly dissipate the capital risk among selected stocks.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
Manual Selection - BasicTemplateFrameworkAlgorithm.py Python Fundamental Selection - QC500UniverseSelectionModel.py Python Scheduled Selection - ScheduledUniverseSelectionModelRegressionAlgorithm.py Python

Universe Selection

Universe Settings

Introduction

Universe settings and security initializers enable you to configure some properties of the securities in a universe.

Resolution

The `resolution` setting defines the time period of the asset data. The `Resolution` enumeration has the following members:

To view which resolutions are available for the asset class of your universe, follow these steps:

1. Open the Asset Classes documentation page.
2. Click an asset class.
3. Click Requesting Data .
4. On the Requesting Data page, in the table of contents, click Resolutions .

The default value is `Resolution.MINUTE` . To change the resolution, in the Initialize method, adjust the algorithm's `UniverseSettings` before you create the Universe Selection model.

```
PY
# Use daily data and select SPY ETF constituents as the trading universe.
self.universe_settings.resolution = Resolution.DAILY
self.add_universe_selection(ETFConstituentsUniverseSelectionModel("SPY"))
```

Leverage

The `leverage` setting is a `float` that defines the maximum amount of leverage you can use for a single asset in a non-derivative universe. The default value is `Security.NULL_LEVERAGE` (0). To change the leverage, in the Initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

```
PY
# Set unviverse leverage to 2x to increase potential returns from each asset in the non-derivative universe to increase potential returns.
self.universe_settings.leverage = 2.0
# Select universe based on EMA cross signals to identify and trade assets with trending behaviors.
self.add_universe_selection(EmaCrossUniverseSelectionModel())
```

Fill Forward

The `fill_forward` setting is a `bool` that defines whether or not too fill forward data. The default value is `True` . To disable fill forward in non-derivative universes, in the Initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

```
PY
# Disable fill forward to ensure each day's data is used exclusively, preventing previous day's data from influencing decisions and
select options for SPY with a daily update interval to track specific option contracts.
self.universe_settings.fill_forward = False
self.add_universe_selection(
    OptionUniverseSelectionModel(
        timedelta(1), lambda _: [Symbol.create("SPY", SecurityType.OPTION, Market.USA)]
    )
)
```

Extended Market Hours

The `extended_market_hours` setting is a `bool` that defines the trading schedule. If it's `True` , your algorithm receives price data for all trading hours. If it's `False` , your algorithm receives price data only for regular trading hours. The default value is `False` .

You only receive extended market hours data if you create the subscription with an intraday resolution. If you create the subscription with daily resolution, the daily bars only reflect the regular trading hours.

To view the trading schedule of an asset, open Asset Classes , click an asset class, then click Market Hours .

To enable extended market hours, in the initialize method, adjust the algorithm's `UniverseSettings` before you create the Universe Selection model.

```py
# Enable extended market hours for universe and manually select equity symbols for SPY, QQQ, and IWM to ensure trading during pre-market
and after-hours sessions.
self.universe_settings.extended_market_hours = True
tickers = ["SPY", "QQQ", "IWM"]
symbols = [ Symbol.create(ticker, SecurityType.EQUITY, Market.USA) for ticker in tickers]
self.add_universe_selection(ManualUniverseSelectionModel(symbols))
```

Minimum Time in Universe

The `minimum_time_in_universe` setting is a `timedelta` object that defines the minimum amount of time an asset must be in the universe before the universe can remove it. The default value is `timedelta(1)` . To change the minimum time, in the Initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

```py
# Keep each security in the universe for a minimum of 7 days.
self.universe_settings.minimum_time_in_universe = timedelta(7)
self.add_universe_selection(ETFConstituentsUniverseSelectionModel("QQQ"))
```

Data Normalization Mode

The `data_normalization_mode` setting is an enumeration that defines how historical data is adjusted. This setting is only applicable for US Equities and Futures.

In the case of US Equities, the data normalization mode affects how historical data is adjusted for corporate actions . To view all the available options, see Data Normalization . To change the data normalization mode, in the initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

In the case of Futures, the data normalization mode affects how historical data of two contracts is stitched together to form the continuous contract . To view all the available options, see Data Normalization .

The default value is `DataNormalizationMode.ADJUSTED` . To change the data normalization mode, in the Initialize method, adjust the algorithm's `UniverseSettings` before you create the Universe Selection model.

```py
# Set universe data normalization to use raw prices to reflect actual traded values without adjustments and manually select equities
MSTR, MSFT, and IBM.
self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW

tickers = ["MSTR", "MSFT", "IBM"]
symbols = [ Symbol.create(ticker, SecurityType.EQUITY, Market.USA) for ticker in tickers]
self.add_universe_selection(ManualUniverseSelectionModel(symbols))
```

Contract Depth Offset

The `contract_depth_offset` setting is an `int` that defines which contract to use for the continuous Futures contract. 0 is the front month contract, 1 is the following back month contract, and 3 is the second back month contract. The default value is 0. To change the contract depth offset, in the Initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

```py
# Select the following month contract dynamically to avoid low liquidity and high volatility, for S&P 500 E-mini futures with daily
updates.
self.universe_settings.contract_depth_offset = 1
self.add_universe_selection(
    FutureUniverseSelectionModel(
        timedelta(1),
        lambda _: [Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)]
    )
)
```

Asynchronous Selection

The `asynchronous` setting is a `bool` that defines whether or not LEAN can run universe selection asynchronously, utilizing concurrent execution to

increase the speed of your algorithm. The default value for this setting is `False` . If you enable this setting, abide by the following rules:

- Don't make any history requests in your filter function . History requests can only provide data up to the algorithm time , but if your filter function runs asynchronously, the algorithm time may not be what you expect.
- Don't use the portfolio, security, or orders state. The `portfolio` , `securities` , and `transactions` objects are also functions of the algorithm time.
- If your filter function updates a class variable, don't update the class variable anywhere else in your algorithm.

To enable asynchronous universe selection, in the Initialize method, adjust the algorithm's `universe_settings` before you create the Universe Selection model.

```
PY
# Run universe selection asynchronously to speed up your algorithm.
self.universe_settings.asynchronous = True
self.add_universe_selection(EmaCrossUniverseSelectionModel())
```

Schedule

The `Schedule` setting defines the selection schedule of the universe. Most universes run on a daily schedule. To change the selection schedule, call the `universe_settings.schedule.on` method with an `IDateRule` object before you create the Universe Selection model.

```
PY
# Trigger selections at the begining of each month.
self.universe_settings.schedule.on(self.date_rules.month_start())
self.add_universe_selection(ETFConstituentsUniverseSelectionModel("QQQ"))
```

The following table describes the supported `DateRules` :

Member	Description
`self.date_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `DateRules` object used in all methods in this table. The default time zone is the algorithm time zone .
`self.date_rules.on(year: int, month: int, day: int)`	Trigger an event on a specific date.
`self.date_rules.on(dates: list[datetime])`	Trigger an event on specific dates.
`self.date_rules.every_day()`	Trigger an event all seven days of the week.
`self.date_rules.every_day(symbol: Symbol, extended_market_hours: bool = False)`	Trigger an event every day a specific symbol is trading.
`self.date_rules.every(days: list[DayOfWeek])`	Trigger an event on specific days throughout the week. To view the `DayOfWeek` enum members, see DayOfWeek Enum in the .NET documentation.
`self.date_rules.month_start(days_offset: int = 0)`	Trigger an event on the first day of each month plus an offset.
`self.date_rules.month_start(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the first tradable date of each month for a specific symbol plus an offset.
`self.date_rules.month_end(days_offset: int = 0)`	Trigger an event on the last day of each month minus an offset.
`self.date_rules.month_end(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the last tradable date of each month for a specific symbol minus an offset.
`self.date_rules.week_start(days_offset: int = 0)`	Trigger an event on the first day of each week plus an offset.
`self.date_rules.week_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each week for a specific symbol plus an offset.
`self.date_rules.week_end(days_offset: int = 0)`	Trigger an event on the last day of each week minus an offset.
`self.date_rules.week_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each week for a specific symbol minus an offset.
`self.date_rules.year_start(days_offset: int = 0)`	Trigger an event on the first day of each year plus an offset.
`self.date_rules.year_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each year for a specific symbol plus an offset.
`self.date_rules.year_end(days_offset: int = 0)`	Trigger an event on the last day of each year minus an offset.
`self.date_rules.year_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each year for a specific symbol minus an offset.
`self.date_rules.today`	Trigger an event once today.
`self.date_rules.tomorrow`	Trigger an event once tomorrow.

To define custom date rules, create a `FuncDateRule` object. The `FuncDateRule` constructor expects a `name` argument of type `str` and a `get_dates_function` argument of type `Callable[[datetime, datetime], list[datetime]]` . The `get_dates_function` function receives the start and end dates of the algorithm and returns a list of dates for the date rule. In live trading, the end date is 12/31/2025. The following example demonstrates how to define a date rule that represents the 10th day of each month:

```py
# Create a date rule that specifies the 10th day of each month.
date_rule = FuncDateRule(
    name="10th_day_of_the_month",
    get_dates_function=lambda start, end: [
        datetime(year, month, 10)
        for year in range(start.year, end.year) for month in range(1,12)
    ]
)
```

Some date rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.date_rules.every_day('ES')` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
PY
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

In live trading, scheduled universes run at roughly 8 AM Eastern Time to ensure there is enough time for the data to process.

Configure Securities

Instead of configuring global universe settings, you can individually configure the settings of each security in the universe with a security initializer. Security initializers let you apply any security-level reality model or special data requests on a per-security basis. To set the security initializer, in the `initialize` method, call the `set_security_initializer` method and then define the security initializer.

```
PY
# A custom security initializer can override default models such as
# setting new fee and fill models for the security.
self.set_security_initializer(self._custom_security_initializer)

def _custom_security_initializer(self, security: Security) -> None:
    security.set_fee_model(ConstantFeeModel(0, "USD"))
```

For simple requests, you can use the functional implementation of the security initializer. This style lets you configure the security object with one line of code.

```
PY
# Disable the trading fees for each security by using lambda function
# for the set_security_initializer argument.
self.set_security_initializer(lambda security: security.set_fee_model(ConstantFeeModel(0, "USD")))
```

If you call the `set_security_initializer` method, it overwrites the default security initializer. The default security initializer uses the security-level reality models of the brokerage model to set the following reality models of each security:

- Fill
- Slippage
- Fee
- Buying Power
- Settlement
- Short availability
- Margin Interest Rate

The default security initializer also sets the leverage of each security and intializes each security with a seeder function. To extend upon the default security initializer instead of overwriting it, call `add_security_initializer` before you create the subscriptions.

```
PY
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite some of the reality models
        security.set_fee_model(ConstantFeeModel(0, "USD"))
```

Examples

The following examples demonstrate some common practices for universe settings.

Example 1: Weekly-Updating Liquid Universe

The following algorithm demonstrates daily EMA cross, trading on the most liquid stocks. The universe is set to be updated weekly. Various universe settings have been set to simulate the brokerage environment best and for trading needs.

PY

```
class UniverseSettingsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # To avoid over-updating the universe, which causes insufficient time to earn the trend, update the universe weekly.
        self.universe_settings.schedule.on(self.date_rules.week_start())
        self.universe_settings.minimum_time_in_universe = timedelta(7)
        # Rebalancing frequency is daily, so we only need to subscribe to daily resolution.
        self.universe_settings.resolution = Resolution.DAILY
        # To best simulate IB's margin requirement, set the leverage to 50%.
        self.universe_settings.leverage = 2
        # Since we trade by market order, we cannot use extended market hours data to fill.
        self.universe_settings.extended_market_hours = False
        # We want to trade the EMA with raw price but not altered by splits.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.SPLIT_ADJUSTED

                # Only trade on the top 10 most traded stocks since they have the most popularity to drive trends.
        self.add_universe(self.universe.top(10))

    def on_data(self, slice: Slice) -> None:
        for symbol, bar in slice.bars.items():
            # Trade the trend by EMA crossing.
            ema = self.securities[symbol].ema.current.value
            if bar.close > ema:
                self.set_holdings(symbol, 0.05)
            else:
                self.set_holdings(symbol, -0.05)

    def on_securities_changed(self, changes: SecurityChanges) -> None:
        for removed in changes.removed_securities:
            # Liquidate the ones leaving the universe since we will not trade their EMA anymore.
            self.liquidate(removed.symbol)
            # Deregister the EMA indicator to free resources
            self.deregister_indicator(removed.ema)

        for added in changes.added_securities:
            # Create EMA indicator for trend trading signals.
            added.ema = self.ema(added.symbol, 50, Resolution.DAILY)
            # Warm up the EMA indicator to ensure its readiness for immediate use.
            self.warm_up_indicator(added.symbol, added.ema, Resolution.DAILY)
```

Universe Selection

Manual Universes

Introduction

The `ManualUniverseSelectionModel` selects a static, fixed set of assets. It is similar to adding securities with the traditional `add_security` API methods. If your algorithm has a static universe, you can use automatic indicators instead of manual indicators in your algorithm.

Manual universes can be prone to look-ahead bias . For example, if you select a set of securities that have performed well during the backtest period, you are incorporating information from the future into the backtest and the algorithm may underperform in live mode.

Add Manual Universe Selection

To add a `ManualUniverseSelectionModel` to your algorithm, in the `initialize` method, call the `AddUniverseSelection` method. The `ManualUniverseSelectionModel` constructor expects a list of `Symbol` objects that represent the universe constituents.

```
PY
# Use ManualUniverseSelectionModel method to select a static set of equities, similar to traditional add_security methods, for a fixed
trading universe.
tickers = ["SPY", "QQQ", "IWM"]
symbols = [ Symbol.create(ticker, SecurityType.EQUITY, Market.USA) for ticker in tickers]
self.add_universe_selection(ManualUniverseSelectionModel(symbols))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`symbols`	`List[Symbol]`	Universe constituents	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

To move the universe tickers and `Symbol` objects outside of the algorithm class, create a universe selection model that inherits the `ManualUniverseSelectionModel` class.

```
PY
# Create a custom universe selection model to move universe tickers and Symbol objects outside of the algorithm class for improved
modularity, reusability, and maintainability.
self.add_universe_selection(IndexUniverseSelectionModel())

# Outside of the algorithm class
class IndexUniverseSelectionModel(ManualUniverseSelectionModel):
    def __init__(self):
        tickers = ["SPY", "QQQ", "IWM"]
        symbols = [Symbol.create(ticker, SecurityType.EQUITY, Market.USA) for ticker in tickers]
        super().__init__(symbols)
```

For more information about this model, see the class reference and implementation .

Examples

The following examples demonstrate some common practices for implementing a manual universe selection model.

Example 1: Crypto List Selection Model

The following algorithm selects a list of preset cryptos, given we have information that they will perform better. So, we can use `ManualUniverseSelectionModel` to select this list and buy the listed cryptos.

PY

```
class FrameworkManualUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Add a universe selection model to select the most liquid cryptos manually.
        symbols = [
            Symbol.create("BTCUSD", SecurityType.CRYPTO, Market.COINBASE),
            Symbol.create("ETHUSD", SecurityType.CRYPTO, Market.COINBASE)
        ]
        self.add_universe_selection(ManualUniverseSelectionModel(symbols))

        # Sent insights on buying and holding the most liquid cryptos for one week.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Evenly dissipate the capital risk among selected cryptos.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

Universe Selection

Fundamental Universes

Introduction

A `FundamentalUniverseSelectionModel` selects a universe of US Equities based on `Fundamental` data. Depending on the `Fundamental` properties you use, these universes rely on the US Equity Coarse Universe dataset, the US Fundamental dataset, or both.

These types of universes operate on daily schedule by default. In backtests, they select assets at midnight. In live trading, the selection timing depends on the data provider you use. To adjust the selection schedule, see Schedule .

If you use a fundamental Universe Selection model, the only way to unsubscribe from a security is to return a list from the selection function that doesn't include the security `Symbol` . The `remove_security` method doesn't work with these types of Universe Selection models.

Fundamental Selection

The `FundamentalUniverseSelectionModel` lets you select stocks based on corporate `Fundamental` data. You can specific the selection method, which takes a list of `Fundamental` objects as argument and returns a list of `Symbol` objects.

```py
def initialize(self) -> None:
    # Run asynchronous universe selection to speed up your algorithm.
    # In this case, you can't rely on the method or algorithm state between filter calls.
    self.universe_settings.asynchronous = True
    self.add_universe_selection(FundamentalUniverseSelectionModel(self._fundamental_filter_function))

def _fundamental_filter_function(self, fundamental: List[Fundamental]):
    # Select symbols with fundamental data and a price above $10.
    filtered = [f for f in fundamental if f.price > 10 and not np.isnan(f.valuation_ratios.pe_ratio)]
    # Sort the assets in ascending order by their P/E ratio.
    sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
    # Select the first 10 assets.
    return [f.symbol for f in sorted_by_pe_ratio[:10]]
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`selector`	`Callable[[List[Fundamental]], List[Symbol]]`	Filter function to select assets based on fundamental data.	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

The `Fundamental` objects have the following properties:

To move the selection function outside of the algorithm class, create a universe selection model that inherits the `FundamentalUniverseSelectionModel` class and override its `select` method.

```
# In the initialize method, enable asynchronous universe selection to speed up your algorithm.
self.universe_settings.asynchronous = True
# Add a custom universe selection model that selects undervalued, liquid stocks.
self.add_universe_selection(LiquidAndLowPEUniverseSelectionModel())

# Outside of the algorithm class, define the universe selection model.
class LiquidAndLowPEUniverseSelectionModel(FundamentalUniverseSelectionModel):
    def select(self, fundamental: List[Fundamental]) -> List[Symbol]:
        # Select symbols with fundamental data and a price above $1.
        filtered = [x for x in fundamental if x.price > 1 and not np.isnan(x.valuation_ratios.pe_ratio)]
        # Select the 100 assets with the greatest daily dollar volume.
        most_liquid = sorted(filtered, key=lambda x: x.dollar_volume, reverse=True)[:100]
        # Select the 10 assets with the lowest PE ratio.
        lowest_pe_ratio = sorted(most_liquid, key=lambda x: x.valuation_ratios.pe_ratio, reverse=True)[:10]
        return [x.Symbol for x in lowest_pe_ratio]
```

To return the current universe constituents from the selection function, return `Universe. UNCHANGED`.

For more information about this model, see the class reference and implementation.

EMA Cross Selection

The `EmaCrossUniverseSelectionModel` applies two exponential moving average (EMA) indicators to the price history of assets and then selects the assets that have their fast EMA furthest above their slow EMA on a percentage basis.

```
# Initialize asynchronous universe settings for faster processing and add EmaCrossUniverseSelectionModel to dynamically manage the
universe based on EMA cross signals to identify trending assets.
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.add_universe_selection(EmaCrossUniverseSelectionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`fast_period`	`int`	Fast EMA period	100
`slow_period`	`int`	Slow EMA period	300
`universe_count`	`int`	Maximum number of members of this universe selection	500
`universe_settings`	`UniverseSettings`	The universe settings. If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

For more information about this model, see the class reference and implementation.

Examples

The following examples demonstrate some common practices for implementing the framework fundamental universe model.

Example 1: 50 Stocks >10/Share and >10M in Daily Trading Volume

The following algorithm selects the 50 most liquid US Equities above 10/share and 10M daily volume. We pass a function to the `FundamentalUniverseSelectionModel` to filter the fundamental data for stock selection.

```
class FrameworkFundamentalUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 8)
        self.set_cash(100000)

        # Configure the universe to update at the start of each month. Most of the top 500 don't change very frequently.
        self.universe_settings.schedule.on(self.date_rules.month_start())
        # Add a universe with custom selection rules for filtering.
        self.add_universe_selection(FundamentalUniverseSelectionModel(
            lambda fundamental: [
                x.symbol for x in sorted(
                    # Large-cap non-penny stocks have a more stable trend and demand.
                    [f for f in fundamental if f.price > 10 and f.dollar_volume > 10_000_000],
                    key=lambda f: f.dollar_volume
                )[-500:]
            ]
        ))

        # Sent insights on buying and holding the most liquid cryptos for a week.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(30)))
        # Evenly dissipate the capital risk among selected cryptos.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

Example 2: 10 Stocks Above Their 200-Day EMA With >$1B of Daily Trading Volume

Another common request is to filter the universe by a technical indicator, such as only picking stocks above their 200-day Exponential Moving Average (EMA). The `Fundamental` object has adjusted price and volume information so that you can do any price-related analysis. The following algorithm defines a separate class to contain the indicator of each asset. We pass a function to the `FundamentalUniverseSelectionModel` to filter the fundamental data for stock selection.

```
class FrameworkFundamentalUniverseSelectionAlgorithm(QCAlgorithm):
 # Create a dictionary to store the EMA data for universe selection.
    _selection_data_by_symbol = {}

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Add a universe with custom selection rules for filtering.
        self.add_universe_selection(FundamentalUniverseSelectionModel(self._select_assets))

        # Sent insights on buying and holding the most liquid cryptos for a week.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))
        # Evenly dissipate the capital risk among selected cryptos.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def _select_assets(self, fundamental: List[Fundamental]) -> List[Symbol]:
        for f in fundamental:
            # Create/Update the EMA indicators of each stock.
            if f.symbol not in self._selection_data_by_symbol:
                self._selection_data_by_symbol[f.symbol] = SelectionData(self, f.symbol, 200)
            self._selection_data_by_symbol[f.symbol].update(f.end_time, f.adjusted_price, f.dollar_volume)

        # Select the Equities above their EMA and have a daily volume of $1B.
        # These assets are in an uptrend and are very liquid.
        selected = [x for x in self._selection_data_by_symbol.values() if x.is_above_ema and x.volume > 1_000_000_000]

        # Select the 10 most liquid Equities to avoid extra slippage.
        return [ x.symbol for x in sorted(selected, key=lambda x: x.volume)[-10:] ]

# Create a separate class to contain the EMA information of each asset.
class SelectionData(object):
    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol, period: int):
        # Create an EMA indicator for trend estimation and filtering.
        self.symbol = symbol
        self._ema = ExponentialMovingAverage(period)
        self.is_above_ema = False
        self.volume = 0

        # Warm up the EMA indicator.
        algorithm.warm_up_indicator(symbol, self._ema, Resolution.Daily);

    # Update your variables and indicators with the latest data.
    # You may also want to use the History API here to warm up the indicator.
    def update(self, time: datetime, price: float, volume: float) -> None:
        self.volume = volume
        if self._ema.update(time, price):
            self.is_above_ema = price > self._ema.current.value
```

In this example, the `SelectionData` class group variables for the universe selection and update the indicator of each asset. We highly recommend

you follow this pattern to keep your algorithm tidy and bug-free. The following snippet shows an example implementation of the `SelectionData` class, but you can make whatever you need to store your custom universe filters. Note that the preceding `SelectionData` class uses a manual EMA indicator instead of the automatic version . For more information about universes that select assets based on indicators, see Indicator Universes . You need to use a `SelectionData` class instead of assigning the EMA to the `Fundamental` object because you can't create custom attributes on `Fundamental` objects like you can with `Security` objects.

Example 3: 10 Stocks Furthest Above their 10-day SMA of Volume

The process to get the 10-day Simple Moving Average (SMA) stock volume is the same process as in Example 2. First, define a `SelectionData` class that performs the averaging. This class tracks the ratio of today's volume relative to historical volumes. You can use this ratio to select assets above their 10-day SMA and sort the results by the Equities that have had the most significant jump since yesterday.

We inherit the `FundamentalUniverseSelectionModel` to create a custom universe selection model. Overriding the `select` method, we filter the fundamental data for stocks selection.

```py
class FrameworkFundamentalUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1_000_000)

        # Add a universe with custom selection rules for filtering.
        self.add_universe_selection(VolumeSMAUniverseSelectionModel(10))

        # Sent insights on buying and holding the selected securities.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))
        # Evenly dissipate the capital risk among selected securities.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

class VolumeSMAUniverseSelectionModel(FundamentalUniverseSelectionModel):
 # Create a dictionary to store the EMA data for universe selection.
    _selection_data_by_symbol = {}

    def __init__(self, period: int) -> None:
        self.period = period

    def create_universes(self, algorithm: QCAlgorithm) -> List[Universe]:
        return [FundamentalUniverseFactory(Market.USA, algorithm.universe_settings,
            lambda fundamental: self.select(algorithm, fundamental))]

    def select(self, algorithm: QCAlgorithm, fundamental: List[Fundamental]) -> List[Symbol]:
        # Create/Update the volume SMA indicator of each stock.
        selected = []
        for f in fundamental:
            selection_data = self._selection_data_by_symbol.get(f.symbol, self.SelectionData(algorithm, f.symbol, self.period))
            is_ready = selection_data.update(f.end_time, f.adjusted_price, f.dollar_volume)
            # Select the Equities with higher trading volume than their SMA, indicating higher capital flow.
            if is_ready and selection_data.volume_ratio > 1:
                selected.append(selection_data)
            self._selection_data_by_symbol[f.symbol] = selection_data

        # Select the 10 Equities with the highest relative volume since they have the highest capacity
        # for scalp-trading or intra-day movement.
        return [ x.symbol for x in sorted(selected, key=lambda x: x.volume_ratio)[-10:] ]

    # Create a separate class to contain the EMA information of each asset.
    class SelectionData(object):
        def __init__(self, algorithm: QCAlgorithm, symbol: Symbol, period: int) -> None:
            self.symbol = symbol
            self.volume_ratio = 0
            # Create an SMA of volume to track the popularity of the stock.
            self._sma = SimpleMovingAverage(period)

            # Warm up the volume SMA indicator.
            algorithm.warm_up_indicator(symbol, self._sma, Resolution.DAILY, lambda data: data.volume)

        def update(self, time: datetime, price: float, volume: float) -> bool:
            # Update the SMA with today's data and calculate the relative volume position for filtering.
            if not self._sma.update(time, volume):
                return False
            sma = self._sma.current.value
            self.volume_ratio = volume / sma if sma > 0 else 0
            return True
```

Example 4: 10 "Fastest Moving" Stocks With a 50-Day EMA > 200 Day EMA

You can construct complex universe filters with the `SelectionData` helper class pattern. To view a full example of this algorithm, see the EmaCrossUniverseSelectionAlgorithm in the LEAN GitHub repository or take the related Boot Camp lesson .

PY

```
class FrameworkFundamentalUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # Add a universe with custom selection rules for filtering.
        self.add_universe_selection(EmaUniverseSelectionModel(10))

        # Sent insights on buying and holding the selected securities.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))
        # Evenly dissipate the capital risk among selected securities.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

class EmaUniverseSelectionModel(FundamentalUniverseSelectionModel):
 # Create a dictionary to store the EMA data for universe selection.
    averages = {}

    def __init__(self, selection_number: int) -> None:
        super().__init__()
        self.selection_number = selection_number

    def select(self, algorithm: QCAlgorithm, fundamental: List[Fundamental]) -> List[Symbol]:
        # We are going to use a dictionary to refer to the object that will keep the moving averages
        for f in fundamental:
            if f.symbol not in self.averages:
                self.averages[f.symbol] = self.SelectionData(algorithm, f.symbol)

            # Updates the SymbolData object with current EOD price
            avg = self.averages[f.symbol]
            avg.update(f.end_time, f.adjusted_price)

        # Filter the values of the dict: we only want up-trending securities
        values = list(filter(lambda x: x.is_uptrend, self.averages.values()))

        # Sorts the values of the dict: we want those with greater differences between the moving average
        values.sort(key=lambda x: x.scale, reverse=True)

        # we need to return only the symbol objects
        return [ x.symbol for x in values[:self.selection_number] ]

    # class used to improve the readability of the fundamental selection function
    class SelectionData(object):
        def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
            self.symbol = symbol
            self.tolerance = 1.01
            self.fast = ExponentialMovingAverage(100)
            self.slow = ExponentialMovingAverage(300)
            self.is_uptrend = False
            self.scale = 0

            # Warm up the indicators.
            algorithm.warm_up_indicator(symbol, self.fast, Resolution.DAILY)
            algorithm.warm_up_indicator(symbol, self.slow, Resolution.DAILY)

        def update(self, time: datetime, price: float) -> None:
            if self.fast.update(time, price) and self.slow.update(time, price):
                fast = self.fast.current.value
                slow = self.slow.current.value
                self.is_uptrend = fast > slow * self.tolerance

            if self.is_uptrend:
                # computes an object score of how much larger the fast is than the slow
                self.scale = (fast - slow) / ((fast + slow) / 2.0)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
FundamentalUniverseSelectionAlgorithm.py Python EmaCrossUniverseSelectionFrameworkAlgorithm.py Python

Universe Selection

ETF Constituents Universes

Introduction

The `ETFConstituentsUniverseSelectionModel` selects a universe of US Equities based on the constituents of an ETF. These Universe Selection models rely on the US ETF Constituents dataset. They run on a daily schedule by default. To adjust the selection schedule, see Schedule .

Add ETF Constituents Universe Selection

To add an `ETFConstituentsUniverseSelectionModel` to your algorithm, in the `initialize` method, call the `add_universe_selection` method. The `ETFConstituentsUniverseSelectionModel` constructor expects an ETF ticker.

```
PY
# Run universe selection asynchronously to speed up your algorithm.
self.universe_settings.asynchronous = True
self.add_universe_selection(ETFConstituentsUniverseSelectionModel("SPY"))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`etf_ticker`	`string`	Ticker of the ETF to get constituents for. To view the available ETFs, see Supported ETFs .	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`
`universe_filter_func`	`Callable[[List[ETFConstituentUniverse]], List[Symbol]]`	Function to filter ETF constituents. If you don't provide an argument, the model selects all of the ETF constituents by default.	`None`

If you provide a `universe_filter_func` argument, you can use the following attributes of the `ETFConstituentUniverse` objects to select your universe:

The following example shows how to select the 10 Equities with the largest weight in the SPY ETF:

```
PY
# Initialize asynchronous settings for speed and use the ETFConstituentsUniverseSelectionModel
# to select the top 10 SPY constituents by weight, focusing on blue-chip stocks with minimal risk.
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.add_universe_selection(
        ETFConstituentsUniverseSelectionModel("SPY", universe_filter_func=self._etf_constituents_filter)
    )

def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
    # Select the 10 largest Equities in the ETF.
    selected = sorted(
        [c for c in constituents if c.weight],
        key=lambda c: c.weight, reverse=True
    )[:10]
    return [c.symbol for c in selected]
```

To move the ETF `Symbol` and the selection function outside of the algorithm class, create a universe selection model that inherits the `ETFConstituentsUniverseSelectionModel` class.

```py
# Initialize asynchronous settings for speed and use the LargestWeightSPYETFUniverseSelectionModel
# to select the top 10 blue-chip SPY constituents by weight, focusing on stocks with minimal risk.
self.universe_settings.asynchronous = True
self.add_universe_selection(LargestWeightSPYETFUniverseSelectionModel())

# Outside of the algorithm class
class LargestWeightSPYETFUniverseSelectionModel(ETFConstituentsUniverseSelectionModel):

    def __init__(self) -> None:
        super().__init__('SPY', universe_filter_func=self._etf_constituents_filter)

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Select the 10 largest Equities in the ETF.
        selected = sorted(
            [c for c in constituents if c.weight],
            key=lambda c: c.weight, reverse=True
        )[:10]
        return [c.symbol for c in selected]
```

To return the current universe constituents from the selection function, return `Universe. UNCHANGED` .

For more information about this model, see the class reference and implementation .

Examples

The following examples demonstrate some common practices for implementing the framework ETF constituent universe selection model.

Example 1: Weighted ETF Constituents

A subset of the SPY constituents outperform the SPY while many underperform the overall index. In an attempt to buy the ETF constituents that outperform the index, the following algorithm buys the top 50 weighted assets in the ETF. In this example, we will pass a function to the `ETFConstituentsUniverseSelectionModel` for selection.

```py
class FrameworkETFConstituentsUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)

        self.etf_weight_by_symbol = {}

        # Add a universe of the SPY constituents.
        self.add_universe_selection(
            ETFConstituentsUniverseSelectionModel("SPY", universe_filter_func=self._etf_constituents_filter)
        )
        # Add an Alpha model to trade based on the constituent weights.
        self.add_alpha(EtfAlphaModel(self))
        # Position sizing was handled by insight weight (from ETF weight).
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())

    def _etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Cache the constituent weights in a dictionary for filtering and position sizing.
        self.etf_weight_by_symbol = {x.symbol: x.weight for x in constituents if x.weight}
        # Select the 50 constituents with the largest weight in the ETF.
        # They should have positive excess returns.
        return [x[0] for x in sorted(self.etf_weight_by_symbol.items(), key=lambda x: x[1], reverse=True)[:50]]

class EtfAlphaModel(AlphaModel):
    def __init__(self, algorithm: FrameworkETFConstituentsUniverseSelectionAlgorithm) -> None:
        self._algorithm = algorithm
        self._day = -1

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Daily rebalance only.
        if self._day == slice.time.day:
            return insights

        for symbol, weight in self._algorithm.etf_weight_by_symbol.items():
            # Get the ETF weight of all the assets currently in the universe.
            # To avoid trading errors, skip assets that have no price yet.
            if symbol in slice.bars:
                if not algorithm.securities[symbol].price:
                    continue
                insights.append(Insight.price(symbol, timedelta(1), InsightDirection.UP, weight=weight))

        self._day = slice.time.day

        return insights
```

Example 2: Weight Trend Selection

The following algorithm trades the QQQ constituents with the upward trend of its weight constitution, indicated by an EMA indicator, suggesting its price trend is outperforming other constituents, or its weight was promoted by the NASDAQ index, which brings positive sentiment. To better utilize the EMA indicator, we inherit the `ETFConstituentsUniverseSelectionModel` superclass to create a custom universe selection model.

```py
class FrameworkETFConstituentsUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)

        # Add a universe of the SPY constituents.
        self.add_universe_selection(
            UptrendETFUniverseSelectionModel("QQQ")
        )
        # Add Alpha model to trade based on the selections.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))
        # Equally invest in insights to dissipate the capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Warm up the indicators.
        self.set_warm_up(60, Resolution.DAILY)

class UptrendETFUniverseSelectionModel(ETFConstituentsUniverseSelectionModel):
    ema_by_symbol = {}

    def __init__(self, etf_ticker: str) -> None:
        super().__init__(etf_ticker, universe_filter_func=self.etf_constituent_filter)

    def etf_constituent_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Remove the ones that are not in the ETF anymore.
        to_remove = set(self.ema_by_symbol.keys()).difference(set([x.symbol for x in constituents]))
        for symbol in to_remove:
            del self.ema_by_symbol[symbol]

        for c in constituents:
            symbol = c.symbol
            if symbol in self.ema_by_symbol and c.weight:
                # Update EMA with the latest weight.
                self.ema_by_symbol[symbol].update(c.EndTime, c.weight)
            else:
                # Create an EMA to filter by trend.
                self.ema_by_symbol[symbol] = ExponentialMovingAverage(60)

        # Select the ones with the increasing trend of constituent weight.
        return [symbol for symbol, ema in self.ema_by_symbol.items() if ema.is_ready and ema.current.value > ema.previous.value]
```

Universe Selection

Scheduled Universes

Introduction

A Scheduled Universe Selection model selects assets at fixed, regular intervals. In live trading, you can use this type of model to fetch tickers from Dropbox or to perform periodic historical analysis and select some assets.

Scheduled Universe Selection

The `ScheduledUniverseSelectionModel` selects assets on the schedule you provide. To use this model, provide a `DateRule` , `TimeRule` , and a selector function. The `DateRule` and `TimeRule` define the selection schedule. The selector function receives a `datetime` object and returns a list of `Symbol` objects. The `Symbol` objects you return from the selector function are the constituents of the universe.

```
# Enable asynchronous universe selection to speed up your algorithm.
self.universe_settings.asynchronous = True
# Add a universe that selects assets at the beginning of each month.
self.add_universe_selection(
    ScheduledUniverseSelectionModel(
        self.date_rules.month_start(),
        self.time_rules.midnight,
        # Select SPY for October. Otherwise, select QQQ.
        lambda dt: [Symbol.create("SPY" if dt.month == 10 else "QQQ", SecurityType.EQUITY, Market.USA)]
    )
)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`date_rule`	`IDateRule`	Date rule defines what days the universe selection function will be invoked	
`time_rule`	`ITimeRule`	Time rule defines what times on each day selected by date rule the universe selection function will be invoked	
`selector`	`Callable[[datetime], List[Symbol]]`	Selector function accepting the date time firing time and returning the universe selected symbols	
`settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

The model assumes the `time_rule` argument is in Coordinated Universal Time (UTC). To use a different timezone, pass a `timeZone` argument of type `DateTimeZone` before the `date_rule` argument.

To return the current universe constituents from the selector function, return `Universe. UNCHANGED` .

```py
# Add a universe that selects assets on Monday, Tuesday, and Thursday at 00:00, 06:00, 12:00, and 18:00 UTC.
self.add_universe_selection(ScheduledUniverseSelectionModel(
    self.date_rules.every(DayOfWeek.MONDAY, DayOfWeek.TUESDAY, DayOfWeek.THURSDAY),
    self.time_rules.every(timedelta(hours = 6)),
    self.select_symbols # selection function in algorithm.
))

# Define the selection function.
def select_symbols(self, date_time: datetime) -> List[Symbol]:
    tickers = ['SPY', 'AAPL', 'IBM']
    return [Symbol.create(ticker, SecurityType.EQUITY, Market.USA)
        for ticker in tickers]
```

For more information about this model, see the class reference and implementation .

Date Rules

The following table describes the supported `DateRules` :

Member	Description
`self.date_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `DateRules` object used in all methods in this table. The default time zone is the algorithm time zone .
`self.date_rules.on(year: int, month: int, day: int)`	Trigger an event on a specific date.
`self.date_rules.on(dates: list[datetime])`	Trigger an event on specific dates.
`self.date_rules.every_day()`	Trigger an event all seven days of the week.
`self.date_rules.every_day(symbol: Symbol, extended_market_hours: bool = False)`	Trigger an event every day a specific symbol is trading.
`self.date_rules.every(days: list[DayOfWeek])`	Trigger an event on specific days throughout the week. To view the `DayOfWeek` enum members, see DayOfWeek Enum in the .NET documentation.
`self.date_rules.month_start(days_offset: int = 0)`	Trigger an event on the first day of each month plus an offset.
`self.date_rules.month_start(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the first tradable date of each month for a specific symbol plus an offset.
`self.date_rules.month_end(days_offset: int = 0)`	Trigger an event on the last day of each month minus an offset.
`self.date_rules.month_end(symbol: Symbol, daysOffset: int = 0)`	Trigger an event on the last tradable date of each month for a specific symbol minus an offset.
`self.date_rules.week_start(days_offset: int = 0)`	Trigger an event on the first day of each week plus an offset.
`self.date_rules.week_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each week for a specific symbol plus an offset.
`self.date_rules.week_end(days_offset: int = 0)`	Trigger an event on the last day of each week minus an offset.
`self.date_rules.week_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each week for a specific symbol minus an offset.
`self.date_rules.year_start(days_offset: int = 0)`	Trigger an event on the first day of each year plus an offset.
`self.date_rules.year_start(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the first tradable date of each year for a specific symbol plus an offset.
`self.date_rules.year_end(days_offset: int = 0)`	Trigger an event on the last day of each year minus an offset.
`self.date_rules.year_end(symbol: Symbol, days_offset: int = 0)`	Trigger an event on the last tradable date of each year for a specific symbol minus an offset.
`self.date_rules.today`	Trigger an event once today.
`self.date_rules.tomorrow`	Trigger an event once tomorrow.

To define custom date rules, create a `FuncDateRule` object. The `FuncDateRule` constructor expects a `name` argument of type `str` and a `get_dates_function` argument of type `Callable[[datetime, datetime], list[datetime]]` . The `get_dates_function` function receives the start and end dates of the algorithm and returns a list of dates for the date rule. In live trading, the end date is 12/31/2025. The following example demonstrates how to define a date rule that represents the 10th day of each month:

```py
# Create a date rule that specifies the 10th day of each month.
date_rule = FuncDateRule(
    name="10th_day_of_the_month",
    get_dates_function=lambda start, end: [
        datetime(year, month, 10)
        for year in range(start.year, end.year) for month in range(1,12)
    ]
)
```

Some date rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.date_rules.every_day('ES')` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

PY

Time Rules

The following table describes the supported `TimeRules` :

Member	Description
`self.time_rules.set_default_time_zone(time_zone: DateTimeZone)`	Sets the time zone for the `TimeRules` object used in all methods in this table, except when a different time zone is given. The default time zone is the algorithm time zone .
`self.time_rules.before_market_open(symbol: Symbol, minutes_before_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_open(symbol: Symbol, minutes_after_open: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market open for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.before_market_close(symbol: Symbol, minutes_before_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes before market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.after_market_close(symbol: Symbol, minutes_after_close: float = 0, extended_market_open: bool = False)`	Trigger an event a few minutes after market close for a specific symbol (default is 0). This rule doesn't work for Crypto securities or custom data.
`self.time_rules.every(interval: timedelta)`	Trigger an event every period interval starting at midnight. This time rule triggers every period, regardless of whether or not the market is open.
`self.time_rules.now`	Trigger an event at the current time of day.
`self.time_rules.midnight`	Trigger an event at midnight.
`self.time_rules.noon`	Trigger an event at noon.
`self.time_rules.at(hour: int, minute: int, second: int = 0)`	Trigger an event at a specific time of day (e.g. 13:10).
`self.time_rules.at(hour: int, minute: int, second: int, time_zone: DateTimeZone)`	Trigger an event at a specific time of day in the given time zone (e.g. 13:10 UTC).

To define custom time rules, create a `FuncTimeRule` object. The `FuncTimeRule` constructor expects a `name` argument of type `str` and a `create_utc_event_times_function` argument of type `Callable[[list[datetime]], list[datetime]]` . The function receives the list of dates from the date rule and then returns a list of `datetime` that define the time rule.

```
time_rule = FuncTimeRule(
    name="CustomTimeRule",
    create_utc_event_times_function=lambda dates: [d + timedelta(hours=10) for d in dates]
)
```

PY

Some time rules require a Symbol. However, they accept a string representation of the Symbol as well. If your algorithm doesn't add the security , it assumes the string refers to a US Equity Symbol. For example, if you use the `self.time_rules.before_market_open('ES', 15)` method, it assumes that 'ES' refers to a US Equity. To create a Symbol without adding a security to your algorithm, call the `Symbol.create` method.

```
symbol = Symbol.create('ES', SecurityType.FUTURE, Market.CME)
```

PY

Examples

The following examples demonstrate some common practices for implementing a dateless scheduled universe selection model.

Example 1: From External Source

The following example selects its universe every week starting before the market opens from a source URL of Dropbox. It reloads on every selection so that you can hook up to the continuously updating source file.

```py
from io import StringIO

class FrameworkScheduledUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10000000)

        # Add a universe of that read from a Dropbox source to select the stocks 30 minutes before the week's first trading day.
        spy = Symbol.create("SPY", SecurityType.EQUITY, Market.USA)
        self.add_universe_selection(ScheduledUniverseSelectionModel(
            self.date_rules.week_start(spy),
            self.time_rules.before_market_open(spy, 10),
            self.select_symbols
        ))
        # Add Alpha model to trade based on the selections; the signals last for a week until the next selection.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Equally invest in insights to dissipate the capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def _download_universe_file(self, dt: datetime) -> str:
        # Download the universe CSV file. Dropbox links require the "dl=1" URL parameter.
        file = self.download(
            "https://www.dropbox.com/scl/fi/fbrxitk4ec3w91nse1raa/df.csv?rlkey=7r042rukzkthp7y1srloyhkov&st=5r4sdfwd&dl=1"
        )
        # Convert the CSV file data into a dictionary where the key is the date and
        # the value is a comma-separated string of stock tickers.
        df = pd.read_csv(StringIO(file), index_col=0).iloc[:, 0]
        df.index = pd.to_datetime(df.index).date
        date = dt.date()
        # Return the universe if the date matches.
        if date in df:
            return df.loc[dt.date()]
        return ''

    def select_symbols(self, dt: datetime) -> List[Symbol]:
        # Re-download the CSV file each day to get today's row.
        tickers = self._download_universe_file(dt)
        # If there isn't an entry for the current date, return an empty universe.
        if not tickers:
            return []
        # Convert the stock tickers in the CSV file to Symbol objects.
        return [Symbol.create(x, SecurityType.EQUITY, Market.USA) for x in tickers.split(",")]
```

Example 2: Seasonality

The following algorithm selects between SPY and short-term bond ETF according to the current month. It holds SPY on May, June, July, November, and December since they are historically better-performing months while holding SHV to preserve funds and earn interest in the remaining months.

```py
class FrameworkScheduledUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add a universe of that select based on the time to trade seasonality.
        self.add_universe_selection(ScheduledUniverseSelectionModel(
            self.date_rules.week_start(),
            self.time_rules.at(0, 0),
            self.select_symbols
        ))
        # Add an Alpha model to trade based on the selections; the signals will last until the next selection.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Equally invest in insights to dissipate the capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def select_symbols(self, date_time: datetime) -> List[Symbol]:
        # May/Jun/Jul/Nov/Dec are statistically better-performing months of SPY.
        if date_time.month in [5, 6, 7, 11, 12]:
            return [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
        # In other months, hold short-term bonds to preserve funds while earning interest.
        return [Symbol.create("SHV", SecurityType.EQUITY, Market.USA)]
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
ScheduledUniverseSelectionModelRegressionAlgorithm.py Python

Universe Selection

Futures Universes

Introduction

A Future Universe Selection model selects contracts for a set of Futures.

Future Universe Selection

The `FutureUniverseSelectionModel` selects all the contracts for a set of Futures you specify. To use this model, provide a `refresh_interval` and a selector function. The `refresh_interval` defines how frequently LEAN calls the selector function. The selector function receives a `datetime` object that represents the current Coordinated Universal Time (UTC) and returns a list of `Symbol` objects. The `Symbol` objects you return from the selector function are the Futures of the universe.

```
# Run universe selection asynchronously to speed up your algorithm.
# This setting means you cannot rely on the method or algorithm state between filter calls.
self.universe_settings.asynchronous = True
# Add a universe of E-mini S&P 500 Futures contracts.
self.add_universe_selection(
    FutureUniverseSelectionModel(
        # Refresh the universe daily.
        timedelta(1),
        lambda _: [Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)]
    )
)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`refresh_interval`	`timedelta`	Time interval between universe refreshes	
`future_chain_symbol_selector`	`Callable[[datetime], List[Symbol]]`	A function that selects the Future symbols for a given Coordinated Universal Time (UTC). To view the supported assets in the US Futures dataset, see Supported Assets .	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

The following example shows how to define the Future chain Symbol selector as an isolated method:

```
# In the initialize method, add the FutureUniverseSelectionModel with a custom selection function.
def initialize(self) -> None:
    self.set_universe_selection(
        FutureUniverseSelectionModel(timedelta(days=1), self.select_future_chain_symbols)
    )

def select_future_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
    # Add E-mini S&P 500 and Gold Futures to the universe.
    return [
        Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME),
        Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
    ]
```

This model uses the default Future contract filter, which doesn't select any Futures contracts. To use a different filter, subclass the `FutureUniverseSelectionModel` and define a `filter` method. The `filter` method accepts and returns a `FutureFilterUniverse` object to select the Futures contracts. The following table describes the filter methods of the `FutureFilterUniverse` class:

`standards_only()` Selects standard contracts
`front_month()` Selects the front month contract
`back_months()` Selects the non-front month contracts
`back_month()` Selects the back month contracts
`expiration(min_expiry: timedelta, max_expiry: timedelta)` Selects contracts that expire within a range of dates relative to the current day
`expiration(min_expiry_days: int, max_expiry_days: int)` Selects contracts that expire within a range of dates relative to the current day
`contracts(contracts: list[Symbol])` Selects a list of contracts
`contracts(contractSelector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects

The contract filter runs at the first time step of each day.

To move the Future chain Symbol selector and the contract selection function outside of the algorithm class, create a universe selection model that inherits the FundamentalUniverseSelectionModel class and override its Select method.

```
# In the initialize method, define the universe settings and add data.
self.universe_settings.asynchronous = True
self.add_universe_selection(FrontMonthFutureUniverseSelectionModel())

# Outside of the algorithm class, define the universe selection model.
class FrontMonthFutureUniverseSelectionModel(FutureUniverseSelectionModel):
    def __init__(self) -> None:
        # Refresh the universe daily.
        super().__init__(timedelta(1), self.select_future_chain_symbols)

    def select_future_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
        # Add E-mini S&P 500 and Gold Futures to the universe.
        return [
            Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME),
            Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
        ]

    def filter(self, filter: FutureFilterUniverse) -> FutureFilterUniverse:
        # Select the front month contracts.
        return filter.front_month()
```

Some of the preceding filter methods only set an internal enumeration in the `FutureFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `FutureFilterUniverse` .

The `add_universe_selection` method doesn't return a `Future` object like the add_future method. The `Future` object contains `Symbol` and `mapped` properties, which reference the continuous contract and the currently selected contract in the continuous contract series, respectively. To get the

Future object, define the on_securities_changed method in your algorithm class or framework models and check the result of the is_canonical method.

```
# Save the Future object if the security added to the universe is the canonical asset.
def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        if security.symbol.is_canonical():
            self.future = security
```

For more information about this model, see the class reference and implementation .

Open Interest Future Universe Selection

The OpenInterestFutureUniverseSelectionModel is an extension of the FutureUniverseSelectionModel that selects the contract with the greatest open interest on a daily basis.

```
# Enable asynchronous universe settings for faster performance.
self.universe_settings.asynchronous = True
# Add an OpenInterestFutureUniverseSelectionModel for E-mini S&P 500 Futures, incorporating contracts with high
# open interest into the trading universe.
self.add_universe_selection(
    OpenInterestFutureUniverseSelectionModel(
        self,
        lambda utc_time: [Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)]
    )
)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
algorithm	IAlgorithm	Algorithm	
future_chain_symbol_selector	Callable[[datetime], List[Symbol]]	A function that selects the Future symbols for a given Coordinated Universal Time (UTC). To view the supported assets in the US Futures dataset, see Supported Assets .	
chain_contracts_lookup_limit	int/None	Limit on how many contracts to query for open interest	6
results_limit	int/None	Limit on how many contracts will be part of the universe	1

The following example shows how to define the Future chain Symbol selector as an isolated method:

```
# In the Initialize method, define the universe settings and add a universe.
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.add_universe_selection(
        OpenInterestFutureUniverseSelectionModel(self, self.select_future_chain_symbols)
    )

# Define the selection function, which returns Symbol objects.
def select_future_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
    return [
        Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME),
        Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)
    ]
```

To move the Future chain Symbol selector outside of the algorithm class, create a universe selection model that inherits the OpenInterestFutureUniverseSelectionModel class.

```py
# In the Initialize method, define the universe settings and add a universe.
self.universe_settings.asynchronous = True
self.add_universe_selection(GoldOpenInterestFutureUniverseSelectionModel(self))

# Outside of the algorithm class, define the universe selection model.
class GoldOpenInterestFutureUniverseSelectionModel(OpenInterestFutureUniverseSelectionModel):
    def __init__(self, algorithm: QCAlgorithm, chain_contracts_lookup_limit: int=6, results_limit: int=1):
        super().__init__(algorithm, self.select_future_chain_symbols, chain_contracts_lookup_limit, results_limit)

    def select_future_chain_symbols(self, utcTime: datetime) -> List[Symbol]:
        return [Symbol.create(Futures.Metals.GOLD, SecurityType.FUTURE, Market.COMEX)]
```

The `add_universe_selection` method doesn't return a `Future` object like the add_future method. The `Future` object contains `Symbol` and `mapped` properties, which reference the continuous contract and the currently selected contract in the continuous contract series, respectively. To get the `Future` object, define the `on_securities_changed` method in your algorithm class or framework models and check the result of the `is_canonical` method.

```py
# Save the Future object if the security added to the universe is the canonical asset.
def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
    for security in changes.added_securities:
        if security.symbol.is_canonical():
            self.future = security
```

For more information about this model, see the class reference and implementation .

Examples

The following examples demonstrate some common practices for implementing a Futures universe selection model.

Example 1: Front Month Contracts

The following algorithm selects a list of Futures to buy and hold equally. To ensure liquidity and efficiency, we will select only their front-month contracts. We can do so in the algorithm framework through a child class inheriting the `FutureUniverseSelectionModel` while providing the list of Futures and overriding the `filter` method to select the front month contracts.

```py
class FrameworkFutureUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # It is usual to trade Futures during extended market hours.
        self.universe_settings.extended_market_hours = True
        # Add a universe with custom selection rules for filtering.
        self.add_universe_selection(FrontMonthFutureUniverseSelectionModel(7))

        # Sent insights on buying and holding the selected securities.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Evenly dissipate the capital risk among selected securities.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())


class FrontMonthFutureUniverseSelectionModel(FutureUniverseSelectionModel):
    '''Creates futures chain universes that select the front month contract and run a user-defined
    futureChainSymbolSelector every day to enable choosing different futures chains'''
    def __init__(self, rebalance_period: int = 7) -> None:
        super().__init__(timedelta(rebalance_period), self.select_future_chain_symbols)

    def filter(self, filter: FutureFilterUniverse) -> FutureFilterUniverse:
        # Defines the futures chain universe filter to select only the front-month contracts at market open.
        return filter.front_month()

    def select_future_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
        return [
            Symbol.create(Futures.Indices.VIX, SecurityType.FUTURE, Market.CFE),
            Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME),
            Symbol.create(Futures.Indices.NASDAQ_100_E_MINI, SecurityType.FUTURE, Market.CME),
            Symbol.create(Futures.Indices.DOW_30_E_MINI, SecurityType.FUTURE, Market.CBOT),
            Symbol.create(Futures.Energy.GASOLINE, SecurityType.FUTURE, Market.NYMEX),
            Symbol.create(Futures.Energy.HEATING_OIL, SecurityType.FUTURE, Market.NYMEX),
            Symbol.create(Futures.Energy.NATURAL_GAS, SecurityType.FUTURE, Market.NYMEX),
            Symbol.create(Futures.Grains.CORN, SecurityType.FUTURE, Market.CBOT),
            Symbol.create(Futures.Grains.OATS, SecurityType.FUTURE, Market.CBOT),
            Symbol.create(Futures.Grains.SOYBEANS, SecurityType.FUTURE, Market.CBOT),
            Symbol.create(Futures.Grains.WHEAT, SecurityType.FUTURE, Market.CBOT),
        ]
```

Example 2: Seasonal Contracts

Some Futures exhibit a seasonality effect, i.e., a yearly cycle in its price fluctuation. Studies show that the Natural Gas contract has a yearly cycle of maximizing its price in early December and minimizing it in early April. It might be due to the high demand for natural gas during winter warming. The following algorithm will hold Natural Gas contracts from summer to winter with a similar universe selection model in Example 1, but using a custom selection function with a time argument to select the contract.

PY

```
class FrameworkFutureUniverseSelectionAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        # It is usual to trade Futures during extended market hours.
        self.universe_settings.extended_market_hours = True
        # We hold the Future and will not frequently trade it; using Daily resolution is sufficient for computational efficiency.
        self.universe_settings.resolution = Resolution.DAILY
        # Add a universe with custom selection rules for filtering.
        # Rescan every week to ensure we are on our plan.
        self.add_universe_selection(FrontMonthFutureUniverseSelectionModel(7))

        # Sent insights on buying and holding the selected securities.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Place orders for a single contract to control risk exposure and avoid over-leveraging.
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())

class FrontMonthFutureUniverseSelectionModel(FutureUniverseSelectionModel):
    '''Creates futures chain universes that select the front month contract and run a user-defined
     futureChainSymbolSelector every day to enable choosing different futures chains'''
    def __init__(self, rebalance_period: int = 7) -> None:
        super().__init__(timedelta(rebalance_period), self.select_future_chain_symbols)

    def filter(self, filter: FutureFilterUniverse) -> FutureFilterUniverse:
        # Defines the futures chain universe filter to select only the front-month contracts.
        return filter.front_month()

    def select_future_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
        # We hold Natural Gas from summer to winter since it is in an upward cycle.
        if 4 <= utc_time.month <= 11:
            return [ Symbol.create(Futures.Energy.NATURAL_GAS, SecurityType.FUTURE, Market.NYMEX) ]
        # We do not hold any contracts during the time of the downward cycle.
        return []

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Use integer target to create a portfolio target to trade a single contract
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Universe Selection

Options Universes

Introduction

An Option Universe Selection model selects contracts for a set of Options.

Options Universe Selection

The `OptionUniverseSelectionModel` selects all the available contracts for the Equity Options, Index Options, and Future Options you specify. To use this model, provide a `refresh_interval` and a selector function. The `refresh_interval` defines how frequently LEAN calls the selector function. The selector function receives a `datetime` object that represents the current Coordinated Universal Time (UTC) and returns a list of `Symbol` objects. The `Symbol` objects you return from the selector function are the Options of the universe.

```
# Run universe selection asynchronously to speed up your algorithm.
# In this case, you can't rely on the method or algorithm state between filter calls.
self.universe_settings.asynchronous = True
# Add a universe of SPY Options.
self.set_universe_selection(
    OptionUniverseSelectionModel(
        # Refresh the universe daily.
        timedelta(1), lambda _: [Symbol.create("SPY", SecurityType.OPTION, Market.USA)]
    )
)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`refresh_interval`	`timedelta`	Time interval between universe refreshes	
`option_chain_symbol_selector`	`Callable[[datetime], List[Symbol]]`	A function that selects the Option symbols	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

The following example shows how to define the Option chain Symbol selector as an isolated method:

```
# In the initialize method, add the OptionUniverseSelectionModel with a custom selection function.
def initialize(self) -> None:
    self.add_universe_selection(
        OptionUniverseSelectionModel(timedelta(days=1), self.select_option_chain_symbols)
    )

# Define the selection function.
def select_option_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
    # Equity Options example:
    #tickers = ["SPY", "QQQ", "TLT"]
    #return [Symbol.create(ticker, SecurityType.OPTION, Market.USA) for ticker in tickers]

    # Index Options example:
    #tickers = ["VIX", "SPX"]
    #return [Symbol.create(ticker, SecurityType.INDEX_OPTION, Market.USA) for ticker in tickers]

    # Future Options example:
    future_symbol = Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)
    return [Symbol.create_canonical_option(contract.symbol) for contract in self.futures_chain(future_symbol)]
```

This model uses the default Option filter, which selects all of the available Option contracts at the current time step. To use a different filter for the contracts, subclass the `OptionUniverseSelectionModel` and define a method. The method accepts and returns an `OptionFilterUniverse` object to

select the Option contracts. The following table describes the methods of the `OptionFilterUniverse` class:

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

The contract filter runs at the first time step of each day.

To move the Option chain Symbol selector outside of the algorithm class, create a universe selection model that inherits the `OptionUniverseSelectionModel` class.

```
# In the initialize method, define the universe settings and add data.
self.universe_settings.asynchronous = True
self.add_universe_settings(EarliestExpiringAtTheMoneyCallOptionUniverseSelectionModel(self))

# Outside of the algorithm class, define the universe selection model.
class EarliestExpiringAtTheMoneyCallOptionUniverseSelectionModel(OptionUniverseSelectionModel):
    def __init__(self, algorithm):
        self.algo = algorithm
        super().__init__(timedelta(1), self.select_option_chain_symbols)

    def select_option_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
        # Equity Options example:
        #tickers = ["SPY", "QQQ", "TLT"]
        #return [Symbol.create(ticker, SecurityType.OPTION, Market.USA) for ticker in tickers]

        # Index Options example:
        #tickers = ["VIX", "SPX"]
        #return [Symbol.create(ticker, SecurityType.INDEX_OPTION, Market.USA) for ticker in tickers]

        # Future Options example:
        future_symbol = Symbol.create(Futures.Indices.SP_500_E_MINI, SecurityType.FUTURE, Market.CME)
        return [Symbol.create_canonical_option(contract.symbol) for contract in self.algo.futures_chain(future_symbol)]

    # Create a filter to select contracts that have the strike price within 1 strike level and expire within 7 days.
    def Filter(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
        return option_filter_universe.strikes(-1, -1).expiration(0, 7).calls_only()
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

To override the default pricing model of the Options, set a pricing model in a security initializer.

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

For more information about this model, see the class reference and implementation .

Option Chained Universe Selection

An Option chained universe subscribes to Option contracts on the constituents of a US Equity universe .

```
# Configure the universe to use price data unadjusted for splits and dividends ("raw") into the algorithm.
# Options require raw Equity prices.
self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
self.universe_settings.asynchronous = True
self.add_universe_selection(
    OptionChainedUniverseSelectionModel(
        # Add a universe of the 10 most liquid US Equities.
        self.add_universe(self.universe.top(10)),
        # Select call Option contracts on the underlying Equities that have the strike price within 2 strike levels.
        lambda option_filter_universe: option_filter_universe.strikes(-2, +2).front_month().calls_only()
    )
)
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`universe`	`Universe`	The universe to chain onto the Option Universe Selection model	
`option_filter`	`Callable[[OptionFilterUniverse], OptionFilterUniverse]`	The Option filter universe to use	
`universe_settings`	`UniverseSettings`	The universe settings . If you don't provide an argument, the model uses the `algorithm.universe_settings` by default.	`None`

The `option_filter` function receives and returns an `OptionFilterUniverse` to select the Option contracts. The following table describes the methods of the `OptionFilterUniverse` class:

The following table describes the filter methods of the `OptionFilterUniverse` class:

`strikes(min_strike: int, max_strike: int)` Selects contracts that are within `m_strike` strikes below the underlying price and `max_strike` strikes above the underlying price.
`calls_only()` Selects call contracts.
`puts_only()` Selects put contracts.
`standards_only()` Selects standard contracts.
`weeklys_only()` Selects weekly contracts.
`front_month()` Selects the front month contract.
`back_months()` Selects the non-front month contracts.
`back_month()` Selects the back month contracts.
`expiration(min_expiryDays: int, max_expiryDays: int)` Selects contracts that expire within a range of dates relative to the current day.
`contracts(contracts: list[Symbol])` Selects a list of contracts.
`contracts(contract_selector: Callable[[list[Symbol]], list[Symbol]])` Selects contracts that a selector function selects.

The preceding methods return an `OptionFilterUniverse` , so you can chain the methods together.

The following example shows how to define the Option filter as an isolated method:

```
# In the initialize method, define the universe settings and add the universe selection model.
def initialize(self) -> None:
    self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
    self.universe_settings.asynchronous = True
    self.add_universe_selection(
        OptionChainedUniverseSelectionModel(
            self.add_universe(self.universe.top(10)), self.option_filter_function
        )
    )

# Define the contract filter function to select front month call contracts with a strike price within 2 strike levels.
def option_filter_function(self, option_filter_universe: OptionFilterUniverse) -> OptionFilterUniverse:
    return option_filter_universe.strikes(-2, +2).front_month().calls_only()
```

Some of the preceding filter methods only set an internal enumeration in the `OptionFilterUniverse` that it uses later on in the filter process. This subset of filter methods don't immediately reduce the number of contract `Symbol` objects in the `OptionFilterUniverse` .

For more information about this model, see the class reference and implementation .

Examples

The following examples demonstrate some common practices for implementing the framework Option Universe Selection Model.

Example 1: Horizontal Jelly Roll

The following algorithm selects SPX index options to construct a Jelly Roll strategy. It filters for ATM calls and puts with 30 days and 90 days till expiration. Using the SMA indicator to predict the interest rate cycle, it longs Jelly Roll if the cycle is considered uprising, otherwise selling the Jelly Roll.

```
class FrameworkOptionUniverseSelectionAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(500_000)

        # Add a universe that selects the needed option contracts.
        self.add_universe_selection(AtmOptionHorizontalSpreadUniverseSelectionModel())
        # Add Alpha model to trade Jelly Roll, using interest rate data.
        self.add_alpha(JellyRollAlphaModel(self))
        # Invest in the same number of contracts per leg in the Jelly Roll.
        self.set_portfolio_construction(SingleSharePortfolioConstructionModel())


class AtmOptionHorizontalSpreadUniverseSelectionModel(OptionUniverseSelectionModel):
    # 30d update with the SelectOptionChainSymbols function since the filter returns at least 30d expiry options.
    def __init__(self) -> None:
        super().__init__(timedelta(30), self.selection_option_chain_symbols)

    def selection_option_chain_symbols(self, utc_time: datetime) -> List[Symbol]:
        # We will focus only on SPX options since they have a relatively stable dividend yield, which we assume will remain the same over
time.
        # Also, assignment handling is not required since it is cash-settled.
        return [Symbol.create("SPX", SecurityType.INDEX_OPTION, Market.USA)]

    def filter(self, filter: OptionFilterUniverse) -> OptionFilterUniverse:
        # Jelly Roll is one of the best strategies for trading interest rates using options.
        # It is market-neutral but sensitive to interest rate and dividend yield changes.
        # We target to trade the market speculation between 30d and 90d options interest rate.
        return filter.jelly_roll(0, 30, 90)


class JellyRollAlphaModel(AlphaModel):
    _symbol = Symbol.create("SPX", SecurityType.INDEX_OPTION, Market.USA)
    # Use a 365d SMA indicator of daily interest rate to estimate if the interest rate cycle is upward or downward.
    _sma = SimpleMovingAverage(365)

    def __init__(self, algorithm: QCAlgorithm) -> None:
        self._algorithm = algorithm

        # Warm up the SMA indicator.
        current = algorithm.time
        provider = algorithm.risk_free_interest_rate_model
        dt = current - timedelta(365)
        while dt <= current:
            rate = provider.get_interest_rate(dt)
            self._sma.update(dt, rate)
            self._was_rising = rate > self._sma.current.value
            dt += timedelta(1)

        # Set a schedule to update the interest rate trend indicator every day.
        algorithm.schedule.on(
            algorithm.date_rules.every_day()
```

```
            algorithm.date_rules.every_day(),
            algorithm.time_rules.at(0, 1),
            self.update_interest_rate
        )

    def update_interest_rate(self) -> None:
        # Update interest rate to the SMA indicator to estimate its trend.
        rate = self._algorithm.risk_free_interest_rate_model.get_interest_rate(self._algorithm.time)
        self._sma.update(self._algorithm.time, rate)
        self._was_rising = rate > self._sma.current.value

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Hold one position group at a time.
        chain = slice.option_chains.get(self._symbol)
        if algorithm.portfolio.invested or not chain:
            return insights

        # Obtain the Jelly Roll constituents from the option chain.
        calls = sorted([x for x in chain if x.right == OptionRight.CALL], key=lambda x: x.expiry)
        puts = sorted([x for x in chain if x.right == OptionRight.PUT], key=lambda x: x.expiry)
        near_call = calls[0]
        far_call = calls[-1]
        near_put = puts[0]
        far_put = puts[-1]

        # Emit insight of the Jelly Roll constituents, with directions depending on the interest rate trend given by SMA.
        rate = algorithm.risk_free_interest_rate_model.get_interest_rate(algorithm.time)
        # During the rising interest rate cycle, order a long Jelly Roll.
        if rate > self._sma.current.value:
            insights.extend([
                Insight.price(near_call.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(far_call.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(near_put.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(far_put.symbol, timedelta(30), InsightDirection.DOWN)
            ])
        # During the downward interest rate cycle, order short Jelly Roll.
        elif rate < self._sma.current.value:
            insights.extend([
                Insight.price(near_call.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(far_call.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(near_put.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(far_put.symbol, timedelta(30), InsightDirection.UP)
            ])
        # If the interest rate cycle is steady for a long, we expect a flip in the cycle coming up.
        elif self._was_rising:
            insights.extend([
                Insight.price(near_call.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(far_call.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(near_put.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(far_put.symbol, timedelta(30), InsightDirection.UP)
            ])
        else:
            insights.extend([
                Insight.price(near_call.symbol, timedelta(30), InsightDirection.DOWN),
                Insight.price(far_call.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(near_put.symbol, timedelta(30), InsightDirection.UP),
                Insight.price(far_put.symbol, timedelta(30), InsightDirection.DOWN)
            ])

        return insights

class SingleSharePortfolioConstructionModel(PortfolioConstructionModel):
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        for insight in insights:
            if algorithm.securities[insight.symbol].is_tradable:
                # Use integer target to create a portfolio target to trade a single contract
                targets.append(PortfolioTarget(insight.symbol, insight.direction))
        return targets
```

Example 2: Options of an Equity Universe

The following hybrid algorithm example chains a fundamental universe and an Equity Options universe . It first selects 10 stocks with the lowest PE ratio and then selects their front-month call Option contracts. It buys one front-month call Option contract every day.

To override the default pricing model of the Options, set a pricing model in a security initializer.

To override the initial guess of implied volatility , set and warm up the underlying volatility model .

PY

```
# Example code to chain a fundamental universe and an Equity Options universe by selecting top 10 stocks with lowest PE, indicating
potentially undervalued stocks and then selecting their from-month call Option contracts to target contracts with high liquidity.
from AlgorithmImports import *

class ChainedUniverseAlgorithm(QCAlgorithm):
    _day = 0
    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.seed_initial_prices = True
        self.universe_settings.asynchronous = True
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        self.add_security_initializer(self._custom_security_initializer)

        self.add_universe_selection(OptionChainedUniverseSelectionModel(
            # Add a universe of the top 10 stocks with lowest PE
            self.add_universe(self.fundamental_function),
            # Select call Option contracts on the underlying Equities that have the strike price within 2 strike levels.
            lambda option_filter_universe: option_filter_universe.strikes(-2, +2).front_month().calls_only()
            )
        )

    def fundamental_function(self, fundamental: List[Fundamental]) -> List[Symbol]:
        filtered = (f for f in fundamental if not np.isnan(f.valuation_ratios.pe_ratio))
        sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
        return [f.symbol for f in sorted_by_pe_ratio[:10]]

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the price model
        if security.type == SecurityType.OPTION: # Option type
            security.price_model = OptionPriceModels.binomial_cox_ross_rubinstein()

    def on_data(self, data: Slice) -> None:
        if self.is_warming_up or self._day == self.time.day:
            return

        for symbol, chain in data.option_chains.items():
            underlying = self.securities.get(chain.underlying.symbol)
            if not underlying:
                continue

            if underlying.invested:
                self.liquidate(underlying.symbol)

            spot = chain.underlying.price
            contract = sorted(chain, key=lambda x: abs(spot-x.strike))[0]
            tag = f"IV: {contract.implied_volatility:.3f} Δ: {contract.greeks.delta:.3f}"
            self.market_order(contract.symbol, 1, True, tag)
            self._day = self.time.day
```

Universe Selection

Legacy Fundamental Universes

Introduction

Warning: This API for Universe Selection was deprecated on November 2023. Please refer to the new Fundamental Selection API .

A `FundamentalUniverseSelectionModel` selects a universe of US Equities based on `CoarseFundamental` and, sometimes, `FineFundamental` data. If the model uses `CoarseFundamental` data, it relies on the US Equity Coarse Universe dataset. If the Universe Selection model uses `FineFundamental` data, it relies on the US Fundamental dataset.

These types of universes operate on daily schedule. In backtests, they select assets at midnight. In live trading, the selection timing depends on the data provider you use.

If you use a fundamental Universe Selection model, the only way to unsubscribe from a security is to return a list from the selection functions that doesn't include the security `Symbol` . The `remove_security` method doesn't work with these types of Universe Selection models.

Coarse Fundamental Selection

The `CoarseFundamentalUniverseSelectionModel` selects assets based on `CoarseFundamental` data. To use this model, define a coarse selection function. The coarse selection function receives a list of `CoarseFundamental` objects and returns a list of `Symbol` objects. The `Symbol` objects you return from the coarse selection function are the constituents of the universe.

```
# Set asynchronous universe settings for improved speed and use CoarseFundamentalUniverseSelectionModel with a custom function to select
the top 100 most liquid assets with available fundamental data, ensuring the universe consists of liquid and fundamentally strong
securities for stable returns.
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.add_universe_selection(CoarseFundamentalUniverseSelectionModel(self.select_coarse))

def select_coarse(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
    selected = [c for c in coarse if c.has_fundamental_data]
    sorted_by_dollar_volume = sorted(selected, key=lambda c: c.dollar_volume, reverse=True)
    return [c.symbol for c in sorted_by_dollar_volume[:100]] # Return most liquid assets w/ fundamentals
```

PY

To move the coarse selection function outside of the algorithm class, create a universe selection model that inherits the `CoarseFundamentalUniverseSelectionModel` class.

```
# Move the coarse selection function outside of the algorithm class by creating a custom universe selection model inheriting from
CoarseFundamentalUniverseSelectionModel, allowing for modification and reusability of the selection logic.
self.universe_settings.asynchronous = True
self.add_universe_selection(MostLiquidFundamentalUniverseSelectionModel(self.universe_settings))

# Outside of the algorithm class
class MostLiquidFundamentalUniverseSelectionModel(CoarseFundamentalUniverseSelectionModel):
    def __init__(self, universe_settings: UniverseSettings) -> None:
        super().__init__(self.select_coarse, universe_settings)

    def select_coarse(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        selected = [c for c in coarse if c.has_fundamental_data]
        sorted_by_dollar_volume = sorted(selected, key=lambda c: c.dollar_volume, reverse=True)
        return [c.symbol for c in sorted_by_dollar_volume[:100]]
```

PY

To return the current universe constituents from the coarse selection function, return `Universe.Unchanged` .

For more information about this model, see the class reference and implementation .

Fine Fundamental Selection

The `FineFundamentalUniverseSelectionModel` selects assets based on `CoarseFundamental` and `FineFundamental` data. This is the only model that provides corporate fundamental data to your algorithm. To use this model, define a coarse selection function and a fine selection function. The coarse selection function receives a list of `CoarseFundamental` objects and returns a list of `Symbol` objects. To filter the `CoarseFundamental` down to the securities that have fundamental data, add a `has_fundamental_data` filter to the coarse selection function. The fine selection function receives a

subset of `FineFundamental` objects generated from coarse selection results and returns a list of `Symbol` objects. The `Symbol` objects you return from the fine selection function are the constituents of the universe.

```py
# FineFundamentalUniverseSelectionModel integrates corporate fundamental data into the algorithm through a two-step filtering process:
coarse selection ensures liquidity and fundamental data availability, while fine selection focuses on valuation metrics, refining the
universe of selected stocks.
def initialize(self) -> None:
    self.universe_settings.asynchronous = True
    self.add_universe_selection(FineFundamentalUniverseSelectionModel(self.select_coarse, self.select_fine))

def select_coarse(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
    selected = [c for c in coarse if c.has_fundamental_data]
    sorted_by_dollar_volume = sorted(selected, key=lambda c: c.dollar_volume, reverse=True)
    return [c.symbol for c in sorted_by_dollar_volume[:100]] # Return most liquid assets w/ fundamentals

def select_fine(self, fine: List[FineFundamental]) -> List[Symbol]:
    sorted_by_pe_ratio = sorted(fine, key=lambda x: x.valuation_ratios.pe_ratio, reverse=False)
    return [c.symbol for c in sorted_by_pe_ratio[:10]] # Return assets with lowest P/E ratios
```

To move the selection functions outside of the algorithm class, create a universe selection model that inherits the `FineFundamentalUniverseSelectionModel` class.

```py
# Create a custom universe selection model inheriting from FineFundamentalUniverseSelectionModel to move both coarse and fine selection
functions outside the algorithm class, enabling easier modification and reusability of the filtering and ranking logic.
self.universe_settings.asynchronous = True
self.add_universe_selection(LowPERatioUniverseSelectionModel(self.universe_settings))

# Outside of the algorithm class
class LowPERatioUniverseSelectionModel(FineFundamentalUniverseSelectionModel):
    def __init__(self, universe_settings: universe_settings = None) -> None:
        super().__init__(self.select_coarse, self.select_fine, universe_settings)

    def select_coarse(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        selected = [c for c in coarse if c.has_fundamental_data]
        sorted_by_dollar_volume = sorted(selected, key=lambda c: c.dollar_volume, reverse=True)
        return [c.symbol for c in sorted_by_dollar_volume[:100]]

    def select_fine(self, fine: List[FineFundamental]) -> List[Symbol]:
        sorted_by_pe_ratio = sorted(fine, key=lambda x: x.valuation_ratios.pe_ratio, reverse=False)
        return [c.symbol for c in sorted_by_pe_ratio[:10]]
```

To return the current universe constituents from the coarse or fine selection function, return `Universe. UNCHANGED` .

If you add a `FineFundamentalUniverseSelectionModel` to your algorithm, you can access fundamental data in the fine selection function or from the `Equity` object. To access fundamental data from the `Equity` object, use the `Equity. fundamentals` property.

For more information about this model, see the class reference and implementation .

Algorithm Framework

Alpha

Alpha

Key Concepts

Introduction

The Alpha model predicts market trends and signals the best moments to trade. These signals, or `Insight` objects, contain the `Direction` , `Magnitude` , and `Confidence` of a market prediction and the suggested portfolio `Weight` . You should generate insights on the set of assets provided by the Universe Selection model and only generate them when your predictions change.

Add Models

To add an Alpha model, in the `initialize` method, call the `AddAlpha` method.

```
# Add EmaCrossAlphaModel to generate market trend insights and trading signals based on EMA crossovers, providing Direction, Magnitude,
and Confidence for the universe-selected assets.
self.add_alpha(EmaCrossAlphaModel())
```

PY

To view all the pre-built Alpha models, see Supported Models .

Multi-Alpha Algorithms

You can add multiple Alpha models to a single algorithm and generate Insight objects with all of them.

```
# Add RsiAlphaModel and EmaCrossAlphaModel to generate trading signals based on RSI and EMA crossovers, utilizing multiple alpha models to
capture different market signals and trends for robust predictions.
self.add_alpha(RsiAlphaModel())
self.add_alpha(EmaCrossAlphaModel())
```

PY

Each Alpha model has a unique name. The Insight objects they generate are automatically named according to the Alpha model name.

If you add multiple Alpha models, each alpha model receives the current slice in the order that you add the Alphas. The combined stream of Insight objects is passed to the Portfolio Construction model that defines how the Insight collection is combined. If you have a hybrid algorithm, the combined stream of Insight objects is passed to the event handler .

Model Structure

Alpha models should extend the `AlphaModel` class. Extensions of the `AlphaModel` class must implement the `update` method, which receives a Slice object and returns an array of `Insight` objects. Extensions should also implement the `on_securities_changed` method to track security changes in the universe.

```
# Algorithm framework model that produces insights
class MyAlphaModel(AlphaModel):

    def update(self, algorithm: QCAlgorithm, data: Slice) -> List[Insight]:
        # Updates this Alpha model with the latest data from the algorithm.
        # This is called each time the algorithm receives data for subscribed securities
        # Generate insights on the securities in the universe.
        insights = []
        return insights

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # Security additions and removals are pushed here.
        # This can be used for setting up algorithm state.
        # changes.added_securities
        # changes.removed_securities
        pass
```

Method Parameters

The `algorithm` parameter that the methods receive is an instance of the base `QCAlgorithm` class, not your subclass of it. To access members of your algorithm object, either use a global variable or pass an algorithm reference to Alpha model constructor. Both of these approaches violate the separation of concerns principle.

The `data` parameter contains the latest data available to the algorithm.

The `changes` parameter contains the universe changes.

Model Names

To ensure your Alpha model is compatible with all Portfolio Construction models , assign a unique name to your Alpha model. Some Portfolio Construction models can combine multiple Alpha models together, so it can be important to distinguish between the Alpha models.

```
# Assign a unique name to RsiAlphaModel to ensure compatibility and distinguish it when combining with other Alpha models in portfolio
construction.
class RsiAlphaModel(AlphaModel):
    name = "RsiAlphaModel"
```

By default, LEAN uses the string representation of a Guid object to set the Alpha model name. An example default name is "0f8fad5b-d9cb-469f-a165-70867728950e". This default name is different for every backtest you run and every live algorithm you deploy. For consistent behavior, manually set the Alpha model name.

Example

To view a full example of an `AlphaModel` subclass, see the ConstantAlphaModel in the LEAN GitHub repository.

Track Security Changes

The Universe Selection model may select a dynamic universe of assets, so you should not assume a fixed set of assets in the Alpha model. When the Universe Selection model adds and removes assets from the universe, it triggers an `on_securities_changed` event. In the `on_securities_changed` event handler, you can initialize the security-specific state or load any history required for your Alpha model. If you need to save data for individual securities, add custom members to the respective `Security` object .

```
# Handle dynamic asset changes by initializing indicators for new securities and removing them for removed securities to keep the Alpha
model adaptive and accurate.
class MyAlphaModel(AlphaModel):
    _securities = []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            security.indicator = algorithm.sma(security.symbol, 20)
            algorithm.warm_up_indicator(security.symbol, security.indicator)
            self._securities.append(security)

        for security in changes.removed_securities:
            if security in self._securities:
                algorithm.deregister_indicator(security.indicator)
                self._securities.remove(security)
```

Insights

An Insight is a single prediction for an asset. The `update` method returns an array of Insight objects. You can think of these as actionable trading signals, indicating the asset direction, magnitude, and confidence in the near future. All insights can take a weight parameter to set the desired weighting for the insight. If you change the cash of the algorithm, it will affect the orders, but not necessarily the insights.

The Portfolio Construction model consumes the Insight objects from the Alpha model. It's up to the Portfolio Construction model to define how Insight objects are converted into `PortfolioTarget` objects. In the pre-built Portfolio Construction models, down insights translate to short-biased trades, up insights translate to long-biased trades, and flat insights usually close open positions, but this doesn't have to be the case in custom Portfolio Construction models .

Insight Properties

`Insight` objects have the following properties:

Create Insights

To create an `Insight` , call the `Insight` constructor.

```
# Insight(symbol, period, type, direction, magnitude=None, confidence=None, source_model=None, weight=None, tag='')
insight = Insight("IBM", timedelta(minutes=20), InsightType.PRICE, InsightDirection.UP)
```

In the `Insight` constructor, the `period` argument can be a `timedelta` object or a function that receives a `datetime` object and returns the expiry `datetime` . Some of the constructor arguments are optional to create the `Insight` object, but some of the Portfolio Construction models may require these properties. To check which `Insight` properties a Portfolio Construction model requires, see Supported Models .

You can also use the helper method to create Insight objects of the Price type.

```
# Insight.price(symbol, period, direction, magnitude=None, confidence=None, source_model=None, weight=None, tag='')
insight = Insight.price("IBM", timedelta(minutes = 20), InsightDirection.UP)

# Insight.price(symbol, resolution, barCount, direction, magnitude=None, confidence=None, source_model=None, weight=None, tag='')
insight = Insight.price("IBM", Resolution.MINUTE, 20, InsightDirection.UP)
```

In the `price` method, the `period` argument can be a `timedelta` object, a `datetime` object, or a function that receives a `datetime` object and returns the expiry `datetime` .

Group Insights

Sometimes an algorithm's performance relies on multiple insights being traded together, like in pairs trading and options straddles. These types insights should be grouped. Insight groups signal to the Execution model that the insights need to be acted on as a single unit to maximize the alpha created.

To mark insights as a group, call the `Insight.Group` method.

```
# Generate and group insights for a set of tickers with a 20-minute prediction horizon; each Insight object provides directional signals,
magnitudes, and confidence for price movement.
tickers = ["IBM", "SPY", "AAPL"]
insights = [Insight.price(ticker, timedelta(minutes = 20), InsightDirection.UP) for ticker in tickers]
return Insight.group(insights)
```

Cancel Insights

If an Alpha model in your algorithm emits an Insight to enter a position but it determines the trading opportunity has pre-maturely ended, you should cancel the Insight. For example, say you want your algorithm to enter a long position in a security when its Relative Strength Index (RSI) moves below 20 and then liquidate the position when the security's RSI moves above 30. If you emit an Insight that has a duration of 30 days when the RSI moves below 20 but the RSI moves above 30 in 10 days, you should cancel the Insight.

To cancel an Insight, call its `cancel` / `Expire` method with the algorithm's Coordinated Universal Time (UTC).

```
# Cancel any outstanding insights to ensure only current predictions are considered and to avoid outdated signals influencing trading
decisions.
self.insight.cancel(algorithm.utc_time)
```

If you don't have a reference to the Insight you want to cancel, get it from the InsightManager .

When you cancel an insight, it's `close_time_utc` property is set to one second into the past.

Stop Loss Orders Workaround

In some cases, if you add a Risk Management model that uses stop loss logic, the Risk Management model generates `PortfolioTarget` objects with a 0 quantity to make the Execution model liquidate your positions, but then the Portfolio Construction model generates `PortfolioTarget` objects with a non-zero quantity to make the Execution model re-enter the same position. This issue can occur if your Portfolio Construction model rebalances and your Alpha model still has an active insight for the liquidated securities. To avoid this issue, add the stop loss order logic to the Alpha model. When the stop loss is hit, emit a flat insight for the security. Note that this is a workaround, but it violates the separation of concerns principle since the Alpha Model shouldn't react to open positions.

Universe Timing Considerations

If the Alpha model manages some indicators or consolidators for securities in the universe and the universe selection runs during the indicator sampling period or the consolidator aggregation period, the indicators and consolidators might be missing some data. For example, take the following scenario:

- The security resolution is minute
- You have a consolidator that aggregates the security data into daily bars to update the indicator
- The universe selection runs at noon

In this scenario, you create and warm-up the indicator at noon. Since it runs at noon, the history request that gathers daily data to warm up the indicator won't contain any data from the current day and the consolidator that updates the indicator also won't aggregate any data from before noon. This process doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Examples

The following examples demonstrate common practices for implementing the framework alpha model.

Example 1: EMA Cross On Liquid Stocks

The following algorithm selects the top 500 liquid US stocks traded within the SPY ETF, then emits trade signals 60-day versus 200-day EMA cross through the `EmaCrossAlphaModel` . The `EqualWeightingPortfolioConstructionModel` ensures each asset with an insight gets an allocation of equal magnitude.

```
class FrameworkAlphaModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe of liquid equities since they have capital to support demand and trends.
        self.universe_settings.resolution = Resolution.DAILY
        self.add_universe_selection(ETFConstituentsUniverseSelectionModel("SPY"))
        # Using an EMA cross-alpha model to trade on medium versus long-term trends.
        self.add_alpha(EmaCrossAlphaModel(60, 200, Resolution.DAILY))
        # Equally invested in insights to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

Example 2: Interest Rate Effect On Forex

The following algorithm implements a custom alpha model that uses the interest rate cycle as a signal to trade Forex. We trade the top 6 forexes: AUDUSD, EURUSD, GBPUSD, USDCAD, USDCHF, and USDJPY. For the ones with USD as quote currencies, we buy them during the US interest rate down cycle since USD is relatively weak while shorting them during the upcycle. The opposite is true for the ones with USD as the base currency.

PY

```
class FrameworkAlphaModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Use daily resolution since the interest rate signal is daily.
        self.universe_settings.resolution = Resolution.DAILY
        # Add a universe of selected list of Forex.
        forex = [
            Symbol.create("AUDUSD", SecurityType.FOREX, Market.OANDA),
            Symbol.create("EURUSD", SecurityType.FOREX, Market.OANDA),
            Symbol.create("GBPUSD", SecurityType.FOREX, Market.OANDA),
            Symbol.create("USDCAD", SecurityType.FOREX, Market.OANDA),
            Symbol.create("USDCHF", SecurityType.FOREX, Market.OANDA),
            Symbol.create("USDJPY", SecurityType.FOREX, Market.OANDA)
        ]
        self.add_universe_selection(ManualUniverseSelectionModel(forex))
        # Using a custom alpha model to trade forex based on interest rate.
        self.add_alpha(InterestRateAlphaModel(self))
        # Equally invested in insights to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

class InterestRateAlphaModel(AlphaModel):
    # Use a 365d SMA indicator of daily interest rate to estimate if the interest rate cycle is upward or downward.
    _sma = SimpleMovingAverage(365)

    def __init__(self, algorithm: QCAlgorithm) -> None:
        self._algorithm = algorithm

        # Warm up the SMA indicator.
        current = algorithm.time
        provider = algorithm.risk_free_interest_rate_model
        dt = current - timedelta(365)
        while dt <= current:
            rate = provider.get_interest_rate(dt)
            self._sma.update(dt, rate)
            self._was_rising = rate > self._sma.current.value
            dt += timedelta(1)

        # Set a schedule to update the interest rate trend indicator every day.
        algorithm.schedule.on(
            algorithm.date_rules.every_day(),
            algorithm.time_rules.at(0, 1),
            self.update_interest_rate
        )

    def update_interest_rate(self) -> None:
        # Update interest rate to the SMA indicator to estimate its trend.
        rate = self._algorithm.risk_free_interest_rate_model.get_interest_rate(self._algorithm.time)
        self._sma.update(self._algorithm.time, rate)
        self._was_rising = rate > self._sma.current.value

    def update(self, algorithm: QCAlgorithm, slice: Slice) -> List[Insight]:
        insights = []

        # Split the forexes by whether the quote currency is USD.
        quote_usd = []
        base_usd = []
        for symbol, security in algorithm.securities.items():
            if security.quote_currency.symbol == Currencies.USD:
                quote_usd.append(symbol)
            else:
                base_usd.append(symbol)

        rate = algorithm.risk_free_interest_rate_model.get_interest_rate(algorithm.time)
        # During the rising interest rate cycle, long the forexes with USD as the base currency and short the ones with USD as the quote
currency.
        if rate > self._sma.current.value:
            insights.extend(
                [Insight.price(symbol, timedelta(1), InsightDirection.UP) for symbol in base_usd] +
                [Insight.price(symbol, timedelta(1), InsightDirection.DOWN) for symbol in quote_usd]
            )
        # During the downward interest rate cycle, short the forexes with USD as the base currency and long the ones with USD as the
quote currency.
        elif rate < self._sma.current.value:
            insights.extend(
                [Insight.price(symbol, timedelta(1), InsightDirection.DOWN) for symbol in base_usd] +
                [Insight.price(symbol, timedelta(1), InsightDirection.UP) for symbol in quote_usd]
            )
        # If the interest rate cycle is steady for a long time, we expect a flip in the cycle.
        elif self._was_rising:
            insights.extend(
                [Insight.price(symbol, timedelta(1), InsightDirection.DOWN) for symbol in base_usd] +
                [Insight.price(symbol, timedelta(1), InsightDirection.UP) for symbol in quote_usd]
            )
        else:
            insights.extend(
                [Insight.price(symbol, timedelta(1), InsightDirection.UP) for symbol in base_usd] +
                [Insight.price(symbol, timedelta(1), InsightDirection.DOWN) for symbol in quote_usd]
            )

        return insights
```

Alpha

Supported Models

Introduction

This page describes the pre-built Alpha models in LEAN. The number of models grows over time. To add a model to LEAN, make a pull request to the GitHub repository . If none of these models perform exactly how you want, create a custom Alpha model .

Null Model

The `NullAlphaModel` doesn't emit any insights. It's the default Alpha model.

```py
# NullAlphaModel doesn't emit insights.
self.add_alpha(NullAlphaModel())
```

For more information about this model, see the class reference and implementation .

Constant Model

The `ConstantAlphaModel` always returns the same insight for each security.

```py
# Emit up insights that expire in 30 days for all assets.
self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(30)))
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`type`	`InsightType`	The type of insight	
`direction`	`InsightDirection`	The direction of the insight	
`period`	`timedelta`	The period over which the insight will come to fruition	
`magnitude`	`float/NoneType`	The predicted change in magnitude as a +/- percentage	`None`
`confidence`	`float/NoneType`	The confidence in the insight	`None`
`weight`	`float/NoneType`	The portfolio weight of the insights	`None`

For more information about this model, see the class reference and implementation .

Historical Returns Model

The `HistoricalReturnsAlphaModel` buys securities that have a positive trailing return and sells securities that have a negative trailing return. It sets the magnitude of the Insight objects to the trailing rate of change.

```py
# Add HistoricalReturnsAlphaModel to leverage historical return data for generating alpha signals, identifying trends based on past
performance.
self.add_alpha(HistoricalReturnsAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`lookback`	`int`	Historical return lookback period	1
`resolution`	`Resolution`	The resolution of historical data	`Resolution.DAILY`

This model cancels all the active insights it has emit for a security when the security has a 0% historical return.

For more information about this model, see the class reference and implementation .

EMA Cross Model

The `EmaCrossAlphaModel` uses an exponential moving average (EMA) cross to create insights. When the fast EMA crosses above the slow EMA, it emits up insights. When the fast EMA crosses below the slow EMA, it emits down insights. It sets the duration of Insight objects to be the product of the `resolution` and `fast_period` arguments.

```
# Add EmaCrossAlphaModel to generate trading insights based on the crossing of fast and slow exponential moving averages identifying
trends and potential reversals. The duration of insights is set by the resolution and fast_period to ensure timely and relevant trading
signals.
self.add_alpha(EmaCrossAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`fast_period`	`int`	The fast EMA period	12
`slow_period`	`int`	The slow EMA period	26
`resolution`	`Resolution`	The resolution of data sent into the EMA indicators	`Resolution.DAILY`

For more information about this model, see the class reference and implementation .

MACD Model

The `MacdAlphaModel` emits insights based on moving average convergence divergence (MACD) crossovers. If the MACD signal line is 1% above the security price, the model emits an up insight. If the MACD signal line is 1% below the security price, the model emits a down insight. If the MACD signal line is within 1% of the security price, the model cancels all the active insights it has emitted for the security.

```
# Use MacdAlphaModel to generate insights based on MACD crossovers, emitting up signals when the MACD signal line is 1% above the security
price and down signals when it is 1% below, while canceling insights if within 1% capturing significant trend changes while avoiding
noise from minor fluctuations.
self.add_alpha(MacdAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`fast_period`	`int`	The MACD fast period	12
`slow_period`	`int`	The MACD slow period	26
`signal_period`	`int`	The smoothing period for the MACD signal	9
`moving_average_type`	`MovingAverageType`	The type of moving average to use in the MACD	`MovingAverageType. EXPONENTIAL`
`resolution`	`Resolution`	The resolution of data sent into the MACD indicator	`Resolution.DAILY`

For more information about this model, see the class reference and implementation .

RSI Model

The `RsiAlphaModel` generates insights based on the relative strength index (RSI) indicator values. When the RSI value passes above 70, the model emits a down insight. When the RSI value passes below 30, the model emits an up insight. The model uses the Wilder moving average type and sets the duration of Insight objects to be the product of the `resolution` and `period` arguments.

```
# The RsiAlphaModel generates insights based on the RSI indicator: emits a down insight when RSI exceeds 70 (overbought) and an up insight
when RSI falls below 30 (oversold). This helps identify potential price reversals based on momentum.
self.add_alpha(RsiAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`period`	`int`	The RSI indicator period	14
`resolution`	`Resolution`	The resolution of data sent into the RSI indicator	`Resolution.DAILY`

For more information about this model, see the class reference and implementation .

Base Pairs Trading Model

The `BasePairsTradingAlphaModel` analyzes every possible pair combination from securities that the Universe Selection model selects. This model calculates a ratio between the two securities by dividing their historical prices over a lookback window. It then calculates the mean of this ratio by taking the 500-period EMA of the quotient. When the ratio diverges far enough from the mean ratio, this model emits generates alternating long ratio/short ratio insights emitted as a group to capture the reversion of the ratio.

```
# Use BasePairsTradingAlphaModel to analyze security pairs from the universe, generating long/short insights based on deviations from a
500-period EMA of their price ratio to capture mean-reversion opportunities.
self.add_alpha(BasePairsTradingAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`lookback`	`int`	Lookback period of the analysis	1
`resolution`	`Resolution`	Analysis resolution	`Resolution.DAILY`
`threshold`	`float`	The percent [0, 100] deviation of the ratio from the mean before emitting an insight	1

For more information about this model, see the class reference and implementation .

Pearson Correlation Pairs Trading Model

The `PearsonCorrelationPairsTradingAlphaModel` ranks every pair combination by its Pearson correlation coefficient and trades the pair with the highest correlation. This model follows the same insight logic as the `BasePairsTradingModel` .

```
# Add PearsonCorrelationPairsTradingAlphaModel to rank and trade the most correlated security pairs based on Pearson correlation
coefficients, using a similar insight logic as BasePairsTradingModel to capture mean-reversion opportunities.
self.add_alpha(PearsonCorrelationPairsTradingAlphaModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`lookback`	`int`	Lookback period of the analysis	15
`resolution`	`Resolution`	Analysis resolution	`Resolution.MINUTE`
`threshold`	`float`	The percent [0, 100] deviation of the ratio from the mean before emitting an insight	1
`minimum_correlation`	`float`	The minimum correlation to consider a tradable pair	0.5

For more information about this model, see the class reference and implementation .

Algorithm Framework

Portfolio Construction

Portfolio Construction

Key Concepts

Introduction

The Portfolio Construction model receives `Insight` objects from the Alpha model and creates `PortfolioTarget` objects for the Risk Management model. A `PortfolioTarget` provides the number of units of an asset to hold.

Set Models

To set a Portfolio Construction model, in the `initialize` method, call the `set_portfolio_construction` method.

```
PY
# Create PortfolioTarget objects to form an equal weighted portfolio based on the Insight objects.
self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

To view all the pre-built Portfolio Construction models, see Supported Models .

Model Structure

Portfolio Construction models should extend the `PortfolioConstructionModel` class or one of the supported models . Extensions of the `PortfolioConstructionModel` class should implement the `create_targets` method, which receives an array of `Insight` objects from the Alpha model at every time step and returns an array of `PortfolioTarget` objects. The Portfolio Construction model seeks to answer the question, "how many units should I buy based on the insight predictions I've been presented?".

If you don't override the `create_targets` method, the base class implementation calls the model's `is_rebalance_due` , `determine_target_percent` , and `get_target_insights` helper methods. The `get_target_insights` method, in turn, calls the model's `should_create_target_for_insight` method. You can override any of these helper methods. If you don't override the `create_targets` method from the `PortfolioConstructionModel` class, your class must at least override the `determine_target_percent` method.

```
PY
 # Portfolio construction scaffolding class; basic method arguments.
class MyPortfolioConstructionModel(PortfolioConstructionModel):
    # Create list of PortfolioTarget objects from Insights.
    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        return super().create_targets(algorithm, insights)

    # Determine if the portfolio should rebalance based on the provided rebalancing function.
    def is_rebalance_due(self, insights: List[Insight], algorithmUtc: datetime) -> bool:
        return super().is_rebalance_due(insights, algorithmUtc)

    # Determine the target percent for each insight.
    def determine_target_percent(self, activeInsights: List[Insight]) -> dict[Insight, float]:
        return {}

    # Get the target insights to calculate a portfolio target percent.
    # They will be piped to the determine_target_percent method.
    def get_target_insights(self) -> List[Insight]:
        return super().get_target_insights()

    # Determine if the portfolio construction model should create a target for this insight.
    def should_create_target_for_insight(self, insight: Insight) -> bool:
        return super().should_create_target_for_insight(insight)

    # Track universe changes.
    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        super().on_securities_changed(algorithm, changes)
```

The Portfolio Construction model should remove expired insights from the Insight Manager . The `create_targets` definition of the base `PortfolioConstructionModel` class already removes them during each rebalance. Therefore, if you override the `create_targets` method and don't call the `create_targets` definition of the base class, your new method definition should remove expired insights from the Insight Manager.

The model should also remove all a security's insights from the Insight Manager when the security is removed from the universe . The `on_securities_changed` definition of the base `PortfolioConstructionModel` class already does this. Therefore, if you override the `on_securities_changed` method and don't call the `on_securities_changed` definition of the base class, your new method definition should remove the security's insights from the Insight Manager.

The `algorithm` argument that the methods receive is an instance of the base `QCAlgorithm` class, not your subclass of it.

You may use the `PortfolioBias` enumeration in the definition of Portfolio Construction model methods. The `PortfolioBias` enumeration has the following members:

To view a full example of a `PortfolioConstructionModel` subclass, see the EqualWeightingPortfolioConstructionModel in the LEAN GitHub repository.

Multi-Alpha Algorithms

If you add multiple Alpha models , each Alpha model receives the current slice in the order that you add the Alphas. The combined stream of Insight objects is passed to the Portfolio Construction model.

Each Portfolio Construction model has a unique method to combine Insight objects. The base `PortfolioConstructionModel` that most PCM's inherit from doesn't combine information from Insight objects with the same `Symbol` - but just gets the most recent active insight. To combine the active insights differently, override the `get_target_insights` , and return all active insights. The `determine_target_percent` method implements the combination criteria and determines the target for each `Symbol` .

```
PY
# Implement MultipleAlphaPortfolioConstructionModel to handle and utilize insights from multiple Alpha models.
# The get_target_insights method retrieves current active insights, and determine_target_percent allocates portfolio weights accordingly
for integrating and balancing multiple Alpha signals within the portfolio.
class MultipleAlphaPortfolioConstructionModel(PortfolioConstructionModel):
    def get_target_insights(self) -> List[Insight]:
        return self.algorithm.insights.get_active_insights(self.algorithm.utc_time)

    def determine_target_percent(self, activeInsights: List[Insight]) -> dict[Insight, float]:
        return {}
```

Portfolio Targets

The Portfolio Construction model returns `PortfolioTarget` objects, which are passed to the Risk Management model.

To create a `PortfolioTarget` object based on a quantity, pass the `Symbol` and quantity to the `PortfolioTarget` constructor.

```
PY
# Create a new portfolio target for 1200 IBM shares.
target = PortfolioTarget("IBM", 1200)
```

To create a `PortfolioTarget` object based on a portfolio weight, call the `percent` method. This method is only available for margin accounts.

```
PY
# Calculate target equivalent to 10% of portfolio value
target = PortfolioTarget.percent(algorithm, "IBM", 0.1)
```

To include more information in the `PortfolioTarget` object, pass a `tag` argument to the constructor or the `percent` method.

The `create_targets` method of your Portfolio Construction model must return an array of `PortfolioTarget` objects.

```
PY
return [ PortfolioTarget("IBM", 1200) ]
```

Portfolio Target Collection

The `PortfolioTargetCollection` class is a helper class to manage `PortfolioTarget` objects. The class manages an internal dictionary that has the

security `Symbol` as the key and a `PortfolioTarget` as the value.

Add Portfolio Targets

To add a `PortfolioTarget` to the `PortfolioTargetCollection` , call the `add` method.

```
self.targets_collection.add(portfolio_target)
```

To add a list of `PortfolioTarget` objects, call the `add_range` method.

```
self.targets_collection.add_range(portfolio_targets)
```

Check Membership

To check if a `PortfolioTarget` exists in the `PortfolioTargetCollection` , call the `contains` method.

```
target_in_collection = self.targets_collection.contains(portfolio_target)
```

To check if a Symbol exists in the `PortfolioTargetCollection` , call the `contains_key` method.

```
symbol_in_collection = self.targets_collection.contains_key(symbol)
```

To get all the Symbol objects, use the `keys` property.

```
symbols = self.targets_collection.keys
```

Access Portfolio Targets

To access the `PortfolioTarget` objects for a Symbol, index the `PortfolioTargetCollection` with the Symbol.

```
portfolio_target = self.targets_collection[symbol]
```

To iterate through the `PortfolioTargetCollection` , call the `get_enumerator` method.

```
enumerator = self.targets_collection.get_enumerator()
```

To get all the `PortfolioTarget` objects, use the `values` property

```
portfolio_targets = self.targets_collection.values
```

Order Portfolio Targets by Margin Impact

To get an enumerable where position reducing orders are executed first and the remaining orders are executed in decreasing order value, call the `order_by_margin_impact` method.

```
for target in self.targets_collection.order_by_margin_impact(algorithm):
    # Place order
```

This method won't return targets for securities that have no data yet. This method also won't return targets for which the sum of the current holdings and open orders quantity equals the target quantity.

Remove Portfolio Targets

To remove a `PortfolioTarget` from the `PortfolioTargetCollection` , call the `remove` method.

```
PY
remove_successful = self.targets_collection.remove(symbol)
```

To remove all the `PortfolioTarget` objects, call the `clear` method.

```
PY
self.targets_collection.clear()
```

To remove all the `PortfolioTarget` objects that have been fulfilled, call the `clear_fulfilled` method.

```
PY
self.targets_collection.clear_fulfilled(algorithm)
```

Rebalance Frequency

If you use a Portfolio Construction model that is a subclass of the `PortfolioConstructionModel` class, you can set the rebalancing frequency of the model with a function. The rebalancing function receives the Coordinated Universal Time (UTC) of the algorithm and should return the next rebalance UTC time or `None` . If the function returns `None` , the model doesn't rebalance unless the rebalance settings trigger a rebalance. For a full example of a custom rebalance function, see the PortfolioRebalanceOnCustomFuncRegressionAlgorithm .

If you use a Portfolio Construction model with the following characteristics, you can also set the rebalancing frequency of the model with a `timedelta` , `Resolution` , or DateRules :

- The model is a subclass of the `EqualWeightingPortfolioConstructionModel` class.
- The model constructor calls the `EqualWeightingPortfolioConstructionModel` constructor.
- The model doesn't override the `create_targets` method.

To check which of the pre-built Portfolio Construction models support this functionality, see Supported Models .

Rebalance Settings

By default, portfolio construction models create `PortfolioTarget` objects to rebalance the portfolio when any of the following events occur:

- The model's rebalance function signals it's time to rebalance
- The Alpha model emits new insights
- The universe changes

To disable rebalances when the Alpha model emits insights or when insights expire, set `rebalance_portfolio_on_insight_changes` to false.

```
PY
# Disable automatic portfolio rebalancing upon insight change, allowing for manual control over when portfolio adjustments are made based
on insights.
self.settings.rebalance_portfolio_on_insight_changes = False
```

To disable rebalances when security changes occur, set `rebalance_portfolio_on_security_changes` to false.

```
PY
# Disable automatic portfolio rebalancing upon security change, allowing for manual control over when portfolio adjustments are made
based on security additions or removals.
self.settings.rebalance_portfolio_on_security_changes = False
```

Portfolio Optimizer Structure

Some portfolio construction models contain an optimizer that accepts the historical returns of each security and returns a list of optimized portfolio weights. Portfolio optimizer models must implement the `IPortfolioOptimizer` interface, which has an `optimize` method.

```py
# Implement an equal-weighted portfolio optimizer to assign equal weights to all securities, providing basic diversification to reduce
risk compared to a concentrated portfolio.
class MyPortfolioOptimizer:

    def optimize(self, historicalReturns: pd.DataFrame, expectedReturns: pd.Series = None, covariance: pd.DataFrame = None) -> pd.Series:
        # Create weights
        #  For example, equal-weighting:
        num_assets = historical_returns.shape[1]
        weights = [1/num_assets] * num_assets

        return weights
```

The following table describes the arguments the `optimize` method accepts:

Argument	Data Type	Description	Default Value
`historical_returns`	`DataFrame`	Matrix of historical returns where each column represents a security and each row returns for the given date/time (size: K x N)	
`expected_returns`	`Series`	Array of double with the portfolio annualized expected returns (size: K x 1)	`None`
`covariance`	`DataFrame`	Multi-dimensional array of double with the portfolio covariance of annualized returns (size: K x K)	`None`

The method should return a K x 1 array of double objects that represent the portfolio weights.

To view all the pre-built portfolio optimization algorithms, see Supported Optimizers .

To view a full example of an `IPortfolioOptimizer` implementation, see the MaximumSharpeRatioPortfolioOptimizer in the LEAN GitHub repository.

If you define a custom optimizer and want to use it as the `optimizer` argument for one of the pre-built Portfolio Construction models , import the Python version of the Portfolio Construction model into your project file. For example, to pair your optimizer with the Black Litterman Optimization Model , add the following line:

```py
from Portfolio.BlackLittermanOptimizationPortfolioConstructionModel import BlackLittermanOptimizationPortfolioConstructionModel
```

Track Security Changes

The Universe Selection model may select a dynamic universe of assets, so you should not assume a fixed set of assets in the Portfolio Construction model. When the Universe Selection model adds and removes assets from the universe, it triggers an `on_securities_changed` event. In the `on_securities_changed` event handler, you can initialize the security-specific state or load any history required for your Portfolio Construction model. If you need to save data for individual securities, add custom members to the respective `Security` object .

```py
class MyPortfolioConstructionModel(PortfolioConstructionModel):
    _securities = []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        super().on_securities_changed(algorithm, changes)
        for security in changes.added_securities:
            # Store and manage Symbol-specific data
            security.indicator = algorithm.sma(security.symbol, 20)
            algorithm.warm_up_indicator(security.symbol, security.indicator)

            self._securities.append(security)

        for security in changes.removed_securities:
            if security in self.securities:
                algorithm.deregister_indicator(security.indicator)
                self._securities.remove(security)
```

Universe Timing Considerations

If the Portfolio Construction model manages some indicators or consolidators for securities in the universe and the universe selection runs during the indicator sampling period or the consolidator aggregation period, the indicators and consolidators might be missing some data. For example, take the following scenario:

- The security resolution is minute
- You have a consolidator that aggregates the security data into daily bars to update the indicator
- The universe selection runs at noon

In this scenario, you create and warm-up the indicator at noon. Since it runs at noon, the history request that gathers daily data to warm up the indicator won't contain any data from the current day and the consolidator that updates the indicator also won't aggregate any data from before noon. This process doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Examples

The following examples demonstrate common practices for implementing the framework portfolio construction model.

Example 1: All-Weather Portfolio

The following algorithm uses `RiskParityPortfolioConstructionModel` to construct an all-weather portfolio that dissipates 1-year daily variance equally among ETFs that represent different asset classes, including stock market (SPY), bond (TLT), gold (GLD), oil (USO), and agricultural(DBA).

```
class FrameworkPortfolioConstructionModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add a universe of selected lists of ETFs that represent various asset classes.
        etfs = [
            Symbol.create("SPY", SecurityType.EQUITY, Market.USA),  # stock market
            Symbol.create("TLT", SecurityType.EQUITY, Market.USA),  # bond
            Symbol.create("GLD", SecurityType.EQUITY, Market.USA),  # gold
            Symbol.create("USO", SecurityType.EQUITY, Market.USA),  # oil
            Symbol.create("DBA", SecurityType.EQUITY, Market.USA)   # agricultural
        ]
        self.add_universe_selection(ManualUniverseSelectionModel(etfs))
        # Emit insights for all selected ETFs.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(1)))# Using risk parity PCM to construct an
all-weather portfolio to dissipate risk.
        self.set_portfolio_construction(RiskParityPortfolioConstructionModel())
```

Example 2: Protective Position

The following algorithm implements an equal-weighting portfolio on the given insights. To hedge catastrophic downside risk, each position would order a protective call/put 10% OTM.

PY

```
class FrameworkPortfolioConstructionModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)

        # Set to raw data normalization for a strike and the underlying price fair comparison.
        self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
        # Add a universe of the market representative SPY.
        spy = [Symbol.create("SPY", SecurityType.EQUITY, Market.USA)]
        self.add_universe_selection(ManualUniverseSelectionModel(spy))
        # Emit insights for all selected ETFs. Each insight lasts for 7 days for the option hedge expiry.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Using a custom PCM to control the order size and hedge option position ordering with a 10% OTM threshold.
        self.set_portfolio_construction(ProtectivePositionPortfolioConstructionModel(0.1))

class ProtectivePositionPortfolioConstructionModel(PortfolioConstructionModel):
    def __init__(self, threshold: float) -> None:
        self.threshold = threshold

    def create_targets(self, algorithm: QCAlgorithm, insights: List[Insight]) -> List[PortfolioTarget]:
        targets = []
        if not insights:
            return targets

        # Equally invest in each position group to dissipate the capital risk.
        fund_per_insight = algorithm.portfolio.total_portfolio_value / len(insights)

        for insight in insights:
            underlying = insight.symbol
            # Hedge position option right: long position should purchase OTM put, while short should purchase OTM call.
            right = OptionRight.PUT if insight.direction == InsightDirection.UP else OptionRight.CALL
            threshold = 1 - self.threshold if insight.direction == InsightDirection.UP else 1 + self.threshold

            # Select a protective option position that expires in 7 days.
            hedge = sorted([x for x in algorithm.option_chain(underlying) if x.expiry < algorithm.time + timedelta(7) and x.right ==
right],
                           key=lambda x: (x.expiry, -abs(x.strike - x.underlying_last_price * threshold)),
                           reverse=True)[0]
            # Request the protective position data for trading.
            hedge_symbol = algorithm.add_option_contract(hedge).symbol

            # Each insight will be ordered by the position group of the underlying and the hedging option positions.
            contract_multiplier = algorithm.securities[underlying].symbol_properties.contract_multiplier
            quantity = fund_per_insight // (contract_multiplier * (hedge.bid_price + hedge.underlying_last_price))
            targets.append(PortfolioTarget(underlying, int(quantity * contract_multiplier)))
            targets.append(PortfolioTarget(hedge_symbol, int(quantity)))

        return targets
```

Portfolio Construction

Supported Models

Introduction

This page describes the pre-built Portfolio Construction models in LEAN. The number of models grows over time. To add a model to LEAN, make a pull request to the GitHub repository . If none of these models perform exactly how you want, create a custom Portfolio Construction model .

Null Model

The `NullPortfolioConstructionModel` is the default Portfolio Construction model. It doesn't return any `PortfolioTarget` objects. It's useful if you need to analyze an Alpha model in isolation.

```
PY
# Set portfolio construction to NullPortfolioConstructionModel to bypass custom logic, useful for simplifying testing or initial
development when you want to manually manage or skip automatic portfolio adjustments.
self.set_portfolio_construction(NullPortfolioConstructionModel())
```

For more information about this model, see the class reference and implementation .

Equal Weighting Model

The `EqualWeightingPortfolioConstructionModel` assigns an equal share of the portfolio to the securities with active insights. This weighting scheme is useful for universe rotation based on simple portfolio strategies.

```
PY
# Set portfolio construction to EqualWeightingPortfolioConstructionModel to allocate capital equally across selected securities, ensuring
balanced exposure and reducing the influence of any single asset on the portfolio.
self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`rebalance`	`Resolution`	Rebalancing frequency	`Resolution.DAILY`

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Confidence Weighted Model

The `ConfidenceWeightedPortfolioConstructionModel` generates target portfolio weights based on the `Insight. confidence` for the last Insight of each Symbol. If the Insight has a direction of `InsightDirection. UP` , the model generates long targets. If the Insight has a direction of `InsightDirection. DOWN` , the model generates short targets. If the sum of all the last active Insight per Symbol is greater than 1, the model factors down each target percent holdings proportionally so the sum is 1. The model ignores `Insight` objects that have no `Confidence` value.

```
PY
# Use ConfidenceWeightedPortfolioConstructionModel to assign portfolio weights based on the confidence level of the latest Insights,
creating long or short positions accordingly, and normalizing weights if the total exceeds 1, leveraging the strength of predictions for
better allocation and risk management.
self.set_portfolio_construction(ConfidenceWeightedPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`rebalance`	`Resolution`	Rebalancing frequency	`Resolution.DAILY`
`portfolio_bias`	`PortfolioBias`	The bias of the portfolio	`PortfolioBias. LONG_SHORT`

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Insight Weighting Model

The `InsightWeightingPortfolioConstructionModel` generates target portfolio weights based on the `Insight. weight` for the last Insight of each Symbol. If the Insight has a direction of `InsightDirection. UP` , the model generates long targets. If the Insight has a direction of `InsightDirection. DOWN` , the model generates short targets. If the sum of all the last active Insight per Symbol is greater than 1, the model factors down each target percent holdings proportionally so the sum is 1. The model takes the absolute value of the `Weight` of each `Insight` object and ignores `Insight` objects that have no `Weight` value.

```
# Use InsightWeightingPortfolioConstructionModel to allocate capital based on the latest Insight weights, generating long or short targets
as per the Insight direction and adjusting allocations if total weights exceed 1, ensuring portfolio weights reflect current market
insights and maintains proper risk management.
self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`rebalance`	`Resolution`	Rebalancing frequency	`Resolution.DAILY`
`portfolio_bias`	`PortfolioBias`	The bias of the portfolio	`PortfolioBias. LONG_SHORT`

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Sector Weighting Model

The `SectorWeightingPortfolioConstructionModel` generates target portfolio weights based on the `company_reference.industry_template_code` provided by the US Fundamental dataset . The target percent holdings of each sector is 1/S where S is the number of sectors and the target percent holdings of each security is 1/N where N is the number of securities of each sector. If the insight has a direction of `InsightDirection. UP` , the model generates long targets. If the insight has a direction of `InsightDirection. DOWN` , the model generates short targets. The model ignores `Insight` objects for Symbols that have no `company_reference.industry_template_code` .

```
# Use SectorWeightingPortfolioConstructionModel to ensure diversified sector exposure and balanced allocation, aligning portfolio weights
with sector insights while managing long and short positions based on insight direction.
self.set_portfolio_construction(SectorWeightingPortfolioConstructionModel())
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
rebalance	Resolution	Rebalancing frequency	Resolution.DAILY

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Accumulative Insight Model

The AccumulativeInsightPortfolioConstructionModel generates target portfolio weights based on all the active insights of a security. For each active Insight of direction InsightDirection. UP , it increases the position size by a fixed percent. For each active Insight of direction InsightDirection. DOWN , it decreases the position size by a fixed percent. For each active Insight of direction InsightDirection.Flat , it moves the position size towards 0 by a fixed percent.

```
# Use AccumulativeInsightPortfolioConstructionModel to adjust portfolio weights based on the cumulative value of all active insights, increasing position size for insights with UP direction to capitalize on expected upward movements, while reducing position size with down or flat insights to mitigate potential losses.
self.set_portfolio_construction(AccumulativeInsightPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
rebalance	Any of the following types: • timedelta • Resolution • DateRules • None Callable[[datetime], datetime]	Rebalancing parameter. If it's a timedelta , DateRules or Resolution , it's converted into a function. If it's None , it's ignored. The function returns the next expected rebalance time for a given algorithm UTC DateTime. The function returns None if unknown, in which case the function will be called again in the next loop. If the function returns the current time, the portfolio rebalances.	None
portfolio_bias	PortfolioBias	The bias of the portfolio	PortfolioBias. LONG_SHORT
percent	float	The percentage amount of the portfolio value to allocate to a single insight	0.03 (3%)

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Mean Variance Optimization Model

The MeanVarianceOptimizationPortfolioConstructionModel seeks to build a portfolio with the least volatility possible and achieve a target return.

```
# Use MeanVarianceOptimizationPortfolioConstructionModel to determine portfolio weights by optimizing for the best risk-return trade-off balancing the expected returns against the risk of each security to construct a portfolio aiming to maximize returns for a given level of risk or minimize risk for a given level of returns.
self.set_portfolio_construction(MeanVarianceOptimizationPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`rebalance`	`Resolution`	Rebalancing frequency	`Resolution.DAILY`
`portfolio_bias`	`PortfolioBias`	The bias of the portfolio	`PortfolioBias. LONG_SHORT`
`lookback`	`int`	Historical return lookback period	1
`period`	`int`	The time interval of history price to calculate the weight	63
`resolution`	`Resolution`	The resolution of the history price	`Resolution.DAILY`
`target_return`	`float`	The target portfolio return	0.02 (2%)
`optimizer`	`IPortfolioOptimizer`	The portfolio optimization algorithm	`None`

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency . If you don't provide an `optimizer` argument, the default one is the MinimumVariancePortfolioOptimizer with upper and lower weights that respect the `portfolioBias` .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Black Litterman Optimization Model

The `BlackLittermanOptimizationPortfolioConstructionModel` receives `Insight` objects from multiple Alphas and combines them into a single portfolio. These multiple sources of Alpha models can be seen as the "investor views" required in the classical model.

```
# Use BlackLittermanOptimizationPortfolioConstructionModel to incorporate market equilibrium portfolio and the investor views to create
an improved portfolio aligning with both market expectations and investor insights.
self.set_portfolio_construction(BlackLittermanOptimizationPortfolioConstructionModel())
```

PY

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
rebalance	Resolution	Rebalancing frequency	Resolution.DAILY
portfolio_bias	PortfolioBias	The bias of the portfolio	PortfolioBias. LONG_SHORT
lookback	int	Historical return lookback period	1
period	int	The time interval of history price to calculate the weight	63
resolution	Resolution	The resolution of the history price	Resolution.DAILY
risk_free_rate	float	The risk free rate	0.0
delta	float	The risk aversion coefficient of the market portfolio	2.5
tau	float	The model parameter indicating the uncertainty of the CAPM prior	0.05
optimizer	IPortfolioOptimizer	The portfolio optimization algorithm	None

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency . If you don't provide an `optimizer` argument, the default one is the MinimumVariancePortfolioOptimizer with upper and lower weights that respect the `portfolioBias` .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Mean Reversion Model

The `MeanReversionPortfolioConstructionModel` implements an on-line portfolio selection technique, named "On-Line Moving Average Reversion" (OLMAR). The basic idea is to represent multi-period mean reversion as "Moving Average Reversion" (MAR), which explicitly predicts next price relatives using moving averages and then forms portfolios with online learning techniques.

```
# Use MeanReversionPortfolioConstructionModel to build a portfolio that capitalizes on the expected mean-reversion of asset prices,
adjusting allocations based on deviations from historical averages to potentially enhance returns by exploiting price reversions.
self.set_portfolio_construction(MeanReversionPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
rebalance	Resolution	Rebalancing frequency	Resolution.DAILY
portfolio_bias	PortfolioBias	The bias of the portfolio	PortfolioBias. LONG_SHORT
reversion_threshold	float	Reversion threshold	1
window_size	int	The window size of mean price	20
resolution	Resolution	The resolution of the history price	Resolution.DAILY

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is

removed from the universe .

For more information about this model, see the class reference and implementation .

Reference:

- Li, B., Hoi, S. C. (2012). On-line portfolio selection with moving average reversion. arXiv preprint arXiv:1206.4626 .

Risk Parity Model

The `RiskParityPortfolioConstructionModel` seeks to build a portfolio with the equal contribution of risk to the total portfolio risk from all assets.

```
# Use RiskParityPortfolioConstructionModel to allocate portfolio weights based on risk contributions, ensuring that each asset
contributes equally to the portfolio's overall risk, aiming for balanced risk exposure and potentially improved risk-adjusted returns.
self.set_portfolio_construction(RiskParityPortfolioConstructionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`rebalance`	`Resolution`	Rebalancing frequency	`Resolution.DAILY`
`portfolio_bias`	`PortfolioBias`	The bias of the portfolio	`PortfolioBias. LONG_SHORT`
`lookback`	`int`	Historical return lookback period	1
`period`	`int`	The time interval of history price to calculate the weight	252
`resolution`	`Resolution`	The resolution of the history price	`Resolution.DAILY`
`optimizer`	`IPortfolioOptimizer`	The portfolio optimization algorithm	`None`

This model supports other data types for the rebalancing frequency argument. For more information about the supported types, see Rebalance Frequency . If you don't provide an `optimizer` argument, the default one is the RiskParityPortfolioOptimizer .

This model removes expired insights from the Insight Manager during each rebalance. It also removes all insights for a security when the security is removed from the universe .

For more information about this model, see the class reference and implementation .

Portfolio Construction

Supported Optimizers

Introduction

This page describes the pre-built Portfolio Optimizer models in LEAN. The number of models grows over time. To add a model to LEAN, make a pull request to the GitHub repository . If none of these models perform exactly how you want, create a custom Portfolio Optimizer model .

Maximum Sharpe Ratio Optimizer

The `MaximumSharpeRatioPortfolioOptimizer` seeks to maximize the portfolio Sharpe Ratio .

```
optimizer = MaximumSharpeRatioPortfolioOptimizer()
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`minimum_weight`	`float`	The lower bounds on portfolio weights	-1
`maximum_weight`	`float`	The upper bounds on portfolio weights	1
`risk_free_rate`	`float`	The risk free rate	0

For more information about this model, see the class reference and implementation .

Minimum Variance Optimizer

The `MinimumVariancePortfolioOptimizer` seeks to minimize the portfolio variance and achieve a target return.

```
optimizer = MinimumVariancePortfolioOptimizer()
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`minimum_weight`	`float`	The lower bounds on portfolio weights	-1
`maximum_weight`	`float`	The upper bounds on portfolio weights	1
`target_return`	`float`	The target portfolio return	0.02 (2%)

For more information about this model, see the class reference and implementation .

Unconstrained Mean Variance Optimizer

The `UnconstrainedMeanVariancePortfolioOptimizer` seeks to find the optimal risk-adjusted portfolio that lies on the efficient frontier.

```
optimizer = UnconstrainedMeanVariancePortfolioOptimizer()
```

For more information about this model, see the class reference and implementation .

Risk Parity Optimizer

The `RiskParityPortfolioOptimizer` seeks to equalize the individual risk contribution to the total portfolio risk from each asset.

```
optimizer = RiskParityPortfolioOptimizer()
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`minimum_weight`	`float`	The lower bounds on portfolio weights	1e-05
`maximum_weight`	`float`	The upper bounds on portfolio weights	`sys.float_info.max`

For more information about this model, see the class reference and implementation .

Algorithm Framework

Risk Management

Risk Management

Key Concepts

Introduction

The Risk Management model seeks to manage risk on the `PortfolioTarget` collection it receives from the Portfolio Construction model before the targets reach the Execution model. There are many creative ways to manage risk. Some examples of risk management include the following:

- *"Trailing Stop Risk Management Model"*
  Create and manage trailing stop-loss orders for open positions.
- *"Option Hedging Risk Management Model"*
  Purchase options to hedge large equity exposures.
- *"Sector Exposure Risk Management Model"*
  Reduce position sizes when overexposed to sectors or individual assets, keeping the portfolio within diversification requirements.
- *"Flash Crash Detection Risk Management Model"*
  Scan for strange market situations that might be precursors to a flash crash and attempt to protect the portfolio when they are detected.

Add Models

To set a Risk Management model, in the `initialize` method, call the `add_risk_management` method.

```
# Add the null risk management model, which doesn't affect the portfolio targets.
self.add_risk_management(NullRiskManagementModel())
```

To view all the pre-built Risk Management models, see Supported Models .

Multi-Model Algorithms

To add multiple Risk Management models, in the `initialize` method, call the `AddRiskManagement` method multiple times.

```
# Add multiple Risk Management models to sequentially adjust portfolio targets, with each model refining the targets passed from the
previous model, managing risk across individual securities and sectors.
self.add_risk_management(MaximumDrawdownPercentPerSecurity())
self.add_risk_management(MaximumSectorExposureRiskManagementModel())
```

If you add multiple Risk Management models, the original collection of `PortfolioTarget` objects from the Portfolio Construction model is passed to the first Risk Management model. The risk-adjusted targets from the first Risk Management model are passed to the second Risk Management model. The process continues sequentially until all of the Risk Management models have had an opportunity to adjust the targets.

Model Structure

Risk Management models should extend the `RiskManagementModel` class. Extensions of the `RiskManagementModel` class must implement the `manage_risk` method, which receives an array of `PortfolioTarget` objects from the Portfolio Construction model at every time step and should return an array of risk-adjusted `PortfolioTarget` objects. The method should only return the adjusted targets, not all of targets. If the method creates a `PortfolioTarget` object to liquidate a security, cancel the security's insights to avoid re-entering the position.

```
# Extend the RiskManagementModel class by implementing the manage_risk method, which receives PortfolioTarget objects from the Portfolio
Construction model and returns only the risk-adjusted targets.
class MyRiskManagementModel(RiskManagementModel):
    # Adjust the portfolio targets and return them. If no changes emit nothing.
    def manage_risk(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> List[PortfolioTarget]:
        return []

    # Optional: Be notified when securities change
    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # Security additions and removals are pushed here.
        # This can be used for setting up algorithm state.
        # changes.added_securities
        # changes.removed_securities
        pass
```

The `algorithm` argument that the methods receive is an instance of the base `QCAlgorithm` class, not your subclass of it.

To view a full example of a `RiskManagementModel` subclass, see the MaximumDrawdownPercentPerSecurity in the LEAN GitHub repository.

Track Security Changes

The Universe Selection model may select a dynamic universe of assets, so you should not assume a fixed set of assets in the Risk Management model. When the Universe Selection model adds and removes assets from the universe, it triggers an `on_securities_changed` event. In the `on_securities_changed` event handler, you can initialize the security-specific state or load any history required for your Risk Management model. If you need to save data for individual securities, add custom members to the respective `Security` object .

```
class MyRiskManagementModel(RiskManagementModel):
    _securities = []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        super().on_securities_changed(algorithm, changes)
        for security in changes.added_securities:
            # Store and manage Symbol-specific data
            security.indicator = algorithm.sma(security.symbol, 20)
            algorithm.warm_up_indicator(security.symbol, security.indicator)

            self._securities.append(security)

        for security in changes.removed_securities:
            if security in self.securities:
                algorithm.deregister_indicator(security.indicator)
                self._securities.remove(security)
```

Portfolio Target Collection

The `PortfolioTargetCollection` class is a helper class to manage `PortfolioTarget` objects. The class manages an internal dictionary that has the security `Symbol` as the key and a `PortfolioTarget` as the value.

Add Portfolio Targets

To add a `PortfolioTarget` to the `PortfolioTargetCollection` , call the `add` method.

```
self.targets_collection.add(portfolio_target)
```

To add a list of `PortfolioTarget` objects, call the `add_range` method.

```
self.targets_collection.add_range(portfolio_targets)
```

Check Membership

To check if a `PortfolioTarget` exists in the `PortfolioTargetCollection` , call the `contains` method.

```
target_in_collection = self.targets_collection.contains(portfolio_target)
```

To check if a Symbol exists in the `PortfolioTargetCollection` , call the `contains_key` method.

```
symbol_in_collection = self.targets_collection.contains_key(symbol)
```

To get all the Symbol objects, use the `keys` property.

```
symbols = self.targets_collection.keys
```

Access Portfolio Targets

To access the `PortfolioTarget` objects for a Symbol, index the `PortfolioTargetCollection` with the Symbol.

```
portfolio_target = self.targets_collection[symbol]
```

To iterate through the `PortfolioTargetCollection` , call the `get_enumerator` method.

```
enumerator = self.targets_collection.get_enumerator()
```

To get all the `PortfolioTarget` objects, use the `values` property

```
portfolio_targets = self.targets_collection.values
```

Order Portfolio Targets by Margin Impact

To get an enumerable where position reducing orders are executed first and the remaining orders are executed in decreasing order value, call the `order_by_margin_impact` method.

```
for target in self.targets_collection.order_by_margin_impact(algorithm):
    # Place order
```

This method won't return targets for securities that have no data yet. This method also won't return targets for which the sum of the current holdings and open orders quantity equals the target quantity.

Remove Portfolio Targets

To remove a `PortfolioTarget` from the `PortfolioTargetCollection` , call the `remove` method.

```
remove_successful = self.targets_collection.remove(symbol)
```

To remove all the `PortfolioTarget` objects, call the `clear` method.

```
self.targets_collection.clear()
```

To remove all the `PortfolioTarget` objects that have been fulfilled, call the `clear_fulfilled` method.

```
self.targets_collection.clear_fulfilled(algorithm)
```

Universe Timing Considerations

If the Risk Management model manages some indicators or consolidators for securities in the universe and the universe selection runs during the indicator sampling period or the consolidator aggregation period, the indicators and consolidators might be missing some data. For example, take the following scenario:

- The security resolution is minute
- You have a consolidator that aggregates the security data into daily bars to update the indicator
- The universe selection runs at noon

In this scenario, you create and warm-up the indicator at noon. Since it runs at noon, the history request that gathers daily data to warm up the indicator won't contain any data from the current day and the consolidator that updates the indicator also won't aggregate any data from before noon. This process doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Examples

The following examples demonstrate some common practices for implementing a risk management model.

Example 1: Take Profit Stop Loss

The following algorithm trades 20-60 EMA crosses on the top 500 liquid US stocks in equal weighting. To minimize risk and capture early profit, we can implement both `MaximumUnrealizedProfitPercentPerSecurity` to take profit and `TrailingStopRiskManagementModel` to stop loss by trailing high price.

```
class FrameworkRiskManagementAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(1000000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe of the most liquid stocks since their trend is more capital-supported.
        self.add_universe_selection(QC500UniverseSelectionModel())
        # Emit insights on EMA cross, indicating the trend changes. We use short-term versus medium-term for more trade opportunities.
        self.add_alpha(EmaCrossAlphaModel(20, 60, Resolution.DAILY))
        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Take profit at 10%.
        self.add_risk_management(MaximumUnrealizedProfitPercentPerSecurity(0.1))
        # Trailing stop loss at 5%.
        self.add_risk_management(TrailingStopRiskManagementModel(0.05))
```

Example 2: Tail Value At Risk

The following algorithm implements a custom risk management model that liquidates the position if an asset has PnL lower than the tail value-at-risk (TVaR) throughout the insight to avoid large catastrophic losses. To calculate the TVaR, we create a log return indicator and use the following formula for the calculation: $\textrm{TVaR}_{\alpha} = \mu + \sigma \times \phi[\Phi^{-1}(\alpha)] / (1 - \alpha)$

```
from scipy.stats import norm

class FrameworkRiskManagementAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 9, 15)
        self.set_cash(1000000)

        # Add a universe of the most liquid stocks since their trend is more capital-supported.
        self.add_universe_selection(QC500UniverseSelectionModel())
        # Emit insights all for selected stocks, rebalancing every 2 weeks.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(14)))
        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Liquidate on extreme catastrophic events.
        self.add_risk_management(TailValueAtRiskRiskManagementModel(0.05, 14))

class TailValueAtRiskRiskManagementModel(RiskManagementModel):
    _log_ret_by_symbol = {}

    def __init__(self, alpha: float = 0.05, num_days: float = 7) -> None:
        # The alpha level on TVaR calculation.
        self.alpha = alpha
        # The number of days that the insight signal lasts for.
        self.days = num_days

    # Adjust the portfolio targets and return them. If there are no changes, emit nothing.
```

```
    def manage_risk(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> List[PortfolioTarget]:
        targets = []
        for kvp in algorithm.securities:
            security = kvp.value
            if not security.invested:
                continue

            pnl = security.holdings.unrealized_profit_percent
            symbol = security.symbol
            # If the %PnL is worse than the preset level TVaR, we liquidate it.
            if pnl < self.get_tvar(symbol):
                # Cancel insights to avoid reordering afterward.
                algorithm.insights.cancel([symbol])
                # Liquidate.
                targets.append(PortfolioTarget(symbol, 0, tag="Liquidate due to TVaR"))

        return targets

    def get_tvar(self, symbol: Symbol) -> float:
        symbol_data = self._log_ret_by_symbol.get(symbol)
        if not symbol_data:
            return 0

        # TVaR = \mu + \sigma * \phi[\Phi^{-1}(p)] / (1 - p)
        # Scale up to the days of the signal. By stochastic calculus, we multiply by sqrt(days).
        daily_tvar = symbol_data.mean_log_ret + symbol_data.sd_log_ret * np.sqrt(self.days) * norm.pdf(norm.ppf(self.alpha)) / (1 -
self.alpha)
        # We want the left side of the symmetric distribution.
        return -daily_tvar

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            # Add SymbolData class to handle log returns.
            self._log_ret_by_symbol[added.symbol] = SymbolData(algorithm, added.symbol)

        for removed in changes.removed_securities:
            # Stop subscription on the data to release computational resources.
            symbol_data = self._log_ret_by_symbol.pop(removed.symbol, None)
            if symbol_data:
                symbol_data.dispose()

class SymbolData:
    def __init__(self, algorithm: QCAlgorithm, symbol: Symbol) -> None:
        self.algorithm = algorithm
        self.symbol = symbol

        # Since the return is assumed log-normal, we use the log return indicator to calculate TVaR later.
        self.log_ret = LogReturn(1)
        # Register the indicator for automatic updating for daily log returns.
        algorithm.register_indicator(symbol, self.log_ret, Resolution.DAILY)
        # Set up a rolling window to save the log return for calculating the mean and SD for TVaR calculation.
        self.window = RollingWindow(252)
        # Add a handler to save the log return to the rolling window.
        self.log_ret.updated += lambda _, point: self.window.add(point.value)
        # Warm up the rolling window.
        history = algorithm.history[TradeBar](symbol, 253, Resolution.DAILY)
        for bar in history:
            self.log_ret.update(bar.end_time, bar.close)

    @property
    def is_ready(self) -> bool:
        return self.window.is_ready

    @property
    def mean_log_ret(self) -> float:
        # Mean log return for TVaR calculation.
        return np.mean(list(self.window))

    @property
    def sd_log_ret(self) -> float:
        # SD of log return for TVaR calculation.
        return np.std(list(self.window), ddof=1)

    def dispose(self) -> None:
        # Stop subscription on the data to release computational resources.
        self.algorithm.deregister_indicator(self.log_ret)
```

Example 3: Bracket Risk Model

The following algorithm implements a custom bracket risk management model that combines a trailing stop-loss with a take-profit target. Positions are exited when unrealized profit exceeds a threshold or when the price falls by more than a specified percentage from the trailing high. It is used alongside an XGBoost Alpha model that trades Equities near their earnings dates.

```
PY

from sklearn.model_selection import RandomizedSearchCV, train_test_split
from sklearn.preprocessing import MinMaxScaler
import xgboost as xgb
```

```
class ModulatedNadionReplicator(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)
        self.settings.free_portfolio_value_percentage = 0.05
        self.settings.seed_initial_prices = True
        self._spy = self.add_equity("SPY", Resolution.HOUR)

        # Create universe parameters.
        self.universe_settings.resolution = Resolution.HOUR
        self.universe_settings.asynchronous = True
        date_rule = self.date_rules.month_start(self._spy)
        self.universe_settings.schedule.on(date_rule)
        self.add_universe_selection(EarningsVolumeUniverseSelectionModel(10))

        # Add the other framework models.
        self.add_alpha(XGBoostAlphaModel(self, date_rule, self._spy))
        self.set_portfolio_construction(InsightWeightingPortfolioConstructionModel())
        self.set_risk_management(BracketRiskModel(0.05, 0.15))

        # Add a warm up so the algorithm has insights on deployment day.
        self.set_warm_up(timedelta(45))


class EarningsVolumeUniverseSelectionModel(FundamentalUniverseSelectionModel):
    # Selects symbols by liquidity and nearest earnings report date.
    def __init__(self, universe_size=10):
        self._universe_size = universe_size
        super().__init__(self._select)

    def _select(self, fundamental):
        # Sort the top 30 by price and dollar volume.
        liquid = sorted(
            [f for f in fundamental if f.has_fundamental_data and 20 <= f.price <= 200 and f.dollar_volume > 5_000_000],
            key=lambda f: f.dollar_volume
        )[-30:]

        # Select symbols with nearest earnings report dates.
        selected = sorted(
            [f for f in liquid if f.earning_reports.file_date],
            key=lambda f: str(f.earning_reports.file_date)
        )[:self._universe_size]

        return [f.symbol for f in selected]


class XGBoostAlphaModel(AlphaModel):

    def __init__(self, algorithm, date_rule, spy):
        self._algorithm = algorithm
        self._universe = []
        self._insights = []
        self._feature_window = 24
        self._history_bars = 48
        self._rsi_period = 12
        self._scaler = MinMaxScaler(feature_range=(-1, 1))
        self._spy = spy

        # Create rate-of-change indicator for SPY with specified window.
        self._spy.rocp = algorithm.rocp(spy, self._feature_window)
        self._spy.rocp.window.size = self._feature_window
        algorithm.indicator_history(self._spy.rocp, spy, self._spy.rocp.period + self._spy.rocp.window.size)

        algorithm.train(date_rule, algorithm.time_rules.at(8, 0), self._train_models)
        algorithm.schedule.on(date_rule, algorithm.time_rules.after_market_open(self._spy, 30), self._create_insights)

    def _create_insights(self):
        self._insights.clear()

        # Extract predictions from all trained models.
        for security in self._universe:
            # Skip prediction if model not yet trained.
            if not (security.model and security.rocp.window.is_ready and self._spy.rocp.window.is_ready):
                continue
            features, _ = self._build_features(security)
            # Get prediction from the trained model.
            magnitude = security.model.predict(features)[-1]
            # Determine signal direction based on prediction magnitude.
            direction = InsightDirection.FLAT
            if magnitude > 0.05:
                direction = InsightDirection.UP
            elif magnitude < -0.05:
                direction = InsightDirection.DOWN
            # Generate price insights with computed weights and directions.
            self._insights.append(Insight.price(security, timedelta(1), direction, weight=0.3 * abs(magnitude)))

    def update(self, algorithm, data):
        if algorithm.is_warming_up:
            return []

        insights = self._insights.copy()
        self._insights.clear()
```

```
        return insights

    def on_securities_changed(self, algorithm, changes):
        for security in changes.added_securities:
            if security == self._spy or security in self._universe:
                continue
            # Add security and register indicators to universe.
            self._universe.append(security)
            security.model = None
            security.rocp = algorithm.rocp(security, self._feature_window)
            security.rsi = algorithm.rsi(security, self._rsi_period, MovingAverageType.SIMPLE)
            security.atr = algorithm.atr(security, self._feature_window, MovingAverageType.SIMPLE)
            # Configure window sizes and populate history for indicator and window.
            bars = algorithm.history[TradeBar](security, self._feature_window * 2)
            for indicator in [security.rocp, security.rsi, security.atr]:
                indicator.window.size = self._feature_window
                for bar in bars:
                    indicator.update(bar)

        for security in changes.removed_securities:
            if security not in self._universe:
                continue
            # Remove security and deregister indicators from universe.
            self._universe.remove(security)
            for indicator in [security.rocp, security.rsi, security.atr]:
                algorithm.deregister_indicator(indicator)

    def _build_features(self, security):
        # Compute momentum and volatility indicators and normalize all features to consistent range.
        scaled = self._scaler.fit_transform(np.hstack(
            [self._get_indicator_history(self._spy.rocp)]
            + [self._get_indicator_history(indicator) for indicator in [security.rocp, security.rsi, security.atr]]
        ))

        return scaled[:, :3], scaled[:, [3]]

    def _get_indicator_history(self, indicator):
        return np.array([x.value for x in indicator.window])[::-1].reshape(-1, 1)

    def _train_models(self):
        # Iterate through all security in the universe for model training.
        for security in self._universe:
            if not (security.rocp.window.is_ready and self._spy.rocp.window.is_ready):
                continue

            features, scaled_rocp = self._build_features(security)
            target = scaled_rocp.ravel()

            # Prepare training and validation splits from the feature matrix.
            x_train, x_valid, y_train, y_valid = train_test_split(
                features, target, test_size=0.35, random_state=42,
            )

            # Define hyperparameter grid for randomized search optimization.
            parameters = {
                'n_estimators': [100, 200, 300, 400],
                'learning_rate': [0.001, 0.005, 0.01, 0.05],
                'max_depth': [8, 10, 12, 15],
                'gamma': [0.001, 0.005, 0.01, 0.02],
                'random_state': [42]
            }
            eval_set = [(x_train, y_train), (x_valid, y_valid)]
            base_model = xgb.XGBRegressor(objective="reg:squarederror", verbosity=0)
            model = RandomizedSearchCV(
                estimator=base_model, param_distributions=parameters, n_iter=5, scoring="neg_mean_squared_error", cv=4, verbose=0,
            )

            # Execute randomized search for optimal model hyperparameters.
            model.fit(x_train, y_train, eval_set=eval_set, verbose=False)
            security.model = model


class BracketRiskModel(RiskManagementModel):

    def __init__(self, drawdown_pct, profit_pct):
        # Store drawdown and profit thresholds as percentage values.
        self._drawdown_pct = -abs(drawdown_pct)
        self._profit_pct = abs(profit_pct)

    def manage_risk(self, algorithm, targets):
        # Build list of risk-adjusted targets.
        adjusted_targets = []

        # Iterate through all securities to evaluate positions.
        for symbol, security in algorithm.securities.items():
            # Reset trailing high for non-invested securities.
            if not security.invested:
                security.trailing_high = None
            # Take profit when unrealized gains exceed the threshold.
            elif security.holdings.unrealized_profit_percent > self._profit_pct:
                adjusted_targets.append(PortfolioTarget(symbol, 0))
                algorithm.insights.cancel([symbol])
            # Initialize trailing high from the entry price.
```

```
            elif security.trailing_high is None:
                security.trailing_high = security.holdings.average_price
            # Update trailing high if a new high is reached.
            elif security.trailing_high < security.high:
                security.trailing_high = security.high
            # Exit position when drawdown from trailing high exceeds limit.
            elif (security.low / security.trailing_high) - 1 < self._drawdown_pct:
                adjusted_targets.append(PortfolioTarget(symbol, 0))
                algorithm.insights.cancel([symbol])

        return adjusted_targets
```

Risk Management

Supported Models

Introduction

This page describes the pre-built Risk Management models in LEAN. The number of models grows over time. To add a model to LEAN, make a pull request to the GitHub repository . If none of these models perform exactly how you want, create a custom Risk Management model .

Null Model

The `NullRiskManagementModel` is the default Risk Management model. It doesn't adjust any of the `PortfolioTarget` objects it receives from the Portfolio Construction model.

```
# Adding the null risk management model doesn't affect the portfolio targets.
self.add_risk_management(NullRiskManagementModel())
```

For more information about this model, see the class reference and implementation .

Maximum Security Drawdown Model

The `MaximumDrawdownPercentPerSecurity` model monitors the unrealized profit percentage of each security in the portfolio. When the percentage drops below a threshold relative to the opening price, it liquidates the position and cancels all insights in the Insight Manager that are for the security. This model can operate even when the Portfolio Construction model provides an empty list of `PortfolioTarget` objects.

```
# Add risk management model to limit maximum drawdown per security, reducing position size to prevent further losses.
self.add_risk_management(MaximumDrawdownPercentPerSecurity())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maximum_drawdown_percent`	`float`	The maximum percentage drawdown allowed for any single security holding	0.05 (5%)

For more information about this model, see the class reference and implementation .

Maximum Portfolio Drawdown Model

The `MaximumDrawdownPercentPortfolio` model monitors the portfolio drawdown . The drawdown can be relative to the starting portfolio value or the maximum portfolio value. When the portfolio value drops below a percentage threshold, the model liquidates the portfolio and cancels all insights in the Insight Manager . To liquidate the portfolio, the model must receive at least 1 `PortfolioTarget` after the drawdown threshold is passed. After the portfolio is liquidated, the model resets. This model can operate even when the Portfolio Construction model provides an empty list of `PortfolioTarget` objects.

```
# Add risk management model to limit maximum drawdown for the entire portfolio, protecting overall portfolio value from significant
losses.
self.add_risk_management(MaximumDrawdownPercentPortfolio())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maximum_drawdown_percent`	`float`	Maximum spread accepted comparing to current price in percentage	0.05 (5%)
`is_trailing`	`bool`	If `False` , the drawdown is relative to the starting value of the portfolio. If `True` , the drawdown is relative the last maximum portfolio value	`False`

For more information about this model, see the class reference and implementation .

Maximum Unrealized Profit Model

The `MaximumUnrealizedProfitPercentPerSecurity` model monitors the unrealized profit of each security in the portfolio. When the unrealized profit exceeds a profit threshold, it liquidates the position and cancels all insights in the Insight Manager that are for the security. This model can operate even when the Portfolio Construction model provides an empty list of `PortfolioTarget` objects.

```py
# Add risk management model to secure gains by limiting the maximum unrealized profit for each security, locking in profits before
potential reversals.
self.add_risk_management(MaximumUnrealizedProfitPercentPerSecurity())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maximum_unrealized_profit_percent`	`float`	The maximum percentage unrealized profit allowed for any single security holding	0.05 (5%)

For more information about this model, see the class reference and implementation .

Maximum Sector Exposure Model

The `MaximumSectorExposureRiskManagementModel` limits the absolute portfolio exposure in a each industry sector to a predefined maximum percentage. If the absolute portfolio exposure exceeds the maximum percentage, the weight of each Equity in the sector is scaled down so the sector doesn't exceed the maximum percentage. This process requires assets that are selected by Morningstar fundamental data . This model can operate even when the Portfolio Construction model provides an empty list of `PortfolioTarget` objects.

```py
# Add risk management model to limit exposure to any single sector, ensuring diversification and reducing sector-specific risk.
self.add_risk_management(MaximumSectorExposureRiskManagementModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maximum_sector_exposure`	`float`	The maximum exposure for any sector	0.2 (20%)

For more information about this model, see the class reference and implementation .

Trailing Stop Model

The `TrailingStopRiskManagementModel` monitors the drawdown of each security in the portfolio. When the peak-to-trough drawdown of the unrealized profit exceeds a threshold, it liquidates the position and cancels all insights in the Insight Manager that are for the security. This model can operate even when the Portfolio Construction model provides an empty list of `PortfolioTarget` objects.

```
# Add risk management model to implement trailing stops, which adjusts stop-loss levels as the price moves favorably, helping to lock in
gains and limit losses.
self.add_risk_management(TrailingStopRiskManagementModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`maximum_drawdown_percent`	`float`	The maximum percentage drawdown allowed for algorithm portfolio compared with the highest unrealized profit	0.05 (5%)

For more information about this model, see the class reference and implementation .

Algorithm Framework

Execution

Execution

Key Concepts

Introduction

The Execution model receives an array of risk-adjusted `PortfolioTarget` objects from the Risk Management model and places trades in the market to satisfy the targets. The Execution model only receives updates to the portfolio target share counts. It doesn't necessarily receive all of the targets at once.

Set Models

To set an Execution model, in the `initialize` method, call the `set_execution` method.

```
# Satisify portfolio targets by immediately sending market orders.
self.set_execution(ImmediateExecutionModel())
```

To view all the pre-built Execution models, see Supported Models .

Model Structure

Execution models should extend the `ExecutionModel` class. Extensions of the `ExecutionModel` must implement the `execute` method, which receives an array of `PortfolioTarget` objects at every time step and is responsible for reaching the target portfolio as efficiently as possible. The Portfolio Construction model creates the `PortfolioTarget` objects, the Risk Management model may adjust them, and then the Execution model places the orders to fulfill them.

```
# Execution Model scaffolding structure example
class MyExecutionModel(ExecutionModel):

    # Fill the supplied portfolio targets efficiently
    def execute(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> None:
        pass

    # Optional: Securities changes event for handling new securities.
    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        # Security additions and removals are pushed here.
        # This can be used for setting up algorithm state.
        # changes.added_securities
        # changes.removed_securities
        pass

    # Optional: Order event for handling new order events.
    def on_order_event(self, algorithm: QCAlgorithm, order_event: OrderEvent) -> None:
        # Order events are pushed here.
        # This can be used for tracking order status or handling invalid and cancelled orders.
        pass
```

The `algorithm` argument that the methods receive is an instance of the base `QCAlgorithm` class, not your subclass of it.

The following table describes the properties of the `PortfolioTarget` class that you may access in the Execution model:

Property	Data Type	Description
`symbol`	`Symbol`	Asset to trade
`quantity`	`float`	Number of units to hold

To view a full example of an `ExecutionModel` subclass, see the ImmediateExecutionModel in the LEAN GitHub repository.

Track Security Changes

The Universe Selection model may select a dynamic universe of assets, so you should not assume a fixed set of assets in the Execution model. When the Universe Selection model adds and removes assets from the universe, it triggers an `on_securities_changed` event. In the `on_securities_changed` event handler, you can initialize the security-specific state or load any history required for your Execution model. If you need to save data for individual securities, add custom members to the respective `Security` object .

```
class MyExecutionModel(ExecutionModel):
    _securities = []

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for security in changes.added_securities:
            # Store and manage Symbol-specific data
            security.indicator = algorithm.sma(security.symbol, 20)
            algorithm.warm_up_indicator(security.symbol, security.indicator)

            self._securities.append(security)

        for security in changes.removed_securities:
            if security in self.securities:
                algorithm.deregister_indicator(security.indicator)
                self._securities.remove(security)
```

PY

Portfolio Target Collection

The `PortfolioTargetCollection` class is a helper class to manage `PortfolioTarget` objects. The class manages an internal dictionary that has the security `Symbol` as the key and a `PortfolioTarget` as the value.

Add Portfolio Targets

To add a `PortfolioTarget` to the `PortfolioTargetCollection` , call the `add` method.

```
self.targets_collection.add(portfolio_target)
```

PY

To add a list of `PortfolioTarget` objects, call the `add_range` method.

```
self.targets_collection.add_range(portfolio_targets)
```

PY

Check Membership

To check if a `PortfolioTarget` exists in the `PortfolioTargetCollection` , call the `contains` method.

```
target_in_collection = self.targets_collection.contains(portfolio_target)
```

PY

To check if a Symbol exists in the `PortfolioTargetCollection` , call the `contains_key` method.

```
symbol_in_collection = self.targets_collection.contains_key(symbol)
```

PY

To get all the Symbol objects, use the `keys` property.

```
symbols = self.targets_collection.keys
```

Access Portfolio Targets

To access the `PortfolioTarget` objects for a Symbol, index the `PortfolioTargetCollection` with the Symbol.

```
portfolio_target = self.targets_collection[symbol]
```

To iterate through the `PortfolioTargetCollection` , call the `get_enumerator` method.

```
enumerator = self.targets_collection.get_enumerator()
```

To get all the `PortfolioTarget` objects, use the `values` property

```
portfolio_targets = self.targets_collection.values
```

Order Portfolio Targets by Margin Impact

To get an enumerable where position reducing orders are executed first and the remaining orders are executed in decreasing order value, call the `order_by_margin_impact` method.

```
for target in self.targets_collection.order_by_margin_impact(algorithm):
    # Place order
```

This method won't return targets for securities that have no data yet. This method also won't return targets for which the sum of the current holdings and open orders quantity equals the target quantity.

Remove Portfolio Targets

To remove a `PortfolioTarget` from the `PortfolioTargetCollection` , call the `remove` method.

```
remove_successful = self.targets_collection.remove(symbol)
```

To remove all the `PortfolioTarget` objects, call the `clear` method.

```
self.targets_collection.clear()
```

To remove all the `PortfolioTarget` objects that have been fulfilled, call the `clear_fulfilled` method.

```
self.targets_collection.clear_fulfilled(algorithm)
```

Universe Timing Considerations

If the Execution model manages some indicators or consolidators for securities in the universe and the universe selection runs during the indicator sampling period or the consolidator aggregation period, the indicators and consolidators might be missing some data. For example, take the following scenario:

- The security resolution is minute
- You have a consolidator that aggregates the security data into daily bars to update the indicator
- The universe selection runs at noon

In this scenario, you create and warm-up the indicator at noon. Since it runs at noon, the history request that gathers daily data to warm up the indicator won't contain any data from the current day and the consolidator that updates the indicator also won't aggregate any data from before noon. This process doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Examples

The following examples demonstrate some common practices for implementing the execution model.

Example 1: Iceberg Execution

The following algorithm simulates an account with large capital that equally holds the most liquid stocks. To hide the footprint and avoid a large market impact that might erode the profit margin, we can set up an iceberg execution system, submitting only 10% of the volume of the bid/ask side order book to the order direction.

```
class FrameworkExecutionModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000000)

        # Add a universe of the most liquid stocks since their trend is more capital-supported.
        self.add_universe_selection(QC500UniverseSelectionModel())
        # Emit insights all for selected stocks.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Iceberg ordering to hide traces and avoid market impact.
        # Since quote data will be used, ensure the asset class and resolution are compatible.
        self.set_execution(IcebergExecutionModel(0.1))

class IcebergExecutionModel(ExecutionModel):
    def __init__(self, order_percent_volume: float = 0.1) -> None:
        self.targets_collection = PortfolioTargetCollection()
        # The maximum order size is taken from the order book in percentage.
        self.maximum_order_quantity_percent_volume = order_percent_volume

    def execute(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> None:
        # update the complete set of portfolio targets with the new targets
        self.targets_collection.add_range(targets)

        if not self.targets_collection.is_empty:
            for target in self.targets_collection.order_by_margin_impact(algorithm):
                symbol = target.symbol

                # Calculate the remaining quantity to be ordered
                unordered_quantity = OrderSizing.get_unordered_quantity(algorithm, target)
                # adjust order size to respect maximum order size based on a percentage of current volume
                order_size = self.get_order_size_for_percent_volume(algorithm.securities[symbol],
self.maximum_order_quantity_percent_volume, unordered_quantity)

                if order_size != 0:
                    algorithm.market_order(symbol, order_size)

            self.targets_collection.clear_fulfilled(algorithm)

    def get_order_size_for_percent_volume(self, security: Security, maximum_percent_current_volume: float, desired_order_size: float) ->
float:
        # Take N% from the order book according to the order direction.
        max_order_size = maximum_percent_current_volume * (security.bid_size if desired_order_size > 0 else security.ask_size)
        order_size = min(max_order_size, abs(desired_order_size))
        return np.sign(desired_order_size) * OrderSizing.adjust_by_lot_size(security, order_size)
```

Example 2: More Favorable Than Signal Time

Some algorithms are signaling based on daily close data. However, their orders often emit when the next market opens, making the filling price slip. To avoid this slippage negatively impacting your expectations, we can construct a custom execution model to place orders only when the price is more favorable than the last close price. Although we did not set a time limit in this example, you may consider doing so.

PY

```
class FrameworkExecutionModelAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 1)

        # Add a universe of the most liquid stocks since their trend is more capital-supported.
        self.add_universe_selection(QC500UniverseSelectionModel())
        # Emit insights all for selected stocks.
        self.add_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(7)))
        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Only place an order if the price is more favorable than the price at the insight signal.
        self.set_execution(FavorableExecutionModel())

class FavorableExecutionModel(ExecutionModel):
    def __init__(self) -> None:
        self.targets_collection = PortfolioTargetCollection()
        self.lag_price_by_symbol = {}

    def execute(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> None:
        self.targets_collection.add_range(targets)
        if not self.targets_collection.is_empty:
            for target in self.targets_collection.order_by_margin_impact(algorithm):
                symbol = target.symbol
                security = algorithm.securities[symbol]
                # Calculate the remaining quantity to be ordered
                quantity = OrderSizing.get_unordered_quantity(algorithm, target, security, True)

                if quantity != 0:
                    above_minimum_portfolio = BuyingPowerModelExtensions.above_minimum_order_margin_portfolio_percentage(
                        security.buying_power_model,
                        security,
                        quantity,
                        algorithm.portfolio,
                        algorithm.settings.minimum_order_margin_portfolio_percentage)
                    if above_minimum_portfolio:
                        # Cache the price at signal emission to compare if the current price is more favorable.
                        if not symbol in self.lag_price_by_symbol:
                            history = algorithm.history[TradeBar](symbol, 1, Resolution.DAILY)
                            self.lag_price_by_symbol[symbol] = list(history)[-1].close

                        # Only order if the price is more favorable than the price at signal emission.
                        if (security.price <= self.lag_price_by_symbol[symbol] and quantity > 0)\
                        or (security.price >= self.lag_price_by_symbol[symbol] and quantity < 0):
                            algorithm.market_order(security, quantity)
                    elif not PortfolioTarget.minimum_order_margin_percentage_warning_sent:
                        # will trigger the warning if it has not already been sent
                        PortfolioTarget.minimum_order_margin_percentage_warning_sent = False

            self.targets_collection.clear_fulfilled(algorithm)
```

Execution

Supported Models

Introduction

This page describes the pre-built Execution models in LEAN. The number of models grows over time. To add a model to LEAN, make a pull request to the GitHub repository . If none of these models perform exactly how you want, create a custom Execution model .

Null Model

The `NullExecutionModel` doesn't place any trades.

```
# Set the execution model to NullExecutionModel, which disables execution functionality for debugging or when the execution logic is not
needed for the current strategy.
self.set_execution(NullExecutionModel())
```

For more information about this model, see the class reference and implementation .

Immediate Model

The `ImmediateExecutionModel` is the default Execution model. It uses market orders to immediately fill portfolio targets. It's similar to placing market orders in line with your algorithm logic.

```
# Use ImmediateExecutionModel for executing trades instantly at market price to minimize execution delay. This may incur extra slippage.
self.set_execution(ImmediateExecutionModel())
```

For more information about this model, see the class reference and implementation .

Spread Model

The `SpreadExecutionModel` executes trades with market orders if the exchange is open and the bid-ask spread is within a threshold percentage. The model only works if the security subscription provides QuoteBar data. The spread percentage is calculated as

$$\frac{a - b}{p}$$

where a is the ask price, b is the bid price, and p is the price.

```
# Use SpreadExecutionModel to manage trade execution based on bid-ask spreads, aiming to optimize trade execution costs and reduce
slippage.
self.set_execution(SpreadExecutionModel())
```

The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`accepting_spread_percent`	`float`	Maximum spread accepted comparing to current price in percentage	0.005 (0.5%)

For more information about this model, see the class reference and implementation .

Standard Deviation Model

The `StandardDeviationExecutionModel` seeks to fill orders when the price is more than 2 standard deviations lower than the average price over a trailing period. The intent is to find dips in the market to place trades. In strongly trending markets, this procedure can result in delayed trade placement since it might be a while before the next price spike or dip.

```
# Use StandardDeviationExecutionModel to execute trades based on volatility measures, aiming to adapt trade sizes and timing according to
market volatility.
self.set_execution(StandardDeviationExecutionModel())
```

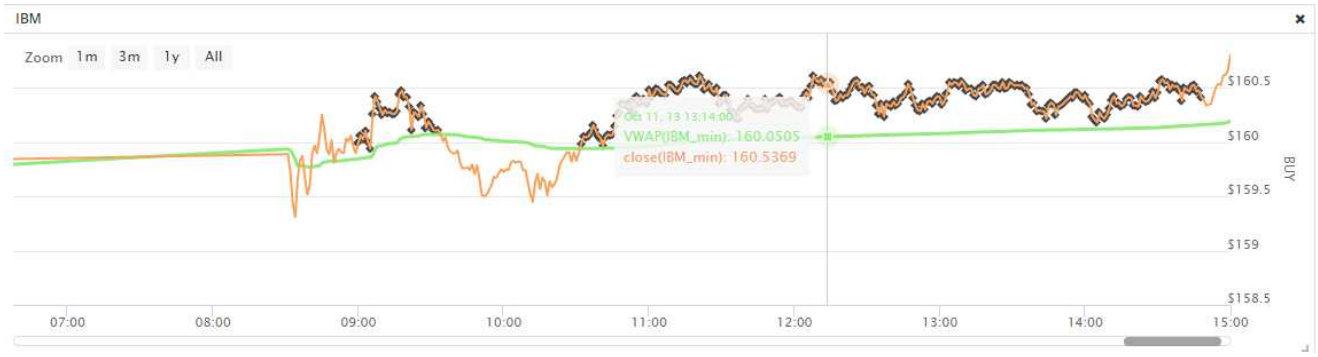
The following table describes the arguments the model accepts:

Argument	Data Type	Description	Default Value
`period`	`float`	Period of the standard deviation indicator	60
`deviations`	`int`	The number of deviations away from the mean before submitting an order	2
`resolution`	`Resolution`	The resolution of the STD and SMA indicators	`Resolution.MINUTE`

For more information about this model, see the class reference and implementation .

VWAP Model

The `VolumeWeightedAveragePriceExecutionModel` works to fill your orders at or better than the volume-weighted average price (VWAP) for the trading day. This is a best-effort algorithm, so no guarantee can be made that it will reach the VWAP.



VWAP Execution Model Fill Placements

The model uses market orders and it only works if the security subscription provides intraday data. It sets a maximum order size of 1% of the current bar's volume, but you can update the maximum order size through the `maximum_order_quantity_percent_volume` member.

```
# Use VolumeWeightedAveragePriceExecutionModel to execute trades based on volume-weighted average price, aiming to minimize market impact
and achieve better execution prices.
self.set_execution(VolumeWeightedAveragePriceExecutionModel())
```

For more information about this model, see the class reference and implementation .

Algorithm Framework

Hybrid Algorithms

Introduction

Classic style algorithms can also use Algorithm Framework modules. This allows you to get the best of both styles of algorithm design, combining easily pluggable modules with the superior control of classic format.

Examples of popular use-cases of a hybrid approach are:

- Using framework universe selection models as the assets to select.
- Using portfolio construction models to decide portfolio allocations.
- Using a risk control model for free portfolio risk monitoring.
- Passing data between modules freely without concerns of the module interface.

Universe Selection

You can add one or more Framework Universe Selection Models to your algorithm and it will operate normally. You can also combine it with the `add_universe` method of the classic approach. The following example initializes a classic algorithm with framework universe selection models:

```py
# Use a hybrid approach by combining Framework Universe Selection Models with classic methods to leverage both the modular flexibility of
Framework modules and the control of classic design. This allows for better universe selection, portfolio allocation, and risk
management.
def initialize(self):
    self.universe_settings.asynchronous = True
    self.set_universe_selection(ManualUniverseSelectionModel([ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]))
    self.add_universe_selection(ManualUniverseSelectionModel([ Symbol.create("AAPL", SecurityType.EQUITY, Market.USA) ]))
    self.add_universe(self.universe.top(5))
```

Alpha

You can add one or multiple Alpha models to your classic algorithm and place the orders using `Insight` objects without a Portfolio Construction model. To receive the collection of `Insight` objects in a classic algorithm, implement the `insights_generated` event handler:

```py
# Add one or multiple Alpha models to a classic algorithm allowing for direct trading based on generated Insight objects, bypassing the
need for a Portfolio Construction model.
def initialize(self):
    self.add_alpha(EmaCrossAlphaModel())
    # The insights_generated event handler is used to receive and process the collection of insights, facilitating flexible and dynamic
trading strategies.
    self.insights_generated += self.on_insights_generated

def on_insights_generated(self, algorithm: IAlgorithm, insights_collection: GeneratedInsightsCollection) -> None:
    insights = insights_collection.insights
```

Portfolio Construction

You can add a Portfolio Construction model to your classic algorithm and have it place orders without returning `Insight` objects from an Alpha model. To emit insights without an Alpha model, in the `on_data` method, call the `EmitInsights` method.

The following example uses a Portfolio Construction Framework model with `EmitInsights` method in a classic algorithm:

```py
# Add a Portfolio Construction model to place orders directly without relying on Alpha-generated insights.
def initialize(self):
    self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

# Use the emit_insights method in the on_data method to provide trading signals for executing trades, ensuring balanced investment with
the chosen Portfolio Construction model.
def on_data(self, slice):
 self.emit_insights(Insight("GOOG", timedelta(minutes=20), InsightType.PRICE, InsightDirection.UP))
 self.emit_insights([
  Insight("AAPL", timedelta(minutes=20), InsightType.PRICE, InsightDirection.UP),
  Insight("MSFT", timedelta(minutes=20), InsightType.PRICE, InsightDirection.UP)
 ])
```

Risk Management

Some Risk Management Models don't require a Portfolio Construction model to provide `PortfolioTarget` objects, allowing them to directly monitor the portfolio holdings and liquidate positions when neccessary. To see which pre-built Risk Management models don't need the Portfolio Construction model to provide `PortfolioTarget` objects, see Supported Models .

You can add one or more Risk Management Models to your algorithm and it will operate normally. The following example initializes a classic algorithm with framework risk management models:

```py
# Add Risk Management models to monitor portfolio risk directly, without needing PortfolioTargets from a Portfolio Construction model.
These models can independently manage holdings and trigger liquidations as needed.
def initialize(self):
    self.add_risk_management(MaximumDrawdownPercentPerSecurity(0.05))
    self.add_risk_management(MaximumUnrealizedProfitPercentPerSecurity(0.1))
```

Execution

Execution models can place orders for your strategy instead of placing them manually. LEAN routes PortfolioTarget objects to the `execute` method of the Execution Model to place orders.

The following example uses a Framework Execution Model in a classic style algorithm:

```py
# Use set_execution to place orders via the Execution Model, which routes PortfolioTarget objects to its
# execute method, instead of placing orders manually. This ensures automated and consistent order execution.
def initialize(self):
    # Execute the market order imediately to fill the portfolio targets.
    self.set_execution(ImmediateExecutionModel())
```

Universe Timing Considerations

If the Alpha, Portfolio Construction, Risk Management, or Execution model manages some indicators or consolidators for securities in the universe and the universe selection runs during the indicator sampling period or the consolidator aggregation period, the indicators and consolidators might be missing some data. For example, take the following scenario:

- The security resolution is minute
- You have a consolidator that aggregates the security data into daily bars to update the indicator
- The universe selection runs at noon

In this scenario, you create and warm-up the indicator at noon. Since it runs at noon, the history request that gathers daily data to warm up the indicator won't contain any data from the current day and the consolidator that updates the indicator also won't aggregate any data from before noon. This process doesn't cause issues if the indicator only uses the close price to calculate the indicator value (like the simple moving average indicator) because the first consolidated bar that updates the indicator will have the correct close price. However, if the indicator uses more than just the close price to calculate its value (like the True Range indicator), the open, high, and low values of the first consolidated bar may be incorrect, causing the initial indicator values to be incorrect.

Examples

The following examples demonstrate common practices for implementing hybrid algorithms.

Example 1: Corresponding Alpha

The following algorithm trades two separate logics:

- EMA cross on the 20 most liquid stocks
- Buy and hold the top 20 weighted SPY constituents

To do so, we can save the universe and allow an access point in the corresponding alpha model to filter the signals that are only applicable to the selected equities from each universe.

```py
from Alphas.EmaCrossAlphaModel import EmaCrossAlphaModel, SymbolData

class FrameworkHybridAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe of the most liquid stocks since their trend is more capital-supported to trend EMA cross.
        self.universe1 = self.add_universe(self.liquid_selection)
        # Emit insights if the trend of selected liquid stocks changes.
        self.add_alpha(CustomEmaCrossAlphaModel(self))

        # Add another universe set to pick the most weighted SPY constituents since they usually have excess return compared to the rest.
        self.universe2 = self.add_universe(self.universe.etf("SPY", Market.USA, self.universe_settings, self.etf_constituents_filter))
        # Emit insights for all selected SPY constituents.
        self.add_alpha(CustomConstantAlphaModel(self))

        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

    def liquid_selection(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select the top 20 liquid equities for EMA cross trading.
        return [x.symbol for x in sorted(fundamentals, key=lambda f: f.dollar_volume, reverse=True)[:20]]

    def etf_constituents_filter(self, constituents: List[ETFConstituentUniverse]) -> List[Symbol]:
        # Select the top 20 weighted stocks of SPY constituents to hold.
        return [x.symbol for x in sorted([c for c in constituents if c.weight], key=lambda c: c.weight, reverse=True)[:20]]

class CustomEmaCrossAlphaModel(EmaCrossAlphaModel):
    def __init__(self, algorithm: FrameworkHybridAlgorithm, fast_period: int = 20, slow_period: int = 60, resolution: Resolution =
Resolution.DAILY) -> None:
        self.algorithm = algorithm
        super().__init__(fast_period, slow_period, resolution)

    def on_securities_changed(self, algorithm: QCAlgorithm, changes: SecurityChanges) -> None:
        for added in changes.added_securities:
            # Only trade the liquid universe for this alpha model.
            if added.symbol in self.algorithm.universe1.selected:
                symbol_data = self.symbol_data_by_symbol.get(added.symbol)
                if symbol_data is None:
                    symbol_data = SymbolData(added, self.fast_period, self.slow_period, algorithm, self.resolution)
                    self.symbol_data_by_symbol[added.symbol] = symbol_data
                else:
                    # A security that was already initialized was re-added, reset the indicators.
                    symbol_data.fast.reset()
                    symbol_data.slow.reset()

        for removed in changes.removed_securities:
            data = self.symbol_data_by_symbol.pop(removed.symbol, None)
            if data is not None:
                # clean up our consolidators.
                data.remove_consolidators()

class CustomConstantAlphaModel(ConstantAlphaModel):
    def __init__(self, algorithm: FrameworkHybridAlgorithm) -> None:
        self.algorithm = algorithm
        # Daily insight length due to daily signals.
        super().__init__(InsightType.PRICE, InsightDirection.UP, timedelta(1))

    def update(self, algorithm: QCAlgorithm, data: Slice) -> List[Insight]:
        insights = super().update(algorithm, data)
        # Only trade the selected SPY constituents universe for this alpha model.
        return [i for i in insights if i.symbol in self.algorithm.universe2.selected]
```

Example 2: Bracket Order On Insight

The following algorithm uses framework universe selection and alpha model to filter the universe and generate insights. However, the portfolio construction model cannot handle bracket orders easily. Hence, we will access the active insights to place bracket orders using the classic implementation, with stop losses at 1% and take profits at 2%.

PY

```
class FrameworkHybridAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)

        # Add a universe of the most liquid stocks since their trend is more capital-supported to trend EMA cross.
        self.universe1 = self.add_universe(self.liquid_selection)
        # Emit insights if the trend of selected liquid stocks changes.
        self.add_alpha(EmaCrossAlphaModel(20, 60, Resolution.DAILY))

        self.ordered_insights = []

    def liquid_selection(self, fundamentals: List[Fundamental]) -> List[Symbol]:
        # Select the top 20 liquid equities for EMA cross trading.
        return [x.symbol for x in sorted(fundamentals, key=lambda f: f.dollar_volume, reverse=True)[:20]]

    def on_data(self, slice: Slice) -> None:
        # Exit positions if insight is expired.
        for insight in self.ordered_insights.copy():
            # If the insight is expired or profit or loss are taken.
            if insight.is_expired(self.utc_time) or not self.portfolio[insight.symbol].invested:
                # Discontinue the insight.
                self.insights.cancel([insight])
                # Liquidate any positions and any open (bracket) orders.
                self.liquidate(insight.symbol)
                # Remove from the cache.
                self.ordered_insights.remove(insight)

        # Place orders for active insights if it has not yet been ordered.
        active_insights = self.insights.get_active_insights(self.utc_time)
        for insight in active_insights:
            if insight not in self.ordered_insights:
                # Position sizing by given weight or equal weighting to dissipate capital risk.
                if insight.weight:
                    size = insight.weight
                else:
                    size = 1 / len(active_insights)
                self.set_holdings(insight.symbol, size)
                # Add to cache to avoid re-order.
                self.ordered_insights.append(insight)

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            if order_event.ticket.order_type == OrderType.MARKET and self.portfolio[order_event.symbol].quantity:
                # Stop loss order at 1%.
                stop_price = order_event.fill_price * (0.99 if order_event.fill_quantity > 0 else 1.01)
                self.stop_market_order(order_event.symbol, -self.portfolio[order_event.symbol].quantity, stop_price)
                # Take profit order at 2%.
                take_profit_price = order_event.fill_price * (1.02 if order_event.fill_quantity > 0 else 0.98)
                self.limit_order(order_event.symbol, -self.portfolio[order_event.symbol].quantity, take_profit_price)
            elif order_event.ticket.order_type == OrderType.STOP_MARKET or order_event.ticket.order_type == OrderType.LIMIT:
                # Cancel any open order if stop loss or take profit order filled.
                self.transactions.cancel_open_orders()
```

Algorithm Framework

Insight Manager

Introduction

The `InsightManager` tracks all of the Insight objects in your algorithm. It's an `InsightCollection` , which stores all of your insights in an internal dictionary that has the security `Symbol` as the key and a list of `Insight` objects as the value. You can access the manager anywhere in your algorithm where you have reference to the algorithm class. If you want to use an `InsightCollection` in your algorithm that's seperate from the `InsightManager` , create a new `InsightCollection` .

Check Membership

To check if an insight exists in the `InsightManager` , call the `contains` method.

```
PY
# Check the algorithms central insight collection (InsightManager) to avoid emitting duplicates, reducing the processing work.
if algorithm.insights.contains(insight):
    pass
```

To check if the `InsightManager` has an insight for a `Symbol` , call the `contains_key` method.

```
PY
# Check the algorithms central insight collection (InsightManager) if there's already been an insight for this symbol across the other
alphas.
if algorithm.insights.contains_key(symbol):
    pass
```

To check if the `InsightManager` has active insights for a `Symbol` at a specific Coordinated Universal Time (UTC), call the `has_active_insights` method.

```
PY
# Check if there are active insights for a specific symbol at the given time to determine if alpha should emit more insights
if algorithm.insights.has_active_insights(symbol, utc_time):
    pass
```

Get Insights

To get the insights for a `Symbol` , index the `InsightManager` with the `Symbol` .

```
PY
if algorithm.insights.contains_key(symbol):
    insights = algorithm.insights[symbol]
```

To get the insights that pass a filter, call the `get_insights` method.

```
PY
insights = algorithm.insights.get_insights(lambda insight: insight.direction == Insightdirection.up)
```

To iterate through the `InsightManager` , call the `get_enumerator` method.

```
PY
enumerator = algorithm.insights.get_enumerator()
```

To get all of the insights that will be active at a certain UTC time, call the `get_active_insights` method.

```
PY
active_insights = algorithm.insights.get_active_insights(utc_time)
```

Get the Next Expiry Time

To get the next insight expiry time in UTC, call the `get_next_expiry_time` method.

```
next_expiry = algorithm.insights.get_next_expiry_time()
```

Remove Insights

Only the Portfolio Construction model should remove insights from the `InsightManager` . It should remove insights when the insights expire and when the corresponding security leaves the universe .

To remove an insight from the `InsightManager` , call the `remove` method.

```
remove_successful = algorithm.insights.remove(insight)
```

To remove all the insights for a set of `Symbol` objects, pass a list of `Symbol` objects the `clear` method.

```
algorithm.insights.clear(symbols)
```

To remove all the insights that will be expired at a certain UTC time, call the `remove_expired_insights` method.

```
expired_insights = algorithm.insights.remove_expired_insights(utc_time)
```

Cancel Insights

In some cases, you may want to cancel an Insight. For example, if a Risk Management model in your algorithm liquidates a security, it should also cancel all of the active insights for the security. If you don't cancel the insights, the Portfolio Construction model might create a new PortfolioTarget to re-enter the position.

Another example of a situtation where you would want to cancel an Insight is when an Alpha model in your algorithm determines the trading opportunity has pre-maturely ended. For instance, say you want your algorithm to enter a long position in a security when its Relative Strength Index (RSI) moves below 20 and then liquidate the position when the security's RSI moves above 30. If you emit an Insight that has a duration of 30 days when the RSI moves below 20 but the RSI moves above 30 in 10 days, you should cancel the Insight when the RSI moves above 30.

To cancel insights, call the `cancel` / `Expire` method with a list of `Insight` objects.

```
algorithm.insights.cancel(insights)
```

To cancel all the insights for some securities, call the `cancel` / `Expire` method with a list of `Symbol` objects.

```
algorithm.insights.cancel(symbols)
```

When you cancel an active insight, it's `close_time_utc` property is set to one second into the past.

Add Insights to Self-Managed Insight Collections

To add insights to the `InsightManager` , return a list of `Insight` objects from the update method of your Alpha model or call the `emit_insights` method. If you manage an `InsightCollection` that's seperate from the `InsightManager` , there are some methods to add `Insight` objects to it.

To add an insight to an `InsightCollection` , call the `add` method.

```
# You can manually add an Insight to an InsightCollection object to organize, fetch, or cancel insights an alpha has generated.
self.insight_collection.add(insight)
```

To add a list of insights, call the `add_range` method.

```
# You can manually add a list of Insights to an InsightCollection object to organize, fetch, or cancel insights an alpha has generated.
self.insight_collection.add_range(insights)
```

Preserve Insights Between Deployments

Follow these steps to use the Object Store to preserve the algorithm state across live deployments:

1. Create an algorithm that defines a storage key and adds insights to the Insight Manager .

```
class ObjectStoreChartingAlgorithm(QCAlgorithm):
    def initialize(self):
        self.insight_key = f"{self.project_id}/insights"
        self.set_universe_selection(ManualUniverseSelectionModel([ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]))
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(5), 0.025, None))
```

2. At the top of the algorithm file, add the following imports:

```
from Newtonsoft.Json import JsonConvert
from System.Collections.Generic import List
```

   `Insight` objects are C# objects, so you need the preceding C# libraries to serialize and deserialize them.

3. In the on_end_of_algorithm event handler of the algorithm, get the Insight objects and save them in the Object Store as a JSON object.

```
def on_end_of_algorithm(self):
    insights = self.insights.get_insights(lambda x: x.is_active(self.utc_time))
    content = ','.join([JsonConvert.SerializeObject(x) for x in insights])
    self.object_store.save(self.insight_key, f'[{content}]')
```

4. At the bottom of the `initialize` method, read the Insight objects from the Object Store and add them to the Insight Manager .

```
if self.object_store.contains_key(self.insight_key):
    insights = JsonConvert.DeserializeObject[List[Insight]](
        self.object_store.read(self.insights_key)
    )
    self.insights.add_range(insights)
```

The following algorithm provides a full example of preserving the Insight state between deployments:

```
from Newtonsoft.Json import JsonConvert
from System.Collections.Generic import List


class ObjectStoreInsightsAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.universe_settings.resolution = Resolution.DAILY

        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)

        self.insights_key = f"{self.project_id}/insights"

        # Read the file with the insights
        if self.object_store.contains_key(self.insights_key):
            # Load cached JSON key value pair from object store, then add to algorithm insights.
            insights = JsonConvert.DeserializeObject[List[Insight]](
                self.object_store.read(self.insights_key)
            )
            self.log(f"Read {len(insights)} insight(s) from the Object Store")
            self.insights.add_range(insights)

            # Delete the key to reuse it
            self.object_store.delete(self.insights_key)

        self.set_universe_selection(ManualUniverseSelectionModel([ Symbol.create("SPY", SecurityType.EQUITY, Market.USA) ]))
        self.set_alpha(ConstantAlphaModel(InsightType.PRICE, InsightDirection.UP, timedelta(5), 0.025, None))
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel(Resolution.DAILY))

    def on_end_of_algorithm(self) -> None:
        # Get all active insights.
        insights = self.insights.get_insights(lambda x: x.is_active(self.utc_time))
        # If we want to save all insights (expired and active), we can use
        # insights = self.insights.get_insights(lambda x: True)
        self.log(f"Save {len(insights)} insight(s) to the Object Store.")
        content = ','.join([JsonConvert.SerializeObject(x) for x in insights])
        self.object_store.save(self.insights_key, f'[{content}]')
```

Properties

The `InsightManager` has the following properties:

Examples

The following examples demonstrate some common practices for using insight manager.

Example 1: Cancel Insight

If the risk management model only submits the `PortfolioTarget` to liquidate the security, it will be re-ordered immediately due to the active insight. Hence, we also need to add logic to cancel the insight in the `manage_risk` method of the risk management model. The below example only permits 1-day holding time per each insight through a custom risk management model.

PY

```
class FrameworkInsightManagerAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 11, 15)
        self.set_cash(1000000)
        # Seed the price of each asset with its last known price to avoid trading errors.
        self.settings.seed_initial_prices = True
        # Add a universe of the most liquid stocks since their trend is more capital-supported.
        self.add_universe_selection(QC500UniverseSelectionModel())
        # Emit insights on EMA cross, indicating the trend changes. We use short-term versus medium-term for more trade opportunities.
        self.add_alpha(EmaCrossAlphaModel(20, 60, Resolution.DAILY))
        # Equal weighting on each insight to dissipate capital risk evenly.
        self.set_portfolio_construction(EqualWeightingPortfolioConstructionModel())

        # Add a risk management model that controls maximum holding time.
        self.add_risk_management(MaximumHoldingTimeRiskManagementModel(timedelta(5)))

class MaximumHoldingTimeRiskManagementModel(RiskManagementModel):
    def __init__(self, maximum_holding_time: timedelta) -> None:
        self.maximum_holding_time = maximum_holding_time
        # A dictionary to hold the holding time of each position.
        self.holding_time = {}

    def manage_risk(self, algorithm: QCAlgorithm, targets: List[PortfolioTarget]) -> List[PortfolioTarget]:
        targets = []

        for symbol, holding in algorithm.portfolio.items():
            if holding.invested:
                if symbol not in self.holding_time:
                    # For newly open positions, save the entry time.
                    self.holding_time[symbol] = algorithm.time
                elif algorithm.time - self.holding_time[symbol] > self.maximum_holding_time:
                    # Liquidate the position if the holding time exceeds the permitted level.
                    targets.append(PortfolioTarget(symbol, 0))
                    # Cancel the insight to avoid repeated re-ordering.
                    algorithm.insights.cancel([symbol])
                    # Remove from the dictionary.
                    self.holding_time.pop(symbol)

        return targets
```

Charting

Introduction

We provide a powerful charting API that you can use to build many chart types.

Charts

Charts contain a collection of series, which display data on the chart. To add a chart to an algorithm, create a `Chart` object and then call the `add_chart` method.

```py
chart = Chart("<chartName>")
self.add_chart(chart)
```

The `Chart` constructor expects a name argument. The following chart names are reserved:

- Assets Sales Volume
- Exposure
- Portfolio Margin

Series

A chart series displays data on the chart. To add a series to a chart, create a `Series` object and then call the `add_series` method.

```py
series = Series("<seriesName>")
chart.add_series(series)
```

Arguments

There are several other headers for the `Series` constructor.

```
Series(name, type)
Series(name, type, index)
Series(name, type, index, unit)
Series(name, type, unit)
Series(name, type, unit, color)
Series(name, type, unit, color, symbol)
```

The following table describes the constructor arguments:

Argument	Data Type	Description
`name`	`str`	Name of the series
`type`	`SeriesType`	Type of the series
`index`	`int`	Index position on the chart of the series
`unit`	`str`	Unit for the series axis
`color`	`Color`	Color of the series
`symbol`	`ScatterMarkerSymbol`	Symbol for the marker in a scatter plot series

The default `Series` is a line chart with a "$" unit on index 0.

Names

The `Series` constructor expects a name argument. If you add a series to one of the default charts, some series names may be reserved. The following table shows the reserved series name for the default charts:

Chart Name	Reserved Series Names
Strategy Equity	Equity, Return
Capacity	Strategy Capacity
Drawdown	Equity Drawdown
Benchmark	Benchmark
Portfolio Turnover	Portfolio Turnover

Types

The `SeriesType` enumeration has the following members:

A `Line` series connects plotted values with a continuous line. This is the default series type.

```
chart.add_series(Series("EMA", SeriesType.LINE, "$", Color.ORANGE))
```

A `Scatter` series plots individual data points without connecting lines. Use the `ScatterMarkerSymbol` parameter to set the marker shape.

```
chart.add_series(Series("Signal", SeriesType.SCATTER, "$", Color.GREEN, ScatterMarkerSymbol.TRIANGLE))
```

A `Candle` series displays OHLC data as candlesticks. Use the `CandlestickSeries` helper class and plot a `TradeBar` to populate all four values.

```
chart.add_series(CandlestickSeries("SPY", "$"))
```

A `Bar` series draws vertical bars for each plotted value, which is useful for volume or count data.

```
chart.add_series(Series("Volume", SeriesType.BAR, "", Color.GRAY))
```

To create a `StackedArea` chart, add multiple series to the same chart. Each series contributes an area band that stacks on top of the others.

```
chart.add_series(Series("Stocks", SeriesType.STACKED_AREA, "%"))
chart.add_series(Series("Bonds", SeriesType.STACKED_AREA, "%"))
```

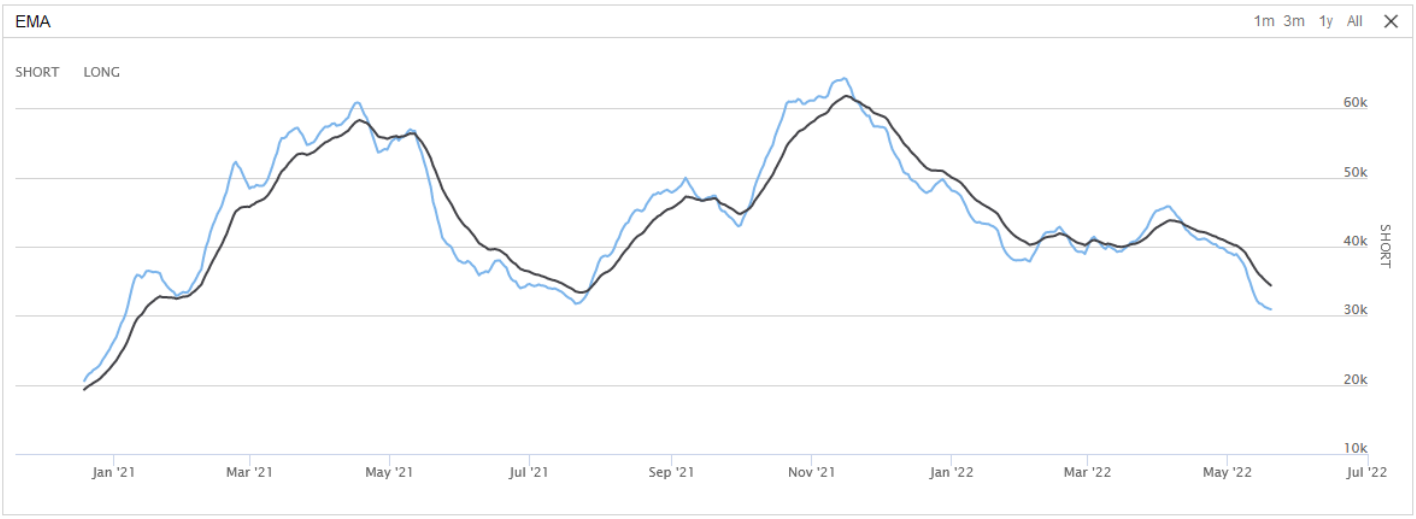
To create a `Treemap` chart, add multiple series to the same chart. Each series becomes a tile and its plotted value determines the tile size.

```
chart.add_series(Series("SPY", SeriesType.TREEMAP, "$"))
chart.add_series(Series("AAPL", SeriesType.TREEMAP, "$"))
```

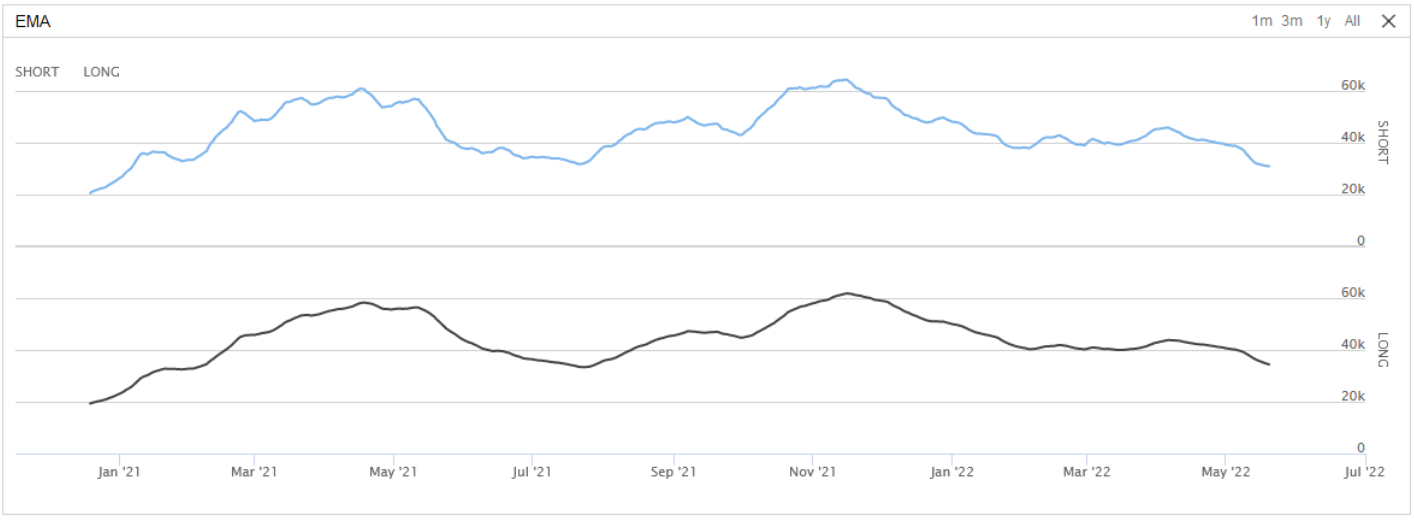
For a full example that demonstrates all series types, see Examples .

Index

The series index refers to its position in the chart. If all the series are at index 0, they lay on top of each other. If each series has its own index, each series will be separate on the chart. The following image shows an EMA cross chart with both EMA series set to the same index:



The following image shows the same EMA series, but with the short EMA on index 0 and the long EMA on index 1:



Colors

To view the available `Color` options, see the Color Struct Properties in the .NET documentation.

Scatter Marker Symbols

The `ScatterMarkerSymbol` enumeration has the following members:

Candlestick Series

A chart candlestick series displays candlesticks on the chart. To add a candlestick series to a chart, create a `CandlestickSeries` object and then call the `add_series` method.

```py
candlestick_series = CandlestickSeries("<seriesName>")
chart.add_series(candlestick_series)
```

There are several other headers for the `CandlestickSeries` constructor.

```
CandlestickSeries(name)
CandlestickSeries(name, index)
CandlestickSeries(name, index, unit)
CandlestickSeries(name, unit)
```

The following table describes the constructor arguments:

Argument	Data Type	Description
`name`	`str`	Name of the series
`index`	`int`	Index position on the chart of the series
`unit`	`str`	Unit for the series axis

The default `CandlestickSeries` has 0 index and "$" unit.

Plot Series

To add a data point to a chart series, call the `plot` method. If you haven't already created a chart and series with the names you pass to the `plot` method, the chart and/or series is automatically created.

```
PY
self.plot("<chartName>", "<seriesName>", value)
```

The `value` argument can be an integer or decimal number. If the chart is a time series, the value is added to the chart using the algorithm time as the x-coordinate.

To plot the current value of indicators, call the `plot` method. The method accepts up to four indicators.

```
PY
# In Initialize
symbol = self.add_equity("SPY")
sma_short = self.sma(symbol, 10)
sma_long = self.sma(symbol, 20)

# In OnData
self.plot("<chartName>", sma_short, sma_long)
```

To plot all of the values of some indicators, in the `initialize` method, call the `plot_indicator` method. The method plots each indicator value as the indicator updates. The method accepts up to four indicators.

```
PY
symbol = self.add_equity("SPY")
sma_short = self.sma(symbol, 10)
sma_long = self.sma(symbol, 20)
self.plot_indicator("<chartName>", sma_short, sma_long)
```

Plot Candlestick

To add a sample of open, high, low, and close values to a candlestick series, call the `plot` method with the data points. If you haven't already created a chart and series with the names you pass to the `plot` method, the chart and/or series is automatically created.

```
PY
self.plot("<chartName>", "<seriesName>", open, high, low, close)
```

The `open` , `high` , `low` , and `close` arguments can be an integer for decimal number. If the chart is a time series, the values are added to the chart using the algorithm time as the x-coordinate.

To plot the current trade bar, call the `plot` method with a `TradeBar` argument in the `on_data` method.

```
PY
# In Initialize
equity = self.add_equity("SPY")
forex = self.add_forex("EURUSD")

# In OnData
trade_bar = slice.bars["SPY"];
collapsed = slice.quote_bars["EURUSD"].collapse()   # Collapses QuoteBar into TradeBar object
self.plot("<chartName>", "<seriesName>", trade_bar)
self.plot("<chartName>", "<seriesName>", collapsed)
```

To plot consolidated bars, call the `plot` method with a `TradeBar` argument in the consolidation handler.

```py
# In Initialize
equity = self.add_equity("SPY")
self.consolidate(equity.symbol, timedelta(minutes=10), self._consolidation_handler)

# Define the consolidation handler
def _consolidation_handler(self, consolidated_bar: TradeBar) -> None:
    self.plot("<chartName>", "<seriesName>", consolidated_bar)
```

Plot Asset Data

Asset plots display the trade prices of an asset and the following order events you have for the asset:

Order Event	Icon
Submissions	Gray circle
Updates	Blue circle
Cancellations	Gray square
Fills and partial fills	Green (buys) or red (sells) arrows

The following image shows an example asset plot for AAPL:

The order submission icons aren't visible by default.

For more information about these charts, including how to view them in QC Cloud, see Asset Plots for backtests or live trading .

View Charts

The following table describes where you can access your charts, depending on how to deploy your algorithms:

Location	Algorithm Lab Algorithms	CLI Cloud Algorithms	CLI Local Algorithms
Backtest results page	✓	✓	
Live results page	✓	✓	
/backtests/read endpoint	✓	✓	
/live/read endpoint	✓	✓	
ReadBacktest method	✓	✓	
ReadLiveAlgorithm method	✓	✓	
Local JSON file in your <projectName> / backtests / <timestamp> or <projectName> / live / <timestamp> directory		✓	✓

Quotas

When you run backtests, you must stay within the plotting quotas to avoid errors.

Cloud Quotas

Intensive charting requires hundreds of megabytes of data, which is too much to stream online or display in a web browser. The number of series and the number of data points per series you can plot depends on your organization tier . The following table shows the quotas:

Tier	Max Series	Max Data Points per Series
Free	10	4,000
Quant Researcher	10	8,000
Team	25	16,000
Trading Firm	25	32,000
Institution	100	96,000

If you exceed the series quota, your algorithm stops executing and the following message displays:

Exceeded maximum chart series count, new series will be ignored. Limit is currently set at <quota>.

If you exceed the data points per series quota, the following message displays:

Exceeded maximum points per chart, data skipped

If your plotting needs exceed the preceding quotas, create the plots in the Research Environment instead .

Local Quotas

If you execute local backtests, the charting quotas are set by the `maximum-chart-series` and `maximum-data-points-per-chart-series` configuration settings .

Examples

The following example demonstrates the `Line` , `Scatter` , `Candle` , `Bar` , `StackedArea` , and `Treemap` series types. The algorithm trades a three-asset portfolio and plots EMA crossover signals, candlestick prices, volume bars, portfolio allocation as a stacked area, and sales volume as a treemap.

```py
class AllSeriesTypesAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 1, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100000)
        self.settings.automatic_indicator_warm_up = True

        tickers = ["AAPL", "MSFT", "GOOG"]
        self._symbols = []
        self._ema_fast = {}
        self._ema_slow = {}
        for ticker in tickers:
            symbol = self.add_equity(ticker, Resolution.DAILY).symbol
            self._symbols.append(symbol)
            self._ema_fast[symbol] = self.ema(symbol, 10)
            self._ema_slow[symbol] = self.ema(symbol, 50)

        # Line and Scatter chart: EMA values with crossover markers.
        price_chart = Chart("EMA Crossover")
        self.add_chart(price_chart)
        price_chart.add_series(Series("EMA Fast", SeriesType.LINE, "$", Color.ORANGE))
        price_chart.add_series(Series("EMA Slow", SeriesType.LINE, "$", Color.BLUE))
        price_chart.add_series(Series("Cross Up", SeriesType.SCATTER, "$",
            Color.GREEN, ScatterMarkerSymbol.TRIANGLE))
        price_chart.add_series(Series("Cross Down", SeriesType.SCATTER, "$",
            Color.RED, ScatterMarkerSymbol.TRIANGLE_DOWN))

        # Bar chart: daily volume.
        volume_chart = Chart("Volume")
        self.add_chart(volume_chart)
        volume_chart.add_series(Series("Volume", SeriesType.BAR, "", Color.GRAY))

        # Candlestick chart: daily OHLC for the first symbol.
        candle_chart = Chart("Candlestick")
        self.add_chart(candle_chart)
```

```
        candle_chart.add_series(CandlestickSeries(self._symbols[0].value, "$"))

        # Stacked area chart: portfolio allocation by asset over time.
        allocation_chart = Chart("Allocation")
        self.add_chart(allocation_chart)
        for symbol in self._symbols:
            allocation_chart.add_series(Series(symbol.value, SeriesType.STACKED_AREA, "%"))

        # Treemap chart: total sales volume per asset.
        treemap_chart = Chart("Sales Volume")
        self.add_chart(treemap_chart)
        for symbol in self._symbols:
            treemap_chart.add_series(Series(symbol.value, SeriesType.TREEMAP, "$"))

    def on_data(self, slice: Slice) -> None:
        # Trade on EMA crossovers for the first symbol.
        symbol = self._symbols[0]
        if not self._ema_fast[symbol].is_ready:
            return

        if (self._ema_fast[symbol].current.value > self._ema_slow[symbol].current.value and
                self._ema_fast[symbol][1] < self._ema_slow[symbol][1]):
            self.set_holdings(symbol, 0.4)
            self.set_holdings(self._symbols[1], 0.3)
            self.set_holdings(self._symbols[2], 0.3)
        elif (self._ema_fast[symbol].current.value < self._ema_slow[symbol].current.value and
                self._ema_fast[symbol][1] > self._ema_slow[symbol][1]):
            self.liquidate()

    def on_end_of_day(self, symbol: Symbol) -> None:
        if symbol != self._symbols[0]:
            return

        # Plot EMA line and scatter crossover markers.
        self.plot("EMA Crossover", "EMA Fast", self._ema_fast[symbol].current.value)
        self.plot("EMA Crossover", "EMA Slow", self._ema_slow[symbol].current.value)
        if self._ema_fast[symbol].is_ready and self._ema_fast[symbol][1] != 0:
            if (self._ema_fast[symbol].current.value > self._ema_slow[symbol].current.value and
                    self._ema_fast[symbol][1] < self._ema_slow[symbol][1]):
                self.plot("EMA Crossover", "Cross Up", self.securities[symbol].price)
            elif (self._ema_fast[symbol].current.value < self._ema_slow[symbol].current.value and
                    self._ema_fast[symbol][1] > self._ema_slow[symbol][1]):
                self.plot("EMA Crossover", "Cross Down", self.securities[symbol].price)

        # Plot candlestick and volume bar.
        data = self.securities[symbol].get_last_data()
        if data:
            self.plot("Candlestick", symbol.value, data)
            self.plot("Volume", "Volume", data.volume)

        # Plot allocation and treemap.
        total_value = self.portfolio.total_portfolio_value
        if total_value > 0:
            for s in self._symbols:
                weight = 100 * self.portfolio[s].holdings_value / total_value
                self.plot("Allocation", s.value, weight)
                self.plot("Sales Volume", s.value, self.portfolio[s].total_sale_volume)
```

Logging

Introduction

Algorithms can record string messages ('log statements') to a file for analysis after a backtest is complete, or as a live algorithm is running. These records can assist in debugging logical flow errors in the project code. Consider adding them in the code block of an `if` statement to signify an error has been caught.

It's good practice to add logging statements to live algorithms so you can understand its behavior and keep records to compare against backtest results. If you don't add logging statements to a live algorithm and the algorithm doesn't trade as you expect, it's difficult to evaluate the underlying problem.

Log Messages

Log statements are added to the log file while your algorithm continues executing. Logging dataset information is not permitted. Use `log` method statements to debug your backtests and live trading algorithms.

If you execute algorithms in QuantConnect Cloud, log length is capped by organization tier . If your organization hits the daily limit, contact us .

If you log the same content multiple times, only the first instance is added to the log file. To bypass this rate-limit, add a timestamp to your log messages.

For live trading, the log files of each cloud project can store up to 100,000 lines for up to one year. If you log more than 100,000 lines or some lines become older than one year, we remove the oldest lines in the files so your project stays within the quota.

To record the algorithm state when the algorithm stops executing, add log statements to the `on_end_of_algorithm` event handler.

```
self.log("My log message")
```

Debug Messages

Debug statements are the same as log statements, but `debug` method statements are orange in the Cloud Terminal. Use these statements when you want to give more attention to a message in the Cloud Terminal. Debug messages can be up to 200 characters in length. If you send multiple debug statements within 1 second, your messages are rate-limited to avoid crashing your browser.

```
self.debug("My debug message")
```

Error Messages

Error statements are the same as log statements, but `error` method statements are displayed in red text in the Cloud Terminal. Use these statements when you want to give the most attention to a message in the Cloud Terminal. Error statements are rate-limited like debug statements.

```
self.error("My error message")
```

Quit Messages

Quit statements cause your project to stop running and may log some data to the log file and Cloud Terminal. These statements are orange in the Cloud Terminal. When you call the `quit` method method, the program continues executing until the end of the method definition. If you want to quit execution immediately, `return` after you call `quit` method.

```
self.quit("My quit message")
```

Get Logs

The following tables describe how to access the logs of your backtests and live algorithms:

Deployment Target	Execution Mode	Access Tools
QuantConnect Cloud	Backtest	• Backtest results page in the Algorithm Lab • Backtest results page on Local Platform
QuantConnect Cloud	Live Trading	• Live results page in the Algorithm Lab • Live results page on Local Platform • /live/read/log endpoint
Local	Backtest	• Backtest results page on Local Platform • <organizationWorkspace> / <projectName> / backtests / <unixTimestamp> / <algorithmId>-log.txt • lean logs CLI command
Local	Live Trading	• Live results page on Local Platform • <organizationWorkspace> / <projectName> / live / <timeStamp> / / L-<deploymentId>-log.txt • lean logs CLI command

Example

The following examples demonstrate some common practices for logging.

Example 1: On Events And Trading Logic

The following example trades a simple EMA cross trend-following strategy on SPY. We log or debug messages in event handlers and trading logic changes to get information on any event flow.

PY

```
class LoggingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)

        # Warm up the algorithm for indicator readiness.
        self.set_warm_up(20, Resolution.DAILY)

    def on_warmup_finished(self) -> None:
        # Signals on warm-up finished.
        self.log("Warm up finished")

    def on_data(self, slice: Slice) -> None:
        bar = None if self.is_warming_up else slice.bars.get(self.spy)
        if bar:
            # Trend-following strategy using price and EMA.
            # If the price is above EMA, SPY is in an uptrend, and we buy it.
            if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
                self.debug("Trend changes to upwards")
                self.set_holdings(self.spy, 1)
            elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
                self.debug("Trend changes to downwards")
                self.set_holdings(self.spy, -1)

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            # Log the order details if being filled.
            self.log(f"Order filled - Quantity: {order_event.quantity}")
```

Statistics

LEAN automatically monitors a set of statistics as your algorithms execute. You can even create custom statistics for further analysis.

Algorithm Statistics

Trade statistics and rolling KPIs

Runtime Statistics

Performace at a single moment in time

Trade Statistics

Get and customize trade statistics

See Also

Cloud Backtest Results

Statistics

Algorithm Statistics

Introduction

The `StatisticsResults` object tracks all the statistics of your algorithm. You can access the statistics anywhere in your algorithm where you have a reference to the algorithm class. Some of these statistics are a function of the risk free interest rate and the number of trading days per year .

Properties

The `StatisticsResults` object has the following properties:

Add Statistics

To add custom statistics to the `StatisticsResults` object, call the `set_summary_statistic` method with the statistic name and value.

```
self.set_summary_statistic("name", value)
```

Get Values

To get the `StatisticsResults` , use the `statistics` member of the algorithm class.

```
sharpe = self.statistics.total_performance.portfolio_statistics.sharpe_ratio
```

To get the value of a custom statistic while your algorithm runs, index the `summary` member with the statistic name. The values in the `summary` dictionary are strings, so you may need to cast the value that returns to a different data type.

```
value = self.statistics.summary["name"]
```

The following table describe other ways to get the statistics after your backtest completes:

Deployment Target	Access Tools
QuantConnect Cloud	• Backtest results page in the Algorithm Lab • Backtest results page on Local Platform • ReadBacktest API method • /backtests/read endpoint
Local	• Backtest results page on Local Platform • <organizationWorkspace> / <projectName> / backtests / <unixTimestamp> / <algorithmId>.json

Examples

The following examples demonstrate common practices for getting algorithm statistics.

Example 1: Portfolio Statistics

To evaluate the algorithm's best domain or regime, we can log the portfolio statistics daily along the backtest to analyze. To do so, we log the Sharpe Ratio, Beta, and 95% Value-at-Risk at the end of each day, storing them in the object store for assessment later.

```py
class AlgorithmStatisticsAlgorithm(QCAlgorithm):
    _statistics = ["Time,Sharpe,Beta,95VaR"]

    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Warm-up indicator for immediate readiness.
        self.warm_up_indicator(self.spy, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Trend-following strategy using price above or below EMA.
            # If the price is above EMA, SPY is in an uptrend, and we buy it.
            if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
                self.set_holdings(self.spy, 1)
            elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
                self.set_holdings(self.spy, -1)

    def on_end_of_day(self, symbol: Symbol) -> None:
        # Obtain the algorithm statistics interested.
        sharpe = self.statistics.total_performance.portfolio_statistics.sharpe_ratio
        b = self.statistics.total_performance.portfolio_statistics.beta
        var = self.statistics.total_performance.portfolio_statistics.value_at_risk_95

        # Plot the statistics.
        self.plot("Statistics", "Sharpe", sharpe)
        self.plot("Statistics", "Beta", b)
        self.plot("Statistics", "Value-at-Risk", var)

        # Write to save the statistics.
        self._statistics.append(f'{self.time.strftime("%Y%m%d")},{sharpe},{b},{var}')

    def on_end_of_algorithm(self) -> None:
        # Save the logged statistics for later access in the object store.
        self.object_store.save(f'{self.project_id}/algorithm-statistics', '\n'.join(self._statistics))
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithms
StatisticsResultsAlgorithm.py Python

Statistics

Runtime Statistics

Introduction

Runtime statistics show the performace of your algorithm at a single moment in time. Some of these statistics are a function of the risk free interest rate and the number of trading days per year .

Default Statistics

The following table describes the default runtime statistics:

Statistic	Description
Equity	The total portfolio value if all of the holdings were sold at current market rates. Equity equals the sum of cash and the market value of all open positions.
Fees	The total quantity of fees paid for all the transactions during the algorithm's trading period. Total fees include brokerage commissions, exchange fees, and other transaction costs.
Holdings	The absolute sum of the items in the portfolio. Holdings represent the total market value of all positions, regardless of whether they are long or short.
Net Profit	The dollar-value return across the entire trading period.
PSR	The probability that the estimated Sharpe ratio of an algorithm is greater than a benchmark.
Return	The rate of return across the entire trading period.
Unrealized	The amount of profit a portfolio would capture if it liquidated all open positions and paid the fees for transacting and crossing the spread. Unrealized profit becomes realized profit when the positions are closed.
Volume	The total value of assets traded for all of an algorithm's transactions during the trading period.

Add Statistics

To add a custom runtime statistic, call the `set_runtime_statistic` method with a `name` and `value` . The `value` argument can be a `string` or a number.

```
self.set_runtime_statistic(name, value)
```

Don't try to set a value for any of the preceding default runtime statistics. LEAN overwrites the value that you try to set.

Get Values

To get the value of a runtime statistic in an algorithm, index the `runtime_statistics` member of the algorithm class with the statistic name. The values of the `runtime_statistics` dictionary are strings, so you may need to cast the result to a different data type.

```
value = self.runtime_statistics[name]
```

The following table describe other ways to get the runtime statistics of your backtests and live algorithms:

Deployment Target	Execution Mode	Access Tools
QuantConnect Cloud	Backtest	• Backtest results page in the Algorithm Lab • Backtest results page on Local Platform • ReadBacktest API method • /backtests/read endpoint
QuantConnect Cloud	Live Trading	• Live results page in the Algorithm Lab • Live results page on Local Platform • ReadLiveAlgorithm API method • /live/read endpoint
Local	Backtest	• Backtest results page on Local Platform • <organizationWorkspace> / <projectName> / backtests / <unixTimestamp> / <algorithmId>.json
Local	Live Trading	• Live results page on Local Platform • <organizationWorkspace> / <projectName> / live / <timeStamp> / / L-<deploymentId>.json

Examples

The following examples demonstrate common practices for setting runtime statistics.

Example 1: Display Banner

The following algorithm demonstrates how to put calculated runtime statistics of interest in the top display banner in the backtest interface and plot the data for analysis.

PY

```
class RuntimeStatisticsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Warms up indicator for immediate readiness.
        self.warm_up_indicator(self.spy, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar:
            # Trend-following strategy using price and EMA.
            # If the price is above the EMA, SPY is in an uptrend, and we buy it.
            if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
                self.set_holdings(self.spy, 1)
            elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
                self.set_holdings(self.spy, -1)

    def on_end_of_day(self, symbol: Symbol) -> None:
        # Set the 95VaR and fee-to-profit ratio to the display banner.
        var = self.statistics.total_performance.portfolio_statistics.value_at_risk_95
        self.set_runtime_statistic("95VaR", var)
        fee_to_profit = self.statistics.total_performance.trade_statistics.total_fees /\
            self.statistics.total_performance.trade_statistics.total_profit_loss \
            if self.statistics.total_performance.trade_statistics.total_profit_loss != 0 else 0
        self.set_runtime_statistic("Fee-to-Profit", fee_to_profit)

        # Plot the calculated statistics.
        self.plot("Statistics", "95VaR", var)
        self.plot("Statistics", "fee-to-profit", fee_to_profit)

    def on_end_of_algorithm(self) -> None:
        # Log the 95VaR when the algorithm stops
        value_at_risk_95 = self.runtime_statistics["95VaR"]
        self.log(f'Final {value_at_risk_95=}')
```

Statistics

Trade Statistics

Introduction

The `TradeBuilder` tracks the trades of your algorithm and calculates some statistics. Some of these statistics are a function of the risk free interest rate and the number of trading days per year . You can adjust how the `TradeBuilder` defines a trade and how it matches offsetting order fills.

Set Trade Builder

To set the `TradeBuilder` , in the `initialize` method, call the `set_trade_builder` method.

```
self.set_trade_builder(TradeBuilder(grouping_method, matching_method))
```

The following table describes the arguments the `TradeBuilder` constructor accepts:

Argument	Data Type	Description	Default Value
`grouping_method`	`FillGroupingMethod`	The method used to group order fills into trades	
`matching_method`	`FillMatchingMethod`	The method used to match offsetting order fills	

The `FillGroupingMethod` enumeration has the following members:

The `FillMatchingMethod` enumeration has the following members:

Default Behavior

The default `TradeBuilder` uses `FillGroupingMethod.FILL_TO_FILL` and `FillMatchingMethod.FIFO` .

Check Open Positions

To check if you have a position open for a security as defined by the `FillGroupingMethod` , call the `has_open_position` method.

```
# Check if the trade builder has recorded any active trades (unclosed positions)
# for a security.
has_open_position = self.trade_builder.has_open_position(self._symbol)
```

Get Closed Trades

To get your closed trades, use the `closed_trades` property.

```
trades = self.trade_builder.closed_trades
```

This property returns a list of `Trade` objects, which have the following attributes:

Examples

The following examples demonstrate common practices for using trade statistics.

Example 1: Kelly Criterion Sizing Using Previous Trades

The following algorithm uses the previous trades to update the win probability and expected return of each trade of the EMA cross strategy of SPY, by the concept of Bayes Theorem. The updated figures are used for position sizing through Kelly Criterion.

PY

```
class TradeStatisticsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Warm-up indicator for immediate readiness.
        self.warm_up_indicator(self.spy, self._ema, Resolution.DAILY)

        # Set up a trade builder to track the trade statistics; we are interested in open-to-close round trade.
        self.set_trade_builder(TradeBuilder(FillGroupingMethod.FLAT_TO_FLAT, FillMatchingMethod.FIFO))

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if not bar:
            return
        # Trend-following strategy using price and EMA.
        # If the price is above EMA, SPY is in an uptrend, and we buy it.
        sign = 0
        if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
            sign = 1
        elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
            sign = -1
        else:
            return

        size = 1
        trades = self.trade_builder.closed_trades

        if len(trades) > 4:
            # Use the trade builder to obtain the win rate and % return of the last five trades to calculate position size.
            last_five_trades = sorted(trades, key=lambda x: x.exit_time)[-5:]
            prob_win = len([x for x in last_five_trades if x.is_win]) / 5
            win_size = np.mean([x.profit_loss / x.entry_price for x in last_five_trades])
            # Use the Kelly Criterion to calculate the order size.
            size = max(0, prob_win - (1 - prob_win) / win_size)

        self.set_holdings(self.spy, size * sign)
```

Live Trading

QuantConnect enables you to run your algorithms in live mode with real-time market data. You can use the same algorithm for backtesting and live trading with different brokerages and data providers.

Key Concepts

Synchonize your account and algorithm

Brokerages

Stream data into your algorithm

Data Providers

Send orders to your brokerage

Trading and Orders

Reconciliation

Differences from backtesting

Notifications

Stay informed on algorithm decisions

Commands

Display information on algorithm decisions

Arbitrage

Charting and Logging

Signal Exports

Send portfolio targets to third-party services

See Also

QuantConnect Live Trading
Supported Brokerages

Live Trading

Key Concepts

Introduction

Algorithms should generally work seamlessly shifting from backtesting to live trading with no changes required. In backtesting algorithm events are triggered as fast as possible, whereas in live trading events are triggered in realtime.

Algorithm Portfolio

The live trading algorithm portfolio is loaded from the brokerage holdings, including open orders. When there are unsupported asset types in the portfolio the live trading strategy is stopped as we cannot be sure how to model the asset.

In paper trading the algorithm starts from no portfolio, and attempts to remember the previous portfolio state from the last time the portfolio was deployed.

Your algorithm should not assume it starts from an empty portfolio as you may have holdings from previous deployments.

Stateful Strategies

There is no automatic state management of live strategies in QuantConnect. We recommend algorithms reconstruct state or indicator needs using the WarmUp and History methods.

Once the algorithm has warmed up some objects you can use the Object Store to save the data for the next algorithm restart.

In the event the algorithm is terminated unexpectedly, you should review the live algorithm portfolio at the brokerage to confirm it will behave as expected.

Data Delivery Times

Most data is available at the same time as in backtesting, with the exception of daily data which is often delayed by 4-8 hours (4am - 8am).

US Equities universe selection data is processed at 7-8am ET.

Latency

Live data takes time to travel from the source to your algorithm. The QuantConnect latencies vary depending on the data provider, but for US Equities, we have a latency of 5-40 milliseconds. A much more significant source of latency is the round trip order times from brokers, which can vary from 100ms to 5 seconds. QuantConnect is not intended for high-frequency trading, but we have integrations to high-speed brokers if you need.

Other Brokerages

If we do not support your brokerage reach out to let us and we can attempt to add them to the roadmap. With enough community requests we can build and open-source the brokerage integration.

Live Trading

Brokerages

Introduction

Brokerages provide you with a connection to the market so you can fill trades. To avoid placing invalid orders for execution in live trading, LEAN validates your orders before sending them to the real brokerage. To view all of the integrated brokerages, see Brokerages .

Portfolio

In live trading, LEAN populates the `portfolio` object with your account holdings and the `transactions` object with your open positions. If you don't manually subscribe to the assets in your account, LEAN subscribes to them with the lowest resolution of the subscriptions in your algorithm. For example, say you hold AAPL shares in your account and create the following subscriptions in your algorithm:

```
self.add_equity("SPY", Resolution.HOUR)
self.add_equity("MSFT", Resolution.SECOND)
```

In this case, LEAN subscribes to second-resolution data for AAPL since the lowest resolution in your algorithm is

`Resolution.SECOND` .

Deposits and Withdrawals

You can deposit and withdraw cash from your brokerage account while you run an algorithm that's connected to the account. We sync the algorithm's cash holdings with the cash holdings in your brokerage account every day at 7:45 AM Eastern Time (ET).

Monitor the Brokerage Connection

We notify your algorithm when your brokerage connection disconnects and reconnects.

Lost Connections

If the brokerage connection breaks, we notify your algorithm through the `on_brokerage_disconnect` event handler.

```
def on_brokerage_disconnect(self) -> None:
    self.debug("Brokerage connection lost")
```

Restored Connections

When the brokerage connection restores after it disconnects, we notify your algorithm through the `on_brokerage_reconnect` event handler.

```
def on_brokerage_reconnect(self) -> None:
    self.debug("Brokerage connection restored")
```

When LEAN reconnects with your brokerage, it synchronizes the state of your live orders. For example, say the brokerage fills your limit order during the period of disconnection. When the connection restores, LEAN updates the state of your portfolio and orders to reflect the filled order, but you won't receive an order event .

Example Algorithm

For a full example algorithm that implements the `on_brokerage_disconnect` and `on_brokerage_reconnect` methods, see the BrokerageActivityEventHandlingAlgorithm in the LEAN GitHub repository.

Monitor Brokerage Messages

When your brokerage sends you a message, we notify your algorithm through the `on_brokerage_message` event handler.

```py
def on_brokerage_message(self, message_event: BrokerageMessageEvent) -> None:
    self.debug(f"Brokerage message received: {message_event.message}")
```

`BrokerageMessageEvent` objects have the following attributes:

For a full example algorithm that implements the `on_brokerage_message` method, see the BrokerageActivityEventHandlingAlgorithm in the LEAN GitHub repository.

To handle brokerage messages outside of your algorithm class, create and set a `BrokerageMessageHandler` . For a more information example, see the Reality Modeling > Brokerage Message Handler .

Brokerage Models

Brokerage features and limitations are realistically modelled with Brokerage Models. These reality models set the supported order and asset types for the brokerage. To learn how to set your specific brokerage, see Supported Models .

Examples

The following examples demonstrate common practices for handling live brokerage messages.

Example 1: Pre-Open Order Error

The following algorithm simulate extended market hour trading on Tradier Brokerage . We place a market-on-open order at 7:30am and expect the order cannot submit to the broker since it is not supported, resulting into an error.

```py
class OrderErrorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Simulate Tradier brokerage, which does not support the market-on-open orders.
        self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE, AccountType.CASH)

        # Request extended market hour SPY data for trading.
        self.spy = self.add_equity("SPY", extended_market_hours=True).symbol

        # Set a scheduled event to trade 2 hours pre-open to place the market-on-open-order.
        self.schedule.on(
            self.date_rules.every_day(self.spy),
            self.time_rules.before_market_open(self.spy, 120),
            self.open_position
        )

    def open_position(self) -> None:
        # Buy on market open will result in an error since Tradier does not support it.
        self.market_on_open_order(self.spy, 10)

    # It will be triggered on live trading.
    def on_brokerage_message(self, message: BrokerageMessageEvent) -> None:
        if message.type == BrokerageMessageType.ERROR:
            self.log(f"{self.time}: {message.type}: Message: {message.message}")
```

Live Trading

Data Providers

Introduction

Data providers provide a stream of asset prices and quotes delivered to your trading algorithm during live execution. You need live data provider to inject data into your algorithm so that you can make real-time trading decisions and so that the values of the securities in your portfolio update in real-time. You can source data from QuantConnect, brokerage, or a third-party source. To view the available data providers you can use in QuantConnect cloud, see Datasets . To view the available data providers you can use with local algorithms, see the Data Providers .

Data Providers

You can source data from QuantConnect, your brokerage, or third-party providers. The following table lists the data providers:

Data Provider	Cloud	Local
QuantConnect	✓	
Alpaca	✓	✓
Alpha Vantage		✓
Binance	✓	✓
Bitfinex	✓	✓
Bybit	✓	✓
Charles Schwab	✓	✓
Coinbase	✓	✓
FactSet		✓
Interactive Brokers	✓	✓
IQFeed		✓
Kraken	✓	✓
Polygon	✓	✓
Samco	✓	✓
Theta Data		✓
TradeStation	✓	✓
Terminal Link		✓
Tradier	✓	✓
Zerodha	✓	✓

Bar Building

We aggregate ticks to build bars.

In live trading, bars are built using the exchange timestamps with microsecond accuracy. This microsecond-by-microsecond processing of the ticks can mean that the individual bars between live trading and backtesting can have slightly different ticks. As a result, it's possible for a tick to be

counted in different bars between backtesting and live trading, which can lead to bars having slightly different open, high, low, close, and volume values.

Latency

Live data takes time to travel from the source to your algorithm. The QuantConnect latencies vary depending on the data provider, but for US Equities, we have a latency of 5-40 milliseconds. A much more significant source of latency is the round trip order times from brokers, which can vary from 100ms to 5 seconds. QuantConnect is not intended for high-frequency trading, but we have integrations to high-speed brokers if you need.

History Requests

In live trading, if you make a history request for minute data at noon and the history period covers the start of the previous day to the present moment, the data from the previous day will be backtest data. The data of the current day will be live data that we collected throughout the morning. If you make this history request in a backtest, you might get slightly different data for the current day because of post-processing from the data vendor.

In cloud deployments, QuantConnect is the default data provider. If you remove it and use a different data provider, you won't get any price data from QuantConnect. However, we provide the following datasets:

- US Equity Security Master
- US Futures Security Master
- US Equity Option Universe
- US Index Option Universe
- Universe selection datasets
- Non-streaming alternative datasets

Warm Up Periods

LEAN supports an automated fast-forward system called "Warm Up". It simulates winding back the date you deployed your algorithm by a specific time period. In backtests, it changes the start date of your algorithm to an earlier time. In live trading, it fetches data from before your deployment and fast forwards through your strategy up to the present day. Warm Up is a great way to prepare your algorithm and its indicators for trading. You can't place trades during the warm-up period because the data feed is replaying historical data for setting algorithm state.

Live Trading

Trading and Orders

Introduction

LEAN has dozens of methods to create, update, and cancel orders. You can place orders automatically with helper methods or manually through methods on the algorithm API. You can fetch, update, and cancel manual orders with order tickets . As the state of your orders change, LEAN creates events that notify your algorithm.

In backtesting, LEAN simulates order fills with historical data, but you can create your own fill, fee, slippage, and margin models via plugin points. You control how optimistic or pessimistic order fills are with transaction model classes. For more information about these types of models, see Reality Modeling .

In live trading, orders fill asynchronously. We send your order to the API of your brokerage and wait for their response to update the state of your algorithm and portfolio. The timing of live order fills doesn't depend on the resolution of your security subscriptions. When your order fills, the fill price and fee is set by your brokerage. You can add event handlers to your algorithm to monitor the brokerage connection and brokerage messages .

In backtesting, the trade fill timing depends on the resolution of your security subscription. For example, if you subscribe to a security with minute resolution data, the algorithm only receives data in one-minute time slices . As a result, the fill model can only evaluate if the order should fill on a minute-by-minute frequency.

Order Life Cycle

When you call one of the order methods to place an order, LEAN performs pre-order checks to ensure that the order meets some requirements and uses the last known price to check you have enough capital . To see the requirements of each order, see the Requirements section of the documentation for the order types and the Orders section of your brokerage model . If you can place the order, LEAN creates `Order` and OrderTicket objects. The order ticket is sent to your brokerage. As the brokerage processes your order, it returns another order ticket and it's compared against the order to see if the order is satisfied. Orders fill asynchronously in live trading, so if you want to change an order, you must request it with the order ticket. Order changes are not guaranteed since your order may fill by the time brokerage receives the request.

When you create an order ticket, LEAN validates it to avoid sending orders that will be rejected by your brokerage. If the order is invalid, it's status is set to `OrderStatus.Invalid` . Otherwise, LEAN sets the order status to `OrderStatus.New` , sends the order to your brokerage, and waits for the brokerage message. If your brokerage accepts the order, the order status becomes `OrderStatus.Submitted` . Otherwise, it becomes `OrderStatus.Canceled` .

You can update orders until they fill or the brokerage prevents modifications. If you place an order update request, LEAN validates it to avoid sending orders that will be rejected by you brokerage. If the order update is valid, LEAN sends the update request and waits for the brokerage message. If your brokerage accepts the update, the order status becomes `OrderStatus.UpdateSubmitted` . Otherwise, it becomes `OrderStatus.Canceled` .

The brokerage notifies LEAN when you order fills (`OrderStatus.FILLED`), partially fills (`OrderStatus.PartiallyFilled`) or is canceled (`OrderStatus.Canceled`). The brokerage can cancel your order before it completely fills. For example, if you place a market on open and the asset price moves before the market open to where you can't afford the quantity of the order, the brokerage rejects your order. The brokerage can also cancel your market on open order after partial fills if there is no shares left to trade.

Transaction Manager

LEAN doesn't save orders and order tickets across live deployments.

When you deploy your algorithm, LEAN fetches the open orders from your brokerage and populate the algorithm transactions manager (`SecurityTransactionManager`).

Disable Buying Power

You can disable order margin checks and opt to let your brokerage decide to accept or reject the trades. This is helpful in live trading if you have a more permissive brokerage margin allowance that what LEAN models. The default position group buying power models are helpful for Option trading

strategies. However, it can be counterproductive if it's not a supported Option strategy . To disable the validations of the default position group buying power model, use the `NullSecurityPositionGroupModel` . To set the `NullSecurityPositionGroupModel` for the portfolio, during initialization , call the `set_positions` method with the `SecurityPositionGroupModel. NULL` argument.

```py
self.portfolio.set_positions(SecurityPositionGroupModel.NULL)
```

To disable the validations of the default buying power model , use the `NullBuyingPowerModel` . To set the `NullBuyingPowerModel` for a security subscription, call the `set_buying_power_model` method with the `BuyingPowerModel. NULL` argument.

```py
def initialize(self) -> None:
    equity = self.add_equity("SPY")
    # To disable any buying power of the selected security
    equity.set_buying_power_model(BuyingPowerModel.NULL)
    # Alias:
    # equity.set_margin_model(SecurityMarginModel.NULL)
```

You can also set the `NullBuyingPowerModel` in a security initializer . If your algorithm has a universe, use the security initializer technique. To set the buying power of securities in the security initializer, set the brokerage model , set the security initializer , and then create security subscriptions.

```py
class AddSecurityInitializerExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the security initializer to set models of assets.
        self.add_security_initializer(self._custom_security_initializer)

    def _custom_security_initializer(self, security: Security) -> None:
        # Overwrite the security buying power
        security.set_buying_power_model(BuyingPowerModel.NULL)
```

Trading and Orders

Financial Advisors

Introduction

Financial Advisor accounts enable certified professionals to use a single trading algorithm to manage several client accounts. Our Interactive Brokers integration enables you to place FA group orders if your IB account code starts with F, FA, or I.

Group Routing

To place trades using a subset of client accounts, create Account Groups in Trader Workstation and then define the `InteractiveBrokersOrderProperties` when you create orders.

```py
self.default_order_properties = InteractiveBrokersOrderProperties()
self.default_order_properties.fa_group = "TestGroupEQ"
self.default_order_properties.fa_method = "Equal"
self.default_order_properties.account = "DU123456"
```

SecurityHolding objects aggregate your positions across all the account groups. If you have two groups where group A has 10 shares of SPY and group B has -10 shares of SPY, then `self.portfolio["SPY"].quantity` is zero.

Allocation Methods

LEAN supports several allocation methods for FA group orders. If you intend to use the same group allocation method for every order, set the `default_order_properties` of your algorithm, which sets the order properties for all of your orders.

```py
def initialize(self) -> None:
    # Set the default order properties
    self.default_order_properties = InteractiveBrokersOrderProperties()
    self.default_order_properties.fa_group = "TestGroupEQ"
    self.default_order_properties.fa_method = "Equal"
    self.default_order_properties.account = "DU123456"

def on_data(self, slice: Slice) -> None:
    # Use default order order properties
    LimitOrder(_symbol, quantity, limitPrice);
```

To adjust the order properties of an order, change the `default_order_properties` or pass an order properties object to the order method . The following sections explain the FA group allocation methods.

Equal

This group allocation method distributes shares equally between all accounts in the group. When you use this method, you need to specify an order quantity.

For example, say your Account Group includes four accounts and you place an order to buy 400 shares of a stock. In this case, each account receives 100 shares. If your Account Group includes six accounts, each account receives 66 shares, and then 1 share is allocated to each account until all are distributed. After you submit the order, your algorithm receives order events to track the order progress. When all the shares are bought, the order status is `OrderStatus.FILLED` . If one of the accounts in the group can't afford 10 of the shares it needs to buy, 10 shares are cancelled and you'll only end up buying 390 shares in total.

```py
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "Equal"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Net Liquidation Value

This group allocation method distributes shares based on the net liquidation value of each account. The system calculates ratios based on the net

liquidation value in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with Net Liquidation values of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```py
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "NetLiq"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Available Equity

This group allocation method distributes shares based on the amount of available equity in each account. The system calculates ratios based on the available equity in each account and allocates shares based on these ratios. When you use this method, you need to specify an order quantity.

For example, say your account group includes three accounts, A, B and C with available equity of 25,000, 50,000 and $100,000, respectively. In this case, the system calculates a ratio of 1:2:4. If you place an order for 700 shares of a stock, it allocates 100 shares to Client A, 200 shares to Client B, and 400 shares to Client C.

```py
order_properties = InteractiveBrokersOrderProperties()
order_properties.fa_method = "AvailableEquity"
self.limit_order(self._symbol, quantity, limit_price, order_properties=order_properties)
```

Subscription Requirements

To use FA group orders through our Interactive Brokers integration, you need to connect as a member of a Trading Firm and Institution organization. If you aren't currently on either of these tiers, upgrade your organization .

Live Trading

Reconciliation

Introduction

Algorithms usually perform differently between backtesting and live trading over the same time period. Backtests are simulations where we model reality as close as possible, but the modeling isn't always perfect. To measure the performance differences, we run an out-of-sample (OSS) backtest in parallel to all of your live trading deployments. The live results page displays the live equity curve and the OOS backtest equity curve of your algorithms.

If your algorithm is perfectly reconciled, it has an exact overlap between its live and OOS backtest equity curves. Deviations mean that the performance of your algorithm has differed between the two execution modes. Several factors can contribute to the deviations.

To generate the OOS reconciliation curve for a live deployment and overlay live vs. backtest equity and order fills from the Research Environment, see Live Reconciliation .

Differences From Data

The data that your algorithm uses can cause differences between backtesting and live trading performance.

Data Providers

QuantConnect provides high-quality historical data for all supported asset classes . However, our live data provider lacks data feeds for equity options, index options, and futures options. If your algorithm trades these asset classes, you must rely on brokerage or third-party data feeds.

Look-Ahead Bias

The Time Frontier minimizes the risk of look-ahead bias in backtests, but it does not completely eliminate the risk of look-ahead bias. For instance, if you use a custom dataset that contains look-ahead bias, your algorithm's live and backtest equity curves may deviate. To avoid look-ahead bias with custom datasets, set a `period` on your custom data points so that your algorithm receives the data points after the `time + period` .

Discrete Time Steps

In backtests, we inject data into your algorithm at predictable times, according to the data resolution. In live trading, we inject data into your algorithm when new data is available. Therefore, if your algorithm has a condition with a specific time (i.e. time is 9:30:15), the condition may work in backtests but it will always fail in live trading since live data has microsecond precision. To avoid issues, either use a time range in your condition (i.e. 9:30:10 < time < 9:30:20), use a rounded time, or use a Scheduled Event.

Custom Data Emission Times

Custom data is often timestamped to midnight, but the data point may not be available in reality until several days after that point. If your custom dataset is prone to this delay, your backtest may not fetch the same data at the same time or frequency that your live trading algorithm receives the data, leading to deviations between backtesting and live trading. To avoid issues, ensure the timestamps of your custom dataset are the times when the data points would be available in reality.

In backtesting, LEAN and custom data are perfectly synchonized. In live trading, daily and hourly data from a custom data source are not because of the frequency that LEAN checks the data source depends on the `resolution` argument. The following table shows the polling frequency of each resolution:

Resolution	Update Frequency
Daily	Every 30 minutes
Hour	Every 30 minutes
Minute	Every minute
Second	Every second
Tick	Constantly checks for new data

Split Adjustment of Indicators

Backtests use adjusted price data by default. Therefore, if you don't change the data normalization mode , the indicators in your backtests are updated with adjusted price data. In contrast, if a split or dividend occurs in live trading, your indicators will temporarily contain price data from before the corporate event and price data from after the corporate event. If this occurs, your indicators will produce different signals in your backtests compared to your live trading deployment. To avoid issues, reset and warm up your indicators when your algorithm receives a corporate event.

Tick Slice Sizes

In backtesting, we collect ticks into slices that span 1 millisecond before injecting them into your algorithm. In live trading, we collect ticks into slices that span up to 70 milliseconds before injecting them into your algorithm. This difference in slice sizes can cause deviations between your algorithm's live and OOS backtest equity curves. To avoid issues, ensure your strategy logic is compatible with both slice sizes.

Opening and Closing Auctions

The opening and closing price of the day is set by very specific opening and closing auction ticks. When a stock like Apple is listed, it’s listed on Nasdaq. The open auction tick on Nasdaq is the price that’s used as the official open of the day. NYSE, BATS, and other exchanges also have opening auctions, but the only official opening price for Apple is the opening auction on the exchange where it was listed.

We set the opening and closing prices of the first and last bars of the day to the official auction prices. This process is used for second, minute, hour, and daily bars for the 9:30 AM and 4:00 PM Eastern Time (ET) prices. In contrast, other platforms might not be using the correct opening and closing prices.

The official auction prices are usually emitted 2-30 seconds after the market open and close. We do our best to use the official opening and closing prices in the bars we build, but the delay can be so large that there isn't enough time to update the opening and closing price of the bar before it's injected into your algorithms. For example, if you subscribe to second resolution data, we wait until the end of the second for the opening price but most second resolution data won’t get the official opening price. If you subscribe to minute resolution data, we wait until the end of the minute for the opening auction price. Most of the time, you’ll get the actual opening auction price with minute resolution data, but there are always exceptions. Nasdaq and NYSE can have delays in publishing the opening auction price, but we don’t have control over those issues and we have to emit the data on time so that you get the bar you are expecting.

Data Updates

Data issues are incorrect or missing values in a dataset. These issues are generally a result of human error or from mistakes in the data collection process. Data issues can be reported by any QuantConnect member. When data issues are reported and verified, our Data Team works to quickly resolve them. Thanks to the communal efforts of the QuantConnect members, the QuantConnect data is reviewed and fixed by over 250,000 people, giving you a very high-quality source of data.

Data updates change backtest results and, consequently, cause differences between backtesting running with fixed data and live trading. For more information on common data issues, please refer to the data issues documentation .

Differences From Modeling

The modeling that your algorithm uses can cause differences between backtesting and live trading performance.

Reality Modeling Error

We provide brokerage models to model fees, slippage, and order fills in backtests. However, these model predictions may not always match the fees that your live algorithm incurs, leading to deviations between backtesting and live trading. You can adjust the reality models that your algorithm uses to more accurately reflect the specific assets that you're trading. For more information about reality models, see Reality Modeling .

Market Impact

We don't currently model market impact. So, if you are trading large orders, your fill prices can be better during backtesting than live trading, causing deviations between backtesting and live trading. To avoid issues, implement a custom fill model in your backtests that incorporates market impact.

Fills

In backtests, orders fill immediately. In live trading, they are sent to your brokerage and take about half a second to execute. If you fill an order in a backtest with stale data, deviations between backtesting and live trading can occur because the order is filled at a price that is likely different from the real market price. Stale order fills commonly occur in backtests when you create a Scheduled Event with an incompatible data resolution. For instance, if you subscribe to hourly data, place a Scheduled Event for 11:15 AM, and fill an order during the Scheduled Event, the order will fill at a stale price because the data between 11:00 AM and 11:15 AM is missing. To avoid stale fills, only place orders when your algorithm receives price data.

In live trading, your brokerage provides the fill price of your orders. Since the backtesting brokerage models do not know the price at which live orders are filled, the fill price of backtest orders is based on the best price available in the current backtesting data. Similarly, limit orders can fill at different prices between backtesting and live trading. In backtesting, limit orders fill as soon as the limit price is hit. In live trading, your brokerage may fill the same limit order at a different price or fail to fill the order, depending on the position of your order in their order book.

Borrowing Costs

By default, LEAN doesn't simulate the cost of borrowing shorts in backtests, but you can enable them by setting a margin interest model.

Differences From Brokerage

The brokerage that your algorithm uses can cause differences between backtesting and live trading performance.

Portfolio Allocations on Small Accounts

If you trade a small portfolio, it's difficult to achieve accurate portfolio allocations because shares are usually sold in whole numbers. For instance, you likely can't allocate exactly 10% of your portfolio to a security. You can use fractional shares to achieve accurate portfolio allocations, but not all brokerages support fractional shares. To get the closest results when backtesting and live trading over the same period, ensure both algorithms have the same starting cash balance.

Different Backtest Parameters

If you don't start your backtest and live deployment on the same date with the same holdings, deviations can occur between backtesting and live trading. To avoid issues, ensure your backtest parameters are the same as your live deployment.

Non-deterministic State From Algorithm Restarts

If you stop and redeploy your live trading algorithm, it needs to restart in a stateful way or else deviations can occur between backtesting and live trading. To avoid issues, redeploy your algorithm in a stateful way using the `set_warm_up` and `history` methods. Furthermore, use the Object Store to save state information between your live trading deployments.

Existing Portfolio Securities

If you deploy your algorithm to live trading with a brokerage account that has existing holdings, your live trading equity curve reflects your existing positions, but the backtesting curve won't. Therefore, if you have existing positions in your brokerage account when you deploy your algorithm to live trading, deviations will occur between backtesting and live trading. To avoid issues, deploy your algorithm to live trading using a separate brokerage account or subaccount that does not have existing positions.

Brokerage Limitations

We provide brokerage models that support specific order types and model your buying power. In backtesting, we simulate your orders with the brokerage model you select. In live trading, we send your orders to your brokerage for execution. If the brokerage model that you use in backtesting is not the same brokerage that you use in live trading, deviations may occur between backtesting and live trading. The deviations can occur if your live brokerage doesn't support the order types that you use or if the backtesting brokerage models your buying power with a different methodology than the real brokerage. To avoid brokerage model issues, set the brokerage model in your backtest to the same brokerage that you use in live trading.

Differences From Third-Party Indicators

The values of our indicators can sometimes produce different results than the indicators on other platforms. There can be several reasons for these discrepancies.

Price Differences

If you find differences in indicator values, compare the prices that are fed into the indicator on each platform. If the input data is slightly different, the indicator values will be different. To test if it's a difference in price data, feed in the data from the third-party platform into our indicators. We validate the indicators against third-party sources and when the values are the same, the indicator values are similar too.

Timestamp Differences

We timestamp our data to the time when the period ends. Many other platforms timestamp to the beginning of the candle.

Implementation Differences

Some indicators can have slightly different default arguments for their implementation. A common difference across platforms is to use a different `MovingAverageType` for the indicators. To view the default arguments for all our indicators, see Supported Indicators . To view the full implementation of our indicators, see the LEAN GitHub repository .

Warm Up Period Differences

Some platforms use all of the historical data to warm up their indicators while LEAN indicators have a fixed number of bars they need to warm up. As a result, indicators with long memory like the Exponential Moving Average can have slightly different values across platforms.

Risk Free Interest Rate Differences

Some indicators, like Sharpe ratios and Sortino ratios , are a function of the risk free interest rate . LEAN provides several different ways to define the interest rate, which can lead to different indicator values compared to other platforms.

Differences From Real-time Scheduled Events

In live trading, Scheduled Events execute in a parallel thread based on a real-time clock. If you set a Scheduled Event to fire at 10:00 AM, it executes at exactly 10:00 AM. In backtesting, Scheduled Events are part of the main algorithm manager loop, so they may not execute exactly when you set them. For example, if your algorithm subscribes to minute resolution US Equity data with regular trading hours and you set a Scheduled Event to occur at 2:00 AM, your Scheduled Event will execute at 9:31 AM when the next bar is fed into your algorithm.

The difference between live trading and backtesting is important to note because it can affect your algorithm's behavior. There are two common scenarios to consider.

Execution Timing and Backtest Timeouts

Take the following scenario:

- You set Scheduled Events for 2:00 AM, 3:00 AM, and 4:00 AM each day.
- Each Scheduled Event takes eight minutes to execute.
- Your algorithm only subscribes to US Equity securities without extended market hours (9:30 AM - 4:00 PM).

In this scenario, the Scheduled Events each fire at the correct time and execute without error in live trading. In backtesting, all of the Scheduled Events execute at 9:31 AM when your algorithm receives the first bar of the trading day. Since all the Scheduled Events take eight minutes to execute, the algorithm tries to execute all the Scheduled Events but reaches the 10-minute timeout and the backtest stops execution.

Live Data Delays

In backtests, your algorithm receives data at perfect timing. If you request minute resolution data, your algorithm receives the bars at the top of each minute. In live trading, bars have a slight delay, so you may receive them milliseconds after the top of each minute. Take the following scenario:

- You subscribe to minute resolution data
- You set a Scheduled Event for 10:00 AM
- The Scheduled Event checks the current asset price

In live trading, the Scheduled Event executes at exactly 10:00 AM but your algorithm may receive the 9:59-10:00 AM bar at 10:00:00.01 AM. Therefore, when you check the price in the Scheduled Event, the price from the 9:58-9:59 AM bar is the latest price. In backtesting, the Scheduled Event gets the price from the 9:59-10:00 AM bar since your algorithm receives the bar at perfect timing.

Live Trading

Notifications

Introduction

Set up some live trading notifications so that you are notified of market events and your algorithm's performance. QuantConnect Cloud supports email, SMS, webhooks, and Telegram notifications for live trading algorithms. If you run local algorithm with LEAN or the LEAN CLI, the notification services aren't available. To view the number of notification you can send for free in QuantConnect Cloud, see the Live Trading Notification Quotas . Notifications are different from commands because notifications enable you to send information out of your algorithm while commands enable you to manually call methods in your algorithm as it runs.

Email

Email notifications can include up to 10KB of text content in the message body. These notifications can be slow since they go through your email provider. If you don't receive an email notification that you're expecting, check your junk folders.

To send email notifications in your code files, run:

```
PY
message = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.email(address="quant@office.com", subject="Last Trade", message=message)
```

You can include an attachment and set its name.

```
PY
attachment = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
headers = { "filename": "trades.csv" }
self.notify.email(address="quant@office.com", subject="Last Trade", message="See attachment", data=attachment, headers=headers)
```

If you don't provide headers, the attachment name is attachment.txt .

FTP

FTP notifications send files to your FTP server. You can use password and SSH authentication.

To send a file to your FTP server in your code files, run:

```
PY
file_content = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.ftp(hostname="myftpserver.com", username="admin", password="12345", file_path="trades.csv", file_content=file_content,
port=21)
```

To send a file to your SFTP server in your code files, run:

```
PY
file_content = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.sftp(hostname="mysftpserver.com", username="admin", password="12345", file_path="trades.csv", file_content=file_content,
port=21)
```

You can use password authentication over SFTP using SSH keys.

```
PY
file_content = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.sftp(hostname="mysftpserver.com", username="admin", private_key="12345", private_key_passphrase="67890",
file_path="trades.csv", file_content=file_content, port=21)
```

The `file_path` parameter is path to file on the FTP server. The `port` parameter is optional.

SMS

SMS notifications are the only type of notification that you don't need an internet connection to receive. They can include up to 1,600 characters of text content in the message body.

To send SMS notifications in your code files, run:

```py
message = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.sms(phone_number="+100503016366", message=message)
```

Telegram

Telegram notifications are automated messages to a Telegram group.

To send Telegram notifications in your code files, run:

```py
message = "2023-07-13T12:20:00.2033226Z,EURUSD,1.11853,89187,Market,Filled,99758.33511"
self.notify.telegram(id="-503016366", message=message, token="1234")
```

The `id` argument is your Telegram group Id. Your group Id is in the URL when you open your group chat in the Telegram web interface. For example, the group Id of web.telegram.org/z/#-503016366 is -503016366.

The `token` optional argument is a token of a bot you must add to your Telegram group. To create a bot, chat with @BotFather and follow its instructions. If you want to use our bot, the username is @quantconnect_notifications_bot.

To add emoticons to your `message` , convert the emoticon character to UTF-32 with the Unicode Converter on the Branah website. For example, the rocket emoji is `0001f680` in UTF-32 format, so use `\U001f680` in your message.

Webhooks

Webhook notifications are an HTTP-POST request to a URL you provide. The request is sent with a timeout of 300s. You can process these notifications on your web server however you want. For instance, you can inject the content of the notifications into your server's database or use it to create other notifications on your own server.

To send webhook notifications in your code files, run:

```py
data = ""
headers = { 'Authorization': 'Basic XXXXX' }
self.notify.web(address="http://api.myserver.com", data=data, headers=headers)
```

The `data` argument can be a `string` , `float` , or `int` object .

To send a webhook to a Discord server, generate a webhook URL (for instructions, see Intro to Webhooks in the Discord Help Center) and serialize the data in a dictionary with a `content` key.

```py
discord_url = "https://discord.com/api/webhooks/.../..."
self.notify.web(discord_url, json.dumps({'content': 'Hello world!'}))
```

Receive Notifications

To receive external notification in your algorithm, see Commands .

Terms of Use

The notification system can't be used for data distribution.

Examples

The following examples demonstrate common practices for sending notifications in live algorithms.

Example 1: Pre-Open Order Error

The following algorithm simulate extended market hour trading on Tradier Brokerage . We place a market-on-open order at 7:30am and expect the

order cannot submit to the broker since it is not supported, resulting into an error.

```py
class OrderErrorsAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        # Simulate Tradier brokerage, which does not support the market-on-open orders.
        self.set_brokerage_model(BrokerageName.TRADIER_BROKERAGE, AccountType.CASH)

        # Request extended market hour SPY data for trading.
        self.spy = self.add_equity("SPY", extended_market_hours=True).symbol

        # Set a scheduled event to trade 2 hours pre-open to place the market-on-open-order.
        self.schedule.on(
            self.date_rules.every_day(self.spy),
            self.time_rules.before_market_open(self.spy, 120),
            self.open_position
        )

    def open_position(self) -> None:
        # Buy on market open will result in an error since Tradier does not support it.
        self.market_on_open_order(self.spy, 10)

    # It will be triggered on live trading.
    def on_brokerage_message(self, message: BrokerageMessageEvent) -> None:
        if message.type == BrokerageMessageType.ERROR:
            self.log(f"{self.time}: {message.type}: Message: {message.message}")
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
LiveFeaturesAlgorithm.py Python

Live Trading

Commands

Introduction

Commands enable you to manually call methods in your live algorithm as it runs. You can command live projects to update your algorithm state, place orders, or run any other logic. Commands are different from notifications because notifications enable you to send information *out* of your algorithm while commands enable you to send information *into* your algorithm.

Basic Usage

To receive and handle generic commands in your algorithm, define the `on_command` method. This method receives the data you want to inject into your algorithm and returns a boolean that represents if the command was successful.

```
PY
class BasicCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_benchmark(lambda x: 1) # So the algorithm needs no asset data.

    def on_command(self, data):
        self.log(f'Got command at {self.time} with data: {data}')
        return True
```

To invoke the `on_command` event handler, send a payload (for example, `{"ticker": "AAPL", "quantity": 1}`) through the QuantConnect REST API or LEAN CLI . The keys of the payload become the members of the `data` object.

Encapsulate Event Handlers

To encapsulate the command handler logic, define isolated classes. Command classes should extend the `Command` class. Extensions of the `Command` class must implement a `run` method, which receives the payload you send and returns a boolean that represents if the command was successful. To add the command to your algorithm, call the `add_command` method.

```
PY
class EncapsulatedCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        MyCommand.ALGORITHM = self
        self.add_command(MyCommand)
        self.set_benchmark(lambda x: 1) # So the algorithm doesn’t need asset data.

    def do_something(self):
        self.log('Something was done!')

class MyCommand(Command):
    ALGORITHM = None
    ticker = None
    quantity = None
    parameters = {}

    def run(self, algorithm):
     if not MyCommand.ALGORITHM:
            algorithm.log('Please set the static reference to the ALGORITHM attribute in the initialize method: `MyCommand.ALGORITHM =
self`')
            return False
        algorithm.log(f"ticker: {self.ticker}; quantity: {self.quantity}; parameters: {self.parameters}")
        MyCommand.ALGORITHM.do_something()
        return True
```

To invoke the `run` method of the class you define, send a payload with a `$type` key (for example, `{"ticker": "AAPL", "quantity": 1, "$type": "MyCommand"}`). The value of the `$type` key must be the `Command` class you define. The other keys of the payload set the members of the class. If you don’t provide a `$type` key, LEAN runs the `on_command` method instead of the `run` method in your custom `Command` class.

To access your algorithm's attributes, you need to create a global static variable , and assign the reference to the global variable in the algorithm's `initialize` . Then, in the `run` merhod, you can acess your algorithm's attributes through the global.

Send Command Link

You can create the command in your algorithm and then send yourself an email notification with a link inside it. When you click the link, it then executes the event handler in your algorithm. To create the link, call the `link` method. The following algorithm demonstrates how send command requests for the `on_command` event handler:

```
PY
class BasicLinkedCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_benchmark(lambda x: 1) # So the algorithm needs no asset data.
        link = self.link({"ticker": "AAPL", "quantity": 1})
        self.notify.email("email@address.com", "Run Command?", f"Click here to run: {link}")

    def on_command(self, data):
        self.log(f'Got command at {self.time} with data: {data}')
        return True
```

The following algorithm demonstrates how to send command requests for an event handler you encapsulate in a class:

```
PY
class EncapsulatedAndLinkedCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        self.add_command(MyCommand)
        self.set_benchmark(lambda x: 1) # So the algorithm doesn't need asset data.
        potential_command = MyCommand()
        potential_command.ticker = 'AAPL'
        potential_command.quantity = 1
        potential_command.parameters = {"tag": "Signal X"}
        link = self.link(potential_command)
        self.notify.email("email@address.com", "Run Command?", f"Click here to run: {link}")

class MyCommand(Command):
    ticker = None
    quantity = None
    parameters = {}

    def run(self, algorithm):
        algorithm.log(f"ticker: {self.ticker}; quantity: {self.quantity}; parameters: {self.parameters}")
        return True
```

Broadcast Command

You can create the command in your algorithm and then broadcast it to all live algorithms in your organization, except the source algorithm and commands of unregisted type name. The following algorithm demonstrates how broadcast a command to the `on_command` event handler in all your live algorithms:

```
PY
class BasicBroadcastCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_benchmark(lambda x: 1) # So the algorithm needs no asset data.
        link = self.broadcast_command({"ticker": "AAPL", "quantity": 1})

    def on_command(self, data):
        self.log(f'Got command at {self.time} with data: {data}')
        return True
```

You can control the command data by encapsulating the event handler in a class. The following algorithm demonstrates how to broadcast a command for an event handler you encapsulate in a class:

```
class EncapsulatedAndBroadcastCommandAlgorithm(QCAlgorithm):

    def initialize(self):
        self.add_command(MyCommand)
        self.set_benchmark(lambda x: 1) # So the algorithm doesn't need asset data.
        potential_command = MyCommand()
        potential_command.ticker = 'AAPL'
        potential_command.quantity = 1
        potential_command.parameters = {"tag": "Signal X"}
        broadcast_result = self.broadcast_command(potential_command)

class MyCommand(Command):
    ticker = None
    quantity = None
    parameters = {}

    def run(self, algorithm):
        algorithm.log(f"ticker: {self.ticker}; quantity: {self.quantity}; parameters: {self.parameters}")
        return True
```

Send Commands by API

To send a command to your algorithm, send a payload through the QuantConnect REST API or LEAN CLI . Follow these steps to send a command from the command line. In this case, the script calls the `on_command` method.

1. Install the `requests` library, if you haven't already.

```
pip install requests
```

2. Save the following code as `send_command.py` .

```
from base64 import b64encode
from hashlib import sha256
from time import time
from sys import argv
from requests import post

# Edit these values:
USER_ID = 0
API_TOKEN = '_____'
PROJECT_ID = <project_id>

BASE_URL = 'https://www.quantconnect.com/api/v2/'

def get_headers():
    # Get timestamp
    timestamp = f'{int(time())}'
    time_stamped_token = f'{API_TOKEN}:{timestamp}'.encode('utf-8')

    # Get hased API token
    hashed_token = sha256(time_stamped_token).hexdigest()
    authentication = f'{USER_ID}:{hashed_token}'.encode('utf-8')
    authentication = b64encode(authentication).decode('ascii')

    # Create headers dictionary.
    return {
        'Authorization': f'Basic {authentication}',
        'Timestamp': timestamp
    }

def create_command(project_id, command):
    return post(
        f'{BASE_URL}/live/commands/create',
        headers=get_headers(),
        json={"projectId": project_id, "command": command}
    ).json()

# Read the ticker, quantity, and (optional) command type from the command line:
ticker = argv[1]
quantity = int(argv[2])
command = {'ticker': ticker, 'quantity': quantity}
if len(argv) > 3:
    command['$type'] = argv[3]
print(create_command(PROJECT_ID, command))
```

3. Replace the placeholder values at the top of the file with your user Id, API token, and project Id.
4. Run the script, passing the ticker and quantity as arguments. The first form below calls the `on_command` method; the second adds an optional third argument that sets the `$type` key to route to an encapsulated `Command` subclass handler.

```
python send_command.py AAPL 1
python send_command.py AAPL 1 MyCommand
```

To get your user Id and API token, see Request API Token .

Broadcast Commands by API

To broadcast a command to your live algorithms, send a payload through the QuantConnect REST API or LEAN CLI . Follow these steps to broadcast a command to all projects in your organization from the command line.

1. Install the `requests` library, if you haven't already.

```
pip install requests
```

2. Save the following code as `broadcast_command.py` .

```
from base64 import b64encode
from hashlib import sha256
from time import time
from sys import argv
from requests import post

# Edit these values:
USER_ID = 0
API_TOKEN = '_____'
ORGANIZATION_ID = '_____'

BASE_URL = 'https://www.quantconnect.com/api/v2/'

def get_headers():
    # Get timestamp
    timestamp = f'{int(time())}'
    time_stamped_token = f'{API_TOKEN}:{timestamp}'.encode('utf-8')

    # Get hased API token
    hashed_token = sha256(time_stamped_token).hexdigest()
    authentication = f'{USER_ID}:{hashed_token}'.encode('utf-8')
    authentication = b64encode(authentication).decode('ascii')

    # Create headers dictionary.
    return {
        'Authorization': f'Basic {authentication}',
        'Timestamp': timestamp
    }

def broadcast_command(command):
    return post(
        f'{BASE_URL}/live/commands/broadcast',
        headers=get_headers(),
        json={"organizationId": ORGANIZATION_ID, "command": command}
    ).json()

# Read the ticker, quantity, and (optional) command type from the command line:
ticker = argv[1]
quantity = int(argv[2])
command = {'ticker': ticker, 'quantity': quantity}
if len(argv) > 3:
    command['$type'] = argv[3]
print(broadcast_command(command))
```

3. Replace the placeholder values at the top of the file with your user Id, API token, and organization Id.
4. Run the script, passing the ticker and quantity as arguments. The first form below calls the `on_command` method; the second adds an optional third argument that sets the `$type` key to route to an encapsulated `Command` subclass handler.

```
python broadcast_command.py AAPL 1
python broadcast_command.py AAPL 1 MyCommand
```

To get your user Id, API token, and organization Id, see Request API Token and Get Organization Id .

Examples

The following examples demonstrate common practices for implementing live commands.

Example 1: Grey Box

The following algorithm trades a simple EMA cross strategy on SPY. However, instead of direct ordering by the algorithm as a "black box", it sends an email to ask for clicking a confirmation link before ordering, making each order decision an informed manual decision. The algorithm is called a "grey box".

```py
class LiveCommandAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)
        # Warm-up indicator for immediate readiness.
        self.warm_up_indicator(self.spy, self._ema, Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        bar = slice.bars.get(self.spy)
        if bar and self.live_mode:
            # Trend-following strategy using price and EMA.
            # If the price is above EMA, SPY is in an uptrend, and we buy it.
            # We sent a link to our email address and await confirmation.
            if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
                link = self.link({"ticker": "SPY", "size": 1})
                self.notify.email("email@address.com", "Trade Confirmation Needed", f"Click here to run: {link}")
            elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
                link = self.link({"ticker": "SPY", "size": -1})
                self.notify.email("email@address.com", "Trade Confirmation Needed", f"Click here to run: {link}")

    def on_command(self, data: DynamicData) -> Optional[bool]:
        # The algorithm will place the order if we click the email link to confirm the trade.
        self.set_holdings(data["ticker"], data["size"])
        return True
```

Example 2: Parent-Child Communication

Parent algorithm

The parent algorithm is connected to a brokerage. It receives commands from other algorithms and places market orders based on those commands.

```py
class ParentAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(100_000)

        # Remove benchmark so doesn't need asset data.
        self.set_benchmark(lambda x: 1)

        # Seed each asset with it's price to trade right away.
        self.settings.seed_initial_prices = True

    def on_command(self, payload):
        self.log(f'{self.time} - Execute on_command(): {payload}')
        equity = self.add_equity(payload.ticker)

        if payload.email:
            self.notify.email(address=payload.email, subject=f"Order Executed from {payload.source}", message=f"{equity.symbol},
{payload.quantity}, {payload.source}")

        self.market_order(equity.symbol, payload.quantity, asynchronous=True, tag=payload.source)
        return True
```

Child algorithm

The child algorithm is connected to a paper trading account. It contains the trading logic and sends commands to the parent algorithm for execution.

```py
class ChildAlgorithm (QCAlgorithm):
    def initialize(self):
        # Set the initials to identify the sender and the cash this algorithm will use.
        self._initials = 'C1'
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_cash(10_000)

    def on_order_event(self, order_event: OrderEvent):
        # When an order is filled, we send a command to the parent algorithm with the order details.
        if order_event.status == OrderStatus.FILLED:
            self.log(f'{self.time} - Sending command')
            response = send(self, order_event.symbol.value, order_event.fill_quantity, self._initials)
            self.log(f'{self.time} - Command response {response}')

def send(algorithm, ticker, quantity, name):
    if not algorithm.live_mode:
        return ''
    # Set the project ID with the ID of your parent algorithm.
    algorithm.project_id = 30123456
    # Generate the command link with the desired data payload, and execute with download_data.
    link = algorithm.link({'ticker': ticker, 'quantity': quantity, 'name': name})
    return Extensions.download_data(link)
```

Other Examples

For more examples, see the following algorithms:

Demonstration Algorithm
CallbackCommandRegressionAlgorithm.py Python

Live Trading

Arbitrage

Introduction

Arbitrage is the practice of taking advantage of a price difference between two or more markets. QuantConnect supports arbitrage strategies by allowing you to run multiple live algorithms deployments simultaneously and communicate between them using commands .

Basic Implementation

You can broadcast a command from and execute the event handler in another project. Add the project ID to the command data to identify the source of the command:

```py
class BasicBroadcastCommandAlgorithm(QCAlgorithm):
    def initialize(self):
        self.set_benchmark(lambda x: 0) # So the algorithm needs no asset data.
        broadcast_result = self.broadcast_command({"sender": self.project_id, "ticker": "AAPL", "quantity": 1})

    def on_command(self, data):
        # Receive the command from the other algorithm
        self.log(f'Got command at {self.time} with data: {data}')
        return True
```

Examples

The following examples demonstrate common practices for implementing arbitrage algorithms.

Example 1: Long-short Bitcoin in two exchanges

The following algorithms trade an arbitrage strategy with BTCUSDT.

Primary algorithm

The primary algorithm trades a simple EMA cross strategy on BTCUSDT in Kraken. It broadcasts a command with the order filled quantity to all live deployments, and logs the command data it receives from the secondary algorithm.

```py
class CryptoArbitrageA(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_account_currency('USDT', 100000)

        # Set the brokerage model to trade on Kraken.
        self.set_brokerage_model(BrokerageName.KRAKEN, AccountType.MARGIN)
        # Request BTCUSDT data to trade.
        self._btc = self.add_crypto("BTCUSDT").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self._btc, 22)
        # Warm-up indicator for immediate readiness.
        self.warm_up_indicator(self._btc, self._ema)
        # Trade when the EMA is updated.
        self._ema.updated += self.trade
        # Set a threshold value for the EMA cross.
        self._threshold = 0.005

    def trade(self, _, current: IndicatorDataPoint) -> None:
        ema = current.value
        holdings = self.portfolio[self._btc]
        price = holdings.security.price

        # Trend-following strategy using price and EMA.
        # If the price is above EMA, SPY is in an uptrend, and we buy it.
        if not holdings.is_long and price > ema * (1+self._threshold):
            self.set_holdings(self._btc, 1)
        elif not holdings.is_short and price < ema * (1-self._threshold):
            self.set_holdings(self._btc, -1)

    def on_command(self, data: DynamicData) -> Optional[bool]:
        # The algorithm will log the command data from the secondary algorithm.
        self.log(f'Got command at {self.time} with data: {data}')
        return True

    def on_order_event(self, order_event: OrderEvent) -> None:
        # If an order is filled, we call the broadcast_command method to send a command to all live deployments.
        # You can add the project ID to the command data to identigy the sender.
        if not self.live_mode or order_event.status != OrderStatus.FILLED:
            return
        broadcast_result = self.broadcast_command({"sender": self.project_id, "ticker": f"{order_event.symbol.value}", "quantity":
self.portfolio[order_event.symbol].quantity})
```

Secondary algorithm

The secondary algorithm trades the opposite direction of the primary in Bybit using the command it receives from the primary algorithm. It broadcasts a command with the order filled quantity to all live deployments.

```py
class CryptoArbitrageB(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 1)
        self.set_end_date(2024, 12, 31)
        self.set_account_currency('USDT', 100000)

        # Set the brokerage model to trade on Bybit.
        self.set_brokerage_model(BrokerageName.BYBIT, AccountType.MARGIN)
        # Request BTCUSDT data to trade.
        self._btc = self.add_crypto("BTCUSDT").symbol

    def on_command(self, data: DynamicData) -> Optional[bool]:
        # The algorithm will trade the opposite direction of the primary algorithm.
        self.log(f'Got command at {self.time} with data: {data}')
        quantity = -(self.portfolio[self._btc].quantity + float(data.quantity))
        if quantity != 0:
            self.market_order(self._btc, quantity, tag=str(data))
        return True

    def on_order_event(self, order_event: OrderEvent) -> None:
        # If an order is filled, we call the broadcast_command method to send a command to all live deployments.
        # You can add the project ID to the command data to identify the sender.
        if not self.live_mode or order_event.status != OrderStatus.FILLED:
            return
        broadcast_result = self.broadcast_command({"sender": self.project_id, "ticker": f"{order_event.symbol.value}", "quantity":
self.portfolio[order_event.symbol].quantity})
```

Live Trading

Charting and Logging

Introduction

The live results page shows your algorithm's live trading performance. You can add custom charts and logs to the results page.

Charting

Charts are sampled every one and ten minutes. If you create 1-minute resolution custom charts, the IDE charting will downgrade the granularity and display the 10-minutes sampling after a certain amount of samples. There are not quotas in live trading.

Logging

Algorithms can record string messages ('log statements') to a file for analysis after a backtest is complete, or as a live algorithm is running. These records can assist in debugging logical flow errors in the project code. Consider adding them in the code block of an `if` statement to signify an error has been caught.

It's good practice to add logging statements to live algorithms so you can understand its behavior and keep records to compare against backtest results. If you don't add logging statements to a live algorithm and the algorithm doesn't trade as you expect, it's difficult to evaluate the underlying problem.

If you run algorithms on QuantConnect, you must stay within the log quota . To only log when your algorithm is live, use the `live_mode` property.

```
PY
if self.live_mode:
    self.log("My log message")
```

Runtime Statistics

Runtime statistics show the performace of your algorithm at a single moment in time.

The following table describes the default runtime statistics:

Statistic	Description
Equity	The total portfolio value if all of the holdings were sold at current market rates. Equity equals the sum of cash and the market value of all open positions.
Fees	The total quantity of fees paid for all the transactions during the algorithm's trading period. Total fees include brokerage commissions, exchange fees, and other transaction costs.
Holdings	The absolute sum of the items in the portfolio. Holdings represent the total market value of all positions, regardless of whether they are long or short.
Net Profit	The dollar-value return across the entire trading period.
PSR	The probability that the estimated Sharpe ratio of an algorithm is greater than a benchmark.
Return	The rate of return across the entire trading period.
Unrealized	The amount of profit a portfolio would capture if it liquidated all open positions and paid the fees for transacting and crossing the spread. Unrealized profit becomes realized profit when the positions are closed.
Volume	The total value of assets traded for all of an algorithm's transactions during the trading period.

For more information about runtime statistics, see Runtime Statistics .

Examples

The following examples demonstrate some common practices for logging in live algorithm.

Example 1: On Events And Trading Logic

The following example trades a simple EMA cross trend-following strategy on SPY. We log or debug messages in event handlers and trading logic changes to get information on any event flow.

```py
class LoggingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2024, 9, 12)
        self.set_end_date(2024, 10, 1)
        self.set_cash(1000000)

        # Request SPY data to trade.
        self.spy = self.add_equity("SPY").symbol
        # Create an EMA indicator to generate trade signals.
        self._ema = self.ema(self.spy, 20, Resolution.DAILY)

        # Warm up the algorithm for indicator readiness.
        self.set_warm_up(20, Resolution.DAILY)

    def on_warmup_finished(self) -> None:
        # Signals on warm-up finished.
        self.log("Warm up finished")

    def on_data(self, slice: Slice) -> None:
        bar = None if self.is_warming_up else slice.bars.get(self.spy)
        if bar:
            # Trend-following strategy using price and EMA.
            # If the price is above EMA, SPY is in an uptrend, and we buy it.
            if bar.close > self._ema.current.value and not self.portfolio[self.spy].is_long:
                self.debug("Trend changes to upwards")
                self.set_holdings(self.spy, 1)
            elif bar.close < self._ema.current.value and not self.portfolio[self.spy].is_short:
                self.debug("Trend changes to downwards")
                self.set_holdings(self.spy, -1)

    def on_order_event(self, order_event: OrderEvent) -> None:
        if order_event.status == OrderStatus.FILLED:
            # Log the order details if being filled.
            self.log(f"Order filled - Quantity: {order_event.quantity}")
```

Live Trading

Signal Exports

QuantConnect Signal Exports allows you to collect distributed, crowd-sourced, algorithm algorithm's trading and portfolio positions to your fund or a third-party service..

Key Concepts

Getting started with Signal Exports

Collective2

Equities, Equity Options, Forex, Futures, Index Options

Numerai

Equities

vBase

See Also

Datasets

Signal Exports

Key Concepts

Introduction

QuantConnect Signal Exports allows you to transmit the state of an algorithm's trading and portfolio positions to your fund or a third-party service. QuantConnect implements two signal services out of the box: Collective2 and Numerai . To send portfolio targets to your fund, you need to create a custom Signal Export implementation.

Add Providers

To export signals to a signal provider, during initialization , add a signal export provider.

```
self.signal_export.add_signal_export_provider(CustomSignalExport())
```

You can add multiple signal export providers to a single algorithm.

Send Portfolio Targets

The signal export manager automatically sends signals when your portfolio holdings change to your fund or a third-party service. By default, it waits five seconds after an order is filled to aggregate all the state changes into a single post. Set the `automatic_export_time_span` property to change the waiting time.

```
self.signal_export.automatic_export_time_span = timedelta(seconds=1)
```

To send targets that aren't based on your portfolio holdings, see Manual Exports .

To disable automatic exports, set the `automatic_export_time_span` property to `None` .

```
self.signal_export.automatic_export_time_span = None
```

Custom Signal Exports

Signal export providers must implement the `ISignalExportTarget` interface. Extensions of the `ISignalExportTarget` interface must define the `send` and the `dispose` methods.

The `send` receives a `SignalExportTargetParameters` object with a list of `PortfolioTarget` objects you can use to create the API request payload.

The `dispose` method is called when the algorithm is disposed. You can use this method to release unmanaged resources such as HTTP Clients.

```py
class CustomSignalExportExampleAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # In the Initialize method, set the custom signal export provider
        self.add_equity("SPY")
        self.signal_export.add_signal_export_providers(CustomSignalExport())

    def on_data(self, slice: Slice) -> None:
        # Buy the SPY ETF when the algorithm is not invested
        if not self.portfolio.invested:
            self.set_holdings("SPY", 0.1)

# Define the custom signal export provider
from requests import post
class CustomSignalExport:
    def send(self, parameters: SignalExportTargetParameters) -> bool:
        # Create the request payload using the PortfolioTarget objects in the parameters
        # You can use the PortfolioTarget.Percent to calculate the target quantity
        # from the target percentage
        # The Tag property can be used to send additional information about the target
        # It's only available when you create the PortfolioTarget object with the tag parameter
        # See the Manual Exports documentation for more information
        targets = [PortfolioTarget.percent(parameters.algorithm, x.symbol, x.quantity, x.tag)
           for x in parameters.targets]
        data = [ {'symbol' : x.symbol.value, 'quantity': x.quantity, 'tag': x.tag} for x in targets ]

        # Send the request to your custom endpoint
        request_uri = "https://your-custom-endpoint.com/api/signal-export"
        response = post(request_uri, json = data)
        result = response.json()

        # Log the result of the request for debugging purposes
        success = result.get('success', False)
        parameters.algorithm.log(f"Send #{len(parameters.targets)} targets. Success: {success}")
        return success

    def dispose(self) -> None:
        pass
```

Manual Exports

In some cases, you may need to send signals to your fund or a third-party manually. For example, the signal export manager sends signals after a fill event and you need to submit signals when the market is closed or your signals aren't based on your portfolio holdings.

To do this, pass a `PortfolioTarget` object or a list of `PortfolioTarget` objects to the `set_target_portfolio` method. The method returns a boolean that represents if the targets were successfully sent to its destination. In this situation, the number you pass to the `PortfolioTarget` constructor represents the portfolio weight. Don't use the `PortfolioTarget.percent` method.

```py
target = PortfolioTarget(self._symbol, weight, tag="My tag")
success = self.signal_export.set_target_portfolio(target)
```

If you use a margin account, you can send your current portfolio holdings by calling the `set_target_portfolio_from_portfolio` method.

```py
success = self.signal_export.set_target_portfolio_from_portfolio()
```

Signal Exports

Collective2

Introduction

Collective2 is a service that connects strategy developers and capital allocators. With our Collective2 integration, you can send trading signals from your live algorithms to Collective2 .

Add Providers

To export signals to Collective2 from your algorithm, during initialization , add a Collective2 signal export provider.

```
self.signal_export.add_signal_export_provider(Collective2SignalExport(api_key, system_id, use_white_label_api))
```

The `Collective2SignalExport` constructor accepts the following arguments:

Argument: `api_key` Your Collective2 API key. To get your API key, see Where can I find my API key? on the Collective2 website. Data Type: `str`
Argument: `system_id` Your Collective2 strategy Id. To get your strategy Id, see How do I find my strategy ID? on the Collective2 website. Data Type: `int`
Argument: `use_white_label_api` Whether to use the white-label API instead of the general one. Data Type: `bool` | Default Value: `False`

You can add multiple signal export providers to a single algorithm.

Asset Classes

Our Collective2 integration supports signals for the following asset classes:

- US Equity
- Equity Options
- Forex
- Futures
- Index Options

Send Portfolio Targets

The signal export manager automatically sends signals when your portfolio holdings change to Collective2. By default, it waits five seconds after an order is filled to aggregate all the state changes into a single post. Set the `automatic_export_time_span` property to change the waiting time.

```
self.signal_export.automatic_export_time_span = timedelta(seconds=1)
```

To send targets that aren't based on your portfolio holdings, see Manual Exports .

To disable automatic exports, set the `automatic_export_time_span` property to `None` .

```
self.signal_export.automatic_export_time_span = None
```

PY

Examples

Demonstration Algorithms
Collective2SignalExportDemonstrationAlgorithm.py Python Collective2PortfolioSignalExportDemonstrationAlgorithm.py Python

Signal Exports

Numerai

Introduction

Numerai is a platform where you can earn rewards by uploading unique live trading signals for a universe of US Equities. The regular Numerai Tournament is built around a free dataset they provide , but with Numerai Signals , you can now use any dataset to generate your trading signals. Through our Numerai integration, you can use our rich library of alternative datasets or your own custom datasets to create and automate your Numerai Signal submissions.

Add Providers

To export signals to Numerai from your algorithm, during initialization , add a Numerai signal export provider.

```
self.signal_export.add_signal_export_provider(NumeraiSignalExport(public_id, secret_id, model_id, file_name))
```

The `NumeraiSignalExport` constructor accepts the following arguments:

Argument: `public_id` Your Numerai API key. To create and view your API keys, see the Automation section of your Account page on the Numerai website. Data Type: `str` | Default Value: `None`
Argument: `secret_id` Your Numerai API secret. You get your API secret when you create a new API key. Data Type: `str` | Default Value: `None`
Argument: `model_id` The Id of the Numerai model. Follow these steps to get the model Id: 1. Open the Models page on the Numerai website. 2. In the Models section, click the three dots next to a model. 3. Click Copy Model ID . Data Type: `str` | Default Value: `""`
Argument: `file_name` The name for the signal file. If you use a file name that already exists, the new file overwrites the old one. Data Type: `str` | Default Value: `"predictions.csv"`

You can add multiple signal export providers to a single algorithm.

Asset Classes

Our Numerai integration supports signals for US Equities .

Universe Selection

The Numerai Signals stock market universe covers roughly the top 5,000 largest stocks in the world. The universe available on QuantConnect that's the closest match to the Numerai Signals universe is the CRSP US Total Market Index, which represents approximately 100% of the investable US Equity market regularly traded on the New York Stock Exchange and Nasdaq. This Index doesn't contain all of the stocks in the Numerai Signals universe, but you don't need to submit signals for all the stocks in the Numerai Signals universe.

To get the constituents of the CRSP US Total Market Index, add an ETF constituents universe for the Vanguard Total Stock Market ETF, VTI.

```py
self.universe_settings.asynchronous = True
self.etf_symbol = self.add_equity("VTI").symbol
self.add_universe(self.universe.etf(self.etf_symbol))
```

Schedule Submissions

Every Tuesday, Wednesday, Thursday, Friday, and Saturday of the week, a new round is open for you to submit signals. Saturday rounds open at 18:00 UTC, and the submission window is open until Monday 14:30 UTC. Weekday rounds open at 13:00 UTC, and the submission window is open for 1 hour. For more information about the competition rounds, see Rounds in the Numerai documentation.

To schedule your submissions, you can set a Scheduled Event during initialization.

```py
self.schedule.on(
    self.date_rules.every_day(self.etf_symbol),
    self.time_rules.at(13, 0, TimeZones.UTC),
    self.submit_signals)
```

Send Portfolio Targets

To send targets, pass a list of `PortfolioTarget` objects to the `set_target_portfolio` method. The method returns a boolean that represents if the targets were successfully sent to Numerai Signals. In this situation, the number you pass to the `PortfolioTarget` constructor represents the portfolio weight. Don't use the `PortfolioTarget.percent` method.

```py
success = self.signal_export.set_target_portfolio(targets)
```

Signal Requirements

When you submit signals to Numerai, abide by the following rules:

- All signals must have a quantity between 0 and 1 (exclusive)
- You must submit at least 10 different signals per submission
- At least one of the weights in every batch of signals must be unique

Examples

Demonstration Algorithms
NumeraiSignalExportDemonstrationAlgorithm.py Python

Signal Exports

vBase

Introduction

validityBase enables you to create a globally credible, independently verifiable audit trail for trading track records, models and signals. This ensures anyone viewing your results can confirm that your results are point-in-time, complete, free from cherry-picking and fully out-of-sample.

QuantConnect users can use validityBase to publish live, verifiable, indices of their trading strategies and signals - increasing their credibility with allocators, clients and potential employers, and significantly increasing their commercial value. Integration is simple. With a single function call from your QuantConnect code, you can automatically build an immutable blockchain-backed audit trail for your signals and models.

Add Providers

To export signals to vBase from your algorithm, during initialization , add a vBase signal export provider.

```
self.signal_export.add_signal_export_provider(VBaseSignalExport(api_key, collection_name, store_stamped_file=True, idempotent=False))
```

The `VBaseSignalExport` constructor accepts the following arguments:

Argument: `api_key` Your vBase API key. You can find it in your profile Account Settings tab. Data Type: `str`
Argument: `collection_name` The name of the vBase collection to which the signals will be sent. You can create and manage collections in the vBase app Collections tab. Think of a collection like a strategy or dataset name. Data Type: `str`
Argument: `store_stamped_file` Whether to store a stamped copy of the signal in vBase. **Important** if this is marked "no", then you will be responsible for maintaining an exact copy of your stamped data. Losing or accidentally modifying stamped outputs will prevent your work from being verifiable. Data Type: `bool` | Default Value: `True`
Argument: `idempotent` Whether to make the signal export idempotent. If true, sending the same signal multiple times will only create one record in vBase. Data Type: `bool` | Default Value: `False`

You can add multiple signal export providers to a single algorithm.

Asset Classes

Our vBase integration supports all asset classes.

Send Portfolio Targets

To send targets, pass a list of `PortfolioTarget` objects to the `set_target_portfolio` method. The method returns a boolean value indicating whether the targets were successfully sent to vBase.

What gets stamped? The provider builds a `sym,wt` (symbol-weight) CSV directly from your portfolio targets. Each target's `Quantity` value is treated as the portfolio weight (percentage of holdings), so you should pass percentage-based targets (e.g., 0.25 for 25% of the portfolio). Default behavior can be customized by overriding the `BuildCsv` method of the `VBaseSignalExport` class.

```py
success = self.signal_export.set_target_portfolio(targets)
```

Examples

Demonstration Algorithms
VBaseSignalExportDemonstrationAlgorithm.py Python

Strategy Library

Introduction

The Strategy Library is a collection of tutorials written by the QuantConnect team and community members. Review these tutorials to learn about trading strategies found in the academic literature and how to implement them with QuantConnect/LEAN.

Tutorials

Search Strategy Tutorials

Strategy Name
CAPM Alpha Ranking Strategy on Dow 30 Companies Applies CAPM model to rank Dow Jones 30 companies.
Combining Mean Reversion and Momentum in Forex Market Combines momentum and mean reversion techniques in the forex markets.
Pairs Trading-Copula vs Cointegration Applies Copula and Cointergration method to pairs trading.
The Dynamic Breakout II Strategy A demonstration of dynamic breakout II strategy.
Dual Thrust Trading Algorithm A demontration of Dual Thrust Intraday strategy.
Can Crude Oil Predict Equity Returns Applies regression method to predict the return from the stock market and compare it to the short-term U.S. T-bill rate.
Intraday Dynamic Pairs Trading using Correlation and Cointegration Approach A high frequency pairs trading algorithm based on cointegration.
The Momentum Strategy Based on the Low Frequency Component of Forex Market Applies high frequency filter to the momentum strategy.
Stock Selection Strategy Based on Fundamental Factors MorningStar Fundamental factors universe selection algorithm.
Short-Term Reversal Strategy in Stocks A short term reversal algorithm which gives the opposite signal by analyzing recent period price action.

Fundamental Factor Long Short Strategy

A basic monthly rebalance long short algorithm based on fundamental factors.

Asset Class Trend Following

Selects ETFs over ten-month moving average and assigns an equally weighted allocation.

Source: Quantpedia

Asset Class Momentum

Selects ETFs in different asset classes with the highest momentum and assigns an equally weighted allocation.

Source: Quantpedia

Residual Momentum

Constructs a long/short portfolio based on trailing residual momentum normalized by its standard deviation

Source: Quantpedia

Sector Momentum

Selects ETFs in different sectors with the highest momentum and assigns an equally weighted allocation.

Source: Quantpedia

Overnight Anomaly

Buy SPY ETF at its closing price and sell it at the opening each day.

Source: Quantpedia

Forex Carry Trade

Goes long the currency with the highest central bank interest rate and goes short the currency with the lowest interest rate.

Source: Quantpedia

Volatility Effect in Stocks

Constructs equally weighted portfolios by selecting stocks with the lowest volatility in the past one year.

Source: Quantpedia

Forex Momentum

Goes long currencies with strongest 12 month momentum against USD and goes short currencies with the lowest 12 month momentum against USD.

Source: Quantpedia

Pairs Trading with Stocks

Looks for the security that minimizes the sum of squared deviations and long-short position is opened when pair prices have diverged by multiple of standard deviations.

Source: Quantpedia

Short Term Reversal

Goes long stocks with the lowest return in the previous month and goes short stocks with the greatest return from the previous month.

Source: Quantpedia

Momentum Effect in Stocks

Goes long stocks with the best 12-month momentum in the large-cap universe.

Source: Quantpedia

Momentum Effect in Country Equity Indexes

Goes long stocks with the best 12-month momentum in the country equity indexes ETFs.

Source: Quantpedia

Mean Reversion Effect in Country Equity Indexes

Goes long country equity indexes ETFs with the worst 36-month return and short ETFs with the best 36-month return.

Source: Quantpedia

Liquidity Effect in Stocks

Goes long stocks with the lowest turnover and short on stocks with the highest turnover from the lowest market-cap quartile.

Source: Quantpedia

Volatility Risk Premium Effect

Sells at-the-money straddle with one month until maturity and buys an offsetting 15% out-of-the-money puts each month.

Source: Quantpedia

Momentum Effect in Commodities Futures

Goes long commodity futures with the highest momentum and short on futures with the lowest momentum.

Source: Quantpedia

Small Capitalization Stocks Premium Anomaly

Goes long stocks with the lowest market capitalization and rebalances the portfolio once a year.

Source: Quantpedia

Paired Switching

Goes long asset with better performance over the last period and rebalances portfolio every quarter.

Source: Quantpedia

Term Structure Effect in Commodities

Buys each month the 20% of commodities with the highest roll-returns and shorts the 20% of commodities with the lowest roll-returns and holds the long-short positions for one month.

Source: Quantpedia

Momentum Effect Combined with Term Structure in Commodities

Portfolios are formed based on roll returns and the algorithm goes long and short contracts with the highest and lowest one-month performance.

Source: Quantpedia

Book-to-Market Value Anomaly

Quintile portfolios are formed based on the Book-to-Market ratio and the highest quintile is held for one year.

Source: Quantpedia

Gold Market Timing

Goes long gold when the Fed model shows that the market is undervalued (the earnings yield is higher than the bond yield and their ratio is at least 2).

Source: Quantpedia

Turn of the Month in Equity Indexes

Buys SPY the day before the end of the month and liquidates position on 3rd trading day of new month.

Source: Quantpedia

Momentum - Short Term Reversal Strategy

Goes long stocks with the decreasing return from the winner group and short stocks with the increasing return from the loser group.

Source: Quantpedia

Pairs Trading with Country ETFs

Identifies the price divergence from two highly correlated country ETFs and takes a market neutral position.

Source: Quantpedia

Sentiment and Style Rotation Effect in Stocks

Creates long-short positions of growth and value stocks based on the investment sentiment.

Source: Quantpedia

Asset Growth Effect

Creates long-short positions of stocks based on the annual change of their total assets.

Source: Quantpedia

Momentum and State of Market Filters

Goes long and short stocks with the highest and lowest six-month momentum respectively if the previous 12 months return on the broad market index was positive.

Source: Quantpedia

Accrual Anomaly

Decile portfolios are formed based on balance sheet based accruals and highest decile is shorted while lowest decile is bought for a year.

Source: Quantpedia

Momentum in Mutual Fund Returns

Forms a long-short portfolio of asset management firms based on trailing rate of change and nearness to trailing high.

Source: Quantpedia

Momentum and Style Rotation Effect

Goes long style index ETF with the highest 12-month momentum and short ETF with the lowest 12-month momentum.

Source: Quantpedia

Trading with WTI BRENT Spread

Goes long the spread if the spread is below 20-day moving average and short if the spread is above 20-day moving average.

Source: Quantpedia

Momentum Effect in REITs

Tercile portfolios are formed based on momentum and the best performing portfolio is held.

Source: Quantpedia

Option Expiration Week Effect

Goes long S&P 100 index ETF during option expiration week and stays in cash during other days.

Source: Quantpedia

Earnings Quality Factor

Goes long stocks with high earnings quality and short stocks with low earnings quality based on composite factor score.

Source: Quantpedia

January Effect in Stocks

Invests into small cap stocks at the beginning of each January and stays invested in large cap stocks for rest of the year.

Source: Quantpedia

Momentum and Reversal Combined with Volatility Effect in Stocks

Goes long on stocks from the highest performing quintile from the highest volatility group and short on stocks from the lowest performing quintile from the highest volatility group.

Source: Quantpedia

ROA Effect within Stocks

Goes long on stocks with highest ROA and short stocks with the lowest ROA from each market capitalization group.

Source: Quantpedia

January Barometer

Invested in equity market with ETF only if January return is positive otherwise switch investments to T-Bills.

Source: Quantpedia

Lunar Cycle in Equity Market

Goes long in emerging market index ETF 7 days before the new moon and switch to a short position on emerging market index ETF 7 days before the full moon.

Source: Quantpedia

VIX Predicts Stock Index Returns

Goes long on equity index ETF if the VIX is in the highest percentile short if VIX is in the lowest percentile in the last two-year history.

Source: Quantpedia

Combining Momentum Effect with Volume

Goes long stocks with the highest volume from the top momentum decile and short stocks with the highest volume from the bottom momentum decile.

Source: Quantpedia

Short Term Reversal with Futures

Goes long (short) on futures from the high-volume, low-open interest group with the lowest (greatest) returns in the previous week.

Source: Quantpedia

Pre-holiday Effect

Invests in equity market 2 days preceding holiday days and stays in cash during the other trading days.

Source: Quantpedia

Beta Factors in Stocks

Goes long stocks with the bottom beta and short stocks with the top beta, securities are weighted by the ranked betas.

Source: Quantpedia

Exploiting Term Structure of VIX Futures

Buys or sells the nearest VIX futures based on the daily roll and hedge against the open positions with E-mini S&P500 futures.

Source: Quantpedia

12 Month Cycle in Cross-Section of Stocks Returns

Reviews the returns from last January, going long on the top 10% winners and short the bottom 10%.

Source: Quantpedia

Momentum Effect in Stocks in Small Portfolios

Goes long in the 10 stocks with the highest performance and goes short in the 10 stocks with the lowest performance in the previous one year.

Source: Quantpedia

Value Effect within Countries

Invests in the cheapest 33% of country ETFs according to CAPE ratios.

Source: Quantpedia

Beta Factor in Country Equity Indexes

Goes long on the low-beta portfolio and short on the high-beta portfolio in country indexes ETFs.

Source: Quantpedia

Price to Earnings Anomaly

Invests in stocks with low P/E ratio.

Fama French Five Factors

Stock selecting strategy based on Fama-French Five Factors Model.

Source: NYU

Mean-Reversion Statistical Arbitrage Strategy in Stocks

Apply statistical arbitrage to take advantage of pricing inefficiencies in stocks.

Source: NYU

Expected Idiosyncratic Skewness

Stock selection strategy that calculates expected idiosyncratic skewness using Fama-French three-factor model, sorts stocks based on the calculated skewness, and longs the bottom 5%.

Source: NYU

Risk Premia in Forex Markets

A strategy based on asymmetric tail risks and excess returns in forex markets.

Source: NYU

Seasonality Effect based on Same-Calendar Month Returns

A strategy that takes long and short positions based on historical same-calendar month returns

Source: NYU

Standardized Unexpected Earnings

Stock selection strategy that calculates the unexpected earnings, standardizes the unexpected earnings, goes long on the top 5%, and rebalances the portfolio monthly.

Source: NYU

Price and Earnings Momentum

A momentum strategy based on quarterly returns and earnings growth

Source: NYU

Improved Momentum Strategy on Commodities Futures

An advanced momentum strategy that modifies the basic momentum strategies by introducing Baltas and Kosowski weights and rebalances the portfolio monthly. The new weighing scheme incorporates trend strength into the trading signal, uses an efficient volatility estimator, and adds a dynamic leverage mechanism.

Source: NYU

Commodities Futures Trend Following

A simple trend following strategy on commodities futures.

Source: NYU

Forecasting Stock Prices using a Temporal CNN Model

Applying a Temporal Convolutional Neural Network to forecasting future stock prices.

Source: Tampere University

Leveraged ETFs with Systematic Risk Management

We apply Simple Moving Averages to manage the risk of holding leveraged ETFs in an attempt to beat the S&P500

Source: The Lead-Lag Report

Ichimoku Clouds in the Energy Sector

A techincal indicator crossover strategy trading the largest energy companies.

Source: SSRN

Intraday ETF Momentum

A momentum strategy based on returns of the market open

Source: NYU

Intraday Arbitrage Between Index ETFs

A strategy that tracks the price paths of two correlated ETFs and takes advantage of mis-pricings that arise when the price paths diverge

Source: SSRN

Optimal Pairs Trading

Mathematically Deriving the Optimal Entry and Liquidation Values of a Pairs Trading Process

Source: arXiv

G-Score Investing

Applying G-Score Investing to Invest in a Portfolio of Technology Stocks

Source: SSRN

SVM Wavelet Forecasting

Forecasting EURJPY prices with an SVM Wavelet model

Source: Academia

Gradient Boosting Model

Forecasts future intraday returns with a gradient boosting model trained on technical indicators

Source: arXiv

Using News Sentiment to Predict Price Direction of Drug Manufacturers

Analyzes the news releases of drug manufacturers and places intraday trades for the stocks with positive news.

Source: arXiv

Gaussian Naive Bayes Model

Forecasts the next day's return of technology stocks by fitting a gaussian naive bayes model to the historical returns of the technology sector constituents.

Source: Academia

API Reference

The QuantConnect API Reference is a comprehensive technical document that details every class, method, property, and event available in the LEAN algorithmic trading framework. It serves as the definitive source for understanding how to interact with LEAN, helping you quickly look up syntax and explore available features. The LEAN engine is written in C#, but you can create algorithms with either C# or Python since LEAN uses Python.Net to bridge between your Python code and the underlying C# engine. For your convenience, we provide an API Reference for both languages.

Python

C#

Migrations

Migrations

Zipline

The following pages explain how to migrate trading algorithms from Zipline to LEAN:

Initialization

Using Data

Ordering

Logging and Plotting

Quick Reference

Zipline

Initialization

Initializing State

On QuantConnect, algorithm initialization behavior is equivalent to Quantopian's `initialize` method. `initialize` is the method equivalent, and it is only ran once at the start of the algorithm. It is responsible for initially adding data to the algorithm, adding a universe, setting the brokerage, setting the algorithm's start and end date, etc.

The algorithm structure in QuantConnect differs from Quantopian's as well. On QuantConnect, we create a class that derives from `QCAlgorithm` and define our overrides/algorithm functionality there. An example of an algorithm with initialization is provided.

Quantopian

```
def initialize(context):
    # Adds AAPL sid to `context` for use later in `handle_data(...)`
    context.aapl = sid(24)
```

QuantConnect

```
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        # Adds minutely AAPL data to the algorithm
        self.aapl = self.add_equity("AAPL", Resolution.MINUTE)
```

On QuantConnect, the `self` parameter is the equivalent of the `context` parameter in Quantopian algorithms. You can use `self` to access algorithm resources, such as the Portfolio, Orders, Order Tickets, Securities, place orders in your algorithm, and much more.

You can read more about algorithm initialization at the Initializion documentation page.

Attaching Pipelines

The functional equivalent of Pipelines on QuantConnect are Universes. Universes can be used to dynamically select assets your algorithm wants to trade or request data from. To add a universe to your algorithm, you can call the `AddUniverse()` method and provide the required Coarse and Fine universe selection functions.

Coarse Universe selection is the first step of universe selection. It provides a means to filter data in a lightweight fashion before proceeding to Fine Universe selection. Coarse universe will allow you to filter assets based off top-level fundamental factors, such as Dollar Volume for the day, if a company has fundamental data, and market of the asset.

Fine Universe selection is the second step of universe selection. It provides fundamental data to your user-defined function, and can be used to filter in greater detail based on an asset's fundamental data, such as earnings, EPS, etc.

An example of coarse and fine universe selection to filter assets in our algorithm is provided. Adding universes can be used as an alternative to using `add_equity` if you prefer dynamic asset selection.

```
class MyUniverseAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2020, 2, 1)

        self.universe_settings.resolution = Resolution.MINUTE
        self.add_universe(self._coarse_selection_function, self._fine_selection_function)

    def _coarse_selection_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        return [i.symbol for i in coarse if i.dollar_volume > 500000 and i.has_fundamental_data]

    def _fine_selection_function(self, fine: List[FineFundamental]) -> List[Symbol]:
        return [i.symbol for i in fine if i.earning_reports.basic_eps.one_month > 0]
```

PY

You can configure universe settings by modifying the existing `UniverseSettings` attribute of the algorithm.

```
class MyUniverseAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.universe_settings.resolution = Resolution.MINUTE
        self.universe_settings.data_normalization_mode = DataNormalizationMode.ADJUSTED
        self.universe_settings.leverage = 1.0
        self.universe_settings.extended_market_hours = False
```

PY

You can read more about universe selection at the Universe documentation page.

Scheduling Functions

Scheduled Events on QuantConnect are the equivalent of `schedule_function()` from Quantopian. Examples are provided comparing the Scheduled Events functionality between QuantConnect and Quantopian.

Quantopian

```
import quantopian.algorithm as algo

def my_scheduled_function(context, data):
    pass

def initialize(context):
    # Runs a function every day 60 minutes before U.S. Equities market close
    algo.schedule_function(
        func=my_scheduled_function,
        date_rule=algo.date_rules.every_day(),
        time_rule=algo.time_rules.market_close(minutes=60),
        calendar=algo.calendars.US_EQUITIES
    )
```

PY

QuantConnect

```
class MyAlgorithm(QCAlgorithm):
    def my_scheduled_function(self):
        pass

    # Runs a function every day 60 minutes before SPY ETF's market close
    def initialize(self) -> None:
        self.schedule.on(
            self.date_rules.every_day(),
            self.time_rules.before_market_close("SPY", 60),
            self.my_scheduled_function
       )
```

PY

You can read more about scheduled events at the Scheduled Events documentation page.

Setting Slippage and Commissions

QuantConnect supports applying slippage and commissions per individual security.

To add a slippage model to a security, we must set the `SlippageModel` property of the `Security` object to our model. A `Security` is returned when data is added to the algorithm. Examples of setting slippage in an algorithm is provided below for both Quantopian and QuantConnect.

Quantopian

```
PY
import quantopian.algorithm as algorithm
from zipline.finance.slippage import FixedSlippage


    def initialize(context):
        # This will set slippage for all U.S. equities
        algorithm.set_slippage(us_equities=FixedSlippage(0.01))
```

QuantConnect

```
PY
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        spy = self.add_equity("SPY", Resolution.MINUTE)
        # `ConstantSlippageModel` is the same as `FixedSlippage` on Quantopian.
        # This will set slippage only for the "SPY" security.
        spy.slippage_model = ConstantSlippageModel(0.01)
```

Commissions

To add a fee model (otherwise known as Commissions on Quantopian) to a security, we must set the `FeeModel` property of the `Security` object to our model. Examples are provided below for Quantopian and QuantConnect.

Quantopian

```
PY
import quantopian.algorithm as algorithm
from zipline.finance.commission import PerShare


    def initialize(context):
        # Approximates Interactive Brokers' equities trading commissions.
        algorithm.set_commission(us_equities=Pershare(cost=0.0075))
```

QuantConnect

```
PY
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
       aapl = self.add_equity("AAPL", Resolution.MINUTE)
       spy = self.add_equity("SPY", Resolution.MINUTE)

       # Sets the fee model to the IB fee model, which simulates
       # the fees we would encounter in live trading.
       spy.fee_model = InteractiveBrokersFeeModel()

       # Sets the fee model to a constant fee model
       aapl.fee_model = ConstantFeeModel(0.01, "USD")
```

You can read more about Securities and their attributes at the Securities and Portfolio and Reality Modeling documentation pages.

Manual Asset Lookup

On QuantConnect, we provide the `Symbol` class to reference a security across time with a changing ticker. This is similar to Quantopian's `symbol()` function, which enables the same functionality.

When you add data to your algorithm, a `Security` object is returned. The `Symbol` is accessible from the `Security` as shown below.

```
PY
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2000, 1, 1)
        self.set_end_date(2020, 1, 1)
        # TWX will change to AOL when backtesting in the early 2000s, and will
        # change to TWX as the backtest advances past the date it was renamed.
        twx_security = self.add_equity("TWX", Resolution.MINUTE)
        twx_symbol = twx_security.symbol
```

To manually create a reference to an old equity, you can use the `Symbol.create()` method.

You can read more about Symbols and Security Identifiers (SID) at the Security Identifiers documentation page.

Zipline

Using Data

Pipeline in Algorithms

As covered in the Initialization section, Pipelines can be replicated using Universe Selection in QuantConnect, albeit with some additional steps in between and a performance impact.

In this section, we will construct and define an equivalent pipeline model using universe selection in QuantConnect. We will filter our data set in Coarse and Fine, and apply an additional filter to historical data with a rolling window.

We first create a skeleton algorithm definition to begin setting up our Pipeline. Note that this algorithm will allow all equities through, which will have a substantial performance impact on our algorithm.

```py
class MyPipelineAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2020, 10, 20)

        self.add_universe(self.coarse_selection_function, self.fine_selection_function)

    def coarse_selection_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        # Allows all Symbols through, no filtering applied
        return [coarse_data.symbol for coarse_data in coarse]

    def fine_selection_function(self, fine: List[FineFundamental]) -> List[Symbol]:
        # Allows all Symbols through, no filtering applied
        return [fine_data.symbol for fine_data in fine]
```

The skeleton algorithm is the equivalent of the Pipeline call below.

```py
from quantopian.pipeline import Pipeline
from quantopian.pipeline.domain import US_EQUITIES
from quantopian.research import run_pipeline

pipe = Pipeline(columns={}, domain=US_EQUITIES)
run_pipeline(pipe, '2020-01-01', '2020-10-20')
```

The equivalent of `Pipeline(screen=...)` resolves to the filter applied at the Coarse and Fine stages of universe selection. Let's define a filter of stocks with a dollar volume greater than $50000000 USD, as well as a rolling thirty day return greater than 2%. Once we've initially filtered the Symbols in Coarse Universe Selection, let's define a final filter only allowing stocks with EPS greater than 0. Beware of making `History()` calls with many Symbols. It could potentially cause your algorithm to run out of system resources (i.e. RAM) and reduce performance of your algorithm on universe selection.

```py
from datetime import datetime, timedelta

class MyPipelineAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2020, 10, 20)

        self.add_universe(self._coarse_selection_function, self._fine_selection_function)

    def _coarse_selection_function(self, coarse: List[CoarseFundamental]) -> List[Symbol]:
        # Allows all Symbols through, no filtering applied
        dollar_volume_filter_symbols = [coarse_data.symbol for coarse_data in coarse if coarse_data.dollar_volume > 50000000]

        # Make a history call to calculate the 30 day rolling returns
        df = self.history(dollar_volume_filter_symbols, self.time - timedelta(days=60), self.time, Resolution.DAILY)

        # Compute the rolling 30 day returns
        df = df['close'].groupby(level=0).filter(lambda x: len(x) >= 30).groupby(level=0).apply(lambda x: (x.iloc[-1] - x.iloc[-30]) /
x.iloc[-30])

        # Finally, apply our filter
        dataframe_screen = df[df > 0.02]

        # Filters out any Symbol that is not in the DataFrame
        return [s for s in dollar_volume_filter_symbols if str(s) in dataframe_screen]

    def _fine_selection_function(self, fine: List[FineFundamental]) -> List[Symbol]:
        # We receive the filtered symbols from before, and we filter by EPS > 0 in this step
       return [s.symbol for s in fine if s.earning_reports.basic_eps.three_months > 0]
```

This class definition is now roughly equivalent to the following `CustomFactor` and Pipeline call in Quantopian, excluding the EPS filtering.

```py
from quantopian.pipeline.filters import QTradableStocksUS
from quantopian.pipeline.factors import AverageDollarVolume

class PercentageChange(CustomFactor):
    def compute(self, today, assets, out, values):
        out[:] = (values[-1] - values[0]) / values[0]

    dollar_volume = AverageDollarVolume(window_length=5)
    dollar_volume_filter = (dollar_volume > 50000000)

    pipe = Pipeline(
        columns={
            "percent_change": PercentageChange(inputs=[USEquityPricing.close], window_length=30)
        },
        screen=(QTradableStocksUS() & dollar_volume_filter)
    )
```

An example of the shape of the DataFrame returned from History is shown below. The DataFrame has a MultiIndex, with `level=0` being the Symbol, and `level=1` being the Time for that point of data. You can index the Symbol/ `level=0` index by using either the SecurityIdentifier string (e.g. `df.loc["AAPL R735QTJ8XC9X"]`) or with the ticker of the Symbol (e.g. `df.loc["AAPL"]`)

		close	high	low	open	volume
symbol	time					
IBM	2019-04-26	138.62	139.82	137.70	139.82	1419038.0
	2019-04-27	139.44	139.89	138.81	139.28	1275702.0
AAPL	2019-04-26	205.24	207.76	205.12	206.84	10105760.0
	2019-04-27	204.29	205.00	202.12	204.83	9984854.0

BarData Lookup

Similar but different, the Quantopian `BarData` object, and the QuantConnect `Slice` object both provide data to the user's algorithm as point-in-time data.

In Quantopian, data is handled via the `handle_data(context, data)` function. In QuantConnect, data is handled via the `OnData(self, slice)` method. Both of these functions accept data whenever new data exists for a given point in time. Although these two functions share the same method signature, the handling of the data is different.

`BarData` vs. `Slice`

`BarData` is the primary mechanism to retrieve the point-in-time data, as well as requesting history for any given securities in Quantopian. The

following code retrieves daily historical data from 30 days into the past, as well as getting the most recent data for AAPL at the current point-in-time.

Quantopian

```py
def initialize(context):
    context.aapl = sid(24)

def handle_data(context, data):
    # Gets a DataFrame of AAPL history going back 30 days
    aapl_history = data.history(context.aapl, fields=["open", "high", "low", "close", "volume"], 30, "1d")
    # Gets a pandas Series of the most recent AAPL data
    aapl_current = data.current(context.aapl, fields=["open", "high", "low", "close", "volume"])
```

QuantConnect

```py
from datetime import timedelta

class MyHistoryAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.aapl = self.add_equity("AAPL", Resolution.DAILY)

    def on_data(self, slice: Slice) -> None:
        # Gets a DataFrame of AAPL history going back 30 days
        aapl_history = self.history([self.aapl.symbol], timedelta(days=30), Resolution.DAILY)

        # Gets the most recent AAPL Trade data (OHLCV)
        aapl_current = slice.bars[self.aapl.symbol]
```

`Slice` represents a single point in time, and does not provide the functionality to access historical data itself. To access historical data in an algorithm, use the algorithm's `self.History()` call to request a pandas DataFrame of historical data. In `Slice` , the data that is accessed is not a pandas DataFrame, but rather a single object containing data for a single point in time.

The `TradeBar` class, for example, only contains scalar values of OHLCV, rather than return a DataFrame of OHLCV values. Since the data `Slice` contains is point-in-time, there will be only a single trade/quote bar per Symbol whenever `OnData(self, data)` is called.

QuantConnect provides Quote (NBBO) data for use in your algorithm, otherwise known as a `QuoteBar` . Quote data is only accessible when an algorithm is set to consume Tick, Second, or Minutely data.

You can access Trade (OHLCV) data by accessing the `Bars` attribute of `Slice` . You can access Quote (Bid(OHLCV), Ask(OHLCV)) data by accessing the `QuoteBars` attribute of `Slice` .

Both of the `Bars` and `QuoteBars` attributes are similar to Python dictionaries, and can be used as such. To check to see if there exists a new piece of data for a given security, you can use Python's `in` operator on `Bars` and or `QuoteBars` . You can also choose to iterate on all data received by calling the `Values` attribute of the `Bars` or `QuoteBars` attributes, which will return either a list of `TradeBar` or `QuoteBar` objects.

The `TradeBar` object contains the `Open` , `High` , `Low` , `Close` , `Volume` , `time` , `end_time` , and `Symbol` attributes. The `QuoteBar` object contains the following attributes:

- `Bid.Open` , `Bid.High` , `Bid.Low` , `Bid.Close` , `LastBidSize`
- `Ask.Open` , `Ask.High` , `Ask.Low` , `Ask.Close` , `LastAskSize`
- `time` , `end_time` , and `Symbol` .

Note that the `Bid` and `Ask` attributes can potentially be `None` if no bid/ask data exists at a given point-in-time.

The example below shows the different ways to access `TradeBar` and `QuoteBar` data, as well as requesting 30 days of AAPL historical data.

PY

```
from datetime import datetime, timedelta


class MyDataAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.aapl_security = self.add_equity("AAPL", Resolution.DAILY)
        self.aapl_symbol = self.aapl_security.symbol

    def on_data(self, slice: Slice) -> None:
        # Gets 30 days of AAPL history
        aapl_history = self.history([self.aapl_symbol], timedelta(days=30), Resolution.DAILY)

        # We must first check to make sure we have AAPL data, since this point-in-time
        # might not have had any trades for AAPL (this is in case you trade a low
        # liquidity asset. The data can potentially be missing for a point-in-time).
        if self.aapl_symbol in slice.bars:
            aapl_current_trade = slice.bars[self.aapl_symbol]
            self.log(f"{self.time} :: TRADE :: {self.aapl_symbol} - O: {aapl_current_trade.open} H: {aapl_current_trade.high} L:
{aapl_current_trade.low} C: {aapl_current_trade.close} V: {aapl_current_trade.volume}")

        # Check to make sure we have AAPL data first, since there might not have
        # been any quote updates since the previous for AAPL (this is in case you trade
        # a low liquidity asset. The data can potentially be missing for a point-in-time).
        if self.aapl_symbol in slice.quote_bars:
            aapl_current_quote = slice.quote_bars[self.aapl_symbol]
            if aapl_current_quote.bid is not None:
                bid = aapl_current_quote.bid
                self.log(f"{self.time} :: QUOTE :: {self.aapl_symbol} - Bid O: {bid.open} Bid H: {bid.high} Bid L: {bid.low} Bid C:
{bid.close} Bid size: {aapl_current_quote.last_bid_size}")

            if aapl_current_quote.ask is not None:
                ask = aapl_current_quote.ask
                self.log(f"{self.time} :: QUOTE :: {self.aapl_symbol} - Ask O: {ask.open} Ask H: {ask.high} Ask L: {ask.low} Ask C:
{ask.close} Ask size: {aapl_current_quote.last_ask_size}")
```

Zipline

Ordering

Placing Orders

Both Quantopian and QuantConnect offer several methods for placing orders. Both platforms have `MarketOrder` , `LimitOrder` , `StopOrder` , and `stop_limit_order` classes to create specific order types, although the `StopOrder` order is named `StopMarketOrder` on our platform. On Quantopian, creating these orders is done by passing a `style` argument to the `order` method.

```py
from zipline.finance.execution import (
    LimitOrder,
    MarketOrder,
    StopLimitOrder,
    StopOrder,
)

def initialize(context):
    context.stock = sid(8554)
    context.ordered = False

def handle_data(context, data):
    if not context.ordered:
        close = data.current(context.stock, 'close')
        order(context.stock, 10, style=MarketOrder())
        order(context.stock, 10, style=LimitOrder(limit_price=close * 0.9))
        order(context.stock, -10, style=StopOrder(stop_price=close * 0.85))
        order(context.stock, -10, style=StopLimitOrder(limit_price=close * 0.75, stop_price=close * 0.85))
        context.ordered = True
```

On QuantConnect, the same orders can be created with

```py
class MyAlgorithm(QCAlgorithm):

    def initialize(self) -> None:
        self.set_start_date(2020, 3, 1)
        self.set_cash(100000)
        self._symbol = self.add_equity("SPY", Resolution.MINUTE).symbol
        self.ordered = False

    def on_data(self, slice: Slice) -> None:
        if not self.ordered and slice.contains_key(self._symbol):
            close = slice[self._symbol].close
            self.market_order(self._symbol, 10)
            self.limit_order(self._symbol, 10, close * 0.9)
            self.stop_market_order(self._symbol, -10, close * 0.85)
            self.stop_limit_order(self._symbol, -10, close * 0.85, close * 0.75)
            self.ordered = True
```

In addition to the order types above, QuantConnect has several other order types available. Refer to our Trading and Orders documentation for a comprehensive list.

Quantopian's `order_optimal_portfolio` method computes the optimal portfolio weights using an objective and constraints, then places the orders to achieve the desired portfolio. For example, the code below uses the Quantopian API to create an equal-weighted dollar-neutral portfolio.

```py
import quantopian.algorithm as algo
import quantopian.optimize as opt

objective = opt.TargetWeights(weights)
constraints = [
    opt.MaxGrossExposure(1.0),
    opt.DollarNeutral()
]
algo.order_optimal_portfolio(objective, constraints)
```

To acheive the same functionality with QuantConnect, we utilize portfolio construction models . For instance, to create an equal-weighted dollar-neutral portfolio, we could define the following portfolio construction model and attach it to the algorithm.

```
class MyAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.set_start_date(2020, 5, 1)
        self.set_cash(100000)

        self.set_portfolio_construction(EqualWeightedDollarNeutralPortfolioConstructionModel())


class EqualWeightedDollarNeutralPortfolioConstructionModel(PortfolioConstructionModel):

    def determine_target_percent(self, activeInsights: List[Insight]):
        result = {}

        longs = 0
        shorts = 0
        for insight in activeInsights:
            if insight.direction == InsightDirection.UP:
                longs += 1
            elif insight.direction == InsightDirection.DOWN:
                shorts += 1
            result[insight] = 0

        if longs == 0 or shorts == 0:
            return result

        for insight in activeInsights:
            if insight.direction == InsightDirection.UP:
                result[insight] = 0.5 / longs
            elif insight.direction == InsightDirection.DOWN:
                result[insight] = -0.5 / shorts
        return result
```

When algorithms require manual control of their orders, both Quantopian and QuantConnect have lower-level ordering methods. Quantopian's `order` method, which places an order for a fixed number of shares, directly maps to our `Order` method. Additionally, Quantopian's `order_target_percent` method places an order to adjust a position to a target percent of the current portfolio value. Here's an example use case of these methods on the Quantopian platform.

```
order_target_percent(sid(8554), 10) # Allocate 10% of portfolio
order(sid(8554), 10) # Order 10 shares
```

On QuantConnect, these same orders can be placed using the `set_holdings` and `Order` methods.

```
self.set_holdings(symbol, 0.1) # Allocate 10% of portfolio
self.order(symbol, 10) # Order 10 shares
```

Quantopian and QuantConnect don't have equivalents for all the ordering techniques, although we can create a workaround for most situations. For instance, Quantopian's `order_percent` method places an order in the specified asset corresponding to the given percent of the current portfolio value.

```
order_percent(sid(8554), 10) # Allocate 10% more of portfolio
```

To accomplish this on QuantConnect, we can determine the weight of the asset in the `portfolio` , then manually determine the new target percent to pass to `set_holdings` .

```
current_weight = self.portfolio[symbol].holdings_value / self.portfolio.total_portfolio_value
self.set_holdings(symbol, current_weight + 0.1) # Allocate 10% more of portfolio
```

The Porfolio Object

Quantopian's `portfolio` class provides read-only access to the current portfolio state. The state includes starting cash, current cash, portfolio value, and positions. The `portfolio` object is accessed through the `context` object, as seen below.

```py
context.portfolio.starting_cash     # Amount of cash in the portfolio at the start of the backtest
                 .cash              # Amount of cash currently held in portfolio
                 .portfolio_value   # Current liquidation value of the portfolio's holdings
                 .positions         # Dict-like object for currently-held positions
```

All of this information, and more, is attainable on QuantConnect by using our `portfolio` object. Listed below is an example of doing so. Although there isn't a property for starting cash, this value can be saved during Initialization to be referenced later.

```py
self.portfolio.cash                 # Sum of all currencies in account (only settled cash)
              .total_portfolio_value   # Portfolio equity
              .keys                 # Collection of Symbol objects in the portfolio
              .values               # Collection of SecurityHolding objects in the portfolio
```

On Quantopian, by iterating through the `positions` dictionary, we can access information about currently-held positions. This includes the number of shares held, the last traded price & date of the asset, and the position's cost basis.

```py
for sid, position in context.portfolio.positions.iteritems():
    position.amount           # Number of shares in the position
            .cost_basis       # Volume weighted average price paid per share
            .last_sale_price  # Price at last sale of the asset on the exchange
            .last_sale_date   # Date of last sale of the asset on the exchange
```

Since the `portfolio` on QuantConnect is a `Dictionary<Symbol, SecurityHolding>` , we can get information on current positions by indexing the `portfolio` object with a `Symbol` or ticker. The `SecurityHolding` object that is returned contains information related to a single security in the portfolio. For instance, we have

```py
self.portfolio[symbol].quantity                # Number of shares held
                      .average_price            # Average price of shares held (aka cost basis)
                      .price                   # Last traded price of the security
```

For more information on our `portfolio` object refer to the Securities and Portfolio documentation.

Zipline

Logging and Plotting

Logging

In QuantConnect, the `self.Debug()` method is the equivalent of the `log.info()` function in Quantopian. Debug output is viewable via the QuantConnect Terminal Console.

Output to the console via `self.Debug()` is rate-limited. If you need long-term storage of your logs and don't want to overflow the console, consider using `self.Log()` instead. An example comparing logging between Quantopian and QuantConnect is shown below.

Quantopian

```
def initialize(context):
    log.info("Hello, console!")
```

PY

QuantConnect

```
class MyLoggingAlgorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.debug("Hello, console!")
        self.log("Hello, backtesting logs!")
```

PY

Logging via `self.Log()` has daily limits in QuantConnect. On the free tier, users are limited to 10KB of logs per day for your account/organization. To increase this limit, you can upgrade your account to a different tier. Calls to `self.Debug()` will also count towards the 10KB log limit, but are always outputted to the Terminal Console regardless of your daily log limit.

To view pricing and upgrade your account, visit the Organization Pricing page.

You can view the logs generated via `self.Log()` for a backtest by first accessing a backtest, and clicking the "Logs" tab near the bottom of the page. To download the logs, you can click the "Download Logs" link to download a text file containing your backtest's logs.

Plotting

In Quantopian, plotting/charting is done with the `record(series1_name=scalar_value, series2_name=scalar_value, ...)` function. In QuantConnect, plotting is accessible via the `self.plot("chart name", "series name", scalar_value)` method.

Plotting multiple series in one function call is possible in Quantopian. However, in QuantConnect, a separate call is needed for each series requiring an update. All series with the same chart name will appear on the same window/pane. New charts can be created by calling `self.plot()` method with a unique chart name.

An example of plotting in Quantopian vs. QuantConnect is shown below. The QuantConnect code will create two charts, one named "Chart1" with two series of data, and another named "Chart2" with one series of data. Note that you can also use `self.plot("series name", value)` to place the custom series on the same chart as the equity curve.

Quantopian

```py
import numpy as np
from quantopian.algorithm import record

    def initialize(context):
        context.i = 0

    def handle_data(context, data):
        context.i += 0.25

        record(
            series1=np.sin(context.i),
            Series2=np.cos(context.i),
            Series3=np.tan(context.i)
        )
```

QuantConnect

```py
import numpy as np

class Algorithm(QCAlgorithm):
    def initialize(self) -> None:
        self.i = 0

    def on_data(self, slice: Slice) -> None:
        self.i += 0.25

        self.plot('Chart1', 'Series1', np.sin(self.i))
        self.plot('Chart1', 'Series2', np.cos(self.i))
        self.plot('Chart2', 'Chart2 Series1', np.tan(self.i))
```

For more information on plotting in QuantConnect, visit the Charting documentation page.

Zipline

Quick Reference

User-Implemented Functions

The following table describes functions that are treated specially when defined in the IDE.

Algorithms are required to implement one method: Initialize(self).

An optional scheduled event can be defined to behave like before_trading_start(). OnData() is also optional but is treated especially if defined.

Quantopian	**QuantConnect**
`initialize` (context)	`Initialize` (self)
Required function called once at the start of a backtest.	
`before_trading_start` (context, data)	`self.Schedule.On(self.DateRules.EveryDay(), self.TimeRules.AfterMarketOpen(symbol, 10), self.before_trading_start)`
Optional function called prior to market open on every trading day.	
`handle_data` (context, data)	`OnData` (self, slice)
Optional function called at the end of each market minute.	

Interface Classes

The following table describes classes that are passed to `initialize()` , `handle_data()` , `before_trading_start()` and any scheduled with `schedule_function()` .

In QuantConnect/Lean, the portfolio state and the positions are class attributes of QCAlgorithm, available via self keyword. The scheduled function is parameterless and can algorithms can self.CurrentSlice.

Quantopian	**QuantConnect**
`quantopian.algorithm.interface.AlgorithmContext`	`QCAlgorithm`
Shared object for storing custom state.	
`quantopian.algorithm.interface.BarData`	`Slice`
Provides methods for accessing minutely and daily price/volume data from Algorithm API functions.	
`quantopian.algorithm.interface.Portfolio`	`self.Portfolio`
Object providing read-only access to current portfolio state.	
`quantopian.algorithm.interface.Positions`	`self.Transactions`
A dict-like object containing the algorithm's current positions.	

The features from the BarData methods are found in the Slice object (current data), History method (past data) and class attribute Securities.

Quantopian	QuantConnect
`quantopian.algorithm.interface.BarData.current`	`Slice[symbol]`
Returns the "current" value of the given fields for the given assets at the current simulation time.	
`quantopian.algorithm.interface.BarData.history`	`self.History(symbols, bar_count)`
Returns a trailing window of length `bar_count` containing data for the given assets, fields, and frequency.	
`quantopian.algorithm.interface.BarData.can_trade`	`self.Securities[symbol].IsTradable`
For the given asset or iterable of assets, returns True if all of the following are true:	
`quantopian.algorithm.interface.BarData.is_stale`	`not Slice[symbol].IsFillForward`
For the given asset or iterable of assets, returns True if the asset is alive and there is no trade data for the current simulation time.	

Pipeline

The following table describes functions used to schedule and retrieve results of Universe selection. For more information on the Universe Selection API, see the Univeres API Reference.

Quantopian	QuantConnect
`quantopian.algorithm.attach_pipeline` (...)	`universe = self.AddUniverse` (...) name = universe.Configuration.Symbol
Register a pipeline to be computed at the start of each day.	
`quantopian.algorithm.pipeline_output` (name)	`self.UniverseManager` [name]
Get results of the pipeline attached by with `name` .	

Scheduling Functions

The following tables describes functions that can be used to schedule functions to run periodically during your algorithm.

Quantopian	QuantConnect
`quantopian.algorithm.schedule_function` (func)	`self.Schedule.On(IDateRule, ITimeRule, func)`
Schedule a function to be called repeatedly in the future.	

Date Rules

The following table contains functions that can be used with the `date_rule` parameter of `schedule_function()` .

Quantopian	QuantConnect
`quantopian.algorithm.date_rules.every_day` ()	`self.DateRules.EveryDay(symbol)`
Create a rule that triggers every day.	
`quantopian.algorithm.date_rules.month_start` ([...])	`self.DateRules.MonthStart(symbol, daysOffset)`
Create a rule that triggers a fixed number of trading days after the start of each month.	
`quantopian.algorithm.date_rules.month_end` ([...])	`self.DateRules.MonthEnd(symbol, daysOffset)`
Create a rule that triggers a fixed number of trading days before the end of each month.	
`quantopian.algorithm.date_rules.week_start` ([...])	`self.DateRules.WeekStart(symbol, daysOffset)`
Create a rule that triggers a fixed number of trading days after the start of each week.	
`quantopian.algorithm.date_rules.week_end` ([...])	`self.DateRules.WeekEnd(symbol, daysOffset)`
Create a rule that triggers a fixed number of trading days before the end of each week.	

Time Rules

The following table contains functions that can be used with the `time_rule` parameter of `schedule_function()` .

Quantopian	QuantConnect
`quantopian.algorithm.time_rules.market_open` ([...])	`self.TimeRules.AfterMarketOpen(symbol, minutes_after_open = 0)`
Create a rule that triggers at a fixed offset from market open.	
`quantopian.algorithm.time_rules.market_close` ([...])	`self.TimeRules.BeforeMarketClose(symbol, minutes_before_close = 0)`
Create a rule that triggers at a fixed offset from market close.	

Ordering

There are several ways to place orders from an algorithm. For most use-cases, we recommend using SetHoldings method which correctly accounts the porfolio buying power model and fees. At the moment, there is no equivalent for `order_optimal_portfolio()` . However a similar behavior can be replicated with a Portfolio Construction Model.

Algorithms that require explicit manual control of their orders can use the lower-level ordering functions.

Placing Orders

Quantopian	QuantConnect
`quantopian.algorithm.order_optimal_portfolio` (...)	`N/A`
Calculate an optimal portfolio and place orders toward that portfolio.	
`quantopian.algorithm.order` (asset, amount[, ...])	`self.Order(symbol, amount)`
Place an order for a fixed number of shares.	
`quantopian.algorithm.order_value` (asset, value)	`N/A`
Place an order for a fixed amount of money.	
`quantopian.algorithm.order_percent` (asset, ...)	`self.SetHoldings(symbol, percent)`
Place an order in the specified asset corresponding to the given percent of the current portfolio value.	
`quantopian.algorithm.order_target` (asset, target)	`N/A`
Place an order to adjust a position to a target number of shares.	
`quantopian.algorithm.order_target_value` (...)	`N/A`
Place an order to adjust a position to a target value.	
`quantopian.algorithm.order_target_percent` (...)	`self.SetHoldings(symbol, percent)`
Place an order to adjust a position to a target percent of the current portfolio value.	

Controlling Order Execution

Quantopian	QuantConnect
`zipline.finance.execution.ExecutionStyle`	`FillModel`
Base class for order execution styles.	
`zipline.finance.execution.MarketOrder` ([exchange])	`self.MarketOrder` (symbol, quantity)
Execution style for orders to be filled at current market price.	
`zipline.finance.execution.LimitOrder` (limit_price)	`self.LimitOrder` (symbol, quantity, limit_price)
Execution style for orders to be filled at a price equal to or better than a specified limit price.	
`zipline.finance.execution.StopOrder` (stop_price)	`self.StopMarketOrder` (symbol, quantity, stop_price)
Execution style representing a market order to be placed if market price reaches a threshold.	
`zipline.finance.execution.StopLimitOrder` (...)	`self.StopLimitOrder` (symbol, quantity, stop_price, limit_price)
Execution style representing a limit order to be placed if market price reaches a threshold.	

Managing Existing Orders

Quantopian	QuantConnect
`quantopian.algorithm.get_order` (order_id)	`self.Transactions.GetOrderById` (order_id)
Lookup an order based on the order Id returned from one of the order functions.	
`quantopian.algorithm.get_open_orders` ([asset])	`self.Transactions.GetOpenOrders` (symbol)
Retrieve all of the current open orders.	
`quantopian.algorithm.cancel_order` (order_param)	`self.Transactions.CancelOpenOrders` (symbol)
Cancel an open order.	

Customizing the Simulation

Slippage

Algorithms can customize the simulation of order fills by calling SetSlippageModel() with a ISlippageModel which is any class that implements GetSlippageApproximation method. Unlike Quantopian, the slippage model is a security attribute, not a global attribute. Different securities can have different slippage models.

The default slippage model on QuantConnect is a constant slippage model with zero slippage percent (no slippage).

Quantopian	QuantConnect
`quantopian.algorithm.set_slippage` ([...])	`self.Securities[symbol].SetSlippageModel(ISlippageModel)`
Set the slippage models for the simulation.	
`zipline.finance.slippage.SlippageModel` ()	`N/A (class must implement GetSlippageApproximation method)`
Abstract base class for slippage models.	
`zipline.finance.slippage.FixedBasisPointsSlippage` ([...])	`ConstantSlippageModel(slippage_percent)`
Model slippage as a fixed percentage difference from historical minutely close price, limiting the size of fills to a fixed percentage of historical minutely volume.	
`zipline.finance.slippage.NoSlippage` ()	`NullSlippageModel.Instance`
A slippage model where all orders fill immediately and completely at the current close price.	
`zipline.finance.slippage.FixedSlippage` ([spread])	`N/A`
Simple model assuming a fixed-size spread for all assets.	
`zipline.finance.slippage.VolumeShareSlippage` ([...])	`N/A`
Model slippage as a quadratic function of percentage of historical volume.	

Commissions

Algorithms can customize the simulation of order fees by calling SetFeeModel() with a IFeeModel which is any class that implements GetOrderFee method. Unlike Quantopian, the fee model is a security attribute, not a global attribute. Different securities can have different fee models.

The default fee model on QuantConnect depends on the security type and the brokerage. For instance, the default fee model for equity is the Interactive Brokers fee model.

Quantopian	QuantConnect
`quantopian.algorithm.set_commission` ([...])	`self.Securities[symbol].SetFeeModel(IFeeModel)`
Sets the commission models for the simulation.	
`zipline.finance.commission.CommissionModel`	`N/A (class must implement GetOrderFee method)`
Abstract base class for commission models.	
`zipline.finance.commission.PerShare` ([cost, ...])	`N/A`
Calculates a commission for a transaction based on a per share cost with an optional minimum cost per trade.	
`zipline.finance.commission.PerTrade` ([cost])	`N/A`
Calculates a commission for a transaction based on a per trade cost.	
`zipline.finance.commission.PerDollar` ([cost])	`N/A`
Model commissions by applying a fixed cost per dollar transacted.	
`zipline.finance.commission.NoCommission`	`FeeModel()`
Model commissions as free.	

Benchmark

The default benchmark is SPY.

Quantopian	QuantConnect
`quantopian.algorithm.set_benchmark` (benchmark)	`self.SetBenchmark` (benchmark)
Set the benchmark asset.	

Processing math: 0%

Processing math: 8%